

Released 18 May 2026



Guide to the Systems Engineering Body of Knowledge (SEBoK)

Version 2.14



Contents

Articles

Front Matter	1
Editor's Corner	1
Governance and Editorial Boards	2
Acknowledgements and Release History	10
SEBoK Sponsors	14
Get Involved	15
Bkcase Wiki:Copyright	18
Table of Contents	19
SEBoK Table of Contents	19
Part 1: SEBoK Introduction	27
SEBoK Introduction	27
Introduction to the SEBoK	29
Introduction to the SEBoK	29
Scope of the SEBoK	30
Structure of the SEBoK	32
Introduction to Systems Engineering	36
Introduction to Systems Engineering	36
Systems Engineering Overview	38
Fundamentals for Digital Engineering	42
Economic Value of Systems Engineering	48
A Brief History of Systems Engineering	52
Systems Engineering: Historic and Future Challenges	56
Systems Engineering and Other Disciplines	60
Fundamentals for Future Systems Engineering	63
SEBoK Users and Uses	66
SEBoK Users and Uses	66
Guidance for Systems Engineering Novices	70
Guidance for Systems Engineers	73
Guidance for Engineers	77

Guidance for Systems Engineering Customers	80
Guidance for Educators and Researchers	84
Guidance for General Managers	87
Part 2: Foundations of Systems Engineering	90
Foundations of Systems Engineering	90
Knowledge Area: Systems Engineering Fundamentals	97
Systems Engineering Fundamentals	97
Introduction to Systems Engineering Fundamentals	100
Systems Engineering Core Concepts	109
Systems Engineering Principles	114
Systems Engineering Heuristics	121
Knowledge Area: The Nature of Systems	126
The Nature of Systems	126
Engineered Systems	131
Natural Systems	136
Principles and Attributes of Natural Systems	139
Value Proposition for Systems Engineering	142
Identity and Togetherness of Systems	147
Behavior and Dynamics of Systems	152
Cycles and the Phases of Systems	157
Purpose and the Capability of Systems	166
Value and the Quality of Systems	173
Consciousness and the Experience of Systems	178
Knowledge Area: Systems Science	183
Systems Science	183
From Nature's Patterns to Systems Science	187
History and Foundations of Systems Science	192
Worldviews and Philosophies of Systemness	197
Patterns and Principles in Systems Science	205
Structure, Identity, and Function in Systems	211
Emergence and Complexity	216
System Dynamics and Control	223
Information, Organization, and Entropy in Systems	228
Systems, Levels, and Holarchy	235
Systems Science and the Scientific Enterprise	242

Knowledge Area: Systems Thinking	248
Systems Thinking	248
What is Systems Thinking?	251
Concepts of Systems Thinking	255
Principles of Systems Thinking	260
Patterns of Systems Thinking	266
Knowledge Area: Representing Systems with Models	275
Representing Systems with Models	275
What is a Model?	277
Why Model?	282
Types of Models	285
System Modeling Concepts	290
Integrating Supporting Aspects into System Models	293
Modeling Standards	299
Knowledge Area: Systems Approach Applied to Engineered Systems	304
Systems Approach Applied to Engineered Systems	304
Overview of the Systems Approach	306
Engineered System Context	313
Identifying and Understanding Problems and Opportunities	317
Synthesizing Possible Solutions	321
Analysis and Selection between Alternative Solutions	326
Implementing and Proving a Solution	329
Deploying, Using, and Sustaining Systems to Solve Problems	331
Applying the Systems Approach	334
Part 3: Systems Engineering and Management	344
Systems Engineering and Management	344
Systems Engineering STEM Overview	346
Model-Based Systems Engineering (MBSE)	357
Part 3a: Life Cycle Concepts	363
Knowledge Area: Life Cycle Terms and Concepts	364
Life Cycle Terms and Concepts	364
Life Cycle Concepts	365
Life Cycle Models	367

Life Cycle Stages	371
Technical Reviews and Audits	373
Knowledge Area: Development Approaches	377
Development Approaches	377
Development Approach Concepts	378
Sequential Development Approach	381
Incremental Development Approach	384
Evolutionary Development Approach	388
Agile Development Approach	391
Lean Engineering	394
Knowledge Area: Agile Approaches	397
Agile Approaches	397
Agile Systems Engineering	398
Industrial DevOps	402
Knowledge Area: Life Cycle Model Selection and Adaptation	405
Life Cycle Model Selection and Adaptation	405
Selecting the Life Cycle Model	406
Adapting the Life Cycle Model	408
Selecting the Development Approach	409
Adapting the Development Approach	411
Applying Life Cycle Processes	413
Part 3b: Process Concepts	420
Knowledge Area: Process Concepts	421
Process Concepts	421
Process Description	422
Process Concurrency, Iteration, and Recursion	425
Knowledge Area: Process Selection and Tailoring	427
Process Selection and Tailoring	427
Guidelines for the Selection and Tailoring of Processes	428
Using ISO/IEC/IEEE 24748-2	430
Tailoring	433
Part 3c: Processes	438

Knowledge Area: Technical Management Processes	439
Technical Management Processes	439
Project Planning	441
Project Assessment and Control	446
Decision Management	450
Requirements Management	463
Risk Management	468
Configuration Management	477
Configuration Baselines	485
Configuration Management Implementation	491
Information Management	497
Quality Management	509
Measurement	515
 Knowledge Area: System Concept Definition	 527
System Concept Definition	527
Business or Mission Analysis	530
Stakeholder Needs Definition	536
System Requirements Definition	544
 Knowledge Area: System Architecture Design Definition	 556
System Architecture Design Definition	556
Functional Architecture	567
Logical Architecture	574
Physical Architecture	583
System Detailed Design Definition	591
System Analysis	595
System Realization	602
System Implementation	606
System Integration	611
System Verification	621
System Transition	629
System Validation	633
System Operation	644
 Knowledge Area: System Maintenance	 647
System Maintenance	647
Logistics	650

Service Life Management	656
Service Life Extension	661
Capability Updates, Upgrades, and Modernization	667
System Disposal and Retirement	675
Part 3d: Relevant Standards	681
Knowledge Area: Systems Engineering Standards	682
Systems Engineering Standards	682
Why Standards?	683
Evolution of Systems Engineering Standards	687
Systems Engineering Related Standards Landscape	692
Harmonization of Systems Engineering Standards	699
Application of Systems Engineering Standards	706
References	
Article Sources and Contributors	711
Image Sources, Licenses and Contributors	715

Contents

Articles

Part 4: Applications of Systems Engineering	1
Applications of Systems Engineering	1
Knowledge Area: Product Systems Engineering	3
Product Systems Engineering	3
Product Systems Engineering Background	12
Product as a System Fundamentals	20
Business Activities Related to Product Systems Engineering	27
Product Systems Engineering Key Aspects	32
Product Systems Engineering Special Activities	43
Knowledge Area: Service Systems Engineering	51
Service Systems Engineering	51
Service Systems Background	56
Fundamentals of Services	62
Properties of Services	69
Scope of Service Systems Engineering	73
Value of Service Systems Engineering	78
Service Systems Engineering Stages	84
Knowledge Area: Enterprise Systems Engineering	91
Enterprise Systems Engineering	91
Enterprise Systems Engineering Background	97
The Enterprise as a System	106
Related Business Activities	114
Enterprise Systems Engineering Key Concepts	122
Enterprise Systems Engineering Process Activities	128
Enterprise Capability Management	137
Knowledge Area: Systems of Systems (SoS)	143
Systems of Systems (SoS)	143
Architecting Approaches for Systems of Systems	152
Systems Engineering for Systems of Systems	157
Systems of Systems Analytic Approaches	160

System of Systems and Complexity	163
Socio-Technical Features of Systems of Systems	168
Capability Engineering	172
Mission Engineering	174
Knowledge Area: Healthcare Systems Engineering	177
Healthcare Systems Engineering	177
Overview of the Healthcare Sector	184
Systems Engineering in Healthcare Delivery	188
Systems Biology	194
Lean in Healthcare	198
Part 5: Enabling Systems Engineering	204
Enabling Systems Engineering	204
Knowledge Area: Enabling Businesses and Enterprises	208
Enabling Businesses and Enterprises	208
Systems Engineering Organizational Strategy	210
Determining Needed Systems Engineering Capabilities in Businesses and Enterprises	219
Organizing Business and Enterprises to Perform Systems Engineering	227
Assessing Systems Engineering Performance of Business and Enterprises	234
Developing Systems Engineering Capabilities within Businesses and Enterprises	241
Barriers to Successful Embedding of Systems Engineering into Organizations	248
Culture	254
Knowledge Area: Enabling Teams	261
Enabling Teams	261
Team Capability	263
Team Dynamics	271
Diversity, Equity, and Inclusion	274
Technical Leadership in Systems Engineering	280
Knowledge Area: Enabling Individuals	294
Enabling Individuals	294
Roles and Competencies	296
Assessing Individuals	306
Developing Individuals	310
Ethical Behavior	316

Part 6: Related Disciplines	320
Related Disciplines	320
Knowledge Area: Systems Engineering and Environmental Engineering	324
Systems Engineering and Environmental Engineering	324
Knowledge Area: Systems Engineering and Geospatial/Geodetic Engineering	330
Systems Engineering and Geospatial/Geodetic Engineering	330
Overview of Geospatial/Geodetic Engineering	331
Relationship between Systems Engineering and Geospatial/Geodetic Engineering	335
Further Insights into Geospatial/Geodetic Engineering	340
Knowledge Area: Systems Engineering and Industrial Engineering	350
Systems Engineering and Industrial Engineering	350
Knowledge Area: Systems Engineering and Project Management	358
Systems Engineering and Project Management	358
The Nature of Project Management	359
An Overview of the PMBOK® Guide	362
Relationships between Systems Engineering and Project Management	365
The Influence of Project Structure and Governance on Systems Engineering and Project Management Relationships	368
Cost Estimating and Analysis in Systems Engineering	372
Procurement and Acquisition	382
Portfolio Management	388
Knowledge Area: Systems Engineering and Software Engineering	393
Systems Engineering and Software Engineering	393
Software Engineering in the Systems Engineering Life Cycle	395
The Nature of Software	400
An Overview of the SWEBOK Guide	402
Key Points a Systems Engineer Needs to Know about Software Engineering	408
Software Engineering Features - Models, Methods, Tools, Standards, and Metrics	413
Knowledge Area: Systems Engineering and Aerospace Engineering	417
Systems Engineering and Aerospace Engineering	417
Knowledge Area: Systems Engineering and Electrical Engineering	418

Systems Engineering and Electrical Engineering	418
Knowledge Area: Systems Engineering and Mechanical Engineering	419
Systems Engineering and Mechanical Engineering	419
Knowledge Area: Systems Engineering and Civil Engineering	424
Systems Engineering and Civil Engineering	424
Knowledge Area: Systems Engineering and Economics	425
Systems Engineering and Economics	425
Knowledge Area: Systems Engineering and Enterprise IT	426
Systems Engineering and Enterprise IT	426
Knowledge Area: Systems Engineering and Quality Attributes	439
Systems Engineering and Quality Attributes	439
A Framework for Viewing Quality Attributes from the Lens of Loss	442
Human Systems Integration	448
Manufacturability and Producibility	455
System Affordability	456
System Hardware Assurance	459
System Reliability, Availability, and Maintainability	465
System Resilience	480
Resilience Modeling	493
System Resistance to Electromagnetic Interference	497
System Safety	503
System Security	506
Part 7: SE Implementation Examples	515
Systems Engineering Implementation Examples	515
Matrix of Implementation Examples	518
Implementation Examples	521
Defense System Examples	524
Submarine Warfare Federated Tactical Systems	524
Virginia Class Submarine	532
Information System Examples	535
Complex Adaptive Taxi Service Scheduler	535
Successful Business Transformation within a Russian Information Technology Company	541

Federal Bureau of Investigation (FBI) Virtual Case File System	548
Management System Examples	552
Project Management for a Complex Adaptive Operating System	552
Medical System Examples	557
Next Generation Medical Infusion Pump	557
Medical Radiation	562
Design for Maintainability	565
Space System Examples	568
Global Positioning System	568
Global Positioning System II	572
Russian Space Agency Project Management Systems	579
How Lack of Information Sharing Jeopardized the NASA/ESA Cassini/Huygens Mission to Saturn	584
Hubble Space Telescope	589
Applying a Model-Based Approach to Support Requirements Analysis on the Thirty-Meter Telescope	592
Miniature Seeker Technology Integration Spacecraft	600
Apollo 1 Disaster	602
Transportation System Examples	607
Denver Airport Baggage Handling System	607
Federal Aviation Administration (FAA) Advanced Automation System (AAS)	609
Federal Aviation Administration (FAA) Next Generation Air Transportation System	611
Reverse Engineering a UAV Prototype using Agile Practices	616
UK West Coast Route Modernisation Project	621
Standard Korean Light Transit System	623
Utilities Examples	627
Northwest Hydro System	627
Singapore Water Management	632
Part 8: Emerging Knowledge	635
Emerging Knowledge	635
Emerging Topics	638
Emerging Topics	638
Introduction to SE Transformation	640

Socio-technical Systems	642
Artificial Intelligence	645
Verification and Validation of Systems in Which AI is a Key Element	651
Transitioning Systems Engineering to a Model-based Discipline	660
Model-Based Systems Engineering Adoption Trends 2009-2018	663
Digital Engineering	671
Set-Based Design	673
Emerging Research	679
Emerging Research	679
References	
Article Sources and Contributors	682
Image Sources, Licenses and Contributors	686

Front Matter

Editor's Corner

The Editor's Corner provides perspective from the Editor in Chief on critical topics for systems engineering, either through their own words or by inviting a guest writer.

18 May 2026

SEBoK Together: A Living Body of Knowledge Requires a Living Community

As we at the SEBoK have engaged with the systems engineering community over the past several months, one idea has continued to surface: the discipline is evolving rapidly, and the way we steward knowledge must evolve with it. That realization sits at the heart of the SEBoK Together initiative. When we launched SEBoK Together earlier this year, our goal was straightforward but ambitious: create a lightweight, approachable way for the global systems community to continuously review, improve, and evolve the SEBoK. Rather than treating updates as isolated editorial activities, we wanted to create a sustained rhythm of community engagement one article, one insight, and one conversation at a time.

The response so far has been incredibly encouraging.

To date, we've gathered feedback from dozens of people, many of whom have never before provided input to the SEBoK. Reviews have spanned topics from systems thinking and lifecycle concepts to verification, modernization, enabling teams, and applied systems domains. Several articles have already received multiple independent reviews, helping us identify recurring themes, opportunities for clarification, and areas where current practice is evolving faster than our existing content. What has been especially exciting is that participation has come from a wide range of perspectives. Some contributors are long-time SEBoK editors and authors. Others are practitioners, educators, researchers, and students engaging with the SEBoK community for the first time. That diversity matters. A living body of knowledge can only remain relevant if it reflects the experiences and needs of the broader community it serves.

SEBoK Together was intentionally designed to lower the barrier to participation. You do not need to be a lead editor, published author, or recognized expert to contribute meaningful insight. Sometimes the most valuable feedback comes from someone actively trying to apply systems engineering concepts in practice, teach them in the classroom, or explain them to a new audience. In many ways, this initiative reflects the original spirit of and collaborative vision that has shaped the SEBoK from the beginning. The SEBoK has always been more than a reference document. It is a shared community effort, one that depends on continual stewardship, thoughtful discussion, and the willingness of practitioners across domains to help refine and strengthen it over time. As we look ahead toward future releases and the longer-term vision for SEBoK, community engagement becomes even more important. We are exploring new forms of content and interaction, including improved navigation, companion media, article summary videos, and podcasts. But regardless of how the SEBoK evolves, its core strength will continue to come from the people who use it, challenge it, and improve it together.



So here is my invitation to you:

Pick an article that interests you. One you use frequently. One you teach from. One you disagree with. One you think is missing something important. Spend fifteen minutes reviewing it and share your thoughts with us.

Every review helps.

Every perspective matters.

Every contribution strengthens the SEBoK for the broader systems engineering community.

You can learn more about the initiative and access the review forms here: [SEBoK Together](#) Whether your feedback is a few sentences or a detailed analysis, we hope you will join us in helping shape the future of the SEBoK.

Together, we review. Together, we improve. Together, we evolve.

Sincerely,



Governance and Editorial Boards

BKCASE Governing Board

The three SEBoK steward organizations – the International Council on Systems Engineering (INCOSE), the Institute of Electrical and Electronics Engineers Systems Council (IEEE-SYSC), and Stevens Institute of Technology – provide the primary funding and resources needed to sustain and evolve the SEBoK and make it available as a free and open resource to all. The stewards appoint the BKCASE Governing Board to be their primary agents to oversee and guide the SEBoK. The stewards appoint the SEBoK Editor in Chief to manage the SEBoK and oversee the Editorial Board.

The BKCASE Governing Board includes:

- Representing the **International Council on Systems Engineering (INCOSE)**
 - Art Pyster (Governing Board Chair), Emma Sparks
- Representing the **Systems Engineering Research Center (SERC)**
 - Thomas McDermott, Cihan Dagli
- Representing the **IEEE Systems Council (IEEE-SYSC)**
 - Andy Chen (President of IEEE Systems Council), Steve Li (Treasurer of IEEE Systems Council)

Past governors include Stephanie White, Bob Rassa, Richard Fairley, Kevin Forsberg, Paul Frenz, Richard Hilliard, John Keppler, Bill Miller, David Newbern, Ken Nidiffer, Dave Olwell, Massood Towhidnejad, Jon Wade, David Walden, and Courtney Wright. The governors would especially like to acknowledge Andy Chen and Rich Hilliard, IEEE Computer Society, who were instrumental in helping the governors to work within the IEEE CS structure and who supported the SEBoK transition to the IEEE Systems Council. Andy Chen has returned to his governor role after several years off, this time representing the IEEE Systems Council.

Editorial Board

The SEBoK Editorial Board is chaired by the *Editor in Chief*, who provides the strategic vision for the SEBoK. The EIC is supported by a group of Editors, each of whom are responsible for a specific aspect of the SEBoK. The EIC and Editorial Board are supported by the Managing Editor, who handles all day-to-day operations. The EIC, Managing Editor, and Editorial Board are supported by Student Editors throughout the year whose hard work and dedication are greatly appreciated.

SEBoK Editor in Chief



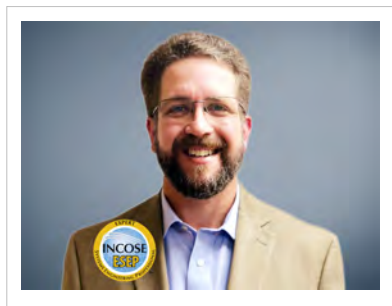
Nicole Hutchison

Virginia Tech National Security Institute

nicole.hutchison@stevens.edu^[1] or emtnicole@gmail.com^[2]

Responsible for the appointment of SEBoK Editors and for the strategic direction and overall quality and coherence of the SEBoK.

SEBoK Managing Editor



Chris Hoffman

INCOSE, Cummins Inc.

christopher.hoffman@incose.net^[3]

Responsible for the day-to-day operations of the SEBoK and supports the Editor in Chief.

You can reach out to both the Editor in Chief and the Managing Editor by emailing sebok@incose.net^[4].

Each Editor has their area(s) of responsibility, or shared responsibility, highlighted in the table below.

SEBoK Part 1: SEBoK Introduction

Lead Editor: Nicole Hutchison

Virginia Tech National Security Institute

emtnicole@gmail.com^[2]

SEBoK Part 2: Foundations of Systems Engineering

Lead Editor: Gary Smith (UK) *Airbus and International Society for the System Sciences*
grs0036@gmail.com^[5]

Responsible for the System Science Foundations of System Engineering.



Assistant Editor: Dov Dori

*Massachusetts Institute of Technology (USA) and Technion Israel
Institute of Technology (Israel)*
dori@mit.edu^[6]

Responsible for the Representing Systems with Models knowledge
area



Assistant Editor: Duane Hybertson

MITRE (USA)
dhyberts@mitre.org^[7]

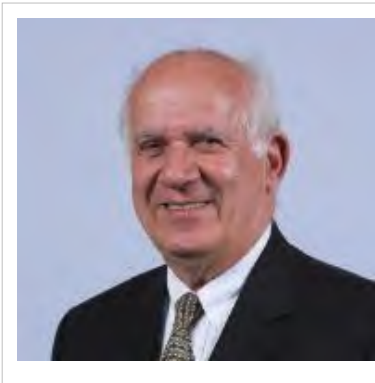
Jointly responsible for the Systems Fundamentals, Systems Science and
Systems Thinking knowledge areas.



Assistant Editor: Peter Tuddenham

College of Exploration (USA)

Peter@coexploration.net^[8]



Assistant Editor: Cihan Dagli

Missouri University of Science & Technology (USA)

dagli@mst.edu^[9]

Responsible for the Systems Approach Applied to Engineered Systems knowledge areas.

SEBoK Part 3: Systems Engineering and Management



Lead Editor: David Endler

Lead Editor of ISO/IEC/IEEE 15288 and ISO/IEC/IEEE 24748-2, Systems Engineering Consultant – Freelancer

de@davidendler.de^[10]



Assistant Editor: Cecelia Haskins

INCOSE

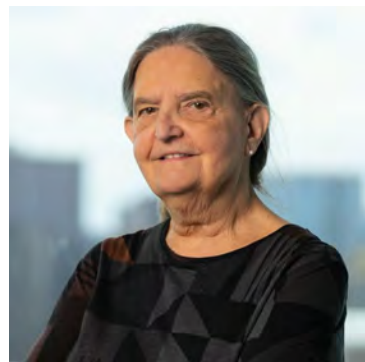
cecilia.haskins@incose.net^[11]

SEBoK Part 4: Applications of Systems Engineering**Lead Editor: Tom McDermott**

Systems Engineering Research Center (SERC)
tmcdermo@stevens.edu^[12]

**Assistant Editor: Javier Calvo-Amodio**

University of Idaho
javier.calvo-amodio@incose.net^[13]

**Assistant Editor: Judith Dahmann**

MITRE Corporation (USA)
jdahmann@mitre.org^[14]

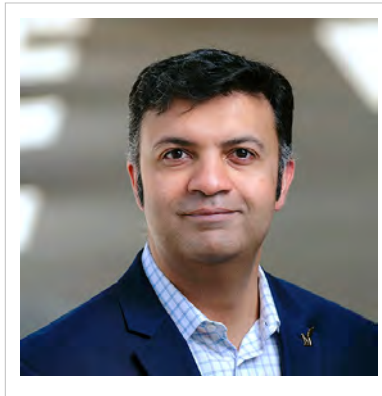
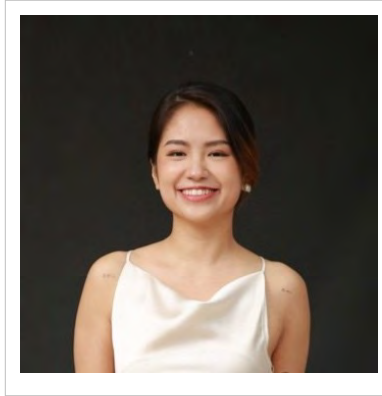
Jointly responsible for Product Systems Engineering and Systems of Systems (SoS) knowledge areas.

**Assistant Editor: Michael Henshaw**

Loughborough University (UK)
M.J.d.Henshaw@lboro.ac.uk^[15]

Jointly responsible for Product Systems Engineering and Systems of Systems (SoS) knowledge areas

SEBoK Part 5: Enabling Systems Engineering**Lead Editor: Nicole Hutchison***Virginia Tech National Security Institute*emtnicole@gmail.com^[2]**Assistant Editor: Rick Hefner***California Institute of Technology*rhefner@caltech.edu^[16]**SEBoK Part 6: Related Disciplines****Lead Editor: Caitlyn Singam***University of Maryland, College Park*Caitlyn.Singam@incose.net^[17] or csingam@umd.edu^[18]**SEBoK Part 7: Systems Engineering Implementation Examples****Lead Editor: Chris Hoffman***Cummins Inc.*christopher.hoffman@incose.net^[3]

SEBoK Part 8: Emerging Knowledge**Lead Editor: Ali Raz***George Mason University*araz@gmu.edu^[19]**Assistant Editor: Ha Phuong Le***Purdue University*happle104@gmail.com^[20]**SEBoK Cross-Cutting: Digital Engineering****Lead Editor: Ray Madachy***Naval Postgraduate School*rjmadach@nps.edu^[21]

SEBoK Cross-Cutting: Standards**Lead Editor: David Endler**

Lead Editor of ISO/IEC/IEEE 15288 and ISO/IEC/IEEE 24748-2, Systems Engineering Consultant – Freelancer
de@davidendler.de^[10]

The Cross-Cutting Standards team consists of the following reviewers, who provide valuable recommendations to the SEBoK Editorial Board for updates and improvements:

- **Bill Bearden**
- **Dr. Judith Dahmann**
- **Alan Harding**
- **Garry Roedler**
- **Dr. Mike Yokell**

Student Editor

SEBoK Student Editors play a key role in supporting the maintenance, visibility, and growth of the Systems Engineering Body of Knowledge (SEBoK). This position includes both traditional editorial tasks and high-impact contributions to outreach and analytics. It is ideal for students interested in systems engineering, digital publishing, data visualization, and science/engineering communication.

SEBoK Student Editors

Student Editor: Gianna Conover **Student Editor: Dixie Poteet**

Purdue

gconover@purdue.edu^[22]

Colorado State University

dixiepot@rams.colostate.edu^[23]

Interested in Editing?

The Editor in Chief is looking for additional assistant editors to support the evolution of the SEBoK. Assistant editors are responsible for maintaining and updating one to two knowledge areas, including recruiting and working with authors, ensuring the incorporation of community feedback, and maintaining the quality of SEBoK content.

View the current openings at [Get Involved](#).

If you are interested in being considered for participation on the Editorial Board, please contact the SEBoK Staff directly at sebok@incose.net^[4].

References

- [1] <mailto:nicole.hutchison@stevens.edu>
- [2] <mailto:emtnicole@gmail.com>
- [3] <mailto:christopher.hoffman@incose.net>
- [4] <mailto:sebok@incose.net>
- [5] <mailto:grs0036@gmail.com>
- [6] <mailto:dori@mit.edu>
- [7] <mailto:dhyberts@mitre.org>
- [8] <mailto:Peter@coexploration.net>
- [9] <mailto:dagli@mst.edu>
- [10] <mailto:de@davidendler.de>
- [11] <mailto:cecilia.haskins@incose.net>
- [12] <mailto:tmcdermo@stevens.edu>
- [13] <mailto:javier.calvo-amodio@incose.net>
- [14] <mailto:jdahmann@mitre.org>
- [15] <mailto:M.J.d.Henshaw@lboro.ac.uk>
- [16] <mailto:rhefner@caltech.edu>
- [17] <mailto:Caitlyn.Singam@incose.net>
- [18] <mailto:csingam@umd.edu>
- [19] <mailto:araz@gmu.edu>
- [20] <mailto:haple104@gmail.com>
- [21] <mailto:rjmadach@nps.edu>
- [22] <mailto:gconover@purdue.edu>
- [23] <mailto:dixiepot@rams.colostate.edu>

Acknowledgements and Release History

This article describes the contributors to the current version of the SEBoK. For information on contributors to past versions of the SEBoK, please follow the links under "SEBoK Release History" below.

The BKCASE Project began in the fall of 2009. Its aim was to add to the professional practice of systems engineering by creating two closely related products:

- *Guide to the Systems Engineering Body of Knowledge (SEBoK)*
- *Graduate Reference Curriculum for Systems Engineering (GRCSE)*

BKCASE History, Motivation, and Value

The **Guide to the Systems Engineering Body of Knowledge (SEBoK)** is a living authoritative guide that discusses knowledge relevant to Systems Engineering. It defines how that knowledge should be structured to facilitate understanding, and what reference sources are the most important to the discipline. The curriculum guidance in the **Graduate Reference Curriculum for Systems Engineering (GRCSE)** (Pyster and Olwell et al. 2015) makes reference to sections of the SEBoK to define its core knowledge; it also suggests broader program outcomes and objectives which reflect aspects of the professional practice of systems engineering as discussed across the SEBoK.

Between 2009 and 2012, BKCASE was led by Stevens Institute of Technology and the Naval Postgraduate School in coordination with several professional societies and sponsored by the U.S. Department of Defense (DoD), which provided generous funding. More than 75 authors and many other reviewers and supporters from dozens of companies, universities, and professional societies across 10 countries contributed many thousands of hours writing the SEBoK articles; their organizations provided significant other contributions in-kind.

The SEBoK came into being through recognition that the systems engineering discipline could benefit greatly by having a living authoritative guide closely related to those groups developing guidance on advancing the practice,

education, research, work force development, professional certification, standards, etc.

At the beginning of 2013, BKCASE transitioned to a new governance model with shared stewardship between the Systems Engineering Research Center (SERC) ^[1], the International Council on Systems Engineering (INCOSE) ^[2], and the Institute of Electrical and Electronics Engineers Computer Society (IEEE-CS) ^[3]. This governance structure was formalized in a memorandum of understanding between the three stewards that was finalized in spring of 2013 and subsequently updated. In January 2020, the IEEE Systems Council ^[4] replaced the IEEE-CS in representing IEEE as a steward. The stewards have reconfirmed their commitment to making the SEBoK available at no cost to all users, a key principle of BKCASE.

At of the end of 2024, SEBoK articles have had over 8.6M pageviews from more than 3.5M unique visitors.

We hope the SEBoK will regularly be used by thousands of systems engineers and others around the world as they undertake technical activities such as eliciting requirements, creating systems architectures, or analyzing system test results; and professional development activities such as developing career paths for systems engineers, and deciding new curricula for systems engineering university programs.

Governance

The SEBoK is shaped by the SEBoK Editorial Board and is overseen by the BKCASE Governing Board. A complete list of members for each of these bodies can be found on the BKCASE Governance and Editorial Board page.

Content and Feature Updates for version 2.14

The SEBoK Editorial Board has introduced version 2.14 with a comprehensive refresh of the Systems Science Knowledge Area—adding eight new articles, revising foundational content, and strengthening coverage of complexity, structure, and scientific context—alongside updates to systems engineering standards articles, alignment with the latest PMBOK, and a set of general improvements to content quality, consistency, and platform infrastructure.

New articles are highlighted in the left-hand outline, in the SEBoK Table of Contents, and are elaborated below.

We welcome you to Get Involved with SEBoK!

Part 2

The Systems Science knowledge area (KA) has been significantly refreshed and updated with eight new articles, two revamped articles, and explainer videos provided for several articles. This KA provides a guide to some of the major developments in systems science, which is an interdisciplinary field of science that studies the nature of complex systems in nature, society, and engineering. The articles included are:

- From Nature's Patterns to Systems Science - [New Article](#)
 - History and Foundations of Systems Science - [Renamed Article](#), was "History of Systems Science"
 - Worldviews and Philosophies of Systemness - [New Article](#)
 - Patterns and Principles in Systems Science - [New Article](#)
 - Structure, Identity, and Function in Systems - [New Article](#)
 - Emergence and Complexity - [Integrated and revised from existing articles](#) - "Emergence" and "Complexity"
 - System Dynamics and Control - [New Article](#)
 - Information, Organization, and Entropy in Systems - [New Article](#)
 - Systems, Levels, and Holarchy - [New Article](#)
 - Systems Science and the Scientific Enterprise - [New Article](#)
-

Part 3

- New article Evolution of Systems Engineering Standards created from the existing Why Standards? article.
- Renamed "Alignment and Comparison of Systems Engineering Standards" article to Harmonization of Systems Engineering Standards

Part 6

- The INCOSE PM-SE Working Group updated the article, An Overview of the PMBOK® Guide, reflecting the latest version 8 of the PMBoK.

General Updates

The INCOSE logo has been updated, spelling corrections, acronym corrections, repairing broken links, and a link to the SEBoK Style Guide has been added to the Get Involved page. From an operational perspective, the underlying MediaWiki version has been updated to LTS Version 1.43 from 1.39. Each Knowledge Area has been adjusted to remove duplicate article lists within that KA, as that section is now programmatically generated.

SEBoK Release History

There have been 26 releases of the SEBoK to date.

Main Releases

- Version 2.14 - Current version.
 - Version 2.13 - The SEBoK Editorial Board released version 2.13 with substantial rework of existing content and expansion of key knowledge areas. Major updates include a significant expansion of the The Nature of Systems Knowledge Area, introduction of new Process Concepts content, and updates to Technical Management Processes to reflect current practices and terminology.
 - Version 2.12 - This release included many changes to reflect v. 5 of the INCOSE *Handbook* and the 2023 version of [ISO/IEC/IEEE 15288]. In some instances, substantive rework was completed to reflect the current versions and changes will continue into the next releases of the SEBoK.
 - Version 2.11 - Three new articles were added in this release, Barriers to Successful Embedding of Systems Engineering into Organizations, Cost Estimating and Analysis in Systems Engineering, and Resilience Modeling with an updated introduction to the Related Disciplines article.
 - Version 2.10 - This release formally invited readers to Get Involved through a new content review form, renamed and reorganized several articles in Part 3 to reflect current practice, and published a new System Architecture Design Definition article.
 - Version 2.9 - The first release with Editor-in-Chief Nicole Hutchison and Managing Editor Chris Hoffman. New articles included Reverse Engineering a UAV Prototype using Agile Practices, System Security, and An overview of the SWEBOK Guide. Updated articles on Loss Driven Systems Engineering, System Resilience, and a change to persona-driven SEBoK user guidance instead of use cases that were introduced in v. 1.0.
 - Version 2.8 - This release included new articles on systems engineering and enterprise IT, and system adaptability; minor updates to several articles throughout the wiki; improvements to the wiki infrastructure; and Rob Cloutier's final edition of the SEBoK as Editor-in-Chief.
 - Version 2.7 - This release included new articles on loss-driven systems engineering and the history of systems engineering; updates to the article on systems and industrial engineering; and minor updates to improve resources and align with evolving practices throughout part 3, including in the articles around systems engineering standards and to the articles in Part 5, particularly the addition of new resources. There were also some improvements in the SEBoK wiki infrastructure.
-

- Version 2.6 - This update included substantial evolution of Parts 2 and 3, the foundations of systems engineering and systems engineering approaches, methods, processes, and tools. The version also included more information on Model-Based Systems Engineering (MBSE) and Digital Engineering, refinement of systems science foundations of systems engineering, and a new article on agile approaches. In Part 6 there were many new articles and updates to existing articles on the relationships between systems engineering and other disciplines.
 - Version 2.5 - This version included an update of the main page; creation of the Editor's Corner; new sponsors and sponsorship packages; new navigation in the left-hand menu; small edits to address the comments received from the community. This release also updated to the latest version of MediaWiki, tightened up the IT infrastructure, and made some adjustments to improve performance.
 - Version 2.4 - This was a minor release, including reorganizations of Part 6 and 8 to handle new knowledge areas and topics. In addition, several new articles were added, including, Systems Engineering Heuristics, Diversity, Equity, and Inclusion, Systems Engineering and Geospatial/Geodetic Engineering Knowledge Area, System Hardware Assurance, Socio-technical Systems, Verification and Validation of Systems in Which AI is a Key Element, and an introductory article on Artificial Intelligence. The content on Systems of Systems (SoS) was also updated.
 - Version 2.3 - This was a minor release, including two new articles: Cycles and the Phases of Systems and Portfolio Management. A number of additional minor edits, including a new overview graphic for the SEBoK, cleanup of existing pages, software updates, etc. were incorporated.
 - Version 2.2 - This was a significant release, including the first new Part to be added since v. 1.0 - Emerging Knowledge - which is a place to highlight new topics in systems engineering that are important but may not yet have a large body of literature. Recent dissertations around emerging topics are also included. A new case study on Apollo 1 was added to Part 7, which has also been reorganized around topics. Additional minor updates have occurred throughout.
 - Version 2.1 - This was a significant release with new articles, new functionality, and minor updates throughout.
 - Version 2.0 - This was a major release of the SEBoK which included incorporation of multi-media and a number of changes to the functions of the SEBoK.
 - Version 1.9.1 - This was a micro release of the SEBoK which included updates to the editorial board, and a number of updates to the wiki software.
 - Version 1.9 - This was a minor update which included updates to the System Resilience article in Part 6: Related Disciplines, as well as a major restructuring of Part 7: Systems Engineering Implementation Examples. A new example has been added around the use of model based systems engineering for the thirty-meter telescope.
 - Version 1.8 - This was a minor update, including an update of the Systems of Systems (SoS) knowledge area in Part 4: Applications of Systems Engineering where a number of articles were updated on the basis of developments in the area as well as on comments from the SoS and SE community. Part 6: Related Disciplines included updates to the Manufacturability and Producibility and Reliability, Availability, and Maintainability articles.
 - Version 1.7 - This was a minor update, including a new Healthcare SE Knowledge Area (KA), expansion of the MBSE area with two new articles, Technical Leadership and Reliability, Availability, and Maintainability and a new case study on the Northwest Hydro System.
 - Version 1.6 - This was a minor update, including a reorganization of Part 1 SEBoK Introduction, a new article on the Transition towards Model Based Systems Engineering and a new article giving an overview of Healthcare Systems Engineering, a restructure of the "Systems Engineering and Specialty Engineering" (now Systems Engineering and Quality Attributes) KA.
 - Version 1.5 - This was a minor update, including a restructure and extension of the Software Engineering Knowledge Area, two new case studies, and a number of corrections of typographical errors and updates of outdated references throughout the SEBoK.
-

- Version 1.4 - This was a minor update, including changes related to ISO/IEC/IEEE 15288:2015 standard, three new case studies and updates to a number of articles.
- Version 1.3 - This was a minor update, including three new case studies, a new use case, updates to several existing articles, and updates to references.
- Version 1.2 - This was a minor update, including two new articles and a revision of several existing articles.
- Version 1.1 - This was a minor update that made modest content improvements.
- Version 1.0 - This was the first official version of the SEBoK intended for broad use and was released 15 September 2012.

Click on the links above to read more information about each release.

References

- [1] <https://www.sercuarc.org>
- [2] <https://www.incose.org>
- [3] <https://www.computer.org>
- [4] <https://ieeesystemscouncil.org/>

SEBoK Sponsors

Sponsors provide critical funding that supports the infrastructure around the SEBoK. These organizations have demonstrated their commitment to systems engineering through their support of the SEBoK and we are very grateful for their support.

Global Sponsors



Corporate Partners

Coming soon!

Academic Partners





Sponsorship Packages

If you are interested in sponsoring the SEBoK, please contact us at sebok@incose.net ^[4].

	Level		
	Global Sponsor	Corporate Partner	Academic Partner
Logo on all SEBoK Pages (Rotating)	10 seconds	5 seconds	5 seconds
Logo on SEBoK Sponsor Page	☐	☐	☐
Logo Linked to Specified URL	☐	☐	☐
Video(s) on SEBoK Sponsor Page	Up to 2 (incl.)	\$250	\$250
Social Media Announcement on Sign up/Renewal	☐	☐	☐
Sponsored Knowledge Area	Up to 2 (incl.)	1 (incl.)	\$250
Annual Impact Statement	☐	☐	☐
	\$5,000/year	\$2,000/year	\$1,000/year

Get Involved

The SEBoK is a living resource — and *you* can help shape its future!

The SEBoK is built by the community, for the community. Every article, concept, and framework in the SEBoK exists because professionals like you contributed their time, insight, and expertise to improve our shared understanding of systems engineering.

Whether you're a seasoned practitioner, a researcher, an educator, or a student, there's a place for you in this global effort.

SEBoK Together

The SEBoK Together initiative is an open invitation for the entire community to help shape what comes next. This effort will guide our work over the next year as we continue to modernize the SEBoK's content, structure, and integration with the broader systems engineering ecosystem featuring weekly opportunities for engagement. Whether you contribute new material, provide feedback, or simply share how you use the SEBoK in your organization, your participation matters. Collaboration has always been the SEBoK's greatest strength. Together we can ensure that it remains not just a guide to systems engineering knowledge, but a living example of what collaboration can achieve.

Why Get Involved?

There are a variety of reasons to get engaged with the SEBoK:

- **Make an Impact:** Influence how systems engineering is understood, practiced, and taught around the world.
- **Grow Your Network:** Collaborate with thought leaders, peers, and contributors from diverse domains.
- **Expand Your Expertise:** Deepen your knowledge of key SE topics while contributing to a trusted and evolving resource.
- **Pay It Forward:** Help the next generation of systems engineers by shaping the foundational knowledge they'll rely on.

Provide Feedback

Even if you're not ready to write or edit, your feedback is valuable. Tell us what works, what doesn't, and what's missing.

Eligibility

Anyone is welcome to review the SEBoK articles, knowledge areas, glossary, and other pages. Over the life of the SEBoK, we have had several hundred reviewers, and they provide critical value by helping us ensure that our articles are accessible to the community.

To review an article of SEBoK, please use our Google Form ^[1] or for more general notes, email us at sebok@incose.net ^[4].

Become a Content Contributor

Write or revise content in your area of expertise. Whether it's a new section, an update to existing material, or integration of new methods (like model-based or digital engineering), we welcome your contributions.

Eligibility

Authors create content specifically for the SEBoK per the style-guide (PDF). They work with the Editorial Board to define the right scope for a topic and develop the content in compliance with our style guide. Authors are subject matter experts who can speak authoritatively and without bias and may propose new articles or edits to existing articles. Time commitment is flexible, ranging from one-time contributions to ongoing engagement.

If you are interested in becoming a SEBoK author, please contact us at sebok@incose.net ^[4].

Join the Editorial Board

Editorial board members guide the content, review contributions, and shape SEBoK strategy. We are always looking for individuals passionate about systems engineering and committed to the growth of the field. There are two main roles:

- **Assistant Editors** have responsibility for specific knowledge areas (collections of topics) within the SEBoK. They are subject matter experts who help to ensure that the content of their particular knowledge area is up to date and meets the quality standards of the SEBoK. Assistant editors work with our authors to improve upon or create articles.
 - **Lead Editors** have responsibility for a specific part of the SEBoK (shown in Structure of the SEBoK). They help create the strategic vision for their part and coordinate the efforts of the Assistant Editors within their parts. They are responsible for the overall quality of their part and work directly with authors as appropriate.
-

Eligibility

To become a SEBoK editor, you need to have proven expertise in an area of the SEBoK. Prior experience with the SEBoK is a plus but not required.

Ready to Make a Difference?

Join us in advancing the SEBoK - reach out to sebok@incose.net^[4] to start the conversation!

SEBoK Releases

The SEBoK is updated twice a year, typically the third weeks of May and November. After each release the Editorial Board and Authors begin working on the next version. This means we are always looking for inputs - it's always a good time to engage!

Below are the deadlines for the upcoming Fall 2026 release. Deliverables are strongly encouraged to be completed sooner than the deadlines.

- **12 weeks to publication (24 August 2026):** Lead Editors have submitted proposed changes to SEBoK articles (titles and intent only, not content).
- **9 weeks to publication (14 September 2026):** Authors have submitted Intellectual Property releases for new authors and new figures.
- **6 weeks to publication (5 October 2026):** Editors have submitted SEBoK Architecture (Table of Contents) changes (adds, removes, renames, moves).
- **4 weeks to publication (19 October 2026):** Editors have reviewed feedback from SEBoK Together and incorporated changes into the draft SEBoK.
- **2 weeks to publication (2 November 2026):** Authors and Editors have finalized and uploaded all content into the draft SEBoK.
- **1 week to publication (9 November 2026):** Lead Editors have reviewed, made final edits, and approved the uploaded content.
- **Weekend prior & day of publication (16 November 2026):** Managing Editor and Editor-in-Chief final review, creation of PDF, content is published.

Open Positions

The SEBoK Editorial Board is currently looking for Assistant Editors to support Part 3: Systems Engineering and Management (particularly MBSE/digital engineering practitioners), Part 4: Applications of Systems Engineering, and Part 6: Related Disciplines. We are also looking for a Lead Editor for Part 5: Enabling Systems Engineering and Part 7: Systems Engineering Implementation Examples. If you have an interest in any of these roles, please contact us at sebok@incose.net^[4].

Contact Us

If you have any questions about how to engage with the SEBoK, please reach out to the Editorial Board at sebok@incose.net^[4]. We look forward to hearing from you!

References

[1] <https://forms.gle/hMYn2TNM1kMYsuqZ8>

Bkcase Wiki:Copyright

Please read this page which contains information about how and on what terms you may use, copy, share, quote or cite the Systems Engineering Body of Knowledge (SEBoK):

Copyright and Licensing

A compilation copyright to the SEBoK is held on behalf of the BKCASE Board of Governors by The Trustees of the Stevens Institute of Technology ©2026 ("Stevens") and copyright to most of the content within the SEBoK is also held by Stevens. Prominently noted throughout the SEBoK are other items of content for which the copyright is held by a third party. These items consist mainly of tables and figures. In each case of third party content, such content is used by Stevens with permission and its use by third parties is limited. If you have questions about content used with permission in the SEBoK, contact us at sebok@incose.net ^[4].

Stevens is publishing those portions of the SEBoK to which it holds copyright under a Creative Commons Attribution-NonCommercial ShareAlike 3.0 Unported License. See https://creativecommons.org/licenses/by-nc-sa/3.0/deed.en_US for details about what this license allows. This license does not permit use of third party material but gives rights to the systems engineering community to freely use the remainder of the SEBoK within the terms of the license. Stevens is publishing the SEBoK as a compilation including the third party material under the terms of a Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported (CC BY-NC-ND 3.0). See <https://creativecommons.org/licenses/by-nc-nd/3.0/> for details about what this license allows. This license will permit very limited noncommercial use of the third party content included within the SEBoK and only as part of the SEBoK compilation. Additionally, the U.S. government has limited data rights associated with the SEBoK based on their support for the SEBoK development.

Attribution

When **using text material from the SEBoK**, users who have accepted one of the Creative Commons Licenses described above **must** attribute the work as follows:

This material is used under a Creative Commons Attribution-NonCommercial ShareAlike 3.0 Unported License from The Trustees of the Stevens Institute of Technology.

When **citing the SEBoK in general**, please refer to the format described on the Cite the SEBoK page.

When **using images, figures, or tables from the SEBoK**, please note the following intellectual property (IP) classifications:

- Materials listed as "SEBoK Original" may be used in accordance with the Creative Commons attribution (above).
- Materials listed as "Public Domain" may be used in accordance with information in the public domain.
- Materials listed as "Used with Permission" are copyrighted and *permission must be sought from the copyright owner* to reuse them. If you have any questions about this, please contact us at sebok@incose.net ^[4].

Table of Contents

SEBoK Table of Contents

Navigating the SEBoK
Parent Article Next Article > • <i>Previous</i> - The "Previous" link will take you back one article in the table of contents. • <i>Next</i> - The "Next" link will take you forward one article in the table of contents. • <i>Parent</i> - The "Parent" link takes you up one level in the table of contents.)

Part 1: SEBoK Introduction

- Introduction to the SEBoK
 1. Scope of the SEBoK
 2. Structure of the SEBoK
- Introduction to Systems Engineering
 1. Systems Engineering Overview
 2. Fundamentals for Digital Engineering
 3. Economic Value of Systems Engineering
 4. A Brief History of Systems Engineering
 5. Systems Engineering: Historic and Future Challenges
 6. Systems Engineering and Other Disciplines
 7. Fundamentals for Future Systems Engineering
- SEBoK Users and Uses
 1. Guidance for Systems Engineering Novices
 2. Guidance for Systems Engineers
 3. Guidance for Engineers
 4. Guidance for Systems Engineering Customers
 5. Guidance for Educators and Researchers
 6. Guidance for General Managers

Part 2: Foundations of Systems Engineering

- Knowledge Area: Systems Engineering Fundamentals
 1. Introduction to Systems Engineering Fundamentals
 2. Systems Engineering Core Concepts
 3. Systems Engineering Principles
 4. Systems Engineering Heuristics
 - Knowledge Area: The Nature of Systems
 1. Engineered Systems
 2. Natural Systems
 3. Principles and Attributes of Natural Systems
 4. Value Proposition for Systems Engineering
-

-
5. Identity and Togetherness of Systems
 6. Behavior and Dynamics of Systems
 7. Cycles and the Phases of Systems
 8. Purpose and the Capability of Systems
 9. Value and the Quality of Systems
 10. Consciousness and the Experience of Systems
- Knowledge Area: Systems Science - [Updated](#)
 1. From Nature's Patterns to Systems Science - [New Article](#)
 2. History and Foundations of Systems Science - [Renamed Article, was "History of Systems Science"](#)
 3. Worldviews and Philosophies of Systemness - [New Article](#)
 4. Patterns and Principles in Systems Science - [New Article](#)
 5. Structure, Identity, and Function in Systems - [New Article](#)
 6. Emergence and Complexity - [Renamed Article, was "Complexity"](#)
 7. System Dynamics and Control - [New Article](#)
 8. Information, Organization, and Entropy in Systems - [New Article](#)
 9. Systems, Levels, and Holarchy - [New Article](#)
 10. Systems Science and the Scientific Enterprise - [New Article](#)
 - Knowledge Area: Systems Thinking
 1. What is Systems Thinking?
 2. Concepts of Systems Thinking
 3. Principles of Systems Thinking
 4. Patterns of Systems Thinking
 - Knowledge Area: Representing Systems with Models
 1. What is a Model?
 2. Why Model?
 3. Types of Models
 4. System Modeling Concepts
 5. Integrating Supporting Aspects into System Models
 6. Modeling Standards
 - Knowledge Area: Systems Approach Applied to Engineered Systems
 1. Overview of the Systems Approach
 2. Engineered System Context
 3. Identifying and Understanding Problems and Opportunities
 4. Synthesizing Possible Solutions
 5. Analysis and Selection between Alternative Solutions
 6. Implementing and Proving a Solution
 7. Deploying, Using, and Sustaining Systems to Solve Problems
 8. Applying the Systems Approach
-

Part 3: Systems Engineering and Management

- Systems Engineering STEM Overview
- Model-Based Systems Engineering (MBSE)

Part 3A - Life Cycle Concepts

- Knowledge Area: Life Cycle Terms and Concepts
 1. Life Cycle Concepts
 2. Life Cycle Models
 3. Life Cycle Stages
 4. Technical Reviews and Audits
- Knowledge Area: Development Approaches
 1. Development Approach Concepts
 2. Sequential Development Approach
 3. Incremental Development Approach
 4. Evolutionary Development Approach
 5. Agile Development Approach
 6. Lean Engineering
- Knowledge Area - Agile Approaches
 1. Agile Systems Engineering
 2. Industrial DevOps
- Knowledge Area - Life Cycle Model Selection and Adaptation
 1. Selecting the Life Cycle Model
 2. Adapting the Life Cycle Model
 3. Selecting the Development Approach
 4. Adapting the Development Approach
 5. Applying Life Cycle Processes

Part 3B - Process Concepts

- Knowledge Area: Process Concepts
 1. Process Description
 2. Process Concurrency, Iteration, and Recursion
- Knowledge Area – Process Selection and Tailoring
 1. Guidelines for the Selection and Tailoring of Processes
 2. Using ISO/IEC/IEEE 24748-2
 3. Tailoring

Part 3C: Processes

- Knowledge Area: Technical Management Processes
 1. Project Planning
 2. Project Assessment and Control
 3. Decision Management
 4. Requirements Management
 5. Risk Management
 6. Configuration Management
 7. Configuration Baselines
 8. Configuration Management Implementation
 9. Information Management
-

- 10. Quality Management
- 11. Measurement
- Knowledge Area: System Concept Definition
 - 1. Business or Mission Analysis
 - 2. Stakeholder Needs Definition
- System Requirements Definition
- Knowledge Area: System Architecture Design Definition
 - 1. Functional Architecture
 - 2. Logical Architecture
 - 3. Physical Architecture
- System Detailed Design Definition
- System Analysis
- System Realization
- System Implementation
- System Integration
- System Verification
- System Transition
- System Validation
- System Operation
- Knowledge Area: System Maintenance
 - 1. Logistics
 - 2. Service Life Management
 - 3. Service Life Extension
 - 4. Capability Updates, Upgrades, and Modernization
 - 5. System Disposal and Retirement

Part 3D - Relevant Standards

- Knowledge Area: Systems Engineering Standards
 - 1. Why Standards?
 - 2. Evolution of Systems Engineering Standards - [New Article](#)
 - 3. Systems Engineering Related Standards Landscape
 - 4. Harmonization of Systems Engineering Standards - [Renamed Article, was "Alignment and Comparison of Systems Engineering Standards"](#)
 - 5. Application of Systems Engineering Standards

Part 4: Applications of Systems Engineering

- Knowledge Area: Product Systems Engineering
 - 1. Product Systems Engineering Background
 - 2. Product as a System Fundamentals
 - 3. Business Activities Related to Product Systems Engineering
 - 4. Product Systems Engineering Key Aspects
 - 5. Product Systems Engineering Special Activities
 - Knowledge Area: Service Systems Engineering
 - 1. Service Systems Background
 - 2. Fundamentals of Services
 - 3. Properties of Services
-

- 4. Scope of Service Systems Engineering
- 5. Value of Service Systems Engineering
- 6. Service Systems Engineering Stages
- Knowledge Area: Enterprise Systems Engineering
 - 1. Enterprise Systems Engineering Background
 - 2. The Enterprise as a System
 - 3. Related Business Activities
 - 4. Enterprise Systems Engineering Key Concepts
 - 5. Enterprise Systems Engineering Process Activities
 - 6. Enterprise Capability Management
- Knowledge Area: Systems of Systems (SoS)
 - 1. Architecting Approaches for Systems of Systems
 - 2. Systems Engineering for Systems of Systems
 - 3. Systems of Systems Analytic Approaches
 - 4. System of Systems and Complexity
 - 5. Socio-Technical Features of Systems of Systems
 - 6. Capability Engineering
 - 7. Mission Engineering
- Knowledge Area: Healthcare Systems Engineering
 - 1. Overview of the Healthcare Sector
 - 2. Systems Engineering in Healthcare Delivery
 - 3. Systems Biology
 - 4. Lean in Healthcare

Part 5: Enabling Systems Engineering

- Knowledge Area: Enabling Businesses and Enterprises
 - 1. Systems Engineering Organizational Strategy
 - 2. Determining Needed Systems Engineering Capabilities in Businesses and Enterprises
 - 3. Organizing Business and Enterprises to Perform Systems Engineering
 - 4. Assessing Systems Engineering Performance of Business and Enterprises
 - 5. Developing Systems Engineering Capabilities within Businesses and Enterprises
 - 6. Barriers to Successful Embedding of Systems Engineering into Organizations
 - 7. Culture
 - Knowledge Area: Enabling Teams
 - 1. Team Capability
 - 2. Team Dynamics
 - 3. Diversity, Equity, and Inclusion
 - 4. Technical Leadership in Systems Engineering
 - Knowledge Area: Enabling Individuals
 - 1. Roles and Competencies
 - 2. Assessing Individuals
 - 3. Developing Individuals
 - 4. Ethical Behavior
-

Part 6: Related Disciplines

- Knowledge Area: Systems Engineering and Environmental Engineering
 - Knowledge Area: Systems Engineering and Geospatial/Geodetic Engineering
 1. Overview of Geospatial/Geodetic Engineering
 2. Relationship between Systems Engineering and Geospatial/Geodetic Engineering
 3. Further Insights into Geospatial/Geodetic Engineering
 - Knowledge Area: Systems Engineering and Industrial Engineering
 - Knowledge Area: Systems Engineering and Project Management
 1. The Nature of Project Management
 2. An Overview of the PMBOK® Guide
 3. Relationships between Systems Engineering and Project Management
 4. The Influence of Project Structure and Governance on Systems Engineering and Project Management Relationships
 5. Cost Estimating and Analysis in Systems Engineering
 6. Procurement and Acquisition
 7. Portfolio Management
 - Knowledge Area: Systems Engineering and Software Engineering
 1. Software Engineering in the Systems Engineering Life Cycle
 2. The Nature of Software
 3. An Overview of the SWEBOK Guide
 4. Key Points a Systems Engineer Needs to Know about Software Engineering
 5. Software Engineering Features - Models, Methods, Tools, Standards, and Metrics
 - Knowledge Area: Systems Engineering and Aerospace Engineering - **Desired Article!**
 - Knowledge Area: Systems Engineering and Electrical Engineering - **Desired Article!**
 - Knowledge Area: Systems Engineering and Mechanical Engineering
 - Knowledge Area: Systems Engineering and Civil Engineering - **Desired Article!**
 - Knowledge Area: Systems Engineering and Economics - **Desired Article!**
 - Knowledge Area: Systems Engineering and Enterprise IT
 - Knowledge Area: Systems Engineering and Quality Attributes
 1. A Framework for Viewing Quality Attributes from the Lens of Loss
 2. Human Systems Integration
 3. Manufacturability and Producibility
 4. System Adaptability
 5. System Affordability
 6. System Hardware Assurance
 7. System Reliability, Availability, and Maintainability
 8. System Resilience
 9. Resilience Modeling
 10. System Resistance to Electromagnetic Interference
 11. System Safety
 12. System Security
-

Part 7: Systems Engineering Implementation Examples

- Matrix of Implementation Examples
 - Implementation Examples
 - Construction System Examples - **Desired Article!**
 - CyberPhysical System Examples - **Desired Article!**
 - Defense System Examples
 1. Submarine Warfare Federated Tactical Systems
 2. Virginia Class Submarine
 - Geospatial System Examples - **Desired Article!**
 - Information System Examples
 1. Complex Adaptive Taxi Service Scheduler
 2. Successful Business Transformation within a Russian Information Technology Company
 3. FBI Virtual Case File System
 - Management System Examples
 1. Project Management for a Complex Adaptive Operating System
 - Medical System Examples
 1. Next Generation Medical Infusion Pump
 2. Medical Radiation
 3. Design for Maintainability
 - Space System Examples
 1. Global Positioning System Case Study
 2. Global Positioning System Case Study II
 3. Russian Space Agency Project Management Systems
 4. How Lack of Information Sharing Jeopardized the NASA/ESA Cassini/Huygens Mission to Saturn
 5. Hubble Space Telescope
 6. Applying a Model-Based Approach to Support Requirements Analysis on the Thirty-Meter Telescope
 7. Miniature Seeker Technology Integration Spacecraft
 8. Apollo 1 Disaster
 - Transportation System Examples
 1. Denver Airport Baggage Handling System
 2. FAA Advanced Automation System (AAS)
 3. Federal Aviation Administration Next Generation Air Transportation System
 4. UK West Coast Route Modernisation Project
 5. Reverse Engineering a UAV Prototype using Agile Practices
 6. Standard Korean Light Transit System Vignette
 - Utilities Examples
 1. Northwest Hydro System
 2. Singapore Water Management
-

Part 8: Emerging Knowledge

- Emerging Topics
 1. Introduction to SE Transformation
 2. Socio-technical Systems
 3. Artificial Intelligence
 4. Verification and Validation of Systems in Which AI is a Key Element
 5. Transitioning Systems Engineering to a Model-based Discipline
 6. Model-Based Systems Engineering Adoption Trends 2009-2018
 7. Digital Engineering
 8. Set-Based Design
- Emerging Research

[Go to First SEBoK Article >](#)

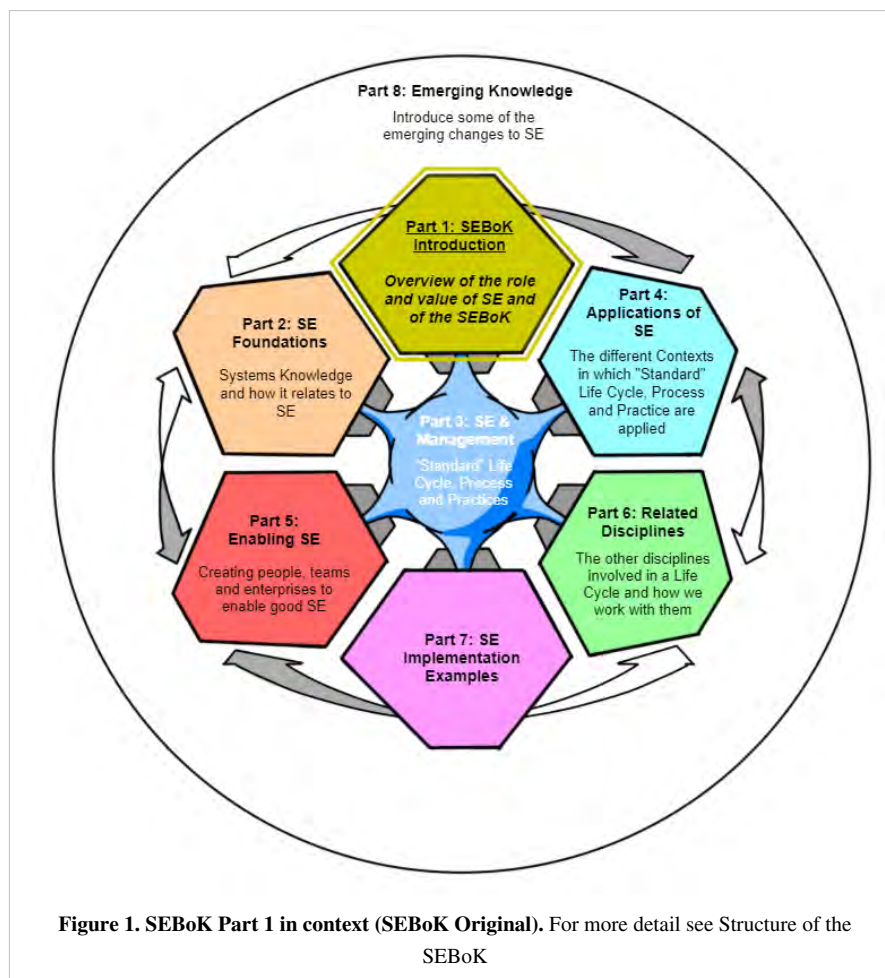
Part 1: SEBoK Introduction

SEBoK Introduction

Contents of this Part

- Introduction to the SEBoK
- Introduction to Systems Engineering
- SEBoK Users and Uses

The purpose of the *Guide to the Systems Engineering Body of Knowledge (SEBoK)* is to provide a widely accepted, community-based, and regularly updated baseline of systems engineering (SE) knowledge. SEBoK Part 1 contains an introduction to both the discipline of SE, and an introduction to and guide for the use of the SEBoK wiki.



Part 1 also includes an introduction to some of the emerging aspects of systems engineering and a discussion of how these are transforming the discipline. As this knowledge matures, it will be migrated into the main body of the SEBoK

Scope and Context of the SEBoK

While Part 1 introduces Systems Engineering knowledge areas, the remaining SEBoK content (Parts 2 – 6) focuses on domain-independent information—that which is universal to systems engineering regardless of the domain in which it is applied. Part 7 includes examples from real projects. These illustrate the concepts discussed elsewhere in the SEBoK, while detailing considerations relevant to domains such as aerospace, medicine, and transportation. Part 8 is intended to introduce some of the more significant emerging changes to systems engineering, both in general topics and emerging research.

SE in the context of engineered systems (ES) is the primary scope for the SEBoK, though general systems concepts are also discussed in Part 2. The SEBoK also covers considerations for the disciplines of software engineering and project management, which are strongly intertwined with the practice of SE (see Part 6).

References

Works Cited

None

Primary References

None

Introduction to the SEBoK

Introduction to the SEBoK

Contents of this Knowledge Area

- Scope of the SEBoK
- Structure of the SEBoK

The SEBoK provides a widely accepted, community-based, and regularly updated baseline of systems engineering (SE) knowledge. Therefore, it is a curated body of knowledge which is updated on a semi-annual basis. This baseline strengthens the mutual understanding across the many disciplines involved in developing and operating systems.

References

Works Cited

None.

Primary References

INCOSE. 2015. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, Fourth Edition. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-004.

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.

Sage, A. and W. Rouse, Eds. 2009. *Handbook of Systems Engineering and Management*, 2nd ed. Hoboken, NJ, USA: John Wiley and Sons, Inc.

Additional References

None.

Scope of the SEBoK

The SEBoK is a large, curated compendium of information about systems engineering. It:

- is a guide to the body of SE knowledge which provides references to detailed sources for additional information; it is not a self-contained knowledge resource
- focuses on Engineered Systems contexts, that is socio-technical systems with a recognized SE life cycle, while treating social and natural systems as relevant and important environmental considerations (see Part 2)
- describes generic SE life cycle and process knowledge (see Part 3)
- recognizes that SE principles can be applied differently to different types of products, services, enterprises, and systems of systems (SoS) context (see Part 4)
- provides resources for organization support of SE activities (see Part 5)
- explores the interaction between SE and other disciplines, highlighting what systems engineers need to know about these disciplines (see Part 6)
- is domain-independent, with implementation examples to provide domain-specific context (see Part 7)

Each of these considerations depends upon the definition and scope of SE itself, which is the subject of the next section.

SEBoK Purposes

Ongoing studies of system cost and schedule failures (Gruhl & Stutzke 2005; Johnson 2006, GAO 2016) and safety failures (Leveson 2012) have shown that the failures have mostly come not from their domain disciplines, but from lack of adequate Systems Engineering (NDIA 2003, 2006, 2016). To provide a foundation for the mutual understanding of SE needed to reduce these failures, the SEBoK describes the boundaries, terminology, content, and structure of SE. In so doing, the SEBoK systematically and consistently supports six broad purposes, described in Table 1.

Table 1. SEBoK Purposes. (SEBoK Original)

#	Purpose	Description
1	Inform Practice	Inform systems engineers about the boundaries, terminology, and structure of their discipline and point them to useful information needed to practice SE in any application domain.
2	Inform Research	Inform researchers about the limitations and gaps in current SE knowledge that should help guide their research agenda.
3	Inform Interactors	Inform performers in interacting disciplines (system implementation, project and enterprise management, other disciplines) and other stakeholders of the nature and value of SE.
4	Inform Curriculum Developers	Inform organizations defining the content that should be common in undergraduate and graduate programs in SE.
5	Inform Certifiers	Inform organizations certifying individuals as qualified to practice systems engineering.
6	Inform SE Staffing	Inform organizations and managers deciding which competencies practicing systems engineers should possess in various roles ranging from apprentice to expert.

The SEBoK is a guide to the body of SE knowledge, not an attempt to capture that knowledge directly. It provides references to more detailed sources of knowledge, all of which are generally available to any interested reader. No proprietary information is referenced, but not all referenced material is free—for example, some books or standards must be purchased from their publishers. The criterion for including a source is simply that the authors & editors believed it offered the best generally available information on a particular subject.

The SEBoK is global in applicability. Although SE is practiced differently from industry to industry and country to country, the SEBoK is written to be useful to systems engineers anywhere. The authors & editors were chosen from diverse locales and industries, and have refined the SEBoK to broaden applicability based on extensive global reviews of several drafts.

The SEBoK aims to inform a wide variety of user communities about essential SE concepts and practices in ways that can be tailored to different enterprises and activities while retaining greater commonality and consistency than would be possible without the SEBoK. Because the world in which SE is being applied continues to evolve and is dynamic, the SEBoK is designed for easy, continuous updating as new sources of knowledge emerge.

SEBoK Uses

The communities involved with SE include its various specialists, engineers from disciplines other than systems engineering, managers, researchers, and educators. This diversity means that there is no single best way to use the SEBoK. The SEBoK includes use cases that highlight potential ways that particular communities can draw upon the content of the SEBoK, identify articles of interest to those communities, and discuss primary users (those who use the SEBoK directly) and secondary users (those who use the SEBoK with assistance from a systems engineer). For more on this, see the article SEBoK Users and Uses.

SEBoK Domain Independent Context

The SEBoK uses language and concepts that are generally accepted for domain-independent SE. For example, the domain-independent conceptual foundations of SE are elaborated in Part 2: Foundations of Systems Engineering. However, each of the numerous domains in which SE is practiced — including telecommunications, finance, medicine, and aerospace — has its own specialized vocabulary and key concepts. Accordingly, the SEBoK is designed to show how its domain-independent material relates to individual domains in two ways.

Firstly, by means of examples that tell stories of how SE is applied in particular domains. Part 7: Systems Engineering Implementation Examples) consists of examples (case studies and vignettes), each set in a particular domain such as aerospace, medicine, or software, and featuring vocabulary and concepts special to that domain. There are similar vignettes in some of the Use Cases in Part 1. These examples demonstrate the effect of domain on the application of SE and complement the domain-independent information elsewhere in the SEBoK. They show how a concept works in a given domain and provide a fair opportunity for reviewers to reflect on whether there are better ways to capture application-dependent aspects of SE knowledge.

In addition, the SEBoK will contain knowledge areas in Part 4: Applications of Systems Engineering which explicitly describe the domain specific language, approaches, specialized processes and tools, etc. of particular application domains. In this version of the SEBoK, there are a limited set of domain knowledge areas.

References

Works Cited

- GAO. 2016. *Weapon System Requirements. Detailed Systems Engineering Prior to Product Development Positions Programs for Success*. Government Accounting Office Report to Congressional Committees.
- Gruhl, W. and Stutzke, R. 2005. "Werner Gruhl analysis of SE investments and NASA overruns," in R. Stutzke, *Estimating Software-Intensive Systems*. Boston, MA, USA: Addison Wesley, page 290.
- Johnson, J. 2006. *My Life Is Failure: 100 Things You Should Know to Be a Better Project Leader*. Boston, MA, USA: Standish Group International.
- Leveson, N. 2012. *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA, USA: MIT Press, NDIA (National Defense Industrial Association). 2003. *Top 5 Systems Engineering Issues within DOD and*
-

Defense Industry: Task Report. Version 9, released 1/23/03. Available at: https://www.atcourses.com/sampler/TopFiveSystemsEngineeringIssues_In_DefenseIndustry.pdf^[1]. Accessed October 25, 2019.

NDIA (National Defense Industrial Association). 2006. *Top 5 Systems Engineering Issues within DOD and Defense Industry DOD and Defense Industry: Task Report*. Version 8a, released July 26-27, 2006. Available at: <https://ndiastorage.blob.core.usgovcloudapi.net/ndia/2006/systems/Wednesday/rassa5.pdf>. Accessed October 25, 2019.

NDIA (National Defense Industrial Association). 2016. *Top Systems Engineering Issues In US Defense Industry 2016*. Version 7c. Available at: <https://www.ndia.org/-/media/sites/ndia/divisions/systems-engineering/studies-and-reports/ndia-top-se-issues-2016-report-v7c.ashx?la=en>. Accessed October 25, 2019.

Primary References

None.

Additional References

None.

References

[1] <https://ndiastorage.blob.core.usgovcloudapi.net/ndia/2006/systems/Wednesday/rassa5.pdf>

Structure of the SEBoK

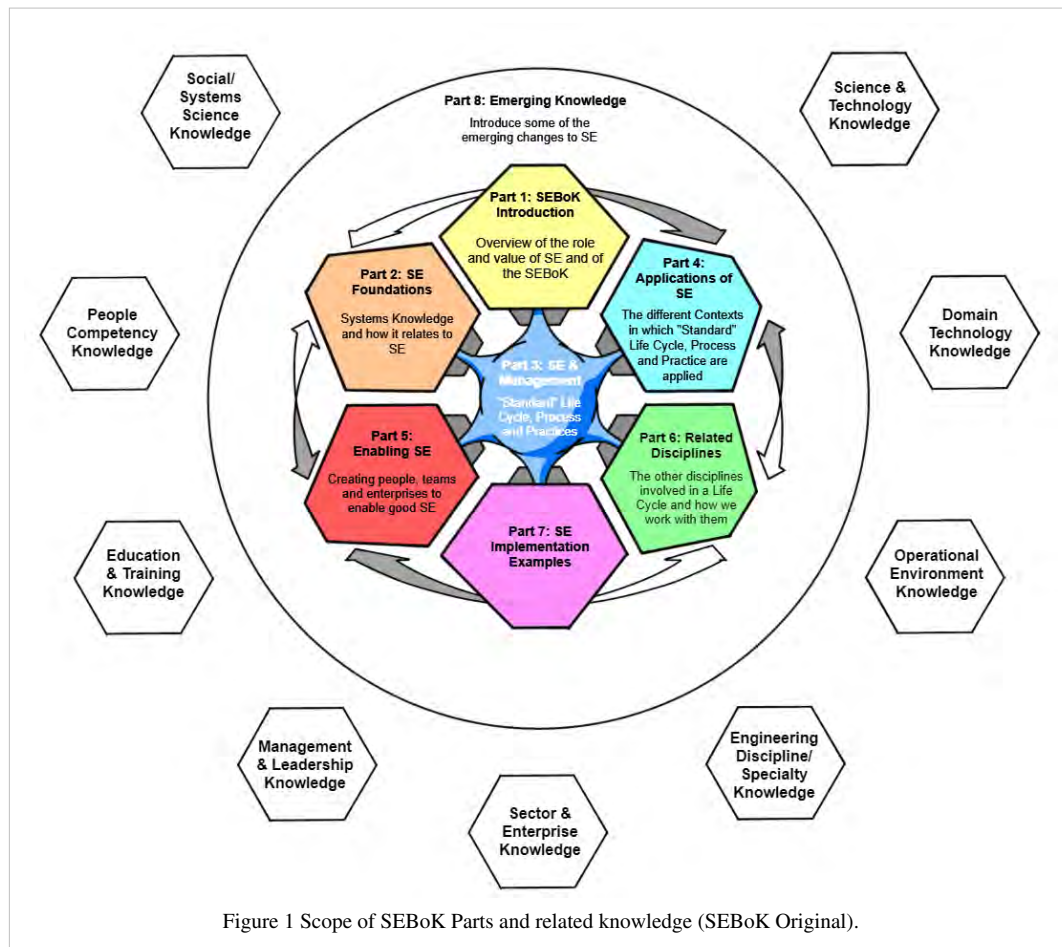
The **Guide to the Systems Engineering Body of Knowledge (SEBoK)** is a living authoritative guide that discusses knowledge relevant to Systems Engineering. SEBoK does not contain all of this knowledge itself but provides a starting point and key resources to allow the reader to navigate the wider body of knowledge that exists in published sources. To do this, SEBoK:

- Defines relevant knowledge and structures it to facilitate understanding.
- Provides short discussions of key ideas, principles and concepts within that structure.
- Points to reference sources important to the discipline, which explore these ideas in more detail.

In doing this, it is inevitable that differences in terminology, alternative approaches, and even fundamentally different ways of thinking within the knowledge will appear. SEBoK attempts were possible to provide clarity of similar or overlapping ideas, or to highlight real differences and the reasons behind them. In particular, the SEBoK Glossary of Terms contains the most used or generally agreed upon definitions of terms when it can, but may highlight more than one definition if needed to show breadth of current thinking.

SEBoK Structure

Figure 1, below, illustrates the eight parts of the SEBoK and how they are related.



The scope of each part and the key relationships amongst them are briefly discussed below. For a more detailed discussion of how this structure was evolved, see (Adcock et al, 2016).

Overview of Parts

Part 1: SEBoK Introduction

This part explains the scope, context, and structure of the SEBoK, and of systems engineering (SE).

An overview of who should use the SEBoK, and for what purpose, is followed by detailed use cases. Systems engineering's economic value, history, and relationship to other disciplines are discussed. Part 1 also contains a section which discusses the future evolution of the SEBoK and allows for new areas of content to be introduced before being transitioned into other SEBoK parts.

Part 2: Foundations of Systems Engineering

This part provides an introduction and overview of areas of knowledge which provide the foundations of SE.

A discussion of the definitions and basic concepts of systems is followed by an overview of the principles, concepts, methods, models and patterns of some of the key foundational areas of systems science. This includes a detailed consideration of the foundational knowledge related to systems models and modelling.

Part 2 looks in more detail at two aspects of this foundational knowledge of particular value to SE. The first is to discuss aspects of systems knowledge related to a systems approach to complex problems and opportunities. This approach provides foundations for how SE is defined and practiced (see Parts 3 and 5 below). The second is to describe the different ways in which system concepts are applied to real-world concerns. The SEBoK defines an engineered system (ES) as the primary focus for the application of SE (see Part 4 below).

Part 3: Systems Engineering and Management

This part describes generic knowledge on the practice of SE and related management activities.

Part 3 begins with the life cycle models common in SE and the general principles behind their application. It then moves on to SE management activities. It covers both technical activities such as requirements, architecture, test and evaluation; and management activities such as planning, measurement, and risk. Next is product and service life management, a distinct area of SE management that emphasizes the entire life cycle including retirement and disposal. An account of SE standards concludes this part.

Focused on what many think of as the main body of SE, including best practices and common pitfalls, this part constitutes a substantial proportion of the SEBoK. As already discussed, the knowledge in Part 3 is based on the systems approach from Part 2. The links between Part 3 and the other parts of the SEBoK are discussed below.

Part 4: Applications of Systems Engineering

This part describes how to apply SE principles to different types of system context.

Part 4 focuses on four major engineered system contexts in turn: products, services, enterprises, and systems of systems (SoS). For each one, the system abstraction, commercial relationships and application of generic SE is described.

The generalized contexts above should be viewed as overlapping models of how SE can be applied in different kinds of situations. Combinations of one or more of them are fully realized when applied in an application domain. Part 4 currently describes this application in a small number of such domains. This will be expanded in later updates. The applications of SE in this part describe the real-world practice of SE. The generalized knowledge in both Parts 2 and 3 evolves through what we learn from these applications. Part 2 includes a discussion of this relationship between theory and practice.

Part 5: Enabling Systems Engineering

This part describes approaches to organization that may enable the successful performance of SE activities.

Part 4 covers knowledge at the enterprise, team, or individual level. The range of considerations extends from value proposition, business purpose, and governance, down to competency, personal development as a systems engineer, and ethics.

All of these relate to the baseline definitions of SE in Part 3, further generalized in the levels of application in Part 4. The systems approach in Part 2 should also form a foundation for this part. Since the practice of SE is transdisciplinary, Part 5 also has a link to Part 6 as discussed below.

Part 6: Related Disciplines

This part describes the relationships between SE and other disciplines.

Part 6 covers the links between SE and software engineering (SwE), project management (PM), industrial engineering (IE) and procurement. It also describes how SE is related to specialty engineering, which describes the various system “-ilities” (like reliability, availability, and maintainability) that SE must balance and integrate.

The knowledge in this part provides an interface to other bodies of knowledge, focused on how it is linked to Parts 3, 4 and 5 above.

Part 7: Systems Engineering Implementation Examples

A set of real-world examples of SE activities demonstrates implementations of the systems engineering knowledge in previous parts of the SEBoK. These examples come in two forms: case studies, which refer the reader to and summarize published examinations of the successes and challenges of SE programs, and vignettes, which are brief, self-contained wiki articles. This part is a key place to look within the SEBoK for lessons learned, best practices, and patterns. Many links connect material in the examples to the conceptual, methodological, and other content elsewhere in the SEBoK.

Part 8: Emerging Knowledge

One of the challenges associated with a body of knowledge is that cutting edge and/or emerging ideas are difficult to include. Bodies of knowledge are based on existing literature and resources, and these often do not exist for new topics. To address this, Part 8 of the SEBoK contains those emerging ideas and items that are not easily covered in the other sections of the SEBoK. As these areas mature and as a body of literature is created around them, they will be moved into the other Parts of the SEBoK.

Addenda

The SEBoK contains a Glossary of Terms, which provides authoritatively-referenced definitions of key terms. This information is displayed when the reader hovers the mouse pointer over a glossary term within an article. It also contains a list of Primary References, with additional information about each reference. Quicklinks in the left margin provide additional background information, including a table of contents, a listing of articles by topic ^[1], and a list of Acronyms.

References

Works Cited

Adcock, R., Hutchison, N., Nielsen, C., 2016, "Defining an architecture for the Systems Engineering Body of Knowledge," Annual IEEE Systems Conference (SysCon) 2016.

Primary References

None.

Additional References

None.

References

[1] <https://sebokwiki.org/1.1.1/index.php?title=Category:Topic>

Introduction to Systems Engineering

Introduction to Systems Engineering

Contents of this Knowledge Area

- Systems Engineering Overview
- Fundamentals for Digital Engineering (Emiliya Suprun and Sondoss Elsawah)
- Economic Value of Systems Engineering
- A Brief History of Systems Engineering (Robert Cloutier) (Michael Pennotti)
- Systems Engineering: Historic and Future Challenges
- Systems Engineering and Other Disciplines
- Fundamentals for Future Systems Engineering (Rick Adcock) (Duane Hybertson)

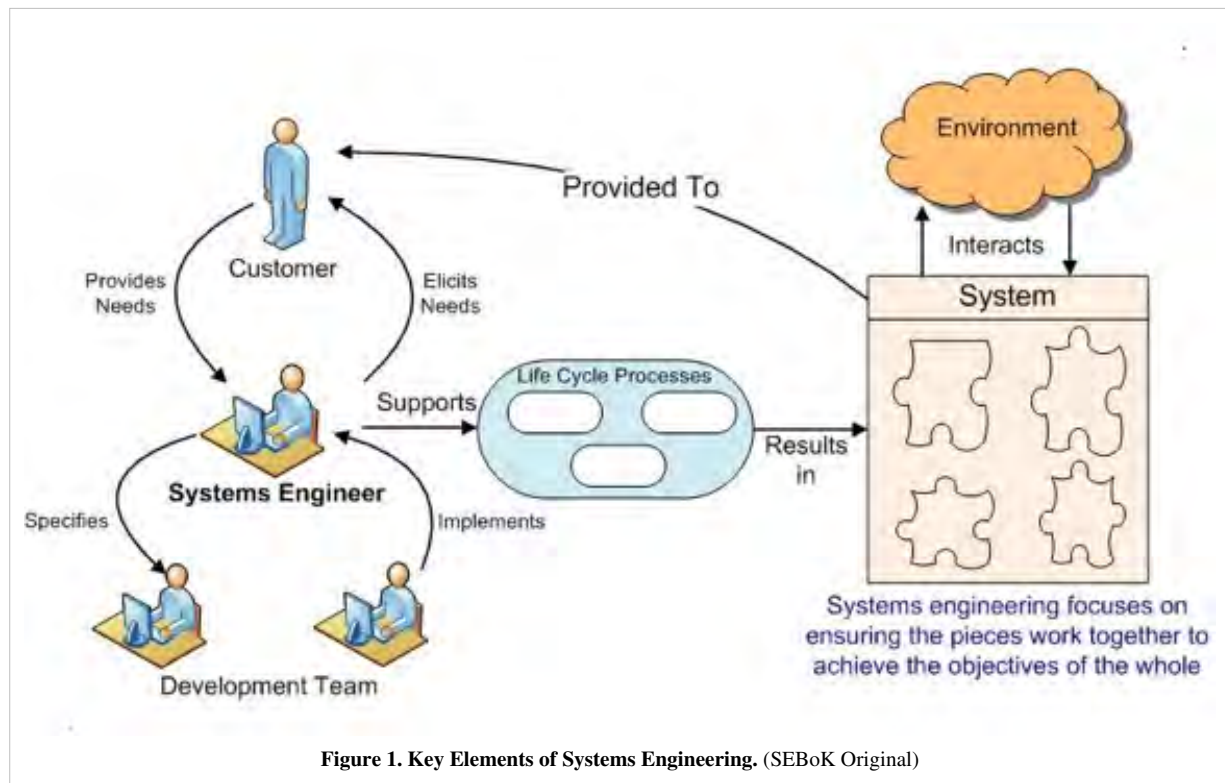
The primary focus of the SEBoK is on the current baseline of knowledge describing the practice of domain independent systems engineering (SE). This Knowledge Area (KA) contains topic articles which provide an overview of SE practice and discuss its economic value, historic evolution and key relationships.

Systems Engineering

Systems Engineering is a transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems, using systems principles and concepts, and scientific, technological, and management methods. (INCOSE 2023)

Successful systems must satisfy the needs of their customers, users and other stakeholders. Some key elements of systems engineering are highlighted in Figure 1 and include:

- The principles and concepts that characterize a system, where a system is an interacting combination of system elements that accomplish a defined objective(s). The system interacts with its environment, which may include other systems, users, and the natural environment. The system elements that compose the system may include hardware, software, firmware, people, information, techniques, facilities, services, and other support elements.
 - A systems engineer is a person or role who supports this transdisciplinary approach. In particular, the systems engineer often serves to elicit and translate customer needs into specifications that can be realized by the system development team.
 - In order to help realize successful systems, the systems engineer supports a set of life cycle processes beginning early in conceptual design and continuing throughout the life cycle of the system through its manufacture, deployment, use and disposal. The systems engineer must analyze, specify, design, and verify the system to ensure that its functional, interface, performance, physical, and other quality characteristics, and cost are balanced to meet the needs of the system stakeholders.
 - A systems engineer helps ensure the elements of the system fit together to accomplish the objectives of the whole, and ultimately satisfy the needs of the customers and other stakeholders who will acquire and use the system.
-



References

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.

Systems Engineering Overview

Systems engineering (SE) is a transdisciplinary approach and means to enable the realization of successful systems. Successful systems must satisfy the needs of their customers, users and other stakeholders. This article discusses the complexities of systems and systems engineering (SE), highlighting how SE manages ambiguities in terminology and integrates with broader disciplines. It introduces specific SE definitions, emphasizing its focus on engineered systems and their contexts. SE's scope excludes activities like manufacturing. The article outlines SE's transdisciplinary approach and its role within the project life cycle, where it collaborates with various agents to create and evolve successful engineered systems. Notably, SE involves negotiating changes proposed by stakeholders and mediated by systems engineers. Overall, the article underscores SE's pivotal role in realizing and managing complex engineered systems within diverse environments.

Systems and Systems Engineering

In the broad community, the term “system” may mean a collection of technical, natural or social elements, or a combination of all three. This may produce ambiguities at times: for example, does “management” refer to management of the SE process, or management of the system being engineered? As with many special disciplines, SE uses terms in ways that may be unfamiliar outside the discipline. For example, in systems science and therefore SE, “open” means that a system is able to interact with its environment--as opposed to being “closed” to its environment. However, in the broader engineering world we would read “open” to mean “non-proprietary” or “publicly agreed upon.” In such cases, the SEBoK tries to avoid misinterpretation by elaborating the alternatives e.g. “system management” or “systems engineering management”.

A useful introduction to systems engineering from Colorado Technical University can be found here: <https://www.youtube.com/watch?v=aZXVRvZJT4I>.

The SEBoK seeks to position SE within the broader scope of knowledge which considers systems as part of its foundations. To do this without attempting to re-define general systems terminology, SEBoK introduces two related definitions specific to SE:

- An engineered system is a technical or socio-technical system which is the subject of an SE life cycle. It is a system designed or adapted to interact with an anticipated operational environment to achieve one or more intended purposes while complying with applicable constraints.
- An engineered system context centers around an engineered system but also includes in its relationships other engineered, social or natural systems in one or more defined environments.

Since the province of SE is engineered systems, most SE literature assumes this in its terminology. Thus, in an SE discussion, “system architecture” would refer to the architecture of the system being engineered (e.g., a spacecraft) and not the architecture of a natural system outside its boundary (e.g., the solar system). In fact, a spacecraft architecture would cover the wider system context including external factors such as changes in gravity and external air pressure and how these affect the spacecraft's technical and human elements. Thus, the term “system architecture” more properly refers to the engineered system context. The SEBoK tries to be more explicit about this, but may still make these kinds of assumptions when referring directly to other SE literature.

An extensive glossary of terms identifies how terms are used in the SEBoK and shows how their meanings may vary in different contexts. As needed, the glossary includes pointers to articles providing more detail.

For more about the definition of systems, see the article What is a System? in Part 2. The primary focus of SEBoK Part 3: Systems Engineering and Management and Part 4: Applications of Systems Engineering is on how to create or change an engineered system to fulfill the goals of stakeholders within these wider system contexts. The knowledge in Part 5: Enabling Systems Engineering and Part 6: Systems Engineering and Other Disciplines examines the need for SE itself to be integrated and supported within the human activity systems in which it is

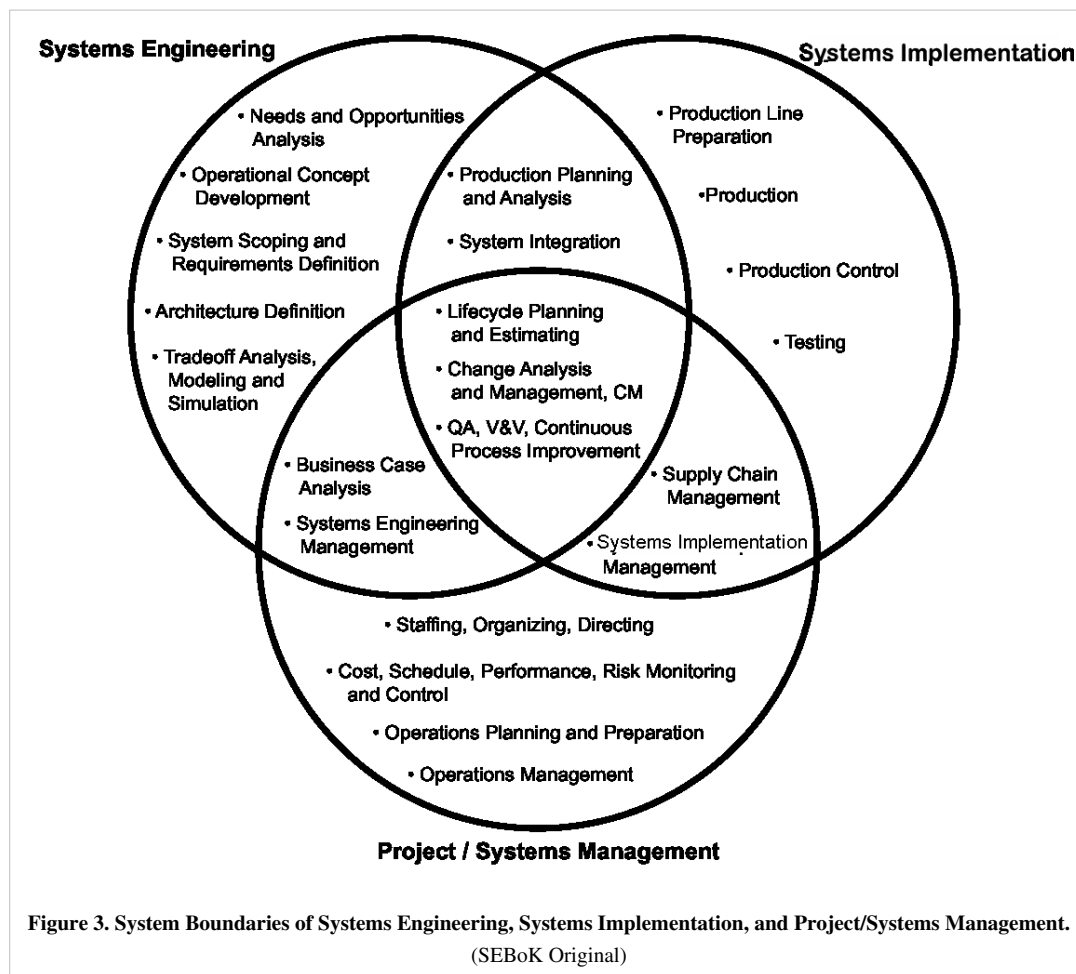
performed, and the relationships between SE and other engineering and management disciplines.

The MATLAB team created this 15-minute overview of systems engineering as a discipline: <https://www.youtube.com/watch?v=pSfZutP9H-U>.

Scope of Systems Engineering within the Engineered Systems Domain

The scope of SE does not include everything involved in the engineering and management of an engineered system. Activities can be part of the SE environment, but other than the specific management of the SE function, are not considered to be part of SE. Examples include system construction, manufacturing, funding, and general management. This is reflected in the International Council on Systems Engineering (INCOSE) top-level definition of systems engineering as, “A transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems, using systems principles and concepts, and scientific, technological, and management methods” (Fellows 2019). Although SE can *enable* the realization of a successful system, if an activity that is outside the scope of SE, such as manufacturing, is poorly managed and executed, SE cannot *ensure* a successful realization.

A convenient way to define the scope of SE within engineering and management is to develop a Venn diagram. Figure 3 shows the relationship between SE, system implementation, and project/systems management. Activities, such as analyzing alternative methods for production, testing, and operations, are part of SE planning and analysis functions. Such activities as production line equipment ordering and installation, and its use in manufacturing, while still important SE environment considerations, stand outside the SE boundary. Note that as defined in Figure 3, system implementation engineering also includes the software production aspects of system implementation. Software engineering, then, is not considered a subset of SE.



Traditional definitions of SE have emphasized sequential performance of SE activities, e.g., “documenting requirements, then proceeding with design synthesis ...” (INCOSE 2012). Originally, the SEBoK authors & editors departed from tradition to emphasize the inevitable intertwining of system requirements definition and system design in the following revised definition of SE:

Systems Engineering (SE) is an interdisciplinary approach and means to enable the realization of successful systems. It focuses on holistically and concurrently understanding stakeholder needs; exploring opportunities; documenting requirements; and synthesizing, verifying, validating, and evolving solutions while considering the complete problem, from system concept exploration through system disposal. (INCOSE 2012, modified.)

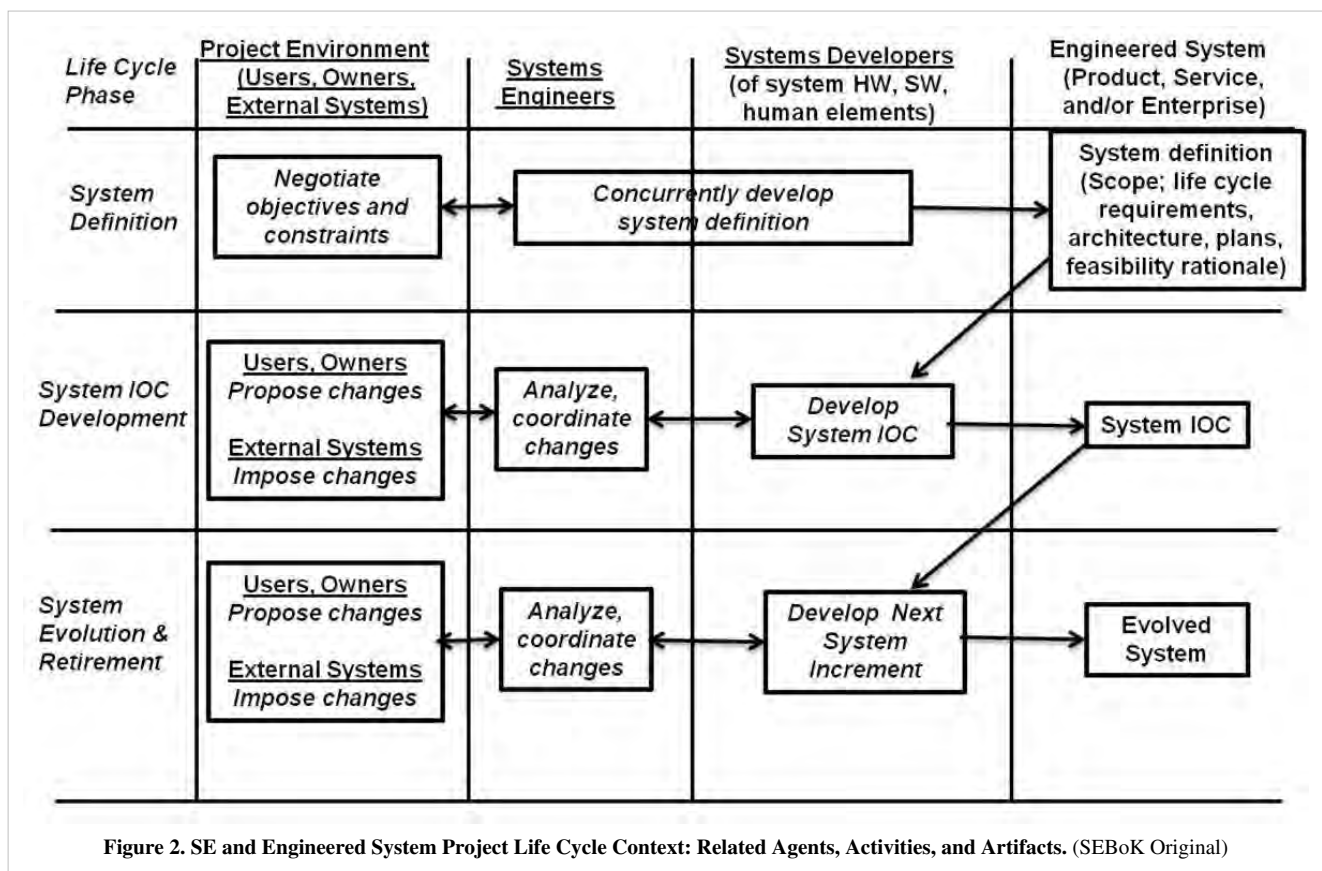
More recently, the INCOSE Fellows have offered an updated definition of SE that has been adopted as the official INCOSE definition:

A transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems, using systems principles and concepts, and scientific, technological, and management methods.

Part 3: Systems Engineering and Management elaborates on the definition above to flesh out the scope of SE more fully.

Systems Engineering and Engineered Systems Project Life Cycle Context

SE is performed as part of a life cycle approach. Figure 2 summarizes the main agents, activities, and artifacts involved in the life cycle of SE, in the context of a project to create and evolve an engineered system.



For each primary project life cycle phase, we see activities being performed by primary agents, changing the state of the ES.

- Primary project life cycle phases appear in the leftmost column. They are system definition, system initial operational capability (IOC) development, and system evolution and retirement.
- Primary agents appear in the three inner columns of the top row. They are systems engineers, systems developers, and primary project-external bodies (users, owners, external systems) which constitute the project environment.
- The ES, which appears in the rightmost column, may be a product, a service, and/or an enterprise.

In each row:

- boxes in each inner column show activities being performed by the agent listed in the top row of that column
- the resulting artifacts appear in the rightmost box.

Arrows indicate dependencies: an arrow from box A to box B means that the successful outcome of box B depends on the successful outcome of box A. Two-headed arrows indicate two-way dependencies: an arrow that points both from box A to box B and from box B to box A means that the successful outcome of each box depends on the successful outcome of the other.

For example, consider how the inevitable changes that arise during system development and evolution are handled:

- One box shows that the system's users and owners may propose changes.
- The changes must be negotiated with the systems developers, who are shown in a second box.
- The negotiations are mediated by systems engineers, who are shown in a third box in between the first two.
- Since the proposed changes run from left to right and the counter-proposals run from right to left, all three boxes are connected by two-headed arrows. This reflects the two-way dependencies of the negotiation.

An agent-activity-artifact diagram like Figure 1 can be used to capture complex interactions. Taking a more detailed view of the present example demonstrates that:

- The system's users and owners (stakeholders) propose changes to respond to competitive threats or opportunities, or to adapt to changes imposed by independently evolving external systems, such as Commercial-off-the-Shelf (COTS) products, cloud services, or supply chain enablers.
- Negotiation among these stakeholders and the system developers follows, mediated by the SEs.
- The role of the SEs is to analyze the relative costs and benefits of alternative change proposals and synthesize mutually satisfactory solutions.

References

Works Cited

- Checkland, P. 1981. *Systems Thinking, Systems Practice*. Hoboken, NJ, USA: Wiley.
- INCOSE. 2012. *Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.
- Fellows. 2019. *INCOSE Fellows Briefing to INCOSE Board of Directors*. January 2019.
- Rechtin, E. 1991. *Systems Architecting*. Upper Saddle River, NJ, USA: Prentice Hall.
-

Primary References

INCOSE. 2012. *Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.

Additional References

None.

Fundamentals for Digital Engineering

- Lead Authors:
- Emiliya Suprun and Sondoss Elsayah

The world has witnessed exponential growth in technological advancement in recent years, and systems engineering (SE) is no exception. SE is transforming the way we design, analyze, and manage complex systems. This transformational change leverages the use of advanced technologies such as Artificial Intelligence (AI), Big Data, and Internet of Things (IoT). However, the evolution of SE goes well beyond just technology advancements. This chapter discusses digital engineering (DE) as a systemic intervention, a holistic approach to SE transformation that involves people, processes, data, and technology.

INCOSE Vision 2035: Transforming SE

The INCOSE Vision 2035 statement (INCOSE 2021) provides insights into the global context for SE, highlighting key trends and influencing factors expected to drive changes in the practice of SE. Some of the principal takeaways include but are not limited to the impacts of digital transformation:

- Digital transformation, sustainability, smart systems and complexity growth, and advancements in modeling, simulation, and visualization are trends that impact enterprise competitiveness.
- The future SE will leverage digital transformation in its tools and methods and move towards a fully model-based SE environment.
- The changing nature of systems includes more embedded and application software, increasing amounts of data to process, and cyber-physical systems.
- The systems of the future will be engineered by an evolving, diverse workforce reshaped by digital transformation and systems of systems.
- Data science techniques will be infused into the SE practice, with trusted data being managed as a critical asset.
- Digital technology, including the broader application of AI, enables the transformation of how enterprises capture, reuse, exploit, and protect knowledge through digital representation and semantic integration of all information.
- Evolving digital ecosystems will enable automation and autonomy to perform increasingly complex tasks.

All engineering disciplines today have evolved, and modern SE needs to be supported by large amounts of data. This requires the transformation of SE practices to DE in which technological innovations are assembled to allow for an integrated digital approach that supports life cycle activities and develops a culture among stakeholders of working more efficiently and effectively (DoD 2018). DE is not a destination but a journey towards the digital transformation of engineering and enterprises. This journey requires a paradigm shift from a traditional paper-based SE to a more integrated approach that leverages digital technology to accelerate the formalization and integration of knowledge, fostering collaboration and adopting a more agile approach to SE.

What is DE?

DE provides an integrated digital approach that uses authoritative sources of systems data and models as a continuum across disciplines to support lifecycle activities from concept through disposal (DoD, 2018). ABAB (2021) defines DE as a collaborative way of working, using digital processes to enable more productive methods of planning, designing, constructing, operating, and maintaining assets. Collaboration with suppliers and partners is improved through DE, reducing the risk of errors and delays. DE is reshaping the landscape of how capabilities, assets and projects are designed and managed across many industries, including defense, transportation, construction, healthcare delivery, and other sectors. Examples include:

- In the defense sector, DE enables the development of innovative weapon systems, reduction in acquisition timelines and improvement of the force's readiness (Zimmerman et al. 2019).
- In transportation, DE improves the design and optimization of infrastructure systems, such as roads, bridges, and airports, to improve safety and efficiency (Wu et al. 2022). DE drives the improvement of the resilience of critical infrastructure systems, such as power grids, water systems, and transportation networks (Suprun et al. 2022).
- In construction, DE enables the design and construction of smarter and more sustainable buildings and infrastructure and the reduction of construction costs and waste (Opoku et al. 2021).
- In healthcare, DE enables development of new medical devices, optimization of hospital operations, and improvement in patient outcomes (Awad et al. 2021). The list goes on.

DE as a Systemic Intervention in Its Own Right

DE is underpinned by the view of enterprises as socio-cultural-technical systems (Sony & Naik, 2020). DE is a systemic intervention that comprises four fundamental components: Why, What, Who and How. These components work together to enable fundamental shifts in organizational culture, practices, and technologies to achieve enterprise digital transformation. Figure 1 provides a holistic view of the dimensions and elements that make the DE intervention.

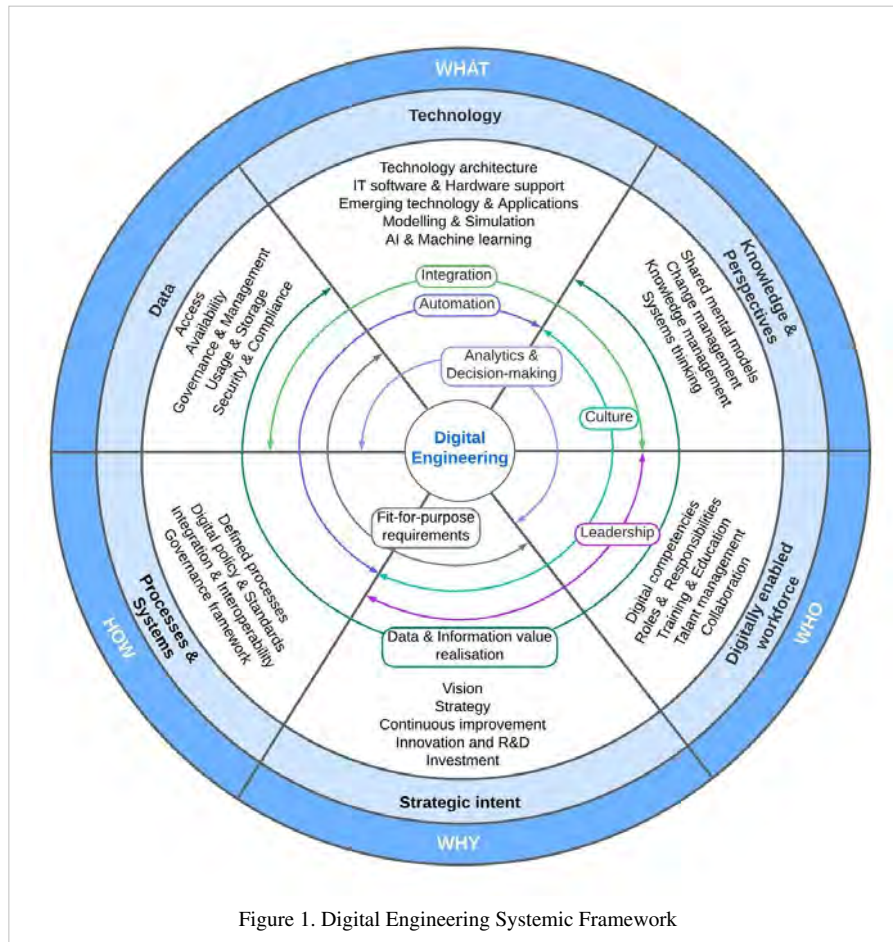


Figure 1. Digital Engineering Systemic Framework

Why: Strategic Intent

The *why* component is critical for establishing the goals and objectives of the DE initiative, as well as for ensuring that it aligns with the broader organizational strategy. DE is not exclusively about technology but the fact that technology enables organizations to transform their business operations and systems and create better-informed decisions around their organizational objectives and strategy.

As part of the process, the immediate targets of transforming SE practices include the digitalization of engineering artifacts, information, and model sharing, while the long-term strategic vision is focused on aligning investments into technological innovations and advanced engineering practices with organizational goals and objectives (Huang, 2020). The organizational view of digital transformation opportunities and innovation should be considered as part of a more extensive continuous improvement process.

A clear DE strategy provides a roadmap for digital transformation, outlining the steps needed to achieve organizational outcomes. They should also consider the organization's culture and how DE transformation will impact the workforce as well as its existing systems and processes. A successful strategy involves stakeholder collaboration to ensure buy-in and alignment with the organization's goals and leadership development. DoD DE

Strategy (DoD 2018) identifies the following goals that could be applied to DE activities: (i) model use for decision-making; (ii) developing the authoritative source of truth (AST); (iii) enhancing technological innovation; (iv) establishing collaborative environments; and (v) transforming the workforce and cultural.

A systemic approach to DE drives a shift from a technology-focused acquisition approach to a whole ecosystem view that supports all enablers for genuine transformation, including workforce readiness and awareness of the legal and policy environment. Moreover, the strategic view on DE allows organizations to realize the value of data and information. Industries must collectively recognize the importance of valuing and managing data and information as critical assets.

Who: Digitally Enabled Workforce

The *who* component of DE focuses on the workforce. This component is critical for ensuring industries have the necessary skills and capabilities to embark on the DE journey. It involves identifying the skills, roles, responsibilities, and capabilities needed for the successful digital transformation of engineering and enterprises. It is essential to have a digitally enabled workforce trained and capable of using technological innovation to effectively leverage digital tools and methods.

DE requires a new comprehensive set of skills and competencies beyond traditional SE and IT. It requires a combination of technical and specific data management expertise, such as modeling and simulation, data analytics, AI, and cybersecurity, as well as soft skills, such as communication, collaboration, and leadership. The DE competency framework developed by Hutchison and Tao (2022) identifies the foundational digital competencies: (i) digital literacy; (ii) DE value proposition; (iii) DoD Policy/Guidance; (iv) decision-making; (v) software literacy.

Social and cultural aspects of DE also include the need for a shift in mindset and culture. DE requires a more collaborative agile approach to design, develop, and deploy complex systems and a willingness to adapt and embrace change. It also requires a culture of innovation, continuous improvement and learning as technologies and tools evolve rapidly. The growing worldwide demand for digital and systems engineers exceeds the available supply (INCOSE 2021). Many enterprises initiate internal in-house training programs to develop their workforce further. Nevertheless, there is a need for training and education programs and a life-long learning pipeline that empower more system engineers with strong multi- and transdisciplinary competencies, including digital, business, leadership and systems thinking, needed to enable collaboration across a broad range of the engineering and management workforce globally.

What: Data, Technology, and Knowledge and Perspectives

The *what* component captures dimensions of integration, being data, technology, and knowledge and perspectives. SE faces a significant obstacle today due to the extensive fragmentation among the engineering tools and data landscape (INCOSE 2021). In DE, the use of common integrated models helps overcome these challenges. Integration across different tools, technological solutions and data is becoming a focus for enabling collaboration, analytics, and data-informed decision-making. Integration also ensures that the data generated by different tools are standardized and can be seamlessly shared across different engineering disciplines.

Data is at the heart of DE, and enterprises need to invest in the capabilities for seamless collection, management, and analysis of data to support their digital transformation. Real-time data can improve how complex systems and assets are operated, enable informed decision-making, and allow better responses to disruptions, failures, and environmental concerns. Nevertheless, collecting the trusted data alone is insufficient to improve SE practices. The key is to support an integrated approach enabling the use of high-quality data, security, privacy, and accessibility in a form that allows sharing, visualization, and analysis. For the systems and assets to be managed efficiently, it is crucial to enable consistent, informed decision-making that relies on high-quality, robust data. An organization's insights are only as good as its data. In other words, the value maximization from the physical system requires the

value maximization from the data and information. Valuing and managing data and information as critical assets allow a fundamental shift towards data-driven decision-making built on a 'digital by default' culture.

Knowledge is another critical enterprise asset. Both data and knowledge must be managed appropriately for an enterprise to continue to learn and advance. The knowledge and perspectives component of DE encompasses the importance of understanding and sharing common mental models across stakeholders and establishing a shared understanding of DE. Failure to address the underlying cultural shift, a change in mindset, and a new way of thinking will result in "putting a new coat of technological paint over the same crumbling organizational walls" (Forsythe & Rafoth 2022). Hence, DE must be approached as a cultural transformation, where stakeholders across the enterprise are encouraged to adopt new ways of thinking and working together. This requires a top-down commitment to the new approach and investment in training, education, and tools to enable all stakeholders to understand and work within the new paradigm. This shift in mindset and culture can be difficult and requires leadership to drive the change (Laskey et al. 2021).

The technology component is critical for ensuring organizations have the infrastructure, tools, and platforms to support DE. Technology transforms how enterprises capture, reuse, exploit, and protect knowledge through digital representation and semantic integration of all information. Evolving technology, including digital twins, modeling and simulation tools, and data analytics and visualization tools with the broader application of AI, ML, and IoT, will enable automation and autonomy to perform increasingly complex tasks, providing further opportunities for humans to add value through innovation.

The key shift enabled by DE as an integrated approach is a move to collaborative data-driven problem-solving practices. This drives culture transformation and organizational commitment to a 'single source of truth' approach to decision-making.

How: Processes and Systems

The *how* component of DE focuses on processes and systems. As the future of SE, DE is model-based, leveraging next-generation modeling and simulation powered by global digital transformation (INCOSE 2021). The right processes and systems in place provide consistent digital transformation to deliver value throughout system elements, disciplines, the life cycle, and the enterprise. Defined processes and structured procedures ensure that information fits its intended purpose and can be shared and reused. Moreover, organizations must develop a systematic approach to integrate DE tools and techniques into their existing processes and systems to allow for a seamless flow of data, information, knowledge, and expertise.

DE opens new opportunities but also introduces new integration challenges. One of the critical challenges is the need for interoperability and standardization. Standardized or commonly shared digital representation forms, semantics, and vocabulary are critical for sharing digitalized engineering artefacts, especially digital models.

The core concept in DE is using digital models integrated with simulation, multi-disciplinary analysis, and immersive visualization environments. In other words, one of DE's key technical aspects is using model-based systems engineering (MBSE). MBSE provides a common language and framework for communicating and managing system requirements, design, and implementation. MBSE also enables system designers and engineers to simulate and test system behavior in a virtual environment, reducing the need for physical prototypes. The transformation to DE is happening step by step, and the evolution goes beyond just MBSE. While MBSE is a critical component of DE, it is only one part of a more significant shift towards digital transformation (McDermott et al. 2022). DE involves a fundamental shift in mindset and requires organizations to embrace a more collaborative approach to problem-solving, with stakeholders from across the organization working together to develop more efficient, effective, and responsive solutions to rapidly changing industry demands.

The key shift enabled by DE processes and systems is a move towards the end-to-end digital representation of the enterprise to address long-standing challenges associated with complexity, uncertainty, and rapid change in deploying and using systems. DE also drives a shift from confined and ad-hoc use of models for immediate benefits

to continuous and coherent use of models across the lifecycle to drive and accelerate organizational outcomes.

Conclusions

DE goes beyond using specific software tools and models. It builds upon the principles of SE, leveraging technologies and modeling and simulation methods to enhance data-driven decision-making and optimize system performance. DE is about creating a culture of innovation, collaborative problem-solving, and continuous improvement, changing how we connect, understand, and navigate our environments.

References

Works Cited

- ABAB. 2021. Digital twins: An ABAB Position paper. Australasian BIM Advisory Board.
- Awad, A., S.J. Trenfield, T.D. Pollard, J.J. Ong, M. Elbadawi, L.E. McCoubrey, A. Goyanes, S. Gaisford, and AW Basit. 2021. "Connected healthcare: Improving patient care using digital health technologies." *Advanced Drug Delivery Reviews*, 178, 113958.
- DoD. 2018. Digital Engineering Strategy. US Department of Defense.
- Forsythe, J., and J. Rafoth. 2022. Being Digital: Why Addressing Culture and Creating a Digital Mindset are Critical to Successful Transformation. *INSIGHT*, 25: 25-28.
- Huang, J., A. Gheorghe, H. Handley, P. Pazos, A. Pinto, S. Kovacic, A. Collins, C. Keating, A. Sousa-Poza, G. Rabadi, R. Unal, T. Cotter, R. Landaeta, and C. Daniels. 2020. Towards digital engineering: the advent of digital systems engineering. *International Journal of System of Systems Engineering*, 10(3), 234-261.
- Hutchison, N., and H.Y.S. Tao. 2022. The Digital Engineering Competency Framework (DECF): Critical Skillsets to Support Digital Transformation. *INSIGHT*, 25: 35-39.
- INCOSE. 2021. Engineering solutions for a better world: Systems Engineering Vision 2035. International Council on Systems Engineering.
- Laskey, K.J., M.L. Farinacci, and O.C. Diaz. 2021. Digital Engineering Fundamentals: A Common Basis for Digital Engineering Discussions. MITRE Technical Report. The MITRE Corporation, McLean, VA.
- McDermott, T., K. Henderson, A. Salado, and J. Bradley. 2022. Digital Engineering Measures: Research and Guidance. *INSIGHT*, 25: 12-18.
- Opoku, D.G.J., S. Perera, R. Osei-Kyei, and M. Rashidi. 2021. Digital twin application in the construction industry: A literature review. *Journal of Building Engineering*, 40, 102726.
- Sony, M., and S. Naik. 2020. Industry 4.0 integration with socio-technical systems theory: A systematic review and proposed theoretical model. *Technology in Society*, 61, 101248.
- Suprun, E., S. Mostafa, R.A. Stewart, H. Villamor, K. Sturm, and A. Mijares. 2022. Digitisation of Existing Water Facilities: A Framework for Realizing the Value of Scan-to-BIM. *Sustainability*, 14, 6142.
- Wu, J., X. Wang, Dang, Y., and Z. Lv. 2022. Digital twins and artificial intelligence in transportation infrastructure: Classification, application, and future research directions. *Computers and Electrical Engineering*, 101, 107983.
- Zimmerman, P., T. Gilbert, and F. Salvatore. 2019. Digital engineering transformation across the Department of Defense. *The Journal of Defense Modelling and Simulation*, 16(4):325-338.

Primary References

DoD. 2018. *Digital Engineering Strategy*. Arlington, VA: US Department of Defense. Available at: https://ac.cto.mil/wp-content/uploads/2019/06/2018-Digital-Engineering-Strategy_Approved_PrintVersion.pdf^[1]

Hutchison, N., and H.Y.S. Tao. 2022. "The Digital Engineering Competency Framework (DECF): Critical Skillsets to Support Digital Transformation." *INCOSE INSIGHT*, 25: 35-39.

Laskey, K.J., M.L. Farinacci, and O.C. Diaz. 2021. *Digital Engineering Fundamentals: A Common Basis for Digital Engineering Discussions*. MITRE Technical Report. The MITRE Corporation, McLean, VA.

McDermott, T., K. Henderson, A. Salado, and J. Bradley. 2022. "Digital Engineering Measures: Research and Guidance." *INCOSE INSIGHT*, 25: 12-18.

Zimmerman, P., T. Gilbert, and F. Salvatore. 2019. "Digital engineering transformation across the Department of Defense." *The Journal of Defense Modelling and Simulation*, 16(4):325-338.

Additional References

None.

References

[1] https://ac.cto.mil/wp-content/uploads/2019/06/2018-Digital-Engineering-Strategy_Approved_PrintVersion.pdf

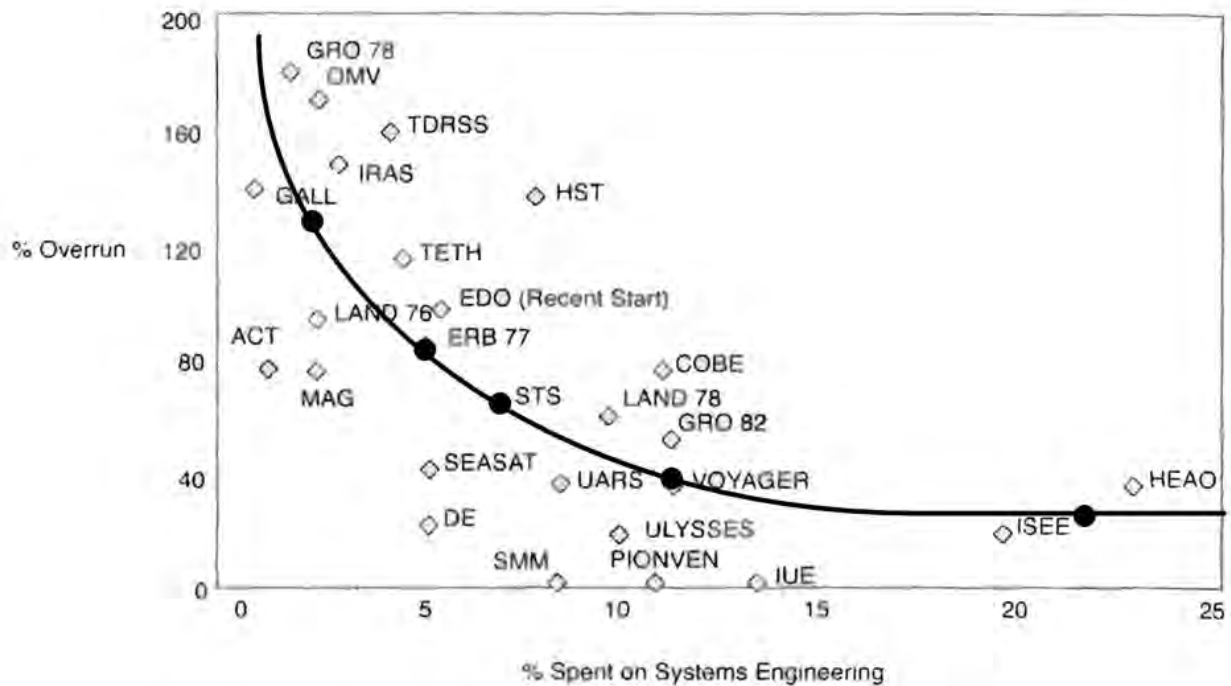
Economic Value of Systems Engineering

The Increasing Value of Systems Engineering

With traditional projects, such as railroads, reservoirs, and refrigerators, a systems engineer faced a self-contained system that typically had relatively stable requirements, a sound scientific base, and numerous previous precedents. As most modern systems become parts within one or more evolving systems of systems (SoS), the performance of effective SE now takes on an ever-higher economic value, as the systems feature a rapidly increasing scale, dynamism, interdependence, human-intensiveness, number of sources of vulnerability, and novelty.

This is corroborated by the implementation examples in Part 7. Shortfalls in SE lead to either cancellation of already expensive systems or even more expensive systems in terms of total cost of ownership or loss of human life. Part 7 presents the problems in the United States Federal Aviation Administration (FAA) Advanced Automation System (AAS), United States Federal Bureau of Investigation (FBI) Virtual Case File System, the Hubble Space Telescope Case Study, and the Therac-25 medical linear accelerator.

On the other hand, the Global Positioning System (GPS), Miniature Seeker Technology Integration Project (MSTI), and Next Generation Medical Infusion Pump Project all demonstrate that investment in thorough SE results in highly cost-effective systems. Figure 1 summarizes the analyses data by Werner Gruhl, which relates investment levels in SE to cost overruns of the United States National Aeronautics and Space Administration (NASA) projects (Stutzke 2005). The results indicate that there is a general correlation between the amount invested in SE within a program and cost overruns, demonstrating the critical role of properly allocating SE resources.

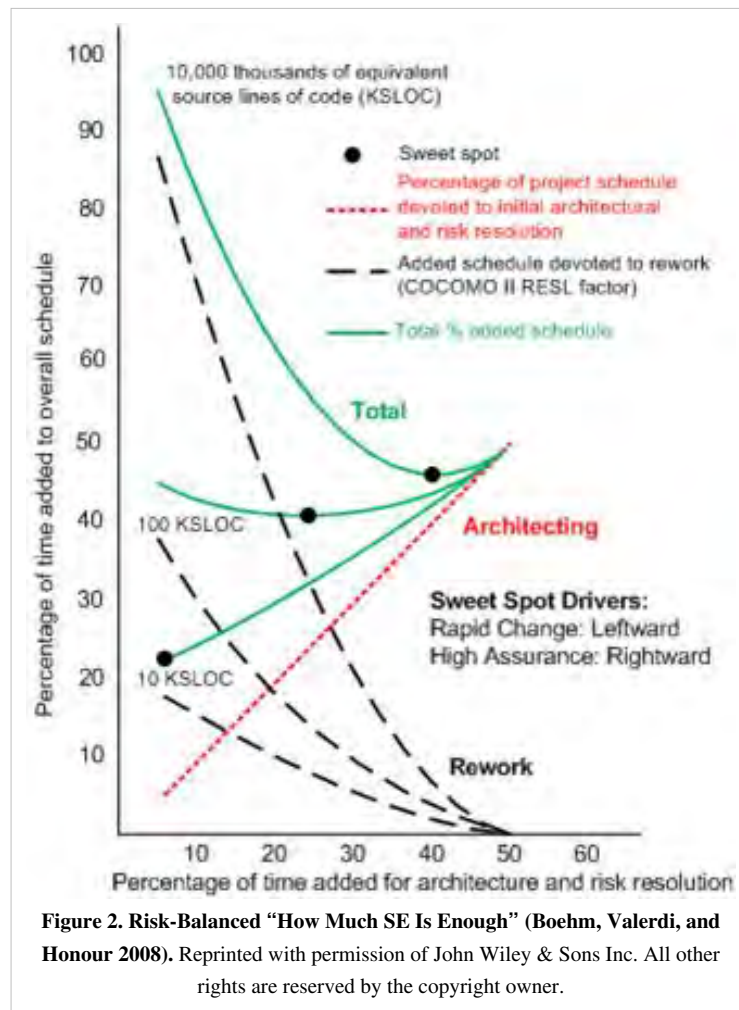


*Source: Werner M. Gruhl, Chief Cost & Economics Analysis Branch, NASA Headquarters

Figure 1. Relation of SE Investments to NASA Program Cost Overruns (Stutzke 2005). Released by NASA HDQRT/Gruhl.

Further Quantitative Evidence of the Value of Systems Engineering

Analysis of the effects of shortfalls in systems architecture and risk resolution (the results of insufficient SE) for software-intensive systems in the 161-project Constructive Cost Model II (COCOMO™ II) database shows a statistically significant increase in rework costs as a function of project size measured in source lines of code (SLOC): averages of 18% rework for ten-thousand-SLOC projects and 91% rework for ten-million-SLOC projects. This data has influenced many major system projects to reconsider initial underinvestment in SE (e.g., Boehm et al. 2004), as well as to address “how much SE is enough” by balancing the risks of under-investing in SE against those of over-investing (often called “analysis paralysis”), as shown in Figure 2 (Boehm, Valerdi, and Honour 2008).



Typically, small projects can quickly compensate for neglected SE interface definition and risk resolution; however, as projects grow larger and have more independently-developed components, the cost of late rework negates any savings in reduced SE effort. Additionally, medium-sized projects have relatively flat operating regions, while very large projects pay extremely large penalties for neglecting thorough SE. Extensive surveys and case study analyses corroborate these results.

Survey data on software cost and schedule overruns in *My Life Is Failure: 100 Things You Should Know to Be a Better Project Leader* (Johnson 2006) indicates that the primary sources of the roughly 50% of commercial projects with serious "software overruns" are the result of shortfalls in SE (lack of user input, incomplete requirements, unrealistic expectations, unclear objectives, and unrealistic schedules). The extensive survey of 46 government-contracted industry projects conducted by the Software Engineering Institute (SEI)/National Defense Industrial Association (NDIA) illustrated a strong correlation between higher project SE capability and higher project performance (Elm et al. 2007). Ongoing research that combined project data and survey data reported in "Toward an Understanding of The Value of SE" (Honour 2003) and "Effective Characterization Parameters for Measuring SE" (Honour 2010) has provided additional evidence as to the economic value of SE and further insights on critical factors that affect SE success.

The Constructive Systems Engineering Cost Model (COSYSMO), a calibrated model for determining "how much SE is enough," has been developed and is discussed in (Valerdi 2008). It estimates the number of person-months that a project needs for SE as a function of system size (i.e., requirements, interfaces, algorithms, and operational scenarios), modified by 14 factors (i.e., requirements understanding,

technology risk, personnel experience, etc.), which dictates the amount of SE effort needed. Other economic considerations of SE include the costs and benefits of reuse (Wang, Valerdi and Fortune 2010), the management of SE assets across product lines (Fortune and Valerdi 2013), the impact of SE on project risk (Madachy and Valerdi 2010), and the role of requirements volatility on SE effort (Pena and Valerdi 2010).

References

Works Cited

- Boehm, B., Brown, A.W., Basili, V., and Turner, R. 2004. "Spiral acquisition of software-intensive systems of systems," *CrossTalk*. May, pp. 4-9.
- Boehm, B., R. Valerdi, and E.C. Honour. 2008. "The ROI of systems engineering: Some quantitative results for software-intensive systems," *Systems Engineering*, vol. 11, no. 3, pp. 221-234.
- Elm, J. P., D.R. Goldenson, K. El Emam, N. Donatelli, and A. Neisa. 2008. *A Survey of Systems Engineering Effectiveness-Initial Results* (with Detailed Survey Response Data). Pittsburgh, PA, USA: Software Engineering Institute, CMU/SEI-2008-SR-034. December 2008.
- Fortune, J., and R. Valerdi. 2013. "A framework for systems engineering reuse," *Systems Engineering*, vol. 16, no. 2.
- Honour, E.C. 2003. "Toward an understanding of the value of systems engineering," Proceedings of the First Annual Conference on Systems Integration, Hoboken, NJ, USA, March 2003.
- Honour, E.C. 2010. "Effective characterization parameters for measuring systems engineering," Proceedings of the 8th Annual Conference on Systems Engineering Research (CSER), Hoboken, NJ, USA, March 17-19, 2010.
- Johnson, J. 2006. *My Life Is Failure: 100 Things You Should Know to Be a Better Project Leader*. Boston, MA, USA: Standish Group International.
- Madachy, R., and R. Valerdi. 2010. *Automating systems engineering risk assessment*, 8th Conference on Systems Engineering Research, Hoboken, NJ.
- Pena, M., and R. Valerdi. 2010. "Characterizing the impact of requirements volatility on systems engineering effort," 25th Forum on COCOMO and Systems/Software Cost Modeling, Los Angeles, CA.
- Stutzke, R. 2005. *Estimating Software-Intensive Systems*. Boston, MA, USA: Addison Wesley.
- Valerdi, R. 2008. *The Constructive Systems Engineering Cost Model (COSYSMO): Quantifying the Costs of Systems Engineering Effort in Complex Systems*. Saarbrücken, Germany: VDM Verlag.
- Wang, G., R. Valerdi, and J. Fortune. 2010. "Reuse in systems engineering," *IEEE Systems Journal*, vol. 4, no. 3, pp. 376-384.

Primary References

- Boehm, B., R. Valerdi, and E.C. Honour. 2008. "The ROI of systems engineering: Some quantitative results for software-intensive systems," *Systems Engineering*, vol. 11, no. 3, pp. 221-234.
- Honour, E.C. 2010. "Effective characterization parameters for measuring systems engineering," Proceedings of the 8th Annual Conference on Systems Engineering Research (CSER), Hoboken, NJ, USA, March 17-19, 2010
- Valerdi, R. 2008. *The Constructive Systems Engineering Cost Model (COSYSMO): Quantifying the Costs of Systems Engineering Effort in Complex Systems*. Saarbrücken, Germany: VDM Verlag.

Additional References

- Hughes, T.P. 2000. *Rescuing Prometheus: Four Monumental Projects that Changed the Modern World*. New York, NY, USA: Vintage Books.
- Vanek, F., R. Grzybowski, P. Jackson, and M. Whiting. 2010. "Effectiveness of systems engineering techniques on new product development: Results from interview research at corning Incorporated." Proceedings of the 20th Annual INCOSE International Symposium, Chicago, IL, USA, July 12-15, 2010.

A Brief History of Systems Engineering

- Lead Author:
- Robert Cloutier
- Contributing Author:
- Michael Pennotti

While many will attribute systems thinking to great accomplishments such as the Egyptian pyramids, Incan ziggurats, and the Roman aqueduct system, this article will pick up with the early mentions of systems engineering as a discipline. One only has to look to find a healthy body of works on the history of systems engineering. Many of those articles are cited in the References section below. The purpose of this article is to highlight the evolution of both the practice and definition of systems engineering as it emerged in the 20th century.

The Forties and Fifties (1940-1959)

During World War II, systems engineering began to emerge to deal with the myriad new technologies developed in support of the war effort. For example, an analysis of the RAF Fighter Command C2 System, which was crucial to the United Kingdom's prevailing in the 1940 Battle of Britain, performed by Derek Hitchens in 2005, concluded that its design represented "systems engineering of the highest caliber, even though that term had not yet been invented". (Hitchins 2005)

The first known use of the term "systems engineering" was in a March 1950 presentation to the Royal Society of London by Mervin J. Kelly, then Executive Vice President of Bell Telephone Laboratories. The transcript of Kelly's presentation entitled, "The Bell Telephone Laboratories - An Example of an Institute of Creative Technology", was published in the Proceedings of the Royal Society later that year. (Kelly 1950) In it, Kelly discussed the progress of Bell Labs over the first half of the twentieth century. He stated that the organization "grew in size and matured in the scope and character of its work during the period of rapid expansion in research in the physical sciences". From there, he went on to describe the organization of their work. The first organizational heading was Research and Fundamental Development. The second was 'Systems Engineering,' at the same level of importance as the research and development departments. Kelly described the responsibility of the systems engineering organization as "... the determination of new specific systems and facilities development projects – their operational and economic objectives and the broad technical plan to be followed. 'Systems engineering' controls and guides the use of the new

knowledge obtained from the research and fundamental development programs in the creation of new telephone services and the improvement and lowering of cost of services already established...it attempts to ensure that the technical objectives of the development projects undertaken can be realized within the framework of the new knowledge available in the reservoir and present engineering practice.”

The first published paper specifically about systems engineering appeared in 1956. In it, Kenneth Schlager of General Motors wrote, “Increased complexity in the fields of communications, instruments, computation, and control has led to an emphasis on the field of systems engineering.” He went on to say, “Though engineers with system functions can be found in almost all phases of the modern electronics and aircraft industries, there seems to be no commonly agreed upon definition of the term “systems engineering.” Nevertheless, he said, “the rise of systems engineering as a separate field has resulted in some organizational changes in the engineering departments of many companies... “typical instance is that of a systems engineering group which has established itself on an equal level with other electrical and mechanical design groups in the engineering department.” (Schlager 1956)

E.W. Engstrom grew up in the Radio Corporation of America (RCA) Laboratories. He became the president and CEO of RCA in 1961 and 1966 respectively. However, in 1957 he published another early paper on systems engineering in *Electrical Engineering*. Engstrom explained the concept of systems engineering in terms of its evolution and characteristics and described the engineering of a color television system and of a specific weapons system to illustrate its application. He stated [that “the task of adapting our increasingly complex devices and techniques to the requirements and limitations of the people who must use them has presented modern engineering with its greatest challenge. To meet this challenge, we have come to rely increasingly during recent years upon the comprehensive and logical concept known as systems engineering. He defined only two requirements for successful systems engineering. “First, a determination of the objective that is to be reached; and second, a thorough consideration of all the factors that bear upon the possibility of reaching the objective and the relationships among these factors.” (Engstrom 1957)

The first textbook on the subject of systems engineering was *Systems Engineering: An Introduction to the Design of Large-Scale Systems* written by Goode and Machol. It sold for \$10 when published in 1957. In it, the authors stated, “this book develops no general theory. It presents experience, the parts and the pieces, and the relationships among them; occasionally, in a small area, it attempts to tie several parts together with a general observation”. (Goode and Machol 1957)

The Sixties and Seventies (1960-1979)

Arthur Hall, who also worked for Bell Telephone Laboratories, taught one of the earliest Systems Engineering courses at MIT. In his book “A Methodology for Systems Engineering”, Hall identified five traits of the ideal systems engineer (1962):

1. an affinity for the systems
2. faculty of judgment
3. Creativity
4. facility in human relations, and
5. facility for expression

He also wrote: “Systems engineering is most effectively conceived of as a process that starts with the detection of a problem and continues through problem definition, planning and designing of a system, manufacturing or other implementing section, its use, and finally on to its obsolescence. Further, systems engineering is not a matter of tools alone; it is a careful coordination of process, tools, and people.”

Another interesting historical description of systems engineering appeared in a 1967 report to the Committee on Science and Astronautics of the U.S. House of Representatives. In the report, Hendrik Bode of Bell Laboratories wrote: “...the systems engineer resembles an architect, who must generally have adequate substantive knowledge of building materials, construction methods, and so on, to ply his trade. Like architecture, systems engineering is in

some ways an art as well as a branch of engineering. Thus, aesthetic criteria are appropriate for it also. For example, such essentially aesthetic ideas as balance, proportion, proper relation of means to ends, and economy of means are all relevant in a systems-engineering discussion. Many of these ideas develop best through experience.” (Panel on Applied Science and Technological Progress 1967)

Several common themes emerge in the early references to systems engineering as practiced during its first three decades:

- Systems engineers were deeply immersed in, and knowledgeable of, their application domains,
- Their analysis relied on domain-specific science, math, and engineering,
- The systems engineering effort transcended multiple technical and non-technical disciplines (e.g. including economics, psychology, and operations). It was, in fact, “transdisciplinary” from its inception, though that term has only been applied recently and
- As practiced, systems engineering was informal in nature, essentially the application of systems thinking to the engineering of systems.

Despite the lack of formal structure during these early decades, systems engineering played an essential role in the achievement of perhaps the greatest technical accomplishment of that era, the landing of a man on the moon and his safe return to the earth in July of 1969.

Coincidentally, the same month as the Apollo landing saw the introduction of the first formal systems engineering process. Military Standard (MIL-STD)-499: *System Engineering Management*, was published by the U.S. Air Force. (1969) Later extended to the entire DoD, the intent of this Mil-Std was to provide program managers and contractors with guidance for managing the systems engineering process. Later, in 1974, the DoD updated their guidance with MIL-STD-499A. It too covered the process but added guidelines for the Systems Engineering Management Plan (SEMP) and task statements that could be selectively applied to a DoD acquisition program.

The Eighties and Nineties (1980-1999)

The National Council on Systems Engineering (NCOSE) grew out of the need for formally trained systems engineers. Meetings between industry and academia began in 1989 and continued through 1991. Notable names included Jeffrey Grady (GD), Dr. David Sworder (UCSD), Dr. Brian Mar (U of Washington), Dr. Terry Bahill and Dr. Ron Askin (U of Arizona), and Gerald Chasko (DSMC Regional Director). The group grew to include industry and DoD representatives from the USAF, TRW, Lockheed, Martin Marietta, MacDonnell Douglas, Aerospace Corp, Bechtel, TI, Boeing, Unisys, IBM and many others. In 1989, Dr. Brian Mar took the lead to begin the International Council on Systems Engineering and is recognized as the Father of INCOSE. (Grady, 2013).

Hughes hosted the January 1992 business meeting in Los Angeles with NCOSE, now a formally incorporated organization (Honour, 1998). Later, this organization would change its name and become known as the International Council on Systems Engineering. The first edition "Systems Engineering", the journal for NCOSE was published in July/September 1994.

In 1995, the NASA Systems Engineering Handbook (NASA/SP-6105) was published to bring the fundamental concepts and techniques of systems engineering to the National Aeronautics and Space Administration (NASA) personnel in a way that recognized the nature of NASA systems and the NASA environment.

Finally, version 1 of INCOSE's *Systems Engineering Handbook* first appeared in 1997.

21st Century

In 2005, the International Organization for Standardization (ISO) published their first standard defining systems engineering application and management. The purpose of this ISO standard was to define the interdisciplinary tasks which are required throughout a system's life cycle to transform customer needs, requirements and constraints into a system solution. In addition, it defines the entire systems engineering lifecycle. A number of related standards followed, to include ISO/IEC TR 24748-1:2010, 15288, and 12207.

There are a many great articles documenting a more thorough history of systems engineering that are found in the References section of this brief article.

References

Works Cited

- Engstrom, E.W. 1957 "Systems engineering: A growing concept." in *Electrical Engineering*. 76(2): 113-116, Feb. 1957, doi: 10.1109/EE.1957.6442968.
- Goode, H. and Machol, R. 1957. *Systems Engineering: An Introduction to the Design of Large-Scale Systems*. New York, NY: McGraw-Hill Book Co, Inc. pp 551.
- Hall, A.D. 1962. *A Methodology for Systems Engineering*. Van Nostrand.
- Hitchins, D.K. 2005. "Systems Engineering of the Battle of Britain." *Systems World*. Published online November 2005.
- Kelly, M. J. 1950. The Bell Telephone Laboratories-An Example of an Institute of Creative Technology. Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences, 203(1074), 287–301. <https://www.jstor.org/stable/98407>
- National Research Council. 1967. *Applied Science and Technological Progress: A Report to the Committee on Science and Astronautics, U.S. House of Representatives*. Washington, DC: The National Academies Press. <https://doi.org/10.17226/21281>.
- Schlager, K.J. 1957. "Systems Engineering – Key to Modern Development." *IRE Transactions on Engineering Management*. July 1956; 64-66.

Primary References

- Ferris, T.L. 2007. "7.4.4 Some Early History of Systems Engineering – 1950's" in IRE Publications (Part 2): The Solution. INCOSE International Symposium, 17.
- Ferris, T.L. 2007. "7.4.3 Some Early History of Systems Engineering – 1950's" in IRE Publications (Part 1): The Problem. INCOSE International Symposium, 17.
- Ferris, T.L. 2008. "1.2.1 Early History of Systems Engineering (Part 3) – 1950's in Various Engineering Sources." INCOSE International Symposium, 18: 46-57. <https://doi.org/10.1002/j.2334-5837.2008.tb00790.x>
- Honour, E. 2018. "A historical perspective on systems engineering." February 2018. *Systems Engineering*. 21(3). DOI: 10.1002/sys.21432
- Hossain, N.U.I., R.M. Jaradat, M.A. Hamilton, C.B. Keating, S.R. Goerger. 2019. "A Historical Perspective on Development of Systems Engineering Discipline: A Review and Analysis." *Journal of Systems Science and Systems Engineering*. 10.1007/s11518-019-5440-x, 29(1): (1-35).
- Hossain, N.U.I., R.M. Jaradat, M.A. Hamilton, C.B. Keating, S.R. Goerger. 2021. *A Historical Perspective on Development of Systems Engineering Discipline*. US Army Corps of Engineers, Engineering Research and Development Center. ERDC/ITL MP-21-3. Downloadable at <https://apps.dtic.mil/sti/pdfs/AD1127676.pdf>

Additional References

Grady, Jeffrey (2013) A History of INCOSE, Presented by JOG System Engineering for the San Diego Chapter Mini Conference. Downloaded 4/18/2022 from <https://sdincose.org/wp-content/uploads/2013/11/2-Jeff-Grady-105A.pdf>

INCOSE website (2022). Downloaded from <https://www.incose.org/about-incose>

NCOSE (1994). Inaugural Issue, Systems Engineering, The Journal of the National Council of Systems Engineering. Vol. 1, Number 1.

Buede, D. (2000) History of Systems Engineering. <https://www.incose.org/about-systems-engineering/history-of-systems-engineering>. Last accessed 8/8/2020

Hallam, C. (2001). An Overview of Systems Engineering - The Art of Managing Complexity. Submitted on October 16th, 2001, for ESD.83. <https://web.mit.edu/esd.83/www/notebook/syseng.doc>. Last accessed 8/8/2020

MITRE, The Evolution of Systems Engineering, downloaded 4/18/2022 from <https://www.mitre.org/publications/systems-engineering-guide/systems-engineering-guide/the-evolution-of-systems>

Page, A. (2015). The Evolution of Systems Engineering in the US Department of Defense. Located at <https://sdm.mit.edu/the-evolution-of-systems-engineering-in-the-us-department-of-defense/>. Last accessed 8/8/2020

The MITRE Corporation, August 2007, Evolving Systems Engineering, Bedford, MA. Available at https://www.mitre.org/sites/default/files/pdf/mitre_ese.pdf

Systems Engineering: Historic and Future Challenges

Humans have faced increasingly complex challenges and have had to think systematically and holistically in order to produce successful responses to these challenges. From these responses, generalists have developed generic principles and practices for replicating success. Some of these principles and practices have contributed to the evolution of systems engineering as a discipline.

Historical Perspective

Some of the earliest relevant challenges were in organizing cities. Emerging cities relied on functions such as storing grain and emergency supplies, defending the stores and the city, supporting transportation and trade, providing a water supply, and accommodating palaces, citadels, afterlife preparations, and temples. The considerable holistic planning and organizational skills required to realize these functions were independently developed in the Middle East, Egypt, Asia, and Latin America, as described in Lewis Mumford's *The City in History* (Mumford 1961).

Megacities, and mobile cities for military operations, such as those present in the Roman Empire, emerged next, bringing another wave of challenges and responses. These also spawned generalists and their ideological works, such as Vitruvius and his *Ten Books on Architecture* (Vitruvius: Morgan transl. 1960). "Architecture" in Rome meant not just buildings, but also aqueducts, central heating, surveying, landscaping, and overall planning of cities.

The Industrial Revolution brought another wave of challenges and responses. In the nineteenth century, new holistic thinking and planning went into creating and sustaining transportation systems, including canal, railroad, and metropolitan transit. General treatises, such as *The Economic Theory of the Location of Railroads* (Wellington 1887), appeared in this period. The early twentieth century saw large-scale industrial enterprise engineering, such as the Ford automotive assembly plants, along with treatises like *The Principles of Scientific Management* (Taylor 1911).

The Second World War presented challenges around the complexities of real-time command and control of extremely large multinational land, sea, and air forces and their associated logistics and intelligence functions. The postwar period brought the Cold War and Russian space achievements. The U.S. and its allies responded to these challenges by investing heavily in researching and developing principles, methods, processes, and tools for military defense systems, complemented by initiatives addressing industrial and other governmental systems. Landmark results included the codification of operations research and SE in *Introduction to Operations Research* (Churchman et. al 1957), Warfield (1956), and Goode-Machol (1957) and the Rand Corporation approach as seen in *Efficiency in Government Through Systems Analysis* (McKean 1958). In theories of system behavior and SE, we see cybernetics (Weiner 1948), system dynamics (Forrester 1961), general systems theory (Bertalanffy 1968), and mathematical systems engineering theory (Wymore 1977).

Two further sources of challenge began to emerge in the 1960s and accelerated in the 1970s through the 1990s: awareness of the criticality of the human element, and the growth of software functionality in engineered systems.

Concerning awareness of the human element, the response was a reorientation from traditional SE toward “soft” SE approaches. Traditional hardware-oriented SE featured sequential processes, pre-specified requirements, functional-hierarchy architectures, mathematics-based solutions, and single-step system development. A Soft Systems approach to SE is characterized by emergent requirements, concurrent definition of requirements and solutions, combinations of layered service-oriented and functional-hierarchy architectures, heuristics-based solutions, and evolutionary system development. Good examples are societal systems (Warfield 1976), soft systems methodology (Checkland 1981), and systems architecting (Rechtin 1991 and Rechtin-Maier 1997). As with Vitruvius, “architecting” in this sense is not confined to producing blueprints from requirements, but instead extends to concurrent work on operational concepts, requirements, structure, and life cycle planning.

The rise of software as a critical element of systems led to the definition of Software Engineering as a closely related discipline to SE. The Systems Engineering and Software Engineering knowledge area in Part 6: Related Disciplines describes how software engineering applies the principles of SE to the life cycle of computational systems (in which any hardware elements form the platform for software functionality) and of the embedded software elements within physical systems.

Evolution of Systems Engineering Challenges

Since 1990, the rapidly increasing scale, dynamism, and vulnerabilities in the systems being engineered have presented ever-greater challenges. The rapid evolution of communication, computer processing, human interface, mobile power storage and other technologies offers efficient interoperability of net-centric products and services, but brings new sources of system vulnerability and obsolescence as new solutions (clouds, social networks, search engines, geo-location services, recommendation services, and electrical grid and industrial control systems) proliferate and compete with each other.

Similarly, assessing and integrating new technologies with increasing rates of change presents further SE challenges. This is happening in such areas as biotechnology, nanotechnology, and combinations of physical and biological entities, mobile networking, social network technology, cooperative autonomous agent technology, massively parallel data processing, cloud computing, and data mining technology. Ambitious projects to create smart services, smart hospitals, energy grids, and cities are under way. These promise to improve system capabilities and quality of life but carry risks of reliance on immature technologies or on combinations of technologies with incompatible objectives or assumptions. SE is increasingly needed but increasingly challenged in the quest to make future systems scalable, stable, adaptable, and humane.

It is generally recognized that there is no one-size-fits-all life cycle model that works best for these complex system challenges. Many systems engineering practices have evolved in response to this challenge, making use of lean, agile, iterative and evolutionary approaches to provide methods for simultaneously achieving high-effectiveness, high-assurance, resilient, adaptive, and life cycle affordable systems;. The emergence of system of systems (SoS)

approaches have also been introduced, in which independent system elements developed and deployed within their own life cycle are brought together to address mission and enterprise needs.

Creating flexible and tailored life cycles and developing solutions using combinations of engineered systems, each with its own life cycle focus, creates its own challenges of life cycle management and control. In response to this, enterprise systems engineering (ESE) approaches have been developed, which consider the enterprise itself as a system to be engineered. Thus, many of the ambitious smart system projects discussed above are being delivered as a program of managed life cycles synchronized against a top down understanding of enterprise needs. It is important that within these approaches we create the flexibility to allow for bottom-up solutions developed by combining open, interoperable system elements to emerge and be integrated into the evolving solutions.

More recently, emerging technologies such as artificial intelligence, machine learning, deep learning, mechatronics, cyberphysical systems, cybersecurity, Internet of Things (IoT), additive manufacturing, digital thread, Factory 4.0, etc. are challenging approaches to SE.

Many of the challenges above, and the SE response to them, increase the breadth and complexity of the systems information being considered. This increases the need for up to date, authoritative and shared models to support life cycle decisions. This has led to the development and ongoing evolution of model-based systems engineering (MBSE) approaches.

Future Challenges

The INCOSE Systems Engineering Vision 2025 (INCOSE 2014) considers the issues discussed above and from this gives an overview of the likely nature of the systems of the future. This forms the context in which SE will be practiced and give a starting point for considering how SE will need to evolve:

- Future systems will need to respond to an ever growing and diverse spectrum of societal needs in order to create value. Individual engineered system life cycles may still need to respond to an identified stakeholder need and customer time and cost constraint. However, they will also form part of a larger synchronized response to strategic enterprise goals and/or societal challenges. System life cycles will need to be aligned with global trends in industry, economy and society, which will, in turn, influence system needs and expectations.
- Future systems will need to harness the ever-growing body of technology innovations while protecting against unintended consequences. Engineered system products and services need to become smarter, self-organized, sustainable, resource efficient, robust and safe in order to meet stakeholder demands.
- These future systems will need to be engineered by an evolving, diverse workforce which, with increasingly capable tools, can innovate and respond to competitive pressures.

These future challenges change the role of software and people in engineered systems. The Systems Engineering and Software Engineering knowledge area considers the increasing role of software in engineered systems and its impact on SE. In particular, it considers the increasing importance of cyber-physical systems in which technology, software and people play an equally important part in the engineered systems solutions. This requires a SE approach able to understand the impact of different types of technology, and especially the constraints and opportunities of software and human elements, in all aspects of the life cycle of an engineered system.

All of these challenges, and the SE responses to them, make it even more important that SE continues its transition to a model-based discipline.

The changes needed to meet these challenges will impact the life cycle processes described in Part 3: Systems Engineering and Management and on the knowledge, skills and attitudes of systems engineers and the ways they are organized to work with other disciplines as discussed in Part 5: Enabling Systems Engineering and Part 6: Related Disciplines. The different ways in which SE is applied to different types of system context, as described in Part 4: Applications of SE, will be a particular focus for further evolution to meet these challenges. The Introduction to SE Transformation knowledge area in SEBoK Part 1 describes how SE is beginning to change to meet these challenges.

References

Works Cited

- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. New York, NY, USA: George Braziller.
- Checkland, P. 1981. *Systems Thinking, Systems Practice*. Hoboken, NJ, USA: Wiley, 1981.
- Churchman, C.W., R. Ackoff, and E. Arnoff. 1957. *Introduction to Operations Research*. New York, NY, USA: Wiley and Sons.
- International Council on Systems Engineering (INCOSE), 2014, *Systems Engineering Vision 2025 July*, 2014; Available at: <https://www.incose.org/docs/default-source/aboutse/se-vision-2025.pdf?sfvrsn=4>. Accessed February 16.
- Ferguson, J. 2001. "Crouching dragon, hidden software: Software in DoD weapon systems," *IEEE Software*, July/August, p. 105–107.
- Forrester, J. 1961. *Industrial Dynamics*. Winnipeg, Manitoba, Canada: Pegasus Communications.
- Goode, H. and R. Machol. 1957. *Systems Engineering: An Introduction to the Design of Large-Scale Systems*. New York, NY, USA: McGraw-Hill.
- McKean, R. 1958. *Efficiency in Government Through Systems Analysis*. New York, NY, USA: John Wiley and Sons.
- Mumford, L. 1961. *The City in History*. San Diego, CA, USA: Harcourt Brace Jovanovich.
- Rechtin, E. 1991. *Systems Architecting*. Upper Saddle River, NJ, USA: Prentice Hall.
- Rechtin, E. and M. Maier. 1997. *The Art of Systems Architecting*. Boca Raton, FL, USA: CRC Press.
- Taylor, F. 1911. *The Principles of Scientific Management*. New York, NY, USA and London, UK: Harper & Brothers.
- Vitruvius, P. (transl. Morgan, M.) 1960. *The Ten Books on Architecture*. North Chelmsford, MA, USA: Courier Dover Publications.
- Warfield, J. 1956. *Systems Engineering*. Washington, D.C., USA: US Department of Commerce (DoC).
- Wellington, A. 1887. *The Economic Theory of the Location of Railroads*. New York, NY, USA: John Wiley and Sons.
- Wiener, N. 1948. *Cybernetics or Control and Communication in the Animal and the Machine*. New York, NY, USA: John Wiley & Sons Inc.
- Wymore, A. W. 1977. *A Mathematical Theory of Systems Engineering: The Elements*. Huntington, NY, USA: Robert E. Krieger.

Primary References

- Boehm, B. 2006. "Some future trends and implications for systems and software engineering processes," *Systems Engineering*, Wiley Periodicals, Inc., vol. 9, no. 1, pp. 1-19.
- INCOSE Technical Operations. 2007. *Systems Engineering Vision 2020*, version 2.03. Seattle, WA: International Council on Systems Engineering, Seattle, WA, INCOSE-TP-2004-004-02.
- International Council on Systems Engineering (INCOSE). 2014. *Systems Engineering Vision 2025*, July 2014. Available at: <https://www.incose.org/docs/default-source/aboutse/se-vision-2025.pdf?sfvrsn=4> ^[1]. Accessed February 16.
- Warfield, J. 1956. *Systems Engineering*. Washington, D.C., USA: US Department of Commerce (DoC). Report PB111801.
- Warfield, J. 1976. *Societal Systems: Planning, Policy, and Complexity*. New York, NY, USA: John Wiley & Sons.

Wymore, A. W. 1977. *A Mathematical Theory of Systems Engineering: The Elements*. Huntington, NY, USA: Robert E. Krieger.

Additional References

Hitchins, D. 2007. *Systems Engineering: A 21st Century Methodology*. Chichester, England: Wiley.

The MITRE Corporation. 2011. "The evolution of systems engineering," in *The MITRE Systems Engineering Guide*. Available at: [2]. Accessed 8 March 2012.

Sage, A. and W. Rouse (eds). 1999. *Handbook of Systems Engineering and Management*. Hoboken, NJ, USA: John Wiley and Sons, Inc.

References

[1] <https://www.incose.org/wp-content/uploads/legacy/se-vision-2025/se-vision-2025/incose-se-vision-2025.pdf>

[2] https://www.mitre.org/work/systems_engineering/guide/evolution_systems.html

Systems Engineering and Other Disciplines

As discussed in the Scope of the SEBoK article, there are many touch points and overlaps between systems engineering (SE) and other disciplines. Systems engineers should have a basic understanding of the nature of these other disciplines, and often need to understand aspects of another discipline in detail. This article describes the landscape of disciplines that are intertwined with SE. For a closer view of the individual disciplines, see Part 6.

Engineering Disciplines Other than Systems Engineering

Engineering disciplines are mostly component-oriented and value-neutral in their intellectual content (Boehm and Jain 2006). Their underlying laws and equations, such as Ohm's Law, Hooke's Law, Newton's Laws, Maxwell's equations, the Navier-Stokes equations, Knuth's compendia of sorting and searching algorithms, and Fitts's Law of human movement, pertain to performance in a system-of-interest. They do not address how that performance contributes to the value propositions of stakeholders.

In contrast, SE is more holistic than component-oriented, and more stakeholder value-oriented than value-neutral, performance-oriented in its intellectual content. Realizing successful systems requires reasoning with stakeholders about the relative value of alternative realizations, and about the organization of components and people into a system that satisfies the often-conflicting value propositions of stakeholders. Stakeholders who are critical to the system's success include funders, owners, users, operators, maintainers, manufacturers, and safety and pollution regulators.

In some disciplines, the engineer evaluates and integrates design elements into a system that satisfies proxies of value. The wider the scope of the SoI, the broader the set of SE skills the engineer needs.

For example, an aeronautical engineer might integrate mechanical, electrical, fluid, combustion-chemical, software, and cockpit design elements into a system that satisfies proxies of value like flight range, payload capacity, fuel consumption, maneuverability, and cost of production and maintenance. In so doing, the engineer operates partly as a systems engineer. The SoI is the aircraft itself and the engineer applies aircraft-domain expertise.

However, the same engineer could participate in the engineering of passenger services, airport configurations, baggage handling, and local surface transportation options. All of these contribute to the value propositions of success-critical stakeholders. The SoIs are wider, and the engineer needs broader SE knowledge, skills, and abilities to operate as a systems engineer. The aircraft-domain expertise remains needed for effective engineering of the wider systems. As discussed in (Guest 1991), most good systems engineers are "T-shaped" people, with both a working

knowledge of wider-system considerations, and a deep expertise in a relevant domain, such as aeronautical, manufacturing, software, or human factors engineering.

Engineering disciplines that are intertwined with SE include software engineering (SwE), human factors engineering, and industrial engineering. SwE and SE are not just allied disciplines, they are intimately intertwined (Boehm 1994). Most functionality of commercial and government systems is now implemented in software, and software plays a prominent or dominant role in differentiating competing systems in the marketplace. Software is usually prominent in modern systems architectures and is often the “glue” for integrating complex system components.

The scope of SwE includes both software SE and software construction, but does not include hardware SE. Thus, neither SwE nor SE is a subset of the other. See Figure 1 in Scope of the SEBoK. For a definition of the relationship between the SEBoK and the *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, which is published by the Institute of Electrical and Electronics Engineers (IEEE) (Bourque and Fairley 2014), see Systems Engineering and Software Engineering.

Human factors engineering, from micro-ergonomics to macro-ergonomics, is intertwined with SE (Booher 2003; Pew and Mavor 2007). See Human Systems Integration in Part 6.

Industrial engineering overlaps significantly with SE in the industrial domain, but also includes manufacturing and other implementation activities outside of SE. See Systems Engineering and Industrial Engineering in Part 6.

Finally, to field a successful system, a systems engineer may need to know one or more of the many specialty fields in engineering, e.g., security, safety, reliability, availability, and maintainability engineering. Most of these are considered professional disciplines in their own right and many have their own bodies of knowledge. For explanations of how these disciplines relate to SE, overviews of what most systems engineers need to know about them, and references within their bodies of knowledge, see Systems Engineering and Quality Attributes in Part 6.

Non-Engineering Disciplines

SE is intimately intertwined with two non-technical disciplines: technical management (TM), and procurement and acquisition (also known as acquisition and procurement). TM often falls within the purview of a systems engineer. Many SE textbooks, competency models, and university programs include material about TM. TM is a specialization of project management (PM). SE and PM have significant common content in TM, but neither is a subset of the other. See Figure 1 in the article Scope of the SEBoK. For a definition of the relationship between the SEBoK and the *Guide to the Project Management Body of Knowledge (PMBOK)*, which is published by the Project Management Institute (PMI) (PMI 2013), see Systems Engineering and Project Management in Part 6.

Procurement and acquisition practitioners draw upon SE to determine the scope and overall requirements of the system to be procured or acquired. They then prepare requests for proposals and statements of work, determine evaluation criteria, and design source selection processes. Once a leading source is selected, they decide upon contracting options that encompass payments, reviews, audits, incentive fees, acceptance criteria, procedures, and the nature of deliverables. Finally, they monitor progress with respect to plans (including those for SE), and negotiate and execute changes and corrective actions. Many of these activities amount to specialty disciplines within procurement and acquisition. See the article Related Disciplines in Part 6.

References

Works Cited

- Boehm, B. W. "Integrating Software Engineering and Systems Engineering." *The Journal of NCOSE* Vol. 1 (No. 1): pp. 147-151. 1994
- Boehm, B. and A. Jain. 2006. "A value-based theory of systems engineering," *Proceedings, INCOSE IS 2006*. Also available at: <https://sunset.usc.edu/csse/TECHRPTS/2006/usccse2006-619/usccse2006-619.pdf>.
- Booher, H. 2003. *Handbook of Human-Systems Integration*. New York, NY, USA: John Wiley & Sons Inc.
- Bourque, P. and R.E. Fairley. Eds. 2014. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.swebok.org>.
- Guest, D. 1991. "The hunt is on for the Renaissance Man of computing." *The Independent*. London, England: September 17, 1991.
- INCOSE. 2011. *Systems Engineering Handbook*, version 3.2.1. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.
- Pew, R. and A. Mavor. 2007. *Human-System Integration in the System Development Process*. Washington, D.C., USA: The National Academies Press.
- PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Primary References

- Booher, H. 2003. *Handbook of Human-Systems Integration*. New York, NY, USA: John Wiley & Sons Inc.
- Bourque, P. and R.E. Fairley Eds. 2014. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.swebok.org>.
- Gallagher, B., M. Phillips, K. Richter, and S. Shrum. 2011. *CMMI For Acquisition: Guidelines for Improving the Acquisition of Products and Services*, second ed. Upper Saddle River, NJ, USA: Addison Wesley.
- Paulk, M., C. Weber, B. Curtis, and M. Chrissis. 1995. *The Capability Maturity Model: Guidelines for Improving the Software Process*. Upper Saddle River, NJ, USA: Addison Wesley.
- Pyster, A. Ed. 2009. *Graduate Software Engineering 2009 (GSWE2009): Curriculum Guidelines for Graduate Degree Programs in Software Engineering*. Integrated Software & Systems Engineering Curriculum Project. Hoboken, NJ, USA: Stevens Institute of Technology, September 30, 2009.
- Pew, R. and A. Mavor. 2007. *Human-System Integration in the System Development Process*. Washington, D.C., USA: The National Academies Press.
- PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Additional References

None.

Fundamentals for Future Systems Engineering

- Lead Author:
- Rick Adcock
- Contributing Author:
- Duane Hybertson

This article forms part of the Systems Fundamentals knowledge area (KA). It considers future trends in SE and how these might influence the evolution of future fundamentals.

The SEBoK contains a guide to generalized knowledge about the practice of SE. It does not pass judgement on that knowledge. However, it can be useful in some cases to indicate which parts of the knowledge are rooted in existing practice and which point towards the future evolution of SE.

This article provides a sketch of how SE is changing and suggests how these changes may affect the future of systems engineering, the SEBoK, and the foundations in Part 2.

INCOSE Vision

The INCOSE Vision 2025 statement (INCOSE 2014) depicts some future directions in:

Broadening SE Application Domains

- SE relevance and influence will go beyond traditional aerospace and defense systems and extend into the broader realm of engineered, natural and social systems
- SE will be applied more widely to assessments of socio-physical systems in support of policy decisions and other forms of remediation

More Intelligent and Autonomous Systems

- Systems of the future need to become smarter, self-organized, sustainable, resource-efficient, robust and safer
- The number of autonomous vehicles and transportation systems needs to increase
- Systems become more “intelligent” and dominate human-safety critical applications

Theoretical Foundations

- SE will be supported by a more encompassing foundation of theory and sophisticated model-based methods and tools allowing a better understanding of increasingly complex systems and decisions in the face of uncertainty
- Challenge: A core body of systems engineering foundations is defined and taught consistently across academia and forms the basis for systems engineering practice

In this article we will consider how the fundamentals of SE might need to evolve to support this vision.

How will SE Change?

In Engineered Systems, we describe three general contexts in which a SE life cycle can be applied. In a product system context, the outputs of SE focus on the delivery of technological systems. While such systems are designed to be used by people and fit into a wider problem-solving context, this context has been seen as largely fixed and external to SE. The service system context allows SE to consider all aspects of the solution system as part of its responsibility. This is currently seen as a special case of SE application largely focused on software intensive solutions. The enterprise system context offers the potential for a direct application of SE to tackle complex socio-technical problems, by supporting the planning, development and use of combinations of service systems. While this is done, it can be difficult to connect to the product focused life cycles of many SE projects.

The role of the systems engineer has already begun to change somewhat due to the first two of the future trends above. Changes to the scope of SE application and the increased use of software intensive reconfigurable and

autonomous solutions will make the service system context the primary focus of most SE life cycles. To enable this, most product systems will need to become more general and configurable, allowing them to be used in a range of service systems as needed. These life cycles are increasingly initiated and managed as part of an enterprise portfolio of related life cycles.

In this evolution of SE, the systems engineer cannot consider as many aspects of the context to be fixed, making the problem and possible solution options more complex and harder to anticipate. This also means the systems engineer has greater freedom to consider solutions which combine existing and new technologies and in which the role of people and autonomous software can be changed to help deliver desired outcomes. For such systems to be successful, they will need to include the ability to change, adapt and grow both in operation and over several iterations of their life cycle. This change moves SE to be directly involved in enterprise strategy and planning, as part of an ongoing and iterative approach to tackling the kinds of societal problems identified in the INCOSE vision.

This evolution of both the role and scope of SE will also see the system of systems aspects of all system contexts increase. We can expect System of Systems Engineering to become part of the systems engineering of many, if not most, SE life cycles.

Evolution of Fundamentals

These ongoing changes to SE place more emphasis on the role of autonomous agents in systems engineering, and agency will be an area of increased emphasis in the systems engineering and SEBoK of the future. Hybertson (2019) spells out in more detail the increased role of agents and agency in future SE. Moving from a total control model to a shared responsibility model changes the nature of engineering to something more like collective actualization, as proposed by Hybertson (2009 and 2019). Systems will represent a combination and interplay of technology and social factors, and they can range from technical product to service provider to social entity. In many cases they will be a socio-technical combination or hybrid.

The above trends have an impact on SE foundations, including technical aspects, social aspects, and ethical aspects. Inclusion of people in systems implies significant expansion of foundation sciences, to provide principles, theories, models, and patterns of the human, biological, social, and agent realm as well as the technical and physical realm. Emphasis on agents implies a revised conceptualization of system change, from the traditional model of mechanistic and controlled fixes and upgrades to a more organic change model that involves growth, self-learning, self-organizing, and self-adapting. Ethical considerations will include how to allocate responsibility for a system in a shared responsibility model. Further discussion of the expanded foundation and a list of foundation disciplines for future SE are presented in (Hybertson 2009 and 2019).

References

Works Cited

- Hybertson, D. (2009). *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*, Boca Raton, FL, USA: Auerbach/CRC Press.
- Hybertson, D. (2020 forthcoming). *Systems Engineering Science*. Chapter in G. S. Metcalf, H. Deguchi, and K. Kijima (editors in chief). *Handbook of Systems Science*. Tokyo: Springer.
- INCOSE (2014). "A world in motion: Systems engineering vision 2025." International Council on Systems Engineering.
-

Primary References

None.

Additional References

None.

SEBoK Users and Uses

SEBoK Users and Uses

Contents of this Knowledge Area

- Guidance for Systems Engineering Novices
- Guidance for Systems Engineers
- Guidance for Engineers
- Guidance for Systems Engineering Customers
- Guidance for Educators and Researchers
- Guidance for General Managers

The SEBoK is intended to be a resource that can help anyone understand more about systems engineering. Systems engineers should obviously benefit, but so can undergraduate engineering students, those new to the SE discipline, or people who work with systems engineers and want to understand more about what to expect from systems engineering activities. This article explores a variety of personas. Once you determine which persona best represents you, read on to get practical advice on how to utilize the SEBoK.

Personas

In marketing, "personas" are used to illustrate potential users or customers. Personas are fictional characters that represent a demographic of a target audience. For the SEBoK, we've developed personas that represent different types of people who we anticipate will utilize the SEBoK:

- An engineering undergraduate student
- A new systems engineer
- An experienced systems engineer
- A chief systems engineer
- An organizational manager
- A systems engineering educator or researcher
- A general manager

Note that initial work on personas was published in (Hutchison et al. 2023). The personas published in this reference were built upon and edited to support this article.

An Undergraduate Engineering Student

Stueti is in her final year studying electrical engineering at a private university. In her capstone, she works on a multi-disciplinary team, including students from the mechanical and software engineering departments as well. As the team comes together, each student starts talking about using the most advanced techniques and technologies available in their part of the project. The discussion devolves into an argument about how to do this in the time and budget available. Her faculty advisor says that in order to be successful, they have to work together to create a useful product, which means that they can't each focus on their own parts in isolation. The advisor suggests that they all need to come to the next team meeting prepared to discuss their project as a system. Stueti Googles "systems" when she gets back to her dorm and stumbles on the SEBoK.

For Stueti, the SEBoK begins to expose her to systems concepts. In particular, Stueti could benefit from reading the articles in Part 2: Foundations of Systems Engineering, particularly The Nature of Systems, which will introduce her

to different views on and core definitions of a system. From there, the articles in the Introduction to Systems Engineering knowledge area would be useful. Finally, Stueti reads the article Systems Engineering and Mechanical Engineering, which relates to this new subject area that she has spent several years learning. In her reading, Stueti finds a few references that she thinks will help her better understand this SE stuff and help her apply it.

This gives Stueti a better understanding of what her professor meant – that all the different elements of the project (or system) must work together to achieve their project goal. As the capstone progresses, Stueti returns to the SEBoK to find additional resources as new issues come up.

If you identify with Stueti, review the recommendation for SE novices.

A New Systems Engineer

Hans completed a master's in computer science three years ago and has since been working for a small company in their IT department. The team he is on has been operating and maintaining the existing customer support system, which has become increasingly cumbersome and costly. Hans's manager noticed that he was always asking good questions, and several times Hans's curiosity helped the team uncover the root causes of problems. Hans has also exhibited "out of the box" thinking, which has helped create some novel solutions. Hans's computer science and programming skills are competent. The company creating a new customer support system and, based on his experience with the current system and his inquisitiveness, Hans's manager asks if he would consider taking a role as the software systems engineer. In that role, he would be reporting directly to the chief systems engineer on the project. Hans looks forward to new responsibilities, but other than a few brief interactions with systems engineers, he's not sure what they do. He is enrolled in the company's introductory systems engineering course in a few weeks, but he wants to start understanding more about it now.

Hans's manager recommends he take a look at the SEBoK in preparation for his upcoming course. Hans, feeling a bit nervous, decides to search the SEBoK for "computer science" and finds the page, Exploring the Relationship between Systems Engineering and Software Engineering. This is a page about a paper by the same title, which Hans looks up to read later. In the SEBoK, he notices that this is used in something called, Software Engineering in the Systems Engineering Life Cycle. Hans reads this article and finds a number of related articles on the relationship between systems engineering and software engineering, which highlight the relationships between the two. Hans is now feeling more confident that this "systems stuff" isn't completely different from what he has learned by applying his computer science degree on the job.

Hans continues to explore the SEBoK, finding a collection of articles about systems thinking, which he finds useful and which leads him to articles about the systems engineering process. He learns about lifecycle approaches and finds an article on agile systems engineering. He is very familiar with agile approaches from his work over the last three years. While reading the SEBoK, Hans looks at the references and identifies 6 articles and 2 books he wants to read to get more familiar with SE. He is feeling much more confident about his ability to tackle the upcoming course and his new role and bookmarks the SEBoK for future reference.

If you identify with Hans, review the recommendations for experienced engineers new to systems engineering.

An Experienced Systems Engineer

Yan has been working in a large company that makes medical devices for the last 10 years, ever since she earned her undergraduate degree in electrical engineering. Within a few years of joining the company, Yan was selected for a "high potential" program to develop new systems talent in the company. As part of the program, Yan completed her graduate degree in systems engineering last year. Currently, she is a lead designer on one of the company's flagship products. She has taken all the courses her company offers around systems design and engineering and is looking for more resources to continue to develop her skills.

Yan is aware of the SEBoK and has read a few articles but has not used it often. She decides to search the SEBoK for "knowledge skills abilities" (KSA) as her company uses this term when evaluating performance. She finds

articles on enabling individuals, ethical behavior, and roles and competencies. She reads the roles and competencies articles and learns about several different competency models. Primary references in these articles point her to these models, and she decides that the NASA's Systems Engineering Competencies model seems to align well with how her company views systems engineering. She decides to look through the competency model and see what competencies she has and which she might want to work on. She is generally comfortable with the competencies around the systems engineering lifecycle. She has had a few minor leadership roles, but some of the technical management competencies are less familiar. Yan decides that learning more about technical management will improve her skillset and make her more valuable to the company.

She skims the articles on Technical Management Processes in the SEBoK, and the overview content is pretty familiar. Looking through the works cited and primary references, however, she finds a few really useful references. This leads to the definition of technical management in the SEBoK Glossary of Terms, which points to a joint INCOSE/Project Management Institute (PMI) working group on the subject. Yan reaches out to this group and identifies further resources for self-study. After reviewing these resources, Yan is now more familiar with the vocabulary of technical management and looks for related short courses to help her hone her skills.

If you identify with Yan, review the SEBoK guidance for systems engineers.

A Chief Systems Engineer

Jacquie is a chief systems engineer at a mid-size electronics company. She has led teams to successfully deliver several projects, over the years coordinating hundreds of engineers, project managers, and specialists. Jacquie has recently been asked to step in and support the lead engineers in several smaller projects to resolve some common challenges. She has noted that in almost every case, the issues have two root causes: lack of systems perspective and lack of coordination between teams of different disciplines. Jacquie has seen several initiatives to move the workforce in one direction or another fail over her 18 years with the company, so is wary of top-down mandates saying that everyone needs to become a systems thinker or something similar.

Jacquie is a long-time user of the SEBoK and remembers that there's a section somewhere about enabling systems engineers. She quickly finds the section in the Outline (Part 5: Enabling Systems Engineering) and she notices a knowledge area on Enabling Businesses and Enterprises and starts reading. The articles on determining what capabilities organizations need and developing those capabilities are particularly helpful and lead her to a Harvard Business Review article on change management that is really useful.

Armed with this information, Jacquie gets a group of her peers together to discuss the challenges and some potential solutions.

If you identify with Jacquie, review the recommendations for experienced systems engineers.

An Organizational Manager

The primary audience of the SEBoK is individuals working in the SE discipline, but articles are written in a way that should enable non-systems engineers to grasp the basics.

Juan is a mid-level manager at a large firm that makes major weapons systems for his nation's defense department. The organization is trying to alter the culture toward a more integrative and collaborative model. This includes a focus on systems and systems thinking. Juan's leadership has emphasized that it is critical for all levels of management to show support for this initiative and that, without their support, the initiative cannot be successful. Juan has never heard the term "systems thinking" before and the only connotation he has with "systems" is the company's IT systems that are always causing headaches.

Juan Google's "systems thinking" and is immediately overwhelmed by the search results. Inundated with definitions from companies trying to sell systems thinking services or training, universities advertising their programs, and blog posts, which could be by experts or by people who only think they know what systems thinking is, Juan is quickly

overwhelmed. He stumbles upon something that links “systems thinking” to “systems engineering”, which is a term he knows, though he has never been and has no desire to be an “engineer”. He refines his search to include both “systems thinking” and “systems engineering” in Google Scholar to try to find more reputable sources. Again there are tens of thousands of results. He goes to a systems engineer whom he often sees in the break room, and asks him what to do. The systems engineer points him to the SEBoK.

In the SEBoK, Juan quickly finds a knowledge area on systems thinking and several articles and papers that he believes he can trust to give authoritative information. After reading through the articles and references, Juan now feels he has a basic understanding of what systems thinking is and he now understands why this is such a critical part of the company’s strategy for change. Juan now has the vocabulary and knowledge to support the initiative as a member of the management team.

If you identify with Juan, review the guidance for general managers.

References

Works Cited

Hutchison, N., A. Pyster, and R. Cloutier. 2023. "Using the Systems Engineering Body of Knowledge (SEBoK)." in Verma, D. (ed). *Systems Engineering for the Digital Age*. Hoboken, NJ: John Wiley and Sons.

Primary References

Hutchison, N., A. Pyster, and R. Cloutier. 2023. "Using the Systems Engineering Body of Knowledge (SEBoK)." in Verma, D. (ed). *Systems Engineering for the Digital Age*. Hoboken, NJ: John Wiley and Sons.

Additional References

None.

Guidance for Systems Engineering Novices

Some users of the Systems Engineering Body of Knowledge (SEBoK) may be new to the field. This article provides recommended readings for such a user.

Learn the Basic Terms

As discussed in the Introduction to the SEBoK, there are four key terms that you should first understand when learning about systems engineering (SE):

- A system is “a collection of elements and a collection of inter-relationships amongst the elements such that they can be viewed as a bounded whole relative to the elements around them.” Open systems exist in an environment described by related systems with which they may interact and conditions to which they may respond. While there are many definitions of the word “system,” the SEBoK authors believe that this definition encompasses most of those which are relevant to SE.
- An engineered system is an open system of technical or sociotechnical elements that exhibits emergent properties not exhibited by its individual elements. It is created by and for people; has a purpose with multiple views; satisfies key stakeholders’ value propositions; has a life cycle and evolution dynamics; has a boundary and an external environment; and is part of a system-of-interest hierarchy.
- Systems engineering is “an interdisciplinary approach and means to enable the realization of successful (engineered) systems.” It focuses on holistically and concurrently understanding stakeholder needs; exploring opportunities; documenting requirements; and synthesizing, verifying, validating, and evolving solutions while considering the complete problem, from system concept exploration through system disposal.
- A systems engineer is “a person who practices systems engineering” as defined above, and whose systems engineering capabilities and experience include sustained practice, specialization, leadership, or authority over SE activities. These activities may be conducted by any competent person regardless of job title or professional affiliation.

Get an Overview

The next step for someone new to SE is get an overview of the discipline. Part 1: SEBoK Introduction contains four articles particularly helpful to one new to SE.

- The article Systems Engineering Overview frames systems engineering inside the larger topic of ‘Systems Science.’
 - The article Economic Value of Systems Engineering makes the business case for investing in systems engineering as a way to reduce total ownership cost.
 - The article Systems Engineering and Other Disciplines discusses briefly how systems engineers and other engineers interact as they develop complex systems together.
 - Finally, the article Systems Engineering: Historic and Future Challenges gives a quick history of the discipline and discusses what lies ahead.
-

Learn About Systems

Engineering is often described as the application of science to develop new products or systems. Part 2: Foundations of Systems Engineering describes some of the underlying systems principles that form the foundation for systems engineering.

- The Knowledge Area on Systems Fundamentals contains five articles. What is a System? is recommended for a new user.
- The Knowledge Area on Systems Science presents two articles on its history and approaches. Both are recommended.
- The Knowledge Area on Systems Thinking has four articles. The first, What is Systems Thinking?, is recommended on a first reading.
- One of the most important current research and practice areas of SE is Model Based Systems Engineering (MBSE). The Knowledge Area Representing Systems with Models provides the foundation for MBSE. The first three of the five articles in the KA are recommended.

Learn How the Systems Approach Is Applied to Engineered Systems

The Knowledge Area Systems Approach Applied to Engineered Systems describes how systems science and systems thinking lead to the practice of systems engineering. All eight articles are recommended.

- Overview of the Systems Approach
- Engineered System Context
- Identifying and Understanding Problems and Opportunities
- Synthesizing Possible Solutions
- Analysis and Selection between Alternative Solutions
- Implementing and Proving a Solution
- Deploying, Using, and Sustaining Systems to Solve Problems
- Stakeholder Needs Definition
- Applying the Systems Approach

Explore the Methods of Systems Engineering

The SEBoK uses a life-cycle framework to describe the processes that comprise systems engineering. Part 3: SE and Management contains the plurality of the content of the SEBoK in eight knowledge areas. A new user should be familiar with the introductions to each of these Knowledge Areas, and should read further in those KAs of interest.

- Life Cycle Models
 - System Concept Definition
 - System Definition
 - System Realization
 - System Deployment and Use
 - Technical Management Processes
 - Product and Service Life Management
 - Systems Engineering Standards
-

Explore the Applications of Systems Engineering

The SEBoK partitions the body of knowledge between methods and areas of application. Areas of application are classified as:

- Product Systems Engineering
- Service Systems Engineering
- Enterprise Systems Engineering
- Systems of Systems (SoS)

A new user should read the introduction to Part 4: Applications of Systems Engineering and to the four knowledge areas listed above. The reader's interests can then suggest which further reading should be done.

Read Case Studies

Finally, the new user should scan the case studies and vignettes in Part 7: SE Implementation Examples and read a few of those in areas that appeal to the reader. This will help reinforce the fundamentals as well as illustrate the practice of SE.

The following case studies are included:

- Successful Business Transformation within a Russian Information Technology Company
- Federal Aviation Administration Next Generation Air Transportation System
- How Lack of Information Sharing Jeopardized the NASA/ESA Cassini/Huygens Mission to Saturn
- Hubble Space Telescope Case Study
- Global Positioning System Case Study
- Medical Radiation Case Study
- FBI Virtual Case File System Case Study
- MSTI Case Study
- Next Generation Medical Infusion Pump Case Study

For Later Reading

Part 6: Related Disciplines contains a broad selection of Knowledge Areas and Topics that describe how systems engineers work with other disciplines. The Knowledge Area on SE and Software Engineering is particularly important, as modern systems get much of their functionality from software.

Part 5: Enabling Systems Engineering has KAs describing how individuals, teams, and organizations can develop to practice effective systems engineering.

A person new to SE should become familiar with several references that are beyond the SEBoK. They include the INCOSE Handbook, several standards (listed in Systems Engineering Related Standards Landscape), and the main journals of systems engineering (including but not limited to *Systems Engineering*, the *Journal of Enterprise Transformation*, and *Systems, Man, and Cybernetics*).

References

Works Cited

None.

Primary References

None.

Additional References

None.

Guidance for Systems Engineers

Both for the entry-level systems engineer learning the discipline of systems engineering (SE), and the more experienced systems engineer seeking the knowledge required to accomplish a work activity, the SEBoK serves as a primary information source and a quick, comprehensive reference for SE information.

What these system engineers find in the SEBoK includes:

- definitions of terms,
- explanations of basic concepts and principles,
- useful discussions of topics,
- references to articles and textbooks that cover topics in-depth, and
- pointers to additional sources.

How Systems Engineers Use Topics

Researching SE-related subjects, identifying educational resources, and connecting with individuals or organizations which offer specialized expertise are all part of the job for the practicing systems engineer. The time available to the SE for these activities can be quite limited. The SEBoK is designed to ease the pressure on the systems engineer in this situation, in several ways:

- Because its content is based on research, proven practices, and emerging knowledge, the SEBoK makes high-quality information available to the systems engineer right away.
- Being composed of articles of 2000 words or less in most cases, the SEBoK enables the systems engineer to quickly get an overview of relevant topics.
- By providing primary references, each topic offers a direct route to more detailed information.
- Even greater detail, breadth, and a sense of what's relevant in the SE literature are available through the additional references each topic provides.
- Since the SEBoK sources have been reviewed and vetted by a team of experts, the SEBoK helps the systems engineer avoid less reliable information which can be hard to eliminate within Internet search results.
- The systems engineer who needs to connect with educators and researchers can find relevant names and institutions in SEBoK topics and references.

Systems engineers using the SEBoK may choose one or more of several approaches:

- searching on keywords or article names, using the text field, Search ^[1] button, and Go ^[2] button at the top right of each SEBoK page
 - scanning the Quick Links, Outline (where the table of contents is located), or Navigation indexes that appear at the left of each SEBoK page, and following links from there to articles that seem likely to be of interest
-

- searching on keywords using an Internet search engine
- reading through one or more of Parts 1 through 7 in sequence

Reading the SEBoK in sequence is especially suitable for the practicing engineer who is new to SE or is enrolled in an SE-related training course. For this engineer, SE (or some aspect of it) is a subject to be learned comprehensively. This is made easier by navigation links from each article to the previous, next, and parent articles as found in the Table of Contents.

For practicing systems engineers, having the SEBoK makes it possible to gain knowledge more quickly and reliably than they would otherwise. The goal is to spend less time searching for and compiling new information from disparate sources and more time getting work done.

For a team of practicing engineers, the gap in knowledge between more- and less-experienced engineers can be a major obstacle. The SEBoK serves as a tool for the team to build a framework of agreed-upon definitions and perspectives. The consistency of such a framework enhances communication across the team. New teams, especially, can benefit from bridging the gap between legacy and more-recently-acquired knowledge. For more information, see Enabling Teams in Part 5.

How Systems Engineers Use the Examples

The SEBoK is written, for the most part, independent of any particular domain of practice. By design, parts 1 through 6 focus on the discipline of SE and not the numerous domains where SE can be applied.

This lack of domain-specific content is partly offset by Part 7, Systems Engineering Implementation Examples, which consists of case studies and examples drawn from a number of domains where SE is applied. Each example demonstrates the impact of a particular application domain upon SE activities. Examples are generally most useful to the systems engineer when they are aligned with the domain in which the he or she is working, but sometimes ideas from an example in one domain can be usefully applied to situations in another.

Example: Model-Based Systems Engineering Practitioners

For practitioners of model-based systems engineering (MBSE), the Representing Systems with Models knowledge area is of central importance within the SEBoK.

Academic faculty who use the SEBoK to support curriculum development and assessment can refer to the same knowledge area to ensure that their curricula accurately cover the languages and/or methodologies such as System Modeling Language (SysML) and Object-Process Methodology (OPM).

SE researchers, too, can adopt an MBSE approach, making their research products more formal and rigorous by basing them on models.

In MBSE, models of systems support system life cycle activities, including requirements engineering, high-level architecture, detailed design, testing, usage, maintenance, and disposal.

Vignette: Systems Engineering for Medical Devices

Tara Washington has worked as an engineer for the HealthTech medical device company for seven years. Besides continuing to improve her strong software skills, she has shown an aptitude for systems thinking. To better understand the products that her software supports, Tara has taken courses in electrical engineering, mechanical engineering, and physiology. The coursework has helped her to perform effectively as a software system analyst on the SE teams of her last two projects.

HealthTech's Research Division proposes a new concept for a highly programmable radiation therapy device that monitors the effects of the radiation on various parts of the body and adjusts the parameters of the radiation dosage to maximize its effectiveness, subject to a number of safety constraints. The software-intensiveness of the device

leads Tara's project manager to recommend her as the lead systems engineer for the design and development of the product.

Tara welcomes the opportunity, knowing that she possesses enough domain knowledge to take the lead SE role. Even so, she realizes that she has picked up SE skills mainly by intuition and needs to build them up more systematically. Tara begins to consult some of HealthTech's lead systems engineers, and to study the SEBoK.

After reading the SEBoK Introduction, Tara feels that she has a solid overview of the SEBoK. Tara finds that the next topic, Scope and Context of the SEBoK, outlines the key activities that she expects to lead, along with others which will require her to collaborate with systems developers and project and systems management personnel.

The same topic identifies those parts of the SEBoK that Tara needs to study in preparation for her lead systems engineer role:

- SE concepts, principles, and modeling approaches in Part 2 (Representing Systems with Models knowledge area (KA))
- life cycle processes, management, technical practices, in Part 3 (Systems Engineering and Management KA)
- approaches for specifying, architecting, verifying and validating the hardware, software, and human factors aspects of the product, as well as common pitfalls to avoid and risks to manage, also in Systems Engineering and Management
- guidelines for the systems engineering of products, in Part 4: Applications of Systems Engineering, including references
- SE knowledge, skills, abilities, and attitudes (KSAAAs) needed for a project in Part 5: Enabling Systems Engineering including references
- specialty engineering disciplines that may be key to the project's success, in Part 6: Related Disciplines

Tara's awareness of the deaths caused by the Therac-25 radiation therapy device motivates her to study not only the System Safety topic in Part 6, but all of its key references as well.

While reading about SE life cycle process models in Systems Engineering and Management in Part 3, Tara notes the reference to the Next Generation Medical Infusion Pump Case Study in Part 7. This case study strikes Tara as highly relevant to her medical-device work, and she observes that it is organized into phases similar to those used at HealthTech. From the case study, Tara gains understanding of how a project such as hers would progress: by concurrently evaluating technology opportunities, by discovering the needs of various device stakeholders such as patients, nurses, doctors, hospital administrators, and regulatory agencies, and by working through increasingly detailed prototypes, specifications, designs, plans, business cases, and product safety analyses.

The case study mentions its source: Human-System Integration in the System Development Process ^[3] (Pew and Mavor 2007), published by the U.S. National Research Council. Tara obtains this book. In it, she finds numerous good practices for human-systems needs analysis, organizational analysis, operations analysis, prototyping, usability criteria formulation, hardware-software-human factors integration, process decision milestone review criteria, and risk management.

As a result of her SEBoK-based study, Tara feels better-qualified to plan, staff, organize, control, and direct the SE portion of the HealthTech radiation therapy device project and to help bring the project to a successful conclusion.

How Systems Engineers Use the Emerging Knowledge

The SE discipline continues to mature and evolve, incorporating new ideas, processes, and technologies. The SEBoK part on Emerging Knowledge describes some of these trends. For example, it currently includes references to the incorporation of artificial intelligence (AI) to support systems engineering.

Summary

In the SEBoK, practicing engineers have an authoritative knowledge resource that can be accessed quickly to gain essential high-level information, and to identify the best references for in-depth study and research into SE topics when an individual's initial level of understanding is not adequate to get the job done.

The SEBoK is also a resource for practicing engineers who teach, as well as those taking training courses.

References

Works Cited

Pew, R. and A. Mavor. 2007. *Human-System Integration in the System Development Process: A New Look*. Washington, DC, USA: The National Academies Press.

Primary References

None.

Additional References

None.

References

- [1] <https://www.mediawiki.org/wiki/Help:Searching>
- [2] https://meta.wikimedia.org/wiki/Help:Go_button
- [3] https://www.nap.edu/catalog.php?record_id=11893

Guidance for Engineers

The realization of successful complex systems requires experts from many disciplines to work together. This makes the SEBoK useful to engineers with backgrounds in biomedical, civil, electrical, chemical, civil, materials, mechanical, software, and many other engineering disciplines.

Studying the SEBoK enables engineers from disciplines other than systems engineering (SE) to:

- see why good systems engineering practice must involve multiple disciplines,
- appreciate a broader view of systems beyond their specialties,
- understand how their contributions fit into the larger systems picture, and
- prepare to solve more difficult and encompassing problems.

In many cases, engineers who study systems engineering as a supplement to their area of specialization find their professional value enhanced when they put the new knowledge into practice.

Use of Topics

For engineers from non-SE backgrounds, each part of the SEBoK contributes something to the experience of learning about systems engineering.

- Part 1: SEBoK Introduction provides an overview both of systems engineering and of the SEBoK itself
- Part 2: Foundations of Systems Engineering highlights the areas of systems knowledge most relevant to systems engineering, providing a foundation for the theory and practice of systems engineering as explained in Parts 3, 4 and 5
- Part 3: Systems Engineering and Management includes the knowledge areas of Life Cycle Models, System Definition, System Realization, and System Deployment and Use, all highly important when approaching the study of SE from another discipline
 - Also in Part 3, Technical Management Processes includes such relevant topics as risk management, measurement, configuration management, and quality management
- Part 4: Applications of Systems Engineering identifies the SE activities for four kinds of engineered systems, namely products, services, enterprises, and systems of systems (SoS)
 - The primary references and glossary terms — not just the content — for a given type of system are essential reading for an engineer developing or modifying a system of that kind
- Part 5: Enabling Systems Engineering, especially Team Capability, explains how systems engineers and other types of engineers fit into the larger picture of enabling individuals and teams to perform systems engineering activities, and into the larger picture of systems engineering organizational strategies
- Part 6: Systems Engineering and Other Disciplines is key for engineers from non-SE backgrounds
 - Within Part 6, Systems Engineering and Project Management should be of interest to almost all readers, while Systems Engineering and Software Engineering and Systems Engineering and Quality Attributes are naturally most essential for engineers in the respective disciplines
- Part 7: Systems Engineering Implementation Examples illustrates how systems engineering practices, principles, and concepts are applied in real settings and contain universally useful insights

Engineers may be tempted to skip over knowledge areas or topics that sound more like management than engineering stories, for example Technical Management Processes in Part 3 or Part 5. This temptation should be resisted, because these topics are actually about how SE orchestrates the efforts of multiple disciplines, not management in the administrative sense.

Finally, the extensive lists of references throughout the SEBoK provide a basis for further readings.

Vignette: Software Engineer

José Wilks is an entrepreneurial software engineer who wants to learn more about systems engineering principles applied to embedded systems for advanced document identification and verification. He wants to implement best practices in developing highly secure systems for real-time image processing and forensic verification of documents. His company provides a rapid, secure and cost-effective solution for verifying the authenticity of identification, travel, and financial documents, with technology that runs on proprietary tablet computers for portable and fixed locations.

José is knowledgeable about computer hardware engineering, low-level interfaces between hardware and software, and the related trade-offs in embedded devices. His company has developed research prototypes, but without the stringent security requirements for actual field usage linked to government identification databases. The few experimental units which have been sold have fared well in limited testing, but José wants to expand into markets for government agencies, law enforcement departments and the private sector. To make headway into those diverse markets, he will need to confront abundant new constraints and challenges.

José begins his study of SE by skimming the SEBoK Introduction and the Scope and Context of the SEBoK to get an overview of the SEBoK contents. As he reads, he sometimes refers to the *Software Engineering Body of Knowledge (SWEBoK)* (Bourque and Fairley 2014), which José already knows from his many years of experience on software projects. In the SEBoK, José is looking for nuggets of knowledge and pointers that can help his enterprise expand. Here are his notes:

- Part 3: Systems Engineering and Management has concepts that are new to us and that may work. Extra system-level verification and validation (V&V) gates identified in Life Cycle Models can be incorporated in company processes, and the references can help with implementation details. There is also material about system-wide procedures beyond software V&V, and about where to find testing and regulation standards used by various government entities. Together with the traditional software testing already in place, these processes could ensure conformity to the regulations and expedite the product's approval for use.
- Though the system concept is proven, the company must still convince potential buyers of the system's financial benefits while demonstrating that all security criteria are satisfied. To do that, we must better understand the needs of the stakeholders. In expressing system requirements and benefits, we need to start using the terminology of users, corporate/government purchasers, and regulatory agencies. Stakeholder Needs Definition is relevant here. The company needs to quantify expected return on investment (ROI) for its products.
- System Realization addresses our broader V&V concerns. We need to demonstrate the measures we are taking to boost reliability of system performance. The standard models and measures for system reliability described in the SEBoK are new to us — now staff must develop tests to quantify important attributes. We may want to model reliability and system adherence to regulations using a form of model-based systems engineering (MBSE). We can learn more about this from the references.
- Technical Management Processes makes it clear that new configuration management (CM) and information management (IM) procedures need to be adopted for federal database controls and integrity. We can use the references in Systems Engineering Standards to learn how to define processes and develop test cases.
- Part 5: Enabling Systems Engineering makes a convincing case that having the right people for a new systems engineering culture is critical. We should probably hire a systems engineer or two to augment our engineering department expertise.
- Our application must deal with private data concerns, and Part 7: Systems Engineering Implementation Examples, particularly the FBI Virtual Case File System Case Study, could help us avoid pitfalls that have hurt others in similar situations. We can put this in context based on System Security in Part 6: Related Disciplines, and then follow up with further study based on the references.

Now José feels that he is better prepared to adapt his processes for new system lifecycles and environments, and that he can see a clear path through the morass of agencies and regulations. His priorities are to quantify the value

proposition for his technology innovations, make inroads into new markets, and strengthen his staff for the long-term enterprise.

Vignette: Mechanical Engineer

Cindy Glass is a mechanical engineer whose experience in the petroleum industry has focused on large-scale oil extraction equipment in the field. Now Cindy is tasked with helping to manage the development of new offshore oil platforms featuring robotic technology and computer networks. This calls for incorporating SE principles from day one to cope with the systems considerations, which are broader than anything in Cindy's previous experience.

Some of the drilling is to be done with remote-controlled, unmanned underwater vehicles (UUVs). Along with safety, which was always a major concern, cybersecurity now takes center stage. Hostile state actors, "hacktivists," or others could cause havoc if they succeed in taking control of the remote vehicles or other infrastructure. Unfortunately, software system implementation is completely new to Cindy, who realizes that this entails dealing with many more engineering disciplines and dimensions of system constraints than she previously encountered.

Cindy is accustomed to implementing minor design changes in existing equipment, with automation and safety guidelines already in place. Now she is starting from scratch with the earliest stages of the platform lifecycle. While Cindy understands trade-offs involving mechanical sub-systems like rigs and drilling materials, she must now broaden her system analysis to include new environmental constraints and system security.

Cindy consults the SEBoK and discovers that for her effort to understand system design with many "-ilities," System Realization is a good starting point and its references should provide the in-depth information she needs.

The project lifecycle requires pursuing several major activities concurrently:

- engineering platform sub-components
- evaluating technology opportunities
- understanding the needs of all stakeholders inside and outside the company
- progressing through increasingly detailed prototypes, working slices of software, system specifications, designs, plans, business cases, and security and safety analyses of the platform architecture and its operations.

To understand how to manage such a project lifecycle, Cindy turns to Part 3: Systems Engineering and Management. The planning section provides detailed advice for starting out. Cindy expects to conduct her management activities on a rigorous basis, to consider the interfaces between the engineering specialties, and to produce a project plan that calls for a broad set of integrated management and technical plans.

Being new to the software development world, Cindy reads *The Nature of Software and Key Points a Systems Engineer Needs to Know about Software Engineering*, and consults the SWEBoK ^[1] for references on software engineering.

These readings show Cindy how closely systems engineering and software engineering are intertwined. For example, they remind her to include security specialists at both the software level and the systems level from the beginning.

From her initial plunge into study of the SEBoK, Cindy has gained an appreciation of the wide range of system constraints for which she must account, and the many engineering disciplines she must work with as a result. She plans to consult the references in the SEBoK on each unfamiliar subject that she encounters throughout the architecting, design, development and deployment of the new platforms.

Summary

Engineers from disciplines other than systems engineering benefit from the insights about SE principles that the SEBoK provides. Studying the knowledge areas highlighted in this use case and the sources to which their references point can help such engineers become more interdisciplinary. Ultimately, they can consider broadening their work responsibilities, rendering them more valuable to their employers and society.

References

Works Cited

Bourque, P. and R.E. Fairley. Eds. 2014. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.swebok.org>

Primary References

None.

Additional References

None.

References

[1] <https://www.computer.org/portal/web/swebok>

Guidance for Systems Engineering Customers

Customers of systems engineering (SE) provide resources to SE organizations and individuals, and receive SE products and services in return. They are among the stakeholders for a system-of-interest (SoI). They and other stakeholders express needs and expectations for results that systems engineers provide.

Although their main SE activity is helping to define the system, customers must take account of all life cycle aspects. The better they understand the activities that systems engineers perform, the better customers know what to request, how to request it, how much to pay for it, and how to judge the quality and value of the results of systems engineering. In short, what customers need to grasp is how systems engineers participate in the realization of engineered systems resulting in products, services, enterprises, and systems of systems (SoS).

The SEBoK assists the customers of systems engineering by providing a broad, comprehensive treatment of the concepts, principles, theory, and practice related to systems in general and SE in particular. Its references inform customers about books and articles that provide important perspectives on systems and SE.

Customers of SE include:

- sponsors of internal SE organizations,
- organizations that maintain long-term customer-domain relationships with external SE organizations, and
- organizations that outsource SE functions to general-purpose SE organizations.

The two vignettes below show how the SEBoK can assist SE customers. In one, the customer of an internal, corporate SE organization leads the transition to a mobile supply chain management system. In the other, the customer of a mixture of customer-domain and other SE organizations presides over the SE of a catastrophe-response SoS, which entails integration over multiple domains.

Use of Topics

For customers of SE, most parts of the SEBoK offer immediately relevant knowledge about SE.

Part 1: SEBoK Introduction:

- explains the relationship between SE, system development, and project management,
- summarizes overall trends in the rate of growth of systems interdependency, complexity, assurance levels, and pace of change, and of the evolving nature of integrated hardware-software-human systems, and
- provides pointers to other parts of the SEBoK of interest to customers.

Part 3: Systems Engineering and Management:

- explains evolving system life cycle models and their elements, indicating which elements are SE-intensive (see Life Cycle Models),
- provides overall perspectives on customer participation in SE activity,
- identifies customer influence points on SE activity, and
- explains how customers can express their concerns in the form of needs, expectations, and requirements.

Part 4: Applications of Systems Engineering:

- explains how the SE function varies by class of system product, service, enterprise, and systems of systems engineering).

Part 6: Systems Engineering and Other Disciplines:

- explains how SE relates to project management, procurement and acquisition, and specialty engineering for such customer-intensive specialties as safety, security, maintainability, usability, and affordability.

Part 7: Systems Engineering Implementation Examples:

- provides case studies and examples to illustrate how the parts have been used in similar situations, presenting successes to emulate and failures to avoid.

If there is a central theme here, it is that the quality of customer input is critical. That is because the systems engineer evaluates customer input, then uses it in formulating an approach to defining and realizing the system. Part 3 addresses this, explaining that the customer should expect the systems engineer to provide:

- a well-architected product, service, enterprise, or system of systems that meets customer needs and expectations (again, this depends on high quality input from stakeholders — see System Concept Definition)
- a managed life cycle model from the customer need and requirements to the delivered product, service, enterprise or system of systems (see Life Cycle Models)
- both verification that the system-of-interest (SoI) meets the needs and requirements of the stakeholders, and validation that the final result, when deployed in an operational environment, provides the value added that was desired are critical to systems engineering (see System Realization and System Deployment and Use).

Implementation Examples

Good examples provide a basis for deeper understanding. In Part 7, the SEBoK provides summaries of and references to full case studies and examples. These are linked back to the appropriate areas of the SEBoK and a matrix is provided that shows the primary areas of the SEBoK addressed by each example. Readers can use the matrix to find case studies and examples- and through these, references - that relate to their concerns.

Vignette: Mobile Supply Chain Management

Barbara Bradley is the Director of Supply Chain Management Systems for a large manufacturing company. Her main area of expertise is transportation logistics. She has led the evolution of a highly successful corporate supply chain management system based on desktop and mainframe technology, more by making incremental strategic choices than by applying formal SE.

Now, many of her suppliers and distributors adopt mobile devices and cloud services and Barbara sees that her own company must do the same. The company's status quo approach of incremental, ad hoc choices is clearly inadequate for a technology transition of this magnitude. Not only that, but the company must evolve to the new mode of operation while providing continuity of service to the supply chain stakeholders.

Barbara decides that these challenges require formal SE. As a first step, she plans to put together a Next-Generation Supply Chain Management System integrated product team (IPT). Members of the IPT will include Barbara's supply chain experts, her supply-chain success-critical stakeholders, and the corporate SE organization.

Barbara has never used the corporate SE organization before and wants to better understand an SE organization's overall capabilities and modes of operation. She turns to the SEBoK for answers to the questions about SE that are on her mind:

- How do we maintain continuity of service while pursuing incremental development?
 - What choices about life cycle models can make this possible?
- What is the role of the customer in defining systems of interest (SoIs)?
 - How do we provide guidance to the customer in expressing needs, concerns, and requirements?
- What is the role of the customer at early decision milestones?
 - How do we ensure that results of our interaction with the customer include well-architected products and thorough development plans, budgets, and schedules?
- What is the role of the customer in product acceptance, specifically when we verify stakeholder requirements and when we validate the final result?

Barbara seeks the answer to one question in Part 4: Applications of Systems Engineering:

- Given that a supply chain management system combines product, service, enterprise, and SoS views, how do we understand what goes into all those views, and keep the overall picture clear?

Barbara's final question is addressed in Part 6: Systems Engineering and Other Disciplines:

- How do we integrate SE and software engineering (SwE)?

Once in command of the answers to these questions, Barbara is ready to lead the IPT in analyzing, negotiating, and defining an approach that is satisfactory to all of the success-critical stakeholders. By having the IPT members read the portions of the SEBoK that she has found most valuable, Barbara begins to build a shared vision within the IPT. As the IPT defines a Next-Generation Supply Chain Management System and prepares the transition from the old system to the new, the SEBoK is an important tool and resource.

Vignette: Catastrophe-Response System of Systems

Ahmed Malik is the Information Systems Division General Manager in his country's Department of Natural Resources. The country suffers frequent wildfires that destroy crops, forests, villages, and parts of cities, and also cause problems with emergency care, crime prevention, and the water supply.

During a recent catastrophic wildfire, personnel responsible for firefighting, crime prevention, traffic control, water supply maintenance, emergency care facilities, and other key capabilities found themselves unable to communicate with each other. As a result, the Minister for Natural Resources has been tasked with improving the country's catastrophe response capabilities, and has named Ahmed as the SE customer lead for this effort.

The Minister suggests that Ahmed organize a workshop to scope the problem and explore candidate solutions to the communications problems. Ahmed invites the various actors involved in catastrophe response — medical, insurance, and news media organizations from both public and private sectors. He also invites SE organizations with SoS experience.

Ahmed has strong experience in information SE, but none in the development of SoSs. To come up to speed in his role as the SE customer lead, Ahmed turns to the SEBoK Part 3: Systems Engineering and Management. To better

understand the challenges of SoS SE, he studies the SoS knowledge area in Part 4, and its references. Ahmed also schedules meetings with the leading SoS SE provider organizations, who are eager to tell him about their capabilities. Overall, Ahmed looks for both guidance and pointers to candidate solution sources in the SEBoK.

Thus prepared, Ahmed structures the workshop to address three key challenges:

- mutual understanding of organization roles, responsibilities, and authority
- summary analyses of previous catastrophe response communication gaps and needs
- candidate solution capabilities in communications, data access, geolocation services, public emergency warning systems, coordinating evacuation procedures, architectural connector approaches for improving interoperability, and sharable models for evaluating alternative solution approaches.

The workshop brings the primary organizations involved in catastrophe responses together with the most capable SoS SE provider organizations. The results of their discussions provide Ahmed and his Minister with sufficient information to prepare a phased plan, budget, and schedule for incremental development of improved catastrophe response capabilities, beginning with simple interoperability aids and analyses of architecture alternatives for performance, scalability, and feasibility of evolution from the initial simple fixes. The plan is then iterated with the key stakeholders and converged to a common-consensus approach for achieving strong, credible early improvements and a way forward to a much more scalable and cost-effective catastrophe-response SoS.

This vignette is based on the Regional Area Crisis Response SoS (RACRS) in (Lane and Bohn 2010).

Summary

For the customers of SE, the SEBoK provides both general and specific knowledge that will help users gain important insight in relating to systems engineers. Key to this is learning about life cycles, the definition of SoIs, and how to provide guidance in expressing needs, concerns, and requirements. Further, customers need to know what to expect as a result of SE activities in the form of well-architected products, services, enterprises, or systems of systems and a managed life cycle. The results of verification of stakeholder requirements and the validation of the final result in respect to fulfilling the user needs are vital.

References

Works Cited

Lane, J. and T. Bohn. 2010. *Using SysML to Evolve Systems of Systems*. Los Angeles, CA, USA: USC CSSE Technical Report. USC-CSSE-2010-506.

Primary References

INCOSE. 2011. *Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.

Jamshidi, M. Ed. 2009. *Systems of Systems Engineering - Principles and Applications*. Boca Raton, FL, USA: CRC Press.

Sage, A., and Rouse, W. Eds. 1999. *Handbook of Systems Engineering and Management*. Hoboken, NJ, USA: John Wiley and Sons, Inc.

U.S. Department of Defense. 2008. *Systems Engineering Guide for System of Systems*, version 1.0. Arlington, VA, USA: U.S. Department of Defense. Available at: <https://www.acq.osd.mil/se/docs/SE-Guide-for-SoS.pdf>. Accessed April 18, 2013.

Additional References

P. Bourque and R.E. Fairley Eds. 2014. *Guide to the Software Engineering Body of Knowledge*, Version 3.0. Los Alamitos, CA, USA: IEEE Computer Society. Available at <https://www.swebok.org>.

Guidance for Educators and Researchers

For educators or researchers, the SEBoK should be used together with GRCSE (*Graduate Reference Curriculum for System Engineering*). The SEBoK is a guide to the knowledge that constitutes the systems engineering domain, while GRCSE ^[1] “describes a program for a professional master’s degree focused on developing student ability to perform systems engineering tasks and roles” (Pyster et al. 2012).

An educator, for purposes of this use case, is a university faculty member or a professional trainer. Educators use the SEBoK and the GRCSE to develop curricula or courses focused on systems engineering (SE) generally, on domain-centric systems engineering, or on another engineering discipline that touches on SE. The SEBoK and GRCSE are means to assure accuracy, completeness, and effective assessment at all levels, from lessons through objectives.

A researcher, for purposes of this use case, is a person actively contributing to the body of SE knowledge.

The Use of Topics

Educators can use SEBOK topics and their primary and additional references as:

- assigned readings for courses,
- supplemental references for student research, and
- content for curriculum development.

Educators can also use the concepts, perspectives, and references to develop or refine course objectives and the techniques for assessing them.

Researchers can use SEBoK topics and their primary and additional references to learn about the state of the art in the subject areas of interest, for summaries of the literature, and to look for opportunities to advance those areas by further research.

A good course or research topic should reflect multiple perspectives, which the SEBoK provides. As well, cataloging the wide diversity in accepted practices across SE is an important function of the SEBoK from the researcher’s perspective.

For both educators and researchers, the fact that the SEBoK provides both primary and additional references in each topic is useful. So is the fact that the SEBoK is a wiki, which allows frequent updates to keep pace with the dynamic evolution of the systems engineering domain. See Acknowledgements and Release History.

Implementation Examples

Good examples make for good teaching. The Systems Engineering Implementation Examples in the SEBoK consist of relatively in-depth case studies and shorter examples, which are linked back to appropriate areas of the SEBoK. A matrix shows which SEBoK topics are addressed by each case study or vignette.

Each case study in the SEBoK is actually a summary of an original case found in the SE literature, and is accompanied by a reference to the full, published case study. Case study summaries or examples from the SEBoK may be incorporated in curricula.

Educator

University faculty may use the SEBoK and GRCSE to develop:

- a complete SE curriculum,
- a single course in systems engineering, either for use in an SE curriculum, or in a curriculum that belongs to some other discipline, or
- assessment criteria for curricula or courses.

Likewise, professional trainers use the SEBoK to develop training material, or to evaluate or update existing training courses.

Both faculty and trainers pursue professional development, in the form of SE study, using the SEBoK.

Vignette: Curriculum and Course Development

A university designates a faculty team to investigate the feasibility of developing a graduate degree in SE.

Results of preliminary feasibility analysis (including evaluating the market, competing degree programs, and so on) are encouraging. The faculty team then begins to design the program, by identifying:

- program constituents
- potential objectives, outcomes and entrance requirements, based on review of GRCSE
- one half of the curriculum content based on review of the typical curriculum architecture (GRCSE chapter 5) and the core body of knowledge (CorBoK) (chapter 6) of GRCSE and
- the other half of the curriculum content based on review of the SEBoK (Parts 2 through 7) .

According to the GRCSE, 50% of the total knowledge conveyed in a graduate program should be based on the CorBoK, to ensure a common foundation among programs offered at different institutions. At the same time, restricting the CorBoK to no more than 50% encourages a healthy variety in those programs.

Once these steps are complete, the overall architecture and the content and scope of the curriculum are defined. Now the faculty designs the courses themselves, defining in turn:

- the prerequisites for each course
- the overall course sequencing for the curriculum, based on the course prerequisites
- the objectives and goals for each course and
- the expected outcomes of each course.

Finally, the faculty is ready to develop the content for each individual course.

Defining course content is done based on topics in the SEBoK that cover the subject of the course.

Using primary and additional references as much as the topics themselves, the faculty responsible for course design define:

- the scope of the course content
- the course coverage, that is, what within the course content scope is actually taught in the course.

Given the scope and coverage, the next and final step is to develop the course material.

A professional trainer designing the training material performs the same kinds of activities. To customize the training course for a specific industry or customer, the trainer may integrate domain-specific content as well.

Researcher

Researchers use SEBoK topics and their primary and additional references to learn about the state of the art in the subject areas of topics, and to look for opportunities to advance those areas by further research.

Vignette: Software Engineering Research

William McGregor, a software engineer, wants to learn more about software intensive systems (SIS). Initially, William wants to answer the question: Do the activities and practices used to develop SIS represent special treatments of standard activities and practices?

William has already reviewed the SWEBoK and its primary references extensively for an answer to his question. In the course of his research, William learns about the SEBoK and decides to look there, too.

William finds no specific discussion of the SIS within the SEBoK. As he looks through the SEBoK, though, he realizes that there are activities throughout the system development life cycle which can be adapted or customized for the development of SIS. Accordingly, William decides to replace his original question with two new ones: (a) what best practices are applied throughout the software development life cycle and (b) how can those practices be adapted to SIS?

William now focuses on Part 3 to learn about the system development life cycle and identify development activities and practices that he can customize for software intensive systems.

Summary

Educators use the SEBoK as a framework or a resource which helps them:

- determine what subject matter should be included in a new curriculum
- identify gaps in an existing curriculum and craft plans to address those gaps, and
- design individual courses.

The case studies and vignettes in the SEBoK may be used by educators in the classroom.

To develop curricula at the program level, educators should use the SEBoK in tandem with the GRCSE.

Researchers use the SEBoK to learn about the state of the systems engineering discipline, and to look for opportunities to advance that state by further research.

References

Works Cited

Bloom, B.S., M.D. Engelhart, E.J. Furst, W.H. Hill, and D.R. Krathwohl. 1956. *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. London, UK: Longman Group Ltd.

Primary References

Pyster, A., D.H. Olwell, T.L.J. Ferris, N. Hutchison, S. Enck, J.F. Anthony, D. Henry, and A. Squires. Eds. 2015. *Graduate Reference Curriculum for Systems Engineering (GRCSE™)*, version 1.1. Hoboken, NJ, USA: The Trustees of the Stevens Institute of Technology ©2015. Available at: <https://www.bkcase.org/grcse-2/>.

Additional References

None.

References

[1] <https://s3.us-east-2.amazonaws.com/serc.public/GRCSEv1%2B1+Final.pdf>

Guidance for General Managers

General managers preside over system development projects, system acquisitions, product lines, systems of systems (SoSs), and commercial and government organizations. For general managers, the SEBoK serves as a primary information source and quick, comprehensive reference for systems engineering information.

In particular, the SEBoK helps the general manager understand:

- the boundaries and synergies among systems engineering (SE), systems development, project management (PM), and life cycle support
- how those boundaries and synergies are likely to evolve with increasing use of evolutionary development, lean and agile methods, and systems that provide purchased services as opposed to salable products
- how to best balance a mix of hardware, software, human factors, domain, and specialty-area systems engineers and
- how an organization can evolve to take advantage of the trend towards cross-discipline systems engineers.

Use of Topics

For general managers, most parts of the SEBoK offer immediately relevant knowledge about SE.

Part 1:

- explains the relationship between SE, system development, and project management
- summarizes overall trends in the nature of systems interdependency, complexity, assurance levels, and pace of change
- describes the evolving nature of integrated hardware-software-human systems and
- provides pointers to other parts of the SEBoK of interest to general managers.

Part 3:

- explains evolving system life cycle models and their elements, indicating which elements are SE-intensive (see Life Cycle Models) and
- provides overall guidance on the management of SE activity.

Part 4:

- explains how the SE function varies by class of system product, service, enterprise, and systems of systems engineering).

Part 5:

- explains SE governance and competence development.

Part 6:

- explains how SE relates to software engineering, project management, industrial engineering, procurement and acquisition, and specialty engineering for such specialties as safety, security, maintainability, and usability.

Part 7:

- provides case studies and vignettes to illustrate how the parts have been used in similar situations in successes to emulate and failures to avoid.

Vignette: Emerging Nation Satellite System

Tom Lee is General Manager for Telecommunications in a ministry of a large emerging nation. The government does not have much existing capability for developing capital-intensive infrastructure projects. The government decides to use a major investment in technology as a vehicle to develop national enterprise capabilities.

To accomplish this, the minister assigns Tom to lead a project to develop a national satellite system for telecommunications and earth resources observation. Tom understands that this is a very complex system and decides to do some background research. During this research, Tom discovers the SEBoK and decides that it may be a useful resource.

Tom first reads:

- Part 1 for an overview and pointers to relevant sections of Parts 3 through 6,
- portions of Part 3, Part 4, Part 5, and Part 6 to learn about the life cycle, nature, scope, and management aspects of enterprise SE,
- the successful satellite system case studies in Part 7 (Global Positioning System, Miniature Seeker Technology Integration spacecraft) for approaches to emulate, and
- the satellite system case study in Part 7 which describes development and integration problems (Hubble Space Telescope) for pitfalls to avoid.

Tom continues by carefully reading Part 5. He realizes that he must simultaneously develop individuals, teams, and the enterprise. The knowledge areas (KAs) from Part 5 give useful background. For this project, Tom enlists both a proven multi-national satellite SE company and some of his brightest aerospace systems engineers. Tom expects his local systems engineers to learn from the SE company, and he plans to use them as the core group of the national satellite system as it ultimately develops and operates.

He realizes that correct problem definition and requirements setting will be critical first steps. He carefully reads the System Concept Definition KA. As his team develops the stakeholder needs and the system requirements, he makes sure they follow good practices as listed in the SEBoK. Once architectural designs have been proposed and approved, he requires his team to perform cost-benefit trade-off analyses of alternative solutions.

Thus prepared, Tom is confident that he can formulate and execute a successful approach.

Vignette: Commercial Safety Equipment Company

Maria Moreno is General Manager at Safety First Equipment Company, specialists in hardware-intensive safety equipment. Maria's background is in electromechanical systems. Safety First is highly successful but beginning to lose market share to competitors who offer software-intensive capabilities and user amenities.

Maria is preparing an initiative to make Safety First into a leading software-intensive safety equipment provider. She decides to make the SEBoK a primary resource for gathering concepts and insights for the initiative. She begins by skimming through all of Parts 1 through 6, both to become familiar with the SEBoK itself and to start organizing her thoughts on SE.

Now Maria is ready to focus on subjects of prime importance to her task. Here are those subjects, listed with the places in the SEBoK where she finds information about them.

In Systems Engineering and Software Engineering in Part 6:

- the nature of software,
 - differences between hardware and software architectures and practices and
 - key aspects of managing software teams.
-

In the article Human Systems Integration in the Systems Engineering and Quality Attributes knowledge area, also in Part 6:

- the SE of user amenities.

In the Next Generation Medical Infusion Pump Case Study in Part 7:

- the software aspects of safety practices, such as software fault tree analysis and failure modes and effects analysis and
- overall approaches for concurrent engineering of the hardware, software, and human factors aspects of safety-critical equipment.

In the Medical Radiation Case Study in Part 7:

- hardware-software pitfalls to avoid in safety-critical equipment.

Maria chose the last two items from among the case studies in Part 7 because being safety-critical, they contain lessons directly applicable to her initiative at Safety First.

With this framework of concepts and practical information in place, Maria begins assembling a core team of Safety First systems engineers, complemented by external experts in software and human factors engineering. Maria wants the team to begin by developing a shared vision. To that end, she asks them to read the portions of the SEBoK that she has found most valuable in assessing the challenges of transitioning Safety First into a leading software-intensive, user-friendly safety equipment provider.

Summary

For the general manager whose organization includes systems engineers, the relationship between SE, systems development, project management, and life cycle support is a central concern. The SEBoK provides insights and guidance about this and other aspects of SE principle and practice, and explains the role of SE in a variety of management challenge areas and application domains.

The SEBoK complements the general management guidance available in sources such as the *PMBOK® Guide* (PMI 2013).

References

Works Cited

PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Primary References

PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Additional References

P. Bourque and R.E. Fairley. Eds. 2014. *Guide to the Software Engineering Body of Knowledge*, Version 3.0. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.swebok.org>

Booher, H. 2003. *Handbook of Human-Systems Integration*. New York, NY, USA: John Wiley & Sons Inc.

Hooks, I.F. and K. Farry. 2000. *Customer Centered Products: Creating Successful Products Through Smart Requirements Management*. New York, NY, USA: AMACON/American Management Association.

Pew, R. and A. Mavor. 2007. *Human-System Integration in the System Development Process*. Washington, D.C., USA: The National Academies Press.

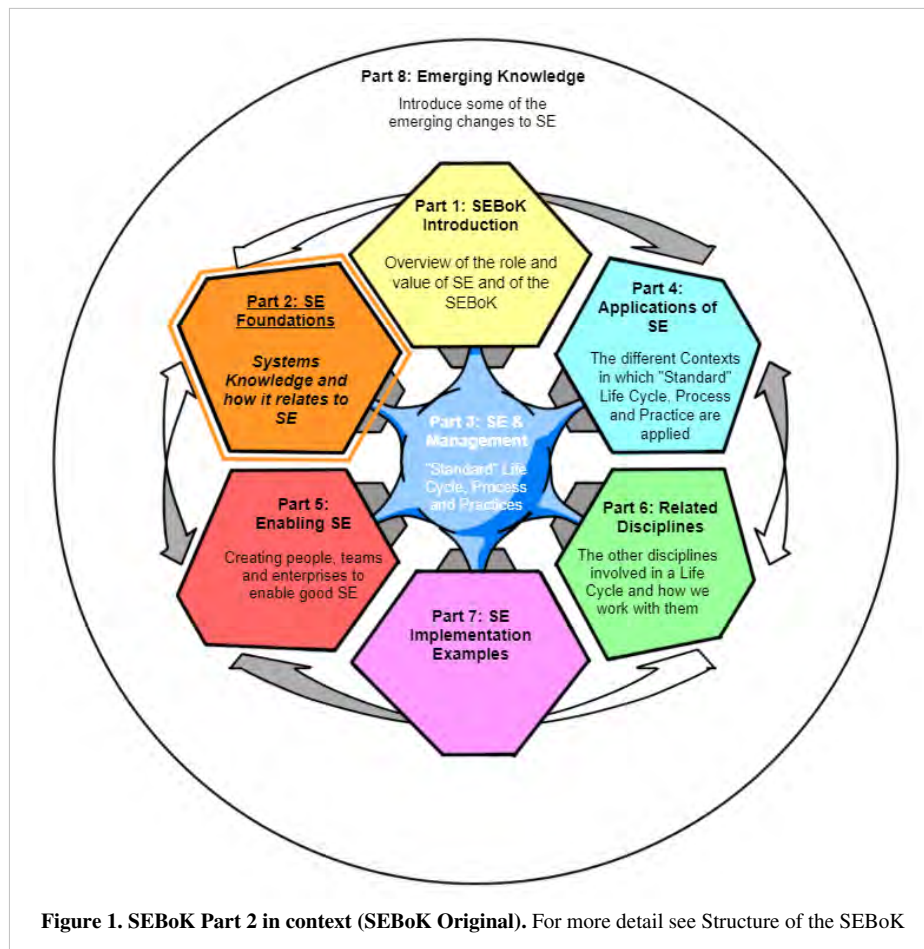
Part 2: Foundations of Systems Engineering

Foundations of Systems Engineering

Contents of this Part

- Systems Engineering Fundamentals (Rick Adcock) (Janet Singer, Duane Hybertson, and Gary Smith)
- The Nature of Systems (Randall Anway, Paul McGoey, Curt McNamara, and Gary Smith)
- Systems Science (Gary Smith)
- Systems Thinking (Rick Adcock)
- Representing Systems with Models (Sanford Friedenthal) (Dov Dori and Yaniv Mordecai)
- Systems Approach Applied to Engineered Systems (Rick Adcock) (Janet Singer and Duane Hybertson)
- Lead Author:
- Rick Adcock
- Contributing Authors:
- Scott Jackson, Janet Singer, Duane Hybertson, and Gary Smith

Part 2 of the SEBoK provides the conceptual foundations of systems engineering. It equips systems engineers with an understanding of the principles, theories, and practices that underpin their discipline, while also connecting them to the broader traditions of systems science and systems practice. By presenting this wider integrative context, Part 2 makes systems knowledge more accessible both within engineering and to a wider community of practitioners and researchers working across diverse domains.



This knowledge is included in the SEBoK firstly to help systems engineers benefit from an understanding of the foundations of their discipline, and to provide them with access to some of the theories and practices of systems science and other fields of systems practice. Including this wider integrative systems science context in the SEBoK should also help to make SE knowledge more accessible to a wider audience outside of its traditional domains.

Knowledge Areas in Part 2

Each part of the SEBoK is divided into knowledge areas (KAs), which are groupings of information with a related theme. Part 2 contains the following KAs:

- Systems Fundamentals introduces essential definitions, concepts, principles, and heuristics that underpin all of SE.
- The Nature of Systems explores the diversity of systems in the world, from natural and engineered to socio-technical, and the universal patterns they share.
- Systems Science surveys theoretical advances and enduring insights that support systemic understanding across domains.
- Systems Thinking describes the perspectives, principles, and heuristics that guide how we frame and approach complex problems.
- Representing Systems with Models highlights the role of abstraction and modeling in understanding, designing, and managing systems.
- Systems Approach Applied to Engineered Systems presents structured methods for applying systems concepts directly to real-world engineering practice

Introduction

Most systems engineers are practitioners, applying processes and methods that have been developed and evolved over decades. SE is a pragmatic approach, inherently interdisciplinary, yet specialized in its focus on integration and whole-system effectiveness. Practitioners typically work within a specific domain, using approaches tailored to that domain's problems, constraints, risks, and opportunities. These approaches capture the accumulated knowledge of domain experts regarding the most effective ways to apply systems engineering.

Specific domains in which systems approaches are used and adapted include:

- Technology products, integrating multiple engineering disciplines
- Information-rich systems, such as command and control, air traffic management, or health informatics.
- Platforms, e.g. aircraft, civil airliners, cars, trains, etc.
- Organizational and enterprise systems, which may be focused on delivering service or capability
- Civil engineering/infrastructure systems, e.g. roads networks, bridges, builds, communications networks, etc.

Each domain involves different kinds of systems and requires different skill sets. Yet across this diversity, there are underlying **unifying systems principles** that can enhance the effectiveness of systems approaches in any domain. In particular, shared knowledge of systems principles and terminology improves communication and enables systems engineers to integrate complex systems that span traditional domain boundaries (Sillitto 2012). This integrated approach is increasingly needed to solve today's complex system challenges, but as these different communities come together, they may find that assumptions underpinning their worldviews are not shared.

General Systems Engineering Foundations

To bridge the gap between different domains and communities of practice, it is essential to establish the intellectual foundations of systems engineering and a common language for describing its key concepts and paradigms. An integrated systems approach to complex problems must combine elements of systems theories with practical methods. This ranges from the technical-systems focus that has traditionally dominated engineering to the learning-systems focus used in social and organizational interventions. A shared framework and terminology allow communities with diverse worldviews and skills to collaborate effectively toward common goals.

The SEBoK provides principles and concepts that can support all potential applications of systems engineering, while remaining adaptable to particular contexts. Much of the published knowledge in systems engineering originates from specific domains, such as defense, transportation, health, or enterprise systems, or from particular business models and technology areas. In the SEBoK, this specialized knowledge is generalized where possible, so that it can be applied across related applications.

Generalized knowledge may take different forms. In many cases, it is informal generalization, representing current best understanding across domains or capturing common practices. In other cases, it is based on stronger theoretical considerations, offering explanatory power and predictive value across all special cases. SEBoK typically presents the informally generalized view, while identifying and linking specific knowledge to its broader foundations.

Looking ahead, the *INCOSE Vision 2035* highlights the aspiration for systems engineering to mature into a discipline with a formally defined theoretical basis. Such a theory, once developed, would largely reside within Part 2 of the SEBoK. At present, Part 2 provides generalized foundations with pragmatic value to describe and improve current practice, and it will evolve to incorporate any emerging general theory of systems engineering as it becomes available.

The Systems Praxis Framework

The term systems praxis refers to the combined intellectual and practical effort to create holistic solutions to today's complex challenges. *Praxis* means “translating an idea into action” (WordNet 2012), and in this context highlights the need to integrate appropriate theory with effective practice drawn from many different sources. Because complex challenges cut across disciplinary and organizational boundaries, systems praxis requires diverse communities to work together. To collaborate, they must first establish communication; and to communicate, they must first find shared points of connection.

To support this, members of the International Council on Systems Engineering (INCOSE) and the International Society for the Systems Sciences (ISSS), working through the International Federation for Systems Research (IFSR), developed the *Systems Praxis Framework* (IFSR 2012). This framework represents a first step toward a common language for systems praxis, and it has been adopted in the SEBoK as a way of organizing foundational knowledge. It helps make the concepts and principles of systems thinking and practice more accessible to anyone applying a systems approach to engineered system problems.

The Systems Praxis Framework highlights the flows and interconnections among elements of a broader *knowledge ecosystem* of systems theory and practice. This ecosystem situates systems engineering within an integrative context that links foundations, theories, representations, and approaches to practice.

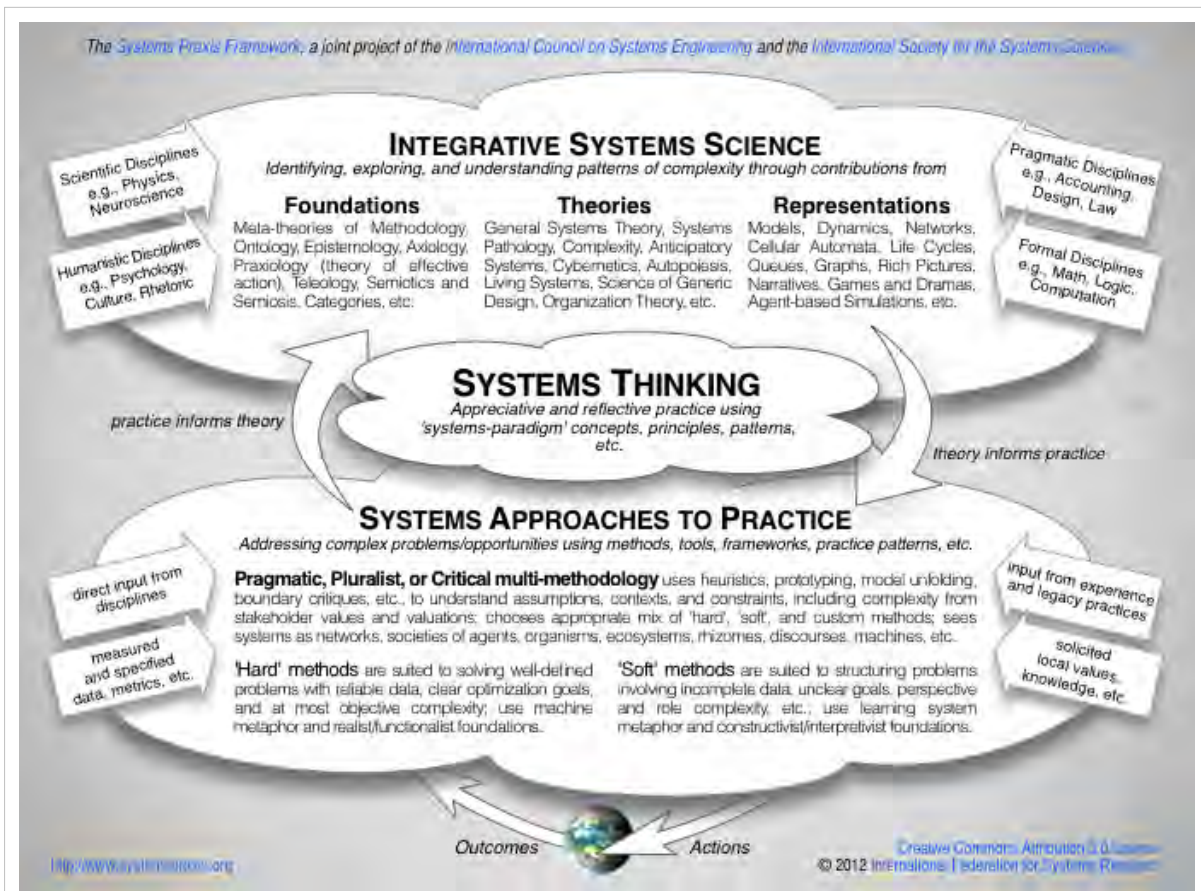


Figure 2. The Systems Praxis Framework, Developed as a Joint Project of INCOSE and ISSS. (© 2012 International Federation for Systems Research) Released under Creative Commons Attribution 3.0 License. Source is available at <https://systemspraxis.org/framework.pdf>.

In this framework, the following elements are connected:

Systems Thinking is the core integrative element. It connects the foundations, theories, and representations of systems science with the hard, soft, and pragmatic approaches of systems practice. Like any discipline underpinned by science, systems praxis involves a continuous interplay between theory and practice: theory informs practice, and

practice generates insights that refine theory. Systems thinking provides the ongoing capability to assess and appreciate the system context, and to guide appropriate adaptation throughout the praxis cycle.

Integrative Systems Science provides the conceptual base and is grouped into three broad areas:

- *Foundations*, organizing knowledge and supporting discovery (e.g., methodology, ontology, epistemology, axiology, praxiology, teleology, semiotics, category theory).
- *Theories*, abstracted from specific domains but universally applicable (e.g., general system theory, complexity, anticipatory systems, cybernetics, autopoiesis, living systems, design science, organization theory, systems pathology).
- *Representations*, models and formalisms for describing, analyzing, and predicting system behaviors and contexts (e.g., system dynamics, networks, cellular automata, life cycles, graphs, rich pictures, narratives, games, agent-based simulations).

Systems Approaches to Practice aim to act on real-world situations to produce desired outcomes while minimizing unintended consequences. No single branch of systems science or practice is sufficient for every problem; therefore, a pragmatic blend is often required:

- *Hard approaches* are suited to well-defined problems with clear goals and reliable data. They emphasize quantitative analysis, optimization, and technical systems, often grounded in “machine” metaphors.
- *Soft approaches* are suited to problems with incomplete data, ambiguous goals, and social complexity. They emphasize learning, communication, and multiple perspectives, drawing from “humanist” philosophies.
- *Pragmatic approaches* combine elements of both, selecting methods, tools, and heuristics as appropriate to the situation. This includes techniques such as boundary critique or model unfolding, enabling diverse stakeholders to make assumptions and constraints explicit.

The “clouds” that collectively represent systems praxis sit within a wider **ecosystem of knowledge, learning, and action**. Successful integration requires drawing on the sciences (e.g., physics, neuroscience), formal disciplines (e.g., mathematics, logic, computation), the humanities (e.g., psychology, culture, rhetoric), and pragmatic fields (e.g., accounting, design, law). Systems practice depends not only on theories and models, but also on data, metrics, local knowledge, and lived experience.

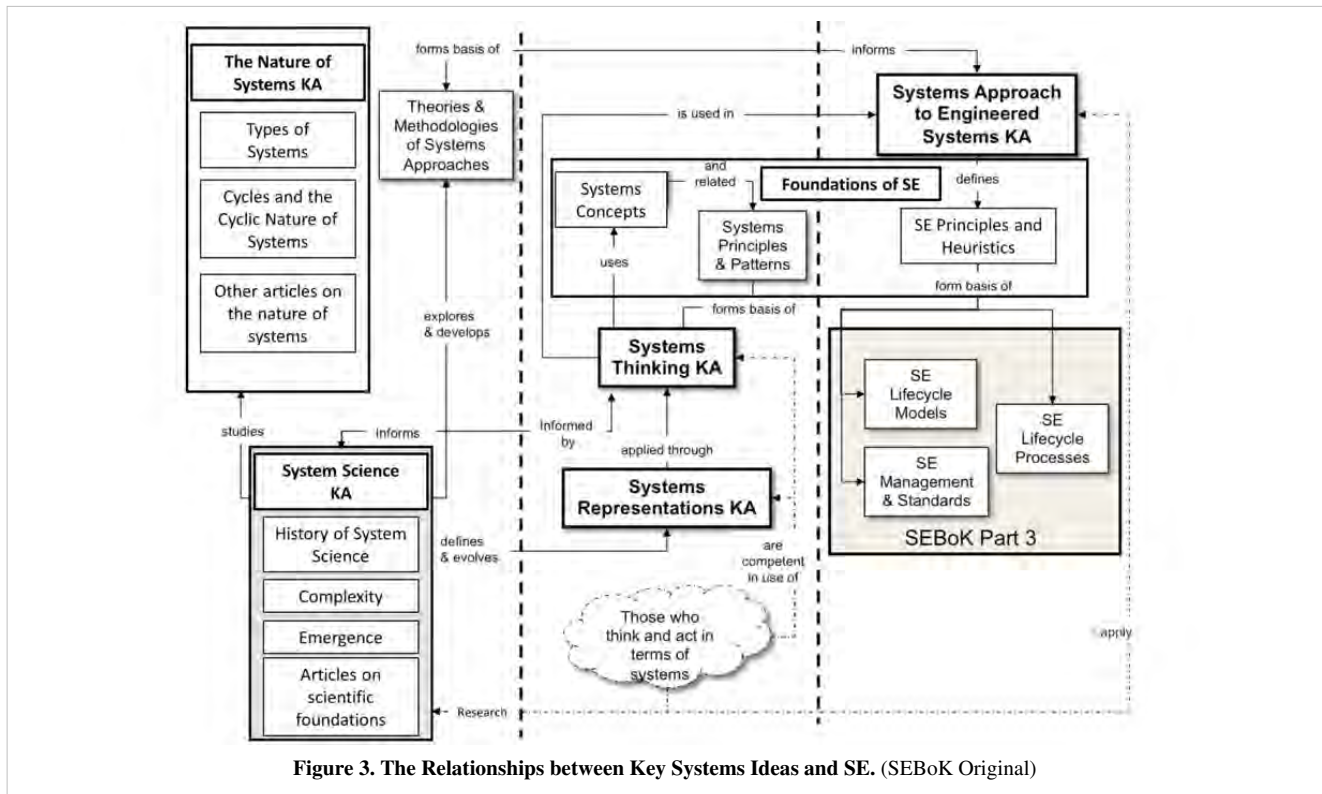
In summary, integrative systems science provides the concepts and patterns that allow us to understand complexity, while systems approaches to practice apply these insights to address real problems. Systems thinking binds them together, creating an adaptive cycle of theory and practice that underpins all systems engineering foundations.

Scope of Part 2

Part 2 of the SEBoK provides a guide to systems knowledge that is most relevant for understanding and practicing systems engineering. It does not attempt to capture the entirety of systems science or systems practice. Instead, it offers an overview of key aspects of systems theory and application that are especially important for systems engineers.

The organization of Part 2 is based on the *Systems Praxis Framework* (IFSR 2012), which emphasizes the need for a clear and shared foundation to support effective communication and collaboration across diverse domains. As the discipline evolves, it is expected that the coverage of systems knowledge in Part 2 will continue to expand.

The following diagram summarizes the way in which the knowledge in SEBoK Part 2 is organized..



The six Knowledge Areas in Part 2 each highlight a different but complementary aspect of systems foundations:

1. Systems Fundamentals, introduces essential definitions, concepts, principles, and heuristics. These provide systems engineers with a baseline vocabulary and the means to distinguish different kinds of systems, including product, service, enterprise, and system-of-systems,
2. The Nature of Systems, surveys the diversity of natural, engineered, and socio-technical systems, drawing attention to universal patterns that underpin systems science and form the basis of systems thinking and systems engineering.
3. Systems Science, presents influential theories and movements in systems research, including the chronological development of systems knowledge and the foundations that inform systems approaches.
4. Systems Thinking, describes the perspectives, principles, and patterns that characterize a systemic worldview. It emphasizes the role of reflective practitioners who integrate research and practice in order to frame and address complex problems.
5. Representing Systems with Models, considers the central role of models in both advancing systems theory and supporting engineering practice, from conceptual understanding to design and analysis..
6. Systems Approach Applied to Engineered Systems, defines a structured approach for problem and opportunity discovery, solution synthesis, analysis, implementation, and use. This knowledge area provides the most direct link between systems foundations and practical SE application.

Taken together, these Knowledge Areas form a “**system of ideas**” that connects research, understanding, and practice. They provide a shared foundation of systems knowledge that underpins a wide range of scientific, management, and engineering disciplines, and that can be applied across all types of domains.

References

Works Cited

IFSR. 2012. *The Systems Praxis Framework, Developed as a Joint Project of INCOSE and ISSS*. Vienna, Austria: International Federation for Systems Research (IFSR). Available at: <https://systemspraxis.org/framework.pdf>.

Sillitto, H.G., 2012. "Integrating systems science, systems thinking, and systems engineering: Understanding the differences and exploiting the synergies", Proceedings of the 22nd INCOSE International Symposium, Rome, Italy, 9-12 July, 2012.

Primary References

Bertalanffy, L., von. 1968. *General System Theory: Foundations, Development, Applications*, rev. ed. New York, NY, USA: Braziller.

Checkland, P. B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons.

INCOSE (2022). *INCOSE Systems Engineering Vision 2025*. San Diego, CA: International Council on Systems Engineering.

Additional References

Blanchard, B., & Fabrycky, W. (2010). *Systems Engineering and Analysis* (5th ed.). Saddle River, NJ: Prentice Hall.

Lawson, H. (2010). *A Journey Through the Systems Landscape*. London: College Publications.

Meadows, D. (2008). *Thinking in Systems*. White River Junction, VT: Chelsea Green.

Mobus, G. (2022). *System Science: Theory, Analysis, Modeling and Design*. Springer.

Ostrom, E. (2009). "A General Framework for Analyzing Sustainability of Social-Ecological Systems." *PNAS*, 104(51): 15181–15187.

Senge, P.M. (1990). *The Fifth Discipline: The Art & Practice of the Learning Organization*. New York: Doubleday.

Troncale, L. (1985–2010). *Selected works on Systems Isomorphies and Systems Processes*.

Volk, T. (2017). *Quarks to Culture: How We Came to Be*. Columbia University Press.

Zwick, M. (2023). *Elements and Relations: Aspects of a Scientific Metaphysics*. Springer.

Knowledge Area: Systems Engineering Fundamentals

Systems Engineering Fundamentals

Contents of this Knowledge Area

- Introduction to Systems Engineering Fundamentals (Rick Adcock) (Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson)
- Systems Engineering Core Concepts (Gary Smith) (Shingai Thornton)
- Systems Engineering Principles (Michael Watson) (Bryan Mesmer, Garry Roedler, David Rousseau, Chuck Keating, Rob Gold, Javier Calvo-Amodio, Cheryl Jones, William D. Miller, Scott Lucero, Aileen Sedmak, and Art Pyster)
- Systems Engineering Heuristics
- Lead Author:
- Rick Adcock
- Contributing Authors:
- Janet Singer, Duane Hybertson, and Gary Smith

This knowledge area (KA) provides a guide to some of the most important knowledge about system engineering fundamentals. System Engineering benefits from systems thinking in combination with an essential understanding of The Nature of Systems. It applies systems approaches to practice grounded in systems science.

This is part of the wider systems knowledge, which can help to provide a common language and intellectual foundation and make practical systems concepts, principles, patterns and tools accessible to systems engineering (SE) as discussed in Part 2: Foundations of Systems Engineering.

Introduction

A concept is a mental representation that can be shared with a word or other media such as visualizations, pictures or displays of emotion (Daniel-Allegro B, Smith G.R. 2016). Systems Engineering Core Concepts provide the mental building blocks that when brought together can support an expression of principles. “A principle is a fundamental truth or proposition that serves as the foundation for a system of belief or behavior or for a chain of reasoning”. As an example, if you have three colors, red, blue, yellow (all words that serve as mental concepts). A principle is if you mix some yellow with some blue then you get green. A principle can provide guidance for acting with some certainty of the result. Principles learnt from practical experience that yet lack a scientific understanding as to the explanation of “why” are referred to as heuristics. Often in the progression of science these observations of ‘what is happening’ form the starting point for asking the question ‘why does this happen’ and prompts research into the underpinning scientific theory.

The evolution of Systems Engineering Heuristics and associated Systems Engineering Principles is described. The objective of system science is to provide the theoretical basis for these principles (the ‘why’ behind the phenomena).

The word system is a concept used in many areas of human activity and at many levels. But what do System Engineers mean when they use the word system? The answer is surprisingly not as simple as you might think.

(Sillitto, Griego et al. 2018) established in their research that there are at least seven distinct, and to a considerable extent mutually incompatible, worldviews on systems within a relatively small population of systems engineering

experts in the INCOSE Fellows' and System Science Working Group (SSWG) communities. The worldviews identified ranged from constructivist ("systems are purely a mental construct") to "extreme realist" ("systems only exist in the real world"). Some considered the threshold of systemness" to be as simple as "two or more inter-related elements"; while others demanded a wide range of attributes, including those of living systems, before considering something to be a system.

That such a wide range of worldviews was discovered within the Systems Engineering community was a surprising result. We might expect to find an even wider range of worldviews across a more diverse community; but a non-rigorous examination of the results of the survey and of the wider literature suggests that the range of "system" worldviews held by the surveyed group of Systems Engineers is broadly representative of the systems field. In the 2019 IFSR conversation "What is System Science?" (Metcalf, G.S, Edson M.C, and Chroust G. 2018), it was concluded that there would be great benefit to achieve a synthesis across the apparently conflicting worldviews of constructivism and realism, expressed in terms of these two key aspects: a) the nature of systems in the physical universe; and b) the way humans perceive and interact with systems.

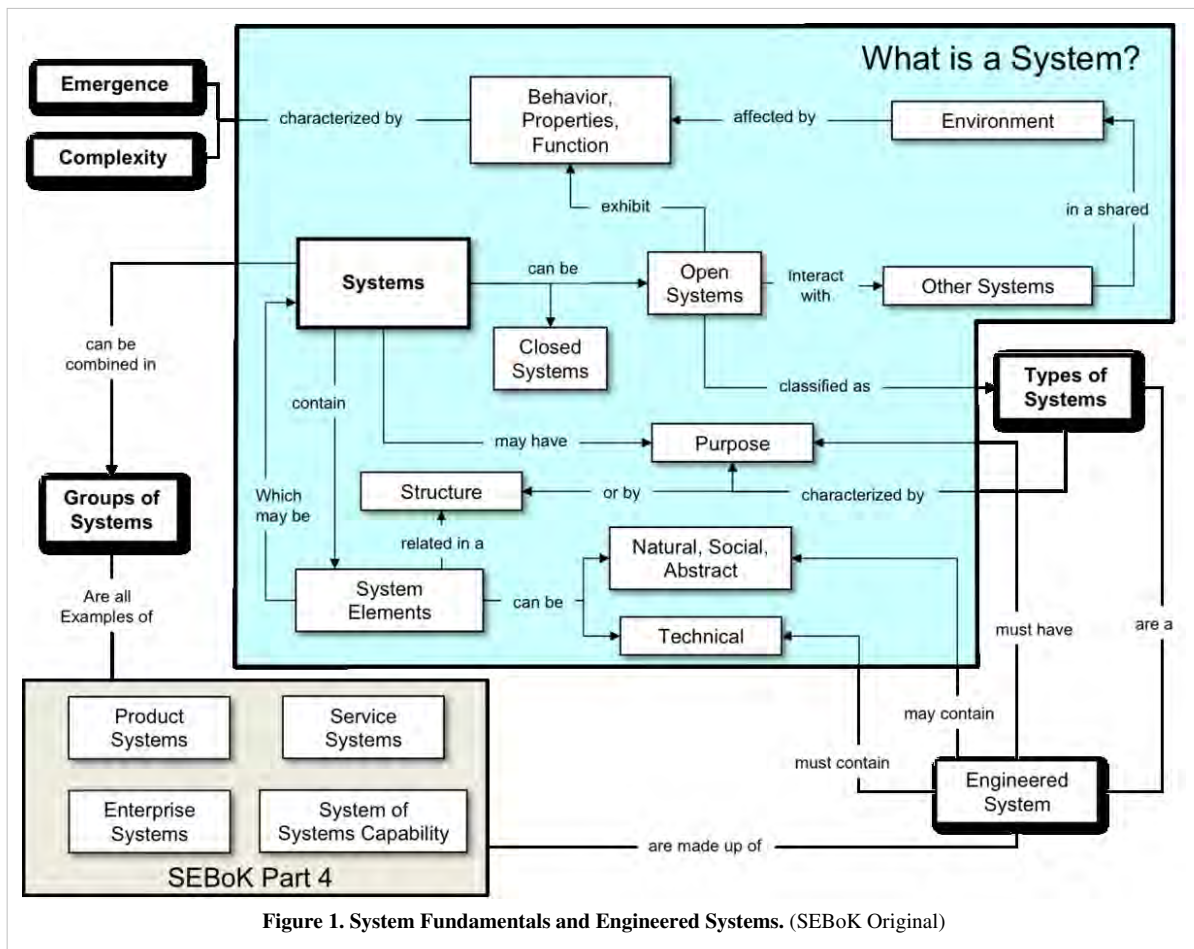
The philosophical foundations of systemness and system research are described (M.C Edson, Henning P, Sankaran. 2017), this work provides a meta-theoretical orientation of systems and an orientation for systems research processes and practices.

In practical terms, in the Introduction to Systems Engineering Fundamentals, the concepts of open system and closed system are explored. Open systems, described by a set of elements and relationships, are used to describe many real-world phenomena. Conceptually closed systems have no interactions with their environment.

Some systems classifications, characterized by type of element or by purpose, are presented.

An engineered system is defined within the SEBoK as encompassing combinations of technology and people in the context of natural, social, business, public or political environments, created, used and sustained for an identified purpose. The application of the Systems Approach Applied to Engineered Systems requires the ability to position problems or opportunities in the wider system containing them, to create or change a specific engineered system-of-interest, and to understand and deal with the consequences of these changes in appropriate wider systems. The concept of a system context allows all of the system elements and relationships needed to support this to be identified.

Two particular aspects of the system phenomena complexity and emergence, are described in the System Science KA. Between them, these two concepts represent many of the challenges which drive the need for systems thinking and an appreciation of systems science in SE. In The Nature of Systems KA, a rich portrayal of the landscape of systems and the system phenomena is provided.



The discussions of engineered system contexts includes the general idea of groups of systems to help deal with situations in which the elements of an engineered system are themselves independent engineered systems. To help provide a focus for the discussions of how SE is applied to real-world problems, four engineered system contexts are introduced in the KA:

1. product system context
2. service system context
3. enterprise system context
4. system of systems (sos) context

The details of how SE is applied to each of these contexts are described in Part 4: Applications of Systems Engineering.

References

Works Cited

- Daniel-Allegro B, Smith G.R. 2016. Exploring the branches of the system landscape, Les editions Allegro Brigitte D. ISBN 978-2-9538007-1-5.
- Edson, M.C., Henning P, and Sankaran. 2017. A Guide to Systems Research - Philosophy, Processes and Practice. Springer.
- Metcalf, G.S., M.C. Edson, and G. Chroust, Systems: from science to practice: Proceedings of the Nineteenth IFSR Conversation 2018, St. Magdalena, Linz, Austria. 2019: Books on Demand.

Sillitto, H., R. Griego, et al. 2018. What do we mean by “system”?—System Beliefs and Worldviews in the INCOSE Community. INCOSE International Symposium, Wiley Online Library.

Primary References

Bertalanffy, L., von. 1968. *General System Theory: Foundations, Development, Applications*, rev. ed. New York, NY, USA: Braziller.

Magee, C. L., O.L. de Weck. 2004. "Complex system classification." Proceedings of the 14th Annual International Council on Systems Engineering International Symposium, Toulouse, France, 20-24 June 2004.

Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press.

Sheard, S.A. and A. Mostashari. 2009. "Principles of complex systems for systems engineering." *Systems Engineering*, vol. 12, no. 4. pp. 295-311.

Tien, J.M. and D. Berg. 2003. "A case for service systems engineering." *Journal of Systems Science and Systems Engineering*, vol. 12, no. 1, pp. 13-38.

Additional References

None.

Introduction to Systems Engineering Fundamentals

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson

This article forms part of the Systems Engineering Fundamentals knowledge area (KA). It provides various perspectives on systems, including definitions, scope, and context.

This article provides a guide to some of the basic concepts of systems developed by systems science and discusses how these relate to the definitions to be found in systems engineering (SE) literature. The concept of an engineered system is introduced as the system context of critical relevance to SE.

Overview

In the System Fundamentals KA we will define some terms and ideas which are foundational to the understanding and practice of Systems Engineering (SE). In particular, a number of views of system are explored; these are summarized below and described in more detail with links to relevant references in the rest of this article.

- The INCOSE Fellows' Initiative on System and Systems Engineering Definitions ^[1] expresses that "**A system is an arrangement of parts or elements that together exhibit behavior or meaning that the individual constituents do not**". The premise of the system phenomena and the basis for system science is that there are theories, models and approaches that are transversal and useful to anyone trying to understand, create, transform or experience systems, independent of what the system is made of or the application domain considering it.
 - Systems can be either physical or conceptual, or a combination of both. Systems in the physical universe are composed of matter and energy, may embody information encoded in matter-energy carriers, and exhibit observable behavior. Conceptual systems are abstract systems of pure information, and do not directly exhibit
-

behavior, but exhibit “meaning”. In both cases, the system’s properties (as a whole) result, or emerge from: the parts or elements and their individual properties; AND the relationships and interactions between and among the parts, the system and its environment.

- Many of these common systems ideas relate to complex networks or hierarchies of related system elements. A **System Context is a set of system interrelationships associated with a particular system of interest (SoI) within a real-world environment**. One or more views of a context allow us to focus on the SoI but not lose sight of its broader, holistic relationships and influences. Context can be used for many kinds of system but is particularly useful for scoping problems and enabling the creation of solutions which combine people and technology and operate in the natural world. These are referred to as socio-technical system contexts.
- Systems Engineering is one of the disciplines interested in socio-technical systems across their whole life. This includes where problems come from and how they are defined, how we identify and select candidate solutions, how to balance technology and human elements in the wider solution context, how to manage the complex organizational systems needed to develop new solutions, and how developed solutions are used, sustained and disposed of. To support this, we define an **Engineered System as a socio-technical system which is the focus of a Systems Engineering life cycle**.
- While SE is focused on the delivery of an engineered system of interest, an **SE should consider the full Engineered System Context so that the necessary understanding can be reached and the right systems engineering decisions can be made across each Life Cycle**.

A General View of Systems

The idea of a system whole can be found in both Western and Eastern philosophy. Many philosophers have considered notions of holism, the concept that ideas, people or things must be considered in relation to the things around them to be fully understood (M’Pherson 1974). Broader still, the concept of holarchy, that everything in the universal is an organized whole (a system) and also at the same time a part of a, at least one, greater whole (Koestler 1967).

One influential systems science definition of a system comes from general system theory (GST):

A System is a set of elements in interaction. (Bertalanffy 1968)

The parts of a system may be conceptual organizations of ideas in symbolic form or real objects. GST considers **abstract systems** to contain only conceptual elements and **concrete systems** to contain at least two elements that are real objects, e.g. people, information, software, and physical artifacts, etc.

Similar ideas of wholeness can be found in systems engineering literature. For example:

We believe that the essence of a system is ‘togetherness’, the drawing together of various parts and the relationships they form in order to produce a new whole... (Boardman and Sauser 2008).

The cohesive interactions between a set of parts suggest a system boundary and define what membership of the system means. For **closed systems**, all aspects of the system exist within this boundary. This idea is useful for abstract systems and for some theoretical system descriptions.

The boundary of an **open systems** defines elements and relationships which can be considered part of the system and describes how these elements interact across the boundary with related elements in the environment. The relationships among the elements of an open system can be understood as a combination of the systems structure and behavior. The structure of a system describes a set of system elements and the allowable relationships between them. System behavior refers to the effects or outcomes produced when an instance of the system interacts with its environment. An allowable configuration of the relationships among elements is referred to as a system state. A stable system is one which returns to its original, or another stable, state following a disturbance in the environment. *System wholes entities often exhibit emergence, behavior which is meaningful only when attributed to the whole, not to its parts* (Checkland 1999).

The identification of a system and its boundary is ultimately the choice of the observer, the recognition of a system and to establish a relation with it is a decision made cognitively such as by a human actor or through affordances through physical relationships in the case of chemical substances. In the case of humans, this may be through observation and classification of sets of elements as systems, through an abstract conceptualization of one or more possible boundaries and relationships in a given situation, or a mixture of this concrete and conceptual thinking. This underlines the fact that any particular identification of a system by humans is a construct used by us to help make better sense of a set of things and to share that understanding with others if needed. It is important to recognize that this notion of observer is not limited to human perception. The observation and development of relationships between systems is a natural phenomenon inherent in all systems.

Many natural, social and man-made things can be better understood by recognizing the subjects of our interest as open systems as this takes into account this key dependency with the external, such as the stakeholder or actors with these systems. Additionally, one of the reasons we find the transversal nature of systems useful is that it is possible to identify shared concepts which apply to many system views. These recurring concepts or isomorphies can give useful insights into many situations, independently of the kinds of elements of which a particular system is composed. The ideas of structure, behavior, emergence and state are examples of such concepts. The identification of these shared system ideas is the basis for systems thinking and their use in developing theories and approaches in a wide range of fields of study the basis for system sciences.

Systems Engineering (SE), and a number of other related disciplines use systems concepts, patterns and models in the creation of useful outcomes or things. The concept of a network of open systems created, sustained and used to achieve a purpose within one or more environments is a powerful model that can be used to understand many complex real-world situations and provide a basis for effective problem solving within them.

System Context

Bertalanffy (1968) divided open systems into nine real-world types ranging from static structures and control mechanisms to socio-cultural systems. Other similar classification systems are discussed in the article Engineered Systems.

The following is a simple classification of system elements which we find at the heart of many of these classifications:

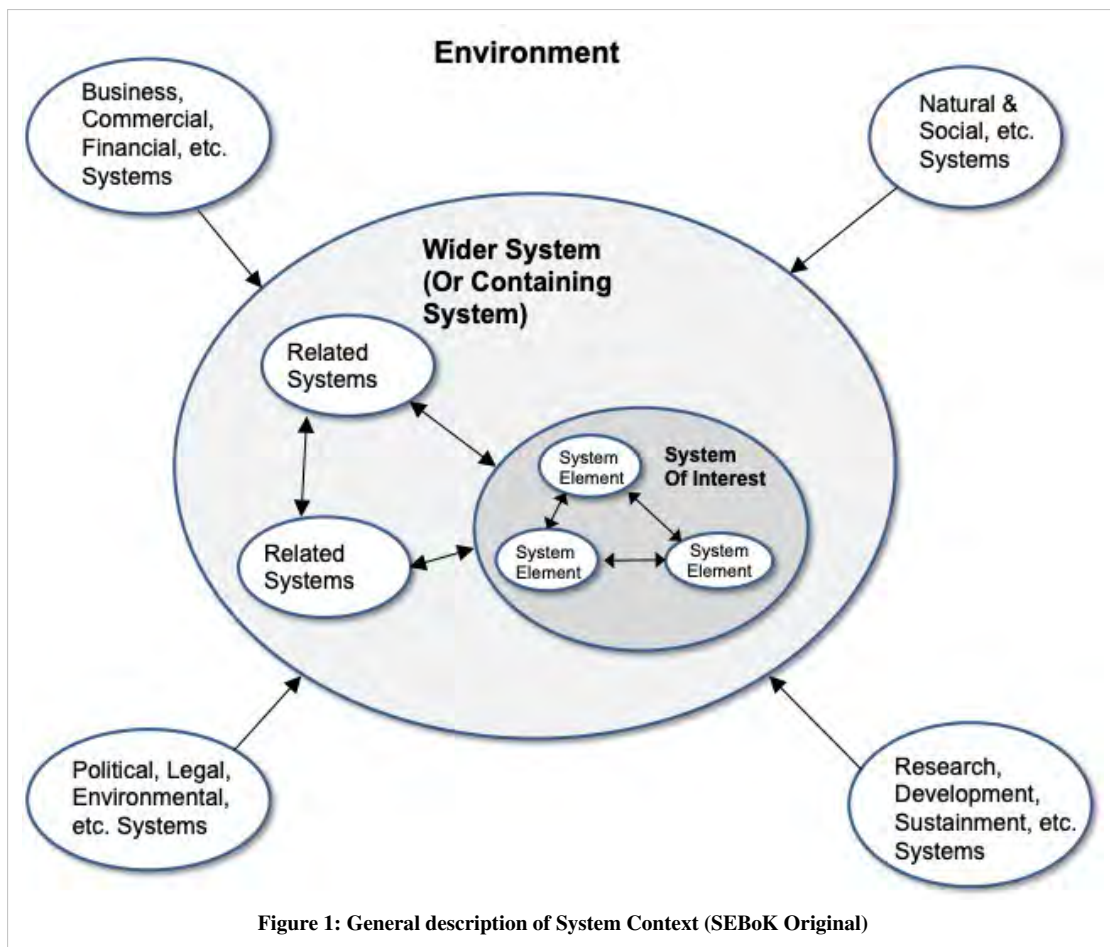
- Natural system elements, objects or concepts which exist outside of any practical human control. Examples: the real number system, the solar system, planetary atmosphere circulation systems.
- Social system elements, either abstract human types or social constructs, or concrete individuals or social groups.
- Technological System elements, man-made artifacts or constructs; including physical hardware, software and information.

While the above distinctions can be made as a general abstract classification, in reality there are no hard and fast boundaries between these types of systems: e.g., natural systems are operated by, developed by, and often contain social systems, which depend on technical systems to fully realize their purpose. Systems which contain technical and either human or natural elements are often called socio-technical systems. The behavior of such systems is determined both by the nature of the technical elements and by their ability to integrate with or deal with the variability of the natural and social systems around them.

Many of the original ideas upon which GST and other branches of system study are based come from the study of systems in the natural and social sciences. Many natural and social systems are initially formed as simple structures through the inherent cohesion among a set of elements. Once formed, they will tend to stay in this structure, as well as combine and evolve further into more complex stable states to exploit this cohesion in order to sustain themselves in the face of threats or environmental pressures. Such complex systems may exhibit specialization of elements, with elements taking on roles which contribute to the system purpose, but losing some or all of their separate identity outside the system. Such roles might include management of resources, defense, self-regulation or problem solving,

and control. Natural and social systems can be understood through an understanding of this wholeness, cohesion and specialization. They can also be guided towards the development of behaviors which not only enhance their basic survival, but also fulfill other goals of benefit to them or the systems around them. In *The Architecture of Complexity*, Simon (1962) has shown that natural or social systems which evolve via a series of stable “hierarchical intermediate forms” will be more successful and resilient to environmental change.

Thus, it is often true that the environment in which a particular system sits and the elements of that system can themselves be considered as open systems. It can be useful to consider collections of related elements as both a system and a part of one or more other systems. For example, a “holon” or system element was defined by Koestler as something which exists simultaneously as a whole and as a part (Koestler 1967). At some point, the nature of the relationships between elements within and across boundaries in a hierarchy of systems may lead to complex structures and emergent behaviors which are difficult to understand or predict. Such complexity can often best be dealt with not only by looking for more detail, but also by considering the wider open system relationships.



A system context describes all of the external elements which interact across the boundary of a particular system of interest (SoI) and a sufficient view of the elements within its boundary, to allow the SoI to be better understood as part of a wider systems whole. To fully understand the context, we also need to identify the environment in which the SoI and wider system sit and the systems in the environment which influence them.

Many man-made systems are designed as networks and hierarchies of related system elements to achieve desirable behaviors and the kinds of the resilience seen in natural systems. While such systems can be deliberately created to take advantage of system properties such as holism and stability, they must also consider system challenges such as complexity and emergence. Considering different views of a SoI and its context over its life can help enable this understanding. Considering systems in context allows us to focus on a SoI while maintaining the necessary wider, holistic systems perspective. This is one of the foundations of the Systems Approach described in SEBoK part 2,

which forms a foundation of systems engineering.

Systems and Systems Engineering

Some of the systems ideas discussed above form part of the systems engineering body of knowledge. Systems engineering literature, standards and guides often refer to “the system” to characterize a socio-technical system with a defined purpose as the focus of SE, e.g.

- “A system is a value-delivering object” (Dori 2002).
- “A system is an array of components designed to accomplish a particular objective according to plan” (Johnson, Kast, and Rosenzweig 1963).
- “A system is defined as a set of concepts and/or elements used to satisfy a need or requirement” (Miles 1973).

The International Council on Systems Engineering Handbook (INCOSE 2015) generalizes this idea, defining system as “an interacting combination of elements to accomplish a defined objective. These include hardware, software, firmware, people, information, techniques, facilities, services, and other support elements.” While these definitions cover the socio-technical systems created by SE, it is also necessary to consider the natural or social problem situations in which these systems sit, the social systems which developed, sustained and used them, and the commercial or public enterprises in which these all sit as systems (Martin 2004).

Hence, while many SE authors talk about systems and systems ideas, they are often based on a particular world view which related to engineered artifacts. It would also be useful to take a broader view of the context in which these artifacts sit, and to consider through life relationships as part of that context. To help promote this, the SEBoK will attempt to be more precise with its use of the word system, aligned with the work of the fellows initiative and distinguish between general systems principles and the specific socio-technical systems created by SE.

The term socio-technical system is used by many in the systems community and may have meanings outside of that relevant to SE. Hence, we will define an engineered system as a socio-technical system which forms the primary focus or system of interest (SoI) for an application of SE. A SE life cycle will consider an engineered system context, from initial problem formulation through to final safe removal from use (INCOSE 2015). A more detailed discussion of engineered system context and how it relates to the foundations of systems engineering practice can be found below.

Introduction to Engineered Systems

An engineered system defines a context containing both technology and social or natural elements, developed for a defined purpose by an engineering life cycle.

Engineered system contexts:

- are created, used and sustained to achieve a purpose, goal or mission that is of interest to an enterprise, team, or an individual.
- require a commitment of resources for development and support.
- are driven by stakeholders with multiple views on the use or creation of the system, or with some other stake in the system, its properties or existence.
- contain engineered hardware, software, people, services, or a combination of these.
- exist within an environment that impacts the characteristics, use, sustainment and creation of the system.

Engineered systems typically:

- are defined by their purpose, goal or mission.
 - have a life cycle and evolution dynamics.
 - may include human operators (interacting with the systems via processes) as well as other social and natural components that must be considered in the design and development of the system.
 - are part of a system-of-interest hierarchy.
-

Open systems are a useful way to understand many complex situations. Traditional engineering disciplines have become very good at building up detailed models and design practices to deal with the complexity of tightly integrated collections of elements within a technology domain. It is possible to model the seemingly random integration of lots of similar elements using statistical approaches. Systems Engineering makes use of both these aspects of system complexity, as discussed in the Emergence and Complexity article.

SE also considers the complexity of relatively small numbers of elements taken from a range of design disciplines together with people who may not always be experienced or have detailed training in their use. Such engineered systems may be deployed in uncertain or changing environments and be used to help people achieve a number of loosely defined outcomes. Relatively small changes in the internal working of these engineered systems' elements, or in how those elements are combined, may lead to the emergence of complex or un-expected outcomes. It can be difficult to predict and design for all such outcomes during an engineered system's creation, or to respond to them during its use. Iterative life cycle approaches which explore the complexity and emergence over a number of cycles of development and use are needed to deal with this aspect of complexity. The ways that system engineering deals with these aspects of complexity in the definition of life cycle and life cycle processes applied to an engineered system context is fully explored in Part 3

Life Cycle Definitions

As well as being a kind of system, an engineered system is also the focus of a life cycle and hence part of a commercial transaction. Historically,

Economists divide all economic activity into two broad categories, goods and services. Goods-producing industries are agriculture, mining, manufacturing, and construction; each of them creates some kind of tangible object. Service industries include everything else: banking, communications, wholesale and retail trade, all professional services such as engineering, computer software development, and medicine, nonprofit economic activity, all consumer services, and all government services, including defense and administration of justice.... (Encyclopedia Britannica 2011).

The following diagram defines some terms related to an engineered system life cycle and the development of goods (products) and services.

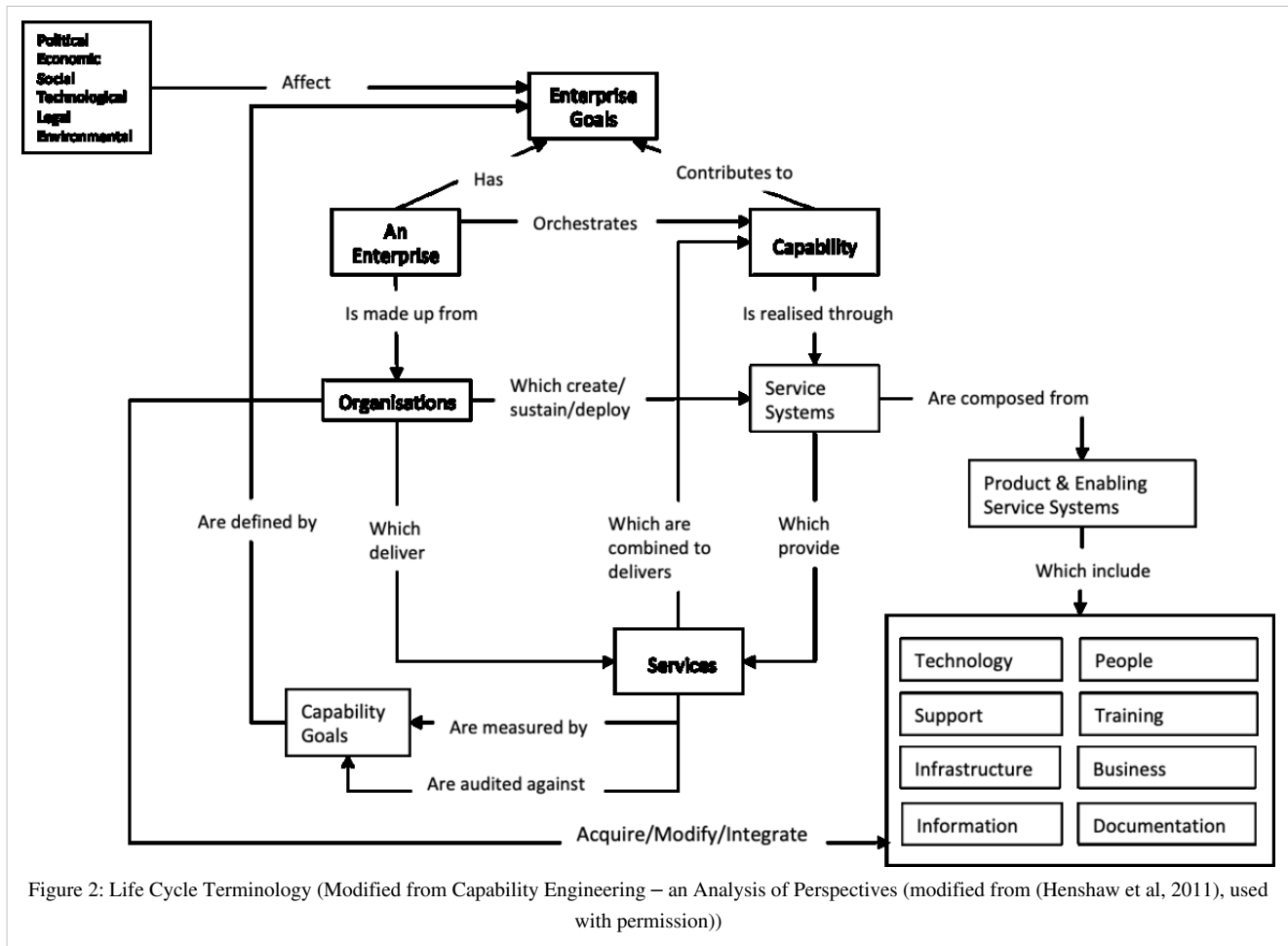
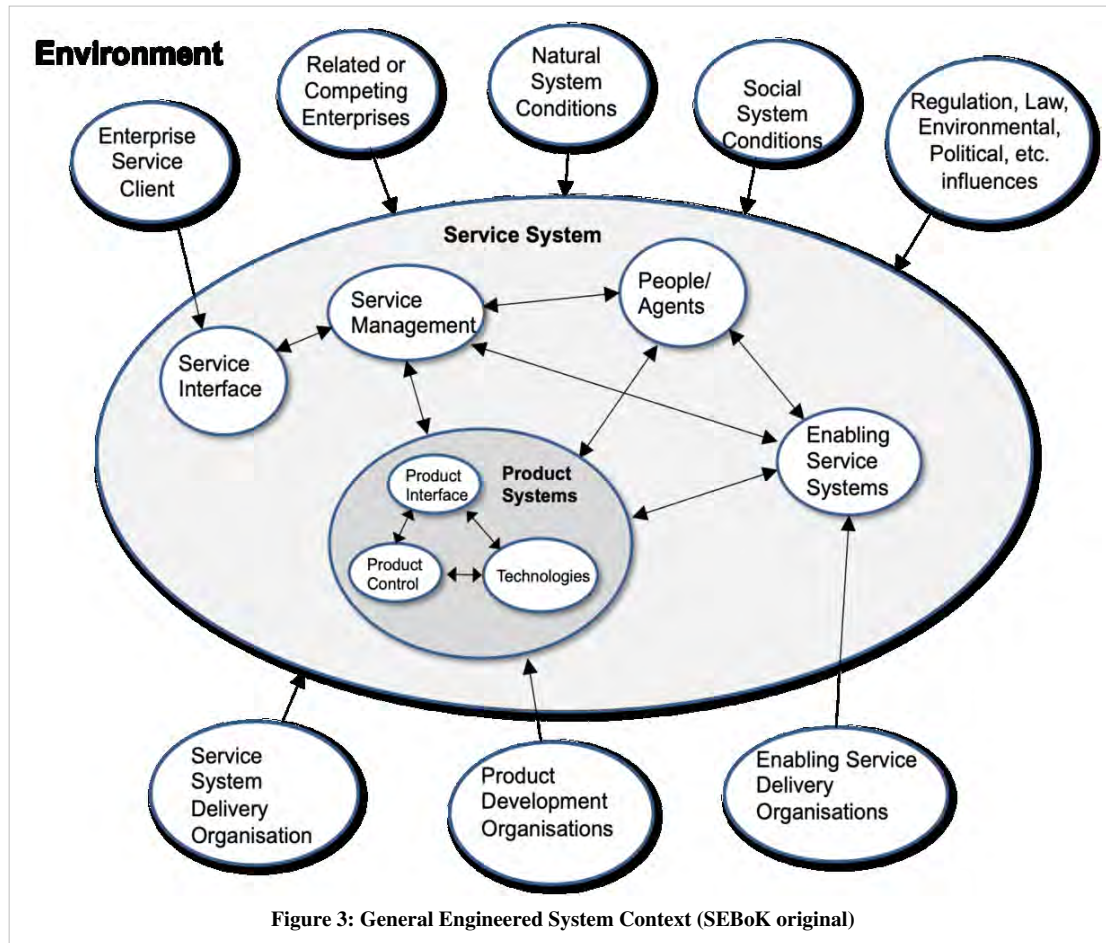


Figure 2: Life Cycle Terminology (Modified from Capability Engineering – an Analysis of Perspectives (modified from (Henshaw et al, 2011), used with permission))

In the above figure the capability needed to enable an enterprise to achieve its goals is delivered by the synchronized use of services. Those services are provided by a service system, which is created, sustained and deployed by one or more organizations. A service system is composed of people, technology, information, and access to related services and other necessary resources. Some of these resources are provided by enabling services and the technological elements may be developed and supplied as product systems. An enterprise system describes a collection of related capabilities and associated services which together enable the achievement of the overall purpose of an enterprise as a government, business or societal entity. Measurement and review of enterprise goals may define needs for change which require an organization to acquire or modify, and integrate the elements needed to evolve its service systems. The general terminology above is described briefly in the associated glossary definitions and expanded in related articles in Part 4: Applications of Systems Engineering.

Engineered System Context

Engineered systems are developed as combinations of products and services within a life cycle. The figure below gives a general view of the full context for any potential application of a SE life cycle.



In this view a service system related directly to a capability need sets the overall boundary. This need establishes the problem situation or opportunity which encapsulates the starting point of any life cycle. Within this service system are the related services, products and people (or intelligent software agents) needed to fully deliver a solution to that need. The environment includes any people, organizations, rules or conditions which influence or constrain the service system or the things within it. The SoI for a particular SE life cycle may be defined at any level of this general context. While the focus of the context will vary for each life cycle it is important that some version of this general context is considered for all SE life cycles, to help maintain a holistic view of problem and solution. This is discussed in Engineered Systems.

An engineered system context describes the context for a SoI so that the necessary understanding can be reached and the right systems engineering decisions can be made across the life of that SoI. This will require a number of different views of the context across a SE life cycle, both to identify all external influence on the SoI and to guide and constraint the systems engineering of the elements of the SoI. A full engineered systems context will include the problem situation from which a need for a SoI is identified, one or more socio technical solutions, the organizations needed to create and sustain new solutions and the operational environment within which those solutions must be integrated, used and eventually disposed. The kinds of views which can be used to represent a SoI context over its life and how those views can be combined into models is discussed in the Representing Systems with Models KA in Part 2. The activities which use those models are described conceptually in the Systems Approach Applied to Engineered Systems KA in part 2 and related to more formal SE life cycle processes in Part 3.

References

Works Cited

- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*, rev. ed. New York: Braziller.
- Boardman, J. and B. Sauser. 2008. *Systems Thinking: Coping with 21st Century Problems*. Boca Raton, FL, USA: Taylor & Francis.
- Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: Wiley and Sons, Inc.
- Dori, D. 2002. *Object-Process Methodology – A Holistic Systems Paradigm*. New York, NY, USA: Springer.
- Henshaw, M., D. Kemp, P. Lister, A. Daw, A. Harding, A. Farncombe, and M. Touchin. 2011. "Capability engineering – An analysis of perspectives." Presented at International Council on Systems Engineering (INCOSE) 21st International Symposium, Denver, CO, USA, June 20-23, 2011.
- Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4, pp. 59-63.
- INCOSE. 2015. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 4.0. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Johnson, R.A., F.W. Kast, and J.E. Rosenzweig. 1963. *The Theory and Management of Systems*. New York, NY, USA: McGraw-Hill Book Company.
- Koestler, A. 1990. *The Ghost in the Machine*, 1990 reprint ed. Sturgis, Michigan, USA: Penguin Group.
- Martin, J. 2004. "The seven samurai of systems engineering: Dealing with the complexity of 7 interrelated systems." Proceedings of the 14th Annual International Council on Systems Engineering International Symposium, 20-24 June, 2004, Toulouse, France, 20-24 June, 2004.
- Miles, R.F. (ed). 1973. *System Concepts*. New York, NY, USA: Wiley and Sons, Inc.
- M'Pherson, P.K. 1974. "A perspective on systems science and systems philosophy." *Futures*. Vol. 6, no. 3, pp. 219-39.
- Simon, H.A. 1962. "The architecture of complexity." *Proceedings of the American Philosophical Society*. Vol. 106, no. 6 (Dec. 12, 1962), pp. 467-482.

Primary References

- Bertalanffy, L., von. 1968. *General System Theory: Foundations, Development, Applications*, rev. ed. New York, NY, USA: Braziller.
- INCOSE. 2015. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 4.0. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Additional References

- Hybertson, Duane. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boca Raton, FL, USA: CRC Press.
- Hubka, Vladimir, and W. E. Eder. 1988. *Theory of Technical Systems: A Total Concept Theory for Engineering Design*. Berlin: Springer-Verlag.
- Laszlo, E., ed. 1972. *The Relevance of General Systems Theory: Papers Presented to Ludwig von Bertalanffy on His Seventieth Birthday*. New York, NY, USA: George Brazillier.
-

References

[1] <https://www.incose.org/about-systems-engineering/system-and-se-definitions>

Systems Engineering Core Concepts

- Lead Author:
- Gary Smith
- Contributing Author:
- Shingai Thornton

This article is part of the *Foundations of Systems Engineering* knowledge area. It integrates core **systems engineering** concepts, as presented in the SEBoK article *Systems Engineering Core Concepts*, with a general system model derived from **systems science**. The purpose is to enrich the conceptual foundation of systems engineering by showing how recent formal developments in systems science offer deeper theoretical structure to existing SE constructs. The integration is grounded in George Mobus's axiomatic model (Mobus 2022), which provides mathematical formalisms for systems structure and behavior. This article shows how these axioms can underpin and clarify SE concepts such as the System-of-Interest, System Context, Life Cycle Process, and *Emergence and Complexity*. It also provides a generalized system model that enables consistency between conceptual understanding and quantitative modeling.

Concepts are the fundamental building blocks for the construction of theories, principles, thoughts and beliefs. They play an important role in all aspects of cognition and communication.

Purpose and Uses of the Systems Engineering Concept Model

The Systems Engineering Concept Model (SECM) captures the concepts that are referred to in the Systems Engineering Body of Knowledge. The SECM provides a means to evaluate the consistency and coverage across the broad set of Systems Engineering concepts, and can facilitate communication to better understand and evolve these concepts. Although the primary reference for the SECM was the SEBoK, the concepts were cross-checked against other industry references including ISO/IEC/IEEE 15288:2015 and the INCOSE Systems Engineering Handbook version 5, enabling the SECM to be used to evaluate, understand, and evolve the concepts in these references as well. A small team developed the SECM to support the requirements for the next generation of the OMG Systems Modeling Language (OMG SysML™) to ensure SysML is consistent with the three leading industry standards.

SECM Approach

A concept model, called the Concept Map, was developed by the original SEBoK team prior to release of the SEBoK v1.0 in 2012 and was used to support integration of the initial concepts across the SEBoK topic areas. The Concept Map included a concept model and a mapping of the concepts to the glossary terms and to the sections of the SEBoK.

The SECM captures the Systems Engineering concepts and their relationship that are contained in today's SEBoK. The small subset of UML constructs and symbols, shown in Figure 1, are used to represent the SECM model. The choice of notations is intended to balance simplicity, understandability, and precision.

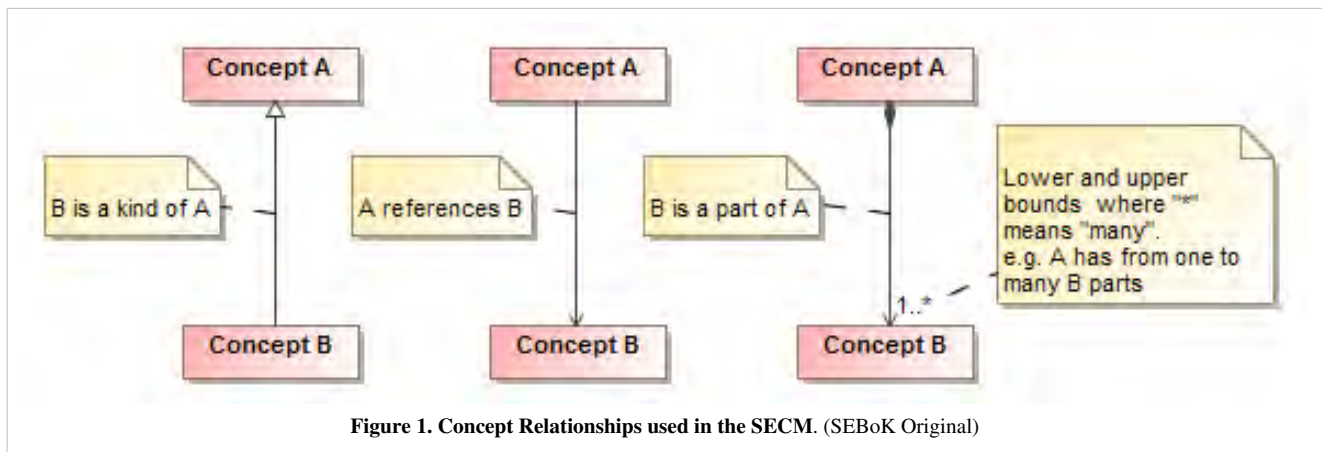
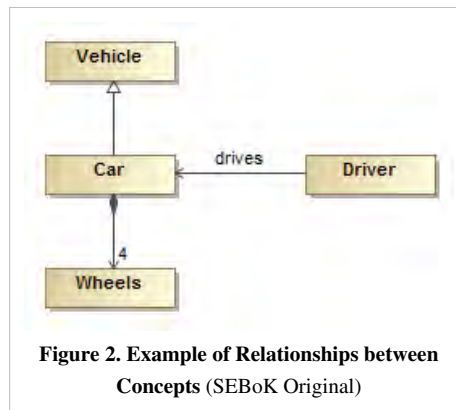


Figure 2 below shows a usage of these constructs and symbols when applied to a simple example of a Car. This diagram shows that a Car is kind of a Vehicle, and that the Driver drives the car (a reference relationship), and that there are four wheels that are parts of the Car.



The SECM is presented in a series of diagrams that generally represent concepts for particular knowledge areas and topics in the SEBoK, and also includes references to glossary terms in the SEBoK.

The approach used to capture the SECM content from the SEBoK is described in the following steps:

1. A topic within a SEBoK knowledge area is selected and the text is evaluated to identify key Systems Engineering concepts.
2. From the sentences containing the concepts, the subjects, i.e. the concepts, and the predicate, i.e. the relationship between concepts, are identified. The predicate is a statement that says something about the subject.
3. A search is conducted for this concept in other areas of the SEBoK, and the accompanying text is evaluated to further refine the concept and its relationships to other concepts.
4. The definition of the concept in the SEBoK glossary, if available, is evaluated.
5. The use and definition of this concept in the other two industry references is evaluated.
6. The concept and a derived definition from the above evaluation are added to the SECM.
7. The discovered relationships between the concepts are added to the model and to the relevant diagrams. The diagrams group related concepts that are often associated with a SEBoK Knowledge Area or Topic.
8. As new contributions are made to the SEBoK, this approach can be used to identify new concepts and relationships, and add them to the SECM.

Introduction to Core Concepts

The SECM was developed independently of the BKCASE project to support the evolution of the SysML standard. These models can be used to identify inconsistencies and/or areas requiring additional coverage within the SEBoK to support future updates.

The Core Concept Diagram shown in Figure 3 provides a high-level view of the some of the key concepts presented in the SEBoK. It should be emphasized that this figure is intended to be a representative interpretation of the current SEBoK without modifying or adding concepts, and no claim of the completeness of the SEBoK concepts is made. The colors correspond to logical groupings of concepts.

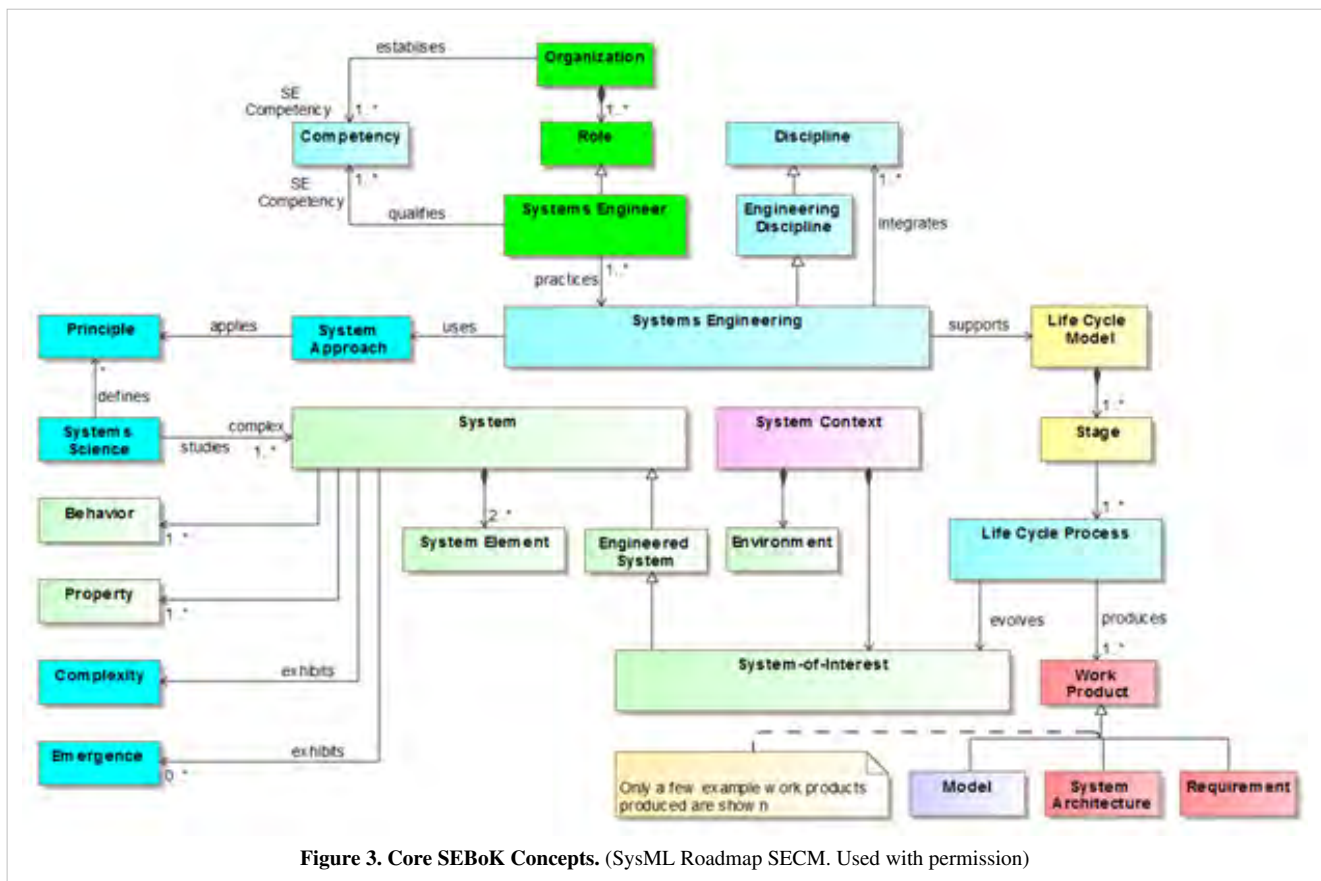


Figure 3. Core SEBoK Concepts. (SysML Roadmap SECM. Used with permission)

A brief description of the core concepts model is provided below.

A *Systems Engineer* is a role within an *Organization* that practices the *Engineering Discipline of Systems Engineering* (SE), and is qualified by a set of *SE Competencies*. *Systems Engineering* integrates other *Disciplines* to support the *Life Cycle Model*. The *Life Cycle Model* is composed of life cycle *Stages* that typically include conceptual, realization, production, support, utilization and retirement stages (not shown). Each life cycle *Stage* refers to *Life Cycle Processes* that produce various kinds of *Work Products*. The *Life Cycle Processes* evolve the *System-of-Interest* (SoI).

There are many kinds of systems, including natural systems, social systems, and technological systems (not shown). Systems that are created by and for people are referred to as *Engineered Systems*. An *Engineered System* whose life cycle is under consideration is referred to as a *System-of-Interest* (SoI).

A *System-of-Interest* is part of a broader *System Context*, which also includes an *Environment*. The *Environment* consists of other open systems (not shown) that can influence the SoI. Systems are composed of *System Elements* and have *Behavior* and *Properties*. Systems can exhibit *Emergence and Complexity* and *Emergence and Complexity*.

Systems Engineering uses a *System Approach* which applies established system *Principles*. *Systems Science* is an interdisciplinary field of science that studies complex systems and helps define and update the *Principles* that are

applied by the *System Approach*, which is used by the discipline of Systems Engineering.

Axiomatic Foundations from Systems Science

Mobus (2022) introduces a formal system model expressed as: $S_{i,l} = [C,N,G,B,T,H,\Delta t]_{i,l}$

This model includes:

- C: Components
- N: Internal network structure
- G: External interaction graph (context/environment)
- B: System boundary and interface
- T: Transformation functions (behavior)
- H: System history (state evolution)

These map to axioms that characterize system properties:

- Relationality:** Components interact through measurable flows, creating a network of quantifiable relationships.
- Hierarchy:** Systems decompose recursively into simpler subsystems until reaching atomic components.
- Context & environment:** Systems interact with their environment through defined interfaces and measurable flows.
- Emergence & holism:** System behaviors arise from component interactions and transformations across levels.
- Feedback:** Information and material flows create regulatory patterns through circular causation.
- Conservation & transformation:** Systems maintain and transform flows while preserving conservation laws.

Integrating SEBoK Concepts with Systems Science

The following table maps SEBoK concepts to Mobus’s axiomatic framework:

SEBoK Concept	Mobus Element	Description
System-of-Interest (SoI)	$S_{\{i,l\}}$	The engineered system modeled as a whole
System Elements	C	Components forming the system
Internal Relationships	N	Networked structure of interactions
Environment & Context	G	External systems and flow connections
Interfaces & Boundaries	B	Delineated inputs/outputs across system edges
System Behavior	T	Transformation of component states and flows
Lifecycle & History	H, Δt	Evolution and traceability over time
Emergence and Complexity	T, H, hierarchy	Outcomes of internal interactions and nested systems

This alignment enhances both theoretical rigor and modeling precision, complementing the SECM's conceptual focus with mathematical structure.

A General System Model for Systems Engineering

The general system model such as that provided by Mobus could provide scientific underpinning for:

Unified structural and behavioral modeling

Quantitative basis for feedback and emergence

Compatibility with systems modeling languages (e.g., SysML)

Support for lifecycle traceability and transformation

For example, in an engineered mobility system:

C: Subsystems (powertrain, sensors, chassis)

N: Wiring and data bus interconnections

G: Urban infrastructure, regulations, users

B: Interfaces such as GPS, brakes, communications

T: Acceleration, routing, braking logic

H: System logs, performance metrics

Δt : Millisecond-level telemetry samples

Implications for Systems Engineering Practice

By grounding SE concepts in a general systems science model, systems engineering would gain:

A formalized system context for MBSE

Improved capability to represent feedback, hierarchy, and emergence

Alignment with system science education and interdisciplinary fields

References

Works Cited

Mobus, G. E. (2022). *Systems Science: Theory, Analysis, Modeling, and Design*. Springer Nature.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions/Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

INCOSE. 2015. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 4.0. Hoboken, NJ, USA: John Wiley and Sons, Inc., ISBN: 978-1-118-99940-0.

OMG. *SysML Roadmap: Systems Engineering Concept Model Workgroup (SECM)* Available at: https://www.omgwiki.org/OMGSysML/doku.php?id=sysml-roadmap%3Asystems_engineering_concept_model_workgroup.

Primary References

ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions/Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc.

Additional References

None

Systems Engineering Principles

- Lead Author:
- Michael Watson
- Contributing Authors:
- Bryan Mesmer, Garry Roedler, David Rousseau, Chuck Keating, Rob Gold, Javier Calvo-Amodio, Cheryl Jones, William D. Miller, Scott Lucero, Aileen Sedmak, and Art Pyster

Every discipline has a set of underlying principles which fundamentally characterize it. Discipline knowledge and practice are either explicitly or implicitly derived from its principles. For example, one of the principles of physics is that the speed of light is constant. This specific principle has been validated countless times in practice and is broadly accepted as how the universe works. One common economic principle is that “Every choice has an opportunity cost”. (University of Minnesota n.d.) Another is the principle of supply, which states that “The quantity of a good supplied rises as the market price rises and falls as the price falls”. (Ehrbar 2007) Generally, disciplines do not have one authoritative complete set of principles. Different authors emphasize different aspects of a discipline in the principles they identify and explain. As fields mature, new principles emerge, and old principles may change. For example, Einstein’s Theory of Relativity revealed that the principles underlying Newtonian physics were not absolutely true. Yet, they are still immensely valuable most of the time.

Introduction

Systems engineering is a young discipline. The emergence of a set of systems engineering principles occurred over the past 30 years within the discipline. The development of these principles derives from many sources including heuristics Systems Engineering Heuristics (i.e., experience based), sociology, physical sciences, and mathematics. The INCOSE Systems Engineering Principles Action Team (SEPAT) reviewed various sources of systems principles and systems engineering principles identified in literature over this period. This article surveys this work and identifies a set of systems engineering principles based on this review.

Systems engineering principles are a form of guidance proposition which provide guidance in application of the systems engineering processes and a basis for the advancement of systems engineering. Systems engineering has many kinds of guidance propositions that can be classified by their sources, e.g. heuristics (derived from practical experience), conventions (derived from social agreements), values (derived from cultural perspectives), and models (based on theoretical mechanisms). Although these all support purposeful judgement or action in a context, they can vary greatly in scope, authority, and conferred capability. They can all be refined, and as they mature, they gain in their scope, authority, and capability, while the set becomes more compact. A key moment in their evolution occurs with gaining insight into why they work, at which point they become principles. Principles can have their origins associated in referring to them as “heuristic principles”, “social principles”, “cultural principles”, and “scientific principles”, although in practice it is usually sufficient to just refer to principles. Systems engineering principles are derived from principles of these various origins providing a diverse set of transcendent principles based on both practice and theory (Rousseau, Pennotti, & Brook 2022).

System principles differ from *systems engineering* principles differ in important ways (Watson 2018a). System principles address the behavior and properties of all kinds of systems, looking at the scientific basis for a system and characterizing this basis in a system context via specialized instances of a general set of system principles. SE principles are a specialized and contextualized instantiation of systems principles that address the approach to the realization, use, and retirement of systems. SE principles build on systems principles that are general for all kinds of

systems (Rousseau, 2018a) and for all kinds of human activity systems (Senge 1990, Calvo-Amodio & Rousseau 2019). Hence, system principles guide the definition and application of SE processes – a strong systems engineer must be the master of both system principles and SE principles.

A number of people have proposed SE underlying principles, in part by building on system principles. Perhaps because of its youth, a consensus among the community is still developing on which SE principles are most central. Yet, it is valuable to examine a number of proposed SE principles as they offer insight into what some of the best minds in the field think are fundamental to the discipline. In reviewing various published SE principles, a set of criteria emerged for valid SE principles. A principle:

- transcends a particular lifecycle model or phase
- transcends system types
- transcends a system context
- informs a world view on SE
- is not a “how to” statement
- is supported by literature or widely accepted by the community; i.e. has proven successful in practice across multiple organizations and multiple system types
- is focused, concise, and clearly worded

Thus, system type, the context in which the system is developed and operated, or a life cycle phase do not narrowly define systems engineering principles. Systems engineering principles transcend these system characteristics and inform a worldview of the discipline. Principles are not “how to” statements, which are embodied in the processes, but provide guidance in making decisions in applying the systems engineering processes. Principles should have a strong reference basis supported by literature, or widely accepted in practice (i.e. heuristic basis) (keeping in mind that this success must transcend the system characteristics mentioned above), or both. Principles are focused, concise, and clear in well-constructed principles statements.

Literature currently contains several good articles on system principles. These principles provide a basis for the functioning of a system and seek to group scientific axioms, laws, and principles into a set of system principles. The main themes seen in the literature on system principles include system governance, system theory axioms, and system pathologies with a focus on complex systems and system of systems. Complex System Governance provides a set of nine Metasystem functions “To provide control, communication, coordination, and integration of a complex system”. These functions provide a basis from which to understand the functions of complex systems and how to manage their acquisition (i.e., governance). (Keating et al. 2017a) These Metasystem functions also extend to systems of systems engineering (Keating et al. 2017b).

Advances in system theory produced a set of unified propositions stated as seven axioms “From which all other propositions in systems theory may be induced”. These seven axioms map to 30 scientific laws and principles (Adams et al. 2014). These axioms focus on the scientific basis of systems. Further work on these axioms provides an integration construct and a slightly different mapping to the underlying scientific laws and principles (Whitney et al. 2015). This work provides a strong integration and advancement in system theory, focusing on the principles behind the scientific basis of a system.

System science approaches also incorporate system theory leading to 10 concepts of systems theory and systems thinking (Sillitto 2014). These 10 concepts focus on system principles providing a definition of system characteristics. A further development in system sciences produced a list of 12 systems sciences principles that also focus on the characteristics of systems (Mobus and Kalton 2015). Rousseau formally derived a statement and derivation of three principles of systems (Rousseau 2018a). In addition, an architecture of systemology and typology of system principles provides a good classification of scientific principles spanning from system philosophy through system practice (Rousseau 2018b). This work led to a framework for understanding system science principles (Rousseau 2018c).

Other early work included a set of seven system science principles exhibited by systems (Hitchins 1992). Organizational principles were also defined as a set of 11 principles dealing with how to work successfully within an organization (Senge 1990). Principles of Systems Thinking describes a set of 20 system thinking principles captured and integrated from a variety of sources.

System pathologies is another interesting approach to understand “Circumstances that act to limit system performance or lessen system viability (continued existence) and as such they reduce the likelihood of a system meeting performance expectations”. These pathologies define diagnostics for understanding systems derived from a set of 45 system laws and principles. (Katina et al. 2016)

INCOSE compiled an early list of principles. This list consisted of 8 principles and 61 sub-principles (Defoe 1993). These principles were important considerations in practice for the success of system developments and ultimately became the bases for SE processes. These principles are reflective of how SE works in general. Following this work, several early versions of SE principles were compiled leading up to one of the first documented sets of SE processes. Project Performance International (Halligan 2019) has a set of SE principles that follow along the model set by Defoe providing considerations in the practice of SE, focusing on specific aspects within life cycle phases.

The Korean Council on Systems Engineering provided a survey article of 8 works on SE principles spanning the time from Defoe’s principles through 2004 (Han 2004), including an early version of the PPI principles. These 8 works showed evolution of systems engineering principles from practice focused to more transcendent focused principles. In 1997, the INCOSE Systems Engineering Principles Working Group (no longer active) generated a set of 8 principles building from the work of Defoe over the course of several years of discussions. These principles were a mixture of process basis, modeling guidelines, and an early world view of the SE focus. The Institute of Electrical Engineers (IEE 2000), now part of the Institution of Engineering and Technology (IET), produced a set of 12 principles that also provided some basis for the systems engineering processes which are no longer extant. Lawrence Berkley National Laboratory (LNBL 2001) produced a set of systems engineering principles that embody the concepts captured by the INCOSE SE processes. In England, the Defence Engineering Group (DEG 2002) produced an SE Handbook with a brief set of principles guiding their processes and capturing some aspects of systems principles. Iowa State University is reported to have produced an SE Student Handbook containing a short list of SE heuristic phrases stated as principles. The KCOSE paper also referenced a lecture on SE principles from a course at the University of Southern California (USC) (Jackson, 2003). This lecture defined a principle as “A statement or generalization of a truth reflected in the systems engineering process”, showing the focus on processes in the early SE principle development.

Some early forms of SE principles were also contained in textbooks on complex system development. (Adamsen II 2000) This set of principles assume a hierarchical system representation (complex systems have since shown to be more networks than hierarchies) and include statements on SE processes. Finally, system architecting books also included some early SE heuristics (Maier and Rechtin 2002). These heuristics read as sayings about some aspect of systems engineering practice.

The KCOSE Technical Board reviewed these 8 sources and voted that 8 of the principles from these sources as a set of SE principles, leading to an early form of transcendent principles consistent with the criteria defined above. These sources all show the early evolution stages of the SE principles as people looked at both formal and informal (i.e., course notes and student handbooks) sources to try to understand SE principles. The definition of the SE processes in works such as the INCOSE Systems Engineering Handbook fulfilled some of the objectives of these early works on SE principles and consolidated a lot of the work in this area. Recently, the need for more transcendent SE principles has been recognized, as a guide for applying the processes, which is the focus of current work by the SEPAT.

Over the last several years, a fresh look at the set of SE principles has emerged from the SEPAT. (Watson, et al. 2019) (INCOSE 2022) Its effort is based on SE postulates, principles, and hypotheses from the NASA Systems Engineering Research Consortium. This consortium followed the approach of Ludwig Boltzmann in defining his postulates on gas distribution laws. Boltzmann’s work is an early example of how to characterize the interactions of

complex systems. A postulate is something assumed without proof to be true, real, or necessary. (Webster 1988) This led to the articulation of a set of postulates and hypotheses underlying SE which were expanded into a proposed set of SE principles. The underlying SE postulates and hypotheses matured over the course of 4 years (Watson et al. 2014; Watson et al. 2015; Watson & Farrington 2016). As the postulates matured so did the SE principles, providing more specifics in the application of SE, and the proof of a hypothesis becoming a principle (Watson et al. 2018; Watson 2018b). The final version of the principles is contained in a NASA Technical Publication on theory of systems engineering (Watson, et. al. 2020a) and a NASA Technical Publication on application of systems engineering approaches in practice. (Watson, et. al. 2020b)

The SEPAT developed 15 principles and 3 hypotheses, some expanded by subprinciples, described in INCOSE (2022). These were peer reviewed by several professional societies and represent the first steps towards consensus on systems engineering principles. These principles are:

1. SE in application is specific to stakeholder needs, solution space, resulting system solution(s), and context throughout the system life cycle.
2. SE has a holistic system view that includes the system elements and the interactions amongst themselves, the enabling systems, and the system environment.
3. SE influences and is influenced by internal and external resources, and political, economic, social, technological, environmental, and legal factors.
4. Both policy and law must be properly understood to not overly constrain or under-constrain the system implementation.
5. The real system is the perfect representation of the system. (models are only representations of real systems)
6. A focus of SE is a progressively deeper understanding of the interactions, sensitivities, and behaviors of the system, stakeholder needs, and its operational environment.
7. Systems Engineering addresses changing stakeholder needs over the system life cycle.
8. SE addresses stakeholder needs, taking into consideration budget, schedule, and technical needs, along with other expectations and constraints.
9. SE decisions are made under uncertainty accounting for risk.
10. Decision quality depends on knowledge of the system, enabling system(s), and interoperating system(s) present in the decision-making process.
11. SE spans the entire system life cycle.
12. Complex systems are engineered by complex organizations.
13. SE integrates engineering and scientific disciplines in an effective manner.
14. SE is responsible for managing the discipline interactions within the organization.
15. SE is based on a middle range set of theories.

The SEPAT's recent articulation of SE principles elaborates on points made earlier by Defoe and emphasizes additional aspects of current SE practices, but there is nothing inconsistent between the two sets.

Principles of SE such as those proposed by Defoe and more recently articulated by the SEPAT are domain independent; i.e. they apply independent of the type of system being built, whether it is for transportation, healthcare, communication, finance, or any other business or technical domain. As they are applied, these principles can take more specialized forms, and/or can be complemented by other context-specific principles. Indeed, general SE principles such as these have been successfully applied in virtually every domain.

References

Works Cited

- Adams, K. M., Hester, P. T., Bradley, J. M., Meyers, T. J., & Keating, C. B. 2014. *Systems Theory as the Foundation for Understanding Systems*. *Systems Engineering*, vol. 17, no. 1, pp. 112–123.
- Adamsen II, Paul B. 2000. *A Framework for Complex System Development*, Chapter 7. Boca Raton, FL, USA: CRC Press.
- Calvo-Amodio, J., & Rousseau, D. 2019. "The Human Activity System: Emergence from Purpose, Boundaries, Relationships, and Context," *Procedia Computer Science*, vol. 153, pp. 91-99.
- Cutler, William. 1997. Presented at INCOSE Principles WG Session, Tuesday, August, 5th, 1997, Los Angeles, CA
- Defoe, J.C., Ed. 1993. *National Council on Systems Engineering: An Identification of Pragmatic Principles, Final Report*. SE Practice Working Group. Subgroup on Pragmatic Principles. Bethesda, MD, USA: NCOSE WMA Chapter.
- DEG (Defence Engineering Group). 2002. *The Defence Systems Engineering Handbook*, London, UK: University College London. pg. 12.
- Edwards, B., Ed. 2001. *A Systems Engineering Primer for Every Engineer and Scientist*. Berkeley, CA, USA: Lawrence Berkeley National Laboratory. pp. 6-7.
- Ehrbar, A. 2007. "Supply," in *The Concise Encyclopedia of Economics*. The Library of Economics and Liberty, [online document]. Available: The Library of Economics and Liberty ^[1]. Accessed May 14, 2020.
- Jackson, S. 2003. "Principles of Systems Engineering" from the lecture note used in the course called Systems Engineering Theory and Practice at University of Southern California.
- Han, M-D. 2004. "Systems engineering principles revisited." Proceedings of the 14th INCOSE International Symposium, Session 6 Track 2: Researching SE Methodologies & Approaches in SE Research, Toulouse, France, June 20-24, 2004.
- Halligan, Robert. 2019. Project Performance International Systems Engineering, Systems Engineering Principles.
- Hitchins, D. 1992. *Putting Systems to Work*. Chichester, UK: John Wiley & Sons. pp. 60–71.
- INCOSE. 2022. International Council on Systems Engineering Systems Engineering Principles Publication.
- Katina, P. F. 2016. "Systems theory as a foundation for discovery of pathologies for complex system problem formulation," in *Applications of Systems Thinking and Soft Operations Research in Managing Complexity*. Cham, Switzerland: Springer. pp. 227–267.
- Keating, C. B., Katina, P. F., Jaradat, R., Bradley, J. M. and Gheorghe, A. V. 2017. "Acquisition system development: A complex system governance perspective," *INCOSE International Symposium*, vol. 27, pp. 811–825. doi:10.1002/j.2334-5837.2017.00395.x
- Keating, C. B., Katina, P. F., Gheorghe, A. V. and Jaradat, R. 2017. Complex System Governance: Advancing Prospects for System of Systems Engineering Applications.
- Meir, M. W. and Rechtin, E. 2002. "Appendix A: Heuristics for systems-level architecting," in *The Art of Systems Architecting*, 2nd ed. Boca Raton, FL, USA: CRC Press.
- Mobus, G. E., & Kalton, M. C. 2015. *Principles of Systems Science*. New York, NY, USA: Springer. pp. 17–30.
- Neufeldt, V. and Guralnik, D. B., Eds. 1988. *Webster's New World Dictionary*, Third College Edition. New York, NY, USA: Simon & Schuster. pg. 1055.
- Oxford College of Marketing. 2016. "What is a PESTEL analysis?" Available: <https://blog.oxfordcollegeofmarketing.com/2016/06/30/pestel-analysis/>. Accessed May 2, 2022.

- Rousseau, D. 2018a. "Three general systems principles and their derivation: Insights from the philosophy of science applied to systems concepts," in Madni et. al., Eds., *Disciplinary Convergence in Systems Engineering Research*. Cham, Switzerland: Springer. pp. 665–681.
- Rousseau, D. 2018b. "On the architecture of systemology and the typology of its principles." *Systems*, vol. 6, no. 1, pg. 7.
- Rousseau, D. 2018c. "A framework for understanding systems principles and methods." Proceedings of the INCOSE International Symposium, Washington, DC, USA, July 7–12, 2018.
- Rousseau, D., Pennotti, M., Brook, P. (2022). *Systems Engineering's Evolving Guidelines*. Report of the INCOSE Bridge Team, presented to the INCOSE Systems Science Working Group on January 31, 2022. Available: <https://drive.google.com/file/d/1JibL44sUh0ztefZQ5Rfy4kGiodXIy63n/view>. Accessed March 30, 2022.
- Senge, P. M. 1990. *The Fifth Discipline: The Art and Practice of the Learning Organization*. London, UK: Random House.
- Sillitto, H. 2014. *Architecting Systems. Concepts, Principles and Practice*. London, UK: College Publications. pp. 33–38.
- University of Minnesota. N.D. Available: <https://open.lib.umn.edu/macroeconomics/chapter/1-1-defining-economics/#:~:text=Every%20choice%20has%20an%20opportunity,forgone%20in%20making%20that%20choice.> . Accessed May 2, 2022.
- Watson, M.D. 2018a. "Engineering elegant systems: Postulates, principles, and hypotheses of systems engineering," AIAA Complex Aerospace Systems Exchange (CASE) 2018, Future of Systems Engineering Panel, Orlando, FL, September 2018.
- Watson, M.D. 2018b. "Engineering elegant systems: Systems engineering postulates, principles, and hypotheses related to systems principles," Proceedings of the International Society for the Systems Sciences, Corvallis, OR, July 2018.
- Watson, M.D., B.L. Mesmer, G. Roedler, D. Rousseau, R. Gold, J. Calvo-Amodio, C. Jones, W.D. Miller, D., D. Long, S. Lucero, R.W. Russell, A. Sedmak, and D. Verma. 2019. "Systems engineering principles and hypotheses", *INCOSE INSIGHT Magazine*, vol. 21, no. 1, pp. 18-28.
- Watson, M.D., S. Long, E-H Ng, and C. Downings ed. 2014. "Building a path to elegant design," Proceedings of the American Society for Engineering Management 2014 International Annual Conference, Virginia Beach, Virginia, USA, October 15-18, 2014.
- Watson, M.D. and P.A. Farrington. 2016. "NASA systems engineering research consortium: Defining the path to elegance in systems", Proceedings of the 2016 Conference on Systems Engineering Research, Huntsville, AL, USA, Mar 22-24, 2016.
- Watson, M.D., B.L. Mesmer, B. and P.A. Farrington. 2018. "Engineering elegant systems: Postulates, principles, and hypotheses of systems engineering", Proceedings of the 16th Conference on Systems Engineering Research, Charlottesville, VA, USA, May 2018.
- Watson, M.D., B.L. Mesmer, and P.A. Farrington ed. 2020a. NASA Systems Engineering Research Consortium: "Engineering Elegant Systems: Theory of Systems Engineering," NASA/TP–20205003644, NASA Marshall Space Flight Center, Huntsville, AL.
- Watson, M.D., B.L. Mesmer, and P.A. Farrington. ed. 2020b. NASA Systems Engineering Research Consortium: "Engineering Elegant Systems: The Practice of Systems Engineering," NASA/TP–20205003646, NASA Marshall Space Flight Center, Huntsville, AL.
- Whitney, K., J.M. Bradley, D.E. Baugh, and C.W. Chesterman. 2015. "Systems theory as a foundation for governance of complex systems," *International Journal of System of Systems Engineering*, vol. 6, nos. 1–2, pp.

15–32.

Primary References

Defoe, J.C., Ed. 1993. *National Council on Systems Engineering: An Identification of Pragmatic Principles, Final Report*. SE Practice Working Group. Subgroup on Pragmatic Principles. Bethesda, MD, USA: NCOSE WMA Chapter.

Han, M-D. 2004. “Systems Engineering Principles Revisited.” Proceedings of the 14th INCOSE International Symposium, Session 6 Track 2: Researching SE Methodologies & Approaches in SE Research, Toulouse, France, June 20-24, 2004.

Rousseau, D. 2018b. “On the Architecture of Systemology and the Typology of its Principles.” *Systems*, vol. 6, no. 1, pg. 7.

Rousseau, D. 2018c. “A Framework for Understanding Systems Principles and Methods.” Proceedings of the INCOSE International Symposium, Washington, DC, USA, July 7–12, 2018.

Watson, M.D., B.L. Mesmer, G. Roedler, D. Rousseau, R. Gold, J. Calvo-Amodio, C. Jones, W.D. Miller, D., D. Long, S. Lucero, R.W. Russell, A. Sedmak, and D. Verma. 2019. “Systems Engineering Principles and Hypotheses”, *INCOSE INSIGHT Magazine*, vol. 21, no. 1, pp. 18-28.

Additional References

None.

References

[1] <https://www.econlib.org/library/Enc/Supply.html>

Systems Engineering Heuristics

Heuristics provide a way for an established profession to pass on its accumulated wisdom. This allows practitioners and others interested in how things are done to gain insights from what has been found to work well in the past and to apply the lessons learned. Heuristics will usually take the form of short expressions in natural language. These can be memorable phrases encapsulating rules of thumb, shortcuts or "words to the wise", giving general guidelines on professional conduct or rules, advice, or guidelines on how to act under specific circumstances. Common heuristics do not summarize all there is to know, yet they can act as useful entry points for learning more. This article overviews heuristics in general as well as some of those specifically supporting Systems Engineering practice.

Overview

Heuristics have always played an important part in the history of engineering and shaped its progress, especially before science developed to the point when it could also assist engineers. Systems Engineering is still at a stage at which there is no sufficiently reliable scientific basis for many of the systems being built, which has triggered a renewed interest in heuristics to fill the gap. This is especially true as the practice of Systems Engineering is extended to provide solutions to inherently complex, unbounded, ill-structured, or "wicked" problems (Churchman 1967).

Using heuristics does not guarantee success under all circumstances, but usefulness of a heuristic can be maximized if the known extent of its applicability is made clear. At their best, heuristics can act as aids to decision making, value judgements, and assessments.

Heuristics have the potential to be useful in a number of ways:

- reduce the amount of thinking (or computation) needed to make a good decision or a choice
- help in finding an acceptable solution to a problem
- identify the most important factors to focus on while addressing a complex problem
- improve the quality of decisions by drawing on best practices
- avoid repeating avoidable mistakes
- act as an entry point to wider knowledge of what has been found to work

Historical Background

Engineering first emerged as a series of skills acquired while transforming the ancient world, principally through buildings, cities, infrastructure, and machines of war. Since then, mankind has sought to codify the knowledge of "how to." Doing so allows each generation to learn from its predecessors, enabling more complex structures to be built with increasing confidence while avoiding repeated real-world failures. Setting out the aims for engineering in the 1st Century BCE, Vitruvius proposed a set of enduring principles: Strength, Utility and Beauty. He provided many examples of their applications to the fields of engineering of the time.

Vitruvius' writings were rediscovered in the Middle Ages, forming the basis of the twin professions of architecture and engineering. Early cathedral builders encapsulated their knowledge in a small number of rules of thumb, such as: "maintain a low center of gravity," "put 80% of the mass in the pillars," and "observe empirical ratios between cross-section and span for cross-members." Designs were conservative, with large margins, the boundaries of which were largely unknown. Numerous excellent structures resulted, many of which have endured to this day. When the design margins were exceeded, for example out of a desire to build higher and more impressive structures, a high price could be paid, with the collapse of a roof, a tower, or even a whole building. From such failures, new empirical rules emerged. Much of this took place before the science behind the strength of materials or building secure foundations was understood. Only in recent times has computer simulation revealed the contribution towards certain failures played by dynamic effects, such as those of wind shear on tall structures.

Since then, engineering and applicable sciences have co-evolved: science providing the ability to predict and explain performance of engineered artifacts with greater assurance, and engineering developing new and more complex systems, requiring new scientific explanations and driving research agendas. In the modern era, complex and adaptive systems are being built which challenge conventional engineering sciences, with the builders turning to social and behavioral sciences, management sciences, and increasingly systems science to deal with some of the new forms of complexity involved and to guide the profession accordingly.

Koen (1985), went further, and argued that the whole of engineering was essentially heuristic in nature, since engineers could never know everything about whether what they built would meet their original intent, and how it would interact with an uncertain world. According to Koen, we must do what we can to tie these things down but make allowances for what we cannot know. He defined the engineering method as: *the strategy for causing the best change in poorly understood or uncertain situation within available resources*. He further claimed that heuristics are the only way of guiding our actions in these circumstances, and that the set of all heuristics agreed or used by a profession at any one time represents its 'state of the art'. He also put forward a number of universal heuristic principles for the conduct of engineering, covering areas such as: rules of thumb, factors of safety, engineers' attitude towards their work, managing risk and allocating resources. Although much of what Koen (a chemical engineer by profession) had to say remains relevant to systems engineers today, there is renewed optimism that scientific SE principles can be uncovered to underpin the discipline, just as they have done through the ages. (see Systems Engineering Principles)

Modern Interest

Renewed interest in the application of heuristics to the field of Systems Engineering stems from the seminal work of Rechlin and Maier (2009). Their book remains the best single repository of such knowledge, although efforts are now under way within INCOSE to update them for the 21st Century. Their motivation was to provide guidance for the emerging role of system architect as the person or team responsible for coordinating engineering effort towards devising solutions to complex problems and overseeing their implementation. Rechlin and Maier observed that it was in many cases better to apply "rules of thumb" than to attempt detailed analysis, especially when this was precluded by the number of variables involved, the complexity of the interactions between stakeholders, and the internal dynamics of system solutions and the organizations responsible for their realization.

An argument in favor of the wider use of heuristics was also made by Mervyn King (2016), who was Governor of the Bank of England during the 2008 global financial crash. Looking back, he said "it is better to be roughly right than precisely wrong": banks which observed the old bankers' rule of maintaining capital assets equal to 70% of their loan book survived, while those who relied on complex (and flawed) mathematical models of derivatives failed. He has become a powerful advocate for the use of heuristics alongside formal economics to allow bankers and others to deal with the uncertainties of global financial affairs in the modern interconnected world.

Further backing for the contemporary use of heuristics comes from Simon (1957), who coined the term "satisficing" for a situation in which people seek solutions, or accept choices or judgments, that are "good enough" for their purposes, regardless of whether they can be further optimized by precise analysis. He made the point that some heuristics were scientifically derived from experiment or systematic collection and analysis of real-world data, while others were just rules of thumb based on real-world observation or experience.

This idea has been further developed over a number of years by Gigerenzer and co-workers (for example, Gigerenzer and Selten (2001)), who have conducted research on heuristics which assist in making rapid decisions in areas of limited predictability, when probability theory is no longer helpful. They have developed a series of what they call "fast and frugal" rules of general applicability which have outperformed more conventional analysis in such areas as medical diagnosis and performance science (Raab and Gigerenzer 2015). The application to Systems Engineering has so far been little explored.

Practical Use

Experience of using heuristics suggests they should be memorable and can be most effective when phrased informally. The best examples are more than just literal expressions; they should resonate with the readers and suggest additional meaning, encouraging them to find out more about why and when to use them.

There is an underlying issue here: if you haven't experienced the situation for which the heuristic applies, it may mean little to you, and the inherent value may well be lost; but if you already have relevant experience, you may find it obvious. The use of heuristics is therefore linked to how we learn. In a world where experience is vital, but the number of opportunities to learn on the job is limited by the number of big projects one systems engineer works on in a lifetime, the learning process has to be accelerated. Heuristics can help here if linked to other knowledge sources and accessible at the right points in a career, as part of life-long learning.

A repository of heuristics can also act as a knowledge base in its own right, especially if other media, such as video clips or training materials, or even interactive media are added to encourage discussion and feedback. Such a repository might also link to other established knowledge sources or company websites. It can be organized to reflect accepted areas of practice or in a data base tagged with metadata to allow flexible retrieval. Maier and Rechtin (2009) suggested that a repository might also act as a "reading room," allowing users to move freely among associated subjects as if they were following their curiosity in a library or a bookshop. Such a repository could also allow users to assemble a set of heuristics most meaningful to them, relevant to their personal interest or professional sphere of activity.

A further possible use of heuristics was demonstrated by Beasley, et al (2014), who used a selection in a survey to uncover how key aspects of Systems Engineering were addressed within their organization. They asked their staff to mark the heuristics according to their importance to the business and whether they were observed in practice. Analysis of the answers allowed attention to be given to areas requiring improvement.

Broadly speaking, interest in heuristics now centers on their use in two main contexts: first, encapsulating engineering knowledge in an accessible form, where the practice is widely accepted and the underlying science understood; and second, overcoming the limitations of more analytical approaches, where the science is still of limited use. In either case, a good set of heuristics requires active maintenance to reflect the evolution of practice and our constantly developing understanding of what works best. A well-curated collection of heuristics allows practitioners to retain and represent the accumulated practical wisdom of the community of the Systems Engineering profession.

This article finishes with a few simple examples applicable to early stages, with some commentary to suggest their hidden meanings and where they might lead.

- ***Don't assume that the original statement of the problem is necessarily the best, or even the right one.*** The same point is repeated in many heuristics, such as: "The hidden assumptions are likely to be the most damaging," and "The customer may know what he wants, but not what he needs." All this has to be handled with tact and respect for the user, but experience shows that failure to reach mutual understanding early on is a fundamental cause of failure, and strong relationships forged in the course of doing such work can pay off when solving more difficult issues which might arise later on.
 - ***In the early stages of a project, unknowns are a bigger issue than known problems.*** Sometimes what is being asked for is obscure, and the whole context of the systems engineering remit difficult to know, especially in areas of high uncertainty. The starting question may be less "What are the Requirements for this?" and more "What's going on here?". (Kay and King 2020) Unpeeling the layers of meaning behind the second question may require methods drawn from systems thinking – looking for root causes, for example – and can take a project into unexpected areas. Again, soft skills such as empathy and intuition may be more important than those of conventional engineering.
 - ***Model before build, wherever possible.*** This heuristic draws one in to the deeper question of the general use and limitation of models as a way that systems engineering tries to predict desirable and undesirable emergent system
-

properties before committing to build it. A related heuristic states "When you test a model of a system in the real world, you validate the model not the system," and a Theorem from System Science states "The only complete model of the system is the system itself." One heuristic leads to another, leading to reflection on how systems are built in the highly uncertain world of hyperconnected IT systems, where one might postulate that "The only complete model of the system in its environment is the system in its environment," which leads into using evolutionary lifecycles, rapid deployment of prototypes, agile life cycles, and so on. The original heuristic opens a door into 21st Century systems.

- ***Most of the serious mistakes are made early on.*** This heuristic summarizes much of what has just been said. Just to show that much what is believed now might have been known for some time, here is a quote from Plato: "The beginning is the most important part of the work." (Plato 375 BCE).

Finally a heuristic about heuristics – where they come from and what they are for – is taken from a fortune cookie and provides a summary: "The work will tell you how to do it." (Wilczek 2015) The serious point being made here – apart from showing that folk wisdom can have a part to play – is that the most important lessons about how to do Systems Engineering ultimately derive from the collective experience of the community of systems engineers as they undertake their profession. Even the science used to support heuristics must prove itself in practical situations.

References

Works Cited

- Beasley, R., A. Nolan, and A.C. Pickard. 2014. "When 'Yes' is the Wrong Answer." INCOSE International Symposium, Las Vegas, NV, Jun 30-Jul 3, 2014.
- Churchman, C.W. 1967. "Wicked Problems". *Management Science*. 14(4): B-141–B-146.
- Gigerenzer, G. and R. Selten (eds.). 2001. *Bounded Rationality*. MIT Press.
- Kay, J. and M. King. 2020. *Radical Uncertainty: Decision-Making for an Unknowable Future*. The Bridge St. Press.
- King, M. 2016. *The End of Alchemy: Money, Banking and the Future of the Global Economy*. New York and London: W.W. Norton and Company.
- Koen, B.V. 1985. *Definition of the Engineering Method*. Washington, DC: American Society for Engineering Education. ISBN-0-57823-101-3.
- Maier, M. and E. Rechtin. 2009. *The Art of Systems Architecting*, 3rd edition. CRC Press.
- Plato. 375 BCE. *The Republic*.
- Raab, M. and G. Gigerenzer. 2015. "The Power of Simplicity: A Fast-and-Frugal Heuristics Approach to Performance Science", *Frontiers in Psychology*, October 29, 2015.
- Simon, H.A. 1957. *Models of Man, Social and Rational: Mathematical Essays on Rational Human Behavior in a Social Setting*. New York, NY: John Wiley and Sons.
- Wilczek, F. 2015. *A Beautiful Question: Finding Nature's Deep Design*. New York, NY: Penguin Press.

Primary References

Gigerenzer, G. and R. Selten (eds.). 2001. *Bounded Rationality*. MIT Press.

Maier, M. and E. Rechtin. 2009. *The Art of Systems Architecting*, 3rd edition. CRC Press.

Rechtin, E. 1991. *Systems Architecting: Creating and Building Complex Systems*. Englewood Cliffs, NJ: Prentice-Hall.

Simon, H.A. 1957. *Models of Man, Social and Rational: Mathematical Essays on Rational Human Behavior in a Social Setting*. New York, NY: John Wiley and Sons.

Additional References

Royal Academy of Engineering. 2010-2011. *Philosophy of Engineering*, Volumes 1 and 2. Proceedings of seminars held at Royal Academy of Engineering, June 2010 and October 2011. Accessed April 9, 2021. Available: <https://www.raeng.org.uk/policy/supporting-the-profession/engineering-ethics-and-philosophy/philosophy>

Knowledge Area: The Nature of Systems

The Nature of Systems

Contents of this Knowledge Area

- Engineered Systems (Rick Adcock) (Brian Wells, Scott Jackson, and Gary Smith)
- Natural Systems (Dennis Tuckowski, Allison Lyle, Randall Anway, and Jacquelyn Nagel) (Gary Smith)
- Principles and Attributes of Natural Systems (Dennis Tuckowski, Allison Lyle, Randall Anway, and Jacquelyn Nagel) (Gary Smith)
- Value Proposition for Systems Engineering (Dennis Tuckowski, Allison Lyle, and Jacquelyn Nagel) (Gary Smith)
- Identity and Togetherness of Systems (Gary Smith)
- Behavior and Dynamics of Systems (Gary Smith)
- Cycles and the Phases of Systems (Gary Smith) (Olaf Brugman, Helene Finadori, John Kineman, Tom Marzolf, George Mobus, Peter Tuddenham, Lynn Rasmussen, Hillary Sillitto, William Smith, Len Troncale, and Tyler Volk)
- Purpose and the Capability of Systems (Gary Smith)
- Value and the Quality of Systems (Gary Smith)
- Consciousness and the Experience of Systems (Gary Smith)
- Lead Authors:
- Randall Anway, Paul McGoey, Curt McNamara, and Gary Smith

This article introduces The Nature of Systems knowledge area (KA). Systems cannot be separated from their environment and human conceptions of system are contingent upon relations within specific (constrained) situations. Whilst the previous sections are concerned with the Introduction to System Fundamentals, or "The Engineering of Useful Artefacts" (Mobus, Kalton 2014); This section aims to illustrate the nature of systems, a compact description of essential themes relevant to a 'systems sciences' approach to this understanding, and to recognizing interdependencies between systems science and practices of systems engineering.

Introduction

This Knowledge Area (KA) introduces foundational perspectives on the nature of systems across disciplines, including natural, social, and engineered domains. It provides a conceptual bridge between systems science and systems engineering by clarifying essential systemic concepts such as identity, behavior, purpose, and value.

Where previous sections of the SEBoK focus on the engineering of useful artefacts, this KA explores the systemic qualities that underlie all types of systems, whether physical or conceptual, naturally occurring or human-made. It supports systems engineers in gaining a deeper understanding of how systems behave, evolve, and interact with their environments, and how this understanding can improve the design, implementation, and governance of complex engineered systems.

System World Views

While a broad spectrum of perspectives on concrete systems derives from practices of Systems Engineering (Sillitto, et al 2018) and other systems-cantered disciplines, the discipline of Systems Science seeks abstract, generalized, and universally applicable concepts, principles, and theories of systems. The overarching concept 'system' applies to any physical or conceptual subject of interest, including abstract domains of knowledge.

A system is a set of interrelated components. The components are variously linked to one another, and the system itself is connected to its surroundings. All systems have a boundary. In the case of designed or physical systems, the boundary is clear. In social and metaphysical systems, the boundary is typically more dynamic, less obvious, and more open to interpretation. In other words, observers are likely to perceive the boundary of a social or metaphysical system differently. According to Bogdanov (Bogdanov 1996), a system (or complex) is not simply a collection, aggregate (or vector) of components and their relationships. A system is a process, or continuous flux of independent granular processes, concatenated in self-triggering circles of build-up and degradation. Bogdanov's system cannot be separated from its environment, because it does not simply exist or interact with its environment: it is structurally coupled with its environment and thus evolves its own environment while co-evolving with it.

A common observation about systems is that they display properties which are not apparent from the properties of the components themselves. This is sometimes referred to as synergy, or as emergence.

While a system is often perceived as a structure, the unique properties come from the relations between the components within this structure. For example, trees provide shade from the sun and so can function as a refuge from heat. In the practice of systems engineering, a system is often identified with its functions. Designed systems exhibit one or more functions. Functions are useful properties in the context of a higher level system. The functions being an emergent property of the desired system's behavior, and the system's behavior enabled by the structure of the system.

Systems can be both 'physical' and / or 'conceptual' things. Conceptual things, mental processes, often correlate with physical causes (eg, events observed in brains) and may be considered to 'constitute' a separate class of relational processes. If 'conceptual' things can be grounded in relational processes correlated with physical causes, it stands to reason that communicating meaningfully about physical causes at various spatial and temporal scales provides a theoretical basis for including conceptual and physical things in the domain of 'concrete' systems.

In this narrow sense it may be said that all subjects of interest take part in a 'unitary' system of 'concrete' (physical) nature. That is, as an irreducible physical unity, the universe is composed of conceptual and physical things existing in interlocking and subsuming processes of interrelation that may be described in subjectively relatable terms, in other words, 'understood'.

This 'systemness' (existing understandably in relational processes) is a general characteristic of all things. System Science provides useful frames of reference for meaningful approaches to both realms generally (eg, physical and social).

Definitions and Problems of terminology

In this overview article we recognize the (apparent) huge body of literature (a google scholar search on "nature of systems" returns 13,000 results) on the nature of systems. While a few basic definitions of systems (Sillitto, et al. 2018) are widely accepted in practice, significant differences in specialized usage exist.

Thus, here we try to use commonly understood terms meaningful to both practitioners and scientists. While potentially useful to consider this diversity in terms of a more broadly organized taxonomy, it's helpful to appreciate that terminology is unlikely to be permanently 'fixed' or fully normalized; this contingent relation with human expression is a feature of the universe and dynamic interactions within it.

The dual nature of such contingency is notably paradoxical. On one hand, incommensurate terminology can present problems. On the other: opportunity. Together, in the open-ended relation between 'problem' and 'opportunity',

interesting things can occur.

The available knowledge about systems is very rich indeed but it is yet chaotic (the system of system knowledge often yields more questions than answers) and the domain of system science relates variously with many knowledge domains (specializations) in the broader culture. One of the goals of system science, therefore, is to purposively help reconcile such 'geographical' concerns - literally and figuratively. This leads to validating questions of valuation and realization for systems knowledge and systems science pursuits.

Value of systems knowledge

The nature of systems, as one of the most powerful and widely used paradigmatic conceptions is a common thread across human existence. Generally, systems in the universe do not embody human agency - humans and system scientists do. The validation, legitimation, and valuation of systems knowledge generally occurs in its relation between human situations. More specifically, system science in the service of systems engineering (or vice versa) recognizes and is capable of anticipating and delivering value for system stakeholders - who are typically situated in systems of other derivation (cultures that do not necessarily share a common system language).

Systems science (as an activity system) adds value by organizing and tailoring concepts, theories, principles and assets which render useful expressions of fact, concern, effect or degree pertaining to forms, functions, and fitness of systems in the landscape of systems competencies.

These patterns of expression help stakeholders anticipate complex dynamic situations and compose descriptions and sequences of action (eg, plans) in multiple relatable - and reliable - ways. System scientists must communicate with a range of stakeholders in order for systems to be engineered to operate predictably well. In turn, stakeholders must be conversant in the 'local' language(s) of their systemic interest; often this takes priority and the value of system science is in expressing knowledge in terms stakeholders understand rather than the other way around.

Clearly, the nature of systems can be elusive and pursuit of systems knowledge a great challenge. Future articles in this KA are continually being developed to complement (and also improve upon) existing articles. The concepts in the titles of the articles can always be improved upon but the essence of the content should be consistent with this compact description, as it captures foundational themes in our understanding of the relation between human minds and the universe we are present to.

A Set of Systems Concerns

One of the aims of this section is to illustrate the wide array of concerns in systems science literature. Looking across the literature, there is a notable variety of useful categorical mappings and taxonomies. Here we introduce ten representative concerns of systems science; the list is intended to be illustrative rather than definitive.

1. Identity: Bounded networks of relations among simplified elements constitute a nominal and semantically meaningful unit, pragmatically speaking. This 'systemness' is a general characteristic of all things and can often be represented (expressed) as a non-random (informative) network of symbols.
 2. Processes: layers, levels, and dimensions of dynamically *changing structure and function*.
 3. Networks of relations between elements: connectivity, structure, and holistic properties such as resilience, criticality, efficiency.
 4. Dynamics on multiple time scales: states and sequences of change such as growth, collapse, cycling, pattern-formation, criticality.
 5. Complexity : variability in number of elements and dimensions of element relations including additivity, connectivity, and inter-adaptability (contingency, dependency).
 6. Evolution: progression of qualitative, quantitative, and/or semantic change over time.
 7. Information: matter and energy 'encoded' within network(s) of relation or exchange between senders and receivers.
-

8. Governance: modes of mutual regulation and adaptation between elements, typically involving synergetic co-operation or competitive interference.
9. Contingency: degrees of freedom within a network of constraints which the system is subject to.
10. Methods of interaction. Profoundly, every system has a kind of signature; a unique way of showing up to human cognition, through which it reveals or exchanges information and yields knowledge about its state and identity; Practically, this involves (human) stakeholders and their respective processes of attention. In regard to system science and engineering - attention to specific exchanges of energies *through* bounded interfaces (eg, scientific instrumentation; sensing/measuring systems), rather than narrowing-in on surface experience of limitations and boundaries. In other words, system science is a process of open ended learning.

Gaining a 'feel' for systems

Systems are an ordinary occurrence; we are immersed in systems. It is a challenge, however, to discover the unity in the immense diversity of ways systems manifest in experience. Considering the far-ranging Body of Knowledge underlying Systems Engineering, 'natural kinds of value' are foundational. All systems exist in our natural context and can be experienced as values. These values relate to utilities stemming from systems knowledge and its application, ranging from guiding personal day-to-day actions to addressing global challenges.

Within the structure of this section of the SEBOK we shall be giving presentation to interwoven concepts of Togetherness, Character/Behaviour, Cycles, Purpose, Value, and Learning/Experience (Rousseau, et al. 2018) as a way of providing a cognitive system of values, and a holistic perspective on situated systems, systems in practice, that can deeply align theoretical and practical efficiencies of the systems engineering discipline.

The form of things: Togetherness provides a framework for collaborative and integrated operations, emphasizing the indispensability of harmonious interrelations at the heart of successful systems. Behavior informs the adaptable and responsive dynamics that are essential for the resilience and robustness of engineered entities.

The function of things: Cycles highlight the recent and iterative nature of systems, underscoring the importance of feedback mechanisms in persistent operation and effectiveness. Purpose anchors systems with a definitive direction and intentionality, centering on outcomes that deliver meaningful impact.

The fitness of things: Value signifies the critical evaluation of a system's effects, ensuring that resource utilization and stakeholder interests are aligned. Experience centers the human-centric aspects of system concepts, definition, realization and operation, ensuring human appreciation, influence, and subsidiarity.

These six intertwined concepts provide a conceptual scaffolding upon which systems engineering may appreciate the nature of systems when fulfilling the requirements of Form, Fit and Function.

In practice, these general concepts and the theoretical models that build upon them translate through to the application of specific assets such as that utilized in our lifecycle management processes as discussed in the article in this KA.

Applications engendered by these assets play out in the creation and modification of systems artefacts and complex entities involving living systems. Here, the natural kinds of value are further actualized through the actions of embedded computational networks (themselves products of evolutionary refinement); continuous regeneration of self-sufficient organic systems, promotion of sustainable resource cycles, alignment of system functionalities within overarching frameworks of values, elevation of stakeholder value, and enhancement of user interaction with technology.

In general, guidance offered for practical use of systems knowledge spans various dimensions of human activity. It helps direct rationalizations taking place in personal decision-making through to the strategic considerations necessary for addressing complex global issues. By operationalizing the six concepts at every level—from nanotechnology to large-scale infrastructure, systems engineering integrates natural kinds of value with a complex of systems critical to technical, social and natural systems safety and well-being.

In summary, the six system concepts serve as theoretical keystone that, together with methods and applications (facilitated by relevant assets at hand), aim to deliver natural kinds of value at every scale of human enterprise. Integrating these theories, methods, and applications fosters an approach to systems engineering that thrives on adding net intrinsic value. Through such an approach engineered systems transcend mere functionality, endowing them with qualities that support and enrich the human experience in its broadest context.

This is by no means a complete landscape of systems concepts, rather it may help to scaffold a useful entry point into the diverse inspirations and instrumental concepts for systems engineering.

Value of a Systems Science Foundation for Systems Engineers

The urgency of understanding common foundations of systems for practicing SE's can be summarized in three points:

1. Civilization depends upon highly evolved physical and conceptual systems and recent evidence from planetary science <https://www.ipcc.ch/reports> ^[1] indicates that planetary support systems are changing significantly and relatively quickly. Civilization is an ongoing co-evolutionary System of Systems with Humankind's Planetary Support system, and recent evidence indicates that the rate of change is increasing; Civilization depends upon highly evolved planetary systems and recent evidence from planetary science (limits to growth update) indicates that these are changing significantly and relatively quickly;
2. The practice of Systems Engineering is embedded in and co-evolves with the proliferation of socio-technical systems that constitute modern civilization;
3. Designing, integrating, and evolving socio-technical systems becomes more complex and challenging as new technical specialists become involved in Systems Engineering processes. For instance, adding environmental, economic, social, and governance specialties increases workload for the key Systems Engineering role of System Integrator and Communicator.

The considerable value of Systems Science could be demonstrated in helping address these key Systems Engineering challenges by:

1. Framing common-view characteristics of all systems relevant to a given System of Interest and the environment of interacting System of Systems 's so that appropriate tools, techniques, and processes can be employed and/or developed for efficiently working across the system design and development community;
2. Facilitating effective cooperation between diverse technical specialists bringing unique concepts, models, and vocabularies and promoting inclusive equity for enhanced project/program/system success: it's essential for all stakeholders to appreciate their specific situational role(s) in systems of interest co-evolution with interacting systems of systems, to more effectively engage in realizing critical shared objectives.

References

Works Cited

- Bogdanov, A. A. (1996). Bogdanov's tektology. Book 1. Hull, Centre for Systems Studies, University of Hull.
- IPCC AR6. (2022) Mitigation of Climate Change. Climate Change.
- Sillitto, H. Griego, R. Arnold, E. Dori, D. Martin, J. McKinney, D. Godfrey, P. Krob, D. Jackson, S. (2018). What do we mean by "system"? - System Beliefs and Worldviews in the INCOSE Community. INCOSE International Symposium. 28. 1190-1206. 10.1002/j.2334-5837.2018.00542.x.
- Rousseau, D., J. Billingham, and J. Calvo-Amodio, Systemic Semantics: A Systems Approach to Building Ontologies and Concept Maps. Systems, 2018. 6(3): p. 32.
- Metcalf, G. S. Kijima K; Deguchi H. (2021). Handbook of Systems Sciences.

Primary References

Capra, F., & Luisi, P. L. (2014). The systems view of life: A unifying vision. Cambridge University Press.

Mobus, G. E. and M. C. Kalton (2014). Principles of Systems Science, Springer New York.

Smith, E. and H.J. Morowitz, The Origin and Nature of Life on Earth: The Emergence of the Fourth Geosphere. 2016: Cambridge University Press.

Trefil, J.S., The Nature of Science: An A-Z Guide to the Laws and Principles Governing Our Universe. 2003: Houghton Mifflin.

Additional References

None.

References

[1] <https://www.ipcc.ch/reports>

Engineered Systems

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, and Gary Smith

Introduction

Engineered systems are purposeful, human-defined systems created to achieve specific objectives. They are central to systems engineering practice and include products, services, enterprises, and systems of systems (SoS). These systems must be conceived, developed, integrated, and sustained across complex life cycles, often within dynamic and unpredictable environments.

This article complements related entries on **Natural Systems** and **Socio-Technical Systems**. Together, these perspectives provide a more holistic understanding of system types encountered in practice.

Importantly, although engineered systems are designed and assembled by humans (and increasingly with AI assistance), they are **not separate from nature**. They are embedded within the physical world, governed by natural laws, and constrained by the same energy, material, and information flows that structure all natural systems. This duality, that engineered systems are both *products of intentional design* and *participants in broader natural systems*, is a key theme throughout this Knowledge Area.

Engineered Systems as Natural Phenomena

Engineered systems are often conceptualized as distinct from natural systems. However, systems science reminds us that they are **subsets of natural systems**:

- They are composed of physical materials, constrained by the same thermodynamic and ecological principles that govern all natural processes.
- Their behaviors emerge from feedback loops, control mechanisms, and relational dynamics consistent with other systems in the universe.
- They generate effects, intended or not, that interact with ecosystems, societies, and planetary systems.

The design of engineered systems therefore requires not only technical ingenuity, but also **an appreciation for their ecological and systemic context**. Engineering that is blind to natural laws or environmental coupling risks unsustainable or unstable outcomes. Conversely, engineered systems that align with natural cycles, resilience principles, and ecological limits can enable regenerative and adaptive capabilities at scale.

This perspective resonates with foundational concepts explored in the articles on:

- **Cycles and Phases** – highlighting lifecycle rhythms, renewal, and iteration
- **Value and Qualities** – emphasizing stakeholder and systemic outcomes beyond function
- **Experience and Consciousness** – recognizing human-system interactions and feedback
- **Natural Systems** – describing self-organizing systems as constraints and inspirations

System Classification

Engineered systems are one class within broader taxonomies of systems developed in systems science. Several influential frameworks include:

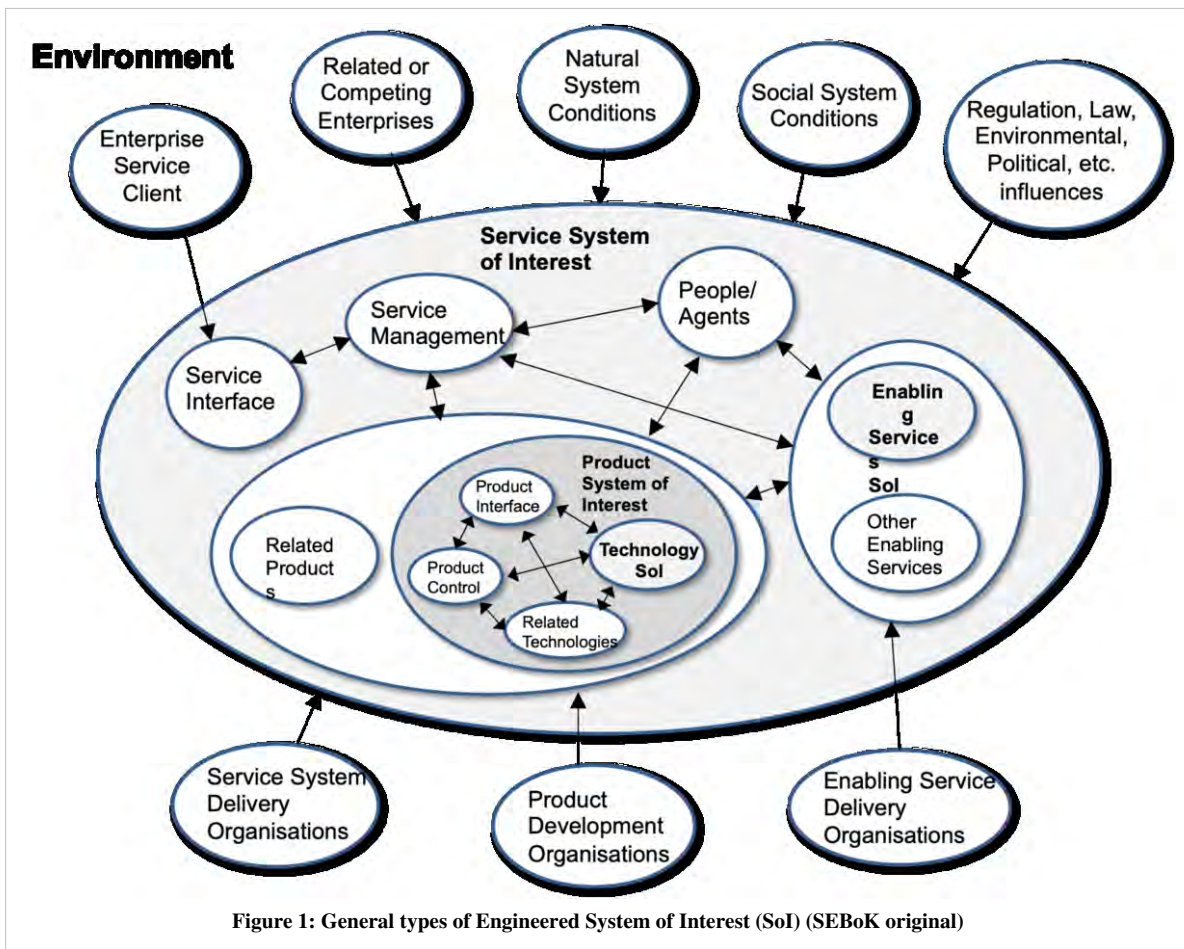
- **Boulding (1956)**: A hierarchical classification ranging from structures and organisms to social and transcendental systems.
- **Checkland (1999)**: Five categories: natural systems, designed physical systems, designed abstract systems, human activity systems, and transcendental systems.
- **Magee and de Weck (2004)**: A functional classification of systems by process (transform, transport, store, exchange, or control) and by the entity on which they operate (matter, energy, information, or value).

Engineered systems are typically situated in the class of **designed physical and abstract systems**, where intentional function, lifecycle logic, and stakeholder goals are dominant, but still operating within natural constraints..

Types of Engineered System

Engineered systems can be grouped into four general contexts that are commonly recognized in systems engineering practice: **product systems**, **service systems**, **enterprise systems**, and **systems of systems (SoS)**. Each represents a possible *system of interest (SoI)* within a life cycle.

Figure 1 illustrates these contexts. A product may exist as a technology-focused system, integrated into a service; services are delivered and sustained by enterprises; and in many cases, systems are combined to form larger systems of systems



Products and Product Systems

A **product system** is an engineered system focused on the creation and delivery of tangible or intangible products, such as hardware, software, or information artifacts. The life cycle of a product system typically includes design, production, operation, sustainment, and eventual retirement.

Products do not exist in isolation: they interact with people (operators, maintainers, producers) and are deployed within service systems that deliver capabilities to an enterprise or society. Effective product system engineering therefore requires attention to both the product itself and its wider context.

Services and Service Systems

A **service system** is an engineered system that delivers outcomes or benefits to users. Services are processes or performances co-created with clients or stakeholders. Examples include transportation, healthcare, communications, and information technology services.

Service systems are often information-intensive and software-defined. They may involve combinations of products, people, and supporting infrastructure, integrated close to the point of use. Systems engineers address both the design of the service and the management of its delivery, ensuring performance, quality, and adaptability over time.

Enterprise Systems

An **enterprise system** is a purposeful network of people, processes, organizations, and technologies that interact to achieve shared goals. Enterprises are unique in that they are constantly evolving, rarely have fixed requirements, and typically balance multiple objectives such as customer satisfaction, stakeholder value, and long-term sustainability.

Systems engineering supports enterprises through **enterprise architecture** and related modeling approaches, which describe current capabilities and plan for future ones. These tools allow enterprises to align their strategy with the product and service systems that support their operations.

Systems of Systems

A **system of systems (SoS)** is an arrangement of independent systems that retain operational and managerial autonomy but are integrated to provide new capabilities. Examples include national defense networks, air traffic management systems, and smart cities.

SoS engineering involves unique challenges, including governance, interoperability, and lifecycle coordination across independently managed systems. As integration technologies become more common, SoS considerations are increasingly central to modern systems engineering practice.

Applying Engineered System Contexts

Real-world engineering efforts often span multiple system types. For example:

- A satellite product is operated within a telecommunications service,
- Managed by an aerospace enterprise,
- And integrated into a defence SoS.

Systems engineers must therefore define the **system of interest (SoI)** within a given project or activity, while remaining aware that it exists within broader systems-of-systems and natural environments.

This awareness includes:

- Defining clear boundaries and interfaces
- Understanding enabling and interacting systems
- Recognizing cycles of feedback and adaptation
- Designing with foresight into ecological and social impacts

The **article on Identity and Togetherness** provides further perspective on boundary definition and system coherence across contexts.

Interfacing with Natural and Socio-Technical Systems

Engineered systems:

- **Depend on natural systems** for materials, energy, and stability (see *Natural Systems: Principles and Attributes*)
- **Shape and are shaped by social systems**, including regulation, usage, and cultural norms (see *Value and the Quality of Systems*)
- **Must evolve to fit within dynamic environmental conditions**, respecting planetary boundaries and long-term resilience requirements

As engineered systems become more complex and intelligent, **they increasingly resemble natural systems**, exhibiting self-organization, learning, and emergent behavior. This convergence suggests the need for system engineers to understand:

- The **limits and affordances of physical law**
 - The **patterns of natural system organization**
 - The **ethical responsibilities of system design** in relation to society and ecology
-

Summary

Engineered systems are intentional, human-defined systems developed to fulfill specific functions. Yet, they are also deeply **natural**, built within and governed by the physical world, and inevitably connected to broader ecological and social systems.

By situating engineered systems within the wider landscape of systems types, this article supports a more integrated and reflective approach to engineering practice. It complements other articles in this Knowledge Area that explore the shared principles, behaviors, and values that cut across both natural and artificial systems.

Understanding this interconnection is essential for designing **systems that are not only effective, but also sustainable, adaptable, and ethically coherent** in the complex systems landscape of the 21st century.

References

Works Cited

- Boulding, K. E. (1956). "General systems theory: The skeleton of science." *Management Science*, 2(3), 197–208.
- Checkland, P. B. (1999). *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons.
- Magee, C. L., & de Weck, O. L. (2004). "Complex system classification." *Proceedings of the 14th Annual INCOSE International Symposium*, Toulouse, France, June 2004.

Primary References

- ISO/IEC/IEEE 15288:2023. *Systems and Software Engineering, System Life Cycle Processes*. Geneva, Switzerland: ISO/IEC.
- INCOSE. (2022). *INCOSE Systems Engineering Vision 2035*. San Diego, CA: International Council on Systems Engineering.
- Dahmann, J., Baldwin, K. J., & Goodnight, J. (2020). "Systems of Systems Engineering: Essential Concepts and Practical Examples." *INCOSE International Symposium*, Cape Town, South Africa.

Additional References

- Blanchard, B. S., & Fabrycky, W. J. (2010). *Systems Engineering and Analysis* (5th ed.). Upper Saddle River, NJ: Prentice Hall.
- Maier, M., and E. Rechtin. 2009. *The Art of Systems Architecting*, 3rd Ed. Boca Raton, FL, USA: CRC Press.
- Rebovich, G., & White, B. E. (eds.). (2011). *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL: CRC Press.
-

Natural Systems

- Lead Authors:
- Dennis Tuckowski, Allison Lyle, Randall Anway, and Jacquelyn Nagel
- Contributing Author:
- Gary Smith

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

Natural systems are **self-organizing, dynamic systems** that occur in nature without direct human intervention. They consist of interacting physical, chemical, biological, and ecological components that have existed and evolved over billions of years, and operate according to the laws of physics and natural processes (i.e. evolution, natural selection). Natural systems typically exhibit feedback loops, energy flows, material and temporal cycles, and can be viewed as open systems governed by boundary conditions and conservation laws (e.g., mass, energy, momentum). Inputs such as solar energy or precipitation drive outputs such as runoff, heat, and carbon flux, creating continuous cycles that sustain life and ecological balance.(Capra & Luisi, 2014; Morowitz, 2002).

Understanding natural systems is essential for systems engineers because it:

- Reveals recurring **systemic principles** that govern stability, resilience, adaptation, and transformation;
- Provides **constraints** that must be respected in engineered solutions;
- Offers **models and metaphors** that can inspire innovation, sustainability, and long-term viability;
- Helps situate engineering within the broader fabric of planetary systems and ecological interdependence (Mobus & Kalton, 2015).

What Makes a System “Natural”?

Natural systems differ from engineered systems not because they are simpler or less functional, but because their **organization arises from internal and environmental interactions**, not from top-down design (Jantsch, 1980; Bunge, 1979). Key characteristics of natural systems include:

- **Self-organization:** Structure and function emerge without centralized control
- **Openness:** Exchange of energy, matter, and information with the environment
- **Multi-scale dynamics:** Nested spatial and temporal scales of operation
- **Evolutionary adaptation:** Variation, feedback, and selection processes
- **Constraint-driven behavior:** Shaped by physical and thermodynamic laws (Odum, 1983)

These systems may be living (organisms, ecosystems), non-living (climate, plate tectonics), or complex hybrids (e.g., microbiome–host systems).

Natural Systems in the Engineering Context

Natural systems are deeply entangled with engineered systems in three primary ways:

1. Constraint

Engineered systems must respect the physical and ecological limits of the environment, such as energy flows, entropy, material cycles, and climate stability (Rockström et al., 2009).

2. Context

Natural systems form the surrounding conditions into which engineered systems are embedded. This includes local ecosystems, weather, geological features, and socio-ecological dynamics (Folke et al., 2010).

3. Inspiration

Natural systems offer design patterns and strategies, such as modularity, feedback regulation, and redundancy, that can be abstracted and applied to engineered systems (Benyus, 1997; Vincent et al., 2006).

This triad (constraint, context, inspiration) aligns with the broader themes in this Knowledge Area, particularly:

- *Cycles and Phases* – natural systems are inherently cyclic (e.g., water cycle, nutrient flow)
- *Purpose and Capabilities* – while not consciously goal-seeking, natural systems exhibit coherent and evolving functions
- *Value and Qualities* – natural systems provide ecosystem services and intrinsic values increasingly recognized in engineering decisions

Properties of Natural Systems

Natural systems exhibit a range of properties, some of which are shared with engineered systems, while others are distinctive.

Shared System Properties	Distinctive Natural Properties	Engineering Relevance
Resilience: Return to stable state after disturbance	Growth: Increase in capacity and structure over time	Informs scalable and sustainable design
Robustness: Maintain function across variable conditions	Regenerability: Restore function after degradation	Inspires recovery and renewal strategies
Adaptability: Reconfigure to meet new conditions	Anti-fragility: Improve through exposure to stress	Basis for learning systems, evolutionary computation
Sustainability: Persist within changing environments	Flourishing: Support reciprocal relationships	Guides circular economy, cooperative architectures

These properties arise from systems’ embeddedness in feedback-rich, evolving environments (Meadows, 2008; Gunderson & Holling, 2002). Systems engineers can draw on these traits to inform lifecycle design, risk management, and adaptive response.

Examples Across Disciplines

Engineering Domain	Relevant Natural Systems
Civil & Environmental	Watersheds, floodplains, geochemical cycles
Aerospace & Mechanical	Atmospheric dynamics, thermodynamics
Electrical & Bioengineering	Neural networks, electrochemical signaling
Systems Engineering	Ecosystem modeling, food web resilience, system dynamics

These examples illustrate how natural systems can serve as **reference architectures**, **benchmarks**, or **integrated constraints** in engineering thinking.

Implications for Systems Engineering

Systems engineers increasingly work in domains where **natural and engineered systems intersect**, from sustainable infrastructure to adaptive software agents embedded in biological environments.

Understanding natural systems supports:

- **Transdisciplinary modeling** (e.g., integrating ecological and technical models)
- **Systems-of-systems planning** under conditions of uncertainty and change
- **Resilient and regenerative design strategies**
- **Ethical awareness** of systemic externalities and responsibilities

As engineering challenges become increasingly complex and planetary in scope, the **principles of natural systems provide a guide** for designing with humility, foresight, and systemic alignment.

Summary

Natural systems are not external to engineering, they are its context, constraint, and sometimes its inspiration. By learning from natural systems, engineers can design more adaptive, sustainable, and resilient systems that contribute positively to the complex web of life.

This article supports further exploration in the KA through:

- *Principles and Attributes of Natural Systems*
- *Value Proposition for Systems Engineering*
- *Cycles and the Phases of Systems*
- *Behaviour and Dynamics of Systems*

References

Works Cited

- Benyus, J. M. (1997). *Biomimicry: Innovation Inspired by Nature*. HarperCollins.
- Capra, F., & Luisi, P. L. (2014). *The Systems View of Life: A Unifying Vision*. Cambridge University Press.
- Folke, C., et al. (2010). *Resilience Thinking: Integrating Resilience, Adaptability, and Transformability*. Ecology and Society, 15(4).
- Gunderson, L., & Holling, C. S. (Eds.). (2002). *Panarchy: Understanding Transformations in Human and Natural Systems*. Island Press.
- Jantsch, E. (1980). *The Self-Organizing Universe*. Pergamon Press.
- Meadows, D. H. (2008). *Thinking in Systems: A Primer*. Chelsea Green.
- Mobus, G. E., & Kalton, M. C. (2015). *Principles of Systems Science*. Springer.
- Odum, H. T. (1983). *Systems Ecology*. Wiley-Interscience.
- Rockström, J., et al. (2009). "A Safe Operating Space for Humanity." *Nature*, 461(7263), 472–475.

- Vincent, J. F. V., et al. (2006). "Biomimetics: Its Practice and Theory." *Journal of the Royal Society Interface*, 3(9), 471–482.

Primary References

- Capra, F., & Luisi, P. L. (2014). *The Systems View of Life*.
- Mobus, G., & Kalton, M. (2015). *Principles of Systems Science*.
- Gunderson, L., & Holling, C. S. (2002). *Panarchy*.
- Meadows, D. H. (2008). *Thinking in Systems*.
- Odum, H. T. (1983). *Systems Ecology*.
- Benyus, J. M. (1997). *Biomimicry*.
- Jantsch, E. (1980). *The Self-Organizing Universe*.

Additional References

None

Principles and Attributes of Natural Systems

- Lead Authors:
 - Dennis Tuckowski, Allison Lyle, Randall Anway, and Jacquelyn Nagel
- Contributing Author:
 - Gary Smith

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Natural Systems: Principles and Attributes

Natural systems are the most enduring systems in the known universe. Shaped by billions of years of evolution, they exhibit systemic patterns that support resilience, adaptability, and sustainability. While engineered systems are often designed with a focus on efficiency and function, natural systems offer a deeper repertoire of systemic principles and attributes that are relevant to modern systems engineering, especially in the face of global complexity, planetary limits, and socio-technical interdependence.

This article explores those principles and attributes, drawing on foundational work from systems science, including **systems processes and isomorphies** (Troncale 2014), and **visualizing dynamic patterns** (Rasmussen 2024). By studying natural systems, systems engineers can draw inspiration and guidance for designing systems that are not only technically effective but also ecologically integrated and future-fit.

Principles, Attributes, and Engineering Implications

The following table presents core **principles of natural systems**, the **characteristic attributes** they give rise to, and **implications for systems engineering practice**. These principles emerge through long-term interactions within and across multiple scales, yielding patterns of systemic health.

Principle of Natural Systems	Characteristic Attribute	Systems Engineering Implication
Self-Organization – ordered patterns and functions emerge without centralized control	Emergent complexity	Supports decentralized architectures (e.g., swarm robotics, adaptive SoS) and reduces reliance on single points of failure
Adaptation through Feedback – positive and negative feedback regulate behavior	Dynamic responsiveness	Informs control theory, adaptive algorithms, and real-time monitoring in cyber-physical systems
Decentralization and Modularity – distributed control and modular subsystems	Scalability & autonomy	Guides modular open system architectures (MOSA), enabling independent upgrades and resilience
Resource Efficiency – optimal use of materials and energy in closed loops	Sustainability, ecosystem services & circularity	Encourages lifecycle efficiency, closed-loop supply chains, and energy/material minimization
Redundancy and Diversity – multiple pathways and variation buffer against disruption	Robustness & fault tolerance	Informs fault-tolerant design, diversity in redundancy strategies, and resilience engineering
Hierarchical Organization – nested levels of structure and function	Multi-scale integration	Supports hierarchical system decomposition and layered architectures (e.g., C4ISR, enterprise systems)
Evolutionary Iteration – improvement through variation, selection, and retention	Iterative improvement & adaptability	Reinforces agile development, evolutionary algorithms, and continuous verification/validation
Co-evolution and Symbiosis – systems adapt in relation to one another	Interoperability & cooperation between ecosystem level interfaces	Informs SoS engineering, ecosystem integration, and collaborative architectures
Anti-Fragility – capacity to recover or improve through stress	Resilience, enhancements in capability	Provides design principles for contested or high-risk environments, emphasizing system responsiveness, recovery and learning
Cyclical Temporality – system behaviors often occur in recurrent patterns or cycles	Dynamic responsiveness to repeating events across system elements and hierarchies	System scheduling, buffering, and pacing in response to its environment

These principles are observed in ecosystems, biological processes, and geophysical phenomena. Collectively, they create systems capable of thriving under uncertainty, maintaining coherence across scales, and adapting to long-term environmental change.

Toward a Nature-Informed Paradigm for Systems Engineering

Natural systems offer both **conceptual insight** and **practical reference models** for engineering. They exemplify the integration of **form**, **function**, and **fitness**, a triad that aligns with the **Fit–Form–Function** lens applied across this SEBoK knowledge area. Natural systems do not aim for single-purpose optimization, but instead balance multiple system goals (e.g., stability, diversity, and renewal).

Adopting a nature-informed perspective allows systems engineers to:

- Design for **resilience**, not just reliability
- Develop systems that **coexist** with natural ecosystems
- Build **adaptive capacity** into complex socio-technical infrastructures
- Leverage principles such as **circularity** and **interconnectedness** to mitigate unintended consequences

Reference Models from Nature

Natural systems can be used as reference models across different engineering challenges:

- **Ecosystems** (e.g., forests, coral reefs): inspire distributed regulation, symbiosis, and multi-species cooperation
- **Biological processes** (e.g., photosynthesis, decomposition): illustrate closed-loop material flows and energy efficiency
- **Geophysical systems** (e.g., plate tectonics, erosion): show long-term adaptation, thresholds, and transformation
- **Atmospheric/oceanic dynamics** (e.g., jet streams, ocean currents): model distributed control, feedback loops, and buffering across vast scales

These models are particularly useful for **biomimicry**, **systems-of-systems design**, and **regenerative infrastructure**. They also reinforce the importance of **temporal design**, a theme central to both Troncale's cyclical isomorphies and Rasmussen's visualization of systemic patterns.

Design Heuristics Inspired by Nature

Systems engineers can draw upon the following heuristics informed by natural systems:

- **Design for redundancy with variation**, not just replication
- **Foster self-regulation** through feedback and monitoring
- **Embrace partial control**, allowing room for adaptation
- **Optimize for relationships**, not just parts
- **Co-evolve systems with their environments**
- **Respect planetary boundaries**, designing within ecological constraints

Implications for Problem-Solving, Design, and System Interactivity

- **Modeling**: bridging reference models to design and production models using analogies, particularly at functional levels
- **Biomimicry**: Innovative solutions that emulate nature's patterns for sustaining life.
- **Systems Thinking**: Holistic understanding of complex problems, recognizing dependencies and connectivity.
- **Circular Economy**: Value production systems that emulate natural cycles and functional relationships to help manage waste, promote reuse, and circulate materials and energy.
- **Resilient Infrastructure**: Embedding principles such as adaptability and self-organization can contribute to infrastructure responsiveness, reusability, and multi-functionality.

Relationship to Other Articles in the Knowledge Area

This article complements:

- **Natural Systems** (definition and scope)
- **Identity and Togetherness of Systems** (distinction, boundary, binding and wholeness)
- **Behaviour and Dynamics of Systems** (response patterns and change processes)
- **Cycles and the Phases of Systems** (temporal system states and changes)
- **Purpose and the Capability of Systems** (teleology and emergent function)
- **Value and the Quality of Systems** (systemic contributions to life and flourishing)
- **Consciousness and the Experience of Systems** (sentience, awareness, and information exchange)

Together, these articles scaffold a **unified systems view** integrating insights from **natural, engineered, and socio-technical domains**.

References

Works Cited

- Rasmussen, L. (2024). *Seeing: Patterns in Living Systems*. Synearth Publishing.
- Troncale, L. (2014). SPT I.: IDENTIFYING FUNDAMENTAL SYSTEMS PROCESSES FOR A GENERAL THEORY OF SYSTEMS. *Proceedings of the 56th Annual Meeting of the ISSS - 2012*,

Primary References

- Capra, F., & Luisi, P. L. (2014). *The Systems View of Life: A Unifying Vision*. Cambridge University Press.
- Holling, C. S. (1973). "Resilience and Stability of Ecological Systems." *Annual Review of Ecology and Systematics*.
- Mobus, G. E., & Kalton, M. C. (2014). *Principles of Systems Science*. Springer.
- Meadows, D. H. (2008). *Thinking in Systems: A Primer*. Chelsea Green.
- Odum, E. P. (1996). *Ecology: A Bridge Between Science and Society*.
- Strogatz, S. (2003). *Sync: The Emerging Science of Spontaneous Order*.
- Troncale, L.R. (1978). Nature's Enduring Patterns. California State Polytechnic University.

Additional References

None

Value Proposition for Systems Engineering

- Lead Authors:
- Dennis Tuckowski, Allison Lyle, and Jacquelyn Nagel
- Contributing Author:
- Gary Smith

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

Natural systems represent the most time-tested systems on Earth, shaped by billions of years of interaction, adaptation, and co-evolution across scales. These systems have persisted and transformed under conditions of uncertainty, constraint, and change. Their enduring success provides a foundational knowledge base for understanding systemic viability, resilience, and integration.

This article presents the **value proposition** for bringing natural systems perspectives into systems engineering. It outlines practical and strategic reasons why nature's principles, structures, and patterns are relevant to the future of systems thinking and engineering practice, especially as we confront planetary-scale complexity and cascading systemic risks

By studying and abstracting strategies from natural systems, engineers can expand the solution space across domains such as structure, function, behavior, interfaces, and process dynamics. This can improve system performance and reliability, reduce risk, and increase efficiency in engineering processes (e.g., Benyus 1997; Mobus 2022).

Why Natural Systems Matter to Systems Engineering

Systems engineering has historically focused on the purposeful design of human-made systems to meet specified objectives. However, engineered systems increasingly operate within dynamic, interconnected environments that resemble the complex behavior of natural systems. These contexts, ranging from cyber-physical infrastructure to social-ecological systems, demand design practices capable of dealing with:

- Multi-scale feedback and cascading effects
- Long-term resilience and regenerative capability
- Evolving stakeholder needs and boundary conditions
- Integration with planetary systems and natural limits

Natural systems provide:

- **Proven patterns of systemic health**, such as resilience, redundancy, adaptability, and regeneration
- **Functional strategies shaped by evolutionary constraint and selection**, offering robust solutions under real-world pressures
- **Conceptual bridges** that align science, engineering, sustainability, and policy
- **A reference architecture for systems of systems**, grounded in observed performance across temporal and spatial scales. Applying natural principles to engineered systems has already led to advances in areas such as materials science, functional capabilities, and system architectures. Examples include the lightweight strength of bone inspiring aerospace structures, neural processes informing artificial intelligence and control systems, and ecological resilience informing redundancy and diversity in system-of-systems architectures. In practice, engineers can learn from natural strategies to optimize system attributes, supporting performance criteria such as:
 - Size-weight-power-function optimization
 - Sensing and perception accuracy
 - Cognitive adaptation and decision-making
 - Resilience and anti-fragility under stress

Recognizing natural systems as both **context and inspiration** encourages systems engineers to move from isolated optimization to integrated co-evolution within the broader Earth system.

Functional and Strategic Value of Natural Systems Thinking

Domain	Value Contribution
Design Innovation	Biological and ecological dynamics inspire novel control architectures, scalable morphologies, and multifunctional materials (e.g., swarm robotics, self-healing surfaces)
Performance Optimization	Natural models inform optimization strategies for power-to-weight ratios, feedback timing, and multi-objective trade-offs (e.g., vascular flow, neural signal propagation)
Resilience and Robustness	Nature achieves systemic resilience through diversity, redundancy, and dynamic feedback, offering key heuristics for SoS and infrastructure design
Lifecycle Adaptability	Iterative change through variation, selection, and retention mirrors agile development, adaptive control, and evolutionary algorithmic approaches
Societal and Ecosystem Integration	Nature-based design supports circular economy, ecosystem services modeling, and integrated planning across human–environment interfaces

These insights can be applied not only at the product or project level, but across enterprise transformation, infrastructure resilience, and planetary stewardship.

Established Contributions from Natural Systems to Engineering Practice

Systems engineering has already begun to draw on nature-derived ideas in domains such as:

- **Neural networks and cognition-inspired architectures**
- **Bio-inspired materials**, such as lotus-effect surfaces and bone-mimicking composites
- **Distributed sensing and regulatory feedback**, modelled on biological networks
- **Ecological resilience frameworks**, applied to infrastructure planning and disaster response
- **Circular resource systems**, echoing nutrient cycling and metabolic closure
- **Holarchic decomposition**, reflecting the nested nature of ecosystems and organisms (see *Holarchy and Fit–Form–Function*)

Key contributors to this convergence include:

- **Janine Benyus's** articulation of *biomimicry* as a design philosophy
- **Len Troncale's** *Systems Processes Theory*, which identifies recurring system processes across natural and artificial systems
- **Lynn Rasmussen's** *Seeing*, which visualizes patterns and rhythms in living systems as design referents for system dynamics

These contributions illustrate that nature does not merely inspire aesthetics, it offers scalable, transdisciplinary insights that can ground both practice and theory.

Systems Engineering Benefits Across Levels

Level	Benefit
Project	More robust, adaptive, and context-sensitive designs; reduced failure modes and lifecycle vulnerabilities
Organizational	Facilitates interdisciplinary collaboration across science, engineering, ecology, and sustainability
Professional	Deepens creative and ethical engagement with complex challenges; supports careers attuned to 21st-century imperatives
Societal	Contributes to regenerative, equitable, and sustainable system outcomes aligned with long-term planetary viability

A Responsibility to Design Within and With Nature

Nature is not merely a constraint, it is a **partner, precedent, and teacher**. As engineering increasingly intersects with planetary limits, it becomes vital to:

- **View Earth as a system-of-systems**, not a passive backdrop
- **Recognize and value ecosystem services** as part of design trade-offs
- **Design systems that co-evolve with their environments**, respecting scale and phase relationships
- **Foster long-term viability**, regeneration, and fairness, not just short-term output and optimization

This perspective contributes to an emerging **ethics of systems** grounded in **stewardship, relationality, and life-enhancing reciprocity**.

Toward a Nature-Informed Paradigm

As systems engineers confront 21st-century challenges, such as changing climate, advancement of cyber-physical systems, and space exploration, natural systems provide not only technical inspiration but also a paradigm for sustainability and stewardship. Recognizing the Earth as a system of systems encourages the design of human systems that thrive with, rather than at the expense of, the natural environment.

Significant System of Systems Reference Models:

- **Ecosystems:** Forests, Coral Reefs, or Grasslands can provide insights into how diverse components continually self-organize.
- **Biological Processes:** Photosynthesis, Decomposition, or Nutrient Cycling can reveal evolved patterns of energy conversion, waste management, and resource allocation.
- **Geological Systems:** Plate Tectonics, Weathering, or Erosion can provide perspectives on long-term change, stability, and different components of the Earth's system.
- **Atmospheric and Oceanic Processes:** Circulation patterns, Currents, and Mass-Energy Balance can inform our understanding of global systems, feedback loops, and the interconnectedness of our planet.
- **Critical Zone Interfaces:** The dynamic and complex interactions between the atmosphere, biosphere and geosphere can offer insight into how nature operates through multiple levels and scales of complexity, highlighting functional exchanges of material and energy at the system boundaries.

The benefits of incorporating natural systems thinking also extend beyond individual projects. At an organizational level, it can create productive intersections between scientific, technological, and engineering functions. At a professional level, it fosters creativity, engagement, and satisfaction in developing holistic solutions. At a societal level, it supports the design of systems that are more sustainable and more symbiotic with their natural counterparts (Ostrom 2009; Volk 2017).

Ultimately, integrating knowledge of natural systems into systems engineering can improve outcomes across the lifecycle—enhancing performance, value, and sustainability. It also strengthens awareness of the Earth as a system-of-systems, highlighting the value of ecosystem services and the responsibility to design human systems that thrive with, rather than at the expense of, natural systems.

Connections to Other Articles in This Knowledge Area

This article is part of *The Nature of Systems* and is closely linked to:

- **Natural Systems: Principles and Attributes** – systemic patterns underpinning design insight
 - **Natural Systems (Definition)** – defining characteristics of natural systems and their boundaries
 - **Value and the Quality of Systems** – understanding systems through intrinsic and systemic worth
 - **Cycles and the Phases of Systems** – temporal dynamics of development, stability, and transformation
 - **Consciousness and the Experience of Systems** – feedback, identity, and sensing as systemic phenomena
-

References

Works Cited

- Benyus, J. M. (1997). *Biomimicry: Innovation Inspired by Nature*. HarperCollins.
- Mobus, G. E., & Kalton, M. C. (2014). *Principles of Systems Science*. Springer.
- Ostrom, E. (2009). "A General Framework for Analyzing Sustainability of Social-Ecological Systems." *Science*, 325(5939), 419–422.
- Rasmussen, L. (2024). *Seeing: Patterns in Living Systems*. Synearth Publishing
- Troncale, L. (2014). SPT I.: IDENTIFYING FUNDAMENTAL SYSTEMS PROCESSES FOR A GENERAL THEORY OF SYSTEMS. *Proceedings of the 56th Annual Meeting of the ISSS - 2012*,

Primary References

- Capra, F., & Luisi, P. L. (2014). *The Systems View of Life: A Unifying Vision*. Cambridge University Press.
- Holling, C. S. (1973). "Resilience and Stability of Ecological Systems." *Annual Review of Ecology and Systematics*.
- Kauffman, S. A. (1995). *At Home in the Universe: The Search for Laws of Self-Organization and Complexity*. Oxford University Press.
- Volk, T. (2017). *Quarks to Culture: How We Came to Be*. Columbia University Press.
- Troncale, L. (2006), "Towards A Science of Systems" *Systems Research and Behavioral Science*, Special Issue on J.G. Miller, Founding Editor (G.A. Swanson, Ed.)

Additional References

None

Identity and Togetherness of Systems

- Lead Author:
- Gary Smith

This article is part of The Nature of Systems knowledge area (KA).

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Systems are distinguished not only by what they do, but by what they are. This article explores two foundational and interdependent properties of all systems: **identity**, which sustains their distinctiveness and coherence, and **togetherness**, which enables the organization and integration of parts into functioning wholes. Together, these properties underpin system persistence, adaptation, and recognition across scales.

This article complements other entries in this knowledge area that examine behavior, purpose, value, and cycles, offering systems engineers a deeper understanding of how systems maintain continuity while participating in complex, evolving environments.

Introduction: Identity, the Persistence of Distinctiveness

Identity and togetherness are fundamental properties of systems that underpin their recognition, persistence, and coherence. *Identity* refers to what makes a system distinct from its environment, while *togetherness* refers to the forces and organizing principles that bind components into a functioning whole. These concepts are central to understanding Form in the Fit–Form–Function framework and are essential for defining, modelling, and engineering systems of interest.

In natural systems, identity is expressed through mechanisms such as atomic structure, genetic codes, or ecological niches, while togetherness emerges from binding forces, membranes, flows, and interdependencies. In social systems, shared values and institutions sustain collective identity and cohesion. Engineered systems achieve identity and togetherness through standards, interfaces, and architectures that stabilize interactions across components and stakeholders.

Patterns of identity and togetherness can be observed across levels of emergence, from quarks and molecules to ecosystems, societies, and systems of systems. At each level, new forms of binding lead to new forms of identity: particles form atoms, cells form organisms, and institutions form cultures and enterprises. This progression illustrates how complexity grows through successive layers of coherence.

For systems engineering, attention to identity and togetherness has practical implications. Defining the System of Interest (SoI) requires clarity of boundaries and coherence. Modularity and cohesion guide system integration and interoperability. Identity supports verification, validation, and lifecycle continuity. At enterprise and SoS scales, maintaining togetherness across independent systems requires governance, shared purpose, and mission alignment. Finally, recognizing the togetherness of human systems with natural systems emphasizes the need for sustainable and symbiotic design.

By studying identity and togetherness, systems engineers and practitioners gain a foundation for addressing complexity, ensuring resilience, and designing systems that persist and adapt within broader contexts.

Definitions and Perspectives

- **Identity:** refers to the enduring properties by which a system is recognized as the same system over time, even as its components and context may change. It arises from:
 - **Structure:** The pattern of internal relationships that provide cohesion and functional capability.
 - **Boundaries:** A distinction from the surrounding environment, whether physical, informational, or conceptual.
 - **History:** Continuity through memory, record, or inherited structure (e.g., genetic code, configuration baselines).
 - **Intent:** In engineered systems, purpose contributes to identity, what the system is *for* shapes what it *is*. Systems engineers commonly encounter identity in the form of **system-of-interest (SoI)** boundaries, configuration states, mission definitions, and stakeholder values. However, identity is also an emergent, relational phenomenon—it is not always fixed, and in socio-technical or natural systems, may evolve or be contested..
- **Togetherness:** refers to the organizing forces, constraints, or relations that bind a system's parts into a coherent whole. This includes:
 - **Binding Forces:** Physical (e.g., gravity, electromagnetism), informational (e.g., feedback), or organizational (e.g., hierarchy, contracts).
 - **Interdependence:** Components rely on one another to fulfill system functions.
 - **Integration Mechanisms:** Interfaces, modularity, and protocols enable cooperation and coordination.
 - **Contextual Unity:** Shared environment or purpose that drives collective behavior. In engineering practice, togetherness shows up in **systems integration, interface control, team dynamics, and value co-creation**. The stronger and more aligned the forces of togetherness, the more functional and resilient the system..

Perspectives from systems science include:

- **General Systems Theory** (Boulding, 1956): Identity persists through levels from structures to transcendent systems.
- **Relational Science** (Kineman, 2011): Identity emerges through a system's relation to self, others, and whole.
- **Holonic Models** (Koestler, 1967): Systems as nested wholes/parts, each with autonomous identity and embeddedness.
- **Pattern Science** (Troncale, 1978; Rasmussen, 2024): Identity arises from pattern recurrence, togetherness from coupling processes.

Patterns and Archetypes of Identity & Togetherness

Identity and togetherness emerge through recurring archetypes across levels of scale. Each level demonstrates how binding mechanisms create coherence and how new forms of identity arise. These archetypes also provide analogies useful for systems engineering practice.

Quarks and Atoms

- *Binding:* Strong and electromagnetic forces hold particles together.
- *Identity:* Stable elements with distinctive properties.
- *Engineering Analogy:* Interface standards act like physical forces, binding components into stable configurations while preserving distinct properties (e.g., USB, TCP/IP).

Molecules

- *Binding:* Chemical bonds integrate atoms into compounds with new behaviors.
- *Identity:* Distinct compounds with emergent properties beyond their elements.
- *Engineering Analogy:* Software modules or hardware components bound by design rules; emergent system behavior arises from well-structured integration.

Cells

- *Binding*: Membranes, genetic codes, and metabolic cycles maintain integrity.
- *Identity*: The cell as a self-sustaining unit of life.
- *Engineering Analogy*: Subsystems with clear boundaries and internal processes (e.g., avionics suites, autonomous vehicles) that preserve autonomy while interacting with wider systems.

Organisms

- *Binding*: Physiology, behavior, and coordination of subsystems.
- *Identity*: The organism as a functioning individual capable of adaptation.
- *Engineering Analogy*: Complex engineered products (e.g., aircraft, satellites) that integrate diverse subsystems into a coherent, adaptive whole.

Ecosystems

- *Binding*: Energy flows, interdependence, and ecological niches.
- *Identity*: The ecosystem as a resilient, adaptive collective.
- *Engineering Analogy*: System-of-systems such as transportation networks or power grids, where diversity and redundancy enhance resilience.

Societies

- *Binding*: Shared values, culture, and institutions sustain collective identity.
- *Identity*: The society as a coordinated human system.
- *Engineering Analogy*: Enterprises and large-scale programs, where governance, standards, and culture bind people and technologies into effective organizations.

Cultures, Enterprises, and Systems of Systems (SoS)

- *Binding*: Governance structures, shared purpose, and adaptive institutions.
 - *Identity*: The adaptive whole, capable of learning and transformation.
 - *Engineering Analogy*: Multi-stakeholder SoS (e.g., smart cities, global telecommunications) where coherence depends on aligning governance, mission, and interoperability across independently managed systems.
-

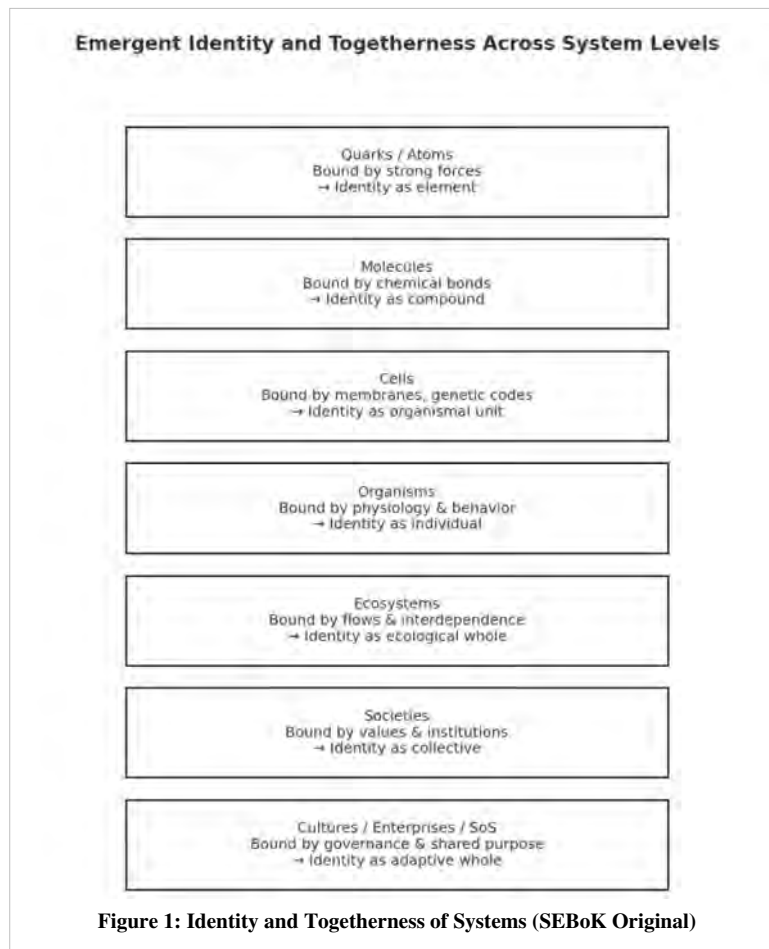


Figure 1 (Archetypes of Identity and Togetherness Across System Levels) illustrates this progression. At each step, new forms of togetherness enable the emergence of new identities, providing both inspiration and analogy for engineered systems.

Identity and togetherness shape all stages of the systems engineering life cycle:

- **System of Interest and Requirements:** Defining identity clarifies scope, boundaries, and stakeholder needs. Poorly specified identity often leads to scope creep or mismatched expectations (e.g., aerospace programs with shifting SoI definitions).
- **Architecture and Design:** Togetherness informs modularity, cohesion, and interoperability. Standards and protocols provide “membranes” for interaction, as in Modular Open System Architectures (MOSA).
- **Integration, Verification, and Validation:** Preserving identity ensures continuity and traceability across lifecycle stages. Togetherness supports reliable integration without loss of coherence.
- **Enterprise and System of Systems Engineering:** Identity at scale is maintained through governance, shared mission, and institutional alignment. Smart city SoS, for example, require governance frameworks to sustain coherence across autonomous subsystems.
- **Sustainability and Ecological Context:** Engineered systems always interact with natural systems. Designing for sustainability requires respecting ecological identities and fostering symbiotic forms of togetherness (e.g., circular resource flows, ecosystem resilience).

In practice, identity defines *what the system is*, while togetherness guides *how the parts are bound*. Integrating these concepts explicitly into lifecycle activities enables engineers to manage complexity, ensure resilience, and design systems that are effective, sustainable, and adaptive.

Implications for Systems Engineering

Attention to identity and togetherness strengthens the conceptual foundation of systems engineering by clarifying what gives a system coherence, persistence, and purpose across its life cycle. These principles illuminate the underlying logic of ISO/IEC/IEEE 15288 processes and support the broader aims of INCOSE Vision 2035 to engineer “solutions for a better world.”

Defining and Maintaining System Identity

Identity provides the basis for distinguishing the System of Interest (SoI) from its environment. Establishing and maintaining this identity enables clear scope definition, stakeholder alignment, and configuration control. Through life-cycle stages, traceability of requirements, architecture, and verification activities depends on the persistence of system identity. When identity becomes ambiguous, boundaries blur, leading to scope creep, fragmented ownership, and loss of coherence.

Achieving Coherence through Togetherness

Togetherness expresses the organizing principles that bind parts into functioning wholes. In engineering terms, this corresponds to modularity, cohesion, and well-defined interfaces that enable integration while preserving autonomy. Standards and protocols act as the “binding forces” of engineered systems, ensuring predictable interaction among subsystems. Applying these principles supports reliable integration and effective interoperability across suppliers and organizations.

Supporting Lifecycle Continuity and Change

Systems evolve through upgrades, maintenance, and contextual change. Understanding how togetherness operates allows engineers to manage change without eroding system integrity. Configuration management, interface control, and verification and validation all rely on maintaining the balance between preserving identity and accommodating adaptation.

Governance and Alignment in Enterprises and Systems of Systems

At enterprise and SoS scales, identity equates to shared purpose and mission coherence among independently managed systems. Togetherness is achieved through governance, standards, and culture that sustain collaboration and mutual trust. Effective SoS engineering depends on these higher-order forms of cohesion as much as on technical interoperability.

Sustainability and Societal Integration

Engineered systems inevitably interact with natural and social systems. Recognizing the togetherness of human and ecological systems highlights the responsibility to design for sustainability. Identity must extend beyond the engineered artifact to include its role within broader environmental and societal contexts. Concepts such as circular resource flows, resilience, and regenerative design exemplify this systems-level coherence.

Methodological Implications

Incorporating identity and togetherness encourages a shift from purely reductionist decomposition toward re-compositional and coherence-based design. Systems engineers can use these principles as diagnostic lenses to assess systemic integrity, identify weak coupling or fragmentation, and guide integration strategies that reinforce coherence across boundaries and life-cycle stages.

Summary

By embedding the principles of identity and togetherness within systems engineering practice, engineers can design and sustain systems that remain coherent, resilient, and ethically aligned with their wider human and ecological environments. These concepts provide a bridge between systems science and engineering application, clarifying how systems maintain continuity through change and how collective purpose and structure emerge from well-governed interconnection.

References

Works Cited

- Boulding, K.E. (1956). "General Systems Theory: The Skeleton of Science." *Management Science*, 2(3), pp. 197–208.
- Kineman, J.J. (2011). "Relational Science: A Synthesis." *Axiomathes*, 21(3), pp. 393–437.
- Koestler, A. (1967). *The Ghost in the Machine*. London, UK: Hutchinson.
- Rasmussen, L. (2024). *Seeing: Patterns in Living Systems*. Synearth Publishing.
- Troncale, L.R. (1978). *Nature's Enduring Patterns*. California State Polytechnic University, Pomona.

Primary References

- Capra, F. and Luisi, P.L. (2014). *The Systems View of Life: A Unifying Vision*. Cambridge, UK: Cambridge University Press.
- von Bertalanffy, L. (1968). *General System Theory: Foundations, Development, Applications*. New York, NY: George Braziller.
- Rosen, R. (1991). *Life Itself: A Comprehensive Inquiry into the Nature, Origin, and Fabrication of Life*. New York, NY: Columbia University Press

Additional References

None

Behavior and Dynamics of Systems

- Lead Author:
- Gary Smith

Acknowledgement

This article is part of The Nature of Systems area (KA). The article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

Behavior and dynamics describe how systems act and change over time. *Behavior* refers to the externally observable responses or outputs of a system, while *dynamics* refers to the internal processes and interactions that give rise to these behaviors. Together, they provide an internal perspective on **Form**, complementing concepts such as identity and togetherness, and distinct from the temporal rhythms described in *Cycles and Phases of Systems*.

Understanding behavior and dynamics is critical for systems engineers. Internal dynamics such as feedback, thresholds, and attractors shape how systems behave under stress, adapt to change, or collapse into dysfunction. These mechanisms underpin, but are not the same as, the recurring temporal patterns of Function described in *Cycles and Phases*.

Definitions and Perspectives

- **Behavior:** The observable outcomes of a system as it interacts with its environment.
- **Dynamics:** The internal mechanisms, feedback, flows, state transitions, that drive and constrain behavior.

Key perspectives include:

- Levels of system behavior from simple reactive to goal-seeking and learning (Boulding 1956)
- System dynamics, focusing on stocks, flows, and feedback loops (Forrester 1961)
- Systems isomorphies such as Feedback, Oscillation, and Regulation Troncale (2001)
- Anticipatory systems, where internal models shape behavior (Rosen 1985).
- Principles of systemic dynamics including stability, chaos, and adaptation (Mobus & Kalton 2015)

Transversal Context

Examples across domains illustrate how dynamics generate behavior:

- **Physical systems:** oscillations of a pendulum, turbulence, critical transitions in fluids.
- **Biological systems:** homeostasis, neural firing, adaptation to stimuli.
- **Ecological systems:** predator–prey growth and decline, resilience, sudden regime shifts.
- **Social systems:** reinforcing loops in markets, diffusion of innovations, collective behaviors.
- **Engineered systems:** feedback control in aircraft, congestion dynamics in networks, emergent effects in software.

These cases show that while behavior is what we observe, it is dynamics that explain *why* systems act as they do.

Patterns and Archetypes of System Behavior and Dynamics

Systems exhibit recurring dynamic modes generated by internal mechanisms such as feedback, nonlinearity, and state dependence. These archetypes form the **mechanistic basis** for the cycles and phases described in the companion article *Cycles and Phases of Systems*.

Stability and Equilibrium

- **Description:** A system resists change and maintains its state through balancing feedback.
- **System example:** The body's temperature regulation through sweating and shivering.
- **Engineering example:** An aircraft autopilot holding altitude by adjusting control surfaces.
- **Lesson for SE:** Stability depends on effective balancing feedback and control; poorly tuned loops can cause drift or oscillation.

Oscillation and Feedback Loops

- **Description:** Interactions of reinforcing and balancing loops generate cycles around an equilibrium.
- **System example:** Predator–prey population swings in ecosystems.
- **Engineering example:** Underdamped suspension systems producing oscillations in vehicles.
- **Lesson for SE:** Oscillations can be beneficial (rhythmic processes) or harmful (instability); understanding feedback strengths is critical.

Attractors and Dynamic Regimes

- **Description:** Systems converge toward stable points, cycles, or chaotic attractors that define long-term behavior.
- **System example:** The Lorenz “butterfly” attractor in weather models.
- **Engineering example:** Internet traffic stabilizing at a particular level of congestion.
- **Lesson for SE:** Attractors help anticipate possible end states; design should account for which regime the system may settle into.

Nonlinear Growth and Saturation

- **Description:** Reinforcing feedback leads to exponential growth until constraints produce saturation.
- **System example:** Logistic growth of bacteria in a nutrient-rich medium.
- **Engineering example:** Adoption of new technologies following an S-curve.
- **Lesson for SE:** Anticipating resource constraints and saturation points avoids overdesign and unexpected decline in performance.

Criticality and Tipping Points

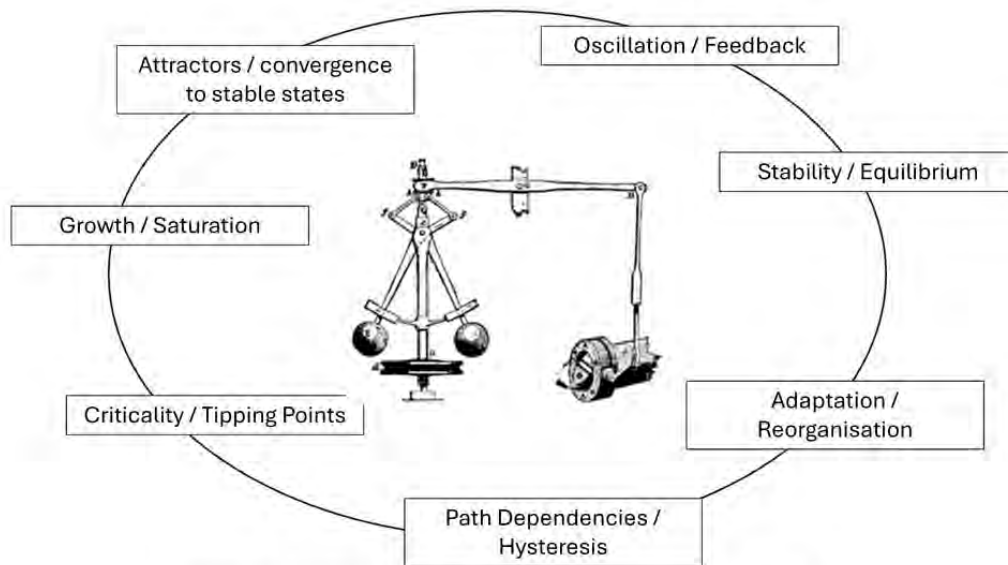
- **Description:** Small changes can push systems across thresholds, causing large regime shifts.
- **System example:** Sudden collapse of fisheries once population drops below a critical threshold.
- **Engineering example:** Cascading failures in a power grid triggered by a single line outage.
- **Lesson for SE:** Early warning indicators of criticality (stress, load, redundancy loss) are essential for risk management.

Path Dependence and Hysteresis

- **Description:** The trajectory of system behavior depends on past states, and recovery does not follow the same path as decline.
- **System example:** Ecosystems degraded by deforestation may not recover even if regrowth occurs.
- **Engineering example:** Legacy IT systems locking organizations into architectures that are difficult to replace.
- **Lesson for SE:** History matters; decisions create lock-in, so architects should anticipate long-term dependencies.

Adaptive Dynamics

- **Description:** Systems may reorganize their internal structures and feedback processes in response to disturbances, producing new modes of behavior.
- **System example:** An ecosystem reorganizing after a disturbance, shifting species dominance but maintaining function.
- **Engineering example:** Resilient infrastructure rerouting flows after a component failure.
- **Lesson for SE:** Adaptive dynamics increase resilience by enabling systems to absorb shocks and reconfigure, but can also introduce unpredictability and unintended consequences.



Governor and Throttle-Value. Figure 4. By R. Routledge - Image from "Discoveries & Inventions of the Nineteenth Century" by R. Routledge, 13th edition, published 1900., Public Domain.

Figure 1: Archetypes of System Behavior and Dynamics (SEBoK Original)

This figure illustrates seven archetypal modes of system dynamics: stability, oscillation, attractors, growth, criticality, path dependence, and adaptive dynamics. The Watt flyball governor, shown at the centre, symbolizes the internal mechanisms of feedback and regulation that generate such behaviors.

The governor demonstrates in mechanical form how centrifugal force, feedback, and thresholds can stabilize a system against perturbations, or, if mis-tuned, lead to oscillation and collapse. Similarly, across natural, social, and engineered systems, internal processes such as balancing and reinforcing feedback, attractors, and nonlinear

thresholds drive recurrent dynamic modes.

These archetypes provide analogies for systems engineering practice:

- Stability in control loops and architectural balance.
- Oscillation in under-damped systems or cyclical market behaviors.
- Attractors in enterprise dynamics and traffic flow equilibria.
- Growth and saturation in technology adoption.
- Criticality in cascading failures such as power grids.
- Path dependence in legacy architectures.
- Adaptive reorganization in resilient infrastructures.

By focusing on dynamics as the *engine* of behavior, the figure highlights how internal processes shape observable system outcomes, and how understanding these mechanisms helps systems engineers anticipate, design for, and manage complex behavior.

Theories and Frameworks

- Models using stocks, flows, and feedback - **System Dynamics** (Forrester 1961, Meadows et al 1972)
- Regulation, homeostasis, and purposeful feedback control - **Control Theory** (Åström, Murray. 2008) and **Cybernetics** (Wiener 1948)
- Emergent behaviors, criticality, sensitivity to initial conditions - **Chaos** (Gleick 1978) and **Complexity** (Mitchell 2009).
- Domain-independent isomorphies of feedback, oscillation, and regulation - **Systems Processes Theory** (Troncale 2014).
- Systems that model their environment internally and respond proactively - **Anticipatory Systems** (Rosen 1985)

Implications for Systems Engineering

System dynamics have direct consequences across the lifecycle:

- **Requirements and SoI Definition:** Anticipating desired behaviors requires attention to dynamic mechanisms, not just static functions. Poorly specified dynamics can produce instability or unintended responses.
 - **Architecture and Design:** Togetherness of parts must be structured through feedback and control. Examples include avoiding oscillations in flight control or ensuring damping in mechanical systems.
 - **Modelling and Simulation:** Dynamic models such as system dynamics, MBSE, agent-based simulation, are vital tools to test how internal processes shape external behavior.
 - **Integration, Verification, and Validation:** Traceability depends on ensuring that the integrated dynamics generate the intended behaviors across conditions.
 - **Risk and Safety:** Understanding reinforcing loops, tipping points, and collapse dynamics is essential to preventing failure cascades (e.g., power grid blackouts).
 - **Enterprise and SoS Engineering:** Emergent dynamics arise from interactions among independently managed systems; governance and coordination are needed to maintain coherence.
 - **Sustainability:** Designing with feedback from natural and social systems in mind helps create resilient and adaptive engineered systems.
-

References

Works Cited

- Åström, K. J., and R. M. Murray. (2008). *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton University Press. Available at: <https://fbswiki.org>
- Boulding, K. E. (1956). General systems theory: The skeleton of science. *Management Science*, 2(3), 197–208. <https://doi.org/10.1287/mnsc.2.3.197>
- Forrester, J. W. (1961). *Industrial dynamics*. Cambridge, MA: MIT Press.
- Gleick, J. (1987). *Chaos: Making a New Science*. New York, NY, USA: Viking.
- Meadows, D. H., Meadows, D. L., Randers, J., and W. W. Behrens III. (1972). *The Limits to Growth*. New York, NY, USA: Universe Books.
- Mitchell, M. (2009). *Complexity: A Guided Tour*. Oxford, UK: Oxford University Press.
- Mobus, G. E., & Kalton, M. C. (2015). *Principles of systems science*. New York, NY: Springer.
- Rosen, R. (1985). *Anticipatory systems: Philosophical, mathematical and methodological foundations*. Oxford, UK: Pergamon Press.
- Troncale, L. R. (2001). The future of the natural systems sciences. In G. Ragsdell & J. Wilby (Eds.), *Understanding complexity* (pp. 309–330). Boston, MA: Springer. https://doi.org/10.1007/978-1-4615-1313-1_25.
- Troncale, L. R. (2014). SPT I: Identifying fundamental systems processes for a general theory of systems. *Proceedings of the 56th Annual Meeting of the International Society for the Systems Sciences (ISSS)*, San Jose, CA, USA. Retrieved from <https://journals.iss.org/index.php/proceedings56th/article/view/2145>
- Wiener, N. (1948). *Cybernetics: Or Control and Communication in the Animal and the Machine*. Cambridge, MA: MIT Press.

Primary References

- Holling, C. S. (1973). Resilience and stability of ecological systems. *Annual Review of Ecology and Systematics*, 4, 1–23. <https://doi.org/10.1146/annurev.es.04.110173.000245>
- ISO/IEC/IEEE. (2023). *ISO/IEC/IEEE 15288:2023 systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization.
- INCOSE. (2022). *INCOSE systems engineering vision 2035: Engineering solutions for a better world*. San Diego, CA: International Council on Systems Engineering.
- Meadows, D. H. (2008). *Thinking in systems: A primer*. White River Junction, VT: Chelsea Green Publishing.
- Taleb, N. N. (2012). *Antifragile: Things that gain from disorder*. New York, NY: Random House.

Additional References

None

Cycles and the Phases of Systems

- Lead Author:
- Gary Smith
- Contributing Authors:
- Olaf Brugman, Helene Finadori, John Kineman, Tom Marzolf, George Mobus, Peter Tuddenham, Lynn Rasmussen, Hillary Sillitto, William Smith, Len Troncale, and Tyler Volk

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the lead author, who retains full responsibility.

Framing and Definitions

This article is part of The Nature of Systems knowledge area (KA). **Cycles** and **Phases** are general system concepts studied across many sciences and recognized as fundamental patterns of Function. They complement the Form-focused concepts of identity, togetherness, behavior, and dynamics, by describing how systems unfold and repeat through time:

- Cycles refer to patterns of **recurrence** - sequences of events that return to their starting conditions or repeat in recognizable rhythm.
- Phases refer to patterns of **progression** - the sequential stages through which systems emerge, develop, mature, decline, or transform

Crucially, **each phase is characterized by distinctive underlying behaviors and functions, not simply by its position in sequence.** For example:

- Early phases emphasize **exploration** and **innovation**.
- Growth phases emphasize **structuring** and **scaling**.
- Maturity phases emphasize **conservation** and **optimization**.
- Decline and release phases emphasize **breakdown** and **transformation**, opening the way for renewal.

Several definitions illustrate the breadth of the cycle concept:

- “A series of events that are regularly repeated in the same order; or move in or follow a regularly repeated sequence of events.” — Oxford English Dictionary
- “Cycles define and make things. Equally things contain cycles.” — (Volk 1995)
- “A temporal pattern.” — (Mobus and Kalton 2015)
- “The longer and more regular the series is repeated, the more predictable it becomes, until it cannot reasonably be considered a coincidence.” — Foundation for the Study of Cycles
- “Circularity is the essence of the early notion of feedback (circular causality). The notion of circularity is found in recursive computation (the use of DO loops, for example).” — (Krippendorff 1984)

Phases are equally universal. Troncale (1978) observed that “all living (even non-living) entities of the same class exhibit the same phases in their life cycle – birth, maturation, death, dissolution.” Such phase patterns are instantiated in natural systems, ecological succession, social processes, and engineered lifecycles.

Position in the Fit–Form–Function Framework:

- **Form concepts** (identity, togetherness, behavior, dynamics) describe what a system is and how it acts internally.
 - **Function concepts** (cycles, phases, purpose, capabilities) describe what a system does through time and the recurring temporal structures it expresses.
-

- **Cycles and Phases** are the first major categorization of Function, forming a bridge between the internal dynamics of systems and the observable patterns of activity that structure their engagement with context.

Introduction: Ubiquity of Cycles and Phases

Movement and evolution are fundamental in the dynamics of nature, and cycles and phases are evident across nearly every domain of inquiry. Because of their generality, they are key concepts for Systemists they provide a common language, inspire systemic thinking, and offer practical means for engaging with systems.

- **Cycles** appear as repeated patterns or rhythms of activity, from planetary orbits and ecological renewal to decision-making routines and enterprise management. They provide continuity and predictability by connecting events across time.
- **Phases** appear as ordered stages of progression, such as birth, growth, maturity, decline, and renewal in both living and non-living systems. Each phase expresses distinctive behaviors and functions, shaping the way systems interact with their environment and with one another.

Examples across the sciences illustrate their ubiquity:

- **Ecology:** adaptive cycles of exploitation, conservation, release, and reorganization (Allen & Hoekstra 1992; Gunderson 2013).
- **Social science:** planning and intervention cycles, often structured as sequential phases of inquiry and action (Sankaran et al. 2015).
- **Business and learning:** reinforcing cycles of cause and effect, as in Senge's insight that *"today's problems come from yesterday's solutions"* (Senge 2006).
- **Cybernetics:** circular causality and feedback cycles that underpin control and regulation (Krippendorff 1984).
- **Physics:** coherence and decoherence cycles in quantum mechanics (Hsiang & Ford 2009).
- **Systems biology:** modeling and interpretation cycles in living systems (Rosen 1991a, 1991b).

Cycles and phases are equally evident in engineered practice:

- Project and system lifecycles,
- Enterprise planning horizons,
- Iterative and spiral development methods,
- The recursive structure of the scientific method itself.

Together, these examples demonstrate that cycles and phases are among the most pervasive and useful concepts in systems science. They provide one of the most powerful entry points for recognizing **Function** in systems, linking natural phenomena, social processes, and engineered practice.

Cycles and Phases in Frameworks

Cycles and phases are embedded in frameworks that guide inquiry, management, and engineering practice. These illustrate how recurrence and progression structure systemic understanding and action.

- **Cynefin Framework** – A situational sense-making model highlighting transitions between ordered, complex, and chaotic domains. These transitions represent *phases* of context, while iterative exploration and re-categorization illustrate *cycles* of learning and adaptation.
- **Panarchy** – A theory of adaptive cycles in ecosystems and social–ecological systems (Gunderson & Holling 2002). It describes *phases* of exploitation, conservation, release, and reorganization, while *nested cycles* link local dynamics with broader cross-scale interactions.
- **OODA Loop** – The Observe–Orient–Decide–Act decision framework developed in military contexts. It presents a sequential set of *phases* that, when repeated rapidly, form a *cycle* enabling agility under pressure.
- **Spiral Model of Software Development** – An iterative engineering lifecycle combining sequential *phases* (requirements, design, implementation, testing) with recurring *cycles* of refinement and risk assessment.

- **Problem/Solution Systems and Action Research** – Frameworks such as Plan–Act–Observe–Reflect represent explicit *phases* of inquiry, embedded in recursive *cycles* of practice and reflection that build cumulative learning.
- **Senge’s First Law** – “*Today’s problems come from yesterday’s solutions*” (Senge 2006). This highlights the recursive nature of interventions, in which each solution cycle produces conditions that shape subsequent *phases* of systemic response.
- **Relational Holon** – Building on Rosen and extended by Kineman, the holon models the reciprocity of system and context. It frames *phases* of origin, realization, interaction, and transformation as aspects of a recurring *cycle* that repeats across scales of organization.
- **Life Cycle Management** – The management of software, products, services, and enterprises is structured by sequential *phases* of concept, development, operation, and retirement, interwoven with renewal *cycles* such as upgrades and iterations.
- **Scientific Method** – A canonical example of sequential *phases* (hypothesis, experiment, analysis, conclusion) that are repeated across *cycles* of inquiry to build cumulative knowledge.
- **Systems Engineering Life Cycle** – The discipline of systems engineering itself is organized around project *phases* (concept, development, operation, retirement) linked by iterative *cycles* of feedback, review, and refinement.

Together, these frameworks demonstrate that cycles and phases are not abstractions but **operational tools**, shaping how problems are framed, solutions are developed, and systems are managed across contexts. They also reveal a pattern: **phases provide structure for progression, while cycles enable iteration and renewal**, a distinction that will be integrated in the models and archetypes described in the following sections.

Integrated Models of Cycles and Phases

While individual frameworks illustrate specific forms of recurrence and progression, some models demonstrate how cycles and phases can be aligned across theory and practice. These integrations show coherence between systemic concepts and engineering application.

- **Systems Engineering Lifecycle and Relational Holon** – The phases of the systems engineering lifecycle (concept, development, operation, retirement) can be mapped onto the relational holon’s phases of origin, realization, interaction, and transformation. This highlights how theoretical models of system–context reciprocity can inform practical engineering lifecycles.
- **Cynefin and Lifecycle Phases** – The Cynefin framework complements lifecycle thinking by showing how sense-making transitions (ordered ↔ complex ↔ chaotic) align with, or disrupt, engineering phases. This helps practitioners recognize when iterative cycles are required rather than linear progression.

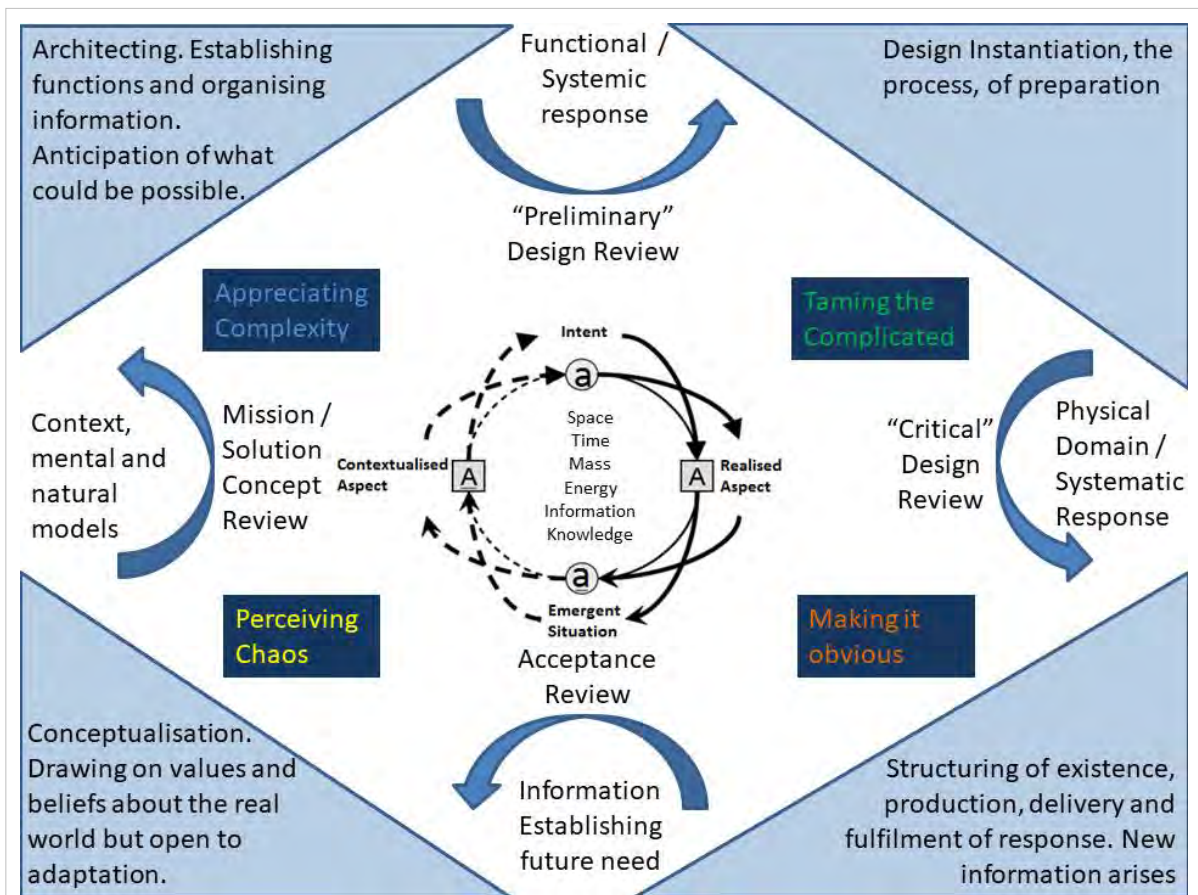


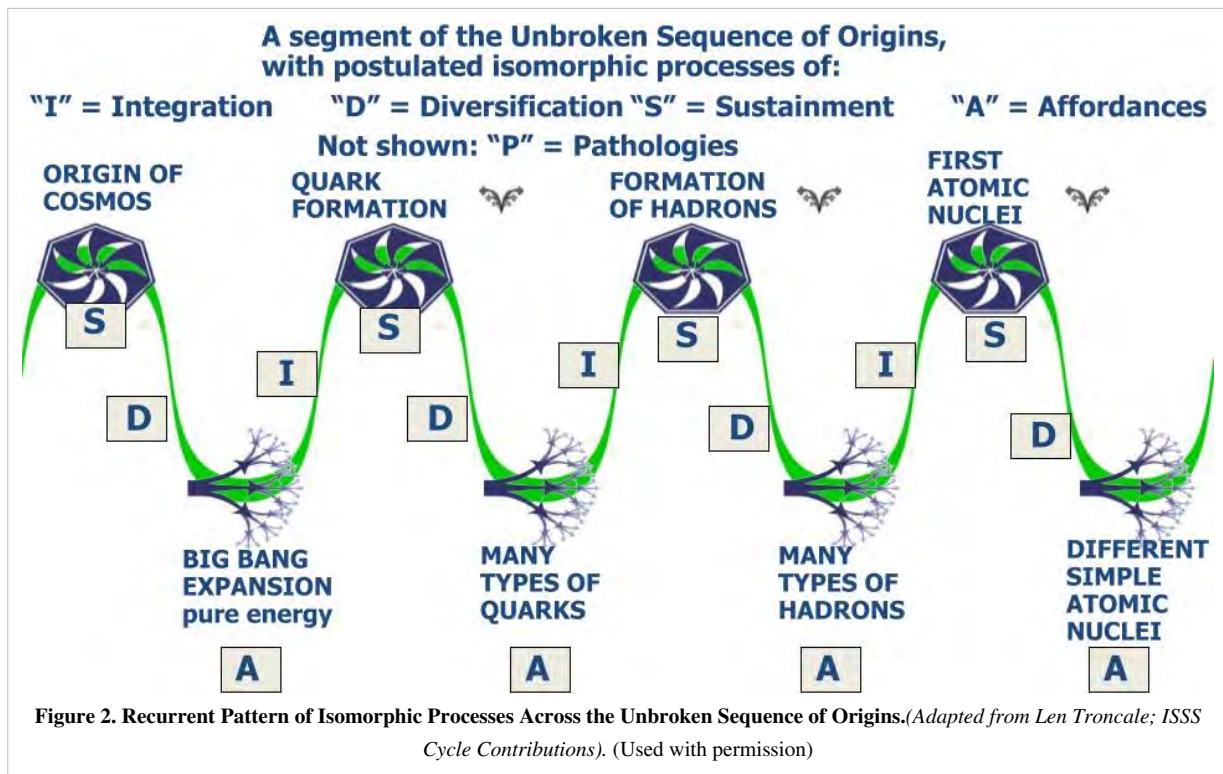
Figure 1. One Spin of the Holon – the Four Phases of System Engineering. An (Adapted from Brigitte Daniel Allegro and Gary Robert Smith.) This figure illustrates how the relational holon framework can be mapped onto the four phases of systems engineering, demonstrating coherence between theory and practice.. Used with permission.

Further integrative perspectives have been contributed through the International Society for the Systems Sciences (ISSS). These include Len Troncale's *Unbroken Sequence of Origins*, which postulates recurrent patterns of system emergence, and Tyler Volk's *metapatterns*, which reveal binding, breaking, and renewal cycles across domains. Video contributions by Troncale, Volk, and Smith further illustrate how cycles and phases connect scientific insight with systems practice (ISSS Cycle Contributions) ^[1].

Recurrent Isomorphies

Beyond applied frameworks, cycles and phases can be understood as **isomorphic processes**, recurring systemic patterns observed across multiple domains of science. These provide a deeper theoretical grounding for systems science and practice.

(Troncale 1978) systematically documented recurrent patterns of cycling, showing evidence of over 250 natural and social phenomena across seven major scientific disciplines. This work forms part of *Systems Process Theory*, which seeks to identify the universal processes underpinning system behavior and to establish a unifying empirical foundation for systems science.



This figure illustrates a waveform pattern of recurrent processes and phases spanning the Unbroken Sequence of Origins. Troncale observed that “all living (and even non-living) entities of the same class exhibit the same phases (form) in their life cycle – birth, maturation, death, dissolution.” Subsequent collaborative work extended this pattern to include additional processes such as affordances (opening new relational possibilities) and sustainment (maintenance of systemic coherence).

These recurrent isomorphies show that cycles and phases are not arbitrary constructs but reflect enduring structures of reality. By recognizing and classifying these universal processes, systemists and engineers can design, maintain, and apply lifecycle processes with greater fidelity. In this sense, isomorphies provide a scientific foundation for connecting systemic Function to practical systems engineering.

Archetypes of Cycles and Phases

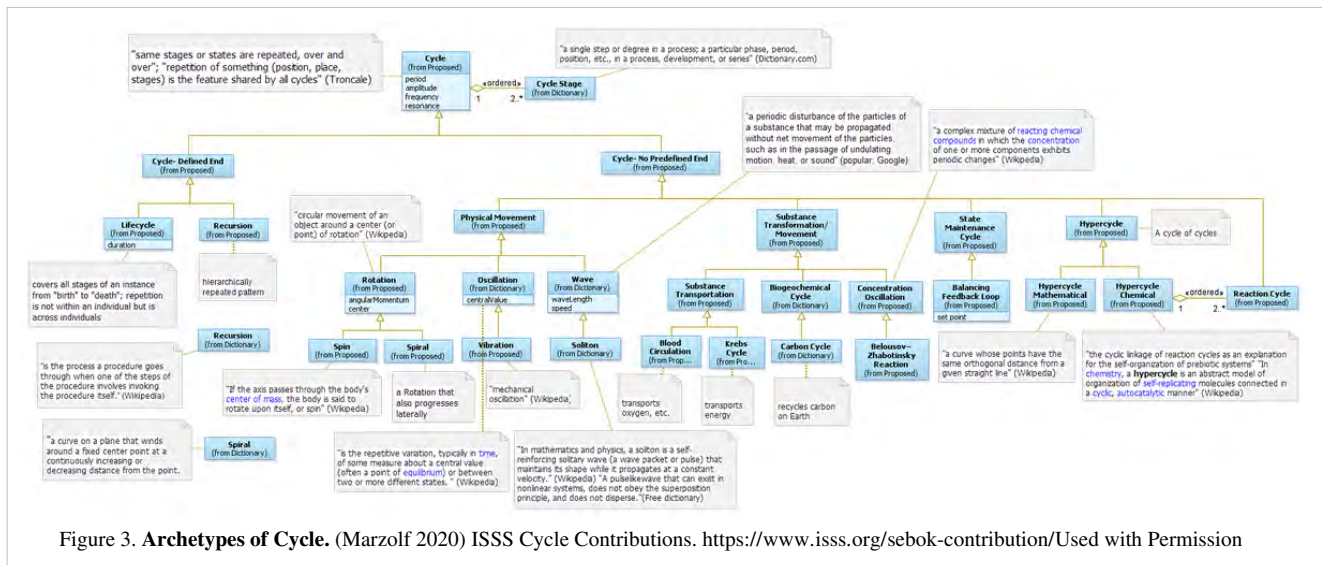
Cycles and phases can be abstracted into **archetypes** — recurrent patterns that generalize across domains of science, society, and engineering. Archetypes make the underlying logic of cycles and phases tangible, and provide a bridge between systemic theory and practical application.

Cycle Archetypes

Cycle archetypes emphasize **recurrence and rhythm** — the repeating patterns that sustain systems over time.

- **Feedback Loops** – reinforcing or balancing interactions that generate recurrence.
 - Found in ecological predator–prey oscillations, organizational decision loops, and engineering control systems.
- **Iterative Cycles** – repetition with refinement, building cumulative improvements.
 - Evident in scientific experimentation, agile sprints, and spiral development models.
- **Nested Cycles** – cycles within cycles operating at multiple scales.
 - Examples include ecological panarchy (local ↔ global) or enterprise planning cycles embedded within strategic horizons.
- **Coupled Cycles** – interacting cycles that synchronize or conflict with one another.

- Examples include economic and political cycles, or product refresh cycles interacting with regulatory approval cycles.



Cycles manifest across domains as feedback, iteration, nesting, and coupling. Each archetype captures a distinctive way in which recurrence is structured and maintained. These archetypes provide analogies for engineering practice, supporting system control, iterative design, multi-level management, and coordination across interdependent cycles.

Phase Archetypes

Phase archetypes emphasize **progression and transformation**, the ordered stages through which systems evolve. Each phase expresses **distinctive behaviors and functions**:

- **Life Phases** – exploration (birth), structuring (growth), optimization (maturity), release (decline), renewal (reorganization).
 - Found in organisms, stars, and engineered product lifecycles.
- **Adaptive Cycles** – exploitation → conservation → release → reorganization.
 - Widely used to describe ecosystem resilience and enterprise renewal.
- **Development Phases** – design, build, operate, sustain.
 - Common to engineered projects, infrastructure systems, and the SE V-model.
- **Decision Phases** – observe, orient, decide, act.
 - Characterize situational decision-making and agile team practices.

Recognizing these archetypes reinforces that phases are not arbitrary labels but **distinct systemic states** with characteristic behaviors and functions.

Integrative Archetypes

Some systemic models combine cycles and phases, illustrating how **recurrence and progression interrelate**:

- **Troncale's isomorphies** – recurrent processes that manifest as ordered sequences of phases across the *Unbroken Sequence of Origins*.
- **Volk's metapatterns** – binding, breaking, and renewal cycles repeated across levels of emergence.
- **Relational Holon** – reciprocal phases of origin, realization, interaction, and transformation, embedded in cyclical relation between system and context.
- **Systems Engineering Lifecycles** – sequential phases (concept, development, operation, retirement) structured by iterative cycles of review and renewal.

Together, these archetypes demonstrate that:

- **Cycles structure recurrence.**
- **Phases structure progression.**
- **Integrative archetypes connect the two**, showing how systemic Function arises from the interplay of recurring rhythms and ordered transformations.

Implications for Systems Engineering

Cycles and phases underpin many methods and lifecycles in systems engineering. Recognizing their archetypes enables practitioners to align practice with systemic Function, anticipate transitions, and design for resilience.

- **Concept Definition**
 - *Phase archetype (life phases – birth)*: emphasizes exploration, creativity, and innovation.
 - *Cycle archetype (feedback loops)*: stakeholder engagement and iterative feedback ensure robustness of early concepts.
- **Design and Development**
 - *Phase archetype (growth)*: emphasizes structuring, scaling, and alignment of resources.
 - *Cycle archetype (iterative cycles)*: spiral development and agile sprints support progressive refinement.
 - *Integrative archetype (relational holon) (Blockley 2022)*: highlights the reciprocity of system–context relations in design choices.
- **Integration, Verification, and Validation (V&V)**
 - *Phase archetype (maturity)*: emphasizes assurance, stability, and optimization of function.
 - *Cycle archetype (feedback loops)*: V-model structures link requirements definition to validation, closing the loop.
 - *Phase archetype (decision phases)*: V&V reviews can be structured as OODA loops, supporting adaptive decision-making.
- **Operation and Sustainment**
 - *Phase archetype (maturity/conservation)*: emphasizes reliability, optimization, and efficiency.
 - *Cycle archetype (renewal cycles)*: upgrades, maintenance, and feedback-informed adjustments sustain long-term performance.
- **Retirement and Renewal**
 - *Phase archetype (decline/release and reorganization)*: emphasizes transformation, resource recovery, and closure.
 - *Cycle archetype (nested cycles)*: feedback from decommissioning informs the birth of next-generation systems.
- **Enterprise and System of Systems (SoS)**
 - *Cycle archetypes (nested and coupled cycles)*: illustrate how enterprise planning horizons, regulatory cycles, and product lifecycles must be harmonized.

- *Integrative archetypes (panarchy)*: reveal how cross-scale cycle alignment supports systemic resilience.
- **Risk and Resilience**
 - *Phase archetype (adaptive cycle transitions)*: tipping points occur at shifts from conservation to release, highlighting fragility.
 - *Cycle archetype (critical feedback loops)*: reinforcing loops can escalate into cascading failures, while balancing loops maintain resilience.
- **Sustainability and Long-Term Viability**
 - *Integrative archetypes (ecological and societal cycles)*: engineered systems exist within planetary boundaries and biogeochemical cycles.
 - *Phase archetypes (life and adaptive cycles)*: awareness of systemic phases supports alignment with sustainability goals and long-term resilience.

Summary:

- **Phases** embody *distinctive systemic behaviors and functions* (exploration, structuring, optimization, release, renewal).
- **Cycles** provide the *recurrence and feedback rhythms* that connect phases across time and scale.
- **Integrative archetypes** demonstrate how recurrence and progression interrelate, ensuring engineering practice remains coherent with both internal system dynamics and external contexts.

References

Works Cited

- Allen, T. F. H., and T. W. Hoekstra. (1992). *Toward a Unified Ecology*. New York, NY, USA: Columbia University Press.
- Bell, S. (2012). DPSIR = A problem structuring method? An exploration from the “Imagine” approach. *European Journal of Operational Research*, 222(2), 350–360.
- Blockley, D., Smith, G. Godfrey, P., Kineman, J. (2022) Relational holon science and Popper's 3 worlds in engineering practice. *Systems Research and Behavioural Science*.
- Gunderson, L. H. (2013). *Panarchy Synopsis: Understanding Transformations in Human and Natural Systems*. Washington, DC, USA: Island Press.
- Hsiang, J.-T., and L. H. Ford. (2009). Decoherence and recoherence in model quantum systems. *International Journal of Modern Physics A*, 24(9), 1705–1712.
- Krippendorff, K. (1984). An epistemological foundation for communication. *Journal of Communication*, 34(3), 21–36.
- Oxford English Dictionary. (2020). s.v. “Cycle.” *Oxford English Dictionary Online*.
- Rosen, R. (1991a). Beyond dynamical systems. *Journal of Social and Biological Structures*, 14(2), 217–220.
- Rosen, R. (1991b). *Life Itself: A Comprehensive Inquiry into the Nature, Origin, and Fabrication of Life*. New York, NY, USA: Columbia University Press.
- Sankaran, S., B. Dick, R. Passfield, and P. Swepson. (2015). *Effective Change Management Using Action Learning and Action Research*. Lismore, NSW, Australia: Southern Cross University.
- Senge, P. M. (2006). *The Fifth Discipline: The Art & Practice of the Learning Organization* (Rev. ed.). New York, NY, USA: Doubleday.
- Wikipedia. (2020). “List of cycles.” Available at: https://en.wikipedia.org/wiki/List_of_cycles

Primary References

- Forsberg, K., H. Mooz, H. Cotterman. 2005. *Visualizing Project Management*, 3rd ed. New York, NY, USA: J. Wiley & Sons.
- Kineman, J.J. 2017. Chapter 2, *Systems Research Framework, A Guide to Systems Research*. Springer.
- Mobus, G.E and M.C. Kalton. 2015. *Principles of System Science*. Springer.
- Troncale, L.R. 1978. *Nature's Enduring Patterns*. California State Polytechnic University.
- Volk, T. 1996. *Metapatterns*. Columbia University Press.

Additional References

- Daniel-Allegro B, Smith G.R. (2016). Exploring the branches of the system landscape, Les editions Allegro Brigitte D. ISBN 978-2-9538007-1-5.
- Kineman, J.J. (2011) Relational Science: A synthesis. *Axiomathes*, 21, 393–437.
- Mobus, G.E. 2022. *System Science: Theory, Analysis, Modeling and Design*. New York, NY, USA: Springer.
- The Foundation for the Study of Cycles. 2020. "Cycle" <https://cycles.org/>.

Videos

Video presentations on the unbroken sequence of origins by Len Troncale and on Cycles by Tyler Volk and Gary Smith are available ISSS Cycle Contributions. <https://www.issc.org/sebok-contribution/>

References

- [1] <https://www.issc.org/sebok-contribution/>

Purpose and the Capability of Systems

- Lead Author:
- Gary Smith

This article is part of The Nature of Systems area (KA).

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Framing and Definitions

This article is part of the *Systems Science* knowledge area (KA). **Purpose and capabilities** are fundamental concepts of Function. They complement the **Form-focused concepts** of identity, togetherness, behavior, and dynamics, as well as the temporal Function concepts of cycles and phases. Together, they describe *why* systems act and *how* they are able to do so.

- **Purpose** refers to the orienting role of systems: the goals, aims, or ends toward which system activity is directed. Purpose may be:
 - **Designed** — explicitly specified in engineered systems.
 - **Emergent** — arising through natural processes of selection or self-organization.
 - **Interpreted** — attributed in social or human systems through meaning-making.
- **Capabilities** refer to the enabling means through which systems realize their purpose. They express what a system can do, its functions, mechanisms, resources, and potentials, and provide the operational foundation for achieving outcomes.

Several perspectives illustrate the breadth of these concepts:

- *"Behavior, purpose, and teleology are fundamental ways of classifying systems."* — (Rosenblueth, Wiener, Bigelow 1943)
- *"Every living system contains subsystems that carry out the functions of reproducer, decider, and transducer — purposeful subsystems enabling system survival."* — (Miller 1995)
- *"The purpose of a system is what it does (POSIWID)."* — (Beer 1984)

Position in the Fit–Form–Function Framework:

- **Form concepts** describe what a system *is*: its identity, cohesion, and patterns of structure and dynamics. Purpose builds on this foundation by orienting those structures and behaviors toward outcomes, while capabilities arise from the behaviors and dynamics that give Form its power to act.
- **Function concepts** describe what a system *does*. Within Function, **purpose provides orientation** (the "why"), while **capabilities provide potential and means** (the "how").
- **Fit concepts** (value and qualities, addressed in subsequent articles) describe how well a system's purposes and capabilities align with contextual needs, constraints, and expectations.

Taken together, purpose and capabilities define the **directed activity of systems**, distinguishing them from passive structures or random processes and preparing the ground for evaluating their value in context

Introduction: Why Purpose and Capabilities Matter

Purpose and capabilities are central to understanding the **Function** of systems. **Purpose** expresses the orienting *why* of a system, while **capabilities** provide the enabling *means*. Together, they distinguish systems as agents of directed activity rather than passive aggregates of parts.

- In **natural systems**, purpose often appears as survival, reproduction, or adaptation. Capabilities such as metabolism, sensing, movement, or communication enable these purposes to be realized.
- In **social systems**, purpose may be shared goals, cultural values, or strategic aims. Capabilities include knowledge, organizational structures, and technologies that allow groups to pursue those purposes.
- In **engineered systems**, purpose is typically specified in terms of stakeholder needs and mission objectives. Capabilities are expressed in functional architectures, technologies, and performance criteria that allow the system to meet those objectives.

The relationship between purpose and capability is **dynamic**:

- Purposes may evolve or shift as systems interact with their environment.
- Capabilities may expand, degrade, or be repurposed over a system's lifecycle.
- Misalignment between declared purpose and actual capabilities is a frequent cause of system failure.

Examples across domains illustrate the universality of these concepts:

- **Biology**: organisms exhibit purposeful behavior (survival, reproduction) supported by capabilities such as immune response or sensory-motor coordination.
- **Organizations**: enterprises define strategic purposes (profit, service, sustainability) and build capabilities such as supply chains, human resources, and digital infrastructure.
- **Engineering practice**: the systems engineering process begins with defining purpose (requirements, mission needs) and proceeds through developing and validating capabilities that fulfil that purpose.

Recognizing purpose and capabilities is thus essential for:

1. Describing what systems are *for* and means how they achieve it.
2. Identifying failure modes when purposes and capabilities diverge.
3. Designing coherent engineered systems that align means with ends.

Connection to Fit and Form:

- Purpose and capabilities build on **Form** concepts, identity, togetherness, and dynamics, by directing them toward outcomes and enabling effective action.
- They also prepare the ground for **Fit**, since the value of purposes and the adequacy of capabilities are judged by their qualities in context.
- By focusing on these twin concepts, systemists and practitioners can better understand how systems generate directed outcomes, adapt to change, and fulfill their role within larger contexts.

Frameworks Addressing Purpose and Capabilities

Purpose and capabilities have been treated in multiple systemic traditions, each emphasizing different aspects of **Function**. Together, these frameworks show how the two concepts provide a foundation for understanding and designing systems.

- **General System Theory (GST)** (Bertalanffy 1968) identified *teleology* (goal-directedness) as a central feature of living systems. He argued that systems are distinguished by their tendency to maintain and pursue states in relation to their environment. Here, **identity and dynamics (Form)** are oriented toward continuity and survival through purposeful activity.
- **Cybernetics and Control Theory** (Rosenblueth, Wiener, and Bigelow 1943) distinguished purposeful from non-purposeful behavior, linking purpose to feedback and control. Cybernetic systems exhibit *goal-seeking*

through regulation and adaptation, showing how **capabilities emerge from system behaviors (Form)** and become directed toward intended outcomes.

- **Miller's Living Systems Theory (LST)** (Miller 1995) described 20 critical subsystems in living systems, including purposeful ones such as the *decider*, and capability-enabling ones such as *input/output transducers*, *reproducers*, and *regulators*. These purposeful and capability subsystems connect **togetherness and dynamics (Form)** with directed systemic Function.
- **Stafford Beer's Viable System Model (VSM)** (Beer 1984) emphasized *viability* as the ultimate purpose of organizations. Capabilities are distributed across recursive levels of management and operations, with systemic purpose emerging from coherent interactions across levels. VSM illustrates how **system purpose and distributed capabilities** must align to maintain Fit within a changing environment.
- **Management Science and Enterprise Models** Purpose is expressed in vision, strategy, and goals, while enterprise capabilities are catalogued and measured through maturity models and frameworks (e.g., Capability Maturity Model Integration, CMMI). These frameworks make explicit the connection between **declared purpose and institutional capabilities**, while increasingly assessing them through *qualities* such as maturity, agility, or resilience.
- **Systems Engineering Practice** In engineered systems, purpose is made explicit through mission statements, stakeholder needs, and requirements. Capabilities are defined in functional architectures, operational concepts (CONOPS), and performance parameters. Modern practice increasingly emphasizes *capability engineering* (e.g., DoDAF, MODAF, NATO Architecture Frameworks), where systems are designed to deliver capabilities that enable mission purposes. Here, the transition to **Fit** becomes clear: capabilities are judged by their *qualities* (e.g., performance, reliability, safety, adaptability) in meeting stakeholder value.

Shared Insight:

Across these traditions, a common pattern emerges: **purpose provides orientation, capabilities provide means, and both must remain aligned for systems to function effectively**. Misalignment leads to inefficiency, fragility, or failure, while coherence supports viability and resilience.

Integrative Perspectives

Across the sciences and systems traditions, **purpose and capabilities are best understood as dual aspects of Function**. Purpose defines *orientation*, the direction or aim of a system's activity, while capabilities define *potential*, the means and capacities that allow that purpose to be realized.

Purpose and Capability as Complementary

- **Purpose without capability** is aspirational but unrealized.
- **Capability without purpose** is unfocused or misapplied.
- **Coherence** arises when purpose and capability are aligned, allowing systems to achieve meaningful outcomes within their context.

Emergence and Scale

- **Purpose** often emerges at higher levels of organization (e.g., the purpose of an organ derives from its role within the body; the purpose of a subsystem derives from its role in the system).
- **Capabilities** are typically instantiated at lower levels (e.g., the cells of the organ provide metabolic and functional capacity).
- Together, this creates a **holarchic relationship**: higher-level purposes guide, while lower-level capabilities enable.

Relational Holon Perspective

From the relational holon viewpoint (Blockley 2025), purpose and capability can be situated in reciprocal quadrants of system activity:

- **Purpose** is associated with reflective and orienting aspects, the capacity to define direction, value, or intent.
- **Capabilities** are associated with realization and action, the capacity to enact, regulate, and sustain system functioning.
- The **cycle of praxis** connects the two: purpose shapes the design and mobilization of capabilities, while capabilities constrain and inform the evolution of purpose.

Systemic Integration

- In **natural systems**, purposes like survival or reproduction are coupled with capabilities such as metabolism, reproduction, and adaptation.
- In **social systems**, collective purposes (e.g., justice, prosperity) are pursued through cultural, institutional, and technological capabilities.
- In **engineered systems**, explicitly declared purposes (e.g., mission needs) are realized through designed capabilities (e.g., functions, performance characteristics).

In summary, purpose and capabilities are **integrative systemic concepts**. They represent the ongoing dialogue between *ends and means*, *orientation and potential*, *why and how*. Their coherence is foundational to the viability and effectiveness of all systems, natural or artificial.

Bridge to Fit: The adequacy of capabilities, and the legitimacy of purposes, are ultimately judged by their **qualities in context**, a theme taken up in the subsequent article on *Value and Qualities of Systems*.

Archetypes of Purpose and Capability

Purpose and capabilities manifest in recurring **archetypes**, systemic patterns that appear across natural, social, and engineered systems. These archetypes make the abstract concepts of purpose and capability tangible and provide useful analogies for systems engineering practice.

Purpose Archetypes

Systems exhibit a range of orienting purposes, from basic survival to transcendent aims. Common archetypes include:

- **Survival** – maintaining existence within an environment.
 - *Examples:* cellular repair in biology; safety-critical functions in engineered systems.
- **Growth and Expansion** – increasing scale, influence, or resources.
 - *Examples:* ecological succession; market expansion in enterprises.
- **Coherence and Stability** – maintaining balance, efficiency, or systemic order.
 - *Examples:* homeostasis in organisms; redundancy and stability measures in engineered systems.
- **Adaptation and Resilience** – adjusting purpose in response to change.
 - *Examples:* species adaptation; organizational pivot strategies.
- **Transcendence** – pursuing goals beyond immediate survival or efficiency.
 - *Examples:* evolution of consciousness; societal aims such as justice or sustainability.

Capability Archetypes

Capabilities are the systemic means by which purposes are enacted. Recurrent capability archetypes include:

- **Sensing** – perceiving and gathering information.
 - *Examples:* sensory organs in animals; sensors in cyber-physical systems.
 - *Miller's Living Systems:* Input transducer subsystem.
- **Processing** – interpreting, evaluating, or deciding.
 - *Examples:* neural processing in organisms; decision-support systems in enterprises.
 - *Miller's Living Systems:* Decider subsystem.

- **Acting** – effecting change through energy or force.
 - *Examples:* muscular movement; actuators in machines.
 - *Miller's Living Systems:* Effector subsystem.
- **Communicating** – transmitting signals or coordinating with others.
 - *Examples:* language in humans; network protocols in ICT systems.
 - *Miller's Living Systems:* Internal and external transducers.
- **Adapting** – modifying structure or behavior to sustain viability.
 - *Examples:* genetic mutation and learning; adaptive control in engineering.
 - *Miller's Living Systems:* Reproducer subsystem.

Integrative Archetypes

Some archetypes combine both purpose and capability, illustrating their interdependence:

- **Adaptive Capacity** – the capability to change purpose in response to context.
- **Purposeful Orientation** – the alignment of multiple capabilities around a shared purpose.
- **Viability** – the systemic capacity to sustain purpose through distributed capabilities across scales (Beer's VSM).

Together, these archetypes show that purpose provides orientation (the “why”), capabilities provide potential (the means “how”), and their integration enables systems to thrive, adapt, and innovate. The qualities of capabilities, such as reliability, resilience, or efficiency, determine whether these archetypes achieve Fit within their context

Implications for Systems Engineering

Purpose and capabilities are central to systems engineering practice. They underpin requirements definition, architecture, verification, and lifecycle management. Recognizing their archetypes helps engineers design systems that are both coherent and viable.

Defining System Purpose

- In SE, purpose is captured in **stakeholder needs, mission statements, and requirements**.
- Purpose archetypes guide orientation: survival (safety), growth (scalability), stability (coherence), adaptation (resilience), or transcendence (sustainability, societal goals).
- A clear statement of purpose provides the orienting “north star” for design and development.

Engineering for Capabilities

- Systems are realized through **capabilities** defined in functional architectures and operational concepts (CONOPS).
- Capability archetypes translate into practice: sensing (data collection), processing (control logic), acting (effectors), communicating (interfaces), and adapting (reconfiguration).
- Capability engineering frameworks (e.g., DoDAF, MODAF, NATO) formalize this process by defining what a system must be able to do before specifying how it will do it.

Alignment of Purpose and Capabilities

- **Alignment is critical:** declared purpose must be achievable by available capabilities.
- Misalignment leads to:
 - *Overreach* — purposes declared without sufficient enabling capabilities.
 - *Drift* — capabilities developed without serving the core purpose.
- Successful alignment enables coherence, stakeholder confidence, and operational success.

Lifecycle Dynamics

- **Purposes evolve** over time as needs and environments change.
- **Capabilities degrade, adapt, or expand** as systems are sustained, upgraded, or reconfigured.

- Engineers must anticipate these dynamics by designing adaptive architectures and sustainment strategies that preserve alignment.

Enterprise and System-of-Systems (SoS) Contexts

- At enterprise and SoS levels, purposes are often plural, reflecting diverse stakeholders.
- Capabilities must therefore be **modular, interoperable, and scalable**, enabling systems to serve multiple or shifting purposes.
- Nested and coupled cycle archetypes (from earlier articles) highlight the importance of coordinating purposes and capabilities across levels.

Risk, Resilience, and Sustainability

- **Risk** arises when critical purposes depend on fragile or narrowly defined capabilities.
- **Resilience** requires redundant or adaptive capabilities to sustain purpose through disruption.
- **Sustainability** demands that engineered purposes and capabilities remain coherent with ecological and societal contexts over the long term

Summary

Purpose and capabilities are twin pillars of Function. Purpose provides orientation, the *why* of a system. Capabilities provide potential, the *how* through which purposes are realized. Together, they distinguish systems as directed, effective entities rather than passive aggregates of parts.

Key insights include:

- Purpose can be **designed** (engineered), **emergent** (natural), or **interpreted** (social).
- Capabilities encompass **sensing, processing, acting, communicating, and adapting**, and can be mapped to purposeful subsystems identified in systems science (e.g., Miller's Living Systems Theory).
- Archetypes of purpose (survival, growth, coherence, adaptation, transcendence) and capability (sensing, processing, acting, communicating, adapting) provide recurring patterns that guide analysis and design.
- Integrative archetypes, such as adaptive capacity, purposeful orientation, and viability, illustrate the interdependence of ends and means.
- Misalignment between purpose and capability is a frequent source of system fragility, while coherence supports resilience and sustainability.

Connection to Form: Purpose builds on the identity and togetherness of systems, orienting their behaviors and dynamics toward directed outcomes. Capabilities arise from these dynamics, providing the means by which systems act.

Connection to Function-in-time: Cycles and phases describe how systems progress and recur through time. Purpose and capabilities define *what those temporal processes are for* and *how they are enabled*.

Connection to Fit: The adequacy of purposes and the effectiveness of capabilities are ultimately judged by their **qualities in context**, such as reliability, adaptability, efficiency, and sustainability. These qualities determine whether systems are truly fit for purpose within their environment.

By clarifying purpose and capabilities, this article establishes a foundation for understanding the **directed activity of systems**, while preparing the way for the next article in this series: *The Value and Qualities of Systems*.

References

Works Cited

- Beer, S. (1984). The viable system model: Its provenance, development, methodology and pathology. *Journal of the Operational Research Society*, 35(1), 7–25. <https://doi.org/10.1057/jors.1984.2>
- Bertalanffy, L. von. (1968). *General System Theory: Foundations, Development, Applications* (Rev. ed.). New York, NY, USA: George Braziller.
- Blockley, D, Smith G. (2025). Relational holon: Systems science and information theory in engineering practice and beyond. *Journal of Information Science*. Advance online publication. <https://doi.org/10.1177/01655515251353192>
- Miller, J. G. (1995). *Living Systems*. Niwot, CO, USA: University Press of Colorado.
- Rosenblueth, A., Wiener, N., & Bigelow, J. (1943). Behavior, purpose and teleology. *Philosophy of Science*, 10(1), 18–24. <https://doi.org/10.1086/286788>

Primary References

- INCOSE. (2023). *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities* (5th ed.). San Diego, CA, USA: International Council on Systems Engineering.
- ISO/IEC/IEEE. (2023). *ISO/IEC/IEEE 15288:2023 Systems and Software Engineering — System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE).
- Blockley, D., Curtis, T., Godfrey, P., & Littlejohn, G. (2023). *Designing for the Unexpected: Principles of Engineering Systems for the 21st Century*. London, UK: Routledge.
- Checkland, P. (1999). *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons.
- Lawson, H. (2010). *A Journey Through the Systems Landscape*. London, UK: College Publications.
- Rousseau, D., Billingham, J., Wilby, J., & Calvo-Amodio, J. (2018). General systems theory: Development of the conceptual basis. *Systems Research and Behavioral Science*, 35(6), 687–694. <https://doi.org/10.1002/sres.2544>

Additional References

None

Value and the Quality of Systems

- Lead Author:
- Gary Smith

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Framing and Definitions

This article is part of the *Nature of Systems* knowledge area (KA). It addresses two essential concepts within the **Fit** dimension of the Fit–Form–Function (FFF) triad:

- **Value** refers to the contextual significance or worth of a system, judged relative to stakeholder goals, ecological constraints, or societal purposes.
- **Qualities** are the attributes through which a system's capabilities are expressed and evaluated, such as reliability, resilience, elegance, or sustainability.

In this framing:

- **Form** describes what a system is.
- **Function** describes what a system does.
- **Fit** describes how well a system aligns with its environment and purpose.

“The quality of a system is the degree to which it satisfies the stated and implied needs of its various stakeholders.”

— (ISO/IEC 25010 2011)

“The purpose of a system is what it does (POSIWID)... but how well it does it determines its viability.”

— (Beer 1984)

“Stability, equilibrium, and regulation are universal qualities observed across natural systems.”

— (Troncale 1978)

Introduction: Why Value and Qualities Matter

Systems engineering often emphasizes purpose and capability, what systems are for and how they perform. However, to determine whether a system is **fit for purpose**, we must evaluate both:

- **Qualities:** How well are capabilities expressed?
- **Value:** Why does it matter in a given context?

These evaluations are **relational and dynamic**:

- In **natural systems**, qualities such as resilience and redundancy determine survival value.
- In **social systems**, value is framed by cultural, ethical, and institutional goals.
- In **engineered systems**, system qualities drive stakeholder satisfaction, mission success, and lifecycle viability.

As contexts evolve, the relative value of qualities may change. For example, climate adaptation shifts emphasis toward sustainability and resilience, qualities once subordinate to efficiency or cost.

Recognizing these shifting balances is essential for:

- Prioritizing system requirements
 - Managing trade-offs among qualities
-

- Ensuring long-term value beyond immediate utility

Frameworks Addressing Value and Qualities

A range of systemic and engineering traditions address Fit through either value, qualities, or both:

Framework	Key Contribution
General Systems Theory (GST)	Emergent system properties, adaptability, coherence, robustness, inform evaluation (Bertalanffy 1968).
Troncale’s System Isomorphies	Recurring qualities like equilibrium and regulation appear across domains (Troncale 1978).
Living Systems Theory	Viability arises from subsystem qualities like throughput, redundancy, and integrity (Miller 1995).
Cybernetics	Control qualities such as feedback responsiveness and robustness determine system effectiveness (Wiener 2019; Ashby 1956).
Viable System Model (VSM)	Organizational viability is tied to variety-handling and recursive regulation (Beer 1984).
INCOSE & ISO Standards	Codify measurable system qualities such as reliability, usability, and sustainability (ISO/IEC 25010; INCOSE SE Handbook 2023).
Management Science	Value is modeled through performance metrics, stakeholder satisfaction, and systemic contribution (Ackoff 1971; Checkland 1999).

Each tradition affirms that qualities shape performance, while value determines contextual worth.

Integrative Perspectives

Value and Qualities as Dual Aspects of Fit

- **Value without qualities** is abstract, a claim without demonstration.
- **Qualities without value** are technical features without systemic relevance.

Together, they form the lens of Fit: **value asks “why it matters”**; **qualities show “how well” it is achieved**.

Emergence and Scale

- **Qualities** emerge from interactions between Form and Function.
- **Value** emerges in relation to systems of reference, stakeholders, environments, ecosystems, or societal structures.

This dual emergence implies that **Fit is always contextual and multi-level**.

Relational Holon Perspective

Building on Blockley (2025), the **relational holon** offers a dynamic view:

- **Qualities** correspond to the realization and actualization quadrants, how the system enacts itself.
- **Value** arises in the reflective and orienting quadrants, where meaning, ethics, and purpose emerge.

Archetypes of Value and Qualities

Value and qualities manifest in recurring evaluative patterns across domains.

Value Archetypes

Archetype	Description	Example
Utility	Direct usefulness	A solar panel's energy output
Exchange	Interoperability or trade	API compatibility
Intrinsic Worth	Independent significance	Biodiversity, cultural assets
Systemic Contribution	Enables higher-level functionality	Keystone species; transport infrastructure
Transformative Value	Enables paradigm shifts	AI systems; emergence of consciousness

Quality Archetypes

Archetype	Description	Example
Reliability	Consistent performance	Fault-tolerant hardware
Resilience	Recovery from disturbance	Forest regrowth; network redundancy
Adaptability	Reconfiguration under change	Agile organizations; evolutionary traits
Efficiency	Resource-performance ratio	Metabolic pathways; lean manufacturing
Sustainability	Long-term endurance in context	Closed-loop cycles; circular economy

Integrative Archetypes

Archetype	Description
Fitness for Purpose	Adequate qualities aligned with intent
Systemic Elegance	Simplicity, sufficiency, and coherence
Alignment with Context	Harmonization with ecological, societal, or stakeholder realities

Implications for Systems Engineering

Defining Value

Value in SE spans:

- **Operational utility** (mission success)
- **Stakeholder satisfaction** (usability, safety)
- **Long-term contribution** (sustainability, resilience)

Value archetypes help clarify these distinctions and link design choices to contextual relevance.

Engineering for Qualities

System qualities are:

- **Specified** as non-functional requirements
- **Assessed** through verification, validation, and model-based methods
- **Balanced** through trade-off analysis

Standards such as ISO/IEC 25010 and 15288 offer a structured vocabulary for evaluating qualities.

(Rousseau 2019) proposes *systemic virtues*, such as coherence, elegance, and resilience, as foundational qualities that link system design to broader systemic contribution and aesthetic value, offering a promising bridge between engineering practice and systems science.

Trade-offs and Lifecycle Change

- Systemic trade-offs (e.g., performance vs. maintainability) must be stakeholder-informed and lifecycle-aware.
- Both **qualities and perceived value change over time**, necessitating re-evaluation across design, operation, and decommissioning.

System-of-Systems and Enterprise Engineering

- Large-scale systems require balancing diverse and sometimes conflicting stakeholder values.
- Enterprise architecture, value network analysis, and resilience engineering help navigate this complexity.

Sustainability and Planetary Fit

Systems that harm ecological integrity lose long-term value, regardless of short-term utility. SE must increasingly:

- Address **planetary boundaries** (Rockström et al. 2009)
- Align value models with **ecosystem services**
- Adopt **circularity** and **regenerative design** principles

Summary

Value and qualities form the evaluative backbone of the *Fit* dimension in systems thinking:

- **Value** is contextual significance—why a system matters to stakeholders, ecosystems, or society.
- **Qualities** describe how well a system performs across dimensions like reliability, adaptability, and sustainability.
- **Together**, they determine whether a system is truly fit for its environment and purpose, not only in theory but across its lifecycle and across scales.

This article completes the transition from what systems are (*Form*) and do (*Function*) to how they are judged (*Fit*). The next article, *Consciousness and the Experience of Systems*, extends this evaluation into perceptual and experiential dimensions.

References

Works Cited

- Ackoff, R. L. (1971). Towards a system of systems concepts. *Management Science*, 17(11), 661–671.
- Ashby, W. R. (1956). *An Introduction to Cybernetics*. London, UK: Chapman & Hall.
- Beer, S. (1984). The viable system model: Its provenance, development, methodology and pathology. *Journal of the Operational Research Society*, 35(1), 7–25.
- Bertalanffy, L. von. (1968). *General System Theory: Foundations, Development, Applications*. Revised edition. New York, NY, USA: George Braziller.
- Blockley, D, Smith G. (2025). Relational holon: Systems science and information theory in engineering practice and beyond. *Journal of Information Science*. <https://doi.org/10.1177/01655515251353192>
- Capra, F., & Luisi, P. L. (2014). *The Systems View of Life: A Unifying Vision*. Cambridge, UK: Cambridge University Press.
- Checkland, P. (1999). *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons.
- INCOSE. (2023). *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities* (5th ed.). San Diego, CA, USA: International Council on Systems Engineering.
- ISO/IEC. (2011). *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. Geneva, Switzerland: International Organization for Standardization and International Electrotechnical Commission.
- ISO/IEC/IEEE. (2023). *ISO/IEC/IEEE 15288:2023 — Systems and Software Engineering — System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers.
- Miller, J. G. (1995). *Living Systems*. Niwot, CO, USA: University Press of Colorado.
- Rockström, J., Steffen, W., Noone, K., Persson, Å., Chapin III, F. S., Lambin, E. F., ... & Foley, J. A. (2009). A safe operating space for humanity. *Nature*, 461(7263), 472–475. <https://doi.org/10.1038/461472a>
- Rousseau, D., Billingham, J., & Calvo-Amodio, J. 2019. "Systemic Virtues as a Foundation for a General Theory of Design Elegance." *Systems Research and Behavioral Science*, 36(5): 656–667. Chichester, UK: Wiley.
- Troncale, L. R. 1978. "Linkage Propositions between Fifty Principal Systems Concepts." In *Applied General Systems Research*, edited by G. J. Klir, NATO Conference Series, vol. 5, 29–52. Boston, MA, USA: Springer. https://doi.org/10.1007/978-1-4757-0555-3_2
- Volk, T. (2017). *Quarks to Culture: How We Came to Be*. New York, NY, USA: Columbia University Press.
- Wiener, N. 2019. *Cybernetics: Or Control and Communication in the Animal and the Machine*, 2nd ed. Cambridge, MA, USA: MIT Press. Available at: <https://direct.mit.edu/books/oa-monograph/4581>

Primary References

- Ackoff, R. L. (1971). Towards a system of systems concepts. *Management Science*, 17(11), 661–671.
- Capra, F., & Luisi, P. L. (2014). *The Systems View of Life: A Unifying Vision*. Cambridge, UK: Cambridge University Press.
- Maier, M. W., & Rechtin, E. (2009). *The Art of Systems Architecting*, 3rd ed. Boca Raton, FL: CRC Press.
- Ulrich, W. (1994). *Critical Heuristics of Social Planning: A New Approach to Practical Philosophy*. Chichester, UK: Wiley.
- Volk, T. (2017). *Quarks to Culture: How We Came to Be*. New York, NY, USA: Columbia University Press.

Additional References

None

Consciousness and the Experience of Systems

- Lead Author:
- Gary Smith

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the lead author, who retains full responsibility.

Framing and Definitions

This article is part of the "Nature of Systems" knowledge area. Experience and consciousness are foundational concepts in the Fit dimension, complementing the Form-oriented concepts of identity, togetherness, behavior, and dynamics, and the Function-oriented concepts of cycles, phases, purpose, and capabilities. These two concepts describe how systems participate in and make sense of their environments.

- **Experience** is how a system encounters, senses, and responds to its environment. All systems experience the world to some degree, but complexity increases the richness and nuance of this encounter.
- **Consciousness** is the integration of experience into awareness. It ranges from basic responsiveness in physical systems to reflective, intentional cognition in higher-order systems.

These concepts reflect the shift from describing systems mechanistically to understanding them as participants in a relational, evolving whole.

"Living systems are cognitive systems, and living as a process is a process of cognition." — (Maturana & Varela 1980)

"Anticipatory systems contain models of themselves and their environment, enabling them to act in present terms for future states." — (Rosen 1985)

"All conscious systems share the capacity to integrate information across a whole, though in differing degrees." — (Tononi 2004)

"Consciousness can be understood at multiple levels, with richer forms emerging with greater systemic complexity." — (Jonkisz 2016)

Position in the Fit–Form–Function Framework:

Experience and consciousness are the apex Fit concepts. They govern how systems interpret qualities, ascribe value, and evolve in relation to their context.

Introduction: Why Experience and Consciousness Matter

Experience and consciousness clarify how systems live in, sense, and adapt to their context. While typically associated with human cognition, these concepts generalize to all systems as properties that scale with complexity.

- **Physical systems** experience interaction through forces and energy exchanges.
- **Biological systems** sense, perceive, and adapt.
- **Social systems** create shared meaning, culture, and governance.
- **Engineered systems** increasingly require attention to user experience, AI autonomy, and situational awareness.

These concepts help bridge:

- **Qualities** (how well a system performs)
- **Value** (why it matters in a context)
- **Experience** (how systems encounter context)
- **Consciousness** (how systems interpret and act on this encounter)

Understanding experience and consciousness allows engineers to design systems that support trust, ethical behavior, and long-term sustainability.

Frameworks Addressing Experience and Consciousness

A variety of traditions offer perspectives on how systems experience and integrate awareness.

Philosophical Traditions

- **Phenomenology** (Husserl, Merleau-Ponty 1931): focuses on lived experience as the foundation of meaning.
- **Mind-body problem & qualia**: address whether consciousness is reducible or emergent.
- **subjective experience**: (Chalmers 1996) consciousness poses a "hard problem" of .

Neuroscience and Cognitive Science

- **Integrated Information Theory (IIT)** (Tononi 2004, 2012): consciousness correlates with the degree of integrated information (Φ).
- **Global Workspace Theory** (Baars 1988): models consciousness as information broadcast across subsystems.
- **Predictive Processing** (Friston 2010): consciousness emerges from prediction-error minimization.

Systems Science

- **Autopoiesis** (Maturana & Varela 1980): living systems produce and regulate themselves; cognition is intrinsic.
- **Anticipatory Systems** (Rosen 1985): consciousness is modeled as the ability to simulate future outcomes.
- **Relational Holon** (Blockley, Smith 2025): integrates system, context, and reflection.
- **Levels of consciousness**: (Jonkisz 2015): presents a graded model of consciousness across system complexity.

Engineering and Applied Frameworks

- **HSI and UX Design**: ensure that human users experience systems meaningfully and safely.
- **Autonomy levels** (e.g., SAE J3016): imply degrees of system situational awareness.
- **AI systems**: simulate perceptual and reflective capabilities without full consciousness.

Integrative Perspectives

Experience and consciousness are deeply interconnected:

- **Experience** is the encounter with environment.
- **Consciousness** is the integration and interpretation of that experience.

Their **systemic coherence** arises from:

- **Complementarity**: One grounds the other; both are needed.
- **Scaling**: All systems experience; more complex systems develop reflective awareness.
- **Holonic integration**: Systems exist within wholes, and consciousness links individual and collective perspectives.

In the **relational holon model** (Blockley, Smith 2025):

- Experience corresponds to engagement at the system-context boundary.
- Consciousness arises in the reflective and orienting quadrants.
- The cycle of praxis connects sensory input, integration, interpretation, and action.

This model bridges:

- **Natural systems** (perception, survival)
- **Social systems** (shared meaning, culture)
- **Engineered systems** (human interaction, AI autonomy)

Archetypes of Experience and Consciousness

These archetypes clarify how systems *participate* in context, and how richer systems *perceive and integrate* meaning.

Experience Archetypes	Description	Example Systems
Sensation	Direct encounter with stimuli	Sensors, immune cells, physical particles
Perception	Organizing and interpreting input	Animals, pattern recognition systems
Emotion/Affect	Valenced evaluation of experience	Human response, social trust mechanisms
Interaction	Reciprocal engagement with environment	Ecosystems, conversational agents

Consciousness Archetypes	Description	Example Systems
Awareness	Capacity to register and respond	Drivers, pilots, automated vehicles
Self-reflection	Awareness of internal state	Humans, adaptive monitoring tools
Intentionality	Goal-directed awareness	Strategic planners, autonomous agents
Collective consciousness	Shared awareness among agents	Cultures, swarm intelligence, organizations

Integrative Archetypes:

- **Anticipatory capacity**: (Rosen 1985)
- **Autopoietic cognition**: (Maturana, Varela 1980)
- **Relational reflexivity**: (Blockley, Smith 2025)

These demonstrate how experience and consciousness scale from system structure to meaningful participation.

Implications for Systems Engineering

Experience and consciousness may appear abstract, but they have direct implications for systems engineering. Engineers increasingly design systems that shape human experience, simulate conscious-like properties, or participate in collective decision processes. Addressing these concepts ensures that engineered systems remain **fit for purpose, trustworthy, and sustainable**.

Area	Implication
Human-System Integration	Design must account for human perception, trust, interpretation, and cognitive load.
Autonomy and AI	AI systems simulate conscious-like behavior (awareness, modeling, adaptation).
Ethical Design	Systems affect lived experience and require consideration of dignity, fairness, and responsibility.
Collective Systems	Organizational awareness and decision-making must align with shared meaning and goals.
Lifecycle Integration	Experience accumulates; consciousness-like properties must be embedded in adaptive feedback loops.

Systems engineering increasingly operates in domains where:

- **User experience** shapes effectiveness.
- **Trust** is essential for adoption.
- **Autonomous systems** interact with uncertain environments.

This makes experience and consciousness not optional curiosities, but core design and lifecycle concerns.

Summary

Experience and **consciousness** are the capstone concepts in the **Fit** dimension of systems. They represent how systems *participate in, perceive, and integrate meaning* from their environment.

Key insights:

- **Experience** is universal: all systems are affected by and interact with their environment. The *richness* of experience scales with complexity.
- **Consciousness** is the integration of experience into *awareness*. It includes responsiveness, anticipation, self-reflection, and intentionality.
- **Archetypes** of experience (sensation, perception, emotion, interaction) and consciousness (awareness, self-reflection, collective meaning) recur across physical, biological, social, and engineered systems.
- **Integrative concepts** such as **anticipation**, **autopoietic cognition**, and **relational reflexivity** demonstrate how experience and consciousness deepen system Fit.
- These properties underpin emerging concerns in **AI, ethics, human–system integration**, and **systemic responsibility**.

By clarifying experience and consciousness as properties of all systems, scaling with complexity from simple responsiveness to reflective awareness, we affirm that systems are participants in the greater whole. This recognition strengthens the bridge between systems science and systems engineering, ensuring that practice attends not only to technical performance but also to **meaning, trust, and responsible participation in life’s unfolding system of systems**.

References

Works Cited

- Baars, B. J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.
- Chalmers, D. J. (1996). *The Conscious Mind: In Search of a Fundamental Theory*. Oxford University Press.
- Friston, K. (2010). "The Free-Energy Principle: A Unified Brain Theory?" *Nature Reviews Neuroscience*, 11, 127–138.
- Husserl, E. (1931). *Ideas: General Introduction to Pure Phenomenology*. London: George Allen & Unwin.
- Jonkisz, J. 2015. "Consciousness: Individuated Information in Action." *Frontiers in Psychology*, 6:1035. <https://doi.org/10.3389/fpsyg.2015.01035>
- Blockley, D, Smith G. (2025). "Relational Holon: Systems Science and Information Theory in Engineering Practice and Beyond." *Journal of Information Science*. <https://doi.org/10.1177/01655515251353192>
- Maturana, H. R., & Varela, F. J. (1980). *Autopoiesis and Cognition: The Realization of the Living*. D. Reidel Publishing.
- Rosen, R. (1985). *Anticipatory Systems: Philosophical, Mathematical and Methodological Foundations*. Pergamon Press.
- Tononi, G. (2004). "An Information Integration Theory of Consciousness." *BMC Neuroscience*, 5(1), 42. <https://doi.org/10.1186/1471-2202-5-42>
- Tononi, G. (2012). *Phi: A Voyage from the Brain to the Soul*. Pantheon Books.

Primary References

- Merleau-Ponty, M. (1945). *Phenomenology of Perception*. Gallimard.
- Tononi, G., & Koch, C. (2015). "Consciousness: Here, There and Everywhere?" *Philosophical Transactions of the Royal Society B*, 370(1668), 20140167.
- Baars, B. J., Franklin, S., & Ramsoy, T. Z. (2013). "Global Workspace Dynamics: Cortical Binding and Propagation Enables Conscious Contents." *Frontiers in Psychology*, 4, 200.
- Varela, F., Thompson, E., & Rosch, E. (1991). *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press.

Additional References

- Wiener, N. (1948). *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press.
- Ulrich, W. (1994). *Critical Heuristics of Social Planning: A New Approach to Practical Philosophy*. Wiley.
- Damasio, A. (1999). *The Feeling of What Happens: Body and Emotion in the Making of Consciousness*. Harcourt Brace.
- Dehaene, S. (2014). *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Viking.

Knowledge Area: Systems Science

Systems Science

Contents of this Knowledge Area

- From Nature's Patterns to Systems Science (Gary Smith)
- History and Foundations of Systems Science (Gary Smith) (Scott Jackson, Janet Singer, Duane Hybertson, and Rick Adcock)
- Worldviews and Philosophies of Systemness (Gary Smith)
- Patterns and Principles in Systems Science (Gary Smith)
- Structure, Identity, and Function in Systems (Gary Smith)
- Emergence and Complexity (Gary Smith) (Hillary Sillitto, Sarah Sheard, and Rick Adcock)
- System Dynamics and Control (Gary Smith)
- Information, Organization, and Entropy in Systems (Gary Smith)
- Systems, Levels, and Holarchy (Gary Smith)
- Systems Science and the Scientific Enterprise (Gary Smith)
- Lead Author:
- Gary Smith

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Overview

Systems science is the study of the general principles, patterns, and processes that govern systems of all kinds, natural, social, engineered, and abstract (Bertalanffy 1968; Boulding 1956). It seeks to understand systemness: how systems form, persist, interact, adapt, and evolve. In doing so, it examines phenomena such as emergence, feedback, organization, hierarchy, adaptation, and information flow that recur across diverse domains.

Unlike traditional disciplines, which focus on specific classes of phenomena, systems science aims to identify structures and behaviors common across domains. These recurring characteristics, often referred to as isomorphies, enable recognition of similarities between otherwise disparate systems (Troncale 1978). This cross-domain perspective provides a unified framework for understanding complexity.

The intellectual foundations of systems science span multiple disciplines. General systems theory sought to identify universal principles of organization (Bertalanffy 1968). Boulding's hierarchy of systems highlighted increasing levels of complexity across natural and social domains (Boulding 1956). Cybernetics introduced the concepts of feedback, control, and communication as mechanisms of regulation (Wiener 1948; Ashby 1956).

Subsequent developments extended these ideas. System dynamics examined how feedback structures shape behavior over time (Forrester 1961). Thermodynamics and complexity science explored the emergence of order in far-from-equilibrium systems (Nicolis and Prigogine 1977). Research on autopoiesis examined how living systems maintain organization through self-production (Maturana and Varela 1980).

Together, these traditions form the foundation of modern systems science: an integrative field that explains how complex systems behave across domains.

Within the SEBoK, systems science provides theoretical foundations that support systems thinking and systems engineering practice. It helps explain how systems organize, interact, and evolve within broader technological, organizational, and environmental contexts (Checkland 1999).

Position within the SEBoK Foundations

This Knowledge Area is part of the Foundations of Systems Engineering and sits alongside several closely related knowledge areas:

- The Nature of Systems defines what systems are and how they can be described.
- Systems Thinking applies systemic concepts in practice.
- Representing Systems with Models provides formal tools for analysis and communication.
- Systems Approach Applied to Engineered Systems applies these concepts in engineering contexts.

Systems science therefore serves as a **bridge between foundational understanding and practical application**.

Scope and Intent

The purpose of this Knowledge Area is to introduce the scientific foundations of systemic understanding. It aims to:

- present the development of systems science as a multidisciplinary field
- describe recurring patterns and principles across systems
- explain phenomena such as emergence, feedback, complexity, and adaptation
- examine how systems maintain organization through structure and information
- explore multi-level organization and holarchies
- clarify relationships between systems science and other disciplines
- provide conceptual grounding for systems thinking and systems engineering

These topics support the development of systems literacy, enabling effective reasoning about complex systems.

Structure of this Knowledge Area

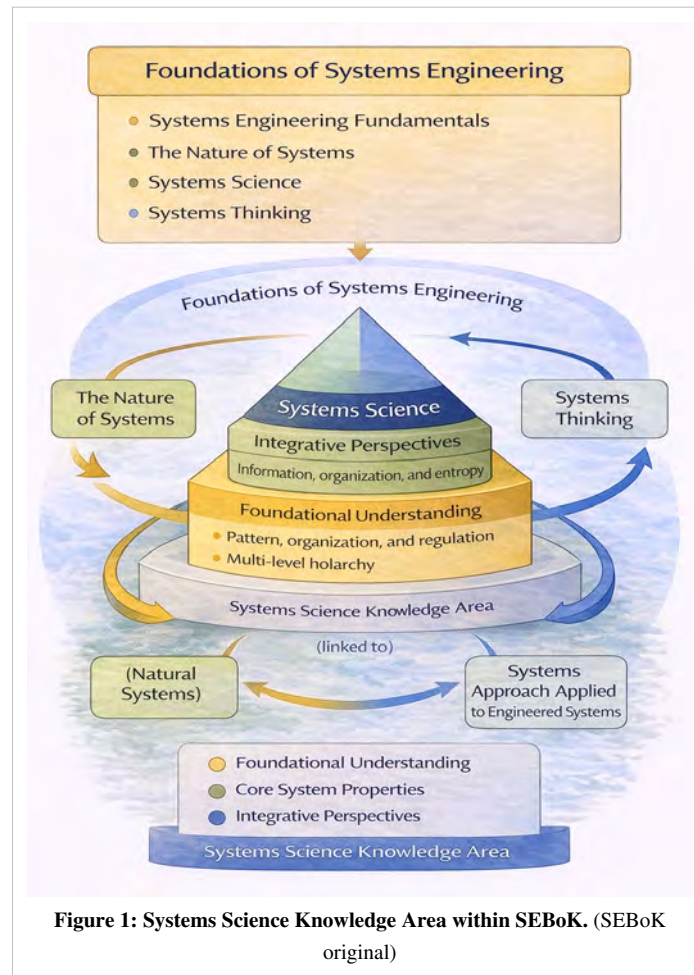
The articles in this Knowledge Area are organized to progressively develop understanding:

- From Nature's Patterns to Systems Science Observations of recurring patterns motivate systemic inquiry.
 - History and Foundations of Systems Science Development of key intellectual traditions.
 - Worldviews and Philosophies of Systemness Philosophical and epistemological perspectives.
 - Patterns and Principles in Systems Science Recurring structures and organizing principles.
 - Structure, Identity, and Function in Systems How systems maintain coherence and perform functions.
 - Emergence and Complexity How system behavior arises from interactions.
 - System Dynamics and Control Feedback, regulation, and temporal behavior.
 - Information, Organization, and Entropy in Systems Information processes underlying organization and change.
 - Systems, Levels, and Holarchy Multi-level organization and cross-scale interactions.
 - Systems Science and the Scientific Enterprise The role of systems science within the broader scientific landscape.
-

Understanding the Knowledge Area as a System

The structure of this Knowledge Area is both sequential and systemic. Each article contributes to a coherent body of knowledge while remaining interdependent with others.

- Early articles establish **empirical and conceptual foundations**
- Intermediate articles **develop core systemic mechanisms**
- Later articles integrate these into **multi-level and scientific perspectives**



The accompanying figure illustrates:

- the Systems Science Knowledge Area as a subsystem within the Foundations of Systems Engineering
- its relationships with adjacent knowledge areas
- the internal progression of topics within the Knowledge Area
- the flow of understanding from observation to integration

Interpretation of the Figure

The figure can be interpreted in three ways:

1. **Progression of understanding** Concepts develop from empirical observation to abstraction and integration.
2. **Interconnected knowledge system** Topics such as structure, dynamics, information, and levels are interrelated and mutually reinforcing.
3. **Embedded context** Systems science is situated within a broader ecosystem of knowledge areas that both inform and apply its insights.

The figure therefore represents the Knowledge Area as a **coherent and interacting system of ideas**, rather than a strictly linear sequence.

Role in Systems Engineering

Systems engineering addresses the design, integration, and lifecycle management of complex systems. Systems science supports this practice by providing:

- insight into feedback and dynamic behavior
- understanding of emergence and unintended consequences
- awareness of multi-level interactions
- principles for managing complexity and adaptation

These insights are particularly important because engineered systems operate within **interconnected social, ecological, and technical environments**.

By applying systems science, systems engineers can:

- anticipate emergent behavior
- design more resilient systems
- manage interactions across system boundaries
- operate effectively under conditions of complexity and uncertainty

Conclusion

Systems science provides a unifying framework for understanding how systems behave across domains. It demonstrates that diverse phenomena are governed by common principles of organization, interaction, and change.

Within the SEBoK, this Knowledge Area is itself structured as a coherent system of knowledge. It progressively develops understanding while maintaining conceptual integration across its topics. In doing so, it forms a critical link between foundational knowledge and applied systems thinking and engineering.

References

Works Cited

- Ashby, W. R. 1956. *An Introduction to Cybernetics*. London: Chapman & Hall.
- Bertalanffy, L. von. 1968. *General System Theory*. New York: Braziller.
- Boulding, K. E. 1956. "General Systems Theory—The Skeleton of Science." *Management Science*, 2(3).
- Checkland, P. 1999. *Systems Thinking, Systems Practice*. Wiley.
- Forrester, J. W. 1961. *Industrial Dynamics*. MIT Press.
- Troncale, L. R. 1978. "Linkage Propositions Between Systems Concepts."
- Wiener, N. 1948. *Cybernetics*. MIT Press.

Primary References

Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.

Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*, Revised ed. New York, NY, USA: Braziller.

Additional References

Maturana, H. R., and F. J. Varela. 1980. *Autopoiesis and Cognition*.

Mobus, G. E., and M. C. Kalton. 2015. *Principles of Systems Science*.

Nicolis, G., and I. Prigogine. 1977. *Self-Organization in Nonequilibrium Systems*.

Volk, T. 1996. *Metapatterns*.

From Nature's Patterns to Systems Science

- Lead Author:
- Gary Smith

This article introduces the development of systems science by tracing its roots in the recognition of recurring patterns in nature and the gradual emergence of a formal scientific field devoted to understanding them. It serves as the opening article of the Systems Science Knowledge Area (KA), building conceptually on the preceding KA on Natural Systems and preparing the ground for the subsequent exploration of the historical development of systems science.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

Across many domains of experience and scientific inquiry, observers have long recognized that the world exhibits recurring forms of organization and behavior. Cycles of growth and decay, nested structures within living organisms, balancing dynamics in ecosystems, and patterns of stability and change in social systems all reveal a patterned structure to reality. These recurring features suggest that systems, whether biological, social, ecological, or technological, share common characteristics governing how they organize, persist, interact, and evolve.

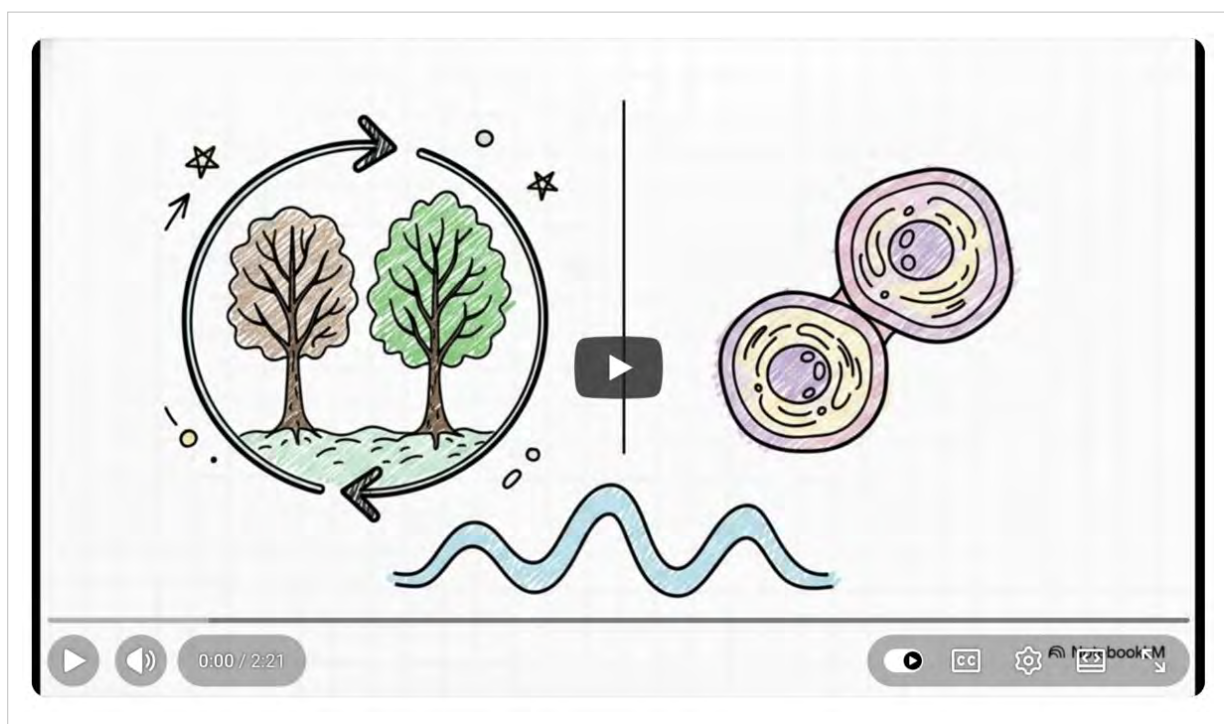
The recognition of such patterns eventually led researchers to ask whether **general principles of organization might apply across different kinds of systems**. The search for such principles ultimately contributed to the emergence of systems science as a formal interdisciplinary field.

Human engagement with systemic patterns can be understood as a progression from observation of natural organization to the development of formal scientific theories of systems. Early recognition of recurring patterns in nature inspired reflection across cultures and disciplines, which gradually evolved into structured scientific investigation.



Figure 1: From natural patterns to the emergence of systems science. (SEBoK original)

Figure 1 illustrates this conceptual progression, from patterns observed in nature, through human observation and cultural interpretation, to the formalization of systems science.



Systemic Insight across Cultures and Traditions

While systems science emerged as a formal discipline primarily during the twentieth century, systemic insight itself has much deeper historical and cultural roots.

Many intellectual and cultural traditions have long recognized patterns of **interdependence, relationality, and cyclic processes** in the natural world. Ancient philosophical traditions in both Eastern and Western cultures often emphasized the interconnected nature of phenomena and the importance of balance, harmony, and dynamic change. Concepts such as cyclical time, nested order, and relational identity appear in early philosophical writings and cosmologies across many civilizations.

Indigenous knowledge traditions around the world frequently emphasize **relational understandings of humans, ecosystems, and the natural world**, recognizing that living systems exist within webs of reciprocal relationships and dynamic environmental processes. Such perspectives often stress stewardship, adaptation, and the maintenance of balance within ecological systems.

These traditions do not constitute systems science in the modern academic sense. However, they reflect enduring human attempts to understand **patterns of organization and interdependence in the world**, demonstrating that systemic awareness is not limited to any single intellectual tradition.

Modern systems science can therefore be understood not as the origin of systemic thinking, but as a **formal scientific articulation of questions that humans have long explored across cultures and disciplines**.

Patterns in Nature and the Origins of Scientific Systems Inquiry

Systemic patterns are particularly evident in the natural world. Observations of natural phenomena reveal recurring structures and dynamics across many domains.

Examples include:

- **feedback processes** that regulate stability in biological and ecological systems
- **cyclic dynamics** such as seasonal cycles, nutrient cycles, and biological rhythms
- **hierarchical organization** across multiple levels of structure
- **flows of energy and information** that sustain system activity
- **self-organization**, where order emerges spontaneously from interacting components

These patterns appear across many scientific disciplines. Feedback regulation is central to physiology and ecology. Cyclic processes appear in ecosystems, climate systems, and planetary dynamics. Hierarchical organization appears in biological organisms, ecosystems, and complex technological infrastructures.

Modern studies of systemic patterns in nature have further highlighted the existence of **recurring structural and dynamic patterns across multiple scales of organization**, sometimes described as **meta-patterns**. These patterns include phenomena such as cycles, branching networks, gradients, nested structures, and feedback regulation that recur across biological, ecological, and physical systems (Volk 2017).

The widespread recurrence of these patterns suggests that **similar organizing principles may operate across many domains of reality**, raising the possibility of a more general science of systems.

The Limits of Reductionism and the "Systems Problem"

During the nineteenth and early twentieth centuries, scientific progress was largely driven by **reductionist approaches**, which sought to understand complex phenomena by analyzing their individual components. This approach produced major advances in physics, chemistry, and biology.

However, certain phenomena resisted explanation through reduction alone. Complex systems such as organisms, ecosystems, societies, and large technological infrastructures exhibit behaviors that arise from interactions among components rather than from the properties of the components themselves.

Recognizing this challenge, scientists began exploring new ways of understanding such phenomena. Warren Weaver famously described these challenges as **"problems of organized complexity"**, referring to situations in which large numbers of interacting elements produce structured and dynamic behavior that cannot be explained through simple statistical or reductionist models (Weaver 1948).

Understanding organized complexity required scientific approaches capable of studying systems as **organized wholes rather than collections of isolated parts**.

Toward a General Science of Systems

Several influential developments during the twentieth century contributed to the emergence of systems science.

One early precursor was the work of **Alexander Bogdanov**, whose early twentieth-century framework of *Tektology* attempted to formulate a universal science of organization across natural and social systems. Although developed independently of later systems traditions, Bogdanov's work anticipated many themes that would later appear in systems theory.

Later developments included **General Systems Theory**, developed by Ludwig von Bertalanffy, which sought to identify general principles governing the organization and behavior of open systems across biological, social, and technological domains (Bertalanffy 1968).

At the same time, **cybernetics** investigated mechanisms of communication, feedback, and control in both biological organisms and machines (Wiener 1948; Ashby 1956). Cybernetics demonstrated how feedback processes enable systems to regulate themselves and adapt to changing environments.

Kenneth Boulding further expanded these ideas by proposing a **hierarchical classification of systems**, illustrating how increasing levels of complexity appear across natural and social systems (Boulding 1956).

Together, these developments contributed to the formation of systems science as a field concerned with identifying **general patterns of organization across many domains of reality**.

Complexity, Hierarchy, and Dynamic Systems

Further developments deepened the scientific study of complex systems.

Herbert Simon examined the **hierarchical architecture of complex systems**, arguing that many complex structures evolve through the gradual aggregation of stable subsystems (Simon 1962). Simon's work helped explain why hierarchical organization is common in both natural and engineered systems.

Research in thermodynamics and nonlinear systems also revealed how **self-organization and emergent order** can arise in systems far from equilibrium (Nicolis and Prigogine 1977). These discoveries contributed to the development of modern complexity science.

Together, these developments expanded the scientific understanding of how complex systems form, behave, and evolve.

From Systemic Patterns to Systems Science

As interest in systemic phenomena grew, several complementary intellectual traditions emerged that contributed to the development of systems science. These traditions include:

- biological systems theory
- cybernetics and control theory
- systems modeling and complexity science
- systems analysis and interdisciplinary systems research

Although these traditions developed in different disciplinary contexts, they share a common focus on **relationships, interactions, and organization within complex systems**.

Systems science gradually emerged from the convergence of these streams of inquiry. The field seeks to understand **systemness itself**, the underlying principles and patterns that characterize systems across natural, social, and engineered domains.

The next article in this Knowledge Area, **History and Foundations of Systems Science**, examines how these ideas developed into a coherent body of scientific thought and how the field has evolved through contributions from multiple disciplines and intellectual traditions.

References

Works Cited

- Ashby, W. R. 1956. *An Introduction to Cybernetics*. London, UK: Chapman & Hall.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. New York, NY: Braziller.
- Bogdanov, A. 1980. *Essays in Tektology*. Seaside, CA: Intersystems.
- Boulding, K. E. 1956. "General Systems Theory—The Skeleton of Science." *Management Science* 2(3), pp. 197–208.
- Simon, H. A. 1962. "The Architecture of Complexity." *Proceedings of the American Philosophical Society*.
- Volk, T. 2017. *Quarks to Culture: How We Came to Be*. New York, NY: Columbia University Press.
- Weaver, W. 1948. "Science and Complexity." *American Scientist* 36, pp. 536–544.
- Wiener, N. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. Cambridge, MA: MIT Press.

Primary References

None.

Additional References

- Miller, J. G. 1978. *Living Systems*. New York, NY: McGraw-Hill.
- Mobus, G. E., and M. C. Kalton. 2015. *Principles of Systems Science*. New York, NY: Springer.
- Nicolis, G., and I. Prigogine. 1977. *Self-Organization in Nonequilibrium Systems*. New York, NY: Wiley.
- Warfield, J. N. 2006. *An Introduction to Systems Science*. Singapore: World Scientific.
-

History and Foundations of Systems Science

- Lead Author:
- Gary Smith
- Contributing Authors:
- Scott Jackson, Janet Singer, Duane Hybertson, and Rick Adcock

The intellectual foundations of systems science emerged through contributions from biology, engineering, mathematics, philosophy, and the social sciences, gradually shaping a more holistic understanding of complex systems and their behaviors. Foundational concepts such as general systems theory, cybernetics, feedback, emergence, and interdisciplinarity helped establish systems thinking as a distinct way of understanding relationships, interactions, and dynamic change. These historical developments flow into contemporary systems engineering practice, informing how complexity, integration, adaptation, and socio-technical interactions are understood across domains. Together, these perspectives provide critical conceptual grounding for modern systems thinking and engineering practice.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

Systems science emerged during the twentieth century as researchers across many disciplines began investigating phenomena that could not be adequately explained through reductionist approaches alone. Complex systems, including organisms, ecosystems, economies, societies, and large technological infrastructures, exhibit behaviors that arise from interactions among their components and from relationships with their environments. Understanding such phenomena required conceptual frameworks capable of studying systems as organized wholes.

The development of systems science did not originate from a single discipline or theory. Instead, it emerged through the convergence of several intellectual traditions concerned with organization, interaction, regulation, and complexity. These traditions arose in fields such as biology, engineering, mathematics, physics, and the social sciences. Together they contributed to a growing recognition that many systems, despite differences in their elements, share common patterns of organization and behavior.

Building on the preceding article, *From Nature's Patterns to Systems Science*, which describes how systemic patterns observed in nature motivated the search for general principles of organization, this article outlines the major intellectual streams that contributed to the development of systems science as a formal interdisciplinary field.

An explainer video is also provided here ^[1] to support the introduction to the topic.

Early Foundations of Systems Inquiry

Prior to the formal emergence of systems science, several scientific traditions had already begun to recognize the importance of studying organized wholes rather than isolated components.

In biology, researchers increasingly emphasized the integrated nature of living organisms. Organisms could not be fully understood solely through the study of individual parts; rather, their properties arise from coordinated interactions among multiple subsystems. Similarly, ecological research highlighted the interdependence of species and the dynamic relationships within ecosystems.

At the same time, developments in engineering and mathematics were revealing the importance of communication, control, and feedback in both natural and artificial systems. These developments helped shift scientific attention

toward the study of relationships, interactions, and regulatory processes.

An early attempt to formulate a universal science of organization was developed by **Alexander Bogdanov** in the early twentieth century. His framework, known as *Tektology*, sought to identify general organizational principles applicable across biological, social, and technological systems (Bogdanov 1980). Although largely unknown in Western scientific communities for many decades, Bogdanov's work anticipated many themes later associated with systems theory.

General Systems Theory

One of the most influential contributions to the development of systems science was **General Systems Theory (GST)**, developed by Ludwig von Bertalanffy.

Bertalanffy argued that many systems, particularly living organisms, must be understood as **open systems** that exchange matter, energy, and information with their environments (Bertalanffy 1968). This perspective contrasted with earlier mechanistic views that treated systems as closed and static.

General Systems Theory sought to identify principles of organization that apply across different scientific domains. These included concepts such as open systems, dynamic equilibrium, hierarchical organization, and interactions among system components.

Rather than replacing existing disciplines, Bertalanffy envisioned systems theory as a **transdisciplinary framework** capable of revealing common patterns of organization across biological, social, and technological systems.

Kenneth Boulding expanded these ideas by proposing a **hierarchical classification of systems**, illustrating how increasing levels of complexity emerge across physical, biological, and social domains (Boulding 1956). Boulding's framework highlighted the progressive increase in organizational complexity from simple physical structures to complex social systems.

Cybernetics and Regulation

A second major intellectual stream contributing to systems science was **cybernetics**, developed through the work of Norbert Wiener and colleagues studying communication and control in animals and machines.

Cybernetics introduced powerful concepts for understanding how systems regulate their behavior through **feedback processes** (Wiener 1948). Feedback mechanisms allow systems to monitor their own states and adjust their behavior in response to environmental changes.

W. Ross Ashby further developed these ideas through his work on regulatory systems. Ashby introduced the principle of **requisite variety**, which states that the capacity of a control system must be at least as diverse as the disturbances it must regulate (Ashby 1956). This principle highlighted the relationship between environmental complexity and the regulatory capacity required for system stability.

Cybernetics had profound influence across many disciplines, including neuroscience, engineering, computing, and organizational studies. It established feedback and regulation as central concepts within systems science.

Organizational Cybernetics and Governance Systems

The application of cybernetic principles to organizations was further developed by **Stafford Beer**, whose work on **management cybernetics** explored how organizations can maintain viability in complex and changing environments.

Beer developed the **Viable System Model**, which describes the organizational structures required for adaptive regulation and coordination within complex systems (Beer 1972). His work demonstrated how cybernetic principles could be applied to governance, management, and institutional design.

These ideas helped extend systems science beyond natural and engineering systems into the study of complex social and organizational systems.

Systems Dynamic and Modeling

Another important development within systems science was the emergence of **system dynamics**, pioneered by Jay Forrester.

System dynamics focuses on understanding how feedback structures, delays, and nonlinear interactions shape the behavior of complex systems over time (Forrester 1961). Using mathematical models and computer simulations, system dynamics allows researchers to explore how interactions among system components generate dynamic patterns such as growth, decline, oscillation, and instability.

System dynamics became widely applied in fields such as industrial management, urban planning, environmental policy, and sustainability studies.

Mathematical Systems Theory

Mathematical approaches also played an important role in the development of systems science. Researchers sought to develop formal frameworks capable of describing system behavior across diverse domains.

Anatol Rapoport contributed significantly to the development of mathematical systems theory, exploring how systems concepts could be expressed through formal models and applied to social and behavioral systems (Rapoport 1986). His work helped bridge the gap between abstract systems theory and practical applications in social science and policy.

Complexity and Self-Organization

Later developments in mathematics and physics contributed to the emergence of **complexity science**, which studies how large numbers of interacting components give rise to organized structures and emergent behavior.

Research on nonlinear dynamics and thermodynamics revealed that complex systems can spontaneously generate ordered structures under certain conditions. Studies of **self-organization** demonstrated how patterns and structures can emerge from local interactions without centralized control.

Work by Nicolis and Prigogine showed how systems operating far from equilibrium can develop structured patterns through nonlinear interactions (Nicolis and Prigogine 1977). These discoveries significantly expanded the scientific understanding of dynamic systems.

Today, complexity science investigates phenomena such as network dynamics, adaptive systems, and emergent behavior across domains ranging from physics and biology to economics and social systems.

Systems Science as an Integrative Field

Although the traditions described above emerged within different disciplines, they share a common concern with understanding systems as organized wholes whose behavior arises from relationships among components.

Systems science gradually developed as an **integrative field of inquiry** bringing together insights from biology, engineering, mathematics, physics, and the social sciences. Its central aim is to identify **general principles and patterns of organization** that apply across different types of systems.

These insights have influenced research and practice in fields including ecology, economics, organizational theory, computer science, and systems engineering.

Continuing Evolution of Systems Science

Systems science continues to evolve as researchers investigate increasingly complex socio-technical and socio-ecological systems. Contemporary research explores topics such as:

- complex networks
- adaptive systems
- resilience and sustainability
- multi-scale system dynamics
- interactions between technological, social, and ecological systems

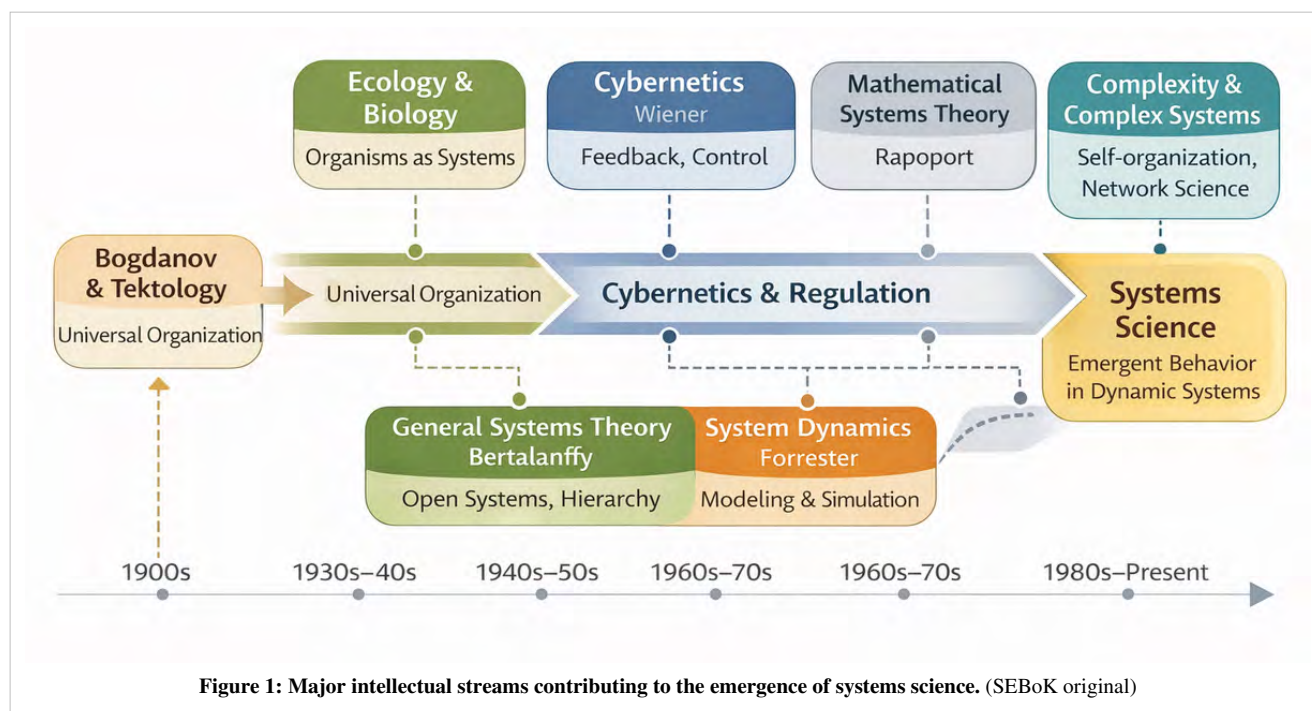
Despite the diversity of approaches, the central goal of systems science remains consistent: to understand how systems organize, interact, adapt, and evolve across different domains of reality.

Subsequent articles in this Knowledge Area explore several of the key concepts that have emerged from this interdisciplinary field.

Major Intellectual Streams in the Development of Systems Science

The development of systems science can be understood as the convergence of several intellectual streams that emerged during the twentieth century. Early work on universal organizational principles was followed by research on feedback and regulation, general systems theory, dynamic modeling, and complexity science.

The timeline in **Figure 1** illustrates several of the major contributions that shaped the development of systems science.



References

Works Cited

- Ashby, W. R. 1956. *An Introduction to Cybernetics*. London, UK: Chapman & Hall.
- Beer, S. 1972. *Brain of the Firm*. London, UK: Allen Lane.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. New York, NY: Braziller.
- Bogdanov, A. 1980. *Essays in Tektology*. Seaside, CA: Intersystems.
- Boulding, K. E. 1956. "General Systems Theory—The Skeleton of Science." *Management Science* 2(3), pp. 197–208.
- Forrester, J. W. 1961. *Industrial Dynamics*. Cambridge, MA: MIT Press.
- Nicolis, G., and I. Prigogine. 1977. *Self-Organization in Nonequilibrium Systems*. New York, NY: Wiley.
- Rapoport, A. 1986. *General System Theory: Essential Concepts and Applications*. Tunbridge Wells: Abacus.
- Wiener, N. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. Cambridge, MA: MIT Press.

Primary References

None.

Additional References

Mobus, G. E., and M. C. Kalton. 2015. *Principles of Systems Science*. New York, NY: Springer.

Prigogine, I., and I. Stengers. 1984. *Order Out of Chaos*. New York, NY: Bantam.

References

[1] <https://youtu.be/QeaavKxTuhM>

Worldviews and Philosophies of Systemness

- Lead Author:
- Gary Smith

Different worldviews and philosophies of "systemness" shape how people define, interpret, and engage with systems across disciplines and contexts. Perspectives grounded in mechanistic, organismic, cybernetic, socio-technical, and holistic traditions each emphasize different aspects of systems, including structure, behavior, purpose, relationships, adaptation, and meaning. These philosophical foundations influence how systems are modeled, analyzed, engineered, and governed, particularly when addressing complex human and organizational challenges. Recognizing and navigating multiple perspectives on systemness strengthens systems thinking by encouraging flexibility, reflection, and a deeper appreciation for the assumptions underlying systems practice.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

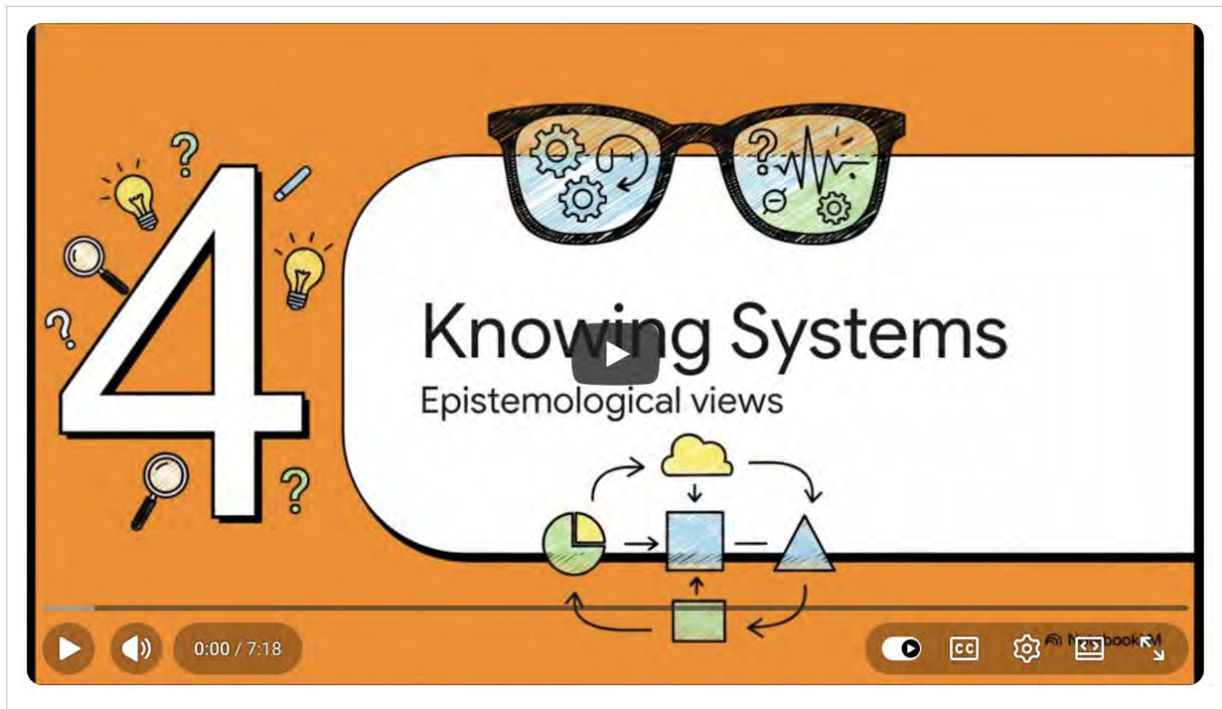
Introduction

The concept of a system is widely used across science, engineering, and many other fields of inquiry. Yet the interpretation of what constitutes a system depends strongly on underlying philosophical assumptions about the nature of reality, knowledge, and meaning. Different disciplines and traditions often interpret systems in different ways: as objective structures present in the world, as conceptual constructs used to organize understanding, or as relational patterns emerging from interactions among entities.

These philosophical assumptions influence how systems are defined, how system boundaries are drawn, how systemic knowledge is generated, and how complex phenomena are interpreted. They therefore play a central role in shaping systemic inquiry.

Unlike the preceding article on the *History and Foundations of Systems Science*, which describes the intellectual development of the field, this article focuses on the philosophical perspectives that influence how systems are perceived and interpreted. These perspectives include ontological assumptions concerning the nature of systems, epistemological perspectives on systemic knowledge, semantic considerations concerning systemic language, and phenomenological insights concerning human experience and perception.

Understanding these philosophical foundations helps clarify how systems are recognized and studied. It also provides an important bridge between systems science and the cognitive and methodological approaches of systems thinking that are addressed in the following section.



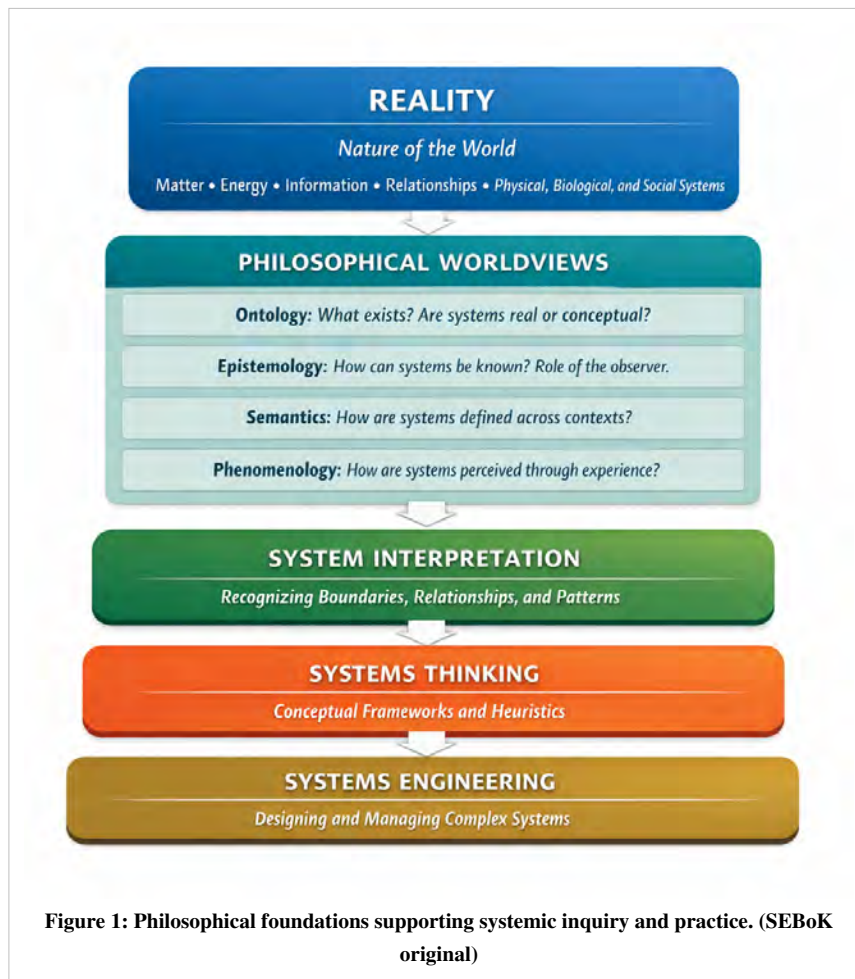
Philosophical Foundations of Systemic Inquiry

The interpretation of complex phenomena as systems depends on underlying philosophical assumptions about reality, knowledge, meaning, and experience. These assumptions shape how observers identify relationships, define system boundaries, and interpret systemic behavior.

Figure 1 illustrates how these philosophical perspectives support systemic inquiry and practice.

At the most fundamental level is **reality itself**, consisting of natural and social phenomena. Observers interpret these phenomena through philosophical worldviews concerning ontology, epistemology, semantics, and lived experience. These perspectives influence how systems are recognized and conceptualized. Systems thinking then provides conceptual frameworks for reasoning about these relationships, while systems engineering applies these insights in the design and management of complex socio-technical systems.

Understanding these conceptual layers clarifies the intellectual foundations that support systemic inquiry.



Philosophical worldviews shape how observers interpret reality as systems. These foundations influence systemic interpretation, which supports systems thinking and ultimately informs systems engineering practice.

As shown in the figure, systemic inquiry involves several conceptual layers:

- **Reality** – the natural and social phenomena that observers encounter
- **Philosophical worldviews** – assumptions concerning ontology, epistemology, semantics, and experience
- **System interpretation** – recognizing relationships, boundaries, and patterns that constitute systems
- **Systems thinking** – conceptual frameworks and heuristics for understanding complex systems
- **Systems engineering** – practical methods for designing and managing complex systems

Understanding these layers helps clarify the intellectual foundations of systems science and its relationship to systems thinking and systems engineering.

The Meaning of "System"

Although the concept of a system is widely used across disciplines, its precise meaning varies depending on context. Most definitions converge on the idea that a system consists of interrelated components forming an integrated whole. For example, von Bertalanffy described systems as organized wholes whose properties arise from interactions among their components (von Bertalanffy 1968). Boulding emphasized hierarchical organization across levels of complexity (Boulding 1956). Ackoff emphasized purposeful systems capable of goal-directed behavior (Ackoff 1971). Meadows described systems as sets of elements interconnected in ways that produce patterns of behavior over time (Meadows 2008).

Despite differences among disciplines, most system definitions include three key characteristics:

- elements or components
- relationships among those elements
- an integrated whole exhibiting emergent behavior

These shared characteristics provide a conceptual foundation for systemic inquiry across scientific and engineering domains.

Ontological Perspectives: Are Systems Real?

A central philosophical question concerns the **ontological status of systems**: whether systems exist as real structures in the world or whether they are primarily conceptual constructs used to interpret complex phenomena.

System Realism

Some philosophers argue that systems represent genuine structures of reality. Bunge proposed the philosophical position known as *systemism*, which holds that the world is fundamentally composed of interconnected systems rather than isolated objects (Bunge 1979; Bunge 2000). From this perspective, systems science investigates real patterns of organization present in nature and society.

Examples of such systems include biological organisms, ecosystems, economies, and technological infrastructures, all of which exhibit organized relationships and dynamic behaviors that arise from interactions among their components.

Systems as Conceptual Models

Other perspectives emphasize that systems are often defined through analytical choices made by observers. System boundaries may be determined by the purpose of investigation rather than by intrinsic divisions in nature. From this viewpoint, systems function as conceptual models that help observers interpret complexity.

This perspective has been influential in systemic problem-structuring approaches. Ackoff, for example, emphasized that systems are often defined in relation to the purposes of inquiry (Ackoff 1971). Churchman similarly argued that system boundaries are ethical and intellectual choices that shape how problems are framed (Churchman 1979).

Rational Ontology

A third perspective emphasizes relationships as the fundamental basis of reality. In relational ontology, entities are defined primarily through their interactions with other entities.

Rosen's relational biology argued that living systems cannot be understood solely through material components but must be understood through the network of relationships that sustain their organization (Rosen 1991). Kineman similarly proposes a relational view of science in which systemic structures emerge through contextual relationships rather than isolated substances (Kineman 2011; Kineman 2017b). Relational ontology has become increasingly influential in fields such as ecology, network science, and systems biology, where interactions and relationships often provide deeper explanatory power than isolated components.

Epistemological Perspectives: How Systems are Known

Systems science also raises important epistemological questions concerning how knowledge about systems is generated.

Cybernetic Epistemology

Cybernetics introduced the insight that observers are themselves part of the systems they study. Wiener defined cybernetics as the study of control and communication in animals and machines (Wiener 1948). Cybernetic analysis revealed that feedback, communication, and regulation are fundamental features of many systems.

Second-order cybernetics later emphasized the role of the observer in constructing descriptions of systems. Von Foerster argued that systemic knowledge is always produced by observers who participate in the systems they describe (von Foerster 1974; von Foerster 2003).

Constructivist Perspectives

Constructivist approaches emphasize that knowledge about systems is shaped by interpretation and context. Checkland highlighted the role of stakeholder worldviews when defining systems of interest in complex social situations (Checkland 1989; Checkland 1999). Similarly, Maturana and Varela showed how living systems generate meaning through structural coupling with their environments (Maturana and Varela 1980).

From this perspective, systemic knowledge arises through interaction between observers and the phenomena they investigate.

Integrative Perspectives

Many contemporary systems scholars combine realist and constructivist insights. Systemic structures exist in the world, yet human observers interpret and model those structures using conceptual frameworks.

This integrative perspective supports systems science as both empirical inquiry and conceptual modelling (Mobus and Kalton 2014; Hieronymi 2013).

Popper's Three Worlds and Systemic Knowledge

Karl Popper proposed that reality can be understood through three interacting domains (Popper 1972):

- **World 1 – Physical Reality** Material objects and natural processes.
- **World 2 – Subjective Experience** Human perception, cognition, and awareness.
- **World 3 – Objective Knowledge** Scientific theories, models, and conceptual knowledge.

Systems science frequently operates across all three domains. Ecological systems exist in World 1, human perception of those systems occurs in World 2, and systemic theories and models belong to World 3.

Recent work has extended this interpretation within engineering and systemic practice. For example, Blockley and colleagues relate Popper's framework to relational approaches to engineering knowledge and systemic reasoning (Blockley et al. 2023).

Information and the Ontology of Systems

Another philosophical perspective relevant to systems science concerns the role of **information** in the structure of reality.

Stonier argued that information should be considered a fundamental property of the universe alongside matter and energy (Stonier 1997; Stonier 2012). From this viewpoint, physical structures can be interpreted as organized information patterns.

Information-based perspectives are increasingly influential in systems biology, network science, and complex adaptive systems. Vedral, for example, argues that information provides a foundational basis for understanding physical reality and organization across scales (Vedral 2018).

These perspectives suggest that systemic organization may often be interpreted in terms of information flows, transformations, and constraints.

Semantics and the Language of Systems

The concept of a system functions as a **semantic abstraction** that enables diverse phenomena to be described using common conceptual frameworks. However, different disciplines interpret systemic language differently.

Engineering systems often emphasize functional architecture and performance. Biological systems emphasize self-maintaining organization. Social systems emphasize institutions and relationships.

These variations highlight the importance of semantic clarity within systems science. Rousseau, Billingham, and Calvo-Amodio emphasize the need for coherent systemic semantics when building shared ontologies and conceptual frameworks across the systems field (Rousseau, Billingham, and Calvo-Amodio 2018).

Dori and colleagues similarly highlight the importance of distinguishing system definitions, worldviews, and systemness characteristics when establishing conceptual clarity (Dori et al. 2019).

Heuristics and Practical Systemic Reasoning

Phenomenological insights connect closely with the role of **heuristics** in systemic practice. Heuristics are practical guiding principles that support reasoning and decision-making under complexity.

Simon showed that decision-making under bounded rationality often relies on heuristic judgment rather than exhaustive optimization (Simon 1957). Kahneman later demonstrated how heuristic reasoning shapes human judgment under uncertainty (Kahneman 2011).

In systems engineering, heuristics help practitioners identify system boundaries, anticipate emergent behavior, and balance competing system objectives.

Historically, systems engineering itself developed largely through heuristic practice derived from engineering experience. Ring explicitly framed systems engineering in terms of value-seeking heuristics that guide systemic design and integration (Ring 1998).

Worldviews and System Boundaries

Worldviews strongly influence how system boundaries are defined. Different observers may interpret the same situation through different conceptual frameworks.

Environmental scientists may define systems using ecological boundaries. Policy makers may define them using administrative jurisdictions. Communities may interpret systems through cultural or historical relationships.

Churchman emphasized that boundary judgments are always ethically and practically significant because they determine what is included, excluded, or valued in systemic inquiry (Churchman 1979).

Implications for Systems Science and Engineering

Understanding the philosophical foundations of systemness has practical implications for systems science and engineering.

Practitioners must make assumptions concerning:

- what constitutes a system
- how system boundaries are defined
- how systemic knowledge is generated
- how values and perspectives influence interpretation

Recognizing these assumptions helps ensure that systemic inquiry remains transparent and coherent.

It also clarifies the relationship between systems science and the next knowledge area in SEBoK: **Systems Thinking**. Systems science investigates the nature and principles of systems, while systems thinking provides cognitive and methodological approaches for reasoning about complex situations.

Conclusion

Systems science rests upon philosophical foundations concerning the nature of reality, the role of observers, and the interpretation of systemic concepts. Ontological, epistemological, semantic, and phenomenological perspectives together shape how systems are recognized and studied.

Understanding these philosophical foundations strengthens the conceptual coherence of systems science and provides an important bridge to the practical methods of systems thinking and systems engineering.

References

Works Cited

- Ackoff, R. L. 1971. "Towards a System of Systems Concepts." *Management Science* 17(11): 661–671.
- Blockley, D., et al. 2023. "Relational Holon Science and Popper's 3 Worlds in Engineering Practice." *Systems Research and Behavioral Science* 40(6): 955–967.
- Boulding, K. E. 1956. "General Systems Theory—The Skeleton of Science." *Management Science* 2(3): 197–208.
- Bunge, M. 1979. "A Systemic World View." In *Treatise on Basic Philosophy: Ontology II: A World of Systems*. Dordrecht: Reidel.
- Bunge, M. 2000. "Systemism: The Alternative to Individualism and Holism." *The Journal of Socio-Economics* 29(2): 147–157.
- Checkland, P. B. 1989. "Soft Systems Methodology." *Human Systems Management* 8: 273–289.
- Checkland, P. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: Wiley.
- Churchman, C. W. 1979. *The Systems Approach and Its Enemies*. New York, NY, USA: Basic Books.

- Dori, D., et al. 2019. "System Definition, System Worldviews, and Systemness Characteristics." *IEEE Systems Journal*.
- Hieronymi, A. 2013. "Understanding Systems Science: A Visual and Integrative Approach." *Systems Research and Behavioral Science* 30(5): 580–595.
- Husserl, E. 1931. *Ideas: General Introduction to Pure Phenomenology*. London, UK: George Allen & Unwin.
- Jonkisz, J. 2021. "Consciousness, Subjectivity, and Gradedness." *Semiotic Studies* 35(1).
- Kahneman, D. 2011. *Thinking, Fast and Slow*. New York, NY, USA: Farrar, Straus and Giroux.
- Kineman, J. J. 2011. "Relational Science: A Synthesis." *Axiomathes* 21(3): 393–437.
- Kineman, J. J. 2017b. "Systems Research Framework." In Edson, M. C., Henning, P. B., and Sankaran, S., eds. *A Guide to Systems Research: Philosophy, Processes and Practice*. Singapore: Springer, pp. 21–57.
- Maturana, H. R., and Varela, F. J. 1980. *Autopoiesis and Cognition: The Realization of the Living*. Dordrecht, The Netherlands: Reidel.
- Meadows, D. H. 2008. *Thinking in Systems: A Primer*. White River Junction, VT, USA: Chelsea Green.
- Merleau-Ponty, M. 1945. *Phenomenology of Perception*. Paris, France: Gallimard.
- Mobus, G. E., and Kalton, M. C. 2014. *Principles of Systems Science*. New York, NY, USA: Springer.
- Popper, K. R. 1972. *Objective Knowledge: An Evolutionary Approach*. Oxford, UK: Clarendon Press.
- Ring, J. 1998. "A Value-Seeking Approach to the Engineering of Systems." In *Proceedings of the 1998 IEEE International Conference on Systems, Man, and Cybernetics*.
- Rosen, R. 1991. *Life Itself: A Comprehensive Inquiry into the Nature, Origin, and Fabrication of Life*. New York, NY, USA: Columbia University Press.
- Rousseau, D., Billingham, J., and Calvo-Amodio, J. 2018. "Systemic Semantics: A Systems Approach to Building Ontologies and Concept Maps." *Systems* 6(3): 32.
- Simon, H. A. 1957. *Models of Man: Social and Rational*. New York, NY, USA: Wiley.
- Stonier, T. 1997. *Information and Meaning: An Evolutionary Perspective*. London, UK: Springer.
- Stonier, T. 2012. *Information and the Internal Structure of the Universe: An Exploration into Information Physics*. London, UK: Springer.
- Vedral, V. 2018. *Decoding Reality: The Universe as Quantum Information*. Oxford, UK: Oxford University Press.
- von Bertalanffy, L. 1968. *General System Theory: Foundations, Development, Applications*. New York, NY, USA: Braziller.
- von Foerster, H. 1974. *Cybernetics of Cybernetics*. Urbana, IL, USA: University of Illinois.
- von Foerster, H. 2003. *Understanding Understanding: Essays on Cybernetics and Cognition*. New York, NY, USA: Springer.
- Wiener, N. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. Cambridge, MA, USA: MIT Press.
-

Primary References

None.

Additional References

None.

Patterns and Principles in Systems Science

- Lead Author:
- Gary Smith

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

A central aim of systems science is to identify recurring patterns and organizing principles that appear across many kinds of systems. While traditional scientific disciplines typically investigate particular classes of phenomena, such as physical systems, biological organisms, or social institutions—systems science seeks to understand characteristics of organization that recur across domains.

Researchers studying systems in fields ranging from physics and biology to ecology, engineering, and the social sciences have repeatedly observed similar forms of organization and behavior. These recurring characteristics suggest that many systems share common structural and dynamic properties despite differences in their components.

Such regularities are often described using terms such as **system principles**, **organizational patterns**, **meta-patterns**, or isomorphies. These concepts reflect a shared effort to understand how complex systems are structured, how they behave, and how they evolve.

Recognizing these patterns provides an important foundation for understanding complex systems in nature, society, and engineering. By identifying cross-domain regularities in system organization, systems science contributes to a deeper understanding of how systems maintain coherence, adapt to changing environments, and generate emergent behavior.

Recurring Patterns Across Systems

Across many scientific disciplines, researchers have observed that systems frequently exhibit similar forms of organization. Examples include:

- **hierarchical organization**, where systems consist of nested levels of subsystems
- **network structures**, in which elements are interconnected through relational links
- **flows of energy, matter, or information** that sustain system processes
- **feedback mechanisms** that regulate system behavior
- **cyclic dynamics** that generate recurring patterns over time
- **boundaries and interfaces** that distinguish systems from their environments
- **emergence**, where system-level properties arise from interactions among components

These patterns appear in systems as diverse as ecosystems, biological organisms, economies, technological infrastructures, and social institutions. Their repeated appearance across domains suggests that they represent

general characteristics of organized systems rather than domain-specific phenomena.

Recognizing these patterns allows researchers to identify common principles that govern the behavior of complex systems.

Nature's Enduring Patterns

Researchers have increasingly sought to identify recurring organizational patterns that appear across many natural systems. These patterns often arise from fundamental interactions among system components and their environments.

The major categories of such recurring patterns are illustrated in **Figure 1**, which highlights several organizational characteristics that appear widely across natural and complex systems.

- **Hierarchies**, representing nested levels of organization
- **Networks**, representing relational connectivity among system elements
- **Flows and exchanges**, involving energy, matter, and information
- **Feedback mechanisms**, enabling regulation and stability
- **Cycles and rhythms**, producing repeating temporal patterns
- **Self-organization**, where order emerges from local interactions
- **Boundaries**, defining interfaces between systems and environments
- **Evolution and co-development**, reflecting adaptive change over time
- **Patterns across scales**, including fractal and scaling relationships

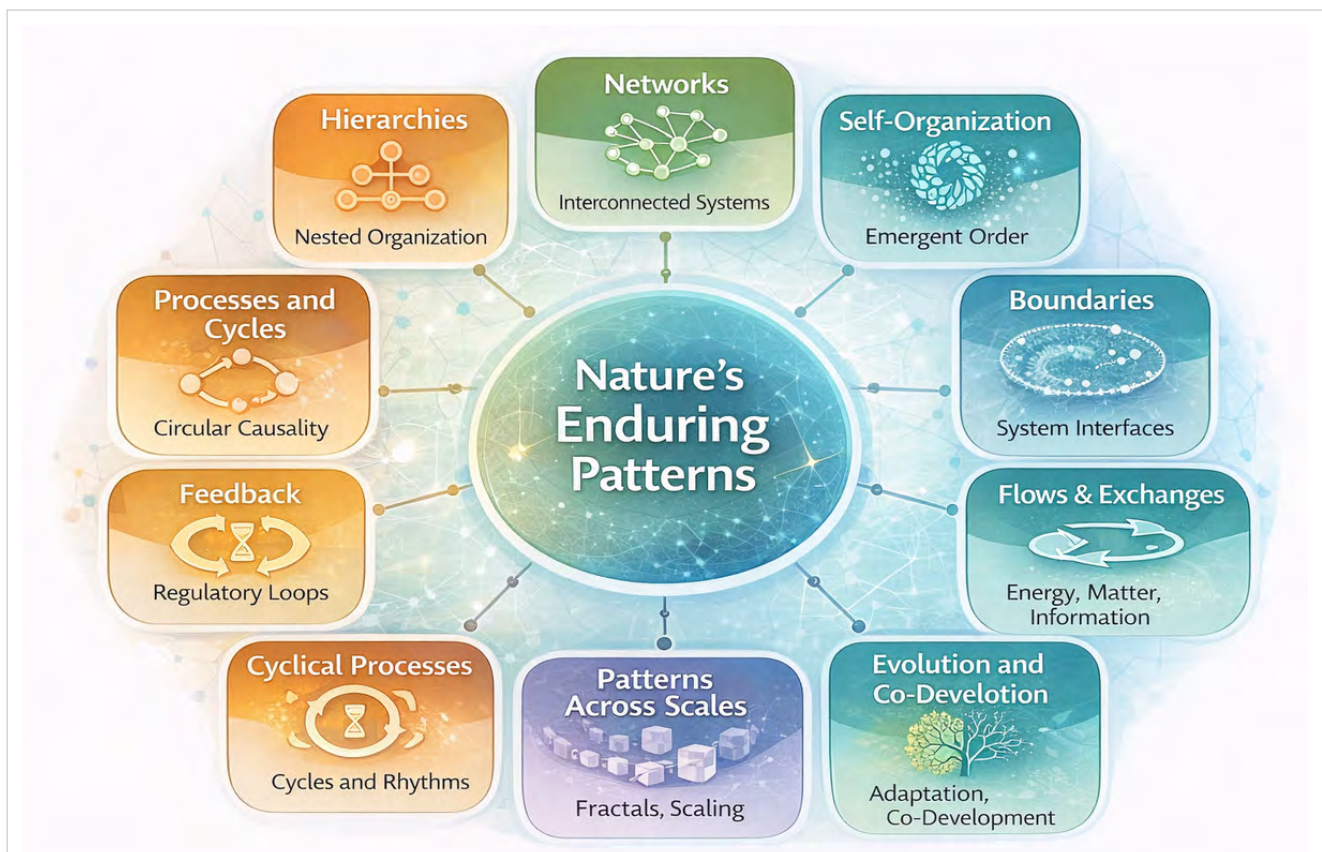


Figure 1: Common patterns found across natural and complex systems. (SEBoK original)

These patterns can be observed in phenomena ranging from physical processes and ecological systems to social networks and technological infrastructures.

The recurrence of these patterns across many domains suggests that they reflect **fundamental properties of system organization**.

Cross-Domain System Principles

The search for cross-domain regularities has been a central goal of systems science since its earliest development. General Systems Theory, cybernetics, and related traditions all sought to identify principles of organization that apply across different types of systems.

Examples of widely recognized system principles include:

- **open systems**, which exchange matter, energy, and information with their environments
- **feedback regulation**, enabling systems to maintain stability and respond to disturbances
- **hierarchical organization**, allowing complex systems to be structured across multiple levels
- **self-organization**, where patterns and structures emerge from local interactions among components

These principles help explain how systems maintain coherence, adapt to environmental change, and evolve over time.

The recognition of such principles has played a key role in establishing systems science as a field concerned with **general characteristics of organization across diverse domains**.

Isomorphies and Cross-Domain System Processes

One of the most systematic efforts to identify recurring system patterns was undertaken by **Len Troncale**, who investigated the existence of cross-disciplinary regularities among systems phenomena.

Troncale introduced the concept of **systems isomorphies**, referring to processes or organizational relationships that appear across multiple scientific domains (Troncale 1985). Through extensive surveys of scientific literature across many disciplines, he documented numerous recurring processes including:

- feedback regulation
- oscillatory and cyclic dynamics
- hierarchical organization
- flow processes
- network interactions
- developmental and evolutionary patterns

Troncale's work represented one of the most comprehensive attempts to empirically document recurring system processes across natural and social systems. His research suggested that many systems exhibit similar behaviors and structures regardless of the specific domain in which they occur.

Although the catalog of isomorphies does not constitute a fully integrated taxonomy of system processes, it highlights the potential for identifying **generalizable system phenomena that recur across many scientific fields**.

Continuing Work on Systemic Patterns

Subsequent work has continued exploring recurring patterns across domains and scales. Rasmussen (2023), for example, emphasizes the importance of recognizing **enduring patterns** that appear across natural and social systems. Her work highlights how structures such as networks, cycles, fractal forms, and hierarchical organization appear repeatedly across levels of organization.

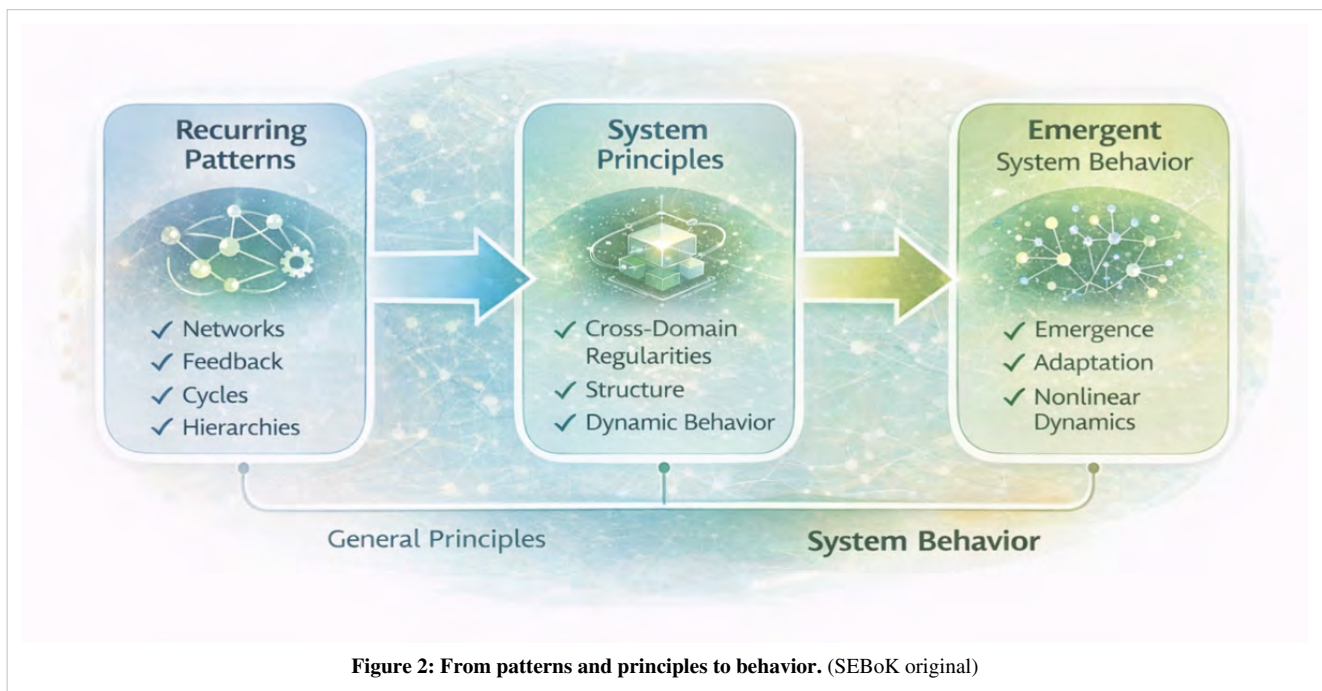
This perspective extends earlier efforts to identify systemic regularities by emphasizing the role of **perception and recognition of patterns** in developing systemic understanding.

Together, the work of Troncale and Rasmussen illustrates the continuing effort within systems science to identify recurring structures and processes that characterize complex systems.

From Patterns to System Behaviour

The relationship between recurring patterns, system principles, and system behavior can be understood as a progression from observation to explanation. Researchers first observe recurring patterns across systems, then formulate principles that explain those patterns, and finally investigate how these principles produce characteristic system behaviors.

This relationship is illustrated in **Figure 2**.



Recurring patterns such as networks, feedback loops, cycles, and hierarchies provide clues to deeper organizing principles. These principles help explain why systems exhibit behaviors such as emergence, adaptation, nonlinear dynamics, and complex interactions.

Understanding this progression helps clarify how systems science links empirical observation with theoretical explanation.

Structural, Dynamic, and Evolutionary Principles

Recurring system patterns can often be grouped into several broad categories.

Structural Principles

Structural principles describe how systems are organized.

Examples include:

- hierarchical organization
- modular structure
- network connectivity
- system boundaries and interfaces

These principles describe the architecture of systems and how their components are arranged.

Dynamic Principles

Dynamic principles describe how systems behave and interact over time.

Examples include:

- feedback and regulation
- flows of energy, matter, or information
- cyclic and oscillatory dynamics
- adaptation to environmental change

These principles explain how systems maintain stability while responding to disturbances.

Evolutionary Principles

Evolutionary principles describe how systems develop and transform.

Examples include:

- emergence of new system properties
- self-organization
- co-evolution with environments
- increasing complexity over time

These principles help explain how systems adapt and evolve across longer timescales.

Implications for Systems Engineering

Understanding recurring patterns in systems has important implications for systems engineering. Engineers frequently design and manage complex systems whose behavior arises from interactions among many components and subsystems.

Recognizing systemic patterns such as feedback effects, hierarchical organization, and emergent behavior can help engineers anticipate potential system behaviors and interactions. Awareness of these principles can therefore support more effective design, integration, and management of complex engineered systems.

Systems science provides conceptual tools that help engineers understand how complex systems behave beyond the properties of their individual components.

Conclusion

Systems science seeks to identify recurring patterns and principles that characterize organized systems across many domains of reality. These patterns, such as hierarchical organization, feedback regulation, network structures, and emergent dynamics, appear repeatedly in systems ranging from physical processes and biological organisms to ecological, social, and engineered systems.

Efforts to identify such patterns have been pursued through concepts such as system principles, meta-patterns, and isomorphies. Research by Troncale, Rasmussen, and others illustrates the continuing effort to document and understand these recurring characteristics of complex systems.

By identifying cross-domain regularities in system organization and behavior, systems science provides a foundation for understanding complex phenomena and supports the development of approaches for studying and managing systems in many fields.

The next article explores how these principles contribute to phenomena such as **complexity, emergence, and adaptive behavior in systems**.

References

Works Cited

- Ashby, W. R. 1956. *An Introduction to Cybernetics*. London: Chapman & Hall.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. New York: Braziller.
- Boulding, K. E. 1956. "General Systems Theory—The Skeleton of Science." *Management Science*.
- Simon, H. A. 1962. "The Architecture of Complexity." *Proceedings of the American Philosophical Society*.
- Troncale, L. 1985. "On the Possible Order of Systems Isomorphies." *General Systems*.
- Volk, T. 2017. *Quarks to Culture: How We Came to Be*. New York: Columbia University Press.
- Rasmussen, L. 2023. *Seeing: A Guide to Systems Thinking*. Axminster: Triarchy Press.

Primary References

None.

Additional References

- Miller, J. G. 1978. *Living Systems*. New York: McGraw-Hill.
- Mobus, G. E., and M. C. Kalton. 2015. *Principles of Systems Science*. New York: Springer.

Structure, Identity, and Function in Systems

- Lead Author:
- Gary Smith

Systems can be understood by looking at three closely connected ideas: structure, identity, and function. Structure describes how a system is organized and how its parts relate to one another, while function reflects what the system does and the outcomes it produces. Identity helps explain what makes a system recognizable over time, even as it changes or adapts to new conditions. Together, these perspectives help explain how systems behave, evolve, and maintain continuity across natural, engineered, and socio-technical contexts.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

Systems maintain coherence through relationships among their **structure, identity, and function**. These elements together determine how systems are organized, how they distinguish themselves from their environments, and how they perform activities that sustain their continued existence.

In systems science, a system is generally understood as a set of interacting components that together form a coherent whole. The properties and behaviors of the whole arise not only from the characteristics of individual components but also from the **relationships among those components**. These relationships define the system's structure and enable processes that support system function.

Understanding the interplay between structure, identity, and function provides a foundation for analyzing both natural and engineered systems. It explains how systems persist over time, how they maintain coherence despite internal change, and how they perform the activities that characterize them as recognizable entities.

System Structure

The **structure of a system** refers to the arrangement of its components and the relationships among them. These relationships may involve physical connections, flows of energy or materials, exchanges of information, or other forms of interaction.

Structure determines how components influence one another and how processes propagate throughout the system. In a biological organism, structural organization includes tissues, organs, and circulatory pathways that enable coordinated physiological processes. In an engineered system such as an aircraft, structural organization includes subsystems, interfaces, and communication links that allow components to operate together as an integrated whole.

Structure therefore represents more than a collection of parts. It reflects an **organized pattern of relationships** that constrains and enables system behavior.

General systems theory emphasized the importance of understanding such relational patterns rather than focusing solely on isolated components. As described by Ludwig von Bertalanffy, many important properties of systems arise from the organization of relationships among elements.

Cybernetics similarly highlights the role of structure in shaping system behavior. W. Ross Ashby showed that the behavior of a system is constrained by its internal structure and the regulatory relationships among its components.

Organization and System Identity

Closely related to structure is the concept of **organization**, which refers to the relationships that define a system as a particular kind of entity. Organization determines which relationships must remain intact for the system to maintain its identity.

Biologists Humberto Maturana and Francisco Varela distinguished between **structure**, which may change over time, and **organization**, which defines the essential relations that characterize a system.

For example, the components of a living organism continually change through metabolic processes and cellular turnover. Despite these structural changes, the organism maintains its identity because the organizational relationships that sustain life remain intact.

Similarly, engineered systems may undergo component replacement and modification throughout their lifecycle while preserving their identity as functioning systems.

System identity therefore depends not only on the presence of particular components but on the **continuity of organizational relationships** that sustain the system as a coherent whole.

Relational perspectives in systems science further emphasize that systemic properties arise from networks of relationships rather than isolated components. Robert Rosen argued that living systems must be understood primarily through the relationships that sustain their organization.

System Boundaries

Systems are typically distinguished from their environments by **boundaries** that define the scope of the system.

Boundaries identify which components and processes belong to the system and which lie outside it. These boundaries may be physical, functional, or conceptual.

A biological membrane forms a physical boundary that separates an organism from its environment while regulating exchanges of materials and energy. In engineered systems, boundaries often correspond to defined interfaces that regulate interactions with external systems.

Importantly, boundaries do not imply isolation. Most systems maintain their identity through **regulated interaction with their environments**, exchanging energy, materials, and information while preserving internal organization.

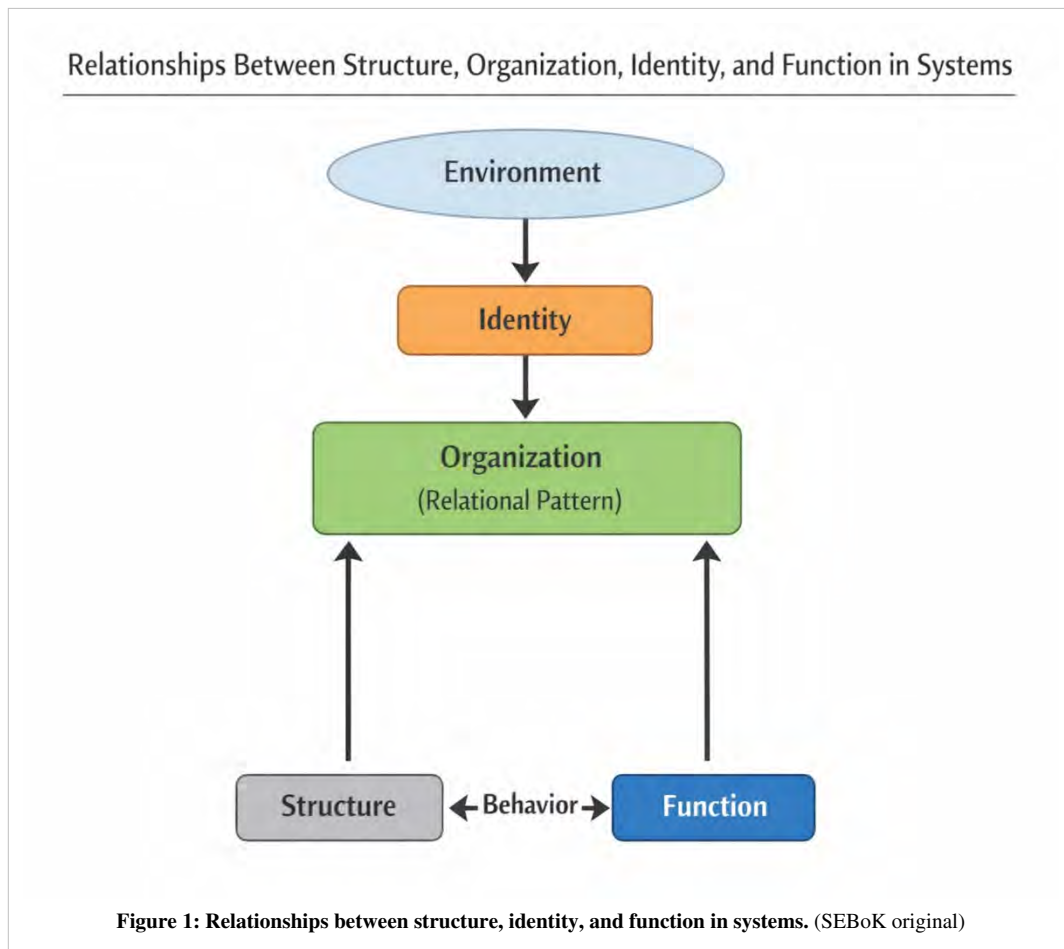
In systems thinking practice, boundaries are also recognized as analytical constructs used to define the scope of a system of interest (Ackoff 1999).

Relationship Between Structure, Identity, and Function

Structure, identity, and function are closely interconnected.

Structure provides the arrangement of components that enables processes to occur. These processes allow the system to perform functions that sustain its continued existence. At the same time, the organization of structural relationships maintains the identity of the system as a coherent whole.

Figure 1 illustrates these relationships.



System structure defines the arrangement of components and their relationships. These relationships sustain system identity by preserving organizational coherence. At the same time, structural organization enables functions and behaviors that allow the system to operate effectively within its environment.

Changes in system structure may alter system behavior, while disruptions to essential organizational relationships may threaten system identity.

Understanding these relationships is therefore central to understanding system behavior and persistence.

Domains of Systems

Systems science applies to a wide range of systems across both the natural world and human society. Although these systems differ in their components and mechanisms, they share common systemic properties such as organization, boundaries, interactions, and functional processes.

Several broad domains of systems can be identified.

- **Physical systems** include structures such as atoms, planetary systems, and engineered artifacts. Their structure arises from physical interactions among components, and their behavior reflects processes such as energy transfer and motion.
- **Living systems** include cells, organisms, and ecosystems. These systems maintain identity through biological organization and metabolic processes that sustain life (Miller 1978).
- **Social systems** include communities, institutions, and organizations. In these systems, structure arises from relationships among individuals and groups, while functions involve communication, coordination, governance, and collective action (Parsons 1951).

- **Conceptual systems** include scientific theories, mathematical frameworks, classification systems, and engineering models. These systems consist primarily of concepts and relationships that organize knowledge and reasoning.

Despite their differences, all of these domains exhibit recognizable relationships among **structure, identity, and function**. Systems science therefore seeks principles that apply across these diverse contexts (Boulding 1956).

Maintaining System Identity and Viability

For a system to persist over time, its structural organization and functional processes must remain sufficiently coherent. This coherence allows the system to maintain identity despite internal change or environmental disturbance.

Systems that lose essential relationships among components may experience degradation or collapse. Biological organisms may lose viability if regulatory processes fail, while engineered systems may fail when structural relationships among subsystems break down.

Maintaining coherence among structure, identity, and function is therefore central to system viability.

Concepts such as regulation, feedback, and adaptation play important roles in maintaining this coherence across many kinds of systems (Ashby 1956; Meadows 2008).

Relevance to Systems Engineering

The relationship between structure, identity, and function is central to **systems engineering practice**.

Engineers must design system architectures that establish appropriate relationships among components while enabling the system to perform required functions. System architecture frameworks, interface definitions, and model-based representations provide conceptual tools for managing system structure and ensuring coherent integration of subsystems.

Understanding how structural relationships influence system behavior helps systems engineers design systems that remain reliable and adaptable within complex operational environments.

Conceptual models and system representations therefore play an important role in enabling engineers to reason about structure and function before systems are physically realized.

Conclusion

Systems maintain coherence through relationships among their structure, organization, and function. Structural arrangements determine how components interact, organizational relationships sustain system identity, and these relationships together enable the system to perform functions within its environment.

These relationships apply across many domains of systems, including physical, living, social, and conceptual systems. Understanding how structure and organization sustain systemic coherence provides an essential foundation for analyzing complex systems.

This foundation also prepares the ground for examining how complex behaviors arise from interactions among system components, which is explored further in the following article on **Emergence and Complexity**.

References

Works Cited

- Ackoff, R. L. (1999). *Ackoff's Best: His Classic Writings on Management*. Wiley.
- Ashby, W. R. (1956). *An Introduction to Cybernetics*. Chapman & Hall.
- Bertalanffy, L. von. (1968). *General System Theory: Foundations, Development, Applications*. George Braziller.
- Boulding, K. E. (1956). "General Systems Theory—The Skeleton of Science." *Management Science*, 2(3).
- Meadows, D. H. (2008). *Thinking in Systems: A Primer*. Chelsea Green.
- Maturana, H. R., & Varela, F. J. (1980). *Autopoiesis and Cognition: The Realization of the Living*. D. Reidel.
- Miller, J. G. (1978). *Living Systems*. McGraw-Hill.
- Parsons, T. (1951). *The Social System*. Free Press.
- Rosen, R. (1991). *Life Itself: A Comprehensive Inquiry into the Nature, Origin, and Fabrication of Life*. Columbia University Press.
- Simon, H. A. (1962). "The Architecture of Complexity." *Proceedings of the American Philosophical Society*, 106(6).
- Troncale, L. R. (2006). "Towards a Science of Systems Processes." *Systems Research and Behavioral Science*.
- Weinberg, G. M. (1975). *An Introduction to General Systems Thinking*. Wiley.

Primary References

None.

Additional References

None.

Emergence and Complexity

- Lead Author:
- Gary Smith
- Contributing Authors:
- Hillary Sillitto, Sarah Sheard, and Rick Adcock

Emergence and complexity describe how systems can exhibit behaviors, patterns, and outcomes that cannot be fully understood by examining individual parts in isolation. As interactions between components increase, systems often become less predictable, more adaptive, and more sensitive to relationships, context, and feedback. Complex systems may display nonlinearity, self-organization, adaptation, and unexpected emergent behaviors that arise from the interactions within the system as a whole. Understanding these dynamics is essential for systems thinking and systems engineering, particularly when working with large-scale, interconnected, or socio-technical systems.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility

Introduction

This article forms part of the **Systems Science** knowledge area (KA). It examines the closely related concepts of **complexity** and **emergence**, which are fundamental to understanding how systems behave as organized wholes.

The preceding article, Structure, Identity, and Function in Systems, examined how systems maintain coherence through relationships among components, boundaries, organization, and functions. This article builds on that foundation by considering how complex behavior arises from such relationships. It prepares the ground for the following article, System Dynamics and Control, which examines how feedback processes and regulatory mechanisms shape system behavior over time.

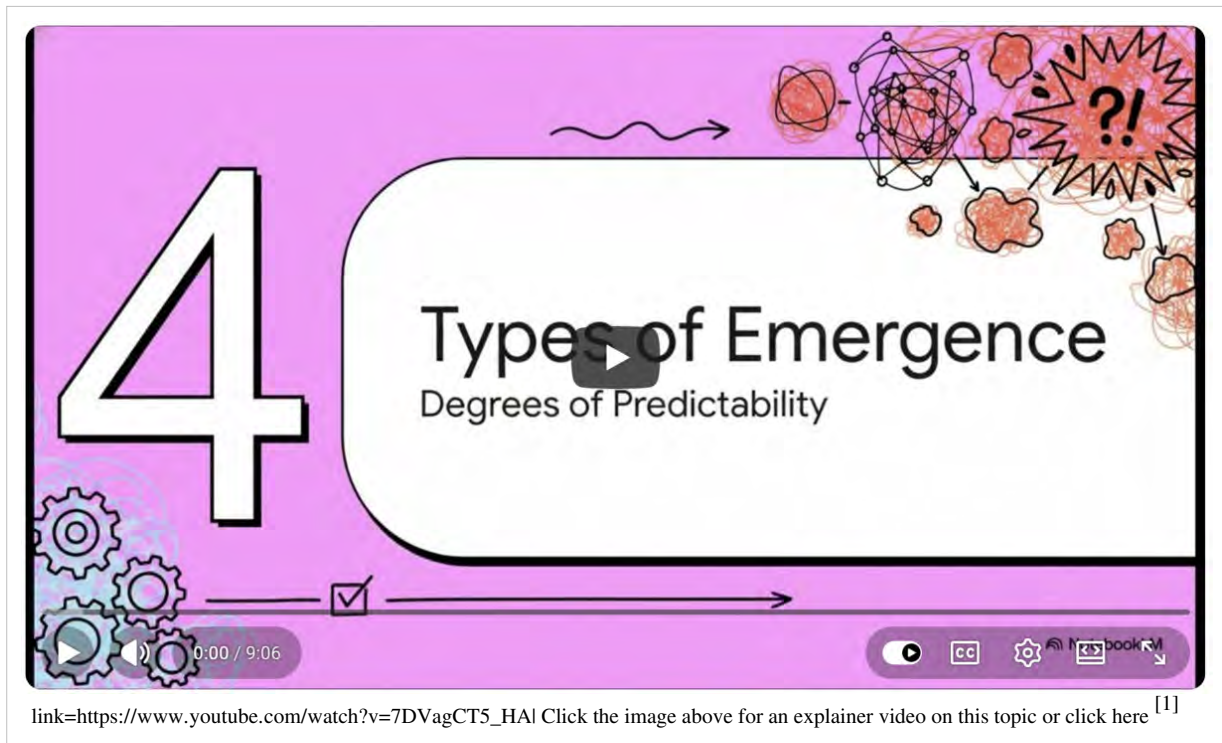
Many systems cannot be fully understood by examining their components in isolation. Instead, system-level properties and behaviors arise from interactions among elements, from the structure of those interactions, and from the system's relationship with its environment. These phenomena are central to systems science and have direct implications for systems engineering.

Early recognition of these issues can be traced to Warren Weaver's distinction between problems of simplicity, disorganized complexity, and organized complexity. Organized complexity arises when many interacting elements form structured patterns of behavior that cannot be adequately understood through simple reduction or statistical aggregation alone (Weaver 1948).

Within this context:

- Complexity refers to the richness, diversity, and interdependence of relationships among system elements and between the system and its environment.
- Emergence refers to system-level properties or behaviors that arise from these interactions and cannot be attributed to any single component.

Understanding complexity and emergence is essential for systems engineering, particularly when designing, integrating, and operating large-scale, adaptive, or socio-technical systems.



Understanding System Complexity

System complexity arises from the number, diversity, organization, and interaction of system elements, and from the relationships between the system and its environment. It reflects the degree to which system behavior cannot be fully understood, predicted, or influenced simply from knowledge of individual components.

Complexity should not be understood only as an intrinsic property of a system considered in isolation. It is also relational: it depends on the system's organization, its environmental coupling, the perspective and purpose of the observer, and the extent to which system behavior can be understood, anticipated, influenced, or controlled.

A useful distinction is often made between **complicated** and **complex** systems.

Complicated systems may contain many components but behave in broadly predictable ways because the relationships among elements are stable and well understood. Many mechanical systems are complicated in this sense. They may be difficult to design, build, or maintain, but their behavior can often be decomposed, analyzed, and predicted.

Complex systems, by contrast, involve interactions that may be nonlinear, adaptive, evolving, or context dependent. Their behavior may change over time as elements interact with one another and with their environment. Ladyman, Lambert, and Wiesner (2013) emphasize that complex systems are typically characterized by multiple interacting parts, nonlinear relations, feedback, robustness, and emergent organization.

The distinction between complicated and complex systems is useful, but should not be treated as absolute. Many engineered systems are complicated in their technical architecture while becoming complex in operation because of changing environments, human interaction, organizational coupling, lifecycle effects, or system-of-systems dependencies. A system may therefore be complicated in design, complex in use, and increasingly complex as it becomes embedded within wider operational and social contexts.

Complexity often arises from several systemic characteristics:

- **Interconnectedness** — elements influence one another through networks of relationships.
- **Diversity** — elements differ in structure, role, capability, or perspective.

- **Nonlinear interaction** — small changes may produce disproportionate effects.
- **Feedback** — reinforcing and balancing loops shape behavior over time.
- **Adaptation** — elements or systems adjust in response to internal or environmental change.
- **Context dependence** — behavior depends on the surrounding environment and conditions of operation.

These characteristics are widely observed in ecosystems, economies, biological organisms, organizations, infrastructure systems, and many engineered systems. Holland (1995) and Mitchell (2009) show how complex adaptive behavior frequently arises when many interacting agents operate within networks and adapt to changing conditions.

From a systems engineering perspective, complexity may arise across several dimensions:

- **Structural complexity**, related to the number and arrangement of elements and interfaces.
- **Dynamic complexity**, related to feedback, delays, and changing behavior over time.
- **Socio-technical complexity**, arising from interactions among people, technologies, organizations, and institutions.
- **Contextual complexity**, arising from environmental uncertainty, stakeholder diversity, and changing operational conditions.

These dimensions often interact, making the behavior of complex systems difficult to predict or control through reductionist analysis alone. Sheard and Mostashari (2011) provide a systems engineering perspective on different types of complexity and how they affect engineering practice.

Emergence in Systems

Emergence refers to properties or behaviors that arise at the level of the system as a whole and cannot be attributed to any individual component.

Emergent properties are meaningful only when the system is considered as an organized whole. Checkland describes emergence as the principle that entities exhibit properties that are meaningful when attributed to the whole, not to its parts (Checkland 1999). In systems science, emergence is closely associated with holism, interaction, organization, and system context.

Examples of emergence include:

- chemical compounds exhibiting properties different from their constituent elements
- coordinated movement in animal groups arising from local interactions
- consciousness arising from networks of neurons
- traffic patterns emerging from interactions among drivers, vehicles, and infrastructure.
- resilience or adaptability emerging from the organization of a socio-technical system

Many important properties of engineered systems are also emergent. These include performance, safety, reliability, resilience, usability, interoperability, and mission effectiveness. Such properties arise from the combined effects of architecture, component behavior, human use, operational context, and environmental conditions.

Emergence is therefore not only a theoretical concept. It is central to systems engineering because many of the properties that matter most to stakeholders exist only at the whole-system level.

At the same time, emergence should not be treated as a mysterious or non-scientific phenomenon. Abbott (2006) argues that emergence can be understood without assuming that higher-level phenomena violate physical explanation. This view is important for systems engineering: emergent behavior may be difficult to predict or manage, but it is still produced through interactions among real components, structures, processes, and environments

Types of Emergence

Several forms of emergence have been described in the systems and complexity literature (Emmeche, Koppe, and Stjernfelt 1997; Bedau and Humphreys 2008). A useful distinction for systems engineering can be made between **simple**, **weak**, and **strong** emergence.

Simple Emergence

Simple emergence occurs when system-level properties arise from combinations of elements and relationships that are sufficiently well understood to be predicted.

For example, the aerodynamic lift generated by an aircraft arises from interactions among aerodynamic surfaces, propulsion systems, control systems, and airflow. Although lift is a system-level property, it can be analyzed using established physical principles and engineering models.

In engineered systems, many system-level performance measures can be understood as forms of simple emergence when their contributing factors are well characterized.

Weak Emergence

Weak emergence occurs when system-level behavior arises from interactions among components but cannot be predicted directly from component properties alone. Such behavior may require simulation, modeling, experimentation, or operational observation to understand.

Examples include:

- traffic congestion patterns
- collective behavior in animal groups
- market behavior
- organizational performance
- distributed behavior in complex software or networked systems

Weak emergence is especially important in systems engineering because it can often be explored through modeling, simulation, prototyping, and iterative development. Page (2009) discusses emergence and complexity in ways that support this practical distinction between behavior that can be analytically derived and behavior that must be explored through models or observation.

Strong Emergence

Strong emergence refers to unexpected system-level behavior that is not anticipated during design or analysis and may only become visible during integration, testing, or operation.

Strong emergence is common in systems of systems, where independently developed systems interact in new operational contexts. Examples include cascading failures in power grids, unexpected interactions between digital platforms, or unintended consequences in large-scale infrastructure systems.

Strong emergence may be beneficial or harmful. It can produce innovation, adaptability, and new capability, but it can also generate instability, failure, or unacceptable risk.

Ryan (2007) argues that emergence is coupled to scope rather than simply to hierarchical level. This is important for systems engineering because emergent properties depend on how the system of interest, its boundaries, its environment, and the purpose of inquiry are defined. A property may appear emergent when one scope is selected but become explainable when a broader or different scope is considered.

Emergence, Self-Organization, and Adaptation

Many emergent phenomena arise through **self-organization**, in which system structure or behavior develops through interactions among elements rather than through centralized control.

Self-organization is observed in many natural systems, including biological development, ecosystem dynamics, flocking behavior, and neural pattern formation. It is also increasingly relevant in engineered and socio-technical

systems, including distributed computing, autonomous systems, adaptive networks, and collaborative organizations.

Research in thermodynamics and nonlinear systems has shown that ordered structures can emerge in systems operating far from equilibrium (Nicolis and Prigogine 1977). In such systems, flows of energy, matter, or information can sustain organized patterns that would not exist in equilibrium conditions.

Complex adaptive systems research has further shown how adaptation can arise when interacting agents adjust their behavior in response to experience and changing environments (Holland 1995; Mitchell 2009). Such systems may evolve new patterns of organization over time.

For systems engineering, these insights are important because modern engineered systems increasingly operate within adaptive environments involving human behavior, organizational learning, digital technologies, and environmental change.

Complexity, Emergence, and System Knowledge

Complexity and emergence can be understood through the interaction of several fundamental dimensions of systems:

- **Structural organization** — how system elements are arranged and related.
- **Dynamic behavior** — how elements interact and change over time.
- **Information and control** — how signals, feedback, and constraints influence behavior.
- **Environmental context** — how surrounding conditions affect the system and are affected by it.

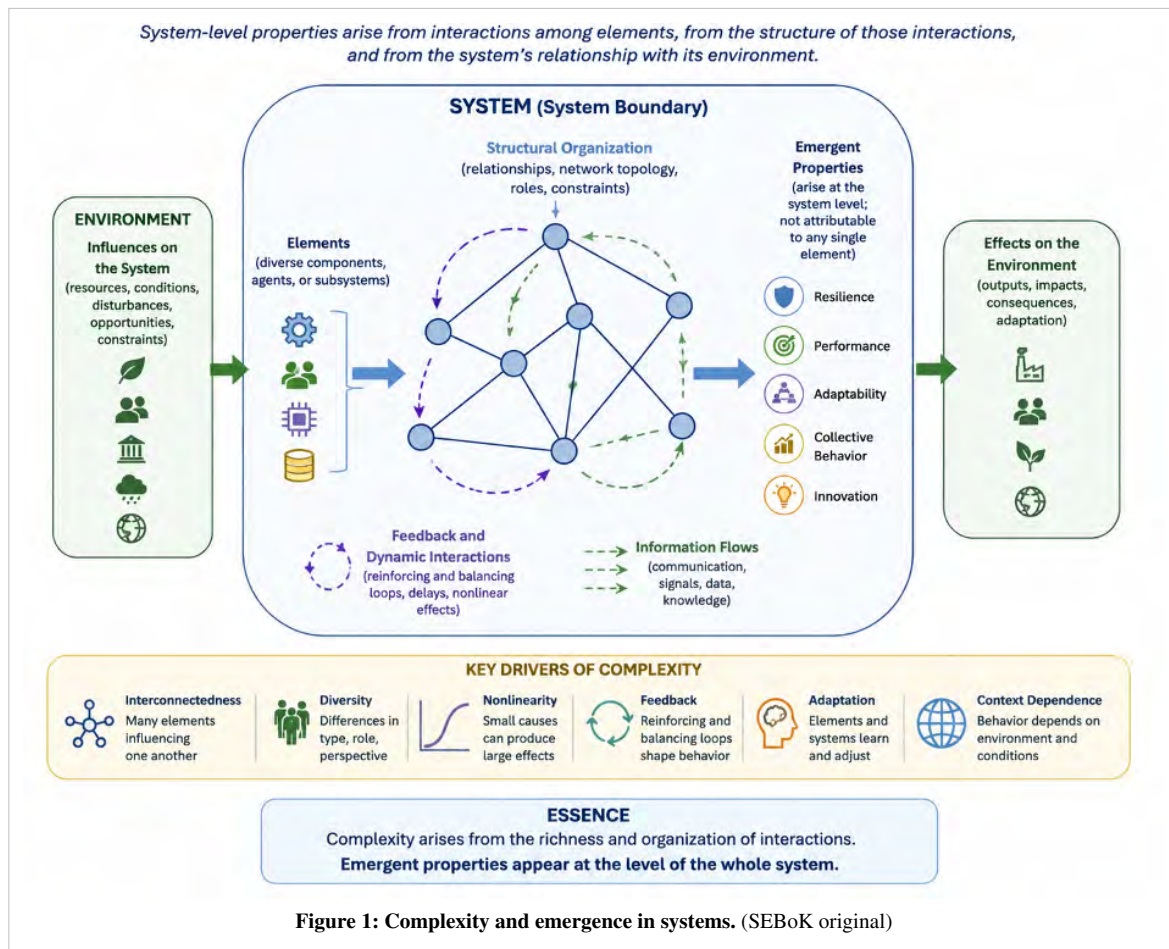
Emergent properties arise from the interaction of these dimensions. Structural arrangements enable and constrain interactions. Dynamic processes generate patterns of behavior over time. Information and feedback support coordination, regulation, and adaptation. Environmental influences shape how systems respond and evolve.

This relationship is important across the Systems Science KA. Complexity and emergence provide a bridge between:

- the structure and identity of systems
- the dynamics and feedback processes that shape behavior
- the information flows that sustain organization
- the multi-level structures through which systems evolve

In this sense, emergence is not an isolated phenomenon. It is a consequence of organized interaction within and across systems.

Work in relational systems theory, including Rosen's account of living systems, further emphasizes that some systems cannot be fully understood only as collections of components; the organization of relations is central to their identity and behavior (Rosen 1991). This reinforces the importance of studying system organization, not only system parts.



System-level properties arise from interactions among elements, from the structure of those interactions, and from the system's relationship with its environment. Complexity increases when relationships are numerous, diverse, nonlinear, adaptive, or context dependent. Emergent behavior appears at the level of the system as a whole and may be anticipated, explored, or discovered through analysis, modeling, experimentation, and operational learning.

Engineering Implications

Modern engineered systems frequently operate within complex socio-technical environments and interact with other systems. As a result, system behavior often cannot be completely predicted during design.

Systems engineering approaches therefore seek to manage complexity and emergence through techniques such as:

- architectural design and modularization
- interface management
- modeling and simulation
- iterative development and experimentation
- prototyping and test environments
- stakeholder engagement and socio-technical analysis
- monitoring and adaptation during operation
- learning from patterns across multiple systems

Simulation and modeling are particularly important for exploring potential emergent behaviors before deployment. However, some emergent behaviors can only be understood through operational experience.

Over time, successful patterns of system architecture and design can be captured and applied to future systems. Hybertson (2009) emphasizes the importance of model-oriented systems engineering knowledge, in which patterns

learned from multiple systems can support future design and analysis.

Research into recurring patterns across complex systems, including work by Troncale on systems isomorphies, suggests that many systemic processes appear across diverse domains of science (Troncale 1985). Recognizing such patterns may help systems engineers anticipate common mechanisms underlying complexity and emergence.

For systems engineers, the practical challenge is not to eliminate emergence. Many desirable system properties are emergent. The challenge is to increase the likelihood of desirable emergence while reducing the likelihood and impact of harmful or unexpected emergence.

Sillitto (2010) emphasizes the importance of design principles for ultra-large-scale systems, where interactions among many independently evolving elements may create both opportunities and risks. This reinforces the need for systems engineering approaches that combine architecture, modeling, iteration, operational learning, and systemic judgement.

Conclusion

Complexity and emergence are central characteristics of many natural, social, and engineered systems. Complexity arises from the organization and interaction of diverse system elements within changing environments. Emergence refers to system-level properties and behaviors that arise from those interactions.

Understanding complexity and emergence is essential for systems engineering practice, particularly when addressing large-scale, adaptive, or socio-technical systems. Many properties that matter most to stakeholders, including resilience, safety, adaptability, usability, and mission effectiveness, exist only at the level of the whole system.

By recognizing how interactions among structure, dynamics, information, and environment produce emergent outcomes, systems engineers can better design architectures, explore system behavior, anticipate unintended consequences, and support systems that remain viable under conditions of complexity and change.

The following article, *System Dynamics and Control*, examines how feedback processes and regulatory mechanisms shape such behavior over time.

References

- Abbott, R. 2006. "Emergence Explained: Getting Epiphenomena to Do Real Work." *Complexity* 12(1): 13–26.
- Ashby, W. R. 1956. *An Introduction to Cybernetics*. London, UK: Chapman & Hall.
- Bedau, M. A., and P. Humphreys, eds. 2008. *Emergence: Contemporary Readings in Philosophy and Science*. Cambridge, MA, USA: MIT Press.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. New York, NY, USA: Braziller.
- Checkland, P. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: Wiley.
- Emmeche, C., S. Koppe, and F. Stjernfelt. 1997. "Explaining Emergence: Towards an Ontology of Levels." *Journal for General Philosophy of Science* 28(1): 83–119.
- Holland, J. H. 1995. *Hidden Order: How Adaptation Builds Complexity*. Reading, MA, USA: Addison-Wesley.
- Hybertson, D. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boca Raton, FL, USA: CRC Press.
- Ladyman, J., J. Lambert, and K. Wiesner. 2013. "What Is a Complex System?" *European Journal for Philosophy of Science* 3: 33–67.
- Meadows, D. H. 2008. *Thinking in Systems: A Primer*. White River Junction, VT, USA: Chelsea Green.
- Mitchell, M. 2009. *Complexity: A Guided Tour*. Oxford, UK: Oxford University Press.
- Nicolis, G., and I. Prigogine. 1977. *Self-Organization in Nonequilibrium Systems*. New York, NY, USA: Wiley.

- Page, S. E. 2009. *Understanding Complexity*. Chantilly, VA, USA: The Teaching Company.
- Rosen, R. 1991. *Life Itself: A Comprehensive Inquiry Into the Nature, Origin, and Fabrication of Life*. New York, NY, USA: Columbia University Press.
- Ryan, A. J. 2007. "Emergence Is Coupled to Scope, Not Level." *Complexity* 13(2): 67–77.
- Sheard, S. A., and A. Mostashari. 2011. "Complexity Types: From Science to Systems Engineering." *INCOSE International Symposium*.
- Sillitto, H. G. 2010. "Design Principles for Ultra-Large-Scale Systems." *Proceedings of the 20th Annual INCOSE International Symposium*.
- Simon, H. A. 1962. "The Architecture of Complexity." *Proceedings of the American Philosophical Society* 106(6): 467–482.
- Troncale, L. R. 1985. "On the Possible Order of Systems Isomorphies." *General Systems* 30: 179–188.
- Weaver, W. 1948. "Science and Complexity." *American Scientist* 36(4): 536–544.

References

[1] https://youtu.be/7DVagCT5_HA

System Dynamics and Control

- Lead Author:
- Gary Smith

System dynamics and control examine how systems change over time and how feedback, interactions, and decision-making influence system behavior. Dynamic systems often exhibit patterns such as growth, delay, oscillation, adaptation, and unintended consequences that emerge through the relationships between system elements and their environments. Control mechanisms, whether automated, organizational, or human-driven, help systems maintain stability, respond to changing conditions, and pursue desired outcomes. Understanding these dynamics supports more effective analysis, prediction, and management of complex systems across engineering, natural, and socio-technical domains.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

Systems exist within changing environments and must continually adapt in order to remain viable. **System dynamics** refers to the patterns of change that arise in systems through interactions among components, feedback processes, and environmental influences. **Control** refers to the mechanisms through which systems regulate these dynamics in order to maintain stability, achieve goals, or respond to disturbances.

Understanding system dynamics and control is therefore fundamental to both systems science and systems engineering. System behavior rarely arises from simple linear cause-and-effect relationships; rather, it emerges from networks of interactions that may produce amplification, stabilization, oscillation, adaptation, or collapse (Meadows 2008; Sterman 2000).

These behaviors occur across many domains, including biological regulation, ecological systems, economic systems, engineered infrastructures, and socio-technical systems. The study of system dynamics and control provides a

foundation for understanding how systems maintain coherence while interacting with their environments.

An explainer video is also provided here ^[1] to support the introduction to the topic.

Dynamics as an Emergent Property of Systems

System dynamics arises from interactions among system elements and between the system and its environment. These interactions generate flows of matter, energy, and information that evolve over time. Because of these interactions, systems often exhibit behaviors that cannot be understood solely by examining individual components.

A number of recurring dynamic patterns appear across many types of systems:

- **Stabilizing behavior**, where feedback processes maintain equilibrium
- **Amplifying behavior**, where reinforcing processes drive growth or decline
- **Oscillatory behavior**, where interactions produce cycles or rhythmic patterns
- **Adaptive behavior**, where systems adjust to environmental change
- **Emergent behavior**, where new structures or patterns arise from collective interactions

These patterns are widely observed in natural systems such as ecosystems, metabolic networks, and planetary processes, as well as in engineered systems such as power grids, transportation systems, and communication networks (Bertalanffy 1968; Sterman 2000).

In many cases, system behavior is best understood as the result of **interacting feedback loops**, rather than simple linear causation.

Feedback and System Regulation

Feedback is one of the most fundamental mechanisms underlying system dynamics. A **feedback loop** occurs when the output of a process influences the conditions that produced it (Ashby 1956; Wiener 1948).

Two major types of feedback are commonly observed.

Reinforcing Feedback

Reinforcing feedback amplifies change within a system. In these loops, an increase in one variable leads to further increases in the same direction.

Examples include:

- population growth in biological systems
- viral spread in information networks
- compound interest in financial systems

Reinforcing feedback can drive rapid growth or decline and is often associated with nonlinear change and tipping points (Meadows 2008).

Balancing Feedback

Balancing feedback counteracts change and tends to stabilize a system around a target condition or equilibrium.

Examples include:

- temperature regulation in organisms
- homeostasis in physiological systems
- automatic control systems in engineering

Balancing feedback allows systems to maintain stability despite disturbances.

Most complex systems contain **multiple interacting feedback loops**, producing behaviors that are often difficult to anticipate without systemic analysis (Sterman 2000).

Co-Evolution of Systems and Their Environments

Systems do not exist in isolation. They are embedded within broader environments with which they continually interact. These interactions shape both the system and its environment over time.

Rather than treating systems as closed entities, systems science recognizes that **systems and environments co-evolve through ongoing exchanges of matter, energy, and information**. This relational perspective has roots in early systems thinking, including the work of Alexander Bogdanov, who emphasized that systems must be understood in terms of their interactions with surrounding processes (Bogdanov 1922/1980).

This insight has also been developed in ecological and social-ecological systems research, where systems and environments are viewed as coupled adaptive systems (Holling 2001; Ostrom 2009).

Engineered systems likewise exhibit this coupling. Infrastructure systems interact with ecological conditions, social behavior, economic forces, and technological change. Similarly, digital platforms evolve through interactions with users and wider technological ecosystems.

Understanding system dynamics therefore requires attention not only to internal structure but also to the **boundary interactions that connect systems with their environments**.

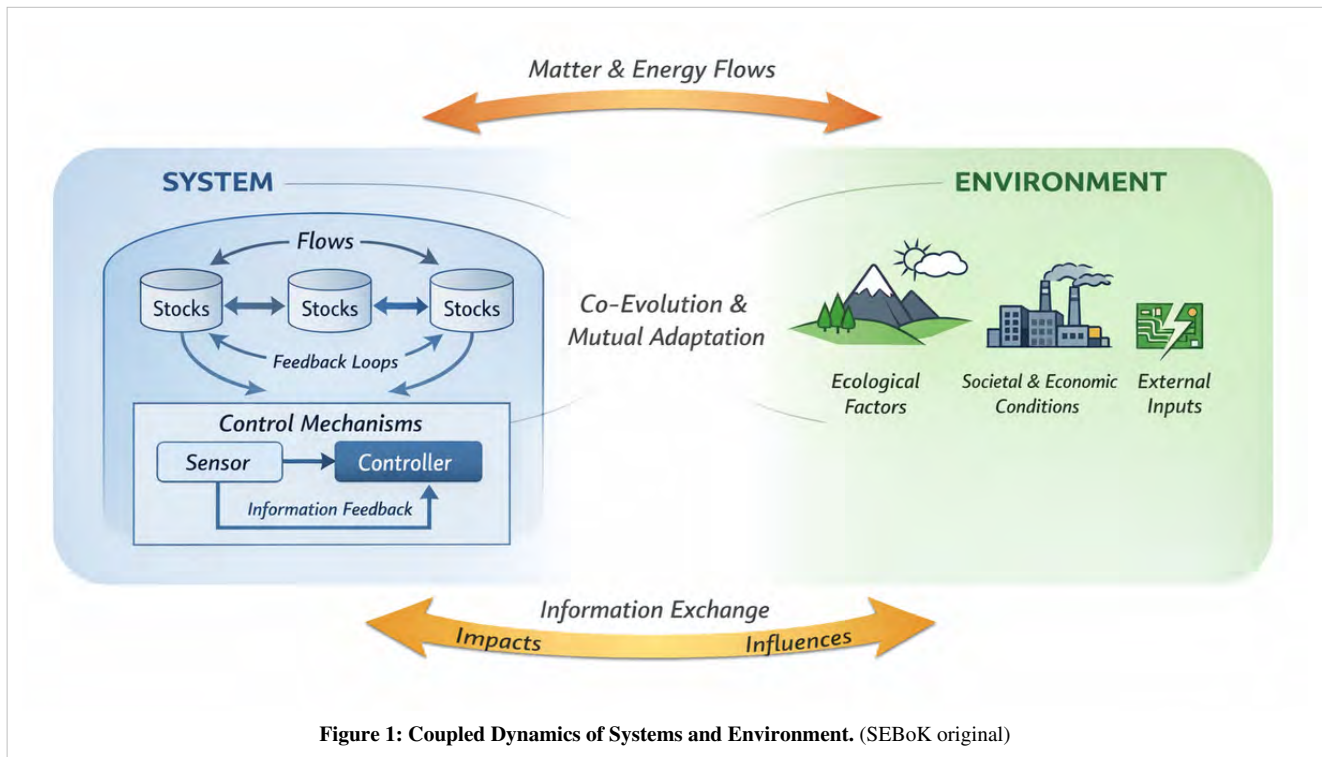


Figure 1 illustrates the coupled dynamics between a system and its environment. The system contains interacting stocks, flows, and feedback loops that generate behavior over time. These internal dynamics are influenced by environmental conditions, which both constrain and are altered by system activity.

The figure highlights several important principles:

- systems contain internal feedback structures that regulate behavior
- systems exchange matter, energy, and information with their environments
- environmental conditions influence system dynamics
- system activity can reshape environmental conditions over time

This bidirectional interaction reflects the view that systems and environments form **interacting processes within a larger dynamic context**, rather than independent entities.

Control Mechanisms in Natural and Engineered Systems

Control refers to the processes by which systems regulate behavior in order to maintain coherence, achieve goals, or respond to changing conditions.

In natural systems, control mechanisms often emerge through evolutionary processes. Examples include:

- biological homeostasis
- ecological regulation
- neural control in organisms

These mechanisms allow living systems to maintain stability while adapting to environmental variation (Ashby 1956; Maturana and Varela 1980).

In engineered systems, control mechanisms are often deliberately designed. Examples include:

- feedback controllers in industrial processes
- flight control systems in aircraft
- traffic management systems in urban infrastructure

Modern engineering increasingly recognizes that control must often be **distributed rather than centralized**, particularly in large-scale systems such as networks, transportation systems, and socio-technical infrastructures.

Dynamics in Social-Ecological-Technical Systems

Many contemporary systems combine technological infrastructure, human behavior, and environmental processes. These **social-ecological-technical systems** exhibit complex dynamics that cannot be understood by examining any single domain alone.

Examples include:

- climate and energy systems
- urban infrastructures
- food and water systems
- digital information ecosystems

In these contexts, system dynamics emerges from the interaction of:

- physical processes
- technological capabilities
- human decision-making
- institutional structures

Research on complex adaptive systems and social-ecological systems has increasingly emphasized the importance of understanding these coupled dynamics when addressing large-scale societal challenges (Holland 1995; Ostrom 2009).

Implications for Systems Engineering

For systems engineering, understanding system dynamics and control is essential for designing systems that are stable, adaptable, and resilient.

Engineered systems often operate within uncertain and evolving environments. Effective system design therefore requires attention to:

- feedback structures
- dynamic interactions between components
- system responses to disturbance
- adaptation and learning over time

Model-based systems engineering (MBSE) and simulation tools increasingly support the analysis of system dynamics during design and operation. These approaches allow engineers to explore how feedback processes and environmental interactions influence system behavior across the system lifecycle (INCOSE 2023).

By incorporating dynamic thinking into system design, engineers can better anticipate unintended consequences and improve system resilience.

Relationship to Information, Organization, and Entropy

System dynamics and control depend fundamentally on flows of information and the organization of system structures. Feedback processes rely on the detection of system states and the transmission of signals that influence future behaviour.

The next article, **Information, Organisation, and Entropy**, explores how information flows support the maintenance of system organization and how entropy-related processes influence system stability, adaptation, and transformation.

Together, these concepts deepen understanding of how systems maintain coherence while evolving within changing environments.

References

Works Cited

- Ashby, W. Ross. 1956. *An Introduction to Cybernetics*. London: Chapman & Hall.
- Bertalanffy, Ludwig von. 1968. *General System Theory: Foundations, Development, Applications*. New York: George Braziller.
- Bogdanov, Alexander. 1922/1980. *Essays in Tektology: The General Science of Organization*. Seaside, CA: Intersystems Publications.
- Holland, John H. 1995. *Hidden Order: How Adaptation Builds Complexity*. Reading, MA: Addison-Wesley.
- Holling, C. S. 2001. "Understanding the Complexity of Economic, Ecological, and Social Systems." *Ecosystems* 4(5): 390–405.
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 5th Edition.
- Maturana, Humberto R., and Francisco J. Varela. 1980. *Autopoiesis and Cognition: The Realization of the Living*. Dordrecht: D. Reidel.
- Meadows, Donella. 2008. *Thinking in Systems: A Primer*. White River Junction, VT: Chelsea Green.
- Ostrom, Elinor. 2009. "A General Framework for Analyzing Sustainability of Social-Ecological Systems." *Science* 325(5939): 419–422.

Sterman, John. 2000. *Business Dynamics: Systems Thinking and Modeling for a Complex World*. Boston: Irwin/McGraw-Hill.

Wiener, Norbert. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. Cambridge, MA: MIT Press.

Primary References

None.

Additional References

None.

References

[1] <https://youtu.be/PSaDCPyqn44>

Information, Organization, and Entropy in Systems

- Lead Author:
- Gary Smith

Information, organization, and entropy help explain how systems create, maintain, and lose structure over time. Information supports coordination, communication, adaptation, and decision-making within and across systems, while organization reflects the patterns and relationships that give systems coherence and purpose. Entropy describes the tendency toward disorder, uncertainty, or loss of structure when energy, information, or control are insufficient to sustain the system. Together, these concepts provide insight into how systems evolve, remain stable, adapt to change, and respond to disruption across natural, engineered, and socio-technical contexts.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

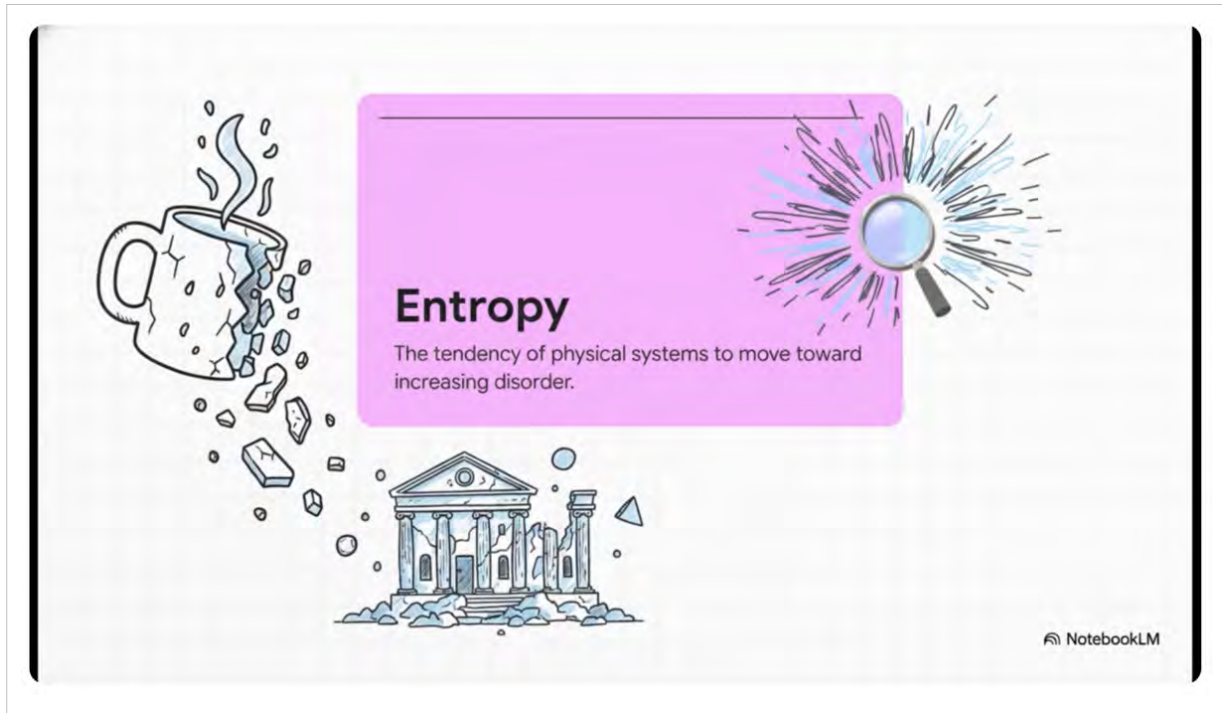
This article forms part of the Systems Science knowledge area (KA). It examines the relationships between **information, organization, and entropy**, concepts that help explain how organized systems arise and persist within a universe governed by thermodynamic constraints.

Earlier articles in this knowledge area examined recurring patterns in natural systems, the historical development of systems science, philosophical perspectives on systemness, structural organization, emergent phenomena, and the dynamic processes of feedback and regulation. These topics address how systems behave and regulate themselves over time. This article focuses on a more fundamental question: **how organized systems are able to exist and persist at all**.

Across physical, biological, social, and engineered domains, systems maintain organization through continuous **flows of energy, matter, and information** with their environments. These exchanges allow systems to sustain ordered structures, regulate internal processes, and adapt to environmental change.

Systems science investigates how organized structures can arise and persist in environments where physical processes tend toward increasing entropy. Real systems exist through **ongoing interaction with their environments**. Fully closed systems rarely occur in practice and are primarily theoretical idealizations used in physical modeling. Organized systems persist instead through dynamic coupling with their surroundings.

Understanding the relationships among **information, organization, and entropy** helps explain how complex systems maintain coherence despite the universal tendency toward increasing disorder.



Information in Systems

Information plays a central role in systemic processes because it enables systems to **sense conditions, coordinate internal activity, and regulate interactions with their environments**.

In systems science, information is often understood more broadly than in classical communication theory. It refers to **distinctions, signals, or constraints that influence the state or behavior of a system**. This systemic interpretation includes, but is not limited to, the formal definitions developed in communication theory.

Gregory Bateson described information as *“a difference that makes a difference,”* emphasizing that information becomes meaningful only when distinctions in the environment influence system behavior (Bateson 1972).

The formal study of information expanded significantly with the development of **information theory**. Shannon demonstrated that communication systems transmit information by reducing uncertainty within signals (Shannon 1948). Informational entropy provides a quantitative measure of uncertainty associated with possible system states.

Information processes appear across many types of systems:

- **Biological systems**, where genetic information regulates cellular processes
- **Ecological systems**, where environmental signals influence species interactions
- **Social systems**, where communication networks coordinate collective activity
- **Engineered systems**, where sensors and communication channels enable monitoring and control

Across these domains, information enables systems to interpret conditions and coordinate behavior.

Organization and System Structure

While information influences system behavior, **organization** refers to the relational structure that enables systems to function as coherent wholes.

Systems maintain identity through patterns of organization that persist even when individual components change. Living organisms illustrate this principle clearly: cellular components are continuously replaced through metabolic processes, yet the organism maintains continuity of identity.

Maturana and Varela described this phenomenon as **organizational closure**, where networks of processes continually regenerate the components necessary to sustain the system (Maturana and Varela 1980).

More generally, organization can be understood as the network of relationships that connects system elements and governs their interactions. These relationships determine:

- how information flows through the system
- how energy and resources are distributed
- how system behavior emerges from interactions among components

Organizational structure therefore provides the framework within which informational and energetic processes operate.

Entropy and the Maintenance of Organization

Entropy originates in thermodynamics and describes the tendency of physical systems toward increasing disorder.

If systems were completely isolated, entropy would steadily increase until equilibrium was reached and organized structures would eventually disappear. However, most real systems are **environmentally coupled systems** that exchange energy and matter with their surroundings.

Such systems can maintain internal organization by exporting entropy to their environments.

Schrödinger highlighted this phenomenon when he observed that living systems maintain order by drawing on “**negative entropy**” from their environments (Schrödinger 1944).

Later developments in nonequilibrium thermodynamics demonstrated that energy flows through such systems can generate organized structures. Prigogine and Nicolis showed that systems operating far from equilibrium may develop **dissipative structures**, organized patterns sustained by energy flows (Nicolis and Prigogine 1977).

Examples include:

- convection patterns in heated fluids
- oscillating chemical reactions
- metabolic cycles in biological organisms
- ecological nutrient cycles

These phenomena demonstrate that organization can arise from interactions between energy flows and system structure.

Information and Thermodynamics

The relationship between information and thermodynamics became clearer through developments in physics and computation.

A classic thought experiment illustrating this connection is **Maxwell’s demon**, proposed by James Clerk Maxwell in the nineteenth century. The demon was imagined as a microscopic observer capable of selectively allowing faster or slower molecules to pass between chambers, apparently reducing entropy by sorting particles. Later analysis revealed that the act of acquiring and processing information itself carries thermodynamic consequences, preserving the second law of thermodynamics.

Landauer demonstrated that information processing has **unavoidable physical consequences**, showing that logically irreversible operations, such as erasing information, require energy dissipation (Landauer 1961). This principle establishes that information processing is fundamentally constrained by thermodynamic laws.

Information therefore cannot be treated as purely abstract; it must always be embodied in physical systems.

Some physicists have suggested that information may play a deeper role in the structure of physical reality. Wheeler summarized this idea with the phrase “**it from bit**,” suggesting that informational distinctions may play a fundamental role in physical phenomena (Wheeler 1990). This formulation is best understood as a hypothesis highlighting the close relationship between physical processes and informational structure rather than as an established scientific consensus.

Stonier similarly proposed that organized structures can be interpreted as manifestations of accumulated information, suggesting that the evolution of complex systems involves the progressive transformation of informational organization (Stonier 1997).

These perspectives emphasize the deep connections between **information, physical processes, and systemic organization**.

Information, Energy, and System Organization

System organization emerges through the interaction of **energy flows, informational constraints, and structural relationships**.

Energy flows provide the physical processes that sustain system activity. Information constrains these processes by regulating interactions among system components. Organizational structure channels these flows into coordinated behavior.

Living systems illustrate this relationship clearly. Biological organisms maintain complex internal structures through metabolic processes that transform energy while genetic and biochemical information regulate cellular behavior.

Similar relationships appear in many other domains:

- ecosystems organize through flows of energy and nutrients regulated by ecological interactions
- economic systems organize through flows of resources coordinated by institutions and information
- engineered systems rely on sensing, communication, and control mechanisms to regulate physical processes

Across these domains, organization arises through the integration of **energy flows and information processing within structured networks of interaction**.

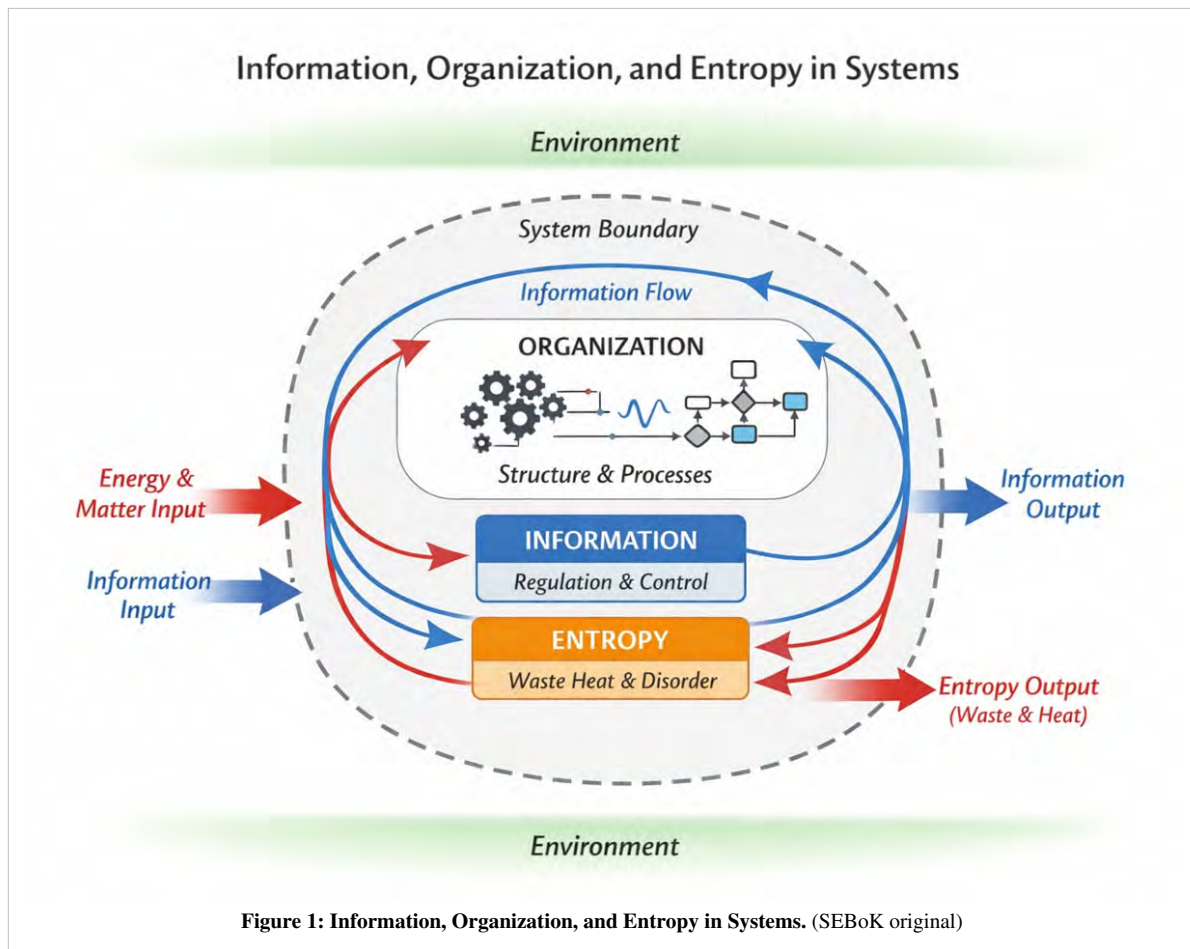


Figure 1 illustrates the relationships among **information flows, organizational structure, and entropy** within an environmentally coupled system.

The boundary represents a system interacting dynamically with its surrounding environment. Energy and matter enter the system and support internal processes. Informational inputs allow the system to sense conditions and regulate behavior.

Within the system boundary, organizational structures transform energy and information through networks of interactions. These processes generate coordinated activity and maintain systemic coherence.

At the same time, entropy is continually produced as a consequence of physical processes. To sustain internal order, the system exports entropy, often in the form of heat or waste, back into the environment. The continuous export of entropy allows the system to maintain internal organization despite thermodynamic constraints.

The figure highlights a central insight of systems science: **organized systems persist through the integration of energy flows, information processing, and structural relationships within ongoing environmental interaction.**

Informational Limits and System Viability

Although information enables organization, systems cannot sustain unlimited informational complexity.

Ashby's law of requisite variety states that regulators must possess sufficient informational capacity to respond to environmental disturbances (Ashby 1956). Systems lacking sufficient informational capacity cannot effectively regulate their interactions with their environments.

At the same time, excessive informational complexity may destabilize systems. As complexity increases, coordination becomes more difficult and the cost of maintaining organizational coherence rises.

Simon observed that complex systems often evolve modular or hierarchical structures because such architectures help manage informational complexity (Simon 1962). Tainter similarly argued that societies may collapse when the cost of sustaining increasing organizational complexity exceeds the benefits derived from it (Tainter 1988).

Research in complexity science suggests that adaptive systems frequently operate between extremes of rigid order and chaotic instability, sometimes described as the **edge of chaos** (Kauffman 1993).

Rose articulated a related view in which self-organizing systems maintain stability between extremes of insufficient and excessive informational interaction, suggesting that viable systems exist within ranges where informational exchange can be processed without overwhelming organizational capacity (Rose 1997).

These perspectives suggest that organized systems persist within **bounded regions of informational complexity that balance stability, coordination, and adaptability**.

Implications for Systems Engineering

Understanding the relationships among **information, organization, and entropy** is increasingly important for systems engineering.

Modern engineered systems frequently combine **physical infrastructure, digital information systems, and human organizations** within complex socio-technical networks. Examples include transportation systems, energy infrastructures, communication networks, and cyber-physical systems.

Designing such systems requires attention to:

- the structure and integrity of information flows
- sensing and monitoring architectures
- the ability of control mechanisms to regulate system behavior
- resilience under changing environmental conditions
- the management of informational complexity across distributed networks

Insights from systems science help engineers design systems that maintain organization and functionality within dynamic environments.

Conclusion

Information, organization, and entropy together provide a powerful framework for understanding how systems maintain coherence and evolve.

Information enables systems to sense conditions, communicate internally, and regulate behavior. Organizational structures provide the relational framework within which informational processes operate. Thermodynamic principles explain why systems must continually exchange energy and information with their environments in order to maintain organized structures.

Across many domains of science and engineering, similar relationships between information, structure, and dynamic processes appear repeatedly. Cybernetics, thermodynamics, information theory, and complexity science each contribute insights into these relationships.

By integrating these perspectives, systems science provides a coherent framework for understanding how organized systems emerge, persist, and evolve within a dynamic universe.

References

Works Cited

- Ashby, W. R. 1956. *An Introduction to Cybernetics*. London, UK: Chapman & Hall.
- Bateson, G. 1972. *Steps to an Ecology of Mind*. Chicago, IL, USA: University of Chicago Press.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. New York, NY, USA: George Braziller.
- Kauffman, S. A. 1993. *The Origins of Order: Self-Organization and Selection in Evolution*. New York, NY, USA: Oxford University Press.
- Landauer, R. 1961. "Irreversibility and Heat Generation in the Computing Process." *IBM Journal of Research and Development* 5(3): 183–191.
- Maturana, H., and F. Varela. 1980. *Autopoiesis and Cognition: The Realization of the Living*. Dordrecht, Netherlands: Reidel.
- Nicolis, G., and I. Prigogine. 1977. *Self-Organization in Nonequilibrium Systems*. New York, NY, USA: Wiley.
- Rose, J. N. 1997. *The Integrity Paradigm: An Explanation of the Behavior of Self-Organizing Systems*. Proceedings of the International Conference on Complex Systems.
- Schrödinger, E. 1944. *What Is Life?* Cambridge, UK: Cambridge University Press.
- Shannon, C. E. 1948. "A Mathematical Theory of Communication." *Bell System Technical Journal* 27: 379–423, 623–656.
- Simon, H. A. 1962. "The Architecture of Complexity." *Proceedings of the American Philosophical Society* 106(6): 467–482.
- Stonier, T. 1997. *Information and Meaning: An Evolutionary Perspective*. London, UK: Springer.
- Tainter, J. A. 1988. *The Collapse of Complex Societies*. Cambridge, UK: Cambridge University Press.
- Vedral, V. 2018. *Decoding Reality: The Universe as Quantum Information*. Oxford, UK: Oxford University Press.
- Wheeler, J. A. 1990. "Information, Physics, Quantum: The Search for Links." In W. H. Zurek (ed.), *Complexity, Entropy, and the Physics of Information*. Redwood City, CA: Addison-Wesley.

Primary References

None.

Additional References

None.

Systems, Levels, and Holarchy

- Lead Author:
- Gary Smith

Systems are often organized across multiple levels, with smaller systems interacting to form larger, more complex systems. Concepts such as hierarchy and holarchy help explain how systems can be simultaneously whole entities and parts of broader systems, each with their own structures, behaviors, and purposes. Relationships across levels influence communication, control, adaptation, and emergent behavior within and between systems. Understanding these layered interactions supports systems thinking by providing insight into how complexity, coordination, and interdependence operate across natural, engineered, organizational, and socio-technical systems.

Acknowledgement

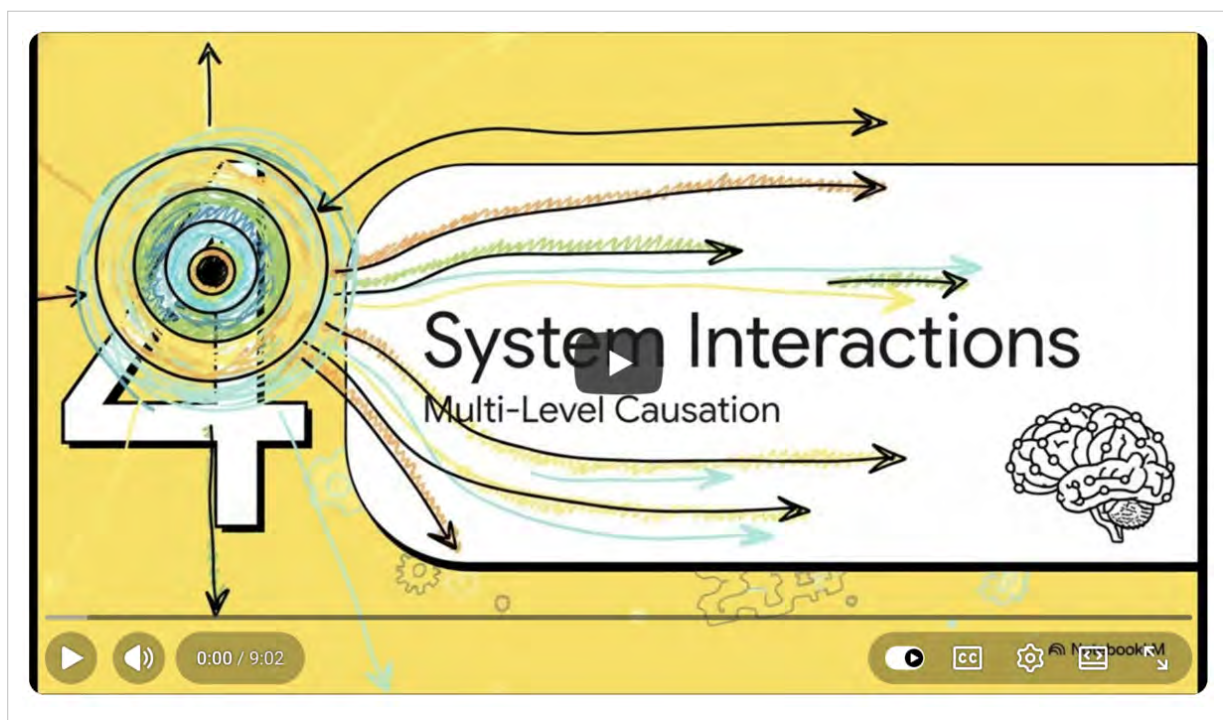
This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

This article forms part of the **Systems Science knowledge area (KA)**. It examines how many systems are organized across multiple levels of structure and scale, and how interactions among these levels influence system behavior.

Earlier articles in this knowledge area have explored the structural organization of systems, the emergence of new properties from interacting components, the dynamic processes that regulate system behavior, and the role of information in maintaining system organization. These discussions reveal that complex systems are rarely isolated entities. Instead, they exist within **nested arrangements of systems within systems**, forming layered structures across multiple scales.

Understanding how systems are organized across levels is therefore essential for understanding complexity. Systems science investigates how these layered structures arise, how interactions occur across levels, and how **systemic properties propagate through these interconnected structures**.



Levels of Organization in Systems

Across many domains of science, complex systems exhibit **nested levels of organization**. Systems are composed of subsystems that themselves contain smaller systems, while also functioning as components of larger systems.

Examples of such multi-level organization include:

- **Physical systems:** particles, atoms, molecules, materials, planetary systems
- **Biological systems:** cells, tissues, organs, organisms, ecosystems
- **Social systems:** individuals, groups, organizations, institutions, societies
- **Engineered systems:** components, subsystems, systems, and systems-of-systems

In each case, higher levels emerge from interactions among elements at lower levels, while simultaneously creating new contexts that influence the behavior of those elements.

The existence of such layered organization has long been recognized in systems science. Boulding (1956) described the universe as consisting of multiple levels of organization characterized by increasing complexity and new forms of behavior. These levels range from simple physical structures to biological organisms and complex social systems.

Hierarchy theory further refines this understanding by emphasizing that levels are often associated with **differences in scale, rate of change, and observational perspective** (Allen and Starr 1982). While these levels can be distinguished analytically, they are not independent. Instead, they form **interconnected structures in which processes occurring at one level both influence and are influenced by processes at other levels**.

Hierarchical Organization and the Architecture of Complexity

Complex systems frequently exhibit **hierarchical organization**, in which subsystems are grouped into larger structures. Simon (1962) argued that such hierarchical structures play a critical role in the evolution and stability of complex systems.

He introduced the concept of **nearly decomposable systems**, where interactions within subsystems are typically stronger than interactions between subsystems. This structural property enables:

- stability through modular organization
- scalability in system growth
- manageable complexity

Hierarchical organization allows complex systems to evolve through stable intermediate forms, reducing the need to manage all interactions simultaneously.

However, hierarchy should not be understood as rigid top-down control. Instead, it reflects a **pattern of organization that supports both independence and interdependence across levels**, enabling systems to balance stability with adaptability.

Holons and Holarchies

Koestler (1967) introduced the concept of the **holon** to describe entities that are simultaneously:

- **wholes in themselves**, and
- **parts of larger systems**

Holons form **holarchies**, nested structures in which each level maintains a degree of autonomy while contributing to larger organizational structures.

Holons exhibit two complementary tendencies:

- a **self-assertive tendency**, which maintains identity and integrity
 - an **integrative tendency**, which enables participation within larger systems
-

This balance between autonomy and integration allows complex systems to maintain coherence while remaining adaptable.

Holonic structures appear widely in biological systems, social institutions, and engineered systems, particularly where modular or layered architectures are present.

Boundary Formation and the Nature of Levels

Levels of organization depend fundamentally on the establishment of **boundaries** that distinguish systems from their environments. These boundaries may be:

- physical (e.g., membranes)
- functional (e.g., organizational roles)
- informational (e.g., models and representations)

Such boundaries are often **actively maintained**, rather than fixed.

Biological systems maintain cellular membranes, social systems maintain institutional and communicative boundaries, and conceptual systems are bounded through definitions and models. These boundary processes enable systems to persist as identifiable entities.

Maturana and Varela (1980) emphasize that living systems continuously produce and maintain their own boundaries, while Pattee (1973) highlights the role of symbolic and informational distinctions in defining system levels.

Levels of organization therefore arise not only from structure, but from the **ongoing processes that maintain distinctions between system and environment**.

Multi-Level Causation

The significance of levels becomes clearer when considering how causation operates across them.

System behavior arises from interactions that occur both within levels and across them. Three broad patterns of causal interaction can be identified:

- **Upward causation:** lower-level interactions give rise to higher-level structures and behaviors
- **Downward causation:** higher-level structures constrain and regulate lower-level processes (Campbell 1974)
- **Reciprocal causation:** feedback processes link multiple levels simultaneously

These causal relationships are central to understanding how systems maintain coherence while remaining responsive to change. Higher-level organization does not merely constrain lower-level processes—it also **enables coordinated behavior by providing structure and context**.

Functional Organization Across Levels

In addition to structural nesting, systems exhibit recurring **functional organization**.

Miller (1978) identified key subsystems required for living systems, including:

- boundary maintenance
- information processing
- resource and energy flows
- regulation and control

Similar functional patterns appear in social and engineered systems. Organizations, for example, require communication, coordination, resource allocation, and governance functions.

This recurring architecture reinforces the idea that levels of organization are not merely structural layers, but **patterns of organized activity that recur across multiple domains of reality**.

Levels of Complexity from Quarks to Culture

The progression of levels in the universe has been described by Volk (2003; 2017) as extending “**from quarks to culture**”, reflecting the emergence of increasingly complex forms of organization across cosmic, biological, and social evolution.

This progression includes:

- elementary particles
- atoms and molecules
- living cells
- multicellular organisms
- social and cultural systems

Transitions between levels often involve **major evolutionary transformations**, where new forms of organization emerge (Maynard Smith and Szathmáry 1995).

Within each level, recurring patterns, such as networks, flows, and feedback loops, appear, suggesting the presence of common systemic principles across domains.

Generative Processes in the Formation of Levels

The progression of levels in the universe has been described by Volk (2003; 2017) as extending “**from quarks to culture**”, reflecting the emergence of increasingly complex forms of organization across cosmic, biological, and social evolution.

This progression includes:

- elementary particles
- atoms and molecules
- living cells
- multicellular organisms
- social and cultural systems

Transitions between levels often involve **major evolutionary transformations**, where new forms of organization emerge (Maynard Smith and Szathmáry 1995).

Within each level, recurring patterns, such as networks, flows, and feedback loops, appear, suggesting the presence of common systemic principles across domains.

Constraints, Stability, and the Persistence of Levels

Not all system configurations persist. The emergence and stability of levels depend on constraints that regulate system behavior.

These include:

- energy and material constraints
- structural coherence
- environmental compatibility

Morowitz (2002) demonstrated that increasing levels of organization arise from **energy flowing through matter under far-from-equilibrium conditions**.

From this perspective, levels exist because systems are able to **maintain organized structure through continuous flows and constraints**. Systems that cannot sustain these conditions remain transient or fail to stabilize.

Cross-Scale Dynamics and Adaptive Cycles

Systems not only exist across levels, they interact dynamically across them.

The concept of **panarchy** (Gunderson and Holling 2002) describes systems as nested adaptive cycles operating across scales.

- “**revolt**”: small, fast processes triggering change at higher levels
- “**remember**”: large, slow processes stabilizing lower levels

These interactions show that levels are not fixed structures but **dynamically coupled processes evolving over time**.

Time Scales and the Separation of Dynamics

Levels of organization are often associated with characteristic **time scales**.

- lower levels → fast dynamics
- higher levels → slow dynamics

This separation of time scales contributes to system stability and enables coordination across levels (Allen and Starr 1982).

Domains of System Organization

Systems can be understood across several broad domains:

- **physical systems** (matter and energy)
- **living systems** (biological organization)
- **social systems** (institutions and societies)
- **conceptual systems** (knowledge, models, designs)

Each domain exhibits distinct forms of organization while sharing common systemic principles.

Hierarchy and Multi-Level Dynamics

The nested structure below represents a **hierarchy**, in which each level functions as both a whole system and a component of a larger system. Arrows in the figure indicate:

- **upward causation**, where new levels emerge
 - **downward causation**, where higher levels constrain and regulate
 - **reciprocal feedback**, linking levels dynamically
-

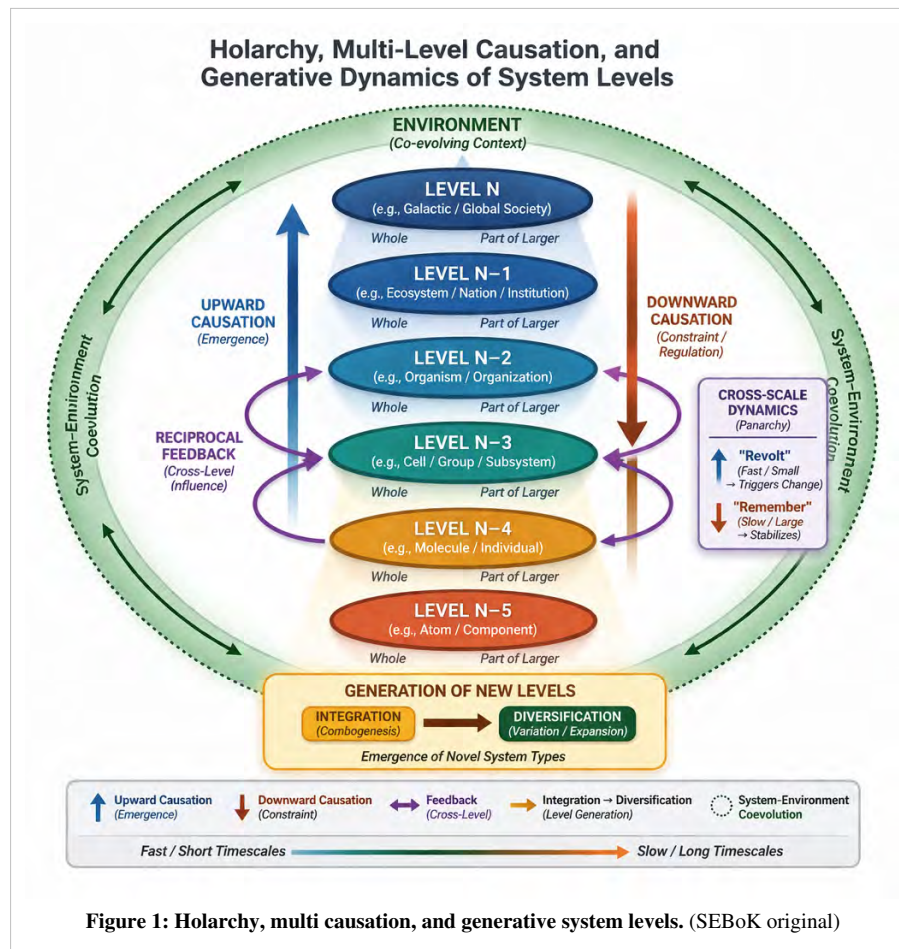


Figure 1 illustrates the organization of systems across levels and the processes that connect them.

The figure also highlights **generative dynamics**, where integration processes give rise to new levels, followed by diversification within those levels.

Surrounding the holarchy is a **co-evolving environment**, emphasizing that systems and environments are mutually defining and continuously interacting.

Cross-scale dynamics, represented by adaptive cycles, show how processes at different levels influence one another over time.

Levels of Organization and the Diversity of Scientific Inquiry

The layered organization of systems is reflected in the diversity of scientific disciplines, each focusing on particular levels of organization.

Systems science plays an **integrative role**, identifying principles that connect knowledge across domains and scales.

Implications for Systems Engineering

Understanding multi-level organization is essential in **systems engineering**, particularly for systems-of-systems.

Systems science helps engineers:

- manage complexity across levels
- anticipate emergent behavior
- design resilient and adaptive systems

These insights are especially important in socio-technical systems, where interactions span technical, organizational, and environmental domains.

Conclusion

Complex systems are organized across multiple levels of structure and scale, forming nested arrangements of systems within systems. Concepts such as hierarchy, holons, and holarchies help explain how these structures maintain coherence while supporting flexibility and adaptation.

Interactions across levels, including upward emergence, downward constraint, and reciprocal feedback, enable systemic properties to propagate through interconnected structures. These interactions are shaped by generative processes, thermodynamic constraints, and cross-scale dynamics.

Taken together, these perspectives reveal that levels of organization are not static features, but **dynamic and evolving patterns of organization that arise, stabilize, and transform through continuous processes of interaction, constraint, and integration within a co-evolving environment.**

References

Works Cited

- Allen, T. F. H., and T. B. Starr. 1982. *Hierarchy: Perspectives for Ecological Complexity*. Chicago, IL, USA: University of Chicago Press.
- Boulding, K. E. 1956. "General Systems Theory—The Skeleton of Science." *Management Science* 2(3): 197–208.
- Campbell, D. T. 1974. "Downward Causation in Hierarchically Organized Biological Systems." In *Studies in the Philosophy of Biology*.
- Gunderson, L. H., and C. S. Holling. 2002. *Panarchy: Understanding Transformations in Human and Natural Systems*. Washington, DC, USA: Island Press.
- Koestler, A. 1967. *The Ghost in the Machine*. London, UK: Hutchinson.
- Maier, M. W. 1998. "Architecting Principles for Systems-of-Systems." *Systems Engineering* 1(4): 267–284.
- Maturana, H. R., and F. J. Varela. 1980. *Autopoiesis and Cognition: The Realization of the Living*. Dordrecht, Netherlands: Reidel.
- Maynard Smith, J., and E. Szathmáry. 1995. *The Major Transitions in Evolution*. Oxford, UK: Oxford University Press.
- Miller, J. G. 1978. *Living Systems*. New York, NY, USA: McGraw-Hill.
- Mobus, G. E., and M. C. Kalton. 2015. *Principles of Systems Science*. New York, NY, USA: Springer.
- Morowitz, H. J. 2002. *The Emergence of Everything: How the World Became Complex*. Oxford, UK: Oxford University Press.
- Pattee, H. H. 1973. "Hierarchy Theory: The Challenge of Complex Systems." New York, NY, USA: George Braziller.
- Simon, H. A. 1962. "The Architecture of Complexity." *Proceedings of the American Philosophical Society* 106(6): 467–482.
- Volk, T. 2003. *Metapatterns: Across Space, Time, and Mind*. New York, NY, USA: Columbia University Press.
- Volk, T. 2017. *Quarks to Culture: How We Came to Be*. New York, NY, USA: Columbia University Press.

Primary References

None.

Additional References

None.

Systems Science and the Scientific Enterprise

- Lead Author:
- Gary Smith

Systems science developed alongside the broader scientific enterprise, drawing from and contributing to disciplines across the natural sciences, engineering, mathematics, and the social sciences. The interdisciplinary nature of systems science encouraged new ways of studying complexity, relationships, feedback, and behavior that could not be fully addressed within traditional disciplinary boundaries alone. Over time, systems-oriented approaches influenced scientific inquiry, modeling, problem solving, and collaboration across increasingly interconnected domains. These perspectives continue to shape how complex problems are investigated and understood in both research and practice.

Acknowledgement

This article reflects established knowledge from systems science and systems engineering, organized and collated for the SEBoK. Drafting support was provided by OpenAI's ChatGPT, with all content reviewed and finalized by the author, who retains full responsibility.

Introduction

This article forms part of the Systems Science knowledge area (KA). It considers the broader role of systems science within the scientific landscape, examining how systems science contributes to the organization, integration, and advancement of scientific knowledge.

Earlier articles in this knowledge area have explored recurring patterns in nature, the historical development of systems science, philosophical perspectives on systemness, and core concepts such as structure, dynamics, information, and multi-level organization. Taken together, these perspectives demonstrate that systems science is not defined by a particular domain of study, but by its focus on general principles of organization, interaction, and behavior that recur across many kinds of systems.

This perspective positions systems science as both:

- an **integrative field**, connecting knowledge across disciplines
- a **generative field**, contributing new concepts and theories
- a **reflexive field**, enabling understanding of science itself as a systemic enterprise

An explainer video is also provided here ^[1] to support the introduction to the topic.

Systems Science as an Integrative Field

Traditional scientific disciplines are typically organized around particular domains of reality:

- physics studies matter and energy
- biology studies living systems
- social sciences study human behavior and institutions

Each discipline develops specialized theories and methods suited to its domain.

Systems science, by contrast, focuses on patterns and principles that recur across these domains. Concepts such as:

- feedback
- emergence
- hierarchy
- adaptation
- information flow

are not confined to any single discipline, but appear across physical, biological, social, and engineered systems.

Bertalanffy (1968) described this as the search for general system laws. Boulding (1956) emphasized structural similarities across levels of organization. Troncale (1978) further articulated this integrative role through linkage propositions that connect concepts across disciplines.

Through these contributions, systems science enables the identification of *isomorphies*, shared patterns across systems, supporting the transfer and integration of knowledge across disciplinary boundaries.

Systems Science and the Organization of Knowledge

The integrative role of systems science extends beyond individual concepts to the organization of scientific knowledge itself.

Scientific disciplines can be understood as focusing on different domains and levels of organization, ranging from physical systems to biological and social systems. While this specialization has enabled deep advances, it has also contributed to fragmentation within the scientific enterprise.

Systems science helps address this fragmentation by:

- identifying common principles across domains and levels
- providing conceptual bridges between disciplines
- supporting the integration of knowledge across diverse fields

This integrative role can be understood in terms of **transversal knowledge**: system concepts, principles, and patterns that cut across domains and can be compared, related, and synthesized without collapsing their differences (Smith 2026). This perspective aligns with broader efforts toward the unity of knowledge, sometimes described as consilience (Wilson 1998), while maintaining a distinct focus on systemic organization, interaction, and structure..

Systems Science as a Generative Field

In addition to integrating knowledge, systems science contributes to the development of new scientific ideas and approaches.

Key developments influenced by systems science include:

- cybernetics (Wiener 1948; Ashby 1956)
 - system dynamics (Forrester 1961)
 - complexity science and self-organization (Nicolis and Prigogine 1977)
 - autopoiesis and biological organization (Maturana and Varela 1980)
-

These developments demonstrate that systems science is not only a framework for organizing knowledge, but also a source of new theoretical insights into how systems form, behave, and evolve.

By focusing on general principles rather than domain-specific details, systems science enables the discovery of patterns and relationships that might otherwise remain hidden.

Systems Science and Complexity

Many contemporary scientific challenges involve complex systems that span multiple domains, including:

- climate systems
- ecological systems
- socio-technical systems
- global economic systems

These systems are characterized by:

- nonlinearity
- feedback processes
- multi-level organization
- dynamic adaptation

Traditional disciplinary approaches alone are often insufficient to fully understand such systems.

Systems science provides tools and concepts that enable scientists and engineers to:

- analyze interactions across domains and levels
- understand emergent behavior
- model dynamic processes
- integrate knowledge across disciplines

In this way, systems science contributes directly to addressing complex, interconnected challenges.

Systems Science as a Science of Sciences

Beyond its integrative and generative roles, systems science contributes to a broader understanding of science itself.

Scientific knowledge is often organized into distinct disciplines. However, systems science highlights the extent to which these disciplines are interconnected through shared structures, processes, and principles.

In this sense, systems science has been described as aspiring toward a **science of sciences**: a field concerned not only with systems in the world, but with the relationships among domains of scientific inquiry themselves. Recent work has similarly emphasized that systems science requires its own internal coherence if it is to integrate diverse traditions without reducing their differences (Smith 2026).

This perspective builds on:

- Boulding's hierarchy of systems (1956)
- Troncale's linkage propositions (1978)

and extends them by emphasizing that knowledge itself is structured systemically, rather than simply accumulated within isolated domains.

Science as a Systemic Enterprise

From a systems perspective, science itself exhibits the characteristics of a complex system.

Scientific disciplines can be understood as interacting subsystems within a broader enterprise of knowledge production. These subsystems are linked through shared concepts, methods, and explanatory frameworks, forming a network of relationships rather than a collection of isolated domains.

Within this perspective:

- disciplines represent domains of inquiry
- theories provide conceptual structures
- methods define processes of investigation
- empirical findings contribute to evolving bodies of knowledge

One way to understand this systemic enterprise is to view science as a purposeful system that transforms observation into understanding and understanding into action. Within this view, scientific knowledge develops through interacting elements such as ideas, frameworks, principles, assets, and engagement with nature and society (Smith 2026). These elements are not linear stages but mutually reinforcing parts of an evolving knowledge system.

This view aligns with perspectives that treat knowledge as relational, structured, and transversal across domains. Systems science plays a central role in making these relationships explicit and intelligible.

Reflexivity and the Evolution of Systems Science

Systems science is itself subject to the principles it studies.

As a field, it exhibits key characteristics of systems:

- it consists of interacting components (disciplines, theories, methods)
- it evolves through the integration of ideas
- it adapts to new challenges and contexts
- it maintains coherence through shared concepts and frameworks

The development of systems science can therefore be understood as a self-organizing and evolving process.

This reflexive characteristic highlights the recursive nature of knowledge: the study of systems contributes to the evolution of systems of knowledge.

Systems Science and Societal Challenges

Systems science is increasingly applied to complex challenges with significant societal implications, including:

- environmental sustainability
- global health systems
- technological transformation
- socio-economic resilience

In such contexts, systems science contributes not only analytical tools, but also integrative perspectives that support more informed decision-making.

This highlights that systems science is not purely descriptive, but also plays an important role in shaping how complex problems are understood and addressed.

The Systems Science Knowledge Area as a System of Knowledge

Taken together, the articles within this Knowledge Area form a coherent system of knowledge about systems.

- Early articles establish **empirical and conceptual foundations**
- Intermediate articles develop **core systemic mechanisms**, including structure, dynamics, and information
- Later articles integrate these into **multi-level, relational, and epistemological perspectives**

These elements are interconnected:

- structure shapes dynamics
- dynamics are mediated by information
- information supports organization
- organization spans levels and holarchies
- interactions across levels give rise to emergence and complexity

This interconnected structure reflects the very principles that systems science seeks to understand. The Knowledge Area is therefore not only about systems, but is itself an example of a systematically organized body of knowledge. A coherent knowledge structure can act as an **enabling constraint**: it does not remove the diversity or complexity of systems science, but helps make the field more navigable, comparable, and useful (Smith 2026).

Systems Science within the SEBoK Content

Within the SEBoK:

- the **Systems Science KA** develops understanding of systemic principles
- the **Systems Thinking KA** applies these principles to reasoning and problem framing
- **Systems engineering** applies these insights to design and practice

This reflects a progression:

understanding systems → thinking systemically → applying systems in practice

Transition to Systems Thinking

While systems science provides an understanding of how systems are organized and behave, it does not in itself prescribe how to engage with complex systems in practice.

That role is addressed by systems thinking.

Systems thinking builds upon systems science by:

- applying systemic concepts to real-world situations
- supporting holistic reasoning and boundary selection
- enabling the exploration of interactions and perspectives
- guiding intervention in complex and uncertain contexts

The transition from systems science to systems thinking represents a shift from:

- understanding systems to
- thinking and acting systemically

As systems become increasingly interconnected and complex, this shift becomes essential for effective analysis, decision-making, and design.

Conclusion

Systems science occupies a distinctive position within the scientific landscape. It serves as both an integrative field, connecting knowledge across disciplines, and a generative field, contributing new theories and concepts that expand scientific understanding.

It also provides a basis for understanding science itself as a systemic enterprise, in which knowledge emerges through structured and relational interactions among domains, methods, and ideas.

Taken together, the Systems Science Knowledge Area presents a coherent framework for understanding how systems are structured, how they behave, how they are organized across levels, and how knowledge about them is itself organized.

These insights provide a foundation for the next stage: the application of systemic understanding through systems thinking, enabling more effective engagement with complex real-world systems.

References

Works Cited

- Ashby, W. R. 1956. *An Introduction to Cybernetics*. London, UK: Chapman & Hall.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. New York, NY, USA: Braziller.
- Boulding, K. E. 1956. "General Systems Theory—The Skeleton of Science." *Management Science* 2(3): 197–208.
- Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.
- Forrester, J. W. 1961. *Industrial Dynamics*. Cambridge, MA, USA: MIT Press.
- Maturana, H. R., and F. J. Varela. 1980. *Autopoiesis and Cognition: The Realization of the Living*. Dordrecht, Netherlands: Reidel.
- Mobus, G. E., and M. C. Kalton. 2015. *Principles of Systems Science*. New York, NY, USA: Springer.
- Nicolis, G., and I. Prigogine. 1977. *Self-Organization in Nonequilibrium Systems*. New York, NY, USA: Wiley.
- Smith, G. R. 2026. "A Meta-Framework for Organizing Transversal Knowledge About Systems." *Systems Research and Behavioral Science*. <https://doi.org/10.1002/sres.70067>.
- Troncale, L. R. 1978. "Linkage Propositions Between Fifty Principal Systems Concepts." In *Applied General Systems Research*, edited by G. J. Klir. Boston, MA, USA: Springer.
- Wiener, N. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. Cambridge, MA, USA: MIT Press.
- Wilson, E. O. 1998. *Consilience: The Unity of Knowledge*. New York, NY, USA: Knopf.

Primary References

None.

Additional References

None.

References

- [1] https://youtu.be/_w6T0hbdi5o

Knowledge Area: Systems Thinking

Systems Thinking

Contents of this Knowledge Area

- What is Systems Thinking? (Rick Adcock) (Brian Wells and Bud Lawson)
- Concepts of Systems Thinking (Rick Adcock) (Scott Jackson, Janet Singer, and Duane Hybertson)
- Principles of Systems Thinking (Rick Adcock) (Scott Jackson, Janet Singer, and Duane Hybertson)
- Patterns of Systems Thinking (Rick Adcock) (Scott Jackson, Janet Singer, and Duane Hybertson)
- Lead Author:
- Rick Adcock

This knowledge area (KA) provides a guide to knowledge about systems thinking which is the integrating paradigm for systems science and systems approaches to practice.

This is part of the wider systems knowledge which can help to provide a common language and intellectual foundation, and make practical systems concepts, principles, patterns and tools accessible to systems engineering (SE), as discussed in the Introduction to Part 2.

Introduction

Systems thinking is concerned with understanding or intervening in problem situations, based on the principles and concepts of the systems paradigm. This KA offers some basic definitions of systems thinking. The following diagram summarizes how the knowledge is presented.

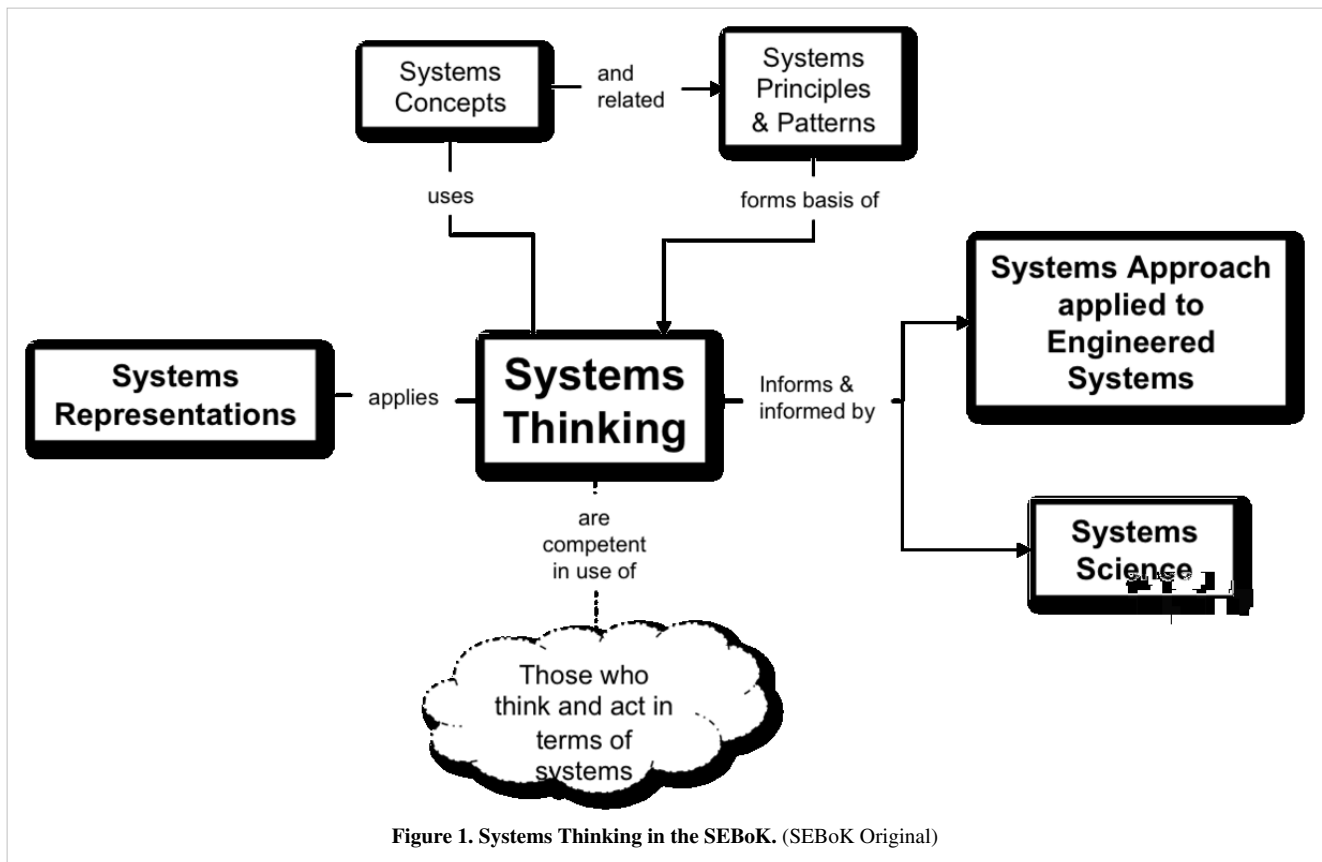


Figure 1. Systems Thinking in the SEBoK. (SEBoK Original)

Systems thinking considers the similarities between systems from different domains in terms of a set of common systems concepts, principles and patterns:

- A principle is a rule of conduct or behavior. To take this further, a principle is a “basic generalization that is accepted as true and that can be used as a basis for reasoning or conduct” (WordWeb.com).
- A concept is an abstraction, or a general idea inferred or derived from specific instances.

Principles depend on concepts in order to state a “truth.” Hence, principles and concepts go hand in hand; principles cannot exist without concepts and concepts are not very useful without principles to help guide the proper way to act (Lawson and Martin 2008).

Many sources combine both concepts and the principles based on them. The Concepts of Systems Thinking article presents concepts extracted from a variety of theory and practice sources. The Principles of Systems Thinking article, in turn, presents a summary of important principles referring back to the concepts upon which they are based.

A pattern is an expression of observable similarities found in systems from different domains. Patterns exist in both natural and man-made systems and are used in systems science and systems engineering. A summary of the different classes of patterns and the use of patterns to support a systems approach is discussed in the final Patterns of Systems Thinking article.

The practical application of systems thinking often employs the use of abstract system representations or models. Some mention of models is made in this KA; additionally, a more complete guide is provided in Representing Systems with Models.

References

Works Cited

Lawson, H., and J.N. Martin. 2008. "On the Use of Concepts and Principles for Improving Systems Engineering Practice," in Proceedings of the 18th Annual International Council on Systems Engineering (INCOSE) International Symposium, Utrecht, The Netherlands, 5-19 June 2008.

WordWeb Online. n.d. "Definition: Principle." Accessed Dec 3, 2014. Available at: WordWeb Online <https://www.wordwebonline.com/en/PRINCIPLE>.

Primary References

Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. Revised ed. New York, NY, USA: Braziller.

Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.

Churchman, C. W. 1968. *The Systems Approach and its Enemies*. New York, NY, USA: Dell Publishing.

Flood, R. L. 1999. *Rethinking the Fifth Discipline: Learning Within the Unknowable*. London UK: Routledge.

INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.1.

Additional References

Ackoff, R.L. 1971. "Towards a system of systems concepts," *Management Science*, vol. 17, no. 11.

Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4.

Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.

Ramage, M. and K. Shipp. 2009. *Systems Thinkers*. Dordrecht, The Netherlands: Springer.

Weinberg, G. M. 1975. *An Introduction to General Systems Thinking*. New York, NY, USA: Wiley.

What is Systems Thinking?

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells and Bud Lawson

This topic is part of the Systems Thinking knowledge area (KA). The scope of systems thinking is a starting point for dealing with real-world situations using a set of related systems concepts discussed in the Concepts of Systems Thinking topic, systems principles discussed in the Principles of Systems Thinking topic, and system patterns discussed in the Patterns of Systems Thinking topic.

Introduction

The concepts, principles, and patterns of systems thinking have arisen both from the work of systems scientists and from the work of practitioners applying the insights of systems science to real-world problems.

Holism has been a dominant theme in systems thinking for nearly a century, in recognition of the need to consider a system as a whole because of observed phenomena such as emergence. Proponents have included Wertheimer, Smuts, Bertalanffy, Weiss, (Ackoff 1979), (Klir 2001), and (Koestler 1967) among many others.

A more detailed discussion of the most important movements in systems theory can be found in History of Systems Science.

Identifying Systems of Interest

When humans observe or interact with a system, they allocate boundaries and names to parts of the system. This naming may follow the natural hierarchy of the system, but will also reflect the needs and experience of the observer to associate elements with common attributes of purposes relevant to their own. Thus, a number of systems of interest (SoIs) (Flood and Carson 1993) must be identified and they must be both relevant and include a set of elements which represent a system whole. This way of observing systems wherein the complex system relationships are focused around a particular system boundary is called **systemic resolution**.

Systems thinking requires an ongoing process of attention and adaptation to ensure that one has appropriately identified boundaries, dependencies, and relationships. Churchman (1968) and others have also considered broader ethical, political, and social questions related to management science with regards to the relative power and responsibility of the participants in system interventions. These are seen by critical systems thinkers as key factors to be considered in defining problem system boundaries.

A system context can be used to define a SoI and to capture and agree on the important relationships between it, such as the systems which it works with directly and the systems which influence it in some way. When this approach is used to focus on part of a larger system, a balance of reductionism and holism is applied. This balance sits at the heart of a systems approach. A systems context provides the tool for applying this balance and is thus an essential part of any systems approach and hence, of systems engineering (SE) as well. Approaches for describing the context of the different types of engineered systems are discussed in the Engineered System Context topic within the Systems Approach Applied to Engineered Systems KA.

Thoughts on Systems Thinking

Senge (1990) discusses systems thinking in a number of ways as

a discipline for seeing wholes ... a framework for seeing interrelationships rather than things ... a process of discovery and diagnosis ... and as a sensibility for the subtle interconnectedness that gives living systems their unique character. (Senge 2006, 68-69)

Churchman came to define a systems approach as requiring consideration of a system from the viewpoint of those outside its boundary (Churchman 1979). There are many demonstrations that choosing too narrow a boundary, either in terms of scope or timeline, results in the problem of the moment being solved only at the expense of a similar or bigger problem being created somewhere else in space, community, or time (Senge 2006) and (Meadows 1977). This is the "shifting the burden" archetype described in Patterns of Systems Thinking topic.

Churchman believes that an important component of system knowledge comes from "others" or "enemies" outside the system; the systems approach begins when first you see the world through the eyes of another (Churchman 1968). In this famous phrase, Churchman suggests that people can step outside a system they are in and mentally try to consider it through the lenses of other people's values. Churchman (1979) identified four main enemies of the systems approach namely: politics, morality, religion and aesthetics.

To Churchman, the "enemies" of the systems approach provide a powerful way of learning about the systems approach, precisely because they enable the rational thinker to step outside the boundary of a system and to look at it. It means that systems thinkers are not necessarily just involved within a system but are essentially involved in reasoning and decisions "outside" of systems rationality.

Some additional perspectives on systems thinking definitions are as follows:

- "Systems thinking requires the consciousness of the fact that we deal with models of our reality and not with the reality itself." (Ossimitz 1997, 1)
- "...what is often called 'systemic thinking' ...is ...a bundle of capabilities, and at the heart of it is the ability to apply our normal thought processes, our common sense, to the circumstances of a given situation." (Dörner 1996, 199)
- "Systems thinking provides a powerful way of taking account of causal connections that are distant in time and space." (Stacey 2000, 9)

Chaos and complexity theories have also impacted the development of systems thinking, including the treatment of such concepts as emergence. According to Gharajedaghi:

Systems thinking is the art of simplifying complexity. It is about seeing through chaos, managing interdependency, and understanding choice. We see the world as increasingly more complex and chaotic because we use inadequate concepts to explain it. When we understand something, we no longer see it as chaotic or complex. (Gharajedaghi 1999, 283)

Kasser considers systems thinking to be one element in a wider system of holistic thinking. Kasser defines holistic thinking as follows: "...the combination of analysis [in the form of elaboration], systems thinking and critical thinking" (Kasser 2010).

Systems Thinking and the Guide to the Systems Engineering Body of Knowledge

From these discussions, one can see systems thinking as both a set of founding ideas for the development of systems theories and practices and also as a pervasive way of thinking needed by those developing and applying those theories.

The SEBoK is particularly focused on how systems thinking can support a systems approach to engineered systems.

In order to examine a SoI in more detail, to understand, use, or change it in some way, practitioners are faced with an apparent “systems thinking paradox.” One can only truly understand a system by considering all of its possible relationships and interactions, inside and outside of its boundary and in all possible future situations (of both system creation and life), but this makes it apparently impossible for people to understand a system or to predict all of the consequences of changes to it.

If this means that all possible system relationships and environmental conditions must be considered to fully understand the consequences of creating or changing a system, what useful work can be done?

In many ways this is the essence of all human endeavors, whether they are technical, managerial, social or political, the so-called *known knowns* and *unknown unknowns*. The systems approach is a way of tackling real-world problems and making use of the concepts, principles and patterns of systems thinking to enable systems to be engineered and used.

The systems principles of encapsulation and separation of concerns in Principles of Systems Thinking relate to this issue. Some of the detail of complex situations must be hidden to allow focus on changes to a system element. The impact must be considered of any changes that might be made across sufficient related system components to fit within the acceptable commercial and social risks that must be considered. Engineering and management disciplines deal with this by gathering as much knowledge as necessary to proceed at a risk level acceptable to the required need. The assessment of what is enough and how much risk to take can, to some extent, be codified with rules and regulations, and managed through processes and procedures; however, it is ultimately a combination of the skill and judgment of the individuals performing the work.

References

Works Cited

- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*, Revised ed. New York, NY, USA: Braziller.
- Churchman, C.W. 1968. *The Systems Approach*. New York, NY, USA: Delacorte Press.
- Churchman, C.W. 1979. *The Systems Approach and Its Enemies*. New York, NY, USA: Basic Books.
- Checkland, P. 1981. *Systems Thinking, Systems Practice*. New York, NY, USA: Wiley.
- Dorner, H., and A. Karpoti. 2008. "Mentored innovation in teacher training using two virtual collaborative learning environments," in *Beyond Knowledge: The Legacy of Competence--Meaningful Computer-Based Learning Environments*, eds. J. Zumbach, N. Schwartz, T. Seufert and L. Kester. Vol. VIII. New York, NY, USA: Springer.
- Flood, R.L. and E.R. Carson. 1993. *Dealing with Complexity: An Introduction to the Theory and Application of Systems Science*, 2nd ed. New York, NY, USA: Plenum Press.
- Gharajedaghi, J. 1999. *Systems Thinking: Managing Chaos and Complexity: A Platform for Designing Business Architecture*, 1st ed. Woburn, MA, USA: Butterworth-Heinemann.
- Jackson, M. 1989. "Which Systems Methodology When? Initial Results from a Research Program," in R. Flood, M. Jackson and P. Keys (eds), *Systems Prospects: The Next Ten Years of Systems Research*. New York, NY, USA: Plenum.

Kasser, J. 2010. "Holistic thinking and how it can produce innovative solutions to difficult problems." Paper presented at 7th Bi-annual European Systems Engineering Conference (EuSEC), Stockholm, Sweden, 24-27 May 2010.

Meadows, D. H. et al. 1977. "Limits to Growth: A Report for the Club of Rome's Project on the Predicament of Mankind." New American Library, paperback, ISBN 0-451-13695-0; Universe Books, hardcover, 1972, ISBN 0-87663-222-3 (scarce).

Ossimitz, G. 1997. "The development of systems thinking skills using system dynamics modeling tools," in Universitat Klagenfurt [database online]. Klagenfurt, Austria: Universitat Klagenfurt. Accessed November 12 2007. Available at: https://www.uni-klu.ac.at/gossimit/sdyn/gdm_eng.htm.

Senge, P.M. 1990, 2006. *The Fifth Discipline: The Art and Practice of the Learning Organization*. New York, NY, USA: Doubleday Currency.

Stacey, R.D., D. Griffin, and P. Shaw. 2000. *Complexity and management: Fad or radical challenge to systems thinking?* London, U.K.: Routledge.

Primary References

Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*, Revised ed. New York, NY: Braziller.

Churchman, C.W. 1979. "The Systems Approach and its Enemies". New York: Basic Books.

Gharajedaghi, J. 1999. *Systems Thinking: Managing Chaos and Complexity: A Platform for Designing Business Architecture*, 1st ed. Woburn, MA, USA: Butterworth-Heinemann.

Senge, P.M. 1990, 2006. *The Fifth Discipline: The Art and Practice of the Learning Organization*. New York, NY, USA: Doubleday Currency.

Additional References

Jackson, M. 2003. *Systems Thinking: Creating Holisms for Managers*. Chichester, UK: Wiley.

Edson, R. 2008. *Systems Thinking. Applied. A Primer*, in: ASYST Institute (ed.). Arlington, VA: Analytic Services.

Klir, G. 2001. *Facets of Systems Science*, 2nd ed. New York, NY, USA: Kluwer Academic/Plenum Publishers.

Koestler, A. 1967. *The Ghost in the Machine*. New York, NY, USA: Macmillan. Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, Kings College, UK.

MITRE. 2012. "Systems Engineering Guide." Accessed September 11, 2012. Available at: https://www.mitre.org/work/systems_engineering/guide.

Rebovich, G., Jr. 2005. "Systems thinking for the enterprise (new and emerging perspectives)," in *Volume 2 of Enterprise Systems Engineering Theory and Practice*. McLean, VA, USA: The MITRE Corporation.

Senge, P.M., A. Klieiner, C. Roberts, R.B. Ross, and B.J. Smith. 1994. *The Fifth Discipline Fieldbook: Strategies and Tools for Building a Learning Organization*. New York, NY, USA: Crown Business.

Concepts of Systems Thinking

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Scott Jackson, Janet Singer, and Duane Hybertson

This article forms part of the Systems Thinking knowledge area (KA). It describes systems concepts, knowledge that can be used to understand problems and solutions to support systems thinking.

The concepts below have been synthesized from a number of sources, which are themselves summaries of concepts from other authors. Ackoff (1971) proposed a system of system concepts as part of general system theory (GST); Skyttner (2001) describes the main GST concepts from a number of systems science authors; Flood and Carlson (1993) give a description of concepts as an overview of systems thinking; Hitchins (2007) relates the concepts to systems engineering practice; and Lawson (2010) describes a system of system concepts where systems are categorized according to fundamental concepts, types, topologies, focus, complexity, and roles.

Wholeness and Interaction

A system is defined by a set of elements which exhibit sufficient cohesion, or "togetherness," to form a bounded whole (Hitchins 2007; Boardman and Sauser 2008).

According to Hitchins, interaction between elements is the "key" system concept (Hitchins 2009, 60). The focus on interactions and holism is a push-back against the perceived reductionist focus on parts and provides recognition that in complex systems, the interactions among parts is at least as important as the parts themselves.

An open system is defined by the interactions between system elements within a system boundary and by the interaction between system elements and other systems within an environment (see *What is a System?*). The remaining concepts below apply to open systems.

Regularity

Regularity is a uniformity or similarity that exists in multiple entities or at multiple times (Bertalanffy 1968). Regularities make science possible and engineering efficient and effective. Without regularities, we would be forced to consider every natural and artificial system problem and solution as unique. We would have no scientific laws, no categories or taxonomies, and each engineering effort would start from a clean slate.

Similarities and differences exist in any set or population. Every system problem or solution can be regarded as unique, but no problem/solution is in fact entirely unique. The nomothetic approach assumes regularities among entities and investigates what the regularities are. The idiographic approach assumes each entity is unique and investigates the unique qualities of entities, (Bertalanffy 1975).

A very large amount of regularity exists in both natural systems and engineered systems. Patterns of systems thinking capture and exploit that regularity.

State and Behavior

Any quality or property of a system element is called an attribute. The state of a system is a set of system attributes at a given time. A **system event** describes any change to the environment of a system, and hence its state:

- **Static** - A single state exists with no events.
- **Dynamic** - Multiple possible stable states exist.
- **Homeostatic** - System is static but its elements are dynamic. The system maintains its state by internal adjustments.

A stable state is one in which a system will remain until another event occurs.

State can be monitored using state variables, values of attributes which indicate the system state. The set of possible values of state variables over time is called the "state space". State variables are generally continuous but can be modeled using a finite state model (or "state machine").

Ackoff (1971) considers "change" to be how a system is affected by events, and system behavior as the effect a system has upon its environment. A system can

- **react** to a request by turning on a light,
- **respond** to darkness by deciding to turn on the light, or
- **act** to turn on the lights at a fixed time, randomly or with discernible reasoning.

A stable system is one which has one or more stable states within an environment for a range of possible events:

- **Deterministic** systems have a one-to-one mapping of state variables to state space, allowing future states to be predicted from past states.
- **Non-Deterministic** systems have a many-to-many mapping of state variables; future states cannot be reliably predicted.

The relationship between determinism and system complexity, including the idea of chaotic systems, is further discussed in the Emergence and Complexity article.

Survival Behavior

Systems often behave in a manner that allows them to sustain themselves in one or more alternative viable states. Many natural or social systems have this goal, either consciously or as a "self-organizing" system, arising from the interaction between elements.

Entropy is the tendency of systems to move towards disorder or disorganization. In physics, entropy is used to describe how organized heat energy is "lost" into the random background energy of the surrounding environment (the 2nd Law of Thermodynamics). A similar effect can be seen in engineered systems. What happens to a building or garden left unused for any time? Entropy can be used as a metaphor for aging, skill fade, obsolescence, misuse, boredom, etc.

"Negentropy" describes the forces working in a system to hold off entropy. Homeostasis is the biological equivalent of this, describing behavior which maintains a "steady state" or "dynamic equilibrium." Examples in nature include human cells, which maintain the same function while replacing their physical content at regular intervals. Again, this can be used as a metaphor for the fight against entropy, e.g. training, discipline, maintenance, etc.

Hitchins (2007) describes the relationship between the viability of a system and the number of connections between its elements. Hitchins's concept of connected variety states that stability of a system increases with its connectivity (both internally and with its environment). (See variety.)

Goal Seeking Behavior

Some systems have reasons for existence beyond simple survival. Goal seeking is one of the defining characteristics of engineered systems:

- A **goal** is a specific outcome which a system can achieve in a specified time
- An **objective** is a longer-term outcome which can be achieved through a series of goals.
- An **ideal** is an objective which cannot be achieved with any certainty, but for which progress towards the objective has value.

Systems may be single-goal seeking (perform set tasks), multi-goal seeking (perform related tasks), or reflective (set goals to tackle objectives or ideas). There are two types of goal seeking systems:

- Purposive systems have multiple goals with some shared outcome. Such a system can be used to provide pre-determined outcomes within an agreed time period. This system may have some freedom to choose how to achieve the goal. If it has memory, it may develop processes describing the behaviors needed for defined goals. Most machines or software systems are purposive.
- Purposeful systems are free to determine the goals needed to achieve an outcome. Such a system can be tasked to pursue objectives or ideals over a longer time through a series of goals. Humans and sufficiently complex machines are purposeful.

Control Behavior

Cybernetics, the science of control, defines two basic control mechanisms:

- **Negative feedback**, maintaining system state against set objectives or levels.
- **Positive feedback**, forced growth or contraction to new levels.

One of the main concerns of cybernetics is the balance between stability and speed of response. A black-box system view looks at the whole system. Control can only be achieved by carefully balancing inputs with outputs, which reduces speed of response. A white-box system view considers the system elements and their relationships; control mechanisms can be embedded into this structure to provide more responsive control and associated risks to stability.

Another useful control concept is that of a "meta-system", which sits over the system and is responsible for controlling its functions, either as a black-box or white-box. In this case, behavior arises from the combination of system and meta-system.

Control behavior is a trade between:

- **Specialization**, the focus of system behavior to exploit particular features of its environment, and
- Flexibility, the ability of a system to adapt quickly to environmental change.

While some system elements may be optimized for specialization, a temperature sensitive switch, flexibility, or an autonomous human controller, complex systems must strike a balance between the two for best results. This is an example of the concept of dualism, discussed in more detail in Principles of Systems Thinking.

Variety describes the number of different ways elements can be controlled and is dependent on the different ways in which they can then be combined. The Law of Requisite Variety states that a control system must have at least as much variety as the system it is controlling (Ashby 1956).

Function

Ackoff defines functions as outcomes which contribute to goals or objectives. To have a function, a system must be able to provide the outcome in two or more different ways. (This is called **equifinality**.)

This view of function and behavior is common in systems science. In this paradigm, all system elements have behavior of some kind; however, to be capable of functioning in certain ways requires a certain richness of behaviors.

In most hard systems approaches, a set of functions are described from the problem statement and then associated with one or more alternative element structures (Flood and Carson 1993). This process may be repeated until a system component (implementable combination of function and structure) has been defined (Martin 1997). Here, function is defined as either a task or activity that must be performed to achieve a desired outcome or as a transformation of inputs to outputs. This transformation may be:

- **Synchronous**, a regular interaction with a closely related system, or
- **Asynchronous**, an irregular response to a demand from another system that often triggers a set response.

The behavior of the resulting system is then assessed as a combination of function and effectiveness. In this case, behavior is seen as an external property of the system as a whole and is often described as analogous to human or organic behavior (Hitchins 2009).

Hierarchy, Emergence and Complexity

System behavior is related to combinations of element behaviors. Most systems exhibit **increasing variety**; i.e., they have behavior resulting from the combination of element behaviors. The term "synergy," or weak emergence, is used to describe the idea that the whole is greater than the sum of the parts. This is generally true; however, it is also possible to get **reducing variety**, in which the whole function is less than the sum of the parts (Hitchins 2007).

Complexity frequently takes the form of hierarchies. Hierarchic systems have some common properties independent of their specific content, and they will evolve far more quickly than non-hierarchic systems of comparable size (Simon 1996). A natural system hierarchy is a consequence of wholeness, with strongly cohesive elements grouping together forming structures which reduce complexity and increase robustness (Simon 1962).

Encapsulation is the enclosing of one thing within another. It may also be described as the degree to which it is enclosed. System encapsulation encloses system elements and their interactions from the external environment, and usually involves a system boundary that hides the internal from the external; for example, the internal organs of the human body can be optimized to work effectively within tightly defined conditions because they are protected from extremes of environmental change.

Socio-technical systems form what are known as control hierarchies, with systems at a higher level having some ownership of control over those at lower levels. Hitchins (2009) describes how systems form "preferred patterns" which can be used to enhance the stability of interacting systems hierarchies.

Looking across a hierarchy of systems generally reveals increasing complexity at the higher level, relating to both the structure of the system and how it is used. The term emergence describes behaviors emerging across a complex system hierarchy.

Effectiveness, Adaptation and Learning

Systems effectiveness is a measure of the system's ability to perform the functions necessary to achieve goals or objectives. Ackoff (1971) defines this as the product of the number of combinations of behavior to reach a function and the efficiency of each combination.

Hitchins (2007) describes effectiveness as a combination of **performance** (how well a function is done in ideal conditions), **availability** (how often the function is there when needed), and **survivability** (how likely is it that the system will be able to use the function fully).

System elements and their environments change in a positive, neutral or negative way in individual situations. An adaptive system is one that is able to change itself or its environment if its effectiveness is insufficient to achieve its current or future objectives. Ackoff (1971) defines four types of adaptation, changing the environment or the system in response to internal or external factors.

A system may also **learn**, improving its effectiveness over time without any change in state or goal.

References

Works Cited

- Ackoff, R.L. 1971. "Towards a system of systems concepts," *Management Science*, vol. 17, no. 11.
- Ackoff, R. 1979. "The future of operational research is past," *Journal of the Operational Research Society*, vol. 30, no. 2, pp. 93–104, Pergamon Press.
- Ashby, W R. 1956. "Chapter 11," in *Introduction to Cybernetics*. London, UK: Wiley.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*, Revised ed. New York, NY, USA: Braziller.
- Bertalanffy, L. von. 1975. *Perspectives on General System Theory*. E. Taschdjian, ed. New York, NY, USA: George Braziller.
- Boardman, J. and B. Sauser. 2008. *Systems Thinking: Coping with 21st Century Problems*. Boca Raton, FL, USA: Taylor & Francis.
- Flood, R.L. and E.R. Carson. 1993. *Dealing with Complexity: An Introduction to the Theory and Application of Systems Science*. New York, NY, USA: Plenum Press.
- Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: John Wiley and Sons.
- Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4, pp. 59-63.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.
- Martin, J. N. 1997. *Systems Engineering Guidebook*. Boca Raton, FL, USA: CRC Press.
- Skyttner, L. 2001. *General Systems Theory: Ideas and Applications*. Singapore: World Scientific Publishing Co., pp. 53-69.
- Simon, H.A. 1962. "The architecture of complexity," *Proceedings of the American Philosophical Society*, vol. 106, no. 6, December 12, pp. 467-482.
- Simon, H. 1996. *The Sciences of the Artificial*, 3rd ed. Cambridge, MA, USA: MIT Press.

Primary References

Ackoff, R.L. 1971. "Towards a system of systems concept," *Management Science*, vol. 17, no. 11.

Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4, pp 59-63.

Additional References

Edson, R. 2008. *Systems Thinking. Applied. A Primer*. Arlington, VA, USA: Applied Systems Thinking Institute (ASysT), Analytic Services Inc.

Jackson, S., D. Hitchins, and H. Eisner. 2010. "What is the systems approach?" *INCOSE Insight*, vol. 13, no. 1, April, , pp. 41-43.

Waring, A. 1996. "Chapter 1," in *Practical Systems Thinking*. London, UK: International Thomson Business Press.

Principles of Systems Thinking

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Scott Jackson, Janet Singer, and Duane Hybertson

This topic forms part of the Systems Thinking knowledge area (KA). It identifies systems principles as part of the basic ideas of systems thinking.

Some additional concepts more directly associated with engineered systems are described, and a summary of systems principles associated with the concepts already defined is provided. A number of additional “laws” and heuristics are also discussed.

Systems Principles, Laws, and Heuristics

A principle is a general rule of conduct or behavior (Lawson and Martin 2008). It can also be defined as a basic generalization that is accepted as true and that can be used as a basis for reasoning or conduct (WordWeb 2012c). Thus, systems principles can be used as a basis for reasoning about systems thinking or associated conduct (systems approaches).

Separation of Concerns

A systems approach is focused on a systems-of-interest (SoI) of an open system. This SoI consists of open, interacting subsystems that as a whole interact with and adapt to other systems in an environment. The systems approach also considers the SoI in its environment to be part of a larger, wider, or containing system (Hitchins 2009).

In the What is Systems Thinking? topic, a “systems thinking paradox” is discussed. How is it possible to take a holistic system view while still being able to focus on changing or creating systems?

Separation of concerns describes a balance between considering parts of a system problem or solution while not losing sight of the whole (Greer 2008). Abstraction is the process of taking away characteristics from something in order to reduce it to a set of base characteristics (SearchCIO 2012). In attempting to understand complex situations, it is easier to focus on bounded problems, whose solutions still remain agnostic to the greater problem (Erl 2012). This process sounds reductionist, but it can be applied effectively to systems. The key to the success of this approach is ensuring that one of the selected problems is the concern of the system as a whole. Finding balance between using abstraction to focus on specific concerns while ensuring the whole is continually considered is at the center of

systems approaches. A view is a subset of information observed of one or more entities, such as systems. The physical or conceptual point from which a view is observed is the viewpoint, which can be motivated by one or more observer concerns. Different views of the same target must be both separate, to reflect separation of concerns, and integrated such that all views of a given target are consistent and form a coherent whole (Hybertson 2009). Some sample views of a system are internal (Of what does it consist?), external (What are its properties and behavior as a whole?), static (What are its parts or structures?); and dynamic (interactions).

encapsulation, which encloses system elements and their interactions from the external environment, is discussed in Concepts of Systems Thinking. Encapsulation is associated with modularity, the degree to which a system's components may be separated and recombined (Griswold 1995). Modularity applies to systems in natural, social, and engineered domains. In engineering, encapsulation is the isolation of a system function within a module; it provides precise specifications for the module (IEEE Std. 610.12-1990).

Dualism is a characteristic of systems in which they exhibit seemingly contradictory characteristics that are important for the system (Hybertson 2009). The yin yang concept in Chinese philosophy emphasizes the interaction between dual elements and their harmonization, ensuring a constant dynamic balance through a cyclic dominance of one element and then the other, such as day and night (IEP 2006).

From a systems perspective, the interaction, harmonization, and balance between system properties is important. Hybertson (2009) defines **leverage** as the duality between:

- **Power**, the extent to which a system solves a specific problem, and
- **Generality**, the extent to which a system solves a whole class of problems.

While some systems or elements may be optimized for one extreme of such dualities, a dynamic balance is needed to be effective in solving complex problems.

Summary of Systems Principles

A set of systems principles is given in Table 1 below. The "Names" segment points to concepts underlying the principle. (See Concepts of Systems Thinking). Following the table, two additional sets of items related to systems principles are noted and briefly discussed: prerequisite laws for design science, and heuristics and pragmatic principles.

Table 1. A Set of Systems Principles. (SEBoK Original)

Name	Statement of Principle
Abstraction	A focus on essential characteristics is important in problem solving because it allows problem solvers to ignore the nonessential, thus simplifying the problem (Sci-Tech Encyclopedia 2009; SearchCIO 2012; Pearce 2012).
Boundary	A boundary or membrane separates the system from the external world. It serves to concentrate interactions inside the system while allowing exchange with external systems (Hoagland, Dodson, and Mauck 2001).
Change	Change is necessary for growth and adaptation, and should be accepted and planned for as part of the natural order of things rather than something to be ignored, avoided, or prohibited (Bertalanffy 1968; Hybertson 2009).
Dualism	Recognize dualities and consider how they are, or can be, harmonized in the context of a larger whole (Hybertson 2009).
Encapsulation	Hide internal parts and their interactions from the external environment (Klerer 1993; IEEE 1990).
Equifinality	In open systems, the same final state may be reached from different initial conditions and in different ways (Bertalanffy 1968). This principle can be exploited, especially in systems of purposeful agents.
Holism	A system should be considered as a single entity, a whole, not just as a set of parts (Ackoff 1979; Klir 2001).
Interaction	The properties, capabilities, and behavior of a system are derived from its parts, from interactions between those parts, and from interactions with other systems (Hitchins 2009 p. 60).
Layer Hierarchy	The evolution of complex systems is facilitated by their hierarchical structure (including stable intermediate forms) and the understanding of complex systems is facilitated by their hierarchical description (Pattee 1973; Bertalanffy 1968; Simon 1996).

Leverage	Achieve maximum leverage (Hybertson 2009). Because of the power versus generality trade-off, leverage can be achieved by a complete solution (power) for a narrow class of problems, or by a partial solution for a broad class of problems (generality).
Modularity	Unrelated parts of the system should be separated, and related parts of the system should be grouped together (Griswold 1995; Wikipedia 2012a).
Network	The network is a fundamental topology for systems that forms the basis of togetherness, connection, and dynamic interaction of parts that yield the behavior of complex systems (Lawson 2010; Martin et al. 2004; Sillitto 2010).
Parsimony	One should choose the simplest explanation of a phenomenon, the one that requires the fewest assumptions (Cybernetics 2012). This applies not only to choosing a design, but also to operations and requirements.
Regularity	Systems science should find and capture regularities in systems, because those regularities promote systems understanding and facilitate systems practice (Bertalanffy 1968).
Relations	A system is characterized by its relations: the interconnections between the elements. Feedback is a type of relation. The set of relations defines the network of the system (Odum 1994).
Separation of Concerns	A larger problem is more effectively solved when decomposed into a set of smaller problems or concerns (Erl 2012; Greer 2008).
Similarity/Difference	Both the similarities and differences in systems should be recognized and accepted for what they are (Bertalanffy 1975 p. 75; Hybertson 2009). Avoid forcing one size fits all, and avoid treating everything as entirely unique.
Stability/Change	Things change at different rates, and entities or concepts at the stable end of the spectrum can and should be used to provide a guiding context for rapidly changing entities at the volatile end of the spectrum (Hybertson 2009). The study of complex adaptive systems can give guidance to system behavior and design in changing environments (Holland 1992).
Synthesis	Systems can be created by "choosing (conceiving, designing, selecting) the right parts, bringing them together to interact in the right way, and in orchestrating those interactions to create requisite properties of the whole, such that it performs with optimum effectiveness in its operational environment, so solving the problem that prompted its creation" (Hitchins 2009: 120).
View	Multiple views, each based on a system aspect or concern, are essential to understand a complex system or problem situation. One critical view is how concern relates to properties of the whole (Edson 2008; Hybertson 2009).

The principles are not independent. They have synergies and trade-offs. Lipson (2007), for example, argued that "scalability of open-ended evolutionary processes depends on their ability to exploit functional modularity, structural regularity and hierarchy." He proposed a formal model for examining the properties, dependencies, and trade-offs among these principles. Edson (2008) related many of the above principles in a structure called the conceptagon, which he modified from the work of Boardman and Sauser (2008). Edson also provided guidance on how to apply these principles. Not all principles apply to every system or engineering decision. Judgment, experience, and heuristics (see below) provide understanding into which principles apply in a given situation.

Several principles illustrate the relation of view with the dualism and yin yang principle, for example, holism and separation of concerns. These principles appear to be contradictory but are in fact dual ways of dealing with complexity. Holism deals with complexity by focusing on the whole system, while separation of concerns divides a problem or system into smaller, more manageable elements that focus on particular concerns. They are reconciled by the fact that both views are needed to understand systems and to engineer systems; focusing on only one or the other does not give sufficient understanding or a good overall solution. This dualism is closely related to the systems thinking paradox described in What is Systems Thinking?.

Rosen (1979) discussed "false dualisms" of systems paradigms that are considered incompatible but are in fact different aspects or views of reality. In the present context, they are thus reconcilable through yin yang harmonization. Edson (2008) emphasized viewpoints as an essential principle of systems thinking; specifically, as a way to understand opposing concepts.

Derick Hitchins (2003) produced a systems life cycle theory described by a set of seven principles forming an integrated set. This theory describes the creation, manipulation and demise of engineered systems. These principles consider the factors which contribute to the stability and survival of man-made systems in an environment. Stability

is associated with the principle of **connected variety**, in which stability is increased by variety, plus the **cohesion** and **adaptability** of that variety. Stability is limited by allowable relations, resistance to change, and patterns of interaction. Hitchins describes how interconnected systems tend toward a **cyclic progression**, in which variety is generated, dominance emerges to suppress variety, dominant modes decay and collapse and survivors emerge to generate new variety.

Guidance on how to apply many of these principles to engineered systems is given in the topic Synthesizing Possible Solutions, as well as in System Definition and other knowledge areas in Part 3 of the SEBoK.

Prerequisite Laws of Design Science

John Warfield (1994) identified a set of laws of generic design science that are related to systems principles. Three of these laws are stated here:

1. "Law of Requisite Variety": A design situation embodies a variety that must be matched by the specifications. The variety includes the diversity of stakeholders. This law is an application of the design science of the Ashby (1956) Law of Requisite Variety, which was defined in the context of cybernetics and states that to successfully regulate a system, the variety of the regulator must be at least as large as the variety of the regulated system.
2. "Law of Requisite Parsimony": Information must be organized and presented in a way that prevents human information overload. This law derives from Miller's findings on the limits of human information processing capacity (Miller 1956). Warfield's structured dialog method is one possible way to help achieve the requisite parsimony.
3. "Law of Gradation": Any conceptual body of knowledge can be graded in stages or varying degrees of complexity and scale, ranging from simplest to most comprehensive, and the degree of knowledge applied to any design situation should match the complexity and scale of the situation. A corollary, called the Law of Diminishing Returns, states that a body of knowledge should be applied to a design situation at the stage at which the point of diminishing returns is reached.

Heuristics and Pragmatic Principles

A heuristic is a common sense rule intended to increase the probability of solving some problem (WordWeb 2012b). In the present context, it may be regarded as an informal or pragmatic principle. Maier and Rechtin (2000) identified an extensive set of heuristics that are related to systems principles. A few of these heuristics are stated here:

- Relationships among the elements are what give systems their added value. This is related to the "Interaction" principle.
- Efficiency is inversely proportional to universality. This is related to the "Leverage" principle.
- The first line of defense against complexity is simplicity of design. This is related to the "Parsimony" principle.
- In order to understand anything, you must not try to understand everything (attributed to Aristotle). This is related to the "Abstraction" principle.

An International Council on Systems Engineering (INCOSE) working group (INCOSE 1993) defined a set of "pragmatic principles" for systems engineering (SE). They are essentially best practice heuristics for engineering a system. For example:

- Know the problem, the customer, and the consumer
- Identify and assess alternatives to converge on a solution
- Maintain the integrity of the system

Hitchins defines a set of SE principles which include principles of holism and synthesis as discussed above, as well as principles describing how systems problems that are of particular relevance to a Systems Approach Applied to Engineered Systems should be resolved (Hitchins 2009).

References

Works Cited

- Ackoff, R. 1979. "The future of operational research is past," *Journal of the Operational Research Society*, vol. 30, no. 2, pp. 93–104, Pergamon Press.
- Ashby, W.R. 1956. "Requisite variety and its implications for the control of complex systems," *Cybernetica*, vol. 1, no. 2, pp. 1–17.
- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. Revised ed. New York, NY, USA: Braziller.
- Bertalanffy, L. von. 1975. *Perspectives on General System Theory*. E. Taschdjian, ed. New York, NY, USA: George Braziller.
- Boardman, J. and B. Sauser. 2008. *Systems Thinking: Coping with 21st Century Problems*. Boca Raton, FL, USA: Taylor & Francis.
- Cybernetics (Web Dictionary of Cybernetics and Systems). 2012. "Principle of Parsimony or Principle of Simplicity." Available at: Web Dictionary of Cybernetics and Systems [https:// pespmc1. vub. ac. be/ ASC/ PRINCI_SIMPL.html](https://pespmc1.vub.ac.be/ASC/PRINCI_SIMPL.html). Accessed December 3, 2014.
- Edson, R. 2008. *Systems Thinking. Applied. A Primer*. Arlington, VA, USA: Applied Systems Thinking (ASysT) Institute, Analytic Services Inc.
- Erl, T. 2012. "SOA Principles: An Introduction to the Service Orientation Paradigm." Available at: Arcitura [https:// www.soaprinciples.com/p3.php](https://www.soaprinciples.com/p3.php). Accessed December 3 2014.
- Greer, D. 2008. "The Art of Separation of Concerns." Available at: Aspiring Craftsman [https://aspiringcraftsman. com/tag/separation-of-concerns/](https://aspiringcraftsman.com/tag/separation-of-concerns/). Accessed December 3 2014
- Griswold, W. 1995. "Modularity Principle." Available at: William Griswold [https://cseweb.ucsd.edu/users/wgg/ CSE131B/Design/node1.html](https://cseweb.ucsd.edu/users/wgg/CSE131B/Design/node1.html). Accessed December 3 2014.
- Hitchins D. K. 2003. *Advanced Systems Thinking Engineering and Management*. Boston, MA, USA: Artech House.
- Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4, pp. 59-63.
- Hoagland, M., B. Dodson, and J. Mauck. 2001. *Exploring the Way Life Works*. Burlington, MA, USA: Jones and Bartlett Publishers, Inc.
- Holland, J. 1992. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. Cambridge, MA, USA: MIT Press.
- Hybertson, D. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boca Raton, FL, USA: Auerbach/CRC Press.
- IEEE. 1990. *IEEE Standard Glossary of Software Engineering Terminology*. Geneva, Switzerland: Institute of Electrical and Electronics Engineers. IEEE Std 610.12-1990.
- IEP (Internet Encyclopedia of Philosophy). 2006. "Yinyang (Yin-yang)." Available at: Internet Encyclopedia of Philosophy <https://www.iep.utm.edu/yinyang/>. Accessed December 3, 2014.
- INCOSE. 1993. *An Identification of Pragmatic Principles - Final Report*. SE Principles Working Group, January 21, 1993.
- Klerer, S. "System management information modeling," *IEEE Communications*, vol. 31, no. 5 May 1993, pp. 38-44.
- Klir, G. 2001. *Facets of Systems Science*, 2nd ed. New York, NY, USA: Kluwer Academic/Plenum Publishers.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College, UK.

- Lawson, H. and J. Martin. 2008. "On the use of concepts and principles for improving systems engineering practice." INCOSE International Symposium 2008, The Netherlands, 15-19 June 2008.
- Lipson, H. 2007. "Principles of modularity, regularity, and hierarchy for scalable systems," *Journal of Biological Physics and Chemistry*, vol. 7 pp. 125–128.
- Maier, M. and E. Rechtin. 2000. *The Art of Systems Architecting*, 2nd ed. Boca Raton, FL, USA: CRC Press.
- Miller, G. 1956. "The magical number seven, plus or minus two: Some limits on our capacity for processing information," *The Psychological Review*, vol. 63, pp. 81–97.
- Odum, H. 1994. *Ecological and General Systems: An Introduction to Systems Ecology (Revised Edition)*. Boulder, CO, USA: University Press of Colorado.
- Pattee, H., Ed. 1973. *Hierarchy Theory: The Challenge of Complex Systems*. New York, NY, USA: George Braziller.
- Pearce, J. 2012. "The Abstraction Principle." Available at: Jon Pearce, San Jose State University <https://www.cs.sjsu.edu/~pearce/modules/lectures/ood/principles/Abstraction.htm>. Accessed December 3 2014.
- Rosen, R. 1979. "Old trends and new trends in general systems research," *International Journal of General Systems*, vol. 5, no. 3, pp. 173-184.
- Sci-Tech Encyclopedia. 2009. "Abstract data type," in *McGraw-Hill Concise Encyclopedia of Science and Technology, Sixth Edition*, New York, NY, USA: The McGraw-Hill Companies, Inc.
- SearchCIO. 2012. "Abstraction." Available at: SearchCIO <https://searchcio-midmarket.techtarget.com/definition/abstraction>. Accessed December 3 2014.
- Sillitto, H. 2010. "Design principles for ultra-large-scale (ULS) systems," *Proceedings of INCOSE International Symposium 2010*, Chicago, IL, 12-15 July 2010.
- Simon, H. 1996. *The Sciences of the Artificial*, 3rd ed. Cambridge, MA, USA: MIT Press.
- Warfield, J.N. 1994. *A Science of Generic Design*. Ames, IA, USA: Iowa State University Press.
- Wikipedia. 2012a. "Modularity." Available at: Wikipedia <https://en.wikipedia.org/wiki/Modularity>. Accessed December 3 2014.
- WordWeb. 2012b. "Dualism." Available at: WordWeb <https://www.wordwebonline.com/en/DUALISM>. Accessed December 3 2014.
- WordWeb. 2012c. "Heuristic." Available at: WordWeb <https://www.wordwebonline.com/en/HEURISTIC>. Accessed December 3 2014.
- WordWeb. 2012d. "Principle." Available at: WordWeb <https://www.wordwebonline.com/en/PRINCIPLE>. Accessed December 3 2014.

Primary References

- Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. Revised ed. New York, NY, USA: Braziller.
- Hybertson, D. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boca Raton, FL, USA: Auerbach/CRC Press.
- Klir, G. 2001. *Facets of Systems Science*, 2nd ed. New York, NY, USA: Kluwer Academic/Plenum Publishers.
-

Additional References

- Francois, F. Ed. 2004. *International Encyclopedia of Systems and Cybernetics*, 2nd ed. Munich, Germany: K. G. Saur Verlag.
- Meyers, R. Ed. 2009. *Encyclopedia of Complexity and Systems Science*. New York, NY, USA: Springer.
- Midgley, G. Ed. 2003. *Systems Thinking*. Thousand Oaks, CA, USA: Sage Publications Ltd.
- Volk, T., and J.W. Bloom. 2007. "The use of metapatterns for research into complex systems of teaching, learning, and schooling. Part I: Metapatterns in nature and culture," *Complicity: An International Journal of Complexity and Education*, vol. 4, no. 1, pp. 25—43.

Patterns of Systems Thinking

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Scott Jackson, Janet Singer, and Duane Hybertson

This topic forms part of the Systems Thinking knowledge area (KA). It identifies systems patterns as part of the basic ideas of systems thinking. The general idea of patterns and a number of examples are described. A brief conclusion discusses the maturity of systems science from the perspective of principles and patterns.

Systems Patterns

This section first discusses definitions, types, and pervasiveness of patterns. Next, samples of basic patterns in the form of hierarchy and network patterns, metapatterns, and systems engineering (SE) patterns are discussed. Then samples of patterns of failure (or “antipatterns”) are presented in the form of system archetypes, along with antipatterns in software engineering and other fields. Finally, a brief discussion of patterns as maturity indicators is given.

Pattern Definitions and Types

The most general definition of pattern is that it is an expression of an observed regularity. Patterns exist in both natural and artificial systems and are used in both systems science and systems engineering (SE). Theories in science are patterns. Building architecture styles are patterns. Engineering uses patterns extensively.

Patterns are a representation of similarities in a set or class of problems, solutions or systems. In addition, some patterns can also represent uniqueness or differences, e.g., uniqueness pattern or unique identifier, such as automobile vehicle identification number (VIN), serial number on a consumer product, human fingerprints, DNA. The pattern is that a unique identifier, common to all instances in a class (such as a fingerprint), distinguishes between all instances in that class.

The term pattern has been used primarily in building architecture and urban planning (Alexander et al. 1977, Alexander 1979) and in software engineering (e.g., Gamma et al. 1995; Buschmann et al. 1996). Their definitions portray a pattern as capturing design ideas as an archetypal and reusable description. A design pattern provides a generalized solution in the form of templates to a commonly occurring real-world problem within a given context. A design pattern is not a finished design that can be transformed directly into a specific solution. It is a description or template for how to solve a problem that can be used in many different specific situations (Gamma et al. 1995; Wikipedia 2012b). Alexander placed significant emphasis on the pattern role of reconciling and resolving competing forces, which is an important application of the yin yang principle.

Other examples of general patterns in both natural and engineered systems include: conventional designs in engineering handbooks, complex system models such as evolution and predator-prey models that apply to multiple application domains, domain taxonomies, architecture frameworks, standards, templates, architecture styles, reference architectures, product lines, abstract data types, and classes in class hierarchies (Hybertson 2009). Shaw and Garlan (Garlan 1996) used the terms *pattern* and *style* interchangeably in discussing software architecture. Lehmann and Belady (Lehmann 1985) examined a set of engineered software systems and tracked their change over time and observed regularities that they captured as evolution laws or patterns.

Patterns have been combined with model-based systems engineering (MBSE) to lead to pattern-based systems engineering (PBSE) (Schindel and Smith 2002, Schindel 2005).

Patterns also exist in systems practice, both science and engineering. At the highest level, Gregory (1966) defined science and design as behavior patterns:

The scientific method is a pattern of problem-solving behavior employed in finding out the nature of what exists, whereas the design method is a pattern of behavior employed in inventing things of value which do not yet exist.

Regularities exist not only as positive solutions to recurring problems, but also as patterns of failure, i.e., as commonly attempted solutions that consistently fail to solve recurring problems. In software engineering these are called antipatterns, originally coined and defined by Koenig (1995). An antipattern is just like a pattern, except that instead of a solution it gives something that looks superficially like a solution but isn't one. Koenig's rationale was that if one does not know how to solve a problem, it may nevertheless be useful to know about likely blind alleys. Antipatterns may include patterns of pathologies (i.e., common diseases), common impairment of normal functioning, and basic recurring problematic situations. These antipatterns can be used to help identify the root cause of a problem and eventually lead to solution patterns. The concept was expanded beyond software to include project management, organization, and other antipatterns (Brown et al. 1998; AntiPatterns Catalog 2012).

Patterns are grouped in the remainder of this section into basic foundational patterns and antipatterns (or patterns of failure).

Basic Foundational Patterns

The basic patterns in this section consist of a set of hierarchy and network patterns, followed by a set of metapatterns and SE patterns.

Hierarchy and Network Patterns

The first group of patterns are representative types of hierarchy patterns distinguished by the one-to-many relation type (extended from Hybertson 2009, 90), as shown in the table below. These are presented first because hierarchy patterns infuse many of the other patterns discussed in this section.

Table 1. Hierarchy Patterns. (SEBoK Original)

Relation	Hierarchy Type or Pattern
Basic: Repeating One-to-Many Relation	General: Tree structure
Part of a Whole	Composition (or Aggregation) hierarchy
Part of + Dualism: Each element in the hierarchy is a holon, i.e., is both a whole that has parts and a part of a larger whole	Holarchy (composition hierarchy of holons) (Koestler 1967) - helps recognize similarities across levels in multi-level systems
Part of + Interchangeability: The parts are clonons, i.e., interchangeable	Composition Hierarchy of Clonons (Bloom 2005). Note: This pattern reflects horizontal similarity.
Part of + Self-Similarity: At each level, the shape or structure of the whole is repeated in the parts, i.e., the hierarchy is self-similar at all scales.	Fractal. Note: This pattern reflects vertical similarity.
Part of + Connections or Interactions among Parts	System composition hierarchy
Control of Many by One	Control hierarchy—e.g., a command structure
Subtype or Sub-Class	Type or specialization hierarchy; a type of generalization
Instance of Category	Categorization (object-class; model-metamodel...) hierarchy; a type of generalization

Network patterns are of two flavors. First, traditional patterns are network topology types, such as bus (common backbone), ring, star (central hub), tree, and mesh (multiple routes) (ATIS 2008). Second, the relatively young science of networks has been investigating social and other complex patterns, such as percolation, cascades, power law, scale-free, small worlds, semantic networks, and neural networks (Boccara 2004; Neumann et al. 2006).

Metapatterns

The metapatterns identified and defined in the table below are from (Bloom 2005), (Volk and Bloom 2007), and (Kapraff 1991). They describe a metapattern as convergences exhibited in the similar structures of evolved systems across widely separated scales (Volk and Bloom 2007).

Table 2. Metapatterns. (SEBoK Original)

Name	Brief Definition	Examples
Spheres	Shape of maximum volume, minimum surface, containment	Cell, planet, dome, ecosystem, community
Centers	Key components of system stability	Prototypes, purpose, causation; Deoxyribonucleic acid (DNA), social insect centers, political constitutions and government, attractors
Tubes	Surface transfer, connection, support	Networks, lattices, conduits, relations; leaf veins, highways, chains of command
Binaries Plus	Minimal and thus efficient system	Contrast, duality, reflections, tensions, complementary/symmetrical/reciprocal relationships; two sexes, two-party politics, bifurcating decision process
Clusters, Clustering	Subset of webs, distributed systems of parts with mutual attractions	Bird flocks, ungulate herds, children playing, egalitarian social groups
Webs or Networks	Parts in relationships within systems (can be centered or clustered, using clonons or holons)	Subsystems of cells, organisms, ecosystems, machines, society
Sheets	Transfer surface for matter, energy, or information	Films; fish gills, solar collectors
Borders and Pores	Protection, openings for controlled exchange	Boundaries, containers, partitions, cell membranes, national borders
Layers	Combination of other patterns that builds up order, structure, and stabilization	Levels of scale, parts and wholes, packing, proportions, tiling

Similarity	Figures of the same shape but different sizes	Similar triangles, infant-adult
Emergence	General phenomenon when a new type of functionality derives from binaries or webs.	Creation (birth), life from molecules, cognition from neurons
Holarchies	Levels of webs, in which successive systems are parts of larger systems	Biological nesting from biomolecules to ecosystems, human social nesting, engineering designs, computer software
Holons	Parts of systems as functionally unique	Heart-lungs-liver (holons) of body
Clonons	Parts of systems as interchangeable	Skin cells (clonons) of the skin; bricks in constructing a house
Arrows	Stability or gradient-like change over time	Stages, sequence, orientation, stress, growth, meanders, biological homeostasis, growth, self-maintaining social structures
Cycles	Recurrent patterns in systems over time	Alternating repetition, vortex, spiral, turbulence, helices, rotations; protein degradation and synthesis, life cycles, power cycles of electricity generating plants, feedback cycles
Breaks	Relatively sudden changes in system behavior	Transformation, change, branching, explosion, cracking, translations; cell division, insect metamorphosis, coming-of-age ceremonies, political elections, bifurcation points
Triggers	Initiating agents of breaks, both internal and external	Sperm entering egg or precipitating events of war
Gradients	Continuum of variation between binary poles	Chemical waves in cell development, human quantitative and qualitative values

Systems Engineering Patterns

Some work has been done on various aspects of explicitly applying patterns to SE. A review article of much of this work was written by Bagnulo and Addison (2010), covering patterns in general, capability engineering, pattern languages, pattern modeling, and other SE-related pattern topics. Cloutier (2005) discussed applying patterns to SE, based on architecture and software design patterns. Haskins (2005), and Simpson and Simpson (2006) discussed the use of SE pattern languages to enhance the adoption and use of SE patterns. Simpsons identified three high-level, global patterns that can be used as a means of organizing systems patterns:

- Anything can be described as a system.
- The problem system is always separate from the solution system.
- Three systems, at a minimum, are always involved in any system activity: the environmental system, the product system, and the process system.

Haskins (2008) also proposed the use of patterns as a way to facilitate the extension of SE from traditional technological systems to address social and socio-technical systems. Some patterns have been applied and identified in this extended arena, described as patterns of success by Rebovich and DeRosa (2012). Stevens (2010) also discussed patterns in the engineering of large-scale, complex “mega-systems.”

A common SE activity in which patterns are applied is in system design, especially in defining one or more solution options for a system-of-interest. See Synthesizing Possible Solutions for a discussion. The more specific topic of using patterns (and antipatterns, as described below) to understand and exploit emergence is discussed in the Emergence and Complexity topic.

Patterns of Failure: Antipatterns

System Archetypes

The system dynamics community has developed a collection of what are called system archetypes. The concept was originated by Forrester (1969), while Senge (1990) appears to have introduced the system archetype term. According to Braun (2002), the archetypes describe common patterns of behavior that help answer the question, “Why do we keep seeing the same problems recur over time?” They focus on behavior in organizations and other complex social systems that are repeatedly but unsuccessfully used to solve recurring problems. This is why they are grouped here under antipatterns, even though the system dynamics community does not refer to the archetypes as antipatterns. The table below summarizes the archetypes. There is not a fixed set, or even fixed names for a given archetype. The table shows alternative names for some archetypes.

Table 3. System Archetypes. (SEBoK Original)

Name (Alternates)	Description	Reference**
Counterintuitive Behavior	Forrester identified three “especially dangerous” counter-intuitive behaviors of social systems, which correspond respectively to three of the archetypes discussed below: (1) Low-Leverage Policies: Ineffective Actions; (2) High Leverage Policies: Often Wrongly Applied; and (3) Long-Term vs. Short-Term Trade-offs	F1, F2
Low-Leverage Policies: Ineffective Actions (Policy Resistance)	Most intuitive policy changes in a complex system have very little leverage to create change; this is because the change causes reactions in other parts of the system that counteract the new policy.	F1, F3, M
High Leverage Policies: Often Wrongly Applied (High Leverage, Wrong Direction)	A system problem is often correctable with a small change, but this high-leverage solution is typically counter-intuitive in two ways: (1) the leverage point is difficult to find because it is usually far removed in time and place from where the problem appears, and (2) if the leverage point is identified, the change is typically made in the wrong direction, thereby intensifying the problem.	F1, F3, M
Long-Term vs. Short-Term Trade-offs (Fixes that Fail, Shifting the Burden, Addiction)	Short-term solutions are intuitive, but in complex systems there is nearly always a conflict or trade-off between short-term and long-term goals. Thus, a quick fix produces immediate positive results, but its unforeseen and unintended long-term consequences worsen the problem. Furthermore, a repeated quick fix approach makes it harder to change to a more fundamental solution approach later.	F1, F3, M, S, B
Drift to Low Performance (Eroding Goals, Collapse of Goals)	There is a strong tendency for complex system goals to drift downward. A gap between current state and goal state creates pressure to lower the goal rather than taking difficult corrective action to reach the goal. Over time the continually lowered goals lead to crisis and possible collapse of the system.	F1, F3, M, B
Official Addiction – Shifting the Burden to the Intervener	The ability of a system to maintain itself deteriorates when an intervener provides help and the system then becomes dependent on the intervener.	M, S
Limits to Growth (a.k.a. Limits to Success)	A reinforcing process of accelerating growth (or expansion) will encounter a balancing process as the limit of that system is approached and continuing efforts will produce diminishing returns as one approaches the limits.	S, B
Balancing Process with Delay	Delay in the response of a system to corrective action causes the correcting agent to either over-correct or to give up due to no visible progress.	S
Escalation	Two systems compete for superiority, with each escalating its competitive actions to get ahead, to the point that both systems are harmed.	B
Success to the Successful	Growth leads to decline elsewhere. When two equally capable systems compete for a limited resource, if one system receives more resources, it is more likely to be successful, which results in its receiving even more resources, in a reinforcing loop.	S, B
Tragedy of the Commons	A shared resource is depleted as each system abuses it for individual gain, ultimately hurting all who share it.	H, S, B

Growth and Underinvestment	In a situation where capacity investments can overcome limits, if such investments are not made, S, B then growth stalls, which then rationalizes further underinvestment.	
Accidental Adversaries	Two systems destroy their relationship through escalating retaliations for perceived injuries.	B
Attractiveness Principle	In situations where a system faces multiple limiting or impeding factors, the tendency is to consider each factor separately to select which one to address first, rather than a strategy based on the interdependencies among the factors.	B

**** B—**(Braun 2002); **F1—**(Forrester 1969); **F2—**(Forrester 1995); **F3—**(Forrester 2009); **H—**(Hardin 1968); **M—**(Meadows 1982); **S—**(Senge 1990).

Relations among system archetypes were defined by Goodman and Kleiner (1993/1994) and republished in Senge et al. (1994).

Software and Other Antipatterns

Antipatterns have been identified and collected in the software community in areas that include: architecture, development, project management, user interface, organization, analysis, software design, programming, methodology, and configuration management (AntiPatterns Catalog 2012, Wikibooks 2012). A brief statement of three of them follows; the first two are organization and the third is software design.

- Escalation of commitment - Failing to revoke a decision when it proves wrong.
- Moral hazard - Insulating a decision-maker from the consequences of his or her decision.
- Big ball of mud - A system with no recognizable structure.

A link between the software community and the system archetypes is represented in a project at the Software Engineering Institute (SEI) (2012), which explores the system archetypes in the context of identifying recurring software acquisition problems as “acquisition archetypes.” They refer to both types of archetypes as patterns of failure.

Another set of antipatterns in the general systems arena has been compiled by Troncale (2010; 2011) in his systems pathologies project. Sample pathology types or patterns include:

- Cyberpathologies - Systems-level malfunctions in feedback architectures.
- Nexopathologies - Systems-level malfunctions in network architectures or dynamics.
- Heteropathologies - Systems-level malfunctions in hierarchical, modular structure & dynamics.

Some treatments of antipatterns, including Senge (1990) and SEI (2012), also provide some advice on dealing with or preventing the antipattern.

Patterns and Maturity

Patterns may be used as an indicator of the maturity of a domain of inquiry, such as systems science or systems engineering. In a mature and relatively stable domain, the problems and solutions are generally understood and their similarities are captured in a variety of what are here called patterns. A couple of observations can be made in this regard on the maturity of systems science in support of systems engineering.

In the arenas of physical systems and technical systems, systems science is relatively mature; many system patterns of both natural physical systems and engineered technical systems are reasonably well-defined and understood.

In the arena of more complex systems, including social systems, systems science is somewhat less mature. Solution patterns in that arena are more challenging. A pessimistic view of the possibility of science developing solutions to social problems was expressed by Rittel and Webber (1973) in their classic paper on wicked problems: “The search for scientific bases for confronting problems of social policy is bound to fail, because . . . they are ‘wicked’ problems, whereas science has developed to deal with ‘tame’ problems.” A more optimistic stance toward social problems has characterized the system dynamics community. They have been pointing out for over 40 years the problems with conventional solutions to social problems, in the form of the system archetypes and associated feedback loop

models. That was an important first step. Nevertheless, they have had difficulty achieving the second step; producing social patterns that can be applied to solve those problems. The antipatterns characterize problems, but the patterns for solving those problems are elusive.

Despite the difficulties, however, social systems do exhibit regularities, and social problems are often solved to some degree. The social sciences and complex systems community have limited sets of patterns, such as common types of organization structures, common macro-economic models, and even patterns of insurgency and counter-insurgency. The challenge for systems science is to capture those regularities and the salient features of those solutions more broadly and make them explicit and available in the form of mature patterns. Then perhaps social problems can be solved on a more regular basis. As systems engineering expands its scope from the traditional emphasis on technical aspects of systems to the interplay of the social and technical aspects of socio-technical systems, such progress in systems science is becoming even more important to the practice of systems engineering.

References

Works Cited

- Alexander, C. 1979. *The Timeless Way of Building*. New York, NY, USA: Oxford University Press.
- Alexander, C., S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, and S. Angel. 1977. *A Pattern Language: Towns – Buildings – Construction*. New York, NY, USA: Oxford University Press.
- ATIS. 2008. *ATIS Telecom Glossary 2007*. Washington, D.C., USA: Alliance for Telecommunications Industry Solutions. Available at: ATIS <https://www.atis.org/glossary/definition.aspx?id=3516>. Accessed December 3, 2014.
- Bagnulo, A. and T. Addison. 2010. *State of the Art Report on Patterns in Systems Engineering and Capability Engineering*. Contract Report 2010-012 by CGI Group for Defence R&D Canada – Valcartier. March 2010.
- Bloom, J. 2005. "The application of chaos, complexity, and emergent (meta)patterns to research in teacher education." *Proceedings of the 2004 Complexity Science and Educational Research Conference* (pp. 155-191), Chaffey's Locks, Canada, Sep 30–Oct 3 2004. Available at: <https://www.complexityandeducation.ca>.
- Boccaro, N. 2004. *Modeling Complex Systems*. New York, NY, USA: Springer-Verlag.
- Braun, T. 2002. "The System Archetypes." Available at: https://www.albany.edu/faculty/gpr/PAD724/724WebArticles/sys_archetypes.pdf
- Brown, W., R. Malveau, H. McCormick, and T. Mowbray. 1998. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. Hoboken, NJ, USA: John Wiley & Sons.
- Buschmann, F., R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal. 1996. *Pattern-Oriented Software Architecture: A System of Patterns*. Chichester, U.K.: John Wiley.
- Cloutier, R. 2005. "Toward the application of patterns to systems engineering." *Proceedings of the Conference on Systems Engineering Research (CSER) 2005*, Hoboken, NJ, USA, March 23-25, 2005.
- Forrester, J. 1969. *Urban Dynamics*. Waltham, MA, USA: Pegasus Communications.
- Forrester, J. 1995. "Counterintuitive behavior of social systems," *Technology Review*, vol. 73, no. 3, Jan. 1971, pp. 52-68.
- Forrester, J. 2009. *Learning through System Dynamics as Preparation for the 21st Century*.
- Gamma, E., R. Helm, R. Johnson, and J. Vlissides. 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley.
- Goodman, G. and A. Kleiner. 1993/1994. "Using the archetype family tree as a diagnostic tool," *The Systems Thinker*, December 1993/January 1994.

- Gregory, S. 1966. "Design and the design method," in S. Gregory, Ed., *The Design Method*. London, England: Butterworth.
- Hardin, G. 1968. "The Tragedy of the Commons," *Science*, vol. 162, 13 December 1968, pp. 1243-1248. DOI: 10.1126/science.162.3859.1243.
- Haskins, C. 2005. "Application of patterns and pattern languages to systems engineering." *Proceedings of the 15th Annual INCOSE International Symposium*, Rochester, NY, USA, July 10-13, 2005.
- Haskins, C. 2008. "Using patterns to transition systems engineering from a technological to social context," *Systems Engineering*, vol. 11, no. 2, May 2008, pp. 147-155.
- Hybertson, D. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boca Raton, FL, USA: Auerbach/CRC Press.
- Kapraff, J. 1991. *Connections: The Geometric Bridge between Art and Science*. New York, NY, USA: McGraw-Hill.
- Koenig, A. 1995. "Patterns and antipatterns," *Journal of Object-Oriented Programming*, vol. 8, no. 1, March/April 1995, pp. 46-48.
- Koestler, A. 1967. *The Ghost in the Machine*. New York, NY, USA: Macmillan.
- Lehmann, M. and L. Belady. 1985. *Program Evolution*. London, England: Academic Press.
- Meadows, D. 1982. "Whole Earth Models and Systems," *The Co-Evolution Quarterly*, Summer 1982, pp. 98-108.
- Odum, H. 1994. *Ecological and General Systems: An Introduction to Systems Ecology (Revised Edition)*. Boulder, CO, USA: University Press of Colorado.
- Rebovich, G. and J. DeRosa. 2012. "Patterns of success in systems engineering of IT-intensive government systems," *Procedia Computer Science*, vol. 8, 2012, pp. 303 – 308.
- Rittel, H. and M. Webber. 1973. "Dilemmas in a general theory of planning," *Policy Sciences*, vol. 4, pp. 155-169.
- Schindel, W. 2005. "Pattern-based systems engineering: An extension of model-based systems engineering," INCOSE TIES tutorial presented at 2005 INCOSE Symposium, Rochester, NY, USA, 10-15 July 2005.
- Schindel, W. and V. Smith. 2002. *Results of Applying a Families-of-Systems Approach to Systems Engineering of Product Line Families*. Technical Report 2002-01-3086. SAE International.
- SEI 2012. *Patterns of Failure: System Archetypes*. Available at: SEI <https://www.sei.cmu.edu/acquisition/research/pofsa.cfm>. Accessed December 3, 2014.
- Senge, P. 1990. *The Fifth Discipline: Discipline: The Art and Practice of the Learning Organization*. New York, NY, USA: Currency Doubleday.
- Senge, P., A. Kleiner, C. Roberts and R. Ross. 1994. *The Fifth Discipline Fieldbook: Strategies and Tools for Building a Learning Organization*. New York, NY, USA: Currency Doubleday.
- Shaw, M. and D. Garlan. 1996. *Software Architecture: Perspectives on an Emerging Discipline*. Upper Saddle River, NJ, USA: Prentice Hall.
- Simpson, J. and M. Simpson. 2006. "Foundational systems engineering patterns for a SE pattern language," *Proceedings of the 16th Annual INCOSE Symposium*, Orlando, FL, USA, July 2006.
- Stevens, R. 2011. *Engineering Mega-Systems: The Challenge of Systems Engineering in the Information Age*. Boca Raton, FL, USA: Auerbach/Taylor & Francis.
- Troncale, L. 2010. "Would a rigorous knowledge base in "systems pathology" add to the S.E. portfolio?" Presented at 2010 LA Mini-Conference, Loyola Marymount University, Los Angeles, CA, 16 October 2010.
- Troncale, L. 2011. "Would a rigorous knowledge base in systems pathology add significantly to the SE portfolio?" *Proceedings of the Conference on Systems Engineering Research (CSER)*, Redondo Beach, CA, April 14-16.
-

Volk, T., and J.W. Bloom. 2007. "The use of metapatterns for research into complex systems of teaching, learning, and schooling. Part I: Metapatterns in nature and culture," *Complicity: An International Journal of Complexity and Education*, vol. 4, no. 1, pp. 25—43.

Wikibooks. 2012a. "AntiPatterns." Available at: https://en.wikibooks.org/wiki/Introduction_to_Software_Engineering/Architecture/Anti-Patterns.

Wikipedia. 2012b. "Software Design Pattern." Available at: https://en.wikipedia.org/wiki/Software_design_pattern.

Primary References

Alexander, C. 1979. *The Timeless Way of Building*. New York, NY, USA: Oxford University Press.

Bertalanffy, L. von. 1968. *General System Theory: Foundations, Development, Applications*. Revised ed. New York, NY, USA: Braziller.

Bloom, J. 2005. "The application of chaos, complexity, and emergent (meta)patterns to research in teacher education." *Proceedings of the 2004 Complexity Science and Educational Research Conference* (pp. 155-191), Chaffey's Locks, Canada, Sep 30–Oct 3, 2004. Available at: <https://www.complexityandeducation.ca>.

Hybertson, D. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boca Raton, FL, USA: Auerbach/CRC Press.

Additional References

Principia Cybernetica. 1996. *Cybernetics and Systems Theory*. Available at: <https://pespmc1.vub.ac.be/CYBSYSTH.html>. Accessed 21 April 2013.

Erl, T. 2009. *SOA: Design Patterns*. Upper Saddle River, NJ, USA: Prentice Hall.

Erl, T. 2008. *SOA: Principles of Service Design*. Upper Saddle River, NJ, USA: Prentice Hall.

Francois, F., Ed.. 2004. *International Encyclopedia of Systems and Cybernetics*, 2nd ed. Munich, Germany: K. G. Saur Verlag.

Meyers, R. Ed. 2009. *Encyclopedia of Complexity and Systems Science*. New York, NY, USA: Springer.

Midgley, G. Ed. 2003. *Systems Thinking*. Thousand Oaks, CA, USA: Sage Publications Ltd.

Principia Cybernetica Web. 2013. "Web Dictionary of Cybernetics and Systems." Available at: <https://pespmc1.vub.ac.be/ASC/indexASC.html>. Accessed 21 April 2013.

Knowledge Area: Representing Systems with Models

Representing Systems with Models

Contents of this Knowledge Area

- What is a Model? (Sanford Friedenthal) (Dov Dori and Yaniv Mordecai)
- Why Model? (Sanford Friedenthal) (Dov Dori and Yaniv Mordecai)
- Types of Models (Sanford Friedenthal) (Dov Dori and Yaniv Mordecai)
- System Modeling Concepts (Sanford Friedenthal) (Dov Dori, Yaniv Mordecai)
- Integrating Supporting Aspects into System Models (Sanford Friedenthal) (Dov Dori and Yaniv Mordecai)
- Modeling Standards (Sanford Friedenthal) (Dov Dori and Yaniv Mordecai)
- Lead Author:
- Sanford Friedenthal
- Contributing Authors:
- Dov Dori and Yaniv Mordecai

A model is a simplified representation of a system at some particular point in time or space intended to promote understanding of the real system. As an abstraction of a system, it offers insight about one or more of the system's aspects, such as its function, structure, properties, performance, behavior, or cost.

Overview

The modeling of systems as holistic, value-providing entities has been gaining recognition as a central process of systems engineering. The use of modeling and simulation during the early stages of the system design of complex systems and architectures can:

- document system functions and requirements,
- assess the mission performance,
- estimate costs,
- evaluate trade-offs, and
- provide insights to improve performance, reduce risk, and manage costs.

Modeling and analysis can complement testing and evaluations which occur later in the life cycle. In some systems, modeling and simulation may be the only way to fully evaluate performance (e.g., ballistic missile defense) or to evaluate system performance in severe scenarios (e.g., response to weapons of mass destruction attacks on the homeland). Furthermore, advanced simulations, e.g. flight simulators, and command and control center simulations, can be a cost-effective technique for personnel training in accompaniment with operational system training (INCOSE 2012).

Modeling serves to make concepts concrete and formal, enhance quality, productivity, documentation, and innovation, as well as to reduce the cost and risk of systems development.

Modeling occurs at many levels: component, subsystem, system, and systems-of-systems; and throughout the life cycle of a system. Different types of models may be needed to represent systems in support of the analysis, specification, design, and verification of systems. This knowledge area provides an overview of models used to represent different aspects of systems.

Modeling is a common practice that is shared by most engineering disciplines, including:

- electrical engineering, which uses electrical circuit design models
- mechanical engineering, which uses three-dimensional computer-aided design models
- software engineering, which uses software design and architecture models.

Each of these disciplines has its own language with its syntax and semantics, serving as a means of communication among professionals in that discipline. Analytic models are used to support power, thermal, structural, and embedded real-time analysis.

Modeling Standards play an important role in defining system modeling concepts that can be represented for a particular domain of interest and enable the integration of different types of models across domains of interest.

References

Works Cited

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Primary References

Dori, D. 2002. *Object-Process Methodology – A Holistic Systems Paradigm*. Berlin, Germany: Springer Verlag.

Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, rev. B. Seattle, WA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02. Available at: https://www.omgsysml.org/MBSE_Methodology_Survey_RevB.pdf. Accessed April 13, 2015.

Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.

Guizzardi, G. 2007. "On ontology, ontologies, conceptualizations, modeling languages, and (meta)models," *Proceedings of the Databases and Information Systems IV Conference*, Amsterdam, Netherlands. Available at: [ACM https://portal.acm.org/citation.cfm?id=1565425](https://portal.acm.org/citation.cfm?id=1565425). Accessed December 4 2014.

INCOSE. 2007. *Systems Engineering Vision 2020*. Seattle, WA, USA: International Council on Systems Engineering. September 2007. INCOSE-TP-2004-004-02.

Wymore, A.W. 1993. *Model-Based Systems Engineering*. Boca Raton, FL, USA: CRC Press, Inc.

Additional References

Holt, J. and S. Perry. 2008. *SysML for Systems Engineering*. Stevenage, UK: Institution of Engineering and Technology. Available at: [Ebrary https://site.ebrary.com/id/10263845](https://site.ebrary.com/id/10263845). Accessed December 4 2014.

Grobshtein, Y. and D. Dori. 2011. "Generating SysML views from an OPM model: Design and evaluation," *Systems Engineering*, vol. 14, no. 3, Sept.

West, P., J. Kobza, and S. Goerger. 2011. "Chapter 4, systems modeling and analysis," in Parnell, G.S., P.J. Driscoll, and D.L. Henderson Eds., *Decision Making for Systems Engineering and Management*, 2nd ed. Wiley Series in Systems Engineering. Hoboken, NJ, USA: Wiley & Sons Inc.

What is a Model?

- Lead Author:
 - Sanford Friedenthal
- Contributing Authors:
 - Dov Dori and Yaniv Mordecai

This topic provides foundational concepts, such as definitions of a model and a modeling language, and expresses their relationships to modeling tools and model-based systems engineering (MBSE).

Definition of a Model

There are many definitions of the word *model*. The following definitions refer to a model as a representation of selected aspects of a domain of interest to the modeler:

- a physical, mathematical, or otherwise logical representation of a system, entity, phenomenon, or process (DoD 1998);
- a representation of one or more concepts that may be realized in the physical world (Friedenthal, Moore, and Steiner 2009);
- a simplified representation of a system at some particular point in time or space intended to promote understanding of the real system (Bellinger 2004);
- an abstraction of a system, aimed at understanding, communicating, explaining, or designing aspects of interest of that system (Dori 2002); and
- a selective representation of some system whose form and content are chosen based on a specific set of concerns; the model is related to the system by an explicit or implicit mapping (Object Management Group 2010).

In the context of systems engineering, a model that represents a system and its environment is of particular importance to the system engineer who must specify, design, analyze, and verify systems, as well as share information with other stakeholders. A variety of system models are used to represent different engineered systems for different modeling purposes. Some of the purposes for modeling systems are summarized in the topic Why Model?, and a simple taxonomy of the different types of models are described in the topic Types of Models. The modeling standards topic refers to some of the standard system modeling languages and other modeling standards that support MBSE.

A model can have different forms as indicated in the first definition above, including a physical, mathematical, or logical representation. A physical model can be a mockup that represents an actual system, such as a model airplane. A mathematical model may represent possible flight trajectories in terms of acceleration, speed, position, and orientation. A logical model may represent logical relationships that describe potential causes of airplane failure, such as how an engine failure can result in a loss of power and cause the airplane to lose altitude, or how the parts of the system are interconnected. It is apparent that many different models may be required to represent a system-of-interest (SoI).

Modeling Language

A physical model is a concrete representation of an actual system that can be felt and touched. Other models may be more abstract representations of a system or entity. These models rely on a modeling language to express their meaning as explained in “On Ontology, Ontologies, Conceptualizations, Modeling Languages, and (Meta)Models” (Guizzardi 2007).

Just as engineering drawings express the 3D structure of mechanical and architectural designs, conceptual models are the means by which systems are conceived, architected, designed, and built. The resulting models are the counterparts of the mechanical design blueprint. However, the difference is that, while blueprints are exact representations of physical artifacts with a precise, agreed-upon syntax and long tradition of serving as a means of communication among professionals, conceptual models are just beginning to make headway toward being a complete and unambiguous representation of a system under development. The articles in the special section of Communications of the Association for Computing Machinery (ACM) (Dori 2003) present the abstract world of systems analysis and architecting by means of conceptual modeling, and how to evaluate, select, and construct models.

Modeling languages are generally intended to be both human-interpretable and computer-interpretable, and are specified in terms of both syntax and semantics.

The abstract syntax specifies the model constructs and the rules for constructing the model. In the case of a natural language like English, the constructs may include types of words such as verbs, nouns, adjectives, and prepositions, and the rules specify how these words can be used together to form proper sentences. The abstract syntax for a mathematical model may specify constructs to define mathematical functions, variables, and their relationships. The abstract syntax for a logical model may also specify constructs to define logical entities and their relationships. A well-formed model abides by the rules of construction, just as a well-formed sentence must conform to the grammatical rules of the natural language.

The concrete syntax specifies the symbols used to express the model constructs. The natural language English can be expressed in text or Morse code. A modeling language may be expressed using graphical symbols and/or text statements. For example, a functional flow model may be expressed using graphical symbols consisting of a combination of graphical nodes and arcs annotated with text, while a simulation modeling language may be expressed using a programming language text syntax such as the C programming language.

The semantics of a language define the meaning of the constructs. For example, an English word does not have explicit meaning until the word is defined. Similarly, a construct that is expressed as a symbol, such as a box or arrow on a flow chart, does not have meaning until it is defined. The language must give meaning to the concept of a verb or noun, and must give specific meaning to a specific word that is a verb or noun. The definition can be established by providing a natural language definition or by mapping the construct to a formalism whose meaning is defined. As an example, a graphical symbol that expresses $\sin(x)$ and $\cos(x)$ is defined using a well-defined mathematical formalism for the sine and cosine function. If the position of a pendulum is defined in terms of $\sin(\theta)$ and $\cos(\theta)$, the meaning of the pendulum position is understood in terms of these formalisms.

Modeling Tools

Models are created by a modeler using modeling tools. For physical models, the modeling tools may include drills, lathes, and hammers. For more abstract models, the modeling tools are typically software programs running on a computer. These programs provide the ability to express modeling constructs using a particular modeling language. A word processor can be viewed as a tool used to build text descriptions using natural language. In a similar way, modeling tools are used to build models using modeling languages. The tool often provides a tool palette to select symbols and a content area to construct the model from the graphical symbols or other concrete syntax. A modeling tool typically checks the model to evaluate whether it conforms to the rules of the language and enforces such rules

to help the modeler create a well-formed model. This is similar to the way a word processor checks the text to see that it conforms to the grammar rules for the natural language.

Some modeling tools are commercially available products, while others may be created or customized to provide unique modeling solutions. Modeling tools are often used as part of a broader set of engineering tools which constitute the systems development environment. There is increased emphasis on tool support for standard modeling languages that enable models and modeling information to be interchanged among different tools.

Relationship of Model to Model-Based Systems Engineering

The International Council of Systems Engineering (INCOSE) (INCOSE Systems Engineering Vision 2020 2007) defines MBSE as “the formalized application of modeling to support system requirements, design, analysis, verification, and validation activities beginning in the conceptual design phase and continuing throughout development and later life cycle phases.” In MBSE, the models of the system are primary artifacts of the systems engineering process, and are managed, controlled, and integrated with other parts of the system technical baseline. This contrasts with the traditional document-centric approach to systems engineering, where text-based documentation and specifications are managed and controlled. Leveraging a model-based approach to systems engineering is intended to result in significant improvements in system specification and design quality, lower risk and cost of system development by surfacing issues early in the design process, enhanced productivity through reuse of system artifacts, and improved communications among the system development and implementation teams.

In addition to creating models, the MBSE approach typically includes methods for model management, which aim to ensure that models are properly controlled, and methods for model validation, which aim to ensure that models accurately represent the systems being modeled.

The jointly sponsored INCOSE/Object Management Group (OMG) MBSE Wiki ^[1] provides additional information on the INCOSE MBSE Initiative, including some applications of MBSE and some key topics related to MBSE such as sections on Methodology and Metrics, and Model Management.

The Final Report of the Model Based Engineering (MBE) Subcommittee, which was generated by the National Defense Industrial Association (NDIA) Modeling and Simulation Committee of the Systems Engineering Division, highlights many of the benefits, risks, and challenges of a model-based approach, and includes many references to case studies of MBE (NDIA 2011).

Brief History of System Modeling Languages and Methods

Many system modeling methods and associated modeling languages have been developed and deployed to support various aspects of system analysis, design, and implementation. Functional modeling languages include the data flow diagram (DFD) (Yourdon and Constantine 1979), Integration Definition for Functional Modeling (IDEF0) (Menzel and Maier 1998), and enhanced functional flow block diagram (eFFBD). Other behavioral modeling techniques include the classical state transition diagram, statecharts (Harel 1987), and process flow diagrams. Structural modeling techniques include data structure diagrams (Jackson 1975), entity relationship diagrams (Chen 1976), and object modeling techniques (Rumbaugh et al. 1991), which combine object diagrams, DFDs, and statecharts.

In 2008, Estefan conducted an extensive survey of system modeling methods, processes, and tools and documented the results in *A Survey of Model-Based Systems Engineering (MBSE) Methodologies* (Estefan 2008). This survey identifies several candidate MBSE methodologies and modeling languages that can be applied to support an MBSE approach. Additional modeling methods are available from the MBSE Wiki ^[1] under the section on Methodology and Metrics ^[2]. The modeling standards section refers to some of the standard system modeling languages and other modeling standards that support MBSE. Since Estefan's report, a number of surveys have been conducted to understand the acceptance and barriers to model-based systems engineering (Bone and Cloutier 2010, 2014; Cloutier 2015).

References

Works Cited

- Bellinger, G. 2004. "Modeling & simulation: An introduction," in *Mental Model Musings*. Available at: <https://www.systems-thinking.org/modsim/modsim.htm>.
- Bone, M. & Cloutier, R. 2010. "The current state of model based systems engineering: Results from the OMG™ SysML request for information 2009." 8h Conference on Systems Engineering Research, Hoboken, NJ, USA, March 17-19, 2010, Paper #1569270919. Available at https://www.omg-sysml.org/SysML_2009_RFI_Response_Summary-bone-cloutier.pdf. Accessed May 26, 2019.
- Cloutier, R. & Bone, M. 2014. "MBSE survey," Presented January 2015 INCOSE IW, Los Angeles, CA. Available at: https://www.omgwiki.org/MBSE/lib/exe/fetch.php?media=mbse:incose_mbse_survey_results_initial_report_2015_01_24.pdf. Accessed May 26, 2019.
- Cloutier, R. 2015. "Current modeling trends in systems engineering," *INCOSE Insight*, vol. 18, no. 2.
- Chen, P. 1976. "The entity relationship model – Toward a unifying view of data," *ACM Transactions on Data Base Systems*, vol. 1, no. 1, pp. 9-36.
- DoD. 1998. "DoD modeling and simulation (M&S) glossary," in *DoD Manual 5000.59-M*. Arlington, VA, USA: US Department of Defense. January. P2.13.22. Available at <https://www.dtic.mil/whs/directives/corres/pdf/500059m.pdf>.
- Dori, D. 2002. *Object-Process Methodology: A Holistic System Paradigm*. New York, NY, USA: Springer.
- Dori, D. 2003. "Conceptual modeling and system architecting," *Communications of the ACM*, vol. 46, no. 10, pp. 62-65. Available at: <https://esml.iem.technion.ac.il/site/wp-content/uploads/2011/02/article169.pdf>.
- Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, rev. B, Seattle, WA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02.
- Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.
- Guizzardi, G. 2007. "On ontology, ontologies, conceptualizations, modeling languages, and (meta)models," Proceedings of Seventh International Baltic Conference, Amsterdam, The Netherlands. Available at: <https://portal.acm.org/citation.cfm?id=1565425>.
- Harel, D. 1987. "Statecharts: A visual formalism for complex systems," *Science of Computer Programming*, vol. 8, no. 3, pp. 231–74.
- Jackson, M.A. 1975. *Principles of Program Design*. New York, NY, USA: Academic Press.
- Menzel, C. and R.J. Mayer. 1998. "The IDEF family of languages" in P. Bernus, K. Mertins, and G. Schmidt, Eds. *Handbook on Architectures for Information Systems*. Berlin, Germany: Springer-Verlag, pp. 209-241.
- OMG. 2010. *MDA Foundation Model*. Needham, MA, USA: Object Management Group. ORMSC/2010-09-06.
- Rumbaugh, J., M. Blaha, W. Premerlani, F. Eddy, and W. Lorenson. 1990. *Object-Oriented Modeling and Design*. Upper Saddle River, NJ, USA: Prentice Hall.
- Press, Y. and L.L. Constantine. 1976. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*. Upper Saddle River, NJ, USA: Prentice Hall.
- Yourdon E. and Constantine L.L. 1973. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*. Upper Saddle River, NJ, USA: Prentice-Hall, Inc. 1st Edition.

Primary References

- Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, Rev. B. San Diego, CA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02. Available at: https://www.incose.org/ProductsPubs/pdf/techdata/MTTC/MBSE_Methodology_Survey_2008-0610_RevB-JAE2.pdf.
- Guizzardi, G. 2007. "On ontology, ontologies, conceptualizations, modeling languages, and (meta)models," Proceedings of Seventh International Baltic Conference, Amsterdam, The Netherlands. Available at: <https://portal.acm.org/citation.cfm?id=1565425>.
- INCOSE. 2007. *Systems Engineering Vision 2020*. Seattle, WA, USA: International Council on Systems Engineering. September 2007. INCOSE-TP-2004-004-02.
- NDIA. 2011. *Final Report of the Model Based Engineering (MBE) Subcommittee*. Arlington, VA, USA: National Defense Industrial Association. Available at: [https://www.ndia.org/Divisions/Divisions/SystemsEngineering/Documents/Committees/M_S%20Committee/Reports/MBE_Final_Report_Document_\(2011-04-22\)_Marked_Final_Draft.pdf](https://www.ndia.org/Divisions/Divisions/SystemsEngineering/Documents/Committees/M_S%20Committee/Reports/MBE_Final_Report_Document_(2011-04-22)_Marked_Final_Draft.pdf)

Additional References

- Downs, E., P. Clare, and I. Coe. 1992. *Structured Systems Analysis and Design Method: Application and Context*. Hertfordshire, UK: Prentice-Hall International.
- Eisner, H. 1988. *Computer-Aided Systems Engineering*. Englewood Cliffs, NJ, USA: Prentice Hall.
- Harel, D. 1987. "Statecharts: A visual formalism for complex systems," *Science of Computer Programming*, vol. 8, no. 3, pp. 231–74.
- Kossiakoff, A. and W. Sweet. 2003. "Chapter 14," in *Systems Engineering Principles and Practice*. New York, NY, USA: Wiley and Sons.
- OMG. "MBSE Wiki." Object Management Group (OMG). Available at: <https://www.omgwiki.org/MBSE/doku.php>. Accessed 11 September 2011.
- Oliver, D., T. Kelliber, and J. Keegan. 1997. *Engineering Complex Systems with Models and Objects*. New York, NY, USA: McGraw-Hill.

References

- [1] <https://www.omgwiki.org/MBSE/doku.php>
- [2] <https://www.omgwiki.org/MBSE/doku.php?id=mbse:methodology>

Why Model?

- Lead Author:
- Sanford Friedenthal
- Contributing Authors:
- Dov Dori and Yaniv Mordecai

System models can be used for many purposes. This topic highlights some of those purposes, and provides indicators of an effective model in the context of model-based systems engineering (MBSE).

Purpose of a Model

Models are representations that can aid in defining, analyzing, and communicating a set of concepts. System models are specifically developed to support analysis, specification, design, verification, and validation of a system, as well as to communicate certain information. One of the first principles of modeling is to clearly define the purpose of the model. Some of the purposes that models can serve throughout the system life cycle are:

- **Characterizing an existing system:** Many existing systems are poorly documented and modeling the system can provide a concise way to capture the existing system design. This information can then be used to facilitate system maintenance or to assess the system with the goal of improving it. This is analogous to creating an architectural model of an old building with overlays for electrical, plumbing, and structure before proceeding to upgrade it to new standards to withstand earthquakes.
 - **Mission and system concept formulation and evaluation:** Models can be applied early in the system life cycle to synthesize and evaluate alternative mission and system concepts. This includes clearly and unambiguously defining the system's mission and the value it is expected to deliver to its beneficiaries. Models can be used to explore a trade-space by modeling alternative system designs and assessing the impact of critical system parameters such as weight, speed, accuracy, reliability, and cost on the overall measures of merit. In addition to bounding the system design parameters, models can also be used to validate that the system requirements meet stakeholder needs before proceeding with later life cycle activities such as synthesizing the detailed system design.
 - **System design synthesis and requirements flowdown:** Models can be used to support architecting system solutions, as well as flow mission and system requirements down to system components. Different models may be required to address different aspects of the system design and respond to the broad range of system requirements. This may include models that specify functional, interface, performance, and physical requirements, as well as other non-functional requirements such as reliability, maintainability, safety, and security.
 - **Support for system integration and verification:** Models can be used to support integration of the hardware and software components into a system, as well as to support verification that the system satisfies its requirements. This often involves integrating lower level hardware and software design models with system-level design models which verify that system requirements are satisfied. System integration and verification may also include replacing selected hardware and design models with actual hardware and software products in order to incrementally verify that system requirements are satisfied. This is referred to as hardware-in-the-loop testing and software-in-the-loop testing. Models can also be used to define the test cases (glossary) and other aspects of the test program to assist in test planning and execution.
 - **Support for training:** Models can be used to simulate various aspects of the system to help train users to interact with the system. Users may be operators, maintainers, or other stakeholders. Models may be a basis for developing a simulator of the system with varying degrees of fidelity to represent user interaction in different usage scenarios.
 - **Knowledge capture and system design evolution:** Models can provide an effective means for capturing knowledge about the system and retaining it as part of organizational knowledge. This knowledge, which can be
-

reused and evolved, provides a basis for supporting the evolution of the system, such as changing system requirements in the face of emerging, relevant technologies, new applications, and new customers. Models can also enable the capture of families of products.

Indicators of an Effective Model

When modeling is done well, a model's purposes are clear and well-defined. The value of a model can be assessed in terms of how effectively it supports those purposes. The remainder of this section and the topics Types of Models, System Modeling Concepts, and Modeling Standards describe indicators of an effective model (Friedenthal, Moore, and Steiner 2012).

Model Scope

The model must be scoped to address its intended purpose. In particular, the types of models and associated modeling languages selected must support the specific needs to be met. For example, suppose models are constructed to support an aircraft's development. A system architecture model may describe the interconnection among the airplane parts, a trajectory analysis model may analyze the airplane trajectory, and a fault tree analysis model may assess potential causes of airplane failure.

For each type of model, the appropriate breadth, depth, and fidelity should be determined to address the model's intended purpose. The model breadth reflects the system requirements coverage in terms of the degree to which the model must address the functional, interface, performance, and physical requirements, as well as other non-functional requirements, such as reliability, maintainability, and safety. For an airplane functional model, the model breadth may be required to address some or all of the functional requirements to power up, take off, fly, land, power down, and maintain the aircraft's environment.

The model's depth indicates the coverage of system decomposition from the system context down to the system components. For the airplane example, a model's scope may require it to define the system context, ranging from the aircraft, the control tower, and the physical environment, down to the navigation subsystem and its components, such as the inertial measurement unit; and perhaps down to lower-level parts of the inertial measurement unit.

The model's fidelity indicates the level of detail the model must represent for any given part of the model. For example, a model that specifies the system interfaces may be fairly abstract and represent only the logical information content, such as aircraft status data; or it may be much more detailed to support higher fidelity information, such as the encoding of a message in terms of bits, bytes, and signal characteristics. Fidelity can also refer to the precision of a computational model, such as the time step required for a simulation.

Indicators of Model Quality

The quality of a model should not be confused with the quality of the design that the model represents. For example, one may have a high-quality, computer-aided design model of a chair that accurately represents the design of the chair, yet the design itself may be flawed such that when one sits in the chair, it falls apart. A high quality model should provide a representation sufficient to assist the design team in assessing the quality of the design and uncovering design issues.

Model quality is often assessed in terms of the adherence of the model to modeling guidelines and the degree to which the model addresses its intended purpose. Typical examples of modeling guidelines include naming conventions, application of appropriate model annotations, proper use of modeling constructs, and applying model reuse considerations. Specific guidelines are different for different types of models. For example, the guidelines for developing a geometric model using a computer-aided design tool may include conventions for defining coordinate systems, dimensioning, and tolerances.

Model-Based Metrics

Models can provide a wealth of information that can be used for both technical and management metrics to assess the modeling effort, and, in some cases, the overall systems engineering (SE) effort. Different types of models provide different types of information. In general, models provide information that enables one to:

- assess progress;
- estimate effort and cost;
- assess technical quality and risk; and
- assess model quality.

Models can capture metrics similar to those captured in a traditional document-based approach to systems engineering, but potentially with more precision given the more accurate nature of models compared to documents. Traditional systems engineering metrics are described in *Metrics Guidebook for Integrated Systems and Product Development* (Wilbur 2005).

A model's progress can be assessed in terms of the completeness of the modeling effort relative to the defined scope of the model. Models may also be used to assess progress in terms of the extent to which the requirements have been satisfied by the design or verified through testing. When augmented with productivity metrics, the model can be used to estimate the cost of performing the required systems engineering effort to deliver the system.

Models can be used to identify critical system parameters and assess technical risks in terms of any uncertainty that lies in those parameters. The models can also be used to provide additional metrics that are associated with its purpose. For example, when the model's purpose is to support mission and system concept formulation and evaluation, then a key metric may be the number of alternative concepts that are explored over a specified period of time.

References

Works Cited

- Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.
- Wilbur, A., G. Towers, T. Sherman, D. Yasukawa, and S. Shreve. 2005. *Metrics Guidebook for Integrated Systems and Product Development*. Seattle, WA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-1995-002-01.

Primary References

- Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.
- Wilbur, A., G. Towers, T. Sherman, D. Yasukawa, and S. Shreve. 2005. *Metrics Guidebook for Integrated Systems and Product Development*. Seattle, WA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-1995-002-01. Accessed April 13 at <https://www.incose.org/ProductsPublications/techpublications/GuideMetrics>

Additional References

None.

Types of Models

- Lead Author:
- Sanford Friedenthal
- Contributing Authors:
- Dov Dori and Yaniv Mordecai

There are many different types of models expressed in a diverse array of modeling languages and tool sets. This article offers a taxonomy of model types and highlights how different models must work together to support broader engineering efforts.

Model Classification

There are many different types of models and associated modeling languages to address different aspects of a system and different engineered systems. Since different models serve different purposes, a classification of models can be useful for selecting the right type of model for the intended purpose and scope.

Formal versus Informal Models

Since a system model is a representation of a system, many different expressions that vary in degrees of formalism could be considered models. In particular, one could draw a picture of a system and consider it a model. Similarly, one could write a description of a system in text and refer to that as a model. Both examples are representations of a system. However, unless there is some agreement on the meaning of the terms, there is a potential lack of precision and the possibility of ambiguity in the representation.

The primary focus of system modeling is to use models supported by a well-defined modeling language. While less formal representations can be useful, a model must meet certain expectations for it to be considered within the scope of model-based systems engineering (MBSE). In particular, the initial classification distinguishes between informal and formal models as supported by a modeling language with a defined syntax and the semantics for the relevant domain of interest.

Physical Models versus Abstract Models

The United States “Department of Defense Modeling and Simulation (M&S) Glossary” asserts that “a model can be [a] physical, mathematical, or otherwise logical representation of a system” (1998). This definition provides a starting point for a high-level model classification. A physical model is a concrete representation that is distinguished from the mathematical and logical models, both of which are more abstract representations of the system. The abstract model can be further classified as descriptive (similar to logical) or analytical (similar to mathematical). Some example models are shown in Figure 1.

Model-Based Systems Engineering

What Kinds of System Models?

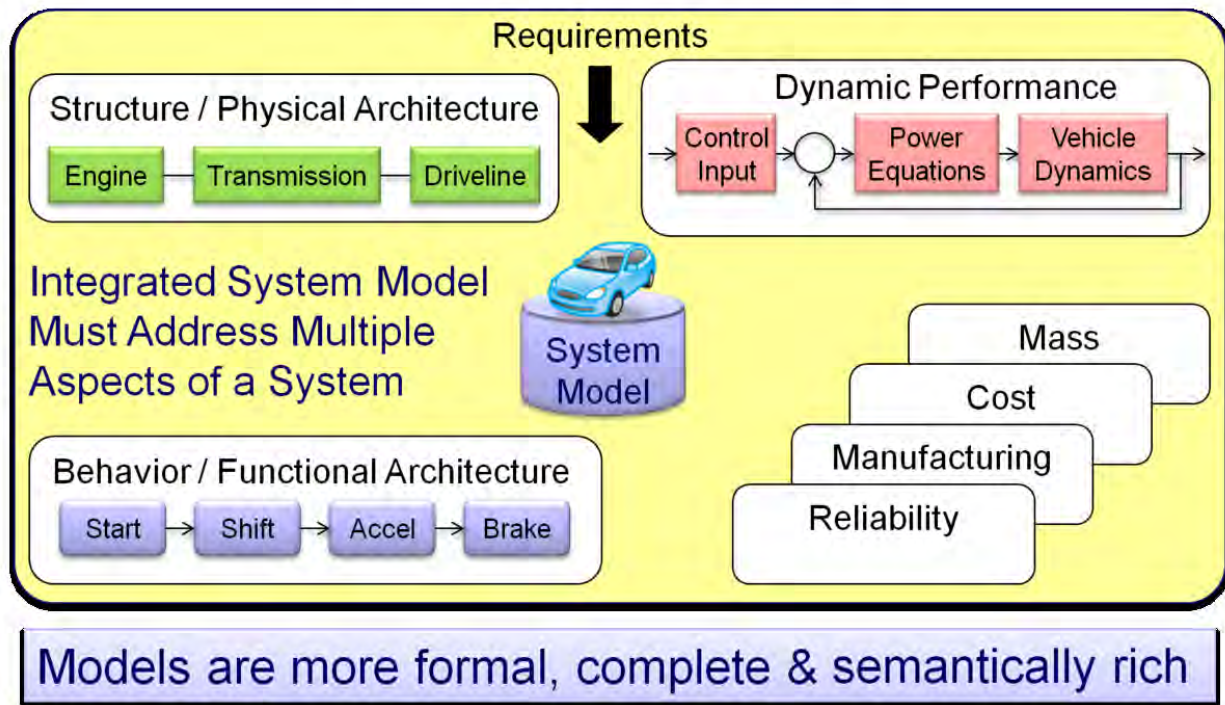


Figure 1. Model-Based Systems Engineering (Paredis 2011). Reprinted with permission of Chris Paredis from Georgia Tech. All other rights are reserved by the copyright owner.

Descriptive Models

A descriptive model describes logical relationships, such as the system's whole-part relationship that defines its parts tree, the interconnection between its parts, the functions that its components perform, or the test cases that are used to verify the system requirements. Typical descriptive models may include those that describe the functional or physical architecture of a system, or the three-dimensional geometric representation of a system.

Analytical Models

An analytical model describes mathematical relationships, such as differential equations that support quantifiable analysis about the system parameters. Analytical models can be further classified into dynamic and static models. Dynamic models describe the time-varying state of a system, whereas static models perform computations that do not represent the time-varying state of a system. A dynamic model may represent the performance of a system, such as the aircraft position, velocity, acceleration, and fuel consumption over time. A static model may represent the mass properties estimate or reliability prediction of a system or component.

Hybrid Descriptive and Analytical Models

A particular model may include descriptive and analytical aspects as described above, but models may favor one aspect or the other. The logical relationships of a descriptive model can also be analyzed, and inferences can be made to reason about the system. Nevertheless, logical analysis provides different insights than a quantitative analysis of system parameters.

Domain-Specific Models

Both descriptive and analytical models can be further classified according to the domain that they represent. The following classifications are partially derived from the presentation on *OWL, Ontologies and SysML Profiles: Knowledge Representation and Modeling* (Web Ontology Language (OWL) & Systems Modeling Language (SysML)) (Jenkins 2010):

- properties of the system, such as performance, reliability, mass properties, power, structural, or thermal models;
- design and technology implementations, such as electrical, mechanical, and software design models;
- subsystems and products, such as communications, fault management, or power distribution models; and
- system applications, such as information systems, automotive systems, aerospace systems, or medical device models.

The model classification, terminology and approach are often adapted to a particular application domain. For example, when modeling an organization or business, the behavioral model may be referred to as a workflow or process model, and the performance modeling may refer to the cost and schedule performance associated with the organization or business process.

A single model may include multiple domain categories from the above list. For example, a reliability, thermal, and/or power model may be defined for an electrical design of a communications subsystem for an aerospace system, such as an aircraft or satellite.

System Models

System models can be hybrid models that are both descriptive and analytical. They often span several modeling domains that must be integrated to ensure a consistent and cohesive system representation. As such, the system model must provide both general-purpose system constructs and domain-specific constructs that are shared across modeling domains. A system model may comprise multiple views to support planning, requirements, design, analysis, and verification.

Wayne Wymore is credited with one of the early efforts to formally define a system model using a mathematical framework in *A Mathematical Theory of Systems Engineering: The Elements* (Wymore 1967). Wymore established a rigorous mathematical framework for designing systems in a model-based context. A summary of his work can be found in *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*.

Simulation versus Model

The term simulation, or more specifically computer simulation, refers to a method for implementing a model over time (DoD 1998). The computer simulation includes the analytical model which is represented in executable code, the input conditions and other input data, and the computing infrastructure. The computing infrastructure includes the computational engine needed to execute the model, as well as input and output devices. The great variety of approaches to computer simulation is apparent from the choices that the designer of a computer simulation must make, which include:

- stochastic or deterministic;
 - steady-state or dynamic;
 - continuous or discrete; and
-

- local or distributed.

Other classifications of a simulation may depend on the type of model that is being simulated. One example is an agent-based simulation that simulates the interaction among autonomous agents to predict complex emergent behavior (Barry 2009). There are many other types of models that could be used to further classify simulations. In general, simulations provide a means for analyzing complex dynamic behavior of systems, software, hardware, people, and physical phenomena.

Simulations are often integrated with the actual hardware, software, and operators of the system to evaluate how actual components and users of the system perform in a simulated environment. Within the United States defense community, it is common to refer to simulations as live, virtual, or constructive, where live simulation refers to live operators operating real systems, virtual simulation refers to live operators operating simulated systems, and constructive simulations refers to simulated operators operating with simulated systems. The virtual and constructive simulations may also include actual system hardware and software in the loop as well as stimulus from a real systems environment.

In addition to representing the system and its environment, the simulation must provide efficient computational methods for solving the equations. Simulations may be required to operate in real time, particularly if there is an operator in the loop. Other simulations may be required to operate much faster than real time and perform thousands of simulation runs to provide statistically valid simulation results. Several computational and other simulation methods are described in *Simulation Modeling and Analysis* (Law 2007).

Visualization

Computer simulation results and other analytical results often need to be processed so they can be presented to the users in a meaningful way. Visualization techniques and tools are used to display the results in various visual forms, such as a simple plot of the state of the system versus time to display a parametric relationship. Another example of this occurs when the input and output values from several simulation executions are displayed on a response surface showing the sensitivity of the output to the input. Additional statistical analysis of the results may be performed to provide probability distributions for selected parameter values. Animation is often used to provide a virtual representation of the system and its dynamic behavior. For example, animation can display an aircraft's three-dimensional position and orientation as a function of time, as well as project the aircraft's path on the surface of the Earth as represented by detailed terrain maps.

Integration of Models

Many different types of models may be developed as artifacts of a MBSE effort. Many other domain-specific models are created for component design and analysis. The different descriptive and analytical models must be integrated in order to fully realize the benefits of a model-based approach. The role of MBSE as the models integrate across multiple domains is a primary theme in the International Council on Systems Engineering (INCOSE) INCOSE Systems Engineering Vision 2020 (INCOSE 2007).

As an example, system models can be used to specify the components of the system. The descriptive model of the system architecture may be used to identify and partition the components of the system and define their interconnection or other relationships. Analytical models for performance, physical, and other quality characteristics, such as reliability, may be employed to determine the required values for specific component properties to satisfy the system requirements. An executable system model that represents the interaction of the system components may be used to validate that the component requirements can satisfy the system behavioral requirements. The descriptive, analytical, and executable system models each represent different facets of the same system.

The component designs must satisfy the component requirements that are specified by the system models. As a result, the component design and analysis models must have some level of integration to ensure that the design

model is traceable to the requirements model. The different design disciplines for electrical, mechanical, and software each create their own models representing different facets of the same system. It is evident that the different models must be sufficiently integrated to ensure a cohesive system solution.

To support the integration, the models must establish semantic interoperability to ensure that a construct in one model has the same meaning as a corresponding construct in another model. This information must also be exchanged between modeling tools.

One approach to semantic interoperability is to use model transformations between different models. Transformations are defined which establish correspondence between the concepts in one model and the concepts in another. In addition to establishing correspondence, the tools must have a means to exchange the model data and share the transformation information. There are multiple means for exchanging data between tools, including file exchange, use of application program interfaces (API), and a shared repository.

The use of modeling standards for modeling languages, model transformations, and data exchange is an important enabler of integration across modeling domains.

References

Works Cited

- Barry, P.S., M.T.K. Koehler, and B.F. Tivnan. 2009. *Agent-Directed Simulation for Systems Engineering*. McLean, VA: MITRE, March 2009, PR# 09-0267.
- DoD. 1998. "DoD modeling and simulation (M&S) glossary," in *DoD Manual 5000.59-M*. Arlington, VA, USA: US Department of Defense. January 1998.
- Wymore, A. 1967. *A Mathematical Theory of Systems Engineering: The Elements*. New York, NY, USA: John Wiley.
- Wymore, A. 1993. *Model-Based Systems Engineering*. Boca Raton, FL, USA: CRC Press.

Primary References

- Law, A. 2007. *Simulation Modeling and Analysis*, 4th ed. New York, NY, USA: McGraw Hill.
- Wymore, A. 1993. *Model-Based Systems Engineering*. Boca Raton, FL, USA: CRC Press.

Additional References

- Estefan, J. 2008. *Survey of Candidate Model-Based Systems Engineering (MBSE) Methodologies*, Revision B. Pasadena, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TD-2007-003-02.
- Hybertson, D. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boca Raton, FL, USA: Auerbach/CRC Press.
- INCOSE. 2007. *Systems Engineering Vision 2020*. Seattle, WA, USA: International Council on Systems Engineering. September 2007. INCOSE-TP-2004-004-02.
- Rouquette, N. and S. Jenkins. 2010. *OWL Ontologies and SysML Profiles: Knowledge Representation and Modeling*. Proceedings of the NASA-ESA PDE Workshop, June 2010.

System Modeling Concepts

- Lead Author:
- Sanford Friedenthal
- Contributing Author:
- Dov Dori, Yaniv Mordecai

A system model represents aspects of a system and its environment. There are many different types of models, as there are a variety of purposes for which they are built. It is useful to have a common way to talk about the concepts underlying the many different types of models (e.g., many modeling techniques enable the understanding of system behavior, while others enable the understanding of system structure). This article highlights several concepts used for modeling systems.

Abstraction

Perhaps the most fundamental concept in systems modeling is abstraction, which concerns hiding unimportant details in order to focus on essential characteristics. Systems that are worth modeling have too many details for all of them to reasonably be modeled. Apart from the sheer size and structural complexity that a system may possess, a system may be behaviorally complex as well, with emergent properties, non-deterministic behavior, and other difficult-to-characterize properties. Consequently, models must focus on a few vital characteristics in order to be computationally and intellectually tractable. Modeling techniques address this complexity through various forms of abstraction. For example, a model may assume that structural characteristics of many individual components of a particular type are all the same, ignoring the small order differences between individuals in instances that occur in real life. In that case, those differences are assumed to be unimportant to modeling the structural integrity of those components. Of course, if that assumption is wrong, then the model could lead to false confidence in that structural integrity. There are two key concepts that are applied in regard to modeling different levels of abstraction, which are: view and viewpoint, and black-box and white-box modeling, which are described below. Although these two modeling methods are the most widely recognized, different modeling languages and tools employ other techniques as well.

View and Viewpoint

IEEE 1471, a standard for architecture modeling, defines "view" and "viewpoint" as follows:

- View - A representation of a whole system from the perspective of a related set of concerns.
- Viewpoint - A specification of the conventions necessary for constructing and using a view; a pattern or template from which to develop individual views by establishing the purposes and audience for a view and the techniques for its creation and analysis.

Even though IEEE 1471 is focused on architecture models, the concepts of view and viewpoint are general and could apply to models for other purposes as well (IEEE 2000). The viewpoint addresses the concerns of the stakeholders and provides the necessary conventions for constructing a view to address those concerns; therefore, the view represents aspects of the system that address the concerns of the stakeholder. Models can be created to represent the different views of the system. A systems model should be able to represent multiple views of the system to address a range of stakeholder concerns. Standard views may include requirements, functional, structural, and parametric views, as well as a multitude of discipline-specific views to address system reliability, safety, security, and other quality characteristics.

Black-Box and White-Box Models

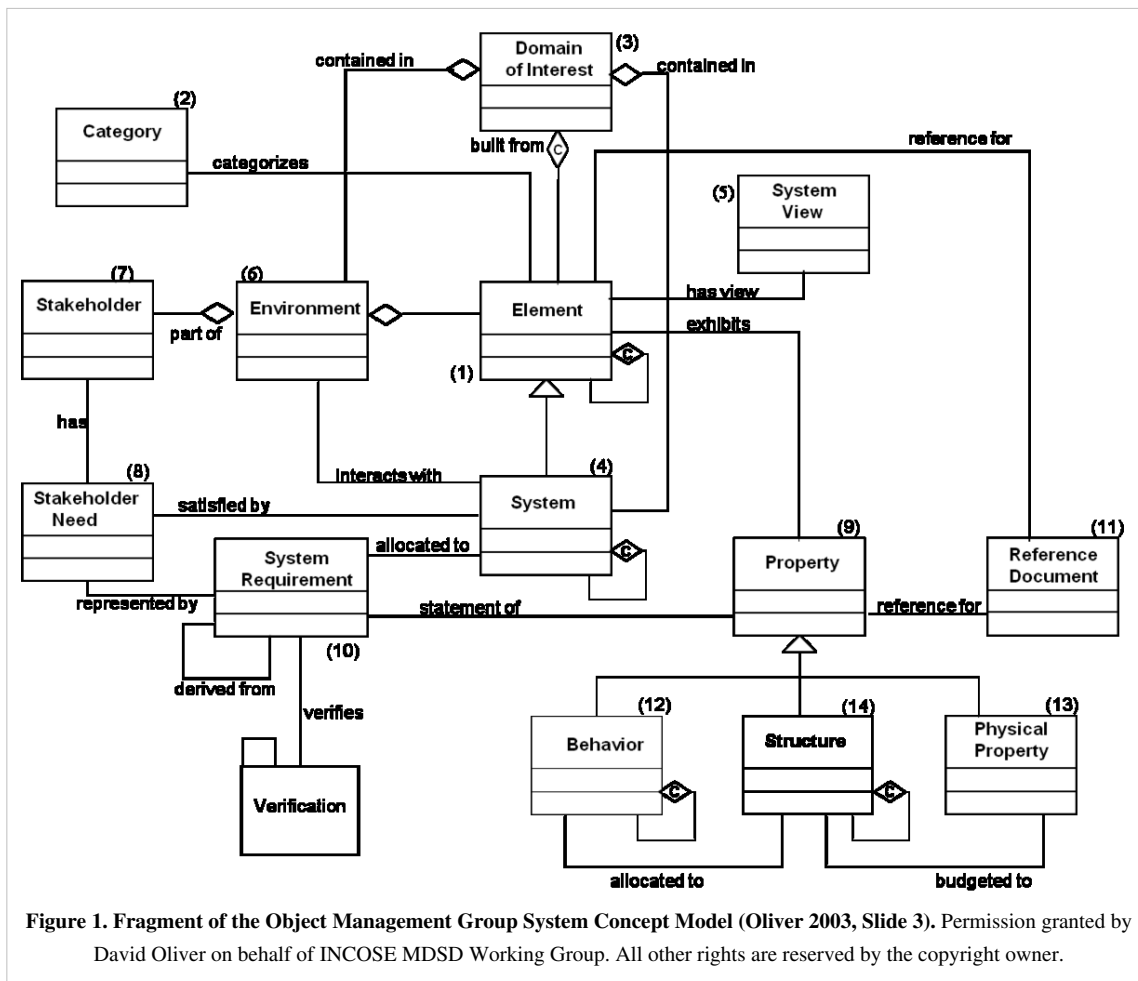
A very common abstraction technique is to model the system as a black-box, which only exposes the features of the system that are visible from an external observer and hides the internal details of the design. This includes externally visible behavior and other physical characteristics, such as the system's mass or weight. A white-box model of a system, on the other hand, shows the internal structure and displays the behavior of the system. Black-box and white-box modeling can be applied to the next level of design decomposition in order to create a black-box and white-box model of each system component.

Conceptual Model

A conceptual model is the set of concepts within a system and the relationships among those concepts (e.g., view and viewpoint). A system conceptual model describes, using one diagram type (such as in Object-Process Methodology (OPM)) or several diagram types (such as in Systems Modeling Language (SysML)), the various aspects of the system. The conceptual model might include its requirements, behavior, structure, and properties. In addition, a system conceptual model is accompanied by a set of definitions for each concept. Sometimes, system concept models are defined using an entity relationship diagram, an object-process diagram (OPD), or a Unified Modeling Language (UML) class diagram.

A preliminary conceptual (or concept) model for systems engineering (Systems Engineering Concept Model) was developed in support of the integration efforts directed toward the development of the Object Management Group (OMG) SysML and the International Organization for Standardization (ISO) AP233 *Data Exchange Standard for Systems Engineering* (ISO 2010). The concept model was originally captured in an informal manner; however, the model and associated concepts were rigorously reviewed by a broad representation of the systems engineering community, including members from the International Council on Systems Engineering (INCOSE), AP233, and SysML development teams.

A fragment from the top level systems engineering concept model is included in Figure 1. This model provides concepts for requirements, behavior, structure and properties of the system, as well as other concepts common to systems engineering and project management, such as the stakeholder. The concept model is augmented by a well-defined glossary of terms called the semantic dictionary. The concept model and the semantic dictionary contributed greatly to the requirements for the OMG Systems Modeling Language written in the UML for Systems Engineering Request for Proposal.



A concept model is sometimes referred to as a meta-model, domain meta-model, or schema, and can be used to specify the abstract syntax of a modeling language (refer to the Model Driven Architecture (MDA®) Foundation Model (OMG 2010)). Several other systems engineering concept models have been developed but not standardized. Future standardization efforts should establish a standard systems engineering concept model. The model can then evolve over time as the systems engineering community continues to formalize and advance the practice of systems engineering.

References

Works Cited

- IEEE. 2000. *Recommended Practice for Architectural Description for Software-Intensive Systems*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1471-2000.
- ISO. 2010. *OMG System Modeling Language (OMG SysML)*, version 1.2. Needham, MA, USA: Object Management Group.
- OMG. 2010. *MDA Foundation Model*. Needham, MA, USA: Object Management Group. Document number ORMSC/2010-09-06.

Primary References

ANSI/IEEE. 2000. *Recommended Practice for Architectural Description for Software-Intensive Systems*. New York, NY, USA: American National Standards Institute (ANSI)/Institute of Electrical and Electronics Engineers (IEEE), ANSI/IEEE 1471-2000.

Dori, D. 2002. *Object-Process Methodology – A Holistic Systems Paradigm*. New York, NY, USA: Springer-Verlag.

Guizzardi, G. 2007. "On ontology, ontologies, conceptualizations, modeling languages, and (meta)models," Proceedings of the 2007 Conference on Databases and Information Systems IV. Available at: <https://portal.acm.org/citation.cfm?id=1565425>.

IEEE. 2000. *Recommended Practice for Architectural Description for Software-Intensive Systems*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1471-2000.

INCOSE. 2003. *Systems Engineering Concept Model. Draft 12 Baseline*. Seattle, WA, USA: International Council on Systems Engineering. Available at: https://syseng.omg.org/SE_Conceptual%20Model/SE_Conceptual_Model.htm.

OMG. 2003. *UML for Systems Engineering Request for Proposal*. Needham, MA, USA: Object Management Group. OMG document number ad/2003-3-41. Available at: <https://www.omg.org/cgi-bin/doc?ad/2003-3-41>.

Additional References

None.

Integrating Supporting Aspects into System Models

- Lead Author:
- Sanford Friedenthal
- Contributing Authors:
- Dov Dori and Yaniv Mordecai

This article discusses the integrated modeling of systems and supporting aspects using Model-Based Systems Engineering methodologies and frameworks. Supporting aspects of systems engineering include:

- Engineering Management
- Project Management
- Requirements Engineering and Management
- Risk Modeling, Analysis, and Management
- Quality Assurance, Testing, Verification, and Validation
- System Integration and Employment
- Analysis of "-ilities" (e.g., Reliability, Availability, Maintainability, Safety, And Security (RAMSS), Manufacturability, Extensibility, Robustness, Resilience, Flexibility, and Evolvability)

These aspects can pertain to physical facets, as well as to functional, structural, behavioral, social, and environmental facets of the core system model. The article focuses on three main aspects:

1. Project and Engineering Management
 2. Risk Modeling, Analysis, and Management
 3. Requirements Definition and Management
-

Background

The model-based approach to systems engineering considers the system model as much more than a plain description of the system; the model is the central common basis for capturing, representing, and integrating the various system aspects listed above. The model is essential to the design and understanding of the system as well as to managing its life cycle and evolution. Modeling languages constitute the basis for standardized, formal descriptions of systems, just like natural languages form the basis for human communication.

As systems progressively become more complex and multidisciplinary, the conceptual modeling of systems needs to evolve and will become more critical for understanding complex design (Dori 2002). In addition to facilitating communication among clients, designers, and developers, conceptual modeling languages also assist in clearly describing and documenting various domains, systems, and problems, and define requirements and constraints for the design and development phases (Wand and Weber 2002). The importance of model-based analysis is demonstrated by the variety of conceptual modeling methodologies and frameworks, although *de facto* standards are slow to emerge. While certain disciplines of engineering design, such as structural analysis or circuit design, have established modeling semantics and notation, the conceptual modeling of complex systems and processes has not yet converged on a unified, consolidated modeling framework (Estefan 2007). The challenge is not only to integrate multiple aspects and support the various phases of the system's life cycle, but also to capture the multidisciplinary nature of the system, which has led to the creation of various frameworks. Nevertheless, the information systems analysis paradigm is currently the most widely used, perhaps due to the need to integrate complex systems via information-intensive applications and interactions.

Integrated Modeling of Systems and Projects

This section discusses the integrated modeling of systems and projects and of the project-system relationship (often called Project-Product Integration). The fields of project management and systems engineering have been advancing hand-in-hand for the last two decades, due to the understanding that successful projects create successful systems. Many of the main systems engineering resources pay considerable attention to project management and consider it to be a critical process and enabler of systems engineering (INCOSE 2012; NASA 2007; Sage and Rouse 2011). The integration of system-related aspects and concepts into project plans is more common than the integration of project-related aspects and concepts into system models. Because the project is a means to an end, it is the process that is expected to deliver the system. Indeed, project activities are often named after or in accord with the deliverables that they are aimed at facilitating (e.g., "console design," "software development," "hardware acquisition," or "vehicle assembly"). Each is a function name, consisting of an object (noun), or the system to be attained, and a process (verb), being the project or part of the project aimed at attaining the end system.

The specific process associated with each of these examples refers to different stages or phases of the project and to different maturity levels of the system or sub-system to which it applies. Moreover, the mere inclusion of system and part names in activity names does not truly associate system model artifacts with these activities. Overall, it is not truly possible to derive the set of activities associated with a particular part or functionality of the system that will be delivered by the project. Project-Product integration is not straightforward, as project models and system models are traditionally disparate and hardly interface. A model-based approach to project-system integration follows a system-centric paradigm and focuses on incorporating project-related aspects and concepts into the core system model, as opposed to the project-centric approach described in the previous paragraph. Such aspects and concepts include schedule, budget and resources, deliverables, work-packages, constraints and previous relations. The integrated system-project model should provide useful information on the mutual effects of project activities and system components and capabilities. Some examples of integrated system-project modeling include the following:

- The set of project activities associated with a particular system component, feature, or capability.
 - The set of resources required for performing a task of designing or developing a particular component of the system.
-

- The team or subcontractor responsible for delivering each system component.
- The preexisting dependencies between activities of system components deployment.
- The cost associated with each system component, feature, or capability.
- The parts of the system negotiated for each delivery, deployment, build, release, or version.

The Work Breakdown Structure (WBS) is designed to support the division of the project scope (work content) amongst the individuals and organizations participating in the project (Golany and Shtub 2001). The WBS is traditionally organization or activity-oriented; however, one of its main cornerstones focuses on the deliverable, which corresponds to the system, sub-system, component, or a capability or feature of one or more of these. A deliverable-oriented WBS, in which the high-level elements correspond to primary sub-systems, is advocated, as it is likely to allow the WBS to be more product-oriented (Rad 1999). An integrated approach to project planning and system modeling (Sharon and Dori 2009) merges the system model with the project's WBS using Object-Process Methodology (OPM) (Dori 2002). The unified OPM model captures both the project activities and the system components and functionalities.

The Design Structure Matrix (DSM) is a common method for enhancing and analyzing the design of products and systems. DSMs can be component-based, task-based, parameter-based, or team-based (Browning 2001). A DSM for an OPM-based project-product model derives a hybrid DSM of project activities and system building blocks from the unified OPM model, accounting for dependencies between project activities and system components, as well as replacing the two monolithic and separate component-based and task-based DSM views (Sharon, De-Weck, and Dori 2012). The underlying OPM model assures model consistency and traceability. The integrated project-product OPM model includes both a diagram and an equivalent auto-generated textual description. The DSM derived from this model visualizes a dependency loop comprising both system components and project activities.

Another model-based approach (Demoly et al. 2010) employs System Modeling Language (SysML) in order to create various views that meet the needs of various system stakeholders, such as the project/process manager. The approach includes both product-oriented and process-oriented views.

Integrated Modeling of Systems and Requirements

Requirements are statements that describe operational, functional, or design-related aspects of a system. Requirements definition and management is an important SE process, as it both initiates and facilitates the entire SE effort by defining the expected functions and performance of the engineered system. Several challenges associated with requirements include:

- Defining the requirements in a structured, controlled manner.
- Tracing these requirements to system components, aspects, and decisions.
- Testing and verifying compliance of the system with these requirements.

The extension of conceptual system models to include requirements has several significant benefits:

1. Requirements provide the rationale for the system's architecture and design by making and justifying architectural and design decisions based on specific requirements.
2. Modeling the internal logic and the hierarchy and dependency relations among requirements enables identification and elimination of redundant and contradictory requirements.
3. Teams and persons responsible for delivering various system components can often take responsibility for satisfying specific requirements. While the advantages of having good requirements engineering is clear, it is often a challenge to directly trace requirements to specific system artifacts, especially when the requirements are defined in a holistic, solution-independent manner.

There are several methods to incorporate requirements into system models, including SysML Requirements Engineering and Object-Process Methodology (OPM)-based Requirements Engineering and Authoring.

SysML-Based Requirements Engineering

The SysML requirements diagram makes it possible to capture the requirements and the relations among them in a visual manner, which is more intuitive than the textual manner in which requirements are traditionally edited and managed. The diagram was added to the basic set of UML diagrams that formed the basis for SysML (Friedenthal, Moore, and Steiner 2006), and is not a native UML diagram. Tracing requirements to the system blocks and artifacts satisfying them can be captured in the SysML Block Definition Diagram, which is primarily designated to capture the relations among types of system elements and components. The < > link between the block and the requirement captures the trace.

OPM-Based Requirements Engineering

Object-Process Methodology (OPM) is a methodology and language for conceptual modeling of complex systems and processes with a bimodal textual and graphical representation (Dori 2002). OPM's textual representation is coordinated with the graphical representation; additionally, each visual model construct in the Object-Process Diagram (OPD) is described by a formal structured textual statement in Object-Process Language (OPL), which is a subset of natural English. OPM facilitates model-based requirements engineering, authoring, and specification, in three possible modes:

1. OPM can be used to generate conceptual models which initially focus on the requirements level—the problem domain, rather than the design level or the solution domain, which facilitates automated model-based requirements generation (Blekhman and Dori 2011). The requirements model is solution-neutral, and it can be the basis for one or more architectural solutions for achieving the functions specified in the requirements.
2. OPM can be utilized in order to generate requirement-oriented OPDs in a manner similar to how the SysML Requirements Diagram enables an engineer to capture the requirements specification as the skeleton for the system model. User-defined tagged structural relations, such as "is realized by" or "is allocated to," provide for associating requirements with system model functions (objects and processes that transform them). This approach is similar to the SysML requirements diagram; however, instead of using a unique notation in a separate diagram type, the requirements are seamlessly incorporated into the single system model.
3. OPM can be used for the purpose of generating visual system models from formally specified requirements by tracing the textually authored requirements to system model inserts and artifacts (Dori et al. 2004).

Integrating Risk into System Models

Risk is an expression and a measure of the negative or adverse impact of uncertainty. Risk exists whenever uncertainty can lead to several results, of which some may be negative (adverse) and some positive. A system faces risks from other systems or from the environment, and it can also pose risks to other systems or to the environment. Systems are characterized by such attributes, such as: goals, objectives, inputs, outputs, variables, parameters, processes, events, states, subsystems, interfaces, mechanisms, and methods. System vulnerability is the system's total potential to be harmed or negatively affected in any one of these attributes. Analogously, system harmfulness is the system's total potential to harm others or to generate negative effects, which can be manifested in one or more of these attributes (Haimes 2009). Model-based risk analysis (MBRA) enables structured analysis and risk-related process control. Several model-based risk analysis approaches are available in the literature. MBRA is presently more common in the information technology and information security domains than in the systems engineering domain; however, some of the methods are generally applicable to complex systems as well. The ISO-IEC-IEEE collaborative software development and operation lifecycle standard (ISO and IEC 2004) proposes a concurrent approach to IT Risk Management. This approach consists of six main activities:

- Plan and Implement Risk Management
 - Manage the Project Risk Profile
 - Perform Risk Analysis
-

- Perform Risk Monitoring
- Perform Risk Treatment
- Evaluate the Risk Management Process

These activities are executed concurrently, affect and provide feedback to each other, and interact with other software life cycle processes, such as the technical management and the design processes (ISO and IEC 2004).

The CORAS approach (Fredriksen et al. 2002; den Braber et al. 2006; Lund, Solhaug, and Stølen 2011) is a UML-derivative for IT security risk modeling and assessment. This framework consists mostly of the UML use case (UC) diagram, extended for misuse cases. Additional notation was added to the UC notation in order to capture risk sources, effects, and results (e.g., the “bad actor” icon, moneybag for asset-in-risk). A misuse diagram can include, for example, the risk of loss of legal protection of proprietary know-how due to information theft and distribution by an unfaithful employee. The treatment for the risk source of insufficient security policy, which contributes to the above risk, is illustrated in a separate treatment diagram.

A quantitative risk assessment method for component-based systems (Grunske and Joyce 2008) supports component vulnerability analysis and specification using modular attack trees. In addition, it provides attacker profiling, which enables supporting econometric approaches to risk response. The methodology utilizes SysML as its underpinning language, especially the SysML block definition diagram and parametric diagram, in order to capture parametric relations and constraints as a means to defining risk profiles.

System-Theoretic Accident Model and Processes (STAMP) is a method for system and component design for safety (Leveson 2011). STAMP reformulates the safety problem as a control problem as opposed to a reliability problem. STAMP is optimized for safety-oriented systems engineering and design and for hazard avoidance and mitigation, specifically in complex socio-technical systems. A model-based adaptation of STAMP was also proposed (Leveson 2004) and was implemented in various safety-critical and mission-critical systems, including aircraft collision avoidance systems (CAS) (Leveson 2004) and ballistic missile defense systems (Pereira, Lee, and Howard 2006). Risk-Oriented Systems Engineering (ROSE) (Mordecai and Dori 2013) is a method based on Object-Process Methodology (OPM) for integrating risk into system models. Being system-centric, ROSE is responsible for capturing risk layers and aspects on top of and in sync with the core system model, while improving and immunizing it against captured risks, as well as for generating system robustness and resilience by design in response to various risk-posing scenarios. The risk handling meta-model includes risk mitigation during the design phase and risk response during the operational phase.

References

Works Cited

- Blekhman, A. and D. Dori. 2011. “Model-Based requirements authoring - Creating explicit specifications with OPM,” in 6th International Conference on Systems Engineering. Herzeliyya, Israel.
- Browning, T.R. 2001. “Applying the design structure matrix to system decomposition and integration problems: A review and new directions,” *IEEE Transactions on Engineering Management*, vol. 48, no 3, pp. 292–306. Available at: IEEE <https://ieeexplore.ieee.org/lpdocs/epic03/wrapper.htm?arnumber=946528>, and doi:10.1109/17.946528. Accessed December 4, 2014.
- Demoly, F., D. Monticolo, B. Eynard, L. Rivest, and S. Gomes. 2010. “Multiple viewpoint modelling framework enabling integrated product–process design,” *International Journal on Interactive Design and Manufacturing (IJIDeM)*, vol. 4, no. 4, October 12, pp. 269–280. Available at: Springer <https://link.springer.com/10.1007/s12008-010-0107-3>, and doi:10.1007/s12008-010-0107-3. Accessed December 4, 2014.
- Den Braber, F., G. Brændeland, H.E.I. Dahl, I. Engan, I. Hogganvik, M.S. Lund, B. Solhaug, K. Stølen, and F. Vraalsen. 2006. *The CORAS Model-based Method for Security Risk Analysis*. Oslo, Norway: SINTEF.

- Dori, D. 2002. *Object-Process Methodology – A Holistic Systems Paradigm*. New York, NY, USA: Springer-Verlag.
- Dori, D., N. Korda, A. Soffer, and S. Cohen. 2004. "SMART: System model acquisition from requirements text," *Lecture Notes in Computer Science: Business Process Management*, vol. 3080, pp. 179–194. Available at: Springer https://link.springer.com/chapter/10.1007/978-3-540-25970-1_12. Accessed December 4, 2014.
- Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, Rev. B. San Diego, CA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02.
- Friedenthal, S., A. Moore, and R. Steiner. 2006. "OMG Systems Modeling Language (OMG SysML™) Tutorial" (July).
- Golany, B. and A. Shtub. 2001. "Work breakdown structure," in Salvendy, G. Ed. 2001. *Handbook of Industrial Engineering, Technology and Operations Management*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, Inc., pp. 1263–1280.
- Grunske, L. and D. Joyce. 2008. "Quantitative risk-based security prediction for component-based systems with explicitly modeled attack profiles," *Journal of Systems and Software*, vol. 81, no. 8, pp. 1327–1345.
- Haimes, Y. 2009. "On the complex definition of risk: A systems-based approach," *Risk Analysis*, vol. 29, no. 12, pp. 1647–1654. Available at: Wiley <https://onlinelibrary.wiley.com/doi/10.1111/j.1539-6924.2009.01310.x/full>. Accessed December 4, 2014.
- ISO/IEC/IEEE. 2004. *Systems and Software Engineering - Life Cycle Processes - Risk management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC/IEEE 16085:2006.
- Leveson, N.G. 2004. "Model-Based Analysis of Socio-Technical Risk." Cambridge, MA, USA: Massachusetts Institute of Technology (MIT) Working Paper Series. ESD-WP-2004-08.
- Leveson, N.G. 2011. *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA, USA: MIT Press.
- Lund, M.S., B. Solhaug, and K. Stølen. 2011. *Model-Driven Risk Analysis: The CORAS Approach*. Berlin and Heidelberg, Germany: Springer Berlin Heidelberg. Available at: Springer <https://www.springerlink.com/index/10.1007/978-3-642-12323-8>, and doi:10.1007/978-3-642-12323-8. Accessed December 4, 2014.
- Mordecai, Y., and D. Dori. 2013. "Model-Based risk-oriented robust systems design with object-process methodology," *International Journal of Strategic Engineering Asset Management*, vol. 1, pp. 331-354 (CESUN 2012 Special Issue).
- NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.
- Pereira, S.J., G. Lee, and J. Howard. 2006. "A system-theoretic hazard analysis methodology for a non-advocate safety assessment of the ballistic missile defense system," Vol. 1606. Available at: Defense Technical Information Center <https://oai.dtic.mil/oai/oai?verb=getRecord&metadataPrefix=html&identifier=ADA466864>. Accessed December 4, 2014.
- Rad, P.F. 1999. "Advocating a Deliverable-Oriented Work Breakdown Structure." *Cost Engineering*, vol. 41, no. 12, p. 35. Sage, Andrew P., and William B. Rouse. 2011. *Handbook of Systems Engineering and Management*. Hoboken, NJ, USA: John Wiley & Sons.
- Sharon, A., O.L. de Weck, and D. Dori. 2012. "Improving project-product lifecycle management with model-based design structure matrix: A joint project management and systems engineering approach," *Systems Engineering*, vol. 16, no. 4, pp. 413-426. Available at: doi:10.1002/sys.21240.
- Sharon, A., and D. Dori. 2009. "A model-based approach for planning work breakdown structures of complex systems projects," in Proc. 14th IFAC Symposium on Information Control Problems in Manufacturing.

Wand, Y., and R. Weber. 2002. "Research Commentary: Information Systems and Conceptual Modeling-A Research Agenda," *Information Systems Research*, vol. 13, no. 4, December, pp. 363–376. Available at: doi:10.1287/isre.13.4.363.69.

Primary References

- Dori, D. 2002. *Object-Process Methodology – A Holistic Systems Paradigm*. New York, NY, USA: Springer-Verlag.
- Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, Rev. B. San Diego, CA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02.
- Golany, B. and A. Shtub. 2001. "Work breakdown structure," in Salvendy, G. Ed. 2001. *Handbook of Industrial Engineering, Technology and Operations Management*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, Inc., pp. 1263–1280.
- INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- NASA. 2007. *Systems Engineering Handbook*. Washington, D.C., USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Additional References

- Kristiansen, B.G. and K. Stolen. 2002. "The CORAS framework for a model-based risk management process," in *Lecture Notes*, S. Anderson, M. Felici, and S. Bologna. Eds., vol. 2434, pp. 94–105. Berlin and Heidelberg, Germany: Springer-Verlag. Available at: doi:10.1007/3-540-45732-1_11.

Modeling Standards

- Lead Author:
- Sanford Friedenthal
- Contributing Authors:
- Dov Dori and Yaniv Mordecai

Different types of models are needed to support the analysis, specification, design, and verification of systems. The evolution of modeling standards enables the broad adoption of Model-Based Systems Engineering (MBSE).

Motivation for Modeling Standards

Modeling standards play an important role in defining agreed-upon system modeling concepts that can be represented for a particular domain of interest and enable the integration of different types of models across domains of interest. Modeling standards are extremely important to support MBSE, which aims to integrate various system aspects across various disciplines, products, and technologies.

Standards for system modeling languages can enable cross-discipline, cross-project, and cross-organization communication. This communication offers the potential to reduce the training requirements for practitioners who only need to learn about a particular system and enables the reuse of system artifacts. Standard modeling languages also provide a common foundation for advancing the practice of systems engineering, as do other systems engineering standards.

Types of Modeling Standards

Many different standards apply to systems modeling. Modeling standards include standards for modeling languages, data exchange between models, and the transformation of one model to another to achieve semantic interoperability. Each type of model can be used to represent different aspects of a system, such as representing the set of system components and their interconnections and interfaces, or to represent a system to support performance analysis or reliability analysis.

The following is a partial list of representative modeling standards, which also includes the common acronym, when applicable, and a reference as to where additional information can be found on the topic.

Modeling Languages for Systems

Descriptive Models - These standards apply to general descriptive modeling of systems:

- Functional Flow Block Diagram (FFBD) (Oliver, Kelliher, and Keegan 1997)
- Integration Definition for Functional Modeling (IDEF0) (NIST 1993)
- Object-Process Methodology (OPM) [[1]] (Dori 2002; ISO/PAS 19450:2015)
- Systems Modeling Language (SysML)(OMG 2010a)
- Unified Profile for United States Department of Defense Architecture Framework (DoDAF) and United Kingdom Ministry of Defence Architecture Framework (MODAF) (OMG 2011e)
- Web ontology language (OWL) (W3C 2004b)

Analytical Models and Simulations - These standards apply to analytical models and simulations:

- Distributed Interactive Simulation (DIS) (IEEE 1998)
- High-Level Architecture (HLA) (IEEE 2010)
- Modelica (Modelica Association 2010)
- Semantics of a Foundational Subset for Executable Unified Modeling Language (UML) Models (FUML) (OMG 2011d)

Data Exchange Standards

These standards enable the exchange of information between models:

- Application Protocol for Systems Engineering Data Exchange (ISO 10303-233) (AP-233) (ISO 2005)
- Requirements Interchange Format (ReqIF) (OMG 2011c)
- Extensible Mark-Up Language - (XML) Metadata Interchange (XMI) (OMG 2003a)
- Resource Description Framework (RDF) (W3C 2004a)

Model Transformations

These standards apply to transforming one model to another to support semantic interoperability:

- Query View Transformations (QVT) (OMG 2011b)
- Systems Modeling Language (SysML)-Modelica Transformation (OMG 2010c)
- OPM-to-SysML Transformation (Grobshtein and Dori 2011)

General Modeling Standards

These standards provide general frameworks for modeling:

- Model-driven architecture (MDA®) (OMG 2003b)
 - IEEE 1471-2000 - Recommended Practice for Architectural Description of Software-Intensive Systems (ANSI/IEEE 2000) (ISO/IEC 2007)
-

Other Domain-Specific Modeling Standards

Software Design Models

These standards apply to modeling application software and/or embedded software design:

- Architecture Analysis and Design Language (AADL) (SAE 2009)
- Modeling and Analysis for Real-Time and Embedded Systems (MARTE) (OMG 2009)
- Unified Modeling Language (UML) (OMG 2010b)

Hardware Design Models

These standards apply to modeling hardware design:

- Very-High-Speed Integrated Circuit (VHSIC) Hardware Description Language (VHDL) (IEEE 2008)

Business Process Models

These standards apply to modeling business processes:

- Business Process Modeling Notation (BPMN) (OMG 2011a)

References

Works Cited

ANSI/IEEE. 2000. *Recommended Practice for Architectural Description for Software-Intensive Systems*. New York, NY, USA: American National Standards Institute (ANSI)/Institute of Electrical and Electronics Engineers (IEEE), ANSI/IEEE 1471-2000.

Grobshtein, Y. and D. Dori. 2011. "Generating SysML views from an OPM model: Design and evaluation," *Systems Engineering*, vol. 14, no. 3 Sept. 2011.

IEEE. 1998. *Distributed Interactive Simulation (DIS)*. Washington, DC, USA: Institute for Electrical and Electronic Engineers. IEEE 1278.1-1995. Available at: IEEE <https://standards.ieee.org/develop/project/1278.2.html>. Accessed December 4, 2014.

IEEE. 2008. *VHSIC Hardware Description Language (VHDL)*. Washington, DC, USA: Institute of Electrical and Electronics Engineers. IEEE Standard 1076-2008. Available at: IEEE <https://standards.ieee.org/findstds/standard/1076-2008.html>. Accessed December 4, 2014.

IEEE. 2010. *Standard for High Level Architecture*. Washington, DC, USA: Institute for Electrical and Electronic Engineers. IEEE Standard 1516. Available at: IEEE <https://standards.ieee.org/develop/intl/intlstds.html>. Accessed December 4, 2014.

ISO. 2005. *Application Protocol for Systems Engineering Data Exchange*. Geneva, Switzerland: International Organization for Standardization. ISO 10303-233. Available at: ISO https://www.iso.org/iso/iso_catalogue/catalogue_ics/catalogue_detail_ics.htm?csnumber=55257. Accessed December 4, 2014.

ISO. 2015. *Automation Systems and Integration - Object Process Methodology*. Geneva, Switzerland: International Organization for Standardization. ISO/PAS 19450:2015. Available at: ISO <https://www.iso.org/obp/ui/#iso:std:iso:pas:19450:ed-1:v1:en>. Accessed March 15, 2020.

ISO/IEC/IEEE. 2011. *Systems and Software Engineering — Architecture Description*. Geneva, Switzerland: International Organization for Standardization/International Electrotechnical Commission/Institute of Electrical and Electronics Engineers. December 1, 2011. ISO/IEC/IEEE 42010:2011. Available at: ISO https://www.iso.org/home/store/catalogue_ics/catalogue_detail_ics.htm?csnumber=50508. Accessed December 4, 2014.

Modelica Association. 2010. *Modelica® - A Unified Object-Oriented Language for Physical Systems Modeling, Language Specification, Version 3.2*. Modelica Association. Available at: Modelica <https://www.modelica.org/documents/ModelicaSpec32.pdf>. Accessed December 4, 2014.

- NIST. 1993. *Integration Definition for Functional Modeling (IDEF0)*. Gaithersburg, MD: National Institute for Standards and Technologies. Available at: IDEF <https://www.edef.com/IDEF0.htm>. Accessed December 4, 2014.
- Oliver, D., T. Kelliher, and J. Keegan. 1997. *Engineering Complex Systems with Models and Objects*. New York, NY, USA: McGraw Hill.
- OMG 2003a. *XML Metadata Interchange (XMI)*, Version 1.1. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/XML/>. Accessed December 4, 2014.
- OMG. 2003b. *Model Driven Architecture (MDA®)*, Version 1.0.1. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/mda>. Accessed December 4, 2014.
- OMG. 2009. *Modeling and Analysis for Real-Time and Embedded Systems (MARTE)*, Version 1.0. Object Management Group. Available at: OMG <https://www.omg.org/spec/MARTE/1.0/>. Accessed December 4, 2014.
- OMG. 2010a. *OMG Systems Modeling Language (SysML)*, Version 1.2. Needham, MA, USA: Object Management Group. Available at: SysML forum <https://www.sysml.org/docs/specs/OMGSysML-v1.2-10-06-02.pdf>. Accessed December 4, 2014.
- OMG. 2010b. *Unified Modeling Language™ (UML)*, Version 2. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/UML/>. Accessed December 4, 2014.
- OMG. 2010c. *SysML-Modelica Transformation Specification*, Beta Version. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/SyM/>. Accessed December 4, 2014.
- OMG. 2011a. *Business Process Modeling Notation (BPMN)*, Version 2.0. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/BPMN/2.0/>. Accessed December 4, 2014.
- OMG. 2011b. *Query View Transformations (QVT)*, Version 1.1. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/QVT/1.1/>. Accessed December 4, 2014.
- OMG. 2011c. *Requirements Interchange Format (ReqIF)*, Version 1.0.1. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/ReqIF/>. Accessed December 4, 2014.
- OMG. 2011d. *Semantics of a Foundational Subset for Executable UML Models (FUML)*, Version 1.0. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/FUML/1.0/>. Accessed December 4, 2014.
- OMG. 2011e. *Unified Profile for DoDAF and MODAF (UPDM)*, Version 1.1. Needham, MA, USA: Object Management Group. Available at: OMG <https://www.omg.org/spec/UPDM/>. Accessed December 4, 2014.
- SAE. 2009. *Architecture Analysis & Design Language (AADL)*. Warrendale, PA, USA: SAE International. Available at: Society of Automotive Engineers <https://standards.sae.org/as5506a/>. Accessed December 4, 2014.
- W3C. 2004a. *Resource Description Framework (RDF)*, Version 1.0. World Wide Web Consortium. Available at: World Wide Web Consortium <https://www.w3.org/RDF/>. Accessed December 4, 2014.
- W3C. 2004b. *Web Ontology Language. (OWL)*. World Wide Web Consortium. Available at: World Wide Web Consortium <https://www.w3.org/2004/OWL>. Accessed December 4, 2014.
-

Primary References

- Dori, D. 2002. *Object-Process Methodology – A Holistic Systems Paradigm*. Berlin and Heidelberg, Germany; New York, NY, USA: Springer Verlag.
- Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.

Additional References

- Fritzson, P. 2004. *Object-Oriented Modeling and Simulation with Modelica 2.1*. New York, NY, USA: Wiley Interscience and IEEE Press.
- Bibliowicz, A. and D. Dori. 2012. "A graph grammar-based formal validation of object-process diagrams," *Software and Systems Modeling*, vol. 11, no. 2, pp. 287-302.
- Blekhman, A. and D. Dori. 2011. "Model-Based Requirements Authoring." INCOSE 2011 – The 6th International Conference on System Engineering. March 2011.
- Dori, D., R. Feldman, and A. Sturm. 2008. *From conceptual models to schemata: An object-process-based data warehouse construction method*, "Information Systems, vol. 33, pp. 567–593.
- Osorio, C.A., D. Dori, and J. Sussman. 2011. *COIM: An object-process based method for analyzing architectures of complex, interconnected, large-scale socio-technical systems*, *Systems Engineering*, vol. 14, no. 3.
- Paredis, C.J.J. et al. 2010. "An overview of the SysML-Modelica transformation specification". Proceedings of the 20th Annual International Council on Systems Engineering (INCOSE) International Symposium, 12-15 July 2010, Chicago, IL.
- Reinhartz-Berger, I. and D. Dori. 2005. "OPM vs. UML—Experimenting with comprehension and construction of web application models," *Empirical Software Engineering*, vol. 10, pp. 57–79, 2005.
- Weilkiens, T. 2008. *Systems Engineering with SysML/UML*. Needham, MA, USA: OMG Press.

References

- [1] <https://www.iso.org/obp/ui/#iso:std:iso:pas:19450:ed-1:v1:en>

Knowledge Area: Systems Approach Applied to Engineered Systems

Systems Approach Applied to Engineered Systems

Contents of this Knowledge Area

- Overview of the Systems Approach
- Engineered System Context (Rick Adcock) (Brian Wells, Scott Jackson, and James Martin)
- Identifying and Understanding Problems and Opportunities (Rick Adcock) (Brian Wells, Scott Jackson, Janet Singer, Duane Hybertson, and Bud Lawson)
- Synthesizing Possible Solutions (Rick Adcock) (Scott Jackson, Janet Singer, and Duane Hybertson)
- Analysis and Selection between Alternative Solutions (Rick Adcock) (Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson)
- Implementing and Proving a Solution (Rick Adcock) (Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson)
- Deploying, Using, and Sustaining Systems to Solve Problems (Rick Adcock) (Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson)
- Applying the Systems Approach (Rick Adcock) (Brian Wells, Scott Jackson, Janet Singer, Duane Hybertson, Hillary Sillitto, Bud Lawson, and James Martin)
- Lead Author:
- Rick Adcock
- Contributing Authors:
- Janet Singer and Duane Hybertson

This knowledge area (KA) provides a guide for applying the systems approach as a means of identifying and understanding complex problems and opportunities, synthesizing possible alternatives, analyzing and selecting the best alternative, implementing and approving a solution, as well as

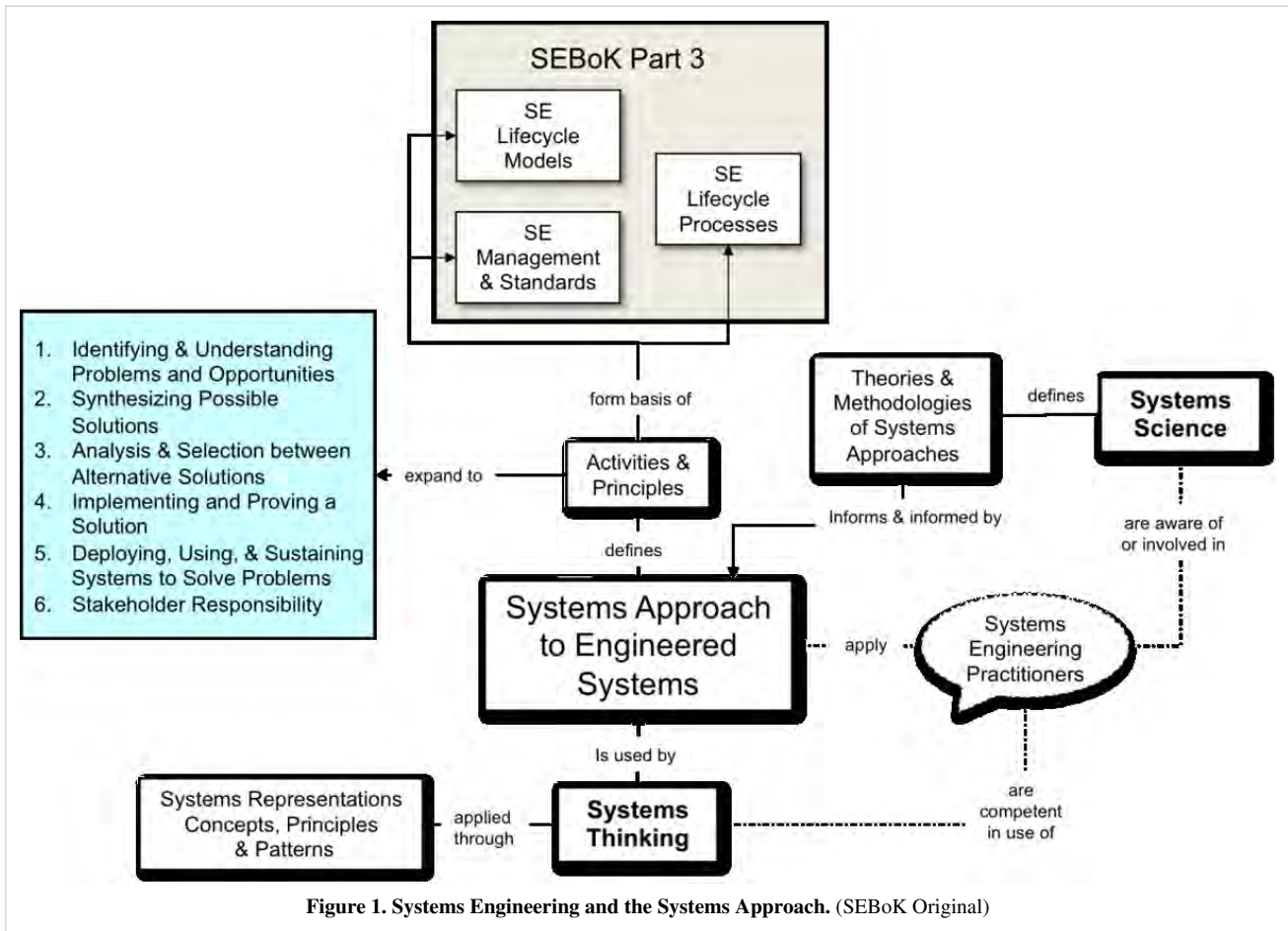
deploying, using and sustaining engineered system solutions. The active participation of stakeholders during all the activities of the systems approach is the key to the success of the systems approach.

In an engineered system context, a systems approach is a holistic approach that spans the entire life of the system; however, it is usually applied in the development and operational/support life cycle stages. This knowledge area defines a systems approach using a common language and intellectual foundation to ensure that practical systems concepts, principles, patterns and tools are accessible to perform systems engineering (SE), as is discussed in the introduction to Part 2: Foundations of Systems Engineering.



Systems Approach

This KA describes a high-level framework of activities and principles synthesized from the elements of the systems approach, as described earlier in Part 2 of the SEBoK, and is mapped to the articles Concepts of Systems Thinking, Principles of Systems Thinking, and Patterns of Systems Thinking. The concept map in Figure 1 describes how the knowledge is arranged in this KA and the linkage to the KA in Part 3.



According to Jackson et al. (2010, 41-43), the systems approach to engineered systems is a problem-solving paradigm. It is a comprehensive problem identification and resolution approach based upon the principles, concepts, and tools of systems thinking and systems science, along with the concepts inherent in engineering problem-solving. It incorporates a holistic systems view that covers the larger context of the system, including engineering and operational environments, stakeholders, and the entire life cycle.

Successful systems practice should not only apply systems thinking to the system being created but should also utilize systems thinking in consideration of the way in which work is planned and conducted. See Part 5: Enabling Systems Engineering for further discussions on how individuals, teams, businesses and enterprises may be enabled to perform systems engineering.

References

Works Cited

Jackson, S., D. Hitchins, and H. Eisner. 2010. "What is the Systems Approach?." *INCOSE Insight*, vol. 13, no. 1, pp. 41-43.

Primary References

Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.

Hitchins, D. 2009. "What are the General Principles Applicable to Systems?" *INCOSE Insight*, vol. 12, no. 4.

Jackson, S., D. Hitchins, and H. Eisner. 2010. "What is the Systems Approach?" *INCOSE Insight*, vol. 13, no. 1, pp. 41-43.

Additional References

Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: John Wiley & Sons.

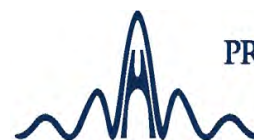
Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.

Senge, P. M. 1990. *The Fifth Discipline: The Art and Practice of the Learning Organization*. New York, NY, USA: Doubleday/Currency.

Overview of the Systems Approach

Lead Author: Rick Adcock, **Contributing Authors:** Janet Singer, Duane Hybertson, Hillary Sillitto, Bud Lawson, Brian Wells, James Martin

This knowledge area (KA) considers how a systems approach relates to engineered systems and to systems engineering (SE). The article Applying the Systems Approach considers the dynamic aspects of how the approach is used and how this relates to elements of SE.



**PROJECT PERFORMANCE
INTERNATIONAL**

The "Systems Approach Applied to Engineered Systems" knowledge area is graciously sponsored by PPI.

Systems Approach and Systems Engineering

The term systems approach is used by systems science authors to describe a systems "*thinking*" approach, as it pertains to issues outside of the boundary of the immediate system-of-interest (Churchman 1979). This systems approach is essential when reductionist assumptions (the notion that the whole system has properties derived directly from the properties of their components) no longer apply to the system-of-interest (SoI) and when emergence and complexity at multiple levels of a system context necessitate a holistic approach.

The systems approach for engineered systems is designed to examine the "*whole system, whole lifecycle, and whole stakeholder community*" as well as to ensure that the purpose of the system (or systemic intervention) is achieved sustainably without causing any negative unintended consequences. This prevents the engineer from "*transferring the burden*" (in systems thinking terms) to some other part of the environment that unable to sustain that burden (Senge 2006). This also deters issues involving *sub-optimization* that could occur when whole systems are not kept

in mind in achieving the purpose of the system (Sillitto 2012).

The systems approach (derived from systems thinking) and systems engineering (SE) have developed and matured, for the most part, independently; therefore, the systems science and the systems engineering communities differ in their views as to what extent SE is based on a systems approach and how well SE uses the concepts, principles, patterns and representations of systems thinking. These two views are discussed in the following sections.

Systems Science View

As discussed in the Systems Science article, some parts of the systems movement have been developed as a reaction to the perceived limitations of systems engineering (Checkland 1999).

According to Ryan (2008):

Systems engineering has a history quite separate to the systems movement. Its closest historical link comes from the application of systems analysis techniques to parts of the systems engineering process . . . The recent popularity of the SoS buzzword in the systems engineering literature has prompted the expansion of systems engineering techniques to include methods that can cope with evolving networks of semi-autonomous systems. This has led many systems engineers to read more widely across the systems literature, and is providing a re-conceptualization of systems engineering as part of the systems movement, despite its historical independence. This is reflected in the latest INCOSE hand-book [INCOSE 2011, page 52], which states “the systems engineering perspective is based on systems thinking”, which “recognizes circular causation, where a variable is both the cause and the effect of another and recognizes the primacy of interrelationships and non-linear and organic thinking—a way of thinking where the primacy of the whole is acknowledged. (emphases added)

Thus, for many in the systems science community, systems thinking is *not* naturally embedded in either SE definitions or practice.

Systems Engineering View

Many SE authors see a clear link between SE and systems thinking. For example, Hitchins (Hitchins 2007) describes generic models for the application of systems thinking to engineered system contexts. He suggests that these could form the foundation for descriptions and standards for the practices of SE. Hitchins also proposes a set of *guiding principles which have been the foundations of SE, apparently since its inception* (Hitchins 2009):

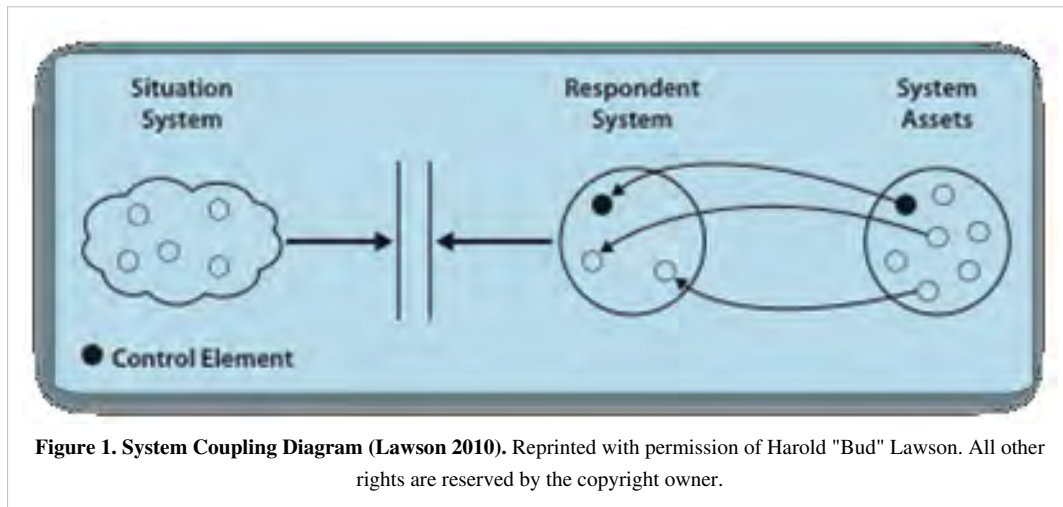
- **SE Principle A: The Systems Approach** - “SE is applied to a system-of-interest (SoI) in a wider systems context”
- **SE Principle B: Synthesis** - “SE must bring together a collection of parts to create whole system solutions”
- **SE Principle C: Holism** - “Always consider the consequences on the wider system when making decisions about the system elements”
- **SE Principle D: Organismic Analogy** - “Always consider systems as having dynamic “living” behavior in their environment”
- **SE Principle E: Adaptive Optimizing** - “Solve problems progressively over time”
- **SE Principle F: Progressive Entropy Reduction** - “Continue to make systems work over time, through maintenance, sustainment and, upgrade activities.”
- **SE Principle G: Adaptive Satisfying** - “A system will succeed only if it makes winners of its success-critical stakeholders, so the lifecycle of a system must be driven by how well its outputs contribute to stakeholder purpose”

Hitchins considers principles A-D as pillars of SE that identify key aspects of systems thinking which should underpin the practice of SE. Principles E-G consider the dynamics of SE life cycle thinking, the *why*, *when* and *how often* of SE.

The following sections consider the systems approach to engineered systems against four themes.

1. Whole System

The system coupling diagram (Figure 1), describes the scope of a systems approach to engineered systems (Lawson 2010).



- The **situation system** is the problem or opportunity either unplanned or planned. The situation may be natural, man-made, a combination of both, or a postulated situation used as a basis for deeper understanding and training (e.g. business games or military exercises).
- The **respondent system** is the system created to respond to the situation. The parallel bars indicate that this system interacts with the situation and transforms it to a new situation. Respondent systems have several names: project, program, mission, task force, or in a scientific context, experiment.
- **System assets** are the sustained assets of one or more enterprises to be used in response to situations. System assets must be adequately managed throughout the life of a system to ensure that they perform their function when instantiated in a respondent system. Examples include: value-added products or services, facilities, instruments and tools, and abstract systems, such as theories, knowledge, processes and methods.

Martin (Martin 2004) describes seven types of system, or *"the seven samurai of systems engineering,"* all of which, system developers need to understand to develop successful systems:

- the context system
- the intervention system
- the realization system
- the deployed system
- collaborating systems
- the sustainment system
- competing systems

Martin contends that all seven systems must be explicitly acknowledged and understood when engineering a solution for a complex adaptive situation.

These views, and others, describe one aspect of the systems approach when applied to engineered systems; in addition, it is applicable to understanding a problem, it organizes the resolution of that problem, and creates and integrates any relevant assets and capabilities to enable that solution.

2. Whole Lifecycle

Ring (Ring 1998) provides a powerful framework for the continuing management and periodic upgrade of long-life and “immortal” systems. It also accurately represents the “continuous” or very rapid product launch and refreshment cycle driven by market feedback and constant innovation that is seen in most product and service system consumer markets.

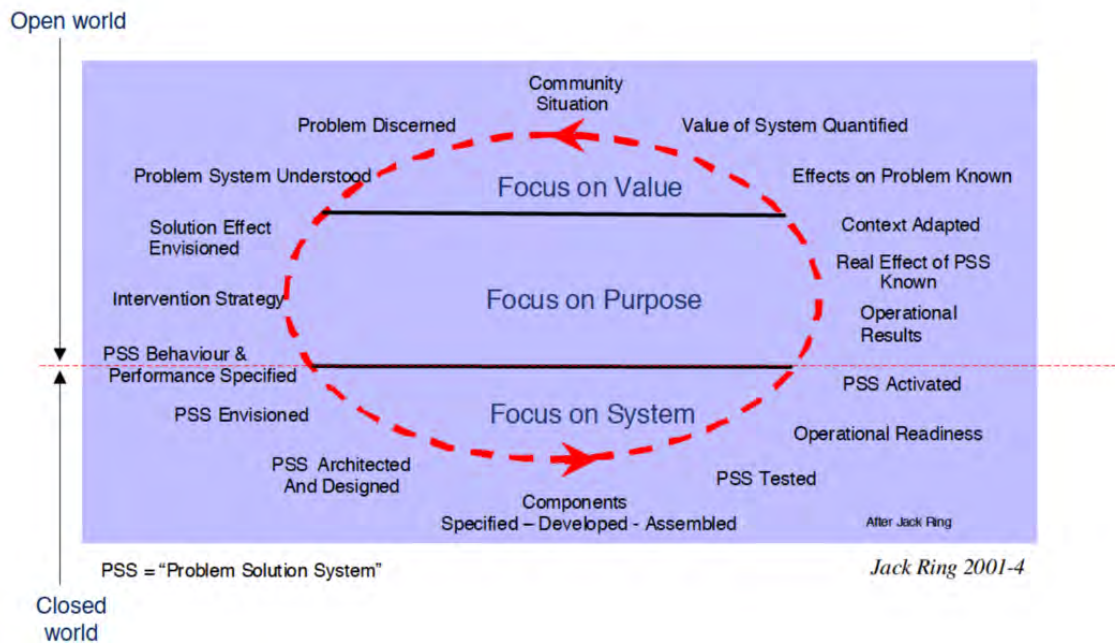


Figure 2. Ellipse Graphic (Ring 1998). © 1998 IEEE. Reprinted, with permission, from Jack Ring, Engineering Value-Seeking Systems, IEEE-SMC. Conference Proceedings. All other rights are reserved by the copyright owner.

Enterprise systems engineering may be considered in multiple concurrent instances of this model for different sub-sets of enterprise assets and services, in order to maintain a capability to pursue enterprise goals in a complex and dynamic external environment.

The dynamic nature of this cycle and its relationship to Life Cycle thinking is discussed in the article Applying the Systems Approach.

3. Whole Problem

The article Identifying and Understanding Problems and Opportunities considers the nature of problem situations. It discusses the relationship between hard system and soft system views of problems and how they relate to engineered systems. Engineered systems are designed to operate with and add value to a containing social and/or ecological system. The scope of problems is captured by frameworks, such as Political, Economic, Social, Technological, Legal and Environmental (PESTLE) (Gillespie 2007) or Social, Technical, Economic, Environmental, Political, Legal, Ethical and Demographic (STEEPLED).

The idea of a wicked problem (Rittel and Webber 1973) is also discussed. These problems cannot be quantified and solved in a traditional engineering sense.

Sillitto (Sillitto 2010) describes a lifecycle model in which the decision as to what parts of problems can be “solved” and what parts must be “managed” is the first key decision and emphasizes the need for a solution approach that provides flexibility in the solution to match the level of uncertainty and change in the problem and stakeholder expectations. It is now normal to view a problem as one that “changes over time” and to promote the belief that value is determined by the perceptions of key stakeholders.

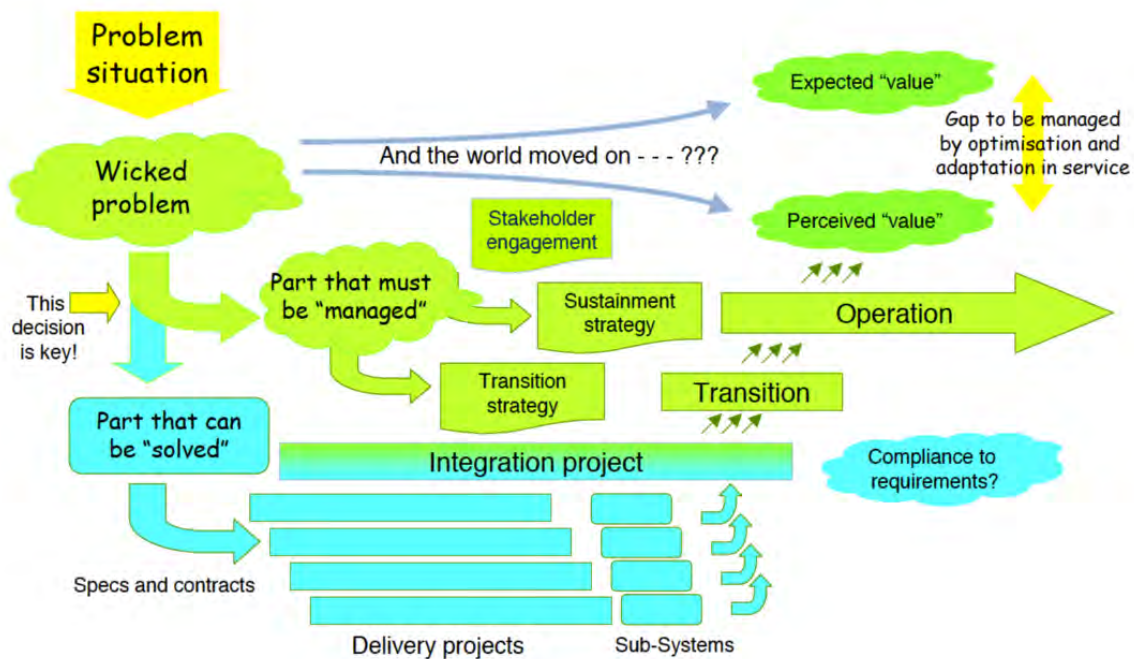


Figure 3. Engineered vs Managed Problems (Sillitto 2010). Reproduced with permission of Hillary Sillitto. All other rights are reserved by the copyright owner.

Thus, a systems approach can be useful when addressing all levels of a problematic situation, from individual technologies to the complex socio-technical issues that come about in the area of engineered systems development.

4. Multi-Disciplinary

As discussed by Sillitto (Sillitto 2012), the methods and thinking applied by many practicing systems engineers have become optimized to the domains of practice. While systems thinking concepts, patterns and methods are used widely, they are not endemic in SE practice. As a result, SE practitioners find it difficult to share systems ideas with others involved in a systems approach. Part 4: Applications of Systems Engineering describes traditional (product based) SE (Lawson 2010) and examines this against the SE approaches that are applicable to service, enterprise, and system of systems capability. These approaches require more use of problem exploration, a broader solution context, and a purpose driven life cycle thinking.

SE and Systems Approach

From the above discussions, there are three ways in which SE could make use of a systems approach:

- in its overall problem- solving approach
- in the scope of problem and solution system contexts considered
- in the embedding of systems thinking and systems thinking tools and in all aspects of the conduct of that approach

The current SE standards and guides, as described in Part 3: Systems Engineering and Management, encapsulate many of the elements of a systems approach. However, they tend to focus primarily on the development of system solutions while the wider purpose-driven thinking of a full systems approach (Ring 1998) and the wider consideration of all relevant systems (Martin 2004) are embedded in the acquisition and operational practices of their application domains.

The inclusion of systems thinking in SE competency frameworks (INCOSE 2010) represents a general move toward a desire for more use of systems thinking in SE practice. There is a wide stakeholder desire to acquire the benefits of a systems approach through the application of SE, particularly in areas where current SE approaches are inadequate or irrelevant. Hence, there is a need for a better articulation of the *systems approach* and how to apply it to non-traditional problems.

Synthesis for SEBOK

The systems approach presented in the SEBoK uses the following activities:

- identify and understand the relationships between the potential problems and opportunities in a real-world situation
- gain a thorough understanding of the problem and describe a selected problem or opportunity in the context of its wider system and its environment
- synthesize viable system solutions to a selected problem or opportunity situation
- analyze and choose between alternative solutions for a given time/cost/quality version of the problem.
- provide evidence that a solution has been correctly implemented and integrated
- deploy, sustain, and apply a solution to help solve the problem (or exploit the opportunity)

All of the above are considered within a life cycle (glossary) framework which may need concurrent, recursive (glossary) and iterative applications of some or all of the systems approach.

When the systems approach is executed in the real world of an engineered system (glossary), a number of engineering and management disciplines emerge, including SE. Part 3: Systems Engineering and Management and Part 4: Applications of Systems Engineering contain a detailed guide to SE with references to the principles of the systems approach, where they are relevant. Part 5: Enabling Systems Engineering provides a guide to the relationships between SE and the organizations and Part 6: Related Disciplines also offers a guide to the relationship between SE and other disciplines.

More detailed discussion of how the systems approach relates to these engineering and management disciplines is included in the article Applying the Systems Approach within this KA.

References

Works Cited

- Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.
- Churchman, C.W. 1979. *The Systems Approach and Its Enemies*. New York, NY, USA: Basic Books.
- Gillespie. 2007. *Foundations of Economics - Additional Chapter on Business Strategy*. Oxford, UK: Oxford University Press.
- Hitchins, D. 2009. "What are the General Principles Applicable to Systems?". *INCOSE Insight*, vol. . 12, no. (4).
- Hitchins, D. 2007. *Systems Engineering, a 21st Century Systems Methodology*. Hoboken, NJ, USA: Wiley.
- INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2010-003.
- INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.1. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.1.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.
- Martin, J., 2004. "The Seven Samurai of Systems Engineering: Dealing with the Complexity of 7 Interrelated Systems." Proceedings of the 14th Annual INCOSE International Symposium, 20-24 June 2004, Toulouse, France,

20-24 June 2004.

Ring, J., 1998. "A Value Seeking Approach to the Engineering of Systems." Proceedings of the IEEE Conference on Systems, Man, and Cybernetics. pp. 2704-2708.

Rittel, H. and M. Webber. 1973. "Dilemmas in a gGeneral tTheory of pPlanning." *Policy Sciences*, v. ol. 4, pp. :155–169.

Ryan, A. 2008. "What is a Systems Approach?" *Journal of Non-linear Science*.

Senge, P.M. 2006. *The Fifth Discipline: The Art and Practice of the Learning Organization*, 2nd ed. New York, NY, USA: Doubleday Currency.

Sillitto, H. 2010. "Design principles for ultra-large scale systems." Proceedings of the INCOSE International Symposium, Chicago, July 2010, re-printed in "The Singapore Engineer," July 2011.

Sillitto, H.G. 2012.: "Integrating sSystems sScience, sSystems tThinking, and sSystems eEngineering: Uunderstanding the differences and exploiting the synergies," Proceedings of the INCOSE International Symposium, Rome, Italy, July 2012.

Primary References

Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.

Hitchins, D. 2009. "What are the general principles applicable to systems," *INCOSE Insight*, vol. . 12, no. (4).

Senge, P.M. 2006. *The Fifth Discipline: The Art and Practice of the Learning Organization*, 2nd ed. New York, NY, USA: Doubleday Currency.

Additional References

Biggs, J.B. 1993. "From tTheory to pPractice: A cCognitive sSystems aApproach". *Journal of Higher Education & Development*. Accessed December 4 2014. Available at: Taylor and Francis from <https://www.informaworld.com/smpp/content~db=all~content=a758503083>.

Boardman, J. and B. Sauser. 2008. *Systems Thinking: Coping with 21st Century Problems*. Boca Raton, FL, USA: CRC Press.

Edson, R. 2008. *Systems Thinking. Applied. A Primer*. Arlington, VA, USA: Applied Systems Thinking (ASysT) Institute, Analytic Services Inc.

Ring J. 2004. "Seeing an Enterprise as a System." *INCOSE Insight*. 6(2): 7-8.

Engineered System Context

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, and James Martin

This article is part of the Systems Approach Applied to Engineered Systems knowledge area (KA). It describes knowledge related to the further expansion of the ideas of an engineered system and engineered system context that were introduced in the Systems Fundamentals KA.

The single most important principle of the systems approach is that it is applied to an engineered system context and not just to a single system (INCOSE 2012). The systems approach includes models and activities useful for the understanding, creation, use, and sustainment of engineered systems to enable the realization of stakeholder needs. Disciplines that use a systems approach (like systems engineering (SE)) consider an engineered system context that defines stakeholder needs, and look for the best ways to provide value by applying managed technical activities to one or more selected engineered systems of interest (SoI).

Generally, four specific types of engineered system contexts are recognized in SE:

- product system
- service system
- enterprise system
- system of systems (SoS) capability

One of the key distinctions between these system contexts pertains to the establishment of how and when the SoI boundary is drawn.

Engineered System-of-Interest

We use the idea of an engineered system context to define an engineered SoI and to capture and agree on the important relationships between it, the systems with which it works directly, and any other systems with which it works. All applications of a systems approach (and hence of SE) are applied in a system context rather than only to an individual system.

A system context can be constructed around the following set of open system relationships (Flood and Carson 1993):

- The **Narrower System-of-Interest** (NSoI) is the system of direct concern to the observer. The focus of this system is driven by the scope of authority or control with implicit recognition that this scope may not capture all related elements.
- The **Wider System-of-Interest** (WSoI) describes a logical system boundary containing all of the elements needed to fully understand system behavior. The observer may not have authority over all of the elements in the WSoI but will be able to establish the relationships between WSoI elements and NSoI elements.
- The WSoI exists in an **environment**. The immediate environment contains engineered, natural, and/or social systems, with which the WSoI (and thus some elements of the NSoI) directly interact for the purpose of exchanging material, information, and/or energy to achieve its goals or objective.
- A **Wider Environment** completes the context and contains systems that have no direct interaction with the SoI, but which might influence decisions related to it during its life cycle.



- "Some Theoretical Considerations of Mathematical Modeling" (Flood 1987) extends this context to include a **meta-system** (MS) that exists outside of the WSoI and exercises direct control over it.

The choice of the SoI boundary for particular activities depends upon what can be changed and what must remain fixed. The SoI will always include one or more NSoI but may also include WSoI and an MS if appropriate, such as when considering a service or an enterprise system.

Applying the System Context

For lower-level and less-complex systems, the WSoI can represent levels of a product system hierarchy. An example of this would be an engine management unit as part of an engine, or an engine as part of a car. The WSoI in a system context may encapsulate some aspects of SoS ideas for sufficiently complex systems. In these cases, the WSoI represents a collection of systems with their own objectives and ownership with which the NSoI must cooperate in working towards a shared goal. An example of this would be a car and a driver contributing to a transportation service.

This view of a SoS context being used as a means to support the engineering of an NSoI product system is one way in which a systems approach can be applied. It can also be applied directly to the SoS. Examples of this include a flexible multi-vehicle transportation service or transportation as part of a commercial enterprise. In this case, the NSoI aspect of the context no longer applies. The WSoI will consist of a set of cooperating systems, each of which might be changed or replaced to aid in the synthesis of a solution. The context may also need to represent **loose coupling**, with some systems moving in or out of the context depending on the need, or **late binding** with systems joining the context only at, or close to, the delivery of the service.

Thus, a context allows a reductionist view of the SoI that is of direct concern to an observer, as it provides for the system relationships and influences that are needed to maintain a holistic view of the consequence of any actions taken.

Product System Context

The distinction between a product and a product system is discussed in the article Engineered Systems.

A product system context would be one in which the SoI is the product itself. The wider system context for a product system can be a higher level of product hierarchy, a service, or an enterprise system that uses the product directly to help provide value to the user. A significant aspect of a product systems context is the clear statement of how the product is intended to be used and ensures that this information is given to the acquirer upon delivery. The customer will be required to accept the system, typically through a formal process, agreeing not to go against the terms of use.

If a systems approach is applied to a product context, it is done with the purpose of engineering a narrow system product to be integrated and used in a wider system product hierarchy or to enable the delivery of a wider system service directly to a user by an enterprise.

This view of the relationship between product and service is specific to product systems engineering. While some engineering of the acquirer's static service system may occur, it is done with a product focus. The definition of service system in a service systems engineering context describes a more dynamic view of service systems.

Service System Context

Services are activities that cause a transformation of the state of an entity (people, product, business, and region or nation) by mutually agreed terms between the service provider and the customer (Spohrer 2008). The distinction between service and a service system is discussed in the article Engineered Systems.

A service system context is one in which the SoI is the service system. This SoI contains all of the technology, infrastructure, people, resources, etc. that are needed to enable the service. The WSoI describes the enterprise providing the service as well as its relationship with other services that impact the success of the enterprise.

If a systems approach is applied to a service system, it is done with the purpose of engineering a service system to enable the outcomes required by an enterprise to satisfy its clients. When operating in the service system context, all options to provide the service must be considered, providing that they fit within the constraints of the enterprise. This will include interfaces to other services, people, and resources in the enterprise. If an option for providing the service makes use of existing products or resources within or outside of the enterprise, it must be ensured that they are available for this use and that this does not adversely affect other services. Part of getting the right service may require the negotiation of changes to the wider enterprise context, but this must be by agreement with the relevant authority.

For a service system, and also when considering the service system context, the value is realized only through service transactions. The end-user co-creates value at the time of the request to use the service. For example, to make a flight reservation using a smart phone, the service system is composed of many service system entities (the caller, the person called, the smart phone, the access network, the core Internet Protocol (IP) network, the Internet Service provider (ISP), the World Wide Web (WWW), data centers, etc. All these are necessary to enable the service. When a caller makes a reservation and then books the flight, the value has been created.

This definition of a service system, as associated with dynamic Information Technology (IT) services, is discussed further in the article Service Systems Engineering.

Enterprise System Context

The distinction between an enterprise and an enterprise system is discussed in the article Engineered Systems.

An enterprise system context is one in which the SoI is the enterprise system. This system contains all of the technology, infrastructure, people, resources, etc. needed to enable the service. The WSoI describes the business environment within which the enterprise sits.

It is to be noted that an enterprise context is not equivalent to an **organization** according to this definition. An enterprise includes not only the organizations that participate in it, but also the people, knowledge, and other assets, such as processes, principles, policies, practices, doctrines, theories, beliefs, facilities, land, and intellectual property that compose the enterprise.

An enterprise may contain or employ service systems along with product systems. An enterprise might even contain sub-enterprises. Enterprise systems are unique when compared to product and service systems in that:

- they are constantly evolving
- they rarely have detailed configuration controlled requirements
- they typically have (constantly changing) goals of providing shareholder value and customer satisfaction
- they exist in a context (or environment) that is ill-defined and constantly changing

The enterprise systems engineer must consider and account for these factors in their processes and methods.

Both product and service systems require an enterprise system context to create them and an enterprise to use the product system and deliver services, either internally to the enterprise or externally to a broader community. Thus, the three types of engineered system contexts are linked in all instances, regardless of which type of system the developers consider as the object of the development effort that is delivered to the customer.

References

Works Cited

- Flood, R.L. 1987. "Some theoretical considerations of mathematical modeling." In *Problems of Constancy and Change* (Proceedings of 31st Conference of the International Society for General Systems Research, Budapest), Volume 1, pp. 354 - 360.
- Flood, R.L. and E.R. Carson. 1993. *Dealing with Complexity: An Introduction to the Theory and Application of Systems Science*, 2nd ed. New York, NY, USA: Plenum Press.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Spohrer, J. 2008. "Service science, management, engineering, and design (SSMED): An emerging discipline-outline & references." *International Journal of Information Systems in the Service Sector*, vol. 1, no. 3 (May).

Primary References

- Chang, C.M., 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. Hoboken, NJ, USA: John Wiley and Sons.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press.
- Rouse, W.B. 2005. "Enterprises as systems: Essential challenges and enterprise transformation". *Systems Engineering*, vol. 8, no. 2, pp. 138-50.
- Tien, J.M. and D. Berg. 2003. "A case for service systems engineering", *Journal of Systems Science and Systems Engineering*, vol. 12, no. 1, pp. 13-38.

Additional References

- ANSI/EIA. 2003. *Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA). ANSI/EIA 632-1998.
- Bernus, P., L. Nemes, and G. Schmidt (eds.). 2003. *Handbook on Enterprise Architecture*. Heidelberg, Germany: Springer.
- Chang, C.M., 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. Hoboken, NJ, USA: John Wiley and Sons.
- DeRosa, J. K. 2006. "An Enterprise Systems Engineering Model." Proceedings of the 16th Annual International Council on Systems Engineering, Orlando, FL, USA, 9-13 July 2006.
- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press.
- Joannou, P. 2007. "Enterprise, systems, and software—The need for integration." *IEEE Computer*, vol. 40, no. 5, May, pp. 103-105.
- Katzan, H. 2008. *Service Science*. Bloomington, IN, USA: iUniverse Books.
- Maglio P., S. Srinivasan, J.T. Kreulen, and J. Spohrer. 2006. "Service Systems, Service Scientists, SSME, and Innovation." *Communications of the ACM*. 49(7) (July).

Martin J.N. 1997. *Systems Engineering Guidebook*. Boca Raton, FL, USA: CRC Press.

Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press.

Rouse, W.B. 2009. "Engineering the enterprise as a system," *Handbook of Systems Engineering and Management*, 2nd ed. A.P. Sage and W.B. Rouse (eds.). New York, NY, USA: Wiley and Sons.

Tien, J.M. and D. Berg. 2003. "A Case for Service Systems Engineering," *Journal of Systems Science and Systems Engineering*, vol. 12, no. 1, pp. 13-38.

Valerdi, R. and D.J. Nightingale. 2011. "An introduction to the journal of enterprise transformation," *Journal of Enterprise Transformation*, vol. 1, no. 1, pp. 1-6.

Identifying and Understanding Problems and Opportunities

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, Janet Singer, Duane Hybertson, and Bud Lawson

This topic is part of the Systems Approach Applied to Engineered Systems knowledge area (KA). It describes knowledge related to the identification and exploration of problems or opportunities in detail. The problem situations described by the activities in this topic may form a starting point for Synthesizing Possible Solutions. Any of the activities described below may also need to be considered concurrently with other activities in the systems approach at a particular point in the life of a system-of-interest (SoI).



The activities described below should be considered in the context of the Overview of the Systems Approach topic at the start of this KA. The final topic in this knowledge area, Applying the Systems Approach, considers the dynamic aspects of how these activities are used as part of the systems approach and how this relates in detail to elements of systems engineering (SE).

The phrase "problem or opportunity" used herein recognizes that the "problem" is not always a negative situation and can also be a positive opportunity to improve a situation.

Introduction

According to Jenkins (1969), the first step in the systems approach is "the recognition and formulation of the problem." The systems approach described in the Guide to the SE Body of Knowledge (SEBoK) is predominantly a hard system approach. The analysis, synthesis, and proving parts of the approach assume a problem or opportunity has been identified and agreed upon and that a "new" engineered system solution is needed.

However, the systems approach does not have to apply to the development and use of a newly designed and built technical solution. Abstract or experimental solutions to potential problems might be explored to help achieve agreement on a problem context. Solutions may involve reorganizing existing systems of systems (SoS) contexts or the modification or re-use of existing products and services. The problem and opportunity parts of the approach overlap with soft system approaches. This is discussed in more detail below.

One thing that must be considered in relation to system complexity is that the opportunity situation may be difficult to fully understand; therefore, system solutions may not solve the problem the first time, but is still useful in increasing the understanding of both problem issues and what to try next to work toward a solution.

Hence, problem exploration and identification is often not a one-time process that specifies the problem, but is used in combination with solution synthesis and analysis to progress toward a more complete understanding of problems and solutions over time (see Applying the Systems Approach for a more complete discussion of the dynamics of this aspect of the approach).

Problem Exploration

Soft system thinking does not look for "the problem," but considers a problematic situation. Forming systems views of this situation can help stakeholders better understand each other's viewpoints and provide a starting point for directed intervention in the current system context. If a full soft systems intervention is undertaken, such as a soft systems methodology (SSM) (Checkland 1999), it will not include formal analysis, synthesis, and proving. However, the SSM method was originally based on hard methodologies, particularly one presented by Jenkins (1969). It follows the basic principles of a systems approach: "analyzing" conceptual models of shared understanding, "synthesizing" intervention strategies, and "proving" improvements in the problematic situation.

Often, the distinction between hard and soft methods is not as clear cut as the theory might suggest. Checkland himself has been involved in applications of SSM as part of the development of information system design (Checkland and Holwell 1998). It is now agreed upon by many that while there is a role for a "pure soft system" approach, the service and enterprise problems now being tackled can only be dealt with successfully by a combination of soft problematic models and hard system solutions. Mingers and White (2009) give a number of relevant examples of this. In particular, they reference "Process and Content: Two Ways of Using SSM" (Checkland and Winters 2006). It is likely in the future that engineered system problems will be stated, solved, and used as part of a predominately soft intervention, which will place pressure on the speed of development needed in the solution space. This is discussed more fully in the topic Life Cycle Models.

The critical systems thinking and multi-methodology approaches (Jackson 1985) take this further by advocating a "pick and mix" approach, in which the most appropriate models and techniques are chosen to fit the problem rather than following a single methodology (Mingers and Gill 1997). Thus, even if the hard problem identification approach described below is used, some use of the soft system techniques (such as rich pictures, root definitions, or conceptual models) should be considered within it.

Problem Identification

Hard system thinking is based on the premise that a problem exists and can be stated by one or more stakeholders in an objective way. This does not mean that hard systems approaches start with a defined problem. Exploring the potential problem with key stakeholders is still an important part of the approach.

According to Blanchard and Fabrycky (2006, 55-56), defining a problem is sometimes the most important and difficult step. In short, a system cannot be defined unless it is possible to clearly describe what it is supposed to accomplish.

According to Edson (2008, 26-29), there are three kinds of questions that need to be asked to ensure we fully understand a problem situation. First, how difficult or well understood is the problem? The answer to this question will help define the tractability of the problem. Problems can be "tame," "regular," or "wicked":

- For tame problems, the solution may be well-defined and obvious.
- Regular problems are those that are encountered on a regular basis. Their solutions may not be obvious, thus serious attention should be given to every aspect of them.

- Wicked problems (Rittel and Webber 1973) cannot be fully solved, or perhaps even fully defined. Additionally, with wicked problems, it is not possible to understand the full effect of applying systems to the problem.

Next, who or what is impacted? There may be elements of the situation that are causing the problem, elements that are impacted by the problem, and elements that are just in the loop. Beyond these factors, what is the environment and what are the external factors that affect the problem? In examining these aspects, the tools and methods of systems thinking can be productively applied.

Finally, what are the various viewpoints of the problem? Does everyone think it is a problem? Perhaps there are conflicting viewpoints. All these viewpoints need to be defined. Persons affected by the system, who stand to benefit from the system, or can be harmed by the system, are called stakeholders. Wasson (2006, 42-45) provides a comprehensive list of stakeholder types. The use of soft systems models, as discussed above, can play an important part in this. Describing a problem using situation views can be useful when considering these issues, even if a single problem perspective is selected for further consideration.

Operations research is a hard systems method which concentrates on solving problem situations by deploying known solutions. The problem analysis step of a typical approach asks questions about the limitation and cost of the current system to identify efficiency improvements that need to be made (Flood and Carson 1993).

Traditional SE methods tend to focus more on describing an abstract model of the problem, which is then used to develop a solution that will produce the benefits stakeholders expect to see (Jenkins 1969). The expectation is often that a new solution must be created, although this need not be the case. Jenkins suggests that SE is just as applicable to a redesign of existing systems. A clear understanding of stakeholder expectations in this regard should produce a better understanding of part of the problem. Do stakeholders expect a new solution or modifications to their existing solutions, or are they genuinely open to solution alternatives which consider the pros and cons of either? Such expectations will influence suggestions of solution alternatives, as discussed in the Synthesizing Possible Solutions article.

An important factor in defining the desired stakeholder outcomes, benefits, and constraints is the operational environment, or scenario, in which the problem or opportunity exists. Armstrong (2009, 1030) suggests two scenarios: the first is the descriptive scenario, or the situation as it exists now, and the second is the normative scenario, or the situation as it may exist sometime in the future.

All of these aspects of problem understanding can be related to the concept of a system context.

Problem Context

The Engineered System Context topic identifies a way by which a complex system situation can be resolved around a system-of-interest (SoI). The initial identification of a "problem context" can be considered as the outcome of this part of the systems approach.

The systems approach should not consider only soft or hard situations. More appropriately, a problem or opportunity should be explored using aspects of both. In general, the application of the systems approach with a focus on engineered system contexts will lead to hard system contexts in which an identified SoI and required outcome can be defined.

An initial description of the wider SoI and environment serves as the problem or opportunity problem scope. Desired stakeholder benefits are expressed as outcomes in the wider system and some initial expression of what the SoI is intended for may be identified. Jenkins (1969) defines a problem formulation approach where one:

- states the aim of the SoI
 - defines the wider SoI
 - defines the objectives of the wider SoI
 - defines the objectives of the system
 - defines economic, informational, and other conditions
-

In a hard system problem context, a description of a logical or ideal system solution may be included. This ideal system cannot be implemented directly, but describes the properties required of any realizable system solution.

To support this problem or opportunity description, a soft context view of the SoI will help ensure wider stakeholder concerns are considered. If a soft system context has been defined, it may include a conceptual model (Checkland 1999) which describes the logical elements of a system that resolve the problem situation and how they are perceived by different stakeholders. Unlike the hard system view, this does not describe the ideal solution, but provides an alternative view on how aspects of any solution would be viewed by potential stakeholders.

In problem contexts with a strong coercive dimension, the problem context should include an identification of the relative power and the importance of stakeholders.

The problem context should include some boundaries on the cost, time to deployment, time in use, and operational effectiveness needed by stakeholders. In general, both the full problem context and an agreed version of the problem to be tackled next are described. (See Applying the Systems Approach.)

References

Works Cited

- Armstrong, Jr., J.E., 2009. "Issue formulation." in A.P. Sage and W.B. Rouse (eds.). *Handbook of Systems Engineering and Management*, 2nd edition. Hoboken, NJ, USA: Wiley.
- Blanchard, B. and W.J. Fabrycky. 2006. *Systems Engineering and Analysis*. Upper Saddle River, NJ, USA: Prentice Hall.
- Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: Wiley.
- Checkland, P. and S. Holwell. 1998. *Information, Systems and Information Systems: Making Sense of the Field*. Hoboken, NJ, USA: Wiley.
- Checkland, P. and M. Winter. 2006. "Process and content: Two ways of using SSM," *Journal of Operational Research Society*, vol. 57, no. 12, pp. 1435-1441.
- Edson, R. 2008. *Systems Thinking. Applied. A Primer*. Arlington, VA, USA: Applied Systems Thinking (ASysT) Institute, Analytic Services Inc.
- Flood, R. L. and E.R. Carson 1993. *Dealing with Complexity: An Introduction to the Theory and Application of Systems Science*, 2nd ed. New York, NY, USA: Plenum Press.
- Jackson, M. 1985. "Social systems theory and practice: The need for a critical approach," *International Journal of General Systems*, vol. 10, pp. 135-151.
- Jenkins, G.M. 1969. "The systems approach," *The Journal of Systems Engineering*, vol. 1, no. (1).
- Mingers, J. and A. Gill. 1997. *Multimethodology: Theory and Practice of Combining Management Science Methodologies*. Chichester, UK: Wiley.
- Mingers, J. and L. White. 2009. *A Review of Recent Contributions of Systems Thinking to Operational Research and Management Science*, Working Paper 197. Canterbury, UK: Kent Business School.
- Rittel, H. and M. Webber. 1973. "Dilemmas in a general theory of planning," *Policy Sciences*, vol. 4, pp. 155-169.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: Wiley.

Primary References

- Blanchard, B. & W.J. Fabrycky. 2006. *Systems Engineering and Analysis*. Upper Saddle River, NJ, USA: Prentice Hall.
- Edson, R. 2008. *Systems Thinking. Applied. A Primer*. Arlington, VA, USA: Applied Systems Thinking (ASysT) Institute, Analytic Services Inc.

Additional References

None

Synthesizing Possible Solutions

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Scott Jackson, Janet Singer, and Duane Hybertson

This topic is part of the Systems Approach Applied to Engineered Systems knowledge area (KA). It describes knowledge related to the synthesis of possible solution options in response to the problem situations described by activities from Identifying and Understanding Problems and Opportunities topic. The solution options proposed by the synthesis activities will form the starting point for the Analysis and Selection between Alternative Solutions. Any of the activities described below may also need to be considered concurrently with other activities in the systems approach at a particular point in the life of a system-of-interest (SoI).



The activities described below should be considered in the context of the Overview of the Systems Approach topic at the start of this KA. The final topic in this KA, Applying the Systems Approach, considers the dynamic aspects of how these activities are used as part of the systems approach and how this relates in detail to elements of systems engineering (SE).

Synthesis Overview

System synthesis is an activity within the systems approach that is used to describe one or more system solutions based upon a problem context for the life cycle of the system to:

- Define options for a SoI with the required properties and behavior for an identified problem or opportunity context.
- Provide solution options relevant to the SoI in its intended environment, such that the options can be assessed to be potentially realizable within a prescribed time limit, cost, and risk described in the problem context.
- Assess the properties and behavior of each candidate solution in its wider system context.

The iterative activity of system synthesis develops possible solutions and may make some overall judgment regarding the feasibility of said solutions. The detailed judgment on whether a solution is suitable for a given iteration of the systems approach is made using the Analysis and Selection between Alternative Solutions activities.

Essential to synthesis is the concept of holism(Hitchins 2009), which states that a system must be considered as a whole and not simply as a collection of its elements. The holism of any potential solution system requires that the behavior of the whole be determined by addressing a system within an intended environment and not simply the

accumulation of the properties of the elements. The latter process is known as reductionism and is the opposite of holism, which Hitchens (2009, 60) describes as the notion that “the properties, capabilities, and behavior of a system derive from its parts, from interactions between those parts, and from interactions with other systems.”

When the system is considered as a whole, properties called emergent properties often appear (see Emergence and Complexity). These properties are often difficult to predict from the properties of the elements alone. They must be evaluated within the systems approach to determine the complete set of performance levels of the system. According to Jackson (2010), these properties can be considered in the design of a system, but to do so, an iterative approach is required.

In complex systems, individual elements will adapt to the behavior of the other elements and to the system as a whole. The entire collection of elements will behave as an organic whole. Therefore, the entire synthesis activity, particularly in complex systems, must itself be adaptive.

Hence, synthesis is often not a one-time process of solution design, but is used in combination with problem understanding and solution analysis to progress towards a more complete understanding of problems and solutions over time (see Applying the Systems Approach topic for a more complete discussion of the dynamics of this aspect of the approach).

Problem or Opportunity Context

System synthesis needs the problem or opportunity that the system is intended to address to have already been identified and described and for non-trivial systems, the problem or opportunity needs to be identified and understood concurrently with solution synthesis activities.

As discussed in Identifying and Understanding Problems and Opportunities, the systems approach should not consider strictly soft or hard situations. In general, the application of the systems approach, with a focus on engineered system contexts, will lead to hard system contexts in which an identified SoI and required outcome are defined. Even in these cases, a soft context view of the SoI context will help ensure wider stakeholder concerns are considered.

The problem context should include some boundaries on the cost, time to deployment, time in use, and operational effectiveness needed by stakeholders. In general, the goal is not to synthesize the perfect solution to a problem, but rather to find the best available solution for the agreed version of the problem.

Synthesis Activities

The following activities provide an outline for defining the SoI: grouping of elements, identification of the interactions among the elements, identification of interfaces between elements, identification of external interfaces to the SoI boundary and common sub-elements within the SoI boundary.

The activities of systems synthesis are built on the idea of a balanced reduction vs. holism approach as discussed in What is Systems Thinking? topic. It is necessary to divide system elements and functions to create a description of the SoI which is realizable, either through combinations of available elements or through the design and construction of new elements. However, if the system is simply decomposed into smaller and smaller elements, the holistic nature of systems will make it more and more difficult to predict the function and behavior of the whole. Thus, synthesis progresses through activities that divide, group, and allocate elements, and then assesses the complete system's properties in context relevant to the user need the SoI will fulfill. Hence, synthesis occurs over the entire life cycle of the system as the system and its environment change.

Identification of the Boundary of a System

Establishing the boundary of a system is essential to synthesis, the determination of the system's interaction with its environment and with other systems, and the extent of the SoI. Buede (2009, 1102) provides a comprehensive discussion of the importance of, and methods of, defining the boundary of a system in a SE context.

Identification of the Functions of the System

The function of a system at a given level of abstraction is critical to synthesis since the primary goal of the synthesis activity is to propose realizable system descriptions which can provide a given function. The function of a system is distinct from its behavior as it describes what the system can be used for or is asked to do in a larger system context.

Buede (2009, 1091-1126) provides a comprehensive description of functional analysis in a SE context.

Identification of the Elements of a System

System synthesis calls for the identification of the elements of a system. Typical elements of an Engineered System Context may be physical, conceptual, or processes. Physical elements may be hardware, software, or humans. Conceptual elements may be ideas, plans, concepts, or hypotheses. Processes may be mental, mental-motor (writing, drawing, etc.), mechanical, or electronic (Blanchard and Fabrycky 2006, 7).

In addition to the elements of the system under consideration (i.e., a SoI), ISO 15288 (ISO/IEC/IEEE 15288 2015) also calls for the identification of the enabling systems. These are systems (or services) utilized at various stages in the life cycle, e.g., development, utilization or support stages, to facilitate the SoI in achieving its objectives.

Today's systems often include existing elements. It is rare to find a true "greenfield" system, in which the developers can specify and implement all new elements from scratch. "Brownfield" systems, wherein legacy elements constrain the system structure, capabilities, technology choices, and other aspects of implementation, are much more typical (Boehm 2009).

Division of System Elements

System synthesis may require elements to be divided into smaller elements. The division of elements into smaller elements allows the systems to be grouped and leads to the SE concept of physical architecture, as described by Levin (2009, 493-495). Each layer of division leads to another layer of the hierarchical view of a system. As Levin points out, there are many ways to depict the physical architecture, including the use of wiring diagrams, block diagrams, etc. All of these views depend on arranging the elements and dividing them into smaller elements. According to the principle of recursion, these decomposed elements are either terminal elements, or are decomposable. The hierarchical view does not imply a top-down analytical approach to defining a system. It is simply a view. In the systems approach, levels of the hierarchy are defined and considered recursively with one level forming the context for the next.

Grouping of System Elements

System synthesis may require that elements be grouped. This leads to the identification of the sub-systems that are essential to the definition of a system. Synthesis determines how a system may be partitioned and how each sub-system fits and functions within the whole system. The largest group is the SoI, also called the relevant system by Checkland (1999, 166). According to Hitchins, some of the properties of a SoI are as follows: the SoI is open and dynamic, the SoI interacts with other systems, and the SoI contains sub-systems (Hitchins 2009, 61). The SoI is brought together through the concept of synthesis.

Identification of the Interactions among System Elements

System synthesis may require the identification of the interactions among system elements. These interactions lead to the SE process of interface analysis. Integral to this aspect is the principle of interactions. Interactions occur both with other system elements as well as with external elements and the environment. In a systems approach, interfaces have both a technical and managerial importance. Managerial aspects include the contracts between interfacing organizations. Technical aspects include the properties of the physical and functional interfaces. Browning provides a list of desirable characteristics of both technical and managerial interface characteristics (Browning 2009, 1418-1419).

System synthesis will include activities to understand the properties of system elements, the structure of proposed system solutions, and the resultant behavior of the composed system. A number of system concepts for describing system behavior are discussed in the Concepts of Systems Thinking topic. It should be noted that in order to fully understand a system's behavior, we must consider the full range of environments in which it might be placed and its allowable state in each. According to Page, in complex systems, the individual elements of the system are characterized by properties which enhance the systems as a whole, such as their adaptability (Page 2009).

Defining the System-of-Interest

Flood and Carson provide two ways to identify system boundaries: a bottom-up, or **structural approach**, which starts with significant system elements and builds out, and a top down, or **behavioral approach**, in which major systems needed to fulfill a goal are identified and then the work flows downward (Flood and Carson 1993). They identify a number of rules proposed by Beishon (1980) and Jones (1982) to help in the selection of the best approach.

In either case, the ways in which system elements are refined, grouped, and allocated must be driven towards the synthesis of a realizable system solution description. A realizable solution must consider elements that are either already available, can be created from existing system elements, or are themselves described as system contexts which will need to be synthesized at a future point. In the third case, it is one of the outcomes of the Analysis and Selection between Alternative Solutions activities that is used to assess the risk that a given element may not be able to be synthesized in the required time limit or cost budget.

A top down approach might start with a system boundary and an overall description of system functions. Through the repeated application of element identification, division, grouping, and allocation of functions, a complete description of the elements needed for the SoI can be defined. In this case, the choice of system elements and allocation of functions may be guided by pre-defined ways of solving a given problem or by identified system patterns; both can support as well as insert bias into the synthesis. For example, one might start with the need to provide energy to a new housing project and propose solution options based around connections to an existing power grid, local power generators, renewable energy sources, increased energy efficiency, etc.

The iterative nature of analysis also reflects the need to change the solution as the life cycle progresses and changes the system's environment; thereby, possibly changing what a "best" solution is.

A bottom up approach starts with major elements and interactions. Again, division, grouping, and identification allows for the construction of a full system description that is capable of providing all the necessary functions, at which point the final SoI boundary can be set. In this case, the choice of system elements and groupings will be driven by the goal of ensuring that the major system elements can be formed together into a viable system whole. For example, there may be a need to replace an existing delivery vehicle and produce solution options that consider vehicle ownership/leasing, driver training, petrol, diesel or electric fuel, etc.

The systems approach aspect of synthesis leads to SE terms such as "design" and "development." Wasson describes synthesis from a SE point of view (Wasson 2006, 390-690). White provides a comprehensive discussion of methods of achieving design synthesis (White 2009, 512-515). The systems approach treats synthesis at the abstract level while the SE process definitions provide the concrete steps.

The SoI brings together elements, sub-systems and systems through the concept of synthesis to identify a solution option.

Synthesis of possible solutions may result in the development of artifacts documenting the synthesis itself and provide the basis for analysis and selection between alternative solutions. These artifacts are dynamic and will change as the SoI changes its environment throughout the system life cycle.

References

Works Cited

- Beishon, J. 1980. *Systems Organisations: The Management of Complexity*. Milton Keynes, UK: Open University Press.
- Blanchard, B. and W.J. Fabrycky. 2006. *Systems Engineering and Analysis*. Upper Saddle River, NJ, USA: Prentice Hall.
- Boehm, B. 2009. "Applying the Incremental Commitment Model to Brownfield System Development". Proceedings of the 7th Annual Conference on Systems Engineering Research (CSER), Loughborough, UK.
- Browning, T.R. 2009. "Using the design structure matrix to design program organizations," in Sage, A.P. and W.B. Rouse (eds.). *Handbook of Systems Engineering and Management*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Buede, D.M. 2009. "Functional analysis," in Sage, A.P. and W.B. Rouse (eds.). *Handbook of Systems Engineering and Management*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Checkand, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.
- Flood, R.L. and E.R. Carson. 1993. *Dealing with Complexity: An Introduction to the Theory and Application of Systems Science*, 2nd ed. New York, NY, USA: Plenum Press.
- Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4, December, pp. 59-63.
- INCOSE. 1998. "INCOSE SE Terms Glossary." INCOSE Concepts and Terms WG (eds.). Seattle, WA, USA: International Council on Systems Engineering.
- Jackson, S., D. Hitchins, and H. Eisner. 2010. "What is the systems approach?" *INCOSE Insight*, vol. 13, no. 1, April, pp. 41-43.
- Jones, L. 1982. "Defining system boundaries in practice: Some proposals and guidelines," *Journal of Applied Systems Analysis*, vol. 9, pp. 41-55.
- Levin, A.H. 2009. "System architectures," in Sage, A.P. and W.B. Rouse (eds.). *Handbook of Systems Engineering and Management*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Page, S.E. 2009. "Understanding Complexity." The Great Courses. Chantilly, VA, USA: The Teaching Company.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.
- White, Jr., K.P. 2009. "Systems design," in Sage, A.P. and W.B. Rouse (eds.). *Handbook of Systems Engineering and Management*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.
-

Primary References

Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4, December, pp. 59-63.

ISO/IEC/IEEE. 2015. Systems and software engineering -- System life cycle processes. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions / Institute of Electrical and Electronics Engineer. ISO/IEC/IEEE 15288:2015.

Jackson, S., D. Hitchins and H. Eisner. 2010. "What is the systems approach?" *INCOSE Insight*, vol. 13, no. 1, April, pp. 41-43.

Additional References

None

Analysis and Selection between Alternative Solutions

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson

This topic is part of the Systems Approach Applied to Engineered Systems knowledge area (KA). It describes knowledge related to the analysis and selection of a preferred solution from the possible options, which may have been proposed by Synthesizing Possible Solutions. Selected solution options may form the starting point for Implementing and Proving a Solution.

Any of the activities described below may also need to be considered concurrently with other activities in the systems approach at a particular point in the life of a system-of-interest (SoI).

The activities described below should be considered in the context of the Overview of the Systems Approach topic at the start of this KA. The final topic in this KA, Applying the Systems Approach, considers the dynamic aspects of how these activities are used as part of the systems approach and how this relates in detail to elements of systems engineering (SE).



System Analysis

System analysis is an activity in the systems approach that evaluates one or more system artifacts created during the activities involved in Synthesizing Possible Solutions, such as:

- Defining assessment criteria based on the required properties and behavior of an identified problem or opportunity system situation.
- Accessing the properties and behavior of each candidate solution in comparison to the criteria.
- Comparing the assessments of the candidate solutions and identification of any that could resolve the problem or exploit the opportunities, along with the selection of candidates that should be further explored.

As discussed in Synthesizing Possible Solutions topic, the problem context for an engineered system will include a logical or ideal system solution description. It is assumed that the solution that "best" matches the ideal one will be

the most acceptable solution to the stakeholders. Note, as discussed below, the “best” solution should include an understanding of cost and risk, as well as effectiveness. The problem context may include a soft system conceptual model describing the logical elements of a system to resolve the problem situation and how these are perceived by different stakeholders (Checkland 1999). This soft context view will provide additional criteria for the analysis process, which may become the critical issue in selecting between two equally effective solution alternatives.

Hence, analysis is often not a one-time process of solution selection; rather, it is used in combination with problem understanding and solution synthesis to progress towards a more complete understanding of problems and solutions over time (see Applying the Systems Approach topic for a more complete discussion of the dynamics of this aspect of the approach).

Effectiveness Analysis

Effectiveness studies use the problem or opportunity system context as a starting point.

The effectiveness of a synthesized system solution will include performance criteria associated with both the system’s primary and enabling functions. These are derived from the system’s purpose, in order to enable the realization of stakeholder needs in one or more, wider system contexts.

For a product system, there are a set of generic non-functional qualities that are associated with different types of solution patterns or technology, e.g., safety, security, reliability, maintainability, usability, etc. These criteria are often explicitly stated as parts of the domain knowledge of related technical disciplines in technology domains.

For a service system or enterprise system, the criteria will be more directly linked to the identified user needs or enterprise goals. Typical qualities for such systems include agility, resilience, flexibility, upgradeability, etc.

In addition to assessments of the absolute effectiveness of a given solution system, systems engineers must also be able to combine effectiveness with the limitations of cost and timescales included in the problem context. In general, the role of system analysis is to identify the proposed solutions which can provide some effectiveness within the cost and time allocated to any given iteration of the systems approach (see Applying the Systems Approach for details). If none of the solutions can deliver an effectiveness level that justifies the proposed investment, then it is necessary to return to the original framing of the problem. If at least one solution is assessed as sufficiently effective, then a choice between solutions can be proposed.

Trade-Off Studies

In the context of the definition of a system, a trade-off study consists of comparing the characteristics of each candidate system element to those of each candidate system architecture in order to determine the solution that globally balances the assessment criteria in the best way. The various characteristics analyzed are gathered in cost analysis, technical risks analysis, and effectiveness analysis (NASA 2007). To accomplish a trade off study, there are a variety of methods, often supported by tooling. Each class of analysis is the subject of the following topics:

- Assessment criteria are used to classify the various candidate solutions. They are either absolute or relative. For example, the maximum cost per unit produced is c\$, cost reduction shall be x%, effectiveness improvement is y%, and risk mitigation is z%.
- **Boundaries** identify and limit the characteristics or criteria to be taken into account at the time of analysis (e.g., the kind of costs to be taken into account, acceptable technical risks, and the type and level of effectiveness).
- **Scales** are used to quantify the characteristics, properties, and/or criteria and to make comparisons. Their definition requires knowledge of the highest and lowest limits, as well as the type of evolution of the characteristic (linear, logarithmic, etc.).
- An assessment score is assigned to a characteristic or criterion for each candidate solution. The goal of the trade-off study is to succeed in quantifying the three variables (and their decomposition in sub-variables) of cost, risk, and effectiveness for each candidate solution. This operation is generally complex and requires the use of

models.

- The **optimization** of the characteristics or properties improves the scoring of interesting solutions.

A decision-making process is not an accurate science; ergo, trade-off studies have limits. The following concerns should be taken into account:

- Subjective Criteria – personal bias of the analyst; for example, if the component has to be beautiful, what constitutes a “beautiful” component?
- Uncertain Data – for example, inflation has to be taken into account to estimate the cost of maintenance during the complete life cycle of a system; how can a systems engineer predict the evolution of inflation over the next five years?
- Sensitivity Analysis – A global assessment score that is designated to every candidate solution is not absolute; thus, it is recommended that a robust selection is gathered by performing a sensitivity analysis that considers small variations of assessment criteria values (weights). The selection is robust if the variations do not change the order of scores.

A thorough trade-off study specifies the assumptions, variables, and confidence intervals of the results.

Systems Principles of System Analysis

From the discussions above, the following general principles of systems analysis can be defined:

- Systems analysis is an iterative activity consisting of trade studies made between various solution options from the systems synthesis activity.
- Systems analysis uses assessment criteria based upon a problem or opportunity system description.
 - These criteria will be based around an ideal system description that assumes a hard system problem context can be defined.
 - The criteria must consider required system behavior and properties of the complete solution in all of the possible wider system contexts and environments.
 - Trade studies require equal consideration to the primary system and the enabling system working as a single system to address the user need. These studies need to consider system requirements for Key Performance Parameters (KPPs), systems safety, security, and affordability across the entire life cycle.
 - This ideal system description may be supported by soft system descriptions from which additional “soft” criteria may be defined (e.g., a stakeholder preference for or against certain kinds of solutions and relevant social, political, or cultural conventions to be considered in the likely solution environment, etc.).
- At a minimum, the assessment criteria should include the constraints on cost and time scales acceptable to stakeholders.
- Trade studies provide a mechanism for conducting analysis of alternative solutions.
 - A trade study should consider a “system of assessment criteria,” designating special attention to the limitations and dependencies between individual criteria.
 - Trade studies need to deal with both objective and subjective criteria. Care must be taken to assess the sensitivity of the overall assessment to particular criteria.

References

Works Cited

Checkland, P.B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

NASA. 2007. *Systems Engineering Handbook*, Revision 1. Washington, D.C., USA: National Aeronautics and Space Administration (NASA). NASA/SP-2007-6105.

Primary References

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions/Institute of Electrical and Electronics Engineer. ISO/IEC/IEEE 15288:2015.

Jackson, S., D. Hitchins and H. Eisner. 2010. "What is the systems approach?" *INCOSE Insight*, vol. 13, no. 1, April, pp. 41-43.

Additional References

None.

Implementing and Proving a Solution

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson

This topic is part of the Systems Approach Applied to Engineered Systems knowledge area (KA). It describes knowledge related to the implementation and proving of a preferred solution that may have been selected by activities described in the Analysis and Selection between Alternative Solutions topic. The activities that apply to an implemented solution during its operational

life are described in Deploying, Using, and Sustaining Systems to Solve Problems topic, and how systems fit into commercial and acquisition relationships is discussed in the Introduction to System Fundamentals topic. Any of the activities described below may also need to be considered concurrently with other activities in the systems approach at a particular point in the life of a system-of-interest (SoI).

The activities described below should be considered in the context of the Overview of the Systems Approach topic at the start of this KA. The final topic in this KA, Applying the Systems Approach, considers the dynamic aspects of how these activities are used as part of the systems approach and how this relates in detail to elements of systems engineering (SE).



Proving the System Overview

This topic covers both the sub-topics of verification and validation.

Verification

Verification is the determination that each element of the system meets the requirements of a documented specification (see principle of elements). Verification is performed at each level of the system hierarchy. In the systems approach, this topic pertains to the more abstract level of providing evidence that the system will accomplish what it was meant to do. In SE, this topic pertains to providing quantitative evidence from tests and other methods for verifying the performance of the system.

Validation

Validation is the determination that the entire system meets the needs of the stakeholders. Validation only occurs at the top level of the system hierarchy. In the systems approach, this topic pertains to the more abstract level of ensuring the system meets the needs of the stakeholders. In SE, this topic pertains to the detailed demonstrations and other methods that are used to promote stakeholder satisfaction.

In a SE context, Wasson provides a comprehensive guide to the methods of both system verification and system validation (Wasson 2006, 691-709).

References

Works Cited

Wasson, C. S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.

Primary References

Jackson, S., D. Hitchins and H. Eisner. 2010. "What is the systems approach?" *INCOSE Insight*, vol. 13, no. 1, April, pp. 41-43.

Additional References

MITRE. 2012. "Verification and validation," in *Systems Engineering Guide*. Available at: https://mitre.org/work/systems_engineering/guide/se_lifecycle_building_blocks/test_evaluation/verification_validation.html. Accessed September 11, 2012.

Wasson, C. S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.

Deploying, Using, and Sustaining Systems to Solve Problems

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, Janet Singer, and Duane Hybertson

This topic is part of the Systems Approach Applied to Engineered Systems knowledge area (KA). It describes knowledge related to the deployment, sustainment, and use of a solution that may have been developed through the activities described in the Implementing and Proving a Solution topic. Discussion of how a deployed system fits into commercial and acquisition

relationships is present in Introduction to System Fundamentals. Any of the activities described below may also need to be considered concurrently with other activities in the systems approach at a particular point in the life of a system-of-interest (SoI).

The activities described below should be considered in the context of the Overview of the Systems Approach topic at the start of this KA. The final topic in this KA, Applying the Systems Approach, considers the dynamic aspects of how these activities are used as part of the systems approach and how this relates in detail to elements of systems engineering (SE).



Introduction

Part 3, Systems Engineering and Management, of the Guide to the SE Body of Knowledge (SEBoK) provides two additional KAs that address the engineering aspects of these steps of the systems approach. KAs Product and Service Life Management and System Deployment and Use in Part 3 explain the SE aspects of deployment, operation, maintenance, logistics, service life extension, updates, upgrades, disposal and the retirement of systems.

A systems approach considers the total system and the total life cycle of the system. This includes all aspects of the system and the system throughout its life until the day users dispose of the system and the external enterprises complete the handling of the disposed system products. Creation of the system is rarely the step that solves the stakeholders' problems. It is the use of the system solution that solves the problem. From this perspective the deployment, use and sustainment of the system are important concepts that must be a part of the systems approach.

Engineered systems are eventually owned by an individual, team, or enterprise. Those who own the system during development may not be the ones who own the system when it is in operation. Moreover, the owners may not be the users; e.g., service systems may be used by the general public but owned by a specific business that offers the service. The transition of a system from development to operations is often itself a complex task, involving such activities as training those who will operate the system, taking legal actions to complete the transfer, and establishing logistical arrangements so that the operators can keep the system running once the transition is completed.

A complete systems approach must also consider the many enterprises involved in the system from initial conception through the completion of the disposal process. These enterprises are all stakeholders with requirements, which all have interfaces that must be considered as part of a total systems approach.

There is very little in the literature pertaining to the application of the systems approach to these phases of the life cycle. However, a basic premise of this KA is that the systems approach pertains to all phases of a system's life cycle. Hence, to properly build systems to solve problems or for other uses, it can be inferred that the systems

approach pertains to the deployment, use, and the sustainment of the systems. Many of the available references in this topic area are from SE literature rather than from literature associated with the systems approach; the reader should also see Part 3 of the SEBoK, Systems Engineering and Management.

Deployment: The Transition from Development to Operation

Transferring custody of the SoI and responsibility for its support from one organization to another occurs during deployment and is often called transition (INCOSE 2011). Transition of a product system includes the integration of the system into the acquiring organization's infrastructure. Deployment and transition involves the activity of moving the system from the development to the operational location(s), along with the support systems necessary to accomplish the relocation.

Transition includes the initial installation of a system and the determination that it is compatible with the wider system and does not cause any significant wider system issues. This process of acceptance and release for use varies between domains and across businesses and enterprises, and can be thought of as an initial assessment of the system's effectiveness (Hitchins 2007). Generally, transition may be considered as having two parts: 1.) ensuring that the new system interoperability with the systems around it and 2.) ensuring the resulting system is safe and possesses other critical operational properties.

It is particularly important to consider emergent properties when a new system is added to the existing organization's system of systems (SoS) network, as well as the complexity of the organization into which the new system is transitioned (see also Emergence and Complexity). The more complex the receiving organization is, the more challenging the transition will be, and the greater the likelihood of unintended interactions and consequences from the new system's insertion. Dealing with the consequences of this complexity starts in transition and continues into operation, maintenance, and disposal.

Transition of a service system is often performed in two stages. First, the service system infrastructure is accepted and released. Second, each realization of the service is accepted and released. There can be significant problems during the second stage if the required responsiveness of the service does not leave sufficient time to ensure that the service meets necessary functional and quality attributes, including interoperability, safety, and security. (See Service Systems Engineering.)

Transition and deployment of a system may introduce unique requirements that are not necessary for operation or use. These requirements can influence the design of the system; therefore, must be considered during the initial requirements and design stages. The most common examples are related to the need to transport the system or system elements, which often limits the size and weight of the system elements.

Transition can also require its own enabling systems, each of which can be realized using a systems approach.

Use: Operation

Use of the system to help enable delivery of user services is often called "operations" (INCOSE 2011). A system's effectiveness is normally considered throughout the operational life of a system. For a complex system, emergent behavior should be considered in three ways:

- to identify and plan for emergent properties within the system realization process (See System Realization KA in Part 3, Systems Engineering and Management)
- to incorporate mechanisms for identifying and handling unexpected emergent properties within the system during its use
- to provide necessary procedures for dealing with wider system consequences of unexpected emergent properties in the enterprise (e.g., emergency responses or medical first aid)

Operations require their own enabling systems, each of which can be realized using a systems approach.

System Sustainment and Maintenance

System sustainment requires maintenance of the system throughout its useful life (INCOSE 2011). In system terms, maintenance implements systems that handle entropy and maintaining the SoI in a viable state. Since an open system maintains its existence by continual exchange of energy, information, and materiel with its environment, one aspect of its maintenance must be the management of resources in the environment.

Hitchins (2007) describes generic approaches to resource management and viability management based on systems concepts. Resource management identifies the need to consider the acquisition, storage, distribution, conversion, and disposal of resources. Viability management should consider systems to maintain homeostasis and a means for ensuring resilience to environmental disturbance and adaptability to environmental change.

Maintenance will require its own enabling systems, each of which can be realized using a systems approach. Maintenance success is more likely if it is considered as part of the system concept and design well before the system enters service.

Disposal

A total life cycle systems approach cannot be considered complete without consideration of how disposal of the system will be accomplished. The purpose of disposal is to remove a system element from the operational environment with the intent of permanently terminating its use, and remove any hazardous or toxic materials or waste products (INCOSE 2011).

During disposal, the entirety of the open system crosses the boundary from the system side to the environment. A complete systems approach must consider how it crosses the boundary and what remains that must be managed by enterprises other than the ones that developed, used or sustained the system. Including disposal in the system approach expands the stakeholders, the enterprises and the external systems that must be considered.

Disposal requires its own enabling systems, each of which can be realized using a systems approach. Some of these may be contained within the system boundaries and others may be external to the system. For the external disposal systems, the interface where the handover occurs must be considered. As with maintenance, a large part of successful disposal requires related issues to have been considered early on in the system's life cycle.

The topic Disposal and Retirement in Part 3, Systems Engineering and Management, of the SEBoK provides information on the engineering aspects of system disposal.

References

Works Cited

Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: John Wiley and Sons.

INCOSE. 2012. *INCOSE Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering. INCOSE-TP-2003-002-03.2.2.

Primary References

INCOSE. 2011. *Systems Engineering Handbook*, version 3.2.1. San Diego, CA, USA: International Council on Systems Engineering. INCOSE-TP-2003-002-03.2.1.

Additional References

MITRE. 2011. "Transformation planning and organizational change," in *Systems Engineering Guide*. Available at: https://www.mitre.org/work/systems_engineering/guide/enterprise_engineering/transformation_planning_org_change/. Accessed December 4, 2014.

Applying the Systems Approach

- Lead Author:
- Rick Adcock
- Contributing Authors:
- Brian Wells, Scott Jackson, Janet Singer, Duane Hybertson, Hillary Sillitto, Bud Lawson, and James Martin

The systems approach relates to both the dynamics of problem resolution and stakeholder value over time, as well as to the levels of system relationship, detailed management, and the engineering activities this implies.

This article builds on the concepts introduced in Overview of the Systems Approach topic. It is part of the Systems Approach Applied to Engineered Systems knowledge area (KA), which describes, primarily through five groups of activities, the application of an approach based around systems thinking to engineered system contexts throughout their lives.



Life Cycle

Engineered Systems provide outcomes which deliver benefits to stakeholders by helping them achieve something of value in one or more problem situations. Ultimately, a system is successful only if it enables successful outcomes for its stakeholders (Boehm and Jain 2006). In complex real-world situations, value can best be provided through a continuing process of adapting the system needs and developing associated solutions in response to changing circumstances, according to the principle of **Progressive Satisfying** (Hitchins 2009).

A value cycle associating the systems approach to the delivery of real-world stakeholder benefits is discussed in the Overview of the Systems Approach topic. A greater understanding of the value of an engineered system within its context enables agreement on the problem situation and appropriate system interventions to be created, deployed, and used overall, in turn enabling a more effective application of the systems approach. Value is fully realized only when considered within the context of time, cost, funding and other resource issues appropriate to key stakeholders (Ring 1998).

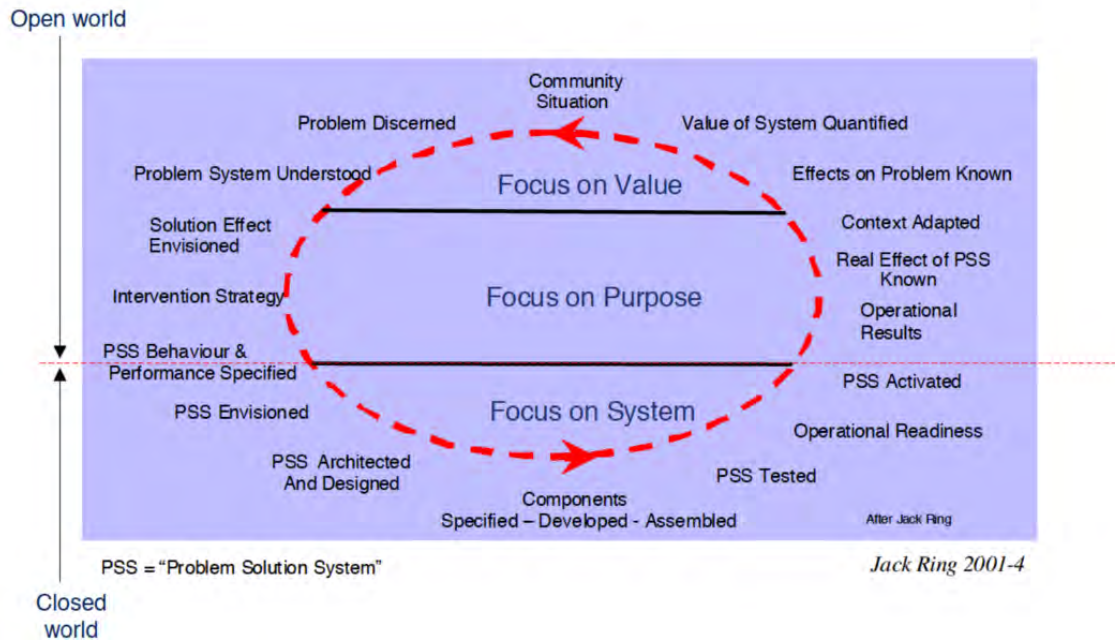


Figure 1. Ellipse Graphic (Ring 1998). © 1998 IEEE. Reprinted, with permission, from Jack Ring, Engineering Value-Seeking Systems, IEEE-SMC. Conference Proceedings. All other rights are reserved by the copyright owner.

The views in Figure 1 apply the idea of **Systemic Intervention** to the resolution of problem situations in which one or more engineered system solutions might be required. For each turn of the cycle, an agreement is made between stakeholders and developers that an Engineered System to solve problem X with effectiveness Y in agreed conditions Z has a chance of delivering value A for which they are willing to invest cost B and other resources C.

It is in the nature of wicked problems that this proposition cannot be a certainty. life cycle approaches to understand and manage the shared risk of tackling such problems are discussed in Life Cycle Models. The idea of Systemic Intervention comes from soft systems thinking (see Systems Approaches).

For each of the engineered system problems, the solutions agreed above must be developed such that they are effective in terms of cost, performance and other properties relevant to the problem domain. A developer must consider not only what to do, but when and how much to do to provide real value (Senge 1990). In systems engineering (SE) and management practices, this leads to the two key concepts (INCOSE 2011):

- **Life Cycles:** Stakeholder value and problem resolution described as a set of life cycle stages over which problems can be explored and resolved, and resources can be managed.
- **Life Cycle Processes:** Systems of activities focused on creation and sharing of knowledge associated with the systems approach, that can be employed to promote a holistic approach over a life cycle.

Life cycle management provides the framework in which to take a systems approach to all aspects of an engineered system context, which includes not only the system product or service but also the systems to create, deploy and support it (Martin 2004). The following sections consider how the systems approach should be applied to an identified problem statement, within the context of the overall value cycle discussed above.

Application Principles

Concurrency

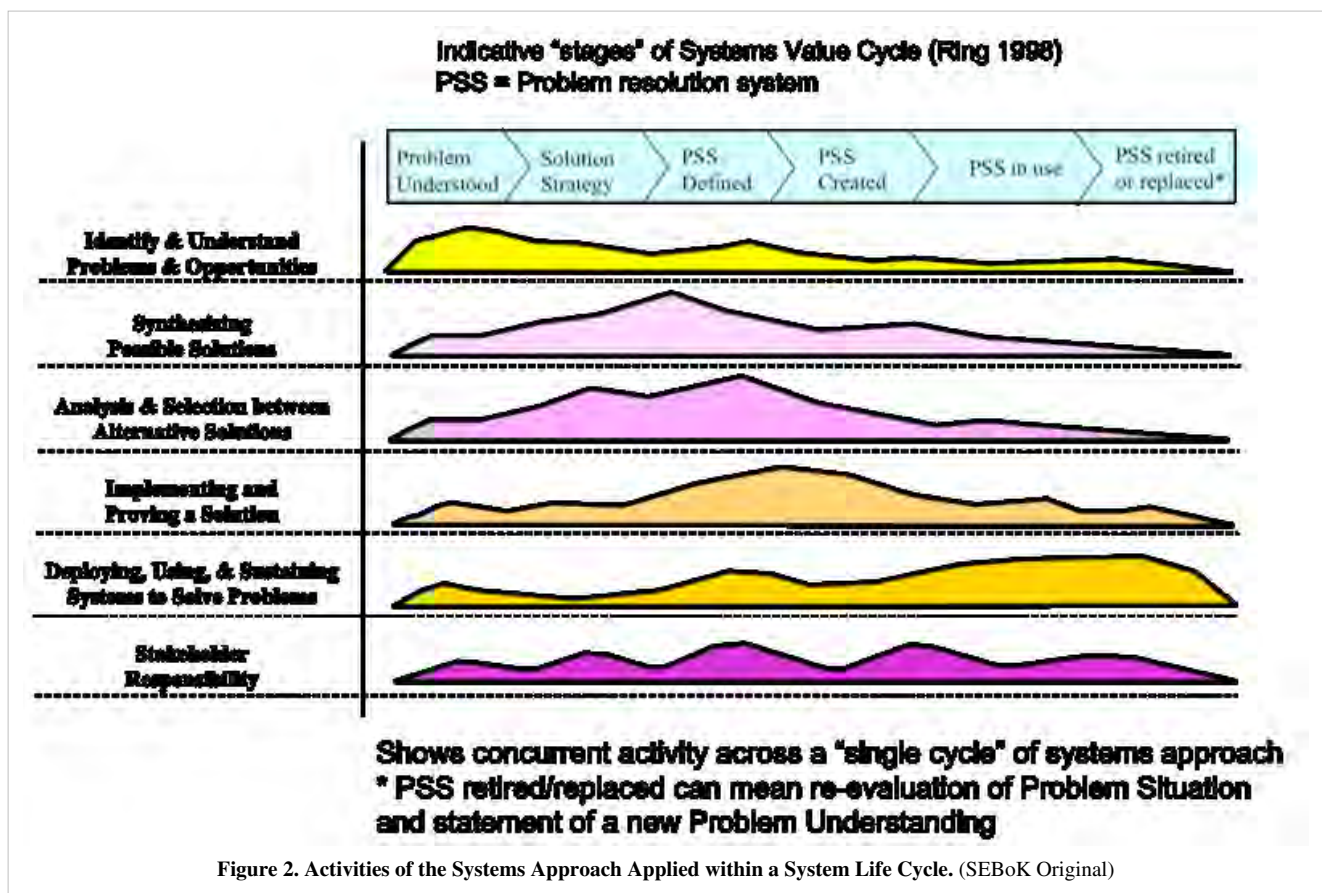
Within any application of the systems approach, the activities of problem identification, solution synthesis and selection, solution implementation and proving, and deployment, sustainment, and use should be applied concurrently, reflecting their interrelationships and dependencies.

The system value cycle (Ring 1998) can be taken as a generic model of the life of an engineered system within a problem resolution cycle driven by stakeholder value. For practical reasons, it is necessary to break this life down into a set of finite stages to allow activities to be organized. We can express the value cycle as six groups of questions to cycle around value, problem, and solution questions that are related to the systems approach:

1. What values do stakeholders want/need?
2. What system outcomes could improve this value?
3. What system can provide these outcomes?
4. How do we create such a system?
5. How do we deploy and use the system to achieve the outcomes?
6. Do these outcomes provide the expected improvement in value?

The above questions focus on each iteration of the systems approach to deliver stakeholder goals within an enterprise context. Activities 1 and 6 are part of the business cycles of providing stakeholder value within an enterprise, whereas activities 2 through 5 can be mapped directly to product, service, and enterprise engineering life cycles. A distinction is made here between the normal business of an enterprise and the longer-term strategic activities of Enterprise Systems Engineering.

The following diagram illustrates the concurrent nature of the activities of a systems approach over time.



The lines on Figure 2 represent activity in each of the activity areas over a simple (not to scale) life cycle based on the questions above. Activities may have a primary focus in certain stages but need to span the whole of life to ensure a holistic approach. For example, problem identification has a large input during the Problem Understanding stage, but problems are refined, reviewed, and reassessed over the rest of the life cycle. Similarly, Implement and Proving activities are conducted during the transition from Create to Use. This is only possible if proving issues, strategies, and risks are considered in earlier stages. This diagram is a schematic representation of these activity mappings, sometimes called a hump diagram (Kruchten 2003).

For the generic systems approach, the following fundamental life cycle principles apply:

- A life cycle has groups of stages which cover understanding stakeholder value; exploration of a problem situation; creation of a system solution (see System Realization); and System Deployment and Use.
- Life cycle processes define a system of engineering and management activities based on the detailed information needed to ensure a systems approach across a life cycle (e.g., requirements, architecture, verification, and validation).
- Activities in any of the processes may be employed in all of the stages to allow for appropriate concurrency.
- The sequence and control of the life cycle stages and concurrent process activities must be tailored to the problem situation and commercial environment (Lawson 2010), thus leading to the selection of an appropriate life cycle model.
- Appropriate management activities must be included in the life cycle to ensure consideration of time, cost, and resource drivers.
- In focusing on the creation of a specific system-of-interest (SoI) to provide solutions within the cycle, it is important to recognize the need to employ the right balance between reductionism and holism by considering the appropriate system context.

The ways in which this idea of concurrent process activity across a life cycle has been implemented in SE are discussed in Systems Engineering and Management.

Iteration

The systems approach can be applied in an iterative way to move towards an acceptable solution to a problem situation within a larger cycle of stakeholder value.

The systems approach can be applied to multiple systems within an engineered system context, as discussed below. At each level, the approach may be applied iteratively to cycle between what is needed and versions of the solutions within a life cycle model.

Hitchins (2009) defines two principles related to iterations:

- **Adaptive Optimizing:** Continual redesign addresses the problem space, detecting and addressing changes in situation, operational environment, other interacting systems, and other factors; it continually conceives, designs, and implements or reconfigures the whole solution system to perform with optimal effectiveness in the contemporary operational environment.
- **Progressive Entropy Reduction:** Continual performance and capability improvement of systems in operation may be undertaken by customer or user organizations with or without support from industry, as they seek to “get the best” out of their systems in demanding situations. In terms of knowledge or information, this process involves progressively reducing entropy, going from a condition of high entropy (that is, disorder) at the outset to low entropy (order) at the finish.

In general, these two cycles of iterations can be realized from combinations of three life cycle types (Adcock 2005):

- **Sequential:** With iteration between the stages to solve detailed issues as they arise, a single application of the systems approach is sufficient.

- Incremental: Successive versions of the sequential approach are necessary for a solution concept. Each increment adds functionality or effectiveness to the growing solution over time.
- Evolutionary: A series of applications of the sequential approach for alternative solutions intended to both provide stakeholder value and increase problem understanding. Each evolutionary cycle provides an opportunity to examine how the solution is used so the lessons learned can be incorporated in the next iteration.

These aspects of the systems approach form the basis for life cycle models in Life Cycle Models.

Recursion

The stakeholder value, problem resolution, and system creation aspects of the system value cycle may each require the use of a focused systems approach. These might be soft systems to prove a better understanding of a situation, product systems and/or service systems solutions to operational needs, enabling systems to support an aspect of the product or service life cycle, or enabling systems used directly by the enterprise system.

Each of these systems may be identified as a system-of-interest (SoI) and require the application of the systems approach. This application may be sequential (the start of one system approach dependent on the completion of another) or parallel (independent approaches which may or may not overlap in time), but will often be recursive in nature.

Recursion is a technique borrowed from computer science. In computer science, recursion occurs when a function calls itself repeatedly to logically simplify an algorithm. In a recursive application applied to systems, the systems approach for one system-of-interest is nested inside another. Examples include cases where:

- trades made at one level of the system require trades to be made for system elements;
- the analysis of a system requires analysis of a system element;
- the synthesis of a solution system requires one or more sub-system elements; and
- the verification of a product system requires verification of system elements.

In each case, the “outer” system approach may continue in parallel with the “inner” to some extent but depends on key outcomes for its own progress.

As with all recursive processes, at some stage the application of the approach must reach a level at which it can be completed successfully. This then “rolls up” to allow higher levels to move forward and eventually complete all nested applications successfully.

The *INCOSE Systems Engineering Handbook* (INCOSE 2011) describes a recursive application of SE to levels of system element with each application representing a system project. Martin (1997) describes the recursive application of SE within a product system hierarchy until a component level is reached, at which point procurement of design and build processes can be used to create solution elements.

The principle of recursive application and how it relates to life cycle models is described in Life Cycle Models. This topic is part of the Systems Approach Applied to Engineered Systems knowledge area (KA). It summarizes various aspects of stakeholder responsibility for acquisition and ownership during the system life cycle processes covered by such sources as the *International Council on Systems Engineering Handbook* (INCOSE 2012). Any of the activities described below may also need to be considered concurrently with other activities in the systems approach at a particular point in the life of a system-of-interest (SoI).

The activities described below should be considered in the context of the Overview of the Systems Approach topic at the start of this KA. The final topic in this KA, Applying the Systems Approach, considers the dynamic aspects of how these activities are used as part of the systems approach and how this relates in detail to elements of systems engineering (SE).

Stakeholder Responsibilities

The general principles of life cycle application discussed above apply as necessary to each application of SE. The following sections clarify the different kinds of stakeholder and the roles they take for the different system contexts discussed in the SEBoK.

Products, Services, and Enterprises

Most often, the terms "product" and "service" describe the effects that are exchanged in a customer and supplier agreement. This may be a commercial agreement, one funded publicly by a charity, or provided by a government agency. The difference between a product and a service is that a product is an artifact acquired to achieve an outcome while a service is an outcome supplied directly to a user.

The terms "customer" and "user" are often used interchangeably in engineering and management disciplines. The *INCOSE Systems Engineering Handbook* (INCOSE 2012) makes the following specific distinctions among the stakeholders associated with a system:

- The acquirer is the stakeholder that acquires or procures a product or service from a supplier.
- The supplier is an organization or individual that enters into an agreement with the acquirer to supply a product or service.
- The operator is an individual or organization that uses knowledge, skills and procedures to perform the functions of the system to provide the product or service.
- The user or customer is the individual or group that benefit from the operation of the system.

These terms define the roles stakeholders take; however, they may not always lie within these distinct entities (e.g. the acquirer may also be the user). This also applies to service systems, as some of the entities may also overlap in roles. Parnell et al. (2011) offer an alternative list of stakeholders that include decision authority, client, owner, user, consumer, and interconnected.

Product systems consist of hardware, software, and humans, and they have traditionally been the focus of SE efforts. These systems are delivered to the acquirer and operated to accomplish the goals that led to the requirements for the system. These requirements were derived from the need to provide products and services to one or more users as part of an enterprise.

The delivery (supplying) of a service is indicative of the direct delivery of an outcome, which is often related to the delivery of products (e.g., a maintenance, training, or cleaning service). This is not the same as the delivery of a service system (see the discussion below).

In traditional SE, the term "service" or "service system" refers to the wider system context that describes the acquirer's need to deliver user value. In this case, the service system is a fixed system definition that dictates the manner in which the acquiring enterprise will utilize the products to enable the delivery of services to users. Product systems are designed to be integrated and operated as appropriate to enable this service to be maintained or improved as required. In this view, a service system is static and contains dedicated products, people, and resources; that is, hierarchies of products are engineered to provide acquirers with the ability to offer predefined services to users or customers.

More recently, the term "service systems" has been used to describe a system that is engineered in a manner that allows enterprises to offer services directly to users, bypassing the need to hold all of the necessary products and services within the enterprise itself. This requires the expansion of the definition of a "supplier" as follows:

- A **product supplier** is an organization or individual that enters into an agreement with an acquirer to supply a product or related product support services.
 - A **service system supplier** is an organization or individual that enters into an agreement with an acquirer to supply a service system.
 - A **service supplier** is an organization or individual that enters into an agreement with a user to supply a service.
-

These service systems tend to be configured dynamically to deal with problems that traditional static services find challenging to address. This view of a service system employs "late binding" with product systems that are not owned by the enterprise but are used to enable the service to be offered as closely to given time demands as possible. This is the definition of a service system used in the Service Systems Engineering topic in Part 4, Applications of Systems Engineering.

Stakeholder Needs

One of the most critical stakeholder responsibilities is to identify the needs and requirements for the system that provides the products or services (INCOSE 2012). These needs and requirements are expressed in agreements between acquirers and suppliers.

There are other stakeholders who shape system requirements based on their needs, but who are not necessarily acquirers or suppliers. The stakeholders and the requirements engineers share the responsibility to identify their needs during the requirements process.

Acquirer/Supplier Agreements

Lawson (2010) provides a perspective on what it means to own systems, trade in system products and services, and the implications of supply chains in respect to the value added and ownership of the systems, its products and services. INCOSE (2012) defines two life cycle processes related to acquisition and supply. The acquisition process includes activities to identify, select, and reach commercial agreements with a product or service supplier.

In many larger organizations, there is a tradition of system ownership vested in individuals or, in some cases, enterprise entities (groups or teams). Ownership implies the authority and responsibility to create, manage, and dispose of a system-of-interest (SoI), as well as sometimes to operate the SoI.

Product Acquire/Supply

In some industries, a supplier works directly with an acquirer to help understand the acquirer's needs and then engineer one or more products to satisfy those needs. In certain cases, a single supplier will provide the complete worthy product system. In other cases, a supply chain will be formed to deliver product systems with a system integrator to ensure they fit together and integrate into the wider context. This is a theoretical view of product systems engineering in which the context is fixed and the product is designed to fit into it. A good systems engineer may suggest changes to the enterprise as a better way to solve the problem and then modify the product system's requirements accordingly. However, at some point, an agreed context will be set and a product system developed to work within it.

For many commercial products, such as mobile phones, a supplier creates a representative user profile to generate the requirement and then markets the product to real users once it is realized. In these cases, the other elements of the systems approach are performed by the acquirer/user and may not follow formal SE processes. It is important that a product supplier takes this into account when considering the best manner in which to engineer a system, as additional help or support services may need to be offered with the purchased product. The idea of a supplier offering support services for users with a type of product purchased elsewhere (e.g., an auto-mechanic servicing different makes of cars) begins to overlap with the service systems context, as discussed in the next topic.

For an institutionalized infrastructure in which SoIs are entirely owned by an enterprise or parties thereof, the entire responsibility of life cycle management, including operation, is often vested with the system owners. These systems belong to the system asset portfolio of an enterprise or multiple enterprises and provide the system resources, including the planned systems that are developed during life cycle management.

Service Acquire/Supply

Organizations providing service systems need not own the individual products and services that they deliver to their users and customers. With this viewpoint, the supplied service system includes the means to identify and gain access to appropriate products or services when needed. The service systems would then be the bundle of products and services assembled for the user; for example, assembling software applications and service agreements for a mobile phone already owned by a user. The enterprises providing service systems may, in turn, offer infrastructure services to a wide range of different technologies or application domains. This can mean that the transition, operation, maintenance and disposal activities associated with system ownership may not be embedded in the acquiring service system enterprise, and will therefore need to be treated as separate system services. More detail can be found in Product Systems Engineering, Service Systems Engineering, and Enterprise Systems Engineering, in Part 4, Applications of Systems Engineering in the *Guide to the Systems Engineering Body of Knowledge* (SEBoK).

The service systems engineer helps the service supplier create and sustain the service system that can be used to discover, integrate, and use specific versions of generic products or services when needed. The realization of service systems requires the ability to make use of product systems; however, these product systems are developed and owned outside of the service system. The service system must be able to gain access to a product or service when needed, as well as to interface with it effectively. The use of open interface standards, such as standard power supplies, interface connections (e.g., Universal Serial Bus (USB)), or file formats (e.g., Portable Document Format (PDF)) can help make this easier.

Enterprise Evolution

A useful distinction between product system design and enterprise system design is that “enterprise design does not occur at a single point in time like the design of most systems. Instead, enterprises evolve over time and are constantly changing, or are constantly being designed” (Giachetti 2010, xiii).

The enterprise developer may also aim to optimize backstage processes (the internal operations) of an organization or an institution by exploiting advances in technology, particularly information technology (IT) and associated processes. In these cases, the engineered systems are considered to be enterprise systems.

Enterprise systems may offer products (goods) and/or services. From an enterprise engineering viewpoint, an enterprise concurrent with its product SE must not only look at the development and delivery of the products but also look at the alignment and optimization of the product delivery within the enterprise objectives. Similarly, in service SE, the main focus is on an intangible value delivery to the end-customer (externally focused: front stage), in which internal and external processes must be synchronized. However, with the rapid advances in information and communications technologies (ICT), in many cases the boundaries between internal and external processes are quite blurred. Current SE research is extending product methods, processes, and tools into the enterprise transformation and service innovation fields to exploit advances in business process methodologies and technologies.

Enterprise SE must not only do the engineering of the enterprise itself but may also be involved in the engineering of the service systems and product systems that are necessary for the enterprise to achieve its goals.

Activity Mapping

This topic belongs to the Systems Approach Applied to Engineered Systems KA from Part 2, Foundations of Systems Engineering. Other topics about activities, from the same KA, relate to high level technical processes defined in KAs in Part 3, Systems Engineering and Management, in the following way:

- Identifying and Understanding Problems and Opportunities topic relates to the System Concept Definition KA.
- Synthesizing Possible Solutions and Analysis and Selection between Alternative Solutions topics relate to the Systems Approach Applied to Engineered Systems KA.
- Implementing and Proving a Solution topic relates to the System Realization KA.
- Deploying, Using, and Sustaining Systems to Solve Problems topic relates to the Product and Service Life Management KA.

Part 3 discusses the principles defined in each of the systems approach activities, and how they help shape the technical processes to which they are mapped.

References

Works Cited

- Adcock, R.D. 2005. "Tailoring systems engineering lifecycle processes to meet the challenges of project and programme". *INCOSE International Symposium 2005*. Volume 15. Issue 1.
- Boehm, B. and A. Jain. 2006. "A value-based theory of systems engineering." Presented at the 16th Annual INCOSE Systems Engineering Conference, Orlando, FL, USA.
- Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4.
- INCOSE. 2011. *INCOSE Systems Engineering Handbook*, version 3.2.1. San Diego, CA, USA: International Council on Systems Engineering. INCOSE-TP-2003-002-03.2.1.
- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press.
- Kruchten, P. 2003. *The Rational Unified Process: An Introduction*, 3rd edition. Boston, MA, USA: Addison Wesley.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.
- Martin J.N. 1997. *Systems Engineering Guidebook*. Boca Raton, FL, USA: CRC Press.
- Martin, J. 2004. "The Seven Samurai of Systems Engineering: Dealing with the Complexity of 7 Interrelated Systems." *INCOSE 2004 - 14th Annual International Symposium Proceedings*.
- Parnell, G.S., P.J. Driscoll, and D.L. Henderson (eds). 2011. *Decision Making for Systems Engineering and Management*, 2nd ed. Wiley Series in Systems Engineering. Hoboken, NJ, USA: Wiley & Sons Inc.
- Ring, J. 1998. "A value seeking approach to the engineering of systems." *Proceedings of the IEEE Conference on Systems, Man, and Cybernetics*. p. 2704-2708.
- Senge, P. 1990. *The Fifth Discipline: The Art and Practice of the Learning Organization*. New York, NY, USA: Doubleday/Currency.

Primary References

Hitchins, D. 2009. "What are the general principles applicable to systems?" *INCOSE Insight*, vol. 12, no. 4.

Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.

Additional References

Blanchard, B. and W.J. Fabrycky. 2006. *Systems Engineering and Analysis*. Upper Saddle River, NJ, USA: Prentice Hall.

Carlock, P.G. and R.E. Fenton. 2001. "System of Systems (SoS) enterprise systems engineering for information-intensive organizations." *Systems Engineering*, vol. 4, no. 4, pp. 242–261.

Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.

Rouse, W.B. 2005. "Enterprises as systems: Essential challenges and approaches to transformation." *Systems Engineering*, vol. 8, no. 2, pp. 138-150.

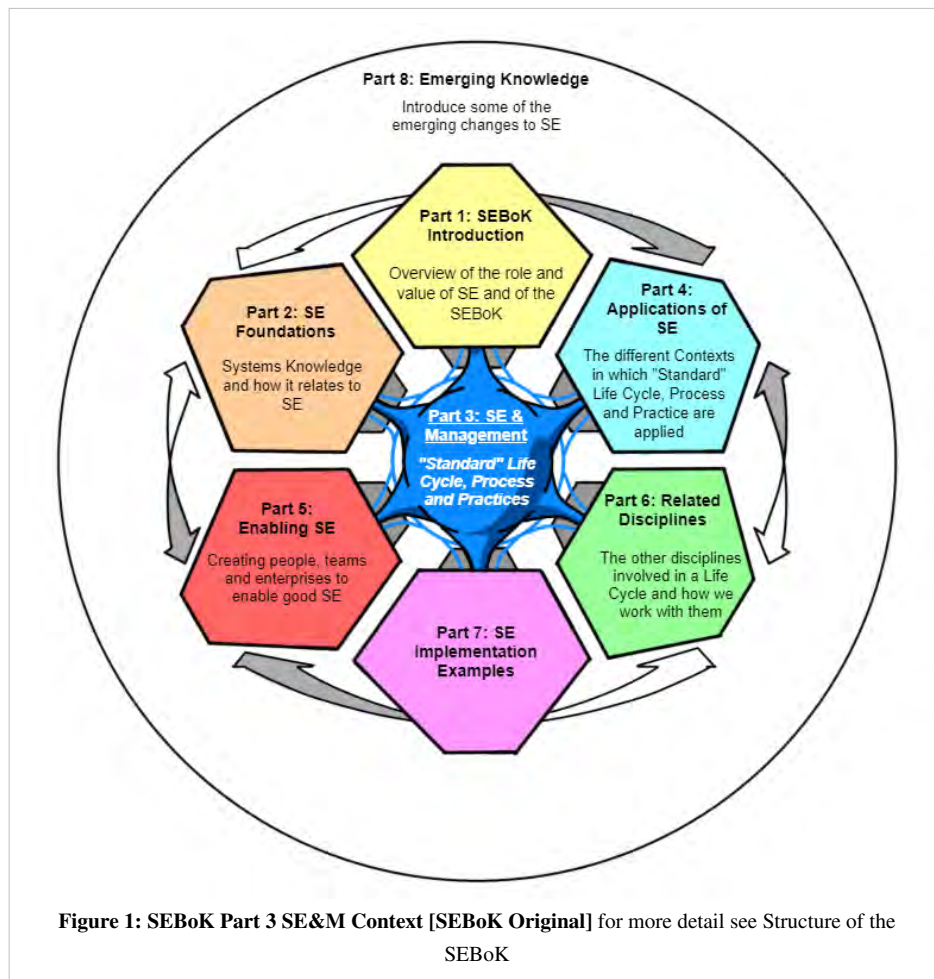
Part 3: Systems Engineering and Management

Systems Engineering and Management

Contents of this Part

- Systems Engineering STEM Overview (Bill Schindel)
- Model-Based Systems Engineering (MBSE) (Caitlyn Singam and Jeffrey Carter)
- Life Cycle Terms and Concepts (Mike Yokell) (David Endler and Garry Roedler)
- Development Approaches (David Endler) (Mike Yokell and Garry Roedler)
- Agile Approaches (David Endler) (Mike Yokell)
- Life Cycle Model Selection and Adaptation (David Endler) (Mike Yokell and Garry Roedler)
- Process Concepts (David Endler) (Mike Yokell and Garry Roedler)
- Process Selection and Tailoring (David Endler) (Mike Yokell and Garry Roedler)
- Technical Management Processes (Ray Madachy and Garry Roedler)
- System Concept Definition (Tami Katz) (Lou Wheatcraft, Mike Ryan, Garry Roedler, and Rick Adcock)
- System Requirements Definition (Tami Katz) (Lou Wheatcraft and Mike Ryan)
- System Architecture Design Definition (Jeffrey Carter and Caitlyn Singam)
- System Detailed Design Definition (Alan Faisandier and Rick Adcock)
- System Analysis (Alan Faisandier and Ray Madachy) (Rick Adcock)
- System Realization (John Snoderly and Alan Faisandier) (Rick Adcock)
- System Implementation (John Snoderly and Alan Faisandier)
- System Integration (John Snoderly, Alan Faisandier, and Scott Jackson)
- System Verification (John Snoderly and Alan Faisandier)
- System Transition (Scott Jackson and Brian Gallagher)
- System Validation (Alan Faisandier) (Rick Adcock)
- System Operation (Scott Jackson and Brian Gallagher) (Leopoldo deCardenas)
- System Maintenance (Scott Jackson and Brian Gallagher) (David Dorgan)
- Systems Engineering Standards (Garry Roedler) (Bill Bearden, David Endler, and Mike Yokell)
- Lead Authors:
- David Endler and Mike Yokell

Systems Engineering and Management (SE&M) articles provide system lifecycle good practices for defining and executing interdisciplinary processes to ensure that customer needs are satisfied with a technical performance, schedule, and cost compliant solution. The figure below depicts the context of SE&M processes and practices guidance within the SEBoK.



The SE&M materials are currently being updated to better align with international standards.

SE&M Knowledge Areas

The SE&M articles provide exemplary processes and practices which can be adapted by an engineering organization to satisfy strategic business goals and individual project objectives including:

- How engineering conducts system development
- The purpose of each engineering artifact generated
- How systems are integrated, and requirements verified
- How new product designs are transitioned to production operations
- How the resulting system is employed and sustained to satisfy customer needs

Part 3A - Life Cycle Concepts

A life cycle is the evolution of a system, product, service, project or other human-made entity from conception through retirement. There are various approaches to properly manage the progression of the system of interest through its life cycle stages as well as the planning for the other stages. In general, the approaches can be roughly divided into sequential, incremental and evolutionary approaches. Agile systems engineering is a principle-based method for designing, building, sustaining, and evolving systems when knowledge is uncertain and/or environments are dynamic. Life cycle models serve as essential frameworks that guide the development and management of systems and services throughout their existence. Tailored to organizational needs, these models help align projects with strategic goals while accommodating industry regulations and internal diversity. Life cycle models and development approaches can be selected and adapted to meet specific needs.

Knowledge Areas

The Part 3A - Life Cycle Concepts articles are organized into the following Knowledge Areas [KAs]:

- Knowledge Area – Life Cycle Terms and Concepts
- Knowledge Area – Development Approaches
- Knowledge Area – Agile Systems Engineering
- Knowledge Area – Life Cycle Model Selection and Adaptation

Systems Engineering STEM Overview

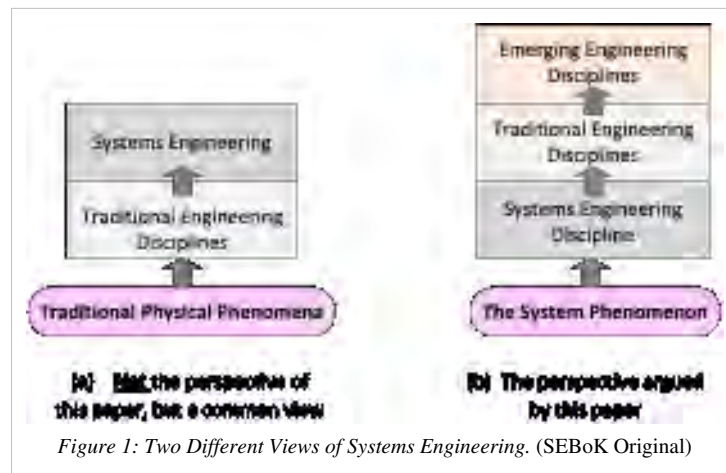
- Lead Author:
- Bill Schindel

Engineering disciplines (ME, EE, CE, ChE) sometimes argue their fields have “real physical phenomena”, “hard science” based laws, and first principles, claiming Systems Engineering lacks equivalent phenomenological foundation. Here we argue the opposite, and how replanting systems engineering in MBSE/PBSE supports emergence of new hard sciences and phenomena-based domain disciplines with deep historical roots. Supporting this perspective is the System Phenomenon, wellspring of engineering opportunities and challenges. Governed by Hamilton’s Principle, it is a traditional path for derivation of equations of motion or physical laws of so-called “fundamental” physical phenomena of mechanics, electromagnetics, chemistry, and thermodynamics.

We argue that laws and phenomena of traditional disciplines are less fundamental than the System Phenomenon from which they spring—an historical fact that was well-known and equally remarkable 200 years earlier to the pioneers of mathematical physics. This is a practical reminder of emerging higher disciplines, with their own phenomena, first principles, and physical laws. Contemporary examples include ground vehicles, aircraft, marine vessels, and biochemical networks; ahead are health care, distribution networks, market systems, ecologies, and the IoT.

Introduction

As a formal body of knowledge and practice, Systems Engineering is much younger than the more established engineering disciplines, such as Civil, Mechanical, Chemical, and Electrical Engineering. Comparing their underlying scientific foundations to some equivalent in Systems Engineering sometimes arises as a dispute, concerning whose profession is “real” engineering based on (or at least later explained by) hard science, with tangible physical phenomena, and accompanied by physical laws and first principles. This paper summarizes the argument for a different perspective altogether (Figure 1), and the reader exploring this paper is warned to avoid the trap of the seemingly familiar in parsing the message. A more complete discussion is provided in (Schindel 2016) and (Schindel 2019).



Beyond that argument, this paper addresses a more pragmatic goal—the means of identifying and representing the tangible physical phenomena that emerge in new system domains, along with their respective physical laws and first principles. This is of more than philosophical or professional significance. Challenged by numerous issues in emerging systems, society has an interest in organizing successful approaches to the scientific understanding of laws and first principles about, and engineering harnessing of, the related phenomena. Individuals entering or navigating the technical professions likewise have personal interests in this evolving roadmap.

While recognizing the formidable works of systems theorists in these still early days of systems engineering (Ashby 1956; Bertalanffy 1969; Braha et al 2006; Cowan et al 1994; Holland 1998; Prigogine 1980; Warfield 2006; Wymore 1967), this paper focuses on even earlier contributions of science and mathematics to the flowering of engineering’s impact over the last three centuries. We will extract the “System Phenomenon” at the center of that foundation and consider its impacts and implications for systems engineering practice. This perspective helps us understand the phase change that Systems Engineering is going through, as model-based representations enable the framework that has already had profound impact in the traditional science/engineering paired disciplines.

Phase Change Evidence: Efficacy of Hard Science, Phenomena-Based, STEM Disciplines

Science, Technology, Engineering, and Mathematics [STEM] —300 Years of Impact

Our pragmatic argument is based on assessing the impact of the physical sciences and mathematics on engineering by their joint efficacy in improving the human condition. In a matter of 300 years (from around Newton), the accelerating emergence of Science, Technology, Engineering, and Mathematics (STEM) has lifted the possibility, quality, and length of life for a large portion of humanity, while dramatically increasing human future potential (Mokyr 2009; Morris 2012; Rogers 2003). By the close of the Twentieth Century, the learning and impacts of STEM along with other factors (e.g., market capitalism as a driver of prosperity, as in (Friedman 1980)) were increasingly recognized as critical to individual and collective human prosperity. During that same period, the human-populated world has become vastly more interconnected, complex, and challenging. New opportunities and threats have emerged, in part out of less positive impacts of human applications of STEM. Understanding and harnessing the possibilities have become even more important than before, from the smallest known constituents of matter and life to the largest scale complexities of networks, economies, the natural environment, and living systems

"Phase Changes": Emergence of Science and Engineering as Phenomena-Based Disciplines

Over those three centuries, the “hard sciences”, along with the engineering disciplines and technologies based on those sciences, are credited with much of this amazing societal progress, as well as some related challenges (Mokyr 2009; Morris 2012; Rogers 2003). Our point here is the enormous impact of these “traditional” (at least, over 300 short years) disciplines, as their foundations emerged in understanding of physical phenomena and related predictive and explanatory models.

How can the foundational roots of Systems Engineering be compared to engineering disciplines already seen as based on the “hard sciences”? The traditional engineering disciplines (ME, EE, ChE, CE) have their technical bases and quantitative foundations in what emerged as physical sciences of what came to be understood as physical phenomena.

It wasn't always this way, as seen from the shift that began to occur just three centuries ago. It is informative to remember the “phase changes” that occurred in what are now considered the traditional disciplines, by recalling the history of physics before Newton, chemistry before Lavoisier and Mendeleev, and electrical science before Faraday, Hertz, and Maxwell, versus what followed for each. (Cardwell 1971; Forbes et al 2014; Pauling 1960; Servos 1996; Westfall 1980) All of these domains had earlier, less effective, bodies of thought, generated by those attempting to answer questions and, in some cases, provide practical benefits. Instead of dismissing alchemy, astrology, pre-Copernican cosmology, and their counterparts, we can instead see them as grappling with phenomena without the benefit of sufficiently powerful physical-mathematical representation and the verification mechanisms of experiment and refutation to test against reality what we would now call models.

Systems Engineering is Still Young

Contemporary specialists in individual engineering disciplines (e.g., ME, EE, CE, ChE) sometimes argue that their fields are based on “real physical phenomena”, founded on physical laws based in the “hard sciences” and first principles. One sometimes hears claims that Systems Engineering lacks the equivalent phenomena-based theoretical foundations. In that telling, Systems Engineering is instead critically portrayed as emphasizing (1) process and procedure, (2) critical and systems thinking and good writing skills, and (3) organizing and accounting for information and risk in particular ways—valuable, but not as based on an underlying “hard science”.

That view is understandable, given the initial trajectory of the first 50 years of Systems Engineering. (Adcock 2015; Checkland 1981; Walden et al 2015) “Science” or “phenomenon” of generalized systems have for the most part been described on an intuitive or qualitative basis, with limited reference to a “physical phenomenon” that might be called the basis of systems science and systems engineering. Some systemic phenomena (e.g., requisite variety, emergence of structure, complexity, chaos theory, etc.) have received attention, but it is challenging to argue that these insights have had as great an impact (yet) on the human condition and engineering practice as the broader STEM illustrations cited above for the most recent three centuries of physical sciences and mathematics. However, INCOSE's own stated vision (Friedenthal et al 2014) calls upon systems engineering for such a result.

Respectful of the contributions of those early thinkers in systems engineering, we also note that their contributions can in some cases be expressed as manifestations of the modeled System Phenomenon described below, advancing the scientific foundations of systems engineering.

MBSE, PBSE: Enabling a Phase Change in Systems Engineering

In the case of systems engineering, a key part of the story is that the role that quantitative system models have played, or not played, during its initial history. Most recently, the broader INCOSE-encouraged role for model-based methods offers to eventually accelerate the “phase change” that the successful earlier history of science, mathematics, and other engineering disciplines suggest is now in progress.

Models are certainly not new to segments of engineering practice. However, we are representing an increasingly fraction of our overall understanding of systems, from stakeholder trade space, to required functionality and performance, to design, and to risk, using explicit and increasingly integrated system models. As in Newton’s day, this also puts pressure on the approaches to model representations, in order that they effectively represent the key ideas concerning the real things they are intended to describe. “Effective” meant these models described observable phenomena, offered explanatory theories of cause, provided verifiable (or falsifiable) predictions, and increased human understanding. In many cases, this understanding was harnessed by practicing engineers to improve human life. The progress of physical sciences did not arise from models that only could describe single unique instances of systems, but instead represented what came to be understood as more general patterns that recur across broad families of systems. Likewise, there is an increasing effort in systems engineering to recognize that these models must often describe patterns of similarity and parameterized variation. The increasing use of explicit model-based patterns in these representations is a part of this phase change (INCOSE Patterns WG 2015; INCOSE MBSE Initiative 2015). Pattern-Based Systems Engineering (PBSE) as an extension of Model-Based Systems Engineering (MBSE) increases emphasis on representation.

This is a more significant change than just the emergence of standards for systems modeling languages and IT toolsets, even though those are valuable steps. We need underlying model structures that are strong enough—remember physics before the calculus of Newton & Leibniz. As a test of “strong enough”, we suggest the ability to have the kinds of impact on humankind summarized in Section 2—beginning with clearer focus on what phenomena are being represented.

Although this sounds challenging, it is not necessary for emerging systems models to “start from scratch” in their search for new system phenomena, and further argue that what is already known from the earlier phase change of Section 2 helps suggest what aspects of our systems models need to be strengthened during the phase change in systems engineering. PBSE further reminds us of a practical lesson from the STEM revolution. Once validated patterns emerge, we (mostly) need to learn and apply those patterns (laws, principles), not how to re-derive them from earlier knowledge. Examples include the Periodic Table and the Gas Laws. While it may be controversial, “learn the model, not modeling” is advice worth considering, in a time when modeling from scratch seems carry more excitement.

The System Phenomenon

The perspective used in this paper defines a system as a collecting of interacting components, where interactions involve the exchange of energy, force, mass, or information, through which one component impacts the state of another component, and in which the state of a component impacts its behavior in future interactions (Schindel 2011).

In this framework, all behavior is expressed through physical interactions (Figure 2). This perspective emphasizes physical interactions as the context in which all the laws of the hard sciences are expressed. (Schindel 2013)



The traditional “Phenomena” of the hard sciences are all cases of the following System Phenomenon:

1. Each component has a specific behavior during a given interaction type, determined by the component’s state. (See (4) below for the source of that component’s behavioral characteristics.)
2. The combined behaviors of the set of interacting components determine a combined system state space trajectory.
3. That trajectory is a collective property of the system components and interaction, and accordingly is not simply the description of possible behaviors of the individual components. For the systems discussed in this paper, by Hamilton’s Principle (Levi 2014; Sussman et al 2001; Hankins 2004), the emergent interaction-based behavior of the larger system is a “stationary” trajectory $X = X(t)$ of the action integral, based on the Lagrangian L of the combined system:

$$S[X] = \int_A^B L(X, \dot{X}, t) dt$$

1. The behavioral characteristics of each interacting component in (1) above are in turn determined by its internal (“subsystem”) components, themselves interacting.

Reduced to simplest forms, the resulting equations of motion (or if not known or solvable, empirically observed paths) provide “physical laws” (or recurring observable behaviors) subject to verification.

Instead of Systems Engineering lacking the kind of theoretical foundation that the “hard sciences” bring to other engineering disciplines, we therefore assert that:

- It turns out that all those other engineering disciplines’ foundations are themselves dependent upon the System Phenomenon, and emerge from it.
- The related underlying math and science of systems (dating to at least Hamilton) provides the theoretical basis already used by all the hard sciences and their respective engineering disciplines.
- It is not Systems Engineering that lacks its own foundation—instead, it has been providing the foundation for the other disciplines! (Refer to Figure 6.)
- This insight was well-known and remarkable to the sciences 200 years ago, and has continued to be remarked upon by leading scientists for its surprising coverage ever since: “It [science] has as its highest principle and most coveted aim the solution of the problem to condense all natural phenomena which have been observed and are still to be observed into one simple principle, that allows the computation of past and more especially of future processes from present ones. ...Amid the more or less general laws which mark the achievements of physical science during the course of the last centuries, the principle of least action is perhaps that which, as regards form and content, may claim to come nearest to that ideal final aim of theoretical research.” (Kline, 1981)

Historical Domain Example 1: Chemistry

Chemists, and Chemical Engineers, justifiably consider their disciplines to be based on the “hard phenomena” of Chemistry (Pauling 1960; Servos 1996):

- This perspective emerged from the scientific discovery and verification of phenomena and laws of Chemistry.
- Prominent among these was the discovery of the individual Chemical Elements and their Chemical Properties, organized by the discovered patterns of the Periodic Table.
- Emerging understanding of related phenomena and behaviors included Chemical Bonds, Chemical Reactions, Reaction Rates, Chemical Energy, and Conservation of Mass and Energy.
- Upon that structure grew further understanding of Chemical Compounds and their Properties:

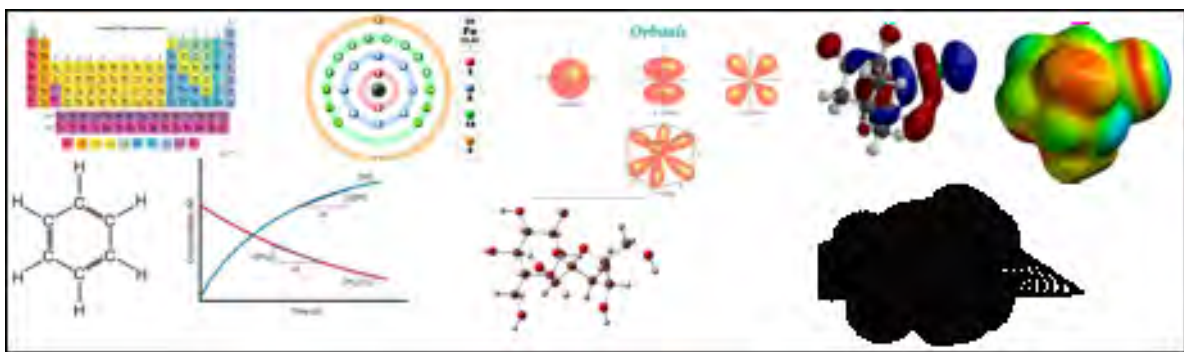


Figure 3: Chemical Interactions, Phenomena, Principles. (SEBoK Original)

Even though these chemical phenomena and laws seemed very fundamental:

- All those chemical properties and behaviors are emergent consequences of interactions that occur between atoms' orbiting electrons (or their quantum equivalents), along with limited properties (e.g., atomic weights) of the rest of the atoms they orbit.
- These lower interactions give rise to visible higher-level Chemical behavior patterns, their own higher-level properties and relationships, expressing “hard science” laws of Chemistry.

This illustrates:

- The “fundamental phenomena” of Chemistry, along with the scientifically-discovered / verified “fundamental laws / first principles” are in fact . . .
- Higher level emergent system patterns and . . .
- Chemistry and Chemical Engineering study and apply those system patterns.

Historical Domain Example 2: The Gas Laws and Fluid Flow

Illustrated by Figure 4, the discovered and verified laws of gases and of compressible and incompressible fluid flow by Boyle, Avogadro, Charles, Gay-Lussac, Bernoulli, and others are rightly viewed as fundamental to science and engineering disciplines. (Cardwell 1971) However, all those fluid and gaseous properties and behaviors are emergent consequences of interactions that occur between atoms or molecules, the containers they occupy, and their external thermal environment. These lower-level interactions give rise to patterns that have their own higher-level properties and relationships, expressed as “hard sciences” laws. So, the “fundamental phenomena” of gases, along with the scientifically-discovered and verified “fundamental laws and first principles” are in fact higher level emergent system patterns. And so, Mechanical Engineers, Thermodynamicists, and Aerospace Engineers can study and apply those system patterns.

[[File: |thumb|center|750px|Figure 4: Gas, Fluid Interactions, Phenomena, Principles. (SEBoK Original)]]

Examples from More Recent History

The practical point of this paper is to emphasize the constant emergence of new scientific and engineering disciplines, in domains arising from higher level system interactions. These include domains that have been important to society, even though they arose later than the more fundamental domains from which they spring. The discovery and exploitation of these higher-level phenomena, principles, and laws is important to future progress and innovation, including enterprises, careers of individuals, and society. These more recent emergent domains, in which formal system patterns are being recognized as describing higher-level phenomena and laws, are illustrated by examples of Figure 8:

1. Ground Vehicles: As in the dynamical laws of vehicle stability that enable vehicular stability controls (Guiggiani 2014)
2. Aircraft: Including the dynamical laws at the aircraft level that enable advanced aircraft design for dynamic performance and top-level flight controls (Pratt 2000)
3. Marine Vessels: Facilitating the design of more efficient hulls and special purpose craft, as well as bulk transports (Perez et al 2007)
4. Biological Regulatory Networks: Advancing our understanding of immune reactions and other regulatory paths in connection with pathologies as well as therapies (Davidson and Levine 2005).

For example, in the case of ground vehicles, dynamical laws of vehicle stability arise from the interactions, modulated through control algorithms, of the distributed mass of the vehicle in motion with the driving surface, transmitted through tractional forces of braking, acceleration, or steering, as further impacted by road surface and tire conditions, along with other factors. It is the overall system interaction of all these domain elements that leads to emergent vehicular laws of motion.

Students of complexity (Cowan et al 1994) will note that nonlinearity, the onset of chaos, and extreme interdependencies are not reasons to avoid representing the interactions manifesting that behavior. Indeed, they provide further reasons to understand those very interactions.

Strengthening the Foundations of MBSE

Like mechanics pre-Newton, models of MBSE require an underlying framework to effectively describe the System Phenomenon in domains of practice. MBSE requires a strong enough underlying Metamodel to support phenomenon-based systems science. As discussed in (Schindel 2013), Interactions play a central role in such frameworks, inspired by Hamilton and three hundred years of pioneers in the emergence of science and engineering. Interactions are acknowledged by and can be modelled in some current system modelling frameworks, but typical practice and underlying structures need related improvement. Figure 9 illustrates a related, Interaction-centric, extract from the S*Metamodel (Schindel 2011).

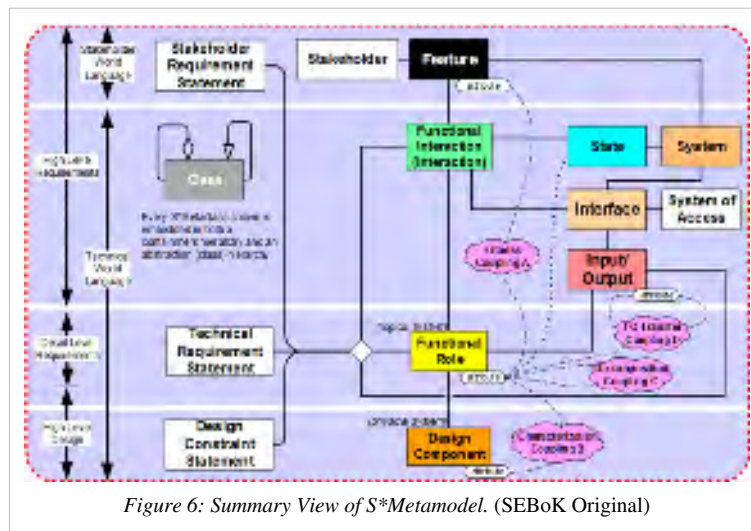


Figure 6: Summary View of S*Metamodel. (SEBoK Original)

This is more than model semantics or ontology alone. It means recognizing that the models we pursue are models of the real physical systems they are about, and not just models of information about business processes concerned with those systems. While that might seem obvious to the physical scientist, a different perspective than that is embedded in forty years of enterprise information system practice. In that history, the traditional (and relatively successful) paradigm is construction of information models that describe information transactions or documents (e.g., purchase of air travel tickets). Symptomatic of that paradigm, today we still encounter MBSE models and human interpretations of them that include notions of databases, “calls”, “methods”, and other successful software notions that are not the same as modeling physical systems.

Conclusions and Implications for Future Action

1. Like the other engineering disciplines, Systems Engineering can be viewed as founded on “real” physical phenomena—the System Phenomenon—for which experimentally verified, mathematically modeled hard science, laws, and first principles have existed for over 150 years, dating to Hamilton, or earlier, to Newton.
2. Systems Engineering not only has its own phenomenon, but the phenomena upon which the traditional engineering disciplines (ME, CE, ChE, EE) are based can themselves all be seen to be derivable from the System Phenomenon. It is SE that has the more fundamental foundation, while the other disciplines are special cases of both the phenomena and mathematics.
3. The System Phenomenon supports the emergence of hard sciences, laws, and first principles for higher level phenomena of critical importance to humankind.
4. Systems Engineering, along with its related scientific foundations, is a young and still emerging discipline. The re-planting of Systems Engineering in a model-based framework is an important step toward strengthening the discipline, but requires a stronger model framework for that to occur, and the System Phenomenon points the way to a key part of that framework.

5. A practical implication for practicing systems engineers and their educators: All models of behavior should be based on interactions. Nature offers no “naked” behavior outside interactions, but current practice and training often seem to overlook this.
6. Systems research emphasis would benefit from more attention to specific emergent domains, each of which will have their own phenomena, instead of over-emphasizing abstract generic systems. This is a well-described but often overlooked observation, as noted in (Anderson 1972)
7. There are additional phenomena in this space. For a discussion of the Value Selection Phenomenon and Group Learning and Model Trust Phenomenon, see (Schindel 2020).

References

Works Cited

- Adcock, Rick, ed., “Guide to the Systems Engineering Body of Knowledge (SEBoK)”, retrieve from: [https://sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_\(SEBoK\)](https://sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_(SEBoK))
- Anderson, P.W., “More Is Different”, *Science*, v177, No. 4047. (Aug. 4, 1972), pp. 393-396.
- Ashby, William Ross, *An Introduction to Cybernetics*, Wiley, 1956.
- Bertalanffy, L. von, (1969). *General System Theory: Foundations, Development, Applications*, George Braziller Inc.; Revised edition, 1969.
- Braha, D., A. Minai, Yaneer Bar-Yam, eds. 2006. *Complex engineered systems: Science meets technology*, City: Springer.
- Braha, Dan, and Bar-Yam, Yaneer, “The Statistical Mechanics of Complex Product Development: Empirical and Analytical Results”, *Management Science*, Vol. 53, No. 7, July 2007, pp. 1127–1145.
- Cardwell, D.S.L. *From Watt to Clausius: The Rise of Thermodynamics in the Early Industrial Age*. London: Heinemann, 1971. Checkland, P., *System Thinking, System Practice*, Wiley, 1999.
- Cowan, George, Pines, David, and Meltzer, David, *Complexity: Metaphors, Models, and Reality*, Proceedings Volume XIX, Santa Fe Institute Studies in Science of Complexity, Addison-Wesley, 1994.
- Davidson, E., Levine, M., eds, “Gene Regulatory Networks”, *Proc Natl Acad Sci USA*. Apr 5 2005; 102(14): 4935. Retrieve from-- <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC556010/>
- Forbes, Nancy, and Mahon, Basil, *Faraday, Maxwell, and the Electromagnetic Field: How Two Men Revolutionized Physics*, Prometheus Books, 2014.
- Friedenthal et al, “A World in Motion: Systems Engineering Vision 2025”, International Council on Systems Engineering, 2014. Friedman, Milton and Friedman, Rose, *Free to Choose: A Personal Statement*, Harcourt, 1980.
- Guiggiani, Massimo, *The Science of Vehicle Dynamics: Handling, Braking, and Ride of Road and Race Cars*, Springer, 2014.
- Hankins, T., *Sir William Rowan Hamilton*, Johns Hopkins University Press, 2011.
- Holdren, John P., Lander, Eric S., et al, “Report to the President--Better Health Care and Lower Costs: Accelerating Improvement Through Systems Engineering”, Executive Office of the President, President’s Council of Advisors on Science and Technology, May 2014. <https://www.whitehouse.gov/ostp/pcast>
- Hoffman, C. and Schindel, W., “Systems Engineering Community of Practice Social Network Pattern”, *Proc. of INCOSE Great Lakes Regional Conference*, 2015.
- Holland, John H., *Emergence: From Chaos to Order*, Perseus, 1998.
- INCOSE MBSE Initiative Patterns Working Group web site, at <https://www.omgwiki.org/MBSE/doku.php?id=mbse:patterns:patterns>

- INCOSE MBSE Patterns Working Group, "MBSE Methodology Summary: Pattern-Based Systems Engineering (PBSE), Based On S*MBSE Models", V1.6.1,
- INCOSE PBSE Working Group, 2019: https://www.omgwiki.org/MBSE/lib/exe/fetch.php?media=mbse:patterns:pbse_extension_of_mbse--methodology_summary_v1.6.1.pdf
- INCOSE, Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities, Fifth Edition, John Wiley and Sons, Inc., Hoboken, NJ, 2023, ISBN: 978-1-119-81429-0.
- Kline, Morris. Mathematics and the Physical World. Dover, 1981.
- Levi, M., Classical Mechanics with Calculus of Variations and Optimal Control, American Mathematical Society, Providence, Rhode Island, 2014.
- MacArthur R. H., and Wilson, E. O., The Theory of Island Biogeography, Princeton U. Press, 2001.
- Mokyr, Joel, The Enlightened Economy: An Economic History of Britain 1700-1850, Yale University Press, 2009.
- Morris, C. R., The Dawn of Innovation: The First American Industrial Revolution, Public Affairs, 2012.
- Pauling, L., The Nature of the Chemical Bond and the Structure of Molecules and Crystals: An Introduction to Modern Structural Chemistry, 3rd edition, Cornell University Press; 1960
- Perez, Tristan, and Fossen, Thor I., "Modelling and Simulation of Marine Surface Vessel Dynamics", Tutorial, IFAC Conference on Control Applications in Marine Systems, Bol, Croatia, 2007.
- Pérez-Arriaga, Ignacio, et al, "From Distribution Networks to Smart Distribution Systems: Rethinking the Regulation of European Electricity DSOs", THINK Project Final Report, European University Institute, 2013.
- Pratt, Roger W., ed., Flight Control Systems: Practical Issues in Design and Implementation, IEE Control Engineering, 2000.
- Prigogine, Ilya, From Being to Becoming: Time and Complexity in the Physical Sciences, Freeman, 1980.
- Rogers, Everett M. Roger, Diffusion of Innovations, Fifth Edition, Free Press, 2003.
- Schindel, W., "Inputs to Theoretical Foundations Section of INCOSE Vision 2035", April, 2020. Retrieve from--
https://www.omgwiki.org/MBSE/lib/exe/fetch.php?media=mbse:patterns:science_math_foundations_for_systems_and_systems_engineering--1_hr_awareness_v2.3.2a.pdf
- Schindel, W., "Got Phenomena? Science-Based Disciplines for Emerging Systems Challenges", in Proc. of INCOSE 2016 International Symposium, 2016.
- Schindel, W., "System Interactions: Making the Heart of Systems More Visible", Proc. of INCOSE Great Lakes Regional Conference, 2013.
- Schindel, W., "What Is the Smallest Model of a System?", Proc. of the INCOSE 2011 International Symposium, International Council on Systems Engineering (2011).
- Schindel, W., and Dove, R., "Introduction to the Agile Systems Engineering Life Cycle MBSE Pattern", in Proc. of INCOSE 2016 International Symposium, 2016.
- Schindel, W., "Improving Design Review", ICTT System Sciences, 2007. Servos, John W., Physical Chemistry from Ostwald to Pauling, Reprint Edition, Princeton University Press, 1996.
- Sussman, G, and Wisdom, J., Structure and Interpretation of Classical Mechanics, Cambridge, MA: MIT Press, 2001.
- Warfield, John N., An Introduction to Systems Science, World Scientific Publ., 2006.
- Westfall, Richard S., Never at Rest: A Biography of Isaac Newton, Cambridge, 1980.
- Wymore, A. Wayne, A Mathematical Theory of Systems Engineering: The Elements, Krieger, 1967.
-

Primary References

None.

Additional References

None.

Model-Based Systems Engineering (MBSE)

- Lead Authors:
- Caitlyn Singam and Jeffrey Carter

Model-based Systems Engineering [MBSE] is a paradigm that uses formalized representations of systems, known as models, to support and facilitate the performance of Systems Engineering [SE] tasks throughout a system's life cycle. MBSE is frequently contrasted with legacy document-based approaches where systems engineering captures system design information via multiple independent documents in various non-standardized formats. MBSE consolidates of system information in system design models, which provide primary SE artifacts. These system models, which are generally expressed in a standardized modelling language such as Systems Modeling Language [SysML®] express key system information in a concise, consistent, correct, and coherent format. When implemented properly, MBSE models permit the standardized consolidation and integration of system knowledge across engineering disciplines and subsystems and streamline key systems engineering tasks while also minimizing developmental risk.

This article provides an overview of key concepts underlying model-based approaches to systems engineering and highlights the benefits of utilizing MBSE on projects.

System Models

During the systems engineering process, a substantial amount of information is collected, generated, and/or maintained regarding the characteristics of the system(s) of interest, composite elements, and interacting entities/environments. MBSE utilizes models as a means of aggregating and managing these disparate pieces of information about a system in a centralized repository that can serve as a 'single source of truth' and technical baseline regarding a system of interest.

Definitions

Models are representations that are used to capture, analyze, and/or communicate information about a system or concept. They can vary in scope, purpose, and type, and can be utilized both individually as stand-alone entities as well as in concert with each other as part of an integrated set (Wymore 1993).

Properties of System Models

A model can be described and classified with respect to the following properties:

- **Scope:** the range of relevance of a model. Models can range from capturing the characteristics and interactions of a system's components (broad scope), to only focusing on the form and function of a single element in isolation (narrow scope).
 - **Domain:** the 'lens' through which the model views a system. Models can be holistic in nature or can focus on only highlighting information relevant to certain domains. Domain-specific models generally are used to highlight certain "perspectives" of a system, whether from the lens of a particular application sector (e.g., aerospace, biomedicine), discipline (e.g., electrical, mechanical, thermal), subsystem, or system property (e.g., power, reliability, fault management).
-

- **Formality:** the model's level of adherence to formalized standards for information expression. Models can express information about systems with varying degrees of precision. The most fundamental of models, which simply express a basic representation of a system in an unspecified format, do not convey information with precision and are considered informal. The most formal models comply with well-developed, pre-defined standards (formalisms) for content and organization, which collectively define 'languages' that enable consistent and precise interpretations of models.
- **Abstraction:** the degree to which a model suppresses or excludes out-of-scope, unimportant, or irrelevant details. Abstraction is a necessity with large and complex systems where it is impractical to replicate every aspect of a given system within a reasonable time and resource expenditure margin.
- **Physical/conceptual:** whether the model is concrete in nature (i.e., a physical model) or fully conceptual (i.e., an abstract model).
- **Descriptive/analytical:** whether a model details qualitative aspects of a system such as requirements, behaviors, or physical architecture (descriptive model), provides a representation of quantitative aspects of the system such as mass, reliability, power consumption via mathematical relationships (analytical model), or both (hybrid model).
- **Fidelity:** the degree to which a model comprehensively captures details about a system's characteristics, ranging from models which only capture general information about a system to those which seek to faithfully capture as much detail about the system as possible.
- **Completeness:** the extent to which a model captures all relevant domain- information within its scope and at its intended level of detail.
- **Integration:** the extent to which a model interacts and interfaces with other relevant models describing the system of interest or other related/interacting entities.
- **Quality:** the degree to which the model (not the system it represents) meets the needs of the individuals performing systems engineering activities. A high-quality model should be readily usable, have minimal ambiguity, and provide accurate, relevant information needed to support tasks associated with the design, development, operation, and/or maintenance of a system.

Of these properties, formalization and abstraction are generally the most frequently discussed in relation to MBSE (Vogelsang et al. 2017) as they have the greatest impact on whether a model can be effectively used as part of an MBSE workflow.

Criteria for Effective MBSE Models

While a successful MBSE workflow can involve the use of several different interconnected or standalone models of various scopes and types based on user needs, the main system model in an MBSE projects generally should have the following characteristics:

1. A scope which matches the scope of the project (i.e., it should encompass the entire system of interest);
 2. Representative of a holistic perspective from all relevant domains.
 3. Strict compliance with a previously established standardized modeling language, whether that be an existing language such as SysML® or a custom formalism.
 4. Fully abstracted, to only include relevant information appropriate for the system of interest and its desired use-case(s).
 5. Conceptual in nature, to permit the capture of intangible information (e.g., system requirements)
 6. Containing a description of the system functional and structural architecture at minimum and supplemented by integrated analytical/quantitative property descriptions as needed.
 7. Demonstrating sufficient fidelity to capture relevant system elements and behavior.
 8. Fully complete given its scope.
 9. Integrated with any necessary auxiliary models.
 10. Sufficiently high-quality as to meet the needs of those designing, developing, or otherwise working on the system.
-

In terms of content, effective system models are expected to capture key system information regarding requirements, system functionality/behavior, structure/form, properties, and interconnections between system components.

Modeling Languages

Modeling languages are specifications which provide standardized guidelines and structures for expressing system information. These languages, which provide both the structures or 'syntax' in which the information can be expressed, as well as the 'semantics' that govern the way in which the information should be interpreted, can be selected based on user preferences and needs. Different languages utilize different formats to express information (e.g., visual or textual means), as well as different paradigms (e.g., object-oriented, functional, etc.) in order to group information. Visual languages are generally preferred for modeling due to being readily readable, and object-oriented modeling languages are frequently used in systems engineering contexts since they readily lend themselves to systems which can be decomposed, or otherwise thought of, in terms of objects.

SysML®, an extension of Unified Modeling Language [UML] for systems engineering, is a one of the more frequently used modeling languages for MBSE. It is a graphical language that utilizes diagrams and tables in order to express system information, and provides a standard set of nine diagram types which can be used to organize and express system information (Friedenthal, Moore, and Steiner 2014). The collective diagrams (each of which can be considered a model in its own right), when interconnected, provide a means of representing system structure, behavior, and requirements in abstracted form. A number of other options have been proposed as architecture description languages [ADLs] for specifically modeling system architectures. ISO/IEC/IEEE 42010 (Systems and software engineering - Architecture description) specifies minimum requirements for a language to qualify as an ADL (ISO 2011).

MBSE users have the option of using SysML®, a similar graphical-language option like UML, a domain or framework specific language, or potentially developing a custom formalism for their team or organization (Bonnet et al. 2016). It is possible to formalize textual documents to create models, though doing so requires the establishment of a domain dictionary in order to remove the ambiguity inherent in diction choice, as well as the use of rigid grammatical structures which may limit readability.

Regardless of what modeling language is used for an MBSE project, it is important that the language be inherently scalable, standardized, readable, reusable, and abstractable to enable the development of effective MBSE models.

Architecture Frameworks

A second layer of structure that exists overtop a modeling language is an architecture framework. Architecture frameworks are used to organize the information expressed via modeling language. Whereas a modeling language provides the structure needed to express multiple 'views' (diagrams) of system elements and their interactions, architecture frameworks enable the user to group those views based on the elements they represent, and organize them in a way that allows traceability, eases navigation through the model, and aids in the identification of missing information (e.g., an omitted element). Architecture frameworks are a specific type of pattern that frequently get defined and standardized for MBSE models. There are also organization- and domain-specific design patterns that can be employed in MBSE models to meet stakeholder needs in more specific model use-cases.

Architecture frameworks and model design patterns play an important role in enabling the re-use of MBSE models (Wu et al. 2019), as certain architectural design patterns may be frequently used across multiple projects even when the specifications of the individual components differ (e.g. building a house with the same structure but different décor). By organizing a system model in a sufficiently abstracted manner, it may be possible to identify the points of difference between an old project and a new one and make the appropriate changes to element properties in the model without having to redo the entire model development process.

Process Frameworks

The MBSE model development workflow can be streamlined using pre-defined process frameworks, which provide tailorable guidelines and patterns for integrating MBSE into the generic systems engineering process. While process frameworks are typically defined on an organizational level, they generally all exhibit some form of configuration management process, access guidelines, practices for updating the model, and means of integrating the MBSE model into all or nearly all systems engineering lifecycle activities. The benefits of MBSE usage are limited when the system model falls out of date or otherwise becomes inaccurate, so regular model updates are a minimum requirement for MBSE process frameworks.

For smaller projects, the MBSE process framework may be as simple as utilizing the version control features that come included as part of many collaborative modeling software platforms and integrating model usage and periodic updates as checkpoints in the systems engineering process. More complex projects can formalize MBSE process frameworks in a manner that can be verified against configuration management and systems engineering management plans (Fisher et al. 2014).

Benefits of MBSE

The MBSE workflow and the creation of a centralized system model emphasizes a holistic, standards-based approach to systems engineering (Madni and Sievers 2018). Since the creation of a system model requires reconciliation of information from multiple domains and subsystems, inconsistencies and defects are readily identifiable during the modeling process (Carroll and Malins 2016) and can be addressed or eliminated earlier on in the system lifecycle process than would otherwise be done in a document-based workflow. Similarly, the centralization and standardization of information ensures a reduction in miscommunications and other development risks since all project team members are using the same source of information for reference. Format standardization also makes it easier to search for and extract information, compared to a document-based workflow where information is stored across multiple documents in different formats.

More broadly, MBSE provides a better means of managing complexity than document-based using formalized structures and abstraction. Cross-referencing within MBSE models makes it possible to begin design verification, requirements validation, and systems assurance earlier on in the system lifecycle, and to continue assessing system design quality throughout a project at minimum cost. Furthermore, models can be reused and adapted for similar systems, which enables accelerated system development with minimal risk.

Digital Transformation

While DBSE has traditionally been the paradigm of preference for artifact generation and for supporting systems engineering efforts in the pre-digital age, digital transformation of the generic systems engineering workflow in recent years has catalyzed the widespread adoption of MBSE and broader model-based [MBx] approaches. Digital environments and software tools have made it easier and faster to generate, maintain, and use system models, especially in a collaborative setting (Ma et al. 2022). If implemented appropriately, digital MBSE models can be used to programmatically identify inconsistencies, enable interactive simulations of system behavior, simultaneously propagate changes across an entire project (rather than updating artifacts one-by-one), automatically generate document-based artifacts, and more. The advent of new software for supporting and automating systems engineering tasks has opened additional avenues for expanding the capabilities of system models, and for increasing the efficiency with which systems engineering tasks can be performed.

Digital Twins

When MBSE models of physical systems are built with sufficient completeness and fidelity, it is possible for them to function as ‘digital twins’ of the systems they represent. Digital twins provide a means of accurately representing a system’s form and function throughout the system’s lifecycle, all within a digital environment. Creating such digital twins provides number of advantages, including allowing individuals to perform testing, analysis, and optimization of systems in a virtual environment at no risk to the actual system of interest and often at a greatly reduced cost/burden (Schluse, Atorf, and Rossmann 2017). Digital twins also make it possible to represent the behavior of systems under conditions which would be impractical or impossible to induce under experimental conditions, thereby making it possible to obtain information not obtainable via study of the original physical system.

MBSE versus DBSE

Although MBSE and document-based approaches are usually presented as alternatives to each other, it is possible to use MBSE and document-based in conjunction with each other on the same project. In work environments where document-based is the norm, stakeholders may expect or require the submission of textual document artifacts, or there may be issues with a lack of familiarity with any modeling languages (Kim, Wagner, and Jimenez 2019); in such instances, it may be necessary to utilize a hybrid approach where documents are generated from the design model as static representations of the system for project milestones.

References

Works Cited

- Bonnet, Stéphane, Jean-Luc Voirin, Daniel Exertier, and Véronique Normand. 2016. “Not (Strictly) Relying on SysML for MBSE: Language, Tooling and Development Perspectives: The Arcadia/Capella Rationale.” In 2016 Annual IEEE Systems Conference (SysCon), 1–6. <https://doi.org/10.1109/SYSCON.2016.7490559>.
- Carroll, Edward Ralph, and Robert Joseph Malins. 2016. “Systematic Literature Review: How Is Model-Based Systems Engineering Justified?.” SAND2016-2607, 1561164. <https://doi.org/10.2172/1561164>.
- Fisher, Amit, Mike Nolan, Sanford Friedenthal, Michael Loeffler, Mark Sampson, Manas Bajaj, Lonnie VanZandt, Krista Hovey, John Palmer, and Laura Hart. 2014. “3.1.1 Model Lifecycle Management for MBSE.” INCOSE International Symposium 24 (1): 207–29. <https://doi.org/10.1002/j.2334-5837.2014.tb03145.x>.
- Friedenthal, Sanford, Alan Moore, and Rick Steiner. 2014. A Practical Guide to SysML: The Systems Modeling Language. Morgan Kaufmann. ISO. 2011. “ISO/IEC/IEEE 42010.” Geneva, Switzerland: International Organization for Standardization (ISO). <https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/05/05/50508.html>.
- Kim, So Young, David Wagner, and Alejandro Jimenez. 2019. “Challenges in Applying Model-Based Systems Engineering: Human-Centered Design Perspective,” September. <https://trs.jpl.nasa.gov/handle/2014/51368>.
- Ma, Junda, Guoxin Wang, Jinzhi Lu, Hans Vangheluwe, Dimitris Kiritsis, and Yan Yan. 2022. “Systematic Literature Review of MBSE Tool-Chains.” Applied Sciences 12 (7): 3431. <https://doi.org/10.3390/app12073431>.
- Madni, Azad M., and Michael Sievers. 2018. “Model-Based Systems Engineering: Motivation, Current Status, and Research Opportunities.” Systems Engineering 21 (3): 172–90. <https://doi.org/10.1002/sys.21438>.
- Schluse, Michael, Linus Atorf, and Juergen Rossmann. 2017. “Experimentable Digital Twins for Model-Based Systems Engineering and Simulation-Based Development.” In 2017 Annual IEEE International Systems Conference (SysCon), 1–8. <https://doi.org/10.1109/SYSCON.2017.7934796>.
- Vogelsang, Andreas, Tiago Amorim, Florian Pudlitz, Peter Gersing, and Jan Philipps. 2017. “Should I Stay or Should I Go? On Forces That Drive and Prevent MBSE Adoption in the Embedded Systems Industry.” In

Product-Focused Software Process Improvement, edited by Michael Felderer, Daniel Méndez Fernández, Burak Turhan, Marcos Kalinowski, Federica Sarro, and Dietmar Winkler, 182–98. Cham: Springer International Publishing. https://doi.org/10.1007/978-3-319-69926-4_14.

Wu, Quentin, David Gouyon, Sophie Boudau, and Éric Levrat. 2019. "Capitalization and Reuse with Patterns in a Model-Based Systems Engineering (MBSE) Framework." In 2019 International Symposium on Systems Engineering (ISSE), 1–8. <https://doi.org/10.1109/ISSE46696.2019.8984571>.

Wymore, A. Wayne. 1993. Model-Based Systems Engineering: An Introduction to the Mathematical Theory of Discrete Systems and to the Tricotyledon Theory of System Design. Boca Raton: CRC Press.

Primary References

Estefan, J. 2008. A Survey of Model-Based Systems Engineering (MBSE) Methodologies, Rev. B. San Diego, CA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02. Available at: https://www.incose.org/ProductsPubs/pdf/techdata/MTTC/MBSE_Methodology_Survey_2008-0610_RevB-JAE2.pdf.

INCOSE. 2021. Systems Engineering Vision 2035. Torrance, CA, USA: International Council on Systems Engineering.

OMG. "MBSE Wiki." Object Management Group (OMG). Available at: <https://www.omgwiki.org/MBSE/doku.php>. Accessed 05 April 2022.

Additional References

Downs, E., P. Clare, and I. Coe. 1992. Structured Systems Analysis and Design Method: Application and Context. Hertfordshire, UK: Prentice-Hall International.

INCOSE. 2007. Systems Engineering Vision 2020. Seattle, WA, USA: International Council on Systems Engineering. September 2007. INCOSE-TP-2004-004-02.

Kossiakoff, A. and W. Sweet. 2003. "Chapter 14," in Systems Engineering Principles and Practice. New York, NY, USA: Wiley and Sons.

NDIA. 2011. Final Report of the Model Based Engineering (MBE) Subcommittee. Arlington, VA, USA: National Defense Industrial Association. Available at: [https://www.ndia.org/Divisions/Divisions/SystemsEngineering/Documents/Committees/M_S%20Committee/Reports/MBE_Final_Report_Document_\(2011-04-22\)_Marked_Final_Draft.pdf](https://www.ndia.org/Divisions/Divisions/SystemsEngineering/Documents/Committees/M_S%20Committee/Reports/MBE_Final_Report_Document_(2011-04-22)_Marked_Final_Draft.pdf)

Oliver, D., T. Kelliber, and J. Keegan. 1997. Engineering Complex Systems with Models and Objects. New York, NY, USA: McGraw-Hill.

Part 3a: Life Cycle Concepts

Knowledge Area: Life Cycle Terms and Concepts

Life Cycle Terms and Concepts

Contents of this Knowledge Area

- Life Cycle Concepts (Mike Yokell) (David Endler and Garry Roedler)
- Life Cycle Models (Mike Yokell) (David Endler and Garry Roedler)
- Life Cycle Stages (Mike Yokell) (David Endler and Garry Roedler)
- Technical Reviews and Audits (Ken Garlington) (David Endler, Garry Roedler, and Mike Yokell)
- Lead Author:
- Mike Yokell
- Contributing Authors:
- David Endler and Garry Roedler

A life cycle is the evolution of a system, product, service, project or other human-made entity from conception through retirement. (ISO/IEC/IEEE 24748-1:2024) It can be helpful to create models to depict and manage the progression of the entity from beginning to end. These models are called life cycle models. A stage is a period within the life cycle of an entity that relates to the state of its description or realization. Stages relate to major progress, achievement milestones, or decision points of the entity through its life cycle. Technical reviews and audits are a mechanism by which sufficiently independent and knowledgeable stakeholders analyze the current state of a system, work product, or set of work products using pre-established criteria.

References

Works Cited

ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers.

Primary References

None.

Additional References

None.

Life Cycle Concepts

- Lead Author:
- Mike Yokell
- Contributing Authors:
- David Endler and Garry Roedler

A life cycle is the evolution of a system, product, service, project or other human-made entity from conception through retirement. (ISO/IEC/IEEE 24748-1:2024) A stage is a period within the life cycle of an entity that relates to the state of its description or realization. Stages relate to major progress, achievement milestones, or decision points of the entity through its life cycle. It can be helpful to create models to depict and manage the progression from beginning to end. These models are called life cycle models.

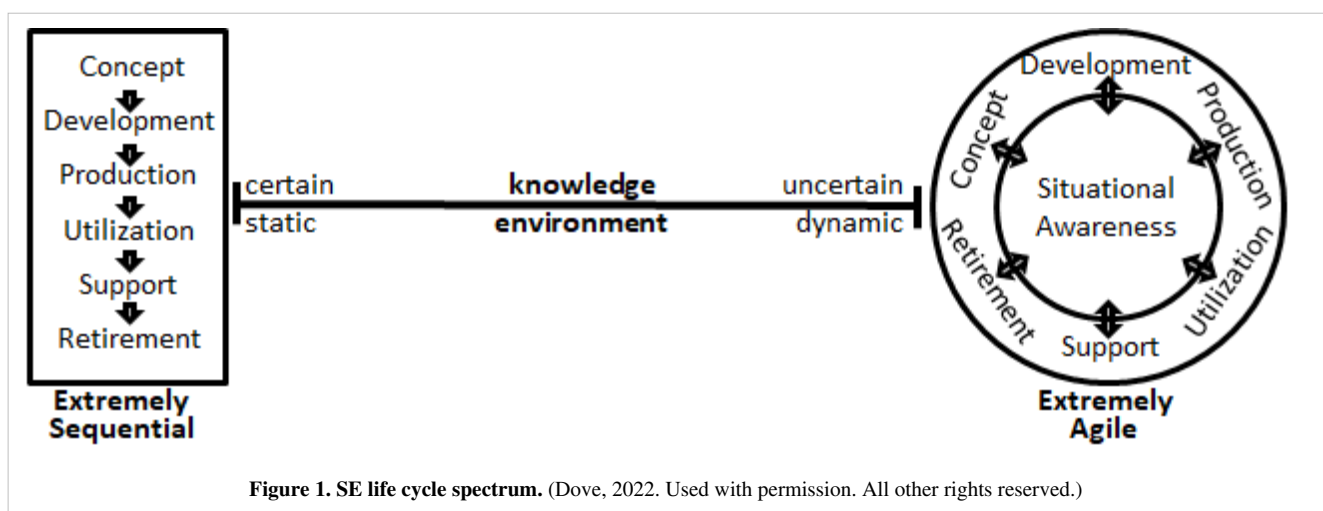
Basic terms and concepts

A life cycle is the evolution of a system, product, service, project or other human-made entity from conception through retirement. (ISO/IEC/IEEE 24748-1:2024)

It can be helpful to create models to depict and manage the progression of an entity from beginning to end. These models are called life cycle models. A life cycle model is a framework of processes and activities concerned with the life cycle which can be organized into stages, acting as a common reference for communication and understanding (ISO/IEC/IEEE 24748-1:2024) See “Knowledge Area – Life Cycle Model Selection and Adaptation” for further elaboration.

A stage is a period within the life cycle of an entity that relates to the state of its description or realization. Stages relate to major progress, achievement milestones, or decision points of the entity through its life cycle. Stages often overlap. (ISO/IEC/IEEE 24748-1:2024) See “Article – Life Cycle Stages” for more information about stages, including entry and exit criteria for progressing between stages, and “Article – Technical Reviews and Audits” for assessing progress within stages.

Figure 1 shows a spectrum of life cycle approaches, depending on the level of uncertainty. All life cycle approaches fall somewhere between the two ends of the spectrum, with neither end of the spectrum being common in practice.



References

Works Cited

INCOSE. 2023. "Chapter 2.1.1" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5> ^[1].

INCOSE. 2023. "Chapter 4.2.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5> ^[1].

ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers.

Primary References

None.

Additional References

None.

References

[1] <https://www.incose.org/publications/se-handbook-v5>

Life Cycle Models

- Lead Author:
- Mike Yokell
- Contributing Authors:
- David Endler and Garry Roedler

A life cycle is the evolution of a system, product, service, project or other human-made entity from conception through retirement. It can be helpful to create models to depict and manage the progression of an entity from beginning to end. These models are called life cycle models.

Life Cycle Models

It can be helpful to create models to depict and manage the progression of an entity from beginning to end. These models are called life cycle models. A life cycle model is a framework of processes and activities concerned with the life cycle which can be organized into stages, acting as a common reference for communication and understanding [SOURCE: ISO/IEC/IEEE 24748-1:2024]

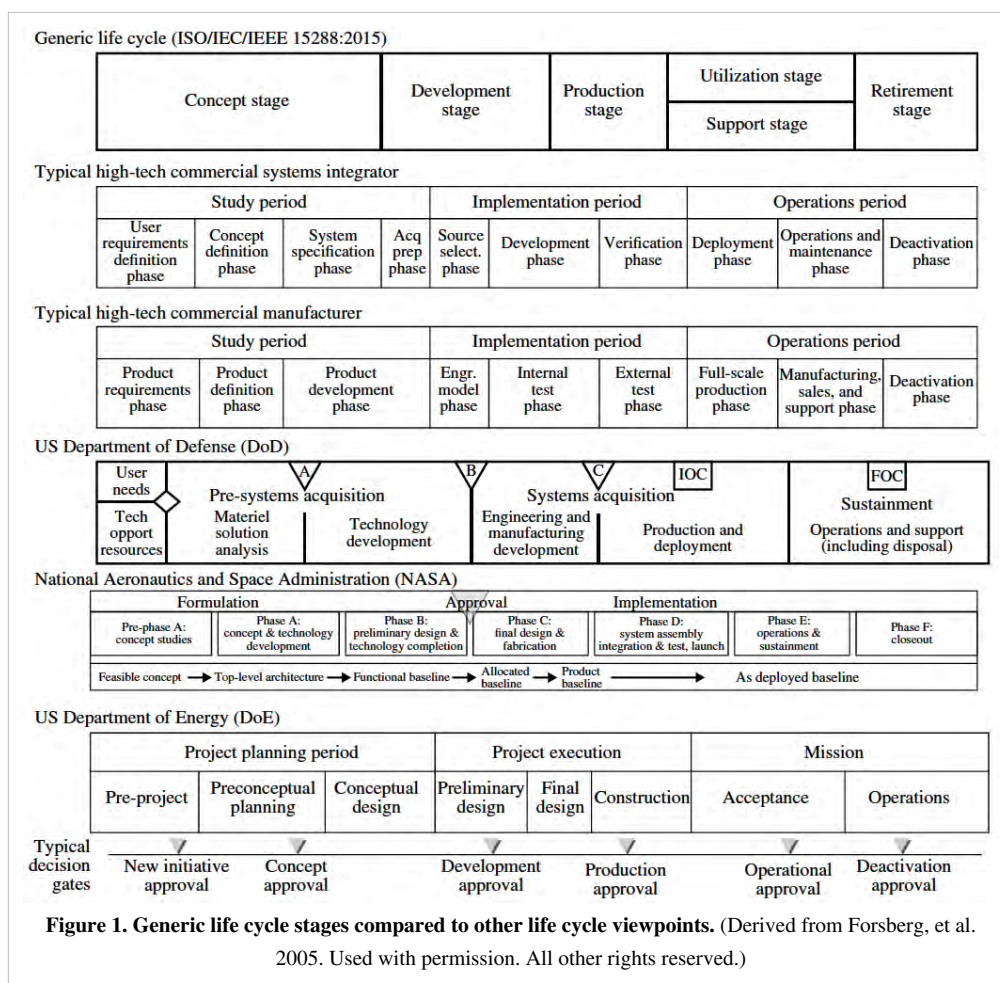
Movement between stages represents a decision point with specific criteria related to stage entry and exit. See Article – Life Cycle Stages for more information. Life cycle models provide a decision-linked segmentation of the conceptualization, development, production, utilization, support, and retirement of the system and its elements.

Any system of interest can be regarded from the perspective of one or more life cycle models. Some life cycle models permit iteration and concurrency of stages while some do not. Which stage(s) a system of interest is in depends on the life cycle model. A system of interest can be in multiple life cycle stages at the same time.

Four common principles associated with a life cycle model are the following:

- a system progresses through specific stages during its life;
- enabling systems should be available for each stage to achieve the outcomes of the stage;
- at specific life cycle stages, quality characteristics such as producibility, usability, supportability and disposability should be specified and designed or implemented into a system;
- stages begin and end based on criteria or external events.

Figure 1 shows models with stage names depending on the context. However, note that some of the models are project models, not life cycle models. That is, they model the phases of a project, not the stages of the life cycle of a system.



Example

Depending on the life cycle model, stages can occur in parallel and multiple times. Progression through the stages of a life cycle can be depicted in many ways. In Figure 2, the concept stage is short, development and production are longer, while utilization, support, and retirement are lengthy.

In this model for a system of interest where multiple units were produced, utilization and support stages began with the first unit. For the software element, support began during early development, well before delivery to operations. Retirement began when the first unit retired or was lost to damage. In this example the development approach envisioned a mid-life upgrade, so the concept, development and production stages were restarted. Consequently, the support stage ran parallel to utilization until the last unit was removed from service. Support ended while units were still being utilized. The retirement stage continued until the last unit was retired. It is not always possible for the initial developer to know if units are still being utilized.

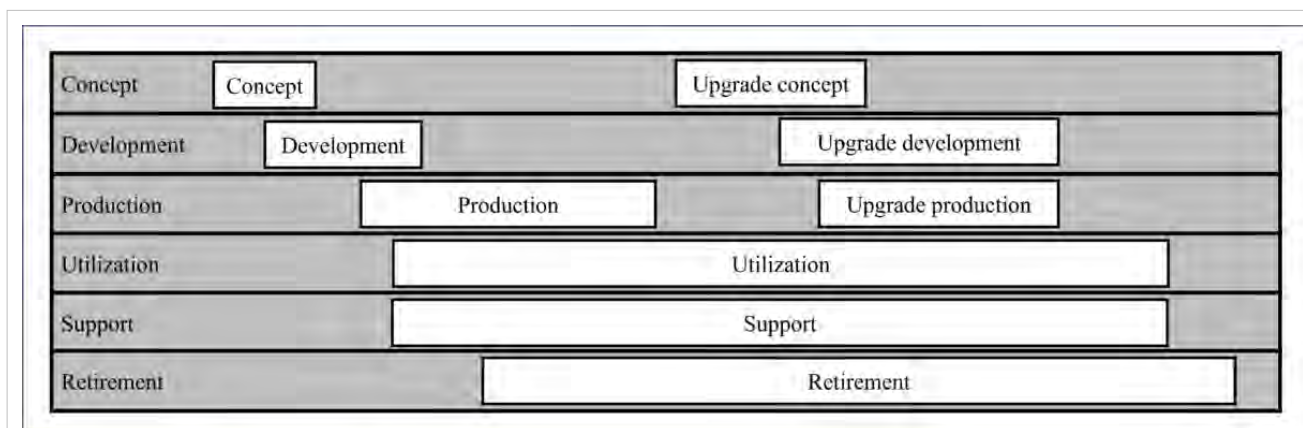


Figure 2. System life cycle stages. (Yokell. Used with permission. All other rights reserved.)

Many life cycle models depict the stages as left to right. DevOps (and DevSecOps) includes a life cycle model that depicts the life cycle stages in an infinity loop as shown in Figure 3. In this model, there is no distinction between development and support; that is, the work is never “done”. The model also assumes that the risks and effort associated with retirement are negligible, and thus it does not need to be a focus of the model.

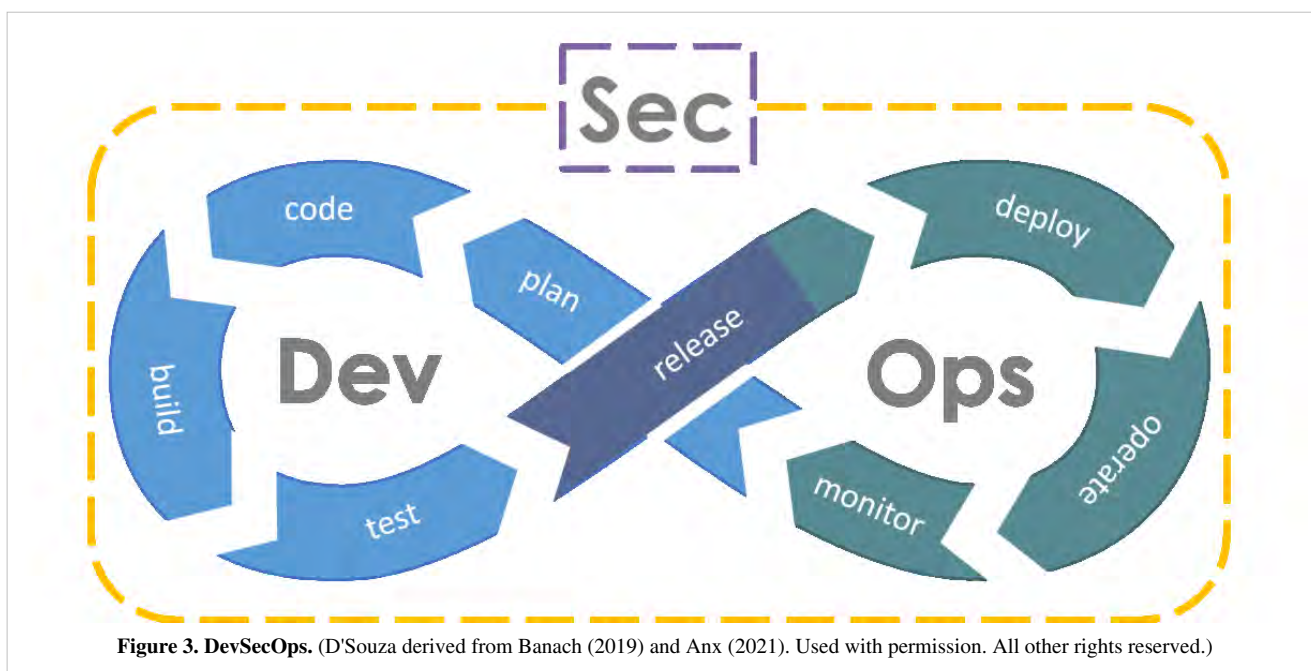


Figure 3. DevSecOps. (D'Souza derived from Banach (2019) and Anx (2021). Used with permission. All other rights reserved.)

Life Cycle Processes

The processes and activities that are integrated into the framework of a life cycle model are often referred to as life cycle processes. They can be seen as one of the enablers to help manage a system solution across the life cycle stages. In some life cycle models the life cycle processes can be applied concurrently, iteratively, and recursively with other enablers (e.g., tools, technology) throughout the stages of the life cycle.

ISO/IEC/IEEE 15288, SAE 1001 and INCOSE SE Handbook define system life cycle processes. In SEBoK, these life cycle processes can be found in KAs Technical Management Processes, System Concept Definition, System Architecture Design Definition, and System Maintenance.

In most cases, organization-specific processes are defined for the life cycle phases in which an organization is usually heavily involved in projects. Organizations that develop systems “from scratch”, often have detailed processes to navigate the SoI through the stages Concept and Development. In case the manufacturing of the SoI is

outsourced, the implementation process is rather thin (it cannot be removed as this process is invoked to identify constraints on the SoI related to implementation) and when the organization manufactures the SoI, the implementation process is more detailed.

References

Works Cited

INCOSE. 2023. "Chapter 2.1.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5> ^[1].

ISO/IEC/IEEE 15288. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/81702.html> ^[1].

ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html> ^[2].

Primary References

None.

Additional References

None.

References

[1] <https://www.iso.org/standard/81702.html>

[2] <https://www.iso.org/standard/84709.html>

Life Cycle Stages

- Lead Author:
- Mike Yokell
- Contributing Authors:
- David Endler and Garry Roedler

A life cycle is the evolution of a system, product, service, project or other human-made entity from conception through retirement. (ISO/IEC/IEEE 24748-1) A stage is a period within the life cycle of an entity that relates to the state of its description or realization. Stages relate to major progress, achievement milestones, or decision points of the entity through its life cycle.

Typical Life Cycle Stages

A system progresses through various life cycle stages that span conception, development, production, utilization, support, and retirement. This naming of stages can be found in (ISO/IEC/IEEE 24748-1), (ISO/IEC/IEEE 15288), and the INCOSE *Systems Engineering Handbook* (2023). Within each stage, the primary goals are:

- Concept Stage - Identify stakeholders' needs. Explore concepts. Propose viable solutions
- Development Stage - Refine system requirements. Create solution description. Build system. Perform system verification and system validation
- Production Stage - Produce systems. Inspect and test
- Utilization Stage - Operate system to satisfy users' needs
- Support Stage - Provide sustained system capability
- Retirement Stage - Store, archive or dispose of system

Movement between stages represents a milestone or decision point with specific criteria related to stage entry and exit. Life cycle models provide a decision-linked segmentation of the conceptualization, development, production, utilization, support, and retirement of the system. Any system of interest can be regarded from the perspective of one or more life cycle models. Some life cycle models permit iteration and concurrency of stages while some do not. Which stage(s) a system of interest is in depends on the life cycle model. A system of interest can be in multiple life cycle stages at the same time.

Entry and Exit Criteria

Entry or exit criteria can be met (diamonds in Figure 1) before the decision, concurrent with the decision, after the decision, or not at all (triangles). Decision criteria can include an assessment of technical debt and its implications for other stages. The existence of technical debt in the system negatively impacts design suitability. Technical debt can accumulate in any life cycle stage.

A system life cycle model can be segmented by stages to facilitate planning for the system of interest. Decision-making reviews or gates can be used to inform the progression between stages to reduce risk and to enable satisfactory progress. These gates can be based on specific entry or exit criteria leading to informed decisions that can facilitate consistent outcomes as the system transitions between stages. It is important to note that not all parts of an SOI will reach the same maturity and therefore decision gate at the same time. Therefore, interim maturity reviews are essential to keep timely awareness. Also, it is necessary to be clear about tailoring the entry/exit criteria - one size does not fit all.

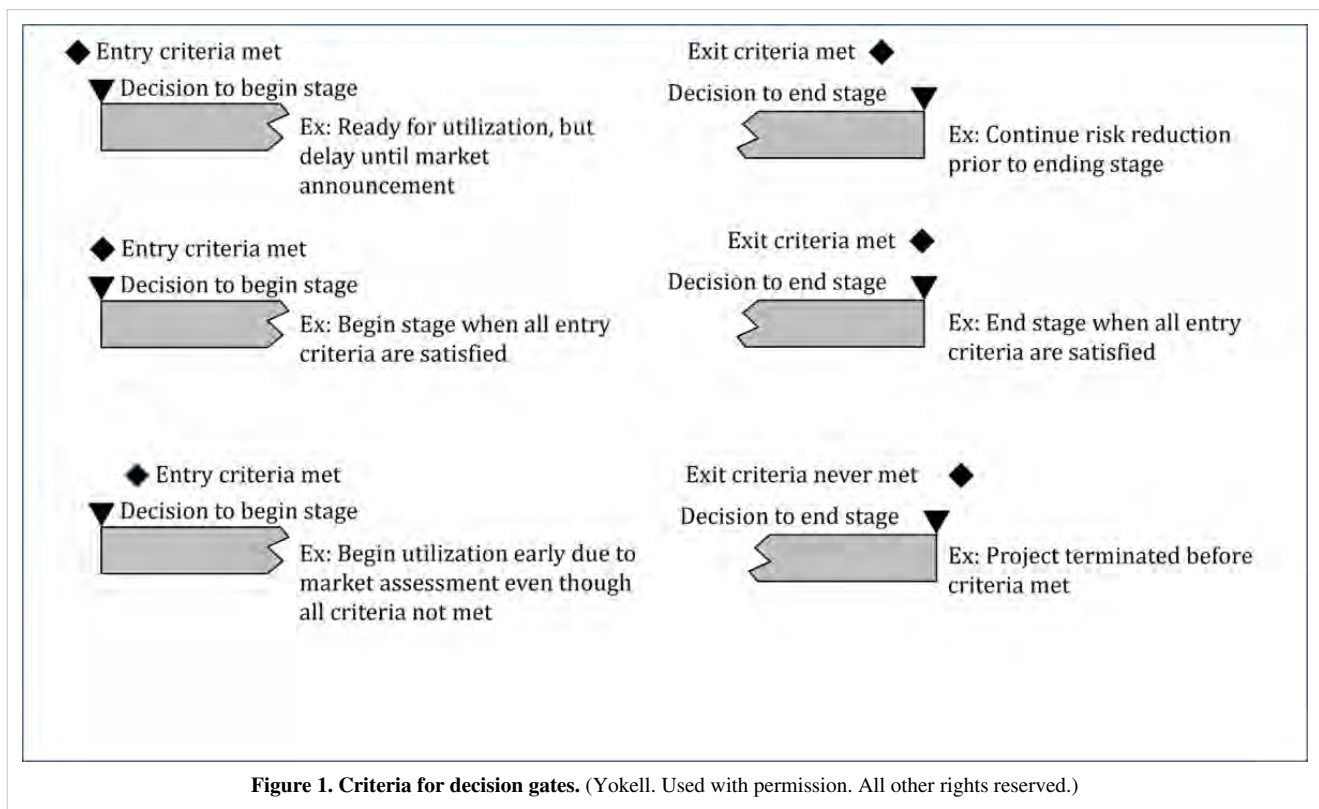
At a decision gate, the actual progress is evaluated against criteria expected for each stage or for an increment of the system that can be independent of other increments. Generally, there are two elements to the review: looking backwards at the progress to date and looking forwards to the preparedness to continue. Criteria determine when a

stage can be exited and when other appropriate stages can be begun. Technical Reviews and Audits provides additional information. ISO/IEC/IEEE 24748-8 provides more rigorous requirements for technical reviews and audits in support of decision gates for defense projects but can be applied to other types of systems.

In two-party acquisition situations, joint stakeholder reviews between the parties to the agreement can be needed. These joint reviews can support the decision gates between stages.

Decision Options for each stage:

- Begin another stage or stages
- Continue this stage
- Go to or restart another stage
- Pause activity
- Terminate



References

Works Cited

- INCOSE. 2023. "Chapter 2.1.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5> ^[1].
- ISO/IEC/IEEE 15288. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/81702.html> ^[1].
- ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org>.

[org/standard/84709.html](https://www.iso.org/standard/84709.html) ^[2].

Primary References

None.

Additional References

None.

Technical Reviews and Audits

- Lead Author:
- Ken Garlington
- Contributing Authors:
- David Endler, Garry Roedler, and Mike Yokell

Technical reviews and audits are a mechanism by which sufficiently independent and knowledgeable stakeholders analyze the current state of a system, work product, or set of work products using pre-established criteria. The results of these analyses are used to support programmatic and technical decisions.

Concepts

A technical review is a series of systems engineering activities conducted at logical transition points in a system life cycle, by which the progress of a project is assessed relative to its technical requirements using a mutually agreed-upon set of criteria (ISO/IEC/IEEE 24748-8:2019). An audit is an independent examination of a work product or set of work products to assess compliance with specifications, standards, contractual agreements, or other criteria (ISO/IEC/IEEE 15288:2023). Different organizations, functional disciplines, etc. can use either term or other terms for these related concepts.

Technical reviews and audits support multiple systems engineering processes, including:

- Project assessment and control. Technical reviews and audits help provide visibility into the project's technical progress and risks, and support decision gates by establishing readiness to proceed to the next milestone or life cycle stage.
- Configuration management. Technical reviews and audits support establishing, revising, and verifying configuration baselines.
- Validation. Through the involvement of appropriate stakeholders (including end users), technical reviews and audits enable early validation of requirements, design, and other work products.

Technical reviews and audits can be applied to a system, system element, or other established technical boundary. In addition to their use at transition points in a system life cycle, they can also be used to authorize risky activities, such as testing with safety or security concerns.

Approach

A typical approach to a technical review or audit includes preparing, conducting, and completing it.

Prepare

Technical reviews and audits assessing the progress of a project typically include:

- Business case or mission.
- Budget.
- Technical maturity, including associated risks.

These must be balanced to ensure that business and technical baselines are acceptable and will lead to satisfactory verification and validation. The decision as to whether or not to proceed is sometimes associated with great uncertainty. For this reason, it is advisable to properly document the assumptions made and the risks identified.

Technical planning is used to define and coordinate the elements of each technical review or audit, including:

- technical review or audit name,
- purpose and expected outcomes,
- scope (project, system, system element, etc.)
- relevant properties of the system, etc. to be assessed,
- timing and frequency, and
- specific activities and tasks.

To ensure objective and comprehensive results that support the expected outcomes, specific entry and exit criteria should be defined and coordinated, and the completion of the entry criteria confirmed, prior to conducting a technical review and audit (see also Life Cycle Stages and Entry/Exit Criteria). In addition, any necessary independence and areas of expertise required should be considered in the selection and coordination of subject matter experts (SMEs) as participants, including representatives from specialty engineering.

For some key stakeholders, particularly for senior technical and programmatic personnel responsible for a portfolio of projects, a technical review or audit can be a primary method of participating in a project. Planning should consider the needs of these and other stakeholders with respect to specific scheduling, coordination, presentation of project context, etc. Roles and responsibilities for each participant should be coordinated prior to conducting the technical review or audit. Most technical reviews and audits require a set of interrelated information items to be assessed for consistency, completeness, etc. The specific configuration of the information items in the set should be identified and made available to participants as needed prior to conducting the technical review or audit.

According to Boehm and Lane, there are three different types of technical reviews:

Review type	Examples	Considerations
Schedule-based	Explicit dates in agreement	• Opportunities: early and frequent feedback • Risks: insufficient information to meet review objectives, failure to make progress
Event-based	End of development phase (e.g. design) based on delivery of artifacts as defined in agreement	• Opportunities: More complete information available • Risks: Potential for insufficient analysis, unresolved technical issues and risks
Evidence-based	Achieving a specific technical risk level based on supplied evidence	• Opportunities: Higher chance of stakeholder commitment due to more complete risk understanding • Risks: Late feedback on latent defects, etc. with resulting cost, schedule, and technical impacts.

For all of these, there can be misunderstandings or adverse drivers (e.g. schedule, cost) that cause the technical review to be held with insufficient preparation, information, etc.

Conduct

Participants support the purpose and expected outcomes of the technical review or audit through the analysis of relevant information using pre-established criteria. The information may be available as documents, models, simulations, prototypes, or other appropriate formats. Usually, different participants will perform their analysis from different perspectives (safety, security, reliability, etc.) consistent with their expertise. Participants may perform their work independently or as a group. Typically, all participants will meet to discuss analysis results and identify actions based on the results.

In many cases, it can be particularly challenging to identify the right people who are able to objectively assess the outcome of a review. If these people are “too close” to the project, they may be biased (e.g., miss out on the next bonus). If these people are “too far away” from the project, critical issues may not be discovered. Thus, it is most effective to build a review team with a combination of people close enough to provide important insights and some people who are far enough removed to have independent viewpoints.

Complete

Once the exit criteria have been met for the technical review or audit, the results are communicated to the participants (and other stakeholders unable to participate), and any identified actions are implemented as needed to support the desired project outcomes.

Timing and Frequency

Technical reviews and audits can be applied to both a system of interest and recursively to constituent system elements as needed. They can occur sequentially, iteratively, incrementally or concurrently. In any case, they should not be conducted until the pre-defined entry criteria have been met.

Representative Technical Reviews and Audits

Technical reviews and audits are defined based on the needs of the organization, domain, life cycle, etc. Although industry and organizational standards in this area can be useful, they should be carefully tailored for the needs of each project. Example applications are described in:

- United States Department of Defense (DoD) *Systems Engineering Guidebook* (Feb 2022),
- North Atlantic Treaty Organization (NATO) *System Life Cycle Processes* (AAP-48, May 2022),
- National Aeronautics and Space Administration *Systems Engineering Processes and Requirements* (NPR 7123.1D, Jul 2023), and
- Incremental Commitment Spiral Model

For additional information on applying technical reviews and audits, see ISO/IEC/IEEE 24748-8.

References

Works Cited

Boehm, B. and Lane, J., “Improved Acquisition Process Through Incremental Commitments,” Tutorial at the 13th Annual NDIA Systems Engineering Conference, The National Defense Industrial Association (NDIA), 2010.

US DoD. 2022. *Systems Engineering Guidebook*. Washington, DC: US Department of Defense. Available at: <https://www.dau.edu/sites/default/files/2024-02/Systems%20Engineering%20Guidebook%2C%20February%202022.pdf>^[1]

ISO/IEC/IEEE 15288. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of

Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/81702.html> ^[1].

ISO/IEC/IEEE 24748-8. 2019. *Systems and software engineering — Life cycle management, Part 8: Technical reviews and audits on defense programs, Edition 1*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/75405.html> ^[2].

NASA. 2023. *NASA Systems Engineering Processes and Requirements*, Updated with Change 2. Washington, DC: National Aeronautics and Space Administration (NASA). NPR 7123.1D. 05 July 2023. Available at: <https://nodis3.gsfc.nasa.gov/displayDir.cfm?t=NPR&c=7123&s=1B> ^[3]

NATO. 2022. "NATO System Life Cycle Processes." Brussels, Belgium: North Atlantic Treaty Organization (NATO). NATO-AAP-48. Available at: <https://standards.globalspec.com/std/14514800/aap-48> ^[4]

Primary References

None.

Additional References

None.

References

[1] <https://www.dau.edu/sites/default/files/2024-02/Systems%20Engineering%20Guidebook%2C%20February%202022.pdf>

[2] <https://www.iso.org/standard/75405.html>

[3] <https://nodis3.gsfc.nasa.gov/displayDir.cfm?t=NPR&c=7123&s=1B>

[4] <https://standards.globalspec.com/std/14514800/aap-48>

Knowledge Area: Development Approaches

Development Approaches

Contents of this Knowledge Area

- Development Approach Concepts (David Endler) (Mike Yokell and Garry Roedler)
- Sequential Development Approach (David Endler) (Mike Yokell and Garry Roedler)
- Incremental Development Approach (David Endler) (Mike Yokell and Garry Roedler)
- Evolutionary Development Approach (David Endler) (Mike Yokell and Garry Roedler)
- Agile Development Approach (David Endler) (Mike Yokell and Garry Roedler)
- Lean Engineering
- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

There are various approaches to properly manage the progression of the system of interest (SoI) through its life cycle stages as well as the planning for the other stages. From a systems engineering perspective, the concept and the development stage are of particular interest. For this reason, the articles in this Knowledge Area focus on these two stages. Even though these approaches cover both concept and the development stage, they are often referred to as development approaches. The main difference between the concept stage and the development stage is that the concept stage has a different or more limited set of stakeholders than the development stage.

As projects can have very different starting points, there are a variety of development approaches. In general, the approaches can be roughly divided into sequential, incremental and evolutionary approaches. The agile development approach can be seen as another type or a subset of the others. The boundaries between the groups are not clear-cut and there can be overlaps. The classification is mainly based on the criteria of the extent to which the requirements for the system of interest are known, whether there are increments or not, and what these increments are used for. It is important to keep in mind that development will always be concurrent and iterative to a certain degree.

References

Works Cited

None.

Primary References

None.

Additional References

None.

Development Approach Concepts

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

A life cycle is the evolution of a system, product, service, project or other human-made entity from conception through retirement. (ISO/IEC/IEEE 24748-1) A stage is a period within the life cycle of an entity that relates to the state of its description or realization. Stages relate to major progress, achievement milestones, or decision points of the entity through its life cycle. It can be helpful to create models to depict and manage the progression from beginning to end. These models are called life cycle models.

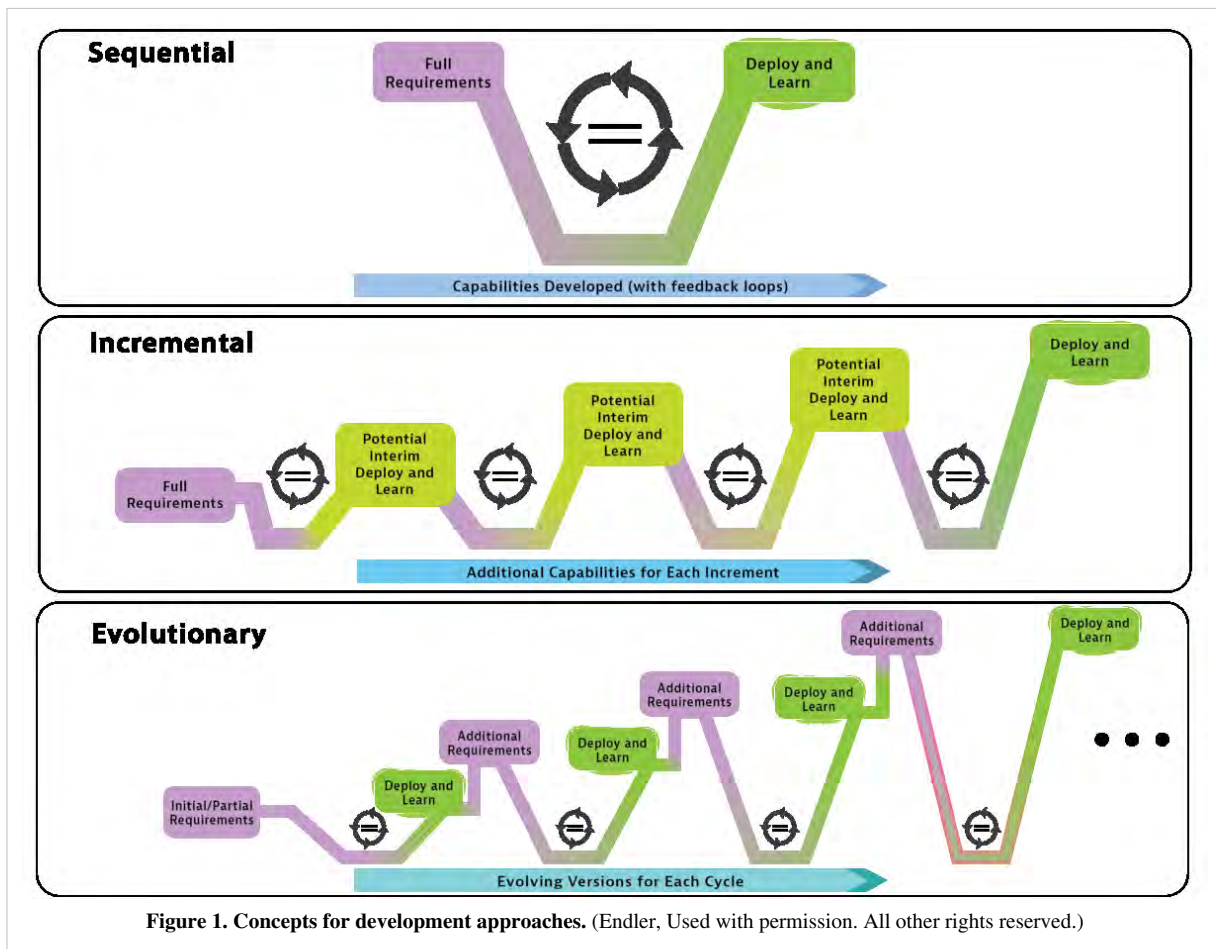
Basic terms and concepts

A life cycle model is a framework of processes and activities concerned with the life cycle. Each of the life cycle stages has its specific needs and goals. In order to support these, approaches should be in place to ensure continuous learning and improvement. In the context of systems engineering, the development stage is of particular importance. Therefore, this article elaborates on concepts for the development stage.

With the exception of services, the terms "waterfall," "incremental," and "evolutionary" almost always occur adjacent to "development," and almost never adjacent to the names of other life cycle stages (e.g. "waterfall production", "incremental utilization" or "evolutionary retirement". These adjectives apply only to the development stage and not to the life cycle as a whole.

As projects can have very different starting points, there are a variety of development approaches; several with combinations of features including sequential, incremental and evolutionary as shown in Figure 1 below. The boundaries between these groups are not clear-cut and there can be overlaps.

The classification into the groups is mainly based on the criteria of the extent to which the requirements for the system of interest are known, whether there are increments or not, and what these increments are used for. It is important to keep in mind that development will always be concurrent and iterative to a certain degree (which is never zero in practice) as is indicated in Figure 1 by the round symbol along the development stage. In practice, it is often the case that a system of interest consists of system elements for which different development approaches are recommended. This can be caused by most common approaches to hardware and software or even because it has differing delivery goals (e.g., ongoing research vs. in-service support).



An agile approach is an iterative and incremental approach to product and service delivery. DevOps (development and operations) is the set of principles and practices which enable better communication and collaboration between relevant stakeholders for the purpose of specifying, developing and operating software, and systems products and services, and continuous improvements in all aspects of the life cycle. Having similar objectives, agile and DevOps approaches are frequently combined.

Agile and DevOps practices are commonly associated with both incremental and evolutionary approaches.

Sequential approaches

Fully sequential approaches are purely theoretical and unfairly maligned because they don't appear in practice. Sequential development approaches typically consist of performing the development tasks in a specific order with some iteration to allow learning and change. Information needs to flow between stages to manage risk. Sequential approaches are intuitive, they have well-defined processes, and it is relatively easy to measure progress. Typically, they are applied to manageable projects in familiar domains where risk is manageable.

Incremental approaches

Incremental approaches are typically applied when a sequence of increments with increasing capabilities or effectiveness are developed. These increments support learning and assessment of potential design solutions. As the development progresses, some of these increments can be released and delivered. The last increment finally meets all requirements and provides all capabilities.

The benefits of incremental approaches are the focus on technical risk and the possibility to introduce emerging technologies.

Evolutionary approaches

Evolutionary approaches start with a limited set of requirements. A series of evolutionary cycles intended to both provide stakeholder value and increase problem understanding is initiated. Each cycle provides an opportunity to examine how the solution is used so the lessons learned can be incorporated in the next iteration.

The benefits of evolutionary approaches are early and steady feedback from stakeholders and the possibility to introduce emerging technologies.

References

Works Cited

INCOSE. 2023. "Chapter 2.1.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5> ^[1].

INCOSE. 2023. "Chapter 4.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5> ^[1].

ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html> ^[2].

Primary References

None.

Additional References

None.

Sequential Development Approach

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Sequential development approaches are commonly used in complex system development projects. Emphasizing a linear process with some iteration, this approach is particularly suited for large-scale, multi-organization efforts and safety-critical systems. It provides a clear framework for managing system development through distinct, well-defined stages. The article also introduces the Vee model as an example for the sequential development approach. The Vee model is a widely recognized variant that highlights the importance of validation, verification, and stakeholder engagement throughout the development lifecycle. By understanding these models, readers can appreciate the benefits of predictability, repeatability, and high assurance in systems engineering.

Concepts

The sequential development approach is a traditional methodology used in systems engineering, characterized by a structured, phase-based process. One of the earliest and most influential models of this approach is the waterfall model, introduced by Dr. Winston W. Royce in 1970. Interestingly, Royce initially presented the model not as a recommendation, but as a cautionary example of what could go wrong without incorporating feedback and iteration. Nevertheless, it became widely adopted in its linear form.

Royce's waterfall model outlines a series of distinct phases: requirements analysis, system design, implementation (or construction), integration and testing, deployment, and maintenance. Each phase is completed before the next begins, and typically, there is limited overlap or feedback between them. Although Royce advocated for some iteration, particularly early in the process, this aspect was often overlooked in practice.

In systems engineering, sequential development approaches provide structured methods to manage complex projects with well-defined requirements and deliverables. Among the most prominent models are the Waterfall and Vee models. Both offer a clear, phase-driven framework, but they differ in how they handle verification and validation.

The sequential approach is best suited for systems where requirements are well-defined, stable, and unlikely to change during development. It offers a clear structure, thorough documentation, and straightforward progress tracking, making it ideal for projects with strict regulatory, safety, or contractual obligations. This is often the case in industries such as aerospace, healthcare, defense, and civil engineering, where precision, predictability, and traceability are critical.

However, the sequential nature of this approach can make it difficult to accommodate changes once development is underway. Late modifications can be costly and time-consuming, particularly when early decisions are locked in by earlier phases. As a result, this model is most effective in environments with low uncertainty and minimal need for flexibility.

Practical considerations

In systems engineering projects that follow a sequential development approach, especially as originally envisioned by Winston Royce, engineers work within a structured, phase-driven process that incorporates planned iterations, particularly in early phases. While the overall development flow remains sequential, iterations are used to refine requirements and validate early design decisions before full-scale implementation begins.

For systems engineers, this means that early engagement in requirement analysis and system design is critical. Iterative feedback loops during these phases allow for stakeholder input, simulations, and feasibility assessments to inform more accurate and reliable specifications. Engineers must be prepared to revisit and revise these specifications multiple times before moving forward, which requires openness to feedback and strong communication skills.

Once the design is finalized, implementation and integration follow a more linear path. Engineers are expected to execute their tasks based on agreed specifications, with little room for ad hoc changes later in the process. Precision and adherence to documented requirements are essential, as late modifications can affect multiple subsystems and introduce costly rework.

Verification and validation typically occur after integration, so systems engineers must anticipate how subsystems will interact. This demands thorough interface definitions and systems thinking during earlier phases. While iterations are encouraged early on, they are less common in later phases unless major issues arise – underscoring the importance of getting the architecture right before committing to detailed development.

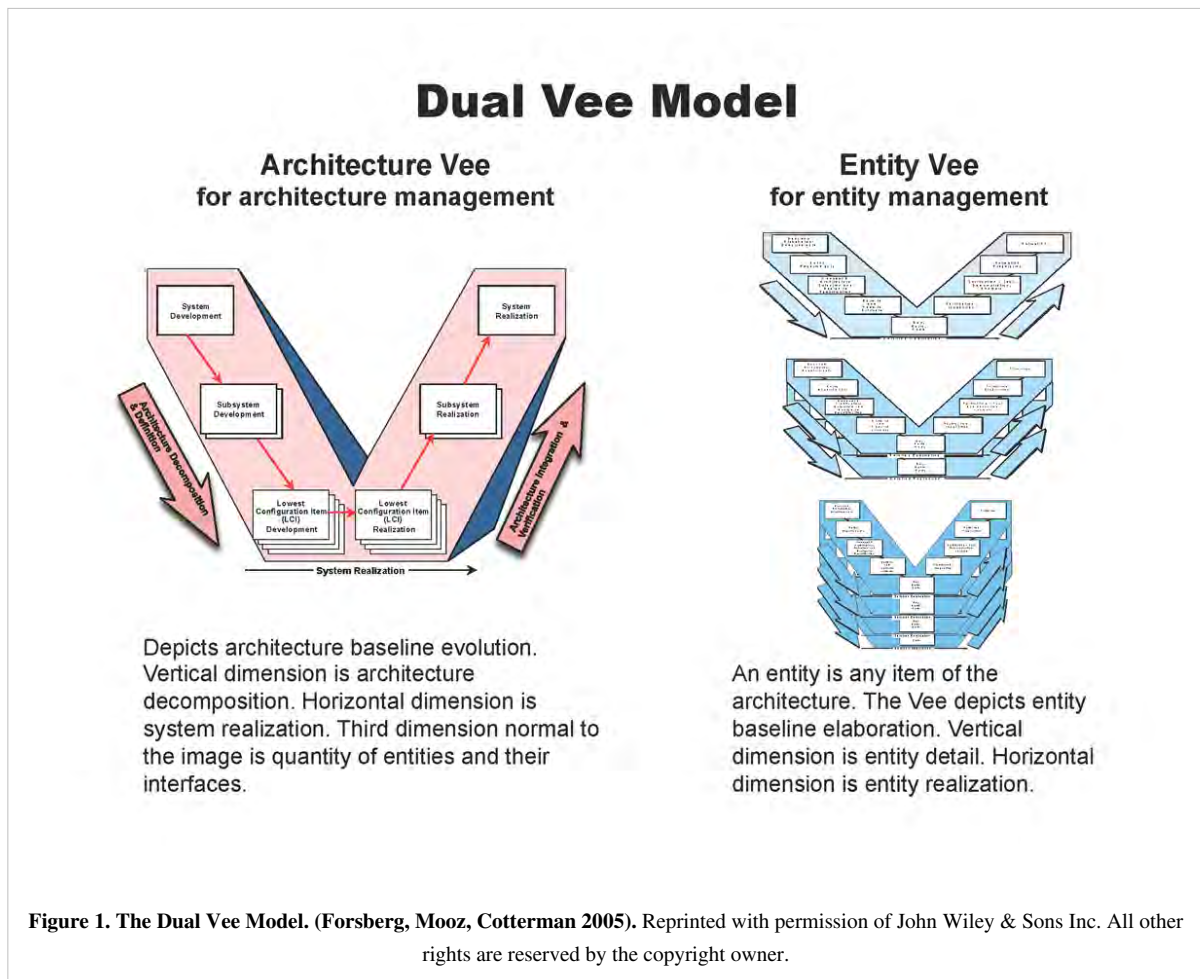
In practical terms, engineers must rely heavily on documentation, formal communication, and system models. Collaboration tends to follow the project structure, with different roles becoming more involved at different points in time. Systems engineers may lead early analysis and design efforts, then shift to supporting roles as implementation and verification proceed.

Ultimately, working within this approach requires a disciplined mindset, a proactive attitude during early iterations, and a strong focus on requirements traceability and design integrity throughout the project lifecycle.

Example development approach

The Vee model was developed as an extension of the Waterfall model to address the concerns that late discovery of defects or requirement mismatches can be costly. The Vee model retains a sequential flow but integrates system verification and validation more explicitly. As shown in Figure 1, it visualizes development in a “V” shape.

Note that because the Vee Model only covers the development stage, and not the full life cycle, it is not a life cycle model. There are other forms of the Vee model that show feedback from the right side of the Vee to the left side of the Vee for validation and verification.



The Vee model is used to visualize key areas for SE focus, associating each development stage with corresponding verification. The Vee highlights the need for continuous validation with the stakeholders, the need to define verification plans during requirements development, and the importance of continuous risk and opportunity assessment.

There are several variations of the Vee model. Typically, the “left” side of the Vee is called system definition and the “bottom” and “right” side of the Vee are called system realization. In the Vee model, time and system maturity conceptually proceed from left to right (down the left side of the Vee and up the right side of the Vee). However, all the system life cycle processes are performed concurrently and iteratively at each level of the system hierarchy and all the system life cycle processes are recursively applied at each level of the system hierarchy. One of the strengths of the Vee model is its depiction of the relationships between the left and right sides of the Vee.

The Vee model is widely used in systems engineering because it makes the relationship between development, verification, and validation more explicit. It encourages engineers to think ahead about how each part of the system will be verified, helping to reduce the risk of late failures.

References

Works Cited

Forsberg, K., H. Mooz, and H. Cotterman. 2005. *Visualizing Project Management*. 3rd ed. New York, NY, USA: J. Wiley & Sons.

INCOSE. 2023. "Chapter 2.2.1" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5> ^[1].

ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html> ^[2].

Winston W. Royce (1970). "Managing the Development of Large Software Systems" in: Technical Papers of Western Electronic Show and Convention (WesCon) August 25–28, 1970, Los Angeles, USA.

Primary References

None.

Additional References

None.

Incremental Development Approach

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Incremental development approaches offer a flexible and adaptive method suited for projects with high uncertainty or evolving requirements. Unlike sequential models, incremental development delivers value in chunks by breaking down the system into manageable increments, each providing a subset of capabilities. This allows for potential early deployment, faster feedback, and continuous improvement based on real-world insights. The article explores practical applications, planning challenges, and the importance of balancing a structured roadmap with flexibility. It also highlights the Incremental Commitment Spiral Model (ICSM), which integrates concurrent development and risk assessment to guide informed decisions across successive development cycles.

Concepts

Incremental development is a fundamental approach in systems engineering where a system evolves through repeated cycles (iterations) of planning, designing, implementing, and evaluating. Unlike sequential approaches, such as the Waterfall model, incremental development approaches allow for continuous refinement of both requirements and solutions based on feedback and learning obtained during each iteration.

The roots of incremental development can be traced back to the 1950s and 60s with early software and systems engineering projects that required more flexibility than rigid linear models could provide. Notably, Barry Boehm's Spiral Model in the 1980s formalized incremental development by emphasizing risk management and incremental progress. It later influenced many agile and incremental development methodologies used today.

When using an incremental approach, not every iteration results in a tangible increment (i.e., a deliverable or prototype). Some iterations serve primarily as learning opportunities -exploring technical feasibility, user needs, or integration challenges. These learning-focused cycles are crucial for reducing uncertainty and guiding future design decisions.

Iterations that do produce increments provide stakeholders with early, partial versions of the system. These early deliverables enable verification, validation, and feedback well before the final system is complete. This responsiveness to change and focus on incremental value helps ensure the system better meets user needs and environmental constraints.

Incremental development approaches are especially useful in complex, high-uncertainty projects such as those involving new technologies, evolving requirements, or significant stakeholder interaction. It supports a more adaptive planning process, enabling engineers to respond to change and reduce risks over time.

By embracing early feedback, learning, and flexibility, incremental development approaches help create more robust, user-centered systems. It aligns engineering efforts with real-world complexities, making it a powerful strategy for certain development environments.

Practical considerations

The incremental development approach typically determines user needs, defines requirements, then divides the remaining development into a series or sequence of iterations, some of which result in a tangible increment and some of which serve for learning.

Early iterations may focus more on exploration (modeling, simulation, or proof-of-concept studies) to reduce technical uncertainty or evaluate design alternatives. Systems engineers must be comfortable with partial or evolving system architectures, often working with incomplete data or preliminary interfaces.

Traceability and configuration management also take on a different character compared to a sequential development approach. Since requirements, design elements, and interfaces may evolve over several iterations, robust version control and clear documentation of assumptions and decisions become essential to prevent confusion or rework later.

Collaboration and communication are especially critical. Incremental development often involves frequent interactions with stakeholders, users, and other engineering disciplines. Systems engineers must facilitate this exchange, using tools like prototypes, models, or incremental demonstrations to ensure alignment across the team and with the project goals.

Validation and verification activities are also distributed more evenly across the project timeline. Rather than being concentrated at the end, these activities are planned into iterations, allowing for early detection of issues. This demands more flexible test planning and a willingness to revise test cases as the system evolves.

Proper planning of the increments is critical and at the same time challenging. On the one hand, stakeholders are expecting a “master plan” for the increments and on the other hand, subsequent increments build on the learnings from the previous increments.

Finally, engineers must be prepared to continuously reassess risks and priorities at each iteration boundary. Trade studies, system impact assessments, and integration planning become recurring tasks, helping to maintain system coherence and feasibility across changing project conditions.

An incremental development approach can also be used when it is critical to hit the market with a product at a certain point in time. In this case, it can be advantageous to bring a product with limited capabilities to the market to get “a foot in the door”. Successive increments follow to close the capability gaps.

Example development approach

Boehm's Incremental Commitment Spiral Model (ICSM) shown in Figure 1 is a modern incremental approach which extends the classic Spiral Model (Boehm, 1987) for software introduced by Boehm for SE. It is designed to address the challenges of complex, high-risk systems engineering projects. It combines the strengths of traditional, plan-driven methods with the flexibility of iterative and agile practices, offering a structured way to deliver systems in well-defined increments while continuously managing uncertainty and stakeholder alignment.

In ICSM, system development is divided into a series of incremental cycles, each leading to a decision point, or commitment review. Rather than committing to a full system design early on, stakeholders and engineers make incremental commitments only as far as the evidence and confidence gathered in each cycle justify. This allows projects to pivot, refine goals, or even stop development early if feasibility or value cannot be demonstrated.

Each cycle involves activities such as stakeholder engagement, feasibility assessments, risk analysis, and prototype development. If the results are promising, the project moves forward to the next increment, where additional capability is developed and validated. Over time, the system grows in maturity and functionality, reducing risk at each stage.

Unlike purely iterative models, ICSM provides clear decision checkpoints and is especially useful in projects requiring formal reviews, traceability, and accountability such as in defense, transportation, or critical infrastructure systems. It accommodates evolving requirements while maintaining a disciplined engineering process.

For systems engineers, working with ICSM means managing incremental baselines, tracking decisions across cycles, and facilitating informed stakeholder commitments. The model encourages early and continuous validation, with learning cycles embedded throughout the project.

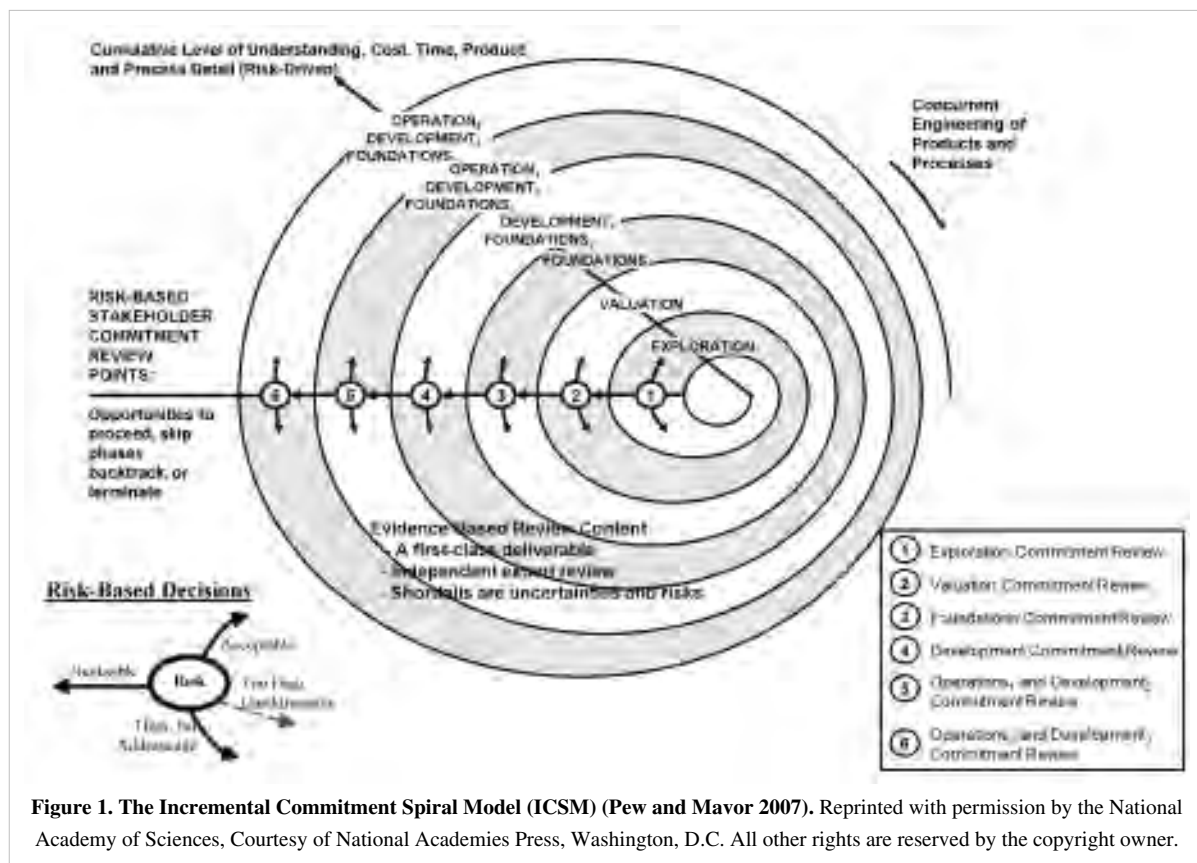


Figure 1. The Incremental Commitment Spiral Model (ICSM) (Pew and Mavor 2007). Reprinted with permission by the National Academy of Sciences, Courtesy of National Academies Press, Washington, D.C. All other rights are reserved by the copyright owner.

References

Works Cited

- INCOSE. 2023. "Chapter 2.2.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.
- Boehm, B. (1987). A spiral model of software development and enhancement, *ACM SIGSOFT Software Engineering Notes*, 11(4), 14–24.
- Boehm, B., Lane, J., Koolmanojwong, S., and Turner, R. (2014). *The Incremental Commitment Spiral Model: Principles and Practices for Successful Systems and Software*, Addison-Wesley Professional.
- ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html>.

Primary References

None.

Additional References

None.

Evolutionary Development Approach

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

The evolutionary development approach is a dynamic and flexible method ideal for projects where not all requirements are known upfront and are likely to evolve over time. Unlike sequential or incremental models, evolutionary development embraces uncertainty, allowing systems to evolve through frequent iterations and stakeholder feedback. Often applied in software development, this approach emphasizes rapid delivery of deployable versions and continuous refinement based on user input. Key considerations include managing compatibility, supporting multiple versions, and designing with adaptability and modularity in mind. The article also highlights DevOps as a representative model, showcasing how continuous integration, automation, and shared ownership drive ongoing development and operational excellence.

Concepts

Evolutionary development approaches are a flexible strategy for developing complex systems in situations where only part of the system's requirements is known at the start. Unlike sequential development approaches that aim to finalize all specifications before implementation begins, this approach allows the system to evolve over time driven by real-world use and continuous feedback.

Emerging in the late 1970s and early 1980s, evolutionary development gained recognition as an effective way to handle uncertainty and changing needs in system design. It has since become widely adopted across industries where adaptability and responsiveness are essential.

In evolutionary development, systems are developed, built, and delivered in a series of working iterations, each of which is released to the market or end users. These early versions provide core functionality and are intended for actual use. Feedback from users, performance data, and market response are gathered and analyzed after each release. This insight is then used to shape the next version of the system.

Each iteration adds new capabilities or improves existing ones, gradually expanding and refining the system. Because the system is in use from early on, stakeholders can interact with real products rather than abstract plans, which helps clarify needs and priorities as development progresses.

This approach is especially effective for innovative or dynamic markets, where requirements are likely to change and speed of delivery is important. It reduces risk by allowing early validation and ensures the system remains aligned with user expectations and market demand.

Key characteristics of evolutionary development include frequent releases, real-world feedback loops, and continuous improvement. It's best suited for projects where the ability to adapt quickly and learn from use is critical to success.

Practical considerations

One key consideration is modular and scalable architecture. Since the system will grow and change over multiple iterations, it must be designed to accommodate extension and modification without major rework. Interfaces should be well-defined and loosely coupled to allow new system elements to integrate smoothly as they are developed.

Another critical aspect is continuous integration and verification. Each iteration is intended for release and real-world use, so it must meet quality standards independently. Systems engineers must ensure that each version is sufficiently verified and that the integration of new system elements does not compromise overall functionality, safety, or compliance. This often involves setting up automated testing frameworks, simulation environments, and regression test suites early in the project.

Traceability and configuration management also take on increased importance. As requirements evolve and the system matures, maintaining clear documentation of what was changed, why it was changed, and what impact it had becomes essential. Systems engineers must use version control systems and maintain accurate configuration baselines across iterations. This discipline not only supports accountability and regulatory compliance but also enables engineers to assess the upgradeability of deployed systems ensuring that new versions can be integrated smoothly into existing operational environments without disrupting functionality or compatibility.

Finally, close and ongoing collaboration with stakeholders is vital. Systems engineers need to actively participate in feedback loops, assess input from the field, and translate user experiences into actionable technical changes. This requires not only technical expertise but also strong communication skills and a user-focused perspective.

Example development approach

Although DevOps as shown in Figure 1 is a set of principles, it is often cited as an example of evolutionary development approaches. At the same time, DevOps can also be understood as a lifecycle model, as it is an infinite repetition of development and operations.

An evolutionary approach to system development focuses on building systems in a way that allows continuous adaptation to changing requirements, user feedback, and external conditions. DevOps is a prime example of an evolutionary approach that blends development, operations, and continuous feedback loops to improve systems over time.

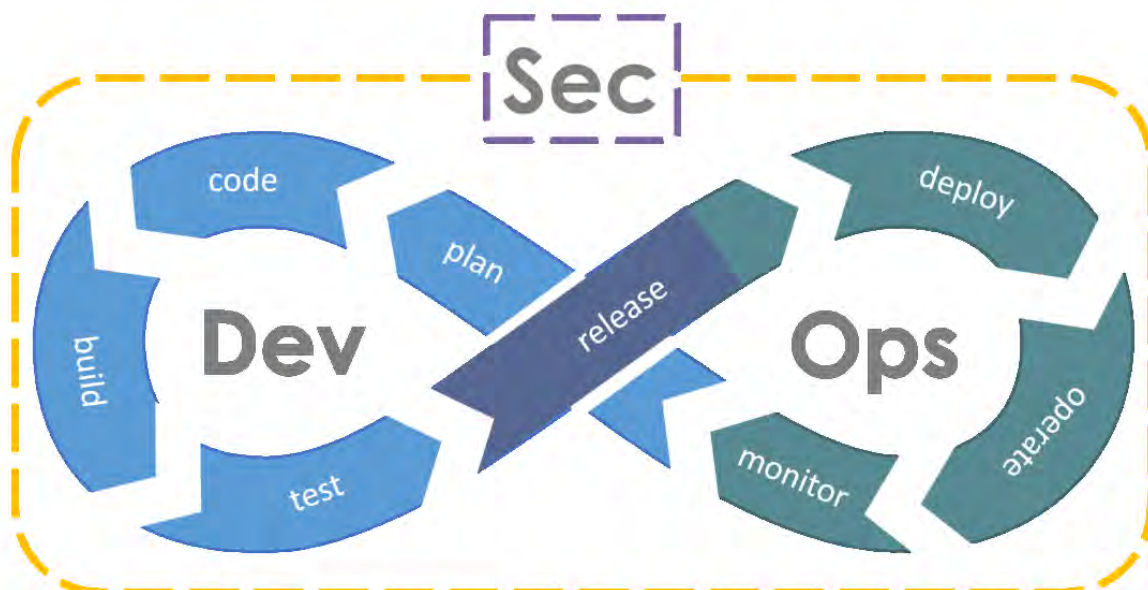


Figure 1. DevSecOps. (D'Souza derived from Banach (2019) and Anx (2021). Used with permission. All other rights reserved.)

As shown in the figure, DevOps promotes continuous integration (CI) and continuous delivery (CD), enabling rapid iteration and frequent releases of system updates. The core idea is to create a development pipeline where small, incremental changes are integrated and tested continuously, making it possible to deploy new features or fixes to production at any time. This method allows the system to evolve based on real-time feedback, whether from users, monitoring tools, or the operational environment.

For systems engineers, DevOps means engaging in constant collaboration with both development and operations teams. It's not just about writing code or designing systems in isolation; it's about iteratively building a system, deploying it into the real world, gathering data, and refining it based on that data. Automation plays a crucial role in DevOps, as it ensures that the evolution process is both fast and reliable. Automated tests, deployment pipelines, and infrastructure-as-code practices enable engineers to deliver frequent updates without sacrificing quality.

DevOps also emphasizes the importance of feedback loops whether it's from customers, users, or system performance monitoring. As the system is deployed and used, systems engineers can gather actionable insights that inform subsequent iterations. This continuous cycle of building, deploying, testing, and adapting is the essence of the evolutionary development approach.

In environments like cloud services, IoT systems, or large-scale software platforms, where conditions are constantly changing and innovation is rapid, DevOps offers a practical, scalable way to ensure systems remain relevant and high performing over time.

References

Works Cited

INCOSE. 2023. "Chapter 2.2.3" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.

ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html>.

Primary References

None.

Additional References

None.

Agile Development Approach

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Agile development is an approach based on iterative development, frequent inspection and adaptation, and incremental deliveries in which requirements and solutions evolve through collaboration in cross-functional teams and through continual stakeholder feedback.

Concepts

Agile simply means able to move quickly and easily. Agile is an adjective, as in an agile development approach or an agile person. Agility is a noun, as in systems engineering agility, or someone who has impressive agility for a person of their size.

Agile development is an approach based on iterative development, frequent inspection and adaptation, and incremental deliveries in which requirements and solutions evolve through collaboration in cross-functional teams and through continual stakeholder feedback (ISO/IEC/IEEE 26515).

Iteration is repeating the application of the same process or set of processes on the same level of the system structure (ISO/IEC/IEEE 15288:2023).

In contrast to sequential approaches, agile development approaches break development into small, manageable units of work called iterations or sprints, usually lasting 1 to 4 weeks. At the end of each iteration, the team delivers a potentially shippable product increment. This allows for frequent reassessment of priorities and adaptation to new information or customer needs.

Agile principles were formally introduced in 2001 with the publication of the Agile Manifesto ^[1], which was specifically designed for software development. The Manifesto outlines four core values for guiding the development of software systems:

1. Individuals and interactions over processes and tools
2. Working systems over comprehensive documentation
3. Customer collaboration over contract negotiation
4. Responding to change over following a plan

While these values were crafted for software development, many systems engineers adopt agile principles in their practices. However, it is important to recognize that these principles were initially formulated within the context of software engineering and may not transfer seamlessly to the broader, more complex domain of systems engineering. Systems engineering involves not only software but also hardware, human elements, processes, and other components that may require different considerations, timelines, and integration points.

The agile methodology is supported by 12 principles that emphasize continuous delivery, responsiveness to change, and high-quality, sustainable development practices. However, these principles were originally intended for software systems, and the direct application of these ideas to complex systems engineering projects can require adaptation. Agile methods can be highly effective in dynamic environments where customer needs evolve rapidly. Teams are encouraged to deliver value early and often, potentially improving responsiveness and customer satisfaction. At the same time, these approaches must be carefully adapted when applied to large-scale, multi-disciplinary systems engineering projects.

Practical considerations

In an agile systems engineering environment, team members collaborate in a fast-moving, feedback-driven manner. A core aspect of agile systems engineering is the use of system-level user stories. These are short, simple descriptions of system features or capabilities from the perspective of the end user. Each story typically follows a format like: "As a <role (who wants to accomplish something)>, I want <activity (what they want to accomplish)> so that <business value (why they want to accomplish that thing)>." These stories help clarify not just the technical requirements but also the rationale behind them.

User stories are often grouped into user journeys or epics, giving context to how individual features contribute to the broader product experience. During sprint planning, the team breaks stories into smaller tasks, estimates their complexity (often using story points), and commits to what can realistically be delivered in the upcoming sprint. Frequent communication is vital in agile systems engineering to share progress, identify obstacles, and coordinate efforts, ensure alignment and foster transparency.

Using an agile development approach also means team members are expected to respond to change. As requirements evolve throughout the project, teams are expected to pivot and adjust without disrupting the overall progress. Continuous engagement with stakeholders, including system architects, designers, and product owners, ensures that changing priorities are clearly communicated and understood.

Sprints are a core element of most agile methodologies, but there are some agile approaches that do not strictly rely on sprints. In case sprints are used, retrospectives at the end of each sprint allow team members to reflect on their processes, discuss what worked well, and identify areas for improvement. This fosters a culture of continuous learning and refinement, encouraging ongoing process optimization. In the agile development approach, decision points tend to avoid the terms "milestones" and "decision gates." With agile development approaches, interaction with stakeholders can be more frequent, smaller in scope and less formal than other approaches. (ISO/IEC/IEEE 24748-1)

Example development approach

Several frameworks have emerged to help teams implement agile principles effectively. The following text introduces a few non-exhaustive examples to demonstrate the existing variation in agile methods and do not constitute an endorsement.

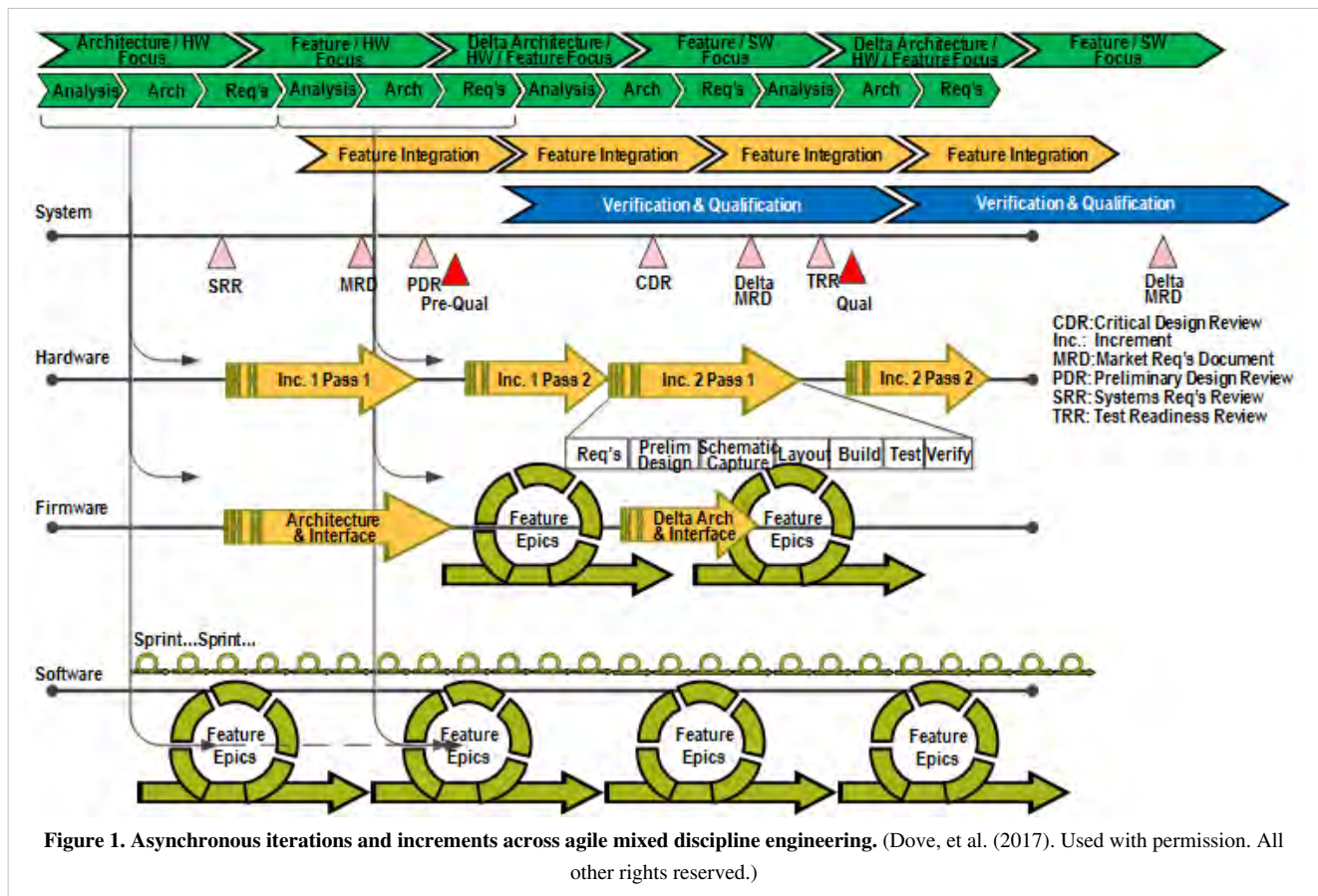
Scrum^[2] is the most widely used. It structures work into time-boxed sprints (sometimes 2 to 4 weeks) and includes defined roles such as Product Owner, Scrum Master, and Development Team. Scrum emphasizes clear priorities, daily stand-ups, sprint planning, reviews, and retrospectives to promote continuous improvement and team alignment.

For larger organizations, SAgile (Scaled Agile Framework)^[3] provides a structured way to scale Agile across multiple teams and departments. It introduces roles like Release Train Engineer and events like PI Planning (Program Increment Planning), aligning teams around shared goals and synchronized delivery cycles. SAgile blends Agile, Lean, and systems thinking to support complex enterprise needs.

LeSS (Large-Scale Scrum)^[4] is a lighter-weight alternative to SAgile. It extends Scrum principles to multiple teams working on the same product. Instead of adding layers of management, LeSS encourages simplification and emphasizes team autonomy and shared product ownership.

Each framework offers different levels of structure, but all aim to improve collaboration, transparency, and adaptability which are key benefits of working in an Agile environment.

Figure 1 shows an example of a mixed approach that is both incremental, evolutionary and agile. In this example, teams working on electronic-board hardware, firmware, and software have different timings for hardware increments and firmware and software epics (versions). The teams accomplish integrated work-in-process test with the latest increments and versions from each of the disciplines.



This example shows bringing the individual strands together at certain points in time. This reduces inconsistencies at the interfaces and strengthens the team spirit.

References

Works Cited

- INCOSE. 2023. "Chapter 2.2 and 4.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.
- ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html>.
- ISO/IEC/IEEE 24748-2. 2024. *Systems and Software Engineering -- Life Cycle Management – Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*. Geneva, Switzerland: International Organisation for Standardisation (ISO) / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84661.html>.
- ISO/IEC/IEEE 26515. 2018. *Systems and software engineering — Developing information for users in an agile environment*. Geneva, Switzerland: International Organisation for Standardisation (ISO) / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/70880.html>.

ISO/IEC/IEEE 15288. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/81702.html>.

Beck, K., et al. (2001) The Agile Manifesto. Agile Alliance. <https://agilemanifesto.org/>

Primary References

None.

Additional References

None.

References

[1] <https://agilemanifesto.org/>

[2] <https://www.scrum.org/>

[3] <https://framework.scaledagile.com/>

[4] <https://less.works/>

Lean Engineering

Lean Systems Engineering (LSE) is the application of lean thinking (Womack 2003) to systems engineering (SE) and related aspects of enterprise and project management. LSE is an approach that is applicable throughout the system life cycle. The goal of LSE is to deliver the best life-cycle value for technically complex systems with minimal waste. Lean engineering is relevant to all of the traditional SE technical processes (see system concept definition, system definition, system realization, system deployment and use, etc.). Lean engineering also interacts with and utilizes many of the specialty engineering disciplines discussed in Part 6.

Lean Systems Engineering

SE is an established, sound practice, but not always delivered effectively. Most programs are burdened with some form of waste such as: poor coordination, unstable requirements, quality problems, delays, rework, or management frustration. Recent U.S. Government Accountability Office (GAO), National Aeronautics and Space Association (NASA), and Massachusetts Institute of Technology (MIT) studies of government programs document major budget and schedule overruns and a significant amount of waste in government programs - some reaching seventy percent of charged time. This waste represents a productivity reserve in programs and major opportunities to improve program efficiency.

LSE is the application of lean thinking to systems engineering and related aspects of enterprise and project management. SE is focused on the discipline that enables development of complex technical systems. Lean thinking is a holistic paradigm that focuses on delivering maximum value to the customer and minimizing wasteful practices. It has been successfully applied in manufacturing, aircraft depots, administration, supply chain management, healthcare, and product development, which includes engineering. LSE is the area of synergy between lean thinking and SE, which aims to deliver the best life-cycle value for technically complex systems with minimal waste. LSE does not mean less SE. It means more and better SE with higher responsibility, authority, and accountability (RAA), leading to better, waste-free workflow with increased mission assurance. Under the LSE philosophy, mission assurance is non-negotiable and any task which is legitimately required for success must be included; however, it should be well-planned and executed with minimal waste.

Lean Principles

Oppenheim (2011) describes the six lean principles for product development (PD) as follows:

1. **Capture the value defined by the customer.** One cannot over-emphasize the importance of capturing task or program value (requirements, CONOPS, etc.) with precision, clarity, and completeness before resource expenditures ramp up to avoid unnecessary rework.
2. **Map the value stream (plan the program) and eliminate waste.** Map all end-to-end linked tasks, control/decision nodes, and the interconnecting information flows necessary to realize customer value. During the mapping process, eliminate all non-value-added activities, and enable the remaining activities to flow (without rework, backflow or stopping). The term *information flow* refers to the packets of information (knowledge) created by different tasks and flowing to other tasks for subsequent value adding, such as: design, analysis, test, review, decision, or integration. Each task adds value if it increases the level of useful information and reduces risk in the context of delivering customer value.
3. **Flow the work through planned and streamlined value-adding steps and processes, without stopping or idle time, unplanned rework, or backflow.** To optimize flow, one should plan for maximum concurrency of tasks, up to near capacity of an enterprise. Legitimate engineering iterations are frequently needed in PD, but they tend to be time consuming and expensive if they extend across disciplines. Lean PD encourages efficient methodology of *fail early - fail often* through rapid architecting and discovery techniques during early design phases. Lean flow also makes every effort to use techniques that prevent lengthy iterations, such as design frontloading, trade space explorations, set designs, modular designs, legacy knowledge, and large margins. Where detailed cross-functional iterations are indeed necessary, lean flow optimizes iteration loops for overall value.
4. **Let customers pull value.** In PD, the pull principle has two important meanings: (1) the inclusion of any task in a program must be justified by a specific need from an internal or external customer and coordinated with them, and (2) the task should be completed when the customer needs the output. Excessively early completion leads to "shelf life obsolescence" including possible loss of human memory or changed requirements and late completion leads to schedule slips. This is the reason that every task owner or engineer needs to be in close communication with their internal customers to fully understand their needs and expectations and to coordinate their work.
5. **Pursue perfection of all processes.** Global competition requires continuous improvements of processes and products. Yet, no organization can afford to spend resources improving everything all the time. Systems engineers must make a distinction between processes and process outputs. Perfecting and refining the *work output* in a given task must be bounded by the overall value proposition (system or mission success, program budget and schedule) which define when an output is "good enough." In contrast, engineering and other *processes* must be continuously improved for competitive reasons.
6. **Respect for people.** A lean enterprise is an organization that recognizes that its people are the most important resource. In a lean enterprise, people are not afraid to identify problems and imperfections honestly and openly in real time, brainstorm about root causes and corrective actions without fear, or plan effective solutions together by consensus to prevent a problem from occurring again.

Lean Enablers for Systems

In 2009, the International Council on Systems Engineering's (INCOSE's) Lean SE Working Group (LSE WG) released an online product entitled *Lean Enablers for Systems Engineering (LEfSE)*. It is a collection of practices and recommendations formulated as "dos" and "don'ts" of SE, based on lean thinking. The practices cover a large spectrum of SE and other relevant enterprise management practices, with a general focus on improving the program value and stakeholder satisfaction and reduce waste, delays, cost overruns, and frustrations. LEfSE are grouped under the six lean principles outlined above. The LEfSE are not intended to become a mandatory practice but should be used as a checklist of good practices. LEfSE do not replace the traditional SE; instead, they amend it with lean thinking.

LEfSE were developed by fourteen experienced INCOSE practitioners, some recognized leaders in lean and SE from industry, academia, and governments (such as the U.S., United Kingdom, and Israel), with cooperation from the 160-member international LSE WG. They collected best practices from the many companies, added collective tacit knowledge, wisdom, and experience of the LSE WG members, and inserted best practices from lean research and literature. The product has been evaluated by surveys and comparisons with the recent programmatic recommendations by GAO and NASA.

Oppenheim (2011) includes a comprehensive explanation of the enablers, as well as the history of LSE, the development process of LEfSE, industrial examples, and other material. Oppenheim, Murman, and Secor (2011) provide a scholarly article about LEfSE. A short summary was also published by Oppenheim in 2009.

References

Works Cited

- Lean Systems Engineering Working Group. 2009. "Lean Enablers for Systems Engineering." Accessed 13 January 2016 at https://www.lean-systems-engineering.org/wp-content/uploads/2012/07/Lean-Enablers-for-SE-Version-1_03-.pdf.
- Lean Systems Engineering Working Group. 2009. "Quick Reference Guide Lean Enablers for Systems Engineering." Accessed 13 January 2016 at <https://www.lean-systems-engineering.org/wp-content/uploads/2012/07/LEfSE-Quick-Reference-Guide-8-pages.pdf>.
- Oppenheim, B.W. 2009. "Process Replication: Lean Enablers for Systems Engineering." *CrossTalk, The Journal of Defense Software Engineering*. July/August 2009.
- Oppenheim, B.W. 2011. *Lean for Systems Engineering, with Lean Enablers for Systems Engineering*. Hoboken, NJ, USA: Wiley.
- Oppenheim, B.W., E.M. Murman, and D. Secor. 2011. "Lean Enablers for Systems Engineering." *Journal of Systems Engineering*. 1(14).
- Womack, J.P. 2003. *Lean Thinking*. Columbus, OH, USA: Free Press.

Primary References

- Lean Systems Engineering Working Group. 2009. "Lean Enablers for Systems Engineering." Accessed 13 January 2016 at https://www.lean-systems-engineering.org/wp-content/uploads/2012/07/Lean-Enablers-for-SE-Version-1_03-.pdf.
- Oppenheim, B., E. Murman, and D. Sekor. 2010. "Lean Enablers for Systems Engineering." *Systems Engineering*. 14(1). Accessed 13 January 2016. Available at <https://www.lean-systems-engineering.org/wp-content/uploads/2012/07/LEfSE-JSE-compressed.pdf>.

Additional References

- Lean Enterprise Institute. 2009. "Principles of Lean." Accessed 1 March 2012 at <https://www.lean.org/WhatsLean/Principles.cfm>.

Knowledge Area: Agile Approaches

Agile Approaches

Contents of this Knowledge Area

- Agile Systems Engineering (Rick Dove, Kerry Lunney, Michael Orosz, and Mike Yokell)
- Industrial DevOps (Suzette Johnson and Robin Yeman)
- Lead Author:
- David Endler
- Contributing Author:
- Mike Yokell

A useful way to frame both Agile Systems Engineering and Industrial DevOps is through the lens of ISO/IEC/IEEE 24748-10, which emphasizes agility as a set of strategic capabilities rather than a fixed method. Modern systems operate in environments characterized by uncertainty, rapid technological change, and evolving stakeholder expectations. In this context, systems engineering must move beyond linear, plan-driven models toward approaches that enable continuous adaptation across the life cycle.

At an abstract level, agility in systems engineering can be understood as the ability to sense changes, respond effectively, and evolve system solutions over time. This requires combining architectural foresight (e.g., modularity and scalability), organizational alignment, and feedback-rich processes that support iterative learning and decision-making. Importantly, agility is not limited to development. It spans the entire system life cycle, including production, utilization, and support.

Within this framing, Agile Systems Engineering provides the conceptual and strategic foundations for achieving agility, while Industrial DevOps represents a practical realization of these principles through integrated practices, toolchains, and value-stream-oriented workflows. Together, they illustrate how systems engineering can balance rigor and adaptability while ensuring both dependable system outcomes and responsiveness to change in complex socio-technical environments.

References

Works Cited

None.

Primary References

ISO/IEC/IEEE 24748-10, 2026. *Systems and software engineering — Life cycle management. Part 10: Guidelines for systems engineering agility*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/90086.html>.

Additional References

None.

Agile Systems Engineering

- Lead Authors:
- Rick Dove, Kerry Lunney, Michael Orosz, and Mike Yokell

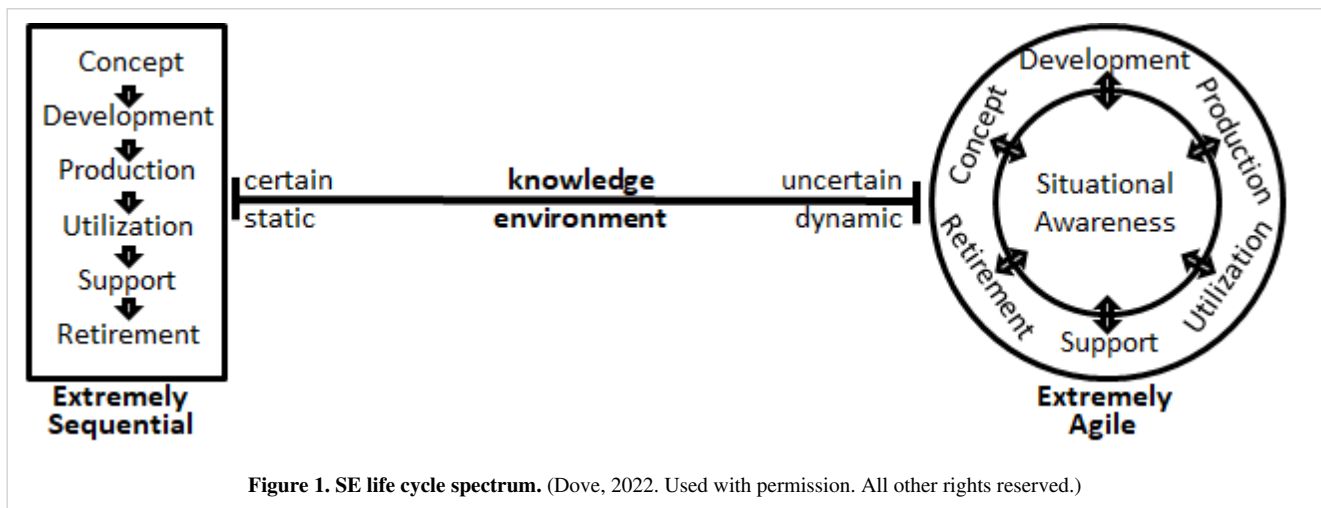
Agile systems engineering (Dove et al, 2023, ISO/IEC/IEEE 24748-10) is a principle-based method for designing, building, sustaining, and evolving systems when knowledge is uncertain and/or environments are dynamic. Agile systems engineering is being agile, not doing agile. Thus, Agile System Engineering is a what, not a how.

Concepts

The common use of the terms agile and agility are fundamental to its application in systems engineering. Agile simply means able to move quickly and easily. Agile is an adjective, as in an agile development approach or an agile person. Agility is a noun, as in systems engineering agility, or he has impressive agility for a person of his size.

Within systems of software engineering, agile is a development approach based on iterative development, frequent inspection and adaptation, and incremental deliveries in which requirements and solutions evolve through collaboration in cross-functional teams and through continual stakeholder feedback (ISO/IEC/IEEE 24748-1:2024, 3.4)

Likewise, Systems engineering (SE) agility is a “strategy-based method for designing, building, sustaining, and evolving purpose-fulfilling creations when knowledge is uncertain and operational environments are dynamic” (Dove et al, 2023, ISO/IEC/IEEE 24748-10). Agile SE is a thoughtful engagement with opportunities and threats in the environment. Figure 1 shows a spectrum of life cycle approaches. All life cycle approaches fall somewhere between the two ends of the spectrum.



Background Context

In 1991, the US Department of Defense (Nagel and Dove, 1991) funded a project to investigate what would drive competition in manufacturing enterprises after the then-active scramble to become more Lean manufacturing (a term related to the Toyota Production System (Liker, 2004)) had stabilized. The results of that project used the concept of agility and the word agile as a way to describe how organizations could accommodate the increasing frequency of change in markets and technologies. Over the next four years, the Agility Forum, funded by DARPA (Defense

Advanced Research Agency) at Lehigh University, led 1200 participants from 125 organizations through a collaborative discovery process across a broad base of business and engineering areas (Dove and LaBarge, 2014). The initial focus on agile manufacturing evolved quickly during the early 1990s into agile systems, agile enterprises, and agile command and control. This led in 2001 to the adoption of the word "agile" to describe a variety of new ways to develop software (e.g. agile software development) (Fowler and Highsmith, 2001). In 2014, INCOSE established "agile systems engineering" as one of its top priorities (Dove and Schindel 2019).

Guidelines for SE Agility

Eight core aspects (Dove et al, 2023, ISO/IEC/IEEE 24748-10) are each explained succinctly as a Why (need) and a What (behavior), with some examples of How (method). Though these aspects are core strategies for any kind of agile engineering, the purpose and descriptions here are for application at the systems engineering level rather than the domain engineering level.

Each of the aspects can individually improve capability to deal with uncertain knowledge and dynamic environments in any engineering process; but to have something intended as an agile engineering process at either domain or system level requires multiple aspects operating in concert. Individual aspects are strategic concepts that can tactically manifest over a range of intensity. Thus, the degree of agility is a product of how many of these aspects are operational as well as how effectively each one contributes to the agility required by the operating environment. Big bang concurrent implementation of all aspects is not necessary to gain agility benefits.

The eight strategic aspects (Dove et al, 2023, ISO/IEC/IEEE 24748-10) of SE agility are:

- adaptable modular architecture
- iterative incremental development
- attentive situational awareness
- attentive decision making
- common-mission teaming
- shared-knowledge management
- continual integration and test
- being agile: operations concept

Adaptable Modular Architectures

Needs: Facilitated product and process experimentation, modification, and evolution.

Behaviors: Composable and reconfigurable product and process designs from variations of reusable assets.

Discussion: One fixed process approach won't fit all projects, so an appropriate process should be easy to compose and evolve according to context and usage experience. Variations of reusable assets are built over time as features are modified for different contextual usage.

A hallmark of agile systems engineering is iterative incremental development, which modifies work in process as suitability is repetitively evaluated. The agility of the process depends upon the agility of the product so both process and product can be easily changed.

Iterative Incremental Development

Needs: Minimize rework, maximize quality, facilitate innovation.

Behaviors: Incremental loops of building, evaluating, correcting, and improving capabilities.

Discussion: Generally, increments create capabilities and iterations add and augment features to improve capabilities.

- Increment cycles are beneficially timed to coordinate events such as integrated testing and evaluation, capability deployment, experimental deployment, or release to production.
- Increments may have constant or variable cadence to accommodate management standards or operational dynamics.
- Iteration cycles are beneficially timed to minimize rework cost as a project learns experimentally and empirically.

Attentive Situational Awareness

Needs: Timely knowledge of emergent risks and opportunities.

Behaviors: Active monitoring and evaluation of relevant internal and external operational environmental factors.

Discussion: Are things being done right (internal awareness) and are the right things being done (external awareness)? Having the agile capability for timely and cost-effective change does little good if you don't know when that ability should be exercised. Situational awareness can be enhanced with systemic methods and mechanisms.

Attentive Decision Making

Needs: Timely corrective and improvement actions.

Behaviors: Systemic linkage of situational awareness to decisive action.

Discussion: Empower decision making at the point of most knowledge. As a counter example, technical debt (a term for knowing something needs correction or improvement but postponing action) is situational awareness without a causal link to prompt action.

Common-Mission Teaming

Needs: Coherent collective pursuit of a common mission.

Behaviors: Engaged collaboration, cooperation, and teaming among all relevant stakeholders.

Discussion: Collaboration, cooperation, and teaming are not synonymous, and need individual support attention. Collaboration is an act of relevant information exchange among individuals, cooperation is an act of optimal give and take among individuals, and teaming is an act of collective endeavor toward a common purpose.

Continual Integration & Test

Needs: Early revelation of system integration issues.

Behaviors: Integrated test and demonstration of work-in-process.

Discussion: Discovering integration issues late in development activities can impact cost and schedule with major rework. Synchronizing multiple domain engineering activities via continual integration and test provides faster and clearer insight into potential system integration issues.

Shared-Knowledge Management

Needs: Accelerated mutual learning and single source of truth for internal and external stakeholders.

Behaviors: Facilitated communication, collaboration, and knowledge curation.

Discussion: There are two kinds of knowledge to consider. Short time frame operational knowledge: what happened, what's happening, what's planned to happen. Long time frame curated knowledge: what do we know of reusable relevance, e.g., digital artifacts, lessons learned, and proven practices.

Being Agile

Needs: Attentive operational response to evolving knowledge and dynamic environments.

Behaviors: Sensing, responding, evolving.

Discussion: Agile systems engineering is not about doing Agile, it is about being agile. Being agile is a behavior, not a procedure—a behavior sensitive to threats and opportunities in the operational environment, decisive when faced with threat or opportunity, and driven to improve these capabilities. Deciding how to implement any of the core aspects, even this one, should be done with sense-respond-evolve principles in mind as aspect objectives.

References

Works Cited

- Dove, R, et al. 2023. Agile Systems Engineering – Eight Core Aspects. INCOSE, 2023
- Dove, R. and LaBarge, R. 2014. Fundamentals of agile systems engineering – part 1 & part 2. International Council on Systems Engineering, International Symposium, Las Vegas, NV, 30Jun-3Jul.
- Dove, R. and Schindel, W. 2019. Agile Systems Engineering Life Cycle Model for Mixed Discipline Engineering. Proceedings International Symposium. International Council on Systems Engineering. Orlando, FL, July 20-25.
- Fowler, M. and Highsmith, J. 2001. The Agile Manifesto. Dr. Dobb's Journal, August.
- Liker, J. Toyota Way, 2004. 14 Management Principles from the World's Greatest Manufacturer, McGraw Hill, 2004.
- Nagel, R and Dove, R. 1991. 21st Century Manufacturing Enterprise Strategy – An Industry-led View, Eds. Goldman, S and Preiss, K. Diane Publishing Company 1991.
- ISO/IEC/IEEE 24748-10, 2026. *Systems and software engineering — Life cycle management. Part 10: Guidelines for systems engineering agility*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/90086.html>.
- INCOSE. 2023. "Chapter 4.2.2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.

Primary References

None.

Additional References

SE Agility Primer. INCOSE.

Industrial DevOps

- Lead Authors:
- Suzette Johnson and Robin Yeman

Industrial DevOps (IDO) extends Lean, Agile, and DevOps principles to the design, development, deployment, and sustainment of cyber-physical systems (CPS). As industries face increasing demands for faster delivery, greater resilience, and digital transformation, IDO offers a framework to improve collaboration, accelerate integration, and reduce risk in complex systems. This article synthesizes insights from practical implementations, principles, and advancements, including AI-enabled digital twins, to illustrate how organizations can build better systems faster.

Concepts

DevOps (development and operations) is a set of principles and practices which enable better communication and collaboration between relevant stakeholders for the purpose of specifying, developing, and operating software and systems, products and services, and continuous improvements in all aspects of the life cycle. (See ISO/IEC/IEEE 32675). Industrial DevOps applies lean, agile and DevOps principles to cyber-physical systems throughout their life cycles. Both are potential approaches to address the strategic aspects of SE agility.

Principles

The principles of Industrial DevOps and their associated benefits are outlined below in Table 1.

Industrial DevOps Principles (SEBoK Original)

Principle	Description	Benefit
1. Organize Around the Flow of Value	Organizing cross functionally around the product delivery value stream.	Reduces handoffs between organizations improving communication and delivery speed.
2. Apply Multiple Horizons of Planning	Links the short-term plans to long-term plans using regular business rhythms to adjust long-term plan based on empirical data.	Increase delivery predictability and reduce risk.
3. Implement Data-Driven Decisions	Use current observations and metrics to determine the state, manage the flow of work across systems of systems.	Increase transparency into product scope, schedule, cost and quality.
4. Architect for Change and Speed	Modular components with standardized interfaces.	Rapidly adapts to change. reducing the lead time and impact of change.
5. Manage Queues and Create Flow	Minimize work in progress and focus on smaller batch sizes.	Reduce bottlenecks and Increase delivery speed.
6. Establish Cadence /Synchronization	Put all teams and their work on the same synchronized cadence.	Enables shorter integration loops resulting in risk reduction.
7. Integrate Early and Often	Integrate products and resources using frequent integration points.	Faster feedback cycles and reduced risk.

8. Shift Left	Shifting left emphasizes a “test-first” mindset encompassing the multiple levels of testing across cyber-physical systems.	Reduced rework, built in quality resulting in increased delivery speed.
9. Adopt a Growth Mindset	Applying a growth mindset expresses the need to continuously learn.	Increased quality and innovation.

Enabling practices

The enabling practices of Industrial DevOps (IDO) shown in the bulleted list below translate its core principles into actionable strategies that drive real-world implementation across complex, cyber-physical systems. These practices support the seamless integration of hardware and software development, enabling organizations to respond more rapidly to change, reduce delivery risk, and enhance product quality throughout the system lifecycle. As industrial systems face increasing demands for speed, adaptability, and resilience, these practices help bridge traditional silos between engineering disciplines, streamline workflows, and foster a culture of continuous improvement.

From value stream management and test automation to the use of AI-enabled digital twins and infrastructure as code, these practices offer the tools and methods necessary to operationalize IDO. They not only improve technical execution but also support organizational transformation by aligning teams, automating compliance, and promoting agile program execution. Together, they form a foundation that empowers industrial enterprises to build better systems faster while maintaining the rigor and reliability required in high-stakes environments.

- **Value Stream Management** - Enables visualization and optimization of work from concept to deployment. It helps identify delays, waste, and inefficiencies across the full value stream.
- **Integrated Digital Engineering (IDE)** - Leverages digital models and simulations (such as digital twins) to support continuous integration and validation of both hardware and software systems.
- **Agile Program Execution** - Focuses on aligning multiple teams through program increment planning, system demos, and synchronized cadences to deliver value effectively and predictably.
- **Test Automation and Virtualization** - Includes test-first approaches, automated testing, and use of virtual environments to verify functionality early and continuously throughout the development cycle.
- **Continuous Integration and Deployment (CI/CD)** - Enables teams to integrate, test, and release updates rapidly and safely, even in complex, safety-critical environments.
- **Infrastructure as Code (IaC)** - Uses code to manage and provision hardware and software infrastructure, enabling repeatable and reliable deployments.
- **AI and Digital Twins** - Supports predictive maintenance, performance optimization, and real-time decision-making by simulating the behavior of physical systems through digital replicas.
- **Lean Governance and Compliance Automation** - Applies automation and lean thinking to ensure that governance, regulatory, and security requirements are met without slowing delivery.

References

Works Cited

ISO/IEC/IEEE 24748-10. (2025 - Under Development). *Systems and software engineering — Life cycle management. Part 10: Guidelines for systems engineering agility*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/90086.html>.

ISO/IEC/IEEE 32675. 2022. *Information technology — DevOps — Building reliable and secure systems including application build, package and deployment*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/83670.html>.

Industrial DevOps Book

Primary References

None.

Additional References

Johnson, Suzette, and Robin Yeman. Industrial DevOps: Build better systems faster. IT Revolution, 2023

Johnson, S., LaFortune, D., Leffingwell, D., Koehnemann, H., Magill, S., Mayner, S., Ofer, A., Wallgren, A., Stroud, R., & Yeman, R. (2018). Industrial DevOps: Applying DevOps and Continuous Delivery to Significant Cyber-Physical Systems. IT Revolution.

Atwell, J., Grinnell, B., Johnson, S., Koehnemann, H., Yeman, R., (2019). Applied Industrial DevOps. IT Revolution
Crook, J., Johnson, S., Koehnemann, H., Shupack, J., Spear, S. J., Yasar, H., Yeman, R., Boleng, J., Wrubel, E., & Kersten, M. (2020). Applied Industrial DevOps 2.0: A Hero's Journey. IT Revolution.

Johnson, S., Yeman, R., Koehnemann, H., Shupack, J., Aizcorbe, M., Cockcroft, A., McKay, M., & Yasar, H. (2021). Building Industrial DevOps Stickiness: Applying Insights. IT Revolution.

Johnson, S., Yeman, R., Koehnemann, H., Shupack, J., Yasar, H., Grinnell, B., Brey, D., Farley, S., & Corman, J. (2022). Overcoming Barriers to Industrial DevOps: Working with the Hardware-Engineering Community.

Fawcett, J., Houston, K., Johnson, S., Moore, B., & Yeman, R. (2024). Accelerating value delivery in highly complex domains: Integrating value stream management, system architecture, and Lean/Agile execution. IT Revolution.

Johnson, S., & Yeman, R. (2024). The Application of Industrial DevOps Using Digital Twins: A Deep Dive Into Building Better Systems Faster. IT Revolution.

Bannon, T., Bensing, B., Brey, D., Johnson, S., Radcliffe, R., Yasar, H., & Yeman, R. (2024). AI-Enabled Digital Twins: Revolutionizing Efficiency for Modern Enterprises. IT Revolution.

Knowledge Area: Life Cycle Model Selection and Adaptation

Life Cycle Model Selection and Adaptation

Contents of this Knowledge Area

- Selecting the Life Cycle Model (David Endler) (Mike Yokell and Garry Roedler)
- Adapting the Life Cycle Model (David Endler) (Mike Yokell and Garry Roedler)
- Selecting the Development Approach (David Endler) (Mike Yokell and Garry Roedler)
- Adapting the Development Approach (David Endler) (Mike Yokell and Garry Roedler)
- Applying Life Cycle Processes (Rick Adcock)
- Lead Author:
 - David Endler
- Contributing Authors:
 - Mike Yokell and Garry Roedler

Life cycle models serve as essential frameworks that guide the development and management of systems and services throughout their existence. Adapted to organizational needs, these models help align projects with strategic goals while accommodating industry regulations and internal diversity. Life Cycle Models and Development Approaches can be selected and adapted to meet specific needs.

References

Works Cited

None.

Primary References

None.

Additional References

None.

Selecting the Life Cycle Model

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Life cycle models serve as essential frameworks that guide the development and management of systems and services throughout their existence. Adapted to organizational needs, these models help align projects with strategic goals while accommodating industry regulations and internal diversity. Far from being static, effective life cycle models evolve through continuous improvement and performance measurement. They enable consistent project execution, facilitate knowledge sharing, and support efficient resource use. Creating the right model requires balancing regulatory demands with best practices and corporate culture, ensuring relevance and stakeholder commitment. Sustained success depends on regularly reviewing and adapting these models to changing conditions.

Align to 24748-1, 6.2.6

Concepts

A life cycle model is a framework of processes and activities concerned with the life cycle which can be organized into stages, acting as a common reference for communication and understanding [SOURCE: ISO/IEC/IEEE 24748-1:2024]. Decision-making reviews or gates based on specific entry or exit criteria are typically part of a life cycle model. Organizations create and maintain life cycle models to develop systems or services that meet their strategic plans, policies, goals and objectives.

Life cycle models should always be organization-specific, even if there are regulatory requirements regarding the life cycle model in certain industries. Creating a life cycle model is one of the biggest challenges for organizations, as it is often the case that a large number of very different projects have to be managed. These differences can be due to the size of the project, the degree of complexity, the proportion of new technologies, customer requirements, or approval regulations when entering new business areas, for example. Accordingly, it can happen that organizations develop several alternative life cycle models and draw up instructions as to which one is to be used in which cases and what adaptation may be needed.

The selected life cycle models should not be seen as rigid constructs. On the one hand, they are adapted to the project-specific requirements for each project and, on the other, periodic reviews should take place to check whether the current life cycle models are still up-to-date and appropriate. The life cycle models are therefore subject to continuous evaluation and improvement, which must keep pace with the speed at which the organization or customer requirements change. For this reason, it is advisable to develop key performance indicators together with the life cycle models that make it possible to measure efficiency and effectiveness in meeting requirements. These key performance indicators should then also be used to continuously improve the life cycle models and associated planning.

Value proposition

The introduction of company-wide processes has many advantages. The associated value propositions are [SOURCE: INCOSE SEHv5, 7.1.1.1]

- Provide repeatable/predictable performance across the projects in the organization (this helps the organization in planning and estimating future projects and in demonstrating reliability to stakeholders)
- Leverage practices that have been proven successful by certain projects and instill those in other projects across the organization (where applicable)
- Enable process improvement across the organization
- Improve ability to efficiently transfer staff across projects as roles are defined and performed consistently
- Enable leveraging lessons that are learned from one project for future projects to improve performance and avoid issues
- Improve startup of new projects (less reinventing the wheel)

Additional benefits can be generated through the use of shared resources (e.g., tool support, document templates, knowledge management database).

Establish life cycle models

The life cycle models created by organizations are usually based on the following inputs:

- Legislative, regulatory, and other government requirements
- Industry SE and management-related standards, training, and capability maturity models
- Academic education, research results, future concepts and perspectives, and requests for financial support

Every organization would do well to create its own life cycle models. Since every organization is unique in terms of its industry, history and culture, the life cycle models should reflect this. Experience shows that life cycle models that work excellently in one organization cannot easily be transferred to other organizations.

In order to create the “right” life cycle models, all relevant stakeholders should be involved. This increases their commitment to the rules created and prevents the not-invented-here syndrome.

The creation of suitable life cycle models is therefore always a balancing act between regulatory requirements, appropriate best practices and the corporate culture or company history.

Maintain life cycle models

As the input variables for selected life cycle models are constantly changing, it is advisable to constantly monitor developments in relation to the inputs listed under “Establish life cycle models” above.

References

Works Cited

INCOSE. 2023. "Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities," 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.

ISO/IEC/IEEE 24748-1. 2024. "Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management." Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html>.

Primary References

None.

Additional References

None.

Adapting the Life Cycle Model

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Adapting a life cycle model to suit a specific project requires thoughtful consideration of both system scope, organizational context, and customer requirements. Modifying stages, criteria, or reviews can help tailor the model to unique needs, but such changes must be guided, documented, and often approved to maintain alignment with organizational standards. Adaptation must also account for multi-party collaborations, where responsibilities shift across different life cycle phases. Additionally, differing domains like hardware and software may call for distinct, but synchronized models. To ensure coherence and efficiency, customized approaches must be selected for each stage (i.e., development through retirement) based on the project's technical and operational demands.

Adaptation of the Model and Stages for Scope

Once an appropriate life cycle model has been selected, it can be adapted to meet the specific needs of the project and system. These specific needs can originate both inside (e.g., product line considerations, follow-on business) and outside (e.g., regulations, customer requirements or preferences) the organization. When adapting a life cycle model, it is important to consider the overall scope. If other projects carried out in the same organization are ignored, some of the value propositions listed in Article “Article – Selecting the Life Cycle Model” may no longer be fulfilled.

When adapting a life cycle model, stages can be added, deleted, combined, or modified as appropriate. Adaption guidelines are usually developed together with the life cycle models, which specify the extent to which life cycle models may be adapted. As soon as these limits are exceeded, it is necessary for these adaptations to be approved by the organization. In general, all adaptations must be documented. Adaption also includes any changes to entry or exit criteria of the reviews or audits to facilitate transitions between stages.

A case could be that organization X buys a concept from organization A, then establishes a project to develop the system, then outsources the production to organization B, then sells the system to organization C, which is responsible for operation and maintenance, and in the end is itself responsible for the disposal of the system. In this case, organization X is responsible for the life cycle stages for development and retirement. At the same time, the transitions to the other life cycle stages must be considered as well.

Adaptation for domains

A life cycle model appropriate for a software system might be different from the life cycle model appropriate for a system dominated by hardware. It is not necessary that the software uses a life cycle appropriate for the hardware, nor is it necessary for the hardware to use a life cycle appropriate for the software. A system that has both hardware and software elements could have a combination of life cycle models, synchronized at certain points. Life cycle models should be adapted based on the domains involved. Each domain's perspective on the system element's life cycle should be considered.

Selection and Adaptation of Approaches for all Stages of the Life Cycle

Regardless of the life cycle model, each of the life cycle stages needs to have an approach selected and adapted to the context of the system and project. Systems engineers will be familiar with the need to select and adapt a development approach for the system. Note that the development approach can be different for different elements of the system. In addition, approaches need to be selected and adapted for the other life cycle stages: production, utilization, support, and retirement. Planning for these should be documented.

References

Works Cited

ISO/IEC/IEEE 24748-1. 2024. "Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management." Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html>.

Primary References

None.

Additional References

None.

Selecting the Development Approach

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Choosing the right development approach is a critical step after selecting and adapting a life cycle model. Each system element or enabling system may require its own approach, especially in complex or innovative projects involving new technologies or business domains. Options like sequential, incremental, evolutionary, or agile must be weighed based on risk, system size, requirements clarity, and other contextual factors. Tools such as risk analysis and frameworks like PMI's Situation Context Framework offer valuable guidance. A shared understanding between acquirer and supplier is essential to ensure alignment on strategy, resources, and delivery expectations throughout the system's development.

Select the Development Approach

Within the selected life cycle model for the system of interest, and after that model has been adapted as appropriate, the specifics of the development approach for the system of interest should be selected and adapted. Determination of the development approach should be performed not just for the system of interest, but for each system element, as well as each enabling system. It is possible, perhaps likely, that the development approaches will differ.

A particular challenge arises when the system of interest involves new technologies or is developed to target a new business area. It is then very likely that development approaches used previously are no longer up-to-date and applicable.

Candidate development approaches include Sequential, Incremental, Evolutionary, and Agile. Depending on what is known and what is not known, a development approach can be more favorable in terms of reducing risk. Risk analysis can be used to compare the development approaches to help guide the selection process.

There are good practices on how to select a suitable development approach, for example in ISO/IEC/IEEE 24748-1. The Project Management Institute (PMI) published their Situation Context Framework (SCF) ^[1] that “defines how to select and tailor a situation-dependent strategy for software development. The SCF is used to provide context for organizing your people, process, and tools for a software-based solution delivery team.” This framework defines seven dimensions (team size, geographic distribution, organizational distribution, skill availability, compliance, domain complexity, and solution complexity) with scaling factors in each dimension.

Considerations:

- How well are the requirements understood
- How large is the system
- How quickly is technology changing
- How constrained is the staff
- How constrained is the budget
- Does the user have preferences on capabilities in first delivery
- Does the user need to phase out an old system at once

There should be agreement between the acquirer and supplier on the development approaches.

References

Works Cited

<https://www.pmi.org/disciplined-agile/agility-at-scale/tactical-agility-at-scale/scaling-factors>

Primary References

ISO/IEC/IEEE 24748-1. 2024. *Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers.

Additional References

None.

References

[1] <https://www.pmi.org/disciplined-agile/agility-at-scale/tactical-agility-at-scale/scaling-factors>

Adapting the Development Approach

- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Adapting development approaches to suit the unique demands of each project is essential for success, especially when facing new technologies, diverse system elements, or changing business strategies. As projects grow more complex, with components spanning different domains like software and hardware, coordinated development becomes increasingly important. Aligning development cycles, managing interface consistency, and considering domain-specific requirements all influence how approaches are adapted. Additionally, certain quality characteristics such as stealth in military systems or safety in healthcare demand further refinement of the development process. Thoughtful adaptation, led by experienced systems engineers, ensures development strategies remain fit for purpose in diverse and evolving contexts.

Adaptation of the Development Approach

Since every project is unique, in most cases it is necessary to adapt not only the life cycle model but also the development approach to the project-specific circumstances. There can be many reasons for this adaptation. For example, if for the organization previously unknown technologies are to be implemented, the approaches used so far are probably no longer up to date. It is also possible that the system elements are very different from one another in terms of their domain characteristics (e.g. software-intensive vs. purely mechanical). Another reason could be that the project is taking a different approach to make or buy than in previous projects.

Adaptation for domains

For larger, complex systems in particular, different development approaches may be used for different system elements. If these development approaches are then also adapted, it is extremely important that the development results of the individual system elements are regularly aligned with each other. These intervals should not be too short. For example, if the software team develops a new release within two weeks, it is unlikely that the associated hardware will have undergone major changes in the same period. At the same time, the intervals should not be too long, as design decisions may then be made without taking other domains into account. As a consequence, there will be inconsistencies at the interfaces that are difficult to resolve.

In most cases, it is the task of experienced systems engineers to adapt the development approaches. In order to come to an educated decision, it is advisable to involve those responsible for the individual system elements. This encourages buy-in from everyone involved and ensures that as many opinions and concerns as possible have been taken into account.

Adaptation for quality characteristics

Some system developments are dominated by certain quality characteristics. For example, when designing a military submarine, it is extremely important that its signature is kept to an absolute minimum. In healthcare projects, topics like system safety and security are usually of particular importance.

In such cases, the development approaches for the system or some of its system elements may have to be adapted to take these peculiarities into account. In the example of the military submarine, it is important that the individual system elements are well coordinated. The reason for this is that even small effects can build up as soon as system elements are integrated into larger assemblies or the overall system of interest. The development approach must therefore aim to ensure that the exchange between the individual departments that develop the system elements functions as smoothly as possible.

In healthcare projects, not only the end product but also the process to it is closely examined by the responsible authorities. The focus in such projects is therefore on predictability, stability, repeatability, and high assurance. All these are characteristics of sequential development approaches.

References

Works Cited

ISO/IEC/IEEE 24748-1. 2024. "Systems and software engineering — Life cycle management, Part 1: Guidelines for life cycle management." Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. Available at <https://www.iso.org/standard/84709.html>.

Primary References

None.

Additional References

None.

Applying Life Cycle Processes

- Lead Author:
- Rick Adcock

The Generic Life Cycle Model describes a set of life cycle stages and their relationships. In defining this we described some of the technical and management activities critical to the success of each stage. While this association of activity to stage is important, we must also recognize the through life relationships between these activities to ensure we take a systems approach.

Systems Engineering technical and management activities are defined in a set of life cycle processes. These group together closely related activities and allow us to describe the relationships between them. In this topic, we discuss a number of views on the nature of the inter-relationships between process activities within a life cycle model.

In general, the technical and management activities are applied in accordance with the principles of concurrency, iteration and recursion. These principles overlap to some extent and can be seen as related views of the same fundamental need to ensure we can take a holistic systems approach, while allowing for some structuring and sequence of our activities. The views presented below should be seen as examples of the ways in which different SE authors present these overlapping ideas.

Life Cycle Process Terminology

Process

A process is a series of actions or steps taken in order to achieve a particular end. Processes can be performed by humans or machines transforming inputs into outputs.

In the SEBoK, processes are interpreted in several ways, including: technical, life cycle, business, or manufacturing flow processes. Many of the Part 3 sections are structured along technical processes (e.g. design, verification); however, Life Cycle Models also describes a number of high-level program life cycle sequence which call themselves processes (e.g. rational unified process (RUP), etc.).

Part 4: Applications of Systems Engineering and Part 5: Enabling Systems Engineering utilize processes that are related to services and business enterprise operations.

Systems Engineering life cycle processes define technical and management activities performed across one or more stages to provide the information needed to make life cycle decisions; and to enable realization, use and sustainment of a system-of-interest (SoI) across its life cycle model as necessary. This relationship between life cycle models and process activities can be used to describe how SE is applied to different system contexts.

Requirement

A requirement is something that is needed or wanted but may not be compulsory in all circumstances. Requirements may refer to product or process characteristics or constraints. Different understandings of requirements are dependent upon process state, level of abstraction, and type (e.g. functional, performance, constraint). An individual requirement may also have multiple interpretations over time.

Requirements exist at multiple levels of enterprise or systems with multiple levels of abstraction. This ranges from the highest level of the enterprise capability or customer need to the lowest level of the system design. Thus, requirements need to be defined at the appropriate level of detail for the level of the entity to which they apply. See the articles System Concept Definition and System Requirements Definition for further detail on the transformation of needs and requirements from the enterprise to the lowest system element across concept definition and system definition.

Architecture

An architecture refers to the organizational structure of a system, whereby the system can be defined in different contexts. Architecting is the art or practice of designing the structures. See below for further discussions on the use of levels of Logical and Physical architecture models to define related system and system elements; and support the requirements activities.

Architectures can apply for a system product, enterprise, or service. For example, Part 3 mostly considers product or service related architectures that systems engineers create, but enterprise architecture describes the structure of an organization. Part 5: Enabling Systems Engineering interprets enterprise architecture in a much broader manner than an IT system used across an organization, which is a specific instance of architecture.

Frameworks are closely related to architectures, as they are ways of representing architectures. See Architecture Framework for definition and examples.

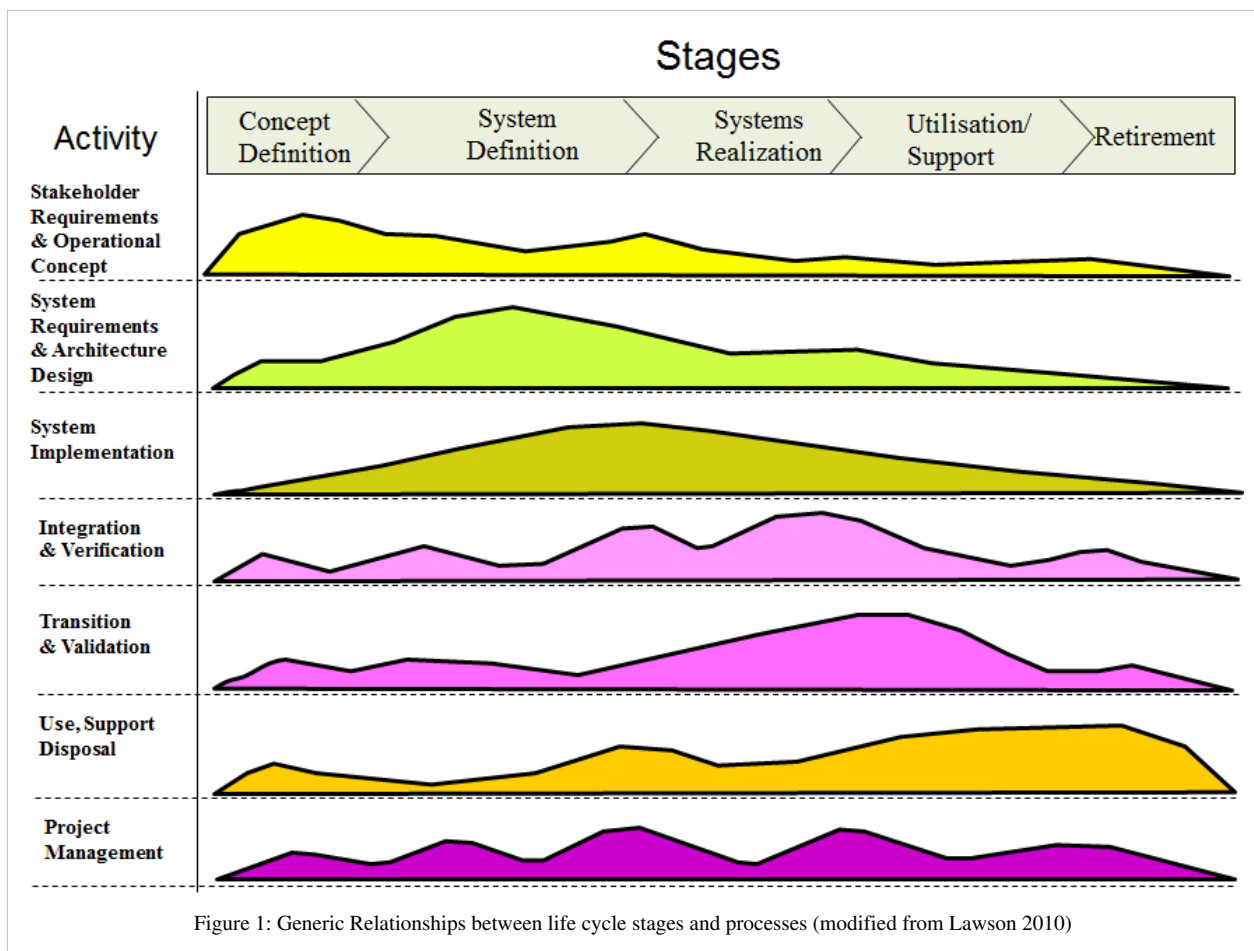
Other Processes

A number of other life cycle processes are mentioned below, including system analysis, integration, verification, validation, deployment, operation, maintenance and disposal; they are discussed in detail in the System Realization and System Deployment and Use knowledge areas.

Life Cycle Process Concurrency

In the Generic Life Cycle Model, we have listed key activities critical to successful completion of each stage. This is a useful way to illustrate the goals of each stage and gives an indication of how processes align with these stages. This can be important when considering how to plan for resources, milestones, etc. However, it is important to observe that the execution of process activities is not compartmentalized to particular life cycle stages (Lawson 2010).

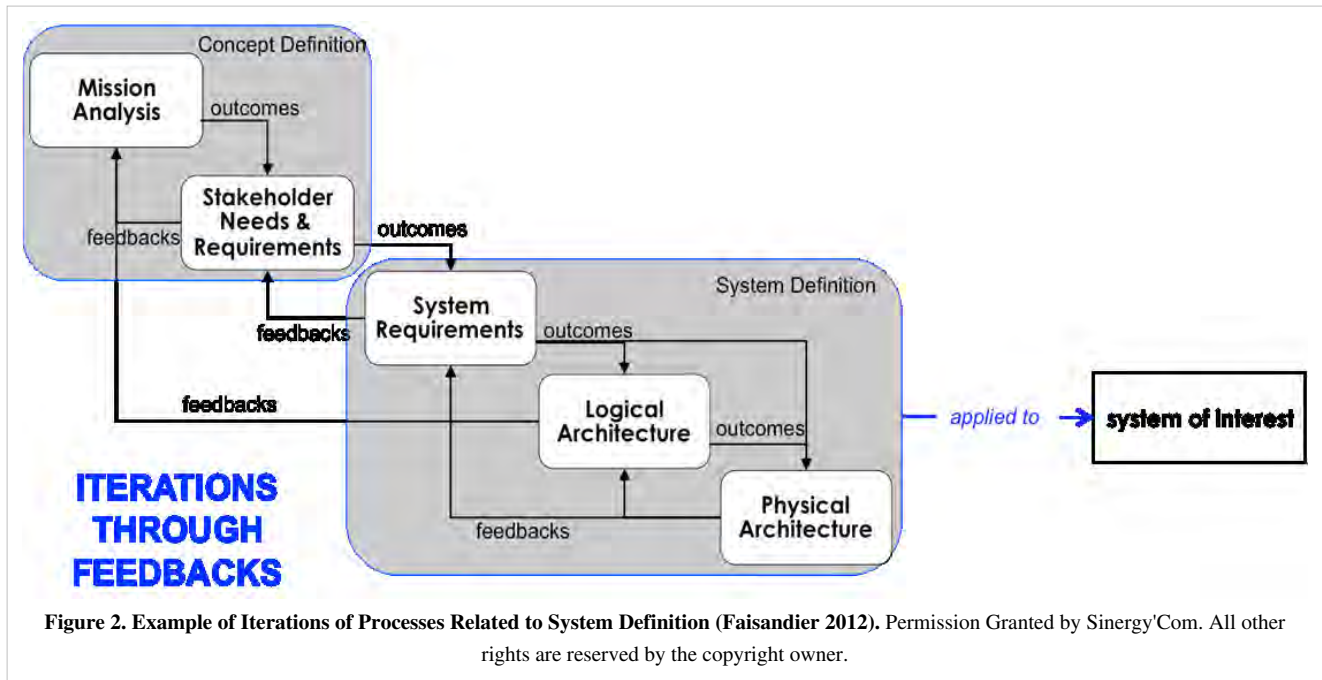
Figure 1 shows a simple illustration of the through life nature of technical and management processes. This figure builds directly on the "hump diagram" principles described in Systems Approach Applied to Engineered Systems.



The lines on this diagram represent the amount of activity for each process over the generic life cycle. The peaks (or humps) of activity represent the periods when a process activity becomes the main focus of a stage. The activity before and after these peaks may represent through life issues raised by a process focus, e.g. how likely maintenance constraints will be represented in the system requirements. These considerations help maintain a more holistic perspective in each stage, or they can represent forward planning to ensure the resources needed to complete future activities have been included in estimates and plans, e.g. all resources needed for verification are in place or available. Ensuring this hump diagram principle is implemented in a way which is achievable, affordable and appropriate to the situation is a critical driver for all life cycle models.

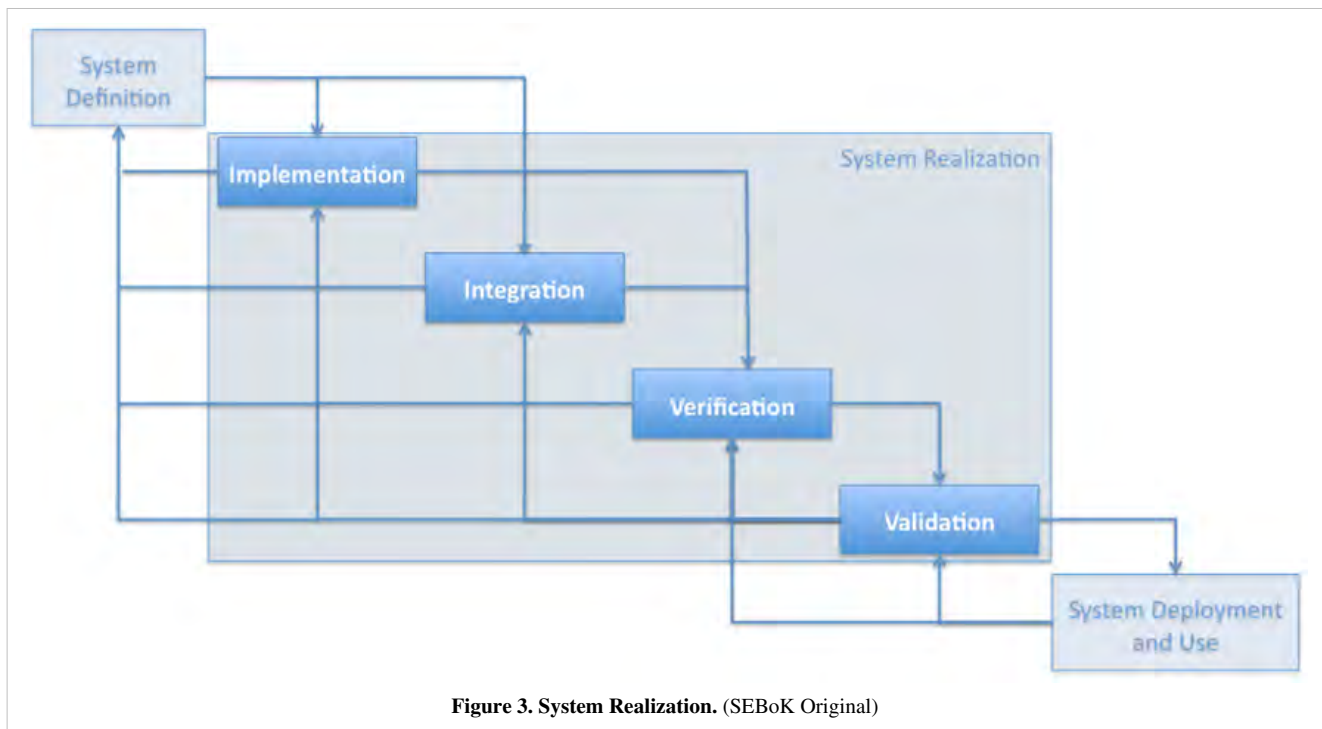
Life Cycle Process Iteration

The concept of iteration applies to life cycle stages within a life cycle model, and also applies to processes. Figure 2 below gives an example of iteration in the life cycle processes associated with concept and system definition.



There is generally a close coupling between the exploration of a problem or opportunity and the definition of one or more feasible solutions; see Systems Approach to Engineered Systems. Thus, the related processes in this example will normally be applied in an iterative way. The relationships between these processes are further discussed in the System Concept Definition KA.

Figure 3 below gives an example of the iteration between the other life cycle processes.

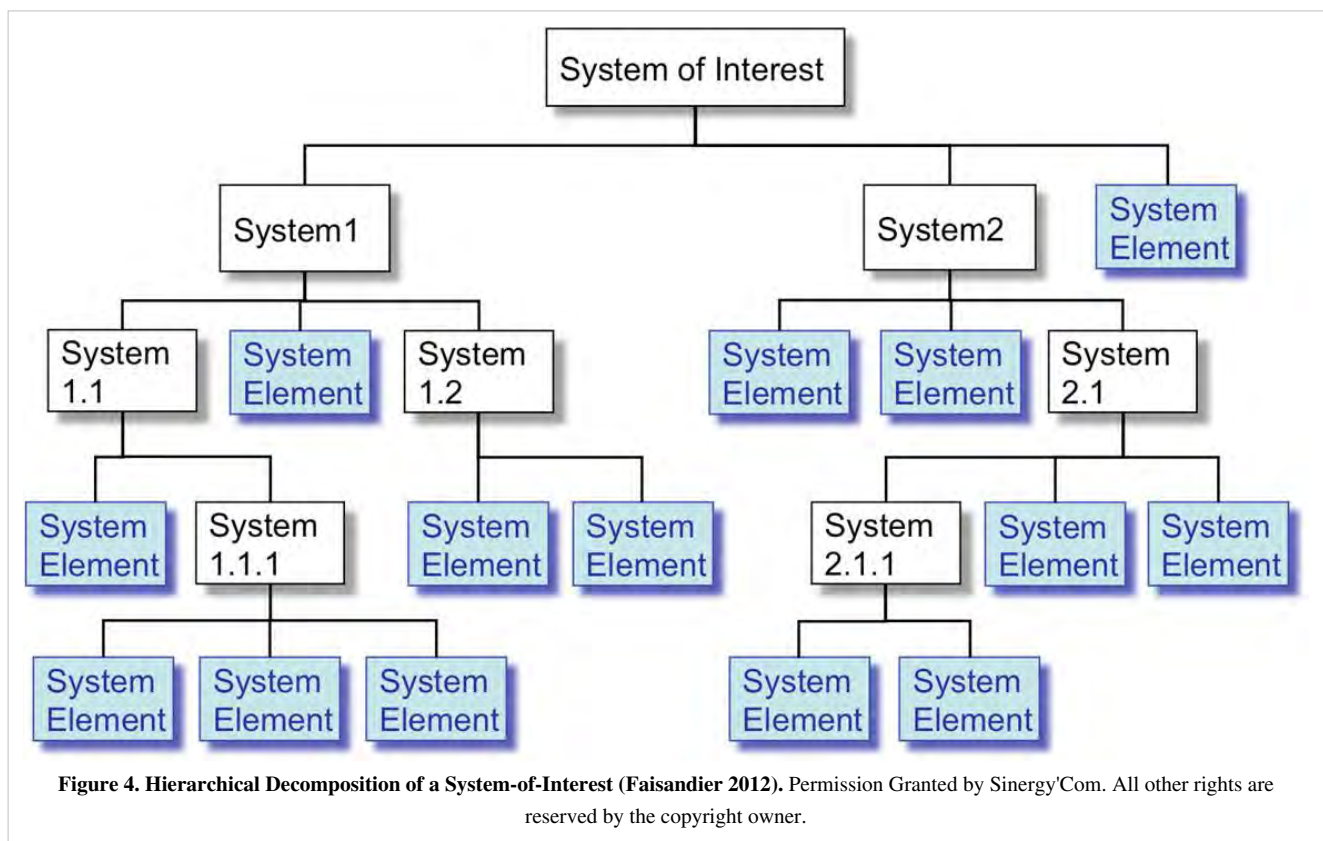


The iterations in this example relate to the overlaps in process outcomes shown in Figure 1. They either allow consideration of cross process issues to influence the system definition (e.g. considering likely integration or verification approaches might make us think about failure modes or add data collection or monitoring elements into the system) or they allow risk management and through life planning activities to identify the need for future activities.

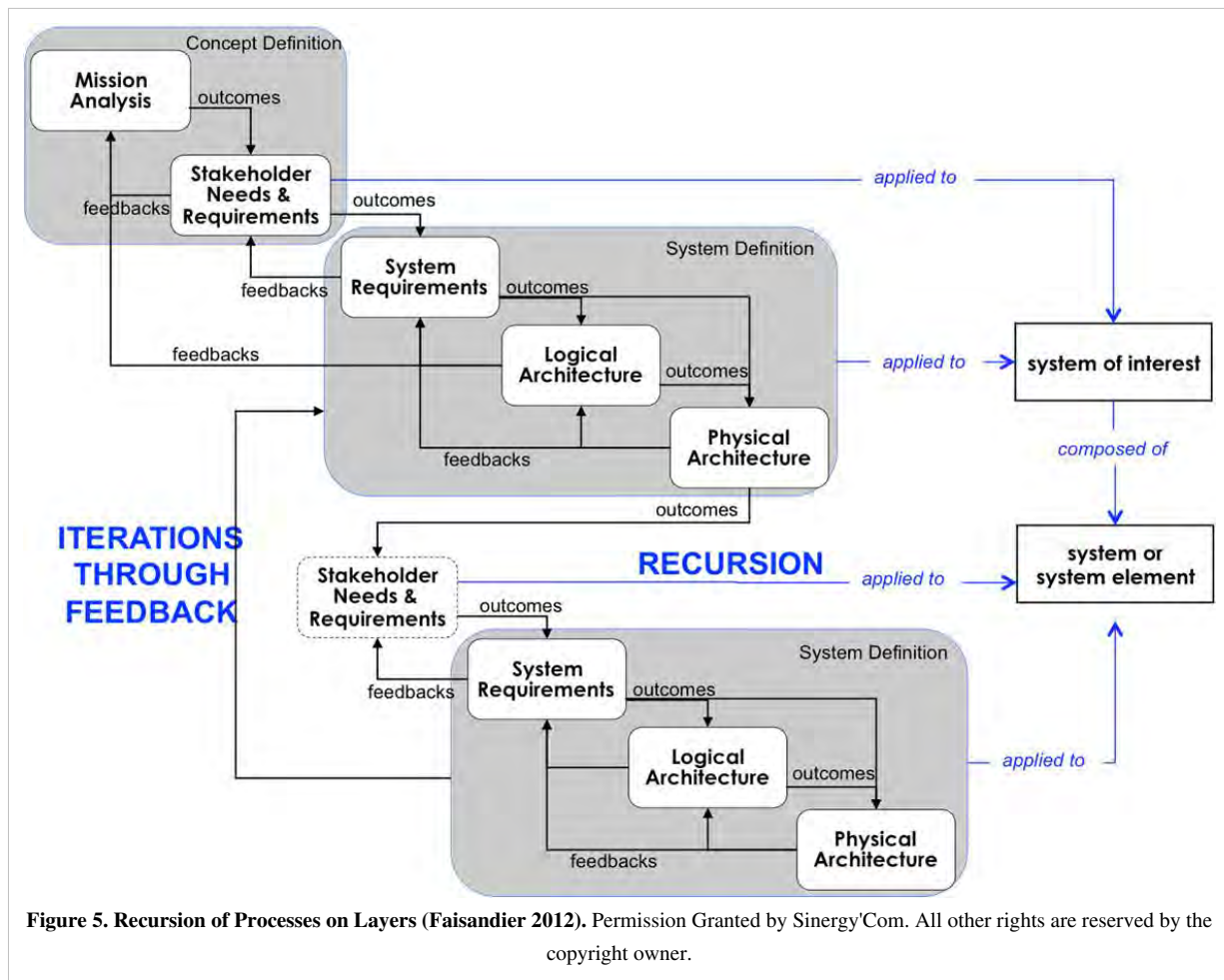
The relationships between these processes are further discussed in system realization and system deployment and use.

Life Cycle Process Recursion

The comprehensive definition of a SoI is generally achieved using decomposition layers and system elements. Figure 4 presents a fundamental schema of a system breakdown structure.



In each decomposition layer and for each system, the System Definition processes are applied recursively because the notion of "system" is in itself recursive; the notions of SoI, system, and system element are based on the same concepts (see Part 2). Figure 5 shows an example of the recursion of life cycle processes.



Systems Approach to Solution Synthesis

The sections above give different perspectives on how SE life cycle processes are related and how this shapes their application. Solution synthesis is described in Part 2 as a way of taking a systems approach to creating solutions. Synthesis is, in general, a mixture of top-down and bottom-up approaches as discussed below.

Top-Down Approach: From Problem to Solution

In a top-down approach, concept definition activities are focused primarily on understanding the problem, the operational needs/requirements within the problem space, and the conditions that constrain the solution and bound the solution space. The concept definition activities determine the mission context, mission analysis, and the needs to be fulfilled in that context by a new or modified system (i.e. the SoI), and address stakeholder needs and requirements.

The system definition activities consider functional, behavioral, temporal, and physical aspects of one or more solutions based on the results of concept definition. System analysis considers the advantages and disadvantages of the proposed system solutions both in terms of how they satisfy the needs established in concept definition, as well as the relative cost, time scales and other development issues. This may require further refinement of the concept definition to ensure all legacy relationships and stakeholders relevant to a particular solution architecture have been considered in the stakeholder requirements.

The outcomes of this iteration between Concept Definition and System Definition define a required system solution and its associated problem context, which are used for System Realization, System Deployment and Use, and Product and Service Life Management of one or more solution implementations. In this approach, problem

understanding and solution selection activities are completed in the front-end portion of system development and design and then maintained and refined as necessary throughout the life cycle of any resulting solution systems. Top-down activities can be sequential, iterative, recursive or evolutionary depending upon the life cycle model.

Bottom-Up Approach: Evolution of the Solution

In some situations, the concept definition activities determine the need to evolve existing capabilities or add new capabilities to an existing system. During the concept definition, the alternatives to address the needs are evaluated. Engineers are then led to reconsider the system definition in order to modify or adapt some structural, functional, behavioral, or temporal properties during the product or service life cycle for a changing context of use or for the purpose of improving existing solutions.

Reverse engineering is often necessary to enable system engineers to (re)characterize the properties of the system-of-interest (SoI) or its elements. This is an important step to ensure that system engineers understand the SoI before beginning modification. For more information on system definition, see the System Concept Definition article.

A bottom-up approach is necessary for analysis purposes, or for (re)using existing elements in the design architecture. Changes in the context of use or a need for improvement can prompt this. In contrast, a top-down approach is generally used to define an initial design solution corresponding to a problem or a set of needs.

Solution Synthesis

In most real problems, a combination of bottom-up and top-down approaches provides the right mixture of innovative solution thinking driven by need, and constrained and pragmatic thinking driven by what already exists. This is often referred to as a “middle-out” approach.

As well as being the most pragmatic approach, synthesis has the potential to keep the life cycle focused on whole system issues, while allowing the exploration of the focused levels of detail needed to describe realizable solutions; see Synthesising System Solutions for more information.

References

Works Cited

Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications.

Primary References

INCOSE. 2015. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 4.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-118-99940-0.

Additional References

None.

Part 3b: Process Concepts

Knowledge Area: Process Concepts

Process Concepts

Contents of this Knowledge Area

- Process Description (David Endler) (Mike Yokell)
- Process Concurrency, Iteration, and Recursion (David Endler) (Mike Yokell)
- Lead Author:
- David Endler
- Contributing Authors:
- Mike Yokell and Garry Roedler

Systems engineering relies on a well-defined set of processes that guide the development of complex systems along the whole life cycle. These processes form the backbone of systematic engineering practice, differentiating it from ad-hoc approaches. ISO/IEC/IEEE 24774 standardizes process descriptions to ensure clarity, consistency, and comparability. However, they are not meant to be executed in a serial, sequential fashion. Concurrency, iteration, and recursion ensure dynamic, non-linear system life cycle management.

References

Works Cited

None.

Primary References

None.

Additional References

None.

Process Description

- Lead Author:
- David Endler
- Contributing Author:
- Mike Yokell

A life cycle model provides a framework of processes and activities guiding a system’s life cycle. ISO/IEC/IEEE 24774 standardizes process descriptions to ensure clarity, consistency, and comparability. Each process includes a name, purpose, and outcomes, with optional activities, tasks, inputs, and outputs. Consistent descriptions enhance understanding, compliance, and improvement, supporting systematic, transparent, and adaptable system engineering practices.

Fundamental Concepts

Article Life Cycle Models describes that a life cycle model is a framework of processes and activities concerned with the life cycle. Processes and activities that are integrated into the framework of a life cycle model are often referred to as life cycle processes.

The purpose of ISO/IEC/IEEE 24774, *Specification for process description*, (available for free at: <https://www.iso.org/standard/78981.html>) is to “encourage uniformity in the description of processes”. This international standard specifies the way process descriptions should be established and builds on the lessons learned from other international, national, and industry standards that describe processes and process models.

Defining processes in a consistent manner ensures that they are understandable, comparable, and manageable. Consistency provides a common language for describing how work is done, which reduces ambiguity and simplifies collaboration. It enables alignment with standards, regulations, and good practices, ensuring that processes are not only efficient but also compliant. When processes follow a consistent structure, it becomes easier to document, analyze, and improve them systematically. This also supports training and knowledge transfer, as users can quickly grasp and apply established methods.

Processes can be specified using the following elements (ISO/IEC/IEEE 24774):

Table 1. Process Elements. (SEBoK Original)

	Required Element	Optional Elements	For Any Other Element
Process	Name	Inputs	Notes
	Purpose	Activities	Controls
	Outcomes	Tasks	Constraints
		Outputs	

According to ISO/IEC/IEEE 24774, at a minimum, process descriptions include the elements **Name, Purpose, and Outcomes**. Additional details, such as inputs, outputs, activities, or tasks, can be added if needed but are not mandatory. The core elements capture the goals and objectives of the process, focusing on the intended results without requiring a detailed structural breakdown. By defining processes in terms of Name, Purpose, and Outcomes, organizations establish a clear and consistent foundation. This approach ensures that processes can be easily understood, implemented, and assessed, while still allowing for further elaboration if deeper analysis or refinement becomes necessary.

Elements of Process Description

Process Name (Required)

The name of a process should be a concise noun phrase that clearly identifies its main focus and distinguishes it from other processes. Verbs should be avoided because they can lead to situations where the process name resembles activities or purposes.

Process Purpose (Required)

The purpose of a process should express one or more high-level goals that define why the process is performed. It helps clarify the scope and boundaries, especially when processes appear to overlap. Ideally, the purpose is stated in a single, concise sentence, beginning with *"The purpose of the xxx process is..."*. It should not summarize activities or outcomes, and the use of "and" should be avoided to prevent listing unrelated goals. Additional context or explanations can be provided separately in notes or supporting text.

Process Outcomes (Required)

Process outcomes represent measurable and observable results or change of state achieved by executing a process. Unlike outputs, they are not defined as documents, records, or information items, but rather as tangible technical or business achievements. Each outcome should be written as a single, present-tense declarative sentence, expressing a clear, positive objective such as delivering a service, achieving a change of state, maintaining a desired condition, or meeting defined requirements. To ensure clarity, outcomes should avoid conjunctions like "and" or "and/or", separate statements should be used instead. Typically, a process should have three to seven concise outcomes, each directly supporting its stated purpose. Outcomes are distinct from benefits, which describe broader organizational gains and can be recorded separately as informative notes.

Process Activities (Optional)

According to ISO/IEC/IEEE 24774, activities are defined as "set of cohesive tasks of a process". Process activities describe the broad actions undertaken to execute a process. They group related tasks (see below) to improve understanding and communication and can, if cohesive and well-defined, be treated as lower-level processes with their own purposes and outcomes. All activities together must achieve the process outcomes and purpose. Activities should be strongly related internally (across its tasks) but loosely connected to other activities. They are not sequential steps but ongoing responsibilities, avoiding imposed timing or order unless explicitly required.

Process Tasks (Optional)

According to ISO/IEC/IEEE 24774, a task is defined as a "required, recommended, or permissible action, intended to contribute to the achievement of one or more outcomes of a process".

Process tasks specify concrete requirements, recommendations, or permissible actions that guide the execution of a process. They are written to support the achievement of process outcomes but do not need to fully cover the scope of an activity. Instead, tasks can be seen as discrete points within the broader space of processes and activities. Additional tasks can be included to clarify conformance requirements or to support higher capability levels. Timing or sequencing should not be assumed unless explicitly stated, ensuring flexibility across different life cycle models.

Process Inputs (Optional)

Process inputs are items that get transformed into outputs through the execution of the process, excluding the human or automated resources that perform the work. Inputs can originate from other processes, organizational resources, suppliers, or external sources. While specifying inputs is optional, it becomes essential in closed-loop life cycles.

Process Outputs (Optional)

Process outputs can be specified in descriptions but are optional if outcomes can be demonstrated. Some outputs are essential deliverables, while others serve only for validation or audit. They are generally categorized as artefacts (e.g., prototypes, models, components, or services) or information items with defined characteristics. Artifacts further include four generic work product types: services, software, hardware, and processed materials.

Process Notes (For Any Other Element)

Process notes contain further information on the what, how and why. In many cases, they explain the interrelationship with other processes and also main contain brief examples.

Process Controls and Constraints

Process controls and process constraints guide or restrict how a process is performed, and to some extent when. Controls can originate from laws, regulations, organizational policies, voluntary standards, or agreements with suppliers and customers. Constraints often stem from external environmental or business conditions. They can be documented in a dedicated section of the process description or included as notes alongside other process elements.

Elaboration

In INCOSE SEHv5 section 2.3 a common structure has been applied to describe the system life cycle processes. To maintain alignment with ISO/IEC/IEEE 15288, the purpose statements from the standard are reproduced verbatim for each process described in INCOSE SEHv5. Similarly, the titles of process activities follow the standard, describing **what** should be done rather than **how** to perform the tasks. Additional elements are included in some cases to summarize industry best practices and recent developments in systems engineering processes. Process elaborations provide further details specific to each life cycle process.

Each system life cycle process is also represented by an input-process-output (IPO) diagram, showing typical inputs, activities, and typical outputs. While the outcomes are included in ISO/IEC/IEEE 15288, they are not included in INCOSE SEHv5. Instead, INCOSE SEHv5 contains typical inputs and outputs for each process.

They can take any form (e.g., database, document, model, etc), and combinations of inputs and outputs in one artifact are also possible. The names of the inputs and outputs can vary depending on domain, company, or project.

In INCOSE SEHv5, controls and enablers are typically excluded from input-process-output (IPO) diagrams, in figure 1 they are noted, showing how inputs are transformed into outputs under process controls.

These diagrams illustrate one possible way to perform the process, not a mandatory approach. Outputs often represent results captured as documents, artifacts, or models, rather than being produced solely because they are listed.

References

Works Cited

INCOSE. 2023. "Chapter 2" in *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.

ISO/IEC/IEEE. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023.

ISO/IEC/IEEE. 2021. *Systems and software engineering — Life cycle management — Specification for process description*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24774:2021.

Primary References

None

Additional References

None

Process Concurrency, Iteration, and Recursion

- Lead Author:
- David Endler
- Contributing Author:
- Mike Yokell

Concurrency, iteration, and recursion ensure dynamic, non-linear system life cycle management. Concurrency allows parallel processes; iteration enables refinement through feedback and evolving requirements; recursion applies processes across system levels for consistency. Together, they promote continuous learning, adaptability, and informed decision-making throughout system definition and realization.

Fundamental Concepts

The concepts of concurrency, iteration, and recursion is also discussed in ISO/IEC/IEEE 15288 and article Applying the Systems Approach. While the article in SEBoK Part 2 explains the application of concurrency , iteration, and recursion in the development of complex systems, this article deals with the application of these concepts to life cycle processes.

The system life cycle processes are sometimes mistakenly perceived as linear and sequential, applied at a single system hierarchy level. In reality, effective system life cycle management relies on exchanging information and insights across processes to ensure a system definition that meets stakeholder needs efficiently and effectively. To do this, it is often necessary to account for learning and feedback that is obtained as processes are applied during the system definition. Applying concurrency, iteration, and recursion supports continuous communication, ongoing learning, and informed decision-making as the technical solution evolves.

Concurrency

Concurrency involves performing two or more processes in parallel. Many processes can proceed simultaneously when they do not strongly depend on each other's outputs. For instance, risk management and measurement processes are typically continuous and concurrent.

Iteration

Iteration refers to repeated application and interaction between processes for a variety of reasons, including to accommodate stakeholder decisions, to incorporate learning and feedback, and evolving understanding, architectural constraints, and trade-offs for affordability, feasibility, resilience, and adaptability. Iteration frequently occurs between processes such as system requirements definition and system architecture definition. Evolving requirements can drive architectural changes, which in turn can prompt updates to requirements, design, or trade-offs.

Recursion

Recursion is the repeated application of life cycle processes throughout the system structure. Technical Management and Technical Processes are applied recursively for many reasons, including defining the system at continually lower levels until decisions on make, buy, or reuse are made. Outputs from one element level serve as inputs for another, whether in system definition or system realization, ensuring consistency and integrity.

References

Works Cited

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.

ISO/IEC/IEEE. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023.

Primary References

None

Additional References

None

Knowledge Area: Process Selection and Tailoring

Process Selection and Tailoring

Contents of this Knowledge Area

- Guidelines for the Selection and Tailoring of Processes (David Endler) (Mike Yokell)
- Using ISO/IEC/IEEE 24748-2 (David Endler) (Mike Yokell)
- Tailoring (David Endler) (Mike Yokell)
- Lead Author:
 - David Endler
- Contributing Authors:
 - Mike Yokell and Garry Roedler

Selecting appropriate processes for use throughout the life cycle of systems is a strategic challenge for organizations. In many cases, ISO/IEC/IEEE 24748-2 is a useful resource for this challenge. When starting projects, it is often beneficial to tailor the organization's life cycle processes to the specific needs of the project.

References

Works Cited

None.

Primary References

ISO/IEC/IEEE. 2024b. *Systems and software engineering — Life cycle management — Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-2:2024.

Additional References

None.

Guidelines for the Selection and Tailoring of Processes

- Lead Author:
- David Endler
- Contributing Author:
- Mike Yokell

Selecting suitable life cycle processes is strategic, requiring flexibility, compliance, and alignment with standards like ISO/IEC/IEEE 24748-2. Key principles include risk-based rigor, tailoring, traceability, tool integration, and governance. Success depends on human factors, learning, reuse, and management support. Effective frameworks balance rigor with agility, using feedback, training, and pilot projects to ensure continuous improvement.

Fundamental Concepts

Selecting appropriate processes for use throughout the life cycle of systems is a strategic challenge for organizations. In many cases, ISO/IEC/IEEE 24748-2 is a useful resource for this challenge. The goal is to create a flexible, adaptable framework that ensures compliance, quality, and efficiency while allowing innovation and agility. This requires aligning with consensus standards, learning from past experience, and tailoring processes to project-specific needs.

Core Principles

When selecting system life cycle processes, consider the following principles:

- Risk and context: Process rigor should match system complexity, criticality, and regulatory environment. For example, a safety-critical avionics system requires more rigorous analysis and stringent verification than a home TV screen.
- Tailoring: Consider tailoring upfront, it will happen. In almost every project, the process framework needs to be tailored to the project-specific context, life cycle model and approach, with documented rationale. A process framework with tailoring guidelines is far more effective than a single, uniform process.
- Traceability: Most systems demand end-to-end traceability between requirements, design, implementation, verification results, and other artifacts.
- Tool and data integration: Processes can connect with product lifecycle management / application lifecycle management / configuration management (PLM/ALM/CM) tools to enable automation, audit trails, and reporting.
- Governance and accountability: Clear process ownership, change control boards, and role definitions prevent fragmentation and ensure accountability.

Additional topics of high relevance include:

- Human Factors and Organizational Aspects
 - Competence Management: Processes are only as good as the people executing them. Skills matrices, role training, and certification of staff should be part of the process framework.
 - Culture & Change Management: Process adoption depends strongly on company culture. Introducing new methods requires structured change management.
 - Communication Interfaces: Most systems often involve multiple disciplines. Defined communication rules and cross-disciplinary forums are essential.
 - Knowledge and Lessons Learned
-

- **Organizational Learning:** Processes should include structured mechanisms for capturing lessons learned, good practices, and failure analyses.
- **Reuse & Reference Architectures:** Reusable components, design patterns, and product and process reference architectures can dramatically reduce risk and cost, but only if processes explicitly encourage and govern reuse.

Good Practices

Implementing effective and efficient processes for most systems goes beyond compliance; practical approaches that teams can consistently follow are required. The following good practices provide some guidance, however, tailoring to different project types, technologies, and organizational contexts should be considered.

- **Lightweight Process Frameworks:** Identify and build around core processes (e.g., requirements, architecture, integration, verification, validation, configuration, risk management). Overly detailed process definitions are often ignored. Focus first on essential practices. Note that the optimal process “weight” depends on mission criticality and risk.
- **Tailoring:** Define decision criteria (e.g., size, safety level, novelty, mission criticality, certification needs) to select mandatory vs. optional activities and level of rigor. Ensure tailoring is documented; clear decision trees and real-world examples improve adoption and compliance.
- **Risk-based Verification:** Scale test rigor and independence according to risk and criticality.
- **Supplier governance:** Require suppliers to demonstrate compliance and maturity.
- **Training & coaching:** Adoption requires skilled practitioners and process champions.
- **Measurement and feedback loops:** Collect data on defects, requirements churn, and lead times to refine processes.
- **Management support:** When process compliance is supported and demanded by management, lengthy debates about necessity are avoided. All levels of the project should be able to see the benefit of performing the processes.
- **Pilot projects:** Test process changes in representative projects before full rollout.
- **Effectiveness vs efficiency:** Processes must be effective before they can be made efficient. Getting the wrong answer faster or cheaper doesn’t count.
- **Process assurance:** Life cycle processes form a basis for execution that avoids unintended omission, unnecessary activities and lack of discipline. For example, the project assessment and control process and decision management process provide for iterative incremental development that is at once responsive and disciplined.

Challenges

While a unified process framework offers consistency, in practice, no single process can perfectly fit every project or technology context. Differences in project size, complexity, regulatory requirements, and technology maturity can create conflicts that a single process cannot address. Typical challenges include:

- **Scaling:** Large programs can need more structure in their governance; small projects can leverage lean, flexible processes.
- **Technology maturity:** New technology requires experimentation; legacy products require strict control.
- **Regulatory heterogeneity:** Some domains demand exhaustive documentation, others allow leaner approaches.
- **Organizational maturity:** Processes that exceed a team’s capability can lead to noncompliance.
- **Culture Change, Change Management:** Noncompliance also results from not having a common vision regarding the necessity, not getting adequate training, not providing for effective transition from previous process approaches, etc.
- **Contractual constraints:** OEMs, regulators, or funding bodies may impose specific processes.
- **Myths about processes:** ISO/IEC/IEEE 15288 “does not prescribe a specific system life cycle model, development methodology, method, modelling approach or technique” (ISO/IEC/IEEE 15288:2023, Clause 1). The processes, activities and tasks in ISO/IEC/IEEE 15288 can be used with agile and other techniques with or without tailoring.

References

Works Cited

ISO/IEC/IEEE. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023.

ISO/IEC/IEEE. 2024. *Systems and software engineering — Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-2:2024.

Primary References

None

Additional References

None

Using ISO/IEC/IEEE 24748-2

- Lead Author:
- David Endler
- Contributing Author:
- Mike Yokell

Effective management of life cycle processes ensures quality, risk reduction, and consistency across projects. ISO/IEC/IEEE 15288 provides the foundational framework for system life cycle processes, often extended by domain-specific standards. ISO/IEC/IEEE 24748-2 guides the practical application of 15288 covering strategy, system, life cycle, organizational, project, process, and conformance aspects to ensure structured, adaptable implementation.

Fundamental Concepts

Management and structuring of life cycle processes are critical to achieving successful results. Life cycle processes provide a structured framework for all life cycle stages of systems or products. For organizations striving to ensure quality, reduce risk, and enhance consistency across projects, adopting recognized standards is not only recommended but often essential.

ISO/IEC/IEEE 15288 can be seen as the foundation for domain-specific standards (e.g., ASPICE, ISO 26262, RTCA DO-178C, RTCA DO-254, IEC 62304, ISO 13485), or as a framework for other standards and guides with specific focuses (e.g., ISO/IEC 27000, NIST 800-160, SAE 1001). In many situations it is therefore advisable to use this standard as a starting point and then overlay domain-specific standards to meet regulatory or certification requirements.

According to ISO/IEC/IEEE 24748-2: ISO/IEC/IEEE 24748-2 is a guideline for the application of ISO/IEC/IEEE 15288. It addresses system, life cycle, organizational, project, and process, concept application, principally through reference to ISO/IEC/IEEE 24748-1 and ISO/IEC/IEEE 15288. It gives guidance on applying ISO/IEC/IEEE 15288 from the aspects of strategy, planning, application in organizations, and application on projects. Whereas ISO/IEC/IEEE 15288 is kept short and concise, ISO/IEC/IEEE 24748-2 elaborates on the following aspects:

- Application strategy
- Application of system concepts
- Application of life cycle concepts
- Application of organizational concepts
- Application of project concepts
- Application of process concepts
- Application of conformance and adaptation concepts

Application Strategy

Guidelines are provided for defining the strategy to apply ISO/IEC/IEEE 15288 to

- a specific project,
- improving an organization's life cycle processes, or
- system life cycle processes usable within a larger process (e.g., an organization's acquisition process).

The strategy described in ISO/IEC/IEEE 24748-2 includes the following steps:

- plan the application,
- adapt ISO/IEC/IEEE 15288, if applicable,
- conduct pilot project(s),
- formalize the approach, and
- institutionalize the approach.

Application of System Concepts

Approaches are summarized on how to define the system-of-interest, the system structure, as well as interfacing, enabling, and interoperating systems.

Application of life cycle concepts

This clause of ISO/IEC/IEEE 24748-2 summarizes
the generic life cycle stages,
the decision gates that separate the stages, and
application approaches.

Application of organizational concepts

Guidelines are provided on which topics should be considered when ISO/IEC/IEEE 15288 is applied internally by an organization or ISO/IEC/IEEE 15288 is applied through an agreement by two or more organizations.

Application of project concepts

This clause of ISO/IEC/IEEE 24748-2 summarizes how ISO/IEC/IEEE 15288 can be used by a project. Each of the four process groups (Agreement Processes, Organizational Project-Enabling Processes, Technical Management Processes, and Technical Processes) can be used to establish a common understanding in a project.

Application of process concepts

Detailed guidelines are provided for the life cycle processes defined in ISO/IEC/IEEE 15288.

Application of conformance and adaptation concepts

Guidelines for conformance and adaptation of the life cycle processes defined in ISO/IEC/IEEE 15288 are provided.

References

Works Cited

- ASPICE. 2025. *Automotive Software-based systems Process Improvement and Capability DEtermination*. Version 4.0. Accessed 29 Oct 2025: <https://vda-qmc.de/en/automotive-spice/>
- IEC. 2006. *Medical device software — Software life cycle processes*. International Electrotechnical Commission (IEC). IEC 62304:2006.
- ISO. 2016. *Medical devices — Quality management systems — Requirements for regulatory purposes*. Geneva, Switzerland: International Organization for Standardization (ISO). ISO 13485:2016.
- ISO. 2028. *Road vehicles — Functional safety*. Geneva, Switzerland: International Organization for Standardization (ISO). ISO 26262 Series, 2018.
- ISO/IEC. 2018. *Information technology — Security techniques — Information security management systems — Overview and vocabulary*. International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC). ISO/IEC 27000:2018.
- ISO/IEC/IEEE. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023.
- ISO/IEC/IEEE. 2024a. *Systems and software engineering — Life cycle management — Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.
- ISO/IEC/IEEE. 2024b. *Systems and software engineering — Life cycle management — Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-2:2024.
- NIST 2022. *Engineering Trustworthy Secure Systems*. National Institute of Standards and Technology. NIST SP 800-160 Vol. 1 Rev. 1, 2022.
- RTCA. 2000. *Design Assurance Guidance for Airborne Electronic Hardware*. Radio Technical Commission for Aeronautics. RTCA DO-254, 2000.
- RTCA. 2018. *Software Considerations in Airborne Systems and Equipment Certification*. Radio Technical Commission for Aeronautics. RTCA DO-178C, 2018.
- SAE. 2018. *Integrated Project Processes for Engineering a System*. Warrendale, PA: SAE International, SAE 1001:2018.
-

Primary References

ISO/IEC/IEEE. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023.

ISO/IEC/IEEE. 2024a. *Systems and software engineering — Life cycle management — Part 1: Guidelines for life cycle management*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.

ISO/IEC/IEEE. 2024b. *Systems and software engineering — Life cycle management — Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-2:2024.

Additional References

None

Tailoring

- Lead Author:
- David Endler
- Contributing Author:
- Mike Yokell

Project-level tailoring adapts ISO/IEC/IEEE 15288 processes to specific contexts, balancing rigor, risk, and flexibility. It is dynamic, evidence-based, and aligned with project planning. Success depends on stakeholder collaboration, documentation, and continuous review. Good practices emphasize fact-based decisions, tool integration, and adaptability, while avoiding over-engineering, uniformity, and neglecting stakeholder engagement.

Fundamental Concepts

While organizations provide governance and overarching frameworks (see articles *Adapting the Life Cycle Model* and *Adapting the Development Approach*), tailoring can be needed on the project level. ISO/IEC/IEEE 15288, along with related elaboration standards (ISO/IEC/IEEE 24748 series), defines a comprehensive set of life cycle processes that can be applied to projects of any size and complexity. However, no project can adopt these processes “off the shelf.” They should be tailored to the specific project environment, stakeholders, and risks.

The Role of Tailoring

Tailoring at the project level involves adjusting life cycle processes to ensure that they meet the unique requirements of the project. These adjustments scale the rigor of processes to a level appropriate to project risk, complexity, and constraints. For example:

- A project with tight budgets and low tolerance for failure can benefit more rigorous verification and validation activities.
 - A small project with low complexity and high trust among stakeholders may streamline certain formal reviews to save cost and time.
-

Tailoring is not a one-time activity, but a dynamic process. As risks and circumstances evolve, tailoring should be revisited throughout the life cycle. As shown in Figure 1, it balances the dangers of two extremes:

Too little rigor, which raises the risk of technical issues, schedule slips, or cost overruns.

Too much rigor, which introduces unnecessary process overhead, reduces flexibility, and inflates cost.

PLACE FIGURE HERE

Figure 1: Tailoring requires balance between risk and process. INCOSE SEH original figure created by Krueger adapted from Salter (2003). Used with permission.

Project Context Factors

According to PMI Scaling Factors, several factors shape how tailoring decisions are made:

1. Team size
2. Geographic distribution
3. Organizational distribution
4. Skill availability
5. Compliance
6. Domain complexity
7. Solution complexity

Other factors that can be considered are:

1. Stakeholder landscape
2. Budget and schedule
3. Risk tolerance
4. Criticality of the project (safety, mission, etc.)

In all cases, tailoring seeks to ensure that processes are fit for purpose, supporting the project's goals while managing risk at an acceptable level.

Concurrency, Iteration, and Recursion

As described in article Process Concurrency, Iteration, and Recursion, life cycle processes are applied in ways that are concurrent, iterative, and recursive. For instance, requirements definition, architecture definition, and verification may occur simultaneously and in multiple cycles as knowledge evolves. The processes are recursively applied throughout the system structure. This recursive application ensures coherence across project activities and maintains traceability of decisions.

Good Practices

Project success depends on disciplined tailoring, transparent agreements, and effective use of tools and methods. Several practices have proven especially valuable.

Fact-Based Tailoring

Tailoring decisions should not rely on intuition alone. Instead, they should be based on facts, evidence, and risk assessment. Independent review or approval can help avoid bias. Using structured Decision Management processes to document alternatives, trade-offs, and rationales creates a traceable record that supports accountability.

Documenting and Validating Assumptions and Rationale

Every tailoring choice should be explicitly justified and recorded. For example, if a project reduces the number of formal design reviews, it should document why (e.g., high team cohesion, proven system element maturity). This documentation ensures that later stakeholders (especially during audits or project transitions) understand the reasoning behind earlier decisions.

Collaboration with Stakeholders

Projects rarely operate in isolation. Tailoring works best when all relevant stakeholders (e.g., acquirers, suppliers, and regulators) are involved in defining and approving the tailoring approach. Collaborative agreements build trust and reduce disputes later in the life cycle. As trust grows or project risks or complexity reduces[GR10], projects can simplify some process activities; conversely, when risks rise, additional rigor can be introduced.

Integration with Project Planning

Tailoring should be conducted in parallel with project planning. Decisions on scope, schedule, budget, milestones, and resource allocation are tightly linked to the processes chosen or how they are applied. For example, aggressive schedules can require automated testing tools to accelerate verification, while budget constraints may limit the number of prototypes built. Alignment ensures that plans are realistic and processes are executable.

Dynamic Assessment and Control

Tailoring is not static. Throughout the project, effectiveness and efficiency of tailoring should be continuously assessed and, if necessary, controlled (see article Project Assessment and Control) in response to changing risks, stakeholder feedback, or external conditions. For example, if a project experiences unanticipated technical problems, additional design reviews can be added. Conversely, if risks decrease, some formalities can be reduced to save time.

Methods and Tools at Project Level

Though ISO/IEC/IEEE 15288 does not prescribe specific tools, projects benefit from structured selection and disciplined use of methods and tools. Key considerations include:

- Purpose-driven selection: Tools should directly support tailored processes; process comes before tool.
- Connectivity: Tools should integrate with other tools in use, ensuring smooth data flow across project functions.
- Training requirements: Staff need adequate preparation to use tools effectively.
- Engineer judgment: Tools are aids, not substitutes for critical thinking.
- Data availability: Tool selection should consider what data is needed and whether that data can easily be obtained and verified.

Practical Applications of Life Cycle Processes

According to ISO/IEC/IEEE 24748-2, ISO/IEC/IEEE 15288 processes can be applied in at least four practical ways:

- to establish agreements with organizational entities external and internal to the project to acquire or supply a product or service (agreement processes);
- to establish the organization's capability to acquire and supply products or services through the initiation, support and control of projects (organizational project-enabling processes);
- to establish and evolve project plans, to execute the project plans, to assess actual achievement and progress against the plans and to control execution of the project through to fulfilment (technical management processes);
- to contribute to the satisfaction of technical objectives for one or more life cycle stages (technical processes).

By focusing on these purposes, projects ensure that process tailoring delivers tangible value rather than administrative burden.

Challenges

Even with clear principles and good practices, projects face substantial challenges when tailoring and applying life cycle processes.

Common Tailoring Traps

Projects often fall into predictable traps, including:

- Reusing baselines blindly: Borrowing a tailored process from another project without revisiting assumptions.
- Over-engineering: Applying all processes “just to be safe,” leading to unnecessary costs and delays.
- Assuming uniformity: Believing that the same set of risks or controls applies across all projects.
- Failing to engage stakeholders: Excluding key voices, which leads to lack of buy-in and increased risk of conflict.

Disciplined decision-making and continuous review can facilitate avoiding these traps.

Complexity of Multi-Stakeholder Projects

Projects often involve multiple stakeholders with different priorities and policies. Achieving alignment requires:

- Establishing consistent processes across parties.
- Negotiating agreements that balance competing needs.
- Using ISO/IEC/IEEE 15288 as a shared reference framework.
- Still, differences in culture, terminology, and expectations remain significant hurdles.

Tool and Method Pitfalls

On many cases, tools pose challenges:

- Lack of integration between tools creates silos and rework.
- Security and data management issues complicate tool lifecycle.
- High training requirements can strain limited project resources.
- Over-reliance on tools risks diminishing engineers’ independent judgment.

Data availability and verification

- “Garbage in, Garbage out”.

Maintaining Stakeholder Commitment

Tailoring of processes benefits from strong stakeholder commitment. If stakeholders view processes as excessive overhead, they can resist compliance. Conversely, if they perceive insufficient rigor, they may lose confidence in the project’s ability to deliver. Active communication and visible leadership support are essential to maintain alignment.

Conclusion

At the project level, tailoring systems engineering life cycle processes is both a necessity and an opportunity. Standards such as ISO/IEC/IEEE 15288 provide a comprehensive framework, but projects should adapt these processes to their specific context, risks, and stakeholder environment.

Good practices include fact-based tailoring, documentation of rationale, stakeholder collaboration, integration with project planning, dynamic monitoring, and thoughtful tool use. By applying processes in ways that are practical and purpose-driven, projects can increase efficiency, reduce risk, and maintain stakeholder confidence.

Nevertheless, challenges remain. Projects should avoid common tailoring traps, balance rigor with agility, manage multi-stakeholder complexity, and ensure that tools support rather than dominate decision-making. Above all, continuous adaptation and transparent communication can help ensure that processes remain aligned with project needs.

In the end, project-level tailoring is not about minimizing process effort or maximizing bureaucracy. It is about applying the right processes, at the right level of rigor, at the right time to deliver systems that meet stakeholder needs effectively and efficiently.

References

Works Cited

ISO/IEC/IEEE. 2023. *Systems and software engineering — System life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023.

ISO/IEC/IEEE. 2024. *Systems and software engineering — Life cycle management — Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-2:2024.

Primary References

None

Additional References

None

Part 3c: Processes

Knowledge Area: Technical Management Processes

Technical Management Processes

Contents of this Knowledge Area

- Project Planning (Ray Madachy, Garry Roedler, and Brian Wells)
- Project Assessment and Control (Ray Madachy, Andy Pickard, and Garry Roedler) (Richard Turner)
- Decision Management (Matt Chilli) (Garry Roedler, Greg Parnell, and Scott Jackson)
- Requirements Management (Tami Katz) (Lou Wheatcraft and Mike Ryan)
- Risk Management (Ed Conrow, Ray Madachy, and Garry Roedler) (Victor Bertolazzo, Leighton Johnson, Bob Parro, Jack Stein, and Richard Turner)
- Configuration Management (John Metcalf, Philip Hallenbeck, and Sandrine Gonthier) (Garry Roedler)
- Configuration Baselines (John Metcalf, Philip Hallenbeck, and Sandrine Gonthier) (Garry Roedler)
- Configuration Management Implementation (John Metcalf, Philip Hallenbeck, and Sandrine Gonthier) (Garry Roedler)
- Information Management (John Metcalf, Philip Hallenbeck, and Sandrine Gonthier) (Garry Roedler and Andy Pickard)
- Quality Management (Quong Wang, Massood Towhidnejad, and David Olwell) (Dick Fairley and Garry Roedler)
- Measurement (Garry Roedler) (Salvatore R. Bruno, Thomas McDermott, and David Endler)
- Lead Authors:
- Ray Madachy and Garry Roedler

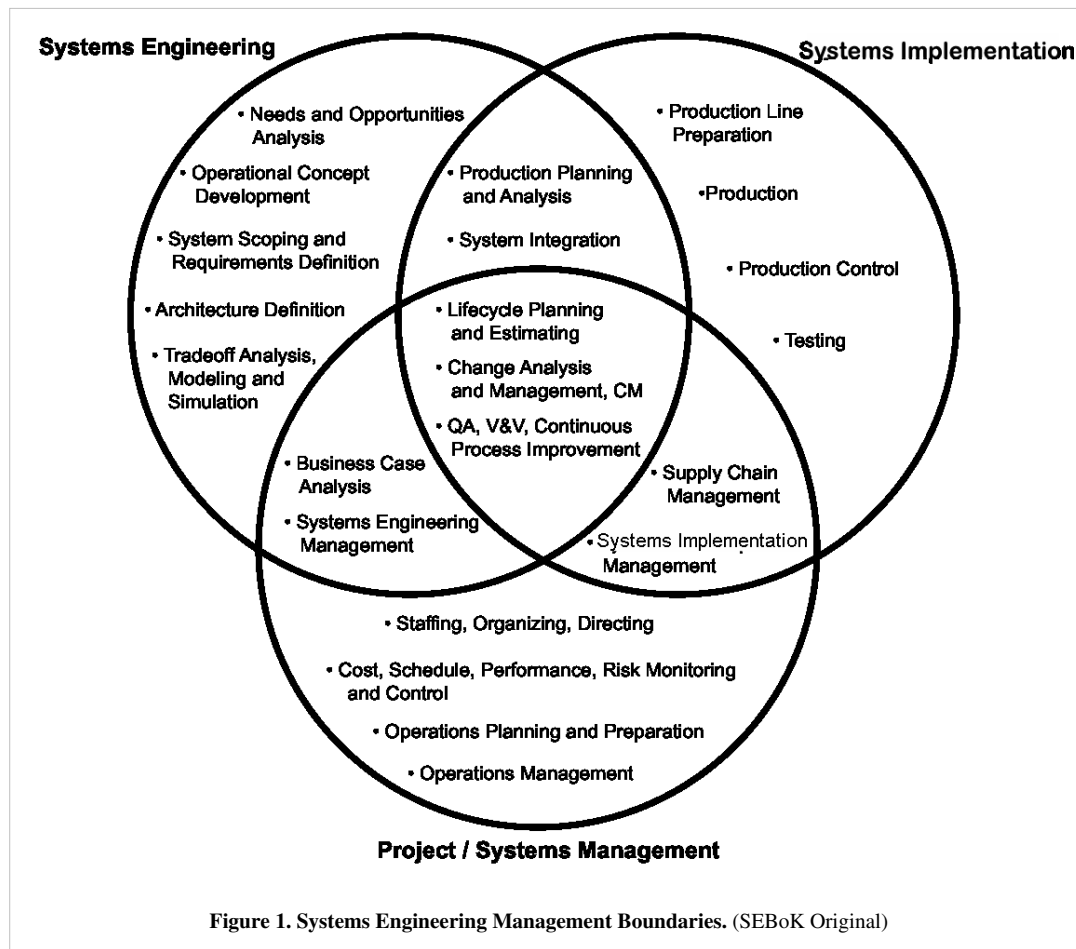
This knowledge area is about managing the resources and assets allocated to perform systems engineering, often in the context of a project or a service, but sometimes in the context of a less well-defined activity. Systems engineering management is distinguished from general project management by its focus on the technical or engineering aspects of a project. SEM also encompasses exploratory research and development (R&D) activities at the enterprise level in commercial or government operations.

See the article Matrix of Implementation Examples for a mapping of case studies and vignettes included in Part 7 to topics covered in Part 3.

Discussion

Implementing systems engineering (SE) requires the coordination of technical and managerial endeavors. Success with the technical is not possible in the absence of the managerial. Management provides the planning, organizational structure, collaborative environment, and program controls to ensure that stakeholder needs are met.

The Venn diagram below provides some context for thinking about SEM. It shows that some functions are managed within the SE function, while others are managed in collaboration with the management of systems implementation and with overall project and systems management.



There is no one-size-fits-all way to define the details of where SEM functions are performed. An in-company SE organization does not run its own accounting system, but relies on the corporate management organization for this aspect of SEM. A company performing only SE *does* include the accounting functions as part of SEM. In all cases, the managers of the SE function must be actively involved in the management of all the activities within the SE system boundary, including working out what collaborative arrangements best fit their situation. They must also remain aware of management events in their environment outside the system boundary that may affect their ability to perform. Part 6 of the SEBoK includes relevant knowledge areas for collaborative management, including Systems Engineering and Software Engineering, Systems Engineering and Project Management, Systems Engineering and Industrial Engineering, Systems Engineering and Procurement/Acquisition, and Systems Engineering and Quality Attributes.

References

Works Cited

None.

Primary References

Blanchard, B.S. 2004. *Systems Engineering Management*, 3rd ed. New York, NY, USA: John Wiley & Sons Inc.

Sage, A.P. and W. Rouse. 2009. *Handbook of Systems Engineering and Management*, 2nd Ed. Hoboken, NJ, USA: John Wiley and Sons.

Rebentisch, E. 2017. *Integrating Program Management and Systems Engineering*, 1st Ed. Hoboken, NJ, USA: John Wiley and Sons.

Additional References

None.

Project Planning

- Lead Authors:
- Ray Madachy, Garry Roedler, and Brian Wells

Planning is an important aspect of technical management processes (SEM). Systems engineering (SE) planning is performed concurrently and collaboratively with project planning. It involves developing and integrating technical plans to achieve the technical project objectives within the resource constraints and risk thresholds. The planning involves the success-critical stakeholders to ensure that necessary tasks are defined with the right timing in the life cycle in order to manage acceptable risks levels, meet schedules, and avoid costly omissions.

SE Planning Process Overview

SE planning provides the following elements:

- Definition of the project from a technical perspective.
 - Definition or tailoring of engineering processes, practices, methods, and supporting enabling environments to be used to develop products or services, as well as plans for transition and implementation of the products or services, as required by agreements.
 - Definition of the technical organizational, personnel, and team functions and responsibilities, as well as all disciplines required during the project life cycle.
 - Definition of the appropriate life cycle model or approach for the products or services.
 - Definition and timing of technical reviews, product or service assessments, and control mechanisms across the life cycle, including the success criteria such as cost, schedule, and technical performance at identified project milestones.
 - Estimation of technical cost and schedule based on the effort needed to meet the requirements; this estimation becomes input to project cost and schedule planning.
 - Determination of critical technologies, as well as the associated risks and actions needed to manage and transition these technologies.
 - Identification of linkages to other project management efforts.
 - Documentation of and commitment to the technical planning.
-

Scope

SE planning begins with analyzing the scope of technical work to be performed and gaining an understanding the constraints, risks, and objectives that define and bound the solution space for the product or service. The planning includes estimating the size of the work products, establishing a schedule (or integrating the technical tasks into the project schedule), identification of risks, and negotiating commitments. Iteration of these planning tasks may be necessary to establish a balanced plan with respect to cost, schedule, technical performance, and quality. The planning continues to evolve with each successive life cycle phase of the project (NASA 2007, 1-360; SEI 1995, 12).

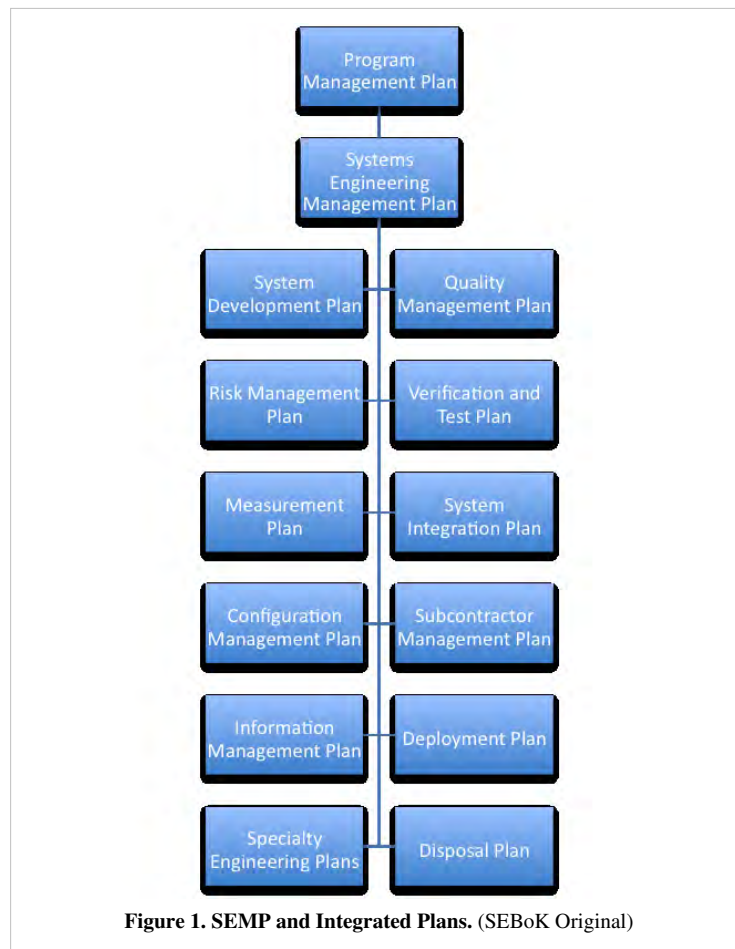
SE planning addresses all programmatic and technical elements of the project to ensure a comprehensive and integrated plan for all of the project's technical aspects and should account for the full scope of technical activities, including system development and definition, risk management, quality management, configuration management, measurement, information management, production, verification and testing, integration, validation, and deployment. SE planning integrates all SE functions to ensure that plans, requirements, operational concepts, and architectures are consistent and feasible.

The scope of planning can vary from planning a specific task to developing a major technical plan. The integrated planning effort will determine what level of planning and accompanying documentation is appropriate for the project.

Integration

The integration of each plan with other higher-level, peer, or subordinate plans is an essential part of SE planning. For the technical effort, the systems engineering management plan (SEMP), also frequently referred to as the systems engineering plan (SEP), is the highest-level technical plan. It is subordinate to the project plan and often has a number of subordinate technical plans providing detail on specific technical focus areas (INCOSE 2011, sec. 5.1.2.2; NASA 2007, appendix J).

In U.S. defense work, the terms SEP and SEMP are not interchangeable. The SEP is a high-level plan that is made before the system acquisition and development begins. It is written by the government customer. The SEMP is the specific development plan written by the developer (or contractor). In this context, intent, and content of these documents are quite different. For example, a SEP will have an acquisition plan that would not be included in a SEMP. Figure 1 below shows the SEMP and integrated plans.



Task planning identifies the specific work products, deliverables, and success criteria for systems engineering efforts in support of integrated planning and project objectives. The success criteria are defined in terms of cost, schedule, and technical performance at identified project milestones. Detailed task planning identifies specific resource requirements (e.g., skills, equipment, facilities, and funding) as a function of time and project milestones.

SE planning is accomplished by both the acquirer and supplier and the activities for SE planning are performed in the context of the respective enterprise. The activities establish and identify relevant policies and procedures for managing and executing the project management and technical effort, identifying the management and technical tasks, their interdependencies, risks, and opportunities, and providing estimates of needed resources/budgets. Plans are updated and refined throughout the development process based on status updates and evolving project requirements (SEI 2007).

Linkages to Other Systems Engineering Management Topics

The project planning process is closely coupled with the measurement, assessment and control, decision management, and risk management processes.

The measurement process provides inputs for estimation models. Estimates and other products from planning are used in decision management. SE assessment and control processes use planning results for setting milestones and assessing progress. Risk management uses the planning cost models, schedule estimates, and uncertainty distributions to support quantitative risk analysis (as desired).

Additionally, planning needs to use the outputs from assessment and control as well as risk management to ensure corrective actions have been accounted for in planning future activities. The planning may need to be updated based on results from technical reviews (from assessment and control) addressing issues pertaining to: measurement, problems that were identified during the performance of risk management activities, or decisions made as a result of

the decision management activities (INCOSE 2010, sec. 6.1).

Practical Considerations

Pitfalls

Some of the key pitfalls encountered in planning and performing SE planning are listed in Table 1.

Table 1. Major Pitfalls with Planning. (SEBoK Original)

Name	Description
Incomplete and Rushed Planning	Inadequate SE planning causes significant adverse impacts on all other engineering activities. Although one may be tempted to save time by rushing the planning, inadequate planning can create additional costs and interfere with the schedule due to planning omissions, lack of detail, lack of integration of efforts, infeasible cost and schedules, etc.
Inexperienced Staff	Lack of highly experienced engineering staff members, especially in similar projects, will likely result in inadequate planning. Less experienced engineers are often assigned significant roles in the SE planning; however, they may not have the appropriate judgment to lay out realistic and achievable plans. It is essential to assign the SE planning tasks to those with a good amount of relevant experience.

Good Practices

Some good practices gathered from the references are in Table 2.

Table 2. Proven Practices with Planning. (SEBoK Original)

Name	Description
Use Multiple Disciplines	Get technical resources from all disciplines involved in the planning process.
Early Conflict Resolution	Resolve schedule and resource conflicts early.
Task Independence	Tasks should be as independent as possible.
Define Interdependencies	Define task interdependencies, using dependency networks or other approaches.
Risk Management	Integrate risk management with the SE planning to identify areas that require special attention and/or trades.
Management Reserve	The amount of management reserve should be based on the risk associated with the plan.
Use Historical Data	Use historical data for estimates and adjust for differences in the project.
Consider Lead Times	Identify lead times and ensure that you account for them in the planning (e.g., the development of analytical tools).
Update Plans	Prepare to update plans as additional information becomes available or changes are needed.
Use IPDTs	An integrated product development team (IPDT) (or integrated product team (IPT)) is often useful to ensure adequate communication across the necessary disciplines, timely integration of all design considerations, as well as integration, testing, and consideration of the full range of risks that need to be addressed. Although there are some issues that need to be managed with them, IPDTs tend to break down the communication and knowledge stovepipes that often exist.

Additional good practices can be found in the *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, *NASA Systems Engineering Handbook*, the *INCOSE Systems Engineering Handbook*, and *Systems and Software Engineering - Life Cycle Processes - Project Management* (Caltrans and USDOT 2005, 278; NASA December 2007, 1-360, sec. 6.1; INCOSE 2011, sec. 5.1; ISO/IEC/IEEE 2009, Clause 6.1).

References

Works Cited

Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research & Innovation/U.S. Department of Transportation (USDOT), SEG for ITS 1.1.

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense, February 19.

INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - Life Cycle Processes - Project Management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 16326:2009(E).

NASA. 2007. *NASA Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

SEI. 1995. *A systems engineering capability maturity model*. Version 1.1. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-95-MM-003.

Primary References

Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research & Innovation/U.S. Department of Transportation (USDOT), SEG for ITS 1.1.

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense.

INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - Life Cycle Processes - Project Management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 16326:2009(E).

NASA. 2007. *NASA Systems Engineering Handbook*. Washington, D.C., USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

SEI. 1995. *A Systems Engineering Capability Maturity Model*, version 1.1. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-95-MM-003.

SEI. 2007. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.2, measurement and analysis process area. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Additional References

- Boehm, B., C. Abts, A.W. Brown, S. Chulani, B.K. Clark, E. Horowitz, R. Madachy, D.J. Reifer, B. Steece. 2000. *Software Cost Estimation with COCOMO II*. Englewood Cliffs, NJ, USA: Prentice Hall
- DeMarco, T. and T. Lister. 2003. *Waltzing with Bears; Managing Risks on Software Projects*. New York, NY, USA: Dorset House.
- ISO/IEC/IEEE. 2009. *Systems and Software Engineering - Life Cycle Processes - Project Management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 16326:2009(E).
- Valerdi, R. 2008. *The Constructive Systems Engineering Cost Model (COSYSMO): Quantifying the Costs of Systems Engineering Effort in Complex Systems*. Saarbrücken, Germany: VDM Verlag Dr. Muller

Project Assessment and Control

- Lead Authors:
- Ray Madachy, Andy Pickard, and Garry Roedler
- Contributing Author:
- Richard Turner

The purpose of systems engineering assessment and control (SEAC) is to provide adequate visibility into the project's actual technical progress and risks with respect to the technical plans (i.e., systems engineering management plan (SEMP) or systems engineering plan (SEP) and subordinate plans). The visibility allows the project team to take timely preventive action when disruptive trends are recognized or corrective action when performance deviates beyond established thresholds or expected values. SEAC includes preparing for and conducting reviews and audits to monitor performance. The results of the reviews and measurement analyses are used to identify and record findings/discrepancies and may lead to causal analysis and corrective/preventive action plans. Action plans are implemented, tracked, and monitored to closure. (NASA 2007, Section 6.7; SEG-ITS, 2009, Section 3.9.3, 3.9.10; INCOSE, 2010, Clause 6.2; SEI, 2007)

Systems Engineering Assessment and Control Process Overview

The SEAC process involves determining and initiating the appropriate handling strategies and actions for findings and/or discrepancies that are uncovered in the enterprise, infrastructure, or life cycle activities associated with the project. Analysis of the causes of the findings/discrepancies aids in the determination of appropriate handling strategies. Implementation of approved preventive, corrective, or improvement actions ensures satisfactory completion of the project within planned technical, schedule, and cost objectives. Potential action plans for findings and/or discrepancies are reviewed in the context of the overall set of actions and priorities in order to optimize the benefits to the project and/or organization. Interrelated items are analyzed together to obtain a consistent and cost-effective resolution.

The SEAC process includes the following steps:

- monitor and review technical performance and resource use against plans
 - monitor technical risk, escalate significant risks to the project risk register and seek project funding to execute risk mitigation plans
 - hold technical reviews and report outcomes at the project reviews
 - analyze issues and determine appropriate actions
 - manage actions to closure
-

- hold a post-delivery assessment (also known as a post-project review) to capture knowledge associated with the project (this may be a separate technical assessment or it may be conducted as part of the project assessment and control process).

The following activities are normally conducted as part of a project assessment and control process:

- authorization, release and closure of work
- monitor project performance and resource usage against plan
- monitor project risk and authorize expenditure of project funds to execute risk mitigation plans
- hold project reviews
- analyze issues and determine appropriate actions
- manage actions to closure
- hold a post-delivery assessment (also known as a post-project review) to capture knowledge associated with the project

Examples of major technical reviews used in SEAC are shown in Table 1 from DAU (2010).

Table 1. Major Technical Review Examples (DAU 2012). Released by Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

Name	Description
Alternative Systems Review	A multi-disciplined review to ensure the resulting set of requirements agrees with the customers' needs and expectations.
Critical Design Review (CDR)	A multi-disciplined review establishing the initial product baseline to ensure that the system under review has a reasonable expectation of satisfying the requirements of the capability development document within the currently allocated budget and schedule.
Functional Configuration Audit	Formal examination of the as-tested characteristics of a configuration item (hardware and software) with the objective of verifying that actual performance complies with design and interface requirements in the functional baseline.
In-Service Review	A multi-disciplined product and process assessment that is performed to ensure that the system under review is operationally employed with well-understood and managed risk.
Initial Technical Review	A multi-disciplined review that supports a program's initial program objective memorandum submission.
Integrated Baseline Review	A joint assessment conducted by the government program manager and the contractor to establish the performance measurement baseline.
Operational Test Readiness Review	A multi-disciplined product and process assessment to ensure that the system can proceed into initial operational test and evaluation with a high probability of success, and also that the system is effective and suitable for service introduction.
Production Readiness Review (PRR)	The examination of a program to determine if the design is ready for production and if the prime contractor and major subcontractors have accomplished adequate production planning without incurring unacceptable risks that will breach thresholds of schedule, performance, cost, or other established criteria.
Physical Configuration Audit	An examination of the actual configuration of an item being produced around the time of the full-rate production decision.
Preliminary Design Review (PDR)	A technical assessment establishing the physically allocated baseline to ensure that the system under review has a reasonable expectation of being judged operationally effective and suitable.
System Functional Review (SFR)	A multi-disciplined review to ensure that the system's functional baseline is established and has a reasonable expectation of satisfying the requirements of the initial capabilities document or draft capability development document within the currently allocated budget and schedule.
System Requirements Review (SRR)	A multi-disciplined review to ensure that the system under review can proceed into initial systems development and that all system requirements and performance requirements derived from the initial capabilities document or draft capability development document are defined and testable, as well as being consistent with cost, schedule, risk, technology readiness, and other system constraints.

System Verification Review (SVR)	A multi-disciplined product and process assessment to ensure the system under review can proceed into low-rate initial production and full-rate production within cost (program budget), schedule (program schedule), risk, and other system constraints.
Technology Readiness Assessment	A systematic, metrics-based process that assesses the maturity of critical technology elements, such as sustainment drivers.
Test Readiness Review (TRR)	A multi-disciplined review designed to ensure that the subsystem or system under review is ready to proceed into formal testing.

Linkages to Other Systems Engineering Management Topics

The SE assessment and control process is closely coupled with the measurement, planning, decision management, and risk management processes. The measurement process provides indicators for comparing actuals to plans. Planning provides estimates and milestones that constitute plans for monitoring as well as the project plan, which uses measurements to monitor progress. Decision management uses the results of project monitoring as decision criteria for making control decisions.

Practical Considerations

Key pitfalls and good practices related to SEAC are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing SE assessment and control are shown in Table 2.

Table 2. Major Pitfalls with Assessment and Control. (SEBoK Original)

Name	Description
No Measurement	Since the assessment and control activities are highly dependent on insightful measurement information, it is usually ineffective to proceed independently from the measurement efforts - what you get is what you measure.
"Something in Time" Culture	Some things are easier to measure than others - for instance, delivery to cost and schedule. Don't focus on these and neglect harder things to measure like quality of the system. Avoid a "something in time" culture where meeting the schedule takes priority over everything else, but what is delivered is not fit for purpose, resulting in the need to rework the project.
No Teeth	Make sure that the technical review gates have "teeth". Sometimes the project manager is given authority (or can appeal to someone with authority) to over-ride a gate decision and allow work to proceed, even when the gate has exposed significant issues with the technical quality of the system or associated work products. This is a major risk if the organization is strongly schedule-driven; it can't afford the time to do it right, but somehow it finds the time to do it again (rework).
Too Early Baselining	Don't baseline requirements or designs too early. Often there is strong pressure to baseline system requirements and designs before they are fully understood or agreed, in order to start subsystem or component development. This just guarantees high levels of rework.

Good Practices

Some good practices gathered from the references are shown in Table 3.

Table 3. Proven Practices with Assessment and Control. (SEBoK Original)

Name	Description
Independence	Provide independent (from customer) assessment and recommendations on resources, schedule, technical status, and risk based on experience and trend analysis.
Peer Reviews	Use peer reviews to ensure the quality of a product's work before they are submitted for gate review.
Accept Uncertainty	Communicate uncertainties in requirements or designs and accept that uncertainty is a normal part of developing a system.
Risk Mitigation Plans	Do not penalize a project at gate review if they admit uncertainty in requirements - ask for their risk mitigation plan to manage the uncertainty.
Just In-Time Baselineing	Baseline requirements and designs only when you need to - when other work is committed based on the stability of the requirement or design. If work must start and the requirement or design is still uncertain, consider how you can build robustness into the system to handle the uncertainty with minimum rework.
Communication	Document and communicate status findings and recommendations to stakeholders.
Full Visibility	Ensure that action items and action-item status, as well as other key status items, are visible to all project participants.
Leverage Previous Root Cause Analysis	When performing root cause analysis, take into account the root cause and resolution data documented in previous related findings/discrepancies.
Concurrent Management	Plan and perform assessment and control concurrently with the activities for Measurement and Risk Management.
Lessons Learned and Post-Mortems	Hold post-delivery assessments or post-project reviews to capture knowledge associated with the project – e.g., to augment and improve estimation models, lessons learned databases, gate review checklists, etc.

Additional good practices can be found in INCOSE (2010, Clause 6.2), SEG-ITS (2009, Sections 3.9.3 and 3.9.10), INCOSE (2010, Section 5.2.1.5), and NASA (2007, Section 6.7).

References

Works Cited

- Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research & Innovation/U.S. Department of Transportation (USDOT), SEG for ITS 1.1.
- DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- NASA. 2007. *Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), December 2007. NASA/SP-2007-6105.
- SEI. 2007. "Measurement and Analysis Process Area," in *Capability Maturity Model Integrated (CMMI) for Development*, version 1.2. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Primary References

Caltrans and USDOT. 2005. *[[Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)],* version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research & Innovation/U.S. Department of Transportation (USDOT), SEG for ITS 1.1.

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

NASA. 2007. *Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), December 2007. NASA/SP-2007-6105.

SEI. 2007. "Measurement and Analysis Process Area," in *Capability Maturity Model Integrated (CMMI) for Development*, version 1.2. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Additional References

ISO/IEC/IEEE. 2009. *ISO/IEC/IEEE 16326\Systems and Software Engineering - Life Cycle Processes - Project Management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 16326:2009(E).

Decision Management

- Lead Author:
- Matt Chilli
- Contributing Authors:
- Garry Roedler, Greg Parnell, and Scott Jackson

Many systems engineering decisions are difficult because they include numerous stakeholders, multiple competing objectives, substantial uncertainty, and significant consequences. In these cases, good decision making requires a formal decision management process. The purpose of the decision management process is:

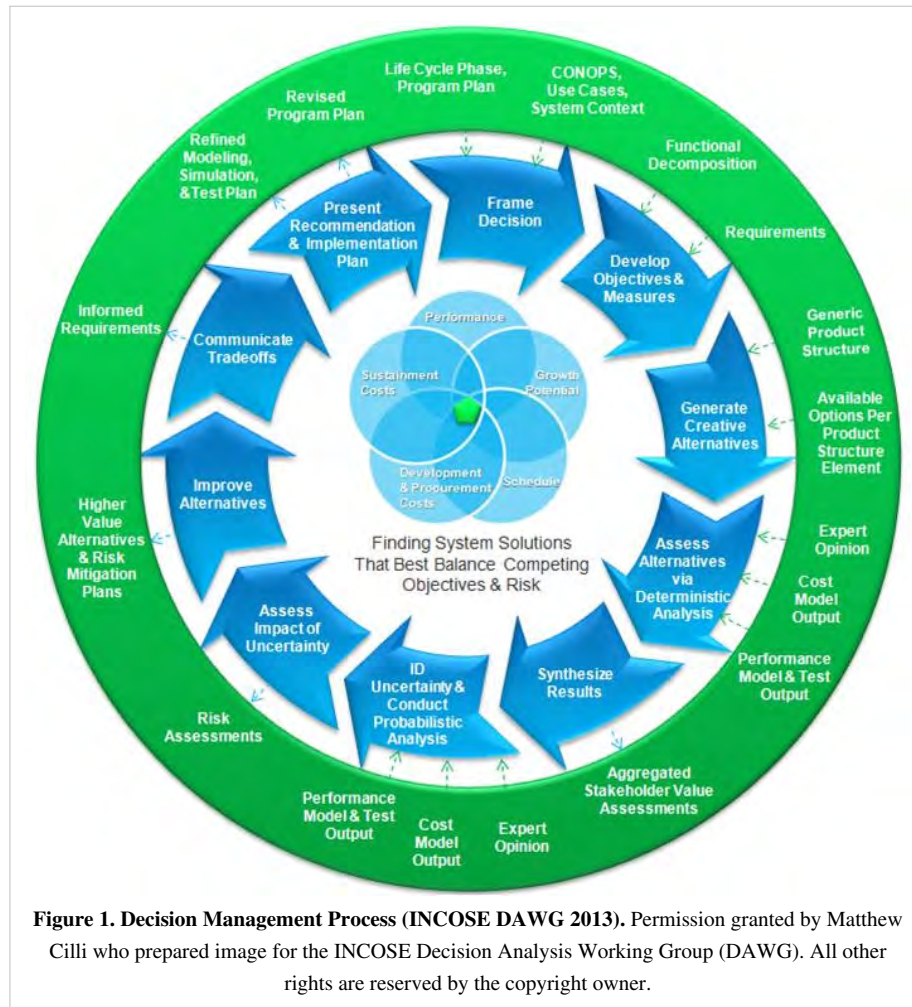
“...to provide a structured, analytical framework for objectively identifying, characterizing and evaluating a set of alternatives for a decision at any point in the life cycle and select the most beneficial course of action.”(ISO/IEC/IEEE 15288)

Decision situations (opportunities) are commonly encountered throughout a system's lifecycle. The decision management method most commonly employed by systems engineers is the trade study. Trade studies aim to define, measure, and assess shareholder and stakeholder value to facilitate the decision maker's search for an alternative that represents the best balance of competing objectives. By providing techniques for decomposing a trade decision into logical segments and then synthesizing the parts into a coherent whole, a decision management process allows the decision maker to work within human cognitive limits without oversimplifying the problem. Furthermore, by decomposing the overall decision problem, experts can provide assessments of alternatives in their area of expertise.

Decision Management Process

The decision analysis process is depicted in Figure 1 below. The decision management process is based on several best practices, including:

- Utilizing sound mathematical technique of decision analysis for trade studies. Parnell (2009) provided a list of decision analysis concepts and techniques.
- Developing one master decision model, followed by its refinement, update, and use, as required for trade studies throughout the system life cycle.
- Using Value-Focused Thinking (Keeney 1992) to create better alternatives.
- Identifying uncertainty and assessing risks for each decision.



The center of the diagram shows the five trade space objectives (listed clockwise): Performance, Growth Potential, Schedule, Development & Procurement Costs, and Sustainment Costs. The ten blue arrows represent the decision management process activities and the white text within the green ring represents SE process elements. Interactions are represented by the small, dotted green or blue arrows. The decision analysis process is an iterative process. A hypothetical UAV decision problem is used to illustrate each of the activities in the following sections.

Framing and Tailoring the Decision

To ensure the decision team fully understands the decision context, the analyst should describe the system baseline, boundaries and interfaces. The decision context includes: the system definition, the life cycle stage, decision milestones, a list of decision makers and stakeholders, and available resources. The best practice is to identify a decision problem statement that defines the decision in terms of the system life cycle.

Developing Objectives and Measures

Defining how an important decision will be made is difficult. As Keeney (2002) puts it:

Most important decisions involve multiple objectives, and usually with multiple-objective decisions, you can't have it all. You will have to accept less achievement in terms of some objectives in order to achieve more on other objectives. But how much less would you accept to achieve how much more?

The first step is to develop objectives and measures using interviews and focus groups with subject matter experts (SMEs) and stakeholders. For systems engineering trade-off analyses, stakeholder value often includes competing objectives of performance, development schedule, unit cost, support costs, and growth potential. For corporate decisions, shareholder value would also be added to this list. For performance, a functional decomposition can help generate a thorough set of potential objectives. Test this initial list of fundamental objectives by checking that each fundamental objective is essential and controllable and that the set of objectives is complete, non-redundant, concise, specific, and understandable (Edwards et al. 2007). Figure 2 provides an example of an objectives hierarchy.

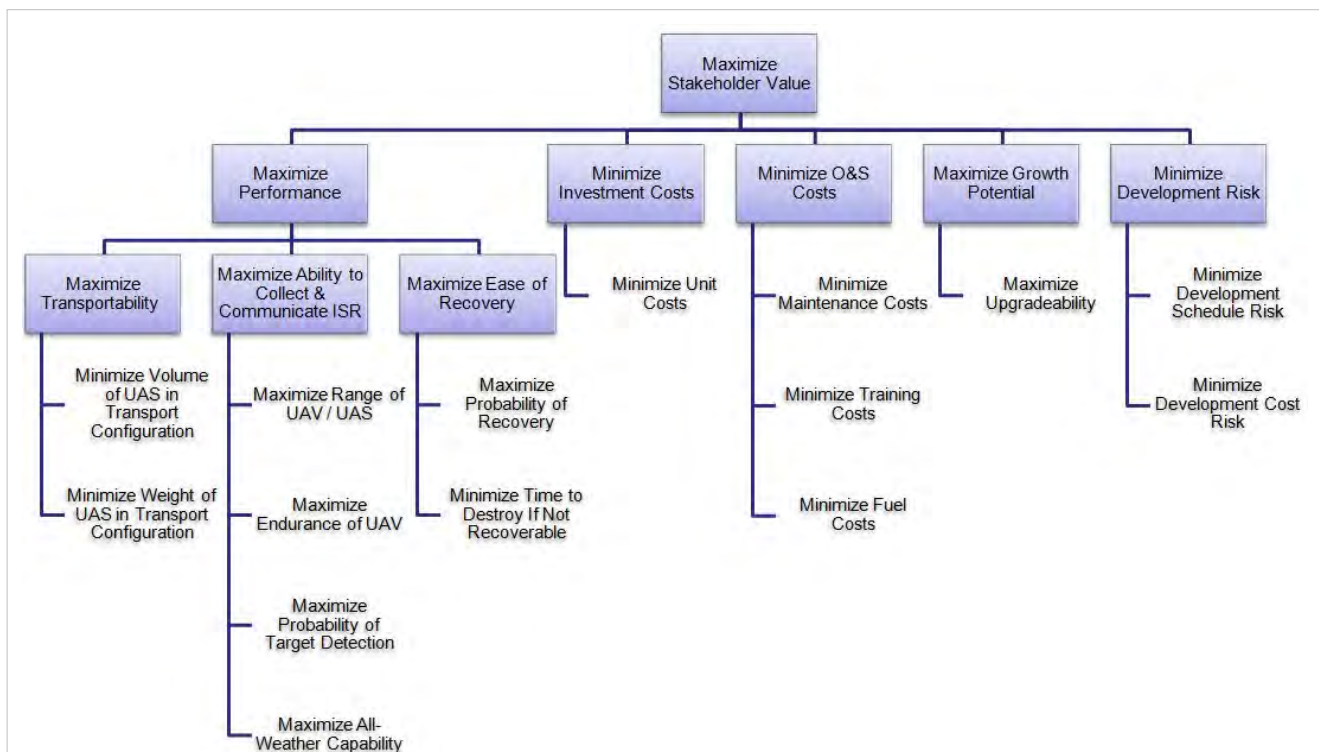


Figure 2. Fundamental Objectives Hierarchy (INCOSE DAWG 2013). Permission granted by Matthew Cilli who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

For each objective, a measure must be defined to assess the value of each alternative for that objective. A measure (attribute, criterion, and metric) must be unambiguous, comprehensive, direct, operational, and understandable (Keeney & Gregory 2005). A defining feature of multi-objective decision analysis is the transformation from measure space to value space. This transformation is performed by a value function which shows returns to scale on the measure range. When creating a value function, the walk-away point on the measure scale (x-axis) must be ascertained and mapped to a 0 value on the value scale (y-axis). A walk-away point is the measure score where

regardless of how well an alternative performs in other measures, the decision maker will walk away from the alternative. He or she does this through working with the user, finding the measure score beyond, at which point an alternative provides no additional value, and labeling it "stretch goal" (ideal) and then mapping it to 100 (or 1 and 10) on the value scale (y-axis). Figure 3 provides the most common value curve shapes. The rationale for the shape of the value functions should be documented for traceability and defensibility (Parnell et al. 2011).

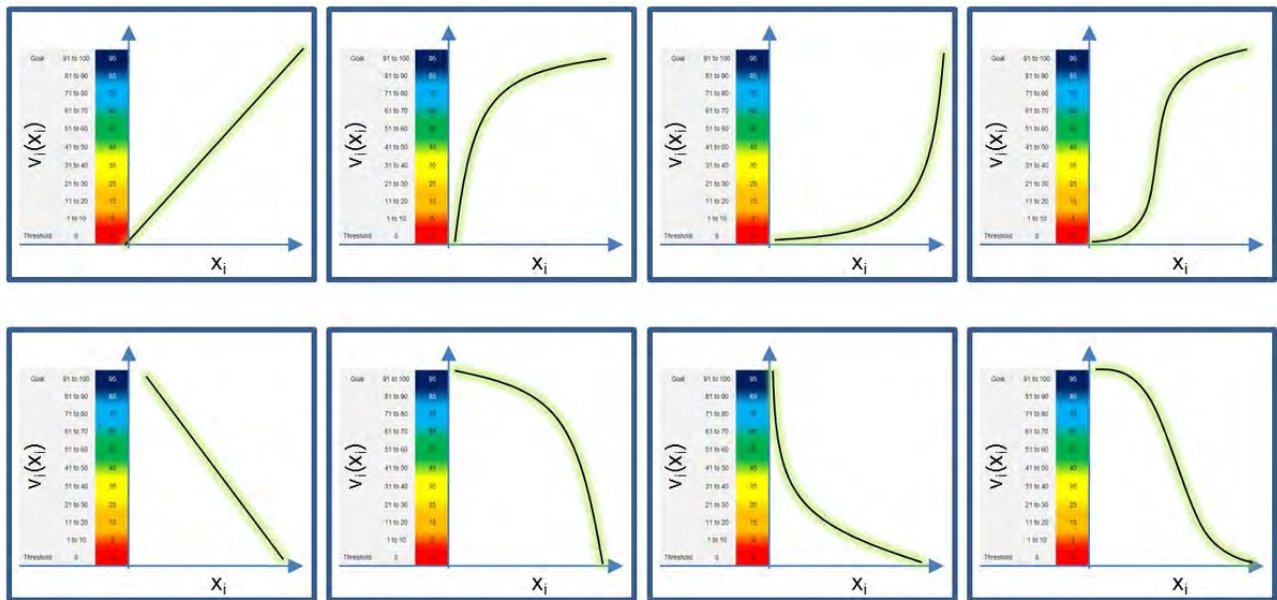


Figure 3. Value Function Examples (INCOSE DAWG 2013). Permission granted by Matthew Cilli who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

The mathematics of multiple objective decision analysis (MODA) requires that the weights depend on importance of the measure and the range of the measure (walk away to stretch goal). A useful tool for determining priority weighting is the swing weight matrix (Parnell et al. 2011). For each measure, consider its importance through determining whether the measure corresponds to a defining, critical, or enabling function and consider the gap between the current capability and the desired capability; finally, put the name of the measure in the appropriate cell of the matrix (Figure 4). The highest priority weighting is placed in the upper-left corner and assigned an unnormalized weight of 100. The unnormalized weights are monotonically decreasing to the right and down the matrix. Swing weights are then assessed by comparing them to the most important value measure or another assessed measure. The swing weights are normalized to sum to one for the additive value model used to calculate value in a subsequent section.

Highest Weight		Level of importance of the value measure								
		Mission Critical	Swt	Mwt	Mission Effectiveness	Swt	Mwt	Mission Enhancing	Swt	Mwt
Capability gap	Large									
		UAV range in nm	100	0.285	All weather capability	50	0.142	UAV P[R]	10	0.028
	Medium									
		UAV endurance in minutes	80	0.228	UAS volume in ft3	40	0.114			
	Small									
		UAV P[D]	50	0.142	UAS weight in pounds	20	0.057	UAV destruct time in seconds	1	0.003

Total swing weight

351

f_i = swing weight

w_i = measure weight

$$w_i = \frac{f_i}{\sum_{i=1}^n f_i}$$

Figure 4. Swing Weight Matrix (INCOSE DAWG 2013). Permission granted by Gregory Parnell who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

Generating Creative Alternatives

To help generate a creative and comprehensive set of alternatives that span the decision space, consider developing an alternative generation table (also called a morphological box) (Buede, 2009; Parnell et al. 2011). It is a best practice to establish a meaningful product structure for the system and to be reported in all decision presentations (Figure 5).

	Cardinal	Buzzard	Crow	Pigeon	Robin	Dove
						
Subsystem	Design Choice	Design Choice	Design Choice	Design Choice	Design Choice	Design Choice
Propulsion System	Electric 300W & Li P	Electric 300W w/ Li Ion	Electric 600W w/ Solar	Elect 600W w Fuel Cell	Piston Engine 2.5 HP	Piston Engine 4.0 HP
Fuel	NA	NA	NA	NA	JP-8	JP-8
Fuel Tank Capacity	NA	NA	NA	NA	5 liter	7 liter
Propeller	18" Rear	20" rear	22" rear	24" Front	26" Front	28" Front
Wing Configuration	5 ft, Conventional	6 ft, Canard	6 ft, Tandem Wing	7 ft, Three Surface	8 ft., Conventional	9 ft., Conventional
Fin Configuration	Twin Boom Conv.	Inverted V	V Tail	Conventional	H Tail	Cruciform
Actuators	Electromagnetic	Hydraulic	MEMS	Hydraulic	Hydraulic	Hydraulic
Fuselage X Section	12" Diameter	14" Diameter	16" Diameter	18" Diameter	20" Diameter	22" Diameter
Airframe Material	Graphite Epoxy	Graphite Epoxy	Aramid-epoxy	Boron-epoxy	Fiberglass-epoxy	Fiberglass-epoxy
Avionics Arch.	Simplex	Simplex	Triplex	Triplex	Triplex	Triplex
Navigation Sensor	MEMS GPS / INS	MEMS GPS / INS	MEMS GPS / INS	MEMS GPS / INS	MEMS GPS / INS	MEMS GPS / INS
External Comms	LOS COMM Link	LOS COMM Link	LOS + SATCOM Link	LOS + SATCOM Link	LOS + SATCOM Link	LOS + SATCOM Link
Internal Comms	MIL-STD-1553B	MIL-STD-1553B	MIL-STD-1553B	MIL-STD-1553B	MIL-STD-1553B	MIL-STD-1553B
Autopilot	Pre-Programmed, Auto	Semi-Autonomous	Remotely Piloted	Pre-Programmed, Auto	Pre-Programmed, Auto	Pre-Programmed, Auto
Launch / Recovery	Hand / Belly	Hand / Belly	Hand / Belly	Hand / Belly	Hand / Belly	Hand / Belly
Acquisition Sensor	Un-cooled IR	Day Video	Day Video, Cooled IR	Day Video, Cooled IR	Day Video	SAR, Acoustic, Day, IR
Sensor Actuation	Pan-tilt	Pan-tilt-roll	Roll-tilt	Pan-tilt	Pan tilt	Pan tilt
Characteristics	Measurement	Measurement	Measurement	Measurement	Measurement	Measurement
Weight	5 lbs	10 lbs	10 lbs	15 lbs	30 lbs	40 lbs
Max Airspeed	60 kph	50 kph	80 kph	70 kph	60 kph	80 kph
Climb Rate	200 m / minute	150 m / minute	250 m / minute	200 m / minute	200 m / minute	250 m / minute

Figure 5. Descriptions of Alternatives (INCOSE DAWG 2013). Permission granted by Matthew Cilli who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

Assessing Alternatives via Deterministic Analysis

With objectives and measures established and alternatives having been defined, the decision team should engage SMEs, equipped with operational data, test data, simulations, models, and expert knowledge. Scores are best captured on scoring sheets for each alternative/measure combination which document the source and rationale. Figure 6 provides a summary of the scores.

Raw Scorecard (expected performance values)			RELOCATE UAV		EMPLOY UAV				RECOVER UAV		GROWTH POT.	UNIT COST	DEVELOPMENT RISK		OPERATION & SUPPORT COST		
			Minimize UAV weight	Minimize UAV volume	Maximize all weather capability	Maximize UAV range	Maximize UAV probability of detection	Maximize UAV endurance	Maximize probability of recovery	Minimize time to destroy if not recoverable	Maximize upgradability	Minimize unit cost	Minimize development schedule risk	Minimize development cost risk	Minimize training cost	Minimize maintenance cost	Minimize fuel cost
ID	Name	Image	(lbs)	ft3	index	km	P[D]	hours	P[R]	sec	index	FY13\$K	index	index	FY13\$	FY13\$	FY13\$
1	Cardinal		5	12	3	10	0.92	0.5	0.6	1	0.3	250	0.9	0.9	300	300	0
2	Buzzard		10	15	1	10	0.9	1	0.7	2	0.6	300	0.8	0.8	300	300	0
3	Crow		10	20	3	70	0.92	1	0.8	2	0.6	350	0.7	0.7	300	300	0
4	Pigeon		15	30	3	80	0.92	1.5	0.9	2	0.6	400	0.6	0.6	300	300	0
5	Robin		30	40	1	90	0.9	2	0.9	2	0.6	500	0.5	0.5	500	500	300
6	Dove		40	50	5	100	0.94	2	0.9	3	0.9	700	0.4	0.4	500	500	500
7	Ideal		5	10	5	100	1	2	0.9	1	1	200	1	1	250	0	0

Figure 6. Alternative Scores (INCOSE DAWG 2013). Permission granted by Richard Swanson who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

Note that in addition to identified alternatives, the score matrix includes a row for the ideal alternative. The ideal is a tool for value-focused thinking, which will be covered later.

Synthesizing Results

Next, one can transform the scores into a value table, by using the value functions developed previously. A color heat map can be useful to visualize value trade-offs between alternatives and identify where alternatives need improvement (Figure 7).

Heat-indexed Value Scorecard			RELOCATE UAV		EMPLOY UAV				RECOVER UAV		GROWTH POT.	UNIT COST	DEVELOPMENT RISK		OPERATION & SUPPORT COST		
			Minimize UAV weight	Minimize UAV volume	Maximize all weather capability	Maximize UAV range	Maximize UAV probability of detection	Maximize UAV endurance	Maximize probability of recovery	Minimize time to destroy if not recoverable	Maximize upgradability	Minimize unit cost	Minimize development schedule risk	Minimize development cost risk	Minimize training cost	Minimize maintenance cost	Minimize fuel cost
ID	Name	Image	0.06	0.11	0.14	0.28	0.14	0.23	0.03	0.01	1	1	0.5	0.5	0.33	0.33	0.33
1	Cardinal		100	97	90	13	59	1	47	100	31	94	83	83	83	85	100
2	Buzzard		86	92	1	13	42	60	77	90	61	88	67	67	83	85	100
3	Crow		86	85	90	83	59	60	90	75	61	82	50	50	83	85	100
4	Pigeon		72	57	90	90	59	80	100	75	61	76	34	34	83	85	100
5	Robin		29	29	1	95	42	100	100	60	61	56	18	18	17	67	51
6	Dove		1	1	100	100	75	100	100	1	91	1	1	1	17	67	18
7	Ideal		100	100	100	100	100	100	100	100	100	100	100	100	100	100	100

Figure 7. Value Scorecard with Heat Map (INCOSE DAWG 2013). Permission granted by Richard Swanson who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

The additive value model uses the following equation to calculate each alternative's value:

$$v(x) = \sum_{i=1}^n w_i v_i(x_i)$$

where

$v(x)$ is the alternative's value

$i = 1$ to n is the number of the measure

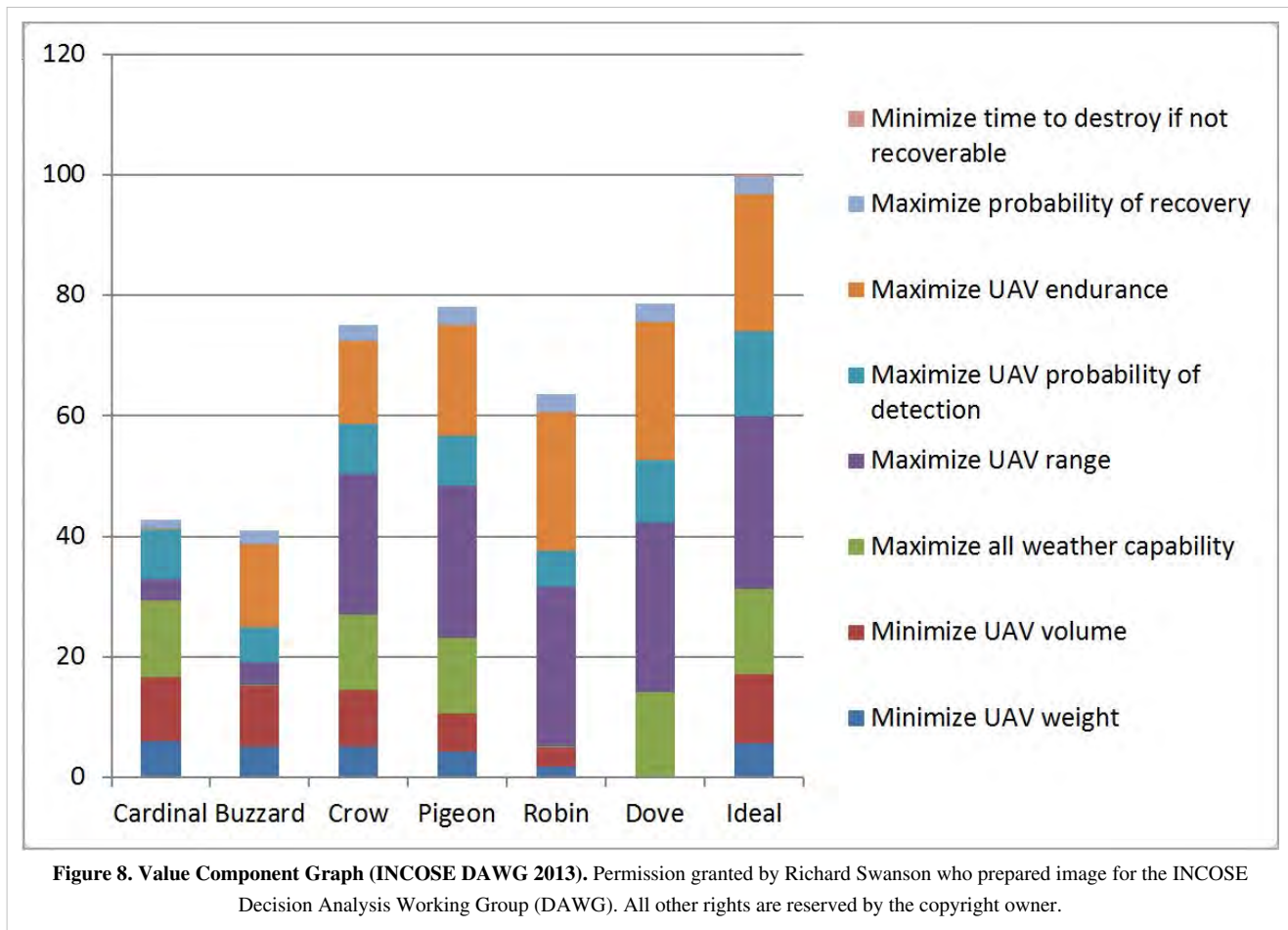
x_i is the alternative's score on the i^{th} measure

$v_i(x_i)$ is the single dimensional value of a score of x_i

w_i is the weight of the i^{th} measure

and $\sum_{i=1}^n w_i = 1$ (all weights sum to one)

The value component chart (Figure 8) shows the total value and the weighted value measure contribution of each alternative (Parnell et al. 2011).



The heart of a decision management process for system engineering trade-off analysis is the ability to assess all dimensions of shareholder and stakeholder value. The stakeholder value scatter plot in Figure 9 shows five dimensions: unit cost, performance, development risk, growth potential, and operation and support costs for all alternatives.

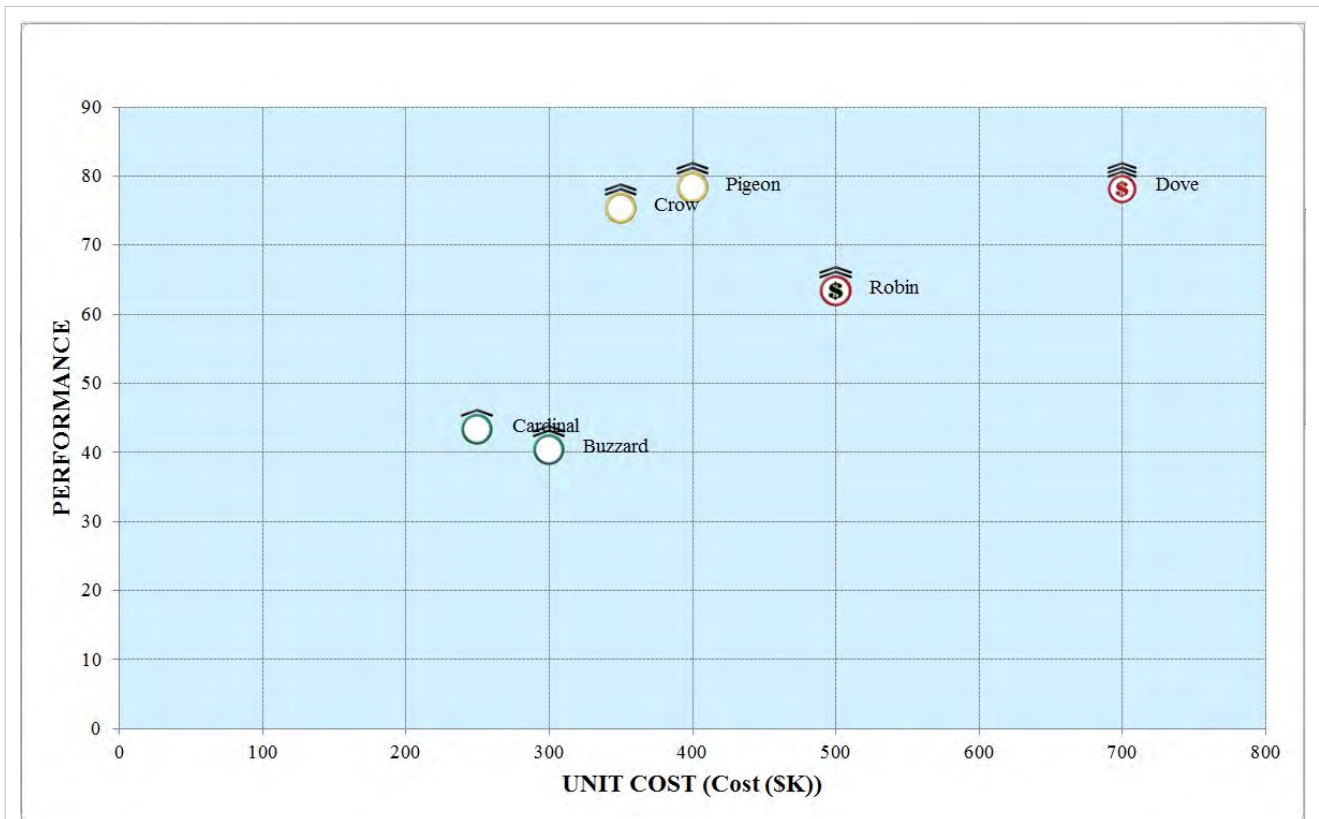


Figure 9. Example of a Stakeholder Value Scatterplot (INCOSE DAWG 2013). Permission granted by Richard Swanson who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

Each system alternative is represented by a scatter plot marker (Figure 9). An alternative's unit cost and performance value are indicated by x and y positions respectively. An alternative's development risk is indicated by the color of the marker (green = low, yellow = medium, red = high), while the growth potential is shown as the number of hats above the circular marker (1 hat = low, 2 hats = moderate, 3 hats = high).

Identifying Uncertainty and Conducting Probabilistic Analysis

As part of the assessment, the SME should discuss the potential uncertainty of the independent variables. The independent variables are the variables that impact one or more scores; the scores that are independent scores. Many times the SME can assess an upper, nominal, and lower bound by assuming low, moderate, and high performance. Using this data, a Monte Carlo Simulation summarizes the impact of the uncertainties and can identify the uncertainties that have the most impact on the decision.

Accessing Impact of Uncertainty - Analyzing Risk and Sensitivity

Decision analysis uses many forms of sensitivity analysis including line diagrams, tornado diagrams, waterfall diagrams and several uncertainty analyses including Monte Carlo Simulation, decision trees, and influence diagrams (Parnell et al. 2013). A line diagram is used to show the sensitivity to the swing weight judgment (Parnell et al. 2011). Figure 10 shows the results of a Monte Carlo Simulation of performance value.

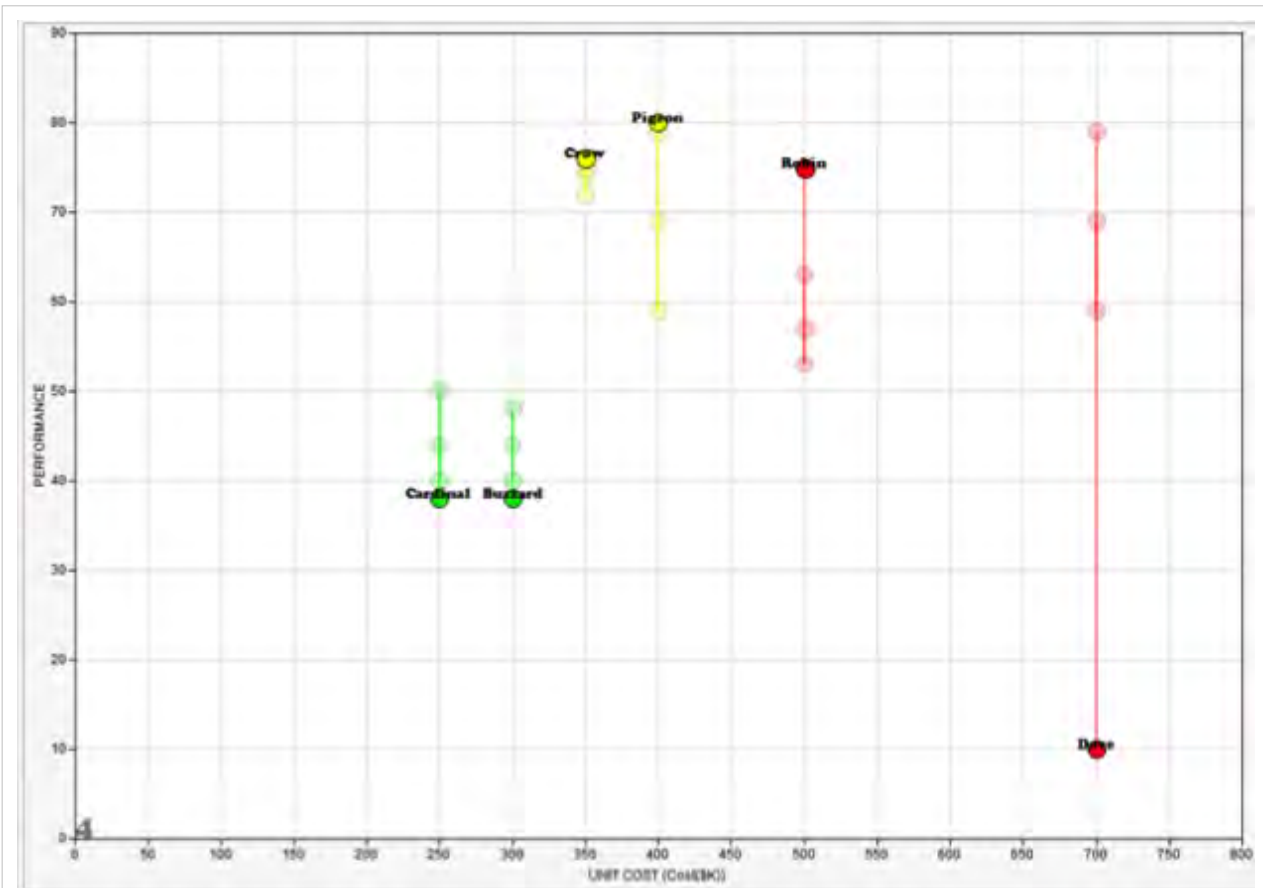


Figure 10. Uncertainty on Performance Value from Monte Carlo Simulation (INCOSE DAWG 2013). Permission granted by Matthew Cilli who prepared image for the INCOSE Decision Analysis Working Group (DAWG). All other rights are reserved by the copyright owner.

Improving Alternatives

Mining the data generated for the alternatives will likely reveal opportunities to modify some design choices to claim untapped value and/or reduce risk. Taking advantage of initial findings to generate new and creative alternatives starts the process of transforming the decision process from "alternative-focused thinking" to "value-focused thinking" (Keeney 1993).

Communicating Trade-offs

This is the point in the process where the decision analysis team identifies key observations about trade-offs and the important uncertainties and risks.

Presenting Recommendations and Implementing Action Plan

It is often helpful to describe the recommendation(s) in the form of a clearly-worded, actionable task-list in order to increase the likelihood of the decision implementation. Reports are important for historical traceability and future decisions. Take the time and effort to create a comprehensive, high-quality report detailing study findings and supporting rationale. Consider static paper reports augmented with dynamic hyper-linked e-reports.

The Cognitive Bias Effect on Decisions

Research by (Kahneman 2011) and (Thaler and Sunstein 2008) has concluded that cognitive bias can seriously distort decisions made by any decision maker. Both Kahneman and Thaler were awarded the Nobel prize for their work. The cause of this distortion is called the cognitive bias. These distorted decisions have contributed to major catastrophes, such as Challenger and Columbia. Other sources attributing major catastrophes are (Murata, Nakamura, and Karwowski 2015) and (Murata 2017).

(Kahneman 2011) and (Thaler and Sunstein 2008) have identified a large number of individual biases, the most well-known of which is the confirmation bias. This bias states that humans have a tendency to interpret new evidence as confirmation of one's existing beliefs or theories. Regarding mitigation of these biases, there is general agreement that self-mitigation by the decision-maker is not feasible for most biases. (Thaler and Sunstein 2008) provide methods to influence the mitigation of most biases. They refer to these influences as “nudges”.

Considering cognitive biases in a systems engineering is discussed by (Jackson 2017, Jackson and Harel 2017), and (Jackson 2018). The primary theme of these references is that rational decisions are rarely possible and that cognitive bias must be taken into account.

Decisions with Cognitive Bias

According to (INCOSE 2015) ideal decisions are made while “objectively identifying, characterizing, and evaluating a set of alternatives for a decision...” Research in the field of behavioral economics has shown that these decisions can be distorted by a phenomenon known as cognitive bias. Furthermore, most decision makers are unaware of these biases. The literature also provides methods for mitigating these biases.

According to (Haselton, Nettle, and Andrews 2005, p. 2) a cognitive bias represents a situation in which “human cognition reliably produces representations that are systematically distorted compared to some aspect of objective reality.” Cognitive biases are typically stimulated by emotion and prior belief. The literature reveals large numbers of cognitive biases of which the following three are typical:

1. The rankism bias. According to (Fuller 2011), rankism is simply the idea that persons of higher rank in an organization are better able to assert their authority over persons of lower rank regardless of the decision involved. Rankism frequently occurs in aircraft cockpits. According to (McCreary et al. 1998), rankism was a factor in the famous Tenerife disaster.
2. The complacency bias. According to (Leveson 1995, pp. 54-55), complacency is the disregard for safety and the belief that current safety measures are adequate. According to (Leveson 1995, pp. 54-55), complacency played a role in the Three Mile Island and Bhopal disasters.
3. The optimism bias. According to (Leveson 1995, pp. 54-55), famous physicist Richard Feynman states that NASA “exaggerates the reliability of the system.” This is an example of the optimism bias.

Mitigation of Cognitive Bias

Various sources have suggested methods to mitigate the effects of cognitive bias. Following are some of the major ones.

1. Independent Review. The idea of independent review is that advice on decisions should come from an outside body, called by the Columbia Accident Investigation Board (CAIB) (NASA 2003, 227) as the Independent Technical Authority (ITA). This authority must be both organizationally and financially independent of the program in question. That is, the ITA cannot be subordinate to the program manager.
2. Crew Resource Management. Following a period of high accident rate, several airlines have adopted the crew resource management (CRM) method. The primary purposes of this method are first to assure that all crew members do their job properly and secondly that they communicate with the pilot effectively when they have a concern. The impetus for this method was the judgment that many pilots were experiencing the rankism bias or were preoccupied with other tasks and simply did not understand the concerns of the other crew members. The result is that this strategy has been successful, and that the accident rate has fallen.
3. The Premortem. (Kahneman 2011) (pp. 264-265) suggests this method of nudging in an organizational context. This method, like others, requires a certain amount of willingness on the part of the decision-maker to participate in this process. It calls for decision-makers to surround themselves with trusted experts in advance of major decisions. According to Kahneman the primary job of the experts is to present the negative argument against any decision. For example, the decision-maker should not authorize the launch now, perhaps later.

References

Works Cited

- Buede, D.M. 2009. *The engineering design of systems: Models and methods*. 2nd ed. Hoboken, NJ: John Wiley & Sons Inc.
- Edwards, W., R.F. Miles Jr., and D. Von Winterfeldt. 2007. *Advances In Decision Analysis: From Foundations to Applications*. New York, NY: Cambridge University Press.
- Fuller, R.W. 2011. "What is Rankism and Why do We "Do" It?" *Psychology Today*. 25 May 2011. <https://www.psychologytoday.com/us/blog/somebodies-and-nobodies/201002/what-is-rankism-and-why-do-we-do-it>
- Haselton, M.G., D. Nettle, and P.W. Andrews. 2005. "The Evolution of Cognitive Bias." *Handbook of Psychology*.
- INCOSE. 2015. *Systems Engineering Handbook*, 4th Ed. Edited by D.D. Walden, G.J. Roedler, K.J. Forsberg, R.D. Hamelin, and T.M. Shortell. San Diego, CA: International Council on Systems Engineering (INCOSE).
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- Kahneman, D. 2011. "Thinking Fast and Slow." New York, NY: Farrar, Straus, and Giroux.
- Keeney, R.L. and H. Raiffa. 1976. *Decisions with Multiple Objectives - Preferences and Value Tradeoffs*. New York, NY: Wiley.
- Keeney, R.L. 1992. *Value-Focused Thinking: A Path to Creative Decision-Making*. Cambridge, MA: Harvard University Press.
- Keeney, R.L. 1993. "Creativity in MS/OR: Value-focused thinking—Creativity directed toward decision making." *Interfaces*, 23(3), p.62–67.
- Leveson, N. 1995. *Safeware: System Safety and Computers*. Reading, MA: Addison Wesley.
- McCreary, J., M. Pollard, K. Stevenson, and M.B. Wilson. 1998. "Human Factors: Tenerife Revisited." *Journal of Air Transportation World Wide*. 3(1).

- Murata, A. 2017. "Cultural Difference and Cognitive Biases as a Trigger of Critical Crashes or Disasters - Evidence from Case Studies of Human Factors Analysis." *Journal of Behavioral and Brain Science*. 7:299-415.
- Murata, A., T. Nakamura, and W. Karwowski. 2015. "Influences of Cognitive Biases in Distorting Decision Making and Leading to Critical Unfavorable Incidents." *Safety*. 1:44-58.
- Parnell, G.S. 2009. "Decision Analysis in One Chart," *Decision Line, Newsletter of the Decision Sciences Institute*. May 2009.
- Parnell, G.S., P.J. Driscoll, and D.L. Henderson (eds). 2011. *Decision Making for Systems Engineering and Management*, 2nd ed. Wiley Series in Systems Engineering. Hoboken, NJ: Wiley & Sons Inc.
- Parnell, G.S., T. Bresnick, S. Tani, and E. Johnson. 2013. *Handbook of Decision Analysis*. Hoboken, NJ: Wiley & Sons.
- Thaler, Richard H., and Cass R. Sunstein. 2008. *Nudge: Improving Decisions About Health, Wealth, and Happiness*. New York: Penguin Books.

Primary References

- Buede, D.M. 2004. "On Trade Studies." Proceedings of the 14th Annual International Council on Systems Engineering International Symposium, 20-24 June, 2004, Toulouse, France.
- Keeney, R.L. 2004. "Making Better Decision Makers." *Decision Analysis*, 1(4), pp.193–204.
- Keeney, R.L. & R.S. Gregory. 2005. "Selecting Attributes to Measure the Achievement of Objectives". *Operations Research*, 53(1), pp.1–11.
- Kirkwood, C.W. 1996. *Strategic Decision Making: Multiobjective Decision Analysis with Spreadsheets*. Belmont, California: Duxbury Press.

Additional References

- Buede, D.M. and R.W. Choisser. 1992. "Providing an Analytic Structure for Key System Design Choices." *Journal of Multi-Criteria Decision Analysis*, 1(1), pp.17–27.
- Felix, A. 2004. "Standard Approach to Trade Studies." Proceedings of the International Council on Systems Engineering (INCOSE) Mid-Atlantic Regional Conference, November 2-4 2004, Arlington, VA.
- Felix, A. 2005. "How the Pro-Active Program (Project) Manager Uses a Systems Engineer's Trade Study as a Management Tool, and not just a Decision Making Process." Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium, July 10-15, 2005, Rochester, NY.
- Jackson, S. 2017. "Irrationality in Decision Making: A Systems Engineering Perspective." *INCOSE Insight*, 74.
- Jackson, S. 2018. "Cognitive Bias: A Game-Changer for Decision Management?" *INCOSE Insight*, 41-42.
- Jackson, S. and A. Harel. 2017. "Systems Engineering Decision Analysis can benefit from Added Consideration of Cognitive Sciences." *Systems Engineering*. 55, 19 July.
- Miller, G.A. 1956. "The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information." *Psychological Review*, 63(2), p.81.
- Ross, A.M. and D.E. Hastings. 2005. "Tradespace Exploration Paradigm." Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium, July 10-15, 2005, Rochester, NY.
- Sproles, N. 2002. "Formulating Measures of Effectiveness." *Systems Engineering*, 5(4), p. 253-263.
- Silletto, H. 2005. "Some Really Useful Principles: A new look at the scope and boundaries of systems engineering." Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium, July 10-15, 2005, Rochester, NY.

Ullman, D.G. and B.P. Spiegel. 2006. "Trade Studies with Uncertain Information." Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium, July 9-13, 2006, Orlando, FL.

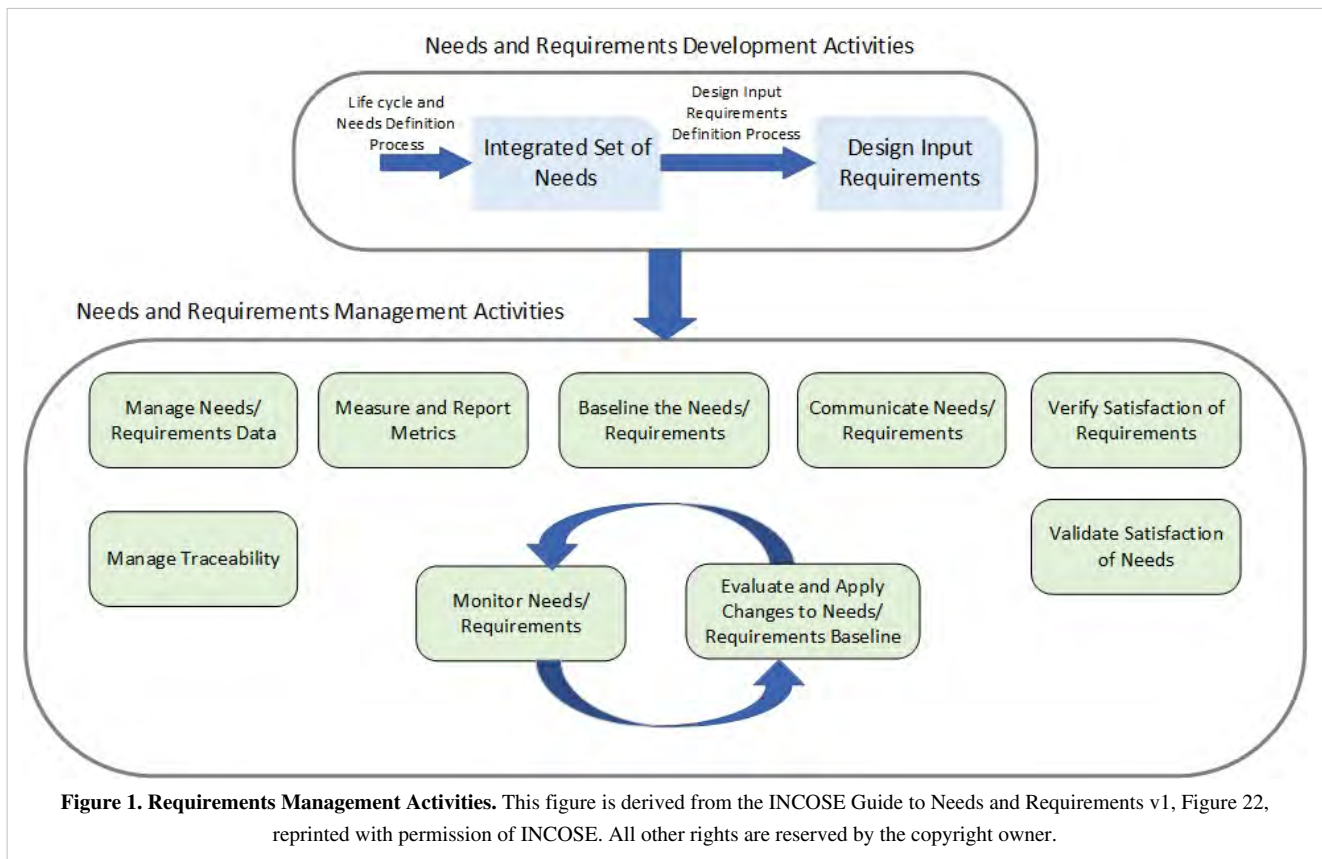
Requirements Management

- Lead Author:
- Tami Katz
- Contributing Authors:
- Lou Wheatcraft and Mike Ryan

Requirements management (RM) is performed to ensure alignment of requirements with other representations, analyses, and system artifacts generated across the life cycle. The scope of RM includes management of artifacts from the Concept Definition processes: Business or Mission Analysis and Stakeholder Needs Definition, as well as traceability to other systems engineering artifacts as described in System Requirements Definition process. RM includes providing an understanding of the requirements, an understanding of the needs from which they originated, establishing a baseline, communicating the needs and requirements, managing changes, providing status, and maintaining traceability with the rest of the artifacts of System Definition (INCOSE NRM 2022).

Purpose and Definition

As shown in Figure 1, RM is a cross-cutting series of activities that involve managing the sets of needs and the sets of design input requirements, including managing the needs and requirement definition activities, baselining the needs and requirements, communicating the needs and requirements, managing the flow-down (allocation and budgeting) of requirements from one level to another, managing interactions (interfaces) both internal and external to the SoI, managing bidirectional traceability, and managing the design and system verification and validation artifacts, and managing change (INCOSE GtNR 2022).



While needs and requirements development addresses elicitation and creation of the needs and requirements as described in System Requirements Definition, the RM activities address how needs and requirements are managed across the project life cycle. These activities leverage other systems engineering management processes including Configuration Management (CM), Interface Management, Systems Analysis, and Information Management.

Process Approach

RM begins at the beginning of a project addressing the management of needs and requirements across the life cycle, ensuring the data and information from the development life cycle activities are captured, configuration controlled, and communicated. Enablers for RM include organizational processes, tools, and trained personnel in those processes and tools.

Configuration Management and Change Control

RM is closely tied to CM activities associated with baseline management and control of the sets of needs and requirements and other related artifacts. When the sets of needs and requirements have been defined, assessed, and approved, they are baselined. The baseline allows the project to define and manage budgets and schedules as well as analyze and understand the impact (technical, cost, and schedule) of any proposed changes.

There are several reasons needs and requirements could change:

- changing stakeholder needs, requirements, expectations,
- customer budget and schedule changes,
- emerging threats and risks,
- re-allocation of system performance, and
- changing operational environments.

Note that due to the iterative and recursive nature of SE across the life cycle there is an expectation of changes to needs and requirements, particularly early in the development life cycle, as requirements are allocated, budgeted, and requirements defined for lower-level architecture entities. This drives the need for an established change control process early in the effort.

Changes to needs and requirements can include modifications, additions, or deletions. Once a need or requirement is baselined, changes must include rationale why the change is necessary, which helps with impact assessment of the change. Impacts could include the cost of making changes at multiple levels in the architecture hierarchy (including suppliers), as well as the cost/schedule/technical impacts associated with any work-in-process updates (design, verification, etc.). Impacts to related needs, requirements, and interfaces also need to be considered when making changes. Traceability, described below, is a powerful enabler to support assessment and impact of need and requirement changes.

Monitoring and Control

The monitoring process uses the Measurement process to enable situational awareness of the status and quality of the RM process activities, including status of needs and requirement definition activities, incorporation of changes, and progress towards obtaining objective evidence during System Verification and System Validation that the needs and requirements have been met.

Controlling the needs and requirements involves actions taken to ensure the integrated set of needs reflect the life cycle concepts, MGOs, and measures from which they were derived, and that the requirements continue to address the intent of the integrated set of needs from which they were transformed. Monitoring and controlling also includes resolving unknowns (To Be Resolved (TBR), To Be Determined (TBD), etc.), addressing the quality and correctness of the needs and requirements, and managing changes to the needs and requirements.

Several types of metrics can be used as part of Monitoring and Controlling:

- number of needs,
- number of requirements,
- number of needs and requirements with TBDs or TBRs
- needs and requirements not traced to a source,
- status of system verification, and
- status of system validation,

The project must define which metrics will be used to monitor and control the needs and requirements as well as choose tools that enable these metrics to be defined and managed as well as tools that can communicate these metrics to the project team. The ability to monitor these metrics is supported through the use of attributes that provide additional content for the need or requirement. Example attributes to support requirements management include Rationale, Status, Criticality, Priority, Stability, and Responsible Person or Owner. Reference System Requirements Definition for more details on defining attributes.

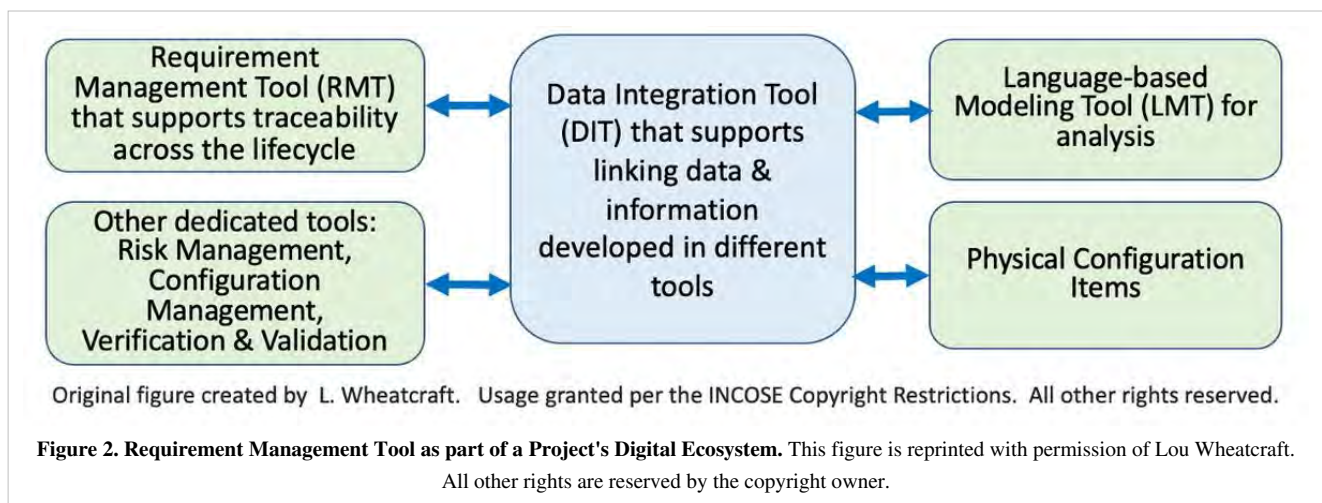
Requirement Management Tools

There are many tools available to provide a supporting infrastructure for needs and requirements management; the best choice is the one that matches the needs and processes of the project or enterprise. Desired capabilities and features of RM tools include definition, collaboration, change control, and traceability to other project data (INCOSE NRM 2022). A requirements management tool (RMT) can enable a project's success by providing several capabilities:

- capturing the needs, requirements, and associated attributes,
- capturing requirement traceability to other data,
- communication of metrics and status,
- management of version control and changes, and
- facilitation of change impact analysis.

Modern RMTs vary in capabilities and costs. Pointers for finding various RMTs and other SE tools can be found in the Primary References section.

It is recommended that RMTs enable the sharing of data and information with other tools in the project toolset as part of a larger digital engineering ecosystem, as highlighted in Figure 2; this ability helps to maximize the ability to ensure consistency with other project data and artifacts generated across the life cycle, which is critical to being able to establish the Authoritative Source of Truth (ASoT) (INCOSE NRM 2022).



RMTs are most effective if they are setup with a common project schema, ontology, and templates, and team members are trained in their use before project initiation, ensuring the project team spends their time on the definition and management of needs and requirements and not on extensive development of the tool infrastructure.

Managing Traceability

As described in System Requirements Definition, traceability can be established between the needs and requirements and other sets of data and artifacts across the life cycle:

- operational Scenarios,
- risks,
- related requirements, and
- verification/validation artifacts.

RM processes are used to monitor traceability of data and ensure they are maintained over the system life cycle. This is enabled through use of toolsets that allow linkages of the needs and requirements to the other data within a digital ecosystem as shown in Figure 2.

Requirements Management Planning

The RM activities involve project resources and must be planned as part of the overall project and systems engineering planning efforts. A Requirements Management Plan (RMP) can be used to define and communicate scope and processes for managing needs and requirements across the SoI life cycle. An RMP keeps all team members and stakeholders on the same page. For smaller and less complex projects, the needs and requirements management process could be captured within the program or project management plan (PMP) or systems engineering management plan (SEMP). For larger and more complex projects, it is recommended a standalone RMP be developed.

There are several topics addressed within an RMP:

- Who will be responsible for the RM activities and deliverables?
- What tool(s) will be used?
- How will the project team interact with the stakeholders?
- What activities are involved in life cycle concepts and needs definition?
- What activities are involved in requirements definition?
- How will the needs and requirements be assessed for quality and correctness?
- How will changes be managed and controlled?
- What types of traceability will be used?
- What is the hierarchy of the sets of needs and requirements and how will it be managed?
- What attributes will be used?
- How will metrics need to be defined and reported?
- How are TBRs and TBDs managed?

Projects should plan for the RM approach based on the scale and complexity of the project and system to be developed and generate an RMP accordingly, and then update the plan if the processes evolve. Note that some projects are required to deliver their RMP to a customer, which may require additional content; however, do not generate a plan merely because a customer directed it, the development of an RMP is critical to project success. Proper planning can ensure desired outcomes when implementing an RM process.

References

Works Cited

- INCOSE. 2022. *INCOSE Needs and Requirements Manual (NRM)*, version 1.1. INCOSE-TP-2021-002-01.
- INCOSE. 2022. *INCOSE Guide to Needs and Requirements (GtNR)*, version 1. INCOSE-TP-2021-003-01.

Primary References

- INCOSE. 2022. *INCOSE Needs and Requirements Manual (NRM)*, version 1.1. INCOSE-TP-2021-002-01.
- INCOSE. 2022. *INCOSE Guide to Needs and Requirements (GtNR)*, version 1. INCOSE-TP-2021-003-01.
- INCOSE and PPI Systems Engineering Tools Database ^[1].
- Pohl, K. (2010). *Requirements Engineering Fundamentals, Principles, and Techniques*.

Additional References

none

References

[1] <https://www.ppi-int.com/resources/setdb/>

Risk Management

- Lead Authors:
- Ed Conrow, Ray Madachy, and Garry Roedler
- Contributing Authors:
- Victor Bertolazzo, Leighton Johnson, Bob Parro, Jack Stein, and Richard Turner

The purpose of risk management is to reduce risks to an acceptable level before they occur, throughout the life of the product or project. Risk management is a continuous, forward-looking process that is applied to anticipate and avert risks that may adversely impact the project, and can be considered both a project management and a systems engineering process. A balance must be achieved on each project in terms of overall risk management ownership, implementation, and day-to-day responsibility between these two top-level processes.

For the SEBoK, risk management falls under the umbrella of Systems Engineering Management, though the wider body of risk literature is explored below.

Risk Management Process Overview

Risk is a measure of the potential inability to achieve overall program objectives within defined cost, schedule, and technical constraints.

The Risk Management Process section of the INCOSE Systems Engineering Handbook: A Guide for Systems Life Cycle Processes and Activities, 5th Edition, provides a comprehensive overview of risk management which is intended to be consistent with the Risk Management Process section of ISO 15288:2023. In this section of the INCOSE handbook two prominent risk concepts which may be encountered in a systems engineering project are discussed and tied to their respective internationally accepted definitions of risk (INCOSE 2023, pg. 84):

- The effect of uncertainty on objectives, see ISO/IEC/IEEE 15288, ISO 31073, ISO/IEC/IEEE 16085, ISO 31000, ISO 27000
- The combination of the probability of occurrence of harm and the severity of that harm, see ISO Guide 51, ISO 22367, ISO 14971

The first definition, which includes a note-to-entry stating that “effects may be negative or positive,” is different from the second in that the second definition accommodates only negative effects. Also, it is important to note that the first definition ties risks to specified objectives, while the second ties risks to identified types of potential harm without regard to objectives. Thus, the second definition is used extensively in safety engineering and in projects that require compliance to safety and risk management standards and regulations intended for the protection of public health, safety, and security.

A long-established approach to risk management is to identify and manage a set of individual risks for the project, while characterizing or expressing each individual risk as the combination of the following two components (DAU 2003a; INCOSE 2023, pg. 84-85):

1. the probability (or likelihood) of failing to achieve a particular outcome
 2. the consequences (or impact) of failing to achieve that outcome
-

Among the benefits of this approach is that it supports the need for simple project risk metrics and familiar visual aids such the risk matrix (or risk cube). However, in practice, it is usually necessary to select and utilize a combination of various methods, techniques, and tools as deemed necessary. International standard ISO 31010:2019 *Risk management – risk assessment techniques* describes and provides guidance on selecting and applying 41 risk assessment techniques, including the risk matrix.

New techniques able to identify and better address undesirable outcomes associated with highly complex (and unpredictable) system behaviors, e.g., System Theoretic Process Analysis (STPA), are emerging and gaining international acceptance (SAE, 2025).

In the domain of catastrophic risk analysis, risk has three components: (1) threat, (2) vulnerability, and (3) consequence (Willis et al. 2005).

Risk management involves defining a risk management strategy, identifying and analyzing risks, treating selected risks, and monitoring the progress in reducing risks to an acceptable level (SEI 2010; DoD 2023; DAU 2003a; DAU 2003b; PMI 2021). The SE community, as a general rule, has recommended that “positive effects” of risks identified using the “effect of uncertainty on objectives” definition of risk be referred to as “opportunities”. Opportunity and opportunity management is discussed in the “Opportunity and Opportunity Management” section below.

The SE risk management process includes the following activities:

- risk management planning
- risk identification
- risk analysis
- risk treatment
- risk monitoring

ISO/IEC/IEEE 16085 provides a detailed set of risk management activities and tasks which can be utilized in a risk management process aligned with ISO 31000, ISO 31073, and ISO 9001, which includes risk-based preventive action requirements.

Risk Management Planning

Risk management planning establishes and maintains a strategy for identifying, analyzing, treating, and monitoring risks within the project. The strategy, both the process and its implementation, is documented in a risk management plan (RMP).

The risk management process and its implementation should be tailored to each project and updated as appropriate throughout the life of the project. The RMP should be transmitted in an appropriate means to the project team and key stakeholders.

The risk management strategy includes as necessary the risk management process of all supply chain suppliers and describes how risks from all suppliers will be raised to the next level(s) for incorporation in the project risk process.

The context description for the risk management process should include a description of stakeholders’ perspectives, risk categories, and a description (perhaps by reference) of the technical and managerial objectives, assumptions and constraints. The risk categories include the relevant technical areas of the system and facilitate identification of risks across the life cycle of the system. As noted in ISO 31000, the aim of this step is to generate a comprehensive list of risks based on those events that might create, enhance, prevent, degrade, accelerate or delay the achievement of objectives.

The RMP should contain key risk management information; Conrow (2003) identifies the following as key components of RMP:

- a project summary
 - project acquisition and contracting strategies
 - key definitions
-

- list of key documents
- process steps
- inputs, tools and techniques, and outputs per process step
- linkages between risk management and other project processes
- key ground rules and assumptions
- risk categories
- buyer and seller roles and responsibilities
- organizational and personnel roles and responsibilities

Generally, the level of detail in an RMP is risk-driven, with simple plans for low risk projects and detailed plans for high risk projects.

Risk Identification

Risk identification is the process of examining the project products, processes, and requirements to identify and document candidate risks. Risk identification should be performed continuously at the individual level as well as through formerly structured events at both regular intervals and following major program changes (e.g., project initiation, re-baselining, change in acquisition phase, etc.).

Conrow (2009) states that systems engineers should use one or more top-level approaches (e.g., work breakdown structure (WBS), key processes evaluation, key requirements evaluation, etc.) and one or more lower-level approaches (e.g., affinity, brainstorming, checklists and taxonomies, examining critical path activities, expert judgment, Ishikawa diagrams, etc.) in risk identification. The top and lower-level approaches are essential but there is no single accepted method — all approaches should be examined and used as appropriate.

Candidate risk management documentation should include the following items where possible, as identified by Conrow (2003 p.198):

- risk title
- structured risk description
- applicable risk categories
- potential root causes
- relevant historical information
- responsible individual and manager

It is important to use structured risk descriptions such as an *if-then* format: *if* (an event occurs--trigger), *then* (an outcome or affect occurs). Another useful construct is a *condition* (that exists) that leads to a potential *consequence* (outcome) (Gluch 1994). These approaches help the analyst to better think through the nature of the risk.

Risk Analysis

Risk analysis is the process of systematically evaluating each identified risk to estimate the probability of occurrence (likelihood) and consequence of occurrence (impact), and then converting the results to a corresponding risk level or rating.

There is no *best* analysis approach for a given risk category. Risk scales and a corresponding matrix, simulations, and probabilistic risk assessments are often used for technical risks, while decision trees, simulations and payoff matrices are used for cost risk; and simulations are used for schedule risk. Risk analysis approaches are sometimes grouped into qualitative and quantitative methods. A structured, repeatable methodology should be used in order to increase analysis accuracy and reduce uncertainty over time.

Once the risk level for each risk is determined, the risks need to be prioritized. Prioritization is typically performed by risk level (e.g., low, medium, high), risk score (the pair of max (probability), max (consequence) values), and other considerations such as time-frame, frequency of occurrence, and interrelationship with other risks (Conrow

2003, pp. 187-364).

Widely used quantitative methods include decision trees and the associated expected monetary value analysis (Clemen and Reilly 2001), modeling and simulation (Law 2007; Mun 2010; Vose 2000), payoff matrices (Kerzner 2009, p. 747-751), probabilistic risk assessments (Kumamoto and Henley 1996; NASA 2002), and other techniques. Risk prioritization can directly result from the quantitative methods employed. For quantitative approaches, care is needed in developing the model structure, since the results will only be as good as the accuracy of the structure, coupled with the characteristics of probability estimates or distributions used to model the risks (Law 2007; Evans, Hastings, and Peacock 2011).

Descriptions and application guidance for a number of the above mentioned, as well as additional, risk assessment methods and techniques, may be found among the 41 contained in ISO 31010. Depending on the project environment and objectives, it may be beneficial to devote resources to researching, developing, and staying abreast of new and emerging analysis and assessment methods, techniques, and tools.

Risk Treatment

Risk treatment is the process that identifies and selects options and implements the desired option to reduce a risk to an acceptable level, given program constraints (budget, other resources) and objectives (DAU 2003a, 20-23, 70-78).

The risk treatment strategy selected is the combination of the most desirable risk treatment option coupled with a suitable implementation approach for that option (Conrow 2003). Risk treatment options include assumption, avoidance, control (mitigation), and transfer. All four options should be evaluated and the best one chosen for each risk. An appropriate implementation approach is then chosen for that option. Hybrid strategies can be developed that include more than one risk treatment option, but with a single implementation approach.

Additional risk treatment strategies can also be developed for a given risk and either implemented in parallel with the primary strategy or be made a contingent strategy that is implemented if a particular trigger event occurs during the execution of the primary strategy. Often, this choice is difficult because of uncertainties in the risk probabilities and impacts. In such cases, buying information to reduce risk uncertainty via prototypes, benchmarking, surveying, modeling, etc. will clarify risk treatment decisions (Boehm 1981).

Risk Monitoring

Risk monitoring is used to evaluate the effectiveness of risk treatment activities against established metrics and provide feedback to the other risk management process steps. Risk monitoring results may also provide a basis to (a) update risk treatment plans (RTPs), and/or the appropriate part(s) of the RMP, and (b) develop additional risk treatment options and approaches, and re-analyze risks. In some cases, monitoring results may also be used to identify new risks, revise an existing risk with a new facet, or revise some aspects of risk management planning (DAU 2003a, p. 20). Some risk monitoring approaches that can be applied include earned value, program metrics, technical performance measures (TPMs), schedule analysis, and variations in risk level. Risk monitoring approaches should be updated and evaluated at the same time and WBS level; otherwise, the results may be inconsistent.

Opportunity and Opportunity Management

In principle, opportunity management is the duality to risk management, with two components: (1) probability of achieving an improved outcome and (2) impact of achieving the outcome. Thus, both should be addressed in risk management planning and execution. In practice, however, a positive opportunity exposure will not match a negative risk exposure in utility space, since the positive utility magnitude of improving an expected outcome is considerably less than the negative utility magnitude of failing to meet an expected outcome (Canada 1971; Kahneman-Tversky 1979). Further, since many opportunity-management initiatives have failed to anticipate serious side effects, all candidate opportunities should be thoroughly evaluated for potential risks to prevent unintended consequences from

occurring.

In addition, while opportunities may provide potential benefits for the system or project, each opportunity pursued may have associated risks that detract from the expected benefit. This may reduce the ability to achieve the anticipated effects of the opportunity, in addition to any limitations associated with not pursuing an opportunity.

Integrated Risk Management (IRM) and Linkages to Other Systems Engineering Management Topics

Per ISO 31000:2018, “integrating risk management with all organizational processes improves the performance of risk management while gaining efficiencies”. Clause 7 of ISO/IEC/IEEE 16085:2021 provides a methodical approach for the integration of risk management and “risk-based thinking” into all systems engineering life cycle processes.

Within Systems Engineering Management the practices of IRM and establishing linkages between risk management and other SE life cycle processes has been an area of interest for a number of decades. For example: The measurement process provides indicators for risk analysis. Project planning involves the identification of risk and planning for stakeholder involvement. Project assessment and control monitors project risks. Decision management evaluates alternatives for selection and treatment of identified and analyzed risks. With respect to linkages and interrelationship between systems engineering and project management, considerable work is documented the PMI-INCOS- MIT collaborative Wiley book publication *Integrating Program Management and Systems Engineering: Methods, Tools, and Organizational Systems for Improving Performance*, (Rebentisch 2017).

Practical Considerations

Key pitfalls and good practices related to systems engineering risk management are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in performing risk management are below in Table 1.

Table 1. Risk Management Pitfalls. (SEBoK Original)

Name	Description
Process Over-Reliance	Over-reliance on the process side of risk management without sufficient attention to human and organizational behavioral considerations.
Lack of Continuity	Failure to implement risk management as a continuous process. Risk management will be ineffective if it's done just to satisfy project reviews or other discrete criteria. (Charette, Dwinnell, and McGarry 2004, 18-24 and Scheinin 2008).
Tool and Technique Over-Reliance	Over-reliance on tools and techniques, with insufficient thought and resources expended on how the process will be implemented and run on a day-to-day basis.
Lack of Vigilance	A comprehensive risk identification will generally not capture all risks; some risks will always escape detection, which reinforces the need for risk identification to be performed continuously.
Automatic Mitigation Selection	Automatically select the risk treatment mitigation option, rather than evaluating all four options in an unbiased fashion and choosing the “best” option.
Sea of Green	Tracking progress of the risk treatment plan, while the plan itself may not adequately include steps to reduce the risk to an acceptable level. Progress indicators may appear “green” (acceptable) associated with the risk treatment plan: budgeting, staffing, organizing, data gathering, model preparation, etc. However, the risk itself may be largely unaffected if the treatment strategy and the resulting plan are poorly developed, do not address potential root cause(s), and do not incorporate actions that will effectively resolve the risk.

- Band-Aid Risk Treatment
- Treating risks (e.g., interoperability problems with changes in external systems) by patching each instance, rather than addressing the root cause(s) and reducing the likelihood of future instances.

Good Practices

Some good practices gathered from the references are below in Table 2.

Table 2. Risk Management Good Practices. (SEBoK Original)

Name	Description
Top Down and Bottom Up	• Risk management should be both “top down” and “bottom up” in order to be effective. The project manager or deputy need to own the process at the top level, but risk management principles should be considered and used by all project personnel.
Early Planning	• Include the planning process step in the risk management process. Failure to adequately perform risk planning early in the project phase contributes to ineffective risk management.
Risk Analysis Limitations	• Understand the limitations of risk analysis tools and techniques. Risk analysis results should be challenged because considerable input uncertainty and/or potential errors may exist.
Robust Risk Treatment Strategy	• The risk treatment strategy should attempt to reduce both the probability and consequence of occurrence terms. It is also imperative that the resources needed to properly implement the chosen strategy be available in a timely manner, else the risk treatment strategy, and the entire risk management process, will be viewed as a “paper tiger.”
Structured Risk Monitoring	• Risk monitoring should be a structured approach to compare actual vs. anticipated cost, performance, schedule, and risk outcomes associated with implementing the RHP. When ad-hoc or unstructured approaches are used, or when risk level vs. time is the only metric tracked, the resulting risk monitoring usefulness can be greatly reduced.
Update Risk Database	• The risk management database (registry) should be updated throughout the course of the program, striking a balance between excessive resources required and insufficient updates performed. Database updates should occur at both a tailored, regular interval and following major program changes.

References

Works Cited

- Boehm, B. 1981. *Software Engineering Economics*. Upper Saddle River, NJ, USA: Prentice Hall.
- Boehm, B. 1989. *Software Risk Management*. Los Alamitos, CA; Tokyo, Japan: IEEE Computer Society Press: 115-125.
- Canada, J.R. 1971. *Intermediate Economic Analysis for Management and Engineering*. Upper Saddle River, NJ, USA: Prentice Hall.
- Carr, M., S. Konda, I. Monarch, F. Ulrich, and C. Walker. 1993. *Taxonomy-based risk identification*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-93-TR-6.
- Charette, R., L. Dwinnell, and J. McGarry. 2004. "Understanding the roots of process performance failure." *CROSSTALK: The Journal of Defense Software Engineering* (August 2004): 18-24.
- Clemen, R., and T. Reilly. 2001. *Making hard decisions*. Boston, MA, USA: Duxbury.
- Conrow, E. 2003. *Effective Risk Management: Some Keys to Success*, 2nd ed. Reston, VA, USA: American Institute of Aeronautics and Astronautics (AIAA).
- Conrow, E. 2008. "Risk analysis for space systems." Paper presented at Space Systems Engineering and Risk Management Symposium, 27-29 February, 2008, Los Angeles, CA, USA.
- Conrow, E. and P. Shishido. 1997. "Implementing risk management on software intensive projects." *IEEE Software*. 14(3) (May/June 1997): 83-9.

- DAU. 2003a. *Risk Management Guide for DoD Acquisition: Fifth Edition*, version 2. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU) Press.
- DAU. 2003b. *U.S. Department of Defense extension to: A guide to the project management body of knowledge (PMBOK(R) guide), first edition*. Version 1. 1st ed. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU) Press.
- DoD. 2023. *Risk, Issue, and Opportunity Management Guide for Defense Acquisition Programs*. Washington, DC, USA: Office of the Deputy Assistant Secretary of Defense for Systems Engineering/Department of Defense.
- Evans, M., N. Hastings, and B. Peacock. 2000. *Statistical Distributions*, 3rd ed. New York, NY, USA: Wiley-Interscience.
- Forbes, C., M. Evans, N. Hastings, and B. Peacock. 2011. "Statistical Distributions," 4th ed. New York, NY, USA.
- Gallagher, B., P. Case, R. Creel, S. Kushner, and R. Williams. 2005. *A taxonomy of operational risk*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-2005-TN-036.
- Gluch, P. 1994. *A Construct for Describing Software Development Risks*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-94-TR-14.
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-05.
- ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.
- Kerzner, H. 2009. *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*. 10th ed. Hoboken, NJ, USA: John Wiley & Sons.
- Kahneman, D., and A. Tversky. 1979. "Prospect theory: An analysis of decision under risk." *Econometrica*. 47(2) (Mar., 1979): 263-292.
- Kumamoto, H. and E. Henley. 1996. *Probabilistic Risk Assessment and Management for Engineers and Scientists*, 2nd ed. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) Press.
- Law, A. 2007. *Simulation Modeling and Analysis*, 4th ed. New York, NY, USA: McGraw Hill.
- Mun, J. 2010. *Modeling Risk*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.
- NASA. 2002. *Probabilistic Risk Assessment Procedures Guide for NASA Managers and Practitioners*, version 1.1. Washington, DC, USA: Office of Safety and Mission Assurance/National Aeronautics and Space Administration (NASA).
- PMI. 2021. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 7th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- Rebentisch, E. 2017. *Integrating Program Management and Systems Engineering: Methods, Tools, and Organizational Systems for Improving Performance*. Hoboken, NJ, USA: John Wiley & Sons.
- SAE. 2025. *System Theoretic Process Analysis (STPA) Standard for All Industries*, Warrendale, PA, USA: SAE International (SAE). SAE J3307_202503.
- Scheinin, W. 2008. "Start Early and Often: The Need for Persistent Risk Management in the Early Acquisition Phases." Paper presented at Space Systems Engineering and Risk Management Symposium, 27-29 February 2008, Los Angeles, CA, USA.
- SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- Vose, D. 2000. *Quantitative Risk Analysis*, 2nd ed. New York, NY, USA: John Wiley & Sons.
-

Willis, H.H., A.R. Morral, T.K. Kelly, and J.J. Medby. 2005. *Estimating Terrorism Risk*. Santa Monica, CA, USA: The RAND Corporation, MG-388.

Primary References

Boehm, B. 1981. *Software Engineering Economics*. Upper Saddle River, NJ, USA: Prentice Hall.

Boehm, B. 1989. *Software Risk Management*. Los Alamitos, CA; Tokyo, Japan: IEEE Computer Society Press, p. 115-125.

Conrow, E.H. 2003. *Effective Risk Management: Some Keys to Success*, 2nd ed. Reston, VA, USA: American Institute of Aeronautics and Astronautics (AIAA).

DoD. 2023. Risk, Issue, and Opportunity Management Guide for Defense Acquisition Programs. Washington, DC, USA: Office of the Deputy Assistant Secretary of Defense for Systems Engineering/Department of Defense.

SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Additional References

Canada, J.R. 1971. *Intermediate Economic Analysis for Management and Engineering*. Upper Saddle River, NJ, USA: Prentice Hall.

Carr, M., S. Konda, I. Monarch, F. Ulrich, and C. Walker. 1993. *Taxonomy-based risk identification*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-93-TR-6.

Charette, R. 1990. *Application Strategies for Risk Management*. New York, NY, USA: McGraw-Hill.

Charette, R. 1989. *Software Engineering Risk Analysis and Management*. New York, NY, USA: McGraw-Hill (MultiScience Press).

Charette, R., L. Dwinnell, and J. McGarry. 2004. "Understanding the roots of process performance failure." *CROSSTALK: The Journal of Defense Software Engineering* (August 2004): 18-24.

Clemen, R., and T. Reilly. 2001. *Making hard decisions*. Boston, MA, USA: Duxbury.

Conrow, E. 2010. "Space program schedule change probability distributions." Paper presented at American Institute of Aeronautics and Astronautics (AIAA) Space 2010, 1 September 2010, Anaheim, CA, USA.

Conrow, E. 2009. "Tailoring risk management to increase effectiveness on your project." Presentation to the Project Management Institute, Los Angeles Chapter, 16 April, 2009, Los Angeles, CA.

Conrow, E. 2008. "Risk analysis for space systems." Paper presented at Space Systems Engineering and Risk Management Symposium, 27-29 February, 2008, Los Angeles, CA, USA.

Conrow, E. and P. Shishido. 1997. "Implementing risk management on software intensive projects." *IEEE Software*. 14(3) (May/June 1997): 83-9.

DAU. 2003a. *Risk Management Guide for DoD Acquisition: Fifth Edition*. Version 2. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU) Press.

DAU. 2003b. *U.S. Department of Defense extension to: A guide to the project management body of knowledge (PMBOK(R) guide)*, 1st ed. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU) Press.

Dorofee, A., J. Walker, C. Alberts, R. Higuera, R. Murphy, and R. Williams (eds). 1996. *Continuous Risk Management Guidebook*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU).

Gallagher, B., P. Case, R. Creel, S. Kushner, and R. Williams. 2005. *A taxonomy of operational risk*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-2005-TN-036.

- Gluch, P. 1994. *A Construct for Describing Software Development Risks*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU), CMU/SEI-94-TR-14.
- Haimes, Y.Y. 2009. *Risk Modeling, Assessment, and Management*. Hoboken, NJ, USA: John Wiley & Sons, Inc.
- Hall, E. 1998. *Managing Risk: Methods for Software Systems Development*. New York, NY, USA: Addison Wesley Professional.
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-05.
- ISO. 2018. *Risk Management Guidelines*. Geneva, Switzerland: International Organization for Standardization (ISO), ISO 31000:2018.
- ISO/IEC. 2019. *Risk management—Risk assessment techniques*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 31010:2019.
- ISO/IEC/IEEE. 2021. *Systems and Software Engineering -- Life cycle processes -- Risk management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 16085:2021.
- ISO. 2022. *Space Systems -- Programme management -- Risk Management*. Geneva, Switzerland: International Organization for Standardization (ISO), ISO 17666:2025.
- Jones, C. 1994. *Assessment and Control of Software Risks*. Upper Saddle River, NJ, USA: Prentice-Hall.
- Kahneman, D. and A. Tversky. 1979. "Prospect theory: An analysis of decision under risk." *Econometrica*. 47(2) (Mar., 1979): 263-292.
- Kerzner, H. 2009. *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*, 10th ed. Hoboken, NJ: John Wiley & Sons.
- Kumamoto, H., and E. Henley. 1996. *Probabilistic Risk Assessment and Management for Engineers and Scientists*, 2nd ed. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) Press.
- Law, A. 2007. *Simulation Modeling and Analysis*, 4th ed. New York, NY, USA: McGraw Hill.
- MITRE. 2012. *Systems Engineering Guide to Risk Management*. Available online: https://www.mitre.org/work/systems_engineering/guide/acquisition_systems_engineering/risk_management/. Accessed on July 7, 2012. Page last updated on May 8, 2012.
- Mun, J. 2010. *Modeling Risk*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.
- NASA. 2002. *Probabilistic Risk Assessment Procedures Guide for NASA Managers and Practitioners*, version 1.1. Washington, DC, USA: Office of Safety and Mission Assurance/National Aeronautics and Space Administration (NASA).
- PMI. 2021. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 7th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- Rebentisch, E. 2017. *Integrating Program Management and Systems Engineering: Methods, Tools, and Organizational Systems for Improving Performance*. Hoboken, NJ, USA: John Wiley & Sons.
- SAE. 2025. *System Theoretic Process Analysis (STPA) Standard for All Industries*, Warrendale, PA, USA: SAE International (SAE). SAE J3307_202503.
- Scheinin, W. 2008. "Start Early and Often: The Need for Persistent Risk Management in the Early Acquisition Phases." Paper presented at Space Systems Engineering and Risk Management Symposium, 27-29 February 2008, Los Angeles, CA, USA.
- USAF. 2005. *SMC systems engineering primer & handbook: Concepts, processes, and techniques*, 3rd ed. Los Angeles, CA, USA: Space & Missile Systems Center/U.S. Air Force (USAF).
-

USAF. 2014. "SMC Risk Management Process Guide. *Version 2. Los Angeles, CA, USA: Space & Missile Systems Center/U.S. Air Force (USAF).*

Vose, D. 2000. *Quantitative Risk Analysis*. 2nd ed. New York, NY, USA: John Wiley & Sons.

Willis, H.H., A.R. Morral, T.K. Kelly, and J.J. Medby. 2005. *Estimating Terrorism Risk*. Santa Monica, CA, USA: The RAND Corporation, MG-388.

Configuration Management

- Lead Authors:
 - John Metcalf, Philip Hallenbeck, and Sandrine Gonthier
- Contributing Author:
 - Garry Roedler

Configuration management (CM) helps teams keep track of changes to a system over its life cycle—ensuring that what's built matches what was planned, and that everyone stays aligned as updates occur. Simply put, CM is about maintaining a clear, accurate picture of a system as it evolves—tracking changes, managing versions, and avoiding confusion.

As stated in ISO/IEC/IEEE 15288 (6.3.5.1): The purpose of the configuration management process is to manage system and system element configurations over their life cycle. The INCOSE *SE Handbook* describes CM as a key technical management process, supported by defined roles, resources, and controls. In this article, we may refer to a "system" or "product" depending on the application of the CM process.

Fundamental Concepts

CM and main CM concepts are defined slightly differently according to the various standards. The objective of this article is to provide a general understanding of CM based on common and general principles, usually aligned with INCOSE *SE Handbook*, ISO 10007, and SAE EIA-649C standards. Before entering in the description of the CM process, it is important to define a few concepts (below).

Applicability

CM applies to any kind of system; any product or process can be considered a system. Therefore, CM addresses hardware, software, services, systems, and systems of systems with equal relevance, or any combination of the above. CM is applicable regardless of the complexity of the system in question.

CM applies throughout the entire life cycle of the system. CM applies to all system life-cycle models and development approaches, with adequate tailoring.

In addition, to master the configuration of a system of interest, the configuration of the associated enabling systems should also be taken into account.

Configuration, Configuration Items, and Configuration Information

The configuration refers to the interrelated functional and physical characteristics outlined in configuration information, as defined by ISO 10007. The system and its individual elements, which fall under Configuration Management, are known as Configuration Items. Their functional and physical characteristics constitute their configuration information.

Configuration information encompasses the comprehensive data necessary to describe and document Configuration Items throughout the entire system life cycle. This includes artifacts produced during the execution of the technical processes, such as requirement specifications, architecture descriptions, design descriptions, and user documentation, as referenced in ISO/IEC/IEEE 15289.

Configuration information is understood as structured and contextualized data related to the system and its artifacts. While data serves as the raw input, information represents the processed output that adds context, relevance, and purpose. The collective set of configuration information associated with a system encapsulates all pertinent elements needed to document its characteristics.

CM fundamental objectives

The objectives of Configuration Management (CM) are to provide means to guarantee consistency between the system and its configuration information, ensure the integrity and traceability of this information and its changes over time, and facilitate reproducibility. These three objectives collectively enable the capabilities to define, realize, transition, operate, maintain, and dispose of the system.

How do you distinguish Configuration Management (CM) from Information Management (IM)?

Configuration Management (CM) and Information Management (IM) processes are interconnected; however, CM focuses on the relevance of information related to the system and its evolution over time, while IM concentrates on the management of the information itself. This includes its generation, collection, validation, transformation, dissemination, and disposal, overseeing the entire life cycle of the information.

CM is concerned with the significance and applicability of information in relation to the specific system elements with which it is associated, referred to as configuration information. While all information can be managed, only configuration information is subject to configuration management.

The CM and IM processes are complementary and closely aligned with the objective of ensuring traceability.

Configuration Management System

To ensure the CM process activities, a dedicated Configuration Management system to enable and support CM.

Process Overview

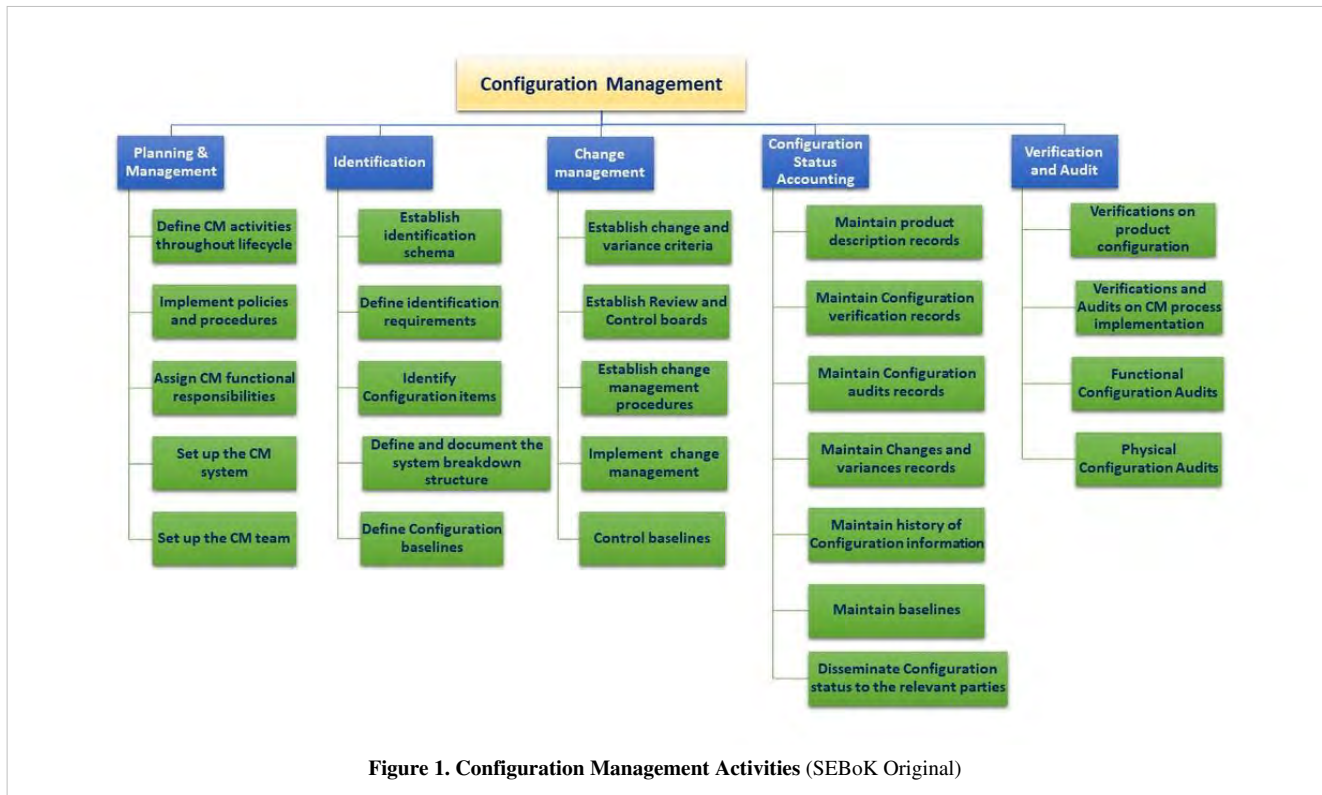
The Configuration Management process relies on appropriate processes, resources, and controls, to establish and maintain an authoritative source of truth on system configuration.

The CM process consists of activities that are widely used in key standards and references, including ISO/IEC/IEEE 15288, SAE EIA 649, and ISO 10007. The names of these activities may differ through various norms and standards, and in this article, the terms used are:

- Configuration Management Planning and Management
 - Configuration Identification
 - Configuration Change Management
 - Configuration Status Accounting
-

- Configuration Verification and Audit.

The benefits of using the CM process are maximized when all five activities are planned and executed. These activities are not strictly sequential; rather, they are interrelated and may be applied concurrently and iteratively as needed throughout the system life cycle.



CM Planning and Management

This first activity involves defining the scope and organization of CM tasks. It is considered good practice to formalize the results of this activity in a dedicated CM plan that guides implementation throughout the life cycle.

In terms of organization, the roles in charge of leading the Configuration Management activities on the project need to be defined early in the system life cycle. These roles constitute the CM team. This team is organized according to the size and complexity of the product to be managed and to the organizational rules of the organization. It should at least have a Configuration Manager, who oversees the CM plan.

CM Planning and Management defines the CM scope relevant for the system and for being able to maintain the consistency between the system and its configuration information: CM activities need to be adjusted to the type of system (hardware, software, service or combination of the above), the system life cycle, and to the system complexity, as well to the type of configuration information to be managed and to their criticality.

The activities depend also on the stages of the life cycle that need to be addressed as defined per contract: if all the stages of the life cycle must be managed from conception until retirement, the CM plan covers all these stages.

The CM plan should address the definition of the CM tasks, their organization, their schedule, the responsibilities of the various stakeholders of the system, and the tools to support the CM. CM activities involve most of the stakeholders of the system.

The CM plan should also be developed in consideration of the organizational context and culture. It must adhere to or incorporate applicable policies, procedures, and standards and it must accommodate acquisition, subcontracting and partnership situations, as well as any specific requirements agreements.

Except very specific use cases, CM planning and management should address the remaining activities as described hereafter.

Additional considerations about the Planning and Management of CM activities are provided in “CM implementation”.

Configuration Identification

This activity overarches the following task:

- Identify which elements of the system and which information should be under configuration management
- Identify where configuration management is needed among the elements of the system: these elements are called “Configuration Items”, the information related to these elements are their configuration information
- Define the identification rules for Configuration items and configuration information to apply to ensure the unicity, the rules to make these identifiers evolve, and the associated labelling constraints
- Structure the items and their information: this structure reflects the relationships between system items
- Define the rules for validating and releasing the items and the information under configuration management
- Define the baselines to be established at dedicated milestones of the system life cycle.

Note 1: a baseline specifies how a system is viewed for the purposes of management, control, and evaluation; each baseline is fixed at a specific point in time in the system life cycle and represents the current approved configuration; it serves as a basis for defining changes, and generally can only be changed through formal change procedures. See CM baselines for more details about typical CM baselines.

Note 2: depending on the practices and methods, the term “configuration item” may be used to designate anything that should be managed in the configuration management system, which leads to include the system elements and its information in the generic term “configuration items”.

Configuration Change Management

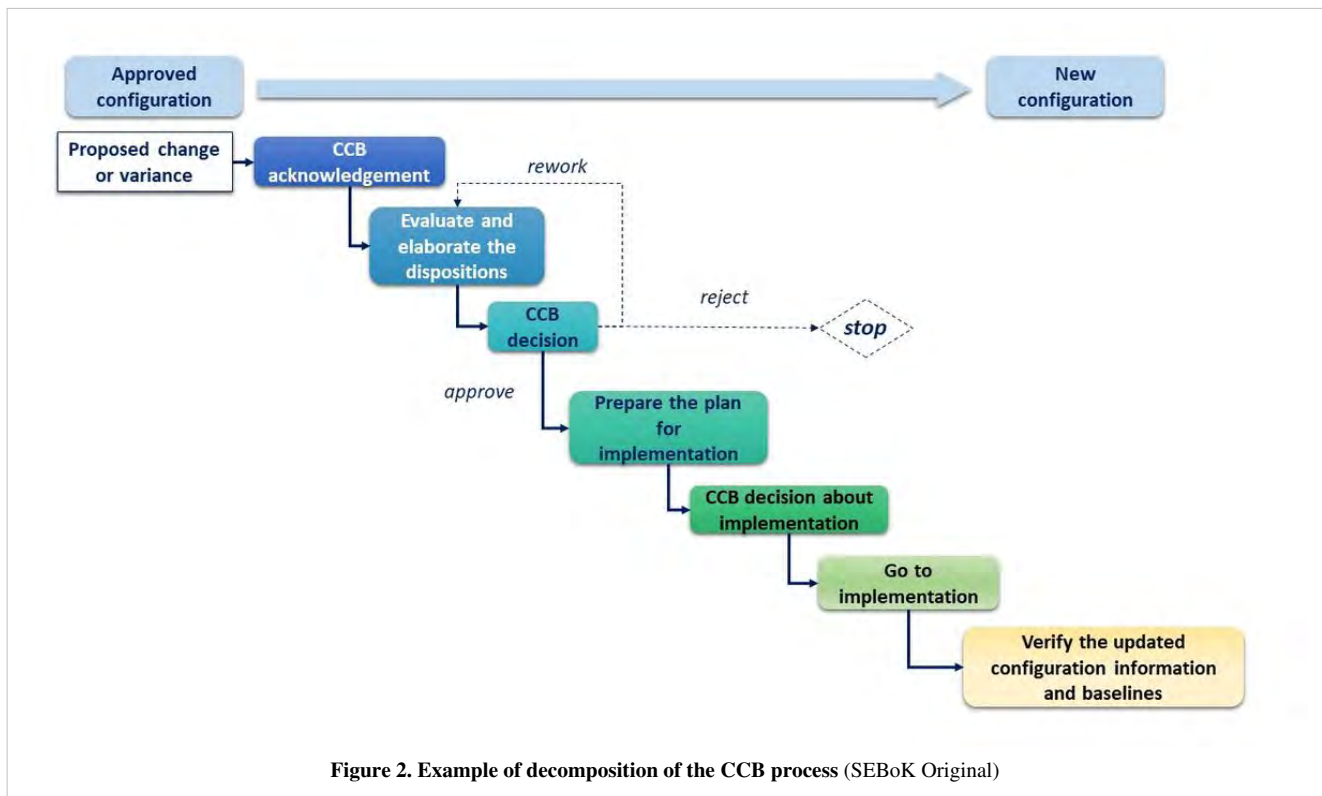
A disciplined Configuration Change Management process is critical for engineering systems.

It encompasses the analysis, justification, evaluation, coordination and disposition of changes to the system and variances to the system (non-compliance to its requirements).

This activity aims to control the evolution of the system, of its elements and of its configuration information throughout the life cycle.

To ensure the missions of the activity, boards are defined to monitor and make key decisions regarding changes and variances process.

These boards are commonly called "Configuration Control Boards (CCBs)" but may also be referred to as "Configuration Steering Boards" or "Change Control Boards (CCBs)". They can also be split into Change Review boards and Change Implementation boards, when separately addressing the decisions about agreeing to the changes and variances and the stage where changes are implemented.



A Configuration Control Board (CCB) typically includes:

- a CCB Chair,
- a Configuration Manager,
- the product or organization systems engineer,
- domain-specific subject-matter experts (SMEs) such as for software or mechanical engineering, product support, and cyber resiliency,
- procurement and contracting specialists.

The CCB dealing with the CM of a dedicated system or product (product CCB) may invite periodic participation from specialized or outside SMEs, including representation from vendors and subcontractors; this however must be carefully managed to ensure that information-access restrictions (especially for competition-sensitive information) is not compromised.

Organizational CCBs may be organized at a wider level and address several systems, products, or product lines and they typically include information technology (IT) and cyber resiliency SMEs who may not be needed on product-focused CCBs.

More advanced considerations about the change management and the related boards are provided in CM implementation.

Configuration Status Accounting

This activity covers all the reporting tasks that aim to provide dedicated reporting about the configuration information and the CM activities. The reports can address a wide range of content, for various purposes: internal reports for the organization and for the internal stakeholders, external reports as part of the deliveries planned in the contract, certificates and conformity reports, etc.

Elaboration

Configuration Management is involved in the management and control of artifacts produced and modified throughout the system life cycle. It is therefore linked to system analysis, system detailed design definition, system realization, system deployment and use, system operation and product and service life management.

CM must also work in conjunction with other technical management processes. These include project planning, project assessment and control, decision management, risk management, information management, and measurement.

This includes CM application to the artifacts of all the other management processes (plans, analyses, reports, statuses, etc.).

Practical Considerations

Refer to CM implementation for detailed considerations about the practical implementation of CM according to context.

Key pitfalls and good practices related to systems engineering CM are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing CM are in Table 1.

Table 1. Configuration Management Pitfalls. (SEBoK Original)

Name	Description
Shallow Visibility	• Not involving all affected disciplines in the change control process.
Poor Tailoring	• Inadequate CM tailoring to adapt to the project scale, number of subsystems, etc.
Limited CM Perspective	• Not considering and integrating the CM processes of all contributing organizations including COTS vendors and subcontractors.
Insufficient CM Awareness	• Insufficient awareness and training of all affected disciplines
Lack of CM Plan	• Not organizing the CM activities and provision of the adequate resources and means
Insufficient CM Checks	• Not verifying CM implementation regularly

Good Practices

Some good practices gathered from the references are provided in Table 2 below.

Table 2. Configuration Management Good Practices. (SEBoK Original)

Name	Description
Cross-Functional CM	• Implement cross-functional communication and CM processes for software, hardware, firmware, data, or other types of items as appropriate.
Full Lifecycle Perspective	• Plan for integrated CM through the life cycle. Do not assume that it will occur as part of the program without explicit planning.
CM Planning	• Processes are documented in a single, comprehensive CM plan early in the project. The plan should be a (systems) CM plan. • Include tools selected and used.
Requirements Traceability	• Initiate requirements traceability at the start of the CM activity.
CCB Hierarchy	• Use a hierarchy of configuration control boards commensurate with the program elements.
Consistent Identification	• Software CI and hardware CI use consistent identification schemes.
CM Automation	• Configuration status accounting should be as automated as possible.

References

Works Cited

ISO/IEC/IEEE 15288:2023, Second Edition. *Systems and software engineering — System life cycle processes* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023 ^[1]

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.

ISO 10007, Third Edition. *Quality Management Systems – Guidelines for Configuration Management*. International Organization for Standardization (ISO), ISO 10007:2017 ^[1]

Blanchard, B.S. and W J. Fabrycky. 2005. *Systems Engineering and Analysis*, 4th ed. Prentice-hall international series in industrial and systems engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.

IEEE SWEBOK Version 4 2024. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>

SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Primary References

ISO/IEC/IEEE 15288:2023, Second Edition. *Systems and software engineering — System life cycle processes* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023 ^[1]

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.

ISO 10007, Third Edition. *Quality Management Systems – Guidelines for Configuration Management*. International Organization for Standardization (ISO), ISO 10007:2017 ^[1]

ANSI/GEIA. 2019. *Configuration Management Standard Implementation Guide*. Arlington, VA, USA: American National Standards Institute/Government Electronics & Information Technology Association, EIA649C ^[2].

GEIA. 2022. *Data Management*. Arlington, VA, USA: Government Electronics & Information Technology Association. GEIA-859B ^[3].

Additional References

ISO/IEC/IEEE 16236:2019. *Systems and Software Engineering - Life Cycle Processes - Project Management*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 16236:2019 ^[4] (Edition 2)

ISO/IEC/IEEE 15289:2019. *Systems and software engineering — Content of life-cycle information items*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 15289:2019 ^[5] (Edition 4)

ISO/IEC/IEEE 24748-1:2024. *Systems and software engineering — Life cycle management - Part 1: Guidelines for life cycle management*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 24748-1:2024 ^[2] (Edition 2)

ISO/IEC/IEEE 24748-2 *Systems and software engineering — Life Cycle Management – Part 2: Guidelines for the Application of ISO/IEC/IEEE 15288*

ISO/IEC/IEEE 24748-2:2024. *Systems and software engineering — Life cycle management - Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 24748-2:2024 (Edition 2)

ECSS-M-ST-40C Rev.1. 2009. *Space project management - Configuration and information management*. Noordwijk, The Netherlands: European Cooperation for Space Standardization (ECSS)

ANSI/EEIA. 2020. *Configuration Management Requirements for Defense Contracts EIA649_1A*. Arlington, VA, USA: Government Electronics & Information Technology Association - EIA649_1A ^[6]

ISO/IEC/IEEE 24748-8:2019 - *Systems and software engineering — Life cycle management Part 8: Technical reviews and audits on defense programs* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 24748-8:2019 ^[2] (Edition 1)

References

- [1] <https://www.iso.org/standard/70400.html#lifecycle>
- [2] <https://www.sae.org/standards/content/eia649c/>
- [3] <https://www.sae.org/standards/content/geia859b/>
- [4] <https://www.iso.org/standard/75276.html>
- [5] <https://www.iso.org/standard/74909.html>
- [6] https://www.sae.org/standards/content/eia649_1a/

Configuration Baselines

- Lead Authors:
- John Metcalf, Philip Hallenbeck, and Sandrine Gonthier
- Contributing Author:
- Garry Roedler

In systems engineering, a configuration baseline is a formally approved snapshot of a system's attributes at a specific point in its development. These baselines—such as the functional, allocated, and product baselines—serve as reference points that guide and control subsequent changes to the system. By establishing baselines, teams can ensure consistency, manage complexity, and maintain alignment with requirements throughout the system's life cycle. Configuration baselines are an important concept in the overall disciplines of Configuration Management.

Fundamental Concepts

Process Overview

The information of a Configuration item are assembled into Configuration baselines at certain points after or within a life cycle stage, as a set of consistent and immutable information that serve as starting points for further changes. Many types of baselines could be defined depending on the complexity of the system and desire to manage certain types of configuration changes at different levels of the organization. The differences between the information item in each baseline could be by information item type (specifications or descriptions), by information item associated with configuration items at different levels of the system hierarchy, or both.

The next subsection presents three main Configuration Management baselines that are focused on specifications and definition characteristics of the system: Functional Configuration Baseline (FBL), Allocated Configuration Baseline (ABL), and Product Configuration Baseline (PBL). These three formal baselines contain different specifications and description information. They are usually generated in order and aligned with life cycle stage events. They are present in most norms and standards, with some variation in their names, definitions, and scope.

Each of these baselines generally have specific related 'configuration documentation': Functional Configuration Documentation (FCD), Allocated Configuration Documentation (ACD), and Product Configuration Documentation (PCD). Configuration baselines and Configuration documentation are crucial in managing the complexity of systems and products, ensuring that all components and configurations are well-documented and understood throughout the lifecycle of the project or product.

Other predefined baselines may exist for the CM of the system, either requested by the applicable normative context, defined in the Configuration Management Plan for specific project's needs, etc. For example, Mission Objective Baseline which focuses on the purpose, constraints, environment and expected capabilities of the system as planned in [ECSS-M-ST-40C].

Finally, it is important to note that the content of the baselines or related configuration documentation is not necessarily limited to physical documents; the information can also include digital artifacts, such as models,

databases, files, and more.

Functional, Allocated and Product baselines and documentation

The following provides definitions of the FBL, ABL and PBL baselines and FCD, ACD and PCD, whose detailed description may differ through various standards.

- **Functional baseline** represents the approved originating set of performance requirements for the system (functional, interoperability, and interface characteristics) and the verification required to demonstrate the achievement of those specified characteristics.
- **Functional Configuration Documentation:** includes information on the functional requirements, interface descriptions, and any changes made to the system over time. It serves as a reference for understanding the system's configuration and for evaluating changes or updates to its functionality. **Comparison:** the functional baseline acts as a foundational reference point, functional configuration documentation may evolve throughout the life cycle stages as design details are finalized and changes are documented.
- **Allocated baseline** represents the "allocation" of requirements to the major elements of the system (lower levels Configuration Items). It is the reference point for the functional and performance requirements that have been assigned. It defines what is expected from each component in terms of functionality and performance.
- **Allocated Configuration Documentation:** it details the specific configurations of the individual system elements that have been derived from the overall system requirements outlined in the allocated baseline. It may include design specifications, performance criteria, and how each system element is configured to meet its designated functions. **Comparison:** The allocated baseline is a conceptual framework or reference point, while the allocated configuration documentation is a detailed artifact that documents the specifics of individual system elements configurations.
- **Product baseline** gathers the approved definition and description information resulting from the system definition activities (e.g., system design definition).
- **Product Configuration Documentation:** This documentation provides comprehensive details about the configuration of a specific product, encompassing its physical and functional characteristics. It typically includes information about the product's design, elements, versions, and any configuration settings that are relevant. It describes how the product should be built and what specifications must be met. **Comparison:** The product baseline is an established reference for the overall product's characteristics, while the product configuration documentation contains the specific details needed to implement those characteristics.

Figure 1 hereunder provides a schema of the interconnections between the FBL, ABL and PBL baselines and associated documentation. It may vary according to the normative standard to be applied. In this figure, the terms 'Product Configuration Information' and 'Product Definition Information' are consistent with SAE-EIA 649C. The organizations may choose other terms.

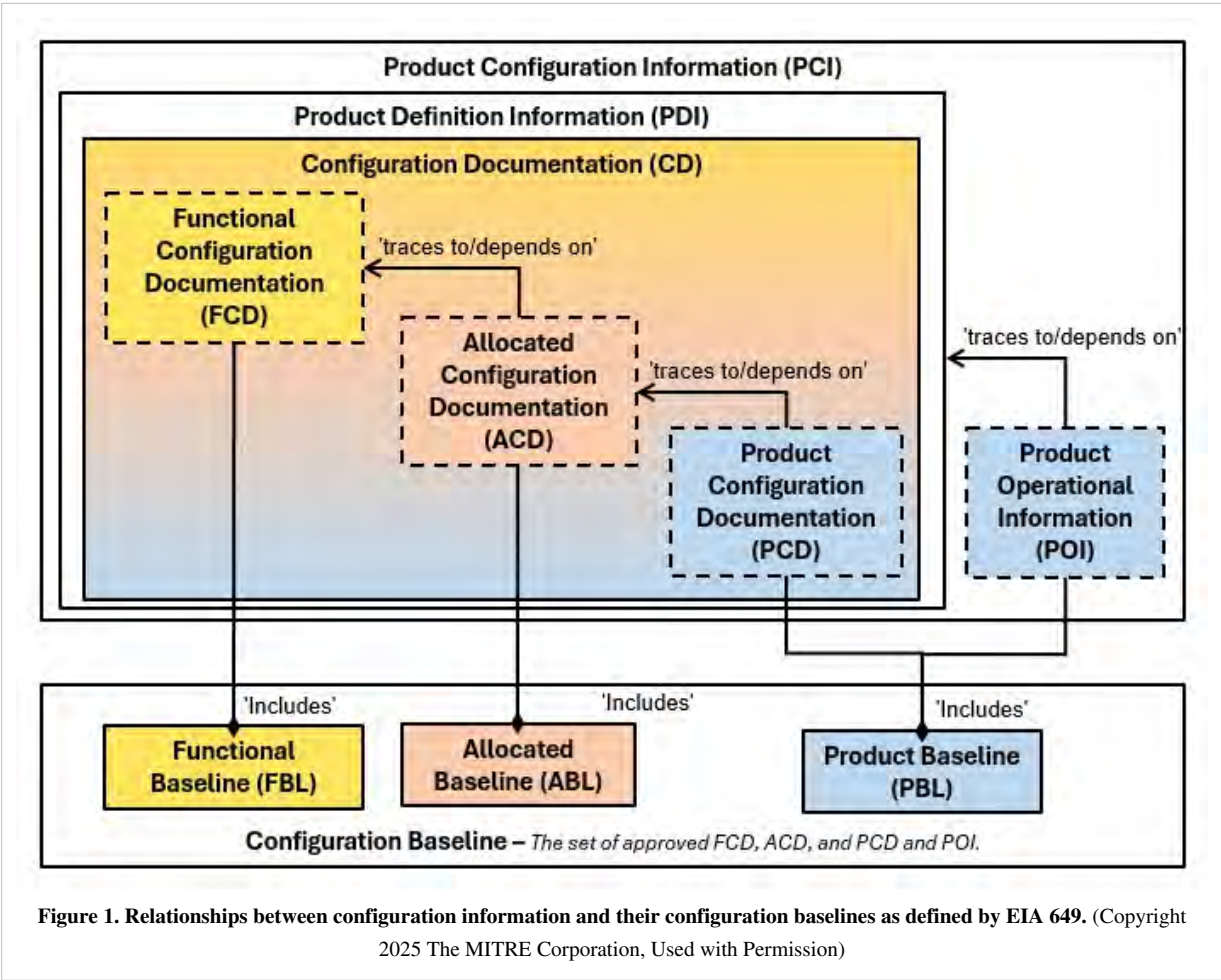
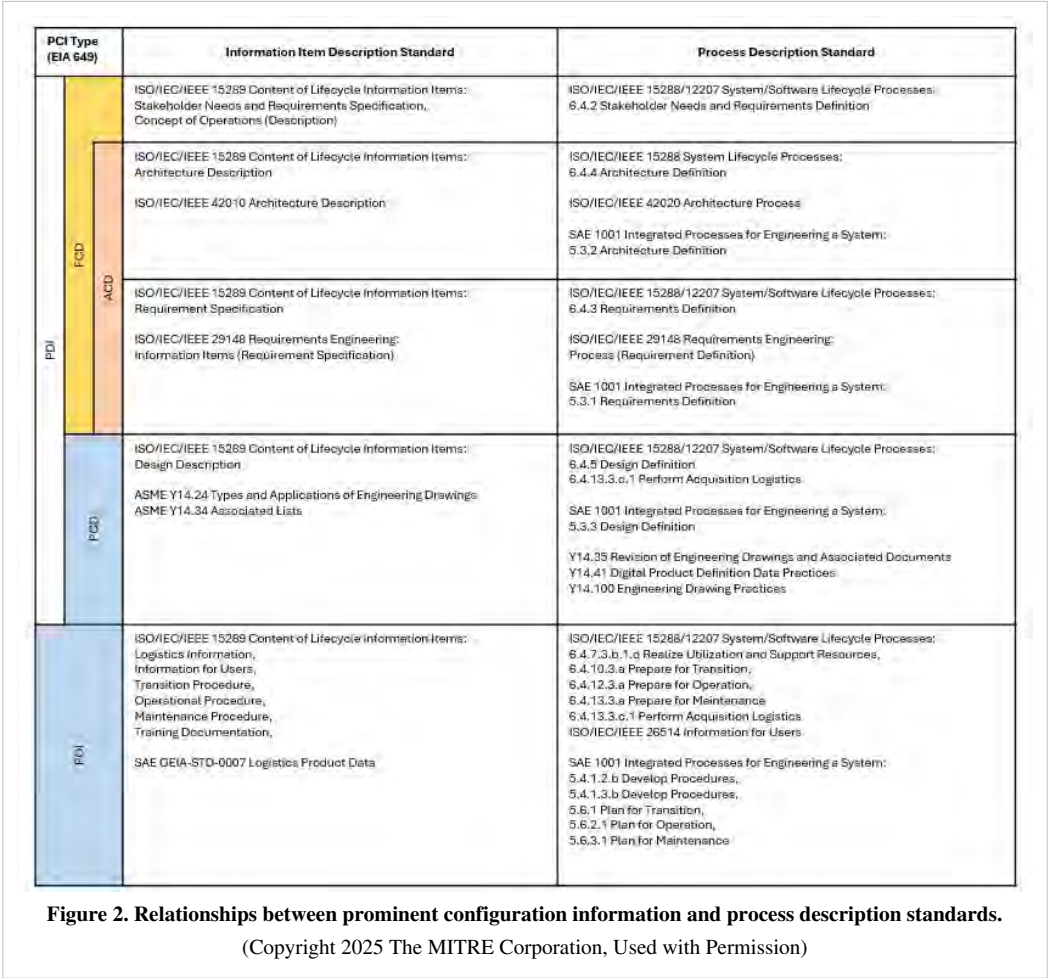


Figure 2 provides a description of the process and information standards related to product configuration information as illustrated in figure 1.



Elaboration

Configuration Management baselines are related to the artifacts produced and modified throughout the system life cycle. Baselining activities are therefore interfacing with system analysis, system detailed design definition, system realization, system deployment and use, system operation and product and service life management.

Practical Considerations

Pitfalls

Some of the key pitfalls encountered in Configuration baselines are in Table 1.

Table 1. Configuration baselines Pitfalls. (SEBoK Original)

Name	Description
Shallow Baselines Visibility	Not involving all affected disciplines in baselines preparation and validation
Baselines needs are not tailored adequately	Insufficient CM tailoring to adapt the baselines needs to the specific context of the project and to the system
Lack of Baselines planning and management	Not planning baselines at the adequate milestones, not identifying rules for their validation and release
Baselines not used by the stakeholders	Insufficient awareness of all stakeholders about the consideration of baselines as key references across the whole project
Missing Baselines checks	Not verifying CM baselines regularly

Good Practices

Some good practices gathered from the references are provided in Table 2 below.

Table 2. Configuration baselines Good Practices. (SEBoK Original)

Name	Description
Cross-Functional Baselines	Define baselines across the whole system, for software, hardware, firmware, data, or other types of items as appropriate
Baselines over the full lifecycle	Define the baselines needed throughout the entire life cycle
Baselines Planning	CM plan describes the needs in terms of baselines
Baselines tooling	CM tools provide the adequate means to create, validate, and disseminate baselines
Baselines immutability	CM baselines should be immutable reference points that serve as starting points for further change

References

Works Cited

ISO/IEC/IEEE 15288:2023, Second Edition. *Systems and software engineering — System life cycle processes* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023 ^[1]

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.

ISO 10007, Third Edition. *Quality Management Systems – Guidelines for Configuration Management*. International Organization for Standardization (ISO), ISO 10007:2017 ^[1]

Blanchard, B.S. and W J. Fabrycky. 2005. *Systems Engineering and Analysis*, 4th ed. Prentice-hall international series in industrial and systems engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.

IEEE SWEBOK Version 4 2024. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>

SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Primary References

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.

ISO 10007, Third Edition. *Quality Management Systems – Guidelines for Configuration Management*. International Organization for Standardization (ISO), ISO 10007:2017 ^[1]

ISO/IEC/IEEE 15288:2023, Second Edition. *Systems and software engineering — System life cycle processes* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023 ^[1]

ANSI/GEIA. 2019. *Configuration Management Standard EIA649C*. Arlington, VA, USA: American National Standards Institute/Government Electronics & Information Technology Association, EIA649C ^[2].

ANSI/GEIA. 2016. *Configuration Management Standard Implementation Guide*. Arlington, VA, USA: American National Standards Institute/Government Electronics & Information Technology Association, EIA649A ^[1].

GEIA. 2022. *Data Management*. Arlington, VA, USA: Government Electronics & Information Technology Association. GEIA-859B ^[3].

Additional References

ISO/IEC/IEEE 16236:2019. *Systems and Software Engineering - Life Cycle Processes - Project Management*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 16236:2019 ^[4] (Edition 2)

ISO/IEC/IEEE 15289:2019. *Systems and software engineering — Content of life-cycle information items*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 15289:2019 ^[5] (Edition 4)

ISO/IEC/IEEE 24748-1:2024. *Systems and software engineering — Life cycle management - Part 1: Guidelines for life cycle management*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 24748-1:2024 ^[2] (Edition 2)

ISO/IEC/IEEE 24748-2 *Systems and software engineering — Life Cycle Management – Part 2: Guidelines for the Application of ISO/IEC/IEEE 15288*

ISO/IEC/IEEE 24748-2:2024. *Systems and software engineering — Life cycle management - Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 24748-2:2024 (Edition 2)

ECSS-M-ST-40C Rev.1. 2009. *Space project management - Configuration and information management*. Noordwijk, The Netherlands: European Cooperation for Space Standardization (ECSS)

ANSI/EEIA. 2020. *Configuration Management Requirements for Defense Contracts EIA649_1A*. Arlington, VA, USA: Government Electronics & Information Technology Association - EIA649_1A ^[6]

References

[1] <https://www.sae.org/standards/content/geiahb649a/>

Configuration Management Implementation

- Lead Authors:
- John Metcalf, Philip Hallenbeck, and Sandrine Gonthier
- Contributing Author:
- Garry Roedler

Implementing Configuration Management (CM) is about putting theory into practice—establishing the processes, tools, and responsibilities needed to manage changes to a system effectively. This involves planning how CM will function within an organization, defining roles, selecting appropriate tools, and integrating CM activities into the broader systems engineering workflow. By thoughtfully implementing CM, organizations can ensure that system changes are tracked, controlled, and communicated, maintaining the integrity and consistency of the system throughout its life cycle. This article is a compliment to Configuration Management.

Process Overview

As described Configuration Management, CM is a process that relies on appropriate practices, methods, and controls to provide the means to establish and maintain a source of authoritative data on system configuration.

This article provides guidance on how to plan the implementation of CM activities with a focus on explaining what elements to consider adapting CM to the dedicated context of the system to be addressed.

Principles and Concepts

Essential Areas to Evaluate for Adjustments

Product Information to be Considered in CM

The Configuration information should encompass product operational information and should not be limited to product definition information (see recommendation from the EIA-649C standard paragraph 5.2.2). Starting from the initial source of information provided by the document describing the operational conditions of the product, information about the product's physical environment of use, and the type of support to be provided in case of in-service support part of the contract, need to be considered.

Here are some examples of valuable product operation information: scope of use, user profiles, quantitative and qualitative requirements, etc.

CM Boards

The organization of CM activities is heavily influenced by the scope of the CM activities to be considered, and by the organizational factors addressed below. In general, complicated and critical products are managed by larger, more comprehensive CM organizations. Conversely, large organizations similarly require more comprehensive CM organizations and processes to manage the workload of key personnel.

Some considerations include the following:

Single CCB for Small Organizations

Small organizations, or those managing relatively simple products, may decide to have a single CCB staffed and operated at the organizational level, controlling not only organizational level configurations, but those of its respective projects or products. This can be a useful cost-saving measure, but one whose benefits must be balanced against the time demands of CCB membership and necessary expertise—especially as products increase in number.

Additional Boards for Larger Organizations and/or for Complex Systems

Larger teams or organizations, or those managing products of significant complexity, may choose to institute a Technical Review Board (TRB) prior to the CCB.

The TRB's function is to ensure thoroughness and correctness of the information supporting each CCB decision. Its membership depends on the demands of the CM subject, but it typically includes the relevant representatives of the major disciplines involved in the project, and subject matter experts as per needed. The TRB should address in detail the investigations and the arbitrations (technical and non-technical ones) prior to CCB to prepare the synthesis of the proposed analysis with predefined solutions to be agreed on in CCB.

CM Augmentation

The CM team must adapt to the system's complexity; greater complexity requires more dedicated and structured CM resources to manage a volume of configuration information that is greater and more complex. It induces the need for a detailed and formalized implementation of CM tasks.

Additionally, the contribution of various stakeholders is essential in performing specific CM functions.

For instance, when evaluating the compliance of a complex system to its requirements in a Functional Configuration Audit (FCA) part of Configuration Verification and Audit activity, the assistance of test engineers and technical writers would be typically required. Status accounting activities on complex systems may require participation from domain experts per expertise area (such as mechanical engineers, radiation engineers, chemical engineers, ...) to ensure the correctness of the process and its artifacts. Systems engineers should ensure that these are accounted for in technical planning, estimation, and resourcing.

CM Training

In general, CM training should adhere to the principle of training to the specific standard of the organization and its processes. Hence, trainers should use generic process areas and principles as a foundation and introduction, only, to their program and its implementation. Important considerations in CM training include:

- Ensure that all organizational stakeholders are trained to a minimum level of awareness to support the Configuration Management (CM) initiative and help prevent interruptions to the CM process.
 - Check that all stakeholders take CM activities into account when scheduling meetings and other activities.
 - Make sure that CCB members and contributors understand the CCB workflow and necessary documents. It's especially important for contributors, like project systems and software engineers, to contribute to properly preparing CCB artifacts, as this avoids wasting time during CCB meetings.
 - Make sure that everyone knows the status accounting procedures and documents, along with those for verification and audits. These activities are often viewed as the most impactful by CM contributors in engineering, production, or support. Therefore, it's crucial to train engineering staff on CM audit procedures and their significance in verifying CM. Similarly, ensuring that all team members understand how to access status accounting documents and their relevance to their roles will help build support for the CM effort.
-

CM Organizational Legacy and Tailoring

A CM organization with a background in previous projects can leverage the legacy of earlier CM plans, tools, and standardized methods as a foundational reference for new projects. However, the Configuration Manager must ensure that the existing processes and infrastructure align with the specific needs of the current CM efforts. If adjustments are necessary, the practices must be appropriately tailored, and such tailoring should be justified.

Constraints and Guidance

Constraints affecting and guiding the CM process come from various sources.

There are a variety of sources for guidance on the development of a CM process. These include the ISO standards on system life cycle processes (refer to latest ISO/IEC/IEEE 15288 version) and configuration management guidelines (refer to latest ISO 10007 version), as well as the *Guide to The Software Engineering Body of Knowledge* (SWEBOK) (2024), and the CMMI for Development V1.3 (SEI 2010).

Policies, procedures, and standards set forth at corporate or other organizational levels might influence or constrain the design and implementation of the CM process.

Guidance should also come from the CM purpose that may differ according to the organizations and/or to the projects, which has a direct influence on the CM effort:

- lifecycle phases to be considered, (operational and maintenance phases included or not), safety, cyber-security, environmental scope included or not,
- specific acquirer processes to comply with
- specific interfaces with authorities to consider,
- agreement(s) with acquirers or suppliers containing provisions affecting the CM process
- tools and associated practices
- other processes used in system development in interface with CM process
- etc.

In all cases, the balance between CM scope and CM costs needs to be addressed.

Highly rigorous CM implementation implies activities, resources and planning, that need to be adjusted to the correct level of control needed for each product. Highly critical products with safety constraints will, for example, need potentially a higher level of Configuration information to be managed and a higher number of CM verifications. The increase of control induces an increase of the cost of CM activities. However, for any product, the dedicated normative expectations of the context will drive the minimum set of CM requirements to be applied.

Organizational Issues

Successful CM planning, management, and implementation requires an understanding of the organizational context for the design and implementation of the CM process and why constraints are placed upon it.

To plan a CM process for a project, it is necessary to understand the organizational context and the relationships among the organizational elements. CM interacts with other organizational elements, which may be structured in various ways. Although the responsibility for performing certain CM tasks might be assigned to other parts of the organization, the overall responsibility for CM often rests with a distinct organizational element or designated individuals that constitute the CM team (Bourque and Fairley 2014).

The following are some examples of elements to be considered for a dedicated project:

- size of the organization,
 - budgets,
 - schedules,
 - contract requirements for CM,
 - approval authority for the configuration,
-

- access and control of CM information, IT-security constraints,
- certification requirements,
- internal context: enterprise management system, processes, and supporting systems to achieve the CM activities,
- external context: Acquirer/Supplier CM interfaces with a contracted relationship to support external trusted interactions,
- IT infrastructure to provide an adequate CM management system supporting the requested CM activities.

Some examples of organizational elements to be considered across the entire organization (this is not an exhaustive list) include:

- IT needs to support the various CM management systems of the projects of the organization
- IT support teams to provide training and support for the usage of the CM tools
- Cyber-resiliency expectations, with the support of the relevant subject-matter experts (SMEs)
- IT infrastructure requirements:
 - Capacity to support the CM activities for all the projects of the organization
 - **Provision and status of backups.** While products of low complexity or criticality may be able to suffer a discontinuous CM tooling with little impact, complicated or safety-critical systems often require continuous access to CM info. Hence, IT may need to include CM tools and repositories in a hot or warm backup scheme.
 - **Traceability and preservation of the history** of all Configuration Information are part of the requirements for CM.
 - **Information hosting and access.** CM information is often proprietary and competition-sensitive and sometimes classified in accordance with national security requirements. Hence, both hosting and access must often be controlled.

Measurement

To carry out certain CM activities, such as status accounting, verification and auditing, as well as to monitor and assess the effectiveness of CM processes, it is necessary to measure and collect data related to CM activities and system artifacts.

CM management system and automated report tools provide convenient access and facilitation of data collection.

Measurement of CM effectiveness would concern address the assessment of CM strategy deployment and maturity, for example:

- are the CM processes clearly documented for all life cycle phases, planned, deployed, known, applied?
- are the stakeholders trained and aware of their involvement in CM activities?
- are the tools providing the adequate capabilities to ensure CM activities?

Measurement of CM performance could address the quantitative aspects of CM activities implementation:

- Volume of Configuration items to manage
- Volume of configuration information
- Volume and costs of changes
- Volume of variances
- Duration of impact analysis to assess the impact of changes and variances
- Time to approve a change, time to implement an approved change
- Proportion of rework of impact analysis (not aiming to decision in CCB at first attempt)
- Number of discrepancies detected in CM verifications
- Number of attempts to get Functional and Physical Configuration Audits successfully passed.

It must be noted that most quantitative measures of CM performance would indeed address the performance of all the disciplines involved in the CM activity:

- the time to perform an impact analysis is mainly illustrating the time needed by the various expertise and engineering stakeholders to evaluate all the induced impacts of a change
- the time to implement a change may be driven by the manufacturing or purchasing constraints that could delay the implementation to include it in a dedicated batch

Tools

CM employs a variety of tools to support the process, that may depend on the nature of the system under CM and of the nature of the system artifacts. In a very general way, the Configuration Management system relies on tools such as Product Lifecycle Management and Application Lifecycle Management applications, and it can be seen as a federated environment. This environment could encompass various functionalities, among which

- information management
- tracking and change management
- version management
- release management
- baseline management
- breakdown structures management.

Practical Considerations

Pitfalls

Some of the key pitfalls encountered in implementing CM are in Table 1.

Table 1. Configuration Implementation Pitfalls. (SEBoK Original)

Name	Description
Poor Tailoring	• Inadequate CM tailoring to adapt to the project scale, number of subsystems, etc.
CM Boards organization not clarified	• Missing a visible and detailed organization of the boards over the whole system
Insufficient CM training	• Insufficient training of all affected disciplines
Insufficient CM tooling	• Lack of tools supporting the CM tasks

Good Practices

Some good practices gathered from the references are provided in Table 2 below.

Table 2. Configuration Implementation Good Practices. (SEBoK Original)

Name	Description
CM activities adjusted to the project needs	• CM activities consider the context of the project, the characteristics of the system, and the context of the organization
CM Boards organization	• The various boards have been set up according to the project needs, and their organization is adjusted along the lifecycle when needed
CM Resources	• The CM team organization is anticipated
CM Tools	• The Configuration Management system is set up to guarantee adequate support to the CM tasks, and to alleviate the administrative tasks.
CM Training	• CM training is available all through the project with adequate content for the involved stakeholders' priorities

References

Works Cited

- ISO/IEC/IEEE 15288:2023, Second Edition. *Systems and software engineering — System life cycle processes* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023 ^[1]
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.
- ISO 10007, Third Edition. *Quality Management Systems – Guidelines for Configuration Management*. International Organization for Standardization (ISO), ISO 10007:2017 ^[1]
- Blanchard, B.S. and W J. Fabrycky. 2005. *Systems Engineering and Analysis*, 4th ed. Prentice-hall international series in industrial and systems engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- IEEE SWEBOK Version 4 2024. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- ISO 10007, Third Edition. *Quality Management Systems – Guidelines for Configuration Management*. International Organization for Standardization (ISO), ISO 10007:2017 ^[1]
- SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Primary References

- ISO/IEC/IEEE 15288:2023, Second Edition. *Systems and software engineering — System life cycle processes* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023 ^[1]
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.
- ISO 10007, Third Edition. *Quality Management Systems – Guidelines for Configuration Management*. International Organization for Standardization (ISO), ISO 10007:2017 ^[1]
- ANSI/GEIA. 2016. *Configuration Management Standard Implementation Guide*. Arlington, VA, USA: American National Standards Institute/Government Electronics & Information Technology Association, EIA649A ^[1].
- ANSI/GEIA. 2019. *Configuration Management Standard Implementation Guide*. Arlington, VA, USA: American National Standards Institute/Government Electronics & Information Technology Association, EIA649C ^[2].
- GEIA. 2022. *Data Management*. Arlington, VA, USA: Government Electronics & Information Technology Association. GEIA-859B ^[3].

Additional References

- ISO/IEC/IEEE 16236:2019. *Systems and Software Engineering - Life Cycle Processes - Project Management*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 16236:2019 ^[4] (Edition 2)
- ISO/IEC/IEEE 15289:2019. *Systems and software engineering — Content of life-cycle information items*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 15289:2019 ^[5] (Edition 4)
- ISO/IEC/IEEE 24748-1:2024. *Systems and software engineering — Life cycle management - Part 1: Guidelines for life cycle management*. International Organization for Standardisation / International Electrotechnical Commissions /

Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 24748-1:2024 ^[2] (Edition 2)

ISO/IEC/IEEE 24748-2 *Systems and software engineering — Life Cycle Management – Part 2: Guidelines for the Application of ISO/IEC/IEEE 15288*

ISO/IEC/IEEE 24748-2:2024. *Systems and software engineering — Life cycle management - Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)* - International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers - ISO/IEC/IEEE 24748-2:2024 (Edition 2)

ECSS-M-ST-40C Rev.1. 2009. *Space project management - Configuration and information management*. Noordwijk, The Netherlands: European Cooperation for Space Standardization (ECSS)

ANSI/EEIA. 2020. *Configuration Management Requirements for Defense Contracts EIA649_1A*. Arlington, VA, USA: Government Electronics & Information Technology Association - EIA649_1A ^[6]

Information Management

- Lead Authors:
- John Metcalf, Philip Hallenbeck, and Sandrine Gonthier
- Contributing Authors:
- Garry Roedler and Andy Pickard

Information management (IM) helps teams access and manage information over its lifecycle, ensuring that the information is relevant, complete, verified, protected, distributed and managed. Simply put, IM is about maintaining a clear, accurate, and accessible picture of information to one or more audiences.

As stated in ISO/IEC/IEEE 15288 (6.3.6.1): 'The purpose of the Information Management process is to generate, obtain, confirm, transform, retain, retrieve, disseminate, and dispose of information to designated stakeholders'.

Fundamental Concepts

The INCOSE SE Handbook describes IM as the process that “plans, executes, and controls the provision of information to designated stakeholders that is unambiguous, complete, verifiable, consistent, traceable, and presentable”.

The IM process ensures that necessary information is created, stored, retained, protected, managed, and made easily available to those with a need and who are permitted access. It also ensures that information is disposed of when it is no longer relevant.

The scope of information to be considered is wide, addressing technical, project, organizational, integration, contractual, agreement and user information, and any other information deemed relevant for the organization, all of these designated as ‘information assets’ in this article.

Effective IM enables business objectives, as illustrated in Table 1 below:

Table 1: IM Enables Business Objectives (SEBoK Original)

Objective	How is IM enabling the objective
Operational efficiency	Through organizing secured processes for the collection, access and distribution of the information
Support analytics and decision-making	Through managing, in a controlled way, the tangible and intangible information assets and preserving them against uncontrolled modifications, threats and loss (cybersecurity dispositions against digital threats, resilient storage structures and pre-defined disaster recovery dispositions)
Quality and security	Through managing, in a controlled way, the tangible and intangible information assets and preserving them against uncontrolled modifications, threats and loss (cybersecurity dispositions against digital threats, resilient storage structures and pre-defined disaster recovery dispositions)
Competitiveness and innovation	Through providing access to the crucial information needed to make decisions and through the capability to extract meaningful insights from information analysis
Compliance	Through planning the adequate dispositions and access controls to comply with legal requirements, intellectual property and regulations applicable to information and data

From a business enterprise perspective, the systems engineer, as well as many other stakeholders involved in systems development, may be involved in several activities that support the management of information across the enterprise and its projects. It may include strategic planning, analyses of technology/business trends, development of applications, understanding operational disciplines, analytics supporting decision-making, resource control techniques, and assessment of organization structures.

Information Versus Data

Terms like artifact, asset, work product, and documentation are commonly encountered in system engineering projects and are often used interchangeably to refer to **information** and **data** without regard to their difference.

ISO/IEC/IEEE 15289 describes **data** as outputs of life cycle processes that can be individually recorded and managed, composable into a structure set and treated as a unit called a **record**, and part of information items (documentation). **Information items** (documentation) *“are produced and communicated for human use and contain formal elements (such as purpose, scope, and summary), intended to make them usable by their intended audience”*.

ISO/IEC/IEEE 15289 also describes the content for the general and specific information items that are inputs and outputs from life cycle processes. (Remark: U.S. DoD projects use MIL-STD-963C to describe the contents of information items).

Similarly, GEIA- STD-927B defines **data** as *“A representation of facts, concepts, or instructions in a formalized manner suitable for communication, interpretation, or processing by humans or by automatic means”* and **information** as *“Data organized in a form that can be interpreted by humans or computers. Note: these definitions are “frequently used to discriminate between uninterpreted measurements (data) and meanings derived from raw data through some form of analysis (information)”*.

Data Management Body of Knowledge (DMBOK) acknowledges that **data** and **information** can be differentiated in a similar way if it will better enable communication within an organization. An example of these relationships is shown in Figure 1. Conversely, GEIA-STD-859B defines data as “information that has been recorded in a form or format convenient to move or process”.

Figure 1 below shows examples of relationships between data and information items in a project. However, it should be noted that this figure only mentions a few examples of data and information items that may have a much broader scope in a project or organization.

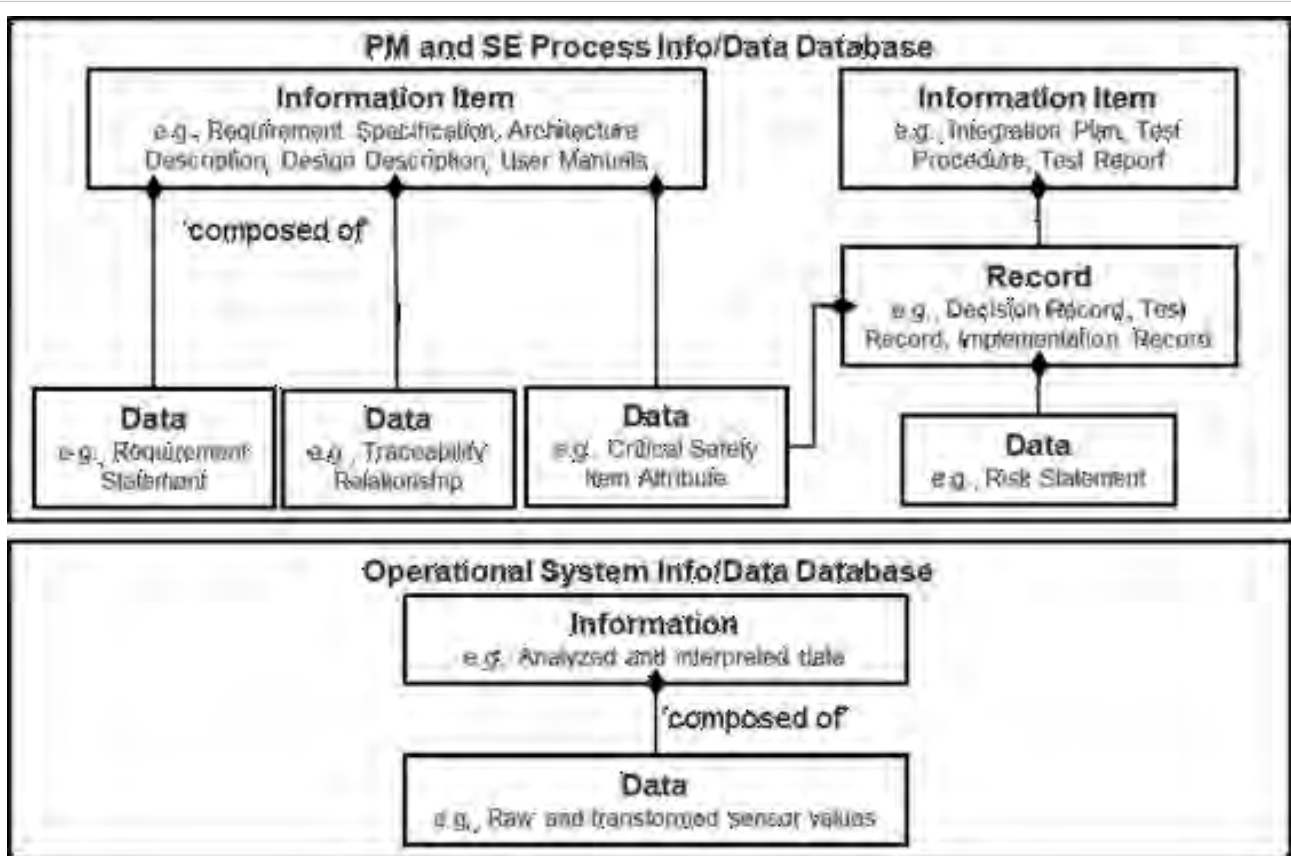


Figure 1 – Example of relationships between information and data encountered in System Engineering (SE), Project Management (PM), and System Operation. (SEBoK Original)

Using the above-mentioned standards, we can synthesize a convenient definition for information and data: **data are raw unstructured values, and information is structured data in context that has been analyzed and interpreted.**

Even though information and data are different, some organizations may choose not to differentiate between management of the two. The definitions, relationships, use, and management of PM, SE or Operational System information and data should be described in the project's management plans or system's configuration information, respectively.

Information Asset

An **information asset** is any piece or collection of information that must be managed independently.

The information items described by ISO/IEC/IEEE 15289 would be considered as information assets.

Information assets:

- can exist in many tangible or intangible forms in an organization,
- are either related to a specific system development project or held at enterprise level and made available to projects as required: e.g., Product Line Engineering information, generic procedures and processes,
- may be held in electronic format and/or in physical form: electronic or paper drawings or documentation, microfiche, photographic record, file, database...

Table 2 gives some examples of criteria that may be considered when choosing between formats:

Table 2: Electronic Format Versus Physical Format (SEBoK Original)

	Electronic Format	Physical Format
Pros	• Ease of access to various stakeholders • Searchability • Maintainability • Ease of multiple use enhanced by the capability to use links towards the information	• No need for access software • Long term storage • Robust to corruption and cyberthreats
Cons	• Potential for data corruption and cyberthreat-sensitive • Need to manage long term storage and archiving solutions • Need to manage the longevity of access software	• Less searchable • Access capabilities driven by physical locations • Long term preservation of the media depending on the physical support

Metadata

Metadata can be thought of as ‘data about data.’ It consists of ‘data’ that characterizes the information or data to be managed: for example, the title of the information, the author(s), the provenance, or the status of the information.

Metadata is particularly useful to organize the information database, to use the information and to search and navigate across the information database. It plays a key role in the organization of the governance of the information and data and contributes to quality and integrity objectives of the IM process.

Information Management versus Knowledge Management

According to SE HBK, **Knowledge Management** (KM) differs from Information Management (IM) as it is managed at Organization level and focused on providing the capability to re-apply existing knowledge.

However, KM is related to IM when it addresses explicit knowledge captured in information assets. Also, IM can be used to aid the objectives of KM by providing the means to manage the information regarding the knowledge assets.

Some examples of information assets, which may be considered as knowledge artifacts at the level of the organization, include:

- Technical content: leveraging and capitalizing on past engineering documentation, manufacturing processes, standard test procedures, design guides...
- Architectural and Design patterns
- Reusable content, i.e. product line information, generic analyses, previous documentation reusable pre-defined templates...
- Process information: process definition documentation, instructions, “How-to” guidance, validation information for analytical tools (e.g. aero-thermo-mechanical analysis tools).

How Does Information Management Relate to Configuration Management?

Information Management (IM) and Configuration Management (CM) are closely interconnected: whenever information assets correspond to identified Configuration Information related to systems or products under Configuration Management, those information assets should be subject to CM.

As noted in SEBoK Configuration Management article, ‘CM is concerned with the significance and applicability of information in relation to the specific system elements with which it is associated, referred to as configuration information’, while the IM process deals with the management of information itself over its own lifecycle: information generation, collection, validation, transformation, dissemination, and disposal.

The IM process is also complementary to CM to ensure the **traceability** of information in Configuration Management perspective.

Information Security Management

IM incorporates the fundamental need to manage the security of information assets.

Information security encompasses the mitigation of all kinds of breaches (digital and physical ones) that may expose personal, proprietary and industrial data, with associated consequences in terms of cost, image, and financial damages. It addresses a broader scope than cybersecurity, which focuses on electronic data, devices, and systems. It must consider all forms of storage, including information on physical formats and even knowledge held in the minds of stakeholders.

It must address the protection of intellectual property and trade secrets and comply with legal and regulatory requirements protecting privacy.

A simple approach for information security, inspired from the detailed methodology available in National Institute of Standards Special Publication 800-160v1r1, *Engineering Trustworthy Secure Systems*, should plan the 5 major steps explained below:

1 - Define Sensitive Information:

Determine the levels of Information sensitivity relative to pre-defined criteria, according to the nature of information to be managed and to the regulations and norms to comply with

Examples: information aiming to identify an individual, health information, national security information, or competition sensitive information.

2 - Define Data and Asset Classifications:

Define a global classification dictionary with clear classification rules, in accordance with the levels of sensitivity and considering the whole set of stakeholders along the system lifecycle; these rules should consider the variability that may exist in case of large systems involving many organizations and countries.

3 - Determine Information Security Controls, that may include the following:

- Information encryption, for both use-cases of information at rest (stored or archived) and under distribution, with considering the induced cost, processing, network and coordination overhead
- Digital signatures Distribution restrictions in accordance with the classification, enforced either by procedures or tools, including non-disclosure agreements (NDAs) between organizations, making them liable in case of disclosure of protected information
- User security education with regular refresher training and monitoring

4 - Implement Information Security Controls:

Ensure controls adapted to each sensitivity classification and to the phasing of the connections across organizations (either existing or under development), with the adequate planned tools and resources

5 – Monitor execution:

Ensure measures to provide insights and status about information security, taking advantage of existing cybersecurity dispositions when available.

Information Management System

A dedicated **Information Management system** is needed to enable and support the IM process and its activities.

The IM system must comply with the involved organizations' security policies along the full lifecycle of information assets.

A typical IMS includes capabilities to do or assist with the following functionalities:

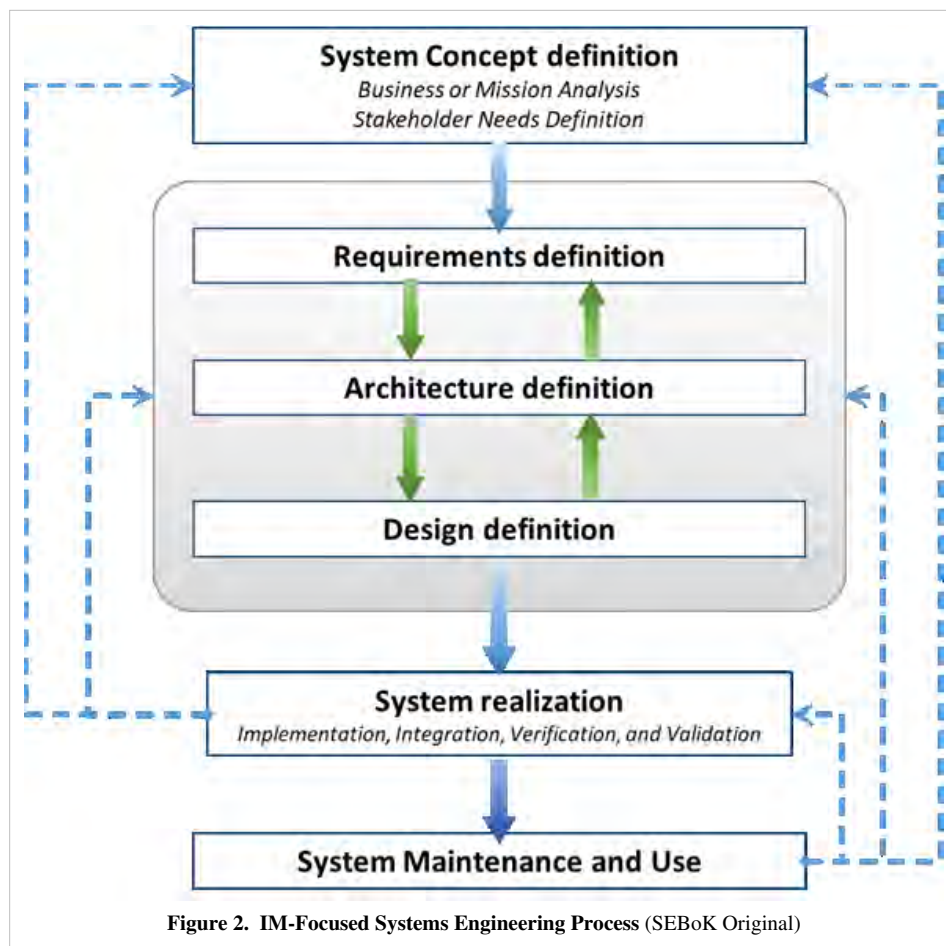
- Collection and organization of data and information
- Storage and retrieval of both raw data and information
- Analysis of data to transform it into information
- Creation of relevant reports/reporting of the results per appropriate context for decisions
- Protection of data and information
- Management of the access rights, responsibilities and authorities over the information database
- Compliance with laws and regulations.

Engineering the Information Management System

The Information Management system (IMS) aims to ensure the functions identified in the previous section while guaranteeing information availability, reliability, provenance traceability, security.

The IMS stakeholders' range may address a wide scope: project management, cybersecurity, systems engineering, specialty engineering disciplines, configuration management, quality management, industry, service functions, ... across one or several organizations.

The life cycle processes can be used to engineer the IMS as the system in charge of answering to the organizational need to manage its information and data. An example of approach is shown in Figure 2:



Each major step in figure 2 includes several considerations of particular importance to IM:

- **In Concept Definition** (Business or Mission Analysis and Stakeholder Needs Definition):
 - The involved stakeholders and organizations and their needs in terms of access controls
 - The needs for security boundaries
 - The needs for audits
 - The applicable regulations, laws, protection requirements

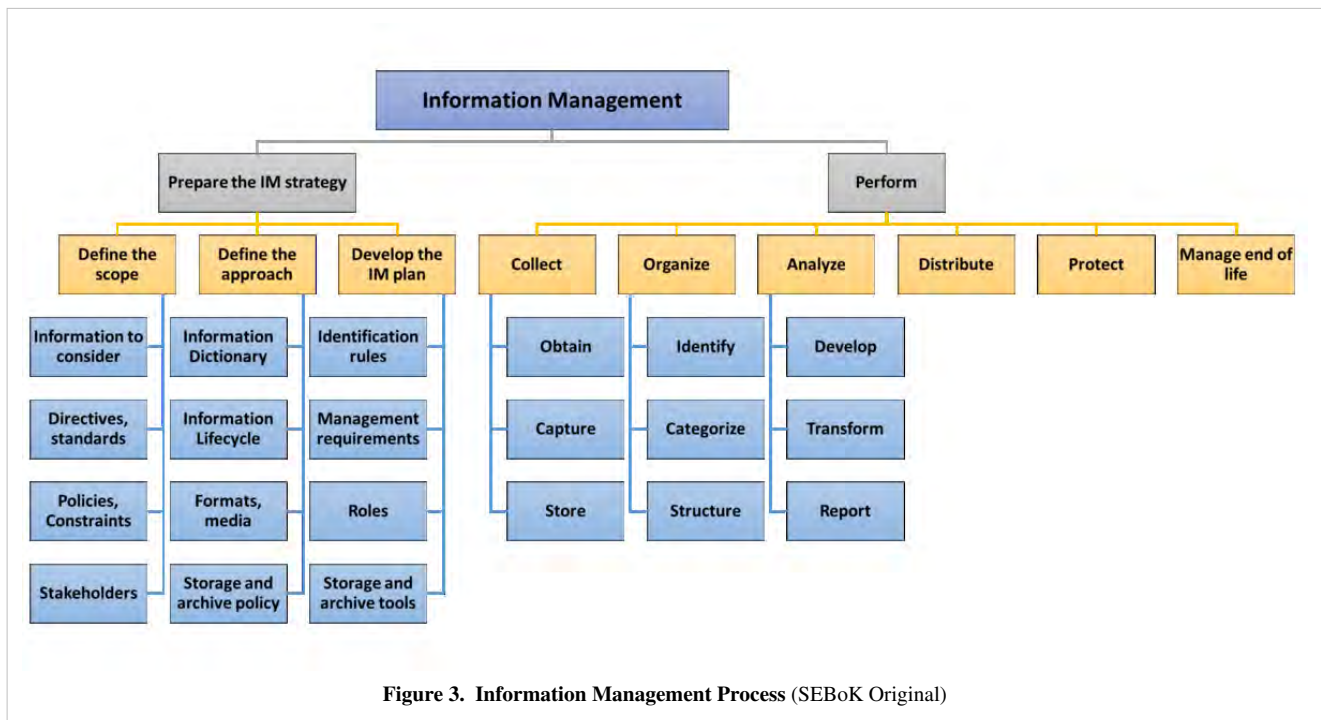
- The information lifecycle requirements per type of information
- **In Requirements Definition and System Architecture Definition:**
 - The existing structures and their constraints: repositories, cybersecurity architectures
 - The existing information and configuration management processes (including problem reporting and change management),
 - The existing syntax and semantics
 - The disaster recovery dispositions
- **In Design definition:**
 - Potential iterations when designing the IMS to consider changes to introduce new requirements for retrieval speed, archiving, metadata, data modelling, access rules
- **In Realization** (Implementation, Integration, Verification, and Validation):
 - Incremental integration, verification, and validation of the IM system to enable a stepwise adoption of new processes
 - Application of information and configuration management rules
 - Involvement of stakeholders in tests to value their feedback
 - Coordination with IT and cybersecurity staff especially during initial implementation steps
- **In Maintenance and Utilization:**
 - Needs to adapt the information lifecycle, always controlled by dedicated processes (information and/or configuration management workflows)
 - Regular disaster recovery and retrieval plan exercise
 - Performance and quality monitoring of a dedicated set of indicators
 - Stakeholders' feedback to maintain their involvement in the overall IMS quality.

Process Overview

The IM process relies on a defined approach that includes appropriate practices, methods, procedures, resources, and controls to establish and maintain a reliable source of Information.

A decomposition is proposed below, inspired by the SE HBK IM section. Examples of decomposition could also be found in ISO/IEC/IEEE 15288 Clause 6.3.6, SAE 1001 Clause 5.2.5, ISO/IEC/IEEE 24748-7 Clause 6.3.6.

The benefits of the IM process are maximized when all activities are planned and executed. These activities are not sequential. They may be applied concurrently and iteratively as needed throughout the information life cycle.



Here are some insights about the proposed sub-activities:

- **Prepare for information management** (preparing the information management strategy):
 - Define the scope:
 - Relevant information and valid sources to be considered (business processes which will provide information)
 - Directives, standards to be considered, including customers' requirements, industry standards...
 - Policies, constraints: organization internal policy and directives, external constraints (e.g. legal, environmental, geographical, regulations)
 - Stakeholders
 - Define the approach:
 - Information dictionary to consider the information to be managed (may be related to the project planning outputs in case of a system's information)
 - Information lifecycle:
 - Stages and status of the information asset, from initialization until obsolescence and disposal
 - Transition triggers between the lifecycle stages, defining workflows
 - Change process to control the evolutions of the information, and when relevant, the traceable incorporation in Configuration Management baselines
 - Authorized formats for capture, retention, transmission, retrieval, distribution
 - Requests for adequate repositories and tools
 - Storage and archive policy: retention duration, end of life disposal process, disaster recovery dispositions to prevent the loss of information in case of failure (back-up system, flawless recovery plan)
 - Develop the IM plan:
 - Identification rules: identification schema, mandatory attributes and metadata aiming to ease of search, rules for identifiers evolution
 - Management requirements
 - applicable to information capture, storage, validation, release, retrieval, and distribution

- problem reporting and change management, for both configuration information and information out of configuration scope
- control and security rules per type of Information, e.g. export control, access or distribution control, Intellectual Property, segregation of information between projects...
- auditing capabilities on the information database
- Roles
 - Responsibilities: roles providing, managing, consuming, distributing information (including external distribution to suppliers, customers, partners)
 - Authorities: roles in charge of IM policy and governance inside the organization, roles defining the IM system requirements...
- Storage and archive tools, consistent with the policy identified in the approach
- **Perform information management**
 - Collect the information: obtain from the valid sources, capture and store according to the IM plan
 - Organize the information: identify and categorize, structure the information to identify their relationships with the system's elements to which they are related
 - Analyze: develop or transform the information, report about the information to support decision-making
 - Distribute the information to designated stakeholders inside the organization and to the relevant external stakeholders in accordance with predefined processes
 - Protect the information: maintain integrity, security, privacy, in accordance with the applicable requirements
 - Manage end-of-life: control information obsolescence, identify obsolete information and assess whether it should be archived or disposed of; it aims to prevent the uncontrolled growth of the information asset database by retaining content that should be archived or destroyed in accordance with applicable policies or agreements.

Elaboration

Information Management is involved in the management and control of artifacts produced and modified throughout the system life cycle and across the organization.

The list of types of information assets to be managed and their cadence in the systems engineering and project lifecycle should be defined in the planning process and documented in the IM plan. For information assets related to configuration management, their cadence should be consistent with CM requirements.

The IM process is therefore linked to configuration management, project planning, project assessment and control, system analysis, system detailed design definition, system realization, system deployment and use, system operation and service life management.

Practical Considerations

Key pitfalls and good practices related to IM are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing IM are provided in Table 3:

Table 3. Information Management Pitfalls. (SEBoK Original)

Pitfall Name	Pitfall Description
No Information / Data Dictionary	Not defining an information/data dictionary for the organization or for the project may result in inconsistencies in identification conventions for information, which reduces the accuracy and completeness of searches for information and adding search time performance.
No Information identification strategy	Not specifying the naming convention for information and identification rules may result in inconsistent information identification, difficulties to retrieve information and impossibility to ensure consistency of the information across the whole project or organization
No Metadata	Not defining the relevant metadata per type of information or tagging them inconsistently may result in ineffective searches based on metadata which are ineffective and can overlook key information.
No predefined Information / data lifecycle	Missing definition of the information lifecycle defining creation and validation workflow, release mechanism, obsolescence management, leading to uncontrolled creation and validation of information, overaccumulation of obsolete data
No security rules	Not defining the security classification levels nor the associated security protection mechanisms induce quickly high levels of risks of information leaks that could engage the legal responsibility of the organization
No clear information communication rules	Not defining the conditions for reception and distribution of information leads to lacks in traceability. Information should be considered as key assets and their communication across the stakeholders need to be organized
Missing content-obsolescence management	Not checking the currency and relevance of the information content, and not tagging obsolescent and obsolete information results in outdated information being retained and used inadequately, and induces an overload in the storage capacity needs
Missing access-obsolescence management	Not checking the currency and relevance of the access rights could lead to maintaining accesses to information that should no longer be proposed and lead to protection breaches
Missing information repositories-obsolescence management	Not considering the information management system applications maintenance and survey could lead to the loss of access to crucial information. The IM plan should include the management of the nominal repositories' lifecycle, back-up solutions as well as a disaster recovery plan.
Inadequate back-up policy	Not ensuring that the information is backed up periodically in a back-up solution having the required retention durability and remains retrievable could lead to late discovery of information loss. The back-up policy should plan regular checks to ensure that information is not corrupted over time.

Good Practices

Some good practices gathered from the references are provided in Table 4:

Table 4. Information Management Good Practices. (SEBoK Original)

Good Practice Name	Good Practice Description
Guidance	- Refer to the applicable Data Management guide or standard or norm applicable to your domain or industrial context. - When applicable, the DAMA Guide to the Data Management Body of Knowledge provides an excellent, detailed overview of IM at both the project and enterprise level.
Information as an Asset	Recognize that information is a strategic asset for the organization and needs to be managed and protected.
Information Storage Capacity	Plan for the information storage capacity that is foreseen in your context, without forgetting to anticipate the increase of information volume based on the regular survey of the amount of information to be managed.
Effective Information Access	Ensure that the users are aware of the presence of the information and the relevant people access the information they need to know. Information that is not accessible or not used loses value that should come from the usage of the information.

Data Modeling	- Invest time and effort in designing data models that are consistent with the underlying structures and information needs of the organizations involved. - Cross-organization data modeling should focus on semantics (the exact meaning of terms or data items). Agreement on formats or table structures is insufficient: Terms often mean different things to different organizations. - Model for interoperability: Assume the data will be used in integrating with new, different systems later on. This requires a focus on identifiers, metadata, and provenance; and structures that can scale across organizational boundaries.
Quality Management	The cost impact of using poor quality information can be enormous. Be rigorous about managing the quality of information, including considering carefully the validation and release steps in the information lifecycle.
Information Repository Design	- The impact of managing information poorly can also be enormous (e.g., violating intellectual property or export control rules). - Make sure that the IM requirements are captured and implemented in the information repository, and that all users of the repository are aware of the rules that they need to follow and the penalties for infringement.
Uncontrolled modification prevention	Organize the information management life cycle and security to prevent from undesirable and/or untraced modification of the information.

References

Works Cited

ISO/IEC/IEEE. 2023. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2023.

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.

DAMA international. 2024, *DAMA International's Guide to the Data Management Body of Knowledge (DAMA-DMBOK2)*. Data Administration Management Association International. <https://technicpub.com/dmbok2/>

Ilana Hamilton. 2025, Article 'Information Security Vs. Cybersecurity: What's The Difference?'. <https://www.forbes.com/advisor/education/it-and-tech/information-security-vs-cybersecurity/>

Shirley M. Radack, National Institute of Standards and Technology. 2009, *The System Development Life Cycle (SDLC)*. <https://www.nist.gov/publications/system-development-life-cycle-sdlc>

ISO/IEC/IEEE15289. 2019. *Systems and software engineering - Content of life-cycle information items (documentation)*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE15289:2019

ANSI/GEIA-859B. 2022. *Data Management GEIA-859B*. Arlington, VA, USA: Government Electronics & Information Technology Association. GEIA-859B Data Management

ANSI/GEIAHB859. 2006. *Implementation Guide for Data Management GEIAHB859* - Arlington, VA, USA: Government Electronics & Information Technology Association. GEIAHB859

ANSI/GEIA-STD-927B. 2013. *Common Data Schema for Complex Systems GEIA-STD-927B*. Arlington, VA, USA: Government Electronics & Information Technology Association. GEIASTD927B

ANSI/GEIAHB927. 2007. *Implementation Guide for Common Data Schema for Complex Systems*. Arlington, VA, USA: Government Electronics & Information Technology Association. GEIAHB927

U.S. Department of Defense. 2019. *MIL-STD-963C Data Item Descriptions (DIDs)*. https://quicksearch.dla.mil/qsDocDetails.aspx?ident_number=202450

NIST Special Publication (SP) 800-160v1r1. 2022. *Engineering Trustworthy Secure Systems*, by Ron Ross, Mark Winstead, and Michael McEvelley. <https://doi.org/10.6028/NIST.SP.800-160v1r1>

Primary References

ISO/IEC/IEEE. 2023. *Systems and Software Engineering — System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2023.

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities* 5th Edition. Ed(s): D. Walden, T.M. Shortell, G.J. Roedler, B.A. Delljicado, O. Mornas, Y. Yew-Seng, D. Endler. San Diego, CA: International Council on Systems Engineering (INCOSE). Available at <https://www.incose.org/publications/se-handbook-v5>.

ISO/IEC 27001. 2022. *Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. <https://www.iso.org/standard/27001>

ISO/IEC/IEEE 15289. 2019. *Systems and software engineering — Content of life-cycle information items (documentation)*. International Organization for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE15289:2019

INCOSE. 2018. *Integrated Data as a Foundation of SE*. INCOSE Requirements working group. INCOSE-TP-2018-001-01.0.

ECSS-M-ST-40C Rev.1. 2009. *Space project management — Configuration and information management*. Noordwijk, The Netherlands: European Cooperation for Space Standardization (ECSS) - ECSS-M-ST-40C Rev.1

Additional References

None.

Quality Management

- Lead Authors:
- Quong Wang, Massood Towhidnejad, and David Olwell
- Contributing Authors:
- Dick Fairley and Garry Roedler

Whether a systems engineer delivers a product, a service, or an enterprise, the deliverable should meet the needs of the customer and be fit for use. Such a deliverable is said to be of high quality. The process to assure high quality is called quality management.

Overview

Over the past 80 years, a *quality movement* has emerged to enable organizations to produce high quality deliverables. This movement has gone through four stages:

1. **Acceptance Sampling** was developed to apply statistical tests to assist in the decision of whether or not to accept a lot of material based on a random sample of its content.
2. **Statistical Process Control (SPC)** was developed to determine if production processes were stable. Instead of necessarily measuring products, processes are measured instead. Processes that departed from a state of statistical control were far more likely to develop low quality deliverables.
3. **Design for Quality** focused on designing processes that were robust against causes of variation, reducing the likelihood that a process would go out of control, and accordingly reducing the monitoring requirements.
4. **Six sigma** methods are applied the tools and power of statistical thinking to improve other aspects of the organization.

Definitions

The American Society for Quality ^[1] provides the following definitions:

- Acceptance Sampling involves the inspection of a sample to decide whether to accept the entire lot. There are two types of sampling:
 - In attributes sampling, the presence or absence of a characteristic is noted in each of the units inspected.
 - In variables sampling, the numerical magnitude of a characteristic is measured and recorded for each inspected unit. This involves reference to a continuous scale of some kind.
- SPC is the application of statistical techniques to control a process. It is often used interchangeably with the term “statistical quality control.”
- Quality is a subjective term for which each person or sector has its own definition. In technical usage, quality can have two meanings:
 - The characteristics of a product or service that bear on its ability to satisfy stated or implied needs.
 - A product or service free of deficiencies. According to Joseph Juran, quality means “fitness for use.” According to Philip Crosby, it means “conformance to requirements.”
- Six Sigma is a method that provides organizations with tools to improve the capability of their business processes. This increase in performance and decrease in process variation leads to defect reduction and improvement in profits, employee morale, and quality of products or services. Six Sigma quality is a term generally used to indicate a process is well controlled ($\pm 6\sigma$ from the centerline in a control chart).

Quality Attributes

Quality attributes, also known as quality factors, quality characteristics, or non-functional requirements, are a set of system functional and non-functional requirements that are used to evaluate the system performance. There are a large number of system quality attributes identified in the literature (e.g. MSDN 2010, Barbacci et al. 1995). Depending on the type of the system being considered, some of these attributes are more prominent than others. Ideally, a system would be optimized for all the quality attributes that are important to the stakeholders, but this is an impossible task. Therefore, it is important to conduct a trade-off analysis to identify the relationship between the attributes and establish whether a change in one attribute would positively or negatively affect any other attributes. An example of such trade-off is shown in Table 1 below. (See SEBoK discussion on Systems Engineering and Quality Attributes for additional information on quality attributes.)

Table 1. Attribute Trade-Off. (SEBoK Original)

	Flexibility	Maintainability	Reliability
Flexibility		+	-
Maintainability	+		+
Reliability	-	+	

Finding the right set of quality attributes is the first step in quality control and management. In order to achieve high quality, quality must be measured, monitored, managed, and improved on. Therefore, in order to increase the overall system quality, it is necessary to:

- identify and prioritize the quality attributes
- identify the metrics that can be used for these attributes
- measure and monitor the attributes
- validate the measurements
- analyze the result of those measurements
- establish processes and procedures that result in improved system quality, based on the analysis.

Quality Attributes for Products

Quality attributes for a product focus on the conformance to the specifications for the product; frequently these are manufacturing specifications. Examples include physical characteristics (length, weight, finish, capacity, etc.) being inside a given tolerance range. The physical characteristics can be related to the function of the product or to aesthetic qualities.

A single product may have a vector of quality attributes of high dimension as well as an associated region in which the vector is expected to be. Often the quality is summarized by saying the item is "in compliance" (if the vector is in the acceptable region) or "defective" (if the vector is outside the acceptable region).

Quality Attributes for Services

Quality of services plays a major role in customer satisfaction, which is the measurement of the overall system quality. Services can be divided into two major categories: primary and secondary. The city public transportation system, the U.S. postal service, or the medical services provided by a hospital are all examples of primary services. Services that provide help to a customer are secondary services, which are typically referred to as a *customer service*. Identifying the appropriate quality attributes is critical in the quality management of services. Some examples of service quality attributes include: affordability, availability, dependability, efficiency, predictability, reliability, responsiveness, safety, security, usability, etc. Again, depending on the type of the service, some of these attributes are more prominent than the others.

For example, in the case of services that are provided by the hospital, one may potentially be more interested in the availability, reliability, and responsiveness than the security (typically hospitals are assumed to be safe) and the affordability (typically insurance covers the majority of the cost). Of course, if the patient does not have a good insurance coverage, then the importance of affordability will increase (de Knoning, 2006).

Quality Attributes for Enterprises

An enterprise typically refers to a large, complex set of interconnected entities that includes people, technologies, processes, financial, and physical elements. Clearly, a typical enterprise has a number of internal and external stakeholders, and as a result there are a large number of quality attributes that will define its quality. Identifying the right set of attributes is typically more challenging in such a complex system. An example of an enterprise is the air traffic management system that is mainly responsible for the safe and efficient operation of the civil aviation within a country or collection of countries. There are many stakeholders that are concerned about the overall quality of the system, some example of these stakeholders and some of the primary quality attributes that they are concerned with are identified in Table 2.

Table 2. Enterprise Stakeholders and their Quality Attributes. (SEBoK Original)

Stakeholders	Primary Quality Attributes
Passengers	Safety, affordability, and reliability
Airlines	Adaptability, efficiency, and profitability
Air Traffic Controller	Safety, reliability, and usability
Hardware & Software Developers	Reliability, fault tolerance, and maintainability
Government/Regulatory Agency	Safety, reliability, affordability, etc.

Measuring Quality Attributes

Quality cannot be achieved if it cannot be measured. The Measurement System Analysis (MSA) (Wheeler and Lynday 1989) is a set of measuring instruments that provide an adequate capability for a team to conduct appropriate measurements in order to monitor and control quality. The MSA is a collection of:

- **Tools** - measuring instruments, calibration, etc.
- **Processes** - testing and measuring methods, set of specifications, etc.
- **Procedures** - policies and procedures and methodologies that are defined by the company and/or regulatory agency
- **People** - personnel (managers, testers, analysis, etc.) who are involved in the measurement activities
- **Environment** - both environmental setting and physical setting that best simulate the operational environment and/or the best setting to get the most accurate measurements

Once the quality attributes are identified and prioritized, then the MSA supports the monitor and control of overall system quality.

Additional details about measurement are presented in the measurement article.

Quality Management Strategies

Acceptance Sampling

In acceptance sampling many examples of a product are presented for delivery. The consumer samples from the lot and each member of the sample is then categorized as either *acceptable* or *unacceptable* based on an attribute (attribute sampling) or measured against one or more metrics (variable sampling). Based on the measurements, an inference is made as to whether the lot meets the customer requirements.

There are four possible outcomes of the sampling of a lot, as shown in Table 3.

Table 3. Truth Table - Outcomes of Acceptance Sampling. (SEBoK Original)

	Lot Meets Requirement	Lot Fails Requirement
Sample Passes Test	No error	Consumer risk
Sample Fails Test	Producer risk	No error

A sample acceptance plan balances the risk of error between the producer and consumer. Detailed ANSI/ISO/ASQ standards describe how this allocation is performed (ANSI/ISO/ASQ A3534-2-1993: *Statistics—Vocabulary and Symbols—Statistical Quality Control*).

Statistical Process Control

SPC is a method that was invented by Walter A. Shewhart (1931) that adopts statistical thinking to monitor and control the behaviors and performances of a process. It involves using statistical analysis techniques as tools in appropriate ways, such as providing an estimate of the variation in the performance of a process, investigating the causes of this variation, and offering the engineer the means to recognize when the process is not performing as it should be based on the data. (Mary et al. 2006, 441). In this context, *performance* is measured by how well the process is performed.

The theory of quality management emphasizes managing processes by fact and maintaining systematic improvement. All product developments are a series of interconnected processes that have variation in their results. Understanding variation with SPC technology can help the process executors understand the facts of their processes and find the improvement opportunities from a systematic view.

Control charts are common tools in SPC. The control chart is also called the Shewhart 3-sigma chart. It consists of 3 limit lines: the center line, which is the mean of statistical samples, and the upper and lower control limit lines, which are calculated using the mean and standard deviation of statistical samples. The observed data points or their statistical values are drawn in the chart with time or other sequence orders. Upper and lower control limits indicate the thresholds at which the process output will be considered as *unlikely*. There are two sources of process variation. One is common cause variation, which is due to inherent interaction among process components. Another is assignable cause, which is due to events that are not part of the normal process. SPC stresses bringing a process into a state of statistical control, where only common cause variation exists, and keeping it in control. A control chart is used to distinguish between variation in a process resulting from common causes and assignable causes.

If the process is in control, and if standard assumptions are met, points will demonstrate a normal distribution around the control limit. Any points outside either of the limits, or in systematic patterns, imply a new source of variation would be introduced. A new variation means increased quality cost. Additional types of control charts exist, including: cumulative sum charts that detect small, persistent step change model departures and moving average charts, which use different possible weighting schemes to detect persistent changes (Hawkins and Olwell 1996).

Design for Quality

Variation in the inputs to a process usually results in variation in the outputs. Processes can be designed, however, to be robust against variation in the inputs. Response surface experimental design and analysis is the statistical technique that is used to assist in determining the sensitivity of the process to variations in the input. Such an approach was pioneered by Taguchi.

Six Sigma

Six sigma methodology (Pyzdek and Keller, 2009) is a set of tools to improve the quality of business processes; in particular, to improve performance and reduce variation. Six sigma methods were pioneered by Motorola and came into wide acceptance after they were championed by General Electric.

Problems resulting in variation are addressed by six sigma projects, which follow a five-stage process:

1. **Define** the problem, the stakeholders, and the goals.
2. **Measure** key aspects and collect relevant data.
3. **Analyze** the data to determine cause-effect relationships.
4. **Improve** the current process or **design** a new process.
5. **Control** the future state or **verify** the design.

These steps are known as **DMAIC** for existing processes and **DMADV** for new processes. A variant of six sigma is called lean six sigma wherein the emphasis is on improving or maintaining quality while driving out waste.

Standards

Primary standards for quality management are maintained by ISO, principally the ISO 9000 series ^[2]. The ISO standards provide requirements for the quality management systems of a wide range of enterprises, without specifying how the standards are to be met. The key requirement is that the system must be audited. ISO standards have world-wide acceptance.

In the United States, the Malcolm Baldrige National Quality Award presents up to three awards in six categories: manufacturing, service company, small business, education, health care, and nonprofit. The Baldrige Criteria ^[3] have become de facto standards for assessing the quality performance of organizations.

References

Works Cited

- Barbacci, M., M.H. Klein, T.A. Longstaff, and C.B. Weinstock. 1995. *Quality Attributes*. Pittsburgh, PA, USA: Software Engineering Institute/Carnegie Mellon University. CMU/SEI-95-TR-021.
- Chrissis, M.B., M. Konrad, and S. Shrum. 2006. *CMMI for Development: Guidelines for Process Integration and Product Improvement*, 2nd ed. Boston, MA, USA: Addison Wesley.
- Evans, J. and W. Lindsay. 2010. *Managing for Quality and Performance Excellence*. Florence, KY, USA: Cengage Southwestern.
- Juran, J.M. 1992. *Juran on Quality by Design: The New Steps for Planning Quality into Goods and Services*. New York, NY, USA: The Free Press.
- Koning, H. de, J.P.S. Verver, J. van den Heuvel, S. Bisgaard, R.J.M.M. Does. 2006. "Lean Six Sigma in Healthcare." *Journal for Healthcare Quality*. 28(2) pp 4-11.
- MSDN. 2010. "Chapter 16: Quality Attributes," in *Microsoft Application Architecture Guide*, 2nd Edition. Microsoft Software Developer Network, Microsoft Corporation. Accessed August 31, 2012. Available online at <https://msdn.microsoft.com/en-us/library/ff650706>.

- Moen, R.D., T.W. Nolan, and L.P. Provost. 1991. *Quality Improvement through Planned Experimentation*. New York, NY, USA: McGraw-Hill.
- Pyzdek, T. and P.A. Keller. 2009. *The Six Sigma Handbook*, 3rd ed. New York, NY: McGraw-Hill.
- Shewhart, W.A. 1931. *Economic Control of Manufactured Product*. New York, NY, USA: Van Nostrand.
- Wheeler, D.J. and R.W. Lyday. 1989. *Evaluating the Measurement Process*, 2nd ed. Knoxville, TN, USA: SPC Press.

Primary References

- Chrissis, M.B, M. Konrad, S. Shrum. 2011. *CMMI for Development: Guidelines for Process Integration and Product Improvement*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional.
- Evans, J. and W. Lindsay. 2010. *Managing for Quality and Performance Excellence*. Florence, KY, USA: Cengage Southwestern.
- Juran, J.M. 1992. *Juran on Quality by Design: The New Steps for Planning Quality into Goods and Services*. New York, NY, USA: The Free Press.
- Moen, R.D., T.W. Nolan, and L.P. Provost. 1991. *Quality Improvement through Planned Experimentation*. New York, NY, USA: McGraw-Hill.
- Pyzdek, T. and P.A. Keller. 2009. *The Six Sigma Handbook*, 3rd ed. New York, NY, USA: McGraw-Hill.
- Wheeler, D.J. and R.W. Lyday. 1989. *Evaluating the Measurement Process*, 2nd ed. Knoxville, TN, USA: SPC Press.

Additional References

- Hawkins, D. and D.H. Olwell. 1996. *Cumulative Sum Charts and Charting for Quality Improvement*. New York, NY, USA: Springer.

References

- [1] <https://asq.org/glossary/index.html>
- [2] https://www.iso.org/iso/iso_catalogue/management_and_leadership_standards/quality_management.htm
- [3] <https://www.nist.gov/baldrige/publications/criteria.cfm>
-

Measurement

- Lead Author:
- Garry Roedler
- Contributing Authors:
- Salvatore R. Bruno, Thomas McDermott, and David Endler

Measurement and the accompanying analysis are fundamental elements of systems engineering (SE) and technical management. SE measurement provides information relating to the products developed, services provided, and processes implemented to support effective management of the processes and to objectively evaluate product or service quality. Measurement supports realistic planning, provides insight into actual performance, and facilitates assessment of suitable actions (Roedler and Jones 2005, 1-65; Frenz et al. 2010).

Appropriate measures and indicators are essential inputs to trade-off analyses to balance cost, schedule, and technical objectives. Periodic analysis of the relationships between measurement results and review of the requirements and attributes of the system provides insights that help to identify issues early, when they can be resolved with less impact. Historical data, together with project or organizational context information, forms the basis for the predictive models and methods that should be used.

Fundamental Concepts

The discussion of measurement in this article is based on some fundamental concepts. Roedler et al. (2005, 1-65) states three key SE measurement concepts that are paraphrased here:

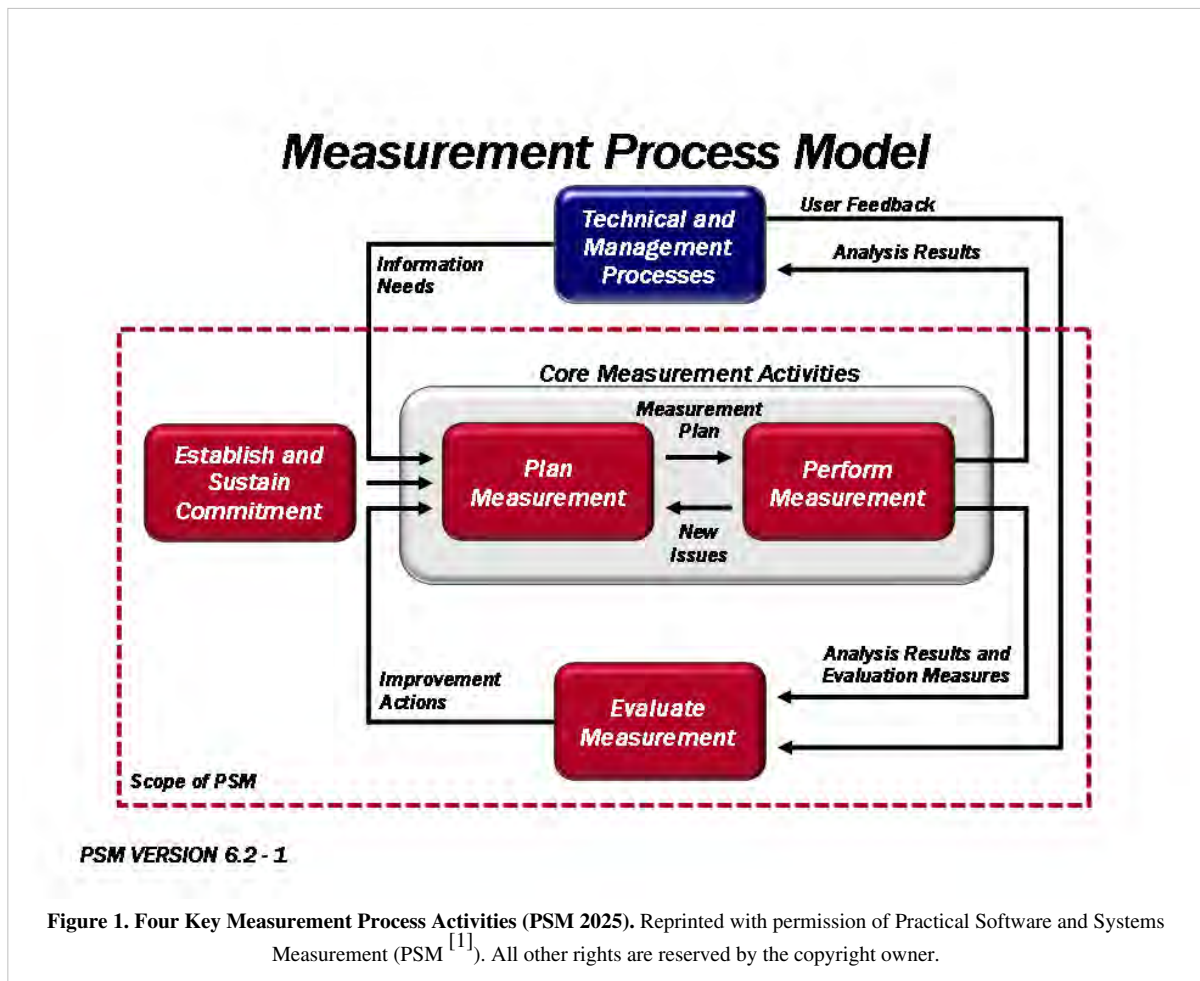
1. **SE measurement is a consistent but flexible process** tailored to the unique information needs and characteristics of a particular project or organization and revised as information needs change.
2. **Decision makers must understand what is being measured.** Key decision-makers must be able to connect *what is being measured* to *what they need to know* and *what decisions they need to make* as part of a closed-loop, feedback control process (Frenz et al. 2010).
3. **Measurement must be used to be effective.** Measurement of the Quality attribute of the technical objectives along with the cost and schedule provides the overall effectiveness of the project.

Measurement Process Overview

The measurement process as presented here consists of four activities from Practical Software and Systems Measurement (PSM 2025) and described in (ISO/IEC/IEEE 15939; McGarry et al. 2002):

1. Establish and sustain commitment
2. Plan measurement
3. Perform measurement
4. Evaluate measurement

This approach has been the basis for establishing a common process across the software and systems engineering communities. This measurement approach has been adopted by the Capability Maturity Model Integration (CMMI) measurement and analysis process area (SEI 2010), as well as by international systems and software engineering standards (ISO/IEC/IEEE 15939; ISO/IEC/IEEE 15288; ISO/IEC/IEEE 12207; SAE 2018). The International Council on Systems Engineering (INCOSE) Measurement Working Group has also adopted this measurement approach for several of their measurement assets, such as the INCOSE SE Measurement Primer (Frenz et al. 2010) and Technical Measurement Guide (Roedler and Jones 2005). This approach has provided a consistent treatment of measurement that allows the engineering community to communicate more effectively about measurement. The process is illustrated in Figure 1 from Roedler and Jones (2005) and McGarry et al. (2002).



Establish and Sustain Commitment

This activity focuses on establishing the resources, training, and tools to implement a measurement process and ensure that there is a management commitment to use the information that is produced. Refer to PSM (2025) and SPC (2011) for additional detail.

Plan Measurement

This activity focuses on defining measures that provide insight into project or organization information needs. This includes identifying what the decision-makers need to know and when they need to know it, relaying these information needs to those entities in a manner that can be measured, and identifying, prioritizing, selecting, and specifying measures based on project and organization processes (Jones 2003, 15-19). This activity also identifies the reporting format, forums, and target audience for the information provided by the measures.

Here are a few widely used approaches to identify the information needs and derive associated measures, where each can be focused on identifying measures that are needed for SE management:

- The PSM approach, which uses a set of information categories, measurable concepts, and prospective measures to aid the user in determining relevant information needs and the characteristics of those needs on which to focus (PSM 2025).
- The (GQM) approach, which identifies explicit measurement goals. Each goal is decomposed into several questions that help in the selection of measures that address the question and provide insight into the goal achievement (Park, Goethert, and Florac 1996).

- ISO/IEC/IEEE 15939:2017, clause 6.3.2 provides specific information on planning for measurement. In clause 6.3.2.4, it covers the identification of information needs and the selection and definition of measures (ISO/IEC/IEEE 15939).

The following are good sources for candidate measures that address information needs and measurable concepts/questions:

- PSM Web Site - Sample Measurement Specifications (PSM 2025)
- Table: Information Categories - Measurable Concepts - Prospective Measures (ICM Table) (PSM 2025)
- PSM Guide, Version 4.0, Chapters 3 and 5 (PSM 2000)
- SE Leading Indicators Guide, Version 2.0, Section 3 (Roedler et al. 2010)
- Technical Measurement Guide, Version 1.0, Section 10 (Roedler and Jones 2005, 1-65)
- Safety Measurement (PSM White Paper), Version 3.0, Section 3.4 (Murdoch 2006, 60)
- Security Measurement (PSM White Paper), Version 3.0, Section 7 (Murdoch 2006, 67)
- Measuring Systems Interoperability, Section 5 and Appendix C (Kasunic and Anderson 2004)
- Measurement for Process Improvement (PSM Technical Report), version 1.0, Appendix E (Statz 2005)
- New Opportunities for Architecture Measurement (Carson and Kohl 2012)
- System Development Performance Measurement (NDIA and PSM 2011)
- Continuous Iterative Development (CID) Measurement Specifications (PSM 2025)
- Digital Engineering Measurement Framework, Version 1.1 (DE Measurement Framework 2022)

The INCOSE *SE Measurement Primer* (Frenz et al. 2010) provides a list of attributes of a good measure with definitions for each attribute; these attributes include *relevance*, *completeness*, *timeliness*, *simplicity*, *cost effectiveness*, *repeatability*, and *accuracy*. Evaluating candidate measures against these attributes can help assure the selection of more effective measures.

The details of each measure need to be unambiguously defined and documented. Templates for the specification of measures and indicators are available on the PSM website (2025) and in Goethert and Sivi (2004).

Perform Measurement

This activity focuses on the collection and preparation of measurement data, measurement analysis, and the presentation of the results to inform decision makers. The preparation of the measurement data includes verification, normalization, and aggregation of the data, as applicable. Analysis includes estimation, feasibility analysis of plans, and performance analysis of actual data against plans.

The quality of the measurement results is dependent on the collection and preparation of valid, accurate, and unbiased data. Data verification, validation, preparation, and analysis techniques are discussed in PSM (2025) and SEI (2010). Per TL 9000, Measurements Handbook, *The analysis step should integrate quantitative measurement results and other qualitative project information, in order to provide managers the feedback needed for effective decision making* (QuEST Forum 2020). This provides richer information that gives the users the broader picture and puts the information in the appropriate context.

There is a significant body of guidance available on good ways to present quantitative information. Edward Tufte has several books focused on the visualization of information, including *The Visual Display of Quantitative Information* (Tufte 2001).

Other resources that contain further information pertaining to understanding and using measurement results include

- PSM (2025)
- ISO/IEC/IEEE 15939:2017, clauses 6.3.3.3 and 6.3.4
- Roedler and Jones (2005), sections 6.4, 7.2, and 7.3
- INCOSE Systems Engineering Handbook, 5th Edition, Section 2.3.4.7 (INCOSE 2023)
- SE Leading Indicators Guide, Version 2.0, Sections 3 & 4 (Roedler et al. 2010)

- System Development Performance Measurement (NDIA and PSM 2011)
- Continuous Iterative Development (CID) Measurement Specifications (PSM 2025)
- Digital Engineering Measurement Framework, Version 1.1 (DE Measurement Framework. 2022)

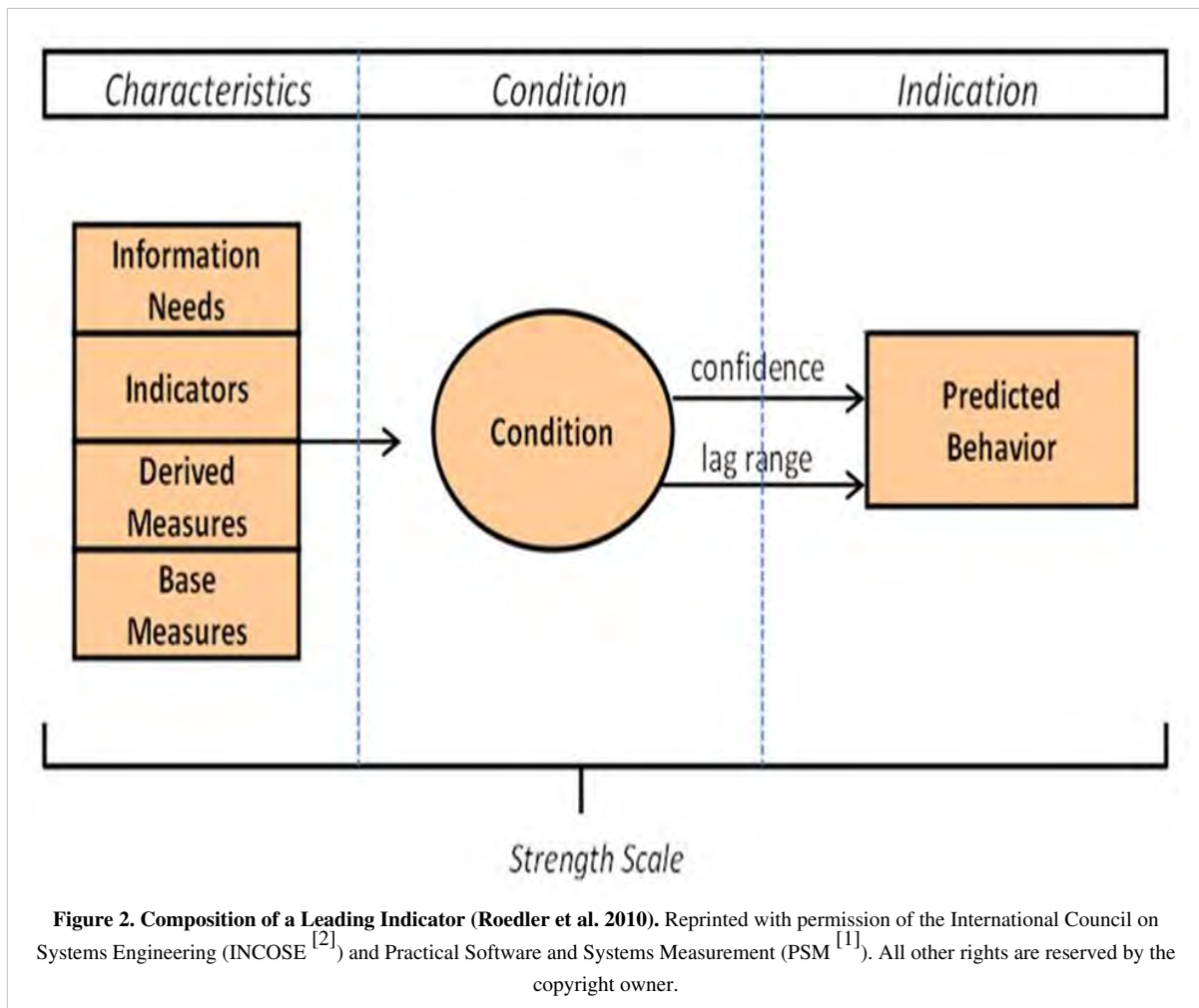
Evaluate Measurement

This activity involves the analysis of information that explains the periodic evaluation and improvement of the measurement process and specific measures. One objective is to ensure that the measures continue to align with the business goals and information needs, as well as provide useful insight. This activity should also evaluate the SE measurement activities, resources, and infrastructure to make sure it supports the needs of the project and organization. Refer to PSM (2025) and *Practical Software Measurement: Objective Information for Decision Makers* (McGarry et al. 2002) for additional detail.

Systems Engineering Leading Indicators

Leading indicators are aimed at providing predictive insight that pertains to an information need. A SE leading indicator is *a measure for evaluating the effectiveness of a how a specific activity is applied on a project in a manner that provides information about impacts that are likely to affect the system performance objectives* (Roedler et al. 2010). Leading indicators may be individual measures or collections of measures and associated analysis that provide future systems engineering performance insight throughout the life cycle of the system; they *support the effective management of systems engineering by providing visibility into expected project performance and potential future states* (Roedler et al. 2010).

As shown in Figure 2, a leading indicator is composed of characteristics, a condition, and a predicted behavior. The characteristics and conditions are analyzed on a periodic or as-needed basis to predict behavior within a given confidence level and within an accepted time range into the future. More information is also provided by Roedler et al. (2010).



Technical Measurement

Technical measurement is the set of measurement activities used to provide information about progress in the definition and development of the technical solution, ongoing assessment of the associated risks and issues, and the likelihood of meeting the critical objectives of the acquirer. This insight helps an engineer make better decisions throughout the life cycle of a system and increase the probability of delivering a technical solution that meets both the specified requirements and the mission needs. The insight is also used in trade-off decisions when performance is not within the thresholds or goals.

Technical measurement includes measures of effectiveness (MOEs), measures of performance (MOPs), and technical performance measures (TPMs) (Roedler and Jones 2005, 1-65). The relationships between these types of technical measures are shown in Figure 3 and explained in the reference for Figure 3. Using the measurement process described above, technical measurement can be planned early in the life cycle and then performed throughout the life cycle with increasing levels of fidelity as the technical solution is developed, facilitating predictive insight and preventive or corrective actions. More information about technical measurement can be found in the *NASA Systems Engineering Handbook, System Analysis, Design, Development: Concepts, Principles, and Practices*, and the *Systems Engineering Leading Indicators Guide* (NASA December 2007, 1-360, Section 6.7.2.2; Wasson 2006, Chapter 34; Roedler et al. 2010).

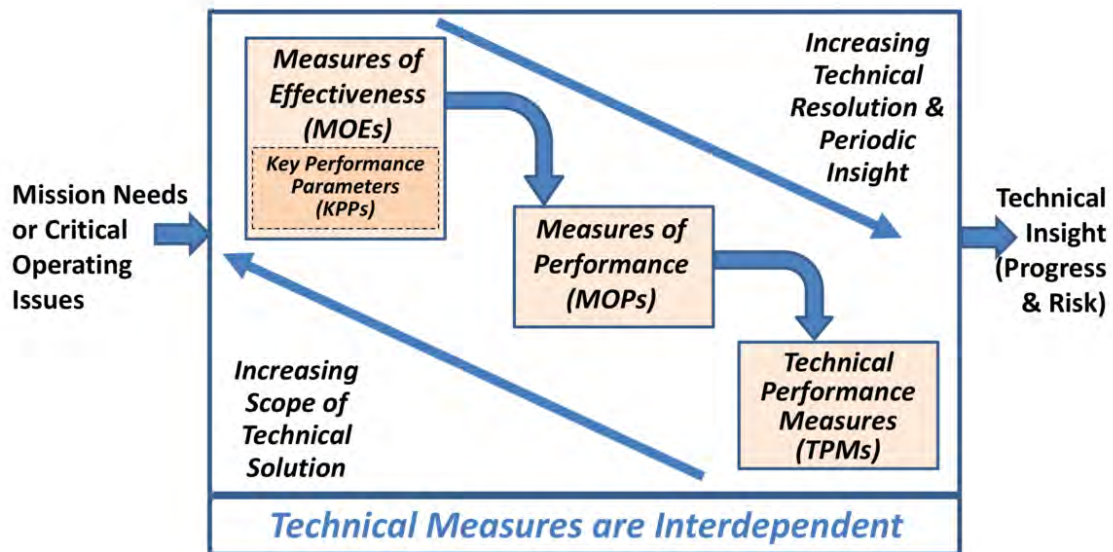


Figure 3. Relationship of the Technical Measures (Roedler et al 2010). Reprinted with permission of the International Council on Systems Engineering (INCOSE^[1]) and Practical Software and Systems Measurement (PSM^[1]). All other rights are reserved by the copyright owner.

Service Measurement

The same measurement activities can be applied for service measurement; however, the context and measures will be different. Service providers have a need to balance efficiency and effectiveness, which may be opposing objectives. Good service measures are outcome-based, focus on elements important to the customer (e.g., service availability, reliability, performance, etc.), and provide timely, forward-looking information.

For services, the terms critical success factors (CSF) and key performance indicators (KPI) are used often when discussing measurement. CSFs are the key elements of the service or service infrastructure that are most important to achieve the business objectives. KPIs are specific values or characteristics measured to assess achievement of those objectives.

More information about service measurement can be found in the *Service Design* and *Continual Service Improvement* volumes of BMP (2010, 1). More information on service SE can be found in the Service Systems Engineering article.

Linkages to Other Systems Engineering Management Topics

SE measurement has linkages to other SEM topics. The following are a few key linkages adapted from Roedler and Jones (2005):

- **Project Planning** – SE measurement provides the historical data and supports the estimation for, and feasibility analysis of, the plans for realistic planning.
- **Project Assessment and Control** – SE measurement provides the objective information needed to perform the assessment and determination of appropriate control actions. The use of leading indicators allows for early assessment and control actions that identify risks and/or provide insight to allow early treatment of risks to minimize potential impacts.
- **Risk Management** – SE risk management identifies the information needs that can impact project and organizational performance. SE measurement data helps to quantify risks and subsequently provides information about whether risks have been successfully treated to a level that they can be managed or eliminated. Risk management relies heavily on measurement, as it quantifies potential impacts and provides data to support informed decision-making, resource allocation, and the evaluation of risk treatments or handling strategies. Thus, the risks and risk treatments identify potential new measures to provide this insight. Additionally, the results of measures may also help to identify potential new risks that should be considered for assessment and handling.
- **Decision Management** – SE Measurement results inform decision making by providing objective insight.

Digital Engineering (DE) Measurement

As stated in the article “Digital Engineering Measures: Research and Guidance” by McDermott et al. in the March 2022 issue of INCOSE Insight magazine, “Digital Engineering (DE) and model-based approaches are two components of enterprise digital transformation that have great promise to improve the efficiency and productivity of engineering activities, particularly for complex engineered systems.” The trends towards Digital Engineering (DE) have continued, so the industry needs to be able to define and perform measurement to manage DE and make informed decisions. For DE, “data and models are the core products undergoing measurement.” (McDermott et al. 2022).

In June 2022, a coalition consisting of INCOSE, NDIA, AIA, SERC, and the US DoD R&E joined with PSM to develop and publish the PSM Digital Engineering Measurement Framework. This framework is intended to supplement the foundational measurement guidance provided by ISO/IEC/IEEE 15939, PSM 2025. And other resources listed in this article. This framework provides “the initial set of constructs ... to isolate those measurements that are most closely linked to the primary benefits of DE.” The guidance focuses on supplemental measures for DE specific information needs that are intended to be used along with relevant measures for other information needs of the project and other engineering activities. The DE measures “focus on the digital information products (data and models) developed and used across a product life cycle.” (DE Framework 2022). This framework can be obtained from the PSM website (www.psmc.com) or from the collaborating organizations. Version 2 of the framework is expected to be published in December 2025.

The DE measurement framework considers measures needed to provide insight for the scope of DE work activities. These include:

- Digital Infrastructure (Environment and Supporting Tools)
 - Life Cycle Models (Phases/Stages and Decision Points)
 - Process Models (Work Processes, Digital Artifacts)
 - Data and Model Ontology (System)
 - Operational, System, and Discipline Specific Data and Models
-

Continuous Iterative Development (CID)

Since most systems have great quantities of software, it would be remiss to ignore recent developments in measurement for software development employing Continuous Iterative Development (CID) approaches (often referred to as agile). Similarly to the DE Measurement Framework, this work is meant to supplement the foundational measurement guidance provided by ISO/IEC/IEEE 15939, PSM 2021 and other resources listed in this article. The CID Measurement Framework was developed and published by a team spanning Practical Software and Systems Measurement (PSM), the National Defense Industrial Association (NDIA), and the International Council on System Engineering (INCOSE). The framework details “common information needs and measures that are effective for evaluating CID approaches. The information needs address the team, product, and enterprise perspectives to provide insight and drive decision-making. The framework also identifies and specifies an initial set of measures identified as practical measures to address these information needs.” (CID Framework 2021) It also includes a series of diagrams and an ontology to describe the development approaches and terminology used. (PSM 2025) The guidance is provided in three volumes that cover basic guidance, prioritized information needs for CID projects, and ten specifications for prospective measures. All of the volumes can be obtained from the PSM website (www.psm-sc.com) or from the collaborating organizations.

Practical Considerations

Key pitfalls and good practices related to SE measurement are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing SE Measurement are provided in Table 1.

Table 1. Measurement Pitfalls. (SEBoK Original)

Name	Description
Golden Measures	- Looking for the one measure or small set of measures that applies to all projects. - No one-size-fits-all measure or measurement set exists. - Each project has unique information needs (e.g., objectives, risks, and issues). - The one exception is that, in some cases with consistent product lines, processes, and information needs, a small core set of measures may be defined for use across an organization.
Single-Pass Perspective	- Viewing measurement as a single-pass activity. - To be effective, measurement needs to be performed continuously, including the periodic identification and prioritization of information needs and associated measures.
Unknown Information Need	- Performing measurement activities without the understanding of why the measures are needed and what information they provide. - This can lead to wasted effort. Avoid "measurement for measurement sake".
Inappropriate Usage	- Using measurement inappropriately, such as measuring the performance of individuals or making interpretations without context information. - This can lead to bias in the results or incorrect interpretations.

Good Practices

Some good practices gathered from the references are provided in Table 2.

Table 2. Measurement Good Practices. (SEBoK Original)

Name	Description
Periodic Review	Regularly review each measure collected.
Action Driven	- Measurement by itself does not control or improve process performance. - Measurement results should be provided to decision makers for appropriate action.
Integration into Project Processes	- SE Measurement should be integrated into the project as part of the ongoing project business rhythm. - Data should be collected as processes are performed, not recreated as an afterthought.
Timely Information	- Information should be obtained early enough to allow necessary action to control or treat risks, adjust tactics and strategies, etc. - When such actions are not successful, measurement results need to help decision-makers determine contingency actions or correct problems.
Relevance to Decision Makers	- Successful measurement requires the communication of meaningful information to the decision-makers. - Results should be presented in the decision-makers' preferred format. - Allows accurate and expeditious interpretation of the results.
Data Availability	Decisions can rarely wait for a complete or perfect set of data, so measurement information often needs to be derived from analysis of the best available data, complemented by real-time events and qualitative insight (including experience).
Historical Data	Use historical data as the basis of plans, measure what is planned versus what is achieved, archive actual achieved results, and use archived data as a historical basis for the next planning effort.
Information Model	The information model defined in ISO/IEC/IEEE (2007) provides a means to link the entities that are measured to the associated measures and the identified information need, and also describes how the measures are converted into indicators that provide insight to decision-makers.

Additional information can be found in the *Systems Engineering Measurement Primer*, Section 4.2 (Frenz et al. 2010), and *INCOSE Systems Engineering Handbook*, Section 5.7.1.5 (2012).

References

Works Cited

- Carson and Kohl. 2013. *New Opportunities for Architecture Measurement*. Proceedings of the INCOSE International Symposium, Philadelphia 2013. Hoboken, New Jersey: Wiley & Sons.
- DE Measurement Framework. 2022. *PSM Digital Engineering Measurement Framework* version 1.1. Practical Software and Systems Measurement. Available at: <https://www.psmc.com/Downloads/DEPaper/DE%20Measurement%20Framework%20ver%201.1%202022-07-27%20final.pdf> or <https://www.psmc.com/DEMeasurement>
- Frenz, P., G. Roedler, D.J. Gantzer, P. Baxter. 2010. *Systems Engineering Measurement Primer: A Basic Introduction to Measurement Concepts and Use for Systems Engineering*. Version 2.0. San Diego, CA: International Council on System Engineering (INCOSE). INCOSE-TP-2010-005-02. Accessed April 13, 2015 at <https://www.incose.org/ProductsPublications/techpublications/PrimerMeasurement>.
- INCOSE Insight. 2022. *Digital Engineering Measures: Research and Guidance*. Hoboken, New Jersey: Wiley. International Council on Systems Engineering (INCOSE) Insight Magazine, March 2022.
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Process and Activities*. 5th ed. D. D. Walden, T. M. Shortell, G. J. Roedler, B. A. Delicado, O. Mornas, Yip Y. S., and D. Endler (Eds.). San Diego, CA: International Council on Systems Engineering. Published by John Wiley & Sons, Inc.

- ISO/IEC/IEEE. 2017a. *Systems and Software Engineering – Software Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 12207:2017.
- ISO/IEC/IEEE. 2017. *ISO/IEC/IEEE 15939|Systems and software engineering - Measurement process*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC/IEEE 15939:2017.
- ISO/IEC/IEEE. 2023. *ISO/IEC/IEEE 15288|Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.
- Kasunic, M. and W. Anderson. 2004. *Measuring Systems Interoperability: Challenges and Opportunities*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- McGarry, J., D. Card, C. Jones, B. Layman, E. Clark, J. Dean, F. Hall. 2002. *Practical Software Measurement: Objective Information for Decision Makers*. Boston, MA, USA: Addison-Wesley.
- NASA. 2007. *NASA Systems Engineering Handbook|Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), December 2007. NASA/SP-2007-6105.
- NDIA and PSM. 2011. *Working Group Report: System Development Performance Measurement*. National Defense Industrial Association (NDIA) and Practical Software and Systems Measurement (PSM). Available at: <https://www.psmc.com/Downloads/Other/NDIA%20System%20Developopment%20Performance%20Measurement%20Report.pdf>
- Park, R.E., W.B. Goethert, and W.A. Florac. 1996. *Goal-Driven Software Measurement – A Guidebook*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU), CMU/SEI-96-BH-002.
- PSM. 2025. *Practical Software and Systems Measurement*. Accessed September 29, 2025. Available at: <https://www.psmc.com/>.
- PSM. 2000. *Practical Software and Systems Measurement (PSM) Guide*, version 4.0c. Practical Software and System Measurement Support Center. Available at: <https://www.psmc.com/PSMGuide.asp>.
- PSM. 2021. *Continuous Iterative Development Measurement*, version 2.1, Volumes 1, 2, and 3. Practical Software and Systems Measurement. Available at: <https://www.psmc.com/CIDMeasurement>
- PSM Safety & Security TWG. 2006. *Security Measurement*, version 3.0. Practical Software and Systems Measurement. Available at: https://www.psmc.com/Downloads/TechnologyPapers/SecurityWhitePaper_v3.0.pdf.
- QuEST Forum. 2020. *TL 9000 Quality Management System (QMS) Measurements Handbook*, Release 5.7. Plano, TX, USA: Quest Forum.
- Roedler, G., D. Rhodes, C. Jones, and H. Schimmoller. 2010. *Systems Engineering Leading Indicators Guide*, version 2.0. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2005-001-03.
- Roedler, G. and C. Jones. 2005. *Technical Measurement Guide*, version 1.0. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-020-01.
- SAE. 2018. *Integrated Project Processes for Engineering a System*. Warrendale, PA: SAE International, SAE 1001:2018.
- SEI. 2010. "Measurement and Analysis Process Area" in *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- Statz, J. et al. 2005. *Measurement for Process Improvement*, version 1.0. York, UK: Practical Software and Systems Measurement (PSM).
-

Tufte, E. 2006. *The Visual Display of Quantitative Information*. Cheshire, CT, USA: Graphics Press.

Wasson, C. 2005. *System Analysis, Design, Development: Concepts, Principles, and Practices*. Hoboken, NJ, USA: John Wiley and Sons.

Primary References

Carson and Kohl. 2013. *New Opportunities for Architecture Measurement*. Proceedings of the INCOSE International Symposium, Philadelphia 2013. Hoboken, New Jersey: Wiley & Sons.

DE Measurement Framework. 2022. *PSM Digital Engineering Measurement Framework*, version 1.1. Practical Software and Systems Measurement. Available at: <https://www.psmc.com/Downloads/DEPaper/DE%20Measurement%20Framework%20ver%201.1%202022-07-27%20final.pdf> or <https://www.psmc.com/DEMeasurement>

Frenz, P., G. Roedler, D.J. Gantzer, P. Baxter. 2010. *Systems Engineering Measurement Primer: A Basic Introduction to Measurement Concepts and Use for Systems Engineering*. Version 2.0. San Diego, CA: International Council on System Engineering (INCOSE). INCOSE-TP-2010-005-02. Accessed April 13, 2015 at <https://www.incose.org/ProductsPublications/techpublications/PrimerMeasurement>.

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Process and Activities (5th ed.)*. D. D. Walden, T. M. Shortell, G. J. Roedler, B. A. Delicado, O. Mornas, Yip Y. S., and D. Endler (Eds.). San Diego, CA: International Council on Systems Engineering. Published by John Wiley & Sons, Inc.

ISO/IEC/IEEE. 2017. *Systems and Software Engineering - Measurement Process*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC/IEEE 15939:2017.

NDIA and PSM. 2011. *Working Group Report: System Development Performance Measurement*. National Defense Industrial Association (NDIA) and Practical Software and Systems Measurement (PSM). Available at: <https://www.psmc.com/Downloads/Other/NDIA%20System%20Development%20Performance%20Measurement%20Report.pdf>

PSM. 2025. *Practical Software and Systems Measurement*. Accessed September 29, 2025. Available at: <https://www.psmc.com/>.^[3]

PSM. 2000. *Practical Software and Systems Measurement (PSM) Guide*, version 4.0c. Practical Software and System Measurement Support Center. Available at: <https://www.psmc.com>.

PSM. 2021. *Continuous Iterative Development Measurement*, version 2.1, Volumes 1, 2, and 3. Practical Software and Systems Measurement. Available at: <https://www.psmc.com/CIDMeasurement>.^[4]

Roedler, G., D. Rhodes, C. Jones, and H. Schimmoller. 2010. *Systems Engineering Leading Indicators Guide*, version 2.0. San Diego, CA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2005-001-03.

Roedler, G. and C. Jones. 2005. *Technical Measurement Guide*, version 1.0. San Diego, CA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-020-01.

Additional References

- Kasunic, M. and W. Anderson. 2004. *Measuring Systems Interoperability: Challenges and Opportunities*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- McGarry, J. et al. 2002. *Practical Software Measurement: Objective Information for Decision Makers*. Boston, MA, USA: Addison-Wesley.
- NASA. 2007. *NASA Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), December 2007. NASA/SP-2007-6105.
- Park, Goethert, and Florac. 1996. *Goal-Driven Software Measurement – A Guidebook*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU), CMU/SEI-96-BH-002.
- PSM. 2011. "Practical Software and Systems Measurement." Accessed August 18, 2011. Available at: <https://www.psmc.com/>.
- PSM Safety & Security TWG. 2006. *Safety Measurement*, version 3.0. Practical Software and Systems Measurement. Available at: https://www.psmc.com/Downloads/TechnologyPapers/SafetyWhitePaper_v3.0.pdf.
- PSM Safety & Security TWG. 2006. *Security Measurement*, version 3.0. Practical Software and Systems Measurement. Available at: https://www.psmc.com/Downloads/TechnologyPapers/SecurityWhitePaper_v3.0.pdf.
- SEI. 2010. "Measurement and Analysis Process Area" in *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- Statz, J. 2005. *Measurement for Process Improvement*, version 1.0. York, UK: Practical Software and Systems Measurement (PSM).
- Tufte, E. 2006. *The Visual Display of Quantitative Information*. Cheshire, CT, USA: Graphics Press.
- Wasson, C. 2005. *System Analysis, Design, Development: Concepts, Principles, and Practices*. Hoboken, NJ, USA: John Wiley and Sons.

References

- [1] <https://www.psmc.com>
- [2] <https://www.incose.com>
- [3] <https://www.psmc.com/>.
- [4] <https://www.psmc.com/CIDMeasurement>.

Knowledge Area: System Concept Definition

System Concept Definition

Contents of this Knowledge Area

- Business or Mission Analysis (Tami Katz) (Lou Wheatcraft and Mike Ryan)
- Stakeholder Needs Definition (Tami Katz) (Lou Wheatcraft and Mike Ryan)
- Lead Author:
 - Tami Katz
- Contributing Authors:
 - Lou Wheatcraft, Mike Ryan, Garry Roedler, and Rick Adcock

Concept Definition is the set of systems engineering (SE) activities in which the problem space as well as the needs and requirements of the business (or enterprise) and stakeholders are closely examined. Concept definition begins before any formal definition of the system-of-interest (SoI) is developed.

The Concept Definition activities include Business or Mission Analysis and Stakeholder Needs Definition. Within these two activities the enterprise or project decision makers, as well as additional key stakeholders, describe *what* a solution should accomplish and *why* it is needed. Both *why* and *what* need to be answered before consideration is given to *how* the problem will be addressed (i.e., what type of solution will be implemented) and *how* the solution will be defined and developed.

Concept Definition Activities

There are two primary activities discussed under Concept Definition: Business or Mission Analysis and the definition of Stakeholder Needs:

1. The Business or Mission Analysis activity defines the problem, threat, or opportunity being addressed which could result in a new or modified product or service. This process also includes identification of major stakeholders, the mission, goals, and objectives of the SoI, the measures of success, identification of business needs and requirements, and identification of the SoI life cycle concepts.
2. The Stakeholder Needs Definition activity uses the inputs from the Business or Mission Analysis effort to identify an integrated set of needs based on inputs from the major stakeholders, higher-level requirements, and an analysis of the life cycle concepts, drivers, constraints, and risks.

The products and artifacts produced during Concept Definition are then used in the System Definition process.

Drivers of Concept Definition

There are many considerations associated with concept definition activities, which are further elaborated below.

The Role of Architecture Development

The activities of Business or Mission Analysis and Stakeholder Needs Definition occur concurrently with the processes of System Architecture Design Definition. The activities to address a full set of needs includes identification of SoI life cycle concepts, external interfaces and constraints, as well as candidate solutions and an exploration of the architecture (logical and functional).

Drivers of Solution on Problem Definition

During Concept Definition, the problem definition and solution exploration depend on each other: solutions should be developed to respond appropriately to well-defined problems; and problem definitions should be constrained by what is feasible in the solution space. System analysis is used to provide the links between problems and solutions.

There are two paradigms that drive the ways in which concept definition is done: push and pull. The pull paradigm is based on providing a solution to an identified problem or gap, such as a missing mission capability for defense or infrastructure. The push paradigm is based on creating a solution to address a perceived opportunity, such as the emergence of an anticipated product or service that is attractive to some portion of the population (i.e. whether a current market exists or not). This can impact other life cycle processes, such as verification and validation, or alpha/beta testing as done in some commercial domains.

As systems generally integrate existing and new system elements in a mixture of push and pull, it is often best to combine a bottom-up approach with a top-down approach to take into account legacy elements, as well as to identify the services and capabilities that must be provided in order to define applicable interface requirements and constraints. This is discussed in Applying Life Cycle Processes.

New System or Modification of Existing System

The activities of concept definition determine whether the enterprise strategic goals and business needs will be addressed by a new system, a change to an existing system, a service, an operational change, or some other solution.

For a new system, the organization or customer has decided to start with a “blank piece of paper”. This is often referred to as a green-field system, and analysis efforts during concept definition characterize the as-is or present-state of the SoI in terms of the problem, threat, or opportunity and then characterize the to-be or future-state of the SoI in obtaining the resolution of the problem, threat, or opportunity.

An existing system can be evolved or transformed into the desired system. This is often referred to as a brown-field system, and the data that has been established for the original system can be used as inputs into the analysis efforts during concept definition activities. The existing system may have been developed for other purposes, the stakeholder needs or the operational environment may have changed, e.g., a change in threats. The analysis effort will explore the problem space and possible solutions to the gaps of the existing system to address the problem, threat or opportunity.

References

Works Cited

None.

Primary References

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-118-99940-0.

INCOSE. 2022. *INCOSE Needs and Requirements Manual*, version 1.1. INCOSE-TP-2021-002-01.

ISO/IEC/IEEE. 2023. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

Additional References

Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: John Wiley & Sons.

ISO/IEC. 2003. *Systems Engineering – A Guide for The Application of ISO/IEC 15288 System Life Cycle Processes*.

ISO/IEC. 2007. *Systems Engineering – Application and Management of The Systems Engineering Process*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 26702:2007.

Jackson, S., D. Hitchins, and H. Eisner. 2010. "What is the Systems Approach?" *INCOSE Insight*. (April 2010): 41-43.

NASA. 2016. *Systems Engineering Handbook*. Washington, D.C., USA: National Aeronautics and Space Administration (NASA). NASA/SP-2016-6105 rev 2.

Business or Mission Analysis

- Lead Author:
- Tami Katz
- Contributing Authors:
- Lou Wheatcraft and Mike Ryan

The primary start to the engineering of any system-of-interest (SoI) is to obtain an understanding the objectives behind the development of the SoI, which is performed as part of the Concept Definition process.

Business or Mission Analysis, the first process performed in Concept Definition, establishes a definition of the overall strategic problem or opportunity and identification of potential solution classes to address the problem (or take advantage of an opportunity) (ISO/IEC/IEEE 15288). The solution could be a new system, a change to an existing system, a service, an operational change or some other solution.

This analysis effort identifies the primary purpose(s) of the SoI (its "mission"). The second Concept Definition process is called Stakeholder Needs Definition, which explores what capabilities are needed to accomplish the mission or the intended use of the system.

Business or Mission Analysis is often performed iteratively with the Stakeholder Needs Definition process to better understand the problem space, as well as options in the solution space.

Purpose and Definition

The purpose of Business or Mission Analysis is to understand a mission or market problem, threat, or opportunity, and to establish the goals, objectives and measures of success of a potential solution class. This consists of a strategic analysis related to emerging needs, capability gaps, threats, opportunities, and potential solutions by a project team or by an organization to obtain its business objectives prior to establishing a project team.

Principles and Concepts

For the SoI under development, success depends on the project team's understanding the data and information that constitutes the purpose of the SoI (*why?*), acceptability or desirability of a solution (*what?*), measures (*how well?*), and the conditions in which the SOI must operate (*in what operating environment?*)

Prior to SoI development, a project champion works with key stakeholders to clearly define the problem, threat, or opportunity for which the project team is to address. Identifying the specific problem, threat, or opportunity will enable the project team to understand if the project is worth doing, why the system is needed, and the expected capabilities of the SoI. The next step is to identify the mission, goals, and objectives (MGOs) based on the defined problem, threat or opportunity, as well as the measures of success.

- The *Mission* statement is based on the analysis of a problem, threat, or opportunity that the project was formed to address, the expected outcome of the SoI being developed, and defines the "why". Why does the project exist? Why is the SoI needed? What value does it bring to the organization?
 - *Goals* are elaborated from the mission statement to communicate what must be achieved to result in a successful mission. Goals allow the organization to divide the mission statement into manageable pieces and promote a shared understanding between the project team and the business operations level stakeholders of what needs to be done to achieve the mission.
 - *Objectives* are elaborated from the goals to provide more details concerning what must be done that will result in the goals and mission to be achieved. What does the project team need to do to achieve the goals? What does the SoI need to do to achieve the goals?
-

- *Measures* are quantitative metrics used to validate the SoI against the objectives as well as to manage system development across the life cycle, expressed as Measures of Effectiveness (MOE) (reference Measurement).

As part of the Business or Mission Analysis effort, an initial assessment by key stakeholders of the SoI life cycle concepts is used to support identification of candidate solution classes. The organization then performs an evaluation of whether to proceed with development of the SoI based on analysis of the data and alignment with the organization's enterprise strategy. Upon agreement to proceed, the data from the Business and Mission Analysis effort is used by the project team to complete the rest of the concept development process for the SoI, as shown in Figure 1.

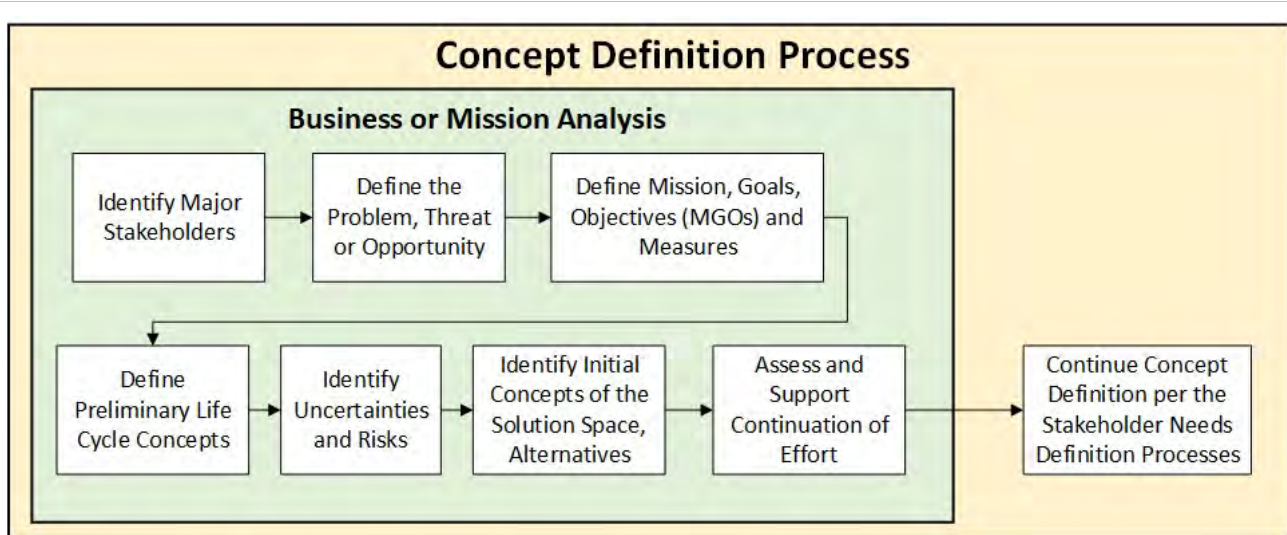


Figure 1. Business or Mission Analysis addresses the effort to generate project success, which is provided to the project team for further concept definition activities. (SEBoK Original)

There are several outputs of the Business or Mission Analysis process:

- identification of major stakeholders,
- definition of the problem, threat, or opportunity to which the project must address,
- elaboration of the MGOs and measures of success,
- identification of preliminary life cycle concepts and the preferred solution classes,
- traceability of strategic problems, threats, opportunities, MGOs, and measures of success to the preferred solution classes, and
- confirmation of organizational support.

The effort of Business or Mission Analysis is often done at a business enterprise level, where the initial assessment results in the authorization for a project and associated budget along with an acquisition concept. For the project team responsible for developing the SoI, this means seeking an understanding of this content to ensure the outcomes of the project align with the organization's overall strategy and rationale for developing that particular SoI.

The output from the Business or Mission Analysis is then provided to the project team for use in additional analysis that establishes an overall set of needs for the solution (described in Stakeholder Needs Definition).

Process Approach

Identify Major Stakeholders

During this process the initial stakeholder identification is performed. This can be captured in a stakeholder register, noting each stakeholder and their involvement with the SoI and project, as well as establishment a ranking of the stakeholders. This list is expanded upon during Stakeholder Needs Definition. At this phase, the stakeholders will often include key members from the organization at the enterprise level.

Define the Problem, Threat, or Opportunity

Identifying the specific problem, threat or opportunity will enable the project team to understand why the system is needed, and which capabilities, functions, performance, and features that are important to the customers, users, operators, maintainers, and disposers of the system.

There are four steps to defining the problem, threat, or opportunity:

1. Identification of the organization's strategic and business operations level stakeholders that are impacted by the problem or threat or those who will benefit by pursuing the opportunity.
2. Collaboration with these stakeholders to understand how they are impacted by the problem or threat, or by those that will benefit by pursuing the opportunity.
3. Clear definition of the problem, threat, or opportunity.
4. Stakeholder concurrence with the problem, threat, or opportunity statement.

Example Problem/Threat/Opportunity statement: Marketing is seeing an increase of work from home personnel that are purchasing more coffee makers. Existing coffee makers have single functions while consumers want a multi-function hot beverage maker to produce a blend of options like traditional brew or espresso. There is a huge opportunity if first to market. (INCOSE GtNR 2022)

Define the Mission, Goals, Objectives and Measures

To achieve the MGOs and measures, the project champion collaborates with the stakeholders that participated in defining the problem, threat, or opportunity to better understand what they view as an acceptable outcome by asking them a series of questions:

- How do they define success?
- What measures would the stakeholders use to determine success?
- What is the intended use of the SoI and in what operating environment?
- What capabilities, features, functions, and performance do they need?
- What are their expectations for quality and compliance (such as with standards and regulations)?
- What specific outcome(s) do they expect once the SoI is delivered?
- What are the measures of success, e.g., measures of effectiveness (MOE), expected of the SoI?

For cases where there is no existing SoI, a common approach is to characterize the “as is” or “present state” in terms of the problem, threat, or opportunity and then characterize the “to be” or “future state” in terms of the resolution of the problem, neutralizing the threat, or the ability to pursue the opportunity.

For existing systems that need to be updated, a common approach is to list the problems or issues with the existing “as-is” system and the reasons it needs to be changed. Key information includes what value they believe will result from the change by addressing: What can the existing SoI no longer do, what performance needs to be improved, what changes need to be made concerning interactions with external systems, what updates are needed as a result of changes to applicable standards and regulations, what updates are needed as a result of changes in the operational environment or new threats (e.g., security)?

It is also important to understand different perspectives. The problem, threat, or opportunity, MGOs, and measures from a business perspective may be different than the user's perspective, both must be addressed. The user does not necessarily care about the developing organization's profits, time to market, market share, etc., they care about how the resulting SoI meets their needs. Thus, there could be several sets of MGOs and measures that need to be defined and met by the project team from both a business perspective and a user perspective of the SoI to be developed. This may lead to conflicts, e.g., product price vs. profitability vs. market share, which need to be prioritized based on the organizational enterprise strategy.

Examples (derived from INCOSE GtNR 2022):

- Mission statement: Increase revenue by providing a home-based, one-stop, multi-function hot beverage maker.
- Consumer Goals: Product a blend of options. Obtain home brew beverage quickly.
- Consumer Objective: Options of brew or espresso. Receive finished home brew coffee within minutes upon request.
- Consumer-focused Measure: Finished home brew provided within 2 minutes of activation at the selected temperature. Rationale: Consumer survey results.
- Business Goals: Increase revenue. Increase market share.
- Business Objective: Increase market share of Company X sales regions.
- Business-focused Measure: Improve current market share of Company X sales regions by 20%. Rationale: Aligns with the enterprise strategic vision.

Capture Preliminary Life Cycle Concepts

Life cycle concepts, such as the operational concept, define what the SoI needs to do and how well during its intended use in the expected operational environment, from multiple perspectives. This includes various use cases with expected interactions with external systems, identification of drivers and constraints, expected risks to success in the context of the MGOs and measures.

Preliminary life cycle concepts are defined by the organization and include preliminary concepts for acquisition, development, manufacturing or coding, verification, validation, deployment, operations, support, and retirement. Operational concepts include high level operational modes or states, operational scenarios, potential use cases, misuse cases, loss scenarios, or usage within a proposed business strategy. These concepts enable feasibility analysis and evaluation of solution options. For example, concerning acquisition and development how much of the development effort will be done inhouse or outsourced to a supplier? Who will be the overall integrator? If development is done in house, what are the concepts for the supply chain for parts and components?

In a Model-Based Systems Engineering effort, these life cycle concepts are generated using mission analysis, such as defining use cases associated with users and life cycle stage (Figure 2). Reference Model-Based Systems Engineering (MBSE) for more information regarding usage of MBSE.

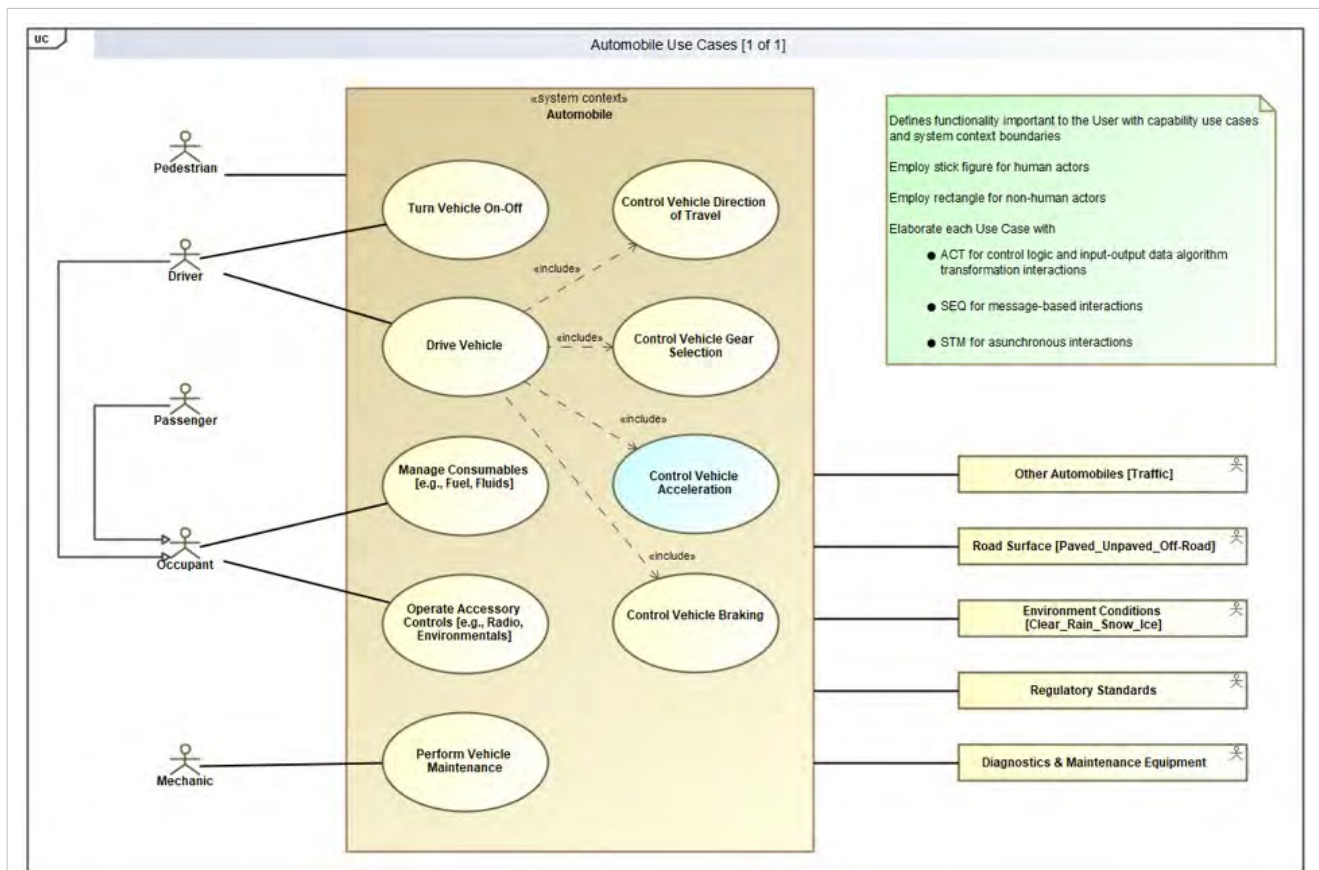


Figure 2. Example Life Cycle Concepts Using MBSE Approaches. (SEBoK Original)

The life cycle concepts will be further refined within the stakeholder needs definition process. As part of the preliminary life cycle concepts, candidate solution classes are suggested for a possible range of approaches to address the MGOs, as part of a Functional Architecture definition activity.

Identify Uncertainties and Risks

The preliminary life cycle concepts include some level of uncertainty. This leads to risks which need to be identified and managed using the Risk Management process. These risks may also serve as inputs to the generation of additional life cycle concepts (such as addressing cyber security or hazards to successful operation).

Identify Preliminary Concepts of the Solution Space, Including Alternatives

A solution class refers to the means of achieving a solution. Examples include development of a new system, modifying or upgrading an existing system, leveraging multiple existing systems, or generating operational changes.

Possible solution classes to address the problem, threat, or opportunity are assessed against the preliminary life cycle concepts, MGOs and measures. Feasibility of a solution class and its capability to meet the strategic needs are key decision criteria, as well as feasibility considerations in terms of cost, schedule, technology, legal, ethical, environmental, sustainability, etc. The Decision Management process is used to evaluate alternatives and to guide selection. The assessment of alternatives can include modeling, simulation, analytical techniques, or expert judgment to understand the risks, feasibility, and value of the alternative solution classes.

The conclusion of this effort results in the identification of preliminary concepts of the solution space, including alternatives, which are traceable to the organization defined problems/threats/opportunities and MGOs/measures.

Assessment of Continuation of Effort

Upon the conclusion of the Business or Mission Analysis process the organization performs an evaluation on whether to commence with development of the SoI. The key determination is alignment with the enterprise strategy. If the effort is to be continued, a project team will take the outcomes of the Business or Mission Analysis effort and commence with the Stakeholder Needs Definition activity.

References

Works Cited

ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

INCOSE. 2022. *INCOSE Needs and Requirements Manual (NRM)*, version 1.1. INCOSE-TP-2021-002-01.

INCOSE. 2022. *INCOSE Guide to Needs and Requirements (GtNR)*, version 1. INCOSE-TP-2021-003-01

Primary References

ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-118-99940-0.

INCOSE. 2022. *INCOSE Needs and Requirements Manual (NRM)*, version 1.1. INCOSE-TP-2021-002-01.

INCOSE. 2022. *INCOSE Guide to Needs and Requirements (GtNR)*, version 1. INCOSE-TP-2021-003-01

Additional References

None.

Stakeholder Needs Definition

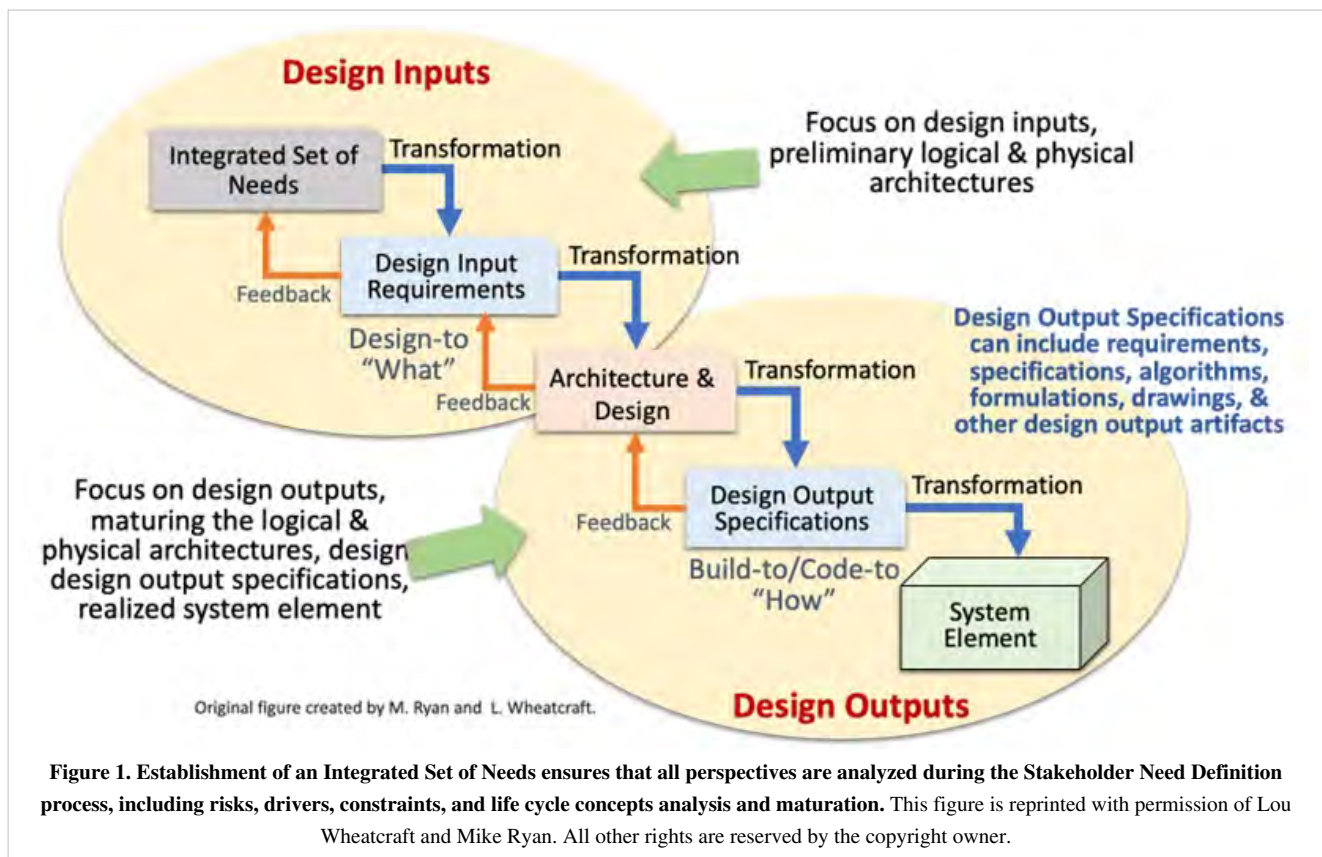
- Lead Author:
- Tami Katz
- Contributing Authors:
- Lou Wheatcraft and Mike Ryan

Stakeholder Needs Definition, the second process in Concept Definition, explores what capabilities are needed by various stakeholders for the system-of-interest (SoI) to accomplish the mission. The outcome of the Stakeholder Needs Definition process is used as the basis of System Validation, as well as input into the System Requirements Definition process.

Note that the first process, Business or Mission Analysis, is often performed iteratively with Stakeholder Needs Definition to better understand the problem, threat, or opportunity space, as well as options of the solution space.

Purpose and Definition

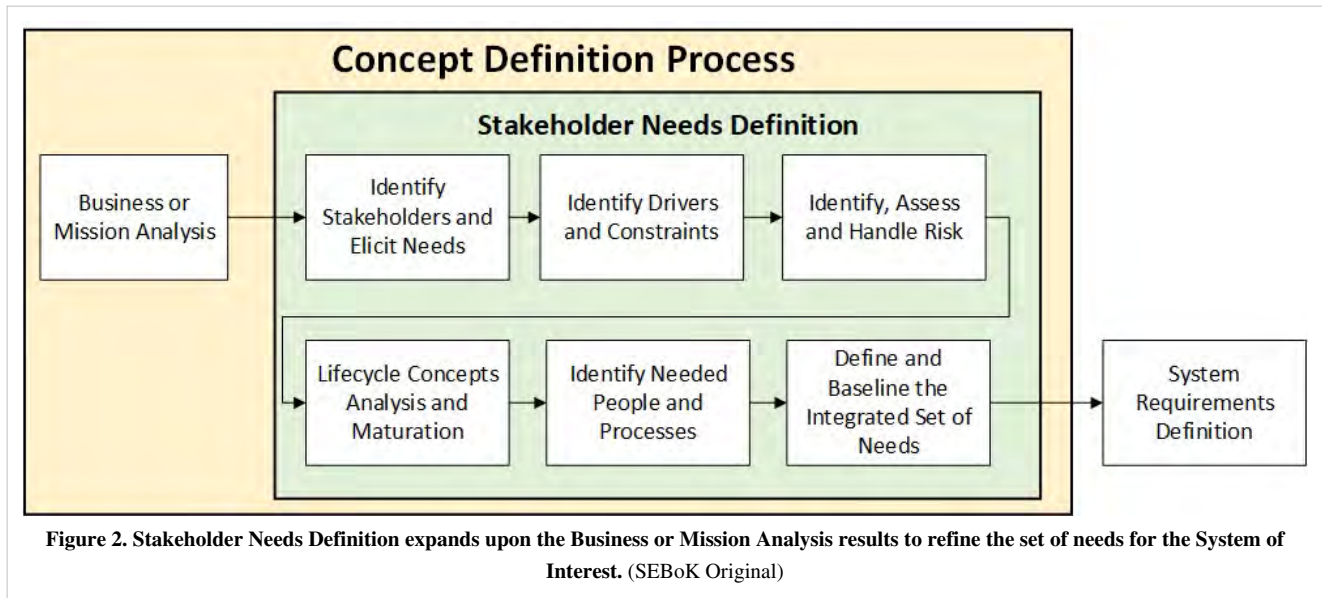
Stakeholder needs represent a user, acquirer, customer, and other stakeholders perspective of the SoI, which are then transformed into system requirements which communicate a developer perspective of the SoI. When stakeholder needs are combined with results of multiple analysis activities that includes risks, drivers, constraints, and life cycle concepts analysis, as shown in Figure 1, the result is an overall integrated set of needs.



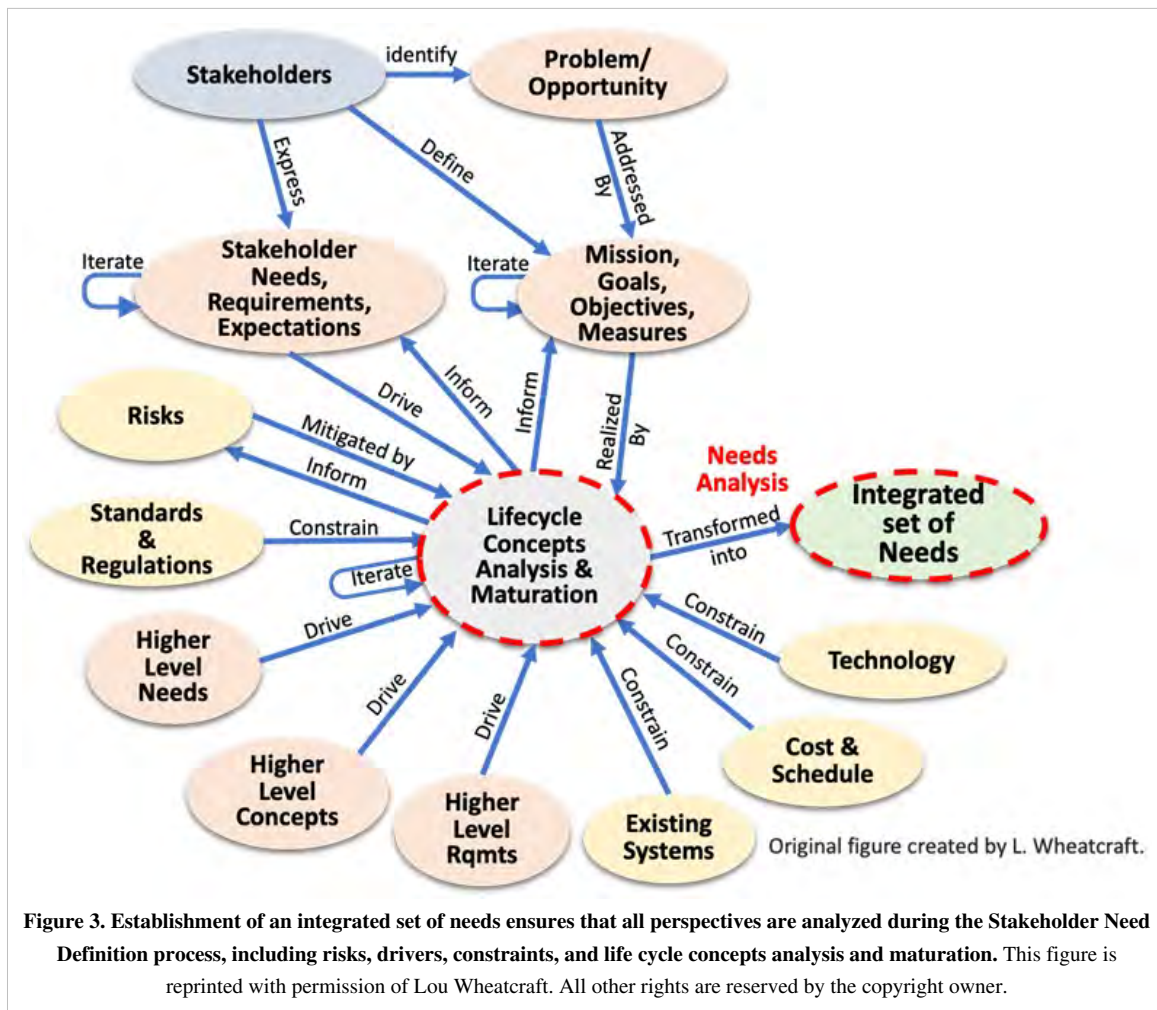
The establishment of the integrated set of needs forms the basis of a full understanding of the capabilities expected of the SoI, and these needs are ultimately transformed into a set of design-input requirements on the SoI as part of the System Requirements Definition process.

Principles and Concepts

The results of the Business or Mission Analysis is provided as inputs into the Stakeholder Needs Definition process (shown in Figure 2). This input includes the problem, threat or opportunity statement capturing why the project is worth doing, the mission, goals and objectives (MGOs) and measures of success used as the criteria for assessing project success, along with identification of major stakeholders, initial life cycle concepts, and initial concepts of the solution space (architecture and design).



The Stakeholder Needs Definition process continues the life cycle concepts definition activities to ensure the system-of-interest (SoI) provides the capabilities needed by users and other stakeholders in the intended operational environment. This process is much more than identification and elicitation of need or requirement statements from various stakeholders, it consists of a series of analysis activities done to ensure that all parameters are captured, including risks, drivers, constraints, as well as the SoI life cycle concepts analysis and maturation; this effort results in an integrated set of needs as shown in Figure 3.



The result of this process is a well-formed integrated set of needs that is correct, consistent, complete, and feasible. It is this set of needs that defines the scope of the project which the organization agrees to provide the resources necessary for developing the SoI, and against which the requirements, design, and the realized SoI will be validated against.

Nomenclature discussion

This process is frequently referred to as the "Stakeholder Needs and Requirements" process. Because various guides, textbooks, and standards refer to stakeholder "expectations, needs, and requirements" as if they are the same, resulting in confusion as to what is an "expectation" versus a "need" and a "requirement", this article focuses on the process of developing an integrated set of stakeholder needs. The term "stakeholder requirements" is considered a set of requirements on the SoI established by the stakeholder, as transformed from their needs, which are provided as additional input towards the life cycle concepts analysis and maturation activities from which the integrated set of needs is defined. In Figure 3, this is designated as both "stakeholder needs, requirements, and expectations" as well as the "higher-level requirements" inputs.

Process Approach

Inputs to the Stakeholder Needs Definition Process

The inputs from the Business or Mission Analysis process includes identification of major stakeholders, definition of the problem, threat, or opportunity, elaboration of the MGOs and measures of success, capture of preliminary life cycle concepts, and identification of initial concepts of the solution space (architecture and design).

Activities of the Process

There are several activities performed during this process:

- Identify additional stakeholders or classes of stakeholders across the life cycle.
- Elicit, capture, consolidate and prioritize stakeholder needs, requirements, and expectations.
- Identify drivers and constraints on the SoI and its development efforts.
- Identify potential risks (such as threats and hazards) that could prevent the SoI from successful operation (see Risk Management for further information on addressing risks).
- Mature and analyze the life cycle concepts.
- Identify, baseline, and manage the integrated set of needs.

The activities behind each of these are described in the following sections.

Identify Stakeholders

Stakeholders are the primary source of needs and requirements, therefore for the project to be successful, all relevant stakeholders must be identified and included from the beginning of the project. Leaving out a relevant stakeholder often results in missing needs and requirements and a failure to pass system validation. Stakeholders can include, but are not limited to, customers, sponsors, organization decision makers, regulatory organizations, developing organizations, integrators, testers, users, operators, maintainers, support organizations, the public at large (within the context of the business and proposed solution), and those involved in the disposal or retirement of the SoI. Stakeholders can be both internal and external to the organization. There can be many stakeholders for a SoI over its life cycle; therefore, considering the life cycle concepts provides a thorough source for stakeholder identification. Examples of stakeholders are provided in Table 1.

Table 1. Stakeholder Identification Based on Life Cycle Stages. (SEBoK Original)

Life Cycle Stage	Example of Related Stakeholders
Concept	Paying customer, sponsor, project team, project manager, procurement, research and development, suppliers, regulating authorities, public, marketing, end users, operators, compliance office, regulators, owners of enabling systems, owners of external systems, Approving Authorities
Development	Acquirer, subject matter experts (SMEs), system architects, design engineers, suppliers, procurement, suppliers (technical domains for components realization), integration team
Production, Integration, Verification and Validation	Production organization, process engineers, quality control, production verification, product acceptance, supply chain, test engineers, system integration engineers, system verification engineers, system validation engineers, operators/users, owners of enabling systems, facility personnel, contracting, approving authorities, regulators, safety personnel, security personnel
Logistics and Maintenance	Customer/technical support, replacement part providers, service technicians, trainers, IT, quality engineer, inspectors, those conducting post development system verification and system validation activities
Operation	Normal users, unexpected users, etc.
Disposal	Operators, waste management, regulators, public

A key part of stakeholder identification is to determine who the approving authorities are within the group of stakeholders. It cannot be assumed that the only stakeholder that has this authority is the "customer". The approving authorities include stakeholders that are responsible for formally certifying, qualifying, and approving the system for use in its operational environment by its intended users. There can be approving authorities both within and external to the organization, especially for highly-regulated systems.

An approach for recording the list of stakeholders is to use a stakeholder register that includes key information for each stakeholder and how they are involved with the SoI. It is recommended that the project team re-evaluate the stakeholder community periodically to ensure successful engagement with stakeholders, keeping them engaged across all life cycle activities, and managing changes in stakeholders and their needs.

Stakeholder Needs Elicitation

For stakeholder needs elicitation, the project team engages the stakeholders to understand their needs, requirements, and expectations for all life cycle stages. The elicitation activities allow the project team to discover what is needed, what processes exist, how stakeholders interact with the SoI, what happens over the SoI's life cycle from their perspective (examples are provided below).

It is recommended that several techniques or methods be considered during elicitation activities to better accommodate the diverse set of sources (INCOSE NRM 2022):

- structured brainstorming workshops,
- interviews and questionnaires,
- workshops or focus groups,
- use of visual and descriptive content associated with the SoI,
- technical, operational, and/or strategy documentation review, and
- feedback from System Verification and System Validation processes.

A broad range of topics are discussed with the stakeholders:

- Obtain feedback on the outputs from the Business and Mission Analysis process (problem, threat, opportunity, MGOs, etc.).
- Identify the life cycle stages the stakeholder represents and their role.
- For each life cycle stage, obtain input on expected and off-nominal use cases, scenarios, misuse cases, and loss scenarios.
- Identify the desired capabilities and functions from their perspective.
- Identify interactions with external systems.
- Obtain input on their view of quality and other "ilities", such as reliability, testability, serviceability, etc.
- Inquire about their view of risks and hazards, along with likelihood and consequence.
- For each need, capture rationale concerning "why".
- Ask about criticality of the stated needs and relative priorities of all inputs obtained.

During elicitation activities, it is important to ask the stakeholders to provide rationale for and prioritize what they are asking for. Some needs will be especially important to the stakeholder, while others may be "nice-to-haves" and not critical to the system being able to accomplish its intended use. There will be some things that the stakeholder may be able to "live without" given budget or schedule constraints. Providing rationale often reveals the real need, especially when the stakeholder expresses a need as an implementation.

The information obtained from the elicitation activities needs to be recorded with trace to the stakeholder register. In a Model-Based Systems Engineering (MBSE) effort, the elicited needs can be included in the MBSE system model and traced to the stakeholders.

Identify Drivers and Constraints

Drivers and constraints are things outside of the project's control that constrain or drive the solution space. Drivers and Constraints can include design constraints (parts, materials, organizational design best practices, etc.), design standards, production constraints (existing technology, facilities, equipment, cost, throughput, etc.), human factors, (human/machine interface - HMI), regulations (law), operating environment (natural, induced), other environment (social, cultural), existing systems: (interactions, interfaces, dependencies), technology maturity, cost, schedule.

Concurrently with the stakeholder elicitation activities, drivers and constraints need to be identified and recorded within the SoI's integrated dataset.

Identify, Assess, and Handle Risk

Risks are anything that could prevent the delivery of a successful SoI (providing what is needed, within budget and schedule, with the needed quality), anything that could impact the intended use of the SoI in its intended environment by its intended users, or anything that would allow unintended users to prevent the intended use of the SoI or to use the SOI in an unintended manner, e.g., hack into an aircraft and use the aircraft as a weapon.

As part of the elicitation activities, issues and risks must be identified and assessed. The identified risks from the Business or Mission Analysis effort should be used as a starting point, and then additional elaboration of risks is needed along with how the project is expected to handle those risks. Stakeholders should be asked specifically about any issues and risk they think could prevent the SoI to be developed and delivered within budget, schedule, or risk during operations. Failing to address risk will result in an incomplete set of needs and resulting design input requirements resulting in a SoI that will fail system validation.

The project must do a risk assessment of each of the risks (see Risk Management). Some risks could lead to development of life cycle concepts as part of the mitigation (such as for hazards), which are expanded further in the next section.

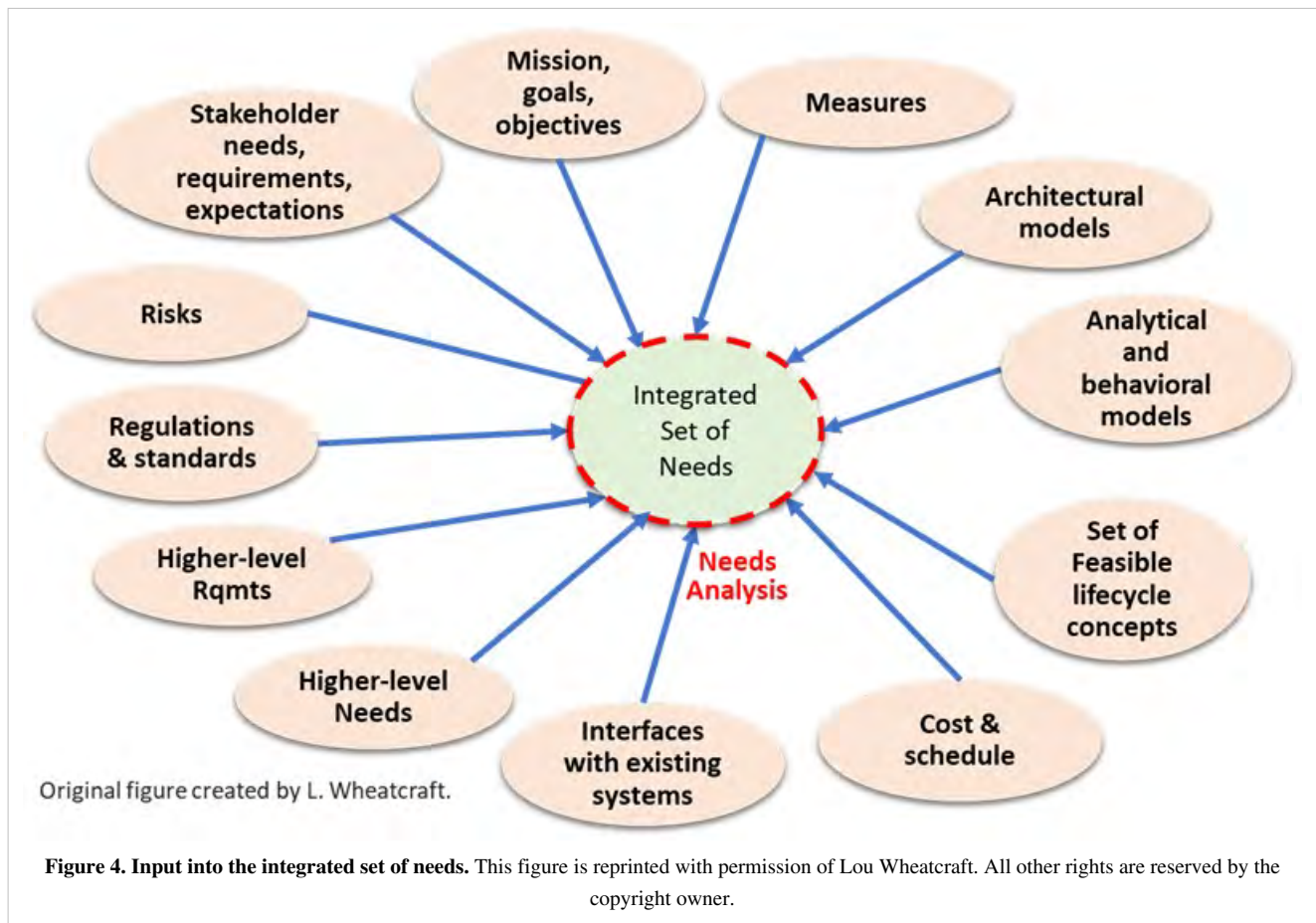
Life Cycle Concepts Analysis and Maturation

As a result of life cycle concept analysis and maturation activities, architectural and analytical/behavioral models are developed. Based on the resulting information, the preliminary set of life cycle concepts established in Business or Mission Analysis are transformed into a mature set of life cycle concepts that are consistent, correct, complete, and feasible. Models and diagrams (such as those used in Model-Based Systems Engineering (MBSE)) are excellent analysis tools for defining and maturing feasible life cycle concepts by providing a context for needs, and are key to help ensure correctness, completeness, and consistency of both individual needs and the integrated set of needs.

The logical architecture defines system boundary and functions, from which more detailed needs can be determined (interactions and interdependencies between logical elements of the system). As part of life cycle concept maturation, functions are defined, grouped logically, and relationships and interactions between those functions are captured. Supporting analytical and behavioral models can be developed to help assess behavior, interactions between parts of the architecture, and determine the performance characteristics of the functions.

Define and Baseline the Integrated Set of Needs

The project team derives an integrated set of needs that reflect the set of feasible system life cycle concepts, MGOs, measures, business operations level and system level stakeholder needs, drivers and constraints, and risk mitigation (Figure 4). These outcomes include results of the life cycle concepts analysis and maturation activity to determine expected functionality (what the stakeholders need the system to do), expected performance and quality ("how well" characteristics), the conditions of action, including triggering events, system states, and operating environments ("under what operating conditions"), as well as compliance with standards and regulations.



This integrated set of needs is what will be transformed into the set of design input requirements. In addition, it is this integrated set of needs which the design input requirements, design, and realized system will be validated against.

Needs are written in a structured, natural language from the perspective of what the stakeholders need the SoI to do. To help distinguish needs from the requirements, the needs statements do not include the word “shall” (or other word that depicts the statement is a requirement). It is recommended that need statements use a different format from requirements, such as: “The stakeholders need the system to” (INCOSE GtWR 2023). See Table 2 for example need statements.

Table 2. Example Need Statements. (SEBoK Original)

ID	Name	Need Statement	Rationale	Source
N1	Variable Temperature Settings.	The user needs the coffee maker to have two temperature settings (warm and hot) for the water temperature.	Focus groups provided input that a multi-select option for temperature is a desired feature.	Consumer input
N2	Prohibit Brew if Container Missing	The user needs the coffee maker to not brew unless a coffee container is in place.	Mitigation of risk of user error prior to starting coffee maker brew process.	Risk mitigation
N3	Coffee Maker Color Options	The company stakeholders need the coffee maker to come in four colors: black, grey, blue and red.	Marketing survey found that offering multiple colors provides competitive advantage with consumers.	Marketing stakeholder
N4	Ease of Use	The user needs the coffee maker to be easy to use (clearly defined user interface and a minimum of steps to get a cup of coffee).	Focus groups provided input that they are more likely to purchase products with simple user interfaces and operation controls.	Consumer input

The integrated set of needs must be recorded within the SoI's integrated dataset in a form and media suitable for review and feedback from the stakeholders, as well as a form that allows traceability. It is critical that the project team has confirmation from the stakeholders that their needs, requirements, expectations, MGOs, measures, drivers and constraints, and risk are properly communicated by integrated set of needs, this is supported by traceability. In a model-based systems engineering (MBSE) approach, the needs can be included in the system model, where they can be traced to their source (life cycle, stakeholder, MGO, risk, etc.).

Once the integrated set of needs is captured, they are used as inputs into the System Requirements Definition process.

References

Works Cited

- INCOSE. 2022. *INCOSE Needs and Requirements Manual*, version 1.1. INCOSE-TP-2021-002-01.
- INCOSE. 2022. *INCOSE Guide to Needs and Requirements*, version 1. INCOSE-TP-2021-003-01.
- INCOSE. 2023. *INCOSE Guide to Writing Requirements (GtWR)*, version 4. INCOSE-TP-2006-004-04.

Primary References

- INCOSE. 2022. *INCOSE Needs and Requirements Manual*, version 1.1. INCOSE-TP-2021-002-01.
- INCOSE. 2022. *INCOSE Guide to Needs and Requirements*, version 1. INCOSE-TP-2021-003-01.
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-118-99940-0.
- ISO/IEC/IEEE. 2018. *Systems and software engineering - Requirements engineering*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission/ Institute of Electrical and Electronics Engineers (IEEE), (IEC), ISO/IEC/IEEE 29148.
- ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Additional References

- INCOSE. 2023. *INCOSE Guide to Writing Requirements*, version 4. INCOSE-TP-2006-004-01.

Relevant Videos

- INCOSE Webinar, Life cycle Concepts and Needs Definition ^[1]
- How to Get Project Requirements from Project Stakeholders ^[2]

References

- [1] <https://youtu.be/hEGfNLvuyXo?list=PLVfZ7HbxxzBVnavoSAMUXjP49triK36KN>
- [2] <https://www.youtube.com/watch?v=YJ4wCbKJeXY>

System Requirements Definition

- Lead Author:
- Tami Katz
- Contributing Authors:
- Lou Wheatcraft and Mike Ryan

The System Requirements Definition process transforms the stakeholder view of desired capabilities into a technical, developer view of how the system can achieve those capabilities. System requirements describe requirements which the system-of-interest (SoI) must fulfill to satisfy the stakeholder needs and are expressed in an appropriate combination of well-formed textual statements and supporting models or diagrams. Inputs into this process are the life cycle concepts and integrated set of needs generated during the System Concept Definition activities.

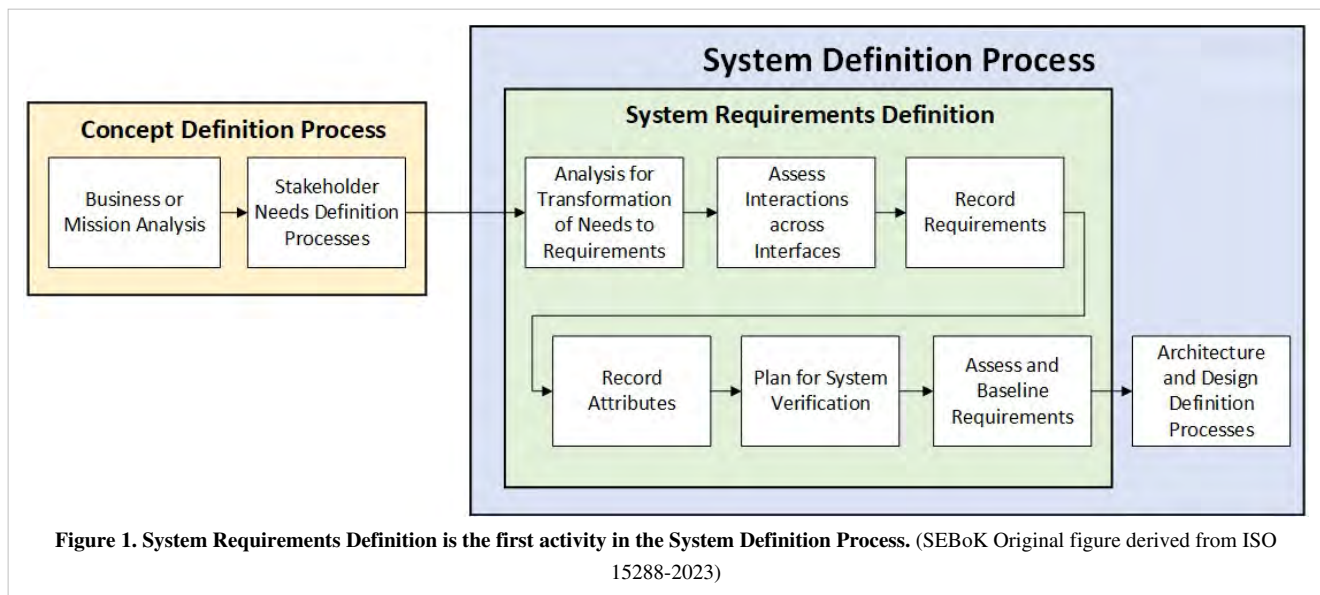
System requirements play major roles in systems engineering, as they:

- Form the basis of system architecture and design activities.
- Form the basis of system integration and verification activities.
- Provide a means of communication between the various project team members that interact throughout the project.

Outputs of the System Requirements Definition process serve as inputs to a number of other technical processes, which include System Design Definition, System Architecture Definition, and System Verification.

Principles and Concepts

System Requirements Definition uses the outcome of the System Concept Definition activities to address what the system must do so that the integrated set of needs will be realized by the SoI. As shown in Figure 1, this information is then provided as input to the System Architecture Definition and System Design Definition processes, as well as the System Verification process.



Defining requirements is not only an exercise in writing it is also an exercise in engineering. Every requirement represents an engineering decision as to what the SoI must do, or a quality the system must have, to meet the needs from which they were transformed. Determination of what the system must do to meet a need is through a process of detailed requirement analysis, which can include the development and use of models, simulations, and prototypes.

A requirement statement is the result of a "formal transformation of one or more needs or parent requirements into an agreed-to obligation for an entity to perform some function or possess some quality within specified constraints with acceptable risk." (INCOSE NRM 2022). In this context the use of the term "quality" means an inherent feature or a distinguishing attribute.

There are many types of requirements; this article describes the requirements which are used as inputs into the System Architecture and System Design Definition processes. As such, these requirements can be referred to as design input requirements, which are defined iteratively and recursively as the integrated system is decomposed into subsystems and system elements (the SoI could exist at any level of a system architecture).

Process for Generating System Requirements

Transforming Needs to System Requirements

The System Requirement Definition activities begin with the transformation of the integrated set of needs into a set of requirements for the SoI. These requirements must be appropriate to the level that the SoI exists within the system architecture and communicate "what" the SoI must do to meet the needs, avoiding requirements that state implementation of "how" to achieve the design realization of the physical SoI.

Needs communicate the stakeholder's perspective concerning their expectations of what they need the system of interest (SoI) to do from an external, black box view.

- Example: The user needs the coffee maker to stop heating water once the user-selected temperature has been achieved.

The sources of needs are described in Stakeholder Needs Definition. These sources include the stakeholder expectations and needs, drivers and constraints, risk analysis, and life cycle concept analysis and maturation activities. These needs are treated as inputs to the System Requirement Definition process, which results in a set of design input requirements for the SoI.

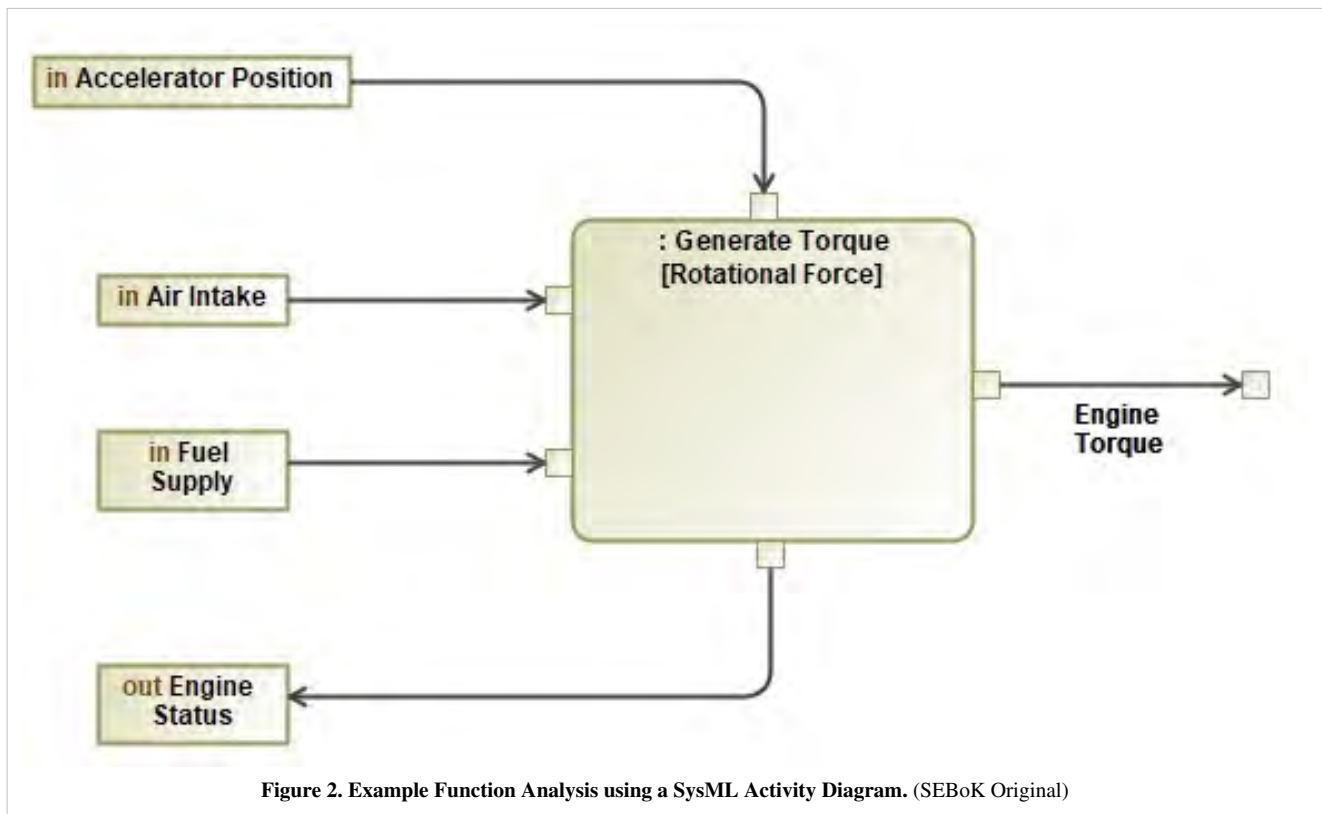
Requirements communicate the technical, developer perspective concerning what the SoI must do to meet the stakeholder needs from an internal white box view.

- Example: The Coffee Maker shall stop the heating function once the selected Brew Temperature has been achieved.

The process of generating a requirement statement is more than changing the subject of the need statement, it is an analysis of what the system must do to achieve what is needed. There are many types of analyses and tools that are used to make this determination:

- functional analysis,
- interface analysis,
- data flow diagrams,
- performance analysis, and
- needs to transformation matrix.

Requirement analysis for functions and performance can be performed by using system models (supported by Model-Based Systems Engineering (MBSE) applications). Key enabling model views include activity diagrams, sequence diagrams, state machine diagrams, and structure diagrams (for hierarchy and interfaces). An example functional analysis is shown in Figure 2.



Close and frequent coordination with the stakeholders is necessary to ensure the transformation is accurate and traceability is maintained. This results in a set of system requirements communicating measurable characteristics which can form the basis for system realization. A detailed analysis of a single need statement may result in multiple requirements expressing what the system must do to meet it, including definition of measurable performance criteria (INCOSE NRM 2022).

Use of Attributes

An attribute is additional information associated with an entity which is used to aid in its definition and management. Well-chosen attributes, when properly defined and tracked, can enable correct interpretation and management of requirements throughout the system life cycle. There are several useful attributes to consider:

- rationale,
- trace to source or parent,
- system verification success criteria,
- owner,
- category,
- status,
- criticality, and
- priority.

The use of the rationale attribute helps communicate why the requirement is needed, any assumptions made, the source of numbers, the results of related design studies, or any other related supporting information. This supports further requirements analysis and decomposition, as well as identifying the source of any requirement value. Defining the system verification success criteria helps ensure the requirement is well-formed, is verifiable, and aids in the planning for the project's verification program (INCOSE NRM 2022).

Categorizing Requirements

It is useful to organize the requirements into categories, grouping similar types of requirements together within a set. An example set of categories is shown in Table 1. While organizations may have different categorizations, for the set of requirements to be complete *each* category topic in Table 1 must be addressed.

Table 1. Example Types of Requirement Categories. (Derived from the INCOSE Needs and Requirements Manual)

Category	Description
Function/ Performance	The primary functions and associated performance that the SoI needs to perform in terms of its intended use. The functions address the capabilities and features the stakeholders expect the SoI to have; performance addresses how well, how many, how fast attributes of the function. Many of the primary functions involve interactions (interfaces) between the SOI and systems external to the SOI. All critical and high priority needs would be included in this category.
Fit/Operational	Requirements dealing with functions that deal with a secondary or enabling capabilities, functions, and interactions between the SoI and external systems needed for the system to accomplish its primary functions. This includes functions concerning the ability of the system to interface with, interact with, connect to, operate within, and become an integral part of the macro system it is a part. Fit includes human system interactions and interfaces as well as both the induced and natural environments (conditions of operations, transportation, storage, maintenance). For example, needs associated with safety, security, power, cooling, transportation and handling, storage, maintenance, and disposal.
Form	Physical Characteristics. The shape, size, dimensions, mass, weight, and other observable parameters and characterizes that uniquely distinguish a system. For software, form could address programming language, lines of code, memory requirements.
Quality	Fitness for use. For example, various “-ilities” such as reliability, testability, operability, availability, maintainability, operability, supportability, manufacturability, and interoperability.
Compliance	Conformance with design and construction standards and regulations.

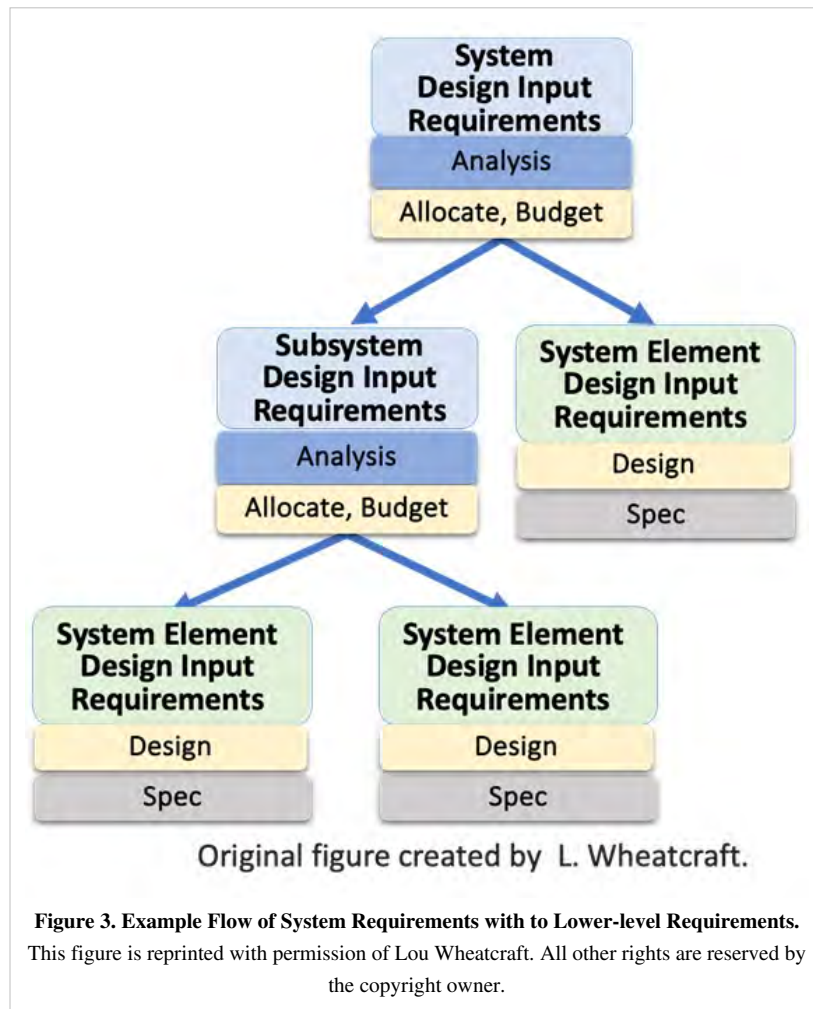
See Table 2 for example requirement statements, attributes and categories.

Table 2. Example Requirement Statements. (SEBoK Original)

Req ID	Requirement Title	Requirement	Rationale	Category
R1	Variable Temperature Settings	The Coffee Maker shall have two user-selectable settings for coffee brewing: Warm: 96°C +/- 2°C, Hot: 100°C +/- 2°C.	Based on focus group inputs for selectable temperature; values are from analysis associated with consumer research surveys and safety regulations.	Function/Performance
R2	Prohibit Brew if Container Missing	If coffee container is not fully inserted into the brew location, the Coffee Maker shall prohibit brew function.	Protective measure related to off-nominal use case.	Function/Performance
R3	Operational Life	The Coffee Maker shall have an operational life greater than or equal to 3 years.	Analysis shows expected operational service of 1,000 hours over three years of usage, ensuring appliance lasts through the warranty period.	Quality
R4	User Inputs	The Coffee Maker shall limit the number of user inputs to: Power On, Brew Temperature, Brew Size, Brew Start.	User inputs are assessed from focus groups to minimum set of inputs to achieve coffee maker full set of life cycle concepts.	Fit/Operation

Requirements At Levels Within the Hierarchy

Requirements definition is an iterative and recursive process performed concurrently with the Architecture Definition Process. Upon determination of the system hierarchy, a requirement tree can be established showing the system level requirements and the supporting requirements at the lower-level elements of the hierarchy. Figure 3 shows an example flow from system requirement to lower-level requirements, and an example requirement tree is shown in Figure 4.



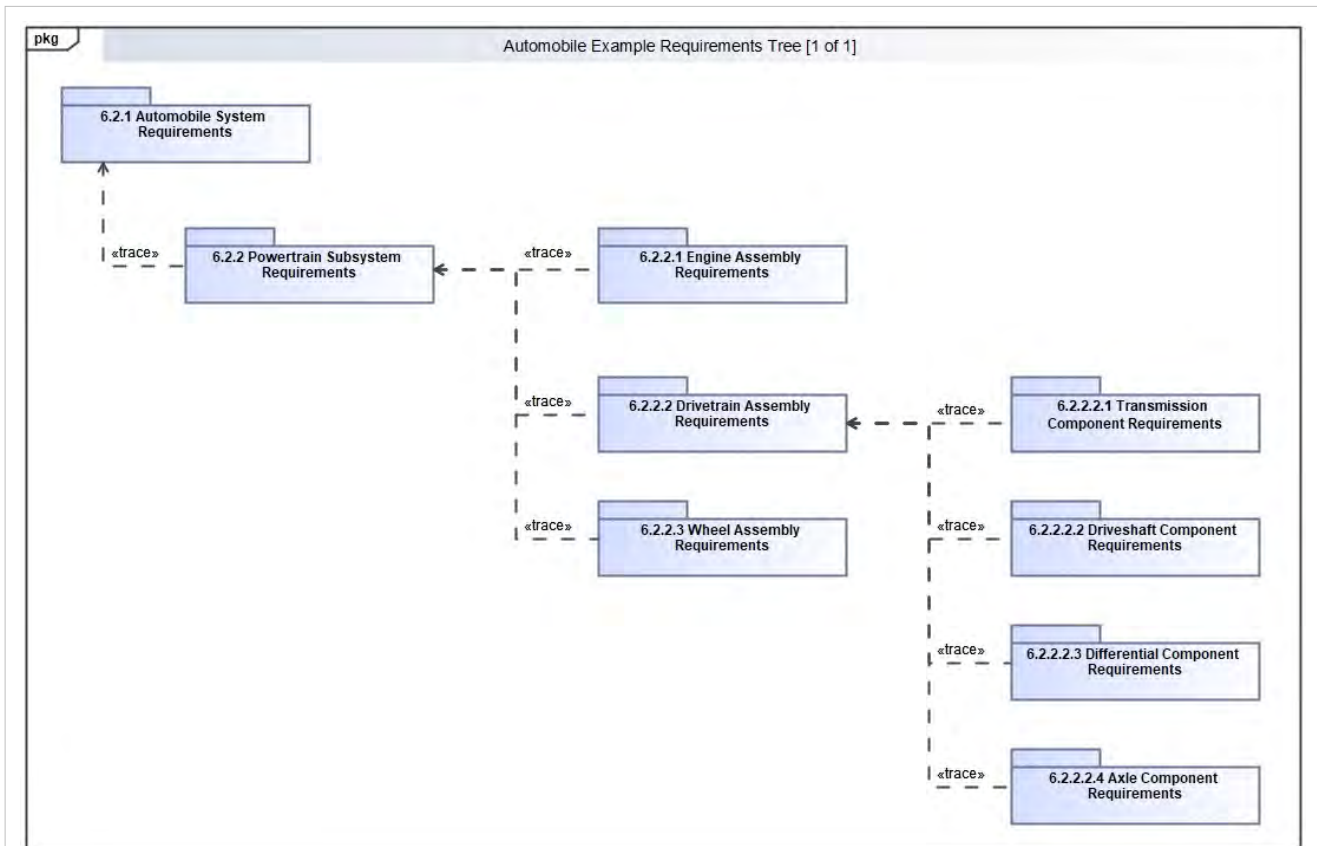


Figure 4. Example Requirement Tree using a SysML Package Diagram. (SEBoK Original)

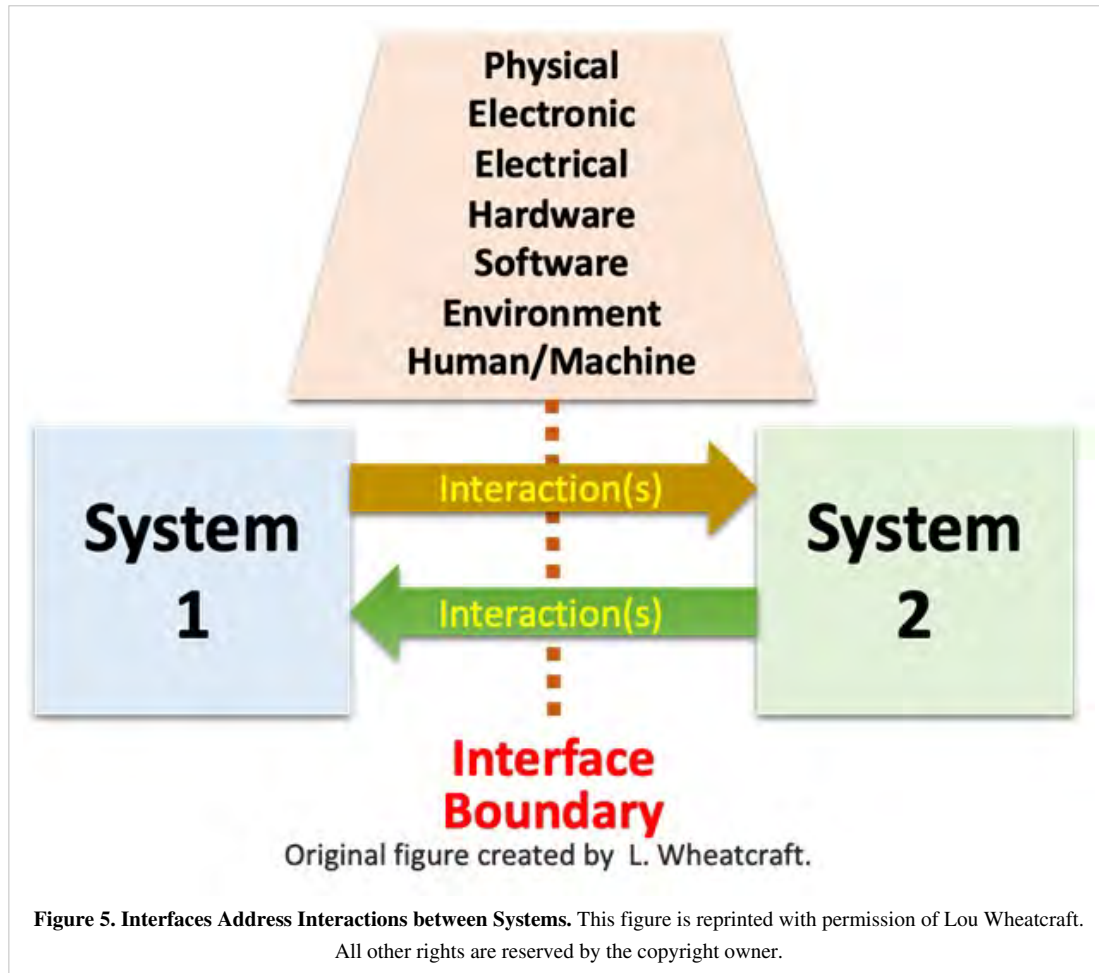
Allocation is the process by which the requirements at one level of the physical architecture are assigned to those entities at the next lower level of the architecture that have a role in the implementation of the allocated requirement. This involves an analysis where the project team determines what “role”, if any, each subsystem or system element at the next level of the architecture has in the implementation of the requirement being allocated. Requirements generated at the lower level are referred to as child requirements of an allocated parent requirement.

An important concept associated with allocation is budgeting. Budget refers to the total value of a parameter defined at the system level. The system requirement value may be decomposed to ensure the lower-level elements contribute their portion, enabling the system level to achieve its total value. Budgeted quantities result in requirements that have a dependency - a change in one will result in the need to change another. Budgeted values can include mass (weight), power usage, bandwidth, time, and quality attributes).

Principles Governing System Requirements

Addressing Interfaces and Interactions

For each part of the architecture, an external interface diagram provides the ability to address interactions between the SoI and external systems (Figure 5).



The phrase “interface requirement” refers to the specific form or template for a functional requirement that deals with an interaction of a system across an interface boundary with another system. Writing interface requirements is a three-step process (INCOSE NRM 2022):

1. Identify the interface boundaries and interactions across those boundaries.
2. Define the interactions across the interface boundaries (the characteristics of what is crossing the boundary, and media involved).
3. Write the interface requirements.

Example interface requirement:

- The System shall send telemetry to the Ground System as defined in ICD 123, Table X.

Model Form of Requirements

Requirements can be recorded and managed via a document, database, or in the form of a model:

- Models provide the capability to address completeness in terms of functions, inputs to those functions, sources of those inputs, outputs, and customers (destinations/users) for those outputs.
- Models help facilitate communication by making complex systems and processes easier to understand (a picture is worth a thousand words....).
- Models enable the capture of interdependencies of requirements to other aspects of the system dataset, such as inputs, needs, architecture, test cases, etc.
- Models enable quantitative analyses and execution of simulations.

In MBSE, the model form of requirements can be expressed as a diagram (Figure 6) or within a table in the modeling tool (Figure 7), using well-formed textual statements as part of a requirement model element. Refer to Requirements Management for further guidance on usage of tools associated with textual and model requirements.

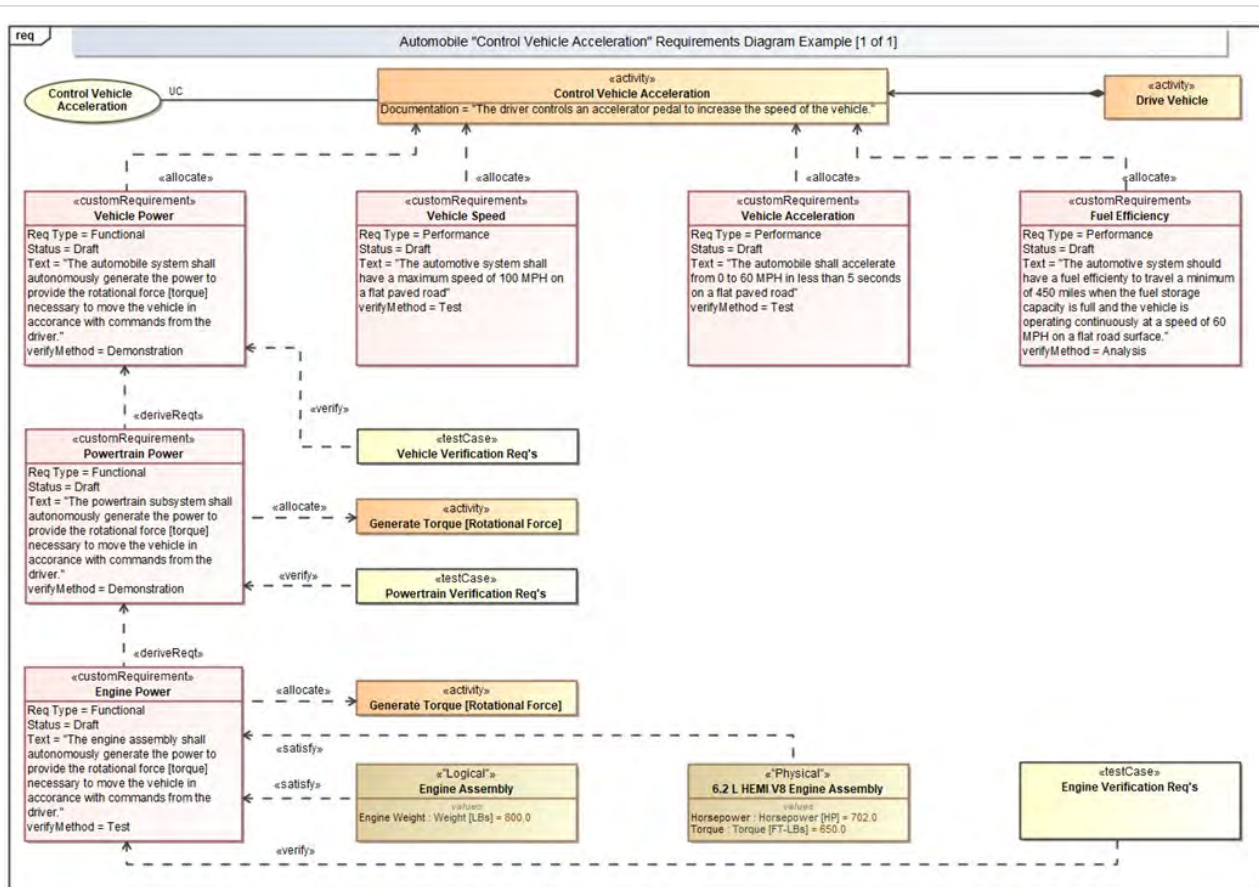


Figure 6. Example of a Model Based Requirement Diagram. (SEBoK Original)

Name	Text Type	Derived From	Req Type	Text	Status	Function Allocation	Satisfied By	Verify Method	Documentation
11 Powertrain Requirements	Header								Powertrain Subsystem Specification
11.1 Powertrain Power	Requirement	12.1 Vehicle Power	Functional	The powertrain subsystem shall autonomously generate the power to provide the rotational force [torque] necessary to move the vehicle in accordance with commands from the driver.	Draft	Generate Torque [Rotational Force]	Engine Assembly 6.2 L HEMI V6 Demonstration Engine Assembly		
11.2 Powertrain Speed	Requirement	12.2 Vehicle Speed	Performance	The powertrain subsystem shall generate sufficient power to maintain a <u>maximum vehicle speed of 100 MPH</u> on a flat paved road.	Draft	Generate Torque [Rotational Force]	Engine Assembly 6.2 L HEMI V6 Test Engine Assembly		
11.3 Powertrain Acceleration	Requirement	12.3 Vehicle Acceleration	Performance	The powertrain subsystem shall generate sufficient power to accelerate the vehicle from 0 to 60 MPH in <u>less than 5 seconds</u> on a flat paved road.	Draft	Generate Torque [Rotational Force]	Engine Assembly 6.2 L HEMI V6 Test Engine Assembly		

Figure 7. Example of a Model Based Requirement Table. (SEBoK Original)

Addressing Unknowns

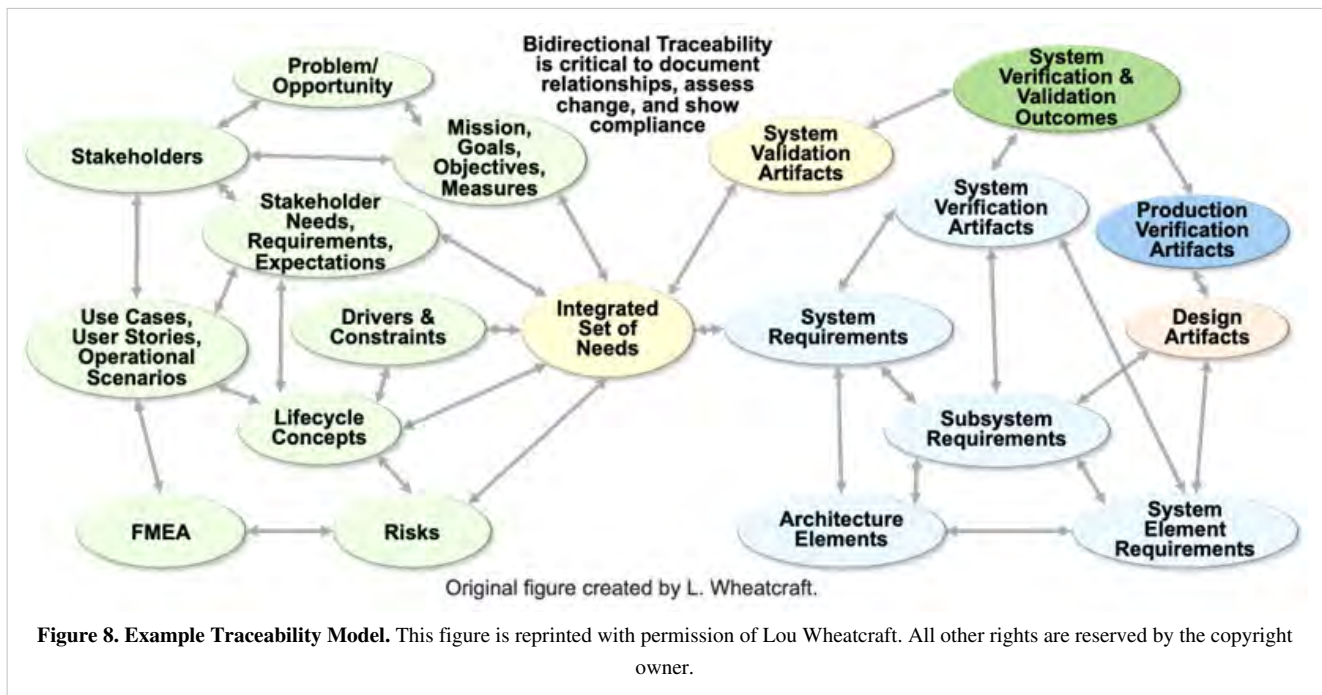
When initially generating a set of requirements, there is often a set of parameters which may require future analysis to determine their actual value. One common method to do this is to use a placeholder in the requirement statement to indicate the requirement is drafted, yet further work is needed before it is considered complete. When a value is unknown, a common approach is to use the "To Be Determined" (TBD) indication within the requirement statement, in place of an actual value. When a value is determined but confidence is low (e.g., feasibility has not been assessed), a method to reflect this is to use the "To Be Resolved" (TBR) indication next to the value. Together, TBRs and TBDs are referred to generically as "TBXs". Keeping track of TBXs allows for awareness of the maturity of the requirements throughout the life cycle and is a valuable metric to evaluate completion and maturity of the integrated set of needs and set of design input requirements.

TBXs can lead to risk if not addressed by the project team prior to significant work in establishing the design solution. TBX management is the process of identifying the TBXs in a requirement set, establishing the work needed to resolve closure of the issue, identifying the owner of the work, and systematically iterating through and completing the actions to enable removal of these placeholders in the requirements.

Traceability

Traceability is a powerful way to manage the design input requirements, especially across levels and across subsystems and system elements within a specific level as well as manage the requirements across the life cycle. Traceability helps ensure the requirements trace to their needs, and that they are correct and complete. Traceability also helps identify allocation of requirements to lower-level system elements of the architecture. Traceability is used to access any impacts of changes to requirements or other types of data across the life cycle, enabling insight into whether that change is beneficial or costly.

Defining a traceability relationship model at the beginning of the project enables an understanding of which relationships will be established and managed via traceability throughout the development of the SoI. An example traceability model is shown in Figure 8 (INCOSE NRM 2022).



Planning for System Verification

A best practice is to do early planning for how the project will perform system verification. Defining the system verification success criteria, strategy, and method when the requirements are generated helps ensure the requirement statements are worded properly, are within the scope of the project plans, and demonstrate they have the characteristic of "verifiable". This information can be captured within the attributes defined for the requirement.

Assessing the Requirements

Requirements should be assessed to gauge whether they are well-formed and that they meet the intent of the needs from which they were transformed. Concepts behind assessment of the requirements are captured with the terms "requirement verification" and "requirement validation", which are not the same concepts described for system verification and system validation.

- Requirement verification activities address: Are the requirements well-formed, i.e., do the individual requirements and sets of requirements have the characteristics and follow the rules defined in the organizations criteria for well-formed requirements?
- Requirement validation activities address: Do we have the right requirements, i.e., do the requirements clearly communicate the intent of the needs, parent requirements, and other sources from which they were transformed, in a language understandable by the architectural definition, design definition, and manufacturing/coding teams?
Validation addresses if the requirements are correct and are also achievable.

There are several methods that can be used to verify and validate requirements, such as standard peer review techniques and comparison of each requirement against a set of requirements characteristics and the integrated set of needs. For requirements communicated in a textual form, guidelines exist for writing well-formed requirements; they include recommendations about the syntax of requirements statements, wording (exclusions, representation of concepts, etc.), and characteristics (specific, measurable, achievable, feasible, testable, etc.) (INCOSE GtWR 2023).

System Requirement Definition Artifacts

The System Requirements Definition process results in a variety of artifacts:

- well-formed requirements recorded in models, databases, or documentation,
- requirement tree,
- traceability of requirements to other data,
- defined interactions across boundaries,
- performance budgets, and
- initial system verification plans.

Requirements Management

Requirements Management (RM) is performed to ensure alignment of the system, subsystem, and system element requirements with other representations, analyses, and artifacts of the system across the life cycle. It includes providing an understanding of the requirements, obtaining commitment, managing changes, maintaining bi-directional traceability among the requirements and with the rest of the system definition, and alignment with project resources and schedule. The Requirements Management article provides additional guidance on methods for performing requirements management.

References

Works Cited

INCOSE. 2022. *INCOSE Needs and Requirements Manual (NRM)*, version 1.1. INCOSE-TP-2021-002-01.

INCOSE. 2023. *INCOSE Guide to Writing Requirements (GtWR)*, version 4. INCOSE-TP-2006-004-04.

Primary References

ISO/IEC/IEEE. 2018. *Systems and Software Engineering - Requirements Engineering*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 29148.

ISO/IEC/IEEE. 2023. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023.

INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-118-99940-0.

INCOSE. 2022. *INCOSE Needs and Requirements Manual (NRM)*, version 1.1. INCOSE-TP-2021-002-01.

INCOSE. 2022. *INCOSE Guide to Needs and Requirements (GtNR)*, version 1. INCOSE-TP-2021-003-01.

INCOSE. 2023. *INCOSE Guide to Writing Requirements (GtWR)*, version 4. INCOSE-TP-2006-004-04.

Additional References

INCOSE. 2022. *INCOSE Guide to Verification and Validation (GtVV)*, version 1. INCOSE-TP-2021-004-01.

Relevant Videos

- INCOSE Requirements Working Group YouTube Channel ^[1]

References

[1] <https://www.youtube.com/@incoserwg891>

Knowledge Area: System Architecture Design Definition

System Architecture Design Definition

Contents of this Knowledge Area

- Functional Architecture (Caitlyn Singam and Jeffrey Carter)
- Logical Architecture (Alan Faisandier and Garry Roedler) (Rick Adcock)
- Physical Architecture (Alan Faisandier and Rick Adcock)
- Lead Authors:
- Jeffrey Carter and Caitlyn Singam

The system architecture design defines system behavior and structure characteristics in accordance with derived requirements. The architecture ensures that the system elements operate together in the applicable operating environment to satisfy Stakeholder needs. Figure 1 illustrates the system design phenomenon based on Hamilton's Principle: a system is composed of interacting elements exchanging data, energy, or mass to impact the state of cooperating elements resulting in continuous, discrete, or emergent behaviors at progressive levels of aggregation or decomposition.

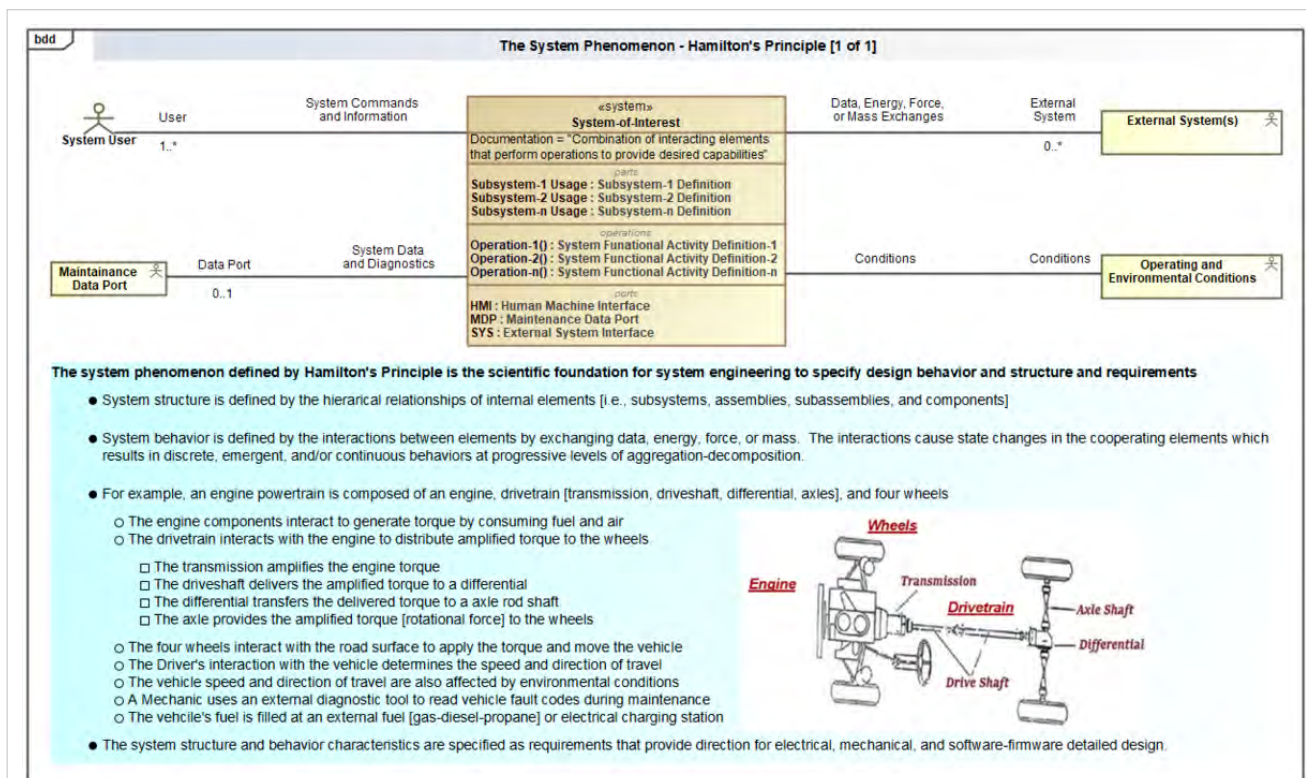


Figure 1. The System Phenomenon – Hamilton's Principle (SEBoK Original)

Systems Engineering has traditionally applied intuitive domain-specific [e.g., aerospace, defense, automotive, consumer, ...] product practices emphasizing processes and procedures. The practices in conjunction with proficient writing skills manually organize design configuration information in a disparate collection of documents. The documents depict the system architecture design with textual description and graphical figures using unique

organizational adopted notation without formal semantics. The documents require manual updating to synchronize design configuration information resulting in consistency issues in addition to cost and schedule inefficiencies.

This article describes a model-based system architecture design definition approach with the Systems Modeling Language [SysML] for product development, integration, and verification. This approach enables digital transformation design initiatives to employ Model-Based System Engineering [MBSE] practices similar to other design engineering disciplines [EE, ME, SW].

SysML provides industry standard graphical and textual [added with SysML-v2] notations with formal semantics to define system design requirements, behavior and structure characteristics with traceability to the associated requirements. The system design model provides an Authoritative Source of Truth [ASoT] digital repository of the project technical baseline including requirement, functional, logical, and physical design representations for each system, subsystem, and component design abstraction level. Integrated system design model simulation capabilities with verification criteria of end-to-end system “digital threads” enable evaluation of key performance parameters to verify design decision or discover and resolve design defects in digital environments. The system design model with integrated simulation capabilities provides a system “digital twin” as an authoritative representation of the physical system including digital thread end-to-end simulations with all the data, models, and infrastructure necessary to optimize the system design digitally and explore design alternatives for system improvements to evolving missions and threats. The approach is tailorable to satisfy specific project objectives.

System Architecture Design Overview

System architecture design definition includes system behavioral and structural analyses to define a holistic technical solution that satisfies Stakeholder needs. The model-based system architecture design definition approach provides an implementation for the ISO/IEC/IEEE-15288 Architecture Definition technical lifecycle process. The approach defines practices to develop, integrate, and employ digital system design models for cross-domain collaboration by applying product domain-specific design knowledge and expertise.

The system architecture design defines the structural elements and their behavior interactions in the applicable operating environment to satisfy Stakeholder needs. The architecture includes a project overview with representations of system requirements, functional behavior & interactions, logical and physical configuration items. Figure 2 illustrates the system architecture design definition approach.

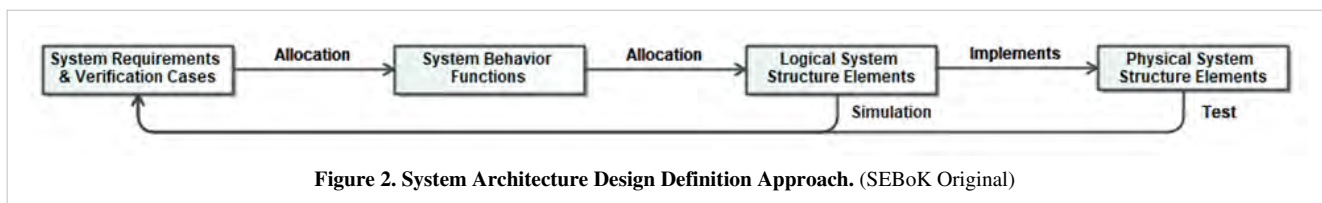


Figure 2. System Architecture Design Definition Approach. (SEBoK Original)

The following describes the model-based system architecture design definition approach activities and information for the digital system design model.

1. Provide a project overview by describing the scope of the project and the system/project boundaries.
 1. System mission description relative to the project's Stakeholder needs with requirements traceable to the system needs.
 2. Glossary of terms with descriptions of the design model organization, environment, development approach, SysML conventions and custom profiles
2. Define the system requirements and verification criteria derived from the Stakeholder Needs to specify the system capabilities, functions, interfaces, performance, characteristics, operating conditions and environments.
 1. System use cases reflect black-box functionality, solution constraints, boundaries, external human and system interfaces.

2. Capabilities and requirements derived from the Stakeholder's needs verified with engineering analysis, demonstration, inspection, and test results throughout the hierarchical system structure of elements.
 3. Define the system functional hierarchical functions and subfunctions with their behavior interactions to satisfy Stakeholder needs. Each function defines input-to-output transformations to satisfy one or more allocated requirement(s).
 1. System use case elaboration identifies system capabilities with activity, sequence, and state machine diagrams to describe system functional operations decomposed into User tasks, system functions and subfunctions with the associated requirements and verification criteria.
 2. System element interactions define how the functions and subfunctions operate together via interfaces to provide capabilities.
 4. Define the logical system structural elements to identify logical configuration items without a specific design implementation, referred to as the technology Platform Independent Model [PIM]. Each logical configuration item has a set of allocated functions and subfunctions with specified property attributes, interfaces, and functional performance requirements for programmatic make vs. buy decisions. Also, each logical element has a corresponding physical design element with additional lower-level design details. For example, an automobile design will specify a logical combustion engine with requirements to establish a contractual baseline for production/procurement.
 1. Recursive white-box analyses identify logical configuration items with allocated functions, subfunctions, and interfaces for implementation by the corresponding physical configuration item.
 2. Logical configuration items include hierarchical subsystems, assemblies, subassemblies, and components based on the system complexity.
 5. Define the physical system structural elements to identify a specific design implementation for each logical structural element, referred to as a technology Platform Specific Model [PSM]. The physical configuration items comply with the allocated requirements and have a specific manufacturer's part number identification. For example, an automobile will have a specific V8 gas, diesel, or propane combustion engine in accordance with the requirements and include additional physical engine component configuration items [e.g., pistons, spark plugs, cam shaft, ...].
 1. Recursive design analyses with electrical, mechanical, and software engineering to identify the specific design implementation for physical configuration items [i.e., manufacture and part number identification].
 2. Physical configuration items provide a design implementation for the corresponding logical element including the required property attributes and functional operations.
-

System Architecture Design Definition Model

Figure 3 provides an example for the organization of the digital system design model. The design model provides an Authoritative Source of Truth [ASoT] repository for a project's system technical baseline including requirements, functional, logical, and physical design representations for each system, subsystem, and component design abstraction level. The design model also provides design libraries for product-line management and reuse.

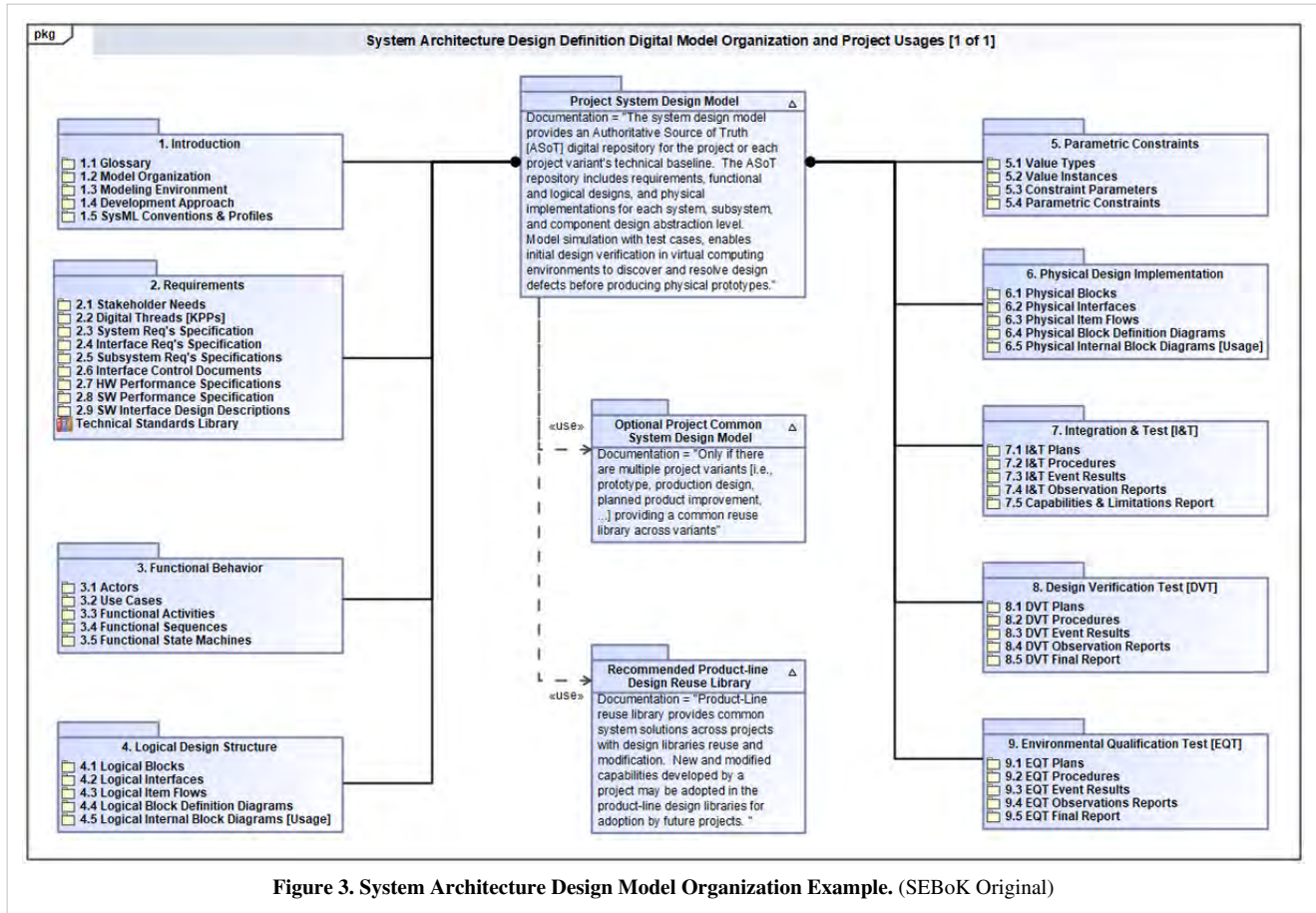


Figure 3. System Architecture Design Model Organization Example. (SEBoK Original)

The digital system design model enables the following:

- Specifies system behavior and structure characteristics with traceability to the associated requirements.
- Provides an Authoritative Source of Truth [ASOT] digital repository of a project's approved technical baseline across engineering, production, and sustainment including requirement, functional, logical, and physical design representations for each system, subsystem, and component design abstraction level for collaboration and decision making.
- Provides product design libraries to adapt product designs to stakeholder needs with new, modified, and existing system capabilities.
- Integrated system design model simulation capabilities with verification test cases of end-to-end system “digital threads” enable evaluation of key performance parameters to verify design decision, discover and resolve design defects in digital environments before the expense of producing prototypes.
- Assessment of design changes in all model usages for compliance, with any issue(s) flagged for corrective action.
- Provides a system design model with integrated simulation capabilities providing a system “digital twin” representing an authoritative representation of the physical system including digital thread end-to-end simulations with all the data, models, and infrastructure necessary to optimize the system design digitally and explore design alternatives for system improvements to evolving missions and threats.
- Provides design model scripts to export functional & interface specifications, design & requirements traceability and design descriptions reports.

System Architecture Design Definition Model Viewpoints and Views

The architecture model includes multiple views of the system design features with the information necessary to address each stakeholder's viewpoint and concerns. Figures 4a and 4b provide typical system views and viewpoints which are tailorable by specific projects, if necessary.

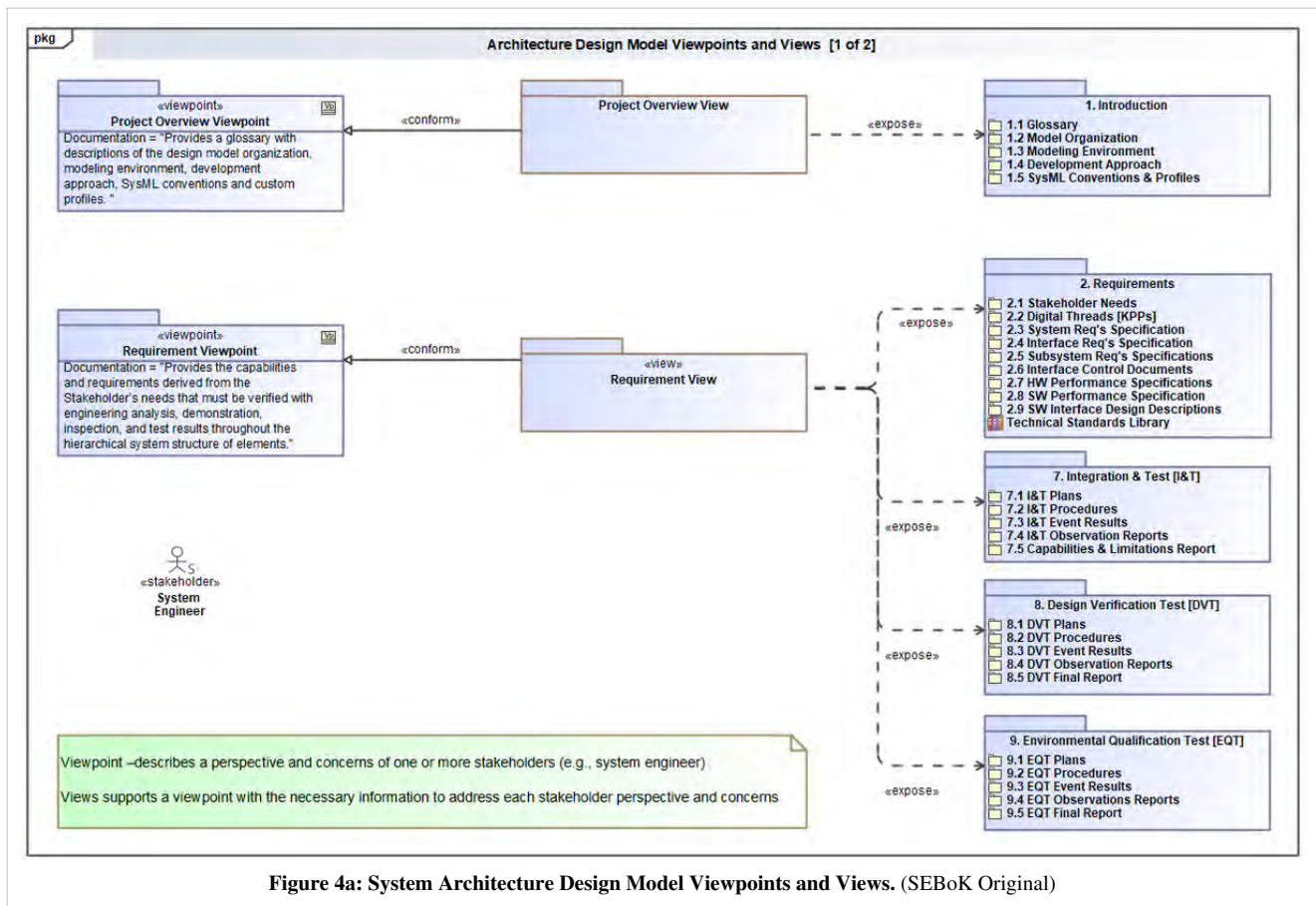


Figure 4a: System Architecture Design Model Viewpoints and Views. (SEBoK Original)

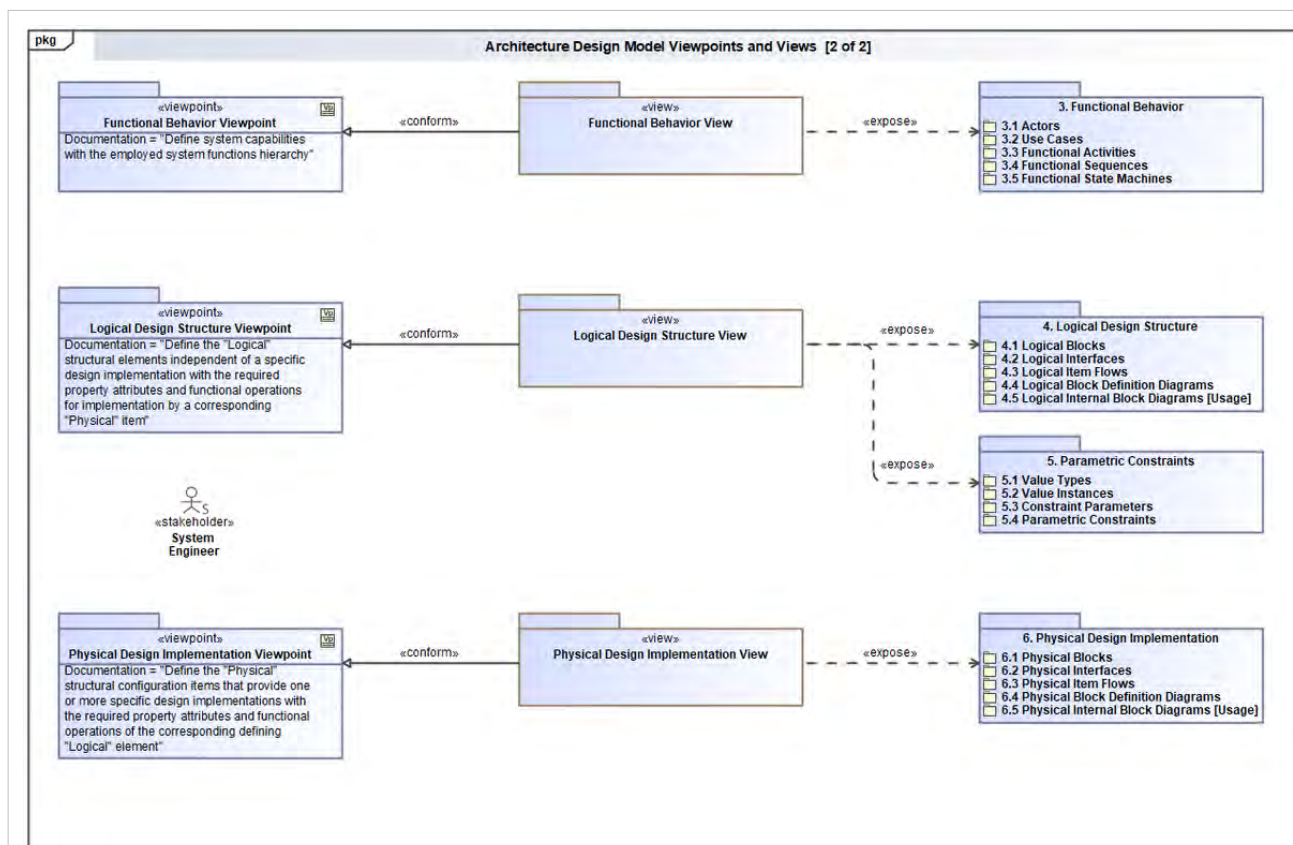


Figure 4b: System Architecture Design Model Viewpoints and Views. (SEBoK Original)

System Architecture Design Definition Modeling Ecosystem

Digital system design model integration within an engineering development ecosystem including electrical, mechanical, software and specialty engineering models enables system-level simulations. The system simulations provide initial design verification in digital environments to discover and resolve design defects before producing physical prototypes. Figure 5 provides an example of a model-based system design ecosystem.

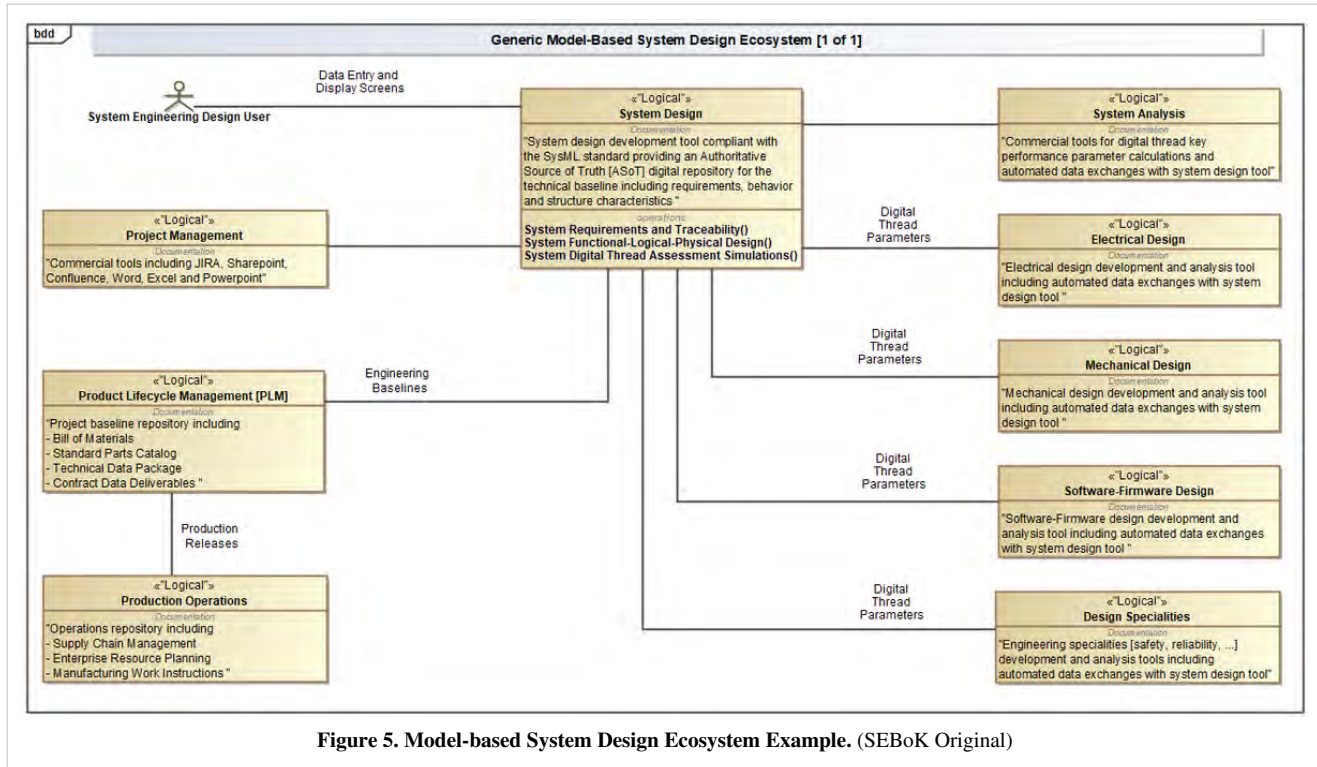


Figure 5. Model-based System Design Ecosystem Example. (SEBoK Original)

The digital system design model provides an executable representation of a system in terms of the hierarchical structural elements and their behavior interactions to satisfy Stakeholder needs in the applicable operating environment.

- Hierarchical structural elements include subsystems, assemblies, subassemblies, and components enabling separation of concerns between each design abstraction level.
- Element interactions involve the exchange of data, energy, force, or mass that trigger state changes in the cooperating elements resulting in discrete, emergent, or continuous behaviors at progressive levels of aggregation or decomposition.

The executable design model enables digital simulations to confirm system functional performance, accuracy, timeliness, and stability with verification criteria. In addition, the system response to operating condition variations is evaluated including deterministic and stochastic disturbances to discover design defects and enhancements before the expense of producing physical prototypes. The governing principle is to develop digital system design models with high-fidelity simulation capabilities to realistically emulate systems to create a digital twin with digital threads to evaluate design alternatives in virtual computing environments without producing prototypes as shown in Figure 6.

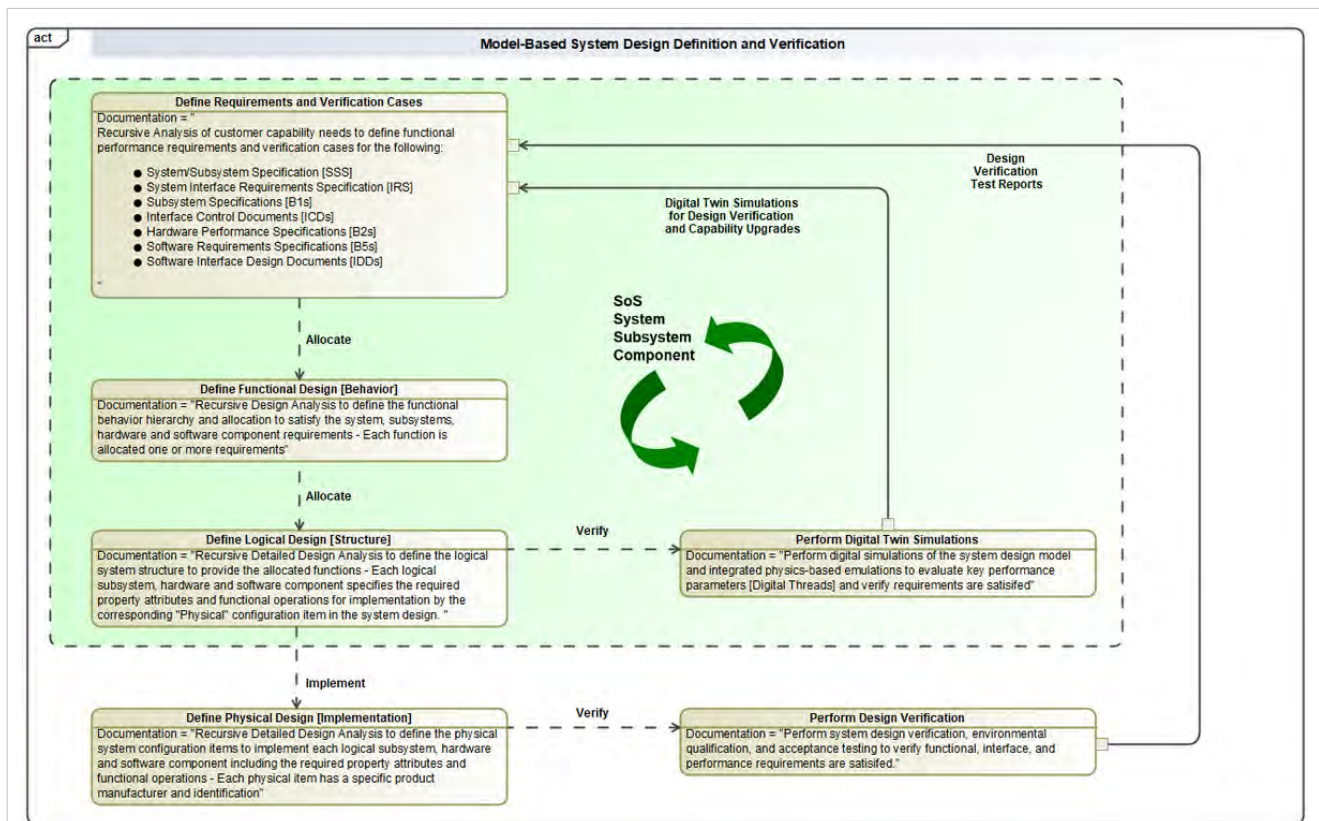


Figure 6. System Architecture Design Definition and Verification. (SEBoK Original)

- Digital threads are analytical frameworks providing end-to-end system simulation representations to evaluate key performance parameters in virtual environments by exchanging information between different modeling tools across the lifecycle.
- Digital twins are authoritative representations of physical systems including the digital thread end-to-end connections with all the data, models, and infrastructure needed to create and optimize a system's lifecycle digitally. Digital twins enable project team collaboration, system simulation performance assessments, design change impact evaluations, and product-line management reuse libraries.

System Architecture Design Definition Illustrative Example

Figure 7 provides an example automobile product to illustrate how the digital design model can specify, analyze, and design the system. The scope of the example is an automotive powertrain subsystem consisting of an engine, drivetrain, and four-wheel assemblies. The drivetrain assembly consists of transmission, driveshaft, differential, and two axle shaft rod subassemblies. The intent is to demonstrate the system architecture design definition approach and design solution accuracy is not a high priority.

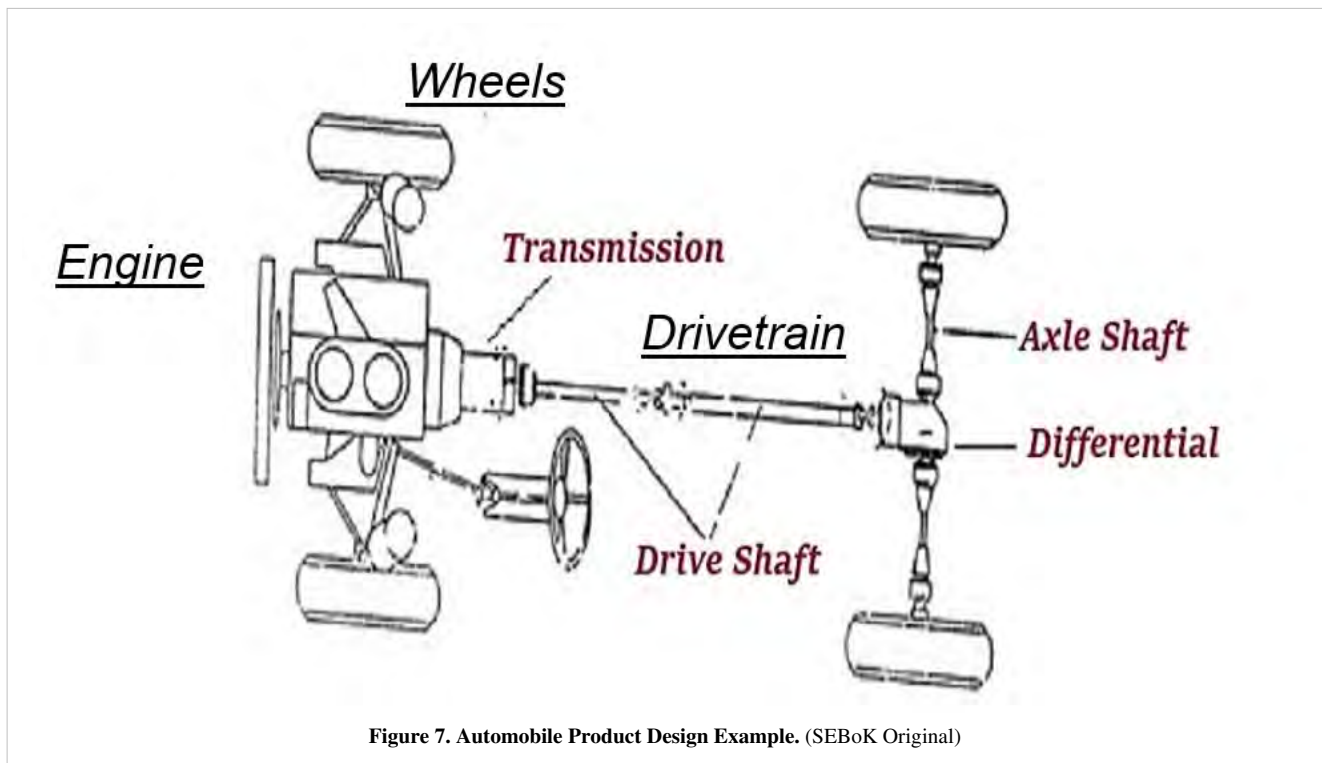


Figure 8 provides the automobile system context and use cases. The diagram defines the automobile system boundary, Stakeholder capability needs, and external interfaces with. The design analyses will focus on the “Drive Vehicle” capability which includes the “Control Vehicle Acceleration” function allocated to a logical and physical system “Powertrain” subsystem.

- The powertrain subsystem is composed of engine, drivetrain, and wheel assemblies.
- The engine assembly components interact to generate torque by consuming fuel and air.
- The drivetrain assembly is composed of transmission, driveshaft, differential, and two axle rod shaft subassemblies. The drivetrain interacts with the engine to distribute amplified torque to the wheels.
 - The transmission subassembly amplifies the engine torque.
 - The driveshaft subassembly delivers the amplified torque to a differential.
 - The differential subassembly transfers the delivered torque to axle rod shaft.
 - The axle shaft rod subassembly provides the amplified torque [rotational force] to the four wheels.
- The four-wheel assemblies interact with the road surface to apply the torque and move the vehicle.
- The Driver's interaction with the vehicle determines the speed and direction of travel.
- The vehicle speed and direction of travel is affected by environmental conditions.
- A Mechanic uses an external diagnostic tool to read vehicle fault codes during maintenance.
- The automobile's fuel is filled at an external fuel [gas-diesel-propane] or electrical charging station.

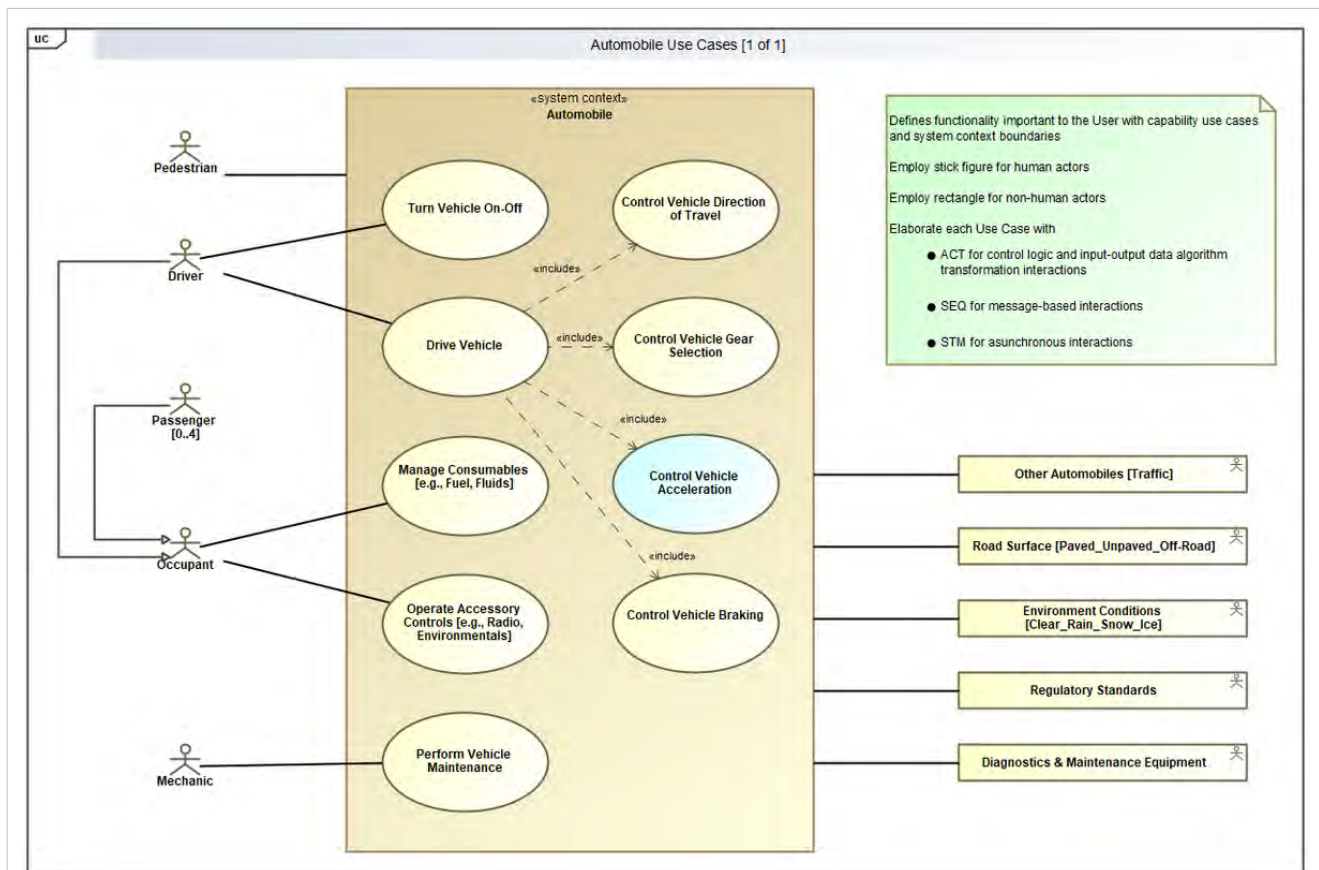


Figure 8. Automobile System Context and Use Cases Example. (SEBoK Original)

The following three sub-articles [functional, logical, and physical architecture] illustrate the model-based system architecture design definition approach for the automobile system example to define a holistic solution. The functional architecture article has been provided. The logical and physical architecture articles will be updated in future SEBoK releases.

References

Works Cited

Bill Schindel; *SEBoK Part-3: Systems Engineering STEM Overview*

Will Roper. *There is No Spoon: The New Digital Acquisition Reality*. US Air Force: Assistant Secretary of the Air Force, 07-October 2020

U.S. DOD; *Digital Engineering Strategy*; Office of the Deputy Assistant Secretary of Defense for Systems Engineering. June 2018.

Primary References

- INCOSE, *Systems Engineering Handbook – 5th Edition: A Guide for System Life Cycle Processes and Activities*, John Wiley & Sons Ltd., 2023
- ISO/IEC/IEEE-15288, *Systems and Software Engineering: System Life Cycle Processes*, 2023
- ISO/IEC/IEEE-42010, *Software, Systems and Enterprise – Architecture Description*, 2022
- OMG; *Systems Modeling Language [SysML] Standard – v1.7*; August 2022
- OMG; *Systems Modeling Language [SysML] Standard – v2 beta 2*; April 2024
- OMG; *Model Driven Architecture Guide*; 2003
- Lenny Delligatti; *SysML Distilled: A brief Guide to the Systems Modeling Language* Addison-Wesley Professional; 2013
- Sanford Friedenthal, Alan Moore, Rick Steiner; *A Practical Guide to SysML: The Systems Modeling Language – 3rd Edition*; Morgan Kaufmann; October 2014
- Tim Weilkiens, Jesko G. Lamm, Stephan Roth, Markus Walker; *Model-Based System Architecture – 2nd Edition*; John Wiley & Sons Ltd., 2023
- Tim Weilkiens; *Systems Engineering with SysML/UML: Modeling, Analysis, Design*; Morgan Kaufmann; 2011
- Mark Maier; *The Art of Systems Architecting – 3rd Edition*; CRC Press; 2009
- Jon Holt, Simon Perry; *SysML for Systems Engineering*; The Institution of Engineering and Technology; 2008

Additional References

- Charles Wasson; *System Analysis, Design, and Development – Concepts, Principles, and Practices*; John Wiley & Sons; 2006
- Katsuhiko Ogata; *Modern Control Engineering*; Prentice-Hall Inc; 1970
-

Functional Architecture

- Lead Authors:
- Caitlyn Singam and Jeffrey Carter

A system's functional architecture is the inter-related set of transformative processes and purposeful input-output tasks (i.e., functions) that a system performs, or can perform, on input(s) from external or internal sources in order to produce output(s) that supports the achievement of mission objectives. More simply, functional architecture defines the various actions a system can perform in support of specific goals, and how those actions relate to each other in order to collectively give the system the appropriate capabilities to meet those goals. The handling of internal inputs and outputs (such as those generated by and passed between sub-functions) are encompassed in functional architecture as well.

Since a system's functional architecture only defines the system in relation to transformative functions and their inputs/outputs, it is not in and of itself a complete definition of all aspects of the system design. Functional architecture is thus considered as an architectural "view" or "perspective", rather than as a standalone sub-unit of the overall system architecture. Other system architectural views, such as logical architecture and physical architecture, can be used to provide limited context for a given system's functional architecture without the need to examine all aspects of a complete system architecture.

Functional architectures are typically represented via abstract system models when used as part of documentation capturing and/or defining the characteristics and behaviors of a system of interest. Capturing a system's functional architecture is a key part of system architecture design definition^[1] and is typically done early on in a system's life cycle. The closely associated task of functional architecture modeling is generally performed at the same time and in advance of any system development efforts, though there are instances (e.g., in studies of natural systems) where there may be an interest in documenting the functional architecture of a system already in operation. Once developed, functional architecture models provide a useful means of elucidating the processes which underlie a system's abilities and behaviors to stakeholders, and consequently can be used later on in the system life cycle as well, in support of verification and validation (V&V) efforts and system design review activities.

This article discusses how functional architecture relates as a concept to other facets of system architecture, and discusses key principles related to the development and documentation of functional architecture in practice.

Purpose

The purpose of functional architecture is essentially stated directly in the name: it defines what functions and sub-functions exist within a system of interest (SoI), and the metaphorical "layout" or architecture which connects or otherwise relates these functions. Specifically, these function definitions involve the identification of the system's inputs and outputs (both main and intermediary), and the expression of system activities in terms of the actions they perform, either directly or in association with auxiliary system processes, to transform those inputs into output elements and behavioral outcomes (ISO/IEC/IEEE 15288:2023). The idea of a "function" in the specific context of functional architecture is thus arguably closer to how it is used in mathematics (in the sense of "y is a function of x") or computer programming than in other aspects of systems engineering, where "function" takes on a broader definition generally along the lines of actions performed in pursuit of a desired outcome (Hitchins 2008) and not directly tied to input-output transformations.

Relation to Behavioral Architecture

Functional architecture is distinct from the closely related concept of behavioral architecture, which it can be easily conflated with due to the similar technical meanings of the terms "behavior" and "function". Both functional and behavioral architecture concern purposeful actions undertaken by a given system, and the manner in which those actions enable the production of observable features such as system modes and mode transitions, but differ in terms of the lenses through which they view and define the system's actions.

Functional architecture relates to input-output transformations: the manner in which a system operates on, and/or in relation to, intangible and tangible inputs from the entities, users, and environments which exist around it in the surrounding system context through the transformation of inputs into outputs. Behavioral architecture, in contrast, is more concerned with the sequencing and execution of system actions, and how system actions interact with checkpoint conditions and the initiation/completion of other system tasks in order to produce behavior.

Application in practice

An overall system architecture is generally defined very early on in the system life cycle, as it plays a key role in translating the system concept definition into a viable system design (INCOSE 2023). Functional architecture, as one of many potential "views" of that system architecture, offers a lens through which one can consider the system in question in order to ensure that key elements of the system architecture are not omitted or overlooked, and to furthermore ensure that changes in aspects of the system architecture that were defined outside the scope of the functional view do not conflict or otherwise cause issues with the system's functionality.

The functional architecture view of a completed system architecture should, at minimum, define the following:

1. the functional capabilities that the system uses to fulfill mission objectives and meet stakeholder needs;
2. the required inputs and outputs for those system functions;
3. the steps through which the system will transform provided inputs into the desired outputs;
4. the grouping of those steps into independently executable sub-functions, if sub-functions are being employed, with corresponding definitions of the inputs and outputs for those sub-functions;
5. the variations in the inputs, outputs, and execution of the system's function/sub-functions for each system mode, if the system has multiple modes; and
6. the internal and external interfaces enabling input flow and output flow for each defined function, sub-function, and auxiliary function.

Given that functional architecture is merely a partial "view" of the overall system architecture, it is generally a good practice to consider a system's functional architecture in juxtaposition with other architectural views for context. Logical architecture, physical architecture, and functional architecture are often discussed in conjunction with each other (Pineda and Smith 2010; Borky and Bradley 2019; Stief et al. 2018) since they provide distinct but mutually relevant perspectives of how the system's capabilities are implemented in a given system design (Broy et al. 2009). Taken together as a trio, they provide a fairly comprehensive portrait of the overall system architecture, and can be as key lenses through which to develop design concepts for new systems or examine the architecture of pre-existing ones. Figure 1 provides the functional architecture viewpoint and view within the system design model.

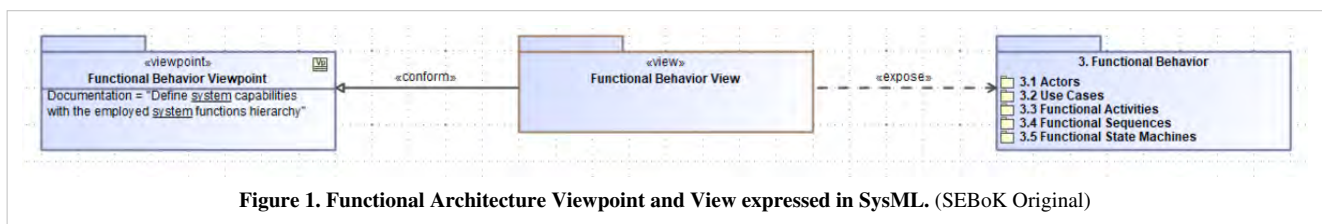


Figure 1. Functional Architecture Viewpoint and View expressed in SysML. (SEBoK Original)

Modeling

A system's functional architecture is an abstract concept; consequently, while it might be possible to deduce the functional architecture of an existing system through observation and testing of the system while it is in operation (as might be necessary when examining/documenting a natural system), doing so is generally a non-trivial, time-intensive task. Thus, a model of a system's functional architecture, which provides a representation of the system's functionality in a manner that can be accessed and understood by relevant parties, is a useful means through which individuals can interact with information about system's functional architecture concept. It is worth noting, however, that a functional architecture model is not equivalent to the system's functional architecture itself: while a system's functional architecture refers to the conceptual idea of how a system transforms its inputs into outputs, a functional architectural model serves as a representation or expression of that concept to communicate understanding of the system's functionality. As with any model, a functional architectural model will not necessarily capture every possible nuance of what is being modeled, and in cases where one is attempting to model the functional architecture of a system without complete knowledge of the system requirements, etc. that impacted the details system's functionality (natural systems again being an example of this), it is entirely feasible to have a functional architecture model that does not match the functional architecture of the system. The terms "functional architecture view" and "functional architecture model" should thus not be used interchangeably as the former refers an idea (a specific aspect of the overall system architecture), and the latter to a representation used to communicate the key aspects of that idea in a manner that may be imperfect and/or incomplete.

There are multiple options for how one can express a system's functional architecture in model form. The optimal type of model for representing any one given system's functional architecture is something that is usually best assessed on a case-by-case basis depending on the system and scope of the model's intended use. Text-based descriptive architecture models, for instance, are suitable solutions for systems with straightforward functional architectures that only involve a few functions and inputs/outputs since those architectures can be described in paragraph form without too much difficulty. In cases where the functional architecture involves a large number of conditional operations or highly interconnected processes, a visual diagram-based model (Youngs et al. 1999) or digital software model (Lemazurier, Chapurlat, and Grossetête 2017; Brinkkemper and Pachidi 2010) are likely to be preferable to text-based descriptions in terms of accurately representing the full gamut of system functionality while still making it feasible to extract information from the model without undue effort (Nowodzienski and Navas 2023; Younse, Cameron, and Bradley 2021). The principles of model-based systems engineering (MBSE) tend to be useful in ensuring the development of a quality system architecture model, and it is thus recommended for individuals to apply such principles when performing architecture modeling in various contexts (Estefan 2007; Kaslow et al. 2017). Software packages for generating digital MBSE-friendly models sometimes include tools that automatically check the model for errors (e.g., incorrectly expressed data flows), which can be useful when modeling informational flow through system functions.

For individuals using Unified Modeling Language (UML), Systems Modeling Language (SysML), or other standardized graphical modeling approaches to express their system concept, the functional architecture and behavioral architecture views are often considered and evaluated jointly due to their coordinated expression in SysML diagrams. Figures 2-5 are presented below for the reader's edification as illustrative exemplars of some of the diagram types of relevance in a functional architecture view of a SysML model, including a Use Case Diagram for a system (Figure 2), a Block Definition Diagram of hierarchical system functions (Figure 3), an Activity Diagram describing an individual system function (Figure 4), and a State Machine Diagram of the overall system (Figure 5). The diagrams below are presented in the example context of an automobile powertrain application.

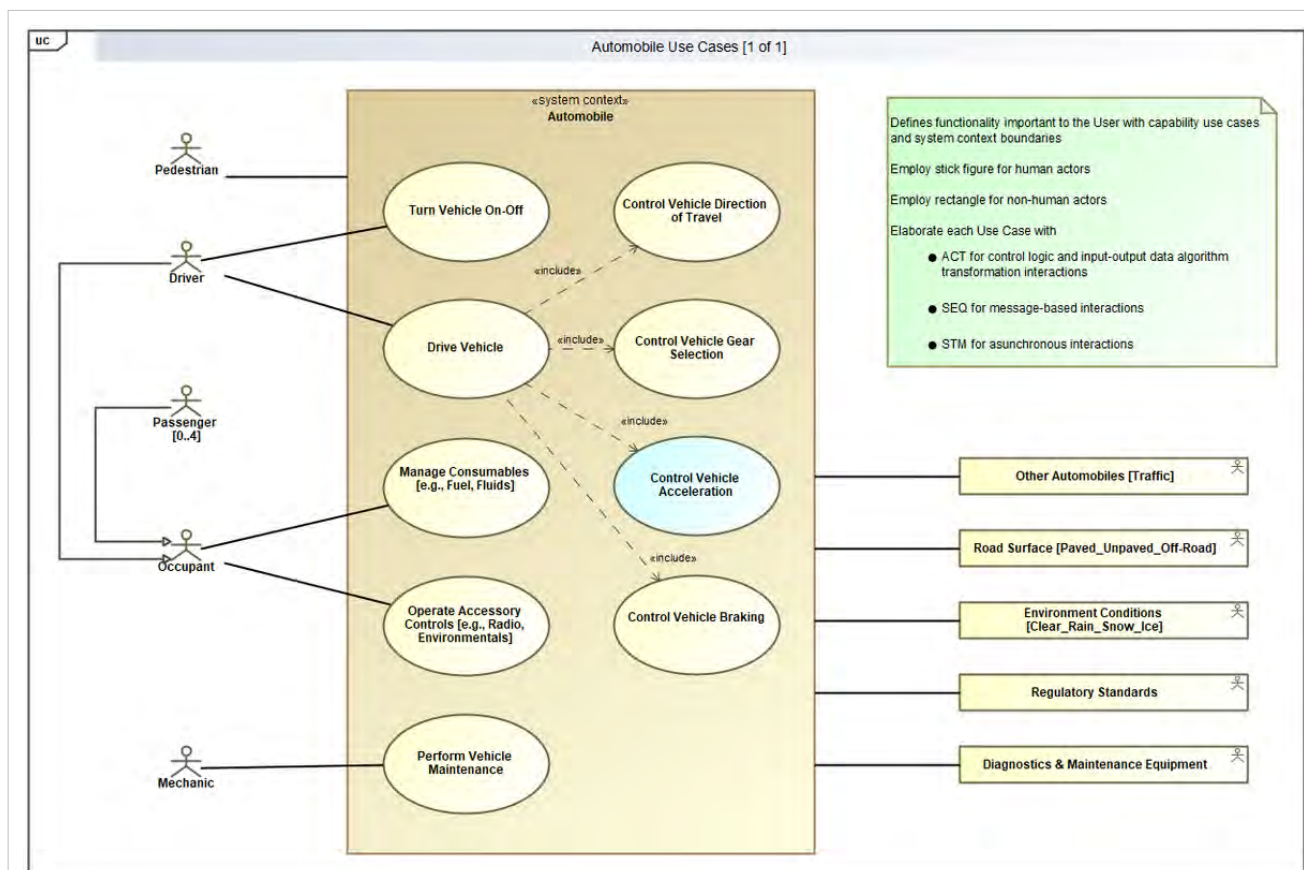


Figure 2. Example SysML Use Case Diagram for an Automotive System. (SEBoK Original)

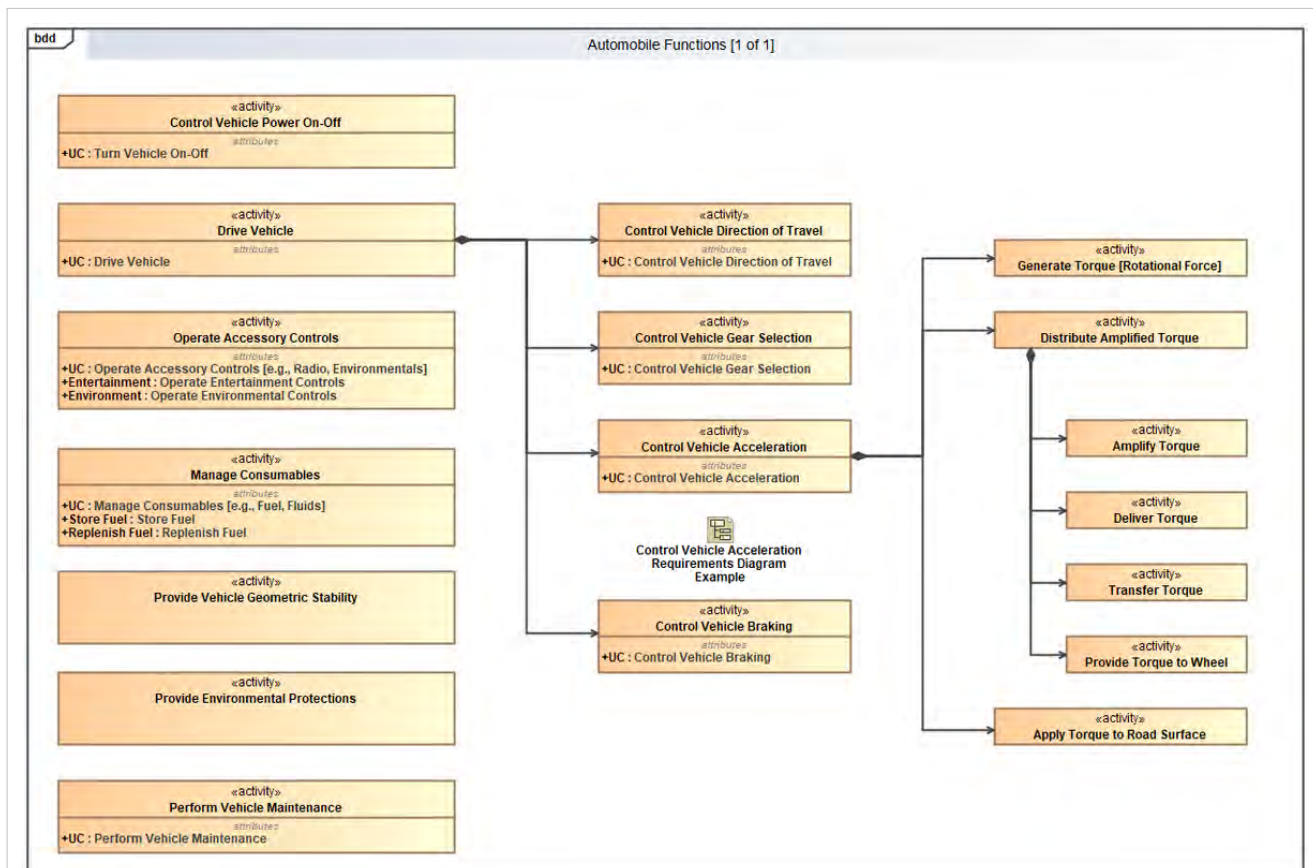


Figure 3. Example SysML Block Definition Diagram Representation of Hierarchical Functions for an Automotive System. (SEBoK Original)

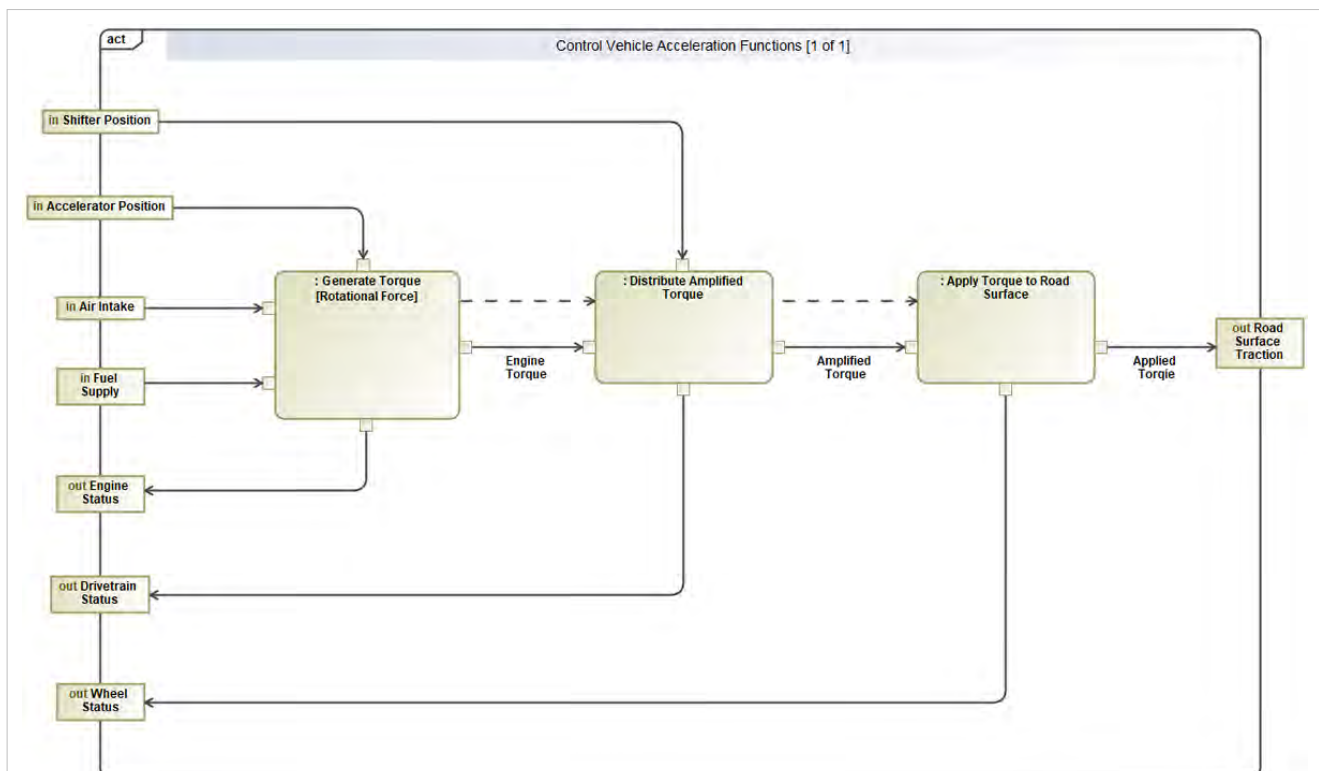


Figure 4. Example SysML Activity Diagram for the Function "Control Vehicle Acceleration" in an Automotive System. (SEBoK Original)

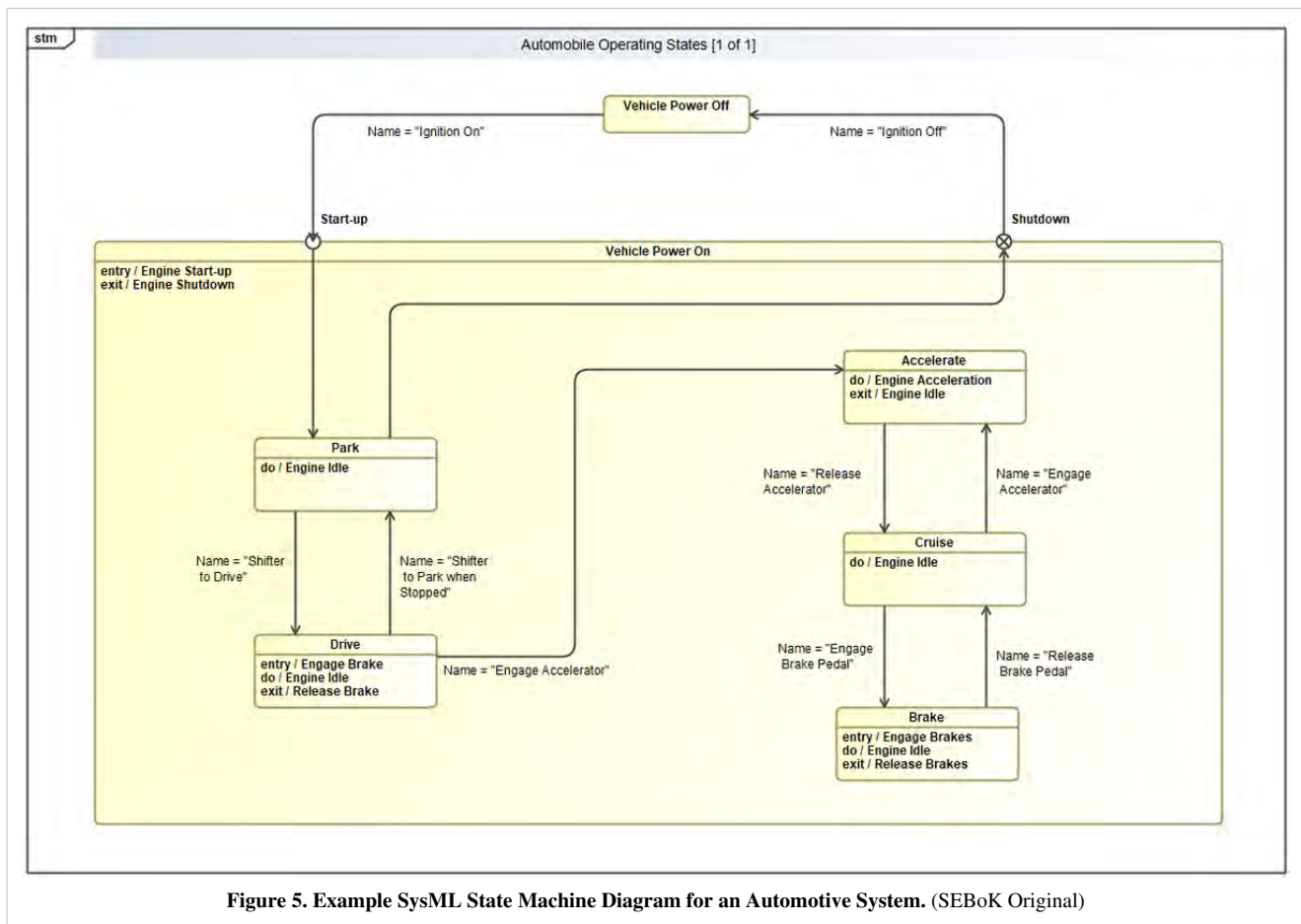


Figure 5. Example SysML State Machine Diagram for an Automotive System. (SEBoK Original)

References

Works Cited

- Borky, John M., and Thomas H. Bradley. 2019. "Designing in a Logical/Functional Viewpoint." In *Effective Model-Based Systems Engineering*, edited by John M. Borky and Thomas H. Bradley, 153–216. Cham: Springer International Publishing. https://doi.org/10.1007/978-3-319-95669-5_5.
- Brinkkemper, Sjaak, and Stella Pachidi. 2010. "Functional Architecture Modeling for the Software Product Industry." In *Software Architecture*, edited by Muhammad Ali Babar and Ian Gorton, 198–213. Berlin, Heidelberg: Springer. https://doi.org/10.1007/978-3-642-15114-9_16.
- Broy, Manfred, Mario Gleirscher, Stefano Merenda, Doris Wild, Peter Kluge, and Wolfgang Krenzer. 2009. "Toward a Holistic and Standardized Automotive Architecture Description." *Computer* 42 (12): 98–101. <https://doi.org/10.1109/MC.2009.413>.
- Estefan, Jeff A. 2007. "Survey of Model-Based Systems Engineering (MBSE) Methodologies." *INCOSE MBSE Focus Group* 25 (8): 1–12.
- Hitchins, Derek K. 2008. *Systems Engineering: A 21st Century Systems Methodology*. John Wiley & Sons.
- INCOSE, ed. 2023. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 5th edition. Hoboken, NJ: Wiley.
- Kaslow, David, Bradley Ayres, Philip T. Cahill, Laura Hart, and Rose Yntema. 2017. "A Model-Based Systems Engineering (MBSE) Approach for Defining the Behaviors of CubeSats." In *2017 IEEE Aerospace Conference*, 1–14. <https://doi.org/10.1109/AERO.2017.7943865>.

- Lemazurier, L., V. Chapurlat, and A. Grossetête. 2017. "An MBSE Approach to Pass from Requirements to Functional Architecture." *IFAC-PapersOnLine*, 20th IFAC World Congress, 50 (1): 7260–65. <https://doi.org/10.1016/j.ifacol.2017.08.1376>.
- Nowodziński, Pierre, and Juan Navas. 2023. "From Model-Based to Model and Simulation-Based Systems Architectures—Achieving Quality Engineering through Descriptive and Analytical Models." *INSIGHT* 26 (1): 40–50. <https://doi.org/10.1002/inst.12428>.
- Pineda, Ricardo L., and Eric D. Smith. 2010. "Functional Analysis and Architecture." In *Systems Engineering Tools and Methods*. CRC Press.
- Stief, Paul, Jean-Yves Dantan, Alain Etienne, and Ali Siadat. 2018. "A New Methodology to Analyze the Functional and Physical Architecture of Existing Products for an Assembly Oriented Product Family Identification." *Procedia CIRP*, 28th CIRP Design Conference 2018, 23-25 May 2018, Nantes, France, 70 (January): 47–52. <https://doi.org/10.1016/j.procir.2018.02.026>.
- Youngs, R., D. Redmond-Pyle, P. Spaas, and E. Kahan. 1999. "A Standard for Architecture Description." *IBM Systems Journal* 38 (1): 32–50. <https://doi.org/10.1147/sj.381.0032>.
- Younse, Paulo J., Jessica E. Cameron, and Thomas H. Bradley. 2021. "Comparative Analysis of a Model-Based Systems Engineering Approach to a Traditional Systems Engineering Approach for Architecting a Robotic Space System through Knowledge Categorization." *Systems Engineering* 24 (3): 177–99. <https://doi.org/10.1002/sys.21573>.

Primary References

- ANSI/IEEE. 2000. *Recommended Practice for Architectural Description for Software-Intensive Systems*. New York, NY, USA: American National Standards Institute (ANSI)/Institute of Electrical and Electronics Engineers (IEEE), ANSI/IEEE 1471-2000.
- INCOSE. 2023. *Systems Engineering Handbook - A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- ISO/IEC/IEEE. 2011. *Systems and Software Engineering - Architecture Description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.

Additional References

None.

Logical Architecture

- Lead Authors:
- Alan Faisandier and Garry Roedler
- Contributing Author:
- Rick Adcock

Logical Architecture Model Development may be used as a task of the activity "Develop candidate architectures models and views," or a sub-process of the System Architecture Design Definition process. Its purpose is to elaborate models and views of the functionality and behavior of the future engineered system as it should operate while in service. The logical architecture model of an engineered system of interest (SoI) is composed of a set of related technical concepts and principles that support the logical operation of the system. It may include a functional architecture view, a behavioral architecture view, and a temporal architecture view. Other additional views are suggested in architecture frameworks, depending on the domain.

Note: The term *Logical Architecture* is a contraction of the expression *Logical View of the System Architecture*.

Concepts and Principles

Functional Architecture Model

A functional architecture model is a set of functions and their sub-functions that defines the transformations performed by the system to complete its mission.

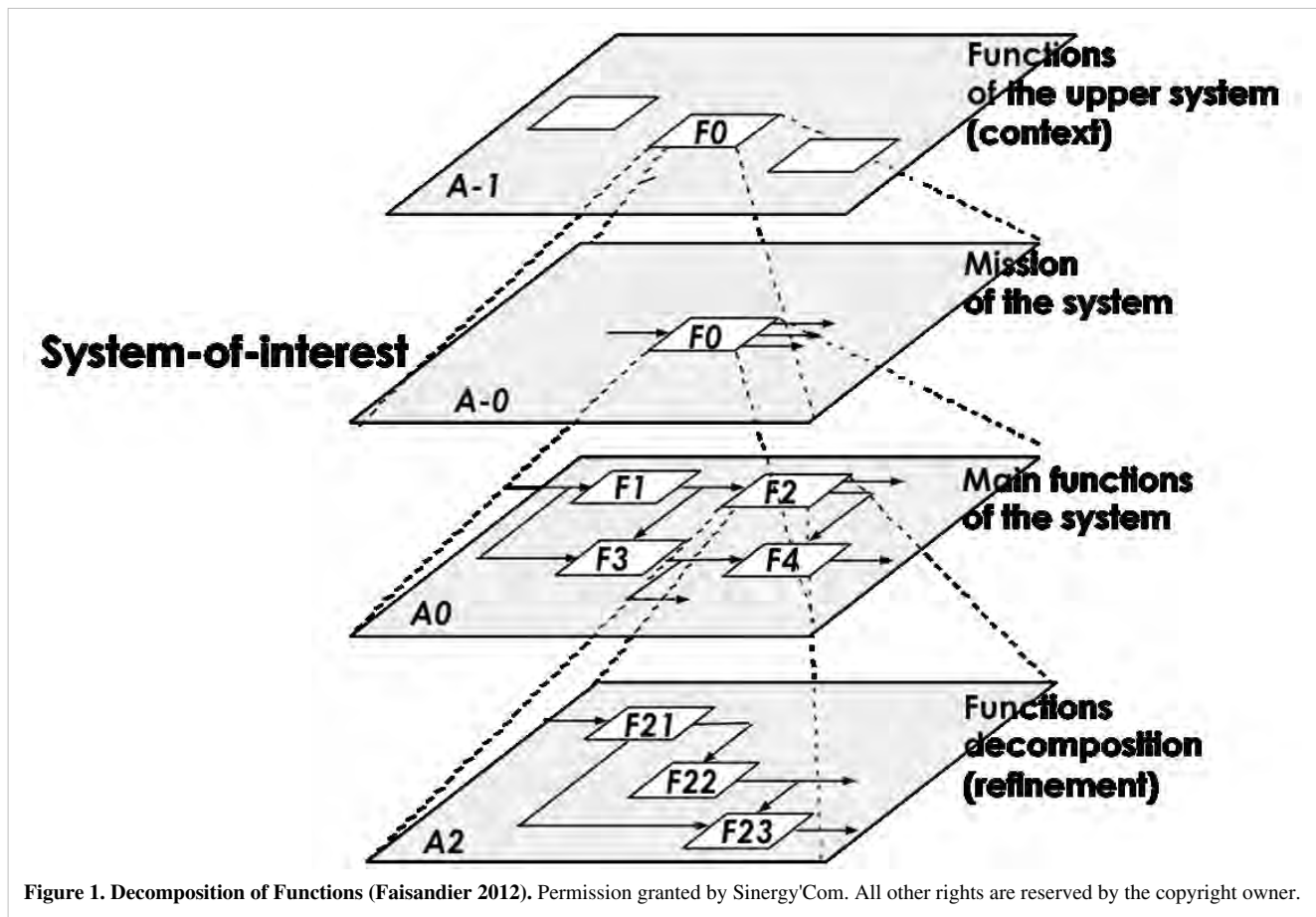
Function and Input-Output Flow - In the context of System Architecture, functions and input-output flows are architecture entities. A function is an action that transforms inputs and generates outputs, involving data, materials, and/or energies. These inputs and outputs are the flow items exchanged between functions. The general mathematical notation of a function is $y = f(x, t)$, in which y and x are vectors that may be represented graphically and t = time.

In order to define the complete set of functions of the system, one must identify all the functions necessitated by the system and its derived requirements, as well as the corresponding inputs and outputs of those functions. Generally speaking, there are two kinds of functions:

1. Functions that are directly deduced from functional and interface requirements. These functions express the expected services of a system necessary to meet its system requirements.
2. Functions that are derived and issued from the alternative solutions of the physical architecture model and are dependent upon the result of the design; additionally, they rely upon on technology choice to implement the logical architecture model elements.

Functional Hierarchy/Decomposition of Functions - At the highest level of a hierarchy (Figure 1), it is possible to represent a system as a unique, central function (defined as the system's mission) that in many ways is similar to a "black box" ("F0" in plan A-0 in Figure 1). In order to understand, in detail, what the system does, this "head-of-hierarchy" (F0) is broken down into sub-functions (F1, F2, F3, F4) grouped to form a sub-level of the hierarchy (plan A0), and so on. Functions of the last level of a functional hierarchy can be called leaf-functions (F21, F22, F23, F24 in plan A2). Hierarchies (or breakdowns) decompose a complex or global function into a set of functions for which physical solutions are known, feasible, or possible to imagine.

This view of functional hierarchy represents a static view of functions which would be populated at different levels over a number of iterations, depending upon the synthesis approach used. In general, it is not created by a single top-down decomposition. A static functional hierarchy on its own does not represent how effectively the flows of inputs and outputs are exchanged, and may need to be viewed alongside the other models below.



Behavioral Architecture Model

A behavioral architecture model is an arrangement of functions and their sub-functions as well as interfaces (inputs and outputs) that defines the execution sequencing, conditions for control or data-flow, and performance level necessary to satisfy the system requirements (ISO/IEC 26702:2007). A behavioral architecture model can be described as a set of inter-related scenarios of functions and/or operational modes.

Control (Trigger) - A control flow is an element that activates a function as a condition of its execution. The state of this element, or the condition it represents, activates or deactivates the function (or elements thereof). A control flow can be a signal or an event, such as a switch being moved to the *on* position, an alarm, a trigger, a temperature variation, or the push of a key on a keyboard.

Scenario (of Functions) - A scenario of functions is a chain of functions that are performed as a sequence and synchronized by a set of control flows to work to achieve a global transformation of inputs into outputs, as seen in the figures below. A scenario of functions expresses the dynamic of an upper level function. A behavioral architecture is developed by considering both scenarios for each level of the functional hierarchy and for each level of the system hierarchy. When representing scenarios of functions and behavioral architecture models, it is appropriate to use diagrams as modeling techniques, such as functional flow block diagrams (FFBD) (Oliver, Kelliher, and Keegan 1997) or activity diagrams, developed with SysML (OMG 2010). Figures 2 and 3 provide examples of these diagrams.

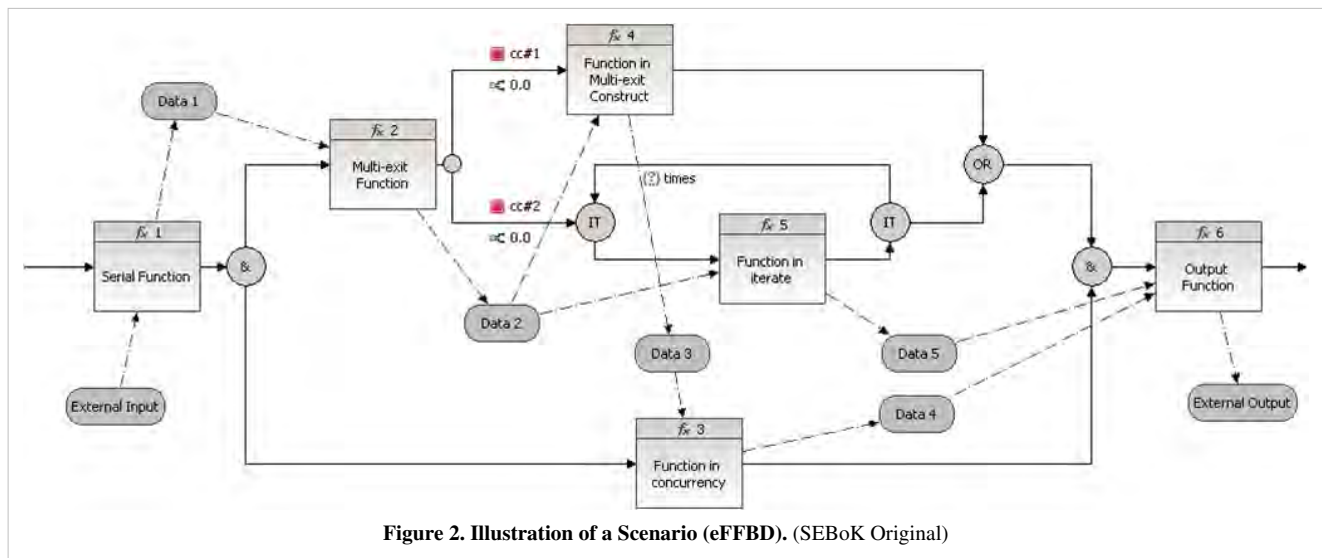


Figure 2. Illustration of a Scenario (eFFBD). (SEBoK Original)

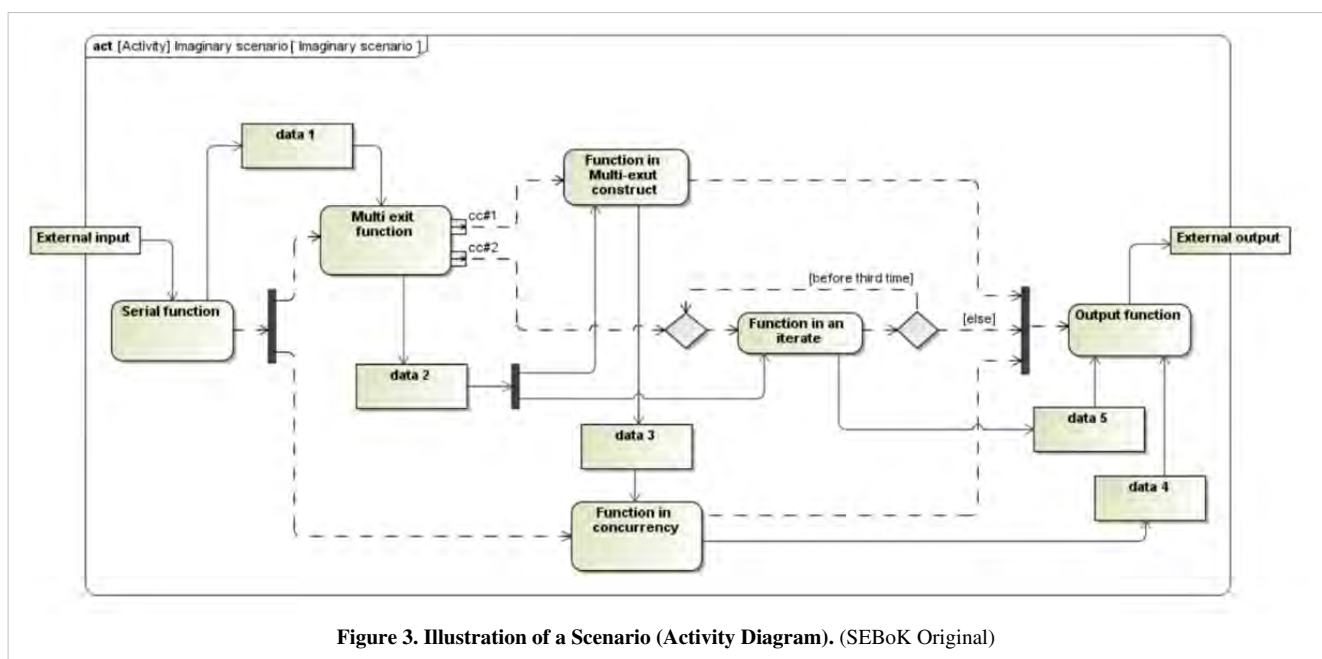
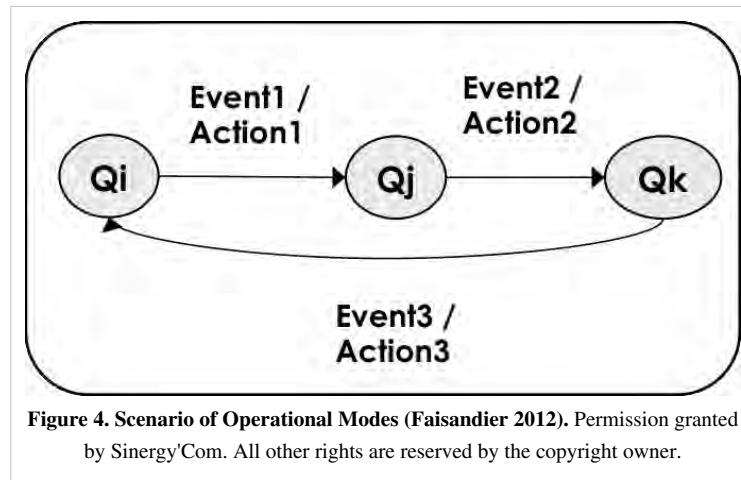


Figure 3. Illustration of a Scenario (Activity Diagram). (SEBoK Original)

Operational Mode - A scenario of functions can be viewed by abstracting the transformation of inputs into outputs of each function and focusing on the active or non-active state of the function and its controls. This view is called a *scenario of modes*, which is a chain of modes performed as a sequence of transitions between the various modes of the system. The transition from one mode to another is triggered by the arrival of a control flow (event/trigger). An action (function) can be generated within a transition between two modes following the arrival of an event or a trigger, as demonstrated in Figure 4 below.



Behavioral Patterns - When defining scenarios or behavioral architecture models, architects may opt to recognize and use known models to represent the expected transformations and behaviors. Patterns are generic basic models that may be more or less sophisticated depending on the complexity of the treatment (Gamma, Helm, Johnson, and Vlissides 1995). A pattern can be represented with different notations. Behavioral patterns are classified into several categories, which can be seen in the following examples (see also SEBoK Part 2: Patterns of Systems Thinking):

- Basic patterns or constructs linking functions - such as sequence, iteration, selection, concurrence, multiple exits, loops with an exit, and replication.
- Complex patterns - such as monitoring a treatment, exchanging a message, man machine interfaces, modes monitoring, real-time monitoring of processes, queue management, and continuous monitoring with supervision.
- Failure detection, identification, and recovery (FDIR) patterns - such as passive redundancies, active redundancies, semi-active redundancies, and treatments with reduced performance.

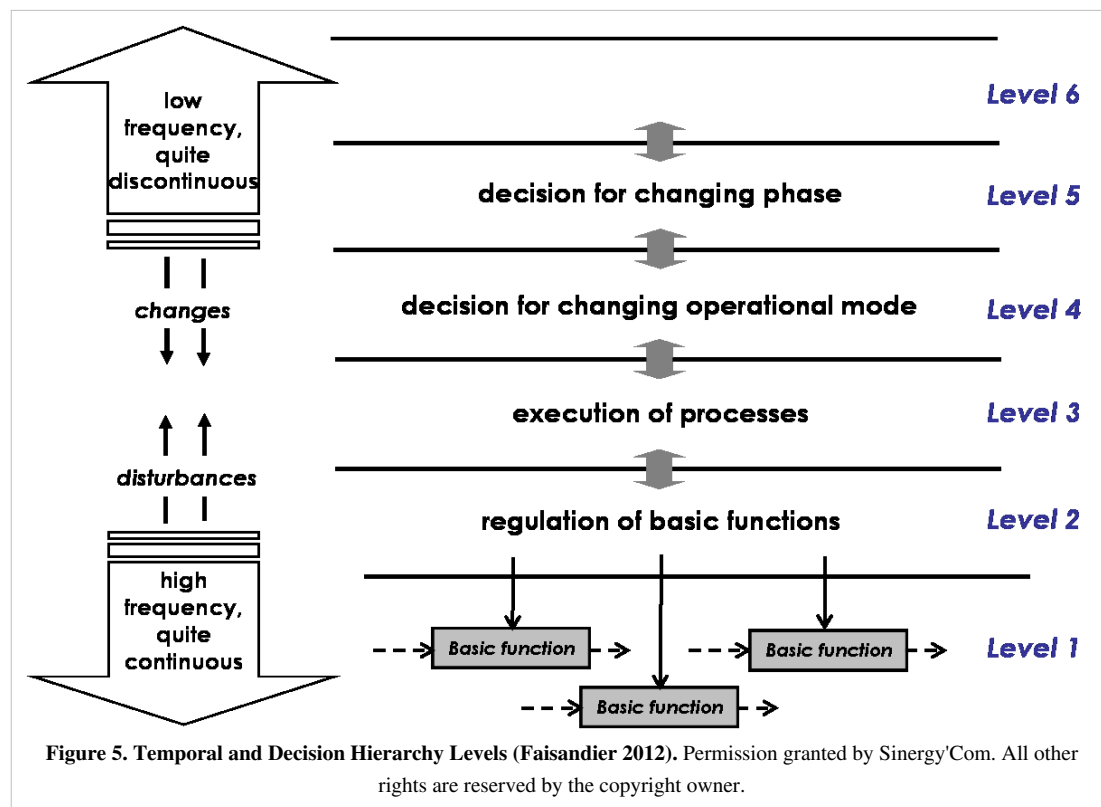
Temporal Architecture Model

A temporal architecture model is a classification of the functions of a system that is derived according to the frequency level of execution. Temporal architecture models include the definition of synchronous and asynchronous aspects of functions. The decision monitoring that occurs inside a system follows the same temporal classification because the decisions are related to the monitoring of functions.

Temporal and Decisional Hierarchy Concept - Not every function of a system is performed at the same frequency. The frequencies change depending on the time and the manner in which the functions are started and executed. One must therefore consider several classes of performance. There are synchronous functions that are executed cyclically and asynchronous functions that are executed following the occurrence of an event or trigger.

To be more specific, *real-time* systems and *command-control* systems combine cyclical operations (synchronous) and factual aspects (asynchronous). Cyclical operations consist of sharing the execution of functions according to frequencies, which depend on either the constraints of capture or dispatching the input/output and control flows. Two types of asynchronous events can be distinguished:

1. Disturbances on High Frequencies (bottom of figure 5) - Decisions that are made at either the level they occur or one level above. The goal is to deter disturbances from affecting the low frequencies so that the system continues to achieve its mission objectives. This is the way to introduce exception operations, with the typical example relating to operations concerns, breakdowns, or failures.
2. Changes on Low Frequencies (top of figure 5) - Decisions pertaining to changes that are made at the upper levels. The ultimate goal is to transmit them toward bottom levels to implement the modifications. A typical example relates to operator actions, maintenance operations, etc.



Process Approach

Purpose

The purpose of the Logical Architecture Model Development is to define, select, and synthesize a system's logical architecture model to provide a framework against which to verify that a future system will satisfy its system requirements in all operational scenarios, within which trade-offs between system requirements can be explored in developing such systems.

Generic inputs to the process include system requirements, generic architecture patterns that architects identify and use to answer requirements, outcomes from system analysis processes, and feedback from system verification and validation processes. Depending on the Life Cycle Model that is chosen, there will be iterations through which these inputs and outputs, and the relationships between them evolve and change throughout the process (see also Applying Life Cycle Processes).

Generic outputs from the process are either a single logical architecture model or a set of candidate logical architecture models together with the selected independent logical architecture model and a rationale for its selection. They include, at minimum, views and models. These involve functional, behavioral and temporal views, a traceability matrix between logical architecture model elements and system requirements.

Activities of the Process

Major activities and tasks performed during this process include the following:

- Identify and analyze functional and behavioral elements:
 - Identify functions, input-output flows, operational modes, transition of modes, and operational scenarios from system requirements by analyzing the functional, interface, and operational requirements.
 - Define necessary inputs and controls (energy, material, and data flows) to each function and outputs that result in the deduction of the necessary functions to use, transform, move, and generate the input-output flows.
- Assign system requirements to functional and behavioral elements:
 - Formally characterize functions expressions and their attributes through the assignment of performance, effectiveness, and constraints requirements. In particular, study the temporal aspects from requirements to assign duration, response time, and frequency to functions.
 - Formally characterize the input, output, and control flows expressions and their attributes through assignment of interface, effectiveness, operational, temporal and constraints requirements.
 - Establish traceability between system requirements and these functional and behavioral elements.
- Define candidate logical architecture models for each candidate:
 - Analyze operational modes as stated in the system requirements (if any) and/or use previously defined elements to model sequences of operational modes and the transition of modes. Eventually decompose the modes into sub-modes and then establish for each operational mode one or several scenarios of functions recognizing and/or using relevant generic behavioral patterns.
 - Integrate these scenarios of functions in order to get a behavioral architecture model of the system (a complete picture of the dynamic behavior).
 - Decompose previously defined logical elements as necessary to look towards implementation.
 - Assign and incorporate temporal constraints to previously defined logical elements, such as the period of time, duration, frequency, response-time, timeout, stop conditions, etc.
 - Define several levels of execution frequency for functions that correspond to levels of decision, in order to monitor system operations, prioritize processing on this time basis, and share out functions among those execution frequency levels to get a temporal architecture model.
 - Perform functional failure modes and effects analysis and update the logical architecture elements as necessary.
 - Execute the models with simulators (when possible) and tune these models to obtain the expected characteristics.
- Synthesize the selected independent logical architecture model:
 - Select the logical architecture by assessing the candidate logical architecture models against assessment criteria (related to system requirements) and compare them, using the system analysis process to perform assessments and decision management process for the selection (see the System Analysis and Decision Management topics). This selected logical architecture model is called *independent logical architecture model* because, as much as possible, it is independent of implementation decisions.
 - Identify and define derived logical architecture model elements created for the necessity of design and corresponding with the derived system requirements. Assign these requirements to the appropriate system (current studied system or external systems).
 - Verify and validate the selected logical architecture models (using as executable models as possible), make corrections as necessary, and establish traceability between system requirements and logical architecture model elements.
- Feedback logical architecture model development and system requirements. This activity is performed after the physical architecture model development process:

- Model the *allocated logical architecture* to systems and system elements, if such a representation is possible, and add any functional, behavioral, and temporal elements as needed to synchronize functions and treatments.
- Define or consolidate derived logical and physical elements induced by the selected logical and physical architecture models. Define the corresponding derived requirements and allocate them to appropriate logical and physical architectures elements. Incorporate these derived requirements into the requirements baselines of impacted systems.

Artifacts, Methods and Modeling Techniques

Logical architecture descriptions use modeling techniques that are grouped under the following types of models. Several methods have been developed to support these types of models (some are executable models):

- Functional Models – These include models such as the structured analysis design technique (SADT/IDEF0), system analysis & real time (SA-RT), enhanced Functional Flow Block Diagrams (eFFBD), and the function analysis system technique (FAST).
- Semantic Models- These include models such as entities-relationships diagrams, class diagrams, and data flow diagrams.
- Dynamic Models – These include such models as state-transition diagrams, state-charts, eFFBDs, state machine diagrams (SysML), activity diagrams (SysML) (OMG 2010), and petri nets.

Depending on the type of domain (e.g. defense, enterprise), architecture frameworks provide descriptions that can help to represent additional aspects/views of architectures - see the section 'Enterprise Architecture Frameworks & Methodologies' in Enterprise Systems Engineering Key Concepts. See also practical means for using general templates related to ISO/IEC/IEEE 42010 (ISO 2011).

Practical Considerations

As stated above, the purpose of the logical architecture model is to provide a description of what a system must be able to do to satisfy the stated need. This should help to ensure that the needs and/or concerns of all stakeholders are addressed by any solution, and that innovative solutions, as well as those based on current solution technologies, can be considered. In practice it is human nature for problem stakeholders to push their own agendas and for solution architects or designers to offer their familiar solutions. If a logical architecture model is not properly enforced with the chosen life cycle, it is easy for both problem and solution stakeholders to ignore it and revert to their own biases (see Part 5: Enabling Systems Engineering). This is exacerbated if the logical architecture model becomes an end in its own right or disconnected from the main lifecycle activities. This can occur either through the use of abstract language or notations, levels of detail, time taken, or an overly complex final architecture that does not match the purpose for which it was created. If the language, scope, and timeliness of the architecture are not matched to the problem stakeholder or solution providers, it is easier for them to overlook it. Key pitfalls and good practices which can help to avoid problems related to logical architecture models are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in developing logical architecture are provided in Table 1.

Table 1. Pitfalls with Logical Architecture Development. (SEBoK Original)

Pitfall	Description
Problem Relevance	The logical architecture model should relate back to the operational scenarios produced by mission analysis.
Inputs for Architecture Model	The major input for architecture definition activity involves the set of system requirements and the instances in which they do not address the right level of architecture. The consequence is that the architect allows the requirements to fall to the side and invents a solution with what he or she understands through the input.
Decomposition Too Deep	A common mistake made by many beginners in architecture consists of decomposing the functions too deeply or having too many functions and input/output flows in scenarios or in the functional architecture model of the current system block.
Not Considering Inputs and Outputs Together with Functions	A common mistake is to consider only the actions supported by functions and decomposing them, while forgetting the inputs and the outputs or considering them too late. Inputs and outputs are integral parts of a function.
Considering Static Decomposition of Functions Only	Static function decomposition is the smallest functional architecture model task and answers the basic question, "How is this done?" The purpose of the static decomposition is to facilitate the management of or navigation through the list of functions. The static decomposition should be established only when scenarios have been created and the logical architecture is close to complete.
Mixing Governance, Management, and Operation	Governance (strategic monitoring), management (tactical monitoring), and basic operations are often mixed in complex systems. Logical architecture model should deal with behavioral architecture model as well as with temporal architecture model.

Proven Practices

Some proven practices gathered from the references are provided in Table 2.

Table 2. Proven Practices with Logical Architecture Development. (SEBoK Original)

Practice	Description
Constitute Scenarios of Functions	Before constituting a decomposition tree of functions, one must model the behavior of the system, establish scenarios of functions, and decompose functions as scenarios of sub-functions.
Analysis and Synthesis Cycles	When facing a system that contains a large number of functions, one should attempt to synthesize functions into higher abstraction levels of functions with the assistance of criteria. Do not perform analysis only; instead, conduct small cycles of analysis (decomposition) and synthesis. The technique of using scenarios includes this design practice.
Alternate Functional and Behavioral Views	A function (action verb; e.g. "to move") and its state of execution/operational mode (e.g. "moving") are two similar and complimentary views. Utilize this to consider a behavioral view of the system that allows for the transition from one operational mode to another.
The Order to Create a Scenario of Functions	When creating a scenario of functions, it is more efficient to first establish the (control) flow of functions, then to add input and output flows, and finally to add triggers or signals for synchronization.

References

Works Cited

- Gamma, E., R. Helm, R. Johnson, and J. Vlissides. 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA, USA: Addison-Wesley.
- Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.
- ISO/IEC. 2007. *Systems Engineering – Application and Management of the Systems Engineering Process*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 26702:2007.
- ISO/IEC/IEEE. 2011. *Systems and Software Engineering - Architecture Description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.
- Oliver, D., T. Kelliher, and J. Keegan. 1997. *Engineering Complex Systems with Models and Objects*. New York, NY, USA: McGraw-Hill.
- OMG. 2010. *OMG Systems Modeling Language Specification*, version 1.2, July 2010. Available at: https://www.omg.org/technology/documents/spec_catalog.htm.

Primary References

- ANSI/IEEE. 2000. *Recommended Practice for Architectural Description for Software-Intensive Systems*. New York, NY, USA: American National Standards Institute (ANSI)/Institute of Electrical and Electronics Engineers (IEEE), ANSI/IEEE 1471-2000.
- INCOSE. 2023. *Systems Engineering Handbook - A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.
- ISO/IEC. 2007. *Systems Engineering – Application and Management of the Systems Engineering Process*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 26702:2007.
- ISO/IEC/IEEE 15288:2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commissions / Institute for Electrical and Electronics Engineers.
- ISO/IEC/IEEE 15288:2023.
- ISO/IEC/IEEE. 2011. *Systems and Software Engineering - Architecture Description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.
- Maier, M. and E. Rechtin. 2009. *The Art of Systems Architecting*, 3rd ed. Boca Raton, FL, USA: CRC Press.

Additional References

- Alexander, C., S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, and S. Angel. 1977. *A Pattern Language: Towns, Buildings, Construction*. New York, NY, USA: Oxford University Press.
- Buede, D.M. 2009. *The Engineering Design of Systems: Models and Methods*. 2nd ed. Hoboken, NJ, USA: John Wiley & Sons Inc.
- Oliver, D., T. Kelliher, and J. Keegan. 1997. *Engineering Complex Systems with Models and Objects*. New York, NY, USA: McGraw-Hill.
- The Open Group. 2011. *TOGAF*, version 9.1. Hogeweg, The Netherlands: Van Haren Publishing. Accessed August 29, 2012. Available at: <https://www.opengroup.org/ogsys/jsp/publications/PublicationDetails.jsp?catalogno=g116>.
- Zachman, J. 2008. "John Zachman's Concise Definition of The Zachman Framework™." *Zachman International Enterprise Architecture*. Accessed August 29, 2012. Available at: <https://www.zachman.com/about-the-zachman-framework>.

Physical Architecture

- Lead Authors:
- Alan Faisandier and Rick Adcock

Physical Architecture Model Development may be used as a task of the activity "Develop candidate architectures models and views," or a sub-process of the System Architecture Design Definition process. Its purpose is to elaborate models and views of a physical, concrete solution that accommodates the logical architecture model and satisfies and trades-off system requirements. Once a logical architecture model is defined (see Logical Architecture Model Development), concrete physical elements have to be identified that can support functional, behavioral, and temporal features as well as the expected properties of the system deduced from non-functional system requirements (e.g. constraint of replacement of obsolescence, and/or continued product support).

A physical architecture model is an arrangement of physical elements, (system elements and physical interfaces) that provides the solution for a product, service, or enterprise. It is intended to satisfy logical architecture elements and system requirements ISO/IEC/IEEE 26702 (ISO 2007). It is implementable through technological system elements. System requirements are allocated to both the logical and physical architectures. The resulting system architecture is assessed with system analysis and when completed becomes the basis for system realization.

In some cases, particularly when multiple systems are to be defined to a common physical architecture model, one of the drivers for the physical architecture model may be interface standards; these physical interfaces may well be one of the most important concerns for these systems. It is quite possible that such interface standards are mandated at a high level in the system requirements. On the other hand, it is equally possible for standards to be derived during physical architecture model development and these can be critical enablers for desirable engineering outcomes, such as: families of systems, technology insertion, interoperability and "open systems". For example, today's video, hi-fi, and computer systems have all benefited from adoption of interface standards. Other examples exist in most fields of engineering from nuts and bolts, plumbing, electrical installations, rail gauges, TCP/IP, IT systems and software to modular defense and space systems.

Note: The term *Physical Architecture* is a contraction of the expression *Physical View of the System Architecture*.

Concepts and Principles

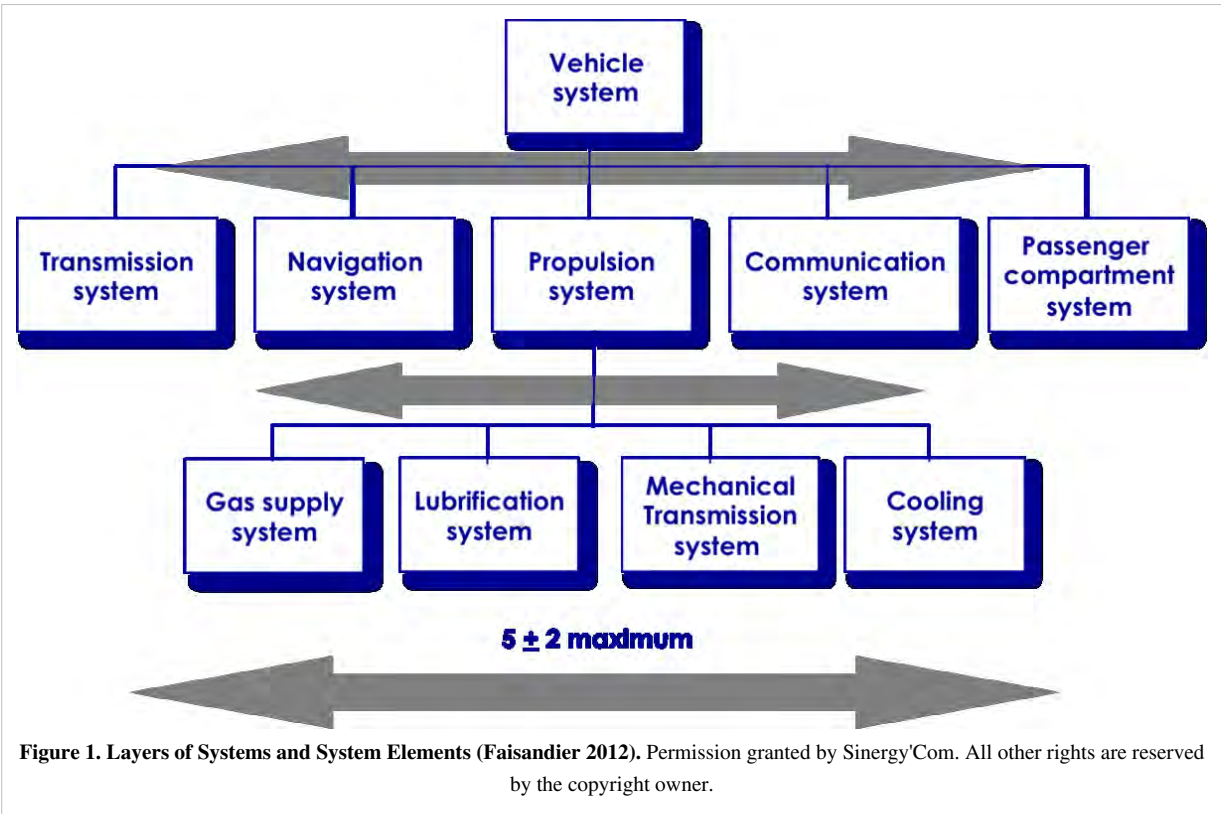
System Element, Physical Interface, and Physical Architecture Model

A system element is a discrete part of a system that can be implemented to fulfill design properties. A system element can be hardware, software, data, humans, processes (e.g., processes that provide a service to users), procedures (e.g., operator instructions), facilities, materials, and naturally occurring entities (e.g., water, organisms, and minerals), or any combination of these ISO/IEC/IEEE 15288 (ISO 2023). A physical interface binds two system elements together; this is similar to a link or a connector. Table 1 provides some examples of system elements and physical interfaces.

Table 1. Types of System Elements and Physical Interfaces. (SEBoK Original)

Element	Product System	Service System	Enterprise System
System Element	• Hardware Parts (mechanics, electronics, electrical, plastic, chemical, etc.) • Operator Roles • Software Pieces	• Processes, Data Bases, Procedures, etc. • Operator Roles • Software Applications	• Corporate, Direction, Division, Department, Project, Technical Team, Leader, etc. • IT Components
Physical Interface	* Hardware Parts, Protocols, Procedures, etc.	* Protocols, Documents, etc.	* Protocols, Procedures, Documents, etc.

A complex system composed of thousands of physical and/or intangible parts may be structured in several layers of systems and system elements. The number of elements in a level of the structure of one system is limited to only a few, in order to facilitate managing the system definition; a common guideline is *five plus or minus two* elements (see illustration in Figure 1).



A physical architecture model is built from systems, system elements, and all necessary physical interfaces between these elements, as well as from external elements (neighboring or enabling systems and/or system elements in the considered layer and concerned elements in the context of the global system-of-interest) - see illustration in Figure 2.

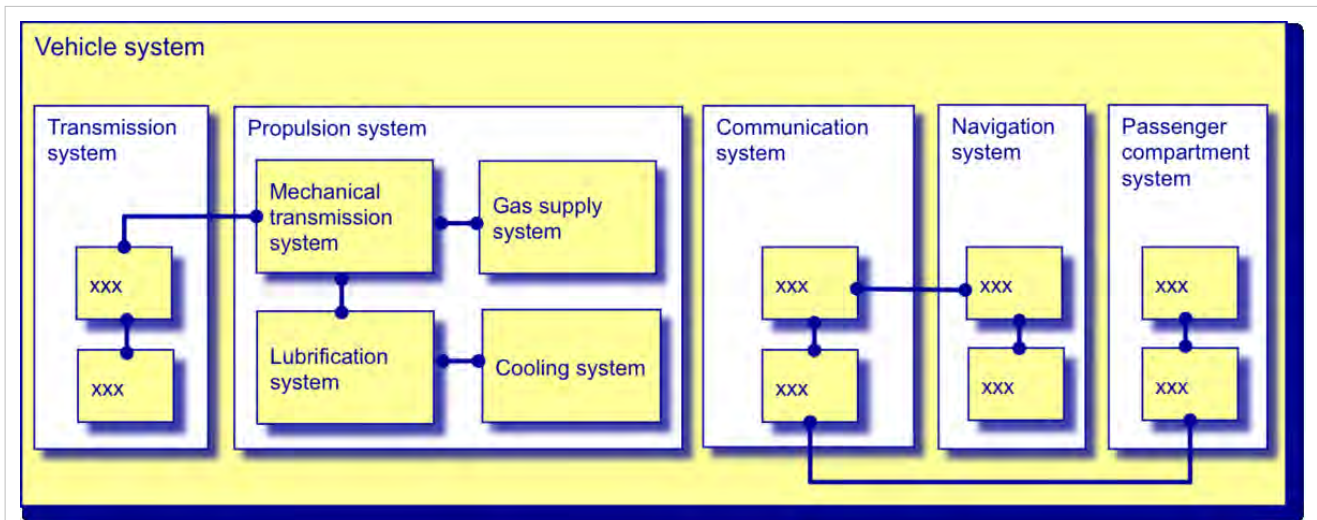


Figure 2. Physical Architecture Model Representation (Faisandier 2012). Permission granted by Sinergy'Com. All other rights are reserved by the copyright owner.

Design Property

A design property is a property that is obtained during system architecture and created through the assignment of non-functional requirements, estimates, analyses, calculations, simulations of a specific aspect, or through the definition of an existing element associated with a system element, a physical interface, and/or a physical architecture. If the defined element complies with a requirement, the design property will relate to (or may equal) the requirement. Otherwise, one has to identify any discrepancy that could modify the requirement or design property and detect any deviations.

Stakeholders have concerns that correspond to the expected behavior of a system within operational, environmental, and/or physical constraints as well as to more general life cycle constraints. Stakeholder needs and requirements and system requirements express these concerns as expected capabilities from the system (e.g., usability, interoperability, security, expandability, environment suitability, etc.). Architects and/or designers identify these capabilities from requirements and deduce corresponding quantitative or qualitative design properties to properly equip their physical architecture model (e.g., reliability, availability, maintainability, modularity, robustness, operability, climatic environment resistance, dimensions limits, etc.). For further discussion on how some of these properties may be included in architecture and design, please see the article Systems Engineering and Quality Attributes in the Related Disciplines Part.

Allocation of Logical Elements to Physical Elements and Partitioning

Developing a candidate physical architecture model for a system consists of first identifying the system elements that can perform functions of the logical architecture model as well as identifying the interfaces capable of carrying out the input-output flows and control flows. When identifying potential elements, a systems engineer needs to allocate design properties within the logical architecture; these properties are deduced from the system requirements. Partitioning and allocation are activities to decompose, gather, or separate functions in order to facilitate the identification of feasible system elements that support these functions. Either they exist and can be reused or re-purposed, or they can be developed and technically implemented.

Partitioning and allocation use criteria to find potential affinities between functions. Systems engineers use system requirements and/or design properties as criteria to assess and select candidate system elements and partitions of functions, such as similar transformations within the same technology, similar levels of efficiency, exchange of the same type of input-output flows (information, energy, and materials), centralized or distributed controls, execution

with close frequency level, dependability conditions, environment resistance level, and other enterprise constraints.

A concurrent engineering approach is necessary when several different sets of technologies, knowledge, and skills are necessary to establish a candidate physical architecture model. This is particularly true during the partition and allocation of functions to various system elements, in which the systems engineer must account for compatibility issues and emergent properties. (See SEBoK Part 2: Synthesizing Possible Solutions for a discussion of possible approaches.)

Developing Candidate Physical Architecture Models

The goal of physical architecture model development activities is to provide the best possible physical architecture model made of suitable systems, technological system elements, and physical interfaces (i.e., the architecture that answers, at best, all system requirements, depending on agreed limits or margins of each requirement). The best way to do this is to produce several candidate physical architecture models, assess and compare them, and then select the most suitable one.

A candidate physical architecture model is elaborated according to affinity criteria in order to build a set of system elements (i.e., separate, gather, connect, and disconnect the network of system elements and their physical interfaces). These criteria are the same as those used for partitioning and allocating functions to system elements. The physical architecture model development may be focused in different ways, for example, it may address:

- Reduction in the number of physical interfaces
- System elements that can be tested separately
- Compatible technology
- Measures of the proximity of elements in space
- Ease of handling (weight, volume, and transportation facilities)
- Optimization of resources shared between elements
- Modularity (i.e. elements have low interdependence)
- Resilience (i.e. elements which are highly reliable, maintainable or replaceable)

Evaluating and Selecting the Preferred Candidate

Viable physical architecture models enable all required functions or capabilities specified in the logical architecture model to be realized. Architecture and design activity includes evaluation to obtain a balance among design properties, costs, risks, etc. Generally, the physical architecture model of a system is determined more strongly by non-functional requirements (e.g., performance, safety, security, environmental conditions, constraints, etc.) than by functions. There may be many (physical) ways to establish functions but fewer ways of satisfying non-functional requirements. The preferred physical architecture model represents the selection of system elements, their physical relationships, and interfaces. Typically, this physical architecture will still leave further systems engineering to be undertaken to achieve a fully optimized system after any remaining trade-offs are made and algorithms and parameters of the system are finalized. Certain analyses (efficiency, dependability, cost, risks, etc.) are required to get sufficient data that characterize the global behavior and structure of the candidate architectures in regard to system requirements; this is often broadly referred to as system analysis. Other analyses and assessments require knowledge and skills from the different involved technologies and specialties (mechanics, electronics, software, thermodynamics, electro-magnetic compatibility, safety, security etc.). They are performed through corresponding specialist analysis of the system.

Legacy Systems and Systems of Systems

Few systems come into existence or operate without interacting with others in a system context. These interactions may be with other operational systems, or maintenance and support systems, which in turn may be legacy (already in use) or future legacy (under development and likely to operate with the system of interest in the future).

The best chosen approach will be dependent on the strength of interactions between the system-of-interest (SoI) and its wider context. While these interactions are small, they may be accounted for by defining a set of static external interface requirements (for example, technical standards) with which the system must comply, by including these as constraints in the system requirements and ensuring compliance through design assurance.

Where the interactions are more intense (for example, where continuous information is to be exchanged with other systems), these will have to be recognized as part of a system of systems context and will instead be considered as part of an enterprise systems engineering approach.

Another important consideration may be the sharing of technology or system elements between the SoI and other systems, often as part of a family of systems (many examples occur in automotive and aerospace industries) or the re-use of system elements from existing legacy. Here a degree of top-down or middle-out design work will be necessary to ensure the system of interest embodies the required system elements, while conforming as far as possible to the stakeholder and system requirements, with any compromises being understood and managed.

If a System-of-Interest is intended to be used in one or more service systems or system of systems configurations, this will affect its physical architecture model. One of the features of these SoS is the late binding of component systems in use. Such component systems must be architected with open or configurable interfaces, must have clearly defined functions packaged in such a way as to be relevant to the SoS using them, and must include some method by which they can be identified and included in the SoS when needed.

Both service systems and SoS will be defined by a high-level physical architecture model, which will be utilized to define the relevant SoS relationships, interfaces, and constraints that should be included in System Concept Definition. The results will be embedded in the stakeholder and system requirements and handled through interface agreements and across-project communication during development, realization, and use.

See SEBoK Part 4: Applications of Systems Engineering for more information on special considerations for architecting SoS.

Process Approach

Purpose

The purpose of the Physical Architecture Model Development is to define, select, and synthesize a system physical architecture model which can support the logical architecture model. A physical architecture model will have specific properties to address stakeholder concerns or environmental issues and to satisfy system requirements.

Because of the evolution of the context of use or technological possibilities, the physical architecture which is composed of system elements is supposed to evolve along the life cycle of the system in order for it to continue to perform its mission within the limits of its required effectiveness. Depending on whether or not evolution impacts logical architecture model elements, allocations to system elements may change. A physical architecture model is equipped with specific design properties to continuously challenge the evolution.

Generic inputs include the selected logical architecture model, system requirements, generic patterns and properties that architects identify and utilize to answer requirements, outcomes from system analysis, and feedback from system verification and system validation.

Generic outputs are the selected physical architecture model, allocation matrix of functional elements to physical elements, traceability matrix with system requirements, stakeholder requirements of each system and system element composing the physical architecture model, and rejected solutions.

Activities of the Process

Major activities and tasks to be performed during this process include the following:

- Partition and allocate functional elements to system elements:
 - Search for system elements or technologies able to perform functions and physical interfaces to carry input-output and control flows. Ensure system elements exist or can be engineered. Assess each potential system element using criteria deduced from design properties (themselves deduced from non-functional system requirements).
 - Partition functional elements (functions, scenarios, input-outputs, triggers, etc.) using the given criteria and allocate partitioned sets to system elements (using the same criteria).
 - When it is impossible to identify a system element that corresponds to a partitioned functional set, decompose the function until the identification of implementable system elements is possible.
 - Check the compatibility of technologies and the compatibility of interfaces between selected system elements.
- Constitute candidate physical architecture models.
 - Because partitioned sets of functions can be numerous, there are generally too many system elements. For defining controllable architectures, system elements have to be grouped into higher-level system elements known as system element groups, often called sub-systems in industry.
 - Constitute several different system element groups corresponding to different combinations of elementary system elements. One set of system element groups plus one or several non-decomposable system elements forms a candidate physical architecture model of the considered system.
 - Represent (using patterns) the physical architecture model of each system element group connecting its system elements with physical interfaces that carry input-output flows and triggers. Add physical interfaces as needed; in particular, add interfaces with external elements to the system element group.
 - Represent the synthesized physical architecture of the considered system built from system element groups, non-decomposable systems, and physical interfaces inherited from the physical architecture model of system element groups.
 - Enhance the physical architecture model with design properties such as modularity, evolution capability, adaptability to different environments, robustness, scalability, resistance to environmental conditions, etc.
 - If possible, use executable architecture prototypes (e.g., hardware-software (HW-SW)-in-the-loop prototypes) for identifying potential deficiencies and correct the architecture as needed.
- Assess physical architecture model candidates and select the most suitable one:
 - Use the system analysis process to perform assessments (see the System Analysis topic).
 - Use the Decision Management process to support the trades and selection of the preferred alternative (see the Decision Management topic).
- Synthesize the selected physical architecture model:
 - Formalize physical elements and properties. Verify that system requirements are satisfied and that the solution is realistic.
 - Identify the derived physical and functional elements created for the necessity of architecture and design and the corresponding system requirements.
 - Establish traceability between system requirements and physical elements as well as allocate matrices between functional and physical elements.

Artifacts, Methods and Modeling Techniques

Physical architecture descriptions use modeling techniques to create and represent physical architectures. Some common physical models include structural blocks, mass, layout and other models. Modeling techniques may be:

- Physical block diagrams (PBD)
- SysML block definition diagrams (BDD)
- Internal block diagrams (IBD) (OMG 2010)
- Executable architecture prototyping
- Etc.

Depending on the type of domain for which it is to be used (defense, enterprise, etc.), architecture frameworks may provide descriptions that can help to trade off candidate architectures. Please see section 'Enterprise Architecture Frameworks & Methodologies' in Enterprise Systems Engineering Key Concepts.

Practical Considerations

Key pitfalls and good practices related to physical architecture development are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in performing physical architecture model development are provided in Table 3.

Table 3. Pitfalls with Physical Architecture Development. (SEBoK Original)

Pitfall	Description
Too Many Levels in a Single System Block	The current system block includes too many levels of decomposition. The right practice is that the physical architecture model of a system block is composed of one single level of systems and/or system elements.
No Logical Architecture Model	The developers perform a direct passage from system requirements to a physical architecture model without establishing a logical architecture model; this is a common wrong practice that mainly takes place when dealing with repeating systems and products because the functions are already known. The issue is that a function is always associated with input-output flows defined in a specific domain set. If the domain set changes, the performance of the function can become invalid.
Direct Allocation on Technologies	At a high level of abstraction of multidisciplinary systems, directly allocating the functions onto technologies of the lowest level of abstraction, such as hardware or software, does not reflect a system comprehension. The right practice is to consider criteria to decompose the architecture into the appropriate number of levels, alternating logical and physical before reaching the technology level (the last level of the system).

Proven Practices

Some proven practices gathered from the references are provided in Table 4.

Table 4. Proven Practices with Physical Architecture Development. (SEBoK Original)

Practice	Description
Modularity	Restrict the number of interactions between the system elements and consider the modularity principle (maximum of consistency inside the system element, minimum of physical interfaces with outside) as the right way for architecting systems.
Focus on Interfaces	Focusing on interfaces rather than on system elements is another key element of a successful architecture and design for abstract levels of systems.

References

Works Cited

- ISO/IEC. 2007. *Systems Engineering – Application and Management of The Systems Engineering Process*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 26702:2007.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation (ISO) /International Electrotechnical Commissions (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.
- OMG. 2010. *OMG Systems Modeling Language Specification*, version 1.2, July 2010. Available at: https://www.omg.org/technology/documents/spec_catalog.htm.
- Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Primary References

- ANSI/IEEE. 2000. *Recommended Practice for Architectural Description for Software-Intensive Systems*. New York, NY, USA: American National Standards Institute (ANSI)/Institute of Electrical and Electronics Engineers (IEEE), ANSI/IEEE 1471-2000.
- INCOSE. 2023. *Systems Engineering Handbook - A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.
- ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation (ISO)/International Electrotechnical Commissions (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2023. ISO/IEC/IEEE. 2011. *Systems and Software Engineering - Architecture Description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.

Additional References

- Maier, M., and E. Rechtin. 2009. *The Art of Systems Architecting*, 3rd ed. Boca Raton, FL, USA: CRC Press.
- Holland, J.H. 2006. "Studying Complex Adaptive Systems." *Journal of Systems Science and Complexity*. vol. 19, no. 1 pp. 1-8. Available at: <https://hdl.handle.net/2027.42/41486>.
- Thome, B. 1993. *Systems Engineering, Principles & Practice of Computer-Based Systems Engineering*. New York, NY, USA: Wiley.
- The Open Group. 2011. *TOGAF*, version 9.1. Hogeweg, The Netherlands: Van Haren Publishing. Accessed August 29, 2012. Available at: <https://www.opengroup.org/ogsys/jsp/publications/PublicationDetails.jsp?catalogno=g116>.

Zachman, J. 2008. "John Zachman's Concise Definition of The Zachman Framework™." Zachman International Enterprise Architecture. Accessed August 29, 2012. Available at: <https://www.zachman.com/about-the-zachman-framework>.

System Detailed Design Definition

- Lead Authors:
- Alan Faisandier and Rick Adcock

The purpose of the System Design is to supplement the system architecture by providing information and data useful and necessary for implementation of the system elements. Design definition is the process of developing, expressing, documenting, and communicating the realization of the architecture of the system through a complete set of design characteristics described in a form suitable for implementation.

Concepts and Principles

Design Notion

In industrial practices, the term *design* is often used to mean both architecture and design. In the recent past, professionals used the term *design* when they dealt with simpler technological products - ones that do not include several different and interconnected technological components such as hardware, software, operators, services, etc. In the development of new multi-technology products and services, professionals have recognized the usefulness of the notion of *system* in dealing with complexity (interconnections level, multi-techno, emergence, etc.).

It was due to complexity that structuring the elements that comprise a system became necessary. This structure explains the functional, behavioral, temporal, physical, and other aspects of a system as described in System Architecture Design Definition. Practitioners found the term *structure* inadequate to describe all these aspects of a system. The terms *architecture* and *architectural design* have been used for approximately 30 years, especially in software intensive systems and other domains, such as the space industry. The set of different types and interrelated structures can be understood as the architecture of the system.

The trend today is to consider system architecture and system design as different and separate sets of activities, but concurrent and strongly intertwined.

System design includes activities to conceive a set of system elements that answers a specific, intended purpose, using principles and concepts; it includes assessments and decisions to select system elements that compose the system, fit the architecture of the system, and comply with traded-off system requirements. It is the complete set of detailed models, properties, and/or characteristics described into a form suitable for implementation.

Design Characteristics and Design Enablers

Every technological domain or discipline owns its peculiar laws, rules, theories, and enablers concerning transformational, structural, behavioral, and temporal properties of its composing parts of materials, energy, or information. These specific parts and/or their compositions are described with typical design characteristics and enablers. These allow achieving the implementation of every system element through various transformations and exchanges required by design characteristics (e.g., operability level, reliability rate, speed, safeguard level) that have been assigned during the system architecture definition process.

The design definition provides the description of the design characteristics and design enablers necessary for implementation. Design characteristics include dimensions, shapes, materials, and data processing structures. Design enablers include formal expressions or equations, drawings, diagrams, tables of metrics with their values and margins, patterns, algorithms, and heuristics.

- Examples of generic design characteristics in mechanics of solids: shape, geometrical pattern, dimension, volume, surface, curves, resistance to forces, distribution of forces, weight, velocity of motion, temporal persistence
- Examples of generic design characteristics in software: distribution of processing, data structures, data persistence, procedural abstraction, data abstraction, control abstraction, encapsulation, creational patterns (e.g., builder, factory, prototype, singleton), and structural patterns (e.g., adapter, bridge, composite, decorator, proxy)

Relation with System Architecture

System design is intended to be the link between the system architecture (at whatever point this milestone is defined in the specific application of the systems engineering process) and the implementation of technological system elements that compose the physical architecture model of the system.

Design definition is driven by specified requirements, the system architecture, and more detailed analysis of performance and feasibility. It addresses the implementation technologies and their assimilation. Design provides the “how-” or “implement-to” level of the definition.

Design concerns every system element composed of implementation technologies, such as mechanics, electronics, software, chemistry, human operations and services for which specific engineering processes are needed. System design provides feedback to the parent system architecture to consolidate or confirm the allocation and partitioning of architectural characteristics and design properties to system elements.

Design Descriptor

A design descriptor is the set of generic design characteristics and of their possible values. If similar, but not exact system elements exist, it is possible to analyze these in order to identify their basic characteristics. Variations of the possible values of each characteristic determine potential candidate system elements.

Holistic Design

Holistic design is an approach that considers the system being designed as an interconnected whole, which is also part of something larger. Holistic concepts can be applied to the system as a whole along with the system in its context (e.g., the enterprise or mission in which the system participates), as well as the design of mechanical devices, the layout of spaces, and so forth. This approach often incorporates concerns about the environment, considering how the design will impact the environment and attempting to reduce environmental impact. Holistic design is about more than merely trying to meet the system requirements.

Process Approach

Purpose

The purpose of the System Design process is to provide sufficient detailed data and information about the system and its system elements to enable the implementation consistent with architectural entities as defined in models and views of the system architecture (ISO/IEC/IEEE 15288 [ISO 2015]).

Generic inputs include architecture description of the parent system and system element requirements.

Generic outputs are the description of the design characteristics and design enablers necessary for implementation.

Activities of the Process

Major activities and tasks to be performed during this process include the following:

1. Initialize design definition

- Plan for technology management for the whole system. Identify the technologies (mechanics, electricity, electronics, software, biology, operators, etc.) that would compose and implement the system elements and their physical interfaces.
- Determine which technologies and system elements have a risk to become obsolete or evolve during the operation stage of the system. Plan for their potential replacement.
- Identify types of design characteristics or properties for each technology of each system element.
- Periodically assess design characteristics and adjust as the system evolves.
- Document the design definition strategy, including the need for and requirements of any enabling systems, products, or services to perform the design.

2. Establish design characteristics and design enablers related to each system element

- Perform, consolidate or detail system requirements allocation to system elements for all requirements and system elements not fully addressed in the System Architecture process (normally, every system requirement would have been transformed into architectural entities and architectural characteristics within the System Architecture process, which are then allocated to system elements through direct assignment or some partitioning).
- Define the design characteristics relating to the architectural characteristics and check that they are implementable. Use design enablers, such as models (physical and analytical), design heuristics, etc. If the design characteristics are not feasible, then assess other design alternatives or implementation option, or perform trades of other system elements definition.
- Define the interfaces that were not defined by the System Architecture process or that need to be refined as the design details evolve. This includes both internal interfaces between the system elements and the external interfaces with other systems.
- Record the design characteristics of each system element within the applicable artifacts (they depend on the design methods and techniques used).
- Provide rationale about selection of major implementation options and enablers.

3. Assess alternatives for obtaining system elements

- Identify existing implemented system elements (COTS/NDI, reused, or other non-developed system elements). Alternatives for new system elements to be developed may be studied.
 - Assess design options for the system element, using selection criteria that are derived from the design characteristics.
 - Select the most appropriate alternatives.
 - If the decision is made to develop the system element, the rest of the design definition process and the implementation process are used. If the decision is to buy or reuse a system element, the acquisition process may be used to obtain the system element.
-

4. Manage the design

- Capture and maintain the rationale for all selections among alternatives and decisions for the design, architecture characteristics, design enablers, and sources of system elements.
- Assess and control the evolution of the design characteristics, including the alignment with the architecture.
- Establish and maintain traceability between design characteristics and architectural characteristics, and with requirements as necessary.
- Provide baseline information for configuration management.
- Maintain the design baseline and the design definition strategy.

Practical Considerations

Key pitfalls and proven practices related to system design are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in performing system design are provided in Table 1.

Pitfall	Description
Consider the design of each system element separately	This would be conducted using heterogeneous implementation of a given technology or between technologies within the system-of-interest. The design strategy for the complete system is defined to find synergies and/or commonalities that could help operation and maintenance of system elements.

Proven Practices

Some proven practices gathered from the references are provided in Table 2.

Practice	Description
Architecture and design mutual support	Discipline engineers perform the design definition of each system element; they provide strong support (knowledge and competencies) to systems engineers or architects in the evaluation and selection of candidate system architectures and system elements. Inversely, systems engineers, or architects, must provide feedback to discipline engineers to improve knowledge and know-how.

References

Works Cited

- INCOSE. 2015. *INCOSE Systems Engineering Handbook*, Version 4. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.
- Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Primary References

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.

Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Additional References

Baldwin, C.Y. and K.B. Clark. 2000. *Design Rules*. Cambridge, MA, USA: MIT Press.

Buede, D.M. 2009. *The Engineering Design of Systems: Models and Methods*. 2nd ed. Hoboken, NJ, USA: John Wiley & Sons Inc.

DoD. 2010. *DOD Architecture Framework*. Version 2.02. Arlington, VA, USA: US Department of Defense. Available at: <https://cio-nii.defense.gov/sites/dodaf20/>

System Analysis

- Lead Authors:
- Alan Faisandier and Ray Madachy
- Contributing Author:
- Rick Adcock

System analysis allows developers to objectively carry out quantitative assessments of systems in order to select and/or update the most efficient system architecture and to generate derived engineering data. During engineering, assessments should be performed every time technical choices or decisions are made to determine compliance with system requirements.

System analysis provides a rigorous approach to technical decision-making. It is used to perform trade-off studies, and includes modeling and simulation, cost analysis, technical risks analysis, and effectiveness analysis.

Principles Governing System Analysis

One of the major tasks of a systems engineer is to evaluate the engineering data and artifacts created during the systems engineering (SE) process. The evaluations are at the center of system analysis, providing means and techniques:

- to define assessment criteria based on system requirements;
- to assess design properties of each candidate solution in comparison to these criteria;
- to score the candidate solutions globally and to justify the scores; and
- to decide on the appropriate solution(s).

The Analysis and Selection between Alternative Solutions article in the Systems Approach Applied to Engineered Systems knowledge area (KA) of Part 2 describes activities related to selecting between possible system solutions to an identified problem or opportunity. The following general principles of systems analysis are defined:

- Systems analysis is based on assessment criteria based upon a problem or opportunity system description.
 - These criteria will be based around an ideal system description, which assumes a hard system problem context can be defined.
 - Criteria must consider required system behavior and properties of the complete solution, in all possible wider system contexts and environments.
-

- These must consider non-functional issues such as system safety, security, etc. (Please see Systems Engineering and Quality Attributes for additional discussion on incorporating non-functional elements.)
- This "ideal" system description may be supported by soft system descriptions, from which additional "soft" criteria may be defined. For example, a stakeholder preference for or against certain kinds of solutions, relevant social, political or cultural conventions to be considered, etc.
- The assessment criteria should include, at a minimum, the constraints on cost and time scales acceptable to stakeholders.
- Trade studies provide a mechanism for conducting analysis of alternative solutions.
 - A trade study should consider a set of assessment criteria, with appropriate awareness of the limitations and dependencies between individual criteria.
 - Trade studies need to deal with both objective and subjective criteria. Care must be taken to assess the sensitivity of the overall assessment to particular criteria.

Trade-Off Studies

In the context of the definition of a system, a trade-off study consists of comparing the characteristics of each system element and of each candidate system architecture to determine the solution that best globally balances the assessment criteria. The various characteristics analyzed are gathered in cost analysis, technical risks analysis, and effectiveness analysis (NASA 2007).

Guidance on the conduct of trade studies for all types of system context are characterized in the above principles and described in more detail in the Analysis and Selection between Alternative Solutions topic. Of particular interest to SE analysis are technical effectiveness, cost, and technical risk analysis.

Effectiveness Analysis

The effectiveness of an engineered system solution includes several essential characteristics that are generally gathered in the following list of analyses, including (but not limited to): performance, usability, dependability, manufacturing, maintenance or support, environment, etc. These analyses highlight candidate solutions under various aspects.

It is essential to establish a classification that limits the number of analyses to the really significant aspects, such as key performance parameters. The main difficulties of effectiveness analysis are to sort and select the right set of effectiveness aspects; for example, if the product is made for a single use, maintainability will not be a relevant criterion.

Cost Analysis

A cost analysis considers the full life cycle costs. A cost baseline can be adapted according to the project and the system. The global life cycle cost (LCC), or total ownership cost (TOC), may include exemplary labor and non-labor cost items such as those indicated in Table 1.

Table 1. Types of Costs. (SEBoK Original)

Type of Cost	Description and Examples
Development	Engineering, development tools (equipment and software), project management, test-benches, mock-ups and prototypes, training, etc.
Product manufacturing or service realization	Raw materials and supplies, spare parts and stock assets, necessary resources to operation (water, electricity power, etc.), risks and nuances, evacuation, treatment and storage of waste or rejections produced, expenses of structure (taxes, management, purchase, documentation, quality, cleaning, regulation, controls, etc.), packing and storage, documentation required.
Sales and after-sales	Expenses of structure (subsidiaries, stores, workshops, distribution, information acquisition, etc.), complaints and guarantees, etc.
Customer utilization	Taxes, installation (customer), resources necessary to the operation of the product (water, fuel, lubricants, etc.), financial risks and nuisances, etc.
Supply chain	Transportation and delivery.
Maintenance	Field services, preventive maintenance, regulation controls, spare parts and stocks, cost of guarantee, etc.
Disposal	Collection, dismantling, transportation, treatment, waste recycling, etc.

Methods for determining cost are described in the Project Planning topic.

Technical Risks Analysis

Every risk analysis concerning every domain is based on three factors:

1. Analysis of potential threats or undesired events and their probability of occurrence.
2. Analysis of the consequences of these threats or undesired events and their classification on a scale of gravity.
3. Mitigation to reduce the probabilities of threats and/or the levels of harmful effect to acceptable values.

The technical risks appear when the system cannot satisfy the system requirements any longer. The causes reside in the requirements and/or in the solution itself. They are expressed in the form of insufficient effectiveness and can have multiple causes: incorrect assessment of technological capabilities; over-estimation of the technical maturity of a system element; failure of parts; breakdowns; breakage, obsolescence of equipment, parts, or software, weakness from the supplier (non-compliant parts, delay for supply, etc.), human factors (insufficient training, wrong tunings, error handling, unsuited procedures, malice), etc.

Technical risks are not to be confused with project risks, even if the method to manage them is the same. Although technical risks may lead to project risks, technical risks address the system itself, not the process for its development. (See Risk Management for more details.)

Process Approach

Purpose and Principles of the Approach

The system analysis process is used to: (1) provide a rigorous basis for technical decision making, resolution of requirement conflicts, and assessment of alternative physical solutions (system elements and physical architectures); (2) determine progress in satisfying system requirements and derived requirements; (3) support risk management; and (4) ensure that decisions are made only after evaluating the cost, schedule, performance, and risk effects on the engineering or re-engineering of a system (ANSI/EIA 1998). This process is also called the decision analysis process by NASA (2007, 1-360) and is used to help evaluate technical issues, alternatives, and their uncertainties to support decision-making. (See Decision Management for more details.)

System analysis supports other system definition processes:

- Stakeholder needs definition and system requirements definition processes use system analysis to solve issues relating to conflicts among the set of requirements; in particular, those related to costs, technical risks, and effectiveness (performances, operational conditions, and constraints). System requirements subject to high risks, or those which would require different architectures, are discussed.
- The Logical Architecture Model Development and Physical Architecture Model Development processes use it to assess characteristics or design properties of candidate logical and physical architectures, providing arguments for selecting the most efficient one in terms of costs, technical risks, and effectiveness (e.g., performances, dependability, human factors, etc.).

Like any system definition process, the system analysis process is iterative. Each operation is carried out several times; each step improves the precision of analysis.

Activities of the Process

Major activities and tasks performed within this process include:

- Planning the trade-off studies:
 - Determine the number of candidate solutions to analyze, the methods and procedures to be used, the expected results (examples of objects to be selected: behavioral architecture/scenario, physical architecture, system element, etc.), and the justification items.
 - Schedule the analyses according to the availability of models, engineering data (system requirements, design properties), skilled personnel, and procedures.
- Define the selection criteria model:
 - Select the assessment criteria from non-functional requirements (performances, operational conditions, constraints, etc.), and/or from design properties.
 - Sort and order the assessment criteria.
 - Establish a scale of comparison for each assessment criterion and weigh every assessment criterion according to its level of relative importance with the others.
- Identify candidate solutions, related models, and data.
- Assess candidate solutions using previously defined methods or procedures:
 - Carry out cost analysis, technical risks analysis, and effectiveness analysis placing every candidate solution on every assessment criterion comparison scale.
 - Score every candidate solution as an assessment score.
- Provide results to the calling process: assessment criteria, comparison scales, solutions' scores, assessment selection, and possibly recommendations and related arguments.

Artifacts and Ontology Elements

This process may create several artifacts, such as:

- A selection criteria model (list, scales, weighing)
- Costs, risks, and effectiveness analysis reports
- Justification reports

This process handles the ontology elements of Table 2 within system analysis.

Table 2. Main Ontology Elements as Handled within System Analysis. (SEBoK Original)

Assessment Criterion	In the context of system analysis, an assessment criterion is a characteristic used to assess or compare system elements, physical interfaces, physical architectures, functional architectures/scenarios, or any engineering elements that can be compared.
	Identifier; name; description; relative weight; scalar weight
Assessment Selection	In the context of system analysis, an assessment selection is a technical management element based on an assessment score that justifies the selection of a system element, a physical interface, a physical architecture, or a functional architecture/scenario.
Assessment Score	In the context of system analysis, an assessment score is obtained assessing a system element, a physical interface, a physical architecture, a functional architecture/scenario using a set of assessment criteria.
	Identifier; name; description; value
Cost	In the context of systems engineering, a cost is an amount expressed in a given currency related to the value of a system element, a physical interface, and a physical architecture.
	Identifier; name; description; amount; type (development, production, utilization, maintenance, disposal); confidence interval; period of reference; estimation technique
Risk	An event having a probability of occurrence and consequences related to the system mission or on other characteristics. (Used for technical risk in engineering.). A risk is the combination of vulnerability and a danger or threat.
	Identifier; name description; status

Checking Correctness of System Analysis

The main items to be checked within system analysis in order to get validated arguments are:

- Relevance of the models and data in the context of use of the system,
- Relevance of assessment criteria related to the context of use of the system,
- Reproducibility of simulation results and of calculations,
- Precision level of comparisons' scales,
- Confidence of estimates, and
- Sensitivity of solutions' scores related to assessment criteria weights.

See Ring, Eisner, and Maier (2010) for additional perspective.

Methods and Modeling Techniques

- **General usage of models:** Various types of models can be used in the context of system analysis:
 - **Physical models** are scale models allowing simulation of physical phenomena. They are specific to each discipline; associated tools include mock-ups, vibration tables, test benches, prototypes, decompression chamber, wind tunnels, etc.
 - **Representation models** are mainly used to simulate the behavior of a system. For example, enhanced functional flow block diagrams (eFFBDs), statecharts, state machine diagrams (based in systems modeling language (SysML)), etc.
 - **Analytical models** are mainly used to establish values of estimates. We can consider the deterministic models and probabilistic models (also known as stochastic models) to be analytical in nature. Analytical models use equations or diagrams to approach the real operation of the system. They can be very simple (addition) to incredibly complicated (probabilistic distribution with several variables).
- **Use right models** depending on the project progress
 - At the beginning of the project, first studies use simple tools, allowing rough approximations which have the advantage of not requiring too much time and effort. These approximations are often sufficient to eliminate unrealistic or outgoing candidate solutions.

- It is progressively necessary over the development of the project to improve precision of data to compare the candidate solutions still competing. The work is more complicated if the level of innovation is high.
- A systems engineer alone cannot model a complex system; he or she must be supported by skilled people from different disciplines involved.
- **Specialist expertise:** When the values of assessment criteria cannot be given in an objective or precise way, or because the subjective aspect is dominating, we can ask specialists for expertise. The estimates proceed in four steps:
 1. Select interviewees to collect the opinion of qualified people for the considered field.
 2. Draft a questionnaire; a precise questionnaire allows an easy analysis, but a questionnaire that is too closed risks the neglect of significant points.
 3. Interview a limited number of specialists with the questionnaire, including an in-depth discussion to get precise opinions.
 4. Analyze the data with several different people and compare their impressions until an agreement on a classification of assessment criteria and/or candidate solutions is reached.

Often used analytical models in the context of system analysis are summarized in Table 3.

Table 3. Often Used Analytical Models in the Context of System Analysis. (SEBoK Original)

Type of Model	Description
Deterministic models	• Models containing statistics are included in this category. The principle consists of establishing a model based on a significant amount of data and number of results from former projects; they can apply only to system elements/components whose technology already exists. • Models by analogy also use former projects. The system element being studied is compared to an already existing system element with known characteristics (cost, reliability, etc.). Then these characteristics are adjusted based on the specialists' expertise. • Learning curves allow foreseeing the evolution of a characteristic or a technology. One example of evolution: "Each time the number of produced units is multiplied by two, the cost of this unit is reduced with a certain percentage, generally constant."
Probabilistic models (also called stochastic models)	The theory of probability allows classifying the possible candidate solutions compared to consequences from a set of events as criteria. These models are applicable if the number of criteria is limited and the combination of the possible events is simple. Take care that the sum of probabilities of all events is equal to one for each node.
Multi-criteria decisions models	When the number of criteria is greater than ten, it is recommended that a multi-criteria decision model be established. This model is obtained through the following actions: • Organize the criteria as a hierarchy (or a decomposition tree). • Associate each criterion of each branch of the tree with a relative weight compared to each other of the same level. • Calculate a scalar weight for each leaf criterion of each branch, multiplying all the weights of the branch. • Score every candidate solution on the leaf criteria; sum the scores to get a global score for each candidate solution; compare the scores. • Using a computerized tool allows to perform sensitivity analysis to get a robust choice.

Practical Considerations

Key pitfalls and good practices related to system analysis are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing system analysis are provided in Table 4.

Table 4. Pitfalls with System Analysis. (SEBoK Original)

Pitfall	Description
Analytical modeling is not a decision tool	Analytical modeling gives analytical results from analytical data. It has to be considered as an aid and not as a decision tool.
Models and system levels of decomposition	A model can be well adapted to a level n of a system and be incompatible with the model of the higher level which uses the data coming from the lower level. It is essential that the systems engineer ensures the coherence of the various models used.
Optimization is not a sum of optimized elements	The general optimization of the system-of-interest is not the sum of its optimized systems and/or system elements.

Proven Practices

Some proven practices gathered from the references are provided in Table 5.

Table 5. Proven Practices with System Analysis. (SEBoK Original)

Practice	Description
Stay in the operational field	Models can never simulate all the behavior/reactions of a system: they operate only in one limited field with a restricted number of variables. When a model is used, it is always necessary to make sure that the parameters and data inputs are part of the operation field. If not, there is a high risk of irregular outputs.
Evolve models	Models shall evolve during the project: by modification of parameter settings, by entering new data when modified (modification of assessment criteria, functions to perform, requirements, etc.), by the use of new tools when those used reach their limits.
Use several types of models	It is recommended to concurrently use several types of models in order to compare the results and/or to take into account another aspect of the system.
Keep context elements consistent	Results of a simulation shall always be given in their modeling context: tool used, selected assumptions, parameters and data introduced, and variance of the outputs.

References

Works Cited

- ANSI/EIA. 1998. *Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA), ANSI/EIA-632-1998.
- NASA. 2007. *Systems Engineering Handbook*. Washington, D.C., USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.
- Ring, J. H. Eisner, and M. Maier. 2010. "Key Issues of Systems Engineering, Part 3: Proving Your Design." *INCOSE Insight* 13(2).

Primary References

ANSI/EIA. 1998. *Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA), ANSI/EIA 632-1998.

Blanchard, B.S., and W.J. Fabrycky. 2010. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.

NASA. 2007. *Systems Engineering Handbook*. Washington, D.C., USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Additional References

Ring, J, H. Eisner, and M. Maier. 2010. "Key Issues of Systems Engineering, Part 3: Proving Your Design." *INCOSE Insight*. vol. 13, no. 2.

System Realization

- Lead Authors:
- John Snoderly and Alan Faisandier
- Contributing Author:
- Rick Adcock

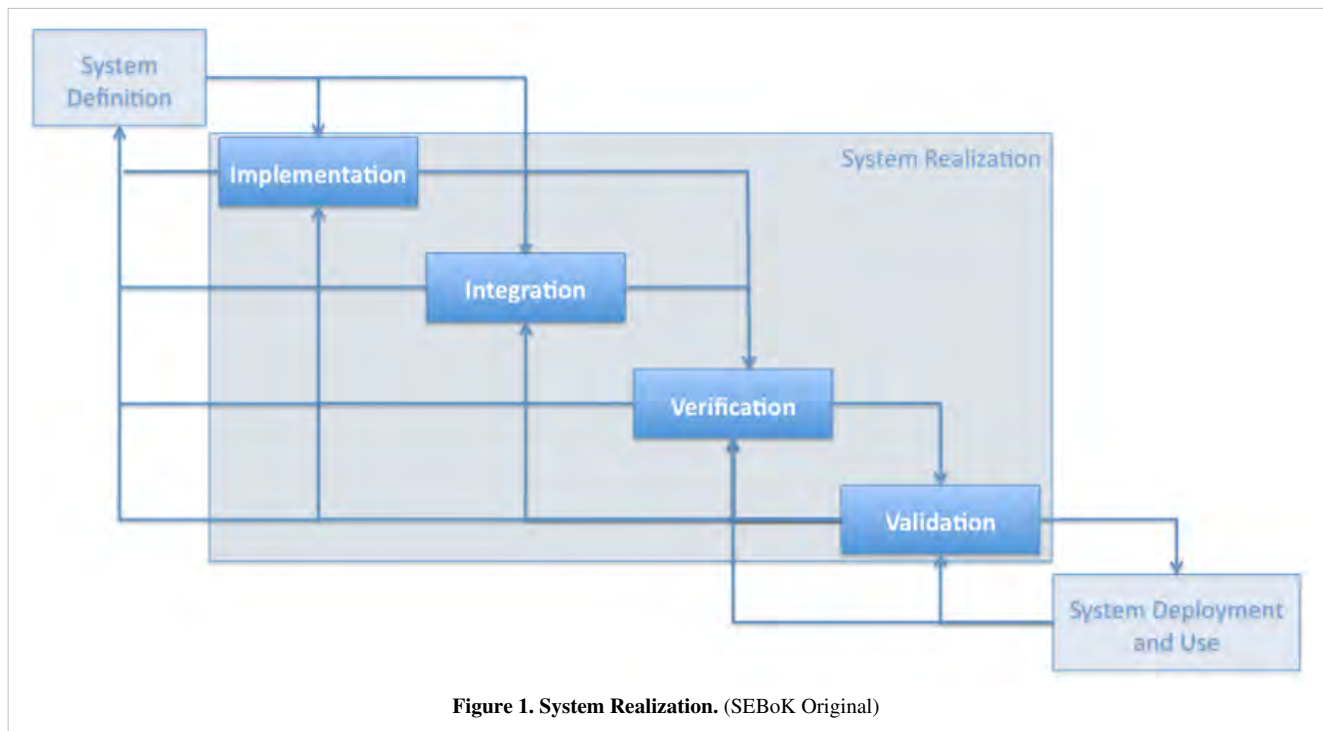
System realization activities are conducted to create and test versions of a system as specified by system definition. The activities are grouped and described as generic processes that are performed iteratively and/or concurrently depending on the selected life cycle model. These activities include those required to build a system (system implementation), integrate disparate system elements (system integration), and ensure that the system meets both the needs of stakeholders (system validation) and aligns with the system requirements and architecture (system verification).

These activities are not sequential, but are performed concurrently, iteratively and recursively depending on the selected life cycle model. Figure 1 (see "Overview", below), also shows how these processes fit within the context of system definition and System Deployment and Use KAs. See also Applying Life Cycle Processes for further discussion of the relationships between process and life cycle model.

Overview

Essentially, the outputs of system definition are used during system implementation to create system elements and during system integration to provide plans and criteria for combining these elements. The requirements are used to verify and validate system elements, systems, and the overall system-of-interest (SoI). These activities provide feedback into the system design, particularly when problems or challenges are identified.

Finally, when the system is considered, verified, and validated, it will then become an input to system deployment and use. It is important to understand that there is overlap in these activities; they do not have to occur in sequence as demonstrated in Figure 1. Every life cycle model includes realization activities, principally, verification and validation activities. The way these activities are performed is dependent upon the life cycle model in use. (For additional information on life cycles, see the Life Cycle Models KA.)



The realization processes are performed to ensure that the system will be ready for transition and has the appropriate structure and behavior to enable the desired operation and functionality throughout the system's life span. Both DAU and NASA include transition in realization, in addition to implementation, integration, verification, and validation (Prosnik 2010; NASA December 2007, 1-360).

Fundamentals

Macro View of Realization Processes

Figure 2 illustrates a macro view of generic outputs from realization activities when using a Vee life cycle model. The left side of the Vee represents various design activities 'going down' the system.

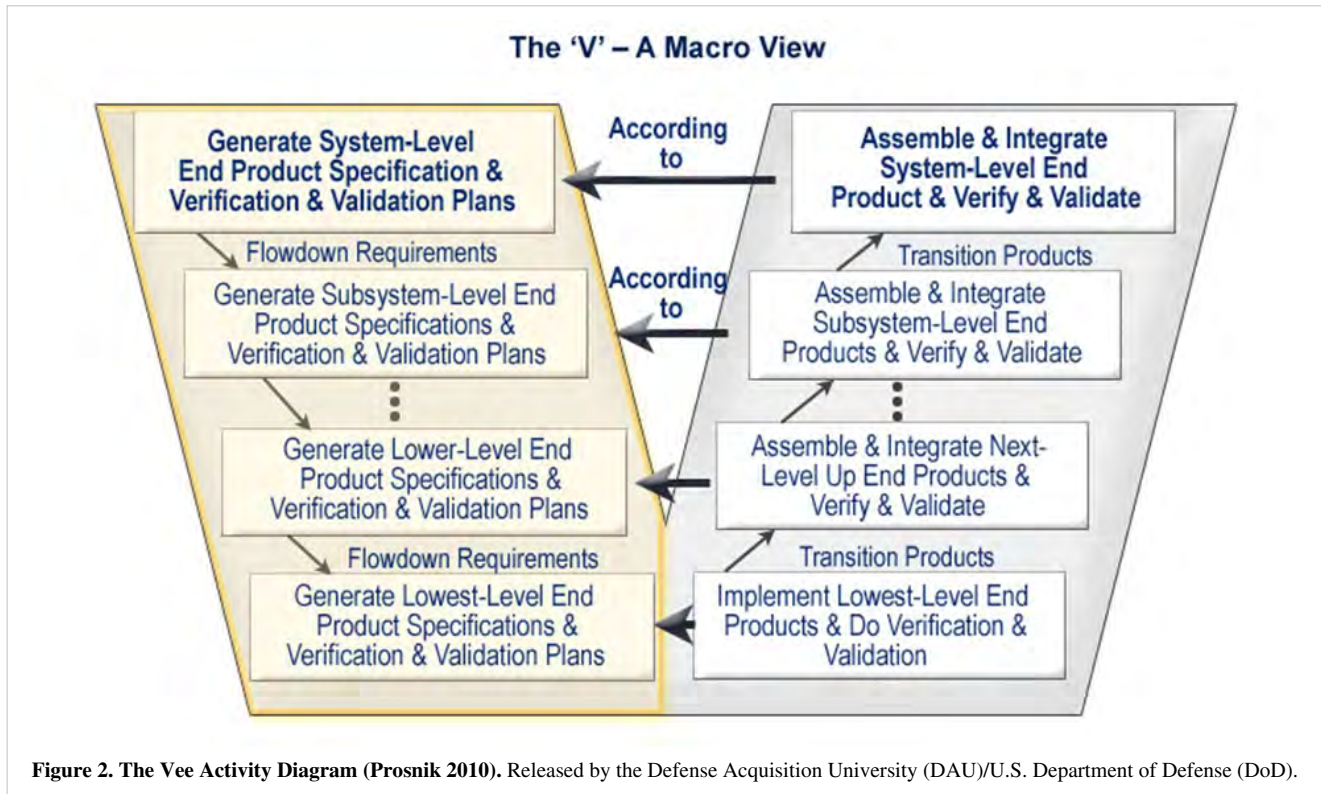


Figure 2. The Vee Activity Diagram (Prosnik 2010). Released by the Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

The left side of the Vee model demonstrates the development of system elements specifications and design descriptions. In this stage, verification and validation plans are developed, which are later used to determine whether realized system elements (products, services, or enterprises) are compliant with specifications and stakeholder needs and requirements. Also, during this stage initial specifications become flow-down requirements for lower-level system models. In terms of time frame, these activities take place early in the system's life cycle. These activities are discussed further in the System Definition KA. However, it is important to understand that some of the system realization activities are initiated at the same time as system definition activities; this is the case with integration, verification and validation planning in particular.

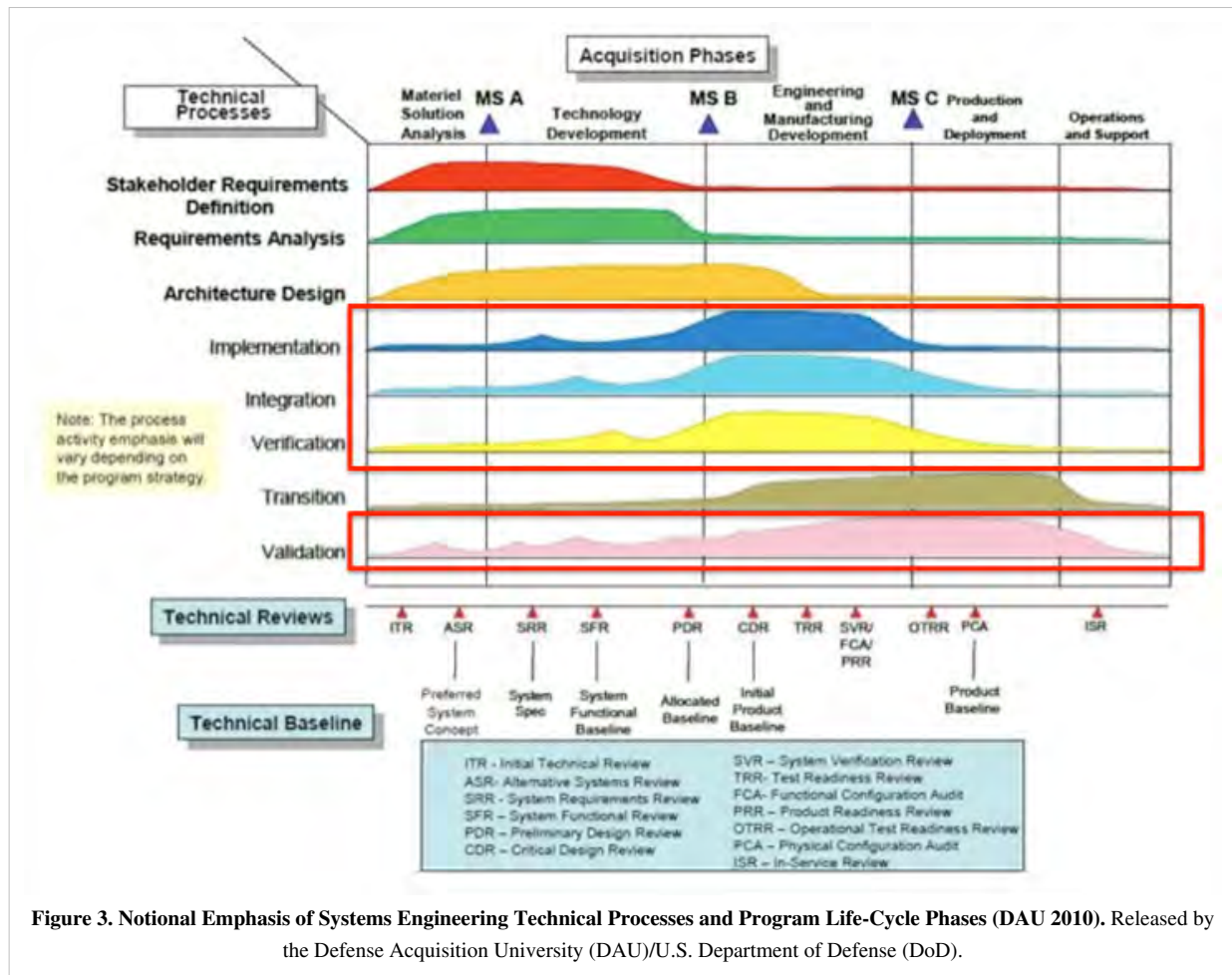
The right side of the Vee model, as illustrated in Figure 2, shows the system elements (products, services, or enterprises) are assembled according to the system model described on the left side of the Vee (integration). Verification and validation activities determine how well the realized system fulfills the stakeholder requirements, the system requirements, and design properties. These activities should follow the plans developed on the left side of the Vee. Integration can be done continuously, incrementally and/or iteratively, supported by verification and validation (V&V) efforts. For example, integration typically starts at the bottom of the Vee and continues upwards to the top of the Vee.

The U.S. Defense Acquisition University (DAU) provides an overview of what occurs during system realization:

Once the products of all system models have been fully defined, Bottom-Up End Product Realization can be initiated. This begins by applying the Implementation Process to buy, build, code or reuse end products. These implemented end products are verified against their design descriptions and specifications, validated against Stakeholder Requirements and then transitioned to the next higher

system model for integration. End products from the Integration Process are successively integrated upward, verified and validated, transitioned to the next acquisition phase or transitioned ultimately as the End Product to the user. (Prosnik 2010)

While the systems engineering (SE) technical processes are life cycle processes, the processes are concurrent, and the emphasis of the respective processes depends on the phase and maturity of the design. Figure 3 portrays (from left to right) a notional emphasis of the respective processes throughout the systems acquisition life cycle from the perspective of the U.S. Department of Defense (DoD). It is important to note that from this perspective, these processes do not follow a linear progression; instead, they are concurrent, with the amount of activity in a given area changing over the system's life cycle. The red boxes indicate the topics that will be discussed as part of realization.



References

Works Cited

- DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.
- Prosnik, G. 2010. Materials from "Systems 101: Fundamentals of Systems Engineering Planning, Research, Development, and Engineering". DAU distance learning program. eds. J. Snoderly, B. Zimmerman. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

Primary References

- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.
- ISO/IEC/IEEE. 2023. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.
- Martin, J.N. 1997. *Systems Engineering Guidebook: A process for developing systems and products*, 1st ed. Boca Raton, FL, USA: CRC Press.
- NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Additional References

- DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.
- DAU. *Your Acquisition Policy and Discretionary Best Practices Guide*. In Defense Acquisition University (DAU)/U.S. Department of Defense (DoD) [database online]. Ft Belvoir, VA, USA. Available at: <https://dag.dau.mil/Pages/Default.aspx> (accessed 2010).
- ECSS. 2009. *Systems Engineering General Requirements*. Noordwijk, Netherlands: Requirements and Standards Division, European Cooperation for Space Standardization (ECSS), 6 March 2009. ECSS-E-ST-10C.
- IEEE. 2012. "Standard for System and Software Verification and Validation". Institute of Electrical and Electronics Engineers. IEEE-1012.

System Implementation

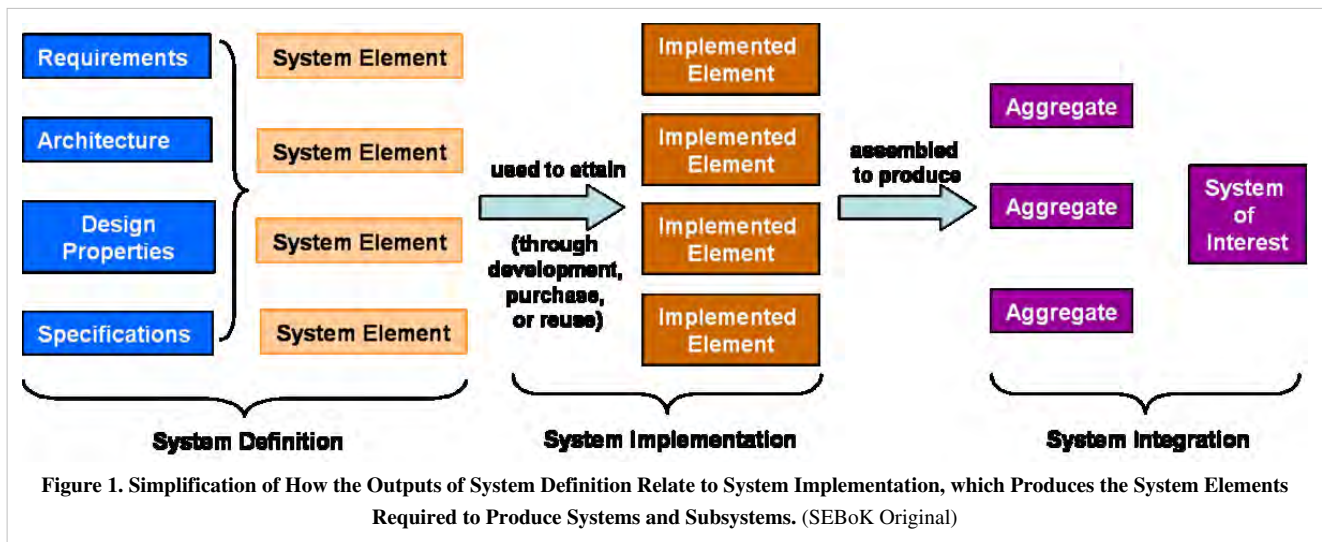
- Lead Authors:
- John Snoderly and Alan Faisandier

System Implementation uses the structure created during architectural design and the results of system analysis to construct system elements that meet the stakeholder needs and requirements and system requirements developed in the early life cycle phases. These system elements are then integrated to form intermediate aggregates and finally the complete system-of-interest (SoI). See System Integration.

Definition and Purpose

Implementation is the process that actually yields the lowest-level system elements in the system hierarchy (system breakdown structure). System elements are made, bought, or reused. Production involves the hardware fabrication processes of forming, removing, joining, and finishing, the software realization processes of coding and testing, or the operational procedures development processes for operators' roles. If implementation involves a production process, a manufacturing system which uses the established technical and management processes may be required.

The purpose of the implementation process is to design and create (or fabricate) a system element conforming to that element's design properties and/or requirements. The element is constructed employing appropriate technologies and industry practices. This process bridges the system definition processes and the integration process. Figure 1 portrays how the outputs of system definition relate to system implementation, which produces the implemented (system) elements required to produce aggregates and the SoI.

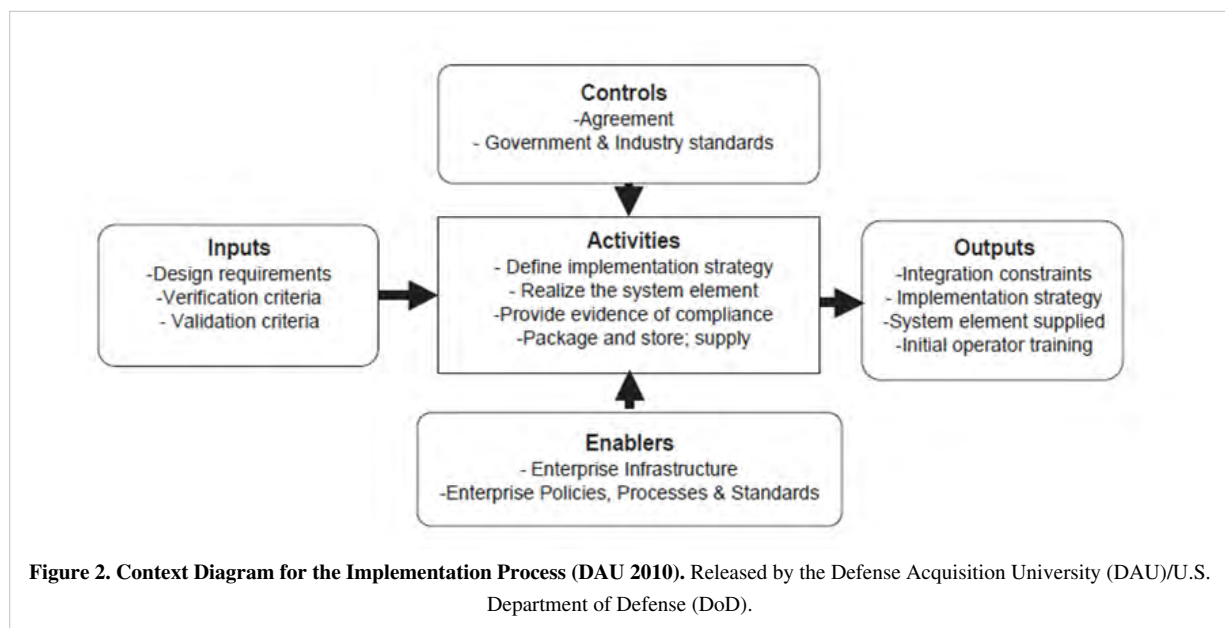


Process Approach

Purpose and Principle of the Approach

During the implementation process, engineers apply the design properties and/or requirements allocated to a system element to design and produce a detailed description. They then fabricate, code, or build each individual element using specified materials, processes, physical or logical arrangements, standards, technologies, and/or information flows outlined in detailed descriptions (drawings or other design documentation). A system element will be verified against the detailed description of properties and validated against its requirements.

If subsequent verification and validation (V&V) actions or configuration audits reveal discrepancies, recursive interactions occur, which includes predecessor activities or processes, as required, to mitigate those discrepancies and to modify, repair, or correct the system element in question. Figure 2 provides the context for the implementation process from the perspective of the U.S. Defense Acquisition University (DAU).



Such figures provide a useful overview of the systems engineering (SE) community's perspectives on what is required for implementation and what the general results of implementation may be. These are further supported by the discussion of implementation inputs, outputs, and activities found in the National Aeronautics and Space

Association's (NASA's) *Systems Engineering Handbook* (NASA 2007). It is important to understand that these views are process -oriented. While this is a useful model, examining implementation only in terms of process can be limiting.

Depending on the technologies and systems chosen when a decision is made to produce a system element, the implementation process outcomes may generate constraints to be applied on the architecture of the higher-level system; those constraints are normally identified as derived system requirements and added to the set of system requirements applicable to this higher-level system. The architectural design must take those constraints into account.

If the decision is made to purchase or reuse an existing system element, it must be identified as a constraint or system requirement applicable to the architecture of the higher-level system. Conversely, the implementation process may involve some adaptation or adjustments to the system requirement in order to be integrated into a higher-level system or aggregate.

Implementation also involves packaging, handling, and storage, depending on the concerned technologies and where or when the system requirement needs to be integrated into a higher-level aggregate. Developing the supporting documentation for a system requirement, such as the manuals for operation, maintenance, and/or installation, is also a part of the implementation process; these artifacts are utilized in the system deployment and use phase. The system element requirements and the associated verification and validation criteria are inputs to this process; these inputs come from the architectural design process detailed outputs.

Execution of the implementation process is governed by both industrial and government standards and the terms of all applicable agreements. This may include conditions for packaging and storage, as well as preparation for use activities, such as operator training. In addition, packaging, handling, storage, and transportation (PHS&T) considerations will constrain the implementation activities. For more information, refer to the discussion of PHS&T in the System Deployment and Use article. The developing or integrating organization will likely have enterprise-level safety practices and guidelines that must also be considered.

Activities of the Process

The following major activities and tasks are performed during this process:

- **Define the implementation strategy** - Implementation process activities begin with detailed design and include developing an implementation strategy that defines fabrication and coding procedures, tools and equipment to be used, implementation tolerances, and the means and criteria for auditing configuration of resulting elements to the detailed design documentation. In the case of repeated system element implementations (such as for mass manufacturing or replacement elements), the implementation strategy is defined and refined to achieve consistent and repeatable element production; it is retained in the project decision database for future use. The implementation strategy contains the arrangements for packing, storing, and supplying the implemented element.
 - **Realize the system element** - Realize or adapt and produce the concerned system element using the implementation strategy items as defined above. Realization or adaptation is conducted with regard to standards that govern applicable safety, security, privacy, and environmental guidelines or legislation and the practices of the relevant implementation technology. This requires the fabrication of hardware elements, development of software elements, definition of training capabilities, drafting of training documentation, and the training of initial operators and maintainers.
 - **Provide evidence of compliance** - Record evidence that the system element meets its requirements and the associated verification and validation criteria as well as the legislation policy. This requires the conduction of peer reviews and unit testing, as well as inspection of operation and maintenance manuals. Acquire measured properties that characterize the implemented element (weight, capacities, effectiveness, level of performance, reliability, availability, etc.).
 - **Package, store, and supply the implemented element** - This should be defined in the implementation strategy.
-

Artifacts and Ontology Elements

This process may create several artifacts such as:

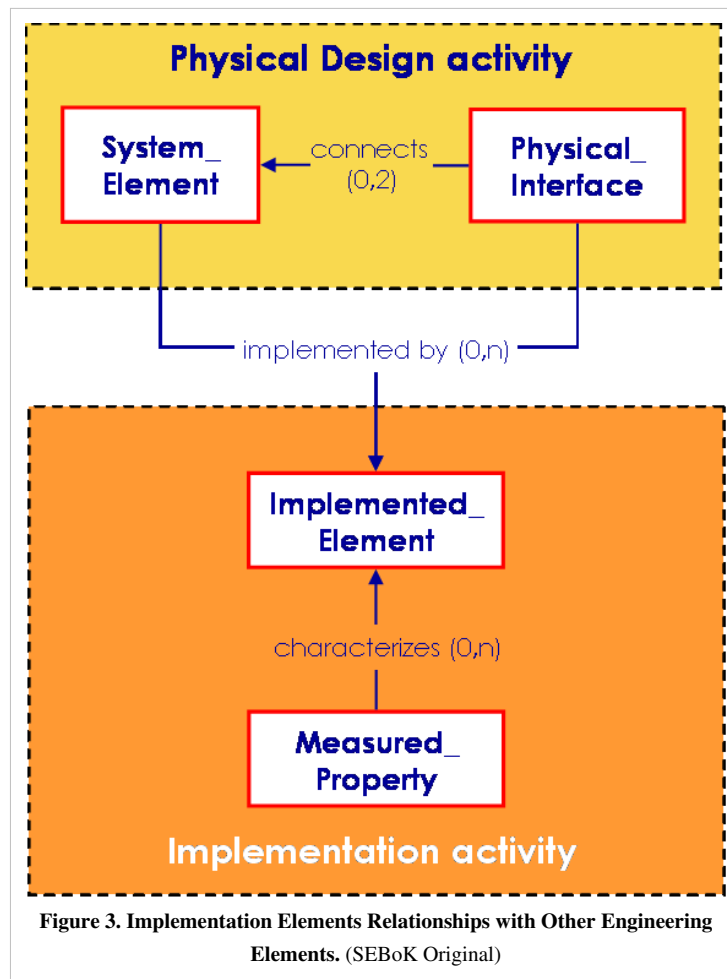
- an implemented system
- implementation tools
- implementation procedures
- an implementation plan or strategy
- verification reports
- issue, anomaly, or trouble reports
- change requests (about design)

This process handles the ontology elements shown in Table 1 below.

**Table 1. Main Ontology Elements as Handled within System Element Implementation.
(SEBoK Original)**

Element	Definition
	Attributes (examples)
Implemented Element	An implemented element is a system element that has been implemented. In the case of hardware it is marked with a part/serial number. Identifier, name, description, type (hardware, software application, software piece, mechanical part, electric art, electronic component, operator role, procedure, protocol, manual, etc.)
Measured Property	A measured property is a characteristic of the implemented element established after its implementation. The measured properties characterize the implemented system element when it is completely realized, verified, and validated. If the implemented element complies with a design property, the measured property should equal the design property. Otherwise one must identify the difference or non-conformance which treatment could conclude to modify the design property and possibly the related requirements, or to modify (correct, repair) the implemented element, or to identify a deviation. Identifier, name, description, type (effectiveness, availability, reliability, maintainability, weight, capacity, etc.), value, unit, etc.

The main relationships between ontology elements are presented in Figure 3.



Methods, Techniques, and Tools

There are many software tools available in the implementation and integration phases. The most basic method would be the use of N-squared diagrams as discussed in Jeff Grady's book *System Integration* (Grady 1994).

Checking and Correctness of Implementation

Proper implementation checking and correctness should include testing to determine if the implemented element (i.e., piece of software, hardware, or other product) works in its intended use. Testing could include mockups and breadboards, as well as modeling and simulation of a prototype or completed pieces of a system. Once this is completed successfully, the next process would be system integration.

References

Works Cited

- DAU. February 19, 2010. *Defense acquisition guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense.
- Grady, J.O. 1994. *System integration*. Boca Raton, FL, USA: CRC Press, Inc.
- NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Primary References

- DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.
- Grady, J.O. 1994. *System Integration*. Boca Raton, FL, USA: CRC Press, Inc.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Additional References

- Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

System Integration

- Lead Authors:
- John Snoderly, Alan Faisandier, and Scott Jackson

System integration consists of taking delivery of the implemented system elements which compose the system-of-interest (SoI), assembling these implemented elements together, and performing the verification and validation actions (V&V actions) in the course of the assembly. The ultimate goal of system integration is to ensure that the individual system elements function properly as a whole and satisfy the design properties or characteristics of the system. System integration is one part of the realization effort and relates only to developmental items. Integration should not be confused with the assembly of end products on a production line. To perform the production, the assembly line uses a different order from that used by integration.

Definition and Purpose

System integration consists of a process that “*iteratively combines implemented system elements to form complete or partial system configurations in order to build a product or service. It is used recursively for successive levels of the system hierarchy.*” (ISO/IEC 15288 2015, 68). The process is extended to any kind of product system, service system, and enterprise system. The purpose of system integration is to prepare the SoI for final validation and transition either for use or for production. Integration consists of progressively assembling aggregates of implemented elements that compose the SoI as architected during design, and to check correctness of static and dynamic aspects of interfaces between the implemented elements.

The U.S. Defense Acquisition University (DAU) provides the following context for integration: *The integration process will be used . . . for the incorporation of the final system into its operational environment to ensure that the system is integrated properly into all defined external interfaces. The interface management process is particularly important for the success of the integration process, and iteration between the two processes will occur* (DAU 2010).

The purpose of system integration can be summarized as below:

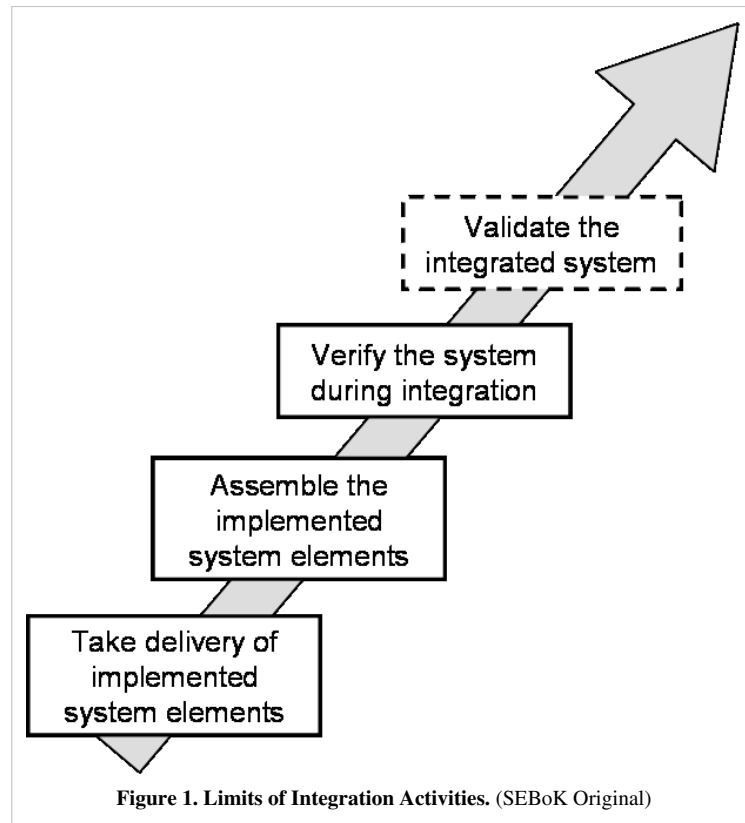
- Completely assemble the implemented elements to make sure that they are compatible with each other.
 - Demonstrate that the aggregates of implemented elements perform the expected functions and meet measures of performance/effectiveness.
 - Detect defects/faults related to design and assembly activities by submitting the aggregates to focused V&V actions.
-

Note: In the systems engineering literature, sometimes the term *integration* is used in a larger context than in the present topic. In this larger sense, it concerns the technical effort to simultaneously design and develop the system and the processes for developing the system through concurrent consideration of all life cycle stages, needs, and competences. This approach requires the "integration" of numerous skills, activities, or processes.

Principles

Boundary of Integration Activity

Integration can be understood as the whole bottom-up branch of the Vee Model, including the tasks of assembly and the appropriate verification tasks. See Figure 1 below:



The assembly activity joins together, and physically links, the implemented elements. Each implemented element is individually verified and validated prior to entering integration. Integration then adds the verification activity to the assembly activity, excluding the final validation.

The final validation performs operational tests that authorize the transition for use or the transition for production. Remember that system integration only endeavors to obtain pre-production prototypes of the concerned product, service, or enterprise. If the product, service, or enterprise is delivered as a unique exemplar, the final validation activity serves as acceptance for delivery and transfer for use. If the prototype has to be produced in several exemplars, the final validation serves as acceptance to launch their production. The definition of the optimized operations of assembly which will be carried out on a production line relates to the manufacturing process and not to the integration process.

Integration activity can sometimes reveal issues or anomalies that require modifications of the design of the system. Modifying the design is not part of the integration process but concerns only the design process. Integration only deals with the assembly of the implemented elements and verification of the system against its properties as designed. During assembly, it is possible to carry out tasks of finishing touches which require simultaneous use of several implemented elements (e.g., paint the whole after assembly, calibrate a biochemical component, etc.). These

tasks must be planned in the context of integration and are not carried out on separate implemented elements and do not include modifications related to design.

Aggregation of Implemented Elements

The integration is used to systematically assemble a higher-level system from lower-level ones (implemented system elements) that have been implemented. Integration often begins with analysis and simulations (e.g., various types of prototypes) and progresses through increasingly more realistic systems and system elements until the final product, service, or enterprise is achieved.

System integration is based on the notion of an aggregate - a subset of the system made up of several implemented elements (implemented system elements and physical interfaces) on which a set of V&V actions is applied. Each aggregate is characterized by a configuration which specifies the implemented elements to be physically assembled and their configuration status.

To perform V&V actions, a V&V configuration that includes the aggregate plus V&V tools is constituted. The V&V tools are enabling products and can be simulators (simulated implemented elements), stubs or caps, activators (launchers, drivers), harness, measuring devices, etc.

Integration by Level of System

According to the Vee Model, system definition (top-down branch) is done by successive levels of decomposition; each level corresponds to the physical architecture of systems and system elements. The integration (bottom-up branch) takes the opposite approach of composition (i.e., a level by level approach). On a given level, integration is done on the basis of the physical architecture defined during system definition.

Integration Strategy

The integration of implemented elements is generally performed according to a predefined strategy. The definition of the integration strategy is based on the architecture of the system and relies on the way the architecture of the system has been designed. The strategy is described in an integration plan that defines the minimum configuration of expected aggregates, the order of assembly of these aggregates in order to support efficient subsequent verification and validation actions (e.g., inspections and/or testing), techniques to check or evaluate interfaces, and necessary capabilities in the integration environment to support combinations of aggregates. The integration strategy is thus elaborated starting from the selected verification and validation strategy. See the System Verification and System Validation topics.

To define an integration strategy, there are several possible integration approaches/techniques that may be used individually or in combination. The selection of integration techniques depends on several factors; in particular, the type of system element, delivery time, order of delivery, risks, constraints, etc. Each integration technique has strengths and weaknesses which should be considered in the context of the SoI. Some integration techniques are summarized in Table 1 below.

Table 1. Integration Techniques. (SEBoK Original)

Integration Technique	Description
Global Integration	Also known as <i>big-bang integration</i>; all the delivered implemented elements are assembled in only one step. • This technique is simple and does not require simulating the implemented elements not being available at that time. • Difficult to detect and localize faults; interface faults are detected late. • Should be reserved for simple systems, with few interactions and few implemented elements without technological risks.
Integration "with the Stream"	The delivered implemented elements are assembled as they become available. • Allows starting the integration quickly. • Complex to implement because of the necessity to simulate the implemented elements not yet available. Impossible to control the end-to-end "functional chains"; consequently, global tests are postponed very late in the schedule. • Should be reserved for well-known and controlled systems without technological risks.
Incremental Integration	In a predefined order, either one or a very few implemented elements are added to an already integrated increment of implemented elements. • Fast localization of faults: a new fault is usually localized in lately integrated implemented elements or dependent of a faulty interface. • Require simulators for absent implemented elements. Require many test cases, as each implemented element addition requires the verification of the new configuration and regression testing. • Applicable to any type of architecture.
Subsets Integration	Implemented elements are assembled by subsets, and then subsets are assembled together (a subset is an aggregate); could also be called "functional chains integration". • Time saving due to parallel integration of subsets; delivery of partial products is possible. Requires less means and fewer test cases than integration by increments. • Subsets shall be defined during the design. • Applicable to architectures composed of sub-systems.
Top-Down Integration	Implemented elements or aggregates are integrated in their activation or utilization order. • Availability of a skeleton and early detection of architectural faults, definition of test cases close to reality, and the re-use of test data sets possible. • Many stubs/caps need to be created; difficult to define test cases of the leaf-implemented elements (lowest level). • Mainly used in software domain. Start from the implemented element of higher level; implemented elements of lower level are added until leaf-implemented elements.
Bottom-Up Integration	Implemented elements or aggregates are integrated in the opposite order of their activation or utilization. • Easy definition of test cases; early detection of faults (usually localized in the leaf-implemented elements); reduce the number of simulators to be used. An aggregate can be a sub-system. • Test cases shall be redefined for each step, drivers are difficult to define and realize, implemented elements of lower levels are "over-tested", and does not allow architectural faults to be quickly detected. • Mainly used in software domain, but can be used in any kind of system.
Criterion Driven Integration	The most critical implemented elements compared to the selected criterion are first integrated (dependability, complexity, technological innovation, etc.). Criteria are generally related to risks. • Allows early and intensive testing of critical implemented elements; early verification of design choices. • Test cases and test data sets are difficult to define.

Usually, a mixed integration technique is selected as a trade-off between the different techniques listed above, allowing optimization of work and adaptation of the process to the system under development. The optimization takes into account the realization time of the implemented elements, their delivery scheduled order, their level of complexity, the technical risks, the availability of assembly tools, cost, deadlines, specific personnel capability, etc.

Process Approach

Activities of the Process

Major activities and tasks performed during this process include:

- **Establishing the integration plan** (this activity is carried out concurrently to the design activity of the system) that defines:
 - The optimized integration strategy – order of aggregates assembly using appropriate integration techniques.
 - The V&V actions to be processed for the purpose of integration.
 - The configurations of the aggregates to be assembled and verified.
 - The integration means and verification means (dedicated enabling products) that may include assembly procedures, assembly tools (harness, specific tools), V&V tools (simulators, stubs/caps, launchers, test benches, devices for measuring, etc.), and V&V procedures.
- **Obtain the integration means** and verification means as defined in the integration plan. The acquisition of the means can be accomplished through various ways such as procurement, development, reuse, and sub-contracting; usually the acquisition of the complete set of means is a mix of these methods.
- **Take delivery** of each implemented element:
 - Unpack and reassemble the implemented element with its accessories.
 - Check the delivered configuration, conformance of implemented elements and compatibility of interfaces, and ensure the presence of mandatory documentation.
- **Assemble the implemented elements** into aggregates:
 - Gather the implemented elements to be assembled, the integration means (assembly tools, assembly procedures), and the verification means (V&V tools and procedures).
 - Connect the implemented elements to each other using assembly tools to constitute aggregates in the order prescribed by the integration plan and in assembly procedures.
 - Add or connect the V&V tools to the aggregates as predefined.
 - Carry out eventual operations of welding, gluing, drilling, tapping, adjusting, tuning, painting, parametering, etc.
- **Verify each aggregate:**
 - Check the aggregate is correctly assembled according to established procedures.
 - Perform the verification process that uses verification and validation procedures and check that the aggregate shows the right design properties/specified requirements.
 - Record integration results/reports and potential issue reports, change requests, etc.

Artifacts and Ontology Elements

This process may create several artifacts such as:

- an integrated system
- assembly tools
- assembly procedures
- integration plans
- integration reports
- issue/anomaly/trouble reports
- change requests (about design)

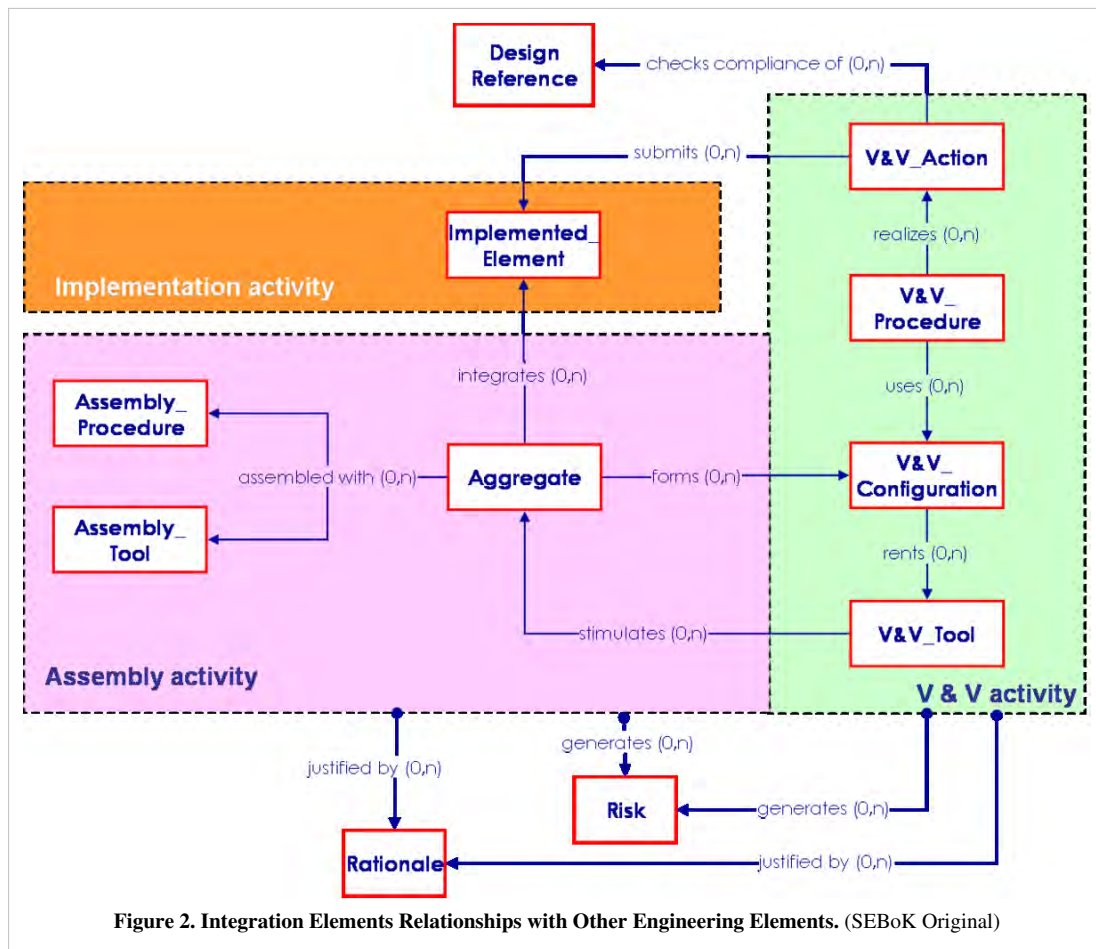
This process utilizes the ontology elements discussed in Table 2.

Table 2. Main Ontology Elements as Handled within System Integration. (SEBoK Original)

Element	Definition
Aggregate	Attributes
	An aggregate is a subset of the system made up of several system elements or systems on which a set of verification actions is applied.
Assembly Procedure	Identifier, name, description
	An assembly procedure groups a set of elementary assembly actions to build an aggregate of implemented system elements.
Assembly Tool	Identifier, name, description, duration, unit of time
	An assembly tool is a physical tool used to connect, assemble, or link several implemented system elements to build aggregates (specific tool, harness, etc.).
Risk	Identifier, name, description
	An event having a probability of occurrence and a gravity degree on its consequence onto the system mission or on other characteristics (used for technical risk in engineering). A risk is the combination of vulnerability and of a danger or a threat.
Rationale	Identifier, name, description, status
	An argument that provides the justification for the selection of an engineering element.
	Identifier, name, description (rationale, reasons for defining an aggregate, assembly procedure, assembly tool)

Note: verification and validation ontology elements are described in the System Verification and System Validation topics.

The main relationships between ontology elements are presented in Figure 2.



Checking and Correctness of Integration

The main items to be checked during the integration process include the following:

- The integration plan respects its template.
- The expected assembly order (integration strategy) is realistic.
- No system element and physical interface set out in the system design document is forgotten.
- Every interface and interaction between implemented elements is verified.
- Assembly procedures and assembly tools are available and validated prior to beginning the assembly.
- V&V procedures and tools are available and validated prior to beginning the verification.
- Integration reports are recorded.

Methods and Techniques

Several different approaches are summarized above in the section Integration Strategy ^[1] (above) that may be used for integration, yet other approaches exist. In particular, important integration strategies for intensive software systems include: vertical integration, horizontal integration, and star integration.

Coupling Matrix and N-squared Diagram

One of the most basic methods to define the aggregates and the order of integration would be the use of N-Squared diagrams (Grady 1994, 190).

In the integration context, the coupling matrices are useful for optimizing the aggregate definition and verification of interfaces:

- The integration strategy is defined and optimized by reorganizing the coupling matrix in order to group the implemented elements in aggregates, thus minimizing the number of interfaces to be verified between aggregates (see Figure 3).

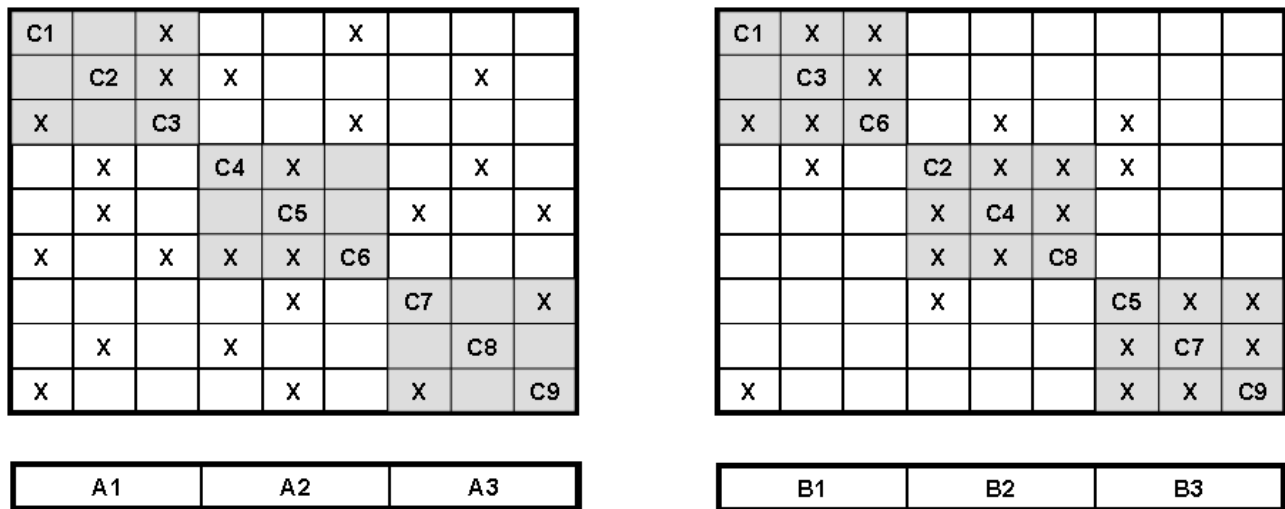


Figure 3. Initial Arrangement of Aggregates on the Left; Final Arrangement After Reorganization on the Right. (SEBoK Original)

- When verifying the interactions between aggregates, the matrix is an aid tool for fault detection. If by adding an implemented element to an aggregate an error is detected, the fault can be related to the implemented element, to the aggregate, or to the interfaces. If the fault is related to the aggregate, it can relate to any implemented element or any interface between the implemented elements internal to the aggregate.

Application to Product Systems, Service Systems, and Enterprise Systems

As the nature of implemented system elements and physical interfaces is different for these types of systems, the aggregates, the assembly tools, and the V&V tools are different. Some integration techniques are more appropriate to specific types of systems. Table 3 below provides some examples.

Table 3. Different Integration Elements for Product, Service, and Enterprise Systems.
(SEBoK Original)

Element	Product System	Service System	Enterprise System
System Element	Hardware Parts (mechanics, electronics, electrical, plastic, chemical, etc.) Operator Roles Software Pieces	Processes, data bases, procedures, etc. Operator Roles Software Applications	Corporate, direction, division, department, project, technical team, leader, etc. IT components
Physical Interface	Hardware parts, protocols, procedures, etc.	Protocols, documents, etc.	Protocols, procedures, documents, etc.
Assembly Tools	Harness, mechanical tools, specific tools Software Linker	Documentation, learning course, etc.	Documentation, learning, moving of office
Verification Tools	Test bench, simulator, launchers, stub/cap	Activity/scenario models, simulator, human roles rehearsal, computer, etc. Skilled Experts	Activity/scenario models, simulator, human roles rehearsal
Validation Tools	Operational environment	Operational environment	Operational environment

Recommended Integration Techniques	Top down integration technique	Subsets integration technique (functional chains)	Global integration technique
	Bottom Up Integration technique		Incremental integration

Practical Considerations

Key pitfalls and good practices related to system integration are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing SE Measurement are provided in Table 4.

Table 4. Major Pitfalls with System Integration. (SEBoK Original)

Pitfall	Description
Delay of expected element	The experience shows that the implemented elements always do not arrive in the expected order and the tests never proceed or result as foreseen; therefore, the integration strategy should allow a great flexibility.
Big-bang not appropriate	The "big-bang" integration technique is not appropriate for a fast detection of faults. It is thus preferable to verify the interfaces progressively all along the integration.
Integration plan too late	The preparation of the integration activities is planned too late in the project schedule, typically when first implemented elements are delivered.

Good Practices

Some good practices gathered from the references are provided in Table 5.

Table 5. Proven Practices with System Integration. (SEBoK Original)

Practice	Description
Start earlier development of means	The development of assembly tools and verification and validation tools can be taken as long as the system development itself. It should be started as early as possible as soon as the preliminary design is nearly frozen.
Integration means seen as enabling systems	The development of integration means (assembly tools, verification, and validation tools) can be seen as enabling systems, using system definition and system realization processes as described in this SEBoK, and managed as projects. These projects can be led by the project of the corresponding system-of-interest, but assigned to specific system blocks, or can be subcontracted as separate projects.
Use coupling matrix	A good practice consists in gradually integrating aggregates in order to detect faults more easily. The use of the coupling matrix applies for all strategies and especially for the bottom up integration strategy.
Flexible integration plan and schedule	The integration process of complex systems cannot be easily foreseeable and its progress control difficult to observe. This is why it is recommended to plan integration with specific margins, using flexible techniques, and integrating sets by similar technologies.
Integration and design teams	The integration responsible should be part of the design team.

References

Works Cited

DAU. February 19, 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Primary References

INCOSE. 2010. *Systems Engineering Handbook: A Guide for Systems Life Cycle Processes and Activities*. Version 3.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.

NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Additional References

Buede, D.M. 2009. *The Engineering Design of Systems: Models and Methods*. 2nd ed. Hoboken, NJ, USA: John Wiley & Sons Inc.

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense. February 19, 2010.

Gold-Bernstein, B. and W.A. Ruh. 2004. *Enterprise integration: The essential guide to integration solutions*. Boston, MA, USA: Addison Wesley Professional.

Grady, J.O. 1994. *System integration*. Boca Raton, FL, USA: CRC Press, Inc.

Hitchins, D. 2009. "What are the General Principles Applicable to Systems?" *INCOSE Insight* 12(4):59-63.

Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Hoboken, NJ, USA: John Wiley & Sons.

Reason, J. 1997. *Managing the Risks of Organizational Accidents*. Aldershot, UK: Ashgate Publishing Limited.

References

[1] https://sebokwiki.org/1.0.1/index.php?title=System_Integration#Integration_Strategy

System Verification

- Lead Authors:
- John Snoderly and Alan Faisandier

System Verification is a set of actions used to check the *correctness* of any element, such as a system element, a system, a document, a service, a task, a requirement, etc. These types of actions are planned and carried out throughout the life cycle of the system. Verification is a generic term that needs to be instantiated within the context it occurs. As a process, verification is a transverse activity to every life cycle stage of the system. In particular, during the development cycle of the system, the verification process is performed in parallel with the system definition and system realization processes and applies to any activity and any product resulting from the activity. The activities of every life cycle process and those of the verification process can work together. For example, the integration process frequently uses the verification process. It is important to remember that verification, while separate from validation, is intended to be performed in conjunction with validation.

Definition and Purpose

Verification is the confirmation, through the provision of objective evidence, that specified requirements have been fulfilled. With a note added in ISO/IEC/IEEE 15288, the scope of verification includes a set of activities that compares a system or system element against the requirements, architecture and design characteristics, and other properties to be verified (ISO/IEC/IEEE 2015). This may include, but is not limited to, specified requirements, design description, and the system itself.

The purpose of verification, as a generic action, is to identify the faults/defects introduced at the time of any transformation of inputs into outputs. Verification is used to provide information and evidence that the transformation was made according to the selected and appropriate methods, techniques, standards, or rules.

Verification is based on tangible evidence; i.e., it is based on information whose veracity can be demonstrated by factual results obtained from techniques such as inspection, measurement, testing, analysis, calculation, etc. Thus, the process of verifying a system (product, service, enterprise, or system of systems (SoS)) consists of comparing the realized characteristics or properties of the product, service, or enterprise against its expected design properties.

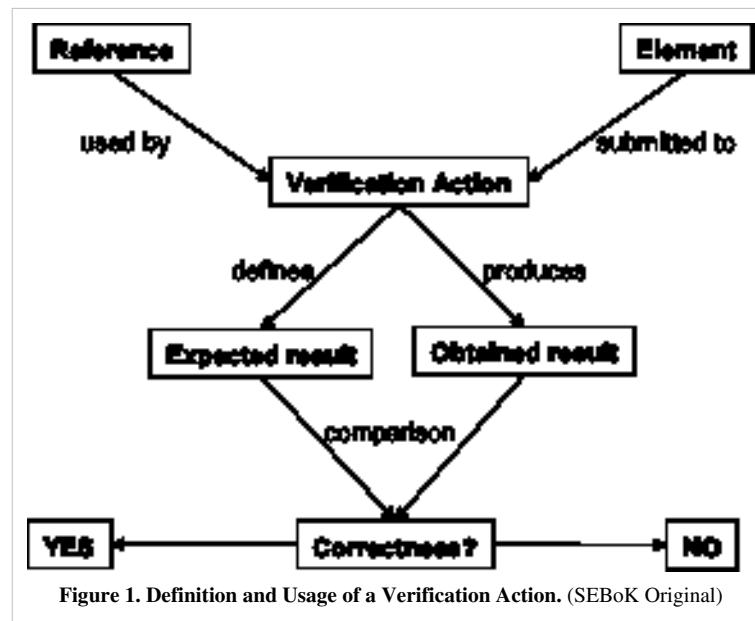
Principles and Concepts

Concept of Verification Action

Why Verify?

In the context of human realization, any human thought is susceptible to error. This is also the case with any engineering activity. Studies in human reliability have shown that people trained to perform a specific operation make around 1-3 errors per hour in best case scenarios. In any activity, or resulting outcome of an activity, the search for potential errors should not be neglected, regardless of whether or not one thinks they will happen or that they should not happen; the consequences of errors can cause extremely significant failures or threats.

A **verification action** is defined, and then performed, as shown in Figure 1.



The definition of a verification action applied to an engineering element includes the following:

- Identification of the element on which the verification action will be performed
- Identification of the reference to define the expected result of the verification action (see examples of reference in Table 1)

The performance of a verification action includes the following:

- Obtaining a result by performing the verification action onto the submitted element
- Comparing the obtained result with the expected result
- Deducing the degree of correctness of the element

What to Verify?

Any engineering element can be verified using a specific reference for comparison: stakeholder requirement, system requirement, function, system element, document, etc. Examples are provided in Table 1.

Table 1. Examples of Verified Items. (SEBoK Original)

Items	Explanation for Verification
Document	To verify a document is to check the application of drafting rules.
Stakeholder Requirement and System Requirement	To verify a stakeholder requirement or a system requirement is to check the application of syntactic and grammatical rules, characteristics defined in the stakeholder requirements definition process, and the system requirements definition process such as necessity, implementation free, unambiguous, consistent, complete, singular, feasible, traceable, and verifiable.
Design	To verify the design of a system is to check its logical and physical architecture elements against the characteristics of the outcomes of the design processes.
System	To verify a system (product, service, or enterprise) is to check its realized characteristics or properties against its expected design characteristics.
Aggregate	To verify an aggregate for integration is to check every interface and interaction between implemented elements.
Verification Procedure	To verify a verification procedure is to check the application of a predefined template and drafting rules.

Verification versus Validation

The term *verification* is often associated with the term *validation* and understood as a single concept of V&V. Validation is used to ensure that *one is working the right problem*, whereas verification is used to ensure that *one has solved the problem right* (Martin 1997). From an actual and etymological meaning, the term verification comes from the Latin *verus*, which means truth, and *facere*, which means to make/perform. Thus, verification means to prove that something is *true* or correct (a property, a characteristic, etc.). The term validation comes from the Latin *valere*, which means to become strong, and has the same etymological root as the word *value*. Thus, validation means to prove that something has the right features to produce the expected effects. (Adapted from "Verification and Validation in plain English" (Lake INCOSE 1999).)

The main differences between the verification process and the validation process concern the references used to check the correctness of an element, and the acceptability of the effective correctness.

- Within verification, comparison between the expected result and the obtained result is generally binary, whereas within validation, the result of the comparison may require a judgment of value regarding whether or not to accept the obtained result compared to a threshold or limit.
- Verification relates more to one element, whereas validation relates more to a set of elements and considers this set as a whole.
- Validation presupposes that verification actions have already been performed.
- The techniques used to define and perform the verification actions and those for validation actions are very similar.

Integration, Verification, and Validation of the System

There is sometimes a misconception that verification occurs after integration and before validation. In most cases, it is more appropriate to begin verification activities during development or implementation and to continue them into deployment and use.

Once the system elements have been realized, they are integrated to form the complete system. Integration consists of assembling and performing verification actions as stated in the integration process. A final validation activity generally occurs when the system is integrated, but a certain number of validation actions are also performed parallel to the system integration in order to reduce the number of verification actions and validation actions while controlling the risks that could be generated if some checks are excluded. Integration, verification, and validation are intimately processed together due to the necessity of optimizing the strategy of verification and validation, as well as the strategy of integration.

Process Approach

Purpose and Principle of the Approach

The purpose of the verification process is to confirm that the system fulfills the specified design requirements. This process provides the information required to effect the remedial actions that correct non-conformances in the realized system or the processes that act on it - see ISO/IEC/IEEE 15288 (ISO/IEC/IEEE 2015).

Each system element and the complete system itself should be compared against its own design references (specified requirements). As stated by Dennis Buede, *verification is the matching of [configuration items], components, sub-systems, and the system to corresponding requirements to ensure that each has been built right* (Buede 2009). This means that the verification process is instantiated as many times as necessary during the global development of the system. Because of the generic nature of a process, the verification process can be applied to any engineering element that has conducted to the definition and realization of the system elements and the system itself.

Facing the huge number of potential verification actions that may be generated by the normal approach, it is necessary to optimize the verification strategy. This strategy is based on the balance between what must be verified and constraints, such as time, cost, and feasibility of testing, which naturally limit the number of verification actions and the risks one accepts when excluding some verification actions.

Several approaches exist that may be used for defining the verification process. The International Council on Systems Engineering (INCOSE) dictates that two main steps are necessary for verification: planning and performing verification actions (INCOSE 2012). NASA has a slightly more detailed approach that includes five main steps: prepare verification, perform verification, analyze outcomes, produce a report, and capture work products (NASA December 2007, 1-360, p. 102). Any approach may be used, provided that it is appropriate to the scope of the system, the constraints of the project, includes the activities of the process listed below in some way, and is appropriately coordinated with other activities.

Generic inputs are baseline references of the submitted element. If the element is a system, inputs are the logical and physical architecture elements as described in a system design document, the design description of internal interfaces to the system and interfaces requirements external to the system, and by extension, the system requirements. **Generic outputs** define the verification plan that includes verification strategy, selected verification actions, verification procedures, verification tools, the verified element or system, verification reports, issue/trouble reports, and change requests on design.

Activities of the Process

To establish the verification strategy drafted in a verification plan (this activity is carried out concurrently to system definition activities), the following steps are necessary:

- Identify verification scope by listing as many characteristics or properties as possible that should be checked. The number of verification actions can be extremely high.
- Identify constraints according to their origin (technical feasibility, management constraints as cost, time, availability of verification means or qualified personnel, and contractual constraints that are critical to the mission) that limit potential verification actions.
- Define appropriate verification techniques to be applied, such as inspection, analysis, simulation, peer-review, testing, etc., based on the best step of the project to perform every verification action according to the given constraints.
- Consider a trade-off of what should be verified (scope) taking into account all constraints or limits and deduce what can be verified; the selection of verification actions would be made according to the type of system, objectives of the project, acceptable risks, and constraints.
- Optimize the verification strategy by defining the most appropriate verification technique for every verification action while defining necessary verification means (tools, test-benches, personnel, location, and facilities) according to the selected verification technique.
- Schedule the execution of verification actions in the project steps or milestones and define the configuration of elements submitted to verification actions (this mainly involves testing on physical elements).

Performing verification actions includes the following tasks:

- Detail each verification action; in particular, note the expected results, the verification techniques to be applied, and the corresponding means required (equipment, resources, and qualified personnel).
 - Acquire verification means used during system definition steps (qualified personnel, modeling tools, mocks-up, simulators, and facilities), and then those used during the integration step (qualified personnel, verification tools, measuring equipment, facilities, verification procedures, etc.).
 - Carry out verification procedures at the right time, in the expected environment, with the expected means, tools, and techniques.
-

- Capture and record the results obtained when performing verification actions using verification procedures and means.

The obtained results must be analyzed and compared to the expected results so that the status may be recorded as either *compliant* or *non-compliant*. Systems engineering (SE) practitioners will likely need to generate verification reports, as well as potential issue/trouble reports, and change requests on design as necessary.

Controlling the process includes the following tasks:

- Update the verification plan according to the progress of the project; in particular, planned verification actions can be redefined because of unexpected events.
- Coordinate verification activities with the project manager: review the schedule and the acquisition of means, personnel, and resources. Coordinate with designers for issues/trouble/non-conformance reports and with the configuration manager for versions of the physical elements, design baselines, etc.

Artifacts and Ontology Elements

This process may create several artifacts such as:

- verification plans (contain the verification strategy)
- verification matrices (contain the verification action, submitted element, applied technique, step of execution, system block concerned, expected result, obtained result, etc.)
- verification procedures (describe verification actions to be performed, verification tools needed, the verification configuration, resources and personnel needed, the schedule, etc.)
- verification reports
- verification tools
- verified elements
- issue / non-conformance / trouble reports
- change requests to the design

This process utilizes the ontology elements displayed in Table 2 below.

Table 2. Main Ontology Elements as Handled within Verification. (SEBoK Original)

Element	Definition
	Attributes (examples)
Verification Action	A verification action describes what must be verified (the element as reference) on which element, the expected result, the verification technique to apply, on which level of decomposition.
	Identifier, name, description
Verification Procedure	A verification procedure groups a set of verification actions performed together (as a scenario of tests) in a given verification configuration.
	Identifier, name, description, duration, unit of time
Verification Tool	A verification tool is a device or physical tool used to perform verification procedures (test bench, simulator, cap/stub, launcher, etc.).
	Identifier, name, description
Verification Configuration	A verification configuration groups all physical elements (aggregates and verification tools) necessary to perform a verification procedure.
	Identifier, name, description

Risk	An event having a probability of occurrence and a gravity degree on its consequence onto the system mission or on other characteristics (used for technical risk in engineering). A risk is the combination of vulnerability and of a danger or a threat.
Rationale	An argument that provides the justification for the selection of an engineering element.
	Identifier, name, description (rationale, reasons for defining a verification action, a verification procedure, for using a verification tool, etc.)

Methods and Techniques

There are several verification techniques to check that an element or a system conforms to its design references or its specified requirements. These techniques are almost the same as those used for validation, though the application of the techniques may differ slightly. In particular, the purposes are different; verification is used to detect faults/defects, whereas validation is used to provide evidence for the satisfaction of (system and/or stakeholder) requirements. Table 3 below provides descriptions of some techniques for verification.

Table 3. Verification Techniques. (SEBoK Original)

Verification Technique	Description
Inspection	Technique based on visual or dimensional examination of an element; the verification relies on the human senses or uses simple methods of measurement and handling. Inspection is generally non-destructive, and typically includes the use of sight, hearing, smell, touch, and taste, simple physical manipulation, mechanical and electrical gauging, and measurement. No stimuli (tests) are necessary. The technique is used to check properties or characteristics best determined by observation (e.g. paint color, weight, documentation, listing of code, etc.).
Analysis	Technique based on analytical evidence obtained without any intervention on the submitted element using mathematical or probabilistic calculation, logical reasoning (including the theory of predicates), modeling and/or simulation under defined conditions to show theoretical compliance. Mainly used where testing to realistic conditions cannot be achieved or is not cost-effective.
Analogy or Similarity	Technique based on evidence of similar elements to the submitted element or on experience feedback. It is absolutely necessary to show by prediction that the context is invariant that the outcomes are transposable (models, investigations, experience feedback, etc.). Similarity can only be used if the submitted element is similar in design, manufacture, and use; equivalent or more stringent verification actions were used for the similar element, and the intended operational environment is identical to or less rigorous than the similar element.
Demonstration	Technique used to demonstrate correct operation of the submitted element against operational and observable characteristics without using physical measurements (no or minimal instrumentation or test equipment). Demonstration is sometimes called 'field testing'. It generally consists of a set of tests selected by the supplier to show that the element response to stimuli is suitable or to show that operators can perform their assigned tasks when using the element. Observations are made and compared with predetermined/expected responses. Demonstration may be appropriate when requirements or specification are given in statistical terms (e.g. mean time to repair, average power consumption, etc.).
Test	Technique performed onto the submitted element by which functional, measurable characteristics, operability, supportability, or performance capability is quantitatively verified when subjected to controlled conditions that are real or simulated. Testing often uses special test equipment or instrumentation to obtain accurate quantitative data to be analyzed.
Sampling	Technique based on verification of characteristics using samples. The number, tolerance, and other characteristics must be specified to be in agreement with the experience feedback.

Practical Considerations

Key pitfalls and good practices related to this topic are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing System Verification are provided in Table 4.

Table 4. Major Pitfalls with System Verification (SEBoK Original)

Pitfall	Description
Confusion between verification and validation	Confusion between verification and validation causes developers to take the wrong reference/baseline to define verification and validation actions and/or to address the wrong level of granularity (detail level for verification, global level for validation).
No verification strategy	One overlooks verification actions because it is impossible to check every characteristic or property of all system elements and of the system in any combination of operational conditions and scenarios. A strategy (justified selection of verification actions against risks) must be established.
Save or spend time	Skip verification activity to save time.
Use only testing	Use only testing as a verification technique. Testing requires checking products and services only when they are implemented. Consider other techniques earlier during design; analysis and inspections are cost effective and allow discovering early potential errors, faults, or failures.
Stop verifications when funding is diminished	Stopping the performance of verification actions when budget and/or time are consumed. Prefer using criteria such as coverage rates to end verification activity.

Proven Practices

Some proven practices gathered from the references are provided in Table 5.

Table 5. Proven Practices with System Verification. (SEBoK Original)

Practice	Description
Start verifications early in the development	The earlier characteristics of an element are verified in the project, the easier the corrections are to do and the consequences on schedule and cost will be fewer.
Define criteria ending verifications	Carrying out verification actions without limits generates a risk of drift for costs and deadlines. Modifying and verifying in a non-stop cycle until arriving at a perfect system is the best way to never supply the system. Thus, it is necessary to set limits of cost, time, and a maximum number of modification loops back for each verification action type, ending criteria (percentages of success, error count detected, coverage rate obtained, etc.).
Involve design responsible with verification	Include the verification responsible in the designer team or include some designer onto the verification team.

References

Works Cited

- Buede, D.M. 2009. *The Engineering Design of Systems: Models and Methods*. 2nd ed. Hoboken, NJ, USA: John Wiley & Sons Inc.
- INCOSE. 2012. *INCOSE Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- Lake, J. 1999. "V & V in Plain English." International Council on Systems Engineering (INCOSE) 9th Annual International Symposium, Brighton, UK, 6-10 June 1999.
- NASA. 2007. *Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), December 2007. NASA/SP-2007-6105.

Primary References

- INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), December 2007. NASA/SP-2007-6105.

Additional References

- Buede, D.M. 2009. *The Engineering Design of Systems: Models and Methods*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons Inc.
- DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.
- ECSS. 2009. *Systems Engineering General Requirements*. Noordwijk, Netherlands: Requirements and Standards Division, European Cooperation for Space Standardization (ECSS), 6 March 2009. ECSS-E-ST-10C.
- MITRE. 2011. "Verification and Validation." in *Systems Engineering Guide*. Accessed 11 March 2012 at [[1]].
- SAE International. 1996. *Certification Considerations for Highly-Integrated or Complex Aircraft Systems*. Warrendale, PA, USA: SAE International, ARP475.
- SEI. 2007. "Measurement and Analysis Process Area" in *Capability Maturity Model Integrated (CMMI) for Development, version 1.2*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
-

Relevant Videos

- Systems Engineering-Test, Evaluation, and Validation ^[2]

References

- [1] https://mitre.org/work/systems_engineering/guide/se_lifecycle_building_blocks/test_evaluation/verification_validation.html
[2] https://www.youtube.com/watch?v=pcrkmaAx_QA

System Transition

- Lead Authors:
- Scott Jackson and Brian Gallagher

As part of system deployment, on-site installation, check-out, integration, and testing must be carried out to ensure that the system is fit to be deployed into the field and/or put into an operational context. *Transfer* is the process that bridges the gap between qualification and use; it deals explicitly with the handoff from development to logistics, operations, maintenance, and support.

Definition & Purpose

There are many different approaches to transition or deployment, and many different views on what is included within transition. The SEBoK uses the ISO/IEC/IEEE 15288 definition of transition, as seen below (ISO/IEC/IEEE 15288 2015):

[The transition] process installs a verified system, together with relevant enabling systems, e.g., operating system, support system, operator training system, user training system, as defined in agreements. This process is used at each level in the system structure and in each stage to complete the criteria established for exiting the stage.

Thinking in a linear fashion, the system is transitioned into operation and then would be used and maintained in the operational environment. However, there are other views on transition. For example, the NASA *Systems Engineering Handbook* states that transition can include delivery for end-use as well as delivery of components for integration (NASA 2007). Using this view, transition is the mechanism for moving system components from implementation activities into integration activities. The NASA discussion of transition also implies that transition can include sustainment activities:

The act of delivery or moving of a product from the location where the product has been implemented or integrated, as well as verified and validated, to a customer.

Many systems are deployed using an iterative or evolutionary approach where operationally useful capabilities are developed and deployed incrementally. While these operationally useful capabilities are fully deployed and transitioned into operational use, transition of logistics, maintenance, and support may occur incrementally or be delayed until after the full system capability is delivered.

Process Approaches

Just as there are multiple views on the definition of transition and deployment, there are also several ways to divide the activities required for transition. For example, the NASA *Systems Engineering Handbook* definition of transition states: *This act can include packaging, handling, storing, moving, transporting, installing, and sustainment activities* (2007). However, the SEBoK includes the topic of *sustainment* as separate from transition; this is instead covered under the maintenance and logistics topics. The International Council on Systems Engineering (INCOSE) views the transition process as two-step: planning and performance. Though there are several processes for deployment and

transition, most generally include the following activities:

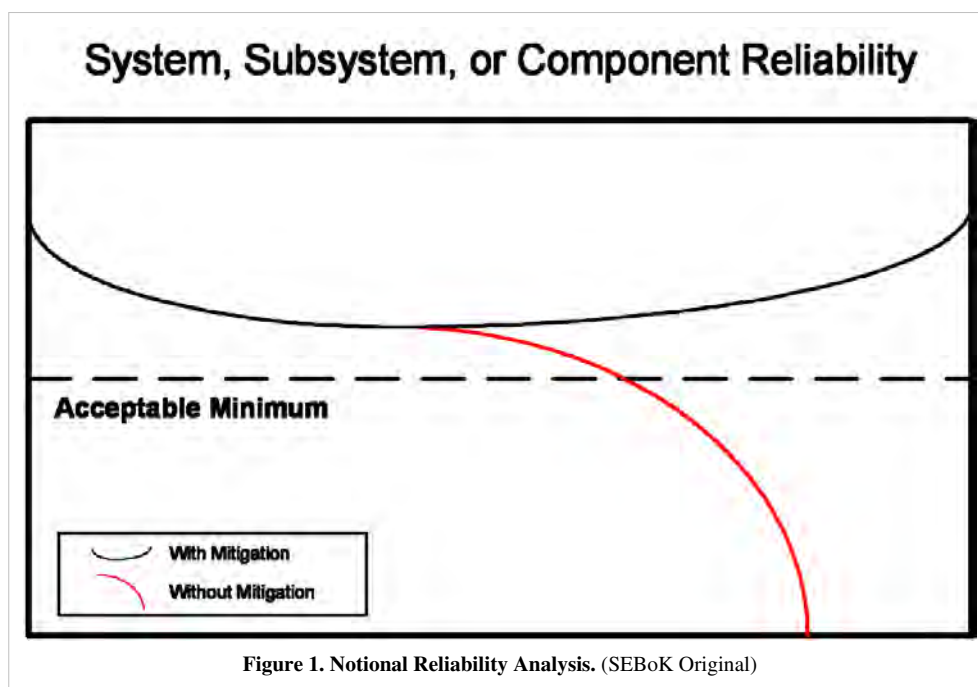
- **Develop a Deployment/Transition Strategy** - Planning for transition activities would ideally begin early in the SE life cycle, though it is possible to conduct these activities concurrently with realization activities. Planning should generally include some consideration of the common lower-level activities of installation, checkout, integration, and testing. Such activities are crucial to demonstrate that the system and the interfaces with the operational environment can function as intended and meet the contractual system specifications. For these activities to be effectively managed and efficiently implemented, the criteria, responsibility, and procedures for carrying out these activities should be clearly established and agreed upon during the planning phase.
- **Develop Plans for Transitioning Systems** - or system capabilities into operational use and support. Transition plans for the system or incremental system capabilities should be consistent with the overall transition strategy and agreed to by relevant stakeholders. Planning for transition will often include establishing a strategy for support, which may include organic support infrastructures, contractor logistics support, or other sources (Bernard et al. 2005, 1-49). It can also include defining the levels of support to be established. The strategy is important because it drives most of the other transition planning activities, as well as product design considerations. Transition plans should include considerations for coordination with the following activities:
 - **Installation** - Installation generally refers to the activities required to physically instate the system; this will likely include connecting interfaces to other systems such as electrical, computer, or security systems, and may include software interfaces as well. Installation planning should generally document the complexity of the system, the range of environmental conditions expected in the operational environment, any interface specifications, and human factors requirements such as safety. When real-world conditions require changes in the installation requirements, these should be documented and discussed with the relevant stakeholders.
 - **Integration** - Though system integration activities will generally be performed prior to installation, there may be additional steps for integrating the system into its operational setting. Additionally, if the system is being delivered incrementally, there will likely be integration steps associated with the transition (for more information on integration, please see the System Realization knowledge area (KA)).
 - **Verification and Validation (V&V)** - At this stage, V&V for physical, electrical, and mechanical checks may be performed in order to verify that the system has been appropriately installed. Acceptance tests conducted after delivery may become part of this process (for additional information on V&V, please see the System Realization KA). There are several types of acceptance tests which may be used:
 - **On-site Acceptance Test (OSAT)** - This test includes any field acceptance testing and is performed only after the system has successfully been situated in the operational environment. It may consist of functional tests to demonstrate that the system is functioning and performing properly.
 - *Field Acceptance Test* - This test includes flight and sea acceptance tests; it is performed, if applicable, only after the system has successfully passed the OSAT. The purpose of field testing is to demonstrate that the system meets the performance specifications called for in the system specifications in the actual operating environment.
 - *Operational Test and Evaluation (OT&E)* - An OT&E consists of a test series designed to estimate the operational effectiveness of the system.
 - *Evaluate the readiness of the system to transition into operations* - This is based upon the transition criteria identified in the transition plan. These criteria should support an objective evaluation of the system's readiness for transition. The integration, verification, and validation activities associated with transition may be used to gauge whether the system meets transition criteria.
 - *Analyze the results of transition activities throughout and any necessary actions* - As a result of analysis, additional transition activities and actions may be required. The analysis may also identify areas for improvement in future transition activities.

Some common issues that require additional considerations and SE activities are the utilization or replacement of legacy systems. It is also common for an organization to continue testing into the early operational phase. The following activities support these circumstances:

- **System Run-In** - After the successful completion of the various acceptance tests, the system(s) will be handed over to the user or designated post-deployment support organization. The tested system(s) may have to be verified for a stated period (called the system run-in, normally for one to two years) for the adequacy of reliability and maintainability (R&M) and integrated logistics support (ILS) deliverables. R&M are vital system operational characteristics having a dominant impact upon the operational effectiveness, the economy of in-service maintenance support, and the life cycle cost (LCC).
- **Phasing-In/Phasing-Out** - The need for phasing-in will usually be identified during the system definition, when it is clear that the new system entails the replacement of an existing system(s) (for additional information, please see the System Definition KA). These activities should help to minimize disruption to operations and, at the same time, minimize the adverse effect on operational readiness. It is also important that the phasing-in of a new system and the phasing-out of an existing system occur in parallel with the systems activities of the system run-in to maximize resource utilization. Other aspects of phasing-in/phasing-out to be considered include:
 - Proper planning for the phasing out of an existing system (if necessary).
 - For multi-user or complex systems, phase-by-phase introduction of the system according to levels of command, formation hierarchy, etc.
 - Minimum disruption to the current operations of the users.
 - Establishment of a feedback system from users on problems encountered in operation, etc.
 - Disposal process including handling of hazardous items, cost of disposal, approval etc.

Applicable Methods & Tools

A system may have to undergo reliability demonstration testing (RDT) to ensure that it meets its contractual R&M guarantees. RDT is conducted under actual field conditions, especially for large systems purchased in small quantity. During RDT, the system is operated in the field within stated test duration and all field data are systematically recorded. At the end of the test period, analysis of the RDT data is performed. Data analysis should facilitate determination of system reliability. One possible output of this analysis is shown in Figure 1 below.



References

Works Cited

Bernard, S., B. Gallagher, R. Bate, H. Wilson. 2005. *CMMI® Acquisition Module (CMMI-AM)*, version 1.1. Pittsburg, PA, USA: Carnegie Mellon University (CMU)/Software Engineering Institute (SEI) CMU/SEI-2005-TR-011.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.

NASA. 2007. *Systems Engineering Handbook*. Washington, D.C., USA8: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Primary References

INCOSE. 2011. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.1. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.1.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.

NASA. 2007. *Systems Engineering Handbook*. Washington, D.C., USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105.

Additional References

None.

System Validation

- Lead Author:
- Alan Faisandier
- Contributing Author:
- Rick Adcock

System Validation is a set of actions used to check the compliance of any element (a system element, a system, a document, a service, a task, a system requirement, etc.) with its purpose and functions. These actions are planned and carried out throughout the life cycle of the system. Validation is a generic term that needs to be instantiated within the context it occurs. When understood as a process, validation is a transverse activity to every life cycle stage of the system. Particularly during the development cycle of the system, the validation process is performed in parallel with the system definition and system realization processes and applies to any activity and product resulting from this activity. The validation process is not limited to a phase at the end of system development, but generally occurs at the end of a set of life cycle tasks or activities, and always at the end of each milestone of a development project. It may be performed on an iterative basis on every produced engineering element during development and may begin with the validation of the expressed stakeholder needs and requirements. When the validation process is applied to the system when completely integrated, it is often called *final validation*. It is important to remember that while system validation is separate from verification, the activities are complementary and intended to be performed in conjunction.

Definition and Purpose

Validation is the confirmation, through the provision of objective evidence, that the requirements for a specific intended use or application have been fulfilled. With a note added in ISO 9000:2005: *validation is the set of activities that ensure and provide confidence that a system is able to accomplish its intended use, goals, and objectives (i.e., meet stakeholder requirements) in the intended operational environment* (ISO 2005).

The purpose of validation, as a generic action, is to establish the compliance of any activity output as compared to inputs of the activity. It is used to provide information and evidence that the transformation of inputs produced the expected and *right* result. Validation is based on tangible evidence; i.e., it is based on information whose veracity can be demonstrated by factual results obtained from techniques or methods such as inspection, measurement, test, analysis, calculation, etc. Thus, to validate a system (product, service, or enterprise) consists of demonstrating that it satisfies its system requirements and eventually the stakeholder's requirements depending on contractual practices. From a global standpoint, the purpose of validating a system is to acquire confidence in the system's ability to achieve its intended mission, or use, under specific operational conditions.

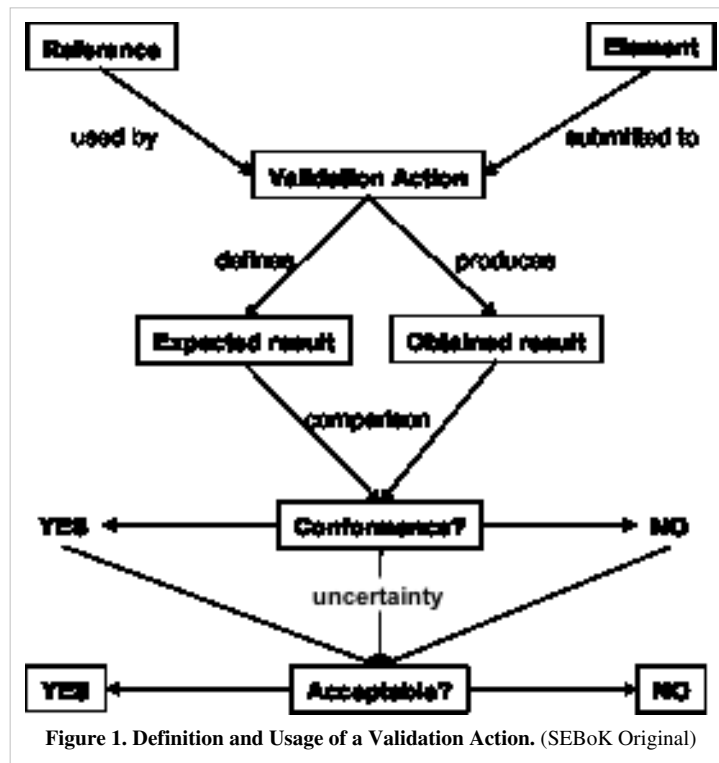
Principles

Concept of Validation Action

Why Validate?

The primary goal of systems engineering (SE) is to develop a solution that meets the needs and requirements of stakeholders. Validation is the process by which engineers ensure that the system will meet these needs and requirements.

A **validation action** is defined and then performed (see Figure 1, below).



A validation action applied to an engineering element includes the following:

- Identification of the element on which the validation action will be performed.
- Identification of the reference that defines the expected result of the validation action.

Performing the validation action includes the following:

- Obtaining a result by performing the validation action onto the submitted element.
- Comparing the obtained result with the expected result.
- Deducing the degree of compliance of the element.
- Deciding on the acceptability of this compliance, because sometimes the result of the comparison may require a value judgment to decide whether to accept the obtained result as compared to the relevance of the context of use.

Note: If there is uncertainty about compliance, the cause could come from ambiguity in the requirements.

What to Validate?

Any engineering element can be validated using a specific reference for comparison, such as stakeholder requirements, system requirements, functions, system elements, documents, etc. Examples are provided in Table 1 below:

Table 1. Examples of Validated Items (SEBoK Original)

Items	Explanation for Validation
Document	To validate a document is to make sure its content is compliant with the inputs of the task that produced the document.
Stakeholder Requirement and System Requirement	To validate a stakeholder requirement is to make sure its content is justified and relevant to stakeholders' expectations, complete and expressed in the language of the customer or end user. To validate a system requirement is to make sure its content translates correctly and/or accurately a stakeholder requirement to the language of the supplier.
Design	To validate the design of a system (logical and physical architectures) is to demonstrate that it satisfies its system requirements.
System	To validate a system (product, service, or enterprise) is to demonstrate that the product, service, or enterprise satisfies its system requirements and/or its stakeholder requirements.
Activity	To validate an activity or a task is to make sure its outputs are compliant with its inputs.
Process	To validate a process is to make sure its outcomes are compliant with its purpose.

Validation versus Verification

The Verification versus Validation section of the System Verification article gives fundamental differences between the two concepts and associated processes. The Table 2 provides information to help understand these differences.

Table 2. Verification and Validation Differences (may vary with context). (SEBoK Original)

Point of View	Verification	Validation
Purpose of the Activity	Detect, identify faults/defects (supplier oriented)	Acquire confidence (end user oriented)
Idea behind the Term	Based on truth (objective/unbiased)	Based on value judgement (more subjective)
Level of Concern	Detail and local	Global in the context of use
Vision	Glass box (how it runs inside)	Black box (application of inputs provides the expected effect)
Basic Method	Fine-tooth comb	Traceability matrix
System (Product, Service, Enterprise)	"Done Right" (respects the state of the art); focus on (physical) characteristics	"Does Right" (produces the expected effect); focus on services, functions
Baseline Reference for Comparison (Product, Service, Enterprise)	System design	System requirements (and stakeholder requirements)
Order of Performance	First	Second
Organization of Activity	Verification actions are defined and/or performed by development/designer team	Validation actions are defined and/or performed by experts and external members to development/designer team

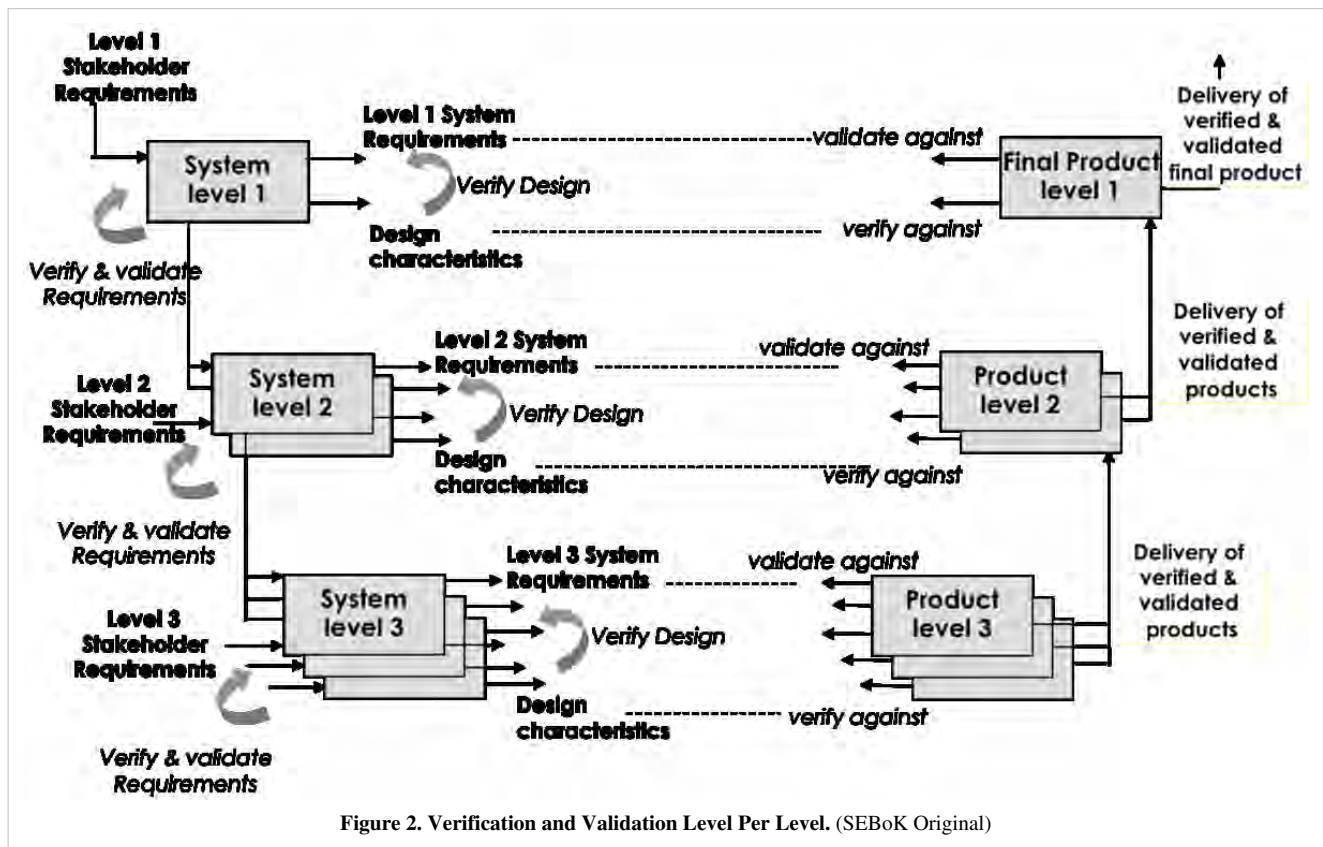
Validation, Final Validation, and Operational Validation

System validation concerns the global system seen as a whole and is based on the totality of requirements (system requirements, stakeholders' requirements, etc.), but it is obtained gradually throughout the development stage in three non-exclusive ways:

- accumulating the results of verification actions and validation actions provided by the application of corresponding processes to every engineering element;
- performing final validation actions to the complete, integrated system in an industrial environment (as close as possible to the operational environment); and
- performing operational validation actions on the complete system in its operational environment (context of use).

Verification and Validation Level per Level

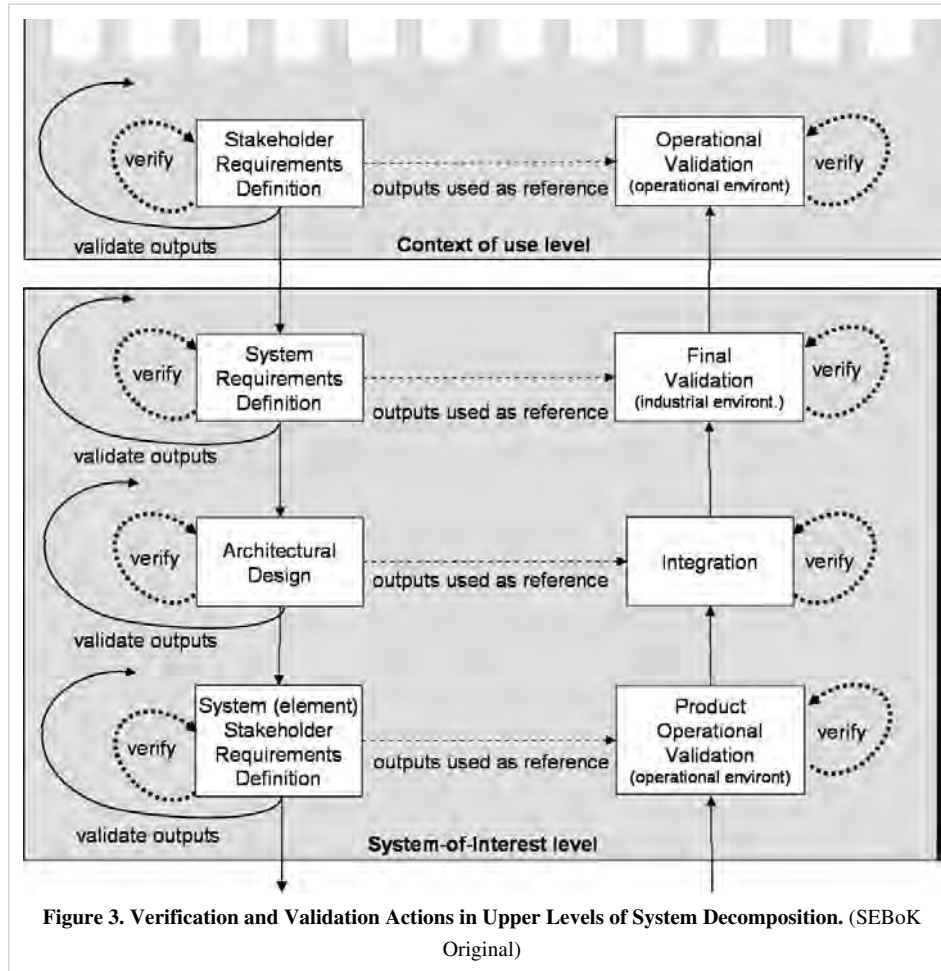
It is impossible to carry out only a single global validation on a complete, integrated complex system. The sources of faults/defects could be important, and it would be impossible to determine the causes of non-conformance manifested during this global check. Generally, the system-of-interest (SoI) has been decomposed during design in a set of layers of systems. Thus, every system and system element is verified, validated, and possibly corrected before being integrated into the parent system of the higher level, as shown in Figure 2.



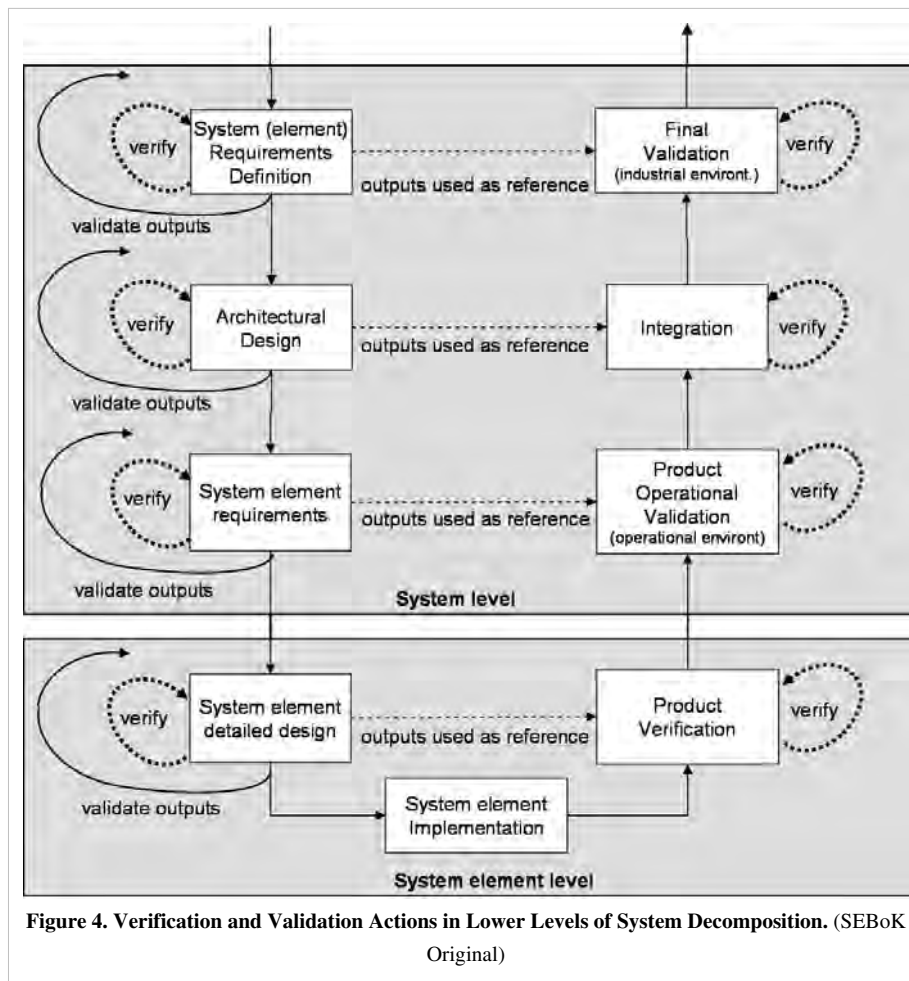
As necessary, systems and system elements are partially integrated in subsets in order to limit the number of properties to be verified within a single step. For each level, it is necessary to perform a set of final validation actions to ensure that features stated at preceding levels are not damaged. Moreover, a compliant result obtained in a given environment can turn into a non-compliant result if the environment changes. Thus, as long as the system is not completely integrated and/or doesn't operate in the real operational environment, no result should be regarded as definitive.

Verification Actions and Validation Actions Inside and Transverse to Levels

Inside each level of system decomposition, verification actions and validation actions are performed during system definition and system realization. This is represented in Figure 3 for the upper levels, and in Figure 4 for the lower levels. Stakeholder requirements definition and operational validation make the link between the two levels of the system decomposition.



Operational validation of system element requirements and products makes the link between the two lower levels of the decomposition. See Figure 4 below.



Note: The last level of system decomposition is dedicated to the realization of system elements and the vocabulary and number of activities may be different from what is seen in Figure 4.

Verification and Validation Strategy

The difference between verification and validation is especially useful for elaborating on the integration strategy, the verification strategy, and the validation strategy. In fact, the efficiency of system realization is gained by optimizing the three strategies together to form what is often called the verification and validation strategy. This optimization consists of defining and performing the minimum number of verification and validation actions but detecting the maximum number of errors/faults/defects and achieving the maximum level of confidence in the system. The optimization takes into account the risks potentially generated if some verification actions or validation actions are excluded.

Process Approach

Purpose and Principles of the Approach

The purpose of the validation process is to provide objective evidence that the services provided by a system in use comply with stakeholder requirements and achieve its intended use in its intended operational environment (ISO/IEC/IEEE 15288 2015). The validation process performs a comparative assessment and confirms that the stakeholder requirements are correctly defined. Where variance is identified, it is recorded to guide future corrective actions. System validation is ratified by stakeholders (ISO/IEC/IEEE 15288 2015).

The validation process demonstrates that the realized end product satisfies its stakeholders' (customers' or other interested parties') expectations within the intended operational environments with validation performed by anticipated operators and/or users (NASA 2007, 1-360). Each system element, system, and the complete SoI are compared against their own applicable requirements (system requirements and stakeholder requirements). This means that the validation process is instantiated as many times as necessary during the global development of the system.

In order to ensure that validation is feasible, the implementation of requirements must be verifiable onto a defined element. It is essential to ensure that requirements are properly written, i.e., quantifiable, measurable, unambiguous, etc. In addition, verification/validation requirements are often written in conjunction with stakeholder and system requirements and provide a method for demonstrating the implementation of each system requirement or stakeholder requirement.

Generic inputs are references of requirements applicable to the submitted element. If the element is a system, inputs are system requirements and stakeholder requirements.

Generic outputs are the validation plan that includes validation strategy, selected validation actions, validation procedures, validation tools, validated elements or systems, validation reports, issue/trouble reports, and change requests on requirements or on the system.

Activities of the Process

Major activities and tasks performed during this process include the following:

- Establish a validation strategy (often drafted in a validation plan). This activity is carried out concurrently to system definition activities:
 - Identify the validation scope that is represented by (system and/or stakeholder) requirements; normally, every requirement should be checked as the number of validation actions can be high.
 - Identify constraints according to their origin (technical feasibility, management constraints as cost, time, availability of validation means or qualified personnel, and contractual constraints that are critical to the mission) that limit or increase potential validation actions.
 - Define appropriate verification/validation techniques to be applied, such as inspection, analysis, simulation, review, testing, etc., depending on the best step of the project to perform every validation action according to constraints.
 - Consider a trade-off of what should be validated (scope) while taking into account all constraints or limits and deduce what can be validated objectively; selection of validation actions would be made according to the type of system, objectives of the project, acceptable risks, and constraints.
 - Optimize the validation strategy to define the most appropriate validation technique for every validation action, define necessary validation means (tools, test-benches, personnel, location, and facilities) according to the selected validation technique, schedule the execution of validation actions in the project steps or milestones, and define the configuration of elements submitted to validation actions (this is primarily about testing on physical elements).
- Perform validation actions, including the following tasks:
 - Detail each validation action. In particular, note the expected results, the validation technique to be applied, and the corresponding means necessary (equipment, resources, and qualified personnel).
 - Acquire validation means used during the system definition steps (qualified personnel, modeling tools, mocks-up, simulators, and facilities), then those means used during integration and final and operational steps (qualified personnel, validation tools, measuring equipment, facilities, validation procedures, etc.).
 - Carry out validation procedures at the right time, in the expected environment, with the expected means, tools, and techniques.

- Capture and record results obtained when performing validation actions using validation procedures and means.
- Analyze the obtained results and compare them to the expected results. Decide if they comply acceptably. Record whether the decision and status are compliant or not, and generate validation reports and potential issue/trouble reports, as well as change requests on (system or stakeholder) requirements as necessary.
- Control the process using following tasks:
 - Update the validation plan according to the progress of the project; in particular, planned validation actions can be redefined because of unexpected events.
 - Coordinate validation activities with the project manager regarding the schedule, acquisition of means, personnel, and resources. Coordinate with the designers for issue/trouble/non-conformance reports. Coordinate with the configuration manager for versions of physical elements, design baselines, etc.

Artifacts and Ontology Elements

This process may create several artifacts, such as:

- a validation plan (contains the validation strategy)
- a validation matrix (contains for each validation action, submitted element, applied technique, step of execution, system block concerned, expected result, obtained result, etc.)
- validation procedures (describe the validation actions to be performed, the validation tools needed, the validation configuration, resources, personnel, schedule, etc.)
- validation reports
- validation tools
- the validated element
- issue, non-conformance, and trouble reports
- change requests on requirements, products, services, and enterprises

This process utilizes the ontology elements of Table 3.

Table 3. Main Ontology Elements as Handled within Validation. (SEBoK Original)

Element	Definition
	Attributes (examples)
Validation Action	A validation action describes what must be validated (the element as reference), on which element, the expected result, the verification technique to apply, on which level of decomposition. Identifier, name, description
Validation Procedure	A validation procedure groups a set of validation actions performed together (as a scenario of tests) in a given validation configuration. Identifier, name, description, duration, unit of time
Validation Tool	A validation tool is a device or physical tool used to perform validation procedures (test bench, simulator, cap/stub, launcher, etc.). Identifier, name, description
Validation Configuration	A validation configuration groups the physical elements necessary to perform a validation procedure. Identifier, name, description

Risk	An event having a probability of occurrence and a gravity degree on its consequence onto the system mission or on other characteristics (used for technical risk engineering).
	Identifier, name, description, status
Rationale	An argument that provides the justification for the selection of an engineering element.
	Identifier, name, description (rationale, reasons for defining a validation action, a validation procedure, for using a validation tool, etc.)

Methods and Techniques

The validation techniques are the same as those used for verification, but their purposes are different; verification is used to detect faults/defects, whereas validation is used to prove the satisfaction of (system and/or stakeholder) requirements.

The **validation traceability matrix** is introduced in the stakeholder requirements definition topic. It may also be extended and used to record data, such as a validation actions list, selected validation techniques to validate implementation of every engineering element (stakeholder and system requirements in particular), expected results, and obtained results when validation actions have been performed. The use of such a matrix enables the development team to ensure that selected stakeholder and system requirements have been checked, or to evaluate the percentage of validation actions completed.

Practical Considerations

Key pitfalls and good practices related to system validation are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in planning and performing system validation are provided in Table 4.

Table 4. Major Pitfalls with System Validation. (SEBoK Original)

Pitfall	Description
Start validation at the end of the project	A common mistake is to wait until the system has been entirely integrated and tested (design is qualified) to perform any sort of validation. Validation should occur as early as possible in the [product] life cycle (Martin 1997).
Use only testing	Use only testing as a validation technique. Testing requires checking products and services only when they are implemented. Consider other techniques earlier during design; analysis and inspections are cost effective and allow discovering early potential errors, faults, or failures.
Stop validation when funding is diminished	Stop the performance of validation actions when budget and/or time are consumed. Prefer using criteria such as coverage rates to end validation activity.

Proven Practices

Some good practices gathered from the references are provided in Table 5.

Table 5. Proven Practices with System Validation. (SEBoK Original)

Practice	Description
Start Validation Plan Early	It is recommended to start the drafting of the validation plan as soon as the first requirements applicable to the system are known. If the writer of the requirements immediately puts the question to know how to validate whether the future system will answer the requirements, it is possible to: • detect the unverifiable requirements • anticipate, estimate cost, and start the design of validation means (as needed) such as test-benches, simulators • avoid cost overruns and schedule slippages
Verifiable Requirements	According to Buede, a requirement is verifiable if a "finite, cost-effective process has been defined to check that the requirement has been attained." (Buede 2009) Generally, this means that each requirement should be quantitative, measurable, unambiguous, understandable, and testable. It is generally much easier and more cost-effective to ensure that requirements meet these criteria while they are being written. Requirement adjustments made after implementation and/or integration are generally much more costly and may have wide-reaching redesign implications. There are several resources which provide guidance on creating appropriate requirements - see the system definition knowledge area, stakeholder requirements, and system requirements topics for additional information.
Document Validation Actions	It is important to document both the validation actions performed and the results obtained. This provides accountability regarding the extent to which system, system elements, and subsystems fulfill system requirements and stakeholders' requirements. These data can be used to investigate why the system, system elements, or subsystems do not match the requirements and to detect potential faults/defects. When requirements are met, these data may be reported to organization parties. For example, in a safety critical system, it may be necessary to report the results of safety demonstration to a certification organization. Validation results may be reported to the acquirer for contractual aspects or to internal company for business purpose.
Involve Users with Validation	Validation will often involve going back directly to the users to have them perform some sort of acceptance test under their own local conditions.
Involve	Often the end users and other relevant stakeholders are involved in the validation process.

The following are elements that should be considered when practicing any of the activities discussed as a part of system realization:

- Confusing verification and validation is a common issue. Validation demonstrates that the product, service, and/or enterprise as provided, fulfills its intended use, whereas verification addresses whether a local work product properly reflects its specified requirements. Validation actions use the same techniques as the verification actions (e.g., test, analysis, inspection, demonstration, or simulation).
- State who the witnesses will be (for the purpose of collecting the evidence of success), what general steps will be followed, and what special resources are needed, such as instrumentation, special test equipment or facilities, simulators, specific data gathering, or rigorous analysis of demonstration results.
- Identify the test facility, test equipment, any unique resource needs and environmental conditions, required qualifications and test personnel, general steps that will be followed, specific data to be collected, criteria for repeatability of collected data, and methods for analyzing the results.

References

Works Cited

Buede, D. M. 2009. *The engineering design of systems: Models and methods*. 2nd ed. Hoboken, NJ: John Wiley & Sons Inc.

INCOSE. 2012. *INCOSE Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105, December 2007.

Primary References

INCOSE. 2012. *Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2015. *Systems and software engineering - system life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC 15288:2015.

NASA. 2007. *Systems Engineering Handbook*. Washington, D.C.: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105, December 2007.

Additional References

Buede, D.M. 2009. *The engineering design of systems: Models and methods*. 2nd ed. Hoboken, NJ: John Wiley & Sons Inc.

DAU. February 19, 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense.

ECSS. 2009. Systems engineering general requirements. Noordwijk, Netherlands: Requirements and Standards Division, European Cooperation for Space Standardization (ECSS), ECSS-E-ST-10C. 6 March 2009.

MITRE. 2011. "Verification and Validation." *Systems Engineering Guide*. Accessed 11 March 2012 at [[1]].

SAE International. 1996. *Certification considerations for highly-integrated or complex aircraft systems*. Warrendale, PA, USA: SAE International, ARP475.

SEI. 2007. *Capability maturity model integrated (CMMI) for development*, version 1.2, measurement and analysis process area. Pittsburg, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

System Operation

- Lead Authors:
- Scott Jackson and Brian Gallagher
- Contributing Author:
- Leopoldo deCardenas

The role of systems engineering (SE) during the operation of a system consists of ensuring that the system maintains key mission and business functions and is operationally effective. The systems engineer is one of the stakeholders who ensures that maintenance actions and other major changes are performed according to the long-term vision of the system. Both the maintenance actions and any implemented changes must meet the evolving needs of owning and operating stakeholders consistent with the documented and approved architecture. SE considerations will also include the eventual decommissioning or disposal of the system so that the disposal occurs according to disposal/retirement plans. Those plans must account for and be compliant with relevant laws and regulations (for additional information on disposal or retirement, please see the Product and Service Life Management knowledge area (KA)). When the system-of-interest (SoI) replaces an existing or legacy system, it may be necessary to manage the migration between systems such that stakeholders do not experience a breakdown in services (INCOSE 2012).

Definition & Purpose

This process assigns personnel to operate the system and monitors the services and operator-system performance. In order to sustain services, it identifies and analyzes operational problems in relation to agreements, stakeholder requirements, and organizational constraints (ISO/IEC/IEEE 2015).

The concept of operations (ConOps) establishes the foundation for initial design specifications according to the long-term vision. It is also possible that pre-planned program improvements (P3I) had been generated based on expected evolving requirements. Throughout the systems life cycle, the operation of the system requires the systems engineer to be an active participant in reviews, change management and integrated master schedule activities to ensure the system operations continue to meet the evolving needs of stakeholders, and are consistent with the architecture through the eventual decommissioning or disposal of the system. In the event of decommissioning, a systems engineer must ensure disposal/retirement plans are compliant with relevant laws and regulations (for additional information on disposal or retirement, see the Product and Service Life Management KA).

Two additional areas are of interest to the systems engineer during system operation require special attention. First, it may be determined that a system is at the end of its life cycle, but the cost of replacing the system with a completely new design is too expensive. In this case, there will be intense engineering activities for service life extension program (SLEP). The SLEP solution will take into account obsolescence issues, diminishing manufacturing sources and material shortages (DMSMS), and changes in ConOps. Secondly, in the event that a new SoI is designed and produced as a complete replacement for an existing or legacy system, it will be necessary to manage the migration between systems such that stakeholders do not experience a breakdown in services (INCOSE 2012).

Process Approaches

During the operational phase, SE activities ensure the system maintains certain operational attributes and usefulness throughout its expected life span. Maintaining operational effectiveness consists of evaluating certain operationally relevant attributes and trends, taking actions to prevent degradation of performance, evolving the system to meet changing mission or business needs (see the Product and Service Life Management KA), and eventually decommissioning the system and disposing of its components. During operation, data would be collected to evaluate the system and determine if changes should be made. It is important to include the process for data collection during operations when considering design and ConOps. In some cases, data may be collected by sensors and reported

autonomously. In other cases, operators will identify and report on performance during operations. The systems engineer needs to understand how all data will be collected and presented for further analysis. The systems engineer will be involved in analysis of this data in several areas, including the following:

- Updating training and development of new training as required for operational and support personnel. Training is generally developed early with system design and production and executed during integration and operations. Determination of training updates or changes will be based on evaluation of the operational and support personnel.
- Evaluation of operational effectiveness. Early in the planning phases of a new system or capability, measures of operational effectiveness are established based on mission and business goals. These measures are important during system operation. These attributes are unique for each system and represent characteristics describing the usefulness of the system as defined and agreed to by system stakeholders. Systems engineers monitor and analyze these measurements and recommend actions.
- Failure reporting and corrective actions (FRACA) activities will involve the collection and analysis of data during operations. FRACA data will provide trends involving failures that may require design or component changes. Some failures may also result in safety issues requiring operational modifications until the offending elements under analysis can be corrected. If components or systems must be returned to maintenance facilities for corrective repairs, there will be operational and business impacts due to increased unavailability and unplanned transportation cost.

Applicable Methods & Tools

Operations manuals generally provide operators the steps and activities required to run the system.

Training and Certification

Adequate training must be provided for the operators who are required to operate the system. There are many objectives of training:

- Provide initial training for all operators in order to equip them with the skill and knowledge to operate the system. Ideally, this process will begin prior to system transition and will facilitate delivery of the system. It is important to define the certification standards and required training materials up front (for more information on material supply, please see Logistics).
- Provide continuation training to ensure currency of knowledge.
- Monitor the qualification/certification of the operators to ensure that all personnel operating the system meet the minimum skill requirements and that their currency remains valid.
- Monitor and evaluate the job performance to determine the adequacy of the training program.

Practical Considerations

The operation process sustains system services by assigning trained personnel to operate the system, as well as by monitoring operator-system performance and monitoring the system performance. In order to sustain services, the operation process identifies and analyzes operational problems in relation to agreements, stakeholder needs and requirements, and organizational constraints. When the system replaces an existing system, it may be necessary to manage the migration between systems such that persistent stakeholders do not experience a breakdown in services.

Results of a successful implementation of the operation process include:

- Definition and refinement of an operation strategy along the way
 - Delivery of services that meet stakeholder requirements
 - Satisfactory completion of approved, corrective action requests
 - Continued stakeholder satisfaction
-

Outputs of the operation process include:

- Operational strategy, including staffing and sustainment of enabling systems and materials
- System performance reports (statistics, usage data, and operational cost data)
- System trouble/anomaly reports with recommendations for appropriate action
- Operational availability constraints to influence future design and specification of similar systems or reused system elements

Activities of the operation process include:

- Providing operator training to sustain a pool of operators
- Tracking system performance and accounting for operational availability
- Performing operational analysis
- Managing operational support logistics
- Documenting system status and actions taken
- Reporting malfunctions and recommendations for improvement

References

Works Cited

INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation (ISO)/International Electrotechnical Commissions (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.

Primary References

Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th Edition. Englewood Cliffs, NJ, USA: Prentice Hall.

Institute of Engineers Singapore. 2009. *Systems Engineering Body of Knowledge*. Provisional version 2.0. Singapore: Institute of Engineers Singapore.

INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation (ISO)/International Electrotechnical Commissions (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.

Additional References

None.

Knowledge Area: System Maintenance

System Maintenance

Contents of this Knowledge Area

- Logistics (Scott Jackson and John Snoderly) (Garry Roedler)
- Service Life Management (William Stiffler)
- Service Life Extension (William Stiffler) (Brian Wells)
- Capability Updates, Upgrades, and Modernization (William Stiffler) (Brian Wells)
- System Disposal and Retirement (Brian Wells)
- Lead Authors:
- Scott Jackson and Brian Gallagher
- Contributing Author:
- David Dorgan

System Maintenance planning begins early in the acquisition process with development of a maintenance concept. Maintenance planning is conducted to evolve and establish requirements and tasks to be accomplished for achieving, restoring, and maintaining operational capability for the life of the system. For a system to be sustained throughout its system life cycle, the maintenance process has to be executed concurrently with the operations process (ISO/IEC/IEEE 15288 2015, Clause 6.4.9).

See the article Matrix of Implementation Examples for a mapping of case studies and vignettes included in Part 7 to topics covered in Part 3.

Overview

The initial requirements for maintenance have to be defined during the stakeholder needs definition process (Clause 6.4.1) (ISO/IEC/IEEE 15288 2015) and continue to evolve during the development and operation of the system. Considerations include:

- Maximizing system availability to meet the operational requirements. This has to take into account the designed-in reliability and maintainability of the system and resources available.
- Preserving system operating potential through proper planning of system scheduled maintenance. This requires a reliability-centered maintenance strategy that incorporates preventive maintenance in order to preempt failures, thereby extending the mean time between corrective maintenance, as well as enhancing the availability of the system.
- Segmenting maintenance activities for potential outsourcing of non-critical activities to approved maintenance subcontractors as to optimize scarce technical manpower resources and maintenance/repair turn-around times.
- Harnessing IT technology for maintenance management. This involves rigorous and systematic capturing and tracking of operating and maintenance activities to facilitate analysis and planning.

Maintenance management is concerned with the development and review of maintenance plans, as well as securing and coordinating resources, such as budget, service parts provisioning, and management of supporting tasks (e.g., contract administration, engineering support, and quality assurance). Maintenance planning relies on level of repair analysis (LORA) as a function of the system acquisition process. Initial planning addresses actions and support necessary to ensure a minimum life cycle cost (LCC).

Process Approaches

The purpose of the maintenance process is to sustain the capability of a system to provide a service. This process monitors the system's capability to deliver services, records problems for analysis, takes corrective, adaptive, perfective, and preventive actions, and confirms restored capability. As a result of the successful implementation of the maintenance process:

- a maintenance strategy is developed
- maintenance constraints are provided as inputs to requirements
- replacement system elements are made available
- services meeting stakeholder requirements are sustained
- the need for corrective design changes is reported
- failure and lifetime data are recorded

The project should implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the maintenance process:

- scheduled servicing, such as daily inspection/checks, servicing, and cleaning
- unscheduled servicing (carrying out fault detection and isolation to the faulty replaceable unit and replacement of the failed unit)
- re-configuration of the system for different roles or functions
- scheduled servicing (higher level scheduled servicing but below depot level)
- unscheduled servicing (carrying out more complicated fault isolation to the faulty replaceable unit and replacement of the failed unit)
- minor modifications
- minor damage repairs
- major scheduled servicing (e.g., overhaul and corrosion treatment)
- major repairs (beyond normal removal and replacement tasks)

The maintenance plan specifies the scheduled servicing tasks and intervals (preventive maintenance) and the unscheduled servicing tasks (adaptive or corrective maintenance). Tasks in the maintenance plan are allocated to the various maintenance agencies. A maintenance allocation chart is developed to tag the maintenance tasks to the appropriate maintenance agencies. These include: in-service or in-house work centers, approved contractors, affiliated maintenance or repair facilities, original equipment manufacturer (OEMs), etc. The maintenance plan also establishes the requirements for the support resources.

Related activities such as resource planning, budgeting, performance monitoring, upgrades, longer term supportability, and sustenance also need to be managed. These activities are planned, managed, and executed over a longer time horizon and they concern the well-being of the system over the entire life cycle.

Proper maintenance of the system (including maintenance-free system designs) relies very much on the availability of support resources, such as support and test equipment (STE), technical data and documentation, personnel, spares, and facilities. These have to be factored in during the acquisition agreement process.

Training and Certification

Adequate training must be provided for the technical personnel maintaining the system. While initial training may have been provided during the deployment phase, additional personnel may need to be trained to cope with the increased number of systems being fielded, as well as to cater to staff turnover. Timely updates to training materials and trained personnel may be required as part of system upgrades and evolution. It is important to define the certification standards and contract for the training materials as part of the supply agreement.

Practical Considerations

The organization responsible for maintaining the system should have clear thresholds established to determine whether a change requested by end users, changes to correct latent defects, or changes required to fulfill the evolving mission are within the scope of a maintenance change or require a more formal project to step through the entire systems engineering life-cycle. Evaluation criteria to make such a decision could include cost, schedule, risk, or criticality characteristics.

References

Works Cited

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.

Primary References

Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th Edition. Upper Saddle River, NJ, USA: Prentice Hall.

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense.

INCOSE. 2012. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Institute of Engineers Singapore. 2009. *Systems Engineering Body of Knowledge*, Provisional version 2.0. Singapore: Institute of Engineers Singapore.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015.

Additional References

None.

Logistics

- Lead Authors:
- Scott Jackson and John Snoderly
- Contributing Author:
- Garry Roedler

There are several definitions for logistics within systems engineering (SE) and the definition used will determine what activities are considered part of logistics. The SEBoK defines logistics as the science of planning and implementing the acquisition and use of the resources necessary to sustain the operation of a system.

Overview

The ability to *sustain the operation of a system* is determined by the inherent supportability of the system (a function of design) and the processes used to sustain the functions and capabilities of the system in the context of the end user. Figure 1, below, shows a Defense Acquisition University (DAU) model of the SE aspects for consideration in logistics and logistics planning (DAU 2010).

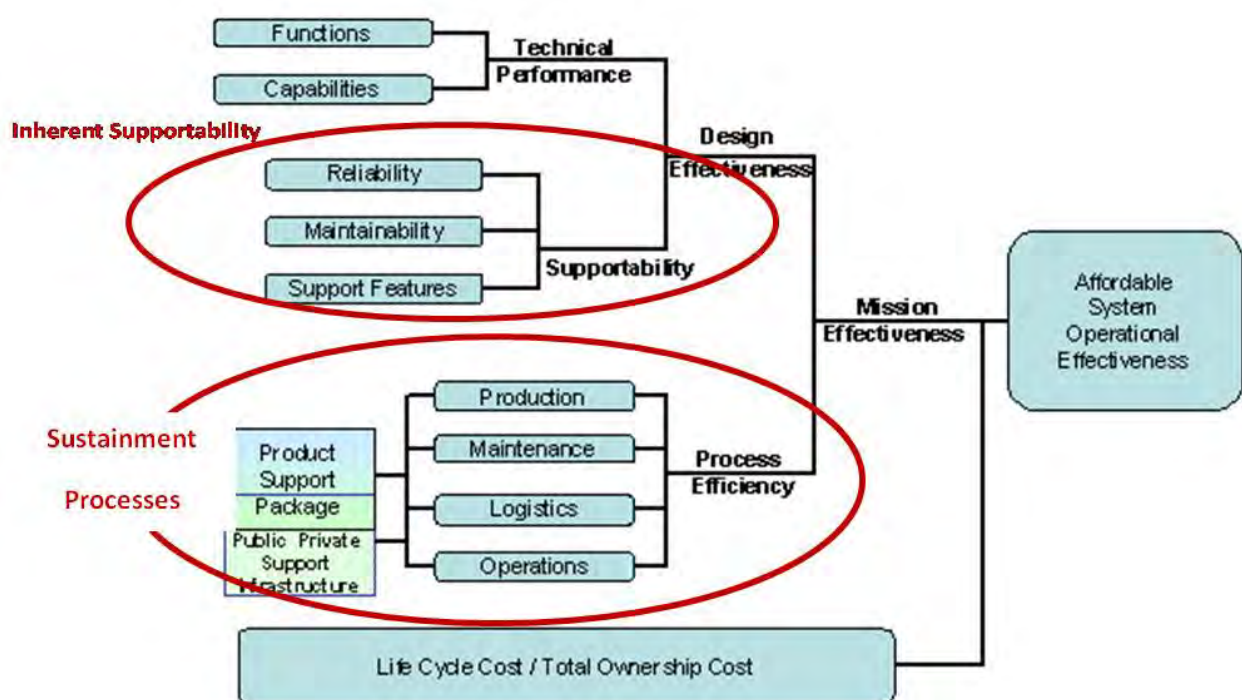


Figure 1. Affordable System Operational Effectiveness (DAU Guidebook 2010). Released by Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

Sustainment Planning

The focus of sustainment planning is to influence the inherent supportability of the system and to plan the sustainment capabilities and processes that will be used to sustain system operations.

Influence Inherent Supportability (Operational Suitability)

Sustainment influence requires an understanding of the concept of operations (ConOps), system missions, mission profiles, and system capabilities to understand the rationale behind functional and performance priorities. Understanding the rationale paves the way for decisions about necessary trade-offs between system performance, availability, and life cycle cost (LCC), with impact on the cost effectiveness of system operation, maintenance, and logistics support. There is no single list of sustainment considerations or specific way of grouping them as they are highly inter-related. They include: compatibility, interoperability, transportability, reliability, maintainability, manpower, human factors, safety, natural environment effects (including occupational health, habitability; see Systems Engineering and Environmental Engineering); diagnostics & prognostics (including real-time maintenance data collection), and corrosion protection & mitigation. The following are key design considerations:

- **Architecture Considerations** - The focus on openness, modularity, scalability, and upgradeability is critical to implementing an incremental acquisition strategy. In addition, the architecture attributes that expand system flexibility and affordability can pay dividends later when obsolescence and end-of-life issues are resolved through a concerted technology refreshment strategy. Trade-offs are often required relative to the extent each attribute is used.
- **Reliability Considerations** - Reliability is critical because it contributes to a system's effectiveness as well as its suitability in terms of logistics burden and the cost to fix failures. For each system, there is a level of basic reliability that must be achieved for the system to be considered useful. Reliability is also one of the most critical elements in determining the logistics infrastructure and footprint. Consequently, system reliability should be a primary focus during design (along with system technical performance, functions, and capabilities). The primary objective is to achieve the necessary probability of operational success and minimize the risk of failure within defined availability, cost, schedule, weight, power, and volume constraints. While performing such analyses, trade-offs should be conducted and dependencies should be explored with system maintainability and integrated with the supportability analysis that addresses support event frequency (i.e. reliability), event duration, and event cost. Such a focus will play a significant role in minimizing the necessary logistics footprint, while maximizing system availability.
- **Maintainability Considerations** - The design emphasis on maintainability is to reduce the maintenance burden and supply chain by reducing the time, personnel, tools, test equipment, training, facilities and cost to maintain the system. Maintainability engineering includes the activities, methods, and practices used to design minimal system maintenance requirements (designing out unnecessary and inefficient processes) and associated costs for preventive and corrective maintenance as well as servicing or calibration activities. Maintainability should be a designed-in capability and not an add-on option because good maintenance procedures cannot overcome poor system and equipment maintainability design. The primary objective is to reduce the time it takes for a properly trained maintainer to detect and isolate the failure (coverage and efficiency) and affect repair. Intrinsic factors contributing to maintainability are:
 - **Modularity** - Packaging of components such that they can be repaired via remove and replace action vs. on-board repair. Care should be taken not to *over modularize*, and trade-offs to evaluate replacement, transportation, and repair costs should be accomplished to determine the most cost-effective approach.
 - **Interoperability** - The compatibility of components with standard interface protocols to facilitate rapid repair and enhancement/upgrade through black box technology using common interfaces. Physical interfaces should be designed so that mating between components can only happen correctly.

- **Physical accessibility** - The designed-in structural assurance that components which require more frequent monitoring, checkout, and maintenance can be easily accessed. This is especially important in low observable platforms. Maintenance points should be directly visible and accessible to maintainers, including access for corrosion inspection and mitigation.
- Designs that require *minimum preventative maintenance* including corrosion prevention and mitigation. Emphasis should be on balancing the maintenance requirement over the life cycle with minimal user workload.
- **Embedded training and testing** when it is determined to be the optimal solution from a total ownership cost (TOC) and materiel availability perspective.
- **Human Systems Integration (HSI)** to optimize total system performance and minimize life-cycle costs by designing systems and incorporating technologies that (a) require minimal manpower, (b) provide effective training, (c) can be operated and maintained by users, (d) are suitable (habitable and safe with minimal environmental and occupational health hazards), and (e) are survivable (for both the user and the equipment).
- **Support Considerations** - Support features cannot be easily *added-on* after the design is established. Consequently, supportability should be a high priority early in the program's planning and integral to the system design and development process. Support features cut across reliability, maintainability, and the supply chain to facilitate detection, isolation, and timely repair/replacement of system anomalies. These include features for servicing and other activities necessary for operation and support including resources that contribute to the overall support of the system. Typical supportability features include diagnostics, prognostics (see CBM+ Guidebook), calibration requirements, many HSI issues (e.g. training, safety, HFE, occupational health, etc.), skill levels, documentation, maintenance data collection, compatibility, interoperability, transportability, handling (e.g., lift/hard/tie down points, etc.), packing requirements, facility requirements, accessibility, and other factors that contribute to an optimum environment for sustaining an operational system.

Planning Sustainment Processes

Process efficiency reflects how well the system can be produced, operated, serviced (including fueling) and maintained. It reflects the degree to which the logistics processes (including the supply chain), infrastructure, and footprint have been balanced to provide an agile, deployable, and operationally effective system.

Achieving process efficiency requires early and continuing emphasis on the various logistics support processes along with the design considerations. The continued emphasis is important because processes present opportunities for improving operational effectiveness even after the *design-in* window has passed via lean-six sigma, supply chain optimization, or other continuous process improvement (CPI) techniques.

Sustainment Analysis (Product Support Package)

The product support package documents the output of supportability analysis and includes details related to the following twelve elements (links below are to excerpts from (NATO RTO 2001):

- Product/information technology (IT) system/medical system support management (integrated life cycle sustainment planning)
 - product/IT system/medical system support strategies
 - life cycle sustainment planning
 - requirements management
 - total ownership costs (TOC)/life cycle costs (LCC) planning & management
 - Integration and management of product support activities
 - configuration management
 - production & distribution
 - energy, environmental, safety and health (EESH) management
 - policies & guidance

- risk management
 - Design Interface ^[1]
 - reliability
 - maintainability
 - supportability
 - affordability
 - configuration management
 - safety requirements
 - environmental and hazardous materials (HAZMAT) requirements
 - human systems integration (HSI)
 - calibration
 - anti-tamper
 - habitability
 - disposal
 - legal requirements
 - Sustainment Engineering
 - failure reporting, analysis, and corrective action system (FRACAS)
 - value engineering
 - diminishing manufacturing sources and material shortages (DMSMS)
 - Supply Support (materiel planning) ^[2]
 - Maintenance Planning ^[3]
 - reliability centered maintenance (RCM)
 - maintenance concepts
 - levels of maintenance (level of repair analysis)
 - condition-based maintenance
 - prognostics & health management
 - Support Equipment ^[4]
 - Technical Data ^[5]
 - Manpower & Personnel ^[6]
 - Training & Training Support ^[7]
 - Facilities & Infrastructure ^[8]
 - Packaging, Handling, Storage, & Transportation ^[9]
 - Computer Resources ^[10]
-

Sustainment Implementation

Once the system becomes operational, the results of sustainment planning efforts need to be implemented. SE supports the execution of the twelve integrated product support elements of a sustainment program that strives to ensure the system meets operational performance requirements in the most cost-effective manner over its total remaining life cycle, as illustrated in Figure 2.

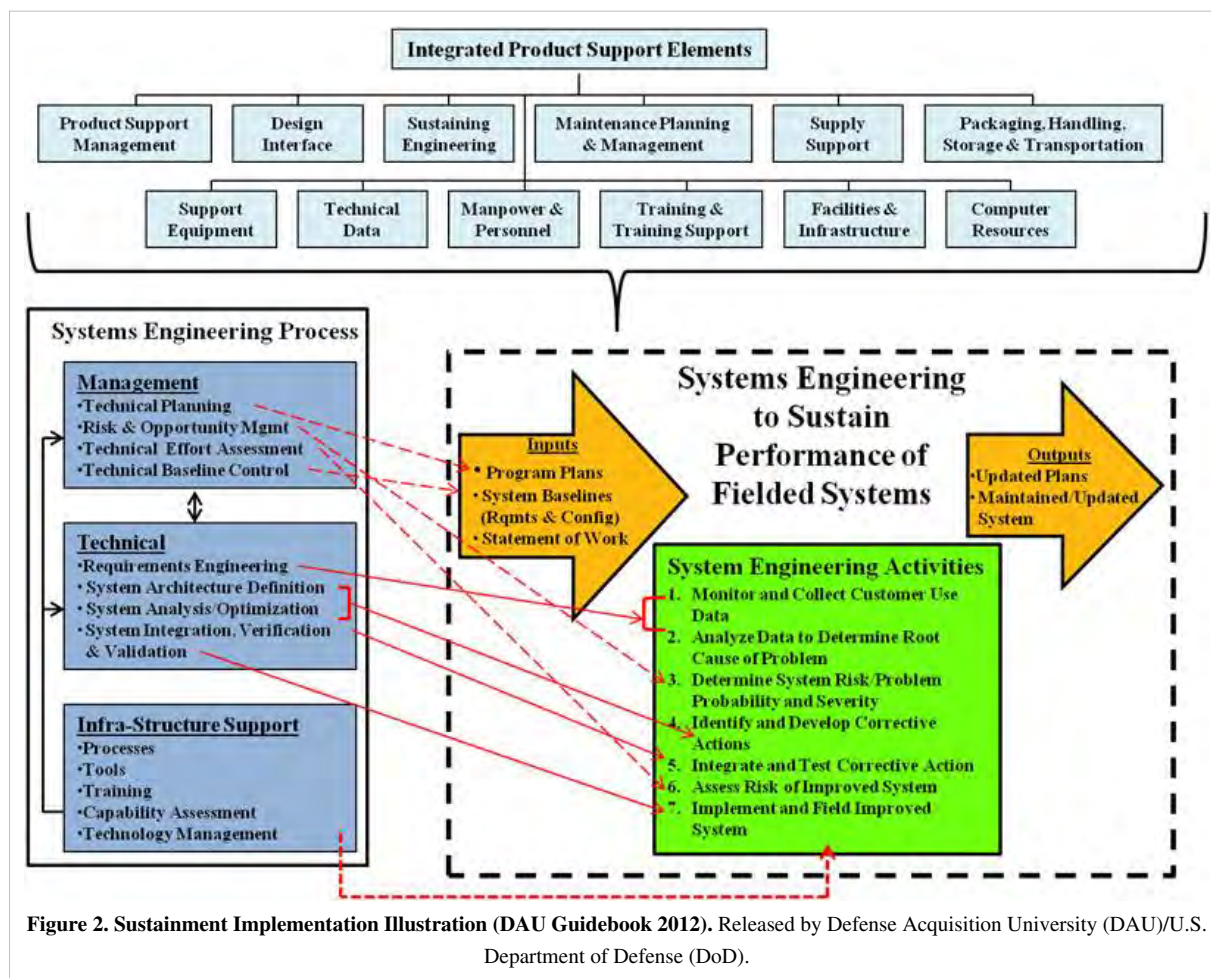


Figure 2. Sustainment Implementation Illustration (DAU Guidebook 2012). Released by Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

Once a system is put into use, SE is often required to correct problems that degrade continued use, and/or to add new capabilities to improve product performance in the current or a new environment. In the context of integrated product support, these SE activities correspond to the integrated product support (IPS) element *Sustaining Engineering*. Changes made to fielded systems to correct problems or increase performance should include any necessary adjustments to the IPS elements, and should consider the interrelationships and integration of the elements to maintain the effectiveness of the system's support strategy.

The degree of change required to the product support elements varies with the severity of the problem. Minor problems may require a simple adjustment to a maintenance procedure, a change of supplier, a training course modification or a change to a technical manual. In contrast, problems that require system or component redesign may require engineering change proposals and approvals, IPS element trade studies, business case analysis, and updates to the product support strategy. The focus is to correct problems that degrade continued use, regardless of the degree of severity.

Evolutionary systems provide a strategy for acquisition of mature technology; the system delivers capabilities incrementally, planning for future capability enhancements. A system of systems (SoS) perspective is required for these systems to synchronize the primary and sustainment systems.

For more information refer to: *An Enterprise Framework for Operationally Effective System of Systems Design* (Bobinis and Herald 2012.).

References

Works Cited

Bobinis, J. and T. Herald. 2012. "An enterprise framework for operationally effective system of systems design." *Journal of Enterprise Architecture*. Vol. 8, no. 2, May 2012. Available at: <https://www.mendling.com/publications/JEA12-2.pdf>.

DAU. 2010. Defense Acquisition Guidebook (DAG). Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

NATO RTO. 2001. *Logistics Test and Evaluation in Flight Test*. Flight Test Techniques Series – Volume 20. Quebec, Canada: North Atlantic Treaty Organization (NATO) Research and Technology Organization (RTO). RTO-AG-300 Vol. 20, AC/323(SCI-010)TP/38. Table of contents available at: [https://ftp.rta.nato.int/public//PubFullText/RTO/AG/RTO-AG-300-V20///AG-300-V20-\\$\\$TOC.pdf](https://ftp.rta.nato.int/public//PubFullText/RTO/AG/RTO-AG-300-V20///AG-300-V20-$$TOC.pdf)

Primary References

Blanchard, B.S. 1998. *Logistics Engineering and Management*. Upper Saddle River, NJ, USA: Prentice Hall.

Blanchard, B. and W. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th Ed. Englewood Cliffs, NJ, USA: Prentice-Hall.

Bobinis, J. and T. Herald. 2012. "An enterprise framework for operationally effective system of systems design." *Journal of Enterprise Architecture*. Vol. 8, no. 2, May 2012. Available at: <https://www.mendling.com/publications/JEA12-2.pdf>.

Daganzo, C. 2005. *Logistics Systems Analysis*, 4th Edition. New York, NY, USA: Springer.

Fabrycky, W.J. and B.S. Blanchard. 1991. *Life-Cycle Cost and Economic Analysis*. Upper Saddle River, NJ, USA: Prentice-Hall.

Ghiani, G., G. Laporte, and R. Musmanno. 2004. *Introduction to Logistics Systems Planning and Control*. Hoboken, NJ, USA: Wiley-Interscience.

Jones, J.V. 1995. *Integrated Logistics Support Handbook*. New York, NY, USA: McGraw Hill.

Additional References

Barros, L.L. 1998. "The optimization of repair decision using life-cycle cost parameters." *IMA Journal of Management Mathematics*. Vol. 9, no. 4, p. 403.

Berkowitz, D., J.N. Gupta, J.T. Simpson, and J.B. McWilliams. 2005. *Defining and Implementing Performance-Based Logistics in Government*. Washington, DC, USA: Defense Technical Information Center. Accessed 6 Sept 2011. Available at: <https://handle.dtic.mil/100.2/ADP018510>.

Gajpal, P.P., L.S. Ganesh, and C. Rajendran. 1994. "Criticality analysis of spare parts using the analytic hierarchy process." *International Journal of Production Economics*. Vol. 35, nos. 1-3 pp. 293-297.

MITRE. 2011. "Integrated logistics support." *Systems Engineering Guide*. Accessed 11 March 2012. Available at: [\[\[11\]\]](#).

Murthy, D.N.P. and W.R. Blischke. 2000. "Strategic warranty management: A life-cycle approach." *Engineering Management*. Vol. 47, no. 1, pp. 40-54.

Northrop Grumman Corporation. 2000. *Logistics Systems Engineering*. Accessed 6 Sept 2011. Available at: <https://www.northropgrumman.com/Capabilities/NavigationSystemsLogisticsSystemsEngineering/Documents/>

nsd_logistics.pdf.

Solomon, R., P.A. Sandborn, and M.G. Pecht. 2000. "Electronic part life cycle concepts and obsolescence forecasting." *IEEE Transactions on Components and Packaging Technologies*. Vol. 23, no. 4, pp. 707-717.

Spengler, T. and M. Schroter. 2003. "Strategic management of spare parts in closed-loop supply chains: A system dynamics approach." *Interfaces*. pp. 7-17.

References

- [1] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-12.pdf>
- [2] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-06.pdf>
- [3] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-03.pdf>
- [4] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-05.pdf>
- [5] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-07.pdf>
- [6] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-04.pdf>
- [7] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-08.pdf>
- [8] https://www.decisionlens.com/docs/WP_Strategic_Facilities_and_Infrastructure_Planning.pdf
- [9] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-11.pdf>
- [10] <https://ftp.rta.nato.int/public/PubFullText/RTO/AG/RTO-AG-300-V20/AG-300-V20-09.pdf>
- [11] https://www.mitre.org/work/systems_engineering/guide/acquisition_systems_engineering/integrated_logistics_support/

Service Life Management

- Contributing Author:
- William Stiffler

Product and service life management deals with the overall life cycle planning and support of a system. The life of a product or service spans a considerably longer period of time than the time required to design and develop the system. Systems engineers need to understand and apply the principles of life management throughout the life cycle of the system. (See Life Cycle Models for a general discussion of life cycles.) Specifically, this article focuses on changes to a system after deployment, including extension, modernization, disposal, and retirement.

Overview

Product and service life management is also referred to as *system sustainment*. Sustainment involves the supportability of operational systems from the initial procurement to disposal. Sustainment is a key task for systems engineering that influences product and service performance and support costs for the entire life of the program.

Sustainment activities include: design for maintainability, application of built-in test, diagnostics, prognostics and other condition-based maintenance techniques, implementation of logistics footprint reduction strategies, identification of technology insertion opportunities, identification of operations and support cost reduction opportunities, and monitoring of key support metrics. Life cycle sustainment plans should be created for large, complex systems (DAU 2010). Product and service life management applies to both commercial systems (e.g. energy generation and distribution systems, information management systems, the Internet, and health industries) and government systems (e.g. defense systems, transportation systems, water-handling systems, and government services).

It is critical that the planning for system life management occur during the requirements phase of system development. (See System Requirements and System Definition). The requirements phase includes the analysis of life cycle cost alternatives, as well as gaining the understanding of how the system will be sustained and modified once it is operational.

The body of knowledge associated with product and service life management includes the following areas:

1. **Service Life Extension** - Systems engineers need to understand the principles of service life extension, the challenges that occur during system modifications, and issues involved with the disposal and retirement after a system has reached the end of its useful life.
2. **Modernization and Upgrades** - Managing service life extension uses the engineering change management process with an understanding of the design life constraints of the system. Modernizing existing legacy systems requires special attention and understanding of the legacy requirements and the importance of having a complete inventory of all the system interfaces and technical drawings.
3. **Disposal and Retirement** - Disposal and retirement of a product after reaching its useful life requires attention to environmental concerns, special handling of hazardous waste, and concurrent operation of a replacement system as the existing system is being retired.

Principles and Standards

The principles of product and service life management apply to different types of systems and domains. The type of system (commercial or government) should be used to select the correct body of knowledge and best practices that exist in different domains. For example, U.S. military systems would rely on sustainment references and best practices from the Department of Defense (DoD) (e.g., military services, Defense Acquisition University (DAU), etc.) and military standardization bodies (e.g., the American Institute of Aeronautics and Astronautics (AIAA), the Society of Automotive Engineers (SAE), the Society of Logistics Engineers (SOLE), the Open Geospatial Consortium (OGC), etc.).

Commercial aviation, power distribution, transportation, water-handling systems, the Internet, and health industries would rely on system life management references and best practices from a combination of government agencies, local municipalities, and commercial standardization bodies and associations (e.g., in the U.S.- the Department of Transportation (DOT), State of Michigan, International Organization for Standardization (ISO), Institute of Electrical and Electronics Engineers (IEEE), International Council on Systems Engineering (INCOSE), etc.).

Some standardization bodies have developed system life management practices that bridge both military and commercial systems (e.g., INCOSE, SOLE, ISO, IEEE, etc.). There are multiple commercial associations involved with defining engineering policies, best practices, and requirements for commercial product and service life management. Each commercial association has a specific focus for the market or domain area where the product is used. Examples of such commercial associations in the U.S. include: American Society of Hospital Engineering (ASHE); Association of Computing Machinery (ACM); American Society of Mechanical Engineers (ASME); American Society for Testing & Materials (ASTM) International; National Association of Home Builders (NAHB); and Internet Society (ISOC), including Internet Engineering Task Force (IETF), and SAE.

In addition, there are several specific resources which provide useful information on product and service life management:

- The *INCOSE Systems Engineering Handbook*, version 3.2.2, identifies several relevant points regarding product and service life management (2011).
- The *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1, provides guidance on product changes and system retirement (Caltrans and USDOT 2005).
- *Systems Engineering and Analysis* emphasizes design for supportability and provides a framework for product and service supportability and planning for system retirement (Blanchard and Fabrycky 2006).
- *Modernizing Legacy Systems* identifies strategies for product and service modernization (Seacord, Plakosh, and Lewis 2003).
- "Logistics and Materiel Readiness" (<https://www.acq.osd.mil/log/>^[1]) provides online policies, procedures, and planning references for product service life extension, modernization, and retirement (OUSD(AT&L) 2011).
- *A Multidisciplinary Framework for Resilience to Disasters and Disruptions* provides insight into architecting a system for extended service life (Jackson 2007).

Good Practices

Major pitfalls associated with systems engineering (SE) after the deployment of products and services can be avoided if the systems engineer:

- Recognizes that the systems engineering process does not stop when the product or service becomes operational.
- Understands that certain life management functions and organizations, especially in the post-delivery phase of the life cycle, are part of the systems engineering process.
- Identifies that modifications need to comply with the system requirements.
- Considers that the users must be able to continue the maintenance activities drawn up during the system requirement phase after an upgrade or modification to the system is made.
- Accounts for changing user requirements over the system life cycle.
- Adapts the support concepts drawn up during development throughout the system life cycle.
- Applies engineering change management to the total system.

Not addressing these areas of concern early in development and throughout the product or service's life cycle can have dire consequences.

References

Works Cited

Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.

Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research & Innovation/U.S. Department of Transportation (USDOT), SEG for ITS 1.1.

DAU. 2010. "Acquisition Community Connection (ACC): Where the DoD AT&L workforce meets to share knowledge." Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/US Department of Defense (DoD). <https://acc.dau.mil>.

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Jackson. 2007. "A Multidisciplinary Framework for Resilience to Disasters and Disruptions." *Journal of Integrated Design and Process Science*. 11(2).

OUSD(AT&L). 2011. "Logistics and Materiel Readiness On-line policies, procedures, and planning references." Arlington, VA, USA: Office of the Under Secretary of Defense for Acquisition, Transportation and Logistics (OUSD(AT&L)). <https://www.acq.osd.mil/log/>.

Seacord, R.C., D. Plakosh, and G.A. Lewis. 2003. *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Pearson Education.

Primary References

- Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, ver 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research and Innovation and U.S. Department of Transportation (USDOT), SEG for ITS 1.1.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Jackson, S. 2007. "A Multidisciplinary Framework for Resilience to Disasters and Disruptions." *Journal of Integrated Design and Process Science*. 11(2).
- OUSD(AT&L). 2011. "Logistics and Materiel Readiness On-line policies, procedures, and planning references." Arlington, VA, USA: Office of the Under Secretary of Defense for Acquisition, Transportation and Logistics (OUSD(AT&L)). <https://www.acq.osd.mil/log/>.
- Seacord, R.C., D. Plakosh, and G.A. Lewis. 2003. *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Pearson Education.

Additional References

- Blanchard, B.S. 2010. *Logistics engineering and management*, 5th ed. Englewood Cliffs, NJ, USA: Prentice Hall: 341-342.
- Braunstein, A. 2007. "Balancing Hardware End-of-Life Costs and Responsibilities." Westport, CT, USA: Experture Group, ETS 07-12-18.
- Brown, M., R. Weyers, and M. Sprinkel. 2006. "Service Life Extension of Virginia Bridge Decks afforded by Epoxy-Coated Reinforcement." *Journal of ASTM International (JAI)*. 3(2): 13.
- DLA. 2010. "Defense logistics agency disposition services." In Defense Logistics Agency (DLA)/U.S. Department of Defense [database online]. Battle Creek, MI, USA, accessed June 19 2010: 5. Available at: <https://www.dtc.dla.mil>.
- EPA. 2010. "Wastes In U.S. Environmental Protection Agency (EPA)." Washington, D.C. Available at: <https://www.epa.gov/epawaste/index.htm>.
- Finlayson, B. and B. Herdlick. 2008. *Systems Engineering of Deployed Systems*. Baltimore, MD, USA: Johns Hopkins University: 28.
- FSA. 2010. "Template for 'System Retirement Plan' and 'System Disposal Plan'." In Federal Student Aid (FSA)/U.S. Department of Education (DoEd). Washington, DC, USA. Accessed August 5, 2010. Available at: <https://federalstudentaid.ed.gov/business/lcm.html>.
- Gehring, G., D. Lindemuth, and W.T. Young. 2004. "Break Reduction/Life extension Program for CAST and Ductile Iron Water Mains." Paper presented at NO-DIG 2004, Conference of the North American Society for Trenchless Technology (NASTT), March 22-24, New Orleans, LA, USA.
- Hovinga, M.N., and G.J. Nakoneczny. 2000. "Standard Recommendations for Pressure Part Inspection during a Boiler Life Extension Program." Paper presented at ICOLM (International Conference on Life Management and Life Extension of Power Plant), May, Xi'an, P.R. China.
- IEC. 2007. *Obsolescence Management - Application Guide*, ed 1.0. Geneva, Switzerland: International Electrotechnical Commission, IEC 62302.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical

and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Ihii, K., C.F. Eubanks, and P. Di Marco. 1994. "Design for Product Retirement and Material Life-Cycle." *Materials & Design*. 15(4): 225-33.

INCOSE UK Chapter. 2010. *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook*, version 3.2, issue 1.0. Foresgate, UK: International Council on Systems Engineering (INCOSE) UK Chapter: 10, 13, 23.

Institute of Engineers Singapore. 2009. *Systems Engineering Body of Knowledge, provisional*, version 2.0. Singapore.

Jackson, S. 1997. *Systems Engineering for Commercial Aircraft*. Surrey, UK: Ashgate Publishing, Ltd.

Koopman, P. 1999. "Life Cycle Considerations." Pittsburgh, PA, USA: Carnegie Mellon. Accessed August 5, 2010. Available at: https://www.ece.cmu.edu/~koopman/des_s99/life_cycle/index.html.

L3 Communications. 2010. "Service Life Extension Program (SLEP)." Newport News, VA, USA: L3 Communications, Flight International Aviation LLC.

Livingston, H. 2010. "GEB1: Diminishing Manufacturing Sources and Material Shortages (DMSMS) Management Practices." McClellan, CA, USA: Defense MicroElectronics Activity (DMEA)/U.S. Department of Defense (DoD).

Minneapolis-St. Paul Chapter of SOLE. 2003. "Systems Engineering in Systems Deployment and Retirement, presented to INCOSE." Minneapolis-St. Paul, MN, USA: International Society of Logistics (SOLE), Minneapolis-St. Paul Chapter.

NAS. 2006. *National Airspace System (NAS) System Engineering Manual*, version 3.1 (volumes 1-3). Washington, D.C.: Air Traffic Organization (ATO)/U.S. Federal Aviation Administration (FAA), NAS SEM 3.1.

NASA. 2007. *Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105, December 2007.

Nguyen, L. 2006. "Adapting the Vee Model to Accomplish Systems Engineering on Change Projects." Paper presented at 9th Annual National Defense Industrial Association (NDIA) Systems Engineering Conference, San Diego, CA, USA.

Office of Natural Gas and Oil Technology. 1999. *Reservoir LIFE Extension Program: Encouraging Production of Remaining Oil and Gas*. Washington, DC, USA: U.S. Department of Energy (DoE).

Paks Nuclear Power Plant. 2010. "Paks Nuclear Power Plant: Service Life Extension." In Paks Nuclear Power Plant, Ltd.. Hungary, accessed August 5, 2010. Available at: <https://paksnuclearpowerplant.com/service-life-extension>.

Ryen, E. 2008. *Overview of the Systems Engineering Process*. Bismarck, ND, USA: North Dakota Department of Transportation (NDDOT).

SAE International. 2010. "Standards: Commercial Vehicle--Maintenance and Aftermarket." Warrendale, PA, USA: Society of Automotive Engineers (SAE) International.

SAE International. 2010. "Standards: Maintenance and Aftermarket." Warrendale, PA, USA: Society of Automotive Engineers (SAE) International.

Sukamto, S. 2003. "Plant Aging and Life Extension Program at Arun LNG Plant Lhokseumawe, North Aceh, Indonesia." Paper presented at 22nd Annual World Gas Conference, June 1-5, Tokyo, Japan.

References

[1] <https://www.acq.osd.mil/log/>

Service Life Extension

- Lead Author:
- William Stiffler
- Contributing Author:
- Brian Wells

Product and service life extension involves continued use of a product and/or service beyond its original design life. Product and service life extension involves assessing the risks and the life cycle cost (LCC) of continuing the use of the product or service versus the cost of a replacement system.

Service life extension (SLE) emphasizes reliability upgrades and component replacement or rebuilding of the system to delay the system's entry into *wear-out* status due to issues such as expensive sustainment, reliability, safety, and/or performance requirements that can no longer be met. The goal is typically to return the system to as new a condition as possible while remaining consistent with the economic constraints of the program.

SLE is regarded as an environmentally friendly way to relieve rampant waste by prolonging the useful life of retiring products and preventing them from being discarded too early when they still have unused value. However, challenged by fast-changing technology and physical deterioration, a major concern in planning a product SLE is considering to what degree a product or service is fit to have its life extended.

Topic Overview

SLE is typically required in the following circumstances:

- The system no longer meets the system performance or reliability requirements.
- The cost of operation and maintenance exceeds the cost of SLE, or the available budgets.
- Parts are no longer available for repair and maintenance.
- Operation of the system violates rules or regulations, such as environmental or safety regulations.
- Parts of the system are about to reach their operations life limits, which will result in the issue listed above occurring.

It is best if systems engineers use a proactive approach that predicts ahead, so that SLE can be accomplished before the system fails to meet its requirements and before the operations and support costs rise above acceptable limits.

Key factors that must be considered by the systems engineer during service life extension include:

- current life cycle costs of the system
- design life and expected remaining useful life of the system
- software maintenance
- configuration management
- warranty policy
- availability of parts, subsystems, and manufacturing sources
- availability of system documentation to support life extension

System design life is a major consideration for SLE. System design life parameters are established early on during the system design phase and include key assumptions involving safety limits and material life. Safety limits and the properties of material aging are critical to defining system life extension. Jackson emphasizes the importance of architecting for system resiliency in increasing system life. He also points out that a system can be architected to withstand internal and external disruptions (2007, 91-108). Systems that age through use, such as aircraft, bridges,

and nuclear power plants, require periodic inspection to ascertain the degree of aging and fatigue. The results of inspections determine the need for actions to extend the product life (Elliot, Chen, and Swanekamp 1998, sec. 6.5).

Software maintenance is a critical aspect of SLE. The legacy system may include multiple computer resources that have been in operation for a period of many years and have essential functions that must not be disrupted during the upgrade or integration process. Typically, legacy systems include a computer resource or application software program that continues to be used because the cost of replacing or redesigning it is prohibitive. The Software Engineering Institute (SEI) has addressed the need for SLE of software products and services and provides useful guidance in the online library for Software Product Lines (SEI 2010, 1). (See Systems Engineering and Software Engineering for additional discussion of software engineering (SwE) factors to consider.)

Systems engineers have found that service life can be extended through the proper selection of materials. For example, transportation system elements such as highway bridges and rail systems are being designed for extended service life by using special epoxy-coated steel (Brown, Weyers, and Spinkel 2006, 13). Diminishing manufacturing sources and diminishing suppliers need to be addressed early in the SLE process. Livingston (2010) in *Diminishing Manufacturing Sources and Material Shortages (DMSMS) Management Practices* provides a method for addressing product life extension when the sources of supply are an issue. He addresses the product life cycle model and describes a variety of methods that can be applied during system design to minimize the impact of future component obsolescence issues.

During product and service life extension, it is often necessary to revisit and challenge the assumptions behind any previous life cycle cost analysis (and constituent analyses) to evaluate their continued validity and/or applicability early in the process.

Application to Product Systems

Product life extension requires an analysis of the LCC associated with continued use of the existing product versus the cost of a replacement product. In the INCOSE Systems Engineering Handbook, Chapter 3.3 points out that the support stage includes service life extension (2012). Chapter 7 provides a framework to determine if a product's life should be extended (INCOSE 2012). In Systems Engineering and Analysis, Chapter 17 provides an LCC methodology and emphasizes the analysis of different alternatives before deciding on product life extension (Blanchard and Fabrycky 2011).

For military systems, service life extension is considered a subset of modification or modernization. Military systems use well-developed and detailed guidance for SLE programs (SLEP). The Office of the Under Secretary of Defense for Acquisition, Technology, and Logistics (OSD AT&L) provides an online reference for policies, procedures, planning guidance, and whitepapers for military product service life extension (DAU 2011). Continuous military system modernization is a process by which state-of-the-art technologies are inserted continuously into weapon systems to increase reliability, lower sustainment costs, and increase the war fighting capability of a military system to meet evolving customer requirements throughout an indefinite service life.

Aircraft service life can be extended by reducing the dynamic loads which lead to structural fatigue. The Boeing B-52 military aircraft and the Boeing 737 commercial aircraft are prime examples of system life extension. The B-52 was first fielded in 1955 and the Boeing 737 has been fielded since 1967; both aircraft are still in use today.

For nuclear reactors, system safety is the most important precondition for service life extension. System safety must be maintained while extending the service life (Paks 2010). Built-in tests, automated fault reporting and prognostics, analysis of failure modes, and the detection of early signs of wear and aging may be applied to predict the time when maintenance actions will be required to extend the service life of the product. (For additional discussion, see System Safety.)

Application to Service Systems

For systems that provide services to a larger consumer base, SLE involves continued delivery of the service without disrupting consumer use. This involves capital investment and financial planning, as well as a phased deployment of changes. Examples of these concepts can be seen in transportation systems, water treatment facilities, energy generation and delivery systems, and the health care industry. As new technologies are introduced, service delivery can be improved while reducing LCC's. Service systems must continuously assess delivery costs based upon the use of newer technologies.

Water handling systems provide a good example of a service system that undergoes life extension. Water handling systems have been in existence since early civilization. Since water handling systems are in use as long as a site is occupied (e.g., the Roman aqueducts) and upgrades are required as the population expands, such systems are a good example of "systems that live forever." For example, there are still U.S. water systems that use a few wooden pipes since there has been no reason to replace them. Water system life extension must deal with the issue of water quality and the capacity for future users (Mays 2000). Water quality requirements can be further understood from the AWWA Manuals of Water Supply Practices (AWWA 2010).

Application to Enterprises

SLE of a large enterprise, such as the National Astronautics and Space Administration's (NASA) national space transportation system, involves SLE on the elements of the enterprise, such as the space vehicle (shuttle), ground processing systems for launch operations and mission control, and space-based communication systems that support space vehicle tracking and status monitoring. SLE of an enterprise requires a holistic look across the entire enterprise. A balanced approach is required to address the cost of operating older system components versus the cost required to implement service life improvements.

Large enterprise systems, such as oil and natural gas reservoirs which span broad geographical areas, can use advanced technology to increase their service life. The economic extraction of oil and natural gas resources from previously established reservoirs can extend their system life. One such life extension method is to pump special liquids or gases into the reservoir to push the remaining oil or natural gas to the surface for extraction (Office of Natural Gas & Oil Technology 1999).

Other Topics

Commercial product developers have been required to retain information for extended periods of time after the last operational product or unit leaves active service (for up to twenty years). Regulatory requirements should be considered when extending service life (INCOSE 2012).

Practical Considerations

The cost associated with life extension is one of the main inputs in the decision to extend service life of a product or a service. The cost of SLE must be compared to the cost of developing and deploying a new system, as well as the functional utility the user will obtain from each of the alternatives. It is often the case that the funding required for SLE of large complex systems is spread over several fiscal planning cycles and is therefore subject to changes in attitude by the elected officials that appropriate the funding.

The challenges with upgrading a system while it is still being used, which is often the case with SLE, must be understood and planned to avoid serious disruptions to the services the systems provide.

Any SLE must also consider the obsolescence of the systems parts, (e.g., software, amount of system redesign that is required to eliminate the obsolete parts, etc.).

References

Works Cited

- AWWA. 2010. "AWWA Manuals of Water Supply Practices." In American Water Works Association (AWWA). Denver, CO. Accessed August 5, 2010. Available at: <https://www.awwa.org/Resources/standards.cfm?ItemNumber=47829&navItemNumber=47834>.
- Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Brown, M., R. Weyers, and M. Sprinkel. 2006. "Service Life Extension of Virginia Bridge Decks afforded by Epoxy-Coated Reinforcement." *Journal of ASTM International (JAI)*, 3(2): 13.
- DAU. 2010. "Acquisition community connection (ACC): Where the DoD AT&L workforce meets to share knowledge." In Defense Acquisition University (DAU)/US Department of Defense (DoD). Ft. Belvoir, VA, USA, accessed August 5, 2010. <https://acc.dau.mil/>.
- Elliot, T., K. Chen, and R.C. Swanekamp. 1998. "Section 6.5" in *Standard Handbook of Powerplant Engineering*. New York, NY, USA: McGraw Hill.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Jackson. 2007. "A Multidisciplinary Framework for Resilience to Disasters and Disruptions." *Journal of Integrated Design and Process Science*. 11(2).
- Livingston, H. 2010. "GEB1: Diminishing Manufacturing Sources and Material Shortages (DMSMS) Management Practices." McClellan, CA: Defense MicroElectronics Activity (DMEA)/U.S. Department of Defense (DoD).
- Mays, L. (ed). 2000. "Chapter 3" in *Water Distribution Systems Handbook*. New York, NY, USA: McGraw-Hill Book Company.
- Office of Natural Gas and Oil Technology. 1999. *Reservoir LIFE Extension Program: Encouraging Production of Remaining Oil and Gas*. Washington, DC, USA: U.S. Department of Energy (DoE).
- Paks Nuclear Power Plant. 2010. "Paks Nuclear Power Plant: Service Life Extension." In Paks Nuclear Power Plant, Ltd. Hungary, accessed August 5, 2010. Available at: <https://paksnuclearpowerplant.com/service-life-extension>.
- SEI. 2010. "Software Engineering Institute." In Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU). Pittsburgh, PA, accessed August 5, 2010. <https://www.sei.cmu.edu>.

Primary References

- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research and Innovation and U.S. Department of Transportation (USDOT), SEG for ITS 1.1.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Jackson. 2007. "A Multidisciplinary Framework for Resilience to Disasters and Disruptions." *Journal of Integrated Design and Process Science*. 11(2).
- OUSD(AT&L). 2011. "Logistics and Materiel Readiness On-line policies, procedures, and planning references." Arlington, VA, USA: Office of the Under Secretary of Defense for Acquisition, Transportation and Logistics

(OUSD(AT&L)). <https://www.acq.osd.mil/log/>.

Seacord, R.C., D. Plakosh, and G.A. Lewis. 2003. *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Pearson Education.

Additional References

AWWA. 2010. "AWWA Manuals of Water Supply Practices." In American Water Works Association (AWWA). Denver, CO, USA. Accessed August 5, 2010. Available at: <https://www.awwa.org/Resources/standards.cfm?ItemNumber=47829&navItemNumber=47834>.

Blanchard, B.S. 2010. *Logistics engineering and management*, 5th ed. Englewood Cliffs, NJ, USA: Prentice Hall, 341-342.

Braunstein, A. 2007. "Balancing Hardware End-of-Life Costs and Responsibilities." Westport, CT, USA: Experture Group, ETS 07-12-18.

Brown, M., R. Weyers, and M. Sprinkel. 2006. "Service Life Extension of Virginia Bridge Decks afforded by Epoxy-Coated Reinforcement." *Journal of ASTM International (JAI)*, 3(2): 13.

Caltrans and USDOT. 2005. *Systems engineering guidebook for ITS*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research & Innovation/U.S. Department of Transportation (USDOT), SEG for ITS 1.1: 278, 101-103, 107.

Casetta, E. 2001. *Transportation Systems Engineering: Theory and methods*. New York, NY, USA: Kluwer Publishers Academic, Springer.

DAU. 2010. "Acquisition community connection (ACC): Where the DoD AT&L workforce meets to share knowledge." In Defense Acquisition University (DAU)/US Department of Defense (DoD). Ft. Belvoir, VA, USA, accessed August 5, 2010. <https://acc.dau.mil/>.

DLA. 2010. "Defense logistics agency disposition services." In Defense Logistics Agency (DLA)/U.S. Department of Defense [database online]. Battle Creek, MI, USA, accessed June 19 2010: 5. Available at: <https://www.dtc.dla.mil>.

Elliot, T., K. Chen, and R.C. Swanekamp. 1998. "Section 6.5" in *Standard Handbook of Powerplant Engineering*. New York, NY, USA: McGraw Hill.

FAA. 2006. "Section 4.1" in "Systems Engineering Manual." Washington, DC, USA: US Federal Aviation Administration (FAA).

FCC. 2009. "Radio and Television Broadcast Rules." Washington, DC, USA: US Federal Communications Commission (FCC), 47 CFR Part 73, FCC Rule 09-19: 11299-11318.

Finlayson, B. and B. Herdlick. 2008. *Systems Engineering of Deployed Systems*. Baltimore, MD, USA: Johns Hopkins University: 28.

Gehring, G., D. Lindemuth, and W.T. Young. 2004. "Break Reduction/Life extension Program for CAST and Ductile Iron Water Mains." Paper presented at NO-DIG 2004, Conference of the North American Society for Trenchless Technology (NASTT), March 22-24, New Orleans, LA, USA.

Hovinga, M.N. and G.J. Nakoneczny. 2000. "Standard Recommendations for Pressure Part Inspection during a Boiler Life Extension Program." Paper presented at ICOLM (International Conference on Life Management and Life Extension of Power Plant), May, Xi'an, P.R. China.

IEC. 2007. *Obsolescence Management - Application Guide*, ed 1.0. Geneva, Switzerland: International Electrotechnical Commission, IEC 62302.

IEEE. 2010. *IEEE Standard Framework for Reliability Prediction of Hardware*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE STD 1413.

- IEEE. 1998. *IEEE Standard Reliability Program for the Development and Production of Electronic Systems and Equipment*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE STD 1332.
- IEEE. 2008. *IEEE Recommended practice on Software Reliability*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE STD 1633.
- IEEE. 2005. *IEEE Standard for Software Configuration Management Plans*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE STD 828.
- IEEE. 2010. *IEEE Standard Framework for Reliability Prediction of Hardware*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE STD 1413.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- Ihii, K., C.F. Eubanks, and P. Di Marco. 1994. "Design for Product Retirement and Material Life-Cycle." *Materials & Design*. 15(4): 225-33.
- INCOSE UK Chapter. 2010. *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook*, version 3.2, issue 1.0. Foresgate, UK: International Council on Systems Engineering (INCOSE) UK Chapter: 10, 13, 23.
- Institute of Engineers Singapore. 2009. "Systems Engineering Body of Knowledge, provisional," version 2.0. Singapore.
- ISO/IEC. 2003. "Industrial Automation Systems Integration-Integration of Life-Cycle Data for Process Plants including Oil, Gas Production Facilities." Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC).
- ISO/IEC. 1997. "Systems Engineering for Commercial Aircraft." Surrey, UK: Ashgate Publishing Ltd.
- Koopman, P. 1999. "Life Cycle Considerations." In Carnegie-Mellon University (CMU). Pittsburgh, PA, USA, accessed August 5, 2010. Available at: https://www.ece.cmu.edu/~koopman/des_s99/life_cycle/index.html.
- L3 Communications. 2010. "Service Life Extension Program (SLEP)." Newport News, VA, USA: L3 Communications, Flight International Aviation LLC.
- Livingston, H. 2010. "GEB1: Diminishing Manufacturing Sources and Material Shortages (DMSMS) Management Practices." McClellan, CA: Defense MicroElectronics Activity (DMEA)/U.S. Department of Defense (DoD).
- Mays, L. (ed). 2000. "Chapter 3" in *Water Distribution Systems Handbook*. New York, NY, USA: McGraw-Hill Book Company.
- MDIT. 2008. *System Maintenance Guidebook (SMG)*, version 1.1: A companion to the systems engineering methodology (SEM) of the state unified information technology environment (SUITE). MI, USA: Michigan Department of Information Technology (MDIT), DOE G 200: 38.
- NAS. 2006. *National Airspace System (NAS) System Engineering Manual*, version 3.1 (volumes 1-3). Washington, D.C., USA: Air Traffic Organization (ATO)/U.S. Federal Aviation Administration (FAA), NAS SEM 3.1.
- NASA. 2007. *Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105, December 2007.
- Office of Natural Gas and Oil Technology. 1999. *Reservoir LIFE Extension Program: Encouraging Production of Remaining Oil and Gas*. Washington, DC, USA: U.S. Department of Energy (DoE).
- Paks Nuclear Power Plant. 2010. "Paks Nuclear Power Plant: Service Life Extension." In Paks Nuclear Power Plant, Ltd. Hungary, accessed August 5, 2010. Available at: <https://paksnuclearpowerplant.com/service-life-extension>.
- Reason, J. 1997. *Managing the Risks of Organizational Accident*. Aldershot, UK: Ashgate.

Ryen, E. 2008. *Overview of the Systems Engineering Process*. Bismarck, ND, USA: North Dakota Department of Transportation (NDDOT).

SAE International. 2010. "Standards: Automotive--Maintenance and Aftermarket." Warrendale, PA: Society of Automotive Engineers (SAE) International.

Schafer, D.L. 2003. "Keeping Pace With Technology Advances When Funding Resources Are Diminished." Paper presented at Auto Test Con. IEEE Systems Readiness Technology Conference, Anaheim, CA, USA: 584.

SEI. 2010. "Software Engineering Institute." In Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU). Pittsburgh, PA, accessed August 5, 2010. <https://www.sei.cmu.edu>.

SOLE. 2009. "Applications Divisions." In The International Society of Logistics (SOLE). Hyattsville, MD, USA, accessed August 5, 2010. <https://www.sole.org/appdiv.asp>.

Sukanto, S. 2003. "Plant Aging and Life Extension Program at Arun LNG Plant Lhokseumawe, North Aceh, Indonesia." Paper presented at 22nd Annual World Gas Conference, June 1-5, Tokyo, Japan.

Capability Updates, Upgrades, and Modernization

- Lead Author:
- William Stiffler
- Contributing Author:
- Brian Wells

Modernization and upgrades involve changing the product or service to include new functions and interfaces, improve system performance, and/or improve system supportability. The logistic support of a product or service reaches a point in its life where system modernization is required to resolve supportability problems and to reduce operational costs. The *INCOSE Systems Engineering Handbook* (INCOSE 2012) and *Systems Engineering and Analysis* (Blanchard and Fabrycky 2005) both stress the importance of using life cycle costs (LCC) when determining if a product or service should be modernized. Systems can be modernized in the field or returned to a depot or factory for modification.

Design for system modernization and upgrade is an important part of the system engineering process and should be considered as part of the early requirements and design activities. Engineering change proposals (ECPs) are used to initiate updates and modifications to the original system. Product and service upgrades can include new technology insertion, removing old equipment, or adding new equipment. Form, fit, function, and interface (F3I) is an important principle for upgrades where backward compatibility is a requirement.

Topic Overview

Product and service modernization involves the same systems engineering (SE) processes and principles that are employed during the upfront design, development, integration, and testing. The primary differences between product and service modernization are the various constraints imposed by the existing system architecture, design, and components. Modernizing a legacy system requires a detailed understanding of the product or service prior to making any changes. The constraints and the existence of design and test artifacts make it necessary for the systems engineers performing modernization to tailor the traditional development processes to fit the situation.

Product and service modernization occurs for many reasons, including the following:

1. The system or one of its subsystems is experiencing reduced performance, safety, or reliability.
 2. A customer or other stakeholder desires a new capability for the system.
-

3. Some system components may be experiencing obsolescence, including the lack of spare parts.
4. New uses for the system require modification to add capabilities not built into the originally deployed system.

The first three reasons above are discussed in more detail in *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook*. (INCOSE UK Chapter 2010).

The UK chapter of the INCOSE developed *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook* (INCOSE UK Chapter 2010). This guidance document applies to any system for which multiple systems are produced. These systems may be buildings, transmission networks, aircraft, automobiles or military vehicles, trains, naval vessels, and mass transit systems.

Government and military products provide a comprehensive body of knowledge for system modernization and updates. Key references have been developed by the defense industry and can be particular to their needs.

Key factors and questions that must be considered by the systems engineer when making modifications and upgrades to a product or service include:

- type of system (space, air, ground, maritime, and safety critical)
- missions and scenarios of expected operational usage
- policy and legal requirements that are imposed by certain agencies or business markets
- product or service LCCs
- electromagnetic spectrum usage expected, including change in RF emissions
- system original equipment manufacturer (OEM) and key suppliers, and availability of parts and subsystems
- understanding and documenting the functions, interfaces, and performance requirements, including environmental testing and validation
- system integration challenges posed by the prevalence of system-of-systems solutions and corresponding interoperability issues between legacy, modified, and new systems
- amount of regression testing to be performed on the existing software

Key processes and procedures that should be considered during product and service modernization include:

- legislative policy adherence review and certification
- safety critical review
- engineering change management and configuration control
- analysis of alternatives
- warranty and product return process implementation
- availability of manufacturing and supplier sources and products

Application to Product Systems

Product modernization involves understanding and managing a list of product deficiencies, prioritizing change requests, and handling customer issues associated with product usage. The *INCOSE Systems Engineering Handbook* emphasizes the use of Failure Modes, Effects, and Criticality Analysis (FMECA) to understand the root causes of product failures and provide the basis for making any product changes.

Product modernization uses the engineering change management principle of change control boards to review and implement product changes and improvements. The U.S. Office of the Under Secretary of Defense for Acquisition, Technology, and Logistics (OUSD AT&L) provides an online reference for product modernization and the use of an ECP to document planned product or service modernization efforts.

Product modernization and upgrades require the use of system documentation. A key part of the product change process is to change the supporting system documentation functions, interfaces, modes, performance requirements, and limitations. Both INCOSE (2012) and Blanchard and Fabrycky (2005) stress the importance of understanding the intended usage of the product or service documented in the form of a concept of operations.

If system documentation is not available, reverse engineering is required to capture the proper “as is configuration” of the system and to gain understanding of system behavior prior to making any changes. Seacord, Plakosh, and Lewis's *Modernizing Legacy Systems* (2003) explains the importance of documenting the existing architecture of a system, including documenting the software architecture prior to making any changes. Chapter 5 of Seacord, Plakosh, and Lewis provides a framework for understanding and documenting a legacy system (2003). The authors point out that the product or service software will undergo a transformation during modernization and upgrades. Chapter 5 introduces a horseshoe model that includes functional transformation, code transformation, and architecture transformation (Seacord, Plakosh, and Lewis 2005).

During system verification and validation (after product change), it is important to perform regression testing on the portions of the system that were not modified to confirm that upgrades did not impact the existing functions and behaviors of the system. The degree and amount of regression testing depends on the type of change made to the system and whether the upgrade includes any changes to those functions or interfaces involved with system safety. INCOSE (2012) recommends the use of a requirements verification traceability matrix to assist the systems engineer during regression testing.

It is important to consider changes to the system support environment. Change may require modification or additions to the system test equipment and other support elements such as packaging and transportation.

Some commercial products contain components and subsystems where modernization activities cannot be performed. An example of these types of commercial systems can be seen by looking at consumer electronics, such as radios and computer components. The purchase price of these commercial systems is low enough that upgrades are not economical and are considered cost prohibitive.

Application to Service Systems

Service system modernization may require regulatory changes to allow the use of new technologies and new materials. Service system modernization requires backward compatibility to previous provided service capability during the period of change. Service system modernization also generally spans large geographical areas, requiring a phase-based change and implementation strategy. Transportation systems, such as highways, provide service to many different types of consumers and span large geographical areas. Modernization of transportation systems often requires reverse engineering prior to making changes to understand how traffic monitoring devices such as metering, cameras, and toll tags interface with the rest of the system. The California Department of Transportation's (CDOT's) Systems Engineering Guidebook for Intelligent Transportation Systems (ITS) (2005) adds reverse engineering to the process steps for system upgrade. In addition, this reference points out the need to maintain system integrity and defines integrity to include the accurate documentation of the system's functional, performance, and physical requirements in the form of requirements, design, and support specifications.

Software modernization is discussed in the *Guide to the Software Engineering Body of Knowledge (SWEBOK)* (Bourque and Fairley, 2014).

Application to Enterprises

Enterprise system modernization must consider the location of the modification and the conditions under which the work will be performed. The largest challenge is implementing the changes while the system remains operational. In these cases, disruption of ongoing operations is a serious risk. For some systems, the transition between the old and new configuration is particularly important and must be carefully planned.

Enterprise system modernization may require coordination of changes across international boundaries. Enterprise modifications normally occur at a lower level of the system hierarchy. Change in requirements at the system level would normally constitute a new system or a new model of a system.

The *INCOSE UK Chapter Supplementary Guidance* (2010) discusses the change to the architecture of the system. In cases where a component is added or changed, this change will constitute a change to the architecture. As an example, the global positioning system (GPS) is an enterprise system implemented by the United States military but used by both commercial and government consumers worldwide. Modernization may involve changes to only a certain segment of the enterprise, such as the ground user segment to reduce size, weight, and power. Modernization may only occur in certain geographical areas of operation. For example, the air transportation system consists of multiple countries and governing bodies dispersed over the entire world. Changes can occur locally or can require coordination and integration world-wide.

Other Topics

The Vee Model for Modifications

Figure 1 below illustrates how the standard Vee model would be applied to a system modification. This Vee model is for the entire system; the key point is that if a modification is being initiated at a lower level of the system hierarchy, the Vee model must be entered at that level as shown in the figure. The figure shows three entry points to the Vee model. As the *INCOSE UK Chapter Supplementary Guidance* (2010) points out, the Vee model may be entered multiple times during the life of the system.

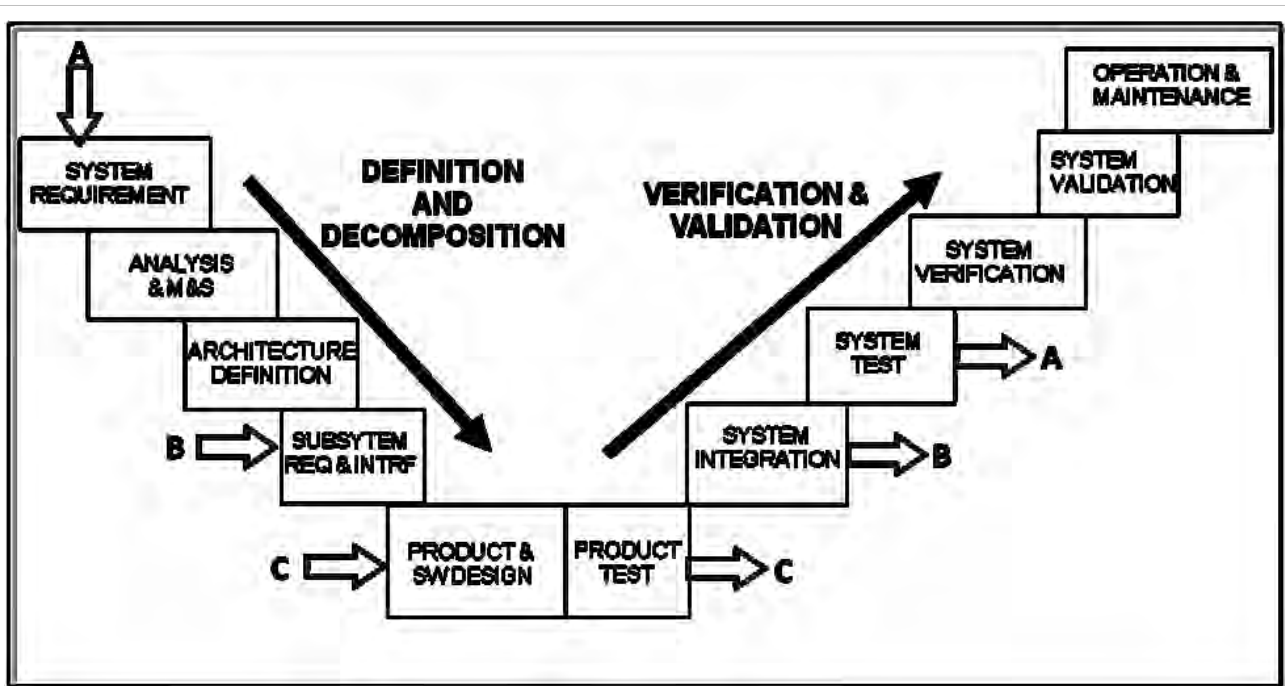


Figure 1. The Vee Model for Modifications at the Three Different Levels. (SEBoK Original)

A change to the system that does not change the system capabilities but does change the requirements and design of a subsystem that may be introduced into the process at point B on the Vee model (see Figure 1). Changes of this type

could provide a new subsystem, such as a computer system, that meets the system-level requirements but has differences from the original, which necessitates modifications to the lower-level requirements and design, such as changing disk memory to solid state memory. The process for implementing changes starting at this point has been described by Nguyen (2006). Modification introduced at points B or C (in Figure 1) necessitate flowing the requirements upward through their “parent” requirements to the system-level requirements.

There are many cases where the change to a system needs to be introduced at the lowest levels of the architectural hierarchy; here, the entry point to the process is at point C on the Vee model. These cases are typically related to obsolete parts caused by changes in technology or due to reliability issues with subsystems and parts chosen for the original design. A change at this level should be F3I compatible so that none of the higher-level requirements are affected. The systems engineer must ensure there is no impact at the higher levels; when this does occur, it must be immediately identified and worked out with the customer and the other stakeholders.

In “Life extension of Civil Infrastructural works - a systems engineering approach” van der Laan (2008) provides a maintenance process that interacts with the system engineering process, represented by the Vee model. His life extension (or modernization) process model includes reverse engineering to obtain the system definition necessary for the modernization process. Consideration of the total lifecycle of the system will result in the capture of all of the records necessary for later upgrade; however, for many reasons, the systems engineer will find that the necessary information has not been captured or maintained.

Practical Considerations

As pointed out by the *INCOSE UK Chapter Supplementary Guidance* (2010) there may be multiple modifications to a system in its lifetime. Often these modifications occur concurrently. This situation requires special attention and there are two methods for managing it. The first is called the “block” method. This means that a group of systems are in the process of being modified simultaneously and will be deployed together as a group at a specific time. This method is meant to ensure that at the end state, all the modifications have been coordinated and integrated so there are no conflicts and no non-compliance issues with the system-level requirements. The second method is called continuous integration and is meant to occur concurrently with the block method. Information management systems provide an example of a commercial system where multiple changes can occur concurrently. The information management system hardware and network modernization will cause the system software to undergo changes. Software release management is used to coordinate the proper timing for the distribution of system software changes to end-users (Michigan Department of Information Technology, 2008).

Application of Commercial-Off-the-Shelf Components

Currently, a prominent consideration is the use of commercial-off-the-shelf (COTS) components. The application of COTS subsystems, components, and technologies to system life management provides a combination of advantages and risks. The first advantage is the inherent technological advancements that come with COTS components. COTS components continue to evolve toward a higher degree of functional integration. They provide increased functionality, while shrinking in physical size. The other advantage to using COTS components is that they typically have a lower cost.

The risks associated with using COTS during system life management involve component obsolescence and changes to system interfaces. Commercial market forces drive some components to obsolescence within two years or less. Application of COTS requires careful consideration to form factor and interface (physical and electrical).

References

Works Cited

- Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Bourque, P. and R.E. Fairley (eds.). 2014. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. Available at: <https://www.swebok.org>
- Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research and Innovation and U.S. Department of Transportation (USDOT), SEG for ITS 1.1.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- INCOSE UK Chapter. 2010. *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook*, version 3.2, issue 1.0. Foresgate, UK: International Council on Systems Engineering (INCOSE) UK Chapter: 10, 13, 23.
- MDIT. 2008. *System Maintenance Guidebook (SMG)*, version 1.1: A companion to the systems engineering methodology (SEM) of the state unified information technology environment (SUITE). MI, USA: Michigan Department of Information Technology (MDIT), DOE G 200: 38.
- Nguyen, L. 2006. "Adapting the Vee Model to Accomplish Systems Engineering on Change Projects." Paper presented at 9th Annual National Defense Industrial Association (NDIA) Systems Engineering Conference, San Diego, CA, USA.
- OSD(AT&L). 2012. "On-line policies, procedures, and planning references." Office of the Under Secretary of Defense for Acquisition, Transportation and Logistics, US Department of Defense (DoD). Accessed on August 30, 2012. Available at: <https://www.acq.osd.mil/log/>
- Seacord, R.C., D. Plakosh, and G.A. Lewis. 2003. *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Pearson Education.
- van der Laan, J. 2008. "Life extension of Civil Infrastructural works - a systems engineering approach." Proceedings of the 18th annual International Symposium of the International Council on Systems Engineering, Utrecht, the Netherlands.

Primary References

- Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research and Innovation and U.S. Department of Transportation (USDOT), SEG for ITS 1.1.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Jackson. 2007. "A Multidisciplinary Framework for Resilience to Disasters and Disruptions." *Journal of Integrated Design and Process Science*. 11(2).
- OSD(AT&L). 2011. "Logistics and Materiel Readiness On-line policies, procedures, and planning references." Arlington, VA, USA: Office of the Under Secretary of Defense for Acquisition, Transportation and Logistics

(OUSD(AT&L)). <https://www.acq.osd.mil/log/>.

Seacord, R.C., D. Plakosh, and G.A. Lewis. 2003. *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Pearson Education.

Additional References

Braunstein, A. 2007. "Balancing Hardware End-of-Life Costs and Responsibilities." Westport, CT, USA: Experture Group, ETS 07-12-18.

Casetta, E. 2001. *Transportation Systems Engineering: Theory and methods*. New York, NY, USA: Kluwer Publishers Academic, Springer.

DAU. 2010. "Acquisition community connection (ACC): Where the DoD AT&L workforce meets to share knowledge." In Defense Acquisition University (DAU)/US Department of Defense (DoD). Ft. Belvoir, VA, USA, accessed August 5, 2010. <https://acc.dau.mil/>.

Elliot, T., K. Chen, and R.C. Swanekamp. 1998. "Section 6.5" in *Standard Handbook of Powerplant Engineering*. New York, NY, USA: McGraw Hill.

FAA. 2006. "Section 4.1" in "Systems Engineering Manual." Washington, DC, USA: US Federal Aviation Administration (FAA).

FCC. 2009. "Radio and Television Broadcast Rules." Washington, DC, USA: US Federal Communications Commission (FCC), 47 CFR Part 73, FCC Rule 09-19: 11299-11318.

Finlayson, B. and B. Herdlick. 2008. *Systems Engineering of Deployed Systems*. Baltimore, MD, USA: Johns Hopkins University: 28.

IEC. 2007. *Obsolescence Management - Application Guide*, ed 1.0. Geneva, Switzerland: International Electrotechnical Commission, IEC 62302.

IEEE. 2010. *IEEE Standard Framework for Reliability Prediction of Hardware*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1413.

IEEE. 1998. *IEEE Standard Reliability Program for the Development and Production of Electronic Systems and Equipment*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1332.

IEEE. 2008. *IEEE Recommended practice on Software Reliability*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1633.

IEEE. 2005. *IEEE Standard for Software Configuration Management Plans*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 828.

INCOSE. 2010. "In-service systems working group." San Diego, CA, USA: International Council on Systems Engineering (INCOSE).

INCOSE UK Chapter. 2010. *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook, version 3.2, issue 1.0*. Foresgate, UK: International Council on Systems Engineering (INCOSE) UK Chapter: 10, 13, 23.

Institute of Engineers Singapore. 2009. "Systems Engineering Body of Knowledge, provisional," version 2.0. Singapore.

ISO/IEC. 2003. "Industrial Automation Systems Integration-Integration of Life-Cycle Data for Process Plants including Oil, Gas Production Facilities." Geneva, Switzerland: International Organization for Standardization (ISO)/International Electro technical Commission (IEC).

Jackson, S. 2007. "A Multidisciplinary Framework for Resilience to Disasters and Disruptions." *Journal of Design and Process Science*. 11(2): 91-108, 110.

Jackson, S. 1997. *Systems Engineering for Commercial Aircraft*. Surrey, UK: Ashgate Publishing, Ltd.

- Koopman, P. 1999. "Life Cycle Considerations." In Carnegie-Mellon University (CMU). Pittsburgh, PA, USA, accessed August 5, 2010. Available at: https://www.ece.cmu.edu/~koopman/des_s99/life_cycle/index.html.
- Livingston, H. 2010. "GEB1: Diminishing Manufacturing Sources and Material Shortages (DMSMS) Management Practices." McClellan, CA, USA: Defense MicroElectronics Activity (DMEA)/U.S. Department of Defense (DoD).
- Mays, L. (ed). 2000. "Chapter 3" in *Water Distribution Systems Handbook*. New York, NY, USA: McGraw-Hill Book Company.
- MDIT. 2008. *System Maintenance Guidebook (SMG)*, version 1.1: A companion to the systems engineering methodology (SEM) of the state unified information technology environment (SUITE). MI, USA: Michigan Department of Information Technology (MDIT), DOE G 200: 38.
- NAS. 2006. *National Airspace System (NAS) System Engineering Manual*, version 3.1 (volumes 1-3). Washington, D.C., USA: Air Traffic Organization (ATO)/U.S. Federal Aviation Administration (FAA), NAS SEM 3.1.
- NASA. 2007. *Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105, December 2007.
- Nguyen, L. 2006. "Adapting the Vee Model to Accomplish Systems Engineering on Change Projects." Paper presented at 9th Annual National Defense Industrial Association (NDIA) Systems Engineering Conference, San Diego, CA, USA.
- Reason, J. 1997. *Managing the Risks of Organizational Accident*. Aldershot, UK: Ashgate.
- Ryen, E. 2008. *Overview of the Systems Engineering Process*. Bismarck, ND, USA: North Dakota Department of Transportation (NDDOT).
- SAE International. 2010. "Standards: Automotive--Maintenance and Aftermarket." Warrendale, PA, USA: Society of Automotive Engineers (SAE) International.
- Schafer, D.L. 2003. "Keeping Pace With Technology Advances When Funding Resources Are Diminished." Paper presented at Auto Test Con. IEEE Systems Readiness Technology Conference, Anaheim, CA, USA: 584.
- SEI. 2010. "Software Engineering Institute." In Software Engineering Institute (SEI)/Carnegie-Mellon University (CMU). Pittsburgh, PA, USA, accessed August 5, 2010. <https://www.sei.cmu.edu>.
- SOLE. 2009. "Applications Divisions." In The International Society of Logistics (SOLE). Hyattsville, MD, USA, accessed August 5, 2010. <https://www.sole.org/appdiv.asp>.
-

System Disposal and Retirement

- Lead Author:
- Brian Wells

Product or service disposal and retirement is an important part of system life management. At some point, any deployed system will become one of the following: uneconomical to maintain; obsolete; or unrepairable. A comprehensive systems engineering process includes an anticipated equipment phase-out period and takes disposal into account in the design and life cycle cost assessment. (See other knowledge areas in Part 3 for discussion on life cycle metrics and assessment.)

A public focus on sustaining a clean environment encourages contemporary systems engineering (SE) design to consider recycling, reuse, and responsible disposal techniques. (See Systems Engineering and Environmental Engineering for additional discussion.)

Topic Overview

According to the INCOSE Systems Engineering Handbook (2012), “The purpose of the disposal process is to remove a system element from the operation environment with the intent of permanently terminating its use; and to deal with any hazardous or toxic materials or waste products in accordance with the applicable guidance, policy, regulation, and statutes.” In addition to technological and economical factors, the system-of-interest (SoI) must be compatible, acceptable, and ultimately address the design of a system for the environment in terms of ecological, political, and social considerations.

Ecological considerations associated with system disposal or retirement are of prime importance. The most concerning problems associated with waste management include:

- Air Pollution and Control,
- Water Pollution and Control,
- Noise Pollution and Control,
- Radiation, and
- Solid Waste.

In the US, the Environmental Protection Agency (EPA) and the Occupational Safety and Health Administration (OSHA) govern disposal and retirement of commercial systems. Similar organizations perform this function in other countries. OSHA addresses hazardous materials under the 1910-119A List of Highly Hazardous Chemicals, Toxics, and Reactives (OSHA 2010). System disposal and retirement spans both commercial and government developed products and services. While both the commercial and government sectors have common goals, methods differ during the disposition of materials associated with military systems.

US DoD Directive 4160.21-M, *Defense Material Disposition Manual* (1997) outlines the requirements of the Federal Property Management Regulation (FPMR) and other laws and regulations as appropriate regarding the disposition of excess, surplus, and foreign excess personal property (FEPP). Military system disposal activities must be compliant with EPA and OSHA requirements.

Application to Product Systems

Product system retirement may include system disposal activities or preservation activities (e.g., mothballing) if there is a chance the system may be called upon for use at a later time.

Systems Engineering and Analysis has several chapters that discuss the topics of design for goals such as “green engineering,” reliability, maintainability, logistics, supportability, producibility, disposability, and sustainability. Chapter 16 provides a succinct discussion of green engineering considerations and ecology-based manufacturing. Chapter 17 discusses life cycle costing and the inclusion of system disposal and retirement costs (Blanchard and Fabrycky 2011).

Some disposal of a system's components occurs during the system's operational life. This happens when the components fail and are replaced. As a result, the tasks and resources needed to remove them from the system need to be planned well before a demand for disposal exists.

Transportation of failed items, handling equipment, special training requirements for personnel, facilities, technical procedures, technical documentation updates, hazardous material (HAZMAT) remediation, all associated costs, and reclamation or salvage value for precious metals and recyclable components are important considerations during system planning. Phase-out and disposal planning addresses when disposal should take place, the economic feasibility of the disposal methods used, and what the effects on the inventory and support infrastructure, safety, environmental requirements, and impact to the environment will be (Blanchard 2010).

Disposal is the least efficient and least desirable alternative for the processing of waste material (Finlayson and Herdlick 2008).

The EPA collects information regarding the generation, management and final disposition of hazardous wastes regulated under the Resource Conservation and Recovery Act of 1976 (RCRA). EPA waste management regulations are codified at 40 C.F.R., parts 239-282. Regulations regarding management of hazardous wastes begin at 40 C.F.R. part 260. Most states have enacted laws and promulgated regulations that are at least as stringent as federal regulations.

Due to the extensive tracking of the life of hazardous waste, the overall process has become known as the “cradle-to-grave system”. Stringent bookkeeping and reporting requirements have been levied on generators, transporters, and operators of treatment, storage, and disposal facilities that handle hazardous waste.

Unfortunately, disposability has a lower priority compared to other activities associated with product development. This is due to the fact that typically, the disposal process is viewed as an external activity to the entity that is in custody of the system at the time. Reasons behind this view include:

- There is no direct revenue associated with the disposal process and the majority of the cost associated with the disposal process is initially hidden.
- Typically, someone outside of SE performs the disposal activities. For example, neither a car manufacturer nor the car's first buyer may be concerned about a car's disposal since the car will usually be sold before disposal.

The European Union's Registration, Evaluation, Authorization, and Restriction of Chemicals (REACH) regulation requires manufacturers and importers of chemicals and products to register and disclose substances in products when specific thresholds and criteria are met (European Parliament 2007). The European Chemicals Agency (ECHA) manages REACH processes. Numerous substances will be added to the list of substances already restricted under European legislation; a new regulation emerged when the Restriction on Hazardous Substances (RoHS) in electrical and electronic equipment was adopted in 2003.

Requirements for substance use and availability are changing across the globe. Identifying the use of materials in the supply chain that may face restriction is an important part of system life management. System disposal and retirement requires upfront planning and the development of a disposal plan to manage the activities. An important consideration during system retirement is the proper planning required to update the facilities needed to support the system during retirement, as explained in the *California Department of Transportation Systems Engineering*

Guidebook (2005).

Disposal needs to account for environmental and personal risks associated with the decommissioning of a system and all hazardous materials involved. The decommissioning of a nuclear power plant is a prime example of hazardous material control and exemplifies the need for properly handling and transporting residual materials resulting from the retirement of certain systems.

The US Defense Logistics Agency (DLA) is the lead military agency responsible for providing guidance for worldwide reuse, recycling, and disposal of military products. A critical responsibility of the military services and defense agencies is demilitarization prior to disposal.

Application to Service Systems

An important consideration during service system retirement or disposal is the proper continuation of services for the consumers of the system. As an existing service system is decommissioned, a plan should be adopted to bring new systems online that operate in parallel with the existing system so that service interruption is kept to a minimum. This parallel operation needs to be carefully scheduled and can occur over a significant period of time.

Examples of parallel operation include phasing in new Air Traffic Control (ATC) systems (FAA 2006), the migration from analog television to new digital television modulation (FCC 2009), the transition to Internet protocol version 6 (IPv6), maintaining water handling systems, and maintaining large commercial transportation systems, such as rail and shipping vessels.

The *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)* provides planning guidance for the retirement and replacement of large transportation systems. Chapter 4.7 identifies several factors which can shorten the useful life of a transportation system and lead to early retirement, such as the lack of proper documentation, the lack of effective configuration management processes, and the lack of an adequate operations and maintenance budget (Caltrans, and USDOT 2005).

Application to Enterprises

The disposal and retirement of large enterprise systems requires a phased approach, with capital planning being implemented in stages. As in the case of service systems, an enterprise system's disposal and retirement require parallel operation of the replacement system along with the existing (older) system to prevent loss of functionality for the user.

Other Topics

See the OSHA standard (1996) and EPA (2010) website for references that provide listings of hazardous materials. See the DLA Disposal Services website ^[1] for disposal services sites and additional information on hazardous materials.

Practical Considerations

A prime objective of systems engineering is to design a product or service such that its components can be recycled after the system has been retired. The recycling process should not cause any detrimental effects to the environment.

One of the latest movements in the industry is green engineering. According to the EPA, green engineering is the design, commercialization, and use of processes and products that are technically and economically feasible while minimizing:

- the generation of pollutants at the source; and
- the risks to human health and the environment.

See *Systems Engineering and Environmental Engineering* for additional information.

References

Works Cited

- Blanchard, B. S. 2010. *Logistics Engineering and Management*, 5th ed. Englewood Cliffs, NJ, USA: Prentice Hall, 341-342.
- Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research and Innovation and U.S. Department of Transportation (USDOT), SEG for ITS 1.1.
- DoD. 1997. *Defense Materiel Disposition Manual*. Arlington, VA, USA: US Department of Defense, DoD 4160.21-M.
- DLA. 2010. "Defense logistics agency disposition services." In Defense Logistics Agency (DLA)/U.S. Department of Defense [database online]. Battle Creek, MI, USA, accessed June 19 2010: 5. Available at: <https://www.dtc.dla.mil>.
- ECHA. 2010. "European Chemicals Agency (ECHA)." In European Chemicals Agency (ECHA). Helsinki, Finland. Available at: https://echa.europa.eu/home_en.asp.
- EPA. 2010. "Wastes In U.S. Environmental Protection Agency (EPA)." Washington, D.C., USA. Available at: <https://www.epa.gov/epawaste/index.htm>.
- European Parliament. 2007. Regulation (EC) no 1907/2006 of the European parliament and of the council of 18 December 2006 concerning the registration, evaluation, authorisation and restriction of chemicals (REACH), establishing a European chemicals agency, amending directive 1999/45/EC and repealing council regulation (EEC) no 793/93 and commission regulation (EC) no 1488/94 as well as council directive 76/769/EEC and commission directives 91/155/EEC, 93/67/EEC, 93/105/EC and 2000/21/EC. Official Journal of the European Union 29 (5): 136/3,136/280.
- FAA. 2006. "Section 4.1" in "Systems Engineering Manual." Washington, DC, USA: US Federal Aviation Administration (FAA).
- FCC. 2009. "Radio and Television Broadcast Rules." Washington, DC, USA: US Federal Communications Commission (FCC), 47 CFR Part 73, FCC Rule 09-19: 11299-11318.
- Finlayson, B. and B. Herdlick. 2008. *Systems Engineering of Deployed Systems*. Baltimore, MD, USA: Johns Hopkins University: 28.
- OSHA. 1996. "Hazardous Materials: Appendix A: List of Highly Hazardous Chemicals, Toxics and Reactives." Washington, DC, USA: Occupational Safety and Health Administration (OSHA)/U.S. Department of Labor (DoL), 1910.119(a).
- Resource Conservation and Recovery Act of 1976*, 42 U.S.C. §6901 et seq. (1976)

Primary References

- Blanchard, B.S. and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Caltrans and USDOT. 2005. *Systems Engineering Guidebook for Intelligent Transportation Systems (ITS)*, version 1.1. Sacramento, CA, USA: California Department of Transportation (Caltrans) Division of Research and Innovation and U.S. Department of Transportation (USDOT), SEG for ITS 1.1.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE),

INCOSE-TP-2003-002-03.2.2.

Jackson, S. 2007. "A Multidisciplinary Framework for Resilience to Disasters and Disruptions." *Journal of Integrated Design and Process Science*. 11(2).

OUSD(AT&L). 2011. "Logistics and Materiel Readiness On-line policies, procedures, and planning references." Arlington, VA, USA: Office of the Under Secretary of Defense for Acquisition, Transportation and Logistics (OUSD(AT&L)). <https://www.acq.osd.mil/log/>.

Seacord, R.C., D. Plakosh, and G.A. Lewis. 2003. *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Pearson Education.

Additional References

Blanchard, B. S. 2010. *Logistics Engineering and Management*, 5th ed. Englewood Cliffs, NJ, USA: Prentice Hall, 341-342.

Casetta, E. 2001. *Transportation Systems Engineering: Theory and methods*. New York, NY, USA: Kluwer Publishers Academic, Springer.

DAU. 2010. "Acquisition community connection (ACC): Where the DoD AT&L workforce meets to share knowledge." In Defense Acquisition University (DAU)/US Department of Defense (DoD). Ft. Belvoir, VA, USA, accessed August 5, 2010. <https://acc.dau.mil/>.

DLA. 2010. "Defense logistics agency disposition services." In Defense Logistics Agency (DLA)/U.S. Department of Defense [database online]. Battle Creek, MI, USA, accessed June 19 2010: 5. Available at: <https://www.dtc.dla.mil>.

ECHA. 2010. "European Chemicals Agency (ECHA)." In European Chemicals Agency (ECHA). Helsinki, Finland. Available at: https://echa.europa.eu/home_en.asp.

Elliot, T., K. Chen, and R.C. Swanekamp. 1998. "Section 6.5" in *Standard Handbook of Powerplant Engineering*. New York, NY, USA: McGraw Hill.

EPA. 2010. "Wastes In U.S. Environmental Protection Agency (EPA)." Washington, D.C., USA. Available at: <https://www.epa.gov/epawaste/index.htm>.

European Parliament. 2007. Regulation (EC) no 1907/2006 of the European parliament and of the council of 18 December 2006 concerning the registration, evaluation, authorisation and restriction of chemicals (REACH), establishing a European chemicals agency, amending directive 1999/45/EC and repealing council regulation (EEC) no 793/93 and commission regulation (EC) no 1488/94 as well as council directive 76/769/EEC and commission directives 91/155/EEC, 93/67/EEC, 93/105/EC and 2000/21/EC. Official Journal of the European Union 29 (5): 136/3,136/280.

FAA. 2006. "Section 4.1" in "Systems Engineering Manual." Washington, DC, USA: US Federal Aviation Administration (FAA).

FCC. 2009. "Radio and Television Broadcast Rules." Washington, DC, USA: US Federal Communications Commission (FCC), 47 CFR Part 73, FCC Rule 09-19: 11299-11318.

Finlayson, B. and B. Herdlick. 2008. *Systems Engineering of Deployed Systems*. Baltimore, MD, USA: Johns Hopkins University: 28.

FSA. 2010. "Template for 'System Retirement Plan' and 'System Disposal Plan'." In Federal Student Aid (FSA)/U.S. Department of Education (DoEd). Washington, DC, USA. Accessed August 5, 2010. Available at: <https://federalstudentaid.ed.gov/business/lcm.html>.

IEEE 2005. *IEEE Standard for Software Configuration Management Plans*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 828.

- Ihii, K., C.F. Eubanks, and P. Di Marco. 1994. "Design for Product Retirement and Material Life-Cycle." *Materials & Design*. 15(4): 225-33.
- INCOSE. 2010. "In-service systems working group." San Diego, CA, USA: International Council on Systems Engineering (INCOSE).
- INCOSE UK Chapter. 2010. *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook, version 3.2*, issue 1.0. Foresgate, UK: International Council on Systems Engineering (INCOSE) UK Chapter: 10, 13, 23.
- Institute of Engineers Singapore. 2009. "Systems Engineering Body of Knowledge, provisional," version 2.0. Singapore: Institute of Engineers Singapore.
- Mays, L. (ed). 2000. "Chapter 3" in *Water Distribution Systems Handbook*. New York, NY, USA: McGraw-Hill Book Company.
- MDIT. 2008. *System Maintenance Guidebook (SMG)*, version 1.1: A companion to the systems engineering methodology (SEM) of the state unified information technology environment (SUITE). MI, USA: Michigan Department of Information Technology (MDIT), DOE G 200: 38.
- Minneapolis-St. Paul Chapter of SOLE. 2003. "Systems Engineering in Systems Deployment and Retirement, presented to INCOSE." Minneapolis-St. Paul, MN, USA: International Society of Logistics (SOLE), Minneapolis-St. Paul Chapter.
- NAS. 2006. *National Airspace System (NAS) System Engineering Manual*, version 3.1 (volumes 1-3). Washington, D.C., USA: Air Traffic Organization (ATO)/U.S. Federal Aviation Administration (FAA), NAS SEM 3.1.
- NASA. 2007. *Systems Engineering Handbook*. Washington, DC, USA: National Aeronautics and Space Administration (NASA), NASA/SP-2007-6105, December 2007.
- OSHA. 1996. "Hazardous Materials: Appendix A: List of Highly Hazardous Chemicals, Toxics and Reactives." Washington, DC, USA: Occupational Safety and Health Administration (OSHA)/U.S. Department of Labor (DoL), 1910.119(a).
- Ryen, E. 2008. *Overview of the Systems Engineering Process*. Bismarck, ND, USA: North Dakota Department of Transportation (NDDOT).
- SAE International. 2010. "Standards: Automotive--Maintenance and Aftermarket." Warrendale, PA, USA: Society of Automotive Engineers (SAE) International.
- Schafer, D.L. 2003. "Keeping Pace With Technology Advances When Funding Resources Are Diminished." Paper presented at Auto Test Con. IEEE Systems Readiness Technology Conference, Anaheim, CA, USA: 584.
- SOLE. 2009. "Applications Divisions." In The International Society of Logistics (SOLE). Hyattsville, MD, USA, accessed August 5, 2010. <https://www.sole.org/appdiv.asp>.

References

[1] <https://www.dtc.dla.mil>

Part 3d: Relevant Standards

Knowledge Area: Systems Engineering Standards

Systems Engineering Standards

Contents of this Knowledge Area

- Why Standards? (Garry Roedler) (Bill Bearden, David Endler, and Mike Yokell)
- Evolution of Systems Engineering Standards (Garry Roedler) (Bill Bearden, David Endler, and Mike Yokell)
- Systems Engineering Related Standards Landscape (Garry Roedler) (Bill Bearden, David Endler, and Mike Yokell)
- Harmonization of Systems Engineering Standards (Garry Roedler) (William Bearden, David Endler, and Mike Yokell)
- Application of Systems Engineering Standards (Garry Roedler) (William Bearden, David Endler, and Mike Yokell)
- Lead Author:
- Garry Roedler
- Contributing Authors:
- Bill Bearden, David Endler, and Mike Yokell

This knowledge area (KA) focuses on the standards and key references (hereafter referred to as standards) that are relevant to systems engineering. This KA examines the importance of standards, the types of standards, some of the key standards, the alignment efforts to achieve a consistent set of standards, and the suitable application of these standards. Note that many of these standards have been used as references throughout Part 3.

See the article Matrix of Implementation Examples for a mapping of case studies and vignettes included in Part 7 to topics covered in Part 3.

References

None.

Why Standards?

- Lead Author:
- Garry Roedler
- Contributing Authors:
- Bill Bearden, David Endler, and Mike Yokell

There are many standards and guides across the industry that are related to Systems Engineering. A common question that comes up when considering the use of standards and guides is “Why Standardize?.” This article addresses this question. This article considers standards as including formal documents that are developed by a Standards Development Organization (SDO) (such as ISO or IEEE), de facto standards that are generally accepted works, bodies of knowledge, and key guides that provide a consensus view. A list of the types of standards is provided in Systems Engineering Related Standards Landscape, Standards Taxonomies and Types of Standards, Table 1.

Why Standardize? The Need and Challenges for Standards in SE

ISO defines a standard as “a document, established by consensus and approved by a recognized body, that provides rules, guidelines, or characteristics for activities or their results.” Standards can help ensure quality, consistency, safety, efficiency, and effectiveness for products and projects throughout the life cycles, leading to improved communication, reduced errors, and faster cycle times. Standardization is the process of establishing standards. (Example: ISO, IEC, IEEE and INCOSE were among the organizations that collaborated to standardize the technical content in the ISO/IEC/IEEE 15288 standard.) Although there are many benefits from standardization, there are also some potential limitations or hinderances that can come from standardization. Table 1 provides a look at the Pros and Cons of standardization. For most projects and organizations, the Pros tend to outweigh the Cons. The use of the term standardization in this KA includes the definition, adaptation, tailoring, assessment and improvement of standards.

Table 1. Pros and Cons of SE Standardization (G. Roedler, Used with Permission)

Focus	Pros	Cons
Timing & Effectiveness	Can incorporate broad lessons as they are learned as well as proven practices. Can help ensure technologies are not only effective, but also secure and safe for users.	Potential immaturity of a topic for standardization; i.e., standard can be developed too early, prior to having evidence of effectiveness or efficiency. It can take a long time to create an effective standard; building consensus and collecting evidence can take time.
Innovation	Can provide a baseline for tool development, methods and applications allowing engineers to focus on innovation for new solutions, rather than reinventing the wheel for performing basic tasks on projects. As innovation occurs, the standardization helps to spread the knowledge.	Can somewhat stifle innovation when new approaches are needed due to novel situations.
Quality and Consistency	Standardized processes and system elements can help ensure that products and systems are built consistently, reducing unproductive variations and improving overall quality. Process standards help ensure consistency in the performance of processes and achieving their desired outcomes.	In some cases, standardized processes can challenge responsiveness or agility.
Communication & Collaboration	Can help converge practices, which aids communication, teaming/collaboration, knowledge management. Standardized processes, practices, formats, symbols, and conventions can facilitate clear and efficient communication among engineers and other stakeholders.	Can drive communication paths or workflows that are not optimal.

Efficiency & Productivity	Standardized designs and processes can promote faster product development cycles, as engineers can reuse existing, pre-approved elements and methods. Streamlined processes can improve efficiency by reducing waste: time and resources. Time and resources then can be focused on the solution or business requirements rather than on how to perform the engineering.	All projects are not the same. So, if tailoring for the project specific characteristics is not done, it can lead to inefficiencies, which can be more pronounced for projects with high levels of uncertainty and/or change..
Knowledge Management	Standards can help to document lessons learned for processes and products. For engineering, standards can help provide the organizational memory that could otherwise be lost when people leave an organization or project. Documenting knowledge assets using standards or standard references can improve searches for specific information.	When there is a high likelihood of change (technology, processes, environments, etc.), especially in short intervals, standards as a means of capturing the “memory” can lead to obsolescence of the knowledge assets shortly after the documentation.
Costs	Standardization can lead to lower material costs for product lines and designs with more stability. Standardization can also help reduce the cost of training, tools, and knowledge transfer.	When there is a high likelihood of change, standardization will not necessarily reduce the cost of materials, training, etc. If the change has not been accounted for due to the standardization, there can be an additional level of resistance to overcome. Adaptability becomes more important.
Scalability	Easier to scale engineering processes and services, as common practices and tools can be reused across multiple projects.	
Interoperability	Standardization can help ensure that different systems and elements can work together, promoting interoperability and reducing integration challenges.	

Considerations for Standardization

Implementing Standards With a Full Life Cycle Approach:

When an organization or project implements a standard, it should be a full life cycle approach with respect to the standard in order to gain the greatest benefit. This includes the evaluation of alternative standards, selection of standards, adaptation or tailoring, assessment of results from applying the standards, and improvement actions. ISO/IEC/IEEE 24748-1 and ISO/IEC/IEEE 24748-2 provide guidance on life cycle management, process tailoring, and other life cycle process topics that can help in the implementation of standards.

Assessing the Readiness of a Technical Topic for the Definition or Usage of Standards:

For the definition of a new topic for standardization, it is beneficial to assess the readiness for standardization based on the maturity of the concepts, convergence of the principles and theory, acceptance of proven practices, and industry consensus on the topic.

It is also useful to assess the relevance and maturity of a standard with respect to the project application of the topic of the standard. For example, if the project is using a technology covered by a standard, but the project’s implementation of the technology is beyond that of the standard, then it could be an impediment to try to use the standard.

Agility and Standardization:

These two things are not necessarily counter to each other. Engineering organizations or projects can tailor and adapt the standards, such as ISO/IEC/IEC 15288, to help define their processes and approaches considering agility necessary for situations with more uncertainty and likelihood for change (See ISO/IEC/IEEE 24748-10, Guidelines for SE Agility). A paper from the NDIA SED and INCOSE titled “ISO/IEC/IEEE 15288 Meets Lean Agile.” details the use of 15288 in an agile approach in the defense environment. However, much of the content applies to projects

and systems in any environment. It describes the use of 15288 in a lean agile approach focusing on lean agile principles, such as customer centricity, systems thinking, multiple horizons of planning, assuming variability and preserving options, iterative development with incremental deliveries and rapid feedback loops, data-driven decisions, decentralized decisions, and more. ISO/IEC/IEEE 24748-1 and ISO/IEC/IEEE 24748-2 provide guidance on life cycle management, process tailoring, and other life cycle process topics that can help in adapting standards for project specific needs, such as agility.

The Balance Between Standardization and Project Specific Needs:

For standards to be most effective, there is usually a need to determine the project specific needs and tailor the standardized elements (process, design, tools, methods, etc.) to best meet the needs of the project. Projects are unique; copy and paste or boilerplates often don't work because the context and many variables are different. However, it is also important to avoid the "not invented here" mindset for the use standards. It is important to find the right balance when many organizations are involved in the project.

Perspectives from key stakeholders

The following are a couple quotes regarding the importance and use of standards from ISO and IEEE, two key engineering standards development organizations (SDOs).

Per ISO:

Standards ensure consistency of essential features of goods and services, such as quality, ecology, safety, economy, reliability, compatibility, interoperability, efficiency and effectiveness.

Standards codify ... technology and facilitate its transfer. Standards are therefore an invaluable source of knowledge.

(https://www.iso.org/sites/ConsumersStandards/1_standards.html#:~:text=How%20do%20standards%20help?,are%20aligned%20in%20this%20way.20250331)

Per IEEE:

Standards help fuel compatibility and interoperability and simplifies product development, and speeds time-to-market. Standards also make it easier to understand and compare competing products. As standards are globally adopted and applied in many markets, they also fuel international trade.

(<https://standards.ieee.org/beyond-standards/what-are-standards-why-are-they-important/#:~:text=This%20helps%20fuel%20compatibility%20and,they%20also%20fuel%20international%20trade.20250331>)

And finally, a quote the sums up the importance from a government project and contractor perspective:

"Technical standards provide the corporate process memory needed for a disciplined systems engineering approach and help ensure that the government and its contractors understand the critical processes and practices necessary to take a system from design to production, and through sustainment."
(Welby, S. 2013)

References

Works Cited

Roedler, G. 2023. "SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.

https://www.iso.org/sites/ConsumersStandards/1_standards.html#:~:text=How%20do%20standards%20help?,are%20aligned%20in%20this%20way.20250331

<https://standards.ieee.org/beyond-standards/what-are-standards-why-are-they-important/#:~:text=This%20helps%20fuel%20compatibility%20and,they%20also%20fuel%20international%20trade.20250331>

Welby, S. 2013. Guest Editorial, M&S Journal, Vol. 8, No. 1 (Spring 2013), Modeling and Simulation Coordination

NDIA. 2025. "ISO/IEC/IEEE 15288 Meets Lean Agile: Integrating Lean and Agile Principles with Systems Engineering Processes for Modern Defense Acquisition". National Defense Industrial Association (NDIA), Arlington, VA. May 2025.

INCOSE. 2023. INCOSE Systems Engineering Handbook 5th Edition. International Council on Systems Engineering. Hoboken, NJ, USA: John Wiley & Sons. INCOSE SE Handbook.

ISO/IEC/IEEE. 2023. Systems and Software Engineering -- System Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

ISO/IEC/IEEE. 2024. "Systems and Software Engineering -- Life Cycle Management – Part 1: Guidelines for life cycle management,". Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.

ISO/IEC/IEEE. 2024. Systems and Software Engineering -- Life Cycle Management – Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes). Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-2:2024.

ISO/IEC/IEEE. 2026. Systems and Software Engineering -- Life Cycle Management – Part 10: Guidelines for systems engineering agility. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-10:2026.

Primary References

Roedler, G. 2023. "SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.

NDIA. 2025. "ISO/IEC/IEEE 15288 Meets Lean Agile: Integrating Lean and Agile Principles with Systems Engineering Processes for Modern Defense Acquisition".

INCOSE. 2023. INCOSE Systems Engineering Handbook 5th Edition. International Council on Systems Engineering. Hoboken, NJ, USA: John Wiley & Sons. INCOSE SE Handbook.

ISO/IEC/IEEE. 2023. Systems and Software Engineering -- System Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288: 2023.

ISO/IEC/IEEE. 2024. "Systems and Software Engineering -- Life Cycle Management – Part 1: Guidelines for life cycle management,". Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE

24748-1:2024.

ISO/IEC/IEEE. 2024. Systems and Software Engineering -- Life Cycle Management – Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes). Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-2:2024.

ISO/IEC/IEEE. 2026. Systems and Software Engineering -- Life Cycle Management – Part 10: Guidelines for systems engineering agility. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-10:2026.

ISO/IEC/IEEE. 2017. Systems and Software Engineering - Vocabulary (SEVocab). Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2017.

Additional References

None.

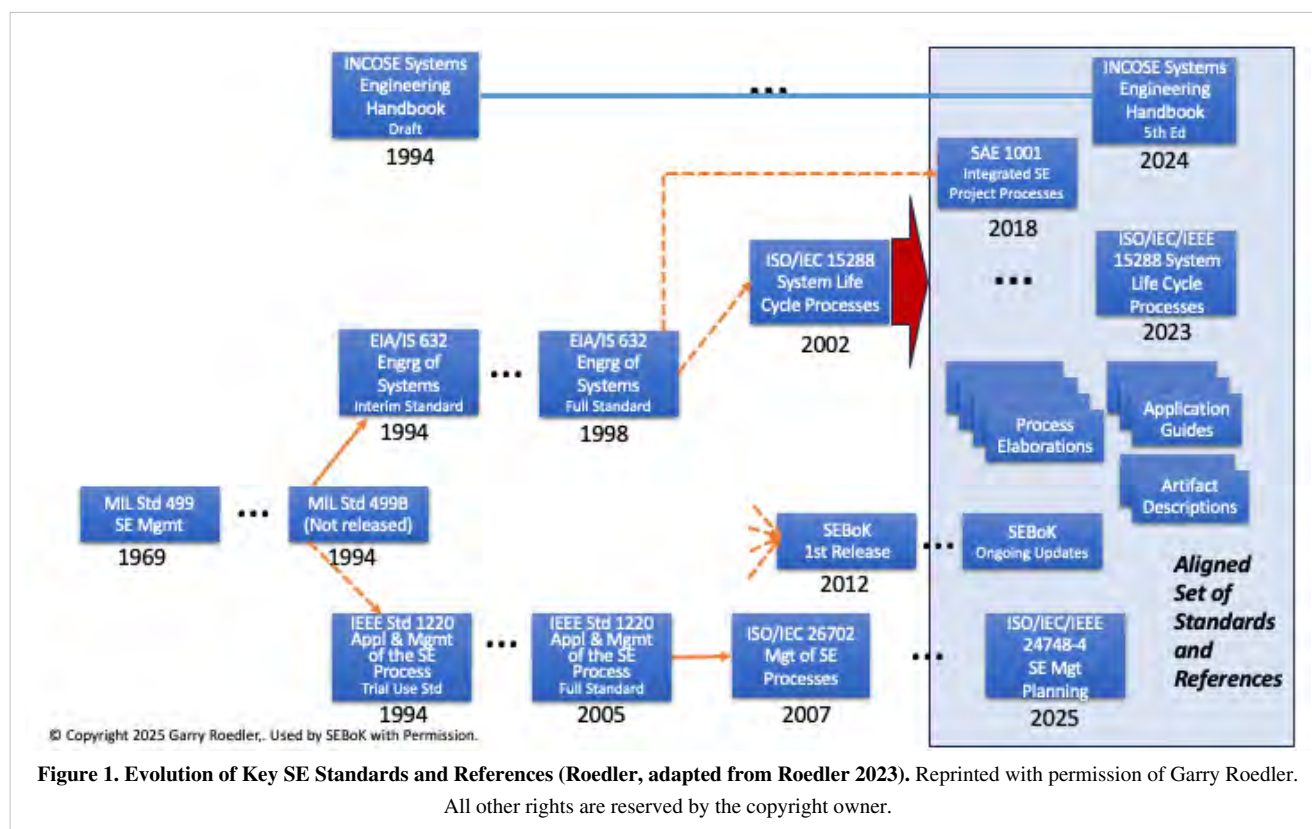
Evolution of Systems Engineering Standards

- Lead Author:
- Garry Roedler
- Contributing Authors:
- Bill Bearden, David Endler, and Mike Yokell

As stated in the article Why Standards, the development of a standard is performed to establish “rules, guidelines, or characteristics for activities or their results”. As the discipline of Systems Engineering evolved, so did the need to standardize the knowledge and practices. This article discusses the journey to develop relevant, useful standards for systems and systems engineering.

Evolution of SE Standards

Systems engineering standards have evolved as the SE discipline has matured and evolved. These references have continued to incorporate the lessons learned and changes in approaches and methods to ensure they reflect current, proven practices. Along this evolution path, there has been collaboration to help align the references across the industry associations and standards development organizations (more is discussed on this in Harmonization of Systems Engineering Standards). Figure 1 provides a graphical view of the evolution of a subset of the key SE related standards.



Until the early 1990's, MIL-STD-499 (initially released in 1969) and subsequent revisions were the first and only SE standards developed. In 1994, it transitioned to an industry standard, EIA/IS 632. Around the same time, INCOSE started development of its Systems Engineering Handbook (SEH). Figure 1 shows the ongoing evolution of these standards, as well as the publication of ISO/IEC 15288, which later also became a joint standard with the IEEE. With the dramatic uptake of 15288, the latest revision of ANSI/EIA 632 was fully aligned with 15288 with a scope change and published as SAE 1001. Finally, the SEBoK, started its development in 2009 and was first published in 2012, as a broader reference that serves as a guide to the SE Body of Knowledge. The following paragraphs provide a little more detail about the evolution of some of the SE standards.

Early Roots: MIL-STD 499 and Its Influence

The U.S. Air Force published MIL-STD-499 ("System Engineering Management") in 1969 to guide program managers and contractors on systems engineering planning, processes, tailoring, documentation, and technical oversight. In 1974, the standard was updated to MIL-STD-499A ("Engineering Management"), which broadened its guidance. A draft MIL-STD-499B was developed later to further modernize SE concepts, but it was never formally released. Over time, due to defense acquisition reform and policy changes, many U.S. military standards, including 499A, were deprecated or withdrawn. Thus, the MIL-STD lineage provided an early, structured, contract-oriented, and fairly prescriptive basis for systems engineering, especially in defense domains.

Emergence of Commercial / Process-Based Standards: ANSI/EIA-632 and IEEE 1220

After U.S. defense policy shifted away from mandating military standards, standards development responsibility increasingly moved to industry and voluntary bodies.

In 1994, the EIA (Electronic Industries Association), with the help of INCOSE and the DoD community, produced an interim standard EIA/IS-632 which closely mirrored MIL-STD-499B to describe a systems engineering process more suitable for commercial and defense applications alike.

That interim standard was developed into the full ANSI/EIA-632 (“Processes for Engineering a System”). ANSI/EIA-632 defined 13 processes across categories (technical, management, acquisition, supply), focusing on lifecycle engineering of systems rather than prescribing methods. It was intended to support enterprise-level systems as well as product development.

Independently, IEEE developed a systems engineering standard, IEEE 1220 (“Application and Management of the Systems Engineering Process”), first published in 1995 as a trial-use standard to specify application and management of systems engineering processes in engineering organizations. Later versions (e.g. IEEE 1220-2005) were revised to better align with ISO/IEC 15288, with a slightly narrower or more development-focused scope. In effect, IEEE 1220 can be viewed as an application of ANSI/EIA-632 concepts within the electrical/electronic and systems engineering domains, refined for industry use.

In 2007, a joint project between the IEEE, ISO/IEC, and INCOSE worked to move the SE management content of IEEE 1220-2005 into the new ISO/IEC/IEEE 26702 standard (“Management of Systems Engineering Processes”) and ensured that the process content was adequately included in ISO/IEC/IEEE 15288:2008.

These two parallel paths (ANSI/EIA-632 as a broad, enterprise-level systems process standard, and IEEE 1220 as a complementary development-oriented standard) contributed to the discourse on how to specify SE processes more generically, flexibly, and applicable across domains.

Harmonization and Creation of ISO/IEC/IEEE 15288

In the mid-1990s, there was growing recognition of the need for a common, internationally accepted, life-cycle process standard for systems engineering that integrates hardware, software, and human/system concerns. ISO/IEC JTC1/SC7 (the software/systems engineering committee) initiated work (from about 1994 onward) to define such a standard, leveraging contributions from IEEE, INCOSE, EIA, and ISO national bodies. The goal was to provide a process framework, common vocabulary, life-cycle management approach, and enable alignment across systems and software standards.

The first edition of ISO/IEC 15288 was published on 1 November 2002. In 2004, IEEE adopted it as IEEE 15288, fostering dual branding and further adoption. Subsequent revisions were published: 2008 (aligned with 12207), 2015 (joint ISO/IEC/IEEE version), and 2023 (latest). The standard today defines 30 processes (grouped into Organizational Project-Enabling Processes, Agreement Processes, Technical Management Processes, and Technical Processes) with purpose, outcomes, activities, and tasks for each process. The expectation within the standards community is that ISO/IEC/IEEE 15288 will continue to be the most relevant SE standard. And many other standards and references have adopted its process framework or been developed to elaborate on specific processes, focus on specific applications, or provide a technical basis for systems concepts. More information about the evolving standards taxonomy is provided in the article Systems Engineering Related Standards Landscape. Also, more information on the harmonization of the SE standards is provided in the article Harmonization of Systems Engineering Standards.

Overview of the Results of the SE Standards Evolution and Harmonization

Figure 2 provides a view of the SE standards evolution and harmonization depicting a tree whose roots are the standards from which ISO/IEC/IEEE 15288 originated. ISO/IEC/IEEE 15288 is the trunk of the tree and the branches reflect development or revisions of other works as a result of harmonization with ISO/IEC/IEEE 15288. More information on the details of the harmonization efforts is provided in SEBoK article Harmonization of Systems Engineering Standards. Also, the article Systems Engineering Related Standards Landscape provides a large listing of the SE standards and references that are in use today.

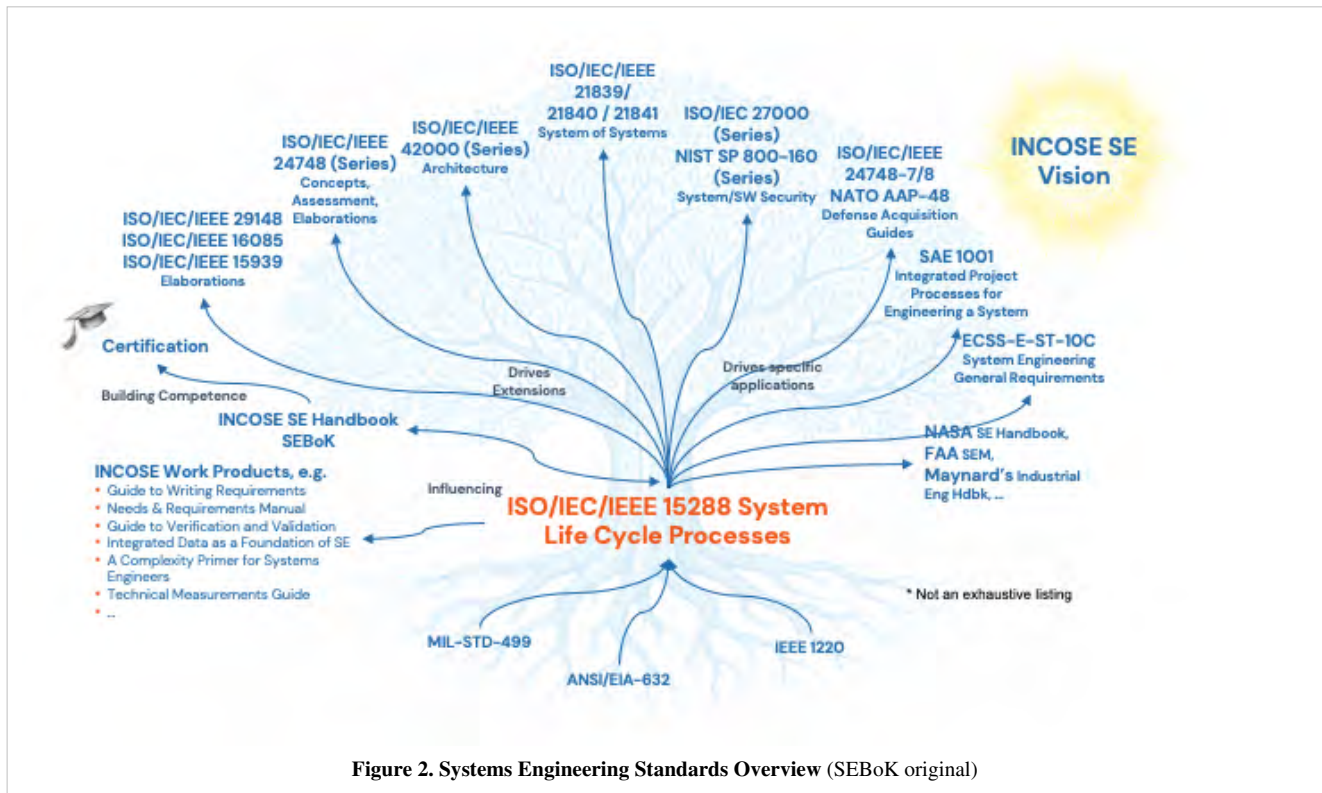


Figure 2. Systems Engineering Standards Overview (SEBoK original)

References

Works Cited

- Roedler, G. 2023. "SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.
- Roedler, G. 2015. "Evolution of SE Standards and Practices – ISO/IEC/IEEE 15288 Based Harmonization." National Defense Industrial Association (NDIA) Conference, Springfield, VA, USA.
- ANSI/EIA. 1998. Processes for Engineering a System. Warrendale, PA, USA: SAE International. ANSI/EIA 632.
- IEEE. 2005. Application and Management of the Systems Engineering Process. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). IEEE 1220-2005
- INCOSE. 2023. INCOSE Systems Engineering Handbook 5th Edition. International Council on Systems Engineering. Hoboken, NJ, USA: John Wiley & Sons. INCOSE SE Handbook.
- ISO/IEC. 2007. Management of Systems Engineering Processes. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 26702:2007.
- ISO/IEC/IEEE. 2023. Systems and Software Engineering -- System Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

ISO/IEC/IEEE. 2026. Systems and Software Engineering -- Life Cycle Management – Part 4: Systems engineering management planning. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-4:2026.

SAE. 2018. Integrated Project Processes for Engineering a System. Warrendale, PA, USA: SAE International. SAE 1001.

US DoD. 1974. Military standard: System engineering management (MIL-STD-499A). Washington, DC: Department of Defense. MIL-STD 499A.

Primary References

Roedler, G. 2023. "SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.

Roedler, G. 2015. "Evolution of SE Standards and Practices – ISO/IEC/IEEE 15288 Based Harmonization." National Defense Industrial Association (NDIA) Conference, Springfield, VA, USA.

Roedler, G; Shaw, B.; Davis, D. 2015. "Extending Industry Standards to Meet the Systems Engineering Needs of Defense Programs." Defense Standardization Program Journal, Arlington, VA. March 2015.

ANSI/EIA. 1998. Processes for Engineering a System. Warrendale, PA, USA: SAE International. ANSI/EIA 632.

IEEE. 2005. Application and Management of the Systems Engineering Process. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). IEEE 1220-2005

INCOSE. 2023. INCOSE Systems Engineering Handbook 5th Edition. International Council on Systems Engineering. Hoboken, NJ, USA: John Wiley & Sons. INCOSE SE Handbook.

ISO/IEC. 2007. Management of Systems Engineering Processes. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 26702:2007.

ISO/IEC/IEEE. 2026. Systems and Software Engineering -- Software Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 12207: 2026.

ISO/IEC/IEEE. 2023. Systems and Software Engineering -- System Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288: 2023.

ISO/IEC/IEEE. 2024. "Systems and Software Engineering -- Life Cycle Management – Part 1: Guidelines for life cycle management,". Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.

ISO/IEC/IEEE. 2024. Systems and Software Engineering -- Life Cycle Management – Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes). Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-2:2024.

ISO/IEC/IEEE. 2026. Systems and Software Engineering -- Life Cycle Management – Part 4: Systems engineering management planning. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-4:2026.

ISO/IEC/IEEE. 2017. Systems and Software Engineering - Vocabulary (SEVocab). Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2017.

SAE. 2018. Integrated Project Processes for Engineering a System. Warrendale, PA, USA: SAE International. SAE 1001.

US DoD. 1974. Military standard: System engineering management (MIL-STD-499A). Washington, DC: Department of Defense. MIL-STD 499A.

Additional References

None.

Systems Engineering Related Standards Landscape

- Lead Author:
- Garry Roedler
- Contributing Authors:
- Bill Bearden, David Endler, and Mike Yokell

There are a multitude of standards across a number of standards development organizations (SDOs) that are related to systems engineering and systems domains. This topic examines the types of standards and provides a summary of the relevant standards for systems engineering (SE).

Standards Taxonomies and Types of Standards

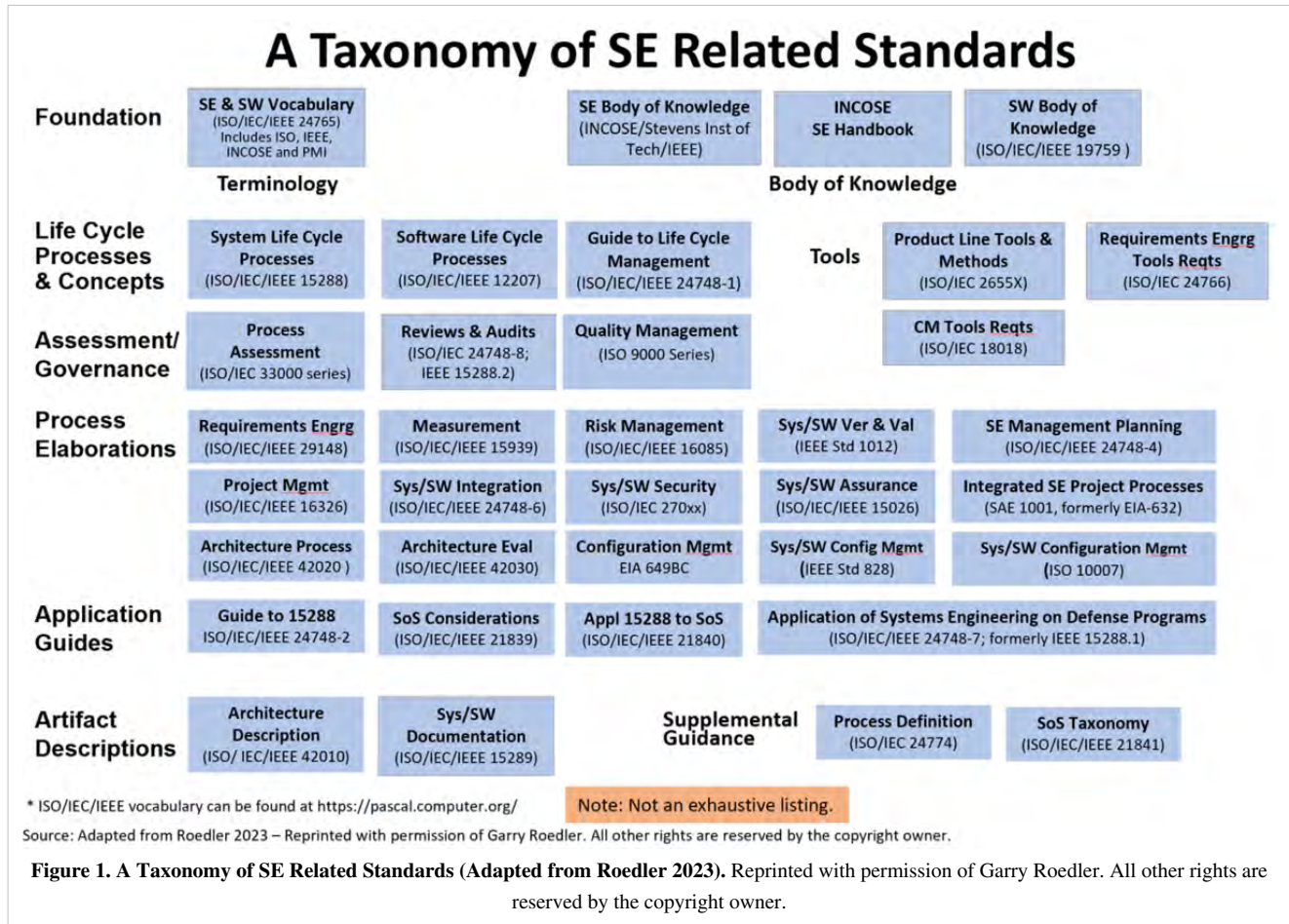
There are many types of standards that focus on different aspects of SE. Table 1 provides the types of the current standards and a description of each type.

Table 1. Types of Systems Engineering Standards. (SEBoK Original)

Standard Type	Description of Type
Concepts and Terminology	• Defines the terminology and describes the concepts of a specific domain.
Process	• Defines a specific process or set of processes, giving normative requirements for the essential elements of the process or processes. • It can give guidance to the requirements.
Requirements	• Defines the requirements for something. • Most often used for actions, activities, or practices and not objects (see specifications).
Procedure (Practice, Activity)	• Defines instructions, approach, or requirements on how to do something. • Sometimes a description of best practices.
Application Guides / Guidance	• Defines guidelines or provides interpretation of a published standard.
Management System	• Defines requirements for management of a specific focus area.
Specification	• Defines, describes, or identifies something precisely, including its requirements. • Usually defines requirements, design, and functionality of a product, system, or project and is normative.
Artifact Description	• Defines the form, attributes, content requirements, or properties of an artifact.
Reference Model	• Defines an abstract or conceptual framework that provides a structured means to understand and communicate a system, architecture or element. • Often used as a mechanism to gain commonality and interoperability.
Process Reference Model (PRM)	• Defines a collection of processes necessary and sufficient to achieve a nominated business outcome.
Process Assessment Model (PAM)	• Defines requirements and guidance for assessing attributes of nominated processes or attributes of a nominated collection of processes.

Guide to Body of Knowledge • Defines a collection and description of the current knowledge sources in a domain, or a guide to knowledge (BOK) sources that comprise the body of knowledge.

Additionally, a taxonomy of the SE related standards can help to understand the available set of standards. A standards taxonomy is defined as an orderly classification of standards according to their attributes and relationships. Figure 1 provides an example of a taxonomy for the SE related standards.



Description of the SE Standards Categories Presented in the Sample Taxonomy

Figure 1 illustrates the focus and level of detail for many of the SE related standards. This figure shows that there are different purposes or objectives for these knowledge assets. The following provides a brief description of the objective for each category in the sample taxonomy. Each of the categories provides specific utility to the SE stakeholders.

Foundation – Vocabulary: This provides a common set of terminology and definitions from across a wide range of standards and references for systems engineering. It includes the vocabulary from ISO/IEC JTC1/SC7, IEEE, INCOSE, and PMI. The vocabulary is both published in ISO/IEC/IEEE 24765, Vocabulary and in an online resource called SEVOCAB at <https://pascal.computer.org/>.

Foundation – Body of Knowledge: This includes resources that are either published as bodies of knowledge for systems engineering or related areas or are recognized consensus references that cover the breadth of systems engineering. The SEBoK is the sole body of knowledge for SE. The INCOSE SE Handbook covers the breadth of SE and is the key resource used for SE Certification.

Life Cycle Processes and Concepts: This includes the top-level standards and references that identify and define the system life cycle processes and the concepts that are essential for understanding, planning, and executing system life cycles and processes.

Assessment / Governance: This includes standards that are focused on the assessment or governance of processes, products, or other concerns in the engineering of systems or their life cycles.

Process Elaborations: This includes standards that focus specifically on one or more processes. These standards provide lower level details about the definitions, application, and other considerations that aid the planning and execution of the processes. The processes trace to the system life cycle processes identified in ISO/IEC/IEEE 15288, the INCOSE SE Handbook, and/or the SEBoK.

Application Guides: This includes the standards and references that provide key insights and guidance for the other standards and help users to understand the potential considerations for planning and executing the standards. Some of these standards and guides provide useful elaborations or notes regarding specific information or requirements in the standards or topics they refer to, such as ISO/IEC/IEEE 24748-2, A guide for the application of ISO/IEC/IEEE 15288.

Artifact Descriptions: This includes descriptions and/or requirements for specific artifacts (or information items) related to the system life cycle processes.

Tools: This includes standards that provide information or requirements related to tools that are applicable to the system life cycle processes.

Other Guidance: This includes any guidance that is outside of the scope of the other categories in the taxonomy.

Systems Engineering Related Standards

Summary of Systems Engineering Related Standards

Table 2 contains a partial list of SE related standards. This table does not include all SE related standards, as there are many are focused on a specific domain, sector, or user group (e.g., it does not include standards from a specific government agenda). The table does include standards that are considered to be widely applicable to systems engineering and systems life cycle management system life cycle processes, such as ISO/IEC/IEEE 15288 (2023). A link to the official abstract or information page has been provided in the first column for each standard.

Table 2. Summary of Systems Engineering Standards and Guides. (SEBoK Original)

Standard or Guide	Title
ISO/IEC/IEEE 15288 ^[1]	System life cycle processes
SEBoK	Guide to the Systems Engineering Body of Knowledge (SEBoK)
INCOSE SE Handbook 5th Ed ^[1]	INCOSE Systems Engineering Handbook (5th Edition)
SAE 1001 ^[1]	Integrated Project Processes for Engineering a System (replaced ANSI/EIA 632)
NATO AAP-48 ^[4]	NATO System Life Cycle Processes
SE Vision 2035 (INCOSE) ^[2]	Systems Engineering Vision 2035: Engineering Solutions for a Better World
ISO/IEC/IEEE 15026-1 ^[3]	Systems and software assurance concepts and vocabulary
ISO/IEC/IEEE 15026-2 ^[4]	Assurance case
ISO/IEC/IEEE 15026-3 ^[5]	System integrity levels

ISO/IEC/IEEE 15026-4 ^[6]	Assurance in the life cycle
ISO/IEC/IEEE 15289 ^[5]	Content of life-cycle information items (documentation)
ISO/IEC/IEEE 15939 ^[7]	Measurement process
ISO/IEC/IEEE 16085 ^[8]	Risk management
ISO/IEC/IEEE 16326 ^[4]	Project management
ISO/IEC/IEEE 21839 ^[9]	System of systems (SoS) considerations in life cycle stages of a system
ISO/IEC/IEEE 21840 ^[10]	Guidelines for the utilization of ISO/IEC/IEEE 15288 in the context of system of systems (SoS)
ISO/IEC/IEEE 21841 ^[11]	Taxonomy of system of systems (SoS)
ISO/IEC/IEEE 24641 ^[11]	Methods and tools for model-based systems and software engineering
ISO/IEC/IEEE 24748-1 ^[2]	Guidelines for life cycle management
ISO/IEC/IEEE 24748-2 ^[12]	Guidelines for the application of ISO/IEC/IEEE 15288
ISO/IEC/IEEE 24748-4 ^[13]	Systems engineering management planning
ISO/IEC/IEEE 24748-6 ^[14]	System and software integration
ISO/IEC/IEEE 24748-7 ^[15]	Application of systems engineering on defense programs (previously IEEE 15288.1)
ISO/IEC/IEEE 24748-8 ^[16]	Technical reviews and audits on defense programs (previously IEEE 15288.2)
ISO/IEC/IEEE 24748-9 ^[17]	Application of system and software life cycle processes in epidemic prevention and control systems
ISO/IEC/IEEE 24748-10 ^[18]	Guidelines for systems engineering agility
ISO/IEC/IEEE 24765 ^[19]	Systems and software engineering - Vocabulary
ISO/IEC/IEEE 26550 ^[20]	Reference model for product line engineering and management
ISO/IEC/IEEE 26580 ^[21]	Methods and tools for the feature-based approach to software and systems product line engineering
ISO/IEC/IEEE 29148 ^[22]	Requirements engineering
ISO/IEC/IEEE 42010 ^[23]	Architecture description
ISO/IEC/IEEE 42020 ^[24]	Architecture processes
ISO/IEC/IEEE 42024 ^[25]	Architecture fundamentals
ISO/IEC/IEEE 42030 ^[26]	Architecture evaluation framework
ISO/IEC/IEEE 42042 ^[27]	Reference architectures
ISO/IEC 29110 ^[28]	Standards and guides for very small entities (VSEs) (Multi-part set)
ISO/IEC TS 33060 ^[29]	Process assessment model for system life cycle processes
ISO 19450 ^[30]	Object Process Methodology (OPM)
ISO 9001 ^[31]	Quality Management Systems - Requirements
ISO 10007 ^[32]	Guidelines for configuration management
ISO 10303-233 ^[33]	Product data representation and exchange Part 233: Application protocol: Systems engineering

ISO 15704 ^[34]	Requirements for Enterprise - Reference Architectures and Methodologies
ISO 19439 ^[35]	Framework for Enterprise Modeling
ISO 31000 ^[36]	Risk management - Guidelines
IEC 31010 ^[37]	Risk assessment techniques
ANSI/AIAA G-043B ^[38]	Guide to the Preparation of Operational Concept Documents
ANSI/EIA-649C (SAE) ^[2]	National Consensus Standard for Configuration Management
CMMI- Dev V3.0 ^[39]	CMMI® for Development V3.0
ECSS-E-ST-10C ^[40]	Systems Engineering General Requirements
ECSS-E-ST-10-02 ^[41]	Space Engineering - Verification
ECSS-E-ST-10-06 ^[42]	Space Engineering - Technical Requirements Specification
ECSS-E-ST-10-24 ^[43]	Space Engineering - Interface Control
ECSS-M-ST-10 ^[44]	Space Project Management - Project Planning and Implementation
ECSS-M-ST-40 ^[45]	Space Project Management - Configuration and Information Management
ECSS-M-ST-60 ^[46]	Space Project Management – Cost and Schedule Management
ECSS-M-ST-80 ^[47]	Space Project Management - Risk Management
ECSS-M-00-03 ^[48]	Space Project Management - Risk Management
EIA 748D (SAE) ^[49]	Earned Value Management System
CEN EN 9277	Programme management - Guide for the management of Systems Engineering
IEC 62853 ^[50]	Open systems dependability
IEEE 828 ^[51]	Configuration Management in Systems and Software Engineering
NIST SP 800-160 Vol 1 ^[52]	Systems Security Engineering: Considerations for a Multidisciplinary Approach in the Engineering of Trustworthy Secure Systems
NIST SP 800-160 Vol 2 ^[53]	Developing Cyber-Resilient Systems: A Systems Security Engineering Approach
OMG SysML TM V2.0 ^[54]	OMG Systems Modeling Language

Practical Considerations

Key pitfalls and good practices related to systems engineering standards are described in the next two sections.

Pitfalls

Some of the key pitfalls encountered in the selection and use of SE standards are provided in Table 3.

Table 3. Pitfalls in Using Systems Engineering Standards. (SEBoK Original)

Pitfall Name	Pitfall Description
Turnkey Process Provision	• Expecting the standard to fully provide your SE processes without any elaboration or tailoring.
No Need for Product Knowledge	• Expecting that the standard can be used without any functional or domain knowledge since the standard is the collective industry knowledge.
No Process Integration	• Lack of integrating the standards requirements with the organization or project processes.

Good Practices

Some good practices as gathered from the references are provided in Table 4.

Table 4. Good Practices in Using Systems Engineering Standards. (SEBoK Original)

Good Practice Name	Good Practice Description
Tailor for Business Needs	• Tailor the standard within conformance requirements to best meet business needs.
Integration into Project	• Requirements of the standard should be integrated into the project via processes or product/service requirements.

References

Works Cited

Roedler, G. 2010. "An Overview of ISO/IEC/IEEE 15288, System Life Cycle Processes." Proceedings of the 4th Asian Pacific Council on Systems Engineering (APCOSE) Conference, 4-6 October 2010, Keelung, Taiwan.

Roedler, G. 2023. " SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.

Roedler, G. 2011. "Towards Integrated Systems and Software Engineering Standards." National Defense Industrial Association (NDIA) Conference, San Diego, CA, USA.

Roedler, G. 2015. "Evolution of SE Standards and Practices – ISO/IEC/IEEE 15288 Based Harmonization." National Defense Industrial Association (NDIA) Conference, Springfield, VA, USA.

Primary References

INCOSE. 2023. INCOSE Systems Engineering Handbook 5th Edition. International Council on Systems Engineering, Hoboken, NJ, USA: John Wiley & Sons. INCOSE SE Handbook.

ISO/IEC/IEEE. 2023. Systems and Software Engineering -- System Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288: 2023.

ISO/IEC/IEEE. 2024. "Part 1: Guidelines for life cycle management," in Systems and software engineering--life cycle management.. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.

ISO/IEC/IEEE. 2024. Systems and Software Engineering -- Life Cycle Management – Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes). Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-2:2024.

- ISO/IEC/IEEE. 2017. Systems and Software Engineering - Vocabulary (SEVocab). Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2017.
- NATO. 2022. NATO System Life Cycle Processes. Brussels, Belgium: North Atlantic Treaty Organization (NATO). NATO AAP-48.
- SAE. 2018. Integrated Project Processes for Engineering a System. Warrendale, PA, USA: SAE International. SAE 1001.
- Roedler, G. 2010. An Overview of ISO/IEC/IEEE 15288, System Life Cycle Processes. Proceedings of the 4th Asian Pacific Council on Systems Engineering (APCOSE) Conference, 4-6 October 2010, Keelung, Taiwan.
- Roedler, G. 2023. "SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.
- Roedler, G. 2011. "Towards Integrated Systems and Software Engineering Standards." National Defense Industrial Association (NDIA) Conference, San Diego, CA, USA.
- Roedler, G. 2015. "Evolution of SE Standards and Practices – ISO/IEC/IEEE 15288 Based Harmonization." National Defense Industrial Association (NDIA) Conference, Springfield, VA, USA.

Additional References

None.

References

- [1] <https://www.sae.org/standards/content/sae1001/>
- [2] <https://www.incose.org/publications/se-vision-2035>
- [3] <https://www.iso.org/standard/89808.html>
- [4] <https://www.iso.org/standard/80625.html>
- [5] <https://www.iso.org/standard/84444.html>
- [6] <https://www.iso.org/standard/74396.html>
- [7] <https://www.iso.org/standard/71197.html>
- [8] <https://www.iso.org/standard/74371.html>
- [9] <https://www.iso.org/standard/91120.html>
- [10] <https://www.iso.org/standard/71956.html>
- [11] <https://www.iso.org/standard/79111.html>
- [12] <https://www.iso.org/standard/84661.html>
- [13] <https://www.iso.org/standard/56887.html>
- [14] <https://www.iso.org/standard/81563.html>
- [15] <https://www.iso.org/standard/90139.html>
- [16] <https://www.iso.org/standard/91563.html>
- [17] <https://www.iso.org/standard/82474.html>
- [18] <https://www.iso.org/standard/90086.html>
- [19] <https://www.iso.org/standard/71952.html>
- [20] <https://www.iso.org/standard/69529.html>
- [21] <https://www.iso.org/standard/43139.html>
- [22] <https://www.iso.org/standard/72089.html>
- [23] <https://www.iso.org/standard/74393.html>
- [24] <https://www.iso.org/standard/68982.html>
- [25] <https://www.iso.org/standard/87510.html>
- [26] <https://www.iso.org/standard/73436.html>
- [27] <https://www.iso.org/standard/87310.html>
- [28] <https://www.iso.org/standard/82669.html>
- [29] <https://www.iso.org/standard/87701.html>
- [30] <https://www.iso.org/standard/84612.html>
- [31] <https://www.iso.org/standard/62085.html>
- [32] <https://www.iso.org/standard/70400.html>

- [33] <https://www.iso.org/standard/55257.html>
- [34] <https://www.iso.org/standard/71890.html>
- [35] <https://www.iso.org/standard/33833.html>
- [36] <https://www.iso.org/standard/65694.html>
- [37] <https://www.iso.org/standard/72140.html>
- [38] <https://arc.aiaa.org/doi/book/10.2514/4.105487>
- [39] <https://cmmiinstitute.com/cmml/intro>
- [40] <https://ecss.nl/standard/ecss-e-st-10c-rev-1-system-engineering-general-requirements-15-february-2017/>
- [41] <https://ecss.nl/standard/ecss-e-st-10-02c-rev-1-verification-1-february-2018/>
- [42] <https://ecss.nl/standard/ecss-e-st-10-06c-technical-requirements-specification/>
- [43] <https://ecss.nl/standard/ecss-e-st-10-24c-rev-1-interface-management-15-november-2024/>
- [44] <https://ecss.nl/standard/ecss-m-st-10c-rev-1-project-planning-and-implementation/>
- [45] <https://ecss.nl/standard/ecss-m-st-40c-rev-1-configuration-and-information-management/>
- [46] <https://ecss.nl/standard/ecss-m-st-60c-cost-and-schedule-management/>
- [47] <https://ecss.nl/standard/ecss-m-st-80c-risk-management/>
- [48] https://www.everyspec.com/ESA/ecss-m-00-03a_2569/
- [49] <https://www.sae.org/standards/content/eia748d/>
- [50] <https://webstore.iec.ch/en/publication/26367>
- [51] <https://standards.ieee.org/ieee/828/5367/>
- [52] <https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final>
- [53] <https://csrc.nist.gov/pubs/sp/800/160/v2/r1/final>
- [54] <https://www.omgsysml.org/>

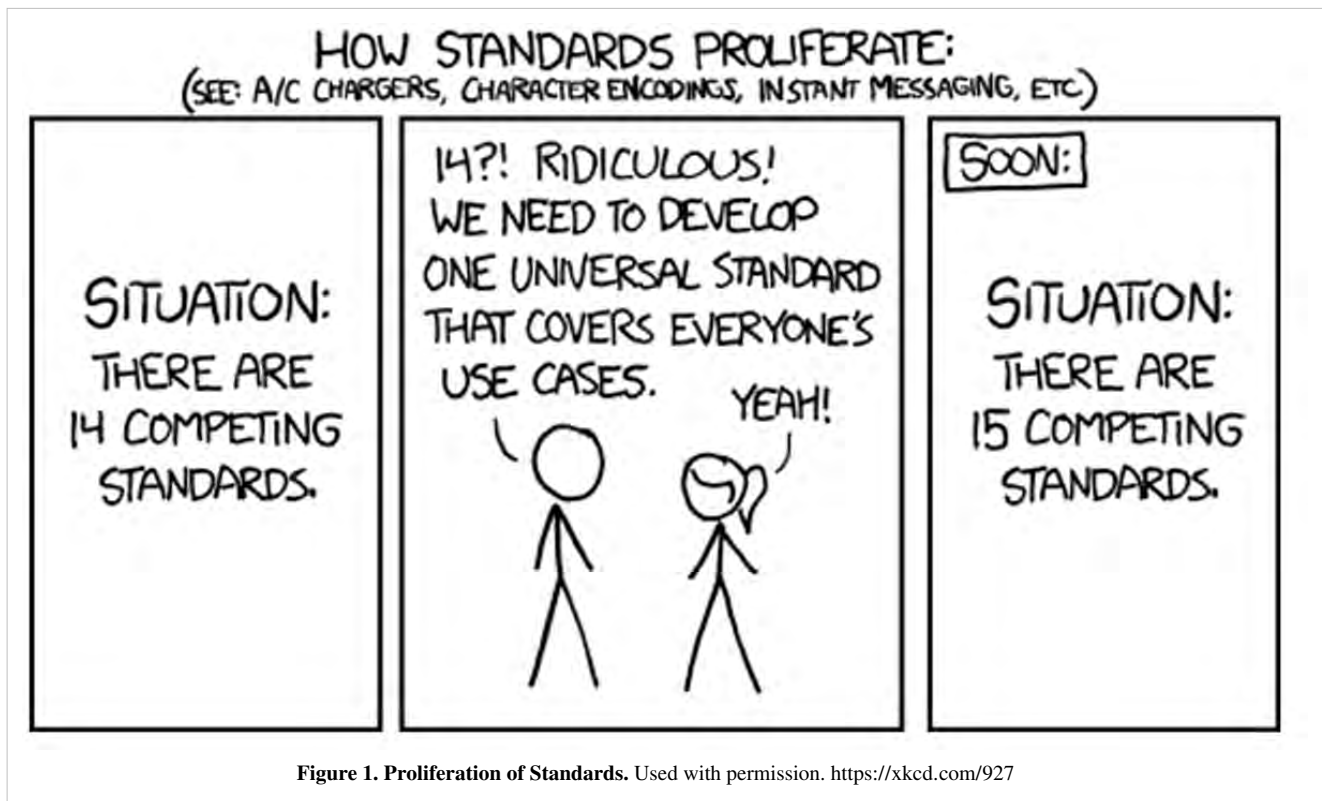
Harmonization of Systems Engineering Standards

- Lead Author:
- Garry Roedler
- Contributing Authors:
- William Bearden, David Endler, and Mike Yokell

Standards development organizations (SDOs) and other industry associations have been collaborating to align the systems engineering (SE) and software engineering (SwE) standards. The objective is to have a set of standards that can easily be used concurrently throughout all engineering disciplines, with common terminology and concepts.

Problem Statement

To address the gaps in knowledge within or across technical disciplines, many organizations attempted to define or codify their best practices. Not surprisingly, this resulted in the use of different terminology, process sets, concepts, and levels of prescription. It existed both within each technical disciplines, but was even more pronounced across SE and SwE due to trying to address the needs of their respective audiences. The problem has been exacerbated by competing standards (Roedler 2010) trying to create a “better standard” or addressing different interests that introduced inconsistencies. Additionally, new challenges and opportunities emerged, such as agile, model-based, and digital approaches, as the disciplines continued to evolve. This also introduced more possibilities for inconsistency in the standards. Through great effort, the SDOs became more open to collaboration and alignment. If alignment efforts do not have the commitment from all of the stakeholders, it can further drive proliferation of standards, as is indicated in Figure 1.



Root Cause

The root causes of the problem of inconsistencies across the standards include (Roedler 2010):

- **Competition** – Many SDOs exist. Each SDO is motivated to have its own set of effective standards that contribute to revenue. From the perspective of other stakeholders, competition has driven significant variation in new approaches, moving away from consensus, as competing organizations, tool vendors, and consultants try to promote what they provide as differentiating for success in the industry. All of this has driven proliferation of standards over the years. (Primarily SDO driven.)
- **Organizational** – Structures with different SDOs, teams, committees, etc. This is further challenged by changes in team structures as agile approaches became more common. And different teams within a single organization can be implementing very different norms as the organization transitions. (Driven by both SDO and project environment.)
- **Culture** – Concepts such as “we’re different,” “not invented here,” etc. Culture is complicated by the vast number of considerations influencing engineering teams and environments. (Primarily driven by project environment.)
- **Domains** – A focused, narrow view often doesn’t look beyond the domain for commonality. (Primarily driven by project environment.)

Impact

The impact of this problem includes the following (Roedler 2010):

- Less effective or efficient processes that are not focused on leveraging commonalities or lessons learned from a wider base of practitioners. This causes redundancy and has resulted in incompatibilities and inconsistencies between the standards making it difficult to use them concurrently.
- Less effective solutions that are not focused on a common approach to solve a problem or need.
- Obstacles for communicating (at all levels), working in integrated teams, and leveraging resources.
- Stove-piping due to the incompatibilities and inconsistencies of and lack of leveraging commonalities.

Alignment and Harmonization of Systems Engineering Standards

Approach

Collaboration between ISO/IEC JTC1/SC7 (Information Technology, Systems and Software Engineering), the IEEE Computer Society, the International Council on Systems Engineering (INCOSE), and others is being used to achieve alignment and harmonization objectives, including common vocabulary, concepts, and process structures; integrated process sets; and considerations for different audiences, domains, applications and tailoring.. This process produced a foundational set of vocabulary, process definition conventions, and life cycle management concepts provided in ISO/IEC/IEEE 24765 (Vocabulary), ISO/IEC/IEEE 24774 (Specification for process description), and ISO/IEC/IEEE 24748-1 (Guide to Life Cycle Management), respectively. At the heart of the approach is the alignment of ISO/IEC/IEEE 15288 (System life cycle processes) and ISO/IEC/IEEE 12207 (Software life cycle processes), which provide the top-level process framework for life cycle management of systems and software. This enables concurrent and consistent use of the standards to support both systems and software life cycle management on a single project. The approach includes a set of supporting standards and technical reports for elaboration of specific processes, description of practices for specific purposes (e.g., systems/software assurance), description of artifacts, and guidance for the application of the standards.

Collaboration With a Common Vision

INCOSE, IEEE, ISO, and other SDOs and industry associations collaborated to align the key SE related standards and resources to be consistent and to manage their evolution to enable concurrent usage. A key part of the harmonization effort was aligning ISO/IEC/IEEE 15288 with ISO/IEC/IEEE 12207 (Software Life Cycle Processes), such that they share consistent concepts, structure, and terminology so projects combining systems and software can use them coherently. After the release of ISO/IEC 15288 in 2002, the focus shifted from the development of a comprehensive system life cycle process standard to harmonization of ISO/IEC 15288 with other standards. Garry Roedler developed a vision for harmonization in the early 2000s and facilitated the initiative from 2004 to 2020. The harmonization work evolved into a broader community effort guided by the vision with collaboration across multiple professional societies and standards organizations, including INCOSE, ISO, IEEE, IEC, SAE, NDIA, government groups, and industry.

Figure 2 shows ISO/IEC/IEEE 15288 is at the center of this Co-evolution model (Roedler 2023). This is due to it serving as the basis for terminology and concepts as a baseline. As part of this alignment effort, INCOSE adopted the ISO/IEC/IEEE 15288 process set, definitions, and terminology for shaping the INCOSE SE Handbook in 2006, reinforcing coherence across INCOSE, ISO, and IEEE concepts and vocabularies. As the Systems Engineering Body of Knowledge (SEBoK) started its development in 2009, the project team also adopted ISO/IEC/IEEE 15288 and the INCOSE SE Handbook as key resources for guiding its content. Each of these resources have their unique purposes and scope, but have found utility in aligning. Over time the three resources, shown as a set of three key resources in the middle of the figure, have reached a point where each positively influences and strengthens the other. In turn, these resources have been a catalyst for alignment of the other SE related documents shown.

As part of the alignment effort, ISO/IEC JTC 1/SC 7 formed a Life Cycle Process Harmonization Advisory Group in 2005 that evaluated the standards for systems and software engineering. The objective of the group was to provide a set of recommendations for further harmonization of the industry standards. Specifically, its charter included:

- Perform an architectural analysis and recommend a framework for an integrated set of process standards in software and systems domains.
- Make recommendations regarding the future content, structure, and relationships of ISO/IEC/IEEE 12207, ISO/IEC/IEEE 15288 and their guides, as well as other related SC7 documents.

This work produced a well aligned set of system and software standards, as shown in the Taxonomy of SE Related Standards figure in the Systems Engineering Related Standards Landscape article.

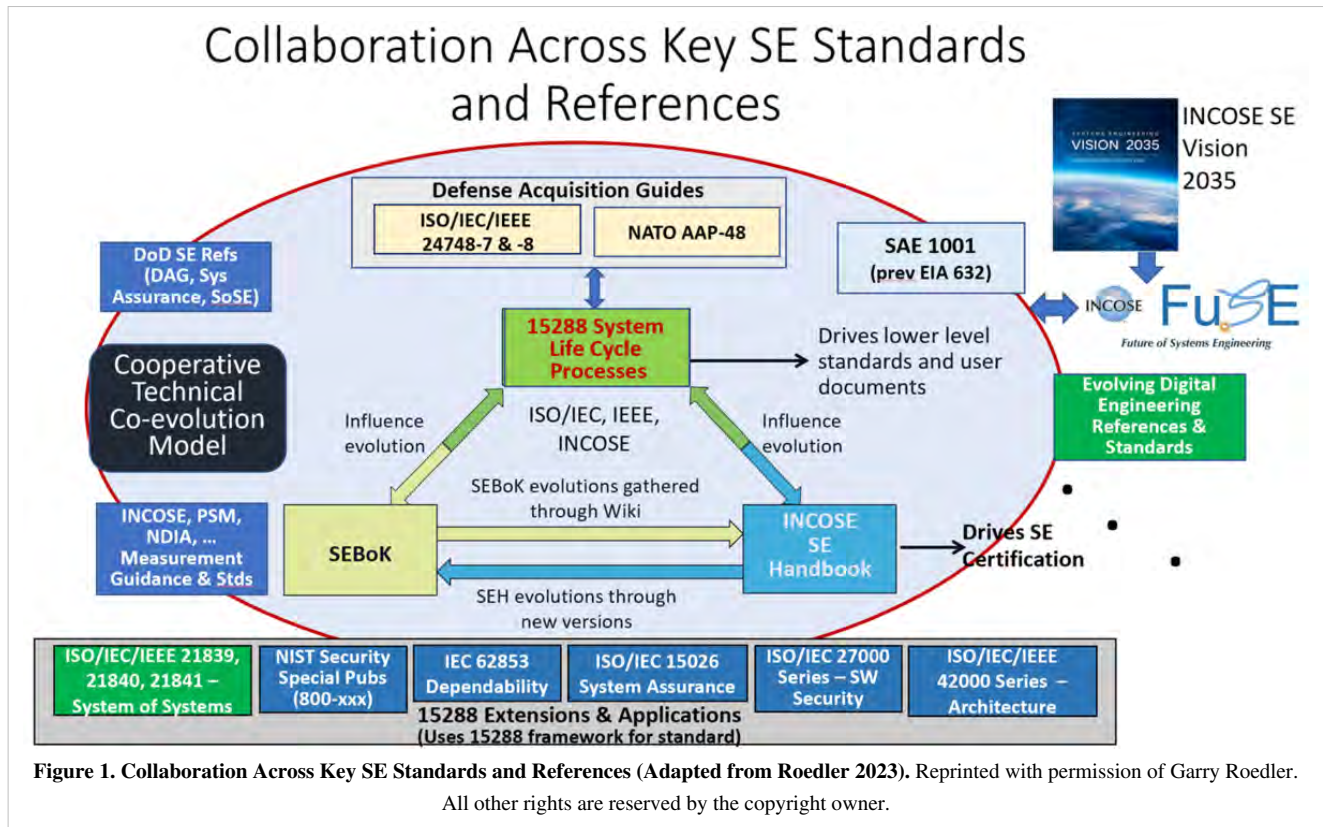


Figure 1. Collaboration Across Key SE Standards and References (Adapted from Roedler 2023). Reprinted with permission of Garry Roedler. All other rights are reserved by the copyright owner.

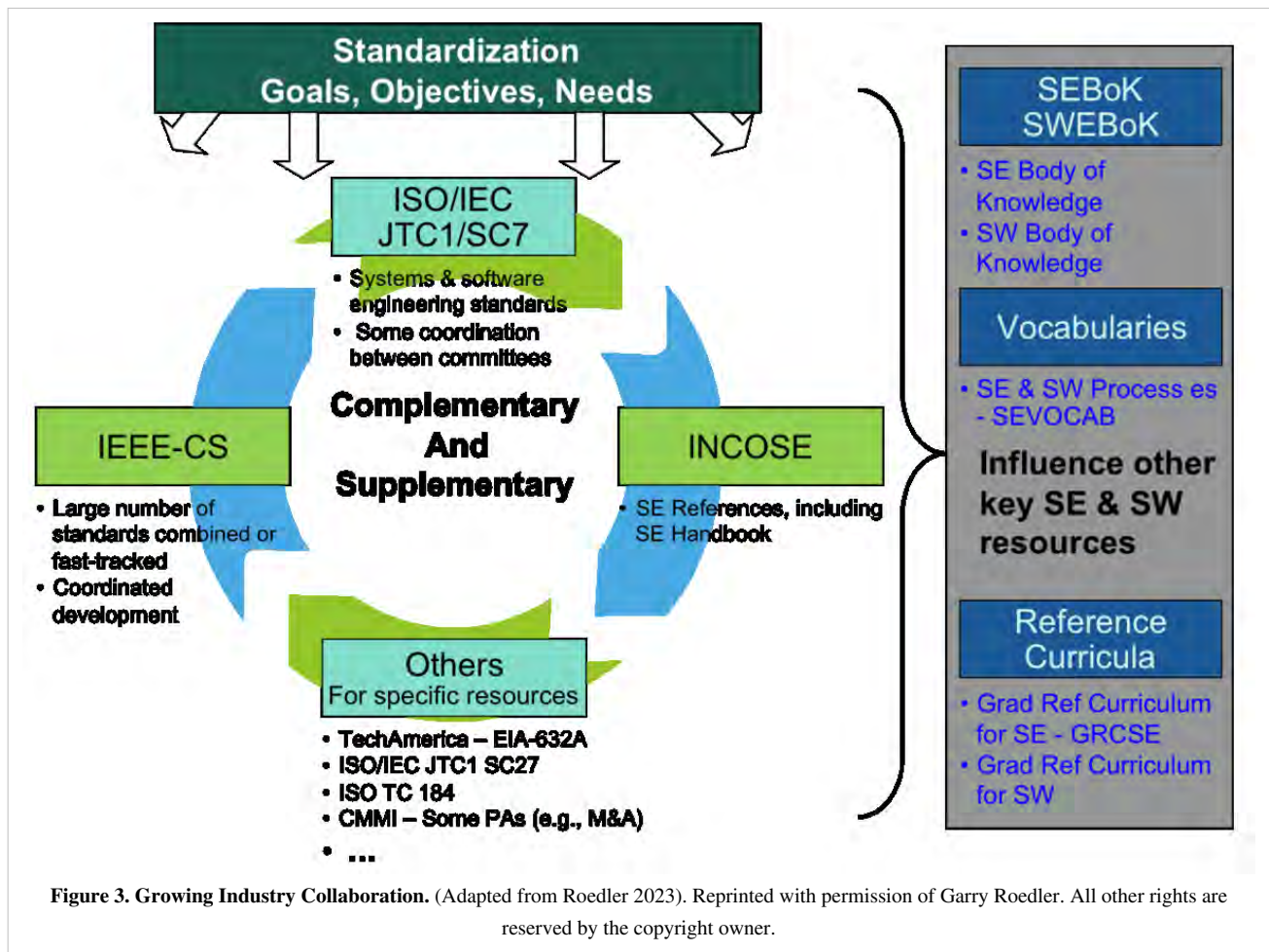
Current Efforts

The industry continues to harmonize SE related standards and references. The standards development community continues to evolve the standards together, promoting convergence instead of divergence. ISO/IEC/IEEE 15288 is widely accepted as the central canonical systems engineering process standard, supported by INCOSE, IEEE, ISO, and used in many industries. The collaboration has grown to include the work of the organizations and projects shown in Figures 2 and 3. These organizations are working towards the goal of completing a complementary and supplementary set of systems engineering resources that use the same terminology, principles, concepts, practices, and processes and can be used concurrently without issues.

Challenges and opportunities to be considered as the SE standards continue to evolve include:

- Where the standards are in their life cycle can influence timing and ability to harmonize.
- Agreements between collaborating parties regarding the basis of the aligned content.
- Standards still proliferate.
- Emerging topics do not necessarily align with the core or existing standards.

These challenges continue to be considered and addressed.



Key Outcomes of the Harmonization Efforts

1. ISO/IEC/IEEE 12207 (software) was brought into alignment with ISO/IEC/IEEE 15288, creating a coherent and comprehensive systems engineering lifecycle.
2. A common vocabulary was established with the SEVocab application and ISO/IEC/IEEE 24765 standard developed to provide a shared lexicon for ISO, IEEE, INCOSE, and PMI.
3. Process-specific elaboration documents were produced to support implementation across sectors or for specific applications.
4. Multiple IEEE documents were adopted by ISO, often with revisions to achieve consistency.
5. The process concepts, process set and terminology of ISO/IEC/IEEE 15288 was adopted for the development of SAE 1001, Integrated Project Processes for Engineering a System.
6. The ISO/IEC/IEEE 15288 process framework was adopted for the process section of the INCOSE Systems Engineering Handbook, the SEBoK (Systems Engineering Body of Knowledge), and other references.
7. The standard was adopted by the U.S. Department of Defense and other defense organizations, and defense-addendum standards (ISO/IEC/IEEE 24748-7 and -8, originally IEEE 15288.1/2) were developed.
8. Additional Harmonization Items – Several other cross-standard initiatives were completed, strengthening interoperability across the systems-engineering community.

Summary of the SE Standards Evolution and Harmonization

Figure 2 in Evolution of Systems Engineering Standards provides another view of the SE standards evolution and harmonization depicting a tree whose roots are the standards from which ISO/IEC/IEEE 15288 originated. ISO/IEC/IEEE 15288 is the trunk of the tree and the branches reflect development or revisions of other works as a result of harmonization with ISO/IEC/IEEE 15288. More information on the various evolution efforts is provided in SEBoK article Evolution of Systems Engineering Standards. Also, the article Systems Engineering Related Standards Landscape provides a large listing of the SE standards and references that are in use today.

Practical Considerations

Key pitfalls and good practices related to systems engineering standards are described in the Systems Engineering Related Standards Landscape article.

There are also instances in which standards groups for program management, safety, or other disciplines create standards on topics addressed within systems engineering, but use different terminology, culture, etc. One such example is risk management, which has been dealt with by many professional societies from a number of perspectives.

Systems engineers must also be aware of the standards that govern the specialty disciplines that support systems engineering, as discussed in Part 6.

Finally, the challenges listed above under the Current Efforts are also reflective of some of the Pitfalls when trying to align the standards and references.

References

Works Cited

Roedler, G. 2023. "SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.

Roedler, G. 2010. An Overview of ISO/IEC/IEEE 15288, System Life Cycle Processes. Asian Pacific Council on Systems Engineering (APCOSE) Conference.

ISO/IEC/IEEE. 2023. Systems and Software Engineering — System life cycle processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

ISO/IEC. 2021. Systems and software engineering — Life cycle management -- Specification for process description. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions. ISO/IEC/IEEE 24774:2021.

ISO/IEC/IEEE. 2019. Systems and Software Engineering — Vocabulary (SEVocab). Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2019.

ISO/IEC/IEEE. 2024. "Systems and Software Engineering — Life Cycle Management – Part 1: Guidelines for life cycle management,". Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.

ISO/IEEE. 2026. Systems and Software Engineering — Software Life Cycle Processes. Geneva, Switzerland: International Organization for Standards (ISO)/Institute of Electrical & Electronics Engineers (IEEE) Computer Society, ISO/IEEE 12207:2026.

Roedler, G. 2011. "Towards Integrated Systems and Software Engineering Standards." National Defense Industrial Association (NDIA) Conference, San Diego, CA, USA.

Proliferation of Standards figure: <https://xkcd.com/927/>

Primary References

Roedler, G. 2010. "An Overview of ISO/IEC/IEEE 15288, System Life Cycle Processes." Asian Pacific Council on Systems Engineering (APCOSE) Conference.

Roedler, G. 2023. "SE Standards and Guidance – Revisions and Trends." International Council on Systems Engineering, Boston, MA, USA.

Roedler, G. 2011. "Towards Integrated Systems and Software Engineering Standards." National Defense Industrial Association (NDIA) Conference, San Diego, CA, USA.

INCOSE. 2023. INCOSE Systems Engineering Handbook 5th Edition. International Council on Systems Engineering. Hoboken, NJ, USA: John Wiley & Sons. INCOSE SE Handbook.

ISO/IEC/IEEE. 2023. Systems and Software Engineering — System life cycle processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

ISO/IEC. 2021. Systems and software engineering — Life cycle management -- Specification for process description. Geneva, Switzerland: International Organisation for Standardisation/International Electrotechnical Commissions. ISO/IEC/IEEE 24774:2021.

ISO/IEC/IEEE. 2019. Systems and Software Engineering — Vocabulary (SEVocab). Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2019.

ISO/IEC/IEEE. 2024. "Systems and Software Engineering — Life Cycle Management – Part 1: Guidelines for life cycle management,". Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.

ISO/IEC/IEEE. 2024. Systems and Software Engineering — Life Cycle Management – Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes). Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-2:2024.

NATO. 2022. NATO System Life Cycle Processes. Brussels, Belgium: North Atlantic Treaty Organization (NATO). NATO AAP-48.

SAE. 2018. Integrated Project Processes for Engineering a System. Warrendale, PA, USA: SAE International. SAE 1001.

Additional References

INCOSE. 2022. Systems Engineering Vision 2035: Engineering Solutions for a Better World. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE SE Vision 2035. 2022.

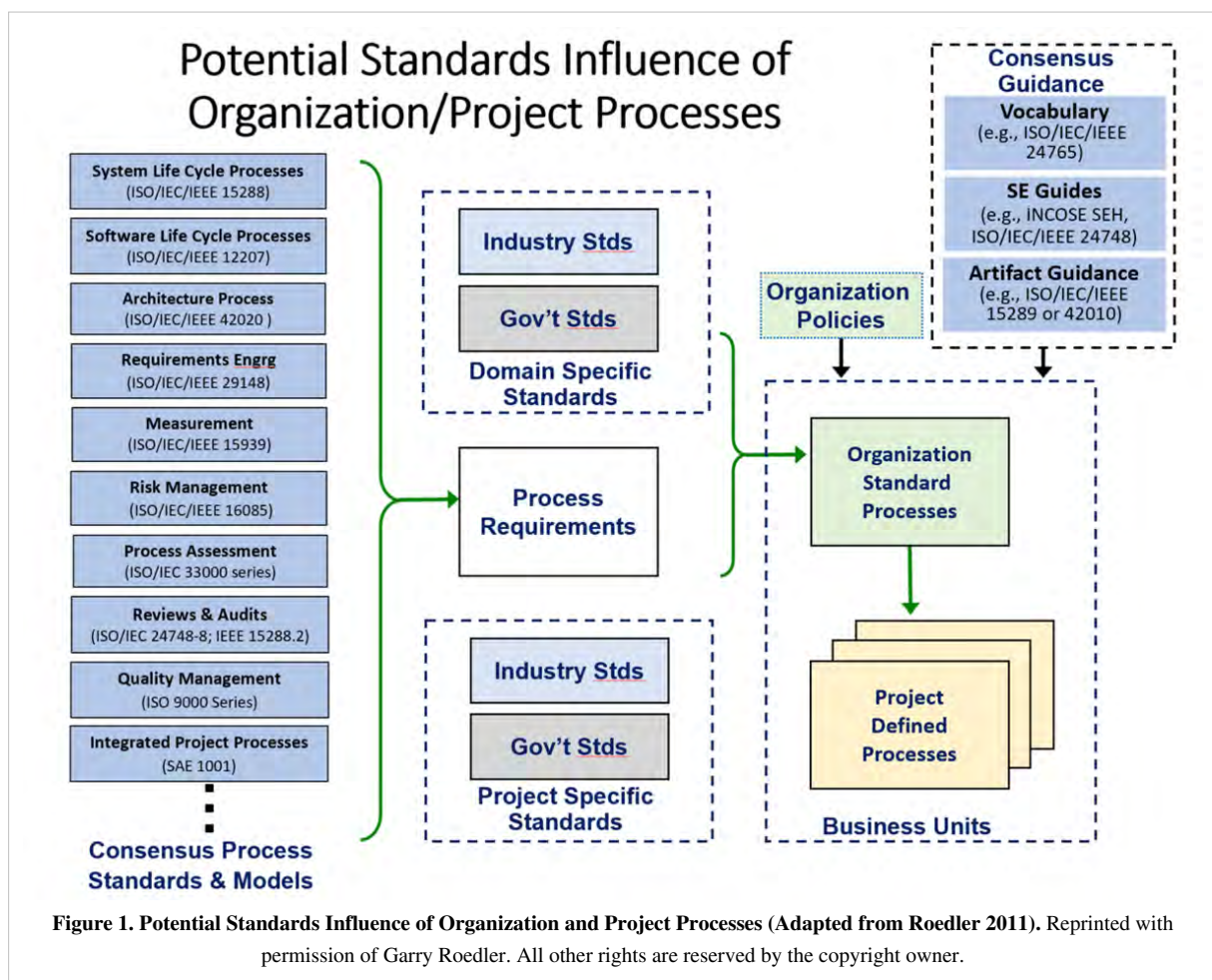
Application of Systems Engineering Standards

- Lead Author:
- Garry Roedler
- Contributing Authors:
- David Endler, Mike Yokell, and William Bearden

Standards can have an influence on organizations and their projects as indicated in Figure 1 (below). The standards provide sets of process, project, and product requirements and associated guidance that organizations use to influence the definition of their organization and project specific processes. Some pitfalls and good practices in utilizing standards are also identified in the article on Systems Engineering Related Standards Landscape. In this article, several additional factors related to the utilization of the standards in systems engineering (SE) are presented.

Standards and their Utilization

A standard is an agreed upon, repeatable way of doing something. It is a published document that contains a technical specification or other precise criteria designed to be used consistently as a rule, guideline, or definition. Standards help to make life simpler and to increase the reliability and the effectiveness of many goods and services we use, as detailed in Why Standards. Standards are created by bringing together the experience and expertise of relevant stakeholders, such as the producers, sellers, buyers, users, and regulators of a particular material, product, process, or service.



Standards are designed for voluntary use and do not impose any regulations. However, laws and regulations may address certain standards and may make compliance with them compulsory.

Further, organizations and their enterprises may choose to use standards as a means of providing uniformity in their operations and/or the products and services that they produce. The standard becomes a part of the corporate culture. In this regard, it is interesting to note that the ISO/IEC/15288 (2023) standard has provided such guidance and has provided a strong framework for systems engineers, as well as systems engineering and business management, as forecast earlier by Arnold and Lawson (2004).

ISO directives ^[1] state the following:

A standard does not in itself impose any obligation upon anyone to follow it. However, such an obligation may be imposed, for example, by legislation or by a contract. In order to be able to claim compliance with a standard, the user (of the standard) needs to be able to identify the requirements he is obliged to satisfy. The user needs also to be able to distinguish these requirements from other provisions where a certain freedom of choice is possible. Clear rules for the use of verbal forms (including modal auxiliaries) are therefore essential.

Requirements, Recommendations, and Permissions

In order to provide specificity, standards employ verb forms that convey requirements, recommendations, and permissions. For example, the ISO directives specify the following verb usages:

- The word *shall* indicates requirements strictly to be followed in order to conform to the standard and from which no deviation is permitted.
- The word *should* indicates that among several possibilities, one is recommended as particularly suitable without mentioning or excluding others, or that a certain course of action is preferred, but not necessarily required, or that (in the negative form) a certain possibility or course of action is deprecated but not prohibited.
- In the past, the word *may* indicates a course of action permissible within the limits of the standard. More recently, the use of *may* is not preferred for use in a standard and should be avoided. *Can* is the preferred term in most situations.

The directive also indicates that standards should avoid the use of *will*, *must*, and other imperatives. Furthermore, the use of the term *ensure* should be avoided, since it is rare that anything can be absolutely ensured. It reflects a guarantee. However, the phrase *help ensure* is acceptable.

Certification, Conformance, and Compliance

In the context of the management system standards (such as ISO 9001 or ISO 14000), certification refers to the issuing of written assurance (the certificate) by an independent external body that it has audited a management system and verified that it conforms to the requirements specified in the standard.

Typically, other more specific systems engineering standards are not the subject of certification. They are self-imposed in order to improve uniformity of organization and enterprise operations or to improve the quality of products and services. Alternatively, they may be dictated by legislation, policy, or as part of a formal agreement between an acquirer and a supplier.

In the context of engineering standards, compliance typically refers to adhering to mandatory legal or regulatory requirements, while conformance refers to adhering to voluntary standards or specifications. Compliance is often driven by external authorities, while conformance is often a company's internal commitment.

Conformance to standards is generally considered to be the voluntary action of an organization to meet the requirements or tailored requirements of a consensus standard or other guidance or specification. Many standards include a Conformance Clause that identifies what is required to make a claim of conformance or tailored conformance to the standard. This is especially true for engineering process standards.

Sometimes Conformance testing is a focus for product standards. Conformance testing, or type testing, is testing to determine whether a product or system meets some specified standard that has been developed for efficiency or interoperability. To aid in this, many test procedures and test setups have been developed either by the standard's maintainers or by external organizations, such as the Underwriters Laboratory (UL), specifically for testing conformity to standards.

Conformance testing is often performed by external organizations, which is sometimes the standards body itself, to give greater guarantees of compliance. Products tested in such a manner are then advertised as being certified by that external organization as complying with the standard. Service providers, equipment manufacturers, and equipment suppliers rely on this data to ensure quality of service (QoS) through this conformance process.

When compliance to a or more standards is needed, the agreement or contract has usually identified the standard(s) compliance as a requirement. Compliance with engineering standards is crucial for applications with a high need for safety, reliability, and functionality of products, systems, and infrastructure. It involves adhering to established rules, regulations, and best practices to meet industry requirements and legal obligations. Compliance is often enforced by external authorities, such as government agencies or industry bodies, and failure to comply can result in penalties or sanctions.

Key aspects of standards compliance include:

- Identifying and selecting the relevant standards that will be part of the agreement.
- Identifying and understand the applicable requirements of the standards.
- Integrating the requirements of the standards into the organization or project processes, product requirements, or project requirements with appropriate adaptation and tailoring; and documenting the incorporation and compliance demonstration needed.
- Verifying compliance through appropriate and valid means and documenting it.

Some challenges for compliance include:

- Keeping current – standards and other requirements can change, then requiring changes ot project planning and implementation, if a specific version is not part of the agreement.
- Balancing between compliance with standards and leveraging innovation in products or processes.
- Capturing and maintaining accurate documentation of the compliance can require structured and ongoing data management.

Tailoring of Standards

Since the SE standards provide guidelines, they are most often tailored to fit the needs of organizations and their enterprises in their operations and/or for the products and services that they provide, as well as to provide agreement in a contract. Tailoring is a process described in an annex to the ISO/IEC/IEEE 15288 (2023) standard. ISO/IEC/IEEE 15288 (2023) addresses the issues of conformance, compliance, and tailoring as follows:

- Full conformance, or a claim of full conformance, first declares the set of processes or other requirements for which conformance is claimed. Full conformance is achieved by demonstrating that all of the requirements have been satisfied using the outcomes or other criteria as evidence.
 - Tailored conformance for processes uses the standard as a basis for establishing a set of processes and their requirements. However, the specific requirements are selected, excluded, or modified in accordance with the tailoring process. A clear statement is made regarding the scope of the claimed tailored conformance.
 - The tailored text for which tailored conformance is claimed is declared. Tailored conformance is achieved by demonstrating that requirements for the processes, as tailored, have been satisfied using the outcomes as evidence.
 - When the standard is used to help develop an agreement between an acquirer and a supplier, clauses of the standard can be selected for incorporation in the agreement with or without modification. In this case, it is more appropriate for the acquirer and supplier to claim compliance with the agreement than conformance with the
-

standard, since the agreement usually becomes legally binding.

- Any organization (e.g., a national organization, industrial association, or company) imposing the standard as a condition of trade should specify and make public the minimum set of required processes, activities, and tasks which constitute a supplier's conformance with the standard.

Per ISO/IEC/IEEE 24748-2, "Life cycle models, as well as the processes from ISO/IEC/IEEE 15288, may be adapted for an individual project to reflect the variations appropriate to the organization, project and system while still being able to claim tailored conformance."

ISO/IEC/IEEE 24748-2 provides an informational list of circumstances that influence tailoring.

References

Works Cited

Arnold, S., and H. Lawson. 2004. "Viewing systems from a business management perspective." *Systems Engineering*, 7 (3): 229.

ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

Roedler, G. 2010. "An Overview of ISO/IEC/IEEE 15288, System Life Cycle Processes. Asian Pacific Council on Systems Engineering." Asia-Pacific Council on Systems Engineering (APCOSE) Conference, Keelung, Taiwan.

Roedler, G. 2011. "Towards Integrated Systems and Software Engineering Standards." National Defense Industrial Association (NDIA) Conference, San Diego, CA, USA.

Primary References

Roedler, G. 2010. "An Overview of ISO/IEC/IEEE 15288, System Life Cycle Processes." Proceedings of the 4th Asian Pacific Council on Systems Engineering (APCOSE) Conference, 4-6 October 2010, Keelung, Taiwan.

ISO. 2015. *Quality management systems -- Requirements*. Geneva, Switzerland: International Organisation for Standardisation. ISO 9001:2015.

ISO. 2015. *Environmental management systems -- Requirements with guidance for use*. Geneva, Switzerland: International Organisation for Standardisation. ISO 14000:2015

ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288: 2023.

ISO/IEC/IEEE. 2024. "Systems and Software Engineering -- Life Cycle Management – Part 1: Guidelines for life cycle management,". Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24748-1:2024.

ISO/IEC/IEEE. 2024. *Systems and Software Engineering -- Life Cycle Management – Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 24748-2:2024.

INCOSE. 2023. *INCOSE Systems Engineering Handbook 5th Edition*. International Council on Systems Engineering. Hoboken, NJ, USA: John Wiley & Sons. INCOSE SE Handbook.

Additional References

None.

References

[1] <https://www.iso.org/directives>

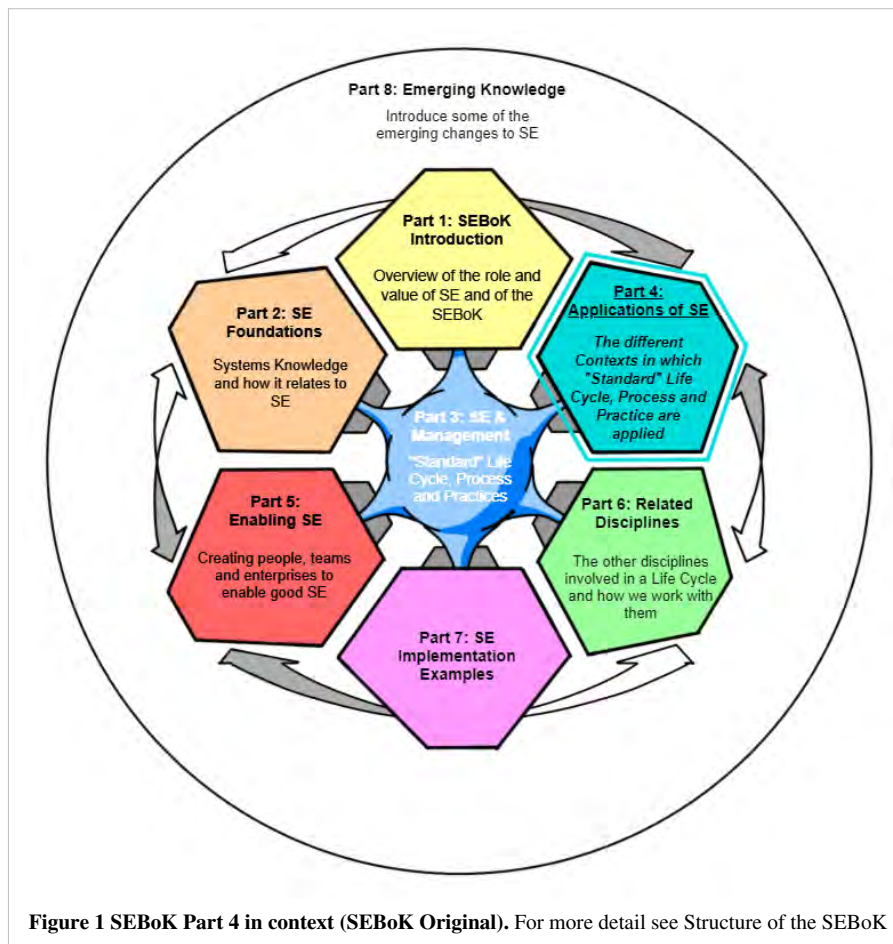
Part 4: Applications of Systems Engineering

Applications of Systems Engineering

Contents of this Part

- Product Systems Engineering (Bud Lawson and Ricardo Pineda) (Rick Adcock)
- Service Systems Engineering (Ricardo Pineda, Bud Lawson, and Richard Turner)
- Enterprise Systems Engineering (James Martin, Dick Fairley, and Bud Lawson)
- Systems of Systems (SoS) (Mike Henshaw and Judith Dahmann)
- Healthcare Systems Engineering
- Lead Author:
- Bud Lawson
- Contributing Author:
- Rick Adcock

Part 4 of the Guide to the SE Body of Knowledge (SEBoK) focuses on the application of systems engineering to the creation and life cycle management of various types of systems.



In particular, the part covers product systems, service systems, enterprise systems, and systems of systems (SoS). It also contains a knowledge area describing Healthcare SE as a domain extension of these general SE approaches. This is the first of a number of planned domain-based extensions.

Systems Engineering Application

The different ways in which each of these contexts shapes the application of the generic SE Life Cycle and Process knowledge in Part 3 are discussed in detail in the KA above.

It is important to note that none of the contexts above are intended to be wholly separate or mutually exclusive from the others. They should be seen as overlapping and related frameworks which provide a starting point for how generic SE might be used to fulfil real-world needs. We might think of each as a model of how SE can work in the real world. Each gives advice on how to use generic SE life cycle and process knowledge against its own viewpoint. If necessary, each may also develop new or extended knowledge relevant to its context, which becomes part of the extended toolkit of SE. Like any set of models, each has its own simplifications, strengths and weaknesses. As a general principle we would always select the simplest model available which fits the purpose and use that. For a complex outcome the combination of a number of models may be needed.

The real-world application of SE is no different. In most real projects, combinations of Product, Service, Enterprise and SoS knowledge may be needed to achieve success. The extent to which these combinations are taken from pre-determined approaches vs. the need for systems engineers to create such combinations as part of the application of SE is a key question for how SE is used. The final part of this knowledge, how SE is applied in the real world, sits within the knowledge base of the various application domain. Some domains have a very detailed set of procedures, guidelines and standards relevant to that domain, while others take general SE and apply it as needed using the judgement of those involved. In general, all domains have parts of both domain specific guidelines and experienced people. The SEBoK was originally written to be domain independent, other than through the application examples in part 7. To complete the SEBoK, we intend to create a series of Domain Application KA. These will give an overview of how SE application maps to domain practice. They are aimed at both the general SE reader who wants to know more about a domain and those working within a domain.

The Healthcare SE KA contained in this version of the SEBoK is the first such domain specific extension of the SEBoK.

References

Works Cited

None

Primary References

None.

Additional References

None.

Knowledge Area: Product Systems Engineering

Product Systems Engineering

Contents of this Knowledge Area

- Product Systems Engineering Background (Ricardo Pineda)
- Product as a System Fundamentals (Ricardo Pineda)
- Business Activities Related to Product Systems Engineering (Ricardo Pineda)
- Product Systems Engineering Key Aspects (Ricardo Pineda)
- Product Systems Engineering Special Activities (Ricardo Pineda)
- Lead Authors:
- Bud Lawson and Ricardo Pineda
- Contributing Author:
- Rick Adcock

Product systems engineering (PSE) is at the core of the new product development process (NPDP) that is needed to successfully develop and deploy products into different market segments. A market can be consumer based (e.g., private enterprises or general consumers) or it can be public (not-for-profit). Public

markets address the strategic needs of a country or region, such as military, healthcare, educational, transportation, and energy needs. NPDP has two significantly overlapping and integrated activities:

1. **Systems engineering:** This includes concept generation, engineering design/development, and deployment
2. **Market development:** This includes market research, market analysis, product acceptance and market growth (diffusion), and rate of adoption

NPDP also includes manufacturability/producibility, logistics and distribution, product quality, product disposal, conformance to standards, stakeholder's value added, and meeting customer's expectations. The internal enterprise competence and capabilities such as customer support, sales & marketing, maintenance and repair, personnel training, etc., must also be taken into account.

Topics

The Product Systems Engineering Background article discusses product types and the need for a product to be aligned with the business objectives of the enterprise. It also discusses the relationships between PSE, product development and technology development.

Various types of connections between product elements, and the concept of enabling systems are introduced in the Product as a System Fundamentals article. It also discusses product architecture, modeling, analysis, and integration with various specialty engineering areas.

Product launching and product offerings have close linkages to different business processes. The major linkages are to business development, marketing, product lines, quality management, project management, operations management, supply chain management, etc. These and other topics are described in the Business Activities Related



to Product Systems Engineering article.

Products emerge when they are realized based upon a system definition. Realizing the system results in instances of the system that are either delivered as products to a specific acquirer (based upon an agreement) or are offered directly to buyers and users. Key Aspects of PSE are discussed in the Product Systems Engineering Key Aspects article, which discusses aspects such as acquired vs. offered products, product lifecycle and adoption rates, integrated product teams (IPTs) and integrated product development teams (IPDTs), product architectures, requirements and standards, etc.

The last article, Product Systems Engineering Special Activities, covers some of the special activities carried out by PSE during the different stages from concept through product deployment.

Key Terms and Concepts

Product

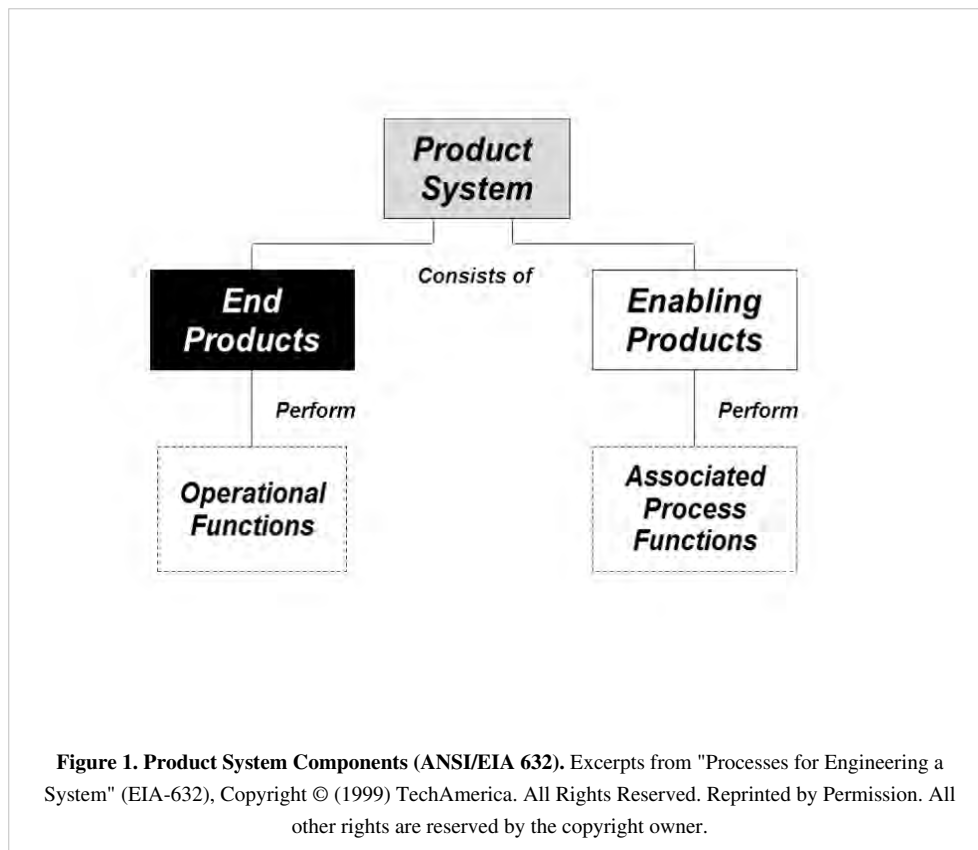
A product is an artifact that is created by some person or by some process such as a manufacturing process, software source code compilation and integration, building construction, creative writing process, or data processing.

In general, a business product is defined as a *thing produced by labor or effort*, or the *result of an act or a process*. It stems from the verb *produce*, from the Latin *prōdūce(re)* (*to*) *lead or bring forth*. Since 1575, the word *product* has referred to anything produced, and since 1695, the word *product* has referred to *a thing or things produced*.

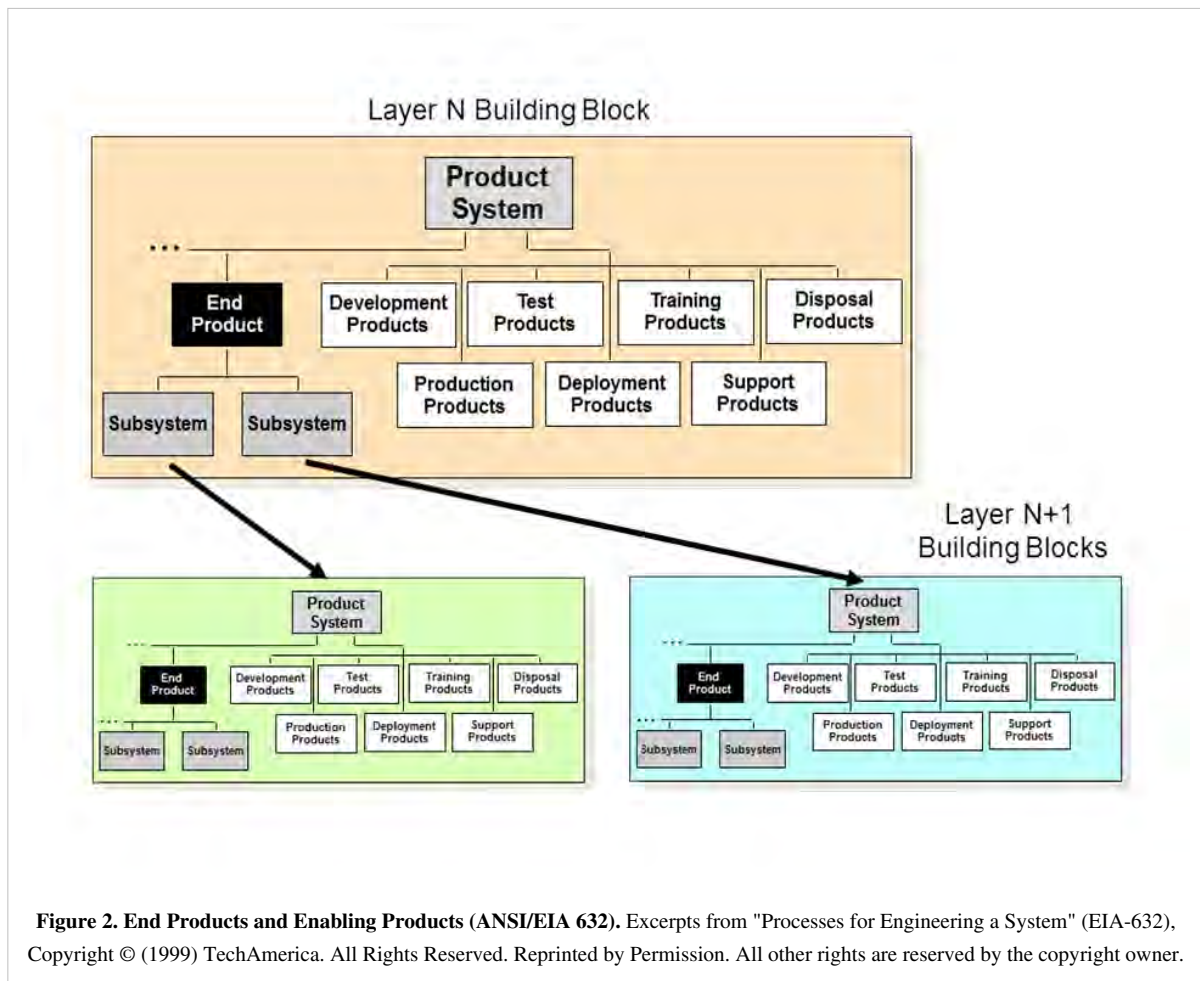
In economics and commerce, products belong to a broader category of *goods*. The economic meaning of the word *product* was first used by political economist Adam Smith. In marketing, a product is anything that can be offered to a market that might satisfy a want or a need. In retail industries, products are called merchandise. In manufacturing, products are purchased as raw materials and sold as finished goods. Commodities are usually raw materials, such as metals and agricultural products, but a commodity can also be anything widely available in the open market. In project management, products are the formal definitions of the project deliverables that make up or contribute to delivering the objectives of the project. In insurance, the policies are considered products offered for sale by the insurance company that created the contract.

Product System

A product system is the combination of end products and the enabling products for those end products. This concept of a product system is illustrated in Figure 1. In the ANSI/EIA 632-2003 standard, just the term *system* is used, but the scope of the standard is clearly restricted to product systems.



The end product can also be considered as a system with its own elements or subsystems, each of which has its own enabling products as illustrated in Figure 2. The product development process usually focuses only on the engineering of the end product. PSE is essential when the enabling products are by themselves complex or their relationship to the end product is complex. Otherwise, the use of a traditional product development process is sufficient.



Product Realization System

The product realization system is a related system that enables the *realization* of the product system. It consists of *all the resources to be applied in causing the Intervention System [i.e., the product system, in this case] to be fully conceived, developed, produced, tested, and deployed* (Martin 2004). Lawson (2010) refers to this as a respondent system in the system coupling diagram. The intervention system is the system that is to be realized (or conceived and brought into being) in order to address some perceived problem in the context as shown in Figure 3.

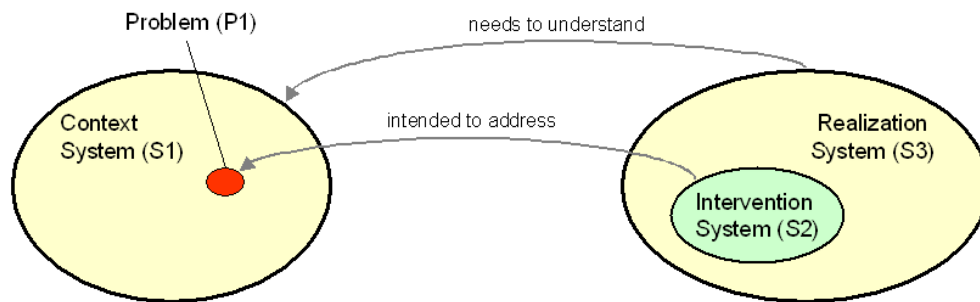


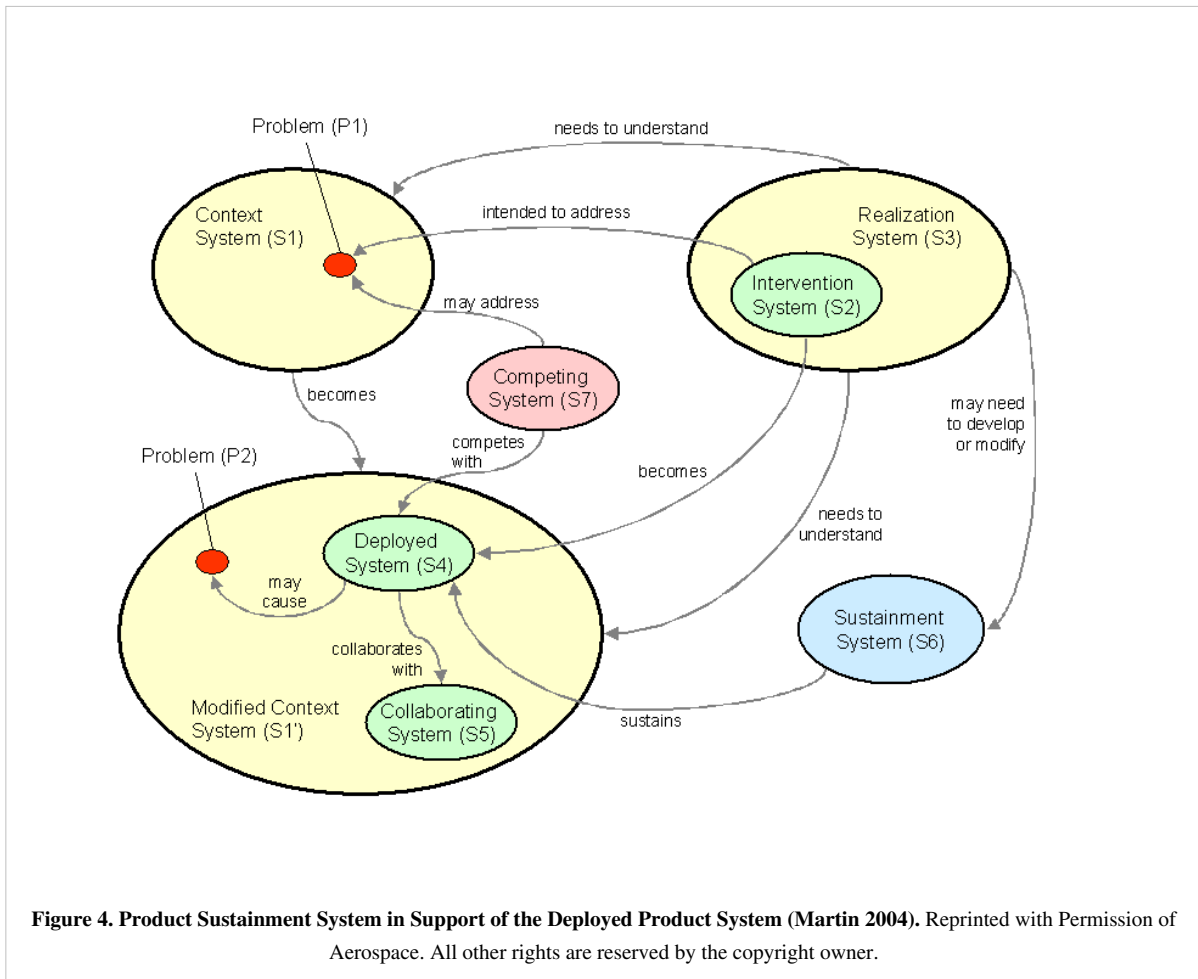
Figure 3. Realization System That Creates the Intervention to Solve a Problem (Martin 2004). Reprinted with Permission of Aerospace. All other rights are reserved by the copyright owner.

The realization system can be a service system (as described in knowledge area Service Systems Engineering) or an enterprise system (as described in the knowledge area Enterprise Systems Engineering). When the realization system is a service system, then the service could partially realize the system by just designing the product system without developing or creating it. This design service system can pass the design to a manufacturing service system that turns the design into a physical artifact. Usually an enterprise is established to orchestrate the disparate services into a cohesive whole that is efficiently and effectively performed to achieve the strategic goals of that enterprise.

The product realization system utilizes a product realization process as described in (Magrab et al 2010) or a product development process as described in (Wheelwright and Clark 1992).

Product Sustainment System

When the realization system delivers the product system into its intended environment, the product often needs a set of services to keep that product operational. This other system, when needed, is called the product sustainment system. It consists of various enabling products and operational services. The sustainment system in relation to the realization system and the deployed product system is illustrated in Figure 4. Notice that the realization may need to develop or modify the sustainment for the particular intervention (product) system under development.



Product Systems Engineering, Service Systems Engineering and Enterprise Systems Engineering

PSE is in line with Traditional Systems Engineering (TSE) as captured in most textbooks on the subject, such as Wasson (2006), Sage and Rouse (2009), and Blanchard and Fabrycky (2011). However, they do not cover the full breadth of PSE since they tend to focus on hardware and software products only. Other kinds of products to be engineered include personnel, facilities, data, materials, processes, techniques, procedures, and media (Martin 1997; Lawson 2010). Further discussions on the distinctions between the various kinds of products is provided in the Product Systems Engineering Background article. Product system domains could be data-intensive (e.g. transportation system), facilities-intensive (e.g. chemical processing plant), hardware-intensive (e.g. defense systems), or technique-intensive (e.g. search and rescue system). Most product systems are a composite of several different kinds of products that must be fully integrated to realize the complete value-added potential for the different stakeholders.

When compared to Service Systems Engineering (SSE) and Enterprise Systems Engineering (ESE), PSE has some unique considerations:

- Often a product is part of a product line where both the product line and the products that make up that product line must be engineered simultaneously.
- Products are often composed of parts and sub-assemblies produced by several suppliers. This entails the need to work closely with the supply chain to ensure a successful product offering.
- Large complex products often require a lengthy and complicated series of steps for assembly, integration and test. During integration, many of the key assumptions made during the initial product design could be challenged.

- Products will usually require certification as to their safety or other factors like energy conservation and environmental compatibility. Electronic products often require certification to ensure electromagnetic compatibility and limited electronic emissions into the radio frequency spectrum. Transportation products require certification for safe operations.
- Products often have a complicated distribution network since they are not always developed where the end user may require it. There could be depots, warehouses, multi-modal transportation, wholesalers, dealers, and retail stores as part of this distribution network. This introduces challenges in delivery, maintenance and support of the product.
- Products must be engineered along with the realization system and the sustainment system. Sometimes it is necessary to make trade-offs between the features and functions of the product system, the realization system and the sustainment system.

These considerations and others will be addressed in the articles under this knowledge area. One of the responsibilities of ESE is to manage these various considerations across multiple product lines across the enterprise while maximizing profits and customer satisfaction. SSE is often dependent on the products resulting from the PSE. A service will often be based on a product infrastructure that includes elements like data processing, hardware, software, data storage, data entry devices, display devices, service delivery facilities and techniques, service desk technicians, maintenance personnel, service offering catalogs and printed materials. Each of these products in the service infrastructure may need to be separately engineered using its own PSE lifecycle.

Creating Value

An enterprise that creates products must also create value that exceeds the cost of the product in the eyes of the customer. This applies to both for-profit and not-for-profit private and public enterprises. The creation and delivery of products may be the result of an acquisition agreement or an offering directly to buyers or users. To remain competitive, enterprises also need to understand the effects of the global economy, trends in industry, technology development needs, effects of new technology breakthroughs, market segments creation and their expectations, and most importantly, ever evolving customer expectations.

Ring (1998) defines a system value cycle with three levels that a systems approach must consider to deliver real-world benefit:

1. stakeholder value
2. customer need
3. system solution

Value will be fully realized only when it is considered within the context of time, cost, and other resources appropriate to key stakeholders.

Aligning product characteristics with associated operational activities

The user of a product views the product as an asset that can be utilized in one's own systems of interest (Lawson 2010). Thus, in supplying the product, the expected form of operation becomes a driving factor in determining the characteristics of the product. In several contexts, in particular for military related products, the desired operational activities are termed concept of operations (ConOps) and in the case of commercial enterprises the intended use of the system is described through some form of *Market Service Description* of the product. The intended use of the product is market/customer driven and so the product characteristics must be aligned with the operational intent.

Architectures as basis for value assessment

Architectures can be used by enterprises to shift product development from individual products to an underlying product line architecture that incorporates the flexibility required by the enterprise to rapidly tailor new technologies and features to specific customer requirements (Phillips 2001). In determining the architecture of the product system, various alternative designs may arise. Each of the architecture alternatives is to be evaluated with respect to its value contribution to end users and other stakeholders.

Role of evaluation criteria in selection between product alternatives

In assessing the product system value, one must consider the measures that are to be used to determine the *goodness* of the product alternatives (alternative architectures and technologies) with respect to producibility, quality, efficiency, performance, cost, schedule and most importantly, the coverage provided in meeting the customer's requirement or market opportunity.

Role of trade-offs in maximizing value

The evaluation of alternatives must include the trade-offs between conflicting properties. For example, in striving for superior quality and efficiency, trade-offs must be made with respect to schedule and cost. See article on Measurement in Part 3. Trade-offs are made during different stages of the development process: at the product or system level, at the subsystem and architecture definition level, and at the technology level (Blanchard and Fabrycky 2011).

There are a variety of methods for performing trade-off analysis such as: utility theory, analytic hierarchical process, the Pugh selection method, multi-objective decision, multi-attribute utility analysis, and multi-variate analysis. For software, the Software Engineering Institute (SEI) provides 'The Architecture Tradeoff Analysis Method (ATAM)' (Kazman et al., 2000) for evaluating software architectures relative to quality attribute goals. ATAM evaluations expose architectural risks that potentially inhibit the achievement of an organization's business goals. The ATAM not only reveals how well an architecture satisfies particular quality goals, but also provides insight into how those quality goals interact with each other and how they trade off against each other.

Expanding role of software in creation of product value

Software has an increasing role in providing the desired functionality in many products. The embedding of software in many types of products (such as transportation vehicles, home appliance, and production equipment) accounts for an ever-increasing portion of the product functionality. The current trend is the development of a network of systems that incorporate sensing and activating functions. The use of various software products in providing service is described in the Service Systems Engineering knowledge area.

References

Works Cited

- ANSI/EIA. 2003. *Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA). ANSI/EIA 632-2003.
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.
- ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.
-

- Kazman, R., M. Klein, and P. Clements. 2000. *ATAM: Method for Architecture Evaluation*. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU). CMU/SEI-2000-TR-004, ESC-TR-2000-004.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications.
- Magrab, E., S. Gupta, P. McCluskey, and P. Sandborn. 2010. *Integrated Product and Process Design and Development - The Product Realization Process*. Boca Raton, FL, USA: CRC Press.
- Martin, J.N. 1997. *Systems Engineering Guidebook: A process for developing systems and products*, 1st ed. Boca Raton, FL, USA: CRC Press.
- Martin, J. 2004. "The Seven Samurai of Systems Engineering: Dealing with the Complexity of 7 Interrelated Systems." Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium, 2004, Toulouse, France.
- Phillips, F. 2001. *Market-oriented Technology Management - Innovating for Profit in Entrepreneurial Times*. Berlin, Germany: Springer-Verlag.
- Pugh, S. 1990. *Total Design: Integrated Methods for Successful Product Engineering*. Upper Saddle River, NJ, USA: Prentice Hall.
- Ring, J. 1998. "A Value Seeking Approach to the Engineering of Systems." Proceedings of the IEEE Conference on Systems, Man, and Cybernetics, 1998, San Diego, CA, USA. p. 2704-2708.
- Sage, A., and W. Rouse. (eds.) 1999. *Handbook of Systems Engineering and Management*. Hoboken, NJ, USA: John Wiley and Sons, Inc.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. New York, NY, USA: John Wiley & Sons.
- Wheelwright, S., and K. Clark. 1992. *Managing New Product and Process Development: Text and Cases*. Columbus, OH, USA: Free Press.

Primary References

- ANSI/EIA. 2003. *Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA). ANSI/EIA 632-2003.
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.
- Magrab, E., S. Gupta, P. McCluskey, and P. Sandborn. 2010. *Integrated Product and Process Design and Development - The Product Realization Process*. Boca Raton, FL, USA: CRC Press.
- Martin, J. 2004. "The Seven Samurai of Systems Engineering: Dealing with the Complexity of 7 Interrelated Systems." Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium, 2004, Toulouse, France.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. New York, NY, USA: John Wiley & Sons.

Additional References

None.

Product Systems Engineering Background

- Lead Author:
- Ricardo Pineda

Product Types

A system is by definition composed of elements that interact. The system itself is usually an element of a larger system, and each element can often also be viewed as a system on its own.

A system element consists of one or more products.

Products need to be produced or acquired. Some can be acquired or procured as-is, without need for fabrication or modification. Others need to be engineered, and in some cases, systems-engineered (Martin 1997). Basic product types are depicted in the figure below.

Types of products are not limited to hardware or software. Many other types of products perform functions necessary to meet stakeholder needs. Some are only relevant to certain industries or domains, such as structures for civil engineering, or ships for shipping or the naval domain. Systems engineers must remember not to allocate the required behavior for a system to hardware and software elements alone.

While we may associate the idea of a product with concrete objects like computer chip, phones, aircraft, or even command, control and communications centers, an organization or a process can also be a product. Sometimes a product is not complex enough to justify performing Product SE, and only needs product design engineering. Enterprise SE and Service SE should determine whether a product needs Product SE.



Product Taxonomy

For any system being developed, the systems engineers must decide what are the right elements to be included. This is not self-evident, because basic product types are not necessarily mutually exclusive. For example, some would consider that facilities contain hardware and people. Others would consider facilities to be separate from hardware and people. Some would include material as part of hardware, while others would not. Creating a taxonomy of product types can help the systems engineer think clearly and thoroughly about what components to include.

Business Objectives and Products

When it develops and launches a new product, an enterprise must align that product with its business goals, internal capabilities, and competition. It must align the end product with the systems expected to realize and sustain it.

The new product concept must be based on analysis that, besides product potential, also explores the ability of the enterprise to exploit that potential, including factors like organizational culture, focus, goals, and processes. Present and future markets and technology must be analyzed. So must several dimensions of competition: competitors' offerings and their plans, for entry into new markets and for product expansion including new functionality, features, or services. These, and the ability of the enterprise to react to them, must also be monitored for the enterprise to remain competitive in the long term.

Accelerating economic globalization since the 1970s has forced enterprises to respond to global needs, not just local or regional ones. Enterprises in the resulting hyper-competitive business environment must analyze their financial goals, their market positions, and the business segments in which they participate, in order to understand what products are required.

This is true for completely new products, product enhancements, penetration of new markets, and growth within existing markets.

Relationship between Product Systems Engineering and Product Development

Product development is the process of bringing a new product to market. Product SE (PSE) considers the complete product system—that is the product in the context of all its enabling elements. PSE takes a full life cycle perspective, “from cradle to grave” or “dust to dust.”

Technology-based product development may be thought of as coming from two sources. One, where innovation enhances existing technology, is aimed at relatively short-term market windows. The other involves long-term research to identify the technology developments required to realize the concept. These may be technologies whose availability is not foreseen in the near future, meaning that substantial investment and long lead times may be required before the proof of concept, initial operational capabilities (IOCs) or prototyping stages are reached, let alone the commitment to realize the actual product offering. Some authors claim that the systems engineering process and the new product development (NPD) process for this second source are one and the same.

It is from the second source that strategic initiatives (long-term applied research) realize new products in areas like military aircraft or bioengineering. When research resolves fundamental questions on matters of science or national/regional interest technology, breakthroughs occur.

This article concentrates on the first source of technology-based product development, that is, the one driven by ever-evolving market needs to enhance existing technology.

Product Development Patterns

When existing or near-future technology innovations are exploited to generate new product ideas, product development may follow any one of following scenarios (Phillips 2001):

- Product development may use well-established technologies to help the enterprise improve the efficiency of current operations.
- Product development may use well-established technologies to help the enterprise into new kinds of operations.
- Product development may use leading edge technologies to improve the efficiency of current operations.
- Product development may use leading edge technologies to help the enterprise into new kinds of operations.

The product itself may simply be a modification of an existing product or its presentation, it may possess new or different characteristics that offer additional benefits to the customer, and/or it may be entirely new and satisfy a newly-defined customer want or market niche ([https:// www. businessdictionary. com/ definition/ product-development.html](https://www.businessdictionary.com/definition/product-development.html) ^[1]).

Existing realization or sustainment systems may not be adequate to develop a given product. For example, it might be necessary to change development practices, use different testing methods or facilities, or upgrade manufacturing equipment and procedures. There might need to be improved customer support procedures and newly trained support personnel, upgraded maintenance facilities and tools, or modified spare parts delivery techniques.

Market Pressures

The product development process must be dynamic, adaptive, and competitive on cost, time-to-market, performance, and quality on a global scale. This is because in the global economy continuous technology innovation and constantly evolving markets and customer needs demand a rapid response.

Products themselves are often multidisciplinary in nature; product development must have close participation, not only from the different specialty engineering fields (mechanical, electrical, industrial, materials, and so on), but also from the finance field to analyze the total cost of development (including production), marketing and sales to understand market behavior and acceptance, manufacturers and distributors, and legal and public relations experts.

All this has mandated enterprises to assess how they create their products and services. The result has been an effort to streamline the development process. One example of this is seen by the deployment of integrated product teams (IPTs) sometimes known as integrated product development teams (IPDTs).

Product Systems Engineering

Product systems engineering strives for the efficient use of company resources in order to achieve business objectives and deliver a quality product. Product systems engineering activities range from concept to analysis to design and determine how conceptual and physical factors can be altered to manufacture the most cost-effective, environmentally friendly product that satisfies customer requirements. Engineering the product system requires an interdisciplinary approach that includes an analysis of the product and its related elements such as manufacturing, maintenance, support, logistics, phase-out, and disposal; these are all activities which belong to either the realization system or the sustainment system. The proper application of systems engineering and analysis ensures the timely and balanced use of human, financial, and technological assets, and technology investments to minimize problems, harmonize overall results, and maximize customer satisfaction and company profits.

Products are as diverse as the customers that acquire them and there are no universally accepted methods, processes, and technologies (MPTs) for end-to-end analysis of products and their supporting subsystems. Every product needs to adapt existing MPTs based on prior experiences and best practices, such as Toyota (Hitchens 2007), MITRE (Trudeau 2010), and NASA (NASA SELDP 2011). Product systems engineering helps develop the end-to-end analysis of products and sub-systems by performing the following tasks:

- determining the overall scope of needs for the product system;
- defining product and system requirements;
- considering all interactions between the different elements of the product system;
- organizing and integrating engineering disciplines; and
- establishing a disciplined approach that includes review, evaluation and feedback, and ensures orderly and efficient progress.

Constantly evolving needs and requirements, along with constant technology innovations, may render a committed product development obsolete even before deployment. This has led to debate among systems engineering professionals on the need for the systems engineering process to become more rapidly adaptable. Platform-based solutions to resolve some of these challenges (infrastructure as a service, platform as a service, and software as a service) are being studied and proposed (MITRE 2010; Boehm 2010).

Integrated Product Development Process

The integrated product development process (IPDP) starts with understanding market needs and developing a strategy that creates products that satisfy or exceed customer expectations, respond to evolving customer demands, adapt to changing business environments, and incorporate systems thinking to generate novel ideas and co-create value with extensive stakeholders' participation. IPDP is a continuously evolving process that strives to realize products whose cost, performance, features, and time-to-market help increase company profitability and market

share. Magrab, et al. (2010) discussed the IPDP in terms of four different stages; Figure 1 provides a snapshot of an IPDP and the main tasks carried out at each stage.

Stage I: Product Identification

During the product identification stage, the enterprise aims to identify an enterprise-wide strategy that flows down to individual product strategies resulting in a good business investment for the company. During this stage addressable markets for the product are identified in addition to geographical coverage of the product. The developments through this stage result in demonstration of strong customer need, determination of potential markets and geographic scope, the fitness of enterprise core capabilities to the product strategy, business profitability (return on investment, profit & loss), etc.

During this stage an integrated product team (IPT) first develops the IPDP for the project, usually by tailoring a corporate IPDP standard. The IPT assesses required technology innovation, feasibility of existing technologies, estimated time and cost of technology development, and the risks associated with markets, finances and technologies risk, etc. This stage also takes into account inputs from the continuous improvement (CI) process to develop new features and enhancements in existing products to address new market needs or customer demands.

Stage II: Concept Development

The main goal of the Concept Development stage is to generate feasible concepts for the potential product and develop MPTs that will satisfy the product's performance goals of economic viability and customer satisfaction. These concept designs must ensure that the company's core competencies can satisfy the requirements to produce the products while taking into account the market viability, manufacturability, and technical feasibility through an extensive analysis of alternative process.

During this stage SE supports the IPT in identifying different operational scenarios and modes of operation, functional requirements of the products, technology and performance risks, and the main components of the products and required interfaces among them, etc. This stage involves a highly interactive and iterative exchange of concepts among several IPTs and, depending on complexity of the products, a Systems Engineering Integration Team may be required to ensure analysis of all the possible solutions. During this stage inputs from the CI process helps analyze new technologies/processes including upgrades to existing technologies and create products that result in enhanced customer experiences.

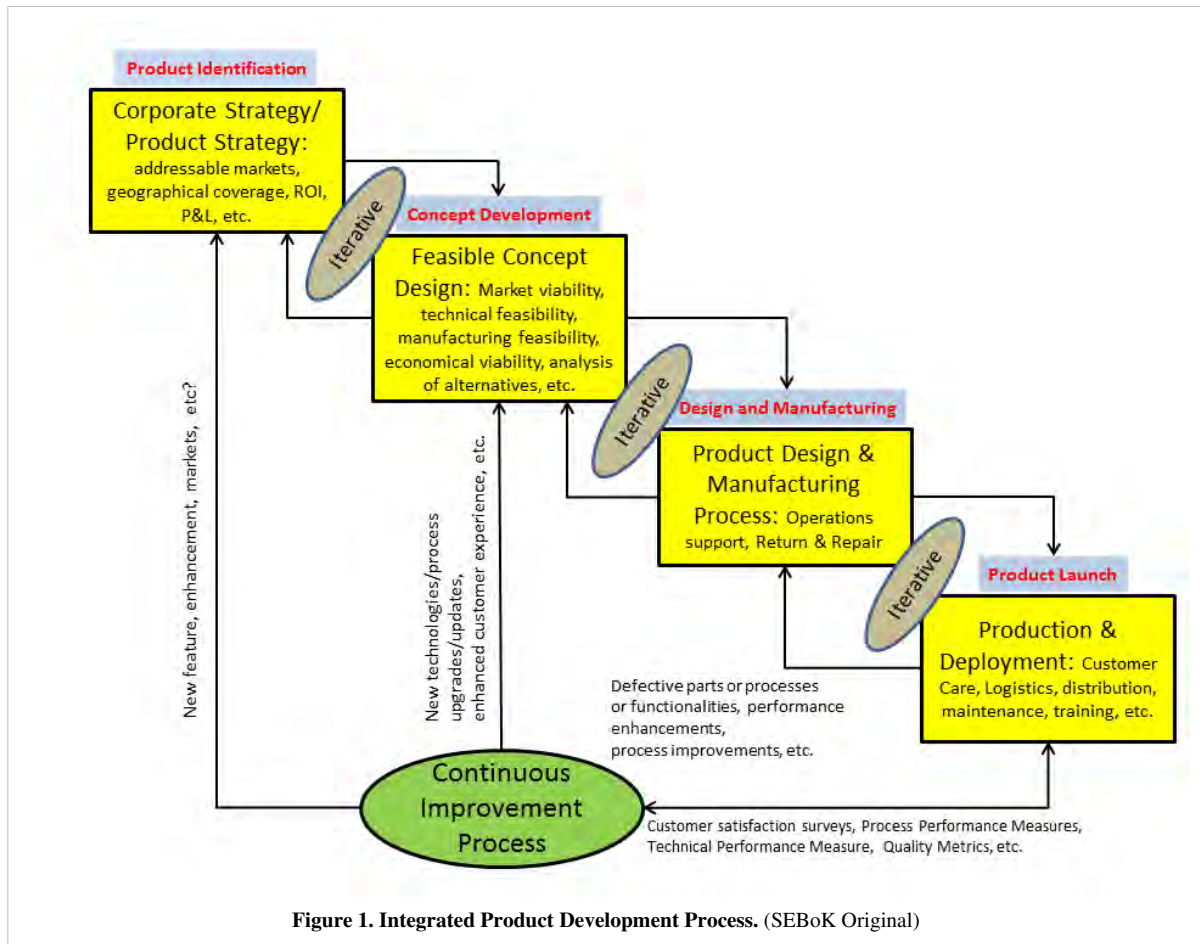
Stage III: Design and Manufacturing

During the design and manufacturing stage the actual product is realized and manufactured. This stage starts with creating engineering drawings for the product, product configuration items specs, "design for X" (DFX), manufacturing design plans, production plans and schedules, test production run to ensure that the product meets customer requirements and quality criteria, and a plan for full production, logistics and distribution.

During this stage the product design & manufacturing engineering team works closely with operations managers to create MPTs to manage the technical effort for the product from an end-to-end perspective. Some of the SE activities during this stage include product integration, verification and validation plans; modeling, simulation, test & evaluation of the product system under critical scenarios; launch readiness plans including end-user test plans, operational readiness, etc. During this stage MPTs are developed and documented for proper handling of defective parts, processes, or functionalities. The CI process inputs include product and process performance enhancements and sustained life-cycle operations support.

Stage IV: Product Launch

During the product launch stage, the product is delivered to its potential markets. During production and deployment, MPTs are developed to ensure that the product meets its quality goals, satisfies customer requirements, and realizes the business plan goals. This requires provisions for customer care, logistics, maintenance, training etc., and a CI process to monitor product and product system technical performance and product quality. The CI process is realized through extensive data collection using customer satisfaction surveys and remotely or manually observing, recording, and analyzing process performance metrics, technical performance measure, quality metrics, etc.



Relationship between Product Systems Engineering and Technology Development

As technological advancement accelerates, product life cycles become shorter, especially for high technology products. As a result, enterprises risk having outdated or obsolete products that have lost pace with markets trends, technology trends, or customer expectations.

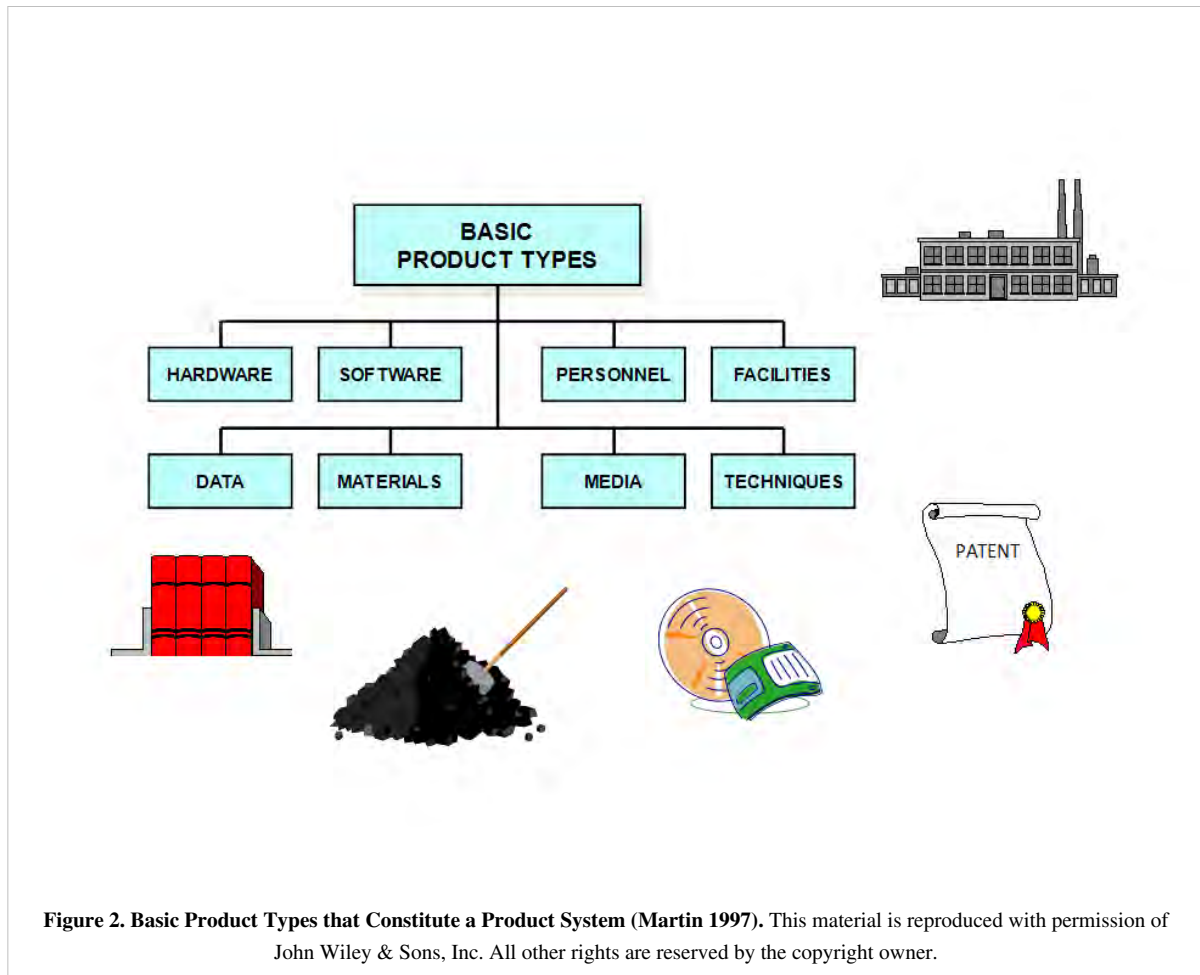
Product systems engineering should bring awareness of technology changes and trends to the analysis of new product ideas or innovations. This affects the time and cost inputs into the technical feasibility analysis of the product. The result should include a road map of required technology developments, which is then used to create the overall road map for the new product offering.

In these cases, new product ideas impose requirements on new technology developments.

On the other hand, when technology developments or breakthroughs drive product innovation or the generation of new markets, the technology developments may also generate requirements on product features and functionalities. Factors which dictate decisions about introducing products include the technology readiness levels (TRL), the integration readiness levels (IRL), the manufacturing readiness levels (MRL), the system readiness levels (SRL), and

the operational readiness of the enterprise to launch the product system. See the "Readiness Levels" section in the Product Systems Engineering Special Activities article.

Understanding the entities (i.e., components or elements) that compose the product is not a trivial task for systems engineers. It is not unusual for a new product to require developments in several technologies, including new materials, electronic components, software, maintenance and repair procedures, processes, or organizational structures. All of these developments must be factored into the IPDP for the successful deployment and proper use of the product.



Product Type Examples

Examples of each product type are shown below (Martin 1997).

Table 1. Product Types (Martin 1997). This material is reproduced with permission of John Wiley & Sons, Inc.

Type	Examples
Hardware	Computer processor unit, radar transmitter, satellite payload, telephone, diesel engine, data storage device, network router, airplane landing gear
Software	Computer operating system, firmware, satellite control algorithm, robot control code, telephone switching software, database application
Personnel	Astronaut, computer operator, clerk, business executive, Laika (the cosmonaut dog), bus driver, cashier, maintenance technician
Facilities	Space rocket launch pad, warehouse building, shipping docks, airport runway, railroad tracks, conference room, traffic tunnel, bridge, local area network cables
Data	Personnel records, satellite telemetry data, command and control instructions, customer satisfaction scores
Materials	Graphite composite, paper, gold, concrete, stone, fiberglass, radar absorption material, clad metals, integrated circuit substrate, magnetic memory core
Media	Data storage media (tape, disc, memory card), signal transport media (twisted pair wire, fiber optic cable, RF spectrum), communications media (television, radio, magazines), social media (blogs, Twitter, Facebook)
Techniques	Soldering, trouble ticket response process, change notice handling, telephone answering protocol, project scheduling, data sorting algorithm

Materials could be thought of as basic raw materials, like steel, or as complex materials, like clad metals, graphite composites, or building aggregate material. Personnel are not normally thought of as a “product,” but that can change depending on the type of system in question. The National Aeronautics and Space Administration (NASA) space program “system” certainly produces astronauts. When personnel are considered product(s), it is not usually possible to simply find and hire personnel with the requisite knowledge, skills, and experience. These personnel “products” can often be developed using a product SE approach (Martin 1996). For example, you could specify requirements (i.e., required knowledge, skills, and experience) for each person that is part of the system. Interfaces can be specified for each person, and an assessment can be made as to the maturity of each person (i.e., each potential product). These are a few examples of how product SE can be applied to personnel products.

In enterprise systems engineering, we may need education and training systems to make up a part of our personnel system in order to produce people with the right competencies and capabilities.

References

Works Cited

- Academy of Program/Project and Engineering Leadership (APPEL). 2009. *NASA's Systems Engineering Competencies*. Washington, D.C., USA: US National Aeronautics and Space Association. Available at: https://www.nasa.gov/offices/oce/appel/pm-development/pm_se_competency_framework.html ^[2].
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.
- Boehm, B. 2010. *Systems 2020 Strategic Initiative*. Hoboken, NJ, USA: Systems Engineering Research Center (SERC), SERC-2010-TR-009.
- Grady, J. 2010. *Systems Synthesis - Product and Process Design*. Boca Raton, FL, USA: CRC Press.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

- Magrab, E., S. Gupta, P. McCluskey, and P. Sandborn. 2010. *Integrated Product and Process Design and Development - The Product Realization Process*. Boca Raton, FL, USA: CRC Press.
- Martin, J.N. 1997. *Systems Engineering Guidebook: A process for developing systems and products*, 1st ed. Boca Raton, FL, USA: CRC Press.
- MITRE. 2010. *Platform as a Service: A 2010 Marketplace Analysis*, Cloud Computing Series. Bedford, MA, USA: Systems Engineering at MITRE.
- Morse, L., and D. Babcock. 2007. *Managing Engineering and Technology*, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall.
- Phillips, F. 2001. *Market Oriented Technology Management: Innovating for Profit in Entrepreneurial Times*. New York, NY, USA: Springer.
- Trudeau, P.N. 2010. *Designing and Enhancing a Systems Engineering Training and Development Program*. Bedford, MA, USA: The MITRE Corporation.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. New York, NY, USA: John Wiley & Sons.

Primary References

- Grady, J. 2010. *Systems Synthesis - Product and Process Design*. Boca Raton, FL, USA: CRC Press.
- Magrab, E., S. Gupta, P. McCluskey, and P. Sandborn. 2010. *Integrated Product and Process Design and Development - The Product Realization Process*. Boca Raton, FL, USA: CRC Press.
- Martin, J.N. 1997. *Systems Engineering Guidebook: A process for developing systems and product*, 1st ed. Boca Raton, FL, USA: CRC Press.

Additional References

- Academy of Program/Project and Engineering Leadership (APPEL). 2009. *NASA's Systems Engineering Competencies*. Washington, D.C., USA: US National Aeronautics and Space Association. Available at: https://www.nasa.gov/offices/oce/appel/pm-development/pm_se_competency_framework.html ^[2].
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.
- Boehm, B. 2010. *Systems 2020 Strategic Initiative*. Hoboken, NJ, USA: Systems Engineering Research Center (SERC), SERC-2010-TR-009.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- MITRE. 2010. *Platform as a Service: A 2010 Marketplace Analysis*, Cloud Computing Series. Bedford, MA, USA: Systems Engineering at MITRE.
- Morse, L., and D. Babcock. 2007. *Managing Engineering and Technology*, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall.
- Phillips, F. 2001. *Market Oriented Technology Management: Innovating for Profit in Entrepreneurial Times*. New York, NY, USA: Springer.
- Trudeau, P.N. 2010. *Designing and Enhancing a Systems Engineering Training and Development Program*. Bedford, MA, USA: The MITRE Corporation.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. New York, NY, USA: John Wiley & Sons.
-

References

- [1] <https://www.businessdictionary.com/definition/product-development.html>
- [2] https://www.nasa.gov/offices/oc/e/appeal/pm-development/pm_se_competency_framework.html

Product as a System Fundamentals

- Lead Author:
- Ricardo Pineda

This article introduces fundamental concepts of product systems.

Product Elements and Connections

Product systems consist of product elements and two kinds of connections: connections among elements, and connections between elements and things in the system environment. That portion of the environment that can be influenced by the system or that can influence the system is called the “context.”

Connections between elements contain interactions and relationships (Hybertson 2009). A connection is more than a mere interface.

Interactions occur across *interfaces* between the elements inside or outside the system, and can be defined as exchanges of data, materials, forces, or energy. Connections with an interactive nature can be represented in various engineering artifacts: schematic block diagrams, data flow diagrams, free body diagrams, interface control diagrams, port specifications, energy transfer diagrams, and so on. Product systems engineering (PSE) usually defines interactions in an interface control document, interface design document, interface requirements document, or the equivalent.

Connections also encompass relationships between elements. These relationships can be spatial, motion-related, temporal, or social.

Spatial relationships:

- one element is underneath another
- two elements are x units apart
- one element is inside another

Motion-related relationships:

- the relative velocity of two elements is v units
- the relative acceleration between two elements is a units

Temporal relationships:

- one element exists before another
- two elements must exist at the same time
- two elements must be separated in time by t units

Social relationships:

- a human element feels a particular way about a system
- a human element owns another (non-human) element
- a human element understands the operation of a system in a particular way

Relationships that are not about time can still change over time. For example, an element that is inside another element during one mode of operation can be outside of it during a different mode of operation. Therefore, one



should not assume that non-temporal relationships are necessarily static in time.

Relationships can be represented in engineering artifacts, including the timing diagram, timeline diagram, mission reference profile, capability road map, and project schedule chart.

Social relationships include the implicit or explicit social obligations or expectations between the roles that human elements play in a system. These roles may be assigned different authorities, responsibilities, and accountabilities. See the discussion on organization behavior in the article Team Dynamics. Organizational behavior theories and human factors may need to be considered when engineering such a product system.

There can also be social relationships between the humans and the non-human elements of the system. This may involve how the human “feels” about things in the system or perhaps even the system as a whole. Humans inside or outside the system-of-interest may have different degrees of “understanding” with respect to how the system operates, its limitations and capabilities, and the best way to operate it safely and effectively. The “ownership” relationship can be important in determining things like who can operate or change some configuration or mode of the system.

There are many such social relationships in a product system that are often ignored or misunderstood when performing PSE. Social relationships can affect the overall performance or behavior of a product system to the point of determining its success or failure.

Core Product and its Enabling Products & Operational Services

A variety of systems (themselves being products or services) enable the development, delivery, operation and eventual disposal of a product, as shown in Figure 1. The concept of enabling systems is defined in the ISO/IEC 15288 standard (2015).

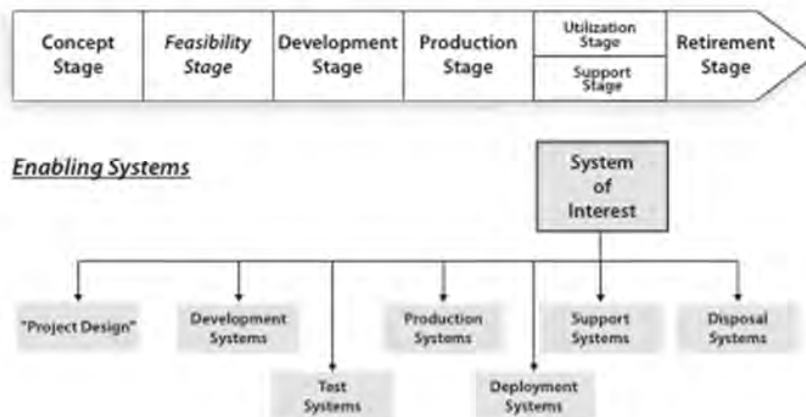


Figure 1. Example of Enabling Systems (Lawson 2010). Reprinted with permission of Harold "Bud" Lawson. All other rights are reserved by the copyright owner.

In the figure, the system-of-interest (SoI) goes into operation as a delivered product or offered service in the utilization stage while maintenance and logistics are provided (by a product sustainment system) simultaneously in the support stage. These two stages are commonly executed in parallel, and they offer opportunities to observe any need for changes in the properties of the product or service or how it is operated and supported. Making changes iterates the life cycle and results in new or improved products or features.

The delivered product and its enabling systems collectively form a wider system-of-interest (WSOI). The project design enabling system is an enterprise based system asset that establishes the strategy and means of organizing the projects to be executed along the life cycle. In many larger organizations, this type of enabling system is institutionalized and can be based upon recommendations of the Project Management Institute (PMI).

Product systems should be viewed as enabling service systems. That is, once deployed, a product system provides a service that contributes to an enterprise's operations. To the acquirer, the SoI provides operational services to users. This is true at several levels:

- Hardware and software products are used to enable the provisioning of service systems,
- Enterprises incorporate products as system assets and use them to enable operations, and
- Provided products are integrated into the system of systems.

Product Architecture, Modeling, and Analysis

IEEE standard 1471-2000 defines architecture as "the fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution" (IEEE 2000). Similarly, ISO/IEC 42010-2011 defines architecture as "fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution" (ISO/IEC 2011).

A product's purpose (stakeholder's need) is realized by a product system (the SoI). Because product systems are composed of different entities (components, assemblies, subsystems, information, facilities, processes, organizations, people, etc.) that together produce the results unachievable by any of the entities alone, architecting the product is based on a whole systems approach. To architect with a whole systems approach means to define, document, design, develop, manufacture, distribute, maintain, improve, and to certify proper implementation of the product's objectives in terms of the functional (the "what"), the behavioral (the use, or intended operations), the logical (interaction and relationships between entities) and the physical constructs (Wasson 2006; Maier 2009; Blanchard and Fabrycky 2011).

The system architect starts at the highest level of abstraction, concentrating on needs, functions, systems characteristics and constraints (concerns) before identifying components, assemblies, or subsystems. This is the systems view, and it is used to represent the stakeholder's market service description or the concept of operations (understanding of the opportunity/problem space).

Next to be documented, as needs become better understood, are architectural descriptions at different levels of abstraction, representing various stakeholders interests. These are the architecture models. They define the possible solution spaces for the product purpose in the form of detailed system, operational, behavioral, and physical requirements of the product system.

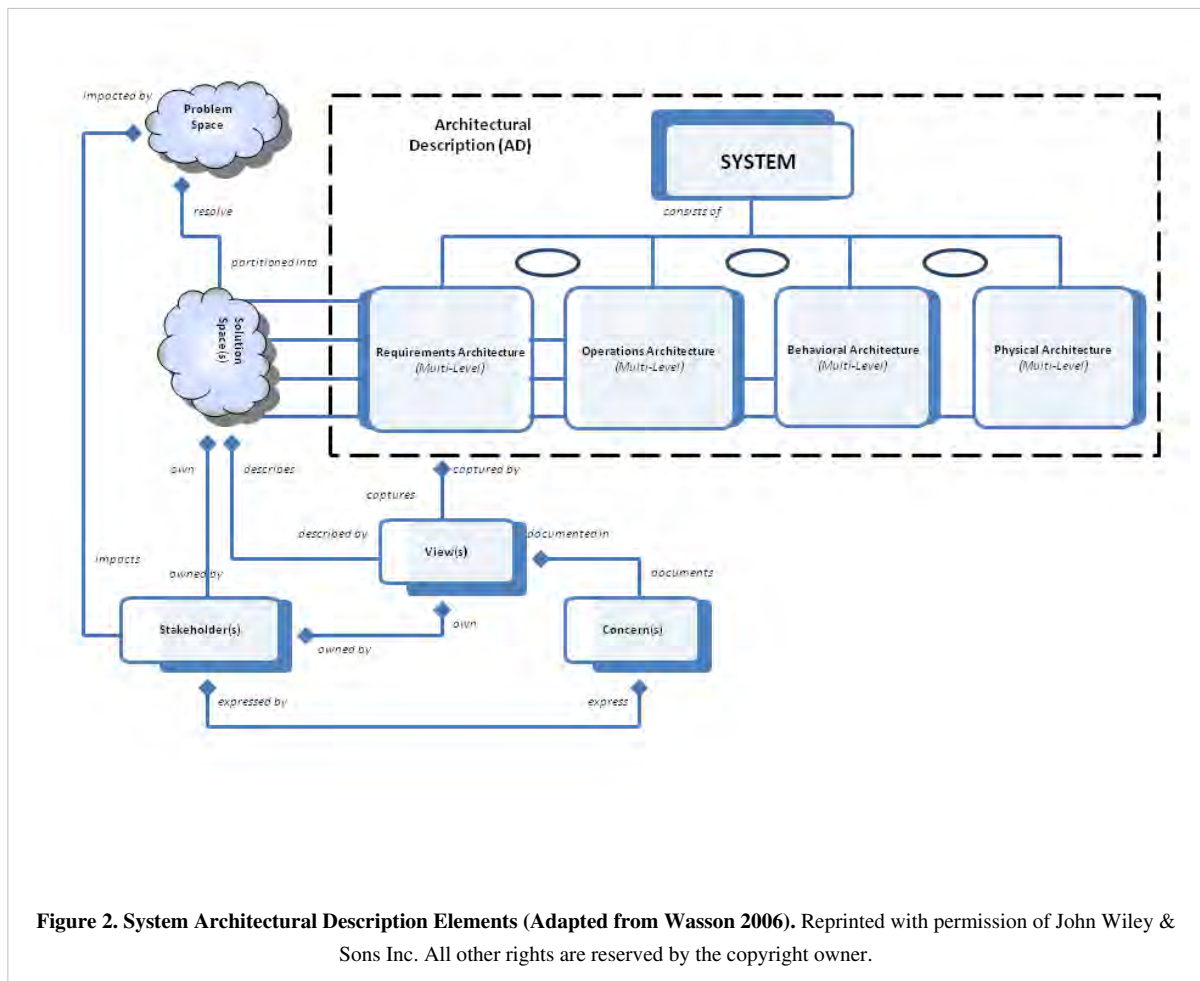
Different modeling techniques are then used to analyze different types of requirements. For operational scenarios and different modes of operation, there are hierarchical decomposition and allocation, architectural block diagrams (ABD), functional block diagrams (FBD), functional flow block diagrams (FFBD), and use case diagrams. For interactions and relationships among hardware and/or software components there are sequence diagrams, activity diagrams, state diagrams, and data flow diagrams. See (Maier 2009) Chapter 8 for an introduction to models and modeling.

Analysis of the solution space makes it possible to produce detailed technical specs, engineering drawings, blueprints, software architectures, information flows, and so on, that describe the entities in the product system. An entity's requirements bound its attributes and characteristics, levels of performance, operational capabilities, and design constraints. During design and integration, entity characteristics can be traced back to requirements (requirements traceability being a key aspect of SE). Verification and validation plans created during the requirements phase are the basis of testing certification that the product does what it was intended to do.

Overall, what occurs is the transformation of a set of requirements into products and processes that satisfy the stakeholder's need. The architecture is represented by a set of models that communicate an integrated view of the product's intent and purpose, and the interactions and interfaces required among all the different participating entities. The product's purpose is articulated in terms of business objectives (market, cost, functionality, performance, and time to deliver). The set of models includes sufficient variety to convey information appropriately to the stakeholders, designers/developers, specialty engineering, operations, manufacturers, management, and marketing and sales personnel.

Different architecture frameworks have been developed to guide product teams in defining the product architecture for commercial and for public enterprises. In general, an architecture framework describes a "view," meaning a "collection of models that represent the whole system from the perspective of a set of related stakeholder concerns." Prime examples of architecture frameworks are the Zachman framework (Zachman 1992), The Open Group Architecture Framework (TOGAF) (TOGAF 2011), the Enhanced-Telecom Operations Map (e-TOM), just to mention a few in the commercial sector. In the case of public enterprises a few architecture frameworks include the Department of Defense Architecture Framework (DODAF 2.0) (DoD 2010), the Federal Enterprise Architecture Framework (FEAF) (FEA 2001), the British Ministry of Defense Architecture Framework (MODAF) (MOD 2004), etc.

Differences between acquired products and offered products play an important role in defining product system requirements. Acquired products are life cycle-managed directly by the acquirer; for instance, acquired defense systems are defined, developed, tested, owned, operated, maintained and upgraded by the defense agency. See the article Product Systems Engineering Key Aspects within this KA.



Specialty Engineering Integration

The INCOSE *SE Handbook* defines specialty engineering as:

“Analysis of specific features of a system that requires special skills to identify requirements and assess their impact on the system life cycle.”

Areas of expertise that fall under this umbrella definition include logistics support, electromagnetic compatibility analysis, environmental impact, human factors, safety and health analysis, and training. The unique characteristics, requirements, and design challenges of a system-of-interest all help determine the areas of specialty that apply.

A number of specialty engineering areas are typically important to systems engineers working on the development, deployment, and sustainment of product systems. For example, logistics support is essential for fielded product systems that require maintenance and repair. The delivery of services, materials, parts, and software necessary for supporting the system must all be considered very early in the development activity. These factors should usually be considered before the system requirements and concept definition are complete. To integrate these specialty disciplines sufficiently early on, the systems engineer needs to know what specialties relate to the system under development, how they relate to the systems engineering process, and how to integrate them into the life cycle process.

For product systems with significant hardware content and that operate in challenging environments, the following specialty engineering areas must usually be considered:

- manufacturability,
- reliability and maintainability,
- certification (essential where human safety is an issue),

- logistics support,
- electromagnetic compatibility (if they radiate),
- environmental impact,
- human factors,
- safety and health, and
- training.

The relationship of these specialty areas to the systems engineering process must be understood and considered. The key aspects of the relationship are:

- when the specialty needs to be considered,
- what essential data or information it provides,
- the consequences of not including the specialty in the systems engineering process, and
- how the systems engineers should interact with the specialty engineers.

Grady (2006) provides an overview, with references, for many of the specialty engineering disciplines, including reliability engineering; parts, materials, and process engineering (PMP); maintainability engineering, availability, producibility engineering, design to cost/life cycle cost (DTC/LCC), human factors engineering, corrosion prevention and control (CPC), system safety engineering, electromagnetic compatibility (EMC) engineering, system security engineering, mass properties engineering, and environmental impact engineering.

Eisner (2008) lists specialty engineering as one of the “thirty elements” of systems engineering. “Specialty engineering refers to a set of engineering topics that have to be explored on some, but not all, systems engineering efforts. In other words, some systems involve these special disciplines and some do not. Examples of specialty engineering areas include electromagnetic compatibility and interference, safety, physical security, computer security, communications security, demand forecasting, object-oriented design, and value engineering.” Some of what we consider specialty engineering in the present article, Eisner includes among his “thirty elements” of systems engineering, but not as part of the specialty engineering element.

There is no standard list of specialty engineering disciplines. What is considered specialty engineering varies according to the community to which the systems engineering belongs, and sometimes to the preferences of the customer.

References

Works Cited

- ANSI/IEEE. 2000. *Recommended practice for architectural description for software-intensive systems*. New York, NY, USA: American National Standards Institute (ANSI)/Institute of Electrical and Electronics Engineers (IEEE), ANSI/IEEE 1471-2000.
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.
- Eisner, H. 2008. "Chapter 7. Essentials of Project and Systems Engineering Management," in *The Thirty Elements of Systems Engineering*, 3rd ed. New York, NY, USA: John Wiley & Sons.
- Grady, J. 2006. "Chapter 3.7. System Requirements Analysis," in *Specialty Engineering Requirements Analysis*. New York, NY, USA: Academic Press.
- Grady, J. 2006. *System Requirements Analysis*. New York, NY, USA: Academic Press.
- Grady, J. 2010. *Systems Synthesis - Product and Process Design*. Boca Raton, FL, USA: CRC Press.
- Hybertson, D. 2009. *Model-oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boston, MA, USA: Auerbach Publications.
-

- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- ISO/IEC/IEEE. 2015. *Systems and software engineering - system life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC 15288:2015.
- ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications.
- Maier, M., and E. Rechtin. 2009. *The Art of Systems Architecting*, 3rd ed. Boca Raton, FL, USA: CRC Press.
- MOD. 2004. *Ministry of Defence Architecture Framework (MODAF)*, version 2. London, UK: UK Ministry of Defence.
- The Open Group. 2011. *TOGAF*, version 9.1. Hogeweg, The Netherlands: Van Haren Publishing. Accessed August 29, 2012. Available at: <https://www.opengroup.org/ogsys/jsp/publications/PublicationDetails.jsp?catalogno=g116>.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.
- Zachman, J.A. 1992. "Extending and Formalizing the Framework for Information Systems Architecture." *IBM Systems Journal*. 31 (3): 590-616.

Primary References

- Eisner, H. 2008. "Chapter 7. Essentials of Project and Systems Engineering Management," in *The Thirty Elements of Systems Engineering*, 3rd ed. New York, NY, USA: John Wiley & Sons.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.

Additional References

- ANSI/IEEE. 2000. *Recommended practice for architectural description for software-intensive systems*. New York, NY, USA: American National Standards Institute (ANSI)/Institute of Electrical and Electronics Engineers (IEEE), ANSI/IEEE 1471-2000.
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.
- Grady, J. 2006. "Chapter 3.7. System Requirements Analysis," in *Specialty Engineering Requirements Analysis*. New York, NY, USA: Academic Press.
- Grady, J. 2006. *System Requirements Analysis*. New York, NY: Academic Press.
- Grady, J. 2010. *Systems Synthesis- Product and Process Design*. Boca Raton, FL, USA: CRC Press.
- Hybertson, D. 2009. *Model-oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boston, MA, USA: Auerbach Publications.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- ISO/IEC. 2008. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE
-

15288:2008.

ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.

Maier, M., and E. Rechtin. 2009. *The Art of Systems Architecting*, 3rd ed. Boca Raton, FL, USA: CRC Press.

Zachman, J. 2008. "John Zachman's Concise Definition of The Zachman Framework™." Zachman International Enterprise Architecture. Accessed August 29, 2012. Available at: <https://www.zachman.com/about-the-zachman-framework>.

Business Activities Related to Product Systems Engineering

- Lead Author:
- Ricardo Pineda

This topic discusses the interfaces between product systems engineering and other 'back office' and management activities that take place in an enterprise.



Marketing, Product Life Cycle Management, & Quality Management

Product systems engineering (PSE) includes critical and robust interfaces with related business activities, such as marketing, product life cycle management (PLM), and quality. Traditionally, PLM has been a critical stage in the integrated product development process (IPDP) and continues to be an important tool for life cycle management after product deployment. PLM provides an important component of the PSE end-to-end view. The other component is the “breadth” component that captures everything relevant to the system at each life cycle stage. Recently, the focus has started to shift from the idea of managing just the life of the product, to an expanded view that includes the management of product-lines (families) or product platforms themselves. This provides an increase in sustainability, flexibility, reduced development times, and important reductions in costs as new or enhanced products are not launched from scratch every time.

PSE also includes interfaces with the marketing function; in particular, PSE works closely with the business and market development organizations to elicit product needs and intended operations in target markets to define product roll-out and possible phases of product introduction. Analysis of the market is critical during the entire product life cycle from conception through retirement with the understanding that each life cycle phase requires very different marketing approaches. During concept development, marketing has to help determine the potential market, the addressable market segments, define products, and product/innovations requirements for those markets. During the product introduction stage, marketing has to create demand and prompt early customers to try the product. During the growth and maturity phases, marketing has to drive public awareness, develop the brand, and differentiate the product and its features and feature releases to compete with new market entrants. During saturation, marketing must help manage diminishing volumes and revenues as focus shifts from top line (increased market share) to bottom line (increased production and distribution efficiencies) considerations. See the article on Procurement and Acquisition.

The links between PSE and quality are just as critical. The relationships between PSE and quality also reflect the broad view which includes the product and opportunity, but also the company's internal goals, processes, and capabilities. Quality schemes which focus on a tangible product have been extensively used historically. More recent approaches that acknowledge and match PSE's holistic view have come into use. Issued during 1988, ISO 9000 is a family of standards which focuses on processes and the organization instead of the product itself. In addition, it calls out specific requirements for the design of products and services. ISO 9001 has served as a "platform" for many other schemes which are tailored to specific domains. A collaborative effort of the International Aerospace Quality Group, AS9100 contains all of the base requirements of ISO 9100 and expands further requirements which are critical and specific to the aviation, space, and defense industries. Similarly QS-16949 is a technical standard based on ISO 9001 but expanded to meet specific requirements in the worldwide automotive industry. PSE should play an important role in the design and implementation of any quality management system. See the article on Quality Management.

Capability Maturity Model Integrated (CMMI) for Development is a process improvement approach whose goal is to help organizations improve their performance. CMMI can be used to guide process improvement across a project, a division, or an entire organization. Although initially used in software engineering and organizational development, CMMI use is spreading to other domains since it provides organizations with the essential elements for effective process improvement. According to the Carnegie Mellon Software Engineering Institute, CMMI describe "an evolutionary improvement path from ad hoc, immature processes to disciplined, mature processes with improved quality and effectiveness." (SEI 2010).

Project Management & Business Development

The end-to-end view mandated by PSE requires strong relationships with project management and business development activities. The 'concurrent' thinking encouraged by PSE necessarily requires multiple projects to move forward in parallel, but with a high level of coordination. In this sense, PSE and project management (see Systems Engineering and Project Management) are two heavily intertwined disciplines which have been shown to generate synergy and added value for the stakeholders.

The systems engineering management plan (SEMP) is the key document covering the activities, milestones, organization, and resource requirements necessary to ensure the system-of-interest accomplishes its intended goals and mission. A key objective of the SEMP, which is usually developed during the conceptual design phase, is to provide the structure, policies, and procedures to foster the optimum management of projects and activities needed for system development and implementation (Blanchard and Fabrycky 2011).

An effective and agile PSE function can make important contributions to business development for an enterprise or company. The primary goal of business development activities is to identify new types of business/product/services which are believed to address existing or potential needs and gaps (new markets), to attract new customers to existing offerings, and to break into existing markets. PSE's end-to-end view of the life cycle can support market development by intelligence gathering, feedback on market acceptance or rejection, strategic analysis, and proposition development and campaign development. Finally, PSE should encourage the consideration of several factors within the new product development which may enhance market development. For example, in well-established companies, business development can often include setting up strategic alliances with other, third-party companies. In these instances, the companies may leverage each other's expertise and/or intellectual property to improve the probability for identifying, researching, and bringing to market new businesses and new products. See (Sørensen 2012).

Supply Chain Management & Distribution Channel Management

PSE provides the following information to the supply chain management function in an enterprise:

- product specifications (including intended uses of the product),
- product acceptance criteria (for accepting delivery of the product from the supplier),
- product testing and qualification plans and procedures, including which ones are responsibility of the supplier and which ones are responsibility of the acquirer,
- interface specifications associated with each product,
- supplier certification criteria (including a list of pre-certified suppliers), and
- feedback on quality of products delivered by suppliers.

Supply chain management will, as necessary, manage the identification and certification of qualified suppliers with the concurrence of, and coordination with, systems engineering and product engineers.

PSE provides the following information to the distribution channel management function in the enterprise:

- product specifications (including intended uses of the product),
- product user manuals (including installation and maintenance documentation),
- product packaging (for safe delivery of product and for display in retail channels),
- product qualification data (to prove that product meets its design requirements),
- product certification data (to prove product is certified for safe and secure operation),
- user support instructions, and
- operator certification criteria.

Distribution channel management will, as necessary, manage the identification and certification of qualified distributors with the concurrence of, and coordination with, systems engineering and product engineers.

Capability Management & Operations Management

Capability is defined in various ways, but each definition is consistent with the notion of "the ability to do something useful." Products and services are acquired by end users to enable and improve their operational capability to let them do something useful, whether in a military context (e.g., weapon systems improve the capability to conduct effective military operations), or a social context (e.g., a car may improve the ability to satisfy the transport needs of a family). Users acquire products (e.g., military equipment, cars, "productized" service offerings from airlines and taxi companies, etc.) to contribute to satisfying their capability needs.

Capability management involves identifying and quantifying capabilities (existing, new, or modified) that will be needed to meet enterprise goals, as well as selecting a coherent set of product and services across all components of capability that will be integrated to provide the needed capabilities. So normally, requirements for "product systems" are derived from capability management. Capability management is likely to include trade-off processes to make the best use of existing products or low-risk evolutions of them, and conversely identifying when a capability need can only be satisfactorily met by a new-generation product, or even a new type of product altogether. In some cases, new offered products or disruptive technologies (e.g., jet engine, nuclear weapons, and internet) create opportunities for new or improved capabilities, in which case capability management focuses on ensuring that all needed components of capability are put in place to exploit the opportunity provided by the new product or technology. See Capability Engineering.

Operations management uses an integrated set of product systems to deliver value to the enterprise and its stakeholders. Operations management involves bringing new product systems into operation, normally while maintaining business continuity, so transition plans and relevant metrics are critical; next, operations management addresses some of the following: working up to full operational efficiency across all components of capability, coping with incidents, contingency plans to deal with major disruptions, adjusting the system to cope with new ways of working and to deliver new services to meet new enterprise requirements and accommodate new product systems

entering service, and eventually planning transitions out of service or major in-service upgrades. PSE supports operations management by defining all dependencies for successful operation on other systems and services, and by providing ongoing engineering support for spares and repairs, obsolescence management, and system upgrades. Systems engineering in the in-service phase has been analyzed (Elliott et al. 2008) and is best viewed as the same basic systems engineering process conducted at a much higher tempo (Kemp and Linton, 2008) and requiring detailed understanding of constraints imposed by the current environment and usage. Configuration management and configuration status accounting during operation is very important for high value and high integrity systems to ensure that any changes are designed to fit the "as-is" system, which may be significantly different from the "as-originally intended" specification and design.

Product Engineering, Assembly, Integration, & Test

Product engineering typically results in an engineering model that is used as the "blueprint" for assembling, integrating, and testing (AIT) a product system. These AIT activities may be performed on prototype versions, as well as final production versions to be delivered to end users. There is significant experience in domain specific industries in performing AIT for complex products. Unfortunately, very little is written in the general literature. Wasson (2006) and de Jong (2008) cover some of these aspects. See also System Integration and System Verification.

For software products, the collection of code modules are integrated via some form of integration program (typically called "make"). The integrated modules are then subjected to tests to exercise the various potential paths through the software. Since software can be easily changed, it is common to use some form of regression testing based upon test suites in order to verify software correctness. Another common means of testing is by fault injection as described by Voas and McGraw (1998).

Manufacturing, Test, & Certification

Systems engineers usually work with manufacturing indirectly through the electrical and mechanical design teams. There are times in the development cycle when a direct interface and working relationship between systems engineering and manufacturing is appropriate and can improve the probability of program and system success. Early in the program the system concept must be examined to determine if it is manufacturable. The requirements and the concept design should be reviewed with the manufacturing engineers to obtain an assessment of the risks associated with the production of the system. If substantial risks are identified, then actions that improve the manufacturing capabilities of the organization, modify the design, and perhaps change the requirements may be needed to reduce the identified risks to acceptable levels. Manufacturing prototypes or proof of manufacture (POM) units may be necessary to reduce the risk and to demonstrate readiness to proceed with the design and the system development. Similarly, the systems engineers must establish that the system will be testable early in the product development phase. The requirements should be mapped to verification methods of inspection, analysis, demonstration, and test before they are released to the design team. All requirements mapped to test must be examined to determine the test methods and the risk associated with accomplishing the necessary tests as part of the product qualification, acceptance, and release process. Where risks are identified, the systems engineers must work with the test engineers to develop the necessary test capabilities.

Product Delivery & Product Support

Most products live much longer in the usage phase than in the development phase. The costs associated with product support are usually greater than the cost of developing the product. These two facts make it very important for the product systems engineer to consider the product delivery and support as part of the earliest activities during development. The design of the product dictates the maintenance and support that will be required. The systems requirements are the first means of influencing the design to achieve the desired product support. If maintenance, reliability, and support requirements have not been defined by the customer, then the systems engineer must define these to achieve the support methods and costs that the customer, users, and the organization responsible for support will find financially acceptable.

References

Works Cited

- de Jong, I. 2008. *Integration and Test Strategies for Complex Manufacturing Machines: Integration and Testing Combined in a Single Planning and Optimization Framework*. Saarbrücken, Germany: Verlag.
- Elliott, B. et al. INCOSE UK Chapter Working Group on Applying Systems Engineering to In-Service Systems, Final Report. Somerset, UK: INCOSE UK Chapter Working Group. 2008. Accessed November 11, 2014 at INCOSE UK https://www.incoseonline.org.uk/Documents/Groups/InServiceSystems/is_tr_001_final_report_final_1_0.pdf.
- Kemp, D., and R. Linton. 2008. "Service Engineering." Proceedings of the 18th Annual International Symposium of the International Council on Systems Engineering, June 15-19, 2008, Utrecht, The Netherlands.
- CMMI Product Team. *CMMI for Development Version 1.3* (CMU/SEI-2010-TR-033). 2010. Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University. Accessed on 10 Nov 2014 at Software Engineering Institute Library <https://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=9661>
- Sørensen, H.E. 2012. *Business Development: a market-oriented perspective*. Hoboken, NJ, USA: John Wiley & Sons.
- Voas, J.M., and G. McGraw. 1998. *Software Fault Injection*. Hoboken, NJ, USA: John Wiley & Sons.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.

Primary References

- de Jong, I. 2008. *Integration and Test Strategies for Complex Manufacturing Machines: Integration and Testing Combined in a Single Planning and Optimization Framework*. Saarbrücken, Germany: Verlag.
- Voas, J.M., and G. McGraw. 1998. *Software Fault Injection*. Hoboken, NJ, USA: John Wiley & Sons.
- Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.

Additional References

- Phillips, F.Y. 2001. *Market-Oriented Technology Management: Innovating for Profit in Entrepreneurial Times*, 1st ed. New York, NY, USA: Springer.

Product Systems Engineering Key Aspects

- Lead Author:
- Ricardo Pineda

Acquired Products versus Offered Products

The emphasis for traditional systems engineering (TSE) is in the provisioning of products and related services that meet stakeholder needs and requirements. For acquired products, an acquirer specifies the needs and requirements, selects a supplier for development and provisioning, and then receives the needed products and services. The acquirer, after acceptance, usually owns, operates, and maintains the product and the support systems supplied by the developer. Offered products are provided by suppliers based on opportunities to develop and offer products and services to potential users of the product based on business objectives usually measured in terms of value addition to the stakeholder.

In the provisioning of product systems and related services, the enterprise owning and provisioning the product and services typically makes agreements with other suppliers to also provide elements, methods, and tools that are used during their entire life cycle. The supplying enterprises, in turn, may make further agreements with suppliers in regards to building a supply chain. The complexities of dealing with supply chains must be accounted for with respect to cost, risk, and schedule and thus can have an impact upon product or service maturity. (See articles under the Systems Engineering Organizational Strategy knowledge area (KA) in Part 5.)



Acquired Products

Specific needs for a product or service typically result in some form of an agreement between the acquirer and a supplier as specified in the agreement processes of ISO/IEC 15288 (2015). The acquirer specifies the need and requirements for the properties of the expected product or service and may or may not place specific requirements upon how the supplier plans to organize their life cycle treatment of the product or system.

The degree of formality involved with the agreement varies and is strongly influenced by whether the customer is a government entity or a commercial entity. Government contracts usually incorporate strict specifications and other unique requirements that are rarely found in commercial contracts. Government acquisition agents often specify design characteristics in addition to functional and performance specifications. Design specifications place constraints on product systems engineering (PSE) by explicitly defining the details of a product's physical characteristics. The government acquirer may also specify how the product is to be designed and developed or how it is to be produced. Government specifications tend to be longer, more detailed, and more complex than functional specifications and much longer than specifications used in a commercial environment.

When contracting with the government or similar enterprises, the PSE must identify disagreements related to the meaning of a particular provision in a contract, and work with contracts to get a written resolution of all ambiguities and issues in the specifications. Failure to do this can lead to legal disputes and government claims of product substitution which can prevent acceptance of the product system and result in financial penalties.

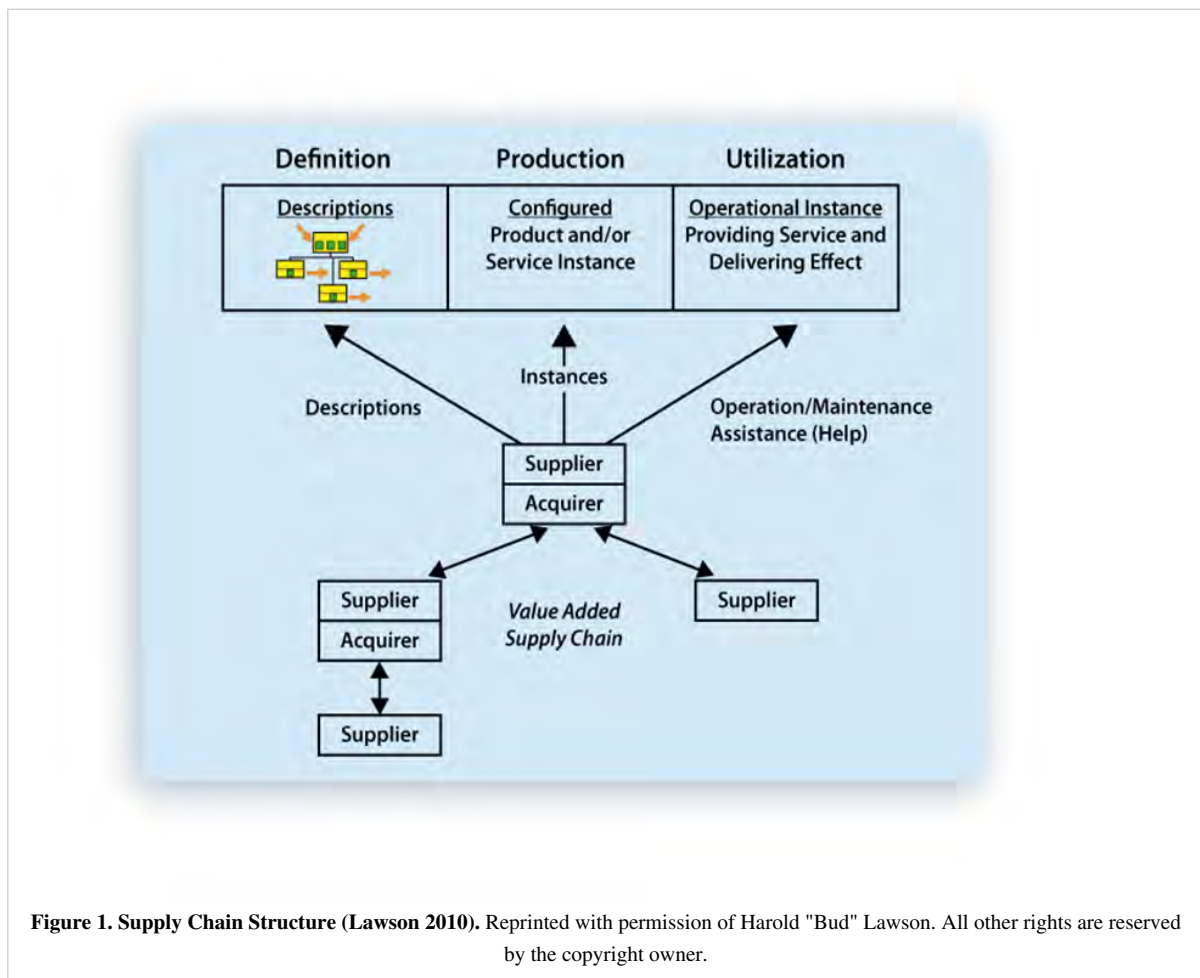
Developing product systems for government customers requires PSE to do a thorough review and perform internal coordination within the enterprise to prevent it from submitting proposals that are non-compliant because the requirements are not fully understood.

Offered Products

Given an opportunity or perceived opportunity, an enterprise may decide to develop and offer products or services to a broader potential marketplace. The properties of the product or service are often determined through surveying and/or forecasting the potential market penetration. The supplier determines the structure and operation of an appropriate life cycle model for achieving the desired results (Pugh 1990).

Supply Chains and Distribution Channels

The supply of products and services to the owner of a product or service that is acquired or offered at various points during the life cycle is vital to success. It is this wider system-of-interest (WSOI) that is the outsourcing holism that must be treated properly in order to provide successful products or services. A portrayal of supply chain structure is provided in Figure 1 below.



In Figure 1, it is important to note that in an agreement with a supplier, the outsourcing can involve delivering complete system description solutions or portions thereof. For example, a supplier could, given a set of stakeholder requirements developed by the acquirer, develop and supply a system that conforms to the architectural solution. The supplier in turn can be an acquirer of portions of their delivered results by outsourcing to other suppliers.

In regards to production, the outsourcing agreement with a supplier can vary from total production responsibility to merely supplying instances of system elements to be integrated by the acquirer. Once again, these suppliers can be acquirers of portions of their delivery from outsourcing to other suppliers.

In regards to utilization, for non-trivial systems, outsourcing agreements can be made with a supplier to provide for operational services, for example, operating a health care information system. Further agreements with suppliers can involve various forms of logistics aimed at sustaining a system product or service or for supplying assistance in the

form of help desks. Once again, suppliers that agree to provide services related to utilization can be acquirers of the services of other suppliers.

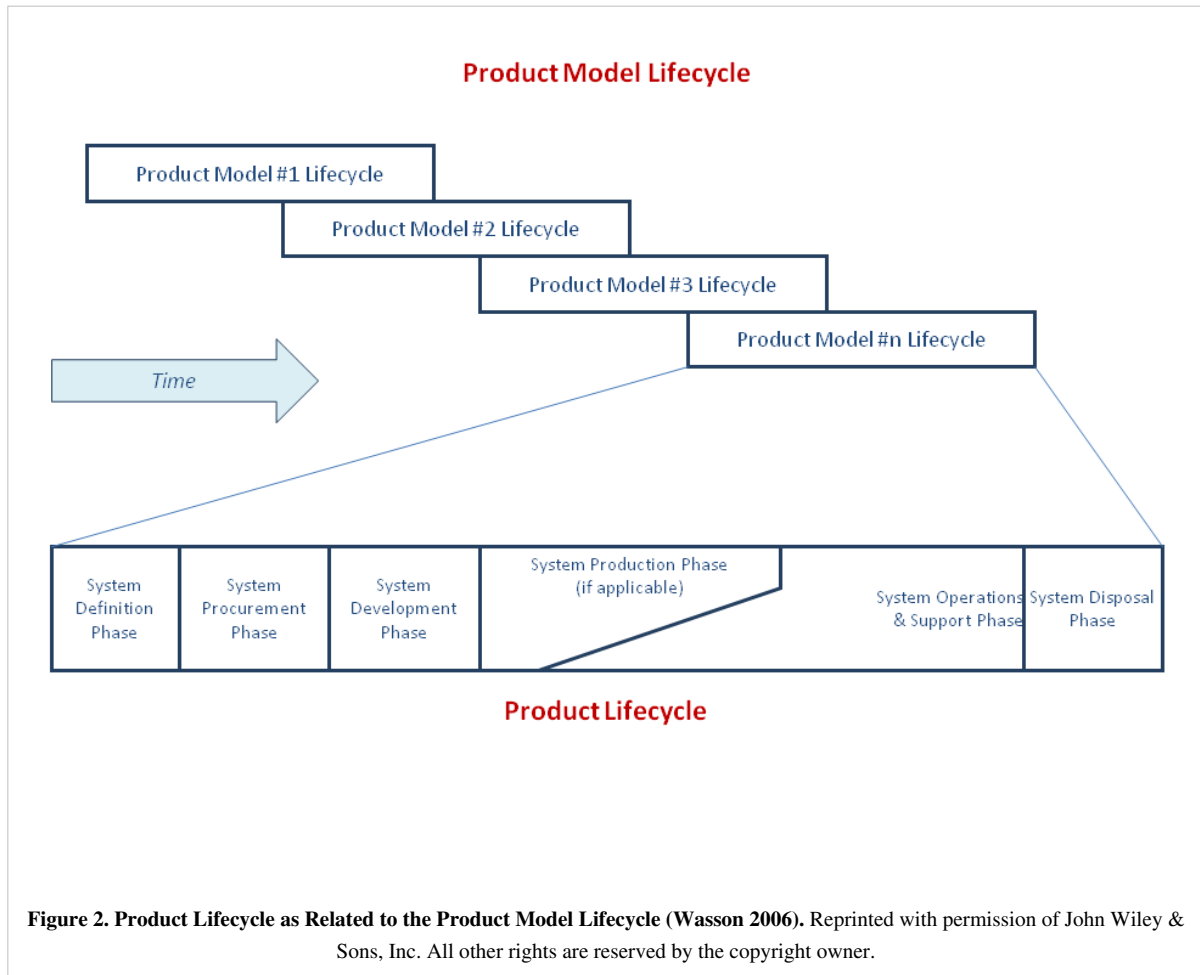
Important to all supply chains is the concept that supplying parties contribute some form of added value to the life cycle of a system-of-interest. The proper management of a supply chain system asset is a vital part of the operations of an enterprise. In fact, the supply chain itself is an enterprise system-of-interest that is composed of acquirers and suppliers as system elements. There is definitely a structure tied together by agreement relationships. Further, the operation of the supply chain results in an emergent behavior. The supply chain system becomes a vital infrastructure asset in the system portfolios of enterprises and forms the basis for extended enterprises.

Similar to a supply chain, the distribution channels for a product system can be a complex web of relationships between the product supplier and various distributors, for example, package delivery companies, warehouses, service depots, wholesale outlets, retail sales establishments, operator training and certification organizations, and so on. The nature of the distribution channels could have a significant impact on the architecture or design of a product system.

PSE may need to include special features in the product design to accommodate for the needs of distribution channel elements, for example, heavy load tie down or lifting brackets, protective shipping packages, retail marketing displays, product brochures, installation manuals, operator certification packages, training materials, and so on. Sometimes it may be necessary to create special versions (or instances) of the product for the training of operators and users for certifying safe or secure operations, for environmental testing and qualification, for product demonstration and user testing, for patent application, for load testing and scalability demonstrations, and for interface fit checking and mass balance certification, to name some examples.

Product Lifecycle and Product Adoption Rates

The life cycle of each product follows the typical incremental development phases shown below (Wasson 2006, 59-65). A particular product to be engineered could be preceded by a previous “model” of that product as shown in the product model life cycle below, and could be superseded later by a newer model of that product. It is worth noting that there is no standard set of life cycle phases. The example below is one of many ways that the phases can be structured.



From an industry perspective, managing a product's life cycle involves more than just the engineering aspects:

Product lifecycle management (PLM) is the process of managing the entire lifecycle of a product from its conception through design and manufacture to service and disposal. PLM integrates people, data, processes and business systems, and provides a product information backbone for companies and their extended enterprise. (CIMdata 2012)

There are many PLM tools and services available for facilitating the development and management of complicated product life cycles and especially for product line management (insert link to product line mgmt section here).

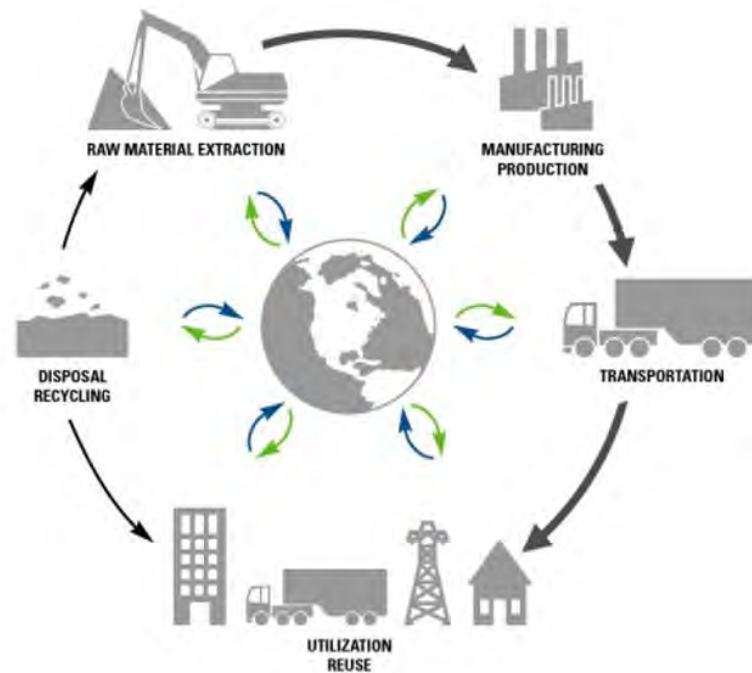


Figure 3. Product Lifecycle from an Industry Perspective. (Source: https://commons.wikimedia.org/wiki/File:Product%E2%80%99s_lifecycle.jpg#filelinks Accessed February 6, 2012. NIST Programs of the Manufacturing Engineering Laboratory, Released by US Federal Government, Public Domain)

The product and product model life cycles are driven by the product adoption rate, illustrated below, that is commonly experienced by most engineered products (Rogers 2003). As products reach market saturation (i.e., on the down slope of the curve below) then there would typically be a new, upgraded version of the product ready for delivery to the marketplace. PSE serves a critical role in determining the best timing for delivery of this new version and the set of features and functions that would be of the greatest value at that time.

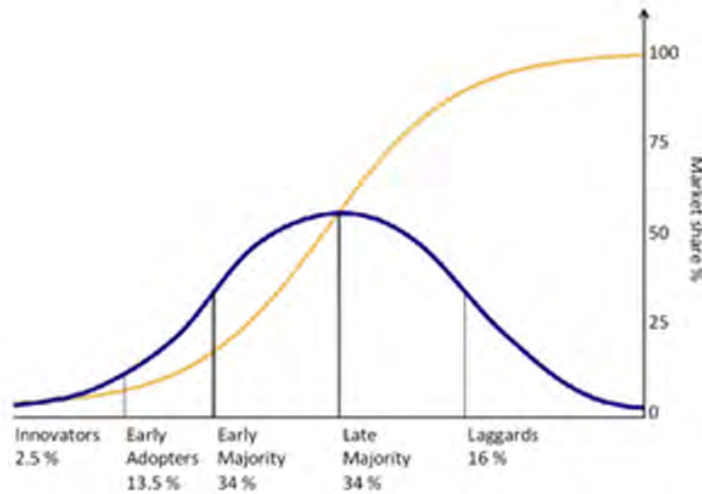


Figure 4. Rogers Innovation Adoption Curve. (Source: <https://en.wikipedia.org/wiki/File:Diffusionofideas.PNG> Accessed February 6, 2012, Released by Tungsten, Public Domain)

Integrated Product Teams and Integrated Product Development

Product systems as discussed throughout this KA mandate the participation of different disciplines for their success during their entire lifecycle from concept to product disposal or retirement. Rapid technology innovations and market pressures in the mid '90s demanded development process (mostly input-output serial) to shorten their development time and development cost, and to improve product quality to remain competitive. For commercial enterprises, the typical development times of 18-24 months to deploy new products into markets of the '90s have in many cases been reduced to 6-12 months and even 3-6 months for the highly competitive leading edge information technology products.

An initial response to these pressures was concurrent engineering. Concurrent engineering is "... a systematic approach to the integrated, concurrent design of products and their related processes, including manufacturing and support to cause developers, from the outset to consider all elements of the product lifecycle from conception through disposal, including quality, cost, schedule and end user requirements." This definition has evolved into the integrated product development (IPD) as more descriptive of this concurrency to describe the continuous integration of the entire product team, including engineering, manufacturing, test, and support through the life cycle. Later, as the importance of the process was recognized, the terminology was modified to integrated product and process development or IPPD (INCOSE 2012).

The INCOSE *Systems Engineering Handbook* v. 3.2.2 provides a good description of the IPT and IPDT process; the different types of IPDT; the steps in organizing and running an IPDT; good examples of IPDT, particularly for acquired systems; and a good discussion on IPDT pitfalls to avoid. (INCOSE 2012)

IPD/IPPD helps plan, capture, execute, and evaluate programs to help design, test, build, deliver, and support products that satisfy diverse stakeholder requirements. IPD/IPPD outlines the necessary infrastructure needed to deploy, maintain, evaluate and continuously improve processes and tools by aligning people (IPTs) and processes to realize product goals (customer satisfaction). The implementation of Integrated Product Development Processes (IPDP) requires an integrated approach for program planning and generally includes the following: Business Strategy, Program Management and Control, Project Planning, Product Requirements and Architecture Development, Product Design and Development, Production and Deployment, Product Verification and Validation, and Operations and Maintenance Support.

At each development stage, there is a decision gate that helps decide if the IPDP is feasible to enter the next stage of product development. IPD utilizes multi-functional IPTs to optimize the individual product and processes to meet overall cost and performance objectives. IPTs are a cross-functional group of people typically including representatives of all the relevant stakeholders in the project, who are brought together for delivering an integrated product to an external or internal customer using relevant IPDP. The main function of the IPTs is to ensure the business, technical and economical integrity and overall success of the product that is delivered to its eventual customer. IPTs carry out tailored IPDPs and follow relevant SE processes to deliver products that satisfy customer needs, overcomes external constraints, and adheres to the overall program strategy.

In the case of commercial enterprises, product development is tightly coupled with business strategies (short and long term), stakeholder value added measured in terms of return on investments (ROI), market presence/coverage, and other strategies as defined by the business objectives. Thus, product integration teams include strategic planners, business managers, financial managers, market managers, quality assurance managers, customer representatives, and end-users, as well as other disciplines required for acquired products. Phillips (2001), Annachino (2003), and Morse (2007) provide good discussions on this topic.

Role of Architectures, Requirements, and Standards

The architectural properties of a product system are influenced by the concerns of the various stakeholders as indicated in the ISO/IEC 42010 standard (ISO/IEC 2011). The stakeholders have various views that they express based on their specific perspective. These views are vital in establishing requirements and are inputs to those responsible for defining the functions, structures, and relationships needed to achieve the desired product or service.

A number of stakeholders have been identified in the discussions of product systems. It would be possible to identify a set of important stakeholders based on the life cycle thinking provided by the ISO/IEC 15288 standard (2015), for example, one such set could consist of owners, conceivers, developers, producers, users, and maintainers as discussed by Lawson (2010). As mentioned earlier, these stakeholders should cooperate at all stages of the life cycle in specifying requirements, verifying that the requirements are met, and validating that the products produced provide needed capabilities.

In addition to the two standards that have been identified, there are a variety of standards related to specialty aspects of products, such as safety and security, as well as standards that are applicable for project management and life cycle considerations, such as requirements and quality management.

Role of Modeling, Simulation, Prototyping, and Experimentation

Modeling, simulation, prototyping, and experimentation are techniques that have the purpose of improving stakeholder knowledge and shared understanding about aspects of the system to de-risk system development and operation before heavy commitment of time and funds. Examples of this are found below:

- Understanding future needs: “Warfighting experiments are the heart of the Army’s warfighting requirements determination process. Progressive and iterative mixes of high fidelity constructive, virtual, and live simulations using real soldiers and units in relevant, tactically competitive scenarios provide Army leaders with future operational capability insights” (US Army 2012),
- Simulation is used to predict and optimize aspects of system performance for which there are good mathematical or logical models before committing the final physical design, and also to verify and validate the system design in scenarios where physical testing is too difficult, dangerous, or expensive, for example, checking the performance envelope of military systems in a wide range of engagement scenarios where test firing thousands of rounds to get statistically valid data is clearly unaffordable, ensuring that the safety features in a nuclear power station will operate correctly in a wide range of stressing scenarios, etc.,
- Prototyping (physical and virtual) is used in a wide variety of ways to check out aspects of system performance, usability, utility, and to validate models and simulations as part of the iterative process of converging on a final design,
- In a manufacturing context, the first units produced are often “prototypes” intended to make sure the production process is working properly before committing to high rate production, and are often not shipped to end users, but used for intensive testing to qualify the design, and
- Simulation is also used extensively for training and marketing purposes. For training, an accurate model of the human machine interface and representation of the operational context allows operators to do most of their training without putting operational hours on the real system enabling them to learn emergency procedures for combat and accident scenarios in a safe and repeatable environment; for example, airline and military pilots now train mainly on simulators. System simulators of various levels of fidelity are used to familiarize customers and end users with the potential characteristics and benefits of the system, available options and trade-offs, and integration issues early in the development and acquisition process.

All of these methods use a variety of physical and mathematical representations of the system and its environment so modeling is an enabler for simulation, prototyping, and experimentation.

Increasing Role of Software in Product Functionality

An important trend in commercial products is the increasing importance of software in an increasingly wide range of products. Everything from phones, cameras, cars, test gear, and medical equipment now has essential functionality implemented in software. Software has had an increasing role in providing the desired functionality in many products. The embedding of software in many types of products accounts for increasing portions of product functionality. In tangible products such as cars, software helps improve functionality and usability (cruise control, climate control, etc.). In intangible products such as insurance, software helps in improving operational efficiency, data accessibility, etc.

The movement toward the internet of “things” where sensing and activating functions are incorporated is now starting to permeate. The use of various software products in proving service is also described in the Service Systems Engineering article.

Recent advancements in IT and software have assisted in their increased use in PSE. Although software development is already a very complex field, the role of software in the development and functionality of products is growing larger each day.

There is a need to broaden the horizons of software engineers to think of problem solving not only in software terms, but also using the systems thinking approach. For this purpose, software engineers need to be able to think critically about the problem and also the possible solutions to the problem or opportunity and its implication for business objectives.

Product Integration and Interface Control

Integration is "the set of activities that bring together smaller units of a system into larger units" (Eisner 2008). Products may consist of several systems, subsystems, assemblies, parts, etc., which have to work together as a whole to deliver the offered product's functionalities at specified performance levels in the intended operations environment. Product integration entails not only the working together of hardware and software components, but also the organization, processes, people, facilities, and the resources for the manufacturing, distribution, maintenance, customer support, sales channels, etc. Grady (2010) groups the above information into three fundamental integration components: functional organization, product integration, and process integration.

PSE plays an important role to ensure well-defined interfaces, interactions, relationships, information exchange, and processes requirements between product components. These requirements are baseline, documented, traced, verified, and validated for the end-to-end Product integration and to maintain and ensure product offering integrity during its life cycle. The systems engineering hierarchical decomposition level allows requirement definition and allocations at different levels of abstraction to define the building blocks of the product architecture; these building blocks are assigned to integrated product development teams (IPDTs) for detailed design and development. The IPDTs or the systems engineering integration team (SEIT) must interact with all involved players to generate appropriate architectural block specifications at the lower tier of development for a product's architectural configuration and configuration tracking. As the building blocks are put together, interface requirements, information exchange, and interaction and relationships among entities are verified against the baseline. Once a configuration item has been built and tested against the baseline, test and verification at higher levels are conducted to obtain the final product configuration; the final product configuration can only be changed by a formal approval from a configuration control board (CCB). Note: the acronym CCB is often used to mean the change control board that, in addition to configuration control, makes decisions of any aspect of a project or an enterprise.

Interface agreements, specifications, and interface designs are usually documented through the interface control documents (ICD) and the interface design descriptions (IDD); in some instances, depending on the complexity of the product and the type of internal and/or external interfaces, an interface control working group (ICWG) is created to analyze and baseline changes to an interface for further recommendation to the CCB.

A configuration item (CI) may be hardware (HWCI), software (SWCI), firmware, subsystems, assemblies, non-development items, commercial off-the-shelf (COTS) items, acquirer furnished equipment, and/or processes. Please see Wasson (2006), Grady (2006), and INCOSE *SE Handbook* v. 3.2.2 for a more detailed description of configuration and interface control.

A product may experience hundreds of changes during its life cycle due to new product releases/enhancements, repair/replacement of parts, upgrades/updates in operating systems, computer infrastructure, software modules, organizational changes, changes in processes and/or methods and procedures, etc. Thus, strong mechanisms for bookkeeping and activity control need to be in place to identify, control, audit, account and trace interfaces, interactions, and relationships between entities that are required to maintain product configuration status (Eisner 2008). The product configuration and CI's are then controlled through the configuration management process.

Configuration Management and Risk Management

Configuration management (CM) deals with the identification, control, auditing, status accounting, and traceability aspects of the product, and broadly covers the book-keeping and control activities of the systems engineering process (Eisner 2001). Any product configuration changes to the baseline (configuration item, operational baseline, functional baseline, behavior baseline) or product baseline are submitted to a configuration control board (CCB) through an engineering change request (ECR) and/or a configuration change request (CCR). The CCB then analyzes the request to understand CI impacts and the feasibility (time and cost) of authorization or rejection of change request(s). The lack of proper control and tracking of CI and product baselines may result in a loss of features, functionality, data, interfaces, etc., leading to backtracking and CI version losses which may affect the offered product. All approved changes will have to be baselined, documented, and tested for backward compatibility and to ensure compliance with the integrated product functionality. Thus, successful implementation and life cycle management of the product mandates a highly disciplined CM process that maintains proper control over the product and its components. Please see the INCOSE *Systems Engineering Handbook* v. 3.2.2 (2012) for a detailed description of the CM Process.

Risk management deals with the identification, assessment, and prioritization of technical, cost, schedule, and programmatic risks in any system. Almost all engineered systems are designed, constructed, and operated under some level of risks and uncertainty while achieving multiple, and often conflicting, objectives. As greater complexities and new technologies are introduced in modern systems, the potential of risks have significantly increased. Thus, the overall managerial decision-making process should involve an extensive cost-benefit analysis of all identified, qualified, and evaluated risks (Haimes 2008). Risk management involves the coordinated and most cost-effective application of resources to minimize, monitor, and control the probability and/or impact of all identified risks within the systems engineering process. The risk management process requires the involvement of several disciplines and encompasses empirical, quantitative, and normative judgmental aspects of decision-making. Furthermore, risk assessment and management should be integrated and incorporated within the broader holistic approach so technology management can help align the risk management requirements to the overall systems engineering requirements. Thus, the inclusion of a well-defined risk management plan that deals with the analysis of risks, within the systems engineering master plan is vital for the long term and sustained success of any system (Blanchard and Fabrycky 2011).

References

Works Cited

- Annachino, M. 2003. *New Product Development: From Initial Idea to Product Management*. Amsterdam, Netherlands: Elsevier.
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.
- Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. New York, NY, USA: John Wiley & Sons.
- Grady, J. 2006. *System Requirements Analysis*. New York, NY, USA: Academic Press.
- Haimes, Y. 2008. *Risk Modeling, Assessment, and Management*, 3rd ed. New York, NY, USA: John Wiley & Sons.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- ISO/IEC/IEEE. 2015. *Systems and software engineering - system life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of
-

Electrical and Electronics Engineers. ISO/IEC 15288:2015.

ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.

Kass, R. 2006. "The logic of warfighting experiments." DOD Command and Control Research Program (CCRP). August 2006. Accessed 23 April 2013 at https://www.dodccrp.org/files/Kass_Logic.pdf.

Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications.

Morse, L., and D. Babcock. 2007. *Managing Engineering and Technology*. International Series in Industrial and Systems Engineering. Upper Saddle River, NJ, USA: Prentice Hall.

Phillips, F. 2001. *Market Oriented Technology Management: Innovating for Profit in Entrepreneurial Times*. New York, NY, USA: Springer.

Pugh, S. 1990. *Total Design: Integrated Methods for Successful Product Engineering*. Englewood Cliffs, NJ, USA: Prentice Hall.

Reinertsen, D. 1997. *Managing the Design Factory: A Product Developers Tool Kit*. New York, NY, USA: Simon & Schuster Ltd.

Rogers, E.M. 2003. *Diffusion of Innovations*, 5th ed. New York, NY, USA: Free Press.

Smith, P., and D. Reinertsen. 1997. *Developing products in half the time – new rules, new tools*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.

US Army. 2012. "Chapter 2, section A.4" in *Army Science and Technology Master Plan*. Accessed January 12, 2012. Available at: <https://www.fas.org/man/dod-101/army/docs/astmp/c2/P2A4.htm>.

Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.

Primary References

Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice Hall.

Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. New York, NY, USA: John Wiley & Sons.

Wasson, C.S. 2006. *System Analysis, Design, and Development*. Hoboken, NJ, USA: John Wiley & Sons.

Additional References

Annachino, M. 2003. *New Product Development: From Initial Idea to Product Management*. Amsterdam, Netherlands: Elsevier.

Haimes, Y. 2008. *Risk Modeling, Assessment, and Management*, 3rd ed. New York, NY, USA: John Wiley & Sons.

Kass, R. 2006. "The logic of warfighting experiments." DOD Command and Control Research Program (CCRP). August 2006. Accessed 23 April 2013 at https://www.dodccrp.org/files/Kass_Logic.pdf.

Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications.

Morse, L., and D. Babcock. 2007. *Managing Engineering and Technology*. International Series in Industrial and Systems Engineering. Upper Saddle River, NJ, USA: Prentice Hall.

Phillips, F. 2001. *Market Oriented Technology Management: Innovating for Profit in Entrepreneurial Times*. New York, NY, USA: Springer.

Pugh, S. 1990. *Total Design: Integrated Methods for Successful Product Engineering*. Englewood Cliffs, NJ, USA: Prentice Hall.

Reinertsen, D. 1997. *Managing the Design Factory: A Product Developers Tool Kit*. New York, NY, USA: Simon & Schuster Ltd.

Rogers, E.M. 2003. *Diffusion of innovations*, 5th ed. New York, NY: Free Press.

Smith, P., and D. Reinertsen. 1997. *Developing products in half the time – new rules, new tools*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.

US Army. 2012. "Chapter 2, section A.4" in *Army Science and Technology Master Plan*. Accessed January 12, 2012. Available at: <https://www.fas.org/man/dod-101/army/docs/astmp/c2/P2A4.htm>.

Product Systems Engineering Special Activities

- Lead Author:
- Ricardo Pineda

Product systems engineering has activities that are unique to products. This article discusses many of them.

Readiness Level Assessments

As a new system is developed, it is essential to verify and validate that the developed system is mature enough to be released as an operational product or service. Technology readiness assessments (TRA) are established tools used to qualify technology development and help make investment decisions within complex development programs in order to deploy systems or elements of technology to an end user in a timely fashion.

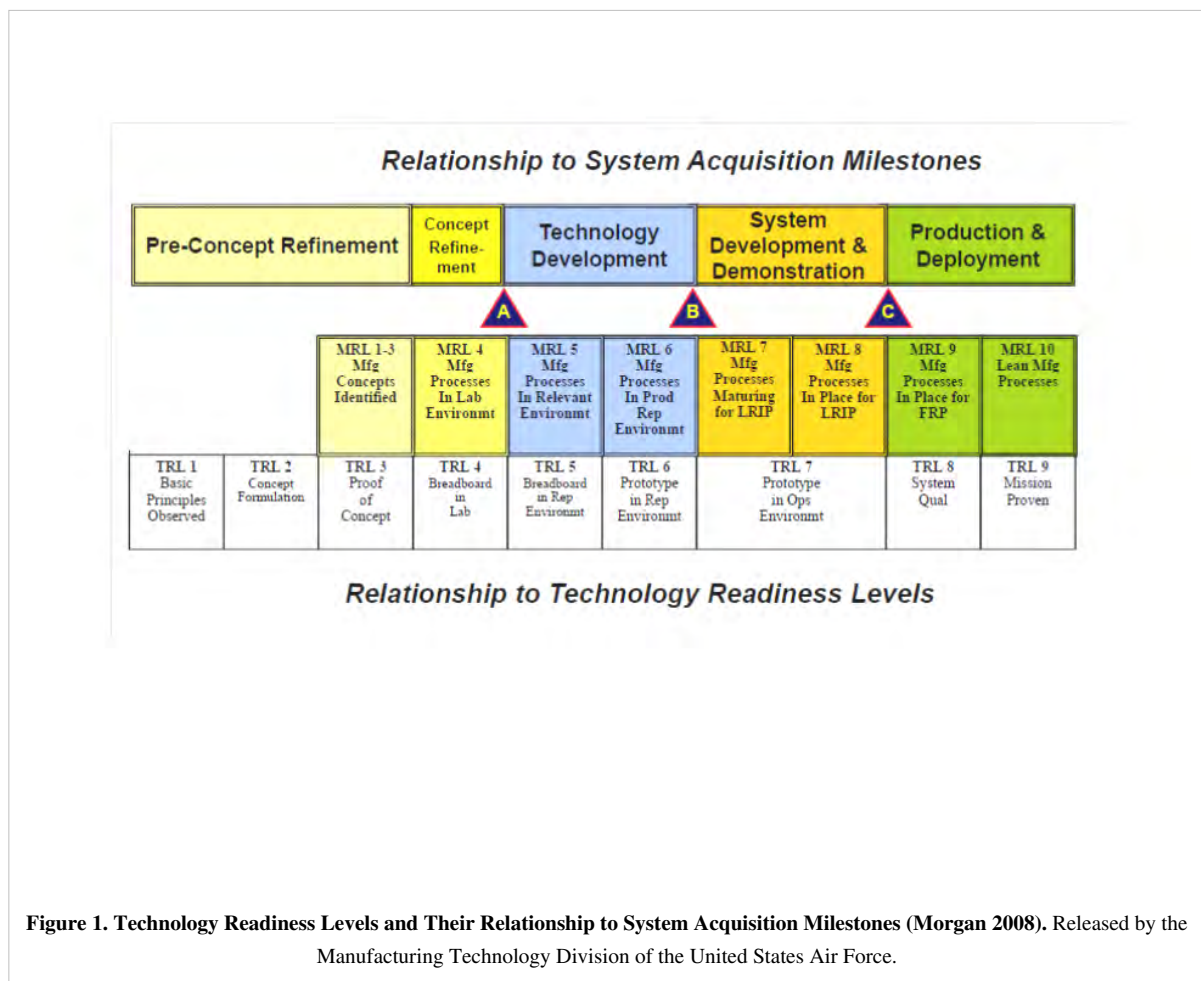
This notion of maturity was formalized by the US National Aeronautics and Space Administration (NASA) (Mankins 1995) and later modified for use by the Department of Defense (DoD), the Air Force Research Laboratory (AFRL), and the US Department of Energy (DoE), as well as a growing number of non-governmental organizations. Technology readiness levels (TRL) are a metric developed to summarize the degree of maturity of a technology. The original NASA TRL scale has nine different levels from the *basic principles observed and reported* (TRL 1) to *actual systems "flight proven" through successful mission operations* (TRL 9). The TRL scale utilized by the DoD is portrayed in Table 1.



**Table 1. Technology Readiness Levels for Assessing Critical Technologies (Mankins 1995).
Released by the Advanced Concept Office, Office of Space Access and Technology, NASA.**

Technology Readiness Level	+ 1. Basic principles observed and reported.	+ 2. Technology concept and/or application formulated.	+ 3. Analytical and experimental critical function and/or characteristic proof of concept.	+ 4. Component validation in laboratory environment.	+ 5. Component validation in relevant environment.	+ 6. Prototype demonstration in a relevant environment.	+ 7. Prototype demonstration in an operational environment.	+ 8. System qualified through test and demonstration.	+ 9. System proven through successful mission operations.	Actual application of the technology in its final form and under mission conditions, such as those encountered in operational test and evaluation.
----------------------------	--	--	--	--	--	---	---	---	---	--

The utilization of TRLs has an impact on the structure and operation of life cycles as described in Part 3; they allow better management and control of risks inherent with technology, as well as better control of costs and the schedule of program development. However, TRLs do not provide an assessment of the programmatic influence on a TRL, technology criticality and priority, software aging and readiness context, as pointed out by Smith (2005). While TRLs have proven to be useful in evaluating a technology's performance, as demonstrated in the laboratory or in a test environment, they do not inform one whether or not the technology product can actually be produced in an affordable manner. The concept of manufacturing readiness levels (MRL) has been incorporated to expand the TRL idea so that it can incorporate producibility concerns. The MRL approach addresses questions such as the level of technology reproducibility, the cost of production, and technology manufacturing production environment early in the development phase (GAO 2003, DoD 2011).



Readiness levels are an active research area within academia and government agencies in regards to the integration of technology components into complex systems (integration readiness levels (IRLs)) to address interface maturity among existing and maturing technology developments. TRLs apply to the critical enabling technologies, which are usually embodied at the subsystem, assembly level, or system component level. Systems readiness levels (SRL) are used when going from individual technologies to the whole system. The SRL model is a function of the individual TRLs in a system and their subsequent integration points with other technologies, the IRL (Sausser 2006).

Another maturity aspect is related to the provisioning of products that are readily available and referred to as commercial off-the-shelf (COTS). Such products, be they hardware, software, or a mixture of both, have hopefully achieved the degree of maturity so that those acquiring them can rely upon their operational properties and that the documentation of the COTS products is sufficient to provide the proper guidance in their use.

The PSE should realize that the TRL assessment for COTS changes dramatically if the operational environment or other requirements are imposed that exceed the design limits of the COTS product (e.g., operations at very high or very cold temperatures, high shock, or vibration levels).

Product Certification

Product certifications are both domain and product specific, and typically relate to human safety and health, the need to meet a specific government regulation, or are required by underwriters for insurance purposes. Certifications are performed by a third party (independent of the developer) who provides a guarantee of the quality, safety, and reliability of the product to the customer or user.

The INCOSE *SE Handbook* defines product certification as "the process of certifying that a certain product has passed performance or quality assurance tests or qualification requirements stipulated in regulations such as a building code or nationally accredited test standards, or that it complies with a set of regulations governing quality or minimum performance requirements." (INCOSE 2012)

The INCOSE *SE Handbook* also defines four methods for verification: inspection, analysis, demonstration, and testing (INCOSE 2012). In addition, it defines certification as a fifth verification method, which is defined as verification against legal or industrial standards by an outside authority without direction to that authority as to how the requirements are to be verified. For example, electronic devices require a CE certification in Europe, and a UL certification in the US and Canada (INCOSE 2012).

The best known certification is the airworthiness certification, which relates to the safety of flight for aircraft. In the US, the test for this certification is performed by the Federal Aviation Administration (FAA). Government certifications are also common in the medical systems field where the Federal Drug Administration (FDA) is the primary certification agency. Some certifications are based on standards defined by technical societies, such as the American Society of Mechanical Engineers (ASME). The combination of the technical standards and a certification allows product developers to perform certifications that meet government standards without having the government directly involved in the process.

There are equivalent government organizations in other countries and for other regulated areas, such as communications, building safety, nuclear systems, transportation systems to include ships, trains and automobiles, environmental impact, and energy use. Systems engineers must be aware of the certifications that are required for the domain and product being developed. Certification agencies must be involved early in the development effort to ensure the necessary certifications are included in the system requirements, the system development plan, and the funding provided to accomplish the development. When system changes and upgrades are necessary, the systems engineers must determine if product re-certification is necessary and include it in the plans and funding for the system upgrade.

Enabling Product Certifications

There may be other certifications for enabling products that must be considered and appreciated by PSE, such as an operator certification of airplane pilots to ensure flight safety, and certification of nuclear plant operators to ensure prevention or mitigation of nuclear radiation effects. An example of this is shown in the certification program by the North American Electric Reliability Corporation (NERC):

In support of NERC's mission, the System Operator Certification Program's mission is to ensure that employers have a workforce of system operators that meet minimum qualifications. These minimum qualifications are set through internationally recognized processes and procedures for agencies that certify persons. The Certification Program promotes excellence in the area of system operator performance and encourages system operators to be inquisitive and informed. (NERC 2012)

Production qualification testing (PQT) is another type of certification which DAU (2005) describes as:

A technical test completed prior to the full-rate production (FRP) decision to ensure the effectiveness of the manufacturing process, equipment, and procedures. This testing also serves the purpose of providing data for the independent evaluation required for materiel release so that the evaluator can address the adequacy of the materiel with respect to the stated requirements. These tests are conducted

on a number of samples taken at random from the first production lot, and are repeated if the process or design is changed significantly and when a second or alternative source is brought online.

Security certification and accreditation (C&A) is often required for the deployment of computing and networking equipment in a classified environment. Facility certification may be required to ensure that a building housing the equipment can provide the proper environment for safe and efficient operation of the equipment. High-altitude electromagnetic pulse (HEMP) certification may be required to ensure that a building and its equipment can withstand the effects of HEMP from nuclear weapons. A similar type of certification to HEMP is TEMPEST testing to ensure that sensitive electronic emissions are not allowed to leave high security facilities. TEMPEST is a code name referring to investigations and studies of compromising emission, and is not an acronym.

Technology Planning and Insertion

Technology planning can be an enterprise function or a program function. Technology planning as an enterprise function typically occurs on an annual basis to determine the funding necessary for independent research and development in the coming year. Technology planning as a program function occurs early in the program and often continues throughout the life of the system. The design of the product system is highly dependent on the availability of technologies that have acceptable risks and that meet the customer's cost, schedule, and performance requirements. These critical technologies will only be available when necessary if the systems engineers perform concept designs, technology assessments, and trade studies that define the critical technologies and the capabilities necessary before the system development activities that will use the critical technologies begin.

The MITRE *Systems Engineering Guide* (MITRE 2011) provides the following definition for technology planning:

Technology Planning is the process of planning the technical evolution of a program or system to achieve its future vision or end-state. Technology planning may include desired customer outcomes, technology forecasting and schedule projections, technology maturation requirements and planning, and technology insertion points. The goal is a defined technical end-state enabled by technology insertion over time.

Systems engineers who participate in technical planning must understand the future vision and system requirements, and relate these to the current and expected future technologies that can be applied to the system design during current development stages, as well as for potential future upgrades to the system. To do this, systems engineers must acquire and maintain knowledge of the existing and developing technology in their design domain. The systems engineer will also provide the essential connection between the system user and research communities to provide alignment between the technology developers and the system designers.

Technology planning and insertion usually requires that the systems engineer perform technology readiness assessments that rate the maturity levels and the risks associated with the planned technologies. Immature, risky technologies require risk reduction activities that include prototyping and product development and test activities that provide quantification of the capabilities and risks. The risk reduction activities provide the data necessary to assess and update the design to reduce its risk.

Product Road Mapping and Release Planning

Product road maps provide an outline that shows when products are scheduled for release and include an overview of the product's primary and secondary features. Both internal and external product road maps should be created. The form of the road map will depend on the development methodology being used. Waterfall, iterative, and spiral development models result in different road maps and release plans. The systems engineer must be an integral member of the team that creates road maps. Requirements should be mapped onto each of the planned releases. Test plans must be adapted to the development model and the release plans.

Product road maps should be aligned with the technology road maps that are applicable to the product. Technology maturity should be accomplished before the technologies are included in the product development plans and the road map for the product release that includes those technologies.

Product road maps are essential for software intensive systems that have many releases of software and capability upgrades. The identification of the requirements, the test plans, and the features provided for each release are an essential driver of the product development process. Clear definition of these items can make the difference between delivering the capabilities the customer is looking for and will support, or a product that fails to meet the needs of the customer and is abandoned.

Intellectual Property Management

Systems engineers must also manage intellectual property as part of their job. Existing systems engineering literature rarely covers this topic. However, there are many textbooks and management related literature that provide additional information, such as “Intellectual Property Rights for Engineers” (Irish 2005). Intellectual property may be considered as *intangible output of the rational thought process that has some intellectual or informational value and is normally protected via using copyrights, patents, and/or trade secrets* (Irish 2005). Listed below are some of the more important intellectual property types with brief explanations:

- **Proprietary Information:** Any information which gives a company (or enterprise) an advantage over its competitors is usually proprietary.
- **Patents:** A patent is the principle mechanism for protecting rights for an invention or discovery. In exchange for a full disclosure of how to practice it, the issuing government will grant the right to exclude others from practicing the invention for a limited amount of time, usually 15 to 20 years (in the US, a patent usually lasts for 17 years from the date of issue).
- **Design Patents:** In some countries, these are referred to by the more appropriate term *design registrations* or some other name. They protect rights in ornamental designs, provided the designs are new and inventive, i.e., *non-obvious* at the time they are made. In the US, the maximum length of a design patent is 14 years.
- **Trademarks:** A trademark identifies the source of origin for goods in commerce, and is not stronger than the actual use to which it has been put to and the diligence with which it has been protected from infringement, encroachment, or dilution. Under some circumstances, a trademark may be registered with governmental agencies. Among a company's most valuable assets is the corporate name, which also is the company's primary trademark.
- **Copyrights:** A claim of copyright protects such works as writings, musical compositions, and works of art from being copied by others, i.e., from plagiarism. A notice of claim of copyright must be made in the manner prescribed by law at the time of a protected work's first publication.

Parts, Materials, and Process Management

The consequences of mission failure or an inability to deploy the system on time due to parts, materials, and process (PM&P) issues needs to be clearly understood by the systems engineer since these elements are fundamental to the overall mission reliability and program success. PM&P management is especially important in harsh environments (like outer space and underwater) and in situations where system failure can have catastrophic impacts on public safety (like nuclear power, bridges and tunnels, and chemical processing plants).

Generally, original equipment manufacturers (OEMs) engaged in the design and fabrication of electronic systems have a documented policy that deals with PM&P, sometimes in the form of a PM&P Management Manual. The elements of a PM&P control program include things such as

- PM&P requirements that apply to a system;
- the generation number of a program or project approved parts list (PAPL);

- the appointment of a PM&P control board (PMPCB);
- the development of a part stress derating policy and a part parameter derating policy for end of life use; and
- a definition of the minimum qualifications, quality controls, and screening requirements for parts.

PM&P management guidance is provided by MIL-HDBK-512 (DoD 2001) and ANSI/AIAA R-100 (2001), which identify the overall management process elements of a PM&P program. Additional issues to be addressed by PM&P include the following: hazardous materials, rare earth elements, conflict materials, and counterfeit materials.

References

Works Cited

- ANSI/AIAA. 2001. *Recommended Practice for Parts Management*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/American Institute of Aeronautics and Astronautics (AIAA), ANSI/AIAA R-100A-2001.
- DAU. 2005. *Glossary of Defense Acquisition Acronyms & Terms*, 12th ed. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/US Department of Defense (DoD). Available at: https://www.dau.mil/pubscats/PubsCats/13th_Edition_Glossary.pdf.
- DoD. 2000. *Department of Defense Directive (DoD-D) 5000.01: The Defense Acquisition System*. Arlington, VA, USA: US Department of Defense, Acquisition, Technology, and Logistics (AT&L). Available at: <https://www.dtic.mil/whs/directives/corres/pdf/500001p.pdf>.
- DoD. 2001. *Department of Defense Handbook: Parts Management*. Arlington, VA, USA: Department of Defense (DoD). MIL-HDBK-512A.
- DoD. 2011 *Department of Defense Technology Readiness Assessment (TRA) Guidance*, Assistant Secretary of Defense for Research and Engineering (ASD(R&E)), May 2011.
- FAA. 2011. *Airworthiness Certificates Overview*. Washington, DC, USA: Federal Aviation Administration (FAA). Available at: https://www.faa.gov/aircraft/air_cert/airworthiness_certification/aw_overview/.
- GAO. 2003. *Defense acquisitions: Assessments of Major Weapon Programs*, GAO-03-476, US Government Accountability Office, May 2003.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- Irish, V. 2005. *Intellectual Property Rights for Engineers*, 2nd ed. Herts, UK: Institution of Engineering and Technology (IET).
- Mankins, J. 1995. *Technology Readiness Levels—A White Paper*. Washington, DC, USA: Advanced Concepts Office, Office of Space Access and Technology, National Aeronautics and Space Administration (NASA).
- MITRE. 2011. "Systems Engineering Guide." Accessed September 11, 2012. Available at: https://www.mitre.org/work/systems_engineering/guide/.
- Morgan, J 2007. *Manufacturing Readiness Levels (MRLs) and Manufacturing Readiness Assessments (MRAs)*. ADA510027 Air Force Research Lab Wright-patterson Afb Oh Manufacturing Technology Directorate. September 2007. Accessed 06 November 2014 at Defense Technical Information Center <https://www.dtic.mil/get-tr-doc/pdf?AD=ADA510027>
- NERC. 2012. "North American Electric Reliability Corporation (NERC)." Accessed September 11, 2012. Available at: <https://www.nerc.com>.
- Sausser, B., D. Verma, J. Ramirez-Marquez, and R. Gove. 2006. *From TRL to SRL: The Concept of System Readiness Levels*. Proceedings of the Conference on Systems Engineering Research (CSER), April 7-8, 2006, Los Angeles, CA, USA.

Smith, J. 2005. *An Alternative to Technology Readiness Levels for Non-Developmental Item (NDI) Software*. Proceedings of the 38th Hawaii International Conference on Systems Sciences, January 3-6, 2005, Island of Hawaii, USA.

Primary References

Mankins, J. 1995. *Technology Readiness Levels—A White Paper*. Washington, DC, USA: Advanced Concepts Office, Office of Space Access and Technology, National Aeronautics and Space Administration (NASA).

MITRE. "Systems Engineering Guide." Available at https://www.mitre.org/work/systems_engineering/guide/

Sauser, B., D. Verma, J. Ramirez-Marquez, and R. Gove. 2006. *From TRL to SRL: The Concept of System Readiness Levels*. Proceedings of the Conference on Systems Engineering Research (CSER), Los Angeles, CA, April 7-8, 2006.

Additional References

ANSI/AIAA. 2001. *Recommended Practice for Parts Management*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/American Institute of Aeronautics and Astronautics (AIAA), ANSI/AIAA R-100A-2001.

DoD. 2000. *Department of Defense Directive (DoD-D) 5000.01: The Defense Acquisition System*. Arlington, VA, USA: US Department of Defense, Acquisition, Technology, and Logistics (AT&L). Available at: <https://www.dtic.mil/whs/directives/corres/pdf/500001p.pdf>.

DoD. 2001. *Department of Defense Handbook: Parts Management*. Arlington, VA, USA: Department of Defense (DoD). MIL-HDBK-512A.

FAA. 2011. "Airworthiness Certificates Overview." Washington, DC, USA: Federal Aviation Administration (FAA). Available at: https://www.faa.gov/aircraft/air_cert/airworthiness_certification/aw_overview/.

Irish, V. 2005. *Intellectual Property Rights for Engineers*, 2nd ed. Herts, UK: Institution of Engineering and Technology (IET).

Smith, J. 2005. *An Alternative to Technology Readiness Levels for Non-Developmental Item (NDI) Software*. Proceedings of the 38th Hawaii International Conference on Systems Sciences, January 3-6, 2005, Island of Hawaii, USA.

Knowledge Area: Service Systems Engineering

Service Systems Engineering

Contents of this Knowledge Area

- Service Systems Background
- Fundamentals of Services (Ricardo Pineda and Bud Lawson)
- Properties of Services (Ricardo Pineda, Bud Lawson, and Brian Wells)
- Scope of Service Systems Engineering (Ricardo Pineda and Bud Lawson)
- Value of Service Systems Engineering (Ricardo Pineda, Bud Lawson, and Richard Turner)
- Service Systems Engineering Stages (Ricardo Pineda, Bud Lawson, and Richard Turner)
- Lead Authors:
- Ricardo Pineda, Bud Lawson, and Richard Turner

The growth of services in the ever-evolving global economy has brought much needed attention to service science and service systems engineering (SSE). Research focuses on developing formal methodologies to understand enterprise-end-user (customer) interactions from both socio-economic and technological perspectives, and to enable value co-creation and productivity improvements. Service systems require trans-disciplinary collaborations between society, science, enterprises, and engineering. Service transactions are customized and personalized to meet a particular customer need. This requires a disciplined and systemic approach among stakeholders and resources to emphasize end-user satisfaction in the design and delivery of the service (Hipel et al. 2007; Tien and Berg 2003; Vargo and Akaka 2009; Maglio and Spohrer 2008; Maglio et al. 2010).

Introduction

New Service Development (NSD) has usually been a proprietary process closely guarded by product businesses and service businesses for their competitive advantage. Traditional systems engineering practices have been primarily applied in aerospace and defense sectors while SSE practices have been applied by information and communications technologies (ICT) service providers (Booz, Allen, and Hamilton 1982; Johnson et al. 2000; Eppinger 2001; Freeman 2004; Whitten and Bentley 2007; AT&T SRP 2008; Lin and Hsieh 2011).

These early efforts were, and in some instances remain, very important for product and service businesses. However, the growth and ubiquity of the World Wide Web, advances in computer science and ICT, and business process management through “social networking,” support the realization of closely interrelated service systems. Product business (manufacturing, agriculture, etc.) and service business distinctions are going away (Spohrer 2011).

These services, or service innovations, must take into account social aspects, governance processes, business processes, operational processes, as well as design and development processes. The customer, service provider, product provider, and intermediaries need to collaborate toward the optimization of customer experiences and customer provided value (through co-creation). The interrelations among different stakeholders and resources require that methodologies, processes, and tools be dynamically tailored and delivered for either foreseen or newly discovered services to rapidly adapt to changing enterprise and end-user environments.

Even in the case of static, predetermined, interaction rules, the major problems faced in the definition, design, and implementation of services have been in understanding the integration needs among different systems, system

entities, stakeholders, and in defining the information flows required for the governance, operations, administration, management and provisioning (OAM&P) of the service. (Maier 1998; Jamshidi 2008; Pineda 2010; Luzeaux and Ruault 2013). Thus, the 21st century technology-intensive services are "information-driven, customer centric, e-oriented, and productivity-focused" as discussed by Chesbrough (2011), Chang (2010), Tien and Berg (2003), and Hipel et al. (2007). A detailed discussion of these characteristics is given in the Value of Service Systems Engineering article within this KA.

Service Systems Engineering Knowledge Area Topics

This knowledge area (KA) describes best practices in SSE during the service design process and outlines current research on methods, processes, and tools. It does not attempt to describe the initial efforts and research in service science that were proposed and introduced by International Business Machines (IBM) (Maglio and Spohrer 2008), but it does recognize their leadership in championing these concepts in undergraduate and graduate curricula.

The rest of the KA is organized in the following way:

The Service Systems Background article presents some background on the transition from a manufacturing economy toward the service economy brought by the World Wide Web through co-creation of end-user value. It describes how this transformation is impacting industries, such as healthcare, agriculture, manufacturing, transportation, supply chain, environmental, etc. The article also describes the scope of the SSE discipline's contributions to meeting the needs of the service sector companies in strategic differentiation and operational excellence (Chang 2010) by pointing out some differences between product-oriented systems engineering and SSE.

The Fundamentals of Services and Properties of Services articles take the reader through a general discussion of services and current attempts to classify different types of services, in particular, attention is paid to the properties of service systems for the service sector, such as transportation, environmental and energy services, consulting services, healthcare, etc.

The Scope of Service Systems Engineering and Value of Service Systems Engineering articles cover the value of SSE, defining (or using when available) service architecture frameworks, and the stages of the service development process from concept to life cycle management.

The Service Systems Engineering Stages article summarizes the major SSE process activities that need to be carried out during the service design process and the needed output (work products) in each of the service design process stages.

Service Innovation and Value-Co-creation

Service innovation has several dimensions. Service innovation can come about through the creation of a service concept which is sufficiently different that it is not merely an improved service, but in reality is a new service concept. To maintain the rigor and value of innovation, it is necessary to distinguish between an improved service, which may generate some additional value, and a truly new and innovative service concept, which may generate a great deal of value. Dr. Noriaki Kano, a renowned quality management guru, has suggested that every service concept has its inherent attributes and we should strive to continuously improve upon these; but this is not innovation (Kano 1996).

To be innovative, the change in a value proposition cannot be incremental, but it must be enough to significantly impact customer and competitor behavior (e.g., new market creation). Value innovation involves a shift in perspective of customer needs that requires a rethinking of what service value proposition is delivered (Kano 1996).

Innovation can also come through a significant change in the way or the reason the customer is engaged or connected. In a service value chain the customer may well change from being just a receiver of service value to becoming a co-creator, or an active participant in the design and delivery, i.e., service transaction of service value. At the retail level, when a customer designs the time, route, and price selection for a plane ticket purchased online, he is

co-creating the service. Value innovation involves a shift in perspective of customer needs that requires a rethinking of how a service value proposition is delivered (Bettencourt 2010).

Finally, service innovation can come through significant changes in the way the enterprise is organized to create a service value proposition from concept through delivery. A considerable improvement in the enterprise structure and/or governance can be seen as innovation. Value innovation involves a shift in perspective of customer needs that requires a rethinking of how an enterprise organizes to support a service value proposition.

Continuous improvement can be reasonably planned and predicted while innovation and breakthroughs cannot. The most effective way to obtain innovation and breakthroughs is to encourage the culture, environment, and atmosphere that are conducive to innovation and breakthroughs. Innovative co-creation requires the integration of people, ideas, and technology for the purpose of creating value for themselves, their customers, companies, and society.

The lone inventor sees a problem and must work to create the solutions to all dimensions of the problem. Co-creators see the problem and realize that there may already be several creators, each already having a piece of the solution. Co-creation embraces the value of things “not invented here” because of the velocity they can bring to ideation and time to market. This service innovation process is facilitated by modern mass (and at the same time, personal) communication technology evident in social networking platforms.

Towards a Discipline of Service Systems Engineering

Mindful of the evolution taking place in the global economy and the world markets, it would be futile to attempt covering all the major advances and the boundless possibilities in the services sector for the rest of the century. The services sector covers wide areas of application studied in many different fields (e.g., business science, social science, cognitive science, political science, etc.). The field of service systems, a trans-disciplinary analysis and study of services, was only introduced 10 to 15 years ago. As a consequence, much of the existing literature on services and service-innovation is scattered. The main objective of this KA is to document the systems engineering processes, methodologies, and existing tools as applied to the service design process, and to introduce critical SSE challenges and research areas.

References

Works Cited

- AT&T SRP. 2008. *Technical Approach to Service Delivery*. General Services Administration, AT&T Bridge Contract No. GS00Q09NSD0003. Accessed on June 1, 2011. Available at: https://www.corp.att.com/gov/contracts/fts_bridge/technical/07_vol_I_section_1.pdf.
- Bettencourt, L. 2010. *Service Innovation: How to Go from Customer Needs to Breakthrough Services*. New York, McGraw-Hill Professional. July 2010.
- Booz, Allen, and Hamilton. 1982. *New Products Management for the 1980s*. New York, NY, USA: Booz, Allen, and Hamilton Inc.
- Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. New York, NY, USA: John Wiley & Sons, Inc.
- Chesbrough, H. 2011. *Open Services Innovation: Rethinking Your Business to Grow and Compete in a New Era*. San Francisco, CA, USA: Jossey-Bass.
- Eppinger, S. 2001. "Innovation at the Speed of Information" *Harvard Business Review*. 79 (1): 149-158.
- Freeman, R.L. 2004. *Telecommunication Systems Engineering*, 4th ed. New York, NY, USA: John Wiley & Sons.
- Hipel, K.W., M.M. Jamshidi, J.M. Tien, and C.C. White. 2007. "The Future of Systems, Man, and Cybernetics: Application Domains and research Methods. IEEE Transactions on Systems, Man, and Cybernetics - Part C:

Applications and Reviews. 37 (5): 726-743.

Jamshidi M, *System of Systems Engineering: Innovations for the Twenty-First Century*. New York, NY, USA: John Wiley & Sons. November 2008.

Johnson, S.P., L.J. Menor, A.V. Roth, and R.B. Chase. 2000. "A critical evaluation of the new service development process: integrating service innovation and service design," in Fitzsimmons, J.A., and M.J. Fitzsimmons (eds.). *New Service Development - Creating Memorable Experiences*. Thousand Oaks, CA, USA: Sage Publications. p. 1-32.

Kano, N. 1996. *Guide to TQM in Service Industry*. Tokyo, Japan: Asian Productivity Organization.

Lin, F.R., and P.S Hsieh. 2011. A SAT View on New Service Development." *Service Science*. 3 (2): 141-157.

Luzeaux, D. and Ruault. J., *System of Systems*. New York, NY, USA: John Wiley & Sons. March 2013.

Maglio, P., C. Kieliszewski, and J. Spohrer. 2010. *Handbook of Service Science*, 1st ed. New York, NY, USA: Springer Science + Business Media.

Maglio, P., and J. Spohrer. 2008. "Fundamentals of Service Science." *Journal of the Academy of Marketing Science*. 36 (1): 18-20.

Maier, M.W., 1998. "Architecting Principles for System of Systems." *Systems Engineering*. 1 (4): 267-284.

Pineda, R. 2010. "Understanding Complex Systems of Systems Engineering." Presented at Fourth General Assembly, Cartagena Network of Engineering, September 21-24, 2010, Metz, France.

Spohrer, J.C. 2011. "Service Science: Progress & Directions." Presented at International Joint Conference on Service Science, May 25-27, 2011, Taipei, Taiwan.

Tien, J.M., and D. Berg. 2003. "A Case for Service Systems Engineering." *Journal of Systems Science and Systems Engineering*. 12 (1): 13-38.

Vargo, S.L., and R.F. Lusch. 2004. "The Four Service Marketing Myths – Remnants of a Goods-Based Manufacturing Model." *Journal of Service Research*. 6 (4): 324-335.

Vargo, S.L., and M.A. Akaka. 2009. "Service-Dominant Logic as a Foundation for Service Science: Clarifications." *Service Science*. 1 (1): 32-41.

Whitten, J., and L. Bentley. 2007. *Systems Analysis and Design Methods*. New York, NY, USA: McGraw-Hill Higher Education.

Primary References

Maglio, P., C. Kieliszewski, and J. Spohrer. 2010. *Handbook of Service Science*, 1st ed. New York, NY, USA: Springer Science + Business Media.

Tien, J.M., and D. Berg. 2003. "A Case for Service Systems Engineering." *Journal of Systems Science and Systems Engineering*. 12 (1): 13-38.

Vargo, S.L., and R.F. Lusch. 2004. "The Four Service Marketing Myths – Remnants of a Goods-Based Manufacturing Model." *Journal of Service Research*. 6 (4): 324-335.

Additional References

- AT&T SRP. 2008. *Technical Approach to Service Delivery*. General Services Administration, AT&T Bridge Contract No. GS00Q09NSD0003. Accessed on June 1, 2011. Available at: https://www.corp.att.com/gov/contracts/fts_bridge/technical/07_vol_I_section_1.pdf.
- Booz, Allen, and Hamilton. 1982. *New Products Management for the 1980s*. New York, NY, USA: Booz, Allen, and Hamilton Inc.
- Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. New York, NY, USA: John Wiley & Sons, Inc.
- Eppinger, S. 2001. "Innovation at the Speed of Information." *Harvard Business Review*. 79 (1): 149-158.
- Freeman, R.L. 2004. *Telecommunication Systems Engineering*, 4th ed. New York, NY, USA: John Wiley & Sons.
- Hipel, K.W., M.M. Jamshidi, J.M. Tien, and C.C. White. 2007. "The Future of Systems, Man, and Cybernetics: Application Domains and research Methods." *IEEE Transactions on Systems, Man, and Cybernetics - Part C: Applications and Reviews*. 37 (5): 726-743.
- Johnson, S.P., L.J Menor, A.V. Roth, and R.B. Chase. 2000. "A critical evaluation of the new service development process: integrating service innovation and service design," in Fitzsimmons, J.A., and M.J. Fitzsimmons (eds.). *New Service Development - Creating Memorable Experiences*. Thousand Oaks, CA, USA: Sage Publications. p. 1-32.
- Kano, N. 1996. *Guide to TQM in Service Industry*. Tokyo, Japan: Asian Productivity Organization.
- Lin, F.R., and P.S Hsieh. 2011. A SAT View on New Service Development." *Service Science*. 3 (2): 141-157.
- Luzeaux, D. and Ruault, J. 2013. *Systems of Systems*, Wiley.
- Maier, M.W. 1998. "Architecting Principles for System of Systems." *Systems Engineering*. 1 (4): 267-284.
- Maglio, P., and J. Spohrer. 2008. "Fundamentals of Service Science." *Journal of the Academy of Marketing Science*. 36 (1): 18-20.
- Pineda, R. 2010. "Understanding Complex Systems of Systems Engineering." Presented at Fourth General Assembly, Cartagena Network of Engineering, September 21-24, 2010, Metz, France.
- Spohrer, J.C. 2011. "Service Science: Progress & Directions." Presented at International Joint Conference on Service Science, May 25-27, 2011, Taipei, Taiwan.
- Vargo, S.L., and M.A. Akaka. 2009. "Service-Dominant Logic as a Foundation for Service Science: Clarifications." *Service Science*. 1 (1): 32-41.
- Whitten, J., and L. Bentley. 2007. *Systems Analysis and Design Methods*. New York, NY, USA: McGraw-Hill Higher Education.
-

Service Systems Background

Economies are pre-disposed to follow a developmental progression that moves them from heavy proportional reliance on agriculture and mining toward the development of manufacturing, and finally toward more service-based economic activity. As reported by the Organization for Economic Co-Operation and Development (OECD) in its "Science, Technology, and Industry (STI) Forum on The Service Economy":

The reason that we see a services economy today, and gather to talk about it and recognize its importance is because technology has allowed service industries to gain the operational leverage that manufacturing achieved 100 years ago. In addition to banks, health systems, telephone and telecommunications networks, and distribution and retailing firms are further examples of sectors that have been able to benefit from economies of scale. As a result, we are now living in a world where global-scale service companies exist for the first time, whereas we have seen global manufacturing companies for 50 years or more. (OECD 2000, 8)

Evolution Toward Service-Based Economies

The typical industry example given of this progression toward services is the company International Business Machines (IBM). Even though IBM still produces hardware, they view their business as overwhelmingly service-oriented wherein hardware plays only an incidental role in their business solutions services; the fastest line of business growth within IBM has been the business-to-business (B2B) services: information technology (IT); for example, data centers and call centers; business process outsourcing/re-engineering; systems integration; and organizational change.

Business to government (B2G) is forecasted to have the fastest growth in the years to come (Spohrer 2011). For IBM, this trend started in 1989 with the launch of business recovery services; it accelerated with the acquisition of Price-Waterhouse Coopers Consultants in 2002 and culminated with the 2005 sale of the laptop (ThinkPad) manufacturing, their last major hardware operation.

IBM exemplifies the services trend which has accelerated in the last 25-30 years and as of 2006, the services produced by private industry accounted for 67.8% of U.S. gross domestic product (GDP). The top sub-sectors included real estate, financial, healthcare, education, legal, banking, insurance, and investment. Production of goods accounted for 19.8% of GDP. The top product sub-sectors included manufacturing, construction, oil and gas, mining, and agriculture (Moran 2006).

Beginning in the mid-1990s, the concept of a product-service system (PSS) started to evolve. PSSs have been adopted by businesses interested in using the model to bring not only added value to their existing offerings, but capital-intensive, environmentally favorable products to market (Mont and Tukker 2006).

There are some definitional issues in any discussion of PSS, including the fact that services can sometimes be considered as products, and services invariably need physical products to support their provisioning or delivery (2006). A PSS is comprised of tangibles and intangibles (activities) in combination to fulfill specific customer requirements, or ideally, to allow applications to be co-created flexibly by linking loosely coupled agents, typically over a network (Domingue et al. 2009). Research has shown that manufacturing firms are more amenable to producing "results" rather than solely products as specific artifacts and that end users are more amenable to consuming such results (Cook 2004; Wild et al. 2007).

The popularity of wikis, blogs, and social networking tools is strong evidence that "Enterprise 2.0" is already well under way; Andrew McAfee describes Enterprise 2.0 as "the use of emergent social software platforms within companies, or between companies and their partners or customers" (McAfee 2009). However, the integrated access to people, media, services, and things, provided by the Future Internet, will enable new styles of societal and economic interactions at unprecedented scales, flexibility, and quality. These applications will exploit the wisdom of

crowds and allow for mass collaboration and value co-creation.

The future internet will provide location independent, interoperable, scalable, secure, and efficient access to a coordinated set of services (Tselentis et al. 2009), but such a broad vision demands a sound and well-defined approach for management and governance.

Current application service providers like Amazon, Facebook, Twitter, eBay, and Google must mediate between the business challenges enabled by network and IT convergence and customers (enterprise or consumer) demanding new and more value-adding services enabled by social networks (TMFORUM 2008). The differences between IT and communications technologies are disappearing; internally-focused processes (back-stage processes) for operations optimization are now being strongly tied to the customer facing (front-stage) processes for value co-creation and delivery. In this scenario, the enterprise's internal organization and employees are embedded in the service value chain to benefit customers and stakeholders. In the service-dominant logic (S-DL) for marketing (Vargo and Lusch 2004), service is the application (through deeds, processes, and performances) of specialized operant resources (knowledge and skills) for the benefit of another entity or the entity itself. The emphasis is on the process of doing something for, and with, another entity in order to create value; a service system is thus a system of interacting and interdependent parts (people, technologies, and organizations) that is externally oriented to achieve and maintain a sustainable competitive advantage (IFM 2008; Maglio and Spohrer 2008).

The future internet is expected to be more agile, scalable, secure, and reliable, demanding rapidly emerging applications/services with different requirements and implications for the Future Internet design that pose a significant set of problems and challenges, in particular, "the fragmentation of knowledge and the isolation of the specialist as well as the need to find new approaches to problems created by earlier 'solution of problems,'" (Skyttner 2006). The service systems engineering discipline may inform the discussion and offer potential multidisciplinary environments and trans-disciplinary solutions.

The internet has been successfully deployed for several decades due to its high flexibility in running over different kinds of physical media and in supporting different high-layer protocols and applications, including traditional file transfer, email, and client-server-based Web applications, among others.

Business Dependence on Service Systems

Most people and enterprises are heavily dependent on service interactions, including entertainment, communications, retail, education, healthcare, etc., brought about by emerging services, such as video on demand, web conferencing, time-shift services, place-shift and device-shift services, enterprise applications (e.g., enterprise resource planning (ERP), customer relationship management (CRM), manufacturing resource management (MRM), software configuration management (SCM), etc.), software as a service (SaaS), platform as a service (PaaS), cloud services, peer-to-peer (P2P) services, etc. A common denominator in the set of services mentioned is that applications are offered as services by the interaction of service system entities and thus they are service based applications (SBA).

Thus, "A service based application is obtained by composing various service system entities to satisfy the desired functionality" (Andrikopoulos et al. 2010). SBAs are heavily dependent on web services development, such as Web services 2.0 (WS). Software systems engineering (SwSE) plays a very important role in a business dependent on a service system. However, another important role is played by human interfaces, organizational development and technology development; for instance, governance (rules & regulations) and technology research and development are required for future services in healthcare services, intelligent transportation services, environmental services, energy services, etc. to address societal challenges of the 21st century (sustainability, energy, etc.) as presented by (Vest 2010) if we were to face those challenges as an ecosystem.

Service System Example

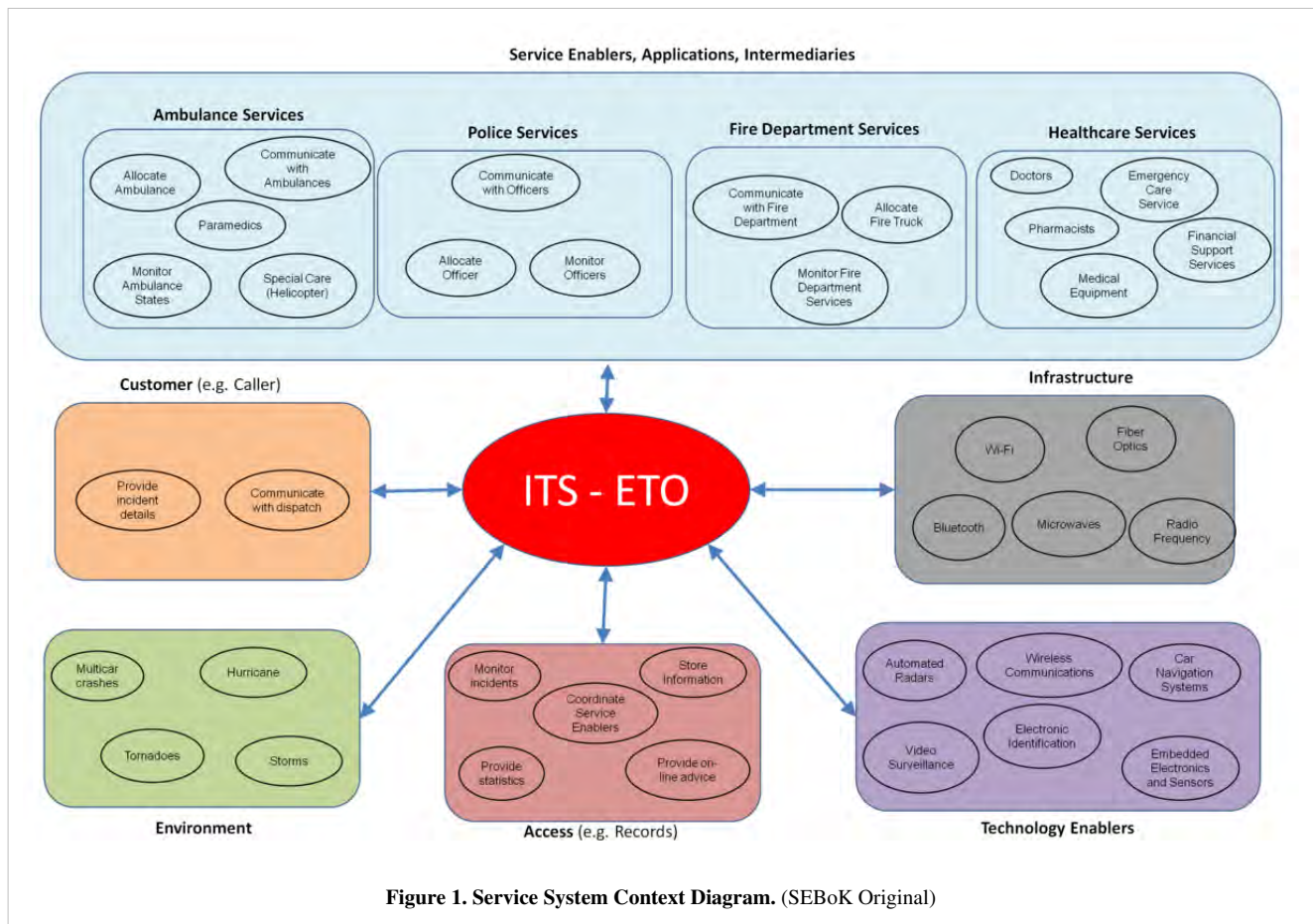
In an intelligent transport system-emergency transportation operation (ITS-ETO), the service goal is to provide safe evacuation, prompt medical care, and improved emergency management service. Typically, a traveler can request service through an emergency call or automated crash report feature, or a public safety officer on location can request service based on customer features and access rights.

The ITS-ETO service system utilizes advances in communication and information systems (technology and information enabler) to access essential, real-time data about conditions on routes throughout the affected area and coordinate operational and logistical strategies in cooperation within all service entities (organization processes). In a critical emergency situation, when patient conditions are continuously changing, ITS can help identify the appropriate response and get the correct equipment (infrastructure enabler), such as a helicopter and emergency personnel (people enabler), to and from the scene quickly and safely.

Efficient and reliable voice, data, and video communications (application enabler) further provide agencies with the ability to share information related to the status of the emergency, the operational conditions of the transportation facilities, and the location of emergency response resources to help communicate and coordinate operations and resources in real time. Advances in logistical and decision-making tools can enable commanders and dispatchers to implement strategies as conditions change (decision making).

It is also critical to receive information on the environmental conditions (storm, hazardous materials, multi-vehicle crashes, etc.) and/or road closures when coordinating evacuations. The availability of real-time data about transportation conditions, coupled with decision-making tools, enables more effective responses and coordination of resources during emergencies. ITS-ETO also enhances the ability of transportation agencies to coordinate responses with other stakeholders/entities.

As a result, increased data accuracy, timeliness, and automation leads to better use of resources, and reuse of exchanges, resulting in time and cost savings. Enhanced response and management leads to greater situational awareness and more effective reactions with the ability to identify and utilize the appropriate equipment, resulting in a more efficient response at the right time (output) (US DOT 2011). Figure 1 below lists the possible stakeholders in a service system.



As seen in the above example, the service activities are knowledge-intensive; well-defined linkages (including access rights) and relationships among different entities give rise to the needed service systems interactions for the service system to be successful. As the world becomes more widely interconnected, and people become better educated, the services networks created by the interaction of the service systems will be accessible from anywhere, at any time, by anyone with the proper access rights.

Knowledge agents are then humans creating new linkages of information to create new knowledge which “can later be embedded in other people, technology, shared information, and organizations.” Thus, people can be considered as individual service systems with “finite life cycles, identities (with associated histories and expectations), legal rights and authority to perform certain functions, perform multitasking as a way to increase individual productivity output in a finite time, and engage in division-of-labor with others to increase collective productive output in finite time” through service transactions enabled by their access rights (Spohrer and Kwan 2008).

References

Works Cited

- Andrikopoulos, V., A. Bucchiarone, E. Di Nitto, R. Kazhamiakin, S. Lane, V. Mazza, and I. Richardson. 2010. "Chapter 8: Service Engineering," in *Service Research Challenges and Solutions for the Future Internet S-Cube – Towards Engineering, Managing and Adapting Service-Based Systems*, edited by M. Papazoglou, K. Pohl, M. Parkin, and A. Metzger. Berlin Heidelberg, Germany: Springer-Verlag. p. 271-337.
- Cook, M. 2004. "Understanding The Potential Opportunities Provided by Service-Orientated Concepts to Improve Resource Productivity," in *Design and Manufacture for Sustainable Development 2004*, edited by T. Bhamra and B. Hon. Bury St. Edmonds, Suffolk, UK: Professional Engineering Publishing Limited. p. 123-134.
- Domingue, J., D. Fensel, J. Davies, R. González-Cabero, and C. Pedrinaci. 2009. "The Service Web: a Web of Billions of Services," in *Towards the Future Internet - A European Research Perspective*, edited by G. Tselentis, J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zehariadis. Amsterdam, The Netherlands: IOS Press.
- IFM. 2008. *Succeeding through Service Innovation: A service perspective for education, research, business and government*. University of Cambridge Institute for Manufacturing (IfM) and International Business Machines Corporation (IBM) report. Cambridge Service Science, Management and Engineering Symposium, July 14-15, 2007, Cambridge, UK.
- Maglio P., and J. Spohrer 2008. "Fundamentals of Service Science." *Journal of the Academy of Marketing Science*. 36 (1): 18-20. DOI: 10.1007/s11747-007-0058-9.
- Maglio, P., Weske, M., Yang, J. and Fantinato, Marcelo. 2010. *8th International Conference on Service Oriented Computing (ICSOC 2010)*. Lecture Notes in Computer Science. Vol. 6470. Springer-Verlag, San Francisco, California. December 2010.
- McAfee, A. 2009. *Enterprise 2.0: New Collaborative Tools for Your Organization's Toughest Challenges*. Boston, MA, USA: Harvard Business School Press.
- Mont, O., and A. Tukker. 2006. "Product-Service Systems." *Journal of Cleaner Production*. 14 (17): 1451-1454.
- Moran, M. 2006. *Servicizing Solar Panels*. Industry Course Report. Lund University International Master's Programme in Environmental Studies and Sustainability Science Department (LUMES), Lund University, Sweden.
- Organization for Economic Co-operation and Development (OECD). 2000. *The Service Economy*. Science Technology Industry (STI) Business and Industry Policy Forum Series. Paris, France: OECD. Available: <https://www.oecd.org/dataoecd/10/33/2090561.pdf>.
- Skyttner, L. 2006. *General Systems Theory: Perspectives, Problems, Practice*, 2nd ed. Singapore: World Scientific Publishing Company.
- Spohrer, J., and S.K. Kwan. 2009. "Service Science, Management, Engineering, and Design (SSMED): An Emerging Discipline - Outline & References." *International Journal of Information Systems in the Service Sector*. 1 (3): 1-31.
- Spohrer, J.C. 2011. "Service Science: Progress & Directions." Presented at the International Joint Conference on Service Science, May 25-27, 2011, Taipei, Taiwan.
- TM Forum. 2008. *Service Delivery Framework (SDF) Overview, Release 2.0*. Morristown, NJ: TeleManagement Forum. Technical Report 139.
- Tselentis, G., J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zahariadis (eds.). 2009. *Towards the Future Internet - A European Research Perspective*. Amsterdam, The Netherlands: IOS Press.
- US DOT. 2011. "Emergency Transportation Operations." Research and Innovative Technology Administration. Accessed June 23, 2011. Last updated June 16, 2011. Available: <https://www.its.dot.gov/eto/index.htm>.

- Vargo, S.L., and R.F. Lusch. 2004. "The Four Service Marketing Myths – Remnants of a Goods-Based Manufacturing Model." *Journal of Service Research*. 6 (4): 324-335.
- Vest, C.M., 2013. "Educating Engineers for 2020 and Beyond" *The Bridge*. Washington DC, National Academy of Engineering.
- Wild, P.J., J. Jupp, W. Kerley, C. Eckert, and P.J. Clarkson. 2007. "Towards A Framework for Profiling of Products and Services." Presented at 5th International Conference on Manufacturing Research (ICMR), September 11-13, 2007, Leicester, UK.

Primary References

- IfM. 2008. *Succeeding through Service Innovation: A service perspective for education, research, business and government*. University of Cambridge Institute for Manufacturing (IfM) and International Business Machines Corporation (IBM) report. Cambridge Service Science, Management and Engineering Symposium, July 14-15, 2007, Cambridge, UK.
- Organization for Economic Co-operation and Development (OECD). 2000. *The Service Economy*. Science Technology Industry (STI) Business and Industry Policy Forum Series. Paris, France: OECD. Available: <https://www.oecd.org/dataoecd/10/33/2090561.pdf>.
- Vargo, S.L., and R.F. Lusch. 2004. "The Four Service Marketing Myths – Remnants of a Goods-Based Manufacturing Model." *Journal of Service Research*. 6 (4): 324-335.

Additional References

- Andrikopoulos, V., A. Bucchiarone, E. Di Nitto, R. Kazhamiakin, S. Lane, V. Mazza, and I. Richardson. 2010. "Chapter 8: Service Engineering," in *Service Research Challenges and Solutions for the Future Internet S-Cube – Towards Engineering, Managing and Adapting Service-Based Systems*, edited by M. Papazoglou, K. Pohl, M. Parkin, and A. Metzger. Berlin Heidelberg, Germany: Springer-Verlag. p. 271-337.
- Cook, M. 2004. "Understanding The Potential Opportunities Provided by Service-Orientated Concepts to Improve Resource Productivity," in *Design and Manufacture for Sustainable Development 2004*, edited by T. Bhamra and B. Hon. Bury St. Edmunds, Suffolk, UK: Professional Engineering Publishing Limited. p. 123-134.
- Domingue, J., D. Fensel, J. Davies, R. González-Cabero, and C. Pedrinaci. 2009. "The Service Web: a Web of Billions of Services," in *Towards the Future Internet - A European Research Perspective*, edited by G. Tselentis, J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zehariadis. Amsterdam, The Netherlands: IOS Press.
- Maglio P., and J. Spohrer 2008. "Fundamentals of Service Science." *Journal of the Academy of Marketing Science*. 36 (1): 18-20. DOI: 10.1007/s11747-007-0058-9.
- McAfee, A. 2009. *Enterprise 2.0: New Collaborative Tools for Your Organization's Toughest Challenges*. Boston, MA, USA: Harvard Business School Press.
- Mont, O., and A. Tukker. 2006. "Product-Service Systems." *Journal of Cleaner Production*. 14 (17): 1451-1454.
- Moran, M. 2006. *Servicizing Solar Panels*. Industry Course Report. Lund University International Master's Programme in Environmental Studies and Sustainability Science Department (LUMES), Lund University, Sweden.
- Skyttner, L. 2006. *General Systems Theory: Perspectives, Problems, Practice*, 2nd ed. Singapore: World Scientific Publishing Company.
- Spohrer, J., and S.K. Kwan. 2009. "Service Science, Management, Engineering, and Design (SSMED): An Emerging Discipline - Outline & References." *International Journal of Information Systems in the Service Sector*. 1 (3): 1-31.

Spohrer, J.C. 2011. "Service Science: Progress & Directions." Presented at the International Joint Conference on Service Science, May 25-27, 2011, Taipei, Taiwan.

TM Forum. 2008. *Service Delivery Framework (SDF) Overview, Release 2.0*. Morristown, NJ: TeleManagement Forum. Technical Report 139.

Tselentis, G., J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zahariadis (eds.). 2009. *Towards the Future Internet - A European Research Perspective*. Amsterdam, The Netherlands: IOS Press.

US DOT. 2011. "Emergency Transportation Operations." Research and Innovative Technology Administration. Accessed June 23, 2011. Last updated June 16, 2011. Available: <https://www.its.dot.gov/eto/index.htm>.

Wild, P.J., J. Jupp, W. Kerley, C. Eckert, and P.J. Clarkson. 2007. "Towards A Framework for Profiling of Products and Services." Presented at 5th International Conference on Manufacturing Research (ICMR), September 11-13, 2007, Leicester, UK.

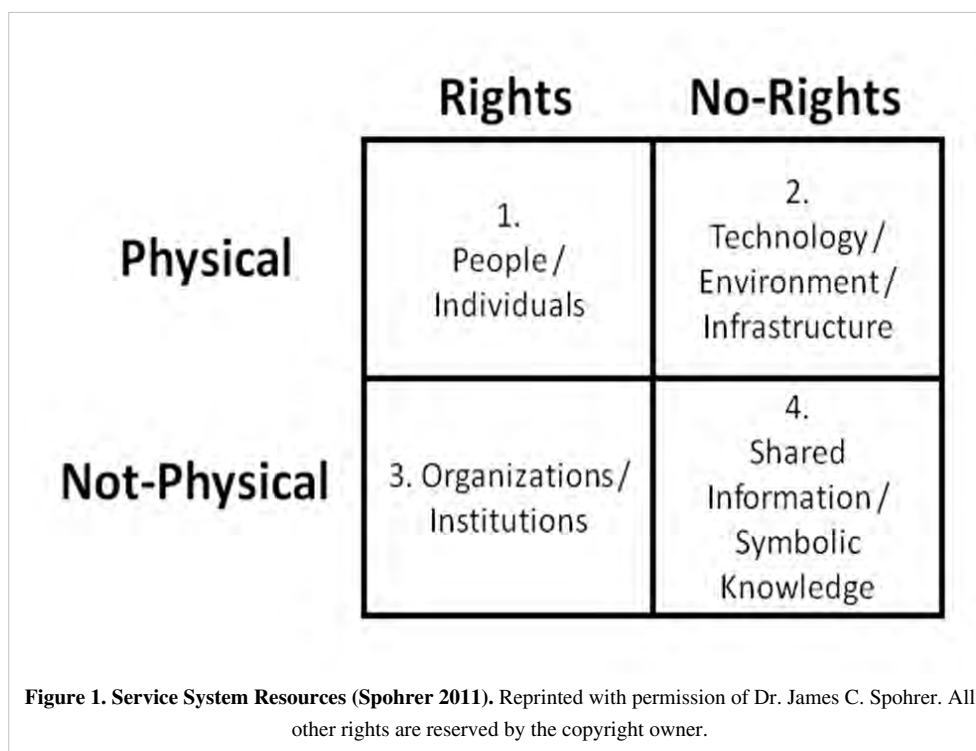
Fundamentals of Services

- Lead Author:
- Ricardo Pineda & Bud Lawson

Services are activities that cause a transformation of the state of an entity (a person, product, business, region, or nation) by mutually agreed terms between the service provider and the customer. Individual services are relatively simple, although they may require customization and significant back-stage support (e.g., database, knowledge management, analysis, forecasting, etc.) to assure quality and timely delivery. Product services are also relatively straightforward as product specifications, performance standards, quality control, installation guidelines, and maintenance procedures require good communication and understanding between providers and users. Business services can be rather complex; some may involve intensive negotiations, work process alignment, quality assurance, team collaboration, and service coproduction. Moreover, Chang (2010) states that: "Regional and National services are even more complex, as they may affect policy, custom regulations, export permits, local business practices, logistics, distribution, and other such issues" (see also Emergence and Complexity).

Service Systems

The service and/or set of services developed and accessible to the customer (individual consumer or enterprise) are enabled by a service system. Service system stakeholders may interact to create a particular service value chain to be delivered with a specific objective (Spohrer and Maglio 2010). Service system entities dynamically configure four types of resources: people, technology/environment infrastructure, organizations(glossary)/institutions, and shared information/symbolic knowledge. Service systems can be either formal or informal in nature. In the case of formal service systems, the interactions are contracted through service level agreements (SLA). Informal service systems can promise to reconfigure resources without a written contractual agreement; in the case of the emergency transports operations example discussed in the Service Systems Background article, there is no formal contractual agreement (i.e., SLA) between the user requesting the service and the agency providing the service other than a "promise" for a quick and efficient response. SLAs are written contracts between and among service system entities, as well as the legal system for enforcing the contracts. The study of informal service systems contains the study of relationships (communications, interactions, and promises) between service systems and social systems, cultural norms and beliefs, as well as political systems that can maintain those relationships (Spohrer and Kwan 2008). The resources are either physical or non-physical and have rights or no rights. See Figure 1 below:



Service Value Chain

SLAs and policies specify the conditions under which services system entities reconfigure access rights to resources by mutually agreed value propositions. Current management frameworks typically focus on single service system entity interfaces. They neither use SLAs for managing the implementation and delivery of services nor do they recognize/support the fact that many services may be composed of lower-level services, involve third-party providers, and rely on possibly complex relationships and processes among participating businesses, information communications, and technologies (CoreGRID 2007). While SLAs are mapped to the respective customer requirements, policies are provider-specific means to express constraints and rules for their internal operations. These rules may be independent of any particular customer (Theilmann 2009).

In service systems practice, we describe the service value chain in terms of links among the entities connected via the Network Centric operations of service systems. For instance, value could then be created and delivered in terms of e-services, such as business-to-business (B2B), business to consumer (B2C), business to government (B2G), government-to-business (G2B), government-to-government (G2G), government-to-consumer (G2C), etc. The emerging service in this case interacts or “co-produces” with their customer via the World Wide Web as compared to the physical environment in which the traditional, or brick and mortar, service enterprises interact with their customers.

The services sector requires information as input, involves the customer at the production/delivery stage, and employs mostly qualitative measures to assess its performance, i.e., technology-intensive services are “information-driven, customer centric, e-oriented, and productivity-focused” (Tien and Berg 2003; Hipel et al. 2007; Chesbrough 2011). Chang (2010) defines these features in this manner:

- **Information Driven:** The creation, management, and sharing of information is crucial to the design, production, and delivery of services.
- **Customer Centric:** Customers are generally the co-producer of the services, as in the case of self-service. Customers require a certain degree of self-adaptation or customization and customers must be satisfied with the rendered services.

- **E (electronics) Oriented:** Services are becoming increasingly e-oriented. Thus, e-access, e-commerce, and e-customer management are crucial to e-services.
- **Productivity Focused:** Both efficiency and effectiveness are important in the design, delivery, and support of services.
- **Value Adding:** Services need to provide some value for the target clients. For profit-seeking service companies, the value produced for customers assures the company's profitability. For non-profit service entities, the value produced for customers reinforces the quality of a service entity's policy.

A service system is defined by its value co-creation chain in which stakeholders work in open collaboration to deliver consistently high quality service according to business goals, service goals, and customer goals. A value proposition can be viewed as a request from one service system to another to run an algorithm (the value proposition) from the perspectives of multiple stakeholders according to culturally determined value principles. The four primary stakeholder's perspectives in regards to value are the customer, provider, authority, and the competitors. Figure 2 below depicts value calculations from multiple stakeholder perspectives.

**Table 1. Value Calculation from Different Stakeholders' Perspectives (Spohrer 2011).
Reprinted with permission of Dr. James C. Spohrer. All other rights are reserved by the
copyright owner.**

Stakeholder Perspective (the players)	Measure Impacted	Pricing Decision	Basic Questions	Value Proposition Reasoning
1. Customer	Quality (Revenue)	Value Based	Should we? (offer it)	Model of customer: Do customers want it? Is there a market? How large? Growth rate?
2. Provider	Productivity (Profit, Mission, Continuous Improvement, Sustainability)	Cost Plus	Can we? (deliver it)	Model of self: Does it play to our strengths? Can we deliver it profitability to customers? Can we continue to improve?
3. Authority	Compliance (Taxes and Fines, Quality of Fire)	Regulated	May we? (offer and deliver it)	Model of authority: Is it legal? Does it compromise our integrity in any way? Does it create a moral hazard?
4. Competitor (Substitute)	Sustainable Innovation (Market Share)	Strategic	Will we? (invest to make it so)	Model of competitor: Does it put us ahead? Can we stay ahead? Does it differentiate us from the competition?

From an engineering design point of view, the service and business goals are an entry point through which to analyze the business architectures (including organization and processes) needed, which in turn demand alignment between the information technology (IT) components and technology architecture to achieve the goals. From a systems engineering perspective, the next step is to identify service system entities that could participate in the service delivery (people, organizations, technologies, processes, etc.).

Service System Entities

Spath and Fahnrich (2007) defined a service meta-model comprised of nine types of **entities**:

1. **Customers:** customer features, customer attitudes, and customer preferences;
2. **Goals:** business goals, service goals, customer goals, and enterprise culture goals;
3. **Inputs:** physical, human beings, information, knowledge, currency, and constraints;
4. **Outputs:** physical, human beings, information, knowledge, currency, and waste;
5. **Processes:** service provision, service operations, service support, customer relationships, planning and control, and call center management;
6. **Human Enablers:** service providers, support providers, management, and owner organization (enterprise);
7. **Physical Enablers:** owner organization (physical), buildings, equipment, furnishings, and location;
8. **Informatics Enablers:** information, knowledge, procedures and processes, decision support, and skill acquisition; and
9. **Environment:** political factors, economic factors, social factors, technological factors, environmental factors, legal factors (PESTEL), and physical factors.

Thus, a service or service offering is created by the relationships among service system entities (including information flows) through business processes into strategic capabilities that consistently provide superior value to the customer. If we were to represent the service as a network diagram (as in Figure 3 below), then the entities represent the nodes and the links represent the relationships between nodes.

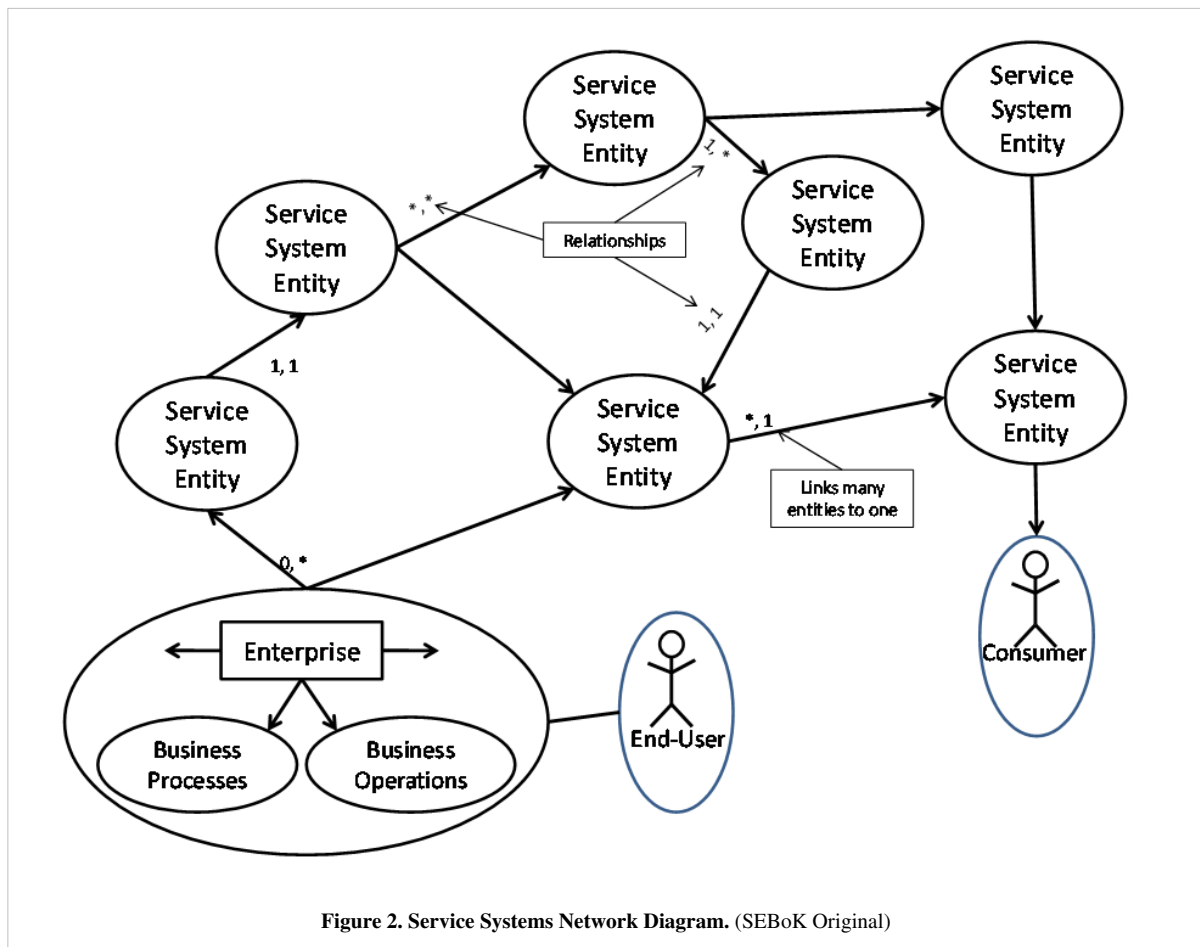


Figure 2. Service Systems Network Diagram. (SEBoK Original)

Service System Hierarchy

Systems are part of other systems which are often expressed by systems hierarchies (Skyttner 2010) to create a multilevel hierarchy, and thus the service system is composed of service system entities that interact through processes defined by governance and management rules to create different types of outcomes in the context of stakeholders with the purpose of providing improved customer interaction and value co-creation. Examples of service system entities are business enterprises, nations, or in the simplest form, a person (consumes and produces services).

Using the hierarchical approach, Spohrer conceptualizes an ecosystem at the highest level in which a service system is an entity of its own. This concept is extended to create the service system hierarchy as described in Figure 4 below (Spohrer 2011; Maglio and Spohrer 2008; Maglio et al. 2010).

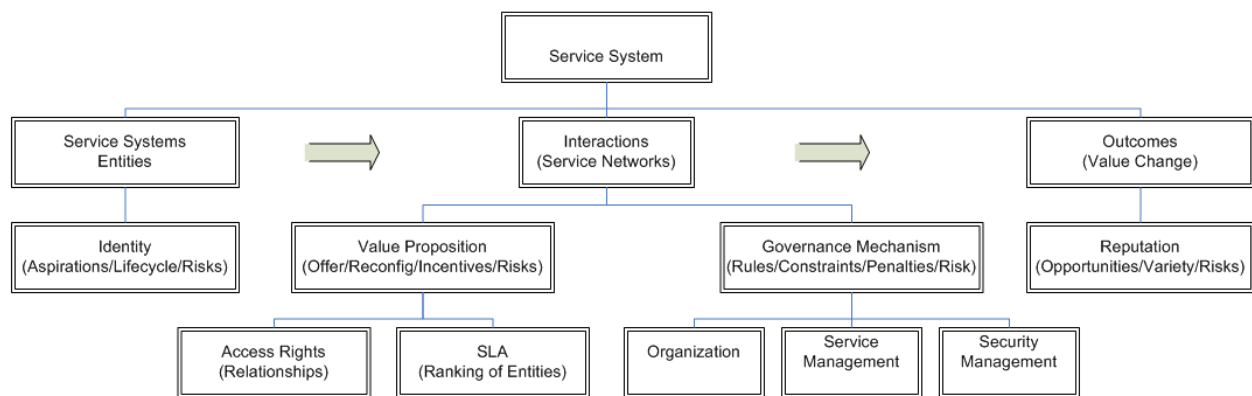


Figure 3. Service System Conceptual Framework (Spohrer 2011). Reprinted with permission of Dr. James C. Spohrer. All other rights are reserved by the copyright owner.

Service System Attributes

The fundamental attributes of a service system include togetherness, structure, behavior, and emergence. As mentioned earlier, today's global economy is very competitive and a service system may be very competitive in a given environment at a given time (the business space). The service system's trajectory should be well controlled as time goes by (Qiu 2009) since services are "real time in nature and are consumed at the time they are co-produced" (Tien and Berg 2003), that is, during service transactions.

The service system should evolve and adapt to the conditions within the business space in a manner which ensures that the customized service behaves as expected. This adaptive behavior of service systems implies that their design must be truly trans-disciplinary:

They must include techniques from social science (i.e., sociology, psychology, and philosophy) and management (i.e., organization, economics, and entrepreneurship). As a consequence, Systems, Man, and Cybernetics (SMC) must expand their systems (i.e., holistic oriented), man (i.e., decision-oriented), and cybernetics methods to include and be integrated with those techniques that are beyond science and engineering. (Hipel et al. 2007)

References

Works Cited

- Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. Hoboken, NJ, USA: John Wiley & Sons, Inc.
- Chesbrough, H. 2011. *Open Services Innovation: Rethinking Your Business to Grow and Compete in a New Era*. San Francisco, CA, USA: Jossey-Bass.
- CoreGRID. 2007. *Using SLA for Resource Management and Scheduling - A Survey. Technical Report 0096*. Jülich & Dortmund, Germany: European Research Network on Foundations, Software Infrastructures and Applications for Large Scale Distributed, GRID and Peer-to-Peer Technologies, Institute on Resource Management and Scheduling. Accessed June 4, 2011. Available: <https://www.coregrid.net/mambo/images/stories/TechnicalReports/tr-0096.pdf>.
- Hipel, K.W., M.M. Jamshidi, J.M. Tien, and C.C. White. 2007. "The Future of Systems, Man, and Cybernetics: Application Domains and Research Methods." *IEEE Transactions on Systems, Man, and Cybernetics - Part C: Applications and Reviews*. 37 (5): 726-743.
- Maglio, P., C. Kieliszewski, and J. Spohrer. 2010. *Handbook of Service Science*. New York, NY, USA: Springer Science and Business Media.
- Maglio, P., and J. Spohrer. 2008. "Fundamentals of Service Science." *Journal of the Academy of Marketing Science*. 36 (1): 18-20.
- Qiu, R. 2009. "Computational Thinking of Service Systems: Dynamics and Adaptiveness Modeling." *Service Science*. 1 (1): 42-55.
- Skyttner, L. 2006. *General Systems Theory: Perspectives, Problems, Practice*, 2nd ed. Singapore: World Scientific Publishing Company.
- Spath, D., and K.P. Fähnrich (eds.). 2007. *Advances in Services Innovations*. Berlin & Heidelberg, Germany: Springer-Verlag.
- Spohrer, J., and S.K. Kwan. 2009. "Service Science, Management, Engineering, and Design (SSMED): An Emerging Discipline - Outline & References." *International Journal of Information Systems in the Service Sector*, 1 (3): 1-31.
- Spohrer, J., and P.P. Maglio. 2010. "Chapter 1: Service Science: Toward a Smarter Planet," in *Introduction to Service Engineering*, edited by G. Salvendy and W. Karwowski. Hoboken, NJ: John Wiley & Sons.
- Spohrer, J.C. 2011. "Service Science: Progress & Directions." Presented at International Joint Conference on Service Science, May 25-27, 2011, Taipei, Taiwan.
- Theilmann, W., and L. Baresi. 2009. "Multi-level SLAs for Harmonized Management in the Future Internet," in *Towards the Future Internet - A European Research Perspective*, edited by G. Tselentis, J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zehariadis. Amsterdam, The Netherlands: IOS Press.
- Tien, J.M., and D. Berg. 2003. "A Case for Service Systems Engineering." *Journal of Systems Science and Systems Engineering*. 12 (1): 13-38.
- Vargo, S.L., and M.A. Akaka. 2009. "Service-Dominant Logic as a Foundation for Service Science: Clarifications." *Service Science*. 1 (1): 32-41.

Primary References

- Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. Hoboken, NJ, USA: John Wiley & Sons, Inc.
- Hipel, K.W., M.M. Jamshidi, J.M. Tien, and C.C. White. 2007. "The Future of Systems, Man, and Cybernetics: Application Domains and Research Methods." *IEEE Transactions on Systems, Man, and Cybernetics - Part C: Applications and Reviews*. 37 (5): 726-743.
- Spath, D., and K.P. Fährnich (eds.). 2007. *Advances in Services Innovations*. Berlin & Heidelberg, Germany: Springer-Verlag.
- Spohrer, J.C. 2011. "Service Science: Progress & Directions." Presented at International Joint Conference on Service Science, May 25-27, 2011, Taipei, Taiwan.

Additional References

- Chesbrough, H. 2011. *Open Services Innovation: Rethinking Your Business to Grow and Compete in a New Era*. San Francisco, CA, USA: Jossey-Bass.
- CoreGRID. 2007. *Using SLA for Resource Management and Scheduling - A Survey. Technical Report 0096*. Jülich & Dortmund, Germany: European Research Network on Foundations, Software Infrastructures and Applications for Large Scale Distributed, GRID and Peer-to-Peer Technologies, Institute on Resource Management and Scheduling. Accessed June 4, 2011. Available: <https://www.coregrid.net/mambo/images/stories/TechnicalReports/tr-0096.pdf>.
- Maglio, P., C. Kieliszewski, and J. Spohrer. 2010. *Handbook of Service Science*. New York, NY, USA: Springer Science and Business Media.
- Maglio, P., and J. Spohrer. 2008. "Fundamentals of Service Science." *Journal of the Academy of Marketing Science*. 36 (1): 18-20.
- Qiu, R. 2009. "Computational Thinking of Service Systems: Dynamics and Adaptiveness Modeling." *Service Science*. 1 (1): 42-55.
- Skyttner, L. 2006. *General Systems Theory: Perspectives, Problems, Practice*, 2nd ed. Singapore: World Scientific Publishing Company.
- Spohrer, J., and S.K. Kwan. 2009. "Service Science, Management, Engineering, and Design (SSMED): An Emerging Discipline - Outline & References." *International Journal of Information Systems in the Service Sector*, 1 (3): 1-31.
- Spohrer, J., and P.P Maglio. 2010. "Chapter 1: Service Science: Toward a Smarter Planet," in *Introduction to Service Engineering*, edited by G. Salvendy and W. Karwowski. Hoboken, NJ: John Wiley & Sons.
- Theilmann, W., and L. Baresi. 2009. "Multi-level SLAs for Harmonized Management in the Future Internet," in *Towards the Future Internet - A European Research Perspective*, edited by G. Tselentis, J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zehariadis. Amsterdam, The Netherlands: IOS Press.
- Tien, J.M., and D. Berg. 2003. "A Case for Service Systems Engineering." *Journal of Systems Science and Systems Engineering*. 12 (1): 13-38.
- Vargo, S.L., and M.A. Akaka. 2009. "Service-Dominant Logic as a Foundation for Service Science: Clarifications." *Service Science*. 1 (1): 32-41.

Properties of Services

- Lead Authors:
- Ricardo Pineda, Bud Lawson, and Brian Wells

A service is realized by the service system through the relationships of service system entities that interact (or relate) in a particular way to deliver the specific service via a service level agreement (SLA). Current management frameworks typically only focus on the interfaces of single service system entities. Meanwhile, SLAs are mapped to the respective customer requirements. These policies are provider-specific means to express constraints and rules for their internal operations. These rules may be independent of any particular customer (Theilmann 2009).

Services not only involve the interaction between the service provider and the consumer to produce value, but have other attributes, like an intangible quality of service (e.g., an ambulance service's availability and response time to an emergency request). The demand for a service may have varying loads dependent on the time of day, day of week, season, or other unexpected needs (e.g., natural disasters, product promotion campaigns, etc.). In the US for instance, travel services have peak demands during Christmas week; Mother's day is usually the highest volume handling day for a telecommunications provider and tax services peak during extended periods (January through mid-April). Services cannot be inventoried; they are rendered at the time they are requested.

Additionally, for a business enterprise, delivering the service at the minimum cost while maximizing its profits may be the service objective. In contrast, for a non-profit organization the objective may be to maximize customer satisfaction while optimizing the resources required to render the service (e.g., during a natural disaster). Thus, the design and operations of service systems "is all about finding the appropriate balance between the resources devoted to the systems and the demands placed on the system so that the quality of service to the customer is as good as possible" (Daskin 2010).

Service Level Agreement

A SLA is a set of technical (functional) and non-technical (non-functional) parameters agreed among customers and service providers. SLAs can and do contain administrative level (non-functional) business related parameters, such as SLA duration, service availability for the SLA duration, consequences for variations, failure reporting, priorities, and provisions for modifications to the SLA. However, for service level management, the service level (technical) parameters need to be defined, monitored, and assessed; these parameters may include such things as throughput; quality; availability; security; performance; reliability, for example, mean time between failure (MTBF), maximum downtime, and time-to-repair; and resource allocation.

An SLA represents the negotiated service level requirements (SLR) of the customer and should establish valid and reliable service performance measures since it is usually the basis for effective service level management (SLM). The goal of SLM is to ensure that service providers meet and maintain the prescribed quality of service (QoS). However, care should be taken since in some domains the term QoS refers only to resource reservation control mechanisms rather than the achieved service quality (e.g., internet protocol (IP) networks). Some terms used to mean the "achieved service quality" include quality of experience (QoE), user-perceived performance, and degree of satisfaction of the user; these other terms are more generally used across service domains.

Non-functional properties fall into two basic categories: business properties, such as price and method of payment, and environmental properties, such as time and location. Business and environmental properties are classified as "context properties" by Youakim Badr (Badr et al. 2008). QoS properties are characteristics such as availability, resilience, security, reliability, scalability, agreement duration, response times, repair times, usability, etc. Therefore services evaluation measures are customer oriented and include not only traditional performance metrics (productivity, quality, etc.), but also require a comprehensive analysis of the service system from an end-to-end perspective. Service evaluation typically includes customer demand-supply to ensure economic viability across the

lifecycle of the service system. Furthermore, the service delivery is evaluated using the key technical performance metrics listed above, adding also Service Process Measures (provisioning time, time-to-restore/repair, etc.) and Technical Performance Measures (end-to-end response times, latency, throughput, etc.). Finally, the service system's SLAs are then the composition of these categories evaluated on a systemic level to ensure consistency, equity, and sustainability of the service to assure that the desired/contracted SLA for customer satisfaction, value co-creation, and high system robustness are realized. (Spohrer 2011; Tien and Berg 2003; Theilmann and Baresi, 2009)

Service Key Performance Indicators

Service key performance indicators (KPI) are defined and agreed to in the SLA; the service KPIs are decomposed into service process measures (SPM) and technical performance measures (TPM) during the analysis stage of the service systems engineering (SSE) process. In the design process, the KPIs and TPM are allocated to service system entities and their components, as well as to the business processes and their components so as to ensure compliance with SLAs. The allocated measures generate derived requirements (SLR) for the system entities and their relationships, as well as for the service entities' components and the data and information flows required in the service systems to monitor, measure, and assess end-to-end SLA. These allocations ensure that the appropriate performance indicators apply to each of the links in the service value chain.

TPMs are typically categorized by the number of defective parts in a manufacturing service, data transmission latency and data throughput in an end-to-end application service, IP QoS expressed by latency, jitter delay, and throughput; SPMs are typically categorized by service provisioning time, end-to-end response times to a service request (a combination of data and objective feedback), and quality of experience (QoE verified by objective feedback). Together, the KPI (TPM combined with SPM) and perception measures make up the service level management function. A quality assurance system's (QAS) continuous service improvement (CSI), processes, and process quality management and improvement (PQMI) should be planned, designed, deployed, and managed for the capability to continuously improve the service system and to monitor compliance with SLAs (e.g., PQMI, capability maturity model integration (CMMI) (SEI 2007), International Organization for Standardization (ISO) Standards 9001 (ISO/IEC 2008), Telecom Quality Management System Standards (TL 9000) (QuEST Forum 2012), Information Technology Infrastructure Library (ITIL) v. 3 (OGC 2009), etc.).

As discussed earlier, QoS needs to correlate customer perceived quality (subjective measures) with objective SPM and TPM measures. There are several techniques available to help monitor, measure, and assess TPM's, but most are a variation on the theme of culling information from TPM's using, for example, perceptual speech quality measure (PSQM) and perceptual evaluation of video quality (PEVQ) and enhancing or verifying this information with customer or end-user perception of service by extending mean opinion score (MOS) techniques/customer opinion models (Ray 1984). Telecommunication systems engineering (TCSE) played an important role in finding methodologies for correlation between perception and objective measures for the services of the twentieth century; SSE should continue to encourage multidisciplinary participation to equally find methodologies, processes, and tools to correlate perceived service quality with TPM and with SPM for the services of the twenty-first century (Freeman 2004).

Subjective (qualitative) service quality is the customer's perceived conformity of the service with the expected objective. Word-of-mouth, personal needs, and past experiences create customer expectations regarding the service. The customers' perception of the service must be captured via surveys and interviews. The customers' perception of the service is then compared with their expectations for the service; this process captures the perceived service quality. Care should be taken to understand that subjective measures appear to measure customer attitudes, and attitudes may be the result of several encounters with the service, as well as numerous encounters with similar services.

In summary, the SLA documents the SLRs and establishes reliable and valid service performance measures, technical parameters, and the agreed performance levels for the technical parameters. The technical parameters are

then monitored and continuously compared against both objective and subjective data culled from multiple internal and external sources (service level management). The goal is not to report the level of service in a given period, but to develop and implement a dynamic system capable of predicting and driving service level improvement over time (i.e., continual service improvement (CSI)).

Evolution of Services

The second, third, and fourth decades of the twenty-first century will almost certainly see similar, and probably accelerated, technology development as seen in the prior three decades. Mass collaboration will become an established mode of operation. The beginnings of mass collaboration have manifested in developments such as value co-creation where loosely entangled actors or entities come together to create value in unprecedented ways, but ways that meet mutual and broader market requirements. Further developments in the technology, use, and acceptance of social media will continue to fuel the acceleration of these developments.

The next decades will see the grounding of concepts, such as crowdsourcing, coined by Jeff Howe in a June 2006 *Wired* magazine article; open innovation, promoted by Henry Chesbrough, a professor and executive director at the Center for Open Innovation at Berkeley; and mass collaboration and open source innovation supported by Enterprise 2.0 tools, as conceived by Wikinomics consultant Don Tapscott.

Roberto Saracco, a telecommunications expert specializing in analyzing economical impacts of technology evolution, argues that: "Communications will be the invisible fabric connecting us and the world whenever and wherever we happen to be in a completely seamless way, connecting us so transparently, cheaply, and effortlessly that very seldom will we think about it." The ubiquity and invisibility of these communications will greatly facilitate the creation and destruction of ad hoc collectives (groups of entities that share or are motivated by at least one common issue or interest, or work together on a specific project(s) to achieve a common objective). This enterprise may engender the concept of the hive mind (the collective intelligence of many), which will be an intelligent version of real-life super organisms, such as ant or bee nests (Hölldobler and Wilson 2009).

These models will most certainly give rise to issues of property rights and liabilities; access rights for both the provider and the customer can be owned outright, contracted/leased, shared, or have privileged access (Spohrer 2011). For now, we are on the cusp of a management revolution that is likely to be as profound and unsettling as the one that gave birth to the modern industrial age. Driven by the emergence of powerful new collaborative technologies, this transformation will radically reshape the nature of work, the boundaries of the enterprise, and the responsibilities of business leaders (McAfee 2009).

The service-providing industry in the US is divided into thirteen sectors (Chang 2010):

1. professional and business services,
2. healthcare and social assistance,
3. state and local government,
4. leisure and hospitality,
5. other services,
6. educational services,
7. retail trade,
8. financial activities,
9. transportation and warehousing,
10. wholesale trade,
11. information,
12. federal government, and
13. utilities.

Spohrer (2011) goes beyond the service sectors to propose three types of service systems:

1. **Systems that focus on flow of things:** transportation and supply chains, water and waste recycling, food and products, energy and electric Grid, information/ICT & cloud;
2. **Systems that focus on Human Activities and Development:** buildings and construction, retail and hospitality / media and entertainment industries, banking and finance / business consulting industries, healthcare and family life systems, education and work life / jobs and entrepreneurship; and
3. **Systems that focus on Governing:** cities, states, and nations.

Categorizing types and sectors of services is an important beginning because it can lead to a better understanding of the emerging rules and relationships in service value chains. This approach can further enhance the value co-creation capabilities of innovative service concepts that contribute to our quality of life. The classification also helps in identifying different objectives and constraints for the design and operations of the service system. Some examples include strategic policies under limited budget: education, strategic with readiness for quick response; national defense; business enterprise, maximizing profit while minimizing cost; etc.

In addition, this classification is being used to determine the overlap and synergies required among different science disciplines to enable trans-disciplinary collaboration and educational programs.

References

Works Cited

- Badr, Y., A. Abraham, F. Biennier, and C. Grosan. 2008. "Enhancing Web Service Selection by User Preferences of Non-Functional Features." Presented at 4th International Conference on Next Generation Web Services Practices, October 20-22, 2008, Seoul, South Korea.
- Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. New York, NY, USA: John Wiley & Sons, Inc.
- Daskin, M.S. 2010. *Service Science*. New York, NY, USA: John Wiley & Sons.
- Freeman, R.L. 2004. *Telecommunication Systems Engineering*, 4th ed. New York, NY, USA: John Wiley & Sons.
- Hölldobler, B., and E.O. Wilson. 2009. *The Super-organism: The Beauty, Elegance, and Strangeness of Insect Societies*. New York, NY, USA: W.W. Norton & Company.
- ISO. 2008, ISO 9001:2008, *Quality management systems -- Requirements*. Geneva, Switzerland: International Organisation for Standardisation.
- McAfee, A. 2009. *Enterprise 2.0: New Collaborative Tools for Your Organization's Toughest Challenges*. Boston, MA, USA: Harvard Business School Press.
- OGC (Office of Government Commerce). 2009. *ITIL Lifecycle Publication Suite Books*. London, UK: The Stationery Office.
- QuEST Forum. 2012. *Quality Management System (QMS) Measurements Handbook*, Release 5.0. Plano, TX, USA: Quest Forum.
- Ray, R.F. (ed). 1984. *Engineering and Operations in Bell System*, 2nd ed. Florham Park, NJ, USA: AT&T Bell Labs.
- SEI. 2007. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.2. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- Spohrer, J.C. 2011. "Service Science: Progress & Directions." Presented at the International Joint Conference on Service Science, 25-27 May 2011, Taipei, Taiwan.
- Theilmann, W., and L. Baresi. 2009. "Multi-level SLAs for Harmonized Management in the Future Internet," in *Towards the Future Internet - A European Research Perspective*, edited by G. Tselentis, J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zehariadis. Amsterdam, The Netherlands: IOS Press.

Tien, J.M., and D. Berg. 2003. "A Case for Service Systems Engineering." *Journal of Systems Science and Systems Engineering*. 12 (1): 13-38.

Primary References

Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. New York, NY, USA: John Wiley & Sons, Inc.

Theilmann, W., and L. Baresi. 2009. "Multi-level SLAs for Harmonized Management in the Future Internet," in *Towards the Future Internet - A European Research Perspective*, edited by G. Tselentis, J. Domingue, A. Galis, A. Gavras, D. Hausheer, S. Krco, V. Lotz, and T. Zehariadis. Amsterdam, The Netherlands: IOS Press.

Additional References

None.

Scope of Service Systems Engineering

- Lead Authors:
- Ricardo Pineda and Bud Lawson

Service systems engineering (SSE) involves all aspects of the enterprise. This topic discusses different aspects of the scope of SSE, from organizational strategy, to interoperability, to the life cycle of services, and then to their design.

SSE and the Enterprise

Enterprises plan, develop, and manage the enhancements of their infrastructure, products, and services, including marketing strategies for product and service offerings. These plans propose new products or service offerings based on new, unexplored, or unforeseen customer needs with clearly differentiated value propositions. Service strategies are the internal business processes required to design, operate, and deliver services. The mission of service strategies is to develop the capacity to achieve and maintain a strategic advantage (OGC 2009).

Taking the systems engineering (SE) approach to service systems, or (SSE), is imperative for the service-oriented, customer-centric holistic view to select and combine service system entities. The SSE approach can then define and discover relationships among service system entities to plan, design, adapt, or self-adapt to co-create value. The SSE approach should identify linkages, relationships, constraints, challenges/problems, new technologies, interoperability standards, interface agreements, or process development requirements among service entities required for the planned service or for potential future services (Lefever 2005).

SSE mandates participation not only from engineering, business operations, and customers, but also from various different domains, such as management science, behavioral science, social science, systems science, network science, computer science, decision informatics, etc.

Hipel et al. (2007) have presented a table for service science in terms of the domains and methods, including not only service systems, but also infrastructure and transportation systems, environmental and energy systems, and defense and space systems. The collaboration domains in Figure 1 below are a first approximation to the collaboration required from different disciplines for the SSE paradigm.

Table 1. Service Systems Engineering Domain Collaboration. (Hipel et al. 2007) Reprinted with permission of © Copyright IEEE - All rights reserved.

SEE	Collaboration Domains
SSE Management	• Management Science • Business Process Management • Cognitive Science • Decision Science
Service Realization Process (SRP)	• All engineering fields • Business Operations • Infrastructure Operations • Social Science • Computer Science • Management Science • Behavioral Science • Network Science • Computational Science • Systems Science • Decision Science
Methodologies, Processes, and Tools (MPT)	• Natural Science • Business Science (BPMN) • Mathematical • All engineering fields

Major challenges faced by SSE include the dynamic nature of service systems evolving and adapting to constantly changing operations and/or business environments, and the need to overcome silos of knowledge. Interoperability of service system entities through interface agreements must be at the forefront of the SSE design process for the harmonization of operations, administration, maintenance, and provisioning procedures of the individual service system entities (Pineda 2010).

In addition, service systems require open collaboration among all stakeholders, but recent research on mental models of multidisciplinary teams shows integration and collaboration into cohesive teams has proven to be a major challenge (Carpenter et al. 2010) (See also Team Dynamics). Thus, the emphasis on multidisciplinary (e.g., scientific, engineering, management, and social) education and training programs required to foster systems thinking helps bridge the gaps created by these silos of knowledge.

In the SSE approach, the social, governance, business, service, operations, and management activities are linked together through the service life cycle; service systems are by themselves a type of system of systems (SoS) where traditional systems engineering (TSE) practices need to be extended to include service systems entities' relationships (e.g., interface agreements among people, organizations, processes, and technologies) through information flows, technical interoperability, governance, and access rights within a system of systems.

Interoperability of Services

Interoperability among the different service system entities becomes highly relevant in SSE since the constituent entities are designed according to stakeholder needs; the entity is usually managed and operated to satisfy its own objectives independently of other system entities. The objectives of individual service system entities may not necessarily converge with the overall objectives of the service system. Thus, the need to include the following in the definition of a service system: analysis and design of the service system, governance frameworks to align political objectives, service strategies, business objectives, information and communications technologies (ICT) objectives, technology objectives and end-to-end operations, administration and maintenance procedures, and allocation of these procedures to individual entities (Luzeaux and Ruault 2010).

The previous discussion relates to a new service system development. There may be instances where a service is planned for delivery in phases of deployment (transition/deployment phase), or as presented earlier, if there is already a service system defined and deployed, then it's possible that the new request is for a service based application (SBA), in which case, the process is more focused on the adaptations needed to deploy the new application. For SBA, instances of advances in computer engineering, computer science, and software development already permit the adaptation and creation of SBA in a run-time environment for the discovery, development, and publishing of applications (Maglio et al. 2010).

The service design process (SDP) for new services is triggered by the market concept of the intended service and considers the stakeholder(s), service value chain(s), target market(s), target customer(s), proposed SLA, demand forecast, pricing strategy, and customer access privileges, which together comprise the service strategy. The SDP process then adapts the TSE as a life cycle approach (concept/definition, design/development, deployment/transition, operations, life cycle management/utilization/CSI, and retirement) as discussed in Life Cycle Models. A more detailed list of the SSE process activities is described in Value of Service Systems Engineering and Service Systems Engineering Stages.

Service Lifecycle Stages

The SDP stages and notation are depicted in Table 2 below; due to the complexity of service systems (see also Emergence and Complexity) the documents generated are becoming more model-based electronic documents than written binders depending on the methodologies and tools used.

Table 2. Service Realization Process: Life Cycle Stages. (SEBoK Original)

<html>

Life Cycle Changes		Purpose	Decision Gates
Service Strategy/Concept	New Service identification	<i>Elicit enterprise needs</i> <i>Explore service concepts</i> <i>Identify service system entities</i> <i>Propose viable HL black box solutions</i> Output: Service Description	Decision Options - Go, No-GO - Continue this stage - Go to preceding stage - Hold project activity - Terminate project - Test - Deploy
	Feasibility Phase		
	HL Analysis		
Service Design/Development	Service Requirement Analysis and Engineering	<i>Refine service system requirements</i> Output: Service Requirement Document <i>Create solution description</i> <i>Identify Interfaces among entities</i> Output: Preliminary Design	
	Service Development	<i>Develop service system detailed architecture and specs</i> Output: Service Specification Document <i>Verify and Validate system requirements</i> Output: service JV & V Plans	
	Service Integration, Verification, and Validation		
Service Transition/Deployment		<i>Service Insertion Plans</i> <i>Deploy service system</i> <i>Manage deployment activities</i> <i>Inspect and test (verify)</i> Output: Service Operation Plans, Operations Technical Plans, Operational Readiness Plans	
Service Operations and / Continuous Service Improvement		<i>Operate a reliable service system to satisfy customer needs</i> <i>Monitor, Measure, & Assess</i> <i>Provide sustained system capability</i> <i>Troubleshoot potential issues</i> <i>Store, archive, or dispose of the service system</i>	

</html>

All the life cycle stages are included for completeness, but very often during the concept analysis phase it may be determined that not all of the stages are needed. In these cases, a recommendation should be made regarding which stages are specifically required for the realization of the service in question.

Service Design Management

Another important role of SSE is the management of the service design process. SSE utilizes TSE practices to manage the resource and asset allocation to perform the activities required to realize the service through the value chain for both the customer and the service provider. The main focus of the service design process management is to provide for the planning, organizational structure, collaboration environment, and program controls to ensure that stakeholder's needs are met from an end-to-end customer perspective.

The service design process management process aligns business objectives and business operational plans with end-to-end service objectives, including customer management plans, service management and operations plans, and operations technical plans. The main SSE management activities are

- planning;
- assessment and control;
- decision management;
- risk management;
- configuration management; and
- information management.

SSE plays a critical role in describing the needs of the intended service in terms of the service's day-to-day operations, including customer care center requirements, interface among service system entities, such as: manufacturing plant, smart grid, hospital, network infrastructure provider(s), content provider(s) and service provider(s), service based application provider(s), applications providers, and the customer management process for the service.

Current research in computer engineering and software systems engineering is looking at the development of run-time platforms to allow real time or near real time customer service discovery and publishing (Spark 2009). The service-centric systems engineering (SeCSE) consortium has a well-defined service design process that is being applied to SBA. In this approach, there are design time and run-time sub-processes for the composition, provisioning, orchestration, and testing for service publishing (Lefever 2005). There is particular interest from the research community to include human-computer interactions (HCI) and behavioral science to address current social networking services (Facebook, Twitter, LinkedIn, Google+, etc.) used to share unverified information via audio, messaging, video, chats, etc.

This research is gaining relevance because of the thin line between the customer (consumer, enterprise) and content providers in regards to security, privacy, information authentication, and possible misuse of the user-generated content. Even as the research progresses, these networking services are examples of business models organizing communities of interest for innovation. Hsu says, "If we understand this networking, then we may be able to see through the business strategies and systems design laws that optimize connected value co-creation" (2009).

References

Works Cited

- Carpenter, S., H. Delugach, L. Etzkorn, J. Fortune, D. Utley, and S. Virani. 2010. "The Effect of Shared Mental Models on Team Performance." Presented at Industrial Engineering Research Conference, Institute of Industrial Engineers, 2010, Cancun, Mexico.
- Hipel, K.W., M.M. Jamshidi, J.M. Tien, and C.C. White. 2007. "The Future of Systems, Man, and Cybernetics: Application Domains and research Methods." *IEEE Transactions on Systems, Man, and Cybernetics-Part C: Applications and Reviews*. 37 (5): 726-743.
- Hsu, C. 2009. "Service Science and Network Science." *Service Science*, 1 (2): i-ii.
- Lefever, B. 2005. *SeSCE Methodology*. Rome, Italy: SeCSE Service Centric Systems Engineering. SeCSE511680. Available: https://www.secse-project.eu/wp-content/uploads/2007/08/a5_d4-secse-methodology-v1_3.pdf.
- Luzeaux, D., and J.R. Ruault (eds.). 2010. *Systems of Systems*. New York, NY, USA: John Wiley & Sons.
- Maglio, P., M. Weske, J. Yang, and M. Fantinato (eds.). 2010. *Proceedings of the 8th International Conference on Service Oriented Computing: ICSOC 2010*. Berlin & Heidelberg, Germany: Springer-Verlag.
- OGC (Office of Government Commerce). 2009. *ITIL Lifecycle Publication Suite Books*. London, UK: The Stationery Office.
- Pineda, R. 2010. "Understanding Complex Systems of Systems Engineering." Presented at Fourth General Assembly Cartagena Network of Engineering, 2010, Metz, France.
- Spark, D. 2009. "Real-Time Search and Discovery of the Social Web." Spark Media Solutions Report. Accessed September 2, 2011. Available: <https://www.sparkminute.com/2009/12/07/free-report-real-time-search-and-discovery-of-the-social-web/>.

Primary References

- Lefever, B. 2005. *SeSCE Methodology*. Rome, Italy: SeCSE Service Centric Systems Engineering. SeCSE511680. Available: https://www.secse-project.eu/wp-content/uploads/2007/08/a5_d4-secse-methodology-v1_3.pdf.
- Luzeaux, D., and J.R. Ruault (eds.). 2010. *Systems of Systems*. New York, NY, USA: John Wiley & Sons.

Additional References

None.

Value of Service Systems Engineering

- Lead Authors:
- Ricardo Pineda, Bud Lawson, and Richard Turner

Service systems engineering (SSE) is a multidisciplinary approach to manage and design value co-creation of a service system. It extends the holistic view of a system to a customer-centric, end-to-end view of service system design. Service systems engineers must play the role of an integrator by considering the interface requirements for the interoperability of service system entities, not only for technical integration, but also for the processes and organization required for optimal customer experience during service operations.

Service systems engineering uses disciplined approaches to minimize risk by coordinating/orchestrating social aspects, governance (including security), environmental, human behavior, business, customer care, service management, operations, and technology development processes. Therefore, systems engineers must have a good understanding of cross disciplinary issues to manage, communicate, plan, and organize service systems development and delivery of service. Service systems engineering also brings a customer focus to promote service excellence and to facilitate service innovation through the use of emerging technologies to propose creation of new service systems and value co-creation.

The service design process includes the definition of methods, processes, and procedures necessary to monitor and track service requirements verification and validation, in particular as they relate to the operations, administration, maintenance, and provisioning procedures of the whole service system and its entities. These procedures ensure that failures by any entity are detected and do not propagate and disturb the operations of the service (Luzaux and Ruault 2010).

Research on service systems needs to fuse business process management, service innovation, and social networks for the modeling of service system value chain (Carroll et al. 2010). The systems engineering approach helps to better understand and manage conflict, thereby helping both private and public organizations optimize their strategic decision making. The use of a systemic approach reduces rework, overall time to market, and total cost of development.

Service SE Knowledge & Skills

The world's economies continue to move toward the creation and delivery of more innovative services. To best prepare tomorrow's leaders, new disciplines are needed that include and ingrain different skills and create the knowledge to support such global services. "In this evolving world, a new kind of engineer is needed, one who can think broadly across disciplines and consider the human dimensions that are at the heart of every design challenge" (Grasso and Martinelli 2007).

Service systems engineers fit the T-shaped model of professionals (Maglio and Spohrer 2008) who must have a deeply developed specialty area, as well as a broad set of skills and capabilities (See the Enabling Individuals article). Chang (2010) lists the following twelve service system management and engineering (SSME) skills:

1. Management of Service Systems. These skills include scheduling, budgeting and management of information systems/technologies, and leadership;
2. Operations of Service Systems. Engineers should be proficient in process evaluation and improvement, quality improvement, customer relationships, and uncertainty management;
3. Service Processes. These skills include performance measurements, flow charting, work task breakdown;
4. Business Management. Business skills include project costing, business planning, and change management;
5. Analytical Skills. These skills include problem solving, economic decision analysis, risk analysis, cost estimating, probability and statistics;

6. **Interpersonal Skills.** Increasingly, service systems engineers are expected to excel in professional responsibility, verbal skills, technical writing, facilitating, and team building;
7. **Knowledge Management.** Service systems engineers should be familiar with definition, strategies, success factors, hurdles, and best practices in industry;
8. **Creativity and Innovation in Services.** These skills include creative thinking methods, success factors, value chain, best practices, and future of innovation;
9. **Financial and Cost Analysis and Management.** Additional business skills include activity-based costing, cost estimation under uncertainty, T-account, financial statements, ratio analysis, balanced scoreboards, and capital formation;
10. **Marketing Management.** Market forecast, market segmentation, marketing mix- service, price, communications and distribution- are important marketing tools;
11. **Ethics and Integrity.** Service Systems Engineers must be held to high ethical standards. These include practicing ethics in workplace and clear guidelines for making tough ethical decisions, corporate ethics programs, affirmation action, and workforce diversity, as well as global issues related to ethics. (See Ethical Behavior); and
12. **Global Orientation.** Increasingly, engineers must be aware of emerging business trends and challenges with regards to globalization drivers, global opportunities, and global leadership qualities.

Service Architecture, Modeling & Views

Successful deployment of service value chains is highly dependent on the alignment of the service with the overall enterprise service strategy, customer expectations, and customer's service experience. The importance of service-oriented customer-centric design has been recognized for several years by traditional service providers (telecommunications, information technology (IT), business reengineering, web services, etc.) through the creation of process-driven architecture frameworks.

Architecture frameworks are important for creating a holistic system view. They promote a common understanding of the major building blocks and their interrelation in systems of systems or complex systems of systems (see also Emergence and Complexity). An architecture is a model of the system created to describe the entities, the interactions and interoperability among entities, as well as the expected behavior, utilization, and properties of the end-to-end system. The architectures become the main tool to guide stakeholders, developers, third-party providers, operations managers, service managers, and users in the understanding of the end-to-end service system, as well as to enable governance at the service management and the service development levels.

These architecture frameworks have been defined through standards bodies and/or by private enterprises that recognize their advantage—standard processes that integrate the business-strategic processes and operations with the information technology and technology infrastructure (See Systems Engineering Standards). Most architecture frameworks model different scopes and levels of detail of business strategies, product and service offerings, business operations, and organizational aspects. Unfortunately, there are currently no frameworks that cover all the aspects (views) required to model the service systems. Some frameworks focus on business strategies, others in business process management, others in business operations, still others in aligning IT strategy or technology strategy to business strategy. Thus, a combination of architecture frameworks is required to create the enterprise service system model. For instance, an enterprise may use an enterprise business architecture (EBA) model covering strategic goals and objectives, business organization, and business services and processes where driven by market evolution, technology evolution, and customer demands. However, a reference framework would be needed to model the IT strategy (e.g., Information Technology Infrastructure Library (ITIL) v. 3 (OGC 2009)) and the organizations and processes needed to deliver, maintain, and manage the IT services according to the business strategy.

Service Architecture Frameworks

Prime examples of Service Architecture Frameworks are listed below.

Standards:

- Zachmann Framework (Zachman 2003)
- Business Process Modeling (BPM) (Hantry et al. 2010)
- The Open Group Architecture Framework (TOGAF) (TOGAF 2009)
- Enhanced-Telecomm Operations Map (eTOM) by the TeleManagement Forum (eTOM 2009)
- Service Oriented Architecture (SOA) (Erl 2008)
- National Institute of Standards and Technology (NIST) Smart Grid Reference Model (NIST 2010)
- Web services business process execution language (WS-BPEL) (OASIS 2007)
- Department of Defense Architecture Framework (DoDAF) (DoD 2010)
- Others.

Proprietary Enterprise Architecture Frameworks:

- Hewlett - Packard IT Service Management Reference Model (HP ITSMRM 2000)
- International Business Machines Systems Management Solutions Life Cycle, IBM Rational Software.
- Microsoft Operations Framework

This list represents only a sample of the existing service architecture frameworks.

One great example of architecture frameworks applications for service systems, the “High Level Reference Model for the Smart Grid,” developed by NIST in 2010 under the “Energy Independence and Security Act of 2007” (EISA), is presented below:

EISA designated the development of a Smart Grid as a national policy goal, specifying that an interoperability framework should be “flexible, uniform and technology neutral. The law also instructed that the framework should accommodate “traditional, centralized generation and distribution resources” while also facilitating incorporation of new, innovative Smart Grid technologies, such as distributed renewable energy resources and energy storage. (NIST 2010)

The NIST reference model was developed as “a tool for identifying the standards and protocols needed to ensure interoperability and cyber security, and defining and developing architectures for systems and subsystems within the smart grid.” Figure 1 illustrates this model and the strategic (organizational), informational (business operations, data structures, and information exchanges required among system entities), and technical needs of the smart grid (data structures, entities specifications, interoperability requirements, etc.).

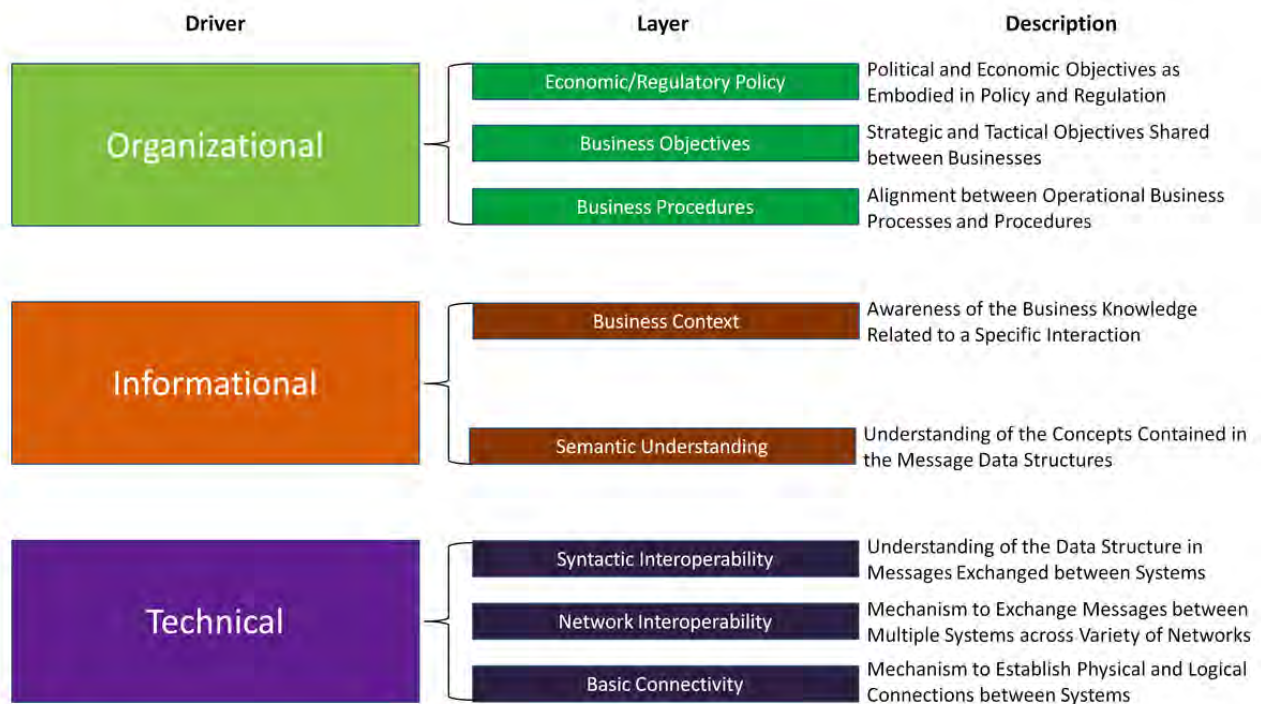


Figure 1. The Grid-Wide Architecture Council's Eight-Layered Stack (NIST and US Dept. of Commerce 2010). Released.

The NIST reference model uses this architecture framework to identify existing standards, identify new standards required for interoperability among interconnected networks, and to enable innovations where smart grid components (energy sources, bulk generation, storage, distribution, transmission, metering, cyber infrastructure, markets, service providers, customers, etc.) are supported by a broad range of interoperable options by well-defined interfaces useful across industries, including security. Emerging/innovative service development with massively scaled, well-managed, and secured networks will enable a dynamic market driven ecosystem representing new economic growth (NIST 2010).

This architecture framework is being used today by different standards organizations, such as the Smart Grid Interoperability Panel (SGIP), and several smart grid working groups. For details on priorities, working programs, and working group charters, see "High Level Reference Model for the Smart Grid" (NIST 2010).

For service systems, the application of any of these frameworks requires modifications/adaptations to create dynamic frameworks aware of environmental changes due to competitor's offerings, market demands, and customer co-creation. Most frameworks are static in nature; this requires business operations to manage changes through pre-defined (pre-programmed) processes for service configuration and change control. Dynamic frameworks would allow real-time, or near real-time, analysis of impacts of newly discovered service on business processes, organizations, and revenue for run-time environment deployment.

Automatic service configuration and change control are being incorporated into the management process via service oriented architecture (SOA) for service automation (Gu et al. 2010) and service oriented computing (Maglio et al. 2010). In particular, progress has been made over the last ten years on the standards for adaptation, orchestration and creation of web services (WS) for service based applications (SBA). A good summary of existing life cycle approaches for adaptable and evolvable SBA is presented in (Papazoglou et al. 2010). Some examples of this are

- web services development life cycle (SDLC);

- rational unified process (RUP) for SOA;
- service oriented modeling and architecture (SOMA); and
- service oriented analysis and design/decision Modeling (SOAD).

Further research is required to understand the architectural implications of dynamic service configuration, including research on human behavior, social aspects, governance processes, business processes, and implications of dynamic service level agreements (SLA) for an enterprise service system. New ways are needed to include adaptation requirements for new technologies that will exchange information with the service system entities and may have their own specifications. These technologies include robots, sensors, renewable energy, nanotechnologies, three dimensional printers, and implantable medical devices.

References

Works Cited

- Carroll, N., E. Whelan, and I. Richardson. 2010. "Applying Social Network Analysis to Discover Service Innovation within Agile Service Networks." *Service Science*. 2 (4): 225-244.
- Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. New York, NY, USA: John Wiley & Sons, Inc.
- DoD. 2010. *DoD Architecture Framework (DoDAF)*, version 2.0. Arlington, VA, USA: US Department of Defense (DoD).
- Erl, T. 2008. *SOA Principles of Service Design*. Boston, MA, USA: Prentice Hall, Pearson Education.
- eTOM. 2009. "Business Process Framework." Morristown, NJ: TeleManagement Forum. Accessed May 30, 2011 at <https://www.tmforum.org/BusinessProcessFramework/1647/home.html>.
- Grasso, D., and D. Martinelli. 2007. "Section B: Holistic Engineering." *The Chronicle Review, The Chronicles of Higher Education*. Vol. 53, Issue 28. Page 8B. March 2007.
- Gu, Q., Cuadrado, F., Lago, P. and Duenas, J.C. 2010. "Architecture views illustrating the service automation aspect of SOA". *Service research challenges and solutions for the future internet*. 339-372.
- Hantry, F., M.P. Papazoglou, W. van den Heuvel, R. Haque, E. Whelan, N. Carroll, D. Karastoyanova, F. Leymann, C. Nikolaou, W. Lamersdorf, and M. Hacid. 2010. "Business Process Management," in *Service Research Challenges and Solutions for the Future Internet S-Cube – Towards Engineering, Managing and Adapting Service-Based Systems*, edited by M. Papazoglou, K. Pohl, M. Parkin, and A. Metzger. Berlin and Heidelberg, Germany: Springer-Verlag. 27-54.
- HP ITSMRM. 2000. "HP IT Service Management Reference Model. Technical White Paper." Palo Alto, California, USA: Hewlett – Packard Company. Accessed September 2, 2011. Available: ftp://ftp.hp.com/pub/services/itsm/info/itsm_rmwp.pdf.
- Luzeaux, D., and J.R. Ruault (eds.). 2010. *Systems of Systems*. New York, NY, USA: John Wiley & Sons.
- Maglio, P., and J. Spohrer. 2008. "Fundamentals of Service Science." *Journal of the Academy of Marketing Science*. 36 (1): 18-20.
- National Institute of Standard and Technology (NIST). 2010. *NIST Framework and Roadmap for Smart Grid Interoperability Standards Release 1.0*. Gaithersburg, MD, USA: Office of the National Coordinator for Smart Grid Interoperability, US Department of Commerce. Accessed September 2, 2011. Available: <https://www.nist.gov/smartgrid/upload/FinalSGDoc2010019-corr010411-2.pdf>.
- OASIS. 2007. "Web Services Business Process Execution Language Version 2.0." Organization for Advancement of Structured Information Standards (OASIS) Standard. Accessed September 2, 2011. Available: [https:// docs.oasis-open.org/webscgm/v2.0/OS/webscgm-v2.0.pdf](https://docs.oasis-open.org/webscgm/v2.0/OS/webscgm-v2.0.pdf).

OGC (Office of Government Commerce). 2009. *ITIL Lifecycle Publication Suite Books*. London, UK: The Stationery Office.

Papazoglou, M., K. Pohl, M. Parkin, and A. Metzger. 1998. "Service Research Challenges and Solutions for the Future Internet," in *Service Research Challenges and Solutions for the Future Internet S-Cube – Towards Engineering, Managing and Adapting Service-Based Systems*. Berlin and Heidelberg, Germany: Springer-Verlag.

TOGAF. 2009. "The Open Group Architecture Framework," version 9. The Open Architecture Group. Accessed September 2, 2011. Available: <https://www.opengroup.org/togaf>.

Zachman, J. 2003. "The Zachman Framework for Enterprise Architecture: Primer for Enterprise Engineering and Manufacturing." Accessed September 2, 2011. Available: <https://www.zachmanframeworkassociates.com/index.php/ebook>.

Primary References

Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. New York, NY, USA: John Wiley & Sons, Inc.

Erl, T. 2008. *SOA Principles of Service Design*. Boston, MA, USA: Prentice Hall, Pearson Education.

Hantry, F., M.P. Papazoglou, W. van den Heuvel, R. Haque, E. Whelan, N. Carroll, D. Karastoyanova, F. Leymann, C. Nikolaou, W. Lamersdorf, and M. Hacid. 2010. "Business Process Management," in *Service Research Challenges and Solutions for the Future Internet S-Cube – Towards Engineering, Managing and Adapting Service-Based Systems*, edited by M. Papazoglou, K. Pohl, M. Parkin, and A. Metzger. Berlin and Heidelberg, Germany: Springer-Verlag. p. 27-54.

National Institute of Standard and Technology (NIST). 2010. *NIST Framework and Roadmap for Smart Grid Interoperability Standards Release 1.0*. Gaithersburg, MD, USA: Office of the National Coordinator for Smart Grid Interoperability, US Department of Commerce. Accessed September 2, 2011. Available: <https://www.nist.gov/smartgrid/upload/FinalSGDoc2010019-corr010411-2.pdf>.

Additional References

None.

Service Systems Engineering Stages

- Lead Authors:
- Ricardo Pineda, Bud Lawson, and Richard Turner

This article describes the stages of the service systems development process (SSDP) and expected outputs for each stage; for a closer alignment with the traditional systems engineering (TSE) process, the concept and feasibility phases have been combined into a single service strategy/concept as discussed in the SEBoK Systems Engineering and Management article. All of the stages of the SSDP take a similar iterative approach to fully understand the enterprise capabilities, enterprise process impact, information technology (IT), and technology impacts and customer expectations. Lin and Hsieh (2011) provide a good summary on New service Development processes. The Information Technology Infrastructure Library (ITIL) stage names have been purposely added to the SSDP to show the needed alignment between IT and technology. The reader should keep in mind that even though IT is crucial to the overall end-to-end system, service technology development needs must be taken into consideration in all the stages of SSDP.

Service Strategy/Concept

A service strategy/concept is the entry into the SSDP. The concept may be generated by an end-user (enterprise customer or consumer), a business manager, an engineering organization, new web service designers, new technology developments, and/or information technology trends. The service concept is the highest level of the service idea and it usually addresses what service is being proposed to what markets and to whom within these markets.

A high-level feasibility assessment of the concept is then carried out by the integrated service development team (ISDT) to assess the needs/impacts on enterprise process capabilities, operational capabilities, and/or new technology developments (access, infrastructure, operations support systems (OSS), service support systems (SSS), and business support systems (BSS). It should also consider any impacts on service governance, social, cultural, and human behaviors. The feasibility assessment also gives a plus or minus 30% estimate on the time to develop and the cost of development, which are entry points into the business case to evaluate whether the service is viable to develop and to market given the constraints and estimates. At this time, a decision (decision gate) determines if the service is to be developed.

If the business case is viable, then a detailed business description of the service is developed. This includes functions and features to be included, phases of development, markets to be addressed, customers within the markets to be targeted, and customer experiences expected from the service (i.e., defining the non-functional requirements of the service, such as the quality of service (QoS), availability, reliability, and security considerations and offerings within the service). This description allows detailed studies of expected human-computer interactions, social networking, technology requirements, and operations requirements. Governance and organizational process requirements should also be included to generate the “service description” as the main output from this stage.

Service systems engineering (SSE) takes an important role in understanding and eliciting the enterprise service concepts. Clearly, understood end-to-end business processes required for the intended service are fundamental to its successful development, deployment, and customer satisfaction. SSE works with business process management (BPM), social science, and cognitive science to elicit intended service operations, including target audiences, pre-sale, sale, and post-sale customer care processes.

Requirements Analysis and Engineering

A service requirements document is created that describes the service functions, the service entities, the intended interaction among entities, and the customer-facing and internal-facing functions/processes that are required to support the service. This description should conceptually include intended service level agreements (SLAs) and the obligations of the service provider process should there be any degree of non-compliance during service operation.

In addition to the TSE activities described earlier, the SSE requirements analysis and engineering process must develop a customer-centric view of the service to analyze SLA, QoS, value co-creation, monitoring, and assessment requirements to comply with the expected/planned SLA. This analysis will determine whether dynamic changes of the service are required during service operation to correct faults, reconfigure, administer, or to adapt/self-adapt for possible performance degradations.

Beyond the traditional service life cycle management (LCM) processes, the requirements must also be developed for service level management (SLM) processes and systems. These are needed to monitor, measure, and assess key performance indicators (KPIs), technical performance measures (TPMs), and service performance measures (SPMs) according to the SLA.

The SSE requirements analysis addresses the support systems for the governance, business, service, operations, and support processes to derive requirements for technologies, information systems, processes, and enterprise organizations. Interface requirements, information flows, and data requirements are also within the scope of requirements analysis. The main output is the service requirements document (SRD).

SSE plays a critical role in describing the services needs for day-to-day operations. These include customer care centers requirements and interfaces between network infrastructure provider(s), content provider(s), service provider(s), service based application provider(s), and the customer management process for the service. All of these are described in detail in the service operations plans (SOPs) and the operations technical plans (OTPs).

Systems Design/Development

The SRD, SOP, and OTP have enough detail regarding the service functions, operations, interfaces, and information flows required among the different service system entities to analyze, identify, and recommend end-to-end applicable architecture frameworks; to carry out trade-off analyses for the alternatives among service system entities; and to describe and allocate relationships (interactions) among entities at all levels of the service architecture. Detailed requirements are worked at lower levels to generate specifications for entity developers including data structures, data flow diagrams, and allocated performance requirements.

ITIL v. 3 (OGC 2007) recommends inclusion of the following service design processes:

- service catalog management,
 - service level management,
 - capacity management,
 - availability management,
 - service continuity management,
 - security management, and
 - supplier/provider management.
-

Service Integration, Verification & Validation

SSE defines integration and interface requirements for the seamless operation of the service. In this regard, the system engineer takes an integrator role to ensure proper data generation and flow through all the different systems composing the service offered. The goal is to ensure customers (consumer or internal) are getting the information required to carry out the tasks required in the business, operations, service, and customer processes. The service integration, verification, and validation plans need to include end-to-end verification and validation procedures for any new development or adaptations required for planned dynamic configuration/re-configuration of previously tested service systems. (See also System Verification and System Validation.)

The systems engineer creates these plans using a number of different perspectives. These include:

- end-to-end service (service validation test plans),
- customer care (operational readiness test plans),
- service provider (network validation test plans),
- service system entities interoperability/interface test plans,
- content provider (content validation test plans), and
- application (user acceptance test plans).

Service Transition/Deployment

Service systems may change very rapidly and new enhancements, new features, or new applications can be added as incremental developments, new developments, or adaptation to service offerings. Service systems engineers review new requirements to assess the feasibility of the changes to the service system entities, technologies, processes, and organizations, as well as their impacts on the service offerings. The service transition/deployment stage takes input from service development to plan for service insertion, technology insertion, processes adaptations, and implementation with minimal impact to existing services. During this stage, special care is taken with integration, verification, and validation test plans and regression testing to ensure new developments work flawlessly with existing services.

ITIL v. 3 (OGC 2007) recommends the following processes in the transition/deployment stage:

- transition planning and support,
- change management,
- service asset and configuration management,
- release and deployment management,
- service validation and testing,
- evaluation, and
- knowledge management.

Service Operations/Continuous Service Improvement (CSI)

Service operation manages the day-to-day activities of all aspects of the end-to-end service delivery to the customer. It manages the operations, administration, maintenance, and provisioning of the service, technology, and infrastructure required to deliver the contracted service to the customer within the specified service levels. The main service operations processes in ITIL v. 3 are

- event management,
 - incident management,
 - problem management,
 - request fulfillment, and
 - access management.
-

A continuous service improvement (CSI) plan for the implementation of technologies and tools for the continuous improvement of the service, monitoring, measuring, and analyzing process and service metrics is essential.

Service Systems Engineering Tools & Technologies

Tools and technologies from a broad spectrum of fields are extensively used during the different stages of SSE. Not only are they used for the development of the hardware, software, information systems and technology components, but also for the modelling, definition, and design of the organization, processes, and data structures of the service system (See also Representing Systems with Models). These tools and technologies include modelling, simulation, development, test bed, and social environmental aspects of the intended or to be designed service. The tools fall into three main domains:

1. business process management (BPM),
2. service design process, and
3. service design management.

Business process management (BPM) generally deals with process management scenarios to coordinate people and systems, including sequential workflow, straight through processing, case management, content life cycle management, collaborative process work, and value chain participation. Systems engineers work with service managers to align the business architectures with the technology and IT architecture. The business process modeling notation (BPMN) is a graphic notation standard that is implemented to describe a process's realization within any given workflow. This notation is linked with web services business process execution language (WS-BPEL), a format used to perform an automated business process by implementing web services technology. For an extensive review of existing BPM tools and BPM suites, please see Hantry et al. (2010), Carroll et al. (2010), Andrikoupolous et al. (2010), Lin and Hsieh (2011), and Ward-Dutton (2010).

Service design process: Architecture frameworks (AF) and enterprise architectures (EAs) are standards that help split complex systems (see also Emergence and Complexity) into an interrelated, structured form. They describe the different characteristics of the products and services. Systems engineering modeling tools, such as the unified modeling language (UML) (OMG 2010a) and system modeling language (SysML) (OMG 2010b), help develop the AF and EA and greatly impact the continued evolution and successful implementation of complex projects. Service oriented architecture (SOA) and systems and software engineering architecture (ISO/IEC/IEEE 2011) are standards that apply architecture principles for specialized applications. Successful implementation of the architecture tools helps identify critical interfaces and improves understanding of the allocations between components and functions.

Model-based systems engineering (MBSE), model driven architectures (MDA), and model oriented systems engineering (MOSES) are examples of commonly used tools for logical (functional), behavioral (operational), and physical design of the IT. UML, UML 2.0, and SysML are extensively used to describe operational scenarios, modes of operations, use cases, and entity relationships. For an extensive review of MBSE, MDA, and MOSES, please see Friedenthal (1998), Estefan (2008), Pezuela (2005), Andrikopoulos et al. (2010), and Hybertson (2010).

In addition, trade-off and engineering analyses use different optimization methodologies. Since services exhibit a significant level of randomness, statistical analysis, demand forecasting, multi-objective optimization, queuing theory, and stochastic optimization methodologies are tools used to model and simulate the service system behavior. These methodologies support decision making in areas as diverse as resource allocation, number of facilities, facilities' geographical locations, fleet routing and optimization, service systems reliability and prognosis, and network optimization. A good overview of these methodologies can be found in Daskin (2010).

During the service design process (SDP), planning for the implementation of technologies and tools for the continuous improvement of the service is performed. These tools support monitoring, measuring, and analyzing process and service performance metrics. The Deming cycle (plan, do, check, and act (PDCA) is widely used as the foundation for quality improvements across the service. Lean manufacturing, six sigma, swim lanes, balanced scoreboard, benchmarking, and gap analysis methodologies are commonly used for service evaluation and

continuous improvement.

Service design management: There are standards for implementing and managing systems engineering processes (IEEE 1220 (1998)) that help coordinate and synchronize all the service systems engineering processes leading to improved organizational collaboration and improved service delivery (see also Systems Engineering Standards). Standards have been developed in software engineering for product evaluation (ISO/IEC 14598 (1998)) and product quality (ISO/IEC 9126 series (2003a, 2003b, & 2004)), as well as information security management (ISO 27001 (2005)) and evaluation series (ISO 15408 (2008a, 2008b, & 2009)). The ITIL v. 3 describes best practices for IT service management, which can be extended to include service systems.

References

Works Cited

- Adams, S., A. Cartlidge, A. Hanna, S. Rance, J. Sowerby, and J. Windebank. 2009. *ITIL V3 Foundation Handbook*. London, England, UK: The Stationary Office.
- Andrikopoulos, V., A. Bucchiarone, E. Di Nitto, R. Kazhamiakin, S. Lane, V. Mazza, and I. Richardson. 2010. "Chapter 8: Service Engineering," in *Service Research Challenges and Solutions for the Future Internet S-Cube – Towards Engineering, Managing and Adapting Service-Based Systems*, edited by M. Papazoglou, K. Pohl, M. Parkin, and A. Metzger. Berlin and Heidelberg, Germany: Springer-Verlag. p. 271-337.
- Carroll, N., E. Whelan, and I. Richardson. 2010. "Applying Social Network Analysis to Discover Service Innovation within Agile Service Networks." *Service Science*. 2 (4): 225-244.
- Daskin, M.S. 2010. *Service Science*. New York, NY, USA: John Wiley & Sons.
- Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, rev B. Seattle, WA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02.
- Friedenthal, S. 1998. "Object Oriented System Engineering: Process Integration for 2000 and beyond." Presented at System Engineering & Software Symposium, Lockheed Martin Corporation, 1998, New Orleans, LA.
- Hantry, F., M.P. Papazoglou, W. van den Heuvel, R. Haque, E. Whelan, N. Carroll, D. Karastoyanova, F. Leymann, C. Nikolaou, W. Lamersdorf, and M. Hacid. 2010. "Business Process Management," in *Service Research Challenges and Solutions for the Future Internet S-Cube – Towards Engineering, Managing and Adapting Service-Based Systems*, edited by M. Papazoglou, K. Pohl, M. Parkin, and A. Metzger. Berlin and Heidelberg, Germany: Springer-Verlag. p. 27-54.
- Hybertson, D.W. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boston, MA, USA: Auerbach Publications.
- IEEE. 1998. IEEE 1220-1998, *IEEE Standard for Application and Management of the Systems Engineering Process*. Washington, DC, USA: Institute of Electrical and Electronics Engineers.
- ISO/IEC. 1998. ISO/IEC 14598-5:1998, *Information technology — Part 5: Process for evaluators*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- ISO/IEC. 2003a. ISO/IEC TR 9126-2:2003, *Software engineering — Product quality — Part 2: External metrics*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- ISO/IEC. 2003b. ISO/IEC TR 9126-3:2003, *Software engineering — Product quality — Part 3: Internal metrics*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- ISO/IEC. 2004. ISO/IEC TR 9126-4:2004, *Software engineering — Product quality — Part 4: Quality in use metrics*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.

- ISO/IEC. 2005. ISO/IEC 27001:2005, *Information technology — Security techniques — Information security management systems — Requirements*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- ISO/IEC. 2008a. ISO/IEC 15408-2:2008, *Information technology — Security techniques — Evaluation criteria for IT security — Part 2: Security functional components*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- ISO/IEC. 2008b. ISO/IEC 15408-3:2008, *Information technology — Security techniques — Evaluation criteria for IT security — Part 3: Security assurance components*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- ISO/IEC. 2009. ISO/IEC 15408-1:2009, *Information technology — Security techniques — Evaluation criteria for IT security — Part 1: Introduction and general model*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- ISO/IEC/IEEE. 2011. ISO/IEC/IEEE 42010:2011, *Systems and software engineering — Architecture description*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commission.
- Lefever, B. 2005. "SeSCE Methodology." Rome, Italy: SeCSE Service Centric Systems Engineering. SeCSE511680. Available: https://www.secse-project.eu/wp-content/uploads/2007/08/a5_d4-secse-methodology-v1_3.pdf.
- Lin, F., and P. Hsieh. 2011. "A SAT View on New Service Development." *Service Science*. 3 (2): 141-157.
- OGC (Office of Government Commerce). 2007. *ITIL Lifecycle Publication Suite Books*. London, England, UK: The Stationery Office.
- OMG. 2010a. *Unified Modeling Language™ (UML)*, version 2. Needham, MA, USA: Object Management Group. Available: <https://www.omg.org/spec/UML/>.
- OMG. 2010b. *OMG Systems Modeling Language (SysML)*, version 1.2. Needham, MA, USA: Object Management Group. Available: <https://www.sysml.org/docs/specs/OMGSysML-v1.2-10-06-02.pdf>.
- Pezuela, C. 2005. "Collection of Existing Service Centric Prototypes." Report A5.D1. Brussels, Belgium: European Union, Information Society Technology. Accessed September 5, 2011. Available: <https://www.secse-project.eu/wp-content/uploads/2007/08/a5d1-collection-of-existing-service-centric-prototypes.pdf>.
- Ward-Dutton, N. 2010. "BPM Technology: Vendor Capability Comparison." MWD Premium Advisory Report. Horsham, West Sussex, UK: Macehiter Ward-Dutton (MWD) Limited. MWD Advisors. Accessed September 5, 2011. Available: <https://www.mwdadvisors.com/library/detail.php?id=380>.

Primary References

- Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, rev B. Seattle, WA, USA: International Council on Systems Engineering. INCOSE-TD-2007-003-02.
- Hybertson, D.W. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Boston, MA, USA: Auerbach Publications.
- Lefever, B. 2005. "SeSCE Methodology." Rome, Italy: SeCSE Service Centric Systems Engineering. SeCSE511680. Available: https://www.secse-project.eu/wp-content/uploads/2007/08/a5_d4-secse-methodology-v1_3.pdf.
- OGC (Office of Government Commerce). 2007. *ITIL Lifecycle Publication Suite Books*. London, England, UK: The Stationery Office.
-

Additional References

None.

Knowledge Area: Enterprise Systems Engineering

Enterprise Systems Engineering

Contents of this Knowledge Area

- Enterprise Systems Engineering Background (James Martin, Dick Fairley, and Bud Lawson) (Alan Faisandier)
- The Enterprise as a System (James Martin, Bud Lawson, and Judith Dahmann)
- Related Business Activities (James Martin, Dick Fairley, and Bud Lawson)
- Enterprise Systems Engineering Key Concepts (James Martin, Bud Lawson, and Alan Faisandier)
- Enterprise Systems Engineering Process Activities (James Martin, Bud Lawson, and Alan Faisandier)
- Enterprise Capability Management (James Martin, Bud Lawson, and Alan Faisandier)
- Lead Authors:
- James Martin, Dick Fairley, and Bud Lawson

Enterprise systems engineering (ESE) is the application of systems engineering principles, concepts, and methods to the planning, design, improvement, and operation of an enterprise.

Introduction

This knowledge area provides an introduction to systems engineering (SE) at the enterprise level in contrast to “traditional” SE (TSE) (sometimes called “conventional” or “classical” SE) performed in a development project or to “product” engineering (often called product development in the SE literature).

The concept of enterprise was instrumental in the great expansion of world trade in the 17th century (see note 1) and again during the Industrial Revolution of the 18th and 19th centuries. The world may be at the cusp of another global revolution enabled by the information age and the technologies and cultures of the Internet (see note 2). The discipline of SE now has the unique opportunity of providing the tools and methods for the next round of enterprise transformations.

Note 1. “The Dutch East India Company... was a chartered company established in 1602, when the States-General of the Netherlands granted it a 21-year monopoly to carry out colonial activities in Asia. It was the first multinational corporation in the world and the first company to issue stock. It was also arguably the world's first mega-corporation, possessing quasi-governmental powers, including the ability to wage war, negotiate treaties, coin money, and establish colonies.” (emphasis added, National Library of the Netherlands 2010)

Note 2. This new revolution is being enabled by cheap and easily usable technology, global availability of information and knowledge, and increased mobility and adaptability of human capital. The enterprise level of analysis is only feasible now because organizations can work together to form enterprises in a much more fluid manner.

ESE is an emerging discipline that focuses on frameworks, tools, and problem-solving approaches for dealing with the inherent complexities of the enterprise. Furthermore, ESE addresses more than just solving problems; it also deals with the exploitation of opportunities for better ways to achieve the enterprise goals. A good overall description of ESE is provided by in the book by Rebovich and White (2011).

Key Terms

Enterprise

An enterprise consists of a purposeful combination (e.g., a network) of interdependent resources (e.g., people, processes, organizations, supporting technologies, and funding) that interact with

- each other to coordinate functions, share information, allocate funding, create workflows, and make decisions, etc.; and
- their environment(s) to achieve business and operational goals through a complex web of interactions distributed across geography and time (Rebovich and White 2011, 4-35).

The term enterprise has been defined as follows:

(1) *One or more organizations sharing a definite mission, goals, and objectives to offer an output such as a product or service.* (ISO 2000);

(2) *An organization (or cross organizational entity) supporting a defined business scope and mission that includes interdependent resources (people, organizations and technologies) that must coordinate their functions and share information in support of a common mission (or set of related missions).* (CIO Council 1999);

(3) *The term enterprise can be defined in one of two ways. The first is when the entity being considered is tightly bounded and directed by a single executive function. The second is when organizational boundaries are less well defined and where there may be multiple owners in terms of direction of the resources being employed. The common factor is that both entities exist to achieve specified outcomes.* (MOD 2004); and

(4) *A complex, (adaptive) socio-technical system that comprises interdependent resources of people, processes, information, and technology that must interact with each other and their environment in support of a common mission.* (Giachetti 2010)

An enterprise must do two things: (1) develop things within the enterprise to serve as either external offerings or as internal mechanisms to enable achievement of enterprise operations, and (2) transform the enterprise itself so that it can most effectively and efficiently perform its operations and survive in its competitive and constrained environment.

Enterprise vs Organization

It is worth noting that an enterprise is not equivalent to an "organization" according to the definition above. This is a frequent misuse of the term enterprise. The figure below shows that an enterprise includes not only the organizations that participate in it, but also people, knowledge, and other assets such as processes, principles, policies, practices, doctrine, theories, beliefs, facilities, land, intellectual property, and so on.

Some enterprises are organizations, but not all enterprises are organizations. Likewise, not all organizations are enterprises. Some enterprises have no readily identifiable "organizations" in them. Some enterprises are self-organizing (i.e., not organized by mandate) in that the sentient beings in the enterprise will find for themselves some way in which they can interact to produce greater results than can be done by the individuals alone. Self-organizing enterprises are often more flexible and agile than if they were organized from above (Dyer and Ericksen 2009; Stacey 2006).

One type of enterprise architecture that supports agility is a non-hierarchical organization without a single point of control. Individuals function autonomously, constantly interacting with each other to define the vision and aims, maintain a common understanding of requirements and monitor the work that needs to be done. Roles and responsibilities are not predetermined but rather emerge from individuals' self-organizing activities and are constantly in flux. Similarly, projects are generated

everywhere in the enterprise, sometimes even from outside affiliates. Key decisions are made collaboratively, on the spot, and on the fly. Because of this, knowledge, power, and intelligence are spread through the enterprise, making it uniquely capable of quickly recovering and adapting to the loss of any key enterprise component. (https://en.wikipedia.org/wiki/Business_agility)

In spite of this lack of "organization" in some enterprises, SE can still contribute much in the engineering of the enterprise, as described in the articles below. However, SE must be prepared to apply some non-traditional approaches in doing so. Hence the need for embracing the new discipline called enterprise systems engineering (ESE).

Giachetti (2010) distinguishes between enterprise and organization by saying that an organization is a view of the enterprise. The organization view defines the structure and relationships of the organizational units, people, and other actors in an enterprise. Using this definition, we would say that all enterprises have some type of organization, whether formal, informal, hierarchical or self-organizing network.

Extended Enterprise

Sometimes it is prudent to consider a broader scope than merely the "boundaries" of the organizations involved in an enterprise. In some cases, it is necessary (and wise) to consider the "extended enterprise" in modeling, assessment, and decision making. This could include upstream suppliers, downstream consumers, and end user organizations, and perhaps even "sidestream" partners and key stakeholders. The extended enterprise can be defined as:

Wider organization representing all associated entities - customers, employees, suppliers, distributors, etc. - who directly or indirectly, formally or informally, collaborate in the design, development, production, and delivery of a product (or service) to the end user. (<https://www.businessdictionary.com>)

Enterprise Systems Engineering

Enterprise systems engineering (ESE), for the purpose of this article, is defined as the application of SE principles, concepts, and methods to the planning, design, improvement, and operation of an enterprise (see note 3). To enable more efficient and effective enterprise transformation, the enterprise needs to be looked at "as a system," rather than merely as a collection of functions connected solely by information systems and shared facilities (Rouse 2009). While a systems perspective is required for dealing with the enterprise, this is rarely the task or responsibility of people who call themselves systems engineers.

Note 3. This form of systems engineering (i.e., ESE) includes (1) those traditional principles, concepts, and methods that work well in an enterprise environment, plus (2) an evolving set of newer ideas, precepts, and initiatives derived from complexity theory and the behavior of complex systems (such as those observed in nature and human languages).

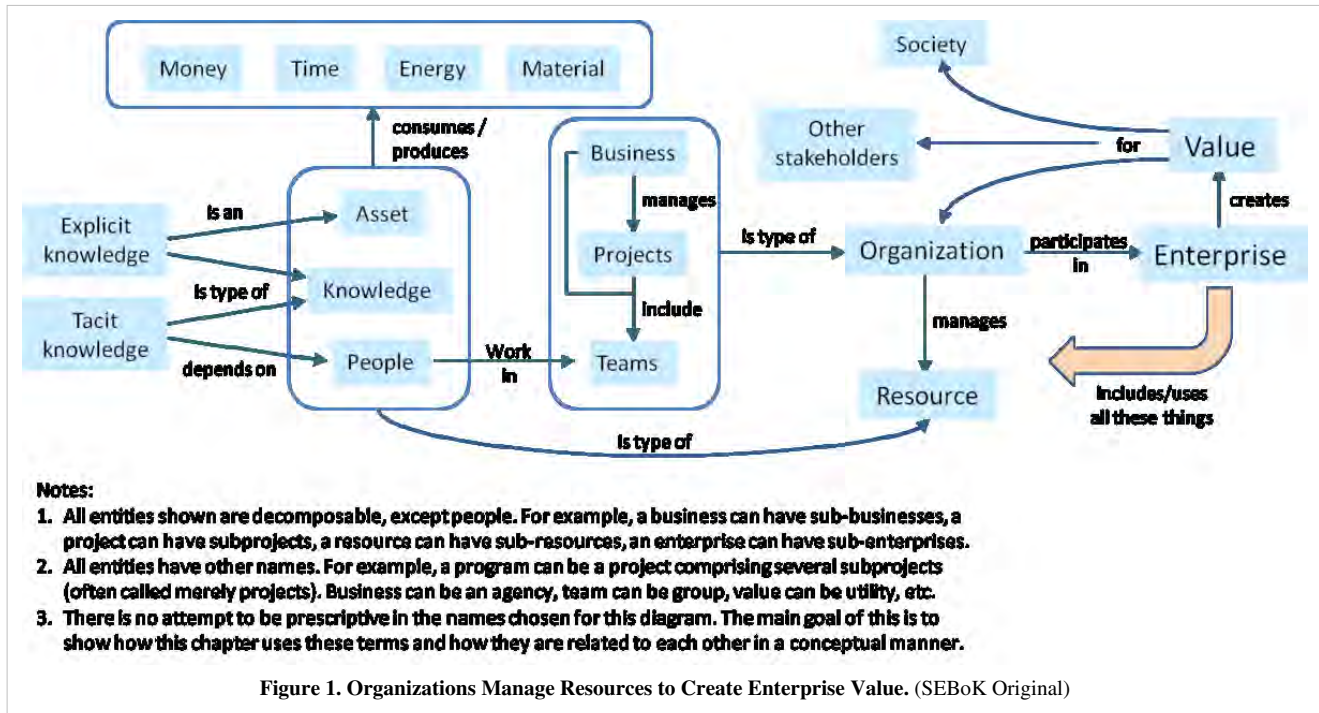
Creating Value

The primary purpose of an enterprise is to create value for society, other stakeholders, and for the organizations that participate in that enterprise. This is illustrated in Figure 1 that shows all the key elements that contribute to this value creation process.

There are three types of organizations of interest: businesses, projects, and teams (see note 4). A typical business participates in multiple enterprises through its portfolio of projects. Large SE projects can be enterprises in their own right, with participation by many different businesses, and may be organized as a number of sub-projects.

Note 4. The use of the word "business" is not intended to mean only for-profit commercial ventures. As used here, it also includes government agencies and not-for-profit organizations, as well as commercial ventures. Business is the activity of providing goods and services involving financial, commercial, and industrial

aspects.



Resource Optimization

A key choice for businesses that conduct SE is to what extent, if at all, they seek to optimize their use of resources (people, knowledge, assets) across teams, projects, and business units. Optimization of resources is not the goal in itself, but rather a means to achieve the goal of maximizing value for the enterprise and its stakeholders. At one extreme, in a product-oriented organization, projects may be responsible for hiring, training, and firing their own staff, as well as managing all assets required for their delivery of products or services. (The term "product-oriented organization" is not meant in the sense of product-oriented SE, but rather in the sense of this being one of the basic constructs available when formulating organizational strategy.)

At the other extreme, in a functional organization, the projects delegate almost all their work to functional groups. In between these two extremes is a matrix organization that is used to give functional specialists a "home" between project assignments. A full discussion of organizational approaches and situations along with their applicability in enabling SE for the organization is provided in the article called Systems Engineering Organizational Strategy.

The optimization debate can be handled as described in the book called "Enterprise Architecture as Strategy" (Ross, Weill, and Robertson 2006). In other words, an enterprise can choose (or not) to unify its operations and can choose (or not) to unify its information base. There are different strategies the enterprise might adopt to achieve and sustain value creation (and how ESE helps an enterprise to choose). This is further addressed in the section on Enterprise Architecture Formulation & Assessment in the article called Enterprise Capability Management.

Enabling Systems Engineering in the Organization

SE skills, techniques, and resources are relevant to many enterprise functions, and a well-founded SE capability can make a substantial contribution at the enterprise level, as well as at the project level. The article called Systems Engineering Organizational Strategy discusses enabling SE in the organization, while the article called Enabling Businesses and Enterprises focuses on the cross-organizational functions at the business and enterprise levels. The competence of individuals is discussed in the article called Enabling Individuals.

Kinds of Knowledge Used by the Enterprise

Knowledge is a key resource for ESE. There are generally two kinds of knowledge: explicit and tacit. Explicit knowledge can be written down or incorporated in computer codes. Much of the relevant knowledge, however, is “tacit knowledge” that only exists within the heads of people and in the context of relationships that people form with each other (e.g., team, project, and business level knowledge). The ability of an organization to create value is critically dependent on the people it employs, on what they know, how they work together, and how well they are organized and motivated to contribute to the organization’s purpose.

Projects, Programs & Businesses

The term “program” is used in various ways in different domains. In some domains a team can be called a program (e.g., a customer support team is their customer relationship “program”). In others, an entire business is called a program (e.g., a wireless communications business unit program), and in others the whole enterprise is called a program (e.g., the Joint Strike Fighter program and the Apollo Space program). And in many cases, the terms project and program are used interchangeably with no discernible distinction in their meaning or scope. Typically, but not always, there are program managers who have profit and loss (P&L) responsibility and are the ultimate program decision makers. A program manager may have a portfolio of items (services, products, facilities, intellectual property, etc.) that are usually provided, implemented, or acquired through projects.

The Office of Government Commerce provides a useful distinction between programs and projects:

The ultimate goal of a Programme is to realise outcomes and benefits of strategic relevance. To achieve this a programme is designed as a temporary flexible organisation structure created to coordinate, direct and oversee the implementation of a set of related projects and activities in order to deliver outcomes and benefits related to the organisation’s strategic objectives...

A programme is likely to have a life that spans several years. A Project is usually of shorter duration (a few months perhaps) and will be focussed on the creation of a set of deliverables within agreed cost, time and quality parameters. (OGC 2010)

Practical Considerations

When it comes to performing SE at the enterprise level, there are several good practices to keep in mind (Rebovich and White 2011):

- Set enterprise fitness as the key measure of system success. Leverage game theory and ecology, along with the practices of satisfying and governing the commons.
- Deal with uncertainty and conflict in the enterprise through adaptation: variety, selection, exploration, and experimentation.
- Leverage the practice of layered architectures with loose couplers and the theory of order and chaos in networks.

Enterprise governance involves shaping the political, operational, economic, and technical (POET) landscape. One should not try to control the enterprise like one would in a TSE effort at the project level.

References

Works Cited

- BusinessDictionary.com, "Extended Enterprise." Accessed September 12, 2012. Available: <https://www.businessdictionary.com/definition/extended-enterprise.html>.
- CIO Council. 1999. *Federal Enterprise Architecture Framework (FEAF)*. Washington, DC, USA: Chief Information Officer (CIO) Council.
- Dyer, L., and J. Ericksen. 2009. "Complexity-based Agile Enterprises: Putting Self-Organizing Emergence to Work," in *The Sage Handbook of Human Resource Management*, edited by A. Wilkinson et al. London, UK: Sage. p. 436–457.
- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.
- ISO. 2000. ISO 15704:2000, *Industrial Automation Systems -- Requirements for Enterprise-Reference Architectures and Methodologies*. Geneva, Switzerland: International Organization for Standardization (ISO).
- MOD. 2004. *Ministry of Defence Architecture Framework (MODAF)*, version 2. London, UK: UK Ministry of Defence.
- National Library of the Netherlands. 2010. "Dossier VOC (1602-1799)." Accessed September 12, 2012. Available: <https://www.kb.nl/dossiers/voc/voc.html> (in Dutch).
- OGC (Office of Government Commerce). 2010. *Guidelines for Managing Programmes: Understanding programmes and programme management*. London, UK: The Stationery Office.
- Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group, Auerbach.
- Ross, J.W., P. Weill, and D. Robertson. 2006. *Enterprise Architecture As Strategy: Creating a Foundation for Business Execution*. Boston, MA, USA: Harvard Business Review Press.
- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of Systems Engineering and Management*, 2nd ed., edited by A.P. Sage and W. B. Rouse. New York, NY, USA: Wiley and Sons, Inc.
- Stacey, R. 2006. "The Science of Complexity: An Alternative Perspective for Strategic Change Processes," in *Complexity and Organization: Readings and Conversations*, edited by R. MacIntosh et al. London, UK: Routledge. p. 74–100.
- Wikipedia contributors, "Business agility," *Wikipedia, The Free Encyclopedia*. Accessed November 28, 2012. Available: https://en.wikipedia.org/w/index.php?title=Business_agility&oldid=503858042.

Primary References

- Bernus, P., L. Nemes, and G. Schmidt (eds.). 2003. *Handbook on Enterprise Architecture*. Berlin and Heidelberg, Germany: Springer-Verlag.
- Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group, Auerbach.
- Rouse, W.B. 2005. "Enterprise as Systems: Essential Challenges and Enterprise Transformation." *Systems Engineering, the Journal of the International Council on Systems Engineering (INCOSE)*. 8 (2): 138-50.
- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of Systems Engineering and Management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY, USA: Wiley and Sons, Inc.
- Valerdi, R. and D.J. Nightingale. 2011. "An Introduction to the Journal of Enterprise Transformation." *Journal of Enterprise Transformation*. 1 (1): 1-6.

Additional References

- Drucker, P.F. 1994. "The theory of business." *Harvard Business Review*. 72 (5): 95-104.
- Fox, M., J.F. Chionglo, and F.G. Fadel. 1993. "A common sense model of the enterprise." Presented at the 3rd Industrial Engineering Research Conference, 1993, Norcross, GA, USA.
- Joannou, P. 2007. "Enterprise, systems, and software—the need for integration." *Computer*. 40 (5): 103-105.
- Journal of Enterprise Architecture*. Available: <https://www.globalaea.org/?page=JEAOverview>.
- MITRE. 2012. "Enterprise Engineering," in *Systems Engineering Guide*, MITRE Corporation. Accessed 8 July 2012. Available: https://www.mitre.org/work/systems_engineering/guide/enterprise_engineering/.
- Nightingale, D., and J. Srinivasan. 2011. *Beyond the Lean Revolution: Achieving Successful and Sustainable Enterprise Transformation*. New York, NY, USA: AMACOM Press.
- Nightingale, D., and R. Valerdi (eds). *Journal of Enterprise Transformation*. London, UK: Taylor & Francis. Available: <https://www.tandf.co.uk/journals/UJET>.
- Saenz, O.A. 2005. "Framework for Enterprise Systems Engineering," in *FIU Electronic Theses and Dissertations*. Miami, FL, USA: Florida International University. Accessed September 12, 2012. Available: <https://digitalcommons.fiu.edu/cgi/viewcontent.cgi?article=1055&context=etd>.

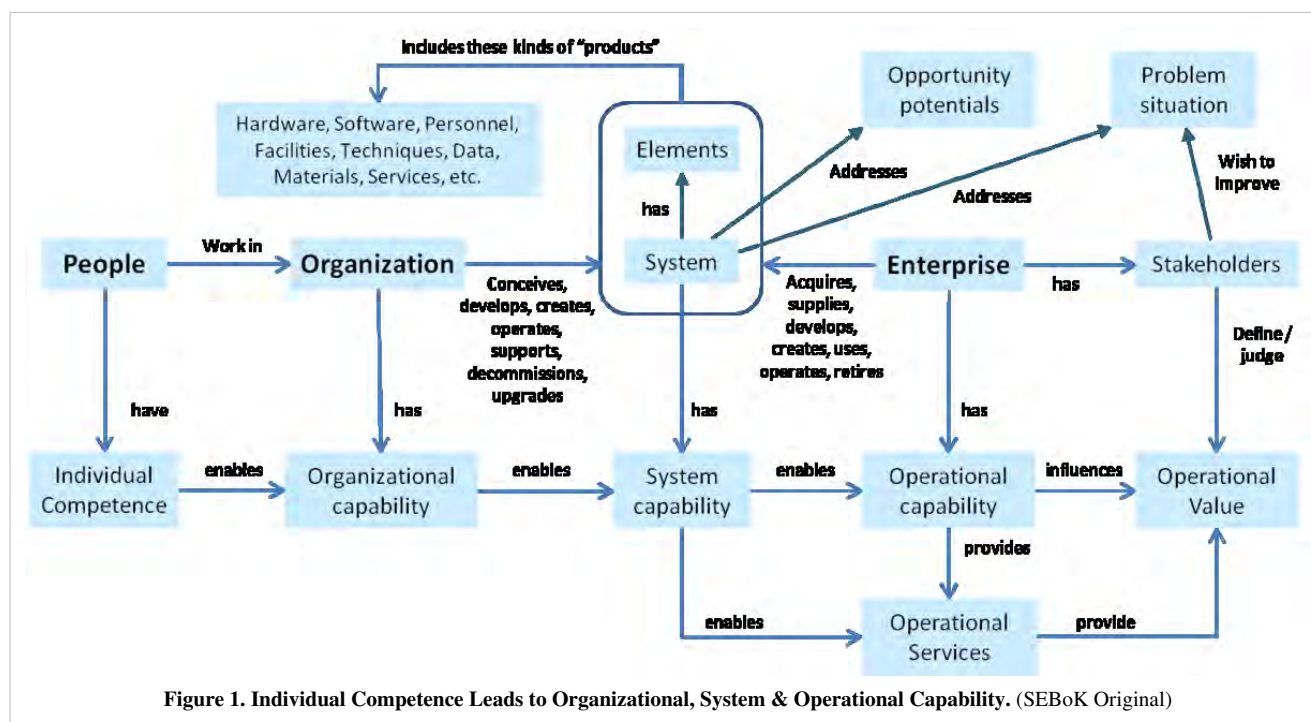
Enterprise Systems Engineering Background

- Lead Authors:
 - James Martin, Dick Fairley, and Bud Lawson
- Contributing Author:
 - Alan Faisandier

This article provides a common context for the succeeding topics in the knowledge area.

Capabilities in the Enterprise

The enterprise acquires or develops systems or individual elements of a system. The enterprise can also create, supply, use, and operate systems or system elements. Since there could possibly be several organizations involved in this enterprise venture, each organization could be responsible for particular systems or perhaps for certain kinds of elements. Each organization brings their own organizational capability with them and the unique combination of these organizations leads to the overall operational capability of the whole enterprise. These concepts are illustrated below.



Organizational structure is tightly tied to creating value for the enterprise's various stakeholders. Since the enterprise is made up of various elements including people, processes, technologies, and assets, the organizational structure of the people and the allocation of responsibilities for executing portions of the value stream is a "design decision" for the enterprise and hence is a key element of properly performing ESE. Organizational design is increasingly influenced by the portfolio of products and services and the degree of coupling between them. This organizational design will be based on organizational design patterns and their trade-offs, as discussed in the article on Systems Engineering Organizational Strategy. Browning (2009) discusses one approach for modeling and analysis of an organization.

Operational Capabilities & Operational Services

As you can see in this figure, operational capabilities provide operational services that are enabled by system capabilities. These system capabilities are inherent in the system that is conceived, developed, created and/or operated by an enterprise. ESE concentrates its efforts on maximizing operational value for various stakeholders, some of whom may be interested in the improvement of some problem situation.

ESE, however, addresses more than just solving problems; it also deals with the exploitation of opportunities for better ways to achieve the enterprise goals. This opportunity might involve lowering of operating costs, increasing market share, decreasing deployment risk, reducing time to market, and any number of other enterprise goals. The importance of addressing opportunity potentials should not be underestimated in the execution of ESE practices.

This article focuses on the *operational capabilities* of an enterprise and the contribution of these capabilities to *operational value* (as perceived by the stakeholders). Notice that the organization or enterprise can deal with either the system as a whole or with only one (or a few) of its elements. These elements are not necessarily hard items, like hardware and software, but can also include "soft" items, like people, processes, principles, policies, practices, organizations, doctrine, theories, beliefs, and so on.

Services vs. Products vs. Enterprises

A service system is a collection of items (or entities) that perform the operations, administration, management and provisioning (OAM&P) of resources that together provide the opportunities to co-create value by both the service provider and the service consumer.

A collection of services is not necessarily a service system. In fact, this collection of services is often merely a product system that is one of the resources being OAM&P'ed by the service system. A product system can be composed of hardware, software, personnel (see note 1), facilities, data, materials, techniques, and even services. Each of these product system elements can be "engineered."

Note 1. Even personnel are engineered in the sense that their roles and responsibilities are specified precisely and trade-offs are made about which functions are performed by these people versus by hardware or software. People are "produced" in the sense that untrained people are trained to perform their allocated system functions, unknowledgeable people are educated to find or create the information they need to do their assigned task, and uninformed people are taught how to get access to the data they need, and how to extract relevant information from that data.

It is important to understand the difference between the services "enabled" by a service system versus the services that are the elements of a service system entity. See the Service Systems Engineering article for more information about services and how they are engineered.

Likewise, a collection of services is not necessarily an enterprise system. An enterprise may be composed of service systems, along with product systems, as well as policies, procedures, properties, knowledge, financial capital, intellectual capital, and so on. An enterprise might even contain sub-enterprises. Enterprise SE must do the engineering not only across the enterprise itself, but may also get involved in the engineering of the service systems and products systems that the enterprise depends on in order to achieve its goals.

Enterprise Components

The above depictions of enterprise-related things do not show the components of an enterprise. The components of an enterprise when it is viewed as a “system” are different than the components of a product or service system (which is the focus of most literature on systems engineering). The figure below shows the typical kinds of components (shown here as “domains”) in an enterprise (Troux 2010) that could be utilized in achieving the desired enterprise *operational capability* as shown in Figure 1. It is this operational capability that drives ultimate value for the enterprise’s customers and other stakeholders. Further discussion on enterprise components is provided by Reese (2010) and Lawson (2010, chap. 8).

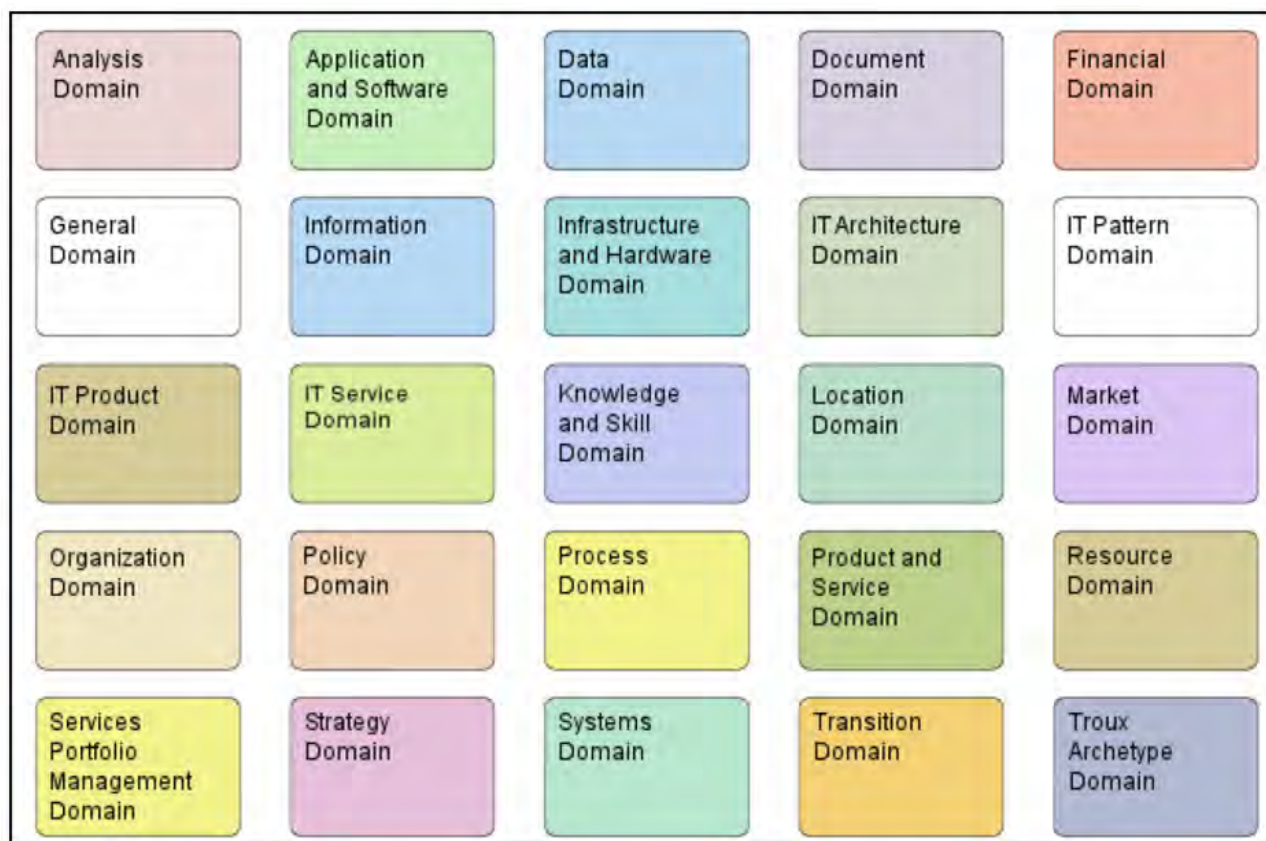


Figure 2. Categories of Enterprise Components (Trouw Technologies, 2010). Reprinted with permission of Copyright © 2010 Troux Technologies. All other rights are reserved by the copyright owner.

The application/software and infrastructure/hardware domains (shown above) are likely the most familiar to systems engineers. The application/software domain contains things like the deployed software itself plus applications, modules, servers, patches, functions, and messages. The infrastructure/hardware domain contains things like the hardware itself plus networks and different kinds of hardware like computing hardware, cabinets, and network devices. There might be different subtypes of computing hardware like computers, servers, desktops, laptops, and mainframes.

This particular “semantic model” had its origins in the area of information technology (IT) management but has been successfully expanded beyond the IT domain (Martin 2003 and 2005). You can see from this elaboration of these domains that an enterprise architecture “schema” can be quite extensive in the kinds of things it can model. The less technical domains would be things like policy, market, strategy, transition, financial, knowledge and skill, and analysis. In a typical enterprise architecture schema like this there could be over a hundred types of modeling objects grouped into these domains.

Various tools used in modeling the enterprise are described at https://www.enterprise-architecture.info/EA_Tools.htm (IEAD 2011). The TOGAF metamodel (<https://pubs.opengroup.org/architecture/togaf9-doc/arch/chap34>.

html) used in The Open Group Architecture Framework (TOGAF) is another useful depiction of the various modeling entities involved in modeling the enterprise (TOGAF 2009).

Scope of Enterprise SE

Computer and communications technologies make it easier to integrate activities across the enterprise, but this does not necessarily make the enterprise more effective and efficient. To enable this to happen, one needs to look at the whole enterprise as a system, rather than as a collection of functions connected solely by information systems and shared facilities.

Essential Challenges

Enterprises face strategic challenges that are essential to address in order to ensure that the enterprise will succeed (Rouse 2009):

- **Growth:** Increasing impact, perhaps in saturated/declining “markets”,
- **Value:** Enhancing relationships of processes to benefits and costs,
- **Focus:** Pursuing opportunities and avoiding diversions,
- **Change:** Competing creatively while maintaining continuity,
- **Future:** Investing in inherently unpredictable outcomes,
- **Knowledge:** Transforming information to insights to programs, and
- **Time:** Carefully allocating the organization’s scarcest resource.

To address these challenges, one recognizes that the central source of value in the enterprise is in its people. “Understanding and supporting the interests of an enterprise’s diverse stakeholders — and finding the ‘sweet spot’ among the many competing interests — is a central aspect of discerning the work of the enterprise as a system and creating mechanisms to enhance this work” (Rouse 2009).

Enterprise Transformation

Enterprises are constantly transforming, whether at the individual level (wherein individuals alter their work practices) or at the enterprise level (large-scale planned strategic changes) (Srinivasan 2010). These changes are a response on the part of the enterprise to evolving opportunities and emerging threats. It is not merely a matter of doing work better, but doing different work, which is often a more important result. Value is created through the execution of business processes. However, not all processes necessarily contribute to overall value (Rouse 2005, 138-150). It is important to focus on process and how they contribute to the overall value stream.

After gaining a good understanding of business processes, the next main concern is how best to deploy and manage the enterprise’s human, financial, and physical assets. The key challenge in transforming an enterprise is, in the midst of all this change, continuing to *satisfice* key stakeholders (see note 2).

Note 2. “Satisfice” means to decide on and pursue a course of action satisfying the minimum requirements to achieve a goal. For the enterprise as a whole, it is often impossible to completely satisfy all stakeholders given their competing and conflicting concerns and interests. Therefore, the concept of “satisficing” is a very important element in the execution of ESE practices. It has less stringent criteria than the concept of “satisfaction,” which is commonly used in product/service systems engineering.

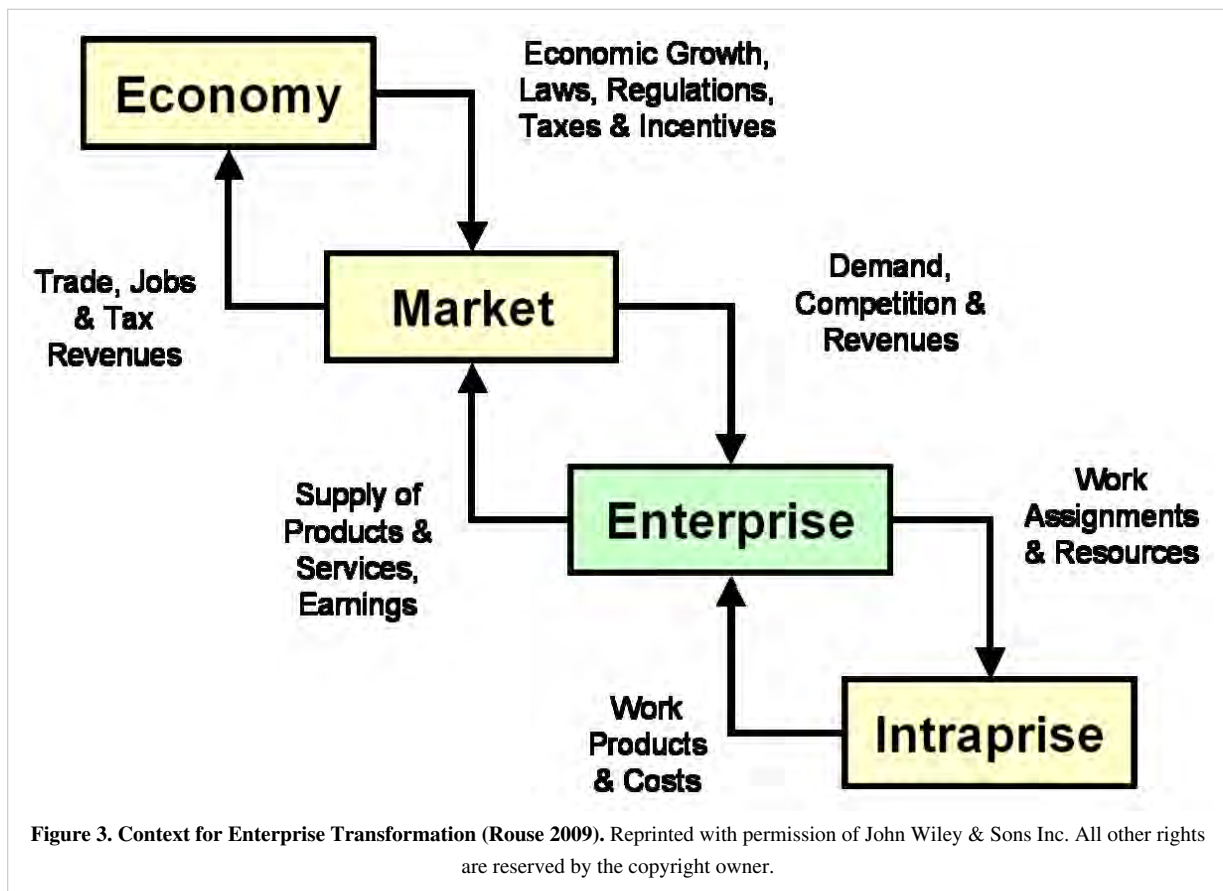
Systems engineers have to respond to an increased recognition of the ‘connectedness’ of products and systems, brought about by a number of trends, for example: the capability of (mainly digital) technology, working across multiple systems, to transform businesses and operational systems; the need to create systems in families to increase product diversity and reuse technology, in order to reduce development and operating costs; and the need to build systems which can be brought together flexibly in operations, even if such co-operation was not foreseen at the time of development.

There has also been an increase in collaborative systems development activities, often spanning national boundaries. This has proceeded alongside a growth in the development of what might be called *meta-systems*, that is systems comprising parts which would previously have been considered as complex in their own right a generation ago, now conceived of and developed as a whole, and thus requiring fresh approaches, of the adaption of old ones.

Tackling these issues requires an approach that transcends the technical and process domain. ESE needs to address integration at the organizational and value chain level.

Transformation Context

Enterprise transformation occurs in the external context of the economy and markets as shown in the figure below (Rouse 2009). The “market” for the enterprise can be thought of as the context in which the enterprise operates. Of course, in the public sector, the enterprise’s “market” is commonly known as its “constituency.”



The term “intraprise” is used here to denote the many systems internal to the enterprise. This includes “information systems such as... ERP [enterprise resource planning] systems, as well as social and cultural systems. More specifically, work assignments are pursued via work processes and yield work products, incurring costs” (Rouse 2009). The social and cultural aspects of an enterprise are addressed further in the article called Enabling Businesses and Enterprises.

Modeling the Enterprise

Models of the enterprise can serve as the basis for understanding the enterprise in its context of markets and economies. The figure below shows the various drivers (or inputs) of an enterprise and its potential outcomes (or outputs) (Rouse 2009). Enterprise architecture can be a key enabler for modeling and can serve as a basis for transformation (Vernadat 1996; Bernus, Laszlo, and Schmidt 2003; Nightingale and Rhodes 2004). Enterprise architecture can be used to provide a model to understand how the parts of the enterprise fit together (or do not)

(Giachetti 2010) (See also Representing Systems with Models). For a good review of the subject see Lillehagen and Krogstie (2008).

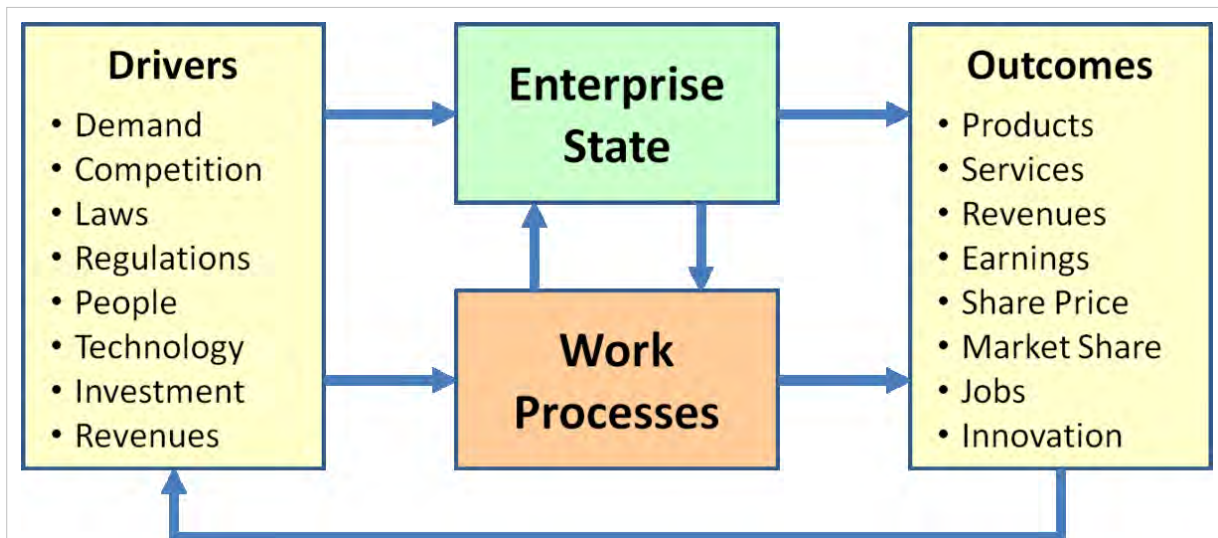


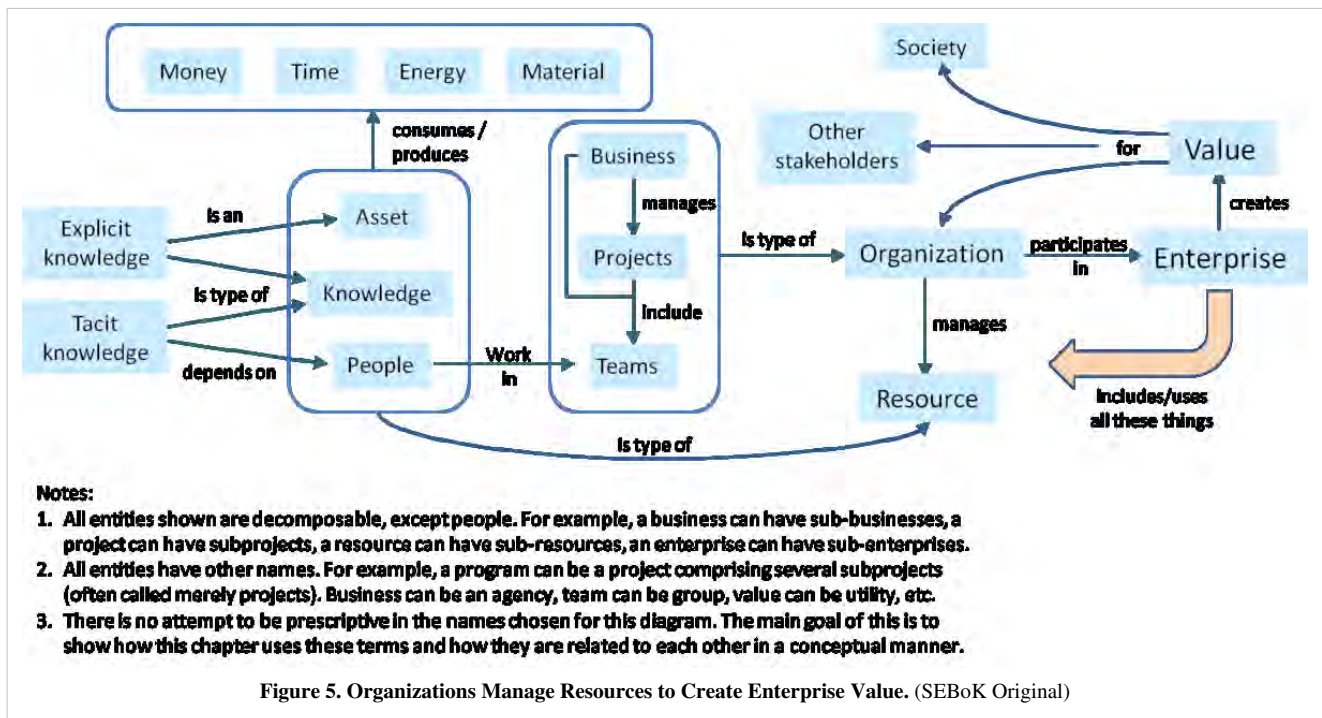
Figure 4. Drivers and Outcomes for the Enterprise (Rouse 2009). Reprinted with permission of John Wiley & Sons Inc. All other rights are reserved by the copyright owner.

In Pursuit of Value

Based on his theory of enterprise transformation, Rouse (2005, 279-295) has identified four alternative perspectives that tend to drive the need for transformation:

1. **Value Opportunities:** The lure of greater success via market and/or technology opportunities prompts transformation initiatives.
2. **Value Threats:** The danger of anticipated failure due to market and/or technology threats prompts transformation initiatives.
3. **Value Competition:** Other players' transformation initiatives prompt recognition that transformation is necessary to continued success.
4. **Value Crises:** Steadily declining market performance, cash flow problems, etc., prompt recognition that transformation is necessary for the enterprise to survive.

Work processes can be enhanced, streamlined, eliminated, and invented to help in the pursuit of enhanced value. These process changes should be aligned with enterprise strategy to maximize value produced by the enterprise (Hammer and Champy 1993). As shown below, there are many entities involved in helping the enterprise create value for society, participating organizations, and other stakeholders.



References

Works Cited

- Browning, T.R. 2009. "Using the Design Structure Matrix to Design Program Organizations," in *Handbook of Systems Engineering and Management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY: Wiley and Sons, Inc.
- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.
- Hammer, M., and J. Champy. 1993. *Reengineering the Corporation: A Manifesto for Business Revolution*. New York, NY: Harper Business, HarperCollins Publishers.
- IEAD. 2011. "Enterprise Architecture Tools." Institute for Enterprise Architecture Developments. Accessed September 12, 2012. Available: https://www.enterprise-architecture.info/EA_Tools.htm.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. Kings College, UK: College Publications.
- Lillehagen, F., and J. Krogstie. 2008. "Chapter 4: State of the Art of Enterprise Modelling," in *Active Knowledge Management of Enterprises*. New York, NY, USA: Springer.
- Martin, J.N. 2003. "On the Use of Knowledge Modeling Tools and Techniques to Characterize the NOAA Observing System Architecture." Presented at 13th Annual International Council on Systems Engineering (INCOSE) International Symposium, 2003, Arlington, VA, USA.
- Martin, J.N. 2005. "Using an Enterprise Architecture to Assess the Societal Benefits of Earth Science Research." Presented at 15th Annual International Council on Systems Engineering (INCOSE) International Symposium, 2005, Rochester, NY, USA.
- Miller, J., and S. Page. 2007. *Complex Adaptive Systems: An Introduction to Computational Models of Social Life*. Princeton, NJ, USA: Princeton University Press.
- Reese, R.J. 2010. *Troux Enterprise Architecture Solutions*. Birmingham, UK: Packt Publishing Ltd.

- Rouse, W.B. 2005. "Enterprise as Systems: Essential Challenges and Enterprise Transformation." *Systems Engineering, the Journal of the International Council on Systems Engineering (INCOSE)*. 8 (2): 138-150.
- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of Systems Engineering and Management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY: Wiley and Sons, Inc.
- Srinivasan, J. 2010. "Towards a Theory Sensitive Approach to Planning Enterprise Transformation." Presented at 5th European Institute for Advanced Studies in Management (EIASM) Workshop on Organizational Change and Development, September 23-24, 2010, Vienna, Austria.
- TOGAF 2009. "The Open Group Architecture Framework," version 9. The Open Architecture Group. Accessed September 2, 2011. Available: <https://www.opengroup.org/togaf>.
- Troux. 2010. *Metamodeling and modeling with Troux Semantics*, version 9. Austin, TX, USA: Troux Technologies.
- White, B.E. 2009. "Complex Adaptive Systems Engineering (CASE)." Presented at IEEE Systems Conference, March 23-26, 2009, Vancouver, Canada.

Primary References

- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.
- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of Systems Engineering and Management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY: Wiley and Sons, Inc.
- Srinivasan, J. 2010. "Towards a Theory Sensitive Approach to Planning Enterprise Transformation." Presented at 5th European Institute for Advanced Studies in Management (EIASM) Workshop on Organizational Change and Development, September 23-24, 2010, Vienna, Austria.
- White, B.E. 2009. "Complex Adaptive Systems Engineering (CASE)." Presented at IEEE Systems Conference, March 23-26, 2009, Vancouver, Canada.

Additional References

- McCarter, B.G., and B.E. White. 2009. "Emergence of SoS, sociocognitive aspects," in *Systems of systems engineering: Principles and applications*, edited by M. Jamshidi. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group. p. 71-105.
- Rouse, W.B. 2008. "Health Care as a Complex Adaptive System: Implications for design and management." *The Bridge, National Academy of Engineering*. 38 (1): 17-25.
- Sage, A.P. 2000. "Transdisciplinarity Perspectives in Systems Engineering and Management," in *Transdisciplinarity: Recreating Integrated Knowledge*, edited by M.A. Somerville and D. Rappaport. Oxford, UK: EOLSS Publishers. p. 158-169.
- von Bertalanffy, L. 1968. *General System Theory: Foundations, Development, Applications*, revised ed. New York, NY, USA: Braziller.
- Weinberg, G., and D. Weinberg. 1988. *General Principles of Systems Design*. New York, NY, USA: Dorset House Publishing Company.
- White, B.E. 2007. "On Interpreting Scale (or View) and Emergence in Complex Systems Engineering." Presented at 1st Annual IEEE Systems Conference, April 9-12, 2007, Honolulu, HI, USA.

The Enterprise as a System

- Lead Authors:
- James Martin, Bud Lawson, and Judith Dahmann

To enable more efficient and effective enterprise transformation, the enterprise needs to be looked at “as a system,” rather than as a collection of functions connected solely by information systems and shared facilities (Rouse 2005 and 2009; Lawson 2010). What distinguishes the design of enterprise systems from product systems is the inclusion of people as a component of the system, not merely as a user/operator of the system.

The term 'enterprise system' has taken on a narrow meaning of only the information system an organization uses. Research and project experience has taught us that to design a good enterprise system, we need to adopt a much broader understanding of enterprise systems. The greater view of enterprise systems is inclusive of the processes the system supports, the people who work in the system, and the information [and knowledge] content of the system. (Giachetti 2010)

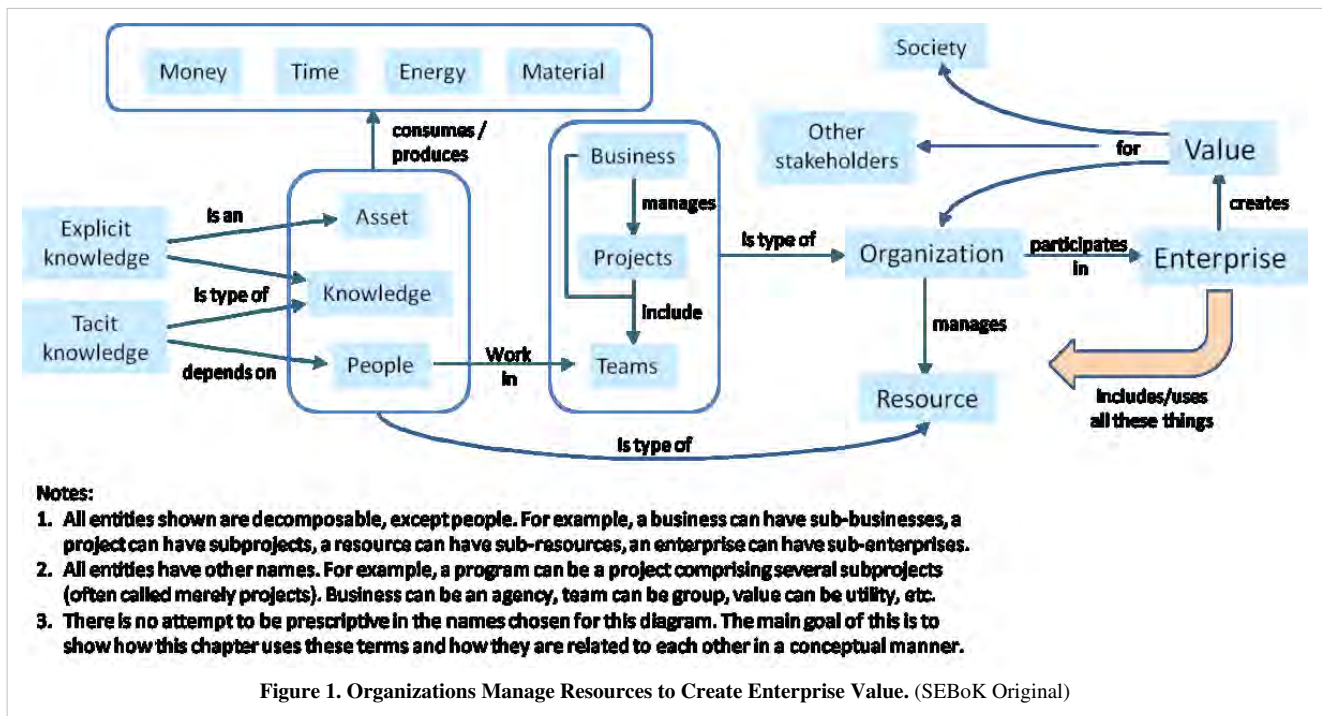
It is worth noting that the concept of "service" systems also includes people in the system. The thoughts above do not take this into account, primarily since their perspectives come mainly from a product system experience. The practice of service systems engineering is relatively new and is an emerging discipline. For more information on this, see the articles on Service Systems Engineering.

Creating Value

The primary purpose of an enterprise is to create value for society, other stakeholders, and for the organizations that participate in that enterprise. This is illustrated in Figure 1 that shows all the key elements that contribute to this value creation process. These elements in the enterprise can be treated as a "system" and the processes, methods, and tools ESE can be applied.

There are three types of organizations of interest: businesses, projects, and teams (see note 1). A typical business participates in multiple enterprises through its portfolio of projects. Large SE projects can be enterprises in their own right, with participation by many different businesses, and may be organized as a number of sub-projects.

Note 1. The use of the word "business" is not intended to mean only for-profit commercial ventures. As used here, it also includes government agencies and not-for-profit organizations, as well as commercial ventures. Business is the activity of providing goods and services involving financial, commercial, and industrial aspects.



Resource Optimization

A key choice for businesses that conduct SE is to what extent, if at all, they seek to optimize their use of resources (people, knowledge, assets) across teams, projects, and business units. Optimization of resources is not the goal in itself, but rather a means to achieve the goal of maximizing value for the enterprise and its stakeholders. At one extreme, in a product-oriented organization, projects may be responsible for hiring, training, and firing their own staff, as well as managing all assets required for their delivery of products or services. (The term "product-oriented organization" is not meant in the sense of product-oriented SE, but rather in the sense of this being one of the basic constructs available when formulating organizational strategy.)

At the other extreme, in a functional organization, the projects delegate almost all their work to functional groups. In between these two extremes is a matrix organization that is used to give functional specialists a "home" between project assignments. A full discussion of organizational approaches and situations along with their applicability in enabling SE for the organization is provided in the article called Systems Engineering Organizational Strategy.

The optimization debate can be handled as described in the book called "Enterprise Architecture as Strategy" (Ross, Weill, and Robertson 2006). In other words, an enterprise can choose (or not) to unify its operations and can choose (or not) to unify its information base. There are different strategies the enterprise might adopt to achieve and sustain value creation (and how ESE helps an enterprise to choose). This is further addressed in the section on Enterprise Architecture Formulation & Assessment in the article called Enterprise Capability Management.

Enabling Systems Engineering in the Organization

SE skills, techniques, and resources are relevant to many enterprise functions, and a well-founded SE capability can make a substantial contribution at the enterprise level, as well as at the project level. The article called Systems Engineering Organizational Strategy discusses enabling SE in the organization, while the article called Enabling Businesses and Enterprises focuses on the cross-organizational functions at the business and enterprise levels. The competence of individuals is discussed in the article called Enabling Individuals.

Kinds of Knowledge Used by the Enterprise

Knowledge is a key resource for ESE. There are generally two kinds of knowledge: explicit and tacit. Explicit knowledge can be written down or incorporated in computer codes. Much of the relevant knowledge, however, is “tacit knowledge” that only exists within the heads of people and in the context of relationships that people form with each other (e.g., team, project, and business level knowledge). The ability of an organization to create value is critically dependent on the people it employs, on what they know, how they work together, and how well they are organized and motivated to contribute to the organization’s purpose.

Projects, Programs, and Businesses

The term “program” is used in various ways in different domains. In some domains a team can be called a program (e.g., a customer support team is their customer relationship “program”). In others, an entire business is called a program (e.g., a wireless communications business unit program), and in others the whole enterprise is called a program (e.g., the Joint Strike Fighter program and the Apollo Space program). And in many cases, the terms project and program are used interchangeably with no discernible distinction in their meaning or scope. Typically, but not always, there are program managers who have profit and loss (P&L) responsibility and are the ultimate program decision makers. A program manager may have a portfolio of items (services, products, facilities, intellectual property, etc.) that are usually provided, implemented, or acquired through projects.

The Office of Government Commerce provides a useful distinction between programs and projects:

The ultimate goal of a Programme is to realise outcomes and benefits of strategic relevance. To achieve this a programme is designed as a temporary flexible organisation structure created to coordinate, direct and oversee the implementation of a set of related projects and activities in order to deliver outcomes and benefits related to the organisation’s strategic objectives...

A programme is likely to have a life that spans several years. A Project is usually of shorter duration (a few months perhaps) and will be focussed on the creation of a set of deliverables within agreed cost, time and quality parameters. (OGC 2010)

Enabling the Enterprise

ESE, by virtue of its inherent trans-disciplinarity (Sage 2000, 158-169) in dealing with problems that are large in scale and scope, can better enable the enterprise to become more effective and efficient. The complex nature of many enterprise problems and situations usually goes beyond the abilities of standard tools and techniques provided to business school graduates (See also Emergence and Complexity). ESE can augment the standard business management methods using the tools and methods from the SE discipline to more robustly analyze and evaluate the enterprise as a holistic system. A more general viewpoint, or “view,” for dealing with the enterprise consisting of scale, granularity, mindset, and time frame is provided by White (2007) and by McCarter and White (2009, 71-105).

ESE can provide the enablers to address the concerns of enterprise executives as shown in Table 1 (Rouse 2009). The methods for dealing with, and the special characteristics of, complex adaptive systems must be properly considered when adapting traditional systems engineering (TSE) practices for use at the enterprise level—many of which come out of the systems science and systems thinking domains (von Bertalanffy 1968; Weinberg and Weinberg 1988; Miller and Page 2007; Rouse 2008, 17-25). For an approach to complex adaptive systems (CAS) engineering, refer to White (2009, 1-16) and to McCarter and White (2009, 71-105).

Table 1. Executive Concerns and SE Enablers (Rouse 2009). Reprinted with permission of John Wiley & Sons Inc. All other rights are reserved by the copyright owner.

Executive Concerns	SE Enablers
Identifying ends, means, and scope and candidate changes	System complexity analysis to compare "as is" and "to be" enterprises
Evaluating changes in terms of process behaviors and performance	Organizational simulation of process flows and relationships
Assessing economics in terms of investments, operating costs, and returns	Economic modeling in terms of cash flows, volatility, and options
Defining the new enterprise in terms of processes and their integration	Enterprise architecting in terms of workflow, processes, and levels of maturity
Designing a strategy to change the culture for selected changes	Organizational and cultural change via leadership, vision, strategy, and incentives
Developing transformation action plans in terms of what, when, and who	Implementation planning in terms of tasks, schedule, people, and information

Enterprise Engineering

Another distinction is that "enterprise design does not occur at a single point in time like the design of most systems. Instead, enterprises evolve over time and are constantly changing, or are constantly *being designed*" (Giachetti 2010) [emphasis in original]. Giachetti calls this new discipline "enterprise engineering." We consider the enterprise engineering set of practices to be equivalent to what we call enterprise systems engineering (ESE) in this article.

The body of knowledge for enterprise engineering is evolving under such titles as enterprise engineering, business engineering, and enterprise architecture Many systems and software engineering principles are applicable to enterprise engineering, but enterprise engineering's unique complexities require additional principles.... Enterprise engineering's intent is to deliver a targeted level of enterprise performance in terms of shareholder value or customer satisfaction Enterprise engineering methods include modeling; simulation; total quality management; change management; and bottleneck, cost, workflow, and value-added analysis. (Joannou 2007)

Supersystem Constructs

System of Systems (SoS)

The phrase "system of systems" (SoS) is commonly used, but there is no widespread agreement on its exact meaning, nor on how it can be distinguished from a conventional system. A system is generally understood to be a collection of elements that interact in such a manner that it exhibits behavior that the elements themselves cannot exhibit. Each element (or component) of the system can be regarded as a system in its own right. Therefore, the phrase "system of systems" can technically be used for any system and, as such, would be a superfluous term. However, the meaning of this phrase has been examined in detail by (Maier 1998, 267-284), and his definition has been adopted by some people (AFSAB 2005). Maier provides this definition:

A SoS is an assemblage of components which individually may be regarded as systems, and which possess two additional properties:

- **Operational Independence of the Components:** *If the system-of-systems is disassembled into its component systems the component systems must be able to usefully operate independently. That is, the components fulfill customer-operator purposes on their own; and*
- **Managerial Independence of the Components:** *The component systems not only can operate independently, they do operate independently. The component systems are separately acquired and integrated but maintain a*

continuing operational existence independent of the system-of-systems. (Maier 1998, 267-284)

Maier goes on further saying that “the commonly cited characteristics of systems-of-systems (complexity of the component systems and geographic distribution) are not the appropriate taxonomic classifiers” (Maier 1998, 267-284). Four kinds of SoS have been defined (Dahmann, Lane, and Rebovich 2008).

For further details on SoS, see the *Systems Engineering Guide for SoS* developed by the US Department of Defense (DoD) (DUS(AT) 2008). Also, see the Systems of Systems (SoS) knowledge area.

Federation of Systems (FoS)

Different from the SoS concept, but related to it in several ways, is the concept called “federation of systems” (FoS). This concept might apply when there is a very limited amount of centralized control and authority (Sage and Cuppan 2001, 325-345; Sage and Rouse 2009). Each system in an FoS is very strongly in control of its own destiny, but “chooses” to participate in the FoS for its own good and the good of the “country,” so to speak. It is a coalition of the willing. An FoS is generally characterized by significant autonomy, heterogeneity, and geographic distribution or dispersion (Krygiel 1999). Krygiel defined a taxonomy of systems showing the relationships among conventional systems, SoSs, and FOSs.

This taxonomy has three dimensions: autonomy, heterogeneity, and dispersion. A FoS would have a larger value on each of these three dimensions than a non-federated SoS. An “Enterprise System,” as described above, could be considered to be an FoS if it rates highly on these three dimensions. However, it is possible for an enterprise to have components that are not highly autonomous, that are relatively homogeneous, and are geographically close together. Therefore, it would be incorrect to say that an enterprise is necessarily the same as an FoS.

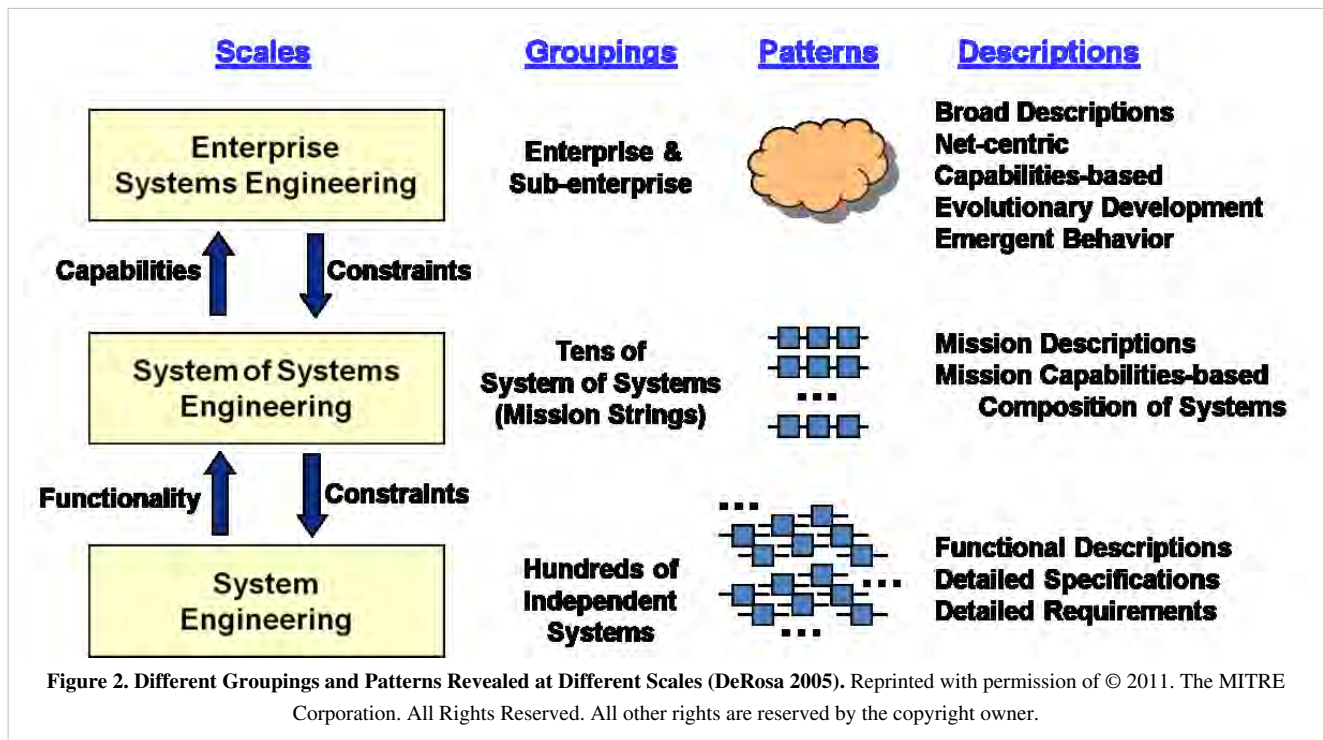
Dove points out that in order for a large enterprise to survive in the twenty-first century, it must be more agile and robust (Dove 1999 and 2001). Handy (1992, 59-67) describes a federalist approach called “New Federalism” which identifies the need for structuring of loosely coupled organizations to help them adapt to the rapid changes inherent in the Information Age. This leads to the need for virtual organizations where alliances can be quickly formed to handle the challenges of newly identified threats and a rapidly changing marketplace (Handy 1995, 2-8). Handy sets out to define a number of federalist political principles that could be applicable to an FoS. Handy’s principles have been tailored to the domain of systems engineering (SE) and management by Sage and Cuppan (2001, 325-345):

- Subsidiarity,
- Interdependence,
- Uniform and standardized way of doing business,
- Separation of powers,
- Dual citizenship, and
- Scales of SE.

Scales of SE

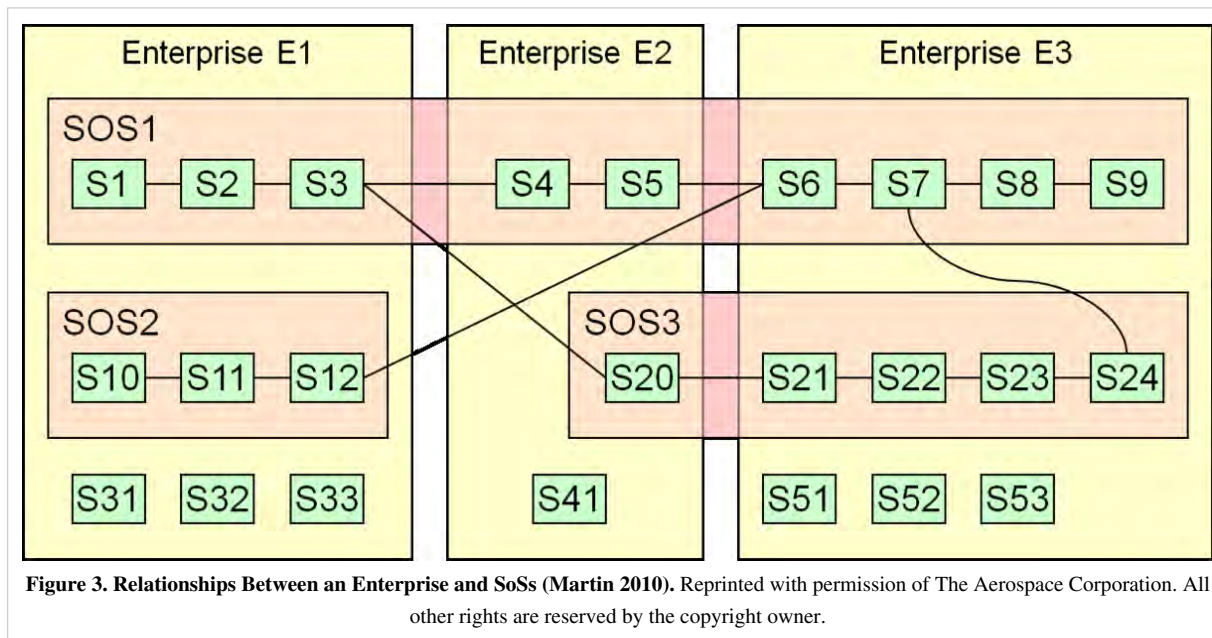
According to Maier’s definition, not every enterprise would be called a SoS since the systems within the enterprise do not usually meet the criteria of operational and managerial independence. In fact, one of the key purposes of an enterprise is to explicitly establish operational dependence between systems that the enterprise owns and/or operates in order to maximize the efficiency and effectiveness of the enterprise as a whole. Therefore, it is more proper to treat an enterprise system and an SoS as different types of things, with different properties and characteristics. This distinction is illustrated in the figure below, where three corresponding categories of SE are shown (DeRosa 2005; Swarz et al. 2006).

It is true that an enterprise can be treated as a system itself and is comprised of many systems within the enterprise, but this discussion will reserve the term SoS to those systems that meet the criteria of operational and managerial independence. This distinction was also used within the MITRE Corporation in their ESE Office (Rebovich and White 2011).



Relationships between Enterprise and SoS

An enterprise may require a particular operational capability that is brought into being by connecting together a chain of systems that together achieve that capability. Any one of these systems in the chain cannot by itself provide this capability. The desired capability is the emergent property of this chain of systems. This chain of systems is sometimes called an SoS. However, the enterprise that requires this capability rarely has direct control over all the systems necessary to provide this full capability. This situation is illustrated in the figure below (Martin 2010).



Enterprise E1 (in the example above) has full control over SoS2, but not full control over SoS1. TSE can be applied to the individual systems (S1, S2, ..., S53) shown within each enterprise, but needs to be augmented with additional activities to handle SoS and enterprise kinds of issues.

There is a general issue regarding dealing with enterprises in this situation: there are at least two enterprises related to any particular SoS. First, there is the enterprise of builders/developers comprising projects and programs, which have to be organized appropriately and adopt special types of architectural principles. Second, there is the enterprise of users (those who use the products and service provided by the first enterprise), which has to exercise its own sort of agility. How the first enterprise designs systems to allow the second to operate is the core issue.

References

Works Cited

- AFSAB. 2005. *Report on System-of-Systems Engineering for Air Force Capability Development*. Washington, DC: US Air Force Scientific Advisory Board (AFSAB), US Air Force. SAB-TR-05-04.
- Dahmann, J.S., J.A. Lane, and G. Rebovich. 2008. "Systems Engineering for Capabilities." *CROSSTALK: The Journal of Defense Software Engineering*. 21 (11): 4–9.
- DeRosa, J.K. 2005. "Enterprise Systems Engineering." Presented at Air Force Association, Industry Day, Day 1, August 4, 2005, Danvers, MA, USA.
- Dove, R. 2001. *Response Ability: The Language, Structure, and Culture of the Agile Organization*. New York, NY, USA: John Wiley & Sons.
- Dove, R. 1999. "Knowledge Management, Response Ability, and the Agile Enterprise," in Paradigm Shift International [database online]. Accessed September 6, 2011. Available: <https://www.parshift.com/docs/KmRaAeX.htm>.
- DUS(AT). 2008. *Systems Engineering Guide for Systems of Systems*, version 1.0. Washington, DC, USA: Deputy Under Secretary of Defense for Acquisition and Technology (DUS(AT)) / US Department of Defense (DoD). Accessed September 6, 2011. Available: <https://www.acq.osd.mil/se/docs/SE-Guide-for-SoS.pdf>.
- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.
- Handy, C. 1995. "Trust and the Virtual Organization." *Harvard Business Review*. 73 (3): 2-8.
- Handy, C. 1992. "Balancing Corporate Power: A New Federalist Paper." *Harvard Business Review*. 70 (6): 59-67.
- Joannou, P. 2007. "Enterprise, Systems, and Software—The Need for Integration." *Computer*. 40 (5): 103-105.
- Krygiel, A.J. 1999. *Behind the Wizard's Curtain: An Integration Environment for a System of Systems*. Arlington, VA, USA: C4ISR Cooperative Research Program (CCRP).
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. Kings College, UK: College Publications.
- Maier, M.W. 1998. "Architecting Principles for Systems-of-Systems." *Systems Engineering, the Journal of the International Council on Systems Engineering (INCOSE)*. 1 (4): 267-84.
- Martin, J.N. 2010. "An Enterprise Systems Engineering Framework." Presented at 20th Anniversary International Council on Systems Engineering (INCOSE) International Symposium, July 12-15, 2010, Chicago, IL, USA.
- McCarter, B.G., and B.E. White. 2009. "Emergence of SoS, sociocognitive aspects," in *Systems of systems engineering: Principles and applications*, edited by M. Jamshidi. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group. p. 71-105.
- OGC (Office of Government Commerce). 2010. *Guidelines for Managing Programmes: Understanding programmes and programme management*. London, UK: The Stationery Office.
- Ross, J.W., P. Weill, and D. Robertson. 2006. *Enterprise Architecture As Strategy: Creating a Foundation for Business Execution*. Boston, MA, USA: Harvard Business Review Press.

- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of systems engineering and management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY, USA: Wiley and Sons, Inc.
- Rouse, W.B. 2008. "Health Care as a Complex Adaptive System: Implications for design and management." *The Bridge, National Academy of Engineering*. 38 (1): 17-25.
- Rouse, W.B. 2005. "Enterprise as Systems: Essential Challenges and Enterprise Transformation." *Systems Engineering*. 8 (2): 138-50.
- Sage, A.P. 2000. "Transdisciplinarity Perspectives in Systems Engineering and Management." in *Transdisciplinarity: Recreating Integrated Knowledge*, edited by M.A. Somerville and D. Rappaport. Oxford, UK: EOLSS Publishers. p. 158-169.
- Sage, A., and C. Cuppan. 2001. "On the Systems Engineering and Management of Systems of Systems and Federations of Systems." *Information-Knowledge-Systems Management Journal*. 2 (4): 325-345.
- Sage, A.P., and W.B. Rouse (eds). 2009. *Handbook of System Engineering and Management*, 2nd ed. New York, NY, USA: John Wiley & Sons.
- Swarz, R.S., J.K. DeRosa, and G. Rebovich. 2006. "An Enterprise Systems Engineering Model." Proceedings of the INCOSE International Symposium, July 9-13, 2006, Orlando, FL, USA.
- von Bertalanffy, L. 1968. *General System Theory: Foundations, Development, Applications*, revised ed. New York, NY, USA: Braziller.
- Weinberg, G., and D. Weinberg. 1988. *General Principles of Systems Design*. New York, NY: Dorset House Publishing Company.
- White, B.E. 2007. "On Interpreting Scale (or View) and Emergence in Complex Systems Engineering." Presented at 1st Annual IEEE Systems Conference, 9-12 April, 2007, Honolulu, HI, USA.

Primary References

- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.
- Joannou, P. 2007. "Enterprise, Systems, and Software—The Need for Integration." *Computer*. 40 (5): 103-105.
- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of systems engineering and management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY, USA: Wiley and Sons, Inc.
- Rouse, W.B. 2005. "Enterprise as Systems: Essential Challenges and Enterprise Transformation." *Systems Engineering*. 8 (2): 138-50.

Additional References

- Arnold, S., and H. Lawson. 2004. "Viewing Systems From a Business Management Perspective." *Systems Engineering*. 7 (3): 229.
- Beimans, F.P.M., M.M. Lankhorst, W.B. Teeuw, and R.G. van de Wetering. 2001. "Dealing with the Complexity of Business Systems Architecting." *Systems Engineering*. 4 (2): 118-33.
- Nightingale, D., and D. Rhodes. 2004. "Enterprise systems architecting: Emerging art and science within engineering systems." Presented at Engineering Systems Symposium, Massachusetts Institute of Technology (MIT), 29-31 March, 2004, Boston, MA, USA.
- Rebovich, G. 2006. "Systems Thinking for the Enterprise: New & Emerging Perspectives." Presented at IEEE/SMC International Conference on System of Systems Engineering, April 2006, Los Angeles, CA, USA.
- Rechtin, E. 1999. *Systems Architecting of Organizations: Why Eagles can't Swim*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.

Ring, J. 2004. "Intelligent Enterprises." *INCOSE INSIGHT*. 6 (2).

Ring, J. 2004. "Seeing an Enterprise as a System." *INCOSE INSIGHT*. 6(2).

Valerdi, R., D. Nightingale, and C. Blackburn. 2009. "Enterprises as Systems: Context, Boundaries, and Practical Implications." *Information-Knowledge-Systems Management Journal*. 7 (4): 377-399.

Related Business Activities

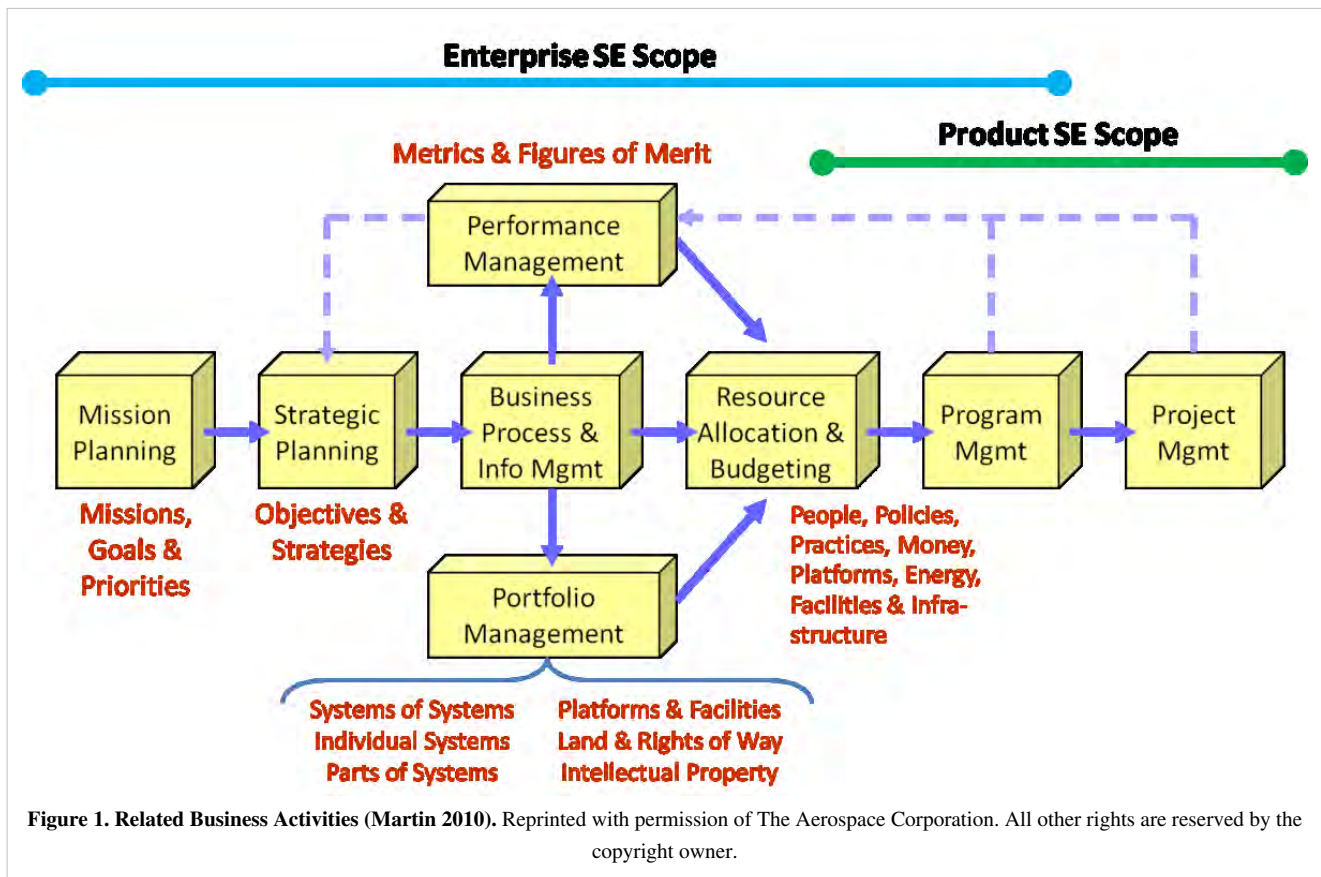
- Lead Authors:
- James Martin, Dick Fairley, and Bud Lawson

The following business management activities can be supported by enterprise systems engineering (ESE) activities:

- mission and strategic planning,
- business processes and information Management,
- performance management,
- portfolio management,
- resource allocation and budgeting, and
- program and project management.

Introduction

The figure below shows how these business activities relate to each other as well as the relative scope of ESE and product systems engineering (PSE) (Martin 2010 and 2011). PSE is mainly involved at the project level and collaborates with project management activities, and gets somewhat involved in program management and the resource allocation and budgeting activities. On the other hand, ESE is heavily involved in the higher level activities from the program management level and up. Both ESE and PSE have key roles to play in the enterprise.

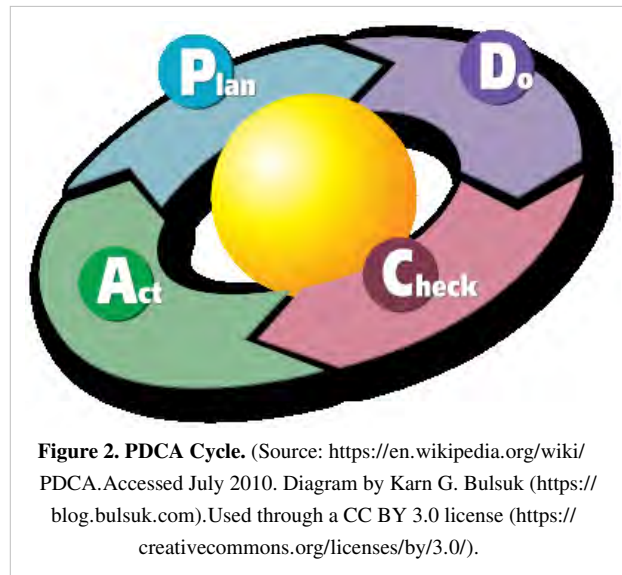


Shown in this manner, these business activities can be considered to be separate processes with a clear precedence in terms of which process drives other processes. TSE uses “requirements” to specify the essential features and functions of a system. An enterprise, on the other hand, typically uses goals and objectives to specify the fundamental characteristics of desired enterprise operational capabilities. The enterprise objectives and strategies are used in portfolio management to discriminate between options and to select the appropriate balanced portfolio of systems and other enterprise resources.

The first three activities listed above are covered in Enabling Businesses and Enterprises. The other business management activities are described in more detail below in regards to how they relate to ESE.

Business Management Cycles

PDCA stands for “plan-do-check-act” and is a commonly used iterative management process as seen in the figure below. It is also known as the Deming circle or the Shewhart cycle after its two key proponents (Deming 1986; Shewhart 1939). ESE should use the PDCA cycle as one of its fundamental tenets. For example, after ESE develops the enterprise transformation plan, execution of the planned improvements are monitored (i.e., “checked” in the PDCA cycle) to ensure they achieve the targeted performance levels. If not, then action needs to be taken (i.e., “act” in the PDCA cycle) to correct the situation and re-planning may be required. ESE can also use the PDCA cycle in its support of the ‘business as usual’ efforts, such as the annual budgeting and business development planning activities.



It is also worth mentioning the utility of using Boyd's OODA loop (observe, orient, decide, and act) to augment PDCA. This could be accomplished by first using the OODA loop (https://en.wikipedia.org/wiki/OODA_loop), which is continuous in situation awareness, and then followed by using the PDCA approach, which is discrete, having goals, resources, usually time limits, etc. (Lawson 2010).

Portfolio Management

Program and project managers direct their activities as they relate to the systems under their control. Enterprise management, on the other hand, is involved in directing the portfolio of items that are necessary to achieving the enterprise goals and objectives. This can be accomplished by using portfolio management:

Project Portfolio Management (PPM) is the centralized management of processes, methods, and technologies used by project managers and project management offices (PMOs) to analyze and collectively manage a group of current or proposed projects based on numerous key characteristics. The objectives of PPM are to determine the optimal resource mix for delivery and to schedule activities to best achieve an organization's operational and financial goals—while honoring constraints imposed by customers, strategic objectives, or external real-world factors. (https://en.wikipedia.org/wiki/Project_portfolio_management)

The enterprise may not actually own these portfolio items. They could rent or lease these items, or they could have permission to use them through licensing or assignment. The enterprise may only need part of a system (e.g., one bank of switching circuits in a system) or may need an entire system of systems (SoS) (e.g., switching systems, distribution systems, billing systems, provisioning systems, etc.). Notice that the portfolio items are not just those items related to the systems that systems engineering (SE) deals with. These could also include platforms (like ships and oil drilling derricks), facilities (like warehouses and airports), land and rights of way (like railroad property easements and municipal covenants), and intellectual property (like patents and trademarks).

The investment community has been using portfolio management for a long time to manage a set of investments to maximize return for a given level of acceptable risk. These techniques have also been applied to a portfolio of “projects” within the enterprise (Kaplan 2009). However, it should be noted that an enterprise is not merely a portfolio of projects. The enterprise portfolio consists of whatever systems, organizations, facilities, intellectual property, and other resources that are needed to help the enterprise achieve its goals and objectives.

Portfolio management in the context of ESE is well addressed in the following article: https://www.mitre.org/work/systems_engineering/guide/enterprise_engineering/enterprise_planning_management/portfolio_management.html (MITRE 2010).

Resource Allocation and Budgeting

The resource allocation and budgeting (RA&B) activity is driven by the portfolio management definition of the optimal set of portfolio elements. Capability gaps are mapped to the elements of the portfolio, and resources are assigned to programs (or other organizational elements) based on the criticality of these gaps. Resources come in the form of people and facilities, policies and practices, money and energy, and platforms and infrastructure. Allocation of resources could also involve the distribution or assignment of corporate assets, like communication bandwidth, manufacturing floor space, computing power, intellectual property licenses, and so on. Resource allocation and budgeting is typically done on an annual basis, but more agile enterprises will make this a more continuous process. Some of the resource allocation decisions deal with base operational organizations that are not project related.

It is sometimes the case that RA&B is part of portfolio management (PfM). But as can be seen in Figure 1, it is sometimes useful and practical to separate these two activities. PfM usually recommends changes to the enterprise portfolio, but RA&B takes these PfM considerations into mind along with inputs from the business process and information management and the performance management activities. Furthermore, PfM is usually an annual or biannual activity whereas RA&B is often done more frequently. RA&B may need to execute *ad hoc* when perturbations happen, such as funding cuts, schedule slips, performance targets missed, strategic goals changed, and so on.

Program and Project Management

Within the enterprise, TSE is typically applied inside a project to engineer a single system (or perhaps a small number of related systems). If there is a SoS or a large, complex individual system to be engineered, then this might be handled at the program level, but is sometimes handled at the project level, depending on the size and complexity of the system-of-interest (See also Emergence and Complexity).

There are commonly three basic types of projects in an enterprise. A development project takes a conceptual notion of a system and turns this into a realizable design. A production project takes the realizable design for a system and turns this into physical copies (or instantiations). An operations "project" directly operates each system or supports the operation by others. (Base operations are sometimes called "line organizations" and are not typically called projects per se, but should nonetheless be considered as key elements to be considered when adjusting the enterprise portfolio.) The operations project can also be involved in maintaining the system or supporting maintenance by others. A program can have all three types of projects active simultaneously for the same system, as in this example:

- Project A is developing System X version 3.
- Project B is operating and maintaining System X version 2.
- Project C is maintaining System X version 1 in a warehouse as a backup in case of emergencies.

Project management uses TSE as a tool to ensure a well-structured project and to help identify and mitigate cost, schedule, and technical risks involved with system development and implementation. The project level is where the TSE process is most often employed (Martin 1997; ISO/IEC/IEEE 2015; Wasson 2006; INCOSE 2010; Blanchard and Fabrycky 2010).

The Office of Government Commerce provides a useful distinction between programs and projects:

The ultimate goal of a Programme is to realise outcomes and benefits of strategic relevance. To achieve this, a programme is designed as a temporary flexible organisation structure created to coordinate, direct and oversee the implementation of a set of related projects and activities in order to deliver outcomes and benefits related to the organisation's strategic objectives...

A programme is likely to have a life that spans several years. A Project is usually of shorter duration (a few months perhaps) and will be focussed on the creation of a set of deliverables within agreed cost, time and quality parameters. (OGC 2010)

Enterprise Governance

ESE is also concerned with the way in which organizations and embedded management and technical functions work together to achieve success at the enterprise level. Governance frameworks provide the essential additional structure and controls needed to both ‘steer a steady ship’ (during business as usual) and to ‘plot a course to a new place’ (during business transformation).

Such frameworks can be designed by recognizing that there are enduring management concerns that need to be addressed and by applying the principle of economy. For example, a particular concern for most organizations is linking the control of projects to business drivers and objectives. This leads to a requirement for a governance body to both approve the initiation of projects, and to regularly review their progress, continuing relevance, and if necessary, mutual coherence in the light of developments inside and outside the enterprise.

This might be achieved by delegating some or all of the roles; depending on circumstances, the enterprise might be driven towards top-down or a more collective, peer-to-peer approach—or even a combination of the two for different functions. Governance bodies and management roles can be engineered in this way against a common set of management concerns. Governance may also include the maintenance of common technical standards and their promulgation and use throughout relevant projects. See Bryant (2012) for more information on governance.

Multi-Level Enterprises

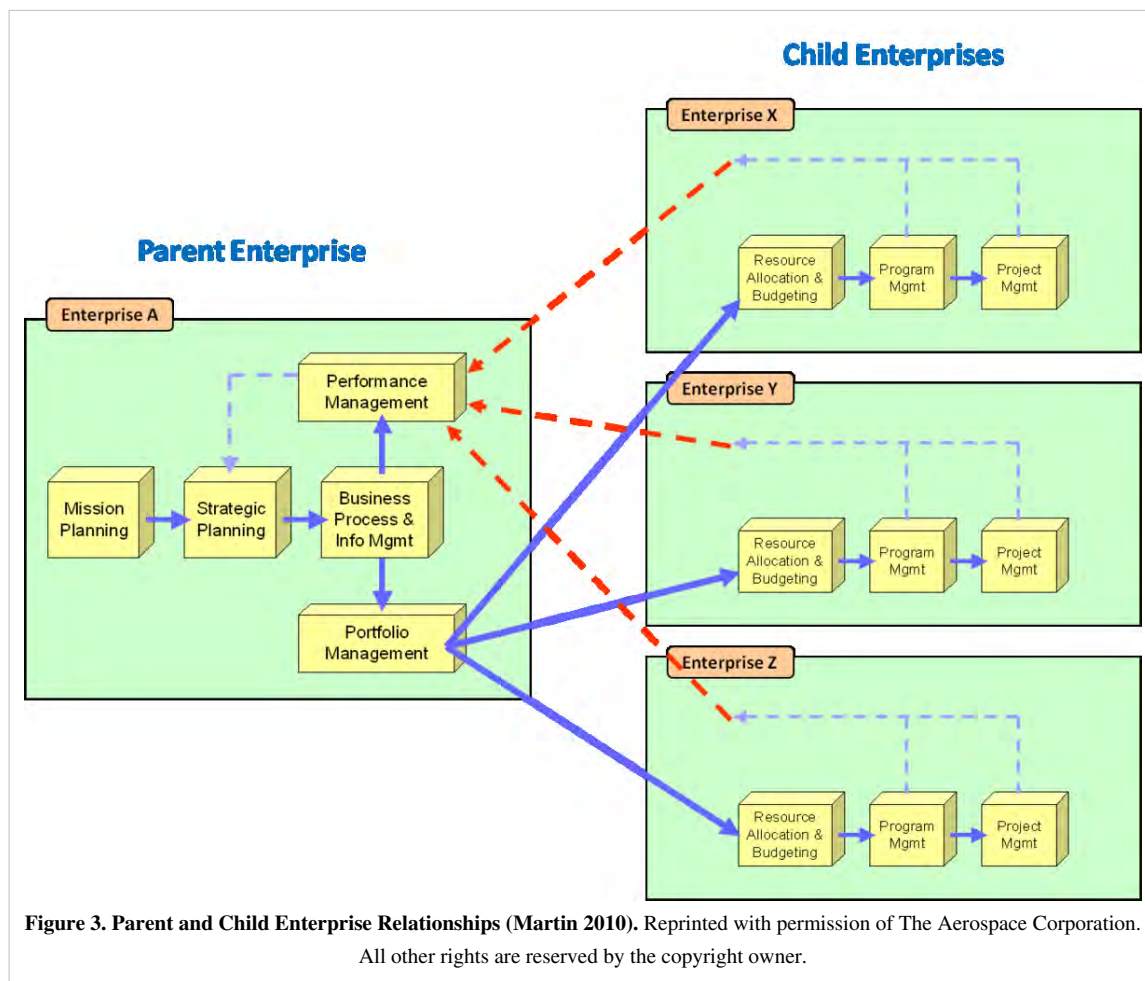
An enterprise does not always have full control over the ESE processes. In some cases, an enterprise may have no direct control over the resources necessary to make programs and projects successful. For example, the Internet Engineering Task Force (IETF) is responsible for the “smooth operation of the Internet,” yet it controls none of the requisite resources.

The Internet Engineering Task Force (IETF) is a large open international community of network designers, operators, vendors, and researchers concerned with the evolution of the Internet architecture and the smooth operation of the Internet. ... The actual technical work of the IETF is done in its working groups, which are organized by topic into several areas (e.g., routing, transport, security, etc.). Much of the work is handled via mailing lists. The IETF holds meetings three times per year. (IETF 2010a)

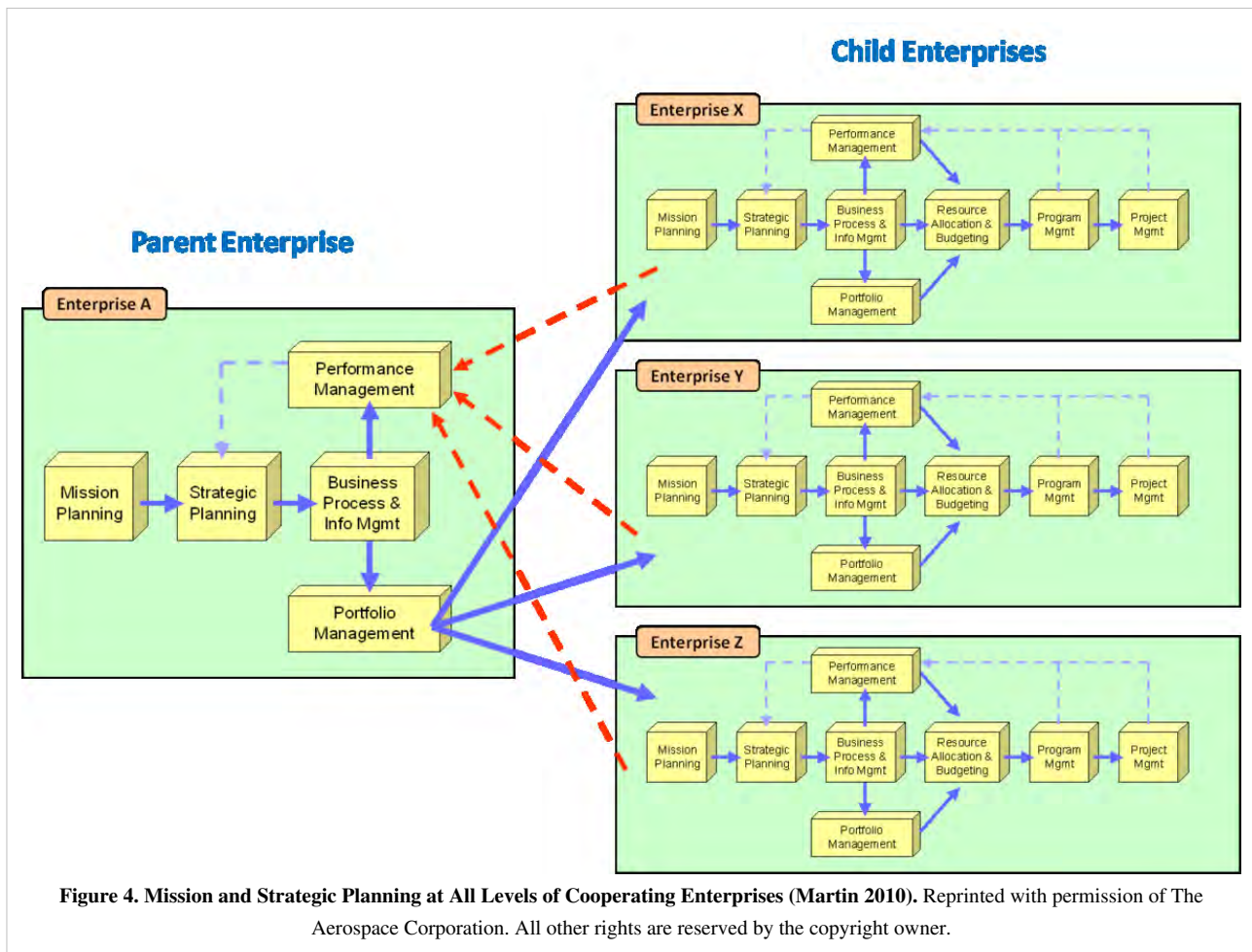
The IETF has “influence” over these resources even though it does not have direct control: “The IETF is unusual in that it exists as a collection of happenings, but is not a corporation and has no board of directors, no members, and no dues” (IETF 2010b).

The ESE processes might be allocated between a “parent” enterprise and “children” enterprises, as shown in the figure below (Martin 2010). The parent enterprise, in this case, has no resources. These resources are owned by the subordinate child enterprises. Therefore, the parent enterprise does not implement the processes of resource allocation and budgeting, program management, and project management.

The parent enterprise may have an explicit contract with the subordinate enterprises, or, as in some cases, there is merely a “working relationship” without the benefit of legal obligations. The parent enterprise will expect performance feedback from the lower level to ensure that it can meet its own objectives. Where the feedback indicates a deviation from the plan, the objectives can be adjusted or the portfolio is modified to compensate.



Enterprises X, Y, and Z in the situation shown above will cooperate with each other to the extent that they honor the direction and guidance from the parent enterprise. These enterprises may not even be aware of each other, and, in this case, would be unwittingly cooperating with each other. The situation becomes more complex if each enterprise has its own set of strategic goals and objectives as shown in the figure below.



These separate, sub-enterprise objectives will sometimes conflict with the objectives of the parent enterprise. Furthermore, each subordinate enterprise has its own strategic objectives that might conflict with those of its siblings. The situation shown here is not uncommon, and illustrates an enterprise of enterprises, so to speak. This highlights the need for the application of SE at the enterprise level to handle the complex interactions and understand the overall behavior of the enterprise as a whole. TSE practices can be used, to a certain extent, but these need to be expanded to incorporate additional tools and techniques.

References

Works Cited

- Bryant, P. 2012. "Modelling Governance within Business Architecture using Topic Mapping." Presented at 22nd Annual International Council on Systems Engineering (INCOSE) International Symposium, July 9-12, 2012, Rome, Italy.
- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Deming, W.E. 1986. *Out of the Crisis*. Cambridge, MA, USA: MIT Press, MIT Center for Advance Engineering Study.
- IETF. 2010a. "Overview of the IETF," in Internet Engineering Task Force, Internet Society (ISOC) [database online]. Accessed September 6, 2011. Available: <https://www.ietf.org/overview.html>.
- IETF. 2010b. "The Tao of IETF: A Novice's Guide to the Internet Engineering Task Force (draft-hoffman-cao4677bix-10)," in Internet Engineering Task Force, Internet Society (ISOC) [database online].

Accessed September 6, 2011. Available: <https://www.ietf.org/tao.html#intro>.

INCOSE. 2012. *Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2015.

Kaplan, J. 2009. *Strategic IT portfolio management: Governing enterprise transformation*. Waltham, Massachusetts, USA: Pittiglio, Rabin, Todd & McGrath, Inc. (PRTM).

Lawson, H. 2010. *A Journey Through the Systems Landscape*. Kings College, UK: College Publications.

Martin, J.N. 2011. "Transforming the Enterprise Using a Systems Approach." Presented at 21st Anniversary International Council on Systems Engineering (INCOSE) International Symposium, June 20-23, 2011, Denver, CO, USA.

Martin, J.N. 2010. "An Enterprise Systems Engineering Framework." Presented at 20th Anniversary International Council on Systems Engineering (INCOSE) International Symposium, July 12-15, 2010, Chicago, IL, USA.

Martin, J.N. 1997. *Systems Engineering Guidebook: A Process for Developing Systems and Products*, 1st ed. Boca Raton, FL, USA: CRC Press.

MITRE. 2012. "Enterprise Engineering," in *Systems Engineering Guide*. Bedford, MA, USA: MITRE Corporation. Accessed July 8, 2012. Available: https://www.mitre.org/work/systems_engineering/guide/enterprise_engineering/.

OGC (Office of Government Commerce). 2010. *Guidelines for Managing Programmes: Understanding programmes and programme management*. London, UK: The Stationery Office.

Shewhart, W.A. 1939. *Statistical Method from the Viewpoint of Quality Control*. New York, NY, USA: Dover Publications.

Wasson, C.S. 2006. *System Analysis, Design and Development*. Hoboken, NJ, USA: John Wiley and Sons Ltd.

Primary References

Martin, J.N. 2011. "Transforming the Enterprise Using a Systems Approach." Presented at 21st Anniversary International Council on Systems Engineering (INCOSE) International Symposium, June 20-23, 2011, Denver, CO, USA.

Martin, J.N. 2010. "An Enterprise Systems Engineering Framework." Presented at 20th Anniversary International Council on Systems Engineering (INCOSE) International Symposium, July 12-15, 2010, Chicago, IL, USA.

Additional References

Arnold, S., and H. Lawson. 2004. "Viewing Systems from a Business Management Perspective." *Systems Engineering*. 7 (3): 229.

Beimans, F.P.M., M.M. Lankhorst, W.B. Teeuw, and R.G. van de Wetering. 2001. "Dealing with the Complexity of Business Systems Architecting." *Systems Engineering*. 4 (2): 118-133.

Drucker, P.F. 1994. "The Theory of Business." *Harvard Business Review*. 72 (5): 95-104.

Haeckel, S.H. 2003. "Leading on demand businesses—Executives as architects." *IBM Systems Journal*. 42 (3): 405-13.

Kaplan, R., and D. Norton. 1996. *The balanced scorecard: Translating strategy into action*. Cambridge, MA, USA: Harvard Business School Press.

Lissack, M.R. 2000. "Complexity Metaphors and the Management of a Knowledge Based Enterprise: An Exploration of Discovery." PhD Dissertation in Business Administration. Henley-on-Thames, UK: Henley Management College, University of Reading.

Rechtin, E. 1999. *Systems Architecting of Organizations: Why Eagles Can't Swim*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.

Enterprise Systems Engineering Key Concepts

- Lead Authors:
- James Martin, Bud Lawson, and Alan Faisandier

The purpose of traditional systems engineering (TSE) is to bring together a diversity of discipline experts to address a wide range of problems inherent in the development of a large, complex "single" system (Blanchard and Fabrycky 2010; Hall 1989; Sage and Rouse 2009). Enterprise systems engineering (ESE) expands beyond this traditional basis to "consider the full range of SE services increasingly needed in a modern organization where information-intensive systems are becoming central elements of the organization's business strategy" (Carlock and Fenton 2001, 242-261). The traditional role of systems engineering (SE) is heavily involved in system acquisition and implementation, especially in the context of government acquisition of very large, complex military and civil systems (e.g., F22 fighter jet and air traffic control systems).

ESE encompasses this traditional role in system acquisition, but also incorporates enterprise strategic planning and enterprise investment analysis (along with others as described below). These two additional roles for SE at the enterprise level are "shared with the organization's senior line management, and tend to be more entrepreneurial, business-driven, and economic in nature in comparison to the more technical nature of classical systems engineering" (Carlock and Fenton 2001, 242-261).

Closing the Gap

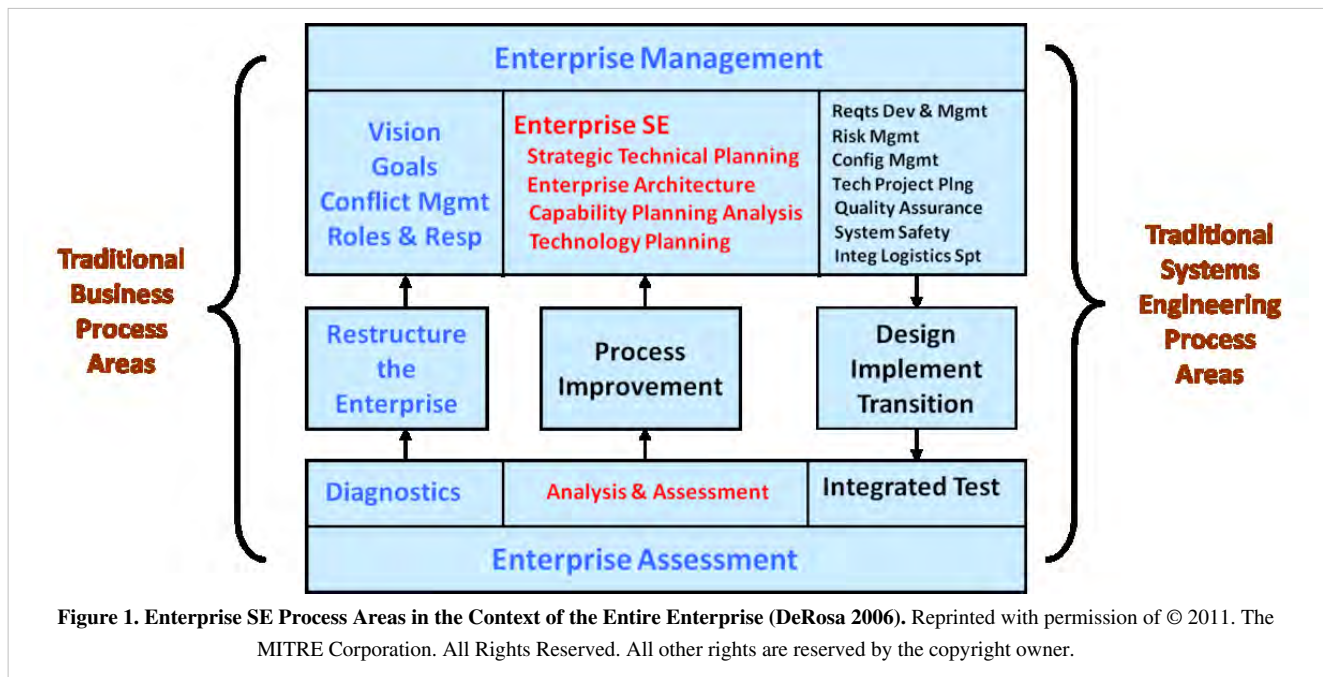
ESE practices have undergone significant development recently.

Today the watchword is enterprise systems engineering, reflecting a growing recognition that an 'enterprise' may comprise many organizations from different parts of government, from the private and public sectors, and, in some cases, from other nations. (MITRE 2004)

Rebovich (2006) says there are "new and emerging modes of thought that are increasingly being recognized as essential to successful systems engineering in enterprises." For example, in addition to the TSE process areas, MITRE has included the following process areas in their ESE process (DeRosa 2005) to close the gap between ESE and PSE:

- strategic technical planning,
- enterprise architecture,
- capabilities-based planning analysis,
- technology planning, and
- enterprise analysis and assessment.

These ESE processes are shown in the context of the entire enterprise in the figure below (DeRosa 2006). The ESE processes are shown in the middle with business processes on the left and TSE processes on the right. These business processes are described in the article called Related Business Activities. The TSE processes are well documented in many sources, especially in the ISO/IEC/IEEE 15288 standard (2015).



SE is viewed by many organizations and depicted in many process definitions as bounded by the beginning and end of a system development project. In MITRE, this restricted definition was referred to as TSE. Many have taken a wider view seeking to apply SE to the “whole system” and “whole life cycle.” For example, Hitchins (1993) sets out a holistic, whole-life, wider system view of SE centered on operational purpose. Elliott and Deasley (2007) discuss the differences between development phase SE and in-service SE.

In contrast to TSE, the ESE discipline is more like a “regimen” (Kuras and White 2005) that is responsible for identifying “outcome spaces,” shaping the development environment, coupling development to operations, and rewarding results rather than perceived promises (DeRosa 2005). ESE must continually characterize the operational environmental and the results of enterprise or SoS interventions to stimulate further actions within and among various systems in the enterprise portfolio. Outcome spaces are characterized by a set of desired capabilities that help meet enterprise objectives, as opposed to definitive “user requirements” based on near-term needs. Enterprise capabilities must be robust enough to handle unknown threats and situations in the future. A detailed description of previous MITRE views on ESE can be found in a work by Rebovich and White (2011).

Role of Requirements in ESE

TSE typically translates user needs into system requirements that drive the design of the system elements. The system requirements must be “frozen” long enough for the system components to be designed, developed, tested, built, and delivered to the end users (which can sometimes take years, and in the case of very large, complicated systems like spacecraft and fighter jets, more than a decade).

ESE, on the other hand, must account for the fact that the enterprise must be driven not by requirements (that rarely can even be defined, let alone made stable), but instead by continually changing organizational visions, goals, governance priorities, evolving technologies, and user expectations. An enterprise consists of people, processes, and technology where the people act as “agents” of the enterprise:

Ackoff has characterized an enterprise as a 'purposeful system' composed of agents who choose both their goals and the means for accomplishing those goals. The variety of people, organizations, and their strategies is what creates the inherent complexity and non-determinism in an enterprise. ESE must account for the concerns, interests and objectives of these agents. (Swarz, DeRosa, and Rebovich 2006)
(See also Emergence and Complexity)

Whereas TSE focuses on output-based methodologies (e.g., functional analysis and object-oriented analysis), ESE is obligated to emphasize outcomes (e.g., business analysis and mission needs analysis), especially those related to the enterprise goals and key mission needs.

Enterprise Entities and Relationships

An enterprise “system” has different entities and relationships than you might find in a product/service system (see note 1). These can be usefully grouped into two categories: asset items and conceptual items. An example of an asset is hardware and software. Examples of conceptual items are things like analysis, financial elements, markets, policies, process, and strategy.

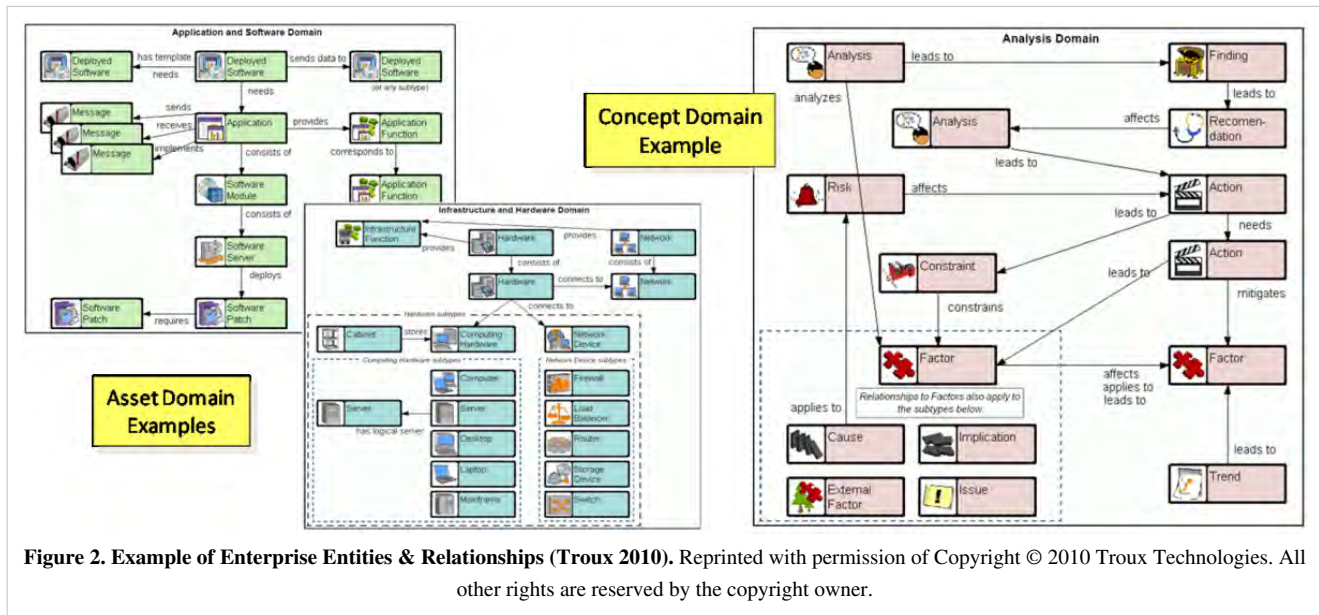
Note 1. An “enterprise system” should not be confused with the enterprise “perceived as a system.” An enterprise system is a product (or service) system used across the enterprise, such as payroll, financial accounting, or enterprise resource planning applications, and consolidated data center, data warehouse, and other such facilities and equipment used across one or more organizations.

Products and services are sometimes treated as “assets” as shown in the figure below (Troux 2010). This categorization of enterprise items comes from the semantic model (i.e., metamodel) used in the Troux Architect modeling tool for characterization and analysis of an enterprise architecture. Other enterprise entities of interest are things like information, knowledge, skills, finances, policies, process, strategy, markets, and resources, but these are categorized as “concept” items (in this particular schema). Further details on how to use this metamodel’s entities and relationships are provided by Reese (2010).

Table 1. Asset Domain and Concept Domain Categories for Enterprise Entities. (Troux 2010) Reprinted with permission of Copyright © 2010 Troux Technologies. All other rights are reserved by the copyright owner.

Asset Domains	Concept Domains
Application and Software Domain	Analysis Domain
Data Domain	Financial Domain
Document Domain	General Domain
Infrastructure and Hardware Domain	Information Domain
IT Product Domain	IT Architecture Domain
IT Service Domain	Knowledge and Skill Domain
Location Domain	Market Domain
Organization Domain	Policy Domain
Product and Service Domain	Process Domain
Services Portfolio Management Domain	Resource Domain
	Strategy Domain
	Timeline Domain
	Transition Domain

The application/software and infrastructure/hardware domains are likely the most familiar to systems engineers (as illustrated in the figure below). The application/software domain contains things like the deployed software itself, plus applications, modules, servers, patches, functions, and messages. The infrastructure/hardware domain contains things like the hardware itself, plus networks and different kinds of hardware like computing hardware, cabinets, and network devices. There might be different subtypes of computing hardware like computers, servers, desktops, laptops, and mainframes. You can see from this elaboration of these domains that an enterprise architecture “schema” can be quite extensive in the kinds of things it can model.



The less technical domains would be things like policy, market, strategy, transition, financial, knowledge and skill, and analysis. In a typical enterprise architecture schema like this, there could be over a hundred types of modeling objects grouped into these domains. The examples give above are from the Trous Semantics metamodel used in the Trous Architect modeling tool for enterprise architecture activities. Other enterprise modeling tools have similar metamodels (sometimes called “schemas”). See Reese (2010) for more details on how to use the metamodel shown in the figure above.

Enterprise Architecture Frameworks & Methodologies

Enterprise architecture frameworks are collections of standardized viewpoints, views, and models that can be used when developing architectural descriptions of the enterprise. These architecture descriptions can be informal, based on simple graphics and tables, or formal, based on more rigorous modeling tools and methods. ISO/IEC 42010 (2011) specifies how to create architecture descriptions.

These frameworks relate to descriptive models of an enterprise, with conventions agreed in particular communities. There are various frameworks and methodologies available that assist in the development of an enterprise architecture.

Urbaczewski and Mrdalj (2006) provide an overview and comparison of five prominent architectural frameworks, including:

- the Zachman Framework for Enterprise Architecture (Zachman 1992),
- the Department of Defense Architecture Framework (DoDAF) (DoD 2010),
- the Federal Enterprise Architecture Framework (FEAF) (FEA 2001),
- the Treasury Enterprise Architecture Framework (TEAF) (US Treasury 2000),
- and The Open Group Architectural Framework (TOGAF) (TOGAF 2009).

References

Works Cited

- Blanchard, B.S., and W.J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Englewood Cliffs, NJ, USA: Prentice-Hall.
- Carlock, P., and R. Fenton. 2001. "System of Systems (SoS) Enterprise Systems Engineering for Information-Intensive Organizations." *Systems Engineering*. 4 (4): 242-261.
- CIO Council. 1999. *Federal Enterprise Architecture Framework (FEAF)*, version 1.1. Washington, DC, USA: Federal Chief Information Officers Council.
- DeRosa, J.K. 2005. "Enterprise Systems Engineering." Presented at Air Force Association, Industry Day, Day 1, August 4, 2005, Danvers, MA, USA.
- DoD. 2010. *DoD Architecture Framework (DoDAF)*, version 2.0. Washington, DC: U.S. Department of Defense (DoD).
- Elliott, C., and P. Deasley. 2007. *Creating Systems that Work--Principles of Engineering Systems for the 21st Century*. London, England, UK: Royal Academy of Engineering.
- FEA. 2001. "Federal Enterprise Architecture – Practical Guide, version 1.0, February 2001." Available: https://secure.cio.noaa.gov/hpcc/docita/files/a_practical_guide_to_federal_enterprise_architecture.pdf.
- Friedman, G., and A.P. Sage. 2004. "Case Studies of Systems Engineering and Management in Systems Acquisition." *Systems Engineering*. 7 (1): 84-96.
- Hall, A.D. 1989. *Metasystems Methodology: A New Synthesis and Unification*, 1st ed. Oxford, UK: Pergamon Press.
- Hitchins, D. 1993. *Putting Systems to Work*. New York, NY, USA: John Wiley & Sons.
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- Kuras, M.L., and B.E. White. 2005. "Engineering Enterprises Using Complex-Systems Engineering." Annotated presentation at 15th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 10-15, 2005, Rochester, NY, USA.
- MITRE. 2004. *MITRE 2004 Annual Report*". McLean, VA, USA: MITRE Corporation.
- Rebovich, G. 2006. "Systems Thinking for the Enterprise: New & Emerging Perspectives." Presented at IEEE/SMC International Conference on System of Systems Engineering, April 2006, Los Angeles, CA, USA.
- Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.
- Reese, R.J. 2010. *Troux Enterprise Architecture Solutions*. Birmingham, UK: Packt Publishing Ltd.
- Sage, A.P., and W.B. Rouse (eds). 2009. *Handbook of System Engineering and Management*, 2nd ed. New York, NY, USA: John Wiley & Sons.
- Swarz, R.S., J.K. DeRosa, and G. Rebovich. 2006. "An Enterprise Systems Engineering Model." Proceedings of the 16th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 9-13, 2006, Orlando, FL, USA.
- TOGAF. 2009. "The Open Group Architecture Framework," version 9. Accessed September 7, 2011. Available: <https://www.opengroup.org/togaf/>.
- Troux. 2010. *Metamodeling and modeling with Troux Semantics*, version 9. Austin, TX, USA: Troux Technologies.
- Urbaczewski, L., and S. Mrdalj. 2006. "A Comparison of Enterprise Architecture Frameworks." *Issues in Information Systems*. 7 (2): 18-26.
-

US Treasury. 2000. *Treasury Enterprise Architecture Framework*, version 1. Washington, DC, USA: US Department of the Treasury Chief Information Officer Council.

Zachman, J.A. 1992. "Extending and Formalizing the Framework for Information Systems Architecture." *IBM Systems Journal*. 31 (3): 590-616.

Zachman, J.A. 1987. "A Framework for Information Systems Architectures." *IBM Systems Journal*. 26 (3): 276-92.

Primary References

Kuras, M.L., and B.E. White. 2005. "Engineering Enterprises Using Complex-Systems Engineering." Annotated presentation at 15th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 10-15, 2005, Rochester, NY, USA.

Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.

Swarz, R.S., J.K. DeRosa, and G. Rebovich. 2006. "An Enterprise Systems Engineering Model." Proceedings of the 16th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 9-13, 2006, Orlando, FL, USA.

Additional References

Journal of Enterprise Architecture. Available: <https://www.globalaea.org/?page=JEAOverview>.

Minoli, D. 2008. *Enterprise Architecture A to Z: Frameworks, Business Process Modeling, SOA, and Infrastructure Technology*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group, An Auerbach Book.

TRAK. 2011. "TRAK Enterprise Architecture Framework." Accessed September 7, 2011. Available: <https://trak.sourceforge.net/index.html>.

Vernadat, F.B. 1996. *Enterprise Modelling and Integration - Principles and Applications*. London, UK: Chapman and Hall.

Enterprise Systems Engineering Process Activities

- Lead Authors:
- James Martin, Bud Lawson, and Alan Faisandier

The application of the key concepts of Enterprise Systems Engineering requires processes. These processes span and can transform the enterprise.

Systems Engineering Role in Transforming the Enterprise

Enabling Systematic Enterprise Change

The systems engineering (SE) process as applied to the enterprise as a whole could be used as the “means for producing change in the enterprise ... [where the] ... Seven Levels of change in an organization [are defined] as effectiveness, efficiency, improving, cutting, copying, differentiating and achieving the impossible” (McCaughin and DeRosa 2006). The essential nature of enterprise systems engineering (ESE) is that it “determines the balance between complexity and order and in turn the balance between effectiveness and efficiency. When viewed as the fundamental mechanism for change, it goes beyond efficiency and drives adaptation of the enterprise” (McCaughin and DeRosa 2006). McCaughin and DeRosa (2006) provide a reasonably good definition for an enterprise that captures this notion of balance:

Enterprise: People, processes and technology interacting with other people, processes and technology, serving some combination of their own objectives, those of their individual organizations and those of the enterprise as a whole.

Balancing Effectiveness versus Efficiency

Ackoff tells us that:

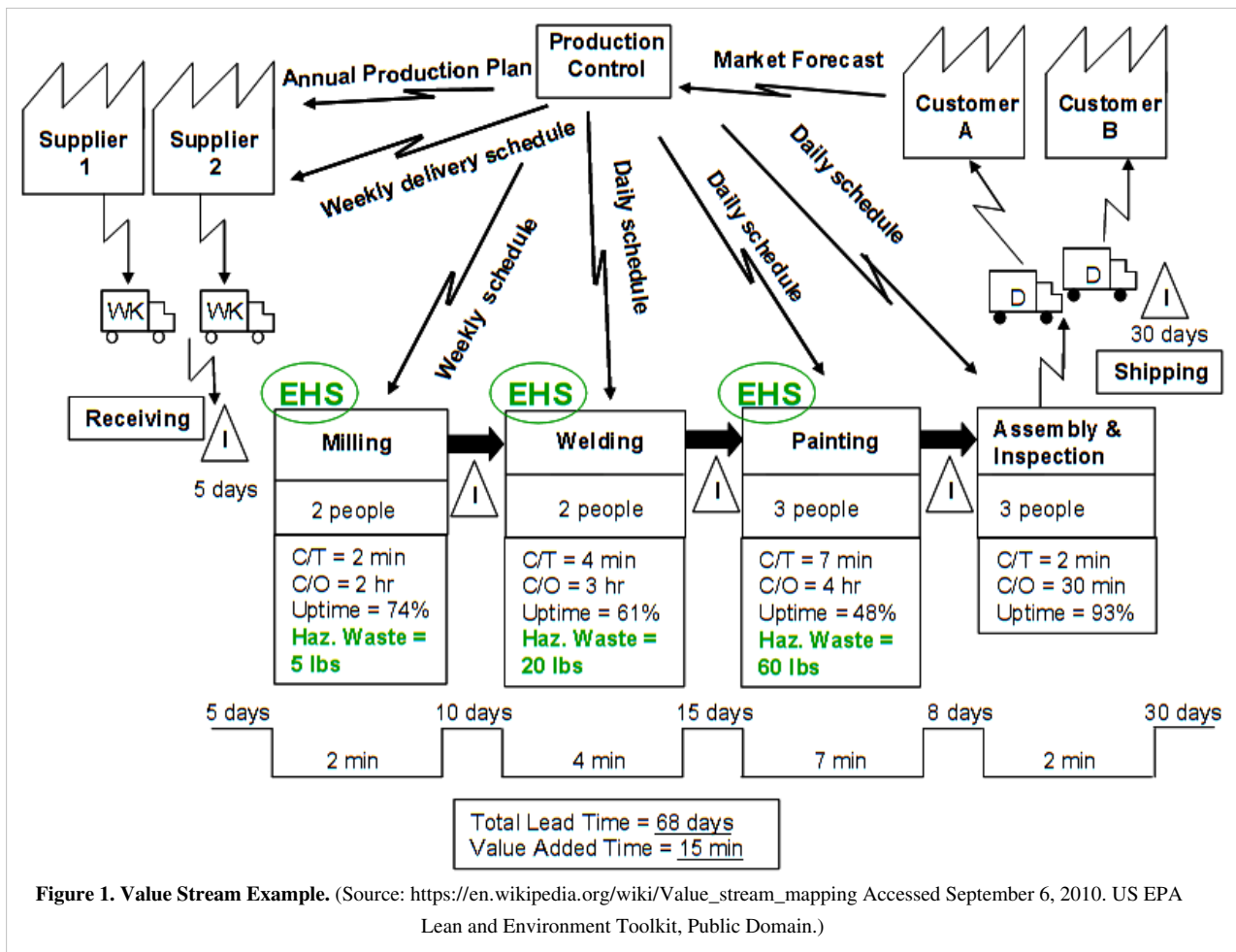
*Data, information, knowledge and understanding enable us to increase efficiency, not effectiveness. The value of the objective pursued is not relevant in determining efficiency, but it is relevant in determining effectiveness. Effectiveness is evaluated efficiency. It is efficiency multiplied by value. **Intelligence** is the ability to increase efficiency; **wisdom** is the ability to increase effectiveness.*

*The difference between efficiency and effectiveness is reflected in the difference between development and growth. Growth does not require an increase in value; development does. Therefore, development requires an increase in **wisdom as well as understanding, knowledge and information**. ((Ackoff 1989, 3-9), emphasis added)*

ESE has a key role to play in establishing the right balance between effectiveness and efficiency in enterprise operations and management. Value stream analysis is one technique, among others, that can help ESE determine where inefficiencies exist or ineffective results are being achieved.

Value Stream Analysis

Value stream analysis is one way of treating the enterprise as a system. It provides insights regarding where in the sequence of enterprise activities value is added as it moves towards the final delivery to customer or user (Rother and Shook 1999). It relates each step to the costs entailed in that step in terms of resource consumption (i.e., money, time, energy, and materials). In addition to direct costs, there may also be indirect costs due to overhead factors or infrastructure elements. This activity commonly involves drawing a flowchart of the value stream for the enterprise as illustrated in the figure below.



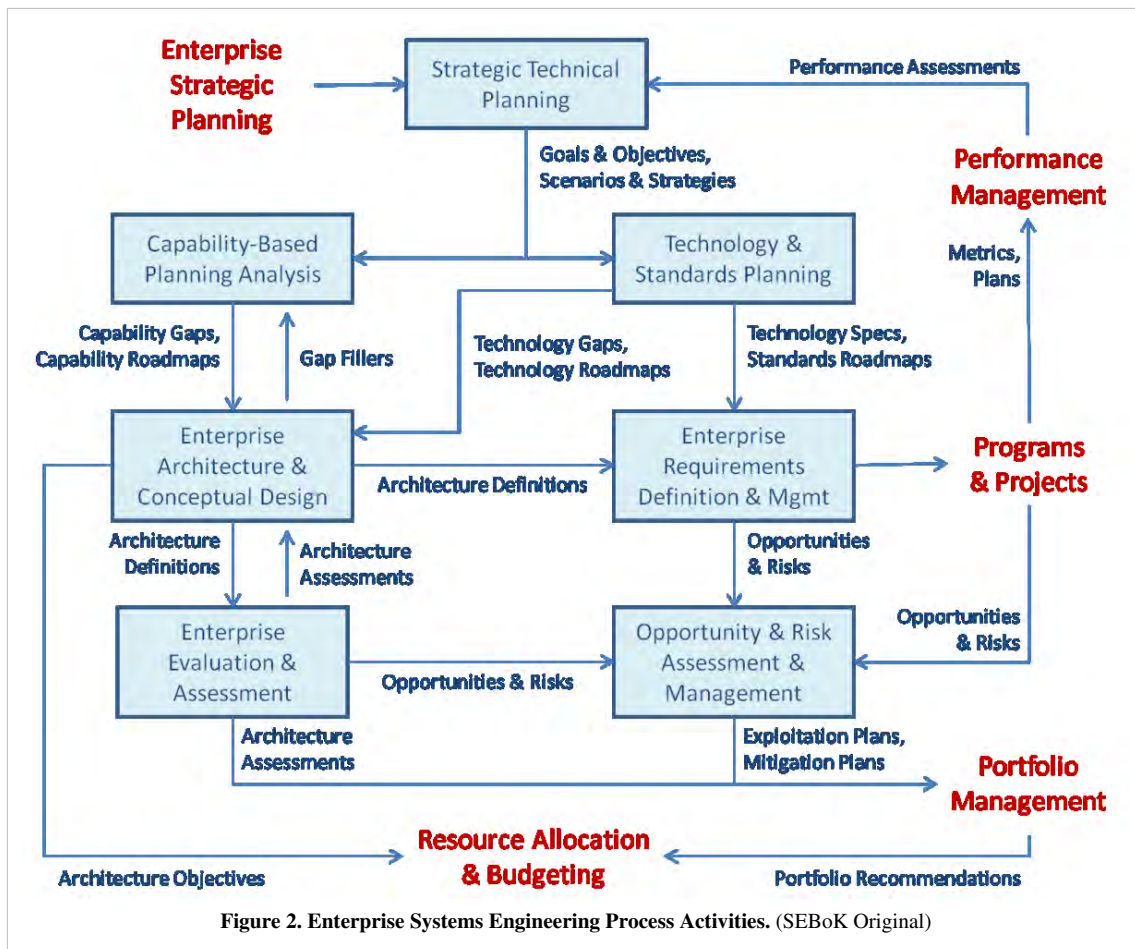
Analysis of this value stream diagram can highlight unnecessary space, excessive distance traveled, processing inefficiencies, and so on. Value stream mapping is associated with so-called “lean enterprise” initiatives. At Toyota, where the technique originated, it is known as “material and information mapping” (Rother 2009). Various value stream mapping tools are available (Hines and Rich 1997).

Enterprise Management Process Areas

Martin (2010) has determined that the following four processes are needed in ESE beyond the traditional SE processes in support of enterprise management activities:

1. Strategic technical planning,
2. Capability-based planning analysis,
3. Technology and standards planning, and
4. Enterprise evaluation and assessment.

The interactions between these four processes are illustrated below, along with their interactions with other processes that deal with architecture, requirements, risk, and opportunity.



Strategic Technical Planning

The purpose of strategic technical planning (STP) is to establish the overall technical strategy for the enterprise. It creates the balance between the adoption of standards (see also Systems Engineering Standards) and the use of new technologies, along with consideration of the people aspects driven by the relevant trans-disciplinary technical principles and practices from psychology, sociology, organizational change management, etc.

This process uses the roadmaps developed during technology and standards planning (TSP). It then maps these technologies and standards against the capabilities roadmap to determine potential alignment and synergy. Furthermore, lack of alignment and synergy is identified as a risk to avoid or an opportunity to pursue in the technical strategy. The technical strategy is defined in terms of implementation guidance for the programs and projects.

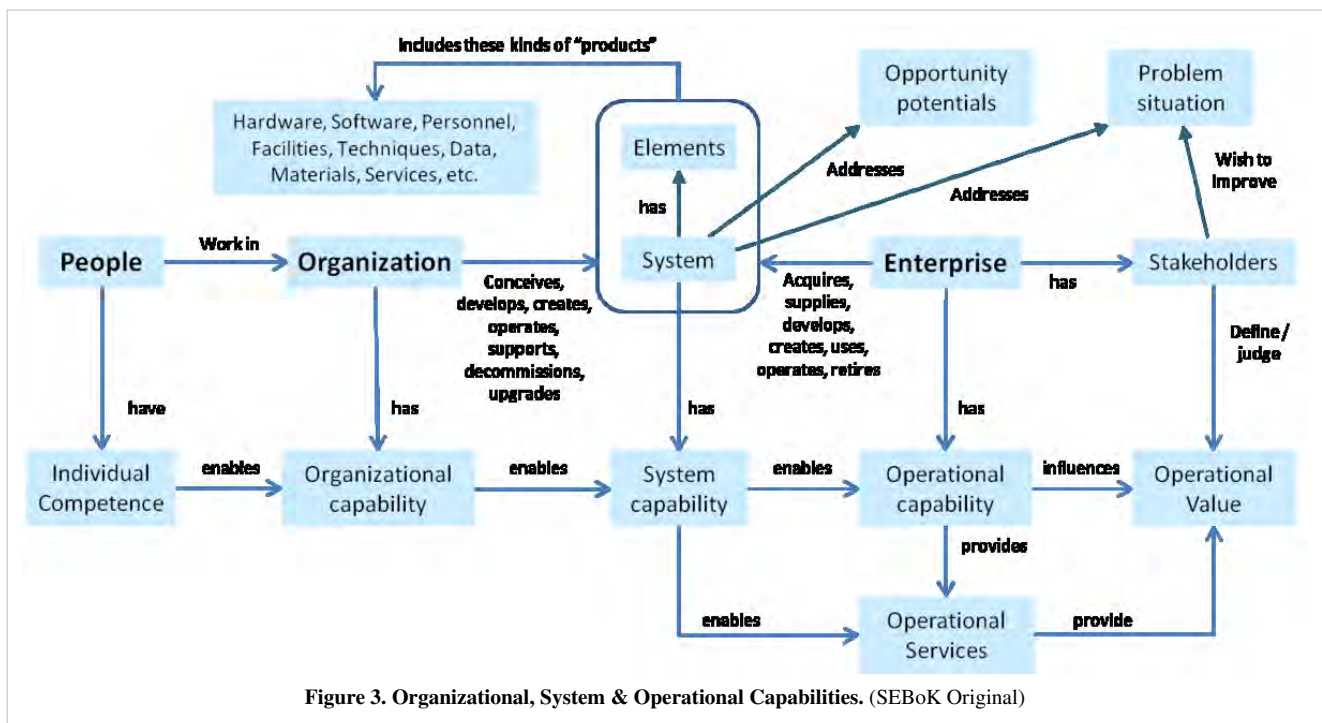
One reason that STP and TSP are separate processes is that they are often done by different groups in the enterprise and they involve different skill sets. TSP is often done by the technology and science groups. TSP is done closer to (if not in) the chief architect and budget planning groups. Sometimes the great technology proposed by TSP just doesn't line up with the capabilities needed in the requisite time frame. STP does this balancing between technology push and capability pull.

Capability-Based Planning Analysis

The purpose of *Capability-based Planning Analysis* is to translate the enterprise vision and goals into a set of current and future capabilities that helps achieve those goals. Current missions are analyzed to determine their suitability in supporting the enterprise goals. Potential future missions are examined to determine how they can help achieve the vision. Current and projected capabilities are assessed to identify capability gaps that prevent the vision and technical strategy from being achieved. These capability gaps are then used to assess program, project, and system opportunities that should be pursued by the enterprise. This is defined in terms of success criteria of what the enterprise is desired to achieve.

There are different types of capabilities, as shown in the figure below. It is common practice to describe capabilities in the form of capability hierarchies and capability roadmaps. Technology roadmaps (discussed below under Technology Planning) are usually related to the system capabilities while business capability roadmaps (BCRMs) are related to the operational capabilities of the enterprise as a whole (ref: Business-Capability Mapping: Staying Ahead of the Joneses, <https://msdn.microsoft.com/en-us/library/bb402954.aspx>). The BCRM development is usually done as part of enterprise strategic planning, which is one level higher than, and a key driver for, the strategic technical planning activity described above.

In some domains there may be competency roadmaps dealing with the organizational capabilities, with perhaps the desired competency levels of individuals mapped out in terms of the jobs or roles used in the enterprise or perhaps in terms of the knowledge and skills required for certain activities. (For more information on systems engineering competency, see the Enabling Individuals article.)



Technology and Standards Planning

The purpose of *Technology Planning* is to characterize technology trends in the commercial marketplace and the research community. This activity covers not just trend identification and analysis, but also technology development and transition of technology into programs and projects. It identifies current, and predicts future, technology readiness levels for the key technologies of interest. Using this information, it defines technology roadmaps. This activity helps establish the technical strategy and implementation guidance in the strategic technical plan. The business capabilities roadmap (BCRM) from the strategic planning activity is used to identify which technologies can contribute to achieved targeted levels of performance improvements.

The purpose of *Standards Planning* is to assess technical standards to determine how they inhibit or enhance the incorporation of new technologies into systems development projects. The future of key standards is forecast to determine where they are headed and the alignment of these new standards with the life cycles for the systems in the enterprise's current and projected future portfolios. The needs for new or updated standards are defined and resources are identified that can address these needs. Standardization activities that can support development of new or updated standards are identified (See also Systems Engineering Standards).

Enterprise Evaluation and Assessment

The purpose of enterprise evaluation and assessment (EE&A) is to determine if the enterprise is heading in the right direction. It does this by measuring progress towards realizing the enterprise vision. This process helps to “shape the environment” and to select among the program, project, and system opportunities. This is the primary means by which the technical dimensions of the enterprise are integrated into the business decisions.

This process establishes a measurement program as the means for collecting data for use in the evaluation and assessment of the enterprise. These measures help determine whether the strategy and its implementation are working as intended. Measures are projected into the future as the basis for determining discrepancies between what is observed and what had been predicted to occur. This process helps to identify risks and opportunities, diagnose problems, and prescribe appropriate actions. Sensitivity analysis is performed to determine the degree of robustness and agility of the enterprise.

Roberts states that EE&A must go beyond traditional system evaluation and assessment practices (Roberts 2006). He says that this process area:

must de-emphasize the utility of comparing detailed metrics against specific individual requirement values, whether the metrics are derived from measurement, simulation or estimation... [it] must instead look for break points where capabilities are either significantly enhanced or totally disabled.

Key characteristics of this activity are the following:

- Multi-scale analysis,
- Early and continuous operational involvement,
- Lightweight command and control (C2) capability representations,
- Developmental versions available for assessment,
- Minimal infrastructure,
- Flexible modeling and simulation (M&S), operator-in-the-loop (OITL), and hardware-in-the-loop (HWIL) capabilities, and
- In-line, continuous performance monitoring and selective forensics. (Roberts 2006)

Enterprise architecture (EA) can be used as a primary tool in support of evaluation and assessment. EA can be used to provide a model to understand how the parts of the enterprise fit together (or do not) (Giachetti 2010). The structure and contents of the EA should be driven by the key business decisions (or, as shown in the six-step process presented by Martin (2005), the architecture should be driven by the “business questions” to be addressed by the architecture).

The evaluation and assessment success measures can be put into the EA models and views directly and mapped to the elements that are being measured. An example of this can be seen in the US National Oceanographic and Atmospheric Agency (NOAA) EA shown by Martin (2003a and 2003b). The measures are shown, in this example, as success factors, key performance indicators, and information needs in the business strategy layer of the architecture.

EA can be viewed as either the set of artifacts developed as “views” of the enterprise, or as a set of activities that create, use, and maintain these artifacts. The literature uses these terms in both senses and it is not always clear in each case which sense is intended.

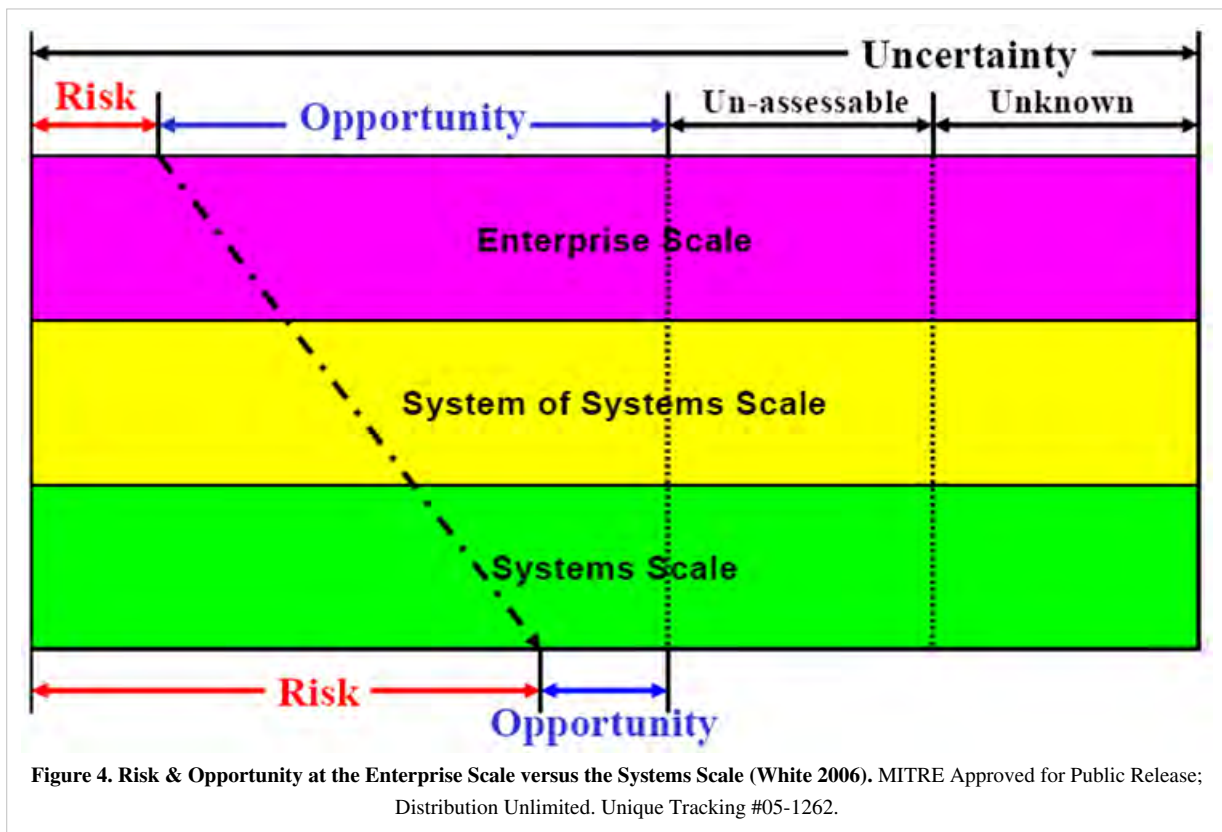
Enterprise Portfolio Considerations

Opportunity Assessment and Management

The management activities dealing with opportunities (as opposed to just risk) are included in ESE. According to White (2006), the “greatest enterprise risk may be in not pursuing enterprise opportunities.” Hillson believes there is:

a systemic weakness in risk management as undertaken on most projects. The standard risk process is limited to dealing only with uncertainties that might have negative impact (threats). This means that risk management as currently practiced is failing to address around half of the potential uncertainties—the ones with positive impact (opportunities). (Hillson 2004)

White claims that “in systems engineering at an enterprise scale the focus should be on opportunity, and that enterprise risk should be viewed more as something that threatens the pursuit of enterprise opportunities” (White 2006). The figure below (Rebovich and White 2011, chapter 5) shows the relative importance of opportunity and risk at the different scales of an individual system, a system of systems (SoS), and an enterprise. The implication is that, at the enterprise level, there should be more focus on opportunity management than on risk management.



Enterprise Architecture and Requirements

EA goes above and beyond the technical components of product systems to include additional items such as strategic goals and objectives, operators and users, organizations and other stakeholders, funding sources and methods, policies and practices, processes and procedures, facilities and platforms, infrastructure, and real estate. EA can be used to provide a model to understand how the parts of the enterprise fit together (or don't) (Giachetti 2010). The EA is not strictly the province of the chief information officer (CIO), and is not only concerned with information technology. Likewise, enterprise requirements need to focus on the cross-cutting measures necessary to ensure overall enterprise success. Some of these enterprise requirements will apply to product systems, but they may also apply to business processes, inter-organizational commitments, hiring practices, investment directions, and so on (Bernus, Nemes, and Schmidt 2003).

Architecture descriptions following the guidelines of an architecture framework have been used to standardize the views and models used in architecting efforts (Zachman 1987 and 1992; Spewak 1992). Architecture descriptions have also been developed using a business-question based approach (Martin 2003b; Martin 2006). The standard on Architecture Description Practices (ISO/IEC 42010) (ISO/IEC 2011) has expanded its scope to include requirements on architecture frameworks.

Government agencies have been increasingly turning to SE to solve some of their agency-level (i.e., enterprise) problems. This has sometimes led to the use of an architecture-based investment process, especially for information technology procurements. This approach imposes a requirement for linking business strategies to the development of EAs. The Federal Enterprise Architecture Framework (FEAF) (CIO Council 1999) and the DoD Architecture Framework (DoDAF) (DoD 2010) were developed to support such an architecture-based investment process. There have been several other architecture frameworks also developed for this purpose (ISO 2000; ISO/IEC 1998; NATO 2004; TOGAF 2009; MOD 2010; TRAK 2010).

ESE Process Elements

As a result of the synthesis outlined above, the ESE process elements to be used at the enterprise scale are as follows:

1. Strategic Technical Planning,
2. Capability-Based Planning Analysis,
3. Technology and Standards Planning,
4. Enterprise Evaluation and Assessment,
5. Opportunity and Risk Assessment and Management,
6. Enterprise Architecture and Conceptual Design,
7. Enterprise Requirements Definition and Management,
8. Program and Project Detailed Design and Implementation,
9. Program Integration and Interfaces,
10. Program Validation and Verification,
11. Portfolio and Program Deployment and Post Deployment, and
12. Portfolio and Program Life Cycle Support.

The first seven of these elements were described in some detail above. The others are more self-evident and are not discussed in this article.

References

Works Cited

- Ackoff, R.L. 1989. "From Data to Wisdom." *Journal of Applied Systems Analysis*. 16 (1): 3-9.
- Bernus, P., L. Nemes, and G. Schmidt (eds.). 2003. *Handbook on Enterprise Architecture*. Berlin and Heidelberg, Germany: Springer-Verlag.
- CIO Council. 1999. *Federal Enterprise Architecture Framework (FEAF), Version 1.1*. Washington, DC, USA: Federal Chief Information Officers Council.
- DoD. 2010. *DoD architecture framework (DoDAF)*, version 2.0. Washington, DC: US Department of Defense (DoD).
- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.
- Hillson, D. 2004. *Effective Opportunity Management for Projects: Exploiting Positive Risk*. Petersfield, Hampshire, UK; New York, NY, USA: Rick Doctor & Partners; Marcel Dekker, Inc.

- Hines, P., and N. Rich. 1997. "The Seven Value Stream Mapping Tools." *International Journal of Operations & Production Management*. 1 (17): 46-64.
- ISO. 2000. ISO 15704:2000, *Industrial Automation Systems — Requirements for Enterprise-Reference Architectures and Methodologies*. Geneva, Switzerland: International Organization for Standardization (ISO).
- ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.
- ISO/IEC. 1998. ISO/IEC 10746:1998, *Information Technology — Open Distributed Processing — Reference Model: Architecture*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC).
- Martin, J.N. 2010. "An Enterprise Systems Engineering Framework." Presented at 20th Anniversary International Council on Systems Engineering (INCOSE) International Symposium, July 12-15, 2010, Chicago, IL, USA.
- Martin, J.N. 2006. "An Enterprise Architecture Process Incorporating Knowledge Modeling Methods." PhD dissertation. Fairfax, VA, USA: George Mason University.
- Martin, J.N. 2005. "Using an Enterprise Architecture to Assess the Societal Benefits of Earth Science Research." Presented at 15th Annual International Council on Systems Engineering (INCOSE) International Symposium, 2005, Rochester, NY, USA.
- Martin, J.N. 2003a. "An Integrated Tool Suite for the NOAA Observing System Architecture." Presented at 13th Annual International Council on Systems Engineering (INCOSE) International Symposium, 2003, Arlington, VA, USA.
- Martin, J.N. 2003b. "On the Use of Knowledge Modeling Tools and Techniques to Characterize the NOAA Observing System Architecture." Presented at 13th Annual International Council on Systems Engineering (INCOSE) International Symposium, 2003, Arlington, VA, USA.
- McCaughin, K., and J.K. DeRosa. 2006. "Process in Enterprise Systems Engineering." Presented at 16th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 9-13, 2006, Orlando, FL, USA.
- MOD. 2010. *Ministry of Defence Architecture Framework (MODAF)*, version 1.2.004. London, England, UK: UK Ministry of Defence. Accessed September 8, 2011. Available: <https://www.mod.uk/NR/rdonlyres/04B5FB3F-8BBC-4A39-96D8-AFA05E500E4A/0/20100602MODAFDownload12004.pdf>.
- NATO. 2010. *NATO Architecture Framework (NAF)*, version 3.1. Brussels, Belgium: North Atlantic Treaty Organization.
- Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group, Auerbach.
- Roberts, J.L. 2006. "Enterprise Analysis and Assessment." Presented at 16th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 9-13, 2006, Orlando, FL, USA.
- Rother, M. 2009. *Toyota Kata: Managing People for Improvement, Adaptiveness, and Superior Results*. New York, NY, USA: McGraw-Hill.
- Rother, M., and J. Shook. 1999. *Learning to See: Value-Stream Mapping to Create Value and Eliminate MUDA*. Cambridge, MA, USA: Lean Enterprise Institute.
- Spewak, S.H. 1992. *Enterprise Architecture Planning: Developing a Blueprint for Data, Applications and Technology*. New York, NY, USA: Wiley and Sons, Inc.
- TOGAF. 2009. "The Open Group Architecture Framework," version 9. Accessed September 2, 2011. Available: <https://www.opengroup.org/togaf>.
-

TRAK. 2011. "TRAK Enterprise Architecture Framework." Accessed September 7, 2011. Available: <https://trak.sourceforge.net/index.html>.

White, B.E. 2006. "Enterprise Opportunity and Risk." Presented at 16th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 9-13, 2010, Orlando, FL, USA.

Zachman, J.A. 1992. "Extending and Formalizing the Framework for Information Systems Architecture." *IBM Systems Journal*. 31 (3): 590-616.

Zachman, J.A. 1987. "A Framework for Information Systems Architectures." *IBM Systems Journal*. 26 (3): 276-292.

Primary References

Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.

Martin, J.N. 2010. "An Enterprise Systems Engineering Framework." Presented at 20th Anniversary International Council on Systems Engineering (INCOSE) International Symposium, July 12-15, 2010, Chicago, IL, USA.

Rebovich, G., and B.E. White (eds.). 2011. *Enterprise Systems Engineering: Advances in the Theory and Practice*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group, Auerbach.

Additional References

DeRosa, J.K. 2005. "Enterprise Systems Engineering." Presented at Air Force Association, Industry Day, Day 1, August 4, 2005, Danvers, MA, USA.

Holt, J., and S. Perry. 2010. *Modelling enterprise architectures*. Stevenage, England, UK: Institution of Engineering and Technology (IET).

Kaplan, R., and D. Norton. 1996. *The Balanced Scorecard: Translating Strategy into Action*. Cambridge, MA, USA: Harvard Business School Press.

McGovern, J., S. Ambler, M. Stevens, J. Linn, V. Sharan, and E. Jo. 2004. *A Practical Guide to Enterprise Architecture*. New York, NY, USA: Prentice Hall.

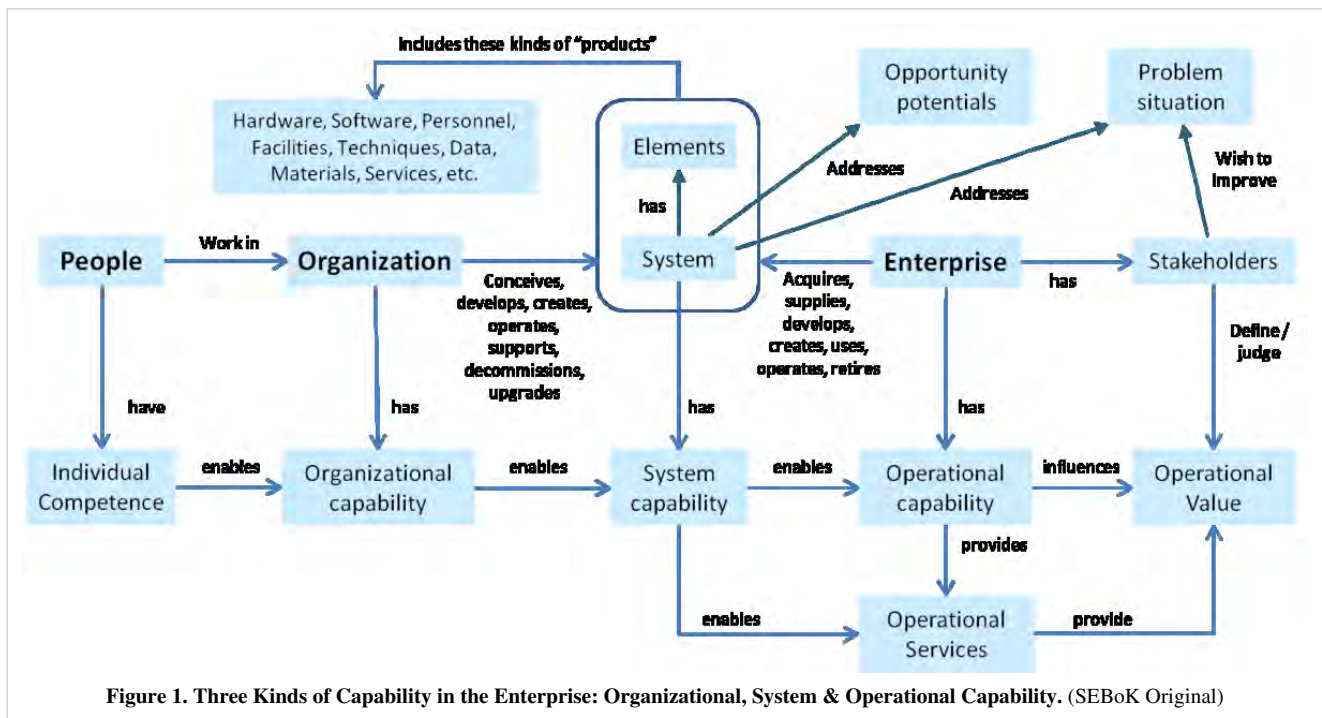
Swarz, R.S., J.K. DeRosa, and G. Rebovich. 2006. "An Enterprise Systems Engineering Model." INCOSE Symposium Proceedings, July 9-13, 2006, Orlando, FL, USA.

Enterprise Capability Management

- Lead Authors:
- James Martin, Bud Lawson, and Alan Faisandier

Introduction

There are three different kinds of capability: organizational capability, system capability, and operational capability. Management of organizational capability is addressed in the article called Enabling Businesses and Enterprises. Management of system capability is addressed by the Systems Engineering (SE) management activities described in the articles called Technical Management Processes and Product and Service Life Management. Management of operational capability is described herein.



The enterprise has a current and planned (baseline) operational capability, based on its past activities and on its current plans for change. The purpose of the enterprise capability management function is to ensure the possibility of “vectoring” the enterprise away from the current baseline trajectory to a more desirable position where it can better meet its enterprise strategic goals and objectives, given all its resource constraints and other limitations.

Operational capability may need to include elements identified in the Information Technology Infrastructure Library (ITIL) best practices for operations management, starting with strategic operation planning (OGC 2009).

The ITIL is a set of practices for IT service management (ITSM) that focuses on aligning IT services with the needs of business. In its current form ..., ITIL is published in a series of five core publications, each of which covers an ITSM lifecycle stage.

ITIL describes procedures, tasks and checklists that are not organization-specific, used by an organization for establishing a minimum level of competency. It allows the organization to establish a baseline from which it can plan, implement, and measure. It is used to demonstrate compliance and to measure improvement. (https://en.wikipedia.org/wiki/Information_Technology_Infrastructure_Library).

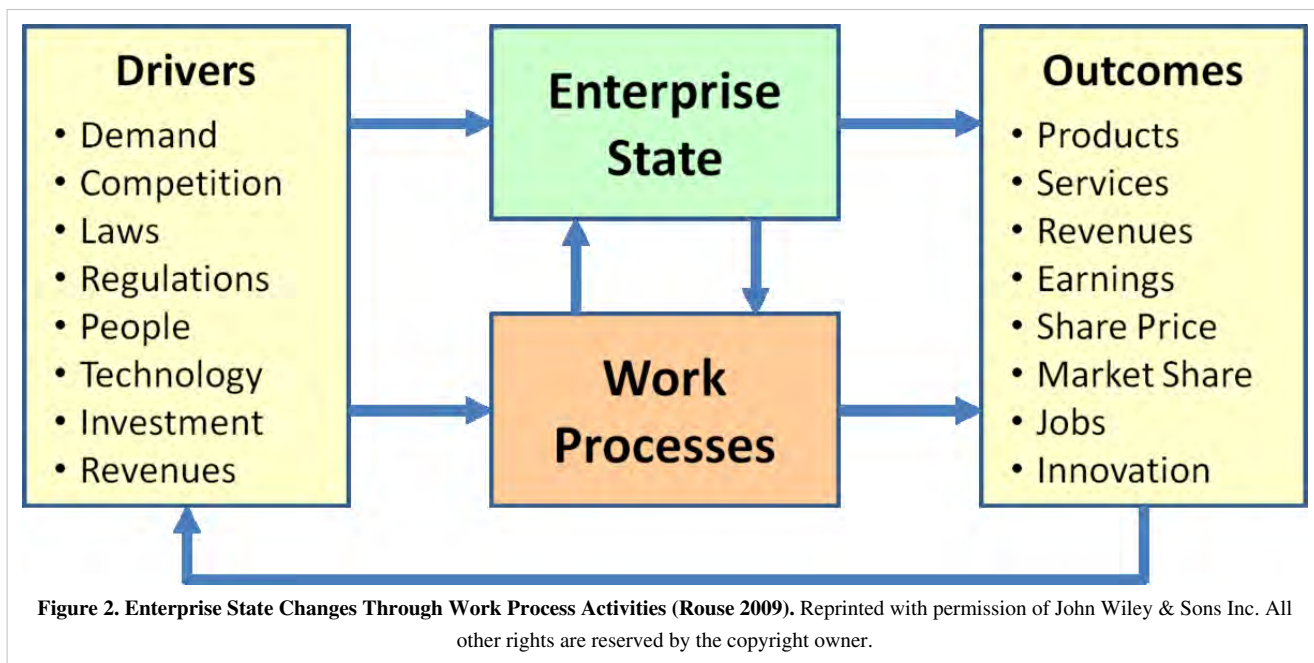
Needs Identification & Assessment

The enterprise has key stakeholders that have operational needs they would like the enterprise to address. These operational needs must be identified and assessed in terms of their relevance to the enterprise and the relative priorities of these needs compared to each other and to the priorities of the enterprise itself. The enterprise exists to meet these needs. An operational need is an expression of something desirable in direct support of the enterprise's end user activities. End user activities include such things as retail sales, entertainment, food services, and business travel. An example of an operational need is: "Provide transportation services to commuters in the metropolitan area of London."

Enterprise needs can be much more than eliminating waste, and the challenge for ESE might relate to any or all of the following: countering a perceived threat (business or military), meeting a policy goal (as in government), doing existing business more efficiently, taking advantage of technological opportunities, meeting new operational needs, replacing obsolete systems, creating integrated enterprises with others (on a temporary or permanent basis), and so on.

In addition to operational needs, there are enterprise needs that relate to enabling assets the enterprise has in place that allow the mission to be accomplished. Enabling assets are things such as personnel, facilities, communication networks, computing facilities, policies and practices, tools and methods, funding and partnerships, equipment and supplies, and so on. An enterprise need is an expression of something desirable in direct support of the enterprise's internal activities. Internal activities include such things as market forecast, business development, product development, manufacturing, and service delivery.

The purpose of the enterprise's enabling assets is to effect state changes to relevant elements of the enterprise necessary to achieve targeted levels of performance. The enterprise "state" shown in the figure below is a complex web of past, current and future states (Rouse 2009). The enterprise work processes use these enabling assets to accomplish their work objectives in order to achieve the desired future states. Enterprise architecture (EA) can be used to model these states and the relative impact each enabling asset has on the desired state changes.



Enterprise needs are related to the enterprise efficiencies achieved through the performance of enterprise activities. The main goal of enterprise needs is to maximize the efficient utilization of enterprise assets, or in other words, enhance productivity, and find and eliminate waste. Waste represents that which does not contribute to the enterprise mission or that cannot reasonably be expected to be accomplished by the enterprise. An example of an enterprise need is: "Decrease power required for operation of enterprise data centers." (Power is a limited asset that consumes

scarce enterprise funds that could be used for delivery of other more valuable services to its customers.)

Capability Identification & Assessment

The capabilities of an enterprise should exist for the sole purpose of meeting mission and enterprise needs. Hence, there will be both mission and enterprise capabilities to identify and assess how well they meet these needs. An example of an operational capability is: "Transport 150,000 passengers per hour among 27 nodes in the network." A supporting enterprise capability might be: "Process 200,000 tickets per hour during peak loading." There is a baseline capability due to capability development up to that point in time, plus any additional capability planned for the future. The desired levels of capability (based on needs assessment) are compared to the baseline capability to determine the capability gaps for the enterprise. This activity will also determine points of excess capability.

The gaps should be filled and the excesses should be eliminated. The projected gaps and excesses are sometimes mapped into several future timeframes to get a better understanding of the relative timing and intensity of change that might be required. It is typical to use time "buckets" like near-term, mid-term, and far-term, which, for some long-lasting capabilities, might correspond to five, ten, and twenty years out respectively. Of course, for fast-changing capabilities (like consumer products) these timeframes would necessarily be shorter in duration, for example, one, two and three years out.

Enterprise Architecture Formulation & Assessment

Enterprise architecture analysis can be used to determine how best to fill these capability gaps and minimize the excess capabilities (or "capacities"). Usually a baseline architecture is characterized for the purpose of understanding what one currently has and where the enterprise is headed under the current business plans. The needs and gaps are used to determine where in the architecture elements need to be added, dropped, or changed. Each modification represents a potential benefit to various stakeholders, along with associated costs and risks for introducing that modification. Enterprise architecture can be used to provide a model to understand how the parts of the enterprise fit together (or do not) (Giachetti 2010).

The enterprise architecture effort supports the entire capability management activity with enterprise-wide views of strategy, priorities, plans, resources, activities, locations, facilities, products, services, and so on (ISO/IEC/IEEE 15288 (ISO/IEC/IEEE 2015) and architectural design process: ISO/IEC 42010 (ISO/IEC 2011) and ISO 15704 (ISO 2000)).

Opportunity Identification & Assessment

The enterprise architecture is used to help identify opportunities for improvement. Usually these opportunities require the investment of time, money, facilities, personnel, and so on. There might also be opportunities for "divestment," which could involve selling of assets, reducing capacity, canceling projects, and so on. Each opportunity can be assessed on its own merits, but usually these opportunities have dependencies and interfaces with other opportunities, with the current activities and operations of the enterprise, and with the enterprise's partners. Therefore, the opportunities may need to be assessed as a "portfolio," or, at least, as sets of related opportunities. Typically, a business case assessment is required for each opportunity or set of opportunities.

Enterprise Portfolio Management

If the set of opportunities is large or has complicated relationships, it may be necessary to employ portfolio management techniques. The portfolio elements could be bids, projects, products, services, technologies, intellectual property, etc., or any combination of these items. Examples of an enterprise portfolio captured in an architecture modeling tool can be found in Martin (2005), Martin et al. (2004), and Martin (2003). See Kaplan's work (2009) for more information on portfolio management, and ISO/IEC (2008) for information on projects portfolio management

process.

Enterprise Improvement Planning & Execution

The results of the opportunity assessment are compiled and laid out in an enterprise plan that considers all relevant factors, including system capabilities, organizational capabilities, funding constraints, legal commitments and obligations, partner arrangements, intellectual property ownership, personnel development and retention, and so on. The plan usually goes out to some long horizon, typically more than a decade, depending on the nature of the enterprise's business environment, technology volatility, market intensity, and so on. The enterprise plan needs to be in alignment with the enterprise's strategic goals and objectives and with leadership priorities.

The planned improvements are implemented across the enterprise and in parts of the extended enterprise (glossary) where appropriate, such as suppliers in the supply chain, distributors in the distribution chain, financiers in the investment arena, and so on. The planned changes should have associated performance targets and these metrics should be monitored to ensure that progress is being made against the plan and that the intended improvements are being implemented. As necessary, the plan is adjusted to account for unforeseen circumstances and outcomes. Performance management of enterprise personnel is a key element of the improvement efforts.

Enterprise Capability Change Management

In an operational context (particularly in defense) the term "capability management" is associated with developing and maintaining all aspects of the ability to conduct certain types of missions in a given threat environment. In an industrial context, capability refers to the ability to manage certain classes of product and service through those parts of their life cycle that are relevant to the business. Changes to enterprise capability should be carefully managed to ensure that current operations are not adversely affected (where possible) and that the long term viability of the enterprise is maintained. The following seven lenses can be used to facilitate change management: strategic objectives, stakeholders, processes, performance metrics, current state alignment, resources, and maturity assessment (Nightingale and Srinivasan 2011).

Capability management is becoming more often recognized as a key component of the business management tool suite:

Capability management aims to balance economy in meeting current operational requirements, with the sustainable use of current capabilities, and the development of future capabilities, to meet the sometimes competing strategic and current operational objectives of an enterprise. Accordingly, effective capability management assists organizations to better understand, and effectively integrate, re-align and apply the total enterprise ability or capacity to achieve strategic and current operational objectives; and develops and provides innovative solutions that focus on the holistic management of the defined array of interlinking functions and activities in the enterprise's strategic and current operational contexts.
(Saxena 2009, 1)

There is a widespread perception that capability management is only relevant to defense and aerospace domains. However, it is becoming more widely recognized as key to commercial and civil government efforts.

References

Works Cited

- Giachetti, R.E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.
- ISO. 2000. ISO 15704:2000, *Industrial Automation Systems — Requirements for Enterprise — Reference Architectures and Methodologies*. Geneva, Switzerland: International Organization for Standardization (ISO).
- ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.
- ISO/IEC/IEEE. 2015. *Systems and software engineering - system life cycle processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC 15288:2015.
- Kaplan, J. 2009. *Strategic IT Portfolio Management: Governing Enterprise Transformation*. Waltham, MA, USA: Pittiglio, Rabin, Todd & McGrath, Inc. (PRTM).
- Martin, J.N. 2005. "Using an Enterprise Architecture to Assess the Societal Benefits of Earth Science Research." Presented at 15th Annual International Council on Systems Engineering (INCOSE) International Symposium, 2005, Rochester, NY, USA.
- Martin, J.N. 2003. "On the Use of Knowledge Modeling Tools and Techniques to Characterize the NOAA Observing System Architecture." Presented at 13th Annual International Council on Systems Engineering (INCOSE) International Symposium, 2003, Arlington, VA, USA.
- Martin, J.N., J. Conklin, J. Evans, C. Robinson, L. Doggrell, and J. Diehl. 2004. "The Capability Integration Framework: A New Way of doing Enterprise Architecture." Presented at 14th Annual International Council on Systems Engineering (INCOSE) International Symposium, June 20-24, 2004, Toulouse, France.
- Nightingale, D., and J. Srinivasan. 2011. *Beyond the Lean Revolution: Achieving Successful and Sustainable Enterprise Transformation*. New York, NY, USA: AMACOM Press.
- OGC (Office of Government Commerce). 2009. *ITIL Lifecycle Publication Suite Books*. London, UK: The Stationery Office.
- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of Systems Engineering and Management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY, USA: Wiley and Sons, Inc.
- Saxena, M.S. 2009. *Capability Management: Monitoring & Improving Capabilities*. New Dehli: Global India Publications Pvt Ltd.
- Wikipedia contributors. "Information Technology Infrastructure Library." *Wikipedia, The Free Encyclopedia*. Accessed November 28, 2012. Available at: [https:// en. wikipedia. org/ wiki/ Information_Technology_Infrastructure_Library](https://en.wikipedia.org/wiki/Information_Technology_Infrastructure_Library).
-

Primary References

- Kaplan, J. 2009. *Strategic IT Portfolio Management: Governing Enterprise Transformation*. Waltham, MA, USA: Pittiglio, Rabin, Todd & McGrath, Inc. (PRTM).
- Nightingale, D., and J. Srinivasan. 2011. *Beyond the Lean Revolution: Achieving Successful and Sustainable Enterprise Transformation*. New York, NY, USA: AMACOM Press.
- Rouse, W.B. 2009. "Engineering the Enterprise as a System," in *Handbook of Systems Engineering and Management*, 2nd ed., edited by A.P. Sage and W.B. Rouse. New York, NY, USA: Wiley and Sons, Inc.

Additional References

- Dahmann, J.S., J.A. Lane, and G. Rebovich. 2008. "Systems Engineering for Capabilities." *CROSSTALK: The Journal of Defense Software Engineering* 21 (11): 4-9.
- Hillson, D. 2004. *Effective Opportunity Management for Projects: Exploiting Positive Risk*. Petersfield, Hampshire, UK; New York, NY: Rick Doctor & Partners; Marcel Dekker, Inc.
- Lillehagen, F., J. Kostie, S. Inella, H.G. Solheim, and D. Karlsen. 2003. "From enterprise modeling to enterprise visual scenes." Presented at International Society for Pharmaceutical Engineering (ISPE) Conference on Concurrent Engineering (CE), July 26-30, 2003, Madeira Island, Portugal.
- McGovern, J., S. Ambler, M. Stevens, J. Linn, V. Sharan, and E. Jo. 2004. *A Practical Guide to Enterprise Architecture*. New York, NY: Prentice Hall.
- Rechtin, E. 1999. *Systems Architecting of Organizations: Why Eagles Can't Swim*. Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.
- Roberts, J.L. 2006. "Enterprise Analysis and Assessment." Presented at 16th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 9-13, 2006, Orlando, FL, USA.

Knowledge Area: Systems of Systems (SoS)

Systems of Systems (SoS)

Contents of this Knowledge Area

- Architecting Approaches for Systems of Systems (Judith Dahmann and Mike Henshaw)
- Systems Engineering for Systems of Systems (Mike Henshaw and Judith Dahmann)
- Systems of Systems Analytic Approaches (Mike Henshaw and Judith Dahmann)
- System of Systems and Complexity (Judith Dahmann)
- Socio-Technical Features of Systems of Systems (Judith Dahmann, Mike Henshaw, and Bud Lawson) (Heidi Davidz and Alan Faisandier)
- Capability Engineering (Duncan Kemp, Judith Dahmann, and Mike Henshaw)
- Mission Engineering (Judith Dahmann) (Ron Giachetti, Andy Hernandez, and Rhys Kissell)
- Lead Authors:
- Mike Henshaw and Judith Dahmann

System of systems engineering (SoSE) is not a new discipline; however, this is an opportunity for the systems engineering community to define the complex systems of the twenty-first century (Jamshidi 2009). SoSE represents a challenge for system engineers, since it requires considerations beyond those usually associated with engineering to include socio-technical and sometimes socio-economic phenomena.

Characteristics and Definition of Systems of Systems

Maier (1998) postulated five key characteristics (not criteria) of SoS: operational independence of component systems, managerial independence of component systems, geographical distribution, emergent behavior, and evolutionary development processes, and identified operational independence and managerial independence as the two principal distinguishing characteristics for applying the term 'systems-of-systems.' A system that does not exhibit these two characteristics is not considered a system-of-systems regardless of the complexity or geographic distribution of its components.

In the Maier characterization, emergence is noted as a common characteristic of SoS particularly in SoS composed of multiple large existing systems, based on the challenge (in time and resources) of subjecting all possible logical threads across the myriad functions, capabilities, and data of the systems in an SoS. As introduced in the article Emergence and Complexity, there are risks associated with unexpected or unintended behavior resulting from combining systems that have individually complex behavior. These become serious in cases which safety, for example, is threatened through unintended interactions among the functions provided by multiple constituent systems in a SoS.

ISO/IEC/IEEE 21839 (ISO, 2019) provides a definition of SoS and constituent system:

System of Systems (SoS) — *Set of systems or system elements that interact to provide a unique capability that none of the constituent systems can accomplish on its own. Note: Systems elements can be necessary to facilitate the interaction of the constituent systems in the system of systems*

Constituent Systems — *Constituent systems can be part of one or more SoS. Note: Each constituent is a useful system by itself, having its own development, management goals and resources, but interacts within the SoS to provide the unique capability of the SoS.*

In addition, there are several definitions of system(s) of systems (SoS), some of which are dependent on the particularity of an application area (Jamshidi, 2005).

It should be noted that formation of a SoS is not necessarily a permanent phenomenon, but rather a matter of necessity for integrating and networking systems in a coordinated way for specific goals such as robustness, cost, efficiency, etc.

The US DoD (2008) defines Systems of Systems Engineering as “planning, analyzing, organizing, and integrating the capabilities of a mix of existing and new systems into an SoS capability greater than the sum of the capabilities of the constituent parts”.

Because of the independence of the constituent systems, systems engineering processes are in most cases implemented for engineering both the constituent systems and the system of systems and need to be tailored to support the characteristics of SoS. These processes are shown in the table.

Table 1. Differences Between Systems and Systems of Systems as They Apply to Systems Engineering.

SE Process	Implementation as Applied to SoS
Agreement processes	Because there is often no top level SoS authority, the SoS often operate without explicit agreements and in some cases effective agreements among the systems in the SoS may be key to successful SoSE.
Organizational project enabling processes	When organizational processes exist, SoSE develops and maintains those for the SoS within the constraints of the system level processes.
Technical management processes	SoSE implements technical management processes applied to the particular considerations of SoS engineering - planning, analyzing, organizing, and integrating the capabilities of a mix of existing and new systems into a system-of-systems capability while systems continue to be responsible for technical management of their systems.
Technical processes	SoSE technical processes define the cross-cutting SoS capability, through SoS level business/mission analysis and stakeholder needs and requirements definition. SoS architecture and design frame the planning, organization and integration of the constituent systems, constrained by system architectures. Development, integration, verification, transition and validation are implemented by the systems. with SoSE monitoring and review. SoSE integration, verification, transition and validation applies when constituent systems are integrated into the SoS and performance is verified and validated.

Finally, based on work done by the INCOSE Systems of Systems Work Group (Dahmann, 2014), the major challenges facing SoSE have been catalogued in terms of seven pain points. These challenges are presented in the SoSE section of the INCOSE SE Handbook. (INCOSE 2023). These challenges include:

- **SoS Authorities.** In a SoS each constituent system has its own local ‘owner’ with its stakeholders, users, business processes and development approach. As a result, the type of organizational structure assumed for most traditional systems engineering under a single authority responsible for the entire system is absent from most SoS. In a SoS, SE relies on cross-cutting analysis and on composition and integration of constituent systems which, in turn, depend on an agreed common purpose and motivation for these systems to work together towards collective objectives which may or may not coincide with those of the individual constituent systems.
- **Leadership.** Recognizing that the lack of common authorities and funding pose challenges for SoS, a related issue is the challenge of leadership in the multiple organizational environment of a SoS. This question of leadership is experienced where a lack of structured control normally present in SE of systems requires alternatives to provide coherence and direction, such as influence and incentives.
- **Constituent Systems’ Perspectives.** Systems of systems are typically comprised, at least in part, of in-service systems, which were often developed for other purposes and are now being leveraged to meet a new or different application with new objectives. This is the basis for a major issue facing SoS SE; that is, how to technically address issues which arise from the fact that the systems identified for the SoS may be limited in the degree to which they can support the SoS. These limitations may affect the initial efforts at incorporating a system into a

SoS, and systems' commitments to other users may mean that they may not be compatible with the SoS over time. Further, because the systems were developed and operate in different situations, there is a risk that there could be a mismatch in understanding the services or data provided by one system to the SoS if the particular system's context differs from that of the SoS.

- **Capabilities and Requirements.** Traditionally (and ideally) the SE process begins with a clear, complete set of user requirements and provides a disciplined approach to develop a system to meet these requirements. Typically, SoS are comprised of multiple independent systems with their own requirements, working towards broader capability objectives. In the best case the SoS capability needs are met by the constituent systems as they meet their own local requirements. However, in many cases the SoS needs may not be consistent with the requirements for the constituent systems. In these cases, the SoS SE needs to identify alternative approaches to meeting those needs through changes to the constituent systems or additions of other systems to the SoS. In effect this is asking the systems to take on new requirements with the SoS acting as the 'user'.
- **Autonomy, Interdependencies and Emergence.** The independence of constituent systems in a SoS is the source of a number of technical issues facing SE of SoS. The fact that a constituent system may continue to change independently of the SoS, along with interdependencies between that constituent system and other constituent systems, add to the complexity of the SoS and further challenges SE at the SoS level. In particular, these dynamics can lead to unanticipated effects at the SoS level leading to unexpected or unpredictable behavior in a SoS even if the behavior of constituent systems is well understood.
- **Testing, Validation, and Learning.** The fact that SoS are typically composed of constituent systems which are independent of the SoS poses challenges in conducting end-to-end SoS testing as is typically done with systems. Firstly, unless there is a clear understanding of the SoS-level expectations and measures of these expectations, it can be very difficult to assess level of performance as the basis for determining areas which need attention, or to assure users of the capabilities and limitations of the SoS. Even when there is a clear understanding of SoS objectives and metrics, testing in a traditional sense can be difficult. Depending on the SoS context, there may not be funding or authority for SoS testing. Often the development cycles of the constituent systems are tied to the needs of their owners and original ongoing user base. With multiple constituent systems subject to asynchronous development cycles, finding ways to conduct traditional end-to-end testing across the SoS can be difficult if not impossible. In addition, many SoS are large and diverse making traditional full end-to-end testing with every change in a constituent system prohibitively costly. Often the only way to get a good measure of SoS performance is from data collected from actual operations or through estimates based on modeling, simulation and analysis. Nonetheless the SoS SE team needs to enable continuity of operation and performance of the SoS despite these challenges.
- **SoS Principles.** SoS is a relatively new area, with the result that there has been limited attention given to ways to extend systems thinking to the issues particular to SoS. Work is needed to identify and articulate the cross cutting principles that apply to SoS in general, and to developing working examples of the application of these principles. There is a major learning curve for the average systems engineer moving to a SoS environment, and a problem with SoS knowledge transfer within or across organizations.

Types of SoS

In today's interconnected world, SoS occur in a broad range of circumstances. In those situations where the SoS is recognized and treated as a system in its right, an SoS can be described as one of four types (Maier, 1998; Dahmann and Baldwin, 2008, ISO 21839, 2019):

- **Directed** - The SoS is created and managed to fulfill specific purposes and the constituent systems are subordinated to the SoS. The component systems maintain an ability to operate independently; however, their normal operational mode is subordinated to the central managed purpose;
- **Acknowledged** - The SoS has recognized objectives, a designated manager, and resources for the SoS; however, the constituent systems retain their independent ownership, objectives, funding, and development and sustainment

approaches. Changes in the systems are based on cooperative agreements between the SoS and the system;

- **Collaborative** - The component systems interact more or less voluntarily to fulfill agreed upon central purposes. The central players collectively decide how to provide or deny service, thereby providing some means of enforcing and maintaining standards; and
- **Virtual** - The SoS lacks a central management authority and a centrally agreed upon purpose for the SoS. Large-scale behavior emerges—and may be desirable—but this type of SoS must rely on relatively invisible mechanisms to maintain it.

This taxonomy is based on the degree of independence of constituents and it offers a framework for understanding SoS based on the origin of the SoS objectives and the relationships among the stakeholders for both the SoS and its constituent systems. In most actual cases, an SoS will reflect a combination of SoS types which may change over time. This taxonomy is in general use. It is presented in ISO 21841, "Taxonomy of Systems of systems". Other taxonomies may focus on nature/type of components, their heterogeneity, etc. (Cook, 2014)

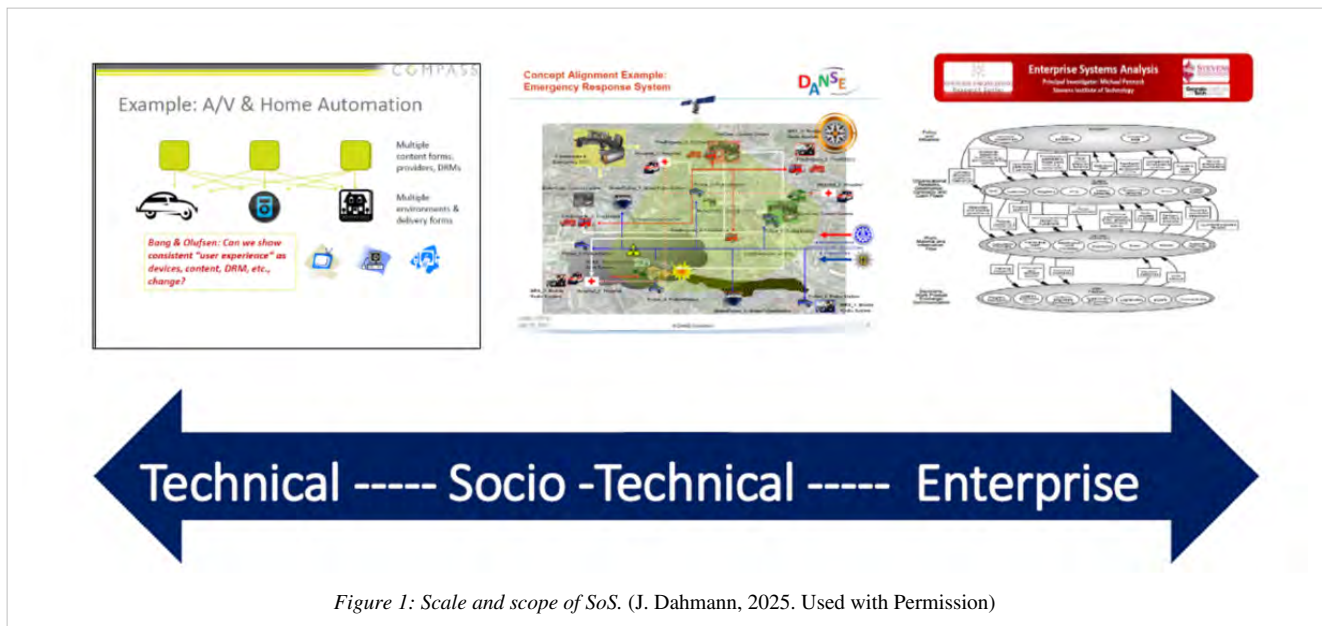
As noted above, many SoS exist in an unrecognized state; this is increasingly true as the levels of interconnectivity between modern systems keeps increasing. Kemp et al (2013) describe such systems as "accidental" but they can be described as "discovered" (Dahmann and Henshaw, 2016) because it is only when they become significant for some reason that we recognize them, at which point they can usually fall into one of the above four categories, since their significance means they must now operate, with management, in some defined way.

From the SoSE point of view, another potential classification would consider the level of anticipation/preparation of SoSE with respect to SoS operations and level of stability of the SoS objectives; this is referred to as variability by Kinder et. al. (2012). This could range from an SoS which responds to a particular trigger and is put immediately in place when needs are expressed. An example of such an SoS would be a crisis management SoS. This type of SoS is updated dynamically during the operation. At the other end of the spectrum there are well-specified and stable SoS developed to answer to specified ongoing needs. An example of such a persistent SoS is an air traffic management system. This type of SoS is acquired and qualified in a well-defined environment and any need for evolution will imply a formal SE evolution and re-qualification.

While much of the early attention to SoS has focused on Acknowledged SoS where current SE practices can be adapted and applied, there is an increasing recognition that the predominance of SoS exist in the collaborative and virtual types (Honour 2016), and in those areas where SoS may not be officially recognized but affect many of the broader capabilities in today's interconnected world. In these cases, the focus is shifting to understanding SoS as socio-technical, complex adaptive systems rather than extensions of current technical systems with a focus on understand and addressing the inherent complexity of these types of SoS.

Scale and Scope of SoS

Earliest reference to systems of systems (Ackoff, 1971) described systems of systems as largely composed of organizations. As indicated above, SoS can range in scale and scope incorporating both technical systems as well as organizations, as shown in Figure 1.



In some cases, SoS may focus almost entirely on technical integration with an example from an early European Commission (EC) research project COMPASS [Fitzgerald et al., 2012] which used as a case study in SoS the integration of the components of a consumer audio-visual ‘system’. In other cases, there are socio-technical systems of systems where not only are the systems integral to the SoS capability but also the organizations and their processes. The figure shows the example of a disaster response SoS, a case example developed by the DANSE [Lochow, 2016] project. Finally, broader SoS address the scale and scope of enterprise level concerns. The example in the figure is the case of counterfeiting as an enterprise comprised of a wide variety of systems, organizations, policies, and competing efforts. [Bodner, 2015]

SoSE Application Domains

Application of SoSE is broad and is expanding into almost all walks of life. After initial attention in space systems (Krygiel, 1980), SoSE was initially applied across defense. SoSE application is now much broader and still expanding. The early work in the defense sector has provided the initial basis for SoSE, including its intellectual foundation, technical approaches, and practical experience. In addition, parallel developments in information services and rail have helped to develop SoSE practice (Kemp and Daw, 2015). Now, SoSE concepts and principles apply across other governmental, civil and commercial domains.

A review of SoS IEEE publications on SoS from 2021-2023 [Dahmann, 2024] found that SoS is an area of global interest (see Table 2). Note that this analysis includes only references from IEEE publications and does not account for other published works (e.g. in INCOSE).

**Table 2: Geographical distribution of authors of IEEE SoS papers from 2021-2023
[Dahmann, 2024, Used with Permission]**

Top Countries	#	%	Top Regions	#	%
US	38	22.5%	Europe	77	45.6%
China	32	18.9%	US	38	22.2%
Germany	20	11.8%	Asia	38	22.2%
France	12	7.1%	Rest of the world	16	9.5%
Brazil	8	4.7%			
Netherlands	5	3.0%			
Italy	5	3.0%			
Australia	4	2.4%			
Countries with 1 or 2 authors	45	26.6%			

Table 3 shows the topics addressed in these papers, again showing the diverse application of systems of systems in the literature. Increased networking and interconnectedness of systems today contributes to growth in the number and domains where SoS are becoming the norm, particularly with the considerable convergence among systems of systems, cyber-physical systems and the internet of things. (Henshaw, 2016).

Table 3: Topics Addressed in IEEE SoS papers from 2021-2023 [Dahmann, 2024, Used with Permission]

Application Areas	#	%
Defense	31	18.5%
Transportation	22	13.1%
Health	10	6.0%
IOT/CPS	8	4.8%
Energy	7	4.2%
Emergency/Crisis Management	6	3.6%
Space	4	2.4%
Search and Rescue	4	2.4%
Education	3	1.8%
Environment	3	1.8%
Remainder (<3)	70	41.7%

SoSE Standards

The first standards for system of systems engineering have been adopted by the International Organization for Standardization. These were initiated in 2016 response to the report of an ISO SoS Standards study group (ISO, 2016) recognizing the increased attention to SoS and the value to standards to the maturation of SoSE. Three standards are (INCOSE 2020):

- ***System of Systems (SoS) Considerations in Life Cycle Stages of a System - ISO/IEC/IEEE 21839***

This standard provides a set of critical considerations to be addressed at key points in the life cycle of systems created by humans and refers to a constituent system that will interact in a system of systems as the system of interest (SOI). These considerations are aligned with ISO/IEC/IEEE 15288 and the ISO/IEC/IEEE 24748-1 framework for system life cycle stages and associated terminology.

- ***Guidelines for the utilization of ISO/IEC/IEEE 15288 in the context of System of Systems (SoS) Engineering - ISO/IEC/IEEE 21840***

This standard provides guidance for the utilization of ISO/IEC/IEEE 15288 in the context of SoS. While ISO/IEC/IEEE 15288 applies to systems (including constituent systems), this document provides guidance on application of these processes to SoS. However, ISO/IEC/IEEE 21840 is not a self-contained SoS replacement for ISO/IEC/IEEE 15288. This document is intended to be used in conjunction with ISO/IEC/IEEE 15288, ISO/IEC/IEEE 21839 and ISO/IEC/IEEE 21841 and is not intended to be used without them.

- ***Taxonomy of Systems of Systems - ISO/IEC/IEEE 21841***

The purpose of this standard is to define normalized taxonomies for systems of systems (SoS) to facilitate communications among stakeholders. It also briefly explains what a taxonomy is and how it applies to the SoS to aid in understanding and communication.

References

Works Cited

- Ackoff, R.L.(1971) "Towards a Systems of Systems Concepts," *Manage. Sci.*, vol. 17, no. 11, pp. 661–671.
- Bodner, Douglas. Mitigating Counterfeit Part Intrusions with Enterprise Simulation. *Procedia Computer Science* 61 (2015) 233 – 239.
- Cook, S. C. and Pratt, J. M., "Towards designing innovative SoSE approaches for the Australian defence force," *Proc. 9th Int. Conf. Syst. Syst. Eng. Socio-Technical Perspect. SoSE 2014*, pp. 295–300, 2014.
- Dahmann, J, and M.J.D Henshaw. 2016. "Introduction to Systems of Systems Engineering", INCOSE INSIGHT, October 2016, Volume 19, Issue 3, pages 12-16.
- Dahmann, Judith. *Systems of Systems Characterization and Types*. NATO Publication Ref NBR EN-SCI-276-01.2015.
- Dahmann, Judith. 2015. *Systems of Systems Pain Points*, INCOSE International Symposium, Seattle, WA.
- Dahmann, J., and K. Baldwin. 2008. "Understanding the Current State of US Defense Systems of Systems and the Implications for Systems Engineering." Presented at IEEE Systems Conference, April 7-10, 2008, Montreal, Canada.
- DoD. 2008. *Systems Engineering Guide for Systems of Systems*. Arlington, VA: US Department of Defense, Director, Systems and Software Engineering, Deputy Under Secretary of Defense (Acquisition and Technology), Office of the Under Secretary of Defense (Acquisition, Technology and Logistics). Accessed 2 /26/2022. Available: <https://acqnotes.com/wp-content/uploads/2014/09/DoD-Systems-Engineering-Guide-for-Systems-of-Systems-Aug-2008.pdf>
- Fitzgerald, John, Jeremy Bryans, and Richard Payne. "A formal model-based approach to engineering systems-of-systems." *Collaborative Networks in the Internet of Services: 13th IFIP WG 5.5 Working Conference on Virtual Enterprises, PRO-VE 2012, Bournemouth, UK, October 1-3, 2012. Proceedings 13*. Springer Berlin

- Heidelberg, 2012. <https://dl.ifip.org/db/conf/ifip5-5/prove2012/FitzgeraldBP12.pdf>, accessed 4/6/205
- Henshaw, M.J.d., "Systems of Systems. Cyber-Physical Systems, the Internet of Things... Whatever Next?" INCOSE INSIGHT, October 2016, Volume 19, Issue 3, pages 51-54.
- Honour, E., "Engineering the Virtual or Collaborative SoS", INCOSE INSIGHT, October 2016, Volume 19, Issue 3, pages 67-69.
- INCOSE, 2020. Systems of Systems Standards Quick reference Guide, INCOSE -2020 -sosstandards.
- INCOSE, 2018. Systems of Systems Primer, INCOSE-TP-2018-003-01.0.
- INCOSE SE Handbook. 2023
- INCOSE, 2020. Systems of Systems Standards Quick Reference Guide, INCOSE -2020 -sosstandards.
- International Organization for Standardization (ISO). 2019. ISO/IEC/IEEE 21839 —Systems and Software Engineering—System of systems considerations in life cycle stages of a system.
- International Organization for Standardization (ISO). 2019. ISO/IEC/IEEE 21840 —Guidelines for the utilization of ISO/IEC/IEEE 15288 in the context of system of systems.
- International Organization for Standardization (ISO). 2019. ISO/IEC/IEEE 21481 —Systems and Software Engineering—Taxonomy of systems of systems.
- International Standards Organization, 2016. Report of the SC7 SG on Systems of Systems Engineering.
- International Organization for Standardization (ISO). 2015. ISO/IEC/IEEE 15288—Systems and Software Engineering—System life cycle processes.
- Jamshidi, M. (ed). 2009a. *Systems of Systems Engineering – Innovations for the 21st Century*. Hoboken, NJ, USA: Wiley.
- Jamshidi, Mo. (2005). System-of-Systems Engineering-a Definition. In International Conference on Systems, Man, and Cybernetics. IEEE
- Kemp, D., et. al.. 2013. *Steampunk System of Systems Engineering: A case study of successful System of Systems engineering in 19th century Britain.* Presented at INCOSE International Symposium, June 24–27, 2013, Philadelphia, PA.
- Kinder, A., Barot, V., Henshaw, M., & Siemieniuch, C. (2012). System of Systems: "Defining the system of interest." In Proc. 7th Int. Conf. Systems of Systems Eng., Genoa, Italy (pp. 463–468). Retrieved from https://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=6384211
- Lochlow, T (2016) "Tool and Techniques – DANSE". INSIGHT, 19: 55-58. <https://doi.org/10.1002/inst.12110>
- Maier, M.W. 1998. "Architecting Principles for Systems-of-Systems." *Systems Engineering*. 1 (4): 267-284.
- Rebovich Jr., G. 2009. "Chapter 6: Enterprise System of Systems," in *Systems of Systems Engineering - Principles and Applications*. Boca Raton, FL, USA: CRC Press.

Primary References

- Dahmann, J., and K. Baldwin. 2008. "Understanding the Current State of US Defense Systems of Systems and the Implications for Systems Engineering." Presented at IEEE Systems Conference, April 7-10, 2008, Montreal, Canada.
- DoD. 2008. Systems Engineering Guide for Systems of Systems, version 1.0. Washington, DC, USA: US Department of Defense (DoD). Available: <https://www.acq.osd.mil/se/docs/SE-Guide-for-SoS.pdf>.
- INCOSE Systems Engineering Handbook, 4th Edition, John Wiley & Sons Inc., 2015
- International Organization for Standardization, 2016. Report of the SC7 SG on Systems of Systems Engineering.
- Jamshidi, M. (ed). 2009a. *Systems of Systems Engineering – Innovations for the 21st Century*. Hoboken, NJ, USA: Wiley.

Maier, M.W. 1998. "Architecting Principles for Systems-of-Systems." *Systems Engineering*. 1 (4): 267-284.

Additional References

Barot, V., S. Henson, M. Henshaw, C. Siemieniuch, M. Sinclair, S.L. Lim, M. Jamshidi, and D. DeLaurentis. 2012. *Trans-Atlantic Research and Education Agenda in Systems of Systems (T-AREA-SoS) SOA Report*. Longborough, England, UK: Longborough University. Ref. TAREA-RE-WP2-R-LU-7.

Boardman, J., and B. Sauser. 2006. "System of Systems - the Meaning of Of." IEEE Conference on Systems of Systems Engineering, April 24-26, 2006, Los Angeles, CA.

Carlock, P., and J.A. Lane. 2006. *System of Systems Enterprise Systems Engineering, the Enterprise Architecture Management Framework, and System of Systems Cost Estimation*. Los Angeles, CA, USA: Center for Systems and Software Engineering (CSSE), University of Southern California (USC). USC-CSE-2006-618.

Checkland, P.B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

Dahmann, J., Rebovich, G., Lane, J., Lowry, R. & Baldwin, K. 2011. "An Implementer's View of Systems Engineering for Systems of Systems." IEEE Systems Conference, April 4-7, 2011, Montreal, Canada. p. 212-217.

Keating C.B., J.J. Padilla, and K. Adams. 2008. "System of systems engineering requirements: Challenges and guidelines". *EMJ - Engineering Management Journal*. 20 (4): 24-31.

Luzeaux, D., and J.R. Ruault. 2010. *Systems of Systems*. London, UK: ISTE.

MITRE. "System of Systems Engineering Collaborators Information Exchange (SoSECIE) Webinar Archive," MITRE, 2019. Available: [1]

Neaga, E.I., M.J.d. Henshaw, and Y. Yue. 2009. "The influence of the concept of capability-based management on the development of the systems engineering discipline." *Proceedings of the 7th Annual Conference on Systems Engineering Research*, April 20-23, 2009, Loughborough University, Loughborough, England, UK.

Poza, A.S., S. Kovacic, and C. Keating. 2008. "System of Systems Engineering: An Emerging Multidiscipline". *International Journal of System of Systems Engineering*. 1 (1/2).

Ring J. 2002. "Toward an ontology of systems engineering." *INSIGHT*. 5 (1): 19-22.

Relevant Videos

- Systems of Systems (Somerville) ^[2]

References

[1] <https://mitre.tahoe.appsembler.com/blog>

[2] <https://www.youtube.com/watch?v=ryLeFaHarPQ>

Architecting Approaches for Systems of Systems

- Lead Authors:
- Judith Dahmann and Mike Henshaw

Essentially, the role of the Systems Engineer with respect to system of systems SoS is to compose the constituent systems to meet the goals and needs of the SoS. In practice, this will largely require management of interfaces between constituent systems and leveraging the existing functionality of the constituent systems. However, it may also include changing the functionality of some constituent systems, the performance of interfaces, or the introduction of additional capability the purpose of which is solely to facilitate SoS operation.

Systems architectures and the process of architecting is covered by the ISO/IEC/IEEE 42000 series of standards and is overviewed by System Architecture Design Definition. In this article, we are concerned with the specific concerns related to architecting SoS.

ISO/IEC/IEEE 42010 (2022) defines architecting as ‘conceiving, defining, expressing, documenting, communicating, certifying proper implementation of, maintaining and improving an architecture throughout the life cycle of an entity of interest’. For a SoS, the level of control varies according to its type; for a directed SoS, the owner, as architect, may be able to treat it as a monolithic system, whereas for a collaborative or virtual SoS, the architect may have little knowledge of many of the constituent systems. Furthermore, the notion of lifecycle is somewhat different because of the manner in which SoS evolve. Some may evolve over long periods of time and be reasonably stable, whereas others may be assembled transiently to achieve a specific purpose and then disassembled. The architectural approaches may need to be different accordingly.

(Wilkinson *et al.*, 2010) analyzed different belief systems in systems architecting and highlighted two important differences pertinent to the approach to architecting and the contrast between a monolithic system and a SoS. Firstly, they determined that whereas some architects through the purpose of architecting is to ‘enable the building of better systems’, i.e. a design approach which they termed *Forward Architecting*, others regarded its purpose as ‘architecting is to understand existing parts of the environment as systems’, referred to as *Reverse Architecting*. Secondly, they contrasted the view of Maier & Rectin (2009) of the purpose to be to deliver maximum value to the client (monolithic system) with MODAF (2008) that sought coherence across the capability of the SoS. MODAF, a contemporary architecture framework (now withdrawn) that has been described by Hause (2010).

The key themes for architecting SoS are interoperability and the management of interfaces, design for evolution and reconfiguration, and increasingly the role of MBSE and simulation within an architectural context.

The Role of System of Systems Architecting

In a SoS, the architecture is the technical framework for the systems comprising the SoS which designates how the systems will be employed by the users in an operational setting (sometimes called the concept of operations (CONOPs), the internal and external relationships and dependencies among the constituent systems and their functions and, finally, the end-to-end functionality and data flow as well as communications among the systems.

The management and/or operational independence of the constituent systems drive the special considerations for architecting a SoS. (Maier, 1998) has provided four principles for architecting SoS, motivated by these characteristics:

- The architect must pay attention to intermediate steps in a planned evolution of the SoS. This is realized in in the Wave Model [ref], which employs stable intermediate steps. This principle seems appropriate to the directed and acknowledged types of SoS, but is unlikely to be possible with the other types.
 - Application of standards and protocols through, what Maier terms, policy triage. This recognizes that the opportunities to influence the constituent system design are limited and that the points of influence must be chosen carefully.
-

- As noted above, architecting of SoS largely focuses on the interfaces, this means that the architect must leverage the interfaces to realize the capability that the SoS is assembled to deliver. Agreement on interface definition is essential and the use of standards and, in particular, Open Systems Standards, is a means through which competent composition and flexible evolution may be achieved. It is worth noting that the inclusion of COTS (Commercial Off-The- Shelf) systems within the SoS relies on compliance with standards both during configuration of the SoS and the design of constituent systems. COTS, and MOTS (Modified Off-The-Shelf) are frequently used to reduce cost and risk in systems development.
- Ensuring cooperation. Maier observes that collaboration between independent systems is only possible if the systems owners choose to collaborate. Thus, incentivizing collaboration, which requires an appreciation of motivations and non-technical aspects of assembly (such as commercial benefits and protection of IP) must be considered.

SoS are generally characterized by complexity and their assembly requires a mindset that thinks of design not as defined by an endpoint but by continual change and adaptation. The INCOSE Complexity Primer for Systems Engineers [INCOSE, 2021] articulates this well as ‘Think like a gardener, not a watchmaker’. This is the mindset that the SoS architect must adopt.

Interoperability as a Key Feature of Architecting for SoS

Interoperability within a SoS implies that each system can interact with appropriate other members of the SoS to enable higher level functions unachievable by the single systems, regardless of their hardware and software characteristics or nature. This implies that each constituent member (and potential new members) of a SoS should be able to exchange data/information with others without compatibility issues in the operating systems, communication hardware, and so on. In general, interoperability focuses on data and information, although physical interoperability may sometimes be needed.

Data exchange patterns are comprised of three elements: an architectural pattern, a data format, a communication protocol (Charest *et al.*, 2020). There are many common languages for exchange of information and data, with XML (eXtensible Markup Language) being a popular choice.

However, interoperability must be achieved at many levels and not just at the data/network level. There are a number of frameworks that describe the levels of interoperability. From military applications, the NCOIC (Network Centric Operations Industry Consortium) Interoperability Framework (Osvalds, 2009) covers three broad levels of interoperability, subdivided into further layers as indicated below:

- Network Transport:
 - Physical Interoperability and
 - Connectivity and Network Interoperability;
- Information Services:
 - Data/Object Model Interoperability,
 - Semantic/Information Interoperability, and
 - Knowledge/Awareness of Actions Interoperability; and
- People, Processes and Applications:
 - Aligned Procedures,
 - Aligned Operations,
 - Harmonized Strategy/Doctrine, and
 - Political or Business Objectives.

This spectrum of interoperability layers requires appropriate coherence at each layer consistent with the SoS shared goals. Indeed, the vertical consideration of the framework is essential because, for example, agreement about the data/object model interoperability level requires alignment of business objectives through collaborative or

cooperative engagement.

There exist interoperability frameworks in other fields of activity. An example is the European Interoperability Framework (European Commission 2017), which focuses on enabling business (particularly e-business) interoperability and has six levels within a political context

- Interoperability Governance
- Integrated Public Service governance
- Legal Interoperability,
- Organizational Interoperability,
- Semantic Interoperability, and
- Technical Interoperability.

The interoperability between the component systems of a SoS is a fundamental design consideration for SoS that may be managed through the application of standards.

Challenges in Architecting SoS

It is rare that architecting a SoS can begin unencumbered by extant designs and legacy systems or infrastructure. In general, the SoS must be composed with extant constituent systems, perhaps designed to different standards and for different purposes. Moreover, the architect's knowledge of existing systems or those managed by alternative organizations, even competitors, is constrained. Thus, a mixed black-box, white-box approach to architecture definition may be needed. Even if the interfaces are well-defined, hidden functionality could potentially lead to undesirable emergent behavior.

The independence of the constituent systems means that these systems are typically designed to optimize system behavior, which may be at the expense of optimization for the SoS objectives. Indeed, it could be the case that the architect requires a constituent system to operate sub-optimally at the system level in order to achieve overall SoS effectiveness. (Rebovich 2009) has articulated this difficulty as a fundamental problem of SoS:

From the single-system community's perspective, its part of the SoS capability represents additional obligations, constraints and complexities. Rarely is participation in an (sic) SoS seen as a net gain from the viewpoint of single-system stakeholders.

The development and implementation of a SoS architecture may be significantly constrained by a reluctance to make changes or invest in the constituent systems, which could be very mature (e.g. in sustainment) or currently productively supporting other uses. In this case, approaches such as gateways and wrapping may be used to incorporate these systems into the SoS without making significant changes in the other systems.

The co-simulation community seek to integrate different types of models within an overall framework, e.g. Gomes et al, 2018. This can be extended to consider models of different constituent systems using an MBSE approach. This has been demonstrated by (Dahmann *et al.*, 2017) in which a SoS architecture built with SysML includes executable models of the platforms, weapons, communications and data links.

(Maier, 1998) third principle, concerning leveraging the interfaces in SoS architecting and assembly is illustrated by (Shames, Sarrel and Friedenthal, 2016) using the example of a space data system. They highlight the role of standards in modelling the SoS interfaces using SysML defining the Interface Modelling Method that employs a black-box / white-box approach and the final step of which is to specify the standards that define interface compliance. The Object Management Group (OMG) are active in the development of standards to support interface design, as described by (Fosse and Delp, 2013).

Open Systems Architecture (OSA)

Open Systems Architectures is a particular approach to standards that seeks to enable commercial, technological, and operational agility (Henshaw *et al.*, 2011) by publishing standards enabling competition amongst suppliers and opportunities to rapidly deploy new systems (or sub-systems) within a SoS. It should be emphasized that Open Systems Architectures refers to an approach to the use of standards within acquisition and procurement, rather than to a particular class of architecture. Typically, government or tier 1 integrators may see benefits by enabling greater competition within the supply chain, and lower tier organizations may see benefits due to reduced barriers to entry. (Henshaw *et al.*, 2011) define an open architecture as: 'An Open (Systems) Architecture applies to a system in which the architecture is published in sufficient detail to enable change and subsequent evolution through the introduction or replacement of modules and/or components from any supplier', whereas the Defense Acquisition Glossary uses the definition: 'A technical architecture that adopts open standards supporting a modular, loosely coupled and highly cohesive system structure that includes publishing of key interfaces within the system and full design disclosure' [DAU, n.d.], whereas Radoman et al [2025] has provided a systemigram description of Open Systems Architecture, combining many different views and presenting an in depth analysis of benefits and actions to implement the approach.

OSA are used across several domains and, typically, open architecture standards are domain specific. Perhaps the broadest applicability is the MOSA (Modular Open Systems Approach), <https://www.cto.mil/sea/mosa> that integrates business and technical strategy across the US DoD. Within the automotive industry, AUTOSAR, is an industry developed set of standards adopted by major automotive manufacturers as an effective supply chain management approach focusing on software. However, this might be regarded as an approach for complicated software-driven systems rather than SoS.

(Radoman *et al.*, 2023) has reviewed more than 80 open Architecture Standards in the military domain and provided a detailed database of her review.

References

Works Cited

- Cláudio Gomes, Casper Thule, David Broman, Peter Gorm Larsen, and Hans Vangheluwe. 2018. Co-Simulation: A Survey. *ACM Comput. Surv.* 51, 3, Article 49 (May 2019), 33 pages. <https://doi.org/10.1145/3179993>
- Charest, G. *et al.* (2020) *Advisory note: Data Exchange Methods and Considerations*. Available at: https://enterprisearchitecture.harvard.edu/sites/hwpi.harvard.edu/files/enterprise/files/data_exchange_advisory_v1_final.pdf?m=1581437469.
- Dahmann, J. *et al.* (2017) 'SysML Executable Systems of System Architecture Definition: A Working Example', in *2017 Annual IEEE International Systems Conference (SysCon)*. Montreal, CA: IEEE.
- Defense Acquisition University, Glossary, Fort Belvoir, VA, <https://www.dau.edu/glossary/open-systems-architecture>, accessed 03/25/25
- The European Interoperability Framework in Detail, 2017, <https://interoperable-europe.ec.europa.eu/collection/nifo-national-interoperability-framework-observatory/european-interoperability-framework-detail>,
- Fosse, E. and Delp, C.L. (2013) 'Systems Engineering Interfaces: A Model Based Approach', in *2012 IEEE Aerospace Conference*. Pasadena: IEEE. Available at: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6497322> (Accessed: 30 January 2025).
- M. Hause, "The Unified Profile for DoDAF/MODAF (UPDM) enabling systems of systems on many levels," *2010 IEEE International Systems Conference*, San Diego, CA, USA, 2010, pp. 426-431, doi: 10.1109/SYSTEMS.2010.5482450.

- Henshaw, M. *et al.* (2011) *Assessment of Open Architectures within Defence Procurement, Crown owned copyright [2011]*. Available at: <https://dspace.lboro.ac.uk/dspace-jspui/handle/2134/8828>.
- INCOSE (2021) "A Complexity Primer for Systems Engineers, Revision 1"
- Maier, M.W. (1998) 'Architecting principles for systems-of-systems', *Systems Engineering*, 1(4), pp. 267–284. Available at: [https://doi.org/10.1002/\(SICI\)1520-6858\(1998\)1:4<267::AID-SYS3>3.0.CO;2-D](https://doi.org/10.1002/(SICI)1520-6858(1998)1:4<267::AID-SYS3>3.0.CO;2-D).
- Osvalds, G., Grumman, N., Yanosy, J., Collins, R., Robert, L. S.-S., & Lebas -Thales, P.-S. F.-X. (2009). NIF WG NIF Solution Document Reference Manual NSD-RM. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=df42971f88001761b69a0da3fea763a85bbf8b59>
- Radoman, R.L.V. *et al.* (2023) 'Mapping of Open Architectures Applied to Military Systems', *2023 18th Annual System of Systems Engineering Conference, SoSe 2023*, pp. 1–8. Available at: <https://doi.org/10.1109/SoSE59841.2023.10178668>.
- Radoman, Raquel Lampaça Vieira; Henshaw, Michael; King, Melanie; Rabbets, Tim (2023). Mapping of Open Architecture Standards Applied to Military Systems (dataset). Loughborough University. Dataset. <https://doi.org/10.17028/rd.lboro.22241656.v2>.
- Radoman, R.L.V.; Henshaw, M.; King, M.; Rabbets, T. Enabling Open Architecture in Military Systems: A Systemic and Holistic Analysis. *Systems* 2025, 13, 207. <https://doi.org/10.3390/systems13030207>
- Shames, P.M., Sarrel, M.A. and Friedenthal, S.A. (2016) 'Modeling systems-of-systems interfaces with SysML', in *14th International Conference on Space Operations, 2016*. American Institute of Aeronautics and Astronautics Inc, AIAA. Available at: <https://doi.org/10.2514/6.2016-2500>.
- Wilkinson, M. *et al.* (2010) 'Belief systems in systems architecting: Method and preliminary applications', *2010 5th International Conference on System of Systems Engineering, SoSE 2010*, pp. 1–7. Available at: <https://doi.org/10.1109/SYBOSE.2010.5544095>.

Primary References

None

Additional References

None

Systems Engineering for Systems of Systems

- Lead Authors:
- Mike Henshaw and Judith Dahmann

Cook and Unewisse [2017] in “A SoS Approach for Engineering Capability Programs” outline the various approaches which have been adopted for SoSE. These are provided here using text drawn from this paper.

Enhanced Traditional Systems Engineering (ETSE)

ETSE applies traditional top-down systems engineering at the level of a portfolio of projects by adapting systems engineering processes and using architecture frameworks to represent the artifacts. Ideal for centralized, directed SoS management in a slowly changing environment. Key references: Levis and Wagenhals, (2000), USN, (2006 a&b).

Complex Systems Engineering (CSE)

CSE is based on complexity theory and strongly advocates that the top-down ETSE approach is not well suited to SoSE because it cannot handle the complexity of SoS and because the preconditions for TSE to be successful are not evident in SoS. CSE changes the focus from “...here is the solution designed from the requirements, now go implement it...” to “...here are the selective pressures acting on the elements present, now resolve or reduce them...” Ideal for rapidly integrating SoS from pre-existing Constituent Systems (CS) where little centralized control exists. Key reference: Bar-Yam, (2003); Norman and Kuras, (2006).

Dynamic Optimization of SOS using Value Measurement (DOSVM)

DOSVM draws on complexity theory and recognizes that it is the Constituent System Project Offices (CSPO) that have the resources and means to change, and that in many SoS, the actual authority and resources of any central element are never going to be sufficient to do more than guide the evolution of the SoS. In DOSVM, each CSPO views the SoS in terms of its utility to itself, seeking to “optimize” the SoS (from its point of view) through the influences available to it. DOSVM is ideal for collaborative SoS in which there is little or no central control. Key references: Honour (2016), Honour and Browning, (2007).

SoS Governance (SoSG)

SoSG, through its origins in complexity theory, seeks to expand SoSE away from the “technology first and technology only” perspective of earlier versions of SoSE. It includes appreciating the context to determine what initiatives might be feasible; identifying areas that can improve the SoS; and adopting a “long-term view” of the evolutionary development of the SoS. SoSG appreciates that de-centralized control can be expected and is suited to a wide range of collaborative and acknowledged SoS challenges. Key references: Keating (2015), Morris et al. (2006).

US Department of Defence SE for SoS: The Wave Model

The wave model is an evolutionary model that comprises five main process elements that incorporate experiential learning from many SoSE programs. It was originally designed for acknowledged SoS that have a small SoSE team but has also been applied to collaborative SoS challenges. It is a meta-methodology, i.e. one that guides the design of SoSE methodologies and is extensively documented. The wave model is widely applicable and is tailorable to the expected Australian Program level of SoSE effort. Key references: DoD, (2008), Dahmann et al (2011), Lane et al. (2010) & Dahmann and Heilmann (2012).

US Navy Mission Engineering Approach

Mission engineering arose to support the assessment of naval systems and capabilities through a SoS approach to analyze the impact of making naval investments across the diverse domains of surface, undersea, air, land, and networks as well as maritime coalition force integration. Mission engineering assessments are executed following a systematic, quantifiable, and iterative approach, which combines the structure of systems engineering (SE) and the tactical insights from operational planning. This approach incorporates a mission focus into integrated capability development. Note that mission engineering activities can be aligned to the elements of the wave model. Key references, Moreland (2015), TTCP (2016).

The British Systems Thinking Approach (BSTA)

BSTA embodies soft systems thinking, systems theory, social theory, and the pragmatism of problem solving to achieve shared meaning and objectives across the stakeholder group to deal with the social and technical aspects of SoS simultaneously. Key to the approach is the mapping of the SoS of interest onto more detailed SE approaches. The BSTA is the ideal approach for SoS problems where there is no obvious consensus of what represents a good SoS outcome trajectory and where the SoS is already established and running. It works well for a wide range of problem definition challenges within the CLC. It can be combined with other approaches. Key references: Checkland and Scholes (1990), Hitchins, (2007).

Systemic Strategic Planning and Execution (SSPE)

(SSPE) is a comprehensive but austere multi-methodology inspired firstly by strategic planning, systems theory and BSTA to achieve inclusivity and stakeholder engagement in a systemic way and secondly by ideas from system engineering to achieve structured abstraction and trade-offs between candidate force structures. SSPE is ideally suited to force design and has been used successfully in Australia; it needs to be enhanced with additional technical approaches to cover technical integration aspects. References: Hodge and Cook (2014a, b &c).

A Hybrid SoSE Approach Based on the Wave Model

This is an austere hybrid of the wave model with key elements from other approaches such as mission engineering. The wave model is inherently evolutionary and can be tailored to be agile, pragmatic, and austere and it is a proven approach that is well documented and has a track record of successful implementation, see, for example, Scrapper et al. (2016). Furthermore, for subsequent iterations, the wave model has the richness to support the technical aspects of integration, and the performance and behavioral analysis required for mission engineering. In addition, Dahmann (2012) has shown how the wave model can incorporate Program-level SoS test and evaluation. The proposed hybrid approach seeks to draw on the ability to design and manage enduring SoS capabilities across multiple defined development stages (as per the wave model) as well as the focus provided by considering the operational missions that instantiations of the SoS are to undertake thereby enabling assessment (as per mission engineering).

References

Works Cited

- Bar-Yam Y., 2003, "Why Systems Engineering Fails – Towards Complex Systems Engineering", IEEE International Conference on Systems, Man and Cybernetics.
- Checkland, P. & Scholes, J., 1990. *Soft systems methodology in action*, Wiley.
- Cook S.C. & Unewisse M. H., 2017, 'A SoS Approach for Engineering Capability Programs', 27th Annual INCOSE International Symposium (IS 2017) Adelaide, Australia, July 15-20, 2017
- Dahmann, J., Rebovich, G., Lowry, R., Lane, J., & Baldwin, K., 2011, "An implementers' view of systems engineering for systems of systems", 2011 IEEE International Systems Conference (SysCon), pp. 212-217. IEEE.
- Dahmann J. & Heilmann R. 2012 "SoS Systems Engineering and Test and Evaluation", NDIA T&E Conference.
- Lane, J., Dahmann, J., Rebovich, G. and Lowry, R., 2010, "Key system of systems engineering artefacts to guide engineering activities", NDIA Systems Engineering Conference.
- Levis, A.H. and Wagenhals, L.W., 2000, "C4ISR architectures: I. Developing a process for C4ISR architecture design", *Systems engineering*, 3(4), pp.225-247. Hitchins, D.K., 2008, *Systems engineering: a 21st century systems methodology*. Wiley.
- Hodge, R.J. and Cook, S.C., 2013, "Achieving action to improve the framework for defence strategy and execution: A case study", *System of Systems Engineering (SoSE)*, 2013 8th International Conference on (pp. 314-319). IEEE.
- Hodge R.J. and Cook S.C., 2014a, "A system of systems methodologies for Strategic Planning in Complex Defence Enterprises" in Gorod A., White B. Ireland V., Gandhi S.J. & Sauser B., *Case Studies in System of System, Enterprise Systems, and Complex Systems*, Taylor and Francis.
- Hodge R.J. and Cook S.C., 2014b, "Assessing whole-of-nation capabilities to respond to serious and unusual emergencies" in Gorod A., White B. Ireland V., Gandhi S.J. & Sauser B., *Case Studies in System of System, Enterprise Systems, and Complex Systems*, Taylor and Francis.
- Honour, E. and Browning T., 2007, "Dynamic Optimization of Systems of Systems using Value Measurement," *Transactions of the Society for Design and Process Science* 11 (1); 1-11.
- Honour, E., 2016, "Engineering the Virtual or Collaborative SoS", *INSIGHT*, Oct 2016.
- Keating, C.B., 2015, "Complex system governance: Theory to practice challenges for system of systems engineering", *System of Systems Engineering Conference (SoSE)*, pp. 226-231 IEEE.
- Moreland J.D Jr, 2015, "Mission Engineering Integration and Interoperability", *Leading Edge*, Jan 2015.
- Morris E., Place P. & Smith D., 2006, *System-of-Systems Governance: New Patterns of Thought*, Software Engineering Institute, CMU/SEI-2006-TN-036.
- Norman D.O. and Kuras M.L., 2006, "Engineering Complex Systems", in *Complex Engineered Systems*, Springer, ISBN 978-3-540-32831-5, 206-245.
- USN, 2006a, US Naval "Systems of Systems" Systems Engineering Guidebook Volume 1, ASN(RDA), US Navy.
- USN, 2006b, US Naval "Systems of Systems" Systems Engineering Guidebook Volume 2, ASN(RDA), US Navy.

Systems of Systems Analytic Approaches

- Lead Authors:
- Mike Henshaw and Judith Dahmann

A review of SoS IEEE publications on SoS from 2021-2023 [Dahmann, 2024] looked across the SoS papers published in IEEE from 2020-2023 addressing a range of domains, incorporated several technical approaches including model-based approaches, graph analysis and artificial intelligence.

Model-based Approaches

First, as systems engineering moves towards becoming a model-based discipline, SoS engineering is part of this movement. Looking across the 138 domain focused papers in this review, the vast majority (120 or 90.9%) apply some type of model-based approach. These model-based approaches take different forms, including:

Network Models

These models represent the system of systems (SoS) as a network of interconnected nodes. They are used to analyze the robustness, resilience, and other properties of the system. Examples include the combat network model, network modeling of kill-web, and network model of a weapon technology system-of-systems.

Agent-Based Models

These models represent the SoS as a collection of agents, each with its own behavior. They are used to simulate the behavior of the system and evaluate policy changes. Examples include the agent-based modeling to represent the kidney transplant system of systems [Threlkeld, R et al. 2022].

Mathematical and Optimization Models

These models represent the SoS using mathematical equations or optimization problems. They are used to analyze and optimize the system's performance. Examples include the mixed-integer programming model for the combat system-of-systems architecture design problem and the mathematical model for the set covering problem.

Model-Based Systems Engineering (MBSE) and SysML

These tools are used to describe the structure and behavior of the SoS. They support activities like requirement analysis, design, verification, operation, and maintenance activities.

Simulation Models

These models are used to simulate the behavior of the SoS under different conditions. Examples include the simulation models to represent the kidney transplant system of systems [Threlkeld, R et al. 2022] and the digital real-time simulators.

Machine Learning Models

These models use machine learning algorithms to predict or classify outcomes based on input data. Examples include the use of machine learning algorithms to predict building maintenance and optimize energy use, and the use of a supervised learning model for classification and prediction of the failure status system.

Conceptual Models

These models represent the high-level structure and concepts of the SoS. Examples include the conceptual model for an interoperable and vendor-neutral communication framework for the agricultural domain and the conceptual model based on Hierarchical Systems (HS) technology.

Graph Analytic Approaches

By their nature networks of interconnected systems, SoS lend themselves to graph analytic approaches. From this set of SoS domain papers, 29 (of 132 or 22%) of the domain application papers employ graph analysis in some form. This includes:

Network Modeling

This includes modeling SoS architectures as single-layer or multilayer networks, using complex network methods for robustness assessment, and using network modeling and adjacency matrices to represent the kill-web.

Hypergraph Theory

This includes using hypergraphs to model the system of systems and represent the constraints between subsystems and utilizing hypergraph theory to manage the evolutionary development property in the complicated internal structure of SoS during physical faults.

Graph Theory

This includes applying graph theory and social network analysis to model communications, healthcare and other types of systems-of-systems.

Knowledge and Information Representation

This includes using graphical representations for the organizational modeling of the system of systems.

Dependency and Interconnection Analysis

This includes using Design Structure Matrices (DSM) for analyzing interdependencies of elements within a SoS.

Complex Network Analysis

This includes using complex network theory to Construction of a combat network based on the kill web, which can be represented as a graph with nodes and edges.

Artificial Intelligence Approaches

Finally, a quarter of these papers (36 out of 132, or 26%) applied artificial intelligence (AI) in their SoS analysis approach. The AI approaches used in these papers can be categorized as follows:

Machine Learning Algorithms

These include general machine learning techniques for fault detection and isolation, feature selection, and data processing. An example is the paper: "Federated Feature Selection for Cyber-Physical Systems of Systems" [P. Cassará et al. 2022].

Neural Networks and Deep Learning

This category includes applications to human-machine collaboration and swarm techniques for search and rescue operations with example papers: "A Robotic System of Systems for Human-Robot Collaboration in Search and Rescue Operations" [Chan T. H. et al. 2023] and "A Capability Fitting and Data Reconstruction Model Based on Particle Swarm Optimization-Bidirectional Deep Neural Network for Search and Rescue System of Systems" [Gao, Y. et al. 2023].

Genetic Algorithms

Genetic algorithms are used for mission planning, sensor allocation, and meta-architecture optimization. An example here is "A System of Systems for the Optimal Allocation of Pollutant Monitoring Sensors" [C. Carnevale. C. et al. 2022]

Artificial Intelligent Agents

The concept of Non-Human Knowledge Workers is introduced, which are artificial intelligent agents designed to relieve human knowledge workers of cognitive tasks in the one paper in this category: "A New Technology Rises Non-Human Knowledge Workers and Decision-Making in a System of Complex Systems" [Mortimore D. Et al, 2023]

Fuzzy Logic and Inference Systems

Fuzzy Inference Systems are used to assess the overall fitness measure of the System of Systems (SoS) and to calculate the overall fitness value in a genetic algorithm. An example here is "System of Systems Meta-Architecture Approach to Improve Legacy Metrorails for Enhanced Customer Experience" [Polley, M. et al. 2021].

AI in Autonomous Systems

The papers discuss the use of AI and machine learning in enhancing the level of autonomy in systems, including autonomous vehicles and offshore wind farms, as presented in "Symbiotic System of Systems Design for Safe and Resilient Autonomous Robotics in Offshore Wind Farms" [Mitchell, D. et al. 2021].

References

Works Cited

- Threlkeld, R., Ashiku, L., & Dagli, C. (2022). Complex System Methodology for Meta Architecture Optimization of the Kidney Transplant System of Systems. In 2022 17th Annual System of Systems Engineering Conference (SOSE). IEEE. DOI: 10.1109/SOSE55472.2022.9812668
- Cassará, P. A. Gotta, and L. Valerio, "Federated Feature Selection for Cyber-Physical Systems of Systems," in IEEE Transactions on Vehicular Technology, vol. 71, no. 9, pp. 9937-9950, September 2022, doi: 10.1109/TVT.2022.3178612.
- Chan, T. H., Halim, J. K. D., Tan, K. W., Tang, E., Ang, W. J., Tan, J. Y., Cheong, S., Ho, H.-N., Kuan, B., Shalihan, M., Liu, R., Soh, G. S., Yuen, C., Tan, U.-X., Heng, L., & Foong, S. (2023). A Robotic System of Systems for Human-Robot Collaboration in Search and Rescue Operations. In 2023 IEEE/ASME International Conference on

Advanced Intelligent Mechatronics (AIM) (pp. 878-885). IEEE. DOI: 10.1109/AIM46323.2023.10196185

Gao, Y., Liu, H., Niu, F., & Tian, Y. (2023). A Capability Fitting and Data Reconstruction Model Based on Particle Swarm Optimization-Bidirectional Deep Neural Network for Search and Rescue System of Systems. *IEEE Access*, 11, 10366-10383. doi:10.1109/ACCESS.2023.100083

C. Carnevale, L. Sangiorgi, E. De Angelis, R. Mansini, and M. Volta, "A System of Systems for the Optimal Allocation of Pollutant Monitoring Sensors," in *IEEE Systems Journal*, vol. 16, no. 4, pp. 6393-6400, December 2022.

Dahmann, Judith (2023). "Current Landscape of System of Systems Engineering", IEEE Systems of Systems Conference; Tacoma, Washington.

Mortimore, D., Aten, K., & Buettner, R. R. (2023). A New Technology Rises: Non-Human Knowledge Workers and Decision-Making in a System of Complex Systems. In *Proceedings of the 2023 18th Annual System of Systems Engineering Conference (SoSe)* (pp. 1-10). IEEE. DOI: 10.1109/SOSE59841.2023.10178624

Experience. In *2021 16th International System of Systems Engineering Conference (SoSE)* (pp. 191-196). IEEE. DOI: 10.1109/SOSE52739.2021.9497492

Mitchell, D., Blanche, J., Zaki, O., Roe, J., Kong, L., Harper, S., Robu, V., Lim, T., & Flynn, D. (2021). Symbiotic System of Systems Design for Safe and Resilient Autonomous Robotics in Offshore Wind Farms. *IEEE Access*, 9, 141421-141445. <https://doi.org/10.1109/ACCESS.2021.3095331>

System of Systems and Complexity

- Lead Author:
- Judith Dahmann

Systems of Systems are generally characterized as complex (Sheard, 2019) (Luzeau et al., 2011) (Simpson, 2009) (DeLaurentis, 2007) (Ireland, 2014) (Magee, 2004), as is noted in the systems of systems (SoS) knowledge area of the SEBoK.

The question for those seeking to perform SoS Engineering (SoSE) then is how to address/use SoS complexity. Work has been performed in INCOSE to apply characterization of complexity to SoS. This work looks at how complexity affects SoS (Watson, 2020) and guiding principles to complexity thinking can be applied to SoSE (INCOSE, 2016).

Complexity Dimensions Applied to Systems of Systems

How and why are systems of systems (SoS) characterized by different dimensions of complexity? Drawing on the dimensions of complexity (Watson, 2019) it is clear that by their nature SoS are rich in complexity, as described below. For each complexity dimension, the dimension is defined and the characteristics of SoS which make them subject to this dimension are briefly presented. Diversity "encompasses the structural, behavior, and system state varieties that characterize a system and/or its environment." (Watson, 2019) By their nature, SoS are composed of independent systems. (Maier, 1998) (ISO, 2019) SoS can exhibit tremendous diversity across the constituent systems which provide a range of behaviors, functionality, and technical approaches. SoS are comprised of multiple independent systems with their own users, management structures, requirements etc. often developed prior to their membership in an SoS, increasing the likelihood of diversity among the constituents of a SoS.

Connectivity "characterizes connection of the system between its functions and the environment. This connectivity is characterized by the number of nodes, diversity of node types, number of links, and diversity in link characteristics." (Watson, 2019) SoS include connectivity within each constituent system, among SoS constituents and between the SoS and its environment. Discontinuities (breaks in a pattern of connectivity at one or more layers)

are often found in SoS. This links directly to dimensions of 'Interactivity' and 'disproportionate effects', since connectivity may lead to complex cascading interactions. Adaptability is defined as the characteristics of complex systems which "proactively and/or reactively change function, relationships, and behavior to balance changes in environment and application to achieve system goals." (Watson, 2019) SoS are composed of operationally independent systems [3]; hence the operators of each of system has their own rules of engagement and may react or adapt to changes in different ways based on their local objectives.

Multi-perspective refers to the fact that "multiple perspectives, some of which are orthogonal, are required to comprehend the complex system." (Watson, 2019) SoS are typically comprised of multiple independent systems which were developed and operated prior to the existence of the SoS; hence they each bring with them their own perspectives, which may or may not be aligned with the SoS. An understanding of the SoS considers all of these different perspectives. This links directly to the dimension of 'Multi-scale' since SoS may contain constituent systems of varying scales.

Complex system behavior "cannot be described fully as a response system. Complex system behavior includes nonlinearities. Optimizing system behavior cannot often be done focusing on properties solely within the system." (Watson, 2019) While the behaviors of the individual systems may be predictable, particularly when the numbers of systems are large and have multiple internal behaviors, SoS behavior can become unpredictable. Each constituent has been designed to be operated independently and safely within its own context. without regard to the potential impact on the behavior of itself, on other systems and on overall SoS behavior.

Dynamics in "complex systems may have equilibrium states or may have no equilibrium state. Complex system dynamics have multiple scales or loops. Complex systems can stay within the dynamical system or generate new system states or state transitions due to internal system changes, external environment changes, or both." (Watson, 2019) Following the discussion of behavior, these effects can be dynamic and impact other systems or have feedback loops leading to dynamic complexity. Notably, constituent systems may not only be managed independently, but they may also operate independently, increasing prospects of dynamic complexity.

Representations of "complex systems can be difficult to properly construct with any depth. It is often impossible to predict future configurations, structures, or behaviors of a complex system, given finite resources. Causal & influence networks create a challenge in developing 'requisite' conceptual models within these time and information resource constraints." (Watson, 2019) One feature of SoS is that boundary conditions can be hard to define, which includes not only which constituent systems are members of an SoS, but also which behaviors of a constituent system play a role in the SoS making representation of an SoS a challenge. Evolution is a dimension of complexity as "changes over time in complex system states and structures (physical and behavioral) can result from various causes. Complex system states and structures are likely to change as a result of interactions within the complex system, with the environment, or in application. A complex system can have disequilibrium (i.e., non-steady) states and continue to function." (Watson, 2019) Rarely do we 'develop and field' an SoS, rather SoS are typically composed of existing systems; changes in an SoS result from changes in one or more of the constituent systems (or in the environment), making SoS development an evolutionary process.

System emergence leading to unpredictable behavior is driven by unexpected emergence; "emergent properties of the holistic system unexpected (whether predictable or unpredictable) in the system functionality/response. Unpredictable given finite resources. Behavior not describable as a response system." (Watson, 2019) By definition, SoS are comprised of multiple independent systems. (Maier, 1998) (ISO, 2019). Changes in one system could lead to new behavior in another, leading to unpredictable results. Indeterminate boundaries result from the fact that "complex system boundaries are intricately woven with their environment and other interacting systems. Their boundaries can be non-deterministic. The boundary cannot be distinguished based solely on processes inside the system." (Watson, 2019) One feature of SoS is that boundary conditions can be hard to define, including which constituent systems should be included in representation of an SoS, and which behaviors of a constituent play a role in the SoS.

Guiding Principles to Complexity Thinking Applied in Systems of Systems Engineering

The INCOSE Complexity Primer (INCOSE, 2016) outlines guiding principles to complexity thinking. As SoS exhibit complexity as discussed above, the next question is how these principles might then be applied to SoS.

Think like a gardener, not a watchmaker: “Consider the complexity of the environment and the solution and think about evolving a living solution to the problem rather than constructing a system from scratch.” (INCOSE, 2016) SoS are often composed of current and new systems which support desired SoS capabilities and have evolved through interaction. Understanding these ‘natural’ interaction effects can be important in understanding the responses of constituent systems to various interventions.

Combine courage with humility: “It takes courage to acknowledge complexity, relinquish control, encourage variety, and explore unmapped territory. It takes humility to accept irreducible uncertainty, to be skeptical of existing knowledge, and to be open to learning from failure. A combination of courage and humility enables the complex systems engineer to risk genuine innovation and learn fast from iterative prototyping of solutions in context.” (INCOSE, 2016) This principle aligns with the recognition in SoS, the SoS engineer is moving into new territory where there are large differences between the degree of control, the diversity of system technical and functional capabilities, and multiple overlapping authorities.

Take an adaptive stance: “Systems engineers should mimic how living systems cope with complexity by identifying and creating variation, selecting the best versions, and amplify the fit of the selected versions. This means, for example, to think “influence” and “intervention” rather than “control” and “design.” (INCOSE, 2016) For most SoS, the successful SoS engineer recognizes that influence and intervention are the name of the game, since to a large extent control continues to rest with the constituents, and SoS architecture and design needs to accommodate the state of the constituent systems while addressing SoS capability objectives. Identify and use patterns: “Patterns are exhibited by complex systems, can be observed and understood, and are a key mechanism in the engineering of complex systems. Patterns are the primary means of dealing specifically with emergence and side effects—that is, the means of inducing desired emergence and side effects, and the means of avoiding undesired emergence and side effects.” (INCOSE, 2016) Understanding systems, their behaviors and interactions is a core element of SoSE. By modeling these and treating them as opportunities, patterns can be an effective SoSE approach.

Zoom in and zoom out: Because complex systems cannot be understood at a single scale of analysis, systems engineers must develop the habit of looking at their project at many different scales, by iteratively zooming in and zooming out.” (INCOSE, 2016) Effective SoSE is often called a ‘middle out’ process, where there is a need to understand the top-down drivers for the SoS, but also to respect the bottoms-up needs and capabilities of the constituents. Dynamics between these two perspectives reflects this ‘zoom in and zoom out’ principle in SoSE thinking.

Achieve Balance: “Optimization is often counterproductive within a complex system. Either the whole is sub-optimized when a part is optimized, or an optimized whole becomes rigid, unable to flex with changing conditions. Instead of optimizing, complex systems engineers should seek balance among competing tensions within the project. Systems engineers can leverage integrative thinking to generate improved solutions and avoid binary either/or tradeoffs.” (INCOSE, 2016) In SoSE, if you are trying to ‘optimize’ you probably don’t understand the situation. Multiple, often competing, objectives of the SoS and the constituents, requires options which continue to meet the constituent objectives but also address SoS capabilities. This links directly to both the ‘Collaborate’ and ‘See through new eyes’ principles.

Learn from problems: “In a changing context, with an evolving system, where elements are densely interconnected, problems and opportunities will continually emerge. Moreover, they will emerge in surprising ways, due to phase transitions, cascading failures, fat tailed distributions, and black swan events.” (INCOSE, 2016) SoS development is recognized as evolutionary (Maier, 1998). One life cycle approach, the SoS ‘Wave Model’ (Dahmann, 2011) explicitly sees each iteration of SoS evolution as starting with assessment of changes since the last wave, recognizing

the importance of learning from problems and adapting.

Meta-cognition: “Meta-cognition, or reflecting on how one reflects, helps to identify bias, make useful patterns of thinking more frequent, and improve understanding of a complex situation.” (INCOSE, 2016) One of the SoS pain points (Dahmann, 2010) is the need for “SoS Thinking’ – a form of meta-cognition.

Focus on desired regions of outcome space rather than specifying detailed outcomes: “Instead of zeroing in on an exact solution, focus on what range of solutions will have the desired effects, and design to keep out of forbidden ranges.” (INCOSE, 2016) It is important to define SoS needs in terms of broad capabilities (versus detailed solutions) since there are a larger number of factors to be considered in an SoS. In SoS terminology, the SoS ‘requirements space’ reflects this perspective.

Understand what motivates autonomous agents: “Changing rewards will shape collective behavior. Implement incentives that will move the system toward a more desired state.” (INCOSE, 2016) A core element of SoSE (DoD, 2008) is to understand constituent systems and their relationships including the objectives and long-term goals for these systems. This provides the basis for assessing of changes systems will welcome, and potential for motivating constituents to provide needed SoS functionality and services aligned with their goals. This links directly to the “Use Free order’ principle emphasizing value of promoting self-organization in complex systems.

Maintain adaptive feedback loops: “Adaptive systems correct for output variations via a feedback mechanism. Over time, feedback loops can either hit the limit of their control space or may be removed in the interest of maintaining stability. To maintain robustness, periodically revisit feedback and ensure that adaptation can still occur.” (INCOSE, 2016) As noted above, under the SoS wave model (Dahmann, 2011) is constructed around this principle as applied to the SoS development and evolution life cycle. Integrate problems: “Focus on the relationships among problems rather than addressing each problem separately. This allows fewer solutions that take care of multiple problems in an integrative fashion.” (INCOSE, 2016) In SoS methods (Cook, 2014) including the SoS Wave Model (Dahmann 2010) and the DoD SoS SE Guide, (DoD, 2008) there is an emphasis on understanding the full SoS context, enabling understanding connections, patterns, and opportunities for this type of integrated view.

References

Works Cited

- Cook, S.C. and J.M. Pratt. 2014. “Towards designing innovative SoSE approaches for the Australian defence force.” Proceedings of the 9th International Conference on Systems of Systems Engineering Socio-Technical Perspectives (SoSE 2014). pp. 295–300, 2014.
- Dahmann, J., G. Rebovich, J. Lane, R. Lowry, and K. Baldwin. 2011. “An Implementer's View of Systems Engineering for Systems of Systems.” IEEE Systems Conference, April 4-7, 2011, Montreal, Canada. p. 212-217.
- Dahmann, J. 2015. “Systems of Systems Pain Points”. International Council on Systems Engineering (INCOSE) International Symposium, 2015, Seattle, WA.
- DoD. 2008. *Systems Engineering Guide for Systems of Systems*, version 1.0. Washington, DC, USA: US Department of Defense (DoD). Accessed 2 /26/2022. Available: [https:// acqnotes. com/ wp-content/ uploads/ 2014/ 09/ DoD-Systems-Engineering-Guide-for-Systems-of-Systems-Aug-2008.pdf](https://acqnotes.com/wp-content/uploads/2014/09/DoD-Systems-Engineering-Guide-for-Systems-of-Systems-Aug-2008.pdf).
- DeLaurentis, D. 2007. “Role of Humans in Complexity of Systems of Systems.” In V.G. Duffy (Ed.): Digital Human Modeling, HCII 2007, LNCS 4561, pp. 363–371. Springer-Verlag Berlin Heidelberg.
- INCOSE. 2016. *A Complexity Primer for Systems Engineers*. TP-2016-001-01.0.
- INCOSE, 2018. *Systems of Systems Primer*, INCOSE-TP-2018-003-01.0.
- INCOSE, 2020. *Systems of Systems Standards Quick reference Guide*, INCOSE -2020 -sosstandards.

- Ireland, V. 2014. "SoS Benefiting from Complex Systems Research.in Complex Adaptive Systems, Publication 4. Cihan Dagli (Ed) Conference Organized by Missouri University of Science and Technology. Philadelphia, PA.
- International Organization for Standardization (ISO). 2019. *ISO/IEC/IEEE 21839 — Systems and Software Engineering—System of systems considerations in life cycle stages of a system*.
- Luzeau, D, J-R Rualt, and J-L Wippler (Eds). 2011. *Complex Systems and Systems of Systems Engineering*. ISTE Ltd, and John Wiley and Sons. Great Britain and UAS.
- Magee, C.I. and O.L. deWeck. "Complex System Classification." International Council on Systems Engineering (INCOSE) International Symposium, 20–24 June 2004, Toulouse, France,
- Maier, M.W. 1998. "Architecting Principles for Systems-of-Systems." *Systems Engineering*. 1 (4): 267-284.
- Sheard, S. 2019. "Complexity in a Systems of Systems Context." International Council on Systems Engineering (INCOSE) International Symposium, 20-25 July 2018, Orlando, FL.
- Simpson J. J. and M. J. Simpson.2009. "System of systems complexity identification and control," 2009 IEEE International Conference on System of Systems Engineering (SoSE), pp. 1-6.
- Watson, M., R. Anway, D. McKinney, L.A. Rosser, and J. MacCarthy. 2019. "Appreciative Methods Applied to the Assessment of Complex Systems." International Council on Systems Engineering (INCOSE) International Symposium, 20-25 July 2018, Orlando, FL.

Primary References

- Watson, M., R. Anway, D. McKinney, L.A. Rosser, and J. MacCarthy. 2019. "Appreciative Methods Applied to the Assessment of Complex Systems." International Council on Systems Engineering (INCOSE) International Symposium, 20-25 July 2018, Orlando, FL.
- INCOSE. 2021. *A Complexity Primer for Systems Engineers*. TP-2016-001-01.0.

Additional References

None.

Socio-Technical Features of Systems of Systems

- Lead Authors:
- Judith Dahmann, Mike Henshaw, and Bud Lawson
- Contributing Authors:
- Heidi Davidz and Alan Faisandier

In perhaps the earliest reference to Systems of Systems (SoS), Ackoff (1971) describes a concept that is mostly concerned with organizations, i.e. social. However, this section is concerned with the socio-technical aspects of technical SoS, which are composed of interdependent resources, such as, people, processes, information, and technology that interact with each other and with their environment in support of a common mission (glossary).

The Socio-Technical Nature of Systems of Systems

Rebovich (2009) [has captured the essence of the SoS problem as:

“From a single-system community’s perspective, its part of the SoS capability represents additional obligations, constraints and complexities. Rarely is participation in an (sic) SoS seen as a net gain from the viewpoint of single-system stakeholders.”

Three of the persistent SoS challenges, or pain points, identified by Dahmann (2015) are directly related to this problem of stakeholder perspective and the local optimization of constituent system performance at the expense, or to the detriment of, the overall SoS performance. These are: SoS Authority, Leadership, and Autonomy, Interdependencies & Emergence. Thus, the sociological aspects affecting decision making and human behaviors must be given similar weight to the technical aspects of SoS.

Turning to views outside of Systems Engineering, Ergonomists regard socio-technical systems as having the following characteristics (Maguire, 2014):

- There are collective operational tasks,
- They contain social and technical sub-systems,
- They are open systems (i.e. strongly interacting with their environments), and
- The concept of the system being an unfinished system.

These are also characteristics of Systems of Systems. Klein (2014) has noted that approaches to socio-technical systems can take the two perspectives of “system affects people” or “people affect system”, depending upon how the system boundary is drawn. It is generally true for systems that consideration of their context requires socio-technical aspects to be taken into account.

Although focused largely on IT systems, Baxter and Sommerville (2011) have noted that the introduction of new business SoS are generally carried out in conjunction with a change process. They argue that frequently the social and organizational aspects are disruptive and that inadequate attention is paid to the connection between change processes and systems development processes. They propose two types of Socio-Technical Systems Engineering activities:

- Sensitizing and awareness activities, designed to sensitize stakeholders to the concerns of other stakeholders.
- Constructive engagement activities, which are largely concerned with deriving requirements accurately and meaningfully.

The extent to which these activities can be effective may be challenged by independent management or operation of constituent systems in a SoS.

Although there are many matters concerning the socio-technical aspects of SoS, there are two important issues, that are dealt with her. The first is the need for appropriate governance structures, given that operational and/or managerial independence affects top-down direction of the SoS and may compromise achievement of the SoS

goal(s). The second issue is a lack of situational awareness of managers, operators, or other stakeholders of the SoS, so that they may not understand the impact of their local decisions on the wider SoS.

SoS Governance

Generally, design and operation of complex systems is concerned with control, but the classification of SoS (Dahmann, et. al., 2008) is based on the notion of diminishing central control, as the types go from directed to virtual. Sauser, et. al. (2009) has described the ‘control paradox of SoS’ and asserted that for SoS, ‘management’ is replaced by ‘governance’. ‘Control is a function of rules, time, and bandwidth; whereas command is a function of trusts, influence, fidelity, and agility’.

Some practitioners have found the Cynefin framework, developed by David Snowden, helpful in understanding the nature of complexity that may arise in SoS. Developed from knowledge management considerations, Kurtz and Snowden (2003) propose three reasons why the behavior of systems involving people may be difficult to predict. Firstly, humans are not limited to one identity, and so modelling human behaviors using norms may not be reliable. Secondly, humans are not limited to acting in accordance with predetermined rules. Thirdly, humans are not limited to acting on local patterns. These reasons all undermine control, so that the sociological aspects of SoS make their behaviours hard to predict and, possibly indeterminate. The Cynefin framework considers systems to be classified in four domains:

- Known – simple systems with predictable and repeatable cause and effect
- Knowable – amenable to systems thinking and analytical/reductionist methods
- Complex – adaptive systems where cause and effect are only discernable in retrospect and do not repeat
- Chaotic – no cause and effect relationships are perceivable

The different types of SoS (directed, acknowledged, collaborative, and virtual) could all be described in any of the above domains, depending on many factors internal to the SoS, but in all cases it is the sociological element of the socio-technical SoS that is most likely to give rise to ambiguity in predicting behavior.

A major governance issue for SoS is understanding the ownership of, and making reliable estimates of risk (Fovino & Masera, 2007). High levels of connectivity, and the potential for emergent behavior due to the interactions of separately owned/operated constituent systems, means that significant risks may go unacknowledged and their mitigations unplanned.

In general, governance can be summed up by asking three connected questions (Siemieniuch and Sinclair, 2014):

- Are we doing the right things (leadership)?
- Are we doing those things right (management)?
- How do we know this (metrics and measurements)?

Currently, there is no accepted framework for addressing these questions in a SoS context, but Henshaw et. al. (2013) highlighted architectures as an important means through which governance may be clarified. They postulate that a SoS can be regarded as a set of trust and contract relationships between systems (i.e. including both informal and formal relationships). The systems architect of a constituent system must, therefore, address trust issues for each participating organization in the overall enterprise with which his/her system must interoperate. For SoS, technical engineering governance is concerned with defining and ensuring compliance with trust at the interface between constituent systems. An example of difficulty managing the interfaces in a SoS is provided in the Cassini-Huygens mission case study .

Situational Awareness

Situational awareness is a decision maker's understanding of the environment in which he/she takes a decision; it concerns information, awareness, perception, and cognition. Endsley (1995) emphasizes that situational awareness is a state of knowledge. There are numerous examples of SoS failure due to the operator of one constituent system making decisions based on inadequate knowledge of the overall SoS (big picture).

On the other hand, SoS development is also viewed as the means through which improved situational awareness may be achieved (Van der Laar, et. al., 2013). In the defense environment, Network Enabled Capability (NEC) was a system of systems approach motivated by the objective of making better use of information sharing to achieve military objectives. NEC was predicated on the ability to share useful information effectively among the stakeholders that need it. It is concluded that improving situational awareness will improve SoS performance, or at least reduce the risk of failures at the SoS level. Thus, the principles which govern the organization of the SoS should support sharing information effectively across the network; in essence, ensuring that every level of the interoperability spectrum is adequately serviced. Operators need insight into the effect that their own local decisions may have on the changing SoS or environment; similarly they need to understand how external changes will affect the systems that they own.

Increasingly, SoS include constituent systems with high levels of autonomous decision making ability, a class of system that can be described as cyber-physical systems (of systems). The relationship to SoS is described by Henshaw (2016). Issues arise because autonomy can degrade human situational awareness regarding the behavior of the SoS, and also the autonomous systems within the SoS have inadequate situational awareness due to a lack of competent models of humans (Sowe, 2016)

References

Works Cited

- Ackoff, R.L.(1971) "Towards a Systems of Systems Concepts," *Manage. Sci.*, vol. 17, no. 11, pp. 661–671.
- Baxter, G. and I. Sommerville, (2011) "Socio-technical systems: From design methods to systems engineering," *Interact. Comput.*, vol. 23, no. 1, pp. 4–17.
- Dahmann, J. S. & Baldwin, K. J. (2008) Understanding the Current State of US Defense Systems of Systems and the Implications for Systems Engineering, 2nd Annual IEEE Systems Conference, 1–7. <https://doi.org/10.1109/SYSTEMS.2008.4518994>
- Dahmann, J.S. (2015) "Systems of Systems Characterization and Types," in *Systems of Systems Engineering for NATO Defence Applications (STO-EN-SCI-276)*, pp. 1–14.]
- Endsley, M. R. (1995) Toward a Theory of Situation Awareness in Dynamic Systems, *J. Human Factors and Ergonomics Soc.*, 37(1), 32–64. <https://doi.org/10.1518/001872095779049543>
- Fovino, I. N., & Masera, M. (2007) Emergent disservices in interdependent systems and system-of-systems, in *Proc. IEEE International Conference on Systems, Man and Cybernetics*, Vol. 1, pp. 590–595. <https://doi.org/10.1109/ICSMC.2006.384449>
- Henshaw, M. J. de C., Siemieniuch, C. E., & Sinclair, M. A. (2013) Technical and Engineering Governance in the Context of Systems of Systems, in *NATO SCI Symp. Architecture Assessment for NEC* (pp. 1–10). Tallinn, Es. NATO STO.
- Henshaw, M. (2014) A Socio-Technical Perspective on SoSE, in *Lecture Series in Systems of Systems Engineering for NATO Defence Applications (SCI-276)*. NATO CSO.
- Henshaw, M. (2016). *Systems of Systems, Cyber-Physical Systems, The Internet-of-Things...Whatever Next?* INSIGHT, 19(3), pp.51–54.

- Klein, L. (2014) What do we actually mean by 'sociotechnical'? On values, boundaries and the problems of language, *Appl. Ergon.*, vol. 45, no. 2 PA, pp. 137–142.
- Kurtz, C.F. and D. J. Snowden (2003) "The New Dynamics of Strategy: Sense-making in a Complex-Complicated World," *IBM Syst. J.*, vol. 42, no. 3, pp. 462–483.
- Maguire, M. (2014) Socio-technical systems and interaction design - 21st century relevance, *Appl. Ergon.*, vol. 45, no. 2 PA, pp. 162–170.:/mil/ ^[1]
- Rebovich, G. (2009) "Enterprise systems of Systems," in *Systems of Systems Engineering - Principles and Applications*, M. Jamshidi, Ed. Boca Raton: CRC Press, pp. 165–191.
- Sauser, B., Boardman, J., & Gorod, A. (2009) System of Systems Management, in *System of Systems Engineering: Innovations for the 21st Century*, M. Jamshidi (Ed.), (pp. 191–217) Wiley.
- Siemieniuch, C.E. & Sinclair, M.A. (2014) Extending systems ergonomics thinking to accommodate the socio-technical issues of Systems of Systems, *Appl. Ergon.*, V 45, Issue 1, Pages 85-98
- Sowe, S.K. et al. (2016) Cyber-Physical-Human Systems - putting people in the loop. *IT Professional*, 18(February), pp.10–13.]
- Van der Laar, P., Tretmans, J., & Borth, M. (2013) *Situational Awareness with Systems of Systems*. Springer.

Primary References

- Checkland, P.B. 1981. *Systems Thinking, Systems Practice*. Chichester, West Sussex, England, UK: John Wiley & Sons, Ltd.

Additional References

- Bruesburg, A., and G. Fletcher. 2009. *The Human View Handbook for MODAF*, draft version 2, second issue. Bristol, England, UK: Systems Engineering & Assessment Ltd. Available: <https://www.hfidtc.com/research/process/reports/phase-2/hv-handbook-issue2-draft.pdf>.
- IFIP-IFAC Task Force. 1999. "The Generalised Enterprise Reference Architecture and Methodology," V1.6.3. Available: <https://www.cit.gu.edu.au/~bernus/taskforce/geram/versions/geram1-6-3/v1.6.3.html>.
- ISO. 1998. ISO 14258:1998, *Industrial automation systems — Concepts and rules for enterprise models*. Geneva, Switzerland: International Organization for Standardization.
- ISO. 2006. ISO 19439:2006, *Enterprise integration — Framework for enterprise modelling*. Geneva, Switzerland: International Organization for Standardization.
- ISO. 2007. ISO 19440:2007, *Enterprise integration — Constructs for enterprise modelling*. Geneva, Switzerland: International Organization for Standardization.
- Miller, F.P., A.F. Vandome, and J. McBrewster. 2009. *Enterprise Modelling*. Mauritius: Alphascript Publishing, VDM Verlag Dr. Müller GmbH & Co. KG.

References

- [1] <https://www.dtic.mil/dtic/tr/fulltext/u2/a468785.pdf>

Capability Engineering

- Lead Authors:
- Duncan Kemp, Judith Dahmann, and Mike Henshaw

Capability has been used to describe the Systems Engineering of Operational Capabilities. The INCOSE UK Capability System Engineering Guide (Kemp and Daw) built on this analysis and describes:

- That Capabilities are realized through a combination of people, processes, information as well as equipment;
- They are concerned with delivering outcomes, rather than outputs;
- They are enduring, with capabilities being upgraded rather than replaced; The term emerged in defense in the early 2000s; however, the concepts go back far earlier (Checkland, 1997).
- The concepts of Capability Systems Engineering have been used in Rail (Dogan, 2012) and Healthcare (Royal Academy of Engineering, 2017).

Capability is widely used across many industrial sectors and has begun to take on various specific meanings across, and even within, those sectors. Terms such as capability-based acquisition, capability engineering and management, life capability management, capability sponsor, etc. are now ubiquitous in defense and elsewhere. Henshaw et al. (2011) have identified at least eight worldviews of capability and capability engineering and concluded that the task of capability engineering is not consistently defined across the different communities.

The aim of capability systems engineering is to ensure that the upgraded capability meets stakeholders needs. Good Capability Systems Engineering provides a clear line of sight from the purpose of the capability, through the operational concept and whole system design down to specific requirements and interfaces.

Capability engineering is concerned with the whole lifecycle; the “Fuzzy front end” of capability trade-offs, the conventional ‘V’ product lifecycle, and the “Messy in-service” support phase.

Capability Systems Engineering uses standard SE tools, applied from the perspective of the asset owner-operators (i.e. the military user or rail transportation provider).

Kemp and Daw (2014) note several differences between Capability Systems Engineering and the more traditional product Systems Engineering:

- Using persuasion and influence as much as command and control to implement decisions
- Building in flexibility where possible, as the capability will change.
- Implementing the transition to the improved capability as both an engineering and cultural change.
- Recognizing that capabilities are often Complex Adaptive Systems. As the capability improves, users or competitors change their behavior, reducing the effectiveness of the capability.
- Capability trade-offs are not about simple comparisons, between similar things – often they are choices between new equipment, better training, or new processes.

There is a strong relationship between Capability Engineering and system of systems (SoS). To some a Capability is a type of system/SoS, to others it is what the system/SoS does. This is explored in Henshaw et al. (2011), who describe at least eight worldviews of capability and capability engineering

References

Works Cited

Henshaw, M., D. Kemp, P. Lister, A. Daw, A. Harding, A. Farncombe, and M. Touchin. 2011. "Capability Engineering - An Analysis of Perspectives." Presented at International Council on Systems Engineering (INCOSE) 21st International Symposium, June 20-23, 2011, Denver, CO, USA.

Checkland P. and Holwell, S, 1997, Information, Systems and Information Systems: Making Sense of the Field

Dogan, H., Henshaw, M. and Johnson, J., 2012. An incremental hybridisation of heterogeneous case studies to develop an ontology for Capability Engineering. In: INCOSE, ed. The 22nd Annual INCOSE International Symposium 9-12 July 2012 Rome, Italy.

Royal Academy of Engineering, 2017, Engineering better care a systems approach to health and care design and continuous improvement

Erl, T. 2008. *SOA Principles of Service Design*. Boston, MA, USA: Prentice Hall Pearson Education.

Hitchins, D.K. 2003. *Advanced Systems Thinking, Engineering and Management*. Norwood, MA, USA: Artech House, Inc.

OGC (Office of Government Commerce). 2009. *ITIL Lifecycle Publication Suite Books*. London, UK: The Stationery Office.

Primary References

Henshaw, M., D. Kemp, P. Lister, A. Daw, A. Harding, A. Farncombe, and M. Touchin. 2011. "Capability Engineering - An Analysis of Perspectives." Presented at International Council on Systems Engineering (INCOSE) 21st International Symposium, June 20-23, 2011, Denver, CO, USA.

Kemp D., Daw A. 2014, INCOSE UK Capability Systems Engineering Guide

Additional References

Davies, J.K. 2011. *Survey of Background Literature for Capability Engineering*. INCOSE UK Capability Working Group Report.

Mission Engineering

- Lead Author:
- Judith Dahmann
- Contributing Authors:
- Ron Giachetti, Andy Hernandez, and Rhys Kissell

This article describes mission engineering, especially as defined by the United States Department of Defense (US DoD). The article defines mission engineering and describes the systems engineering activities involved in mission engineering.

Definition of Mission Engineering

Mission engineering is an interdisciplinary process encompassing the entire technical effort to analyze, design, and integrate current and emerging operational needs and capabilities to achieve desired mission outcomes. In mission engineering, the mission itself becomes a system of interest. The US DoD's Mission Engineering Guide (2023) states:

The mission engineering process decomposes missions into constituent parts to explore and assess relationships and impacts in executing the end-to-end mission.

System engineering focuses on designing systems (including SoS) to achieve specified technical performance metrics. Mission engineering goes one step further to evaluate the performance of the SoS in achieving the mission or capability objectives when implemented in a simulated realistic scenario. Mission engineering evaluates whether the SoS achieves the expected or desired effects and determines whether those effects contribute to mission success.

Goal of Mission Engineering

The goal of mission engineering is to engineer missions by identifying the right things (i.e., technologies, systems, SoS, or processes) to achieve the intended mission outcomes and provide mission-based inputs into the systems engineering process to aid the Department in building things right. [US DoD, 2023]

The US DoD developed the first guidance on mission engineering, and there is a growing body of practice in mission engineering competencies, education and methods (Hutchison et al., 2018) (Von Bossuyt et al, 2019) (Beam, 2015). Zimmerman and Dahmann (2018) reviewed the challenges that face mission engineering and presented the case for the use of digital engineering to support mission engineering, as well as the engineering of systems, which was the initial focus of the US DoD Digital Engineering Strategy (Dahmann, 2019). Use of systems modelling language (SysML) to develop mission architecture models that support mission engineering practices is now considered good practice (Dahmann and Parasidis, 2024).

Key Concepts in Mission Engineering

Key concepts in mission engineering are:

Mission: The “task, together with the purpose, that clearly indicates the action to be taken and the reason thereby. More simply, a mission is a duty assigned to an individual or unit” [US DoD, 2011].

Mission Thread (MT): The “activities of a given mission approach” [US DOD, 2023]

Mission Engineering Thread (MET): “How the mission activities related to the actors, systems, and organizations are executed in a specific mission context” [US DOD, 2023]. METs are equivalent in concept to kill chains or effects chains but are formalized representations of mission execution, used to model system and actor interactions in a specific operational context.

Mission Architecture: “An interwoven effects web, or kill web, comprised of many mission threads and mission engineering threads” [US DoD, 2023]

Implementing Mission Engineering

The US DoD Mission Engineering Guidebook 2.0 lays out a methodology for implementing mission engineering as shown below.

Mission Engineering Methodology from the DoD MEG 2.0 (US DoD,2023)

The key activities in implementing mission engineering are (Dahmann and Parasidis, 2024):

Mission Problem Definition: ME starts with defining and understanding the problem to be addressed, which drives the focus and scope of the ME and analysis activity. ME problems may be based on a proactive interest in the mission outcomes of a priority or pressing scenario, a reaction to a perceived problem in mission results or by an interest in the potential for a new or emerging technology or new operational concept to impact the success of the mission.

Mission Characterization: Mission characterization is a description of the operational mission context for the problem of interest including factors such as the epoch, physical environment, threat, blue force mission objectives, force laydown and CONOPs, as well as the objectives of the mission, which are the measure of mission outcomes.

Mission Architecture: Data extracted from mission characterization drives a model of baseline architecture including the baseline MT and METs. This is used as the blueprint for operational analysis of the impact of the baseline architecture on mission outcomes. Once the baseline operational analysis is complete and gaps in mission outcomes have been identified, the mission architecture models are updated to incorporate alternative approaches and concepts to be considered in the analysis.

Mission Engineering Analysis: The mission impact of the baseline architecture is analyzed in an appropriate analysis environment, typically an operational analysis simulation to assess the mission outcomes of the baseline in the selected scenario and vignette. The analysis is then run with the changes that represent the selected alternative concepts or capabilities to assess the mission impact of the alternative architectures.

Results and Recommendations: Using the results of the ME analysis, comparing the baseline mission outcomes with the incorporated outcomes of alternative concepts, provides the base for results and recommendations.

As stated in the DoD ME Guidebook,” [t]he results of mission engineering are used for a variety of purposes. For instance, findings can inform technology investments, suggest alternative ways to use current systems, identify mission gaps and preferred approaches to addressing these gaps, and trigger the initiation of a new acquisition to meet capability gaps.” [US DoD, 2023].

Mission engineering continues to evolve as a critical capability within defense systems development, enabling the integration of complex capabilities around mission outcomes rather than isolated system performance with potential application beyond defense.

References

Works Cited

- Beam, D.F. 2015. *Systems engineering and integration as a foundation for mission engineering*. Monterey, CA, USA: Naval Postgraduate School.
- Beery, P., E. Paulo. 2019. "Application of Model-Based Systems Engineering Concepts to Support Mission Engineering." *IEEE Systems*. 7(3): 44.
- Dahmann, J. Keynote Address: "Mission engineering: System of systems engineering in context." Proceedings of the IEEE System of Systems Engineering Conference, 19–22 May 2019, Anchorage, AK, USA.
- Dahmann, J, and G. Parasidis. 2024. Mission Engineering. *ITEA Journal*. 24(3).
- Dahmann, J. 2024. Mission Engineering – Extending Systems of Systems Engineering to Mission. 34th Annual INCOSE International Symposium, July. Dublin Ireland.
- Department of Defense. 2023. Department of Defense Mission Engineering Guide Version 2.0. Office of the Under Secretary of Defense for Research and Engineering, https://ac.cto.mil/wp-content/uploads/2023/11/MEG_2_Oct2023.pdf
- Giachetti, R., S. Wangert, R. Eldred. 2019. "Interoperability analysis method for mission-oriented system of systems engineering". Proceedings of IEEE International Systems Conference (SysCon), Orlando, FL, USA, 8-11 April 2019, pp. 1-6.
- Hutchison, N.A.C., S. Luna, W.D. Miller, H.Y. See Tao, D. Verma, G. Vesonder, and J. Wade. 2018. "Mission engineering competencies." Proceedings of the American Society for Engineering Education (ASEE) Annual Conference and Exposition, vol. 2018.
- Kipling, R. "The Elephant's Child, Just So Stories". 1902.
- US Department of Defense. Mission Engineering Guide. November 2020.
- US Department of Defense. Mission Engineering Guide 2.0. October 2023, pp. 3.
- US Department of Defense. Joint Publication 3-0 Joint Operations. 11 August 2011.
- Van Bossuyt, D.L., P. Beery, B.M. O'Halloran, A. Hernandez, E. Paulo. 2019. "The Naval Postgraduate School's Department of Systems Engineering approach to mission engineering education through capstone projects." *IEEE Systems* 7(3): 38.
- Zimmerman, P and J Dahmann. Digital Engineering Support to Mission Engineering. 21st Annual National Defense Industrial Association Systems and Mission Engineering Conference. October 2018.

Primary References

None.

Additional References

None.

Knowledge Area: Healthcare Systems Engineering

Healthcare Systems Engineering

Contents of this Knowledge Area

- Overview of the Healthcare Sector (Chris Unger) (Cyrus Hillsman)
- Systems Engineering in Healthcare Delivery (Cyrus Hillsman) (Chris Unger and Nicole Hutchison)
- Systems Biology (Bridgette Daniel Allegro and Gary Smith) (Chris Unger and Nicole Hutchison)
- Lean in Healthcare (Bohdan Oppenheim) (Chris Unger and Nicole Hutchison)

This article provides an overview of the role of systems engineering in the engineering or re-engineering of healthcare systems to meet a number of modern day challenges. The role of SE in medical devices, healthcare IT, pharmaceuticals, and public health systems are considered and contrasted to "traditional" SE practices discussed elsewhere in the SEBoK. See Overview of the Healthcare Sector for details of the stakeholders and constraints of these different parts of the sector.

Healthcare and Systems Engineering

Healthcare today faces many challenges related to safety (e.g. Hospital Safety Score 2013, Andel et al. 2012, Institute of Medicine 1999), affordability, access, and the means for reliably producing positive outcomes for all patients of all ages and across all care environments. Furthermore, the health of individuals is challenged by many threats such as environmental and behavioral norms, emerging natural infectious diseases, and acute and chronic conditions that are becoming more prevalent because of longer lifespans. Re-engineering today's healthcare to address these challenges requires a systems approach – an approach that develops solutions to contend with the complexity of healthcare-related policy, economics, social dynamics, and technology. Systems Engineers are trained to grapple with this kind of complexity by thinking holistically and to work with trans-disciplinary teams to develop solutions making re-engineering healthcare a natural fit for systems engineers and the tools of systems engineering.

The disciplines involved in re-engineering healthcare are far reaching across academia, government, industry, private, and public sectors including the patients and families the healthcare field serves. Systems Engineers involved in this re-engineering draw on several tools when working with these stakeholders to develop solutions. In doing so they follow the general systems principles described in the Systems Approach Applied to Engineered Systems knowledge area in SEBoK Part 2. First, with so many diverse stakeholders involved in this field, it is vitally important for the Systems Engineer to help clarify the problem or opportunity and to conceive of the objective of the re-engineering. They need to "envision the solution" without being entirely prescriptive of the solution's specific implementation, see Identifying and Understanding Problems and Opportunities. Then, drawing from best practices, the Systems Engineer guides the stakeholders through the synthesis of possible solutions and the analysis and selection between alternatives. Systems engineers are also involved in the implementation and testing and the deployment, use and sustainment of healthcare systems to provide stakeholder value. The systems approach in healthcare must be particularly mindful to not exclusively focused on technical aspects of the effort since the solutions to healthcare's challenges exist not only in technical areas but the integration of culture, workflow and processes, with technology as a tool to support the delivery of safe, affordable, and accessible care.

To achieve this system approach healthcare projects follow a version of the SE life cycle described in SEBoK Part 3. This included the creation of Stakeholder and System Requirements, Systems Architecture and Design and System

Integration, Verification and Validation. The SE life cycle extends to include System Deployment, Operation, Maintenance and Logistics. Healthcare project will also follow some of the Technical Management Processes processes described in Part 3.

It is vitally important for the healthcare systems engineer to ensure socio-technical integration and interoperability among system components are part of any project – the last thing healthcare needs is another standalone innovation that perpetuates the silos that exist in the field today. Remaining focused on the objective and problem to solve, managing scope creep, disciplined design, implementation, and project management are key activities the Systems Engineer is responsible for in healthcare systems engineering.

Systems Engineering for Medical Device Development

Systems Engineering for medical device development is essentially an application of Product Systems Engineering as described in SEBoK Part 4 with a few customizations:

- The life cycle has to comply with specific healthcare regulations, which constrain aspects of the life cycle, as exemplified by FDA regulations in the US (21CFR 820.30)
- The products are market driven, with little customization allowed by the manufacturer at the customer site
- The markets are midsized, with the market for a given technology or product line often being in the \$1-10B range
- Medical device development programs are mid-sized...many from 10-100 man-years of development, lasting 1-2 years
- Time to market is critical, with the first mover or first with a complete solution capturing the majority of the profits
- Most products are cyber-physical, with software becoming a larger part of the product. Many products include significant aspects of physiology or chemistry
- There is a special tension between “efficacy” and “safety”. Efficacy requires the vast majority to be helped. Safety is compromised if only a very small minority is adversely affected. Truly safe systems require a special approach to systems engineering . (Leveson 2011)
- Customer feedback may be constrained by safety issues as well as HIPAA regulations

Device Development in a Market Environment

One critical difference between many “traditional” systems engineering industries (defense and aerospace) and healthcare device development is that most healthcare device development is market driven, rather than contract driven. Some key differences between market and contract systems engineering:

- The program size (budget) and dates are not ‘fixed’, they are set by the business leadership designed to maximize return on investment across a portfolio of product programs
 - Program scope and requirements are not fixed externally; they can be changed fairly rapidly by negotiation between functions and the executive committee.
 - The goal for the product development isn’t necessarily a feature set, it is a market share and price premium relative to the competition...which can be a moving target. A competitive announcement will often force a change in the program scope
 - In a contract based program there is an identified customer, with a set of applications and workflows. In a market driven program the workflow and use cases are defined by the developer, and the buyer needs to ‘own’ the integration of the offering into their specific systems and workflows.
 - For specific medical products the FDA can require pre-market trials and post market studies . (FDA 2014)
 - The different types of healthcare reimbursement across the world (universal coverage private insurance, national single provider, national single payer, private insurance, and out of pocket) creates dramatically different market dynamics (for individuals, healthcare providers, and product developers) . (Reid 2010)
-

Regulations for Medical Device Development

As with all regulated products, there are many regulations governing the development of medical devices. The medical device industry specific regulations are primarily driven by the US (FDA), Europe (European Commission), and Canada (Health Canada). Within the US, the FDA governs medical devices primarily through 21 CFR 820.30 (Quality Systems Regulation, Subpart C Design Controls), which contains requirements similar to ISO 13485. The sections of the Quality Systems Regulation for Design Controls can be mapped fairly directly to ISO/IEC/IEEE 15288 (2015) and the INCOSE *SE Handbook* (INCOSE 2015).

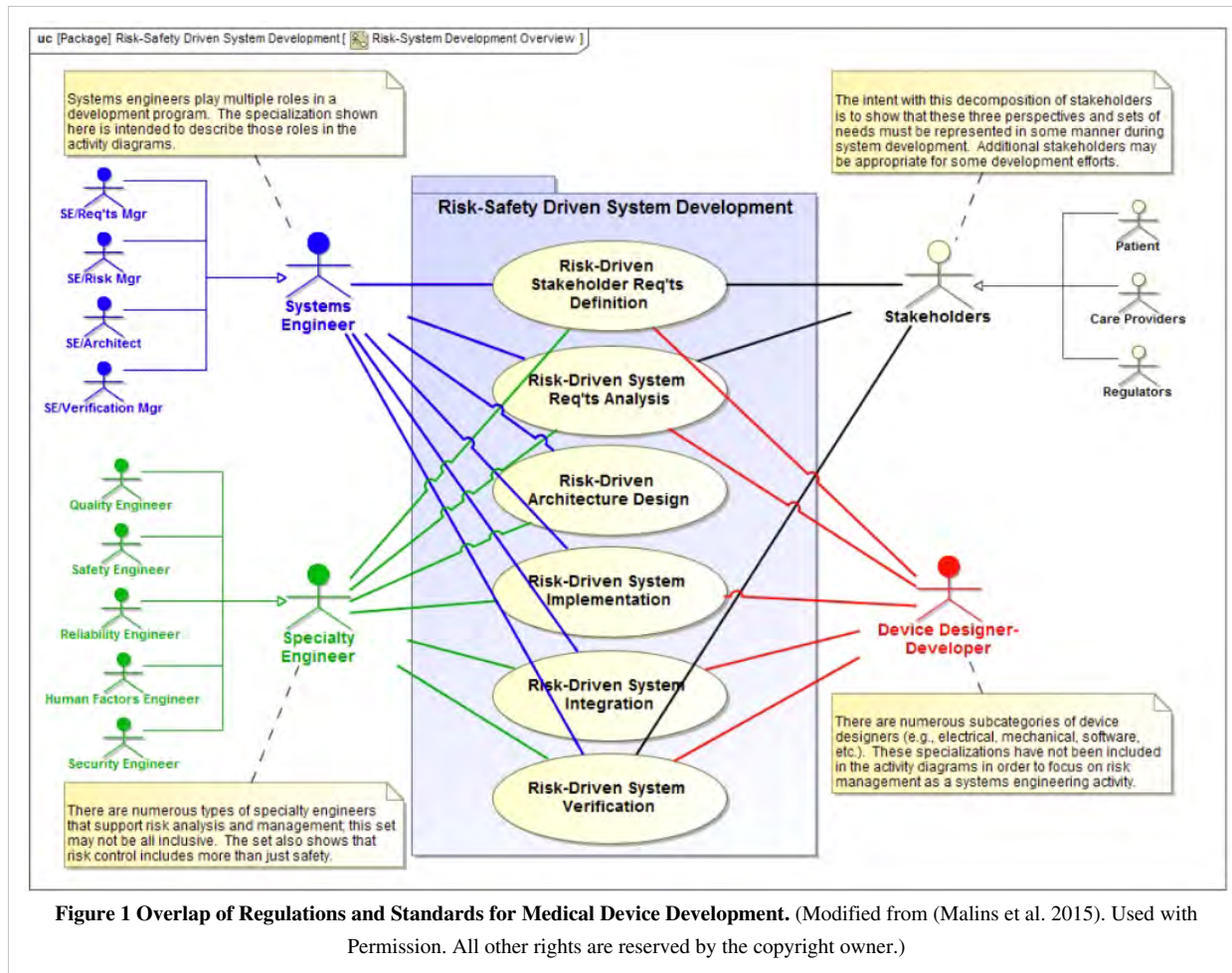
Table 1. Comparison of Healthcare Safety Regulations with ISO/IEC/IEEE 15288 and the INCOSE *SE Handbook*.

21CFR820.30	ISO/IEC/IEEE 15288:2015	INCOSE <i>SE Handbook</i> v4 (2015)
(b) Design and development planning	6.3.1 Project Planning Process	5.1 Project Planning Process
(c) Design input.	6.4.2 Stakeholder needs and requirements definition process 6.4.3 Systems requirements definition process	4.2 Stakeholder needs and requirements definition process 4.3 Systems requirements definition process
(d) Design output	6.4.5 Design definition process 6.4.7 Implementation process	4.5 Design definition process 4.7 Implementation process
(e) Design review	6.3.2 Project Assessment and Control process	5.2 Project Assessment and Control process
(f) Design verification	6.4.9 Verification Process	4.9 Verification Process
(g) Design validation	6.4.11 Validation Process	4.11 Validation Process
(h) Design transfer	6.4.10 Transition Process	4.10 Transition Process
(i) Design changes	6.3.5 Configuration Management Process 6.4.13 Maintenance Process	5.5 Configuration Management Process 4.13 Maintenance Process
(j) Design history file	6.2.6 Knowledge Management Process	5.6 Information Management Process

In the biomedical and healthcare environment, an important differentiator in Risk Management activities compared to other industries (see Risk Management) is that the users and patients are the center of risk analysis rather than technical or business risks. Risk management is an important element of the design control process, as preliminary hazard analysis drive initial design inputs. Traceability between identified risks, risk mitigations, design inputs, and design outputs is a key factor in product clearance through regulatory agencies. Most regulatory bodies have recognized ISO 14971: Medical devices -- Application of risk management to medical devices as a methodology for assessing and documenting product safety and effectiveness.

Usability Engineering is an important subset of risk management activities. ISO 62366-1 Medical devices – Part 1: Application of usability engineering to medical devices provides a “process for a manufacturer to analyze, specify, develop and evaluate the usability of a medical device as it relates to safety. This usability engineering (human factors engineering) process permits the manufacturer to assess and mitigate risks associated with correct use and use errors, i.e., normal use.”. (IEC 62366-2015) For example, for a device designed for home care use, there are many complex interfaces that product designers must consider. Patients may be physically or cognitively affected (age, medication, injury, etc.); they may be untrained or cared for people who are untrained; they are not professionals used to technical systems, etc. Even in the hospital setting, untrained patients may have physical access to systems. This puts a critical focus on usability and human factors considerations and the complexity of the use environment.

Further, as medical devices incorporate more software and become cyber-physical devices, the regulators are also focusing on privacy and security (ISO 21827) and software life cycle management (ISO 62304).



Medical Device regulations, guidance, and technical standards are constantly changing, adding a complex dynamic to manage and incorporate throughout the product development life cycle.

Systems Engineering for Healthcare IT

Systems Engineering for Healthcare Information Technology is very similar to other IT developments, with the addition of medical regulations. Healthcare Information Technology is critical to efficient flow of information and delivery of services. (Presidents Council of Advisors on Science and Technology 2010) The product development is a mix of contract driven development (with a target customer, such as healthcare.gov), and market driven (where there are more standard products, with minimal customization). Much of the market, especially for hospitals and hospital chains, is a mix of standard products with large amounts of customization to the customer's specific needs, terminology, and workflows.

Systems Engineering for Pharmaceuticals

The pharmaceutical industry leverages systems that include hardware, software and sometimes single-use components in different part of their value chain, for example complex analytical systems during drug discovery, complex bioreactors and downstream filtration and chromatography systems in manufacturing and drug delivery devices for the use of their drugs. These systems are subject to very different regulations, e.g. GMP or medical devices, depending on the use. One challenging aspect of these systems is that the users have different skill sets and working under different environments. And in all of the examples below, biological and/or chemical processes run on these systems, requiring deep domain knowledge of the system development teams.

The in-vitro diagnostic industry also uses many systems, small devices (e.g. self-testing blood glucose or coagulation monitoring systems) all the way to large, fully automated, high throughput systems for the use in centralized laboratories. Very often, these systems operate as a closed system, so that the reagents used for the diagnostics tests, are proprietary and the vendor of the system only guarantees high quality results only when using the proprietary chemistry. This enables the vendors to often 'place' the instruments at highly competitive prices when the actual profit is generated through the consumables.

For the chemistry part of pharma, understanding the scientific method, using a systems thinking approach, and using six sigma approaches to managing variation and interdependencies is critical. Once you create a product which includes software and physical parts (including manufacturing equipment), systems engineering of the functional design, design analysis, and integration and verification of the solution become critical.

Systems Challenges for Public Health

Summits and inquiries into problems or shortcomings in the public health space have consistently uncovered the same issues: systemic failures in the way that public health is approached that make it nearly impossible to adequately respond to major health events. Examples can be seen from the US response to Hurricane Katrina (e.g. The White House 2006), the 2011 Thoku tsunami (e.g. Carafano 2011, The Heritage Foundation 2012), or even the 2014-2015 Ebola outbreak in West Africa (e.g. GHTC 2015).

The White House report provides insights into just a few of potential challenges for the health aspects of disasters or large-scale emergencies (2006, Chapter 6):

- Tens of thousands of people may require medical care.
- Large portions of a population with chronic medical conditions may themselves without access to their usual medications and sources of medical care.
- Hospitals and other healthcare facilities may be totally destroyed or otherwise rendered inoperable and the area's health care infrastructure may sustain extraordinary damage.

The types of public health challenges will also change over time: Immediate challenges include the identification, triage and treatment of acutely sick and injured patients; the management of chronic medical conditions in large numbers of evacuees with special health care needs; the assessment, communication and mitigation of public health risk; and the provision of assistance to State and local health officials to quickly reestablish health care delivery systems and public health infrastructures. (The White House 2006) As time passes, longer-term infectious disease outbreaks may occur or environmental impacts may cause health risks (e.g. Fukushima nuclear meltdown after the 2010 tsunami). And over time, the public health and overall healthcare infrastructure must be re-established and repaired.

But the public health "system" in most countries, as currently structured, is not prepared to deal with these types of challenges. In talking about the US, Salinsky and Gursky state, "Despite recent attention to the biodefense role of public health, policymakers have not developed a clear, realistic vision for the structure and functionality of the governmental public health system. Lack of leadership and organizational disconnects across levels of government have prevented strategic alignment of resources and undermined momentum for meaningful change. A transformed public health system is needed to address the demands of emergency preparedness and health protection. ... The future public health system cannot afford to be dictated by outmoded tools, unworkable structures, and outdated staffing models." (2006)

The framing of the challenge as a systems one requires the application of a systems approach, and the use of tools capable of supporting systems views, to enable better understanding of the challenges for public health and for creating ways to address these challenges. The SEBoK knowledge areas on Enterprise Systems Engineering and Systems of Systems (SoS) at least partially consider some of these challenges from a systems engineering perspective.

Systems Biology for Healthcare

As systems science is a foundation for system engineering, systems biology is becoming recognized as a foundational discipline for healthcare systems engineering. Systems biology is an emerging discipline and is recognized as strategically important when tackling complex healthcare problems. The development of systems biology is also an emerging environment for systems engineers.

According to Harvard University, “Systems biology is the study of systems of biological components, which may be molecules, cells, organisms or entire species. Living systems are dynamic and complex and their behavior may be hard to predict from the properties of individual parts. To study them, we use quantitative measurements of the behavior of groups of interacting components, systematic measurement technologies such as genomics, bioinformatics and proteomics, and mathematical and computational models to describe and predict dynamical behavior. Systems problems are emerging as central to all areas of biology and medicine.” (Harvard University 2010)

As systems biology matures, its integration into healthcare approaches is expected to lead to advanced practices such as personalized and connected healthcare and the resolution of complex diseases.

Conclusion

While systems engineering practices apply to the healthcare domain, they face different challenges than other industries and need to be tailored. In fact, different segments of the healthcare industry can take significantly different approaches to effective systems engineering and systems thinking.

References

Works Cited

- 21 CFR 820.30. “Part 820 – Quality System Regulation: Subpart C – Design Controls.” *Title 21 – Food and Drugs: Chapter I – Food and Drug Administration, Department of Health and Human Services: Subchapter H – Medical Devices*. Available at: <https://www.accessdata.fda.gov/scripts/cdrh/cfdocs/cfCFR/CFRSearch.cfm?fr=820.30>
- Andel, C., S.L. Davidow, M. Hollander, D.A. Moreno. 2012. “The economics of health care quality and medical errors.” *Journal of Health Care Finance*. 39(1):39:50. Abstract available at: <https://www.ncbi.nlm.nih.gov/pubmed/23155743> 200,000 Americans die from preventable medical errors including facility-acquired conditions and millions may experience errors. In 2008, medical errors cost the United States \$19.5 billion.
- Carafono, J.J. 2011. *The Great Eastern Japan Earthquake: Assessing Disaster Response and Lessons for the U.S.* Washington, DC: The Heritage Foundation. Special Report #94 for Japan. May 25, 2011.
- FDA. 2014. “Premarket Approval (PMA)”. Washington, DC: U.S. Food and Drug Administration (FDA). Accessed February 17, 2016. Available at: <https://www.fda.gov/Medicaldevices/Deviceregulationandguidance/Howtomarketyourdevice/Premarket/submissions/Premarketapprovalpma/Default.htm>
- GHTC. 2015. “Will We Learn from the Lessons of the Ebola Outbreak?” 2015 Policy Report. Washington, DC: Global Health Technologies Coalition (GHTC).
- Gursky, E. 2005. *Epidemic Proportions: Building National Public Health Capabilities to Meeting National Security Threats*. Arlington, VA: Analytic Services Inc. (ANSER).
- Harvard University. 2010. “Department of Systems Biology.” Cambridge, MA: Harvard University. Accessed February 17, 2016. Available at: <https://sysbio.med.harvard.edu/>
- Hospital Safety Score. 2013. “Hospital Errors are the Third Leading Cause of Death in U.S., and New Hospital Safety Scores Show Improvements are Too Slow.” Washington, DC: The LeapFrog Group. Accessed February 17, 2016. Available at: <https://www.hospitalsafetyscore.org/newsroom/display/hospitalerrors-thirdleading-causeofdeathinus-improvementstooslow>

- IEC. 2015. *Medical devices – Part 1: Application of usability engineering to medical devices*. Geneva, Switzerland: International Electrotechnical Commissions. IEC 62366-1:2015. Available at: https://www.iso.org/iso/catalogue_detail.htm?csnumber=63179
- INCOSE. 2015. "Section 8.2.2." *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 4.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-118-99940-0
- Institute of Medicine. 1999. *To Err is Human: Building a Safe Health System*. Washington, DC: The National Academy Press, The National Academy of Sciences. November 1999. Available at: <https://iom.nationalacademies.org/~media/Files/Report%20Files/1999/To-Err-is-Human/To%20Err%20is%20Human%201999%20%20report%20brief.pdf>
- ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.
- Leveson, N.G. 2011. *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA: Massachusetts Institute of Technology (MIT).
- Presidential Council of Advisors on Science and Technology. 2010. *Report to the President: Realizing the Full Potential of Health Information Technology to Improve Healthcare for Americans: The Path Forward*. Washington, DC: Presidential Council of Advisors on Science and Technology, The White House. December 2010. Available at: <https://www.whitehouse.gov/sites/default/files/microsites/ostp/pcast-health-it-report.pdf>.
- Reid, T.R. 2010. *The Healing of America: A Global Quest for Better, Cheaper, and Fairer Health Care*. New York, NY: Penguin Books.
- Salinsky, E. and E. Gursky. 2006. "The Case for Transforming Governmental Public Health." *Health Affairs*. 25(4). 1017-2018. Available at: <https://content.healthaffairs.org/content/25/4/1017.full>
- The Heritage Foundation. 2012. *One Year Later: Lessons from Recover After the Great Eastern Japan Earthquake*. Washington, DC: The Heritage Foundation. Special Report #108 on Asia and the Pacific. April 26, 2012.
- The White House. 2006. *The Federal Response to Hurricane Katrina Lessons Learned*. Washington, DC: The White House. February 2006. Available at <https://library.stmarytx.edu/acadlib/edocs/katrinawh.pdf>

Primary References

None.

Additional References

None.

Overview of the Healthcare Sector

- Lead Author:
- Chris Unger
- Contributing Author:
- Cyrus Hillsman

This article describes some of the stakeholders of the healthcare sector and the factors which influence the application of systems engineering within it. For an overview of healthcare systems engineering and how it deals with these influences see the Healthcare Systems Engineering article.

The healthcare sector is a complex system made up of people, facilities, laws and regulations. It addresses current health, tries to ensure wellness, treats medical problems; creates new medication and medical devices; manages the health both individuals and populations; and helps determine regulations for safety, privacy, the environment, and healthcare delivery itself.

Stakeholders

There are many types of stakeholders in the healthcare sector. The space covers everyone from the general public – who have a stake in their own health and the health of those around them for issues like infectious disease – to the individual researchers who investigate current healthcare problems. The high-level groups of stakeholders include:

- The general public;
- Healthcare providers (such as doctors, nurses, clinics, and hospitals);
- Payers (such as insurance companies);
- Public health organizations;
- Researchers, scientists, and corporations in the pharmaceutical industry;
- Medical device manufacturers;
- Policy makers (particularly those with interest in public health, healthcare safety or privacy policies);
- Healthcare information technology technicians and organizations; and
- Professional organizations and societies relevant to the various aspects of the space.

The healthcare sector is an enormous area financially as well. For example, out of \$2.87 trillion on healthcare spent in the US in 2010, the breakdown of components is:

US Healthcare Expenditures in 2010 (information from Emmanual 2014)

Hospital Care	\$921B
Physician Services	\$555B
Prescription Drugs	\$280B
Nursing Home Care	\$151B
Other Medical Products	\$113B
Dental Services	\$93B
Government Public Health	\$84B
Other Professional Services	\$79B
Home Health Care	\$77B
Research	\$48B

The sections below provide insight into the landscape for these the stakeholder groups where there is sufficient information currently available. More detail will be added as the healthcare aspects of the SEBoK mature and the team will take particular care to incorporate additional information from outside the US going forward.

Healthcare Delivery

The largest share of the money spent on healthcare in the US healthcare is in hospitals (almost a third). The number of hospitals has been relatively flat for the last 20 years. However, due to the growing cost pressures and increasing paperwork, there has been a general consolidation of hospitals into chains, and independent physician providers into hospitals or group practices. (Emmanuel 2014)

Overall Hospital Landscape (information from (AHA 2014))

Total Number of All U.S. Registered * Hospitals	5,627
Total Number of U.S. Community ** Hospitals	4,926
Total Number of Nongovernment Not-for-Profit Community Hospitals	2,870
Total Number of Investor-Owned (For-Profit) Community Hospitals	1,053
Total Number of State and Local Government Community Hospitals	1,003
Total Number of Federal Government Hospitals	213
Total Number of Nonfederal Psychiatric Hospitals	403

Hospitals range from small community hospitals to the New York-Presbyterian Hospital/Weill Cornell Medical Center with 2,259 beds (Becker 2011)], or the University of Pittsburgh Medical Center Presbyterian with \$12B in revenue in 2013. (Becker 2013).

Hospital chains tend to be less than 10 hospitals, with less than 10 chains having more than 10 hospitals (Becker 2015). The largest two have almost 200 hospitals (Community Health Systems with 188 and Hospital Corporation of America with 166). The largest systems by revenue are Kaiser Permanente and the Veterans Health Administration with revenue or budget of slightly over \$50B each.

Medical Devices Manufacturers

The medical device development landscape is diverse, composed of many markets of intermediate size (many being above \$10B in size, with high single digit to double digit growth rates). Some examples are, with projected market sizes in 2020, are:

Types of Medical Devices and Projected Market Share (Emmanuel 2014)

<i>Medical Device Type</i>	<i>Projected Market Share</i>
In-vitro diagnostics (IVD)	\$75B
Endoscopy	\$33B
Interventional Cardiology	\$27B
Infection control	\$17B
Minimally invasive surgery	\$14B
Defibrillators	\$13B
Dental Implants	\$10B
Infusion pump	\$ 7B

Magnetic Resonance	\$ 7B
Digital Xray	\$ 5B

As described in Healthcare Systems Engineering, this is the area of the healthcare sector that is most closely aligned with classic product-focused businesses.

Healthcare IT

There is a large uncertainty in what constitutes Healthcare IT. The most visible segment is the Electronic Medical Record (EMR) or Electronic Health Record (EHR), but there is also large markets in billing management, clinical decision support, image management, etc. But there is a divergence of market sizes with estimates around \$60B [Bain, FierceIT] and some around \$104B [Markets and Markets, MedGadget, and PRNewswire].

An EHR installation at a hospital is similar to an Oracle database installation at a company, where much of the cost is customizing the database and workflows to the institution's policies and workflows, and in training the users to the new system and standardized practices which come with IT and automation.

The top 10 Healthcare IT solution providers in 2015 (information from (Healthcare Informatic 2015))

<i>Company</i>	<i>2015 Revenue</i>
Optum	\$5.2B
Cerner Corp.	\$3.4B
McKesson	\$3.1B
Dell	\$2.9B
Cognizant	\$2.7B
Philips	\$2.7B
Xerox	\$2.4B
Siemens	\$2.0B
Epic Systems Corp.	\$1.8B
GE Healthcare	\$1.5B

Public Health Systems

The World Health Organization (WHO) defines public health as “all organized measures ... to prevent disease, promote health, and prolong life among the population as a whole. Its activities aim to provide conditions in which people can be healthy and focus on entire populations, not on individual patients or diseases. Thus, public health is concerned with the total system and not only the eradication of a particular disease.” (WHO 2016) Governments at each level define exactly what “public health” will encompass, but typically there are three areas: epidemiology, provision of health services, and workplace and environmental safety and policy. Epidemiology is the study and control health-related events, including disease. Various methods can be used to carry out epidemiological investigations: surveillance and descriptive studies can be used to study distribution; analytical studies are used to study determinants.” (WHO 2016, “Health topics: Epidemiology”). Health services may include services such as preventive vaccinations, disease screening, or well-baby or well-child programs. Environmental safety can include developing policies for automobile or workplace safety, monitoring the quality of drinking water, or even conducting restaurant health inspections. In addition to these wide varieties of work, public health organizations are increasingly expected to be responsible for the health-related aspects of disaster and emergency response efforts.

In the US, the public health “system” is really a patchwork of independent healthcare departments. Each state or territory defines the scope and responsibilities of its own public health “department”, requiring information and cooperation from hospitals, private physicians, emergency personnel, laboratory networks, and sometimes public health organizations from other states. (Gursky 2005)

Conclusion

In addition to each group of stakeholders being complex in itself, these stakeholders then interact and work together - or sometimes contradict one another. This makes the landscape of the healthcare systems engineering space itself complex and highlights the need for systems thinking and systems approaches when attempting to address any health-related issues or challenges.

References

Works Cited

- AHA. 2014. "Fast Facts on US Hospitals." Chicago, IL: American Hospital Association (AHA). September 2014. Available at: <https://www.aha.org/research/rc/stat-studies/fast-facts.shtml>
- Becker's Healthcare. 2015. "10 largest for-profit hospital systems | 2015". *Becker's Hospital Review*. June 30, 2015. Available at: <https://www.beckershospitalreview.com/lists/10-largest-for-profit-hospital-systems-2015.html>
- Becker's Healthcare. 2014. "100 Largest Hospitals in the US". *Becker's Hospital Review*. August 7, 2014. Available at: <https://www.beckershospitalreview.com/lists/8-7-14-100-largest-hospitals-in-america.html>
- Becker's Healthcare. 2013. "100 Top-Grossing Hospitals in the US". *Becker's Hospital Review*. June 24, 2013. Available at: <https://www.beckershospitalreview.com/lists/100-top-grossing-hospitals-in-america-2013.html>
- Eliades, G., M. Retterath, N. Hueltschmidt, and K. Singh. 2012. *Healthcare 2020*. Amsterdam, The Netherlands: Bain & Company. Available at: https://www.bain.com/Images/BAIN_BRIEF_Healthcare_2020.pdf.
- Emmanuel, E.J. 2014 *Reinventing American Health Care: How the Affordable Care Act will Improve our Terribly Complex, Blatantly Unjust, Outrageously Expensive, Grossly Inefficient, Error Prone System*. New York City, NY: PublicAffairs.
- Gold, A. 2014. "Global healthcare IT market projected to hit \$66 billion by 2020." *FierceHealthIT*. April 1, 2014. Available at: <https://www.fiercehealthit.com/story/global-healthcare-it-market-projected-hit-66-billion-2020/2014-04-01>.
- Gursky, E. 2005. *Epidemic Proportions: Building National Public Health Capabilities to Meeting National Security Threats*. Arlington, VA: Analytic Services Inc. (ANSER). Healthcare Informatics. 2015. "2015 HCI 100." Available at: <https://www.healthcare-informatics.com/hci100/2015-hci-100-list>
- Markets and Markets. 2015. *North American Healthcare IT Market by Product (EHR, RIS, PACS, VNA, CPOE, mHealth, Telehealth, Healthcare analytics, Supply Chain Management, Revenue Cycle Management, CRM, Claims Management) by End User (Provider, Payer) - Forecast to 2020*. October 2015. Available at: <https://www.marketsandmarkets.com/Market-Reports/north-america-healthcare-it-market-1190.html>
- MedGadget. 2015. "Global Healthcare IT Market 2020 - Industry Survey, Market Size, Competitive Trends: Radiant Insights, Inc". November 2, 2015. Available at: <https://www.medgadget.com/2015/11/global-healthcare-it-market-2020-industry-survey-market-size-competitive-trends-radiant-insights-inc.html>
- PRNewswire. 2015. "Healthcare IT Market Size to Reach \$104.5 Billion by 2020: Grand View Research, Inc." October 15, 2015. Available at: <https://www.prnewswire.com/news-releases/healthcare-it-market-size-to-reach-1045-billion-by-2020-grand-view-research-inc-533012831.html>

WHO. 2016. "Health Topics: Epidemiology." Geneva, Switzerland: World Health Organization. Available at: <https://www.who.int/topics/epidemiology/en/>

WHO. 2016. "Trade, foreign policy, diplomacy, and health: Public Health." Geneva, Switzerland: World Health Organization. Available at: <https://www.who.int/trade/glossary/story076/en/>

Primary References

None.

Additional References

None.

Systems Engineering in Healthcare Delivery

- Lead Author:
- Cyrus Hillsman
- Contributing Authors:
- Chris Unger and Nicole Hutchison

The healthcare system is complex and adaptive and confronts significant challenges for which systems engineering tools are useful and necessary. The President's Council of Advisors on Science and Technology (PCAST) prepared a report concluding that healthcare improvement could be accelerated with the use of systems engineering. (PCAST 2014) They noted that the key incentives are wrong (fee for service vs. fee for outcomes), and key enablers are missing (access to useful data, lack accepted systems techniques and people trained in systems engineering)

This article provides an overview of healthcare delivery with some historical context, and describes some different approaches to systems engineering which have been found helpful in addressing healthcare delivery problems.

Human Centered Design

Healthcare delivery is not a product but a service and that makes it different than typical hardware or software design that may be seen in aerospace, defense, or even medical devices. There are three primary factors for these differences. First, quality in services can be difficult to measure objectively. Second, in this service system, care providers are continually making risk, cost, and quality of care decisions at the time of service. Each patient is unique and multiple pathologies and value streams are possible based upon any given patient's needs. Those needs are complemented by a care team that is unique and complex and includes the patients themselves, family support, medical professionals, hospitals, and even the industry in which it resides. Third, if returning to or maintaining wellness is considered to be the core value for a healthcare delivery system, then the patient's behaviors both within and outside any designed care plan has a significant role to play, because roughly half of all healthcare cost is derived from preventable disease. (Conover 2012)

Structure of the Healthcare Delivery Industry

The healthcare industry is large, diverse, and fragmented and this causes considerable complexity. This complexity is experienced both at the macro and micro levels.

At the macro level the healthcare industry is highly fragmented with over 50% of all healthcare workers employed in companies with less than 500 employees. (Griffith & White 2007) Nearly 1 million physicians practice medicine in the US; roughly half of these are in primary care and the rest are in over 30 specialties and many more subspecialties and clinics. In addition to physicians, some 5 million others in some 50 other specialties provide care to patients.

At the micro level, the complexity and pace of change make care difficult. Healthcare is a rapidly developing field with over 700,000 publications produced annually and the pace in fact is increasing. (Smith et al. 2013). That there are already 14,400 codes in the World Health Organization's *International Statistical Classification of Diseases and Related Health Problems* (ICD) complicates the issue. Add to this the regulatory and administrative burden of primary care providers interacting with approximately 200 specialists in any given year and the complexity that care providers face on a daily basis becomes clear.

In short, the healthcare delivery system is itself a complex adaptive system and represents a wicked_problem_, whereby any changes to the system intended to solve an issue will likely create other issues.

Improving Ongoing Operations

As mentioned, above caregivers are faced with many challenges and the goal of in systems engineering in healthcare delivery is to lessen that burden in a systematic way without significant disruption of current operations. To do this successfully requires several factors:

- First, as stated above, systems engineers have to acknowledge that they are dealing with a complex adaptive system that includes many wicked problems. An analogy is that what systems engineers experience in healthcare is like rewiring a house with the power turned on because whatever changes are made are to an existing system that must operate while the changes are being made.
- Second, "the system" in place is difficult to define. The "healthcare system" is actually a combination of many open systems and interdependencies with the system of interest may be unknown.
- Third, patient safety is always a concern and any actions that could affect patient safety must be very carefully considered. Often, "optimizing" a system may introduce a potential risk to patient safety. These system aspects are always in tension.
- Fourth, there is a bias towards the current (known) system versus a change leading to an unknown system. Any change will create a certain amount of disruption to an operational system that may be currently operating at or beyond capacity.
- Fifth, healthcare delivery systems are combinations of patients, providers, process, and products and therefore uncertainty is a daily reality. This level of uncertainty may not be amenable to typical agile approaches of 4-6 week sprints nor traditional waterfall methods.
- Sixth, local factors could play a significant role; therefore no two sites may perform an operation in exactly the same way.
- Seventh, the entire industry acts as a complex adaptive system with multiple intelligent agents working sometimes in partnership and sometimes in conflict with the goals of the system or patient.

Because of these factors and others the tradition of healthcare systems engineering has been to use adaptable human-centered methods. (Checkland 1999)

History of Healthcare Improvement Research

There have been many attempts to understand and improve healthcare both in the public and private domains. Examples include the National Healthcare Service Change Model, the efforts of the Agency for Healthcare Research and Quality, and the Institute for Healthcare Improvement. Here we outline some representative efforts.

Healthcare improvement has been shaped in part by four seminal works by the Institute of Medicine (IOM). *To Err is Human* reported that up to 98,000 patients were killed by healthcare each year. (Kohn, Corrigan, & Donaldson 2000) This put an emphasis on safety as a key quality of care metric. The following year the Institute of Medicine (IOM) broadened the concept of quality beyond safety to include six measures of quality. They determined that healthcare should be safe, effective, patient centered, timely, efficient, and equitable. (Institute of Medicine 2001) This report called *Crossing the Quality Chasm* included an appendix that documented poor quality and the severity of the issues of under use, over use, and potential for harm in medicine. A search for the underlying reasons for poor quality led to three primary reasons for poor quality. The three reasons were the growing complexity of science and technology; the increase in chronic conditions; and the failure to exploit information technology.

To address these concerns the IOM partnered with the National Academy of Engineering (NAE) to see what could be done from a systems engineering perspective to address the real challenges facing the industry in *Building a Better Delivery System*. (Compton et al. 2005). That was followed by the realization that standard systems engineering needed to be modified and healthcare was and would remain a human centered endeavor as stated in *Best Care at Lower Cost* (Smith et al. 2013)

Three Approaches

Although there are many accepted approaches to healthcare systems engineering and improvement, here we outline three that share common characteristics and are representative of most of the other methods.

The first approach is Lean Six Sigma which is a combination of two methods. Lean has its roots in the Toyota Production System (Ohno 1988) and the work of the International Motor Vehicle Program (Womack, Jones, & Roos 1990). Six-Sigma has its roots at Motorola and the work of Bill Smith. These two methods were combined by Michael L. George (see (George 2002) and (George 2003)). It includes techniques like value stream mapping, waste elimination, root cause analysis, and voice of customer. For additional information see *Lean Engineering* and *Lean in Healthcare*.

The second approach is based on industrial engineering, which has its roots in the work of Frederick Taylor and others. This approach includes tools such as discrete event simulation, ergonomics, production control, and operations research as shown in Figure 1. For additional information, see *Systems Engineering and Industrial Engineering*.

Insert Table ES-1 from *Building a Better Delivery System* here once we obtain the proper permissions.

The third approach is healthcare systems engineering. Traditional systems engineering uses a functional decomposition approach; see for example (Defense Systems Management College 2001). However, healthcare problems are often classified as wicked and complex and not amenable to traditional decomposition methods found in other areas of engineering. (Rouse & Serban 2014).

There are many tailored approaches to improving healthcare delivery, but almost all are based on one of these three approaches, or a combination of these.

Healthcare Systems Engineering

The basic systems engineering steps are similar to those for any industry specific applications, but the steps are tailored for healthcare. The traditional waterfall model of requirements, design, implementation, verification, and maintenance is interrupted in favor of almost continuous support. In many cases the closeout and transfer of the project to operational staff is more challenging in healthcare than in many industries.

Below is outlined a general methodology used by the US Department of Veteran's Affairs (VA) that may suit a wide variety of situations and programs, composed of 4 pillars: Define the Problem, Investigate Alternatives, Develop the Solution, and Launch and Assess the Solution. These 4 pillars are similar to classic mistake avoidance, development fundamentals, risk management, and schedule oriented approaches. (McConnell 1996); they are also similar to the Plan/Do/Check/Act methodology.

Define the Problem

As mentioned above, the patient is augmented by a care team consisting of family, friends, clinical staff, and many other support staff the patient will not directly encounter. This care team may not be familiar with the rigors of traditional engineering design. Because of this, a systems engineer may use a paired partnership model where engineers are embedded with clinical and administrative staff, family, and the patients themselves. In this concept, everyone is a designer and our goal is to provide them with the tools to contribute to the system design process. Even at this early stage, configuration management would be considered. Depending on the size of the rollout one alpha site and several beta sites may be used at any phase to avoid local optimal solutions that don't work globally.

Investigate Alternatives

During the proof-of-concept phase, visualizing the result is important for the reasons mentioned above. Therefore, one or more initial prototypes may be developed with the alpha site. The goal is to get to a minimally viable product as soon as possible to demonstrate the viability of the product or methodology. After the initial conversations and meetings, participants have a need to have a common understanding of how the system will work. The systems engineer would embrace the concept of operations with rich pictures, model based systems engineering, story boards, customer journey maps and other tools so that we all have a common understanding of the proposed system.

Develop the Solution

Using what has been learned from the minimally viable product feedback and incorporate that into the future state optimization, one would continue developing the prototype at the initial paired partner alpha site and then the trusted beta demonstration sites. In our case, stakeholders are a part of the development team and not an ancillary function. For this reason, demonstration is considered a key element of the communication plan when developing the solution.

Launch the Solution and Assess the Performance

During evaluation and deployment phase, a systems engineer would have considered the future state optimization with corresponding alpha and beta sites. Live implementation would then be used for further testing and evaluation. At any phase feedback is encouraged and reflected in the next iteration of the solution. As mentioned previously abandonment and closeout even during the live phase may not be practical and in fact could be disadvantageous because not all possible needed configurations or situations would have been encountered.

Example Systems Engineering Tools

Below is a list of systems engineering tools which could be used at each of the four steps.

1. Define the Problem
 1. Establish the scope and context of the problem (define boundary conditions)
 2. Stakeholder identification and management
 3. Lifecycle mapping
 4. Value Stream Process Mapping
 5. SWOT analysis (Operational Deficiencies and Technological Opportunities)
 6. Workflow/Usability/Use Case analysis
 7. Observation Research
 8. Root Cause Analysis (Fishbone diagrams, 5 whys, ...)
2. Investigate Alternatives
 1. Requirements management
 2. SE Evaluation Methods (Decision Trees, Quality Function Deployment (QFD))
 3. Trade-off Analysis
 4. Model-Based Systems Engineering (MBSE)
 5. Technical Risk Management
3. Develop the Solution
 1. Concept Development
 2. Architecting the solution (functional analysis, subsystem decomposition, interface definition and control, modeling)
 3. Define the implementation
 4. Process Redesign Techniques (including Lean Six Sigma)
 5. Active Integration
 6. Agile / Lean Development Principles (iterative development)
4. Launch and Assess the Solution
 1. Managing Change in Organizations
 2. Stakeholder Management, Change Management Techniques
 3. Spiral, Agile, and Lean Startup Delivery Practices (Minimal Viable Product delivery)
 4. Business Risk Management
 5. Metrics and benchmarking

During all phases, elements of cognitive & organizational psychology, industrial engineering, usability engineering, systems engineering, and other facets may be critical to implement a solution. Humans are *the* major part of the system and even the system of systems approach in healthcare.

Conclusion

Systems Engineering for Healthcare delivery shares many aspects with traditional SE, but differs significantly since healthcare delivery is a service (not a product) and due to the domain specific challenges. In particular, problem definition is a particularly 'wicked' problem, and measuring successful outcomes in a clear and objective fashion is challenging.

References

Works Cited

- Checkland, P. 1999. *Systems Thinking, Systems Practice*. Hoboken NJ: John Wiley.
- Compton, W.D., G. Fanjiang, J.H. Grossman, & P.P. Reid. 2005. *Building a better delivery system: a new engineering/health care partnership*. Washington DC: National Academies Press.
- Conover, C.J. 2012. *American health economy illustrated*. Washington DC: American Enterprise Institute.
- Defense Systems Management College. 2001. *Systems engineering fundamentals: Supplementary text*. Fort Belvoir, VA: The Press.
- George, M.L. 2002. *Lean Six Sigma*. New York, NY: McGraw-Hill.
- George, M.L. 2003. *Lean Six Sigma for Service*. New York, NY: McGraw-Hill.
- Griffith, J.R., & K.R. White. 2007. *The Well-Managed Healthcare Organization*. Chicago, IL: Health Administration Press.
- Institute of Medicine. 2001. *Crossing the quality chasm: A new health system for the 21st century*. Washington DC: National Academy Press.
- Kohn, L.T., J. Corrigan, & M.S. Donaldson. 2000. *To err is human: Building a safer health system*. Washington DC: National Academy Press.
- McConnell, S. 1996. *Rapid Development*. Redmond, WA: Microsoft Press.
- Ohno, T. 1988. *Toyota Production System*. Portland OR: Productivity Press.
- PCAST. 2014. *Better Health Care and Lower Costs: Accelerating Improvement through Systems Engineering*. Washington DC.: President's Council of Advisors on Science and Technology (PCAST).
- Rouse, W.B., & N. Serban. 2014. *Understanding and managing the complexity of healthcare*. Cambridge, MA: The MIT Press.
- Smith, M.D., R.S. Saunders, L. Stuckhardt, & J.M. McGinnis. 2013. *Best care at lower cost: The path to continuously learning health care in America*. Washington DC: National Academies Press.
- Womack, J.P., D.T. Jones, and D. Roos. 1990. *The machine that changed the world*. New York, NY: Harper Collins.

Primary References

- PCAST. 2014. *Better Health Care and Lower Costs: Accelerating Improvement through Systems Engineering*. Washington DC.: President's Council of Advisors on Science and Technology (PCAST).

Additional References

None.

Systems Biology

- Lead Authors:
- Bridgette Daniel Allegro and Gary Smith
- Contributing Authors:
- Chris Unger and Nicole Hutchison

Systems biology is the computational and mathematical modelling of complex biological systems. Systems biology is a biology-based inter-disciplinary field of study that focuses on complex interactions within biological systems, using a holistic approach to biological research. From year 2000 onwards, the concept has been used in the biosciences in a variety of contexts. For example, the Human Genome Project is an example of applied systems thinking in biology which has led to new, collaborative ways of working on problems in the biological field of genetics. One of the outreaching aims of systems biology is to model and discover emergent properties of cells, tissues and organisms functioning as a system whose theoretical description is only possible using techniques which fall under the remit of systems biology. These typically involve metabolic networks or cell signaling networks. (Wikipedia Contributors 2016)

Systems Biology: A Vision for Engineering and Medicine

Organisms and Hosts Interact in Communities of Life

There is an increasing appreciation that microbes are an essential part of the ecologically-important traits of their host. Organisms do not live in isolation, but have evolved, and continue to evolve, in the context of complex communities and specific environmental conditions. Evolutionary biologists are increasingly able to integrate information across many organisms, from multiple levels of organization and about entire systems to gain a new integrated understanding that incorporates more and more of the complexity that characterizes interdependent species associations. Only when we begin to understand the molecular base for adaptation and interactions of communities of life, can we start to comprehend how ecosystems are functioning.

Addressing Different Levels of Organization of Organisms

Understanding the function of complex biological systems is one of the greatest challenges facing science. The rewards of success will range from better medicines to new engineering materials. The sequencing of the human genome, although of fundamental importance, does not even provide a complete parts list of the protein molecules that exist in a biological organism because of complexities of downstream processing and complex folding required to make a functioning receptor or enzyme from a long chain of amino acids. Furthermore, protein molecules do not function alone but exist in complex assemblies and pathways that form the building blocks of organelles, cells, tissues, organs, organ systems and organisms, including man. The functioning of brain or muscle, liver or kidney, let alone a whole person, is much greater than the sum of its parts.

Figure 1 - Levels of Structural Organization of the Human Body (source - https://cnx.org/contents/Xh_25wmA@7/Structural-Organization-of-the#fig-ch01_02_01)"

Internalizing the Complexity – Pushes the Boundary of Systems Thinking Capability

To tackle this problem (understanding biological systems) requires an iterative application of biomedical knowledge and experiment with mathematical, computational and engineering techniques to build and test complex mathematical models. Systems and control engineering concepts, a modular approach and vastly increased computing capacity are of critical importance. The models, once developed and validated, can be used to study a vastly greater range of situations and interventions than would be possible by applying classical reductionist

experimental methods that usually involve changes in a small number of variables. This new approach is now termed "Systems Biology". It allows insight into the large amount of data from molecular biology and genomic research, integrated with an understanding of physiology, to model the complex function of cells, organs and whole organisms, bringing with it the potential to improve our knowledge of health and disease. Systems Biology will inevitably become an approach that pervades scientific research, in much the same way that molecular biology has come to underpin the biological sciences. It will transform the vast quantities of biological information currently available into applications for engineering and medicine.

Natural Patterns and Engineered Patterns Can Be a Source of Inspiration - in Both Directions

Biological organisms are much more complicated than any machine designed by man. However, there are similarities between the way in which organs and whole organisms are assembled from molecules and cells and the design methods used by engineers in the construction of complex systems. The application of such methods to biology will, however, require novel engineering tools to be developed since biological systems possess key features that artificial ones do not. Specifically, biological systems have an exceptional capacity for self-organization and assembly, using rules and mechanisms that have been shaped by natural selection. Biological systems also have significant capacity for continuing self-maintenance through turnover and renewal of component parts. Perhaps the property that distinguishes biological systems most is their ability to auto-adapt their organization to changing circumstances through altered gene expression, or more directly, through signal transduction and modification of proteins. This adaptation culminates at higher levels of organization as evidenced by phenomena such as the development of resistance to antibiotic therapy or tolerance to recreational drugs. The mechanisms by which component parts interact are often highly stochastic in nature; that is, susceptible to the play of chance, which becomes particularly important when only a few components are being considered. Nevertheless, biological systems are robust.

Advancements in Methods for Predicting "What If" in the Behavior of Complex Adaptive Systems

Advances in engineering design and techniques carry a significant potential in driving the progress of Systems Biology. Interventions to biological systems intended to improve health, whether environmental, pharmacological or clinical, need to be carefully thought through and carried out to maximize benefit and reduce harm. The refinement of techniques and tools enables devices and systems to achieve a defined performance within precise tolerance limits, potentially allowing better interventions to complex biological systems. They will be increasingly necessary to permit more reliable system-wide predictions of the effects of biomedical advances and to achieve desired clinical results to a predefined tolerance, or at least to have a quantitative bound on the biological uncertainty.

Transdisciplinary Approaches are Needed to Address the Complex Bio-system Problems

Research in the field of Systems Biology requires close interactions and collaborations between many disciplines that have traditionally operated separately such as medicine, biology, engineering, computer science, chemistry, physics and mathematics. Systems Biology demands a focus on the problem as a whole and therefore a combination of skills, knowledge and expertise that embraces multiple disciplines. The success of leaders in the field of Systems Biology will depend strongly on the extent to which they accomplish the creation of the environment that researchers need to develop an understanding of different working cultures, and manage also to implement strategies that integrate these cultures into shared working practices.

Systems Biology: Relevance to Healthcare

Complex Diseases Demand Systemic Approaches

Over the past few decades, pharmaceutical R&D has focused on creating potent drugs directed at single targets. This approach was very successful in the past when biomedical knowledge as well as cures and treatments could focus on relatively simple causality. Nowadays, the medical conditions that affect a significant proportion of the population in industrialized countries are more complex, not least because of their multifactorial nature. The sequencing of the human genome has led to a considerable increase in the number of potential targets that can be considered in drug discovery and promises to shed light on the etiology of such conditions. Yet, the knowledge of the physiological properties and the role that these targets play in disease development is still limited.

Diminishing Returns in the Single Target Approach to Disease

In terms of drug targets, there is a case that much of 'the low hanging fruit' was picked in the period between the late 1940s and the mid-1980s. The decline in output of new molecular entities and medicines recorded over the last 20 years, despite the steadily growing R&D expenditure and significant increase in sales, bears testimony to the fact that advances with new targets are more difficult and that R&D projects have become much more prone to failure. A basic problem is that the many factors that predispose to, and cause, complex diseases are poorly understood let alone the way in which they interact. The very fact that there are multiple drivers for these conditions suggests that a reductionist approach focusing on individual entities in isolation is no longer appropriate and may even be misleading. It is therefore necessary to consider 'novel' drugs designed to act upon multiple targets in the context of the functional networks that underlie the development of complex diseases. Many of the new developments are likely to turn into effective medicines when combinations of drugs are used to exert a moderate effect at each of several points in a biological control system. Indeed, many common diseases such as hypertension and diabetes are already treated with a combination of two or three medicines hitting different targets in the control network that underlies the condition. Investigating the possible combinations by trial and error in man is onerous but feasible with two components. However, it quickly becomes extremely complicated with three components and well-nigh impossible with four or more. Systems Biology, promises to assist in the development of more specific compounds and in the identification of optimal drug targets on the basis of their importance as key 'nodes' within an overall network, rather than on the basis of their properties as isolated components.

Individualized Medicine, Tuned to the Individual and Their Circumstances

Increasingly powerful drugs will be aimed at a decreasing percentage of people and eventually at single individuals. Modelling can be used to integrate in vivo information across species. Coupled with in vitro and in silico data, it can predict pharmacokinetic and pharmacodynamics behavior in humans and potentially link chemical structure and physicochemical properties of the compound with drug behavior in vivo. Large-scale integrated models of disease, such as diabetes and obesity, are being developed for the simulation of the clinical effects resulting from manipulations of one or more drug targets. These models will facilitate the selection of the most appropriate targets and help in planning clinical trials. Coupling this approach with pertinent genomic information holds the promise of identifying patients likely to benefit most from or to be harmed by, a particular therapy as well as helping in the stratification of patients in clinical trials. Symptoms that diagnose a disease do not necessarily equate to a common cause.

Systems Biology is arguably the only research approach that has the potential to disentangle the multiple factors that contribute to the pathogenesis of many common diseases. For example, hypertension, diabetes, obesity and rheumatoid arthritis are known to be polygenetic in origin although individual genes may not have been identified. Ultimately, the prevention of these conditions rests upon a comprehensive approach that engages with each of the more important predisposing factors, genetic and environmental, that operate upon individuals. A systems approach

is already proving valuable in the study of complex scientific subjects and the research aimed at the prevention and management of medical conditions. Illustrative examples are neuroscience, cancer, ageing and infectious diseases.

A Healthcare Paradigm Reinforcing the Causes of Health and Not Just the Treatment of Disease

Notwithstanding the hugely important role that Systems Biology plays in understanding disease and designing drugs that treat them, the greatest opportunities may lie in health maintenance and disease prevention. Even modest measures that could retard the effect of ageing on brain, heart, bones, joints and skin would have a large impact on the quality of life and future healthcare demands of older people and consequently on the provision of health services. Young people are vulnerable too. Multifactorial diseases such as diabetes, obesity, allergies and autoimmune conditions are becoming prevalent in younger people and unless effective measures are taken to prevent an early and significant decline in their health, healthcare demand will increase exponentially. It is apparent that multiple and diverse factors interact in determining health, quality of life and ageing. These include genetic makeup, microbiota, diet, physical activity, stress, smoke and alcohol, therapeutic and social drugs, housing, pollution, education, and only a systems approach will permit the understanding of how best to prevent and delay health decline.

References

Works Cited

Wikipedia contributors. "Systems biology." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 20 Aug. 2016. Web. 12 Sep. 2016.

Primary References

None.

Additional References

Bosch, T.C.G. and M.J. Mc-Fall-Ngai. 2011. "Metaorganisms as the new frontier." *Zoology*. 114(4): 185-190. September 2011. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3992624/>

Dollery, C. and R. Kitney. 2007. *Systems Biology: a vision for engineering and medicine*. London: The Academy of Medical Sciences and The Royal Academy of Engineering. Available at <https://www.acmedsci.ac.uk/viewFile/publicationDownloads/1176712812.pdf>

Endy, D. 2005. "Foundations for engineering biology." *Nature*. 438(7067): 449-453. 24 November 2005. Available at: <https://www.nature.com/nature/journal/v438/n7067/abs/nature04342.html>

Harvard Medical School. 2010. "Department of Systems Biology." Cambridge, MA: Harvard Medical School, Harvard University. Available at: <https://sysbio.med.harvard.edu/>

Kitano, H. 2002. "Systems Biology: A Brief Overview." *Science*. 295(5560): 1662-1664. 01 March 2002. Available at: <https://science.sciencemag.org/content/295/5560/1662>.

Sausser, B., J. Boardman, and D. Verma. 2010. "Toward a Biology of Systems of Systems." *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans*. 40(4): 803 - 814. Available at: <https://ieeexplore.ieee.org/document/5467221/>

Wikipedia contributors. "Systems biology." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 20 Aug. 2016. Web. 12 Sep. 2016.

Lean in Healthcare

- Lead Author:
- Bohdan Oppenheim
- Contributing Authors:
- Chris Unger and Nicole Hutchison

Lean Thinking, or Lean for short, originated in Toyota factories in the 1960s, was “transplanted” to the U.S. in 1992 with the publication of Womack and Jones' *Lean Thinking: Banish Waste and Create Wealth in Your Corporation* (2003), and evolved globally to practically all work domains: healthcare, engineering and systems engineering, science, administration, supply chain, government, banking, aviation, and many others (Oppenheim 2011). Lean has proven itself as the most effective methodology for improving operations identifying and eliminating waste from work processes. (E.g. Womack and Jones 2003; Oppenheim 2011; Graban 2012; Toussaint and Gerard 2010; and Oehmen 2012) Since 2003, Lean has established itself in healthcare operations.

Overview of Lean in Healthcare

Entire medical organizations (e.g., Theda Care, WI; Jefferson Healthcare, WA; Virginia Mason, WA; Geisinger Health (now called ProvenCare), PA; St. Elizabeth, Tilburg, The Netherlands, and numerous others (e.g. Graban 2012; Toussaint and Gerard 2010)) have been transformed with Lean. These sources contain rich data on specific improvements. Most leading healthcare institutions now have Lean centers of excellence or use Lean consultants, including Kaiser Permanente, Mayo Clinic, UCLA, Veterans Administration, and others. Lean has proven itself in reducing turnaround time of clinical tests, the time spent by patients in emergency departments, operating suites, pharmacies and clinics. Lean improvements in healthcare on the order of 30-50% are routine because traditional healthcare operations are burdened with this much waste, which remains “unseen” by the employees unless they are trained in Lean. Lean is now an established paradigm for improving healthcare delivery operations: increasing quality of healthcare, delivering care faster, shortening patient time in the system, increasing the time of medical professionals with the patient, reducing bureaucracy, increasing capacity of operations, and reducing healthcare costs and frustrations. (Graban 2012; Toussaint and Gerard 2010)

Lean does not mean that people have to work faster or “attach roller blades to move around faster”. In Lean, systems employees work at their regular ergonomic and intellectual speeds. The time savings come from finding and eliminating idle states (e.g., waiting in numerous queues in the emergency departments), reduction of mistakes and rework, elimination of non-value adding tasks, and more streamlined movements of patients, staff, equipment, and supplies. And, most emphatically, Lean does not mean “mean layoffs”. Quite the opposite is true: Lean improves human relations at work and changes the culture from the traditional “blaming and shaming” to teamwork and cooperation focused on the good of the patient. (Graban 2012 (in particular see the endorsements from eight medical professionals on pages ii and iii) and (Toussaint and Gerard 2010)

With the endorsement of *Lean for Systems Engineering with Lean Enablers for Systems Engineering in the Wiley Series*, (Oppenheim 2011) the International Council on Systems Engineering (INCOSE) has effectively adopted Lean as one of its essential competencies. This book was followed with a major joint Project Management Institute (PMI)-INCOSE-MIT publication of (Oehmen 2012) integrating Lean with Systems Engineering and Program Management. Indeed, when applied with Systems Engineering and Systems Thinking, Lean becomes a powerful weapon in bending the healthcare cost curve and improving the quality of care.

Three concepts are critical to the understanding of Lean: value, waste, and the process of creating value without waste, which has been captured into the so-called Six Lean Principles, as follows.

- **Value:** M. Porter (2010) suggested that patients value three levels of care: (1) survival and the degree of recovery; (2) the time required to get back to normal activities, and (3) the sustainability (individual and social cost) of

treatments.

- **Waste:** Table 1 lists the eight categories of waste used in healthcare. (Graban 2012; Toussaint 2010)

Table 1. Eight Waste Categories Used in Lean Healthcare (SEBoK Original).

Waste Type	Healthcare Examples
1. Waiting	Patients wait in numerous queues in clinics, test facilities, ERs, pharmacies, and for insurance approvals; MDs wait for next activity to occur (e.g. test results, information, approvals.)
2. Over-processing	Performing work that is not valued or needed, e.g. MDs and RNs spending time on computer filling out bureaucratic forms that nobody will review.
3. Over-production	Performing more work than needed for value. Transport of a patient in a wheelchair performed by expensive medical professionals because of the lack of transporters.
4. Inventory	Excess inventory costs. Expired supplies that must be thrown away.
5. Transportation of Patients	Transportation of patients over long distances to test offices in hospitals. Poor layout of hospitals, EDs, or test facilities.
6. Motion of Staff	Staff walking over long distances to fetch supplies, and between patients and central hospital stations.
7. Defects	Treatment of hospital infections. Failed and repeated tests, repeated paperwork. Surgical cart missing an item. Wrong medicine.
8. Waste of Human Potential	Burnout of medical staff. Frustrated employees quit making suggestions for improvements.

Table 2 lists the six Lean Principles (Graban 2012) and provides healthcare examples.

Table 2. Six Lean Principles (SEBoK Original)

Principle Name	Explanation
1. Value	Specify value from the perspective of the customer: the patient.
2. Value Stream	Identify all the value-added steps across the entire process, crossing all departmental boundaries, linking the steps into a seamless process, and eliminating all steps that do not create value.
3. Flow	Keep the processes flowing smoothly through all the steps, eliminating all causes of delay, such as batches of patients or items, and quality problems.
4. Pull	Avoid pushing work onto the next step or department; let work and supplies be pulled, as needed, when needed.
5. Perfection	Pursue perfection through continuous improvement, Kaizen events, implement best work standards, checklists, training, and promote improvement teams and employee suggestions.
6. Respect People	Create work environment based on synergy of cooperation, teamwork, great communication and coordination. Institute leadership. Abandon the culture of blaming and shaming.

Lean Practices

Lean healthcare strongly promotes engaging and leading employees. Lean places a big value on continuous education and training of employees at all levels. Lean management promotes standardization of best practices (“the best known way of doing it”, but not necessarily “identical”), checklists, redundancies, patient safety and privacy rules, and patient data security and cybersecurity. Lean advocates visual management, with electronic or “black” boards updated in real time and displaying all information important for the local employees to manage their operation efficiently. Patient safety is still a significant problem in the U.S., in 1999 causing almost 250,000 deaths (Institute of Medicine, 1999) and medical errors occur in one of three admissions. Instead of “blaming and shaming” Lean promotes error and harm prevention and deep root-cause analysis, implementing processes and tools that make

it impossible to create an error.

Systems Thinking and Lean

Healthcare is the most complex socio-technological system in our society, consuming nearly 20% of the U.S. GDP. Healthcare should be safe, effective and evidence based (Berwick 2011), as well as affordable and accessible, efficient, patient centered, timely, well integrated, and inclusive of latest science. (Oppenheim 2015) Healthcare has many stakeholders: the patients, medical professionals, medical facilities, hospitals, clinics, labs, medical equipment makers and users, pharmaceuticals, healthcare researchers and academia, insurances, employers, federal & state governments and international disease prevention centers, military and veteran's administration, fire departments and ambulances and others. The number of potential interactions (interfaces) in this hyper-system is extensive, and many interfaces are nonlinear, "wicked" (interacting with unpredictable humans), often creating unintended consequences and emergent behaviors. Because of these vast complexities, healthcare leaders (e.g. Kanter 2015) point out the need for intensive application of systems thinking and Lean when addressing these challenges. Attempting to solve the complex healthcare problems without systems thinking risks myopic and unsafe attempts which create more problems than they solve. Attempting to solve the challenges without Lean inevitably promotes excessive wastes, costs, and inefficiencies. Good healthcare needs both, Systems Thinking and Lean, to be applied simultaneously.

Lean and Agile in Six Healthcare Value Streams

The Healthcare Working Group of INCOSE identified six following value streams for HSE: A. Systems Engineering for medical devices B. Systems Engineering for healthcare informatics and medical records C. Healthcare delivery (operations) D. Biomedicine and big data analytics E. Pharmaceutical value streams F. Healthcare public policy

As described above, Lean is extraordinarily effective and well established in improving healthcare delivery operations (C). Agile is highly effective in (B) because this value stream works with software, the domain from which Agile originated. Since the stream (A) is the most similar to traditional systems engineering, Agile is expected to be effective therein, although Agile is not yet highly popular in healthcare outside of the software domain. Elements of Lean improvements which are localized and weakly convoluted (e.g., Kaizen events) have strong overlap with Agile/Scrum methodology. (Medinilla 2014)

MBSE and Lean

A highly powerful Model Based Systems Engineering (MBSE) is clearly the tool of choice for those applications where the benefit from multiple use of a standardized (reference) architecture and standard model compensates for the significant effort of creating such a model or architecture. (OMG 2016) In healthcare the value streams (A), (B), (E) and potentially F are the most conducive to the application of MBSE. Lean thinking is applicable to any healthcare operation without limitation. The Lean improvements always begin with the so-called Gemba waste walks, during which experts together with local process stakeholders walk along all the process steps, interviewing stakeholders and identifying and measuring the wastes wherever they occur. The rich menu of Lean thinking processes and tools is then applied to eliminate the wastes. Training and active participation of local stakeholders is always required.

Examples of Lean Improvements

1. In Jefferson Healthcare, WA: (Murman 2010)

- In Acute Myocardial Infarction (a severe heart attack) time is critical as the greatest loss of heart muscle is in the first two hours. Recommended treatment is catheter insertion of balloon within 90 min of the contact with the patient (wherever the patient happens to be located). The Lean approach has reduced the treatment time from 165 min to 20-60 min at the patient site, vastly increasing patient survival rate.
- The five Jefferson Healthcare clinics increased the cumulative available clinic hours from 1400 to 5600 in two years of Lean improvements which were focused on reorganizing medical staff schedules and eliminating wasted times, with no staff additions. The available clinic hours directly translate into billable visits: 1175 additional patients have been seen in 2009 compared to 2008 across the five clinics.
- The Operating Room daily “on time start” of actual operations went from 14% to 96% using Lean tools for process planning and workplace organization.
- Harder to measure is the culture change, although the staff participation at Lean improvement events was at 50%.

2. In Kaiser Permanente Southern California: (Oppenheim 2015)

- In nine regional clinical laboratories Lean improvements cut the turnaround time for laboratory results by between 30 and 70%, with significant corresponding reductions of cost, rework, errors and work morale, and without hiring new staff or adding equipment.
- In two Emergency Departments (ED) the average patient length of stay was reduced by 40% by the elimination of various idle states. The ED capacity increased accordingly.
- The amount and cost of inventory of supplies on hand was reduced by nearly 30% by introducing the Just-in-Time tools of Lean.

3. In Alegent Health, NE (Grabau 2012) the turnaround time for clinical laboratory results was reduced by 60% in 2004 without adding new staff or equipment; and by another 33% from 2008 to 2010.

4. In Kingston General Hospital, Ontario (Oehmen) the instrument decontamination and sterilization cycle time was reduced by 54% while improving productivity by 16%.

5. In Allegheny Hospital, PA the central-line associated bloodstream infections were reduced by 76%, reducing patient death from such infections by 95% and saving \$1 million.

6. In UPMC St. Margaret Hospital, PA (Grabau 2012) the readmission rates for chronic obstructive pulmonary disease (COPD) patients were reduced by 48%.

7. In ThedaCare, WI [3] the waiting time for orthopedic surgery was reduced from 14 weeks to 31 hours (from first call to surgery); improved inpatient satisfaction scores of “very satisfied” rose from 68% to 90%.

8. In Avera McKennan, SD [3] the patient length of stay was reduced by 29%, and \$1.25 million in new ED construction was avoided.

9. In Denver Health, CO [3] the bottom-line Lean benefit was increased by \$54 million through cost reduction and revenue growth, and layoffs were avoided.

10. In Seattle Children’s Hospital, WA \$180 million in capital spending was avoided through Lean improvements.

These examples demonstrate that Lean is successful in cost and throughput time reductions, and improvements in quality and patient and staff satisfaction. The improvements of this level are possible, even routine – because the amount of initially-invisible waste in traditional healthcare organizations is so high. The broad range of operations described in the examples manifest that Lean is applicable across the board to healthcare operations, without limitations.

Education in Lean Healthcare

Increasingly, Lean Healthcare becomes an inherent part of Healthcare Systems Engineering (HSE) Master's Programs, e.g. (Loyola Marymount University 2016) which has been developed in collaboration with Kaiser Permanente. The program includes two courses in Lean, basic and advanced, focused on improving operations in clinics, hospitals, emergency departments, clinical laboratories, radiology testing, operating rooms, pharmacies, supply chain, and healthcare administration. After the basic courses in systems engineering, project management, and systems thinking, the students also take courses on healthcare system architecting, modeling and simulations; medical data mining and analytics; systems engineering for medical devices, healthcare enterprise informatics; and healthcare delivery systems. All these advanced courses contain elements of Lean thinking because all these subdomains risk being burdened with waste and poor quality if Lean is ignored. Simply put, Lean is not really an optional extra if you want to achieve efficiency and effectiveness.

References

Works Cited

- B.W. Oppenheim, B.W. 2015. "Lean Healthcare," INCOSE Healthcare Working Group webinar. San Diego, CA: International Council on Systems Engineering. April 30, 2015. Available at: <https://onedrive.live.com/redir?resid=147E5C4249DA0EFB%21142>
- Berwick, D. 2009, "National Forum Keynote, Institute for Healthcare Improvement." Cambridge, MA: Institute for Healthcare Improvement. Available at: <https://www.ihl.org/IHI/Programs/AudioAndWebPrograms/BerwickForumKeynote2009.htm> (accessed July 4, 2011)
- Graban, M. 2012. Lean Hospitals; Improving Quality, Patient Safety, and Employee Engagement. Boca Raton, FL: CRC Press.
- Kanter, M.K. 2015. "Strategic Partnership of Healthcare and Systems Engineering." San Diego: INCOSE Healthcare Working Group presentation, 2015
- Loyola Marymount University. 2016. "MS Degree Program in Healthcare Systems Engineering." Available at: CSE.lmu.edu/graduateprograms/systemsengineering/healthcaresystemsengineeringms/
- Medinilla, Á. 2014. "Agile Kaizen: Managing Continuous Improvement Far Beyond Retrospectives." New York, NY: Springer, 2014
- Murman, E. 2010 "The Lean Aerospace Initiative." Boston MA: Lean Advancement Initiative (LAI) Annual Conference.
- Oehmen, J. 2012. The Guide to Lean Enablers for Managing Engineering Programs. PMI-INCOSE-LAI MIT. May 2012.
- OMG. 2016. "MBSE Wiki." Available at: <https://www.omgwiki.org/MBSE/doku.php> (last accessed March 29, 2016)
- Oppenheim, B.W. 2011. Lean for Systems Engineering with Lean Enablers for Systems Engineering. Hoboken, NJ: Wiley Series in Systems Engineering and Management.
- Porter, M. 2010. "What is Value in Healthcare?" New England Journal of Medicine. 363: 2488-2481. 08 December 2010.
- Toussaint, J. and R. Gerard. 2010. On the Mend: Revolutionizing Healthcare to Save Lives and Transform the Industry. Cambridge, MA: Lean Enterprise Institute. 06 June 2010.
- Womack, J.P. and D. T. Jones. 2003. Lean Thinking: Banish Waste and Create Wealth in Your Corporation. Washington, DC: Free Press.
-

Primary References

None.

Additional References

None.

Part 5: Enabling Systems Engineering

Enabling Systems Engineering

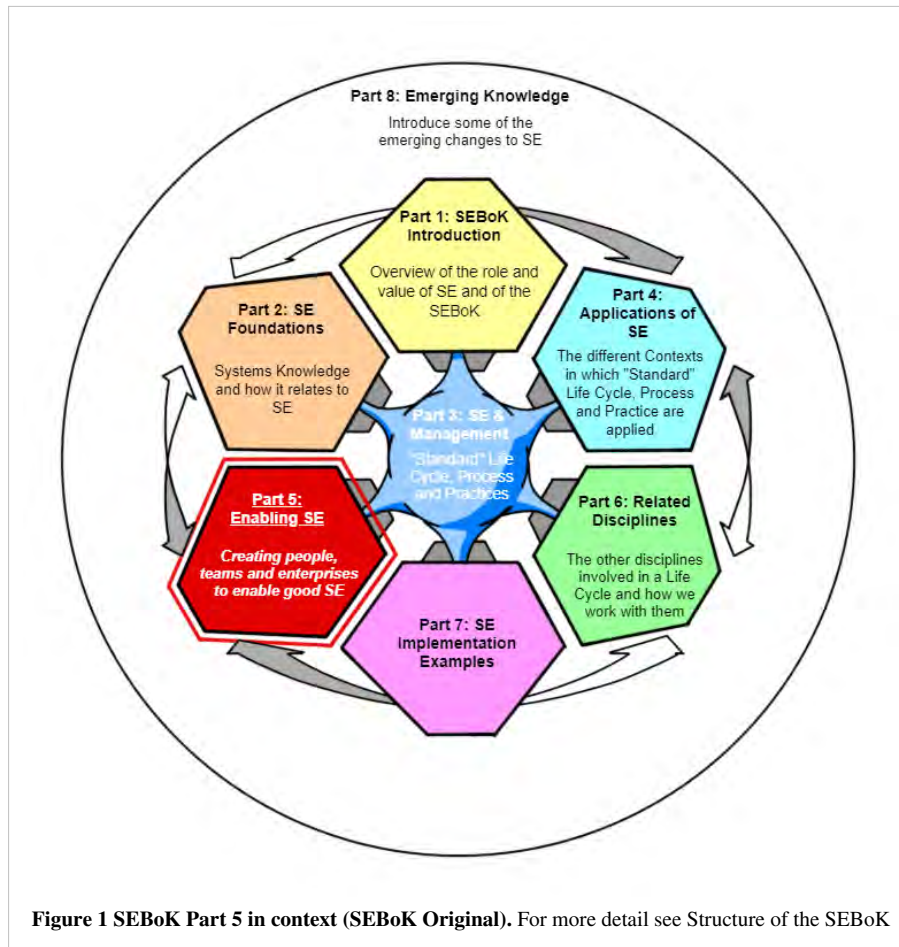
Contents of this Part

- Enabling Businesses and Enterprises (Art Pyster, Deva Henry, and Dave Olwell)
- Enabling Teams (Dick Fairley) (Alice Squires and Art Pyster)
- Enabling Individuals (Heidi Davidz and Dick Fairley) (Alice Squires and Art Pyster)
- Lead Authors:
 - Art Pyster, Hillary Sillitto, and Alice Squires
- Contributing Authors:
 - Dick Fairley, Bud Lawson, Dave Olwell, Deva Henry, and Rick Adcock

Part 5 of the Guide to the SE Body of Knowledge (SEBoK) is a guide to knowledge about how an enterprise prepares and positions itself to effectively perform the systems engineering (SE) activities described elsewhere in the SEBoK.

SE activities — how to develop requirements, select an appropriate life cycle model, and architect a system of systems, and so on — are covered elsewhere, especially in Part 3, Systems Engineering and Management. An organization that desires to do these things effectively must work through questions like whether to allow a project manager to select the systems engineers he or she employs, and, if so, what competencies the project manager might seek in those systems engineers. These are the kinds of questions that Part 5 explores.

The discussion defines three levels of organization: enterprise or organization, team, and individual. To adapt an example to a more complex organizational structure, simply decompose enterprises into sub-enterprises and teams into sub-teams, as needed. For more about the different types of enterprises, see Engineered Systems in Part 2.



Knowledge Areas in Part 5

Each part of the SEBoK is composed of knowledge areas (KA). Each KA groups topics around a theme related to the overall subject of the part.

The KAs in Part 5 explore how to the performance of systems engineering from three different lenses:

- Enabling Businesses and Enterprises
- Enabling Teams
- Enabling Individuals

Common Practices

There are as many different ways to enable SE performance as there are organizations, and every organization's approach is detailed and unique. Nevertheless, common practices, methods, and considerations do exist. Part 5 uses them as a framework to structure the relevant knowledge.

SE activities that support business needs and deliver value are enabled by many factors, including:

- Culture (see Culture),
- SE competencies (see Determining Needed Systems Engineering Capabilities in Businesses and Enterprises) and how the organization grows and deploys its workforce to acquire them, and
- SE tooling and infrastructure (see Systems Engineering and Management in Part 3).

Enterprises and Businesses

The fact that Part 5 uses two terms, “Enterprise” and “Business,” to name a single level of organization, indicates that the two are closely related. In many contexts it is not necessary to make any distinction between them: an enterprise may be a traditional business, and a business can be seen as a special type of enterprise. For the sake of brevity, the more general term “organization” may be used to mean “business or enterprise” throughout Part 5.

Traditional businesses usually have a legal structure and a relatively centralized control structure. Such a business may be a corporation, or a unit of a company or government agency, that creates a product line or offers services.

On the other hand, an enterprise can be structured in a way that excludes description as a business. This happens when an enterprise crosses traditional business boundaries, lacks a centralized legal authority, and has relatively loose governance. One example is the healthcare “system” in the US which encompasses hospitals, insurance companies, medical equipment manufacturers, pharmaceutical companies, and government regulators. Another is the set of companies that form the supply chain for a manufacturer, such as the thousands of companies whose parts and services Apple uses to create, distribute, and support the iPhone.

Significant actions that enable SE are often conducted by traditional businesses rather than by less tightly structured enterprises. Even so, organizational context affects how the business approaches SE and therefore how it enables SE performance. A business that sells to the general commercial marketplace typically has far fewer constraints on its SE practices than one which performs contract work for a government agency. A business that creates systems with very demanding characteristics, such as aircraft, typically has a much more rigorous and planned approach to SE than one which creates less demanding systems, such as a smartphone app.

Traditional businesses are intended to be permanent, and typically offer a portfolio of products and services, introduce new ones, retire old ones, and otherwise seek to grow the value of the business. Sometimes a single product or service has such value and longevity that it spawns a business or enterprise just for its creation, maintenance, and support. The Eurofighter Typhoon aircraft, for example, was developed by a consortium of three corporations that formed a holding company specifically to provide support and upgrade services throughout the in-service life of the aircraft.

For more on the distinction between businesses and enterprises and the value of systems engineering of enterprises to them, see Enterprise Systems Engineering in Part 4. Systems of Systems (SoS), also in Part 4, contrasts the tighter control over SE that is usual for businesses with the looser control that is usual for enterprises lacking a traditional business structure.

Teams

Teams operate within the context of the businesses in which they reside. This context determines how the team is enabled to perform SE.

For example, a business may grant a team wide autonomy on key technical decisions, which are made either by team systems engineers or in consultation with team systems engineers. On the other hand, the same business could instead create a generic set of SE processes that all teams are to tailor and use, constraining the team to adhere to established business policies, practices, and culture. The business could even require that the team gain approval for its tailored SE process from a higher-level technical authority.

Teams are usually formed for a limited duration to accomplish a specific purpose, such as creating a new system or upgrading an existing service or product. Once the purpose has been fulfilled, the team responsible for that effort is usually disbanded and the individuals associated with the effort are assigned to new tasks. Exceptions do happen, however. For example, a team of systems engineers tasked with assisting troubled programs throughout a corporation could persist indefinitely.

References

Works Cited

None.

Primary References

None.

Additional References

None.

Knowledge Area: Enabling Businesses and Enterprises

Enabling Businesses and Enterprises

Contents of this Knowledge Area

- Systems Engineering Organizational Strategy (Alice Squires, Dick Fairley, and Hillary Sillitto) (Art Pyster, Alan Faisandier, Deva Henry, and Rick Adcock)
- Determining Needed Systems Engineering Capabilities in Businesses and Enterprises (Richard Beasley, Hillary Sillitto, and Scott Jackson) (Alice Squires, Heidi Davidz, Garry Roedler, Richard Turner, Art Pyster, and Ray Madachy)
- Organizing Business and Enterprises to Perform Systems Engineering (Richard Beasley, Art Pyster, and Hillary Sillitto) (Alice Squires, Heidi Davidz, Scott Jackson, and Quong Wang)
- Assessing Systems Engineering Performance of Business and Enterprises (Hillary Sillitto, Alice Squires, and Heidi Davidz) (Art Pyster and Richard Beasley)
- Developing Systems Engineering Capabilities within Businesses and Enterprises (Richard Beasley, Hillary Sillitto, and Alice Squires) (Heidi Davidz and Art Pyster)
- Barriers to Successful Embedding of Systems Engineering into Organizations (Bernardo Delicado) (Tom Strandberg, Ivan Mactaggart)
- Culture (Scott Jackson, Hillary Sillitto, and John Snoderly) (Richard Turner, Art Pyster, and Richard Beasley)
- Lead Authors:
- Art Pyster, Deva Henry, and Dave Olwell

Part 5 on Enabling Systems Engineering explores how systems engineering (SE) is enabled at three levels of an organization: the business or enterprise (hereafter usually just called "business" --- See Enabling Systems Engineering for more information), the team, and individuals.

The **Enabling Businesses and Enterprises** Knowledge Area describes the knowledge needed to enable SE at the top level of the organization. Part 3, Systems Engineering and Management, describes how to perform SE once it has been enabled using the techniques described in Part 5. Moreover, a business is itself a system and can benefit from being viewed that way. (See Enterprise Systems Engineering in Part 4.)

Relationship Among Topics

- Systems Engineering Organizational Strategy describes how SE delivers value to the business, who makes decisions about SE in the business, how those decisions are made, how resources are allocated, and how the soundness and performance of those decisions are monitored.
 - Determining Needed Systems Engineering Capabilities in Businesses and Enterprises describes how a business decides what specific SE capabilities are needed; e.g., a business that creates cutting edge products would likely require very strong architecting capabilities, including modeling tools. A business that has a global development team would likely need a very robust collaboration toolset.
 - Organizing Business and Enterprises to Perform Systems Engineering describes various organizational models; e.g., which SE functions should be centralized, which should be distributed, how much SE every engineer should know.
-

- Assessing Systems Engineering Performance of Business and Enterprises describes how a business understands how well it is doing with respect to the SE actually being performed using the techniques described in Systems Engineering and Management.
- Developing Systems Engineering Capabilities within Businesses and Enterprises describes how SE talent that delivers the desired SE capabilities is grown and acquired
- Barriers to Successful Embedding of Systems Engineering into Organizations describes the complexities surrounding the embedding of systems engineering into organizations.
- Finally, Culture describes how the culture of a business affects SE; e.g., a risk-averse business will likely use plan-driven SE processes; an entrepreneurial, fast-paced business will likely use agile SE processes (See Life Cycle Models).

To some extent, these topics have the character of a "plan-do-check-act" cycle, where the "do" part of the cycle is performing SE using the techniques described in Part 3, Systems Engineering and Management (Deming Part 3). For example, if assessing the business' SE performance shows shortfalls, then additional SE capabilities may need to be developed, the organization may need to be adjusted, processes may need to be improved, etc., all working within the existing cultural norms. If those norms prevent the business from successfully performing SE, then transformational efforts to change the culture may be needed as well.

References

Works Cited

Deming, W.E. 1994. *The New Economics*. Cambridge, MA, USA: Massachusetts Institute of Technology, Centre for Advanced Educational Services.

Primary References

Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. Hoboken, NJ, USA: John Wiley and Sons.

Elliott, C. et al. 2007. *Creating Systems That Work – Principles of Engineering Systems for the 21st Century*. London, UK: Royal Academy of Engineering. Accessed September 2, 2011. Available at https://www.raeng.org.uk/education/vps/pdf/RAE_Systems_Report.pdf.

Hofstede, G. 1984. *Culture's Consequences: International Differences in Work-Related Values*. London, UK: Sage.

Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College, UK.

Morgan, J. and J. Liker. 2006. *The Toyota Product Development System: Integrating People, Process and Technology*. New York, NY, USA: Productivity Press.

Rouse, W. 2006. *Enterprise Transformation: Understanding and Enabling Fundamental Change*. Hoboken, NJ, USA: John Wiley and Sons.

Senge, P. M. 2006. *The Fifth Discipline: The Art and Practice of the Learning Organization*, 2nd ed. New York, NY, USA: Currency Doubleday.

Shenhar, A.J. and D. Dvir. 2007. *Reinventing Project Management: The Diamond Approach to Successful Growth and Innovation*. Boston, MA, USA: Harvard Business School Publishing.

Additional References

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2015.

Systems Engineering Organizational Strategy

- Lead Authors:
- Alice Squires, Dick Fairley, and Hillary Sillitto
- Contributing Authors:
- Art Pyster, Alan Faisandier, Deva Henry, and Rick Adcock

Virtually every significant business or enterprise that creates products or services benefits from performing a wide variety of systems engineering (SE) activities to increase the value that those products and services deliver to its owners, customers, employees, regulators, and other stakeholders. (See Stakeholder Needs Definition.)

A business is a specific type of enterprise, usually a legal entity with a management structure that allows for relatively tight control of its components, including how it enables SE. The term business is often used in this article in lieu of enterprise because specific actions to enable SE are typically done by businesses. This is discussed further in the parent article Enabling Systems Engineering. The strategy for organizing to conduct SE activities is important to their effectiveness. For example, every enterprise has a purpose, context, and scope determined by some of its stakeholders and modified over time to increase the value the enterprise offers to them.

Some enterprises are for-profit businesses. Others are not-for-profit businesses that work for the public good. Still others are non-traditional businesses, but more loosely structured entities without legal structure, such as a national healthcare system. Some enterprises are located at a single site, while some others are far-flung global "empires". Some work in highly regulated industries such as medical equipment, while others work with little government oversight and can follow a much wider range of business practices. All these variations shape the strategy for performing SE.

Primary Considerations

SE organizational strategy is driven by the goals of the business and the resources and constraints available to achieve those goals. SE strategy in particular is influenced by several considerations:

- The purpose of the business
 - The value the business offers its stakeholders; e.g., profits, public safety, entertainment, or convenience
 - The characteristics of the system which the SE activities support; e.g., the size, complexity, primary design factors, major components, required products, critical specialties, or areas of life cycle
 - The phases of the life cycle in which the SE activities are being performed; e.g., development, deployment, operations, or maintenance of a product or service
 - The scale of the business, the systems and services of interest; e.g., is it a single site company or a global venture? Is the business creating a relatively modest product for internal use, such as a new Web application to track employee training, or a new hybrid automobile complete with concerns for engineering, manufacturing, servicing, and distribution?
-

- The culture of the business in which the SE activities are performed; e.g., is the business risk-averse? Do people normally collaborate or work in isolated organizations?
- The business structure and how well the current structure aligns with what is needed to create new products and services; e.g., does the structure of the business align with the architecture of its major products and services?
- The degree of change or transformation that the business is undertaking in its operation, products, and markets

Rouse (2006) offers a thorough look at enterprise strategy, especially as it relates to delivering value to the enterprise in various phases of the life cycle, beginning with research and development through operations. Rouse provides a number of techniques to determine and improve the value offered to enterprises using SE methods, especially useful when an enterprise is undergoing significant transformation rather than conducting "business as usual"; e.g., the enterprise could be trying to:

- do current business better (drive down costs or improve quality of its current products and services);
- cope with a disruption in the market, a competitive threat, or changing customer expectations and ways of doing business;
- reposition itself in its value chain (move from being a part supplier to a subassembly supplier); or
- launch a new generation product or enter a new market.

Eisner (2008) provides a thorough look at different SE organizational approaches.

Systems Engineering Strategy Elements

Based on the primary considerations, the SE strategy generally addresses the following:

- How SE activities provide value to the business (See Economic Value of Systems Engineering)
- How SE activities are allocated among the various business entities (See Organizing Business and Enterprises to Perform Systems Engineering)
- What competencies are expected from the parts of the business in order to perform these SE activities (See Deciding on Desired Systems Engineering Capabilities within Businesses and Enterprises)
- How parts of the business gain and improve competencies (See Developing Systems Engineering Capabilities within Businesses and Enterprises)
- Who performs SE activities within each part of the business (See Team Capability)
- How people who perform SE activities interact with others in the business ((See Part 6: Related Disciplines)
- How SE activities enable the business to address transformation (See Enterprise Systems Engineering).

Depending on the business' approach to SE, there may not be a single coherent SE strategy common across the business. Different business units may have their own SE strategies, or development of a strategy may be delegated to individual projects. The SE strategy may not even be explicitly documented or may only be found in multiple documents across the business. Some businesses publish guidebooks and policies that describe their organizational strategy. These are usually proprietary unless the business is a government or quasi-government agency. Two public documents are NASA (2007) and MITRE (2012). The latter has a number of short articles on different topics including an article on Stakeholder Assessment and Management and another on Formulation of Organizational Transformation Strategies.

Product and Service Development Models

There are three basic product and service development models that most businesses employ:

1. Market-driven commercial
2. Product-line
3. Contract

The biggest differences between the three business models are where requirements risks lie and how user needs and usage are fed into the design and delivery process. SE support to the business varies in each case.

Market-driven commercial products and services are sold to many customers and are typically developed by organizations at their own risk. The requirements come from marketing based on understanding the market, relevant regulation and legislation, and good ideas from within the organization (Pugh 1991, Smith and Reinertsen 1997). Sillitto (1999) contends that market-driven commercial product development is a form of systems engineering with adapted techniques for requirements elicitation and validation.

Product-line products and services are variants of the same product and service, usually customized for each customer. Extra investment is required to create the underlying product platform. Architecting such a platform in a way that supports cost-effective customization is usually more complex both technically and organizationally than market-driven commercial products and services.

Systems engineers typically play a central role in establishing the platform architecture, understanding the implications of platform choices on manufacturing and service, etc. There are a number of examples of good practices in product-line products and services; e.g., automobile models from virtually all major manufacturers such as Toyota, General Motors, and Hyundai; Boeing and Airbus aircraft such as the B-737 family and the Airbus 320 family; and Nokia and Motorola cellphones. The Software Engineering Institute has done extensive research on product lines for software systems and has developed a framework for constructing and analyzing them (Northrop et al. 2007). For a reference on product line principles and methods, see Simpson (et al. 2006).

Contract products and services often demand tailor-made system/service solutions which are typically specified by a single customer to whom the solution is provided. The supplier responds with proposed solutions. This style of development is common in defense, space, transport, energy, and civil infrastructure. Customers that acquire many systems often have a specific procurement organization with precise rules and controls on the acquisition process, and mandated technical and process standards. The supplier typically has much less flexibility in SE process, tools, and practices in this model than the other two.

Any single business or enterprise is likely to apply some combination of these three models with varying importance given to one or more of them.

Organizations That Use and Provide SE

There are five basic types of organizations that use SE or provide SE services:

1. A business with multiple project teams
2. A project that spans multiple businesses
3. An SE team within either of the above
4. A business with a single project team
5. An SE service supplier that offers a specific SE capability or service (tools, training, lifecycle process) to multiple clients, either as an external consultancy or as an internal SE function

The kind of business determines the scope and diversity of SE across the organization. This is shown in abstract form in Figure 1, which illustrates the fundamental form of an extended enterprise. This also shows how organizational structure tends to match system structure.

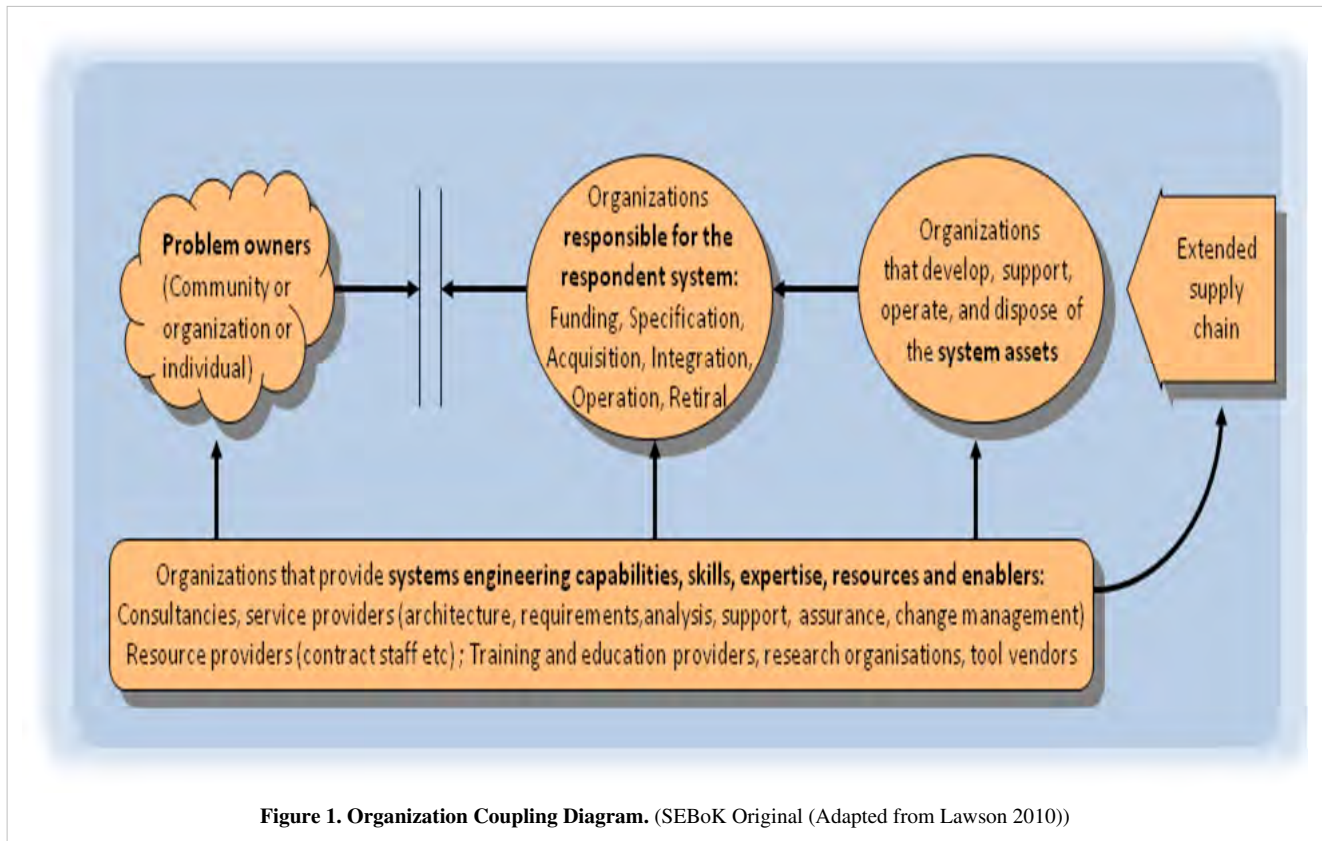
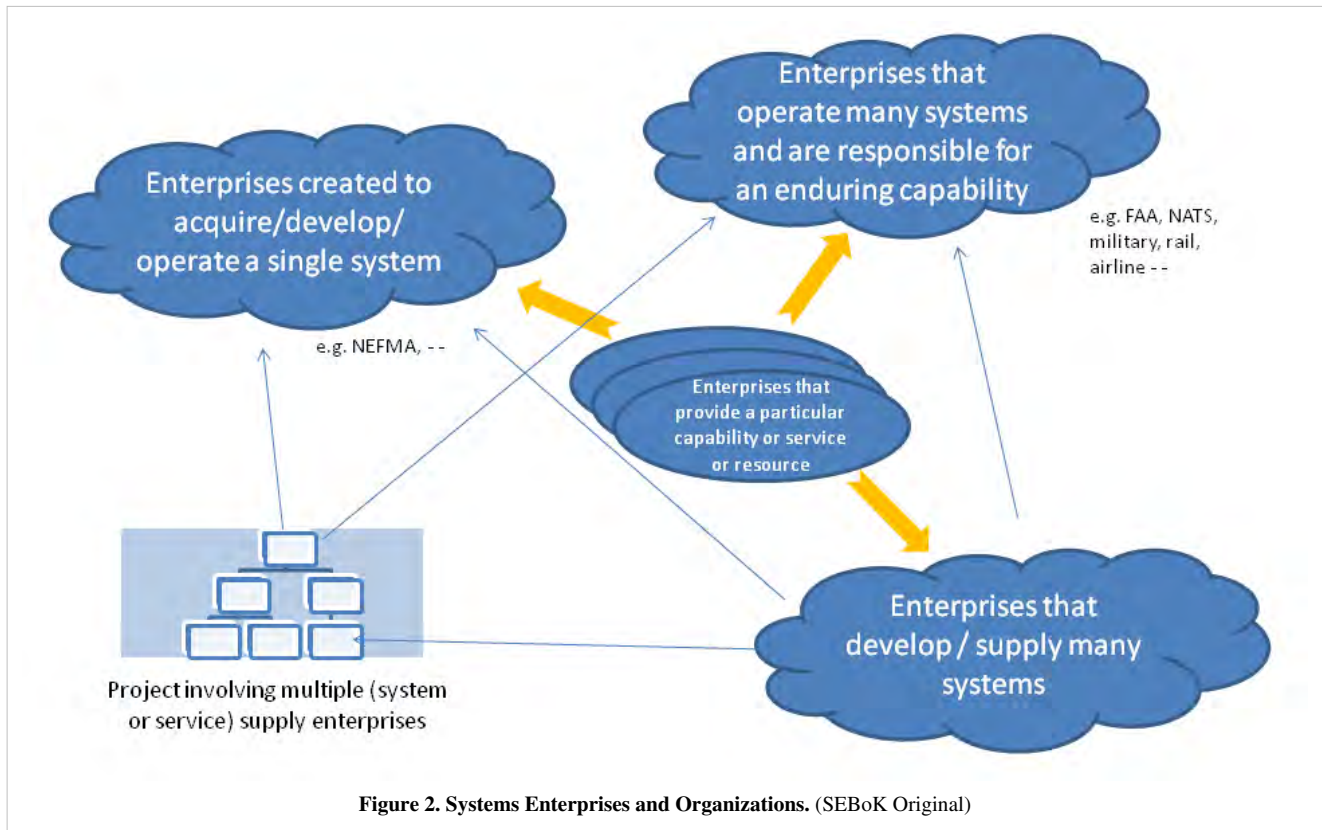


Figure 1. Organization Coupling Diagram. (SEBoK Original (Adapted from Lawson 2010))

The *problem owners* are the people, communities, or organizations involved in and affected by the *problem situation*. They may be seeking to defend a country, to improve transportation links in a community, or to deal with an environmental challenge. The *respondent system* might be a new fighter aircraft, a new or improved transportation infrastructure, or a new low-emission electricity generation systems (respectively). The organizations responsible for the respondent systems would be the Air Force, transport operator or regulator, or electricity supply company. The prime role of these organizations would be to operate the systems of interest to deliver value to the problem owners. They might reasonably be expected to manage the entire system lifecycle.

This same concept is expanded in Figure 2.



Goals, Measures, and Alignment in a Business

The alignment of goals and measures within the business strongly affects the effectiveness of SE and the benefit delivered by SE to the business, and needs to be carefully understood:

- Blockley and Godfrey (2000) describe techniques used successfully to deliver a major infrastructure contract on time and within budget, in an industry normally plagued by adversarial behavior.
- Lean thinking provides a powerful technique for aligning purpose to customer value – provided the enterprise boundary is chosen correctly and considers the whole value stream (Womack and Jones 2003; Oppenheim et al. 2010).
- Fasser and Brettner (2002, 18-19) see an organization as a system, and advocate three principles for organizational design: (1) increasing value for the ultimate customer, (2) strict discipline, and (3) simplicity.
- EIA 632 (ANSI/EIA 2003) advocates managing all the aspects required for the life cycle success of each element of the system as an integrated “building block”. Similarly, Blockley (2010) suggests that taking a holistic view of “a system as a process” allows a more coherent and more successful approach to organization and system design, considering each element both as part of a bigger system-of-interest and as a “whole system” (a “holon”) in its own right.
- Elliott et al. (2007) advocate six guiding principles for making systems that work: (1) debate, define, revise and pursue the purpose, (2) think holistically, (3) follow a systematic procedure, (4) be creative, (5) take account of the people, and (6) manage the project and the relationships.
- For organizations new to SE, the INCOSE UK Chapter has published a range of one-page guides on the subject, including Farncombe and Woodcock (2009a; 2009b).

Governance

SE governance is the process and practice through which a business puts in place the decision rights that enable SE to deliver as much business value as possible. Those rights may be codified in policy, implemented through the business structure, enforced through tools, and understood through measures of compliance and effectiveness.

SE governance in large businesses is often explicit and codified in policy. In small businesses, it is often tacit and simply understood in how the business works. One of the key implementation steps when a business defines its SE strategy is to establish its SE governance model, which should be tailored to the particular context in which the business operates and delivers value. Of course, in practice, this is often incremental, uneven and subject to wide swings based on the current state of the business and the people occupying key management positions.

The term governance for development organizations was first popularized in reference to how Information Technology (IT) is overseen in businesses and enterprises (Weill and Ross 2006; Cantor and Sanders 2007). The recognition in the 1990s and the last decade that IT is a fundamental driver of performance and value for most corporations and government agencies led to the transformation of the Chief Information Officer (CIO) into a key senior manager.

Explicit governance of IT became important to enabling an enterprise to respond to new technology opportunities, emerging markets, new threats, and rapid delivery of new products and services. The term "governance" is now widely used to describe how SE is woven into an enterprise. Governance becomes especially challenging for complex projects in which there are high levels of uncertainty (Cantor 2006) or for system of systems projects in which responsibility for major decisions may be distributed over multiple organizations within an enterprise in which there is no single individual who is "in control" (see Systems of Systems (SoS)). Morgan and Liker (2006) describe the governance model for Toyota, which is one of the largest companies in the world.

SE governance establishes the framework and responsibility for managing issues such as design authority, funding and approvals, project initiation and termination, as well as the legal and regulatory framework in which the system will be developed and will operate. Governance includes the rationale and rules for why and how the enterprise policies, processes, methods and tools are tailored to the context. SE governance may also specify product and process measures, documentation standards, and technical reviews and audits.

The ways in which a team organizes to conduct SE activities either conform to policies established at the level above or are captured in that team's own governance policies, processes, and practices. These policies cover the organizational context and goals, the responsibilities for governance, process, practices and product at the level of interest, and the freedom delegated to and governance and reporting obligations imposed on lower organizational levels. It is good practice to capture the assignment of people and their roles and responsibilities in the form of the Responsible, Accountable, Consult, Inform (RACI) matrix (PMI 2013) or something similar. Responsibility in large organizations can easily become diffused. Sommerville et al. (2009, 515-529) discuss the relationship between information and responsibility, and describe methods to analyze and model responsibility in complex organizations.

Small organizations tend to have relatively informal governance documentation and processes, while larger organizations tend towards more structure and rigor in their governance approach. Government organizations responsible for developing or acquiring large complex systems, such as the US Department of Defense or the US Federal Aviation Administration, usually develop policies that describe governance of their SE activities and SE organizations. See DoD (2012) for the Department of Defense SE policies.

Government contracting typically brings additional regulation and oversight, driving a group to greater rigor, documentation, and specific practices in their SE governance. Development of systems or operating services that affect public safety or security is subject to constraints similar to those seen in government contracting. Think of the creation of medical devices or the operation of emergency response systems, air traffic management, or the nuclear industry. (See Jackson (2010) for example).

Governance models vary widely. For example, Linux, the greatest success of the open source community, has a governance model that is dramatically different than those of traditional businesses. Smith (2009) offers a cogent explanation of how decisions are made on what goes into the Linux kernel. All of the decision rights are completely transparent, posted on the Linux website, and have proven remarkably effective as they have evolved. The classic paper *The Cathedral and The Bazaar* by Eric Raymond (2000) provides great insight into the evolution of Linux governance and how Linus Torvalds responded to changing context and circumstances to keep Linux so successful in the marketplace with a governance model that was radically novel for its time.

The project management literature also contributes to the understanding of SE governance (see Systems Engineering and Project Management). For example, Shenhar and Dvir (2007) offer the "diamond model" for project management, which identifies four dimensions that should guide how development projects are managed: novelty, technology, complexity, and pace. Application of this model to SE governance would influence the available life cycle models for development projects and how those models are applied.

There are numerous examples of projects that went well or badly based largely on the governance practiced by both the acquirer and the supplier organizations. Part 7 of the SEBoK has several examples, notably Singapore Water Management (went well) and FAA Advanced Automation System (AAS) (went less well).

References

Works Cited

- ANSI/EIA. 2003. *Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA). ANSI/EIA 632-1998.
- Blockley, D. 2010. "The Importance of Being Process." *Journal of Civil Engineering and Environmental Systems*. 27(3).
- Blockley, D. and Godfrey, P. 2000. *Doing It Differently – Systems for Rethinking Construction*. London, UK: Thomas Telford, Ltd.
- Cantor, M. 2006. "Estimation Variance and Governance." In IBM developerWorks. Accessed on April 24, 2013. Available at Abstract ^[1].
- Cantor, M. and J.D. Sanders. 2007. "Operational IT Governance." In IBM developerWorks. Accessed on September 15, 2011. Available at https://www.ibm.com/developerworks/rational/library/may07/cantor_sanders/.
- DoD. 2012. "Systems Engineering Policy". Accessed on August 4, 2012. Available at <https://www.acq.osd.mil/se/pg/index.html>.
- Eisner, H. 2008. "Essentials of Project and Systems Engineering Management", 3rd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Elliott, C. et al. 2007. *Creating Systems That Work – Principles of Engineering Systems for the 21st Century*. London, UK: Royal Academy of Engineering. Accessed September 2, 2011. Available at https://www.raeng.org.uk/education/vps/pdf/RAE_Systems_Report.pdf.
- Fasser, Y. and D. Brettner. 2002. *Management for Quality in High-Technology Enterprises*. Hoboken, NJ, USA: John Wiley & Sons-Interscience.
- Farncombe, A. and H. Woodcock. 2009a. "Enabling Systems Engineering". *Z-2 Guide*, Issue 2.0. Somerset, UK: INCOSE UK Chapter. March, 2009. Accessed September 2, 2011. Available at https://www.incoseonline.org.uk/Documents/zGuides/Z2_Enabling_SE.pdf.
- Farncombe, A. and H. Woodcock. 2009b. "Why Invest in Systems Engineering". *Z-3 Guide*, Issue 3.0. Somerset, UK: INCOSE UK Chapter. March 2009. Accessed September 2, 2011. Available at https://www.incoseonline.org.uk/Documents/zGuides/Z3_Why_invest_in_SE.pdf.

- Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Hoboken, NJ, USA: John Wiley & Sons.
- Lawson, H. 2010. "A Journey Through the Systems Landscape". London, UK: College Publications, Kings College, UK.
- MITRE. 2012. "Systems Engineering Guidebook". Accessed on August 4, 2012. Available at https://www.mitre.org/work/systems_engineering/guide/index.html.
- Morgan, J. and J. Liker. 2006. *The Toyota Product Development System: Integrating People, Process and Technology*. New York, NY, USA: Productivity Press.
- NASA. 2007. "NASA Systems Engineering Handbook". Accessed on April 24, 2013. Available at <https://www.acq.osd.mil/se/docs/NASA-SP-2007-6105-Rev-1-Final-31Dec2007.pdf>. Washington, DC, USA: NASA.
- Northrop, L., P. Clements, et al. 2007. *A Framework for Software Product Line Practice*, Version 5.0. With F. Bachmann, J. Bergey, G. Chastek, S. Cohen, P. Donohoe, L. Jones, R. Krut, R. Little, J. McGregor, and L. O'Brien. Pittsburgh, PA, USA: Software Engineering Institute.
- Oppenheim, B., E.M. Murman, D.A. Secor. 2010. Lean Enablers for Systems Engineering. *Systems Engineering*. 14(1): 29-55.
- PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- Pugh, S. 1991. *Total Design: Integrated Methods for Successful Product Engineering*. New York, NY, USA: Addison-Wesley.
- Raymond, E.S. 2000. *The Cathedral and The Bazaar*, version 3.0. Accessed on April 24, 2013. Available at <https://www.catb.org/esr/writings/homesteading/cathedral-bazaar/cathedral-bazaar.ps>.
- Rouse, W. 2006. "Enterprise Transformation: Understanding and Enabling Fundamental Change." Hoboken, NJ, USA: John Wiley & Sons.
- Shenhar, A.J. and D. Dvir. 2007. *Reinventing Project Management: The Diamond Approach to Successful Growth and Innovation*. Boston, MA, USA: Harvard Business School Publishing.
- Sillitto, H. 1999. "Simple Simon Met A System". Proceedings of the 9th Annual International Council on Systems Engineering (INCOSE) International Symposium, 6-10 June, 1999, Brighton, UK.
- Simpson, T.W., Z. Siddique, R.J. Jiao (eds.). 2006. *Product Platform and Product Family Design: Methods and Applications*. New York, NY, USA: Springer Science & Business Media, Inc.
- Smith, J.T. 2009. "2.4 Kernel: How are Decisions Made on What Goes into The Kernel?" Available at <https://www.linux.com/feature/8090>.
- Smith, P.G. and D.G. Reinertsen. 1997. *Developing Products in Half the Time*. New York, NY, USA: Wiley and Sons.
- Sommerville, I., R. Lock, T. Storer, and J.E. Dobson. 2009. "Deriving Information Requirements from Responsibility Models." Paper presented at 21st International Conference on Advanced Information Systems Engineering, Amsterdam, Netherlands.
- Weill, P. and J.W. Ross. 2004. *IT Governance: How Top Performers Manage IT Decision Rights for Superior Results*. Boston, MA, USA: Harvard Business School Publishing.
- Womack, J. and D. Jones. 2003. *Lean Thinking: Banish Waste and Create Wealth in Your Corporation*, Revised Edition. New York, NY, USA: Simon & Schuster.
-

Primary References

- Blockley, D. and Godfrey, P. 2000. *Doing It Differently – Systems for Rethinking Construction*. London, UK: Thomas Telford, Ltd.
- Cantor, M. and J.D. Sanders. 2007. "Operational IT Governance." In IBM developerWorks. Accessed on September 15, 2011. Available at https://www.ibm.com/developerworks/rational/library/may07/cantor_sanders/.
- Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Elliott, C., et al. 2007. *Creating Systems That Work – Principles of Engineering Systems for the 21st Century*. London, UK: Royal Academy of Engineering. Accessed September 2, 2011. Available at https://www.raeng.org.uk/education/vps/pdf/RAE_Systems_Report.pdf.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College, UK.
- Morgan, J. and J. Liker. 2006. *The Toyota Product Development System: Integrating People, Process and Technology*. New York, NY, USA: Productivity Press.
- Northrop, L., P. Clements, et al. 2007. *A Framework for Software Product Line Practice*, version 5.0. With F. Bachmann, J. Bergey, G. Chastek, S. Cohen, P. Donohoe, L. Jones, R. Krut, R. Little, J. McGregor, and L. O'Brien. Pittsburgh, PA, USA: Software Engineering Institute. Accessed on April 25, 2013. Available at https://www.sei.cmu.edu/productlines/frame_report/index.html.
- Rouse, W. 2006. *Enterprise Transformation: Understanding and Enabling Fundamental Change*. Hoboken, NJ, USA: John Wiley & Sons.
- Shenhar, A.J. and D. Dvir. 2007. *Reinventing Project Management: The Diamond Approach to Successful Growth and Innovation*. Boston, MA, USA: Harvard Business School Publishing.

Additional References

- Chastek, D., P. Donohoe, and J.D. McGregor. 2009. *Formulation of a Production Strategy for a Software Product Line*. Pittsburgh, PA, USA: Software Engineering Institute, CMU/SEI-2009-TN-025. Accessed on September 14, 2011. Available at <https://www.sei.cmu.edu/reports/09tn025.pdf>.
- Sillitto, Mazzella, and Fromenteau. 2001. "The development of Product Lines in THALES: methods, examples, lessons learnt," Paper presented at the INCOSE UK Spring Conference.

References

- [1] https://www.ibm.com/developerworks/library/?sort_by=&show_abstract=true&show_all=&search_flag=&contentarea_by=All+Zones&search_by=Estimation+Variance+and+Governance&product_by=-1&topic_by=-1&industry_by=-1&type_by=All+Types&ibm-search=Search

Determining Needed Systems Engineering Capabilities in Businesses and Enterprises

- Lead Authors:
- Richard Beasley, Hillary Sillitto, and Scott Jackson
- Contributing Authors:
- Alice Squires, Heidi Davidz, Garry Roedler, Richard Turner, Art Pyster, and Ray Madachy

Enabling a business or enterprise to perform systems engineering (SE) well requires deciding which specific SE capabilities the business or enterprise needs in order to be successful. (In the rest of this article business or enterprise is usually abbreviated to just "business", because a business is a specific type of enterprise that has sufficiently strong central authority and motivation to take steps to enable SE). SE capabilities should support the Systems Engineering Organizational Strategy and reflect the nature of the business, its products and services, various stakeholders, business leadership focus, etc.

This topic, which is part of the Enabling Businesses and Enterprises knowledge area (KA) of Part 5, summarizes the factors used to decide which SE capabilities a business needs; e.g., the interactions between SE and other functional areas in the business, and consideration of social dynamics and leadership at the team and business levels. Needed capabilities may be decided and developed centrally by a business, or within teams and by individuals, or through some combination of the two. Determination of team SE capability is discussed in the article Team Capability, and individual SE competencies are discussed in the article Roles and Competencies.

Relationship of this Topic to Enterprise Systems Engineering

Enterprise Systems Engineering and Capability Engineering techniques can be used to establish needed SE capabilities. At a high level of abstraction, the following are basic steps that could be used to decide the desired SE capabilities within the business:

1. understand the context;
2. determine the required SE roles;
3. determine the competencies and capabilities needed for each of the SE roles;
4. assess the ability and availability of the needed SE organizations, teams, and individuals;
5. adjust the required SE roles based on the actual ability and availability; and
6. organize the SE function to facilitate communication, coordination, and performance.

See the article Organizing Business and Enterprises to Perform Systems Engineering for additional information. More information on context and required SE roles is provided below.

Contextual Drivers

The following discussion illustrates some of the contextual factors that influence the definition of the SE capability needed by a business.

Where the SE Activities are Performed in the Value Chain

The SE approach adopted by the business should depend on what role the organization plays. Ring (2002) defines a value cycle, and where the business sits in that cycle is a key influence of SE capability need.

- **Problem owner:** focus on identifying and scoping the system problem (defining system-of-interest (SoI)) and understanding the nature of the appropriate respondent system using Enterprise Systems Engineering and Capability Engineering approaches.
-

- **System operator:** focus on establishing all the necessary components of capability to deliver the required services, as well as on integrating new system assets into the system operation as they become available (see Service Systems Engineering). The definition of the components of capability varies by organization - e.g.,
 - The US Department of Defense defines the components of capability as DOTMLPF: doctrine, organization, training, materiel, logistics, people, and facilities.
 - The UK Ministry of Defense defines the components of capability as TEPIDOIL; i.e., training, equipment, people, information, doctrine, organization, infrastructure, and logistics.
 - Other domains and organizations define the components of capability with similar, equivalent breakdowns which are either explicit or implicit.
- **Prime contractor or primary commercial developer:** focus on understanding customer needs and trading alternative solution approaches, then establishing a system team and supply chain to develop, deliver, support, and in some cases, operate the system solution. This may require enterprise SE (see Enterprise Systems Engineering) as well as "traditional" product SE (see Product Systems Engineering).
- **Subsystem/component developer:** focus on understanding the critical customer and system integrator issues for the subsystem or component of interest, defining the component or subsystem boundary, and integrating critical technologies. This may exploit re-usable elements and can be sold in identical or modified forms to several customers. (In Part 4 of the SEBoK, see Systems of Systems, Enterprise Systems Engineering, and Product Systems Engineering for more information and references to the literature.)
- **Specialist service provider:** focus on specific process capabilities and competences which are typically sold on a time and materials or work package basis to other businesses.

Where the Enterprise Operates in the Lifecycle

The SE capabilities required by the business will depend on the system life cycle phase(s) in which it operates (see Life Cycle Models in Part 3).

- **Concept definition phase:** requires the SE capability to identify a "problem situation," define the context and potential concept of operations for a solution system, assess the feasibility of a range of possible solutions in broad terms, and refine the definition to allow the development of system requirements for the solution (see System Concept Definition in Part 3).
- **System Definition phase:** requires the SE capability to influence concept studies (ensure feasible and understood by the development team), establish the trade space that remains at the end of the concept study, perform the system definition activities, including architecture design, and create a detailed definition of the system elements.
- **System realization phase:** requires the SE capability to configure the manufacturing and logistics systems for the system assets, and manufacture system assets (see System Realization in Part 3).
- **System deployment and use:** requires the SE capability to maintain business continuity during the transition to operation, bring the system into service, support system, monitor system performance, and respond to emerging needs (see System Deployment and Use). Elliott et al. (2008) describe the different emphases that should be placed in SE during the "in-service" phase. This phase particularly requires the business to be able to perform SE at an appropriate operational tempo.
- **Retirement phase:** requires the SE capability for ensuring the safe retirement of systems and keeping them in a state ready for re-activation ("mothballed"), safe disposal of the system assets.

Nature of Responsibility to End Users and Society

Depending on the business model and the contracting environment, the business may find that its responsibility to end users is:

- **explicit**, or spelled out by clear requirements and prescriptive legislation; or
- **implicit**; i.e., a legal or ethical obligation to ensure “fitness for purpose” which may be enforced by commercial frameworks, national or international standards, and specific product liability legislation.

Typically, businesses whose business model is contract driven focus on satisfying explicit requirements, whereas market-driven businesses must be more aware of implicit responsibilities.

Nature of Responsibility to Customers

The business may contract with its customers to deliver any of the following:

- **an outcome**: The intended benefits the system is expected to provide, requires enterprise systems engineering;
- **an output**: Deliver or operate the system or part of it against agreed acceptance criteria; requires product systems engineering;
- **an activity**: Perform a specified set of tasks, requires service systems engineering; and
- **a resource**: Provide a specified resource; requires focus on individual competencies - see Enabling Individuals.

Scale of Systems

The business or enterprise may need very different SE approaches depending on the scale of the system at which the business operates. The following categories are based on Hitchins’ five layered system model (Hitchins 2005):

- **Level 1: Subsystem and technical artifacts** – focus on product systems engineering and on technology integration.
- **Level 2: Project systems** – focus on product systems engineering with cross-discipline and human integration.
- **Level 3: Business systems** – focus on enterprise systems engineering, service systems engineering to implement them, and on service management (Chang 2010) and continuous improvement (SEI 2010b); see also Quality Management) for the day to day running of the business.
- **Level 4: Industry systems** – If there is a conscious effort to treat an entire industry as a system, the focus will be on Enterprise Systems Engineering, and on the long-term economic and environmental sustainability of the overall industry.
- **Level 5: Societal systems** – Enterprise systems engineering is used to analyze and attempt to optimize societal systems (see Singapore Water Management in Part 7).

Sillitto (2011) has proposed extending this model to cover sustainability issues by adding two additional layers, the “ecosystem” and the “geosystem”.

Complexity of Systems Integration Tasks and Stupples’ levels

Creating Systems That Work – Principles of Engineering Systems for The 21st century identifies three “kinds” of SE, originally proposed by Stupples (2006), that have to do with the level of cross-disciplinary integration involved (Elliot et al. 2007).

1. Within a discipline (e.g., software, hardware, optics, *or* mechanics), the SE focus is on taking a systems view of the architecture and implementation to manage complexity and scale within a single engineering discipline.
2. In multiple disciplines (e.g., software, hardware, optics, *and* mechanics), the SE focus is on holistic integration of multiple technologies and skills to achieve a balanced system solution.
3. In socio-technical systems integration, the SE focus is on getting people and the non-human parts of the system working synergistically.

Sillitto (2011) proposed extending this model properly to cover sustainability issues by adding one additional level, “Environmental Integration”. He describes this level and show how the Stupples’ levels relate to other dimensions used to categorize systems and professional engineering skills.

Criticality of System and Certification Requirements

The level of rigor in the SE approach adopted by the business will depend on the criticality of various classes of requirement. (See Systems Engineering and Quality Attributes.)

- Safety and security requirements often demand specific auditable processes and proof of staff competence.
- Ethical and environmental requirements may require an audit of the whole supply and value chain.
- Extremely demanding combinations of performance requirements will require more design iteration and more critical control of component characteristics; e.g., see Quality Management and *Management for Quality in High-Technology Enterprises* (Fasser and Brettner 2010).

The Nature of a Contract or Agreement

The nature of the contractual relationship between a business and its customers and end users will influence the style of SE.

- Fixed price, cost plus, or other contracting models influence the mix of focus on performance and cost control and how the business is incentivized to handle risk and opportunity.
- In mandated work share arrangements, the architecture of the product system may be compromised or constrained by the architecture of a viable business system; this is often the case in multi-national projects and high-profile government procurements (Maier and Rechtin 2009, 361-373).
- In self-funded approaches, the priorities will be requirements elicitation approaches designed to discover the latent needs of consumers and business customers, as well as development approaches designed to achieve rapid time to market with a competitive offering, or to have a competitive offering of sufficient maturity available at the most critical time during a customer’s selection process.
- In single phase or whole-life approaches, the business may be able to optimize trade-offs across the development, implementation, and in-service budgets, and between the different components of capability.

The Nature and Predictability of Problem Domain(s)

Well-defined and slowly changing technologies, products, and services permit the use of traditional SE life cycle models based on the waterfall model because the requirements risk and change is expected to be low (see Life Cycle Models).

Poorly defined and rapidly changing problem domains, with operators subject to unpredictable and evolving threats, demand more flexible solutions and agile processes. SE should focus on modular architectures that allow rapid reconfiguration of systems and systems-of-systems, as well as rapid deployment of new technologies at a subsystem level to meet new demands and threats.

Fundamental Risks and Design Drivers in the Solution Domain

When the solution domain is stable, with a low rate of technology evolution, and systems use mature technology, the focus is on optimum packaging and configuration of known and usually well-proven building blocks within known reference architectures, and on low-risk incremental improvement over time.

When there is rapid technology evolution, with pressure to bring new technologies rapidly to market and/or into operational use, the SE approach has to focus on technology maturation, proof of technology and integration readiness, and handling the technology risk in the transition from the lab to the proof of concept to the operational system.

There is usually a trade-off between lead time expectations and the level of integrity/certification. In the development of new systems, short lead times are seldom compatible with high levels of system integrity and rigorous certification.

Competitive Situation and Business Goals

The business drivers for SE deployment may be one or more of the following:

- To perform existing business better;
- To recover from a competitive shock or a shift in clients' expectations;
- To develop a new generation product or service;
- To enter a new market; and/or
- To reposition the business or enterprise in the value chain.

In the first case, SE can be deployed incrementally in parts of the business process where early tangible benefits can be realized. This could be the early steps of a business-wide strategic plan for SE. (See Systems Engineering Organizational Strategy for more on setting SE strategy and Developing Systems Engineering Capabilities within Businesses and Enterprises for improving SE capabilities.)

In the other cases, the business is going through disruptive change and the early priority may be to use systems thinking (see Systems Thinking) and enterprise SE approaches to scope the transformation in the context of a major change initiative.

Type of System or Service

There are three distinct flavors of products or service types (see Systems Engineering Organizational Strategy):

1. In a product or productized service, the focus will be on predicting how the market might change during the development period, eliciting, anticipating, and balancing requirements from a variety of potential customers, and optimizing features and product attractiveness against cost and reliability.
2. In a custom solution (product or service) the focus will be on feasible and low-risk (usually) approaches to meet the stated requirement within budget, using system elements and technologies that are known or expected to be available within the desired development timescale.
3. Tailored solutions based on standard product and/or service elements require a much more sophisticated SE process that is able to use a “product line approach” to blend standard modules with planned adaptation to meet clients' specific needs more quickly and cheaply than would be possible with a single contract solution. The business needs to manage the life cycle and configuration of the standard modules separately from, but coherently with, the life cycle and configuration of each tailored solution.

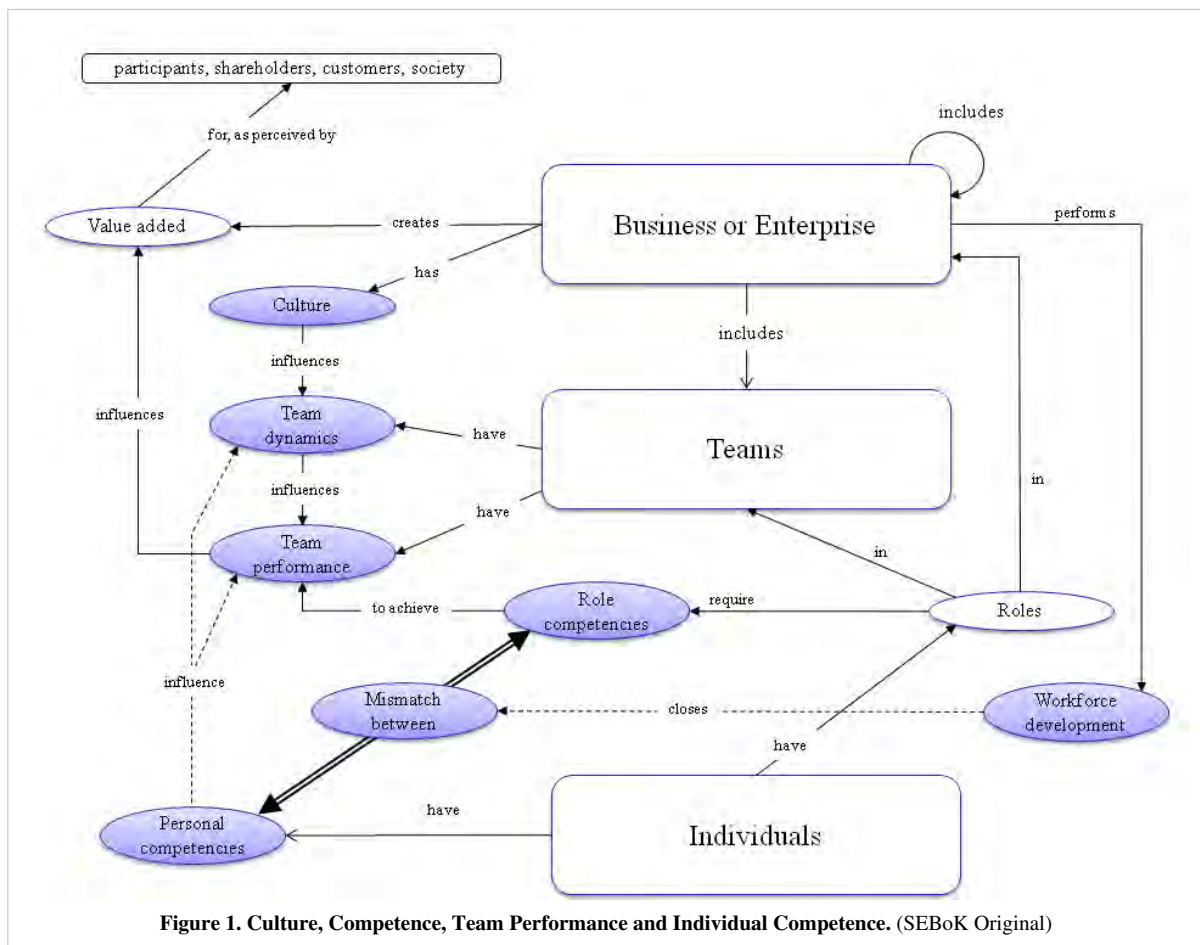
Needed Systems Engineering Roles

After understanding the context for the business, the next step is to determine the SE capabilities required in the role in the business. The SEI Capability Maturity Models for acquisition, development, and services (SEI 2007; SEI 2010a; SEI 2010b) provide a framework for selecting SE capabilities relevant to different types of business. Existing SE competency models can be used to assist in determining the needed capabilities. An example is the INCOSE SE Competencies Framework (INCOSE 2010). (See Roles and Competencies for more information on competency models.)

The spread of SE focus can be a wide spectrum, from SE being focused in a specialist, interface or glue role (Sheard 1996), to the idea that “SE is good engineering with special areas of emphasis... including interfaces between disciplines” (Blanchard and Fabrycky 2005) and so it is shared by all. In any organization where activities and skills are shared, there is always a danger of silos or duplication.

As part of the role definition, the business must define where an individual doing SE fits into career progression (what roles before SE, what after?). Developing Individuals describes how individuals improve SE; the organization must define the means by which that development can be enacted. Businesses need to customize from a range of development strategies; see, for example, Davidz and Martin (2011).

As shown in Figure 1 below, management action on workforce development will be required if there are systemic mismatches between the competencies required to perform SE roles and the actual competencies of individuals. The organizational culture may have a positive or negative effect on team performance and the overall value added by the business (see Culture).



Required SE Processes and Methods

The decisions on how to implement SE capability must be embedded in the businesses processes and its availability methodologies and toolsets. Embedding SE principles, processes, and methods in the organization's quality management system means that senior management and the quality system will help embed SE in the organizational business process and make sure it is applied (INCOSE 2012; ISO/IEC 2008; see Quality Management).

When defining the processes and tools, a balance between the need for a systematic and standardized approach to SE processes, such as that seen in INCOSE (2012), with the flexibility inherent in systemic thinking is critical. Systems thinking helps the organization understand problem situations, remove organizational barriers, and make the most of the organization's technical capabilities (see Beasley (2011)).

Need for Clarity in the SE Approach and the Dangers of Implementing SE

Clarity on how the organization performs SE is important. Typically, implementing SE may be part of an organization's improvement, so Kotter's principles on creating a vision, communicating the vision, and empowering others to act on the vision are extremely relevant (Kotter 1995). The way an organization chooses to perform SE should be part of the vision of the organization and must be understood and accepted by all.

Many of the major obstacles in SE deployment are cultural (see Culture).

One of the lean enablers for SE is to "pursue perfection" (Oppenheim et al. 2010). The means of improvement at a business or enterprise level are discussed in detail elsewhere, but the starting point must be deciding what SE capabilities the organization wants. It needs to be recognized that the needed capabilities change over time (learning, improving, or losing capability). Thus, balancing SE with everything else that it involves is an ever-changing process.

References

Works Cited

- Beasley, R. 2011. "The Three T's of Systems Engineering." Paper presented at the 21st Annual International Council on Systems Engineering (INCOSE) International Symposium. June 2011. Denver, CO, USA.
- Blanchard, B. and W. Fabrycky. 2005. *Systems Engineering and Analysis*, 4th edition. Upper Saddle River, NJ, USA: Prentice Hall.
- Chang, C.M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. Hoboken, NJ, USA: John Wiley and Sons.
- Davidz, H. L. and J. Martin. 2011. "Defining a Strategy for Development of Systems Capability in the Workforce." *Systems Engineering*. 14(2) (Summer, 2011): 141-143
- Elliott, B. et al. 2008. *INCOSE UK Chapter Working Group on Applying Systems Engineering to In-Service Systems*, Final Report. Somerset, UK: INCOSE UK Chapter Working Group. Accessed September 6, 2011. Available at https://www.incoseonline.org.uk/Documents/Groups/InServiceSystems/is_tr_001_final_report_final_1_0.pdf.
- Fasser, Y. and D. Brettner. 2001. *Management for Quality in High-Technology Enterprises*. New York, NY, USA: Wiley.
- Hitchins, D. 2005. *Systems Engineering 5 Layer Model*. Accessed on April 24, 2013. Available at <https://www.hitchins.net/systems/world-class-systems-engineer.html>.
- INCOSE. 2010. *SE Competencies Framework*, Issue 3. Somerset, UK: International Council on Systems Engineering (INCOSE), INCOSE Technical Product 2010-0205.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE),
-

INCOSE-TP-2003-002-03.2.2.

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2008.

Kotter, J. 1995. *Leading Change: Why Transformation Efforts Fail*. Boston, MA, USA: Harvard Business Review (March–April 1995).

Maier, M. and E. Rechtin. 2009. *The Art of System Architecting, Third Edition*. Boca Raton, FL, USA: CRC Press.

Oppenheim et al. 2010. *Lean Enablers for Systems Engineering*. New York, NY, USA: Wiley and Sons, Inc.

Ring J. 2002. *Toward an Ontology of Systems Engineering*. INSIGHT, 5(1): 19-22.

SEI. 2007. *CMMI for Acquisition*. Version 1.2. Technical Report CMU/SEI-2007-TR-017. Pittsburgh, PA, USA: Software Engineering Institute, Carnegie Mellon University.

SEI. 2010a. *Capability Maturity Model Integrated (CMMI) for Development*. Version 1.3. Pittsburgh, PA, USA: Software Engineering Institute, Carnegie Mellon University.

SEI. 2010b. *CMMI for Services*. Version 1.3. Technical Report CMU/SEI-2010-TR-034. Pittsburgh, PA, USA: Software Engineering Institute, Carnegie Mellon University.

Sheard, S. 1996. "12 Systems Engineering Roles." Paper presented at the 6th Annual International Council on Systems Engineering (INCOSE) International Symposium. Boston, MA, USA. Accessed September 14, 2011.

Sillitto, H. 2011. "Unravelling Systems Engineers from Systems Engineering - Frameworks for Understanding the Extent, Variety and Ambiguity of Systems Engineering and Systems Engineers." Paper presented at the 21st Annual International Council on Systems Engineering (INCOSE) International Symposium. 20-23 June 2011. Denver, CO, USA.

Stupples, D. 2006. "Systems Engineering – a road from perdition." Published on Royal Academy of Engineering website. Available at https://www.raeng.org.uk/education/vps/systemdesign/pdf/David_Stupples.pdf

Primary References

Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Chichester, UK: Wiley and Sons, Inc.

Oppenheim, B. 2011. *Lean for Systems Engineering - with Lean Enablers for Systems Engineering*. Hoboken, NJ, USA: Wiley and Sons, Inc.

Sheard, S. 1996. *Twelve Systems Engineering Roles*. Paper presented at the 6th Annual International Council on Systems Engineering (INCOSE) International Symposium. Boston, MA, USA. Accessed September 14, 2011.

Additional References

Rhodes, D., and G. Roedler (eds.). 2007. *Systems Engineering Leading Indicators Guide*, version 1.0. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2005-001-02.

Organizing Business and Enterprises to Perform Systems Engineering

- Lead Authors:
- Richard Beasley, Art Pyster, and Hillary Sillitto
- Contributing Authors:
- Alice Squires, Heidi Davidz, Scott Jackson, and Quong Wang

In order for a business or enterprise to perform systems engineering (SE) well, the team must decide which specific SE capabilities the business or enterprise needs in order to be successful and then organizing to deliver those capabilities. (In the rest of this article, business or enterprise is usually abbreviated to just "business", because a business is a specific type of enterprise that has sufficiently strong central authority and motivation to take steps to enable SE).

SE capabilities and organizational approach should support the Systems Engineering Organizational Strategy and reflect the nature of the business, its products and services, various stakeholders, business leadership focus, etc. This topic, which is part of Part 5, Enabling Businesses and Enterprises, summarizes the factors used to organize a business to perform SE.

Components of Business and Enterprise SE Capability

Organization Issues - Culture, Knowledge, Information, and Infrastructure

The way SE is managed is described in Systems Engineering Organizational Strategy, which both impacts and responds to the SE culture and approach.

Knowledge and Information

Knowledge and Information are key assets in a business, and their management is critical. Fasser and Brettner (2002) discuss knowledge management extensively. They assert that *"We may think that knowledge transfer is just an information technology issue, but in actuality, it is also a psychological, cultural, and managerial issue – in short a human issue"* and *"Only information in action can create knowledge"*.

Organizations need to manage SE know-how, integration of SE with other organizational processes and activities, and knowledge of their business domain. The INCOSE Intelligent Enterprise Working Group's work on knowledge management in an SE context led to the publication of a *"Concept of Operations for a Systems Engineering Educational Community"* (Ring et al. 2004).

Information has to be both shared and protected in complex organizations. Sharing is key to effective collaboration and is constrained by the need to protect intellectual property, as well as commercially and nationally sensitive material. Different cultures and personal styles use information in different ways and in different orders. (Levels of abstraction, big picture first or detail, principles first or practical examples, etc.) Sillitto (2011b) describes the knowledge management challenges for large, multi-national organizations.

Projects need to manage project information and establish configuration control over formal contractual information, as well as the information that defines the product/service being developed, supplied, or operated. A key role of systems engineers is to "language the project" (Ring et al. 2004). Good data management and tool support will allow people to document once, use many times, and ensures consistency of information over time and between different teams.

System information needs to be maintained throughout the life of the system and made available to relevant stakeholders – including those designing new systems that must interface to the system-of-interest - to allow system

management, maintenance, reconfiguration, upgrade and disposal, and forensics after accidents and near-misses. Elliott et al. (2008) suggest that information management is the dominant problem in SE in service systems, and that the cost and difficulty of establishing current state and legacy constraints before starting to implement a change is often underestimated.

"Infostructure" (information infrastructure) to support the system lifecycle will include the following:

- Information assets such as process libraries, document templates, preferred parts lists, component re-use libraries, as-specified and as-tested information about legacy systems, capitalized metrics for organizational performance on previous similar projects, all with appropriate configuration control
- Modeling and simulation tools, data sets and run-time environments
- Shared working environments – workspaces for co-located teams, areas for people to interact with each other to develop ideas and explore concepts, work areas suitable for analysis tasks, meeting rooms, access control provision, etc.
- IT facilities - computer file structures, software licenses, IT equipment, computer and wall displays to support collaborative working, printers, all with appropriate security provision and back-up facilities, procedures for efficient use, and acceptable performance and usability
- Security provisions to protect own, customer, supplier and third party IPR and enforce necessary protective working practices while allowing efficient access to information for those with a need to know

SE is a knowledge activity. Systems engineers need appropriate facilities for accessing, sharing and capturing knowledge, as well as for interacting effectively with the whole set of stakeholders. Warfield (2006) describes collaborative workspaces, environments and processes for developing a shared understanding of a problem situation.

Enabling Infrastructure

The ISO/IEC 15288 (ISO 2008) Infrastructure Management Process provides the enabling infrastructure and services to support organization and project objectives throughout the life cycle. Infrastructure to support the system life cycle will often include the following:

- Integration and test environment – bench and lab facilities, facilities for development testing as well as acceptance testing at various levels of integration, calibration and configuration management of test environments
- Trials and validation environment – access to test ranges, test tracks, calibrated targets, support and storage for trials – equipment, harbor, airfield and road facilities, safe storage for fuel, ordinance, etc.
- Training and support infrastructure – training simulators, embedded training, tools and test equipment for operational support and maintenance, etc.

People

The roles people fill are typically defined by the business/enterprise (see Determining Needed Systems Engineering Capabilities in Businesses and Enterprises), although those decisions may be pushed down to teams. Enabling Teams explains how people are used in teams; Enabling Individuals describes the development of an individual's SE competence.

The implementation of these roles needs further consideration. Sheard (1996) lists twelve system engineering roles. Sheard (2000) draws an important distinction between roles involved in the discovery phase, characterized by a high level of uncertainty, the program phase, which is more deterministic and defined, and the overall systems engineering approach. Kasser et al. (2009) identify five types of systems engineer distinguished by the need to work at increasing levels of abstraction, ambiguity, scope and innovation. Sillitto (2011a) discusses a number of SE roles and the characteristics required of them, in the context of the wider engineering and business professional landscape.

Systems engineering exists within an enterprise "ecosystem." Two key aspects to consider:

- How much should the business/enterprise nurture and value the systems engineer?

- How much should the business/enterprise pull value from systems engineers, rather than wait for systems engineers to "push" value on the business/enterprise?

Process

Many SE organizations maintain a set of organizational standard processes which are integrated in their quality and business management system, adapted to their business, and with tailoring guidelines used to help projects apply the standard processes to their unique circumstances. Guidance on organizational process management is provided by such frameworks as the Capability Maturity Model Integration (CMMI) (SEI 2010), which has two process areas on organizational process: Organizational Process Development (OPD) is concerned with organizational definition and tailoring of the SE lifecycle processes (discussed in detail elsewhere in this document) and Organizational Process Focus (OPF), which is concerned with establishing a process culture in an organization.

To document, assess, and improve SE processes, businesses often establish a systems engineering process group. Members of such groups often create standard process assets and may mentor teams and business units on how to adopt those standard processes and assess how effective those processes are working. There is a large body of literature on SE process improvement based on various process improvement models. Two of the most popular are ISO/IEC 9000 (2000) and CMMI (SEI 2010). The Software Engineering Institute, which created the CMMI, offers many free technical reports and other documents on CMMI at <https://www.sei.cmu.edu/cmmi>.

Assessment and measuring process performance is covered in Assessing Systems Engineering Performance of Business and Enterprises.

Tools and Methods

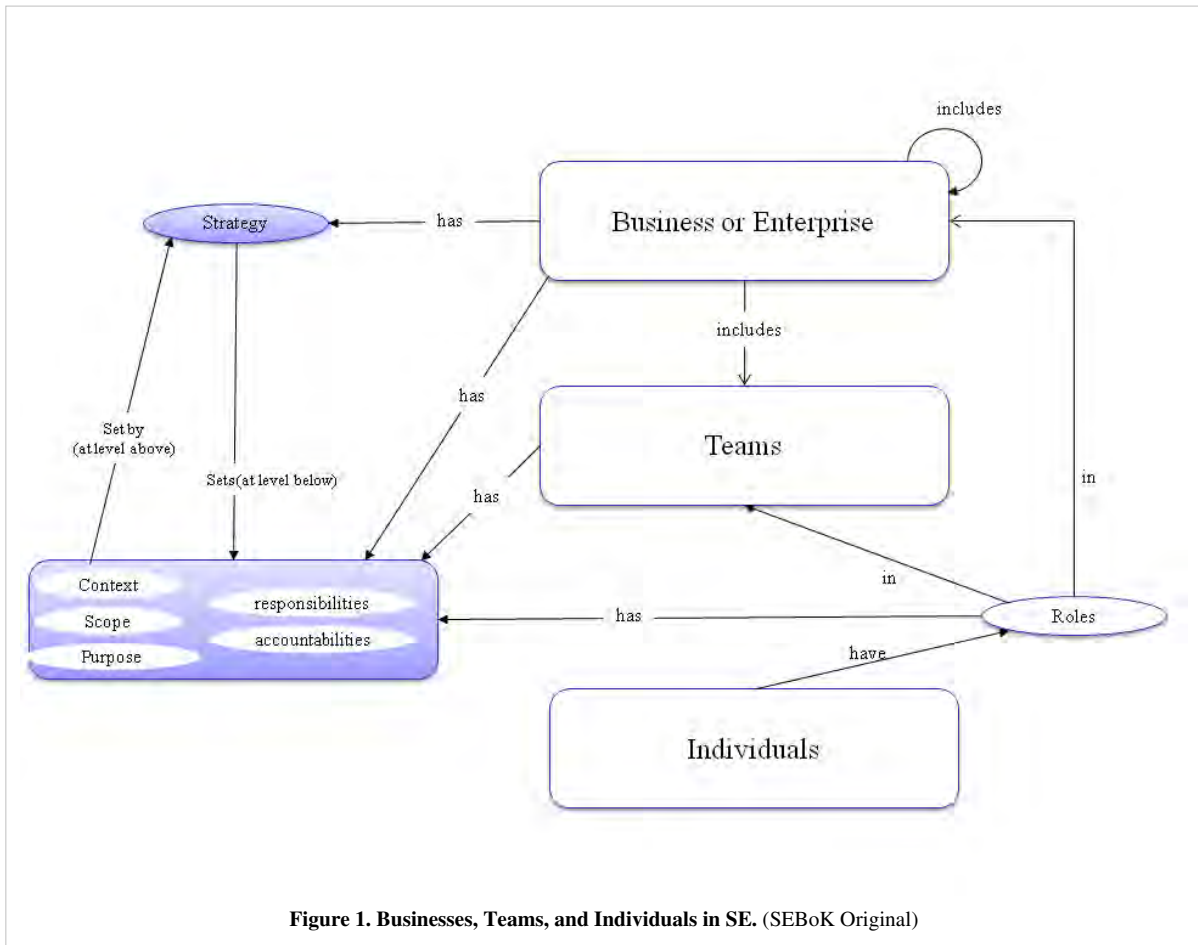
SE organizations often invest in SE tools and models, develop their own, and/or integrate off-the-shelf tools into their particular business/enterprise processes. Tools require great attention to culture and training; to developing a consistent "style" of use so that people can understand each others' work; and proper configuration and management of the information so that people are working on common and correct information.

It is important that methods are used as well as tools, particularly to support Systems Thinking.

It is common practice in large SE organizations to have a tool support infrastructure which ensures that tools support the organizational standard processes and are fully integrated with training, and that projects and teams can use the tools to do their job and are not distracted by tool management issues that are more efficiently handled centrally. Smaller SE organizations often operate more informally.

Fitting It All Together

The concept map in Figure 1 below shows the relationships between the various aspects of organization, resource, responsibility, and governance.



Enterprise Structures and Their Effects on SE

Enterprises manage SE resources in many different ways. A key driver is the extent to which they seek to optimize use of resources (people, knowledge, and assets) across teams and across the enterprise as a whole. Five common ways of organizing resources to support multiple projects are: project; matrix; functional; integrated; and product centered (CM Guide 2009, Handy 1985, PMI 2013, section 2.1.3). A large enterprise would likely apply some combination of these five ways across its constituent sub-enterprises and teams. Browning (2009) offers a way to optimize project organizational structure. Eisner (2008) offers a good overview of different organizational models.

Project Organization

A project organization is one extreme in which projects are responsible for hiring, training, and terminating staff, as well as managing all assets required for delivery. In this model, systems engineers on a project report to the project manager and resources are optimized for the delivery of the project. This model has the advantage of strongly aligning the authority and responsibility of the project with the project manager. However, it operates at the expense of sub-optimizing how the staff is deployed across the larger enterprise, how technology choices are made across projects, etc. *Systems Engineering Fundamentals* (DAU 2001) offers a DoD view of good practice project organizations.

Functional Organization

A functional organization demonstrates the opposite extreme. In a functional organization, projects delegate almost all their work to functional groups, such as the software group, the radar group or the communications group. This is appropriate when the functional skill is fast-evolving and dependent on complex infrastructure. This method is often used for manufacturing, test engineering, software development, financial, purchasing, commercial, and legal functions.

Matrix Organization

A matrix organization is used to give systems engineers a “home” between project assignments. Typically, a SE functional lead is responsible for career development of the systems engineers in the organization, a factor that influences the diversity and length of individual project assignments.

Integrated Organization

In an integrated organization, people do assigned jobs without specific functional allegiance. Those that perform SE tasks are primarily identified as another type of engineer, such as a civil or electrical engineer. They know systems engineering and use it in their daily activities as required.

Product Centered Organization

In accordance with the heuristic that “the product and the process must match” (Rechtin 1991, 132), a common method for creating an organizational structure is to make it match the system breakdown structure (SBS). According to Browning (2009), at each element of the SBS there is an assigned integrated product team (IPT). Each IPT consists of members of the technical disciplines needed to design the product system. The purpose of the IPT is to assure that the interactions among all the technical disciplines are accounted for in the design and that undesirable interactions are avoided.

Interface to Other Organizations

Outside official engineering and SE organizations within an enterprise, there are other organizations whose charter is not technical. Nevertheless, these organizations have an important SE role.

- **Customer Interface Organizations:** These are organizations with titles such as Marketing and Customer Engineering. They have the most direct interface with current or potential clientele. Their role is to determine customer needs and communicate these needs to the SE organization for conversion to product requirements and other system requirements. Kossiakoff and Sweet (2003, 173) discuss the importance of understanding customer needs.
 - **Contracts Organizations:** These organizations interface with both customer and supplier organizations. Their role is to develop clearly stated contracts for the developer or the supplier. These contracts convey tasks and responsibilities for all SE roles of all parties. Technical specifications are attached to the contracts. Responsibilities for verification and validation are specified.
 - **Supplier Management Organizations:** These organizations are responsible for selecting and managing suppliers and assuring that both contractual and technical products are in place. These organizations balance cost and risk to assure that supplier products are delivered, verified, and validated for quality product. Blanchard and Fabrycky (2005, 696-698) discuss the importance of supplier selection and agreement.
-

References

Works Cited

- Blanchard, B. and W. Fabrycky. 2005. *Systems Engineering and Analysis*, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall.
- Browning, T.R. 2009. "Using the Design Structure Matrix to Design Program Organizations." In A.P. Sage and W.B. Rouse (eds.), *Handbook of Systems Engineering and Management*, 2nd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Construction Management (CM) Guide. 2009. *Project Organization Types*. Accessed on September 14, 2011. Available at <https://cmguide.org/archives/319>.
- DAU. 2001. *Systems Engineering Fundamentals*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU), U.S. Department of Defense (DoD). Accessed on September 14, 2011. Available at <https://www.dau.mil/pubscats/PubsCats/SEFGuide%2001-01.pdf>.
- Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Elliott et al. 2008. *INCOSE UK Chapter Working Group on Applying Systems Engineering to In-Service Systems*. Final Report. Somerset, UK: INCOSE UK Chapter Working Group. Accessed September 6, 2011. Available at https://www.incoseonline.org.uk/Documents/Groups/InServiceSystems/is_tr_001_final_report_final_1_0.pdf.
- Fasser, Y. and D. Brettner. 2002. *Management for Quality in High-Technology Enterprises*. Hoboken, NJ, USA: John Wiley & Sons.
- ISO/IEC. 2000. *International standards for quality management*. Genève, Switzerland: International Organization for Standardization. ISO 9000:2000.
- ISO/IEC. 2008. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2008.
- Kasser, J., D. Hitchins, and T. Huynh. 2009. "Re-engineering Systems Engineering." Proceedings of the 3rd Annual Asia-Pacific Conference on Systems Engineering (APCOSE). 20-23 July 2009. Singapore.
- Kossiakoff, A., and W.N. Sweet. 2003. *Systems Engineering: Principles and Practice*. Edited by A. Sage, Wiley Series in Systems Engineering and Management. Hoboken, NJ: John Wiley & Sons.
- PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- Rechtin, E. 1991. *Systems Architecting: Creating and Building Complex Systems*. Englewood Cliffs, NJ, USA: CRC Press.
- Ring, J. and A.W. Wymore (eds.). 2004. *Concept of Operations (conops) of A Systems Engineering Education Community (SEEC)*. Seattle, WA, USA: INCOSE Education Measurement Working Group (EMWG), INCOSE-TP-2003-015-01.
- SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- Sheard, S. 1996. "12 Systems Engineering Roles." Paper presented at the Sixth Annual International Council on Systems Engineering (INCOSE) International Symposium. 7-11 July 1996. Boston, MA, USA.
- Sheard, S. 2000. "The 12 Systems Engineering Roles Revisited." Paper presented at the INCOSE Mid-Atlantic Regional Conference. April 2000. Reston, VA, USA. p 5.2-1 - 5.2-9.
- Sillitto, H. 2011a. "Unravelling Systems Engineers from Systems Engineering - Frameworks for Understanding the Extent, Variety and Ambiguity of Systems Engineering and Systems Engineers." Paper presented at the 21st Annual International Council on Systems Engineering (INCOSE) International Symposium. 20-23 June 2011. Denver, CO,

USA.

Sillitto, H. 2011b. *Sharing Systems Engineering Knowledge through INCOSE: INCOSE as An Ultra-Large-Scale System?* *INCOSE Insight*. 14(1) (April): 20.

Warfield, J. 2006. *An Introduction to Systems Science*. Washington, DC, USA: The National Academies Press, World Scientific.

Primary References

Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. Hoboken, NJ, USA: John Wiley and Sons.

Kotter, J. 1995. "Leading Change: Why Transformation Efforts Fail." *Harvard Business Review*. 73(2): 59–67.

Sheard, S. 2000. "Systems Engineering Roles Revisited." Paper presented at the INCOSE Mid-Atlantic Regional Conference. April 5-8 2000. Reston, VA, USA. p 5.2-1 - 5.2-9.

Additional References

Blanchard, B. and W. Fabrycky. 2005. *Systems Engineering and Analysis*, 4th edition. Upper Saddle River, NJ, USA: Prentice Hall.

Construction Management (CM) Guide. 2009. *Project Organization Types*. Accessed on September 6, 2011. Available at <https://cmguide.org/archives/319>.

Defense Acquisition University (DAU). 2001. *Systems Engineering Fundamentals*. Fort Belvoir, VA, USA: Defense Acquisition University Press. Accessed on September 6, 2011. Available at <https://www.dau.mil/pubscats/PubsCats/SEFGuide%2001-01.pdf>.

Handy, C.B. 1985. *Understanding Organizations*. London, UK: Penguin Business.

Assessing Systems Engineering Performance of Business and Enterprises

- Lead Authors:
- Hillary Sillitto, Alice Squires, and Heidi Davidz
- Contributing Authors:
- Art Pyster and Richard Beasley

At the project level, systems engineering (SE) measurement focuses on indicators of project and system success that are relevant to the project and its stakeholders. At the enterprise level there are additional concerns. SE governance should ensure that the performance of systems engineering within the enterprise adds value to the organization, is aligned to the organization's purpose, and implements the relevant parts of the organization's strategy.

For enterprises that are traditional businesses this is easier, because such organizations typically have more control levers than more loosely structured enterprises. The governance levers that can be used to improve performance include people (selection, training, culture, incentives), process, tools and infrastructure, and organization; therefore, the assessment of systems engineering performance in an enterprise should cover these dimensions.

Being able to aggregate high quality data about the performance of teams with respect to SE activities is certainly of benefit when trying to guide team activities. Having access to comparable data, however, is often difficult, especially in organizations that are relatively autonomous, use different technologies and tools, build products in different domains, have different types of customers, etc. Even if there is limited ability to reliably collect and aggregate data across teams, having a policy that consciously decides how the enterprise will address data collection and analysis is valuable.

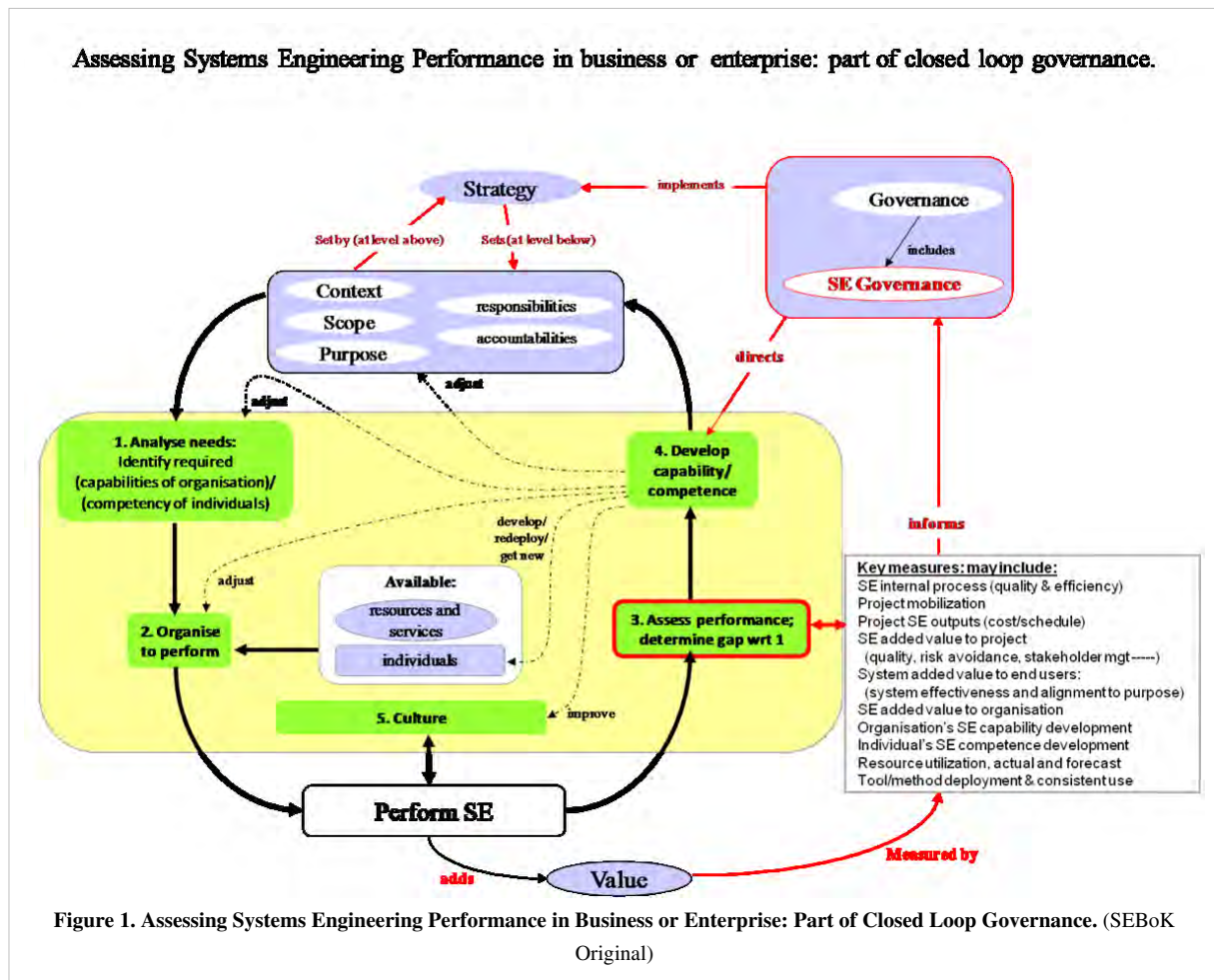
Performance Assessment Measures

Typical measures for assessing SE performance of an enterprise include the following:

- Effectiveness of SE process
- Ability to mobilize the right resources at the right time for a new project or new project phase
- Quality of SE process outputs
- Timeliness of SE process outputs
- SE added value to project
- System added value to end users
- SE added value to organization
- Organization's SE capability development
- Individuals' SE competence development
- Resource utilization, current and forecast
- Productivity of systems engineers
- Deployment and consistent usage of tools and methods

How Measures Fit in the Governance Process and Improvement Cycle

Since collecting data and analyzing it takes effort that is often significant, measurement is best done when its purpose is clear and is part of an overall strategy. The "goal, question, metric" paradigm (Basili 1992) should be applied, in which measurement data is collected to answer specific questions, the answer to which helps achieve a goal, such as decreasing the cost of creating a system architecture or increasing the value of a system to a particular stakeholder. Figure 1 shows one way in which appropriate measures inform enterprise level governance and drive an improvement cycle such as the Six Sigma DMAIC (Define, Measure, Analyze, Improve, Control) model.



Discussion of Performance Assessment Measures

Assessing SE Internal Process (Quality and Efficiency)

A process is a "set of interrelated or interacting activities which transforms inputs into outputs." The SEI CMMI Capability Maturity Model (SEI 2010) provides a structured way for businesses and enterprises to assess their SE processes. In the CMMI, a process area is a cluster of related practices in an area that, when implemented collectively, satisfies a set of goals considered important for making improvement in that area. There are CMMI models for acquisition, for development, and for services (SEI 2010, 11). CMMI defines how to assess individual process areas against Capability Levels on a scale from 0 to 3, and overall organizational maturity on a scale from 1 to 5.

Assessing Ability to Mobilize for a New Project or New Project Phase

Successful and timely project initiation and execution depends on having the right people available at the right time. If key resources are deployed elsewhere, they cannot be applied to new projects at the early stages when these resources make the most difference. Queuing theory shows that if a resource pool is running at or close to capacity, delays and queues are inevitable.

The ability to manage teams through their lifecycle is an organizational capability that has substantial leverage on project and organizational efficiency and effectiveness. This includes being able to

- mobilize teams rapidly;
- establish and tailor an appropriate set of processes, metrics and systems engineering plans;
- support them to maintain a high level of performance;
- capitalize acquired knowledge; and
- redeploy team members expeditiously as the team winds down.

Specialists and experts are used to a review process, critiquing solutions, creating novel solutions, and solving critical problems. Specialists and experts are usually a scarce resource. Few businesses have the luxury of having enough experts with all the necessary skills and behaviors on tap to allocate to all teams just when needed. If the skills are core to the business' competitive position or governance approach, then it makes sense to manage them through a governance process that ensures their skills are applied to greatest effect across the business.

Businesses typically find themselves balancing between having enough headroom to keep projects on schedule when things do not go as planned and utilizing resources efficiently.

Project SE Outputs (Cost, Schedule, Quality)

Many SE outputs in a project are produced early in the life cycle to enable downstream activities. Hidden defects in the early phase SE work products may not become fully apparent until the project hits problems in integration, verification and validation, or transition to operations. Intensive peer review and rigorous modeling are the normal ways of detecting and correcting defects in and lack of coherence between SE work products.

Leading indicators could be monitored at the organizational level to help direct support to projects or teams heading for trouble. For example, the INCOSE Leading Indicators report (Roedler et al. 2010) offers a set of indicators that is useful at the project level. Lean Sigma provides a tool for assessing benefit delivery throughout an enterprise value stream. Lean Enablers for Systems Engineering are now being developed (Oppenheim et al. 2010). An emerging good practice is to use lean value stream mapping to aid the optimization of project plans and process application.

In a mature organization, one good measure of SE quality is the number of defects that have to be corrected "out of phase"; i.e., at a later phase in the life cycle than the one in which the defect was introduced. This gives a good measure of process performance and the quality of SE outputs. Within a single project, the Work Product Approval, Review Action Closure, and Defect Error trends contain information that allows residual defect densities to be estimated (Roedler et al. 2010; Davies and Hunter 2001).

Because of the leverage of front-end SE on overall project performance, it is important to focus on quality and timeliness of SE deliverables (Woodcock 2009).

SE Added Value to Project

SE that is properly managed and performed should add value to the project in terms of quality, risk avoidance, improved coherence, better management of issues and dependencies, right-first-time integration and formal verification, stakeholder management, and effective scope management. Because quality and quantity of SE are not the only factors that influence these outcomes, and because the effect is a delayed one (good SE early in the project pays off in later phases) there has been a significant amount of research to establish evidence to underpin the asserted benefits of SE in projects.

A summary of the main results is provided in the Economic Value of Systems Engineering article.

System Added Value to End Users

System-added value to end users depends on system effectiveness and on alignment of the requirements and design to the end users' purpose and mission. System end users are often only involved indirectly in the procurement process.

Research on the value proposition of SE shows that good project outcomes do not necessarily correlate with good end user experience. Sometimes systems developers are discouraged from talking to end users because the acquirer is afraid of requirements creep. There is experience to the contrary – that end user involvement can result in more successful and simpler system solutions.

Two possible measures indicative of end user satisfaction are:

1. The use of user-validated mission scenarios (both nominal and "rainy day" situations) to validate requirements, drive trade-offs and organize testing and acceptance;
2. The use of technical performance measure (tpm) to track critical performance and non-functional system attributes directly relevant to operational utility. The INCOSE SE Leading Indicators Guide (Roedler et al. 2010, 10 and 68) defines "technical measurement trends" as *"Progress towards meeting the measure of effectiveness (moe) / measure of performance (mop) / Key Performance Parameters (KPPs) and technical performance measure (tpm)"*. A typical TPM progress plot is shown in Figure 2.

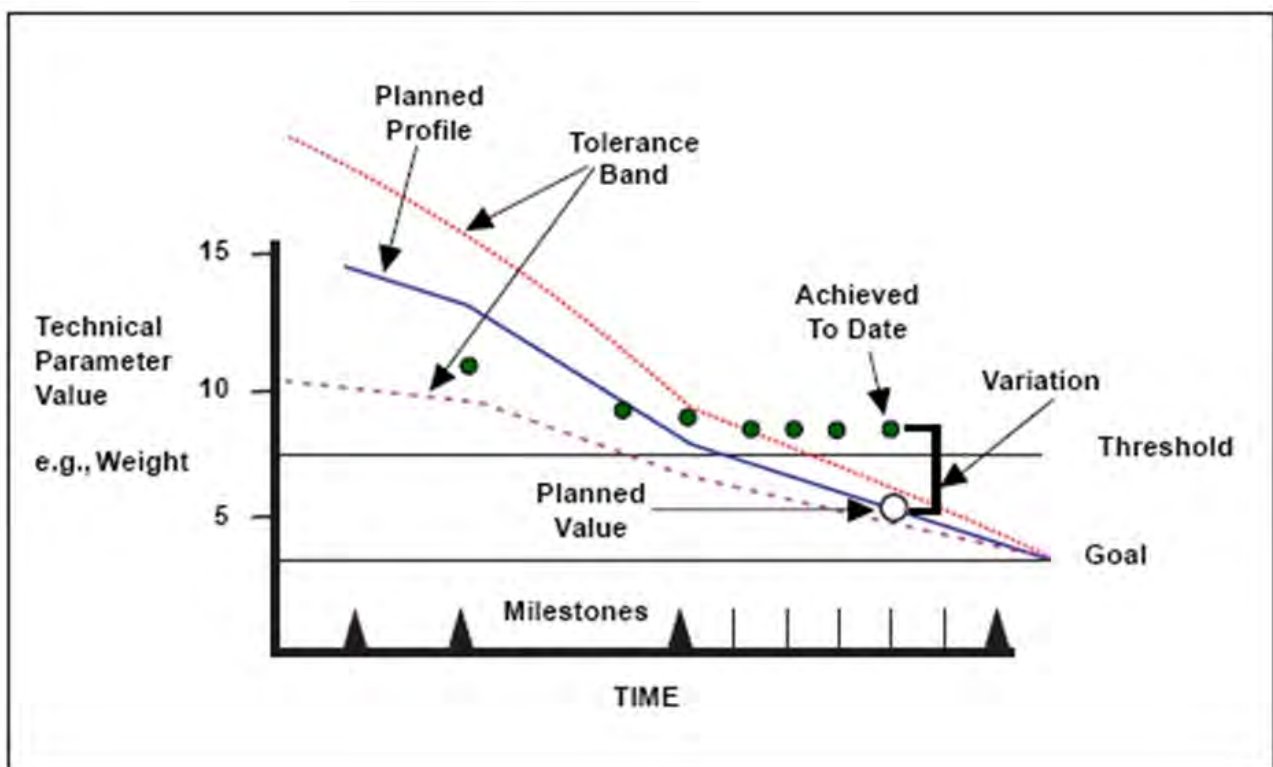


Figure 2. Technical Performance Measure (TPM) Tracking (Roedler et al. 2010). This material is reprinted with permission from the International Council on Systems Engineering (INCOSE). All other rights are reserved by the copyright owner.

SE Added Value to Organization

SE at the business/enterprise level aims to develop, deploy and enable effective SE to add value to the organization's business. The SE function in the business/enterprise should understand the part it has to play in the bigger picture and identify appropriate performance measures - derived from the business or enterprise goals, and coherent with those of other parts of the organization - so that it can optimize its contribution.

Organization's SE Capability Development

The CMMI (SEI 2010) provides a means of assessing the process capability and maturity of businesses and enterprises. The higher CMMI levels are concerned with systemic integration of capabilities across the business or enterprise.

CMMI measures one important dimension of capability development, but CMMI maturity level is not a direct measure of business effectiveness unless the SE measures are properly integrated with business performance measures. These may include bid success rate, market share, position in value chain, development cycle time and cost, level of innovation and re-use, and the effectiveness with which SE capabilities are applied to the specific problem and solution space of interest to the business.

Individuals' SE Competence Development

Assessment of Individuals' SE competence development is described in Assessing Individuals.

Resource Utilization, Current and Forecast

Roedler et al. (2010, 58) offer various metrics for staff ramp-up and use on a project. Across the business or enterprise, key indicators include the overall manpower trend across the projects, the stability of the forward load, levels of overtime, the resource headroom (if any), staff turnover, level of training, and the period of time for which key resources are committed.

Deployment and Consistent Usage of Tools and Methods

It is common practice to use a range of software tools in an effort to manage the complexity of system development and in-service management. These range from simple office suites to complex logical, virtual reality and physics-based modeling environments.

Deployment of SE tools requires careful consideration of purpose, business objectives, business effectiveness, training, aptitude, method, style, business effectiveness, infrastructure, support, integration of the tool with the existing or revised SE process, and approaches to ensure consistency, longevity and appropriate configuration management of information. Systems may be in service for upwards of 50 years, but storage media and file formats that are 10-15 years old are unreadable on most modern computers. It is desirable for many users to be able to work with a single common model; it can be that two engineers sitting next to each other using the same tool use sufficiently different modeling styles that they cannot work on or re-use each others' models.

License usage over time and across sites and projects is a key indicator of extent and efficiency of tool deployment. More difficult to assess is the consistency of usage. Roedler et al. (2010, 73) recommend metrics on "facilities and equipment availability".

Practical Considerations

Assessment of SE performance at the business/enterprise level is complex and needs to consider soft issues as well as hard issues. Stakeholder concerns and satisfaction criteria may not be obvious or explicit. Clear and explicit reciprocal expectations and alignment of purpose, values, goals and incentives help to achieve synergy across the organization and avoid misunderstanding.

"What gets measured gets done." Because metrics drive behavior, it is important to ensure that metrics used to manage the organization reflect its purpose and values, and that they do not drive perverse behaviors (Roedler et al. 2010).

Process and measurement cost money and time, so it is important to get the right amount of process definition and the right balance of investment between process, measurement, people and skills. Any process flexible enough to allow innovation will also be flexible enough to allow mistakes. If process is seen as excessively restrictive or prescriptive, it may inhibit innovation and demotivate the innovators in an effort to prevent mistakes, leading to excessive risk avoidance.

It is possible for a process improvement effort to become an end in itself rather than a means to improve business performance (Sheard 2003). To guard against this, it is advisable to remain clearly focused on purpose (Blockley and Godfrey 2000) and on added value (Oppenheim et al. 2010) as well as to ensure clear and sustained top management commitment to driving the process improvement approach to achieve the required business benefits. Good process improvement is as much about establishing a performance culture as about process.

The Systems Engineering process is an essential complement to, and is not a substitute for, individual skill, creativity, intuition, judgment etc. Innovative people need to understand the process and how to make it work for them, and neither ignore it nor be slaves to it. Systems Engineering measurement shows where invention and creativity need to be applied. SE process creates a framework to leverage creativity and innovation to deliver results that surpass the capability of the creative individuals – results that are the emergent properties of process, organisation, and leadership. (Sillitto 2011)

References

Works Cited

- Basili, V. 1992. "Software Modeling and Measurement: The Goal/Question/Metric Paradigm" Technical Report CS-TR-2956. University of Maryland: College Park, MD, USA. Accessed on August 28, 2012. Available at <https://www.cs.umd.edu/~basili/publications/technical/T78.pdf>.
- Blockley, D. and P. Godfrey. 2000. *Doing It Differently – Systems for Rethinking Construction*. London, UK: Thomas Telford Ltd.
- Davies, P. and N. Hunter. 2001. "System Test Metrics on a Development-Intensive Project." Paper presented at the 11th Annual International Council on System Engineering (INCOSE) International Symposium. 1-5 July 2001. Melbourne, Australia.
- Oppenheim, B., E. Murman, and D. Sekor. 2010. *Lean Enablers for Systems Engineering*. Systems Engineering. 14(1). New York, NY, USA: Wiley and Sons, Inc.
- Roedler, G. D. Rhodes, H. Schimmoller, and C. Jones (eds.). 2010. *Systems Engineering Leading Indicators Guide*, version 2.0. January 29, 2010, Published jointly by LAI, SEARI, INCOSE, and PSM. INCOSE-TP-2005-001-03. Accessed on September 14, 2011. Available at <https://seari.mit.edu/documents/SELI-Guide-Rev2.pdf>.
- SEI. 2010. *CMMI for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute/Carnegie Mellon University. CMU/SEI-2010-TR-033. Accessed on September 14, 2011. Available at <https://www.sei.cmu.edu/reports/10tr033.pdf>.

Sheard, S. 2003. "The Lifecycle of a Silver Bullet." *Crosstalk: The Journal of Defense Software Engineering*. (July 2003). Accessed on September 14, 2011. Available at <https://www.crosstalkonline.org/storage/issue-archives/2003/200307/200307-Sheard.pdf>.

Sillitto, H. 2011. Panel on "People or Process, Which is More Important". Presented at the 21st Annual International Council on Systems Engineering (INCOSE) International Symposium. 20-23 June 2011. Denver, CO, USA.

Woodcock, H. 2009. "Why Invest in Systems Engineering." INCOSE UK Chapter. Z-3 Guide, Issue 3.0. March 2009. Accessed on September 14, 2011. Available at https://www.incoseonline.org.uk/Documents/zGuides/Z3_Why_invest_in_SE.pdf.

Primary References

Basili, V. 1992. "Software Modeling and Measurement: The Goal/Question/Metric Paradigm". College Park, MD, USA: University of Maryland. Technical Report CS-TR-2956. Accessed on August 28, 2012. Available at <https://www.cs.umd.edu/~basili/publications/technical/T78.pdf>.

Frenz, P., et al. 2010. *Systems Engineering Measurement Primer: A Basic Introduction to Measurement Concepts and Use for Systems Engineering*, version 2.0. San Diego, CA, USA: International Council on System Engineering (INCOSE). INCOSE-TP-2010-005-02.

Oppenheim, B., E. Murman, and D. Sekor. 2010. *Lean Enablers for Systems Engineering*. Systems Engineering. 14(1). New York, NY, USA: Wiley and Sons, Inc.

Roedler, G., D. Rhodes, H. Schimmoller, and C. Jones (eds.). 2010. *Systems Engineering Leading Indicators Guide*, version 2.0. January 29, 2010, Published jointly by LAI, SEARI, INCOSE, PSM. INCOSE-TP-2005-001-03. Accessed on September 14, 2011. Available at <https://seari.mit.edu/documents/SELI-Guide-Rev2.pdf>.

Additional References

Jelinski, Z. and P.B. Moranda. 1972. "Software Reliability Research". In W. Freiberger. (ed.), *Statistical Computer Performance Evaluation*. New York, NY, USA: Academic Press. p. 465-484.

Alhazmi O.H. and Y.K. Malaiya. 2005. *Modeling the Vulnerability Discovery Process*. 16th IEEE International Symposium on Software Reliability Engineering (ISSRE'05). 8-11 November 2005. Chicago, IL, USA.

Alhazmi, O.H. and Y.K. Malaiya. 2006. "Prediction Capabilities of Vulnerability Discovery Models." Paper presented at Annual Reliability and Maintainability Symposium (RAMS). 23-26 January 2006. p 86-91. Newport Beach, CA, USA. Accessed on September 14, 2011. Available at <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1677355&isnumber=34933>.

Developing Systems Engineering Capabilities within Businesses and Enterprises

- Lead Authors:
 - Richard Beasley, Hillary Sillitto, and Alice Squires
- Contributing Authors:
 - Heidi Davidz and Art Pyster

The pursuit of continuous improvement is a constant for many organizations. The description of Toyota (Morgan and Liker 2006), the Lean principle of “pursue perfection” (Oppenheim et al. 2010), and the principle of “don’t let up” (Kotter 1995), all drive a need for continuous improvement.

The ability to manage teams through their lifecycle – mobilize teams rapidly, establish and tailor an appropriate set of processes, metrics and systems engineering plans, support them to maintain a high level of performance, capitalize acquired knowledge and redeploy team members expeditiously as the team winds down – is a key organizational competence that has substantial leverage on project and organizational efficiency and effectiveness.

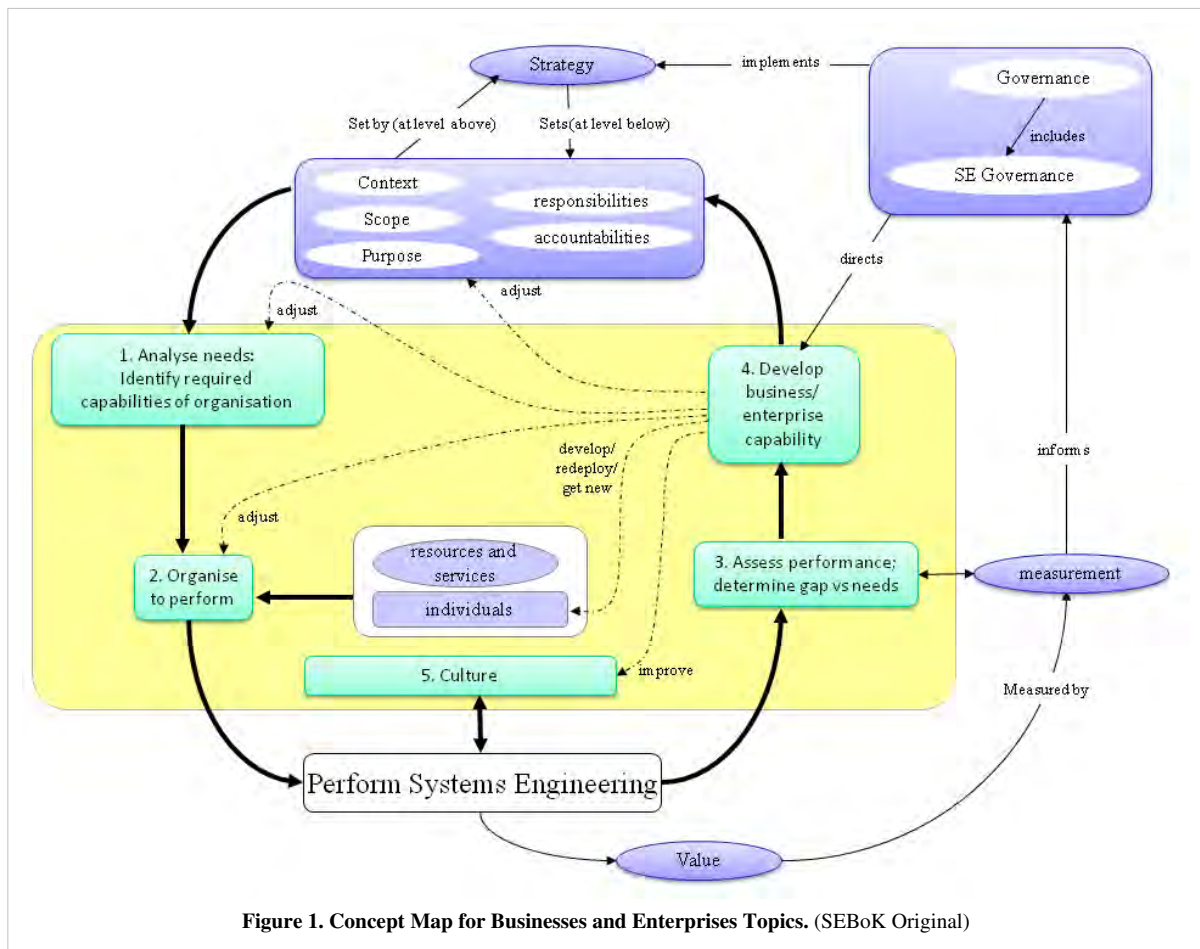
The enterprise provides teams with the necessary resources, background information, facilities, cash, support services, tooling, etc. It also provides a physical, cultural and governance environment in which the teams can be effective. The key functions of the enterprise include generating and maintaining relevant resources, allocating them to teams, providing support and governance functions, maintaining expertise and knowledge (on process, application domain and solution technologies), securing the work that teams perform, organizing finance, and maintaining the viability of the enterprise.

For improvements to persist, they must reside in the enterprise rather than just the individuals, so the improvements can endure as personnel leave. This is reflected in the Capability Maturity Model Integrated (CMMI) (SEI 2010) progression from a “hero culture” to a “quantitatively managed and optimizing process”.

This topic outlines the issues to be considered in capability development and organizational learning.

Overview

Figure 1 shows an “analyze – organize – perform – assess – develop” cycle, which is essentially a reformulation of the Deming (1994) PDCA (Plan Do Check Act) cycle. The analysis step should cover both current and future needs, as far as these can be determined or predicted. Goals and performance assessment, as discussed in *Assessing Systems Engineering Performance of Business and Enterprises*, can be based on a number of evaluation frameworks, such as direct measures of business performance and effectiveness and the CMMI capability maturity models. There is evidence that many organizations find a positive correlation between business performance and CMMI levels (SEI 2010). This is discussed further in the *Economic Value of Systems Engineering*.



Change Levers

SE managers have a number of possible change levers they can use to develop SE capabilities. The amount of time delay between moving a lever and seeing the effect varies with the type of level, size of the enterprise, culture of the enterprise, and other factors.

Adjust Context, Scope, Purpose, Responsibility, Accountability Business Enterprise

If the other change levers cannot achieve the desired effect, the business or enterprise may have to renegotiate its contribution to the higher-level strategy and mission.

Review and Adjust Required Capabilities

In the initial analysis the needed capability may have been over- or under-estimated. The need should be re-evaluated after each rotation of the cycle to make sure the planning assumptions are still valid.

Adjust Organization within Business Enterprise

Adjusting organization and responsibilities so that *"the right people are doing the right things"*, and ensuring that the organization is making full use of their knowledge and skills, is often the easiest change to make (and the one that may have the quickest effect).

A potential risk is that too much organizational churn disrupts relationships and can destabilize the organization and damage performance. Process improvement can be set back by an ill-considered re-organization and can jeopardize any certifications the organization has earned which demonstrate its process capability or performance.

Develop/Train/Redeploy/Get New Resources, Services and Individuals

Resources, services and individuals may include any of the components of organizational SE capability listed in Organizing Business and Enterprises to Perform Systems Engineering.

Leverage includes subcontracting elements of the work, improving information flows, upgrading facilities, and launching short-term training and/or long-term staff development programs. Many organizations consider how they approach these improvements to be proprietary, but organizations such as NASA offer insight on their APPEL website (NASA 2012).

Development of individuals is discussed in Enabling Individuals.

Improve Culture

Culture change is very important and powerful but needs to be handled as a long-term game and given long term commitment.

Adjust and Improve Alignment of Measures and Metrics

Measurement drives behavior. Improving alignment of goals and incentives of different parts of the business/enterprise so that everyone works to a common purpose can be a very effective and powerful way of improving business/enterprise performance. This alignment does require some top-down guidance, perhaps a top-down holistic approach, considering the business/enterprise as a system with a clear understanding of how the elements of enterprise capability interact to produce synergistic value (See Assessing Systems Engineering Performance of Business and Enterprises). It is commonly reported that as an organization improves its processes with respect to the CMMI, its approach to metrics and measurement has to evolve.

Change Methods

Doing Everyday Things Better

There is a wealth of sources and techniques, including Kaizen, Deming PDCA (Deming 1994), Lean (Womack and Jones 2003, Oppenheim et al. 2010), Six-Sigma (Harry 1997), and CMMI.

Value stream mapping is a powerful Lean technique to find ways to improve flow and handovers at interfaces.

Managing Technology Readiness

In high-technology industries many problems are caused by attempting to transition new technologies into products and systems before the technology is mature; to make insufficient allowance for the effort required to make the step from technology demonstration to reproducible and dependable performance in a product; or to overestimate the re-usability of an existing product. NASA's TRL (Technology Readiness Level) construct, first proposed by John Mankins in 1995 (Mankins 1995), is widely and successfully used to understand and mitigate technology transition risk. Several organizations beyond NASA, such as the U.S. Department of Defense, even have automation to aid engineers in evaluating technology readiness.

Variations on TRL have emerged, such as System Readiness Levels (SRL) (Sauter et al. 2006), which recognize that the ability to successfully deliver systems depends on much more than the maturity of the technology base used to create those systems; e.g., there could be surprising risks associated with using two technologies that are relatively mature in isolation, but have never been integrated together before.

Planned Change: Standing Up or Formalizing SE in an Organization

Planned change may include:

- introducing SE to a business (Farncombe and Woodcock 2009);
- improvement/transformation;
- formalizing the way a business or project does SE;
- dealing with a merger/demerger/major re-organization;
- developing a new generation or disruptive product, system, service or product line (Christensen 1997);
- entering a new market; and
- managing project lifecycle transitions: start-up, changing to the next phase of development, transition to manufacture/operation/support, wind down and decommissioning.

CMMI is widely used to provide a framework for planned change in a systems engineering context. Planned change needs to take a holistic approach considering people (knowledge, skills, culture, ability and motivation), process, measurement and tools as a coherent whole. It is now widely believed that tools and process are not a substitute for skills and experience. Instead, they merely provide a framework in which skilled and motivated people can be more effective. Therefore, change should start with people rather than with tools.

Before a change is started, it is advisable to baseline the current business performance and SE capability and establish metrics that will show early on whether the change is achieving the desired effect.

Responding to Unforeseen Disruption

Unforeseen disruptions may be internally or externally imposed. Externally imposed disruptions may be caused by:

- the customer – win/lose contract, mandated teaming or redirection;
- competitors – current offering becomes more/less competitive, a disruptive innovation may be launched in market; or
- governance and regulatory changes – new processes, certification, safety or environmental standards.

Internal or self-induced disruptions may include:

- a capability drop-out due to loss of people, facilities, financing;
- product or service failure in operation or disposal; or
- strategy change (e.g. new CEO, response to market dynamics, or a priority override).

Embedding Change

In an SE context, sustained effort is required to maintain improvements such as higher CMMI levels, Lean and Safety cultures, etc., once they are achieved. There are several useful change models, including Kotter's 8 phases of change (Kotter 1995):

1. Establish a sense of urgency;
2. Create a coalition;
3. Develop a clear vision;
4. Share the vision;
5. Empower people to clear obstacles;
6. Secure short-term wins;
7. Consolidate and keep moving; and
8. Anchor the change.

The first six steps are the easy ones. The Chaos Model (Zuijderhoudt 1990; 2002) draws on complexity theory to show that regression is likely if the short-term wins are not consolidated, institutionalized and anchored. This explains the oft-seen phenomenon of organizations indulging in numerous change initiatives, none of which stick because attention moves on to the next before the previous one is anchored.

Change Management Literature

SE leaders (directors, functional managers, team leaders and specialists) have responsibilities, and control levers to implement them, that vary depending on their organization's business model and structure. A great deal of their time and energy is spent managing change in pursuit of short-, medium- and long-term organizational goals: "doing everyday things better"; making change happen; embedding change and delivering the benefit; and coping with the effects of disruptions. Mergers, acquisitions and project start-ups, phase changes, transitions from "discovery" to "delivery" phase, transition to operation, sudden change in level of funding, can all impose abrupt changes on organizations that can destabilize teams, processes, culture and performance. Table 1 below provides both the general management literature and specific systems engineering knowledge.

Table 1. Change Management – Business and SE References. (SEBoK Original)

Area	Business references	SE references
Doing Every-day Things Better	• Kaizen; Lean (Womack and Jones 2003); 6-Sigma (Harry 1997) • Four Competencies of Learning Organisation – Absorb, Diffuse, Generate, Exploit (Sprenger and Ten Have 1996) • The Seven Habits of Very Effective People (Covey 1989)	• CMMI • Visualizing Project Management (Forsberg and Mooz 2005) • INCOSE IEWG "Conops for a Systems Engineering Educational Community" (Ring and Wymore 2004) • INCOSE Lean Enablers for SE (Oppenheim et al. 2010)
Dealing with Unplanned Disruption	• Managing Crises Before They Happen (Mitroff and Anagnos 2005); • Scenarios: Uncharted Waters Ahead (Wack 1985) • Scenario Planning: Managing for the Future (Ringland 1988)	• Architecting Resilient Systems (Jackson 2010) • Design Principles for Ultra-Large-Scale Systems (Sillitto 2010)
Driving Disruptive Innovation	• The Innovator's Dilemma (Christensen 1997) • Rise and Fall of Strategic Planning, (Mintzberg 2000) • BS7000, Standard for Innovation Management (BSI 2008)	
Exploiting Unexpected Opportunities	• Rise and Fall of Strategic Planning (Mintzberg 2000) • Mission Command (military), Auftragstechnik (Bungay 2002, 32)	• Architecting for Flexibility and Resilience (Jackson 2010) • Open System Architectures; Lean SE; (Oppenheim et al. 2010) • Agile Methodologies
Implementing and Embedding Planned Change	• Kotter's Eight Phases of Change (Kotter 1995), • Berenschot's Seven Forces (ten Have et al. 2003) • Levers of Control (Simons 1995) – Tension between Control, Creativity, Initiative and Risk Taking • Chaos Model from "Complexity Theory Applied to Change Processes in Organisations"; (Zuiderhoudt and Ten Have 1999) • Business Process Re-engineering (Hammer and Champy 1993) • The 5th Discipline (Senge 2006) • Change Quadrants (Amsterdam 1999)	• Doing it differently - Systems for Rethinking Construction (Blockley and Godfrey 2000) • INCOSE UK Chapter Z-guides: • Z-2, Introducing SE to an Organisation (Farncombe and Woodcock 2009); • Z-7, Systems Thinking (Godfrey and Woodcock 2010)
Understanding People's Motivation, Behavior	• Maslow's Hierarchy of Needs • Myers-Briggs Type Indicator; • NLP (Neuro-Linguistic Programming) (See for example: Knight 2009) • Performance by Design: Sociotechnical Systems in North America (Taylor and Felten 1993) • Core Quadrants, (Offman 2001)	• INCOSE Intelligent Enterprise Working Group – "Enthusiasm", Stretch Goals (Ring and Wymore 2004) • Sociotechnical Systems Engineering, Responsibility Mapping, from "Deriving Information Requirements from Responsibility Models" (Sommerville et al. 2009)

- | | |
|--------------------------------------|---|
| Understanding Culture | <ul style="list-style-type: none"> • Cultural Dimensions, from "Culture's Consequences" (Hofstede 1994) • Compliance Typology, from "A Comparative Analysis of Complex Organizations" (Etzione 1961) |
| Helping Individuals Cope with Change | <ul style="list-style-type: none"> • 5 C's of Individual Change, and Rational/Emotional Axes, Kets De Vries, quoted in "Key Management Models" (Ten Have et al. 2003) • Rational/Emotional, NLP and Other Methods, from "Relationships Made Easy" (Fraser 2010) |

References

Works Cited

- Blockley, D. and P. Godfrey. 2000. *Doing It Differently – Systems for Rethinking Construction*. London, UK: Thomas Telford, Ltd.
- Bungay, S. 2002. *Alamein*. London, UK: Aurum press. First published 2002, Paperback 2003.
- BSI. 2008. *Design Management Systems. Guide to Managing Innovation*. London, UK: British Standards Institution (BSI). BS 7000-1:2008.
- Christensen, C. 1997. *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail*. Cambridge, MA, USA: Harvard Business School Press.
- Covey, S.R. 1989. *The Seven Habits of Highly Effective People*. Also released as a 15th Anniversary Edition (2004). New York, NY, USA: Simon & Schuster, 1989.
- Deming, W.E. 1994. *The New Economics*. Cambridge, MA, USA: Massachusetts Institute of Technology, Centre for Advanced Educational Services.
- Etzione, A. 1961. *A Comparative Analysis of Complex Organizations. On Power, Involvement and their Correlates*. New York, NY, USA: The Free Press of Glencoe, Inc.
- Farncombe, A. and H. Woodcock. 2009. "Enabling Systems Engineering." Somerset, UK: INCOSE UK Chapter. *Z-2 Guide*, Issue 2.0 (March 2009). Accessed September 14, 2011. Available at https://www.incoseonline.org.uk/Documents/zGuides/Z2_Enabling_SE.pdf.
- Forsberg, K. and H. Mooz. 2005. *Visualizing Program Management, Models and Frameworks for Mastering Complex Systems*, 3rd ed. New York, NY, USA: Wiley and Sons, Inc.
- Fraser, D. 2010. *Relationships Made Easy: How to Get on with The People You Need to Get on with...and Stay Friends with Everyone Else*. Worcestershire, UK: HotHive Publishing.
- Godfrey, P. and H. Woodcock. 2010. "What is Systems Thinking?" Somerset, UK: INCOSE UK Chapter, *Z-7 Guide*, Issue 1.0 (March 2010). Accessed on September 7, 2011. Available at https://www.incoseonline.org.uk/Documents/zGuides/Z7_Systems_Thinking_WEB.pdf.
- Hammer, M. and J.A. Champy. 1993. *Reengineering the Corporation: A Manifesto for Business Revolution*. New York, NY, USA: Harper Business Books.
- Harry, M.J. 1997. *The Nature of Six Sigma Quality*. Schaumburg, IL, USA: Motorola University Press.
- Hofstede, G. 1984. *Culture's Consequences: International Differences in Work-Related Values*. Newbury Park, CA, USA and London, UK: Sage Publications Inc.
- Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. A. P. Sage (ed.). Wiley Series in Systems Engineering and Management. New York, NY, USA: Wiley & Sons, Inc.
- Knight, S. 2009. *NLP at Work - Neuro Linguistic Programming The Essence of Excellence*, 1st edition 1995. 3rd edition 2009. London, UK and Boston, MA, USA: Nicholas Brealey Publishing.

- Kotter, J. 1995. "Leading Change: Why Transformation Efforts Fail". "Harvard Business Review." (March-April 1995).
- Mintzberg, H. 2000. *The Rise and Fall of Strategic Planning*. Upper Saddle River, NJ, USA: Pearson Education.
- Mitroff, I. and G. Anagnos. 2005. *Managing Crises Before They Happen: What Every Executive and Manager Needs to Know about Crisis Management*. New York, NY, USA: AMACOM Press.
- Morgan, J. and J. Liker. 2006. *The Toyota Product Development System: Integrating People, Process and Technology*. New York, NY, USA: Productivity Press.
- NASA. 2012. "APPEL: Academy of Program/Project & Engineering Leadership." Accessed on September 9, 2012. Available at <https://www.nasa.gov/offices/oce/appel/seldp/nasa-se/index.html>.
- Offman, D.D. 2001. *Inspiration and Quality in Organizations*, (Original title (Dutch): *Bezieling en Kwaliteit in Organisaties*), 12th Edition. Utrecht, The Netherlands: Kosmos-Z&K.
- Oppenheim, B., E. Murman, and D. Sekor. 2010. *Lean Enablers for Systems Engineering*. Systems Engineering. 14(1). New York, NY, USA: Wiley and Sons, Inc.
- Ring, J. and A.W. Wymore (eds.) 2004. *Concept of Operations (conops) of A Systems Engineering Education Community (SEEC)*. Seattle, WA, USA: INCOSE Education Measurement Working Group (EMWG), INCOSE-TP-2003-015-01.
- Ringland, G. 1998. *Scenario Planning: Managing for the Future*. New York, NY, USA: Wiley and Sons, Inc.
- Sauser, B., D. Verma, J. Ramirez-Marque and R. Gove 2006. "From TRL to SRL: The Concept of System Readiness Levels." Proceedings of the Conference on Systems Engineering Research (CSER), Los Angeles, CA.
- SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- Senge, P.M. 2006. *The Fifth Discipline: The Art and Practice of the Learning Organization*, 2nd ed. New York, NY, USA: Doubleday/Currency.
- Simons, R. 1995. *Levers of Control, How Managers Use Innovative Control Systems to Drive Strategic Renewal*. Boston, MA, USA: Harvard Business School Press.
- Sillitto, H. 2010. "Design Principles for Ultra-Large-Scale Systems." Paper in 20th Annual International Council on Systems Engineering (INCOSE) International Symposium. 12-15 July 2010. Chicago, IL, USA. (Reprinted in The Singapore Engineer, IES, April 2011).
- Sommerville, I., R. Lock, T. Storer, and J.E. Dobson. 2009. "Deriving Information Requirements from Responsibility Models. Paper in the 21st International Conference on Advanced Information Systems Engineering (CAiSE). June 2009. Amsterdam, Netherlands. p. 515-529.
- Sprenger, C. and S. Ten Have. 1996. "4 Competencies of a Learning Organisation." (Original title (Dutch): *Kennismanagement Als Motor van Deleurende Organisatie*), *Holland Management Review*, (Sept-Oct): 73-89.
- Taylor, J.C. and D.F. Felten. 1993. *Performance by Design: Sociotechnical Systems in North America*. Englewood Cliffs, NJ, USA: Pearson Education Ltd. (Formerly Prentice Hall).
- ten Have, S., W.T. Have, F. Stevens, and M. van der Elst. 2003. *Key Management Models - The Management Tools and Practices That Will Improve Your Business*. Upper Saddle River, NJ, USA: Pearson Education Ltd. (Formerly Prentice Hall).
- Wack, P. 1985. "Scenarios: Uncharted Waters Ahead." "Harvard Business Review:." (September-October 1985).
- Womack, J. and D. Jones. 2003. *Lean Thinking: Banish Waste and Create Wealth in Your Corporation*, Revised Edition. New York, NY, USA: Simon & Schuster.
- Zuiderhoudt, W. and B. Ten Have. 1999. *Complexity Theory Applied to Change Processes in Organisations*.
-

Primary References

- Kotter, J. 1995. *Leading Change: Why Transformation Efforts Fail*. "Harvard Business Review." (March-April 1995).
- Oppenheim, B., E. Murman, and D. Sekor. 2010. *Lean Enablers for Systems Engineering*. "Systems Engineering." 14(1). New York, NY, USA: Wiley and Sons, Inc.
- SEI. 2010. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.3. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).
- Senge, P.M. 2006. *The Fifth Discipline: The Art and Practice of the Learning Organization*, 2nd ed. New York, NY, USA: Doubleday/Currency.

Additional References

None.

Barriers to Successful Embedding of Systems Engineering into Organizations

- Lead Author:
- Bernardo Delicado
- Contributing Author:
- Tom Strandberg, Ivan Mactaggart

The landscape of industry is evolving rapidly, presenting formidable challenges in the development of complex technical products. Systems Engineering (SE) emerges as a promising solution to tackle these challenges. In recent years, the awareness and significance of SE have surged within industry sectors such as automotive, railway, aerospace, and defense. However, the actual embedding of SE into organizational frameworks poses a significant challenge for many companies operating within these sectors and others.

Introducing SE entails not only dealing with technical complexity but also significant human aspects of change which are critical to ensuring stakeholder engagement, smooth adoption, and the long-term success of the initiative. Various impacts of change, including restructuring, reengineering, and cultural shifts, must be addressed through effective change management techniques to ensure successful SE integration.

This article aims to delve into the complexities surrounding the embedding of SE into organizations. By examining the struggles encountered by companies, we seek to uncover the underlying reasons behind these challenges. Expectations towards SE are scrutinized, and an analysis of the barriers hindering its adoption is presented, drawing insights from the experiences of two non-participating customers. Additionally, we outline ten essential requirements crucial for the successful embedding of SE, comparing them with available introduction methodologies.

Industrial Challenges

Systems Engineering (SE) offers an approach to handle complexity. Despite these potential benefits in product development, the adoption of SE varies significantly by industry, being more prevalent in the aerospace, defense, and automotive sectors than in other areas. Furthermore, SE is more commonly used in large corporations than small and medium-sized enterprises (SMEs). SMEs face challenges in accessing and building SE expertise due to limited resources and varying SE needs depending on the specific projects and applications of each SME.

The complexity of products and their development is rising due to trends like digitalization, globalization, and sustainability. Companies are encountering new hurdles as they transition from traditional mechatronic products to

autonomous, interactive, and dynamically networked systems. This shift, along with the increasing interdisciplinarity of their systems and enhanced networking with other systems, contributes to the escalating complexity in product development.

While SE is a holistic approach to address such challenges, its adoption remains relatively nascent within organizations. Historically, development approaches have been discipline-oriented, lacking the systemic perspective required for tomorrow's products. Consequently, the introduction of SE presents a formidable challenge, given the established discipline-centric mindset prevalent in all industries and application domains.

A Holistic Approach

The Capability Model approach recognizes that building a lasting organizational capability requires more than process and tools. As illustrated in the Figure 1 below, the elements that build up a SE organizational capability include Governance, Organisation, Process, Process support, Information and Technology & Infrastructure.

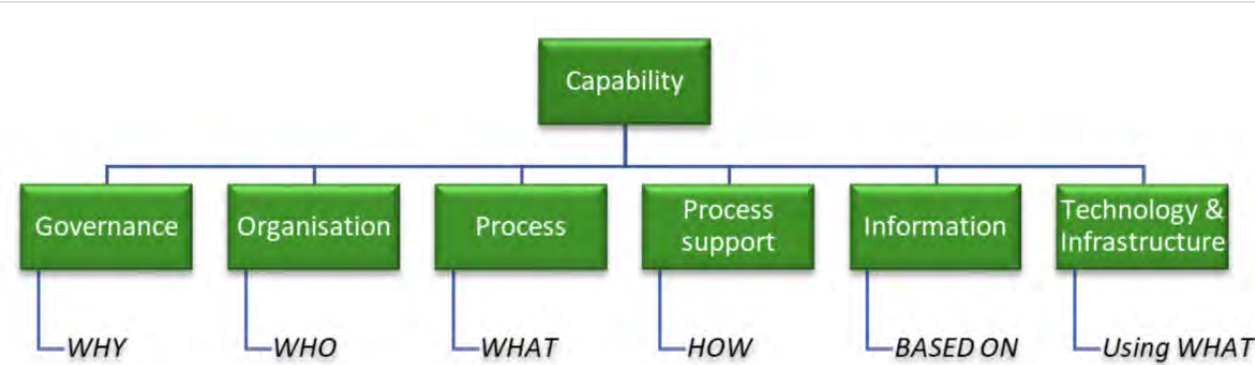


Figure 1. The Capability Model (Strandberg, et al, 2024). Copyright © (2024). All Rights Reserved. Reprinted by Permission. All other rights are reserved by the copyright owner.

Furthermore, the level of organizational capability needed should be based on the context in which it operates. The contextual factors can be divided into Business, Organizational and Product related aspects as shown in Figure 2.

A capability the ability of an entity (department, organization, person, system) to achieve its objectives, specifically in relation to its overall mission. Only when this is used in business operations will the desired effects, such as improved quality, reduced lead-times and enhanced cost-effectiveness, be realized.

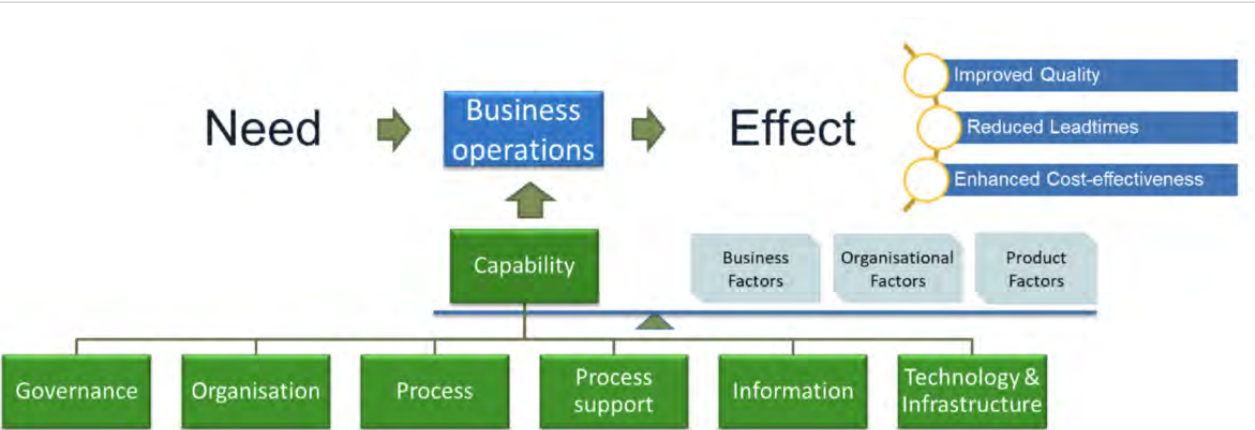
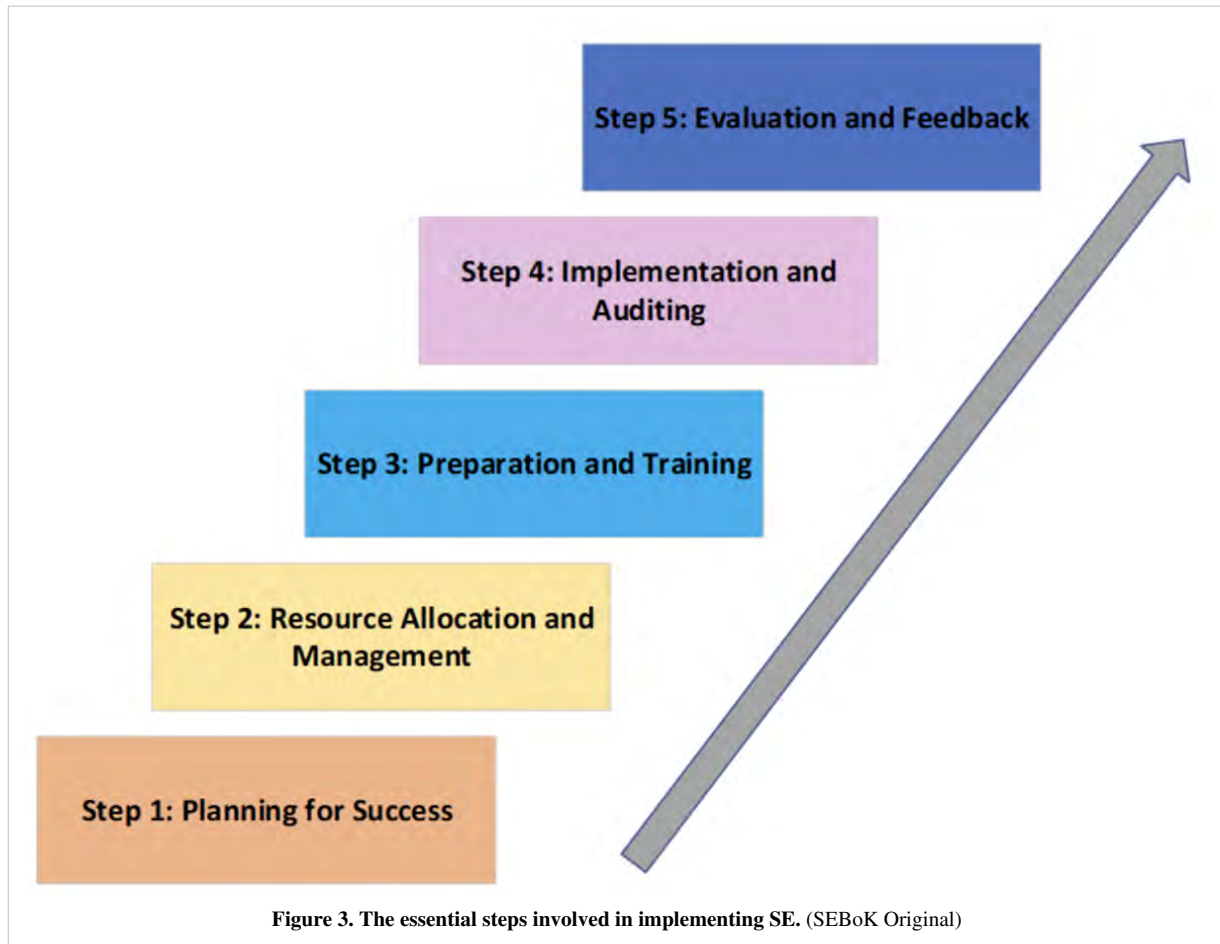


Figure 2. The Capability Model with Contextual Factors (Strandberg, et al, 2024). Copyright © (2024). All Rights Reserved. Reprinted by Permission. All other rights are reserved by the copyright owner.

Implementation of recognized SE standards

Any embedding of SE in an organization should start with a formulation of the Why. As illustrated in the Capability Model, this falls under the Governance element. One common approach involves the tailoring and implementation of recognized standards, such as those outlined by the International Organization for Standardization (ISO). This strategic framework outlines the essential steps involved in implementing SE standards as shown in Figure 3, offering organizations a roadmap for success.



Step 1: Planning for Success

The first stage in implementing an SE standard is thorough planning to ensure smooth integration. This phase includes:

- To understand the current maturity of the organization, baselining what is done or not done and then considering how you might effectively implement.
- Identifying the appropriate industry standards that the organization will seek to comply with and the level of capability desired. Many of these standards describe a list of activities to be performed by an organization and its projects to demonstrate the capability.
- Identifying gaps and areas of non-compliance by mapping current activities against the established activities within the standards for the capability to be achieved.
- Evaluating the necessary process and organizational changes for successful embedding of SE.
- Securing commitment and support from organizational leadership for the resources required (and maintaining this commitment throughout the implementation process).

Step 2: Resource Allocation and Management

Proper resource allocation is critical for successful SE standard implementation. This step involves:

- Appointing a dedicated SE standard implementation manager to oversee the process and ensure timelines and objectives are met.
- Developing a detailed resource plan and schedule, outlining the allocation of personnel, finances, and technological infrastructure needed for SE implementation.
- Gaining approval from organizational leadership for funding and scheduling to ensure smooth execution of the implementation plan.

Step 3: Preparation and Training

Preparing the organization for embedding of SE requires effective communication and comprehensive training initiatives. This phase includes:

- Creating a communication and training plan to disseminate information about SE standards and build organizational readiness.
- Updating organizational policies, procedures, and work instructions to incorporate new SE tasks and methodologies.
- Conducting training sessions at all levels of the organization to provide personnel with the necessary knowledge and skills for SE implementation.

Step 4: Implementation and Auditing

The implementation phase involves systematically executing SE standards, with ongoing monitoring and evaluation. Key activities include:

- Implementing one action or item at a time to ensure gradual and sustainable integration.
- Performing regular audits to assess adherence to SE standards and identify areas for improvement.

Step 5: Evaluation and Feedback

Evaluating the effectiveness of SE implementation is essential to understand its impact and identify opportunities for refinement. This phase involves:

- Managing schedules and resources to facilitate thorough reviews of newly implemented standard tasks.
- Informing the organization upon project completion, highlighting achievements, and outlining future milestones.

Requirements for SE Introduction

The successful embedding of SE into organizational workflows requires careful consideration of various factors and adherence to specific requirements. The integration of SE standards serves as a cornerstone for organizations aiming to optimize their engineering processes and deliver high-quality products and services to market.

By adhering to established standards and best practices, companies can effectively navigate the complexities of modern engineering challenges while ensuring consistency, reliability, and efficiency across all facets of their operations. However, the journey towards SE integration is not without its challenges, as organizations must navigate a myriad of considerations to ensure successful implementation.

As we explore the nine crucial requirements for embedding of SE outlined below, it becomes evident that a strategic and holistic approach is essential for realizing the full potential of SE within an organizational context:

1. Create the vision and roadmap for embedding of SE
 2. Providing a flexible implementation strategy tailored to organizational needs.
 3. Providing comprehensive support for SE rollout within organizations.
-

4. Incorporating change management strategies into SE implementation efforts.
5. Tailoring SE considering the characteristics of companies or organizations (first level of tailoring).
6. Implementing SE through both top-down and bottom-up approaches.
7. Introducing tailoring to adapt SE approaches to specific project requirements (second level of tailoring).
8. Integrating SE with Model-Based Systems Engineering (MBSE).
9. Ensuring the selection of a tool or tools that meet your needs in terms of processes and methods.

Barriers for Organizations

While the adoption of standardized best practices as provided by ISO standards lays a solid foundation for embedding of SE within organizations, the journey is often fraught with challenges and barriers that must be navigated with precision and foresight.

From rigid organizational structures to cultural resistance and the nebulous nature of quantifiable benefits, each barrier presents unique challenges that must be addressed to ensure the successful integration of SE within organizational workflows.

By understanding and addressing these barriers head-on, organizations can navigate the complexities of SE integration with confidence and precision, unlocking the full potential of standardized methodologies to drive innovation, efficiency, and competitiveness in today's dynamic business landscape.

Rigid Organizational Structures

The inherent rigidity of organizational structures poses a significant obstacle to SE integration, hindering the necessary adaptations and restructuring required to accommodate new methodologies and approaches.

Slow Decision-Making Processes

Delays in decision-making processes impede the timely implementation of SE methodologies, prolonging the transition and inhibiting organizational agility and responsiveness.

Resistance to Change

Cultural resistance and entrenched silo thinking contradict the holistic principles of SE, necessitating a fundamental cultural shift and fostering a culture of collaboration and cooperation across organizational boundaries.

Specialization Challenges

While specialization is essential for complex product development, it can complicate the internal allocation of SE roles, which often require a more generalized skill set and interdisciplinary collaboration.

Non-Quantifiable Benefits

The intangible nature of SE benefits, coupled with a lack of key performance indicators (KPIs) to substantiate its efficacy, poses significant challenges to its adoption and integration within organizations.

Insufficient Provision of Resources

Inadequate provision of introduction methods, expertise, and resources further complicates the adoption and successful implementation of SE methodologies, undermining the effectiveness of integration efforts.

Neglecting Non-Technical Aspects

Overlooking non-technical aspects such as stakeholder needs and inadequate change management strategies can undermine the success of SE integration efforts, emphasizing the importance of a holistic and comprehensive approach.

Lack of skilled professionals

One of the major barriers to integrating SE in an organization is the shortage of well-trained and professionally competent personnel in this field. The absence of qualified experts hinders the effective implementation of SE methods and tools, which can slow down projects and impact the quality of outcomes.

Conclusion

Embedding of SE into organizations requires a holistic approach, covering both establishing the appropriate level of capability and the careful implementation thereof.

While the adoption of established SE standards provides a solid foundation for integration, organizations must also recognize the need to tailor these approaches to suit their specific project requirements and organizational objectives. By combining standardized methodologies with bespoke strategies tailored to their unique needs, companies can leverage the full potential of SE to drive value creation and achieve sustainable competitive advantage.

In conclusion, the successful embedding of Systems Engineering into organizations requires a multifaceted approach addressing technical, cultural, and organizational challenges. By understanding and addressing these barriers, companies can unlock the full potential of SE to drive innovation and achieve sustainable growth.

References

Primary References

Bretza, Lukas et al. 2019. An analysis of barriers for the introduction of Systems Engineering. 29th CIRP Design 2019 (CIRP Design 2019). Available online at www.sciencedirect.com.

Castellanos, Octavio 2017. Overview of ISO/IEC/IEEE 15288. INCOSE – North Texas Chapter, May 9th 2017

Wilke, D. et al. Lessons Learned from the Introduction of Systems Engineering. Systems 2023, 11, 119. <https://doi.org/10.3390/systems11030119>

Additional References

De Landsheer, Bram et al. 2006. Implementing Systems Engineering: A Step-By-Step Guide. Fifth European Systems Engineering Conference, EuSec, 18-20 September 2006

Bretza, Lukas et al. 2020. A contribution to the design of organizational structures suitable for Systems Engineering. 30th CIRP Design 2020 (CIRP Design 2020). Available online at www.sciencedirect.com

Strandberg, Tom et al. 2024. Systems Engineering Capability Development using the “Green and Blue Track Approach”, Presentation #475, INCOSE International Symposium 2024

Davidz, Heidi L. and Martin, James N. 2010. Defining a Strategy for Development of Systems Capability in the Workforce. DOI 10.1002/sys.20167

Culture

- Lead Authors:
 - Scott Jackson, Hillary Sillitto, and John Snoderly
- Contributing Authors:
 - Richard Turner, Art Pyster, and Richard Beasley

Establishing and managing cultures, values, and behaviors is a critical aspect of systems engineering, especially in the context of deploying SE within an organization (Fasser and Brettner 2002). The Columbia Accident Investigation Report (NASA 2003, 101), defines *culture* as “the basic values, norms, beliefs, and practices that characterize the functioning of a particular institution.”

Stable safety and process cultures are key to effective SE, and can be damaged by an overly-rapid pace of change, a high degree of churn (see the Nimrod Crash Report, Haddon-Cave 2009), or by change that engineers perceive as arbitrarily imposed by management (see Challenger, discussed below). On the other hand, a highly competitive, adversarial or “blame” culture can impede the free flow of information and disrupt synergies in the workplace.

In the multi-national, multi-business, multi-discipline collaborative projects becoming increasingly prevalent in SE, these factors take on greater importance.

Effective handling of cultural issues is a major factor in the success or failure of SE endeavors.

Systems Thinking and the Culture of the Learning Organization

Improving SE efficiency and effectiveness can be the goal of culture change. This kind of culture change encourages people to learn to think and act in terms of systems, organizations and their enterprises; and, to take a systems approach as described in Overview of Systems Approaches in Part 2, and by Lawson (2010). See the knowledge area Systems Thinking.

Attaining a *learning organization* culture can be another goal of cultural change. And once the learning organization exists, cultural change in general becomes easier to accomplish.

A learning organization aims to absorb, diffuse, generate, and exploit knowledge (Sprenger and Have 1996). Organizations need to manage formal information and facilitate the growth and exploitation of tacit knowledge. They should learn from experience and create a form of *corporate memory* – including process, problem domain and solution space knowledge, and information about existing products and services. Fassner and Brettner (2002, 122-124) suggest that *shared mental models* are a key aspect of corporate knowledge and culture.

A learning organization culture is enabled by disciplines such as:

- **personal mastery**, where a person continually clarifies and deepens personal vision, focuses energy upon it and develops patience in seeking it so as to view reality in an increasingly objective way;
- **mental models**, where people appreciate that mental models do indeed occupy their minds and shape their actions;
- **shared vision**, where operating values and sense of purpose are shared to establish a basic level of mutuality; and
- **team learning**, where people’s thoughts align, creating a feeling that the team as a whole achieves something greater than the sum of what is achieved by its individual members.

Systems thinking supports these four disciplines, and in so doing becomes the **fifth discipline** and plays a critical role in promoting the learning organization (Senge et al. 1994).

Cultural Shortfalls and How to Change Them

Cultural shortfalls that are injurious to a system are described as negative paradigms by Jackson (2010) and others. For example, a cultural reluctance to identify true risks is the hallmark of the **Risk Denial** paradigm as seen in the Challenger and Columbia cases. When individuals believe a system is safe that is in fact unsafe, that is the **Titanic Effect** paradigm, which is of course named for the ocean liner catastrophe of 1912.

Approaches to Change

Jackson and Erlick (Jackson 2010, 91-119) have found that there is a lack of evidence that a culture can be changed from a success point of view. However, they do suggest the Community of Practice (Jackson 2010, 110-112), an approach founded on the principles of organizational psychology, and discuss the pros and cons of other approaches to culture change, including training, coaching, Socratic teaching, use of teams, independent reviews, standard processes, rewards and incentives, use of cost and schedule margins, reliance on a charismatic executive, and management selection. Shields (2006) provides a similarly comprehensive review.

The Columbia Accident (NASA 2003) and the Triangle fire (NYFIC 1912) official reports, among many others, call for cultural issues to be addressed through improved leadership, usually augmented by the more objective approach of auditing. One form of auditing is the Independent Technical Authority, which:

- is separate from the program organization;
- addresses only technical issues, not managerial ones; and
- has the right to take action to avoid failure, including by vetoing launch decisions.

An Independent Technical Authority cannot report to the program manager of the program in question, and it may be formulated within an entirely separate business or enterprise which can view that program objectively. The point of these stipulations is to ensure that the Independent Technical Authority is indeed independent.

Management and leadership experts have identified ways to lead cultural change in organizations, apart from specifically safety-related cultural change. For example, Gordon (1961) in his work on the use of analogical reasoning called synectics is one of several who emphasize creative thinking. Kotter (1995) advocates a series of steps to transform an organization.

How Culture Manifests in Individuals and Groups

As a community's physical, social, and religious environment changes over the generations, cultural beliefs, values, and customs evolve in response, albeit at a slower pace.

Helmreich and Merritt describe the effects of cultural factors in the context of aviation safety and suggest implications for safety cultures in other domains such as medicine. See (Helmreich and Merritt, 2000) and other writings by the same authors.

We can describe the cultural orientation of an individual in terms of:

- national and/or ethnic culture;
- professional culture; and
- organizational culture.

Some particulars of these aspects of culture are sketched below.

National and/or Ethnic Culture

A product of factors such as heritage, history, religion, language, climate, population density, availability of resources, politics, and national culture is acquired in one's formative years and is difficult to change. National culture affects attitudes, behavior, and interactions with others.

National culture may help determine how a person handles or reacts to:

- rules and regulations;
- uncertainty; and
- display of emotion, including one's own.

National culture may also play a role in whether a person

- communicates in a direct and specific style, or the opposite;
- provides leadership in a hierarchical manner, or a consultative one; and
- accepts decisions handed down in superior–inferior relationships, or questions them.

Professional Culture

Professional culture acts as an overlay to ethnic or national culture, and usually manifests in a sense of community and in bonding based on a common identity (Helmreich and Merritt 2000). Well-known examples of professional cultures include those of medical doctors, airline pilots, teachers, and the military.

Elements of professional culture may include:

- a shared professional jargon
- binding norms for behavior
- common ethical values
- self-regulation
- barriers to entry such as selectivity, competition and training
- institutional and/or individual resistance to change
- prestige and status, sometimes expressed in badges or uniforms
- stereotyped notions about members of the profession, in general and/or based on gender

Particularly important elements of professional culture (for example, those that affect safety or survivability) need to be inculcated by extensive training and reinforced at appropriate intervals.

Organizational Culture

An organization's culture builds up cumulatively, determined by factors like its leadership, products and services, relationships with competitors, and role in society.

Compared with one another, organizational cultures are not standardized because what works in one organization seldom works in another. Even so, strength in the following elements normally engenders a strong organizational culture:

- corporate identity;
 - leadership;
 - morale and trust;
 - teamwork and cooperation;
 - job security;
 - professional development and training;
 - empowerment of individuals; and
 - confidence, for example in quality and safety practices, or in management communication and feedback.
-

When the culture of the people in an organization is considered as a whole, organizational culture acts as a common layer shared by all. Despite this, differing national cultures can produce differences in leadership styles, manager-subordinate relationships, and so on, especially in organizations with a high degree of multinational integration.

Because organizations have formal hierarchies of responsibility and authority, organizational culture is more amenable to carefully planned change than are either professional or national cultures. If changes are made in a manner that is sympathetic to local culture (as opposed to that of a distant group head office, for example), they can bring significant performance benefits. This is because organizational culture channels the effects of national and professional cultures into standard working practices.

There are many definitions of culture in the literature. The Columbia Accident Investigation Board (NASA 2003) provides a useful definition for understanding culture and engineering.

Culture and Safety

Reason (1997, 191-220) describes a culture which focuses on safety as having four components:

1. A reporting culture which encourages individuals to report errors and near misses, including their own.
2. A just culture which provides *an atmosphere of trust in which people are encouraged, even rewarded, for providing essential safety-related information.*
3. A flexible culture which abandons the traditional hierarchical reporting structure in favor of more direct team-to-team communications.
4. A learning culture which is willing to draw the right conclusions from safety-related information and to implement reforms when necessary.

Weick and Sutcliffe (2001, 3) introduce the term high reliability organizations (HROs). HROs have *fewer than their fair share of accidents* despite operating *under trying conditions* in domains subject to catastrophic events. Examples include *power grid dispatching centers, air traffic control systems, nuclear aircraft carriers, nuclear power generation plants, hospital emergency departments, and hostage negotiation teams.* There are five hallmarks of HROs (Weick and Sutcliffe 2001, 10):

1. **Preoccupation with Failure**—HROs eschew complacency, learn from near misses, and do not ignore errors, large or small.
2. **Reluctance to Simplify Interpretations**—HROs simplify less and see more. They “encourage skepticism towards received wisdom.”
3. **Sensitivity to Operations**—HROs strive to detect “latent failures,” defined by James Reason (1997) as systemic deficiencies that amount to accidents waiting to happen. They have well-developed situational awareness and make continuous adjustments to keep errors from accumulating and enlarging.
4. **Commitment to Resilience**—HROs keep errors small and improvise “workarounds that keep the system functioning.” They have a deep understanding of technology and constantly consider worst case scenarios in order to make corrections.
5. **Deference to Expertise**—HROs “push decision making down.” Decisions are made “on the front line.” They avoid rigid hierarchies and go directly to the person with the expertise.

The US Nuclear Regulatory Agency (2011) focuses mainly on leadership and individual authority in its policy statement on safety culture.

Historical Catastrophes and Safety Culture

The cases described in the table below are some of the many in which official reports or authoritative experts cited culture as a factor in the catastrophic failure of the systems involved.

Example	Cultural Discussion
Apollo	According to Feynman (1988), Apollo was a successful program because of its culture of <i>"common interest."</i> The <i>"loss of common interest"</i> over the next 20 years then caused <i>"the deterioration in cooperation, which . . . produced a calamity."</i>
Challenger	Vaughn (1997) states that rather than taking risks seriously, NASA simply ignored them by calling them normal—what she terms <i>"normalization of deviance,"</i> whose result was that <i>"flying with acceptable risks was normative in NASA culture."</i>
Columbia	The Columbia Accident Investigation Report (NASA 2003, 102) echoed Feynman's view and declared that NASA had a <i>"broken safety culture."</i> The board concluded that NASA had become a culture in which bureaucratic procedures took precedence over technical excellence.
Texas City - 2005	On August 3, 2005, a process accident occurred at the BP refinery in a Texas City refinery in the USA resulting in 19 deaths and more than 170 injuries. The Independent Safety Review Panel (2007) found that a corporate safety culture existed that <i>"has not provided effective process safety leadership and has not adequately established process safety as a core value across all its five U.S. refineries."</i> The report recommended <i>"an independent auditing function."</i>
The Triangle Fire	On August 11, 1911, a fire at the Triangle shirtwaist factory in New York City killed 145 people, mostly women (NYFIC 1912). The New York Factory Investigating Commission castigated the property owners for their lack of understanding of the <i>"human factors"</i> in the case and called for the establishment of standards to address this deficiency.
Nimrod	On September 2, 2006, a Nimrod British military aircraft caught fire and crashed, killing its entire crew of 14. The Haddon-Cave report (Haddon-Cave 2009) found that Royal Air Force culture had come to value staying within budget over airworthiness. Referencing the conclusions of the Columbia Accident Investigation Report, the Haddon-Cave report recommends creation of a system of detailed audits.

Relationship to Ethics

A business's culture has the potential to reinforce or undermine ethical behavior. For example, a culture that encourages open and transparent decision making and behavior makes it harder for unethical behavior to go undetected. The many differences in culture around the world are reflected in different perspectives on what ethical behavior is. This is often reflected in difficulties that international companies face when doing business globally, sometimes leading to scandals because behavior that is considered ethical in one country may be considered unethical in another. See Ethical Behavior for more information about this.

Implications for Systems Engineering

As SE increasingly seeks to work across national, ethnic, and organizational boundaries, systems engineers need to be aware of cultural issues and how they affect expectations and behavior in collaborative working environments. SEs need to present information in an order and a manner suited to the culture and personal style of the audience. This entails choices like whether to start with principles or practical examples, levels of abstraction or use cases, the big picture or the detailed view.

Sensitivity to cultural issues is a success factor in SE endeavors (Siemieniuch and Sinclair 2006).

References

Works Cited

- Fasser, Y. and D. Brettner. 2002. *Management for Quality in High-Technology Enterprises*. New York, NY, USA: Wiley.
- Feynman, R. 1988. "An Outsider's Inside View of the Challenger Inquiry." *Physics Today*. 41(2) (February 1988): 26-27.
- Gordon, W.J.J. 1961. *Synectics: The Development of Creative Capacity*. New York, NY, USA: Harper and Row.
- Haddon-Cave, C. 2009. *An Independent Review into the Broader Issues Surrounding the Loss of the RAF Nimrod MR2 Aircraft XV230 in Afghanistan in 2006*. London, UK: The House of Commons.
- Helmreich, R.L., and A.C. Merritt. 2000. "Safety and Error Management: The Role of Crew Resource Management." In *Aviation Resource Management*, edited by B.J. Hayward and A.R. Lowe. Aldershot, UK: Ashgate. (UTHFRP Pub250). p. 107-119.
- Independent Safety Review Panel. 2007. *The Report of the BP U.S. Refineries Independent Safety Panel*. Edited by J.A. Baker. Texas City, TX, USA.
- Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Hoboken, NJ, USA: John Wiley & Sons.
- Kotter, J.P. 1995. "Leading Change: Why Transformation Efforts Fail." "Harvard Business Review." (March-April): 59-67.
- Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.
- NASA. 2003. *Columbia Accident Investigation Report*. Washington, D.C., USA: National Aeronautics and Space Administration (NASA). August 2003.
- Nuclear Regulatory Agency. 2011. "NRC Issues Final Safety Culture Policy Statement." 'NRC News' (14 June 2011). Available at: <https://pbadupws.nrc.gov/docs/ML1116/ML11166A058.pdf>.
- NYFIC. 1912. *Preliminary Report of the New York Factory Investigating Commission*. R. F. Wagner (ed). New York, NY, USA: New York Factory Investigating Commission (NYFIC).
- Reason, J. 1997. *Managing the Risks of Organisational Accidents*. Aldershot, UK: Ashgate Publishing Limited.
- Senge, P.M., A. Klieiner, C. Roberts, R.B. Ross, and B.J. Smith. 1994. *The Fifth Discipline Fieldbook: Strategies and Tools for Building a Learning Organization*. New York, NY, USA: Currency Doubleday.
- Shields, J.L. 2006. "Organization and Culture Change." In "Enterprise Transformation," W.B. Rouse (ed.). Hoboken, NJ, USA: John Wiley & Son.
- Siemieniuch, C.E. and M.A. Sinclair. 2006. "Impact of Cultural Attributes on Decision Structures and Interfaces." Paper presented at the 11th ICCRTS Coalition Command and Control in the Networked Era. Cambridge, MA, USA. p. 1-20.
- Sprenger, C. and S.T. Have. 1996. "4 Competencies of a Learning Organization." (Original title: "Kennismanagement Als Moter van Delerende Organisatie"). "Holland Management Review" Sept–Oct, p. 73–89.
- Vaughn, D. 1997. *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. Chicago, IL, USA: University of Chicago Press.
- Weick, K.E. and K.M. Sutcliffe. 2001. *Managing the Unexpected: Assuring High Performance in an Age of Complexity*. San Francisco, CA, USA: Jossey-Bass (Jossey-Bass acquired by Hoboken, NJ, USA: Wiley Periodicals, Inc.).

Primary References

Fasser, Y. and D. Brettner. 2002. *Management for Quality in High-Technology Enterprises*. New York, NY, USA: Wiley.

Helmreich, R.L., and A.C. Merritt. 2000. "Safety and Error Management: The Role of Crew Resource Management." In "Aviation Resource Management," edited by B.J. Hayward and A.R. Lowe. Aldershot, UK: Ashgate. (UTHFRP Pub250). p. 107-119.

Hofstede, G. 1984. *Culture's Consequences: International Differences in Work-Related Values*. London, UK: Sage Publications.

Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Hoboken, NJ, USA: John Wiley & Sons.

NASA. 2003. *Columbia Accident Investigation Report*. Washington, DC, USA: National Aeronautics and Space Administration (NASA). August 2003.

Reason, J. 1997. *Managing the Risks of Organisational Accidents*. Aldershot, UK: Ashgate Publishing Limited.

Senge, P.M., A. Klieiner, C. Roberts, R.B. Ross, and B.J. Smith. 1994. *The Fifth Discipline Fieldbook: Strategies and Tools for Building a Learning Organization*. New York, NY, USA: Currency Doubleday.

Additional References

Hofstede, G. 2001. *Culture's Consequences: Comparing Values, Behaviors, Institutions and Organizations Across Nations*, Second Edition. Thousand Oaks, CA, USA: Sage Publications.

Hofstede, G. 2010. *Cultures and Organizations: Software for the Mind*, Third Edition. New York, NY, USA: McGraw Hill.

Knowledge Area: Enabling Teams

Enabling Teams

Contents of this Knowledge Area

- Team Capability (Dick Fairley) (Alice Squires, Art Pyster, and Heidi Davidz)
- Team Dynamics (Dick Fairley) (Alice Squires and Art Pyster)
- Diversity, Equity, and Inclusion (Alan Harding and Alice Squires)
- Technical Leadership in Systems Engineering (Heidi Davidz)
- Lead Author:
- Dick Fairley
- Contributing Authors:
- Alice Squires and Art Pyster

This knowledge area focuses on enabling a team to perform SE. Once that is done using the techniques described here, the knowledge found in Part 3, Systems Engineering and Management, about how to perform SE can be applied. Part 5, Enabling Systems Engineering, to which this knowledge area belongs, explores how systems engineering (SE) is enabled at three levels of organization: the business or enterprise, the team, and the individual.

For the sake of brevity, the term “business” is used to mean “business or enterprise” throughout most of this knowledge area. For a nuanced explanation of what distinguishes a business from an enterprise, see Enabling Systems Engineering.

Overview

Products, enterprise systems, and services are developed, delivered, and sustained with the contributions of systems engineers, who also coordinate the technical aspects of the multiple projects that comprise a program. These activities require certain individuals to work in a cooperative manner to achieve shared objectives based on a common vision—that is, as teams. Not every group of individuals working together is a team. To perform SE activities efficiently and effectively, the capabilities of and dynamics within the team must be specifically attuned to SE.

Although individuals sometimes perform SE activities, it is more usual to find project teams performing SE activities while providing specialty engineering capabilities (see Systems Engineering and Quality Attributes). Not all who perform SE activities are labeled “systems engineers.” Thus, electrical, mechanical, and software engineers, service providers, or enterprise architects in IT organizations may lead or be members of teams that perform SE tasks. Those individuals are referred to as systems engineers in this knowledge area, regardless of their job titles within their organizations.

This knowledge area is concerned with methods, tools, and techniques for enabling project teams to perform SE activities. Its first topic, Team Capability, answers the questions:

- How do businesses determine value added by SE activities performed by project teams?
- How does an organization determine the efficiency and effectiveness of SE activities performed by project teams?

Its other topic, Team Dynamics, answers the question:

- How are group dynamics crucial to enabling systems engineers to perform work and achieve goals?
-

Topics from elsewhere in the SEBoK that cover related questions include Relationships between Systems Engineering and Project Management and The Influence of Project Structure and Governance on Systems Engineering and Project Management Relationships, which answer the question:

- What do managers need to know about managing systems engineers and project teams that perform SE activities?

References

Works Cited

None.

Primary References

Brooks, F. 1995. *The Mythical Man-Month*, Anniversary Edition. Reading, MA, USA: Addison Wesley.

Curtis, B., W.E. Hefley, and S.A. Miller. 2001. *People Capability Maturity Model (P-CMM)*, Version 2.0. Pittsburg, PA, USA: Software Engineering Institute (SEI). CMU/SEI-2001-MM-01. Accessed on June 8, 2012. Available at <https://www.sei.cmu.edu/library/abstracts/reports/01mm001.cfm>.

DeMarco, T. and T. Lister. 1999. *Peopleware: Productive Projects and Teams*, 2nd ed. New York, NY, USA: Dorset House.

Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. Hoboken, NJ, USA: John Wiley and Sons.

Fairley, R.E. 2009. *Managing and Leading Software Projects*. Hoboken, NJ, USA: John Wiley & Sons.

Forsyth, D.R. 2010. *Group Dynamics*, 5th edition. Belmont, CA, USA: Wadsworth, Cengage Learning.

Hase, S. 2000. "Measuring Organisational Capability: Beyond Competence", Paper presented at Future Research, Research Futures: Australian Vocational Education and Training Research Association (AVETRA) Conference (2000). Accessed on June 8, 2012. Available at https://www.avetra.org.au/abstracts_and_papers_2000/shase_full.pdf.

INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2010-003.

NASA. 2011. *Academy of Program/Project and Engineering Leadership (APPEL), NASA APPEL Performance Enhancement*. Accessed on September 15, 2011. Available at <https://www.nasa.gov/offices/ocel/appeel/performance/index.html>.

Additional References

None.

Team Capability

- Lead Author:
- Dick Fairley
- Contributing Authors:
- Alice Squires, Art Pyster, and Heidi Davidz

The capability of a team to perform systems engineering (SE) depends on having competent personnel, adequate time, sufficient resources and equipment, and appropriate policies and procedures (Torres and Fairbanks 1996).

The team should have a charter. Staff must be proficient in the needed competencies and must work together with the right attitude, under the right organization, and with appropriate tools, training, and processes such as configuration management and peer review.

Those responsible for the team attaining the desired capability need to organize, staff, develop, and assess the team. Techniques for pilot projects, post-mortem analysis, and lessons learned can be applied as well.

Organizing the Team

Project teams, and the roles of systems engineers within those teams, depend on factors such as the nature, size, and scope of the project, the organization's preferred way of organizing teams, and external constraints such as a larger program in which the project may be embedded. Options range from a dedicated team of systems engineers, to Integrated Product Teams, to teams that include other kinds of engineers that perform systems engineering.

Systems engineers and SE teams may play the roles of technical leads, consultants, or advisers; this influences the ways in which SE teams are organized. In some organizations, systems engineers and SE teams provide technical leadership; they perform requirements analysis and architectural design, conduct trade studies, and allocate requirements and interfaces to the various elements of a system. In addition, they work with component specialists, develop integration plans and perform system integration, verification, and validation. Depending on the scope of effort, they may also install the system and train the operators and users; provide ongoing services to sustain the system; and retire/replace an aged system. Systems engineers may be housed within a functional unit of an organization and assigned, in matrix fashion, to projects and programs, or they may be permanently attached to a project or program for the duration of that endeavor. They may be organized based partially on their domain of expertise, such as finance or telecommunications. For additional information on organizational options see *Determining Needed Systems Engineering Capabilities in Businesses and Enterprises*.

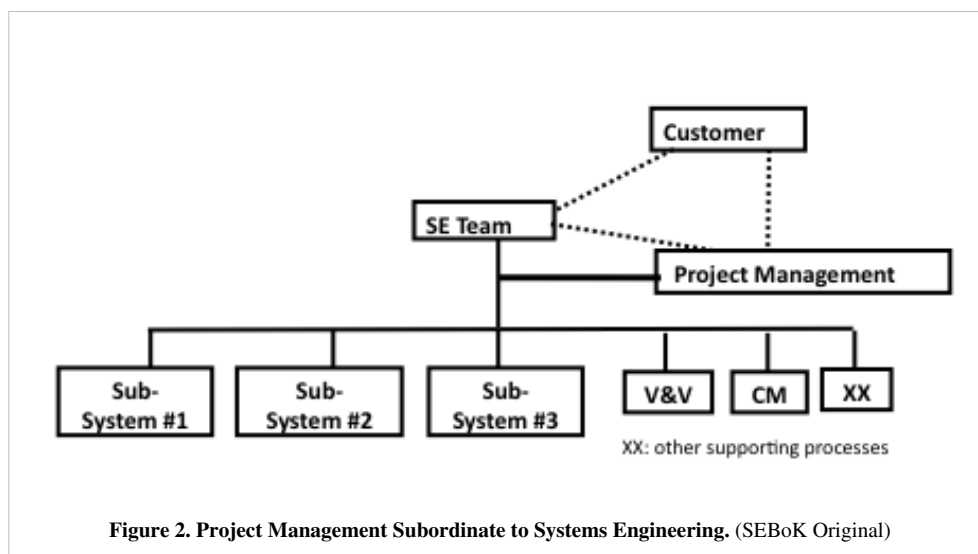
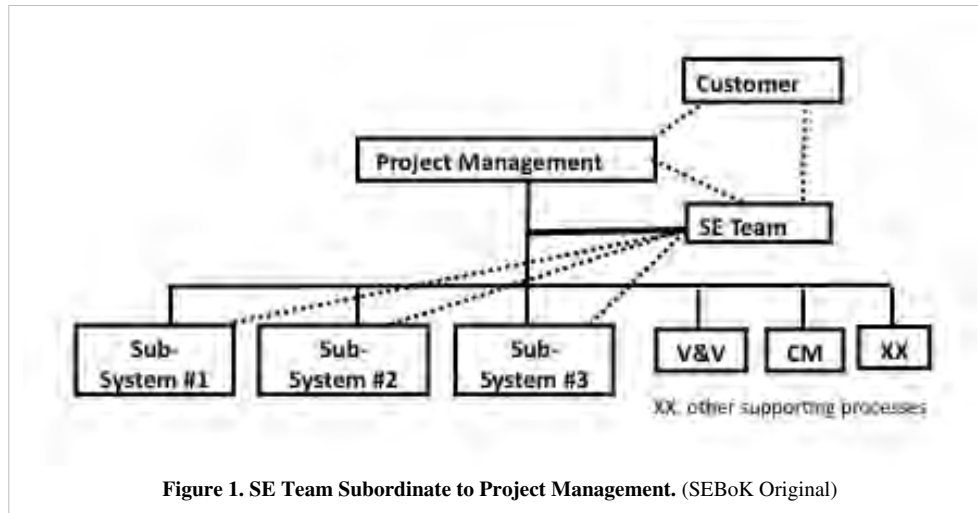
In other cases, one or more systems engineers may provide consulting or advisory services, as requested, to projects and programs. These engineers may be dispatched from a central pool within an organization, or they may be hired from an outside agency.

An SE team can be organized by job specialization, where each SE team member (or each SE sub-team) plays a different role; for example, requirements engineering, system architecture, integration, verification and validation, field test, and installation and training. In this case the various job specializations are typically coordinated by a lead systems engineer.

Alternatively, an SE team can be organized by subsystem where each SE team member (or SE sub-team) performs the previously indicated functions for each of the subsystems with a top-level team to coordinate requirements allocation, interfaces, system integration, and system verification and validation.

Ideally, roles, responsibilities, and authority will be established for each project or program and used to determine the optimal way to organize the team. Sometimes, however, an *a priori* organizational, project, or program structure may determine the structure, roles, responsibilities, and authority of the SE team within a project or program; this may or may not be optimal.

Within a project, a systems engineer or SE team may occupy a staff position subordinate to the project manager, as indicated in Figure 1 or conversely, the SE team may provide the authoritative interface to the customer with the project manager or management team, serving in a staff capacity, as indicated in Figure 2. In both cases, SE and project management must work synergistically to achieve a balance among product attributes, schedule, and budget. Eisner (2008) lays out various approaches to organizing systems engineers. For additional information see Systems Engineering and Project Management.



In scaling up to the program level, the considerations portrayed in Figures 1 and 2 can be generalized so that a top-level SE team provides coordination among the subordinate projects. In this case, each project has an SE team, and within each project the SE team members can be organized in either of the ways indicated in the figures. When scaling up to programs, each of the sub-systems in Figures 1 and 2 are separate, coordinated projects.

The models presented in Figures 1 and 2 can be scaled down to smaller projects, where an individual systems engineer performs the SE activities, either in the subordinate position of Figure 1 or the superior position of Figure 2. In this case, there is a single subsystem (i.e., the system) and the supporting functions may be provided by the systems engineer or by supporting elements of the larger organization.

The roles to be played by members of a SE team are influenced by the structures adopted as part of the organizational strategy of the business in which the team is operating (see Systems Engineering Organizational Strategy). In Product Centered Organizations, for example, an Integrated Product Team (IPT) is assigned to each element of the system breakdown structure (SBS). Each IPT consists of members of the technical disciplines

necessary to perform systems engineering functions for that element of the system.

At the program level there is a top-level IPT commonly called a SE and integration team (SEIT), whose purpose is to oversee all of the lower level IPTs. Some specialists, such as reliability and safety engineers, may be assigned to a team to cover all elements within a given level of the SBS. These teams are sometimes called Analysis and Integration teams (AITs), and are created at various levels of the SBS as needed.

Organizing communication and coordination among a group of systems engineers should follow the well-known 7 ± 2 rule because the number of communication paths among N engineers is $N(N-1)/2$; i.e., the number of links in a fully connected graph (Brooks 1995). There are 10 communication paths among 5 engineers, 21 among 7 engineers, and 36 among 9 engineers. An SE team of more than 10 members (45 paths) should be organized hierarchically with a top-level team leader. Sub-teams can be partitioned by product subsystem or by process work activities (analysis, design, integration).

Staffing the Team

Once the organizational structure of the SE team is understood, the team can be staffed. As noted in Enabling Individuals, competency of an individual is manifest in the knowledge, skills, abilities, and attitudes needed for the individual to perform a specific task efficiently and effectively. Different levels of competency may be needed in different situations. Competencies include occupational competence, social competence, and communication competence. Competent systems engineers, for example, have SE knowledge, skills, and ability; engage in systems thinking; possess emotional intelligence; and have good communication and negotiation skills. In addition, competent systems engineers are typically competent within specific domains (e.g. aerospace, medicine, information technology) and within specific process areas of systems engineering (e.g., requirements, design, verification and validation). (See Part 3, Systems Engineering and Management for more information on specific process areas.) The article on Roles and Competencies includes a summary of SE competency models. Based on the context, these competency models are tailored to match the needs of each project. The roles within the team are defined, and competencies are linked to the roles. The lists of competencies given in those models are most often distributed among the members of a SE team. It is not often that a single individual will possess the full list of competencies given in these models.

In addition to individual competencies to perform SE roles, the collective SE competencies needed by a team depend on additional factors including the domain, the stakeholders, the scope of the effort, criticality of outcome, new initiative versus enhancement, and the responsibilities and authority assigned to the team. For example, collective SE competencies needed to develop the IT enterprise architecture for a small company are quite different from those needed to develop the architecture of an aircraft which is engineered and manufactured in a distributed fashion around the world.

To determine the collective set of competencies an SE team needs to conduct a project or program, perform the following steps:

1. Identify the context, to include:
 1. domain
 2. stakeholders
 3. organizational culture
 4. scope of effort
 5. criticality of the product, enterprise endeavor, or service
 6. new initiative or sustainment project
 2. Clarify the responsibilities, authority, and communication channels of the systems engineering team
 3. Establish the roles to be played by systems engineers, and other project personnel as determined by context, responsibilities, and authority
 4. Determine the required competencies and competency levels needed to fill each of the systems engineering roles
-

5. Determine the number of systems engineers needed to provide the competencies and competency levels for each role
6. Determine the availability of needed systems engineers
7. Adjust based on unavailability of needed systems engineers
8. Organize the systems engineering team in a manner that facilitates communication and coordination within the SE team and throughout the project or program
9. Consult stakeholders to ask “What are we missing?”

Competency models and skills inventories, such as INCOSE (2010) and Curtis et al. (2001), can be used as checklists to assist in determining the needed competencies and competency levels for a product, enterprise, or service. (See Roles and Competencies.)

When the needed competencies, competency levels, and capacities have been determined, one of two situations will arise: In the optimal situation, the number of systems engineers who have the needed competencies and competency levels to fill the identified roles will be available; or, they will be unavailable or cannot be provided because of insufficient funding. For example, a new initiative may need a lead engineer, a requirements engineer, a systems architect and a systems integrator-tester to accomplish systems engineering tasks. Budgetary constraints may indicate that only two of the four roles can be supported. Compromises must be made; perhaps the system architect will be the lead engineer and the requirements engineer will also be assigned the tasks of system integration and testing despite lacking the desired level of skill and experience (i.e., competency level) in integration and testing.

Developing the Team

Before a team that performs SE can be effective, it needs to establish its own identity, norms, and culture. The well-known four stages of “*forming, storming, norming, performing*” (Tuckman 1965, 384-399) indicate that a SE team needs time to form, for the members to get to know and understand each other as well as the tasks to be performed, and to work out how best to work together. It is also important that care is taken to ensure, to the greatest extent possible, assignment of roles and responsibilities that would allow SE team members to satisfy their individual goals (Fraser 2010).

The *cost and time to cohesion* can be minimized by good selection and management of the SE team, consistent training across the business so that team members have a common framework of understanding and language for their work, good “infostructure” to allow easy and useful sharing of information, and shared behavioral norms and values. Conversely, in cross-site, inter-company and international SE teams, more time must be allowed for team formation. SE teams are more effective if attention is given to ensuring that each member's work satisfies their individual goals as well as the team and organizational objectives (Fraser 2010).

According to Stephenson and Weil (1992), capable people are:

those who know how to learn; are creative; have a high degree of self-efficacy, can apply competencies in novel as well as familiar situations; and work well with others. In comparison to competency, which involves the acquisition of knowledge and skills, capability is a holistic attribute.

The results of a survey by Steward Hase (2000) concluded that the following are significant contributors to the human elements of capability:

- Competent People
- Working in Teams
- Visible Vision and Values
- Ensuring Learning Takes Place
- Managing the Complexity of Change
- Demonstrating the Human Aspects of Leadership
- Performing as Change Agents

- Involving People in Change
- Developing Management Talent
- Committing to Organizational Development

These attributes of human capability apply to all members of an organization, including systems engineers, both as individuals and as members of project teams.

DeMarco and Lister (1999) discuss “teamicide” techniques by which management, perhaps unintentionally, practices *sure fire techniques to kill teams*. Teamicide techniques include

- physical separation of team members
- fragmentation of time
- unrealistic schedules
- excessive overtime

Methods for developing and improving SE capabilities within teams include building cohesive teams, conducting pilot projects, participating in and studying post-mortem analyses, and preparing and examining lessons learned. Members of a cohesive systems engineering team have a strong sense of commitment to the work and to the other team members. Commitment creates synergy, which results in performance greater than the sum of the performance of the individual team members.

Some key indicators of a cohesive systems engineering team (Fairley 2009, 411) are:

- clear understanding of systems engineering roles and responsibilities
- shared ownership of systems engineering work products
- willingness of systems engineers to help one another and to help other project members
- good communication channels among systems engineers and with other project elements
- enjoyment of working together

Negations of these indicators—the hallmarks of a dysfunctional team—are:

- confusion of systems engineering roles and responsibilities
- protective ownership of systems engineering work products
- unwillingness to help one another
- absence of good communications among systems engineers and with other project elements
- personal dislike of one or more other systems engineering team members

Techniques for building and maintaining cohesive systems engineering teams include:

- an appropriate number of systems engineering team members
- a correct mix of systems engineering competencies
- celebration of project milestones
- team participation in off-site events
- social events that include family members

Assessing the Team

Performance evaluation is most often conducted for individuals. Robbins (1998, 576) states the historic belief that individuals are the core building blocks around which organizations are built. However, it is also important to assess the team's capability and performance. To design a system that supports and improves the performance of teams, including SE teams, Robbins offers four suggestions:

1. Tie the SE team's performance and the overall project team's results to the organization's goals
 2. Begin with the team's customer and the work process the team follows to satisfy customer's needs
 3. Measure both team and individual performance and compare them to organizational norms and benchmarks
 4. Train the team to create its own measures
-

Robbins' approach can be applied in the context of SE:

1. Tie the SE and overall project team's results to the project's and the organization's goals. Use measures that apply to goals the team must achieve. For SE in particular, the team effort should be tied to the product or service which the organization seeks to deliver. The end product for the SE team should not be only the SE work products but the delivered products and services provided by the project. For more information on general SE assessment, see Systems Engineering Project Assessment and Control.
2. Consider the SE team's customers and more broadly the key stakeholders and the work processes that the SE team follows to satisfy customer needs. SE customers and stakeholders can be internal or external; the internal customers of systems engineering are the other project elements that depend on systems engineering work products and services, which can be evaluated for on-time delivery of quantity and quality. The process steps can be evaluated for waste and cycle time; i.e., efficiency and effectiveness.
3. Assess both individual and team performance. Define the roles of each SE team member in terms of the tasks that must be accomplished to produce the team's work products. For more information on individual assessment, see Assessing Individuals.
4. Finally, have the team define its own measures of achievement of goals. This helps all members of the team to understand their roles, while also building team cohesion.

As an example, NASA's Academy of Program/Project and Engineering Leadership (APPEL) provides a service where team performance is assessed and interventions are provided to the team for specific gaps in performance (NASA 2011). This performance enhancement service increases a project's probability of success by delivering the right support to a project team at the right time. APPEL offers the following assessments:

- Project/Team Effectiveness — Measures effectiveness of a team's behavioral norms
- Individual Effectiveness — Measures effectiveness of an individual's behavioral norms
- Project/Team Process Utilization — Measures the extent of a team's utilization of key processes
- Project/Team Knowledge — Covers topics that NASA project personnel should know in order to perform in their jobs

The APPEL approach can be applied to assessing the performance of a SE team and individual systems engineers.

Further Techniques for Building Team Capability

Further techniques for developing SE capabilities within teams include conducting pilot projects, preparing post-mortem analyses, and participating in and studying lessons learned.

Pilot Projects

Pilot projects are an effective mechanism by which SE teams can build team cohesion, acquire new skills, and practice applying newly acquired skills to projects and programs. Pilot projects can be conducted for the sole purpose of skills acquisition, or they can be conducted to determine the feasibility of a proposed approach to solving a problem. Feasibility studies and acquisition of new team skills can be combined in proof-of-concept studies. Primary inhibitors to conducting SE pilot projects are the time required and diversion of personnel resources.

Post-Mortem Analysis

A post-mortem analysis identifies areas for improvement of SE performance in future projects and programs. Inputs to a post-mortem analysis include:

- personal reflections and recollections of project personnel and other stakeholders;
- email messages, memos, and other forms of communication collected during a project or program;
- successful and unsuccessful risk mitigation actions taken; and
- trends and issues in change requests and defect reports processed by the change control board.

Team participation in a post-mortem analysis allows SE team members to reflect on past efforts, which can lead to improved team capabilities for future projects or, if the present team is being disbanded, improved individual ability to participate in future systems engineering teams.

Inhibitors for effective post-mortem analysis include failure to allocate time to conduct the analysis, failure to effectively capture lessons-learned, failure to adequately document results, reluctance of personnel to be candid about the performance of other personnel, and negative social and political aspects of a project or program. Mechanisms to conduct effective post-mortem analyses of SE projects include using a third-party facilitator, brainstorming, Strength-Weakness-Opportunity-Threat (SWOT) analysis, fishbone (Ishikawa) diagrams, and mind mapping.

Lessons Learned

Lessons learned in SE can be both positive and negative. Experiences gained and documented from past projects and programs can be an effective mechanism for developing and improving the capabilities of a team that performs SE tasks. Studying past lessons learned can aid in team formation during the initiation phase of a new project. Lessons learned during the present project or program can result in improved capabilities for the remainder of the present project and for future projects. Inputs for developing and documenting SE lessons learned include results of past post-mortem analyses plus personal recollections of the team members, informal *war stories*, and analysis of email messages, status reports, and risk management outcomes. Inhibitors for developing and using SE lessons learned include failure to study lessons learned from past projects and programs during the initiation phase of a project, failure to allocate time and resources to developing and documenting lessons learned from the present project or program, and reluctance to discuss problems and issues.

References

Works Cited

- Brooks, F. 1995. *The Mythical Man-Month*. Anniversary Edition. Reading, MA, USA: Addison Wesley.
- Curtis, B., W.E. Hefley, and S.A. Miller. 2001. *People Capability Maturity Model (P-CMM)*, version 2.0. Pittsburgh, PA, USA: Software Engineering Institute (SEI). CMU/SEI-2001-MM-01. Accessed April 24, 2013. Available: <https://www.sei.cmu.edu/library/abstracts/reports/01mm001.cfm>.
- DeMarco, T., and T. Lister. 1999. *Peopleware: Productive Projects and Teams*, 2nd ed. New York, NY, USA: Dorset House.
- Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Fairley, R.E. 2009. *Managing and Leading Software Projects*. Hoboken, NJ, USA: John Wiley & Sons.
- Fraser, D. 2010. *Relationships Made Easy: How to Get on with The People You Need to Get on with...and Stay Friends with Everyone Else*. Worchestershire, UK: HotHive Books.
-

- Hase, S. 2000. "Measuring Organisational Capability: Beyond Competence." Paper presented at Future Research, Research Futures: Australian Vocational Education and Training Research Association (AVETRA) Conference (2000). Accessed September 14, 2011. Available: https://www.avetra.org.au/abstracts_and_papers_2000/shase_full.pdf.
- INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2010-003.
- NASA. 2011. *Academy of Program/Project and Engineering Leadership (APPEL), NASA APPEL Performance Enhancement*. Accessed September 15, 2011. Available: <https://appel.nasa.gov/team/request-support/>.
- Robbins, S.P. 1998. *Organizational Behavior: Concepts, Controversies, Applications*, 8th ed. Upper Saddle River, NJ, USA: Prentice Hall. p. 576.
- Stephenson, J. and S. Weil. 1992. *Quality in Learning: A Capability Approach in Higher Education*. London, UK: Kogan Page.
- Torres, C., and D. Fairbanks. 1996. *Teambuilding: The ASTD Trainer's Sourcebook*. New York, NY, USA: McGraw-Hill.
- Tuckman, B. 1965. "Developmental Sequence in Small Groups." *Psychological Bulletin*. 63 (6): 384-99.

Primary References

- Brooks, F. 1995. *The Mythical Man-Month*. Anniversary Edition. Reading, MA, USA: Addison Wesley.
- Curtis, B., W.E. Hefley, and S.A. Miller. 2001. *People Capability Maturity Model (P-CMM)*, Version 2.0. Pittsburg, PA, USA: Software Engineering Institute (SEI). CMU/SEI-2001-MM-01. Accessed on June 8, 2012. Available at <https://www.sei.cmu.edu/library/abstracts/reports/01mm001.cfm>.
- DeMarco, T. and T. Lister. 1999. *Peopleware: Productive Projects and Teams*. 2nd ed. New York, NY, USA: Dorset House.
- Eisner, H. 2008. *Essentials of Project and Systems Engineering Management*. 3rd ed. Hoboken, NJ, USA: John Wiley & Sons.
- Fairley, R.E. 2009. *Managing and Leading Software Projects*. Hoboken, NJ, USA: John Wiley & Sons.
- Hase, S. 2000. "Measuring Organisational Capability: Beyond Competence". Paper presented at Future Research, Research Futures: Australian Vocational Education and Training Research Association (AVETRA) Conference (2000). Accessed on June 8, 2012. Available at https://www.avetra.org.au/abstracts_and_papers_2000/shase_full.pdf.
- INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2010-003.
- NASA. 2011. *Academy of Program/Project and Engineering Leadership (APPEL), NASA APPEL Performance Enhancement*. Accessed on May 2, 2014. Available at <https://appel.nasa.gov/team/request-support/>.
- Torres, C. and D. Fairbanks. 1996. *Teambuilding: The ASTD Trainer's Sourcebook*. New York, NY, USA: McGraw-Hill.

Additional References

Fasser, T. and D. Brettner. 2002. *Management for Quality in High Technology Enterprises*. New York, NY, USA: Wiley.

INEEL 2004. *A Project Management and Systems Engineering Structure for a Generation IV Very High Temperature Reactor*. Idaho Falls, ID, USA: Idaho National Engineering and Environmental Laboratory, NEEL/CON-04-02175. Accessed on September 14, 2011. Available at <https://www.inl.gov/technicalpublications/Documents/2808490.pdf>.

Team Dynamics

- Lead Author:
- Dick Fairley
- Contributing Authors:
- Alice Squires and Art Pyster

A systems engineering (SE) team is a group of individuals who cooperatively perform a collection of SE tasks based on a shared vision and a common set of engineering objectives. Applying the practical considerations of group dynamics is essential to enabling SE teams to successfully perform SE activities. The interplay of the behaviors of humans in groups is varied, changing, and inescapable. Nevertheless, study of these behaviors has yielded valuable insight and knowledge on the dynamics of individuals within groups. The awareness and application of group dynamics is crucial to facilitating systems engineers' performance of work and achievement of their goals.

The study of group dynamics was initially within the province of psychology and later within sociology. The importance of group dynamics to successful teams has led other disciplines such as business management to study and apply team dynamics.

History

The origins of the study of group dynamics began with Gustave Le Bon. Le Bon wrote *La psychologie des fouls* in 1895, which was translated into English as *The Crowd: A Study of the Popular Mind* a year later. Sigmund Freud wrote *Group Psychology and the Analysis of the Ego* in 1922 responding to Le Bon's work. Kurt Lewin is acknowledged as the "founder of social psychology", coining the term **group dynamics**. He founded the Research Center for Group Dynamics at the Massachusetts Institute of Technology in 1945, relocating in 1948 to the University of Michigan. Wilfred Bion studied group dynamics from a psychoanalytical perspective. He helped found the Tavistock Institute of Human Relations in 1947. In that same year, both the Research Center for Group Dynamics and the Tavistock Institute of Human Relations founded the journal *Human Relations*. The study of group dynamics is now worldwide, active, and well established.

Nature of Groups

Groups are endemic to human existence and experience; humans are by nature social animals. Consequentially, an informed understanding of the nature of groups is very useful in enabling teams to perform SE. Research into group behavior reveals that the nature of a group can be described by interaction, goals, interdependence, structure, unity, and stage. (Forsyth 2010, 5-10)

Interaction

Communication (both verbal and non-verbal) among members within a group produces constantly changing and varied interactions. Group dynamics are more than the sum of the interactions between individual members; group interactions create synergistic behaviors and results. Interactions can be placed into two categories (1) socio-emotional interactions and (2) task interactions (Bales 1950, 1999).

Goals

All groups exist for the purpose of achieving one or more goals. The goals provide the basis for the group's tasks. The tasks accomplished by the group can be categorized into activities and characterized by a Circumplex Model (McGrath 1984, 61), which establishes four quadrants, where the X-axis is *choose* vs. *execute* and the Y-axis is *generate* vs. *negotiate*.

Interdependence

Interdependence is *the state of being dependent to some degree on other people, as when one's outcomes, actions, thoughts, feelings, and experiences are determined in whole or in part by others*. Interdependence can be categorized into five types (1) mutual, reciprocal; (2) unilateral; (3) reciprocal, unequal; (4) serial; and (5) multi-level. (Forsyth 2010, 8)

Structure

Structure includes the organization and patterned behaviors of a group. Structure can be deliberately devised and/or emergently observed. Most groups have both kinds of structures, which are evinced in the roles and norms of the group. *The roles of leader and follower are fundamental ones in many groups, but other roles — information seeker, information giver, elaborator, procedural technician, encourager, compromiser, harmonizer — may emerge in any group* (Benne and Sheats 1948; Forsyth 2010, 9). Norms are the rules that govern the actions of group members; norms can include both formal and informal rules.

Cohesion

The *interpersonal forces that bind the members together in a single unit with boundaries that mark who is in the group and who is outside of it* constitute a group's cohesion (Dion 2000). Cohesion is an essential quality of group; it can vary from weak to strong. A team cannot perform effectively without strong group cohesion.

Stage

Groups exhibit stages of development. Being comprised of people, it is not surprising that groups collectively demonstrate the dynamics and growth of the individuals that constitute the group members. The most well-known and wide-spread model of the stages of group development was developed by Bruce Tuckman. The initial model identified the sequence of group development as (1) Forming, (2) Storming, (3) Norming, and (4) Performing (Tuckman 1965). He later added a final stage to the model: (5) Adjourning (Tuckman and Jensen 1977). While Tuckman's model is sequential, others have observed that groups may actually recursively and iteratively progress through the different stages (Forsyth 2010, 20).

Practical Considerations

The dynamics associated with creating, nurturing, and leading a team that will successfully achieve the team's goals is important and challenging. Although psychologists and sociologists have conducted and continue to conduct research to understand team dynamics, the profession of business management has additionally sought to develop practical guidance for utilizing and applying this knowledge to foster high-performance teams. Accordingly, business management has focused its contribution to the field of team dynamics by publishing practical guidebooks to analyze the problems and focus on developing solutions to the problems of team dynamics (see Additional References). There are many consultancy firms throughout the world that assist organizations with the application of practical knowledge on team dynamics. Successful systems engineering teams would do well to not ignore, but rather take advantage of this knowledge.

References

Works Cited

- Bales, R.F. 1950. *Interaction Process Analysis: A Method for The Study of Small Groups*. Reading, MA, USA: Addison-Wesley.
- Bales, R.F. 1999. *Social Interaction Systems: Theory and Measurement*. New Brunswick, NJ, USA: Transaction.
- Benne, K.D. and P. Sheats. 1948. "Functional Roles of Group Members." *Journal of Social Issues*. 4 (2): 41-49. Blackwell Publishing Ltd.
- Dion, K.L. 2000. "Group Cohesion: From 'Field of Forces' to Multidimensional Construct." *Group Dynamics: Theory, Research, and Practice*. 4 (1): 7-26. Washington DC, USA: American Psychological Association.
- Forsyth, D.R. 2010. *Group Dynamics*, 5th edition. Belmont, CA, USA: Wadsworth, Cengage Learning.
- McGrath, J.E. 1984. *Groups: Interaction and Performance*. Upper Saddle River, NJ, USA: Prentice Hall.
- Tuckman, B.W. 1965. "Developmental Sequence in Small Groups." *Psychological Bulletin*. 63 (6):384-399. Washington DC, USA: American Psychological Association.
- Tuckman, B.W. and M.C. Jensen. 1977. "Stages of Small Group Development Revisited." *Group and Organization Management* 2 (4): 419-427. Thousand Oaks, CA, USA: Sage Publications.

Primary References

- Forsyth, D.R. 2010. *Group Dynamics*, 5th edition. Belmont, CA, USA: Wadsworth, Cengage Learning.

Additional References

- Scholtes, P.R., B.L. Joiner, and B.J. Streibel. 2003. *The Team Handbook*, 3rd edition. Edison, NJ, USA: Oriel Inc.
- Larson, C.E. and F.M.J. LaFaso. 1989. *Teamwork: What Must Go Right, What Can Go Wrong*. Newbury Park, CA, USA: Sage Publications, Inc.
- Lencioni, P. 2002. *The Five Dysfunctions of a Team: A Leadership Fable*. San Francisco, CA, USA: Jossey-Bass.
- Lencioni, P. 2005. *Overcoming the Five Dysfunctions of a Team*. San Francisco, CA, USA: Jossey-Bass.
- McShane, S.L. and M.A. Von Glinow. 2010. *Organizational Behavior: Emerging Knowledge and Practice for the Real World*. New York, NY, USA: McGraw-Hill/Irwin.
-

Diversity, Equity, and Inclusion

- Lead Authors:
- Alan Harding and Alice Squires

Diversity, Equity, and Inclusion (DEI) foster increased engagement, productivity, and innovation in an organization.

DEI in Systems Engineering

Systems engineers play a pivotal role in integrating concepts of diversity, equity, and inclusion within the teams they work on and in system design and development. In particular, systems engineers should:

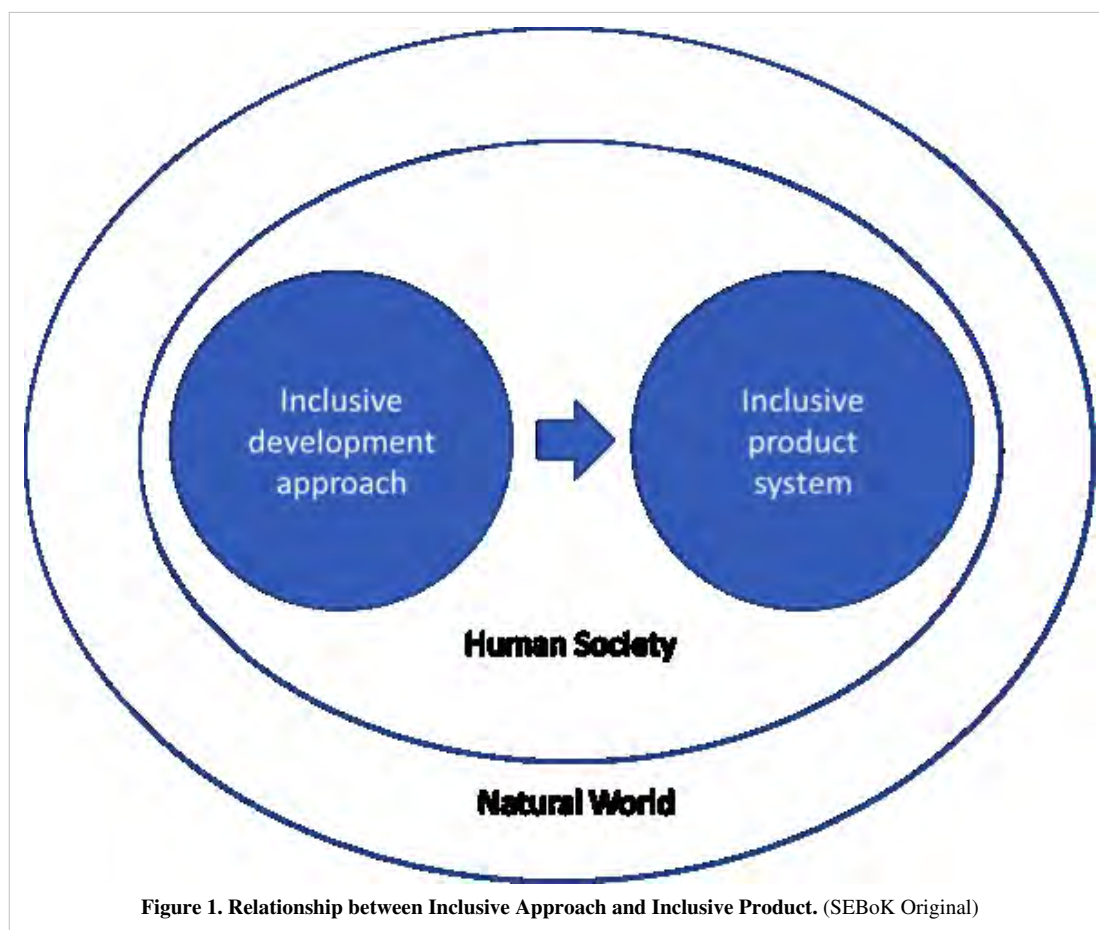
1. Ensure that the systems engineering team and its leadership is inclusive and welcomes a diverse range of talent, and where necessary taking deliberate action to provide equity.
2. Ensures that the systems we realise are as accommodating as possible of the differences within the entire stakeholder community. This is known as “inclusive engineering”.

Failure to address either aspect can result in sub-optimal outcomes whether in terms of missed solutions, lower productivity, or delivering a system that does not fully meet the needs of the whole stakeholder community, i.e. failing to meet the ultimate goal of delivering a total optimal system solution.

Systems engineers are responsible for effectively communicating the importance and value of diversity, equity, and inclusion in enabling, promoting, and advancing systems engineering and systems approaches to address complex societal and technical global challenges.

Figure 1 shows how an inclusive development approach contributes to realizing inclusive solutions, within the context of the human system (whether within an organization, country, or the world), itself set within the context of the natural world. The natural world is shown because of the strong linkage between the full lifecycle of engineered products (from concept to disposal) and sustainable development. For instance:

- Water pollution from industrial plants affecting those who live nearby
 - Air pollution from cars affecting pedestrians and those who live near major roads
 - Product end of life/disposal effects e.g. hazardous substances, contribution to land-fill
-



Definitions of Diversity, Equity, and Inclusion

The following definitions are taken from the Accreditation Board for Engineering and Technology (ABET 2020), where they provide a reference point for conversations and materials about diversity, equity and inclusion.

- Diversity is the range of human differences, encompassing the characteristics that make one individual or group different from another. Diversity includes, but is not limited to, the following characteristics: race, ethnicity, culture, gender identity and expression, age, national origin, religious beliefs, work sector, physical ability, sexual orientation, socioeconomic status, education, marital status, language, physical appearance, and cognitive differences.
- Equity is the fair treatment, access, opportunity and advancement for all people, achieved by intentional focus on their disparate needs, conditions and abilities. Achieving equity requires understanding of historical and systemic patterns of disparity to address and eliminate barriers, and remove participation gaps as part of a comprehensive strategy to achieve equitable outcomes and social justice.
- Inclusion is the intentional, proactive, and continuing efforts and practices in which all members respect, support, and value others. An inclusive environment provides equitable access to opportunities and resources, empowers everyone to participate equally, and offers respect in words and actions for all.

Commonly, the compound term "Diversity, Equity and Inclusion" (abbreviated to DEI) is used to refer to the broad subject area. The definition of diversity given encompasses a wide range of characteristics. As an example, Figure 2 shows 28 of these characteristics recognised by the International Council on Systems Engineering (INCOSE) (Harding and Pickard 2019) grouped into five areas: intrinsic, employment, environment, interaction, and family. The figure shows the relevance of these characteristics to the INCOSE Systems Engineering Certification Program.



Relevance of Diversity, Equity, and Inclusion to Engineering

Engineers apply ingenuity, innovation, and systematic approaches to solve challenging problems. Life experience and academic research shows us that bringing a wide range of skills, knowledge, and thinking styles to bear on a problem is the most effective way to accelerate and improve the intended outcomes. These outcomes include improved "...financial performance, greater innovation and creativity, increased employee productivity and retention, improved customer or client orientation, and increased customer or client satisfaction." (Royal Academy of Engineering 2015). Hunt et al. (2018) have found that companies at the forefront of gender and ethnic/cultural diversity in their leadership perform better financially.

By contrast, a team of people with the same cultural background, life experiences, education, and thinking style could be expected to be relatively less effective and more prone to identifying predictable solutions. The US National Academy of Engineering (2002) notes the opportunity cost of a lack of diversity in terms of "designs not thought of, in solutions not produced."

Inclusion, or ensuring a sense of inclusion in everyone, is necessary to ensure that all team members genuinely feel and believe that they belong and hence are able to use their talents and unique outlook to the maximum degree. By contrast, a lack of inclusion might make someone feel present but not involved or valued with the effect that the team as a whole does not deliver its best possible results.

Equity is not the same as equality, nor is it the same as inequality. It is simply giving more to those who need it, which is proportionate to their own circumstances, in order to ensure that everyone has the same opportunities. In an engineering context this might mean providing more support to a disadvantaged student so they can reach their full potential, or providing additional support or time to a team member with a condition such as dyslexia.

Relevance of Diversity, Equity, and Inclusion to Systems Engineering

DEI is vital to successful systems engineering because of the range of contexts in which it is applied and the consideration of multiple stakeholder viewpoints at the heart of the approach. Systems engineering is applied to a wide range of system types in a broad variety of contexts—engineered systems range from microelectronics to aircraft, from abstract systems to smart cities. Systems engineers may be working with a customer, a prime contractor or integrator, a supplier or product manufacturer, a research/technology organization, or a government body. And these activities take place all over the globe, often as part of consortiums or complex partnered programmes involving multiple organizations, countries, and cultures.

Applying systems engineering requires consideration of multiple viewpoints (such as the user, maintenance, safety, security) to achieve the proper holistic view of problem and solution. This means that the systems engineering team must be able to understand and work with a wide range of stakeholders. The transdisciplinary and integrative nature of systems engineering across other disciplines and activities, again, means that the systems engineering team needs to understand and work well with all the disciplines and specialities involved in realising a system (INCOSE 2020).

Given this diversity of context and of types of systems engineered, and the wide range of stakeholders with whom they need to work, systems engineering workforce and culture should be at the forefront of DEI. In this way, we can represent as many aspects of the diverse community and their needs as possible within the team, and the diverse nature of the team also creates the innovation from which we can realise the best solutions.

Like most of engineering, systems engineering was historically not practiced by a diverse group of people. Therefore, it is necessary to apply the notions of equity (as defined) in order to ensure that the widest range of people are enabled and empowered to become and develop as systems engineers.

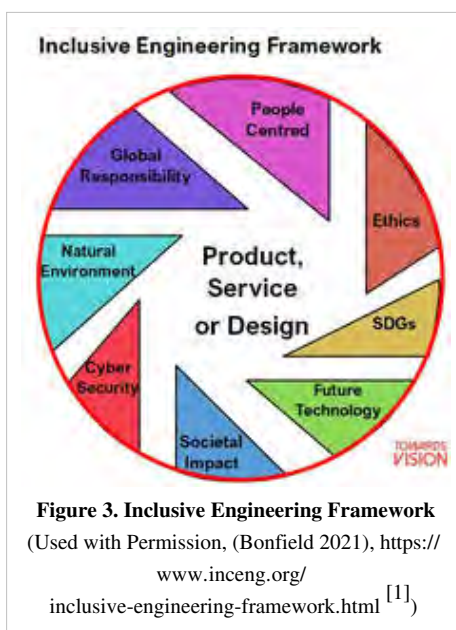
Inclusion (as defined) is about ensuring that the whole (diverse) team is engaged, supported and feels safe and able to give of their best to the team's activity. An inclusive team will produce increased productivity and better-quality outcomes than the alternative. It also provides increased potential for inclusive products because of the greater range of stakeholder views within the team.

Inclusive Engineering

Inclusive Engineering (Inclusive Engineering, n.d.) is the discipline of ensuring that engineering products and services are accessible to and inclusive of all users and are as free as possible from discrimination and bias. This should consider as far as possible all human differences (characteristics of diversity). It is a way of ensuring that engineering is appropriate, ethical, accessible, and as risk free as possible. The extent to which an engineered system is inclusive reduces the degree to which adaptation has to be applied to address the needs of people with differences e.g., differing vision, differing strength or motor functions.

In their enthusiasm for solutions engineers often do not stop to think about whether they have considered all of the things that impact on their design – and in particular all of the non-technical requirements that are not specified by the client or potential beneficiary. As a result, proposed solutions often lack the perspectives of people who have not been involved in their development – and in an industry which is notoriously lacking in diversity – this often means that they fail to include the perspectives of women, people with disabilities, the ageing population, and those with other under-represented characteristics.

Figure 3 shows an inclusive engineering framework (ref. 8) which has eight elements, all of which are important factors for systems engineers to address even if the stated requirements do not cover them.



One way that systems engineers can maximise the potential for inclusive and sustainable solutions is to ensure that DEI considerations are through application of the ISO/IEC/IEEE 15288:2015 Technical Processes. Table 1 illustrates the application of Inclusive Engineering these processes.

Table 1. Mapping of the application of ISO/IEC/IEEE 15288:2015 Technical Processes to Inclusive Engineering considerations.

ISO/IEC/IEEE 15288:2015 Technical Process	Inclusive Engineering considerations
Business or Mission Analysis	Ensure all stakeholders are identified, both the obvious (beneficial) stakeholders, and those potentially affected by the system and the associated project (referred to as “unwilling” stakeholders).
Stakeholder Needs and Requirements Definition	- Identify stated stakeholder needs and the unstated but necessary needs arising from inclusive design – e.g. access, language, disability. - Use understanding of Sustainable Development Goals, circular economies etc. to inform future-looking discussions about needs.
System Requirements Definition	- Ensure that inclusion considerations result in well-specified requirements drawing on standards and legislation as necessary. - Ensure that definition of the System Boundary is informed by its intended and unintended emergent effects on its wider context. - Facilitate careful trade-off and option analysis to optimise equitable outcomes.
Architecture Definition	- Ensure that all necessary Architecture viewpoints are considered to ensure that inclusion and sustainability factors can be given due consideration. - Ensure that Architectures are open to allow future technology adoption where beneficial during the system lifecycle.
Design Definition	Ensure that technology choices and principles for design evolution are informed by Sustainable Development Goals, circular economies etc.
System Analysis	Ensure that System Analysis includes the modelling and prediction of inclusive engineering and sustainable development factors, in particular modelling to understand key emergent outcomes.
Implementation	Ensure that the design team is diverse, properly trained, and understands the principles of diversity, equity, inclusion, sustainability, circular economy, etc.
Integration	Ensure that the integration process is safe, secure, and environmentally compliant.

Verification	• Ensure that planned verification activities fully address all aspects of diversity, equity, and inclusion for all relevant stakeholders and are safe, secure, and environmentally compliant.
Transition	• Ensure that transition plans fully cover all stakeholders and are safe, secure, and environmentally compliant, and that any temporary/transitional arrangements do not have accidental negative effects on unwilling stakeholders or the natural environment i.e., there is an equitable transition plan.
Validation	• Ensure that that planned validation activities fully address all aspects of diversity, equity, and inclusion for all relevant stakeholders including indirect/unwilling stakeholders and the natural environment and are safe, secure, and environmentally compliant.
Operations	• Ensure that selection and training for operations staff is sufficient that a diverse group of operators can correctly operate all aspects of the system such that the system remains safe, secure, and environmentally compliant.
Maintenance	• Ensure that selection and training for maintenance staff is sufficient that a diverse group of maintainers can correctly operate all aspects of the system such that the system remains safe, secure, and environmentally compliant. • Ensure that planned maintenance ensures that the system fully meets its initial specification throughout its service life e.g., energy use, emissions, pollution.
Disposal	• Ensure that selection and training for disposal staff is sufficient that a diverse group of staff can correctly dispose of hazardous materials such that the system remains safe, secure, and environmentally compliant. • Ensure that disposal of hazardous materials is carefully planned, and that recycling and reuse opportunities are maximised.

References

Works Cited

- ABET. 2020. "Diversity, equity, and inclusion." *Accreditation Board for Engineering and Technology (ABET)*. Available: <https://www.abet.org/about-abet/diversity-equity-and-inclusion/>. Accessed December 31, 2020.
- Bonfield, D. 2020. "Inclusive engineering framework." *Inclusive Engineering*. Available: <https://www.inceng.org/inclusive-engineering-framework.html> ^[1]. Accessed December 31, 2020.
- Harding, A. and A. Pickard. "Towards a more diverse INCOSE." *INCOSE INSIGHT Practitioner Magazine*. 22(3). Oct 2019.
- Hunt, V., S. Prince, S. Dixon-Fyle, and L. Yee. 2018. "Delivering Through Diversity." Report, McKinsey & Company. https://www.mckinsey.com/media/McKinsey/Business%20Functions/ Organization/ Our%20Insights/Delivering%20through%20diversity/Delivering-through-diversity_full-report.ashx
- INCOSE. 2020. "Definition of systems engineering." <https://www.incose.org/about-systems-engineering/about-systems-engineering> Accessed 31 Dec 2020.
- Inclusive Engineering. n.d. "Inclusive engineering." Available: <https://www.inceng.org/>. Accessed December 31, 2020.
- National Academy of Engineering. 2002. "Diversity in Engineering: Managing the Workforce of the Future." Washington, DC: The National Academies Press. <https://doi.org/10.17226/10377>
- Royal Academy of Engineering. 2015. "Increasing Diversity and Inclusion in Engineering—A Case Study Toolkit." Case study, Royal Academy of Engineering. <https://www.raeng.org.uk/policy/diversity-in-engineering/diversity-inclusion-toolkit-re-sources/documents/increasing-diversity-and-inclusion-in-engineering>.

Primary References

None.

Additional References

None.

References

[1] <https://www.inceng.org/inclusive-engineering-framework.html>

Technical Leadership in Systems Engineering

- Lead Author:
- Heidi Davidz

Leadership is an important but often overlooked component of technical projects and programs. It addresses the performance of people: their behaviors, their ability to think individually and collectively, and their motivation and energy. Technical leadership in systems engineering creates the environmental conditions conducive to good performance: support of shared understanding, innovation, problem solving, resilience and learning. Leadership is thus complementary to management, which directs specific activities to deliver outputs. A systems engineering leader may lead a team of systems engineers for a project or program, or may be the only systems engineer in a team of diverse members involved in project or program (e.g. other engineers, IT personnel, service providers). There are various models and styles of leadership and key to success is matching leadership to the needs of a situation. “Models” of leadership describe the mechanisms by which leadership arises and operates (e.g. situationally-driven or caused by a charismatic individual). “Styles” of leadership describe the manner in which a leader (or a leadership team) leads (e.g. task-focused or people-focused; autocratic, democratic or “laissez-faire” (Lewin et al., 1939)).

There is a vast amount of literature addressing leadership issues from multiple points of view, including philosophical, psychological and emotional considerations (Yukl, 2012). This article highlights key aspects of leadership theory to help systems engineers understand how they may influence the success of their team and organization. Leadership theory provides the basic building blocks for adapting leadership behaviors at work. The pragmatic aspects of leading team members involved in systems engineering are summarized in section 1.11. This section highlights the need to use different approaches to leadership across the systems engineering context, and it is therefore important be able to understand and adopt the leadership behaviors discussed in the preceding sections, as judged appropriate. Related knowledge areas and articles are in the Part 5 Knowledge Area Part 5 Enabling Systems Engineering and the Part 6 Knowledge Area Systems Engineering and Project Management.

Attributes of Effective Leaders

Traditional Attitudes to Technical Leadership

The need for leadership in an engineering environment has not been widely emphasized or understood. Traditional academic engineering curricula do not cover the development of leadership skills, and industry professionals tend to be task-oriented, with project leaders perceived in terms of power and authority (Toor and Ofori, 2008). In many cases, technical organizations focus on management rather than leadership. “Managers are people who do things right while leaders are people who do the right thing” (Bennis and Nanus, 1985). Doing the right thing is not only about identifying the right approach in the first place; it is also about taking responsibility for understanding and challenging the progression of a project or program in a continuous manner. It is now recognized that leadership is a critical component of successful projects and programs, and that technical leadership is likely to be a distributed

responsibility.

Great Man Theories: Traits and Charisma Models of Leadership

Early concepts of leadership were driven by views of leaders as heroic figures, with particular qualities that made them different to other people. The notion of “charisma” was used to describe the ability to charm and influence followers. Numerous studies have been conducted to try and define the particular personality traits that made someone a born leader. The findings are not clear-cut, partly because there are many different models of personality which produce different results (Hippocrates first identified 4 personality dimensions in the 5th Century BC, and many different conceptualizations have been devised since then). Personality tests should be used with great caution because each test has been developed for specific purposes and contexts, and is only valid within those parameters. For tests to be valid they must undergo a strict set of tests with extensive data sets, and then they must be used exactly as specified by the validation process. The current best consensus of evidence is that there are 5 main dimensions of personality: “Extraversion” (talkative, sociable); “Agreeableness” (good natured, co-operative); “Conscientiousness” (responsible, tidy); “Neuroticism” (general level of anxiety or composure); and “Openness” (to new experiences). This 5-Factor model has good validity across literate populations, but even this model may not be universal (Gurven et al., 2013). There is some evidence that extraversion is associated with leadership roles, but this is not always a predictor of success, and may reflect a cultural stereotype which leads to people who behave like leaders being more likely to get leadership roles. Different contexts will change the value of extraversion (and other traits) in a leader. (See Judge et al. (2002) for a meta-analysis of the literature).

In business settings, the Myers-Briggs Type Indicator (MBTI) personality test is often used as part of guided discussions to assist with self-development (although it lacks the important personality dimension of ‘neuroticism’). People can use their MBTI profiles to help them use their strengths more effectively. Occasionally, MBTI is misused as a basis for selection, especially for leadership roles. The evidence indicates this is not justified (National Research Council, 1991).

Transactional and Transformational Leadership Styles

Certain behaviors have been associated with successful leadership. These behaviors arise from the style of leadership and particularly the attention paid to the task compared to team relationships. Such differences are described as transactional and transformational styles (Burns, 1978). Transactional leadership is closely allied to management, focused on defined task outputs and incentivizing people to follow directions by rewarding and punishing.

Transformational leadership is concerned with achieving outcomes through the development of the people (team building), building trust, developing a shared vision, motivation, cultivating relationships and sharing knowledge. Both types of leadership have value, but transformational leadership is needed for developing the culture of an organization, and for ensuring qualities such as safety, adaptability, learning and improvement. It is usually considered the most valuable form of leadership.

Understanding that different styles have value for different situations provides the basis for leadership models that recognize the interactions between style and situation. Fiedler’s Contingency Model, Hersey et al.’s Situational Model and House’s Path-Goal Theory (all described below) provide useful variations on this approach.

Contingency Model of leadership

Fiedler’s Contingency Model (1964) states that there is no one best style of leadership. Effectiveness is about the match between leadership style (defined as task or relationship-oriented) and situation (defined by: the degree to which the leader is supported by the group; the degree to which the task is clearly structured; and the degree to which the leader can reward and punish team members). Fiedler devised a way of assessing leaders’ styles by measuring their attitude to their ‘least preferred co-worker’ or LPC. In general terms, leaders who are more negative about their LPC are task-oriented and focus on organizing. Leaders who are more positive towards their LPCs are more able to

avoid conflict, promote innovation and learning and are better at making complex decisions. In moderate situations (not extreme in any of the three situation dimensions), the more positive, relationship-oriented leaders appear to be more successful (Valle & Avella, 2003). This contingency model of leadership was found to predict leadership style in an information systems engineering environment, where leadership functions were distributed across technical experts and the end-user (Franz, 1985).

Situational Theory

Situational Theory offers a model of leadership in which any individual leader adapts his or her style according to the needs of the situation. For example, they can learn to change from being task-focused to being relationship focused. They may also adapt according to their own changing status. Hersey, Blanchard and Johnson (2001) describe four modes that leaders can adapt between, according to the nature of the members of the team or organization: delegating, supporting, coaching, and directing. In this situational model, leadership is a learned skill based on understanding context and self-awareness.

Path-Goal Model

The Path-Goal Theory describes the leader's role as helping followers to develop behaviors that allow them to achieve their goals (House and Mitchell, 1974). Leaders are facilitators for others' achievements, e.g. providing resources, associations, knowledge and support. Leaders are members of a community of practice united in a common enterprise and sharing a common culture: history, values, ways of doing things, and ways of talking (Drath and Palus, 1994). In technical leadership, this means helping technical followers to perform effectively in their tasks, and in systems engineering this means facilitating pathways of communication between different areas, encouraging attitudes and behaviors that promote integrated perspectives.

Authentic Leadership

Somewhat in contrast to the principle of leading by adapting style, and thus in effect "acting the part", research on leaders being "authentic" evaluates the effectiveness of staying true to one's own natural style. Successful authentic leaders are described as positive, leading from the heart, concerned with ethics, building on trust, motivating people to achieve challenging tasks. According to the authentic leadership literature (e.g., Gardner et al., 2011; Walumbwa et al., 2008), authentic leaders display four types of behaviors. These include balanced processing (taking evidence from all sides), internalized moral perspective (driven more by morality than external pressures), relational transparency (openly sharing thoughts and feelings), and self-awareness (understanding of self and how others view them) (Gardner et al., 2011). These behaviors are likely to lead to a team having trust in the leader, which will be important in a technical context where safely achieving the right outcome in a complex situation is paramount.

Allied to authentic leadership in terms of behaviors is the concept of Servant Leadership, described as having seven key practices: self-awareness; listening; inverting the pyramid (leadership hierarchy); developing your colleagues; coaching, not controlling; unleashing the energy and intelligence of others; and foresight. Keith (2012), and Sipe and Frick (2009) have a similar list: servant leaders are individuals of character, put people first, are skilled communicators, are compassionate collaborators, use foresight, are systems thinkers, and exercise moral authority.

The servant leadership elements of Empowerment, Standing Back / Sharing Credit, Courage / Risk Taking, Humility, Authenticity, and Stewardship were shown to have a statistically significant correlation with innovation output from engineering teams when applied at a frontline team leadership level (McCleave and Capella, 2015).

Complexity Leadership and the Leadership Process

Authentic and servant leadership styles place a leader in the role of a facilitator, rather than a director; someone who can leverage the capabilities of the team and create synergistic benefits. This perspective is taken a step further in the model of leadership that comes from complexity theory.

Complexity Leadership describes leadership as promoting emergent adaptive outcomes from organizations (such as learning and innovation). Organizations are considered to be complex adaptive systems and leadership can take three forms: administrative, adaptive and enabling. Each form will vary itself according to its locus in an organizational hierarchy. The complex adaptive functions provide the adaptive capability while the bureaucratic functions provide the coordinating structures. Leadership should disentangle these two types of functions in a way that enhances the effectiveness of the organization (Marion & Uhl-Bien, 2001). In this model, leadership is mostly about developing interactions.

Complexity leadership is differentiated from leaders as individuals, because in some cases leadership is about a function rather than a person. In a technical situation such as a Systems Engineering team, this will be an important consideration, as different people will have technical expertise and will be required to provide leadership in areas such as understanding, challenging and communicating. Systems engineering teams consist of members from diverse disciplines with diverse interests. Silos of self-interest must be broken down (or at least effective communication among silos must be established and a balance between global system concerns and provincial disciplinary interests must be maintained.)

Manz and Sims (1989) also see leadership as a process, but they focus on self-leadership within each individual more than the behaviors and actions of a few select people designated as formal leaders in an organization. With this perspective, most people have some contribution to leadership.

Followership

Equally important is the concept of followership. A leader can only lead with effective followers. In technical situations, where a distributed process of leadership may be needed, this is especially important. The study of followership is much less developed than that of leadership, although they are two sides of the same coin. Uhl-Bien et al. (2014) have conducted a review of the literature to date and identify two theoretical frameworks for understanding followership: a role-based approach and a process approach. They warn against too much focus on a leader role and not enough on the leadership process, and suggest that understanding followership can help with:

- Recognizing the importance of follower roles, following behaviors, and the leadership process
- Understanding leadership processes and its outcomes as a function of leaders and followers
- Identifying effective followership behaviors
- Embedding context in the leadership process
- Recognizing that leadership can flow in all directions
- Understanding why and how managers are not always able to co-construct leadership with their subordinates
- Developing followership

This perspective is supportive of a distributed leadership function and is helpful for supporting people who have leadership roles as a consequence of their technical knowledge rather than their desire to lead or comfort with doing so.

Associated with followership development is the nature of motivation within the individuals that the leader wishes to influence. The term “motivation” has been used to describe a range of possible causes of behavior, and no single theory can explain all situations. A useful distinction, however, is the difference between “intrinsic” and “extrinsic” motivation. The former relates to factors arising from emotions, ambitions, expectations and other internal states of an individual, and tends to be the focus of transformational leaders (see section 1.3). The latter relates to factors arising from external factors such as threats, rewards, and social pressure, and tends to be the focus of transactional

leaders (also in section 1.3). It is important to recognize that there are cultural and professional differences in the strength of internal and external causes of motivation. One famous model of motivation by Maslow (1943), the “Hierarchy of Needs”, is useful to assess a range of potential factors, but does not have scientific validity and is based on a rather narrow Western 20th Century perspective. For example, it does not explain why people are willing to undergo physical hardship to conquer higher level challenges; or why some cultures are collectivist while others are individualistic. (A useful review of these culture differences can be found in Triandis et al., 1988).

A more actionable approach to motivation emphasizes an individual's mental model of what is important (valence), what their own role is in achieving it (instrumentality), and how able they are to achieve it (expectancy). This was first described by Vroom (1964) and has led to the concept of ‘empowering’ individuals (e.g. Conger and Kanungo, 1988). The Path-Goal model of leadership (section 1.6) aims to facilitate performance by addressing these aspects of motivation. This approach to motivation, called Expectancy Theory, can help leaders understand how to motivate employees through challenge and self-belief (Isaac, Zerbe, and Pitt, (2001).

An attempt to understand motivation at the organizational level has led to the concept of “organizational energy” (Cole, Bruch and Vogel, 2005). According to the existing overall energy type in an organization, a leader should adopt a different motivational strategy to achieve the optimum “productive” energy, which is described as high intensity and positive. A resignative energy (low intensity, negative) requires the development of a vision, empowerment and challenge. A corrosive energy (high intensity, negative) requires better communication and the development of trust. A comfortable energy (low intensity, positive) requires the identification of an external threat.

Competencies

Leadership competencies are the knowledge and skills required by individuals and teams for making leadership effective. Sometimes traits and other individual differences are added to skills and knowledge to create a “Competency Framework” for the leadership characteristics needed for a role. Communication, managing staff by supporting and providing feedback, and emotional competence are often featured in these frameworks. It is important to distinguish between those characteristics that are learned and those that are based on traits. Learned competencies can be enhanced through personal development; innate individual differences could be acquired for a role through personnel selection (although selection based on personality is not recommended: see section 1.2). As indicated above, leadership depends on many behaviors, including matching style to situations, effective followership, and individual leadership.

A number of roles will be required in a team, and ideally these may be distributed to individuals with the apposite competencies. Emotional competence has been the focus of much recent research and some studies show a strong correlation with effective leadership (e.g. Cavallo and Brienza, 2006, who used the Emotional Competence Inventory©).

Daniel Goleman has extended and publicized the concept of emotional intelligence (an innate characteristic) and the competencies (skills that can be learned) that put it into practice. He describes how emotional aptitudes can preserve relationships, protect our health and improve our success at work (Goleman, 1998).

Goleman differentiates 5 main categories of competence. The first three are about self-management and the last two are about being effective in relationships.

1. Self-awareness: accurate self-assessment, emotional awareness and self-confidence
2. Self-regulation: innovation, adaptability, conscientiousness, trustworthiness and self-control
3. Motivation: optimism, commitment, initiative and achievement, drive
4. Empathy: developing others, service orientation, political awareness, diversity, active listening and understanding others
5. Social skills: communication, influence, conflict management, leadership, bond building, collaboration, cooperation and team capabilities

Emotional Intelligence is most associated with transformational and situational leadership.

Communication skills are also highlighted in most leader competency frameworks. These skills are about communicating to other people and listening and being communicated to by other people. Some skills are about engagement, others about sharing understanding. In particular, avoiding hidden assumptions and understanding others' perspectives are important. Communication can take place in many ways, especially with the help of IT and social media. Each mode of communication has advantages and disadvantages. Consideration should be given to how important it is to have face-to-face communication (usually better, but especially for complex matters and when emotions are involved). Although this takes more time and effort, it will often save time and effort in the long term by reducing misunderstandings and negative emotions. Nikoi (2014) presents a collection of studies that investigated the way in which communication works across media and teams.

Communication may be synchronous or asynchronous, broadcast or individual, dialogue or one-way. Bowman (2004) has a useful summary of the advantages and disadvantages of different communication channels.

Some competencies that are often associated in the literature with good leadership are listed in Table 1. The relevance of these will depend on the style and the situation/context.

Some commonly cited attributes of effective leaders are listed in Table 1 below.

Table 1. Attributes of Effective Leaders (Fairley 2009).

Reprinted with permission of the IEEE Computer Society. All other rights are reserved by the copyright owner.

Listening carefully	Maintaining enthusiasm
Delegating authority	Saying "thank you"
Facilitating teamwork	Praising team for achievements
Coordinating work activities	Accepting responsibility for shortcomings
Facilitating communication	Coaching and training
Making timely decisions	Indoctrinating newly assigned personnel
Involving appropriate stakeholders	Reconciling differences and resolving conflicts
Speaking with individual team members on a frequent basis	Helping team members develop career paths and achieve professional goals
Working effectively with the project/program manager and external stakeholders	Reassigning, transferring, and terminating personnel as necessary

Characteristics that result in effective leadership of systems engineering activities include behavioral attributes, leadership style, and communication style. In addition, a team leader for a systems engineering project or program has management responsibilities that include, but are not limited to: developing and maintaining the systems engineering plan, and establishing and overseeing the relationships between the project/program manager and project/program management personnel.

Implications for technical leadership in systems engineering

Leadership can have a significant impact on engineering performance (Kolb, 1995) and resilience (Flin, 2006). The models and styles of leadership described above emphasize the power of social skills: the ability to relate to and connect with other people. This appears to be particularly true for the sorts of situations that system engineering leaders are likely to find themselves in: working on complex problems with other professionals who are willing to follow but need to be confident in the leader's technical skill and trustworthiness. The technical leader should possess not only essential technical knowledge but should also have positive values, high levels of ethics, morality, leadership from the heart, personal capabilities, out-of-the-box thinking, interpersonal skills, etc. (Lloyd-Walker and

Walker, 2011).

In a systems engineering context it is useful to recognize that different leadership functions may be distributed across a team. Some leadership functions will be knowledge focused, but it may be necessary to have a 'facilitator' (complexity) leader to ensure that the team follows the most appropriate leadership at any time. Each organization will have particular leadership requirements, which should be articulated in a behavioral framework in order to identify the most effective leadership styles and competencies, and where and how they should be applied.

Leadership capability for systems engineers should therefore be seen as a distributed capability to be developed across engineers. NASA takes a systems approach to developing leadership in their Systems Engineering Leadership Development Program (SELDP). They define technical leadership as the 'art' of systems engineering. Technical leadership includes broad technical domain knowledge, engineering instinct, problem solving, creativity, and the leadership and communication skills needed to develop new missions and systems. It focuses on systems design and technical integrity throughout the life cycle. A system's complexity and the severity of its constraints drive the need for systems engineering leadership (Williams and Reyes, 2012).

Selecting leaders by promoting the best technical performers or the most ambitious candidates is not an effective way of ensuring good leadership in an organization or program. For this reason, companies such as General Electric, Motorola, Toyota, Unilever, Raytheon, and Northrop Grumman use internal leadership academies to develop their leadership capability according to their needs (Daniels, 2009). A role model approach may be effective only if the appropriate role model is paired with a candidate, with good leadership characteristics that are valid for the situation (Yukl, 2012).

More effective approaches would involve developing competencies that can be learned through example, experience and reflection. The most effective methods will depend on the competencies needed, the type of organization, and the opportunities. They could include coaching, mentoring, shadowing, 'assistant-to' trial periods, and career management to provide experience (e.g. Fast-track).

There must also be an element of self-development: systems engineers should recognize the impact that people (or 'soft') issues have on the performance of a technical team and organization and learn how to adjust their own behavior and facilitate the behavior of others.

Behavioral Attributes

Behavioral attributes are habitual patterns of behavior, thought, and emotion that remain stable over time (Yukl 2013). Positive behavioral attributes enable a systems engineering leader to communicate effectively and to make sound decisions, while also taking into consideration the concerns of all stakeholders. Desirable behavioral attributes for a systems engineering leader include characteristics such as (Fairley 2009):

- **Aptitude** - This is exhibited by the ability to effectively lead a team. Leadership aptitude is not the same as knowledge or skill but rather is indicative of the ability (either intuitive or learned) to influence others. Leadership aptitude is sometimes referred to as charisma or as an engaging style.
 - **Initiative** - This is exhibited by enthusiastically starting and following through on every leadership activity.
 - **Enthusiasm** - This is exhibited by expressing and communicating a positive, yet realistic attitude concerning the project, product, and stakeholders.
 - **Communication Skills** - These are exhibited by expressing concepts, thoughts, and ideas in a clear and concise manner, in oral and written forms, while interacting with colleagues, team members, managers, project stakeholders, and others.
 - **Team Participation** - This is exhibited by working enthusiastically with team members and others when collaborating on shared work activities.
 - **Negotiation** - This is the ability to reconcile differing points of view and achieve consensus decisions that are satisfactory to the involved stakeholders.
-

- Goal Orientation – This involves setting challenging but not impossible goals for oneself, team members, and teams.
- Trustworthiness - This is demonstrated over time by exhibiting ethical behavior, honesty, integrity, and dependability in taking actions and making decisions that affect others.

Weakness, on the other hand, is one example of a behavioral attribute that may limit the effectiveness of a systems engineering team leader.

Personality Traits

The concept of “personality traits” was initially introduced in the early 1900's by Carl Jung, who published a theory of personality based on three continuums: introversion-extroversion, sensing-intuiting, and thinking-feeling. According to Jung, each individual has a dominant style which includes an element from each of the three continuums. Jung also emphasized that individuals vary their personality traits in the context of different situations; however, an individual's dominant style is the preferred one, as it is the least stressful for the individual to express and it is also the style that an individual will resort to when under stress (Jung 1971). The Myers-Briggs Type Indicator (MBTI), developed by Katherine Briggs and her daughter Isabel Myers, includes Jung's three continuums, plus a fourth continuum of judging-perceiving. These four dimensions characterize 16 personality styles for individuals designated by letters, such as ISTP (Introverted, Sensing, Thinking, and Perceiving). An individual's personality type indicator is determined through the answers the person has provided on a questionnaire (Myers 1995) combined with the individual's self-assessment which is done one to one with a qualified practitioner or in a group setting. MBTI profiles are widely used by coaches and counselors to help individuals assess how their personality type will affect how they might react in a particular profession and make suggestions about which professions might suit their individual preferences. It should never be used to decide which profession would be “most comfortable and effective” as the MBTI measures preference not ability. The MBTI has also been applied to group dynamics and leadership styles. Most studies indicate that groups perform better when a mixture of personality styles work together to provide different perspectives. Some researchers claim that there is evidence that suggests that leadership styles are most closely related to an individual's position on the judging-perceiving scale of the MBTI profile (Hammer 2001). Those on the judging side of the scale are more likely to be “by the book” managers, while those on the perceiving side of the scale are most likely to be “people-oriented” leaders. “Judging” in the MBTI model does not mean judgmental; rather, a judging preference indicates a quantitative orientation and a perceiving preference indicates a qualitative orientation. The MBTI has its detractors (Nowack 1996); however, MBTI personality styles can provide insight into effective and ineffective modes of interaction and communication among team members and team leaders. For example, an individual with a strongly Introverted, Thinking, Sensing, and Judging personality index (ITSJ) may have difficulty interacting with an individual who has a strongly Extroverted, Intuiting, Feeling, Perceiving personality index (ENFP).

Leadership Styles and Communication Styles

There is a vast amount of literature pertaining to leadership styles and there are many models of leadership. Most of these leadership models are based on some variant of Jung's psychological types. One of the models, the Wilson Social Styles, integrates leadership styles and communication styles (Wilson 2004). The Wilson model characterizes four kinds of leadership styles:

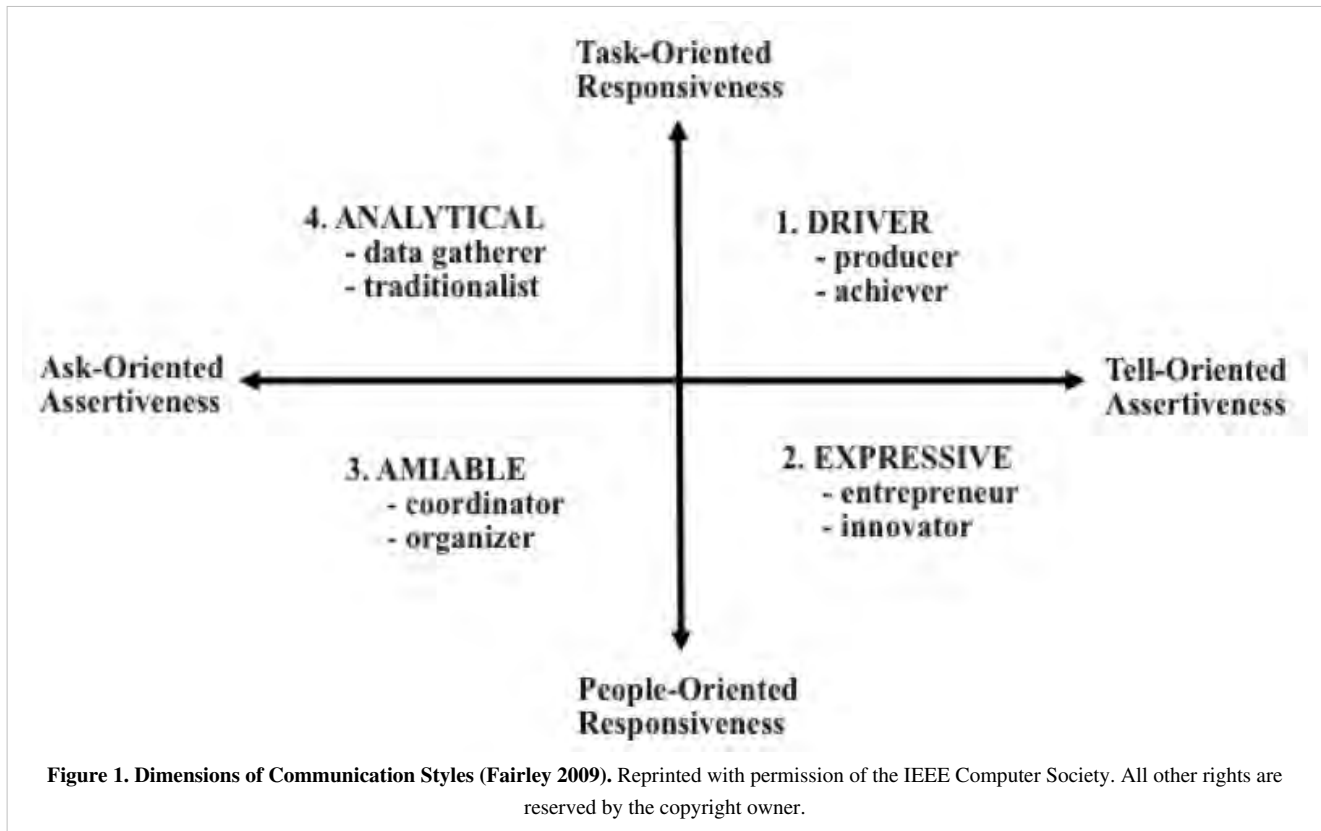
- Driver leadership style - This is exhibited when a leader focuses on the work to be accomplished and on specifying how others must do their jobs.
- Analytical-style leadership - This emphasizes collecting, analyzing, and sharing data and information. An analytical leader asks others for their opinions and recommendations to gather information.
- Amiable leadership style – This is characterized by emphasis on personal interactions and on asking others

for their opinions and recommendations.

- Expressive leadership style – Like the amiable style, this also focuses on personal relationships, but an expressive leader tells others rather than asking for opinions and recommendations. When taken to extremes, each of these styles can result in weakness of leadership. By focusing too intently on the work, "drivers" can provide too much or too little guidance and direction. Too little guidance occurs when the individual is preoccupied with her or his personal work, while too much guidance results in micromanagement, which limits the personal discretion for team members. Drivers may also be insensitive to interpersonal relationships with team members and others. Analytical leaders may provide too much information or may fail to provide information that is obvious to them, but not their team members. They do not like to discuss things they already know or that are irrelevant to the task at hand. Like driver-style leaders, they may be insensitive to interpersonal relationships with other individuals. Amiable leaders focus on interpersonal relationships in order to get the job done. They may exhibit a dislike of those who fail to interact with them on a personal level and may show little concern for those who show little personal interest in them. Expressive leaders also focus on interpersonal relationships. In the extreme, an expressive leader may be more interested in stating their opinions than in listening to others. Additionally, they may play favorites and ignore those who are not favorites. While these characterizations are gross oversimplifications, they serve to illustrate leadership styles that may be exhibited by systems engineering team leaders. Effective team leaders are able to vary their leadership style to accommodate the particular context and the needs of their constituencies without going to extremes; but as emphasized by Jung, each individual has a preferred comfort zone that is least stressful and to which an individual will resort during times of added pressure.

Communication Styles

An additional characterization of the Wilson model is the preferred style of communication for different leadership styles, which is illustrated by the dimensions of assertiveness and responsiveness.



Task-oriented assertiveness is exhibited in a communication style that emphasizes the work to be done rather than the people who will do the work, while the people-oriented communication style addresses personnel issues first and

tasks secondly. A tell-oriented communication style involves telling rather than asking, while an ask-oriented assertiveness emphasizes asking over telling. Movies, plays, and novels often include caricatures of extremes in the assertiveness and responsiveness dimensions of Wilson communication styles. An individual's communication style may fall anywhere within the continuums of assertiveness and responsiveness, from extremes to more moderate styles and may vary considering the situation. Examples include:

- Driver communication style exhibits task-oriented responsiveness and tell-oriented assertiveness.
- Expressive communication style shares tell-oriented assertiveness with the driver style but favors people-oriented responsiveness.
- Amiable communication style involves asking rather than telling (as does the analytical style) and emphasizes people relationships over task orientation (as does the expressive style).
- Analytical communication style exhibits task-oriented responsiveness and ask-oriented assertiveness.

The most comfortable communication occurs when individuals share the same communication styles or share adjacent quadrants in Figure 1. Difficult communication may occur when individuals are in diagonal quadrants; for example, communication between an extreme amiable style and an extreme driver style. Technical leaders and others can improve communications by being aware of different communication styles (both their own and others) and by modifying their communication style to accommodate the communication styles of others.

Management Responsibilities

Leading a systems engineering team involves communicating, coordinating, providing guidance, and maintaining progress and morale. Managing a project, according to the PMBOK® Guide (PMBOK 2013), involves application of the five process groups of project management: initiating, planning, executing, monitoring and controlling, and closing. Colloquially, systems engineering project/program management is concerned with making and updating plans and estimates, providing resources, collecting and analyzing product and process data, working with the technical leader to control work processes and work products, as well as managing the overall schedule and budget. Good engineering managers are not necessarily good technical leaders and good technical leaders are not necessarily good engineering managers; the expression of different personality traits and skill sets is required. Those who are effective as both managers and leaders have both analytical and interpersonal skills, although their comfort zone may be in one of managing or leading. Two management issues that are typically the responsibility of a systems engineering team leader are:

- Establishing and maintaining the division of responsibility among him or herself, the systems engineering team leader, and the project/program manager.
- Developing, implementing, and maintaining the systems engineering plan (SEP).

Relationships between systems engineering and project management are addressed in the Part 6 Knowledge Area (KA) of the SEBoK, Systems Engineering and Project Management. Also see the Part 5 Knowledge Area Enabling Teams for a discussion of the relationships between a project/program manager and a systems engineering technical leader.

The System Engineering Plan (SEP) is, or should be, the highest-level plan for managing the Systems Engineering effort and the technical aspects of a project or program. It defines how a project will be organized and conducted in terms of both performing and controlling the Systems Engineering activities needed to address a project's system requirements and technical content. It can have a number of secondary technical plans that provide details on specific technical areas and supporting processes, procedures, tools. Also, see the Project Planning article in Part 3, which includes a section on Systems Engineering Planning Process Overview.

In United States DoD acquisition programs, the System Engineering Plan (SEP) is a Government produced document which assists in the development, communication, and management of the overall systems engineering (SE) approach that guides all technical activities of the program. It provides direction to developers for program execution. The developer uses the SEP as guidance for producing the System Engineering Management Plan

(SEMP), which is a separate document and usually a contract deliverable that aligns with the SEP. As the SEP is a Government produced and maintained document and the SEMP is a developer/contractor developed and maintained document, the SEMP is typically a standalone, coordinated document.

The following SEP outline from (ODASD 2011) serves as an example.

1. **Introduction – Purpose and Update Plan**
2. **Program Technical Requirements**
 1. Architectures and Interface Control
 2. Technical Certifications
3. **Engineering Resources and Management**
 1. Technical Schedule and Schedule Risk Assessment
 2. Engineering Resources and Cost/Schedule Reporting
 3. Engineering and Integration Risk Management
 4. Technical Organization
 5. Relationships with External Technical Organizations
 6. Technical Performance Measures and Metrics
4. **Technical Activities and Products**
 1. Results of Previous Phase SE Activities
 2. Planned SE Activities for the Next Phase
 3. Requirements Development and Change Process
 4. Technical Reviews
 5. Configuration and Change Management Process
 6. Design Considerations
 7. Engineering Tools

5. **Annex A – Acronyms**

SEP templates are often tailored to meet the needs of individual projects or programs by adding needed elements and modifying or deleting other elements. A systems engineering team leader typically works with other team members, the project/program manager (or management team), and other stakeholders to develop the SEP and maintain currency of the plan as a project evolves. Some organizations provide one or more SEP templates and offer guidance for developing and maintaining an SEP. Some organizations have a functional group that can provide assistance in developing the SEP.

References

Works Cited

- Bennis, W.G. and Nanus, B. 1985. "Leaders: Strategies for Taking Charge." New York, NY, USA: Harper & Row.
- Bowman 2004. "Business Communication: Managing Information and Relationships." Available at: <https://homepages.wmich.edu/~bowman/channels.html>.
- Burns, J.M. 1978. "Leadership". New York, NY, USA: Harper & Row.
- Cavallo, K. and Brienza, D. 2006. "Emotional competence and leadership excellence at Johnson & Johnson," "The Emotional Intelligence and Leadership Study, Europe's Journal of Psychology", Vol.2, No 1.
- Cole, M. S., H. Bruch, & B. Vogel. 2005. "Development and validation of a measure of organizational energy." Academy of Management Best Paper Proceedings. In Weaver, K. M. (Ed.), Proceedings of the Sixty-fourth Annual Meeting of the Academy of Management (CD), ISSN 1543-8643.
- Conger, J.A. and R.N. Kanungo. 1988. "The empowerment process: Integrating theory and practice," "The Academy of Management Review," Vol. 13, No. 3, pp. 471-482.

- Daniels, C. B. 2009. "Improving leadership in a technical environment: A case example of the ConITS leadership institute," *"Engineering Management Journal,"* 21, pp. 47-52.
- Drath, W. H., and C.J. Palus. 1994. "Making Common Sense: Leadership as Meaning-Making in a Community of Practice." Greensboro, NC: Center for Creative Leadership.
- Fairley, R.E. 2009. "Managing and Leading Software Projects." Hoboken, NJ, USA: John Wiley & Sons.
- Fiedler, F. E. 1964. "A Contingency Model of Leadership Effectiveness," *"Advances in Experimental Social Psychology,"* 1, 149–190.
- Flin, R. 2006. "Erosion of Managerial Resilience: From Vasa to NASA." In Hollnagel, E. Woods, D.D., and Levenson, N. (Eds), "Resilience Engineering Concepts and Precepts," pp. 223-233. Aldershot: Ashgate.
- Franz, C. R. 1985. "User Leadership in the Systems Development Life Cycle: A Contingency Model." *"Journal of Management Information Systems,"* 2 (2), 5-25
- Frick, D. 2009. "Seven Pillars of Servant Leadership: Practicing the Wisdom of Leading by Serving," New Jersey: Paulist Press.
- Gardner, W.L., C.C. Coglisier, K.M. Daviss, & M.P. Dickens. 2011. "Authentic leadership: A review of the literature and research agenda." *"Leadership Quarterly,"* 22, 1120-1145
- Goleman D. 1998. "The emotionally competent leader." *"Health Forum Journal,"* 41(2), 38- 76
- Gurven, M., C. Von Rueden,, and C. Kaplan, M.L. Vie, 2013. "How Universal Is the Big Five? Testing the Five-Factor Model of Personality Variation Among Forager–Farmers in the Bolivian Amazon," *"Journal of Personality and Social Psychology,"* Vol. 104, No. 2, 354–370.
- Hersey, P., K.H. Blanchard, and D.E. Johnson. 2001. "Management of Organisational Behaviour Leading Human Resources." NJ, USA: Prentice Hall.
- Herzberg, F., B. Mausnek, and B. Snyderman. 1959. "The Motivation to Work (Second Edition)." New York, New York, USA: John Wiley and Sons.
- House, R. J., and R.R. Mitchell 1974. "Path-goal theory of leadership." *"Journal of Contemporary Business,"* 3(4), pp. 81-98.
- Isaac, R. G., W.J. Zerbe,, & D.C Pitt. 2001. "Leadership and motivation: The effective application of expectancy theory." *"Journal of Managerial Issues,"* 13(2), 212-226.
- Judge, T. A., J.Y. Bono, R. Ilies, & M.W. Gerhardt. 2002. "Personality and leadership: A qualitative and quantitative review." *"Journal of Applied Psychology,"* 87, 765–780.
- Keith, K. 2012. "The Case for Servant-Leadership," Honolulu, Hawaii: Terrace Press, Second Edition.
- Kolb, J. A. 1995. "Leader behaviours affecting team performance: Similarities and differences between leader/member assessments." *"Journal of Business Communication,"* 32, 233-248.
- Lewin, K., R. Lippitt. and R.K. White. 1939. "Patterns of aggressive behavior in experimentally created social climates." *"Journal of Social Psychology,"* 10, 271-301.
- Lloyd-Walker, B. and D. Walker. 2011. "Authentic leadership for 21st century project delivery," *"International Journal of Project Management,"* 29, pp 383-395.
- Sipe, J. and D. Frick. 2009. "The Seven Pillars of Servant Leadership: Practicing the Wisdom of Leading by Serving. Mahwah, NJ, USA: Paulist Press.
- Triandis, H.C., R. Bontempo, , M.J. Villareal, M. Asai, and N. Lucca. 1988. "Individualism and collectivism: Cross-cultural perspectives on self-ingroup relationships," *"Journal of Personality and Social Psychology,"* Vol.54, No. 2. 323-338.
-

- Manz, C. C., and H.P. Sims Jr. 1989. "Superleadership: Leading Others to Leave Themselves". New York, NY, USA: Prentice Hall Press.
- McCleave, E. B., and U. Capella. 2015. "A correlational analysis of frontline leaders as drivers of technical innovation in the aerospace industry based on the servant leadership theory."
- "US Dissertation Abstracts International Section A: Humanities and Social Sciences," Vol 75(8-A)(E).
- Marion, R., and M. Uhl-Bien. 2001. "Leadership in complex organizations," "The Leadership Quarterly," 12, pp. 389–418.
- Maslow, A.H. 1943. "A theory of human motivation," "Psychological Review," 50 (4) 370–96.
- National Research Council. 1991. "In The Mind's Eye," Washington, D.C., USA: National Academy of Science.
- Nikoi, E. (Ed) 2014. "Collaborative Communication Processes and Decision Making in Organizations." Hershey, PA: Business Science Reference.
- Stogdill, R.M. 1948. "Personal factors associated with leadership: A survey of the literature." "Journal of Psychology," Vol. 25.
- Toor, S.R. and G. Ofori. 2008. "Leadership vs. management: How they are different, and why!"
- "Journal of Leadership and Management in Engineering," 8(2), 61- 71.
- Uhl-Bien, M., R.E. Riggio, K.B. Lowec, M.K. Carstend. 2014. "Followership theory: A review and research agenda," "Leadership Quarterly 25th Anniversary Issue, The Leadership Quarterly," Volume 25, Issue 1, Pages 83–104.
- Valle, S., & L. Avella. 2003. "Cross-functionality and leadership of the new product development teams." "European Journal of Innovation Management," 6(1), 32 – 47.
- Vroom, V.H. 1964. "Work and Motivation." New York, NY, USA: Wiley.
- Walumba, F.O., B.J. Avolio, W.L. Gardner, T.S. Wernsing, and S.J. Peterson, 2008 "Authentic leadership: Development and validation of a theory-based measure," "Journal of Management," 34:1, pp. 89-126.
- Williams, C.R. and A. Reyes. 2012. "Developing Systems Engineers at NASA Global Journal of Flexible Systems Management," 13(3), 159–164.
- Yukl, G.A. 2012. "Leadership in Organizations." 8th Ed. Upper Saddle River, NJ, USA: Prentice Hall.

Primary References

- Fairley, R.E. 2009. "Managing and Leading Software Projects." Hoboken, NJ, USA: John Wiley & Sons.
- Myers, I.B., and P.B. Myers. 1995. "Gifts Differing: Understanding Personality Type," 2nd ed. Mountain View, CA: Davies-Black Publishing under special license from CPP, Inc.
- Wilson, Larry. 2004. "The Social Styles Handbook." Belgium: Nova Vista Publishing.
- Barrett, D.J. 2006. "Leadership Communication." Boston: McGraw Hill Education. Bass, B. M., & R. Bass. 2008. "The Bass Handbook of Leadership: Theory, Research, and Managerial Applications." New York, NY, USA: Free Press.
- Bennis, W. 2003. "On Becoming a Leader." New York, NY, USA: Perseus Publishing.
- Northouse, P. G. 2007. "Leadership Theory and Practice." (4th ed.). Thousand Oaks, CA, USA: Sage.

Additional References

- Bass, B. M., & B.J. Avolio. 1994. "Improving Organizational Effectiveness Through Transformational Leadership." Thousand Oaks, CA, USA: Sage.
- Fiedler, F. E. 1964. "A contingency model of leadership effectiveness." "In L. Berkowitz (Ed.), *Advances in experimental social psychology*" (Vol. 1). New York, NY, USA: Academic Press.
- Lowe, K. B., K.G. Kroeck, & N. Sivasubramaniam. 1996. "Effectiveness correlates of transformational and transactional leadership: A meta-analytic review of the MLQ literature." "The Leadership Quarterly," 7(3), 385-415.
- Pandya, K. D. 2014. "Key Competencies of Project Leader Beyond the Essential Technical Capabilities," "IUP Journal of Knowledge Management," Vol. 12 Issue 4, 39-48.
- Ram, C., S. Drotter, and J. Noel. 2001. "The Leadership Pipeline: How to Build the Leadership Powered Company." San Francisco, CA, USA: Jossey-Bass (a Wiley Company).
-

Knowledge Area: Enabling Individuals

Enabling Individuals

Contents of this Knowledge Area

- Roles and Competencies (Heidi Davidz, Dick Fairley, and Tom Hilburn) (Alice Squires and Art Pyster)
- Assessing Individuals (Heidi Davidz) (Alice Squires and Art Pyster)
- Developing Individuals (Heidi Davidz)
- Ethical Behavior (Dick Fairley and Chuck Calvano) (Scott Jackson, Heidi Davidz, Alice Squires, and Art Pyster)
- Lead Authors:
- Heidi Davidz and Dick Fairley
- Contributing Authors:
- Alice Squires and Art Pyster

This knowledge area focuses on enabling an individual to perform SE and addresses the roles of individuals in the SE profession, how individuals are developed for and assessed in these roles, and what ethical behavior is expected of them. Once an individual is enabled to perform SE using the techniques described here, the individual can apply the knowledge found in Part 3, Systems Engineering and Management, about how to perform SE.

Part 5, Enabling Systems Engineering, to which this knowledge area belongs, explores how systems engineering (SE) is enabled at three levels of organization: the business or enterprise, the team, and the individual. Ultimately, individuals perform SE tasks within a team or business.

For the sake of brevity, the term “business” is used to mean “business or enterprise” throughout most of this knowledge area. For a nuanced explanation of what distinguishes a business from an enterprise, see Enabling Systems Engineering.

Topics

Each part of the SEBoK is composed of knowledge areas (KAs]. Each KA groups topics together around a theme related to the overall subject of the part. This KA contains four topics:

- Roles and Competencies discusses allocation of SE roles, which sets of competencies correspond to particular roles, and what competency models are current in the SE world.
- Assessing Individuals discusses how to determine the level of individual proficiency and quality of performance.
- Developing Individuals explains how SE competency is acquired.
- Ethical Behavior describes the ethical standards that apply to individuals and organizations.

Context

The following brief review of terms and concepts provides context for the topics in this knowledge area.

Individuals, Teams, Businesses, and Enterprises

The ability to perform SE resides in individuals, teams, and businesses. An expert systems engineer possesses many competencies at a high level of proficiency, but no one can be highly proficient in all possible competencies. Collectively, a team and a business might possess all needed competencies at a high level of proficiency. A business performs the full range of SE roles, may have dedicated functions to perform specific SE roles, and may have a

strategy for combining individual, team, and business abilities to execute SE on a complex activity. Individuals within the business may be responsible for performing one or more roles.

For descriptions of SE roles and competencies from the literature, see Roles and Competencies.

Competency, Capability, Capacity, and Performance

The final execution and performance of SE is a function of competency, capability, and capacity. There is some complexity here. For example:

- There is disagreement in the literature about whether the term competency applies to the individual level only, or can be correctly used at the team, project, and enterprise levels as well.
- Capability encompasses not just human capital, but processes, machines, tools, and equipment as well. Even if an individual has an outstanding level of competency, having to perform within a limited timeframe might degrade the results. Capacity accounts for this.

Systems Engineering Competency

Competency is built from knowledge, skills, abilities, and attitudes (KSAA). What is inherent in an individual may be subsequently developed through education, training, and experience. Traditionally, SE competencies have been developed primarily through experience, but recently, education and training have taken on a much greater role.

SE competency must be viewed through its relationships to the systems life cycle, the SE discipline, and the domain in which the engineer practices SE.

Competency Models

SE competency models can be used to explicitly state and actively manage the SE competencies within in an organization.

Competency models for SE typically include:

- technical KSAA's;
- "soft" KSAA's such as leadership and communications;
- KSAA's that focus on the domains within which SE is to be practiced;
- a set of applicable competencies; and
- a scale for assessing the level of individual proficiency in each competency (often subjective, since proficiency is not easily measured).

See Roles and Competencies for descriptions of publicly available SE competency models.

References

None.

Roles and Competencies

- Lead Authors:
 - Heidi Davidz, Dick Fairley, and Tom Hilburn
- Contributing Authors:
 - Alice Squires and Art Pyster

Enabling individuals to perform systems engineering (SE) requires an understanding of SE competencies, roles, and tasks; plus knowledge, skills, abilities, and attitudes (KSAA). Within a business or enterprise, SE responsibilities are allocated to individuals through the definition of SE roles associated with a set of tasks. For an individual, a set of KSAs enables the fulfillment of the competencies needed to perform the tasks associated with the assigned SE role. SE competencies reflect the individual's KSAs, which are developed through education, training, and on-the-job experience. Traditionally, SE competencies build on innate personal qualities and have been developed primarily through experience. Recently, education and training have taken on a greater role in the development of SE competencies.

Relationship of SE Competencies and KSAs

There are many ways to define competency. It can be thought of as a measure of the ability to use the appropriate KSAs to successfully complete specific job-related tasks (Whitcomb, Khan, White 2014). Competencies align with the tasks that are expected to be accomplished for the job position (Holt and Perry 2011). KSAs belong to the individual. In the process of filling a position, organizations have a specific set of competencies associated with tasks that are directly related to the job. A person possesses the KSAs that enable them to perform the desired tasks at an acceptable level of competency.

The KSAs are obtained and developed from a combination of several sources of learning including education, training, and on-the-job experience. By defining the KSAs in terms of a standard taxonomy, they can be used as learning objectives for competency development (Whitcomb, Khan, White 2014). Bloom's Taxonomy for the cognitive and affective domains provides this structure (Bloom 1956, Krathwohl 2002). The cognitive domain includes knowledge, critical thinking, and the development of intellectual skills, while the affective domain describes growth in awareness, attitude, emotion, changes in interest, judgment, and the development of appreciation (Bloom 1956). The affective does not refer to additional topics which a person learns about, but rather to a transformation of the person in relation to the original set of topics learned. Cognitive and affective processes within Bloom's taxonomic classification schema refer to levels of observable actions, which indicate learning is occurring. Bloom's Taxonomy for the cognitive and affective domains define terms as categories of levels that can be used for consistently defining KSAA statements (Krathwohl 2002):

Cognitive Domain:

- Remember
- Understand
- Apply
- Analyze
- Evaluate
- Create

Affective Domain:

- Receive
 - Respond
 - Value
 - Organize
-

- Characterize

Both cognitive and affective domains should be included in the development of systems engineering competency models, because the cognitive domain learning concerns the consciously developed knowledge about the various subjects and the ability to perform tasks, whilst the affective learning concerns the interest in or willingness to use particular parts of the knowledge learned and the extent to which the systems engineer is characterized by taking approaches which are inherently systemic. Using the affective domain in the specification of KSAs, is also important as every piece of information we process in our brains goes through our affective (emotional) processors before it is integrated by our cognitive processors (Whitcomb and Whitcomb 2013).

SE Competency Models

Contexts in which individual competency models are typically used include:

- **Recruitment and Selection:** Competencies define categories for behavioral event interviewing (BEI), increasing the validity and reliability of selection and promotion decisions.
- **Human Resources Planning and Placements:** Competencies are used to identify individuals to fill specific positions and/or identify gaps in key competency areas.
- **Education, Training, and Development:** Explicit competency models let employees know which competencies are valued within their organization. Curriculum and interventions can be designed around desired competencies.

Commonality and Domain Expertise

No single individual is expected to be proficient in all the competencies found in any model. The organization, overall, must satisfy the required proficiency in sufficient quantity to support business needs. Organizational capability is not a direct summation of the competency of the individuals in the organization, since organizational dynamics play an important role that can either raise or lower overall proficiency and performance. The articles *Enabling Teams* and *Enabling Businesses and Enterprises* explore this further.

SE competency models generally agree that systems thinking, taking a holistic view of the system that includes the full life cycle, and specific knowledge of both technical and managerial SE methods are required to be a fully capable systems engineer. It is also generally accepted that an accomplished systems engineer will have expertise in at least one domain of practice. General models, while recognizing the need for domain knowledge, typically do not define the competencies or skills related to a specific domain. Most organizations tailor such models to include specific domain KSAs and other peculiarities of their organization.

INCOSE Certification

Certification is a formal process whereby a community of knowledgeable, experienced, and skilled representatives of an organization, such as the International Council on Systems Engineering (INCOSE), provides formal recognition that a person has achieved competency in specific areas (demonstrated by education, experience, and knowledge). (INCOSE nd). The most popular credential in SE is offered by INCOSE, which requires an individual to pass a test to confirm knowledge of the field, requires experience in SE, and recommendations from those who have knowledge about the individual's capabilities and experience. Like all such credentials, the INCOSE certificate does not guarantee competence or suitability of an individual for a particular role, but is a positive indicator of an individual's ability to perform. Individual workforce needs often require additional KSAs for any given systems engineer, but certification provides an acknowledged common baseline.

Domain- and Industry-specific Models

No community consensus exists on a specific competency model or small set of related competency models. Many SE competency models have been developed for specific contexts or for specific organizations, and these models are useful within these contexts.

Among the domain- and industry-specific models is the Aerospace Industry Competency Model (ETA 2010), developed by the Employment and Training Administration (ETA) in collaboration with the Aerospace Industries Association (AIA) and the National Defense Industrial Association (NDIA), and available online. This model is designed to evolve along with changing skill requirements in the aerospace industry. The ETA makes numerous competency models for other industries available online (ETA 2010). The NASA Competency Management System (CMS) Dictionary is predominately a dictionary of domain-specific expertise required by the US National Aeronautics and Space Administration (NASA) to accomplish their space exploration mission (NASA 2009).

Users of models should be aware of the development method and context for the competency model they plan to use, since the primary competencies for one organization might differ from those for another organization. These models often are tailored to the specific business characteristics, including the specific product and service domain in which the organization operates. Each model typically includes a set of applicable competencies along with a scale for assessing the level of proficiency.

SE Competency Models — Examples

Though many organizations have proprietary SE competency models, published SE competency models can be used for reference. Table 1 lists information about several published SE competency models, and links to these sources are shown below in the references section. Each model was developed for a unique purpose within a specific context and validated in a particular way. It is important to understand the unique environment surrounding each competency model to determine its applicability in any new setting.

Table 1. Summary of Competency Models. (SEBoK Original)

Competency Model	Date	Author	Purpose	Development Method	Competency Model Source
INCOSE UK WG	2010	INCOSE	Identify the competencies required to conduct good systems engineering	INCOSE Working Group	(INCOSE 2010), (INCOSE UK 2010)
ENG Competency Model	2013	DAU	Identify competencies required for the DoD acquisition engineering professional	DoD and DAU internal development	(DAU 2013)
NASA APPEL Competency Model	2009	NASA	To improve project management and systems engineering at NASA	NASA internal development - UPDATE IN WORK	(NASA 2009)
MITRE Competency Model	2007	MITRE	To define new curricula for systems engineering and to assess personnel and organizational capabilities	Focus groups as described in (Trudeau 2005)	(Trudeau 2005), (MITRE 2007)
CMMI for Development	2007	SEI	Process improvement maturity model for the development of products and services	SEI Internal Development	(SEI 2007), (SEI 2004)

Other models and lists of traits include: Hall (1962), Frank (2000; 2002; 2006), Kasser et al. (2009), Squires et al. (2011), and Armstrong et al. (2011). Ferris (2010) provides a summary and evaluation of the existing frameworks for personnel evaluation and for defining SE education. Squires et al. (2010) provide a competency-based approach that can be used by universities or companies to compare their current state of SE capability development against a government-industry defined set of needs. SE competencies can also be inferred from standards such as ISO-15288 (ISO/IEC/IEEE 15288 2015) and from sources such as the INCOSE *Systems Engineering Handbook* (INCOSE 2012), the INCOSE Systems Engineering Certification Program, and CMMI criteria (SEI 2007). Whitcomb, Khan,

and White describe the development of a systems engineering competency model for the United States Department of Defense based on a series of existing competency models (Whitcomb, Khan, and White 2013; 2014).

To provide specific examples for illustration, more details about three SE competency model examples follow. These include:

- The International Council on Systems Engineering (INCOSE) UK Advisory Board model (INCOSE 2010), (INCOSE UK 2009);
- The DAU ENG model (DAU 2013); and
- The NASA Academy of Program/Project & Engineering Leadership (APPEL) model (NASA 2009)

INCOSE SE Competency Model

The INCOSE model was developed by a working group in the United Kingdom (Cowper et al. 2005). As Table 2 shows, the INCOSE framework is divided into three theme areas - systems thinking, holistic life cycle view, and systems management - with a number of competencies in each. The INCOSE UK model was later adopted by the broader INCOSE organization (INCOSE 2010).

Table 2. INCOSE UK Working Group Competency (INCOSE UK 2010).

This information has been published with the kind permission of INCOSE UK Ltd and remains the copyright of INCOSE UK Ltd - ©INCOSE UK LTD 2010. All rights reserved.

Systems Thinking	System Concepts	
	Super-System Capability Issues	
	Enterprise and Technology Environment	
Hollistic Lifecycle View	Determining and Managing Stakeholder Requirements	
	Systems Design	Architectural Design
		Concept Generation
		Design For...
		• Functional Analysis
		• Interface Management
		• Maintaining Design Integrity
		• Modeling and Simulation
		• Selecting Preferred Solution
		• System Robustness
	Systems Integration & Verification	
	Validation	
	Transition to Operation	
Systems Engineering Management	Concurrent Engineering	
	Enterprise Integration	
	Integration of Specialties	
	Lifecycle Process Definition	
	Planning, Monitoring, and Controlling	

United States DoD Engineering Competency Model

The model for US Department of Defense (DoD) acquisition engineering professionals (ENG) includes 41 competency areas, as shown in Table 3 (DAU 2013). Each is grouped according to a “Unit of Competence” as listed in the left-hand column. For this model, the four top-level groupings are: analytical, technical management, professional, and business acumen. The life cycle view used in the INCOSE model is evident in the ENG analytical grouping but is not cited explicitly. Technical management is the equivalent of the INCOSE SE management, but additional competencies are added, including software engineering competencies and acquisition. Selected general professional skills have been added to meet the needs for strong leadership required of the acquisition engineering professionals. The business acumen competencies were added to meet the needs of these professionals to be able to support contract development and oversight activities and to engage with the defense industry.

Table 3. DoD Competency Model (DAU 2013) Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

Analytical (11)	1. Mission-Level Assessment
	2. Stakeholder Requirements Definition
	3. Requirements Analysis
	4. Architecture Design
	5. Implementation
	6. Integration
	7. Verification
	8. Validation
	9. Transition
	10. Design Considerations
	11. Tools and Techniques
Technical Management (10)	12. Decision Analysis
	13. Technical Planning
	14. Technical Assessment
	15. Configuration Management
	16. Requirements Management
	17. Risk Management
	18. Data Management
	19. Interface Management
	20. Software Engineering
	21. Acquisition

Professional (10)	22. Problem Solving
	23. Strategic Thinking
	24. Professional Ethics
	25. Leading High-Performance Teams
	26. Communication
	27. Coaching and Mentoring
	28. Managing Stakeholders
	29. Mission and Results Focus
	30. Personal Effectiveness/Peer Interaction
	31. Sound Judgment
Business Acumen (10)	32. Industry Landscape
	33. Organization
	34. Cost, Pricing, and Rates
	35. Cost Estimating
	36. Financial Reporting and Metrics
	37. Business Strategy
	38. Capture Planning and Proposal Process
	39. Supplier Management
	40. Industry Motivation, Incentives, Rewards
	41. Negotiations

NASA SE Competency Model

The US National Aeronautics and Space Administration (NASA) APPEL website provides a competency model that covers both project engineering and systems engineering (APPEL 2009). There are three parts to the model: one that is unique to project engineering, one that is unique to systems engineering, and a third that is common to both disciplines. Table 4 below shows the SE aspects of the model. The project management items include project conceptualization, resource management, project implementation, project closeout, and program control and evaluation. The common competency areas are: NASA internal and external environments, human capital and management, security, safety and mission assurance, professional and leadership development, and knowledge management. This 2010 model is adapted from earlier versions. Squires et al. (2010, 246-260) offer a method that can be used to analyze the degree to which an organization's SE capabilities meet government-industry defined SE needs.

Table 4. SE Portion of the APPEL Competency Model (APPEL 2009). Released by NASA APPEL.

System Design	SE 1.1 - Stakeholder Expectation Definition & Management
	SE 1.2 - Technical Requirements Definition
	SE 1.3 - Logical Decomposition
	SE 1.4 - Design Solution Definition
Product Realization	SE 2.1 - Product Implementation
	SE 2.2 - Product Integration
	SE 2.3 - Product Verification
	SE 2.4 - Product Validation
	SE 2.5 - Product Transition
Technical Management	SE 3.1 - Technical Planning
	SE 3.2 - Requirements Management
	SE 3.3 - Interface Management
	SE 3.4 - Technical Risk Management
	SE 3.5 - Configuration Management
	SE 3.6 - Technical Data Management
	SE 3.7 - Technical Assessment
	SE 3.8 - Technical Decision Analysis

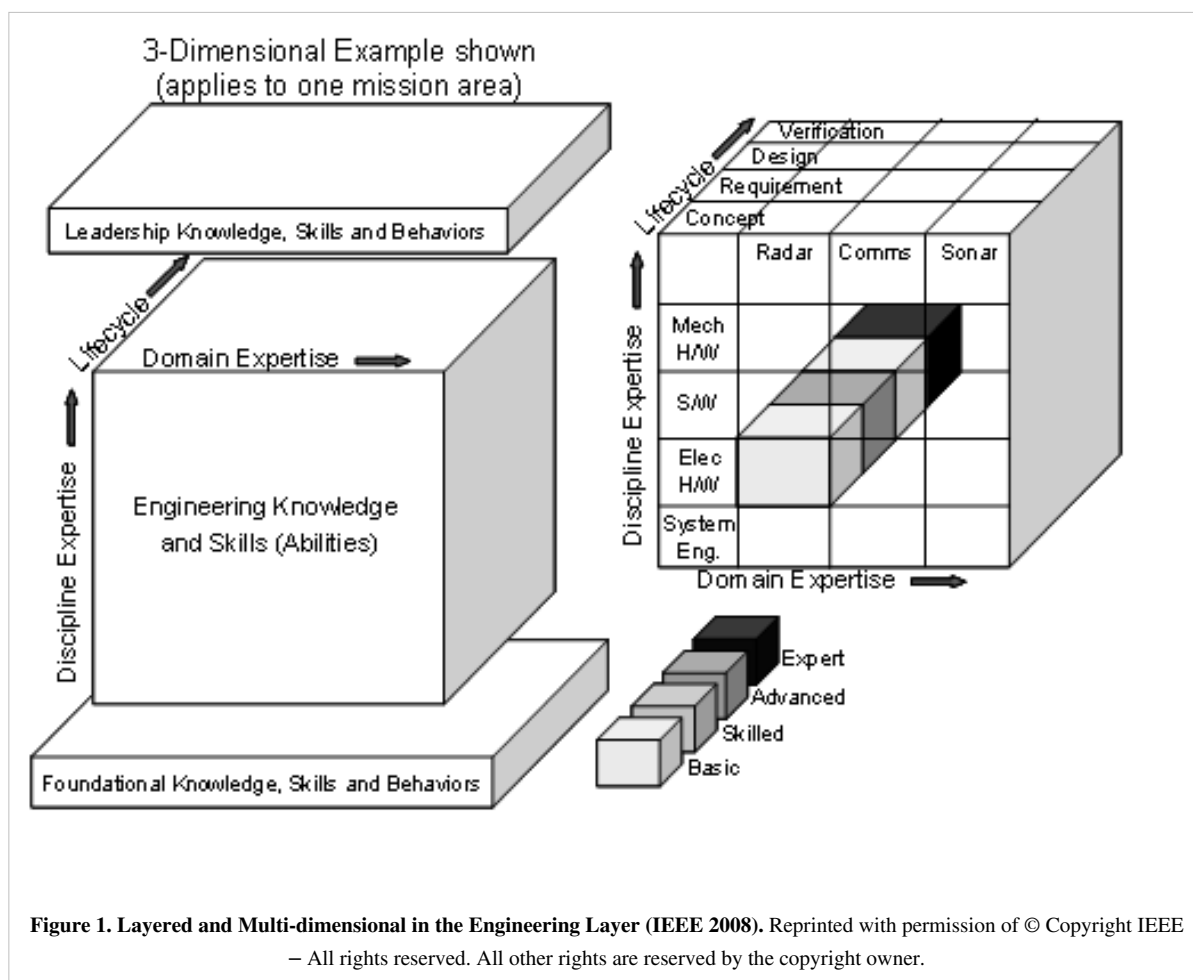
Relationship of SE Competencies to Other Competencies

SE is one of many engineering disciplines. A competent SE must possess KSAs that are unique to SE, as well as many other KSAs that are shared with other engineering and non-engineering disciplines.

One approach for a complete engineering competency model framework has multiple dimensions where each of the dimensions has unique KSAs that are independent of the other dimensions (Wells 2008). The number of dimensions depends on the engineering organization and the range of work performed within the organization. The concept of creating independent axes for the competencies was presented in Jansma and Derro (2007), using technical knowledge (domain/discipline specific), personal behaviors, and process as the three axes. An approach that uses process as a dimension is presented in Widmann et al. (2000), where the competencies are mapped to process and process maturity models. For a large engineering organization that creates complex systems solutions, there are typically four dimensions:

1. **Discipline** (e.g., electrical, mechanical, chemical, systems, optical);
2. **Life Cycle** (e.g., requirements, design, testing);
3. **Domain** (e.g., aerospace, ships, health, transportation); and
4. **Mission** (e.g., air defense, naval warfare, rail transportation, border control, environmental protection).

These four dimensions are built on the concept defined in Jansma and Derro (2007) and Widmann et al. (2000) by separating discipline from domain and by adding mission and life cycle dimensions. Within many organizations, the mission may be consistent across the organization and this dimension would be unnecessary. A three-dimensional example is shown in Figure 1, where the organization works on only one mission area so the mission dimension has been eliminated from the framework.



The discipline, domain, and life cycle dimensions are included in this example, and some of the first-level areas in each of these dimensions are shown. At this level, an organization or an individual can indicate which areas are included in their existing or desired competencies. The sub-cubes are filled in by indicating the level of proficiency that exists or is required. For this example, blank indicates that the area is not applicable, and colors (shades of gray) are used to indicate the levels of expertise. The example shows a radar electrical designer that is an expert at hardware verification, is skilled at writing radar electrical requirements, and has some knowledge of electrical hardware concepts and detailed design. The radar electrical designer would also assess his or her proficiency in the other areas, the foundation layer, and the leadership layer to provide a complete assessment.

References

Works Cited

- Armstrong, J.R., D. Henry, K. Kepcher, and A. Pyster. 2011. "Competencies required for successful acquisition of large, highly complex systems of systems." Paper presented at 21st Annual International Council on Systems Engineering (INCOSE) International Symposium (IS), 20-23 June 2011, Denver, CO, USA.
- Bloom, Benjamin S., Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. 1956. *Taxonomy of Educational Objectives*. New York, NY, USA: David McKay.
- Cowper, D., S. Bennison, R. Allen-Shalless, K. Barnwell, S. Brown, A. El Fatatry, J. Hooper, S. Hudson, L. Oliver, and A. Smith. 2005. *Systems Engineering Core Competencies Framework*. Folkestone, UK: International Council on Systems Engineering (INCOSE) UK Advisory Board (UKAB).

- DAU. 2013. *ENG Competency Model*, 12 June 2013 version. In Defense Acquisition University (DAU)/U.S. Department of Defense Database Online. Accessed on June 3, 2015. Available at https://dap.dau.mil/workforce/Documents/Comp/ENG%20Competency%20Model%2020130612_Final.pdf.
- ETA. 2010. *Career One Stop: Competency Model Clearing House: Aerospace Competency Model*. in Employment and Training Administration (ETA)/U.S. Department of Labor. Washington, DC, USA. Accessed on September 15, 2011. Available at <https://www.careeronestop.org/competencymodel/pyramid.aspx?AEO=Y>.
- Ferris, T.L.J. 2010. "Comparison of systems engineering competency frameworks." Paper presented at the 4th Asia-Pacific Conference on Systems Engineering (APCOSE), Systems Engineering: Collaboration for Intelligent Systems, 3-6 October 2010, Keelung, Taiwan.
- Frank, M. 2000. "Engineering systems thinking and systems thinking." *Systems Engineering*. 3(3): 163-168.
- Frank, M. 2002. "Characteristics of engineering systems thinking – A 3-D approach for curriculum content." *IEEE Transaction on System, Man, and Cybernetics*. 32(3) Part C: 203-214.
- Frank, M. 2006. "Knowledge, abilities, cognitive characteristics and behavioral competences of engineers with high capacity for engineering systems thinking (CEST)." *Systems Engineering*. 9(2): 91-103. (Republished in *IEEE Engineering Management Review*. 34(3) (2006):48-61).
- Hall, A.D. 1962. *A Methodology for Systems Engineering*. Princeton, NJ, USA: D. Van Nostrand Company Inc.
- Holt, J. and S. Perry. 2011. *A Pragmatic Guide to Competency, Tools, Frameworks, and Assessment*. Swindon, UK: BCS, The Chartered Institute for IT.
- INCOSE. 2011. "History of INCOSE Certification Program." Accessed April 13, 2015 at <https://www.incose.org/certification/CertHistory>.
- INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.
- INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2010-003.
- INCOSE UK. 2010. "Systems Engineering Competency Framework," Accessed on June 3, 2015. Available at https://www.incoseonline.org.uk/Normal_Files/Publications/Framework.aspx?CatID=Publications&SubCat=INCOSEPublications.
- Jansma, P.A. and M.E. Derro. 2007. "If you want good systems engineers, sometimes you have to grow your own!" Paper presented at IEEE Aerospace Conference, 3-10 March, 2007, Big Sky, MT, USA.
- Kasser, J.E., D. Hitchins, and T.V. Huynh. 2009. "Reengineering systems engineering." Paper presented at the 3rd Annual Asia-Pacific Conference on Systems Engineering (APCOSE), 2009, Singapore.
- Krathwohl, David. 2002. "A revision of bloom's taxonomy: An overview." "Theory Into Practice," 41(4): 212-218.
- Menrad, R. and H. Lawson. 2008. "Development of a NASA integrated technical workforce career development model entitled: Requisite occupation competencies and knowledge – The ROCK." Paper presented at the 59th International Astronautical Congress (IAC), 29 September-3 October, 2008, Glasgow, Scotland.
- MITRE. 2007. "MITRE Systems Engineering (SE) Competency Model." Version 1.13E. September 2007. Accessed on June 3, 2015. Available at <https://www.mitre.org/publications/technical-papers/systems-engineering-competency-model>.
- NASA. 2009. *NASA Competency Management Systems (CMS): Workforce Competency Dictionary*, revision 7a. Washington, D.C, USA: U.S. National Aeronautics and Space Administration (NASA).
- NASA. 2009. *Project Management and Systems Engineering Competency Model*. Academy of Program/Project & Engineering Leadership (APPEL). Washington, DC, USA: US National Aeronautics and Space Administration (NASA). Accessed on June 3, 2015. Available at <https://appel.nasa.gov/competency-model/>.
-

SEI. 2007. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.2, Measurement and Analysis Process Area. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

SEI. 2004. *CMMI-Based Professional Certifications: The Competency Lifecycle Framework*, Software Engineering Institute, CMU/SEI-2004-SR-013. Accessed on June 3, 2015. Available at <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=6833>.

Squires, A., W. Larson, and B. Sauser. 2010. "Mapping space-based systems engineering curriculum to government-industry vetted competencies for improved organizational performance." *Systems Engineering*. 13 (3): 246-260. Available at <https://dx.doi.org/10.1002/sys.20146>.

Squires, A., J. Wade, P. Dominick, and D. Gelosh. 2011. "Building a competency taxonomy to guide experience acceleration of lead program systems engineers." Paper presented at the Conference on Systems Engineering Research (CSER), 15-16 April 2011, Los Angeles, CA. Wells, B.H. 2008. "A multi-dimensional hierarchical engineering competency model framework." Paper presented at IEEE International Systems Conference, March 2008, Montreal, Canada. Whitcomb, C., R. Khan and C. White. 2014. "Systems Engineering Competency FY14 Technical Report." Naval Postgraduate School Technical Report, Monterey, CA. Available at: <https://calhoun.nps.edu/handle/10945/44705>.

Whitcomb, C., L. Whitcomb. 2013. "Effective Interpersonal and Team Communication Skills for Engineers." Hoboken, NJ, USA: IEEE Press, John Wiley and Sons.

Whitcomb, C., R. Khan, and C. White. 2013. "Systems Engineering Competency FY13 Technical Report." Naval Postgraduate School Technical Report, Monterey, CA. Accessed on June 4, 2015. Available at <https://calhoun.nps.edu/handle/10945/43424>.

Whitcomb, C., R. Khan, and C. White. 2014. "Systems Engineering Competency FY14 Technical Report." Naval Postgraduate School Technical Report, Monterey, CA. Accessed on June 4, 2015. Available at <https://calhoun.nps.edu/handle/10945/44705>.

Widmann, E.R., G.E. Anderson, G.J. Hudak, and T.A. Hudak. 2000. "The taxonomy of systems engineering competency for the new millennium." Presented at 10th Annual INCOSE Internal Symposium, 16-20 July 2000, Minneapolis, MN, USA.

Primary References

DAU. 2013. *ENG Competency Model*, 12 June 2013 version. In Defense Acquisition University (DAU)/U.S. Department of Defense Database Online. Accessed on June 3, 2015. Available at https://dap.dau.mil/workforce/Documents/Comp/ENG%20Competency%20Model%2020130612_Final.pdf.

INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2010-003.

Additional References

Whitcomb, C., J. Delgado, R. Khan, J. Alexander, C. White, D. Grambow, P. Walter. 2015. "The Department of the Navy Systems Engineering Career Competency Model." Proceedings of the Twelfth Annual Acquisition Research Symposium. Naval Postgraduate School, Monterey, CA.

Assessing Individuals

- Lead Author:
- Heidi Davidz
- Contributing Authors:
- Alice Squires and Art Pyster

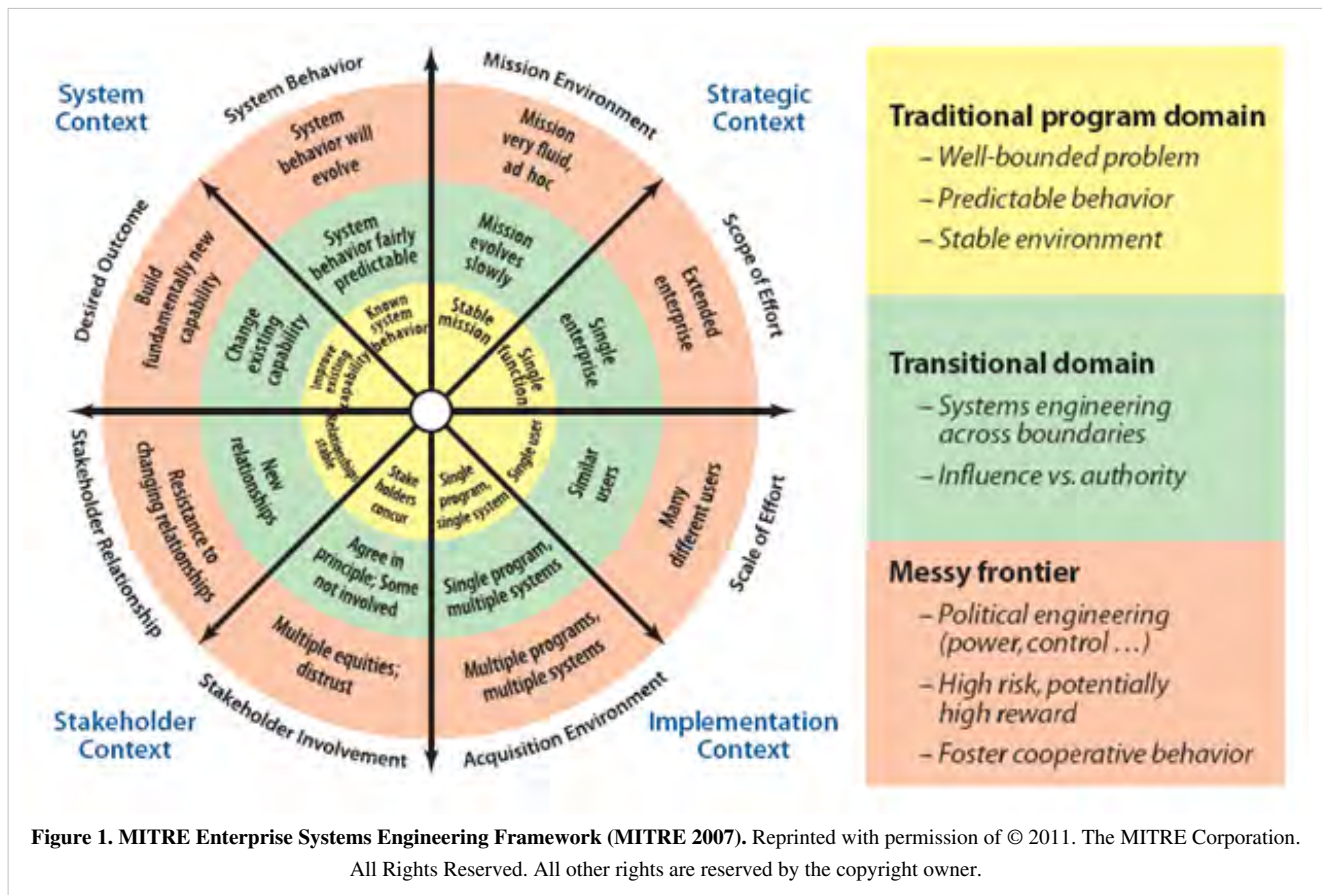
The ability to fairly assess individuals is a critical aspect of enabling individuals. This article describes how to assess the systems engineering (SE) competencies needed and possessed by an individual, as well as that individual's SE performance.

Assessing Competency Needs

If an organization wants to use its own customized competency model, an initial decision is *make vs. buy*. If there is an existing SE competency model that fits the organization's context and purpose, the organization might want to use the existing SE competency model directly. If existing models must be tailored or a new SE competency model developed, the organization should first understand its context.

Determining Context

Prior to understanding what SE competencies are needed, it is important for an organization to examine the situation in which it is embedded, including environment, history, and strategy. As Figure 1 shows, MITRE has developed a framework characterizing different levels of systems complexity. (MITRE 2007, 1-12) This framework may help an organization identify which competencies are needed. An organization working primarily in the *traditional program domain* may need to emphasize a different set of competencies than an organization working primarily in the *messy frontier*. If an organization seeks to improve existing capabilities in one area, extensive technical knowledge in that specific area might be very important. For example, if stakeholder involvement is characterized by multiple equities and distrust, rather than collaboration and concurrence, a higher level of competency in being able to balance stakeholder requirements might be needed. If the organization's desired outcome builds a fundamentally new capability, technical knowledge in a broader set of areas might be useful.



Additionally, an organization might consider both its current situation and its forward strategy. For example, if an organization has previously worked in a traditional systems engineering context (MITRE 2007) but has a strategy to transition into enterprise systems engineering (ESE) work in the future, that organization might want to develop a competency model both for what was important in the traditional SE context and for what will be required for ESE work. This would also hold true for an organization moving to a different contracting environment where competencies, such as the ability to properly tailor the SE approach to *right size* the SE effort and balance cost and risk, might be more important.

Determining Roles and Competencies

Once an organization has characterized its context, the next step is to understand exactly what SE roles are needed and how those roles will be allocated to teams and individuals. To assess the performance of an individual, it is essential to explicitly state the roles and competencies required for that individual. See the references in Roles and Competencies for guides to existing SE standards and SE competency models.

Assessing Individual SE Competency

In order to demonstrate competence, there must be some way to qualify and measure it, and this is where competency assessment is used (Holt and Perry 2011). This assessment informs the interventions needed to further develop individual SE KSAA upon which competency is based. Described below are possible methods which may be used for assessing an individual's current competency level; an organization should choose the correct model based on their context, as identified previously.

Proficiency Levels

In order to provide a context for individuals and organizations to develop competencies, a consistent system of defining KSAs should be created. One popular method is based on Bloom's taxonomy (Bloom 1984), presented below for the cognitive domain in order from least complex to most complex cognitive ability.

- **Remember:** Recall or recognize terms, definitions, facts, ideas, materials, patterns, sequences, methods, principles, etc.
- **Understand:** Read and understand descriptions, communications, reports, tables, diagrams, directions, regulations, etc.
- **Apply:** Know when and how to use ideas, procedures, methods, formulas, principles, theories, etc.
- **Analyze:** Break down information into its constituent parts and recognize their relationships to one another and how they are organized; identify sublevel factors or salient data from a complex scenario.
- **Evaluate:** Make judgments about the value of proposed ideas, solutions, etc., by comparing the proposal to specific criteria or standards.
- **Create:** Put parts or elements together in such a way as to reveal a pattern or structure not clearly there before; identify which data or information from a complex set is appropriate to examine further or from which supported conclusions can be drawn.

One way to assess competency is to assign KSAs to proficiency level categories within each competency. Examples of proficiency levels include the INCOSE competency model, with proficiency levels of: awareness, supervised practitioner, practitioner, and expert (INCOSE 2010). The Academy of Program/Project & Engineering Leadership (APPEL) competency model includes the levels: participate, apply, manage, and guide, respectively (Menrad and Lawson 2008). The U.S. National Aeronautics and Space Administration (NASA), as part of the APPEL (APPEL 2009), has also defined proficiency levels: technical engineer/project team member, subsystem lead/manager, project manager/project systems engineer, and program manager/program systems engineer. The Defense Civilian Personnel Advisory Service (DCPAS) defines a 5-tier framework to indicate the degree to which employees perform competencies as awareness, basic, intermediate, advanced, and expert.

The KSAs defined in the lower levels of the cognitive domain (remember, understand) are typically foundational, and involve demonstration of basic knowledge. The higher levels (apply, analyze, evaluate, and create) reflect higher cognitive ability. Cognitive and affective processes within Bloom's taxonomy refer to levels of observable actions that indicate learning is occurring (Whitcomb et al. 2015). The Bloom's domain levels should not be used exclusively to determine the proficiency levels required for attainment or assessment of a competency. Higher level cognitive capabilities belong across proficiency levels, and should be used as appropriate to the KSA involved. These higher-level terms infer some observable action or outcome, so the context for assessing the attainment of the KSA, or a group of KSAs, related to a competency needs to be defined. For example, applying SE methods can be accomplished on simple subsystems or systems and so perhaps belong in a lower proficiency level such as supervised practitioner. Applying SE methods to complex enterprise or systems of systems, may belong in the practitioner or even the expert level. The determination of what proficiency level is desired for each KSA is determined by the organization and may vary among different organizations.

Quality of Competency Assessment

When using application as a measure of competency, it is important to have a measure of *goodness*. If someone is applying a competency in an exceptionally complex situation, they may not necessarily be successful in this application. An individual may be *managing and guiding*, but this is only helpful to the organization if it is being done well. In addition, an individual might be fully proficient in a particular competency, but not be given an opportunity to use that competency; for this reason, it is important to understand the context in which these competencies are being assessed.

Individual SE Competency versus Performance

Even when an individual is highly proficient in an SE competency, context may preclude exemplary performance of that competency. For example, an individual with high competency in risk management may be embedded in a team or an organization which ignores that talent, whether because of flawed procedures or some other reason. Developing individual competencies is not enough to ensure exemplary SE performance.

When SE roles are clearly defined, performance assessment at least has a chance to be objective. However, since teams are most often tasked with accomplishing the SE tasks on a project, it is the team's performance which ends up being assessed. (See Team Capability). The final execution and performance of SE is a function of competency, capability, and capacity. (See Enabling Teams and Enabling Businesses and Enterprises.)

References

Works Cited

- Academy of Program/Project & Engineering Leadership (APPEL). 2009. *NASA's Systems Engineering Competencies*. Washington, DC, USA: U.S. National Aeronautics and Space Administration. Available at: https://www.nasa.gov/offices/oc/e/appe/pm-development/pm_se_competency_framework.html.
- Bloom, B.S. 1984. *Taxonomy of Educational Objectives*. New York, NY, USA: Longman.
- Holt, J., and S. Perry. 2011. *A Pragmatic Guide to Competency, Tools, Frameworks, and Assessment*. Swindon, UK: BCS, The Chartered Institute for IT.
- INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2010-003.
- Menrad, R. and H. Lawson. 2008. "Development of a NASA integrated technical workforce career development model entitled: Requisite occupation competencies and knowledge – The ROCK." Paper presented at the 59th International Astronautical Congress (IAC). 29 September-3 October 2008. Glasgow, Scotland.
- MITRE. 2007. *Enterprise Architecting for Enterprise Systems Engineering*. Warrendale, PA, USA: SEPO Collaborations, SAE International.
- Whitcomb, C., J. Delgado, R. Khan, J. Alexander, C. White, D. Grambow, P. Walter. 2015. "The Department of the Navy systems engineering career competency model." Proceedings of the Twelfth Annual Acquisition Research Symposium. Naval Postgraduate School, Monterey, CA.

Primary References

- Academy of Program/Project & Engineering Leadership (APPEL). 2009. *NASA's Systems Engineering Competencies*. Washington, DC, USA: U.S. National Aeronautics and Space Administration (NASA). Accessed on May 2, 2014. Available at <https://appe.nasa.gov/career-resources/project-management-and-systems-engineering-competency-model/>.
- INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2010-003.

Additional References

- Holt, J., and S. Perry. 2011. *A Pragmatic Guide to Competency: Tools, Frameworks and Assessment*. Swindon, UK: British Computer Society.

Developing Individuals

- Lead Author:
- Heidi Davidz

Developing each individual's systems engineering (SE) competencies is a key aspect of enabling individuals. The goal may be to develop competency in a broad range of SE competencies or a single aspect of SE, and it is important to know exactly which SE competencies are desired. This article describes strategies to develop SE competencies in individuals.

Closing Competency Gaps

Delivering excellent systems that fulfill customer needs is the primary goal of the organization. Developing the "capability to deliver such systems is a secondary goal, and while necessary, is not sufficient. To attain both of these goals, the organization must assess itself and effect a strategy to identify and close competency gaps.

To identify competency gaps, an organization may take two basic steps:

1. Listing desired competencies, as discussed in Roles and Competencies; and
2. Assessing the competencies of individual systems engineers, as discussed in Assessing Individuals.

Models useful for listing competencies include the International Council on Systems Engineering (INCOSE) United Kingdom Advisory Board model (Cowper et al. 2005; INCOSE 2010), the ENG Competency Model (DAU 2013), and the Academy of Program/Project & Engineering Leadership (APPEL 2009) model (Menrad and Lawson 2008).

Once the organization knows the SE competencies it needs to develop to close the competency gaps it has identified, it may choose from the several methods (Davidz and Martin 2011) outlined in the table below.

Table 1. SE Competency Development Framework. (SEBoK Original)

Goal	Objective	Method
PRIMARY GOAL = Delivery of excellent systems to fulfill customer needs	Focus on successful performance outcome	Corporate initiatives
	Focus on performance of project team	Team coaching of project team for performance enhancement
SECONDARY GOAL = Competency to deliver excellent systems to fulfill customer needs	Develop individual competency	Training courses
		Job rotation
		Mentoring
		Hands-on experience
		Develop a few hand-picked individuals
		University educational degree program
		Customized educational program
		Combination program - education, training, job rotation, mentoring, hands-on experience
		Course certificate program

	Ensure individual competency through certification	Certification program
	Filter those working in systems roles	Use individual characteristics to select employees for systems roles
	Ensure organizational competency through certification	ISO 9000
	Develop organizational systems competency through processes	Process improvement using an established framework
		Concept maps to identify the thought processes of senior systems engineers
		Standardize systems policies and procedures for consistency
		Systems engineering web portal
		Systems knowledge management repository
		On-call organizational experts
		Rotating professor who works at company part-time and is at university part-time

System Delivery

Some organizations mount initiatives which focus directly on successful system delivery. Others focus on project team performance, in some cases by offering coaching, as a means to ensure successful system delivery.

One example of the latter approach is the performance enhancement service of the US National Aeronautics and Space Administration (NASA) Academy of Program/Project & Engineering Leadership (APPEL), which assesses team performance and then offers developmental interventions with coaching (NASA 2010).

Organizations pursue multiple paths towards developing the capability to deliver excellent systems, including:

- developing the competency of individuals;
- developing the competency of the organization through processes (Davidz and Maier 2007); and
- putting measures in place to verify the efficacy of the selected methods.

Individual Competency

An organization may choose a combination of methods to develop individual systems competency. General Electric's Edison Engineering Development Program (GE 2010) and Lockheed Martin's Leadership Development Programs (Lockheed Martin 2010) are examples among the many combination programs offered within companies.

Whether or not the program is specifically oriented to develop systems skills, the breadth of technical training and experience, coupled with business training, can produce a rich understanding of systems for the participant. Furthermore, new combination programs can be designed to develop specific systems-oriented skills for an organization.

Methods for developing individual competency include:

- **classroom or online training courses**, a traditional choice for knowledge transfer and skill acquisition. Here, an instructor directs a classroom of participants. The method of instruction may vary from a lecture format to case study work to hands-on exercises. The impact and effectiveness of this method varies considerably based on the skill of the instructor, the effort of the participants, the presentation of the material, the course content, the quality of the course design process, and the matching of the course material to organizational needs. These types of interventions may also be given online. Squires (2011) investigates the relationship between online pedagogy and

student perceived learning of SE competencies.

- **job rotation**, where a participant rotates through a series of work assignments that cut across different aspects of the organization to gain broad experience in a relatively short time.
- **mentoring**, where a more experienced individual is paired with a protégé in a developmental relationship. Many organizations use mentoring, whose impact and effectiveness vary considerably. Success factors are the tenable pairing of individuals, and the provision of adequate time for mentoring.
- **hands-on experience**, where organizations provide for their engineers to get hands-on experience that they would otherwise lack. A research study by Davidz on enablers and barriers to the development of systems thinking showed that systems thinking is developed primarily by experiential learning (Davidz 2006; Davidz and Nightingale 2008, 1-14). As an example, some individuals found that working in a job that dealt with the full system, such as working in an integration and test environment, enabled development of systems thinking.
- **selecting individuals** who appear to have high potential and focusing on their development. Hand-selection may or may not be accompanied by the other identified methods.
- **formal education**, such as a university degree program. A growing number of SE degree programs are offered worldwide (Lasfer and Pyster 2011). Companies have also worked with local universities to set up customized educational programs for their employees. The company benefits because it can tailor the educational program to the unique needs of its business. In a certificate program, individuals receive a certificate for taking a specific set of courses, either at a university or as provided by the company. There are a growing number of certificate programs for developing systems competency.

Individual Certification

Organizations may seek to boost individual systems competency through certification programs. These can combine work experience, educational background, and training classes. Certifications are offered by local, national, and international professional bodies.

SE organizations may encourage employees to seek certification from the International Council on Systems Engineering (INCOSE 2011) or may use this type of certification as a filter (see **Filters**, below). In addition, many companies have developed their own internal certification measures. For example, the Aerospace Corporation has an Aerospace Systems Architecting and Engineering Certificate Program (ASAECP). (Gardner 2007.)

Filters

Another approach to developing individual competency is to select employees for systems roles based on certain characteristics, or filters. Before using a list of characteristics for filtering, an organization should critically examine:

1. how the list of individual characteristics was determined, and
2. how the characteristics identified enable the performance of a systems job.

Characteristics used as filters should:

- enable one to perform a systems job,
- be viewed as important to perform a systems job, or
- be necessary to perform a systems job.

A necessary characteristic is much stronger than an enabling one, and before filtering for certain traits, it is important to understand whether the characteristic is an enabler or a necessity.

Finally, it is important to understand the extent to which findings are generally applicable, since a list of characteristics that determine success in one organization may not be generalizable to another organization.

Organizational Capability

Once an organization has determined which SE capabilities are mission critical (see Deciding on Desired Systems Engineering Capabilities within Businesses and Enterprises), there are many different ways in which an organization can seek to develop or improve these capabilities. Some approaches seen in the literature include the following:

- Organizations may choose to develop organizational systems capability through processes. One method organizations may choose is to pursue process improvement using an established framework. An example is the Capability Maturity Model® Integration (CMMI) process improvement approach (SEI 2010, 1).
- Concept maps - graphical representations of engineering thought processes - have been shown to be an effective method of transferring knowledge from senior engineering personnel to junior engineering personnel (Kramer 2007, 26-29; Kramer 2005). These maps may provide a mechanism for increasing knowledge of the systems engineering population of an organization.
- An organization may also choose to develop organizational systems competencies by standardizing systems policies and procedures. An example from NASA is their *NASA Systems Engineering Processes and Requirements* (NASA 2007).
- Some organizations use a web portal to store and organize applicable systems engineering knowledge and processes, which assists in developing organizational systems competency. An example is the Mission Assurance Portal for the Aerospace Corporation (Roberts et al. 2007, 10-13).
- Another approach being considered in the community is the development of a rotating professor role, where the person would work at the company and then at a university to strengthen the link between academia and industry.
- Another approach is to alter organizational design to foster and mature a desired competency. For example, an organization that identifies competency in the area of reliability as critical to its SE success may develop a reliability group, which will help foster growth and improvement in reliability competencies.

Organizational Certification

Certification at the organizational level also exists and can be a means for ensuring competency. ISO certification is one example (ISO 2010). Before taking this approach, the organization should verify that the capabilities required by the certification are indeed the systems capabilities it seeks. For more on determining appropriate organizational capabilities, see Deciding on Desired Systems Engineering Capabilities within Businesses and Enterprises.

Repositioning the Product Life Cycle

An organization may also choose to reposition its product life cycle philosophy to maintain system competency. For example, NASA has done this with its APPEL program (APPEL 2009).

Since the systems competencies of individuals are primarily developed through experiential learning, providing experiential learning opportunities is critical. Shortening the product life cycle is one way to ensure that individuals acquire the full range of desired competency sooner.

Maintaining Competency Plans

An organization that has developed an SE competency plan should consider how to maintain it. How, and how often, will the competency plan be re-examined and updated? The maintenance process should account for the ongoing evolution of global contexts, business strategies, and the SEBoK. The process for assessing competencies and taking action to improve them must be part of the normal operations of the organization and should occur periodically.

References

Works Cited

- Academy of Program/Project & Engineering Leadership (APPEL). 2009. NASA's Systems Engineering Competencies. Washington, D.C., USA: U.S. National Aeronautics and Space Association. Accessed on September 15, 2011. Available at https://www.nasa.gov/offices/oc/appe/pm-development/pm_se_competency_framework.html.
- Cowper, D., S. Bennison, R. Allen-Shalless, K. Barnwell, S. Brown, A. El Fatatry, J. Hooper, S. Hudson, L. Oliver, and A. Smith. 2005. *Systems Engineering Core Competencies Framework*. Folkestone, UK: International Council on Systems Engineering (INCOSE) UK Advisory Board (UKAB).
- Davidz, H.L. and J. Martin. 2011. "Defining a strategy for development of systems capability in the workforce". *Systems Engineering*. 14(2): 141-143.
- Davidz, H.L. and M.W. Maier. 2007. "An integrated approach to developing systems professionals." Paper presented at the 17th Annual International Council on Systems Engineering (INCOSE) International Symposium, 24-28 June 2007. San Diego, CA, USA.
- Davidz, H.L., and D. Nightingale. 2008. "Enabling systems thinking to accelerate the development of senior systems engineers." *Systems Engineering*. 11(1): 1-14.
- Davidz, H.L. 2006. *Enabling Systems Thinking to Accelerate the Development of Senior Systems Engineers*. Dissertation. Massachusetts Institute of Technology (MIT), Cambridge, MA, USA.
- Gardner, B. 2007. "A corporate approach to national security space education." *Crosslink*, the Aerospace Corporation Magazine of Advances in Aerospace Technology. 8(1) (Spring 2007):10-5. Accessed April 23, 2013. Available at: <https://aerospace.wpengine.netdna-cdn.com/wp-content/uploads/crosslink/V8N1.pdf>.
- GE. 2010. *Edison Engineering Development Program (EEDP) in General Electric*. Accessed on September 15, 2011. Available at https://www.gecareers.com/GECAREERS/jsp/us/studentOpportunities/leadershipPrograms/eng_program_guide.jsp.
- INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2010-003.
- INCOSE. 2011. "Systems Engineering Professional Certification." In *International Council on Systems Engineering* online. Accessed April 13, 2015. Available at: <https://www.incose.org/certification/>.
- Kramer, M.J. 2007. "Can concept maps bridge the engineering gap?" *Crosslink*, the Aerospace Corporation Magazine of Advances in Aerospace Technology. 8(1) (Spring 2007): 26-9. Accessed April 23, 2013. Available at: <https://aerospace.wpengine.netdna-cdn.com/wp-content/uploads/crosslink/V8N1.pdf>.
- Kramer, M.J. 2005. *Using Concept Maps for Knowledge Acquisition in Satellite Design: Translating 'Statement of Requirements on Orbit' to 'Design Requirements'*. Dissertation. Ft. Lauderdale, FL, USA: Graduate School of Computer and Information Sciences, Nova Southeastern University.
- Lasfer, K. and A. Pyster. 2011. "The growth of systems engineering graduate programs in the United States." Paper presented at Conference on Systems Engineering Research, 15-16 April 2011. Los Angeles, CA, USA.

Lockheed Martin. 2010. *Training and Leadership Development Programs for College Applicants in Lockheed Martin Corporation*. Bethesda, MD, USA. Accessed on August 30, 2012. Available at <https://www.lockheedmartinjobs.com/leadership-development-program.asp>.

NASA. 2010. *Academy of Program/Project & Engineering Leadership (APPEL): Project Life Cycle Support in U.S. National Aeronautics and Space Administration (NASA)*. Washington, D.C., USA: U.S. National Air and Space Administration (NASA). Accessed on September 15, 2011. Available at <https://www.nasa.gov/offices/oce/appeel/performance/lifecycle/161.html>.

NASA. 2007. *NASA Procedural Requirements: NASA Systems Engineering Processes and Requirements*. Washington, D.C., USA: U.S. National Aeronautics and Space Administration (NASA). NPR 7123.1A.

Roberts, J., B. Simpson, and S. Guarro. 2007. "A mission assurance toolbox." *Crosslink*, the Aerospace Corporation Magazine of Advances in Aerospace Technology. 8(2) (Fall 2007): 10-13.

SEI. 2007. *Capability Maturity Model Integrated (CMMI) for Development*, version 1.2, Measurement and Analysis Process Area. Pittsburgh, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

Squires, A. 2011. *Investigating the Relationship between Online Pedagogy and Student Perceived Learning of Systems Engineering Competencies*. Dissertation. Stevens Institute of Technology, Hoboken, NJ, USA.

Primary References

Academy of Program/Project & Engineering Leadership (APPEL). 2009. *NASA's Systems Engineering Competencies*. Washington, D.C., USA: U.S. National Aeronautics and Space Administration (NASA). Accessed on May 2, 2014. Available at <https://appeel.nasa.gov/career-resources/project-management-and-systems-engineering-competency-model/>.

DAU. 2013. *ENG Competency Model*, 12 June 2013 version. in Defense Acquisition University (DAU)/U.S. Department of Defense Database Online. Accessed on September 23, 2014. Available at <https://acc.dau.mil/CommunityBrowser.aspx?id=657526&lang=en-US>.

Davidz, H.L. and J. Martin. 2011. "Defining a strategy for development of systems capability in the workforce". *Systems Engineering*. 14(2): 141-143.

INCOSE. 2010. *Systems Engineering Competencies Framework 2010-0205*. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2010-003.

Additional References

None.

Ethical Behavior

- Lead Authors:
- Dick Fairley and Chuck Calvano
- Contributing Authors:
- Scott Jackson, Heidi Davidz, Alice Squires, and Art Pyster

If the competency of the systems engineer is a matter of KSAA—knowledge, skills, abilities, and attitudes—then the word “attitudes” must have an ethical dimension. The ethical framework that guides the SE's actions ensures that the SE ultimately does good and not harm. Ethical standards apply both to individuals and to organizations. This section discusses the moral foundations of ethics, and the elements of ethical conduct that are especially relevant to systems engineering.

Ethics and Morals in Systems Engineering

Like other people, systems engineers have morals: guiding personal thoughts and feelings about what is right and wrong. All of us also share, with other members of various communities to which we belong, ethics: standards that say what conduct is appropriate and what is not (Whitbeck 2007).

Morals are part of a person's character, the result of upbringing, culture, and other environmental influences. **Ethics** apply morals within the frame of a social system, which could be professional, business, academic, recreational, cultural, political, religious, or even familial. While a person's moral code is usually considered immutable, one's ethics may need to account for new situations as one's profession or role in life changes. Tensions may exist between an engineer's responsibilities to society and those to the customer, the employer, or even the family, resulting in ethical dilemmas, and creating situations where morals come into play.

There is no shortage of discussion on ethics. Ethical codes are promulgated by professional and other organizations. **Professions** here refers to occupations that require learning and advanced knowledge and which safeguard or promote the well-being of others and of society as a whole.

Systems engineers have two ethical responsibilities over and above those of most other engineering professions:

- While engineers in general use their professional skills to address customer needs and desires, systems engineering (SE) helps *determine* those needs and desires in the course of defining and managing requirements. SEs have an obligation to ensure that problem or program definition is influenced solely by the interests of the customer or user, not by those of the systems engineer or the engineer's firm.
- Systems engineers typically integrate and oversee the work of others whose expertise differs from their own. This makes the obligation to widen one's understanding and to seek competent advice from other professionals more acute in SE than in other disciplines.

Caroline Whitbeck's *Ethics in Engineering Practice and Research* explains what ethical behavior means for engineering professionals. Like most books on ethics, this one starts by clarifying the differences between ethics and morals, which can seem somewhat obscure at times (Whitbeck 2007).

A sampling of areas where ethics figure in the engineering of modern systems are described below.

Data Confidentiality and Security, Surveillance, and Privacy

Privacy, confidentiality, and security in systems which touch Personally Identifiable Information (PII) have an ethical dimension for the systems engineers responsible for developing those systems.

Laws and Regulations

Systems are typically developed in societies, sometimes involving international communities, which have laws concerning contracts, intellectual property, freedom of information, and employment. The requirements and restraints of those laws govern the practice of the systems engineer, who must be aware of the laws and must consider their implications for the partnerships that system development entails.

Whether or not they are stated in the system requirements document or provided by the customer, laws and regulations do, in fact, impose system requirements. SEs are responsible for knowing and applying relevant laws and regulations. This means recognizing other people's proprietary interests by safeguarding their intellectual property (trade secrets, copyrights, trademarks, and patents), and giving them credit for performing work and making innovations.

Cultural Issues

Since systems engineers develop and maintain products used by humans globally, it is important that they understand the historical and cultural aspects of their profession and the related context in which their products will be used. System engineers need to be aware of societal diversity and act without prejudice or discrimination.

Ethical Considerations in the Systems Engineering Method

Naturally, SE approaches to meeting customer needs must integrate SE ethics.

Codes of Ethics and Professional Conduct

Codes of ethics are promulgated by the IEEE (IEEE 2009), the National Society of Professional Engineers (NSPE) (NSPE 2007), the International Council on Systems Engineering (INCOSE 2006) and other engineering organizations.

The INCOSE Code of Ethics enunciates fundamental ethical principles like honesty, impartiality, integrity, keeping abreast of knowledge, striving to increase competence, and supporting educational and professional organizations. Based on these principles, the code identifies the systems engineer's fundamental duties to society and the public, and the rules of practice that systems engineers should follow to fulfill those duties.

According to the INCOSE Code of Ethics, it is the systems engineer's duty to:

- guard the public interest and protect the environment, safety, and welfare of those affected by engineering activities and technological artifacts;
 - accept responsibility for one's actions and engineering results, including being open to ethical scrutiny and assessment;
 - proactively mitigate unsafe practice;
 - manage risk using knowledge gained by applying a whole-system viewpoint and understanding of systemic interfaces; and
 - promote the understanding, implementation, and acceptance of prudent SE measures.
-

Enforcing Ethics

Many organizations enforce ethics internally by means of ethics policies. These policies typically include rules such as the following:

- There shall be no exchange of favors between anyone in the organization and entities with which it does business, such as suppliers, customers, or regulatory agencies.
- Product information, for example, test data, shall be reported accurately and completely to the contracting agency.
- There shall be no conflict of interest between the organization and entities with which it does business.

Favors can consist of providing money, reimbursement of travel or entertainment expenses, other items of equivalent value, or inappropriate job offers. Conflict of interest can arise when the personal or professional financial interests or organizational ties of an engineer are potentially at odds with the best interests of the customer or the engineer's employer. Since conflict of interest and other ethical transgressions can be hard to define, care must be taken to design ethics policies that are observable and enforceable. Internal audit functions or external regulatory agencies may enforce ethical rules at the individual, team, organizational, or enterprise level. Punishment for violating ethics policies can include termination and other disciplinary actions.

Unlike self-employed physicians who may choose to not do something specific, many systems engineers are individuals employed by organizations. Depending on the organizational context, an issue in conflict with the company might result in giving up the job. This may result in additional ethical considerations.

Responsibility to Society

Engineers who create products and services for use in society have an obligation to serve the public good. Additionally, the IEEE Code of Ethics states that engineers have an obligation to foster the professional development and ethical integrity of colleagues (IEEE 2015). Because of the criticality and scope of many systems, systems engineers, operating in teams within projects and on behalf of the public in delivery of products, have special responsibility. Poorly designed systems or services can have calamitous effects on society. The INCOSE Code of Ethics asserts the responsibility of systems engineers to "guard the public interest and protect the environment, safety, and welfare of those affected by engineering activities and technological artifacts" (INCOSE 2006).

References

Works Cited

- IEEE. 2009. *IEEE Code of Ethics*. in IEEE [database online]. Accessed September 7, 2012. Available: <https://www.ieee.org/ethics>.
- IEEE. 2015. *IEEE Code of Ethics*. Accessed April 6, 2015. Available: <https://www.ieee.org/about/corporate/governance/p7-8.html>.
- INCOSE. 2006. *INCOSE Code of Ethics*. In International Council on Systems Engineering [database online]. Accessed April 13, 2015. Available: <https://www.incose.org/about/leadershiporganization/codeofethics>.
- NSPE. 2007. *NSPE Code of Ethics for Engineers*. Alexandria, VA, USA: National Society of Professional Engineers. NSPE publication #1102. Accessed on September 15, 2011. Available: <https://www.nspe.org/resources/ethics>.
- Whitbeck, C. 2007. *Ethics in Engineering Practice and Research*. New York, NY, USA: Cambridge University Press.

Primary References

Whitbeck, C. 2007. *Ethics in Engineering Practice and Research*. New York, NY, USA: Cambridge University Press.

Additional References

National Institute for Ethics in Engineering. "National Institute for Engineering Ethics." In the Murdough Center for Engineering Professionalism. Hosted by Texas Technical University. Accessed on September 15, 2011. Available at <https://www.murdough.ttu.edu/pd.cfm?pt=NIEE>.

OnlineEthics.org. "Online Ethics Center (OEC)." Accessed September 8, 2011. Available at: <https://www.onlineethics.org/>.

Martin, M. and R. Schinzinger. 2004. *Ethics in Engineering*, 4th ed. New York, NY, USA: McGraw-Hill.

Penn State. "Ethics: Books." In Penn State College of Engineering. Accessed September 8, 2011. Available at: <https://www.engr.psu.edu/ethics/books.asp>.

Smith, J.H. (ed). 2008. *Engineering Ethics – Concepts, Viewpoint, Cases and Codes*. National Institute for Engineering Ethics, Texas Technical University.

Part 6: Related Disciplines

Related Disciplines

Contents of this Part

- Systems Engineering and Environmental Engineering (Paul Phister and David Olwell)
- Systems Engineering and Geospatial/Geodetic Engineering (Ulrich Lenk)
- Systems Engineering and Industrial Engineering (Gregory S. Parnell) (C. Robert Kenley and Eric Specking, Ed Pohl)
- Systems Engineering and Project Management (Dick Fairley) (Richard Turner and Alice Squires)
- Systems Engineering and Software Engineering (Dick Fairley and Tom Hilburn) (Ray Madachy and Alice Squires)
- Systems Engineering and Mechanical Engineering (Leigh McCue and John Shortle) (Art Pyster)
- Systems Engineering and Enterprise IT (Chuck Walrad and Rich Hilliard)
- Systems Engineering and Quality Attributes (Art Pyster and Dave Olwell) (Richard Turner, Dick Fairley, Scott Jackson, and Alice Squires)
- Lead Author:
- Caitlyn Singam

One of the most fundamental tenets of systems engineering (SE) is that of approaching systems from an integrated, holistic perspective (INCOSE 2023), with an eye towards how sets of interconnected components and their surrounding environments interface and interact with each other to shape the nature, characteristics, and dynamics of systems of interest (SoIs). In a similar fashion, it is possible to regard systems engineering itself as a system, which interacts and intersects with adjoining and related disciplines, communities of practice, and areas of study. These related fields form the systems context in which the field of systems engineering exists, and play a critical role in not only facilitating the effective use of systems engineering in various application areas, but also in shaping the current nature and future evolution of systems engineering as drivers of the overall operational environment in which systems engineering exists (Singam 2022a). Part 6 of the (SEBoK) provides the reader with an overview of many of these related disciplines, discussion on how these disciplines help enrich and enliven systems engineering, and vice versa.

Context

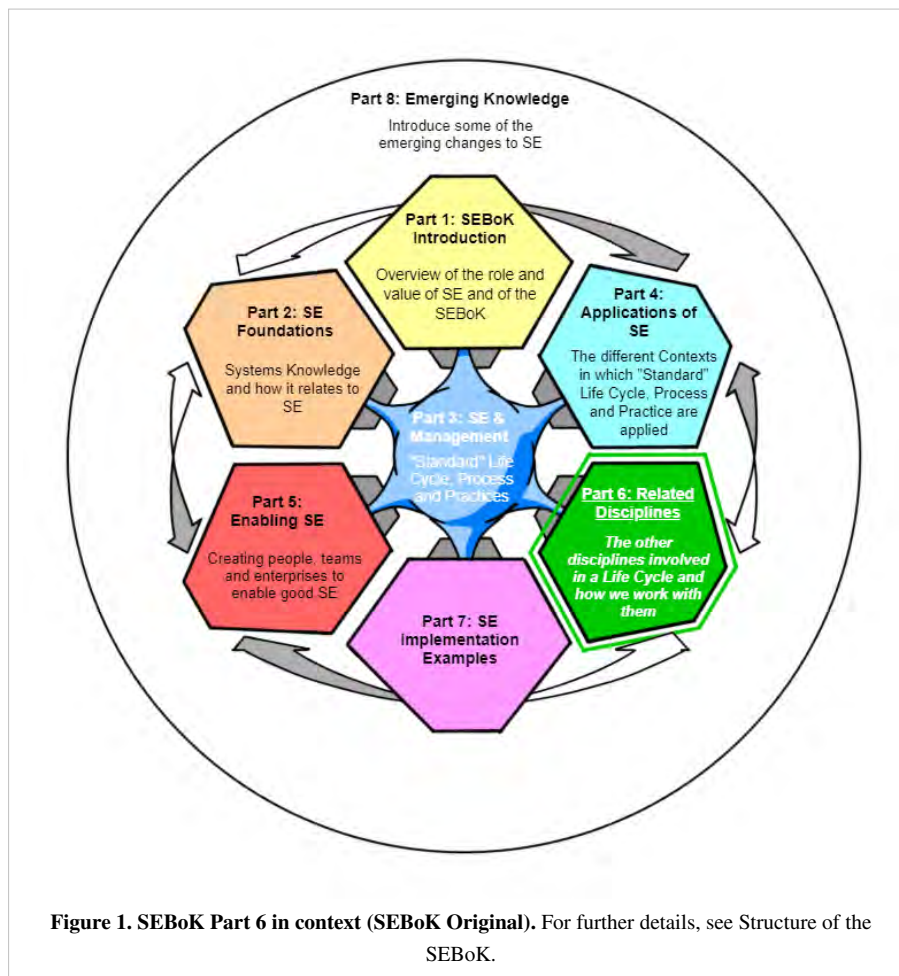
Systems engineering (SE) is a fundamentally transdisciplinary endeavor (INCOSE 2019): its principles and practices are sufficiently abstract and generic as to be readily applicable to any system, regardless of application area or associated discipline(s) relevant to the system of interest (Kossiakoff et al. 2011). The flexibility and versatility of the SE toolkit is part of what makes it such a valuable asset on projects, especially those involving large or complex systems which require the integration of various disparate elements and technical disciplines into a single, unified system (INCOSE 2023, Elm et. al. 2008). In order to realize that versatility, though, SE practitioners need to have a wide range of technical knowledge that extends beyond that of "pure" SE and across the various disciplines relative to the system(s) of interest. A systems engineer who has responsibility for overseeing the interface between a mechanical subsystem and an electrical subsystem in a biomedical device, for instance, would at minimum need knowledge of mechanical engineering and electrical engineering *topics* in order to perform a systems engineering task such as setting interface requirements. However, in the context of a biomedical device for human use, familiarity with concepts in biology, safety engineering, electromagnetic interference, human-systems integration,

and law/policy - just to name a few - are likely to also become relevant in order for the systems engineer to appropriately communicate and integrate information from across the project, as well as ensure that the interface is fit-for-purpose and within acceptable parameters. It thus behooves the well-prepared systems engineer to maintain familiarity with a broad selection of related disciplines.

Purpose and Scope

Part 6 of the SEBoK intends to aid systems practitioners and other individuals with an interest in SE in augmenting their technical knowledge of subjects relevant to systems engineering practice, and providing a foundation for independent exploration of associated topics via relevant resources.

Part 6's purview encompasses the many fields and topics which enrich the discipline of SE, inclusive of both commonly-discussed disciplines such as those classified under the science, technology, engineering, and mathematics (STEM) quartet, as well as oft-overlooked disciplines of relevance in the arts and humanities. Part 6 is therefore arguably the most expansive section of the SEBoK in terms of potential scope, as it extends beyond the boundaries of formal SE and SE application areas, and across the breadth of academic and practical knowledge. Even so, the disciplines discussed in this section are only a subset of the full expanse of those which relate to systems engineering: it is, after all, neigh impossible to find a topic or discipline which does not involve or reckon with system(s), and thus systems engineering, in some way (Singam 2022b). Consequently, rather than attempting a full census of the adjacent or related knowledge areas (KAs) that abut the edges of formal systems engineering, Part 6 instead seeks to proffer a curated smörgåsbord of the many related disciplines which bear relevance to systems engineering, with a focus on KAs that are most likely to be of relevance to a diverse, multi-disciplinary audience of systems-minded professionals, learners, and educators.



Knowledge Areas in Part 6

The KAs covered in Part 6 are all distinct areas of study or practice (i.e., disciplines which are commonly recognized as separate from systems engineering) that meet one or more of the following descriptors:

1. disciplines which are independent from, but overlap with, formal SE practice and/or the core SE body of knowledge (e.g., project management);
2. specialized disciplines focused on in-depth exploration of specific topics that abut or extend beyond the scope of the core/non-specialist SE body of knowledge (e.g., quality engineering);
3. specialist disciplines which are commonly used to describe or govern the characteristics, dynamics, or life cycle of the composite elements of interdisciplinary or multi-disciplinary systems (e.g., physics, biology, mechanical engineering, software engineering, etc.);
4. disciplines which are frequently relevant to the effective and/or ethical practice of SE in a commonly encountered system context (e.g., environmental engineering, information technology (IT) for enterprise systems, law);
5. specialized disciplines which frequently utilize systems engineering methodologies or have a substantial systems engineering/systems science community of practice (e.g., geospatial engineering), and which can provide new generalizable insights into the improvement and evolution of systems engineering as a field;
6. other specialized disciplines which are of substantial interest in non-specialist contexts.

Current Content

At present, Part 6 is organized into the following sub-sections, each representing a key KA:

- Systems Engineering and Environmental Engineering
- Systems Engineering and Geospatial/Geodetic Engineering
- Systems Engineering and Industrial Engineering
- Systems Engineering and Project Management
 - *Other sub-topics:* project structure and governance; procurement and acquisition; portfolio management
- Systems Engineering and Software Engineering
- Systems Engineering and Mechanical Engineering
- Systems Engineering and Enterprise IT
- Systems Engineering and Quality Attributes (This encompasses quality engineering, but is also the home of the disciplines focused on characteristics of systems, sometimes referred to as "-ilities".)
 - *Other subtopics:* human-systems integration (HSI); manufacturability and producibility; adaptability; affordability; hardware assurance; reliability/availability/maintainability; resilience; resistance to electromagnetic interference; safety; security

As reflected by the sub-section titles within Part 6 ("Systems Engineering and...."), each main KA is focused on the intersection between systems engineering and a given topic, and contains at least one article discussing aspects of that KA that would be of import to a systems engineer from outside that particular specialty. Some of the broader KAs, such as those on project management and quality attributes, also include articles on KA-relevant sub-topics alongside the KA overview articles, as per the categorization listed above.

Future Content

Astute readers may note that pre-existing content in Part 6 is almost exclusively focused on areas of specialist engineering; as Part 6's content continues to evolve, it is planned that the KAs covered in this section will be updated to better reflect the current range of SE-adjacent knowledge and practice across various related disciplines, in accordance with broadened interest from both academia and industry (Han et. al. 2023) in exploring interdisciplinary and cross-disciplinary KAs. In particular, it is planned that future versions of Part 6 will include content on related disciplines that are often overlooked in overviews of SE-relevant subject matter, such as topics in the arts and

humanities.

References

Works Cited

- Elm, J. P., D.R. Goldenson, K. El Emam, N. Donatelli, and A. Neisa. 2008. *A Survey of Systems Engineering Effectiveness-Initial Results* (with Detailed Survey Response Data). Pittsburgh, PA, USA: Software Engineering Institute, CMU/SEI-2008-SR-034. December 2008.
- Han, Siqi, Jack LaViolette, Chad Borkenhagen, William McAllister, and Peter S. Bearman. 2023. "Interdisciplinary College Curriculum and Its Labor Market Implications." *Proceedings of the National Academy of Sciences of the United States of America* 120 (43): e2221915120. <https://doi.org/10.1073/pnas.2221915120>.
- INCOSE. 2019. *Systems Engineering and System Definitions*, version 1.0. San Diego, CA, USA: INCOSE. INCOSE-TP-2020-002-06.
- INCOSE. 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-119-81429-0.
- ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.
- Kossiakoff, Alexander, William N. Sweet, Samuel J. Seymour, and Steven M. Biemer. 2011. *Systems Engineering Principles and Practice*. John Wiley & Sons.
- Singam, Caitlyn A. K. 2022. "A Critical Analysis of the Systems Engineering Leadership Pipeline: Closing the Gender Gap." In *Emerging Trends in Systems Engineering Leadership: Practical Research from Women Leaders*, edited by Alice F. Squires, Marilee J. Wheaton, and Heather J. Feli, 195–236. Women in Engineering and Science. Cham: Springer International Publishing. https://doi.org/10.1007/978-3-031-08950-3_7.
- Singam, Caitlyn A. K. 2022. "A Vision for Universal and Standardized Access to Systems Competency Education." *INSIGHT* 25 (3): 30–34. <https://doi.org/10.1002/inst.12395>.

Primary References

None.

Additional References

None.

Knowledge Area: Systems Engineering and Environmental Engineering

Systems Engineering and Environmental Engineering

- Lead Authors:
- Paul Phister and David Olwell

Environmental engineering addresses four issues that arise in system design and operation. They are: (1) design for a given operating environment, (2) environmental impact, (3) green design, and (4) compliance with environmental regulations.

Overview

A system is designed for a particular operating environment. Product systems, in particular, routinely consider conditions of temperature and humidity. Depending on the product, other environmental conditions may need to be considered, including UV exposure, radiation, magnetic forces, vibration, and others. The allowable range of these conditions must be specified in the requirements for the system.

Requirements

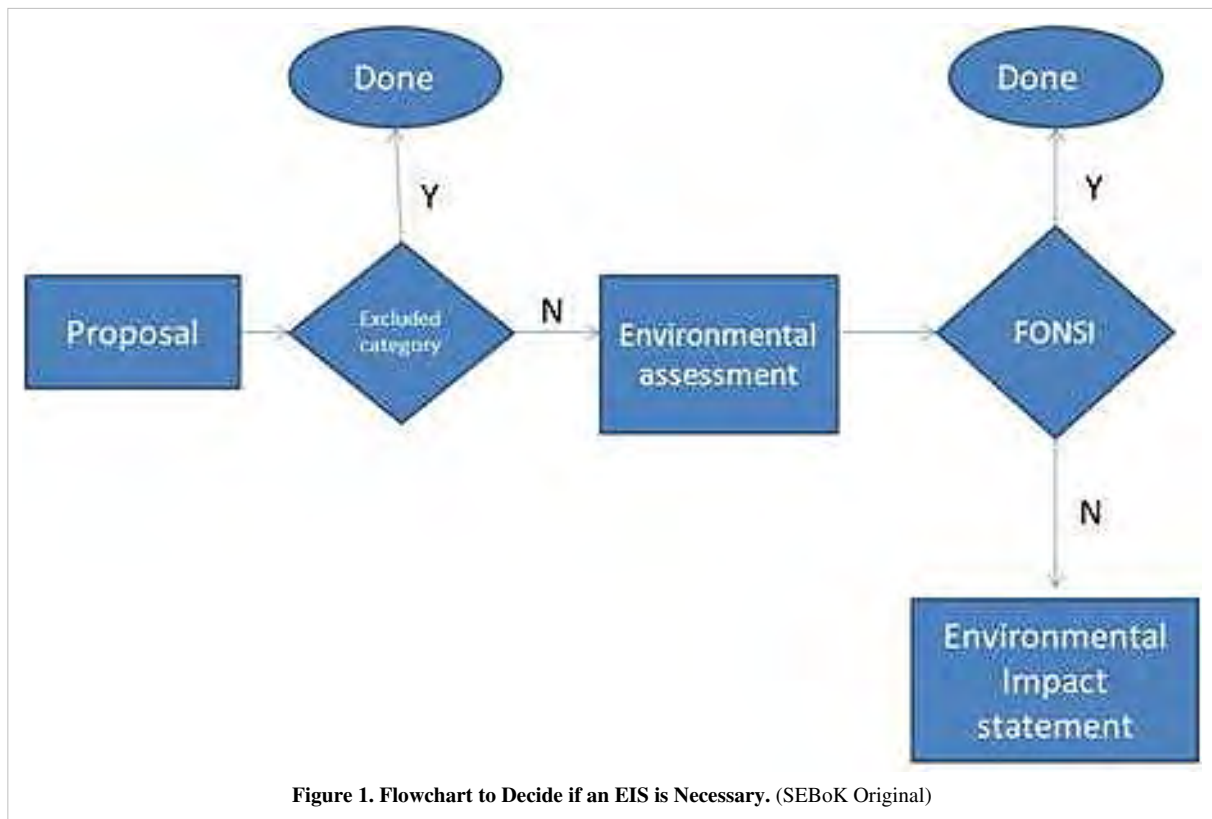
The general principles for writing requirements also apply to specifying the operating environment for a system and its elements. Requirements are often written to require compliance with a set of standards.

Discipline Management

Many countries require assessment of environmental impact of large projects before regulatory approval is given. The assessment is documented in an environmental impact statement (EIS). In the United States, a complex project can require an EIS that greatly adds to the cost, schedule, and risk of the project.

Scope

In the U.S., the process in Figure 1 is followed. A proposal is prepared prior to a project being funded. The regulator examines the proposal. If it falls into an excluded category, no further action is taken. If not, an environmental assessment is made. If that assessment determines a finding of no significant impact (FONSI), no further action is taken. In all other cases, an environmental impact statement is required.



Preparation of an EIS is a resource significant task. Bregman (2000) and Kreske (1996) provide accessible overviews of the process. Lee and Lin (2000) provide a handbook of environmental engineering calculations to aid in the technical submission. Numerous firms offer consulting services.

Legal References

Basic references in the U.S. include the National Environmental Policy Act of 1969 and its implementing regulations (NEPA 1969) and the European Commission directive (EC 1985). State and local regulations can be extensive; Burby and Paterson (1993) discuss improving compliance.

Cost and Schedule Implications

Depending on the scale of the project, the preparation of an EIS can take years and cost millions. For example, the EIS for the Honolulu light rail project took four years and cost \$156M (Hill 2011). While a project may proceed even if the EIS finds a negative impact, opponents to a project may use the EIS process to delay a project. A common tactic is to claim the EIS was not complete in that it omitted some environmental impacts. Eccleston (2000) provides a guide to planning for EIS.

Energy Efficiency

There is a large amount of literature that has been published about design for energy efficiency. Lovins (2010) offers ten design principles. He also provides case studies (Lovins et al. 2011). Intel (2021) provides guidance for improving the energy efficiency of its computer chips. A great deal of information is also available in regard to the efficient design of structures; DOE (2011) provides a good overview.

Increased energy efficiency can significantly reduce total life cycle cost for a system. For example, the Toyota Prius was found to have the lowest life cycle cost for 60,000 miles, three years despite having a higher initial purchase price (Brown 2011).

Carbon Footprint

Increased attention is being paid to the emission of carbon dioxide. BSI British Standards offers a specification for assessing life cycle greenhouse emissions for goods and services (BSI 2011).

Sustainability

Graedel and Allenby (2009), Maydl (2004), Stasinopoulos (2009), Meryman (2004), and Lockton and Harrison (2008) discuss design for sustainability. Sustainability is often discussed in the context of the UN report on Our Common Future (WCED 1987) and the Rio Declaration (UN 1992).

Discipline Relationships

An enterprise must attend to compliance with the various environmental regulations. Dechant et al. (1994) provide the example of a company in which 17% of every sales dollar goes toward compliance activities. They discuss gaining a competitive advantage through better compliance. Gupta (1995) studies how compliance can improve the operations function. Berry (1998) and Nash (2001) discuss methods for environmental management by the enterprise.

Dependencies

ISO14001 sets the standards for organization to comply with environmental regulations. Kwon and Seo (2002) discuss this in a Korean context, and Whitelaw (2004) presents a handbook on implementing ISO14001.

Discipline Standards

Depending on the product being developed, standards may exist for operating conditions. For example, ISO 9241-6 specifies the office environment for a video display terminal. Military equipment may be required to meet MILSTD 810G standard (DoD 2014) in the US, or DEF STAN 00-35 in the UK (MoD 2017).

The U.S. Federal Aviation Administration publishes a list of EIS best practices (FAA 2002).

The U.S. Environmental Protection Agency (EPA) defines green engineering as: the design, commercialization, and use of processes and products, which are feasible and economical, while minimizing (1) generation of pollution at the source and (2) risk to human health and the environment (EPA 2011). Green engineering embraces the concept that decisions to protect human health and the environment can have the greatest impact and cost effectiveness when applied early to the design and development phase of a process or product.

The EPA (2011) offers the following principles of green engineering:

- Engineer processes and products holistically, use systems analysis, and integrate environmental impact assessment tools.
 - Conserve and improve natural ecosystems while protecting human health and well-being.
 - Use life-cycle thinking in all engineering activities.
-

- Ensure that all material and energy inputs and outputs are as inherently safe and benign as possible.
- Minimize depletion of natural resources.
- Strive to prevent waste.
- Develop and apply engineering solutions, while being cognizant of local geography, aspirations, and cultures.
- Create engineering solutions beyond current or dominant technologies; additionally, improve, innovate, and invent (technologies) to achieve sustainability.
- Actively engage communities and stakeholders in development of engineering solutions.

References

Works Cited

- Berry, M. A. "Proactive Corporate Environmental Management: A New Industrial Revolution." *The Academy of Management Executive* 12, no. 2: 38–50. 1998.
- Bregman, J.I. *Environmental Impact Statements*, 2nd ed. Boca Raton, FL: CRC Press. 2000.
- Brown, C. "The Green Fleet Price Tag." *Business Fleet*. 2011. <https://www.businessfleet.com/Article/Story/2011/07/The-Green-Fleet-Price-Tag.aspx>.
- British Standards Institution (BSI). "Specification for the assessment of the life cycle greenhouse gas emissions of goods and service, PAS 2050:2011." British Standards Institution (BSI), London, UK: 2011. <https://shop.bsigroup.com/en/forms/PASs/PAS-2050>.
- BSI. *Specification for the Assessment of the Life Cycle Greenhouse Gas Emissions of Goods and Services: PAS 2050:2011*. London: BSI. 2011. <https://shop.bsigroup.com/en/forms/PASs/PAS-2050> ^[1].
- Burby, R. J., and R. G. Paterson. "Improving Compliance with State Environmental Regulations." *Journal of Policy Analysis and Management* 12, no. 4: 753–772. 1993.
- Dechant, K., B. Altman, R. M. Downing, and T. Keeney. "Environmental Leadership: From Compliance to Competitive Advantage." *Academy of Management Executive* 8, no. 3: 7. 1994.
- Department of Defense (DoD). *Department of Defense Test Method Standard: Environmental Engineering Considerations and Laboratory Tests, MIL-STD-810G Change Notice 1*. Washington, DC: US Army Test and Evaluation Command, US Department of Defense. Accessed May 15, 2022. https://everyspec.com/MIL-STD/MIL-STD-0800-0899/MIL-STD-810G_CHG-1_50560/.2014.
- Eccleston, C. *Environmental Impact Statements: A Comprehensive Guide to Project and Strategic Planning*. New York: Wiley. 2014.
- Environmental Protection Agency (EPA). "Green Engineering." United States Environmental Protection Agency. 2011. <https://www.epa.gov/green-engineering>.
- European Commission (EC). "Council Directive of 27 June 1985 on the Assessment of the Effects of Certain Public and Private Projects on the Environment (85/337/EEC)." 1985. https://www.legislation.gov.uk/eudr/1985/337/pdfs/eudr_19850337_adapted_en.pdf.
- Federal Aviation Administration (FAA). "Best Practices for Environmental Impact Statement (EIS) Management." 2002. https://www.faa.gov/airports/environmental/eis_best_practices/?sect=intro.
- Graedel, T. E., and B. R. Allenby. *Industrial Ecology and Sustainable Engineering*. Upper Saddle River, NJ: Prentice Hall. 2009.
- Gupta, M. C. "Environmental Management and Its Impact on the Operations Function." *International Journal of Operations and Production Management* 15, no. 8: 34–51. 1995.
- Hill, T. "Honolulu Rail's Next Stop?" *Honolulu Magazine*, July 2011.

- Intel. "Corporate Social Responsibility." Intel Corporation. 2021. https://www.intel.com/intel/other/ehs/product_ecology/energy.htm.
- Kreske, D. L. *Environmental Impact Statements: A Practical Guide for Agencies, Citizens, and Consultants*. New York: Wiley. 1996.
- Kwon, D. M., and M. S. Seo. "A Study of Compliance with Environmental Regulations of ISO 14001 Certified Companies in Korea." *Journal of Environmental Management* 65, no. 4: 347–353. 2002.
- Lee, C. C., and S. D. Lin. *Handbook of Environmental Engineering Calculations*. New York: McGraw Hill Professional. 2000.
- Lockton, D., and D. Harrison. "Making the User More Efficient: Design for Sustainable Behaviour." *International Journal of Sustainable Engineering* 1, no. 1: 3–8. 2008
- Lovins, A. "Factor Ten Engineering Design Principles," version 1.0. 2011. https://www.rmi.org/Knowledge-Center/Library/2010-10_10xEPrinciples.
- Lovins, A., et al. "Case Studies." 2011. <https://move.rmi.org/markets-in-motion/case-studies/>.
- Maydl, P. "Sustainable Engineering: State-of-the-Art and Prospects." *Structural Engineering International* 14, no. 3: 176–180. 2004.
- Meryman, H. "Sustainable Engineering Using Specifications to Make It Happen." *Structural Engineering International* 14, no. 3. 2004.
- UK Ministry of Defence (MoD). *Standard 00-35, Environmental Handbook for Defence Materiel (Part 3) Environmental Test Methods*. London: UK Ministry of Defence. 2017. <https://standards.globalspec.com/std/10074175/def-stan-00-035-part-3>.
- Nash, J. *Regulating From the Inside: Can Environmental Management Systems Achieve Policy Goals?* Washington, DC: Resources for the Future Press. 2001.
- National Environmental Policy Act (NEPA). 42 USC 4321–4347. 1969. <https://ceq.doe.gov/laws-regulations/laws.html>.
- Stasinopoulos, P. *Whole System Design: An Integrated Approach to Sustainable Engineering*. London: Routledge. 2009.
- United Nations (UN). "Rio Declaration on Environment and Development." 1992. https://www.un.org/en/development/desa/population/migration/generalassembly/docs/globalcompact/A_CONF.151_26_Vol.I_Declaration.pdf.
- Whitelaw, K. *ISO 14001: Environmental Systems Handbook*, 2nd ed. Oxford: Elsevier. 2004.
- World Commission on Environment and Development (WCED). "Report of the World Commission on Environment and Development: Our Common Future." 1987. <https://sustainabledevelopment.un.org/milestones/wced>.

Primary References

- Bregman, J.I. *Environmental Impact Statements*, 2nd ed. Boca Raton, FL: CRC Press. 2000.
- Graedel, T. E., and B. R. Allenby. *Industrial Ecology and Sustainable Engineering*. Upper Saddle River, NJ: Prentice Hall. 2009.
- Lee, C. C., and S. D. Lin. *Handbook of Environmental Engineering Calculations*. New York: McGraw Hill Professional. 2000.
- Whitelaw, K. *ISO 14001: Environmental Systems Handbook*, 2nd ed. Oxford: Elsevier. 2004.

Additional References

None.

References

[1] [https://knowledge.bsigroup.com/products/](https://knowledge.bsigroup.com/products/assessment-of-life-cycle-greenhouse-gas-emissions-from-horticultural-products-supplementary-requirements-for-the-cradle-to-gate-stages-of-ghg-assessments-of-)

assessment-of-life-cycle-greenhouse-gas-emissions-from-horticultural-products-supplementary-requirements-for-the-cradle-to-gate-stages-of-ghg-assessments-of-

Knowledge Area: Systems Engineering and Geospatial/Geodetic Engineering

Systems Engineering and Geospatial/Geodetic Engineering

Contents of this Knowledge Area

- Overview of Geospatial/Geodetic Engineering (Ulrich Lenk)
- Relationship between Systems Engineering and Geospatial/Geodetic Engineering (Ulrich Lenk)
- Further Insights into Geospatial/Geodetic Engineering (Ulrich Lenk)
- Lead Author:
- Ulrich Lenk

Geographic Information Systems (GIS) and geospatial applications and infrastructures are widely used and widely integrated into other systems. Among the most well-known such systems are those based on the Global Positioning System (GPS) and other Global Navigation Satellite Systems (GNSS). They have enabled such diverse applications as automobile navigation systems, smartphones that location-stamp photographs, and military weapon systems that target enemy locations. Where it is often claimed that 80% of all data may be geospatially referenced, research by Hahmann and Burghardt (2013) indicates that about 60% of all data have a spatial reference; i.e. the data can be related to a physical coordinate in a spatial reference system, or identified by a geographic identifier. Systems and their constituents reside in or operate in space and often need to know where they or their parts, constituents, etc. are; where their mobile components go; or where objects observed by the system are. In other words (Longley et al. 2015): “Almost everything that happens, happens somewhere. Knowing where something happens can be critically important.” Extending this observation, potentially the system(s) and their associated constituents require synchronization of their activities and actions which is often achieved by triggering actions via time stamps; for this purpose, systems need to be time-wise synchronized to a certain extent or accuracy. The Geospatial/Geodetic Engineering (GGE) Knowledge Area provides a broad introduction into this overall topic in order to make the reader aware where relevant technologies are actually used in systems. It reflects the abundant uses and applications and even critical dependencies of geodetic and geospatial technologies in systems, such as GNSS & GPS (satellite positioning systems); GIS; spatial reference systems; processing, analysis and visualization (portrayal) of geographic data. It briefly analyzes to what extent the Systems Engineering Specialty Activities listed in the INCOSE Systems Engineering Handbook (2015) and modeling and simulation may be supported by related subject matter expertise. As a consequence it concludes that GGE activities could be considered as dedicated Specialty Engineering activities themselves within Systems Engineering.

References

Works Cited

Hahmann, S., and D. Burghardt. "How much information is geospatially referenced? Networks and Cognition." *International Journal of Geographical Information Science* 27, no. 6:1171-1189. 2013. <https://www.tandfonline.com/doi/abs/10.1080/13658816.2012.743664>

Primary References

International Council on Systems Engineering (INCOSE). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 4th ed. San Diego, CA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-04. 2015.

Longley, P.A., M. F. Goodchild, D. J. Maguire, and D. W. Rhind. *Geographic Information Science and Systems*. 4th ed. New York, Chichester, Weinheim: John Wiley & Sons, Inc. 2015.

Additional References

None.

Overview of Geospatial/Geodetic Engineering

- Lead Author:
- Ulrich Lenk

This article is part of the Systems Engineering and Geospatial/Geodetic Engineering (GGE) Knowledge Area. It provides a broad introduction into the overall topic including related applications in order to make the reader aware where GGE is used in systems.

GIS and Geospatial Applications

Perhaps the most comprehensive recent standard textbooks on Geographical Information Systems (GIS) are Longley et al. (2015) and Kresse and Danko (2012). Beyond these two books, there are many others on GIS and respective spatial data capture procedures (surveying, photogrammetry, and remote sensing) and management applications. Tomlinson (2019) and Peters (2012) as well as the online successor to this text book, System Design Strategies ^[1], provide valuable insights into aspects of how to set up a GIS system. While Tomlinson (2019) looks more at the management perspective and processes of implementing a GIS, Peters (2012) focuses more on technical aspects.

Domain-specific GIS applications are also documented in numerous textbooks. Domain areas include agriculture and forestry, insurance economics and risk analysis, simulation and environmental impact analysis, hydrology, archaeology, ecology, crime investigation and forensics, disaster management and first responders, marketing, municipalities and cadaster, land administration and urban planning, utility sectors, telecommunications, smart cities and military applications. The latter include Command and Control (C2) systems, or are even extended in Command, Control, Communications, Computers Intelligence, Surveillance and Reconnaissance (C4ISR) systems. Generally speaking, wherever data about events is displayed or portrayed, processed and/or analyzed in a geospatial context, GIS technology is involved. The particular type of user interface doesn't matter. It could be a web interface, a desktop client, or a mobile device such as a smartphone or tablet computer. To provide visualization of this data there must be spatial context for orientation, geographic data such as digital topographic maps, or geographic imagery, Digital Terrain Models (DTM), etc. Beyond such classical geographical data, other types of data are also often used, such as meteorological and other environmental data.

Interoperability is of major concern in geospatial technology. The Open Geospatial Consortium ^[2] (OGC) is probably the most relevant organization that deals with GIS and sensor systems interoperability. The OGC has published a dedicated set of interface specification standards ^[3] on their topics.

Positioning, Navigation and Timing

The previous description of geospatial technology focused mostly on stationary objects, i.e. on non-moving geospatial data. This section is mainly concerned with objects in motion, i.e. objects moving in space and with derived applications such as navigation, monitoring, and tracking such objects. The basic operations needed are (geo-)positioning and navigation. Certainly, the majority of people using smartphones are also using various location based services (LBS) that are provided in conjunction with GIS databases and services, such as Google Maps, a well-known online GIS application. As a consequence, positioning and navigation, which are mainly achieved with satellite positioning systems such as Global Navigation Satellite Systems (GNSS), have become ubiquitous and transparent technologies in the last decade. Clearly the most relevant system in common use in the past has been the US Global Positioning System (GPS) since it was the first of its kind. However, it is not the only one of its kind. Russia developed a GNSS called GLONASS; Europe developed the Galileo system which is close to achieving full operating capability; China is working on its Beidou GNSS. GNSS are used for more than positioning and navigation. Since range measurements conducted by GNSS are based on extremely accurate one-way travel times of signals, these satellites have extremely accurate clocks. GNSS transmit this time for use by other systems, enabling time synchronization of systems and also applications that require frequency normals that can be derived from these time signals. Together, these three GNSS services are called Positioning, Navigation and Timing (PNT). For more on satellite positioning systems and satellite navigation, two excellent sources are Teunissen and Montenbruck (2017) and Hofmann-Wellenhof et al. (2008).

The public does not generally appreciate how many systems used in various domains rely on the availability of GPS/GNSS signals. The majority of national critical infrastructures is now dependent on GNSS (Royal Academy of Engineering 2013; Wallischeck 2016). Thus the availability of open access to GNSS signals is itself considered critical infrastructures. Example infrastructures and applications that depend on GNSS include transport (rail, road, aviation, marine, cycling, walking), agriculture, fisheries, law enforcement, highways management, services for vulnerable people, energy production and management, surveying, dredging, health services, financial services, information services, cartography, safety monitoring, scientific and environmental studies, search and rescue (e.g. as given with the Global Maritime Distress and Safety System, GMDSS), telecommunications, tracking vehicles and valuable or hazardous cargoes, and quantum cryptography (Royal Academy of Engineering 2013).

Geodesy and Geodetic Engineering for Providing the Frameworks for All Spatial Applications

The above sections are application-oriented. However, at a more basic level, all numerical (coordinate-wise) descriptions of natural and man-made stationary and mobile objects, including satellites, need to be referenced to a spatial reference system. It may be a local stationary engineering coordinate reference system, a (moving) internal coordinate reference system that is fixed to an object (in motion), a national spatial reference system, a regional spatial reference system, or even a global spatial reference system, e.g. that given by the World Geodetic System 1984 (WGS84, cf. National Imagery and Mapping Agency 2004), which is the spatial reference system in which GPS works. 2-dimensional ("horizontal") coordinates such as the combination of latitude and longitude in a geodetic coordinate system are fairly straightforward because they are based mainly on mathematical assumptions. The shape of the Earth is approximated by an ellipsoid on whose surface the coordinates are defined, and the ellipsoid is fixed to the Earth via a geodetic datum/geodetic reference frame. Actually also the definition of the ellipsoid itself is part of the geodetic reference frame. Only little input from geophysics is needed (the localization of the rotation axis of the Earth). The third dimension, however, is mostly treated differently. Heights in general are referenced to a

reference surface. For ellipsoidal heights this is the ellipsoid, but since ellipsoidal heights can cause confusion as they do not consider the mass characteristics of the Earth with their distribution, it is more common to use heights that are related to a mean sea level (MSL) surface. The latter is mainly dependent on the (irregular!) distribution of masses on Earth and thus on their physical properties. The typical surface used for referencing these gravity-related heights is the so-called geoid which may be approximated by MSL. Beyond these Earth related aspects, however, there are also celestial spatial reference systems and spatial reference systems on other celestial bodies. Torge and Müller (2012) offers more information on these systems. Thus, geodetic engineering with its sub-disciplines of physical and mathematical geodesy together with related engineering disciplines provide fundamental frameworks for various applications in science and technology including systems of systems.

Portrayal of Geographic Data with Map Projections and Cartography

Because people often rely on visual depictions more so than on verbal descriptions, the complicated surface of the Earth typically needs to be “pressed” onto a flat screen, or a map or chart like in traditional cartography, even when it is a 3-dimensional perspective view on a 2D screen. Depending on the display scale, reducing from 3D to 2D cannot be achieved without somewhat or even significantly distorting the shape of the objects. Mathematical geodesy and map projections based on differential geometry provide the basics to achieve these goals (Grafarend et al. 2014). By carefully selecting an appropriate map projection, different characteristics of land masses and applications can be emphasized. One example may be the difference between the classical Mercator projection that is used for nautical charts from the equator up to medium latitudes, and the Stereographic projection that is often used in aeronautics since the shortest distance between two locations there is a straight line. For the Mercator projection, a straight line is a rhumb line, i.e. the line of constant bearing which eases the use of a magnetic compass for steering a vessel (neglecting variations of magnetic declination on Earth). Here, the shortest distance between two points on the Earth’s surface (the geodesic) is a curved line on the chart whose curve is bent towards the pole of the respective hemisphere.

As portrayal of geographic data is a fundamental functionality of GIS, respective map projection modules are generally included in GIS software packages. Beyond these purely projection-related aspects of visualizing geographic data, cartography offers rules and procedures for what to display and how to visualize geographic data. Kraak and Ormeling (2020) show such data may be abstracted by symbols, lines, and areas, including what color and styles to apply to the graphical elements in a map, how to relate these to each other on a screen or paper map, how to generalize them, i.e. how to simplify their shape and depiction or even discard on display, when the scale of display is changed, etc.

References

Works Cited

- Hahmann, S., and D. Burghardt. “How Much Information Is Geospatially Referenced? Networks and Cognition.” *International Journal of Geographical Information Science* 27, no. 6: 1171–1189. 2013. <https://www.tandfonline.com/doi/abs/10.1080/13658816.2012.743664>
- International Council on Systems Engineering (INCOSE). *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 4th ed. San Diego, CA: INCOSE. INCOSE-TP-2003-002-04. 2015.
- National Imagery and Mapping Agency. *World Geodetic System 1984*. 3rd ed., including Amendment 1 and 2. Department of Defense Technical Report TR8350.2. 2004.
- Royal Academy of Engineering. *Extreme Space Weather: Impacts on Engineered Systems and Infrastructure*. London: Royal Academy of Engineering. 2013.

Wallischeck, E. *GPS Dependencies in the Transportation Sector*. Cambridge, MA: U.S. Department of Transportation, Office of the Assistant Secretary for Research and Technology, John A. Volpe National Transportation Systems Center. 2016.

Primary References

Grafarend, E. W., R. J. You, and R. Syffus. *Map Projections: Cartographic Information Systems*. 2nd ed. Heidelberg; New York; Dordrecht; London: Springer. 2014.

Hofmann-Wellenhof, B., H. Lichtenegger, and E. Wasle. *GNSS - Global Navigation Satellite Systems*. Vienna: Springer-Verlag. 2008.

Kraak, M. J., and F. J. Ormeling. *Cartography: Visualization of Geospatial Data*. 4th ed. London; New York: Taylor & Francis. 2020.

Kresse, W. and D.M. Danko (Eds.). *Springer Handbook of Geographic Information*. Berlin, Heidelberg: Springer. 2012.

Kresse, W., and D. M. Danko, eds. *Springer Handbook of Geographic Information*. Berlin; Heidelberg: Springer. 2012.

Longley, P. A., M. F. Goodchild, D. J. Maguire, and D. W. Rhind. *Geographic Information Science and Systems*. 4th ed. New York; Chichester; Weinheim: John Wiley & Sons. 2015.

Peters, D. *Building a GIS: Geographic Information System Planning for Managers*. 2nd ed. Redlands, CA: Esri Press. 2012.

Teunissen, P., and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Switzerland: Springer International Publishing. 2017.

Tomlinson, R. F. *Thinking About GIS: Geographic Information System Planning for Managers*. 5th ed. Redlands, CA: Esri Press. 2019.

Torge, W. and J. Müller. *Geodesy*. Berlin: De Gruyter. 2012.

Additional References

Freedden, W., and M. Z. Nashed, eds. *Handbook of Mathematical Geodesy: Functional Analytic and Potential Theoretic Methods*. Basel: Birkhäuser. 2018.

Meyer, Th. H. *Introduction to Geometrical and Physical Geodesy: Foundations of Geomatics*. Redlands, CA: Esri Press. 2018.

References

[1] https://www.wiki.gis.com/wiki/index.php/System_Design_Strategies

[2] <https://www.opengeospatial.org>

[3] <https://www.opengeospatial.org/standards>

Relationship between Systems Engineering and Geospatial/Geodetic Engineering

- Lead Author:
- Ulrich Lenk

This article discusses the relationship between Systems Engineering (SE) and Geospatial/Geodetic Engineering (GGE) as reflected through relationships between several Specialty Engineering disciplines listed in INCOSE (2015) and GGE. For most of these disciplines, there are also SEBoK articles in the Knowledge Area SE and Quality Attributes.

Geospatial Aspects in the INCOSE Specialty Engineering Activities

Systems that directly include geospatial components and system elements, or that perform navigation operations or deal with referenceable objects in their broadest interpretations require dedicated contributions from the geodetic/geospatial domain. Those contributions should be achieved by integrating appropriate subject matter experts into SE teams.

The following sections briefly describe possible geospatial solutions or contributions which may directly support some of the Specialty Engineering activities in INCOSE (2015).

Environmental Engineering/Impact Analysis

Analyzing a spatial distribution or dispersal of pollutants typically depends on specific modules that have been integrated into Geographic Information Systems (GIS); e.g. a plume modeler will estimate how chemicals dissolve in the atmosphere under certain meteorological conditions. Other applications determine run-off for flooding simulations, or reveal dependencies between different types of environmental parameters during geospatial analysis. The list of such applications is long. The Knowledge Area (KA) Systems Engineering and Environmental Engineering provides more information.

Interoperability Analysis

Interoperability has been a major issue in geospatial infrastructures for decades. The Open Geospatial Consortium^[1] (OGC), founded in 1994, published its first standard (OpenGIS Simple Features Specification) in 1997. Other organizations also publish standards including the International Organization for Standards^[2] (ISO) with its Technical Committee 211 Geographic information/Geomatics^[3] (see also here^[4]), the International Hydrographic Organization^[5] (IHO) and the North Atlantic Treaty Organization^[6] (NATO). For meteorological data, the World Meteorological Organization^[7] (WMO) standardizes respective services and data formats. Typically, these bodies closely cooperate.

Using these standards can lead to significant cost savings in the development and operation of systems and thus contributes to another INCOSE Specialty Engineering activity: Affordability/Cost-Effectiveness/Life Cycle Cost Analysis. NASA funded a study that was conducted by Booz Allen Hamilton (2005). The study found that the project that adopted and implemented geospatial interoperability standards:

- had a risk-adjusted Return on Investment (ROI) of 119.0%. This ROI is a “Savings to Investment” ratio over the 5-year project life cycle.
- had a risk-adjusted Return on Investment (ROI) of 163.0% over a 10-year period.
- saved 26.2% compared to the project that relied upon proprietary standards.

Another finding was that standards-based projects have lower maintenance and operations costs than those relying exclusively on proprietary products for data exchange.

As a general conclusion from the above, there are substantial contributions from the geospatial domain that support interoperability analyses.

Logistics Engineering

According to INCOSE (2015), “Logistics engineering ... is the engineering discipline concerned with the identification, acquisition, procurement, and provisioning of all support resources required to sustain operation and maintenance of a system.” Amongst others, the following elements supporting logistics engineering are identified in INCOSE (2015) that have a direct relation to geospatial, GIS and PNT/Global Navigation Satellite Systems (GNSS) technologies:

- Sustaining engineering;
- Training and training support;
- Supply support;
- Facilities and infrastructures; and
- Packaging, handling, storage, and transportation (PHS&T).

Typical keywords associated with related activities are:

- “Technical surveillance” where, e.g., fielded systems are monitored with means of geodetic engineering techniques, such as deformation analysis of structures and sites,
- “Simulation” that requires virtual 3D environments and GIS,
- “Facilities” that are nowadays managed with Building Information Modeling (BIM) techniques which have a close connection to GIS, based on cadastre data from local authorities,
- “Transfer” and “transportation” where objects, material and goods are moved in space involving amongst others GIS with navigable map used for planning routes and navigation during transport.

Sometimes, GNSS and real-time GIS technologies are also used for tracking cargo of interest for safety reasons.

Reliability, Availability, and Maintainability

How reliable is a map, or, in the digitized world, a geographic data set displayed on screen or a mobile device? That depends firstly on the source of data, i.e. how reliable the source is, and on the other hand even for trusted data sources on the need to update that data set according to operational requirements and the changes that take place in the landscape of the area of interest. Updating and otherwise maintaining geospatial databases is a costly and sometimes time-consuming operation (again tied to the Specialty Engineering activity “Affordability/Cost-Effectiveness/Life Cycle Cost Analysis”). Efficiently updating geospatial databases is discussed from a technical perspective in Peters (2012) and, at least to a certain extent, must be reflected as well in the design of a geospatial data infrastructure. Using central services to provision geographic data is one possibility to address this issue since then, only one data set needs to be updated according to the single source of information principle. Others will access this data set via services to always receive the latest version of available data. The required availability constraints clearly must be addressed in the design of the IT infrastructure that hosts such a geospatial database, and also IT security aspects need to be considered.

Resilience Engineering

In recent years there has been an increasing awareness of the vulnerabilities of systems depending on GPS/GNSS. Resilient PNT is heavily discussed and alternatives like eLoran and Satelles are often mentioned in this context. According to the Royal Academy of Engineering (2013), “all critical infrastructure and safety critical systems that require accurate GNSS derived time and or timing should be specified to operate with holdover technology for up to three days.” This source also lists other recommendations to be considered for system design.

A source that provides example cases on a regular basis is the Resilient Navigation and Timing Foundation^[8] (RNT Foundation). Examples of official reports in the US and the UK are also Wallischeck (2016) and Royal Academy of Engineering (2011). Jamming and GPS disruptions actually occur and sometimes official warnings are issued, e.g. by the US Coast Guard (DHS 2016). According to an RNT Foundation notice^[9], there was an official warning from flight authorities during the 2017 G20 event in Hamburg, Germany. It cautioned to consider the possibility of GPS disruptions caused by intentionally initiated activities and actions to protect the G20 conference.

Prudent systems engineers will consider such dependencies and ensure to the degree practical that the systems at hand are resilient and fault tolerant, i.e. those systems do not terminate safe and reliable operation in the absence of GNSS signals, or cause major problems when they need to continue to communicate with other systems.

System Safety Engineering

Although it may not be straightforward, even in System Safety Engineering there are aspects that may be supported by geodetic and surveying engineering. One example may be the monitoring of dams, bridges and buildings etc., i.e. to what extent constructions move under differing environmental conditions, especially when subject to wind or water pressure or heat. Another example is the monitoring of natural objects such as volcanoes or slopes to detect early the possibility of future volcanic eruptions or potential landslides, or the monitoring of fracture zones or areas prone to earthquakes.

Usability Analysis/Human Systems Integration

Geographical displays sometimes form a central part of user interfaces. In such cases, proper usability analysis and other aspects of Human Systems Integration (all of these activities are part of Human Factors Engineering, HFE), geospatial expertise may be required. But beyond this, in HFE several other aspects are considered covering the general interaction of users with systems (Stanton et al. 2013). Nevertheless, in displaying virtual environments, HFE is related to the science and application of cartography (Kraak and Ormeling 2020) because the latter deals not only with portrayal of geographic data but also heavily with the different ways of human perception and abstraction of spatial phenomena, especially in dependence of the different scales the data is displayed.

Geospatial Aspects in Modeling and Simulation

Modeling and simulation is a broad field and heavily used in various disciplines and as such also in different SE life cycle processes. Geospatial technologies contribute to these activities amongst others by providing geographic data to create realistic environments, either for 2- or 3-dimensional applications. According to INCOSE (2015), such a model is then termed a “formal geometric model”. When considering temporal aspects and phenomena as well, 4-dimensional models are used. The modeler has to discern what types of geographic information must be modeled, and whether they are discrete objects which can be delimited with boundaries or whether they are continuous fields representing “the real world as a finite number of variables, each one defined at every possible position” (Longley et al. 2015), like temperature. For a comprehensive introduction to the general theory of geographic representation in GIS with continuous fields and discrete objects and how these concepts may be integrated see Longley et al. (2015), Goodchild et al. (2007) and Worboys and Duckham (2004).

In traditional cartography a map model was described by the well-known map legend that explained the portrayal of depicted features or phenomena. Today, fairly straightforward models for perspective visualization of landscapes are created using so-called Digital Terrain Models (DTM) and rendering them with geographic imagery. With these types of models no further descriptive information may be extracted besides geometric information and visual interpretation of the imagery to decide what is actually there. A well-known application for this is Google Earth. Vector models can provide more information. Discrete objects in a vector model may be further described by attributes, e.g. the width of a street. Vector models are created using so-called “feature catalogues” that define which real-world objects and domain values are to be represented. A typical military feature catalogue was created by the

Defence Geospatial Information Working Group (DGIWG) for worldwide military mapping projects and is called the DGIWG Feature Data Dictionary^[10] (DFDD). Feature catalogues vary with different levels of modeling scales, i.e. large-scale models provide a higher granularity than small-scale models that provide more of an overview.

INCOSE (2015) lists the following purposes for models throughout the system life cycle:

- Characterizing an existing system,
- Mission and system concept formulation and evaluation,
- System architecture design and requirements flow-down,
- Support for systems integration and verification,
- Support for training, and
- Knowledge capture and system design evolution.

The second and fifth purposes may be supported by geospatial technologies; i.e. data and software components that create, store, simulate and visualize/portray real-world or virtual models of environments where a system is going to be deployed, or where operations are going to take place (Tolk 2012). “Mission and system concept formulation and evaluation” tie to the definition of the Concept of Operation (ConOPS). By analyzing different variants of system deployment and categorizing them based on defined cost functions, it is possible to optimize a system design to provide a solid basis for decision making.

Conclusions

Geodetic and geospatial technologies and services play a fundamental role in many systems of systems and stand-alone systems. The general public is often not aware how strongly their lives and activities depend on these assets to provide and maintain critical infrastructure such as electric power and communications services. Against this background, systems engineers often need mastery of GGE knowledge and access to GGE subject matter experts and as a consequence, Geospatial and Geodetic Engineering may be considered as well a Specialty Engineering Discipline for Systems and Systems of Systems Engineering endeavors.

References

Works Cited

- Booz Allen Hamilton. *Geospatial Interoperability Return on Investment Study*. NASA Geospatial Interoperability Office, April 2005.
- Department of Homeland Security (DHS). *US Coast Guard Safety Alert 01-16: Global Navigation Satellite Systems – Trust, But Verify*. Washington, DC, January 19, 2016.
- Goodchild, M. F., M. Yuan, and Th. J. Cova. “Towards a General Theory of Geographic Representation in GIS.” *International Journal of Geographical Information Science* 21, no. 3: 239–260. 2007.
- Royal Academy of Engineering. *Extreme Space Weather: Impacts on Engineered Systems and Infrastructure*. London: Royal Academy of Engineering. 2013.
- Royal Academy of Engineering. *Global Navigation Space Systems: Reliance and Vulnerabilities*. London: Royal Academy of Engineering. 2011.
- Stanton, N. A., P. M. Salmon, L. A. Rafferty, G. H. Walker, and C. Baber. *Human Factors Methods: A Practical Guide for Engineering and Design*. 2nd ed. Farnham: Ashgate Publishing Limited. 2013.
- Tolk, A. *Engineering Principles of Combat Modeling and Distributed Simulation*. Hoboken, NJ: John Wiley & Sons. 2012.
- Wallischeck, E. *GPS Dependencies in the Transportation Sector*. Cambridge, MA: U.S. Department of Transportation, Office of the Assistant Secretary for Research and Technology, John A. Volpe National

Transportation Systems Center. 2016.

Worboys, M. F., and M. Duckham. *GIS: A Computing Perspective*. 2nd ed. Bristol, PA: Taylor & Francis. 2004.

Primary References

International Council on Systems Engineering (INCOSE). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 4th ed. San Diego, CA: INCOSE. INCOSE-TP-2003-002-04. 2015.

Kraak, M. J., and F. J. Ormeling. *Cartography: Visualization of Geospatial Data*. 4th ed. London; New York: Taylor & Francis. 2020.

Longley, P. A., M. F. Goodchild, D. J. Maguire, and D. W. Rhind. *Geographic Information Science and Systems*. 4th ed. New York; Chichester; Weinheim: John Wiley & Sons. 2015.

Peters, D. *Building a GIS: Building a GIS: Geographic Information System Planning for Managers*. 2nd ed. Redlands, CA: Esri Press. 2012.

Additional References

None.

References

- [1] <https://www.opengeospatial.org/>
- [2] <https://www.iso.org/home.html>
- [3] <https://www.isotc211.org/>
- [4] <https://committee.iso.org/home/tc211>
- [5] <https://iho.int>
- [6] <https://www.nato.int>
- [7] <https://www.wmo.int/>
- [8] <https://rntfnd.org>
- [9] <https://rntfnd.org/2017/07/28/g20-jams-gps/>
- [10] <https://www.dgiwg.org/FAD/overview.jsp>

Further Insights into Geospatial/Geodetic Engineering

- Lead Author:
- Ulrich Lenk

This article is part of the Systems Engineering and Geospatial/Geodetic Engineering (GGE) Knowledge Area. It discusses in more detail a selected set of topics that a beginner in Geographic Information Systems (GIS) and science or a systems engineer adopting respective techniques might be interested in or should be aware of. Topics discussed include bodies of knowledge on geospatial technologies, various aspects associated with geographic data, and standardization in the geospatial domain.

GIS related Bodies of Knowledge

The emphasis of the article Overview of Geospatial/Geodetic Engineering was to focus on to what extent systems and systems of systems are dependent on GIS related technologies and where potential interfaces or contributions are. In order to provide now an improved but still brief overview of which topics are related in general to geospatial and geodetic engineering and how broad the geospatial domain actually is, a high-level introduction into existing bodies of knowledge in the geospatial domain is provided here.

The work on a body of knowledge (BOK) for the geospatial domain actually goes back into the 1980s (cf. Kemp & Goochild 1991, cited in Stelmaszczuk-Górska et al. 2020), and since then at least two major workstreams have evolved. One in the United States that culminated first in 2006 with the publication of Edition 1 of the Geographic Information Science and Technology Body of Knowledge (GISTBoK) by the University Consortium for Geographic Information Science (UCGIS) ^[1] (DiBiase et al. 2006). For the Geospatial Intelligence (GEOINT) discipline, a refinement was elaborated by the United States Geospatial Intelligence Foundation (USGIF) ^[2]. The UCGIS GISTBoK also formed the nucleus for the other workstream in Europe which started with the GI-N2K: Geographic Information – Need to Know ^[3] project (Vandenbroucke and Vancauwenberghe 2016) that aimed to better reflect European aspects in a BOK and to provide an ontological structure of concepts and relationships (Hofer et al. 2020). The European workstream was then further pursued as part of the Earth Observation for Geoinformation project (EO4GEO) ^[4] that refined and extended the work from GI-N2K (Stelmaszczuk-Górska et al. 2020; Hofer et al. 2020).

UCGIS: Geographic Information Science and Technology Body of Knowledge (GISTBoK)

For the 2006 GISTBoK a hierarchical decomposition of the geospatial domain was conducted into 10 Knowledge Areas which were again divided into 73 Units and then into 329 Topics. There were over 1600 Learning Objectives listed in these topics. With the update that began in 2013 (Wilson 2014), there are also 10 Knowledge Areas in the current GISTBoK but they have changed partly versus the 2006 version. As of the beginning of 2022, there are 54 Units and 363 Topics. The current Knowledge Areas are:

- Foundational Concepts, with 7 Units and 35 Topics;
 - Knowledge Economy, with 4 Units and 20 Topics;
 - Computing Platforms, with 5 Units and 28 Topics;
 - Programming and Development, with 5 Units and 23 Topics;
 - Data Capture, with 8 Units and 35 Topics;
 - Data Management, with 7 Units and 53 Topics;
 - Analytics and Modeling, with 9 Units and 70 Topics;
 - Cartography and Visualization, with 6 Units and 36 Topics;
 - Domain Applications, with 44 Topics (with no categorization into Units); and
-

- GIS&T and Society, with 3 Units and 19 Topics.

It should be noted however that the GISTBoK is constantly evolving and for the latest version the respective GISTBoK online resource ^[5] has to be checked. Additionally, a feature of this BOK is that many Topics are linked with respective citable articles providing insights into the subjects at hand. The UCGIS also provides at its web site (UCGIS BOK ^[6]) information on open educational resources on GIS and GIScience.

USGIF: GEOINT Essential Body of Knowledge

Aside from the activities hosted by the UCGIS that were mainly driven by academia, the USGIF published in 2014 the first version of its GEOINT Essential Body of Knowledge that targeted the GEOINT discipline. Among other sources, it was based on the 2006 GISTBoK (DiBiase et al. 2006) but extending it where necessary to better reflect the broader needs of GEOINT and related industries. The second version (Brooks et al. 2019) was published in 2019 after an 18 months period of preparation with a survey in the GEOINT community involving various subject matter experts that interpreted the results of the survey. It serves as a guide to what skills are required in the GEOINT discipline and acts as a blueprint for respective Certified GEOINT Professional exams (Brooks et al. 2019; Baber 2018). The GEOINT Essential Body of Knowledge version 2.0 is divided into three parts. The first one is related to “Technical Competencies” with the following areas:

- GIS & Analysis Tools;
- Remote Sensing & Imagery Analysis;
- Geospatial Data Management; and
- Data Visualization.

The second part is related to “Cross Functional Competencies” which cover general skills like soft skills or common GEOINT knowledge and practices suitable for the GEOINT practitioner, whereas the third part looks at “Emerging Competencies”, like data science, machine learning techniques, virtual reality, artificial intelligence, and unmanned aerial platforms.

It is worth mentioning that, since 2015, USGIF also publishes the “State and Future of GEOINT Reports” on a yearly basis. These may also serve as a general reference on future trends in geospatial technologies.

Europe: The "GI-N2K: Geographic Information - Need to Know" and the "EO4GEO: Earth Observation for Geoinformation" BOKs

The GI-N2K project funded by the European Union's (EU) Erasmus Lifelong Learning Program and its BOK started as well with the 2006 GISTBoK (DiBiase et al. 2006) and had 10 Knowledge Areas. For these Knowledge Areas 63 sub-concepts were identified and further divided into 301 on level 3. However, in some instances level 3 was even further de-composed into level 4 and partly into level 5 concepts. At the end, 411 concepts were defined on these levels. Additional features that were provided with this BOK were curriculum design tools and a GeoWiki to enable discussion between experts.

The most recent development in European GIS-related BOKs is the EO4GEO BOK that continues and further develops as part of the Erasmus+ Sector Skills Alliance project EO4GEO the work conducted in the GI-N2K project. As Earth Observation (EO) and Geoinformation (GI) data sources, especially from the space sector, are gaining nowadays much more importance for data capture and updates of derivative data, the respective skills for data capture, information processing, standalone and combined analysis and associated applications need to be defined and matched or merged with the previous BOKs to reflect this change in academia, business and applications (Stelmaszczuk-Górska et al. 2020). An analysis revealed that "neither the American nor the European GIS&T (comment: Geographic Information Science and Technology) and GI-N2K BOKs include comprehensive information on EO" (Stelmaszczuk-Górska et al. 2020). Additionally, since there was a criticism that the previous BOKs were too much oriented along education driven by academia and too theoretical with a lack of practical aspects, an emphasis was made to “better align” the academically oriented EO4GEO BOK “with the business,

professional, and industrial perspective” (Hofer et al. 2020) by analyzing a set of relevant business processes with regard to applicable concepts.

The EO4GEO BOK ^[7] has at its highest level 14 subconcepts as follows:

- Analytical Methods, with 14 subconcepts;
- Conceptual Foundations, with 8 subconcepts;
- Cartography and Visualization, with 6 subconcepts;
- Design and Setup of Geographic Information Systems, with 4 subconcepts;
- Data Modeling, Storage and Exploitation, with 5 subconcepts;
- Geocomputation, with 4 subconcepts;
- Geospatial Data, with 4 subconcepts;
- GI and Society, with 6 subconcepts;
- Image processing and analysis, with 6 subconcepts;
- Organizational and Institutional Aspects, with 5 subconcepts;
- Physical principles, with 2 subconcepts;
- Platforms, sensors and digital imagery, with 4 subconcepts;
- Thematic and application domains, with 5 subconcepts; and
- Web-based GI, with 7 subconcepts.

Similar as with the GIN-2K BOK, there are partly also further levels below the subconcepts. In addition to the BOK, it provides an occupational profile tool, a job offer tool, a curriculum design tool, a BOK annotation tool, a BOK matching tool and other educational features. For the concepts, their names are given along with descriptions and references. A set of 5 relationships between the concepts is maintained, and skills explaining the practical use of the EO*GEO knowledge are associated with the concepts (Hofer et al. 2020). The BOK exploration is supported by a graphical tool.

Geographic Data and Metadata

Geographic Data

Geospatial data is actually the fuel needed for any type of geographic application, whether it might be only for visualization purpose, e.g. as background information for real-time situational awareness applications, or for advanced spatial analytics involving different data sources and specific analysis methods. A first categorization into the two fundamental concepts of geographic data has already been given in the SEBoK article Relationship between Systems Engineering and Geospatial/Geodetic Engineering. They are:

- Continuous fields, i.e. spatially distributed phenomena with no clear limits or boundaries and representing “the real world as a finite number of variables, each one defined at every possible position” (Longley et al. 2015). For the case that repeated pattern of positions is used, the term raster data is commonly used, especially for the case that an equidistant matrix pattern is used. However, a matrix could potentially have different resolutions in columns and rows, or theoretically also other regular patterns could be involved, such as hexagonal patterns, but these applications are very rare.
- Discrete objects or features, which are delimited by boundaries and potentially associated with a set of attribute data to describe them further beyond their spatial properties. This type of data is also termed vector data in a GIS context.

Beyond these two fundamental concepts, the different aspects that need to be considered for geographic data when designing a system using this data are very diverse and cannot be treated in full detail here. A selection of important key words coming from practical experiences to be considered when implementing GIS databases include:

- What data is actually needed (examples see below)?

- At what scales shall the data be visualized, i.e. the level of detail needed.
- Dimensionality: typical dimensionalities used in GIS technologies are:
 - 2D, describing the earth surface in a flat plane, like a paper map;
 - 2.5D, with a unique z-value to a position in the horizontal plane;
 - 3D, where all three dimensions are considered; and
 - Time dimension: for the case of 3D data then 4D, but as also the former 2 cases may have variations in time this is treated here as time dimension.
- What are the critical infrastructures that need to be shown, such as transport networks?
- What standards need to be considered, such as feature catalogues, interface and data format standards, data acquisition standards etc.?
- What is the positional accuracy required for the geographic data? This is typically associated with data acquisition method to be selected and obviously with the costs involved.
- What is the level of semantic detail needed, e.g. how many feature attributes shall be captured for features / vector data and how big is the set of domain values from which they shall be selected?
- Are there complex topological relations to be captured and maintained, i.e. to establish connectivity for the vector data, for example, for routing applications or utility networks?
- Questions on updates:
 - How often does the data need to be updated? This is directly related to maintenance costs for the database, i.e. recurring costs to be considered, but also to availability of resources for the updates.
 - How shall the data be updated? Is it possible to use a central service for the data which is updated, i.e. can the responsibility for the updates be delegated?
 - What communication lines are used when data and updates are distributed in a system? Or is a service model the better choice as it realizes a single source of information principle? A systems engineers has to keep in mind that geographic data can reach considerable data volumes (depending on type of data terabytes and petabytes) that cannot be easily distributed over the air for example.
- What are the data sources that may or have to be used? How are bounding conditions on the use of the data?
 - Authoritative data from a spatial data infrastructure, from international or national governmental agencies (or even intergovernmental agencies), such as national surveys like the USGS or the British Ordnance Survey, or on an international level the United Nations.
 - Commercial data sources, such as satellite imagery service providers, or mapping service providers.
 - Open sources, from activities like the Open Street Map ^[8] or Open Seamap ^[9] initiatives.
 - Copyrights and Intellectual Property Rights associated with the data sources.
 - Classification of data.
 - Bounding legal conditions (e.g. export control laws and regulations for export of data, as for example some satellite image resolutions may have export restrictions).
 - Liability aspects for the data, especially for the cases of legal boundaries, i.e. national borders. This is of particular relevance when borders are under dispute between neighboring countries!

For implementation aspects again Tomlinson (2019) and Peters (2012) are referred to as well as the online successor to the latter text book, System Design Strategies ^[1].

Metadata for Geographic Data

Whereas the section above discusses aspects of geographic data itself, it is also of fundamental importance to make this data available to or detectable by potential users. This is done by describing the data by metadata and having the metadata available, for example in a catalog where users can search for it. Whereas the Dublin Core data set (ISO 2017; ISO 2019b) defined by the Dublin Core Metadata Initiative is now used to describe general items, for the special case of geographic data, a set of dedicated ISO standards has been developed (ISO 2014, with its

amendments ISO 2018 & ISO 2022; ISO 2020d). As such, ISO 19115 "provides information about the identification, the extent, the quality, the spatial and temporal aspects, the content, the spatial reference, the portrayal, distribution, and other properties of digital geographic data and services" (ISO 2014).

Geocoding Systems, Localization and Geographic Search

One further particular aspect looked at here is how a spatial reference for a feature may be expressed. Certainly the most well-known way to describe a location technically or mathematically is by coordinates, either in 2 dimensions for the simple case of a plane, or in 3 dimensions or even adding a time dimension. Standardized ways to express geographic coordinates are covered by ISO 6709 (ISO 2009) but also Cartesian coordinate systems are in use. However, a spatial reference may also be given by other types of geographic identifiers where a location is expressed by a specific (sometimes non-numeric) code or name. A gazetteer is used to manage geographic identifiers, such as geographic names, e.g. names of states, provinces, or other geographically identifiable features such as lakes etc. Other codes in use are for example addresses (where it should be remembered that there are also different postal address types in use), country codes (ISO 3166-1, ISO 2020a) and codes of country principal subdivisions such as states and provinces (ISO 3166-2; ISO 2020b), but there are many other, sometimes application or domain specific or even commercially developed geocodes.

An example of a commercially developed and thus proprietary geocode is the What3words^[10] system that is even in use as a postal addressing system in some countries. By dividing the Earth's surface into squares of about 3 meters by 3 meters and assigning unique 3 ordered words to each of them, the codes for each location are established. There are, however, several other systems available, like Geohash^[11] or Mapcode^[12], which have no license restrictions.

In general, geocodes may be categorized into non-hierarchical and hierarchical geocodes, while for the latter the accuracy of location position increases in a refinement/subdivision process, similar to adding more significant digits to a coordinate. A well-known dataset of geographic names often used in GIS applications is provided by geonames.org^[13].

These codes can then also be used to navigate in a geographic display, i.e. by inserting a geocode one can jump directly in the display to the respective position or features (described by positions) associated with the code. In case the code is ambiguous (as it is sometimes the case for geographic names like city names) a disambiguation could be given, in order to clarify the selection. While this approach is mainly used to navigate in a display or to find a location, it should not be confused with the topic of efficiently searching in multidimensional spatial databases. This is not treated here as it relates to database management system design and implementation, including spatial indexing, for example with space filling curves.

Example: The United Nations 14 Global Fundamental Geospatial Data Themes

The set of geographic data to be used in a system will always be dependent on the purpose and goals of the system at hand, and therefore no general purpose structure can be provided here. Some examples of geographic data have already been given in the SEBoK article Relationship between Systems Engineering and Geospatial/Geodetic Engineering in the frame of Geospatial Aspects in Modeling and Simulation. In order to extend this for a better and broader overview of what may be considered as relevant in general, the following list of geospatial data themes may serve as a first indicator. It has been elaborated as "The 14 Global Fundamental Geospatial Data Themes" by the United Nations Committee of Experts on Global Geospatial Information Management (UN-GGIM 2019).

- Global Geodetic Reference Frame;
- Addresses, such as postal addresses, see above;
- Buildings and Settlements;
- Elevation and Depth, e.g. provided by Digital Elevation Models and Digital Terrain Models (DTM);
- Functional Areas, such as administrative or legislative areas;
- Geographical Names, e.g. geographic identifiers managed and provided by a gazetteer, see above;

- Geology and Soils;
- Land Cover and Land Use;
- Land Parcels, e.g. a cadastre or a land register;
- Physical Infrastructure, including industrial and utility facilities;
- Population Distribution;
- Orthoimagery, which is a special case of geographic imagery in orthogonal projection;
- Transport Networks, e.g. rails, roads, waterways and air transport routes associated potentially with connectivity relations; and
- Water, including rivers, lakes and marine features.

The UN-GGIM (2019) provides more insights and information into the themes, e.g. what standards are available and possible sources for data. In GIS where often multiple geographic data sets from different sources are processed in a combined way, these data sets are organized into a stacked set of layers which may be for example switched on and off individually for visualization purpose.

Standardization Organizations active in the Geospatial Domain

In the following selected international civil organizations are briefly introduced that publish standards related to the geospatial domain.

The Open Geospatial Consortium (OGC)

The Open Geospatial Consortium (OGC) was founded in 1994 and publishes open standards and specifications in the geospatial domain. The documents are created in a member-driven consensus process. The most successful standards are the Web Map Service (WMS; OGC 2006) and the Web Feature Service (WFS; OGC 2010), but OGC has published about 70 implementation standards and about 20 abstract specifications.

OGC works closely with ISO TC211 (see next section), and some documents are jointly elaborated and published. For example, the above mentioned WMS is also an ISO standard (namely ISO 2005), as is the WFS (ISO 2010). Another example is the specification of the Geography Markup language (OGC, 2012; and ISO, 2015 & 2020a) that is published by both OGC and ISO. Special care has to be taken which version is published in which document since they are not necessarily published in the same versions at the same time. Besides the cooperation with ISO TC211, OGC has in addition other alliance partners, such as the Object Management Group (OMG), the Organization for Advancement of Structured Information Standards (OASIS), the Web3D Consortium, the World Wide Web Consortium (W3C), the International Hydrographic Organization (IHO) and the World Meteorological Organization (WMO) (both see below), and also with other ISO TCs.

Several companies well-known to the general public such as Amazon Web Services, Apple, Google, Microsoft, Oracle and SAP, and universities, governmental, inter-governmental and non-governmental organizations as well as individuals are members of the OGC at different levels of membership, summing up to more than 500 members.

ISO TC 211 “Geographic information/Geomatics”

The International Organization for Standardization (ISO) is certainly the best known international standardization organization in the world and is organized into technical committees which develop the standards. The Technical Committee (TC) 211 is related to “Geographic information/Geomatics”. TC211 has published more than 80 standards, most of them as part of the 191xx family of standards, including abstract specifications and interface standards together with the OGC. Several ISO TC211 standards are referenced in the list of references below. TC211 also maintains the Online Multi-Lingual Glossary of Terms (MLGT) ^[14] at Geolexica that was used to define terms used in this Knowledge Area. Typically ISO standards are also promulgated as national standards, or as European standards from the respective European standardization organizations.

International Hydrographic Organization (IHO)

Founded in 1921 as the International Hydrographic Bureau and renamed in 1970 to the International Hydrographic Organization (IHO), IHO is an intergovernmental organization that standardizes and coordinates activities in the area of hydrography, nautical cartography and thus nautical charts to ensure initially and still primarily the safety of navigation. With the increasing interest in the marine environment, e.g. for the installation of offshore wind farms, the importance of the activities of IHO has even more increased as it publishes standards for the creation and exchange of digital hydrographic data (IHO 2020; IHO 2017a; IHO 2000, with its appendices) that may serve as a GIS base map layer, and also for the portrayal of data (IHO 2014). The way how hydrographic offices can support the creation of spatial data infrastructures by providing data for the marine environment is discussed in IHO (2017b). With the revision of its standards to adhere to ISO TC211 standards, IHO is now transitioning to the S-100 family of standards (IHO 2018).

World Meteorological Organization (WMO)

Originally founded in 1873 as the International Meteorological Organization and renamed in 1950 into the World Meteorological Organization (WMO), WMO is an intergovernmental organization and specialized agency of the United Nations. According to its mandate as described on its website ^[15], it provides the framework for international cooperation "in the areas of meteorology (weather and climate), operational hydrology and related geophysical sciences" and facilitates "free and unrestricted exchange of data and information, products and services in real- or near-real time on matters relating to safety and security of society, economic welfare and the protection of the environment." WMO defines several data formats for the exchange of weather information (WMO 2019 & 2021a/b).

Clearly the scientific background needed to create meteorological information goes far beyond of what is needed in standard GIS applications. From a GIS perspective, weather information may be treated as one or several information layer(s), and due to the typically required real- or near-real time information respective online interfaces have to be established with weather data providers, whether they are national weather services or commercial companies.

References

Works Cited

- Baber, M. "Geospatial Intelligence and National Security." In *The Geographic Information Science & Technology Body of Knowledge (1st Quarter 2018 Edition)*, edited by John P. Wilson. 2018. <https://doi.org/10.22224/gistbok/2018.1.2>
- Hofer, B., S. Casteleyn, E. Aguilar-Moreno, E. M. Missoni-Steinbacher, F. Albrecht, R. Lemmens, S. Lang, J. Albrecht, M. Stelmaszczuk-Górska, G. Vancauwenberghe, and A. Monfort-Muriach. "Complementing the European Earth Observation and Geographic Information Body of Knowledge with a Business-Oriented Perspective." *Transactions in GIS* 24, no. 3: 587–601.2020. <https://doi.org/10.1111/tgis.12628>. ^[16]
- IHO (International Hydrographic Organization). *IHO Transfer Standard for Digital Hydrographic Data*. Publication S-57, Edition 3.1.0. Monaco: International Hydrographic Bureau. 2000.
- . *Specifications for Chart Content and Display Aspects of ECDIS*. Publication S-52, Edition 6.1(1). Monaco: International Hydrographic Organization. 2014.
- . *ENCs: Production, Maintenance and Distribution Guidance*. Publication S-65, Edition 2.1.0. Monaco: International Hydrographic Organization. 2017.
- . *Spatial Data Infrastructures "The Marine Dimension": Guidance for Hydrographic Offices*. Publication C-17, 2nd ed., Version 2.0.0. Monaco: International Hydrographic Organization. 2017.

- . *IHO Universal Hydrographic Data Model*. Publication S-100, Edition 4.0.0. Monaco: International Hydrographic Organization. 2018.
- . *IHO Standards for Hydrographic Surveys*. Publication S-44, Edition 6.0.0. Monaco: International Hydrographic Organization. 2005.
- ISO (International Organization for Standardization). 2005. *ISO 19128:2005, Geographic information - Web map server interface*. ISO 19128, First edition 2005-12-01. Geneva, Switzerland: International Organization for Standardization. 2020.
- ISO (International Organization for Standardization). 2005. *ISO 19128:2005, Geographic Information – Web Map Server Interface*. Geneva: ISO.
- . *ISO 6709:2009, Standard Representation of Geographic Point Location by Coordinates*. Geneva: ISO. 2009.
- . *ISO 19142:2010, Geographic Information – Web Feature Service*. Geneva: ISO. 2010.
- . *ISO 19115-1:2014, Geographic Information – Metadata – Part 1: Fundamentals*. Geneva: ISO. 2014.
- . *ISO 19136-2:2015, Geographic Information – Geography Markup Language (GML) – Part 2: Extended Schemas and Encoding Rules*. Geneva: ISO. 2015.
- . *ISO 15836-1:2017, Information and Documentation – The Dublin Core Metadata Element Set – Part 1: Core Elements*. Geneva: ISO. 2017.
- . *ISO 19115-1:2018, Geographic Information – Metadata – Part 1: Fundamentals Amendment 1*. Geneva: ISO. 2018.
- . *ISO 19115-2:2019, Geographic Information – Metadata – Part 2: Extensions for Acquisition and Processing*. Geneva: ISO. 2019.
- . *ISO 15836-2:2019, Information and Documentation – The Dublin Core Metadata Element Set – Part 2: DCMI Properties and Classes*. Geneva: ISO. 2019.
- . *ISO 19136-1:2020, Geographic Information – Geography Markup Language (GML) – Part 1: Fundamentals*. Geneva: ISO. 2020.
- . *ISO 3166-1:2020, Codes for the Representation of Names of Countries and Their Subdivisions – Part 1: Country Codes*. Geneva: ISO. 2020.
- . *ISO 3166-2:2020, Codes for the Representation of Names of Countries and Their Subdivisions – Part 2: Country Subdivision Code*. Geneva: ISO. 2020.
- . *ISO 19115-1:2020, Geographic Information – Metadata – Part 1: Fundamentals Amendment 2*. Geneva: ISO. 2020.
- Longley, P. A., M. F. Goodchild, D. J. Maguire, and D. W. Rhind. *Geographic Information Science and Systems*. 4th ed. New York; Chichester; Weinheim: John Wiley & Sons. 2015.
- OGC (Open Geospatial Consortium). *OpenGIS Web Map Server Implementation Specification*. Version 1.3.0, OGC document 06-042. 2006. <https://www.ogc.org/standards/wms>.
- . *OpenGIS Web Feature Service 2.0 Interface Standard*. Version 2.0.0, OGC document 09-025r1. 2010. <https://www.ogc.org/standards/wfs>.
- . *OGC Geography Markup Language (GML) – Extended Schemas and Encoding Rules*. Version 3.3.0, OGC document 10-129r1. 2012. <https://www.ogc.org/standards/wfs>.
- Stelmaszczuk-Górska, M. A., E. Aguilar-Moreno, S. Casteleyn, D. Vandenbroucke, M. Miguel-Lago, C. Dubois, R. Lemmens, G. Vancauwenberghe, M. Olijslagers, S. Lang, F. Albrecht, M. Belgiu, V. Krieger, T. Jagdhuber, A. Fluhrer, M. J. Soja, A. Mouratidis, H. J. Persson, R. Colombo, and G. Masiello. "Body of Knowledge for the Earth Observation and Geoinformation Sector – A Basis for Innovative Skills Development." *International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* XLIII-B5-2020: 15–22. 2020. <https://doi.org/>

10.5194/isprs-archives-XLIII-B5-2020-15-2020.

United Nations Committee of Experts on Global Geospatial Information Management (UN-GGIM). *The Global Fundamental Geospatial Data Themes*. New York: United Nations. 2019.

Vandenbroucke, D., and G. Vancauwenberghe. "Towards a New Body of Knowledge for Geographic Information Science and Technology." *Micro, Macro & Mezzo Geoinformation* 2016, no. 6: 7–19.

Wilson, J. P. *Geographic Information Science & Technology Body of Knowledge 2.0 Project: Final Report*. University Consortium for Geographic Information Science Symposium, Pasadena, CA. 2014.

World Meteorological Organization (WMO) *WMO-No. 306 Manual on Codes – International Codes Volume I.1, Annex II to the WMO Technical Regulations, Part A – Alphanumeric Codes*. Geneva: WMO. 2019.

———. *WMO-No. 306 Manual on Codes – International Codes Volume I.2, Annex II to the WMO Technical Regulations, Part B – Binary Codes & Part C – Common Features to Binary and Alphanumeric Codes*. Geneva: WMO. 2021.

———. *WMO-No. 306 Manual on Codes – International Codes Volume I.2, Annex II to the WMO Technical Regulations, Part D – Representations Derived from Data Models*. Geneva: WMO. 2021.

Primary References

DiBiase, D., M. DeMers, A. Johnson, K. Kemp, A. T. Luck, B. Plewe, and E. Wentz, eds. *Geographic Information Science and Technology Body of Knowledge*. Ed. 1. Ithaca, NY: University Consortium for Geographic Information Science. 2006. <https://www.ucgis.org/gis-t-body-of-knowledge>.

Brooks, T., Kantor, C., Spuria, L. and Quinn, K. (Eds.). *The Geospatial Intelligence Essential Body of Knowledge*, Version 2.0/2019, Compiled by the United States Geospatial Intelligence Foundation. January 2019. 2019. Available at <https://usgif.wpengine.com/wp-content/uploads/2020/11/ebk2019.pdf>.

Brooks, T., C. Kantor, L. Spuria, and K. Quinn, eds. *The Geospatial Intelligence Essential Body of Knowledge*, Version 2.0/2019. Compiled by the United States Geospatial Intelligence Foundation. January 2019. 2019. <https://usgif.wpengine.com/wp-content/uploads/2020>

Peters, D. *Building a GIS: Geographic Information System Planning for Managers*, 2nd ed. Redlands, CA: Esri Press. 2012.

Tomlinson, R. F. *Thinking About GIS: Geographic Information System Planning for Managers*. 5th ed. Redlands, CA: Esri Press. 2019.

Additional References

EO4GEO. *Website of the EO4GEO BOK*. Accessed January 20, 2022. <https://www.eo4geo.eu/bok/>.

International Hydrographic Organization (IHO). *Website of IHO*. Accessed February 2, 2022. <https://iho.int/>.

ISO TC211. *Website of ISO TC211*. Accessed February 02, 2022. <https://www.isotc211.org/>.

ISO. *Websites of ISO TC211 at the ISO Website*. Accessed February 02, 2022. <https://www.iso.org/committee/54904.html> and <https://committee.iso.org/home/tc211>.

Open Geospatial Consortium (OGC). *Website of the OGC*. Accessed February 02, 2022. <https://www.ogc.org/>.

University Consortium for Geographic Information Science (UCGIS). *Website of the UCGIS GISBoK*. Accessed January 20, 2022. <https://gistbok.ucgis.org/>.

World Meteorological Organization (WMO). *Website of WMO*. Accessed February 02, 2022. <https://wmo.int/>.

References

- [1] <https://www.ucgis.org/>
- [2] <https://usgif.org/>
- [3] <https://www.gi-n2k.eu/>
- [4] <https://www.eo4geo.eu/>
- [5] <https://gistbok.ucgis.org/>
- [6] <https://www.ucgis.org/gis-t-body-of-knowledge>
- [7] <https://www.eo4geo.eu/bok/>
- [8] <https://www.openstreetmap.org/>
- [9] <https://www.openseamap.org/>
- [10] <https://what3words.com/>
- [11] <https://geohash.org/>
- [12] <https://www.mapcode.com/>
- [13] <https://www.geonames.org/>
- [14] <https://isotc211.geolexica.org/>
- [15] <https://wmo.int/>
- [16] <https://doi.org/>

Knowledge Area: Systems Engineering and Industrial Engineering

Systems Engineering and Industrial Engineering

- Lead Author:
- Gregory S. Parnell
- Contributing Authors:
- C. Robert Kenley and Eric Specking, Ed Pohl

Systems Engineering (SE) overlaps with many fields, such as Industrial Engineering (IE), Engineering Management, Operations Research, Project Management, and Design Engineering. In fact, the main Industrial Engineering body of knowledge, called the Industrial and Systems Engineering Body of Knowledge (ISEBoK) (IISE 2021), includes the word "systems" in its title and includes a section on systems design and engineering, which references the SEBoK. This article describes the similarities and differences between SE and IE based upon their respective standards, handbooks, and bodies of knowledge. Based on this assessment, this article describes potential roles that systems engineers and industrial engineers perform during a system's life cycle.

Introduction

When systems engineers and industrial engineers are in the same organization, they have different roles and responsibilities. While job titles vary by organization, many organizations have individuals that perform both SE and IE activities. This article tries to help systems engineers and industrial engineers better understand the different perspectives of the fields and the knowledge needed to meet the needs of their organizations and customers. The article compares the use of international standards and the contents of the bodies of knowledge for SE and IE.

Systems Engineering

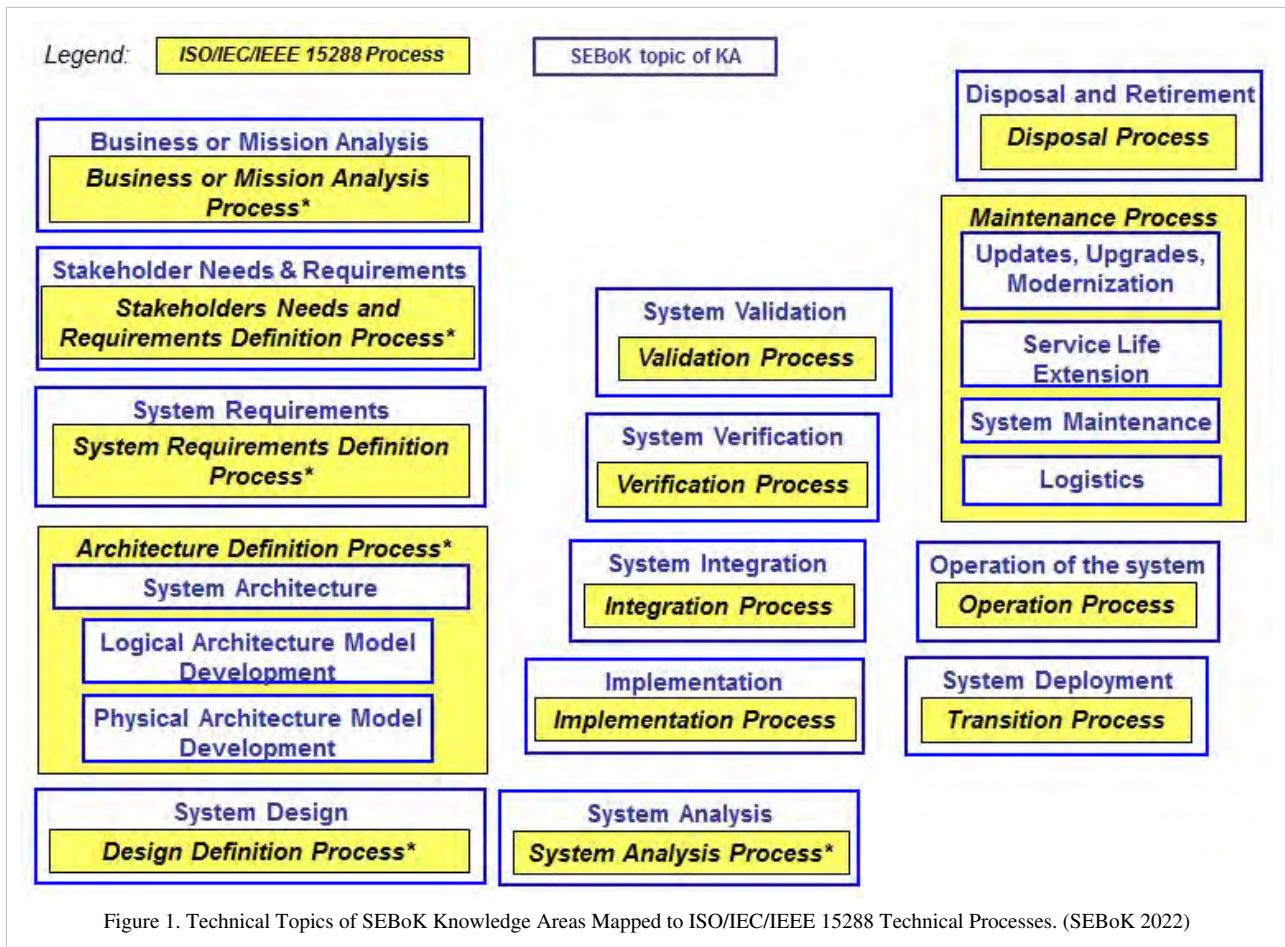
The International Council on Systems Engineering (INCOSE) is a "not-for-profit membership organization founded to develop and disseminate the interdisciplinary principles and practices that enable the realization of successful systems." INCOSE defines systems engineering as

a transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems, using systems principles and concepts, and scientific, technological, and management methods. (INCOSE 2021)

Here, the terms "engineering" and "engineered" are used in their widest sense: "the action of working artfully to bring something about." "Engineered systems" may be composed of any or all of people, products, services, information, processes, and natural elements.

INCOSE aligns its SE Handbook with ISO/IEC/IEEE 15288, System Life Cycle Processes, which focuses on processes. SEBoK Part 3 Systems Engineering and Management, which addresses the major SE technical and management processes, is also organized around 15288 process areas. In this view, SE is process oriented. Each edition of the SE Handbook aligns to an ISO/IEC/IEEE 15288 edition. Figure 1 shows the 15288 processes and how they align with the SE Handbook and SEBoK topic areas. Later in this article, these SEBoK topics and knowledge areas are compared with the knowledge areas of IE. The knowledge areas in Figure 1 align with the system life cycle

if started at the top of the first column and traversed to the bottom, continued at the bottom of the second column and traversed to the top, and then continued at the bottom of the third column and traversed to the top.

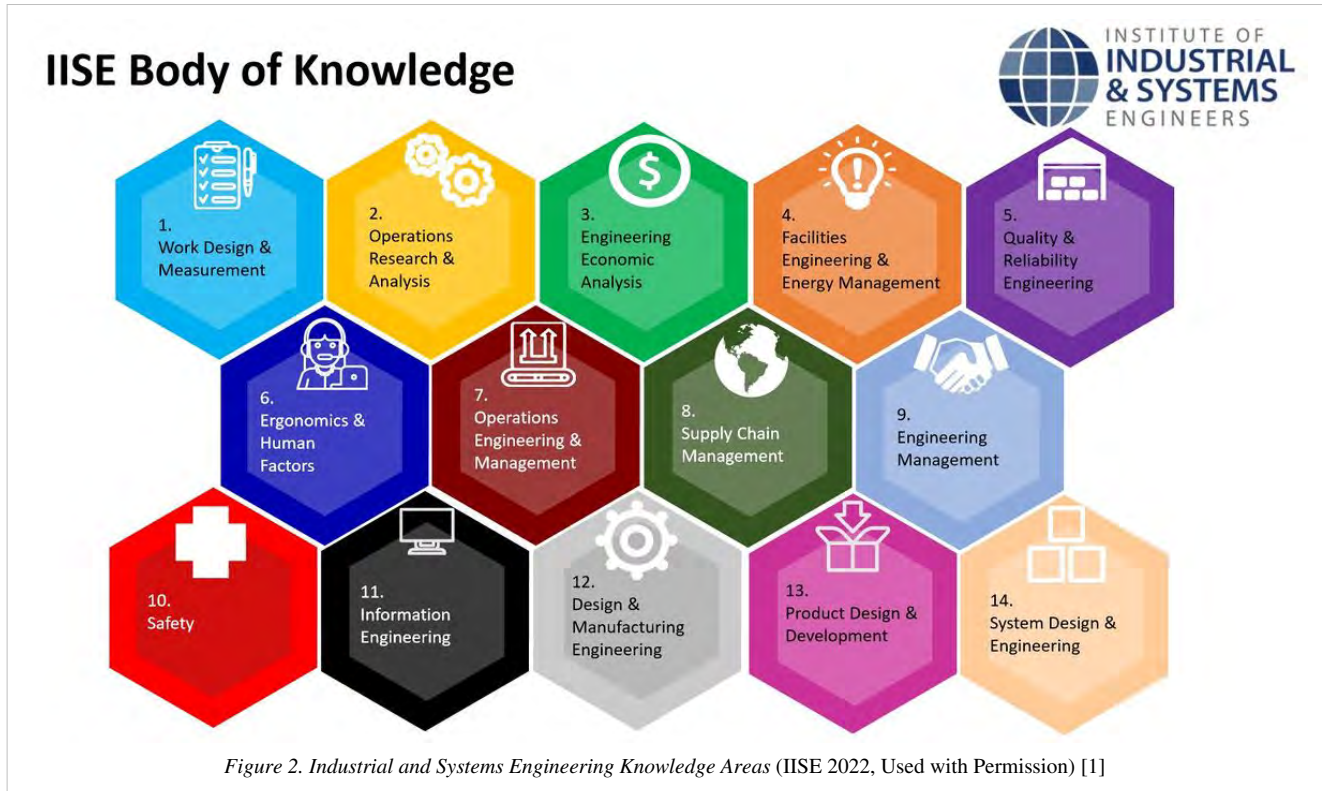


Industrial Engineering

The Institute of Industrial and Systems Engineers (IISE) states that it is “the only international, non-profit, professional society dedicated to advancing the technical and managerial excellence of industrial engineers.” (IISE 2021). IISE started in 1948 as the American Institute of Industrial Engineers. In 1981, the organization was renamed the Institute of Industrial Engineers to reflect its growing international membership. In 2016, the membership voted to change the name to the Institute of Industrial and Systems Engineers. This addition reflects a vote by its membership and aligns with the “changing scope of the profession that, while keeping its industrial base, has seen more industrial and systems engineers working with large scale integrated systems in a variety of sectors”.

At the turn of this century, industrial engineering was well reflected in two prominent publications: *Handbook of Industrial Engineering* (Salvendy 2001) and the fifth edition of *Maynard's Industrial Engineering Handbook* (Zandin 2001). Salvendy (2001) stated that industrial engineers are trained to design and analyze the components of which man-machine systems are composed. They bring together individual elements that are designed via other engineering disciplines and properly synergize these subsystems together with the people components for a completely integrated man-machine system. Industrial engineers are focused on the improvement of any system that is being designed or evaluated. They make individual human tasks more productive and efficient by optimizing flow, eliminating unnecessary motions, utilizing alternate materials to improve manufacturing, improving the flow of product through processes, and optimizing the configuration of workspaces. Fundamentally, the industrial engineer is charged with reducing costs and increasing profitability through ensuring the efficient use of human, material, physical, and/or financial resources.

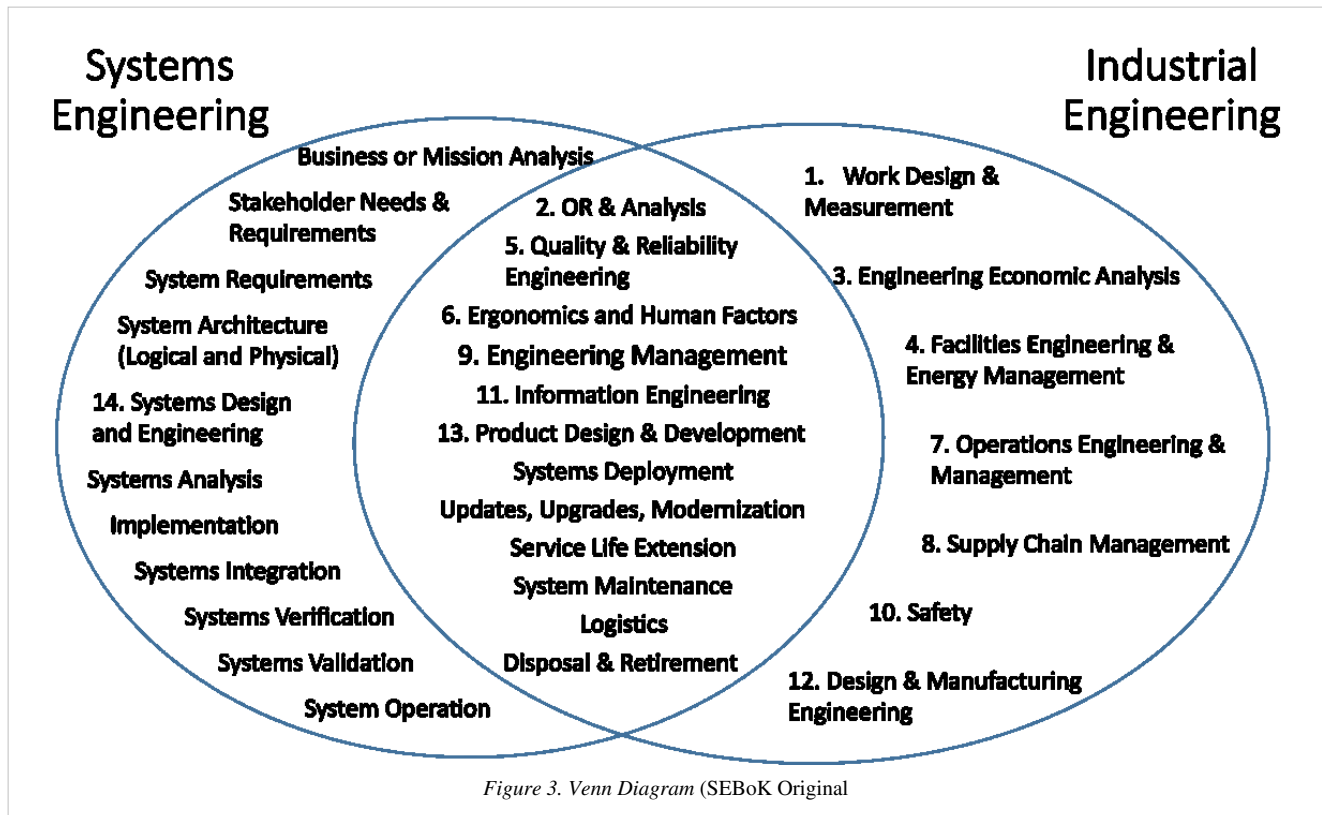
The view of IE has evolved over the last two decades. IISE developed the IISE Body of Knowledge in 2021 (IISE 2021). The sixth edition of *Maynard's Industrial Engineering Handbook* (Zandin 2022) is expected to be published in 2022. The IISEBoK has 14 knowledge areas, as shown in Figure 2. The first 13 knowledge areas identify the Industrial Engineering knowledge. The fourteenth is Systems Design and Engineering, which references the SEBoK. The IISEBoK provides a short description of each knowledge area, a detailed outline of knowledge area topics, and a list of references. The IISEBoK does not use standards as its foundation. In fact, the Standard Practice for Systems Safety (MIL-STD-0-882D) is the only standard cited in the reference section. IISE does not currently have a handbook developed by or for the Institute, although Zandin (2022) is expected to align with the IISEBoK.



The 14 topic areas included in the IISEBoK could be tied to many international standards even though the IISEBoK does not use standards as its foundation or provide references to standards in most of its topic areas.

Venn Diagram Comparison

This section compares the two bodies of knowledge. Figure 3 is a Venn Diagram that identifies knowledge areas that are usually performed by systems engineers, ones usually performed by industrial engineers, and ones that are used by both disciplines.



There are 11 primarily SE knowledge areas, 7 primarily IE knowledge areas, and 12 overlapping knowledge areas. Table 1 provides some illustrative examples of the differences in SE and IE focus in the overlapping knowledge areas.

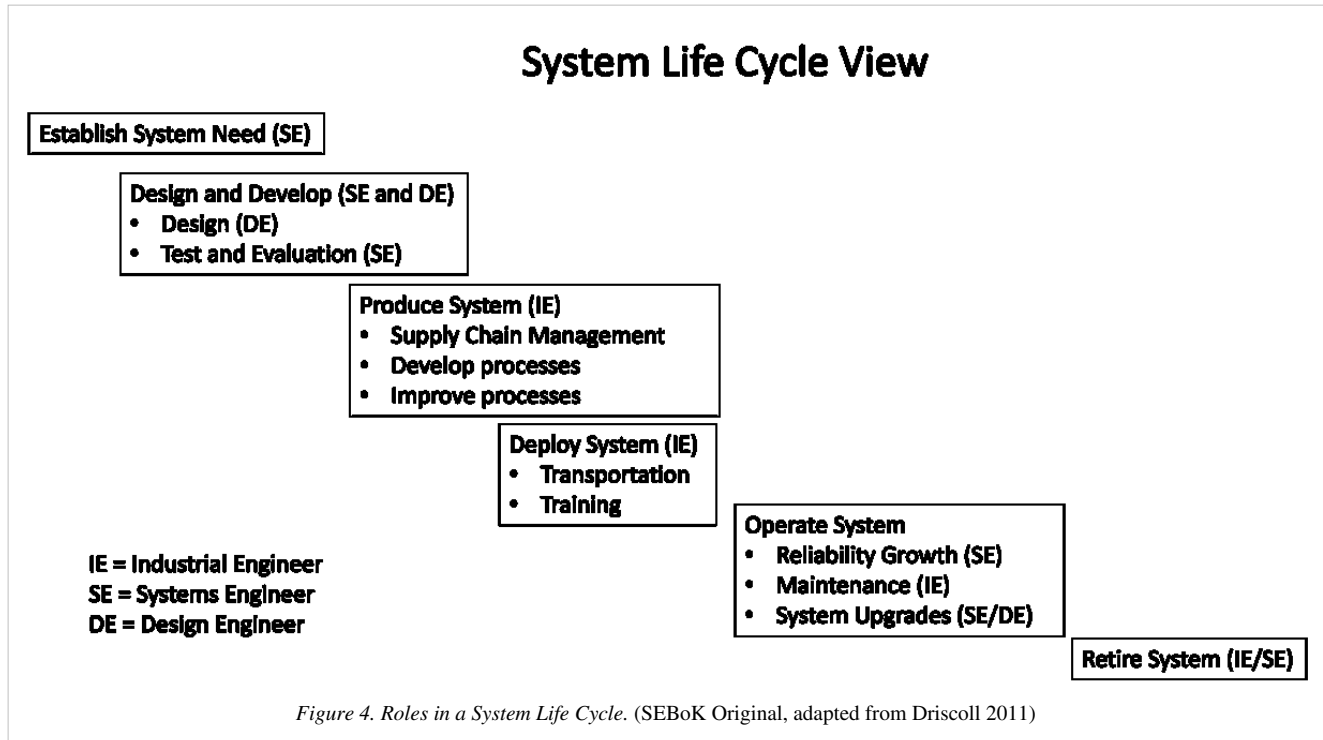
Table 1. Illustrative Examples of Differences in SE and IE Focus

Knowledge Area	Systems Engineering Focus	Industrial Engineering Focus
2. Operations Research (OR) and Analysis	OR and analysis is used in systems analysis to assess system performance and to evaluate system designs (Levis and Wagenhals 2000; Wagenhals, Shin, Kim, and Levis 2000; Wagenhals, Haider, and Levis 2003; Raz, Kenley, and DeLaurentis 2018)	OR and analysis is used to optimize operations, maintenance, and logistics. OR is also used to evaluate and optimize manufacturing systems.
5. Quality & Reliability Engineering	Quality and reliability requirements are treated as system-wide performance requirements (Buede and Miller 2016: 157-159; Wymore 1993: 401) that are assessed at the systems level. One example is availability, A_o , which measures the degree to which a system is either operating or can operate at any time when used in its typical operational and support environment. (DA PAM 70-3 2008: 87)	Quality and reliability are used to evaluate and improve the manufacturing process of goods and services. Reliability is also used to evaluate and improve system operations.

6. Ergonomics and Human Factors	Ergonomics and human factors are considerations in assessing the potential usability of a system for end-users in an operational environment. According to Buede and Miller (2016: 180), "Performance elements of usability are ease of learning (learnability), ease of use (efficiency), ease of remembering (memorability), error rate, and subjectively pleasing (satisfaction)."	Ergonomics and human factors are used to assess and improve manufacturing and operational processes. They are also used to assess and improve the actual usability of products and services.
9. Engineering Management	Engineering managers and SEs work with program managers to develop and improve new products and services.	Engineering managers and IEs support operations managers responsible for manufacturing processes and the operation of systems providing products and services.
11. Information Engineering	Information Engineering is critical to the development of new products and services that are increasingly software intensive. Information engineering is relevant for model-based systems engineering software tools and databases, requirements management software tools and databases, and overall configuration management of the systems design and requirements baseline.	The Information Engineering knowledge area focuses on using data in information systems to facilitate decision-making and business communication.
13. Product Design and Development	SE focuses on the design of systems that provide products and services and the system life cycle. SE includes the technical processes for system design and verification and the technical management processes for project planning, assessment, and control; risk management; and decision management in the <i>Systems Engineering Handbook</i> (2015: 47-83, 104-121).	The knowledge area of the ISEBoK (2021: 53-46) focuses on the design of products and the product life cycle. It closely parallels the technical processes for system design and development of SEBoK.
Systems Deployment	Systems engineers participate in defining requirements, defining the architecture, and verification and validation of deployment systems needed to the deploy the system of interest, e.g., special transport equipment such as the Shuttle Carrier Aircraft (SCA) that NASA used to transport Space Shuttle orbiters. (Jenkins 2000)	Industrial engineers are more focused on air, ground, water, and intermodal logistics to include transportation and distribution of systems and products.
Updates, Upgrades, Modernization	Systems engineers are involved defining requirements, defining the architecture, and verification and validation of updates, upgrades, and modernization of systems. One way that this has been explained is that a second iteration of the systems engineering V-model is completed as the systems remains in service while a system change project is implemented (Ven, Talik, and Hulse 2012).	Industrial engineers are involved in manufacturing processes and supporting operations of systems to provide goods and services. Industrial engineers can help identify the need for updates, upgrades, and modernization of manufacturing and service processes and work with engineering managers, systems engineers, and design engineers to provide improved capabilities.
Service Life Extension	Systems engineers are involved in service life extension efforts in the same way that they are involved in updates, upgrades, and modernization of systems.	Industrial engineers are involved in service life extension efforts in the same way that they are involved in updates, upgrades, and modernization of systems.
System Maintenance	Systems engineers participate in defining requirements for maintenance across the life cycle of the system, determine the impact of maintenance constraints on the system requirements and the system architecture (Walden, et al. 2015: 97-98).	Industrial engineers provide engineering support to production processes maintenance and system maintenance to sustain operation of production and service processes and systems.
Logistics	Systems engineers participate in defining requirements for logistics across the life cycle of the system, determine the impact of maintenance constraints on the system requirements and the system architecture (Walden, et al. 2015: 97-98).	Industrial engineers are very involved in logistics planning and operations including supply chain management, transportation, and distribution.
Disposal and Retirement	Systems engineers identify requirements, define the architecture, and verification and validation of disposal and retirement needed to the disposition or retire the system of interest, e.g., nuclear material stabilization processes and equipment needed to disposition fissile nuclear materials to enable shutdown of the nuclear production facilities (Kenley, et al. 1999).	Industrial engineers plan for disposal and retirement as part of their product design process. Increasingly, industrial engineers must consider environmental impact and sustainability issues.

Roles in a System Life Cycle

Systems engineers and industrial engineers play important roles in a system life cycle. Figure 4 modifies a format from Buede and Miller (2016). It shows the system life cycle stages and, based on analysis in the previous section, identifies and summarizes the major roles of systems engineers, industrial engineers, and design engineers. Some processes have been aggregated to simplify the figure.



Summary

In summary:

- The SEBoK *SE and Management* Part is based on an ISO standard. IE has several related ISO standards. IISE does not link its body of knowledge to standards.
- The SEBoK is more process focused, while IISEBoK focuses more on concepts and techniques.
- SE and IE have overlapping bodies of knowledge.
- The SEBoK and INCOSE's SE Handbook align with the system life cycle. The IISEBoK does not have an analogous organizing structure.
- Systems engineers and industrial engineers both play important roles in the system life cycle with some overlapping responsibilities.

References

Works Cited

- Buede, D. M., and W. D. Miller. *The Engineering Design of Systems: Models and Methods*. Hoboken, NJ: Wiley. 2016.
- Department of the Army. *Army Acquisition Procedures*. DA PAM 70-3, January 28, 2008. Washington, DC: Department of the Army. 2008.
- Environmental Protection Agency. *Guidelines for Preparing Economic Analyses*. Accessed November 19, 2021. <https://www.epa.gov/environmental-economics/guidelines-preparing-economic-analyses>.
- IISE (Institute of Industrial and Systems Engineers). *IISE Body of Knowledge*. 2021. <https://www.iise.org/BodyofKnowledge>.
- Institute of Industrial and Systems Engineers. "Origins of IISE." Accessed November 13, 2021. <https://www.iise.org/details.aspx?id=295>.
- INCOSE (International Council on Systems Engineering). "What Is Systems Engineering?" Accessed February 17, 2022. <https://www.incose.org/about-systems-engineering/system-and-se-definition/systems-engineering-definition>.
- ISO (International Organization for Standardization). "Standards Website." Accessed November 14, 2021. <https://www.iso.org/standards.html>.
- ISO/IEC/IEEE. *Systems and Software Engineering – System Life Cycle Processes*. Geneva: International Organization for Standardization. 2015.
- Jenkins, D. R. *Boeing 747-100/200/300/SP*. Airliner Tech Series, vol. 6. North Branch, MN: Specialty Press. 2000.
- Kenley, B., B. Scott, B. Seidel, D. Knecht, F. Southworth, K. Osborne, N. Chipman, and T. A. Creque. "Program to Stabilize Nuclear Materials as Managed by the Plutonium Focus Area." *Proceedings of Waste Management 1999*. Tucson, AZ. 1999.
- Levis, A. H., and L. W. Wagenhals. "C4ISR Architectures: I. Developing a Process for C4ISR Architecture Design." *Systems Engineering* 3, no. 4: 225–247. 2000.
- Raz, A. K., C. R. Kenley, and D. A. DeLaurentis. "System Architecting and Design Space Characterization." *Systems Engineering* 21, no. 3: 227–242. 2018.
- Salvendy, G., ed. *Handbook of Industrial Engineering, Technology and Operations Management*. 3rd ed. Hoboken, NJ: John Wiley & Sons. 2001.
- Van De Ken, M., J. Talik, and J. Hulse. "An Introduction to Applying Systems Engineering to In-Service Systems." *Proceedings of the 2012 INCOSE International Symposium* 22, no. 1: 879–894. Rome, Italy. 2021. <https://doi.org/10.1002/j.2334-5837.2012.tb01377.x>.
- Wagenhals, L. W., S. Haider, and A. H. Levis. "Synthesizing Executable Models of Object-Oriented Architectures." *Systems Engineering* 6, no. 4: 266–300. 2003.
- Wagenhals, L. W., I. Shin, D. Kim, and A. H. Levis. "C4ISR Architectures: II. A Structured Analysis Approach for Architecture Design." *Systems Engineering* 3, no. 4: 248–287. 2000.
- Walden, D. D., G. J. Roedler, K. J. Forsberg, R. D. Hamelin, and T. M. Shortell, eds. *INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 4th ed. Hoboken, NJ: Wiley. 2015.
- Wymore, A. W. *Model-Based Systems Engineering: An Introduction to the Mathematical Theory of Discrete Systems and to the Tricategory Theory of System Design*. Boca Raton, FL: CRC Press. 1993.
- Zandin, K. B., ed. *Maynard's Industrial Engineering Handbook*. 5th ed. New York: McGraw-Hill. 2001.

Primary References

Salvendy, G., ed. *Handbook of Industrial Engineering, Technology and Operations Management*. 3rd ed. Hoboken, NJ: John Wiley & Sons. 2001.

Zandin, K. B., ed. *Maynard's Industrial Engineering Handbook*. 5th ed. New York: McGraw-Hill. 2001.

Additional References

none.

References

[1] <https://www.iise.org/BodyofKnowledge>

Knowledge Area: Systems Engineering and Project Management

Systems Engineering and Project Management

Contents of this Knowledge Area

- The Nature of Project Management (Heidi Davidz) (Richard Turner and Alice Squires)
- An Overview of the PMBOK® Guide (Karen Wisne, Filippo Maria Ottaviani, Marvin Hammond, Maggie Blakeney, Tina P. Srivastava, Steve Dam, and Tikajit Rai)
- Relationships between Systems Engineering and Project Management (Richard Turner) (Alice Squires)
- The Influence of Project Structure and Governance on Systems Engineering and Project Management Relationships (Alice Squires)
- Cost Estimating and Analysis in Systems Engineering (Gan Wang)
- Procurement and Acquisition (Dick Fairley) (Alice Squires)
- Portfolio Management (Eric Specking, Gregory S. Parnell, and Ed Pohl)
- Lead Author:
- Dick Fairley
- Contributing Authors:
- Richard Turner and Alice Squires

The goal of project management is to plan and coordinate the work activities needed to deliver a satisfactory product, service, or enterprise endeavor within the constraints of schedule, budget, resources, infrastructure, and available staffing and technology. The purpose of this knowledge area (KA) is to acquaint systems engineers with the elements of project management and to explain the relationships between systems engineering (SE) and project management (PM).

References

Works Cited

None.

Primary References

Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, NJ: John Wiley & Sons. 2009.

Forsberg, K., H. Mooz, and H. Cotterman. *Visualizing Project Management*, 3rd ed. New York: John Wiley & Sons. 2005.

PMI (Project Management Institute). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA: Project Management Institute. 2013.

Additional References

None.

The Nature of Project Management

- Lead Author:
- Heidi Davidz
- Contributing Authors:
- Richard Turner and Alice Squires

While *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)* provides an overview of project management for those seeking PMI certification, Fairley (2009) and Forsberg (2005) suggest another way to characterize the important aspects of project management:

- Planning and Estimating
- Measuring and Controlling
- Leading and Directing
- Managing Risk

Introduction

Project managers and systems engineers are both concerned with management issues such as planning, measuring and controlling, leading, directing, and managing risk. In the case of project managers, the project attributes to be managed include project plans; estimates; schedule; budget; project structure; staffing; resources; infrastructure; and risk factors. Product attributes managed by systems engineers include items such as requirements allocation and flow-down; system architecture; structure of and interactions among technical teams; specialty engineering; integration; verification; and validation.

The exact allocation of the SE and PM duties depend on many factors, such as customer and stakeholder interactions, organizational structure of the parent organization, and relationships with affiliate contractors and subcontractors. (See the article on The Influence of Project Structure and Governance on Systems Engineering and Project Management Relationships in this KA.)

Planning and Estimating

Planning

Project Planning a project involves providing answers to the who, what, where, when, and why of every project:

- **Who:** Addresses staffing issues (competencies, numbers of staff, communication and coordination)
- **What:** Addresses the scope of activities
- **Where:** Addresses issues of locale (local, geographically distributed)
- **When:** Addresses scheduling issues
- **Why:** Addresses rationale for conducting a project

Guidance for developing project plans can be found in INCOSE (2012), NASA (2007), and ISO/IEC/IEEE Standard 16326:2009. It is often observed that communication and coordination among stakeholders during project planning are equally as important as (and sometimes more important than) the documented plan that is produced.

In defense work, event-driven integrated master plans and time-driven integrated master schedules are planning products. Chapter 11 of the Defense Acquisition Guidebook provides details (DAU 2010).

Estimating

Estimation is an important element of planning. An estimate is a projection from past to future, adjusted to account for differences between past and future. Estimation techniques include analogy, rule of thumb, expert judgment, and use of parametric models such as the PRICE model for hardware, COCOMO for software projects and COSYSMO for systems projects (Stewart 1990; Boehm et al. 2000; Valerdi 2008).

Entities estimated include (but are not limited to) schedule, cost, performance, and risk.

Systems engineering contributes to project estimation efforts by ensuring that:

- the overall system life cycle is understood;
- dependencies on other systems and organizations are identified;
- the logical dependencies during development are identified; and
- resources and key skills are identified and planned.

Additionally, high-level system architecture and risk assessment provide the basis for both the work breakdown structure and the organizational breakdown structure.

Measuring and Controlling

Measuring and controlling are the key elements of executing a project. Measurement includes collecting measures for work products and work processes. For example, determining the level of coverage of requirements in a design specification can be assessed through review, analysis, prototyping, and traceability. Effort and schedule expended on the work processes can be measured and compared to estimates; earned value tracking can be used for this purpose. Controlling is concerned with analyzing measurement data and implementing corrective actions when actual status does not align with planned status.

Systems engineers may be responsible for managing all technical aspects of project execution, or they may serve as staff support for the project manager or project management office. Organizational relationships between systems engineers and project managers are presented in Team Capability. Other organizational considerations for the relationships between systems engineering and project management are covered in the Enabling Systems Engineering knowledge area.

Additional information on measurement and control of technical factors can be found in the Measurement and Project Assessment and Control articles in Part 3: Systems Engineering and Management.

Leading and Directing

Leading and directing requires communication and coordination among all project stakeholders, both internal and external. Systems engineers may be responsible for managing all technical aspects of project execution, or they may serve as staff support for the project manager or project management office. Organizational relationships between systems engineers and project managers are presented in the article Team Capability in Part 5. Other organizational considerations for the relationships between systems engineering and project management are discussed in Part 5: Enabling Systems Engineering.

Managing Risk

Risk management is concerned with identifying and mitigating potential problems before they become real problems. Systems engineering projects are, by nature, high-risk endeavors because of the many unknowns and uncertainties that are inherent in projects. Because new risk factors typically emerge during a project, ongoing continuous risk management is an important activity for both systems engineers and project managers.

Potential and actual problems may exist within every aspect of a project. Systems engineers are typically concerned with technical risk and project managers with programmatic risk. Sometimes, technical risk factors are identified and confronted by systems engineers and programmatic risk factors are identified and confronted by project managers without adequate communication between them. In these cases, appropriate trade-offs among requirements, schedule, budget, infrastructure, and technology may not be made, which creates additional risk for the successful outcome of a project.

In the last ten years, there has been an increasing interest in opportunity management as the converse of risk management. Hillson (2003), Olsson (2007), and Chapman and Ward (2003) provide highly cited introductions.

Additional information on risk management for systems engineering projects can be found in the Risk Management article in Part 3: Systems Engineering and Management.

References

Works Cited

- Boehm, B., C. Abts, A. W. Brown, S. Chulani, B. K. Clark, E. Horowitz, R. Madachy, D. Reifer, and B. Steece. 2000. *Software Cost Estimation with COCOMO II*. Upper Saddle River, NJ: Prentice Hall.
- Chapman, C., and S. Ward. *Project Risk Management: Processes, Techniques and Insights*. Chichester, West Sussex: John Wiley & Sons. 2003.
- Defense Acquisition University (DAU). *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA: DAU/U.S. Department of Defense, February 19, 2010.
- Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, NJ: John Wiley & Sons. 2009.
- Forsberg, K., H. Mooz, and H. Cotterman. *Visualizing Project Management*. Hoboken, NJ: John Wiley & Sons. 2005.
- Hillson, D. *Effective Opportunity Management for Projects: Exploiting Positive Risk*. Boca Raton, FL: CRC Press. 2003.
- International Council on Systems Engineering (INCOSE). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA: INCOSE. INCOSE-TP-2003-002-03.2.2. 2012.
- ISO/IEC/IEEE. *ISO/IEC/IEEE 16326:2009(E), Systems and Software Engineering – Life Cycle Processes – Project Management*. Geneva: International Organization for Standardization/International Electrotechnical Commission/Institute of Electrical and Electronics Engineers. 2009.
- NASA. *Systems Engineering Handbook*. Washington, DC: National Aeronautics and Space Administration. 2007.
- Olsson, Rolf. "In Search of Opportunity Management: Is the Risk Management Process Enough?" *International Journal of Project Management* 25, no. 8 (2011): 745–752. 2007.
- Stewart, Rodney. *Cost Estimating*. New York: Wiley. 1990.
- Valerdi, R. *The Constructive Systems Engineering Cost Model (COSYSMO): Quantifying the Costs of Systems Engineering Effort*. Saarbrücken, Germany: VDM Verlag.
-

Primary References

Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, NJ: John Wiley & Sons. 2009.

Project Management Institute (PMI). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA: PMI. 2013.

Additional References

Blanchard, B. *System Engineering Management*. Hoboken, NJ: John Wiley & Sons. 2008.

Kerzner, Harold. *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*. 8th ed. Hoboken, NJ: John Wiley & Sons. 2003.

Martin, J. *Systems Engineering Guidebook: A Process for Developing Systems and Products*. London: Taylor & Francis CRC Press. 1997.

An Overview of the PMBOK® Guide

- Lead Authors:
- Karen Wisne, Filippo Maria Ottaviani, Marvin Hammond, Maggie Blakeney, Tina P. Srivastava, Steve Dam, and Tikajit Rai

The Project Management Institute, Inc. (PMI) defines a project as a temporary endeavor undertaken to create a unique product, service, or result. The Guide to the Project Management Body of Knowledge (PMBOK® Guide) is PMI's primary reference that consolidates generally recognized project management (PM) terminology and proven practices.

The PMBOK® Guide is often used as a baseline for technical management processes within systems engineering (SE) projects. It supports the integration of PM frameworks with technical planning, the alignment of project life cycle stages with the system development life cycle (SDLC), the clarification of interface responsibilities across integrated product teams (IPTs), and the shaping of work breakdown structures (WBS) and schedule logic during concept development and detailed design.

The PMBOK® Guide is also the basis for certification exams to qualify Project Management Professionals (PMPs). Many organizations require PMP certification as a basic qualification for the role of project manager.

This article provides overview of the 8th edition of the PMBOK® Guide (PMI 2025).

Fundamental Concepts

Overview

The PMBOK® Guide's 8th Edition combines the 6th and 7th editions to provide both a thorough understanding of individual processes and a broader view of how those processes interact to achieve desired project results, making the PMBOK® Guide more adaptable and useful for project managers working across a variety of methods and in diverse environments.

It preserves a principle-based, value-focused perspective that emphasizes outcomes, and its guidance integrates five domain areas, six principles, and seven performance domains.

Domain Areas

The 8th edition of the PMBOK® Guide re-configures the process groups from the 6th edition of the PMBOK® Guide into a streamlined framework, branded as Project Management Focus Areas. The goal is to preserve a recognizable workflow for practitioners while eliminating the lengthy descriptions of ITTO (inputs, tools, techniques, and outputs) that turned prior editions into memorization-heavy lookup references. There are five domain areas:

- Initiating;
- Planning;
- Executing;
- Monitoring and Controlling; and
- Closing.

Principles

The 8th Edition of the PMBOK® Guide defines and promotes six core principles:

- Adopting a holistic view;
- Focusing on value;
- Embedding quality across processes and deliverables;
- Fostering accountable leadership;
- Integrating sustainability; and
- Building a culture of empowerment.

Performance Domains

The 8th Edition of the PMBOK® Guide is organized around seven project performance domains. These domains are key areas that drive outcomes and apply regardless of the project management approach, whether predictive, incremental, iterative, or adaptive. The seven performance domains covered in the PMBOK® Guide include the following:

- Governance;
- Scope;
- Schedule;
- Finance;
- Stakeholders;
- Resources; and
- Risk.

These seven domains differ from the eight presented in the 7th edition, which moved away from the process group perspective that characterized the 6th edition and earlier versions. The 7th edition instead identified eight performance domains: Stakeholders, Team, Development Approach and Life Cycle, Planning, Project Work, Delivery, Measurement, and Uncertainty.

Like the 7th edition, the 8th edition places great emphasis on tailoring—recognizing that no two projects or organizations are the same—and cautions that applying any framework without adapting it to the appropriate context yields rigid, bureaucratic processes for the sake of process.

Practical Considerations

Pitfalls

The PMBOK® Guide is not intended to dictate a rigid methodology. The principles, domains, and processes are intended to be tailored to the specific details of each project's complexity, customer(s), and goals.

Additionally, following the PMBOK guidance for the sake of process alone may contribute to failed projects which lack context, leadership, or value.

Lastly, because the PMBOK® Guide is industry-agnostic, it lacks the technical domain specificity required for complex systems. Therefore, clarifying the relationship between PM and SE is essential to ensure management and engineering life cycles are aligned.

Good Practices

The concepts presented in the 8th Edition of the PMBOK® Guide are intended to be treated as tools, to be applied in conjunction with SDLC models and not as a standalone methodology. Project artifacts can be aligned with their corresponding SE artifacts by mapping WBS elements and milestones to technical work packages, reviews, and decision gates defined in the SDLC. Conversely, SE artifacts — such as technical risk assessments, interface control documents (ICDs), and systems engineering management plan (SEMP) updates — can be fed back into the project risk domain to support integrated, full-cycle decision making.

References

Works Cited

Project Management Institute (PMI). *A Guide to the Project Management Body of Knowledge and The Standard for Project Management*. 8th ed. *PMBOK® Guide*. Newtown Square, PA: Project Management Institute. 2025.

Primary References

Project Management Institute (PMI). *A Guide to the Project Management Body of Knowledge and The Standard for Project Management*. 8th ed. *PMBOK® Guide*. Newtown Square, PA: Project Management Institute. 2025.

Additional References

Blanchard, Benjamin S., and John E. Blyler. *System Engineering Management*. 5th ed. Hoboken, NJ: John Wiley & Sons. 2016.

Fairley, Richard E. *Managing and Leading Software Projects*. Hoboken, NJ: John Wiley & Sons. 2009. Reprint, 2011.

Martin, James N. *Systems Engineering Guidebook: A Process for Developing Systems and Products*. Taylor and Francis Group, CRC-Press, LLC. 1997. Reprint, 2020.

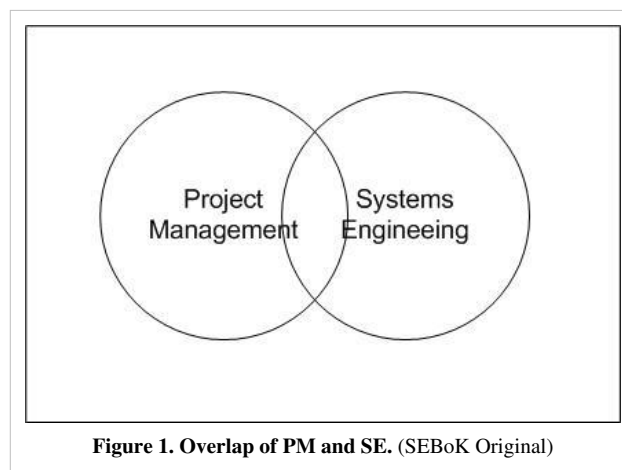
Relationships between Systems Engineering and Project Management

- Lead Author:
 - Richard Turner
- Contributing Author:
 - Alice Squires

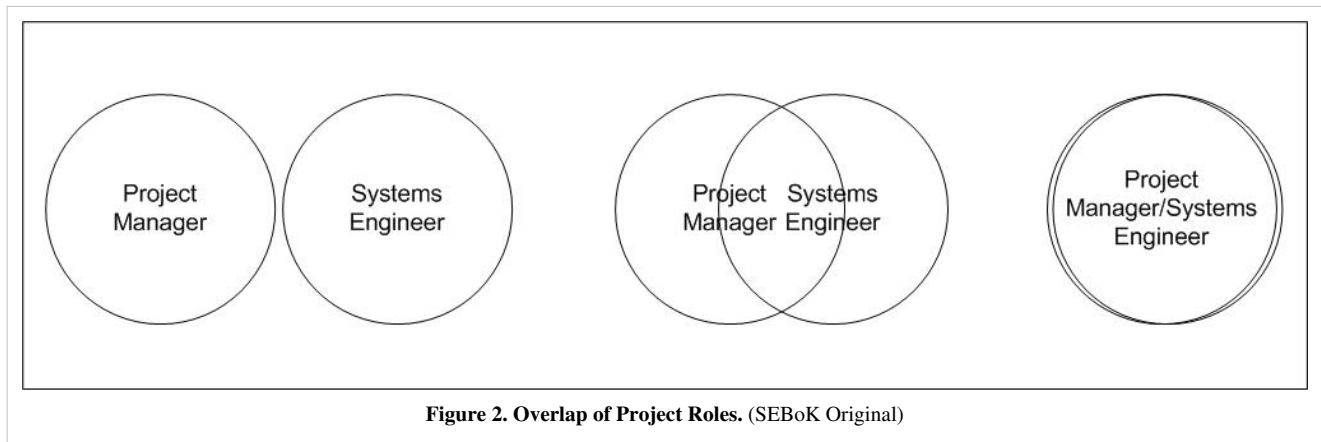
This topic discusses the relationship between systems engineering (SE) and project management (PM). As with software engineering, there is a great deal of overlap. Depending on the environment and organization, the two disciplines can be disjoint, partially intersecting, or one can be seen as a subset of the other. While there is no standard relationship, the project manager and the systems engineer encompass the technical and managerial leadership of a project between them, which requires the enterprise of each project manager and system engineer to work out the particular details for their own context.

Overlap

There is a great deal of significant overlap between the scope of systems engineering, as described here (in the SEBoK), CMMI (2011), and other resources and the scope of project management, as described in the *PMBOK® Guide* (PMI 2013), CMMI (2011), and other resources as illustrated in Figure 1.



These sources describe the importance of understanding the scope of the work at hand, how to plan for critical activities, how to manage efforts while reducing risk, and how to successfully deliver value to a customer. The systems engineer working on a project will plan, monitor, confront risk, and deliver the technical aspects of the project, while the project manager is concerned with the same kinds of activities for the overall project. Because of these shared concerns, at times there may be confusion and tension between the roles of the project manager and the systems engineer on a given project. As shown in Figure 2, on some projects there is no overlap in responsibility. On other projects, there may be shared responsibilities for planning and managing activities. In some cases, particularly for smaller projects, the project manager may also be the lead technical member of the team performing both roles of project manager and systems engineer.



Defining Roles and Responsibilities

Regardless of how the roles are divided up on a given project, the best way to reduce confusion is to explicitly describe the roles and responsibilities of the project manager and the systems engineer, as well as other key team members. The Project Management Plan (PMP) and the Systems Engineering Management Plan (SEMP) are key documents used to define the processes and methodologies the project will employ to build and deliver a product or service.

The PMP is the master planning document for the project. It describes all activities, including technical activities, to be integrated and controlled during the life of the program. The SEMP is the master planning document for the systems engineering technical elements. It defines SE processes and methodologies used on the project and the relationship of SE activities to other project activities. The SEMP must be consistent with and evolve in concert with the PMP. In addition, some customers have technical management plans and expectations that the project's SEMP integrate with customer plans and activities. In the U.S. Department of Defense, most government project teams have a systems engineering plan (SEP) with an expectation that the contractor's SEMP will integrate and remain consistent with customer technical activities. In cases where the project is developing a component of a larger system, the component project's SEMP will need to integrate with the overall project's SEMP.

Given the importance of planning and managing the technical aspects of the project, an effective systems engineer will need to have a strong foundation in management skills and prior experience, as well as possess strong technical depth. From developing and defending basis of estimates, planning and monitoring technical activities, identifying and mitigating technical risk, and identifying and including relevant stakeholders during the life of the project, the systems engineer becomes a key member of the project's management and leadership team. Additional information on Technical Management Processes and Stakeholder Needs Definition can be found in Part 3: Systems Engineering and Management.

Practical Considerations

Effective communication between the project manager and the system engineer is essential for mission accomplishment. This communication needs to be established early and occur frequently.

Resource reallocation, schedule changes, product/system changes and impacts, risk changes: all these and more need to be quickly and clearly discussed between the PM and SE.

References

Works Cited

CMMI. *CMMI for Development: Guidelines for Process Integration and Product Improvement*. Old Tappan, NJ: Pearson Education. 2011.

Project Management Institute (PMI). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA, USA: Project Management Institute. 2013.

Primary References

Chrissis, M. B., M. Konrad, and S. Shrum. *CMMI for Development: Guidelines for Process Integration and Product Improvement*, 3rd ed. Boston, MA: Addison-Wesley Professional. 2011.

PMI. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA: Project Management Institute. 2013.

Additional References

NASA. *Systems Engineering Handbook*, Revision 1. Washington, DC, USA: National Aeronautics and Space Administration. NASA/SP-2007-6105. 2007.

The Influence of Project Structure and Governance on Systems Engineering and Project Management Relationships

- Lead Author:
- Alice Squires

This article reviews various project structures that impact or provide governance to the project and that require key involvement from the program manager and the systems engineer. These structures include: the structure of the organization itself (functional, project, matrix, and specialized teams, such as Integrated Product Teams (IPTs), Change Control Boards (CCBs), and Engineering Review Boards (ERBs). This article also addresses the influence of schedule-driven versus requirements-driven projects on these structures.

Relationships between Systems Engineering and Project Management are covered in a related article.

An Overview of Project Structures

Project management and systems engineering governance are dependent on the organization's structure. For some projects, systems engineering is subordinated to project management and in other cases, project management provides support to systems engineering. These alternatives are illustrated in Figures 1 and 2 of the Organizing the Team section in Team Capability.

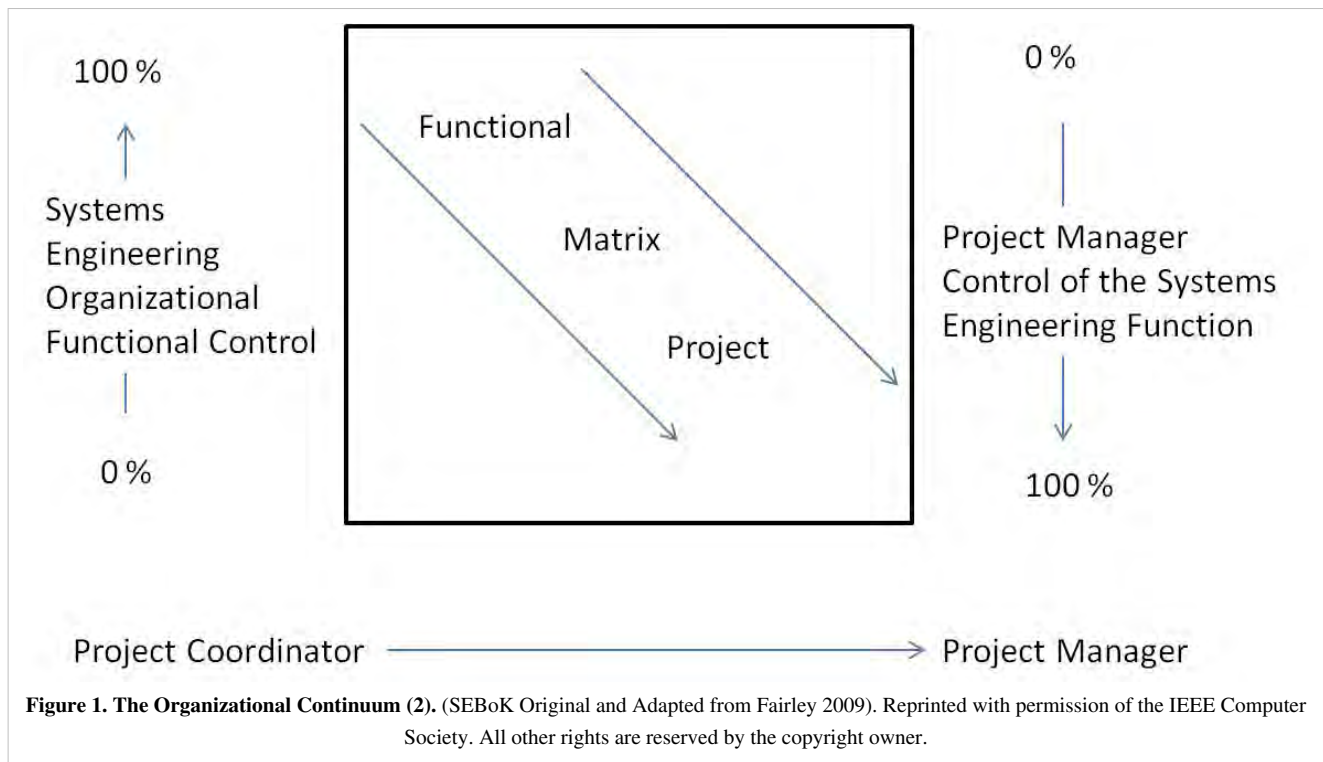
A project exists within the structural model of an organization. Projects are one-time, transient events that are initiated to accomplish a specific purpose and are terminated when the project objectives are achieved. Sometimes, on small projects, the same person accomplishes the work activities of both project management and systems engineering. Because the natures of the work activities are significantly different, it is sometimes more effective to have two persons performing project management and systems engineering, each on a part-time basis. On larger projects there are typically too many tasks to be accomplished for one person to accomplish all of the necessary work. Very large projects may have project management and systems engineering offices with a designated project manager and a designated lead systems engineer.

Projects are typically organized in one of three ways: (1) by functional structure, (2) by project structure, and (3) by a matrix structure (see Systems Engineering Organizational Strategy for a fourth structure and related discussion). In a function-structured organization, workers are grouped by the functions they perform. The systems engineering functions can be: (1) distributed among some of the functional organizations, (2) centralized within one organization or (3) a hybrid, with some of the functions being distributed to the projects, some centralized and some distributed to functional organization. The following figure provides an organizational structure continuum and illustrates levels of governance among the functional organizations and the project.

- In a functional-structured organization, the project manager is a coordinator and typically has only limited control over the systems engineering functions. In this type of organization, the functional manager typically controls the project budget and has authority over the project resources. However, the organization may or may not have a functional unit for systems engineering. In the case where there is a functional unit for systems engineering, systems engineers are assigned across existing projects. Trades can be made among their projects to move the priority of a specific systems engineering project ahead of other projects; thus reducing the nominal schedule for that selected project. However, in the case where there is not a functional unit for systems engineering, the project manager may have to find alternate sources of staffing for systems engineering – for example, hiring systems engineering talent or consultants, promoting or expanding the responsibilities of a current team member, etc.
 - In a project-structured organization, the project manager has full authority and responsibility for managing the budget and resources to meet the schedule requirements. The systems engineer is subject to the direction of the
-

project manager. The project manager may work with human resources or a personnel manager or may go outside the organization to staff the project.

- Matrix-structured organization can have the advantages of both the functional and project structures. For a schedule driven project, function specialists are assigned to projects as needed to work for the project manager to apply their expertise on the project. Once they are no longer needed, they are returned to their functional groups (e.g. home office). In a weak matrix, the functional managers have authority to assign workers to projects and project managers must accept the workers assigned to them. In a strong matrix, the project manager controls the project budget and can reject workers from functional groups and hire outside workers if functional groups do not have sufficient available and trained workers.



In all cases, it is essential that the organizational and governance relationships be clarified and communicated to all project stakeholders and that the project manager and systems engineer work together in a collegial manner.

The Project Management Office (PMO) provides centralized control for a set of projects. The PMO is focused on meeting the business objectives leveraging a set of projects, while the project managers are focused on meeting the objectives of those projects that fall under their purview. PMOs typically manage shared resources and coordinate communication across the projects, provide oversight and manage interdependencies, and drive project-related policies, standards, and processes. The PMO may also provide training and monitor compliance (PMI 2013).

Schedule-Driven versus Requirements-Driven Influences on Structure and Governance

This article addresses the influences on governance relationships between the project manager and the systems engineer. One factor that establishes this relationship is whether a project is schedule-driven or requirements-driven.

In general, a project manager is responsible for delivering an acceptable product/service on the specified delivery date and within the constraints of the specified schedule, budget, resources, and technology.

The systems engineer is responsible for collecting and defining the operational requirements, specifying the systems requirements, developing the system design, coordinating component development teams, integrating the system components as they become available, verifying that the system to be delivered is correct, complete and consistent to its technical specification, and validating the operation of the system in its intended environment.

From a governance perspective, the project manager is often thought of as being a movie producer who is responsible for balancing the schedule, budget, and resource constraints to meet customer satisfaction. The systems engineer is responsible for product content; ergo, the systems engineer is analogous to a movie director.

Organizational structures, discussed previously, provide the project manager and systems engineer with different levels of governance authority. In addition, schedule and requirements constraints can influence governance relationships. A schedule-driven project is one for which meeting the project schedule is more important than satisfying all of the project requirements; in these cases lower priority requirements may not be implemented in order to meet the schedule.

Classic examples of these types of projects are:

- a project that has an external customer with a contractual delivery date and an escalating late delivery penalty, and
- a project for which delivery of the system must meet a major milestone (e.g. a project for an announced product release of a cell phone that is driven by market considerations).

For schedule-driven projects, the project manager is responsible for planning and coordinating the work activities and resources for the project so that the team can accomplish the work in a coordinated manner to meet the schedule. The systems engineer works with the project manager to determine the technical approach that will meet the schedule. An Integrated Master Schedule (IMS) is often used to coordinate the project.

A requirements-driven project is one for which satisfaction of the requirements is more important than the schedule constraint. Classic examples of these types of projects are:

1. exploratory development of a new system that is needed to mitigate a potential threat (e.g. military research project) and
2. projects that must conform to government regulations in order for the delivered system to be safely operated (e.g., aviation and medical device regulations).

An Integrated Master Plan is often used to coordinate event-driven projects.

To satisfy the product requirements, the systems engineer is responsible for making technical decisions and making the appropriate technical trades. When the trade space includes cost, schedule, or resources, the systems engineer interacts with the project manager who is responsible for providing the resources and facilities needed to implement a system that satisfies the technical requirements.

Schedule-driven projects are more likely to have a management structure in which the project manager plays the central role, as depicted in Figure 1 of the Organizing the Team section in Team Capability. Requirement-driven projects are more likely to have a management structure in which the systems engineer plays the central role, as depicted in Figure 2 of the Organizing the Team section in Team Capability.

Along with the Project Management Plan and the Systems Engineering Management Plan, IMP/IMS are critical to this process.

Related Structures

Integrated Product Teams (IPTs), Change Control Boards (CCBs), and Engineering Review Boards (ERBs) are primary examples of project structures that play a significant role in project governance and require coordination between the project manager, systems engineer and other members of the team.

Integrated Product Team

The Integrated Product Team (IPT) ensures open communication flow between the government and industry representatives as well as between the various product groups (see Good Practices in Project Planning). There is typically a top level IPT, sometimes referred to as the Systems Engineering and Integration Team (SEIT) (see Systems Engineering Organizational Strategy), that oversees the lower level IPTs. The SEIT can be led by either the project manager for a specific project or by the systems engineering functional manager or functional lead across many projects. Each IPT consists of representatives from the appropriate management and technical teams that need to collaborate on systems engineering, project management, and other activities to create a high-quality product. These representatives meet regularly to ensure that the technical requirements are understood and properly implemented in the design. Also see Team Capability for more information.

Change Control Board

An effective systems engineering approach includes a disciplined process for change control as part of the larger goal of configuration management. The primary objective of configuration management is to track changes to project artifacts that include software, hardware, plans, requirements, designs, tests, and documentation. Alternatively, a Change Control Board (CCB) with representatives from appropriate areas of the project is set up to effectively analyze, control and manage changes being proposed to the project. The CCB typically receives an Engineering Change Proposal (ECP) from design/development, production, or operations/support and initially reviews the change for feasibility. The ECP may also be an output of the Engineering Review Board (ERB) (see next section). If determined feasible, the CCB ensures there is an acceptable change implementation plan and proper modification and installation procedures to support production and operations.

There may be multiple CCBs in a large project. CCBs may be comprised of members from both the customer and the supplier. As with the IPTs, there can be multiple levels of CCB starting with a top level CCB with CCBs also existing at the subsystem levels. A technical lead typically chairs the CCB; however, the board includes representation from project management since the CCB decisions will have an impact on schedule, budget, and resources.

See Figure 2 under Configuration Management for a flow of the change control process adapted from Blanchard and Fabrycky (2011). See also Capability Updates, Upgrades, and Modernization, and topics included under Enabling Teams. See also the UK West Coast Modernization Project which provides an example where change control was an important success factor.

Engineering Review Board

Another example of a board that requires collaboration between technical and management is the Engineering Review Board (ERB). Examples of ERBs include the Management Safety Review Board (MSRB) (see System Safety). Responsibilities of the ERB may include technical impact analysis of pending change requests (like the CCB), adjudication of results of engineering trade studies, and review of changes to the project baseline. In some cases, the ERB may be the management review board and the CCB may be the technical review board. Alternatively, in a requirement driven organization the ERB may have more influence while in a schedule driven organization the CCB may have more impact.

References

Works Cited

Blanchard, B. S., and W. J. Fabrycky. *Systems Engineering and Analysis*. 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ: Prentice-Hall. 2011.

Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, NJ: IEEE Computer Society, John Wiley & Sons. ISBN 978-0-470-29455-0. 2009.

Project Management Institute (PMI). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 5th ed. Newtown Square, PA: Project Management Institute. 2013.

Primary References

Forsberg, K., H. Mooz, and H. Cotterman. *Visualizing Project Management*. 3rd ed. New York, NY: John Wiley & Sons. 2005.

Additional References

None.

Cost Estimating and Analysis in Systems Engineering

- Lead Author:
- Gan Wang

Cost estimating is a multidisciplinary practice that combines analytical rigor, quantitative analysis, experience, and expert judgment to develop realistic forecasts of future costs. As highlighted in ISO/IEC/IEEE-15288:2023, cost estimating serves diverse purposes across the system life cycle, and is an essential tool for providing stakeholders with insights into resource requirements for system development, acquisition, operations and sustainment. Cost estimating is thus integral to all technical and management processes in systems engineering and project management, and is central to project planning, budgeting, and control, where it is frequently utilized in evaluating trade-offs for analyzing alternative products or services. Cost estimating also plays a pivotal role in the decision-making process for investments and contracting, especially when governmental and/or commercial organizations are involved.

The practice of cost estimating is often considered as a blend of art and science and requires a diverse set of skills and domain expertise, particularly in mathematics, to create a realistic cost forecast for proposed products or services. Historically, cost estimating has been the responsibility of a specialized group of professionals known as cost analysts. However, in recent times, it has increasingly required the involvements and participation of systems engineers to meet the growing demands for deeper product knowledge and system design expertise.

There are established industry best practices and de facto standards. Leading professional associations, including the International Cost Estimating and Analysis Association (ICEAA), with its predecessor, the Society of Cost Estimating and Analysis (SCEA), along with the International Society of Parametric Analysts (ISPA), offer a rich set of references (ICEAA 2024; ISPA 2008; PMI 2021), as well as training courses (ICEAA n.d.; DAU n.d.). Furthermore, different government agencies provide policies and guidelines to communicate requirements and promote best practices (GAO 2020; DoD 2020, 2022; NASA 2015).

This article covers fundamental concepts and key methodologies and processes used in cost estimating and analysis. The goal is to provide a foundational understanding of best practices and to support the seamless integration of these

methodologies into systems engineering processes.

Relationship to Systems Engineering and Project Management

Systems engineering is defined as "a transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems using systems principles and concepts, and scientific, technological, and management methods," according to INCOSE (2021). Integration is the core to systems engineering. It materializes at two different levels.

Firstly, systems engineering integrates the system of interest (SoI) or the prime mission product (PMP) throughout its life cycle, serving as the system design authority from concept definition and design, through development, production, operations, and support. Secondly, systems engineering integrates the project that builds the product, serving as the engineering authority for the project from planning to execution, monitoring, assessment, and control.

Project management involves planning, executing, monitoring and controlling, and closing projects. It encompasses scheduling, resource allocation, risk management, and performance tracking to ensure that the project is completed on time, within budget, and to the specified quality standards.

Cost estimating and analysis is generally considered a practice within the scope of project management, which is a distinctive discipline from systems engineering. However, project management and systems engineering share considerable overlap in scope, especially in the areas of project scope and cost management (INCOSE 2023). The two practices are interdependent and synergistic, particularly in large-scale, complex engineering projects such as those in infrastructure, transportation and mobility, and aerospace and defense systems. In this context, cost estimating plays a crucial role within the project cost management process, underpinning the successful execution and delivery of complex projects.

Furthermore, in practice, engineering cost estimates are generally developed based on the technical scope defined by systems engineers. Cost estimates are heavily influenced by the technical requirements, design complexity, and tasks specified by systems engineering. In other words, systems engineering defines it, cost estimating quantifies it, and project management manages it.

In this context, systems engineers collaborate closely with program or project managers, drawing upon multidisciplinary skills from various fields such as configuration management, information management, risk management, and decision support. Cost estimating and analysis provides a quantitative basis for scope definition, risk management, resource allocation, change management, performance monitoring, trade-off analysis, and informed decision making. As an integral part of both systems engineering and project management, it ultimately ensures the on-time and on-budget completion of complex projects, ultimately leading to the successful delivery of projects and services.

Definitions

- *Cost estimating*: an exercise to *predict* the future cost of an item, product, program or task, through collecting and analyzing historical data and applying quantitative models, techniques, tools, and databases. The purpose of cost estimating is to translate system and functional requirements associated with programs, projects, proposals or processes into reliable budget requirements. The term *cost estimation* is also used, sometime interchangeably with cost estimating. It typically refers to the act or action of developing a cost estimate, while cost estimating refers to the practice applied to estimation.
- *Cost estimate*: is the *product* of a cost estimating process, which specifies the expected labor effort or dollar cost required to perform a stipulated task, or to acquire a specific item. A cost estimate may constitute a single value or a range of values. A cost estimate can normally be broken down to cost elements.
- *Cost element*: an identifiable function, or a common group of functions, which have been established as a separate entity for the purpose of estimating, collecting, controlling, and reporting costs.

- *Cost driver*: typically a physical or performance attribute of a system, or programmatic parameter of a project or contract, that has the most influence on total estimated cost. A cost driver can be another cost quantity.
- *Cost model*: an analytical tool or framework used to systematically to characterize the behavior of the system or program and produce a credible cost estimate. It implements cost estimating methods (see below).
- *Basis of estimate (BOE)*: a document outlining the assumptions, methods, and data used to develop a cost estimate. The BOE explains how the cost estimate is generated and typically includes a detailed description of the work to be performed, the planned approach, the required resources, and the expected schedule or timeline. It is a mandatory component in competitive proposals and formal quotes from potential suppliers to clients.
- *Cost analysis*: a range of activities to review and analyze the estimated or actual costs of a system, a program or a project. It is a comprehensive review of proposed cost estimates or pricing data and includes activities such as sensitivity and what-if analysis that assesses uncertainty or risks associated with the estimates. Relying on the same estimating practice and methodologies, cost analysis can be either retrospective or predictive. Retrospective analysis evaluates the actual costs that have been incurred, while predictive analysis assesses the accuracy and validity of cost estimates before they are finalized. In the United States Department of Defense (DoD) procurement organizations, cost analysis is the evaluation of a contractor's cost and pricing data in order to form an opinion on the degree to which the contractor's proposed costs represent what the performance of the contract should cost, assuming reasonable economy and efficiency.
- *Life Cycle Cost (LCC)*: sometimes called Total Cost of Ownership (TCO) or Total Ownership Cost (TOC), LCC is the total cost of acquisition and ownership of a system or capability over its entire life cycle. LCC generally includes the cost of research, development, testing and evaluation (RDT&E), manufacturing and production, operation and support (O&S), and where applicable, dismantle and disposal.

Types of Cost Estimates

Cost estimates are generally categorized based on the purposes they serve. The following are the most common types of cost estimates.

- *Budget estimate*: an estimate developed for the budgeting and funding purposes associated with a program, project, activity, initiative, or operations of an organization, in order to support its objectives consistent with the defined scope, schedule, and resource requirements.
- *Rough order of magnitude (ROM) estimate*: an estimate intended to grossly approximate the expected costs or effort involved in a project, task, or activity. It is often developed in the very early stage of something and with very little specific available information. It is also known as "quick look," initial, or "ballpark" estimate.
- *Firm cost estimate*: an estimate commonly used in cost proposals and intended to be a binding obligation from an offering organization. It relies on well-defined plans and data, usually in response to a customer's firm request for proposal. Firm estimates demand the highest level of substantiating detail, requiring the preparation of an extensive backup package to facilitate contractual fact-finding and negotiations.
- *Life cycle cost (LCC) estimate*: a comprehensive, "cradle to grave" estimate to cover all anticipated expenses from inception to disposal that includes research, development, testing and evaluation (RDT&E), production, operations and support (O&S) and disposal costs. It is typically created to aid the planning and analysis of alternatives for capability acquisition. It is created regardless of funding source and should be done as early in the life cycle as possible. As described above, it may also be referred to as Total Cost of Ownership (TCO) or Total Ownership Cost (TOC) estimate.
- *Should cost (SC) estimate*: an assessment of what it should reasonably cost to produce a product, deliver a service, or execute a project under ideal conditions, assuming efficiency in execution, optimal resource utilization, continuous improvement and cost optimization. It is a management tool typically associated with the acquirer to actively seek cost reduction, drive efficiency, and promote collaboration between buyers and suppliers to achieve mutual benefits.

- *Estimate at completion (EAC)*: an assessment of the cost to complete the authorized scope of a project based on the historical performance and progress up to a certain point of that project. Often developed at the onset or during the course of the project, it provides an estimate of the total cost expected by the time the project is completed. It is an important *earned value management* (EVM) metric, central to project management.
- *Independent cost evaluation (ICE)*: a cost analysis performed by an independent group of experts to assess the reasonableness, accuracy, and completeness of a cost estimate submitted by an offeror. ICE uses the same data provided by the original estimate and involves analyzing and verifying supporting information, such as cost drivers, assumptions, and uncertainties. It typically develops an independent estimate or view, in comparison to the original estimate, and identifies risks and opportunities associated with the program or project.
- *Not-to-exceed (NTE) and not-less-than (NLT) estimates*: estimates commonly used in contracting and project management to define the maximum allowable and minimum acceptable costs for the project or contract. These estimates provide basic parameters for managing stakeholder expectations, preparing budgets or proposals, and making informed decisions.

Cost Estimating Process

A cost estimating process generally consists of a seven-step process, as depicted in **Figure 1**. In practice, however, the process may be customized based on factors such as the type of estimate being developed, data availability, and the maturity of developmental stage.

Cost estimating is an iterative process characterized by multiple feedback loops. For instance, data analysis may lead to further questions, necessitating additional data collection. Similarly, validating estimates might involve applying different estimating methods, which could require gathering new data.

Estimates evolve continuously as design matures, development progresses, and more data becomes available. Existing estimate must be refreshed based on updated technical and program management baselines to support major program and project milestones.

Step 1: Develop the Work Breakdown Structure (WBS). The WBS establishes a common frame of reference for relating job tasks to each other and aggregating cost elements at the summary level of detail. It is used to capture the scope of work intended for the estimate. A WBS consists of two parts: a tree-like hierarchical structure defining work elements, and a WBS dictionary, which defines the terms used in the structure. The U.S. DoD developed MIL-STD-881 (DoD 2022) to provide the standard framework and instructions for developing a WBS.

Step 2: Establish a technical baseline. A technical baseline includes all the information required for an estimate, captured at a specific point in time. It includes the technical scope approved for the system under development such as the configuration items, the programmatic scope for the project such as schedule and milestones, as well as the ground rules and assumptions made as the basis for the estimate. While the development continues, the baseline must be frozen for the purpose of estimation.

Step 3: *Collect data*. Data collection involves two kinds of data: the data for the system and program under estimation, and the historical data or database from the like systems and historical projects. They may be collected at the same or different times. The data for estimation is collected from the ongoing system design, necessary in identifying required cost drivers. Collecting historical data is required to apply methodologies and develop or update CERs. As emphasized previously in the methodology section, it is critical to interview key stakeholders, including those from the current and historical programs, which is necessary for understanding of the data collected.

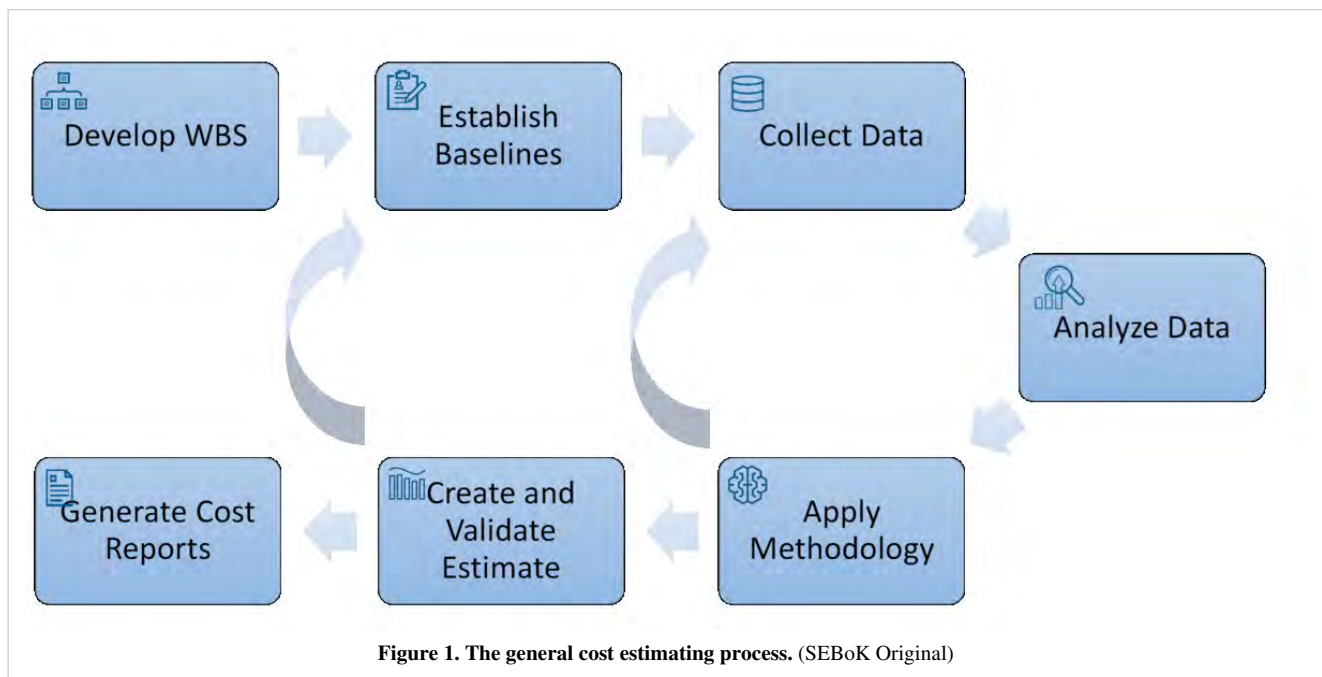
Step 4: *Analyze data*. Collected data must be analyzed to derive useful metrics and enable the application of a cost estimating methodology. Data analysis involves examining collected data to identify useful patterns and gain insights into the appropriate estimating methods to apply to different parts of the estimate. There are commonly deployed tools for data analysis, including scatter plotting, deriving descriptive statistics (e.g., mean, variance, standard deviation), identifying outliers, updating historical database, and calibrating parametric models.

Step 5: Apply estimating methodology and develop cost estimating relationships. Select the best estimating method (see Cost Estimating Methodologies section below) based on available data. Establish the *ground rules* and *assumptions* for estimating, as they will dictate how data is used and influence the final outcome of the estimate. Once patterns are identified from data analysis, CERs can be established for different cost elements. This may involve recalibrating or fine-tuning existing CERs to align with the newly collected data or specific project requirements or characteristics. Commercial off-the-shelf estimating models must be calibrated with collected data before being used to generate estimates.

Step 6: Roll up and validate estimates. Once the CERs are created, estimated cost or effort can be generated based on the cost driver inputs. When appropriate calibrations, commercial off-the-shelf estimating models may be used produce and summarize the estimate. After the estimates are generated, they must be validated to ensure reasonableness and completeness. Sensitivity analysis and cross-technique validation can be applied to key cost elements. Risk assessment should also be conducted.

Step 7: Generate cost reports. Cost report provides a written justification for the cost estimate. This is a formal documentation that captures all aspects of the estimating process and provides sufficient information for a third-party analyst to fully understand how the estimate is developed and to replicate the estimate at a later time, if necessary. In the contracting process, this document is called the *Basis of Estimate* (BOE), which is required to support the bidder's cost proposal.

As indicated previously, cost estimating is always an iterative process. It should be revisited and updated for each major milestone of the development project. There are intermediate feedback loops that may be necessary to complete the process, as shown in **Figure 1**. For example, analysis of data may need additional interviews to fully understand the historical behavior; applying methods and developing CERs may uncover gaps in data, which triggers additional collection efforts.



Cost Estimating Methodologies

Cost estimating methods, also referred to as costing techniques, are the building blocks in developing of a cost estimate. They establish the basic approach to use the historical data and provide the basic structure for statistical relationships. These methods are the basic “algorithms” that relates the historical data to future cost.

Three primary methods are commonly applied when developing a cost estimate (you may hear some other names; but they are mostly synonymous with one of the three methods). These three techniques are *analogy*, *parametric* and *build-up*.

Analogy Method

The *analogy*, also known as *comparative*, method is just what it sounds like – an attempt to estimate costs by comparing the new system with a similar (or analogous) system or by drawing parallels between the current project and a similar past project to estimate costs. It is based on the idea “it’s like one of these.” An analogy can be done at the system, subsystem, component, project, or task level.

The analogy method is not simply a matter of asserting that an item cost an amount, X, and therefore will cost X in the future. Generally, some adjustments must be made to one or more attributes of the old item to estimate the new item. These quantities for adjustment, known as *cost* or *size drivers*, are typically physical (such as size and weight) and performance (like speed and power) characteristics of a system or service that are the primary drivers for cost or effort. They may also include programmatic metrics such as quantity or schedule, as well as supplier performance or economic adjustments for inflation. For example, suppose the current version of an engine generates 10,000 lbs of thrust and costs \$20M dollars to build and that the next-generation version of the same engine is designed to generate 15,000 lbs of thrust. Applying the analogy method, the estimated cost of building the new engine, assuming the use of the same manufacturing technology, would amount to \$30M dollars ($= 15,000 / 10,000 * \$20M$). As a second example, if assembly of the current-generation engine initially took 320 person-hours to assemble and unit test 10 equipment racks. Assuming the same process, it is estimated to require 1,600 person-hours ($= 50 / 10 * 320$ hours) to assemble and unit test 50 racks.

Analogies are generally used in the early stages of the program life cycle, especially when detailed data is lacking but estimates are necessary for decision-making. Many development programs have heritage or legacy systems that can serve as a basis for comparison when estimating costs or efforts for new systems. Additionally, the analogy method can also be used as a validation or crosscheck for estimates created through other methods.

Parametric Method

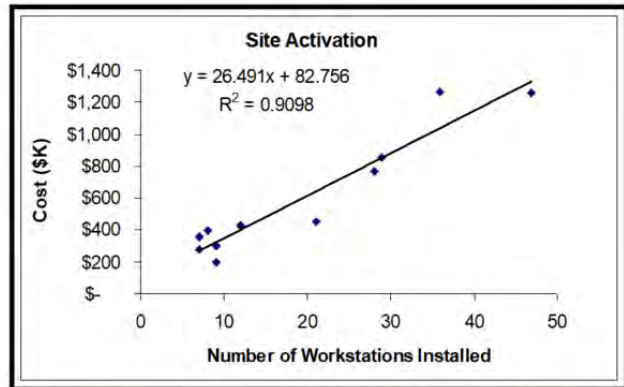
The *parametric*, also known as *algorithmic*, costing technique is based on mathematical relationships between specific system attributes or parameters (such as weight, thrust, power, lines of code) as one or more independent variables, and the estimated cost or effort of the system as a dependent or derived variable. “This pattern holds” is the fundamental concept underlying this methodology. These parametric relationships are typically expressed as mathematical equations, commonly known as *cost estimating relationships* (CERs), in a general form

The CERs are developed with a statistical framework using historical data collected on similar programs or systems. The independent variables (x, y, ...) are known as *cost drivers*. They can be physical characteristics, performance or operational parameters, programmatic variables, or even other costs.

As a hypothetical example, the estimated cost of site activation (C) in \$K is a function of the number of workstations (N_{WS}) installed. One identifies a group of similar site activation projects and collects the historical project data. Analysis of the data results in a scatter plot, as shown in **Figure 2**. A CER is derived using linear regression of the scatter data, expressed as a simple equation.

	X (Cost Driver)	Y
	Number of Workstations	Cost (\$K)
Site 1	8	\$ 398
Site 2	28	\$ 769
Site 3	12	\$ 430
Site 4	9	\$ 199
Site 5	21	\$ 447
Site 6	29	\$ 851
Site 7	47	\$ 1,258
Site 8	7	\$ 359
Site 9	36	\$ 1,267
Site 10	9	\$ 300
Site 11	7	\$ 275

a. Table of num. workstations vs. activation cost



b. Scatter plot of activation cost vs. num. workstation, and a linear regression of the data points

Figure 2. Historical site activation data, in tabulated form and scatter plot, and a linear regression of the scatter data points. (SEBoK Original)

The parametric technique is applicable across a wide range of estimates and throughout all phases of the system life cycle. It is often the preferred method for cost proposals, and is frequently favored by source selection authorities. In addition, it is particularly effective in sensitivity analyses, what-if scenarios, and cost-based trade studies, as it allows for the adjustment of system attribute values acting as cost drivers to directly influence the estimated cost or effort.

Most commercial off-the-shelf (COTS) cost estimating tools rely on parametric-based cost models at their core. These algorithms constitute a significant portion of the commercial market value of these tools and are often considered the "secret sauce" provided by the tool vendors.

Build-up Method

The *build-up* method, also known as *detailed* or *bottom-up* estimating, constructs estimates for higher level cost elements by aggregating or "rolling up" detailed estimates from lower level cost elements. It involves estimating costs at the lowest definable level where data exists and summing it up along a hierarchical structure. A *work breakdown structure* (WBS) is generally used to define this hierarchical structure. It provides a standard framework for organizing cost estimates and, therefore, is the cornerstone of this technique.

Based on the idea "it's made up of these," the build-up method requires breaking down the project into smaller components or work packages and estimating the costs of each individual element. These individual cost estimates are then aggregated to produce the total project cost involves estimating at the lowest definable level where data exists and summing it up along the hierarchical structure. The build-up method works best when low-level detailed cost actuals or estimates are available. It is most effective in operational settings where processes are well-established, and standards and performance factors are clearly defined, enabling effective management of work and measurement of performance.

Other Methods

Two other methods are less rigorous and prevalent, but can be very useful sometimes.

The *expert opinion method*, also known as the *engineering judgment method*, uses an expert or group of experts to estimate the cost of a system. It is often done through one-on-one interviews, round-table discussions with multiple experts, and the Delphi technique, where a group of experts anonymously provides their responses and the results are aggregated.

While expert opinion can offer valuable insights, it is generally considered to be too subjective and prone to biases. Therefore, it should only be utilized as a last resort when other methods are unavailable or to obtain high-level,

low-fidelity estimates for “quick looks.” Additionally, it may serve as a cross-check, corroborate objective data, or adjust estimation based on expert insights.

Extrapolation from actual involves using actual costs from past or current items and historic trends to predict future costs for the same item. It is best applied in production settings to estimate the follow-on production units or lots based on the cost of completed units. There are several variants of this method, including *averages*, *learning curves*, and *estimate-at-completion* (EAC). “Extrapolation from actual” is generally not considered a separate method itself, but a subset of other methods, such as *earned value management* (EVM).

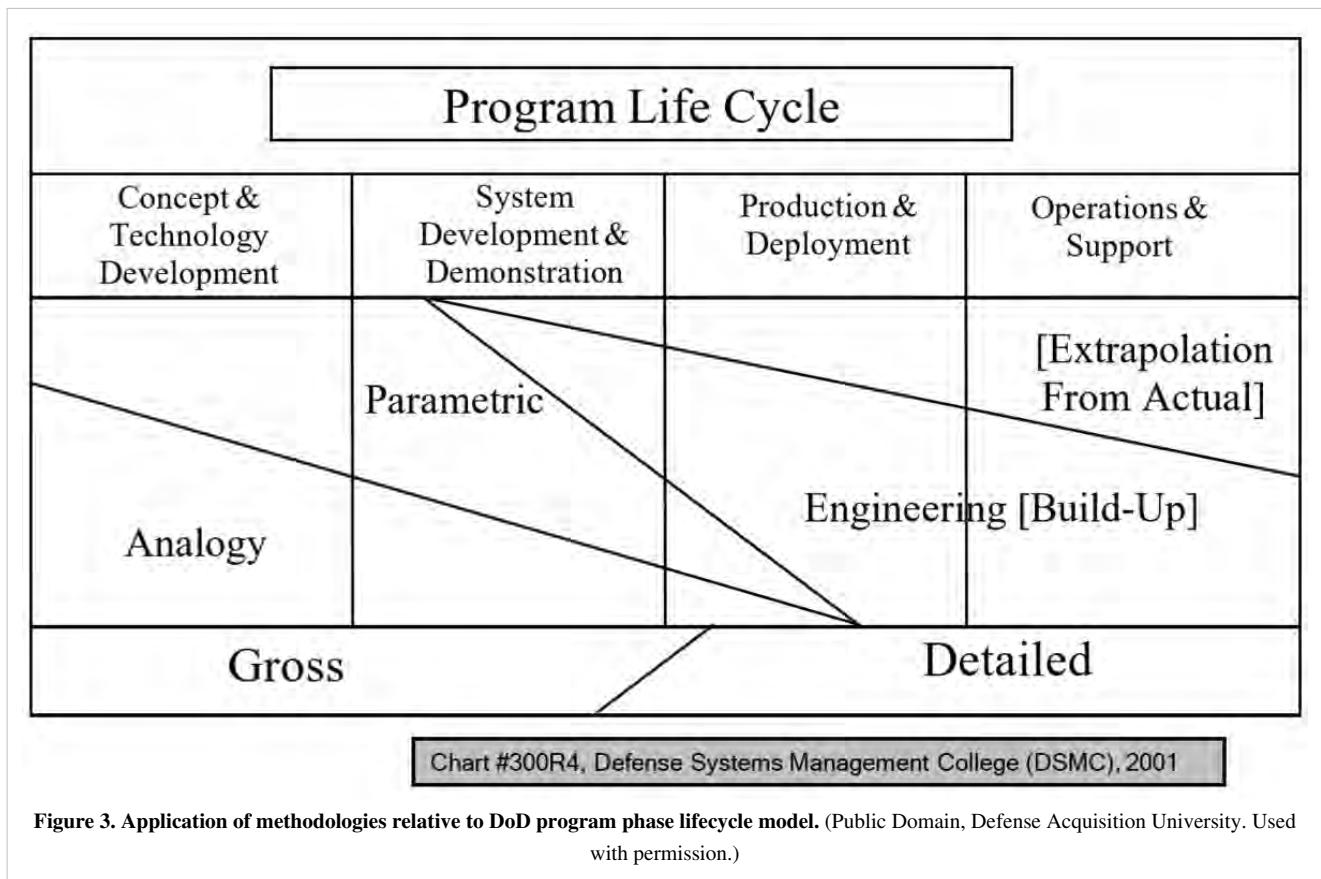
Comparison of Techniques

When developing a cost estimate, it is essential to select the most suitable methods for the given task. While it is possible to use multiple techniques within a single estimate, it is crucial to determine how and when each technique should be applied. Just like any other aspect of project management, each technique comes with its own set of strengths and weaknesses and varying degree of applicability for different stages of a program's life cycle. A comparison of the three techniques is provided in **Table 1**.

Figure 3 presents a guideline provided by the Department of Defense (DoD 2022) regarding the typical application of various costing techniques relative to the DoD program lifecycle model. It outlines the approximate distribution of the different techniques across different phases of the DoD acquisition cycle.

Table 1. Comparison of estimating methodologies.

Methodology	Advantage	Disadvantage
Analogy	• Relatively simple, quick, and low cost • Best used early in program before detailed requirements, or as “sanity check” • Difficult to refute if there is strong historical resemblance	• Subjectivity in making adjustments • Difficult to identify common cost drivers between old and new systems for comparison • Lack of objective validity tests • Limited to stable technology and processes
Parametric	• Relatively simple, quick, and low cost • Easily adjusted for changes by modifying input parameters • Best for sensitivity and impact analyses • Effective for cost-based trade studies • Objective measures of validity • Preferred by cost analysts	• Highly dependent upon quantity and quality of historical database • Expensive to collect and maintain historical database • Must continue to update relationships to adjust for updates in relevant programs, processes, and technologies applied • “Black box syndrome” with pre-existing CERs and commercial models
Build-Up	• Most detailed technique and data, and best inherent accuracy • Easy to see exactly what the estimate includes and contribution of cost elements • Variance factors based on historical data for a given program or a specific manufacturer • Typical required to support organizational cost and pricing proposal	• Expensive and time consuming to develop • Requires large amount of data to be collected, maintained, and analyzed • Detailed specifications and definitions required • May underestimate system integration and other “under the line” system level cost • Small errors can easily magnify • Omissions and duplications are likely



The Role of Historical Data

Data, particularly historical data, is the very foundation for cost estimating. Without it, there is no basis for applying methodologies, and the estimates lack both credibility and defensibility. In essence, without data, there can be no cost estimation, and any estimates made without sufficient data would be nothing more than mere guesses.

Collecting historical data is akin to forensic analysis of the development history, and it is typically done as a separate effort before estimation is required or as part of continuous enterprise knowledge management and process improvement efforts. It can be considered a project in its own right, requiring dedicated resources and tools, as well as access to both technical and financial data. An important aspect of historical data collection is stakeholder interviews, or the ability to speak to the system design and project management authorities of the historical programs, in order to enable understanding of the data collected.

Conclusion

Cost estimating is a multidisciplinary practice that develops future costs for products and services by analyzing historical data and using quantitative methods. It falls in the intersecting areas of systems engineering and project management and demands a wide range of skills and expertise, particularly in mathematics. Effective cost estimating relies on collaboration between systems engineers and cost analysts. This practice is crucial for decision-making, especially in areas involving resource allocation and investment planning.

References

Works Cited

- Defense Acquisition University (DAU). "Business - Cost Estimating". <https://www.dau.edu>.
- International Cost Estimating and Analysis Association (ICEAA), *Cost Estimating Body of Knowledge (CEBoK®)*. <https://www.iceaaonline.com/cebok/>
- ICEAA. *Journal of Cost Analysis & Parametrics*. 2024. <https://www.iceaaonline.com/publications/#journal>.
- International Council on Systems Engineering (INCOSE). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 5.0. Hoboken, NJ: John Wiley and Sons, Inc, ISBN 978-1-119-81429-0.
- International Society of Parametric Analysts (ISPA). *Parametric Estimating Handbook*. 4th ed. ISBN: 0-9720204-7-0. 2008.
- ISO/IEC/IEEE. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization; International Electrotechnical Commissions; Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015. 2023.
- NASA. *NASA Cost Estimating Handbook*. 4th ed. Washington, DC: National Aeronautics and Space Administration. NASA/SP-2007-6105.
- Project Management Institute (PMI). *PMBOK® Guide*, 7th ed. 2021.
- U.S. Department of Defense (DoD). *DoD Cost Estimating Guide*. Washington D.C: Office of the Secretary of Defense (OSD) Cost Assessment and Program Evaluation. 2022.
- U.S. DoD. *Operating and Support Cost-Estimating Guide*. Washington, D.C.: Office of the Secretary of Defense (OSD) Director of Cost Assessment and Program Evaluation (DCAPE). 2020.
- U.S. DoD. *Work Breakdown Structures for Defense Materiel Items, MIL-STD-881F*, AMSC 10316. Washington, D.C.: DoD. 2022
- U.S. Government Accountability Office (GAO). *Cost Estimating and Assessment Guide: Best Practices for Developing and Managing Program Costs, GAO-20-195G*. Washington, D.C.: GAO. 2020.

Primary References

- International Cost Estimating and Analysis Association (ICEAA), *Cost Estimating Body of Knowledge (CEBoK®)*. ICEAA. <https://www.iceaaonline.com/>
- U.S. GAO. *Cost Estimating and Assessment Guide: Best Practices for Developing and Managing Program Costs, GAO-20-195G*. Washington, D.C.: GAO, 2020.
- U.S. DoD. *DoD Cost Estimating Guide*. Washington D.C: Office of the Secretary of Defense (OSD) Cost Assessment and Program Evaluation. 2022.

Additional References

None.

Procurement and Acquisition

- Lead Author:
- Dick Fairley
- Contributing Author:
- Alice Squires

Procurement is the act of buying goods and services. Acquisition covers the conceptualization, initiation, design, development, testing, contracting, production, deployment, logistics support, modification, and disposal of weapons and other systems, as well as supplies or services (including construction) to satisfy organizational needs intended for use in, or in support of, defined missions (DAU 2010; DoD 2001).

Acquisition covers a much broader range of topics than procurement. Acquisition spans the whole life cycle of acquired systems. The procurement of appropriate systems engineering (SE) acquisition activities and levels of SE support is critical for an organization to meet the challenge of developing and maintaining complex systems.

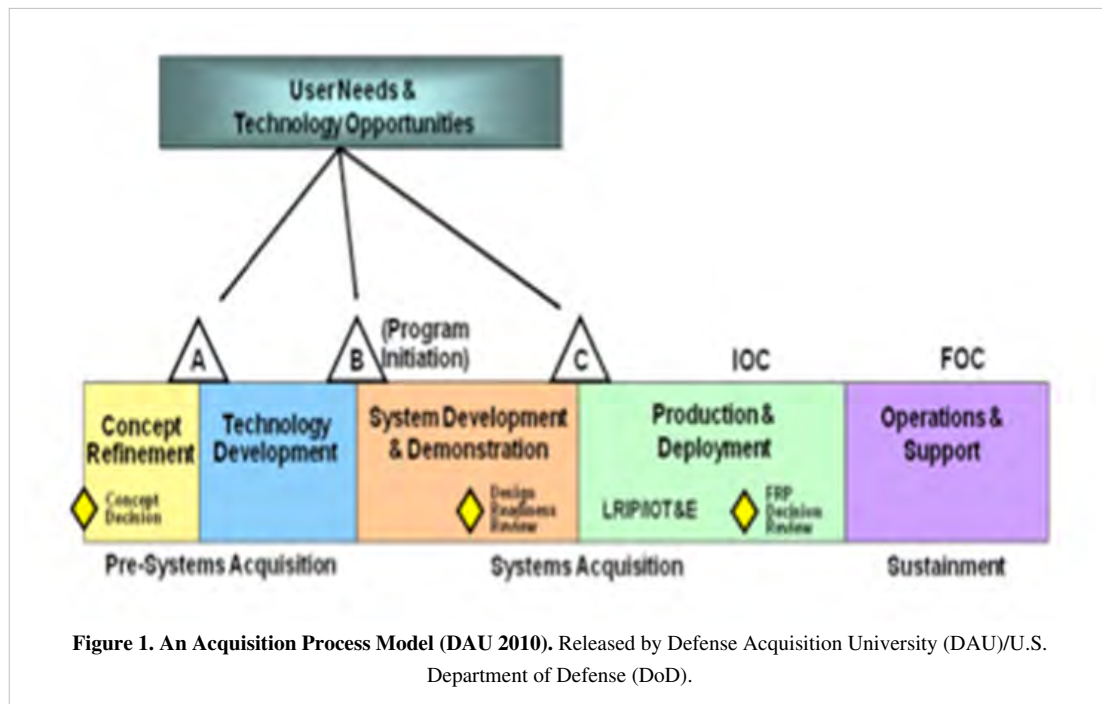
The *Guide for Integrating Systems Engineering into DoD Acquisition Contracts* addresses how systems engineering activities are integrated into the various elements of acquisition and procurement (DoD 2006a).

Acquisition Process Model

Multiple acquisition process models exist. An acquisition process for major systems in industry and defense is shown in Figure 1. The process of acquisition is defined by a series of phases during which technology is defined and matured into viable concepts. These concepts are subsequently developed and readied for production, after which the systems produced are supported in the field.

Acquisition planning is the process of identifying and describing needs, capabilities, and requirements, as well as determining the best method for meeting those requirements (e.g., program acquisition strategy). This process includes procurement; thus, procurement is directly linked to the acquisition process model. The process model present in Figure 1 allows a given acquisition to enter the process at any of the development phases.

For example, a system using unproven technology would enter at the beginning stages of the process and would proceed through a lengthy period of technology maturation. On the other hand, a system based on mature and proven technologies might enter directly into engineering development or sometimes even production.



Systems Engineering Role in the Acquisition Process

The procurement of complex systems usually requires a close relationship between the offeror and supplier SE teams due to the breadth and depth of SE activities. SE is an overarching process that the program team applies in order to transition from a stated capability need to an affordable, operationally effective, and suitable system.

SE is important to every phase of the acquisition process. SE encompasses the application of SE processes across the acquisition life cycle and is intended to be an integrating mechanism for balanced solutions addressing capability needs, design considerations, and constraints. It is also intended to address limitations imposed by technology, budget, and schedule.

SE is an interdisciplinary approach; that is, it is a structured, disciplined, and documented technical effort to simultaneously design and develop system products and processes to satisfy the needs of the customer. Regardless of the scope and type of program, or at what point it enters the program acquisition life cycle, the technical approach to the program needs to be integrated with the acquisition strategy to obtain the best program solution.

Acquisition and procurement in the commercial sector have many characteristics in common with their counterparts in the realm of government contracting, although the processes in the commercial world are usually accomplished with fewer rigors than occur between government and contractor interactions. Offshore outsourcing is commonly practiced in the commercial software arena with the goal of reducing the cost of labor. Commercial organizations sometimes subcontract with other commercial organizations to provide missing expertise and to balance the ebb and flow of staffing needs.

In some cases, relations between the contracting organization and the subcontractor are strained because of the contracting organization's desire to protect its intellectual property and development practices from potential exposure to the subcontractor. Commercial organizations often have lists of approved vendors that are used to expedite the procurement of needed equipment, products, and services. In these situations, commercial organizations have processes to evaluate and approve vendors in ways that are analogous to the qualification of government contractors. Many commercial organizations apply SE principles and procedures even though they may not identify the personnel and job functions as "systems engineers" or "systems engineering."

Importance of the Acquisition Strategy in the Procurement Process

The acquisition strategy is usually developed during the front end of the acquisition life cycle. (For an example of this, see the Technology Development Phase in Figure 1.) The acquisition strategy provides the integrated strategy for all aspects of the acquisition program throughout the program life cycle.

In essence, the acquisition strategy is a high-level business and technical management approach designed to achieve program objectives within specified resource constraints. It acts as the framework for planning, organizing, staffing, controlling, and leading a program, as well as for establishing the appropriate contract mechanisms. It provides a master schedule for research, development, testing, production, fielding, and other SE related activities essential for program success, as well as for formulating functional strategies and plans.

The offeror's program team, including systems engineering, is responsible for developing and documenting the acquisition strategy, which conveys the program objectives, direction, and means of control based on the integration of strategic, technical, and resource concerns. A primary goal of the acquisition strategy is the development of a plan that will minimize the time and cost of satisfying an identified, validated need while remaining consistent with common sense and sound business practices. While the contract officer (CO) is responsible for all contracting aspects, including determining which type of contract is most appropriate, and following the requirements of existing regulations, directives, instructions, and policy memos of an organization, the program manager (PM) works with the CO to develop the best contract/procurement strategy and contract types.

Relating Acquisition to Request for Proposal and Technical Attributes

There are several formats for requesting proposals from offerors for building complex systems. Figure 2 relates acquisition program elements to a representative request for proposal (RFP) topical outline and key program technical attributes that have been used by the Department of Defense. In general, programs have a better chance of success when both the offeror and supplier understand the technical nature of the program and the need for the associated SE activities.

The offeror and supplier need to clearly communicate the technical aspect of the program throughout the procurement process. The offeror's RFP and the associated supplier proposal represent one of the formal communications paths. A partial list of key program technical attributes is presented in Figure 2.

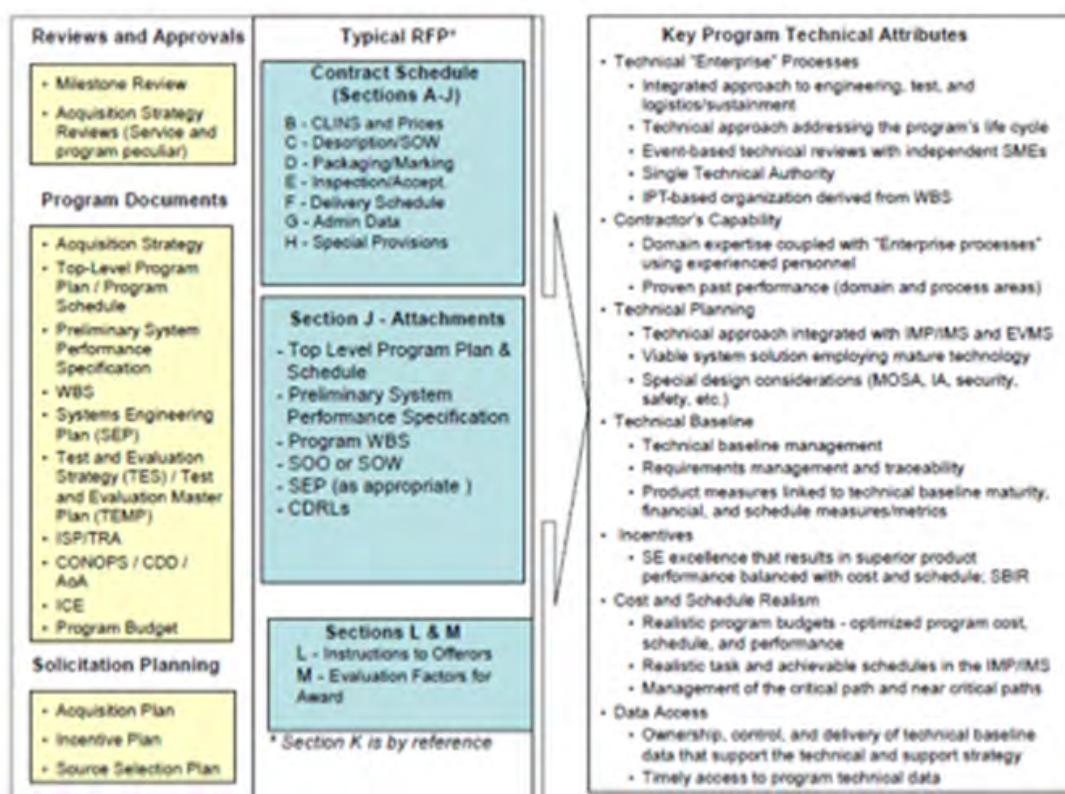


Figure 2. Relating Acquisition to Request for Proposal and Technical Attributes. (DoD 2006a). Released by the U.S. Office of the Secretary of Defense.

Contract-Related Activities and the Offeror's Systems Engineering and Project Management Roles

A clear understanding of the technical requirements is enhanced via the development of a Systems Engineering Plan (SEP). The SEP documents the systems engineering strategy for a project or program and acts as the blueprint for the conduct, management, and control of the technical aspects of the acquisition program (DoD 2011). The SEP documents the SE structure and addresses government and contractor boundaries. It summarizes the program's selected acquisition strategy and identifies and links to program risks. It also describes how the contractor's, and sometimes the subcontractor's and suppliers', technical efforts are to be managed.

Once the technical requirements are understood, a contract may be developed and followed by the solicitation of suppliers. The offeror's PM, chief or lead systems engineer, and CO must work together to translate the program's acquisition strategy and associated technical approach (usually defined in a SEP) into a cohesive, executable contract(s).

Table 1 shows some key contracting-related tasks with indicators of the roles of the PM and LSE.

**Table 1. Offeror's Systems Engineering and Program Management Roles (DoD 2006).
Released by the U.S. Office of the Secretary of Defense.**

Typical Contract-Related Activities	Systems Engineer and Project Manager Roles
1. Identify overall procurement requirements and associated budget. Describe the offer's needs and any constraints on the procurement.	1. Lead systems engineer (LSE) provides program technical requirements. PM provides any programmatic related requirements.
2. Identify technical actions required to successfully complete technical and procurement milestones. The program's SEP is the key source for capturing this technical planning.	2. LSE defines the technical strategy/approach and required technical efforts. This should be consistent with the program's Acquisition Strategy.
3. Document market research results and identify potential industry sources.	3. PM and LSE identify programmatic and technical information needed and assist in evaluating the results.
4. Prepare a Purchase Request, including product descriptions; priorities, allocations and allotments; architecture; government-furnished property or equipment (or Government-Off-The-Shelf (GOTS); government-furnished information; information assurance and security considerations; and required delivery schedules.	4. PM and LSE ensure the specific programmatic and technical needs are defined clearly (e.g., commercial-off-the-shelf (COTS) products).
5. Identify acquisition streamlining approach and requirements, budgeting and funding, management information requirements, environmental considerations, offeror's expected skill sets, and milestones. These should be addressed in the Acquisition Strategy.	5. The procurement team work together, but the CO has the prime responsibility. The PM is the owner of the program Acquisition Strategy. The LSE develops and reviews (and the PM approves) the technical strategy.
6. Plan the requirements for the contract Statement of Objectives (SOO) / Statement of Work (SOW) / specification, project technical reviews, acceptance requirements, and schedule.	6. LSE is responsible for the development of the technical aspects of the SOO/SOW.
7. Plan and conduct Industry Days as appropriate.	7. PM and LSE support the CO in planning the meeting agenda to ensure technical needs are discussed.
8. Establish contract cost, schedule, and performance reporting requirements. Determine an incentive strategy and appropriate mechanism (e.g., Award Fee Plan and criteria).	8. LSE provides technical resource estimates. LSE supports development of the Work Breakdown Structure (WBS) based on preliminary system specifications, determines event-driven criteria for key technical reviews, and determines what technical artifacts are baselined. The PM and LSE advise the CO in developing the metrics/criteria for an incentive mechanism.
9. Identify data requirements.	9. LSE identifies all technical Contractor Data Requirements List (CDRL) and technical performance expectations.
10. Establish warranty requirements, if applicable.	10. LSE works with the CO to determine cost-effective warranty requirements.
11. Prepare a Source Selection Plan (SSP) and RFP (for competitive contracts).	11. PM and LSE provide input to the SSP per the SOO/SOW.
12. Conduct source selection and award the contract to the successful offeror.	12. PM and LSE participate on evaluation teams.

Offeror and Supplier Interactions

There should be an environment of open communication prior to the formal source selection process. This ensures that the supplier understands the offeror's requirements and that the offeror understands the supplier's capabilities and limitations, as well as enhancing the supplier's involvement in the development of a program acquisition strategy. During the pre-solicitation phase, the offeror develops the solicitation and may ask suppliers to provide important insights into the technical challenges, program technical approach, and key business motivations.

For example, potential bidders could be asked for their assessment of a proposed system's performance based on the maturity level of new and existing technologies.

Contracts and Subcontracts

Typical types of contracts include the following:

- **Fixed Price:** In a fixed price contract the offeror proposes a single price for all products and services to implement the project. This single price is sometimes referred to as low bid or lump sum. A fixed price contract transfers the project risks to the supplier. When there is a cost overrun, the supplier absorbs it. If the supplier performs better than planned, their profit is higher. Since all risks are absorbed by the supplier, a fixed price bid may be higher to reflect this.
- **Cost-reimbursement [Cost plus]:** In a cost-reimbursement contract the offeror provides a fixed fee, but also reimburses the contractor for labor, material, overhead, and administration costs. Cost-reimbursement type contracts are used when there is a high level of project risk and uncertainty. With this type of contract, the risks reside primarily with the offeror. The supplier gets reimbursed for all of its costs. Additional costs that arise due to changes or rework are covered by the offeror. This type of contract is often recommended for the system definition of hardware and software development when there is a risk of stakeholder changes to the system.
- **Subcontracts:** A subcontractor performs work for another company as part of a larger project. A subcontractor is hired by a general contractor (also known as a prime or main contractor) to perform a specific set of tasks as part of the overall project. The incentive to hire subcontractors is either to reduce costs or to mitigate project risks. The systems engineering team is involved in establishing the technical contract requirements, technical selection criteria, acceptance requirements, and the technical monitoring and control processes.
- **Outsource contracts:** Outsourced contracts are used to obtain goods or services by contracting with an outside supplier. Outsourcing usually involves contracting a business function, such as software design and code development, to an external provider.
- **Exclusively Commercial Off-the-Shelf (COTS):** Exclusively COTS contracts are completely satisfied with commercial solutions that require no modification for use. COTS solutions are used in the environment without modifying the COTS system. They are integrated into an existing user's platform or integrated into an existing operational environment. The systems engineering team is involved in establishing the technical contract requirements, technical acceptance, and technical selection criteria.
- **Integrated COTS:** Integrated COTS contracts use commercially available products and integrate them into existing user platforms or operational environments. In some cases, integrated COTS solutions modify the system's solution. The cost of integrating the commercial COTS product into the operational environment can exceed the cost of the COTS product itself. As a result, the systems engineering team is usually involved in establishing the technical outsourcing contract requirements, technical selection criteria, technical monitoring and control processes, and technical acceptance and integration processes.
- **COTS Modification:** COTS modification requires the most time and cost because of the additional work needed to modify the COTS product and integrate it into the system. Depending on how complex and critical the need is, the systems engineering team is usually involved in establishing the technical outsource contract requirements, technical selection criteria, technical monitoring and control processes, and technical acceptance requirements.
- **IT services:** IT services provide capabilities that can enable an enterprise, application, or Web service solution. IT services can be provided by an outsourced service provider. In many cases, the user interface for these Web services is as simple as a Web browser. Depending on how complex and critical the needs are, the systems engineering team can be involved in establishing the technical outsourcing contract requirements, technical selection criteria, and technical acceptance process.

References

Works Cited

Defense Acquisition University (DAU). *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: DAU/US Department of Defense (DoD). 2010.

Department of Defense (DoD). *Systems Engineering Plan (SEP) Outline*. Washington, DC, USA: Office of the Undersecretary of Defense for Acquisition, Transportation, and Logistics (AT&L), US DoD. 2011.

DoD. *Guide for Integrating Systems Engineering into DoD Acquisition Contracts*. Washington, DC, USA: Office of the Undersecretary of Defense for AT&L, U.S. DoD. 2006.

DoD. *Systems Engineering Fundamentals*. Washington, DC, USA: DAU Press/US DoD. 2001.

Primary References

DAU. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: DAU/US DoD. 2010.

DoD. *Systems Engineering Plan (SEP) Outline*. Washington, DC, USA: Office of the Undersecretary of Defense for AT&L, US DoD. 2011.

DoD. *Guide for Integrating Systems Engineering into DoD Acquisition Contracts*. Washington, DC, USA: Office of the Undersecretary of Defense for AT&L, US DoD. 2006.

Additional References

MITRE. "Acquisition Systems Engineering." *Systems Engineering Guide*. 2011. Available: https://www.mitre.org/work/systems_engineering/guide/acquisition_systems_engineering/.

Portfolio Management

- Lead Authors:
- Eric Specking, Gregory S. Parnell, and Ed Pohl

Systems engineers need to be aware of and understand the role of systems engineering in project, program, and portfolio management (PfM) processes to achieve maximum stakeholder value. In general, program and project management practices help ensure that organizations execute programs and projects the “right” way. Portfolio management can help to ensure they select the “right” programs and projects, and systems engineering ensures they use the “right” systems thinking (Specking et al. 2020). Not using portfolio management practices increases organizational risk and could decrease stakeholder value.

This article provides:

- a description of the ISO/IEC/IEEE 15288 activities and tasks with regards to portfolio management;
 - a description of the Project Management Institute's (PMI's) portfolio life cycle process based on PMI's Guide to the Project Management Body of Knowledge (PMI 2017a);
 - the connection of the PMI's portfolio life cycle process to ISO/IEC/IEEE 15288;
 - a reference to the principles of portfolio management from the PfM Standard;
 - a realignment of the SE Handbook's common approaches/tips with the PfM Standard's six domains;
 - a discussion to better clarify systems engineering's relationship to portfolio management, as well as systems engineering's relationship to other topics, such as project management and program management; and
 - a description of the systems engineer's role to support portfolio managers that explicitly states how a systems engineer (a) helps evaluate the value of projects to authorize, continue, or terminate enterprise projects and (b) manages their portfolio of systems engineering activities to support portfolio managers and to support enterprise
-

processes, products, and services.

Portfolio Management Overview

ISO/IEC/IEEE 15288 (ISO/IEC/IEEE 2015) establishes portfolio management's importance to systems engineering by including it as an organizational project-enabling process. This organizational project-enabling process uses an organization's strategic plan, portfolio direction and constraints, supply strategy, and project status report as inputs, while a portfolio management plan, organization infrastructure needs, project direction, project portfolio, organization lessons learned, portfolio management report, and portfolio management records are outputs (INCOSE 2015). Portfolio management uses these inputs to perform processes that help enterprises define and authorize projects; evaluate the portfolio of projects and programs; and terminate projects to develop the portfolio management outputs. These activities take place throughout the portfolio life cycle, which includes optimization, initiation, planning, and execution (PMI 2017b).

Portfolio Management Process Implementation

The proper implementation of portfolio management processes results in seven major portfolio management outcomes: 1) qualification and prioritization of business venture opportunities, investments, or necessities; 2) identification of projects; 3) allocation of project resources and budgets; 4) description of project management responsibilities, accountability, and authorities; 5) sustainment of project meeting agreement and stakeholder requirements; 6) redirection or termination of unsatisfactory projects; and 7) closure of successfully completed projects (ISO/IEC/IEEE, 2015). A project should continue if it contributes to organizational strategy; makes progress on achieving its pre-established goals; obeys organizational project directives; is executed according to its pre-approved plan; and continues to add value to the organization with acceptable returns (INCOSE 2015).

Organizations can increase portfolio value by successfully implementing portfolio management processes through six performance management domains throughout the portfolio life cycle (PMI 2017b). These domains, described in Table 1, include portfolio strategic management, portfolio governance, portfolio capacity and capability management, portfolio stakeholder engagement, portfolio value management, and portfolio risk management.

Table 1. Performance Management Domain Descriptions for Portfolio Management (Specking et al. 2020)

Domain	Description
Portfolio Strategic Management	Align portfolio components to one or more strategic objectives and monitor impact
Portfolio Governance	"Through open and transparent governance, including processes for categorizing, prioritizing, selecting, and approving portfolio components, key stakeholders are more likely to accept the decisions and agree with the process, even when they may not fully endorse the decisions made" (PMI 2017b).
Portfolio Capacity and Capability Management	"The selection of portfolio components and the roadmap for their implementation is balanced against the organization's current capacity and capability with the potential of bringing in additional resources" (PMI 2017b).
Portfolio Stakeholder Engagement	"Key portfolio stakeholders require active expectation management" (PMI 2017b).
Portfolio Value Management	Use the organizational strategy to enable investment in a portfolio with the expectation of a pre-determined return
Portfolio Risk Management	Evaluate risk (positive/opportunities, negative/threats) and consider how those risks might impact accomplishing the portfolio strategic plan and objectives

These domains use some or all of the seven PMI fundamental principles for portfolio management (PMI 2017b). The seven PMI fundamental principles for portfolio management include:

- *Strive to achieve excellence in strategic execution;*
- *Enhance transparency, responsibility, accountability, sustainability, and fairness;*
- *Balance portfolio value against overall risks;*
- *Ensure that investments in portfolio components are aligned with the organization's strategy;*
- *Obtain and maintain the sponsorship and engagement of senior management and key stakeholders;*
- *Exercise active and decisive leadership for the optimization of resource utilization;*
- *Foster a culture that embraces change and risk; and*
- *Navigate complexity to enable successful outcomes.*

Portfolio Management Process Best Practices

INCOSE's Systems Engineering Handbook (INCOSE 2015) provides several best practices for performing portfolio management processes. These best practices connect to one or more of the six performance management domains seen in Table 2.

Table 2. SE Handbook Best Practices for Portfolio Management (Modified from (Specking et al. 2020))

Performance Management Domain for Portfolio Management	Key Word for Best Practice	Description of Best Practices
• Portfolio Strategic Management • Portfolio Strategic Engagement • Portfolio Value Management	Right Stakeholders	Include relevant stakeholders when developing the organization's business area plan to enable the organization to determine present and future strategic objectives for focusing resources
• Portfolio Value Management • Portfolio Governance	Measurable Criteria	Prioritize opportunities based upon measurable criteria that contains a stated threshold of acceptable performance
• Portfolio Value Management • Portfolio Governance • Portfolio Capacity and Capability Management	Progress Assessment	Base expected project outcomes on defined, measurable criteria with specific investment assessment information to enable an impartial progress assessment
• Portfolio Governance	Coordinate Interactions	Use some type of coordination organization, such as a program office, to manage the interactions among active projects
• Portfolio Governance	Consider Products and Systems	Use a product line approach for scenarios where multiple customers need the same/similar system but customize the system as necessary
• Portfolio Risk Management	Assess Risk	Assess risk during current project evaluation and cancel/suspend projects with risks that outweigh the investment
• Portfolio Strategic Management • Portfolio Capacity and Capability Management	Align with Strategy	Perform opportunity assessments of ongoing projects and avoid opportunities that contain unacceptable levels of risks, resource demands, or uncertainties or does not aligned with organization capabilities, strategic goals, or strategic objectives
• Portfolio Value Management • Portfolio Governance	Allocate Resources	Allocate resources based upon project requirements
• Portfolio Governance	Effective Governance	Use effective governance processes, which support investment decision making and project management communications

Relationships Between Systems Engineering, Portfolio Management, and Project Management

It is often difficult for systems engineers to understand the relationships between systems engineering, portfolio management, and project management because of their overlapping processes and the use of similar tools. Figure 1 shows the role of project management, systems engineering, and portfolio management for developing and improving an enterprise architecture to provide products or services to many stakeholders. Project and program managers and systems engineers all interact with stakeholders and the enterprise architecture, which consists of products or services and the systems that make them. Systems engineers use the enterprise architecture to develop user and system functional hierarchies, which enable capability development. These capabilities help systems engineers evaluate the enterprise's current and future states. From this evaluation, the Project Management Office (portfolio managers) can develop a potential list of projects or programs to fund. Portfolio managers use best practices in decision analysis (Parnell et al. 2013) to develop a project prioritization value model (e.g. single or multiple objective decision analysis model). Portfolio managers then use optimization techniques to determine which projects or programs to fund based upon the prioritization value model. Project managers then execute the funded projects to ensure that the projects stay on schedule and within budget constraints while meeting desired performance metrics. Information impacting systems engineering documentation discovered from the funded projects updates the functional hierarchies and system architecture. Throughout the project, portfolio managers receive updates from project managers on project progress. Portfolio managers use progress information to evaluate the project for continuation or termination. If the project is terminated upon successful completion of the project, the project's outcomes update the enterprises' architecture. Otherwise (i.e. terminated early or upon an unsuccessful completion), portfolio managers update their list of potential projects and/or programs for reprioritization. This cycle continues and evolves with time based upon the enterprise's strategy.

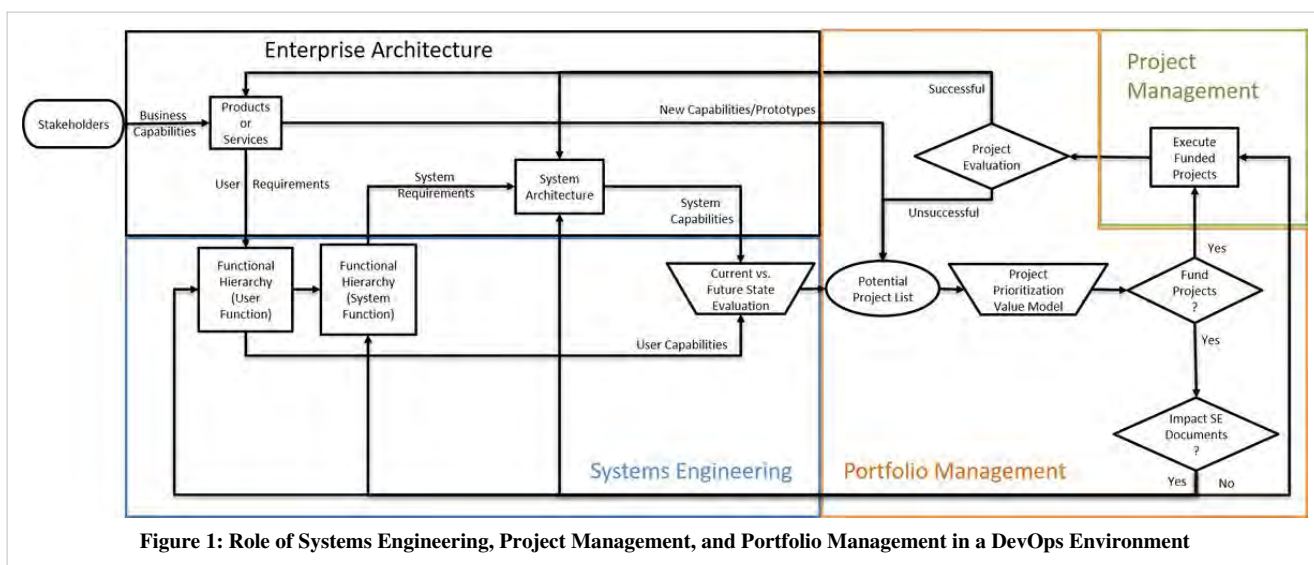


Figure 1: Role of Systems Engineering, Project Management, and Portfolio Management in a DevOps Environment

In addition to helping systems engineers understand the roles of systems engineering, portfolio management, and project management, Figure 1 provides insights into the relationships between the processes from an enterprise DevOps environment perspective.

References

Works Cited

International Council on Systems Engineering (INCOSE). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Edited by David Walden, Garry Roedler, Kevin Forsberg, Douglas Hamelin, and Thomas Shortell. 4th ed. John Wiley & Sons, Inc. 2015.

ISO/IEC/IEEE. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015. 2015.

Parnell G. S., T. A. Bresnic, S. N. Tani, E. R. Johnson. *Handbook of Decision Analysis*. Hoboken, NJ. John Wiley and Sons, Inc. 2013.

Project Management Institute (PMI). *The Guide to the Project Management Body of Knowledge*. 6th ed. Project Management Institute. 2017. <https://www.pmi.org/pmbok-guide-standards/foundational/pmbok>.

PMI. *The Standard for Portfolio Management*. 3rd ed. Project Management Institute. <https://www.pmi.org/pmbok-guide-standards/foundational/standard-for-portfolio-management>.

Specking, E., G. Parnell, E. Pohl. "Comparing INCOSE and PMI portfolio management practices," 30th Annual INCOSE International Symposium, 2020.

Primary References

INCOSE. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Edited by David Walden, Garry Roedler, Kevin Forsberg, Douglas Hamelin, and Thomas Shortell. 4th ed. John Wiley & Sons, Inc. 2015.

ISO/IEC/IEEE. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015. 2015.

Project Management Institute (PMI). *The Guide to the Project Management Body of Knowledge*. 6th ed. Project Management Institute. 2017. <https://www.pmi.org/pmbok-guide-standards/foundational/pmbok>.

PMI. *The Standard for Portfolio Management*. 3rd ed. Project Management Institute. <https://www.pmi.org/pmbok-guide-standards/foundational/standard-for-portfolio-management>.

Additional References

None.

Knowledge Area: Systems Engineering and Software Engineering

Systems Engineering and Software Engineering

Contents of this Knowledge Area

- Software Engineering in the Systems Engineering Life Cycle (Tom Hilburn and Dick Fairley) (Alice Squires)
- The Nature of Software (Tom Hilburn and Dick Fairley) (Alice Squires)
- An Overview of the SWEBOK Guide (Hironori Washizaki, Maria-Isabel Sanchez-Segura, Juan Garbajosa, Steve Tockey, Kenneth E Nidiffer, and Annette D. Reilly)
- Key Points a Systems Engineer Needs to Know about Software Engineering (Dick Fairley) (Alice Squires)
- Software Engineering Features - Models, Methods, Tools, Standards, and Metrics (Tom Hilburn)
- Lead Authors:
- Dick Fairley and Tom Hilburn
- Contributing Authors:
- Ray Madachy and Alice Squires

Software is prominent in most modern systems architectures and is often the primary means for integrating complex system components. Software engineering and systems engineering are not merely related disciplines; they are intimately intertwined. (See Systems Engineering and Other Disciplines.) Good systems engineering is a key factor in enabling good software engineering.

The SEBoK explicitly recognizes and embraces the intertwining between systems engineering and software engineering, as well as defining the relationship between the SEBoK and the Guide to the Software Engineering Body of Knowledge (SWEBOK) (Bourque, and Fairley 2014; IEEE Computer Society 2022).

This knowledge area describes the nature of software, provides an overview of the SWEBOK, describes the concepts that are shared by systems engineers and software engineers, and indicates the similarities and difference in how software engineers and systems engineers apply these concepts and use common terminology. It also describes the nature of the relationships between software engineering and systems engineering and describes some of the methods, models and tools used by software engineers.

Discussion

Software engineers, like systems engineers:

- engage in analysis and design, allocation of requirements, oversight of component development, component integration, verification and validation, life cycle sustainment, and system retirement.
- work with or as a component specialist (for example, user interface, database, computation, and communication specialists) who construct or otherwise obtain the needed software components.
- adapt existing components and incorporate components supplied by customers and affiliated organizations.

These commonalities would make it appear that software engineering is merely an application of systems engineering, but this is only a superficial appearance. The differences between the two disciplines arise from two fundamental concerns:

1. Differences in educational backgrounds (traditional engineering disciplines for SE and the computing disciplines for SWE) and work experiences that result in different approaches to problem solving, and
-

2. Different ways of applying shared concepts based on the contrasting natures of the software medium and the physical media of traditional engineering.

Table 1 itemizes some of the shared concepts that are applied in different ways by systems engineers and software engineers. Each discipline has made contributions to the other. Table 1 indicates the methods and techniques developed by systems engineers adapted for use by software engineers and, conversely, those that have been adapted for use by systems engineers.

Table 1. Adaptation of Methods Across SE and SWE (Fairley and Willshire 2011) Reprinted with permission of Dick Fairley and Mary Jane Willshire. All other rights are reserved by the copyright owner. *

Systems Engineering Methods Adapted to Software Engineering	Software Engineering Methods Adapted to Systems Engineering
• Stakeholder Analysis • Requirements Engineering • Functional Decomposition • Design Constraints • Architectural Design • Design Criteria • Design Trade-offs • Interface Specification • Traceability • Configuration Management • Systematic Verification and Validation	• Model-Driven Development • UML-SysML • Use Cases • Object-Oriented Design • Iterative Development • Agile Methods • Continuous Integration • Process Modeling • Process Improvement • Incremental Verification and Validation

The articles in this knowledge area give an overview of software and software engineering aimed at systems engineers. It also provides more details on the relationship between systems and software life cycles and some of the detailed tools used by software engineers. As systems become more dependent on software as a primary means of delivering stakeholder value, the historical distinction between software and systems engineering may need to be challenged. This is a current area of joint discussion between the two communities which will affect the future knowledge in both SEBoK and SWEBoK.

References

Works Cited

- Bourque, P. and Fairley, R. E. eds. *Guide to the Software Engineering Body of Knowledge (SWEBoK)*. Los Alamitos, CA. IEEE Computer Society. 2014. <https://www.Swebok.org>
- Fairley, R. E. and Willshire M. J. *Teaching systems engineering to software engineering students, CSEET 2011*, Software Engineering Education and Training. 219-226. 2011. ISBN 978-1-4577-0349-2.
- IEEE Computer Society. *SWEBoK Guide Version 4.0 beta*. 2022. <https://waseda.app.box.com/s/elnhhnezdycn2q2zp4fe0f2t1fvse5rn>.

Primary References

- Bourque, P. and R. E. Fairley eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. 2014. <https://www.Swebok.org>
- Brooks, F. *The Mythical Man-Month*, Anniversary Edition. Boston, MA. Addison Wesley Longman Inc. 1995.
- Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, NJ. John Wiley and Sons. 2009.

Additional References

- Pressman, R. *Software Engineering: A Practitioner's Approach*. 7th Ed. New York, NY. McGraw Hill. 2009.
- Schneidewind, N. *Systems and Software Engineering with Applications*. New York, NY. Institute of Electrical and Electronics Engineers. 2009.
- Sommerville, I. *Software Engineering*. 9th ed. Boston, MA. Addison Wesley. 2010.

Software Engineering in the Systems Engineering Life Cycle

- Lead Authors:
- Tom Hilburn and Dick Fairley
- Contributing Author:
- Alice Squires

This article describes how software engineering (SwE) life cycle processes integrate with the SE life cycle. A joint workshop organized by INCOSE, the Systems Engineering Research Center and the IEEE Computer Society was held to consider this relationship (Pyster et al. 2015). This workshop concluded that:

Software is fundamental to the performance, features, and value of most modern engineering systems. It is not merely part of the system, but often shapes the system architecture; drives much of its complexity and emergent behavior; strains its verification; and drives much of the cost and schedule of its development. Given how significant an impact software has on system development and given how complex modern systems are, one would expect the relationship between the disciplines of systems engineering (SE) and software engineering (SWE) to be well defined. However, the relationship is, in fact, not well understood or articulated.

In this article we give some of the basic relationships between SwE and SE and discuss how these can be related to some of the SEBoK knowledge areas.

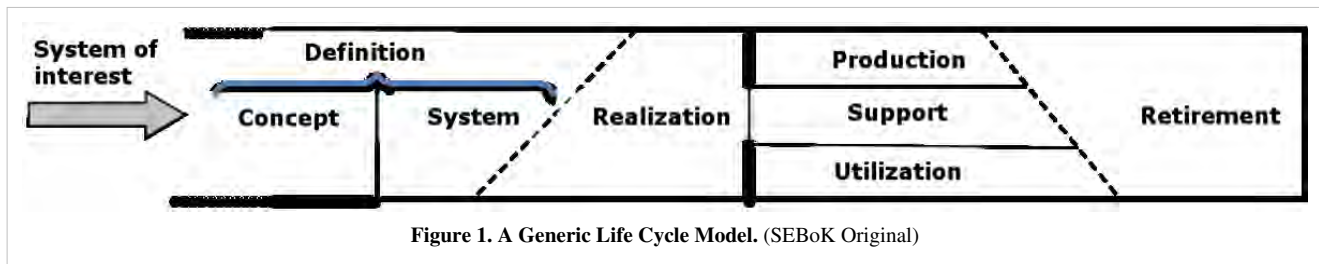
Systems Engineering and Software Engineering Life Cycles

The Guide to the Software Engineering Body of Knowledge (SWEBoK) (Bourque and Fairley, 2014) describes the life cycle of a software product as:

- analysis and design,
- construction,
- testing,
- operation,
- maintenance, and eventually
- retirement or replacement.

This life cycle is common to most other mature engineering disciplines.

In Part 3 of the SEBoK, SE and Management, there is a discussion of SE life cycle models and life cycle processes. A Generic Life Cycle Model is described and reproduced in Fig. 1 below. This is used to describe necessary stages in the life cycle of a typical engineered system.



Part 3 defines a collection of generic SE life cycle processes which define the activities and information needed across the SE life cycle. These processes include activities which contribute across the whole life cycle, with peaks of focused activity in certain stages (see Applying Life Cycle Processes for details).

The following sections provide a brief discussion of how SwE life cycle processes fit into SE life cycle process models. In practice, the details of this relationship are a key part of how a system life cycle is planned and delivered. The relationship will be shaped by the operating domain practice and solution type. Some examples of this are provided in the Implementation Examples.

Systems Engineering and Software Engineering Standards

The Systems Engineering life cycle processes described in Part 3, SE and Management, are largely based on those defined in the ISO/IEC/IEEE SE Life Cycle Processes 15288 Standard (2015).

The SWEBoK references the equivalent ISO/IEC/IEEE Software Engineering Life Cycle Processes 12207 Standard (2008), which defines a very similar set of processes for software systems. Figure 2 shows the relationship between the Enabling, Acquisition, Project and Technical Systems and Software processes in both 15288 and 12207 and the software specific processes of 12207. This alignment is from the last updates of both 12207 and 15288 in 2008. The SE processes have been further updated in 15288:2015, see Systems Engineering and Management for details. This change has not yet been applied to 12207. An update of 12207 was released in 2017, in which the alignment to 15288 was reviewed. See Harmonization of Systems Engineering Standards for more discussion of the relationships between the standards.

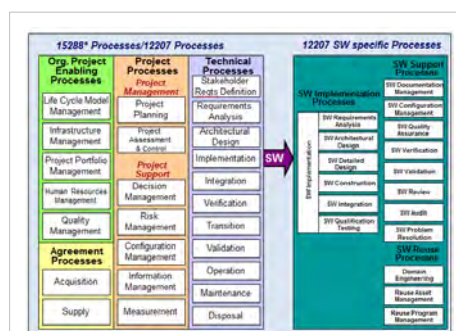


Figure 2. Aligned Process Models for ISO/IEC/IEEE 15288 & 12207: 2008 (Adapted from Roedler 2011). Reprinted with permission of Garry Roedler. All other rights are reserved by the copyright owner.

Systems Engineering and Software Engineering Life Cycle Relationships

Pyster et al. (2015) define two technical dimensions of engineered systems and of the engineering disciplines associated with them. The vertical dimensions of a system are those that modularize around technically focused engineering concerns involving specific elements of the system; the horizontal dimensions of a system involve cross-cutting concerns at the systems level. Examples of vertical concerns include quality attributes and performance effectiveness; and cost, schedule and risk of physical, organizational or human system elements associated with a particular technology domain. Examples of horizontal concerns include addressing evolving customer preferences that drive systems-level quality attributes, trade-off and optimization; resolving system architecture, decomposition and integration issues; implementing system development processes; and balancing system economics, cost, risk and schedule.

In complex systems projects, SE has a horizontal role while traditional engineering disciplines such as electrical, mechanical, and chemical engineering have vertical roles. To the extent that it is responsible for all aspects of the successful delivery of software related elements, SwE can be considered as one of the vertical disciplines. All of these traditional vertical disciplines will have some input to the horizontal dimension. However, the nature of software and its role in many complex systems makes SwE a critical discipline for many horizontal concerns. This is discussed further below.

The ISO/IEC/IEEE 12207 software engineering standard (2008) considers two situations:

- The life cycle of software products, containing minimal physical hardware, should use software specific processes and a simple life cycle
- The life cycle of systems with a significant software content (sometimes called software intensive systems) should integrate the software processes into the SE life cycle

The second of these situations is the one relevant to the practice of SE and requires a significant horizontal contribution from SwE.

The relationship central to this is the way **SwE Implementation Processes** (see Fig 2) are used in the SE life cycle to support the implementation of software intensive system elements. This simple relationship must be seen in the context of the concurrency, iteration and recursion relationship between SE life cycle processes described in Applying Life Cycle Processes. This means that, in general, software requirements and architecture processes will be applied alongside system requirements and architecture processes; while software integration and test processes are applied alongside system integration, verification and validation processes. These interrelationships help with vertical software concerns, ensuring detailed software design and construction issues are considered at the system level. They also help with horizontal concerns, ensuring whole system issues are considered and are influenced by an understanding of software. See the Nature of Software for more details.

The ways these related processes work together will depend on the systems approach to solution synthesis used and how this influences the life cycle. If a top down approach is used, problem needs and system architecture will drive software implementation and realization. If a bottom up approach is used, the architecture of existing software will strongly influence both the system solution and the problem which can be considered. In Applying Life Cycle Processes, a "middle-out" approach is described which combines these two ideas and is the most common way to develop systems. This approach needs a two-way relationship between SE and SwE technical processes.

The **SW Support Processes** may also play these vertical and horizontal roles. Part 3 contains knowledge areas on both System Deployment and Use which includes operation, maintenance and logistics; and Technical Management Processes which covers the project processes shown in Figure 2. SwE support processes focus on the successful vertical deployment and use of software system elements and the management needed to achieve this. They also support their equivalent horizontal SE processes in contributing to the success of the whole system life cycle. The **Software Reuse Processes** have a particularly important role to play in deployment and use and Product and Service Life Management processes. The latter considers Service Life Extension; Capability Updates, Upgrades, and

Modernization; and system Disposal and Retirement. All of these horizontal software engineering activities rely on the associated SE activities having a sufficient understanding of the strengths and limitations of software and SwE (see Key Points a Systems Engineer Needs to Know about Software Engineering).

The Life Cycle Models knowledge area also defines how Vee and Incremental life cycle models provide a framework to tailor the generic life cycle and process definitions to different types of system development. Both models, with some modification, apply equally to the development of products and services containing software. Thus, the simple relationships between SE and SwE processes will form the basis for tailoring to suit project needs within a selected life cycle model.

Software and Systems Challenges

Pyster et al. (2015) define three classes of software intensive systems distinguished by the primary sources of novelty, functionality, complexity and risk in their conception, development, operation and evolution. These are briefly described below:

- **Physical Systems** operate on and generate matter or energy. While they often utilize computation and software technologies as components, those components are not dominant in the horizontal dimension of engineering. Rather, in such systems, they are defined as discrete system elements and viewed and handled as vertical concerns.
- **Computational Systems** include those in which computational behavior and, ipso facto, software are dominant at the systems level. The primary purpose of these systems is to operate on and produce data and information. While these systems always include physical and human elements, these are not the predominant challenges in system development, operation and evolution.
- **Cyber-Physical Systems** are a complex combination of computational and physical dimensions. Such systems are innovative, functionally complex and risky in both their cyber and physical dimensions. They pose major horizontal engineering challenges across the board. In cyber-physical systems, cyber and physical elements collaborate in complex ways to deliver expected system behavior.

Some of the challenges of physical and computational systems are well known and can be seen in many SE and SwE case studies. For example, physical system life cycles often make key decisions about the system architecture or hardware implementation which limit the subsequent development of software architecture and designs. This can lead to software which is inefficient and difficult or expensive to change. Problems which arise later in the life of such systems may be dealt with by changing software or human elements. This is sometimes done in a way which does not fully consider SwE design and testing practices. Similarly, computational systems may be dominated by the software architecture, without sufficient care taken to consider the best solutions for enabling hardware or people. In particular, operator interfaces, training and support may not be considered leading to the need for expensive organizational fixes once they are in use. Many computational systems in the past have been developed without a clear view of the user need they contribute to, or the other systems they must work with to do so. These and other related issues point to a need for system and software engineers with a better understanding of each other's disciplines. Pyster et al. (2015) consider how SE and SwE education might be better integrated to help achieve this aim.

Examples of cyber-physical systems increasingly abound – smart automobiles, power grids, robotic manufacturing systems, defense and international security systems, supply-chain systems, the so-called internet of things, etc. In these systems there is no clear distinction between software elements and the whole system solution. The use of software in these systems is central to the physical outcome and software is often the integrating element which brings physical elements and people together. These ideas are closely aligned with the Service System Engineering approach described in Part 4.

SEBoK Part 3 includes a Business and Mission Analysis process which is based on the equivalent process in the updated ISO/IEC/IEEE 15288 (2015). This process enables SE to be involved in the selection and bounding of the

problem situation which forms the starting point for an engineered system life cycle. For cyber physical systems, an understanding of the nature of software is needed in the formulation of the problem, since this is often fundamentally driven by the use of software to create complex adaptive solution concepts. This close coupling of software, physical and human system elements across the system of interest continues throughout the system life cycle making it necessary to consider all three in most horizontal system level decisions.

The life cycle of cyber physical systems cannot be easily partitioned into SE and SwE achieving their own outcomes but working together on horizontal system issues. It will require a much more closely integrated approach, requiring systems and software engineers with a complementary set of competencies, and changes how the two disciplines are seen in both team and organizational structures. See Enabling Systems Engineering.

References

Works Cited

Bourque, P. and R. E. Fairley eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA. IEEE Computer Society. 2014. <https://www.swebok.org>.

ISO/IEC/IEEE. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015. 2015.

ISO/IEC/IEEE. *Systems and Software Engineering — Software Life Cycle Processes*. Geneva, Switzerland: International Organization for Standards (ISO)/Institute of Electrical & Electronics Engineers (IEEE) Computer Society, ISO/IEC/IEEE 12207:2008(E). 2008.

Pyster, A., Adcock, R., Ardis, M., Cloutier, R., Henry, D., Laird, L., Lawson, H. 'Bud', Pennotti, M., Sullivan, K., Wade J. "Exploring the relationship between systems engineering and software engineering." 13th Conference on Systems Engineering Research (CSER). In *Procedia Computer Science*, Volume 44, 2015, pp. 708-717. 2015.

Roedler, G. "Towards Integrated Systems and Software Engineering Standards." National Defense Industrial Association (NDIA) Conference, San Diego, CA. 2011.

Primary References

Pyster, A., Adcock, R., Ardis, M., Cloutier, R., Henry, D., Laird, L., Lawson, H. 'Bud', Pennotti, M., Sullivan, K., Wade J. Exploring the relationship between systems engineering and software engineering. 13th Conference on Systems Engineering Research (CSER). In *Procedia Computer Science*, Volume 44, 2015, pp. 708-717. 2015.

Additional References

Roedler, G. *An overview of ISO/IEC/IEE 15288, system life cycle processes*. Asian Pacific Council on Systems Engineering (APCOSE) Conference. 2010.

The Nature of Software

- Lead Authors:
- Tom Hilburn and Dick Fairley
- Contributing Author:
- Alice Squires

The nature of the software medium has many consequences for systems engineering (SE) of software-intensive systems. Fred Brooks has famously observed that four properties of software, taken together, differentiate it from other kinds of engineering artifacts (Brooks 1995). These four properties are:

1. complexity,
2. conformity,
3. changeability,
4. invisibility.

Brooks states:

Software entities are more complex for their size than perhaps any other human construct because no two parts are alike (at least above the statement level). If they are, we make the two similar parts into a subroutine — open or closed. In this respect, software systems differ profoundly from computers, buildings, or automobiles, where repeated elements abound. (Brooks 1995, p 82)

Complexity

The complexity of software arises from the large number of unique interacting parts in a software system. The parts are unique because they are encapsulated as functions, subroutines, or objects, and invoked as needed rather than being replicated. Software parts have several different kinds of interactions, including serial and concurrent invocations, state transitions, data couplings, and interfaces to databases and external systems.

Depiction of a software entity often requires several different design representations to portray the numerous static structures, dynamic couplings, and modes of interaction that exist in computer software. Complexity within the parts and in the connections among parts requires that changes undergo substantial design rigor and regression testing. Software provides functionality for components that are embedded, distributed and data centric. Software can implement simple control loops as well as complex algorithms and heuristics.

Complexity can hide defects that may not be discovered easily, thus requiring significant additional and unplanned rework.

Conformity

Software, unlike a physical product, has no underlying natural principles which it must conform to, such as Newton's laws of motion. However, software must conform to exacting specifications in the representation of each of its parts, in the interfaces to other internal parts, and in the connections to the environment in which it operates. A missing semicolon or other syntactic error can be detected by a compiler, but a defect in the program logic or a timing error may be difficult to detect until encountered during operation.

Unlike software, tolerance among the interfaces of physical entities is the foundation of manufacturing and assembly. No two physical parts that are joined together have, or are required to have, exact matches. There are no corresponding tolerances in the interfaces among software entities or between software entities and their environments. There are no interface specifications for software stating that a parameter can be *an integer plus or minus 2%*. Interfaces among software parts must agree exactly in numbers, types of parameters and kinds of couplings.

Lack of conformity can cause problems when an existing software component cannot be reused as planned because it does not conform to the needs of the product under development. Lack of conformity might not be discovered until late in a project, thus necessitating the development and integration of an acceptable component to replace the one that cannot be reused. This requires an unplanned allocation of resources (usually) and can delay project completion.

Changeability

Software coordinates the operation of physical components and provides most of the functionality in software-intensive systems. Because software is the most malleable (easily changed) element in a software-intensive system, it is the most frequently changed element. This is particularly true during the late stages of a development project and during system sustainment. However, this does not mean that software is easy to change. Complexity and the need for conformity can make changing software an extremely difficult task. Changing one part of a software system often results in undesired side effects in other parts of the system, requiring more changes before the software can operate at maximum efficiency.

Invisibility

Software is said to be invisible because it has no physical properties. While the effects of executing software on a digital computer are observable, software itself cannot be seen, tasted, smelled, touched, or heard. Software is an intangible entity because our five human senses are incapable of directly sensing it.

Work products such as requirements specifications, design documents, source code and object code are representations of software, but they are not the software. At the most elemental level, software resides in the magnetization and current flow in an enormous number of electronic elements within a digital device. Because software has no physical presence, software engineers must use different representations at different levels of abstraction in an attempt to visualize the inherently invisible entity.

Uniqueness

One other point about the nature of software that Brooks alludes to but does not explicitly call out is the uniqueness of software. Software and software projects are unique for the following reasons:

- Software has no physical properties;
 - Software is the product of intellect-intensive teamwork;
 - Productivity of software developers varies more widely than the productivity of other engineering disciplines;
 - Estimation and planning for software projects is characterized by a high degree of uncertainty, which can be at best partially mitigated by best practices;
 - Risk management for software projects is predominantly process-oriented;
 - Software alone is useless, as it is always a part of a larger system; and
 - Software is the most frequently changed element of software intensive systems.
-

References

Works Cited

Brooks, F. *The Mythical Man-Month*, Anniversary Edition. Boston, MA. Addison Wesley Longman Inc. 1995.

Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, New Jersey. John Wiley and Sons. 2009.

Primary References

Bourque, P. and R.E. Fairley eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA. IEEE Computer Society. 2014.

Brooks, F. *The Mythical Man-Month*, Anniversary Edition. Boston, MA. Addison Wesley Longman Inc. 1995.

Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, New Jersey. John Wiley and Sons. 2009.

An Overview of the SWEBOK Guide

- Lead Authors:
- Hironori Washizaki, Maria-Isabel Sanchez-Segura, Juan Garbajosa, Steve Tockey, Kenneth E Nidiffer, and Annette D. Reilly

Software is everywhere. Software and systems engineers are leading the way for the digital transformation of social, economic, military, and environmental systems at an increasing rate of change. Advanced software-enabled capabilities allow software and systems engineers to address and anticipate emerging and complex challenges. Most systems of any complexity today are modeled and controlled by software; hence, the term “software intensive systems” is commonly used to describe them. At the end of 2022, the IEEE Computer Society (IEEE-CS) released beta version 4 of the Software Engineering Body of Knowledge (SWEBOK) (IEEE Computer Society 2022). When the final version 4.0 is published, it will replace version 3 which appeared in 2014 (Bourque and Fairley 2014). As have previous versions, SWEBOK 4.0 will help guide both systems engineering and software engineering professionals to long-established software life cycle processes and new insights for dealing with the demands of the digital era. This article uses SWEBOK 4.0 beta to explain how and why the new SWEBOK takes into consideration the needs of digital era software engineers immersed in an industry demanding visionary solutions.

Introduction

ISO/IEC/IEEE Systems and Software Engineering Vocabulary (SEVOCAB) defines software engineering as “the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.” (ISO/IEC/IEEE n.d.)

It is important to note that systems engineering, and software engineering are distinct disciplines with different but somewhat overlapping core competencies. Due to the inherent characteristics of software intensive systems, these disciplines often approach problem solving from different perspectives. For example, systems engineers apply their problem-solving skills to develop physical, computational, social, and hybrid systems. Depending on the system, those skills could be based on a wide range of disciplines including mathematics, any of the physical and social sciences, law, management, and a myriad of other disciplines. Software engineers tend to apply their problem-solving skills on a narrower set of disciplines, especially discrete mathematics and computer science, to develop computational systems and subsystems. Also, software is a logical medium. Software components of software-enabled systems are logical constructions expressed in algorithmic form. This contrasts with the physical and social components of systems that are realized through mechanical, electrical, chemical, biological, social, human and other elements. Both disciplines are critical to solving the complex challenges of the future as evidenced

by disruptive innovation and increasing complexity in new systems and systems of systems which is causing a convergence among disciplines.

SWEBOK Guide Objectives

The Guide to the Software Engineering Body of Knowledge (SWEBOK), published by the IEEE Computer Society (IEEE CS), represents the current state of generally accepted, consensus-based knowledge emanating from the interplay between software engineering theory and practice. (Bourke and Fairley 2014, IEEE Computer Society 2022). The objectives include the provision of guidance for learners, researchers, and practitioners to identify and share a mutual understanding of “generally accepted knowledge” in software engineering. This guidance defines the boundary between software engineering and related disciplines, and provides a foundation for certifications and educational curricula.

The Guide should not be confused with the Body of Knowledge itself which exists in published literature. It is similar in this regard to the SEBoK. For example, the Guide addresses knowledge areas (KAs) in terms of “what” activities are being done versus “how” they are accomplished. Bodies Of Knowledge (BOKs) are often a formal way of referring to core competencies, or what should be known to be successful in a practitioner’s area of expertise. (Washizaki, et al. 2023) Every profession is based on a body of knowledge, although that knowledge is not always defined in a concise manner. In cases where normality exists, the body of knowledge is “generally recognized” by practitioners and may be codified in a variety of ways for a variety of different uses. But in many cases, a guide to a body of knowledge is formally documented, usually in a form that permits it to be used for such purposes as development and accreditation of academic and training programs, certification of specialists, or professional licensing. (IEEE Computer Society 2022, IEEE Computer Society 2014) The purpose of the Guide is to describe the portion of the Body of Knowledge that is accepted, to organize that portion, and to provide topical access to it. The Guide is also aligned with the following objectives:

1. To promote a consistent view of software engineering worldwide
2. To specify the scope of, and clarify the place of software engineering with respect to other disciplines such as computer science, project management, computer engineering, and mathematics
3. To characterize the contents of the software engineering discipline
4. To provide topical access to the Software Engineering Body of Knowledge
5. To provide a foundation for curriculum development and for individual certification and licensing material.

Overview of the SWEBOK Guide

The Guide represents the current state of accepted, consensus-based knowledge emanating from the interplay between software engineering theory and practice. The origins of the Guide go back to the late 1990s. Much like the software engineering discipline, the Guide has continued to evolve over the last 20 years to reflect the educational, industrial, social, technical, and technological changes in society. Version 4 of the Guide will be released in 2023 to improve its currency, readability, consistency, and usability.

Table 1 provides the high-level table of contents from Guide V4. Note that sections with an "*" are new (were not previously included in Guide 3). The current draft Guide V4 contains 18 knowledge areas (KAs), followed by several appendices. A KA is an identified area of software engineering defined by its knowledge requirements and described in terms of its concepts, component processes, practices, inputs, outputs, tools, and techniques. All KAs have been updated in V4 to reflect changes in software engineering since the publication of Guide V3. The new version reflects modern development practices, techniques, and the advancement of standards. Especially, agile and DevOps have been incorporated into almost all KAs since these models have been widely accepted since Guide V3 was published. Agile models typically have frequent demonstrations of working software to a customer in short iterative cycles with agile practices involving KAs. Furthermore, emerging platforms and technologies, including artificial intelligence (AI), machine learning (ML), and Internet of Things (IoT), have been incorporated into the

foundation KAs. Also, there are three new KAs: Software Architecture, Software Operations, and Software Security. It is important to note there has also been more emphasis on Software Security in other relevant KAs such as software requirements. These additions better reflect contemporary software engineering practice, the pressing need to address cybersecurity as early as possible in developing distributed, networked systems, and non-distributed systems for use by the public at large, and the need for specialized skills to be able to work effectively in these areas.

Table 1. SWEBOK v4 Table of Contents (SEBoK Original)

Knowledge Areas

- Requirements
- Architecture*
- Design
- Construction
- Testing
- Operations*
- Maintenance
- Configuration Management
- Engineering Management
- Process
- Models and Methods
- Quality
- Security*
- Professional Practice
- Economics
- Computing Foundations
- Mathematical Foundations
- Engineering Foundations

The 18 KAs in Guide V4 are:

KA1 - Software Requirements: Describes the activities involved in eliciting, analyzing, specifying, validating, and managing software requirements. The importance of requirements documentation for long-term maintenance is added. The “what’s” and “how’s” of work on software requirements in a project should be determined by the type of constructed software and not by the project life cycle. Also, it includes a deeper, broader coverage of requirements specification techniques, including model-driven requirements specification and acceptance criteria-based requirements specification.

KA2 – Software Architecture: This is a new KA resulting from a realization that architecture is a significant and distinct discipline over and above software design. Software architecture consists of fundamental structures of software elements, relations among them, and properties of both elements and relations. As software tends toward larger, ever more complex systems, architecture concerns move beyond construction to connectivity while assuring new levels of quality for safety, security, and dependability. Software architecture aims to “satisfice” all stakeholders, while the focus of software design is on the transformation of that vision into a feasible implementation.

KA 3 - Software Design: Focuses on the process of transforming requirements into a representation of the software system. Software is pervasive and critical in all aspects of modern life. The success of pervasive software depends on broadening the background of people with design skills, especially as emerging technologies, faster delivery times,

and emphasis on agile, lean, and incremental design impact the development process.

KA4 - Software Construction: Covers the activities involved in translating the software design into executable code. This section has been updated to reflect modern construction techniques and practices: managing dependencies, cross-platform development and migration, feedback loops for construction, visual programming, and low-code/zero-code (i.e., called no code) platforms. Furthermore, major updates were made to some of the existing sections, especially about reuse, life-cycle models, construction languages and environments.

KA5 - Software Testing: Deals with the various approaches and techniques used to assess software correctness and verify its conformance to requirements. This material has been revised to align with the shift left testing movement. This KA provides new content about software testing in recent development processes (like Agile, DevOps, and Test-Driven Development), application domains (like automotive, healthcare, mobile, legal, and IoT), emerging technologies (such as AI, blockchain, cloud) and quality attributes (like security and privacy).

KA6 - Software Engineering Operations: This is a new knowledge area that addresses the evolution of the role of software engineers to include DevOps and infrastructure as a Service (IaaS) activity while eliminating the organizational silos between development, maintenance, and operations. This new KA describes operations fundamentals, planning, delivery and control activities, and techniques.

KA 7 - Software Maintenance (SM): Software maintenance addresses fundamentals, processes, and techniques to provide cost-effective support for software in operation. Activities, such as determining the logistical support needed for the transition, are performed, and applied during the pre-delivery stage, whereas software surveillance, modification, training, and operating or interfacing with a help desk are post-stage activities. Software Maintenance categories and the software process have been updated to align with the 2022 version of ISO/IEC/IEEE 14764. (ISO/IEEE/IEEE 2022) Also, continuous integration, delivery, testing, and deployment have been added as a new topic in response to the growing popularity of regrouping development, maintenance, and operations tasks to improve software engineering productivity.

KA 8 - Software Configuration Management (SCM): Focuses on four basic functions of SCM: Configuration Identification: identifying all configuration items (CI) of a software system's state; Control changes: coordinating and limiting access to configurations in a software system; Configuration auditing: a series of reviews confirming that changes are being made to conform to a software system's desired state; and Status Accounting: automatically recording information about CIs, relationships, baselines and changes on any CI developed, as well as why they were made, when they were made, and who made them. In SWEBOK Guide V4, SCM has been explained considering all its facets. It stresses that the SCM process plan must be defined first before all the decisions and commitments included in the plan are somehow incorporated into existing tools for execution rather than the other way around, as was the usual practice in the past. Keeping track of configuration items (CI) relationships is essential to visualize the potential impact of a change on a CI over other CIs.

KA9 - Software Engineering Management: Describes the activities of the planning, organizing, estimating, and tracking of software projects. Practices are changing to meet the needs of society caused by increases in the size and complexity of software as well as greater pressure to quickly release software enabling products to market in rapidly changing environments. The results are fundamental management shifts in how work is performed.

KA10 - Software Engineering Process: Describes the activities, methods, and techniques used to define, implement, assess, and improve the software development process. Software engineers face a challenging and evolving, highly technological landscape due to fast-moving technology, societal events and changes, and a trend towards digitalization involving very many different domains. This is a point of no return that has facilitated the ongoing adoption of Agile and DevOps. Nevertheless, now more than ever, the software engineering processes applied to create products will continue to evolve, profiting from advances in the engineering dimension and learning, new and more advanced tools, and the latest knowledge from the other KAs. The KA describes this new consensus, highlighting the interaction with other KAs.

KA 11 - Software Engineering Models and Methods: Introduces different models, methods, and tools that support the software development process. Models have been updated and accurately classified by type. Aspect-oriented development has been added in the document as well as fundamental model-driven and model-based methods, have also been introduced. Agile methods have been extended a great deal to incorporate modern techniques, such as lean development, as well as large-scale and enterprise agile methods. On this note, DevOps and release engineering have also been introduced to clarify the release aspect of agile methods.

KA12 - Software Quality: Addresses the activities and techniques involved in achieving and evaluating software quality. This chapter was overhauled for alignment with notions of processes/product quality. It now includes new topics: (1) Software Dependability and Integrity Levels, (2) Standards, Models and Certifications, (3) Policies, Processes and Procedures, and (4) Quality Control and Testing.

KA13 – Software Security: This is a new knowledge area that focuses on the broader topics of security, particularly security fundamentals, security management, security tools, and domain-specific security, as well as major security engineering activities (such as security testing) that were briefly described under the Computing Foundations KA in the last SWEBOK edition. The existence of a separate KA emphasizes that security must be a first-class quality attribute in any development since almost any software system is connected to others, resulting in increased security risks.

KA14 - Software Engineering Professional Practice: Covers ethical considerations, professional codes of conduct, and the role of software engineering in society. Additions and revisions address the following areas: (1) Professional practices following generally accepted practices, standards, and guidelines set forth by the applicable professional societies, (2) User interface/user experience (UI/UX) inclusive design, (3) Considerations on diversity and inclusion, and (4) Considerations on agility.

KA15 - Software Engineering Economics: Focuses on the essence of engineering economics, the science of making decisions, and broadening the more traditional, purely financial view of engineering economics. Value does not always derive from money alone; that is, value can also derive from the “unquantifiable” such as, corporate citizenship, employee well-being, environmental friendliness, customer loyalty, and so on. This KA also focuses on more systematic pre-project decisions where a project is not under development but is being envisioned.

Note: Minor improvements in topic presentation in the next three KA 16-18. Discussions of Artificial Intelligence and Machine Learning were added. Furthermore, the revision includes a deeper coverage of measurement, particularly its implications for programming languages, and a more comprehensive discussion of root cause analysis.

KA16 - Computing Foundations: Covers the fundamental concepts and theories that underlie the discipline of software engineering.

KA 17 - Mathematical Foundations: Introduces mathematical techniques and principles relevant to software engineering, such as logic, probability, and discrete mathematics.

KA - 18 Engineering Foundations: Covers engineering principles and practices applicable to software engineering, including system engineering and project management.

Concluding Comments

A significant component of productivity today is the capabilities enabled by software systems. Software is a maturing science and engineering discipline; therefore, a need exists to continuously provide new guidance materials that reflect areas that are becoming important in modern software engineering. The Software Engineering Body of Knowledge Version V4 is a leading product which will help address this issue. It will serve as a reference for educators, practitioners, and organizations to understand the essential concepts and skills needed to design, develop, and maintain current and future software systems.

SWEBOK V4 represents the next step in the evolution of the software engineering profession and is written under the auspices of the Profession and Education Activities Board of the IEEE Computer Society. An international team of subject matter experts was assembled with the goal of developing the update. Given the purpose of this article, a significant portion of this document includes verbatim or near-verbatim elements of the SWEBOK Guide V4 which is about to go out for public review and that of (Washnizaki, et al. 2023) written by several members of the international team.

SWEBOK is described inside the SEBoK and there are some clear similarities and differences between the two BOKs. Each BOK is a comprehensive guide that outlines the core knowledge areas, principles, and best practices. SWEBOK concentrates specifically on the software engineering domain, providing a detailed overview of software development processes and methodologies. SEBoK, on the other hand, has a broader scope and addresses the entire systems engineering domain, including software and beyond, considering the interaction and integration of various components in complex systems.

References

Works Cited

Bourque, P. and R. E. Fairley eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA. IEEE Computer Society. 2014.

IEEE Computer Society. *Software engineering competency model (SWECOM)*. Version 1.0. 2014. <https://www.computer.org/volunteering/boards-and-committees/professional-educational-activities/software-engineering-competency-model>.

IEEE Computer Society. *SWEBOK Guide Version 4.0 beta*. 2022. <https://waseda.app.box.com/s/elnhhnezdycn2q2zp4fe0f2t1fvse5rn>.

ISO/IEC/IEEE. *ISO/IEC/IEEE Systems and Software Engineering – Vocabulary*. 2017. <https://www.iso.org/standard/71952.html><https://www.iso.org/standard/71952.html>.

ISO/IEC/IEEE. *ISO/IEC/IEEE 14764-2022 Software Life Cycle Processes – Maintenance*. 2022. <https://www.iso.org/standard/80710.html>.

Washizaki, H., Sanchez-Segura, M-I., Garbajosa, J., Tockey, S., Nidiffer, K. E. “Envisioning Software Engineer Training Needs in The Digital Era Through the SWEBOK V4 Prism”, Proceedings of the IEEE International Conference on Software Engineering Education and Training (CSEE&T 2023), August 8-9, 2023. Waseda University, Tokyo Japan.

Primary References

Bourque, P. and R. E. Fairley eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA, USA: IEEE Computer Society. 2014. <https://www.swebok.org>.

IEEE Computer Society. *SWEBOK Guide Version 4.0 beta*. 2022. <https://waseda.app.box.com/s/elnhhnezdycn2q2zp4fe0f2t1fvse5rn>.

Additional References

None.

Key Points a Systems Engineer Needs to Know about Software Engineering

- Lead Author:
- Dick Fairley
- Contributing Author:
- Alice Squires

The field of software engineering is extensive and specialized. Its importance to modern systems makes it necessary for systems engineers to be knowledgeable about software engineering and its relationship to systems engineering.

Key Concepts a Systems Engineer Needs to Know about Software Engineering

The following items are significant aspects that systems engineers need to know about software and software engineering. Most are documented in (Fairley and Willshire 2011):

1. **For the time, effort, and expense devoted to developing it, software is more complex than most other system components** - Software complexity arises because few elements in a software program (even down to the statement level) are identical, as well as because of the large number of possible decision paths found even in small programs, with the number of decision paths through a large program often being astronomical. There are several detailed references on software complexity. The SWEBOK (Bourque and Fairley 2014) discusses minimizing complexity as part of software construction fundamentals. Zuse (1991) has a highly cited article on software complexity measures and methods. Chapters 2 and 3 of the SWEBOK also have further references.
2. **Software testing and reviews are sampling processes** - In all but the simplest cases, exhaustive testing of software is impossible because of the large number of decision paths through most programs. Also, the combined values of the input variables selected from a wide combinatorial range may reveal defects that other combinations of the variables would not detect. Software test cases and test scenarios are chosen in an attempt to gain confidence that the testing samples are representative of the ways the software will be used in practice. Structured reviews of software are an effective mechanism for finding defects, but the significant effort required limits exhaustive reviewing. Criteria must be established to determine which components (or sub-components) should be reviewed. Although there are similar concerns about exhaustive testing and reviewing of physical products, the complexity of software makes software testing, reviews, and the resulting assurance provided more challenging. Other points include:
 1. All software testing approaches and techniques are heuristic. Hence, there is no universal "best" approach, practice, or technique for testing, since these must be selected based on the software context.
 2. Exhaustive testing is not possible.
 3. Errors in software tend to cluster within the software structures; therefore, any one specific approach or a random approach to testing is not advised.
 4. Pesticide paradox exists. As a result, running the same test over and over on the same software-system provides no new information.
 5. Testing can reveal the presence of defects but cannot guarantee that there will be no errors, except under the specific conditions of a given test.
 6. Testing, including verification and validation (V&V), must be performed early and continually throughout the lifecycle (end to end).
 7. Even after extensive testing and V&V, errors are likely to remain after long term use of the software.
 8. Chapter 4 of the SWEBOK discusses software testing and provides a bibliography.
3. **Software often provides the interfaces that interconnect other system components** - Software is often referred to as the *glue* that holds a system together because the interfaces among components, as well as the

interfaces to the environment and other systems, are often provided by digital sensors and controllers that operate via software. Because software interfaces are behavioral rather than physical, the interactions that occur among software components often exhibit emergent behaviors that cannot always be predicted in advance. In addition to component interfaces, software usually provides the computational and decision algorithms needed to generate command and control signals. The SWEBOK has multiple discussions of interfaces: Chapter 2 on Software Design is a good starting point and includes a bibliography.

4. **Every software product is unique** - The goal of manufacturing physical products is to produce replicated copies that are as nearly identical as much as possible, given the constraints of material sciences and manufacturing tools and techniques. Because replication of existing software is a trivial process (as compared to manufacturing of physical products), the goal of software development is to produce one perfect copy (or as nearly perfect as can be achieved given the constraints on schedule, budget, resources, and technology). Much of software development involves altering existing software. The resulting product, whether new or modified, is uniquely different from all other software products known to the software developers. Chapter 3 of the SWEBOK provides discussion of software reuse and several references.
 5. **In many cases, requirements allocated to software must be renegotiated and reprioritized** - Software engineers often see more efficient and effective ways to restate and prioritize requirements allocated to software. Sometimes, the renegotiated requirements have system-wide impacts that must be taken into account. One or more senior software engineers should be, and often are, involved in analysis of system-level requirements. This topic is addressed in the SWEBOK in Chapter 1, with topics on the iterative nature of software and change management.
 6. **Software requirements are prone to frequent change** - Software is the most frequently changed component in complex systems, especially late in the development process and during system sustainment. This is because software is perceived to be the most easily changed component of a complex system. This is not to imply that changes to software requirements and the resulting changes to the impacted software can be easily done without undesired side effects. Careful software configuration management is necessary, as discussed in Chapter 6 of the SWEBOK, which includes extensive references.
 7. **Small changes to software can have large negative effects** (A corollary to frequently changing software requirements: *There are no small software changes*) - In several well-known cases, modifying a few lines of code in very large systems that incorporated software negatively impacted the safety, security, and/or reliability of those systems. Applying techniques such as traceability, impact analysis, object-oriented software development, and regression testing reduces undesired side effects of changes to software code. These approaches limit but do not eliminate this problem.
 8. **Some quality attributes for software are subjectively evaluated** - Software typically provides the interfaces to systems that have human users and operators. The intended users and operators of these systems often subjectively evaluate quality attributes, such as ease of use, adaptability, robustness, and integrity. These quality attributes determine the acceptance of a system by its intended users and operators. In some cases, systems have been rejected because they were not judged to be suitable for use by the intended users in the intended environment, even though those systems satisfied their technical requirements. Chapter 10 of the SWEBOK provides an overview of software quality, with references.
 9. **The term *prototyping* has different connotations for systems engineers and software engineers** - For a systems engineer, a prototype is typically the first functioning version of hardware. For software engineers, software prototyping is primarily used for two purposes: (1) as a mechanism to elicit user requirements by iteratively evolving mock-ups of user interfaces, and (2) as an experimental implementation of some limited element of a proposed system to explore and evaluate alternative algorithms. Chapter 1 of the SWEBOK discusses this and provides excellent references.
 10. **Cyber security is a present and growing concern for systems that incorporate software** - In addition to the traditional specialty disciplines of safety, reliability, and maintainability, systems engineering teams increasingly
-

include security specialists at both the software level and the systems level in an attempt to cope with the cyber-attacks that may be encountered by systems that incorporate software. Additional information about System Security can be found in the Systems Engineering and Quality Attributes Part.

11. **Software growth requires spare capacity** - Moore's Law no longer fully comes to the rescue (Moore, 1965). As systems adapt to changing circumstances, the modifications can most easily be performed and upgraded in the software, requiring additional computer execution cycles and memory capacity (Belady and Lehman 1979). For several decades, this growth was accommodated by Moore's Law, but recent limits that have occurred as a result of heat dissipation have influenced manufacturers to promote potential computing power growth by slowing down the processors and putting more of them on a chip. This requires software developers to revise their programs to perform more in parallel, which is often an extremely difficult problem (Patterson 2010). This problem is exacerbated by the growth in mobile computing and limited battery power.
12. **Several Pareto 80-20 distributions apply to software** - These refers to the 80% of the avoidable rework that comes from 20% of the defects, that 80% of the defects come from 20% of the modules, and 90% of the downtime comes from at most 10% of the defects (Boehm and Basili 2001). These, along with recent data indicating that 80% of the testing business value comes from 20% of the test cases (Bullock 2000), indicate that much more cost-effective software development and testing can come from determining which 20% need the most attention.
13. **Software estimates are often inaccurate** - There are several reasons software estimates are frequently inaccurate. Some of these reasons are the same as the reasons systems engineering estimates are often inaccurate: unrealistic assumptions, vague and changing requirements, and failure to update estimates as conditions change. In addition, software estimates are often inaccurate because productivity and quality are highly variable among seemingly similar software engineers. Knowing the performance characteristics of the individuals who will be involved in a software project can greatly increase the accuracy of a software estimate. Another factor is the cohesion of the software development team. Working with a team that has worked together before and knowing their collective performance characteristics can also increase the accuracy of a software estimate. Conversely, preparing an estimate for unknown teams and their members can result in a very low degree of accuracy. Chapter 7 of the SWEBOK briefly discusses this further. Kitchenam (1997) discusses the organizational context of uncertainty in estimates. Lederer and Prasad (1995) also identify organizational and management issues that increase uncertainty; additionally, a recent dissertation from Sweden by Magazinus (2012) shows that the issues persist.
14. **Most software projects are conducted iteratively** - "Iterative development" has a different connotation for systems engineers and software engineers. A fundamental aspect of iterative software development is that each iteration of a software development cycle adds features and capabilities to produce a next working version of partially completed software. In addition, each iteration cycle for software development may occur on a daily or weekly basis, while (depending on the scale and complexity of the system) the nature of physical system components typically involves iterative cycles of longer durations. Classic articles on this include (Royce 1970) and (Boehm 1988), among others. Larman and Basili (2003) provide a history of iterative development, and the SWEBOK discusses this in life cycle processes in Chapter 8.
15. **Teamwork within software projects is closely coordinated** - The nature of software and its development requires close coordination of work activities that are predominately intellectual in nature. Certainly, other engineers engage in intellectual problem solving, but the collective and ongoing daily problem solving required of a software team requires a level of communication and coordination among software developers that is of a different, more elevated type. Highsmith (2000) gives a good overview.
16. **Agile development processes are increasingly used to develop software** - Agile development of software is a widely used and growing approach to developing software. Agile teams are typically small and closely coordinated, for the reasons cited above. Multiple agile teams may be used on large software projects, although this is highly risky without an integrating architecture (Elssamadisy and Schalliol 2002). Agile development

proceeds iteratively in cycles that produce incremental versions of software, with cycle durations that vary from one day to one month, although shorter durations are more common. Among the many factors that distinguish agile development is the tendency to evolve the detailed requirements iteratively. Most agile approaches do not produce an explicit design document. Martin (2003) gives a highly cited overview.

17. Verification and validation (V&V) of software should preferably proceed incrementally and iteratively -

Iterative development of working product increments allows incremental verification, which ensures that the partial software product satisfies the technical requirements for that incremental version; additionally, it allows for the incremental validation (ISO/IEC/IEEE 24765) of the partial product to make certain that it satisfies its intended use, by its intended users, in its intended environment. Incremental verification and validation of working software allows early detection and correction of encountered problems. Waiting to perform integration, verification, and validation of complex systems until later life cycle stages, when these activities are on the critical path to product release, can result in increased cost and schedule impacts. Typically, schedules have minimal slack time during later stages in projects. However, with iterative V&V, software configuration management processes and associated traceability aspects may become complex and require special care to avoid further problems. Chapter 4 of the SWEBOK discusses software testing, and provides numerous references, including standards. Much has been written on the subject; a representative article is (Wallace and Fujii 1989).

18. Performance trade-offs are different for software than systems - Systems engineers use “performance” to denote the entire operational envelope of a system; whereas software engineers use “performance” to mean response time and the throughput of software. Consequentially, systems engineers have a larger design space in which to conduct trade studies. In software, performance is typically enhanced by reducing other attributes, such as security or ease of modification. Conversely, enhancing attributes such as security and ease of modification typically impacts performance of software (response time and throughput) in a negative manner.

19. Risk management for software projects differs in kind from risk management for projects that develop physical artifacts - Risk management for development of hardware components is often concerned with issues such as supply chain management, material science, and manufacturability. Software and hardware share some similar risk factors: uncertainty in requirements, schedule constraints, infrastructure support, and resource availability. In addition, risk management in software engineering often focuses on issues that result from communication problems and coordination difficulties within software development teams, across software development teams, and between software developers and other project members (e.g., hardware developers, technical writers, and those who perform independent verification and validation). See (Boehm 1991) for a foundational article on the matter.

20. Software metrics include product measures and process measures - The metrics used to measure and report progress of software projects include product measures and process (ISO/IEC/IEEE 24765) measures. Product measures include the amount of software developed (progress), defects discovered (quality), avoidable rework (defect correction), and budgeted resources used (technical budget, memory and execution cycles consumed, etc.). Process measures include the amount of effort expended (because of the people-intensive nature of software development), productivity (software produced per unit of effort expended), production rate (software produced per unit time), milestones achieved and missed (schedule progress), and budgeted resources used (financial budget). Software metrics are often measured on each (or, periodically, some) of the iterations of a development project that produces a next working version of the software. Chapter 8 and Chapter 7 of the SWEBOK address this.

21. Progress on software projects is sometimes inadequately tracked - In some cases, progress on software projects is not adequately tracked because relevant metrics are not collected and analyzed. A fundamental problem is that accurate tracking of a software project depends on knowing how much software has been developed that is suitable for delivery into the larger system or into a user environment. Evidence of progress in the form of working software is one of the primary advantages of the iterative development of working software increments.

References

Works Cited

- Belady, L. and M. Lehman. "Characteristics of large systems." In P. Wegner (ed.), *Research Directions in Software Technology*. Cambridge, MA. MIT Press. 1979.
- Boehm, B. "Software risk management: Principles and practices." *IEEE Software* 8, no. 1 (1991) 32-41.
- Boehm, B. and V. Basili. "Software defect reduction Top 10 List." *Computer*. 34, no. 1 (2001) 135-137.
- Brooks, F. *The Mythical Man-Month, Anniversary Edition*. Boston, MA. Addison Wesley Longman Inc. 1995.
- Bullock, J. "Calculating the value of testing." *Software Testing and Quality Engineering*, May-June, 56-62. 2000.
- DeMarco, T. and T. Lister. *Peopleware: Predictive Projects and Teams*. New York, Dorset House. 1987.
- Elssamadisy, A. and G. Schalliol. "Recognizing and responding to 'bad smells' in extreme programming." *Proceedings, ICSE 2002, ACM-IEEE*, 617-622. 2002.
- Fairley, R. E. and M. J. Willshire. "Teaching software engineering to undergraduate systems engineering students." *Proceedings of the 2011 American Society for Engineering Education (ASEE) Annual Conference and Exposition*. 26-29 June 2011. Vancouver, BC, Canada.
- Kitchenham, B. "Estimates, uncertainty, and risk." *IEEE Software* 14, no. 3 (1997) 69-74.
- Larman, C. and V.R. Basili. "Iterative and incremental developments: A brief history." *Computer* 36, no.6 (2003) 47-56.
- Lederer, A. L. and J. Prasad. "Causes of inaccurate software development cost estimates." *Journal of Systems and Software* 31, no. 2 (1995) 125-134.
- Magazinius, A. *Exploring Software Cost Estimation Inaccuracy*. Doctoral Dissertation. Chalmers University of Technology. Goteborg, SE. 2012.
- Martin, R. C. *Agile Software Development: Principles, Patterns and Practices*. Upper Saddle River, NJ. Prentice Hall. 2002.
- Moore, G.E. "Cramming more components onto integrated circuits," *Electronics Magazine*, April 19, 1965.
- Patterson, D. "The trouble with multicore." *IEEE Spectrum*, July, 28-32, 52-53. 2010.
- Royce, W. W. "Managing the development of large software systems." *Proceedings of IEEE WESCON*. August 1970.
- Wallace, D. R. and R. U. Fujii. "Software verification and validation: An overview." *IEEE Software* 6, no. 3 (1989) 10-17.
- Zuse, Horst. *Software complexity: Measures and methods*. Hawthorne, NJ. Walter de Gruyter and Co. 1991.

Primary References

- Bourque, P. and R. E. Fairley eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA. IEEE Computer Society. 2014.
- Brooks, Fred. *The Mythical Man-Month, Anniversary Edition*. Reading, MA. Addison Wesley. 1995.
- Fairley, R. E. *Managing and Leading Software Projects*. Hoboken, NJ. John Wiley & Sons. 2009.
- Project Management Institute (PMI). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA. PMI. 2013.

Additional References

PMI. *Software Extension to the PMBOK® Guide*, 5th ed. PMI and IEEE Computer Society. 2013.

Pyster, A., M. Ardis, D. Frailey, D. Olwell, A. Squires. "Global workforce development projects in software engineering." *Crosstalk - The Journal of Defense Software Engineering*, Nov/Dec, 36-41. 2010. <https://www.dtic.mil/cgi-bin/GetTRDoc?AD=ADA535633>

Software Engineering Features - Models, Methods, Tools, Standards, and Metrics

- Lead Author:
- Tom Hilburn

In recent decades, software has become ubiquitous. Almost all modern engineered systems include significant software subsystems; this includes systems in the transportation, finance, education, healthcare, legal, military, and business sectors. Along with the increase in software utility, capability, cost, and size there has been a corresponding growth in methods, models, tools, metrics and standards, which support software engineering.

Chapter 10 of the SWEBOK discusses modeling principles and types, and the methods and tools that are used to develop, analyze, implement, and verify the models. The other SWEBOK chapters on the software development phases (e.g., Software Design) discuss methods and tools specific to the phase. Table 1 identifies software engineering features for different life-cycle phases. The table is not meant to be complete; it simply provides examples. In Part 2 of the SEBoK there is a discussion of models and the following is one of the definitions offered: "an abstraction of a system, aimed at understanding, communicating, explaining, or designing aspects of interest of that system" (Dori 2002).

For the purposes of Table 1 the definition of a model is extended to some aspect of the software system or its development. As an example, "Project Plan" is listed as a model in the Software Management area. The idea is that the Project Plan provides a model of how the project is going to be carried out: the project team organization, the process to be used, the work to be done, the project schedule, and the resources needed.

Table 1: SWE Features (SEBoK Original)

Life-Cycle Activity	Models	Methods & Tools	Standards
Software Management	• Life-Cycle Process Model • Work Breakdown Structure • Constructive Cost Model (COCOMO) • Project Plan • Configuration Management (CM) Plan • Risk Management Plan	• Effort, Schedule and Cost Estimation • Risk Analysis • Data Collection • Project Tracking • CM Management • Iterative/Incremental Development • Agile Development	• [IEEE 828] • [IEEE 1058] • [IEEE 1540] • [IEEE 12207]
Software Requirements	• Functional Model • User Class Model • Data Flow Diagram • Object Model • Formal Model • User Stories	• Requirements Elicitation • Prototyping • Structural Analysis • Data-Oriented Analysis • Object-Oriented Analysis • Object Modeling Language (OML) • Formal Methods • Requirements Specification • Requirements Inspection	• [IEEE 830] • [IEEE 1012] • [IEEE 12207]

Software Design	• Architectural Model	• Structured Design	• [IEEE 1012]
	• Structure Diagram	• Object-Oriented Design	• [IEEE 1016]
	• Object Diagram	• OML	• [IEEE 12207]
	• Class Specification	• Modular Design	• [IEEE 42010]
	• Data Model	• Integrated Development Environment (IDE)	
Software Construction	• Detail Design Document	• Database Management System (DBMS)	
		• Design Review	
		• Refinement	
		• Detailed Design	• [IEEE 1008]
		• Functional Programming	• [IEEE 1012]
	• Pseudocode	• Object-Oriented Programming	• [IEEE 1016]
	• Flow Chart	• IDE	• [IEEE 12207]
	• Program Code	• DBMS	
	• Unit Test Plan	• Black Box/White Box Testing	
	• Integration Test Plan	• Basic Path Testing	
		• Unit Testing	
		• Code Review	
		• Proof of Correctness	
		• Software Reuse	
		• Integration	
		• Integration Testing	
Software Testing	• System Test Plan	• Usability Testing	• [IEEE 829]
	• Reliability Model	• System Testing	• [IEEE 1012]
	• Software Maintenance Process	• Acceptance Testing	• [IEEE 12207]
		• Regression Testing	
		• Reliability Testing	
		• Non-Functional Software Testing	
Software Maintenance	• Software Maintenance Process	• Automated Testing Tools	• [IEEE 1219]
		• Maintenance Change	• [IEEE 12207]
		• Impact Analysis	• [IEEE 14764]
		• Inventory Analysis	
		• Restructuring	
		• Reverse Engineering	
		• Re-engineering	

Software Metric

A software metric is a quantitative measure of the degree a software system, component, or process possesses a given attribute. Because of the abstract nature of software and special problems with software schedule, cost, and quality, data collection and the derived metrics are an essential part of software engineering. This is evidenced by the repeated reference to measurement and metrics in the SWEBOK. Table 2 describes software metrics that are collected and used in different areas of software development. As in Table 1 the list is not meant to be complete, but to illustrate the type and range of measures used in practice.

Table 2: Software Metrics * (SEBoK Original)

Category	Metrics
Management Metrics	• Size: Lines of Code (LOC*), Thousand Lines of Code (KLOC) • Size: Function points, Feature Points • Individual Effort: Hours • Task Completion Time: Hours, Days, Weeks • Project Effort: Person-Hours • Project Duration: Months • Schedule: Earned Value • Risk Projection: Risk Description, Risk Likelihood, Risk Impact
Software Quality Metrics	• Defect Density - Defects/KLOC (e.g., for system test) • Defect Removal Rate – Defects Removed/Hour (for review and test) • Test Coverage • Failure Rate
Software Requirements Metrics	• Change requests (received, open, and closed) • Change request frequency • Effort required to implement a requirement change • Status of requirements traceability • User stories in the backlog
Software Design Metrics	• Cyclomatic Complexity • Weighted Methods per Class • Cohesion - Lack of Cohesion of Methods • Coupling - Coupling Between Object Classes • Inheritance - Depth of Inheritance Tree, Number of Children
Software Maintenance and Operation	• Mean Time Between Changes (MTBC) • Mean Time to Change (MTTC) • System Reliability • System Availability • Total Hours of Downtime

**Note: Even though the LOC metric is widely used, using it comes with some problems and concerns: different languages, styles, and standards can lead to different LOC counts for the same functionality; there are a variety of ways to define and count LOC— source LOC, logical LOC, with or without comment lines, etc.; and automatic code generation has reduced the effort required to produce LOC.*

References

Works Cited

- Bourque, P. and R. E. Fairley eds. *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. Los Alamitos, CA. IEEE Computer Society. 2014.
- Dori, D. "Conceptual modeling and system architecting." *Communications of the ACM* 46, no. 10 (2003) 62-65.
- IEEE Computer Society. *IEEE Standard for Computer Configuration Management in Systems and Software Engineering*. IEEE Std 828-2012. New York: IEEE, 2012.
- . *IEEE Standard for Software and System Test Documentation*. IEEE Std 829-2008. New York: IEEE, 2008.
- . *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998. New York: IEEE, 1998.
- . *IEEE Standard for Software Unit Testing*. IEEE Std 1008-1987. New York: IEEE, 1987.
- . *IEEE Standard for System and Software Verification and Validation*. IEEE Std 1012-2002. New York: IEEE, 2012.

- . *IEEE Recommended Practice for Software Design Descriptions*. IEEE Std 1016-2002. New York: IEEE, 2002.
- . *IEEE Standard for Software Project Plans*. IEEE Std 1058-1998. New York: IEEE, 1998.
- . *IEEE Standard for Software Maintenance*. IEEE Std 1219-1998. New York: IEEE, 1998.
- . *IEEE Standard for Risk Management*. IEEE Std 1540-2001. New York: IEEE, 2001.
- . *IEEE Standard for Systems and Software Engineering—Software Life Cycle Processes*. IEEE Std 12207-2008. New York: IEEE, 2008.
- . *IEEE Standard for Software Engineering—Software Life Cycle Processes—Maintenance*. IEEE Std 14764-2006. New York: IEEE, 2006.
- . *IEEE Standard for Systems and Software Engineering—Architecture Description*. IEEE Std 42010-2011. New York: IEEE, 2011.

Primary References

None.

Additional References

- Chidamber, S. R., C. F. Kemerer. "A Metrics Suite for Object Oriented Design", *IEEE Transactions on Software Engineering*. Vol. 20, No. 6 (1994).
- Kan, Stephen H. *Metrics and Models in Software Quality Engineering*, 2nd ed. Reading, Massachusetts. Addison-Wesley. 2003.
- Li, M. and Smidts, C. "A ranking of software engineering measures based on expert opinion." *IEEE Transactions on Software Engineering*. 2003.
- McConnell, Steve. *Code Complete*, 2nd ed. Microsoft Press. 2009.
- Moore, James. *Software Engineering Standards: A User's Road Map*. Hoboken, NJ. Wiley-IEEE Computer Society Press. 1997.
- Sommerville, I. *Software Engineering*. 9th ed. Boston, MA. Addison Wesley. 2010.

Knowledge Area: Systems Engineering and Aerospace Engineering

Systems Engineering and Aerospace Engineering

*Lead Author: NAME, Contributing Authors:*Name, Name

Intro Paragraph

References

Works Cited

None.

Primary References

None.

Additional References

None.

< Previous Article | Parent Article | Next Article >

Knowledge Area: Systems Engineering and Electrical Engineering

Systems Engineering and Electrical Engineering

*Lead Author: NAME, Contributing Authors:*Name, Name

Intro Paragraph

References

Works Cited

None.

Primary References

None.

Additional References

None.

< Previous Article | Parent Article | Next Article >

Knowledge Area: Systems Engineering and Mechanical Engineering

Systems Engineering and Mechanical Engineering

- Lead Authors:
- Leigh McCue and John Shortle
- Contributing Author:
- Art Pyster

This treatment opens with a brief overview of subdisciplines of mechanical engineering. The interested reader seeking a more thorough study on Mechanical Engineering (ME) is referred to Sagdeh and Worek (2017) and other references identified below. As stated by Hibbeler (2015), “Mechanics is a branch of the physical sciences that is concerned with the state of rest or motion of bodies that are subjected to the action of forces. In general, this subject can be subdivided into three branches: rigid-body mechanics, deformable-body mechanics, and fluid mechanics.” It is with those branches that this summary begins, before delving into thermodynamics, controls, and mechanical engineering design and the relationship between SE and ME.

Rigid-Body Mechanics and Deformable Body Mechanics

Mechanical engineering and civil engineering structural analysis bear much in common, as both fundamentally seek to create structures to satisfy a design need. While the bridge designer may prioritize material selections that hold up under constant use and varying weather conditions, the race car designer may prioritize light weight while maintaining crash worthiness. In the process of structural analysis, one applies Newton’s laws to analyze loads at joints and calculates internal forces, shear, and moment on structural elements. By understanding the loads a joint or structural element must withstand, one can select materials with appropriate properties to prevent failure. A structural element that changes its shape under load is known as a deformable body, whereas a body for which the distances between points on the body remain constant regardless of loading is known as a rigid body. The study of static structures, solid mechanics, and materials are typically foundational in a mechanical engineering undergraduate curriculum (e.g. (Hibbeler 2015), (Philpot and Thomas 2020), and (Callister and Rethwisch 2019)).

Dynamics

A subset of mechanics is dynamics. Eloquenty summarized by Greenwood (1965), “[t]he science of mechanics is concerned with the study of the interactions of material bodies. Dynamics is that branch of mechanics which consists of the study of the motions of interacting bodies and the description of these motions in terms of postulated laws.” In static rigid-body mechanics, with structures that are not accelerating, we write Newton’s second law as $\Sigma F=0$. In dynamics we utilize $\Sigma F=ma$ and leverage conservation of energy and momentum in analyzing systems. Simple dynamics problems typically involve the swinging of a pendulum or a spring-mass-damper system and evolve in complexity to modern suspension systems, orbital dynamics, and the multi-body challenges one might see when landing a rescue helicopter on the deck of a Coast Guard vessel.

Fluid Mechanics

Fluid mechanics includes experimental, computational, and analytical modeling of forces on a body moving in a liquid. Models of fluid dynamics often utilize the Navier-Stokes equations ^[1], a collection of partial differential equations arising from conservation of mass and momentum. For inviscid flows, the Navier-Stokes equations can be simplified to the Euler equations ^[2]. The Euler equations can be further simplified to Bernoulli's equation ^[3] in the special case of steady, incompressible, irrotational flow along a streamline. Common dimensionless parameters to characterize a flow include Reynolds number ^[4], Froude number, and Mach number. Reynolds number is a measure of the inertial forces to viscous forces, Froude number ^[5] characterizes the ratio of inertial forces to gravitational forces, and Mach number ^[6] provides a ratio of flow speed to the speed of sound. A comprehensive introductory text on fluid mechanics is Batchelor (2000).

Thermodynamics

Mechanical engineering thermodynamics involves products where heat is a primary consideration in the generation or transfer of energy – examples include engines, refrigeration systems, and nuclear reactors. At a broad level, the study of thermodynamics is captured by the following laws:

- *Zeroth law*: “[W]hen two bodies are in thermal equilibrium with a third body, they are in thermal equilibrium with one another.” (Moran and Shapiro 1998).
- *First law*: “[T]he value of the net work done by or on a closed system undergoing an adiabatic process between two given states depends solely on the end states and not on the details of the adiabatic process.” (Moran and Shapiro 1998). In short, energy can be neither created nor destroyed.
- *Second law*: “It is impossible for any system to operate in such a way that the sole result would be an energy transfer by heat from a cooler to a hotter body.” (Moran and Shapiro 1998). Notably, the first and second laws together are what render a perpetual motion machine physically impossible.
- *Third law*: “[T]he entropy of a pure crystalline substance is zero at the absolute zero of temperature...” (Moran and Shapiro 1998).

Relating thermodynamics to the mechanics topics described above, in a comprehensive summary paper, Graham Baker quotes Clifford Truesdell stating “As mechanics is the science of motions and forces, so thermodynamics is the science of forces and entropy.” (Baker 2005).

Controls

Controls refers to “the process of causing a system variable to conform to some desired value, called a reference value.” (Franklin et al. 1994) In application, this requires assimilating knowledge of the above disciplines to devise a system to achieve a desired outcome; e.g. measuring room temperature and using that as feedback to control an HVAC system, or measuring orientation and angular velocity to ideally time the firing of a spacecraft thruster to achieve a target orbit.

Design

In design, one combines knowledge of these specific fields to develop products of need by a customer or society. The Accreditation Board for Engineering and Technology (ABET) definition of design is agnostic to engineering discipline; specifically, they state “Engineering design is a process of devising a system, component, or process to meet desired needs and specifications within constraints. It is an iterative, creative, decision-making process in which the basic sciences, mathematics, and engineering sciences are applied to convert resources into solutions. Engineering design involves identifying opportunities, developing requirements, performing analysis and synthesis, generating multiple solutions, evaluating solutions against requirements, considering risks, and making trade-offs, for the purpose of obtaining a high-quality solution under the given circumstances.” (ABET n.d.) In the design

process, mechanical engineers use their knowledge of these disciplines to address societal needs. Furthermore, a key aspect of mechanical engineering design is the decision-making process. As such, decision theory is a key component to the study of design. (Tebay et al. 1984). Approaches to mechanical engineering design vary from waterfall (sequential) style to agile (parallel) methods, with waterfall methods more likely to be governed by a design spiral leading to a single point design whereas agile methods are iterative. Set-based design has become increasingly common in recent years, gaining benefit from increased flexibility by pursuing a broad design space, eliminating options over time as data and validation of underlying assumptions warrant. (Scaled Agile n.d.)

Application to Related Subdisciplines

These fundamental knowledge areas provide core skills for deeper knowledge in numerous subdisciplines, often taught as standalone multi-disciplinary fields. For example:

Aerospace Engineering

Aircraft and spacecraft require command of each of these core areas. For example fluid-dynamics permits understanding the forces of lift and draft on a body. Thermodynamics allows modeling thermal effects on engines, craft operating at high speeds, or subject to large thermal load fluctuations in space. Application of knowledge of dynamics and control allows engineers to develop safe, operable vehicles with performance traits tailored to design need. Expertise in mechanics and materials allows one to develop light weight structures. And ultimately, the design process enables the development and refinement of prototype and production vehicles. Aircraft provide numerous examples of the ties between mechanical engineering and systems engineering. For example, a Boeing 747 has approximately 6 million parts, provided by over 550 suppliers (Boeing 2013), relying on supply chain management. Furthermore, an airline fleet becomes a system of aircraft with the air traffic network constituting a system of systems.

Naval Engineering

Naval engineers function in many ways as systems engineers. Recognizing, for example, the wide range of engineering complexities involved in designing, building, and operating an aircraft carrier – from hull, mechanical, and electrical system design, to hydrodynamics and maneuvering, to integrated warfare systems, to aircraft operations, to hotel services for crew, the modern naval combatant is a highly complicated system of systems. A representative reference is provided in Lamb (2003).

Codes and Standards

The American Society of Mechanical Engineers (ASME) has been involved in the development of codes and standards for mechanical engineering systems since its first standard, “Code for the Conduct of Trials of Steam Boilers” released in 1884. (ASME n.d.a) As a testament to the complexity of mechanical engineering systems, in the present day, ASME standards are used in more than 100 countries and cover topics as diverse as elevators to nuclear plants with guidance under development for additive manufacturing and robotics, amongst other fields. (ASME n.d.)

Relationship between Mechanical and Systems Engineering

ME emphasizes design, development, and research on dynamic systems, be it turbines, prosthetics, or autonomous vehicles. Mechanical engineering products often form the building blocks for systems of systems; for example, a materials scientist designs the fan blade that goes into another mechanical engineer’s turbine engine design which resides on an aerospace engineer designed fighter jet, operating off an aircraft carrier designed by naval and systems engineers. One of the best examples of the relationship between ME and SE is with regard to human systems integration, in which multidisciplinary teams assess topics such as ergonomics, health, safety, user interface design,

and human performance in designing mechanical systems. Systems engineers often lead such assessments. A comprehensive reference on human systems integration is Booher (2003).

Relatively junior mechanical engineers often focus heavily on the individual subdisciplines of ME, such as those described earlier in this article. They might, for example, conduct a structural analysis of an individual physical assembly. As mechanical engineers gain experience and take on more senior roles, they often become concerned with the larger context in which mechanical components fit and hence take on many of the roles that systems engineers perform but with a focus on the mechanical subsystems. This becomes quite evident when looking at advertisements for senior mechanical engineers. For example, in a May 2022 listing of 20 senior mechanical engineer positions on one online job site, responsibilities included such systems engineering activities as reviewing mechanical documents for areas of conflict with all disciplines, resolving discrepancies between the employer and customer requirements, working with external partners and projects to ensure interoperability of internal and external components, driving internal development methodology, owning the full life cycle for parts, and managing the quality of deliverables while coordinating with other project disciplines.

References

Works Cited

- ABET. "Criteria for Accrediting Engineering Programs, 2020 – 2021," Accreditation Board for Engineering and Technology. Accessed May 9, 2022. <https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-engineering-programs-2020-2021/>.
- ASME. "About ASME Standards and Certification", Accessed May 9, 2022. <https://www.asme.org/codes-standards/about-standards>.
- ASME. "History of ASME Standards", American Society of Mechanical Engineers. Accessed May 9, 2022. <https://www.asme.org/codes-standards/about-standards/history-of-asme-standards>.
- Baker, G. "Thermodynamics in solid mechanics: a commentary," *Philosophical Transactions of the Royal Society A*. 2005. <https://doi.org/10.1098/rsta.2005.1669>.
- Batchelor, G. K. *An Introduction to Fluid Dynamics*, Cambridge University Press. 2000.
- Boeing. "Boeing Celebrates Delivery of 50th 747-8". 2013. <https://boeing.mediaroom.com/2013-05-29-Boeing-Celebrates-Delivery-of-50th-747-8>.
- Booher, H. R. ed. *Handbook of Human Systems Integration*, Hoboken:Wiley. 2003.
- Callister Jr., W. D. and D. G. Rethwisch. *Fundamentals of Materials Science and Engineering: An Integrated Approach*, 5th ed, Hoboken: Wiley. 2015.
- Franklin, G. F., J. D. Powell, and A. Emami-Naeini. *Feedback Control of Dynamic Systems*, 3rd ed. Addison Wesley. 1994.
- Greenwood, D. T. *Principles of Dynamics*, Prentice Hall. 1965.
- Hibbeler, R. C. *Engineering Mechanics: Statics*, 14th ed. Pearson. 2015.
- Lamb, T. ed. *Ship Design and Construction*, The Society of Naval Architects and Marine Engineers. 2003.
- Philpot, T. A. and J. S. Thomas. *Mechanics of Materials: An Integrated Learning System*, 5th ed. Hoboken: Wiley. 2020.
- Moran, M. J. and H. N. Shapiro. *Fundamentals of Engineering Thermodynamics*, 3rd ed. Hoboken: Wiley. 1998.
- Sadegh, A. and W. Worek. *Mark's Standard Handbook for Mechanical Engineers*, 12th ed. McGraw Hill. 2017.
- Scaled Agile. "Set-Based Design," Accessed on May 9, 2022. <https://www.scaledagileframework.com/set-based-design/>.

Tebay, R., J. Atherton, and S. H. Wearne. "Mechanical engineering design decisions: instances of practice compared with theory," Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture, Sage Journals. 1984.

Primary References

Batchelor, G. K. *An Introduction to Fluid Dynamics*, Cambridge University Press. 2000.

Booher, H.R. ed. *Handbook of Human Systems Integration*, Hoboken:Wiley. 2003.

Callister Jr., W. D. and D. G. Rethwisch. *Fundamentals of Materials Science and Engineering: An Integrated Approach*, 5th ed. Hoboken: Wiley. 2015.

Franklin, G. F., J. D. Powell, and A. Emami-Naeini. *Feedback Control of Dynamic Systems*, 3rd ed. Addison Wesley. 1994.

Greenwood, Donald T. *Principles of Dynamics*, Prentice Hall, 1965.

Hibbeler, R. C. *Engineering Mechanics: Statics*, 14th ed. Pearson. 2015.

Moran, M. J. and H. N. Shapiro. *Fundamentals of Engineering Thermodynamics*, 3rd ed. Hoboken: Wiley. 1998.

Philpot, T. A. and J. S. Thomas. *Mechanics of Materials: An Integrated Learning System*, 5th ed. Hoboken: Wiley. 2020.

Sadegh, A. and W. Worek. *Mark's Standard Handbook for Mechanical Engineers*, 12th ed. McGraw Hill. 2017.

Additional References

CFD. "CFD Online," Accessed on May 9, 2022. https://www.cfd-online.com/Wiki/Main_Page

References

- [1] https://en.wikipedia.org/wiki/Navier–Stokes_equations
- [2] [https://en.wikipedia.org/wiki/Euler_equations_\(fluid_dynamics\)](https://en.wikipedia.org/wiki/Euler_equations_(fluid_dynamics))
- [3] https://en.wikipedia.org/wiki/Bernoulli%27s_principle
- [4] https://en.wikipedia.org/wiki/Reynolds_number
- [5] https://en.wikipedia.org/wiki/Froude_number
- [6] https://en.wikipedia.org/wiki/Mach_number

Knowledge Area: Systems Engineering and Civil Engineering

Systems Engineering and Civil Engineering

*Lead Author: NAME, Contributing Authors:*Name, Name

Intro Paragraph

References

Works Cited

None.

Primary References

None.

Additional References

None.

< Previous Article | Parent Article | Next Article >

Knowledge Area: Systems Engineering and Economics

Systems Engineering and Economics

*Lead Author: NAME, Contributing Authors:*Name, Name

Intro Paragraph

References

Works Cited

None.

Primary References

None.

Additional References

None.

< Previous Article | Parent Article | Next Article >

Knowledge Area: Systems Engineering and Enterprise IT

Systems Engineering and Enterprise IT

- Lead Authors:
- Chuck Walrad and Rich Hilliard

SE is integral to the effective functioning of Enterprise Information Technology (EIT) organizations, which are devoted to the successful realization, use, and retirement of engineered systems that are largely based on information technology. Effective means to develop, manage, and support EIT organizations are described in the Enterprise Information Technology Body of Knowledge (EITBOK), which was jointly developed by the IEEE Computer Society and ACM, and is maintained as a public wiki by the IEEE (IEEE/ACM n.d.). This article largely explains EIT organizations through the lens of the EITBOK and how it relates to SE principles, practices, and activities as described elsewhere in the SEBoK.

Enterprise IT Introduction

Systems Engineering is integral to the effective functioning of EIT organizations, whose work is devoted to the successful realization, use, and retirement of engineered systems that are largely based on information technology. Examples of EIT systems include enterprise resource planning, supply chain management and customer relationship management systems, accounting, finance, inventory and order management, marketing, planning, purchasing, and sales. In the late 1980s and throughout the 1990s, EIT organizations became increasingly aware that the success rates of IT projects were very low, and that their enterprise customers were increasingly unhappy with their EIT investments. The Standish Group sought to understand what was going on. Their findings were presented in Standish (1994). The Standish Group research shows a staggering 31.1% of projects were canceled before they ever got completed. Further results indicated 52.7% of projects cost 189% of their original estimates. The cost of these failures and overruns were just the tip of the proverbial iceberg. The lost opportunity costs were not measurable but could easily be in the trillions of dollars.

As a result, commercial IT organizations through the auspicious parallel emergence of the Total Quality Management movement (ASQ n.d.) and the increasingly widespread development of application development methodologies by IT management consulting firms, recognized the need for the principles of SE and cross-functional teams (sometimes called Integrated Product Teams (IPTs)). It should be noted that there is considerable overlap today between SE and Project Management (PM) (see Systems Engineering and Project Management). Note that the following business management activities can be supported by Enterprise Systems Engineering (ESE) activities:

- mission and strategic planning
- business processes and information management
- performance management
- portfolio management
- resource allocation and budgeting
- program and project management

These same areas are often supported by Project Management Offices. However, since a key difference between PM and SE is that the latter includes technical expertise in the product/system of concern, focus should be on SE and IPTs.

With IPTs, each product is shepherded from concept through delivery and support to retirement by a cross-functional team that includes representatives from each area of responsibility, such as customer requirements development, design and development, test, configuration management, release engineering, delivery, and support.

As enterprises have come to depend more and more on technology to perform effectively, EIT organizations have matured, often due to the influence of IT consulting firms that make it their business to keep abreast of best practices.

EIT Concepts

Much of this section builds on concepts found in the EITBOK, the *Guide to the Enterprise Information Technology Body of Knowledge* (IEEE/ACM n.d.), and on general SE concepts elsewhere in the SEBoK.

An enterprise may be viewed from various perspectives: such as a system or system of systems (SoS):

- system-like enterprise: enterprises, like other systems, may have a tight coupling, single-executive function;
- system of systems-like enterprise: may be a loosely coupled group of organizations with blurred boundaries and (yet) with (some) shared outcomes.

The choice of perspective may vary with many factors: the enterprise itself, the mission or purpose, or the particular engineering task at hand. The enterprise can be viewed as a system with inputs and outputs in relation to the outside world, or as a system of systems interacting through its individual component inputs and outputs.

An enterprise usually involves one or more organizations but need not be identified with the organization. As the scope of human and organizational participants of the enterprise widen, there may be no single point of control. For example, an extended enterprise could include a company, its suppliers and its customers.

Whatever perspective is taken (system-like, SoS-like, other), there are common key ingredients of any enterprise: capabilities, individual competencies, and the organizational design. Capabilities can be further categorized into organizational, system, and operational. These capabilities determine what the enterprise is able to produce, which may be external offerings or internal mechanisms. Operational capabilities create services and products which in turn produce value as determined by stakeholders. The capabilities of an enterprise are realized through various enterprise elements including hardware, software, personnel, facilities, data, materials, techniques, and even services which include soft items such as processes, principles, policies, practices, organizations, doctrine, theories, and beliefs.

EIT capabilities exist through lifecycles spanning from creation through eventual replacement and disposal. System life cycle models aid in understanding, designing, planning and managing the capability of interest. As stated in the EITBOK, “A life cycle for a system generally consists of a series of stages regulated by a set of management decisions which confirm that the system is mature enough to leave one stage and enter another. Basic life cycle terms and concepts are discussed in Life Cycle Terms and Concepts.

A life cycle-based approach can be deployed across the several levels and used to manage evolution. These levels are:

- the enterprise itself
- the processes and organizations constituting the enterprise
- the systems enabling the capabilities, services or products provided.

It is important to distinguish between *product* and *project* lifecycle models. A product's life cycle extends from its initial concept approval, through development and distribution, to its retirement. This is the cycle that is managed within an Enterprise IT organization, since EIT must manage the hardware or software product from its introduction (in the case of acquired capabilities) or from its concept approval and development (in the case of capabilities it builds), through its deployment, maintenance and eventual retirement or withdrawal. Each of these stages is usually managed as a project or a series of projects. However, most discussions of life cycles within software and systems discussions focus on *project* life cycles. See, for example, the *Software Extension to PMBOK Guide* (PMI 2013).

System of Systems Context in EIT

The phrase “Enterprise IT” (EIT) is often used both to refer to the technology organization/business unit within the enterprise and to the enterprise’s information technology systems. This can be confusing. Here EIT refers to technology systems. When referring to the organization, “Enterprise IT organization” or EIT organization is used instead.

Enterprise IT organizations are responsible for complex socio-technical systems. Socio-technical systems in a business context (thus, EIT) contain technical elements (software, hardware, networks, storage, databases, etc.) but they also contain essential human components and necessary business processes. As covered in Introduction to Systems Engineering Fundamentals, they exist in an engineered System Context that is a set of system component interrelationships within a real-world environment:

This includes where problems come from and how they are defined, how candidate solutions are identified and selected, how to balance technology and human elements in the wider solution context, how to manage the complex organizational systems needed to develop new solutions, and how developed solutions are used, sustained and disposed of. SE should consider the full engineered system context so that the necessary understanding can be reached and the right systems engineering decisions can be made.

Socio-technical systems also have the following key characteristics: *Openness* in that they strongly interact with their environments, and *unfinished* in that they evolve over time. A complex of EIT systems is essential to the successful functioning of the people in the enterprise. They rely on EIT’s technology for communication, paying employees, people management, business measurement, and often for managing supply chains. Thus, “EIT needs to be viewed as a strategic partner to all the other functions and business units within an enterprise. This means that the results produced by EIT along with its effectiveness and efficiency are all critically important to the success of the enterprise.” (IEEE/ACM n.d. (a))

To accomplish this mission, the EIT organization builds and acquires new systems or system components, integrates them into the mega SoS, and maintains the integrity and interoperability of the individual components as well as the integrity of the entire complex of systems. In addition, security and performance levels must be achieved, benchmarked, and maintained or improved.

Major concerns of the EIT Organization

The EITBOK provides a good deal of information about major organizational concerns. Five of those concerns are summarized here:

- Enterprise Architecture
- Strategy and Governance
- Change
- Interoperability of Component Systems
- Security of Systems and Data

Enterprise Architecture

The EITBOK defines Enterprise Architecture (EA) as the practice of conducting enterprise analysis, design, planning, and implementation using a holistic approach for the successful development and execution of strategy. There are, in fact, many alternative definitions common in the community. See enterprise architecture for several of them, including the one from the EITBOK. EA applies architecture principles and practices to guide organizations through the business, information, process, and technology changes necessary to execute their organization’s strategies. EA builds upon many of the concepts and practices of Systems Architecture and also software architecture (Bourque & Fairley 2014).

This section will discuss EA from the perspective of key Principles of Systems Thinking, designated in this article with ***bold italics***. There are many commonalities in architecture whether one is architecting in systems, software or the enterprise. This article highlights both commonalities and differences between Enterprise Architecture and Systems Architecture. Architecture takes place at the juncture of design problems and their solutions. Design problems rarely arrive fully formulated or “hatched” for the architect; much of the architecture effort is to analyze or formulate the problems to be solved. Often this involves negotiation both within the enterprise and with outside stakeholders. Possible solutions are limited by resources and schedule and therefore often must also be negotiated.

A dominant principle of EA is ***separation of concerns*** (SoC). This is due to the complexity of enterprises, with many forces acting upon them; the many technologies and disciplines involved in their creation and day-to-day operations; and due to the diversity of stakeholders involved in the enterprise (see *EIT Roles* below).

An application of SoC is the ***boundary*** principle which is crucial to separate what is “inside” the enterprise from what is “outside”. While the former is often under the enterprise’s control, the latter is often beyond its control.

For some systems, determining the boundary is a key decision, subject to many possibilities and consequences. This is perhaps less the case for enterprises whose bounds are determined by the extent of organizations, regulations, supply chains or other criteria. Within an established boundary, the ***holism*** principle applies: to treat the enterprise as a unified whole, and, in particular, with respect to the ***interaction*** of the enterprise with its *environment*—including stakeholders, other enterprises and systems. From an EIT perspective, an enterprise is a system of systems. SoC offers a way to focus on the constituent systems in terms of their interconnections to one another to form the enterprise.

Another useful application of SoC is to apply the principle of ***views*** to depict the architecture of the enterprise in different ways to address the diverse interests (i.e., *concerns*) of an enterprise’s various stakeholders. Each *architecture view* presents a specific perspective on the enterprise. To be understandable to its audiences (those various stakeholders), each view should follow the conventions of its *architecture viewpoint* (ISO/IEC/IEEE 2011). Just as a map should have a legend, each view should have a documented viewpoint providing a prescribed set of conventions for looking at the enterprise through the resulting view. A viewpoint definition serves as a contract between the architect and stakeholders on *how* a selected set of concerns are to be addressed. Each viewpoint allows the architect to consider the enterprise in terms of various kinds of elements and relations among them and to present that view to those “concerned” stakeholders. Dominating concerns in enterprise architecture include interoperability, quality of systems and services, operations, managing change, and alignment with the enterprise’s mission, market, and technology. Via the principle of ***dualism***, it is recognized that some viewpoints are paired, such as with process/data duality. Another pervasive duality in the enterprise is of people/technology in terms of what capabilities are automated and which are performed by people.

In support of EA, there are numerous *enterprise architecture frameworks* each providing a pre-defined, integrated set of viewpoints to frame typical, recurring concerns (ISO/IEC/IEEE n.d.). The earliest recorded framework was Hammer et al. (1986), developed in the mid-1980s in response to growing availability of distributed computing to the enterprise (Rivera 2013). In the late 1980s, the US National Institute of Standards and Technology (NIST) created its Enterprise Architecture Model as a guide for US federal departments to organize their IT offerings (Fong and Goldfine 1989). NIST’s 5-layer model remains influential today. Despite never using the term “enterprise,” Zachman’s information systems architecture has had a major influence on how subsequent frameworks have been organized and depicted using grids or cubes. Later frameworks have evolved in response to emerging concepts, such as the move from client-server to service-oriented applications, as in the OASIS SOA Reference Model and its associated framework. The 1990s saw a proliferation of frameworks for defense enterprises including US DODAF, UK MODAF, NATO’s NAF and their successors such as OMG’s UAF.

Yet another application of ***separation of concerns*** is distinguishing today’s needs versus facilitating future needs; balancing ***stability/change*** and highlighting the temporal planning aspect of architecture of the enterprise. In concert with applying SoC, the architect uses the principle of ***abstraction*** to “forget” some details of the subject and thus

focus on essentials. Just as SoC is a way of taking things apart, **abstraction** is a way of unifying or putting like things together through generalization. These two principles are perhaps the most central to architecture. SoC and abstraction work together: SoC picks out subjects and abstraction helps architects to focus upon the essentials of each subject.

Returning to the **boundary** and **holism** principles, the enterprise architect applies these recursively. As architecting proceeds, the internal details of the enterprise become known in terms of **interaction** among its parts and other **relations**. Again, **multiple views** are used to factor specific **relations** into distinct views. Typical relations include Causality, Dependency, Client-Server, and Input-Output.

There are a variety of ways to structure the enterprise based upon stakeholder needs, mission or charter, technologies or capabilities already in place. Among these are structuring principles including:

- **Modularity** (separate and group capabilities into modules by commonalities)
- **Network** (modules are linked together)
- **Encapsulation** (separating architecture modules use from how they are implemented)
- **Layer Hierarchy** (separation through gradual refinement)
- **Regularity** (uniformity of interactions, of structure within layer, within network)
- **Similarity/Difference** (recognizing the limits of Regularity)

Strategy and Governance

EIT governance is the established process to define the strategy for the EIT organization and oversee its execution to achieve enterprise goals. Strategic planning translates the larger enterprise's goals into EIT goals and defines what EIT must do to achieve them. The EIT governance effort should communicate EIT goals, how they support the larger organization's goals, and drive change to achieve them while maintaining agreed upon levels of operation.

The systems thinking principle of **parsimony** applies here, too. For example, the simplest possible measurement and reporting systems are most likely to be used and not corrupted. Similarly, the EIT organizational structure should be straightforward and easy to traverse so that staff can communicate and problem-solve in teams. In governing the EIT organization to meet its dual goals of satisfying its customers and meeting its budget and schedule commitments, EIT management encounters the difficulty of managing in the face of **dualism**. The most obvious instance of handling **dualism** in EIT is the trade-off between budget, project prioritization, and features within projects.

Change

There are two levels of change management discussed in the EITBOK: organizational change management and changes to the IT organization's assets, whether they be software, hardware, or telecommunications systems. In the larger sense, the introduction of new systems and services often also requires changes in people's jobs and in the processes used to accomplish those jobs. Such efforts are intended to increase business value. Such changes require change initiatives because they involve guiding a mixture of stakeholders through the change process. This is one aspect of change management. Change also occurs when existing systems or services must be updated to meet new needs. Depending on the scope of the proposed changes, a change initiative may be required in this situation, too. At any rate, a change request process must be initiated and tracked to disposition (deferral, rejection, or implementation). Changes also come about as a result of defect reports. In such cases, the defect report must be tracked throughout the report's life cycle, to disposition (deferral, rejection, or implementation). These latter types of change management are discussed in detail in the EITBOK.

Interoperability of Component Systems

As explained in the EITBOK, *interoperability* means “the ability of systems (including organizations) to exchange and use exchanged information without knowledge of the characteristics or inner workings of the collaborating systems (or organizations).” This definition is based on an ISO definition: “degree to which two or more systems, products, or components can exchange information and use the information that has been exchanged.” (ISO/IEC 2011) Note that this definition includes hardware components as well as software systems and the organizations in which they are used.

Because technology changes so rapidly, and organizations themselves are frequently changing, achieving and maintaining interoperability is crucial. It is one of the main reasons that providing IT services is difficult and how it differs from engineering and delivering finished products, whether off-the-shelf products or bespoke systems.

Security of Systems and Data

According to former FBI Director Robert Mueller in 2012, “There are only two types of companies: those that have been hacked, and those that will be.” Two years later, his successor, James Comey said “There are two kinds of big companies in the United States. There are those who’ve been hacked by the Chinese and those who don’t know that they’ve been hacked by the Chinese.” Achieving and maintaining acceptable security is daunting but necessary.

While there are many definitions for security, most include the three dimensions of confidentiality, integrity, and availability. As such, the primary goal of EIT security is to preserve the confidentiality, integrity, and availability of information and information systems.

The principles behind an organization’s information security management system should be to design, implement, and maintain a coherent set of policies, processes, and systems that keep the risks associated with its information assets at a tolerable level, and yet, manage the cost and inconvenience of said risk management. As such, according to the EITBOK, EIT security practices should strive to enable the organization to:

- Always understand the current risk tolerance of the enterprise with respect to information and device security.
- Understand the security threats and potential damages to information, devices, and individuals.
- Create and follow policies and procedures that keep cyberattack risk and damages at or below a tolerable level.
- Effectively and efficiently detect and deal with cyberattack incidents.

Ensuring security in EIT has been made even more pressing and more difficult by the proliferation of mobile devices in enterprises and their ever-increasing use to conduct business. Increasingly, personal devices are used, accessing both corporate networks and public networks. The cost of mobile security incidents very easily can outweigh the cost of the mobile device itself.

Quality of Systems and Services

Typically, one of the primary responsibilities of the EIT organization is the timely provision of accurate and relevant information to the larger organization. While the EIT organization itself may not be responsible for creating the data, it is usually responsible for the data’s physical and logical validity and integrity central to constructing information. It is for this reason that defining and using appropriate processes is essential to ensure high-quality operation, including the development or acquisition, implementation and integration of new or changed capabilities.

The EITBOK details the quality concepts of:

- How to approach measuring quality
 - The importance of quality in requirements management
 - Understanding an organization’s quality capability
 - The differences between quality management systems, quality control, and quality assurance
 - Cost of quality in technical debt and the cost of rework
-

Disaster Preparedness

As explained in the EITBOK, disaster preparedness and disaster recovery support business-continuity planning and include planning for EIT resiliency, as well as recovery from adversity, so that critical business services affected are restored to a satisfactory working state within an acceptable timeframe after an event. The EITBoK devotes a chapter to how the EIT organization and its parent must plan in advance for how it will provide sufficient organizational resilience to recover from disasters and return to normal operation as rapidly as possible. It provides the following examples of disasters and service interruptions that could occur and should be considered in prevention and recovery planning:

- Natural disaster affecting datacenters or EIT service operations (flood, fire, earthquake, wind)
- Security breach resulting in a disaster (destruction of data, admin password changes, virus/malware installation, sabotage)
- Usage error (accidental deletion, unplug/turn off system resulting in corruption)
- Utility failure affecting datacenters (loss of power even after UPS)
- Vendor failure (cloud provider security failure, oil spill)
- Staffing issue (employment dispute/walkout, epidemic)

These are examples of unpreparedness:

- Requiring use of computers or printers when power is out
- Requiring use of Internet when power or connectivity is out
- Single point of knowledge/control for administration access
- Lack of offsite backup storage
- Lack of working restoration from backups
- Lack of failover datacenters in separate locations
- Undocumented or out-of-date documentation for system interfaces
- Requiring use of phones that are out of power
- Lack of designation of leaders in restoration efforts (who is in charge of restoring service and they know they are in charge)
- In general, no cohesive, comprehensive EIT service restoration plan

Operations and Support of Delivered Systems and Services

Operations engineering is responsible for the operation of the system. SE is responsible for bringing together the needed experts to insure proper evaluation of proposed changes/upgrades to the system capabilities. It is difficult in some organizational structures to understand how to access requisite information quickly, and what information to share and how to share it. Decisions often must be made quickly, such as when a system is down. Decision effectiveness depends on involving the right decision-makers with a sufficiently complete understanding of the decision context and of criticality of the decision need.

In Operations and Support of deployed technologies, SE is still essential as the overall system of systems goes through maintenance upgrades, new application additions, obsolescence driven re-designs, etc. In addition, during decommissioning and disposal, SE is essential to deal with the proper decoupling of the system and ensuring conformance with policy and laws affecting the system disposal.

Operations and support activities focus on maintaining a normal state of operations in EIT environments, where systems and processes execute according to expectations, with no surprises to users. Normal states can be disrupted either by the expected and planned transition of an asset, or by an unexpected event, be it as large as a disaster or by the discovery of a system defect. In the former case, if there is a disaster recovery plan, it is executed; in the latter, incident management kicks in when a system halts, or problem management is exercised when an operational system is not functioning as expected.

ISO/IEC/IEEE 20000, the International Standard for Service Management (ISO/IEC 2018) may be used by an organization to develop and manage EIT operations, along with complementary information for the operations and infrastructure side of IT, such as the Information Technology Infrastructure Library (ITIL) (Axelos n.d.).

Ethics

Both the IEEE and ACM require their members to adhere to codes of ethics. (IEEE 2020, ACM 2018) In addition, the IEEE Computer Society and the ACM worked together on the Software Engineering Code of Ethics. (ACM/IEEE 1997) Systems engineers in INCOSE have their own code as well. (INCOSE n.d.) All of these codes of ethics are somewhat different in style and length, but serve the same basic purpose.

Per the EITBOK, an EIT organization should create an easily accessible Code of Ethics:

- Ethical assertions should be documented and socialized in EIT in the form of a Code of Ethics, tied to the overall business code of ethics, and signed off by all staff so that violations can be cause for reprimand or dismissal.
- Vendors should be held to similar ethical standards.
- Lists of employees who have signed off on the Code of Ethics should be maintained through EIT governance practices.

The EITBOK has identified EIT ethical topics that include:

- Those actions and decisions that may be illegal
- Consideration of the social impact of the work at hand—if it will cause harm
- Alignment of business and EIT strategies and policies to an ethical standard
- Incorporation of the ideals of professionalism
- Adherence to applicable regulatory intent
- Protection for whistle blowers

Areas to specifically highlight:

- Activities that interfere with or corrupt the proper function of computers, applications and systems
- Activities that interfere with digital privacy or intellectual rights of others
- Respect for confidentiality, privacy, permissions, and access rights
- Inappropriate bias (skewing) of analysis and reporting
- Inaction in the face of likely ethics violations

EIT Roles

Major EIT systems have been disrupted and taken far too long to remedy due to the lack of information flow through the organization. Particularly for large systems, support activities are highly diverse in the roles people play, the technical knowledge required to build and install systems, to deal with downed systems, and in the organizational structure and culture in which people operate.

The roles range across the business functions requesting and supported by Enterprise technology, the evaluators, designers and implementers of those technologies, those who manage the successful transition of new systems into operation and manage and run the technology systems, to those who manage the quality of systems and services via defect reporting and tracking and who interface directly with end users.

Later sections will provide information about some major international models for defining EIT skills and capabilities. Accomplishing the EIT organization's mission requires people filling many roles ranging from enterprise architect, configuration manager, product designer, financial manager, and product manager. SE activities orchestrate the work of these roles so that they function together, rather than in silos or at cross purposes. Just as a systems engineer can play several roles in a given project, so might several people in an EIT project combine to fulfill the SE function. Larger EIT organizations may have many people performing SE functions and SE may be integrated into the EIT organization in many different ways depending on size, culture, organizational mission, and

any number of other factors. Part 5 of the SEBoK on Enabling Systems Engineering explores how systems engineering is woven into various organizations including EIT organizations.

Examples of Systems Engineering in EIT

SE provides a holistic discipline for integrating and managing the contributions of all roles needed to establish and provide EIT products and services. For her doctoral dissertation, Diane Aloisio from Purdue University performed a survey and causal analysis of project failures and of accidents. Aspects of the dissertation were published as a conference paper in Aloisio et al. (2018). The results were similar to those found in reviews of IBM consulting projects in the 1990s.

The following are examples of Systems Engineering interventions that prevented project failures in EIT systems. Almost all point to the lack of the System Engineering balancing act needed for product delivery.

- A widely used document processing product was undergoing a major upgrade. The new VP of Engineering wanted a project review before committing to a schedule. The review showed that the proposed schedule only considered the development effort and did not give sufficient weight to the voices of the testing and documentation organizations, while the needs of the tech support group had been completely overlooked. By establishing an integrated product management team that included technical expertise, as well as expertise in work breakdown and estimating, a realistic effort was defined and the resulting 9-month schedule was met within 3 weeks.
- A Silicon Valley start-up was depending entirely on an offshore development effort to produce its first product. All senior management was in the U.S. The VP of engineering made monthly visits to the development team. The CEO began to feel uneasy about the team's ability to deliver on schedule sensing that a lot of finger pointing within the development team itself signaled problems. There was no SE or IPT guiding the project. The VP of Engineering seemed unable to sort things out. A project review showed that the probable delivery date was at least 6 months beyond what the CEO was being told. The project was shut down.
- A major player in Human Resource Management had made commitments to deliver a new product. The project involved more than 100 people across the U.S. The delivery date kept slipping. Project roll-out would involve both installation of software and hardware at client sites, as well as installation and staff training at the large, centrally located Client Services organization. Executive Management needed to decide whether to reorganize this very expensive effort or cancel it. Again, the successful solution depended on establishing an effective cross-functional team (IPT) that necessarily included representatives from the technical teams. The IPT was supported by a newly established Project Management Office (PMO). The PMO integrated the geographically disparate efforts by establishing communication channels, as well as common estimating and scheduling practices. The PMO also instituted monthly critical metrics reports to show the IPT bottlenecks and inefficiencies across the project, from development efforts to Client Services set-up to Payroll technical support.

Systems Engineering provides a holistic discipline for integrating and managing the contributions of all of the roles needed to provide EIT products and services. It ensures that a single perspective does not dominate outcomes.

Key External Models of EIT Organizational Capability and Maturity

Because EIT is critical to businesses globally, considerable effort has gone into establishing universal frameworks for EIT skills, along with definitions of levels of capability of each skill. In addition, at least 2 models have been developed for evaluating the organizational performance of EIT. The effectiveness of systems engineering is key to the effectiveness of the EIT organization.

IT-CMF: IT-Capability Maturity Framework

The IT-CMF had its origin in a quality improvement effort at Intel Labs Europe in Ireland. Its intent, as described at the CIO Index website was to quantify and demonstrate the true value impact of IT. (CIO Index n.d.) IT-CMF is used to measure, develop, and monitor an organization's EIT capability maturity. It consists of 35 EIT management capabilities organized into four macro capabilities: managing EIT like a business; managing the EIT budget; managing the EIT capability; and managing EIT for business value. Its 35 critical capabilities are described in the EITBOK as "A defined EIT management domain that helps mobilize and deploy EIT-based resources to effect a desired end, often in combination with other resources and capabilities".

CMMI: Capability Maturity Model Integration (US)

The CMMI had its origin at the Software Engineering Institute in 1987 in a model requested by the U.S. Department of Defense for determining the capability maturity of contractor software development organizations. This became the Capability Maturity Model (CMM). Since then, it has been expanded into a set of Capability Maturity Models, all based on five levels of demonstrably increasing maturity as measured by organization performance. That set was then integrated and became the CMMI. The CMMI set addresses development, services, supplier management, people management, and data management. (CMMI Institute. n.d.)

SFIA: Skills Framework for the information Age (Europe, Canada and US)

The SFIA Framework has been downloaded and used by organizations and individuals in nearly 180 countries and is overseen by the SFIA Foundation (SFIA n.d.) It identifies 97 professional skills across EIT and supporting areas with seven levels of responsibility, which provide generic levels of responsibility, taking into account reflect experience and competency. These describe the behaviors, values, knowledge, and characteristics that an individual should have in order to be considered competent at a particular level.

E-CF: European Competency Framework (EU)

The ECF was the first sector-specific implementation of the European Qualifications Framework. The European Norm (EN) 16234-1 European e-Competence Framework (e-CF) provides a reference of 41 competences as applied at the Information and Communication Technology workplace, using a common European reference for competences, skills, knowledge and proficiency levels. The e-CF Explorer web tool is hosted by IT Professionalism Europe (ITPE n.d.).

I Competency Dictionary: EIT skills as Defined by METI/ IPA (Japan)

The Japanese government's Information-Technology Promotion Agency (IPA), established by the Ministry of Economy, Trade and Industry (METI) developed the "I Competency Dictionary" (iCD) to support the IT Engineers Examination. It underlies the METI/IPA Skills Standards for IT Professionals, abbreviated as ITSS, established in 2002. ITSS organizes skills within the information services industry into 11 job categories and 35 specialty fields. In each field, there are seven levels of competence based on individual experience and ability to perform. One appealing feature of ITSS is that this standard allows engineers to draw roadmaps for their own futures and career advancement. Approximately 90% of large enterprises and over 60% of SMEs in Japan have introduced or are

considering using ITSS. Information in English is available at <https://www.ipa.go.jp/en/index.html>.

International Standards Guiding EIT

The following standards guide the EIT community:

- ANSI/AIAA. 2012. *ANSI/AIAA Guide to the Preparation of Operational Concept Documents*. ANSI/AIAA G-043A-2012e
- IEEE 2012 Std 828™-2012, *IEEE Standard for Configuration Management in Systems and Software Engineering*
- ISO 10004:2012, Quality management—Customer satisfaction—Guidelines for monitoring and measuring
- ISO 10007:2003, Quality management systems—Guidelines for configuration management
- ISO 18238:2015, Space systems—Closed loop problem solving management
- ISO/IEC 25010:2011 Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models
- ISO/IEC 16350:2015, Information technology—Systems and software engineering—Application management
- ISO/IEC 19770-1:2012, Information technology—Software asset management—Part 1: Processes and tiered assessment of conformance
- ISO/IEC 20000-1:2011, (IEEE Std 20000-1:2013) Information technology—Service management—Part 1: Service management system requirements
- ISO/IEC 20000-2:2012, Information technology—Service management—Part 2: Guidance on the application of service management systems
- ISO/IEC TR 20000-10:2015, Information technology—Service management—Part 10: Concepts and terminology
- ISO/IEC TR 20000-11:2015, Information technology—Service management—Part 11: Guidance on the relationship between ISO/IEC 20000-1:2011 and related frameworks: ITIL®
- ISO/IEC TR 20000-12—Information technology—IT Service management—Part 12: Guidance on the relationship between ISO/IEC 20000-1:2011 and service management frameworks: CMMI-SVC®
- ISO/IEC/IEEE 14764-2006, Software Engineering—Software Lifecycle Processes—Maintenance
- ISO/IEC 15939:2007, Systems and software engineering—Measurement process
- ISO/IEC/IEEE 42010:2011, Systems and software engineering — Architecture description

References

Works Cited

- Association for Computing Machinery (ACM). *ACM Code of Ethics and Professional Conduct*. 2018. <https://www.acm.org/code-of-ethics>.
- ACM/IEEE. ACM/IEEE Computer Society. *Software Engineering Code*. 1997. <https://ethics.acm.org/code-of-ethics/software-engineering-code/>.
- Alioso, D. C., Marais, K., Sun, Hanxi. "2018 Best PIC II Paper: Systems Engineering Division: Development of a Survey Instrument to Evaluate Student Systems Engineering Ability", in 126th ASEE Annual Conference and Exposition. Tampa, Florida. 2019.
- American Society for Quality (ASQ). "What is Total Quality Management". American Society for Quality. Accessed May 7, 2023. <https://asq.org/quality-resources/total-quality-management>.
- Axelos. "Axelos products and services for professionals". Accessed May 7, 2023. <https://www.axelos.com/for-professionals>.
- Bourke, P. and Fairley, R. E. eds. *Guide to the Software Engineering Body of Knowledge*, Version 3.0, IEEE Computer Society. 2014.

- CIO Index. "IT Capability Maturity Framework". CIO Index. Accessed May 7, 2023. [https://cio-wiki.org/wiki/IT_Capability_Maturity_Framework_\(IT-CMF\)](https://cio-wiki.org/wiki/IT_Capability_Maturity_Framework_(IT-CMF)).
- CMMI Institute. "CMMI". CMMI Institute. Accessed May 7, 2023. <https://cmmiinstitute.com/cmmi>.
- Fong, E. N. and A.H. Goldfine. *Information Management Directions: The Integration Challenge*. National Institute of Standards and Technology (NIST) Special Publication 500-167, September 1989. <https://csrc.nist.rip/library/NIST%20SP%20500-167.pdf>.
- Hammer, M., T. H. Davenport and J. Champy. *Dispersion and Interconnection: Approaches to Distributed Systems Architecture*, Cambridge, MA: CSC Index Inc. and Hammer and Company, Inc. 1986. <https://www.globalaea.org/general/custom.asp?page=Resources>.
- Hilliard, R. "Lessons from the fundamental unity of architecting". *Software Engineering in the Systems Context*. Edited by Ivar Jacobson and Harold 'Bud' Lawson. 2015.
- Hofmeister, C. et al. "A general model of software architecture design derived from five industrial approaches". *The Journal of Systems and Software* 80, no. 1 (2007).
- IEEE. *IEEE Code of Ethics*. 2020. <https://www.ieee.org/about/corporate/governance/p7-8.html>.
- IEEE/ACM. *Guide to the Enterprise IT Body of Knowledge*. IEEE and ACM. Accessed May 7, 2023. <https://eitbokwiki.org>.
- INCOSE. NCOSE Code of Ethics. Accessed May 7, 2023. <https://www.incose.org/about-incose/Leadership-Organization/code-of-ethics>.
- ISO/IEC. *ISO 20000: The International Standard for Service Management*. 2018. <https://www.iso.org/obp/ui/#iso:std:iso-iec:20000:-1:ed-3:v1:en>.
- ISO/IEC. *ISO/IEC 25010:2011 Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models*. 2011. <https://www.iso.org/standard/35733.html>.
- ISO/IEC/IEEE. *Survey of Architecture Frameworks*. ISO/IEC/IEEE 42010 Users Group. Accessed May 7, 2023. <https://www.iso-architecture.org/42010/afs/>.
- ISO/IEC/IEEE. *ISO/IEC/IEEE 42010:2011, Systems and software engineering — Architecture description*. 2011. <https://www.iso.org/standard/50508.html>.
- ITPE. "e-CF Explorer". IT Professional Europe. Accessed May 7, 2023. <https://ecfexplorer.itprofessionalism.org/>.
- Project Management Institute (PMI). *Software Extension to PMBOK® Guide – Fifth Edition*. Newtown Square, PA. PMI. 2013. <https://www.pmi.org/pmbok-guide-standards/foundational/pmbok/software-extension-5th-edition>.
- Rivera, R. "The PRISM Architecture Framework – Was it the Very First Enterprise Architecture Framework?" *Journal of Enterprise Architecture*, November 2013.
- SFIA. "SFIA: The Global Skills and Competency Framework for the Digital World". Accessed May 7, 2023. <https://sfia-online.org/en>.
- Standish Group. *The Chaos Report*. 1994. https://www.standishgroup.com/sample_research_files/chaos_report_1994.pdf.
- Zachman, J. "A Framework for Information Systems Architecture". *IBM Systems Journal*, 454-470, 1987. https://www.zachman.com/images/ZI_PICs/ibmsj2603e.pdf.

Primary Reference

IEEE Computer Society. *Enterprise IT Body of Knowledge*. Accessed May 7, 2023. <https://eitbokwiki.org>.

Additional References

Aloisio, D. C. *Lessons from Systems Engineering Failures: Determining Why Systems Fail, The State of Systems Engineering Education, and Building an Evidence-Based Network to Help Systems Engineers Identify and Fix Problems on Complex Projects*. PhD Dissertation, Purdue University. 2018. [https://hammer.purdue.edu/articles/thesis/](https://hammer.purdue.edu/articles/thesis/Lessons_from_Systems_Engineering_Failures_Determining_Why_Systems_Fail_the_State_of_Systems_Engineering_Education_and_Building_an_Evidence-Based_Network_to_Help_Systems_Engineers_Identify_and_Fix_Problems_on_Complex_Projects)

[Lessons_from_Systems_Engineering_Failures_Determining_Why_Systems_Fail_the_State_of_Systems_Engineering_Education_and_Building_an_Evidence-Based_Network_to_Help_Systems_Engineers_Identify_and_Fix_Problems_on_Complex_Projects](https://hammer.purdue.edu/articles/thesis/Lessons_from_Systems_Engineering_Failures_Determining_Why_Systems_Fail_the_State_of_Systems_Engineering_Education_and_Building_an_Evidence-Based_Network_to_Help_Systems_Engineers_Identify_and_Fix_Problems_on_Complex_Projects)
7488569.

Curley, M., Kenneally, J., Carry, M. eds. *IT Capability Maturity Framework (IT-CMF): The Body of Knowledge Guide*. Van Haren Publishing. 2015.

Knowledge Area: Systems Engineering and Quality Attributes

Systems Engineering and Quality Attributes

Contents of this Knowledge Area

- A Framework for Viewing Quality Attributes from the Lens of Loss (John S. Brtis)
- Human Systems Integration (Ben Schwartz and Holly Handley) (Guy Boy, Alice Squires, Paul Phister, Stuart Booth, and Dick Fairley)
- Manufacturability and Producibility (Dick Fairley and Kevin Forsberg) (Paul Phister, Alice Squires, and Richard Turner)
- System Adaptability (Haifeng Zhu) (Eileen Patrice Arnold)
- System Affordability (Paul Phister) (Ray Madachy)
- System Hardware Assurance (Michael Bear, Donald Davidson, Shawn Fetterolf, Darin Leonhardt, Daniel Radack, Karen Johnson, and Elizabeth A. McDaniel) (Michael Berry, Brian Cohen, Diganta Das, Houman Homayoun, and Thomas McDermott)
- System Reliability, Availability, and Maintainability (Paul Phister and David Olwell)
- System Resilience (John Brtis) (Ken Cureton, Scott Jackson, and Ivan Taylor)
- Resilience Modeling (Ken Cureton) (John Brtis, Scott Jackson, Tim Ferris, and Ivan Taylor)
- System Resistance to Electromagnetic Interference (Paul Phister) (Scott Jackson, Richard Turner, John Snoderly, and Alice Squires)
- System Safety (Dick Fairley) (Art Pyster and Alice Squires)
- System Security (Mark Winstead) (Terri Chan and Keith Willett)
- Lead Authors:
- Art Pyster and Dave Olwell
- Contributing Authors:
- Richard Turner, Dick Fairley, Scott Jackson, and Alice Squires

Every system has a set of properties that are largely a function of the system as a whole rather than just its constituent parts. There are dozens of these properties such as security, reliability, safety, resistance to electromagnetic interference, usability, and resilience. This Knowledge Area (KA) describes many of the most important such properties and how they are realized. The vocabulary to talk about these properties is not standard. Alternatively, they are called *quality attributes*, *non-functional requirements*, *-ilities*, and *specialties*. In this KA, the term specialty engineering refers to the collective engineering of all quality attributes of a system.

Quality attributes are often interdependent rather than orthogonal; e.g. making a system more secure may make it harder to use but also safer. Increasing system reliability often requires using more expensive parts and adding redundant components. Hence, higher reliability often means higher cost to deliver a system – another system property. However, higher reliability might mean lower maintenance cost – another system property. A systems engineer typically makes many decisions and takes many actions with regard to system properties; e.g. specifying which properties are important for a particular system, stating how those properties will be measured, trading off conflicting properties, and verifying that a system has the specified properties. Barry Boehm (2016), as the lead for a project of the Systems Engineering Research Center ^[1], introduced an ontology to describe system quality attributes as a way of enabling smarter trade-offs between them and understanding the impact of those trades on cost. Also, see

the article on Quality Management for more information on how these decisions and actions are managed. de Weck, et al. (2012) has a lengthy and insightful chapter on quality attributes (which they call *-ilities*).

Many consider specialty engineering to be a sub-discipline of SE itself; e.g. they consider security engineering, safety engineering, resilience engineering, etc. to be part of SE. Others consider them to be stand-alone disciplines in their own right with close ties to SE. Either way, their mastery is important to a system engineer. This KA includes topical articles on several specialty engineering disciplines. Some articles will treat the topic of the article as a sub-discipline of SE itself; others will treat it as a closely related but separate discipline. This disparity reflects both the diversity of opinions on this matter and the fact that SEBoK articles are written by a wide array of authors.

Specialty Requirements

The systems engineering team must ensure that requirements about quality attributes (also called specialty requirements) are properly reviewed with regard to their impact on life cycle costs, development schedule, technical performance, and operational utility. For example, security requirements can impact operator workstations, electromagnetic interference requirements can impact the signal in the interfaces between subsystems, and mass-volume requirements may preclude the use of certain materials to reduce subsystem weight.

Engineering specialists audit the evolving design and resulting configuration items to ensure that the overall system performance also satisfies the specialty requirements. Including appropriate specialty engineers within each systems engineering team ensures that all specialty requirements are identified and balanced throughout the development cycle.

Integration of Specialty Engineering

Integration of specialty engineering into a project or program is, or should be, a major objective of systems engineering management. (Endler 2016, Kossiakoff et al. 2020) With properly implemented procedures, the rigor of the systems engineering process ensures participation of the specialty disciplines at key points in the technical decision-making process. Special emphasis on integration is mandatory because a given design could in fact be accomplished without consideration of these specialty disciplines, leading to the possibility of system ineffectiveness or failure when an unexamined situation occurs in the operational environment.

For example, human factors considerations can contribute to reduced workloads and therefore lower error rates by operators in aircraft cockpits, at air-traffic consoles, or in nuclear reactor stations. Similarly, mean-time-to-repair features can significantly increase overall system availability in challenging physical environments, such as mid-ocean or outer space or in safety critical environments such as hospital intensive care units or in traffic control systems in city centers. Specialty engineering requirements are often manifest as constraints on the overall system design space. The role of systems engineering is to balance these constraints with other functionality in order to harmonize total system performance. The end goal is to produce a system that provides utility and effectiveness to the customer at an affordable price.

As depicted in Figure 1, systems engineering plays a leadership role in integrating traditional disciplines, specialty disciplines, and unique system product demands to define the system design. Relationships for this integration process are represented as interactions among three filters.

The first filter is a conceptual analysis that leverages traditional design consideration (structural, electronics, aerodynamics, mechanical, thermodynamics, etc.). The second filter evaluates the conceptual approach using specialty disciplines, such as safety, affordability, quality assurance, human factors, reliability and maintainability, producibility, packaging, test, logistics, and others, to further requirements development. Design alternatives that pass through these two processes go through a third filter that incorporates facility design, equipment design, procedural data, computer programs, and personnel to develop the final requirements for design selection and further detailed development.

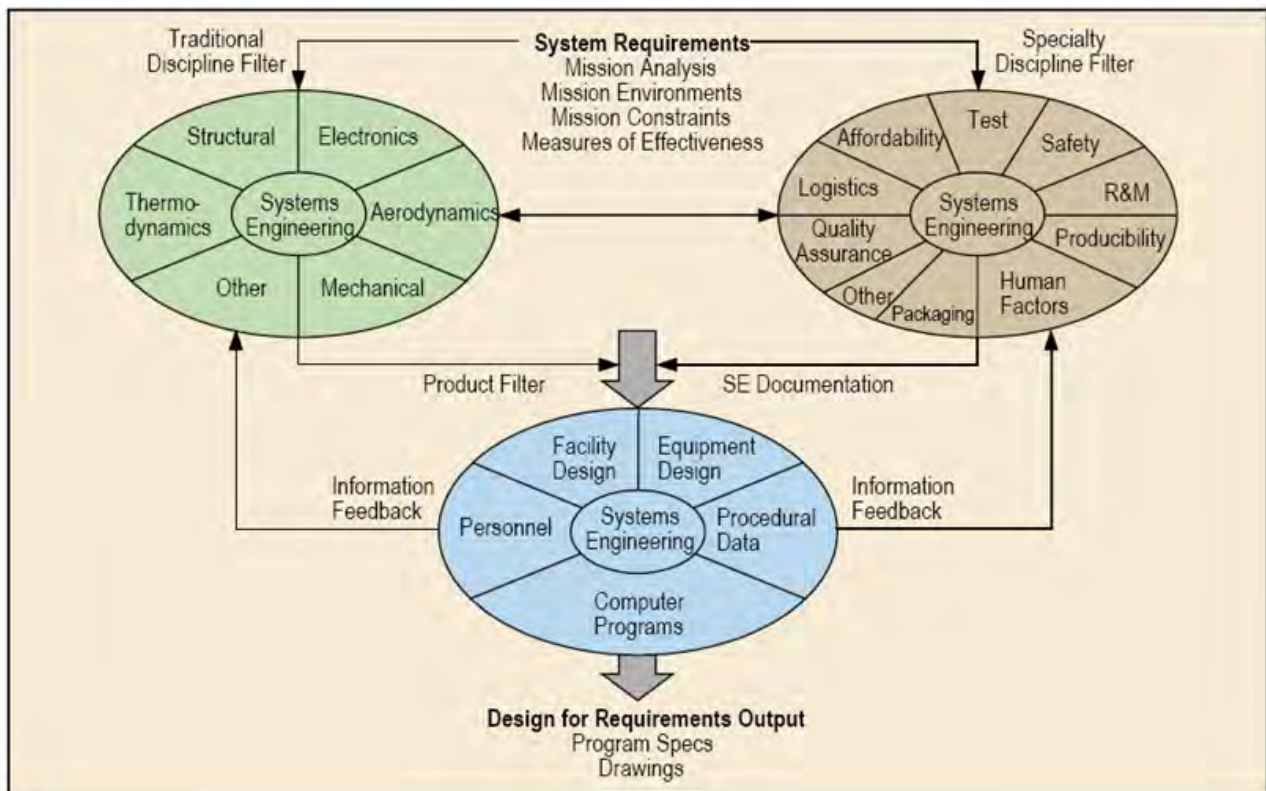


Figure 1. Integration Process for Specialty Engineering (USAF 2000). Released by the U.S. Air Force.

References

Works Cited

- Boehm, B. *System Qualities Ontology, Tradespace, and Affordability (SQOTA) Project - Phase 4*. Systems Engineering Research Center: Stevens Institute of Technology. February 2016. SERC-2016-TR-101. https://www.archive.sercuarc.org/wp-content/uploads/2014/05/SERC-2016-TR-101-Phase-4_RT-137.pdf.
- de Weck, R. D. and C. Magee. *Engineering Systems: Meeting Human Needs in a Complex Technological World*. MIT Press. Chapter 4: "Life-Cycle Properties of Engineering Systems: The Ilities". 2012. <https://www.amazon.com/Engineering-Systems-Meeting-Complex-Technological/dp/0262529947>. Chapter 4 on the -ilities is also available at https://strategic.mit.edu/docs/es_book_004_proof.pdf. Accessed May 21, 2023.
- Endler, D. "Integration of Specialty Engineering Activities." Proceedings of the 26th Annual International Council on Systems Engineering (INCOSE) International Symposium, July 18-21, 2016, Edinburgh, Scotland, UK.
- Kossiakoff, A., W. N. Sweet, S.J. Seymour, and S. M. Biemerl. *Systems Engineering Principles and Practice*, 3rd ed. John Wiley & Sons, Inc: Hoboken, NJ. 2020.
- USAF. *Guidelines for Successful Acquisition and Management of Software-Intensive Systems: Weapon Systems Command and Control Systems Management Information Systems*, version 3.0. Hill AFB: Department of the Air Force Software Technology Support Center. May 2000. <https://onlinelibrary.wiley.com/doi/abs/10.1002/j.2334-5837.1999.tb00315.x>.

Primary References

USAF. *Guidelines for Successful Acquisition and Management of Software-Intensive Systems: Weapon Systems Command and Control Systems Management Information Systems*, version 3.0. Hill AFB: Department of the Air Force Software Technology Support Center. May 2000. <https://onlinelibrary.wiley.com/doi/abs/10.1002/j.2334-5837.1999.tb00315.x>.

Additional References

None.

References

[1] <https://sercuarc.org>

A Framework for Viewing Quality Attributes from the Lens of Loss

- Lead Author:
- John S. Brtis

Important but often under-considered areas of systems engineering address potential losses associated with the development and use of systems. These areas fall under the umbrella category of loss-driven systems engineering (LDSE), i.e., the value-adding unification of the systems engineering specialty areas that address the potential losses associated with systems. LDSE can be defined as an overarching systems engineering process which holistically addresses the quality characteristics concerned with loss (such as system resilience, system safety, and system security).

Overview

Systems engineering methodologies often focus on the delivery of desired capability. Methodology sources such as the Systems Engineering Handbook (Walden 2015) and ISO/IEC/IEEE 15288 (ISO) provide full lifecycle and fully integrated methodologies that focus on the generation and deployment of a system to deliver capabilities. Those methodologies are largely capability-driven and do not provide detailed fully integrated attention to potential loss. Loss and loss-driven specialty areas are largely treated in isolation. Examples of loss-driven specialty areas include resilience, safety, security, operational risk, environmental protection, quality, and availability. There is commonality and synergy among these specialty areas, which should be addressed by systems engineering. Potential synergies include:

- Shared loss scenarios
- Shared requirements
- Shared modeling and analysis techniques
- Shared architecture and design solutions
- Shared risk management

The expected benefits of applying a unified loss-driven viewpoint in the systems engineering processes include:

- Reducing engineering effort by eliminating redundant efforts among the specialty areas
 - Helping to ensure a comprehensive consideration of loss
 - Ensuring cohesion and elimination of conflicts among the loss-driven solutions
 - Identifying highly effective solutions that address the interests of multiple loss-driven specialty areas
-

- Providing a holistic viewpoint addressing the multiple perspectives
- Reducing the load of data generated by multiple specialty areas to a minimal, non-redundant set
- Mutual learning among the loss-driven specialty areas

Background and Origins

Engineers at the MITRE Corporation explored the commonality of “protecting against loss” in the areas of security, safety, and resilience as part of an effort to improve a sponsor’s systems engineering methodologies (Brtis 2020). In parallel, these engineers raised and explored the issue with the INCOSE resilience and security working groups. This led to the realization that these specialty areas had commonalities and synergies with many of the systems engineering specialty areas. The term “loss-driven systems engineering” was coined to identify this area of common interest. These concepts were discussed with the INCOSE Technical Operations Director at the 17th INCOSE International Symposium, and he recommended that the concept be pursued as an INCOSE initiative. An exploratory meeting on LDSE was held at the 18th INCOSE International Symposium. At that meeting participants agreed that this concept should be pursued and decided that as a first step, a special theme issue on loss-driven systems engineering of INCOSE *Insight* magazine should be pursued, to be followed by a section in the SEBoK. That issue of *Insight* was published in December 2021.

The Basis of Commonality

Systems engineering must address all loss-driven specialty areas in a mutually supportive and optimized manner. The exact definitions and demarcation between the different specialty areas are moot. What matters is meeting all the objectives of the loss-driven areas. Loss-driven areas often have common objectives, common concepts and principles, common requirements, common architectural solutions, common design solutions, common analyses, and common methodologies. Engineers responsible for loss-driven areas can often do a better job if they work collaboratively. This is because they are all interested in the same overarching concern: addressing potential losses associated with the system of interest. An inspection of the Systems Engineering Handbook (Walden 2015) identified specialty engineering areas that shared the concerns of loss-driven systems engineering. Those identified include:

- availability
 - environmental impact
 - maintainability
 - resilience engineering
 - reliability
 - risk management
 - system safety engineering
 - system security engineering
 - quality management
-

Attributes Shared by the Loss-Driven Specialty Areas, a Potential for Unification

The loss-driven specialty areas share attributes that specify the scope of each specialty area. All the loss-driven specialty areas have as attributes:

- the types of assets considered
- the types of loss addressed
- the types of adversity addressed
- the coping strategies considered
- the aspects of the system and its environment under consideration

These attributes define the scope of each specialty area. These scopes differ among the loss-driven specialty areas, but in many cases, they overlap. These loss-driven attributes and the possibility of aggregating their overall values provide a basis for integrating the loss-driven specialty areas, by aggregating the range of values of the parameters and then by addressing their aggregate scopes. Brtis (2020) considers each of these attributes. Figure 1 provides an aggregate summary of the scope of considerations identified for loss-driven attributes. Properly engineering a system requires consideration of the full range of each of the loss-driven attributes.



Figure 1: Attributes and Scope of the Integrated Loss-Driven Systems Engineering Problem Space

Core Principles for LDSE

Winstead (2020) investigated the commonality among principles being articulated for individual loss-driven specialty areas. He identified fundamental principles that can be unified across the specialties.

- Candidate principle 1: Systems engineering minimizes hazards.
- Candidate principle 2: Systems engineering seeks to control hazards that cannot be avoided, including assuring transitions from one known acceptable mode or state to another known and acceptable one.
- Candidate principle 3: Systems engineering uses proven and accepted processes, solutions, methods, materials, etc. when the process, etc., achieves the intended trustworthiness.
- Candidate principle 4: The human within the system should be enabled to prevent, minimize, and recover from loss when possible.
- Candidate principle 5: Systems engineering should strive for the simplest solutions.
- Candidate principle 6: Systems engineering produces evolvable systems likely to maintain or improve on loss-driven properties through change.
- Candidate principle 7: Actions should trace to the entity responsible.
- Candidate principle 8: Any critical task should be possible to perform in more than one way.

Winstead (2020) also considered the systems engineering principles of Watson (2019) and discusses minimal changes needed to make them well-suited for application to loss-driven systems engineering.

LDSE in the SE Activities

Brtis (2020) found that several the SE practices identified in ISO 15288 and the INCOSE SE Handbook need to be augmented to adequately address the needs of LDSE and recommended specific additions for the following practice areas:

- Business or Mission Analysis Process
- Stakeholder Needs and Requirements Definition Process
- System Requirements Definition Process
- Architecture Definition Process
- Design Definition Process
- Risk Management Process

LDSE Design Techniques

Winstead (2021) investigates what he calls “loss control” for cyber-physical systems. He investigates loss control independent of any particular systems engineering specialty area. He recommends a set of loss control design principles:

- **Anomaly Detection** - Any salient anomaly in the system or in its environment is detected in a timely manner that enables effective response action.
- **Commensurate Protection** - The strength and type of protection provided to an element must be commensurate with the most significant adverse effect that results from a failure of that element.
- **Commensurate Response** - The design should match the aggressiveness of an engineered response action’s effect to the needed immediacy to control the effects of each loss scenario.
- **Continuous Protection** - The protection provided for an element must be effective and uninterrupted during the time that the protection is required.
- **Defense-in-depth** - Loss is prevented or minimized by employing multiple coordinated techniques and strategies.
- **Distributed Privilege** - Multiple authorized entities must act in a coordinated manner before an operation on the system is allowed to occur.

- **Diversity (Dynamicity)** - The design delivers the required capability through structural, behavioral, data, or control flow variation.
- **Domain Separation** - Domains with distinctly different protection needs should be physically or logically separated.
- **Least Functionality** - Each element should have the capability to accomplish its required functions, but no more.
- **Least Persistence** - System elements and other resources should be available, accessible, and able to fulfill their design intent only for the time they are needed.
- **Least Privilege** - Each element should be allocated privileges that are necessary to accomplish its specified functions, but no more.
- **Least Sharing** - System resources should be shared among system elements only when necessary, and among as few system elements as possible.
- **Loss Margins** - The system is designed to operate in a state space sufficiently distanced below the threshold at which loss occurs.
- **Mediated Access** - All access to and operations on system elements are mediated.
- **Protective Defaults** - The default configuration of the system provides maximum protection effectiveness.
- **Protective Failure** - A failure of a system element should neither result in an unacceptable loss, nor invoke another loss scenario.
- **Protective Recovery** - The recovery of a system element should not result in, nor lead to, unacceptable loss.
- **Redundancy** - The design delivers the required capability by the replication of functions or elements.

Technical Management Considerations

The effective coordination and management of LDSE specialty areas – areas that have often operated independently and in isolation of one another – poses a challenge. Jackson (2020) discusses the concept of siloism in the context of LDSE projects and ways to mitigate that effect. Siloism is the unwillingness of members of any team to share information. Failure to mitigate siloism can reduce the effectiveness of the entire team. Jackson recommends mitigating siloism using integrated product teams (IPTs), which utilize organizational structure and rigorous management to encourage sharing of information among specialties. Brtis (2021) suggests that for LDSE the systems engineer should holistically consider:

- the full spectrum of adversities
- the full spectrum of weaknesses, defects, flaws, exposures, hazards, and vulnerabilities
- the full spectrum of assets and losses
- the full spectrum of timeframes of interest
- the full spectrum of coping mechanisms

and further, suggests the systems engineer:

- elicit, analyze, and capture loss-driven requirements as part of the overall stakeholder and system requirements development
- make the loss-driven architectural decisions holistically across the loss-driven specialty areas
- make the loss-driven design decisions holistically across the loss-driven specialty areas
- integrate the management of risks associated with all loss-driven areas

Endler (2021) considers real life integration of loss-driven systems engineering activities into system development activities. Challenges identified include lack of appreciation of the importance of loss-driven systems engineering activities and organizational barriers. He finds that existing systems engineering standards poorly describe loss-driven systems engineering activities and fail to integrate loss-drive activities with traditional engineering activities. He proposes methods to overcome those barriers based on widely accepted standards. He offers an approach to accomplish the needed integration with emphasis on the need that loss-driven systems engineers participate throughout the system life cycle and be supported by a common understanding of an integrated approach.

Consequences for Model-Based Systems Engineering

Model-based systems engineering data and models need to be augmented to address the shared attributes of loss-driven systems engineering: assets, losses, adversities, and coping techniques and the common information artifacts identified above. Table 1 identifies some of the additional modeling information (in SysML form) that needs to be captured during the various lifecycle stages to support the effective development and documentation of loss management scenarios and loss management requirements.

Table 1. Modeling Information and Artifacts During Lifecycle Phases (Brtis 2020)

Lifecycle Phase	Artifacts and Information
Mission and Stakeholder Needs Analysis	Add adversities to the context diagram as actors • Add loss management scenarios as use cases
Stakeholder Requirements	Develop use case interaction diagrams to document the interaction of actors and architectural modules during the loss management scenarios • Develop sequence diagrams to represent the activity flow during loss management scenarios
System Requirements	Develop activity diagrams to show the states of the system (and adversities) during loss management scenarios
Architecture and System Design	Develop state models of the loss management scenarios • Model events and signals among the architectural nodes
System Design	Propose and select loss management design features • Document loss management related object distribution

Use Case: Manned Space Rescue Vehicle

Cureton (2020) explores the applicability of LDSE to a use case, a hypothetical manned space rescue vehicle, via a thought experiment regarding desirable characteristics for achieving resilience, safety, reliability, security, and other loss-driven goals. Various design reference missions are explored for assessment of required loss-driven capabilities in automated flight operations for a hypothetical manned space rescue vehicle.

Summary

Loss-driven systems engineering offers a valuable unification of loss-driven systems engineering specialty areas. Applying this can integrate currently isolated systems engineering activities, leading to improved system effectiveness and reduced systems engineering costs, while improving the management of potential losses associated with the development and use of systems.

References

Works Cited

- Cureton, K. L. "Role of LDSE for a Hypothetical Manned Space Rescue Vehicle." *INCOSE INSIGHT*, December 19-21. 2020.
- Brtis, J. S. and M. A. McEvilly. "Systems Engineering for Resilience," MITRE Technical Document #190495, The MITRE Corporation. 2019. https://www.researchgate.net/publication/334549424_Systems_Engineering_for_Resilience.
- Endler, D. "Integrating Loss-Driven Systems Engineering Activities." *INCOSE INSIGHT*, December 14-18. 2020.
- International Organization for Standardization (ISO). ISO/IEC/IEEE 15288:2015. Systems and software engineering – System life cycle processes. Geneva, CH: ISO. 2015.

Jackson, S. "Loss-Driven Systems Engineering and Siloism." *INCOSE INSIGHT*, December 32-33. 2020.

Watson, Michael D. "Systems Engineering Principles and Hypotheses." *INCOSE INSIGHT*, May 18-28. 2019.

Winstead, M., D. Hild, M. McEvelley. "Principles of Trustworthy Design of Cyber-Physical Systems." MITRE Technical Report #210263, The MITRE Corporation, June 2021. https://www.researchgate.net/publication/353934929_Principles_for_Trustworthy_Design_of_Cyber-Physical_Systems_Acknowledgments.

Primary References

Brtis, J. S. and M. A. McEvelley. "Unifying Loss-Driven Systems Engineering Activities." *INCOSE INSIGHT*, December 9-13. 2020. https://www.researchgate.net/publication/348502815_Unifying_Loss-Driven_Systems_Engineering_Activities.

Walden, D. D., et al. "Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities." Fourth Edition. San Diego, CA. INCOSE. 2015.

Winstead, M. "An Early Attempt at a Core, Common Set of Loss-Driven Systems Engineering Principles." *INCOSE INSIGHT*, December 22-26. 2020. <https://incose.onlinelibrary.wiley.com/doi/abs/10.1002/inst.12316>.

Additional References

None.

Human Systems Integration

- Lead Authors:
- Ben Schwartz and Holly Handley
- Contributing Authors:
- Guy Boy, Alice Squires, Paul Phister, Stuart Booth, and Dick Fairley

Human systems integration (HSI) is "the management and technical discipline of planning, enabling, coordinating, and optimizing all human-related considerations during system design, development, test, production, use and disposal of systems, subsystems, equipment and facilities." (SAE 2019).

Though used by industries around the world, HSI was initiated by the U.S. Department of Defense (DoD) as part of the "total system approach" to acquisition. The goal of HSI is to "optimize total system performance (hardware, software, and human), operational effectiveness, and suitability, survivability, safety, and affordability." (DoD 2003.) This article itself focuses on HSI within the DoD, although much of it is applicable to other government and commercial organizations worldwide.

HSI activities must be initiated "early in system development (during stakeholder requirements generation) and continuously through the development process to realize the greatest benefit to the final system solution and substantial life cycle cost savings." (INCOSE 2015).

HSI generally incorporates the following seven domains as integration considerations (although some organizations may use a slightly different set): manpower, personnel, training, human factors engineering, safety and occupational health, force protection and survivability, and habitability.

Overview

Historically, insufficient systems engineering resources were dedicated to ensuring proper integration of humans with the rest of the system. Most projects were technology-centered with human considerations being addressed through training. Technological systems were hard to use and maintain resulting in large manpower and training costs, reduced system performance, and increased risk of catastrophic loss, among other impacts.

The U.S. Army was among the first to address this with the Manpower and Personnel Integration (MANPRINT) program in 1986. MANPRINT emphasized the consideration of the HSI domains throughout the system acquisition as a standard part of the systems engineering effort. The approach has since been adopted by the broader DoD, by other militaries, and by civilian government agencies around the world. (Booher 2003). Some organizations, particularly the U.K. Ministry of Defence, use the term Human Factors Integration (HFI).

HSI applies systems engineering processes, tools, and techniques to ensure that human considerations are given proper weight in all system development activities. HSI should not be confused with Human Factors Engineering (HFE); HFE is a domain of HSI focusing on designing human interfaces. HSI is about mutual integration of technology, organizations, and people.

System Description

HSI is more than human factors, human-computer interaction, or systems engineering. It is a technical and managerial set of processes that involves the consideration and integration of multiple domains. In addition, HSI involves complexity analysis and organization design and management. Various organizations represent the HSI domains differently as the number and names of the domains are aligned with existing organizational structures. Booher (2003) first presented the original seven US Army domains as manpower, personnel, training, human factors engineering, soldier survivability, system safety and health hazards. Other countries may have a different number of domains with slightly different names, however, all the technical work of the domains is present. For example, the UK Defence Standard (00-251) includes seven similar domains: manpower, personnel, health hazards, training, human factors engineering, social/ organizational and system safety. The seven domains used across the DoD Instruction (DoD 2017) that tells how to operate the DoD Acquisition System are as follows:

1. **Manpower** Determining the most efficient and cost-effective mix of manpower and contract support necessary to operate, maintain, provide training and support the system. Manpower describes the number and mix of personnel required to carry out a task, multiple tasks, or mission in order to operate, maintain, support, and provide training for a system. Manpower factors are those variables that define manpower requirements. These variables include job tasks, operation/maintenance rates, associated workload, and operational conditions (e.g., risk of operator injury) (DAU 2010).
2. **Personnel:** Determining and selecting the appropriate cognitive, physical, and social capabilities required to train, operate, maintain, and sustain systems based on available personnel inventory or assigned to the mission. Personnel factors are those human aptitudes (i.e., cognitive, physical, and sensory capabilities), knowledge, skills, abilities, and experience levels that are needed to properly perform job tasks. Personnel factors are used to develop occupational specialties for system operators, maintainers, trainers, and support personnel (DAU 2010). The selection and assignment of personnel is critical to the success of a system, as determined by the needs set up by various work-related requirements.
3. **Training:** Developing efficient and cost-effective options that enhance user capabilities and maintain skill proficiencies for individual, collective, and joint training of operators and maintainers. Training is the learning process by which personnel individually or collectively acquire or enhance pre-determined job-relevant knowledge, skills, and abilities by developing their cognitive, physical, sensory, and team dynamic abilities. The "training/instructional system" integrates training concepts and strategies, as well as elements of logistic support to satisfy personnel performance levels required to operate, maintain, and support the systems. It includes the "tools" used to provide learning experiences, such as computer-based interactive courseware, simulators, actual

equipment (including embedded training capabilities on actual equipment), job performance aids, and Interactive Electronic Technical Manuals (DAU 2010).

4. **Human Factors Engineering:** The integration of human characteristics into system definition, design, development, and evaluation to optimize human-system performance under operational conditions. Human factors engineering (HFE) is primarily concerned with designing human-machine interfaces consistent with the physical, cognitive, and sensory abilities of the user population (DAU 2010). It focuses on ensuring that the system design is compatible with the human user. HFE considers required human tasks, and the sensory, perceptual, mental, and physical attributes of the user personnel who will operate, control, maintain, and support the equipment, system, or facility. The objective of HFE is to optimize human-system performance within the desired levels of life-cycle costs. HFE domain considerations span a wide range and include design and layout of working areas, user-equipment interfaces, use of automation and decision aids, and normal, non-normal, and emergency conditions.
5. **Safety & Occupational Health:** Consider environmental, safety and occupational health in determining system design characteristics to enhance job performance and minimize risks of illness, disability, injury and death to operators and maintainers. Safety considers the design features and operating characteristics of a system that serve to minimize the potential for human or machine errors or failure that cause injurious accidents (DAU 2010). Safety also encompasses the administrative procedures and controls associated with the operations, maintenance, and storage of a system. Occupational health factors are those system design features that serve to minimize the risk of injury, acute or chronic illness, or disability, and/or reduce job performance of personnel who operate, maintain, or support the system. Prevalent issues include noise, chemical safety, atmospheric hazards (including those associated with confined space entry and oxygen deficiency), vibration, ionizing and non-ionizing radiation, and human factors issues that can create chronic disease and discomfort such as repetitive motion diseases. Many occupational health problems, particularly noise and chemical management, overlap with environmental impacts. Human factors stress that creating a risk of chronic disease and discomfort overlaps with occupational health considerations (DAU 2010).
6. **Force Protection & Survivability:** Impact system design (e.g., egress, survivability) to protect individuals and units from direct threat events and accidents, including chemical, biological, and nuclear threats. Force Protection and Survivability is the HSI domain that facilitates system operation and personnel safety during and after exposure to hostile situations or environments. Force protection refers to all preventive measures taken to mitigate hostile actions against personnel (to include family members), resources, facilities, and critical information. Survivability denotes the capability of the system and/or personnel manning the system to avoid or withstand manmade hostile environments without suffering an abortive impairment of his/her ability to accomplish its designated mission. Survivability factors consist of those system design features that reduce the risk of fratricide, detection, and the probability of being attacked, and that enable personnel to withstand man-made hostile environments without aborting the mission or objective, or suffering acute chronic illness, disability, or death. Survivability attributes are those that contribute to the survivability of manned systems (DAU 2010).
7. **Habitability:** Establishing and enforcing requirements for individual and unit physical environments, personnel services, and living conditions, to prevent or mitigate risk conditions that adversely impact performance, quality of life and morale, or degrade recruitment or retention. Habitability factors are those living and working conditions that are necessary to sustain the morale, safety, health, and comfort of the user population. They directly contribute to personnel effectiveness and mission accomplishment and often preclude recruitment and retention problems. Examples include lighting, space, ventilation, and sanitation; noise and temperature control (i.e., heating and air conditioning); religious, medical, and food services availability; and berthing, bathing, and personal hygiene. Habitability consists of those characteristics of systems, facilities (temporary and permanent), and services necessary to satisfy personnel needs. Habitability factors are those living and working conditions that result in levels of personnel morale, safety, health, and comfort adequate to sustain maximum personnel effectiveness, support mission performance, and avoid personnel retention problems (DAU 2010).

Discipline Management

In a contractor project organization, the human systems integrator is typically a member of the senior engineering staff reporting to either the systems engineering lead or chief engineer.

HSI activities are documented in the Systems Engineering Management Plan (SEMP). Larger programs may have a stand-alone HSI Plan (HSIP) compatible with and referenced by the SEM. HSI activities are tailored to the needs of the project and the project life cycle (NASA 2016).

Most projects implement a Joint HSI Working Group between the customer and contractor. This enables sharing of priorities, knowledge, and effort to allow each group to achieve their objectives.

Discipline Relationships

Interactions

Interactions include:

- SE: HSI is an integral part of the systems engineering effort and the integrator participates in all relevant systems engineering activities during the whole life cycle of the system being considered.
- HSI domain experts: Domain experts collaborate with the human systems integrator to achieve HSI objectives, though this may or may not be a direct reporting relationship.
- The contractor and customer may each have a human systems integrator and various domain experts; each role should collaborate with their counterparts to the appropriate extent.
- HSI domain experts may participate in integrated product teams (IPTs)/design teams as full participants or consultants as appropriate for the needs of the project.
- HSI shares many concerns with Reliability, Availability, and Maintainability (RAM). The integrator and/or domain experts may collaborate with RAM specialists as appropriate.
- The integrator and/or domain experts should work with the Test & Evaluation team to ensure that HSI is represented in test and evaluation events.
- HSI shares many concerns with logistics and supportability, the integrator and/or domain experts may collaborate with this team as appropriate.

Dependencies

HSI depends on sufficient scope of work and authorization from the project. Proper planning and leadership buy-in is a key enabler.

Discipline Standards

Note: These are standards relevant to the practice of HSI specifically and not each of the HSI domains, which have their own standards and practices.

- National Aeronautics and Space Administration (NASA). 2015. Human Systems Integration Practitioner's Guide. NASA/SP-2015-3709. Houston: Johnson Space Center.
 - SAE International. 2019. Standard Practice for Human Systems Integration. SAE6906. Warrendale, PA: SAE International.
 - U.K. Ministry of Defence. 2016. Defence Standard 00-251: Human Factors Integration for Defence Systems. Glasgow: Defence Equipment and Support.
 - U.S. Army. 2015. Regulation 602-2: Human Systems Integration in the System Acquisition Process. Washington: Headquarters, Department of the Army.
 - U.S. Department of Defense. 2011. Data Item Description: Human Systems Integration Program Plan. DI-HFAC-81743A.
-

- U.S. Navy. 2017. Opnav Instruction 5310.23A: Navy Personnel Human Systems Integration. Washington: Office of the Chief of Naval Operations: Department of the Navy.

Personnel Considerations

HSI is conducted by a human systems integrator. The integrator is part of the systems engineering team responsible for conducting systems engineering related to human and organizational considerations and for coordinating the work of the HSI domain experts.

HSI uses the same techniques and approaches as systems engineering with additional consideration for non-materiel aspects of the system. Therefore, the integrator must be well-versed in the SE process and have a working understanding of each of the domains. The integrator does not need to be an expert in any of the domains.

The human systems integrator's responsibilities include:

- providing inputs to the SEMP and/or creating an HSI Plan (HSIP) compatible with the SEMP and the project life cycle (NASA Systems Engineering Handbook 2016)
- tailoring the scope of HSI efforts to the needs of the project and system life cycle
- ensuring HSI domains are given appropriate consideration across all programmatic and engineering activities
- assisting domain personnel in planning domain activities
- facilitating execution of domain tasks and collaboration among domains
- making trade-offs among domains to optimize the attainment of HSI goals
- optimizing the impact of domains on the acquisition program from the perspectives of performance, sustainability, and cost
- integrating the results of domain activities and representing them to the rest of the acquisition program from a total HSI perspective
- facilitating interactions among domains within the scope of HSI, and between HSI and the rest of the program
- tracking, statusing, and assessing HSI risks, issues and opportunities that have surfaced during the execution of the program

Metrics

Human-System Measures of Effectiveness

A measure of effectiveness (MOE) is a metric corresponding to the accomplishment of the mission objective. MOEs measure system performance in a representative mission context, including with representative users. Effectiveness is typically achieved through a combination of hardware, software, and human components, thus there are not typically HSI-specific MOEs.

MOEs may be decomposed into measures of performance (MOP) and measures of suitability (MOS). There may be HSI-specific MOPs and MOSs. For example, an MOE for an air defense radar might be positive detection probability, with an MOP for the radar's effective resolution and an MOS for the operator's ability to identify the target. It is the human system integrator's responsibility to ensure that relevant MOPs and MOSs are identified and incorporated into modeling, simulation, test, and evaluation efforts. The integrator and domain experts may contribute to these efforts as appropriate.

Models

HSI Process Models

HSI shares common systems engineering models with general systems engineering; e.g. the SE Vee process model. Additionally, a number of HSI-specific models and processes exist. A particularly good resource is "A User-Centered Systems Engineering Framework" by Ehrhart and Sage (in Booher 2003).

Human Performance and Domain Models

A variety of human performance models exist for cognition, behavior, anthropometry, strength, fatigue, attention, situation awareness, etc. Additionally, a variety of models exist for each HSI domain. The integrator should have a good understanding of the types of models available and the appropriate applications. In a project utilizing model-based systems engineering, the system model should include humans. The integrator should ensure sufficient fidelity to meet the needs of the project. Human-in-the-loop simulations should be encouraged during the design process as during the whole life cycle of a product.

Tools

HSI shares common tools with systems engineering. A sample of HSI-specific tools include:

- Command Control and Communications - Techniques for Reliable Assessment of Concept Execution (C3TRACE) ^[1] developed by U.S. Army Research Labs.
- Comprehensive Human Integration Evaluation Framework (CHIEF) ^[2] developed by U.S. Navy.
- Human Analysis and Requirements Planning System (HARPS) ^[3] developed by U.S. Navy Space and Naval Warfare Systems Command.
- Human Systems Integration Framework (HSIF) ^[4] developed by U.S. Air Force. (USAF 2009)
- Improved Performance and Research Integration Tool (IMPRINT) ^[1] developed by U.S. Army Research Labs.

Additionally, each HSI domain has specific tools and approaches for their unique efforts and considerations.

Practical Considerations

Pitfalls

Many organizations assign a human factors engineer to the human systems integrator role. This can be a mistake if the individual is not well versed in the SE process. Relegating HSI to a "specialty engineering" team deprives the integrator of sufficient scope and authority to accomplish their mission.

Proven Practices

Ensure the human systems integrator is a knowledgeable systems engineer with the respect of the other systems engineers and with a good understanding of each of the HSI domains. A human systems integrator should be involved in program planning activities to ensure sufficient budget and schedule. They should be involved in technical planning activities to create sufficient scope in the SEMP/HISPP, identify HSI-related risks and opportunities, recommend HSI trade studies, etc. There is significant overlap and trade space among the HSI domains, therefore the domain experts, led by the integrator, should collaborate throughout the project to optimize the impact of HSI.

References

Works Cited

- Booher, H. R. ed. *Handbook of Human Systems Integration*. Hoboken, NJ. Wiley. 2003.
- Boy, G. A. "Human-Centered Design as an Integrating Discipline." "Journal of Systemics, Cybernetics and Informatics." International Institute of Informatics and Systemics 15, no. 1 (2-15) 25-32. ISSN 1690-4524.
- Defense Acquisition University (DAU). *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA. Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.
- DoD. *DoD Directive 5000.01*, The Defense Acquisition System. 2003. <https://www.esd.whs.mil/Portals/54/Documents/DD/issuances/dodd/500001p.pdf>.
- DoD. *DoD Instruction 5000.02T*, Operation of The Defense Acquisition System. 2017. <https://www.esd.whs.mil/Portals/54/Documents/DD/issuances/dodi/500002p.pdf>.
- National Aeronautics and Space Administration (NASA). *NASA Systems Engineering Handbook Rev. 2*. 2016. <https://www.nasa.gov/connect/ebooks/nasa-systems-engineering-handbook>
- SAE International. *Standard Practice for Human Systems Integration (SAE6906)*. Warrendale, PA. SAE. 2019. <https://saemobilus.sae.org/content/sae6906>.
- UK Defence Standardization. *Def Stan 00-251 Human Factors Integration for Defence Systems*, Revision II. 2016. Accessed April 2, 2021. https://global.ihs.com/doc_detail.cfm?&item_s_key=00670685&item_key_date=831026&input_doc_number=00%2D251&input_doc_title=#product-details-list.
- USAF. *Air Force Human Systems Integration Handbook*. Brooks City-Base, TX, USA: Directorate of Human Performance Integration. 2009. <https://www.acqnotes.com/Attachments/Air%20Force%20Human%20System%20Integration%20Handbook.pdf>.

Primary References

None.

Additional References

- Blanchard, B. S. and W. J. Fabrycky. *Systems Engineering and Analysis*. 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ. Prentice-Hall. 2011.
- Boy, G. A. "Orchestrating Human-Centered Design." Springer, U.K. 2013. ISBN 978-1-4471-4338-3.
- Helander, M., T. K. Landauer, and P. V. Prabhu. *Handbook of Human-Computer Interaction*. Amsterdam, Netherlands. Elsevier. 1997.
- Pew, R. W. and A. S. Mavor. *Human-System Integration in the System Development Process: A New Look*. Washington, DC. National Academies Press. 2007.
- Simpkiss, B. "AFHSIO-001: Human Systems Integration Requirements Pocket Guide." Falls Church, VA. Air Force Human Systems Integration Office. 2009.
- Wickens, C. D., J. D. Lee, Y. Liu, and S. Gordon-Becker. *An Introduction to Human Factors Engineering*, 2nd ed. Saddle River, NJ. Prentice-Hall. 2011.
- Woodson, W. E, B. Tillman, and P. Tillman. *Human Factors Design Handbook: Information and Guidelines for the Design of Systems, Facilities, Equipment, and Products for Human Use*, 2nd ed. New York, NY. McGraw Hill. 1991.

References

- [1] <https://www.arl.army.mil/www/default.cfm?page=3200>
- [2] <https://calhoun.nps.edu/handle/10945/42696>
- [3] <https://www.dau.mil/cop/log/pages/topics/Manpower%20and%20Personnel.aspx>
- [4] https://www.acq.osd.mil/se/webinars/2015_10_06-SoSECIE-Risser-Lacson-brief.pdf

Manufacturability and Producibility

- Lead Authors:
- Dick Fairley and Kevin Forsberg
- Contributing Authors:
- Paul Phister, Alice Squires, and Richard Turner

Manufacturability and producibility is an engineering specialty. The machines and processes used to build a system must be architected and designed. A systems engineering approach to manufacturing and production is necessary because manufacturing equipment and processes can sometimes cost more than the system being built (Maier and Rechtin 2009). Manufacturability and producibility can be a discriminator between competing system solution concepts and therefore must be considered early in the study period, as well as during the maturing of the final design solution.

Overview

The system being built might be intended to be one-of-a-kind or to be reproduced multiple times. The manufacturing system differs for each of these situations and is tied to the type of system being built. For example, the manufacture of a single-board computer would be vastly different from the manufacture of an automobile. Production involves the repeated building of the designed system. Multiple production cycles require the consideration of production machine maintenance and downtime.

Manufacturing and production engineering involve similar systems engineering processes specifically tailored to the building of the system. Manufacturability and producibility are the key attributes of a system that determine the ease of manufacturing and production. While manufacturability is simply the ease of manufacture, producibility also encompasses other dimensions of the production task, including packaging and shipping. Both these attributes can be improved by incorporating proper design decisions that take into account the entire system life cycle (Blanchard and Fabrycky 2010).

Works Cited

- Maier, M. and E. Rechtin. *The Art of Systems Architecting*, 3rd ed. Boca Raton, FL. CRC Press. 2009.
- Blanchard, B. S. and W. J. Fabrycky. *Systems Engineering and Analysis*, 5th Ed. Prentice-Hall International Series in Industrial and Systems Engineering. Englewood Cliffs, NJ. Prentice-Hall. 2010.

Primary References

None.

Additional References

Anderson, D. *Design for Manufacturability & Concurrent Engineering; How to Design for Low Cost, Design in High Quality, Design for Lean Manufacture, and Design Quickly for Fast Production*. Cambria, CA. CIM Press. 2010.

Boothroyd, G., P. Dewhurst, and W. Knight. *Product Design for Manufacture and Assembly*. 3rd ed. Boca Raton, FL. CRC Press. 2010.

Bralla, J. *Design for Manufacturability Handbook*. New York, NY. McGraw Hill Professional. 1998.

System Affordability

- Lead Author:
- Paul Phister
- Contributing Author:
- Ray Madachy

According to MITRE (2014), affordability is the "ability to fund desired investment". "Solutions are affordable if they can be deployed in sufficient quantity to meet mission needs within the (likely) available budget." INCOSE (2015) offers a slightly deeper definition. A system is affordable to the degree that system performance, cost, and schedule constraints are balanced over the system life, while mission needs are satisfied in concert with strategic investment and organizational needs. Design for affordability is the practice of considering affordability as a design characteristic or constraint.

Increasing competitive pressures and the scarcity of resources demand that systems engineering (SE) improve affordability. Several recent initiatives have made affordability their top technical priority. They also call for a high priority to be placed on research into techniques — namely, improved systems autonomy and human performance augmentation — that promise to reduce labor costs, provide more efficient equipment to reduce supply costs, and create adaptable systems whose useful lifetime is extended cost-effectively.

However, methods for cost and schedule estimation have not changed significantly to address these new challenges and opportunities. There is a clear need for:

- new methods to analyze trade-offs between cost, schedule, effectiveness, and resilience;
- new methods to adjust priorities and deliverables to meet budgets and schedules; and
- more affordable systems development processes.

All of this must be accomplished in the context of the rapid changes underway in technology, competition, operational concepts, and workforce characteristics.

Overview

Historically, cost and schedule estimation has been decoupled from technical SE trade-off analyses and decision reviews. Most models and tools focus on evaluating either cost-schedule performance or technical performance, but not the trade-offs between the two. Meanwhile, organizations and their systems engineers often focus on affordability to minimize acquisition costs. They are then drawn into the easiest-first approaches that yield early successes, at the price of being stuck with brittle, expensive-to-change architectures that increase technical debt and life cycle costs.

Two indications that the need for change is being recognized in systems engineering are that the *INCOSE SE Handbook* now includes affordability as one of the criteria for evaluating requirements (INCOSE 2015) and that there is a trend in SE towards stronger focus on maintainability, flexibility, and evolution (Blanchard, Verma, and Peterson 1995).

There are pitfalls for the unwary. Autonomous systems experience several hazardous failure modes, including:

- **system instability due to positive feedback** — where an agent senses a parameter reaching a control limit and gives the system a strong push in the other direction, causing the system to rapidly approach the other control limit, causing the agent (or another) to give it an even stronger push in the original direction, and so on
-

- **self-modifying autonomous agents which fail** after several self-modifications — the failures are difficult to debug because the agent's state has been changing
- **autonomous agents performing weakly at commonsense reasoning** about system control decisions by human operators, and so tend to reach incorrect conclusions and make incorrect decisions about controlling the system
- **multiple agents making contradictory decisions** about controlling the system, and lacking the ability to understand the contradiction or to negotiate a solution to resolve it

Modularization of the system's architecture around its most frequent sources of change (Parnas 1979) is a key SE principle for affordability. This is because when changes are needed, their side effects are contained in a single systems element, rather than rippling across the entire system.

This approach creates the need for three further improvements:

- refocusing the system requirements, not only on a snapshot of current needs, but also on the most likely sources of requirements change, or evolution requirements;
- monitoring and acquiring knowledge about the most frequent sources of change to better identify requirements for evolution;
- evaluating the system's proposed architecture to assess how well it supports the evolution requirements, as well as the initial snapshot requirements.

This approach can be extended to produce several new practices. Systems engineers can

- identify the commonalities and variability across the families of products or product lines, and develop architectures for creating (and evolving) the common elements *once* with plug-compatible interfaces for inserting the variable elements; (Boehm, Lane, and Madachy 2010)
- extrapolate principles for service-oriented system elements that are characterized by their inputs, outputs, and assumptions, and that can easily be composed into systems in which the sources of change were not anticipated;
- develop classes of smart or autonomous systems whose many sensors identify needed changes, and whose autonomous agents determine and effect those changes in microseconds, or much more rapidly than humans can, reducing not only reaction time, but also the amount of human labor needed to operate the systems, thus improving affordability.

Personnel Considerations

Autonomous systems need human supervision, and the humans involved require better methods for trend analysis and visualization of trends (especially undesired ones).

There is also the need, with autonomous systems, to extend the focus from life cycle costs to total ownership costs, which encompass the costs of failures, including losses in sales, profits, mission effectiveness, or human quality of life. This creates a further need to evaluate affordability in light of the value added by the system under consideration. In principle, this involves evaluating the system's total cost of ownership with respect to its mission effectiveness and resilience across a number of operational scenarios. However, determining the appropriate scenarios and their relative importance is not easy, particularly for multi-mission systems of systems. Often, the best that can be done involves a mix of scenario evaluation and evaluation of general system attributes, such as cost, schedule, performance, and so on.

As for these system attributes, different success-critical stakeholders will have different preferences, or utility functions, for a given attribute. This makes converging on a mutually satisfactory choice among the candidate system solutions a difficult challenge involving the resolution of the multi-criteria decision analysis (MCDA) problem among the stakeholders (Boehm and Jain 2006). This is a well-known problem with several paradoxes, such as Arrow's impossibility theorem that describes the inability to guarantee a mutually optimal solution among several stakeholders, and several paradoxes in stakeholder preference aggregation in which different voting procedures produce different winning solutions. Still, groups of stakeholders need to make decisions, and various negotiation

support systems enable people to better understand each other's utility functions and to arrive at mutually satisfactory decisions, in which no one gets everything that they want, but everyone is at least as well off as they are with the current system.

Also see System Analysis for considerations of cost and affordability in the technical design space.

Primary References

Works Cited

- Blanchard, B., D. Verma, and E. Peterson. 1995. *Maintainability: A Key to Effective Serviceability and Maintenance Management*. New York, NY, USA: Wiley and Sons.
- Boehm, B., J. Lane, and R. Madachy. 2010. "Valuing system flexibility via total ownership cost analysis." Proceedings of the NDIA SE Conference, October 2010, San Diego, CA, USA.
- Boehm, B., and A. Jain. 2006. "A value-based theory of systems engineering." Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium (IS), July 9-13, 2006, Orlando, FL, USA.
- INCOSE. 2015. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 4.0. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-04.
- MITRE. 2014. *Systems Engineering Guide*. Bedford, MA, USA. Accessed April 1, 2021. Available at <https://www.mitre.org/publications/technical-papers/the-mitre-systems-engineering-guide>.
- Parnas, D.L. 1979. "Designing Software for Ease of Extension and Contraction." *IEEE Transactions on Software Engineering*. 5(2): 128-138.

Primary References

- INCOSE. 2015. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 4.0. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-04.
- Blanchard, B., D. Verma, and E. Peterson. 1995. *Maintainability: A Key to Effective Serviceability and Maintenance Management*. New York, NY, USA: Wiley and Sons.
- Parnas, D.L. 1979. "Designing Software for Ease of Extension and Contraction." *IEEE Transactions on Software Engineering*. 5(2): 128-138.

Additional References

- Kobren, Bill. 2011. "Supportability as an affordability enabler: A critical fourth element of acquisition success across the system life cycle." *Defense AT&L: Better Buying Power*. Accessed April 1, 2021. Available at <https://apps.dtic.mil/sti/citations/ADA564402>.
- Myers, S.E., P.P. Pandolfini, J.F. Keane, O. Younossi, J.K. Roth, M.J. Clark, D.A. Lehman, and J.A. Dechoretz. 2000. "Evaluating affordability initiatives." *Johns Hopkins APL Tech. Dig.* 21(3): 426–437.
- Redman, Q. 2012. "Why affordability is a systems engineering metric." *Procedia Computer Science*. 8: 376-381. Elsevier. Accessed April 1, 2021. Available: <https://www.sciencedirect.com/science/article/pii/S1877050912000762>.

System Hardware Assurance

- Lead Authors:
- Michael Bear, Donald Davidson, Shawn Fetterolf, Darin Leonhardt, Daniel Radack, Karen Johnson, and Elizabeth A. McDaniel
- Contributing Authors:
- Michael Berry, Brian Cohen, Diganta Das, Houman Homayoun, and Thomas McDermott

This article describes the discipline of hardware assurance, especially as it relates to systems engineering. It is part of the SE and Quality Attributes Knowledge Area.

Overview

System hardware assurance is a set of system security engineering activities (see System Security for more information) undertaken to quantify and increase the confidence that electronics function as intended and only as intended throughout their life cycle, and to manage identified risks. The term *hardware* refers to electronic components, sometimes called integrated circuits or chips. As products of multi-stage processes involving design, manufacturing and post-manufacturing, packaging, and test, they must function properly under a wide range of circumstances. Hardware components – alone and integrated into subcomponents, subsystems, and systems – have weaknesses and vulnerabilities enabling exploitation. Weaknesses are flaws, bugs, or errors in design, architecture, code, or implementation. Vulnerabilities are weaknesses that are exploitable in the context of use (Martin 2014).

Hardware assurance is conducted to minimize risks related to hardware that can enable adversarial exploitation and subversion of functionality, counterfeit production, and loss of technological advantage. Challenges include increasing levels of sophistication and complexity of hardware architectures, integrated circuits, operating systems, and application software, combined with supply chain risks, emergence of new attack surfaces, and reliance on global sources for some components and technologies.

After identifying concerns and applicable mitigations, hardware assurance offers a range of possible activities and processes. At the component level, hardware assurance focuses on the hardware itself and the supply chain used to design and manufacture it; at the subcomponent, subsystems, and system levels, hardware assurance incorporates the software and firmware integrated with the component.

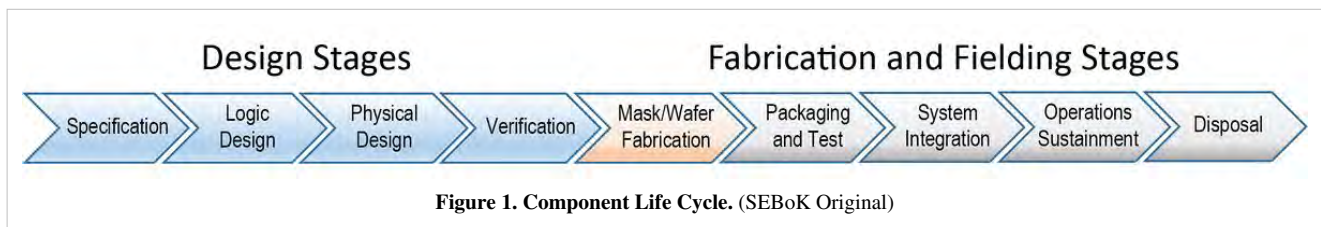
Engineering efforts to enhance trust in hardware have increased in response to complex hardware architectures, the increasing sophistication of adversarial attacks on hardware, and globalization of supply chains. These factors raise serious concerns about the security, confidentiality, integrity, and availability as well as the provenance and authenticity of hardware. The “root of trust” (NIST 2020) of a system is typically contained in the processes, steps, and layers of hardware components and across the systems engineering development cycle. System hardware assurance focuses on hardware components and their interconnections with software and firmware to reduce risks to proper function or other compromises of the hardware throughout the complete life cycle of components and systems. Advances in hardware assurance tools and techniques will strengthen designs, and enhance assurance during manufacturing, packaging, test, and deployment and operational use.

Life Cycle Concerns of Hardware Components

Hardware assurance should be applied at various stages of a component's life cycle from hardware architecture and design, through manufacturing and testing, and finally throughout its inclusion in a larger system. The need for hardware assurance then continues throughout its operational life including sustainment and disposal.

As semiconductor technology advances the complexity of electronic components, it increases the need to “bake-in” assurance. Risks created during architecture, design, and manufacturing are challenging to address during the operational phase. Risks associated with interconnections between and among chips are also a concern. Therefore, improving a hardware assurance posture must occur as early as possible in the life cycle, thereby reducing the cost and schedule impacts associated with “fixing” components later in the life cycle of the system.

A conceptual overview of the typical hardware life cycle (Figure 1) illustrates the phases of the life cycle of components, as well as the subsystems and systems in which they operate. In each phase multiple parties and processes contribute a large set of variables and corresponding attack surfaces. As a result, the potential exists for compromise of the hardware as well as the subcomponents and systems in which they operate; therefore, matching mitigations should be applied at the time the risks are identified.



Both the value of the hardware component and the associated cost of mitigating risks increase at each stage of the life cycle. Therefore, it is important to identify and mitigate vulnerabilities as early as possible. It takes longer to find and fix defects later, thereby increasing the complexity of replacing hardware with “corrected” designs that create system integration issues. In addition to cost savings, early correction and mitigation avoid delays in creating an operational system. It is essential to re-assess risks associated with hardware components throughout the life cycle periodically, especially as operational conditions change.

Hardware assurance during system sustainment is a novel challenge given legacy hardware and designs with their associated supply chains. In long-lived high-reliability systems, hardware assurance issues are compounded by obsolescence and diminished sourcing of components, thereby increasing concerns related to counterfeits and acquisitions from the gray market.

Function as Intended and Only as Intended

Exhaustive testing can check system functions against specifications and expectations; however, checking for unintended functions is problematic. Consumers have a reasonable expectation that a purchased product will perform as advertised and function properly (safely and securely, under specified conditions) – but consumers rarely consider if additional functions are built into the product. For example, a laptop with a web-conferencing capability comes with a webcam that will function properly when enabled, but what if the webcam also functions when turned off, thereby violating expectations of privacy? Given that a state-of-the-art semiconductor component might have billions of transistors, “hidden” functions might be exploitable by adversaries. The statement “function as intended and only intended” communicates the need to check for unintended functions.

Hardware specifications and information in the design phase are needed to validate that components function properly to support systems or missions. If an engineer creates specifications that support assurance that flow down the system development process, the concept of “function as intended” can be validated for the system and mission through accepted verification and validation processes. “Function only as intended” is also a consequence of capturing the requirements and specifications to assure the product is designed and developed without extra

functionality. For example, a Field Programmable Gate Array (FPGA) contains programmable logic that is highly configurable; however, the programmable circuitry might be susceptible to exploitation.

Given the specifications of a hardware component, specialized tools and processes can be used to determine with a high degree of confidence whether the component's performance meets specifications. Research efforts are underway to develop robust methods to validate that a component does not have capabilities that threaten assurance or that are not specified in the original design. Although tools and processes can test for known weaknesses, operational vulnerabilities, and deviations from expected performance, all states of possible anomalous behavior cannot currently be determined or predicted.

Data and information can be used to validate the component's function and should be collected from multiple sources including designers, developers, and members of the user community. Designers and developers can provide deep understanding of the component's intended function and provide tests used to verify its functional performance before fielding. The merging of component design and development information with extensive field data, including third-party evaluation, contributes to assurance that the component is performing specified functions and that no unintended functionality is observed.

Risks to Hardware

Modern systems depend on complex microelectronics, but advances in hardware without attention to associated risks can expose critical systems, their information, and the people who rely on them. "Hardware is evolving rapidly, thus creating fundamentally new attack surfaces, many of which will never be entirely secured". (Oberg 2020) Therefore, it is imperative that risk be modeled through a dynamic risk profile and be mitigated in depth across the entire profile. Hardware assurance requires extensible mitigations and strategies that can and do evolve as threats do. Hardware assurance methods seek to quantify and improve confidence that weaknesses that can become vulnerabilities that create risks are mitigated.

Most hardware components are commercially designed, manufactured, and inserted into larger assemblies by multi-national companies with global supply chains. Understanding the provenance and participants in complex global supply chains is fundamental to assessing risks associated with the components.

Operational risks that derive from unintentional or intentional features are differentiated based on the source of the feature. Three basic operational risk areas related to goods, products, or items are: failure to meet quality standards, maliciously tainted goods, and counterfeit hardware. Counterfeits are usually offered as legitimate products, but they are not. They may be refurbished or mock items made to appear as originals, re-marked products, the result of overproduction, or substandard production parts rejected by the legitimate producer. Counterfeit risks and substandard quality offer avenues for malware insertion and potential impacts to overall system performance and availability.

Failure to follow quality standards including safety and security standards, especially in design, can result in unintentional features or flaws being inadvertently introduced. These can occur through mistakes, omissions, or lack of understanding how features might be manipulated by future users for nefarious purposes. Features introduced intentionally for specific purposes can also make the hardware susceptible to espionage or control of the hardware at some point in its life cycle.

Quantify and Improve Confidence

The quantification of hardware assurance is a key technical challenge because of the complex interplay among designer, manufacturer and supply chains, and adversarial intent, as well as the challenge of defining "security" with respect to hardware function. Quantification is necessary to identify and manage hardware risks within program budgets and timeframes. It enables a determination of the required level of hardware assurance and whether quantification is achievable throughout the hardware's life cycle.

Current methods for quantifying hardware assurance are adapted from the fields of quality and reliability engineering, which use methods like Failure Mode and Effects Analysis (FMEA). (SAE 2021) FMEA is semi-quantitative and combines probabilistic hardware failure data and input from experts. Adapting FMEA to quantify hardware assurance is hampered when it relies on assigning probabilities to human behavior that may be motivated by money, malicious intent, etc. Expert opinion often varies when quantifying and weighting factors used in generating risk matrices and scores. In response, recent efforts are attempting to develop quantitative methods that reduce subjectivity.

Game theoretic analysis (game theory) is the creation of mathematical models of conflict and cooperation between intelligent and rational decision-makers. (Myerson 1991) Models include dynamic, as opposed to static, interactions between attackers and defenders that can quantify the risks associated with potential interactions among adversaries, hardware developers, and manufacturing processes. (Eames and Johnson 2017) Creation of the models forces one to define attack scenarios explicitly and to input detailed knowledge of hardware development and manufacturing processes. Outputs of the model may include a ranking of the most likely attacks to occur based on cost-benefit constraints on the attackers and defenders. (Graf 2017) The results can empower decision-makers to make quantitative trade-off decisions about hardware assurance.

Another quantification method that results in a confidence interval for detecting counterfeit/suspect microelectronics is presented in the SAE AS6171 standard. (SAE 2016) Confidence is based on knowing the types of defects associated with counterfeits, and the effectiveness of different tests to detect those defects. Along the same lines, a standard for hardware assurance might be developed to quantify the confidence interval by testing against a catalogue of known vulnerabilities, such as those documented in the MITRE Common Vulnerabilities and Exposures (CVE) list. (MITRE 2020)

Distributed ledger technology (DLT) is an example of an emerging technology that could enable a standardized approach for quantifying hardware assurance attributes such as data integrity, immutability, and traceability. DLT can be used in conjunction with manufacturing data (such as dimensional measurement, parametric testing, process monitoring, and defect mapping) to improve tamper resistance using component provenance and traceability data. DLT also enables new scenarios of cross-organizational data fusion, opening the door to new classes of hardware integrity checks.

Manage Risks

The selection of specific components for use in subsystems and systems should be the outcome of performance-risk and cost-benefit trade-off assessments in their intended context of use. The goal of risk management and mitigation planning is to select mitigations with the best overall operational risk reduction and the lowest cost impact. The required level of hardware assurance varies with the criticality of a component's use and the system in which it is used.

During a typical development life cycle of a system – architecture, design, code, and implementation – various types of problems can pose risks to the operational functionality of the hardware components provided. These risks include weaknesses or defects that are inadvertent (unintentional), as well as counterfeits that may be either inadvertent or intentionally injected into the supply chain for financial motivations or malicious components designed to change functionality.

Managing risk in the context of hardware assurance seeks to decrease the risk of weaknesses that create attack surfaces that can be exploited, while improving confidence that an implementation resists exploitation. Ideally, risk management reduces risk and maximizes assurance to an acceptable level. Often, risks are considered in the context of likelihood of consequences and the costs and effectiveness of mitigations. However, new operationally impactful risks are recognized continuously over the hardware life cycle and supply chains of components. At the same time hardware weaknesses are often exploited through software or firmware. Therefore, to maximize assurance and minimize operationally impactful risks mitigation-in-depth across all constituent components must be considered.

This highlights the need for a dynamic risk profile.

An example of a post-manufacturing mitigation involves a new hardware risk identified during field operation. A dynamic risk profile can be used to characterize the issue and identify possible resources to address the suspect component function. This profile can also be used to track and address risks throughout its life, including obsolescence-related risk. One means of mitigating this kind of hardware life cycle risk is the use of existing programmable logic.

Just as with software patches and updates, new attack surfaces on hardware may become exposed through the mitigation being applied, and they will likely take a long time to discover. In the example above, the programmable logic is updated to provide a new configuration to protect the hardware. In this context, access to hardware reconfiguration must be limited to authorized parties to prevent an unauthorized update that introduces weaknesses on purpose or by accident. While programmable logic may have mitigated a specific attack surface or type of weakness, additional mitigations are needed to minimize risk more completely. This is mitigation-in-depth – multiple mitigations building upon one another.

Throughout the entire supply chain, critical pieces of information can be inadvertently exposed. The exposure of such information directly enables the creation and exploitation of new attack surfaces. Therefore, the supply chain infrastructure must also be assessed for weaknesses, and the development, use, and maintenance of hardware components assured. The dynamic risk profile offers a framework to balance mitigations in the context of risk and cost throughout the complete hardware and system life cycles.

References

Works Cited

- Eames, B.K. and M.H. Johnson. 2017. "Trust Analysis in FPGA-based Systems." Proceeding of GOMACTech 2017, March 20-23, 2017, Reno, NV.
- Graf, J. 2017. "OpTrust: Software for Determining Optimal Test Coverage and Strategies for Trust." Proceedings of GOMACTech 2017, March 20-23, 2017, Reno, NV.
- Martin, R.A. 2014. "Non-Malicious Taint: Bad Hygiene is as Dangerous to the Mission as Malicious Intent." CrossTalk Magazine. 27(2).
- MITRE. 2020. "Common Vulnerabilities and Exposures." Last Updated December 11, 2020. Accessed March 31, 2021. Available at <https://cve.mitre.org/cve/>,
- Myerson, R.R. 1991. *Game Theory: Analysis of Conflict*. Cambridge, MA: Harvard University Press.
- NIST. 2020. Roots of Trust. Last Updated June 22, 2020. Accessed March 31, 2021. Available at <https://csrc.nist.gov/projects/hardware-roots-of-trust>.
- Oberg, J. 2020. Reducing Hardware Security Risk. Last Updated July 1, 2020. Accessed March 31, 2021. Available at: <https://semiengineering.com/reducing-hardware-security-risk/>.
- SAE. 2016. SAE AS6171, *Test Methods Standard: General Requirements, Suspect/Counterfeit, Electrical, Electronic, and Electromechanical Parts*. SAE International. Accessed March 31, 2021. Available at <https://www.sae.org/standards/content/as6171/>.
- SAE. 2021. SAE J1739_202101, *Potential Failure Mode and Effects Analysis (FMEA) Including Design FMEA, Supplemental FMEA-MSR, and Process FMEA*. SAE International. Last Updated January 13, 2021. Accessed on March 31, 2021. Available at https://www.sae.org/standards/content/j1739_202101/.

Primary References

- Bhunja, S. and M. Tehranipoor. 2018. *Hardware Security: A Hands-on Learning Approach*. Amsterdam, Netherlands: Elsevier Science.
- ENISA. 2017. *Hardware Threat Landscape and Good Practice Guide. Final Version 1.0*. European Union Agency for Cybersecurity. Accessed March 31, 2021. Available at <https://www.enisa.europa.eu/publications/hardware-threat-landscape>
- TAME Steering Committee. 2019. *Trusted and Assured Microelectronics Forum Working Group Reports*. Last Updated December 2019. Accessed March 31, 2021. Available at: <https://dforte.ece.ufl.edu/wp-content/uploads/sites/65/2020/08/TAME-Report-FINAL.pdf>.

Additional References

- DARPA. A DARPA Approach to Trusted Microelectronics. Accessed March 31, 2021. Available at: https://www.darpa.mil/attachments/Background_FINAL3.pdf ^[1].
- Fazzari, S. and R. Narumi. 2019. *New & Old Challenges for Trusted and Assured Microelectronics*. Accessed March 31, 2021. Available at: <https://apps.dtic.mil/dtic/tr/fulltext/u2/1076110.pdf>.
- IEEE. 2008-2020. IEEE International Symposium on Hardware Oriented Security and Trust (HOST). Annual symposium held since 2008 providing wealth of articles on hardware assurance.
- Martin, R. 2019. "Hardware Assurance and Weakness Collaboration and Sharing (HAWCS)." Proceedings of the 2019 Software and Supply Chain Assurance Forum, September 17-18, 2019 in McLean, VA. Accessed March 31, 2021. Available at https://csrc.nist.gov/CSRC/media/Projects/cyber-supply-chain-risk-management/documents/SSCA/Fall_2019/WedPM2.2_Robert_Martin.pdf.
- NDIA. 2017. Trusted Microelectronics Joint Working Group: Team 3 White Paper: Trustable Microelectronics Standard Products. Accessed March 31, 2021. Available at <https://www.ndia.org/-/media/sites/ndia/divisions/working-groups/tmjwg-documents/ndia-tm-jwg-team-3-white-paper-finalv3.ashx>.
- Regenscheid, A. 2019. NIST SP 800-193, *Platform Firmware Resiliency Guidelines*. Accessed March 31, 2021. Available at <https://csrc.nist.gov/publications/detail/sp/800-193/final>.
- Ross, R., V. Pillitteri, R. Graubart, D. Bodeau, R. McQuaid. 2019. NIST SP 800-160 Vol. 2, *Developing Cyber Resilient Systems – A Systems Security Engineering Approach*. Accessed March 31, 2021. Available at: <https://csrc.nist.gov/News/2019/sp-800-160-vol2-developing-cyber-resilient-systems>.

References

- [1] <https://www.darpa.mil/about-us/darpa-approach-to-trusted-microelectronics>

System Reliability, Availability, and Maintainability

- Lead Authors:
- Paul Phister and David Olwell

Reliability, availability, and maintainability (RAM) are three system attributes that are of tremendous interest to systems engineers, logisticians, and users. They are often studied together. Collectively, they affect economic life-cycle costs of a system and its utility.

Overview

Reliability, maintainability, and availability (RAM) are three system attributes that are of great interest to systems engineers, logisticians, and users. Collectively, they affect both the utility and the life-cycle costs of a product or system. The origins of contemporary reliability engineering can be traced to World War II. The discipline's first concerns were electronic and mechanical components. (Ebeling 2010) However, current trends point to a dramatic rise in the number of industrial, military, and consumer products with integrated computing functions. Because of the rapidly increasing integration of computers into products and systems used by consumers, industry, governments, and the military, reliability must consider both hardware, and software.

Maintainability models present some interesting challenges. The time to repair an item is the sum of the time required for evacuation, diagnosis, assembly of resources (parts, bays, tool, and mechanics), repair, inspection, and return. Administrative delay (such as holidays) can also affect repair times. Often these sub-processes have a minimum time to complete that is not zero, resulting in the distributions used to model maintainability having a threshold parameter.

A threshold parameter is defined as the minimum probable time to repair. Estimation of maintainability can be further complicated by queuing effects, resulting in times to repair that are not independent. This dependency frequently makes analytical solution of problems involving maintainability intractable and promotes the use of simulation to support analysis.

System Description

This section sets forth basic definitions, briefly describes probability distributions, and then discusses the role of RAM engineering during system development and operation. The final subsection lists the more common reliability test methods that span development and operation.

Basic Definitions

Reliability

Reliability is defined as the probability of a product performing its intended function under stated conditions without failure for a given period of time. (ASQ 2022) A precise definition must include a detailed description of the function, the environment, the time scale, and what constitutes a failure. Each can be surprisingly difficult to define precisely.

Maintainability

The probability that a given maintenance action for an item under given usage conditions can be performed within a stated time interval when the maintenance is performed under stated conditions using stated procedures and resources. Maintainability has two categories: serviceability (the ease of conducting scheduled inspections and servicing) and repairability (the ease of restoring service after a failure). (ASQ 2022)

Availability

Defined as the probability that a repairable system or system element is operational at a given point in time under a given set of environmental conditions. Availability depends on reliability and maintainability and is discussed in detail later in this topic. (ASQ 2011)

Failure

A failure is the event(s), or inoperable state, in which any item or part of an item does not, or would not, perform as specified. (GEIA 2008) The failure mechanism is the physical, chemical, electrical, thermal, or other process that results in failure (GEIA 2008). In computerized systems, a software defect or fault can be the cause of a failure (Laprie 1992) which may have been preceded by an error which was internal to the item. The failure mode is the way or the consequence of the mechanism through which an item fails. (GEIA 2008, Laprie 1992) The severity of the failure mode is the magnitude of its impact. (Laprie 1992)

Probability Distributions used in Reliability Analysis

Reliability can be thought of as the probability of the survival of a component until time t . Its complement is the probability of failure before or at time t . If we define a random variable T as the time to failure, then:

where $R(t)$ is the reliability and $F(t)$ is the failure probability. The failure probability is the cumulative distribution function (CDF) of a mathematical probability distribution. Continuous distributions used for this purpose include exponential, Weibull, log-normal, and generalized gamma. Discrete distributions such as the Bernoulli, Binomial, and Poisson are used for calculating the expected number of failures or for single probabilities of success.

The same continuous distributions used for reliability can also be used for maintainability although the interpretation is different (i.e., probability that a failed component is restored to service prior to time t). However, predictions of maintainability may have to account for processes such as administrative delays, travel time, sparing, and staffing and can therefore be extremely complex.

The probability distributions used in reliability and maintainability estimation are referred to as models because they only provide estimates of the true failure and restoration of the items under evaluation. Ideally, the values of the parameters used in these models would be estimated from life testing or operating experience. However, performing such tests or collecting credible operating data once items are fielded can be costly. Therefore, approximations sometimes use data from “similar systems”, “engineering judgment”, and other methods. As a result, those estimates based on limited data may be very imprecise. Testing methods to gather such data are discussed below.

RAM Considerations during Systems Development

RAM are inherent product or system attributes that should be considered throughout the development lifecycle. Reliability standards, textbook authors, and others have proposed multiple development process models. (O'Connor 2014, Kapur 2014, Ebeling 2010, DoD 2005) The discussion in this section relies on a standard developed by a joint effort by the Electronic Industry Association and the U.S. Government and adopted by the U.S. Department of Defense (GEIA 2008) that defines 4 processes: understanding user requirements and constraints, design for reliability, production for reliability, and monitoring during operation and use (discussed in the next section).

Understanding User Requirements and Constraints

Understanding user requirements involves eliciting information about functional requirements, constraints (e.g., mass, power consumption, spatial footprint, life cycle cost), and needs that correspondent to RAM requirements. From these emerge system requirements that should include specifications for reliability, maintainability, and availability, and each should be conditioned on the projected operating environments. RAM requirements definition is as challenging but as essential to development success as the definition of general functional requirements.

Design for Reliability

System designs based on user requirements and system design alternatives can then be formulated and evaluated. Reliability engineering during this phase seeks to increase system robustness through measures such as redundancy, diversity, built-in testing, advanced diagnostics, and modularity to enable rapid physical replacement. In addition, it may be possible to reduce failure rates through measures such as use of higher strength materials, increasing the quality components, moderating extreme environmental conditions, or shortened maintenance, inspection, or overhaul intervals. Design analyses may include mechanical stress, corrosion, and radiation analyses for mechanical components, thermal analyses for mechanical and electrical components, and Electromagnetic Interference (EMI) analyses or measurements for electrical components and subsystems.

In most computer-based systems, hardware mean time between failures are hundreds of thousands of hours so that most system design measures to increase system reliability are focused on software. The most obvious way to improve software reliability is by improving its quality through more disciplined development efforts and tests. Methods for doing so are in the scope of software engineering but not in the scope of this section. However, reliability and availability can also be increased through architectural redundancy, independence, and diversity. Redundancy must be accompanied by measures to ensure data consistency, and managed failure detection and switchover. Within the software architecture, measures such as watchdog timers, flow control, data integrity checks (e.g., hashing or cyclic redundancy checks), input and output validity checking, retries, and restarts can increase reliability and failure detection coverage (Shooman 2002).

System RAM characteristics should be continuously evaluated as the design progresses. Where failure rates are not known (as is often the case for unique or custom developed components, assemblies, or software), developmental testing may be undertaken to assess the reliability of custom-developed components. Evaluations based on quantitative analyses assess the numerical reliability and availability of the system and are usually based on reliability block diagrams, fault trees, Markov models, and Petri nets. (O'Connor 2011) Markov models and Petri nets are of particular value for computer-based systems that use redundancy. Evaluations based on qualitative analyses assess vulnerability to single points of failure, failure containment, recovery, and maintainability. The primary qualitative methods are the failure mode effects and criticality analyses (FMECA). (Kececioglu 1991) The development program Discrepancy Reporting (DR) or Failure Reporting and Corrective Action System (FRACAS) should also be used to identify failure modes which may not have been anticipated by the FMECA and to identify common problems that can be corrected through an improved design or development process.

Analyses from related disciplines during design time also affect RAM. Human factor analyses are necessary to ensure that operators and maintainers can interact with the system in a manner that minimizes failures and the restoration times when they occur. There is also a strong link between RAM and cybersecurity in computer-based systems. On the one hand, defensive measures reduce the frequency of failures due to malicious events. On the other hand, devices such as firewalls, policy enforcement devices, and access/authentication servers (also known as "directory servers") can also become single points of failure or performance bottlenecks that reduce system reliability and availability.

Production for Reliability

Many production issues associated with RAM are related to quality. The most important of these are ensuring repeatability and uniformity of production processes and complete unambiguous specifications for items from the supply chain. Other are related to design for manufacturability, storage, and transportation. (Kapur 2014; Eberlin 2010) Large software intensive information systems are affected by issues related to configuration management, integration testing, and installation testing. Testing and recording of failures in the problem reporting and corrective action systems (PRACAS) or the FRACAS capture data on failures and improvements to correct failures. Depending on organizational considerations, this may be the same or a separate system as used during the design.

Monitoring During Operation and Use

After systems are fielded, their reliability and availability are monitored to assess whether the system or product has met its RAM objectives, identify unexpected failure modes, record fixes, and assess the utilization of maintenance resources and the operating environment. The FRACAS or a maintenance management database may be used for this purpose. In order to assess RAM, it is necessary to maintain an accurate record not only of failures but also of operating time and the duration of outages. Systems that report only on repair actions and outage incidents may not be sufficient for this purpose.

An organization should have an integrated data system that allows reliability data to be considered with logistical data, such as parts, personnel, tools, bays, transportation and evacuation, queues, and costs, allowing a total awareness of the interplay of logistical and RAM issues. These issues in turn must be integrated with management and operational systems to allow the organization to reap the benefits that can occur from complete situational awareness with respect to RAM.

Reliability and Maintainability Testing

Reliability Testing can be performed at the component, subsystem, and system level throughout the product or system lifecycle. Examples of hardware related categories of reliability testing are detailed in Ebeling (2010) and O'Connor (2014).

- **Reliability Life Tests:** Reliability life tests are used to empirically assess the time to failure for non-repairable products and systems and the times between failure for repairable or restorable systems. Termination criteria for such tests can be based on a planned duration or planned number of failures. Methods to account for “censoring” of the failures or the surviving units enable a more accurate estimate of reliability. (Meeker, Escobar, Pascual 2022)
 - **Accelerated Life Tests:** Accelerated life testing is performed by subjecting the items under test (usually electronic parts) to increased stress well above the expecting operating range and extrapolating results using model such as an Arrhenius relation (temperature acceleration), inverse power law (voltage), or a cumulative damage model (non-constant stress). (Meeker, Escobar, Pascual 2022)
 - **Highly Accelerated Life Testing/Highly Accelerated Stress Testing (HALT/HASS):** is performed by subjecting units under test (components or subassemblies) to extreme temperature and vibration tests with the objective of identifying failure modes, margins, and design weaknesses.
 - **Parts Screening:** Parts screening is not really a test but a procedure to operate components for a duration beyond the “infant mortality” period during which less durable items fail and the more durable parts that remain are then assembled into the final product or system. This is also known as “burn-in.”
 - **System Level Testing:** Examples of system level testing (including both hardware and software) are detailed in O'Connor (2014) and Ebeling (2010).
 - **Stability Tests:** Stability tests are life tests for integrated hardware and software systems. The goal of such testing is to determine the integrated system failure rate and assess operational suitability. Test conditions must include
-

accurate simulation of the operating environment (including workload) and a means of identifying and recording failures.

- **Reliability Growth Tests:** Reliability growth testing is part of a reliability growth program in which items are tested throughout the development and early production cycle with the intent of assessing reliability increases due to improvements in the manufacturing process (for hardware) or software quality (for software). Also known as "Test-Analyze-And-Fix (TAAF)." (NRC 2015)
- **Failure/Recovery Tests:** Such testing assesses the fault tolerance of a system by measuring probability of switchover for redundant systems. Failures are simulated and the ability of the hardware and software to detect the condition and reconfigure the system to remain operational are tested.
- **Maintainability Tests:** Such testing assesses the system diagnostics capabilities, physical accessibility, and maintainer training by simulating hardware or software failures that require maintainer action for restoration.

Because of its potential impact on cost and schedule, reliability testing should be coordinated with the overall system engineering effort. Test planning considerations include the number of test units, duration of the tests, environmental conditions, and the means of detecting failures.

Data Issues

True RAM models for a system are generally never known. Data on a given system is assumed or collected, used to select a distribution for a model, and then used to fit the parameters of the distribution. This process differs significantly from the one usually taught in an introductory statistics course.

First, the normal distribution is seldom used as a life distribution, since it is defined for all negative times. Second, and more importantly, reliability data is different from classic experimental data. Reliability data is often censored, biased, observational, and missing information about covariates such as environmental conditions. Data from testing is often expensive, resulting in small sample sizes. These problems with reliability data require sophisticated strategies and processes to mitigate them.

One consequence of these issues is that estimates based on limited data can be and usually are very imprecise.

Discipline Management

In most large programs, RAM experts report to the system engineering organization. At project or product conception, top level goals are defined for RAM based on operational needs, lifecycle cost projections, and warranty cost estimates. These lead to RAM derived requirements and allocations that are approved and managed by the system engineering requirements management function. RAM testing is coordinated with other product or system testing through the testing organization, and test failures are evaluated by the RAM function through joint meetings such as a Failure Review Board. In some cases, the RAM function may recommend design or development process changes as a result of evaluation of test results or software discrepancy reports, and these proposals must be adjudicated by the system engineering organization, or in some cases, the acquiring customer if cost increases are involved.

Post-Production Management Systems

Once a system is fielded, its reliability and availability should be tracked. Doing so allows the producer/owner to verify that the design has met its RAM objectives, to identify unexpected failure modes, to record fixes, to assess the utilization of maintenance resources, and to assess the operating environment.

One such tracking system is generically known as a FRACAS system (Failure Reporting and Corrective Action System). Such a system captures data on failures and improvements to correct failures. This database is separate from a warranty database, which is typically run by the financial function of an organization and tracks costs only.

A FRACAS for an organization is a system, and itself should be designed following systems engineering principles. In particular, a FRACAS system supports later analyses, and those analyses impose data requirements. Unfortunately, the lack of careful consideration of the backward flow from decision to analysis to model to required data too often leads to inadequate data collection systems and missing essential information. Proper prior planning prevents this poor performance.

Of particular importance is a plan to track data on units that have not failed. Units whose precise times of failure are unknown are referred to as censored units. Inexperienced analysts frequently do not know how to analyze censored data, and they omit the censored units as a result. This can bias an analysis.

An organization should have an integrated data system that allows reliability data to be considered with logistical data, such as parts, personnel, tools, bays, transportation and evacuation, queues, and costs, allowing a total awareness of the interplay of logistical and RAM issues. These issues in turn must be integrated with management and operational systems to allow the organization to reap the benefits that can occur from complete situational awareness with respect to RAM.

Discipline Relationships

Interactions

RAM interacts with nearly all aspects of the system development effort. Specific dependencies and interactions include:

- **Systems Engineering:** RAM interacts with systems engineering as described in the previous section.
- **Product Management (Life Cycle Cost and Warranty):** RAM interacts with the product or system lifecycle cost and warranty management organizations by assisting in the calculation of expected repair rates, downtimes, and warranty costs. RAM may work with those organizations to perform trade-off analyses to determine the most cost-efficient solution and to price service contracts.
- **Quality Assurance:** RAM may also interact with the procurement and quality assurance organizations with respect to selection and evaluation of materials, components, and subsystems.

Dependencies

- **Systems Safety:** RAM and system safety engineers have many common concerns with respect to managing the failure behavior of a system (i.e., single points of failure and failure propagation). RAM and safety engineers use similar analysis techniques, with safety being concerned about failures affecting life or unique property and RAM being concerned with those failures as well as lower severity events that disrupt operations. RAM and system safety are both concerned with failures occurring during development and test – FRACAS is the primary methodology used for RAM; hazard tracking is the methodology used for system safety.
 - **Cybersecurity:** In systems or products integrating computers and software, cybersecurity and RAM engineers have common concerns relating to the availability of cyber defenses and system event monitoring. However, there are also trade-offs with respect to access control, boundary devices, and authentication where security device failures could impact the availability of the product or system to users.
-

- **Software and Hardware Engineering:** Design and RAM engineers have a common goal of creating dependable products and systems. RAM interacts with the software and hardware reliability functions through design analyses such as failure modes and effects analyses, reliability predictions, thermal analyses, reliability measurement, and component specific analyses. RAM may recommend design changes as a result of these analyses that may have to be adjudicated by program management, the customer, or systems engineering if there are cost or schedule impacts.
- **Testing:** RAM interacts with the testing program during planning to assess the most efficient (or feasible) test events to perform life testing, failure/recovery testing, and stability testing as well as to coordinate requirements for reliability or stress tests. RAM also interacts with the testing organization to assess test results and analyze failures for the implications on product or system RAM.
- **Logistics:** RAM works with logistics in providing expected failure rates and downtime constraints in order for logistics engineers to determine staffing, sparing, and special maintenance equipment requirements.

Discipline Standards

Because of the importance of reliability, availability, and maintainability, as well as related attributes, there are hundreds of standards associated. Some are general but more are specific to domains such as automotive, aviation, electric power distribution, nuclear energy, rail transportation, software, etc. Standards are produced by both governmental agencies, professional associations and international standards bodies such as:

- The International Electrotechnical Commission (IEC), Geneva, Switzerland and the closely associated International Organization for Standardization (ISO)
- The Institute of Electrical and Electronic Engineers (IEEE), New York, NY, USA
- The Society of Automotive Engineers (SAE), Warrendale, PA, USA
- Governmental Agencies – primarily in military and space systems

The following table lists selected standards from each of these agencies. Because of differences in domains and because many standards handle the same topic in slightly different ways, selection of the appropriate standards requires consideration of previous practices (often documented as contractual requirements), domain specific considerations, certification agency requirements, end user requirements (if different from the acquisition or producing organization), and product or system characteristics.

Table 1. Selected Reliability, Availability, Maintainability standards (SEBoK Original)

Organization	Number, Title, and Year	Domain	Comment
IEC	IEC 60812, Analysis techniques for system reliability - Procedure for failure mode and effects analysis (FMEA), 2006	General	
IEC	IEC 61703 Ed 2.0, Mathematical expressions for reliability, availability, maintainability and maintenance support terms [1], 2016	General	
IEC	IEC 62308, Equipment reliability - Reliability assessment methods, 2006	General	
IEC	IEC 62347, Guidance on system dependability specifications, 2006	General	
IEC	IEC 62278, Railway applications – Specification and demonstration of reliability, availability, maintainability and safety (RAMS), 2002	Railways	

IEEE	IEEE Std 352-2016, IEEE Guide for General Principles of Reliability Analysis of Nuclear Power Generating Station Safety Systems and Other Nuclear Facilities ^[2] , 2016	Nuclear Energy	
IEEE	IEEE Std 1044-2009, IEEE Standard Classification for Software Anomalies, 2009	Software	
IEEE	IEEE Std 1633-2008, IEEE Recommended Practice on Software Reliability ^[3] , 2016	Software	
SAE	ARP 4754A, Guidelines for the Development of Civil Aircraft and Systems, 2010	Aviation	
SAE	ARP 5890B, Guidelines for Preparing Reliability Assessment Plans for Electronic Engine Controls ^[4] , 2018	Aviation	
SAE	J2940_202002, Use of Model Verification and Validation in Product Reliability and Confidence Assessments ^[5] , 2020	General	
SAE	SAE-GEIA-STD-0009A, Reliability Program Standard for Systems Design, Development, and Manufacturing ^[6] , 2020	General	Used by the U.S. Dept. of Defense as the primary reliability standard (replaces MIL-STD-785B)
SAE	JA 1002_201205, Software Reliability Program Standard ^[7] , 2012	Software	
U.S. Government	NASA-STD-8729.1A, Planning, Developing and Managing an Effective Reliability And Maintainability (R&M) Program ^[8] , 2017	Space Systems	
U.S. Government	MIL HDBK 470A, Designing and Developing Maintainable Products and Systems ^[9] , 1997	Defense Systems	
U.S. Government	MIL HDBK 217F (Notice 2), Reliability Prediction of Electronic Equipment ^[10] , 1995	Defense Systems	Although formally titled a “Handbook” and more than 2 decades old, the values and methods constitute a de facto standard for some U.S. military acquisitions
U.S. Government	MIL-STD-1629A (Notice 3), Procedures for Performing a Failure Mode Effects and Criticality Analysis -Revision A ^[11] , 1998		The parent of FMEA standards produced by the IEEE, SAE, ISO, and many other agencies. Still valid and in use after 4 decades.

Personnel Considerations

Becoming a reliability engineer requires education in probability and statistics as well as the specific engineering domain of the product or system under development or in operation. A number of universities throughout the world have departments of reliability engineering (which also address maintainability and availability) and more have research groups and courses in reliability and safety – often within the context of another discipline such as computer science, systems engineering, civil engineering, mechanical engineering, or bioengineering. Because most academic engineering programs do not have a full reliability department, most engineers working in reliability have been educated in other disciplines and acquire the additional skills through additional coursework or by working with other qualified engineers. A certification in reliability engineering ^[12] is available from the American Society for Quality (ASQ 2016). However, only a minority of engineers working in the discipline have this certification.

Metrics

The three basic metrics of RAM are (not surprisingly) Reliability, Maintainability, and Availability. Reliability can be characterized in terms of the parameters, mean, or any percentile of a reliability distribution. However, in most cases, the exponential distribution is used, and a single value, the mean time to failure (MTTF) for non-restorable systems, or mean time between failures (MTBF for restorable systems are used). The metric is defined as:

where T is the total operating time and F is the number of failures.

Maintainability is often characterized in terms of the exponential distribution and the mean time to repair and be similarly calculated, i.e.,

Where T_d is the total down time and O is the number of outages.

As was noted above, accounting for downtime requires definitions and specificity. Down time might be counted only for corrective maintenance actions, or it may include both corrective and preventive maintenance actions. Where the lognormal rather than the exponential distribution is used, a mean down time can still be calculated, but both the log of the downtimes and the variance must be known in order to fully characterize maintainability. Availability can be calculated from the total operating time and the downtime, or in the alternative, as a function of MTBF and MTTR (Mean Time To Repair.)

As was the case with maintainability, availability may be qualified as to whether it includes only unplanned failures and repairs (inherent availability) or downtime due to all causes including administrative delays, staffing outages, or spares inventory deficiencies (operational availability).

Probabilistic metrics describe system performance for RAM. Quantiles, means, and modes of the distributions used to model RAM are also useful.

Availability has some additional definitions, characterizing what downtime is counted against a system. For **inherent availability**, only downtime associated with corrective maintenance counts against the system. For **achieved availability**, downtime associated with both corrective and preventive maintenance counts against a system. Finally, **operational availability** counts all sources of downtime, including logistical and administrative, against a system.

Availability can also be calculated instantaneously, averaged over an interval, or reported as an asymptotic value. **Asymptotic availability** can be calculated easily, but care must be taken to analyze whether or not a system settles down or settles up to the asymptotic value, as well as how long it takes until the system approaches that asymptotic value.

Reliability importance measures the effect on the system reliability of a small improvement in a component's reliability. It is defined as the partial derivative of the system reliability with respect to the reliability of a component.

Criticality is the product of a component's reliability, the consequences of a component failure, and the frequency with which a component failure results in a system failure. Criticality is a guide to prioritizing reliability improvement efforts.

Many of these metrics cannot be calculated directly because the integrals involved are intractable. They are usually estimated using simulation.

Models

There are a wide range of models that estimate and predict reliability (Meeker, Escobar, Pascual 2022). Simple models, such as exponential distribution, can be useful for “back of the envelope” calculations.

System models are used to (1) combine probabilities or their surrogates, failure rates and restoration times, at the component level to find a system level probability or (2) to evaluate a system for maintainability, single points of failure, and failure propagation. The three most common are reliability block diagrams, fault trees, and failure modes and effects analyses.

There are more sophisticated probability models used for life data analysis. These are best characterized by their failure rate behavior, which is defined as the probability that a unit fails in the next small interval of time, given it has lived until the beginning of the interval, and divided by the length of the interval.

Models can be considered for a fixed environmental condition. They can also be extended to include the effect of environmental conditions on system life. Such extended models can in turn be used for accelerated life testing (ALT), where a system is deliberately and carefully overstressed to induce failures more quickly. The data is then extrapolated to usual use conditions. This is often the only way to obtain estimates of the life of highly reliable products in a reasonable amount of time. (Nelson 1990)

Also useful are **degradation models**, where some characteristic of the system is associated with the propensity of the unit to fail (Nelson 1990). As that characteristic degrades, we can estimate times of failure before they occur.

The initial developmental units of a system often do not meet their RAM specifications. **Reliability growth models** allow estimation of resources (particularly testing time) necessary before a system will mature to meet those goals. (Meeker, Escobar, and Pascual 2022, NRC 2015)

Maintainability models describe the time necessary to return a failed repairable system to service. They are usually the sum of a set of models describing different aspects of the maintenance process (e.g., diagnosis, repair, inspection, reporting, and evacuation). These models often have threshold parameters, which are minimum times until an event can occur.

Logistical support models attempt to describe flows through a logistics system and quantify the interaction between maintenance activities and the resources available to support those activities. Queue delays, in particular, are a major source of down time for a repairable system. A logistical support model allows one to explore the trade space between resources and availability.

All these models are abstractions of reality, and so at best approximations to reality. To the extent they provide useful insights, they are still very valuable. The more complicated the model, the more data necessary to estimate it precisely. The greater the extrapolation required for a prediction, the greater the imprecision.

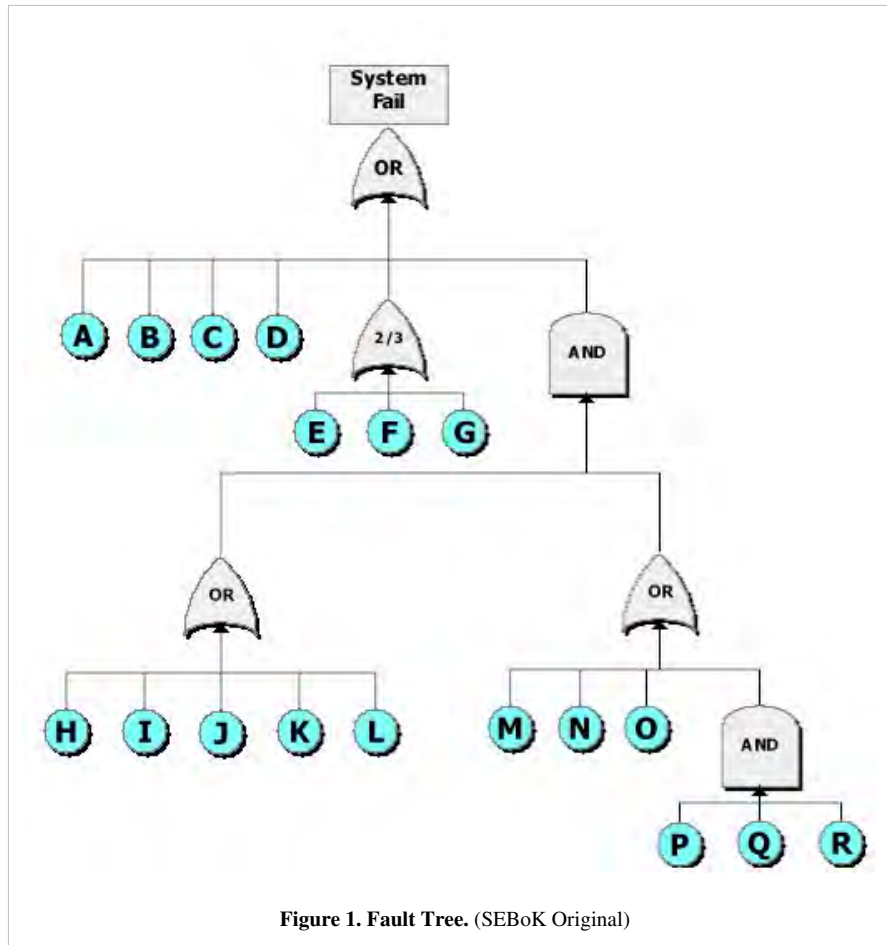
Extrapolation is often unavoidable, because high reliability equipment typically can have long life and the amount of time required to observe failures may exceed test times. This requires strong assumptions be made about future life (such as the absence of masked failure modes) and that these assumptions increase uncertainty about predictions. The uncertainty introduced by strong model assumptions is often not quantified and presents an unavoidable risk to the system engineer.

There are many ways to characterize the reliability of a system, including fault trees, reliability block diagrams, and failure mode effects analysis.

A **Fault Tree** (Kececioglu 1991) is a graphical representation of the failure modes of a system. It is constructed using logical gates, with AND, OR, NOT, and K of N gates predominating. Fault trees can be complete or partial; a partial fault tree focuses on a failure mode or modes of interest. They allow “drill down” to see the dependencies of systems on nested systems and system elements. Fault trees were pioneered by Bell Labs in the 1960s.

A Failure Mode Effects Analysis is a table that lists the possible failure modes for a system, their likelihood, and the effects of the failure. A Failure Modes Effects Criticality Analysis scores the effects by the magnitude of the product

of the consequence and likelihood, allowing ranking of the severity of failure modes. (Kececioglu 1991)



A **Reliability Block Diagram** (RBD) (DOD, 1998) is a graphical representation of the reliability dependence of a system on its components. It is a directed, acyclic graph. Each path through the graph represents a subset of system components. As long as the components in that path are operational, the system is operational. Component lives are usually assumed to be independent in an RBD. Simple topologies include a series system, a parallel system, a k of n system, and combinations of these.

RBDs are often nested, with one RBD serving as a component in a higher-level model. These hierarchical models allow the analyst to have the appropriate resolution of detail while still permitting abstraction.

RBDs depict paths that lead to success, while fault trees depict paths that lead to failure.

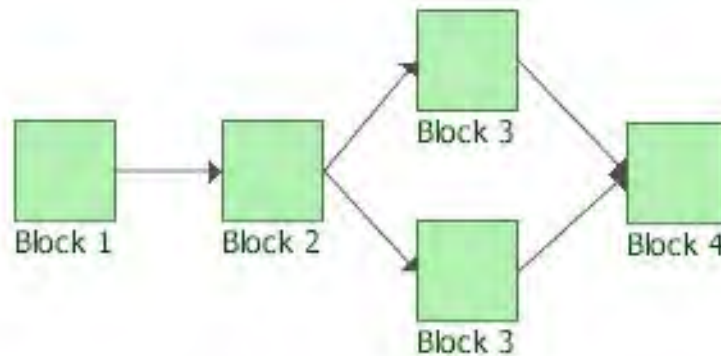


Figure 2. Simple Reliability Block Diagram. (SEBoK Original)

A **Failure Mode Effects Analysis** is a table that lists the possible failure modes for a system, their likelihood, and the effects of the failure. A **Failure Modes Effects Criticality Analysis** scores the effects by the magnitude of the product of the consequence and likelihood, allowing ranking of the severity of failure modes. (Kececioglu 1991)

System models require even more data to fit them well. “Garbage in, garbage out” (GIGO) particularly applies in the case of system models.

Tools

The specialized analyses required for RAM drive the need for specialized software. While general purpose statistical languages or spreadsheets can, with sufficient effort, be used for reliability analysis, almost every serious practitioner uses specialized software.

Minitab (versions 13 and later) includes functions for life data analysis. Win Smith is a specialized package that fits reliability models to life data and can be extended for reliability growth analysis and other analyses. Relex has an extensive historical database of component reliability data and is useful for estimating system reliability in the design phase.

There is also a suite of products from ReliaSoft (2007) that is useful in specialized analyses. Weibull++ fits life models to life data. ALTA fits accelerated life models to accelerated life test data. BlockSim models system reliability, given component data.

Discipline Specific Tool Families

Reliasoft^[13] and PTC Windchill Product Risk and Reliability^[14] produce a comprehensive family of tools for component reliability prediction, system reliability predictions (both reliability block diagrams and fault trees), reliability growth analysis, failure modes and effects analyses, FRACAS databases, and other specialized analyses. In addition to these comprehensive tool families, there are more narrowly scoped tools. Minitab (versions 13 and later) includes functions for life data analysis.

General Purpose Statistical Analysis Software with Reliability Support

Some general-purpose statistical analysis software includes functions for reliability data analysis. Minitab ^[15] has a module for reliability and survival analysis. SuperSmith ^[16] is a more specialized package that fits reliability models to life data and can be extended for reliability growth analysis and other analyses.

R ^[17] is a widely used open source and well-supported general purpose statistical language with specialized packages that can be used for fitting reliability models, Bayesian analysis, and Markov modeling.

Special Purpose Analysis Tools

Fault tree generation and analysis tools include CAFTA ^[18] from the Electric Power Research Institute and OpenFTA ^[19], an open source software tool originally developed by Auvation Software.

PRISM ^[20] is an open source probabilistic model checker that can be used for Markov modeling (both continuous and discrete time) as well as for more elaborate analyses of system (more specifically, “timed automata”) behaviors such as communication protocols with uncertainty.

References

Works Cited

- American Society for Quality (ASQ). 2022. *Glossary: Reliability*. Accessed on May 9, 2022. Available at <https://asq.org/glossary/r.html>.
- American Society for Quality (ASQ). 2016. *Reliability Engineering Certification – CRE*. Accessed May 9, 2022. Available at <https://asq.org/cert/reliability-engineer>.
- DoD. 2005. *DOD Guide for Achieving Reliability, Availability, and Maintainability*. Arlington, VA, USA: U.S. Department of Defense (DoD). Accessed on May 9, 2022. Available at [https://www.acqnotes.com/Attachments/DoD%20Reliability%20Availability%20and%20Maintainability%20\(RAM\)%20Guide.pdf](https://www.acqnotes.com/Attachments/DoD%20Reliability%20Availability%20and%20Maintainability%20(RAM)%20Guide.pdf)
- Ebeling, C.E., 2010. *An Introduction to Reliability and Maintainability Engineering*. Long Grove Illinois, U.S.A: Waveland Press.
- GEIA. 2008. *Reliability Program Standard for Systems Design, Development, and Manufacturing*. Warrendale, PA, USA: Society of Automotive Engineers (SAE), SAE-GEIA-STD-0009.
- IEEE. 2008. *IEEE Recommended Practice on Software Reliability*. New York, NY, USA: Institute of Electrical and Electronic Engineers (IEEE). IEEE Std 1633-2008.
- Kececioglu, D. 1991. *Reliability Engineering Handbook*, Volume 2. Upper Saddle River, NJ, USA: Prentice Hall.
- Laprie, J.C., A. Avizienis, and B. Randell. 1992. *Dependability: Basic Concepts and Terminology*. Vienna, Austria: Springer-Verlag.
- Meeker, W., Escobar, L., and Pascual, F. 2022 *Statistical Methods for Reliability Data*. 2nd ed. Hoboken, NJ: Wiley.
- Nelson, W. 1990. *Accelerated Testing: Statistical Models, Test Plans, and Data Analysis*. New York, NY, USA: Wiley and Sons.
- NRC. 2015. *Reliability Growth: Enhancing Defense System Reliability*. Washington, DC: National Academy Press. Access May 25, 2023. Available at <https://nap.nationalacademies.org/catalog/18987/reliability-growth-enhancing-defense-system-reliability>.
- O'Connor, D.T., and A. Kleyner. 2012. *Practical Reliability Engineering*, 5th Edition. Chichester, UK: J. Wiley & Sons, Ltd.
- ReliaSoft. 2007. *Failure Modes and Effects Analysis (FMEA) and Failure Modes, Effects and Criticality Analysis (FMECA)*. Accessed May 9, 2022. Available at <https://www.weibull.com/basics/fmea.htm>.

Shooman, Martin. 2002. *Reliability of Computer Systems and Networks*. New York, NY, USA: John Wiley & Sons.

Primary References

Blischke, W.R. and D.N. Prabhakar Murthy. 2000. *Reliability Modeling, Prediction, and Optimization*. New York, NY, USA: Wiley and Sons.

Dezfuli, H, D. Kelly, C. Smith, K. Vedros, and W. Galyean. 2009. "Bayesian Inference for NASA Risk and Reliability Analysis", National Aeronautics and Space Administration, NASA/SP-2009-569. Accessed on May 25, 2023. Available at <https://ntrs.nasa.gov/citations/20090023159>.

DoD. 2005. *DOD Guide for Achieving Reliability, Availability, and Maintainability*. Arlington, VA, USA: U.S. Department of Defense (DoD). Accessed September 11, 2011. Available at https://www.acq.osd.mil/se/docs/ram_guide_080305.pdf.

Kececioglu, D. 1991. *Reliability Engineering Handbook*, Volume 2. Upper Saddle River, NJ, USA: Prentice Hall.

Lawless, J.F. 1982. *Statistical Models and Methods for Lifetime Data*. New York, NY, USA: Wiley and Sons.

Lyu, M. 1996. "Software Reliability Engineering". New York, NY: IEEE-Wiley Press. Accessed May 9, 2022. Available at <https://www.cse.cuhk.edu.hk/~lyu/book/reliability/index.html>.

Martz, H.F. and R.A. Waller. 1991. *Bayesian Reliability Analysis*. Malabar, FL, USA: Kreiger.

Meeker, W., Escobar, L., and Pascual, F. 2022 *Statistical Methods for Reliability Data* ^[21]. 2nd ed. Hoboken, NJ: Wiley.

NRC. 2015. Reliability Growth: Enhancing Defense System Reliability ^[22]. Washington, DC: National Academy Press.

DoD. 2011. "MIL-HDBK-189C, Department of Defense Handbook: Reliability Growth Management (14 JUN 2011)." Arlington, VA, USA: U.S. Department of Defense (DoD). Accessed May 9, 2022. Available at https://everyspec.com/MIL-HDBK/MIL-HDBK-0099-0199/MIL-HDBK-189C_34842.

DOD. 1998. "MIL-HDBK-338B, Electronic Reliability Design Handbook" U.S. Department of Defense Air Force Research Laboratory IFTB. Accessed May 9, 2022. Available at https://www.weibull.com/mil_std/mil_hdbk_338b.pdf.

U.S. Naval Surface Weapons Center Carderock Division, NSWC-11. "Handbook of Reliability Prediction Procedures for Mechanical Equipment." Accessed May 9, 2022. Available at https://everyspec.com/USN/NSWC/download.php?spec=NSWC-10_RELIABILITY_HDBK_JAN2010.045818.pdf.

Additional References

IEEE. 2013. *IEEE Recommended Practice for Collecting Data for Use in Reliability, Availability, and Maintainability Assessments of Industrial and Commercial Power Systems*, IEEE Std 3006.9-2013. New York, NY, USA: IEEE.

NIST/SEMATECH Engineering Statistics Handbook 2013. Accessed May 9, 2022. Available at <https://www.itl.nist.gov/div898/handbook/>.

Olwell, D.H. 2001. "Reliability Leadership." *Proceedings of the 2001 IEEE Reliability and Maintainability Symposium*. Philadelphia, PA, USA: IEEE. Accessed May 9, 2022. Available at <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=902431>.

Reliability Analytics Toolkit. n.d. web page containing 31 reliability and statistical analyses calculation aids. Seymour Morris, Reliability Analytics. Accessed May 9, 2022. Available at <https://reliabilityanalyticstoolkit.appspot.com/>

ReliaSoft. 2007. "Availability." Accessed May 9, 2022. Available at https://reliawiki.com/index.php/Introduction_to_Repairable_Systems#Availability.

SAE. 2000a. *Aerospace Recommended Practice ARP5580: Recommended Failure Modes and Effects Analysis (FMEA) Practices for Non-Automobile Applications*. Warrendale, PA, USA: Society of Automotive Engineers (SAE) International.

SAE. 2000b. *Surface Vehicle Recommended Practice J1739: (R) Potential Failure Mode and Effects Analysis in Design (Design FMEA), Potential Failure Mode and Effects Analysis in Manufacturing and Assembly Processes (Process FMEA), and Potential Failure Mode and Effects Analysis for Machinery (Machinery FMEA)*. Warrendale, PA, USA: Society of Automotive Engineers (SAE) International.

Relevant Videos

- Availability ^[23]
- Reliability, Availability ^[24]

References

- [1] <https://webstore.ansi.org/Standards/IEC/IEC61703Ed2016>
- [2] <https://standards.ieee.org/ieee/352/5453/>
- [3] <https://standards.ieee.org/ieee/1633/5726/>
- [4] <https://www.sae.org/standards/content/arp5890b>
- [5] https://www.sae.org/standards/content/j2940_202002
- [6] <https://www.sae.org/standards/content/geiastd0009a>
- [7] https://www.sae.org/standards/content/ja1002_201205
- [8] <https://s3vi.ndc.nasa.gov/ssri-kb/static/resources/nasa-std-8729.1a.pdf>
- [9] <https://www.acqnotes.com/Attachments/MIL-HDBK-470A.pdf>
- [10] https://everyspec.com/MIL-HDBK/MIL-HDBK-0200-0299/download.php?spec=MIL-HDBK-217F_NOTICE-2.014590.PDF
- [11] https://everyspec.com/MIL-STD/MIL-STD-1600-1699/download.php?spec=MIL-STD-1629_NOTICE-3.023100.pdf
- [12] <https://asq.org/cert/reliability-engineer>
- [13] <https://www.reliasoft.com/products.htm>
- [14] <https://www.ptc.com/product-lifecycle-management/windchill/product-risk-and-reliability>
- [15] <https://www.minitab.com/en-us/products/minitab/look-inside/>
- [16] <https://www.barringer1.com/wins.htm>
- [17] <https://www.r-project.org/>
- [18] <https://teams.epri.com/RR/News%20Archives/CAFTAFactSheet.pdf>
- [19] <https://www.openfta.com/>
- [20] <https://www.prismmodelchecker.org/>
- [21] <https://www.wiley.com/en-us/Statistical+Methods+for+Reliability+Data%2C+2nd+Edition-p-9781118115459>
- [22] <https://nap.nationalacademies.org/catalog/18987/reliability-growth-enhancing-defense-system-reliability>
- [23] <https://www.youtube.com/watch?v=ik-rWLnajY>
- [24] <https://www.youtube.com/watch?v=jznHA-C07dg>

System Resilience

- Lead Author:
- John Brtis
- Contributing Authors:
- Ken Cureton, Scott Jackson, and Ivan Taylor

Resilience is a relatively new term in the SE realm, appearing around 2006 and becoming popularized in 2010. The application of “resilience” to engineered systems has led to a proliferation of alternative definitions. While the details of definitions will continue to be debated, the information here should provide a working understanding of the meaning and implementation of resilience, sufficient for an engineer to address it effectively.

Overview

Definition

According to the Oxford English Dictionary, resilience is “the act of rebounding or springing back” (OED 1973). This definition fits materials that return to their original shape after deformation. For human-made or **engineered systems**, the definition of **resilience** can be extended to “maintaining **capability** in the face of an **adversity**.”

Some practitioners limit the definition of resilience to only the system reactions following an encounter with an adversity in what is known as the “reactive perspective” regarding system resilience. The “proactive perspective” defines system resilience to include actions that occur *before* encountering adversity as well. The INCOSE Resilient Systems Working Group (RSWG) asserts that resilience should address both the reactive and proactive perspectives.

The RSWG defines resilience as “the ability to provide required capability when facing adversity”, as depicted in **Figure 1**.

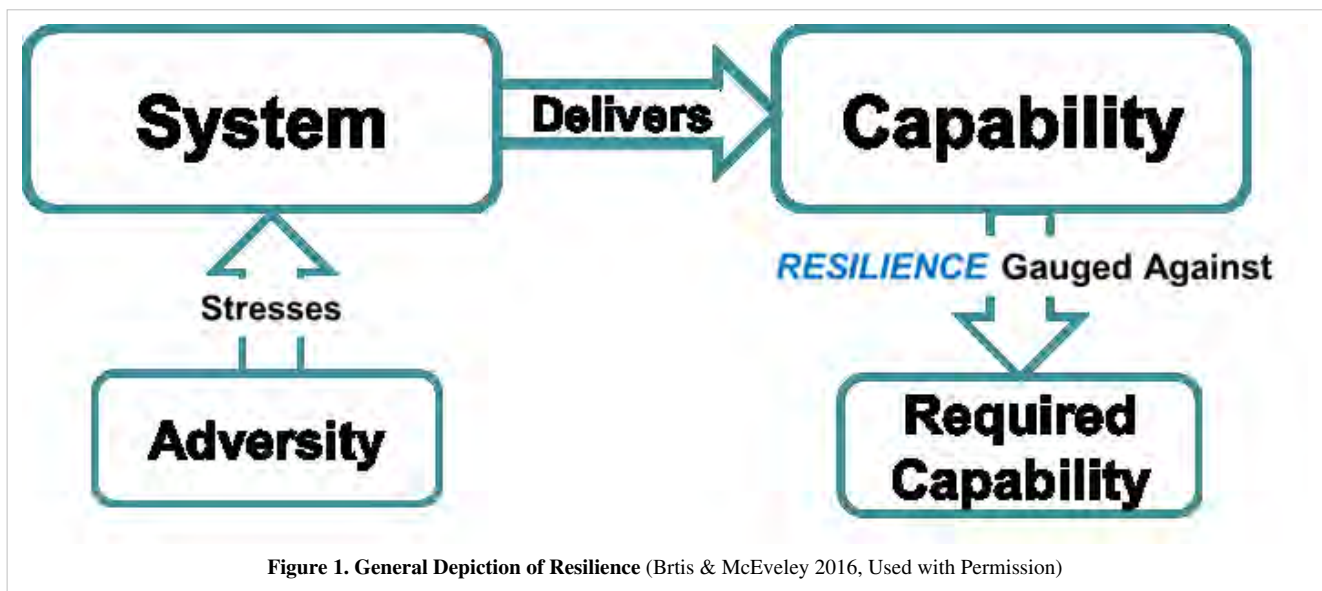


Figure 1. General Depiction of Resilience (Brtis & McEveley 2016, Used with Permission)

Scope of the Means

In applying this definition, one needs to consider the range of means by which resilience is achieved. The high-level means, and fundamental objectives of resilience include avoiding, withstanding, and recovering from adversity (Brtis and McEveley 2019). “Withstanding” and “recovering” from adversity are classic aspects of resilience. For the purpose of engineered systems, “avoiding” adversity is also an important means of achieving resilience (Jackson and Ferris 2016). Finer granularity low-level means of achieving resilience are discussed in the taxonomy portion of this

article.

Scope of the Adversity

An adversity is any condition that could degrade the delivered capability of a system. When discussing system resilience, the full spectrum of sources and types of adversity should be considered. For example, adversity may come from within the system or from the system's environment, and can be expected or unexpected. Adversity may also come from opponents, friendlies or neutral parties, and if from human sources, can be malicious or accidental (Brtis & McEvilly 2020).

"Stress" and "strain" are two useful concepts for understanding adversities. Adversities may affect the system through a causal chain of adversities. The proximate adversities that affect the system directly (rather than indirectly) are called "stresses" on the system. The effects on the system are referred to as "strains" on the system.

It is important to recognize that risks that can impact a system in the future can cause a detrimental **strain** on the system in the present. Systems should be designed to be resilient to future risks, just as they are to current issues.

Taxonomy for Achieving Resilience

A taxonomy describing the fundamental objectives of resilience and the means for achieving those objectives is valuable to the engineer developing a resilient design. It is important to distinguish "fundamental objectives" from "means objectives" and their impact on trades and engineering decision-making (Clemen & Reilly 2001; Keeney 1992).

A three-layer objectives-based taxonomy that implements this distinction is provided below. The first level addresses the fundamental objectives of resilience, the second level addresses the means objectives of resilience, and the third level addresses architecture, design, and operational techniques for achieving resilience. The three layers are related by many-to-many relationships. The terms and their descriptions are, in many cases, tailored to best address the context of resilience. (Most of the taxonomy content came from Brtis (2016), Jackson and Ferris (2013), Winstead (2020), with other content developed over time by the RSWG.)

Taxonomy Layer 1: The Fundamental Objectives of Resilience

Fundamental objectives are the first level decomposition of resilience objectives. They establish the scope of resilience. They identify the values pursued by resilience. They represent an extension of the definition of resilience. They are ends in themselves rather than just means to other ends. They should be relatively immutable. Being resilient means achieving three fundamental objectives:

- **Avoid adversity:** eliminate or reduce exposure to **stress**
- **Withstand adversity:** resist capability degradation when stressed
- **Recover from adversity:** replenish lost capability after degradation

These *fundamental* objectives can be achieved by pursuing *means* objectives. Means objectives are not ends in themselves. Their value resides in helping to achieve the three fundamental objectives.

Taxonomy Layer 2: Means Objectives

Means objectives are not ends in themselves but enable achieving the fundamental objectives in Layer 1. This level tends to reflect the stakeholder perspective. The means objectives are high-level functional requirements that should be considered during the stakeholder needs and requirements definition process. The definitions and descriptions are specific to how each concept is applied for resilience.

- **adapt:** be or become able to deliver required capability in changing situations. This allows systems to function in changing conditions. Adaptability address known-unknowns, and potentially, unknown-unknowns. It may be real-time, near-term, or long-term. The system may adapt itself or be adapted by external actors. The driving situation changes may be in the adversities, environment, system state, required capability, mission, business, stakeholders, stakeholder needs and requirements, system requirements, system architecture, or design. Adaptability is related to **flexibility**, **agility**, and evolution, and serves as a complement to the “preserve integrity” objective. (See the entry for “evolve” below as well.) Examples of adaptability include adding a crumple zone to an automobile, streaming services shifting to different modalities when faced by unexpectedly high loads or anomalies, self-healing systems, etc.
- **anticipate:** (1) consider and understand potential adversities, their consequences, and appropriate responses -- before the system is stressed; and/or (2) develop and maintain courses of design and operation that address predicted adversity. Anticipation facilitates the mitigation of risks before they manifest. The “anticipate” objective is similar to the “prepare” objective. Examples include use of prognostic data, predictive maintenance, hurricane early warning systems, and emergency response planning.
- **constrain:** withstand stress by limiting damage propagation within the system. Examples include fault containment, circuit breakers, fire walls, multiple independent levels of security, vaccines, amputation, etc.
- **continue:** withstand and recover from stress to deliver required capability, while and after being stressed. Examples include fail-safe, backup capability, redundant capability, fault tolerance, rapid recovery, etc.
- **degrade gracefully:** withstand stress by transitioning to desirable system state(s) after degradation. This allows partial useful functionality instead of complete system failure. Examples include automobile crumple zone absorption of kinetic energy, deorbiting a satellite, safety system engagement, and failing over to backup systems.
- **disaggregate:** disperse functions, systems, or system elements. This eliminates a single point of failure and eliminates a single target for opponents. Examples include processor load distribution, mechanical/electrical/thermal load distribution, networks, the nuclear triad, etc.
- **evade:** avoid adversity through design or action. Examples include stealth and maneuver.
- **evolve:** adapt over time. Examples include: B-52 transition to multiple roles, upgrading software-defined radio capabilities, and adoption of fuel injection to address fuel economy needs.
- **fortify:** strengthen the system to withstand stress. Examples include shielding, strengthening weakest “links,” and protective aircraft cockpits.
- **manage complexity:** reduce adversity or its effects by eliminating unnecessary complexity and the resulting unintended consequences. Unintended consequences can be or can create an adversity, or they may cause an inappropriate response to adversity. Examples include the principle of “Keep it Simple Stupid” and suppressing undesirable emergent behavior.
- **minimize adversity:** avoid adversity by reducing the amount or effectiveness of adversities. Examples include battlefield preparation, and fighting an infection (biological).
- **minimize faults:** avoid adversity by reducing the likelihood and severity of adversities arising within the system, such as by using system elements with a long mean time to failure.
- **monitor:** observe and evaluate changes or developments that could lead to degradation. This allows projection and anticipation of future status, to allow early detection and early response. An example would be smart grid real-time power monitoring.
- **preserve integrity:** withstand adversity by remaining complete and unaltered. This is a compliment to “adapt/evolve.” Examples include blockchain, which provides tamper-proof data logging and cryptographic digital

signatures.

- **prevent:** avoid degradation by precluding the realization of strain on the system. Examples include backup power to serve critical system capability, anti-corrosion coatings, etc.
- **reduce vulnerability:** better withstand adversity by identifying system vulnerabilities and modifying the system to reduce the degradation caused by adversity. Examples include performing and acting on Failure Modes and Effects Analysis (FMEA), reducing the attack surface, hardening vulnerable components, and flood-resistant urban infrastructures.
- **repair:** recover by fixing damage.
- **replace:** recover by substituting a capable element for a degraded one.
- **tolerate:** withstand degraded capability. Examples include excess margin, RAID storage, an aircraft hydraulic redundancy.
- **understand:** develop and maintain useful representations of required system capabilities, how those capabilities are generated, the system environment, and the potential for degradation due to adversity. Examples include digital twins, and battlefield simulations.

Taxonomy Layer 3: Architecture, Design, and Operational Techniques to Achieve Resilience Objectives

Architecture, design, and operational techniques that may achieve resilience objectives are listed below. This level tends to represent the system viewpoint, and should be considered during the system requirements, system architecture, design definition, and operation processes. The definitions and descriptions are specific to how each concept is applied for resilience.

- **absorption:** withstanding adversity by assimilating stress without unacceptable degradation of the system capability. Examples include dissipating kinetic energy with automobile crumple zones.
- **adaptability:** see “adapt” in layer 2 above. Adaptability has commonality with flexibility, agility, improvising, overcoming.
- **anomaly detection:** discovering salient abnormalities in the system or its environment to enable effective response.
- **backward recovery:** recovery to a previous stable state of a system.
- **boundary enforcement:** implementing the process, temporal, and spatial limits to protect the system.
- **buffering:** reducing the effect of degradation through excess capacity.
- **coordinated defense:** having multiple, synergistic mechanisms to protect required capability, such as via the defense-in-depth strategy.
- **deception:** confusing and thus impeding an adversary.
- **defense-in-depth:** minimizing loss by employing multiple coordinated mechanisms, each of which can at least partially achieve resilience.
- **detection avoidance:** reducing an adversary's awareness of the system. Examples include stealth.
- **distributed privilege:** requiring multiple authorized entities to act in a coordinated manner before a system function can proceed. Examples include authorization for use of nuclear weapons.
- **distribution:** spreading the system's ability to perform – physically or virtually.
- **diversification:** use of heterogeneous design techniques to minimize common vulnerabilities and common mode failures. Examples include heterogeneous technologies, data sources, processing locations, equipment locations, supply chains, communications paths.
- **domain separation:** physically or logically isolating items with different protection needs.
- **drift correction:** monitoring the system's movement toward the boundaries of proper operation and taking corrective action.
- **dynamic positioning:** relocation of system functionality or components to confound opponent understanding of the system.

- **effect tolerance:** providing required capability despite damage to the system.
 - **error recovery:** detection, control, and correction of internal errors.
 - **fail soft:** prioritizing and gradually terminating affected functions when failure is imminent.
 - **fault tolerance:** providing required capability in spite of faults.
 - **forward recovery:** recovery by restoring the system to a new, not previously occupied state in which it can perform required functions.
 - **human participation:** including people as part of the system. A human in the loop brings a unique capability for agile and adaptable thinking and action.
 - **least functionality:** system elements accomplish their required functions, but no more.
 - **least persistence:** system elements are available, accessible, and fulfill their design intent only while needed.
 - **least privilege:** system elements are allocated authorizations necessary to accomplish their specified functions, but not more.
 - **least sharing:** system resources are accessible by multiple system elements only when necessary, and to as few system elements as possible.
 - **loose coupling:** minimizing the interdependency of elements. This can reduce the potential for propagation of damage.
 - **loss margins:** excess capability, so partial capability degradation is acceptable.
 - **maintainability:** ability to be retained or restored to perform as required.
 - **mediated access:** controlling the ability to access and use system elements.
 - **modeling and analytic monitoring:** developing a representation of the system; and gathering, and analyzing data based on that understanding, to identify vulnerabilities, find indications of potential or actual adverse conditions, identify potential or actual system degradation and evaluate the efficacy of system countermeasures.
 - **modularity:** composing a system of discrete elements so that a change to one component has a minimal impact on other elements.
 - **neutral state:** providing a condition of the system where stakeholders (especially operators) can safely take no action while awaiting determination of the most appropriate action.
 - **protection:** mitigation of stress to the system.
 - **protective defaults:** providing default configurations of the system that provide protection.
 - **protective failure:** ensuring that failure of a system element does not result in an unacceptable loss of capability.
 - **protective recovery:** ensuring that recovery of a system element does not result in unacceptable loss of capability.
 - **realignment:** reconfigure the system architecture to improve the system's resilience.
 - **rearchitect/redesign:** Modify the system elements or system structure for improved resilience.
 - **redeploy:** reorganize resources to provide required capabilities and address adversity or degradation. Examples include rapid launch to replace satellites lost from a constellation, emergency vehicle repositioning during disasters.
 - **redundancy:** having more than one means for performing the required function. This can mitigate single point failures. Examples include: multiple pumps in parallel.
 - **redundancy (functional):** achieving redundancy by heterogeneous means. This can mitigate common mode failures and common case failures. This is also called, "homogeneous redundancy."
 - **redundancy (physical):** achieving redundancy by more than one identical element.
 - **repairability:** the ease with which a system can be restored to an acceptable condition.
 - **replacement:** changing system elements to regain capability.
 - **safe state:** providing the ability to transition to a state that does not lead to unacceptable loss of capability.
 - **segmentation:** separation (logically or physically) of elements to limit the spread of damage.
 - **shielding:** interposition of items (physical or virtual) that inhibit the adversity's ability to stress the system.
 - **substantiated integrity:** ability to ensure and prove that system elements have not been corrupted.
-

- **substitution:** using new system elements to provide or restore capability.
- **unpredictability:** making changes randomly that confound an opponent's understanding of the system.
- **virtualization:** creating a simulated, rather than actual, version of something. This facilitates stealth, dynamic positioning, and unpredictability. Examples include virtual computer hardware platforms, storage devices, and network resources.

The means objectives and architectural and design techniques will evolve as the resilience engineering discipline matures.

The Resilience Process

Resilience should be considered throughout a system's life cycle, but most especially in early life cycle activities that lead to resilience requirements. Once resilience requirements are established, they can and should be managed along with all the other requirements in the trade space. Specific considerations for inclusion in early life cycle activities can include the following items listed below (Brtis and McEvelley 2019).

- **Business or Mission Analysis Process**
 - Defining the problem space should include identification of adversities and expectations for performance under those adversities.
 - ConOps, OpsCon, and solution classes should consider the ability to avoid, withstand, and recover from adversities.
 - Evaluation of alternative solution classes must consider ability to deliver required capabilities under adversity.
- **Stakeholder Needs and Requirements Definition Process**
 - The stakeholder set should include people who understand potential adversities and stakeholder resilience needs.
 - When identifying stakeholder needs, identify expectations for capability under adverse conditions and degraded/alternate, but useful, modes of operation.
 - Operational concept scenarios should include resilience scenarios.
 - Transforming stakeholder needs into stakeholder requirements includes stakeholder resilience requirements.
 - Analysis of stakeholder requirements includes resilience scenarios in the adverse operational environment.
- **System Requirements Definition Process**
 - Resilience should be considered in the identification of requirements.
 - Achieving resilience and other adversity-driven considerations should be addressed in concert.
- **Architecture Definition Process**
 - Selected viewpoints should support the representation of resilience.
 - Resilience requirements can significantly limit and guide the range of acceptable architectures. Resilience requirements must be mature when used for architecture selection.
 - Individuals developing candidate architectures should be familiar with architectural techniques for achieving resilience.
 - Achieving resilience and other adversity-driven considerations should be addressed in concert.
- **Design Definition Process**
 - Individuals developing candidate designs should be familiar with design techniques for achieving resilience.
 - Achieving resilience and the other adversity-driven considerations should be addressed in concert.
- **Risk Management Process**
 - Risk management should be planned to handle risks, issues, and opportunities identified by resilience activities.

Resilience Metrics

Commonly used resilience metrics include the following (Uday and Marais 2015):

- Time duration of failure
- Time duration of recovery
- Ratio of performance recovery to performance loss
- The speed of recovery
- Performance before and after the disruption and recovery actions
- System importance measures

However, resilience principles that are implied or substantiated by recommendations by domain experts are often missing in practice (Jackson and Ferris 2013). This has led to the development of a new metric for evaluating various systems in domains (aviation, fire protection, rail, and power distribution) to address resilience principles that are often omitted (Jackson 2016).

Numerous candidate resilience metrics have also been identified, including the following (Brtis 2016):

- Maximum outage period
- Maximum brownout period
- Maximum outage depth
- Expected value of capability: the probability-weighted average of capability delivered
- Threat resiliency; i.e. the time integrated ratio of the capability provided divided by the minimum needed capability
- Expected availability of required capability; i.e. the likelihood that for a given adverse environment the required capability level will be available
- Resilience levels; i.e. the ability to provide required capability in a hierarchy of increasingly difficult adversity
- Cost to the opponent
- Cost-benefit to the opponent
- Resource resiliency; i.e. the degradation of capability that occurs as successive contributing assets are lost

Multiple metrics may be required depending on the situation. If one has to select a single most effective metric for reflecting the meaning of resilience, consider "the expected availability of the required capability", where the probability-weighted availability is summed across the scenarios under consideration (Brtis 2016).

Resilience Requirements

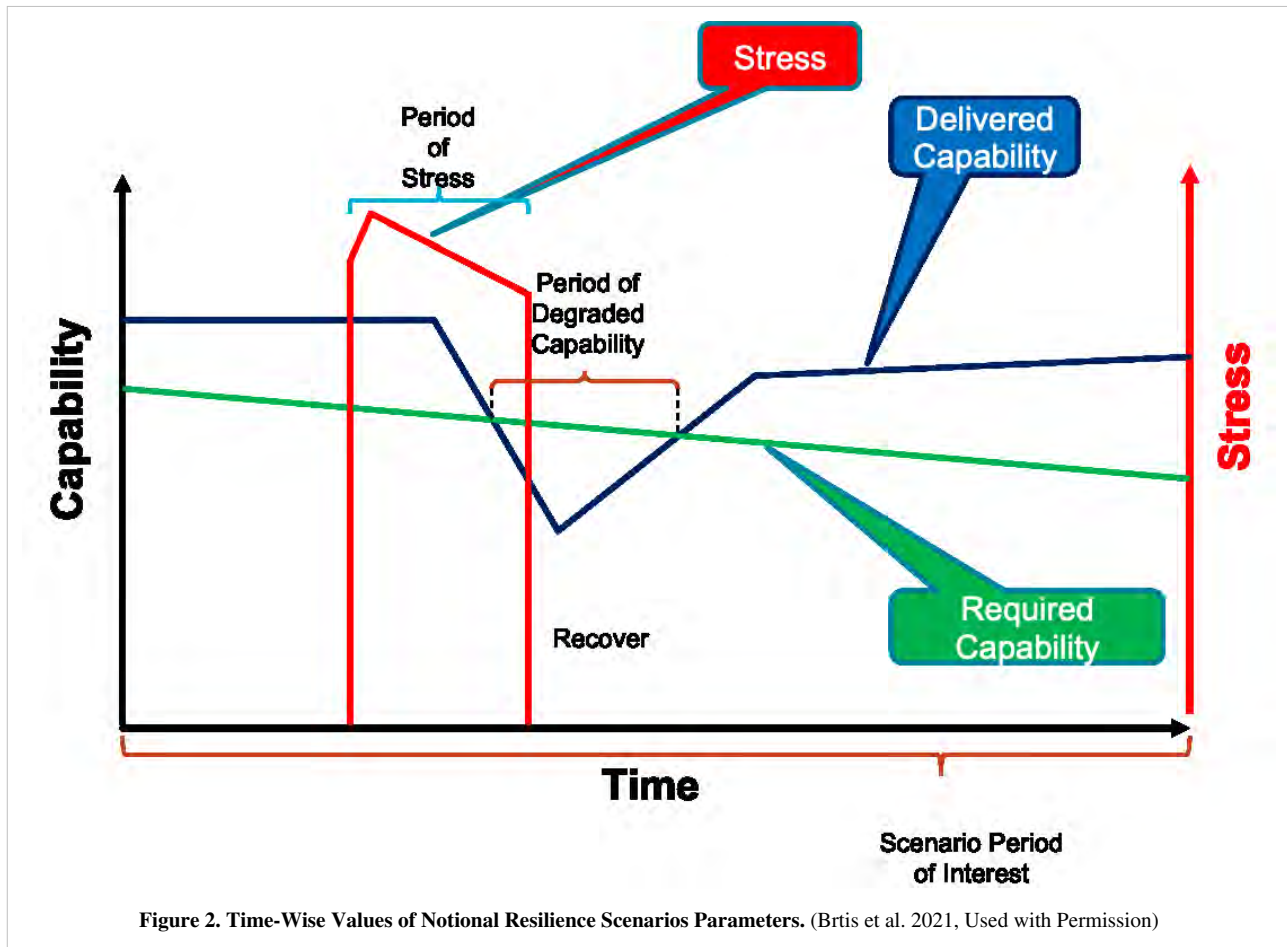
Resilience requirements often take the form of resilience scenarios, which can often be found in ConOps or OpsCons.

The following information is often part of a resilience requirement (Brtis & McEvilley 2019):

- operational concept name
- system or system portion of interest
- capability(s) of interest their metric(s) and units
- target value(s); i.e., the required amount of capabilities
- system modes of operation during the scenario, e.g., operational, training, exercise, maintenance, and update
- system states expected during the scenario
- adversity(s) being considered, their source, and type
- potential stresses on the system, their metrics, units, and values (Adversities may affect the system directly or indirectly. Stresses are adversities that directly affect the system.)
- resilience related scenario constraints, e.g., cost, schedule, policies, and regulations
- timeframe and sub-timeframes of interest
- resilience metric, units, determination methods, and resilience metric target

- impact on other systems if the system of interest fails its resilience goals

Importantly, many of these parameters may vary over the timeframe of the scenario (see **Figure 2**). A single resilience scenario may involve multiple adversities, which may be involved at multiple times during the scenario.

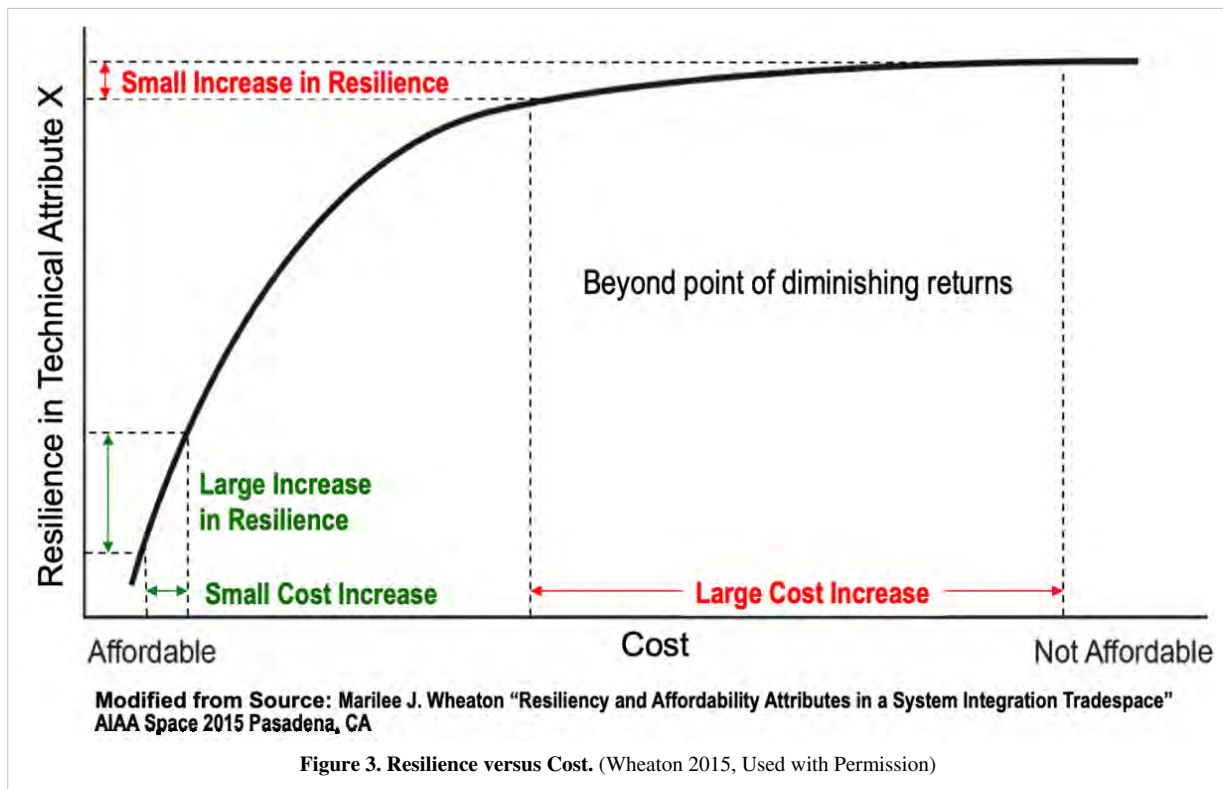


Resilience requirements are compound requirements. They can be classified into three forms: (1) natural language, (2) entity-relationship diagram (data structure), and (3) an extension to SysML. All contain the same information, but are in forms that meet the needs of different audiences (Brtis et al. 2021). An example of a natural language pattern for representing a resilience requirement is:

The <system, mode(t), state(t)> encountering <adversity(t), source, type>, which imposes <stress(t), metric, units, value(t)> thus affecting delivery of <capability(t), metric, units> during <scenario timeframe, start time, end time, units> and under <scenario constraints>, shall achieve <resilience target(t) (include excluded effects)> for <resilience metric, units, determination method>

Affordable Resilience

"Affordable resilience" means achieving an effective balance between cost and system resilience over the system life cycle. Life cycle considerations for affordable resilience should address not only risks and issues associated with known and unknown adversities over time, but also opportunities for gain in known and unknown future environments. This may require balancing the time value of funding vs. the time value of resilience in order to achieve affordable resilience as shown in **Figure 3**.



Once affordable levels of resilience are determined for each key technical attribute, the affordable levels across those attributes can be prioritized via standard techniques, such as Multi-attribute Utility Theory (MAUT) (Keeney and Raiffa 1993) or Analytical Hierarchy Process (AHP) (Saaty 2009).

The priority of affordable resilience attributes for systems is typically domain-dependent. For example:

- Electronic funds transfer systems may emphasize transaction security, meeting regulatory requirements, and liability risks
- Unmanned space exploration systems may emphasize survivability to withstand previously-unknown environments, usually with specified (and often limited) near-term funding constraints
- Electrical power grids may emphasize safety, reliability, and meeting regulatory requirements together with adaptability to changing power generation and storage technologies, shifts in power distribution and usage, etc.
- Medical capacity for disasters may emphasize rapid adaptability, with affordable planning, preparation, response, and recovery levels to meet public health demands. This emphasis must balance potential liability with medical practices, such as triage and "first do no harm".

Discipline Relationships

Resilience has commonality and synergy with many other quality characteristics. Examples include availability, environmental impact, survivability, maintainability, reliability, operational risk management, safety, security, and quality. This group of quality characteristics forms what is called loss-driven systems engineering (LDSE), because they focus on potential losses involved in developing and using systems (INCOSE 2020). These areas frequently share the assets considered, losses considered, adversities considered, requirements, and architectural, design and operational techniques. It is imperative that these areas work closely with one another and share information and decision-making.

The concept of pursuing loss-driven systems engineering, its benefits, and the means by which it can be pursued are addressed in the SEBoK Part 6 article focused on LDSE and the 2020 INCOSE INSIGHT issue on LDSE.

Discipline Standards

Two standards stand out for insight on resilience:

- ASISI (2009) is a standard pertaining to the resilience of organizational systems.
- NIST 800-160 (Ross, R. et al. 2018) considers the resilience of physical systems.

Personnel Considerations

People are important components of systems for which resilience is desired. This aspect is reflected in the human-in-the-loop technique. (Jackson and Ferris 2013). A human in the loop brings a unique capability for agile and adaptable thinking and action. Decisions made by people are at the discretion of the decision-makers in real time, as exemplified by the Apollo 11 mission (Eyles 2009) and similar examples.

Organizational Resilience

Because organizational systems and cyber-physical systems differ significantly, it is not surprising that resilience is addressed differently in each. Organizations, as systems, typically view resilience in terms of managing continuity of operations when facing adverse events and apply a host of processes focused on ensuring that the organization's core functions can withstand disruptions, interruptions, and adversities. ISO 22301 addresses requirements for security and resilience in business continuity management systems. It is an international standard that provides requirements appropriate to the amount and type of impact the organization may or may not accept following a disruption.

Resilient organizations require resilient employees. Resilient organizations and resilient people generally have the following characteristics: (1) they accept the harsh realities facing them, (2) they find meaning in terrible times, and (3) they are creative under pressure, making do with whatever is at hand (Coutu 2002).

Resilient organizations strive to achieve *strategic resilience*, i.e., "the ability to dynamically reinvent business models and strategies as circumstances change...and to change before the need becomes desperately obvious" (Hamel & Valiikangas 2003). An organization with this capability constantly remakes its future rather than defending its past.

Organizational resilience can be measured based on three dimensions: (1) the level of organizational situational awareness, (2) management of organizational vulnerabilities, and (3) organizational adaptive capacity (Lee, Vargo, & Seville 2013). These three items seem well accepted and appear in some form in many papers.

Preparing for the unknown is a recurring challenge of resilience. Organizations must frequently deal with adversities that were previously unknown or unknowable by developing *strategic foresight*. Strategic foresight can be developed by adopting the practice of scenario planning, where multiple adverse futures are envisioned, countermeasures are developed, and coping strategies that appear most frequently are deemed "robust" and candidates for action (Scoblic et. al. 2020). This process also trains personnel to better deal with emerging adversity.

References

Works Cited

- Adams, K. M., P.T. Hester, J.M. Bradley, T.J. Meyers, & C.B. Keating. 2014. "Systems Theory as the Foundation for Understanding Systems." *Systems Engineering*, 17(1):112-123.
- ASIS. 2009. *Organizational Resilience: Security, Preparedness, and Continuity Management Systems--Requirements With Guidance for Use*. Alexandria, VA, USA: ASIS International.
- Billings, C. 1997. *Aviation Automation: The Search for Human-Centered Approach*. Mahwah, NJ: Lawrence Erlbaum Associates.
- Boehm, B. 2013. *Tradespace and Affordability – Phase 2 Final Technical Report*. December 31 2013, Stevens Institute of Technology Systems Engineering Research Center, SERC-2013-TR-039-2. Accessed April 2, 2021. Available at <https://apps.dtic.mil/dtic/tr/fulltext/u2/a608178.pdf>.
- Browning, T.R. 2014. "A Quantitative Framework for Managing Project Value, Risk, and Opportunity." *IEEE Transactions on Engineering Management*. 61(4): 583-598, Nov. 2014. doi: 10.1109/TEM.2014.2326986.
- Brtis, J.S. 2016. *How to Think About Resilience in a DoD Context: A MITRE Recommendation*. MITRE Corporation, Colorado Springs, CO. MTR 160138, PR 16-20151,
- Brtis, J.S. & M.A. McEvilley. 2019. *Systems Engineering for Resilience*. The MITRE Corporation. MP 190495. Accessed April 2, 2021. Available at https://www.researchgate.net/publication/334549424_Systems_Engineering_for_Resilience.
- Brtis, J.S. & M.A. McEvilley. 2020. "Unifying Loss-Driven Systems Engineering Activities," *INCOSE Insight*. 23(4): 7-33. Accessed May 7, 2021. Available at <https://onlinelibrary.wiley.com/toc/21564868/2020/23/4>.
- Brtis, J.S., M.A. McEvilley, & M.J. Pennock. 2021. "Resilience Requirements Patterns." *Proceedings of the INCOSE International Symposium*, July 17-21, 2021.
- Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY: John Wiley & Sons.
- Clemen, Robert T. & T. Reilly. 2001. *Making Hard Decisions, with DecisionTools*. Duxbury Press, Pacific Grove, CA.
- Coutu, Diane L., "How Resilience Works". *Harvard Business Review* 80(5):46-50, 2002.
- Cureton, Ken. 2023. *About Resilience Engineering in (and of) Digital Engineering*. Presented to the Defense Acquisition University, February 23, 2023. Accessed October 2, 2023. Available at https://media.dau.edu/media/t/1_xfmk3vyh.
- DHS. 2017. *Instruction Manual 262-12-001-01 DHS Lexicon Terms and Definitions 2017 Edition – Revision 2*. US Department of Homeland Security. Accessed April 2, 2021. Available at https://www.dhs.gov/sites/default/files/publications/18_0116_MGMT_DHS-Lexicon.pdf.
- Eyles, D. 2009. "1202 Computer Error Almost Aborted Lunar Landing." *Massachusetts Institute of Technology, MIT News*. Accessed April 2, 2021. Available at <https://njohnnetwork.com/2009/07/1202-computer-error-almost-aborted-lunar-landing/>.
- Hamel, G. & L. Valikangas. 2003. "The Quest for Resilience," *Harvard Business Review*, 81(9):52-63.
- Hitchins, D. 2009. "What are the General Principles Applicable to Systems?" *INCOSE Insight*. 12(4):59-6359-63. Accessed April 2, 2021. Available at <https://onlinelibrary.wiley.com/doi/abs/10.1002/inst.200912459>.
- Hollnagel, E., D. Woods, & N. Leveson (eds). 2006. *Resilience Engineering: Concepts and Precepts*. Aldershot, UK: Ashgate Publishing Limited.
- INCOSE. 2015. *Systems Engineering Handbook, a Guide for System Life Cycle Processes and Activities*. New York, NY, USA: John Wiley & Sons.

- INCOSE. 2020. "Special Feature: Loss-Driven Systems Engineering," *INCOSE Insight*. 23(4): 7-33. Accessed May 7, 2021. Available at <https://onlinelibrary.wiley.com/toc/21564868/2020/23/4>.
- Jackson, S. & T. Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering*. 16(2):152-164. doi:10.1002/sys.21228.
- Jackson, S. & T. Ferris. 2016. Proactive and Reactive Resilience: A Comparison of Perspectives. Accessed April 2, 2021. Available at https://www.academia.edu/34079700/Proactive_and_Reactive_Resilience_A_Comparison_of_Perspectives.
- Jackson, W.S. 2016. Evaluation of Resilience Principles for Engineered Systems. Unpublished PhD, University of South Australia, Adelaide, Australia.
- Keeney, R.L. 1992. *Value-Focused Thinking, a Path to Creative Decision-making*, Harvard University Press Cambridge, Massachusetts.
- Keeney, R.L. & H. Raiffa. 1993. *Decisions with Multiple Objectives*. Cambridge University Press.
- Lee, A.V., J. Vargo, & E. Seville. 2013. "Developing a tool to measure and compare organizations' resilience". *Natural Hazards Review*, 14(1):29-41.
- Madni, A. & S. Jackson. 2009. "Towards a conceptual framework for resilience engineering." *IEEE Systems Journal*. 3(2):181-191.
- Neches, R. & A.M. Madni. 2013. "Towards affordably adaptable and effective systems". *Systems Engineering*, 16: 224-234. doi:10.1002/sys.21234.
- OED. 1973. The Shorter Oxford English Dictionary on Historical Principles. edited by C. T. Onions. Oxford: Oxford University Press. Original edition, 1933.
- Ross, R., M. McEvilley, J. Oren 2018. "Systems Security Engineering: Considerations for a Multidisciplinary Approach in the Engineering of Trustworthy Secure Systems." National Institute of Standards and Technology (NIST). SP 800-160 Vol. 1. Accessed April 2, 2021. Available at <https://csrc.nist.gov/publications/detail/sp/800-160/vol-1/final>.
- Saaty, T.L. 2009. *Mathematical Principles of Decision Making*. Pittsburgh, PA, USA: RWS Publications.
- Scoblic, J.P., A. Ignatius, D. Kessler, "Emerging from the Crisis," *Harvard Business Review*, July, 2020.
- Sillitto, H.G. & D. Dori. 2017. "Defining 'System': A Comprehensive Approach." *Proceedings of the INCOSE International Symposium 2017*, Adelaide, Australia.
- Uday, P. & K. Morais. 2015. Designing Resilient Systems-of-Systems: A Survey of Metrics, Methods, and Challenges. *Systems Engineering*. 18(5):491-510.
- Warfield, J.N. 2008. "A Challenge for Systems Engineers: To Evolve Toward Systems Science." *INCOSE Insight*. 11(1).
- Wheaton, M.J. & A.M. Madni. 2015. "Resiliency and Affordability Attributes in a System Integration Tradespace", Proceedings of AIAA SPACE 2015 Conference and Exposition, 31 Aug-2 Sep 2015, Pasadena California. Accessed April 30, 2021. Available at: <https://doi.org/10.2514/6.2015-4434>.
- Winstead, M. 2020. "An Early Attempt at a Core, Common Set of Loss-Driven Systems Engineering Principles." *INCOSE INSIGHT*, December 22-26.
- Winstead, M., D. Hild, & M. McEvilley. 2021. "Principles of Trustworthy Design of Cyber-Physical Systems." MITRE Technical Report #210263, The MITRE Corporation, June 2021. Available: <https://www.mitre.org/publications/technical-papers>.

Primary References

- Hollnagel, E., D.D. Woods, & N. Leveson (Eds.). 2006. *Resilience Engineering: Concepts and Precepts*. Aldershot, UK: Ashgate Publishing Limited.
- Jackson, S. & T. Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering*, 16(2):152-164.
- Jackson, S., S.C. Cook, & T. Ferris. 2015. "Towards a Method to Describe Resilience to Assist in System Specification." *Proceedings of the INCOSE International Symposium*. Accessed May 25, 2023. Available at https://www.researchgate.net/publication/277718256_Towards_a_Method_to_Describe_Resilience_to_Assist_System_Specification
- Jackson, S. 2016. *Principles for Resilient Design - A Guide for Understanding and Implementation*. Accessed April 30, 2021. Available at <https://www.irgc.org/irgc-resource-guide-on-resilience>.
- Madni, A. & S. Jackson. 2009. "Towards a conceptual framework for resilience engineering." *IEEE Systems Journal*. 3(2):181-191.

Additional References

- 9/11 Commission. 2004. 9/11 Commission Report. National Commission on Terrorist Attacks on the United States. Accessed April 2, 2021. Available at <https://9-11commission.gov/report/>.
- Ball, R. E. (2003). *The Fundamentals of Aircraft Combat Survivability Analysis and Design*. 2nd edition. AIAA (American Institute of Aeronautics & Astronautics) Education series. (August 1, 2003)
- Billings, C. 1997. *Aviation Automation: The Search for Human-Centered Approach*. Mahwah, NJ: Lawrence Erlbaum Associates.
- Bodeau, D. K & R. Graubart. 2011. *Cyber Resiliency Engineering Framework*. The MITRE Corporation. MITRE Technical Report #110237.
- DoD. 1985. *MIL-HDBK-268(AS) Survivability Enhancement, Aircraft Conventional Weapon Threats, Design and Evaluation Guidelines*. US Navy, Naval Air Systems Command.
- Henry, D. & E. Ramirez-Marquez. 2016. "On the Impacts of Power Outages during Hurricane Sandy – A Resilience Based Analysis." *Systems Engineering*. 19(1): 59-75. Accessed April 2, 2021. Available at <https://onlinelibrary.wiley.com/doi/10.1002/sys.21338>.
- Jackson, S., S.C. Cook, & T. Ferris. 2015. A Generic State-Machine Model of System Resilience. *INCOSE Insight*. 18(1):1 4-18. Accessed April 2, 2021. Available at <https://onlinelibrary.wiley.com/doi/10.1002/inst.12003>. Accessed on April 2, 2021.
- Leveson, N. 1995. *Safeware: System Safety and Computers*. Reading, Massachusetts: Addison Wesley.
- Pariès, J. 2011. "Lessons from the Hudson." in *Resilience Engineering in Practice: A Guidebook*, edited by E. Hollnagel, J. Pariès, D.D. Woods & J. Wreathhall, 9-27. Farnham, Surrey: Ashgate Publishing Limited.
- Perrow, C. 1999. *Normal Accidents: Living With High Risk Technologies*. Princeton, NJ: Princeton University Press.
- Reason, J. 1997. *Managing the Risks of Organizational Accidents*. Aldershot, UK: Ashgate Publishing Limited.
- Rechtin, E. 1991. *Systems Architecting: Creating and Building Complex Systems*. Englewood Cliffs, NJ: CRC Press.
- Schwarz, C.R. & H. Drake. 2001. *Aerospace Systems Survivability Handbook Series. Volume 4: Survivability Engineering*. Joint Technical Coordinating Group on Aircraft Survivability, Arlington, VA.
- US-Canada Power System Outage Task Force. 2004. Final Report on the August 14, 2003 Blackout in the United States and Canada: Causes and Recommendations. Washington-Ottawa.

Resilience Modeling

- Lead Author:
- Ken Cureton
- Contributing Authors:
- John Britis, Scott Jackson, Tim Ferris, and Ivan Taylor

Overview

Resilience modeling is an emerging topic in digital engineering (DE), model-based systems engineering (MBSE), and artificial intelligence/machine learning (AI/ML). Systems Engineers and developers need to identify, characterize, and accomplish trade-offs regarding cost, schedule, performance, and quality characteristics (including resilience) over the life cycle of a system. If system resilience could be accurately modeled, then quantitative (or at least qualitative) metrics could be used to evaluate a system's resilience characteristics (e.g., via a digital twin). However, no single methodology is accepted for resilience modeling of simple, complicated, or complex systems. This section examines a few potential (and evolving) modeling techniques that practitioners of resilience engineering could use.

Definition

A system resilience model represents a selective abstraction of a system to provide the required capability when facing adversity within the system and its environment. This definition of a resilience model is limited to human-made systems containing software, hardware, humans (e.g., socio-technical, organizational), infrastructures, concepts, and processes.

Potential Resilience Metrics

Please refer to the Resilience Metrics description in the “System Resilience” section.

Modeling, Measuring, & Evaluating System Resilience

Formal Methods of Constructing Models for Systems Resilience—Resilience Contracts

Madni, Erwin, & Sievers (2020) proposed resilience contracts (RCs) as an upgrade to the widely used Contract-Based Design (CBD) approach. They observed that traditional methods like Büchi automata and Linear Temporal Logic (LTL) work for systems that behave predictably. However, many modern systems do not always behave predictably. To handle this, an RC is a mathematical model that extends CBD to account for uncertainty and unpredictability.

An RC is a mixed model that uses fixed rules and flexible assertions and is represented as a Partially Observable Markov Decision Process (POMDP). A POMDP is a special form of a Markov decision process that deals with situations where some states and transitions are not directly observable.

RCs add flexibility to deterministic contracts for systems with random elements by repeatedly checking the environment and system status, choosing the best actions to achieve a goal, and executing those actions. After each action, the system's environment and health are reassessed. The planning function then decides whether to continue with the current plan if the actions are effective or to make changes if they are not.

Application of System Dynamics

System dynamics is suitable for resilience modeling because it captures behavior over time, and resilience takes a behavior over time perspective, as shown in Figure 1 (from the System Resilience article, reproduced below). As with other types of modeling, one of the primary values of system dynamics modeling is that it can be used to build a shared understanding of the issues for all stakeholders.

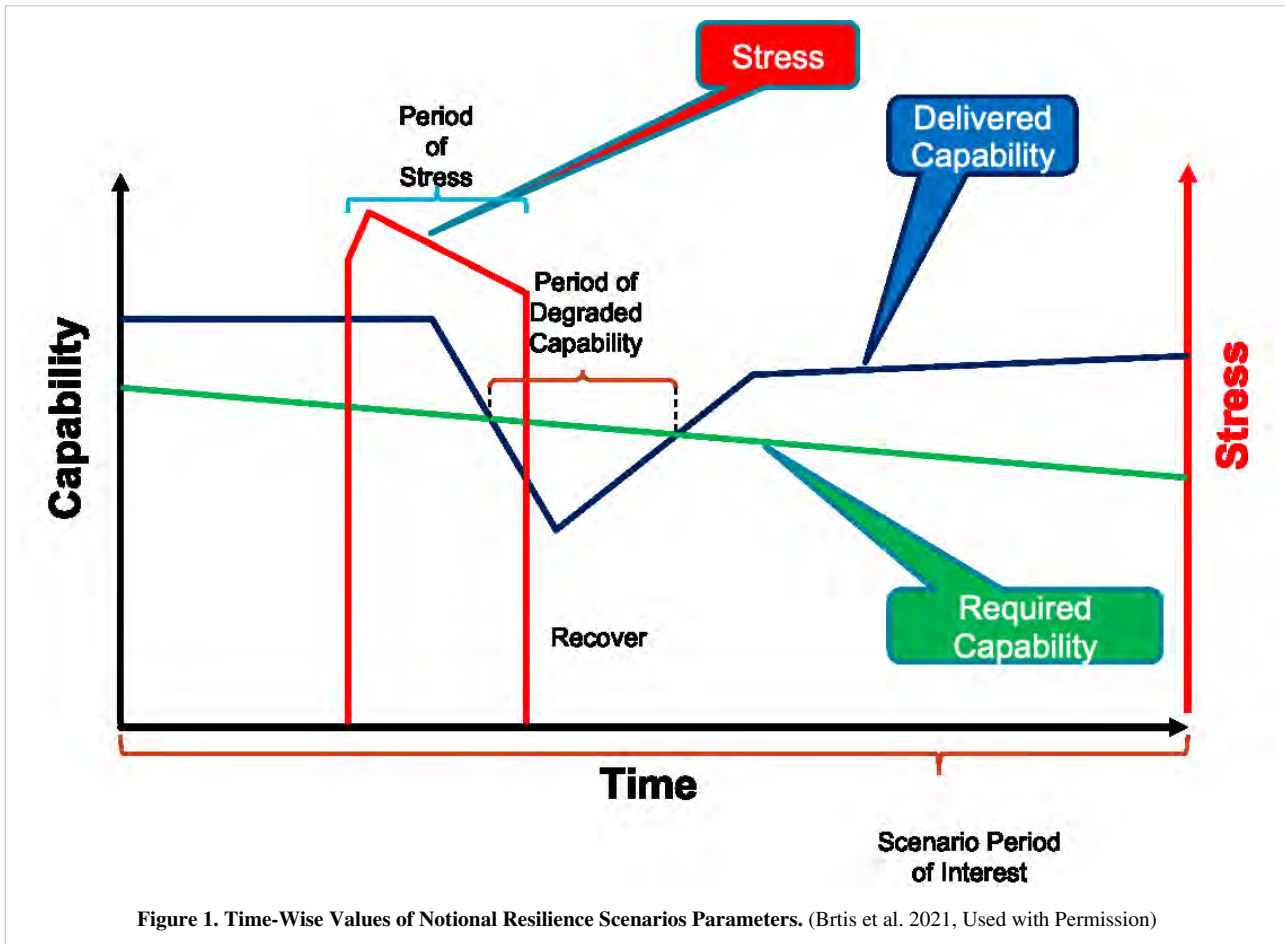


Figure 1. Time-Wise Values of Notional Resilience Scenarios Parameters. (Brtis et al. 2021, Used with Permission)

In this regard, system dynamics's long tradition of participatory model building can be uniquely valuable (Herrera & Kopainsky, 2020). These group model building activities produce causal loop diagrams, which demonstrate the feedback structure in a system in which a change in one component can ripple through the other connected components in the design and return to the original part in a reinforcing way that can lead to catastrophic failure or in a balanced way that can lead to stability and recovery from adversity.

Archetypes are another qualitative tool used in system dynamics modeling (Onyekachi, Onyeagoziri, & Ryan, 2021). In archetypes, a small set of models can examine many behavior types. In terms of resilience modeling, archetypes evaluate the feedback loops in the system that lead to both intended and unintended consequences in behavior where often the unintended consequences are not foreseen when the system is designed.

Quantitative system dynamics models have been applied to resilience modeling (Iturriza et al., 2017; Yabe et al., 2021). In this case, a highly interconnected system of first-order linear differential equations is solved using numerical methods (Radzicki & Taylor, 1997). There are two types of quantitative system dynamics models: exploratory models, which are based on theoretical behavior of a system (Taylor & Willett, 2024), and calibrated models, which use historical time-series data to estimate the model parameters (Taylor & Hossain, 2024).

Software tools can be used to build interactive models applied to resilience modeling (Iturriza et al., 2017). Using interactive models, system engineers could experiment in a virtual environment to test procedures to improve the system's Resilience under conditions that are not economical or even possible in real life.

A system's response to adversity can be analyzed using quantitative system dynamics to determine the effectiveness of the resilience processes. This is effective in learning about the impact of natural disasters on critical infrastructure (Yabe et al., 2021).

Caveats Regarding Resilience Models

Misusing models can lead to problems. It is therefore essential to use a model only for its intended purpose. Modelers must ensure the model is suitable for this purpose, check that all assumptions are valid, and ensure that no constraints are violated.

Neches & Madni (2013) suggest that modeling tools and languages should align with their intended use. Sometimes, modelers must use different tools or languages, which can cause compatibility issues. Because of this, multiple models need to be developed and made to work together, as models must cover various disciplines, aspects, and phenomena. Modelers must also create and manage different models, such as executable, depictional, and statistical models, and multiple categories, including device and environmental physics, communications, sensors, effectors, software, and systems.

Model Analysis with Consideration of Constraint Theory

Friedman & Phan (2017) point out that models face typical “well-posed” problems in mathematics. Modelers must check whether complex models are internally consistent and whether the requested calculations are mathematically allowable.

Complex models, especially those created by diverse teams, often have internal inconsistencies. Even if a model is consistent, many possible calculations might not be allowable due to over-constrained computational sets, with too many input values for the equations. On the other hand, under-constrained calculations, with too many equations and not enough values, can lead to unclear or undefined results.

Most models of complex systems include tight interaction loops called Basic Nodal Squares (BNS), which form the “kernel of intrinsic constraint.” These models often have more extensive, nested interaction loops important for emergent behavior and attributes of Resilience such as adaptability, flexibility, and handling disruptions.

When computational requests that are not allowed are made on models, it often leads to incorrect predictions.

References

Works Cited

- Britis, J.S. 2016. “How to Think About Resilience in a DoD Context: A MITRE Recommendation”. MITRE Corporation, Colorado Springs, CO. MTR 160138, PR 16-20151.
- Friedman, G.J & Phan, P., “Constraint Theory – Multidimensional Mathematical Model Management” Second Edition, ISBN 978-3-319-54791-8. Springer International Publishing AG 2005, 2017
- Herrera, H. & Kopainsky, B. (2020) “Using system dynamics to support a participatory assessment of resilience,” *Environment Systems and Decisions* 40:342–355.
- Iturriza, M., Abdelgawad, A.A., Labaka, L., Radianti, J., Sarriegi, J.M., & Gonzalez, J.J. (2017). “Smart mature resilience system dynamics based interactive learning environment: a beta version,” *International Journal of Safety and Security Engineering*, Vol. 7, No. 3 367–379
- Madni, A.M., Erwin, D., & Sievers, M. “Constructing Models for Systems Resilience: Challenges, Concepts, Formal Methods, and Illustrative Examples”, *MDPI Systems*, 2020, 8,3; doi:10.3390/systems8010003.
- Neches, R. & A.M. Madni. 2013. “Towards affordably adaptable and effective systems”. *Systems Engineering*, 16: 224-234. doi:10.1002/sys.21234.

- Onyekachi J. Onyeagoziri, C.S., & Ryan, T. (2021) "A system dynamics approach for understanding community resilience to disaster risk," *Jambá: Journal of Disaster Risk Studies* 13(1), a1037, <https://doi.org/10.4102/jamba.v13i1.1037>
- Radzicki, M.J. & Taylor, R.A. (1997) "Introduction to System Dynamics: A Systems Approach to Understanding Complex Policy Issues". US Department of Energy. <https://web.nmsu.edu/~lang/files/mike.pdf>
- Taylor, I.W. & Willett, K.D., "Modeling Cybersecurity Operations for Enhanced Security", 34th INCOSE International Symposium, Dublin, Ireland, 2-6 July 2024.
- Taylor, I.W. & Hossain, N.U.I., "A System Dynamics Model of Organizational Resilience," 34th INCOSE International Symposium, Dublin, Ireland, 2-6 July 2024.
- Uday, P. & K. Morais. 2015. "Designing Resilient Systems-of-Systems: A Survey of Metrics, Methods, and Challenges". *Systems Engineering*. 18(5): 491-510.
- Willett, K. D. & Taylor, I., (2022) "Security Modeling and Simulation", in Handbook of Security Science, Masys, A.J. (ed.), Springer Nature Switzerland AG https://doi.org/10.1007/978-3-319-91875-4_65

Primary References

- INCOSE. 2015. *Systems Engineering Handbook, a Guide for System Life Cycle Processes and Activities*. New York, NY, USA: John Wiley & Sons.
- Hollnagel, E., Woods, D. D., & Leveson, N. (Eds.). 2006. *Resilience Engineering: Concepts and Precepts*. Aldershot, UK: Ashgate Publishing Limited.

Additional References

- Brtsis, J.S. & M.A. McEvelley. 2019. *Systems Engineering for Resilience*. The MITRE Corporation. MP 190495. Accessed April 2, 2021. Available: https://www.researchgate.net/publication/334549424_Systems_Engineering_for_Resilience
- Hollnagel, E., Woods, D. D., & Leveson, N. (Eds.). (2006). *Resilience Engineering: Concepts and Precepts*. Aldershot, UK: Ashgate Publishing Limited.
- INCOSE. 2015. *Systems Engineering Handbook, a Guide for System Life Cycle Processes and Activities*. New York, NY, USA: John Wiley & Sons.
- Jackson, S., S.C. Cook, & T. Ferris. 2015. A Generic State-Machine Model of System Resilience. *INCOSE Insight*. 18(1):1 4-18. Accessed April 2, 2021. Available: <https://onlinelibrary.wiley.com/doi/10.1002/inst.12003>
- Jackson, S., & Ferris, T. (2013). "Resilience Principles for Engineered Systems." *Systems Engineering*, 16(2): 152-164.
- Jackson, S., S.C. Cook, & T. Ferris, T. "Towards a Method to Describe Resilience to Assist in System Specification." Proceedings of the INCOSE International Symposium.
- Jackson, S. 2016. *Principles for Resilient Design - A Guide for Understanding and Implementation*. Accessed April 30, 2021. Available at <https://www.irgc.org/irgc-resource-guide-on-resilience>
- Jackson, S. & T. Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering*. 16(2): 152-164. doi:10.1002/sys.21228.
- Jackson, S. & T. Ferris. 2016. Proactive and Reactive Resilience: A Comparison of Perspectives. Accessed April 2, 2021. Available: https://www.academia.edu/34079700/Proactive_and_Reactive_Resilience_A_Comparison_of_Perspectives
- Madni, A. & S. Jackson. 2009. "Towards a conceptual framework for resilience engineering." *IEEE Systems Journal*. 3(2): 181-191.

Madni, A. & S. Jackson. 2009. "Towards a conceptual framework for resilience engineering." *IEEE Systems Journal*. 3(2): 181-191.

Rechtin, E. 1991. *Systems Architecting: Creating and Building Complex Systems*. Englewood Cliffs, NJ: CRC Press.

Yabe, T., Suresh, P., Rao, C. & Ukkusuri, S.V. (2021) "Resilience of Interdependent Urban Socio-Physical Systems using Large-Scale Mobility Data: Modeling Recovery Dynamics," *Sustainable Cities and Society* 75(6):103237 DOI: 10.1016/j.scs.2021.103237

System Resistance to Electromagnetic Interference

- Lead Author:
- Paul Phister
- Contributing Authors:
- Scott Jackson, Richard Turner, John Snoderly, and Alice Squires

Electromagnetic Interference (EMI) is the disruption of operation of an electronic device when it is in the vicinity of an electromagnetic field in the radio frequency (RF) spectrum. Many electronic devices fail to work properly in the presence of strong RF fields. The disturbance may interrupt, obstruct, or otherwise degrade or limit the effective performance of the circuit. The source may be any object, artificial or natural, that carries rapidly changing electrical currents.

Electromagnetic Compatibility (EMC) is the ability of systems, equipment, and devices that utilize the electromagnetic spectrum to operate in their intended operational environments without suffering unacceptable degradation or causing unintentional degradation because of electromagnetic radiation or response. It involves the application of sound electromagnetic spectrum management; system, equipment, and device design configuration that ensures interference-free operation; and clear concepts and doctrines that maximize operational effectiveness (DAU 2010, Chapter 7).

Overview

Spectrum

Each nation has the right of sovereignty over the use of its spectrum and must recognize that other nations reserve the same right. It is essential that regional and global forums exist for the discussion and resolution of spectrum development and infringement issues between bordering and proximal countries that might otherwise be difficult to resolve.

The oldest, largest, and unquestionably the most important such forum, with 193 member countries, is the International Telecommunications Union (ITU) agency of the United Nations, which manages spectrum at a global level. As stated in Chapter 3 of the NTIA Manual, "The International Telecommunication Union (ITU)...is responsible for international frequency allocations, worldwide telecommunications standards and telecommunication development activities" (NTIA 2011, 3-2). The broad functions of the ITU are the regulation, coordination and development of international telecommunications.

The spectrum allocation process is conducted by many different international telecommunication geographical committees. Figure 1 shows the various international forums represented worldwide.



Assigning frequencies is very complicated, as shown in the radio spectrum allocation chart in Figure 2. Sometimes, commercial entities try to use frequencies that are actually assigned to US government agencies, such as the Department of Defense (DoD). One such incident occurred when an automatic garage door vendor installed doors on homes situated near a government installation. Random opening and closing of the doors created a problem for the vendor that could have been avoided.

Four ITU organizations affect spectrum management (Stine and Portigal 2004):

1. World Radio-communication Conference (WRC)
2. Radio Regulations Board (RRB)
3. Radio-communications Bureau (RB)
4. Radio-communication Study Groups (RSG)

The WRC meets every four years to review and modify current frequency allocations. The RB registers frequency assignments and maintains the master international register. The RRB approves the Rules of Procedures used by the BR to register frequency assignments and adjudicates interference conflicts among member nations. The SG analyzes spectrum usage in terrestrial and space applications and makes allocation recommendations to the WRC. Most member nations generally develop national frequency allocation policies that are consistent with the Radio Regulations (RR). These regulations have treaty status.

Dual Management of Spectrum in the US

Whereas most countries have a single government agency to perform the spectrum management function, the US has a dual management scheme intended to insure that

- decisions concerning commercial interests are made only after considering their impact on government systems; and
- government usage supports commercial interests.

The details of this scheme, established by the Communications Act of 1934, are as follows:

- the Federal Communications Commission (FCC) is responsible for all non-government usage
- the FCC is directly responsible to Congress;
- the president is responsible for federal government usage, and by executive order, delegates the federal government spectrum management to the National Telecommunications and Information Administration (NTIA); and
- the NTIA is under the authority of the Secretary of Commerce.

The FCC regulates all non-federal government telecommunications under Title 47 of the Code of Federal Regulations. For example, see FCC (2009, 11299-11318). The FCC is directed by five Commissioners appointed by the president and confirmed by the Senate for five-year terms. The Commission staff is organized by function. The responsibilities of the six operating Bureaus include processing applications for licenses, analyzing complaints, conducting investigations, implementing regulatory programs, and conducting hearings (<https://www.fcc.gov>).

The NTIA performs spectrum management function through the Office of Spectrum Management (OSM), governed by the Manual of Regulations and Procedures for Federal Radio Frequency Management. The IRAC develops and executes policies, procedures, and technical criteria pertinent to the allocation, management, and usage of spectrum. The Spectrum Planning and Policy Advisory Committee (SPAC) reviews the IRAC plans, balancing considerations of manufacturing, commerce, research, and academic interests.

Within the DoD, spectrum planning and routine operation activities are cooperatively managed. Spectrum certification is a mandated process designed to ensure that

1. frequency band usage and type of service in a given band are in conformance with the appropriate national and international tables of frequency allocations;
2. equipment conforms to all applicable standards, specifications, and regulations; and
3. approval is provided for expenditures to develop equipment dependent upon wireless communications.

Host Nation Coordination and Host Nation Approval

In peacetime, international spectrum governance requires military forces to obtain host nation permission — Host Nation Coordination (HNC)/Host Nation Approval (HNA) — to operate spectrum-dependent systems and equipment within a sovereign nation. For example, international governance is honored and enforced within the United States by the US departments of State, Defense, and the user service.

In wartime, international spectrum governance is not honored between warring countries; however, the sovereign spectrum rights of bordering countries must be respected by military forces executing their assigned missions. For example, HNA is solicited by US naval forces to use spectrum-dependent systems and equipment in bordering countries' airspace and/or on bordering countries' soil. HNA must be obtained before the operation of spectrum-dependent systems and equipment within a sovereign nation. The combatant commander is responsible for coordinating requests with sovereign nations within his or her area of responsibility. Because the combatant commander has no authority over a sovereign nation, the HNC/HNA process can be lengthy and needs to be started early in the development of a system. Figure 2 illustrates a spectrum example.

divided into random and impulse sources. These may be transient, continuous or intermittent in occurrence. Examples include unintentional emissions from communication and radar transmitters, electric switch contacts, computers, thermostats, ignition systems, voltage regulators, pulse generators, and intermittent ground connections. (Bagad 2009, G-1)

TEMPEST

TEMPEST is a codename used to refer to the field of emission security. The National Security Agency (NSA) investigations conducted to study compromising emission (CE) were codenamed TEMPEST. National Security Telecommunications Information Systems Security Issuance (NSTISSI)-7000 states:

Electronic and electromechanical information-processing equipment can produce unintentional intelligence-bearing emanations, commonly known as TEMPEST. If intercepted and analyzed, these emanations may disclose information transmitted, received, handled, or otherwise processed by the equipment. (NSTISS 1993, 3)

These compromising emanations consist of electrical, mechanical, or acoustical energy intentionally or unintentionally emitted by sources within equipment or systems which process national security information. Electronic communications equipment needs to be secured from potential eavesdroppers while allowing security agencies to intercept and interpret similar signals from other sources. The ranges at which these signals can be intercepted depends upon the functional design of the information processing equipment, its installation, and prevailing environmental conditions.

Electronic devices and systems can be designed, by means of Radiation Hardening techniques, to resist damage or malfunction caused by ionizing and other forms of radiation (Van Lint and Holmes Siedle 2000). Electronics in systems can be exposed to ionizing radiation in the Van Allen radiation belts around the Earth's atmosphere, cosmic radiation in outer space, gamma or neutron radiation near nuclear reactors, and electromagnetic pulses (EMP) during nuclear events.

A single charged particle can affect thousands of electrons, causing electronic noise that subsequently produces inaccurate signals. These errors could affect safe and effective operation of satellites, spacecraft, and nuclear devices. Lattice displacement is permanent damage to the arrangement of atoms in element crystals within electronic devices. Lattice displacement is caused by neutrons, protons, alpha particles, and heavy ions. Ionization effects are temporary damages that create latch-up glitches in high power transistors and soft errors like bit flips in digital devices. Ionization effects are caused by charged particles.

Most radiation-hardened components are based on the functionality of their commercial equivalents. Design features and manufacturing variations are incorporated to reduce the components' susceptibility to interference from radiation. Physical design techniques include insulating substrates, package shielding, chip shielding with depleted boron, and magneto-resistive RAM. Logical design techniques include error-correcting memory, error detection in processing paths, and redundant elements at both circuit and subsystem levels (Dawes 1991). Nuclear hardness is expressed as susceptibility or vulnerability for given environmental conditions. These environmental conditions include peak radiation levels, overpressure, dose rates, and total dosage.

References

Works Cited

Bagad, V.S. 2009. *Electronic Product Design*, 4th ed. Pune, India: Technical Publications Pune.

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.

NSTISS. 1993. *Tempest Countermeasures for Facilities*. Ft. Meade, MD, USA: National Security Telecommunications and Information Systems Security (NSTISSI). 29 November, 1993. NSTISSI No. 7000.

NTIA. 2011. *Manual of Regulations and Procedures for Federal Radio Frequency Management*, May 2011 Revision of the 2008 Edition. Washington, DC, USA: National Telecommunications and Information Administration, U.S. Department of Commerce.

Stine, J. and D. Portigal. 2004. *An Introduction to Spectrum Management*. Bedford, MA, USA: MITRE Technical Report Spectrum. March 2004.

Van Lint, V.A.J. and A.G. Holmes Siedle. 2000. *Radiation Effects in Electronics: Encyclopedia of Physical Science and Technology*. New York, NY, USA: Academic Press.

Primary References

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.

Additional References

None.

System Safety

- Lead Author:
- Dick Fairley
- Contributing Authors:
- Art Pyster and Alice Squires

In the most general sense, safety is freedom from harm. As an engineering discipline, system safety is concerned with minimizing hazards that can result in a mishap with an expected severity and with a predicted probability. These events can occur in elements of life-critical systems as well as other system elements. MIL-STD-882E defines system safety as "the application of engineering and management principles, criteria, and techniques to achieve acceptable risk, within the constraints of operational effectiveness and suitability, time, and cost, throughout all phases of the system life cycle" (DoD 2012). MIL-STD-882E defines standard practices and methods to apply as engineering tools in the practice of system safety. These tools are applied to both hardware and software elements of the system in question.

Overview

System safety engineering focuses on identifying hazards, their causal factors, and predicting the resultant severity and probability. The ultimate goal of the process is to reduce or eliminate the severity and probability of the identified hazards, and to minimize risk and severity where the hazards cannot be eliminated. MIL STD 882E defines a hazard as "a real or potential condition that could lead to an unplanned event or series of events (i.e., mishap) resulting in death, injury, occupational illness, damage to or loss of equipment or property, or damage to the environment." (DoD 2012).

While systems safety engineering attempts to minimize safety issues throughout the planning and design of systems, mishaps do occur from combinations of unlikely hazards with minimal probabilities. As a result, safety engineering is often performed in reaction to adverse events after deployment. For example, many improvements in aircraft safety come about as a result of recommendations by the U.S. National Air Traffic Safety Board based on accident investigations. Risk is defined as "a combination of the severity of the mishap and the probability that the mishap will occur" (DoD 2012). Failure to identify risks to safety and the according inability to address or "control" these risks can result in massive costs, both human and economic (Roland and Moriarty 1990)."

Personnel Considerations

System Safety Specialists are typically responsible for ensuring system safety. Chapter 11 of Air Force Instruction (AFI) 191-202 (USAF 2020) is a lengthy exposition of the responsibilities of system safety specialists. AFI 191-202 defines system safety as "the application of engineering and management principles, criteria and techniques to achieve acceptable risk within the constraints of operational effectiveness and suitability, time and cost throughout all phases of the system life cycle." The AFI identifies eight activities to achieve systems safety:

1. Documenting the system safety approach
 2. Hazard identification and analysis over the system life cycle
 3. Assessment of risk, expressed as severity and probability of consequences
 4. Identification and assessment of potential risk mitigation measures
 5. Implementation of measures to reduce risks to acceptable levels
 6. Verification of risk reduction
 7. Acceptance of risks by appropriate authorities
 8. Tracking of hazards and risks throughout the system life cycle
-

Although these activities are documented in an Air Force Instruction, they are actually quite generic and applicable to almost any system safety process.

Safety personnel are responsible for the integration of system safety requirements, principles, procedures, and processes into the program and into lower system design levels to ensure a safe and effective interface. Two common mechanisms are the Safety Working Group (SWG) and the Management Safety Review Board (MSRB). The SWG enables safety personnel from all integrated product teams (IPTs) to evaluate, coordinate, and implement a safety approach that is integrated at the system level in accordance with MIL-STD-882E (DoD 2012). Increasingly, safety reviews are being recognized as an important risk management tool. The MSRB provides program level oversight and resolves safety related program issues across all IPTs.

Table 1 provides additional information on safety.

Table 1. Safety Ontology. (SEBoK Original)

Ontology Element Name	Ontology Element Attributes	Relationships to Safety
Failure modes	Manner of failure	Required attribute
Severity	Consequences of failure	Required attribute
Criticality	Impact of failure	Required attribute
Hazard Identification	Identification of potential failure modes	Required to determine failure modes
Risk	Probability of a failure occurring	Required attribute
Mitigation	Measure to take corrective action	Necessary to determine criticality and severity

Table 1 indicates that achieving system safety involves a close tie between Safety Engineering and other specialty Systems Engineering disciplines such as System Reliability, Availability, and Maintainability.

References

Works Cited

- DoD. 2012. *Standard practice for System Safety*. Arlington, VA, USA: Department of Defense (DoD). MIL-STD 882E. Accessed April 2, 2021. Available at [https:// every-spec. com/ MIL-STD/ MIL-STD-0800-0899/ MIL-STD-882E_41682/](https://every-spec.com/MIL-STD/MIL-STD-0800-0899/MIL-STD-882E_41682/).
- Roland, H.E. and B. Moriarty. 1990. *System Safety Engineering and Management*, 2nd Ed. Hoboken, NJ, USA: Wiley.
- USAF. 2020. *The US Air Force Mishap Prevention Program*. Air Force Instruction 91-202. Washington, DC, USA: US Air Force. Accessed April 2, 2021. Available [https:// static. e-publishing. af. mil/ production/ 1/ af_se/ publication/afi91-202/afi91-202.pdf](https://static.e-publishing.af.mil/production/1/af_se/publication/afi91-202/afi91-202.pdf).

Primary References

DoD. 2012. *Standard practice for System Safety*. Arlington, VA, USA: Department of Defense (DoD). MIL-STD 882E. Accessed April 2, 2021. Available at https://everyspec.com/MIL-STD/MIL-STD-0800-0899/MIL-STD-882E_41682/.

Additional References

Bahr, N.J. 2015. *System Safety Engineering and Risk Assessment: A Practice Approach*, 2nd Ed. Boca Raton, FL, USA: CRC Press.

ISSS. 2015a. "System Safety Hazard Analysis Report." The International System Safety Society (ISSS). DI-SAFT-80101C. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-80101C_53803/.

ISSS. 2015b. "Safety Assessment Report." The International System Safety Society (ISSS). DI-SAFT-80102C. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-80102C_53802/. Accessed on April 2, 2021.

ISSS. 2015c. "Engineering Change Proposal System Safety Report." The International System Safety Society (ISSS). DI-SAFT-80103C. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-80103C_52427/.

ISSS. 2015d. "Waiver or Deviation System Safety Report." The International System Safety Society (ISSS). DI-SAFT-80104C. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-80104C_53816/.

ISSS. 2015e. "System Safety Program Progress Report." The International System Safety Society (ISSS). DI-SAFT-80105C. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-80105C_53817/.

ISSS. 2015f. "Health Hazard Assessment Report." The International System Safety Society (ISSS). DI-SAFT-80106C. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-80106C_53814/.

ISSS. 2003. "Explosive Ordnance Disposal Data." The International System Safety Society (ISSS). DI-SAFT-80931B. Accessed April 2, 2021. Available https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-80931B_15713/.

ISSS. 2015. "Explosive Hazard Classification Data." The International System Safety Society (ISSS). DI-SAFT-81299C. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-81299C_53809/.

ISSS. 2001. "System Safety Program Plan (SSPP)." The International System Safety Society (ISSS). DI-SAFT-81626. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-81626_11514/.

ISSS. 2015. "Mishap Risk Assessment Report." The International System Safety Society (ISSS). DI-SAFT-81300B. Accessed April 2, 2021. Available at https://everyspec.com/DATA-ITEM-DESC-DIDs/DI-SAFT/DI-SAFT-81300B_53813/.

Joint Software System Safety Committee. 1999. *Software System Safety Handbook*. Accessed April 2, 2021. Available at: <https://www.acqnotes.com/Attachments/Joint-SW-Systems-Safety-Engineering-Handbook.pdf>.

Leveson, N.G. 2016 reprint edition. *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, Mass: MIT Press. Accessed April 2, 2021. Available at <https://mitpress.mit.edu/books/engineering-safer-world>.

Leveson, N.G. 2012. "Complexity and safety." In *Complex Systems Design & Management*, ed. Omar Hammami, Daniel Krob, and Jean-Luc Voirin, 27–39. Springer, Berlin, Heidelberg. Accessed April 2, 2021. Available at <https://>

/dx.doi.org/10.1007/978-3-642-25203-7_2.

NASA. 2004. *NASA Software Safety Guidebook*. Accessed April 2, 2021. Available at <https://standards.nasa.gov/standard/nasa/nasa-gb-871913>.

SAE. 1996. *Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment*. ARP 4761. Warrendale, PA, USA: Society of Automotive Engineers. Accessed April 2, 2021. Available: <https://www.sae.org/standards/content/arp4761/>.

SAE. 2010. *Guidelines for Development of Civil Aircraft and Systems*. ARP 4754. Warrendale, PA, USA: Society of Automotive Engineers. Accessed April 2, 2021. Available at <https://www.sae.org/standards/content/arp4754a/>.

System Security

- Lead Author:
- Mark Winstead
- Contributing Authors:
- Terri Chan and Keith Willett

Security is freedom from those conditions that may lead to loss of assets (anything of value) with undesired consequences (Ross, Winstead, & McEvilley 2022). Systems Security specifically is concerned with systems delivering capability (an asset) with intended and only intended behaviors and outcomes (no unacceptable consequences such as loss of asset integrity) in contested operational environments (cyberspace or physical).

Restated, Systems Security is about engineering for intended and authorized system behavior and outcomes despite anticipated and unanticipated adversity, conditions that may cause loss (e.g., threats, attacks, hazards, disruptions, exposures). (McEvilley & Winstead 2022)

Note: This is a completely new article inserted in SEBoK 2.9, replacing the previous version by Richard Fairley, Alice Squires, and Keith Willett which originally appeared in SEBoK 1.0 and was periodically updated through SEBoK 2.8.

Overview

Secure systems ideally have three essential characteristics (Ross, Winstead, & McEvilley 2022):

- Enable required system capability delivery despite intentional and unintentional forms of adversity.
- Enforce constraints to ensure only the desired behaviors and outcomes associated with the required capability are realized while realizing the first characteristic.
- Enforce constraints based on a set of rules defining the only authorized interactions and operations allowed to occur while satisfying the second characteristic.

Desired behaviors and outcomes are those reflecting the delivery of the desired system capabilities and features without experiencing loss with undesired consequences, such as loss of information privacy.

While these characteristics are to be achieved to the extent practicable, gaps will occur between the ideal and what can be dependably achieved. A system should be as secure as reasonably practical (ASARP) while meeting minimum stakeholder expectations for security and optimized among other performance objectives and constraints, informed by the principle of commensurate trustworthiness – trustworthy to a level commensurate with the most significant adverse effect resulting from loss or failure. (Hild, McEvilley, & Winstead 2021)

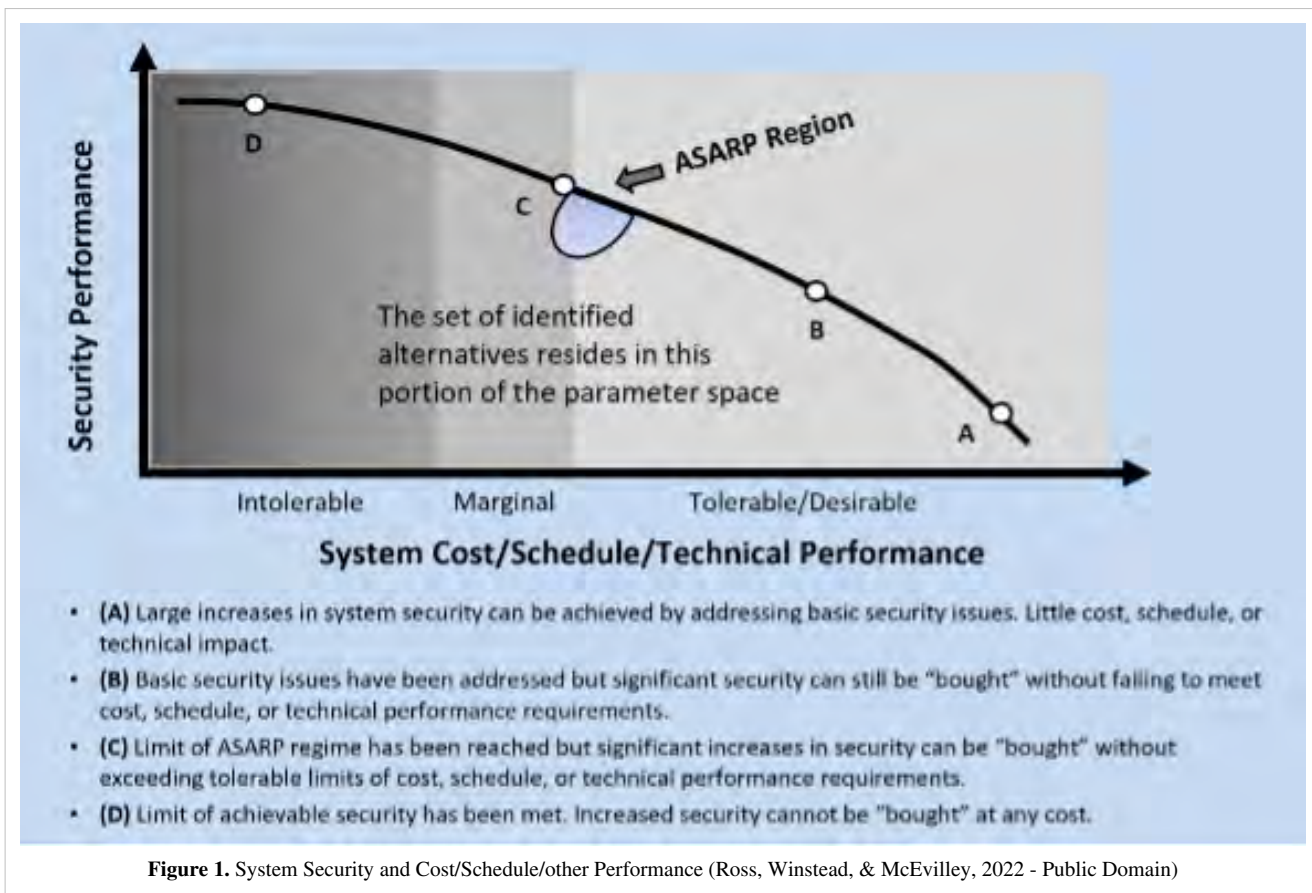
Secure systems ideally have three essential characteristics (Ross, Winstead, & McEvilley 2022):

- Enable required system capability delivery despite intentional and unintentional forms of adversity.
-

- Enforce constraints to ensure only the desired behaviors and outcomes associated with the required capability are realized while realizing the first characteristic.
- Enforce constraints based on a set of rules defining the only authorized interactions and operations allowed to occur while satisfying the second characteristic.

Desired behaviors and outcomes are those reflecting delivery of the desired system capabilities and features without experiencing loss with undesired consequences, such as loss of information privacy.

While these characteristics are to be achieved to the extent practicable, gaps will occur between the ideal and what can be dependably achieved. A system should be as secure as reasonably practical (ASARP) while meeting minimum stakeholder expectations for security and optimized among other performance objectives and constraints, informed by the principle of commensurate trustworthiness – trustworthy to a level commensurate with the most significant adverse effect resulting from loss or failure. (Hild, McEvelley, & Winstead 2021)



To understand the optimization of security among other performance objectives, stakeholders need to be aligned and respectful of each other’s needs. As loss and loss effects or consequences are easily understood, a collaborative understanding of loss tolerances provides a means to alignment as well as forms a basis for metrics across the system lifecycle. (Dove, et al. 2023)

Why Security?

Security, one of 10 acknowledged areas of growing stakeholder expectations in INCOSE’s Systems Engineering Vision 2035, is recognized as needed to become a foundational perspective for system design. (INCOSE 2021) This growing expectation is motivated by increasing cyberspace-based attacks as evidenced by reports such as Security Magazine’s report for computers with Internet access, an attack occurred every 39 seconds in 2017 and has increased since (Security 2017) by observations from the conflict in Ukraine (Carnegie Endowment for International Peace 2023) including attacks on civilian targets such as critical infrastructure, and many well-publicized exploited

vulnerable systems. (Agile IT 2023)

Scope

Adversity, conditions that can cause a loss of assets (e.g., threats, attacks, vulnerabilities, hazards, disruptions, and exposures), occurs throughout the system lifecycle. Such conditions are internal and external to the system, with internal conditions often the result of faults and defects (e.g., missing requirements, implementation errors). Consequently, system security’s scope matches systems engineering’s scope, and every systems engineering process and activity has security considerations. (Ross, Winstead, & McEvelley 2022)

“Unless security is [engineered] into a system from its inception, there is little chance that it can be made secure by retrofit”. (Anderson 1972) Consequently, security must be a foundational perspective from concept exploration.

Assets

An asset is an item of value to a stakeholder. Ross, Winstead, & McEvelley (2022) identified broad asset classes, summarized in Table 1.

Table 1. Common Asset Classes (Ross, Winstead, and McEvelley, 2022 - Public Domain)

Class	Description	Loss Protection Criteria
Material Resources and Infrastructure	Includes - physical property (e.g., buildings, facilities, equipment) - physical resources (e.g., water, fuel). - basic physical and organizational structures and facilities (i.e., infrastructure) needed for an activity or the operation of an enterprise or society. An infrastructure commonly comprised of assets, such as a nation’s national airspace infrastructure, which includes the nation’s airports	<i>Material resources</i> are protected from loss if they are not stolen, damaged, or destroyed or are able to function or be used as intended, as needed, and when needed. <i>Infrastructure</i> is protected from loss if it meets performance expectations while delivering only the authorized and intended capability and producing only the authorized and intended outcomes
System Capability	The set of capabilities and services provided by a system	<i>System capability</i> is protected from loss when it meets its performance expectations while delivering only the authorized and intended capability and producing only the authorized and intended outcomes.
Human Resources	Personnel who are part of the system and personnel affected by the system	<i>Human resources</i> are protected from loss if they are not injured, suffer illness, or killed.
Intellectual Property	Trade secrets, recipes, technology, and other items that constitute an advantage over competitors	<i>Intellectual property</i> is protected from loss if it is not stolen, corrupted, destroyed, copied, substituted in an unauthorized manner, or reverse-engineered in an unauthorized manner.
Data and Information	Includes all types of data and information and all encodings and representations of data and information	<i>Data and information</i> are protected from loss due to unauthorized alteration, exfiltration, infiltration, and destruction.
Derivative Non-Tangible	Includes image, reputation, and trust. Such assets are affected by the success or failure to protect other assets	<i>Non-tangible assets</i> are protected from loss by ensuring the adequate protection of assets in the other classes.

Systems Thinking

Systems thinking is the practice of thinking holistically about systems: relating systems behaviors and concepts to principles based on patterns. It is flexible, conceptual, and strategic in nature: hence generally adapted in systems architecture work. Systems thinking focuses not on immediate cause and effect, but also examines the dynamics of a system to identify secondary effects on behaviors and choices. (Goodman 2018)

Security, especially cybersecurity, has suffered from being treated as a tactics problem, focusing on threat defense and incident response. Security has also become a forensics exercise, steeped in root cause analysis, to add to the known threat defense. Systems thinking realizes the greater objective is assuring a systems' ability to produce the capability (functions and services) that users depend on the system for and contributing to business and mission needs and produce that capability in an acceptable manner (i.e., no harm to stakeholder assets). The need is to focus less on efforts to defend against adversarial action beyond the control of the systems engineer and more on assuring the system performs and protects stakeholder assets, controlling the system from effects of loss, to include avoiding vulnerability that leads to loss when practical. (Young & Leveson 2013) Systems thinking brings security back into the conceptual, design, and implementation of systems capabilities.

Consequently, the need is to focus on a system's trustworthiness rather than singularly focus on risk. (Dove, et al. 2021) Quality evidence such as that generated by verification and validation activities feed assurance arguments that merit trustworthiness, providing a basis for trust by stakeholders. Without such assurance, security functionality is a form of veneer security (Saydjari 2018), providing an unmerited sense of trustworthiness.

Loss

Loss, the experience of having an asset taken away or destroyed or the failure to keep or to continue to have an asset in a desired state or form (Ross, Winstead, & McEvilly 2022), provides language understood by all stakeholders (Dove, et al. 2023). Stakeholder concern is typically the effects of loss caused by adversity, not the adversity itself, and their priorities driven by consequences (e.g., impact to mission). Their needs and requirements can thus be expressed in terms of loss, loss scenarios, loss tolerance, and acceptable loss.

Addressing loss must consider loss results from combinations of adverse events or conditions that cause or lead to unacceptable ramifications, consequences, or impacts. Due to uncertainty (including uncertainty about adversity), guaranteeing a loss will not occur is not possible. The focus must be on controlling loss effects, including cascading or ripple events; e.g., the effect causes additional losses to occur. (Ross, Winstead, & McEvilly 2022)

Loss control objectives frame addressing loss. Loss can be addressed by use of historically-informed practices (known good things to do) and assessing specific loss scenarios for opportunities to address conditions and the potential loss itself.

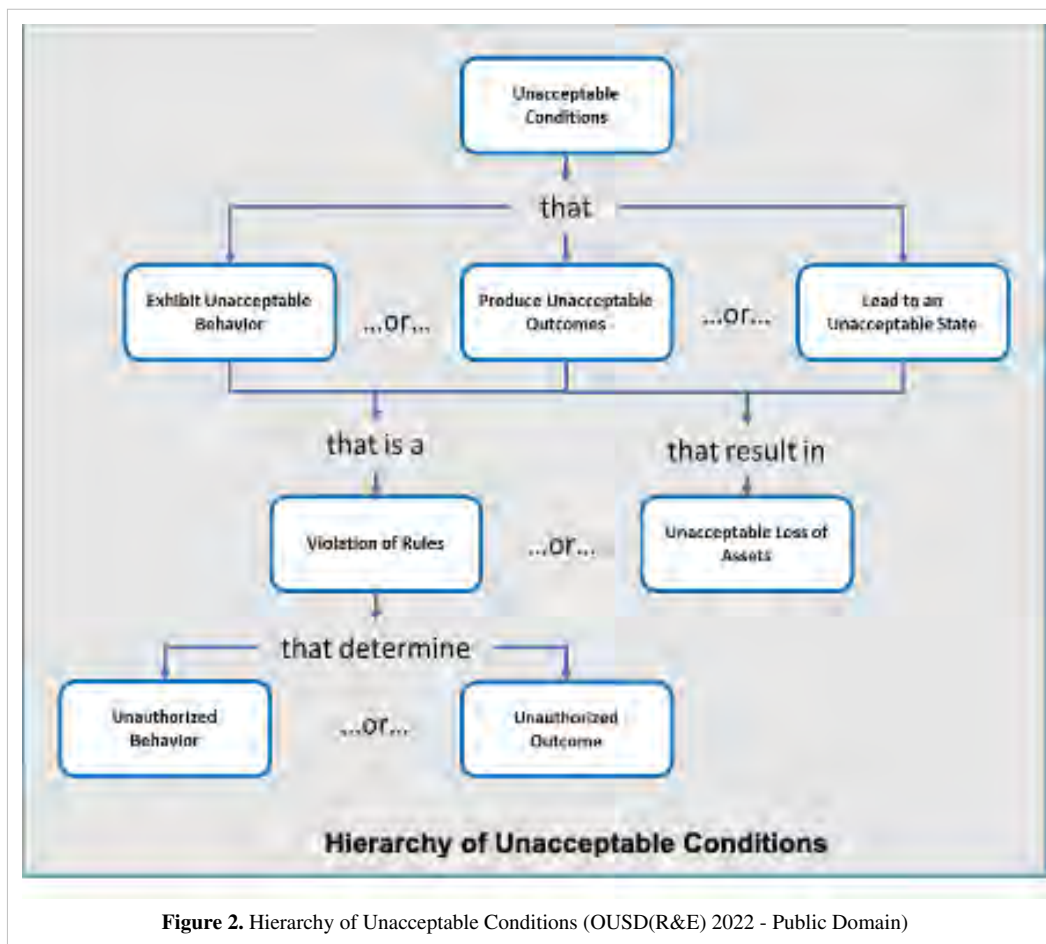
Table 2. Loss Control Objectives (Ross, Winstead, and McEvilly, 2022 - Public Domain)

Loss Control Objective	Discussion
Prevent the Loss from Occurring	*Loss is avoided. Despite the presence of adversity: • The system provides only the intended behavior and produces only the intended outcomes. • Desired properties of the system and assets are retained. • Achieved by combinations of: • Preventing or removing the event or events that cause the loss • Preventing or removing the condition or conditions that allow the loss to occur • Not suffering an adverse effect despite the events or conditions (e.g., fault tolerance)
Limit the Extent of Loss	*Loss can or has occurred. The loss effect extent is to be limited.

- Achieved by combinations of:
 - Limiting dispersion (e.g., propagation, ripple, or cascading effects)
 - Limiting duration (e.g., milliseconds, minutes, days)
 - Limiting capacity (e.g., diminished service or capability)
 - Limiting volume (e.g., bits or bytes of data/information)
- Decisions to limit loss extent may require prioritizing what constitutes acceptable loss across a set of losses (i.e., limiting the loss of one asset requires accepting loss of some other asset).
- Loss recovery and loss delay are two means to limit loss:
 - Loss Recovery: Action is taken by the system or enabled by the system to recover (or allow the recovery of) some or all its ability to function and to recover assets used by the system. Asset restoration can limit the dispersion, duration, capacity, or volume of the loss.
 - Loss Delay: The loss event is avoided until the adverse effect is lessened or when a delay enables a more robust response or quicker recovery.

Loss Scenarios

Loss scenarios describe the events and conditions that lead to unacceptable outcomes. These scenarios do not necessarily lead back to “root causes” in each case, but must capture the internal system (e.g., system states, internal faults) and the external environmental conditions (e.g., loss of power, presence of malicious insider threat) that may lead to a loss, including unauthorized system use (e.g., loss of control). See Figure 2.



Loss scenarios may be analyzed to inform requirements definition and derivation and analyze design alternatives, as well as inform tailoring of historically-informed practice usage.

Enterprise Relationships

Most systems are part of a larger system of systems or enterprises. Adversity often comes through or from these connected systems.

Systems engineering must balance a system’s self-protection capability with opportunities for mutual collaborative protection (Dove, et al. 2021) with trustworthy systems within an enterprise. Enterprises may have dedicated systems for protection, often within network operations and security centers (NOSCs). (Knerler, Parker, & Zimmerman 2022) Systems may also collaborate by sharing situational awareness, with one system alerting others of suspicious activity that may indicate malicious actions others may experience.

Discipline Relationships

Systems Security has commonality and synergy with many other disciplines, such as safety; quality management; reliability, availability, and maintenance (RAM); survivability; operational risk management; and resilience. Overlapping concerns exist with assets, losses, and adversities considered; requirements; and various engineering processes and analyses similarities and opportunities; e.g., System Theoretic Process Analysis, or STPA (Young & Leveson 2013). Some considerations for pursuing these commonalities and synergies were explored in Brtis (2020).

Personnel Considerations

Systems security engineering is a sub-discipline of systems engineering, defined in Ross, Winstead, & McEvelley (2022) as a transdisciplinary and integrative approach to enable the successful secure realization, use, and retirement of engineered systems using systems, security, and other principles and concepts, as well as scientific, technological, and management methods. Systems security engineers are part of the systems engineering teams.

But as security is demonstrated in a system’s behaviors and outcomes, systems engineering responsibility, the systems engineer is ultimately responsible for a system’s security. (Thomas 2013) However, reflecting the maxim “Security is everyone’s job”, all engineering disciplines have security responsibilities (Dove, et al. 2021), not just the systems engineer and various specialists in areas like supply chain assurance, hardware assurance, software assurance, cybersecurity, and physical security.

Security in the Future of Systems Engineering

The INCOSE SE Vision 2035 sets an aim that security “will be as foundational a perspective in systems design as system performance and safety are today”. (INCOSE 2021) To that end and other aims for security within the Vision, the INCOSE Systems Security Engineering Working Group has set objectives and roadmap concepts (Dove 2022).

Roadmap Concept	General Needs to Fill
Security Proficiency in the Systems Engineering Team	System security and its evolution effectively enabled by systems engineering activity.
Education and Competency Development	Education at all levels focused on security of cyber-physical systems.
Stakeholder Alignment	Common security vision and knowledge among all stakeholders.
Loss-Driven Engineering	Standard metrics and abstractions relevant to all system lifecycle phases.
Architectural Agility	Readily composable and re-composable security with feature variants.
Operational Agility	Ability for cyber-relevant response to attack and potential threat; resilience in security system.
Capability-Based Security Engineering	Top-down approach to security starting with desired results/value.
Security as a Functional Requirement	Systems engineering responsibility for the security of systems.

Modeled Trustworthiness	Reinvigorate formal modeling of system trust as a core aspect of system security engineering; address issues of scale with model-based tools and automation.
Security Orchestration	Tightly coupled coordinated system defense in cyber-relevant time.
Collaborative Mutual Protection	Augmented detection and mitigation of known and unknown attacks with components collaborating for mutual protection.

Challenges

The challenging problems for system security are numerous. Some of this is a result of neglecting the addressing of security early in the system life cycle as recognized by the Systems Engineering Vision call for security to be a foundational perspective – as Carl Landwehr wrote “This whole economic boom in [security] seems largely to be a consequence of poor engineering” (Landwehr 2015).

For example, system of systems engineering security faces challenges in part to not knowing or trusting the component systems which may not have been engineered with security in mind, or at least did not consider documenting the evidence that informs trustworthiness. Another system of systems challenge comes as formerly isolated cyber-physical systems (CPSs) are increasingly connected to form larger system of systems, negating assumptions impacting security if security was considered at all. Additionally, legacy CPSs were not built thinking of the need to update the software; i.e. use sustainable security (Rosser 2023)), which creates a challenge in upgrading software to reflect revised assumptions and security models for the CPS in question.

Another challenge comes with the advancing use of Artificial Intelligence (AI). Any new technology presents security challenges until its usage matures, but AI introduces complexity and decreases predictability (INCOSE 2021). Managing complexity and uncertainty is necessary for security (Ross, Winstead, & McEvelley 2022), and AI increasing of both compounds the problem space for systems engineering security.

References

Works Cited

- Agile IT. n.d.. The Top 10 Biggest Cyberattacks of 2022. Accessed September 15, 2023. Available at <https://www.agileit.com/news/biggest-cyberattacks-2022/>.
- Anderson, J. 1972. Computer Security Technology Planning Study, Technical Report ESD-TR-73-51. October 1, 1972. Hanscom AFB: Air Force Electronic Systems Division. Accessed September 15, 2023. Available at <https://apps.dtic.mil/sti/citations/AD0758206>.
- Brtis, J. (ed). 2020. Loss-Driven Systems Engineering. INCOSE Insight, 23(4):7-8. Wiley. Accessed September 15, 2023. Available at <https://incose.onlinelibrary.wiley.com/doi/abs/10.1002/inst.12312>.
- Carnegie Endowment for International Peace. n.d.. Cyber Conflict in the Russia-Ukraine War. Accessed September 15, 2023. Available at <https://carnegieendowment.org/programs/technology/cyberconflictintherussiaukrainewar/>.
- Dove, R. 2022. “Setting Current Context for Security in the Future of Systems Engineering”. INCOSE Insight, 25(2):8-10.
- Dove, R., K. Willett, T. McDermott, H. Dunlap, H. D.P. MacNamara, and C. Ocker. 2021. “Security in the Future of Systems Engineering (FuSE), a Roadmap of Foundational Concepts”. Proceedings of the 31st INCOSE International Symposium, July 17-22. Virtual only,
- Dove, R., M. Winstead, H. Dunlap, M. Hause, A. Scalco, A. Scalco, A. Williams, and B. Wilson. 2023. “Democratizing Systems Security”. Proceedings of the 33rd INCOSE International Symposium. July 15-20. Honolulu, Hawaii: Wiley.

- Goodman, M. 2018. Systems Thinking: What, Why, When, Where, and How? Accessed September 15, 2023. Available at <https://thesystemsthinker.com/systems-thinking-what-why-when-where-and-how/>
- Hild, D., M. McEvilley, and M. Winstead. 2021. Principles for Trustworthy Design of Cyber-Physical Systems. MITRE Technical Report, MTR210263.
- INCOSE. 2021. INCOSE Systems Engineering Vision 2035. Accessed September 15, 2023. Available at <https://www.incose.org/about-systems-engineering/se-vision-2035>
- Knerler, K., I. Parker, and C. Zimmerman. 2022. 11 Strategies of a World-Class Cybersecurity Operations Center (2nd ed.). McLean, VA: MITRE. Accessed September 15, 2023. Available at <https://www.mitre.org/sites/default/files/2022-04/11-strategies-of-a-world-class-cybersecurity-operations-center.pdf>
- Landwehr, C. 2015. "We Need a Building Code for Building Code". *Communications of the ACM*, 58(2):24-26. Accessed September 15, 2023. Available at doi:<https://doi.org/10.1145/2700341>
- McEvilley, M., and M. Winstead. 2022. "Functionally Interpreting Security". *INCOSE Insight*, 25(2):15-17. Wiley. Accessed September 15, 2023. Available at <https://incose.onlinelibrary.wiley.com/doi/abs/10.1002/inst.12380>
- OSD(R&E). 2022. Security and Resilience Interpretation 1.0. Prepared by MITRE. Accessed September 2023. Available at <https://www.crs-bok.org/asset/83a0d3528e6e27272a52b7e2c3758facc16773fd>
- Ross, R., M. Winstead, M., M. McEvilley. 2022. Engineering Trustworthy Secure Systems. NIST SP 800-160 Volume 1 Revision 1. Gaithersburg, MD: NIST. Accessed September 15, 2023. Available at <https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final>
- Rosser, L.A. 2023. "Applying Agility for Sustainable Security". *INCOSE Insight*, 26(2):45-52. Wiley. Accessed September 15, 2023. Available at <https://incose.onlinelibrary.wiley.com/doi/abs/10.1002/inst.12445>
- Saydjari, O.S. (ed.). 2018. Engineering Trustworthy Systems: Get Cybersecurity Design Right the First Time. New York: McGraw Hill. Accessed September 15, 2023. Available at <https://www.amazon.com/Engineering-Trustworthy-Systems-Cybersecurity-Design/dp/1260118177>
- Security. 2017. Hackers Attack Every 39 Seconds. Accessed September 15, 2023. Available at <https://www.securitymagazine.com/articles/87787-hackers-attack-every-39-seconds>
- Thomas, J.A. 2013. "Critical System Behaviors of The Future". *INCOSE Insight*, 16(2):3-5. Wiley.
- Young, W. and N. Leveson. 2013. "Systems Thinking for Safety and Security". *Proceedings of the 29th Annual Computer Security Applications Conference (ACSAC '13)*, pp. 31-35. Accessed September 15, 2023. Available at <https://dspace.mit.edu/handle/1721.1/96965>

Primary References

- Anderson, R. 2020. Security Engineering: A Guide to Building Dependable Distributed Systems (3rd Edition). Wiley. Accessed September 15, 2023. Available at <https://www.amazon.com/Security-Engineering-Building-Dependable-Distributed/dp/1119642787>
- Neumann P. 2004. Principled Assuredly Trustworthy Composable Architectures, CDRL A001 Final Report, SRI International, Menlo Park, CA. Accessed September 15, 2023. Available at <https://ieeexplore.ieee.org/document/1335465>
- Ross, R., M. Winstead, M., and M. McEvilley. 2022. Engineering Trustworthy Secure Systems. NIST SP 800-160 Volume 1 Revision 1. Gaithersburg, MD: NIST. Accessed September 15, 2023. Available at <https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final>

Additional References

Avizienis A., J. Laprie, B. Randell. and C. Landwehr C. "Basic Concepts and Taxonomy of Dependable and Secure Computing". IEEE Transactions on Dependable and Secure Computing 1(1):11-33. Accessed September 15, 2023. Available at <https://www.csl.sri.com/users/neumann/chats4.pdf>

DoD. 1983. DoD Standard 5200.28-STD Trusted Computer System Evaluation Criteria. Accessed September 15, 2023. Available at <https://www.csl.sri.com/users/neumann/chats4.pdf>

National Security Agency. 2002. Information Assurance Technical Framework (IATF), Release 3.1. Accessed September 15, 2023. Available at <https://ntrl.ntis.gov/NTRL/dashboard/searchResults/titleDetail/ADA606355.xhtml>

Schroeder M.D., D.D. Clark, and J.H. Saltzer. 1977. "The Multics Kernel Design Project". Proceedings of Sixth ACM Symposium on Operating Systems Principles. Accessed September 15, 2023. Available at <https://dl.acm.org/doi/pdf/10.1145/800214.806546>

Videos

Keith Willett previously authored a series of videos that explain aspects of systems security. They are not referenced in the above article, but remain quite relevant and interesting viewing.

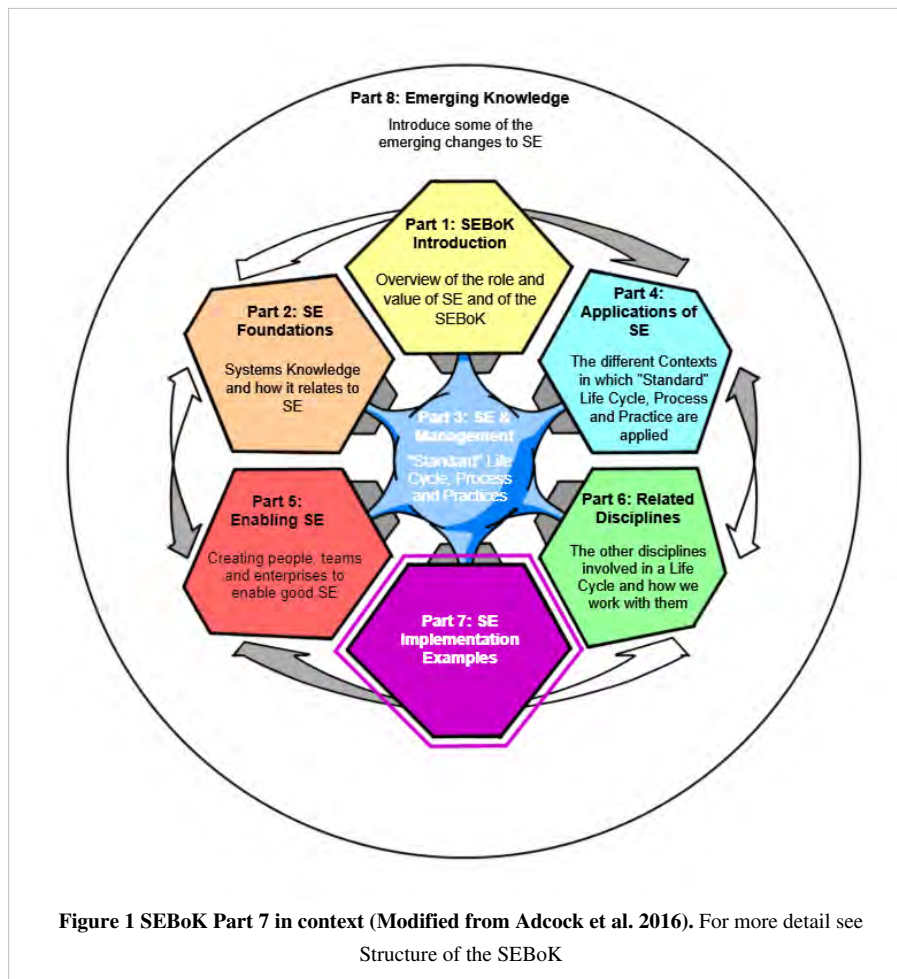
Part 7: SE Implementation Examples

Systems Engineering Implementation Examples

Contents of this Part

- Matrix of Implementation Examples (Heidi Davidz, Alice Squires, and Tom Hilburn) (Alex Lee, John Brackett, and Richard Turner)
- Implementation Examples
- Defense System Examples
- Information System Examples
- Management System Examples
- Medical System Examples
- Space System Examples
- Transportation System Examples
- Utilities Examples
- Lead Authors:
- Heidi Davidz and Richard Turner

Part 7 is a collection of systems engineering (SE) implementation examples to illustrate the principles described in the Systems Engineering Body of Knowledge (SEBoK) Parts 1-6. These examples describe the application of SE practices, principles, and concepts in real settings.



The intent is to provide typical instances of the application of systems engineering (SE) and relate these to key SE principles and concepts from the rest of the SEBoK. This can improve the practice of SE by illustrating to students, educators, and practitioners the benefits of effective practice, as well as the risks and liabilities of poor practice.

A published case study will typically describe aspects of the practice of SE in a particular situation and then provide comments and critique of that practice. Where possible, examples in the SEBoK refer to published case studies and relate the discussions in them to appropriate areas of the SEBoK. In some case, good or bad examples of SE practice are available but have not been documented in a case study. In these cases the SEBoK authors have described and commented on these examples directly.

A matrix of implementation examples is used to map these examples to main topics in the SEBoK which they cover.

More examples will be added over time to highlight the different aspects and applications of SE. In addition, new examples can be added to demonstrate the evolving state of practice, such as the application of model-based SE and the engineering of complex, adaptive systems.

Value of Implementation Examples

Learning from critical examples has been used for decades in medicine, law, and business to help students learn fundamentals and to help practitioners improve their practice. A Matrix of Implementation Examples is used to show the alignment of systems engineering case studies to specific areas of the SEBoK. This matrix is intended to provide linkages between each implementation example to the discussion of the systems engineering principles illustrated. The selection of examples covers a variety of sources, domains, and geographic locations. Both effective and ineffective use of systems engineering principles are illustrated.

The United States Air Force Center for Systems Engineering (AF CSE) has developed a set of case studies "to facilitate learning by emphasizing the long-term consequences of the systems engineering/programmatic decisions on cost, schedule, and operational effectiveness." (USAF Center for Systems Engineering 2011) The AF CSE is using these cases to enhance SE curriculum. The cases are structured using the Friedman-Sage framework (Friedman and Sage 2003; Friedman and Sage 2004, 84-96), which decomposes a case into contractor, government, and shared responsibilities in the following nine concept areas:

1. Requirements Definition and Management
2. Systems Architecture Development
3. System/Subsystem Design
4. Verification/Validation
5. Risk Management
6. Systems Integration and Interfaces
7. Life Cycle Support
8. Deployment and Post Deployment
9. System and Program Management

This framework forms the basis of the case study analysis carried out by the AF CSE. Two of these case studies are highlighted in this SEBoK section, the Hubble Space Telescope Case Study and the Global Positioning System Case Study.

The United States National Aeronautics and Space Administration (NASA) has a catalog of more than fifty NASA-related case studies (NASA 2011). These case studies include insights about both program management and systems engineering. Varying in the level of detail, topics addressed, and source organization, these case studies are used to enhance learning at workshops, training, retreats, and conferences. The use of case studies is viewed as important by NASA since "organizational learning takes place when knowledge is shared in usable ways among organizational members. Knowledge is most usable when it is contextual" (NASA 2011). Case study teaching is a method for sharing contextual knowledge to enable reapplication of lessons learned. The MSTI Case Study is from this catalog.

References

Works Cited

- Adcock, R., N. Hutchison, C. Nielsen, 2016, "Defining an architecture for the Systems Engineering Body of Knowledge," Annual IEEE Systems Conference (SysCon) 2016.
- Friedman, G.R., and A.P. Sage. 2003. *Systems Engineering Concepts: Illustration Through Case Studies*.
- Friedman, G.R., and A.P. Sage. 2004. "Case Studies of Systems Engineering and Management in Systems Acquisition." *Systems Engineering*. 7 (1): 84-96.
- NASA. 2011. *A Catalog of NASA-Related Case Studies*. Goddard Space Flight Center: Office of the Chief Knowledge Officer, National Aeronautics and Space Administration (NASA). Updated June 2011. Accessed September 2011. Available: https://www.nasa.gov/centers/goddard/pdf/450420mainNASA_Case_Study_Catalog.pdf.
- United States Air Force (USAF) Center for Systems Engineering. 2011. *Why Case Studies?*. Wright-Patterson Air Force Base, Ohio, USA: Air Force Institute of Technology (AFIT), US Air Force. Accessed September 2011. Available: <https://www.afit.edu/cse/cases.cfm>.

Primary References

Friedman, G., and A.P. Sage. 2004. "Case studies of systems engineering and management in systems acquisition". *Systems Engineering* 7(1): 84-96.

Gorod, A., B.E. White, V. Ireland, S.J. Gandhi, and B.J. Sauser. 2014. *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*. Boca Raton, FL: CRC Press, Taylor & Francis Group.

NASA. A Catalog of NASA-Related Case Studies. Greenbelt, MD, USA: Office of the Chief Knowledge Officer, Goddard Space Flight Center, National Aeronautics and Space Administration (NASA). Updated June 2011. Accessed December 5, 2014 at NASA https://www.nasa.gov/centers/goddard/pdf/450420mainNASA_Case_Study_Catalog.pdf.

United States Air Force (USAF) Center for Systems Engineering. 2011. Why Case Studies?. Wright-Patterson Air Force Base, OH, USA: Air Force Institute of Technology (AFIT).

Additional References

None.

Matrix of Implementation Examples

- Lead Authors:
- Heidi Davidz, Alice Squires, and Tom Hilburn
- Contributing Authors:
- Alex Lee, John Brackett, and Richard Turner

The following matrix maps the Systems Engineering Implementation Examples to topics in the Systems Engineering Body of Knowledge (SEBoK). It provides both a list of systems engineering implementation examples for topics of interest, and a list of relevant topics for each implementation example. Since the number of topics in the SEBoK is extensive, only a subset is included here for clarity. For additional information, see the example of interest and the corresponding SEBoK topic.

Organization and Mapping of Examples to the SEBoK

The following short titles shown in Table 1 are used for the implementation examples:

Table 1. Short Titles for the SEBoK Examples. (SEBoK Original)

Case Studies	
BT	Business Transformation
ATC	NextGen Air Traffic Control
NASA	NASA's Mission to Saturn
HST	Hubble Space Telescope
GPS	Global Positioning System
GPS II	Global Positioning System II
Radiation	Medical Radiation
FBI VCF	FBI Virtual Case File System
MSTI	Miniature Seeker Technology Integration
Infusion Pump	Next Generation Medical Infusion Pump

DfM	Design for Maintainability
CAS	Complex Adaptive Operating System
PM	Project Management
TS	Taxi Service
SWFTS	SWFTS MBSE
Bag Handling	Denver Airport Baggage Handling System
VA Sub	Virginia Class Submarine
Route Mod	UK West Coast Route Modernisation Project
Water Mgmt	Singapore Water Management
FAA AAS	FAA Advanced Automation System
Light Rail	Standard Korean Light Transit System
TMT	Thirty-Meter Telescope

Table 2 shows how the topics (each row) align with the first set of implementation examples (each column):

Table 2. Implementation Examples based on published case studies. (SEBoK Original)

SEBoK Topic	BT	ATC	NASA	HST	GPS	GPS II	Radiation	FBI VCF	MSTI	Infusion Pump	DfM	CAS	PM	TS	SWFTS	NHS
Systems Thinking					X	X		X	X	X	X	X	X	X		
Models and Simulation		X	X			X							X	X	X	
Product Systems Engineering				X	X		X	X								X
Service Systems Engineering			X			X								X		X
Enterprise Systems Engineering	X	X	X			X							X	X		X
Systems of Systems (SoS)	X	X				X								X		X
Life Cycle Models	X			X	X				X	X	X	X				X
Business or Mission Analysis	X	X				X		X		X			X	X		
Stakeholder Needs Definition	X	X		X		X		X		X			X	X	X	
System Requirements				X	X			X		X			X	X	X	
System Architecture Design Definition	X	X			X	X			X	X						
System Analysis			X		X							X	X	X		
System Implementation					X						X			X		
System Integration	X		X	X	X	X			X		X		X	X		
System Verification			X		X		X		X	X						
System Validation	X				X	X	X		X	X						

System Deployment			X	X						X				
Operation of the System			X	X					X	X		X		X
System Maintenance														X
Logistics														
Project Planning	X	X		X				X		X		X		
Project Assessment and Control						X	X	X				X	X	
Risk Management			X	X	X	X	X		X		X	X		
Measurement								X						
Decision Management	X							X			X	X	X	
Configuration Management				X	X			X		X	X			
Information Management			X								X	X	X	
Quality Management														X
Enabling Systems Engineering			X		X	X		X	X	X	X	X		X
Related Disciplines					X	X	X		X		X			

Table 3 shows how the topics (each row) align with the second set of implementation examples (each column):

Table 3. Implementation Examples Developed for SEBoK. (SEBoK Original)

SEBoK Topic (Part 3)	Bag Handling	VA Sub	Route Mod	Water Mgmt	FAA AAS	Light Rail
Business or Mission Analysis			X	X		X
Stakeholder Needs Definition			X	X		X
System Requirements			X		X	X
System Architecture Design Definition	X	X		X	X	X
System Analysis	X			X	X	X
System Implementation						X
System Integration	X			X		X
System Verification	X				X	X
System Validation	X				X	X
System Deployment						X
Operation of the System				X		X
System Maintenance						X
Logistics	X					
Project Planning	X		X	X	X	X
Project Assessment and Control					X	X
Risk Management	X		X	X	X	X
Measurement						X
Decision Management	X		X	X		

Configuration Management	X	X	X
Information Management		X	
Quality Management			X

References

Works Cited

None.

Primary References

None.

Additional References

None.

Implementation Examples

Characteristics of the organization(s) which perform SE have an impact on the engineering itself. It is for this reason that the examples are grouped into three categories: government examples, commercial examples, and examples which represent issues from both ("combined"). As the SEBoK continues to evolve, additional categorizations may be developed. A matrix is used to map the specific systems engineering principles illustrated in each example to the associated SEBoK topics.

SEBoK Examples

- Construction System Examples - **Coming Soon!**
 - CyberPhysical System Examples - **Coming Soon!**
 - Defense System Examples
 - Submarine Warfare Federated Tactical Systems
 - Virginia Class Submarine
 - Geospatial System Examples - **Coming Soon!**
 - Information System Examples
 - Complex Adaptive Taxi Service Scheduler
 - Successful Business Transformation within a Russian Information Technology Company
 - FBI Virtual Case File System
 - Management System Examples
 - Project Management for a Complex Adaptive Operating System
 - Medical System Examples
 - Next Generation Medical Infusion Pump
 - Medical Radiation
 - Design for Maintainability
 - Space System Examples
 - Global Positioning System Case Study
 - Global Positioning System Case Study II
-

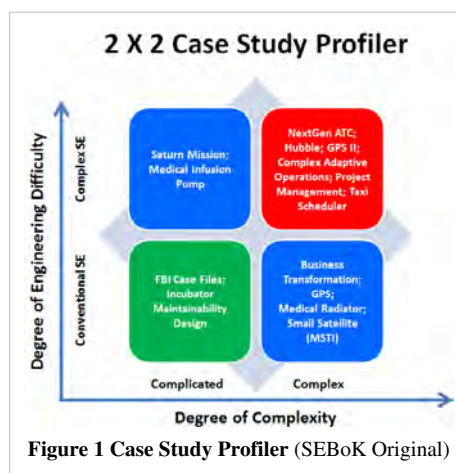
- Russian Space Agency Project Management Systems
- How Lack of Information Sharing Jeopardized the NASA/ESA Cassini/Huygens Mission to Saturn
- Hubble Space Telescope
- Applying a Model-Based Approach to Support Requirements Analysis on the Thirty-Meter Telescope
- Miniature Seeker Technology Integration Spacecraft
- Apollo 1 Disaster
- Transportation System Examples
 - Denver Airport Baggage Handling System
 - FAA Advanced Automation System (AAS)
 - Federal Aviation Administration Next Generation Air Transportation System
 - UK West Coast Route Modernisation Project
 - Standard Korean Light Transit System Vignette
- Utilities Examples
 - Northwest Hydro System
 - Singapore Water Management

Characterization of Examples

Systems engineering (SE) principles described in the SEBoK Parts 1-6 are illustrated in Part 7, Systems Engineering Implementation Examples. These examples describe the application of systems engineering practices, principles, and concepts in real settings and can be used to improve the practice of systems engineering by illustrating to students, practitioners, and those new to SE the benefits of effective practice and the risks of poor practice.

The examples here have been developed by examining previously published case studies from external sources that demonstrate the real-world examples of systems engineering principles. These case studies were then summarized by the BKCASE team. The summaries include links to the original documentation as well as links to the relevant areas of the SEBoK highlighted in the example.

Systems engineering (SE) examples can be characterized in terms of at least two relevant parameters, viz., their degrees of complexity and engineering difficulty. Although a so-called quad chart is likely an oversimplification, a 2 x 2 array can be used to make a first-order characterization, as shown in Figure 1.



The x-axis depicts complicated, the simplest form of complexity, at the low-end on the left, and complex, representing the range of all higher forms of complexity on the right. The y-axis suggests how difficult it might be to engineer (or re-engineer) the system to be improved, using Conventional (classical or traditional) SE, at the low-end on the bottom, and Complex SE, representing all more sophisticated forms of SE, on the top. This upper range is intended to cover system of systems (SoS) engineering (SoSE), enterprise systems engineering (ESE), as well as

Complex SE (CSE). The distinctions among these various forms of SE may be explored by visiting other sections of the SEBoK. In summary, the SEBoK case study editors have placed each case study in one of these four quadrants to provide readers with a suggested characterization of their case study's complexity and difficulty. For sake of compactness the following abbreviations have been used:

- Business Transformation (Successful Business Transformation within a Russian Information Technology Company)
- NextGen ATC (Federal Aviation Administration Next Generation Air Transportation System)
- Saturn Mission (How Lack of Information Sharing Jeopardized the NASA/ESA Cassini/Huygens Mission to Saturn)
- Hubble (Hubble Space Telescope Case Study)
- GPS and GPS II (Global Positioning System Case Study)
- Medical Radiator (Medical Radiation Case Study)
- FBI Case Files (FBI Virtual Case File System Case Study)
- Small Satellite MSTI (MSTI Case Study)
- Medical Infusion Pump (Next Generation Medical Infusion Pump Case Study)
- Incubator Maintainability Design (Design for Maintainability)
- Complex Adaptive Operations (Complex Adaptive Operating System)
- Taxi Scheduler (The Development of the First Real-Time Complex Adaptive Scheduler for a London Taxi Service)
- Project Management (The Development of a Real-Time Complex Adaptive Project Management System)
- SWFTS MBSE (Submarine Warfare Federated Tactical Systems Case Study)

References

Works Cited

None.

Primary References

None.

Additional References

None.

Defense System Examples

Submarine Warfare Federated Tactical Systems

This article describes the transformation of the systems engineering and integration program that produces the common combat system used across the United States Navy (USN) submarine fleet from traditional document-based systems engineering (DBSE) to model-based systems engineering (MBSE).. The topic may be of particular interest to those dealing with programs in the sustainment and evolution phase of their life cycle. For addition information, refer to the links provided in Section V, Lessons Learned below.

Background

Modern submarines are typically in service for 20 - 40 years. Historically, each new class of submarines has been equipped with a new combat system, with a corresponding logistics and sustainment tail unique to that class. Submarines and their internal systems are commonly state-of-the-practice at launch, but most navies find it necessary to upgrade the ship's combat system at least once during the operational lifetime. The evolution of threats, technology and interoperability drives the USN to upgrade their submarine combat systems and key components including the sonar (Fages 1998) (Ford and Dillard 2009) and tactical control systems continuously (Jacobus and Barrett 2002).

Over the last three decades submarine combat systems have evolved from multiple independent systems (sonar, combat control, imaging, electronic warfare, weapon control, etc.) with manual or point-to-point interfaces into networked federations of systems (FoS). Confusingly, these component systems are often referred to as subsystems in the literature.

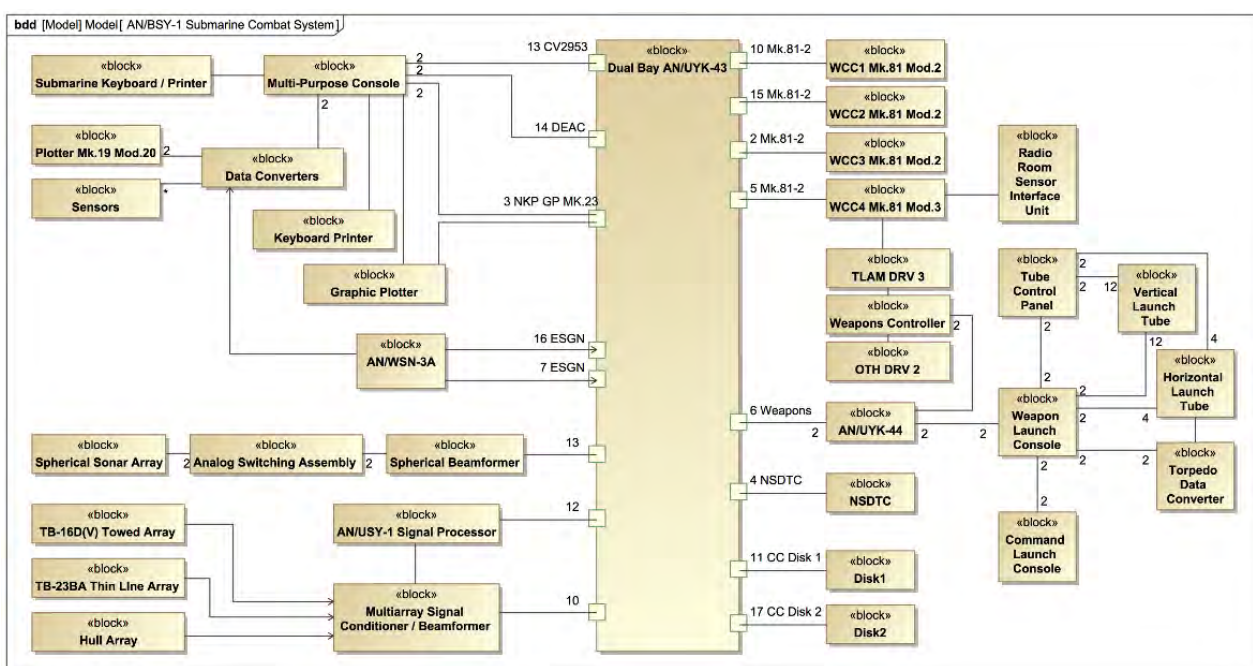


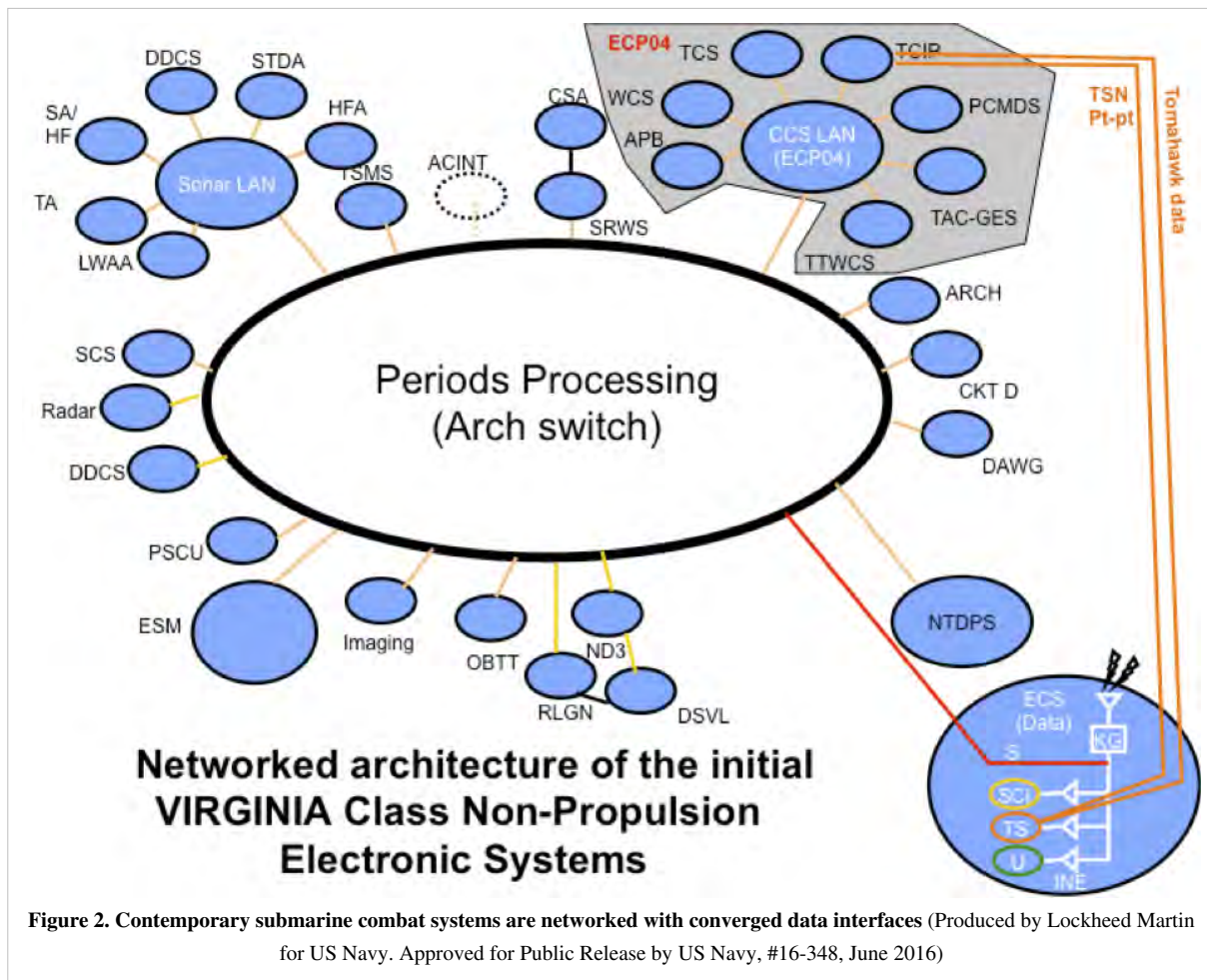
Figure 1. 1985-era submarine combat system composed from independent component systems with point-to-point interfaces (SEBoK Original)

In the USN, each of these component systems has its own acquisition program, customer, and contractor team. Starting as legacy military systems hosted on traditional military-unique computational platforms, these systems have evolved to utilize Commercial Off-The-Shelf (COTS) computational and networking platforms, and leverage large amounts of COTS software.

As the component systems became more tightly interconnected, the acquisition customers established and collaboratively funded a systems engineering and integration (SE&I) program to manage the interfaces between systems, manage technology insertion and the obsolescence of common COTS components, and to integrate and test the production systems (Cooper, Sienkiewicz and Oliver 2006). Starting with the Virginia class, this SE&I program was expanded to encompass both new-production and in-service submarine modernization efforts. Over time, the combat systems of the various USN submarine classes were converged into variants of a single product line (Zingarelli, et al. 2010).

Purpose

The submarine combat system SE&I program delivers an updated production baseline annually, along with product line variants for each submarine class or subclass being built or upgraded that year. Production systems implementing this baseline are delivered to new-build submarines, and to in-service submarines being upgraded on a roughly six-year cycle. The common combat system product line is referred to as the Submarine Warfare Federated Tactical Systems (SWFTS). SWFTS is deployed by the USN on submarines of the Los Angeles (SSN 688), Ohio (SSGN 726, SSBN 730), Seawolf (SSN 21), and Virginia (SSN 774) classes, and by the Royal Australian Navy on the Collins (SSG 73) class. SWFTS is also planned for the next-generation USN Columbia (SSBN) class. Compared to the submarine combat systems that it replaced, SWFTS significantly reduces development, maintenance and training costs while delivering enhanced combat capabilities and facilitating the rapid insertion of new or improved capabilities (Zingarelli, et al. 2010).



Challenges

The USN submarine fleet encompasses substantial platform variability between class, sub-classes, and even individual ships within a sub-class. The RAN Collins class contributes additional variability. Platform variability drives combat system variability.

SWFTS is a Federation of Systems, with each platform hosting a subset of 40 systems produced by 20 different program offices. As is common with system of systems (SoS) and FoS, there is no central program office that can command the compliance of all of the component system programs. Instead, the evolution of SWFTS is executed through negotiation and consensus.

Many baselines must be produced each year: new common hardware baselines are introduced in odd years, while new common software baselines are introduced in even years (Jacobus, Yan and Barrett 2002). In addition, multiple incremental developmental baselines are established each year. Once the annual production baseline for the product line is defined, variants must be developed for each submarine class or subclass built or upgraded that year (Mitchell 2012).

Like most other defense programs, the SWFTS SE&I program is under constant pressure to accomplish more with decreasing resources. There has been steady increase in SE scope despite decreasing budgets. Program leadership has responded in part through continuous SE process improvement. Improvements have included test automation, changes in the requirements management processes and tools (spreadsheets to IBM® Rational® DOORS® to OMG® SysML®), refined tooling for change management, and the DBSE to MBSE transition that is the focus of this case study. Substantial Return on Investment (ROI) has been achieved with each major SE process or tooling improvement (Mitchell 2014, Rogers III and Mitchell 2021).

Systems Engineering Practices

During 2009 the SWFTS SE&I program conducted a Model Driven Architecture (MDA) study to determine if MDA should be the next step in the program's continuous SE process improvement. The MDA study predicted a positive ROI from converting to MBSE. In January 2010 the SWFTS SE&I program office kicked-off a three-year effort to develop and validate a SWFTS FoS model and MBSE process. In 2013 a SWFTS baseline was developed using both DBSE and MBSE in parallel. The DBSE products were used to validate the MBSE products. Based on that successful validation, the SWFTS SE&I program transitioned to MBSE for all ongoing work.

Until the transition in 2013, SWFTS SE was performed using traditional DBSE. Requirements were managed in DOORS, and reviewed and used by engineers in the form of massive spreadsheets with hundreds of columns and thousands of rows. The design of each variant was documented in Microsoft Office files. Baseline Change Requests (BCR) were documented in briefings, and analyzed by all component system programs in parallel for potential impact. Approved BCRs were manually merged into DOORS and into revised baseline documents for each effected variant.

Starting in 2010, the customer community invested in a three-year MBSE transformation effort. The engineering team performed an in-depth tool trade study to select and set up the MBSE environment. That trade study resulted in the selection of MagicDraw™ for system modeling, using Teamwork™ as the model repository.

Once the modeling environment was installed, the MBSE transformation team architected, developed and populated a SysML-based model of the SWFTS FoS interfaces. The team updated the SWFTS SE process to take advantage of the new MBSE environment, with the constraint that the MBSE process produce SE products that were effectively identical to those produced by the DBSE process.

As the SWFTS model was developed, unanticipated benefits emerged. Capturing the architecture in a model improves the depth and quality of baseline products due to the fact that, unlike the spreadsheets it replaced, the model inherently captures relationships between elements. Using the model, one can drill down and explore various aspects of the architecture, e.g., a) the network design that supports data exchanges; b) component systems that use a particular network Virtual Local Area Network (VLAN); c) service level requirements levied upon data providers.

In 2013 the new MBSE environment and process was used to produce a set of SWFTS baseline SE products. In parallel, the DBSE process produced equivalent products. The two sets of baseline products were compared in detail, with all differences traced back to root causes.

The relationships in the model also support automated integrity and consistency checking between model elements, which is also called model concordance. Analysis of these relationships identified a significant number of minor differences that were all traced to errors in the DBSE products. This validated the MBSE process, and demonstrated that while those initial MBSE baseline products were more labor-intensive than the DBSE baseline products, the new MBSE process produced higher quality products. After this validation, the SWFTS SE&I program switched over to MBSE as their basic process.

Since that transformation, SE process improvement has continued apace. Requirements management has moved from DOORS to the system model in MagicDraw. As of 2016, the system model is the baseline for requirements, architecture and the FoS design. BCR impact analysis is now performed in model, leveraging capabilities of the toolset for automated assistance. Variants are documented in the system model as system configurations. Most SE products are generated from the system model on demand.

The MBSE process has been matured and refined over subsequent baseline development efforts. As the team climbed the learning curve, the average cost of processing a BCR declined. New SE artifacts capturing specific aspects of the FoS model were conceived and auto-generated from the model to improve communication with targeted audiences. Where before the transition to MBSE it was not cost-effective to manually generate tailored SE artifacts for individual systems, developing scripts to auto-generate tailored artifacts proved cost effective and improved the efficiency of the component system developers. Additional scripts were developed to tailor the

modeling tool user interface and focus it on the specific activities performed by the SWFTS SE&I engineers. MagicDraw™ has very extensive modeling capabilities with a correspondingly expansive user interface, and tailoring the user interface both reduced the learning curve for new engineers coming onto the program and streamlined routine SWFTS SE activities.

While the initial scope of MBSE was limited to managing the interfaces between component systems, once the transition was successful MBSE started expanding out to encompass additional SWFTS SE&I tasks. MBSE is now beginning to spread into the component system programs as well as the overall submarine combat system SE&I program.

Return on Investment Achieved by Transitioning to MBSE

After the MBSE process reached a reasonable level of maturity, a retrospective analysis (Rogers III and Mitchell 2021) was performed to determine if the anticipated cost savings had been achieved. This analysis compared the requirements database-managed 2010 baselines, which were generated using the mature common requirements baseline and common change management process institutionalized by 2008, with the 2014 baselines built using the mature MBSE process. This analysis made a quantitative comparison of the efficiencies between the legacy interface requirements management process using the IBM DOORS® toolset, and the model-based interface requirements management process employing the MagicDraw™ toolset.

These two distinct SWFTS baseline updates provided a good case study as both

- utilized the same high-level SWFTS process and contract,
- involved updating the lead boat in a new group of submarines added to the SWFTS SE&I program, and
- involved updates to a similar number of submarine classes.

The second bullet is key, since it suggests that the overall level of complexity of the requirements changes is similar between the two sets of baselines.

The 2010 baseline developed using DOORS® will be referred to hereafter as the Legacy Process Baseline. The 2014 baseline developed using MBSE with the MagicDraw™ toolset will henceforth be referred to as the MBSE Baseline. The timing of these two baselines in the context of the MBSE transition is shown in Figure 3.

Figure 3. Timing of the Legacy Process Baseline and the MBSE Baseline relative to other SE process improvement. (Rogers III and Mitchell 2021)

Since a requirement modification is the basic unit of systems engineering work depicting development needs at the FoS level, it is a useful metric to compare the scope of the updates. As can be seen from Table 1, the MBSE Baseline involved 42% more interface requirements changes than the Legacy Process, while consuming only 16% more hours. Another way of looking at these numbers is that the SE hours per requirement decreased from 12.1 in the Legacy baseline to 9.9 in the MBSE baseline. This is equivalent to saying that the MBSE process is 18% more efficient than the older DBSE process. This exceeded the 13% improvement projected by the 2009 SWFTS pilot study (Mitchell 2014).

Table 1. Summary of the SWFTS MBSE ROI Analysis Baselines. (Rogers III and Mitchell 2021)

In addition to the decrease in labor hours per requirements change, measurable improvements in quality were found. A 9% reduction in total interface defects were discovered in the MBSE Baseline compared to the Legacy Process Baseline. In addition, there was an 18% shift of defect discovery from platform integration testing to laboratory integration testing with the MBSE Baseline. Estimates of the cost savings achieved by shifting defect eradication from platform integration to laboratory integration range from 1.6x (Rogers III and Mitchell 2021) to 4x (Feiler et al. 2013), but in any case these savings can contribute significantly to the overall ROI.

Lessons Learned

Seven learning principles (LPs) (Friedman and Sage 2005) were derived that address the more broadly applicable areas of systems engineering knowledge, and inform the areas of the SEBoK that are most strongly related to the case study. They are:

- Requirements traceability (LP1);
- Communications (LP2);
- Productivity (LP3);
- Quality (LP4);
- Managing Change (LP5);
- Managing variants (LP6); and
- Life cycle (LP7).

Requirements traceability LP1: MBSE improves traceability in multiple dimensions, but maintaining requirements traceability between a traditional database and the MBSE system model can be challenging. While DOORS, MagicDraw and Teamwork can interoperate to provide requirements management and traceability, the combination is fragile. Without careful configuration management, synchronization can lead to database corruption. If DOORS and Teamwork are on separate servers, maintaining the connection can run afoul of ever-evolving corporate information assurance policies.

Requirements can be managed using the SysML language inside the system model quite effectively. This approach can reduce the resources needed to keep the system model in sync with a traditional requirements database system and increase overall SE productivity.

Communications LP2: Tailored SE products generated from the system model can substantially enhance communications both within the technical team and between customer stakeholders.

Graphical depictions of the system model often communicate better to human stakeholders than massive spreadsheets and textual documents. Further, the enhanced precision driven by modeling can reduce miscommunications between both technical and programmatic stakeholders.

Having the architecture and design in a system model makes it affordable to generate specialized SE products on demand for particular communications needs while keeping all SE products in concordance. Even technical stakeholders who thought they understood the design can find new insights by looking at it in different representations.

Productivity LP3: MBSE increases productivity by enhancing communications within the team, automating routine tasks and through cost avoidance.

DBSE processes often require substantial revision to achieve the potential productivity gains of MBSE. In particular, review processes should be modified to take advantage of the tooling.

The modeling tools selected constrain how you can practically re-engineer SE processes. Automation can replace a great deal of routine SE work (document generation, identifying potential impacts of changes, etc.).

Developing strong modeling style guidelines and specialized representations, along with training materials to indoctrinate new team members as they join the program, is worth the investment. MBSE does require a trained cadre of modelers, but not all systems engineers have to become skilled modelers.

To effectively quantify the benefits of MBSE, a program needs to plan metrics collection carefully, and then stick to the plan long enough to collect meaningful data.

Quality LP4: Much of the ROI from the MBSE transition can be in improved quality. Improving the quality of SE products enhances early discovery of defects, which reduces integration costs. It also reduces latent defects in systems delivered to the customer, reducing maintenance costs and increasing customer satisfaction.

Models are less tolerant of imprecision than documents. The increased precision improves SE product quality, both in reduced defect generation and in reduced defect escape.

The automation of product generation can make specialized SE products affordable, further enhancing system quality.

Managing change LP5: How a proposed system change is understood and executed is fundamentally different between a model- and a document-centric approach. In the document-centric approach, the focus is on “What should my final output look like?” In the model-based approach, the focus is on “What does this change mean to the system? Which other parts of the system are impacted by this change?”

Change management is hard. When moving from DBSE to MBSE you need to think carefully up front about what approach you are going to take, and then design the system model to facilitate that approach. Change management also impacts tool selection, since different tools align better with different approaches.

Managing variants LP6: The most common process for managing variants in DBSE is ‘clone and own’, where each new product family member takes the then-current baseline and ‘forks’ the baseline for evolution of the variant. This makes synchronizing changes to the core baseline across the product family a very labor-intensive process. Treating variants as deviations from a core baseline in a model greatly reduces the cost of managing variation in a product family.

Variant management is hard. You need to think about what approach you plan to take up front and design your system model to accommodate it. The selected approach impacts tool selection and tailoring.

Design the system model to treat variants as deviations from the core baseline. Then changes to the core baseline are automatically shared among all variants, and impact to product family members is limited to any impact of core baseline change on specific variant deviations. This also facilitates commonality between variants, a key customer goal as commonality reduces logistics and training costs.

Life cycle LP7: MBSE can be applied early or late in the product family life cycle. While most projects using MBSE start off model-based, a program can transition to MBSE late in the life cycle.

Getting from DBSE to MBSE requires serious engineering, careful thought, planning and implementation. The SWFTS MBSE transition required three years of investment by the customer. That time and budget was spent primarily in designing and developing the system model and in re-engineering the SE processes.

Start the transition with carefully defined scope. Once that is accomplished you can expand the scope of MBSE from there.

References

Works Cited

- Cooper, D. J., J. Sienkiewicz, and M. Oliver, "System engineering and integration for submarine combat systems in the COTS environment", NDIA Systems Engineering Conference, San Diego, CA, 23-26 October 2006.
- Fages, H I. 'Submarine programs: A resource sponsor's perspective," *The Submarine Review*, July pp. 53–59, 1998.
- Feiler, Peter H., John B. Goodenough, Arie Gurfinkel, Charles B. Weinstock and Lutz Wrage, "Four Pillars for Improving the Quality of Safety-Critical Software-Reliant Systems", CMU/SEI White Paper, April 2013. Accessed 10 Sept 2017 at <https://www.dtic.mil/get-tr-doc/pdf?AD=ADA585679>
- Ford, D. N., and J. T. Dillard, "Modeling open architecture and evolutionary acquisition: implementation lessons from the ARCI program for the Rapid Capability Insertion process ", *Proceedings of the Sixth Acquisition Research Symposium: Defense Acquisition in Transition 2* (Apr 2009): pp. 207- 235.
- Friedman, G.R. and A.P. Sage, "Case studies of systems engineering and management in systems acquisition." *Systems Engineering*, vol. 7, No. 1, pp. 84-97, 2004.
-

Jacobus, P., P. Yan, and J. Barrett, "Information management: The advanced processor build (tactical)", *JOHNS HOPKINS APL TECHNICAL DIGEST*, vol. 23, No. 4 Jan, pp. 366-372, 2002.

Mitchell, Steven W., "Efficiently Managing Product Baseline Configurations in the Model-Based System Development of a Combat System Product Family", INCOSE International Symposium, Rome, Italy, July 2012.

Zingarelli, M. A., S. R. Wright, R. J. Pallack, and K. C. Matto, "SWFTS - System engineering applied to submarine combat systems," *Engineering the Total Ship*, Falls Church, VA, July 2010.

Primary References

Mitchell, S. W., "Transitioning the SWFTS program combat system product family from traditional document-centric to model-based systems engineering", *Journal of Systems Engineering*, Vol. 17, No. 2, Spring 2014.

Rogers III, Edward B. and Steven W. Mitchell, "MBSE Delivers Significant Return on Investment in Evolutionary Development of Complex SoS", *Systems Engineering*, vol. 24, No. 6, pp. 385-408, November 2021. DOI 10.1002/sys.21592

Additional References

Gibson, B., S. W. Mitchell, and D. Robinson, "Bridging the gap: Modeling federated combat systems," *Third International Conference on Model Based Systems Engineering*, Fairfax, VA, Sept 2010.

Mitchell, S. W., "Model-based system development for managing the evolution of a common submarine combat system," *A FCEA-GMU C4I Center Symposium on Critical Issues in C4I*, 18-19 May 2010.

Mitchell, S. W., "Complex product family modeling for common submarine combat system MBSE," *Third International Conference on Model Based Systems Engineering*, Fairfax, VA, Sept 2010.

Mitchell, S. W., "Efficient management of configurations in the model-based system development of a common submarine combat system," *A FCEA-GMU C4I Center Symposium on Critical Issues in C4I*, 24-25 May 2011.

Note

OMG® and SysML® are registered trademarks of Object Management Group, Inc. in the United States and/or other countries. IBM®, Rational®, and DOORS® are registered trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide.

Virginia Class Submarine

- Lead Authors:
- Heidi Davidz and Alice Squires

This example was developed as a SE example directly for the SEBoK. It describes the Virginia Class submarine sonar system project. In particular, it highlights the approach taken to the development of a sonar system architecture and how this helped in the integration of commercial off the shelf products.

Description

Prior to the Virginia class submarine, sonar systems were comprised of proprietary components and interfaces. However, in the mid-1990s, the United States government transitioned to the use of commercially developed products - or commercial off the shelf (COTS) products - as a cost-saving measure to reduce the escalating costs associated with proprietary-based research and development. The Virginia class submarine system design represented a transition to COTS-based parts and initiated a global change in architectural approaches adopted by the sonar community. The lead ship of the program, Virginia, reduced the number of historically procured parts for nuclear submarines by 60% with the use of standardization. The Virginia class submarine sonar system architecture has improved modularity, commonality, standardization, and reliability, maintainability and testability (RMT) over historical sonar systems.

Architectural Approach: Standardization

Based on the new architectural approach and the success of the transition, system architecture experts developed an initial set of architecture evaluation metrics:

- Commonality
 - Physical commonality (within the system)
 - Hardware (HW) commonality (e.g., the number of unique line replaceable units, fasteners, cables, and unique standards implemented)
 - Software (SW) commonality (e.g., the number of unique SW packages implemented, languages, compilers, average SW instantiations, and unique standards implemented)
 - Physical familiarity (with other systems)
 - Percentage of vendors and subcontractors known
 - Percentage of HW and SW technology known
 - Operational commonality
 - Percentage of operational functions which are automated
 - Number of unique skill codes required
 - Estimated operational training time (e.g., initial and refresh from previous system)
 - Estimated maintenance training time (e.g., initial and refresh from previous system)
 - Modularity
 - Physical modularity (e.g., ease of system element or operating system upgrade)
 - Functional modularity (e.g., ease of adding new functionality or upgrading existing functionality)
 - Orthogonality
 - Level to which functional requirements are fragmented across multiple processing elements and interfaces
 - Level to which throughput requirements span across interfaces
 - Level to which common specifications are identified
 - Abstraction (i.e., the level to which the system architecture provides an option for information hiding)
-

- Interfaces
 - Number of unique interfaces per system element
 - Number of different networking protocols
 - Explicit versus implicit interfaces
 - Level to which the architecture includes implicit interfaces
 - Number of cables in the system
 - Standards-based openness
 - Interface standards
 - Ratio of the number of interface standards to the number of interfaces
 - Number of vendors for products based on standards
 - Number of business domains that apply/use the standard (e.g., aerospace, medical, and telecommunications)
 - Standard maturity
 - Hardware standards
 - Ratio of the number of form factors to the number of line replaceable units (LRUs)
 - Number of vendors for products based on standards
 - Standard maturity
 - Software standards
 - Number of proprietary and unique operating systems
 - Number of non-standard databases
 - Number of proprietary middle-ware
 - Number of non-standard languages
 - Consistency orientation
 - Common guidelines for implementing diagnostics and performance monitor/fault location (PM/FL)
 - Common guidelines for implementing human-machine interface (HMI)
 - Reliability, maintainability, and testability
 - Reliability (fault tolerance)
 - Critical points of fragility (e.g., system loading comprised of percent of processor, memory, and network loading)
 - Maintainability (e.g., expected mean time to repair (MTTR), maximum fault group size, whether the system can be operational during maintenance)
 - Accessibility (e.g., space restrictions, special tool requirements, special skill requirements)
 - Testability
 - Number of LRUs covered by built-in tests (BIT) (BIT coverage)
 - Reproducibility of errors
 - Logging/recording capability
 - Whether the system state at time of system failure can be recreated
 - Online testing (e.g., whether the system is operational during external testing and the ease of access to external test points)
 - Automated input/stimulation insertion
-

Other Points

The Virginia class submarine acquisition exhibited other best practices. These are discussed by Schank (2011), GAO (2008), and General Dynamics (2002).

These best practices included stringent design trades to keep costs under control, careful consideration of technical maturity of components, and the importance of program stability.

Summary

In summary, the work on the Virginia class submarine prompted a change in the traditional architectural approach used in the sonar community to design submarine sonar and validated the cost savings in both research and development (R&D) and in component costs when transitioning from proprietary interfaces to industry standard interfaces. The identification of a list of feasible architecture evaluation metrics was an added benefit of the effort.

References

Works Cited

GAO. *Defense Acquisitions: Assessment of Selected Weapon Programs Report*. Washington, DC, USA: US. Government Accountability Office (GAO). March 2009. GAO-09-326SP.

GD Electric Boat Division. *The Virginia Class Submarine Program: A Case Study*. Groton, CT: General Dynamics. February 2002.

Schank, J.F. et al. *Learning from Experience, Volume 2: Lessons from the U.S. Navy's Ohio, Seawolf, and Virginia Submarine Programs*. Santa Monica, CA, USA: Rand. 2011. Available at https://www.rand.org/content/dam/rand/pubs/monographs/2011/RAND_MG1128.2.pdf^[1]

Primary References

None.

Additional References

None.

References

[1] https://www.rand.org/content/dam/rand/pubs/monographs/2011/RAND_MG1128.2.pdf

Information System Examples

Complex Adaptive Taxi Service Scheduler

- Lead Author:
- Rick Adcock

This article is based around a **London Taxi Service Case Study** (Rzevski and Skobelev, 2014). The case study focuses on the development of a Real-Time Complex Adaptive Scheduler for a London Taxi Service capable of managing the complexity of many hundreds of taxi journeys in an unpredictable and changing environment, while fitting into the goals and values of the Enterprise.

Background

When this project was initiated, the company, the largest and the best-known minicab (taxi) operator in London had a fleet of more than 2,000 vehicles, each with a Global Positioning System (GPS) navigation system. The fleet comprised a variety of vehicles, including minivans and Sport Utility Vehicles (SUVs), some with equipment to match special customer requirements. Under usual circumstances, approximately 700 drivers worked concurrently, competing with each other for customers.

The company had a modern Enterprise Resource Planning (ERP) system and a call center with over 100 operators receiving orders concurrently. Some orders were received through the company website. A large team of skilled dispatchers allocated vehicles to customers.

Main characteristics of the taxi service were as follows:

- More than 13,000 orders per day
- Occasionally more than 1,500 orders per hour (1 order every 2.4 seconds)
- Unpredictable order arrival times and locations
- Various clients, e.g., personal, corporate, Very Important Persons (VIPs), a variety of discounted tariffs, special requirements suitable for the disabled, small children (child seats), transportation of pets, etc.
- Many freelance drivers who leased cars from the company and were allowed to start and finish their shifts at times that suited them, which may have differed from day to day
- Clients in central London were guaranteed pick up times within 15 minutes of order placement
- Fundamentally, the company tried to find the best economic match of vehicle to every client. However, dynamic exceptions to this basic requirement included:
 - Matching drivers going to and from home with passengers travelling in the same direction (to reduce drivers' idle runs); and
 - Giving priority to drivers with less work during a particular day (to increase drivers' satisfaction with working conditions)

No pre-planned taxi schedule was viable because any of the following unpredictable "Disruptive Events" occurred every 2 to 10 seconds:

- Order arrival, change, or cancellation
 - Changes in driver profile, status, or location
 - Client no-show
 - Vehicle failure
 - Delays due to traffic congestion, or queues at airports, railway stations, etc.
-

Purpose

Rescheduling up to 700 independent entities travelling in London under unpredictable conditions that change every few seconds represented an exceedingly complex task, which was not feasible to accomplish using any known mathematical method.

Manual scheduling, as practiced, could not handle the frequent disruptive events. Many perturbations, such as unexpected delays, had to be ignored by the human dispatchers.

Therefore, the project's objective was to provide effective, real-time, automated assistance to accommodate the disruptions that drove the scheduling. Thus, the project purpose became the development of a complex adaptive software system capable of managing the taxi operation complexity described above with the aim of substantially improving: (1) operational profitability; (2) customer service quality; and (3) driver working conditions.

The planned transformation was from a manual to semi-automated managed taxi operation that facilitated optional human dispatcher interactions with a complex adaptive system scheduler. A thorough analysis of contemporary practices showed that such a transformation has never been achieved before. To the best of the project team's knowledge, there were no real-time schedulers of taxi operations in existence anywhere in the world.

Challenges

The team undertaking the development of a new real-time scheduler for this client had vast experience of designing and implementing complex adaptive software, and therefore no particular challenges were anticipated. The multi-agent technology, which underpinned the system, was well understood by the team, and a methodology for managing complexity (Rzevski and Skobelev, 2014) of the task was in place.

Systems Engineering Practices

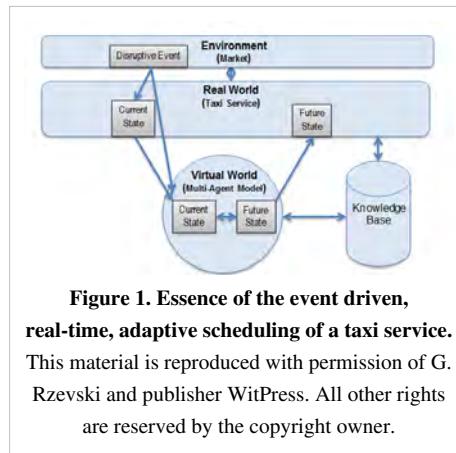
Overview

The complexity of the taxi service ruled out all conventional systems engineering practices. The real-time adaptive scheduler for the client's taxi service was developed using multi-agent software technology. STOPPED HERE The scheduler design consisted of the following major components (Rzevski and Skobelev, 2014):

1. A Knowledge Base containing domain-useful information relevant to the client's taxi service
2. A Multi-agent Virtual World which models the Real World of the taxi service and is capable of managing its complexity
3. Communication channels between the Virtual and Real Worlds which enable the Virtual World management of the Real World with or without human intervention.

The system was designed to behave as follows. In reaction to every disruptive event, Order Agents, assigned to every received order, and Driver Agents, assigned to every working driver, negotiate the most suitable Order-Driver match through the exchange of messages. Before starting negotiations, these software agents consult the Knowledge Base for the current negotiation rules. Once the best possible match (under prevailing circumstances) is agreed upon, the result is communicated to Drivers, who are free to accept or reject the task (Glaschenko et al. 2009). This process is depicted simply in the figure below.

After a successful prototype implementation, a basic version of the complex adaptive scheduler was developed as described below.



Knowledge Base

The Knowledge Base consisted of: (1) Ontology, containing conceptual knowledge as a semantic network; and (2) Values, in standard databases.

The basic Ontology contained two Object Classes: Order and Driver. Order attributes were:

- Location of pick-up and drop-off
- Pick-up(urgent or booked in advance for a certain date and time)
- Type of service (standard car, minivan, VIP, etc.)
- Importance of service (a number from 0 to 100 depending upon the client)
- Special requirements (pet, child chair, etc.)

Driver attributes were:

- Type of vehicle
- Capability to complete special jobs
- Driver experience (novice or experienced)
- Domicile of driver
- Current vehicle location (GPS coordinates)
- Driver status (unavailable, break, working, free, will be free in 5/10 minutes, home transit)

Factual data on Object Instances (Individual Orders and Drivers/Vehicles statuses) were stored in client's databases, including Scenes (i.e., instantaneous models of the taxi service yielding every vehicle location and driver availability).

Virtual World

In the basic version of the scheduler, the allocation of taxis to customers was done by the negotiation between Order Agents, assigned to customers, and Driver Agents, assigned to taxi drivers. Order Agents were active: they compiled lists of available vehicles and initiated negotiations with Driver Agents. In this first version of the system, Driver Agents were designed to be only reactive: they only replied to requests from Order Agents and implemented the option selected by an Order Agent.

In the extended version, hereafter described, Order Agents and Driver Agents competed with each other or co-operated, depending on what was best for the whole enterprise. In addition to Order and Driver Agents, this version used some new types of agents, namely: External Events Agents, Regional Loading Agents, and Orders Allocation Agents.

Agents were designed to use flexible decision-making criteria instead of direct priorities, which is valuable when there is a need to deal with different categories of clients. For example, if a VIP order arrived and there was only one

driver that fully corresponded to the specified requirements and if that driver was already assigned to another job, the system would nevertheless allocate the VIP order to this driver and initiate re-scheduling of the previously agreed matches, if required.

The system first attempted to maximize company profit. Then, other criteria that are important for the business were considered, such as the service level and driver working conditions. For example, when choosing from two approximately equal options the system allocated the order to the driver who had not received orders for a longer time, thus ensuring relatively fair distribution of orders.

This virtual agent-based scheduling system was designed to work effectively with human dispatchers. In a situation where one dispatcher takes a new order and schedules a vehicle to come from north to south to pick up a client, and another dispatcher independently schedules another vehicle to go from south to north for another order, the virtual agents can spot this schedule anomaly and recommend dispatchers change their decisions to be more effective.

To enable improved performance, the taxi allocation system functioned in short cycles rather than as an immediate reaction to every event. Between the cycles, the system collected the events and placed them in a queue. During each cycle, the events from the queue were processed one by one and appropriate agents, in turn, were given control by a designated human system dispatcher. Each event thus initiated a chain of negotiations among virtual agents. When all events were processed and the system dispatcher was satisfied that the best possible schedule was produced for that cycle, the schedule perturbation was implemented in the real world, and the system fell asleep (was idled) until a new event arrived causing the initiation of the next cycle.

To decrease the dimensions of the decision space, a pre-matching mechanism was used, which determined the suitability of Order-Driver matching. This mechanism cuts off unpromising options.

The Order-Driver pairs were evaluated before the final decision was made. An evaluation mark was given to each option and good options were remembered so that the evaluations did not need to be repeated later. The evaluation mark was determined using a multi-criteria model and calculated as a sum of all criteria values multiplied by their (variable) weights.

The following criteria were used for option evaluation: distance to the order, predicted delay of the pick-up, if any, preferences of the driver, driver experience, distance of the driver to overloaded area (to utilize drivers from outlying districts), service level conformity, importance and priority of the order, driver's place in a queue (if he is waiting at an airport), driver's home address (if he is looking for an order to or from home).

Scheduling workflow included the following steps:

1. New order arrives and joins the event queue
2. Possibility of order scheduling is checked
3. A software agent is assigned to the order
4. All drivers that can complete this order are included in pre-matching
5. Evaluation of all Order-Driver pairs is done according to agreed criteria
6. The Order Agent requests order completion costs from selected Driver Agents. This cost includes the cost of transferring the order from the previously allocated driver, if any
7. The Driver Agent receives the information on the reallocation costs by sending a request to its current Order Agent
8. If the revised decision is better than the previous one, it is applied
9. Step 6 continues for all candidate drivers, for whom the initial evaluation (without transfers) was better than the current evaluation
10. If no further changes occur during the cycle, the event processing is considered finished

In order to achieve the best possible solution, the system continued to search for improvements in previously agreed Order-Driver matches until the last moment when it had to issue the instruction to a driver to fulfil an order (commitment time). During this time interval, the Driver was considered to be available for new allocations but only

if the new allocation improved specified performance indicators.

When required, Driver Agents attempted “to come to an agreement” with each other about proposed re-allocation of orders. Occasionally, compensation was offered to the Driver Agent who lost a good client in order to improve overall value of the business, and Driver Agent satisfaction, in particular. Very often the re-scheduling of allocated resources caused a wave of negotiations aimed at the resolution of conflicts between new and old orders. The length of the re-scheduling chain was limited only by the time required for a taxi to reach a customer in a busy city such as London, which normally was sufficient for several changes of the schedule.

To summarize, the system built a schedule and perpetually reviewed it, attempting to improve key performance indicators as long as the time for essential re-scheduling was still available.

The Commitment Time was dynamically calculated for each order, taking into account the priority and service type of the order and some other parameters. The introduction of the dynamic Commitment Time resulted in the increase of the fleet effectiveness by reducing the average task completion time per driver.

An option was introduced for the system to distribute the fleet according to the order-flow forecasts. Having information about the current order-flow and distribution of orders in the past, the expected order-flow was extrapolated, enabling the system to generate short-term (30 minutes) forecasts, which were normally reasonably correct. Based on the forecast, the system sent text messages to unoccupied drivers with recommendations to stay in, or move to, the region where an increased order flow was expected. This feature enabled an improved distribution of the fleet, reducing response times and idle miles and increasing the number of pick-ups.

In cases when forecasts envisaged a probability of a VIP order arrival at a significant distance from the point where drivers were advised to congregate, the system would recommend that a proximate driver to move closer to the likely order point, offering him/her a guaranteed next order in exchange for compliance. This was an important feature because there were usually enough proximate drivers to complete available orders in areas that were not overloaded, and productivity of work in overloaded areas determined the actual fleet effectiveness. The system was also designed with an option to temporarily amend criteria for the allocation of orders to drivers (for example, to extend the area where drivers are allowed to search for orders) to enable drivers to reach critical locations without being intercepted by less important orders from nearby locations.

The forecasting functionality was supported by an agent-based dynamic data mining system, which was, in fact, another complex adaptive system cooperating with the complex adaptive scheduler.

In later versions the system was designed to detect and identify drivers that cheat, i.e., deliberately providing the scheduler with false information to gain personal advantages. Recorded cases include attempts to:

- Reduce their ultimate waiting time by reporting that they were already waiting in an airport queue when, in reality, they may still have been tens of miles from the airport
- Get an earlier next order by indicating “free in 10 minutes” at or near the beginning of a long assignment
- Receive orders in their home direction by indicating “going home” several times during a day.

To reduce cheating, Driver Agents were designed to monitor drivers’ schedules and ignore their messages, when judged inappropriate.

The final version of the complex adaptive taxi service scheduler negotiated only with agents that were affected by a disruptive event and then modified only affected parts of the schedule. This capability was a key feature that improved overall effectiveness.

Connecting Virtual and Real Worlds

The Virtual World, which is a model of reality and resides in the scheduler, is connected with the Real World of customers, dispatchers, and drivers, as follows.

As customers ring the call center or visit the website to place, modify or cancel an order, dispatchers enter the pertinent information into the system. Drivers communicate with the system using GPS, mobile phones, or specialized handheld devices, conveying information on their location, direction of travel, availability, etc., and, in turn, receive instructions to pick up customers.

Lessons Learned

The system began its operation and maintenance phase in March 2008, only 6 months from the beginning of the project.

Results were extremely good: 98.5 % of all orders were allocated automatically without dispatcher's assistance; the number of lost orders was reduced by up to 2 %; the number of vehicles idle runs was reduced by 22.5 %. Each vehicle was able to complete two additional orders per week spending the same time and consuming the same amount of fuel, which increased the yield of each vehicle by 5 – 7 %.

Time required to repay investments was 2 months from the beginning of the operation and maintenance phase. During the first month of operation the fleet utilization effectiveness was increased by 5 – 7 %, which represents potential additional revenue of up to 5 million dollars per year. Such realized additional income has benefited both the company and the taxi drivers. According to available statistics, driver wages have increased by 9 % since 2008, and there is a possibility for an overall fleet growth.

Delayed pick-ups were reduced by a factor of 3, which considerably improved customer service. Urgent order average response time (from booking until taxi pick-up arrival) decreased to 9 minutes, which is the best time among all taxi services in London. For high priority orders, the response time is 5 – 7 minutes or less. Response time reductions are especially noticeable in overloaded areas.

Implementation of "on the way home" orders, an improved allocation mechanism, when compared with a previous system, gives 3 – 4 thousand miles reduction in daily fleet run, greatly benefiting both drivers and the city's ecology.

Further developments targeting business effectiveness improvements may include an analysis of vehicle movements to determine actual vehicle velocities that could improve courier service by increasing the number of orders per courier.

References

Works Cited

- Rzevski, G., Skobelev, P. 2014. *Managing Complexity*. WIT Press, New Forest, Boston. ISBN 978-1-84564-936-4.
- Glaschenko, A., Ivaschenko, A., Rzevski, G., Skobelev, P. "Multi-Agent Real Time Scheduling System for Taxi Companies". Proc. of 8th Int. Conf. on Autonomous Agents and Multiagent Systems (AAMAS 2009), Decker, Sichman, Sierra, and Castelfranchi (eds.), May, 10–15, 2009, Budapest, Hungary. ISBN: 978-0-9817381-6-1, pp. 29-35.
-

Primary References

Rzevski, G., Skobelev, P. 2014. *Managing Complexity*. WIT Press, New Forest, Boston. ISBN 978-1-84564-936-4.

Additional References

None.

Successful Business Transformation within a Russian Information Technology Company

- Lead Author:
- Brian White

This article describes a successful business transformation of an information technology enterprise. The topic may be of particular interest, especially because this transformation was accomplished by a Russian company during the republic's fast-growing economic recovery.

For additional information, refer to the closely related topics of Enabling Businesses and Enterprises and Enterprise Systems Engineering.

Background

In 2001, the top management of the IBS company ^[1] in Moscow initiated a fundamental transformation to change the company's strategy and business model. The company was one of the biggest Russian information technology (IT) systems integrators at that time, with about 900 employees. Annual revenues of about \$80M were mainly generated by information technology (IT) infrastructure projects (complex computing systems, multi-service networks, etc.) and hardware and software distribution. The transformation of the company to form new capabilities in IT services and the associated consulting area is the main topic in the case study.

During the transformation period (from 2001 to the present) IBS was represented as a set of autonomous business units (BUs), called constituent systems, which are virtual, independent businesses with the following characteristics:

- Profit and loss reporting was required for each BU according to management accounting procedures
- BU management established and independently conducted human resources, technology, and product policy
- A centralized back-office was organized to provide supporting functions for each BU. Thus, BUs do not have back-offices; they rely on and "pay" a corporate governing center (CGC) for these services.

A thorough enterprise system (ES) transformation was executed as a set of activities: mission analysis and capabilities decomposition, business architecting, planning of the project program, and implementation of the new business model.

Before and after transformation IBS was an exemplar directed system of systems (sos): the constituent BUs are autonomous but their operations are supervised by CGC. At the same time IBS also has significant features of an acknowledged SoS: the constituent BUs retain their independent development and sustainment approaches, and changes in the company are based on collaboration between the CGC and each constituent; even operations of BUs are not controlled but only supervised/governed by the CGC through "soft" recommendations and coordination.

IBS was a quite mature ES before the transformation, and it was thoroughly upgraded to form new capabilities of the whole system as well as of the constituents.

Purpose

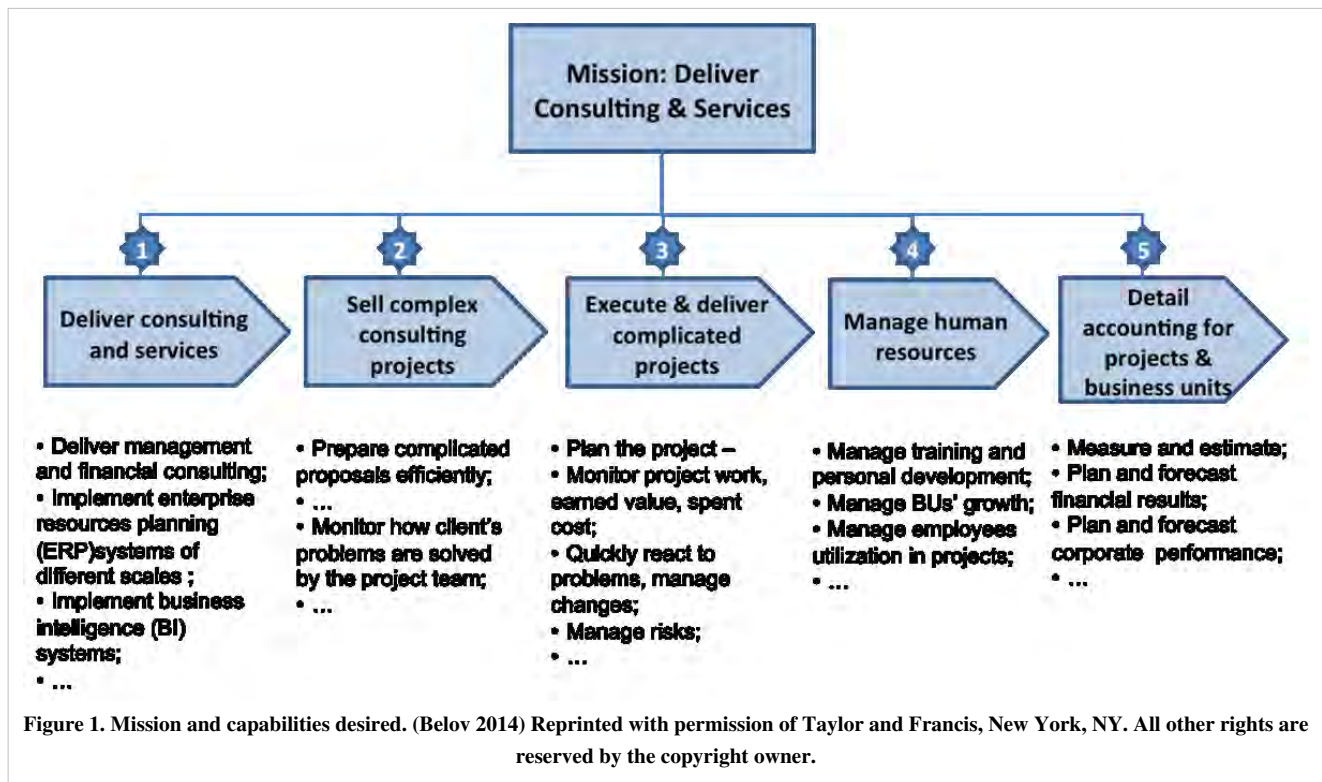
In 2000-2001 IBS management forecasted considerable growth of the Russian IT services and consulting market based on the fast growing Russian economy, which was rapidly recovering from the national financial crisis of 1998. The largest corporations started overseas expansion and borrowed from international markets to finance this growth. IBS predicted corresponding growth in the complexity of business processes and their associated software and hardware systems, all of which should require more consulting and IT services.

Based on this forecast, management established a strategy goal to double the share of IT services and consulting from 25% to 50% over one year; further growth in this business was planned as a long-term trend.

The consulting and IT services business is very complex technologically and organizationally and dramatically differs from IBS's former infrastructure focus. Thus, a fundamental transformation was required, and it was executed during 2002.

Initially detected problems appeared as expenditures exceeding resources, slow delivery of the projects and reworking. Later, as it was expected, new problems appeared, for example, disinterest of BUs' managers in developing new technologies or raising qualified employees' motivation. All those problems were solved during transformation and during further development.

The first step of the transformation included strategic analysis and mission-to-capabilities decomposition. Five major capability groups to be focused on were defined. The groups and exemplar capabilities for each group are represented at Figure 1.



Challenges

All main challenges were caused by knowledge/information deficit described by three factors listed as a, b, and c below.

a. The lack of experience in enterprise transformation (and capability-based approaches, even the lack of any textbooks or guides in those areas) was the major challenge which IBS management faced. The task to be solved did not devolve to organizational changes (which was a well-developed and described area), but was appropriately allocated to enterprise system or system of systems (SoS) engineering. In spite of the lack of experience, it was decided to prepare and execute the transformation based on the company's employees without involving external consultants. The following arguments supported the decision.

- The task to be solved was not typical, so there weren't widely used and well tested algorithms or methods, and there weren't a lot of consultants experienced in exactly what was needed. So only consultants with general experience (strategy consulting, organizational management) might be hired.
- The Russian consulting industry in 2001-2002 was not well developed, so only foreign professionals were available. But foreign consultants would have needed to study Russian specifics; such study would have unduly lengthened the duration and increased the cost of the transformation.
- A joint transformation team would have to be formed, and IBS employees would have to be involved: management would have to be interviewed and be involved in decision making. In any case, all employees would have to participate in change implementations.
- External consultants are not stakeholders; so their level of interest in helping to achieve success might not be very high, and their output also might not be outstanding.
- Unwillingness to open professional secrets and other intellectual property issues to direct competitors were other factors that prevented hiring of external consultants.

Thus, the final decision was to execute the transformation without involvement of external consulting resources. A special BoU responsible for business processes development was established and an agile program management approach was applied to handle challenges and to pursue opportunities as well as to mitigate risks.

b. A very high complexity IBS as an enterprise system or SoS. Management recognized that the company and its environment was very complex, with many different agents, many constituents, and countless relationships; and that an enterprise system or SoS might become even more complex after transformation. This complexification happened as the company became an "extended enterprise," the governing hierarchies weakened, and the demand for more sophisticated relationships increased.

c. The risk of mistaken forecast of IT market development. The expected growth of the consulting and services market might have not happened. In this case the transformation would have been senseless. This challenge generated additional emotional stress for management.

Systems Engineering Practices

The SE task of the transformation was established in the form: to develop required capabilities for an enterprise system or SoS – IBS company. The SE process might be represented by the following specific IBS interpretation of the vee (v) model ("V model") with Stages 1 through 7 (Figure 2).

Initially (Stage 1) the mission was translated to capabilities (Figure 1); "understanding the constituent systems (BUs) and their relationships" was executed. The transformation team found that capabilities might not be directly translated to any business-agent. Neither BUs (they serve as resource pools), nor projects (being temporal elements), nor employees (each of them have a finite set of skills, experience, responsibilities, etc.) might realize necessary capabilities.

Realizing this (Stage 2) transformation team defined several key areas (Figure 2) of company's operations or activities which were supposed to be changed to form new capabilities. Appropriate artifacts (procedures, guides,

documents, software systems) to support new capabilities were developed and implemented for each of the areas; these new assets formed exactly the corporate infrastructure of new business model.

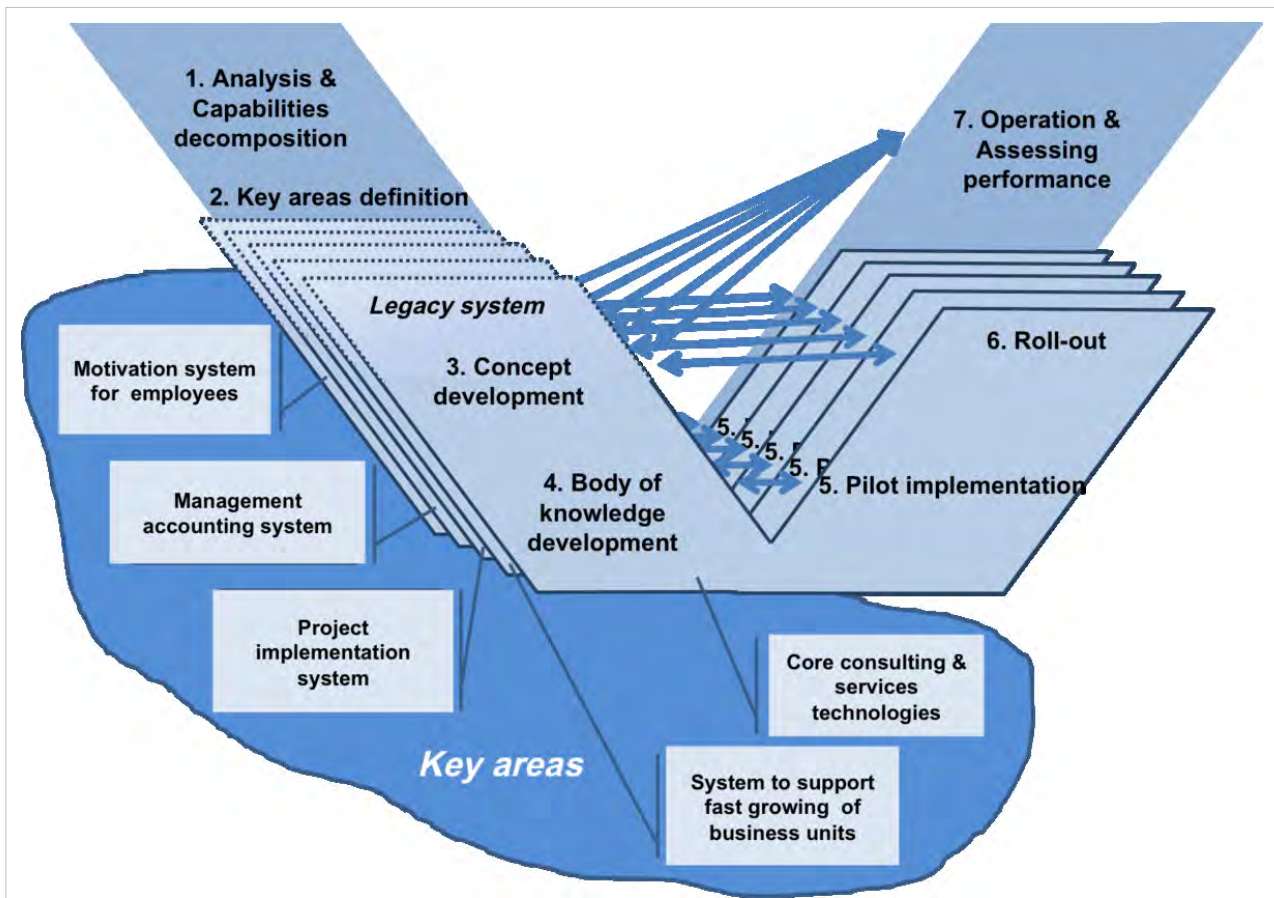


Figure 2. "V model" of the systems engineering process of the transformation. (Belov 2014) Reprinted with permission of Taylor and Francis, New York, NY. All other rights are reserved by the copyright owner..

For each new and legacy system (Stage 3), a set of conceptual design documents was developed, describing approaches, policies, processes, and procedures. The entire set of documents formed the business architecture description of the company. The description connected all key areas and defined a target operation model of the company after transformation. This architecture represented multiple views of the IBS company, and thus aptly reflected its enterprise system or SoS nature.

Somewhat in contrast with the conventional linear systems engineering approach advocated by the V model, Stages 4-6 were conducted in parallel to save time and resources. The company's performance (Stage 7) should be monitored based on indicators' measurements, and improvements should be developed and implemented (arrows from Stage 3 to Stage 7). Such iterations have been executed in practice not only during transformation but also later, when procedures, guides and the whole systems were updated.

Integration and interoperability of the new systems required a thorough integration of parallel development jobs. So joint workgroups were formed of the employees at the level of low officers; and CGC played the role of integrated workgroup at the management level. Actually, multi-level integrated workgroups were formed.

The major complexity and risks derived from the challenges described above.

The transformation team developed and used an approach which is very similar to the agile development approach to address those risks. The following principles were used to manage the portfolio of projects in case of uncertainty and deficit of knowledge.

- Form solutions as fast as possible (but not necessarily with pure quality) to test them in practice faster.
- Recognizing failures are unavoidable, perceive them readily and react rationally.

- In case of failure, analyze the situation and find a new solution, generate changes, and update the plan.
- Work in parallel, verifying and coordinating intermediate results.
- The schedule might be corrected and updated but should not be jeopardized by improper execution.
- Formulate and test the most critical and most questionable solutions at first.
- Start from pilot area and then expand to embrace the entire scope.
- Use high quality monitoring and a “manual control mode” for piloting and testing developing solutions but not additional aspects to limit waste of the resources.

Following those principles, including a very strong discipline of execution, a high level of the sponsorship and all-employee involvement enabled the transformation to be completed on time without hiring consultants while keeping and developing on-going business.

Lessons Learned

IBS's accomplishment of the mission was the major result of ES transformation. Shareholders and management recognized that new capabilities had been formed, that the company could deliver consulting and services, sell and execute complex projects, manage consulting resources effectively, measure its performance, and plan and forecast financial results. Created capabilities are emergent in some sense because they are not directly related to concrete constituents (BUs, or employee, or projects) but are realized by means of integrated end-to-end processes and functions, which are executed in the projects by employees.

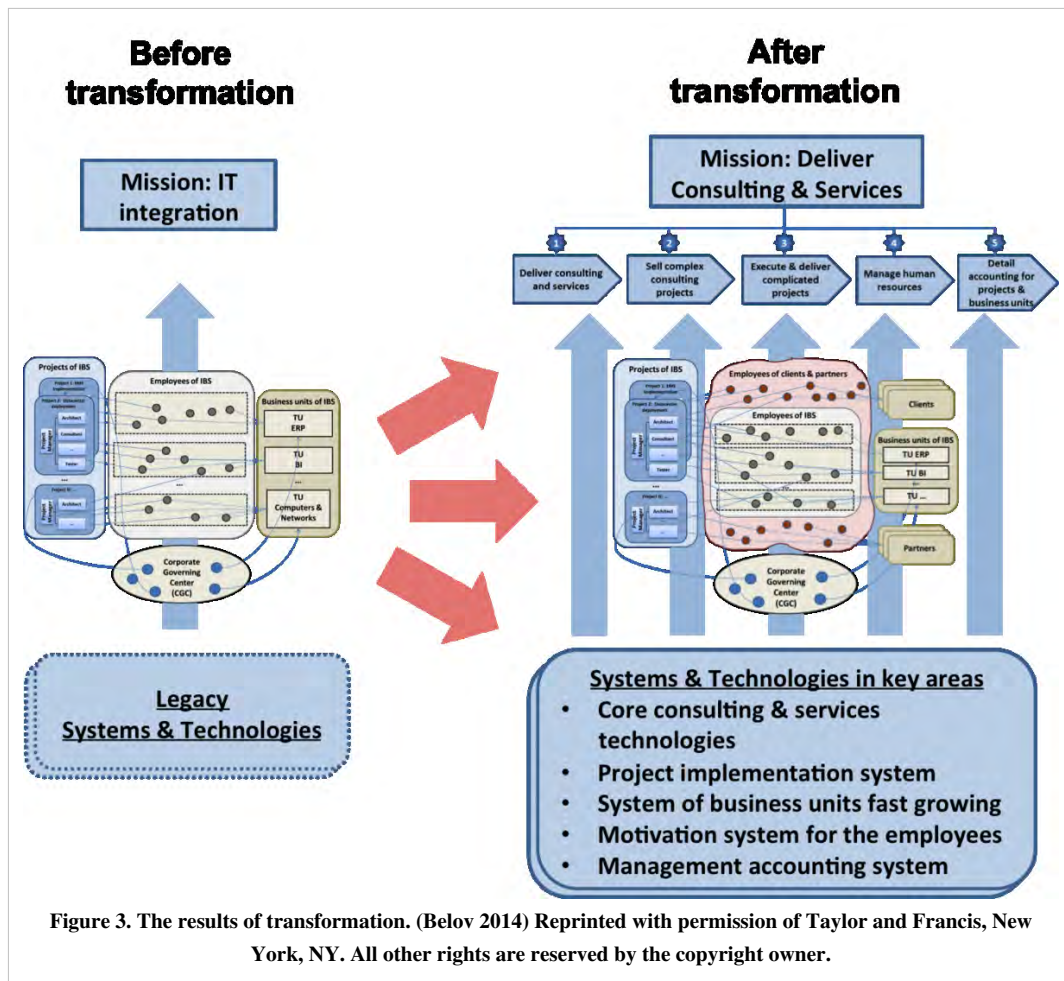
The systems organization did not dramatically change during transformation; “visible structure” was not practically changed: no new types of business-agents appeared, existing types did not change much. Those factors did not create new capabilities. Target capabilities were formed as the result of development and implementation of, it would seem, auxiliary and supporting tools – new capabilities support systems. New capabilities were formed mainly by the changes in the intangible areas of governing media, corporate culture, relations, and personnel competences; as well as by the creation of new capabilities support systems; without considerable changes in main company's business-agents. (Refer to Figure 3.)

The main challenges which management faced (the lack of experience and the ambiguity of market growth forecast) made the uncertainty factor the critical one in the transformation.

What Worked and Why?

An agile program management in general demonstrated its efficiency and applicability to “soft and uncertain” tasks, especially in triggering a pre-established process for dealing with unexpected events; the main aspects of the approach are:

- Senior and credible sponsors
 - Multi-level integrated project team(s)
 - Open information exchange
 - Partnership and collaboration
 - Proactive and motivated parties and constituents
 - Creative and innovative way of development
-



- Prioritizing and focusing on the most ambiguous elements of systems design
- Piloting and subsequent roll-out in realistic environments
- Strong project scope control
- Strong project execution control – time schedule and resources control.

What Did Not Work and Why?

Perhaps corporate knowledge base development was the only more or less serious task which was not solved in transformation. The company's management understood the usefulness of knowledge accumulation and further alienation from the carriers in utilizing their business knowledge, so the goal of developing their own knowledge base was established. Special database and software systems were developed with appropriate guides, reports and data collection forms; but formal regulation to fill in engineering knowledge accumulation templates did not work. However, this issue later progressed quite naturally and simply: common folders were established to store project data in free formats. Such folders served to accumulate knowledge but in flat, unstructured form.

Best Practices and Replication Prospects

The following methods and approaches were proven as efficient and convenient in transformation.

1. Capability-based development approach and capability-based architecting might be recommended to be utilized in creation and transformation of an enterprise system or SoS. These focused all efforts on the required capabilities and involved very important relations from mission to capabilities and to functions in the systems engineering process.
2. An agile program management might be used to solve a wide range of fuzzy and ambiguous problems of different scale in the areas of SE, ES engineering, and SoS engineering where there is much uncertainty and lack of expertise and proven methods or algorithms to solve them. The combination of “soft” and very creative designs with strong planning and progress control is the crucial foundation of this approach.
3. Key area definition and development appropriate to generating new capabilities for support systems (core consulting and services technologies, project implementation systems, systems for business unit growth, management accounting systems, motivation systems). Precisely defining these areas and developing integrated systems in these areas might be considered as quite common for application to a broader group of ESs.

References

Works Cited

Belov, M. 2014. "IBS Group, Eastern European ITS Services – Capability-Based Development for Business Transformation," in *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*, A. Gorod et al., (Eds.). Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.

Primary References

Belov, M. 2014. "IBS Group, Eastern European ITS Services – Capability-Based Development for Business Transformation." *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*. Gorod, A., B. E. White, V. Ireland, S. J. Gandhi, and B. J. Sauser, (Eds.). Boca Raton, FL: CRC Press, Taylor & Francis Group. Scheduled for publication in July 2014. <https://www.taylorandfrancis.com/books/details/9781466502390/>.

Additional References

None

References

[1] <https://www.en.ibs.ru/>

Federal Bureau of Investigation (FBI) Virtual Case File System

- Lead Authors:
- Heidi Davidz and John Brackett

This case study presents systems and software engineering issues encountered in the Federal Bureau of Investigation (FBI) Virtual Case File (VCF) project in the period between 2000-2005. VCF development was abandoned in 2005 after over \$170 million had been spent.

Domain Background

The FBI is an organization within the United States Department of Justice (DoJ) consisting of 23 divisions, including counterintelligence, criminal investigation, and cybercrime. The Bureau's 12,400 agents investigate everything from counter-terrorism leads to kidnappings. They interview witnesses, develop informants, conduct surveillance, hunt for clues, and collaborate with local law enforcement to find and arrest criminals. Agents document every step and methodically build case files. They spend a tremendous amount of time processing paperwork. This system of forms and approvals stretches back to the 1920s when forms for all of the bureau's investigative reports were standardized.

In 2000, the Bureau had hundreds of standardized paper forms and an obsolete information technology (IT) systems. The FBI's 13,000 computers could not run modern software. Most of the agency offices were connected to the FBI Intranet with links operating at about the speed of a 56 kilobits-per-second modem. Agents could not e-mail U.S. Attorneys, federal agencies, local law enforcement, or each other; instead, they typically sent case-related information by fax. The agency's problems in 2000 were summarized in the *9/11 Commission Report*: "The FBI's information systems were woefully inadequate. The FBI lacked the ability to know what it knew; there was no effective mechanism for capturing or sharing its institutional knowledge" (National Commission on Terrorist Acts upon the United States 2004).

In September 2000, Congress approved \$380 million over three years for what was then called the FBI Information Technology Upgrade Program. Eventually divided into three parts, the program became known as the Trilogy Information Technology Modernization Program. The first part would provide all 56 FBI field offices with updated computer terminals, as well as new hardware such as scanners, printers, and servers. The second part would re-implement the FBI Intranet to provide secure local area and wide area networks, allowing agents to share information with their supervisors and each other. The third part was intended to replace the FBI's investigative software applications, including the obsolete Automated Case Support (ACS) system.

In June 2001, the FBI awarded a contract to develop the investigative software applications of Trilogy to Science Applications International Corporation (SAIC) over a three year period. The purpose of the software to be developed was to:

- provide the capability to find information in FBI databases without having prior knowledge of its location, and to search all FBI databases with a single query through the use of search engines;
- Web-enable the existing investigative applications;
- improve capabilities to share information inside and outside the FBI;
- provide access to authorized information from both internal and external databases; and
- allow the evaluation of cases and crime patterns through the use of commercial and FBI-enhanced analytical and case management tools.

After the September 11 terrorist attacks, the inability of FBI agents to share the most basic information about al Qaeda's U.S. activities was front-page news. Within days, the FBI's obsolete technology infrastructure was being discussed in Congress and the FBI was under intense pressure to improve its information sharing capabilities. On

September 4, 2001, Robert S. Mueller III became FBI director, and, in the face of intense public and congressional pressure, Mueller accelerated the Trilogy program. The planned three-year period to develop the investigative software was considered politically unacceptable. In January 2002, the FBI requested an additional \$70 million to accelerate Trilogy; Congress went further, approving \$78 million.

Providing web-enablement of the existing but antiquated and limited ACS system would not provide the investigative case management capabilities required to meet the FBI's post-September 11 mission. In December 2001, the FBI asked SAIC to stop building a Web-based front end for the old programs. Instead, SAIC was asked to devise a new case management system, the Virtual Case File (VCF), to replace ACS. The VCF would contain a major new application, database, and graphical user interface. In order to make both criminal and terrorist investigation information readily accessible throughout the FBI, major changes to the standardized FBI processes would be required. This case study focuses on the VCF component of the Trilogy program.

Case Study Background

The most complete description of the development of the VCF is the report by the DoJ Office of the Inspector General (OIG). The OIG reports to the Attorney General and is independent of the FBI organizations responsible for the Trilogy program. The introduction to the report states, "We conducted this audit to assess the FBI's progress in meeting cost, schedule, technical, and performance targets for the three components of Trilogy. We also examined the extent to which Trilogy will meet the FBI's current and longer-term IT needs" (OIG 2004).

An IEEE *Spectrum* article complements the OIG audit report by detailing the development of the VCF requirements, the contractor's activities, and the project management failures by both the FBI and the contractor. The contractor's viewpoint is presented in testimony given before a subcommittee of the U.S. Senate Appropriations Committee.

These materials, in total, provide a comprehensive view of the VCF program and the reasons for its failure.

Case Study Description

In the political environment following the 9/11 attacks, funding for the VCF project was never a problem. By early 2002, SAIC and the FBI committed to creating an entirely new case management system in 22 months. High-level funding enabled the project to continue gaining momentum in spite of the problems it encountered. The scheduling for the VCF project focused on what was desired, not what was possible. Trilogy's scope grew by approximately 80% from the initial project baseline (Moore 2010).

The reasons for the failure of the VCF project are associated with the non-use or misuse of numerous system engineering practices, especially within stakeholder needs definition, system requirements definition, planning, assessment and control, and risk management. Given the political pressures following the 9/11 attacks, the schedule was accelerated to the point that it was nearly impossible for the developers to follow an appropriate systems engineering process.

The FBI cycled through five people in the role of Chief Information Officer in four years and most decisions were made by committees. In order to compress the schedule, the FBI even proposed replacing the ACS with the VCF over a weekend using an IT procedure called a "flash cut-over." In this proposed implementation, the ACS system would be taken offline and entirely replaced by VCF. Once the cut-over happened, there would be no mechanism to return to ACS, even if the VCF did not work properly.

SAIC worked under a cost-plus-award-fee contract for the VCF as the scope of the project was undefined in early 2002 when work began. Given the schedule pressures, the FBI believed that there was no time to develop formal requirements (glossary), validate them with the various FBI user communities, and then estimate the cost and time required to develop the VCF. The SAIC contract did not require specific completion milestones and the cost-plus contract allowed the scope to increase. VCF was a case of not having the requirements sufficiently defined in terms of completeness and correctness. The continuous redefinition of requirements had a cascading effect on what had

already been designed and produced. Once there was demonstrable software, change requests started arriving—roughly 400 from December 2002 to December 2003.

The new FBI Intranet was specified during 2001, before the start of the VCF project and with little understanding of the network traffic that would result from information sharing. By early 2003, the FBI began to realize how taxing the network traffic would be once all 22,000 users came online. The requirements for the FBI Intranet were modified based on the best guesses for the bandwidth that would be required when the VCF was fully operational. By early 2004, the new FBI Intranet was in operation, although the VCF software was far from complete.

In reaction to the time pressure, SAIC broke its VCF development group into eight teams working in parallel on different functional elements of the program. However, this posed many integration challenges and the eight threads would later prove too difficult for SAIC to combine into a single system. By the time VCF was canceled, SAIC had developed over 700,000 lines of software based upon an incomplete set of requirements that were documented in an 800-page volume.

Summary

The OIG summarizes its conclusions as:

Various reasons account for the delays and associated cost increases in the Trilogy project, including:

- *poorly defined and slowly evolving design requirements,*
- *contracting weaknesses,*
- *IT investment management weaknesses,*
- *lack of an Enterprise Architecture,*
- *lack of management continuity and oversight,*
- *unrealistic scheduling of tasks,*
- *lack of adequate project integration, and*
- *inadequate resolution of issues raised in our previous reports on Trilogy. . . .*

According to the Government Accountability Office (GAO), an Enterprise Architecture is a set of descriptive models such as diagrams and tables that define, in business and technology terms, how an organization operates today, how it intends to operate in the future, and how it intends to invest in technology to transition from today's operational environment to tomorrow's. . . .

As of early 2005 the FBI's operations remain significantly hampered due to the poor functionality and lack of information-sharing capabilities of its current IT systems. . . . (OIG 2005)

In May 2005, FBI director Mueller announced Sentinel, a four-phase, four-year project intended to fulfill the purpose of VCF and provide the Bureau with a web-based case and records management system. During the previous five years, commercial case management software had become available; as a result, Sentinel is intended to utilize commercial off-the-shelf (COTS) software. A report by the OIG in late 2009 describes Sentinel and its status at that time. Sentinel was put online for all employees on July 1, 2012, and it ended up at \$451 million and 2 1/2 years overdue (Yost 2012).

References

Works Cited

Moore, S. 2010. "The Failure of the FBI's Virtual Case File Project." Strategic PPM: Project and Portfolio Management, last modified April 5, accessed on September 11, 2011. Available at <https://strategicppm.wordpress.com/2010/04/05/the-fbis-virtual-case-file-project-and-project-failure>.

National Commission on Terrorist Attacks upon the United States. 2004. *The 9/11 Commission Report: Final Report of the National Commission on Terrorist Attacks Upon the United States*. New York, NY, USA: W. W. Norton & Company.

Office of the Inspector General. 2005. *The Federal Bureau of Investigation's Management of the Trilogy Information Technology Project*. Washington, DC, USA: United States Department of Justice. Audit Report 05-07. February 2005. Accessed on September 11, 2011. Available at <https://www.justice.gov/oig/reports/FBI/a0507>.

Office of the Inspector General. 2009. *Sentinel Audit V: Status of the Federal Bureau of Investigation's Case Management System*. Washington, DC, USA: U.S. Department of Justice. Audit Report 10-03. November 2009. Accessed on September 11, 2011. Available at https://www.justice.gov/oig/reports/FBI/a1003_redacted.pdf.

Yost, P. 2012. "'Sentinel', New FBI Computer System, Finally Tracking Cases -- Years Late and Millions Over Budget." Washington, DC, USA. Accessed on August 6, 2012. Available at https://www.huffingtonpost.com/2012/07/31/sentinel-fbi_n_1725958.html.

Primary References

None.

Additional References

Goldstein, H. 2005. "Who Killed the Virtual Case File?" *IEEE Spectrum*. September. Accessed at September 11, 2011. Available at <https://spectrum.ieee.org/computing/software/who-killed-the-virtual-case-file>.

Testimony before the Subcommittee on Commerce, Justice, State and the Judiciary, U.S. Senate Committee on Appropriations, February 3, 2005 (statement of Arnold Punaro, Executive Vice President, Science Applications International Corporation).

Management System Examples

Project Management for a Complex Adaptive Operating System

- Lead Author:
- Rick Adcock

This article is based on the **Complex Adaptive Operating System: Creating Methods for Complex Project Management** case study (Findlay and Straus, 2015). The case study focuses on creating tools and methods that project managers can use in managing complex adaptive systems projects.

Background

The International Centre for Complex Project Management (ICCPM) is a non-profit organization that was created to address “the international community’s ability to successfully deliver very complex projects and manage complexity across all industry and government sectors” (ICCPM, 2012).

In an ongoing effort to help member organizations successfully undertake major complex projects, ICCPM conducted a bi-annual series of international round-tables. The purpose of the round-tables was to better understand what contributes to the success of complex projects and to identify and develop new and improved tools and approaches. The round-tables were facilitated using a computer-assisted collaborative meeting process that leverages the features of complex adaptive systems—described below—to help people with diverse viewpoints and experience create new collective understanding and plans for action.

Complex major projects are known for being unsuccessful in on-time and on-budget completion. A survey (IBM, 2008) of 1,500 change managers found that only 40% of projects finished on time and on budget. Barriers to success were the inability to change attitudes or mindsets (58%), dysfunctional culture (40%), lack of senior management support (32%), and underestimating the complexity of a project (35%).

However, several new systemic approaches show considerable promise as a way to think about and manage projects. Six frameworks help inform these approaches: systems thinking, the features of complex adaptive systems (CASs), complexity theory, the Complexity Model of Change (Findlay and Straus, 2011), polarity thinking as a way of thinking about and leveraging wicked problems, cognitive complexity, and adult development theory.

Systems thinking recognizes whole systems and the interdependencies of their parts. A system may be defined as “a set of things, organisms, and people that, as a result of their interconnection, produce their own patterns of behavior over time” (Meadows, 2008, p. 2). A system cannot be understood by focusing on its parts alone (Wheatley, 1999). To successfully address and influence a system, such as a complex project, one must understand the whole and how its parts interact.

Some project managers continue to think of a project as a “machine” that operates according to linear or algorithm rules. This persistent and largely unhelpful meme is now being displaced by a new and more robust model, which regards projects as complex adaptive systems (CASs). Unlike strictly mechanical systems, CASs are self-organizing, learn from experience, are emergent, and from time to time undergo large scale phase transitions to new and higher states of the system. This view of large-scale projects—which are heavily influenced by human interaction—is proving to be much more accurate and allows project managers to leverage techniques for exerting influence in environments that have previously presented intractable problems .

Leaders of complex projects would also be wise to consider three fundamental theorems of complexity theory, which apply to CASS and which are critical to project success. These are a robust model of the system, requisite variety and adopting solutions which act at an appropriate scale.

- The robust model axiom considers that “no one can effectively influence a system until they have a thorough understanding of its scope and the connections and interdependencies” (Conant and Ashby, 1970).
- The law of requisite variety contends that “complexity can only be dealt with by equal or greater complexity” (Ashby, 1956, p. 2). In other words, in order to deal effectively with the diversity of problems presented by a complex project, one must have a repertoire of responses which is (at least) as nuanced as the problems themselves (Requisite Variety, 2015).
- The scale condition requires that those who wish to exercise leverage over a system must recognize that “highly complex situations can best be addressed by greater degrees of freedom at the local scale so that innovation and adaptability are maximized” (Bar-Yam, 2004).

A third framework, the Complexity Model of Change, is a model of socio-technological change, comprising a series of growth and decay curves or waves, which helps project managers better understand how to influence systems and design new ones, so the roles, methods, rules of interaction or engagement, technologies, and relationships between people are better aligned with each other and the desired outcome. The overlapping waves represent large-scale eras, for example, the Industrial Age and the Information Age, which have at their core a metaphor, for example, the machine and the computer. The current wave, the Knowledge Age, is based on a network metaphor. The wave we are now entering is the Wisdom Age, which began in 2010. Its core metaphor is the complex adaptive system and the main thrust of this period is the wise application of knowledge.

Another area that project managers may now address differently is that of wicked problems or paradoxes: problems that are ill-defined and recurrent, and which, when attempts are made to solve them with single optimal solutions, create another problem. Polarity thinking regards wicked problems as sets of interdependent values or ideas—like centralizing for efficiency and decentralizing for adaptability—that persist together over time and need each other for the success of the system. If we pay attention to one pole at the expense of the other, we achieve sub-optimal results. When we manage polarities as a system, we realize the benefits of both poles and achieve high performance over time with a minimum of vacillation and the need for correction.

Other disciplines that are critical to project success are understanding and making best use of new ways of thinking about issues and relating to others in more flexible and adaptive ways. Theories of cognitive complexity and adult development theory can contribute to how we think about this problem. For example, “triple-loop learning” (Gragert, 2013), helps us think about issues from a higher level of cognitive complexity. Instead of asking “Are we doing things right (single loop)?”, we might ask, “Are we doing the right things (double loop)?” or “How do we decide what is the most effective paradigm to use to influence and create benefit for the system (triple)?”

Purpose

The purpose of the ICCPM roundtables was to help project managers develop new and better ways to lead complex major projects, by bringing together people from both the buy-side and the supply-side to share their knowledge and experience and to grow a network of practitioners, professionals, researchers, and educators able to deliver leading edge complex project management solutions to client organizations and partners around the world (Findlay and Straus, 2015, p. 489).

Challenges

There are many challenges to be addressed in the complex project management environments. The three top contenders are 1) developing new ways of thinking, acting, and interacting; 2) developing more robust models of the system by getting everyone in the room—the project management team and their stakeholders; and 3) steering projects through multiple disruptive socio-technological shifts using the feature of complex adaptive systems.

“People, their organizations, and their projects need to be capable of reorganizing into new forms, which are a better fit with the new context” (Findlay and Straus, 2015, p. 494).

Systems Engineering Practices

One of the tools Findlay and Straus use to deal with all three challenges in the context of group interaction, such as the ICCPM round-tables, is the Zing complex adaptive meeting process. The process is used to guide conversation in the room, to capture, simultaneously display ideas and to help participants integrate and make meaning from the ideas. The tool was used for the round-tables to help people work together in new ways, develop new and better models of the system together and to design and pilot new and better decision and learning methods.

The technology “provide[s] a container for a suitably representative sample of the people in the system to meet and conceptualize a robust model of the system and develop strategies for how to leverage the system” (Findlay and Straus, 2015, p. 492).

A “talk-type-read-review” (Findlay and Straus, 2015, p. 492) etiquette was employed to organize the session, which, in complexity theory terms, is a simple rule of interaction. Rich, open-ended questions guide the conversations, the ideas are read out loud and the common themes or stand out ideas are recorded by the facilitators.

The open-ended questions are asked one at a time to explore all possibilities and reduce complexity. Although, round-table participants often held opposing views at the beginning of the session, through a processes of continual, iterative feedback, they ultimately arrived at similar or complementary conclusions by the end of the round-table.

The process “automates”—or helps participants engage in—ways of interacting that incorporate a higher level of cognitive complexity than the participants might engage in individually, thus facilitating a shift in the group to a higher level of system performance.

Lessons Learned

The role of leaders of complex projects is to help their organization systems successfully deliver on time and on budget amidst constant change. Their mandate is to deliver amazing new solutions while making few or no mistakes—a challenging goal even in far less complex environments. In order to be successful, project leaders (and their teams) need new systems structures—new tools and methods—that reliably get better results. They need to have a robust and fresh understanding of the systems over which they preside and how they might influence them to greatest effect.

No longer can the complex project leader go off into a corner and design a project and then try to sell it to the community and political leaders, for example. Leaders now need to involve the whole system in the design of a project from its inception through to completion. They need to deal with wicked problems not by looking for the one best solution, but by integrating and leveraging competing ideas. This requires a shift in perspective: from attempting to “control” a complex project system as one might control a mechanical device, to understanding projects as highly complex and interconnected “living” systems that evolve over time. While we do not have “control” over our systems in the classical sense, we can exert influence very effectively, provided that we constantly update our understanding of what is going on and learn new ways to act and interact that are more likely to achieve our desired outcomes.

To achieve this, leaders need to develop the capacity to “anticipate the skills, leadership and coordination roles, technologies, methods, and processes that will be required to successfully surf the waves of change...” (Findlay and Straus, 2015, p. 501).

The 2012 ICCPM round-table series discussion paper (ICCPM, 2012) uses the example of a system undergoing transformation of many levels to illustrate the difficulty that complex project leaders face:

“The issue has been characterized as learning to fly a plane, while the plane is already in the air, and being re-assembled into another kind of transportation technology altogether. And, at the same time, the current passengers are disembarking and another group is boarding that demands a better quality of service or experience at lower cost than ever before. (ICCPM 2015 p. 21)”

This case study illustrates the need, in times of accelerating change, of “a real-time, systems-wide approach to the development of the methods and tools for managing complex projects” (Findlay and Straus, 2015, p. 500) so leaders can deal successfully and creatively with uncertainty and ambiguity.

References

Works Cited

- Ashby, R. 1956. *An Introduction to Cybernetics*. London, UK: Chapman and Hall.
- Bar Yam, Y. 2004. *Making Things Work: Solving Complex Problems in a Complex World*. Cambridge, MA, USA: Knowledge Press.
- Conant, R., and R. Ashby. 1970. "Every good regulator of a system must be a model of that system". *International Journal of System Sciences*. 1(2): 89-97.
- Findlay, J., and A. Straus. 2011. "A shift from systems to complex adaptive systems thinking". In O. Bodrova and N. Mallory (Eds.), *Complex Project Management Task Force Report: Compendium of Working Papers*. Canberra, Australia: International Centre for Complex Project Management: 24-26.
- Findlay, J., and A. Straus. 2015. "Complex Adaptive Operating System: Creating Methods for Complex Project Management". In *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*. Gorod, A., White, B.E., Ireland, V., Gandhi, S.J., and Sauser, B. (Eds.). Boca Raton, Florida, USA: CRC Press: 471-505.
- Gragert, T. 2013. *Triple loop learning*. Thorston's site. Available https://www.thorsten.org/wiki/index.php?title=Triple_Loop_Learning. Accessed 12 June, 2015.
- IBM. 2008. *Making Change Work*. New York, NY, USA: IBM.
- ICCPM. 2012. *Roundtable discussion paper: Complexity in a time of global financial change: Program delivery for the new economy*. Findlay, J., Straus, A., and Hatcher, C. (Eds). Canberra, Australia: International Centre for Complex Project Management.
- Meadows, D. 2008. *Thinking in Systems: A Primer*. White River Junction, Vermont, USA: Chelsea Green Publishing.
- Requisite Variety. 2015. *What is Requisite Variety?*. Available <https://requisitevariety.co.uk/what-is-requisite-variety>. Accessed 1 June, 2015.
- Wheatley, M. 1999. *Leadership and the New Science: Discovering Order in a Chaotic World*. San Francisco, California, USA: Berrett-Koehler.

Primary References

Findlay, J., and Straus, A. 2015. "Complex adaptive operating systems: Creating methods for complex project management". In *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*. Gorod, A., White, B.E., Ireland, V., Gandhi, S.J., and Sauser, B. (Eds.). Boca Raton: CRC Press: 471-505.

Additional References

None.

Medical System Examples

Next Generation Medical Infusion Pump

- Lead Author:
- Heidi Davidz

This case study summarizes the systems engineering aspects of the next-generation Symbiq™ IV (intravenous) medical pump development. Symbiq™ was developed by Hospira Inc. and documented in detail in Chapter 5 of the National Research Council book, *Human-System Integration in the System Development Process*. As described in the book, Symbiq™'s purpose was “to deliver liquid medications, nutrients, blood and other solutions at programmed flow rates, volumes and time intervals via intravenous and other routes to a patient, primarily for hospital use with secondary limited feature use by patients at home” (Pew 2007).

Domain Background

This case study provides insight into the use of systems engineering practices in a medical application.

Case Study Background

The project that is the subject of this report was approved by the Governing Board of the National Research Council, whose members are drawn from the councils of the National Academy of Sciences, the National Academy of Engineering, and the Institute of Medicine.

The study was supported by Award Nos. W911NF-05-0150 and FA5650-06-1-6610 between the National Academy of Sciences, the U.S. Department of the Army, and the U.S. Department of the Air Force.

Case Study Description

In creating a next-generation product, Hospira proposed to introduce new IV pump features, such as:

- multi-channel vs. single-channel liquid delivery;
- the ability to group multi-channeled devices together;
- associated user-programming capabilities and programmable drug libraries for specifying parallel delivery of liquids;
- use of color touchscreen devices;
- integration with numerous types of hospital information systems;
- ease of use for both medical personnel and patients at home;
- handling of potential hardware, software, and human-user faults;
- compliance with U.S. and international safety standards;
- use of alternating-current or battery power; and
- the ability to be cost-competitive and attractive to traditional medical and hospital administration personnel.

Many of these features are highly coupled, such as the multi-channel hardware controls, concurrent software synchronization, distinctive displays and alarms for multi-channel devices, and rigorous medical safety standards.

Views of the resulting medical infusion pump can be found as Figures 5-5 and 5-6 in Chapter 5, page 107 of the Pew and Mavor (2007) book. Systems engineering for the device involved a great deal of concurrent analysis and engineering of its hardware, software, human factors, operational, business, and safety aspects. It has been a

commercial success and won the 2006 Human Factors and Ergonomics Society's User-Centered Product Design Award and the 2007 Medical Design Excellence Award.

Not only were there numerous technical challenges in the development of Symbiq™, but there were also challenges in the systems engineering of a product with a life-cycle operational concept that would produce satisfactory outcomes for a wide variety of product and operational stakeholders whose value propositions were often in some conflict. Some stakeholders wanted numerous features that would require a complex user interface, while others wanted a simple and easy to learn interface. Some users wanted the most advanced color touchscreen displays available, while others wanted a simpler, less-expensive product that was harder to misuse due to inadvertent screen commands. Some organizations felt that a minimal interpretation of the required safety features would be acceptable, while others advocated ultrahigh assurance levels. Some marketing personnel wanted a quick development and fielding of the basic product to capture market share, while maintainers wanted initial built-in life cycle support, maintenance, and diagnostic capabilities.

In such situations, many organizations focus on making quick requirement decisions and rapidly proceed into development. However, Hospira's understanding of the uncertainties and risks caused them to pursue a risk-driven, incremental commitment course of buying information to reduce risk, as emphasized in the SEBoK Part 3 knowledge area on Risk Management. As described in Pew and Mavor (2007), Hospira used a version of the Incremental Commitment Spiral Model (ICSM) summarized in the SEBoK Part 3 Knowledge Area on representative systems engineering process models. The following sections describe the project's incremental system definition progress through the ICSM exploration, valuation, foundations, and Development phases. Some evolution of terminology has occurred, the Pew and Mavor (2007) version uses ICM instead of ICSM and "architecting phase" instead of "foundations phase".

Symbiq™ Exploration Phase Summary

In the exploration phase, the project carried out numerous analyses on stakeholder needs, technical opportunities, and business competition. Using these analyses, the project team determined ranges of preferred options. Stakeholder needs analyses included contextual inquiry via shadowing of nurses using IV pumps and followup interviews, as well as creating task flow diagrams, use environment analyses, and user profiles analyses. Technical opportunity analyses included initial conceptual designs of multi-channel pump configurations, evaluation of commercially available single-color and multicolor display devices with touchscreen capabilities, and software approaches for specifying multi-channel delivery options and synchronizing concurrent processes.

Business competition analyses included hiring a management and marketing planning firm to examine next-generation pump competitor strengths and weaknesses with respect to such capabilities as the number of pump channels, therapies, programming options, air-in-line management, battery and alternating current capabilities, biomedical domain expertise, and alarms. Several key competitive advantages of a next-generation pump were identified, such as the ability to read bar-codes, small size, light weight, stand-alone functional channels, an extensive drug library, a high level of reliability, and clear mapping of screen displays and pumping channels.

Market research and market segment analyses also identified market windows, pricing alternatives, hospital purchasing decision-making trends, and safety aspects. These were iterated by focus groups of key thought leaders in critical care. The results were factored into a product concept plan, cost analysis, and business case analysis. These were independently reviewed by experts as part of the ICSM Valuation Phase Commitment Review process, which resulted in a go-ahead decision with an identification of several risks to be managed.

Symbiq™ Valuation Phase Summary

The valuation phase focused on the major risks highlighted in the Valuation Commitment Review, such as the multi-channel pump options, the types of programmable therapies, the need for tailorable medication libraries, the display screen and user interface options, and the safety considerations. The valuation phase also elaborated the product concept plan for the most attractive general set of options, including a development plan and operations plan, along with an associated cost analysis, risk analysis, and business case for review at the Foundations Commitment Review.

The multi-channel pump options were explored via several hardware industrial design mockups and early usability tests of the mockups. These included evaluation of such desired capabilities as semi-automatic cassette loading, special pole-mounting hardware, stacking of and total number of channels, and tubing management features. The evaluations led to the overall all choice to use a semi-automatic cassette loading capability with a red-yellow-green LED display to indicate concerns with the loading mechanism and with the pump in general.

Field exercises with prototypes of the pole mountings indicated the need for quick release and activation mechanisms, which were subsequently implemented. Risk analyses of alternative stacking mechanisms and the potential number of channels available established a preference for side-by-side stacking, a decision to develop one and two channel units, and to support a maximum of four channels in a stacked configuration.

The types of programmable therapies considered included continuous delivery for a specified time period, patient weight-based dosing, piggyback or alternating delivery between the two channels, tapered or ramped-rate delivery, intermittent-interval delivery, variable-time delivery, and multistep delivery. These were evaluated via prototyping of the software on a simulated version of the pump complexes and were iterated until satisfactory versions were found.

Evaluation of the tailorable medication libraries addressed the issue that different hard and soft safety limits were needed for dosages in different care settings (e.g., emergency room, intensive care, oncology, pediatric care, etc.) which creates a need for hospitals to program their own soft limits (overridable by nurses with permission codes) and hard limits (no overrides permitted). Stakeholder satisfaction with the tailoring features was achieved via prototype exercises and iteration with representative hospital personnel.

A literature review was conducted to determine the relative advantages and disadvantages of leading input and display technologies, including cost and reliability data. After down-selecting to three leading vendors of touch screen color LCD displays and further investigating their costs and capabilities, a business risk analysis focused on the trade-offs between larger displays and customer interest in small-footprint IV pumps. The larger display was selected based on better readability features and the reduced risk of accidental user entries since the larger screen buttons would help to avoid these occurrences. Extensive usability prototyping was done with hardware mockups and embedded software that delivered simulated animated graphic user interface (GUI) displays to a touchscreen interface that was integrated into the hardware case.

The safety risk analysis in the valuation phase followed ISO 14971:2000 standards for medical device design, focusing on Failure Modes and Effects Analyses (FMEAs). This analysis was based on the early high-level design, such as entry of excessive drug doses or misuse of soft safety limit overrides. Subsequent-phase FMEAs would elaborate this analysis, based on the more detailed designs and implementations.

As in the exploration phase, the results of the valuation phase analyses, plans, budgets for the succeeding phases, the resulting revised business case, evidence of solution feasibility, and remaining risks with their risk management plans were reviewed by independent experts and the ICSM Foundations Commitment Review was passed, subject to a few risk level and risk management adjustments.

Symbiq™ Foundations Phase Summary

During the foundations phase, considerable effort was focused on addressing the identified risks such as the need for prototyping the full range of GUI usage by the full range of targeted users, including doctors, home patients, the need for interoperability of the Symbiq™ software with the wide variety of available hospital information systems, and the need for fully detailed FMEAs and other safety analyses. Comparable added effort went into detailed planning for development, production, operations, and support, providing more accurate inputs for business case analyses.

GUI prototyping was done with a set of usability objectives, such as

- 90% of experienced nurses will be able to insert the cassette the first time while receiving minimal training;
- 99% will be able to correct any insertion errors;
- 90% of first time users with no training will be able to power the pump off when directed; and
- 80% of patient users would rate the overall ease of use of the IV pump three or higher on a five-point scale (with five being the easiest to use).

Similar extensive evaluations were done on the efficacy and acceptability of the audio alarms, including the use of a patient and intensive care unit simulator that included other medical devices that produced noises, as well as other distractions such as ringing telephones. These evaluations were used to enable adjustment of the alarms and to make the visual displays easier to understand.

Software interoperability risk management involved extensive testing of representative interaction scenarios between the Symbiq™ software and a representative set of hospital information systems. These resulted in several adjustments to the software interoperability architecture. Also, as the product was being developed as a platform for the next generation of infusion pump products, the software design was analyzed for overspecialization to the initial product, resulting in several revisions. Similar analyses and revisions were performed for the hardware design.

As the design was refined into complete build-to specifications for the hardware and the operational software, the safety analyses were elaborated into complete FMEAs of the detailed designs. These picked up several potential safety issues, particularly involving the off-nominal usage scenarios, but overall confirmed a high assurance level for the safety of the product design. However, the safety risk assessment recommended a risk management plan for the development phase to include continued FMEAs, thorough off-nominal testing of the developing product's hardware and software, and extensive beta-testing of the product at representative hospitals prior to a full release.

This plan and the other development and operations phase plans, product feasibility evidence, and business case analysis updates were reviewed at a Development Commitment Review, which resulted in a commitment to proceed into the development phase.

Symbiq™ Development Phase Systems Engineering Summary

The development phase was primarily concerned with building and testing the hardware and software to the build-to specifications, but continued to have an active systems engineering function to support change management; operations, production, and support planning and preparation; and further safety assurance activities as recommended in the risk management plan for the phase.

For hospital beta-testing, thoroughly bench-tested and working beta versions of the IV pump were deployed in two hospital settings. The hospitals programmed drug libraries for at least two clinical care areas. The devices were used for about four weeks. Surveys and interviews were conducted with the users to capture their “real world” experiences with the pump. Data from the pump usage and interaction memory was also analyzed and compared to the original doctors' orders. The beta tests revealed a number of opportunities to make improvements, including revision of the more annoying alarm melodies and the data entry methods for entering units of medication delivery time in hours or minutes.

Usability testing was also conducted on one of the sets of abbreviated instructions called TIPS cards. These cards serve as reminders for how to complete the most critical tasks. Numerous suggestions for improvement in the TIPS cards themselves, as well as the user interface, came from this work, including how to reset the “Air-in-Line” alarm and how to check all on-screen help text for accuracy.

The above mentioned usability objectives were used as acceptance criteria for the validation usability tests. These objectives were met. For example, the calculated task completion accuracy was 99.66% for all tasks for first time nurse users with minimal training. There were a few minor usability problems uncovered that were subsequently fixed without major changes to the GUI or effects on critical safety related tasks.

The risk analysis was iterated and revised as the product development matured. FMEAs were updated for safety critical risks associated with three product areas: the user interface, the mechanical and electrical subsystems, and the product manufacturing process. Some detailed implementation problems were found and fixed, but overall the risk of continuing into full-scale production, operations, and support was minimal. Systems engineering continued into the operations phase, primarily to address customer change requests and problem reports, and to participate in planning for a broader product line of IV pumps.

Overall, customer satisfaction, sales, and profits from the Symbiq™ IV pump have been strong and satisfaction levels from the management, financial, customer, end user, developer, maintainer, regulatory, and medical-community stakeholders have been quite high (Pew 2007).

Summary

In summary, the Symbiq™ Medical Infusion Pump Case Study provides an example of the use of the systems engineering practices discussed in the SEBoK. As appropriate for a next-generation, advanced technology product, it has a strong focus on risk management, but also illustrates the principles of synthesis, holism, dynamic behavior, adaptiveness, systems approach, progressive entropy reduction, and progressive stakeholder satisfying discussed in Part 2 of the SEBoK. It provides an example of an evolutionary and concurrent systems engineering process, such as the incremental commitment spiral process, and of other knowledge areas discussed in SEBoK Parts 3 and 4, such as system definition, system realization, system engineering management, and specialty engineering.

References

Works Cited

Pew, R. and A. Mavor. 2007. *Human-System Integration in the System Development Process: A New Look*. Washington, DC, USA: The National Academies Press.

Primary References

None.

Additional References

None.

Medical Radiation

- Lead Authors:
- Heidi Davidz and John Brackett

This case study presents system and software engineering issues relevant to the accidents associated with the Therac-25 medical linear accelerator that occurred between 1985 and 1988. The six accidents caused five deaths and serious injury to several patients. The accidents were system accidents that resulted from complex interactions between hardware components, controlling software, and operator functions.

Domain Background

Medical linear accelerators, devices used to treat cancer, accelerate electrons to create high energy beams that can destroy tumors. Shallow tissue is treated with the accelerated electrons. The electron beam is converted to X-ray photons to reach deeper tissues. Accidents occur when a patient is delivered an unsafe amount of radiation.

A radiation therapy machine is controlled by software that monitors the machine's status, accepts operator input about the radiation treatment to be performed, and initializes the machine to perform the treatment. The software turns the electron beam on in response to an operator command. The software turns the beam off whenever the treatment is complete, the operator requests the beam to shut down, or when the hardware detects a machine malfunction. A radiation therapy machine is a reactive system in which the system's behavior is state dependent and the system's safety depends upon preventing entry into unsafe states. For example, the software controls the equipment that positions the patient and the beam. The positioning operations can take a minute or more to execute, thus it is unsafe to activate the electron beam while a positioning operation is in process.

In the early 1980s, Atomic Energy of Canada (AECL) developed the Therac-25, a dual-mode (X-rays or electrons) linear accelerator that can deliver photons at 25 megaelectron volts (MeV) or electrons at various energy levels. The Therac-25 superseded the Therac-20, the previous 20-MeV dual mode accelerator with a history of successful clinical use. The Therac-20 used a DEC PDP-11 (Digital Equipment Corporation Programmed Data Processor) minicomputer for computer control and featured protective circuits for monitoring the electron beam, as well as mechanical interlocks for policing the machine to ensure safe operation. AECL decided to increase the responsibilities of the Therac-25 software for maintaining safety and eliminated most of the hardware safety mechanisms and interlocks. The software, written in PDP-11 assembly language, was partially reused from earlier products in the Therac product line. Eleven Therac-25s were installed at the time of the first radiation accident in June 1985.

The use of radiation therapy machines has increased rapidly in the last 25 years. The number of medical radiation machines in the United States in 1985 was approximately 1000. By 2009 the number had increased to approximately 4450. Some of the types of system problems found in the Therac-25 may be present in the medical radiation devices currently in use. References to more recent accidents are included below.

Case Study Background

The Therac-25 accidents and their causes are well documented in materials from the U.S. and Canadian regulatory agencies (e.g., the U.S. Food and Drug Administration (FDA) and the Canadian Bureau of Radiation and Medical Devices) and in depositions associated with lawsuits brought against AECL. An article by Leveson and Turner (1993) provides the most comprehensive, publicly available description of the accident investigations, the causes of the accidents, and the lessons learned relevant to developing systems where computers control dangerous devices.

Case Study Description

The Therac-25 accidents are associated with the non-use or misuse of numerous system engineering practices, especially system verification and validation, risk management, and assessment and control. In addition, numerous software engineering good practices were not followed, including design reviews, adequate documentation, and comprehensive software unit and integration tests.

The possibility of radiation accidents increased when AECL made the systems engineering decision to increase the responsibilities of the Therac-25 software for maintaining safety and eliminated most of the hardware safety mechanisms and interlocks. In retrospect, the software was not worthy of such trust. In 1983 AECL performed a safety assessment on the Therac-25. The resulting fault tree did include computer failures, but only those associated with hardware; software failures were not considered in the analysis.

The software was developed by a single individual using PDP-11 assembly language. Little software documentation was produced during development. An AECL response to the FDA indicated the lack of software specifications and of a software test plan. Integrated system testing was employed almost exclusively. Leveson and Turner (1993) described the functions and design of the software and concluded that there were design errors in how concurrent processing was handled. Race conditions resulting from the implementation of multitasking also contributed to the accidents.

AECL technical management did not believe that there were any conditions under which the Therac-25 could cause radiation overdoses, and this belief was evident in the company's initial responses to accident reports. The first radiation overdose accident occurred in June 1985 at the Kennestone Regional Oncology Center in Marietta, Georgia, where the Therac-25 had been operating for about 6 months. The patient who suffered the radiation overdose filed suit against the hospital and AECL in October 1985. No AECL investigation of the incident occurred and FDA investigators later found that AECL had no mechanism in place to follow up potential reports of suspected accidents. Additionally, other Therac-25 users received no information that an accident had occurred.

Two more accidents occurred in 1985, including a radiation overdose at Yakima Valley Memorial Hospital in Yakima, Washington that resulted in an accident report to AECL. The AECL technical support supervisor responded to the hospital in early 1986: "After careful consideration, we are of the opinion that this damage could not have been produced by any malfunction of the Therac-25 or by any operator error... there have apparently been no other instances of similar damage to this or other patients."

In early 1986 there were two accidents at the East Texas Cancer Center in Tyler, Texas, both of which resulted in the death of the patient within a few months. On March 21, 1986 the first massive radiation overdose occurred, though the extent of the overdose was not realized at the time. The Therac-25 was shut down for testing the day after the accident. Two AECL engineers, one from the plant in Canada, spent a day running machine tests but could not reproduce the malfunction code observed by the operator at the time of the accident. The home office engineer explained that it was not possible for the Therac-25 to overdose a patient. The hospital physicist, who supervised the use of the machine, asked AECL if there were any other reports of radiation overexposure. The AECL quality assurance manager told him that AECL knew of no accidents involving the Therac-25.

On April 11, 1986 the same technician received the same malfunction code when an overdose occurred. Three weeks later the patient died; an autopsy showed acute high-dose radiation injury to the right temporal lobe of the brain and to the brain stem. The hospital physicist was able to reproduce the steps the operator had performed and measured the high radiation dosage delivered. He determined that data-entry speed during editing of the treatment script was the key factor in producing the malfunction code and the overdose. Examination of the portion of the code responsible for the Tyler accidents showed major software design flaws. Levinson and Turner (1993) describe in detail how the race condition occurred in the absence of the hardware interlocks and caused the overdose. The first report of the Tyler accidents came to the FDA from the Texas Health Department. Shortly thereafter, AECL provided a medical device accident report to the FDA discussing the radiation overdoses in Tyler.

On May 2, 1986 the FDA declared the Therac-25 defective and required the notification of all customers. AECL was required to submit to the FDA a corrective action plan for correcting the causes of the radiation overdoses. After multiple iterations of a plan to satisfy the FDA, the final corrective action plan was accepted by the FDA in the summer of 1987. The action plan resulted in the distribution of software updates and hardware upgrades that reinstated most of the hardware interlocks that were part of the Therac-20 design.

AECL settled the Therac-25 lawsuits filed by patients that were injured and by the families of patients who died from the radiation overdoses. The total compensation has been estimated to be over \$150 million.

Summary

Leveson and Turner (1993) describe the contributing factors to Therac-25 accidents: "We must approach the problems of accidents in complex systems from a systems-engineering point of view and consider all contributing factors." For the Therac-25 accidents, the contributing factors included:

- management inadequacies and a lack of procedures for following through on all reported incidents;
- overconfidence in the software and the resulting removal of hardware interlocks (causing the software to be a single point of failure that could lead to an accident);
- less than acceptable software engineering practices; and
- unrealistic risk assessments along with over confidence in the results of those assessments.

Recent Medical Radiation Experience

Between 2009 and 2011, The New York Times published a series of articles by Walter Bogdanich on the use of medical radiation, entitled "Radiation Boom" (2011).

The following quotations are excerpts from that series:

Increasingly complex, computer-controlled devices are fundamentally changing medical radiation, delivering higher doses in less time with greater precision than ever before." But patients often know little about the harm that can result when safety rules are violated and ever more powerful and technologically complex machines go awry. To better understand those risks, The New York Times examined thousands of pages of public and private records and interviewed physicians, medical physicists, researchers and government regulators. The Times found that while this new technology allows doctors to more accurately attack tumors and reduce certain mistakes, its complexity has created new avenues for error — through software flaws, faulty programming, poor safety procedures or inadequate staffing and training. . . .

Linear accelerators and treatment planning are enormously more complex than 20 years ago,' said Dr. Howard I. Amols, chief of clinical physics at Memorial Sloan-Kettering Cancer Center in New York. But hospitals, he said, are often too trusting of the new computer systems and software, relying on them as if they had been tested over time, when in fact they have not. . . .

Hospitals complain that manufacturers sometimes release new equipment with software that is poorly designed, contains glitches or lacks fail-safe features, records show. Northwest Medical Physics Equipment in Everett, Wash., had to release seven software patches to fix its image-guided radiation treatments, according to a December 2007 warning letter from the F.D.A. Hospitals reported that the company's flawed software caused several cancer patients to receive incorrect treatment, government records show.

References

Works Cited

Bogdanich, W. 2011. Articles in the "Radiation Boom" series. *New York Times*. June 2009-February 2011. Accessed November 28, 2012. Available: https://topics.nytimes.com/top/news/us/series/radiation_boom.

Leveson, N.G., and C.S. Turner. 1993. "An Investigation of the Therac-25 Accidents." *Computer*. 26 (7): 18-41.

Primary References

None.

Additional References

None.

Design for Maintainability

- Lead Author:
- Rick Adcock

This article describes an example of where systems thinking led to a much more practical solution to a common problem. For additional information, refer to Systems Thinking.

This article is excerpted and condensed from Johnson, S. 2010. *Where Good Ideas Come From: The Natural History of Innovation*. New York, NY, USA: Riverhead Books. pp. 25-28.

Background

In the late 1870s, a Parisian obstetrician named Stephane Tarnier was visiting the Paris Zoo where they had farm animals. While there, he conceived the idea of adapting a chicken incubator to use for human newborns, and he hired "the zoo's poultry raiser to construct a device that would perform a similar function for human newborns." At the time infant mortality was staggeringly high "even in a city as sophisticated as Paris. One in five babies died before learning to crawl, and the odds were far worse for premature babies born with low birth weights." Tarnier installed his incubator for newborns at Maternité de Paris and embarked on a quick study of five hundred babies. "The results shocked the Parisian medical establishment: while 66 percent of low-weight babies died within weeks of birth, only 38 percent died if they were housed in Tarnier's incubating box. ... Tarnier's statistical analysis gave newborn incubation the push that it needed: within a few years the Paris municipal board required that incubators be installed in all the city's maternity hospitals." ...

Purpose

"Modern incubators, supplemented with high-oxygen therapy and other advances, became standard equipment in all American hospitals after the end of World War II, triggering a spectacular 75 percent decline in infant mortality rates between 1950 and 1998."... "In the developing world, however, the infant mortality story remains bleak. Whereas infant deaths are below ten per thousand births throughout Europe and the United States, over a hundred infants die per thousand (births) in countries like Liberia and Ethiopia, many of them premature babies that would have survived with access to incubators."

Challenges

But modern incubators are complex, expensive things. A standard incubator in an American hospital might cost more than \$40,000 [about €30,000]. But the expense is arguably the smaller hurdle to overcome. Complex equipment breaks and when it breaks you need both the technical expertise to fix it and replacement parts. In the year that followed the 2004 Indian Ocean tsunami, the Indonesian city of Meulaboh received eight incubators from a range of international relief organizations. By late 2008, when an MIT professor named Timothy Prestero visited the hospital, all eight were out of order, the victims of power surges and tropical humidity, along with the hospital staff's inability to read the English repair manual. The Meulaboh incubators were a representative sample: some studies suggest that as much as 95 percent of medical technology donated to developing countries breaks within the first five years of use.

Systems Engineering Practices

"Prestero had a vested interest in those broken incubators, because the organization he founded, Design that Matters, had been working for several years on a scheme for a more reliable, and less expensive, incubator, one that recognized complex medical technology was likely to have a very different tenure in a developing world context than it would in an American or European hospital. Designing an incubator for a developing country wasn't just a matter of creating something that worked; it was also a matter of designing something that would break in a non-catastrophic way. You couldn't guarantee a steady supply of spare parts, or trained repair technicians. So instead, Prestero and his team decided to build an incubator out of parts that were already abundant in the developing world. The idea had originated with a Boston doctor named Jonathan Rosen, who had observed that even the smaller towns of the developing world seemed to be able to keep automobiles in working order. The towns might lack air conditioning and laptops and cable television, but they managed to keep their Toyota 4Runners on the road. So Rosen approached Prestero with an idea: What if you made an incubator out of automobile parts?

Lessons Learned

"Three years after Rosen suggested the idea, the Design that Matters team introduced a prototype device called NeoNurture. From the outside, it looked like a streamlined modern incubator, but its guts were automotive. Sealed-beam headlights supplied the crucial warmth; dashboard fans provided filtered air circulation; door chimes sounded alarms. You could power the device via an adapted cigarette lighter, or a standard-issue motorcycle battery. Building the NeoNurture out of car parts was doubly efficient, because it tapped both the local supply of parts themselves and the local knowledge of automobile repair. These were both abundant resources in the developing world context, as Rosen liked to say. You didn't have to be a trained medical technician to fix the NeoNurture; you didn't even have to read the manual. You just needed to know how to replace a broken headlight."

References

Works Cited

Johnson, S. 2010. *Where Good Ideas Come From: The Natural History of Innovation*. New York, NY, USA: Riverhead Books. pp. 25-28.

Primary References

Johnson, S. 2010. *Where Good Ideas Come From: The Natural History of Innovation*. New York, NY, USA: Riverhead Books. pp. 25-28.

Additional References

None.

Space System Examples

Global Positioning System

- Lead Authors:
- Heidi Davidz, Richard Freeman, and Alice Squires
- Contributing Authors:
- Tom Hilburn and Brian White

The Global Positioning System (GPS) case study was developed by the United States Air Force Center for Systems Engineering (AF CSE) located at the Air Force Institute of Technology (AFIT). The GPS is a space-based radio-positioning system. A constellation of twenty-four satellites, including three spares, comprise the overall system which provides navigation and timing information to military and civilian users worldwide. GPS satellites, in one of six Earth orbits, circle the globe every twelve hours, emitting continuous navigation signals on two different L-band frequencies. The system consists of two other major segments: a world-wide satellite control network, and the GPS user equipment that can either be carried by a human user or integrated into host platforms such as ships, vehicles, or aircraft.

This case study discussion is based on the original source (O'Brien and Griffin 2007), which provides useful insights into what we might consider a "traditional" SE application. A second global positioning case study looks at the same case study from the perspectives of system of systems (sos) engineering and enterprise systems engineering (ese).

Domain Background

When looking at the Global Positioning System (GPS), it would be difficult to imagine another system that relies so heavily upon such a wide range of domains, with the possible exception of the World Wide Web (WWW). Additionally, the various systems operating within these domains must all function together flawlessly to achieve success. It is evident from reading this case study that it directly relates to the following domains:

- aerospace;
- space;
- communications; and
- transportation.

This is also an example of systems of systems (SoS) and is considered an innovative technology.

The GPS case study includes a detailed discussion of the development of the GPS and its components, as well as other applicable areas. The reader of this study will gain an increased understanding of the effect that GPS has on military and commercial industries in the context of the systems engineering support required to achieve success.

Case Study Background

The United States Air Force Center for Systems Engineering (AF CSE), established in 2002 at the Air Force Institute of Technology (AFIT), was tasked to develop case studies focusing on the application of systems engineering principles within various aerospace programs. The GPS case study (O'Brien and Griffin 2007) was developed by AFIT in support of systems engineering graduate school instruction. The cases are structured using the Friedman-Sage framework (Friedman and Sage 2003; Friedman and Sage 2004, 84-96), which decomposes a case into contractor, government, and shared responsibilities in the following nine concept areas:

1. Requirements Definition and Management
-

2. Systems Architecture Development
3. System/Subsystem Design
4. Verification/Validation
5. Risk Management
6. Systems Integration and Interfaces
7. Life Cycle Support
8. Deployment and Post Deployment
9. System and Program Management

The Friedman-Sage framework (2004) is provided in Appendix A of the case study. This case study is an example where the government - specifically the JPO Systems Engineering Directorate - bore the responsibility for systems integration and configuration management. That is, the government played more than an oversight role in the systems engineering of the GPS system of systems. As mentioned in the case study, JPO developed the CONOPs, mission analysis, requirements and design analysis including security, and developed their own approach to the cryptology methodology. JPO coordinated the Configuration Control Board (CCB) chaired by the Program Director. JPO was also responsible for Level I ICDs and system design configurations; where the contractors were responsible for the system architecture and ICDs within their segment.

Case Study Description

The “Global Positioning System - Systems Engineering Case Study” describes the application of systems engineering during the concept validation, system design and development, and production phases of the GPS program (O'Brien and Griffin 2007). The case examines the applied systems engineering processes, as well as the interactions of the GPS joint program office (JPO), the prime contractors, and the plethora of government agencies that were associated with the program's development and fielding. The systems engineering process is traced from the initiation of studies and the development of key technologies, which established the vision of a satellite navigation system in the 1960s, through to the multiphase joint program that resulted in a fully operational capability release in 1995. This case study does not cover system enhancements incorporated through Blocks IIM, IIF, and III.

The GPS case study derived four learning principles (LPs) that explain the more broadly applicable areas of systems engineering knowledge that are addressed by the case study. These four LPs relate strongly to the SEBoK in the following areas:

- enabling individuals (LP1);
- configuration management (LP2);
- enabling the organization (LP3); and
- risk management (LP4).

Additionally, the GPS case study contains a thorough overview of life cycle management and exemplifies systems thinking principles.

Enabling Individuals

Learning Principle 1: Programs must strive to staff key positions with domain experts.

From the program management team, to the systems engineering, design, manufacturing, and operations teams, the individuals on the program were well-versed in their disciplines and all possessed a systems view of the program. While communications, working relationships, and organization were important, it was the ability of the whole team at all levels to understand the implications of their work on the system that was vital. Their knowledge-based approach for decision making had the effect of shortening the decision cycle because the information was understood and the base and alternative solutions were accurately presented.

Configuration Management

Learning Principle 2: The systems integrator must rigorously maintain program baselines.

The joint program office (JPO) retained the role of managing and controlling the system specification and, therefore, the functional baseline. The JPO derived and constructed a mutually-agreed-to set of system requirements that became the program baseline in 1973. While conducting the development program, the GPS team was able to make performance, risk, cost, and trade analyses against the functional baseline to control both risk and cost. The JPO was fully cognizant of the implications of the functional requirements on the allocated baseline because they managed the interface control working group process. Managing that process gave them first-hand knowledge and insight into the risks at the lowest level. The individual with the system integrator role must rigorously maintain the system specification and functional baseline. There must be appropriate sharing of management and technical responsibilities between the prime contractor and their government counterparts to ensure success.

Enabling the Organization

Learning Principle 3: Achieving consistent and continuous high-level support and advocacy helps funding stability, which impacts systems engineering stability.

Consistent, continuous high-level support provides the requirements and assists funding stability. In this role, the Office of the Secretary of Defense (OSD) provided advocacy and sourced the funding at critical times in the program, promoted coordination among the various services, and reviewed and approved the GPS JPO system requirements. The OSD played the central role in the establishment and survivability of the program. The GPS JPO had clear support from the Director of Defense Development, Research, and Engineering, Dr. Malcolm Currie, and program support from the Deputy Secretary of Defense, Dr. David Packard. Clearly, the armed services – particularly the Navy and the Air Force early on, and later the Army – were the primary users of GPS and the eventual customers. However, each armed service had initial needs for their individual programs, or for the then-current operational navigation systems. Additionally, the secretary of the Air Force provided programmatic support to supply manpower and facilities.

Risk Management

Learning Principle 4: Disciplined and appropriate risk management must be applied throughout the life cycle.

The GPS program was structured to address risk in several different ways throughout the multiphase program. Where key risks were known up front, the contractor and/or the government utilized a classic risk management approach to identify and analyze risk, as well as develop and track mitigation actions. These design (or manufacturing/launch) risks were managed by the office who owned the risks. Identified technical risks were often tracked by technical performance measures (such as satellite weight and software lines of codes) and addressed at weekly chief engineer's meetings.

Serving in the clear role of program integrator allowed the JPO to sponsor risk trade studies at the top level. The JPO would issue study requests for proposals to several bidders for developing concepts and/or preliminary designs. Then, one contractor would be down-selected and the process would continue. This approach provided innovative solutions through competition, as well as helped in defining a lower risk, more clearly defined development program for the fixed-price contracts approach that was being used for development and production.

As the system integrator, the JPO was also closely involved with technical development. To identify unforeseeable unique technical challenges, the JPO would fund studies to determine the optimal approaches to new issues. There were schedule risks associated with the first launch due to unforeseen Block II issues with respect to the space vehicle and control segments (software development). Although it was a catastrophic event, the Challenger accident actually provided much needed schedule relief. Using decision analysis methodology led the JPO to an alternative approach to develop the expendable launch vehicle for the Block II satellites.

Good communication, facilitated by cooperative working relationships, was a significantly positive (though intangible) factor in the success of the GPS program, regardless of whether it was between the contractors and the government (JPO or other agencies), or between contractors and sub-contractors. A true team environment also played a significant role in reducing risk, especially considering the plethora of government agencies and contractors that were involved in the effort.

Life Cycle Management

The GPS case study takes the reader through the initial concept of GPS (March 1942) all the way to the development, production, and operational capability of the system. The current GPS program traces its heritage to the early 1960s when Air Force Systems Command initiated satellite-based navigation systems analyses conducted by The Aerospace Corporation. The case study follows the execution of the GPS program from the inception of the idea to the full operational capability release on April 27th, 1995. The concentration of the case study is not limited to any particular period, and the learning principles come from various times throughout the program's life.

Systems Thinking

The GPS case study highlights the need for systems thinking throughout. GPS satellites, in one of six Earth orbits, circle the globe every twelve hours. These satellites emit continuous navigation signals on two different L-band frequencies. The system consists of two other major segments: a world-wide satellite control network and the GPS user equipment that can either be carried by a human user, or integrated into host platforms such as ships, vehicles, or aircraft. The ability to conceive, develop, produce, field, and sustain the GPS demands the highest levels of systems thinking.

Summary

The GPS case study is useful for global systems engineering learning and provides a comprehensive perspective on the systems engineering life cycle. The study is applicable for detailed instruction in the following areas:

- enabling individuals;
- configuration management;
- enabling the organization;
- risk management;
- life cycle management; and
- systems thinking.

The GPS case study revealed that key Department of Defense personnel maintained a clear and consistent vision for this unprecedented, space-based navigation capability. The case study also revealed that good fortune was enjoyed by the JPO as somewhat independent, yet critical, space technologies matured in a timely manner.

Although the GPS program required a large degree of integration, both within the system and external to the system, amongst a multitude of agencies and contractors, the necessary efforts were taken to achieve success.

Lastly, the reader of the GPS case study will gain an increased understanding of the effect that GPS has on the military and commercial industries in the context of the systems engineering support required to achieve success. The system was originally designed to help “drop five bombs in one hole” which defines the accuracy requirement in context-specific terms. The GPS signals needed to be consistent, repeatable, and accurate to a degree that, when used by munitions guidance systems, would result in the successful delivery of multiple, separately-guided munitions to virtually the identical location anywhere at any time across the planet. Forty to fifty years ago, very few outside of the military recognized the value of the proposed accuracy and most non-military uses of GPS were not recognized before 1990. GPS has increasingly grown in use and is now used every day.

References

Works Cited

Friedman, G.R. and A.P. Sage. 2003. *Systems Engineering Concepts: Illustration Through Case Studies*. Accessed September 2011. Available at: <https://www.afit.edu/cse/docs/Friedman-Sage%20Framework.pdf>.

Friedman, G. and A. Sage. 2004. "Case studies of systems engineering and management in systems acquisition." *Systems Engineering*. 7(1): p. 84-96.

O'Brien, Patrick J., and John M. Griffin. 4 October 2007. *Global Positioning System. Systems Engineering Case Study*. Air Force Center for Systems Engineering (AFIT/SY) Air Force Institute of Technology (AFIT). 2950 Hobson Way, Wright-Patterson AFB OH 45433-7765.

Primary References

O'Brien, Patrick J., and John M. Griffin. 4 October 2007. *Global Positioning System. Systems Engineering Case Study*. Air Force Center for Systems Engineering (AFIT/SY) Air Force Institute of Technology (AFIT). 2950 Hobson Way, Wright-Patterson AFB OH 45433-7765.

Additional References

none.

Global Positioning System II

- Lead Author:
- Brian White

This article highlights some of the differences between the so-called classical, traditional, or conventional systems engineering (SE) approaches and the newer, and, as yet, less defined principles of system of systems (SoS) engineering (SoSE), enterprise systems engineering (ESE), and/or complex systems engineering (CSE) or complex adaptive systems engineering (Gorod et al. 2015). The topic is still somewhat controversial, especially considering those that are skeptical that broader views of SE might work better when one is immersed in trying to cope with our most difficult problems. Indeed, the lack of a unified theory of SE is one of the prime motivations for producing and analyzing case studies to develop more knowledge of what seems to work, what does not seem to work, and reasons why, really challenging SE environments.

For addition information, refer to Systems Engineering: Historic and Future Challenges, Systems Engineering and Other Disciplines, Enterprise Systems Engineering, and System of Systems Engineering.

Rather than modifying the previous discussion of the Global Positioning System Case Study in SEBoK, the focus is on comparing and contrasting the older and newer forms of SE by commenting on quotations from the original case study source documents (O'Brien and Griffin 2007).

Preface

The original case study begins by describing systems engineering (SE) principles. For example,

System requirements are critical to all facets of successful system program development. First, system development must proceed from a well-developed set of requirements. Second, regardless of the evolutionary acquisition approach, the system requirements must flow down to all subsystems and lower-level components. And third, the system requirements must be stable, balanced, and must properly reflect all activities in all intended environments. However, system requirements are not

unchangeable. As the system design proceeds, if a requirement or set of requirements is proving excessively expensive to satisfy, the process must rebalance schedule, costs, and performance by changing or modifying the requirements or set of requirements. (O'Brien and Griffin 2007, p. 9)

The Global Positioning System (GPS), including its multi-various applications, was developed over many years as the result of the efforts of a host of contributors. It is very difficult to believe that the classical, traditional or conventional systems engineering approach described in the above paragraph (especially those phrases highlighted in bold by the present authors) was truly responsible for this remarkable achievement that so profoundly impacts our lives. Rather, some more advanced form of systems engineering (SE), that might be called system of systems (SoS) engineering (SoSE), enterprise systems engineering (ESE), or complex (adaptive) systems engineering (CSE), or a blend and/or combination of these approaches or methodologies, had to be responsible. This premise is supported explicitly and repeatedly in the following case study revision using bold font.

Continuing, the following quoted paragraphs seem flawed in several places highlighted in bold. The bold phrases might be replaced by the phrases in brackets [...]. Such brackets might also include other editorial comments of the present authors.

Systems engineering includes making key system and design trades early in the process to establish the system architecture. "These architectural artifacts" [This architecture] can depict any new system, legacy system, modifications thereto, introduction of new technologies, and overall system-level behavior and performance. Modeling and simulation are generally employed to organize and assess architectural system alternatives at this stage. System and subsystem design follows the functional [system] architecture [as defined from a functional point of view]. System architectures designs are modified if elements are too risky, expensive, or time-consuming. (O'Brien and Griffin 2007, p. 9)

A good architecture, once established, should guide systems development, and not change very much, if at all, at least compared to possible changes in the system design, which, of course, can evolve as one learns more about the problem and potential solutions that may increase the system's capability. Thus, it is crucial to not confuse architecture with designs instantiating the architecture, contrary to what seems to be the case in (Ricci, et al. 2013).

Important to the efficient decomposition and creation of functional and physical architectural designs are the management of interfaces and the integration of subsystems. Interface management and integration is applied to subsystems within a system or across a large, complex system of systems. Once a solution is planned, analyzed, designed, and constructed, validation and verification take place to ensure satisfaction of requirements. Definition of test criteria, measures of effectiveness (MOEs), and measures of performance (MOPs) are established as part of the requirements process, taking place well before any component/subsystem assembly design and construction occurs. (O'Brien and Griffin 2007, p. 10)

In the quoted paragraph just above, bold phrases note the emphasis on a reductionist approach, reductionism, where great attention is paid to the subsystems and managing the interfaces among them. This is the antithesis of a holistic approach where one concentrates on the whole system, recognizing that it is difficult to identify overall system behavior as depending on any particular subsystem or set of subsystems. In a truly complex system that is continually evolving, the above-mentioned requirements process is flawed because the system is continually changing, i.e., the system is evolutionary; the requirements are either ill-defined at the outset, or are modified because stakeholders change their minds, or become somewhat irrelevant because the system environment changes.

There are several excellent representations of the [usual traditional or conventional] systems engineering process presented in the literature. These depictions present the current state of the art in maturity and evaluation of the systems engineering process. One can find systems engineering process definitions, guides, and handbooks from the International Council on Systems Engineering (INCOSE), European Industrial Association (EIA), Institute of Electrical and Electronics Engineers (IEEE), and various Department of Defense (DoD) agencies and organizations. They show the process as it should be applied [Really? In all situations?] by today's experienced practitioner. One of these processes, long

used by the Defense Acquisition University (DAU), is [a model] not accomplished in a single pass. This iterative and nested process gets repeated to the lowest level of definition of the design and its interfaces. (O'Brien and Griffin 2007, p. 10)

The above description appears to be written with pride without any acknowledgement that this SE methodology might fail to work if applied according to these guidelines, or that there might be new SE techniques that could be more effective in some situations. Again, this reflects a reductionist approach that ignores holism and emergent properties that might not be explained even when thoroughly understanding the systems components and their interactions. On the positive side, the next paragraph suggests how the world is changing and hints that something more is needed. Nevertheless, the advice seems to be oriented toward applying the existing SE discipline more vigorously instead of seeking new methods that might be more effective.

The DAU model, like all others, has been documented in the last two decades, and has expanded and developed to reflect a changing environment. Systems are becoming increasingly complex internally and more interconnected externally. The process used to develop aircraft and systems of the past was effective at the time. It served the needs of the practitioners and resulted in many successful systems in our inventory. Notwithstanding, the cost and schedule performance of the past programs are replete with examples of well-managed programs and ones with less-stellar execution. As the nation entered the 1980s and 1990s, large DoD and commercial acquisitions experienced overrunning costs and slipping schedules. The aerospace industry and its organizations were becoming larger and were more geographically and culturally distributed. Large aerospace companies have worked diligently to establish common systems engineering practices across their enterprises. However, because of the mega-trend of teaming in large (and some small) programs, these common practices must be understood and used beyond the enterprise and to multiple corporations. It is essential that the systems engineering process govern integration, balance, allocation, and verification, and be useful to the entire program team down to the design and interface level. (O'Brien and Griffin 2007, p. 11)

Finally, in the next paragraph, there is a suggestion that SE could be made more sophisticated but there is no mention of addressing people problems or advocating a broader transdisciplinary approach.

Today, many factors overshadow new acquisition; including system-of-systems (SoS) context, network centric warfare and operations, and rapid growth in information technology. These factors are driving a more sophisticated systems engineering process with more complex and capable features, along with new tools and procedures. One area of increased focus of the systems engineering process is the informational systems architectural definitions used during system analysis. This process, described in DoD Architectural Framework (DoDAF), emphasizes greater reliance on reusable architectural views describing the system context and concept of operations, interoperability, information and data flows, and network service-oriented characteristics. (O'Brien and Griffin 2007, p. 11)

The last two sections of the systems engineering principles portion of the original case study address case studies themselves, mainly for academic purposes, to help people appreciate systems engineering principles, and the framework used in the case study, namely the rather narrowly defined Friedman-Sage framework that will be discussed briefly in Section II below.

The treatment of the reason for case studies is quite good in that it talks about the benefits of applying systems engineering principles, as highlighted from real-world examples of what works and what does not. Except near the end, where there is allusion to the possibility of new endeavor systems engineering principles, the principles espoused tend to be traditional or conventional.

On the other hand, based upon the original case study (O'Brien and Griffin 2007), if one views the boundary of the GPS system to include primarily the technology associated with the GPS space segment and its controlling ground network, then it can be assumed that system was likely implemented primarily by following traditional or conventional systems engineering processes. If one takes this viewpoint, then all of the above criticism which

attempts to point out some of the shortcomings of conventional systems engineering, may seem vacuous at best, or politically incorrect at worst. It may well be that many would rather not denigrate the original GPS case study by exposing it to the possibilities of a broader system engineering approach.

Unless otherwise indicated, as the present authors have already been doing, unchanged quotations from the existing SEBoK are indented below. Modifications to such quotations are shown in brackets [...]; deletions are not necessarily shown explicitly.

Background

The Global Positioning System (GPS) case study was developed by the United States Air Force Center for Systems Engineering (AF CSE) located at the Air Force Institute of Technology (AFIT). The GPS is a space-based radio-positioning system. A constellation of twenty-four satellites, including three spares, comprise the overall system which provides navigation and timing information to military and civilian users worldwide. GPS satellites, in one of six Earth orbits, circle the globe every twelve hours, emitting continuous navigation signals on two different L-band frequencies. The system consists of two other major segments: a world-wide satellite control network, and the GPS user equipment that can either be carried by a human user or integrated into host platforms such as ships, vehicles, or aircraft.

A user needs to receive signals from at least four GPS satellites simultaneously (satellite orbital positions and terrestrial terrain blockage can be issues that degrade performance) to determine one's position in three dimensions; the altitude determination is typically less accurate than the other two dimensions.

When looking at [GPS], it would be difficult to imagine another system that relies so heavily upon such a wide range of [domains containing systems that must interact effectively to achieve successful GPS operation]. It is evident that [GPS directly relates to many domains and applications including:

- position location and tracking
- time synchronization
- navigation
- transportation
- times of arrival
- air traffic management
- situational awareness
- jam-resistant communications
- business and commerce
- farming
- aerospace
- sensing nuclear detonations from space
- military war-fighting
- targeting
- weapons delivery
- etc.].

[GPS is] an example of [a collaborative (Dahmann, et al. 2008) systems of systems (SoS)]. As such, no one is in charge, and the capabilities (not requirements) flow from the bottom-up, as opposed to top-down.

Purpose

The GPS case study includes a detailed discussion of the development of the GPS and its components, as well as other applicable areas. The reader of this study will gain an increased understanding of the effect that GPS has on military and commercial industries in the context of the systems engineering support required to achieve success.

This may be, but the principal purpose of this revised case study is to suggest a broader view of GPS that discusses signature aspects of SoS, enterprises, and complex systems, and emphasizes SoSE, ESE, and CSE.

[AF CSE] was tasked to develop case studies focusing on the application of [SE] principles within various aerospace programs. The GPS case study [was developed in support of SE] graduate school instruction using the Friedman-Sage framework (Friedman and Sage 2003) (Friedman and Sage 2004).]

However, the Friedman-Sage framework involves only two contractual stakeholders, the Government and the contractor; further, the framework is limited to the traditional or conventional SE life cycle which mainly treats activities in a linear instead of nonlinear fashion; still further, only risks are considered, not a balance of risk and opportunity. Thus, the present authors believe a broader framework embracing SoSE, ESE, and CSE is more appropriate.

Challenges

In the original case study, the first highly technical section (Section 2) was the system description. The original idea derived from trying to determine the precise orbital parameters of the first artificial satellites, such as Sputnik, launched by the Soviets in 1957. Researchers at Johns Hopkins realized the inverse, that if one knew precisely the orbital parameters, the locations of ground stations receiving satellite signals could be determined quite accurately. (O'Brien and Griffin 2007, p. 20.)

GPS got its start in the early 70s (O'Brien and Griffin 2007, p. 19), building upon several previous satellite navigation systems. The primary motive was very accurate position information for the purposes of military applications. For example, the U.S. Air Force wanted to deliver nuclear weapons from bombers with unprecedented accuracy and precision. (O'Brien and Griffin 2007, p. 29)

With such an intense interest from the military, the first real challenge, other than the many technical challenges of making GPS work as well as envisioned, might have been the question of how to make GPS available to the civilian community so they could share the benefits. The study claimed that the system was always offered for civilian use, albeit with some charge. After the Korean airliner went astray and got shot down by a Soviet interceptor aircraft, President Reagan made GPS officially available for civilian use free of charge. (O'Brien and Griffin 2007, p. 14)

The second challenge could be associated with preserving precision capabilities for the military only, and relegating course acquisition (C/A) accuracy to the civilian community. (O'Brien and Griffin 2007, p. 15) Later this dichotomy was essentially eliminated with the realization that a differential GPS configuration involving a fixed ground station with a precisely known location will yield great accuracy. (Kee, et al. 1991)

The GPS satellites used space-borne atomic clocks. To alleviate the need for updating these clocks too often, a successful effort was initiated to revise the international time standard which ended up using relatively infrequent "leap seconds". (O'Brien and Griffin 2007, p. 23) Even these are still annoying for many other applications, such as the continual need to achieve precise synchronization of frequency-hopping radios.

An organizational challenge of inter-service rivalries was overcome with the formation of the Joint Program Office (JPO). (O'Brien and Griffin 2007, p. 25.)

In the early days of satellite communication systems, for example, the satellites were quite small and low powered while the terminals were large and high-powered. By the time GPS came along, the satellites are getting bigger and more sophisticated. Then the challenge to develop relatively low-cost terminals, particularly for mobile users, greatly

increased. (O'Brien and Griffin 2007, p. 29)

A small but interesting challenge was the definition of system of systems (SoS). It was decided that GPS was an SoS because it involved three independent systems, namely, the space vehicle (SV), the control segment (CS), and the user equipment (UE), that "merely" had to interface with each other. (O'Brien and Griffin 2007, p. 30)

Continually changing requirements is usually a problem, although in this case the requirements did not change as often as they could have. (O'Brien and Griffin 2007, p. 31)

Difficulties of defining and updating the many GPS interfaces was largely overcome by the GPS program director, Col. Brad Parkinson, when he convinced his own management, Gen. Schultz at Space and Missile Systems Office (SAMSO) (which eventually became the Space Division) that GPS ought to be defined solely by the signal-structure-in-space and not the physical interfaces. (O'Brien and Griffin 2007, p. 31)

Systems Engineering Practices

Although the systems engineering process in Phase I has been discussed previously, this section will expand on the concepts. For example, one of the user equipment contractors was technically competent, but lacked effective management. The JPO strongly suggested that a systems engineering firm be hired to assist the contractor in managing the program and they agreed. (O'Brien and Griffin 2007, p. 42)

There did not seem to be any mention of what SE firm was hired, if any. The Aerospace Corporation, a non-profit Federally Funded Research and Development Center (FFRDC), which had such a key role in the run-up to GPS was also prominently and centrally involved in development phase of this humungous project. (O'Brien and Griffin 2007, pp. 20, 22, 25, 33, 34, 40, 41, 44, 48, 50-52, 56, 57, 62, 63, 64, 66, 67, 71)

Lessons Learned

Communications was a key ingredient that was fostered throughout GPS development. (O'Brien and Griffin 2007, p. 71)

Yes, from reading the original case study there seems to have been a lot of cooperation among the various organizations, more so than might have been expected in a less compelling case.

Several precepts or foundations of the Global Positioning Satellite program are the reasons for its success. These foundations are instructional for today's programs because they are thought-provoking to those who always seek insight into the program's progress under scrutiny. These foundations of past programs are, of course, not a complete set of necessary and sufficient conditions. For the practitioner, the successful application of different systems engineering processes is required throughout the continuum of a program, from the concept idea to the usage and eventual disposal of the system. Experienced people applying sound systems engineering principles, practices, processes, and tools are necessary every step of the way. Mr. Conley, formerly of the GPS JPO, provided these words: "Systems engineering is hard work. It requires knowledgeable people who have a vision of the program combined with an eye for detail." (O'Brien and Griffin 2007, p. 72)

In very complex systems engineering efforts of this type, it is also important to explore new techniques that attempt to deal with "soft" issues involving people. Those that seem to work can be added to the systems engineering process collection.

Systems engineering played a major role in the success of this program. The challenges of integrating new technologies, identifying system requirements, incorporating a system of systems approach, interfacing with a plethora of government and industry agencies, and dealing with the lack of an operational user early in the program formation required a strong, efficient systems engineering process. The GPS program embedded systems engineering in their knowledge-base, vision, and day-to-day practice to ensure proper identification of system requirements. It also ensured the allocation of those

requirements to the almost-autonomous segment developments and beyond to the subcontractor/vendor level, the assessments of new requirements, innovative test methods to verify design performance to the requirements, a solid concept of operations/mission analysis, a cost-benefit analysis to defend the need for the program, and a strong system integration process to identify and control the “hydra” of interfaces that the program encountered. The program was able to avoid major risks by their acquisition strategy, the use of trade studies, early testing of concept designs, a detailed knowledge of the subject matter, and the vision of the program on both the government and contractor side. (O’Brien and Griffin 2007, p. 72)

This well summarizes the successful systems engineering approach utilized in GPS. Another element of achieving overall balance is the pursuit of opportunities as the “flipside” of risk mitigation.

Finally, here is the list of academic questions offered in the original case study.

QUESTIONS FOR THE STUDENT (O’Brien and Griffin 2007, p. 73) The following questions are meant to challenge the reader and prepare for a case discussion.

- Is this program start typical of an ARPA/ DARPA funded effort? Why or why not?
- Have you experienced similar or wildly different aspects of a Joint Program?
- What were some characteristics that should be modeled from the JPO?
- Think about the staffing for the GPS JPO. How can this be described? Should it be duplicated in today’s programs? Can it?
- Was there anything extraordinary about the support for this program?
- What risks were present throughout the GPS program. How were these handled?
- Requirement management and stability is often cited as a central problem in DoD acquisition. How was this program like, or [un]like, most others?
- Could the commercial aspects of the User Equipment be predicted or planned? Should the COTS aspect be a strategy in other DoD programs, where appropriate? Why or why not?

Other questions might be: What possible influences did the demand for or offering to the public of this GPS capability entail? What differences in the development of GPS might have emerged if the public was more aware of the potential applications for their benefit at the outset?

References

Works Cited

- Dahmann, J. S., G. Rebovich, Jr., and J. A. Lane. November 2008. “Systems engineering for capabilities.” *CrossTalk, The Journal of Defense Software Engineering*. <https://www.stsc.hill.af.mil/crosstalk/2008/11/index.html>. Accessed 12 May 2015).
- Friedman, G.R., and A.P. Sage. 19 January 2003. Systems Engineering Concepts: Illustration Through Case Studies. Accessed September 2011. <https://www.afit.edu/cse/docs/Friedman-Sage%20Framework.pdf>.
- Gorod, A., B. E. White, V. Ireland, S. J. Gandhi, and B. J. Sauser. 2015. *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*. Boca Raton, FL: CRC Press, Taylor & Francis Group. 2015. <https://www.taylorandfrancis.com/books/details/9781466502390/>. Accessed 8 May 2015.
- Friedman, G.R., and A.P. Sage. 2004. “Case Studies of Systems Engineering and Management in Systems Acquisition.” *Systems Engineering* 7(1): 84-96.
- Kee, C., B. W. Parkinson, P. Axelrad. 1991. “Wide area differential GPS.” *Journal of The Institute of Navigation* 38(2): 123-46.
- O’Brien, P. J., and J. M. Griffin. 4 October 2007. *Global Positioning System. Systems Engineering Case Study*. Air Force Center for Systems Engineering (AFIT/SY) Air Force Institute of Technology (AFIT). 2950 Hobson Way, Wright-Patterson AFB OH 45433-7765.

Ricci, N., A. M. Ross, D. H. Rhodes, and M. E. Fitzgerald. 2013. "Considering alternative strategies for value sustainment in systems-of-systems." 6th IEEE International Systems Conference (SysCon). 15-18 April. Orlando, FL.

Primary References

Gorod, A., B. E. White, V. Ireland, S. J. Gandhi, and B. J. Sauser. 2015. *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*. Boca Raton, FL: CRC Press, Taylor & Francis Group. 2015. <https://www.taylorandfrancis.com/books/details/9781466502390/>. Accessed 12 May 2015

Additional References

None

Russian Space Agency Project Management Systems

This article is based around a **Russian space agency Project Management Systems case study** (Rzevski and Skobelev, 2014). The case study focuses on the development of a Real-Time Complex Adaptive Project Management System capable of effectively managing multiple related projects collectively contributing to a defined Enterprise Value.

Background

The essence of every management system should be the same: the allocation of human, physical, financial and intellectual (knowledge) resources to demands (tasks) with the aim of increasing the specified Enterprise Value, where the Enterprise Value is a system of values such as profit, market share, business sustainability, quality of service to customers, quality of working conditions for employees, quality of life in the community, etc. The key difference between the management of a business and management of a project is that businesses are continuously evolving processes whilst projects have specified beginnings and ends.

Standard challenges for project management are:

- Stringent budgets and deadlines
- High competition for limited resource availability such as up-to-date domain knowledge, skills and advanced productivity tools
- Functional organization, which inevitably impedes interdepartmental cooperation
- Bureaucratic management, which is more concerned with lines of command and reporting than with the full use of a project member's initiative and creativity and which negatively affects their motivation
- Rigid project planning, which leads to a rapid divergence between the project plan and reality

Large enterprises commonly operate several projects concurrently. What is best for an individual project is not always best for the enterprise and therefore it is necessary to implement coordination of concurrently run projects with the objective of significantly increasing Enterprise Value.

There are two new key problems, which the 21st century brought to us.

The first is the rapidly increasing complexity of the Internet-based global market, which creates frequent unpredictable disruptive events. This requires real-time adaptive project management, for which there is at present no precedent.

The second is the replacement of capital with knowledge as the key business resource in the economy in which the wealth created by knowledge services is greater than wealth created by producers of goods. There is at present no management system capable of discovering, processing, storing and allocating knowledge to project tasks.

Purpose

The client for this case study was one of the key space technology organizations in Russia, their equivalent of NASA, which operates, at any time, many concurrent mission critical projects.

The purpose of the project management system was to enable the client to effectively manage several related projects (at least ten), collectively contributing to the specified Enterprise Value, with the following requirements:

- Each project may consist of up to 5,000 constituent tasks
- Project members may have different backgrounds and skills and may belong to diverse business cultures
- Project members must have an opportunity to contribute to decision making processes, which affects their domain of work (distributed decision making)
- Both project management and project members must have readily available and up-to-date information on, respectively, project and individual progress, productivity and achievements of goals
- The allocation of resources to tasks must consider 4 types of resources, namely, human, physical, financial and knowledge
- Availability of resources for and constituent tasks of each project may change with short notice and these changes must be rapidly incorporated into the system
- Projects will be subjected to frequent disruptive events such as non-arrival of expected orders, arrival of unexpected orders, sudden and unforeseen emergence of external/internal competitors, cancellations, changes in task specifications, delays, failures, no-shows, etc.
- Rules and regulations governing projects are likely to change rather frequently and any change in rules and regulations must be incorporated immediately and easily into the relevant project management system
- Any discrepancy between project plans and reality in the field must be continuously monitored and rapidly detected and reported
- Projects may cooperate and/or compete for resources in order to increase specified Enterprise Value

A thorough analysis of the client's requirements led to the conclusion that it is necessary to develop a real-time, complex adaptive project management system capable of cooperating and/or competing with other systems, with the overarching goal to continuously increase specified Enterprise Value.

The new system would replace a number of stand-alone, manual or semi-automated project management systems with inadequate monitoring of progress and productivity.

A thorough analysis of contemporary practices showed that such a transformation had never before been achieved. To the best of the team's knowledge, there were no real-time project management systems in existence anywhere in the world.

Challenges

This particular undertaking had a number of challenges.

The most important challenge was the resistance to change by client's managers. The new system with its progress and productivity monitoring capabilities threatened to expose inefficiencies and was not universally welcomed. Two approaches were planned to manage this challenge: the first was education of participants and the second, a proposal for a new payment structure which related salaries to meeting of targets, as reported by the new system.

The scale of the proposed network of systems was an even more important challenge, especially because all projects were mission critical. To manage this challenge, the plan was made to adopt an evolutionary development strategy. The first step was planned to be a fully engineered prototype with a limited functionality, which would be extended

into the first project management system only after a complete acceptance by the client that the prototype was capable of delivering its limited functionality as specified. The network of project management systems would be grown step by step.

The multi-agent technology, which underpinned the system, was well understood by the team, and a methodology for managing complexity (Rzevski and Skobelev, 2014) of the task was in place.

Systems Engineering Practices

Overview

The complexity of client's projects ruled out all conventional project management practices and systems. Instead, for every project, the team designed a complex adaptive project management system, based on multi-agent technology, capable of meeting client requirements.

The system consisted of the following major components (Rzevski and Skobelev, 2014):

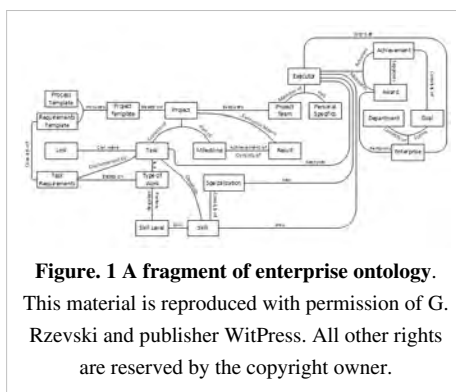
1. Knowledge Base containing domain knowledge relevant to the client's project management processes
2. Multi-agent Virtual World which models the Real World of projects and is capable of managing real-world complexity
3. Interfaces between the Virtual and Real Worlds, which enable the Virtual World to, in-effect, manage the Real World, with or without human intervention

Knowledge Base

Examples of Classes of Objects in the ontology are: Enterprise, Organizational Unit, Project, Task, Project Member (Human Resource), Hardware (Physical) Resource, Document, Software Resource, Knowledge Resource and Process.

Examples of attributes are, for Task: Content, Cost, Duration, Deadline and Preferences; and for Project Member: Organizational Unit, Competences, Profile, Schedule, Current Task, Salary, Achievements and Preferences.

A fragment of enterprise ontology is shown below.



Virtual World

Examples of agents that populate the Virtual World include:

- Task Agent, whose objective is to search for the best resources capable to meet its requirements
- Human Resource Agent, whose objective is to get the best possible task, which will keep the project participant fully occupied, provide opportunities for bonuses and/or enable the participant to learn new skills or get new experience
- Physical Resource Agent, whose objective is to maximize resource utilization
- Project Agent, whose objective is to maximize Project Value
- Enterprise Agent, whose objective is to maximize Enterprise Value

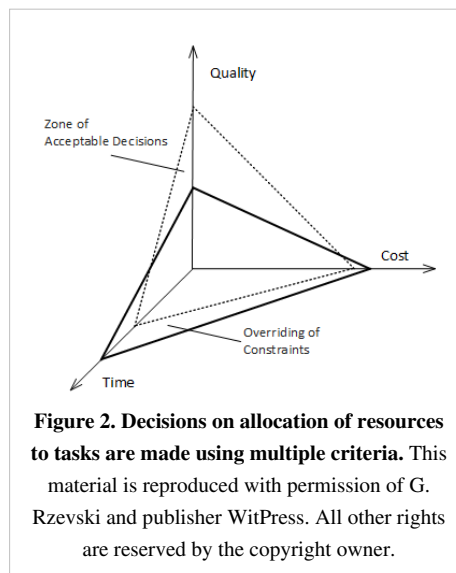
All decisions are made through agent negotiations, as exemplified by the following process:

Task Agents send messages to Human Resource Agents with required competences inviting them to contribute to task fulfilments. Human Resource Agents that are available send their bids. Task Agents offer project participation to those Agents that sent the best bids. Bids are subject to negotiations between affected agents.

A new Task Agent is created whenever a new task is formulated or a previously allocated task is modified. The new Task Agent consults ontology to find out what its objectives are and how to achieve them, and proceeds to send messages to selected Human Resource Agents inviting them to bid for project participation. It is very likely that this invitation will result in re-scheduling, giving an opportunity to Human Resource Agents that were not fully satisfied with their previous allocations to improve their positions. Remuneration, including bonuses, if any, is calculated on the basis of project member participation and achieved results. Enterprise members may participate in several projects.

The allocation of physical, software and knowledge resources is done in an analogous manner. Advanced methods (Rzevski and Skobelev, 2014) have been employed to maximise effectiveness of agent negotiation, such as, virtual microeconomics, agent satisfaction, agent proactivity, enterprise agents, swarm cooperation, etc.

Decisions on allocation of resources to project tasks are made using multiple criteria, for example, decreases in completion time, increases in quality and reducing identified risks, as illustrated below.



Connecting Virtual and Real Worlds

The project member dashboard is the key link between the system and the project member. The exchange of information that could be conducted using the dashboard includes:

- Negotiations of task content, risks, deadlines and budgets
- Acceptance/rejection by the project member of offered project tasks
- Inputs by project members of unexpected disruptive events and comments during the project
- Reports by the system on the project member performance in carrying out accepted tasks

The system may decide to engage two or more available project members in competition with each other to secure an agreement on the acceptable task performance.

The other key link between the virtual and real worlds is the project managers dashboard, which displays detailed project performance data in the form of various diagrams and Gantt charts and allows the manager to examine or modify decisions made autonomously by the system.

Lessons Learned

The first real-time complex adaptive project management system was commissioned and deployed by the client in 2014 achieving the following results:

- 10% to 15% increase in project member productivity;
- 3 to 4 times reduction in manpower required for project scheduling, monitoring and coordination;
- 2 to 3 times reduction of response time to unpredictable disruptive events;
- 15% to 30% increase in the number of projects completion on budget and in time;
- A significant increase in project member motivation;
- A possibility to increase the number of projects operating in parallel without increasing the number of employees.

References

Works Cited

Rzevski, G., Skobelev, P., "Managing Complexity" WIT Press, New Forest, Boston, 2014. ISBN 978-1-84564-936-4.

Primary References

Rzevski, G., Skobelev, P., "Managing Complexity" WIT Press, New Forest, Boston, 2014. ISBN 978-1-84564-936-4.

Additional References

None.

How Lack of Information Sharing Jeopardized the NASA/ESA Cassini/Huygens Mission to Saturn

- Lead Author:
- Brian White

This article describes a deep space mission where more forthright information exchanges between teamed but rival agencies could have both preserved the original plan and saved much time and money. The topic may be of particular interest to those involved in institutional collaborations where there are vested interests in protecting rather than sharing information.

For additional information, refer to the closely related topics of Information Management, Organizing Business and Enterprises to Perform Systems Engineering and Fundamentals of Services.

Background

Before the “Faster, Better, Cheaper” philosophy introduced in the 1990s, the United States National Aeronautics and Space Administration (NASA) focused on three classes of unmanned space missions. In order of increasing cost, these were the Discovery, New Frontiers, and Flagship programs. Flagship programs typically cost more than \$1B, and included the Voyager (outer planets), Galileo (Jupiter), Cassini-Huygens (Saturn), Mars Science Laboratory (Mars), and the James Webb Space Telescope. (Wall 2012)

The concept of the Cassini-Huygens mission was initiated in 1982 as the result of a working group formed by the National Academy of Sciences and the European Science Foundation. This group sought opportunities for joint space missions; several subsequent reports endorsed the working group’s concept of a Saturn orbiter coupled with a Titan (Saturn’s largest moon) lander. (Russell 2003, p. 61.)

By 1988, NASA was politically motivated to reverse earlier tensions with the European Space Agency (ESA) by engaging in a joint mission. Cassini-Huygens was seen as a mechanism to achieve this goal, and the cooperation between NASA and ESA helped the program survive potential budget cuts (since the U.S. was obligated to match ESA commitments). (Russell 2003, p. 62.)

NASA and ESA approved the Cassini-Huygens program, and it proceeded under a traditional management approach. NASA built the Cassini orbiter (the largest and most complex unmanned space probe ever built) and the ESA constructed the Huygens lander. This partition of responsibility almost led to the failure of the Titan survey portion of the mission. Cassini (which would conduct a variety of scientific surveys of the Saturn planetary system) was expected to relay transmissions from Huygens to NASA’s Deep Space Network (DSN); however, the interface between the lander and orbiter was not well-managed and erroneous assumptions about how the orbiter/lander system would behave after separation nearly doomed the Titan exploration portion of the mission. (Oberg 2004.)

Purpose

The intent of the Titan survey portion of the Cassini-Huygens mission was that the Huygens lander would separate from the Cassini orbiter and commence a one-way, 2.5 hour descent into Titan's atmosphere. Its modest transmitter would send data back to the orbiter, which would relay the information to Earth. (Oberg 2004, p. 30.) This effectively made the radio link between the two spacecraft a poorly-characterized single point of failure (SPOF).

Alenia Spazio SpA, the Italian communications vendor that built the radio system, overlooked the Doppler shift (approximately 38 kHz) (Oberg, 2004, p. 31) that would occur when Huygens separated from Cassini and began its descent (Oberg 2004, p. 38). The communications protocol was binary phase-key shifting: "[the] transmission system represents 1s and 0s by varying the phase of the outgoing carrier wave. Recovering these bits requires precise timing: in simple terms, Cassini's receiver is designed to break the incoming signal into 8192 chunks every second. It determines the phase of each chunk compared with an unmodulated wave and outputs a 0 or a 1 accordingly". (Oberg 2004, p. 31.) The receiver was appropriately configured to compensate for the Doppler shift of the carrier wave but would be unable to adjust for the Doppler shift of the encoded data. "In effect, the shift would push the signal out of synch with the timing scheme used to recover data from the phase-modulated carrier." (Oberg 2004, p. 33) Therefore, the communications system would be unable to decode the data from the lander and would then relay scrambled information to NASA. Because of the failure mechanism involved, the data would be completely unrecoverable.

Both Cassini and Huygens had been tested before launch; however, none of the testing accurately reflected the Doppler shift that would be experienced at this critical phase of the mission. An opportunity to conduct a full-scale, high-fidelity radio test was ignored due to budget constraints; the testing would have required disassembly and subsequent recertification of the probes. (Oberg, 2004, p. 30.) Correcting this latent issue via a minor firmware upgrade would have been trivial before the spacecraft were launched (Oberg 2004, p. 33) once they were on the way to Saturn any corrective action would be severely limited and expensive.

Once the mission was underway, the probe coasted along its seven-year trajectory to Saturn and its moons. Claudio Sollazzo, the ESA ground operations manager, was uncomfortable with the untested communications system. He tasked Boris Smeds, an engineer with radio and telemetry experience, with finding a way to test the communications system using an Earth-generated signal. (Oberg 2004, p. 30.)

Smeds spent six months developing the test protocols that would use Jet Propulsion Laboratory (JPL) ground stations and an exact duplicate of Huygens. Simulated telemetry would be broadcast from Earth to Cassini and relayed back; the test signal would vary in power level and Doppler shift to fully exercise the communications link and accurately reflect the anticipated parameters during Huygens's descent into Titan's atmosphere. (ESA 2005)

Challenges

Smeds faced opposition to his test plans from those who felt it was unnecessary, but ultimately prevailed due to support from Sollazzo and Jean-Pierre Lebreton, the Huygens project scientist. More than two years after the mission was launched, Smeds traveled to a DSN site in California to conduct the test. (Oberg 2004, p. 31)

A test signal was broadcast, received by Cassini, re-transmitted to the DSN site, and relayed to ESA's European Space Operation Centre (ESOC) in Darmstadt, Germany for analysis. Testing had to be conducted when the orbiter was in the correct relative position in the sky; it was more than a quarter of a million miles away with a signal round-trip time of nearly an hour. The test immediately exposed an issue; the data stream was intermittently corrupted, with failures not correlated to the power level of the test signal. The first of two days of testing concluded with no clear root cause identified. (Oberg 2004, p. 31.)

Even though the probe was far from its ultimate destination, many science teams were competing for time to communicate with it using the limited bandwidth available. The communications team would not be able to conduct another set of trials for several months. Smeds diagnosed the root cause of the problem; he felt it was the Doppler

shifts induced in the simulated signal. However, the test plan did not include unshifted telemetry (an ironic oversight). He modified his test plan overnight and shortened the planned tests by 60%; this recovered sufficient time for him to inject an unshifted signal into the test protocols. (Oberg 2004, p. 32)

This unshifted signal did not suffer from the same degradation; however, other engineers resisted the diagnosis of the problem. Follow-up testing using probe mockups and other equipment ultimately convinced the ESA of the issue; this took an additional seven months. (Oberg 2004, p. 33.)

By late 2000, ESA informed NASA of the latent failure of the communications link between Cassini and Huygens. Inquiry boards confirmed that Alenia Spazio had reused timing features of a communications system used on Earth-orbiting satellites (which did not have to compensate for Doppler shifts of this magnitude). (Oberg, 2004, p. 33.) In addition, because NASA was considered a competitor, full specifications for the communications modules were not shared with JPL. The implementation of the communications protocols was in the system's firmware; trivial to correct before launch, impossible to correct after. (ESA 2005.)

A 40-man Huygens Recovery Task Force (HRTF) was created in early 2001 to investigate potential mitigation actions. Analysis showed that no amount of modification to the signal would prevent degradation; the team ultimately proposed changing the trajectory of Cassini to reduce the Doppler shift. (ESA 2005) Multiple studies were conducted to verify the efficacy of this remedy, and it ultimately allowed the mission to successfully complete the Titan survey.

Systems Engineering Practices

Space missions are particularly challenging; once the spacecraft is en route to its destination, it is completely isolated. No additional resources can be provided and repair (particularly for unmanned missions) can be impossible. Apollo 13's crew managed to barely survive the notable mishap on its mission because of the resources of the docked Lunar Excursion Module (LEM) and the resourcefulness of the ground control team's experts. A less well-known failure occurred during the Galileo mission to Jupiter. After the Challenger disaster, NASA adopted safety standards that restricted the size of boosters carried in the Space Shuttle. (Renzetti 1995.) Galileo was delayed while the Shuttles were grounded and Galileo's trajectory was re-planned to include a Venus fly-by to accelerate and compensate for a smaller booster. Galileo's main antenna failed to deploy; lubricant had evaporated during the extended unplanned storage (Evans 2003) and limited computer space led to the deletion of the antenna motor-reversing software to make room for thermal protection routines. When the antenna partially deployed, it was stuck in place with no way to re-furl and redeploy it. Engineers ultimately used an onboard tape recorder, revised transmission protocols, the available low-gain antenna, and ground-based upgrades to the DSN to save the mission. (Taylor, Cheung, and Seo 2002.)

The Titan survey was ultimately successful because simulation techniques were able to verify the planned trajectory modifications and sufficient reaction mass was available to complete the necessary maneuvers. In addition, Smeds's analysis gave the mission team the time it needed to fully diagnose the problem and develop and implement the remedy. If this test were conducted the day before the survey, it would merely have given NASA and ESA advance warning of a disaster. The time provided enabled the mission planners to craft a trajectory that resolved the communication issue and then blended back into the original mission profile to preserve the balance of the Saturn fly-bys planned for Cassini. (Oberg 2004, p. 33.)

Lessons Learned

The near-failure of the Cassini-Huygens survey of Titan was averted because a handful of dedicated systems engineers fought for and conducted relevant testing, exposed a latent defect, and did so early enough in the mission to allow for a recovery plan to be developed and executed. Root causes of the issue included politically-driven partitioning, poor interface management, overlooked contextual information, and a lack of appreciation for single-points-of-failure (SPOFs).

The desire to use a joint space mission as a mechanism for bringing NASA and ESA closer together (with the associated positive impact in foreign relations) introduced an unnecessary interface into the system. Interfaces must always be managed carefully; interfaces between organizations (particularly those that cross organizational or political borders) require extra effort and attention. Boeing and Airbus experienced similar issues during the development of the Boeing 787 and A380; international interfaces in the design activities and supply chains led to issues:

...every interface in nature has a surface energy. Creating a new surface (e.g., by cutting a block of steel into two pieces) consumes energy that is then bound up in that surface (or interface). Interfaces in human systems (or organizations), a critical aspect of complex systems such as these, also have costs in the effort to create and maintain them. Second, friction reduces performance. Carl von Clausewitz, the noted military strategist, defined friction as the disparity between the ideal performance of units, organizations, or systems, and their actual performance in real-world scenarios. One of the primary causes of friction is ambiguous or unclear information. Partitioning any system introduces friction at the interface. (Vinarcik 2014, p. 697)

Alenia Spazio SpA's unclear understanding of the Doppler shift introduced by the planned relative trajectories of Huygens and Cassini during the Titan survey led it to reuse a component from Earth-orbiting satellites. Because it considered NASA a competitor and cloaked details of the communications system behind a veil of propriety, it prevented detection of this flaw in the design phase. (Oberg 2004, p. 33)

Because NASA and ESA did not identify this communication link as a critical SPOF, they both sacrificed pre-launch testing on the altar of expediency and cost-savings. This prevented detection and correction of the flaw before the mission was dispatched to Saturn. The resource cost of the later analysis and remedial action was non-trivial and if sufficient time and reaction mass had not been available the mission would have been compromised. It should be noted that a number of recent spacecraft failures are directly attributable to SPOFs (notably, the Mars Polar Lander (JPL 2000) and the Genesis sample return mission (GENESIS, 2005)). Effective SPOF detection and remediation must be a priority for any product development effort. More generally, early in the development process, significant emphasis should be placed on analyses focused on what might go wrong ("rainy day scenarios") in addition to what is expected to go right ("sunny day scenarios").

The success of the Huygens survey of Titan was built upon the foundation established by Boris Smeds by identifying the root cause of the design flaws in a critical communications link. This case study underscores the need for clear contextual understanding, robust interface management, representative testing, and proper characterization and management of SPOFs.

References

Works Cited

- Evans, B. 2003. "The Galileo Trials." *Spaceflight Now*. Available: <https://www.spaceflightnow.com/galileo/030921galileohistory.html>.
- GENESIS. 2005. "GENESIS Mishap Investigation Board Report Volume I." Washington, DC, USA: National Aeronautics and Space Administration (NASA).
- JPL. 2000. "Report on the Loss of the Mars Polar Lander and Deep Space 2 Missions." Special Review Board. Pasadena, CA, USA: NASA Jet Propulsion Laboratory (JPL).
- ESA. 2005. "Modest Hero Sparks Team Response." European Space Agency. Available: https://www.esa.int/Our_Activities/Operations/Modest_hero_sparks_team_response.
- Oberg, J. 2004. "Titan Calling: How a Swedish engineer saved a once-in-a-lifetime mission to Saturn's mysterious moon." *IEEE Spectrum*. 1 October 2004, pp. 28-33.
- Renzetti, D.N. 1995. "Advanced Systems Program and the Galileo Mission to Jupiter." *The Evolution of Technology in the Deep Space Network: A History of the Advanced Systems Program*. Available: https://deepspace.jpl.nasa.gov/technology/95_20/gll_case_study.html.
- Russell, C. 2003. *The Cassini-Huygens Mission: Volume 1: Overview, Objectives and Huygens Instrumentarium*. Norwell, MA, USA: Kluwer Academic Publishers.
- Taylor, J., K.-M. Cheung, and D. Seo. 2002. "Galileo Telecommunications. Article 5." *DESCANSO Design and Performance Summary Series*. Pasadena, CA, USA: NASA/Jet Propulsion Laboratory.
- Vinarcik, M.J. 2014. "Airbus A380 and Boeing 787 — Contrast of Competing Architectures for Air Transportation," in *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*, edited by A. Gorod et al. Boca Raton, FL, USA: CRC Press. p. 687-701.
- Wall, M. 2012. "NASA Shelves Ambitious Flagship Missions to Other Planets." *Space News*. Available: <https://www.space.com/14576-nasa-planetary-science-flagship-missions.html>.

Primary References

None.

Additional References

None.

Hubble Space Telescope

- Lead Authors:
- Heidi Davidz, Alice Squires, and Richard Freeman
- Contributing Author:
- Brian White

This article describes a remarkable engineering feat with vast scientific benefits and implications. The topic may be of particular interest to those facing formidable systems engineering challenges where one might thrive on a thoughtful blend of humility and optimism. For additional information, refer to the links provided in Section V: Lessons Learned below.

Background

The Hubble Space Telescope (HST) Case Study was developed by the United States Air Force Center for Systems Engineering (AF CSE) located at the Air Force Institute of Technology (AFIT). The AF CSE was tasked to develop case studies focusing on the application of systems engineering principles within various aerospace programs. The HST study (Mattice 2005) is one of four initial case studies selected by AFIT for development in support of systems engineering graduate school instruction. The cases are structured using the Friedman-Sage framework (Friedman and Sage 2003; Friedman and Sage 2004, 84-96), which decomposes a case into contractor, government, and shared responsibilities in the following nine concept areas: STOPPED HERE

1. Requirements Definition and Management
2. Systems Architecture Development
3. System/Subsystem Design
4. Verification/Validation
5. Risk Management
6. Systems Integration and Interfaces
7. Life Cycle Support
8. Deployment and Post Deployment
9. System and Program Management

The case study provides a useful example of the rising cost of defect correction through successive life cycle phases, demonstrating how an error (in test fixture specification) that could have been fixed for \$1,000 at the design stage, or detected and fixed with a \$10 million investment in an end-to-end test of the telescope on the ground, ended up costing \$1 billion to fix when the system was in service.

Purpose

The Hubble Space Telescope (HST) is an orbiting astronomical observatory operating in the spectrum from the near-infrared into the ultraviolet. Launched in 1990 and still operational, HST carries and has carried a wide variety of instruments producing imaging, spectrographic, astrometric, and photometric data through both pointed and parallel observing programs. Over 100,000 observations of more than 20,000 targets have been produced for retrieval. The telescope is well known as a marvel of science. This case study hopes to represent the facet of the HST that is a marvel of systems engineering, which, in fact, generated the scientific research and observation capabilities now appreciated worldwide.

Viewed with the clarity that only time and hindsight provide, the HST program certainly represents one of the most successful modern human endeavors on any scale of international scope and complexity. As a systems engineering project the HST had to respond to requirements from the diverse international scientific community at a time when NASA was implementing a different research-development-acquisition philosophy and process than what was

predominately being used in most major government acquisition programs. As with most other large programs, powerful influences outside the systems engineering process itself became issues that HST systems engineers in effect had to acknowledge as integral to their overall system/program/engineering management responsibility.

Challenges

The story of how this remarkable capability came to be is a story of the complicated interactions of a systems engineering process, which we like to believe we understand, with equally demanding political, budgetary, and institutional processes we often fail to understand or comprehend at the time they occur. In the final analysis, these processes are inseparable and integral to attaining program success. The challenge to modern systems engineers is to fully embrace the discipline of the systems engineering process while at the same time learning how to continue to practice it in spite of inevitable external influences and instabilities that often cannot be anticipated.

Major differences revolved around the nature and needs of a very different HST “customer” or user from most DoD systems. The HST had to respond to requirements from the diverse international scientific community instead of from DoD’s combatant commands. In addition, at the time, NASA implemented a different research-development-acquisition philosophy and process than the DoD Acquisition Management Framework described in the DoD 5000 series acquisition reforms. As with most other large programs, powerful influences outside the systems engineering process itself became issues that HST systems engineers in effect had to acknowledge as integral to their overall system/program/engineering management responsibility.

Systems Engineering Practices

During the critical systems engineering phase for the HST program (1970s concept studies thru 1990 launch) there appears to have been no NASA systems engineering master process. Rather, field center processes were operative and possibly even in competition, as centers (especially Marshall and Goddard for HST) were in keen competition for lead management roles and responsibilities. We will see the systems engineering and program management impacts of this competition as it played out for HST, with the science mission objectives and instrumentation payloads being the motivation for Goddard vs. the vehicle/payload access to space motivation of Marshall. In the final analysis, the roles of the major contractors in engineering the system with uneven NASA participation over the system life cycle had a telling effect.

Lessons Learned

Five learning principles (LPs) were derived that address the more broadly applicable areas of systems engineering knowledge. These five LPs inform the areas of the SEBoK that are most strongly related to the case study. The five areas are:

- stakeholder requirements definition (LP1);
- planning (pre-program trade studies) (LP2);
- system integration (LP3);
- life cycle model management (LP4); and
- risk management (LP5).

A synopsis of the HST Learning Principles (LPs) are as follows:

Stakeholder Requirements Definition LP1: Early and full participation by the customer/user throughout the program is essential to success. In the early stages of the HST program, the mechanism for involving the customer was not well defined. The user community was initially polarized and not effectively engaged in program definition and advocacy. This eventually changed for the better, albeit driven heavily by external political and related national program initiatives. Ultimately, institutionalization of the user’s process for involvement ensured powerful representation and a fundamental stake and role in both establishing and managing program requirements. Over time,

the effectiveness of “The Institute” led to equally effective user involvement in the deployment and on-orbit operations of the system as well.

Planning LP 2: The use of Pre-Program Trade Studies (e.g. “Phased Studies” or “Phased Project Planning”) to broadly explore technical concepts and alternatives is essential and provides for a healthy variety of inputs from a variety of contractors and government (NASA) centers. These activities cover a range of feasibility, conceptual, alternative and preliminary design trades, with cost initially a minor (later a major) factor. In the case of HST, several NASA Headquarters and Center organizations funded these studies and sponsored technical workshops for HST concepts. This approach can promote healthy or unhealthy competition, especially when roles and responsibilities within and between the participating management centers have not yet been decided and competing external organizations use these studies to further both technical and political agendas. NASA Center roles and missions can also be at stake depending on political and or budgetary realities. The systems engineering challenge at this stage is to “keep it technical, stupid!”

Systems Integration LP 3: A high degree of systems integration to assemble, test, deploy, and operate the system is essential to success and must be identified as a fundamental program resource need as part of the program baseline. For HST, the early wedding of the program to the Shuttle, prior NASA and NASA contractor experience with similarly complex programs, such as Apollo, and the early requirement for manned, on-orbit servicing made it hard not to recognize this was a big systems engineering integration challenge. Nonetheless, collaboration between government engineers, contractor engineers, as well as customers, must be well defined and exercised early on to overcome inevitable integration challenges and unforeseen events.

Life Cycle Models LP 4: Life Cycle Support planning and execution must be integral from day one, including concept and design phases. The results will speak for themselves. Programs structured with real life cycle performance as a design driver will be capable of performing in-service better, and will be capable of dealing with unforeseen events (even usage in unanticipated missions). HST probably represents a benchmark for building in system sustainment (reliability, maintainability, provision for technology upgrade, built-in redundancy, etc.), while providing for human execution of functions (planned and unplanned) critical to servicing missions. With four successful service missions complete, including one initially not planned (the primary mirror repair), the benefits of design-for-sustainment, or life cycle support, throughout all phases of the program become quite evident. Without this design approach, it is unlikely that the unanticipated, unplanned mirror repair could even have been attempted, let alone been totally successful.

Risk Management LP 5: For complex programs, the number of stakeholders (government and contractor) demands that the program be structured to cope with high risk factors in many management and technical areas simultaneously. The HST program relied heavily on the contractors (especially Lockheed Missiles and Space Company (LMSC) and Perkin-Elmer (P-E)), each of which “owned” very significant and unique program risk areas. In the critical area of optical systems, NASA depended on LMSC as the overall integrator to manage risk in an area where P-E was clearly the technical expert. Accordingly, NASA relied on LMSC and LMSC relied on P-E with insufficient checks, oversight, and independence of the quality assurance function throughout. While most other risk areas were no doubt managed effectively, lapses here led directly to the HST’s going to orbit with the primary mirror defect undetected, in spite of substantial evidence that could have been used to prevent this.

References

Works Cited

Friedman, G.R. and A.P. Sage. 2003. *Systems Engineering Concepts: Illustration Through Case Studies*. Accessed September 2011. Available at: <https://www.afit.edu/cse/docs/Friedman-Sage%20Framework.pdf>.

Friedman, G. and A. Sage. 2004. "Case Studies of Systems Engineering and Management in Systems Acquisition." *Systems Engineering*, 7(1): 84-96.

Mattice, J. "Hubble Space Telescope Case Study." Ft. Belvoir, VA: Defense Acquisition University (DAU). Last modified May 2, 2005. Accessed November 27, 2012. Available at <https://acc.dau.mil/adl/en-US/37600/file/9105/Hubble%20Space%20Telescope%20SE%20Case%20Study%20-%20JJ%20Mattice.pdf>.

Primary References

None.

Additional References

None.

Applying a Model-Based Approach to Support Requirements Analysis on the Thirty-Meter Telescope

This article describes how a model-based systems engineering (MBSE) approach was used to support requirements analysis, system design, and early verification for critical subsystems of the Thirty Meter Telescope (TMT) [8]. The MBSE approach applied the Executable Systems Engineering Method (ESEM) [4] [5] and the Open-source Engineering Environment (OpenMBEE) [7] to specify, analyze, and verify requirements of TMT's Alignment and Phasing System (APS) and the Narrow Field Infrared Adaptive Optics System (NFIRAOS). The value proposition for applying this MBSE approach was to establish precise requirements and fine-grained traceability to system designs, and to verify key requirements using executable SysML [6] models beginning early in development.

Background

The TMT (Figure 1) is a next-generation ground-based extremely large telescope designed to answer key science questions regarding the nature and composition of the Universe. At its core is a wide-field, altitude-azimuth Ritchey-Chrétien design with a 492 segment, 30-meter diameter primary mirror (M1), a fully active secondary mirror, and an articulated tertiary mirror. Each segment's optical performance is sensitive to three rigid body degrees of freedom: piston, tip, and tilt. To obtain optimal image quality, the segmented M1 must perform like a single monolithic mirror, achieved through a multitude of controls working to co-align, co-focus, and co-phase its segments. The APS (Figure 2) is responsible for the overall pre-adaptive optics wavefront quality, using starlight to measure wavefront errors and align the TMT optics. Adaptive optics systems like the NFIRAOS (Figure 2) are designed to sense real-time atmospheric turbulence and correct the telescope's optical beam to remove its effect, enabling diffraction-limited imaging on the ground. In LGS MCAO and NGS AO modes, this is achieved through the use of wavefront sensors to detect laser and natural guide stars and deformable mirrors to direct the corrected wavefront to science instruments. These opto-mechanical designs and complex controls are constrained by several requirements that must be satisfied.



Figure 1. Thirty Meter Telescope. (Used with permission. Permission granted by Jamie Nakawatase.)

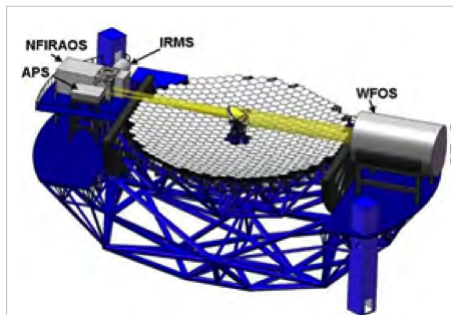


Figure 2. APS and NFIRAOS early light locations. (Used with permission. Permission granted by Jamie Nakawatase.)

TMT International Observatory (TIO), LLC is a non-profit organization of international members, responsible for managing the design, development, and operations of the TMT. The Jet Propulsion Laboratory (JPL) [9] participates in the design and development of several TMT subsystems and delivers an operational APS, where TIO is responsible for providing requirements to JPL. The APS team applies an MBSE approach to analyze requirements, derive an architecture design, and implement a system. TIO also works with JPL to analyze the operational behavior of NFIRAOS through modeling system-level operational scenarios (such as slew, acquisition, and dithering) with Monte-Carlo simulations. Modeling patterns are used to capture functional and physical system characteristics, behavior, requirements, parametric relationships, and use case scenarios. MBSE applications are motivated by optimization to better understand TMT's complex system behaviors.

Purpose

This article describes how MBSE is applied to the development of critical subsystems of a complex interdisciplinary system and the benefits of this approach. While document-based artifacts are necessary throughout the development lifecycle, complex systems engineering relies significantly on the use of models to address concerns from various domains (e.g. mechanics, optics, controls) (Figure 3). MBSE helps to manage implicit dependencies on information contained in these cross-domain documents, understand change impacts, analyze designs, and communicate evolving technical baselines. Models act as the single source of authority for systems engineering information that enables optimization through consistent and automated data exchange, enhanced analyses, and consolidating subsystem design information into separate artifacts needed by various stakeholders.

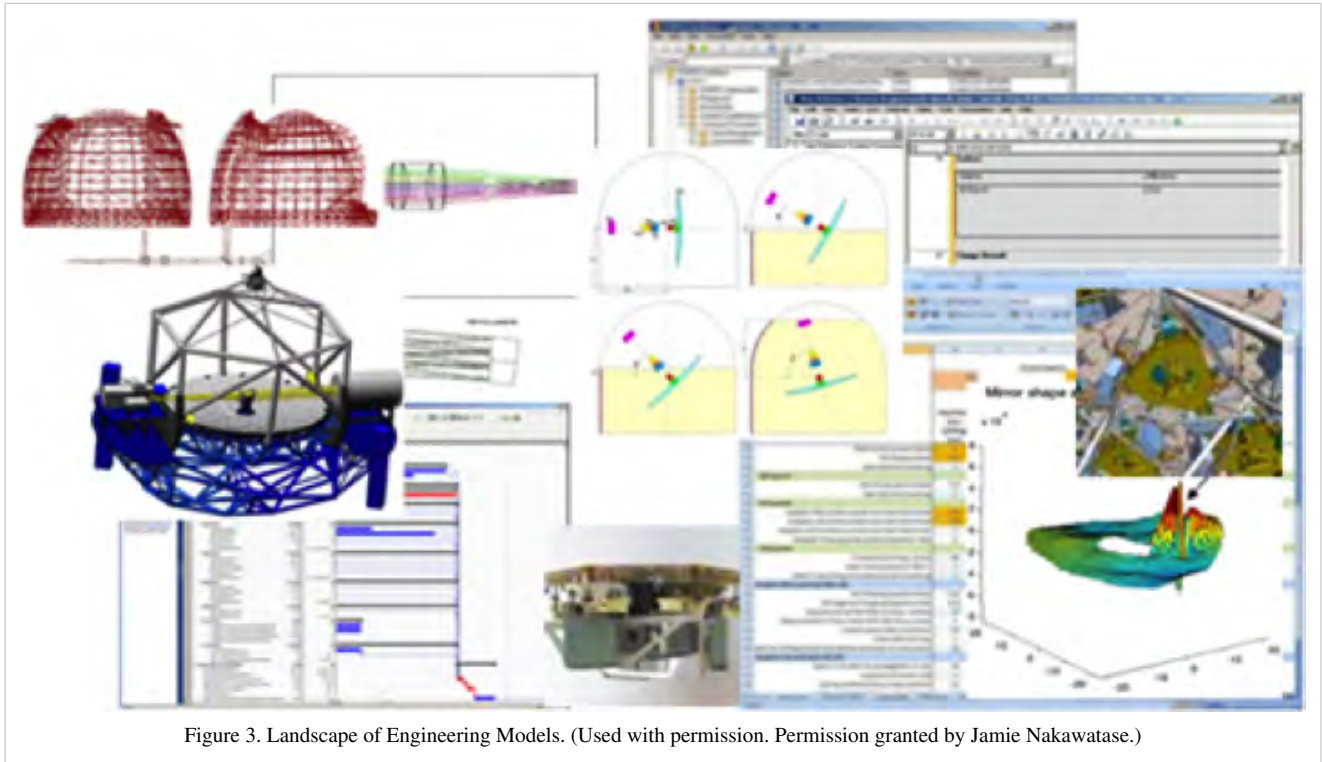


Figure 3. Landscape of Engineering Models. (Used with permission. Permission granted by Jamie Nakawatase.)

MBSE Challenges

Systems engineering (SE) is inherently challenging with concepts spanning several engineering disciplines. Expressing these concepts in a model demands the use of flexible methodology, language, and tools. The modeler must understand how to leverage this flexibility to rigorously specify systems with a broad range of complex design considerations. This poses the initial MBSE challenge—the **learning curve**. However, this challenge is native to any new undertaking and is overcome by hands-on experience, training, and ever-growing resources.

Another challenge is determining how to **apply** MBSE to meet the needs of a project. MBSE is not separate from SE—it is an approach to achieve the fundamental goals of SE more efficiently. MBSE should not be used to duplicate work, but instead replace efforts that can be better accomplished formally through modeling.

Engineers live in a landscape of variability among tools and information models in different domains (e.g. ALM, PLM, CAD), which are often implicitly connected [4]. To benefit from a model-based paradigm, the models require explicit connections. A key challenge is how to leverage data and associated models to enable **cross-domain integration**.

Finally, **standardization** is a challenge for MBSE. A model is only effective if stakeholders can understand it. SysML is an evolving modeling language that is becoming the dominant standard to communicate system architectures, independent of the modeling tools that use it. However, SysML is only one of many standards that must be applied to ensure the models can be consistently interpreted by tools and users.

MBSE Approach

The MBSE approach follows the conventional SE V-process enriched by a model-based paradigm (Figure 4). The scope of this MBSE application addresses models at L2 and L3, and requirements flow-down to L4, although the approach can be applied to support specification and architecture design at other levels as well. Associated modeling artifacts are created early and maintained throughout the development lifecycle.

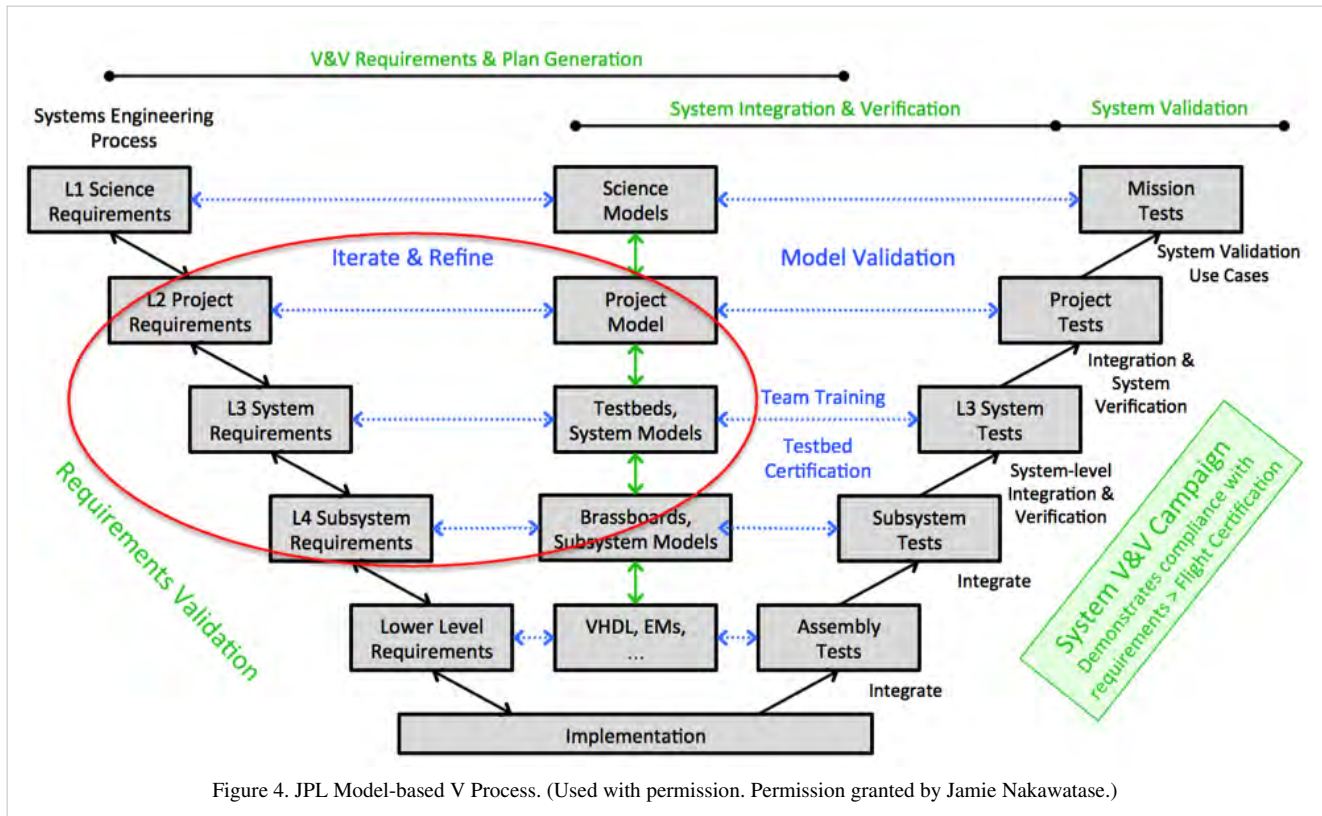


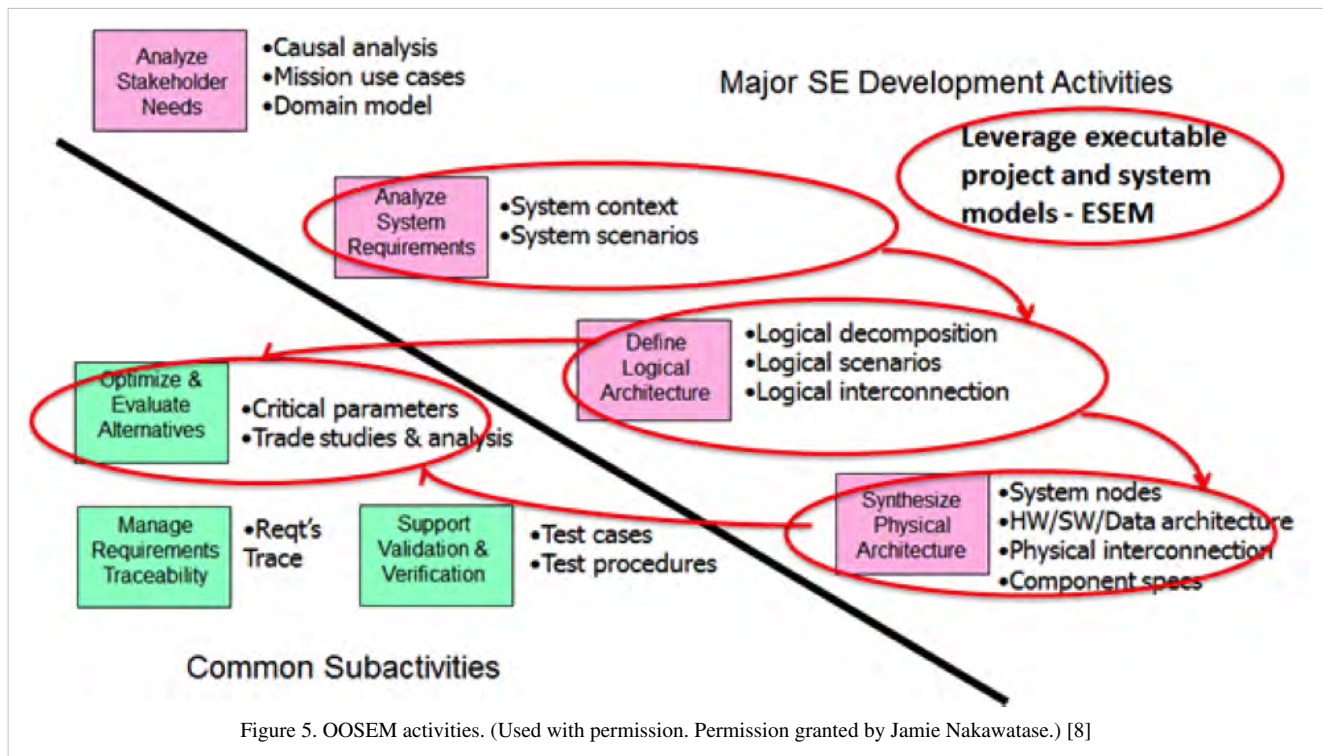
Figure 4. JPL Model-based V Process. (Used with permission. Permission granted by Jamie Nakawatase.)

MBSE articulates the system architecture in a formal, executable model that captures structure, behavior, requirements, and parametric relationships of system elements. Operational scenarios are defined in the model and analyzed through system-level simulation to verify requirements and validate overall system design over an expected range of conditions and parameters. The system model is also the authoritative source for several engineering documents. The MBSE approach is subsequently described by its methodology, tooling infrastructure, and analysis processes.

Methodology

The Executable Systems Engineering Method (ESEM) [1] is used to formalize requirements, specify system designs, characterize components, and specify/run analyses. ESEM augments the Object Oriented Systems Engineering Method (OOSEM) [2] by enabling executable models that enhance understanding, precision, and verification of requirements through applying analysis patterns specified with various SysML diagrams. ESEM also enables integration of supplier/customer models.

Figure 5 shows the major activities common to SE processes using OOSEM. The red circles indicate where ESEM injects formal modeling methods.



ESEM is utilized to model different levels of abstraction that are analyzed using several modeling patterns as detailed in [4]. The system-of-interest is modeled as a black box that interacts with external subsystems, such as controls. Interactions are modeled using ports to identify operations and flows at the system-of-interest interface.

The conceptual model specifies technology-independent system components and captures their behavior. This part of the model is used to analyze characteristics such as duration of operational scenarios. Component behavior is captured using state machines and activity diagrams, and constraint parameters are captured in a table. Communication across internal and external system components is accomplished through the sending and receiving of signals through ports. This model supports production of interface control documents by querying information sent from one component to another over ports.

The conceptual model serves as a basis to specify the realization model of the physical components. The realization model imposes technology-dependent constraints on the design solutions. Both the conceptual (i.e. logical) and realization (i.e. physical) models represent the “as-specified” system.

Tooling

The MBSE approach uses standard SysML language and modeling tools to minimize custom software. The OpenMBEE community promotes an open tooling environment that provides a platform for modeling. It utilizes the Model Management System (MMS) that can be accessed from rich SysML desktop clients like MagicDraw, lightweight web-based clients like View Editor, computational programs like Mathematica, and other tools that utilize RESTful web services. The MMS also provides the basic infrastructure for search, relation management, versioning, workflow, access control, content flexibility, web applications support, web-based API access, and multi-tool/repository integration across engineering and management disciplines.

Figure 6 shows the integration of model artifacts produced by MMS, View Editor, and MagicDraw. System models are constructed, queried, and rendered following the view and viewpoint paradigm [3] from MMS.

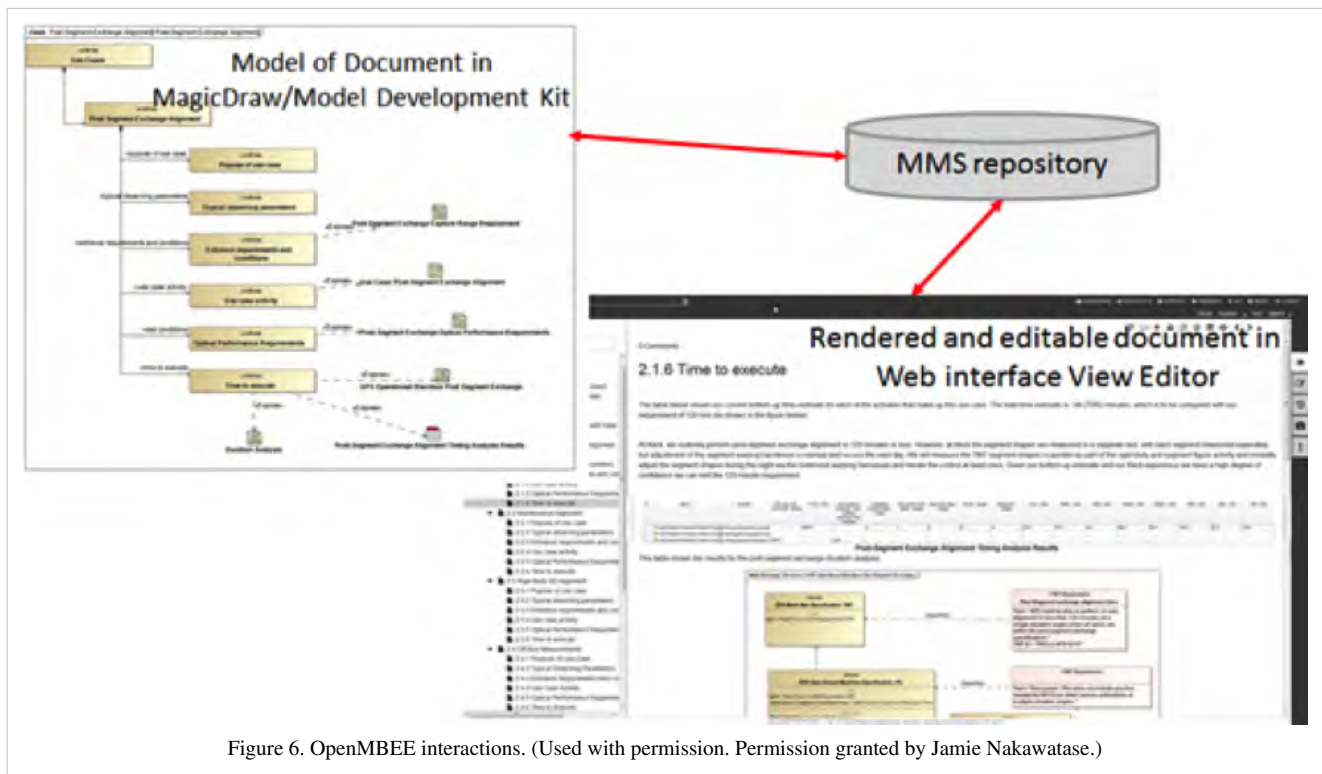


Figure 6. OpenMBEE interactions. (Used with permission. Permission granted by Jamie Nakawatase.)

Analysis

One key SE process is to analyze the impact of changing requirements on the system design. Figure 7 illustrates how the MBSE approach is used to support requirements impact analysis through the following steps:

- Step 1: A changed requirement triggers impact analysis.
- Step 2: MMS integrates DOORS (which manages text requirements) and the SysML model, enabling a DOORS requirement change to propagate to the SysML model.
- Step 3: A property-based requirement is formalized in SysML, enabling requirements specification that can be evaluated by engineering analysis.
- Steps 4-6: The conceptual and/or realization design is automatically verified against the changed requirement, resulting in pass or fail.

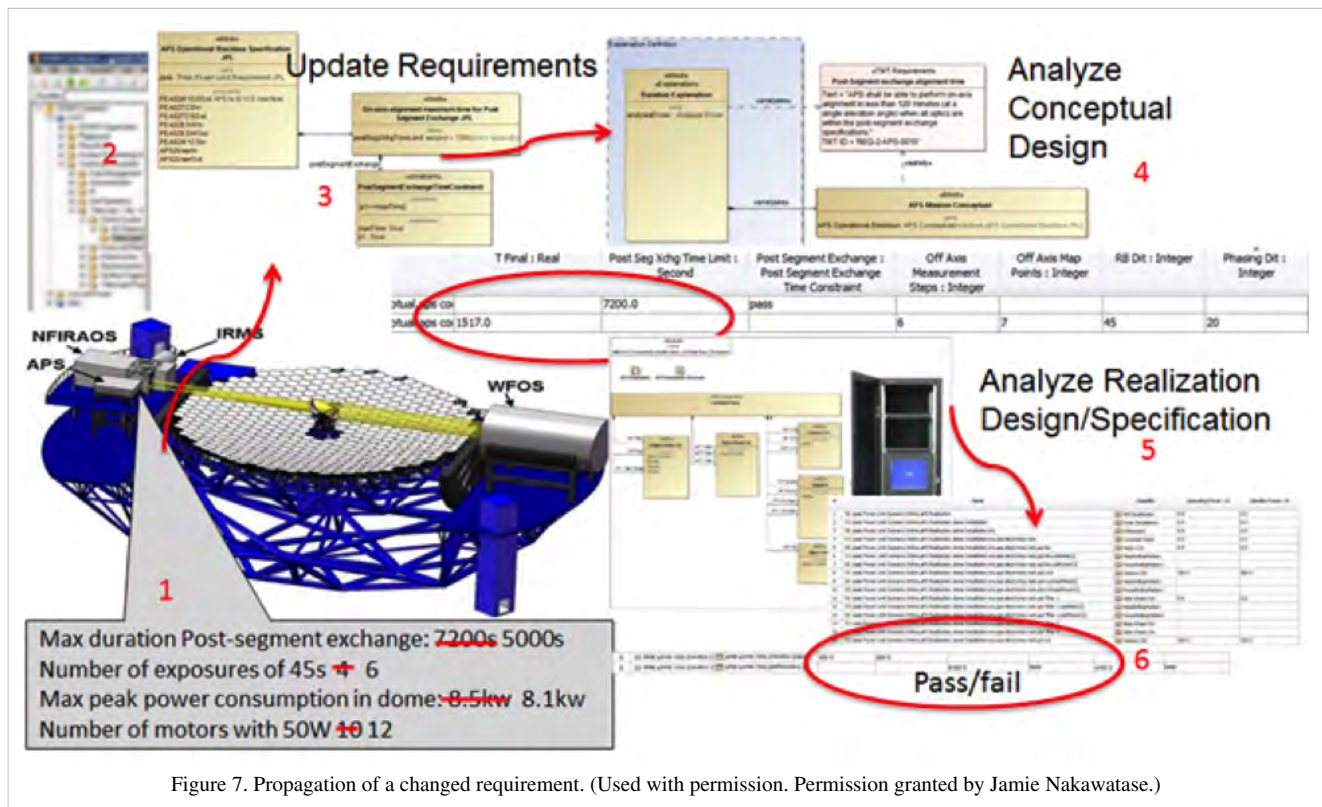


Figure 7. Propagation of a changed requirement. (Used with permission. Permission granted by Jamie Nakawatase.)

In this article, a simplified APS model is shown to illustrate the model-based approach for analyzing changed requirements. The full analysis is available in [4] and [5].

A model-based approach was also applied to APS for error analysis, which was performed to describe the accuracy of expected system performance against requirements. It involved multiple artifacts to analyze a requirement such as “APS shall measure the position of the telescope pupil to an accuracy of 0.03% of the diameter of the pupil.” Defining requirements and parameters in the model indicated the required accuracy and current best estimates of the system design. Defining various roll-up patterns allowed for error decomposition calculations. The benefit is realized in automated requirements verification when applying a parametric solver to formulate results for specified equations in the model. This model-based approach formally integrates accuracy requirements with the system design.

The system model for NFIRAOS LGS MCAO and NGS AO modes was developed to capture sequence behaviors and operational scenarios to run Monte-Carlo simulations for verifying acquisition time, observing efficiency, and operational behavior requirements. The model is particularly useful for investigating the effect of parallelization, identifying interface issues, and re-ordering sequence acquisition tasks.

ESEM enables system analysis by conducting quantitative assessments to select and/or update the most efficient system architecture and generate derived engineering data. System analysis provides rigor for technical decision-making. It includes modeling and simulation, cost, technical risks, and effectiveness analyses, and is used to perform trade studies. In particular, it supports requirements verification, which assesses whether a system design meets its objectives and satisfies the constraints levied by system requirements.

Observed Benefits

The MBSE approach applied to APS and NFIRAOS was motivated by optimization to coordinate the efforts of complex system development. In these applications, implicit dependencies are made explicit in a formal model through the use of ESEM, OpenMBEE, and SysML modeling constructs. Requirements are formalized and tracked directly to the evolving system design. This tight association of requirements within a common environment promotes cross-domain integration and efficient communication among stakeholders. The model is used to automate requirements verification and to generate systems engineering products. The benefit over a traditional document-based approach is that currently disconnected artifacts become related in the model, enabling the production of consistent model-based documentation. Requirements verification is an important analysis conducted in the context of MBSE. To perform this analysis, the requirements, executable behavior, and models predicting the system's performance must be integrated. The ability to integrate these elements using ESEM and the OpenMBEE tooling infrastructure is a significant value proposition for the MBSE approach described in this article. In the formally integrated and executable SysML model, simulations are performed to analyze the impact of changed requirements and verify that requirements are met within specified constraints for various operational scenarios. MBSE enhances information exchange through created visualizations that communicate system behavior. For example, duration analyses were performed to study acquisition time for observing. The use of Monte-Carlo simulations proved how the model-based approach optimized the analysis process. Higher quality analysis results were obtained through the execution of operational scenario runs in an articulated system model, and the model continues to serve as a communication tool across various domains.

Acknowledgements

This research was carried out at the Jet Propulsion Laboratory (JPL), California Institute of Technology, under a contract with the National Aeronautics and Space Administration (NASA), and at NoMagic. The TMT Project gratefully acknowledges the support of the TMT collaborating institutions. They are the California Institute of Technology, the University of California, the National Astronomical Observatory of Japan, the National Astronomical Observatories of China and their consortium partners, the Department of Science and Technology of India and their supported institutes, and the National Research Council of Canada. This work was supported as well by the Gordon and Betty Moore Foundation, the Canada Foundation for Innovation, the Ontario Ministry of Research and Innovation, the Natural Sciences and Engineering Research Council of Canada, the British Columbia Knowledge Development Fund, the Association of Canadian Universities for Research in Astronomy (ACURA), the Association of Universities for Research in Astronomy (AURA), the U.S. National Science Foundation, the National Institutes of Natural Sciences of Japan, and the Department of Atomic Energy of India.

References

Works Cited

[1] Zwemer, D., "Connecting SysML with PLM/ALM, CAD, Simulation, Requirements, and Project Management Tools", Intercax LLC, May 2011. [2] Friedenthal S, Moore A., and Steiner R., "A Practical Guide to SysML 3rd Ed.", Morgan Kaufmann OMG Press, 2014 [3] ISO/IEC, ISO/IEC 42010:2011, Systems and software engineering - Architecture description

Primary References

[4] Karban, R., Jankevičius, N., Elaasar, M. "ESEM: Automated Systems Analysis using Executable SysML Modeling Patterns", INCOSE International Symposium (IS), Edinburgh, Scotland, 2016 [5] Karban, R. "Using Executable SysML Models to Generate System Engineering Products", NoMagic World Symposium, 2016 [6] OMG SysML, "Systems Modeling Language (SysML) Version 1.4", OMG, 2014 [7] Open Source Engineering

Environment: <https://open-mbee.github.io/> [8] TMT, "Thirty Meter Telescope." <https://www.tmt.org> [9] JPL, "Jet Propulsion Laboratory." <https://www.jpl.nasa.gov/> [10] Karban R., Dekens F., Herzig S., Elaasar M, Jankevičius N., "Creating systems engineering products with executable models in a model-based engineering environment", SPIE, Edinburgh, Scotland, 2016

Additional References

<https://github.com/Open-MBEE/TMT-SysML-Model> <https://github.com/Open-MBEE/mdk/tree/support/2.5/manual> <https://www.omgwiki.org/MBSE/doku.php?id=mbse:telescope> <https://groups.google.com/forum/#!forum/openmbee>

Miniature Seeker Technology Integration Spacecraft

- Lead Authors:
- Heidi Davidz, Art Pyster, and Deva Henry
- Contributing Author:
- Dave Olwell

The Miniature Seeker Technology Integration (MSTI) spacecraft was the first of its kind: a rapid development spacecraft, designed and launched in one year. As an aerospace example for a satellite application, the case study, "M.S.T.I.: Optimizing the Whole System" (Grenville, Kleiner, and Newcomb 2004), describes the project's systems engineering approach. Driven by an aggressive schedule, MSTI optimized over the whole project, rather than allowing sub-optimizations at the component level. As a partnership with Phillips Laboratories, the Jet Propulsion Laboratory (JPL), and Spectrum Astro, MSTI went into orbit on November 21, 1992. The MSTI-1 succeeded in meeting all primary mission objectives, surpassing the 6-day data collection mission requirement.

Domain Background

There are many case study examples for aerospace systems. This case is of particular interest because it highlights mechanisms which enabled successful performance following an aggressive schedule. Since this rapid development spacecraft was designed and launched in one year, new ways of structuring the project were necessary. Within this domain, the MSTI project used an innovative approach. Practices from this project led to the Mission Design Center and the System Test Bed at JPL.

Case Study Background

This case study was developed in support of the National Aeronautics and Space Administration (NASA) Program and Project Management Initiative by authors at the Virginia Polytechnic Institute and State University and Scientific Management, Inc. The case study was developed in the interest of continuously improving program and project management at NASA (NASA 2010). Research for this case included comprehensive literature review and detailed interviews. The project was selected based on the potential for providing lessons learned.

Case Study Description

The MSTI case study illustrates many principles described in the Systems Engineering Body of Knowledge (SEBoK). The MSTI team had to make adjustments to the traditional approach to spacecraft development in order to stay within budget and to meet the aggressive timeline of bringing a spacecraft from conception to launch within one year. The team realized that they were "building Porsches not Formula 1s"(Grenville, Kleiner, Newcomb 2004). Meeting the schedule was a crucial factor that affected all decisions. The SEBoK knowledge area on life cycle models describes life cycle design in more detail.

The team took advantage of existing hardware architectures for their architectural design to expedite the project. In addition, at each design phase, the whole system was optimized instead of optimizing subsystems, and the level of optimization at the subsystem level was reduced. A hardware-in-the-loop test bed was used throughout the project, which expedited system integration.

The schedule was maintained only at a high level in the project management office, and the costs were managed using a cost reporting technique for "cost to completion." Rather than report on past spending, the Responsible Engineering Authorities (REAs) were expected to continually evaluate their ability to complete their tasks within projected costs. Faster procurement was achieved using the Hardware Acquisition Team, where a technical team member was matched with a procurement representative for each design function. This pair wrote the specifications together and initiated the purchase requisitions.

From the organizational perspective, increased responsibility and accountability were given to each team member. Individuals took ownership of their work and the decision process was streamlined. The team made more "good decisions," rather than optimal decisions. The team was colocated, and daily meetings were used to assign daily tasks and keep the team focused on the launch. The standard Problem Failure Report (PFR) was streamlined and electronic reports provided snapshots of the resolved and outstanding PFRs. The report helped REAs stay on top of potential problem areas. REAs were responsible for looking forward on the project horizon and notifying the team of any potential problem areas.

The first satellite in the MSTI series, MSTI-1, was launched on November 21, 1992. The spacecraft weighed 150 kg and was built for \$19M in less than 12 months. Over 200,000 photographs were returned from the spacecraft. From a project management standpoint, all mission objectives were completed.

In addition, MSTI had a lasting legacy. Faster procurement developed into an approach JPL now calls "Fast Track Procurement." Hardware acquisition teams are used often in JPL projects. The hardware-in-the-loop test bed was the precursor to the Flight System Test Bed at JPL. Team members moved up quickly in JPL due to the increased responsibility and authority they were given on the MSTI project.

Summary

MSTI demonstrated that an aggressive schedule can be used to design low earth-orbiting spacecraft to optimize the full system. The MSTI experience changed JPL's culture and their approach to spacecraft development and mission management. The insights from this case study example can help both students and practitioners better understand principles described in the SEBoK.

References

Works Cited

Grenville, D., B.M. Kleiner, and J.F. Newcomb. 2004. *M.S.T.I., Optimizing the Whole System*. Blacksburg, VA: Virginia Polytechnic Institute, case study developed in support of the NASA Program and Project Management Initiative. 1-27. Accessed June 3, 2011. Available at https://www.nasa.gov/pdf/293212main_58529main_msti_casestudy_042604.pdf.

NASA. 2010. *A Catalog of NASA-Related Case Studies*, version 1.6. Compiled by the Office of the Chief Knowledge Officer, Goddard Space Flight Center, MD, USA: National Aeronautics and Space Administration (NASA). Accessed June 3, 2011. Available at https://www.nasa.gov/centers/goddard/pdf/450420mainNASA_Case_Study_Catalog.pdf.

Primary References

None.

Additional References

None.

Apollo 1 Disaster

- Lead Authors:
- W. Clifton Baldwin and Anthony Long

On January 27, 1967, the crew of Apollo 204 was training for the first crewed Apollo flight, an Earth orbiting mission scheduled for launch on 21 February. Flight commander Gus Grissom, astronaut Edward White and astronaut Roger Chaffee died when fire swept the Apollo Command Module during this preflight test. After the accident, NASA reclassified Apollo 204 as Apollo 1. This case study examines some of the human factors shortfalls that lead to the Apollo 1 disaster.

Background

On January 27, 1967, the crew of Apollo 204 was training for the first crewed Apollo flight, an Earth orbiting mission scheduled for launch on 21 February. They were involved in a "plugs-out" test on the launch pad just as in the actual launch, except the rocket was not fueled. This test was a simulation, going through an entire countdown sequence. Flight commander Gus Grissom, astronaut Edward White and astronaut Roger Chaffee died when fire swept the Apollo Command Module during this preflight test. After the accident, NASA reclassified Apollo 204 as Apollo 1.

During the test and subsequent accident, emergency teams were not in attendance (Benson and Faherty 1978). The fire crews were only on standby since the vehicle was not fueled (Freiman and Schlager 1995). It was believed that the test did not rate a hazardous classification (Benson and Faherty 1978) (NASA History Office 1967), and the emergency equipment located in the launch tower test room was not designed for the type of fire that resulted (NASA History Office 1967). Within the capsule, there were no design features for fire protection as no one had considered the possibility of a fire from anything other than the rocket engines (NASA History Office 1967). There was not even a fire extinguisher in the cabin (Freiman and Schlager 1995) (Kranz 2000). Astronaut Frank Borman later stated, "None of us gave any serious consideration to a fire in the spacecraft" (Benson and Faherty 1978).

NASA leveraged technical knowledge from the two earlier Mercury and Gemini space programs and utilized their designs as a baseline for the Apollo program (Rosholt 1966). Naturally, some problems were expected from such a huge undertaking. Due in part to a multitude of integration issues, the crew could not escape the fire. After the accident, however, NASA officials admitted that they had concentrated their efforts on "in flight" situations and had not even considered problems on the ground (Benson and Faherty 1978) (Kranz 2000).

Challenges

During the late 1960s, NASA's systems integration group appeared to be largely paperwork focused, although NASA considered the Apollo 204 test as a type of systems integration test (Baldwin and Reilly 2005). Regardless of the systems integration efforts, there were obvious gaps integrating the astronauts leading to the unfortunate consequence of the deaths of the men of Apollo 1.

Integration of the Hatch

Designing and integrating safety into the space capsule was known to be an important factor from the start of the space program. The hatch was the primary means for the astronauts to enter and exit the capsule, and therefore it was a vital component for integrating the astronauts.

Both the Mercury and the Apollo capsules were equipped with a means for escaping from a launch-vehicle failure (Purser, Faget and Smith 1965) (Swenson Jr., Grimwood and Alexander 1998). This escape system consisted of a booster rocket on the capsule that could fly the capsule away from a malfunctioning rocket. The Gemini capsule had ejection seats instead. Due to the dangers during an emergency ejection, the Apollo design went back to an escape tower booster (Purser, Faget and Smith 1965). Without the ejection seats, the quick-opening hatches used by the Gemini program were not required. Initially, the Apollo capsule contractors North American Aviation had recommended a hatch that opens outward with explosive bolts for emergencies. NASA designers disagreed due to the accidental opening of an earlier Mercury capsule with a similar hatch design (Brooks, Grimwood and Swenson 1979). "NASA and North American designers hadn't been as worried about escape contingencies as they were about the possibility of a hatch popping open into the vacuum of space or another inadvertent opening during a water landing" (Kranz 2000).

In order to keep the astronauts safe, "An Apollo mission designer would prefer that the crew never exit the space capsule" (Mendell 1998). Therefore, the designers integrated the hatch to open inward, which allowed the internal pressure to assist in keeping the hatch secure (Murray and Cox 1989). The result of the integration process was a three-part hatch, an inner pressure hatch that opened inward when the capsule was on the ground, an ablative hatch that opened outward when in space, and a boost protective cover to protect the capsule during launch from the escape tower boosters (Freiman and Schlager 1995) (Kranz 2000) (NASA History Office 1967). Furthermore, the designers chose not to have an explosive hatch. As an aside to the Apollo 1 accident, even if the capsule had an explosive hatch, it would not have been armed during the test due to the danger to the support personnel (Murray and Cox 1989).

It took at least 90 seconds to open the hatch under ideal conditions (Freiman and Schlager 1995) (Senate Committee on Aeronautical and Space Sciences 1968). In practice, the crew had never accomplished the egress in the minimum time. Additionally, escaping was a very complicated procedure to perform under emergency circumstances. For example, it required one astronaut to lower another one's headrest in order to actuate a ratchet-type device that would release the first of a series of latches (NASA History Office 1967). When the accident occurred, it took five minutes and 25 seconds to open the hatch (NASA History Office 1967). The Apollo Review Board criticized this problem as well as obviously recommended it to be changed (Benson and Faherty 1978).

The accident of Apollo 1 caused NASA to reconsider its decisions and processes. Although well integrated technically, NASA was lacking in integrating the astronauts with the hatch. To remedy this problem, the hatch was redesigned to be single-hinged that could be unlatched in three seconds and would swing outward with minimal force (Benson and Faherty 1978).

Integration of the Environmental Control System

Perhaps the most complex of all the human factors elements concerned the Environmental Control System (ECS). This system was designed to control the quantity and quality of air delivered to the astronauts, maintain cabin pressure, and heat and cool the astronauts, equipment, and cabin. Extremes of space flight had to be anticipated and this system needed to meet the needs of that harsh environment. Redundancy was built in to provide suitable backup systems and ensure reliability and availability (NASA History Office 1967).

NASA engineers had performed trade studies that concluded a pure oxygen atmosphere in the cabin was preferred. Again, this decision failed to fully consider the astronauts. In 1964, Dr. Emmanuel Roth of the Lovelace Foundation for Medical Education and Research prepared for NASA a paper warning about the dangers of pure oxygen (Benson and Faherty 1978). Natural fabrics, most synthetics, and even allegedly flameproof materials will burn violently in a pure oxygen environment. In that same year, Dr. Frank J. Hendel, a staff scientist with Apollo Space Sciences and Systems at North American Aviation, wrote an article warning against pure oxygen especially on the launch pad (Benson and Faherty 1978). Joe Shea, head of Apollo Spacecraft Program Office at the time, wrote in a memo, "The problem is sticky- we think we have enough margin to keep fire from starting - if one ever does, we do have problems. Suitable extinguishing agents are not yet developed" (Murray and Cox 1989).

Due to the ongoing redesign and test environment in which the ECS was operated, there was a need to change out components quickly and easily to save time on the schedule (NASA History Office 1967). This need resulted in poor wiring placement as well as insulation (Stavnes and Hammoud 1994). Coolant coils were placed in locations that permitted them to be used as a handle to move about in the cabin. This unintended usage led to a leakage of coolant in the cabin, whereby the vapors were flammable and the coolant itself in liquid form was corrosive to the insulation of the nearly 12 miles of electrical wiring in the command module (Freiman and Schlager 1995) (NASA History Office 1967). The cooling system was extensive throughout the capsule, and coolant leakage at solder joints had already been a chronic problem (NASA History Office 1967).

One ECS cable was wedged against the bottom of a door used by the astronauts. When the door was shut, it would scrape the cable. The repeated abrasion eventually exposed two tiny sections of wire on the cable (Murray and Cox 1989). When the insulation became worn away, the wiring system would fail, and sparks could arc (Stavnes and Hammoud 1994). To make matters worse, flammable raschel netting near the scuffed cable was located closer to the cable than it should have been (Murray and Cox 1989).

The astronauts' spacesuits were also not incorporated well into the ECS. A suit-loop provided air quality control, temperature control, pressure control, humidity control, and decontamination to the astronauts and the cabin. There were three astronauts suited up and plugged into the loop with a fourth suit position. This so-called fourth suit position provided forced ventilation and exchange of the cabin air with the suit circuit (Bellcomm, Inc. 1964). This link of the spacesuits to the cabin could not be closed off in an emergency. The result would allow internal toxic gases from a fire to penetrate the astronauts' suits.

Integration of the Egress System

Until the accident, no one seriously considered the possibility of a safety issue within the capsule. The egress system, which would allow astronauts to get away from the launch pad, was not thoroughly explored and several integration problems were missed (NASA History Office 1967). "We all assumed that when a calamity struck, it would be in flight. Our nightmare was an explosion during launch, or a flying coffin, a faulty craft stuck in endless orbit" (Kranz 2000). There were no formal procedures for an in-capsule emergency on the ground for either the crew or the spacecraft pad work team (NASA History Office 1967).

The designers' experience necessitated "The use of an escape system should a malfunction occur during the powered ascent portion of the trajectory" (Purser, Faget and Smith 1965). Unfortunately, the designers only considered escape situations where the astronauts had to remain in the capsule. Hazard analysis was done to "Examine all the hazards that might require escape from the launch vehicle during powered flight" (Purser, Faget and Smith 1965).

Nonetheless, the hazards were thought to come through three operational phases, “1) liftoff and shortly thereafter, 2) transonic through maximum dynamic pressure regimes, and 3) shutdown and staging” (Purser, Faget and Smith 1965). There is no mention of a hazard within the capsule itself.

Additional evidence for unsatisfactory egress can be found in the launch pad environment. Even if they could get the hatch open, there were no contingency preparations to permit escape or rescue of the crew from an internal capsule fire. The umbilical tower access arm contained features such as steps, sliding doors and sharp turns in the egress paths that hindered emergency operations (NASA History Office 1967). Albeit too late for Apollo 1, the Apollo 204 Review Board sharply criticized the fact that the astronauts had no quick means of escaping the capsule (Benson and Faherty 1978).

Integration Lessons

Systems Thinking Approach

A systems approach to integration with respect to the capsule and rocket may have avoided overlooking the users’ needs inside the capsule. For example, the ECS was designed without fully considering the astronauts onboard. Decisions to limit space in order to restrict subsystems from growing in weight were made by rule of thumb. A “whole system” integration approach could have arrived at a design for the ECS while considering the astronauts’ needs. Early integration of the astronauts could have saved time, especially since those changes had to be made eventually, and would have saved lives. For more information, see Part 2, Overview of the Systems Approach.

Use Cases

NASA designers stated they considered three phases of operations which were known to be hazardous. A problem with their analysis is that they failed to consider every phase, such as prelaunch or rather preflight. The designers should have evaluated every feasible scenario, or use case, of the system, even if unlikely. Thorough use case analysis evaluates all potential “normal” and “rainy day” scenarios. Use cases could have been developed for the preflight phase, including any potential failure cases on the ground. The designers could have used these scenarios to contemplate hatch issues, coolant coil issues, and especially the egress process.

Failure Analysis

The plugs-out test of Apollo 1 was ranked as a low risk without much analysis, and NASA officials stated that they were not concerned with problems prior to launch. Although a failure analysis was conducted for the flight of Apollo 204, no failure analysis, such as a Failure Modes and Effects Analysis (FMEA), was conducted for an on-the-ground or prelaunch situation. Once identified, proper mitigation actions could have been implemented. For more information, see Part 3, Risk Management and Part 6, System Safety.

References

Works Cited

- Baldwin, W. C., and C. K. Reilly, interview by Rich Morton. 2005. *Interview with former NASA Technician during January 1967* (Dec. 13).
- Bellcomm, Inc. 1964. *Review of Environmental Control Systems for Apollo*. B-1.
- Benson, C. D., and W. B. Faherty. 1978. *Moonport: A History of Apollo Launch Facilities and Operations*. Vols. NASA Special Publication-4204 in the NASA History Series. Washington D.C., USA: National Aeronautics and Space Administration. Available at: <https://www.hq.nasa.gov/office/pao/History/SP-4204/contents.html>. Accessed Nov. 22, 2005.
-

- Brooks, C., J. Grimwood, and L. Swenson. 1979. *Chariots for Apollo: A History of Manned Lunar Spacecraft*. Vols. NASA Special Publication-4205 in the NASA History Series. Washington D.C., USA: National Aeronautics and Space Administration.
- Freiman, F. L., and N. Schlager. 1995. "Apollo 1 catches fire," Vol. 1, in *Failed Technology: True Stories of Technological Disasters*, by Fran Locher Freiman and Neil Schlager. New York, NY, USA: UXL.
- House Subcommittee on NASA Oversight of the Committee on Science and Astronautics. 1967. *Investigation into Apollo 204 Accident: Hearings*. Washington D.C., USA: 90th Congress, 1st Session, pp. 1-404.
- Kranz, G. 2000. *Failure Is Not an Option*. New York, NY, USA: Simon and Schuster.
- Mendell, W. W. 1998. "Role of lunar development in human exploration of the solar system," *Journal of Aerospace Engineering*, pp. 106 - 110.
- Murray, C., and C. B. Cox. 1989. *Apollo: The Race to the Moon*. New York, NY, USA: Simon & Schuster.
- NASA History Office. 1967. *Findings, Determinations and Recommendations*. Apollo 204 Review Board, National Aeronautics and Space Administration. Washington D.C., USA: NASA Historical Reference Collection. Available at: <https://www.hq.nasa.gov/office/pao/History/Apollo204/find.html>. Accessed Feb. 18, 2020.
- NASA History Office. 1967. *History of The Accident*. Apollo 204 Review Board, National Aeronautics and Space Administration. Washington, D.C., USA: NASA Historical Reference Collection. Available at: <https://www.hq.nasa.gov/office/pao/History/Apollo204/history.html>. Accessed Feb. 18, 2020.

Primary References

None.

Additional References

None.

Transportation System Examples

Denver Airport Baggage Handling System

- Lead Authors:
- Art Pyster and Heidi Davidz

This example was developed as a SE example for the SEBoK. It describes systems engineering (SE) issues related to the development of the automated baggage handling system for the Denver International Airport (DIA) from 1990 to 1995. The computer-controlled, electrical-mechanical system was part of a larger airport system.

Description

In February 1995, DIA was opened 16 months later than originally anticipated with a delay cost of \$500 million (Calleam Consulting Ltd. 2008). A key schedule and cost problem—the integrated automated baggage handling system—was a unique feature of the airport. The baggage system was designed to distribute all baggage automatically between check-in and pick-up on arrival. The delivery mechanism consisted of 17 miles of track on which 4,000 individual, radio-controlled carts would circulate. The \$238 million system consisted of over 100 computers networked together, 5,000 electric eyes, 400 radio receivers, and 56 bar-code scanners. The purpose of the system was to ensure the safe and timely arrival of every piece of baggage. Significant management, mechanical, and software problems plagued the automated baggage handling system. In August 2005, the automated system was abandoned and replaced with a manual one.

The automated baggage system was far more complex than previous systems. As planned, it would have been ten times larger than any other automated system, developed on an ambitious schedule, utilized novel technology, and required shorter-than-average baggage delivery times. As such, the system involved a very high level of SE risk. A fixed scope, schedule, and budget arrangement precluded extensive simulation or physical testing of the full design. System design began late, as it did not begin until well after construction of the airport was underway. The change management system allowed acceptance of change requests that required significant redesigns to portions of work already completed. The design did not include a meaningful backup system; for a system that required very high mechanical and computer reliability, this increased failure risks. The system had an insufficient number of tugs and carts to cope with the volume of baggage expected and this, along with severely limited timing requirements, caused baggage carts to jam in the tracks and for them to misalign with the conveyor belts feeding the bags. This resulted in mutilated and lost bags (Neufville 1994; Gibbs 1994).

The baggage system problems could be associated with the non-use or misuse of a number of systems engineering (SE) concepts and practices: system architecture complexity, project scheduling, risk management, change management, system analysis and design, system reliability, systems integration, system verification and validation/testing, and insufficient management oversight.

Summary

The initial planning decisions, such as the decision to implement one airport-wide integrated system, the contractual commitments to scope, schedule, and cost, as well as the lack of adequate project management (PM) procedures and processes, led to a failed system. Attention to SE principles and practices might have avoided the system's failure.

References

Works Cited

Calleam Consulting Ltd. 2008. *Case Study – Denver International Airport Baggage Handling System – An Illustration of Ineffectual Decision Making*. Accessed on September 11, 2011. Available at https://calleam.com/WTPF/?page_id=2086.

Neufville, R. de. 1994. "The Baggage System at Denver: Prospects and Lessons." *Journal of Air Transport Management*. 1(4): 229-236.

Gibbs, W.W. 1994. "Software's Chronic Crisis." *Scientific American*. September 1994: p. 72-81.

Primary References

None.

Additional References

DOT. 1994. *New Denver Airport: Impact of the Delayed Baggage System*. US Department of Transportation (DOT), Research Innovation Technology Administration. GAO/RCED-95-35BR. Available at <https://ntl.bts.gov/DOCS/rc9535br.html>

Donaldson, A.J.M. 2002. *A Case Narrative of the Project Problems with the Denver Airport Baggage Handling Systems (DABHS)*. Software Forensics Center Technical Report TR 2002-01. Middlesex University, School of Computer Sciences. Available at <https://www.eis.mdx.ac.uk/research/SFC/Reports/TR2002-01.pdf>

Federal Aviation Administration (FAA)

Advanced Automation System (AAS)

- Lead Authors:
 - Tom Hilburn, Alice Squires, and Heidi Davidz
- Contributing Author:
 - Richard Turner

This example was created as a SE example directly for the SEBoK. It describes the Advanced Automation System (AAS), part of the Federal Aviation Administration (FAA) Advanced Automation Program. It describes some of the problems which can occur in a complex, software intensive system program if SE is not applied.

Description

In 1981, the Federal Aviation Administration (FAA) announced the Advanced Automation Program, which was established to modernize air traffic control (ATC) computer systems. A centerpiece of the project was the Advanced Automation System (AAS). AAS was the largest project in FAA's history to modernize the nation's ATC system. AAS would replace computer hardware and software as well as controller workstations at tower, terminal, and en-route facilities and allow the ATC system to accommodate forecasted large increases in traffic through the use of modern equipment and advanced software functions. (GAO 1992)

The FAA originally proposed AAS in 1982 as a project that would cost \$2.5 billion and be completed in 1996. However, substantial cost increases and schedule delays beset the AAS project over its history, caused by numerous problems in AAS development:

- The project began with a design competition between Hughes and IBM. The competition involved numerous extensions and took four years to complete. Analysis by the FAA and others pointed to inadequate consideration of user expectations and improper assessment of the technology risks. (Barlas 1996)
- The FAA pushed for 99.99999% reliability, which was considered by some "more stringent than on any system that has ever been implemented" and extremely costly. (DOT 1998)
- The program created unworkable software testing schedules - "Testing milestones were skipped or shortcut and new software was developed assuming that the previously developed software had been tested and performed." (Barlas 1996)
- There were an extraordinary number of requirements changes. For example, for the Initial Sector Suite System (ISSS), a key component of AAS, there were over 500 requirements changes in 1990. Because of these changes, 150,000 lines of software code had to be rewritten at a cost of \$242 million. (Boppana et al. 2006)
- IBM's cost estimation and development process tracking used inappropriate data, were performed inconsistently, and were routinely ignored by project managers. The FAA conservatively expected to pay about \$500 per line of computer code - five times the industry average. The FAA ended up paying \$700 to \$900 per line for the AAS software. (Gibbs 1994)
- In 1988, FAA estimated that the AAS program - both contract and supporting efforts - would cost \$4.8 billion. By late 1993, the FAA estimated that it would cost \$5.9 billion. Before the program was dramatically restructured in 1994, estimates had risen to as much as \$7 billion, with key segments expected to be behind schedule by as much 8 years. In 1994, with significant cost and schedule overruns, as well as concerns about adequate quality, usability, and reliability, the AAS program ceased to exist as originally conceived, leaving its various elements terminated, restructured, or as parts of smaller programs. (DOT 1998)

The AAS problems could be associated with the non-use or misuse of a number of systems engineering (SE) concepts and practices: system requirements, system architecture complexity, project planning, risk management,

change management, system analysis and design, system reliability, availability, and maintainability, system integration, system verification and system validation/testing, and management oversight.

Summary

The AAS program was the centerpiece of an ambitious effort begun in the 1980s to replace the computer hardware and software throughout the ATC system - including controller workstations, and en-route, terminal, and tower air traffic control facilities. AAS was intended to provide new automated capabilities to accommodate increases in air traffic. After sustaining serious cost and schedule problems, the FAA dramatically restructured the program into more manageable pieces. This action included terminating major segments of the contract. (DOT 1998.)

References

Works Cited

Barlas, S. "Anatomy of a Runaway: What Grounded the AAS." *IEEE Software*. vol. 13, no. 1, Jan., pp. 104-106, 1996.

Boppana, K., S. Chow, O.L. de Weck, C. LaFon, S.D. Lekkakos, J. Lyneis, M. Rinaldi, Z. Wang, P. Wheeler, M. Zborovskiy. "Can models capture the complexity of the systems engineering process?" In *Proceedings of the International Conference on Complex Systems (ICC2006)*, 11-15 June 2006, Istanbul, Turkey.

DOT. *Audit Report: Advance Automation System, Federal Aviation Administration*. Washington, DC, USA: Office of Inspector General, U.S. Department of Transportation, 2005.

GAO. *Advanced Automation System Still Vulnerable to Cost and Schedule Problems*. Washington, DC, USA: United States General Accounting Office (GAO). GAO/RCED-92-264. 1992.

Gibbs, W.W. "Software's Chronic Crisis." *Scientific American*. September 1994.

Primary References

None.

Additional References

Britcher, R. *The Limits of Software: People, Projects, and Perspectives*. Boston: Addison Wesley, 1999.

Federal Aviation Administration (FAA) Next Generation Air Transportation System

- Lead Author:
- Brian White

This article describes a massive undertaking to modernize the air traffic management enterprise. The topic may be of particular interest to those involved in air transportation whether in connection with their careers or as pilots or passengers on airplanes. For additional information, refer to the closely related topics of Enabling Businesses and Enterprises and Enterprise Systems Engineering.

Background

This case study presents the systems engineering and enterprise systems engineering (ese) efforts in the Next Generation (NextGen) Air Transportation Systems by the Federal Aviation Administration (FAA 2008). NextGen is an unprecedented effort by multiple U.S. federal organizations to transform the U.S. air transportation infrastructure from a fragmented ground-based navigation system to a net-centric satellite-based navigation system. This project is unique to the FAA because of its large scale, the huge number of stakeholder(s) involved, the properties of the system of interest, and the revolutionary changes required in the U.S. Air Transportation Network (U.S. ATN) enterprise.

A sociotechnical system like the U.S. ATN is a “large-scale [system] in which humans and technical constituents are interacting, adapting, learning, and coevolving. In [such] systems technical constraints and social and behavioral complexity are of essential essence”. (Darabi and Mansouri 2014). Therefore, in order to understand changes in the U.S. ATN it was seen as necessary to view it through a lens of evolutionary adaptation rather than rigid systems design. The U.S. ATN serves both military and commercial aircraft with its 19,782 airports, including 547 commercial airports. Nineteen major airlines, with more than a billion dollars in annual total revenue, along with other 57 national and regional airlines, transport 793 million passengers and realize 53 billion revenue ton-miles.

The Air Traffic Organization (ATO) is responsible for ensuring aircraft navigation in the U.S. National Air Space (NAS) system using a five-layer architecture. Each aircraft goes through different layers and possibly various zones of this architecture as it takes off from an airport until it lands at another airport (Donohue and Zellweger 2001). However, this architecture is fragmented and many issues are raised: an airplane’s path through its route is not optimized, and the path may change its direction from one zone to another, the destination airport’s capacity is limited by the current regulations of minimum aircraft separation distance due to navigation limitations, the real-time weather information is not integrated into the system, and communications are mainly voice-based, etc.

In NextGen major changes to the U.S. ATN design are planned. As already stated, the navigation system will be changed from ground-based communication to satellite-based navigation. The current fragmented architecture will be integrated into a seamless net-centric information system in which the digital communication will replace the current voice communications. Moreover, weather information will be assimilated into decision making and planning across the system.

Purpose

The FAA's purpose is "to provide the safest, most efficient aerospace system in the world". Toward this end the NextGen project is aimed at enhancing the U.S.'s leadership position in air transportation.

During the last three decades the demand for air transportation shows exponential growth. In just one decade from 1995 to 2005 this demand showed a 44% percent increase. Therefore, the change in infrastructure was inevitable. Moreover, 9/11 attacks on the U.S. ATN emphasized this need for change. The combination of a requirement for a safer and more secure network and increasing demand was the motivation for President Bush to enact the Vision 100-Century of Aviation Reauthorization Act on 2003. A major part of this Act was to revolutionize the U.S. ATN by means of the NextGen project. The first integration plan of the project was released in 2004, and the project is estimated to continue until 2025.

The demand behavior of the U.S. ATN shows diverse degrees of congestion among airports. Although there are multitudes of airports in the system, the top 35 most congested airports carried more than 60% of the total traffic consistently during the period of 2000 to 2008. Because the growth of the network demand is not proportional, the demand in congested airports will be even higher.

A study by the Joint Planning and Development Office (JPDO) shows that flight delays in the current network will cause \$6.5 billion of economic loss until 2015, and \$19.6 billion until 2025. By implementing NextGen the delays are estimated to be reduced by 38% until 2020. Moreover, aircraft CO₂ emissions are a major part of environmental pollution in crowded cities; these will be reduced by 14 million metric tons by 2020. The current level of jet fuel usage is also a known problem because of increasing fuel prices. The NextGen project will improve fuel usage by 1.4 billion gallons cumulative through 2020.

NextGen is pursuing multiple goals to retain the U.S. leadership in aviation, to expand the U.S. ATN capacity, to continue to ensure safety, to increase environment protection, to help ensure national air defense, all generally helping to increase the nation's security (JPDO 2007a).

Eight general capabilities are defined in conducting this mission: (1) network-enabled information access, (2) performance-based operations and services, (3) weather assimilated into decision making, (4) layered adaptive security, (5) positioning, navigation, and timing (PNT) services, (6) aircraft trajectory-based operations (TBO), (7) equivalent visual operations (EVO), and (8) super-density arrival/departure operations.

To create the desired capabilities, general areas of transformations are defined as air traffic management operations, airport operations and infrastructure services, net-centric infrastructure services, shared situational awareness services, layered and adaptive security services, environmental management services, safety management services, and performance management services. The detailed changes in each area are discussed in Concept of Operations for NextGen (JPDO 2007a).

Challenges

An instructive part of this case study is observing evolution in understanding challenges from initial steps of the project through current efforts for delivering it. As an overall conclusion, the perspective on challenges shifted from technical problems and intra-organizational issues to more enterprise-wide issues.

The NextGen Implementation Plan 2008 discussed the following challenges (FAA 2008):

- performance analysis, to understand and assess operational capabilities
 - policy, to balance responsibility between humans and automation, for environmental management processes, and for global harmonization strategies
 - acquisition workforce staffing
 - environmental planning, to resolve conflicts with local environmental constraints
 - security
-

- transition from current ground-based navigation to automatic dependent surveillance – broadcast (ADS-B) technology.

A more recent report on Targeted NextGen Capabilities for 2025 (JPDO 2011) highlights the effect of the multi-stakeholder nature of the project on raising additional challenges. Achieving Interagency Collaboration is the first issue, which is important in implementing security, safety, policy making, and technological advancement.

Increasing capacity, reducing delay and protecting the environment are the main three promises of the NextGen project. However, reaching the defined high standards is not an easy task. A major part of this challenge is integrating new technologies into legacy systems, aircraft, airports, facilities, and organizations. Airlines and general aviation pilots resist the expense of additional avionics and communications equipment, even though it bolsters the common good of air travel.

Maintaining airports and airspace security requires coherent and harmonious work of multiple U.S. agencies. The core of this challenge is not just changing the technology but also the processes, organizational structures, and enterprises to meet the new requirements of security.

Moreover, the need for greater information sharing in this net-centric environment is a challenge. The current culture of limited information sharing in which inter-organizational and intra-organizational information is strictly divided creates tension in a seamless information sharing infrastructure. In addition to that, the responsibility of generating, sharing, and utilizing useful information should be addressed in advance to avoid costly mistakes.

Verification and validation of NextGen deliverables is a major issue. The traditional systems engineering methods of verification and validation are tailored for testing an isolated system, while by definition a project like NextGen requires new methodologies of verification and validation beyond the scope of one system. The knowledge and experience of advancement in systems engineering in this area can be of priceless value for future projects.

Balance between human decision-making and automation is required to ensure a correct policy for increasing traffic and safety concerns. Changes in both human resource and technological facilities are required to effectively address this challenge.

The support of local communities is essential to facilitate development of the U.S. ATN and its physical infrastructure.

Communication, navigation, and surveillance systems in NextGen are going through major changes in terms of capacity and technology. However, planning required backups for them in case of any emergency is an area of challenge in developing NextGen.

The rise of Unmanned Aircraft Systems (UASs) provides significant opportunities for both military and commercial applications. However, integrating them into the NAS and developing policing and strategies for safe and secure use is a concern for the revolutionized U.S. ATN.

And finally realizing the benefits of NextGen is dependent on the critical mass of early adopters, similar to any technological advancement. Therefore, the NextGen project authority requires well-defined policies for motivating stakeholders' participation.

Systems Engineering Practices

The FAA NextGen is not just a revolution of the U.S. air transportation infrastructure, but also a shift in its enterprise. The enterprise architecture document, which is developed by JPDO, provides an overview of the desired capabilities (JPDO 2007b).

The enterprise architecture is described using Department of Defense Architecture Framework (DoDAF) and the Federal Enterprise Architecture (FEA). DoDAF is used to describe the operational aspects of the project. The three views of DoDAF, the Operational View (OV), the Systems View (SV), and the Technical Standards View (TV), are presented in the enterprise architecture document. The Overview and Summary Information (AV-1) is the formal statement about how to use the architecture, the Integrated Dictionary (AV-2) defines the terms in the document, the

Community Model (OV-1) presents a high level depiction of the NextGen community, the Operational Node Connectivity Description (OV-2) presents the information flow among operational nodes in the system, Operational Information Exchange Matrix (OV-3) details the description of information flow in OV-2. Other architectural views of the system based on DoDAF are the Activity Model (OV-5) which documents activities (functions and processes), the Operational Event/Trace Description (OV-6c) is a part of sequence and timing description of activities, the System Functionality Description (SV-4) explains system functional hierarchies, and the Operational Activity to System Functionality Traceability Matrix (SV-5) is specification of relationships between operational activities in architecture and functional activities. However, a challenging part of applying this Enterprise Architecture is transformation from legacy systems to the new NextGen. This transformation is the ultimate test for relevance and comprehensiveness of the developed Enterprise Architecture.

Acquisition is the heart of systems engineering activities in the FAA NextGen project. As mentioned in Challenges above, the current practice of verification of validation in systems engineering (SE) is geared toward single isolated systems, rather than a myriad of interconnected system of systems (sos). Moreover, the capabilities of NextGen are interdependent, and different programs rely on each other to deliver the promises. 250 unique and highly interconnected acquisition programs are identified in the FAA's Capital Investment Plan, and these are to be delivered by 1820 FAA acquisition professionals. In addition, program complexity, budget uncertainty, and the challenge of finding acquisition professionals present other problems. The experience of systems acquisition in NextGen can provide a useful knowledge for future similar projects.

Lessons Learned

Although major portions of the FAA NextGen project are technical transformations and physical infrastructure developments, the transformation in the aviation enterprise is important but to some degree neglected. Part of the issue might be the fact that this transformation is beyond the responsibility and capability of FAA. However, to accomplish NextGen's perceived benefits it is important to realize the effects of legacy systems, and most importantly the legacy enterprise architecture of the U.S. ATN. Many of the actual challenges in the system arose because of this inattention.

The sequestration in the U.S. government, the Budget Control Act of 2011, has cut the project funding substantially in recent years. As a result the project schedule and portfolio are subject to constant and wide-spread changes. The FAA was focused on delivering Optimization of Airspace and Procedures in the Metroplex (OPAM) program which is designed to reduce the delay, fuel consumption, and exhaust emission in busiest airports. The three areas of Houston, North Texas, and Washington D.C. were planned to complete the design phase on 2013 and start implementation.

Out of 700 planned ADS-B ground stations, 445 were operational on February 2013. ADS-B capability is a NextGen descendant of current radar systems and provides situational awareness for the players in the NAS using the Global Positioning System (GPS) and Wide Area Augmentation System (WAAS).

On the enterprise part of the project, the FAA Modernization and Reform Act of 2012 provided financial incentives for airlines and commercial aviation manufacturers to implement the required equipment in their aircraft. These incentives are designed to engage the air transportation community in the project and to create the critical mass of equipped airplanes.

There are considerable practices in applying NextGen. Establishment of the JPDO made the efforts of the project more coherent and integrated. JPDO's main responsibility is to coordinate development of NextGen. The role of this organization is to represent multiple stakeholders of the project, which enables it to resolve possible conflicts of interests inside one entity. Moreover, such an organization provides a venue for technical knowledge-sharing, creating a consensus, and making an integrated system.

Emphasizing delivery of the mid-term objectives of NextGen is another lesson of the project. It was a well-known practice documented by Forman and Maier to establish mid-points for complex projects (Forman 2000). Developing

a mid-level system provides the system designers an opportunity to examine their underlying assumptions, to identify best practices and heuristics in the context of the project, and to reapply the acquired knowledge through evolutionary developments. A major shift in the policy of FAA in recent years was to focus on delivering project mid-term objectives.

There are unique characteristics of NextGen which makes it a valuable case for learning and replicating to other complex transformation projects of sociotechnical systems. The scale of the project for infrastructure transformation is unprecedented. The system includes legacy systems and cutting edge technology, and its performance is based on their coherent work. The project implementation is dependent on involved participation of multiple governmental and commercial organizations. Moreover, this case-study provides a great investigation in enterprise governance and enterprise transformation beyond a single organization.

References

Works Cited

Darabi, H.R., and M. Mansouri. 2014. "NextGen: Enterprise Transformation of the United States Air Transport Network," in *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*, edited by A. Gorod et al. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.

Donohue, G.L., and A.G. Zellweger. 2001. "Air Transportation Systems Engineering." *American Institute of Aeronautics and Astronautics*. 193 (1).

FAA. 2008. *FAA's NextGen Implementation Plan*. Federal Aviation Administration. Washington, DC, USA. Accessed March 29, 2014. Available: https://www.faa.gov/nextgen/media/ng2008_implementation_plan.pdf.

Forman, B. 2000. "The political process and systems architecting," in *The Art of Systems Architecting*, 2nd ed., edited by M.W. Maier and E. Rechtin. Boca Raton, FL, USA: CRC Press LLC.

JPDO. 2007a. *Concept of operations for the next generation air transportation system*. Joint Planning Development Office. Accessed March 29, 2014. Available: https://www.jpdo.gov/library/nextgen_v2.0.pdf.

JPDO. 2007b. *Enterprise Architecture V2.0 for the Next Generation Air Transportation System*. Joint Planning and Development Office. Accessed March 29, 2014. Available: <https://www.jpdo.gov/library/EnterpriseArchitectureV2.zip>.

JPDO. 2011. *Targeted NextGen Capabilities for 2025*. Joint Planning and Development Office. Accessed March 29, 2014. Available: https://www.jpdo.gov/library/2011_Targeted_NextGen-Capabilities_for_2025_v3.25.pdf.

Primary References

Darabi, H.R. and M. Mansouri. 2014. "NextGen: Enterprise Transformation of the United States Air Transport Network." *Case Studies in System of Systems, Enterprise Systems, and Complex Systems Engineering*. Gorod, A., B.E. White, V. Ireland, S. J. Gandhi, and B.J. Sauser. Boca Raton, FL: CRC Press, Taylor & Francis Group. Scheduled for publication in July 2014. <https://www.taylorandfrancis.com/books/details/9781466502390/>. Accessed 29 March 2014.

Additional References

None.

Reverse Engineering a UAV Prototype using Agile Practices

- Lead Author:
- Phyllis Marbach

This example shows how Agile Practices were applied to an unmanned air vehicle (UAV) that had been developed as a prototype and was intended to be produced and marketed (Marbach 2012). At the time it was required to have the Federal Aviation Agency (FAA) approve the use of a UAV in populated areas. FAA required artifacts such as requirements, architecture representations and test procedures to grant this approval. A team was established to reverse engineer the artifacts needed from the operational prototypes being flown at the time. The test manager requested that the life cycle process to produce these artifacts be agile. Stakeholder needs were determined, and a plan was written to describe the problem and the process to be applied. Then the approach was presented to the team, the product backlog was developed, and a training and planning session was started.

Description

The agile process is described in the SEBoK here. When defining requirements (Carlson 2010) proposes that the requirements be identified, gathered, defined, and developed in iterations (or sprints). These requirements are written in the User Story format and controlled and managed in a product backlog. The User Stories in the Product backlog are selected and estimated by the team based on importance and need. The most important user stories (or requirements) are prioritized and moved to the top of the product backlog. Then those User Stories are broken into tasks and the tasks are estimated. The number of tasks that the team can complete are then put into the Sprint Backlog (or iteration backlog) and those tasks are then worked on by the team. For this work the UAV and its code were already operational. We had working code, a test bed, user interfaces and user procedures. The goal was to produce requirements documentation, architecture and design diagrams, a trace matrix of tests to requirements, software test descriptions and a Hazard Analysis to take before the FAA.

The team assembled were experienced engineers, but not with this UAV system. There were UAV subject matter experts (SMEs) still on the program, but they were not always available to answer questions or come to reviews. There was existing documentation in program repositories such as charts and operator procedures, but we did not know where to find this information. Given these challenges it was decided to use collaborative tools to manage the information as it was discovered and make it visible to the team and the UAV SMEs.

The first set of information produced was the product backlog. An example of the product backlog is shown in Figure 1. Epics are a set of User Stories that take more than one iteration to complete. A user story is broken into tasks such as those shown in Figure 2. These templates were developed to understand the scope of each task and what the definition of done for that task was. The goal was to identify tasks that take a maximum of 16 work hours to complete. The product backlog was developed and managed in an Application Lifecycle Management (ALM) Tool. Our team did a trades study when selecting a tool for use. Parameters considered in the trade included ease of use, cost, and features of the tool itself. The tool selected was VersionOne.

The team then determined what collaboration tool would be used to make our discovered information visible. The requirements for this tool were: easy to access, easy to use, easy to comment on and easy to change. At the time these tools were available: Mediawiki, an open source, TWikiTM, another open source, Confluence, SharePoint, and Socialtext. It was decided to use TWikiTM.

For each of the epics in work, such as "Power On", the Description of Functionality was written in the collaboration tool. Figure 3 shows the list of content in the collaboration tool for the artifacts being developed.

- **30 Epics were created from the User Interface Features, examples:**
 - Power On
 - Start Up Feature
 - Shutdown Feature
 - Operate Component
 - Operate Another Component
- **Product Owner prioritized the most important ones**
- **Each epic has 5 significant backlog items (took 3 iterations to reach these 5):**
 - Functional Analysis
 - Requirements
 - Hazard Analysis
 - Draft Test Procedure
 - Finalize Test Procedure

Figure 1. Example Product Backlog for an Unmanned Air Vehicle (UAV) (Marbach 2023, used with permission)

Backlog Item Templates	
Filter ▼	
Move to Project ▼	
Title	ID
User Story Template - OLD	B-01017
Task Template - OLD	B-01018
Update Documentation or Work Products Template	B-01243
Functional Analysis Template	B-01225
Research and Document Functionality	TK-02479
Requirements Template	B-01216
Generate Functional Requirements	TK-02420
Peer Review Requirements	TK-02422
Update and Post Requirements	TK-02469
Hazard Analysis Template	B-01219
Identify and Analyze Potential Hazards	TK-02546
Peer Review Hazard Analysis	TK-02547
Update Hazard Analysis	TK-02548
Draft Test Procedures Template	B-01252
Generate Draft Test Procedures	TK-02543
Peer Review Draft Test Procedures	TK-02544
Update and Post Draft Test Procedures	TK-02545
Finalize Test Procedure Template	B-01251
Run Test Procedures	TK-02539
Update and Post Finalized Test Procedures	TK-02540

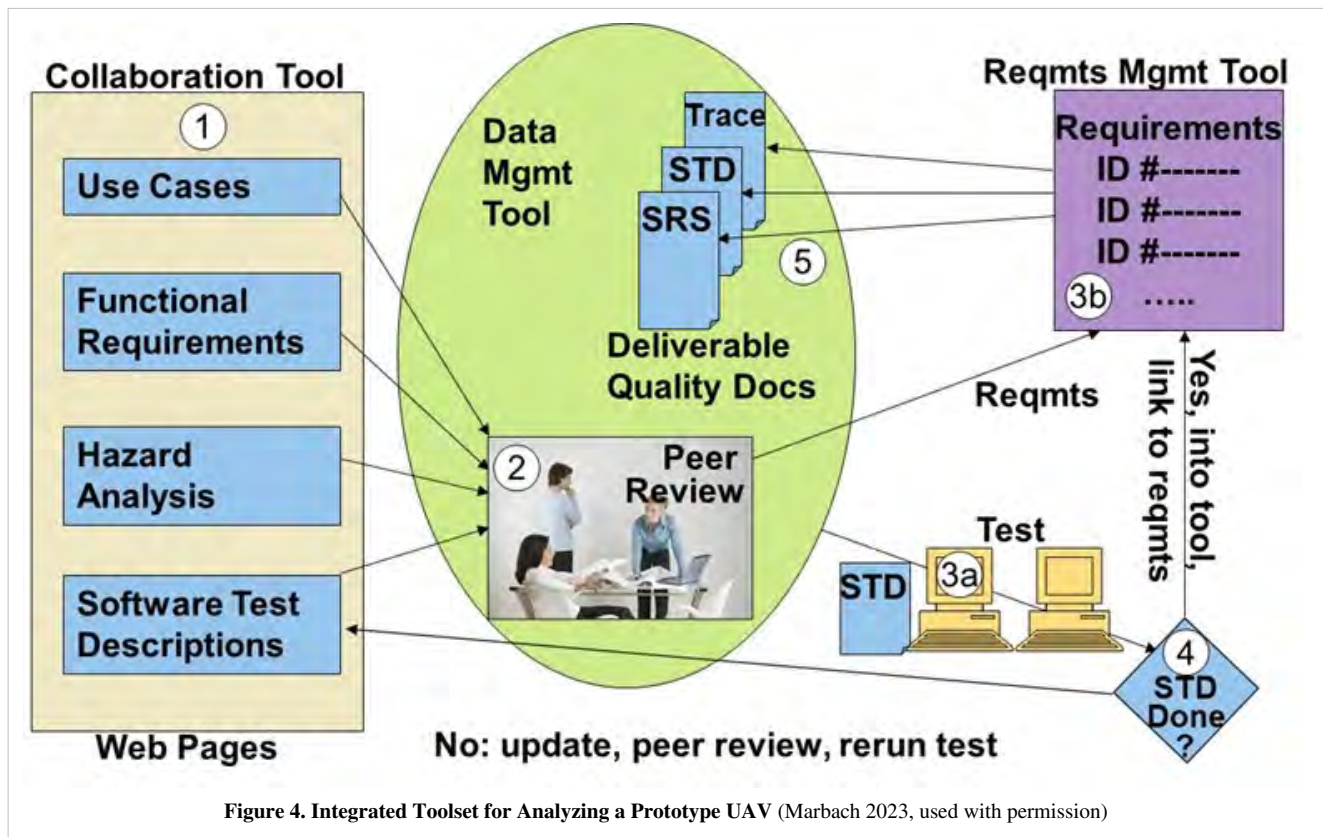
Figure 2. User Story Templates and Task Templates for Consistent Development (Marbach 2023, used with permission)

- ↓ [Description of Functionality](#)
 - ↓ [Overview](#)
 - ↓ [Functional Decomposition](#)
 - ↓ [Use Case Development](#)
 - ↓ [Phase 1 level](#)
- ↓ [Requirements](#)
 - ↓ [Use Case Development](#)
 - ↓ [Phase 1 \(operator/functional\) level](#)
 - ↓ [Functional Requirements](#)
 - ↓ [Requirements Documents](#)
 - ↓ [SRS Document in TWiki](#)
 - ↓ [SRS Document in DOORS](#)
 - ↓ [SRS Document in PIMS](#)
- ↓ [Test Procedures](#)
 - ↓ [Existing Test Procedures](#)
 - ↓ [FQT Team Test Case/Test Procedure Development](#)
 - ↓ [Test Cases](#)
 - ↓ [Test Procedure Document](#)
 - ↓ [Expected Test Results](#)
 - ↓ [Test Procedures to Requirements Trace](#)
 - ↓ [Software Test Description \(STD\)](#)
- ↓ [Test Results](#)
- ↓ [Hazard Analysis/Risk Mitigation](#)
 - ↓ [Hazards/Mitigation](#)

Figure 3. Collaboration Tool Table of Contents for a system being analyzed (Marbach 2023, used with permission)

The Collaboration Home Page had an introduction about the analysis underway. It had links to a list of functional threads that were links to the work products themselves. There were also links to the references used, links to the test environment information and links to templates for the work products with instructions. The work products being developed, as shown in Figure 3, were Collaboration Tool Templates, Functional Descriptions, Requirements including Use Cases, Hazard Analysis and Risk Mitigation, and Test Procedures including Test Cases and Test Descriptions.

Once the requirements were complete for one Epic, such as “Power On” the material in the Collaboration Tool was exported into a Word Document and that was parsed into a Requirements Management tool. This team used the Dynamic Object-Oriented Requirements System (DOORS). The final Software Requirement Specification (SRS) was created from DOORS. After peer review the release documents were baselined into the Data Management Tool Repository that provided Configuration Management control. The Integrated Toolset used for this project is shown in Figure 4.



The sequence of development started with the prioritized user story to be worked first. Then the Collaboration tool was used to capture the information that members of the reverse engineering team worked with SMEs to reach an understanding of the functional requirements, hazard analysis and software test descriptions for the user story in development. A formal peer review was conducted and when agreed it was ready those documents were parsed into the requirements management tool. The Software Test Description (STD) was used to test the UAV code using the test platform. If the STD is determined to be complete, then it is also parsed into the requirements management tool and traced to the requirements that have been verified by testing. From the requirements management tool formal artifacts such as the Software Requirement Specification, the Software Test Description and the Trace Matrix were produced. Those were put into a configuration management tool. If the STD used for the testing was not Done then any markups were made into the Collaboration Tool and the Peer Review was conducted again. Essentially, each iteration resulted in potentially deliverable products that could be delivered to a stakeholder.

The data management tool, shown in Figure 4 by the green oval contained a repository of draft folders, peer review records, action items created, tracking and closure, a repository of release folders, the calendar, meeting notifications, distribution lists, access control to records, configuration management workflow and approvals, and it provided collaboration across companies, subcontractors, and customers.

The documentation created included software requirements specification that was created epic by epic rather than all at once, software test descriptions created as each feature is analyzed, and a Hazard Analysis being performed one epic at a time. These documents were updated each increment. An increment is a set of iterations. Each backlog item included conducting peer reviews of the content, as shown in Figure 2 list of tasks of each user story. Records of the peer reviews were maintained in the data management tool as mentioned above. The definition of done for these artifacts was that the work was not complete until the information was posted into the Requirements Management Tool. The Software Test Descriptions were linked to the requirements in the Requirements Management Tool thus beginning the Trace Matrix.

The Agile Practices described in this article can be mapped to LEAN Disciplines as shown in Table 1.

Table 1. Agile Practices Drive LEAN Disciplines (Used with Permission)

LEAN Disciplines	Agile Requirements Analysis
1. Establish Clear Priorities	1. Product backlog is always prioritized; Team works on highest priority items first
2. Eliminate Bad Multitasking – Focus and Finish	2. Team is shielded from interruptions that cause bad multitasking
3. Limit the Release of Work in Process (WIP) to Deliver Earlier	3. Tasks are pulled from the iteration backlog one at a time to limit individual WIP
4. Prepare! Start to Finish	4. Requirements are not selected from the product backlog until everything needed is available
5. Use Checklists to Prevent Defects and Traveled Risk	5. Checklists and guides are used to prevent costly rework
6. Face into and Resolve Issues Quickly	6. Daily stand-up meetings force issues and risks to be identified and resolved quickly
7. Drive Daily Execution	7. Daily stand-up meetings drive team-based execution

Summary

Systems engineering best practice is to perform requirements analysis, verification and validation as a system is being developed. Artifacts are created and configuration controlled as the system matures. This example describes how an existing operating prototype could be transitioned to a production system by performing requirements analysis, risk mitigation and hazard analysis even after the prototype is developed and operational. There is value in performing these system engineering tasks for an existing prototype to verify it is safe to operate and to achieve approval to fly. Using iterative and incremental development of these artifacts limited the work in process (WIP). The whole team worked one epic at a time to produce artifacts that addressed that one epic and verified the requirements, testing and analysis of hazards relative to that one epic, such as “Power On”. Then they would work the next epic focusing on one capability at a time therefore reinforcing each other’s work quite effectively.

Lessons Learned

- Developing Systems Engineering products such as Systems Requirements Specifications, Hazards Analysis, Test *Procedures, and Verification Trace Matrix in an iterative incremental way is effective.
- Some new to using these principles and methods did resist at first but then saw the value and became advocates of the iterative incremental process.
- The use of tools helped keep the work visible, aiding in communication and accuracy.
- Access to Subject Matter Experts (SMEs) was critical to producing accurate products.
- The team focused on known elements first. Then the knowledge learned was applied to elements with more uncertainty. This applies the Lean Principle of limiting the work in process.
- The team did not start work on an element until they had what was needed to accomplish the analysis. This applies the Lean Principle of working start to finish.

References

Works Cited

Carlson, R., Matzuc, P., 2010, "A Viable Systems Engineering Approach", Proceedings of the Systems and Software Technology Conference (SSTC), 2010, Salt Lake City, Utah, USA.

Marbach, P., 2012, "An Experience Report on Agile Systems Engineering Requirements Analysis," Proceedings of the INCOSE-LA Mini-conference, 2012, Los Angeles, CA, USA.

Primary References

None.

Additional References

None.

UK West Coast Route Modernisation Project

- Lead Authors:
 - Tom Hilburn, Heidi Davidz, and Alex Lee
- Contributing Authors:
 - Alice Squires and Richard Turner

This example was created as a SE example directly for the SEBoK. It describes the United Kingdom West Coast Main Line railway project and some of the problems which occurred on this project before the implementation of SE. It also discussed the value of applying some aspects of SE, even if this is done later in the project.

This example is based on information from a report by the UK National Audit Office (NAO 2006). It also uses an INCOSE publication on systems engineering case studies (INCOSE 2011) to help structure its conclusions.

Description

The West Coast Main Line (WCML) is a principal United Kingdom (UK) railway artery serving London, the Midlands, the North West and Scotland. The Line is responsible for over 2,000 train movements each day, with more than 75 million rail journeys made each year on the route. It accounts for 43% of Britain's UK freight market (Railway People 2011). In 1998, the British government embarked on a modernization program called the West Coast Route Modernisation (WCRM) project, to carry out a significant volume of modernization work between 1998 and 2008, delivering increased capacity and reduced journey times as well as replacing worn-out parts of the railway. It was a challenging job involving 640 kilometers of track—much of which was incapable of carrying high-speed rail cars. Some sections were seriously dilapidated, and new trains would require a complete overhaul of signaling, power supply, and switching systems.

Early on, the WCRM upgrade had serious problems. A major complicating factor was the introduction of a new signaling technology that was designed to allow improved services for new trains running at 140 miles per hour. By 2001, neither the rail infrastructure upgrade nor the new trains were on course for delivery as expected in the 1998 agreement. By May 2002, the projection of the program's final cost had risen from £2.5 billion (in 1998) to £14.5 billion but had delivered only a sixth of the original scope.

In January 2002, the UK Secretary of State instructed the Strategic Rail Authority (SRA) to intervene and find a way to renew and upgrade the WCML. An SRA analysis identified the following issues:

- The program lacked direction and leadership before 2002.
-

- The project did not have a delivery strategy and there was no central point for responsibility and communication.
- There was a lack of openness and communication regarding the program with interested parties before 2002 and there was a lack of stakeholder management.
- Scope changes arose because WCRM did not have an agreed-upon specification that matched required outputs with inputs.
- There was inadequate knowledge about the West Coast asset condition.
- Technology issues related to the decision to replace conventional signaling with unproven moving block signaling introduced major risk into deliverability and cost before 2002. These technology issues caused scope changes and program delay.
- Project management (PM) was weak, with a lack of senior management skills, too many changes in personnel, and ill-defined and fragmented roles and responsibilities. There was no integrated delivery plan and there was limited oversight of contractors. Poor management of contracts added to costs.

In order to remedy the situation, the SRA initiated the following actions, which align with generally accepted systems engineering (SE) practice:

- A clear direction for the project was developed and documented in the June 2003 West Coast Main Line Strategy, specifying desired goals and outcomes.
- A clear, measurable set of program outputs was established, along with more detailed infrastructure requirements, which were then subject to systematic change control and monitoring procedures fixing scope. Contractors were invited to tender complete detailed designs and deliver the work to a fixed price.
- Clear program governance structures were instituted.
- The SRA consulted widely with stakeholders and, in turn, kept stakeholders informed.

A National Audit Office (NAO) report concluded that the new arrangements worked well and that there were benefits to this approach (NAO 2006). Until this time, one of the program's key constraints and cost drivers had been the ability to access certain areas of the track. The new approach facilitated the ability to obtain possession of the track for engineering work, which was crucial to delivery. The new approach also enabled the program to identify opportunities to reduce the total cost by over £4 billion.

The NAO report also discussed a business case analysis by the SRA that identified the following benefits (NAO 2006):

- benefit:cost ratio for the enhancements element was 2.5:1;
- journey times and train frequencies exceeded the targets set out in the 2003 West Coast Strategy;
- growth in passenger numbers exceeded expectations (e.g., by 2005-06, following Phase 1 of the West Coast program, annual passenger journeys on Virgin West Coast grew by more than 20%); and
- punctuality improved (e.g., by September 2006, average time delays on Virgin West Coast trains have been approximately 9.5 minutes, a 43% improvement on the average delay of 17 minutes in September 2004).

The WCRM problems could be associated with a number of systems engineering concepts and practices: stakeholder needs definition, planning, analysis of risks and challenges of new technology and associated risk management, decision management, configuration or change management, information management, and management oversight.

Summary

The WCRM project illustrates that when SE concepts and practices are not used or applied properly, system development can experience debilitating problems. This project also demonstrates how such problems can be abated and reversed when SE principles and methods are applied.

References

Works Cited

INCOSE Transportation Working Group. *Systems Engineering in Transportation Projects: A Library of Case Studies*, version 1.0. Seattle, WA, USA: International Council on Systems Engineering. March 9th, 2011.

NAO. *The Modernisation of the West Coast Main Line, Report by the Comptroller and Auditor General*. London, UK: National Audit Office. November 22, 2006. HC 22 Session 2006-2007.

Railway People. "West Coast Route Modernisation." RailwayPeople. 2011. Accessed July 25, 2011. Available at: <https://www.railwaypeople.com/rail-projects/west-coast-route-modernisation-3.html>.

Primary References

None.

Additional References

None.

Standard Korean Light Transit System

- Lead Authors:
- Heidi Davidz, Alice Squires, and Chuck Calvano
- Contributing Author:
- Richard Turner

This example was created as a SE example directly for the SEBoK. It deals with systems engineering (SE) concepts and guidelines applied to the development of the Standard Korean Light Transit System (SKLTS). In Korea, local authorities had historically been interested in light transit to help resolve their transportation problems. The SKLTS was a joint effort between local authorities and the central government. It was built to provide a standard platform on which any local authority could construct its own light transit system. The issues of stakeholder needs definition, safety, and reliability, availability, and maintainability were critical to the success of this system.

Description

The elements of the SKLTS were classified into four groups (as shown in Figure 1): trains, signal systems, electric and machinery (E&M) systems, and structures. Trains and vehicles were to be automatically operated, without need for human operators. Operation systems and their interfaces were based on digital signals and communications. For SKLTS, SE-based design activities focused on reliability, availability, maintainability, and safety (RAMS), and were integrated into project management (PM) activities during all phases.

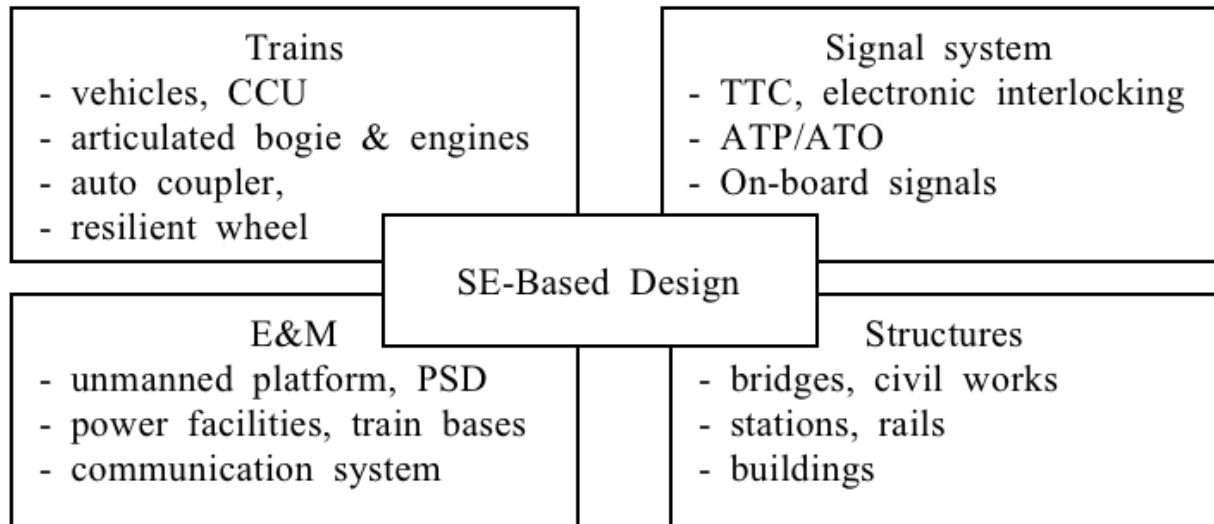


Figure 1. Subsystems of the SKLTS (Ahn, 2005). (Notes: CCU: Central Control Unit; TTC: Total Traffic Control; ATP: Automatic Train Protection; ATO: Automatic Train Operation; PSD: Platform Screen Door) Reprinted with permission of *Journal of the Korean Society for Railway*. All other rights are reserved by the copyright owner.

The project life cycle for the SKLTS is summarized in Figure 2. It consisted of 7 phases: concept studies, concept development, preliminary design, design, system production and testing, performance evaluation, and operation/maintenance/close-out (OMC) - please see (Choi 2007) and (Chung et al. 2010) for further details. These phases, with the exception of the production and test phases, are completed through an evaluation and decision point (EDP) (milestone), depicted as a colored circle in Figure 2. These EDPs correspond to common life cycle artifacts such as requests for proposal (RFPs), proposals, preliminary design reviews (PDRs), and critical design reviews (CDRs).



Figure 2. 7 phases of the SKLTS development (Ahn 2005). Reprinted with permission of the *Journal of the Korean Society for Railway*. All other rights are reserved by the copyright owner.

During the SKLTS development, SE activities were focused on RAMS as summarized in Table 1.

Table 1. The SE Framework of the SKLTS (Ahn 2005). Reprinted with permission of the *Journal of the Korean Society for Railway*. All other rights are reserved by the copyright owner.

Phases	Safety	Reliability	Function	Performance
Concept studies	Requirements analysis	- Identifying RAM conditions - RAM allocation	- System configuration - Interface management	Performance simulation
Concept development & pre-design	- Safety planning - Defining safety procedures & levels	- RAM planning - Initial availability analysis	- Defining scenarios and alarm procedure - Pre-designing command rooms	Interface analysis

Design	• Hazard log • Safety case analysis • Risk analysis	• Reporting RAM analysis • RAM analysis of auxiliary systems	• Defining alarm systems • Train analysis • Functionality analysis of stations	• Interface analysis
Performance evaluation	• Safety test planning & testing	• Verification and Validation (V&V) RAM • Maintainability test	• System test planning and testing	• Performance test planning & testing
Initial Operation	• System acceptance • Driver certification	• RAM monitoring • FRACAS*	• Analyzing systems • Identifying improvement points	• Performance monitoring

*FRACAS: Failure Reporting & Corrective Action System

In the "concept studies" and "concept development" phases, requirements included the RAMS objectives. Planning activities in this phase included the scheduling of various tests and evaluations to be conducted after system design. The basic layout of rails and command rooms was also proposed. Finally, it was during this phase that interface management procedures and relationships between requirements and systems were defined. For RAMS engineering, it was also important to establish associated plans and criteria (e.g., RAM plans, safety plans, service availability, etc.).

During the pre-design phase, the basic architecture of the system was determined for safety planning, RAMS planning, and operational scenarios. Interfaces among subsystems, management procedures for contractors and legal regulations were defined. were defined as well as management procedures for contractors and legal regulations. The functional analysis dealt with timeline, accuracy of stop points, and trip times. Pre-design activities also included the specifications of major system elements such as signal systems, trains, and interfaces. For RAMS engineering, safety scenarios were defined, and the hazard and risk analyses were performed.

During the design and performance evaluation phases, hazard log and RAMS analyses were performed to ensure that each subsystem met safety requirements. The specifications of alarm systems and stations were also defined. In addition, V&V and test procedures were determined for performance evaluation. During the design phase, a design/construction interface manual (D/CIM) was developed and applied to ensure integrated and consistent design. (Bombardier, 2005.)

Because SKLTS was designed as an automatically-driven system, RAMS issues were critical to its success. The safety and reliability of the SKLTS were evaluated on a test railway that was constructed to standard specifications. Data was gathered from existing Korean light rail systems, as well as the light rail systems from other countries, to support V&V activities.

Various methods were applied for achieving the RAMS objectives, including RAMS requirements analysis, safety and RAMS planning, utilization of systems scenarios, and construction risk analysis.

Initial operation of SKLTS was allowed only after the system was formally accepted and operators were properly certified. During test operation, RAMS performance was continuously monitored and system scenarios were used successfully to evaluate the dynamic behavior of the system. A failure reporting and corrective action system (FRACAS) was used to gather accident and failure data. Continuous improvement when the system is in normal operation was identified as a requirement; the results from the FRACAS will be used to support improvement of the system, maintenance, and improvement of procedures.

Summary

Korean local authorities have successfully introduced the SKLTS to their precincts with some modifications. Successful examples include the Inchun Airport Line and the Seoul 9th Subway Line. One lesson learned that was identified was that requirement analysis, especially in the first few phases, should have been more complete.

References

Works Cited

- Ahn, S.H. 2005. "Systems Engineering Applied to the Construction of Unmanned Light Rail Transit Systems." *Journal of the Korean Society for Railway*. 8(2): 41-49.
- Bombardier. 2005. *Design/Construction Interface Manual*. Montréal, Québec, Canada: Bombardier.
- Choi, Y.C. 2007. "Systems eEngineering aApplication mModel for the nNational R&D pProject: Focusing on the rRailway sSystems." Ph.D. dissertation, Ajou University, Suwon, South Korea, 2007.
- Chung, S. Y., S.G. Lee, D.W. Lee, and S.T. Jeon. 2010. "A sStudy on tThe aApplication mModel of sSystems eEngineering to Aadvance the bBusiness of the Light Rail Transit (LRT)." *Proceedings on the Autumn Conference of the Korean Society for Railway*, p. 24-30.

Primary References

None.

Additional References

- Han, S.Y. and A.H. Lee. 2005. *System Engineering for The Technology Development Program of Light Transit Systems: Internal Research Report*. Gyeonggi-do, South Korea: Korea Railroad Research Institute.
- Korean Agency for Technology and Standards. 2009. *KSX ISO/IEC 15288: Life Cycle Processes of IT Systems Based on Systems and Software Engineering, Research Report*. Gyeonggi-do, South Korea: Korean Agency for Technology and Standards.
- Lee, W.D. 2002. "A study on the top-level functional analysis for the automated guideway transit by system engineering tools." *Proceedings of the Autumn Conference of the Korean Society for Railway*, p. 202-207.

Utilities Examples

Northwest Hydro System

This case study presents the Northwest Hydro System (NwHS), which is situated in the northwestern United States (the Northwest). NwHS comprises a large number of loosely coupled autonomous elements (hydroelectric dams) that operate in a complex environment of social, regulatory, and ecological contexts. It is instructive to note that the NwHS is characterized as a project that is concerned with planning, developing, and maintaining a hydroelectric system that has evolved, and continues to evolve, over time. Each of the hydroelectric dams within the NwHS is also referred to as a project, which indicates that the individual elements of the NwHS are also evolving over time (FWEE 2016).

Background

As is shown in this case study, NwHS is an adaptive and reconfigurable system that exists within a framework of policies, rules, regulations, and agreements. The NwHS is analyzed using each of the four provisioning paradigms in SEBoK Part 4: product system engineering; service system engineering; enterprise system engineering; and system of systems engineering.

NwHS encompasses the Columbia River and its tributaries. The headwaters of the Columbia are in the Rocky Mountains of British Columbia, Canada. The river flows south into the United States and then east to west; it forms the north-south border between the states of Washington and Oregon in the U.S. and empties into the Pacific Ocean near Astoria, Oregon. The Columbia River drainage basin (Columbia and tributaries) is roughly the size of France and extends across seven U.S. states and British Columbia (Col 2016).

The Columbia River has 14 hydroelectric dams (hydro dams) on its main stem. In total, there are more than 250 hydro dams in the Columbia River Basin, each of which has a generating capacity of five or more megawatts of electricity. The NwHS produces approximately one-third of all hydroelectric power generated in the United States - more than any other North American hydroelectric system. The amount of electrical power generated by the NwHS fluctuates between 50% and 75% of all electricity used in the Northwest; other sources include coal, natural gas, nuclear, wind, and solar. Excess electrical energy generated by NwHS is sold to other electrical grids. There are more small hydro dams than large dams. Small hydro dams have a generating capacity of 100 kilowatts to 30 megawatts; large hydro dams have a capacity of more than 30 megawatts. In addition, there are numerous micro dams that can each generate fewer than 100 kilowatts. Most micro dams provide power to isolated homes or small communities but some are elements of the NwHS and sell power to utilities (DOE 2016). Utility companies own some of the dams, some are locally and privately owned, and some are owned and operated by federal agencies.

The Bonneville Power Administration (BPA) provides about one-third of the electrical power generated by NwHS and used in the Northwest; BPA is a federal nonprofit agency – it is part of the U.S. Department of Energy. It is self-funded and covers its costs by selling electrical power generated by 31 federal hydro dams in the Columbia River Basin. The U.S. Army Corps of Engineers and the Bureau of Reclamation operate the dams.

The Bureau of Reclamation also operates a network of automated hydrologic and meteorologic monitoring stations located throughout the Northwest. This network and its associated communications and computer systems are collectively called Hydromet. Remote data collection platforms transmit water and environmental data via radio and satellite to provide near-real-time water management capabilities. Other information, as available, is integrated with Hydromet data to provide timely water supply status for river and reservoir operations (USBR 2016).

Distinguishing characteristics of individual hydro dams are their capacity to generate electrical energy and their physical structure. Several factors contribute to the differences in generating capacity of hydro dams: the number of turbines/generators; the flow rate of a river or tributary; the amount of elevation that water falls in order to spin the turbines; and environmental factors such as providing for fish passage and regulating water flow to provide irrigation water and maintain downstream ecosystems.

Dam structures include storage dams, run-of-river dams, and pumped storage dams (DOE 2016). A storage dam retains water in a reservoir for future use. A run-of-river dam harvests the energy in a river as it flows through the dam but does not impede the flow. Pumped storage dams use generated electricity to pump water back up to a storage reservoir during times of low demand for use during times of high demand. Pumped storage dams are not common in the NwHS.

Purpose

NwHS has three primary goals (NwHS 2016):

1. To provide most of the Northwest's "firm-energy" needs.
2. To maximize "non-firm" energy production.
3. To maintain the ecological environment.

Firm energy is the amount of electricity the Northwest will need each year. Planners rely on NwHS and other sources to produce enough firm energy to ensure that sufficient electricity will be generated to meet estimated needs (energy sources include hydroelectric, nuclear, coal, natural gas, solar, and wind). NwHS "firm energy" is the amount of electricity that can be generated by NwHS when the amount of available water is at a historical low, thus guaranteeing the amount of energy the NwHS can provide.

Non-firm energy is the electricity generated when the annual hydrologic cycle makes more water available for power generation than in a historically low-water year. Non-firm electricity generated by hydro dams is generally sold at a lower price than the alternatives of electricity generated by nuclear, coal, or natural gas thus making it more attractive to customers. Excess non-firm electricity is also sold to interconnected regional grids when the demand on those grids exceeds supply.

Other goals for NwHS include flood control, navigation, irrigation, and maintaining the water levels of all reservoirs.

Challenges

The NwHS is a large, complex system that has many challenges to be met.

NwHS hydro dams have varying design details (e.g., the types of turbines, generators, control systems, and fish passage facilities used). This makes routine maintenance, retrofitting, and other sustainment issues unique for each dam.

Safety and security (both physical and cyber) are common challenges for all dams; cyber security is a growing concern. Smaller dams, having fewer resources, may be more susceptible to cyber attacks than larger ones. It has been reported that on 12 occasions in the last decade hackers gained top-level access to key power networks (HLS 2010) (Cyber 2015).

Run-of-river hydro dams do not store water. They may not be able to meet their firm energy commitments when rivers are lower than anticipated and flowing slowly.

The ways in which electricity generated by a dam is transmitted and sold to utilities and large industrial customers varies widely. For instance, the Bonneville Power Administration transmits and sells power generated by federal dams. Non-federal operators must manage transmission and sale of power produced by their dams.

Depending on factors such as size, structure, location, and ownership of each dam, a large number of policies, regulations, and agreements have dramatically different effects on how dams are operated.

Some of the most contentious environmental issues are associated with maintaining the ecology of the rivers, preserving salmon and other fish, and providing sufficient irrigation water while preserving sufficient reservoir water to meet firm-energy demands (Speakout 2016).

Preserving the salmon population endangered by dams is a continuing challenge. Salmon populations have been depleted because dams impede the return of salmon to upstream spawning beds. Native Americans advocate for their traditional fishing rights, which conflict with governmental policies intended to maintain healthy salmon populations in the Columbia River Basin (Impact 2016). The National Marine Fisheries Service has recently declared that salmon recovery is a higher priority than all other purposes except flood control at 14 federal dams.

Systems Engineering Practices

The Northwest Hydro System is a large, complex system composed of loosely coupled autonomous elements; each dam operates semi-independently within a large network of similar entities and contextual constraints. The NwHS evolves over time: some dams have been retrofitted to increase power generation capability or to reconfigure connections to electrical transmission lines; new dams have been constructed; and some existing dams have been decommissioned and removed.

Human elements of NwHS include: operators; maintainers; regulators; and inspectors. Others are suppliers to NwHS (vendors and contractors); some humans are users of the electricity generated by NwHS (businesses and home owners); and some are stakeholders who depend on and are impacted by the NwHS (utilities, large industries, farmers, ranchers, homeowners, ecology advocates, Native Americans, towns).

The context of NwHS includes natural elements (rivers, terrain, weather systems, fish); elements purposefully built by humans (transmission lines, electrical grids); cyber connections (both wired and Internet); and rules, regulations, and agreements at the federal, regional, state, and local levels.

Given the complexity of the NwHS and its context, it is instructive to analyze the NwHS by applying each of the four application paradigms of systems engineering presented in SEBok Part 4: the product, service, enterprise, and system-of-systems application paradigms.

Product System Provisioning

Product system provisioning applies systems engineering processes, methods, tools, and techniques to conceive, develop, and sustain the purposefully developed elements of a system (e.g., a hydro dam or a hydro system). In addition, some of the naturally occurring physical elements of a system may be shaped and configured (e.g., a river channel).

Major product elements of a hydro dam include the physical structure of the dam (including the spillway), the penstock (used to direct water into the turbines), the generating plant (i.e., the turbines used to turn generator rotors, generator stators and rotors that generate the electricity, step-up transformers used to increase the voltage level of electricity produced by the generators, and connections to transmission lines).

Cyber elements sense, measure, regulate, and control water flow, power generation, safety, security, and the structural integrity of the dam. Some turbines, for example, have adjustable vanes that are controlled to harvest maximum energy from the water, depending on the flow rate, power demand, and other factors. The cyber elements include: computing devices; supporting software (operating systems, databases, spreadsheets); data management software (collection, analysis, reporting); application software (displays of monitored status and interfaces for controlling operation of a dam); and communication interfaces to wired linkage and Internet-enabled links. In addition, software support is provided for the analog and digital devices needed to sense, measure, regulate, and control the purposefully built and naturally occurring elements of a dam and its environment.

Product system provisioning is also concerned with other issues that apply to individual dams, elements of dams, and the overall NwHS. They include issues such as: manufacturability/producibility; logistics and distribution; product

quality; product disposal; conformance to policies, laws, regulations, agreements, and standards; value added for stakeholders; and meeting customer's expectations. Many different technologies and engineering disciplines are needed to develop and sustain a hydro dam and the overall Northwest Hydro System. Product system provisioning can provide the coordination and control of systems engineering needed to develop, reconfigure, adapt, analyze, and sustain the hydro dams and the NwHS.

Service System Provisioning

A service is an activity performed by an entity to help or assist one or more other entities. Service system provisioning can be applied within the various contexts of services provided by the NwHS to meet stakeholders' requirements, users' needs, and system interactions with operators, users, and maintainers, plus the interactions with the contextual elements that determined services provided by the NwHS in the social, business, regulatory, and physical environments.

The NWHS provides electricity to a grid that serves commercial, industrial, governmental, and domestic customers. Stakeholders in addition to customers served include those who affect or will be affected by development, operation, and sustainment of a dam. Downstream stakeholders served, for example, include: Native Americans, farmers and ranchers, and communities that receive the service of water released by the dam.

Additional service attributes include: the services that enable operators and maintainers to efficiently and effectively operate and maintain the physical and cyber elements of a dam; release water from the dam in a manner that services the upstream and downstream ecosystems; manage sharing of electrical power with other regional grids; provide emergency responses to power demands that result from electrical brownouts, blackouts, and overloads; and handle system failures that might be caused by earthquakes, terrorist attacks, and other catastrophic events.

Enterprise System Provisioning

An enterprise, such as the NwHS, consists of one or more organizations that share a mission, goals, and objectives to offer an output such as a product or service. The mission and goals of the NwHS are to provide most of the Northwest's firm energy needs and to maximize non-firm energy production while serving stakeholders and preserving affected environmental ecosystems. To meet those goals, the Northwest Hydroelectric Association (NWEA) coordinates the planning, design, improvement, and operation of the hydro dams that constitute the NwHS enterprise.

NWEA members represents all segments of the hydropower industry – independent developers and energy producers; public and private utilities; manufacturers and distributors; and local, state and regional governments including water and irrigation districts. Other NWEA members include contractors, Native American tribes, and consultants: engineers, financiers, environmental scientists, attorneys and others (NWEA 2016).

Note that an enterprise may consist of multiple organizations that are engaged in a common endeavor. The NwHS is a large complex enterprise that has many constituent organizations; namely, the organizations that own and operate the hydro dams and the other stakeholder members of the NWEA. Differences in ownership, structure, location, and size of hydro dams, the special interests of various NWEA members, and a complicated regulatory process, are some of the distinguishing characteristics of the NwHS that can be analyzed by enterprise systems provisioning.

System of Systems Provisioning

Many systems are composed of autonomous elements that are combined to provide increased capabilities that cannot be provided by the elements operating in isolation. The Northwest Hydro System is a system of systems comprised of autonomous hydro dams that have different owners, different operators, different stakeholders, and different regulators. The autonomous hydro dams could not provide the NwHS capabilities without the overall coordination and control that can be managed by applying system of systems provisioning.

Lessons Learned

NwHS is a collection of many interrelated ongoing projects that have shared common goals and shared constraints. The unique characteristics of NwHS make it a useful case study to illustrate how the four provisioning paradigms in SEBoK Part 4 provide essential viewpoints for analyzing large complex systems comprised of loosely coupled, autonomous elements.

Product systems engineering allows the collection of physical and purposefully built NwHS elements and their interconnections to be analyzed by applying systems product engineering processes and methods.

Service systems engineering supports analysis of the NwHS services provided to customers, users, farmers, ranchers, Native Americans, and other stakeholders who rely on NwHS for those services.

Enterprise systems engineering considers the broad scope and impact of the NwHS enterprise, both positive and negative, on the northwestern United States within the context of economic, social, physical, and regulatory environments.

System of Systems engineering applies the principles of planning, coordination, and operation to a collection of semi-autonomous hydro dams that form the Northwest Hydro System. The complexity of adding new dams as well as modifying and decommissioning existing dams in a seamless manner can best be understood by applying system of systems engineering processes and methods.

Taken together, the four provisioning paradigms in SEBoK Part 4 present a comprehensive view of a very large complex system whose many dimensions would be otherwise difficult, if not impossible, to comprehend when the NwHS is examined using only one of the paradigms.

References

Works Cited

- Col. 2016. *Columbia River*. Available at https://en.wikipedia.org/wiki/Columbia_River. Accessed February 15, 2016..
- Cyber. 2015. *US in Fear of New Cyber Attack*. Available at <https://www.independent.co.uk/news/world/americas/bowman-avenue-dam-us-in-fear-of-new-cyber-attack-as-dam-breach-by-iranian-hackers-is-revealed-a6782081.html>. Accessed February 16, 2016.
- DOE. 2016. *Types of Hydropower Plants*. Available at <https://energy.gov/eere/water/types-hydropower-plants>. Accessed February 16, 2016.
- FWEE. 2016. *Foundation for Water and Energy Education*. Available: <https://fwee.org/about-fwee/about/>. Accessed February 17, 2016.
- HLS. 2010. *Dam Sector Roadmap to Secure Systems*. Available at <https://www.damsafety.org/media/Documents2/security/files/DamsSectorRoadmapToSecureControlSystems.pdf>. Accessed February 16, 2016.
- Impact. 2016. *The Impact of the Bonneville Dam on Native American Culture*. Available at <https://www2.kenyon.edu/projects/Dams/bsc02yogg.html>. Accessed February 16, 2016.
- NWHA. 2016. *Northwest Hydroelectric Association*. Available at <https://www.nwhydro.org/nwha/about/>. Accessed February 17, 2016.
- NwHS. 2016. *How the Northwest Hydro System Works*. Available at <https://fwee.org/nw-hydro-tours/how-the-northwest-hydro-system-works/>. Accessed February 15, 2016.
- USBR. 2016. *Hydromet*. Available at <https://www.usbr.gov/pn/hydromet/>. Accessed February 16, 2016.
- Speakout. 2016. *Stop the Hydropower Wish List Bill*. Available at https://act.americanrivers.org/page/speakout/stop-unlockhydro-senate?source=adwords&gclid=CPTF7u3G_MoCFVE0aQodiGgMQg. Accessed February 16,

2016.

Primary References

FCRPS. 2003. *Federal Columbia River Power System*. Available at https://www.bpa.gov/power/pg/fcrps_brochure_17x11.pdf. Accessed February 16, 2016.

HLS. 2010. *Dam Sector Roadmap to Secure Systems*. Available at <https://www.damsafety.org/media/Documents2/security/files/DamsSectorRoadmapToSecureControlSystems.pdf>. Accessed February 16, 2016.

NWHA. 2016. *Northwest Hydroelectric Association*. Available at <https://www.nwhydro.org/nwha/about/>. Accessed February 17, 2016.

NwHS. 2016. *How the Northwest Hydro System Works*. Available at <https://fwee.org/nw-hydro-tours/how-the-northwest-hydro-system-works/>. Accessed February 15, 2016.

Additional References

FWEE1. 2016. *Foundation for Water and Energy Education*. Available at <https://www.fwee.org>. Accessed February 16, 2016.

FWEE2. 2016. *Overview of Hydropower in the Northwest*. Available at <https://fwee.org/education/the-nature-of-water-power/overview-of-hydropower-in-the-northwest/>. Accessed February 16, 2016.

HP. 2016. *Hydro Portal*. Available at <https://energypedia.info/wiki/Portal:Hydro>. Accessed February 16, 2016.

Singapore Water Management

- Lead Authors:
- Heidi Davidz, Alex Lee, and Alice Squires
- Contributing Author:
- Richard Turner

This example was produced as a SE example directly for the SEBoK. It describes a systems engineering approach in the development of a sustainable National Water Management System for the Republic of Singapore. It demonstrates the successful outcome of long-term planning and a systems approach to preempt a critical water shortage. The example is primarily based on information taken from a paper presented at the INCOSE International Symposium in 2008. (Chia 2008.)

Description

When Singapore achieved independence in 1965, water supply depended on water catchment in local reservoirs and two bilateral water agreements with its closest neighbor, Malaysia. These water agreements are registered with the United Nations. The first agreement expired in August 2011, and the second agreement will expire in 2061 (Singapore 2012). After several failed attempts to renegotiate the extension of the first water agreement, Singapore determined that it was necessary to achieve full water self-sufficiency by 2060 in case the second water agreement also could not be extended. An intermediate goal was to match the supply of the first water agreement before it expired. This was achieved in several ways. In 2001, the Public Utilities Board (PUB), the national water agency responsible for treating raw water in Singapore, was charged to also begin managing wastewater and stormwater, allowing for an integrated and holistic approach to water management.

This example examines Singapore's water management system from a large-scale systems engineering perspective, particularly focusing on the goals, boundaries (see Concepts of Systems Thinking), stakeholders (see Stakeholder Needs Definition), and complexities involved in this type of a national system. This approach illustrates how

Systems Thinking (illustrated through causal loop diagrams) and other systems engineering tools may be used to understand systems complexities. Concepts and methodologies of learning organizations were applied to enable understanding of behavioral complexities. Lean thinking facilitated a long-term strategic philosophy, built on the premise of continuous improvements.

Perhaps more importantly, it shows that while systems engineering, especially the Systems Approach, is necessary for the conceptualization and planning of such a complex system, it is not sufficient for success. It is the systemic structures that have been put in place over decades, political will, leadership, people, and culture that make such tasks realizable.

The supply of water in Singapore is managed in totality. Collecting rainwater, purchasing water, and purifying water utilizing reverse osmosis and desalination were all considered. Approaches included even incentivizing consumers to change their habits by making drains and canals recreational areas to encourage the public not to dispose of waste in their drains. By managing sewage and drainage together with water, environmental considerations are taken into account as well. By carefully adjusting organizational boundaries, Singapore has managed to reduce silo thinking and parochial interests. The relationships between the industry innovators, government, suppliers and users, and technology innovators create opportunities for Singapore's water management. This demonstrates how multiple stakeholder interests can be combined to create a viable water management solution. Continuous improvements through the use of technology and elimination of waste, such as reducing water that is not accounted for in the system, help to assure the sustainability of an adequate supply of water for a growing Singapore population. The Singapore Water Management system is already in successful operation and is being studied by the Organization for Economic Co-operation and Development (OECD) and by other nations.

Summary

The supply of water in Singapore is managed in totality through a systems approach, i.e., water catchment, supply, sewage and drainage. The importance of relationships between the stakeholders is also recognized. Industry innovators, political leadership, suppliers, and consumers are all involved; the project has been able to incentivize this diverse group to work together for a common goal, i.e., assuring the sustainability of an adequate supply of water for Singapore into the future.

Utilizing systems engineering and taking into consideration the systemic structures and culture required have helped Singapore achieve its first milestone of supplying its own water resources by 2010. Singapore has been able to overcome the shortfall that would have come about with the expiry of the first water agreement with Malaysia in 2011.

References

Works Cited

Chia, A. 2008. "A large-scale systems engineering perspective of water management in Singapore." Proceedings of the 18th Annual INCOSE International Symposium, 15-19 June 2008, Utrecht, The Netherlands.

Singapore Government. 2012. "The Singapore Water Story." Available at <https://www.pub.gov.sg/water/Pages/singaporewaterstory.aspx>. Accessed August 2012.

Primary References

None.

Additional References

Public Utilities Board. 2007. "PUB Main Website". Available at <https://www.pub.gov.sg>. Accessed August 2011.

Tortajada, C. 2006. "Water management in Singapore." *International Journal of Water Resources Development*. vol. 22, no. 2 p. 227-240.

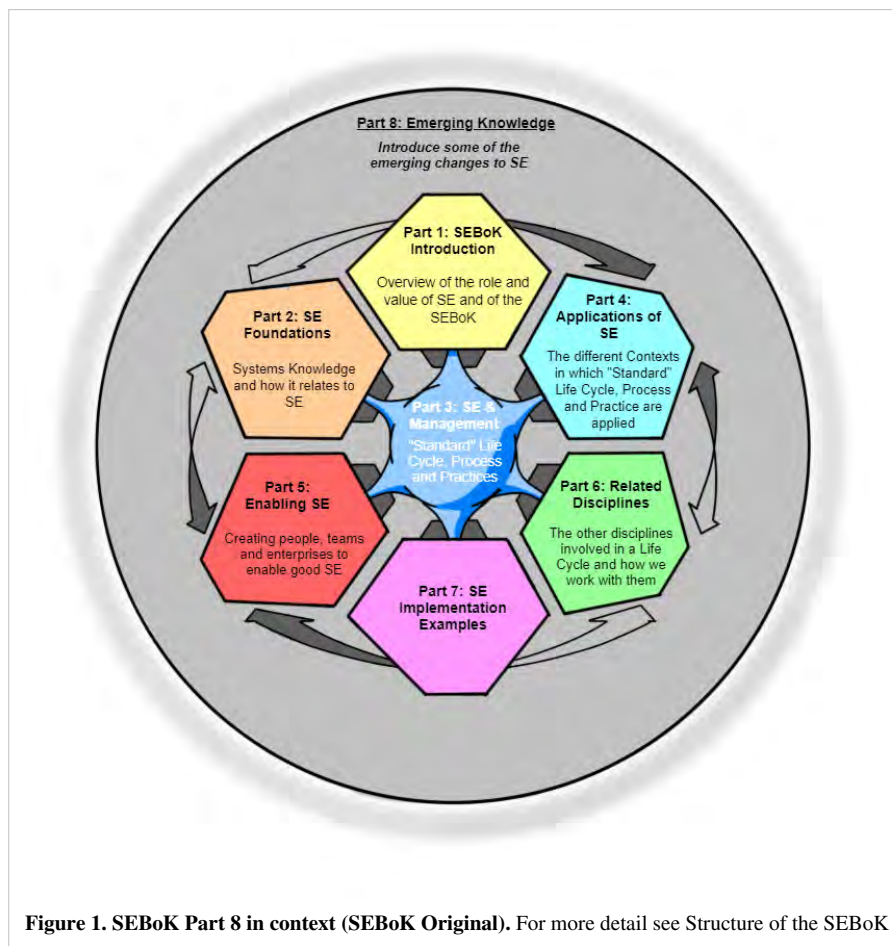
Part 8: Emerging Knowledge

Emerging Knowledge

Contents of this Part

- Emerging Topics (Robert Cloutier)
- Emerging Research (Robert Cloutier and Arthur Pyster)
- Lead Authors:
- Robert Cloutier, Daniel DeLaurentis, and Ha Phuong Le

Like other portions of the SEBoK, the notion and content of Part 8 is evolving. Part 8 consists of two Knowledge Areas (KAs): Emerging Topics and Emerging Research.



Scope and Purpose

While the practice and need for systems engineering began appearing in journals from 1950 onward, the practice currently seems to be gaining momentum in most engineering and even non-engineering circles.

The classically trained systems engineers of the 1970s and even 1980s are faced with a C note shift in thinking brought on by the rapid advance of the software centrality of our systems, cybersecurity, agent-based, object-oriented, and model-based practices. These emerging practices bring their own methods and tools. Hall (1962, p. 5) may have been prescient when he wrote “It is hard to say whether increasing complexity is the cause or the effect of man's effort to cope with his expanding environment. In either case a central feature of the trend has been the development of large and very complex systems which tie together modern society. These systems include abstract or non-physical systems, such as government and the economic system.”

These changes and the rate of change are causing systems engineering to evolve. Some of the practices may not even be recognizable to classically trained systems engineers. This Part of the SEBoK is intended to introduce some of the more significant emerging changes to systems engineering. As topics discussed in this Part evolve and become mainstream, they will be moved into the appropriate Part of the SEBoK.

System of Systems Engineering (SoSE) provides examples in recent times of an emerging topic from Systems Engineering community that generated emerging research, ultimately resulting in a foundational body of knowledge that continues to expand. A recent article describing this evolution from emerging topic to solution is now referenced in Part 4 - Systems of Systems (SoS).

Overview of Emerging Topics

See further: Emerging Topics

The Emerging Topics section is meant to inform the reader on the more significant and emerging changes to the practice of systems engineering. Examples of these emerging topics include:

- What is the potential to change systems engineering processes or the ways in which we perform systems engineering?
- How will the development of artificial intelligence impact systems engineering?
 - Will AI change the way we think of systems architecture?
 - How will we perform V&V of an AI system?
- How will the push towards vertically integrated digital engineering influence systems engineering?
- How are social features becoming more tightly connected to technical features of systems, and how is the modeling of socio-technical systems infusing into practice?

Overview of Emerging Research

See further: Emerging Research

As these emerging topics gain visibility, researchers will begin to investigate them. Corporate R&D may do early work, but academia and government will formalize this research. The Emerging Research section is a place to gather the references to this disparate work into a single repository to better inform systems engineers working on related topics. The references are collected from the following sources:

- PhD dissertations
 - INCOSE publications and events
 - IEEE publications and events
 - Research funded by National Science Foundation (NSF) – Engineering Design and Systems Engineering (EDSE)
 - Research funded by Systems Engineering Research Center (SERC)
-

References

Works Cited

Hall, Arthur D. (1962). *A Methodology for Systems Engineering*. New York, NY, USA: Van Nostrand.

Primary References

None.

Additional References

Engstrom, E.W. (1957). "Systems engineering: A growing concept," in *Electrical Engineering*, vol. 76, no. 2, pp. 113-116, Feb. 1957, doi: 10.1109/EE.1957.6442968.

Goode, H. Herbert., Machol, R. Engel. (1957). *System Engineering: An Introduction to the Design of Large-Scale Systems*. New York, NY, USA: McGraw-Hill.

Kelly, Mervin J. (1950). "The Bell Telephone Laboratories—An example of an institute of creative technology". *Proceedings of the Royal Society B*. Vol. 137, Issue 889. <https://doi.org/10.1098/rspb.1950.0050>.

Emerging Topics

Emerging Topics

Contents of this Knowledge Area

- Introduction to SE Transformation
- Socio-technical Systems (Erika Palmer)
- Artificial Intelligence (Barclay Brown, Tom McDermott, Rael Kopace, Ramakrishnan Raman, Ali Raz, and Kevin Robinson)
- Verification and Validation of Systems in Which AI is a Key Element (Laura Pullum)
- Transitioning Systems Engineering to a Model-based Discipline
- Model-Based Systems Engineering Adoption Trends 2009-2018 (Rob Cloutier) (Ifezue Obiako)
- Digital Engineering (Ron Giachetti)
- Set-Based Design (Eric Specking, Gregory S. Parnell, and Ed Pohl)
- Lead Author:
- Robert Cloutier

The Emerging Topics section is intended to introduce and inform the reader on significant and rapidly emerging needs and trends in practicing systems engineering within the community. It is not intended to be all-inclusive. Instead, those topics that have a high probability of significantly impacting the practice of systems engineering, as determined by the SEBoK editorial board, are covered. If the reader has recommendations of emerging topics that should be covered, please send an email to SEBoK@incose.org, or leave a comment in the comment feature at the bottom of this page.

Introduction to Systems Engineering Transformation

The knowledge covered in this KA reflects the transformation and continued evolution of SE, which are formed by the current and future challenges (see Systems Engineering: Historic and Future Challenges). This notion of SE transformation and the other areas of knowledge which it includes are discussed briefly below.

The INCOSE Systems Engineering Vision 2035 (INCOSE 2021) describes the global context for SE, the current state of SE practice and the possible future state of SE. It describes a number of ways in which SE continues to evolve to meet modern system challenges. These are summarized briefly below.

Systems engineering has evolved from a combination of practices used in a number of related industries (particularly aerospace and defense). These have been used as the basis for a standardized approach to the life cycle of any complex system (see Systems Engineering and Management). Hence, SE practices are largely based on heuristics, with efforts under-way to evolve a theoretical foundation for systems engineering (see Foundations of Systems Engineering) considering foundational knowledge from a variety of sources.

Systems engineering continues to evolve in response to a long history of increasing system **complexity**. Such complexity arises from human and societal needs, global megatrends, grand engineering challenges, and then are shaped by stakeholders expectations, and the enterprise environment. System solutions require both depth and breadth, and the design of those solutions must consider both technical and social aspects (see Socio-technical Systems).

Many systems engineering practices have become standard (e.g. studies, risk analysis) while some other are in transitioning phase (e.g. Model-Based Systems Engineering, agile, systems-of-systems). More recently, the rise of

Artificial Intelligence (AI) introduces unprecedented challenges in verification and validation of AI-infused systems, but also opens up new opportunities to implement AI methodologies in the design of systems.

Systems engineering has gained recognition across industries, academia and governments. However, SE practice varies across industries, organizations, and system types. Cross fertilization of systems engineering practices across industries has begun slowly but surely; however, the global need for systems capabilities has outpaced the progress in systems engineering.

INCOSE Vision 2035 concludes that SE is poised to play a major role in some of the global challenges of the 21st century, that it has already begun to change to meet these challenges and that it needs to undergo a more significant **transformation** to fully meet these challenges. The following bullet points are taken from the summary section of Vision 2035 and define the attributes of a transformed SE discipline in the future:

- The future of systems engineering is model-based, enabled by enterprise digital transformation.
- Systems engineering practices will make significant advancements to deal with systems complexity and enable enterprise agility.
- Systems engineering will leverage practices from other disciplines such as data science to help manage the growth in data.
- Formal systems engineering theoretical foundations will be codified leading to new research and development in the next generation of systems engineering methods and tools.
- AI will both impact the systems engineering practice and the types of systems designed by the systems engineering community.
- There will be a step change in systems engineering education starting with early education with a heavy focus on lifelong learning.

Some of these future directions of SE are covered in the SEBoK. Others need to be introduced and fully integrated into the SE knowledge areas as they evolve. This KA will be used to provide an overview of these transforming aspects of SE as they emerge. This transformational knowledge will be integrated into all aspects of the SEBoK as it matures.

References

Works Cited

None.

Primary References

None.

Additional References

None.

Introduction to SE Transformation

While the primary focus of the SEBoK is on the current practice of domain independent systems engineering, it is also concerned with the future evolution of the discipline.

The topics in this Knowledge Area (KA) summarize SE knowledge which is emerging and transitioning to become part of the practice of systems engineering, such as Model-Based Systems Engineering (MBSE). In general, topics will be introduced here and then expanded into other SEBoK KA's over time.

The knowledge covered in this KA reflects the transformation and continued evolution of SE. For a summary of the current and future challenges that contribute to this evolution, see Systems Engineering: Historic and Future Challenges. This notion of SE transformation and the other areas of knowledge which it includes are discussed briefly below.

Topics

Each part of the SEBoK is divided into Knowledge Areas (KAs), which are groupings of information with a related theme. The KAs, in turn, are divided into topics. This KA contains the following topics:

- Transitioning Systems Engineering to a Model-based Discipline
- Digital Engineering
- Set-Based Design
- Model-Based Systems Engineering Adoption Trends 2009-2018
- Systems Engineering Core Concepts

Systems Engineering Transformation

The INCOSE Systems Engineering Vision 2025 (INCOSE 2014) describes the global context for SE, the current state of SE practice and the possible future state of SE. It describes a number of ways in which SE continues to evolve to meet modern system challenges. These are summarized briefly below.

Systems engineering has evolved from a combination of practices used in a number of related industries (particularly aerospace and defense). These have been used as the basis for a standardized approach to the life cycle of any complex system (see Systems Engineering and Management). Hence, SE practices are still largely based on heuristics. Efforts are under-way to evolve a theoretical foundation for systems engineering (see Foundations of Systems Engineering) considering foundational knowledge from a variety of sources.

Systems engineering continues to evolve in response to a long history of increasing system complexity. Much of this evolution is in the models and tools focused on specific aspects of SE, such as understanding stakeholder needs, representing system architectures or modeling specific system properties. The integration across disciplines, phases of development, and projects continues to represent a key systems engineering challenge.

Systems engineering is gaining recognition across industries, academia and governments. However, SE practice varies across industries, organizations, and system types. Cross fertilization of systems engineering practices across industries has begun slowly but surely; however, the global need for systems capabilities has outpaced the progress in systems engineering.

INCOSE Vision 2025 concludes that SE is poised to play a major role in some of the global challenges of the 21st century, that it has already begun to change to meet these challenges and that it needs to undergo a more significant **transformation** to fully meet these challenges. The following bullet points are taken from the summary section of Vision 2025 and define the attributes of a transformed SE discipline in the future:

- Relevant to a broad range of application domains, well beyond its traditional roots in aerospace and defense, to meeting society's growing quest for sustainable system solutions to providing fundamental needs, in the globally
-

competitive environment.

- Applied more widely to assessments of socio-physical systems in support of policy decisions and other forms of remediation.
- Comprehensively integrating multiple market, social and environmental stakeholder demands against “end-to-end” life-cycle considerations and long-term risks.
- A key integrating role to support collaboration that spans diverse organizational and regional boundaries, and a broad range of disciplines.
- Supported by a more encompassing foundation of theory and sophisticated model-based methods and tools allowing a better understanding of increasingly complex systems and decisions in the face of uncertainty.
- Enhanced by an educational infrastructure that stresses systems thinking and systems analysis at all learning phases.
- Practiced by a growing cadre of professionals who possess not only technical acumen in their domain of application, but who also have mastery of the next generation of tools and methods necessary for the systems and integration challenges of the times.

Some of these future directions of SE are covered in the SEBoK. Others need to be introduced and fully integrated into the SE knowledge areas as they evolve. This KA will be used to provide an overview of these transforming aspects of SE as they emerge. This transformational knowledge will be integrated into all aspects of the SEBoK as it matures.

References

Works Cited

International Council on Systems Engineering (INCOSE). 2014. *Systems Engineering Vision 2025*, July, 2014; Available at <https://www.incose.org/docs/default-source/aboutse/se-vision-2025.pdf?sfvrsn=4>. Accessed February 16.

Primary References

None.

Additional References

None.

Relevant Videos

- Leading the Transformation of Model-Based Engineering ^[1]

References

[1] <https://www.youtube.com/watch?v=VRnNun2EH-o>

Socio-technical Systems

- Lead Author:
- Erika Palmer

Though there are a few specific definitions, there are many ways in which the term “socio-technical system” is used depending on the specific engineering/scientific domain. There are also different approaches for considering socio-technical systems depending on the life cycle stage and the specific systems engineering challenge.

The Concept and Theory

The concept of a socio-technical system describes the interrelationship between humans and machines, and the motivation behind developing research on socio-technical systems was to cope with theoretical and practical work environment problems in industry (Ropohl, 1999).

Socio-technical systems theory has been developing over the past 60 years predominately focusing on new technology and work design (Davis et al., 2014). This theory has developed into socio-technical systems thinking, and research has concentrated in several key areas:

- Human factors and ergonomics (Carayon, 2006)
- Organizational design (Cherns, 1976)
- System design (Clegg, 2000; van Eijnatten, 1998)
- Information systems (Mumford, 2006)

A Design Approach

As a design approach —socio-technical systems design (STSD)—socio-technical systems bring human, social, organizational and technical elements in the design of organizational systems (Baxter and Sommerville, 2011). While Baxter and Sommerville (2011) refer to computer-based systems in their definition of socio-technical systems thinking as a design approach, the generic term “technical system” is also applicable: “The underlying premise of socio-technical systems thinking is that system design should be a process that takes into account both social and technical factors that influence the functionality and usage of computer-based systems” (p.4).

Systems Engineering Context

In a systems engineering context, it has been argued that all systems are socio-technical systems (Palmer, et al., 2019). However, socio-technical systems in a systems engineering context is not well defined though the topic has gained traction in recent years (Donaldson, 2017; Broniatowski, 2018). There are examples in systems engineering literature, where the term socio-technical systems is used to refer to a system where social and technical elements are relevant. These include studies of agent-based modeling of socio-technical systems (Heydari and Pennock, 2018), insurance systems as socio-critical systems (Yasui, 2011) and interdisciplinary systems engineering approaches to influence enterprise systems (Pennock and Rouse, 2016; Wang et al., 2018).

Based on the work that the systems engineering community has produced thus far, the working definition of the term socio-technical systems in a systems engineering context is simply:

Socio-technical systems: Systems operating at the intersection of social and technical systems (Kroes et al., 2006).

Modeling Sociotechnical Systems

There is no “state of the practice” for how to model sociotechnical systems. There are, however, a few examples in systems engineering literature of how systems engineers could analyze these types of systems. Outside systems engineering/engineering literature, there is an ever-increasing number of examples of social system models. The modeling techniques found in these examples can be adapted to evaluate sociotechnical systems in a systems engineering context. Many of these are system dynamics models, and there is a journal dedicated to social system analysis, called the Journal for Artificial Societies and Social Simulation (JASS), which focuses on agent-based modeling.

1) Qualitative Modeling

- Insurance systems as socio-critical systems (Yasui, 2011)

Yasui (2011) provides a new methodology to accommodate stakeholder goals in social system failures. This new methodology is a “soft” systems approach that brings together the Holon concept by Checkland and Scholes (1990) and the Vee Model.

2) Agent-Based Modeling of Sociotechnical Systems in Systems Engineering

- Agent-based modeling of sociotechnical systems (Heydari and Pennock, 2018)

Heydari and Pennock (2018) illustrate how to support the design and governance of sociotechnical systems with agent-based modeling (ABM). Critically, they outline the difference between how ABM is used in physical, natural and social applications versus sociotechnical applications.

- Interdisciplinary systems engineering approaches to influence enterprise systems (Pennock and Rouse, 2016)

Pennock and Rouse (2016) not only provide how to define an enterprise as a system, but they also illustrate this with several ABM examples. They also highlight that when modeling sociotechnical systems versus traditional engineering systems, it is important to focused less on “control” and more on “influence.”

3) Economic modeling

- Social System Modeling Challenges (Wang et al., 2018)

In their book, Social Systems Engineering, Wang et al. (2018) provide an overview of not only modeling and its challenges in evaluating social systems, but they also give insight into how social system modeling is approached in economics.

4) System Dynamics Modeling of Social Systems for Adaptation in an SE Sociotechnical Context

- Social policy (Palmer, 2017)

Palmer (2017) provides an overview of social systems in a systems engineering context, and uses system dynamics modeling of pension and sick leave policy systems to illustrate how to use systems engineering methods for social policy.

- Social Systems Engineering (García-Díaz and Olaya, 2018)

García-Díaz and Olaya (2018) give not only a thorough overview in their book (called Social Systems Engineering) of social systems and various qualitative and quantitative modeling types, but they also highlight participatory system dynamics modeling (stakeholder-led system design).

- Health care (Homer and Hirsch, 2006)

As there is increasing attention in the systems engineering community towards health care technology, Homer and Hirsch’s (2006) paper on system dynamics modeling of public health gives a basis for how to model social systems in this domain. For example, chronic disease prevention, disease outcomes, health and risk behaviors, environmental factors, and health-related resources and delivery systems.

References

Works Cited

- Baxter, G. and Sommerville, I., 2011. Socio-technical systems: From design methods to systems engineering. *Interacting with computers*, 23(1), pp.4-17.
- Broniatowski, DA, 2018, 'Building the tower without climbing it: Progress in engineering systems', *Systems Engineering*, 21 (3), 259-81.
- Carayon, P., 2006. 'Human factors of complex sociotechnical systems.' *Applied ergonomics*, 37(4), pp.525-535.
- Checkland, P. and Scholes, J. 1990. 'Soft systems methodology in action.' Wiley: UK.
- Cherns, A., 1976. The principles of sociotechnical design. *Human relations*, 29(8), pp.783-792.
- Clegg, C.W., 2000. Sociotechnical principles for system design. *Applied ergonomics*, 31(5), pp.463-477.
- Davis, M.C., Challenger, R., Jayewardene, D.N. and Clegg, C.W., 2014. Advancing socio-technical systems thinking: A call for bravery. *Applied ergonomics*, 45(2), pp.171-180.
- Donaldson, W, 2017. 'In Praise of the "Ologies": A Discussion of and Framework for Using Soft Skills to Sense and Influence Emergent Behaviors in Sociotechnical Systems', *Systems Engineering*, 20 (5), 467-78.
- Heydari, B and Pennock, MJ, 2018, 'Guiding the behavior of sociotechnical systems: The role of agent-based modeling', *Systems Engineering*, 21 (3), 210-26.
- Homer, JB and Hirsch, GB, 2006, 'System dynamics modeling for public health: background and opportunities', *American journal of public health*, 96 (3), 452-458.
- Kroes, P, Franssen, M, Poel, IVD and Ottens M, 2006, 'Treating socio-technical systems as engineering systems: some conceptual problems', *Systems Research and Behavioral Science*, 23 (6), 803-814.
- Palmer, E, 2017, 'Systems Engineering Applied to Evaluate Social Systems: Analyzing Systemic Challenges to the Norwegian Welfare State.' University of Bergen: Norway.
- Palmer, E, Presland, I, Rhodes, D, Olaya, C, Haskins, C, Glazner, C, 2019, 'Social Systems-Where Are We and Where Do We Dare to Go?' Panel Discussion. 29th Annual INCOSE Symposium, Orlando, Florida
- Pennock, MJ and Rouse WB, 2016, 'The epistemology of enterprises', *Systems Engineering*, 19 (1), 24-43.
- Ropohl, G., 1999. Philosophy of socio-technical systems. *Society for Philosophy and Technology Quarterly Electronic Journal*, 4(3), pp.186-194.
- van Eijnatten, F.M., 1998. Developments in socio-technical systems design (STSD). P. J. Drenth, H. Thierry, & CJ de Wolff, *Handbook of Work and Organizational Psychology*, 2, pp.61-80.
- Wang, H, Li, S and Wang, Q, 2018. 'Introduction to Social Systems Engineering.' Springer: US.
- Yasui, T, 2011, 'A new systems engineering approach for a Socio-Critical System: A case study of claims-payment failures of Japan's insurance industry,' *Systems Engineering*, 14 (4), 349-63

Primary References

None.

Additional References

None.

Artificial Intelligence

- Lead Authors:
- Barclay Brown, Tom McDermott, Rael Kopace, Ramakrishnan Raman, Ali Raz, and Kevin Robinson

This article provides an overview and definitions of artificial intelligence and machine learning and their importance to and relationships with systems engineering.

Introduction

Artificial Intelligence (AI) is perhaps best defined as the ability of a system to exhibit behavior, which if exhibited by a human being, would be considered intelligent. It's the ability of a machine to process and learn from data, to recognize and understand patterns, to solve problems, and make decisions autonomously. The term "Artificial Intelligence" dates to the early 1950s where it was first introduced to mimic human-level intelligence capabilities in software and hardware systems—a goal which still is far from reachable in the near future despite the rapid technological advancement in the AI field. It is important for systems engineers to note that "AI" does not designate a specific technology—intelligent behavior can be implemented in a system in a number of ways, including conventional software or hardware logic. Modern systems that use AI fall into two major categories: systems where the decision-making logic is explicitly pre-programmed, and systems where the decision-making capability is learned from data. This latter category, known as Machine Learning (ML), is the predominant category for the modern paradigm shift in capabilities enabled by AI.

Machine Learning Foundations of AI

ML builds its foundation from statistics and computer science disciplines, accelerated by the advances in computational power and memory offered by modern hardware systems (i.e., CPUs and GPUs). The ever-increasing number of ML algorithms available today can be classified into three major categories of Supervised, Unsupervised, and Reinforcement learning.

- Supervised Learning is where labelled input-output datasets are available and ML algorithms learn the mapping between input-output pairs presented in the datasets (for example, learning from clearly labelled pictures of cars and tanks)
- Unsupervised Learning is where the labels in data sets are not available and the ML algorithms learn the grouping or clustering of input and outputs (for example, identifying common customer attributes, classifying spam email, or detecting fraudulent transactions)
- Reinforcement Learning is where an AI agent trains itself to make decisions to effect a positive change in the environment based on a defined reward function (e.g., training a robot to avoid obstacles by penalizing crashes)

The implementation approaches for all ML types require large datasets and can be based on statistical models or neural networks including Deep Neural Networks (DNNs). Irrespective of the implementation approaches and the type of ML, the workflow for deploying an ML-based AI solution remains largely common and is shown in Figure 1 (details on each of these steps can be found in the Digital Transformation book by Thomas Siebel).

Figure 1 A typical ML Workflow

Understanding the subtleties of statistical models and neural networks for ML implementation along with the related workflow will become imperative for systems engineers as they begin to incorporate ML solutions in systems and perform systems engineering activities for AI systems.

Systems Engineers and AI

Given the holistic nature of the systems engineering discipline, it is practical to think about AI and its role in the larger context of digital transformation, which can be defined as the confluence of big data, cloud computing, AI and Internet of Things (IoT). Digital transformation is enabling organizations to evolve their business and operational models. As such most organizations are collecting vast amounts of data pertaining to their business, operations, systems, customers, and personnel. Big data has created the conditions that necessitate the application of AI solutions supported by the technological advancements in computational power and memory to aid in the analysis of big data and the automation of labor intensive and time-consuming processes.

The ability to collaborate and communicate with data scientists, AI engineers, cloud computing specialists, application developers, and other experts is becoming a necessity for many systems engineers. Systems engineers must become sufficiently knowledgeable and familiar with as many elements of the digital transformation ecosystem as possible to best serve the needs of the organization and its mission in order to identify the areas of business or the systems that would most benefit from the application of AI. Understanding the benefits as well as the potential pitfalls of an AI development initiatives is also important. To that end, systems engineers must be able to identify the right requirements and architectures, expertise, partnering organizations, technology stack, development platforms, and other relatable elements. The roles and responsibilities of systems engineers versus data scientists, data engineers, application developers, and AI engineers must be differentiated. The market is filled with an ever-growing number of AI platforms and it is virtually impossible for a non-data scientist to be able to master each of them. Moreover, the advances in the AI domain happen at such a rapid pace that nowadays many companies may choose to avoid investing in their own internal data science teams to develop AI solutions. Instead, they can purchase or license models that are ready for implementation from companies that have solved similar problems before. Additionally, there exist unified platforms that aim to standardize workflows and come equipped with the appropriate technology stack to collect, label, and feed data into supervised learning models or alternatively assist with the development of models.

As a result, rather than mirroring the roles and responsibilities of data scientists and other AI domain professionals, it is beneficial for most systems engineering professionals to develop an understanding at various degrees of depth of the relevant technologies that support and are used to create AI applications including but not limited to the following:

- Data integration
- Relational and non-relational databases
- Cloud storage
- Enterprise architecture infrastructure and APIs
- ML learning frameworks
- Batch and online processing
- Executable environments
- Commonly used programming languages
- UI and data visualization tools

Systems Engineering and AI: SE4AI and AI4SE

At an early 2019 Future of Systems Engineering (FuSE) workshop hosted by the International Council on Systems Engineering (INCOSE), the terms AI for SE and SE for AI were first used to describe the dual transformation envisioned for both the SE and AI disciplines. The “AI4SE” and “SE4AI” labels have quickly become symbols for an upcoming rapid evolutionary phase in the SE community. SE’s Digital Engineering transformation will enable both transformation of SE practices using AI for SE and drive the need for new systems engineering practices that support a new wave of automated, adaptive, and learning systems, termed SE for AI. AI4SE applies AI and ML techniques to improve human-driven engineering practices. This goal of “augmented intelligence” includes outcomes such as

achieving scale in model construction and efficiency in designing space exploration systems. SE4AI applies SE methods to the design and operation of intelligent systems, with outcomes such as improved safety, security, ethics, etc.

Systems engineers have long sought to evolve engineered systems to include human-like intelligence, thinking, and autonomous decision-making. In many cases, the inclusion of intelligent capabilities means increasing non-determinism in the system's behavior, exponentially increasing complexity in the design, verification, and validation of such systems. The initial approaches of inculcating intelligence predominantly involved human-in-the-loop for capturing knowledge from experts, codifying it in some form that is machine understandable (for instance, a "knowledge database"), and enabling the system to interpret the codified knowledge so as to make the required optimal decisions and exhibit the desired intelligent behavior. These initial approaches involved human-in-the-loop for the entire gamut of activities from capturing the knowledge and codifying the knowledge to enabling (programming) the system to leverage the knowledge.

More recent contemporary approaches represent the next evolution of inculcating intelligence in systems, wherein the human gathers the required data and programs the system such that the system can learn/build the knowledge from the data provided by the human. Learning from data requires significant computing resources (processing power and memory) but recent advances have expanded the availability of large amounts of both source data and matching computing resources to enable intensive processing and learning from data. Recent ML algorithms are the result of this evolution. The system learns during the design and development phases and is expected to demonstrate limited forms of adaptive behavior.

In both approaches, a human is in the loop to validate the intelligence gained by the system, and to make a conscious decision to deploy the intelligence in the system. The next evolution is expected to be predominantly based on a system learning from the other systems it interacts with (machine-to-machine learning) and building its intelligence from the interactions and from the ecosystem (cloud). Evolution in that direction is as seen by some of the recent progress made in reinforcement learning models and algorithms. The system learns by trial-and-error and once it meets performance expectations, can apply its behavior on a larger scale.

The augmented intelligence scenarios described above will need an underlying and synergistic foundation of both SE4AI and AI4SE which are briefly described in the following subsections.

SE4AI: Addressing Practical Implementation Challenges for AI

The increasing application of AI in systems presents challenges for both the systems engineering community and the AI community. Their common goal is to ensure that the future users of the system can be certain that the behavior and performance of that system is what is needed. That is, the AI system's behavior is verified through a set of activities that check its compliance against system requirements and validate it for fitness to meet user needs. The need for verification and validation presents a challenge for the engineering of AI systems as the failure modes observed in AI systems are different and may not be adequately addressed by traditional Systems Engineering life-cycle approaches. From a systems engineering lifecycle perspective, appropriate tailoring of conventional processes is needed to "engineer" a system that is intelligent—a system that learns during development and is designed for adaptive behavior. New approaches are required for developing requirements, evaluating when these intelligent systems are ready for operations, and for ensuring their adaptive behavior is safe and produces the desired outcomes. The data required for enabling a system to acquire the desired intelligence poses several challenges from being biased (inculcating a biased intelligence reflected in the system's behavior) to not being representative of the various use case scenarios envisaged for the system. The system's intelligence is only as good as the data that was used to train it. There are many considerations pertaining to the broader challenge of engineering safe and effective human interaction with intelligent systems. These include (a) leveraging cognitive strengths of humans and AI, (b) gaining trust, (c) explainable intelligence (d) identifying and understanding bias, and (e) dealing with uncertainty.

Broadly there are three key challenges for the systems engineering of intelligent systems:

1. **New failure modes not previously experienced in the engineering of systems.** The AI community recognizes that there are five main failure modes that cause the AI systems to not behave safely and as expected. These new failure modes include negative side effects, reward hacking, scalable oversight, unsafe exploration, and distributional shift.
2. **The unpredictability of performance due to non-deterministic and evolving behavior.** ML systems initially learn from predetermined data and through the activity of validation, system engineers check the compliance of the system performance against its intended purpose, captured in a Systems Specification. The challenge with AI, and specifically ML, is predicting the performance and behavior of the AI algorithm on unseen data in future operations. ML systems exhibit non-deterministic performance, with the performance of some systems evolving as the system learns (changes performance) during operations. This presents challenges in validating system compliance before the system enters operations.
3. **Lack of trust and robustness in future systems performance.** System validation is based on a basic four step approach: obtaining results from a validation test, comparing the measured results to expect results, deducing the degree of compliance, and deciding on the acceptability of this compliance. A key aspect in deciding the acceptability of compliance is expert judgement. Expert judgement requires an understanding of the result as compared to the relevance of the context of use, and therefore the results need to be explainable. Explainable behavior of AI Systems is problematic, and therefore determining a level of trust and robustness in future systems performance is challenging.

AI4SE: Leveraging AI to Advance State-of-the-Art in Systems Engineering

The next evolution of SE practices is expected to be predominantly driven by AI technologies assisting systems engineers in the engineering development lifecycle activities. AI4SE addresses how AI can enhance the systems engineering lifecycle for engineered systems, across the various lifecycle phases including concept development, requirements, architecture design, implementation, integration, verification, validation, and deployment. Enhancing and assisting systems engineering processes, methods, and tools, with tangible impacts on the quality of the engineered system as well as on the cycle time for the various life cycle activities, would be some of the primary focus areas of AI4SE. For instance, AI technologies can be leveraged to advise system architects on the various architecture and design decisions options based on intelligence built from collective prior experience of decisions made in earlier systems. A second example is leveraging AI technologies to assist in arriving at various corner test cases during verification. Historical systems engineering life cycle data would be the predominant drivers for the use of AI for such applications. A third example would pertain to the simulation aspects, where digital and synthetic environments (e.g., digital twins) are leveraged to understand various lifecycle operation scenarios and providing better insights to systems engineers on understanding the implications of architecture design decisions on the engineered systems. However, some forms of distinction are being envisaged in the community between AI4SE as against automation, digitization and digital linking of various SE life cycle artifacts and work products. While automation and digitization activities may be enhanced by AI, these activities may still be dominated by conventional software, with outcomes focused on significant reduction in life cycle time and efficiency on managing scale.

Landscape of AI Applications in Industry and Systems

Over the last few decades, ML-based solutions have become ubiquitous in technical and social systems and have enabled new business opportunities and even new business models. The rapid commercialization of AI is bringing profound changes to all markets. Capabilities are advancing and it is becoming easier to develop and implement AI solutions given the growing number of AI tools and platforms. Organizations in every industry segment are increasingly realizing that AI is key to market leadership and that they all have processes that are suited for AI. Early adopters of AI have noticed significant tangible benefits and given the lower barriers of entry that exist nowadays,

others are making significant investments to accelerate their AI adoption.

AI is being integrated into the fabric of business and systems and AI solutions are being deployed to solve a wide spectrum of use cases at every layer of the enterprise. The following table provides examples of AI applications across the different fields and sectors.

Domain	General examples of AI usage
Sales	Price optimization, forecasting, performance management, dynamic recommendations (think Amazon, Netflix making product and movie recommendations) [Ref: https://hbr.org/2018/07/how-ai-is-changing-sales]
Security	Breach risk prediction, incident response, early identification and classification of cyber threats [Ref: https://www.ibm.com/case-studies/cargills-bank-ltd]
Anti-fraud	Identification of fraudulent behavior and transactions [Ref: https://www.fico.com/blogs/5-keys-using-ai-and-machine-learning-fraud-detection]
HR	Candidate assessment, screening time reduction, skills to jobs alignment [Ref: https://www.forbes.com/sites/jeannemeister/2019/01/08/ten-hr-trends-in-the-age-of-artificial-intelligence/?sh=426c0c7d3219]
Marketing	Programmatic advertising for target audiences, behavior analysis, interactive marketing through chatbots [Ref: https://www.forbes.com/sites/forbesagencycouncil/2019/08/21/how-artificial-intelligence-is-transforming-digital-marketing/?sh=46ae38e021e1]
Personal Assistant	Control smart home objects, interact with the web and provide answers to questions, manage calendar
Smart tools	Smart thermostats, doorbells, home security system, baby monitor, real time language translation [Ref: https://www.pcmag.com/news/the-best-smart-home-devices-for-2020]
Finance	Automated financial trading and trade validation to protect from obvious errors and irrational price swings, identification and management of risk based on user history, classification of loan and credit applications [Ref: https://builtin.com/artificial-intelligence/ai-finance-banking-applications-companies]
Healthcare	Assisted diagnostic medical imaging, management and accuracy check of electronic medical records, disease risk prevention, personalized health management [Ref: https://www2.deloitte.com/cn/en/pages/technology-media-and-telecommunications/articles/global-ai-development-white-paper.html]
Education	Virtual adaptive teaching and learning, curriculum personalization, virtual tutor [Ref: https://medium.com/towards-artificial-intelligence/artificial-intelligence-in-education-benefits-challenges-and-use-cases-db52d8921f7a]
Autonomous Driving	Real time sensor data processing, dynamic path planning, auto health monitoring, route optimization, real time traffic monitoring [Ref: https://www.embedded.com/the-role-of-artificial-intelligence-in-autonomous-vehicles/]
Retail	Smart inventory, demand forecasting, smart logistics, unmanned store, automated customer service inquiries through chatbots [Ref:]
Manufacturing	Quality and safety checks, improving yield and performance, failure mode prediction, predictive maintenance, adaptive design, energy saving, production forecasting [Ref: https://www2.deloitte.com/cn/en/pages/technology-media-and-telecommunications/articles/global-ai-development-white-paper.html]
Media	Personalized media content, content discovery, real time fact checking, identifying false content [Ref: https://neoteric.eu/blog/10-use-cases-of-ai-in-manufacturing/]
Government and Legal	Non-compliant behavior and tax evasion, policy checks, citizen assistance chatbots, emergency and disaster resource identification and monitoring, automated due diligence on background information, legal analytics, content generation [Ref: https://emerj.com/ai-sector-overviews/ai-in-law-legal-practice-current-applications/]
Agriculture	Pest monitoring, crop return prediction, optimizing yield, monitoring and managing closed environment, weather forecasting [Ref: https://www.intel.com/content/www/us/en/big-data/article/agriculture-harvests-big-data.html]

Logistics	Predictive supply chain network management, demand and capacity planning, route optimization, [Ref: dhl.com/content/dam/dhl/global/core/documents/pdf/glo-core-trend-report-artificial-intelligence.pdf]
Oil and Gas	Oil seep detection with robots, precision drilling, temperature and pressure monitoring, predictive maintenance [Ref: https://www.sparkcognition.com/top-4-ai-applications-oil-gas-industry/]

Acknowledgements

The article is provided by INCOSE's AI working group and includes summarized sections from the AI Primer for Systems Engineers which is currently under development. The AI Primer for Systems Engineers will provide a broader view of AI discipline and its implication for Systems Engineering. The AI working group would like to acknowledge (in alphabetical order) Barclay Brown, Tom McDermott, Rael Kopace, Ramakrishnan Raman, Ali Raz, and Kevin Robinson for their contributions to this article.

References

Works Cited

- Ammanath, B., Jarvis, D. and Hupfer, S., 2021. Thriving in the era of pervasive AI. [online] Deloitte Insights. Available at: <https://www2.deloitte.com/us/en/insights/focus/cognitive-technologies/state-of-ai-and-intelligent-automation-in-business-survey.html> [Accessed 23 April 2021].
- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv preprint arXiv:1606.06565.
- Jordan, Michael I. "Artificial intelligence—the revolution hasn't happened yet." *Harvard Data Science Review* 1.1 (2019).
- Sharma, M., 2021. Navigating the New Landscape of AI Platforms. [online] Harvard Business Review. Available at: <https://hbr.org/2020/03/navigating-the-new-landscape-of-ai-platforms> [Accessed 23 April 2021].
- Siebel, Thomas M. Digital transformation: survive and thrive in an era of mass extinction. Rosetta Books, 2019.
- TechRepublic. 2021. 10 ways data and analytics will impact businesses. [online] Available at: <https://www.techrepublic.com/article/10-ways-data-and-analytics-will-impact-businesses/> [Accessed 23 April 2021].

Primary References

- Siebel, Thomas M. Digital transformation: survive and thrive in an era of mass extinction. Rosetta Books, 2019.

Additional References

None.

Verification and Validation of Systems in Which AI is a Key Element

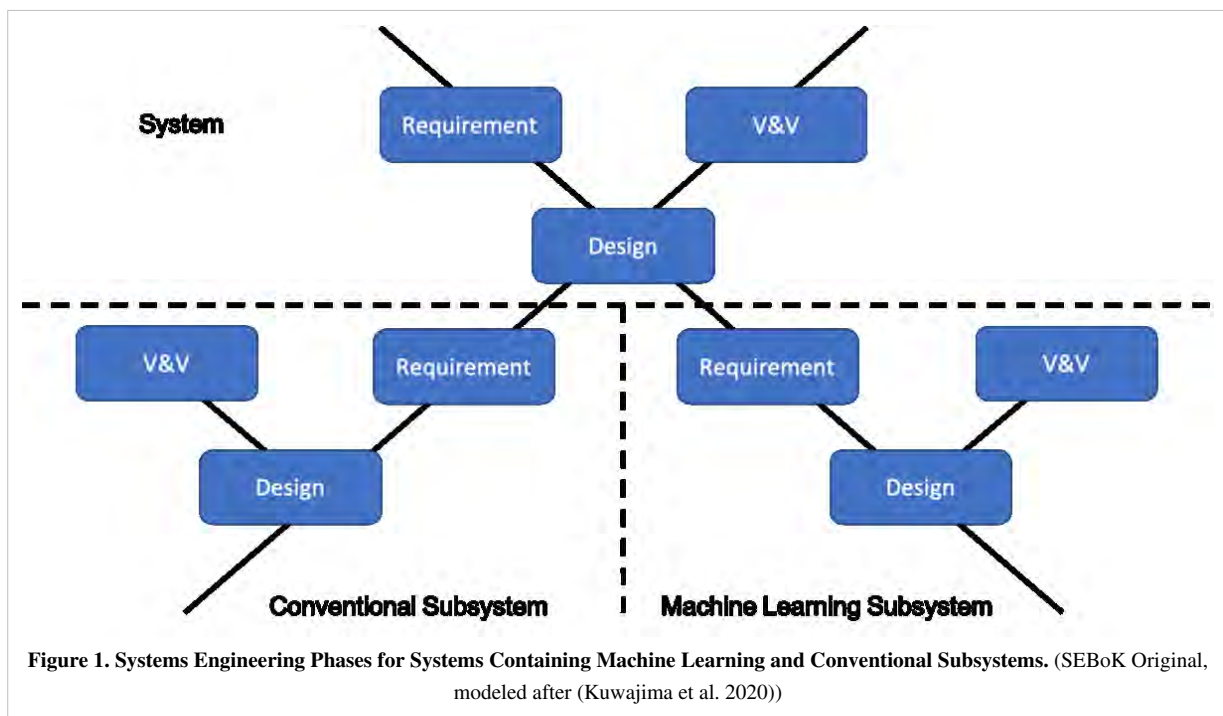
- Lead Author:
- Laura Pullum

Many systems are being considered in which artificial intelligence (AI) will be a key element. Failure of an AI element can lead to system failure (Dreossi et al 2017), hence the need for AI verification and validation (V&V). The element(s) containing AI capabilities is treated as a subsystem and V&V is conducted on that subsystem and its interfaces with other elements of the system under study, just as V&V would be conducted on other subsystems. That is, the high-level definitions of V&V do not change for systems containing one or more AI elements.

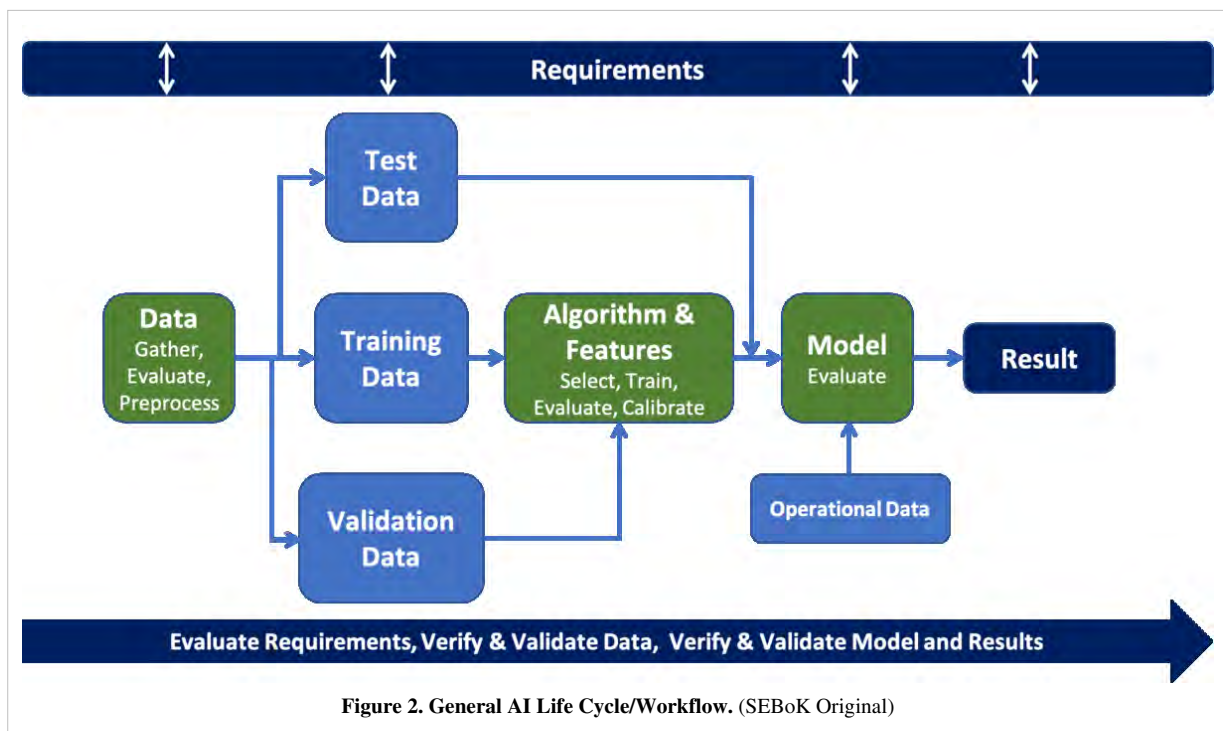
However, AI V&V challenges require approaches and solutions beyond those for conventional or traditional (those without AI elements) systems. This article provides an overview of how machine learning components/subsystems “fit” in the systems engineering framework, identifies characteristics of AI subsystems that create challenges in their V&V, illuminates those challenges, and provides some potential solutions while noting open or continuing areas of research in the V&V of AI subsystems.

Overview of V&V for AI-based Systems

Conventional systems are engineered via 3 overarching phases, namely, requirements, design and V&V. These phases are applied to each subsystem and to the system under study. As shown in Figure 1, this is the case even if the subsystem is based on AI techniques.



AI-based systems follow a different lifecycle than do traditional systems. As shown in the general machine learning life cycle illustrated in Figure 2, V&V activities occur throughout the life cycle. In addition to requirements allocated to the AI subsystem (as is the case for conventional subsystems), there also may be requirements for data that flow up to the system from the AI subsystem.



Characteristics of AI Leading to V&V Challenges

Though some aspects of V&V for conventional systems can be used without modification, there are important characteristics of AI subsystems that lead to challenges in their verification and validation. In a survey of engineers, Ishikawa and Yoshioka (2019) identify attributes of machine learning that make the engineering of same difficult. According to the engineers surveyed, the top attributes with a summary of the engineers' comments are:

- *Lack of an oracle*: It is difficult or impossible to clearly define the correctness criteria for system outputs or the right outputs for each individual input.
- *Imperfection*: It is intrinsically impossible to for an AI system to be 100% accurate.
- *Uncertain behavior for untested data*: There is high uncertainty about how the system will behave in response to untested input data, as evidenced by radical changes in behavior given slight changes in input (e.g., adversarial examples).
- *High dependency of behavior on training data*: System behavior is highly dependent on the training data.

These attributes are characteristic of AI itself and can be generalized as follows:

- Erosion of determinism
- Unpredictability and unexplainability of individual outputs (Sculley et al., 2014)
- Unanticipated, emergent behavior, and unintended consequences of algorithms
- Complex decision making of the algorithms
- Difficulty of maintaining consistency and weakness against slight changes in inputs (Goodfellow et al., 2015)

V&V Challenges of AI Systems

Requirements

Challenges with respect to AI requirements and AI requirements engineering are extensive and due in part to the practice by some to treat the AI element as a "black box" (Gunning 2016). Formal specification has been attempted and has shown to be difficult for those hard-to-formalize tasks and requires decisions on the use of quantitative or Boolean specifications and the use of data and formal requirements. The challenge here is to design effective

methods to specify both desired and undesired properties of systems that use AI- or ML-based components (Seshia 2020).

A taxonomy of AI requirements engineering challenges, outlined by Belani and colleagues (2019), is shown in Table 1.

Table 1: Requirements engineering for AI (RE4AI) taxonomy, mapping challenges to AI-related entities and requirements engineering activities (after (Belani et al., 2019))

RE4AI		AI Related Entities	
RE Activities	Data	Model	System
Elicitation	- Availability of large datasets	- Lack of domain knowledge	- How to define problem /scope
	- Requirements analyst upgrade	- Undeclared consumers	- Regulation (e.g., ethics) not clear
Analysis	- Imbalanced datasets, silos	- No trivial workflows	- No integration of end results
	- Role: data scientist needed	- Automation tools needed	- Role: business analyst upgrade
Specification	- Data labelling is costly, needed	- No end-to-end pipeline support	- Avoid design anti- patterns
	- Role: data engineer needed	- Minimum viable model useful	- Cognitive / system architect needed
Validation	- Training data critical analysis	- Entanglement, CACE problem	- Debugging, interpretability
	- Data dependencies	- High scalability issues for ML	- Hidden feedback loops
Management	- Experiment management	- Difficult to log and reproduce	- IT resource limitations, costs
	- No GORE-like method polished	- DevOps role for AI needed	- Measuring performance
Documentation	- Data & model visualization	- Datasets and model versions	- Feedback from end-users
	- Role: research scientist useful	- Education and training of staff	- Development method
All of the Above	- Data privacy and data safety		
	- Data dependencies		

CACE: change anything, change everything

GORE: goal-oriented requirements engineering

Data

Data is the life-blood of AI capabilities given that it is used to train and evaluate AI models and produce their capabilities. Data quality attributes of importance to AI include accuracy, currency and timeliness, correctness, consistency, in addition to usability, security and privacy, accessibility, accountability, scalability, lack of bias and others. As noted above, the correctness of unsupervised methods is embedded in the training data and the environment.

There is a question of coverage of the operational space by the training data. If the data does not adequately cover the operational space, the behavior of the AI component is questionable. However, there are no strong guarantees on when a data set is 'large enough'. In addition, 'large' is not sufficient. The data must sufficiently cover the operational space.

Another challenge with data is that of adversarial inputs. Szegedy et al. (2014) discovered that several ML models are vulnerable to adversarial examples. This has been shown many times on image classification software, however, adversarial attacks can be made against other AI tasks (e.g., natural language processing) and against techniques other than neural networks (typically used in image classification) such as reinforcement learning (e.g., reward hacking) models.

Model

Numerous V&V challenges arise in the model space, some of which are provided below.

- *Modeling the environment*: Unknown variables, determining the correct fidelity to model, modeling human behavior. The challenge problem is providing a systematic method of environment modeling that allows one to provide provable guarantees on the system's behavior even when there is considerable uncertainty about the environment. (Seshia 2020)
- *Modeling learning systems*: Very high dimensional input space, very high dimensional parameter or state space, online adaptation/evolution, modeling context (Seshia 2020).
- *Design and verification of models and data*: data generation, quantitative verification, compositional reasoning, and compositional specification (Seshia 2020). The challenge is to develop techniques for compositional reasoning that do not rely on having complete compositional specifications (Seshia 2017).
- *Optimization strategy must balance between over- and under-specification*. One approach, instead of using distance (between predicted and actual results) measures, uses the cost of an erroneous result (e.g., an incorrect classification) as a criterion (Faria, 2018) (Varshney, 2017).
- *Online learning*: requires monitoring; need to ensure its exploration does not result in unsafe states.
- *Formal methods*: intractable state space explosion from complexity of the software and the system's interaction with its environment, an issue with formal specifications.
- *Bias* in algorithms from underrepresented or incomplete training data OR reliance on flawed information that reflects historical inequities. A biased algorithm may lead to decisions with collective disparate impact. Trade-off between fairness and accuracy in the mitigation of an algorithm's bias.
- *Test coverage*: effective metrics for test coverage of AI components is an active area of research with several candidate metrics, but currently no clear best practice.

Properties

Assurance of several AI system properties is necessary to enable trust in the system, e.g., the system's trustworthiness. This is a separate though necessary aspect of system dependability for AI systems. Some important properties are listed below and though extensive, are not comprehensive.

- *Accountability*: refers to the need of an AI system to be answerable for its decisions, actions and performance to users and others with whom the AI system interacts
 - *Controllability*: refers to the ability of a human or other external agent to intervene in the AI system's functioning
 - *Explainability*: refers to the property of an AI system to express important factors influencing the AI system results or to provide details/reasons behind its functioning so that humans can understand
 - *Interpretability*: refers to the degree to which a human can understand the cause of a decision (Miller 2017)
 - *Reliability*: refers to the property of consistent intended behavior and results
 - *Resilience*: refers to the ability of a system to recover operations quickly following an incident
 - *Robustness*: refers to the ability of a system to maintain its level of performance when errors occur during execution and to maintain that level of performance given erroneous inputs and parameters
 - *Safety*: refers to the freedom from unacceptable risk
 - *Transparency*: refers to the need to describe, inspect and reproduce the mechanisms through which AI systems make decisions, communicating this to relevant stakeholders.
-

V&V Approaches and Standards

V&V Approaches

Prior to the proliferation of deep learning, research on V&V of neural networks touched on adaptation of available standards, such as the then-current IEEE Std 1012 (Software Verification and Validation) processes (Pullum et al. 2007), areas need to be augmented to enable V&V (Taylor 2006), and examples of V&V for high-assurance systems with neural networks (Schumann et al., 2010). While these books provide techniques and lessons learned, many of which remain relevant, additional challenges due to deep learning remain unsolved.

One of the challenges is data validation. It is vital that the data upon which AI depends undergo V&V. Data quality attributes that are important for AI systems include accuracy, currency and timeliness, correctness, consistency, usability, security and privacy, accessibility, accountability, scalability, lack of bias, and coverage of the state space. Data validation steps can include file validation, import validation, domain validation, transformation validation, aggregation rule and business validation (Gao et al. 2011).

There are several approaches to V&V of AI components, including formal methods (e.g., formal proofs, model checking, probabilistic verification), software testing, simulation-based testing and experiments. Some specific approaches are:

- Metamorphic testing to test ML algorithms, addressing the oracle problem (Xie et al., 2011)
- A ML test score consisting of tests for features and data, model development and ML infrastructure, and monitoring tests for ML (Breck et al., 2016)
- Checking for inconsistency with desired behavior and systematically searching for worst-case outcomes when testing consistency with specifications.
- Corroborative verification (Webster et al., 2020), in which several verification methods, working at different levels of abstraction and applied to the same AI component, may prove useful to verification of AI components of systems.
- Testing against strong adversarial attacks (Useato, 2018); researchers have found that models may show robustness to weak adversarial attacks and show little to no accuracy to strong attacks (Athalye et al., 2018, Uesato et al., 2018, Carlini and Wagner, 2017).
- Use of formal verification to prove that models are consistent with specifications, e.g., (Huang et al., 2017).
- Assurance cases combining the results of V&V and other activities as evidence to support claims on the assurance of systems with AI components (Kelly and Weaver, 2004; Picardi et al. 2020).

Standards

Standards development organizations (SDO) are earnestly working to develop standards in AI, including the safety and trustworthiness of AI systems. Below are just a few of the SDOs and their AI standardization efforts.

ISO is the first international SDO to set up an expert group to carry out standardization activities for AI. Subcommittee (SC) 42 is part of the joint technical committee ISO/IEC JTC 1. SC 42 has a working group on foundational standards to provide a framework and a common vocabulary, and several other working groups on computational approaches to and characteristics of AI systems, trustworthiness, use cases, applications, and big data. (<https://www.iso.org/committee/6794475.html>)

The IEEE P7000 series of projects are part of the IEEE Global Initiative on Ethics of Autonomous and Intelligent Systems, launched in 2016. IEEE P7009, “Fail-Safe Design of Autonomous and Semi-Autonomous Systems” is one of 13 standards in the series. (<https://standards.ieee.org/project/7009.html>)

Underwriters Laboratory has been involved in technology safety for 125 years and has released ANSI/UL 4600 “Standard for Safety for the Evaluation of Autonomous Products”. (<https://ul.org/UL4600>)

The SAE G-34, Artificial Intelligence in Aviation, Committee is responsible for creating and maintaining SAE Technical Reports, including standards, on the implementation and certification aspects related to AI technologies inclusive of any on or off-board system for the safe operation of aerospace systems and aerospace vehicles. (<https://www.sae.org/works/committeeHome.do?comtID=TEAG34>)

References

Works Cited

- Belani, Hrvoje, Marin Vuković, and Željka Car. Requirements Engineering Challenges in Building AI-Based Complex Systems. 2019. IEEE 27th International Requirements Engineering Conference Workshops (REW).
- Breck, Eric, Shanqing Cai, Eric Nielsen, Michael Salib and D. Sculley. What's your ML Test Score? A Rubric for ML Production Systems. 2016. 30th Conference on Neural Information Processing Systems (NIPS 2016), Barcelona Spain.
- Daume III, Hal, and Daniel Marcu. Domain adaptation for statistical classifiers. *Journal of Artificial Intelligence Research*, 26:101–126, 2006.
- Dreossi, T., A. Donzé, S.A. Seshia. Compositional falsification of cyber-physical systems with machine learning components. In Barrett, C., M. Davies, T. Kahsai (eds.) NFM 2017. LNCS, vol. 10227, pp. 357-372. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-57288-8_26
- Faria, José M. Machine learning safety: An overview. In *Proceedings of the 26th Safety-Critical Systems Symposium*, York, UK, February 2018.
- Farrell, M., Luckcuck, M., Fisher, M. Robotics and Integrated Formal Methods. Necessity Meets Opportunity. In: *Integrated Formal Methods*. pp. 161-171. Springer (2018).
- Gao, Jerry, Chunli Xie, and Chuanqi Tao. 2016. Big Data Validation and Quality Assurance – Issues, Challenges and Needs. 2016 IEEE Symposium on Service-Oriented System Engineering (SOSE), Oxford, UK, 2016, pp. 433-441, doi: 10.1109/SOSE.2016.63.
- Gleirscher, M., Foster, S., Woodcock, J. New Opportunities for Integrated Formal Methods. *ACM Computing Surveys* 52(6), 1-36 (2020).
- Goodfellow, Ian, J. Shlens, C. Szegedy. Explaining and harnessing adversarial examples. In International Conference on Learning Representations (ICLR), May 2015.
- Gunning, D. Explainable Artificial Intelligence (XAI). In IJCAI 2016 Workshop on Deep Learning for Artificial Intelligence (DLAI), July 2016.
- Huang, X., M. Kwiatkowska, S. Wang, and M. Wu. Safety Verification of deep neural networks. In Majumdar, R., and V. Kunčák (eds.) CAV 2017. LNCS, vol. 10426, pp. 3-29. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-63387-9_1
- Ishikawa, Fuyuki and Nobukazu Yoshioka. How do Engineers Perceive Difficulties in Engineering of Machine-Learning Systems? - Questionnaire Survey. 2019 IEEE/ACM Joint 7th International Workshop on Conducting Empirical Studies in Industry (CESI) and 6th International Workshop on Software Engineering Research and Industrial Practice (SER&IP) (2019)
- Jones, Cliff B. Tentative steps toward a development method for interfering programs. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 5(4):596–619, 1983.
- Kelly, T., and R. Weaver. The goal structuring notation – a safety argument notation. In Dependable Systems and Networks 2004 Workshop on Assurance Cases, July 2004.
- Klein, G., Andronick, J., Fernandez, M., Kuz, I., Murray, T., Heiser, G. Formally verified software in the real world. *Comm. of the ACM* 61(10), 68-77 (2018).

- Kuwajima, Hiroshi, Hirotoshi Yasuoka, and Toshihiro Nakae. Engineering problems in machine learning systems. *Machine Learning* (2020) 109:1103–1126. <https://doi.org/10.1007/s10994-020-05872-w>
- Lwakatare, Lucy Ellen, Aiswarya Raj, Ivica Crnkovic, Jan Bosch, and Helena Holmström Olsson. Large-scale machine learning systems in real-world industrial settings: A review of challenges and solutions. *Information and Software Technology* 127 (2020) 106368
- Luckcuck, M., Farrell, M., Dennis, L.A., Dixon, C., Fisher, M. Formal Specification and Verification of Autonomous Robotic Systems: A Survey. *ACM Computing Surveys* 52(5), 1-41 (2019).
- Marijan, Dusica and Arnaud Gotlieb. Software Testing for Machine Learning. The Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI-20) (2020)
- Miller, Tim. Explanation in artificial intelligence: Insights from the social sciences. arXiv Preprint arXiv:1706.07269. (2017).
- Pei, K., Y. Cao, J Yang, and S. Jana. DeepXplore: automated whitebox testing of deep learning systems. In The 26th Symposium on Operating Systems Principles (SOSP 2017), pp. 1-18, October 2017.
- Picardi, Chiara, Paterson, Colin, Hawkins, Richard David et al. (2020) Assurance Argument Patterns and Processes for Machine Learning in Safety-Related Systems. In: *Proceedings of the Workshop on Artificial Intelligence Safety* (SafeAI 2020). CEUR Workshop Proceedings, pp. 23-30.
- Pullum, Laura L., Brian Taylor, and Marjorie Darrah, *Guidance for the Verification and Validation of Neural Networks*, IEEE Computer Society Press (Wiley), 2007.
- Rozier, K.Y. Specification: The Biggest Bottleneck in Formal Methods and Autonomy. In: *Verified Software. Theories, Tools, and Experiments*. pp. 8-26. Springer (2016).
- Schumann, Johan, Pramod Gupta and Yan Liu. Application of neural networks in High Assurance Systems: A Survey. In *Applications of Neural Networks in High Assurance Systems*, Studies in Computational Intelligence, pp. 1-19. Springer, Berlin, Heidelberg, 2010.
- Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Machine Learning: the high interest credit card of technical debt. In NIPS 2014 Workshop on Software Engineering for Machine Learning (SE4ML), December 2014.
- Seshia, Sanjit A. Compositional verification without compositional specification for learning-based systems. Technical Report UCB/EECS-2017-164, EECS Department, University of California, Berkeley, Nov 2017.
- Seshia, Sanjit A., Dorsa Sadigh, and S. Shankar Sastry. Towards Verified Artificial Intelligence. arXiv:1606.08514v4 [cs.AI] 23 Jul 2020.
- Szegedy, Christian, Zaremba, Wojciech, Sutskever, Ilya, Bruna, Joan, Erhan, Dumitru, Goodfellow, Ian J., and Fergus, Rob. Intriguing properties of neural networks. ICLR, abs/1312.6199, 2014b. URL <https://arxiv.org/abs/1312.6199>.
- Taylor, Brian, ed. *Methods and Procedures for the Verification and Validation of Artificial Neural Networks*, Springer-Verlag, 2005.
- Thompson, E. (2007). *Mind in life: Biology, phenomenology, and the sciences of mind*. Cambridge, MA: Harvard University Press.
- Tiwari, Ashish, Bruno Dutertre, Dejan Jovanović, Thomas de Candia, Patrick D. Lincoln, John Rushby, Dorsa Sadigh, and Sanjit Seshia. Safety envelope for security. In *Proceedings of the 3rd International Conference on High Confidence Networked Systems* (HiCoNS), pp. 85-94, Berlin, Germany, April 2014. ACM.
- Uesato, Jonathan, O'Donoghue, Brendan, van den Oord, Aaron, Kohli, Pushmeet. Adversarial Risk and the Dangers of Evaluating Against Weak Attacks. *Proceedings of the 35th International Conference on Machine Learning*, Stockholm, Sweden, PMLR 80, 2018.
-

- Varshney, Kush R., and Homa Alemzadeh. On the safety of machine learning: Cyber-physical systems, decision sciences, and data products. *Big Data*, 5(3):246–255, 2017.
- Webster, M., Wester, D.G., Araiza-Illan, D., Dixon, C., Eder, K., Fisher, M., Pipe, A.G. A corroborative approach to verification and validation of human-robot teams. *J. Robotics Research* 39(1) (2020).
- Xie, Xiaoyuan, J.W.K. Ho, C. Murphy, G. Kaiser, B. Xu, and T.Y. Chen. 2011. “Testing and Validating Machine Learning Classifiers by Metamorphic Testing,” *Journal of Software Testing*, April 1, 84(4): 544-558, doi:10.1016/j.jss.2010.11.920.
- Zhang, J., Li, J. Testing and verification of neural-network-based safety-critical control software: A systematic literature review. *Information and Software Technology* 123, 106296 (2020).
- Zhang, J.M., Harman, M., Ma, L., Liu, Y. Machine learning testing: Survey, landscapes and horizons. *IEEE Transactions on Software Engineering*. 2020, doi: 10.1109/TSE.2019.2962027.

Primary References

- Belani, Hrvoje, Marin Vuković, and Željka Car. Requirements Engineering Challenges in Building AI-Based Complex Systems. 2019. IEEE 27th International Requirements Engineering Conference Workshops (REW).
- Dutta, S., Jha, S., Sankaranarayanan, S., Tiwari, A. 2018. Output range analysis for deep feedforward neural networks. In: NASA Formal Methods. pp. 121-138.
- Gopinath, D., G. Katz, C. Păsăreanu, and C. Barrett. 2018. DeepSafe: A Data-Driven Approach for Assessing Robustness of Neural Networks. In: ATVA.
- Huang, X., M. Kwiatkowska, S. Wang and M. Wu. 2017. Safety Verification of Deep Neural Networks. Computer Aided Verification.
- Jha, S., V. Raman, A. Pinto, T. Sahai, and M. Francis. 2017. On Learning Sparse Boolean Formulae for Explaining AI Decisions, *NASA Formal Methods*.
- Katz, G., C. Barrett, D. Dill, K. Julian, M. Kochenderfer. 2017. Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks, <https://arxiv.org/abs/1702.01135>.
- Leofante, F., N. Narodytska, L. Pulina, A. Tacchella. 2018. Automated Verification of Neural Networks: Advances, Challenges and Perspectives, <https://arxiv.org/abs/1805.09938> Marijan, Dusica and Arnaud Gotlieb. Software Testing for Machine Learning. The Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI-20) (2020)
- Mirman, M., T. Gehr, and M. Vechev. 2018. Differentiable Abstract Interpretation for Provably Robust Neural Networks. *International Conference on Machine Learning*.
- Pullum, Laura L., Brian Taylor, and Marjorie Darrah, *Guidance for the Verification and Validation of Neural Networks*, IEEE Computer Society Press (Wiley), 2007.
- Seshia, Sanjit A., Dorsa Sadigh, and S. Shankar Sastry. Towards Verified Artificial Intelligence. arXiv:1606.08514v4 [cs.AI] 23 Jul 2020.
- Taylor, Brian, ed. *Methods and Procedures for the Verification and Validation of Artificial Neural Networks*, Springer-Verlag, 2005.
- Xiang, W., P. Musau, A. Wild, D.M. Lopez, N. Hamilton, X. Yang, J. Rosenfeld, and T. Johnson. 2018. Verification for Machine Learning, Autonomy, and Neural Networks Survey. <https://arxiv.org/abs/1810.01989>
- Zhang, J., Li, J. Testing and verification of neural-network-based safety-critical control software: A systematic literature review. *Information and Software Technology* 123, 106296 (2020).

Additional References

- Jha, Sumit Kumar, Susmit Jha, Rickard Ewetz, Sunny Raj, Alvaro Velasquez, Laura L. Pullum, and Ananthram Swami. An Extension of Fano's Inequality for Characterizing Model Susceptibility to Membership Inference Attacks. arXiv:2009.08097v1 [cs.LG] 17 Sep 2020.
- Sunny Raj, Mesut Ozdag, Steven Fernandes, Sumit Kumar Jha, Laura Pullum, "On the Susceptibility of Deep Neural Networks to Natural Perturbations," *AI Safety 2019* (held in conjunction with IJCAI 2019 - International Joint Conference on Artificial Intelligence), Macao, China, August 2019.
- Ak, R., R. Ghosh, G. Shao, H. Reed, Y.-T. Lee, L.L. Pullum. "Verification-Validation and Uncertainty Quantification Methods for Data-Driven Models in Advanced Manufacturing," *ASME Verification and Validation Symposium*, Minneapolis, MN, 2018.
- Pullum, L.L., C.A. Steed, S.K. Jha, and A. Ramanathan. "Mathematically Rigorous Verification and Validation of Scientific Machine Learning," *DOE Scientific Machine Learning Workshop*, Bethesda, MD, Jan/Feb 2018.
- Ramanathan, A., L.L. Pullum, Zubir Husein, Sunny Raj, Neslisah Totosdagli, Sumanta Pattanaik, and S.K. Jha. 2017. "Adversarial attacks on computer vision algorithms using natural perturbations." In *2017 10th International Conference on Contemporary Computing (IC3)*. Noida, India. August 2017.
- Raj, S., L.L. Pullum, A. Ramanathan, and S.K. Jha. 2017. "Work in Progress: Testing Autonomous cyber-physical systems using fuzzing features derived from convolutional neural networks." In *ACM SIGBED International Conference on Embedded Software (EMSOFT)*. Seoul, South Korea. October 2017.
- Raj, S., L.L. Pullum, A. Ramanathan, and S.K. Jha, "SATYA: Defending against Adversarial Attacks using Statistical Hypothesis Testing," in *10th International Symposium on Foundations and Practice of Security (FPS 2017)*, Nancy, France. (Best Paper Award), 2017.
- Ramanathan, A., Pullum, L.L., S. Jha, et al. "Integrating Symbolic and Statistical Methods for Testing Intelligent Systems: Applications to Machine Learning and Computer Vision." *IEEE Design, Automation & Test in Europe (DATE)*, 2016.
- Pullum, L.L., C. Rouff, R. Buskens, X. Cui, E. Vassiv, and M. Hinchey, "Verification of Adaptive Systems," *AIAA Infotech@Aerospace 2012*, April 2012.
- Pullum, L.L., and C. Symons, "Failure Analysis of a Complex Learning Framework Incorporating Multi-Modal and Semi-Supervised Learning," In *IEEE Pacific Rim International Symposium on Dependable Computing (PRDC 2011)*, 308-313, 2011.
- Haglich, P., C. Rouff, and L.L. Pullum, "Detecting Emergent Behaviors with Semi-Boolean Algebra," *Proceedings of AIAA Infotech @ Aerospace*, 2010.
- Pullum, L.L., Marjorie A. Darrah, and Brian J. Taylor, "Independent Verification and Validation of Neural Networks – Developing Practitioner Assistance," *Software Tech News*, July 2004.
-

Transitioning Systems Engineering to a Model-based Discipline

Systems engineers have always leveraged many kinds of models, including functional models to support requirements development, simulation models to analyze the behavior of systems, and other analytical models to analyze various aspects of the system such as reliability, safety, mass properties, power consumption, and cost. However, the discipline still relies heavily on document-based artifacts to capture much of the system specification and design information, such as requirements, interface control documentation, and system architecture design descriptions. This information is often spread across many different documents including text, informal drawings, and spreadsheets. This document-based approach to systems engineering suffers from a lack of precision, inconsistencies from one artifact to another, and difficulties in maintaining and reusing the information.

Model-Based Systems Engineering

Model-based systems engineering (MBSE) is the formalized application of modeling to support system requirements, design, analysis, verification, and validation activities beginning in the conceptual design phase and continuing through development and later life cycle phases (INCOSE 2007). A distinguishing characteristic of an MBSE approach is that the model constitutes a primary artifact of the systems engineering process. The focus on developing, managing and controlling a model of the system is a shift from the traditional document-based approach to systems engineering, where the emphasis is on producing and controlling documentation about the system. By leveraging the system model as a primary artifact, MBSE offers the potential to enhance product quality, enhance reuse of the system modeling artifacts, and improve communications among the systems development team. This, in turn, offers the potential to reduce the time and cost to integrate and test the system, and significantly reduce cost, schedule, and risks in fielding a system.

MBSE includes a diverse set of descriptive and analytical models that can be applied throughout the life cycle, and from system of systems (SoS) modeling down to component modeling. Typical models may include descriptive models of the system architecture that are used to specify and design the system, and analytical models to analyze system performance, physical characteristics, and other quality characteristics such as reliability, maintainability, safety, and cost.

MBSE has been evolving for many years. The term MBSE was used by Wayne Wymore in his book by this name (Wymore 1993), that provided a state-based formalism for analyzing systems in terms of their input/output characteristics, and value functions for assessing utility of technology independent and technology dependent systems. Simulations have been extensively used across industry to provide high fidelity performance analysis of complex systems. The Standard for Integration Definition for Function Modeling (IDEF0 1993) was introduced in the 1990's to support basic functional modeling. A modeling formalism called the enhanced functional flow block diagram (Long 2000) has been used to model many different types of systems. The Object Management Group (OMG) introduced the concept of a Model Driven Architecture (MDA®) (OMG 2003) that leverages a standards-based approach to modeling. The Systems Modeling Language (OMG SysML™) (OMG 2015) was adopted by the OMG in 2006 as a general-purpose systems modeling language. In addition, the Unified Profile for DoDAF and MODAF (UPDM) (OMG 2013) was adopted by the OMG in 2008 to support enterprise modeling. Several other domain specific modeling languages have been introduced as well.

MBSE Transition

The INCOSE Systems Engineering Vision 2025 (INCOSE 2025, pg 38) describes the current state of MBSE as follows: "Model-based systems engineering has grown in popularity as a way to deal with the limitations of document-based approaches, but is still in an early stage of maturity similar to the early days of CAD/CAE."

SE Vision 2025 also describes a continuing transition of SE to a model-based discipline in which: "Formal systems modeling is standard practice for specifying, analyzing, designing, and verifying systems, and is fully integrated with other engineering models. System models are adapted to the application domain, and include a broad spectrum of models for representing all aspects of systems. The use of internet driven knowledge representation and immersive technologies enable highly efficient and shared human understanding of systems in a virtual environment that span the full life cycle from concept through development, manufacturing, operations, and support." The transition to a more model-based discipline is not without its challenges. This requires both advancements in the practice, and the need to achieve more widespread adoption of MBSE within organizations across industry sectors.

The INCOSE Systems Engineering Vision 2035 (INCOSE 2035, pg 33) states that "The Future of Systems Engineering Is Predominantly Model-Based". Further discussion goes on to project that "Systems engineers routinely compose task-specific virtual models using ontologically linked, digital twin-based model-assets. These connected models are updated in real-time providing a virtual reality-based, immersive design and exploration space. This virtual global collaboration space is cloud-based, enabled by modelling as a service and supports massive simulation leveraging cloud-based high-capacity compute infrastructure. Families of unified ModSim frameworks exist enabling small and medium businesses along with Government agencies to collaborate."

Advancing the practice requires improvements in the modeling languages, methods, and tools. The modeling languages must continue to improve in terms of their expressiveness, precision, and usability. MBSE methods, such as those highlighted in A Survey of Model-Based Systems Engineering (MBSE) Methodologies (Estefan 2008), have continued to evolve, but require further advancements to provide a rigorous approach to modeling a system across the full system lifecycle, while being more adaptable to a diverse range of application domains. The modeling tools must also continue to evolve to support the modeling languages and methods, and to integrate with other multi-disciplinary engineering models and tools in support of the broader model-based engineering effort. The movement towards increased use of modeling standards, that are more widely available in commercial tools, and rigorous model-based methodologies, increase the promise of MBSE.

The adoption of MBSE requires a workforce that is skilled in the application of MBSE. This requires organizations to provide an infrastructure that includes MBSE methods, tools, and training, and a commitment to deploy this capability to their programs. As with any organizational change, this must be approached strategically to grow this capability and learn from their experiences.

Like other engineering disciplines, the transition of systems engineering to a model-based discipline is broadly recognized as essential to meet the challenges associated with increasing system complexity and achieving the productivity and quality improvements. The SEBoK will continue to reflect the growing body of knowledge to facilitate this transition.

References

Works Cited

- Estefan, J. 2008. *A Survey of Model-Based Systems Engineering (MBSE) Methodologies*, rev. B. Seattle, WA: International Council on Systems Engineering.
- INCOSE-TD-2007-003-02. Available at: https://www.omg.sysml.org/MBSE_Methodology_Survey_RevB.pdf. Accessed April 13, 2015.
- INCOSE 2020. INCOSE Technical Operations. 2007. *Systems Engineering Vision 2020*, version 2.03. Seattle, WA: International Council on Systems Engineering, Seattle, WA, INCOSE-TP-2004-004-02.
- INCOSE 2035, INCOSE Technical Operations. 2021. *Systems Engineering Vision 2035* ^[1]. Seattle, WA: International Council on Systems Engineering, Seattle, WA.
- International Council on Systems Engineering (INCOSE) 2014. *Systems Engineering Vision 2025* July, 2014. Available at <https://www.incose.org/docs/default-source/about/se-vision-2025.pdf?sfvrsn=4>. Accessed February 16.
- Long, James E. 2000, *Systems Engineering (SE) 101, CORE®: Product & Process Engineering Solutions*, Vitech Training Materials, Vienna, VA, USA: Vitech Corporation.
- Object Management Group (OMG). 2003. *Model-Driven Architecture (MDA) Guide*, v1.01, June 12 2003. Available at: <https://www.omg.org/mda/>.
- Object Management Group (OMG). 2015. *OMG Systems Modeling Language (OMG SysML™)*, V1.4. OMG document number formal/2015-06-03, September 2015. Available at: <https://www.omg.org/spec/SysML/1.4/>.
- Object Management Group (OMG). 2013. *Unified Profile for DoDAF/MODAF (UPDM) OMG document*. number formal/2013-08-04, August 2013. Available at: <https://www.omg.org/spec/UPDM/2.1/>.
- Computer Systems Laboratory of the National Institute of Standards and Technology (NIST). 1993. *Standard for Integration Definition for Function Modeling (IDEF0)*, Draft Federal Information Processing Standards, Publication 183, December 21, 1993.
- Wymore, W. 1993. *Model-Based Systems Engineering*. Boca Raton, FL: CRC Press.

Primary References

- INCOSE. 2015. *[INCOSE Systems Engineering Handbook] Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, Section 9.2: Model-Based Systems Engineering, version 4.0. Hoboken, NJ, USA: John Wiley and Sons, Inc, ISBN: 978-1-118-99940-0.

Additional References

- OMG. *MBSE Wiki*. Available at: <https://www.omgwiki.org/MBSE/doku.php>. Accessed February 16, 2015.

References

- [1] <https://www.incose.org/about-systems-engineering/se-vision-2035>

Model-Based Systems Engineering Adoption Trends 2009-2018

- Lead Author:
- Rob Cloutier
- Contributing Author:
- Ifezue Obiako

The MBSE Initiative was kicked off at the INCOSE International Workshop (IW) in 2007 at the Albuquerque, NM, USA Embassy Suites. There were approximately 45 INCOSE members for this first meeting, held during the two days preceding IW.

Surveys were conducted in 2009, 2012, 2014, 2018, and 2019 to better understand the adoption trends of model-based systems engineering.

Introduction

Model-based systems engineering (MBSE) is not a new concept. Wymore (1993) published the seminal work on the topic. This book presents the mathematical theory behind MBSE. Since that time, engineering has made significant movement from text-based approaches using office-based tools (e.g. Harvard Graphics, Microsoft PowerPoint, Microsoft Visio, etc.) to an interconnected set of graphical diagrams. These diagrams are generally created in a tool with a specialized graphical user interface.

Today aerospace engineers no longer use drafting boards to create their drawings – they use computer aided design (CAD) tools. Likewise, software engineers seldom use EMACS or Vi (text editors), instead, they use software GUIs that allow them to code, check syntax, compile, link, and run their software all in a single environment.

Broadly speaking, a model_ can be thought of as a facsimile or abstraction of reality. To this end, even a requirements document can be considered a model – it represents what a real system should do in performing its mission or role. While systems engineering has used models for a very long time, MBSE is the systems engineering migration to computer-based graphical user interfaces to perform our analysis and design tasks just as our other engineering brethren have moved to computer-based graphical user interfaces.

A discussion of available tools is beyond the scope of this article, and it is not the practice of the SEBoK to review or promote specific tool offerings. However, it is fair to state that current MBSE tools fall into three broad categories: 1) Functional decomposition tools that use IDEF0 (also called IPO) diagrams, N2 diagrams, functional flow block diagrams, etc., 2) Object-oriented tools that implement the Object Management Group's Systems Modeling Language (SysML), and 3) Mathematical modeling tools.

This migration for systems engineering might have begun in the late 90's. The INCOSE INSIGHT publication proclaimed that MBSE was a new paradigm (INSIGHT 1998). Cloutier (2004) addressed the migration from a waterfall systems engineering approach to an object-oriented approach on the Navy Open Architecture project. At that time, SysML did not exist, and the teams were using the Unified Modeling Language (UML) that was predominately a software modeling tool. Zdanis & Cloutier (2007a, 2007b) addressed the use of activity diagrams instead of sequence diagrams for systems engineering based on the newly released SysML. In 2009, the INCOSE INSIGHT publication proclaimed MBSE was THE new paradigm (INSIGHT 2009).

Approach

In 2009, a survey was commissioned by the Object Management Group (OMG) with the intent of informing the SysML Working Group on necessary changes to SysML since its first release (Cloutier & Bone 2010). That survey focused on process more than adoption. Beginning in 2012, INCOSE has commissioned three more surveys to understand adoption trends and obstacles. The survey instrument remained relatively unchanged for 2012, 2014, and 2018 (Cloutier 2015, Cloutier 2019a). In January of 2019, the Jet Propulsion Lab (JPL) conducted an MBSE Workshop (Cloutier 2019b). A survey of those participants was conducted, and the intent of the questions was to augment knowledge gained from the 2018 survey. The table below shows the number of respondents in each of the surveys.

Table 1. MBSE Survey Purposes and Responses (SEBoK Original)

Year	Survey Purpose	Responses
2012	INCOSE MBSE Initiative	134
2014	INCOSE MBSE Initiative	205
2018	INCOSE MBSE Initiative	661
2019	JPL MBSE Workshop	98

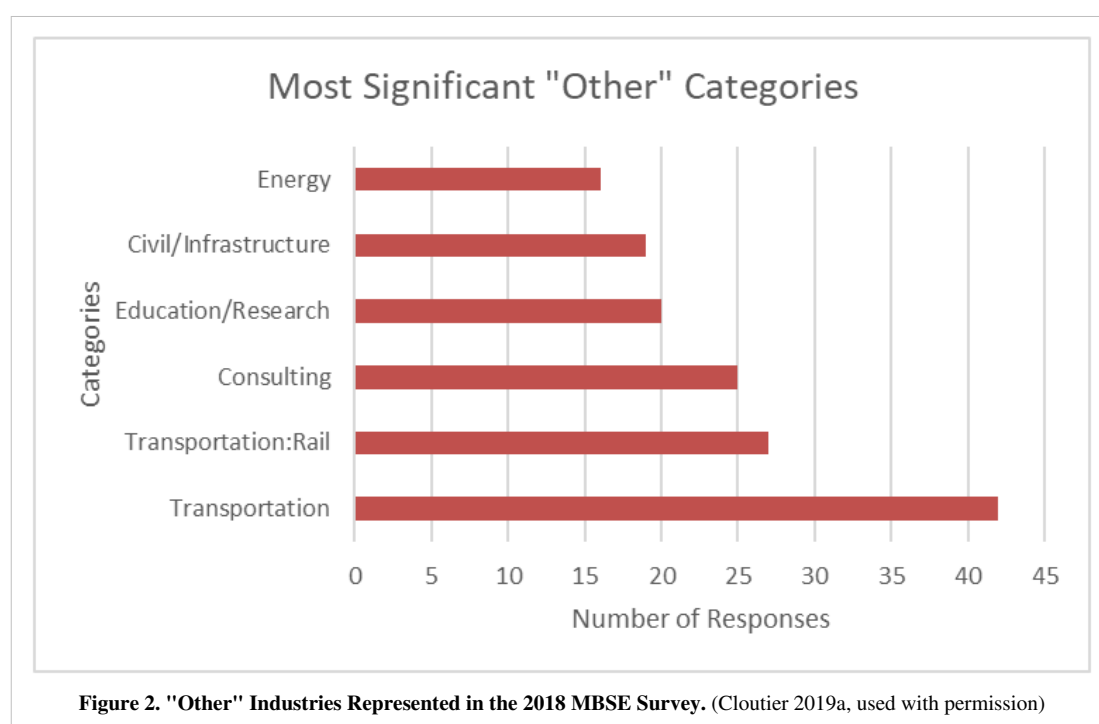
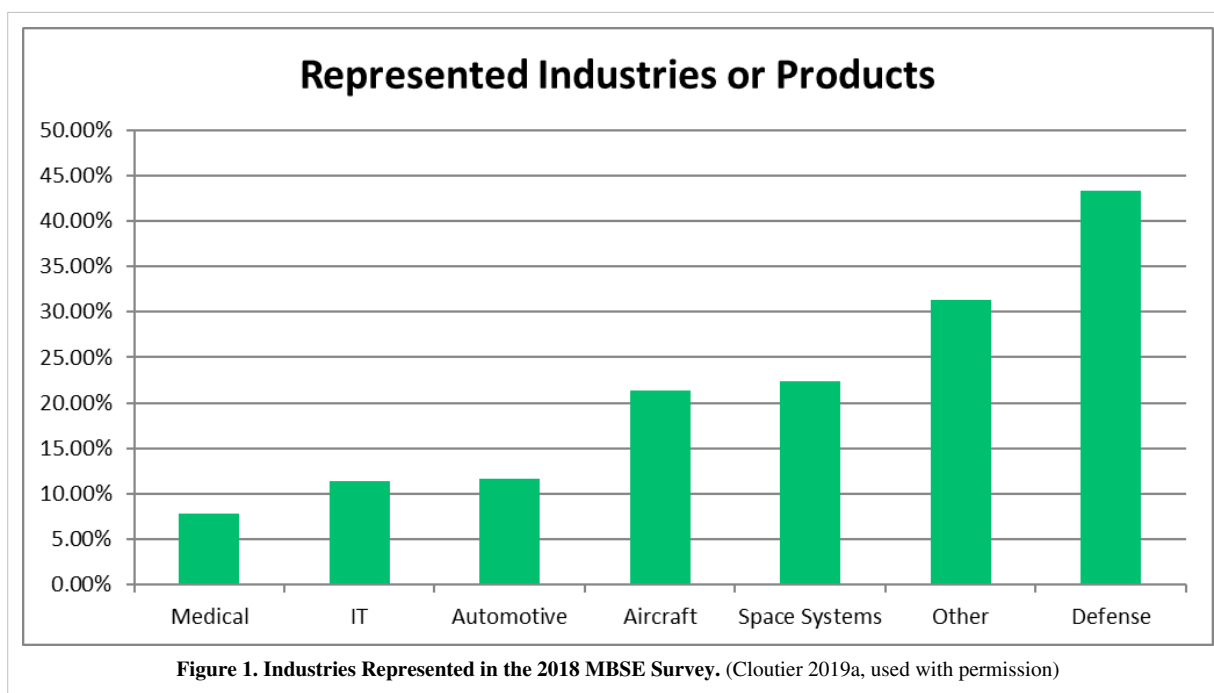
Responses and Response Demographics

Each survey was sent to a diverse group of MBSE practitioners. Table 2 shows that of the 661 responses for the 2018 survey, 410 indicated their country of origin. This international representation is similar to all surveys conducted.

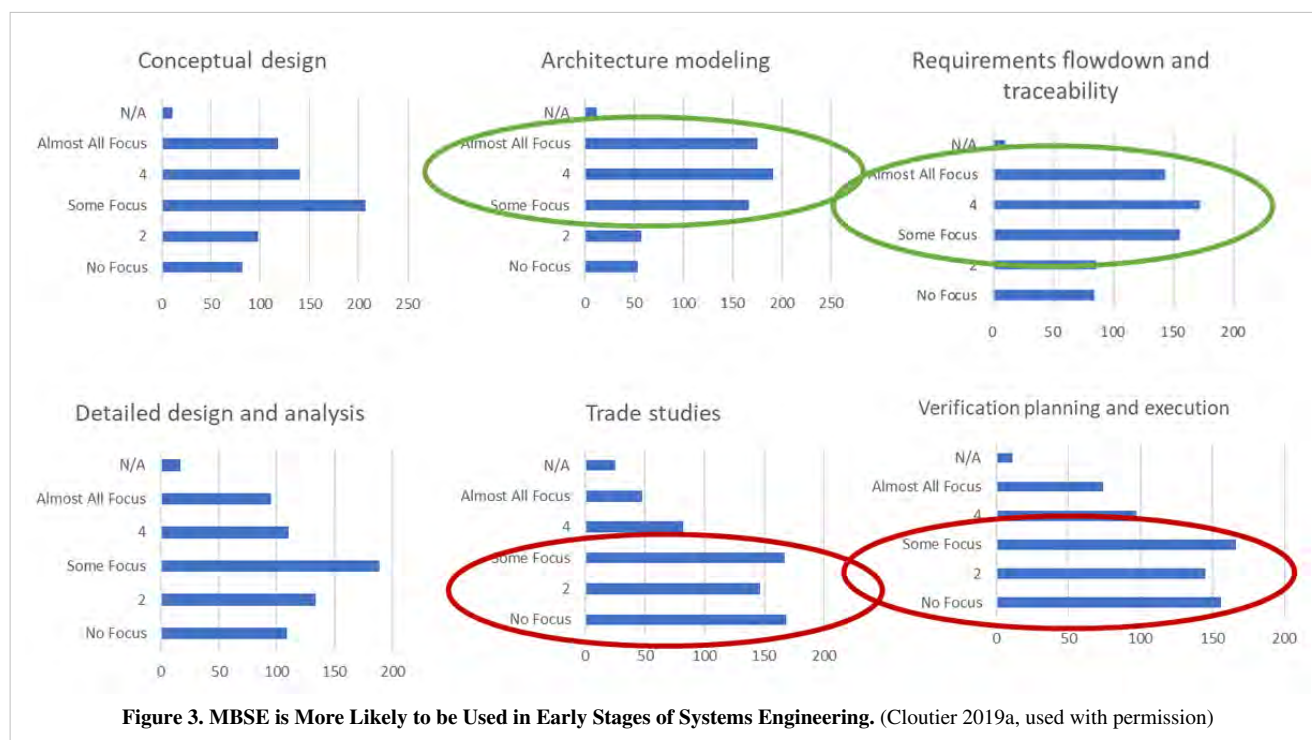
Table 1. MBSE Survey Purposes and Responses (Cloutier 2019, used with permission)

Country	Responses	Country	Responses
USA	197	Israel	4
United Kingdom	52	Singapore	3
France	30	China	2
Germany	28	New Zealand	2
Australia	20	Poland	2
Netherlands	19	Russia	1
Japan	8	Romania	1
Canada	6	Turkey	1
Italy	6	Columbia	1
Sweden	6	Norway	1
South Africa	5	South Korea	1
Switzerland	4	UAE	1
Brazil	4	Belarus	1
India	4		

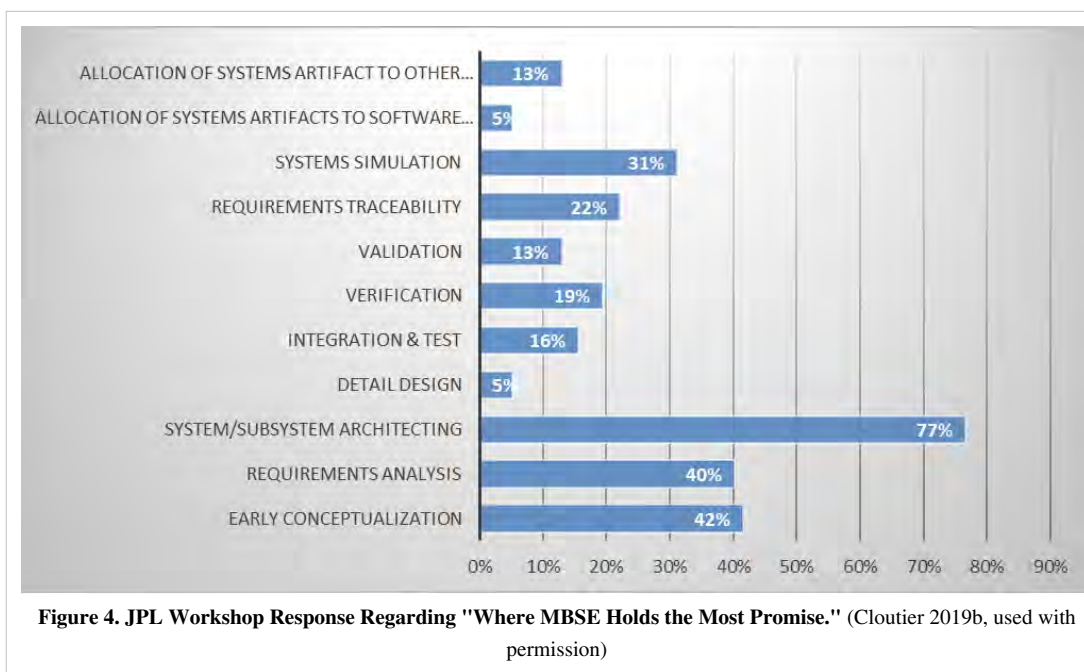
As part of the demographics, Figure 1 shows the represented industries. Because the “Other” category was so large, the data was analyzed to better understand Figure 2.

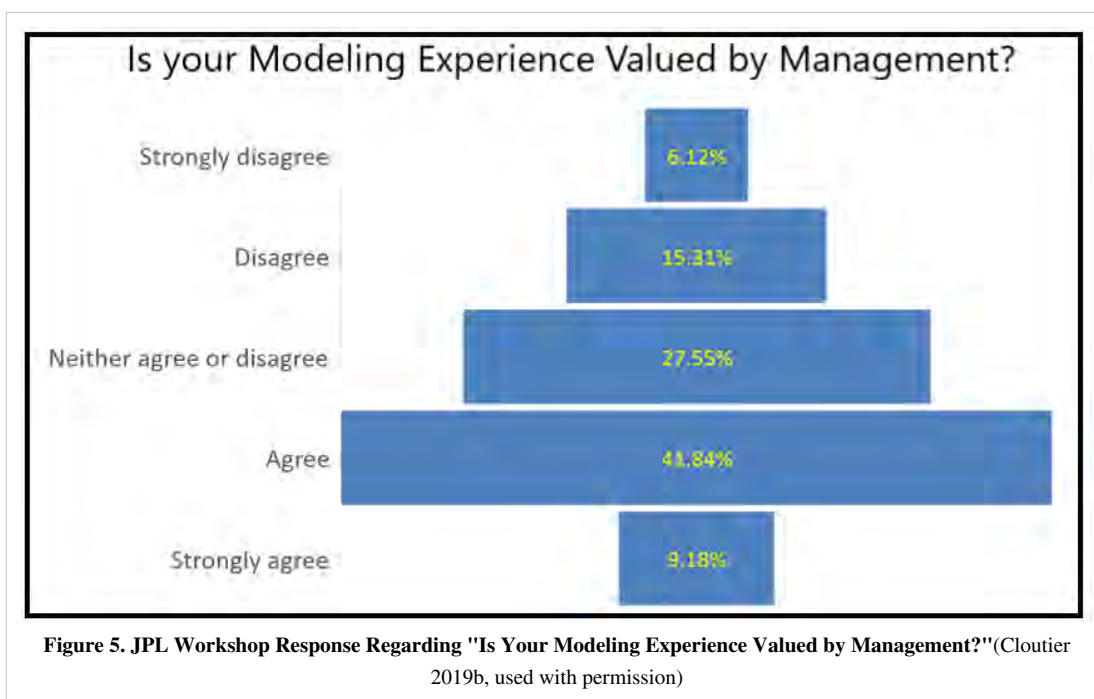


The 2018 survey indicated that there seems to be an increased application of MBSE in traditionally civil engineering industries – specifically energy, infrastructure, and transportation (Figure 2) One of the most interesting aspects of the 2018 survey is the finding that MBSE is being applied in the early phases of systems engineering, and less so in the later phases as shown in Figure 3.



This was confirmed by the JPL question “Where do we believe MBSE holds the most promise?” Figure 4 shows that 76% of the responses indicated system/subsystem architecting, 42% thought requirements analysis, and 39% believed early conceptualization (note: the question allowed for multiple answers).

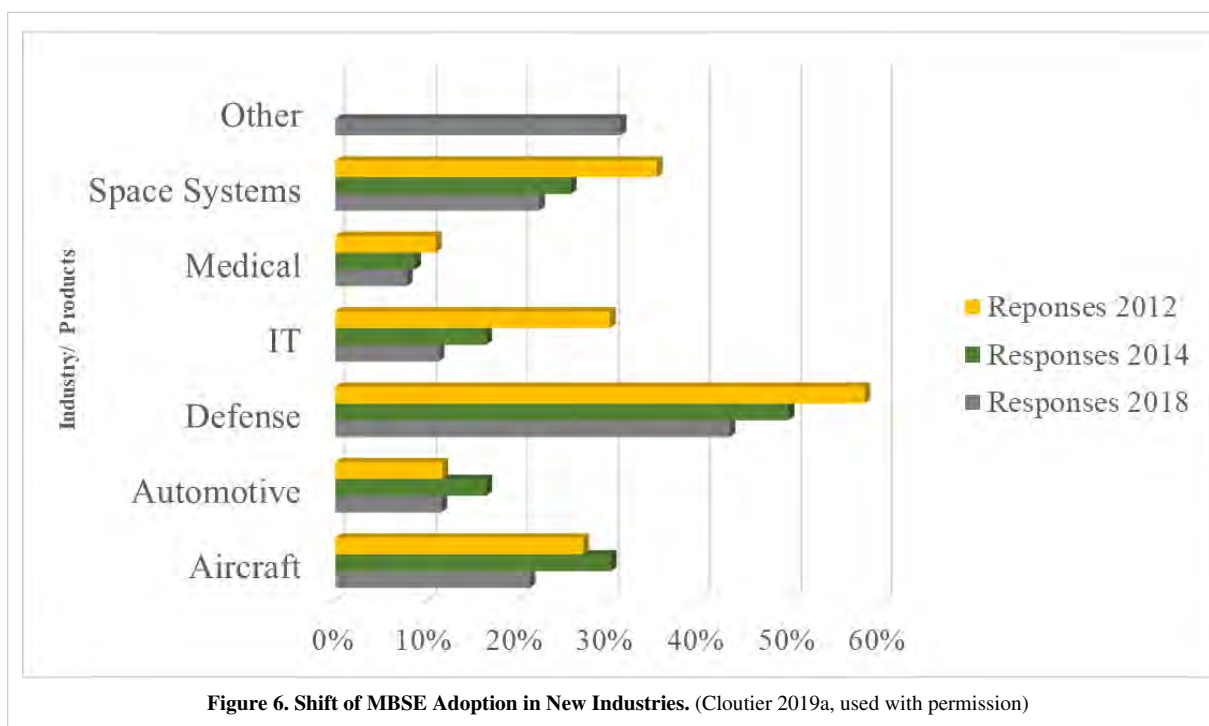




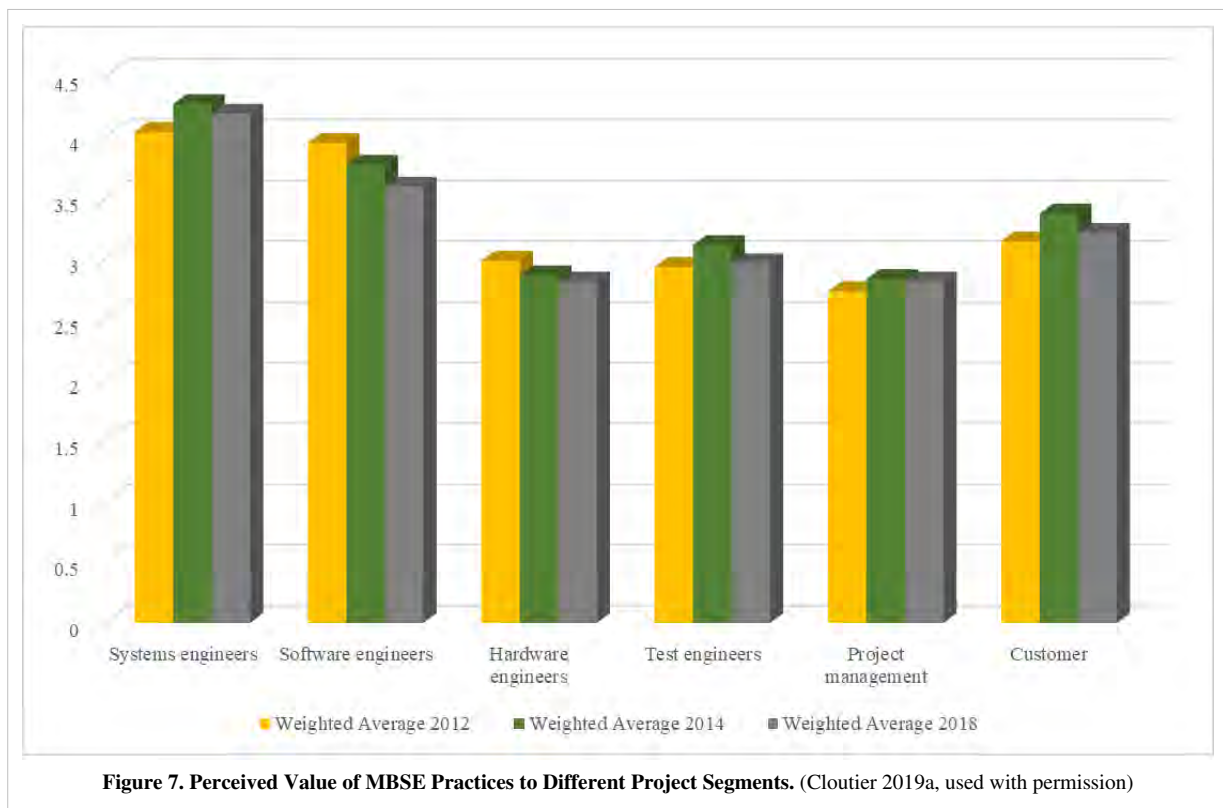
When asked whether the JPL survey respondents believed that their systems modeling experience is recognized as a valued skill supporting career growth of systems engineers in their organization, just over 50% believed management valued their experience. A smaller number, 21%, believed their modeling experience was not valued (Figure 5).

Key Adoption Trends

The remainder of this article will look at some of the trends identified across the surveys, from 2009 to 2018. Figure 6 shows that MBSE is moving from a defense and space dominated practice into other industries as discussed in Figure 4.



Model-based systems engineering seems to be expanding in influence in that it is not just in the purview of systems engineers. While systems and software engineers find value in MBSE practices, Figure 6 demonstrates that the customer is finding value in MBSE practices. It is also interesting that software engineers' perceived value of MBSE is declining from survey to survey.



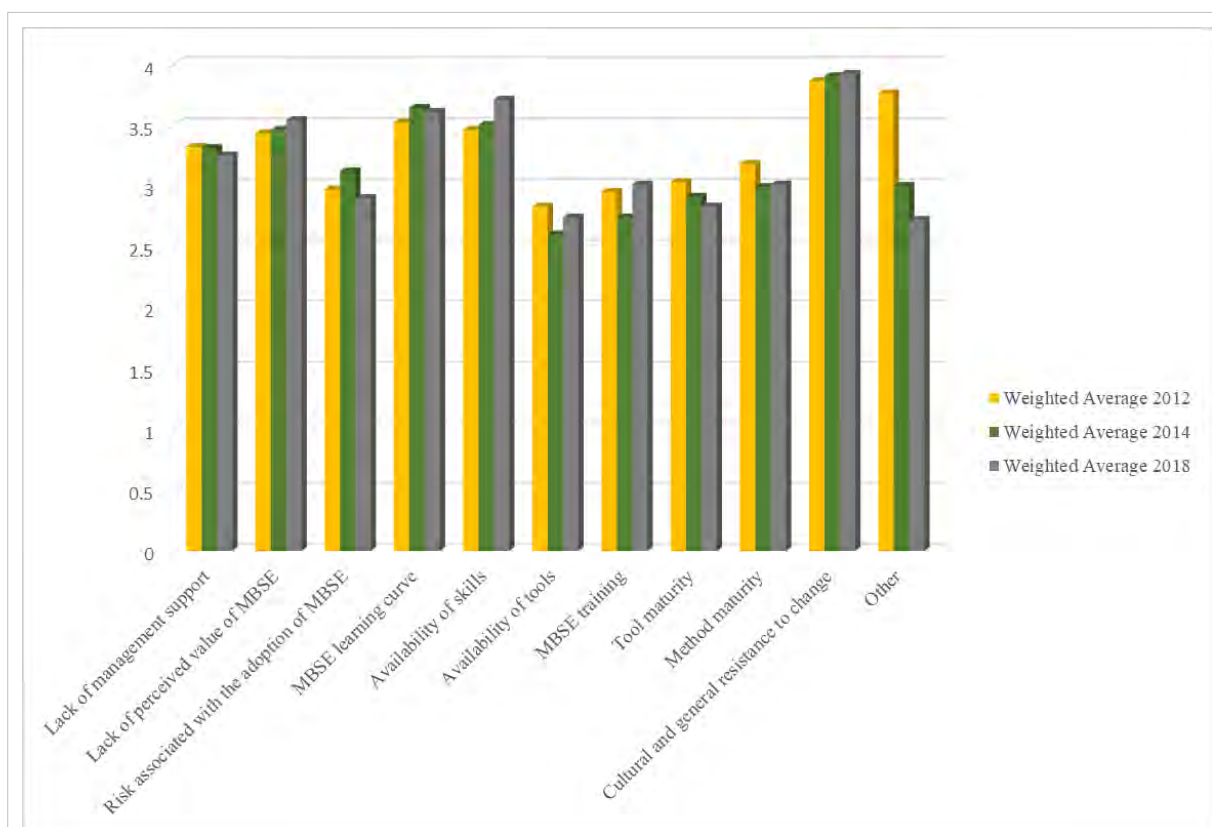


Figure 8. Inhibitors to the Successful Adoption of MBSE within an Organization/Company. (Cloutier 2019a, used with permission)

Figure 8 demonstrates that availability of MBSE skills and cultural and general resistance to change have continued to increase. Lack of perceived value reflects the findings in Figure 6 – software and hardware engineers are not seeing the value of MBSE.

Conclusions

Surveys conducted between 2012 and 2018 demonstrate that MBSE practices are spreading beyond traditional Defense and Space domains. Most MBSE practitioners are finding MBSE is most useful in the early project phases of conceptualization, requirements analysis, and systems architecting. There continues to be a skills shortage, yet companies/organizations are providing less training to improve MBSE skills. Both systems engineers, systems engineering management, and the systems engineering customer are finding value in using models to perform systems engineering.

References

Works Cited

- Cloutier, R. 2004. "Migrating from a waterfall systems engineering approach to an object oriented approach – Lessons learned," ICSE and INCOSE Region II Conference, Las Vegas, NV, USA, September 15, 2004.
- Cloutier, R. and M. Bone. 2010. *Compilation of SysML RFI- Final Report: Systems Modeling Language (SysML) Request For Information*. OMG Document: syseng/2009-06-01. Report date: 02/20/2010. Available at: https://www.omgwiki.org/OMGSysML/lib/exe/fetch.php?media=sysml-roadmap:omg_rfi_final_report_02_20_2010-1.pdf Accessed October 24, 2019.
- Cloutier, R. 2015. "Current modeling trends in systems engineering," *INSIGHT, A Publication of the International Council on Systems Engineering (INCOSE)*, vol. 18, no. 2.

- Cloutier, R. 2019a. "2018 MBSE survey results," In Proceedings of the 2019 INCOSE MBSE Workshop, presented at the INCOSE 2019 International Workshop, Torrance, CA, USA, January 26-29, 2019.
- Cloutier, R. 2019b. "2019 JPL MBSE Survey Results," Presented at the 2019 Jet Propulsion Lab (JPL) MBSE Workshop, Pasadena, CA, USA, January 2019.
- INSIGHT. 1998. *Model-Based Systems Engineering: A New Paradigm*. A Publication of the International Council on Systems Engineering (INCOSE). Volume 1, Issue 3.
- INSIGHT. 2009. *Special Edition on Model-based Systems Engineering: The New Paradigm*. A Publication of the International Council on Systems Engineering (INCOSE). Volume 12, Issue 4.
- Wymore, W. 1993. *Model-Based Systems Engineering*. Boca Raton, FL, USA: CRC Press.
- Zdanis, L. and R. Cloutier. 2007. "The Use of Behavioral Diagrams in SysML." Hoboken, NJ: Stevens Institute of Technology. Proceedings of the Conference on Systems Engineering Research (CSER) 2007, 14-16 March 2007, Hoboken, NJ, USA. ISBN 0-9787122-1-8.
- Zdanis, L. and R. Cloutier. 2007. "The Use of Behavioral Diagrams in SysML." IEEE Long Island Systems, Applications and Technology Conference, Farmingdale, NY, 2007, pp. 1-1. doi: 10.1109/LISAT.2007.4312634

Primary References

- Bone, M. and R. Cloutier. 2010. "The current state of model based systems engineering: Results from the OMG™ SysML request for information 2009," in Proceedings of the 8th Conference on Systems Engineering Research (CSER), Hoboken, NJ, USA, March 17-19, 2010. Available at: https://www.omg-sysml.org/SysML_2009_RFI_Response_Summary-bone-cloutier.pdf.
- Cloutier, R. and M. Bone. 2012. "MBSE survey 2," Presented at the International Council on Systems Engineering (INCOSE) International Workshop, Jacksonville, FL, USA, January 21-24, 2012.
- Cloutier, R. 2019a. "2018 MBSE survey results," In Proceedings of the 2019 INCOSE MBSE Workshop, presented at the INCOSE 2019 International Workshop, Torrance, CA, USA, January 26-29, 2019.
- Cloutier, R. 2019b. "2019 JPL MBSE survey results," Presented at the 2019 Jet Propulsion Lab (JPL) MBSE Workshop, Pasadena, CA, January 2019.

Additional References

- Batareseh, O., L. McGinnis, J. Lorenz. 2012. "MBSE supports manufacturing system design," In Proceedings of the 22nd Annual International Council of Systems Engineering (INCOSE) International Symposium, Rome, Italy, July 9-12, 2012. Paper ID: 2950.
- Kernschmidt, K., B. Vogel-Heuser. 2013. "An interdisciplinary SysML based modeling approach for analyzing change influences in production plants to support the engineering," In Proceedings of the 2013 IEEE International Conference on Automation Science and Engineering (CASE), August 17-20, 2013. pp. 1113-1118. doi: 10.1109/CoASE.2013.6654030.
- Gulan, S., S. Johr, R. Kretschmer, S. Rieger, M. Ditze. 2013. "Graphical modelling meets formal methods," In Proceedings of the 11th IEEE International Conference on Industrial Informatics (INDIN), July 29-31, 2013. pp. 716-721. doi: 10.1109/INDIN.2013.6622972.
- Hoffmann, H. 2012. "Streamlining the development of complex systems through model-based systems engineering," In Proceedings of the 2012 IEEE/AIAA 31st Digital Avionics Systems Conference (DASC), October 14-18, 2012. pp. 6E6-1, 6E6-8. doi: 10.1109/DASC.2012.6382404.
- Paredis, C.J.J. et al. 2010. "An overview of the SysML-Modelica transformation specification," In Proceedings of the 2010 International Council on Systems Engineering (INCOSE) International Symposium, Chicago, IL, USA, July 11-15, 2010.

Pihlanko, P., S. Sierla, K. Thramboulidis, M. Viitasalo. 2013. "An industrial evaluation of SysML: The case of a nuclear automation modernization project," In Proceedings of the 18th IEEE Conference on Emerging Technologies & Factory Automation (ETFA), September 10-13, 2013. pp. 1-8. doi: 10.1109/ETFA.2013.6647945.

Ramos, A.L., J.V. Ferreira, J. Barcelo. "Model-Based Systems Engineering: An Emerging Approach for Modern Systems," *IEEE Transactions on Systems, Man, and Cybernetics*, Part C: Applications and Reviews, vol. 42, no. 1, pp. 101-111. doi: 10.1109/TSMCC.2011.2106495.

Digital Engineering

- Lead Author:
- Ron Giachetti

The US Under Secretary of Defense for Research and Development released the US Department of Defense (DoD) Digital Engineering Strategy in June 2018 describing five goals to streamline the DoD acquisition process through the creation of a digital thread enabling the conception, design, and development of complex weapon systems (DoD 2018; Zimmerman 2017). The crux of digital engineering is the creation of computer readable models to represent all aspects of the system and to support all the activities for the design, development, manufacture, and operation of the system throughout its lifecycle. These computer models would have to be based on shared data schemata so that in effect a digital thread integrates all the diverse stakeholders involved in the acquisition of new weapon systems. The Digital Engineering Strategy anticipates digital engineering will lead to greater efficiency and improved quality of all the acquisition activities.

Relationship with MBSE

Model-based systems engineering (MBSE) is a subset of digital engineering. MBSE supports the systems engineering activities of requirements, architecture, design, verification, and validation. These models would have to be connected to the physics-based models used by other engineering disciplines such as mechanical and electrical engineering. One challenge remaining for digital engineering is the integration of MBSE with physics-based models.

Foundation to digital engineering is the representation of the system data in a format sharable between all stakeholders (Giachetti et al. 2015; Vaneman 2018). SysML 2.0 is one of several future developments promising to provide a representation sufficient to support digital engineering. An ontology defining the entities and relationships between them can be used to define the concepts relevant to systems engineering. Such a representation is necessary to create the digital thread linking all the models together in a cohesive and useful manner.

Digital Engineering as a Transformation

For many organizations, digital engineering represents a transformation of how they normally conduct systems engineering (e.g., see Bone et al. 2018) since most organizations conduct a document-intensive systems engineering process. The adoption of digital engineering requires concomitant changes to how organizations perform system engineering activities. Everything from documenting requirements, technical reviews, architecture design, and so forth would be based on the models in a digital engineering environment (Vaneman and Carlson, 2019). The digital thread would be the authoritative source of truth concerning the system data.

Digital Twin

A digital twin is a related yet distinct concept to digital engineering. The digital twin is a high-fidelity model of the system which can be used to emulate the actual system. An organization would be able to use a digital twin to analyze design changes prior to incorporating them into the actual system.

References

Works Cited

- Bone, M.A., M.R. Blackburn, D.H. Rhodes, D.N. Cohen, and J.A. Guerrero. 2018. "Transforming systems engineering through digital engineering," *The Journal of Defense Modeling and Simulation* (2018): 1548512917751873.
- DoD. 2018. *DoD Digital Engineering Strategy*. Washington, D.C., USA: Office of the Undersecretary of Defense for Research and Engineering (OUSD R&E), US Department of Defense.
- Giachetti, R.E. 2015. "Evaluation of the DoDAF meta-model's support of systems engineering," *Procedia Computer Science*, vol. 61, pp. 254-260.
- Vaneman, W.K. 2018. "Evolving model-based systems engineering ontologies and structures," Proceedings of the International Council on Systems Engineering (INCOSE) International Symposium, Washington D.C., USA, July 7-12, 2018. Symposium Proceedings vol. 28, no. 1, pp. 1027-1036.
- Zimmerman, P. "DoD digital engineering strategy," Proceedings of the 20th Annual National Defense Industrial Association (NDIA) Systems Engineering Conference, Springfield, VA, October 23-26, 2017.

Primary References

- Bone, M.A., M.R. Blackburn, D.H. Rhodes, D.N. Cohen, and J.A. Guerrero. 2018 "Transforming systems engineering through digital engineering," *The Journal of Defense Modeling and Simulation* (2018): 1548512917751873.
- Singh, V. and K.E. Willcox. "Engineering design with digital thread," *AIAA Journal*, vol. 56, no. 11, pp. 4515-4528.
- Zimmerman, P. "DoD digital engineering strategy," Proceedings of the 20th Annual National Defense Industrial Association (NDIA) Systems Engineering Conference, Springfield, VA, October 23-26, 2017. OUSD R&E, DoD Digital Engineering Strategy, (June 2018).

Additional References

None.

Relevant Videos

- Digital Engineering: MBSE Approach for DoD ^[1]

Set-Based Design

- Lead Authors:
- Eric Specking, Gregory S. Parnell, and Ed Pohl

Set-based design (SBD) is a complex design method that enables robust system design by 1) considering a large number of alternatives, 2) establishing feasibility before making decisions, and 3) using experts who design from their own perspectives and use the intersection between their individual sets to optimize a design (Singer, Doerry, and Buckley 2009). Model-based engineering (MBE)/model-based systems engineering (MBSE) with an integrated framework can enable the use of SBD tradespace exploration, for some situations (i.e. early-design stage with low fidelity models), in near-real time (Specking et al. 2018a). This article provides insights on using model-based design to create and assess alternatives with set-based design.

Introduction

SBD analyzes sets of alternatives instead of single solutions. Sets are “two or more design points that have at least one design option in common” (Specking et al. 2018b) or “the range of options for a design factor” (Singer et al. 2017). A design factor is a “solution parameter, characteristic, or relationship that influences the design at the system level” (Singer et al. 2017). Systems engineers should develop sets determining the design factors and separating the design factors into set drivers or set modifiers. Set drivers are “fundamental design decisions that define the system characteristics that enable current and future missions,” while set modifiers are “design decisions that are ‘added on’ to the system and can be modified to adapt for new missions and scenarios” (Specking et al. 2018b).

SBD is not the best design method for every situation. SBD is particularly useful in early-stage design and if the project contains the following attributes:

- A large number of design variables,
- Tight coupling among design variables,
- Conflicting requirements,
- Flexibility in requirements allowing for trades, or
- Technologies and design problems not well understood – learning required for a solution (Singer et al. 2017)

In early-stage design, SBD helps inform requirements analysis and assess design decisions (Parnell et al. 2019). Quantitative SBD requires an integrated MBE environment to assess the effects of constraining and relaxing requirements on the feasible tradespace. For example, Figure 2 demonstrates the effects of constraining or relaxing requirements of an unmanned aerial vehicle case study with all of the explored designs in orange, the tradespace affected by non-requirement constraints (e.g. physics with requirements relaxed to not affect the tradespace) in blue, the original UAV feasible tradespace in yellow, and the relaxed (black)/constrained (red) tradespaces.

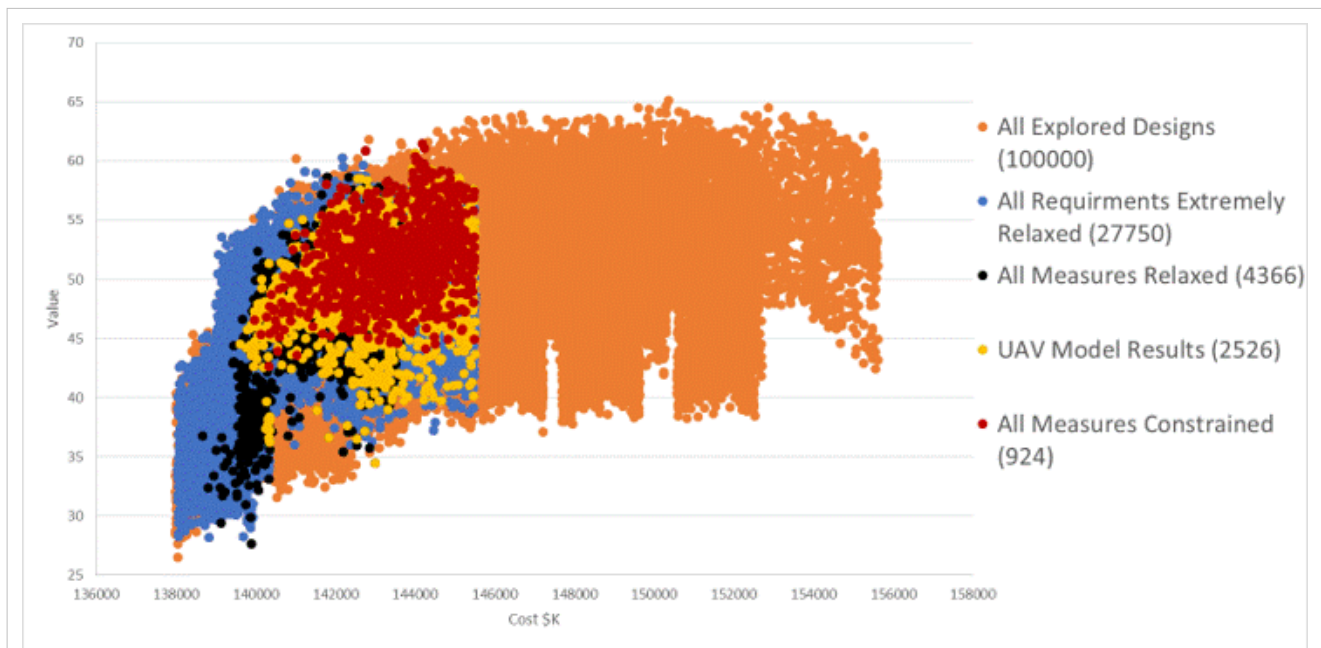


Figure 1. Effects of Requirements on the UAV's Feasible Tradespace (Parnell et al. 2019, used with permission)

The tornado diagram seen in Figure 3 shows results of a one requirement at a time analysis. This makes it easy to see how the constraining/relaxing of each individual requirement affects the feasible tradespace. Figure 3 shows that the requirements “Detect Human Activity at Night” and “Detect Human Activity in Daylight” have the greatest impact on the feasible tradespace.

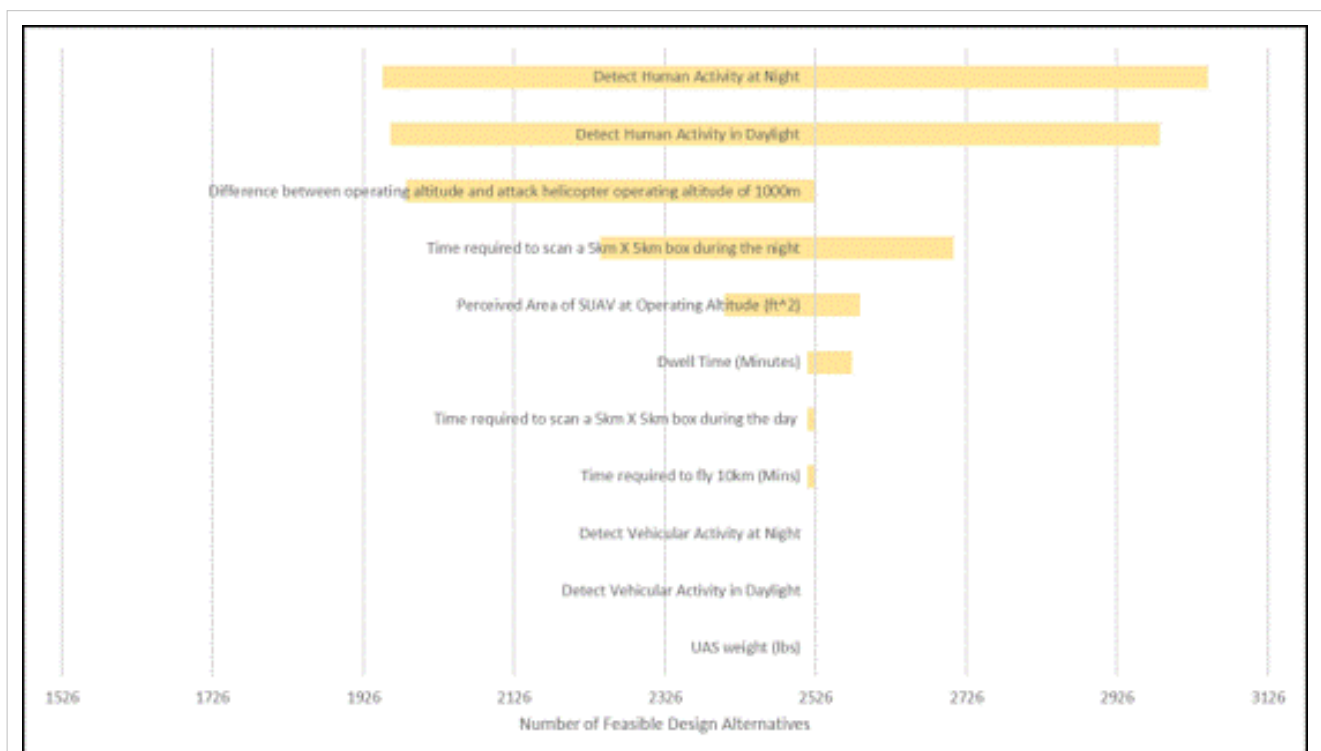


Figure 2. UAV Case Study Results of One-by-One Requirement Analysis (Parnell et al. 2019, used with permission)

Changing the requirements does not always translate to finding improved designs. The individual one requirement at a time analysis scatterplot provides important information, as seen in an example illustration in Figure 4. It is important to carefully analyze the Pareto Frontier created by each change (represented by a different color) and compare it to the Pareto Frontier of the original analysis. If the original requirement level produces better

alternatives, then it does not make sense to change (constrain or relax) the requirement.

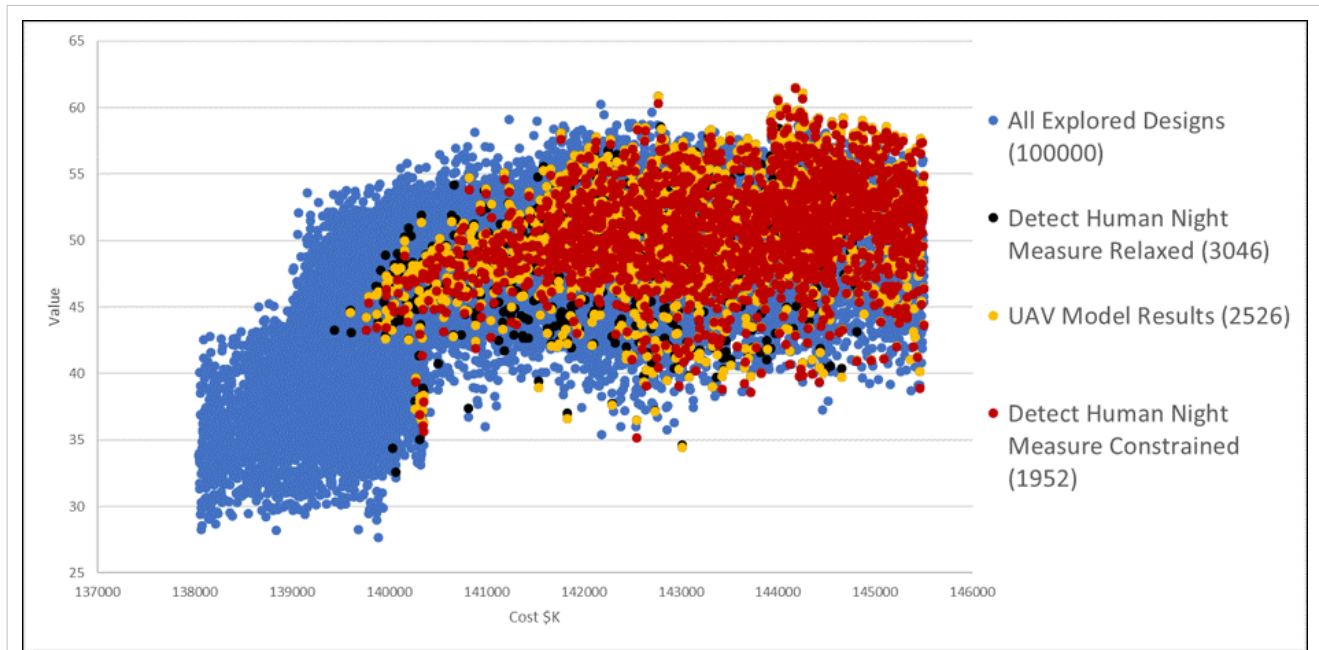


Figure 3. Effect on Feasible Tradespace by Changing Most Sensitive UAV Requirement (Specking et al. 2019, used with permission)

Additionally, using SBD can add value to the overall project and team. Some of the advantages include:

- enabling reliable, efficient communications,
- allowing much greater parallelism in the process, with much more effective use of subteams early in the process,
- allowing the most critical, early decisions to be based on data, and
- promoting institutional learning (Ward et al. 1995).

System Analyst Set-Based Design Tradespace Exploration Process

Figure 4 illustrates SBD as a concept for system design and analysis. This SBD illustration contains 5 distinct characteristics:

1. start by determining the business/mission needs and system requirements;
2. use the business/mission needs and system requirements to perform design and analysis techniques throughout time in the exploratory, concept, and development stages of the system's life cycle;
3. perform design and analysis concurrently as much as possible;
4. inform requirement analysis by using feasibility, performance, and cost data; and
5. consider a large number of alternatives through the use of sets and slowly converge to a single point solution (Specking et al. 2019).

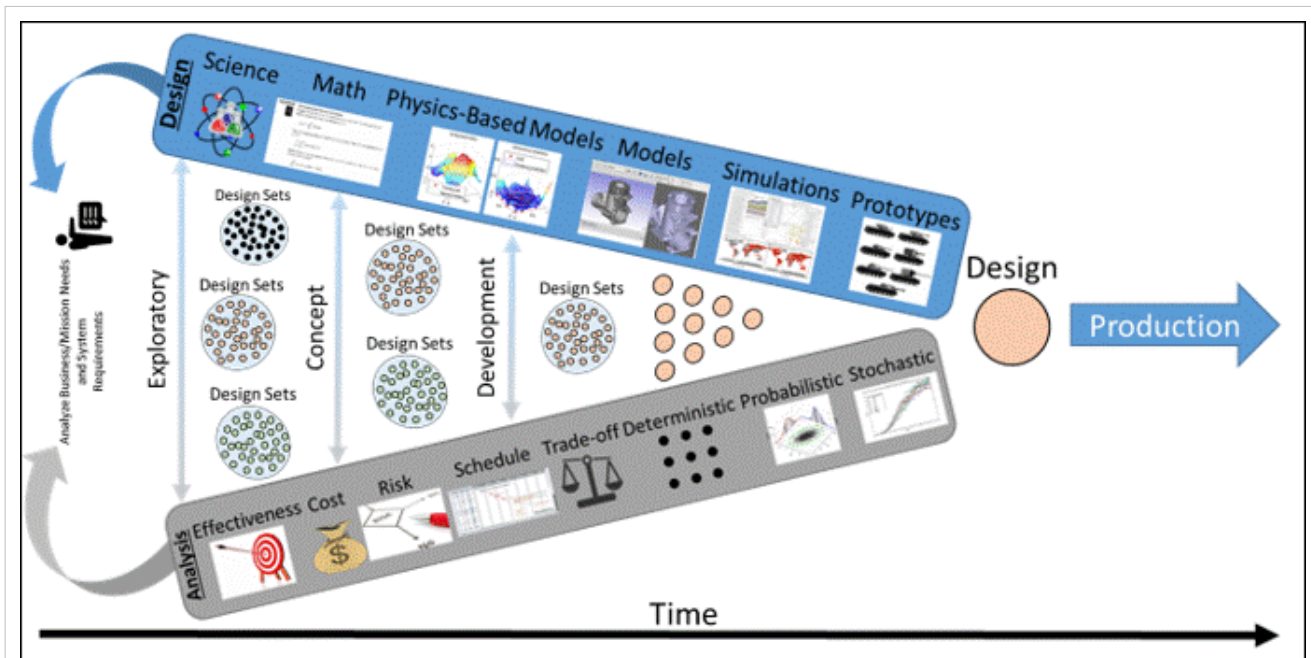
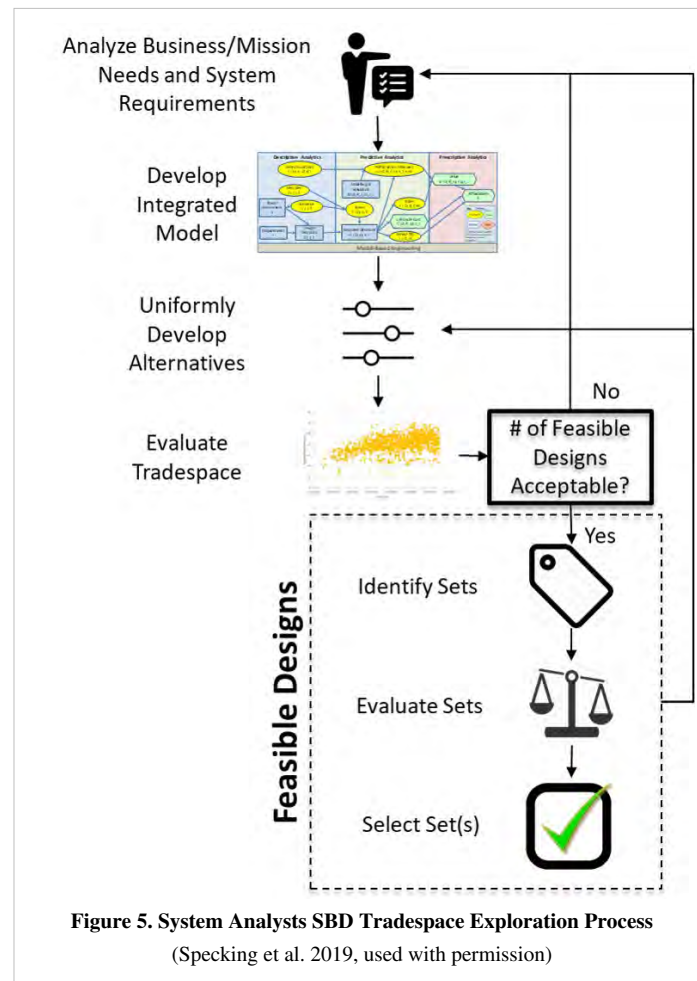


Figure 4. SBD Conceptual Framework for Systems Design (Specking et al. 2019, used with permission)

SBD is a social-technical process and should involve input and interactions from several teams, but Figure 6 provides a SBD tradespace exploration process for system analysts (Specking et al. 2019). This eight-step process is especially useful to perform early-stage design (Specking et al. 2018b). The system analyst starts by analyzing the business/mission needs and system requirements. Systems engineers use this information, along with models and simulations developed by themselves or provided by systems and subsystem teams, to develop an integrated model. Systems engineers include requirements to assess feasible and infeasible alternatives using this integrated model. They explore the tradespace by treating each design decision as a uniform (discrete or continuous) random variable. An alternative consists of an option from every design decision. Systems engineers then use the integrated model to evaluate each alternative and to create the feasible tradespace. Monte Carlo simulation is one method that enables a timely alternative creation and evaluation process. The created tradespace will consist of infeasible and feasible alternatives based upon the requirements and any physics-based performance models and simulations. Systems engineers should work with the appropriate stakeholders to inform requirements when the tradespace produces a significantly small number of or no feasible alternatives. In addition to feasibility, systems engineers should also analyze each design decision by using descriptive statistics and other analyses and data analytics techniques. This information provides insights into how each design factor influences the feasible tradespace. Once the tradespace contains an acceptable number of alternatives, it is then classified by sets. This is an essential part of SBD. If the set drivers or design factors are not known, systems engineers should view the tradespace by each design decision for insights. Systems engineers should use dominance analysis and other optimization methods to find optimal or near optimal alternatives based upon the measures of effectiveness. Systems engineers should explore the remaining sets for additional insights on the feasible tradespace and the requirements. The final part of this process is to select one or more sets to move to the next design-stage. It should be noted that this process contains cycles. At any part of this process, systems engineers should use the available information, such as from tradespace exploration or set evaluation, to inform requirement analysis or update the integrated model. Additionally, the systems engineer should update the integrated model with higher fidelity models and simulations as they become available. The key is to have the “right” information from the “right” people at the “right” time.



References

Works Cited

- Parnell, G.S., E. Specking, S. Goerger, M. Cilli, and E. Pohl. 2019. "Using set-based design to inform system requirements and evaluate design decisions." Proceedings of the 29th Annual International Council on Systems Engineering (INCOSE) International Symposium, Orlando, FL, USA, July 20-25, 2019.
- Singer, D.J., N. Doerry, and M. E. Buckley. 2009. "What is set-based design?" *Naval Engineers Journal*, vol. 121, no. 4, pp. 31–43.
- Singer, D., J. Strickland, N. Doerry, T. McKenney, and C. Whitcomb. 2017. "Set-based design." *SNAME T&R Bulletin SNAME (mt) Marine Technology Technical and Research Bulletin*.
- Specking, E., C. Whitcomb, G. Parnell, S. Goerger, E. Kundeti, and N. Pohl. 2018a. "Literature review: Exploring the role of set-based design in trade-off analytics," *Naval Engineers Journal*, vol. 130, no. 2, pp. 51-62.
- Specking, E., G. Parnell, E. Pohl, and R. Buchanan. 2018b. "Early design space exploration with model-based system engineering and set-based design," *IEEE Systems*, vol. 6, no. 4, pg. 45.
- Ward, A., I. Durward Sobek, J. C. John, and K. L. Jeffrey. 1995. "Toyota, concurrent engineering, and set-based design," in *Engineered in Japan: Japanese Technology-management Practices*. Oxford, England: Oxford University Press. pp. 192–216.

Primary References

None.

Additional References

None.

Emerging Research

Emerging Research

- Lead Authors:
- Robert Cloutier and Arthur Pyster

The Emerging Research topic under the SEBoK Emerging Knowledge is a place to showcase some of the systems engineering research published in the past 3-5 years.

Doctoral Dissertations

Doctoral level systems engineering research has taken root over the last two decades. Additionally, many institutions have either an Industrial Engineering or Systems Engineering Master's program. This has enabled new and interesting research to be conducted. Here you will find bibliographic citations and summaries for recently defended research.

Towards Early Lifecycle Prediction of System Reliability

Salter, C. "Towards early lifecycle prediction of system reliability," Ph.D. dissertation University of South Alabama, Mobile, Alabama, July 2018. Available: ProQuest Store ^[1]

Reliability is traditionally defined as "the probability that an item will perform a required function without failure under stated conditions for a stated period of time" (O'Connor, 2012). This definition is applicable to all levels of a system, from the smallest part to the system as a whole. Predicting reliability requires extensive knowledge of the system of interest, thus making prediction difficult and complex. This problem is further complicated by the desire to predict system reliability early in the acquisition lifecycle. This work set out to develop a model for the prediction of system reliability early in the system lifecycle. The model utilizes eight factors: number of system requirements, number of major interfaces, number of operational environments, requirements understanding, technology maturity, manufacturability, company experience, and performance convergence. These factors come together to form a model much like the software engineering and systems engineering models COCOMO and COSYSMO. This work provides the United States Department of Defense a capability that previously did not exist: the estimation of system reliability early in the system lifecycle. The research demonstrates that information available during early system development may be used to predict system reliability. Through testing, the author found that a model of this type could provide reliability predictions for military ground vehicles within 25% of their actual recorded reliability values.

Toward the Evolution of Information Digital Ecosystems

Lippert, K. "Toward the evolution of information digital ecosystems," Ph.D dissertation, University of South Alabama, Mobile, Alabama, May 2018. Available: ProQuest Store ^[2].

Digital ecosystems are the next generation of Internet and network applications, promising a whole new world of distributed and open systems that can interact, self-organize, evolve, and adapt. These ecosystems transcend traditional collaborative environments, such as client-server, peer-to-peer, or hybrid models (e.g., web services) to become a self-organized, interactive environment. The complexity of these digital ecosystems will encourage evolution through adaptive processes and selective pressures of one member on another to satisfy interaction, adaptive organization, and, incidentally, human curiosity. This work addresses one of the essential parts of the digital

ecosystem – the information architecture. The research, inspired by systems thinking influenced by both biological models and science fiction, applies the TRIZ method to the contradictions raised by evolving data. This inspired the application of patterns and metaphor as a means for coping with the evolution of the ecosystem. The metaphor is explored as a model of representation of rapidly changing information through a demonstration of an adaptive digital ecosystem. The combination of this type of data representation with dynamic programming and adaptive interfaces will enable the development of the various components required by a true digital ecosystem.

Cybersecurity Decision Patterns as Adaptive Knowledge Encoding in Cybersecurity Operations

Willett, K. "Cybersecurity decision patterns as adaptive knowledge encoding in cybersecurity operations", Ph.D. dissertation, Stevens Institute of Technology, Hoboken, NJ, July 2016. Available: <https://pqdtopen.proquest.com/doc/1875237837.html?FMT=ABS>.

Cyberspace adversaries perform successful exploits using automated adaptable tools. Cyberspace defense is too slow because existing response solutions require humans in-the-loop across sensing, sense-making, decision-making, acting, command, and control of security operations (Dõne et al. 2016). Security automation is necessary to provide for cyber defense dynamic adaptability in response to an agile adversary with intelligence and intent who adapts quickly to exploit new vulnerabilities and new safeguards. The rules for machine-encoding security automation must come from people; from their knowledge validated through their real-world experience. Cybersecurity Decision Patterns as Adaptive Knowledge Encoding in Cybersecurity Operations introduces cybersecurity decision patterns (CDPs) as formal knowledge representation to capture, codify, and share knowledge to introduce and enhance security automation with the intent to improve cybersecurity operations efficiency for processing anomalies.

INCOSE & IEEE Periodicals and Events

Every year, the International Council on Systems Engineering (INCOSE) holds one International Workshop and one International Symposium, as well as regular meetings of various working groups, to encourage discussions of emerging needs and sharing of experience within Systems Engineering community. All papers and presentations from these events are available for free for INCOSE members, or with a fee for non-members via Wiley. The library can be access here: <https://www.incose.org/products-and-publications/papers-presentations-library#>

Additionally, INCOSE also publish periodicals, which include: Systems Engineering (SE Journal), INSIGHT (magazine), and INCOSE Members Newsletter. These periodicals are available as PDF, free for INCOSE members and with a fee for non-members, or as hard copies. More information can be found here: <https://www.incose.org/products-and-publications/periodicals>

The Institute of Electrical and Electronics Engineers (IEEE) Systems Council also holds multiple annual conferences, such as the International Systems Conference (SysCon), on systems engineering, resulting in a large pool of publications. These publications can be found via: <https://ieeesystemscouncil.org/publications>

NSF- and SERC-funded Research

The National Science Foundation (NSF), Division of Civil, Mechanical, and Manufacturing Innovation (CMMI) has been funding research in academia on systems engineering under Engineering Design and Systems Engineering (EDSE) program. According to NSF-EDSE website ^[3], the program "seeks proposals leading to improved understanding about how processes, organizational structure, social interactions, strategic decision making, and other factors impact success in the planning and execution of engineering design and systems engineering projects". Research under this program can be found via Award Search feature on NSF website: <https://www.nsf.gov/awardsearch/advancedSearch.jsp> (Enter "CMMI" for NSF Organization and "EDSE" for Program).

The Systems Engineering Research Center (SERC) is a University-Affiliated Research Center of the US Department of Defense, consisting of 22 collaborator universities in the US and funding research on different aspects of Systems Engineering, including Enterprises and System of Systems, Trusted Systems, Systems Engineering and Systems Management Transformation. More information can be found here: <https://sercuarc.org/serc-programs-projects/esos/>

References

None.

References

- [1] https://order.proquest.com/OA_HTML/pqdtibeCAcdLogin.jsp?jsessionid=e0c74b22f5dff64bf4a20c1deef606a0397a02e9aa59ec701b81e5d7cde90387.e34PbxmRc3qPbO0Lbx4Nc3yMbxNe0?ref=https%3A%2F%2Forder.proquest.com%2FOA_HTML%2FpqdtibeCCtpItmDspRte.jsp%3Fdlnow%3D1%26item%3D10840641%26rpath%3Dhttps%253A%252F%252Fsearch.proquest.com%252Fpqdtglobal%252Fredirectfor%253Faccountid%253D14541%26track%3D1SRCH&sitex=10020:22372:US&sitex=10020:22372:US
- [2] https://order.proquest.com/OA_HTML/pqdtibeCAcdLogin.jsp?ref=https%3A%2F%2Forder.proquest.com%2FOA_HTML%2FpqdtibeCCtpItmDspRte.jsp%3Fdlnow%3D1%26item%3D10790760%26rpath%3Dhttps%253A%252F%252Fsearch.proquest.com%252Fpqdtglobal%252Fredirectfor%253Faccountid%253D14541%26track%3D1SRCH&sitex=10020:22372:US&sitex=10020:22372:US
- [3] https://www.nsf.gov/funding/pgm_summ.jsp?pims_id=505478&org=ENG&from=home

Article Sources and Contributors

Editor's Corner *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78827> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dholwell, Eleach, Kguillemette, Mhaas, Radcock, Rcloutier, Smenck2, Wikiexpert

Governance and Editorial Boards *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78767> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Gconover, Gsmith, Jcarter, Kguillemette, Mhaas, Radcock, Rcloutier, Smenck2, WikiWorks, WikiWorks017, WikiWorks753

Acknowledgements and Release History *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78835> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Ddori, Dhenry, Dholwell, Dnewbern, Eleach, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Nicole.hutchison, Radcock, Rcloutier, Smenck2, WikiWorks, Wikiexpert

SEBoK Sponsors *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=69246> *Contributors:* Bkcase, Cdhoffman, WikiWorks, WikiWorks753

Get Involved *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78761> *Contributors:* Bkcase, Cdhoffman

Bkcase Wiki:Copyright *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78760> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Kguillemette, Radcock, Smenck2, Wikiexpert

SEBoK Table of Contents *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78829> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Csingam, Daniel Robbins, Dhenry, Dholwell, Gparnell, Kguillemette, Nicole.hutchison, Smenck2, Thilburn, WikiWorks, Wikiexpert

SEBoK Introduction *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77957> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Nicole.hutchison, Radcock, Rcloutier, Smenck2, WikiWorks, Wikiexpert

Introduction to the SEBoK *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77932> *Contributors:* Bkcase, Cdhoffman, Mhaas, Radcock, Rcloutier, WikiWorks

Scope of the SEBoK *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77259> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Groedler, Janthony, Jgercken, Mhaas, Mhenshaw, Nicole.hutchison, Radcock, Rcloutier, Skmackin, Smenck2, WikiWorks, Wikiexpert

Structure of the SEBoK *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78425> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Gparnell, Janthony, Jgercken, Mhaas, Nicole.hutchison, Radcock, Rcloutier, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert

Introduction to Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77933> *Contributors:* Bkcase, Cdhoffman, Gconover, Mhaas, Radcock, Rcloutier, WikiWorks

Systems Engineering Overview *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77971> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Mhaas, Radcock, Rcloutier, Smenck2, WikiWorks

Fundamentals for Digital Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77262> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Economic Value of Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77268> *Contributors:* Apyster, Asofer, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Eleach, Gparnell, Janthony, Mhaas, Mhenshaw, Radcock, Rvalerdi, Smenck2, WikiWorks, Wikiexpert

A Brief History of Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78514> *Contributors:* Bkcase, Gconover, WikiWorks

Systems Engineering: Historic and Future Challenges *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78173> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Gparnell, Janthony, Jgercken, Mhaas, Radcock, Rcloutier, Smenck2, WikiWorks, Wikiexpert

Systems Engineering and Other Disciplines *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78418> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Gparnell, Janthony, Jgercken, Kguillemette, Mhaas, Nicole.hutchison, Rcloutier, Skmackin, Smenck2, WikiWorks, Wikiexpert

Fundamentals for Future Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77566> *Contributors:* Bkcase, Cdhoffman, Mhaas, Radcock, WikiWorks

SEBoK Users and Uses *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77669> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Jgercken, Kguillemette, Mhaas, Nicole.hutchison, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Guidance for Systems Engineering Novices *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78105> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Dholwell, Gconover, Jcarter, Mhaas, Radcock, WikiWorks

Guidance for Systems Engineers *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78410> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Gparnell, Janthony, Jgercken, Mhaas, Radcock, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Guidance for Engineers *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78239> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Gparnell, Jgercken, Jcarter, Mhaas, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Guidance for Systems Engineering Customers *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78204> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Gparnell, Jgercken, Jcarter, Mhaas, Radcock, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Guidance for Educators and Researchers *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78183> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Jgercken, Mhaas, Mhenshaw, Radcock, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Guidance for General Managers *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78299> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Gparnell, Jgercken, Mhaas, Mhenshaw, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Foundations of Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78453> *Contributors:* Apyster, Asquires, Bkcase, Cdagli, Cdhoffman, Cnielsen, Dcarey, Dhenry, Dholwell, Eleach, Gconover, Gsmith, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Systems Engineering Fundamentals *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77989> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Gconover, Gsmith, Janthony, Jdahmann, Mhaas, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Introduction to Systems Engineering Fundamentals *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78567> *Contributors:* Afaisandier, Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Ddori, Dhenry, Dholwell, Dnewbern, Gconover, Gsmith, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Systems Engineering Core Concepts *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78579> *Contributors:* Bkcase, Cdhoffman, Gconover, Gsmith, Mhaas, WikiWorks

Systems Engineering Principles *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77976> *Contributors:* Apyster, Bkcase, Gconover, Gsmith, Mhaas, WikiWorks

Systems Engineering Heuristics *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78130> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, WikiWorks

The Nature of Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78046> *Contributors:* Bkcase, Cdhoffman, Gconover, Gsmith, Mhaas, WikiWorks

Engineered Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78047> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Gsmith, Janthony, Jdahmann, Jgercken, Mhaas, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Natural Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78048> *Contributors:* Cdhoffman, Gconover, Gsmith

Principles and Attributes of Natural Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77811> *Contributors:* Cdhoffman, Gsmith

- Value Proposition for Systems Engineering** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78713> *Contributors:* Cdhoffman, Geonover, Gsmith
- Identity and Togetherness of Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78050> *Contributors:* Cdhoffman, Geonover, Gsmith
- Behavior and Dynamics of Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78051> *Contributors:* Cdhoffman, Geonover, Gsmith
- Cycles and the Phases of Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77812> *Contributors:* Bkcase, Cdhoffman, Gsmith, Mhaas, Rcloutier, WikiWorks
- Purpose and the Capability of Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78052> *Contributors:* Cdhoffman, Geonover, Gsmith
- Value and the Quality of Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78717> *Contributors:* Cdhoffman, Geonover, Gsmith
- Consciousness and the Experience of Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78718> *Contributors:* Cdhoffman, Geonover, Gsmith
- Systems Science** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78809> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Geonover, Gsmith, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert
- From Nature's Patterns to Systems Science** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78812> *Contributors:* Bkcase, Cdhoffman, Geonover, Gsmith
- History and Foundations of Systems Science** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78773> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Geonover, Gsmith, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Worldviews and Philosophies of Systemness** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78781> *Contributors:* Bkcase, Cdhoffman, Geonover, Gsmith
- Patterns and Principles in Systems Science** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78833> *Contributors:* Cdhoffman, Geonover, Gsmith
- Structure, Identity, and Function in Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78783> *Contributors:* Bkcase, Cdhoffman, Geonover, Gsmith
- Emergence and Complexity** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78790> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gsmith, Janthony, Jgercken, Mhaas, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai
- System Dynamics and Control** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78797> *Contributors:* Bkcase, Cdhoffman, Geonover, Gsmith
- Information, Organization, and Entropy in Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78802> *Contributors:* Bkcase, Cdhoffman, Geonover, Gsmith
- Systems, Levels, and Holarchy** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78805> *Contributors:* Bkcase, Cdhoffman, Geonover, Gsmith
- Systems Science and the Scientific Enterprise** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78806> *Contributors:* Bkcase, Cdhoffman, Geonover, Gsmith
- Systems Thinking** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78277> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Geonover, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert
- What is Systems Thinking?** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78451> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dnewbern, Geonover, Janthony, Jgercken, Mhaas, Radcock, Rturner, Smenck2, WikiWorks, Wikiexpert
- Concepts of Systems Thinking** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78564> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Geonover, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai
- Principles of Systems Thinking** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78452> *Contributors:* Bkcase, Bsauser, Cdhoffman, Cnielsen, Dhenry, Dholwell, Geonover, Janthony, Jtcarter, Mhaas, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert, Ymordecai
- Patterns of Systems Thinking** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78576> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Dhenry, Dnewbern, Geonover, Janthony, Mhaas, Mhenshaw, Radcock, Smenck2, WikiWorks
- Representing Systems with Models** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78312> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Ddori, Dhenry, Dholwell, Dnewbern, Eleach, Geonover, Gparnell, Janthony, Jgercken, Mhaas, Radcock, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert
- What is a Model?** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78353> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Ddori, Dhenry, Geonover, Gparnell, Janthony, Jgercken, Mhaas, Mhenshaw, Rcloutier, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Why Model?** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77311> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Janthony, Jgercken, Mhaas, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Types of Models** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77580> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Gparnell, Janthony, Jgercken, Mhaas, Mhenshaw, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert
- System Modeling Concepts** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78401> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Ddori, Dhenry, Dnewbern, Janthony, Jgercken, Mhaas, Mhenshaw, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Integrating Supporting Aspects into System Models** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78456> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Eleach, Mhaas, Smenck2, WikiWorks, Ymordecai
- Modeling Standards** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78324> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dcarey, Ddori, Dhenry, Dnewbern, Janthony, Jgercken, Mhaas, Mhenshaw, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Systems Approach Applied to Engineered Systems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77937> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dcarey, Dhenry, Dholwell, Dnewbern, Eleach, Geonover, Gparnell, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Overview of the Systems Approach** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78424> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dcarey, Dhenry, Dnewbern, Geonover, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Engineered System Context** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77576> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Identifying and Understanding Problems and Opportunities** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77318> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Rturner, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Synthesizing Possible Solutions** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78577> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Geonover, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Rturner, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Analysis and Selection between Alternative Solutions** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78066> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Geonover, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Implementing and Proving a Solution** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78285> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Janthony, Jgercken, Mhaas, Radcock, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert
- Deploying, Using, and Sustaining Systems to Solve Problems** *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78574> *Contributors:* Apyster, Bkcase, Bwells, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Rturner, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert

Applying the Systems Approach *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78074> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Gparnell, Janthony, Jgercken, Jcarter, Mhaas, Mhenshaw, Radcock, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert

Systems Engineering and Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78533> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Cnielsen, Csingam, Dcarey, Dhenry, Dholwell, Dnewbern, Dward, Elcanez, Gconover, Groedler, Janthony, Jgercken, Jcarter, Mhaas, Mhenshaw, Radcock, Rmadachy, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert

Systems Engineering STEM Overview *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78259> *Contributors:* Bkcase, Cdhoffman, Elcanez, Gconover, Mhaas, WikiWorks

Model-Based Systems Engineering (MBSE) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78221> *Contributors:* Bkcase, Cdhoffman, Csingam, Gconover, Jcarter, Mhaas, WikiWorks

Life Cycle Terms and Concepts *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78537> *Contributors:* Bkcase, Cdhoffman, DEndler, Gconover, Mhaas, Radcock, Sfriedenthal, WikiWorks

Life Cycle Concepts *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78538> *Contributors:* Bkcase, Cdhoffman, DEndler, WikiWorks

Life Cycle Models *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78539> *Contributors:* Bkcase, Cdhoffman, DEndler, WikiWorks

Life Cycle Stages *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78540> *Contributors:* Bkcase, Cdhoffman, DEndler, WikiWorks

Technical Reviews and Audits *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78541> *Contributors:* Bkcase, Cdhoffman, DEndler, WikiWorks

Development Approaches *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77939> *Contributors:* Bkcase, Cdhoffman, DEndler, Gconover, WikiWorks

Development Approach Concepts *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77356> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Sequential Development Approach *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77357> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Incremental Development Approach *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77355> *Contributors:* Cdhoffman, WikiWorks

Evolutionary Development Approach *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77358> *Contributors:* Cdhoffman, WikiWorks

Agile Development Approach *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78159> *Contributors:* Bkcase, Cdhoffman, Gconover, WikiWorks

Lean Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78400> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Jcarter, Mhaas, Smenck2, WikiWorks, Wikiexpert

Agile Approaches *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78735> *Contributors:* Cdhoffman, DEndler

Agile Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78736> *Contributors:* Bkcase, Cdhoffman, DEndler, Mhaas, PMarbach, WikiWorks

Industrial DevOps *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78731> *Contributors:* Cdhoffman, WikiWorks

Life Cycle Model Selection and Adaptation *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77940> *Contributors:* Cdhoffman, Gconover, WikiWorks

Selecting the Life Cycle Model *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77361> *Contributors:* Cdhoffman, WikiWorks

Adapting the Life Cycle Model *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77362> *Contributors:* Cdhoffman, WikiWorks

Selecting the Development Approach *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77364> *Contributors:* Cdhoffman, WikiWorks

Adapting the Development Approach *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77363> *Contributors:* Cdhoffman, WikiWorks

Applying Life Cycle Processes *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78482> *Contributors:* Bkcase, Cdhoffman, Gconover, Jcarter, Mhaas, Radcock, WikiWorks

Process Concepts *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77941> *Contributors:* Cdhoffman, DEndler, Gconover

Process Description *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77836> *Contributors:* Cdhoffman, DEndler

Process Concurrency, Iteration, and Recursion *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77755> *Contributors:* Cdhoffman, DEndler

Process Selection and Tailoring *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78128> *Contributors:* Cdhoffman, DEndler, Gconover

Guidelines for the Selection and Tailoring of Processes *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77761> *Contributors:* Cdhoffman, DEndler

Using ISO/IEC/IEEE 24748-2 *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78129> *Contributors:* Cdhoffman, DEndler, Gconover

Tailoring *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77818> *Contributors:* Cdhoffman, DEndler

Technical Management Processes *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78133> *Contributors:* Bkcase, Cdhoffman, DEndler, Dhenry, Dholwell, Gconover, Gparnell, Groedler, Jgercken, Mhaas, Mhenshaw, Rmadachy, Skmackin, Smenck2, WikiWorks, Wikiexpert

Project Planning *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78062> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Groedler, Janthony, Jgercken, Jcarter, Mhaas, Mhenshaw, Rmadachy, Skmackin, Smenck2, WikiWorks, Wikiexpert

Project Assessment and Control *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77717> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Groedler, Jgercken, Mhaas, Mhenshaw, Rmadachy, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

Decision Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78064> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Eleach, Gconover, Gparnell, Groedler, Jgercken, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Requirements Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77986> *Contributors:* Bkcase, Cdhoffman, Gconover, Tkatz, WikiWorks

Risk Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78386> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, DEndler, Dhenry, Dholwell, Dnewbern, Groedler, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

Configuration Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77371> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Groedler, Jgercken, Jcarter, Mhaas, Mhenshaw, Radcock, Reloutier, Skmackin, Smenck2, WikiWorks, Wikiexpert

Configuration Baselines *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77372> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Configuration Management Implementation *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77373> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Information Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78132> *Contributors:* Bkcase, Cdhoffman, DEndler, Dhenry, Dholwell, Dnewbern, Gconover, Groedler, Jgercken, Jcarter, Mhaas, Mhenshaw, Rmadachy, Skmackin, Smenck2, WikiWorks, Wikiexpert

Quality Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78141> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Groedler, Jgercken, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Measurement *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78292> *Contributors:* Apyster, Bkcase, Cdhoffman, Cjones, DEndler, Dhenry, Dholwell, Dnewbern, Gconover, Groedler, Jgercken, Mhaas, Mhenshaw, Rcarson, Rmadachy, Skmackin, Smenck2, WikiWorks, Wikiexpert

System Concept Definition *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77672> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Eleach, Groedler, Mhaas, Mhenshaw, Radcock, Smenck2, Tkatz, WikiWorks, Wikiexpert, Ymordecai

Business or Mission Analysis *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78134> *Contributors:* Afaisandier, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Groedler, Janthony, Jgercken, Mhaas, Mhenshaw, Radcock, Rturner, Skmackin, Smenck2, Tkatz, WikiWorks, Wikiexpert, Ymordecai

Stakeholder Needs Definition *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78519> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Eleach, Gconover, Groedler, Janthony, Mhaas, Mhenshaw, Radcock, Smenck2, Tkatz, WikiWorks, Wikiexpert, Ymordecai

System Requirements Definition *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78136> *Contributors:* Afaisandier, Apyster, Asofer, Asquires, Bkcase, Cdhoffman, Cjones, Dhenry, Dholwell, Eleach, Gconover, Groedler, Jgercken, Mhaas, Radcock, Rturner, Skmackin, Smenck2, Tkatz, WikiWorks, Wikiexpert, Ymordecai

System Architecture Design Definition *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77674> *Contributors:* Bkcase, Cdhoffman, Csingam, Elcanez, Jtcarter, Mhaas, WikiWorks

Functional Architecture *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77380> *Contributors:* Bkcase, Cdhoffman, Csingam, Jtcarter, Mhaas, WikiWorks

Logical Architecture *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78376> *Contributors:* Afaisandier, Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Elcanez, Eleach, Gconover, Groedler, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Physical Architecture *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78405> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dnewbern, Elcanez, Eleach, Gconover, Mhaas, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert

System Detailed Design Definition *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78151> *Contributors:* Afaisandier, Bkcase, Cdhoffman, Mhaas, Radcock, WikiWorks

System Analysis *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78116> *Contributors:* Afaisandier, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Jtcarter, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

System Realization *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77961> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Elcanez, Eleach, Jgercken, Jtcarter, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

System Implementation *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78027> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Jgercken, Jtcarter, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

System Integration *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78385> *Contributors:* Afaisandier, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Janthony, Jgercken, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

System Verification *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78226> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Jgercken, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

System Transition *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77681> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Jgercken, Jtcarter, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

System Validation *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78411> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Jtcarter, Mhaas, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert

System Operation *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77683> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Jgercken, Ldecardenas, Mhaas, Mhenshaw, Skmackin, WikiWorks, Wikiexpert

System Maintenance *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77962> *Contributors:* Bkcase, Cdhoffman, Ddorgan, Dhenry, Dholwell, Dnewbern, Jgercken, Mhaas, Skmackin, WikiWorks, Wikiexpert

Logistics *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78309> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Groedler, Jgercken, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Service Life Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78240> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Jgercken, Jtcarter, Mhaas, Mhenshaw, Skmackin, WikiWorks, Wikiexpert

Service Life Extension *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78316> *Contributors:* Asquires, Bkcase, Bwells, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Jgercken, Mhaas, Mhenshaw, Skmackin, WikiWorks, Wikiexpert

Capability Updates, Upgrades, and Modernization *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78397> *Contributors:* Asquires, Bkcase, Bwells, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Jgercken, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

System Disposal and Retirement *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78163> *Contributors:* Asquires, Bkcase, Bwells, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Mhenshaw, Skmackin, WikiWorks, Wikiexpert, Ymordecai

Systems Engineering Standards *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77963> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Groedler, Jgercken, Mhaas, Mhenshaw, Skmackin, WikiWorks, Wikiexpert

Why Standards? *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78723> *Contributors:* Bkcase, Cdhoffman, DEndler, WikiWorks

Evolution of Systems Engineering Standards *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78807> *Contributors:* Bkcase, Cdhoffman, DEndler

Systems Engineering Related Standards Landscape *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78655> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Elcanez, Groedler, Janthony, Jgercken, Kguillemette, Kzemrowski, Mhaas, Mhenshaw, Sfriedenthal, Skmackin, Smenck2, WikiWorks, Wikiexpert

Harmonization of Systems Engineering Standards *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78808> *Contributors:* Apyster, Bkcase, Cdhoffman, DEndler, Dhenry, Dholwell, Dnewbern, Elcanez, Groedler, Janthony, Jgercken, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Application of Systems Engineering Standards *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78661> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Elcanez, Groedler, Janthony, Jgercken, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Image Sources, Licenses and Contributors

File:Hutchison,Nicole Profile.jpeg Source: https://sebokwiki.org/draft/w/index.php?title=File:Hutchison,Nicole_Profile.jpeg License: unknown Contributors: Bkcase

File:Hutchison_Signature.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Hutchison_Signature.png License: unknown Contributors: Bkcase

File:Chris profile picture ESEP.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Chris_profile_picture_ESEP.png License: unknown Contributors: Cdhoffman

File:Gary Smith.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Gary_Smith.jpg License: unknown Contributors: User:Bkcase

File:Dov Dori.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Dov_Dori.jpg License: unknown Contributors: User:Bkcase

File:Duane Hybertson.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Duane_Hybertson.jpg License: unknown Contributors: User:Bkcase

File:Peter Tuddenham.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Peter_Tuddenham.jpg License: unknown Contributors: User:Bkcase

File:Cihan Dagli.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Cihan_Dagli.png License: unknown Contributors: User:Bkcase

File:David Endler.png Source: https://sebokwiki.org/draft/w/index.php?title=File:David_Endler.png License: unknown Contributors: User:Bkcase

File:Cecilia-haskins.jpg Source: <https://sebokwiki.org/draft/w/index.php?title=File:Cecilia-haskins.jpg> License: unknown Contributors: Cdhoffman

File:Tom McDermott.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Tom_McDermott.jpg License: unknown Contributors: User:Bkcase

File:Javier Calvo.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Javier_Calvo.jpg License: unknown Contributors: User:Bkcase

File:Judith Dahmann.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Judith_Dahmann.jpg License: unknown Contributors: User:Bkcase

File:Michael Henshaw.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Michael_Henshaw.jpg License: unknown Contributors: User:Bkcase

File:Rick Hefner Caltech (1).jpg Source: [https://sebokwiki.org/draft/w/index.php?title=File:Rick_Hefner_Caltech_\(1\).jpg](https://sebokwiki.org/draft/w/index.php?title=File:Rick_Hefner_Caltech_(1).jpg) License: unknown Contributors: User:Bkcase

File:Singam profile photo.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Singam_profile_photo.jpg License: unknown Contributors: Cdhoffman

File:Ali Raz.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Ali_Raz.jpg License: unknown Contributors: User:Bkcase

File:Ha Phuong Le.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Ha_Phuong_Le.jpg License: unknown Contributors: User:Bkcase

File:Ray Madachy.jpeg Source: https://sebokwiki.org/draft/w/index.php?title=File:Ray_Madachy.jpeg License: unknown Contributors: Cdhoffman

File:PPI.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:PPI.png> License: unknown Contributors: Bkcase

File:PPI_Youtube_Six-Myths-of-Systems-Engineering_final_v2.png Source: https://sebokwiki.org/draft/w/index.php?title=File:PPI_Youtube_Six-Myths-of-Systems-Engineering_final_v2.png License: unknown Contributors: Bkcase

File:Caltech_LOGO-Orange_RGB.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Caltech_LOGO-Orange_RGB.png License: unknown Contributors: Bkcase

File:GMU PLogo RGB.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:GMU_PLogo_RGB.jpg License: unknown Contributors: User:Bkcase

File:USA Systems Eng Logo.png Source: https://sebokwiki.org/draft/w/index.php?title=File:USA_Systems_Eng_Logo.png License: unknown Contributors: Bkcase

File:Wsu stacked.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Wsu_stacked.png License: unknown Contributors: Bkcase

File:SEBoK_Context_Diagram_Inner_P1_Ifezue_Obiako.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P1_Ifezue_Obiako.png License: unknown Contributors: Bkcase

Image:WoYFSO.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:WoYFSO.png> License: unknown Contributors: WikiWorks743

File:SE_Key_Concepts.jpeg Source: https://sebokwiki.org/draft/w/index.php?title=File:SE_Key_Concepts.jpeg License: unknown Contributors: Bkcase, Mhenshaw, Smenck2

File:Scope_BoundariesSE_PM_SM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Scope_BoundariesSE_PM_SM.png License: unknown Contributors: Bkcase, Mhenshaw, Smenck2

File:P1_Scope_and_Con_SE_and_Eng_Sys_Proj_LF_BB.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:P1_Scope_and_Con_SE_and_Eng_Sys_Proj_LF_BB.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:Figure 1. Digital Engineering Systemic Framework (1).jpeg Source: [https://sebokwiki.org/draft/w/index.php?title=File:Figure_1_Digital_Engineering_Systemic_Framework_\(1\).jpeg](https://sebokwiki.org/draft/w/index.php?title=File:Figure_1_Digital_Engineering_Systemic_Framework_(1).jpeg) License: unknown Contributors: Bkcase

File:NASA Image Part 1.png Source: https://sebokwiki.org/draft/w/index.php?title=File:NASA_Image_Part_1.png License: unknown Contributors: Mhenshaw, Smenck2

File:P1_EconValueSE_RiskBalancedfig2_BB.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:P1_EconValueSE_RiskBalancedfig2_BB.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:SEBoK_Context_Diagram_Inner_P2_Ifezue_Obiako.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P2_Ifezue_Obiako.png License: unknown Contributors: Bkcase

File:IFSR_SPF_August_2013.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:IFSR_SPF_August_2013.jpg License: unknown Contributors: Smenck2

File:IFSR_ISA_July_2012_REV.png Source: https://sebokwiki.org/draft/w/index.php?title=File:IFSR_ISA_July_2012_REV.png License: unknown Contributors: Bkcase, Smenck2

File:Fig_1_System_Fundamentals_and_Engineered_Systems_RA.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig_1_System_Fundamentals_and_Engineered_Systems_RA.png License: unknown Contributors: Bkcase, Mhenshaw

File:Part2 Environment 201905.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Part2_Environment_201905.jpg License: unknown Contributors: Bkcase

File:Part2 ModifiedCapabilityEngineering 201905.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Part2_ModifiedCapabilityEngineering_201905.png License: unknown Contributors: Bkcase

File:Part2 ServiceSystem 201905.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Part2_ServiceSystem_201905.png License: unknown Contributors: Bkcase

File:SE Core Concept Relationships.PNG Source: https://sebokwiki.org/draft/w/index.php?title=File:SE_Core_Concept_Relationships.PNG License: unknown Contributors: Bkcase

File:SE Core Concept Relationships Example.PNG Source: https://sebokwiki.org/draft/w/index.php?title=File:SE_Core_Concept_Relationships_Example.PNG License: unknown Contributors: Bkcase

File:Core SEBoK Concepts.PNG Source: https://sebokwiki.org/draft/w/index.php?title=File:Core_SEBoK_Concepts.PNG License: unknown Contributors: Bkcase

File:Part2 ServiceSystemofInterest 201905.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Part2_ServiceSystemofInterest_201905.png License: unknown Contributors: Bkcase

File:Identity and the Togetherness of systems.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Identity_and_the_Togetherness_of_systems.png License: unknown Contributors: Cdhoffman

File:Archetypes of Behaviour and Dynamics.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Archetypes_of_Behaviour_and_Dynamics.png License: unknown Contributors: Cdhoffman

File:One Spin of the Holon.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:One_Spin_of_the_Holon.jpg License: unknown Contributors: User:Bkcase

File:The Unbroken Sequence Of Origins.png Source: https://sebokwiki.org/draft/w/index.php?title=File:The_Unbroken_Sequence_Of_Origins.png License: unknown Contributors: User:Bkcase

File:Archetypes of Cycle.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Archetypes_of_Cycle.png License: unknown Contributors: User:Bkcase

File:System Science knowledge area.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_Science_knowledge_area.png License: unknown Contributors: -

File:Natural patterns to system science.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Natural_patterns_to_system_science.png License: unknown Contributors: -

File:NaturetoSystemsScience_Smith,Gary.png Source: https://sebokwiki.org/draft/w/index.php?title=File:NaturetoSystemsScience_Smith,Gary.png License: unknown Contributors: -

File:History of system science.png Source: https://sebokwiki.org/draft/w/index.php?title=File:History_of_system_science.png License: unknown Contributors: -

File:KnowingSystems_Smith,Gary.png Source: https://sebokwiki.org/draft/w/index.php?title=File:KnowingSystems_Smith,Gary.png License: unknown Contributors: -

File:Philosophy and Worldviews.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Philosophy_and_Worldviews.png License: unknown Contributors: -

File:Common Patterns.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Common_Patterns.png License: unknown Contributors: -

File:Patterns and principles to behavior.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Patterns_and_principles_to_behavior.png License: unknown Contributors: -

File:Structure Identity and Function.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Structure_Identity_and_Function.png License: unknown Contributors: -

File:TypesofEmergence_Smith,Gary.png Source: https://sebokwiki.org/draft/w/index.php?title=File:TypesofEmergence_Smith,Gary.png License: unknown Contributors: -

File:Complexity and Emergence in Systems.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Complexity_and_Emergence_in_Systems.png License: unknown Contributors: -

File:Feedback Stocks and Flows.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Feedback_Stocks_and_Flows.png License: unknown Contributors: -

File:Entropy Screencap_Smith,Gary.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Entropy_Screencap_Smith,Gary.png License: unknown Contributors: -

File:Information, Organisation and Entropy.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Information_Organisation_and_Entropy.png License: unknown Contributors: -

File:SystemInteractions_Smith,Gary.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SystemInteractions_Smith,Gary.png License: unknown Contributors: -

File:Holarchy, Multicausation and generative system levels.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Holarchy_Multicausation_and_generative_system_levels.png License: unknown Contributors: -

File:Fig2_Systems_Thinking_and_Systems_Science_RA.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig2_Systems_Thinking_and_Systems_Science_RA.png License: unknown Contributors: Bkcase, Mhenshaw, Smenck2

File:Picture1.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:Picture1.png> License: unknown Contributors: Mhenshaw, Smenck2

File:060611_SF_System_Concept_Model-Top_Levelnofig10.png Source: https://sebokwiki.org/draft/w/index.php?title=File:060611_SF_System_Concept_Model-Top_Levelnofig10.png License: unknown Contributors: Mhenshaw, Smenck2

File:Fig_1_Systems_Engineering_and_the_Systems_Approach_RA.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig_1_Systems_Engineering_and_the_Systems_Approach_RA.png License: unknown Contributors: Bkcase, Mhenshaw, Smenck2

File:052311_SJ_System_Coupling_Diagram.png Source: https://sebokwiki.org/draft/w/index.php?title=File:052311_SJ_System_Coupling_Diagram.png License: unknown Contributors: Mhenshaw, Smenck2

File:Ellipse Graphic J Ring.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Ellipse_Graphic_J_Ring.png License: unknown Contributors: Bkcase, Mhenshaw, Smenck2

File:Fig_3_Complex_Systems_Engineering_Diagram_Sillitto2010.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig_3_Complex_Systems_Engineering_Diagram_Sillitto2010.png License: unknown Contributors: Bkcase, Mhenshaw

File:Fig_1_SA_Activities_Through_Life_RA.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig_1_SA_Activities_Through_Life_RA.png License: unknown Contributors: Bkcase, Cnielsen, Mhenshaw

File:SEBoK_Context_Diagram_Inner_P3_Ifezue_Obiako.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P3_Ifezue_Obiako.png License: unknown Contributors: Bkcase

File:STEMArticle_DifferentViewsofSE.png Source: https://sebokwiki.org/draft/w/index.php?title=File:STEMArticle_DifferentViewsofSE.png License: unknown Contributors: Bkcase

File:Schindel_TheSystemPerspective.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Schindel_TheSystemPerspective.png License: unknown Contributors: Bkcase

File:Screen_Shot_2022-05-20_at_3.42.18_PM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Screen_Shot_2022-05-20_at_3.42.18_PM.png License: unknown Contributors: Bkcase

File:Chemical_Interactions_Phenomena_Principles.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Chemical_Interactions_Phenomena_Principles.png License: unknown Contributors: Bkcase

File:Ground_and_Marine_Vehicles_Aircraft_Regulation_in_Organisms.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Ground_and_Marine_Vehicles_Aircraft_Regulation_in_Organisms.png License: unknown Contributors: Bkcase

File:Summary_View_of_S*Metamodel.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Summary_View_of_S*Metamodel.png License: unknown Contributors: Bkcase

File:LC Spectrum.png Source: https://sebokwiki.org/draft/w/index.php?title=File:LC_Spectrum.png License: unknown Contributors: Cdhoffman

File:Comparisons of life cycle models.PNG Source: https://sebokwiki.org/draft/w/index.php?title=File:Comparisons_of_life_cycle_models.PNG License: unknown Contributors: Bkcase, Janthony, Mhenshaw

File:System life cycle stages Yokell.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_life_cycle_stages_Yokell.png License: unknown Contributors: Cdhoffman

File:Figure 2-8.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_2-8.png License: unknown Contributors: Cdhoffman

File:Figure 2.3 Criteria for decision gates Yokell.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_2.3_Criteria_for_decision_gates_Yokell.png License: unknown Contributors: Cdhoffman

File:Fig 2 2-1-color.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig_2_2-1-color.png License: unknown Contributors: Cdhoffman

File:@@BKCASE_Wiki_Section_2.5_Fig_2a_PDF_110820.png Source: https://sebokwiki.org/draft/w/index.php?title=File:@@BKCASE_Wiki_Section_2.5_Fig_2a_PDF_110820.png License: unknown Contributors: Mhenshaw, Smenck2

File:KF_IncrementalCommitmentSpiral.png Source: https://sebokwiki.org/draft/w/index.php?title=File:KF_IncrementalCommitmentSpiral.png License: unknown Contributors: Mhenshaw, Smenck2

File:RC Graphic Figure 2.9 SEH original.png Source: https://sebokwiki.org/draft/w/index.php?title=File:RC_Graphic_Figure_2.9_SEH_original.png License: unknown Contributors: Cdhoffman

File:SE Hump diagram.PNG Source: https://sebokwiki.org/draft/w/index.php?title=File:SE_Hump_diagram.PNG License: unknown Contributors: Bkcase

File:Ex_Itera_of_processes_related_to_Sys_Def_AF_052312.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Ex_Itera_of_processes_related_to_Sys_Def_AF_052312.png License: unknown Contributors: Mhenshaw, Smenck2

File:JS_Figure_1.png Source: https://sebokwiki.org/draft/w/index.php?title=File:JS_Figure_1.png License: unknown Contributors: Mhenshaw, Smenck2

File:Hierarchical_decomposition_of_a_system-of-interest_060612.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Hierarchical_decomposition_of_a_system-of-interest_060612.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:Recursion_of_processes_on_layers_060612.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Recursion_of_processes_on_layers_060612.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:semp_and_integrated_plans.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Semp_and_integrated_plans.png License: unknown Contributors: Mhenshaw, Smenck2

File:Decision_Mgt_Process_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Decision_Mgt_Process_DM.png License: unknown Contributors: Smenck2

File:Fund_Obj_Hierarchy_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fund_Obj_Hierarchy_DM.png License: unknown Contributors: Smenck2

File:Value_Function_Example_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Value_Function_Example_DM.png License: unknown Contributors: Smenck2

File:Swing_Weight_Matrix_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Swing_Weight_Matrix_DM.png License: unknown Contributors: Smenck2

File:Descript_of_Alt_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Descript_of_Alt_DM.png License: unknown Contributors: Smenck2

File:ALT_Scores_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ALT_Scores_DM.png License: unknown Contributors: Smenck2

File:Value_Scorecard_w_Heat_Map_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Value_Scorecard_w_Heat_Map_DM.png License: unknown Contributors: Smenck2

File:Eq_1.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Eq_1.jpg License: unknown Contributors: Smenck2

File:Eq_2.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Eq_2.jpg License: unknown Contributors: Smenck2

File:Value_Comp_Graph_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Value_Comp_Graph_DM.png License: unknown Contributors: Smenck2

File:Ex_Stakeholder_Value_Scat_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Ex_Stakeholder_Value_Scat_DM.png License: unknown Contributors: Smenck2

File:Uncertainty_on_Perf_Value_from_Monte_DM.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Uncertainty_on_Perf_Value_from_Monte_DM.png License: unknown Contributors: Smenck2

File:Figure ReqtMgt-1 GtNR Figure 22.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_ReqtMgt-1_GtNR_Figure_22.jpg License: unknown Contributors: User:Cdhoffman

File:Figure ReqtMgt-2 NRM Figure 16-1.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_ReqtMgt-2_NRM_Figure_16-1.jpg License: unknown Contributors: User:Cdhoffman

File:CM section Figure1.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:CM_section_Figure1.jpg License: unknown Contributors: Cdhoffman

File:CM section Figure2.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:CM_section_Figure2.jpg License: unknown Contributors: Cdhoffman

File:CM Figure 25-1053 v1.2.png Source: https://sebokwiki.org/draft/w/index.php?title=File:CM_Figure_25-1053_v1.2.png License: unknown Contributors: Cdhoffman

File:CM Table 25-1054 v1.3.png Source: https://sebokwiki.org/draft/w/index.php?title=File:CM_Table_25-1054_v1.3.png License: unknown Contributors: Cdhoffman

File:PM_SE_System_Operation_Relationships.png Source: https://sebokwiki.org/draft/w/index.php?title=File:PM_SE_System_Operation_Relationships.png License: unknown Contributors: Cdhoffman

File:IM-Focused SE Process.png Source: https://sebokwiki.org/draft/w/index.php?title=File:IM-Focused_SE_Process.png License: unknown Contributors: Cdhoffman

File:Information Management Process.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Information_Management_Process.png License: unknown Contributors: Cdhoffman

File:Measurement_Process_Model-Figure_1.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Measurement_Process_Model-Figure_1.png License: unknown Contributors: Mhenshaw, Smenck2

File:Composition_of_Leading_Indicator-Figure_2.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Composition_of_Leading_Indicator-Figure_2.png License: unknown Contributors: Mhenshaw, Smenck2

File:Measurement Figure3.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Measurement_Figure3.png License: unknown Contributors: Cdhoffman

File:Figure BusMissAnl-1 SEBoK Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_BusMissAnl-1_SEBoK_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure BusMissAnl-3 SEBoK Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_BusMissAnl-3_SEBoK_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure StkhldrNeeds-1 from INCOSE NRM.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_StkhldrNeeds-1_from_INCOSE_NRM.jpg License: unknown Contributors: User:Cdhoffman

File:Figure StkhldrNeeds-2 SEBoK Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_StkhldrNeeds-2_SEBoK_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure StkhldrNeeds-3 from INCOSE NRM.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_StkhldrNeeds-3_from_INCOSE_NRM.jpg License: unknown Contributors: User:Cdhoffman

File:Figure StkhldrNeeds-4 from INCOSE NRM.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_StkhldrNeeds-4_from_INCOSE_NRM.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-1 Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-1_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-2 Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-2_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-3 NRM Figure 6-20.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-3_NRM_Figure_6-20.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-4 Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-4_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-5 NRM Figure 6-10.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-5_NRM_Figure_6-10.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-6 Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-6_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-7 Original.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-7_Original.jpg License: unknown Contributors: User:Cdhoffman

File:Figure SysReq-8 NRM Figure 6-7.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_SysReq-8_NRM_Figure_6-7.jpg License: unknown Contributors: User:Cdhoffman

File:The System Phenomenon – Hamilton's Principal.png Source: https://sebokwiki.org/draft/w/index.php?title=File:The_System_Phenomenon_-_Hamilton's_Principal.png License: unknown Contributors: User:Cdhoffman

File:System Architecture Design Definition Approach.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_Architecture_Design_Definition_Approach.png License: unknown Contributors: User:Cdhoffman

File:System Architecture Design Model Organization Example.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_Architecture_Design_Model_Organization_Example.png License: unknown Contributors: User:Cdhoffman

File:System Architecture Design Model Viewpoints and Views.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_Architecture_Design_Model_Viewpoints_and_Views.png License: unknown Contributors: User:Cdhoffman

File:System Architecture Design Model Viewpoints and Views - Concluded.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_Architecture_Design_Model_Viewpoints_and_Views_-_Concluded.png License: unknown Contributors: User:Cdhoffman

File:Model-based System Design Ecosystem Example.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Model-based_System_Design_Ecosystem_Example.png License: unknown Contributors: User:Cdhoffman

File:System Architecture Design Definition and Verification.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_Architecture_Design_Definition_and_Verification.png License: unknown Contributors: User:Cdhoffman

File:Automobile Product Design Example.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Automobile_Product_Design_Example.png License: unknown Contributors: User:Cdhoffman

File:Automobile System Context and Use Cases Example.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Automobile_System_Context_and_Use_Cases_Example.png License: unknown Contributors: User:Cdhoffman

File:Figure 1 - Functional Behavior Viewpoint-View.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_1_-_Functional_Behavior_Viewpoint-View.png License: unknown Contributors: Cdhoffman

File:2 - Automobile Use Case.bmp Source: https://sebokwiki.org/draft/w/index.php?title=File:2_-_Automobile_Use_Case.bmp License: unknown Contributors: User:Cdhoffman

File:3 - Automobile Functions.bmp Source: https://sebokwiki.org/draft/w/index.php?title=File:3_-_Automobile_Functions.bmp License: unknown Contributors: User:Cdhoffman

File:4 - Control Vehicle Acceleration.bmp Source: https://sebokwiki.org/draft/w/index.php?title=File:4_-_Control_Vehicle_Acceleration.bmp License: unknown Contributors: User:Cdhoffman

File:5 - Automobile Operating States.bmp Source: https://sebokwiki.org/draft/w/index.php?title=File:5_-_Automobile_Operating_States.bmp License: unknown Contributors: User:Cdhoffman

File:Decomposition_of_Functions_AF_071112(2).png Source: [https://sebokwiki.org/draft/w/index.php?title=File:Decomposition_of_Functions_AF_071112\(2\).png](https://sebokwiki.org/draft/w/index.php?title=File:Decomposition_of_Functions_AF_071112(2).png) License: unknown Contributors: Mhenshaw, Smenck2

File:Illustration_of_a_scenario_(eFFBD)_AF_071112.png Source: [https://sebokwiki.org/draft/w/index.php?title=File:Illustration_of_a_scenario_\(eFFBD\)_AF_071112.png](https://sebokwiki.org/draft/w/index.php?title=File:Illustration_of_a_scenario_(eFFBD)_AF_071112.png) License: unknown Contributors: Mhenshaw, Smenck2

File:Illustration_of_a_scenario_Activity_Diagram_AF_071112.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Illustration_of_a_scenario_Activity_Diagram_AF_071112.png License: unknown Contributors: Mhenshaw, Smenck2

File:SEBoKv075_KA-SystDef_Scenario_of_Operational_Modes.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv075_KA-SystDef_Scenario_of_Operational_Modes.png License: unknown Contributors: Mhenshaw, Smenck2

File:SEBoKv05_KA-SystDef_Temporal_and_decision_hierarchy_levels.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv05_KA-SystDef_Temporal_and_decision_hierarchy_levels.png License: unknown Contributors: Mhenshaw, Smenck2

File:SEBoKv075_KA-SystDef_Limited_nb_in_decomposition.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv075_KA-SystDef_Limited_nb_in_decomposition.png License: unknown Contributors: Mhenshaw, Smenck2

File:SEBoKv075_KA-SystDef_Encapsulation.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv075_KA-SystDef_Encapsulation.png License: unknown Contributors: Mhenshaw, Smenck2

File:JS_Figure_2.png Source: https://sebokwiki.org/draft/w/index.php?title=File:JS_Figure_2.png License: unknown Contributors: Mhenshaw, Smenck2

File:JS_Figure_3.png Source: https://sebokwiki.org/draft/w/index.php?title=File:JS_Figure_3.png License: unknown Contributors: Mhenshaw, Smenck2

File:SEBoKv05_KA-SystRealiz_how_outputs_of_Definition_relate_to_Implementation.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv05_KA-SystRealiz_how_outputs_of_Definition_relate_to_Implementation.png License: unknown Contributors: Mhenshaw, Smenck2

File:JS_Figure_5.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:JS_Figure_5.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SEBoKv05_KA-SystRealiz_Implementation_relationships.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv05_KA-SystRealiz_Implementation_relationships.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Limits_of_integration_activities.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Limits_of_integration_activities.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SEBoKv05_KA-SystRealiz_Integration_relationships.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv05_KA-SystRealiz_Integration_relationships.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:JS_Figure_9.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:JS_Figure_9.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Definition_and_usage_of_a_Verification_Action.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Definition_and_usage_of_a_Verification_Action.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:052611_SJ_notional_reliability_analysis.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:052611_SJ_notional_reliability_analysis.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Definition_and_usage_of_a_Validation_Action.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Definition_and_usage_of_a_Validation_Action.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Verification_and_Validation_level_per_level.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Verification_and_Validation_level_per_level.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SEBoKv05_KA-SystRealiz_V&V_Actions_upper_levels.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv05_KA-SystRealiz_V&V_Actions_upper_levels.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SEBoKv05_KA-SystRealiz_V&V_Actions_lower_levels.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SEBoKv05_KA-SystRealiz_V&V_Actions_lower_levels.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Affordable_Sys_Ops_Effect_DAU_GB_Roedler.jpg *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Affordable_Sys_Ops_Effect_DAU_GB_Roedler.jpg *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Sustainment_Implementation_Illustration_Logistics_Roedler.jpg *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Sustainment_Implementation_Illustration_Logistics_Roedler.jpg *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:052411_BSBW_The_Vee_Model.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:052411_BSBW_The_Vee_Model.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Standards Evolution Figure - Final 20250519R1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Standards_Evolution_Figure_-_Final_20250519R1.png *License:* unknown *Contributors:* -

File:20260508_Adapted from Standard TaskForce.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:20260508_Adapted from Standard TaskForce.png *License:* unknown *Contributors:* -

File:A Taxonomy of SE Related Standards.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:A_Taxonomy_of_SE_Related_Standards.png *License:* unknown *Contributors:* Cdhoffman

File: XKCD standards.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:XKCD_standards.png *License:* unknown *Contributors:* Cdhoffman

File:Collaboration Across Key SE Standards and References.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Collaboration_Across_Key_SE_Standards_and_References.png *License:* unknown *Contributors:* Cdhoffman

File:Fig_3_Growing_Industry_Collaboration_GR.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Fig_3_Growing_Industry_Collaboration_GR.png *License:* unknown *Contributors:* Bkcase, Mhenshaw

File:Potential_Standards_Influence_of_Organization_and_Project_Processes.PNG *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Potential_Standards_Influence_of_Organization_and_Project_Processes.PNG *License:* unknown *Contributors:* Cdhoffman, Janthony, Mhenshaw

Article Sources and Contributors

Applications of Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77994> *Contributors:* Bkcase, Cdhoffman, Dcarey, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert

Product Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78009> *Contributors:* Bkcase, Cdhoffman, Dcarey, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Skmackin, Smenck2, WikiWorks, Wikiexpert

Product Systems Engineering Background *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78444> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Product as a System Fundamentals *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78280> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Business Activities Related to Product Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78439> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Rlpineda, WikiWorks, Wikiexpert

Product Systems Engineering Key Aspects *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78318> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Product Systems Engineering Special Activities *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78423> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Service Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78267> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Rturner, Smenck2, WikiWorks, Wikiexpert

Service Systems Background *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78154> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Fundamentals of Services *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78568> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Properties of Services *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77418> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Kguillemette, Mhaas, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Scope of Service Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78558> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Value of Service Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78563> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rlpineda, Rturner, Smenck2, WikiWorks, Wikiexpert

Service Systems Engineering Stages *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78573> *Contributors:* Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Janthony, Jgercken, Kguillemette, Mhaas, Rcloutier, Rlpineda, Rturner, Smenck2, WikiWorks, Wikiexpert

Enterprise Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78150> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Jmartin, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Enterprise Systems Engineering Background *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78251> *Contributors:* Afaisandier, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Jmartin, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

The Enterprise as a System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78570> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Jmartin, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Related Business Activities *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78561> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Jmartin, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Enterprise Systems Engineering Key Concepts *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78571> *Contributors:* Afaisandier, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Jmartin, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Enterprise Systems Engineering Process Activities *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78387> *Contributors:* Afaisandier, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Jmartin, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Enterprise Capability Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78180> *Contributors:* Afaisandier, Asquires, Bkcase, Cdhoffman, Dhenry, Janthony, Jgercken, Jmartin, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Systems of Systems (SoS) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78580> *Contributors:* Apyster, Bkcase, Cdhoffman, Cwright, Dhenry, Dholwell, Dnewbern, Gconover, Janthony, Jdahmann, Jgercken, Kguillemette, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Architecting Approaches for Systems of Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78069> *Contributors:* Apyster, Bkcase, Cdhoffman, Cwright, Dhenry, Dholwell, Dnewbern, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Skmackin, WikiWorks, Wikiexpert

Systems Engineering for Systems of Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77429> *Contributors:* Cdhoffman, WikiWorks

Systems of Systems Analytic Approaches *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77430> *Contributors:* Cdhoffman, WikiWorks

System of Systems and Complexity *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77431> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Socio-Technical Features of Systems of Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78335> *Contributors:* Afaisandier, Bkcase, Cdhoffman, Cwright, Dhenry, Dholwell, Dnewbern, Janthony, Jdahmann, Jgercken, Kguillemette, Mhaas, Mhenshaw, Radcock, Skmackin, WikiWorks, Wikiexpert

Capability Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78041> *Contributors:* Apyster, Bkcase, Cdhoffman, Cwright, Dhenry, Dholwell, Dnewbern, Gconover, Janthony, Jgercken, Kguillemette, Mhaas, Skmackin, WikiWorks, Wikiexpert

Mission Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77434> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Healthcare Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78407> *Contributors:* Bkcase, Cdhoffman, Gconover, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Overview of the Healthcare Sector *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78362> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Systems Engineering in Healthcare Delivery *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77436> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, WikiWorks

Systems Biology *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78470> *Contributors:* Bkcase, Cdhoffman, Gconover, Mhaas, Nicole.hutchison, WikiWorks

Lean in Healthcare *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78462> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, WikiWorks

Enabling Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78478> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Gparnell, Janthony, Jgercken, Kguillemette, Mhaas, Radcock, Smenck2, WikiWorks, Wikiexpert

Enabling Businesses and Enterprises *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78449> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Rbeasley, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Systems Engineering Organizational Strategy *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78368> *Contributors:* Afaisandier, Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Jgercken, Kguillemette, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Determining Needed Systems Engineering Capabilities in Businesses and Enterprises *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78433> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Groedler, Hsillitto, Jgercken, Jtcarter, Kguillemette, Mhaas, Mhenshaw, Radcock, Rbeasley, Rmadachy, Rturner, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Organizing Business and Enterprises to Perform Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78144> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rbeasley, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Assessing Systems Engineering Performance of Business and Enterprises *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78244> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Jgercken, Mhaas, Mhenshaw, Rbeasley, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Developing Systems Engineering Capabilities within Businesses and Enterprises *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78245> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Mhenshaw, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Barriers to Successful Embedding of Systems Engineering into Organizations *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77448> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Culture *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78369> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Gparnell, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rbeasley, Rturner, Smenck2, WikiWorks, Wikiexpert

Enabling Teams *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78535> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dfairley, Dhenry, Dholwell, Gconover, Gparnell, Jgercken, Kguillemette, Mhaas, Smenck2, WikiWorks, Wikiexpert

Team Capability *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78281> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dfairley, Dhenry, Dholwell, Gparnell, Hdavidz, Jgercken, Kguillemette, Mhaas, Skmackin, Smenck2, WikiWorks, Wikiexpert

Team Dynamics *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77453> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dfairley, Dhenry, Dholwell, Janthony, Jgercken, Mhaas, Skmackin, Smenck2, WikiWorks, Wikiexpert

Diversity, Equity, and Inclusion *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78184> *Contributors:* Bkcase, Cdhoffman, Gconover, Mhaas, WikiWorks

Technical Leadership in Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77708> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Eleach, Hdavidz, Kguillemette, Mhaas, Smenck2, WikiWorks

Enabling Individuals *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77442> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gparnell, Hdavidz, Janthony, Jgercken, Mhaas, Skmackin, Smenck2, WikiWorks, Wikiexpert

Roles and Competencies *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78306> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Hdavidz, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Assessing Individuals *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78366> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Hdavidz, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Developing Individuals *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78398> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Hdavidz, Janthony, Jgercken, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert

Ethical Behavior *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78271> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gparnell, Hdavidz, Janthony, Jgercken, Kguillemette, Mhaas, Sackson, Skmackin, Smenck2, WikiWorks, Wikiexpert

Related Disciplines *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77481> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Csingam, Dhenry, Dholwell, Jgercken, Kguillemette, Mhaas, Radcock, SYSIND11, Smenck2, Thilburn, WikiWorks, Wikiexpert, Ymordecai

Systems Engineering and Environmental Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78749> *Contributors:* Apyster, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Dnewbern, Gconover, Kguillemette, Mhaas, Mhenshaw, Phisterp, Radcock, Smenck2, WikiWorks, Wikiexpert

Systems Engineering and Geospatial/Geodetic Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78750> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, Ulenk, WikiWorks

Overview of Geospatial/Geodetic Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78751> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, Ulenk, WikiWorks

Relationship between Systems Engineering and Geospatial/Geodetic Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78752> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, Ulenk, WikiWorks

Further Insights into Geospatial/Geodetic Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78753> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, Ulenk, WikiWorks

Systems Engineering and Industrial Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78814> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Hsillitto, Kguillemette, Mhaas, SYSIND11, Smenck2, WikiWorks, Wikiexpert

Systems Engineering and Project Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78815> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Kguillemette, Mhaas, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

The Nature of Project Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78816> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Kguillemette, Mhaas, Rturner, Smenck2, WikiWorks, Wikiexpert, Ymordecai

An Overview of the PMBOK® Guide *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78817> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Csingam, Dhenry, Dholwell, Gconover, Kguillemette, Mhaas, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Relationships between Systems Engineering and Project Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78818> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Jgercken, Kguillemette, Mhaas, Mhenshaw, Rturner, Smenck2, WikiWorks, Wikiexpert

The Influence of Project Structure and Governance on Systems Engineering and Project Management Relationships *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78819> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Dnewbern, Gconover, Kguillemette, Mhaas, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert

Cost Estimating and Analysis in Systems Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78820> *Contributors:* Bkcase, Cdhoffman, Csingam, Gconover, WikiWorks

Procurement and Acquisition *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78821> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Jgercken, Kguillemette, Mhaas, Mhenshaw, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Portfolio Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78822> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, WikiWorks

Systems Engineering and Software Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78823> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Mhenshaw, Radcock, Skmackin, Thilburn, WikiWorks, Wikiexpert, Ymordecai

Software Engineering in the Systems Engineering Life Cycle *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78824> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Gconover, Mhaas, Radcock, Thilburn, WikiWorks

The Nature of Software *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78840> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Thilburn, WikiWorks, Wikiexpert, Ymordecai

An Overview of the SWEBOK Guide *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78841> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dfairley, Dhenry, Dholwell, Gconover, Jgercken, Kguillemette, Mhaas, Smenck2, Thilburn, WikiWorks, Wikiexpert, Ymordecai

Key Points a Systems Engineer Needs to Know about Software Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78844> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dfairley, Dhenry, Dholwell, Eleach, Gconover, Jgercken, Kguillemette, Mhaas, Mhenshaw, Smenck2, Thilburn, WikiWorks, Wikiexpert, Ymordecai

Software Engineering Features - Models, Methods, Tools, Standards, and Metrics *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78843> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, Thilburn, WikiWorks

Systems Engineering and Aerospace Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76538> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Systems Engineering and Electrical Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76887> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Systems Engineering and Mechanical Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78845> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, WikiWorks

Systems Engineering and Civil Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76257> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Systems Engineering and Economics *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76221> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Systems Engineering and Enterprise IT *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78846> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, WikiWorks

Systems Engineering and Quality Attributes *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78847> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Nicole.hutchison, Phisterp, Radcock, Rturner, S.Jackson, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

A Framework for Viewing Quality Attributes from the Lens of Loss *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78848> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhaas, Rcloutier, WikiWorks

Human Systems Integration *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78849> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Mhenshaw, Phisterp, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

Manufacturability and Producibility *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78850> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Gparnell, Jgercken, Mhaas, Nicole.hutchison, Phisterp, Radcock, Rturner, Skmackin, WikiWorks, Wikiexpert

System Affordability *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78016> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Gparnell, Kguillemette, Mhaas, Phisterp, Radcock, Rmadachy, Smenck2, WikiWorks, Wikiexpert

System Hardware Assurance *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77494> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhaas, WikiWorks

System Reliability, Availability, and Maintainability *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78516> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Dholwell2, Gconover, Jgercken, Mhaas, Mhenshaw, Phisterp, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

System Resilience *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78434> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Csingam, Dhenry, Dholwell, Gparnell, Jbtris, Jgercken, Kguillemette, Mhaas, Mhenshaw, Nicole.hutchison, Phisterp, Radcock, Rturner, S.Jackson, Skmackin, Smenck2, WikiWorks, Wikiexpert

Resilience Modeling *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77549> *Contributors:* Bkcase, Cdhoffman, Csingam, WikiWorks

System Resistance to Electromagnetic Interference *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78332> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gparnell, Jgercken, Mhaas, Mhenshaw, Nicole.hutchison, Phisterp, Radcock, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

System Safety *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78290> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gconover, Jgercken, Mhaas, Mhenshaw, Phisterp, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert, Ymordecai

System Security *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78370> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Cnielsen, Dhenry, Dholwell, Gparnell, Jgercken, Mhaas, Phisterp, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Systems Engineering Implementation Examples *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78172> *Contributors:* Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Hdavidz, Jgercken, Kguillemette, Mhaas, Nicole.hutchison, Radcock, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

Matrix of Implementation Examples *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77733> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Kguillemette, Mhaas, Mhenshaw, Nicole.hutchison, Radcock, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

Implementation Examples *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77503> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Submarine Warfare Federated Tactical Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78394> *Contributors:* Bkcase, Cbaldwin, Cdhoffman, Mhaas, Nicole.hutchison, WikiWorks

Virginia Class Submarine *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78140> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Complex Adaptive Taxi Service Scheduler *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77525> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Successful Business Transformation within a Russian Information Technology Company *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78246> *Contributors:* Bkcase, Bwhite, Cdhoffman, Cnielsen, Kguillemette, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Federal Bureau of Investigation (FBI) Virtual Case File System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78199> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Skmackin, Smenck2, WikiWorks, Wikiexpert

Project Management for a Complex Adaptive Operating System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78365> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Next Generation Medical Infusion Pump *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78020> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Hdavidz, Jgercken, Jcarter, Mhaas, Nicole.hutchison, Smenck2, WikiWorks, Wikiexpert

Medical Radiation *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78436> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Kguillemette, Mhaas, Nicole.hutchison, Smenck2, WikiWorks, Wikiexpert

Design for Maintainability *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77524> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Global Positioning System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78207> *Contributors:* Apyster, Asquires, Bkcase, Bwhite, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Radcock, Smenck2, WikiWorks, Wikiexpert

Global Positioning System II *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78461> *Contributors:* Bkcase, Bwhite, Cdhoffman, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Russian Space Agency Project Management Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77515> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, Radcock, WikiWorks

How Lack of Information Sharing Jeopardized the NASA/ESA Cassini/Huygens Mission to Saturn *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78274> *Contributors:* Bkcase, Bwhite, Cdhoffman, Kguillemette, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Hubble Space Telescope *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78364> *Contributors:* Apyster, Asquires, Bkcase, Bwhite, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Radcock, Smenck2, WikiWorks, Wikiexpert

Applying a Model-Based Approach to Support Requirements Analysis on the Thirty-Meter Telescope *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78375> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, WikiWorks

Miniature Seeker Technology Integration Spacecraft *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78209> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Skmackin, Smenck2, WikiWorks, Wikiexpert

Apollo 1 Disaster *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78175> *Contributors:* Bkcase, Cdhoffman, Gconover, Mhaas, WikiWorks

Denver Airport Baggage Handling System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78186> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Kguillemette, Mhaas, Nicole.hutchison, Radcock, Skmackin, Smenck2, WikiWorks, Wikiexpert

Federal Aviation Administration (FAA) Advanced Automation System (AAS) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78078> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Gconover, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Radcock, Rturner, WikiWorks, Wikiexpert

Federal Aviation Administration (FAA) Next Generation Air Transportation System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78469> *Contributors:* Bkcase, Bwhite, Cdhoffman, Cnielsen, Kguillemette, Mhaas, Nicole.hutchison, Radcock, WikiWorks

Reverse Engineering a UAV Prototype using Agile Practices *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77510> *Contributors:* Bkcase, Cdhoffman, WikiWorks

UK West Coast Route Modernisation Project *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78216> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Radcock, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

Standard Korean Light Transit System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77512> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Jcarter, Mhaas, Mhenshaw, Nicole.hutchison, Radcock, Rturner, Skmackin, Smenck2, WikiWorks, Wikiexpert

Northwest Hydro System *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78311> *Contributors:* Bkcase, Cdhoffman, Mhaas, Nicole.hutchison, WikiWorks

Singapore Water Management *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78575> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Dholwell, Hdavidz, Jgercken, Mhaas, Nicole.hutchison, Radcock, Rturner, Skmackin, WikiWorks, Wikiexpert

Emerging Knowledge *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77530> *Contributors:* Bkcase, Cdhoffman, Ddelaurentis, Hle, Mhaas, WikiWorks

Emerging Topics *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77955> *Contributors:* Bkcase, Cdhoffman, Gconover, Hle, Mhaas, WikiWorks

Introduction to SE Transformation *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78295> *Contributors:* Bkcase, Cdhoffman, Mhaas, Radcock, WikiWorks

Socio-technical Systems *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77534> *Contributors:* Bkcase, Cdhoffman, Hle, Mhaas, WikiWorks

Artificial Intelligence *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78055> *Contributors:* Bkcase, Cdhoffman, Gconover, Mhaas, WikiWorks

Verification and Validation of Systems in Which AI is a Key Element *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78329> *Contributors:* Bkcase, Cdhoffman, Hle, Mhaas, WikiWorks

Transitioning Systems Engineering to a Model-based Discipline *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78404> *Contributors:* Bkcase, Cdhoffman, Mhaas, Radcock, Rcloutier, WikiWorks

Model-Based Systems Engineering Adoption Trends 2009-2018 *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78248> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Digital Engineering *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77539> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Set-Based Design *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77540> *Contributors:* Bkcase, Cdhoffman, Mhaas, Rcloutier, WikiWorks

Emerging Research *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77532> *Contributors:* Bkcase, Cdhoffman, Hle, Mhaas, WikiWorks

Image Sources, Licenses and Contributors

File:SEBoK_Context_Diagram_Inner_P4_Ifezue_Obiako.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P4_Ifezue_Obiako.png *License:* unknown *Contributors:* Bkcase

File:PPI.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:PPI.png> *License:* unknown *Contributors:* Bkcase

File:PSE_Intro_Fig1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_Intro_Fig1.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_Intro_Fig2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_Intro_Fig2.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_Intro_Fig3.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_Intro_Fig3.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_Intro_Fig4.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_Intro_Fig4.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:IPDP PSE Background Figure 1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:IPDP_PSE_Background_Figure_1.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PSEB_Fig2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PSEB_Fig2.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PAAS_Fig1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PAAS_Fig1.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PAAS_Fig2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PAAS_Fig2.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PSEKA_Fig1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PSEKA_Fig1.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PSEKA_Fig2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PSEKA_Fig2.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PSEKA_Fig3.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PSEKA_Fig3.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PSEKA_Fig4.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PSEKA_Fig4.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:PSE_PSESA_Fig1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:PSE_PSESA_Fig1.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SSE_SSB_Fig1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SSE_SSB_Fig1.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SSE_FOS_Fig1_no_white_space.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SSE_FOS_Fig1_no_white_space.png *License:* unknown *Contributors:* Smenck2

File:SSE_FOS_Fig3.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SSE_FOS_Fig3.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SSE_FOS_Fig4.PNG *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SSE_FOS_Fig4.PNG *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:SSE_VoSSE_Fig1v2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SSE_VoSSE_Fig1v2.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F01.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F01.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F02.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F02.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F03.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F03.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F04.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F04.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F05.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F05.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F07.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F07.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F08.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F08.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F09.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F09.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F10.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F10.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F11.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F11.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F12.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F12.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F13.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F13.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F15.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F15.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F06.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F06.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F30_Process_Breakdown.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:ESE-F30_Process_Breakdown.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F21.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F21.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:ESE-F22.png *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:ESE-F22.png> *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Scale and Scope of SoS.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Scale_and_Scope_of_SoS.png *License:* unknown *Contributors:* Cdhoffman

File:Use Cases ISO 14971 15288.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Use_Cases_ISO_14971_15288.png *License:* unknown *Contributors:* Bkcase

File:SEBoK_Context_Diagram_Inner_P5_Ifezue_Obiako.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P5_Ifezue_Obiako.png *License:* unknown *Contributors:* Bkcase

File:Organizational_coupling_diagram_v2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Organizational_coupling_diagram_v2.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:System_enterprises_and_organizations_v2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:System_enterprises_and_organizations_v2.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Culture_competence_team_performance_and_individual_competence.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Culture_competence_team_performance_and_individual_competence.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Part_5_(Organization)_Concept_maps_additions_hgs_15_Aug.png *Source:* [https://sebokwiki.org/draft/w/index.php?title=File:Part_5_\(Organization\)_Concept_maps_additions_hgs_15_Aug.png](https://sebokwiki.org/draft/w/index.php?title=File:Part_5_(Organization)_Concept_maps_additions_hgs_15_Aug.png) *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Picture1_HGS.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Picture1_HGS.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:TPM_Chart_from_INCOSE_SELIG.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:TPM_Chart_from_INCOSE_SELIG.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Concept_map_for_businesses_and_enterprises_topics.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Concept_map_for_businesses_and_enterprises_topics.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:The Capability Model Strandberg 1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:The_Capability_Model_Strandberg_1.png *License:* unknown *Contributors:* Cdhoffman

File:The Capability Model Strandberg 2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:The_Capability_Model_Strandberg_2.png *License:* unknown *Contributors:* Cdhoffman

File:Steps to implement SE.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Steps_to_implement_SE.png *License:* unknown *Contributors:* Cdhoffman

File:Fairley_Fig_1_(2)_Layer_1.png *Source:* [https://sebokwiki.org/draft/w/index.php?title=File:Fairley_Fig_1_\(2\)_Layer_1.png](https://sebokwiki.org/draft/w/index.php?title=File:Fairley_Fig_1_(2)_Layer_1.png) *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Fairley_Fig_2_Layer_1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Fairley_Fig_2_Layer_1.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:DEL_Figure1.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:DEL_Figure1.png *License:* unknown *Contributors:* Bkcase

File:DEL_Figure2.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:DEL_Figure2.png *License:* unknown *Contributors:* Bkcase

File:Inclusive-engineering-framework-new.jpeg *Source:* <https://sebokwiki.org/draft/w/index.php?title=File:Inclusive-engineering-framework-new.jpeg> *License:* unknown *Contributors:* Bkcase

File:Dimensions_of_Communication_Styles.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Dimensions_of_Communication_Styles.png *License:* unknown *Contributors:* Smenck2

File:Layered_and_Multi-dimensional_in_the_Engineering_Layer.PNG *Source:* https://sebokwiki.org/draft/w/index.php?title=File:Layered_and_Multi-dimensional_in_the_Engineering_Layer.PNG *License:* unknown *Contributors:* Janthony, Mhenshaw, Smenck2

File:MITRE_Enterprise_Systems_Engineering_Framework.PNG Source: https://sebokwiki.org/draft/w/index.php?title=File:MITRE_Enterprise_Systems_Engineering_Framework.PNG License: unknown Contributors: Janthony, Mhenshaw, Smenck2

File:SEBoK_Context_Diagram_Inner_P6_Ifezue_Obiako.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P6_Ifezue_Obiako.png License: unknown Contributors: Bkcase

File:Environmental_Engineering_HighRes.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Environmental_Engineering_HighRes.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:Mapping_of_tech_topics_SEBoK_with_ISO_IEC_15288techPro_060612.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Mapping_of_tech_topics_SEBoK_with_ISO_IEC_15288techPro_060612.jpg License: unknown Contributors: Bkcase, Mhenshaw, Smenck2

File:IISE_BoK.jpeg Source: https://sebokwiki.org/draft/w/index.php?title=File:IISE_BoK.jpeg License: unknown Contributors: Bkcase

File:SE&IE_Venn.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SE&IE_Venn.png License: unknown Contributors: Bkcase

File:SE&IE_lifecycle_view.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SE&IE_lifecycle_view.png License: unknown Contributors: Bkcase

File:PM-SE1.jpg Source: <https://sebokwiki.org/draft/w/index.php?title=File:PM-SE1.jpg> License: unknown Contributors: Mhenshaw, Smenck2

File:PM-SE2.jpg Source: <https://sebokwiki.org/draft/w/index.php?title=File:PM-SE2.jpg> License: unknown Contributors: Mhenshaw, Smenck2

File:P6_Fig1_The_Organizational_Continuum_KN.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:P6_Fig1_The_Organizational_Continuum_KN.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:Cost Estimation Figure 1.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Cost_Estimation_Figure_1.png License: unknown Contributors: Cdhoffman

File:Cost Estimation Figure 2.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Cost_Estimation_Figure_2.png License: unknown Contributors: Cdhoffman

File:Cost Estimation Figure 3.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Cost_Estimation_Figure_3.png License: unknown Contributors: Cdhoffman

File:ACQProcessModel_NoWhiteS.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ACQProcessModel_NoWhiteS.png License: unknown Contributors: Dhenry, Mhenshaw, Smenck2

File:RelatingACQtoRFP_NoWhiteS.png Source: https://sebokwiki.org/draft/w/index.php?title=File:RelatingACQtoRFP_NoWhiteS.png License: unknown Contributors: Mhenshaw, Smenck2

File:SE_PM_PfM_DEVopsEnviornment_20201020.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:SE_PM_PfM_DEVopsEnviornment_20201020.jpg License: unknown Contributors: Bkcase

File:Fig_1_A_generic_life_cycle_KF.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig_1_A_generic_life_cycle_KF.png License: unknown Contributors: Bkcase

File:Aligned-Process-Models.PNG Source: <https://sebokwiki.org/draft/w/index.php?title=File:Aligned-Process-Models.PNG> License: unknown Contributors: Bkcase

File:Fig_1_Integration_Process_for_Specialty_Engineering.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Fig_1_Integration_Process_for_Specialty_Engineering.png License: unknown Contributors: Mhenshaw, Smenck2

File:LDSE_Figure_20200702.gif Source: https://sebokwiki.org/draft/w/index.php?title=File:LDSE_Figure_20200702.gif License: unknown Contributors: Bkcase

File:Component_Lifecycle_(rev_a)_-_MJB_ver2.jpg Source: [https://sebokwiki.org/draft/w/index.php?title=File:Component_Lifecycle_\(rev_a\)_-_MJB_ver2.jpg](https://sebokwiki.org/draft/w/index.php?title=File:Component_Lifecycle_(rev_a)_-_MJB_ver2.jpg) License: unknown Contributors: Bkcase

File:Fault_tree.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Fault_tree.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:Simple_RBD.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Simple_RBD.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:System Resilience Figure 1.png Source: https://sebokwiki.org/draft/w/index.php?title=File:System_Resilience_Figure_1.png License: unknown Contributors: Cdhoffman

File:Time-Wise_Values_of_Notional_Resilience_Scenarios_Parameters.PNG Source: https://sebokwiki.org/draft/w/index.php?title=File:Time-Wise_Values_of_Notional_Resilience_Scenarios_Parameters.PNG License: unknown Contributors: Bkcase

File:SystemResilience_Figure_2_Brtis_AffordableResilience.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SystemResilience_Figure_2_Brtis_AffordableResilience.png License: unknown Contributors: Bkcase

File:Figure 1.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_1.png License: unknown Contributors: Mhenshaw, Smenck2

File:Figure 2.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure_2.jpg License: unknown Contributors: Mhenshaw, Smenck2

File:SystemSecurityandPerformance.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:SystemSecurityandPerformance.png> License: unknown Contributors: Bkcase

File:HierarchyofUnacceptableConditions.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:HierarchyofUnacceptableConditions.png> License: unknown Contributors: Bkcase

File:SEBoK_Context_Diagram_Inner_P7_Ifezue_Obiako.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P7_Ifezue_Obiako.png License: unknown Contributors: Bkcase

File:Casestudies-2x2.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:Casestudies-2x2.png> License: unknown Contributors: Bkcase

File:AN_BSY-1 Submarine Combat System.png Source: https://sebokwiki.org/draft/w/index.php?title=File:AN_BSY-1_Submarine_Combat_System.png License: unknown Contributors: Bkcase

File:VA combat system architecture.png Source: https://sebokwiki.org/draft/w/index.php?title=File:VA_combat_system_architecture.png License: unknown Contributors: Bkcase

File:Adaptive-scheduling-taxi.PNG Source: <https://sebokwiki.org/draft/w/index.php?title=File:Adaptive-scheduling-taxi.PNG> License: unknown Contributors: Bkcase

File:MissionAndCapabilities.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:MissionAndCapabilities.png> License: unknown Contributors: Bkcase

File:V_ModelProcess of Transformation.png Source: https://sebokwiki.org/draft/w/index.php?title=File:V_Model_Process_of_Transformation.png License: unknown Contributors: Bkcase

File:Results of Tranformation.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Results_of_Tranformation.png License: unknown Contributors: Bkcase

File:Enterprise-ontology.PNG Source: <https://sebokwiki.org/draft/w/index.php?title=File:Enterprise-ontology.PNG> License: unknown Contributors: Bkcase

File:Resources-allocation.PNG Source: <https://sebokwiki.org/draft/w/index.php?title=File:Resources-allocation.PNG> License: unknown Contributors: Bkcase

File:ThirtyMeterTelescope_Figure1_TelescopeImage.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ThirtyMeterTelescope_Figure1_TelescopeImage.png License: unknown Contributors: Nicole.hutchison

File:ThirtyMeterTelescope_Figure2_EarlyLightLocations.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ThirtyMeterTelescope_Figure2_EarlyLightLocations.png License: unknown Contributors: Nicole.hutchison

File:ThirtyMeterTelescope_Figure3.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ThirtyMeterTelescope_Figure3.png License: unknown Contributors: Nicole.hutchison

File:ThirtyMeterTelescope_Figure4.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ThirtyMeterTelescope_Figure4.png License: unknown Contributors: Nicole.hutchison

File:ThirtyMeterTelescope_Figure5.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ThirtyMeterTelescope_Figure5.png License: unknown Contributors: Nicole.hutchison

File:ThirtyMeterTelescope_Figure6.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ThirtyMeterTelescope_Figure6.png License: unknown Contributors: Nicole.hutchison

File:ThirtyMeterTelescope_Figure7.png Source: https://sebokwiki.org/draft/w/index.php?title=File:ThirtyMeterTelescope_Figure7.png License: unknown Contributors: Nicole.hutchison

File:UAV_Figure 1.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:UAV_Figure_1.jpg License: unknown Contributors: Cdhoffman

File:UAV_Figure_2.png Source: https://sebokwiki.org/draft/w/index.php?title=File:UAV_Figure_2.png License: unknown Contributors: Cdhoffman

File:UAV_Figure_3.png Source: https://sebokwiki.org/draft/w/index.php?title=File:UAV_Figure_3.png License: unknown Contributors: Cdhoffman

File:UAV_Figure_4.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:UAV_Figure_4.jpg License: unknown Contributors: Cdhoffman

File:ChoeKimFigure1.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:ChoeKimFigure1.png> License: unknown Contributors: Mhenshaw, Smenck2

File:ChoeKimFigure2.png Source: <https://sebokwiki.org/draft/w/index.php?title=File:ChoeKimFigure2.png> License: unknown Contributors: Mhenshaw, Smenck2

File:SEBoK_Context_Diagram_Inner_P8_Ifezue_Obiako.png Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_Context_Diagram_Inner_P8_Ifezue_Obiako.png License: unknown Contributors: Bkcase

File:Figure1 systems subsystem.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure1_systems subsystem.png License: unknown Contributors: User:Bkcase

File:Figure2 ML.process.png Source: https://sebokwiki.org/draft/w/index.php?title=File:Figure2_ML.process.png License: unknown Contributors: User:Bkcase

File:MBSE Trends Figure 1.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_1.png License: unknown Contributors: Bkcase

File:MBSE Trends Figure 2.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_2.png License: unknown Contributors: Bkcase

File:MBSE Trends Figure 3.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_3.png License: unknown Contributors: Bkcase

File:MBSE Trends Figure 4.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_4.png License: unknown Contributors: Bkcase

File:MBSE Trends Figure 5.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_5.png License: unknown Contributors: Bkcase

File:MBSE Trends Figure 6.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_6.png License: unknown Contributors: Bkcase

File:MBSE Trends Figure 7.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_7.png License: unknown Contributors: Bkcase

File:MBSE Trends Figure 8.png Source: https://sebokwiki.org/draft/w/index.php?title=File:MBSE_Trends_Figure_8.png License: unknown Contributors: Bkcase

File:SEBok SBD Figure1.gif Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBok_SBD_Figure1.gif License: unknown Contributors: Bkcase

File:SEBok SBD Figure2.gif Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBok_SBD_Figure2.gif License: unknown Contributors: Bkcase

File:SEBok SBD Figure3.gif Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBok_SBD_Figure3.gif License: unknown Contributors: Bkcase

File:SEBok SBD Figure4.gif Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBok_SBD_Figure4.gif License: unknown Contributors: Bkcase

File:SEBoK_SBD_Figure5.jpg Source: https://sebokwiki.org/draft/w/index.php?title=File:SEBoK_SBD_Figure5.jpg License: unknown Contributors: Bkcase

SEBoK Glossary and Acronyms

Contents

Articles

Glossary	1
A	2
Absorption (glossary)	2
Abstract Model (glossary)	2
Abstract Syntax (glossary)	3
Abstraction (glossary)	3
Acceptance Criteria (glossary)	4
Acceptance Sampling (glossary)	4
Acquirer (glossary)	5
Acquisition (glossary)	5
Acquisition Strategy (glossary)	6
Activity (glossary)	6
Adaptability (glossary)	7
Address (glossary)	8
Adversity (glossary)	8
Aggregate (glossary)	8
Agile (glossary)	9
Agility (glossary)	9
Agreement (glossary)	10
Analytical Model (glossary)	10
Antipattern (glossary)	11
Architecting (glossary)	11
Architecture (glossary)	12
Architecture Framework (glossary)	13
Assembly Procedure (glossary)	14
Assembly Tool (glossary)	14
Assessment Criterion (glossary)	15
Assessment Score (glossary)	15
Attribute (glossary)	16
Availability (glossary)	17
B	18

Baseline (glossary)	18
Behavior (glossary)	19
Behavioral Architecture (glossary)	19
Binding (glossary)	20
Black-Box System (glossary)	20
Body of Knowledge (glossary)	21
Boundary (glossary)	22
Business (glossary)	22
Business Process (glossary)	22
Business Process Outsourcing (glossary)	23

C 24

Call Center (glossary)	24
Capability (glossary)	24
Capability Management (glossary)	25
Capacity (glossary)	26
Cartesian coordinate system (glossary)	26
Cartography (glossary)	27
Case Study (glossary)	27
Catalogue (glossary)	28
Chaos (glossary)	28
Closed System (glossary)	29
Coercive (glossary)	29
Cognitive Bias (glossary)	30
Cohesion (glossary)	30
Competency (glossary)	31
Complex (glossary)	32
Complex Adaptive System (CAS) (glossary)	32
Complexity (glossary)	33
Complexity avoidance (glossary)	34
Component (glossary)	34
Computer Simulation (glossary)	35
Concept (glossary)	35
Concept Definition (glossary)	36
Concept of Operations (ConOps) (glossary)	36
Concrete Syntax (glossary)	37
Concurrent (glossary)	37
Concurrently (glossary)	38

Configuration (glossary)	38
Configuration Management (glossary)	39
Constraint (glossary)	39
Control (glossary)	40
Coordinate (glossary)	41
Cost (glossary)	41
Critical Design Review (CDR) (glossary)	41
Critical Systems Thinking (glossary)	42
Culture (glossary)	42
Customer (glossary)	43
Cyber-Physical Systems (glossary)	43
Cybernetics (glossary)	44
Cycle (glossary)	44

D	45
Data Center (glossary)	45
Decision Gate (glossary)	45
Decision Management (glossary)	46
Defense in depth (glossary)	46
Demonstration (glossary)	47
Derived Requirement (glossary)	47
Descriptive Model (glossary)	48
Design (glossary)	48
Design Factor (glossary)	49
Design Life (glossary)	49
Design Property (glossary)	50
DevOps (glossary)	50
Digital Elevation Model (glossary)	51
Digital Terrain Model (DTM) (glossary)	51
Digital Twin (glossary)	52
Disposal (glossary)	52
Disruption (glossary)	53
Diversity (glossary)	53
Domain (glossary)	54
Drift Correction (glossary)	54
Drift Correction (glossary)	55
Dualism (glossary)	55

E	56
E-Services (glossary)	56
Effectiveness (glossary)	56
Element (glossary)	57
Ellipsoid (glossary)	57
Ellipsoidal Height or Geodetic Height (glossary)	58
Emergence (glossary)	58
Emergent Behavior (glossary)	59
Emergent Property (glossary)	59
Enabling System (glossary)	60
Encapsulation (glossary)	60
Engineered System (glossary)	61
Engineering (glossary)	62
Engineering Change Management (glossary)	62
Engineering Change Proposal (ECP) (glossary)	63
Engineering Coordinate Reference System (glossary)	63
Enterprise (glossary)	64
Enterprise Architecture (glossary)	65
Enterprise System (glossary)	66
Enterprise Systems Engineering (ESE) (glossary)	66
Entropy (glossary)	67
Environment (glossary)	68
Equity (glossary)	68
Ethics (glossary)	69
Evolutionary (glossary)	70
Executable System Model (glossary)	70
Extended Enterprise (glossary)	71
F	72
Failure (glossary)	72
Failure Modes and Effects Criticality Analysis (glossary)	73
Feature (glossary)	73
Feature attribute (glossary)	74
Feature Catalog (glossary)	74
Feature catalogue (glossary)	75
Federation of Systems (FoS) (glossary)	75
Flexibility (glossary)	76

Form, Fit, Function, and Interface (F3I) (glossary)	76
Framework (glossary)	77
Function (glossary)	77
Functional Architecture (glossary)	78
Functional redundancy (glossary)	79
G	80
Gazetteer (glossary)	80
General System Theory (glossary)	80
Geodesic or Geodesic Line (glossary)	81
Geodesy (glossary)	81
Geodetic Coordinate System (glossary)	82
Geodetic Datum (glossary)	82
Geodetic Reference Frame (glossary)	83
Geographic coordinates (glossary)	83
Geographic Data (glossary)	84
Geographic Identifier (glossary)	84
Geographic Imagery (glossary)	85
Geographic Information (glossary)	85
Geographic Information System (glossary)	86
Geoid (glossary)	86
GEOINT (glossary)	87
Geopositioning (glossary)	87
Glossary:Principle	88
Governance (glossary)	88
Gravity-Related Height (glossary)	89
Green Engineering (glossary)	89
H	90
Habitability (glossary)	90
Hard System (glossary)	90
Hardware-in-the-Loop Testing (glossary)	91
Healthcare (glossary)	91
Healthcare Systems Engineering (glossary)	92
Height (glossary)	92
Heuristic (glossary)	93
Hierarchy (glossary)	93
High Reliability Organizations (HROs) (glossary)	94

Holism (glossary)	95
Holistic (glossary)	95
Homeostasis (glossary)	95
Human Factors (glossary)	96
Human in the loop (glossary)	96
Human Survivability (glossary)	97
Human Systems Integration (HSI) (glossary)	97
Hydrography (glossary)	98
I	99
Implementation (glossary)	99
In-Process Validation (glossary)	99
Inclusion (glossary)	100
Increment (glossary)	100
Incremental (glossary)	101
Industrial Engineering (glossary)	101
Information and Communications Technologies (ICT) (glossary)	102
Information Category (glossary)	102
Information Need (glossary)	103
Information Technology (glossary)	103
Infrastructure (glossary)	104
Input (glossary)	104
Input-Output Flow (glossary)	105
Installation (glossary)	105
Integrated Product Team (IPT) (glossary)	106
Integration (glossary)	106
Integrity (glossary)	107
Inter-Node Interaction (glossary)	107
Interface (glossary)	108
Internal Coordinate Reference System (glossary)	108
Internode Interaction (glossary)	109
Interoperability (glossary)	109
Isomorphy (glossary)	110
Issue (glossary)	111
Iteration (glossary)	111
K	112
Knowledge (glossary)	112

L	113
Land Cover (glossary)	113
Land Use (glossary)	113
Latent (glossary)	114
Layer (glossary)	114
Leader (glossary)	115
Lean Systems Engineering (LSE) (glossary)	115
Lean Value Stream Mapping (glossary)	116
Legacy System (glossary)	116
Leverage (glossary)	117
Life Cycle (glossary)	117
Life Cycle Cost (LCC) (glossary)	118
Life Cycle Management (glossary)	118
Life Cycle Model (glossary)	119
Life Cycle Process (glossary)	119
Localized Capacity (glossary)	120
Location (glossary)	120
Location-Based Service (LBS) (glossary)	121
Logical Architecture (glossary)	121
Logistics (glossary)	122
Loose Coupling (glossary)	122
Loss-Driven Systems Engineering (glossary)	123
M	124
Maintainability (glossary)	124
Maintenance (glossary)	124
Manpower (glossary)	125
Map (glossary)	125
Map Projection (glossary)	126
Market Analysis (glossary)	126
Mean Sea Level(glossary)	127
Measurable Concept (glossary)	127
Measure (glossary)	128
Measure of Effectiveness (MoE) (glossary)	129
Measure of Performance (MoP) (glossary)	129
Measurement (glossary)	130
Meta-model (glossary)	130

Metadata (glossary)	131
Metric (glossary)	131
Milestone (glossary)	132
Mission (glossary)	132
Mission Analysis (glossary)	133
Mission Engineering (glossary)	133
Model (glossary)	134
Model Management (glossary)	135
Model Transformation (glossary)	135
Model Validation (glossary)	136
Model-Based Systems Engineering (MBSE) (glossary)	136
Modeling Language (glossary)	137
Modeling Tool (glossary)	137
Modularity (glossary)	138
Motion (glossary)	139
N	140
Natural System (glossary)	140
Nautical Chart (glossary)	140
Navigation (glossary)	141
Network (glossary)	141
Neutral State (glossary)	142
Node (glossary)	142
Non-Functional Requirements (glossary)	143
O	144
Occupational Health (glossary)	144
Ontology (glossary)	144
Open System (glossary)	145
Operational (glossary)	145
Operational Capability (glossary)	146
Operational Concept (glossary)	146
Operational Environment (glossary)	147
Operational Mode (glossary)	147
Operational Scenario (glossary)	148
Operations Research (glossary)	148
Opportunity (glossary)	149
Organization (glossary)	150

Organizational (glossary)	150
Organizational Capability (glossary)	151
Orthoimage (glossary)	151
Outcome (glossary)	151
Output (glossary)	152

P 153

Paradigm (glossary)	153
Pattern (glossary)	153
Personnel (glossary)	154
Physical Architecture (glossary)	154
Physical Interface (glossary)	155
Physical Model (glossary)	155
Physical redundancy (glossary)	155
Plan (glossary)	156
Pluralist (glossary)	156
Portfolio (glossary)	157
Portfolio Management (glossary)	157
Portrayal (glossary)	158
Positional accuracy (glossary)	158
Postal address type (glossary)	158
Postmodernist (glossary)	159
Praxis (glossary)	159
Preliminary Design Review (PDR) (glossary)	160
Principle (glossary)	160
Problem (glossary)	161
Process (glossary)	161
Procurement (glossary)	162
Product (glossary)	162
Product System (glossary)	163
Program (glossary)	163
Program Management (glossary)	164
Project (glossary)	164
Project Management (glossary)	165
Prototype (glossary)	165
Proving (glossary)	166
Purpose (glossary)	166
Purposeful (glossary)	167

Purposive (glossary)	167
Q	168
Qualification (glossary)	168
Quality (glossary)	169
R	170
Rationale (glossary)	170
Recovery (glossary)	170
Recursion (glossary)	171
Reductionism (glossary)	171
Redundancy (glossary)	172
Regularity (glossary)	172
Reliability (glossary)	173
Remote sensing (glossary)	173
Reparability (glossary)	174
Requirement (glossary)	174
Resilience (glossary)	175
Resolution (of imagery) (glossary)	175
Restructuring (glossary)	176
Reverse Engineering (glossary)	176
Rhumb Line (glossary)	177
Risk (glossary)	177
Risk Management (glossary)	178
Robustness (glossary)	178
Routing (glossary)	179
S	180
Safety (glossary)	180
Satellite Positioning System (glossary)	180
Scalability (glossary)	181
Scenario (glossary)	181
Scope (glossary)	182
Security (glossary)	182
Semantic Interoperability (glossary)	183
Semantics (glossary)	183
Service (glossary)	184
Service Life Extension (SLE) (glossary)	185

Service Science (glossary)	186
Service System (glossary)	186
Service Systems Engineering (glossary)	187
Service Value Chain (glossary)	188
Set Driver (glossary)	188
Set Modifiers (glossary)	189
Set-Based Design (glossary)	189
Simulation (glossary)	190
Simulator (glossary)	190
Six Sigma (glossary)	191
Social System (glossary)	191
Sociotechnical System (glossary)	192
Soft System (glossary)	192
Soft Systems Methodology (glossary)	193
Software (glossary)	193
Software Assurance (glossary)	194
Software Engineering (glossary)	194
Software System (glossary)	195
Software-in-the-Loop Testing (glossary)	195
Solution (glossary)	196
Spatial Reference (glossary)	196
Spatial Reference System (glossary)	196
Spatial Reference (glossary)	197
Specialty Engineering (glossary)	197
Stage (glossary)	198
Stakeholder (glossary)	198
Stakeholder Needs and Requirements (glossary)	199
State (glossary)	200
Statistical Process Control (glossary)	200
Structure (glossary)	201
Supplier (glossary)	201
Survivability (glossary)	202
Sustainment (glossary)	202
Synectics (glossary)	203
Synergy (glossary)	203
Synthesis (glossary)	204
System (glossary)	204
System Analysis (glossary)	205

System Assurance (glossary)	206
System Boundary (glossary)	206
System Breakdown Structure (glossary)	207
System Capability (glossary)	207
System Context (glossary)	208
System Coupling Diagram (glossary)	208
System Definition (glossary)	209
System Deployment and Use (glossary)	210
System Element (glossary)	210
System Hardware Assurance	211
System of Systems (SoS) (glossary)	217
System Property (glossary)	218
System Realization (glossary)	218
System Requirement (glossary)	219
System-of-Interest (glossary)	219
Systemist (glossary)	220
Systems Approach (glossary)	220
Systems Biology (glossary)	221
Systems Concept (glossary)	221
Systems Development (glossary)	222
Systems Engineer (glossary)	222
Systems Engineering (glossary)	223
Systems Engineering Management (SEM) (glossary)	225
Systems Engineering Plan (SEP) (glossary)	225
Systems Integration (glossary)	226
Systems Science (glossary)	226
Systems Thinking (glossary)	227

T 228

Tailoring (glossary)	228
Task (glossary)	228
Team (glossary)	229
Technical Management (glossary)	229
Technical Performance Measure (TPM) (glossary)	230
Temporal Architecture (glossary)	230
Test Cases (glossary)	231
Threat (glossary)	231
Tolerance (glossary)	232

Total Ownership Cost (glossary)	232
Traceability (glossary)	233
Training (glossary)	233
Transition of Modes (glossary)	234
Transport network (glossary)	234
U	235
Uncertainty (glossary)	235
Unitary (glossary)	235
Use Case (glossary)	236
Useful life (glossary)	236
User (glossary)	237
Utility network (glossary)	237
V	238
Validation (glossary)	238
Value (glossary)	239
Variety (glossary)	240
Vee (V) Model (glossary)	240
Verification (glossary)	241
Verification and Validation Action (glossary)	242
Verification and Validation Configuration (glossary)	242
Verification and Validation Procedure (glossary)	243
Verification and Validation Tool (glossary)	243
Viable (glossary)	244
View (glossary)	244
Viewpoint (glossary)	245
Vignette (glossary)	245
W	246
White-Box System (glossary)	246
Wicked Problem (glossary)	246
Acronyms	247
Acronyms	247
References	
Article Sources and Contributors	261
Image Sources, Licenses and Contributors	272

Glossary

A

Absorption (glossary)

The ability of a system to withstand a disturbance without a fundamental breakdown in the system's performance or structure -- adapted from (Woods 2006)

Absorption is a resilience principle that supports the robustness attribute according to Jackson and Ferris (2013).

Sources

Woods, David D. 2006. "Essential Characteristics of Resilience." In E. Hollnagel, D. D. Woods, and N. Leveson. *"Resilience Engineering: Concepts and Precepts."* Aldershot, UK: Ashgate Publishing Limited.

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

This definition is in the context of resilience for which David Woods is an authority.

Abstract Model (glossary)

(1) A simplified description, especially a mathematical one, of a system or process, to assist calculations and predictions. (Pearsall 2012)

(2) An abstract or conceptual representation of a system that does not have a physical or concrete existence. (Created for SEBoK)

Sources

(1) Pearsall, Judy (editor). 2012. *Oxford Dictionaries Online*. Oxford, England, UK: Oxford University Press. Available at : <https://oxforddictionaries.com/definition/english/model> ^[1].

(2) This definition was developed for the SEBoK v. 1.0.

Discussion

An abstract model contrasts with a concrete physical model. It can be further classified as descriptive or analytical (See article Types of Models).

References

[1] <https://oxforddictionaries.com/definition/english/model>

Abstract Syntax (glossary)

- (1) *A form of representation of data that is independent of machine-oriented structures and encodings and also of the physical representation of the data.* (Dictionary.com 2012)
- (2) *The set of constructs and associated rules to create well-formed models.* (Created for SEBoK)

Sources

- (1) Dictionary.com. 2012. *The Free On-line Dictionary of Computing*. Denis Howe. <https://dictionary.reference.com/browse/abstract%20syntax> (accessed: September 07, 2012).
- (2) This definition was developed for the SEBoK v. 1.0.

Discussion

None.

Abstraction (glossary)

- (1) *A view of an object that focuses on the information relevant to a particular purpose and ignores the remainder of the information.* (ISO/IEC 2010)
- (2) *The process of formulating a view.* (ISO/IEC 2010)
- (3) "A [simplified] replica of the concrete. For example, the abstract architectural principle of redundancy can be realized in the concrete manifestation of a concrete communications system." (Lonergan, 1992)

Source

- (1)-(2) ISO/IEC. 2010. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2010. Available from https://pascal.computer.org/sev_display/index.action.
- (3) Lonergan, Bernard, "Insight" University of Toronto Press, Toronto, 1992, p. 111.

Discussion

None.

Acceptance Criteria (glossary)

The procurement specification, in the context of the overall agreement, should clearly state the criteria by which the acquirer will accept delivery from the supplier. A verification matrix can be used to clarify these criteria. (INCOSE 2011, Section 6.1.15, p 259)

Source

INCOSE. 2011. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2

Discussion

None.

Acceptance Sampling (glossary)

In acceptance sampling many examples of a product are presented for delivery. The consumer samples from the lot and each member of the sample is then either categorized as "acceptable" or "unacceptable" based on an attribute (attribute sampling) or measured against one or more metrics (variable sampling). Based on the measurements, an inference is made as to whether the lot meets the customer requirements. There are four possible outcomes of the sampling of a lot, as shown in Table 1.

Table 1. Truth Table - Outcomes of Acceptance Sampling. (SEBoK Original)

	Lot meets requirement	Lot fails requirement
Sample passes test	No error	Consumer risk
Sample fails test	Producer risk	No error

A sample acceptance plan balances the risk of error between the producer and consumer. Detailed ANSI/ISO/ASQ standards describe how this allocation is performed. (ANSI/ISO/ASQ 1993)

Sources

ANSI/ISO/ASQ. 1993. *Statistics—Vocabulary and Symbols—Statistical Quality Control*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/International Organization for Standardization(ISO)/American Society for Quality(ASQ) :A3534-2-1993.

Discussion

A company cannot test every one of its products due to either the need for destructive testing requirements, or the volume of products being too large. Acceptance sampling solves this by testing a sample of product for defects. The process involves batch size, sample size and the number of defects acceptable in the batch. This process allows a company to measure the quality of a batch with a specified degree of statistical certainty without having to test every unit of product. The statistical reliability of a sample is generally measured by a t-statistic.

Acquirer (glossary)

The stakeholder that acquires or procures a product or a service from a supplier. (ISO/IEC 2010)

Source

ISO/IEC. 2010. *Systems and Software Engineering -- Life Cycle Management -- Part 2: Guide for the Application of ISO/IEC 15288 (systems life cycle processes)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC DTR 24748-2.

Discussion

None.

Acquisition (glossary)

The process of obtaining a system, product or service. (ISO/IEC/IEEE 2015)

Sources

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Discussion

None.

Acquisition Strategy (glossary)

A comprehensive, integrated plan that identifies the acquisition approach and describes the business, technical, and support strategies that management will follow to manage program risks and meet program objectives. The acquisition strategy should define the relationship between the acquisition phases and work efforts, and key program events such as decision points, reviews, contract awards, test activities, production lot/delivery quantities, and operational deployment objectives. (DAU 2010)

Source

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD).

Discussion

None.

Activity (glossary)

According to ISO/IEC/IEEE 24774, activities are defined as “set of cohesive tasks of a process”.
(ISO/IEC/IEEE 2021)

Sources

ISO/IEC/IEEE. 2021. *Systems and software engineering — Life cycle management — Specification for process description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24774:2021, Clause 3.1.

Discussion

None.

Adaptability (glossary)

- (1) *An adaptive system is one that is able to change itself or its environment if its effectiveness is insufficient to achieve its current or future goals or objectives.* (Ackoff 1971)
- (2) *Attributes of software that bear on the opportunity for its adaptation to different specified environments without applying other actions or means than those provided for this purpose for the software considered.* (INCOSE 1998)
- (3) *The ability of a system to acclimate physically and functionally to a new operating environment with a minimal degree of degradation to capability performance.* (Wasson 2006)
- (4) *the ability of the system to become suitable for a new use or purpose and to what range or sources of variation.* According to Jackson (2016) adaptability is an attribute of a resilient system.

Sources

- (1) Ackoff, R.L. 1971. "Towards a System of Systems Concepts." *Management Science* 11: 11.
- (2) INCOSE. 1998. INCOSE SE Terms Glossary. In: *INCOSE Concepts and Terms*, Working Group (ed.). Seattle, WA, USA: International Council on Systems Engineering.
- (3) Wasson, Charles S. 2006. *System Analysis, Design, and Development*. Edited by A. P. Sage, Wiley Series in Systems Engineering and Management. Hoboken, NJ, USA: John Wiley & Sons.
- (4) Jackson, Scott. 2016. "Evaluation of resilience principles for engineered systems." PhD Research, Engineering, University of South Australia.

Discussion

- (1) A system science definition, which applies to any level of system
 - (2) The term in a software context where it is frequently used, the same idea could apply to human systems. The 1998 INCOSE SE Terms Glossary is an authoritative source.
 - (3) A more general definition, which applies to complex engineered systems. The book by Wasson is a standard systems engineering textbook.
-

Address (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Adversity (glossary)

A condition that may degrade the desired capability of a system. (Brtis 2016)

Source

Brtis, John. 2016. How to Think About Resilience in a DoD Context. Colorado Springs, CO: MITRE Corporation.

Discussion

In the context of resilience adversity can include an encounter with a threat or an internal disruption such as human error.

Aggregate (glossary)

A subset of a system made up of several system elements and physical interfaces (indiscriminately system elements or sub-systems) on which a set of verification and validation actions is applied.
(Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

None.

Agile (glossary)

(1) *Response ability state marked by high competence at both proactive and reactive change. (Dove 2001, 69)*

(2) *Project execution methods can be described on a continuum from “adaptive” to “predictive.” Agile methods exist on the “adaptive” side of this continuum, which is not the same as saying that agile methods are “unplanned” or “undisciplined.” (INCOSE 2011, 40, 183)*

Source

(1) Dove, R. 2001. *Response Ability: The Language, Structure, and Culture of the Agile Organization*. New York, NY, USA: John Wiley & Sons.

(2) INCOSE 2011. *INCOSE Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.2.

Discussion

None.

Agility (glossary)

Derivation of the physical ability to act (response ability) and the intellectual ability to find appropriate things to act on (knowledge management). (Dove 2001, p. 9)

Source

Dove, R. 2001. *Response Ability: The Language, Structure, and Culture of the Agile Organization*. New York, NY, USA: John Wiley & Sons.

Discussion

None.

Agreement (glossary)

Mutual acknowledgment of terms and conditions under which a working relationship is conducted.
(ISO/IEEE 2008, 1, Section 4.4)

Source

ISO/IEEE. 2008. *Systems and Software Engineering — Software Life Cycle Processes*. Geneva, Switzerland: International Organization for Standards (ISO)/Institute of Electrical & Electronics Engineers (IEEE) Computer Society, ISO/IEEE 12207:2008(E).

Discussion

None.

Analytical Model (glossary)

Mathematical model into which data are loaded for analysis. (Turban, et al 2010)

Source

Turban, E., R. Sharda, and D. Delen. 2010. *Decision Support and Business Intelligence Systems*, 9th ed. Prentice-Hall.

Discussion

In the context of systems engineering (SE), it may be considered that the model represents a System-of-Interest (SoI).

Antipattern (glossary)

(1) *An antipattern is just like a pattern except that instead of a solution it gives something that looks superficially like a solution but isn't one.* (Koenig 1995)

(2) *Pattern of failure.* (SEI 2012)

Sources

(1) Koenig, A. (March/April 1995). "Patterns and Antipatterns". *Journal of Object-Oriented Programming* 8, (1): 46–48.

(2) SEI 2012. *Patterns of Failure: System Archetypes*. Available at <https://www.sei.cmu.edu/acquisition/research/pofsa.cfm>.

Discussion

A full discussion of Antipatterns and how they relate to systems thinking can be found in *Patterns of Systems Thinking*

Architecting (glossary)

(1) *Process of conceiving, defining, expressing, documenting, communicating, certifying proper implementation of, maintaining and improving an architecture throughout a system's life cycle* (ISO/IEC/IEEE 2011)

(2) *The architecting process sometime involves the use of heuristics to establish the form of architectural options before quantitative analyses can be applied. Heuristics are design principles learned from experience.* (Rechtin 1990)

Source

(1) ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.

(2) Rechtin, E. 1990. *Systems Architecting: Creating & Building Complex Systems*. Upper Saddle River, NJ, USA: Prentice-Hall.

Discussion

None.

Architecture (glossary)

(1) *fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution* (ISO/IEC/IEEE 2015, Section 4.5)

(2) *The organizational structure of a system or component; the organizational structure of a system and its implementation guidelines.* (ISO/IEC 2009, 1)

(3) *Fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution.* (ISO/IEC 2011, section 3.2)

Source

(1) ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

(2) ISO/IEC/IEEE. 2009. *Systems and Software Engineering — System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronic Engineers (IEEE) 2009 ISO/IEC/IEEE 24765:2009 [database online]. Available from https://pascal.computer.org/sev_display/index.action.

(3) ISO/IEC/IEEE. 2011. *Systems and Software Engineering — Architectural Description*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 42010:2011.

Discussion

A few definitions are presented here to illustrate the different ways that authors define architecture. Note that many authors write extensively on architecture without ever defining what they mean by the term.

The use of the word *fundamental* (definitions (1) and (3)) is problematic, since it has no universal definition. In practice, the level of detail in an architecture depends on the context of use and the purpose to which it is being designed. In the early (conceptual) stages it might only contain a high-level description of the system as a whole, but in later stages the key features of all key subsystems need to be mapped out. An architectural description should therefore also justify what is included and what is excluded.

ISO/IEC/IEEE 15288:2015 is normative - see above. The architecture associated with a system-of-interest is conceptual and is realized through an architectural description.

ISO/IEC/IEEE 42010:2011 is normative - see above.

OMG 2010 is normative - “The organizational structure and associated behavior of a system. An architecture can be recursively decomposed into parts that interact through interfaces, relationships that connect parts, and constraints for assembling parts.”

Works Cited

OMG. 2010. *OMG Systems Modeling Language Specification*, version 1.2, July 2010. https://www.omg.org/technology/documents/spec_catalog.htm.

Architecture Framework (glossary)

Conventions, principles and practices for the description of architectures established within a specific domain of application and/or community of stakeholders. (ISO/IEC/IEEE 2011)

Source

ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 42010:2011. Available at <https://www.iso.org/standard/50508.html> ^[1].

Discussion

Examples of architecture frameworks include:

DoDAF: (DOD. 2010. *DoDAF Architecture Framework. Version 2.02*. Arlington, VA, USA: US Department of Defense. Available at: <https://cio-nii.defense.gov/sites/dodaf20/>)

MoDAF: (MOD. 2010. *MOD Architecture Framework. Version 1.2.004*. UK Ministry of Defence. Available at: <https://www.mod.uk/DefenceInternet/AboutDefence/WhatWeDo/InformationManagement/MODAF/>.)

TOGAF: (The Open Group. 2011. *TOGAF*, version 9.1. Hogeweg, The Netherlands: Van Haren Publishing. Accessed August 29, 2012. Available at: <https://www.opengroup.org/ogsys/jsp/publications/PublicationDetails.jsp?catalogno=g116>.)

TRAK: TRAK Enterprise Architecture Framework. Available at: <https://sourceforge.net/projects/trak/>. Recognised by an INCOSE team award (<https://www.incose.org/about-incose/incose-recognition/working-group-awards#2011>)

Zachamn: (Zachman, J. 2008. "John Zachman's Concise Definition of The Zachman Framework™." Zachman International Enterprise Architecture. Accessed August 29, 2012. Available at: <https://www.zachman.com/about-the-zachman-framework>.)

See the section 'Enterprise Architecture Frameworks & Methodologies' in Enterprise Systems Engineering Key Concepts

References

[1] <https://www.iso.org/standard/50508.html>

Assembly Procedure (glossary)

A set of elementary assembly actions to build an aggregate of implemented system elements. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

This definition is most relevant to the usage of this term in the SEBoK (see System Integration article).

Assembly Tool (glossary)

An assembly tool is a physical tool used to connect, assemble or link several implemented system elements and to build aggregates (specific tool, harness, etc.). (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

This definition is most relevant to the usage of this term in the SEBoK (see System Integration article).

Assessment Criterion (glossary)

A characteristic used to assess or compare system elements, physical interfaces, physical architectures, logical architectures, or any such engineering element. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

This definition is most relevant to the usage of this term in the SEBoK in the context of System Analysis. Also see the Analysis and Selection between Alternative Solutions article.

Assessment Score (glossary)

A score that is obtained from assessing a component, a link, a physical architecture, or a functional architecture using a set of assessment criteria. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

This definition is most relevant to the usage of this term in the SEBoK in the context of System Analysis. Also see the Analysis and Selection between Alternative Solutions article.

Attribute (glossary)

An inherent property or characteristic of an entity that can be distinguished quantitatively or qualitatively by human or automated means. (ISO/IEC/IEEE 2007)

Sources

ISO/IEC/IEEE. 2007. *Systems and software engineering - Measurement process*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC/IEEE 15939:2007.

Discussion

There are 8 definitions of attribute in the standards covered by the IEEE *Systems and Software Vocabulary Project* (sevocab). The included definition is the second, but best seems to capture the intended use of the term in SEBoK. Many uses of the term *attribute* are specific to database theory and formal modeling. See www.computer.org/sevocab for more detail.

SEVOCAB^[1] contains the following note:

[An] attribute can refer either to general characteristics such as reliability, maintainability, and usability or to specific features of a software product. ISO 9000 distinguishes two types of attributes: a permanent characteristic existing inherently in something; and an assigned characteristic of a product, process or system (e.g. the price of a product, the owner of a product). The assigned characteristic is not an inherent quality characteristic of that product, process or system. An attribute expresses some characteristic that is generally common to the instances of a class. The name of the attribute is the name of the role that the value class plays in describing the class, which may simply be the name of the value class (as long as using the value class name does not cause ambiguity).

References

[1] <https://www.computer.org/sevocab>

Availability (glossary)

Availability is the probability that a repairable system or system element is operational at a given point in time, under a given set of environmental conditions.

Source

Adapted from the following sources:

ASQ. 2011. "Glossary: Availability." American Society for Quality. Available at <https://asq.org/glossary/a.html>.

DoD. 2010. "Acquipedia: Material Availability." Department of Defense. Available at <https://acc.dau.mil/CommunityBrowser.aspx?id=387703>.

ReliaSoft Corporation. 2003. "Availability and the Different Ways to Calculate it." Available at <https://www.weibull.com/hotwire/issue79/relbasics79.htm>.

Discussion

The definition here is similar to the one given in (ASQ 2011) and in the Department of Defense. Availability has some additional definitions, characterizing what downtime is counted against a system. For **inherent availability**, only downtime associated with corrective maintenance counts against the system. For **achieved availability**, downtime associated with both corrective and preventive maintenance counts against a system. Finally, **operational availability** counts all sources of downtime, including logistical and administrative, against a system.

Availability can also be calculated instantaneously, averaged over an interval, or reported as an asymptotic value. **Asymptotic availability** can be calculated easily from the mean time to failure and the mean time to repair, but care must be taken to analyze whether or not a system settles down or settles up to the asymptotic value, and how long it takes until the system approaches that asymptotic value.

B

Baseline (glossary)

A specification or product that has been formally reviewed and agreed upon, that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures. (ISO/IEEE 2008)

Sources

ISO/IEEE. 2008. *Systems and Software Engineering — Software Life Cycle Processes*. Geneva, Switzerland: International Organization for Standards (ISO)/Institute of Electrical & Electronics Engineers (IEEE) Computer Society, ISO/IEEE 12207:2008(E).

Discussion

There are 7 definitions of baseline in the standards covered by the IEEE sevocab database. The included definition seems most appropriate for SEBoK use. See <https://www.computer.org/sevocab> for more detail.

The SEVOCAB ^[1] entry provides 7 definitions followed by this note: *A baseline should be changed only through formal configuration management procedures. Some baselines may be project deliverables while others provide the basis for further work. Baselines, plus approved changes from those baselines, constitute the current configuration identification.*

References

[1] https://pascal.computer.org/sev_display/index.action

Behavior (glossary)

(1) *Systems behavior is a change which leads to events in itself or other systems. Thus, action, reaction or response may constitute behavior in some cases. (Ackoff 1971)*

(2) *The effect produced when an instance of a complex system or organism is used in its operational environment. (Created for SEBoK)*

Source

(1) Ackoff, R.L. 1971. "Towards a System of Systems Concepts". *Management Science* 17: 11. Hanover, MD, USA: INFORMS.

(2) This definition was developed for the SEBoK.

Discussion

(1) This is the system science definition. Any system has behavior if its actions are in some way visible to systems around it.

(2) This definition associates behavior with an emergent outcome of (complex) deployed system, more analogous to human/animal behavior. Taking this view, the whole organism has behavior but not any of its element systems; e.g., cars have behavior (when driven by people), engines have functions.

Behavioral Architecture (glossary)

(1) *An arrangement of functions and their sub-functions and interfaces (internal and external) which defines the execution sequencing, conditions for control or data-flow and the performance requirements to satisfy the requirements baseline. (ISO/IEC 2010)*

(2) *A set of inter-related scenarios. (Created for SEBoK)*

Sources

(1) ISO/IEC. 2010. Systems and Software Engineering, Part 1: Guide for Life Cycle Management. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 24748-1:2010.

(2) This definition was developed for the SEBoK.

Discussion

Within the terms and definitions related to System Architecture, the present SEBoK tries to fit the real practices and to provide some consistency between those terms.

Definition (1) comes from ISO/IEC/IEEE 24748 - 4 (past IEEE 1220, ISO/IEC 26702) as functional architecture; but modified because in the standard functional and behavioral aspects are mixed. In reality the functional architecture emphasizes more on transformations performed rather than the sequencing of their executions. See definition of functional architecture (glossary).

Definition (2) is a good suggestion to represent a behavioral architecture, because a scenario of functions chains the execution of functions taking into account synchronization between functions and arrival of triggers.

Binding (glossary)

- (1) *The point in time when relationships are established.* (Created for SEBoK)
- (2) *Static binding is done **once** and then the relationship remains unchanged.* (Created for SEBoK)
- (3) *Dynamic binding is established at the point of a request and disappears after the request has been satisfied.* (Created for SEBoK)

Source

These definitions were developed for SEBoK v. 1.0.

Discussion

This definition is most relevant to the usage of this term in the SEBoK (See article Applications of Systems Engineering).

Black-Box System (glossary)

A device, system or object which can be viewed solely in terms of its input, output and transfer characteristics without any knowledge of its internal workings, that is, its implementation is "opaque" (black). (Ashby 1956)

Sources

Ashby, W R. 1956. *Introduction to Cybernetics*. "Chapter 11" London, UK: John Wiley & Sons.

Discussion

None.

Body of Knowledge (glossary)

(1) *The purpose of the Guide to the Software Engineering Body of Knowledge is to provide a consensually validated characterization of the bounds of the software engineering discipline and to provide a topical access to the Body of Knowledge supporting that discipline. (Abran and Moore 2004)*

(2) *Identifies that subset of the project management body of knowledge generally recognized as good practice. "Generally recognized" means the knowledge and practices described are applicable to most projects most of the time, and there is consensus about their value and usefulness. "Good practice" means there is general agreement that the application of these skills, tools, and techniques can enhance the chances of success over a wide range of projects...The PMBOK® Guide also provides and promotes a common vocabulary within the project management profession...The Project Management Institute views this standard as a foundational project management reference for its professional development programs and certifications. (PMI 2013)*

(3) *Document explicitly intended to capture the knowledge for entry into the practice of [an] engineering [discipline] at the professional level. (ASCE 2008)*

Source

(1) Abran, A. and J.W. Moore (exec. eds); P. Bourque and R. Dupuis (eds.). 2004. *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*. Piscataway, NJ, USA: The Institute of Electrical and Electronic Engineers, Inc. (IEEE). Available at: <https://www.computer.org/portal/web/swebok>.

(2) PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

(3) ASCE. 2008. *Civil Engineering Body of Knowledge for the 21st Century: Preparing the Civil Engineer for the Future*, 2nd ed. Reston, VA, USA: American Society of Civil Engineers.

Discussion

None.

Boundary (glossary)

Please see system boundary.

Business (glossary)

A business is an organization engaged in the trade of goods, services, or both to consumers. (Sullivan and Sheffrin 2003)

Source

Sullivan, A. and S.M. Sheffrin. 2003. *Economics: Principles in Action*. Upper Saddle River, NJ, USA: Pearson Prentice Hall.

Discussion

A business may be a commercial company, a not-for-profit organization, or state-owned. Its consumers may be individuals, projects, other businesses, or enterprises.

There are many different legal forms of businesses, which vary across political entities. In the context of the SEBoK, the term **business** would include a government agency.

The word **enterprise** is sometimes used synonymously with "business". However in the SEBoK, a **business** is a specific type of enterprise, usually a legal entity with a form of centralized management. An enterprise is a purposeful activity often involving collaboration across organizational boundaries.

Business Process (glossary)

An inter-related set of cross-functional activities or events that result in the delivery of a specific product or service to a customer. (ISACA 2012)

Source(s)

ISACA. 2012. "Glossary of Terms: Business Process." Information Systems Audit and Control Association. Accessed July 23, 2012. Available at: <https://www.isaca.org/Knowledge-Center/Documents/Glossary/glossary.pdf>.

Discussion

None.

Business Process Outsourcing (glossary)

The transfer of internal business processes, such as customer relationship management, finance & accounting, human resources and procurement, to an external service provider. (Gewald and Rouse 2012)

Source(s)

Gewald, H. and A. Rouse. 2012. "Comparing Business Process and IT Outsourcing Risks--An Exploratory Study in Germany and Australasia," 45th Hawaii International Conference on System Science (HICSS), 4-7 Jan. 2012, 275-284. Available at: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6148641&isnumber=6148595>.

Discussion

The term business process outsourcing is often confounded with "information technology (IT) outsourcing", they are related but not synonymous. IT oriented companies and consultancies often do so. Gartner's definition is a good example: "the delegation of one or more IT-intensive business processes to an external provider that, in turn, owns, administrates and manages the selected processes based on defined and measurable performance metrics." [1]. Any business process may be outsourced.

References

[1] <https://www.gartner.com/it-glossary/business-process-outsourcing-bpo/>

C

Call Center (glossary)

Telephone service facility set up to handle a large number of (usually) both inbound and outbound calls. Some firms, however, specialize only in calls that are inbound (for assistance, help, or ordering) or outbound (for sales promotion or other messages). Most telephone orders are handled by call centers and not by the manufacturers or suppliers of goods or services. (BusinessDictionary.com)

Sources

BusinessDictionary.com, s.v. "Call Center," <https://www.businessdictionary.com/definition/call-center.html> (accessed July 25, 2012).

Discussion

None.

Capability (glossary)

- (1) *The ability to achieve a desired effect under specified (performance) standards and conditions through combinations of ways and means (activities and resources) to perform a set of activities. (DoD 2009)*
- (2) *The ability to execute a specified course of action. It is defined by a user and expressed in non-equipment based operational terms. (MOD 2004)*
- (3) *The ability to execute a specified course of action. A capability may or may not be accompanied by an intention. (DoD 2009)*
- (4) *An outcome or effect which can be achieved through use of features of a system of interest and which contributes to a desired benefit or goal (Created for SEBoK)*

Source

- (1) DoD. 2009. *DoD Architecture Framework (DoDAF)*, version 2.0. Washington, DC, USA: U.S. Department of Defense (DoD).
 - (2) MOD. 2004. *Ministry of Defence Architecture Framework (MODAF)*, version 2. London, UK: U.K. Ministry of Defence.
 - (3) DoD. 2009. *Department of Defense Dictionary of Military and Associated Terms*. Washington, DC, USA: U.S. Department of Defense (DoD), Joint Publication 1-02, 17 March 2009.
 - (4) This definition was developed for the SEBoK.
-

Discussion

Capability provides an umbrella term for the ability to achieve many objectives and effects that may be specified during the SE lifecycle: missions, mission objectives, user needs, user requirements, system

In the defense context capability refers to an operational outcome available to the end user when engineered systems are deployed and fully supported (including trained people, logistics, doctrine, infrastructure, etc.) in an operational environment. It is used either to specify a required capability need (i.e., what is sought) or to describe the currently fielded capability (i.e., the currently available effect) to help identify gaps. The term capability has been defined to encourage military user to describe current and future needs independent of current solution technology

The term capability is less common outside of defense but is sometimes used to describe outcomes a user needs to achieve which connect the systems feature to the business or enterprise benefit. You can say that they lead your customer into understanding how the features you have deliver the benefit.

For example, a fingerprint scanner on a desktop computer can store passwords. This saves five minutes every time the user forgets a password and has to look it up or reset it, it also makes the computer more secure. The fingerprint scanner is the Feature, storing passwords is the Capability and saving time and increasing security the Benefit. This is likely to become a more common terminology as Systems Engineering looks to relate stakeholder needs more explicitly with business benefits, see Stakeholder Needs Definition

Capability Management (glossary)

(1) *Development and maintenance of all aspects of the ability to conduct certain types of missions in a given environment.* (Created for SEBoK)

(2) *The planning, organization, assessment and control of capabilities.* (Created for SEBoK)

Source

(1)-(2) These definitions were developed for SEBoK v. 1.0.

Discussion

None.

Capacity (glossary)

An attribute of resilience that includes all capabilities to enable a system to withstand a disruption – adapted from (Jackson 2010)

Sources

Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Edited by A. P. Sage, Wiley Series in Systems Engineering and Management. Hoboken, NJ, USA: John Wiley & Sons.

Discussion

This term is an accumulation of terms defined by Woods (2006).

Work Cited

Woods, David D. 2006. "Essential Characteristics of Resilience." In *Resilience Engineering: Concepts and Precepts*. Edited by E. Hollnagel, D. D. Woods, and N. Leveson. Aldershot, UK: Ashgate Publishing Limited.

Cartesian coordinate system (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Cartography (glossary)

the science or art of making or drawing maps

Source

Cambridge Online Dictionary. Accessed on May 11, 2022. Available: [https:// dictionary. cambridge. org/ us/ dictionary/english/cartography](https://dictionary.cambridge.org/us/dictionary/english/cartography)

Annotation

None.

Case Study (glossary)

An empirical inquiry that investigates the application of systems engineering practices, principles, and concepts that are based on real-life context. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

This definition describes the approach for the Case Studies included in Part 7: Systems Engineering Implementation Examples.

Catalogue (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Chaos (glossary)

(1) A state of disorder or unpredictability. (Oxford English Dictionary)

(2) A chaotic system has elements which are not interconnected and behave randomly with no adaptation or control. (Oxford English Dictionary)

(3) Chaos theory is applied to certain types of dynamic system (e.g. the weather) which, although they have structure and relationships, exhibit un-predictable behavior. These systems are deterministic; their future behavior is fully determined by their initial conditions with no random elements involved. However, their structure is such that (un-measurably) small perturbations in inputs or environmental conditions may result in unpredictable changes in behavior. This behavior is known as deterministic chaos, or simply chaos. (Kellert, 1993)

Sources

(1) and (2) *Oxford English Dictionary*. s.v. "Chaos."

(3) Kellert, S. 1993. *In the Wake of Chaos: Unpredictable Order in Dynamical Systems*. Chicago, IL, USA: University of Chicago Press. p. 32. ISBN 0-226-42976-8.

Discussion

(1) and (2) are the general usage definitions in which chaos means random and unconnected, e.g. not a system.

(3) is a mathematical definitions of a class of natural systems which appear chaotic, but have underlying mathematical order them.

Closed System (glossary)

A system which has no interactions with its environment. (von Bertalanffy 1968)

Sources

von Bertalanffy, L. 1968. *General system theory: Foundations, Development, Applications*. Revised ed. New York, NY, USA: George Braziller.

Discussion

None.

Coercive (glossary)

A problem situation in which the participants "have few interests in common and, if free to express them, would hold conflicting values and beliefs. Compromise is not possible and so no agreed objectives direct action. Decisions are taken on the basis of who has most power and various forms of coercion employed to ensure adherence to commands." (Jackson 2003, p. 19)

Sources

Jackson, M. 2003. *Systems Thinking: Creating Holisms for Managers*. Chichester, UK: John Wiley & Sons.

Discussion

None.

Cognitive Bias (glossary)

A systematic pattern of deviation from norm or rationality in judgment. (Haselton et al. 2005)

Source

Haselton, M.G., D. Nettle, and P.W. Andrews. 2005. "The evolution of cognitive bias." In D.M. Buss (Ed.). *The Handbook of Evolutionary Psychology*. Hoboken, NJ, US: John Wiley & Sons Inc. pp. 724–746.

Discussion

None.

SEBOK v. 2.2, released 15 June 2020

Cohesion (glossary)

(1) *The attribute of a system that allows it to operate before, during, and after an encounter with a threat.* (Hitchins 2009)

(2) *The manner and degree to which the tasks performed by a single software module are related to one another.* (ISO/IEC/IEEE 2010)

(3) *The act or state of cohering, uniting, or sticking together.* (Dictionary.com 2012)

Sources

(1) Hitchins, D. 2009. "What Are The General Principles Applicable to Systems?" *INCOSE Insight* 12 (4) (Dec 2009): 59-63.

(2) ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

(3) Dictionary.com. 2012. "Cohesion." Available at <https://dictionary.reference.com/browse/cohesion> ^[1].

Discussion

In systems engineering, cohesion refers both to the property of natural and social systems that similar elements are attracted to each other and to ability to operate in a threat environment. The context determines which definition is appropriate.

References

[1] <https://dictionary.reference.com/browse/cohesion>

Competency (glossary)

An observable, measurable set of skills, knowledge, abilities, behaviors, and other characteristics an individual needs to successfully perform work roles or occupational functions. Competencies are typically required at different levels of proficiency depending on the specific work role or occupational function. Competencies can help ensure individual and team performance aligns with the organization's mission and strategic direction. (OPM 2014)

a measure of an individual's ability in terms of their knowledge, skills, and behavior to perform a given role (Holt and Perry, 2011)

Source

U.S. Office of Personnel Management (OPM), Human Capital Assessment and Accountability Framework (HCAAF) Resource Center, "Glossary". https://www.opm.gov/hcaaf_resource_center/glossary.asp ^[1] Last accessed June, 2015.

Holt, Jon, and Perry, Simon, *A Pragmatic Guide to Competency, Tools, Frameworks, and Assessment*. BCS, The Chartered Institute for IT, Swindon, UK, 2011.

Discussion

There is disagreement in the literature on whether competency is only for individuals or if the term competency can be used at the team, project, and enterprise level.

References

[1] https://www.opm.gov/hcaaf_resource_center/glossary.asp

Complex (glossary)

(1) *An adjective describing a system's design or code that is difficult to understand because of numerous components or relationships among components (ISO/IEC/IEEE 2010).*

(2) *An adjective describing a system or component that has a design or implementation that is difficult to understand and verify (ISO/IEC/IEEE 2010).*

Sources

ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

Discussion

Modified from the definition of *complexity* in the SEVOCAB.

Complex Adaptive System (CAS) (glossary)

System where the individual elements act independently but jointly behave according to common constraints and goals. In the natural world, a flock of geese is a Complex Adaptive System (CAS). Human-intensive systems are also CASs since each human in the system is independent. (Weaver 1948, 536; Jackson, Hitchens, and Eisner 2010, 41; Flood and Carson 1993; Lawson 2010)

Sources

Jackson, S., D. Hitchens, and H. Eisner. 2010. "What is the systems approach?" *INCOSE Insight* 13:(1) (March 2010): 41.

Flood, R.L., and E.R. Carson. 1993. *Dealing with complexity: An Introduction to the Theory and Application of Systems Science*. 2nd ed. New York, NY, USA: Plenum Press.

Lawson, H.W. 2010. *A Journey through the Systems Landscape*. London, UK: College Publications, Kings College.

Discussion

None.

Complexity (glossary)

(1) A measure of how difficult it is to understand how a system will behave or to predict the consequences of changing it (Sheard and Mostashari 2009)

(2) The degree to which a system's design or code is difficult to understand because of numerous components or relationships among components (ISO/IEC 2009)

(3) A complex system has elements, the relationship between the states of which are weaved together so that they are not fully comprehended, leading to insufficient certainty between cause and effect (INCOSE 2021)

Source

(1) Sheard, S.A. and A. Mostashari. 2009. "Principles of Complex Systems for Systems Engineering". *Systems Engineering*, 12(4): 295-311.

(2) ISO/IEC. 2009. "Systems and Software Engineering Vocabulary (SEVocab)" - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.

(3) A complexity Primer for systems Engineers, Revision 1, 2021.

Discussion

Emergence and Complexity is a common property of engineered systems and occurs when there is no simple relationship between what an individual element does and what the system as a whole will do, and when the system includes some element of adaptation or problem solving to achieve its goals in different situations. It can be affected by objective attributes of a system such as by the number, types of and diversity of system elements and relationships, or by the subjective perceptions of system observers due to their experience, knowledge, training, or other socio-political considerations.

For a more complete discussion of complexity see the Emergence and Complexity article in Part 2.

Complexity avoidance (glossary)

a system resilience principle which states that a system should not be more complex than is necessary.
Jackson (2016)

Source

Jackson, Scott. 2016. "Principles for Resilient Design - A Guide for Understanding and Implementation." In IRGC Resource Guide on Resilience, edited by I. Linkov. University of Lausanne, Switzerland: International Risk Governance Council (IRGC).

Discussion

Also called reduce complexity

Component (glossary)

(1) *an entity with discrete structure, such as an assembly or software module, within a system considered at a particular level of analysis.* (ISO/IEC 1998)

(2) *One of the parts that make up a system.* (IEEE 2008)

(3) *A set of functional services in the software, which, when implemented, represents a well-defined set of functions and is distinguishable by a unique name.* (ISO/IEC 2008)

Source

(1) ISO/IEC. 1998. *Information Technology — System and Software Integrity Levels* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC. 15026:1998. : 3.1

(2) IEEE. 2008. "IEEE Standard for Software and System Test Documentation". Institute of Electrical and Electronics Engineers (IEEE) Standards Association: IEEE 829-2008: 3.1.6

(3) ISO/IEC. 2008. "Information Technology — Software and Systems Engineering — FiSMA 1.1 Functional Size Measurement Method" Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 29881:2008. A.4.

Discussion

In systems terms, we use component as the generic term for the level of decomposition at which system elements are no longer considered complex, and for which specialist design disciplines can be used.

Computer Simulation (glossary)

A dynamic representation of a model, often involving some combination of executing code, control/display interface hardware, and interfaces to real-world equipment. (DoD 1998)

Sources

DoD. 1998. "DoD Modeling and Simulation (M&S) Glossary" in *DoD Manual 5000.59-M*. Arlington, VA, USA: US Department of Defense. January. P2.3.25. Available at <https://www.dtic.mil/whs/directives/corres/pdf/500059m.pdf>

Discussion

None

Concept (glossary)

An abstraction; a general idea inferred or derived from specific instances. (Oxford Dictionaries Online 2012)

Sources

Oxford Dictionaries Online S.v. "Concept" Accessed February 20, 2012. <https://oxforddictionaries.com/definition/concept>.

Discussion

A concept is an abstraction; a general idea inferred or derived from specific instances. For example, by viewing a pet dog, one can infer that there are other dogs of that "type." Hence, from this observation (or perhaps a set of observations) the concept of a dog is developed in one's mind. Concepts are bearers of meaning, as opposed to agents of meaning and can only be thought about, or designated, by means of a name.

Principles (glossary) depend on concepts in order to state a "truth." Hence, principles and concepts go hand in hand; principles cannot exist without concepts and concepts are not very useful without principles to help guide the proper way to act (Lawson and Martin 2008).

Work Cited

Lawson, H., and J.N. Martin. 2008. "On the Use of Concepts and Principles for Improving Systems Engineering Practice". Proceedings of the 18th Annual International Council on Systems Engineering (INCOSE) International Symposium, 5-19 June 2008, Utrecht, The Netherlands.

Concept Definition (glossary)

A set of core technical activities of systems engineering in which the problem space and the needs of the stakeholders are closely examined. This consists of analysis of the problem space and definition of stakeholder needs for required services within it. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

System Concept Definition is a knowledge area in the SEBoK; it includes systems engineering activities which occur before any formal definition of the system-of-interest (SoI) is developed such as analysis and definition of stakeholder needs and requirements. This generally defines the problem statement which initiates a "concept stage" as discussed in INCOSE (2012). In the SEBoK, System Definition is used to discuss the detailed description of a solution system.

Work Cited

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Concept of Operations (ConOps) (glossary)

Verbal and graphic statement, in broad outline, of an organization's assumptions or intent in regard to an operation or series of operations. (ISO/IEC 2011)

Source

ISO/IEC 2011. *Systems and software engineering - Life cycle processes - Requirements engineering*. Geneva, Switzerland: International Organization for Standardization/International Electrotechnical Commissions. ISO/IEC 29148.

Discussion

The concept of operations frequently is embodied in long-range strategic plans and annual operational plans. In the latter case, the concept of operations in the plan covers a series of connected operations to be carried out simultaneously or in succession. The concept is designed to give an overall picture of the organization operations.

It provides the basis for bounding the operating space, system capabilities, interfaces and operating environment.

See also operational concept which is slightly different.

Concrete Syntax (glossary)

The symbols used to express the constructs of a language. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

There is a reference for "Concrete Syntax" in the computing domain:

"The syntax of a language including all the features visible in the source code such as parentheses and delimiters. The concrete syntax is used when parsing the program or other input, during which it is usually converted into some kind of abstract syntax tree (conforming to an abstract syntax).

In communications, concrete syntax is called transfer syntax."

The Free On-line Dictionary of Computing. Denis Howe. <https://foldoc.org/>(accessed: August 28, 2012).

However, this definition is not directly appropriate as the used in the SEBoK. Consequently, a definition at a higher level of abstraction than the term as used in the computing domain was created.

Concurrent (glossary)

(1) Happening at the same time as something else; operating or acting in conjunction with another. (Oxford English Dictionary Online 2012)

(2) Meeting or tending to meet at the same point; being in accordance; harmonious. (Ma, et al 2008)

(3) Concurrent engineering is a work methodology based on the parallelization of tasks (i.e. performing tasks concurrently). It refers to an approach used in product development in which functions of design engineering, manufacturing engineering and other functions are integrated to reduce the elapsed time required to bring a new product to the market.

(4) A concurrent life cycle approach applies life cycle activities in parallel to ensure the necessary relationships between them are considered within the life cycle. (Created for SEBoK)

Sources

(1) and (2) Oxford English Dictionary. s.v. "Concurrent."

(3) Ma, Y., G. Chen, & G. Thimm. 2008. "Paradigm Shift: Unified and Associative Feature-based Concurrent Engineering and Collaborative Engineering." *Journal of Intelligent Manufacturing*. DOI 10.1007/s10845-008-0128-y, Springer Science+Business Media

(4) This definition was developed for the SEBoK.

Discussion

(1) and (2) are dictionary definitions encapsulating the key ideas of things down at the same time, but within some common cause and joint end result.

(3) comes from the production world, where the idea of concurrency is used to both shorten delivery time and improve quality by putting activities in parallel which need to work closely together.

(4) is the more general lifecycle concept building on these ideas.

Concurrently (glossary)

(1) *Two or more activities occurring within the same interval of time, achieved either by interleaving the activities or by simultaneous execution. (ISO/IEC/IEEE 2010)*

(2) *A problem, process, system, or application in which many activities are happening in parallel, the order of incoming events is not usually predictable, and events often overlap. (ISO/IEC/IEEE 2010)*

Sources

(1) and (2) ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

Discussion

Modified from the definition of 'concurrent'.

Configuration (glossary)

The functional and physical characteristics of hardware or software as set forth in technical documentation or achieved in a product (ISO 24765, 2010)

Source

ISO/IEC/IEEE. 2010. *Systems and Software Engineering -- Vocabulary (SEVocab)* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronic Engineers (IEEE) 2009 ISO/IEC/IEEE 24765:2010 [database online]. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Configuration Management (glossary)

A discipline applying technical and administrative direction and surveillance to: identify and document the functional and physical characteristics of a configuration item, control changes to those characteristics, record and report change processing and implementation status, and verify compliance with specified requirements. (ISO/IEC/IEEE 2010)

Source

ISO/IEC/IEEE. 2010. *Systems and Software Engineering -- Vocabulary (SEVocab)* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronic Engineers (IEEE) 2009 ISO/IEC/IEEE 24765:2010 [database online]. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Constraint (glossary)

(1) A restriction, limit, or regulation imposed on a product, project, or process. (ANSI/EIA 1998)

(2) A type of requirement or design feature that cannot be traded off. (ANSI/EIA 1998)

Sources

(1) and (2) ANSI/EIA. 1998. *ANSI/EIA 632|Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA), ANSI/EIA 632-1998.

Discussion

None.

Control (glossary)

- (1) *In engineering, the monitoring of system output to compare with expected output and taking corrective action when the actual output does not match the expected output. (ISO/IEC 2010)*
- (2) *In engineering, the monitoring of system output to compare with expected output and taking corrective action when the actual output does not match the expected output. (ISO/IEC 2010)*
- (3) *Comparing actual performance with planned performance, analyzing variances, assessing trends to effect process improvements, evaluating possible alternatives, and recommending appropriate corrective action as needed. (PMI 2008)*
- (4) *In an IDEF0 model, a condition or set of conditions required for a function to produce correct output. (IEEE 1998)*
- (5) *In risk management, the formal name for the mitigation risk handling option. (DAU 2003a)*

Source

- (1) and (2) ISO/IEC/IEEE. 2010. *Systems and Software Engineering -- Vocabulary (SEVocab)* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronic Engineers (IEEE) 2009 ISO/IEC/IEEE 24765:2010 [database online]. Available from https://pascal.computer.org/sev_display/index.action.
- (3) PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 4th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- (4) IEEE. 1998. *IEEE Standard for Conceptual Modeling Language Syntax and Semantics for IDEF1X97 (IDEFobject)*. Institute for Electrical and Electronic Engineers (IEEE). IEEE 1320.2-1998 (R2004)
- (5) DAU. 2003. *Risk Management Guide for DoD Acquisition*. Fifth Edition. Version 2. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU) Press.

Discussion

None.

Coordinate (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Cost (glossary)

In the context of systems engineering, a cost is an amount expressed in a given currency related to the value of a system element, a physical interface, a physical architecture. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

None.

Critical Design Review (CDR) (glossary)

A review conducted to verify that the detailed design of one or more configuration items satisfy specified requirements; to establish the compatibility among the configuration items and other items of equipment, facilities, software, and personnel; to assess risk areas for each configuration item; and, as applicable, to assess the results of producibility analyses, review preliminary hardware product specifications, evaluate preliminary test planning, and evaluate the adequacy of preliminary operation and support documents. (ISO/IEC 2010)

Source

ISO/IEC. 2010. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2010. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Critical Systems Thinking (glossary)

Critical systems thinking (CST) or critical management science provides an approach to help select between hard system and soft system methods. (Jackson, 1985)

Sources

Jackson, M. 1985. "Social Systems Theory and Practice: The Need for a Critical Approach." *International Journal of General Systems* 10: 135-151.

Discussion

None.

Culture (glossary)

The basic values, norms, beliefs, and practices that characterize the functioning of a particular institution. (NASA 2003)

Source

NASA. 2003. *Columbia Accident Investigation Report*. R. Godwin. Ed. Washington, DC, USA: National Aeronautics and Space Administration (NASA). p. 101.

Discussion

This is an authoritative source endorsed by NASA on the subject of culture.

Customer (glossary)

The organization or person that receives a product or service. (ISO/IEC/IEEE 2015)

Sources

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2015.

Discussion

There are 9 definitions in the SEVOCAB repository of glossary entries. Several of these are very specific to particular standards. See <https://www.computer.org/sevocab> for more detail.

Cyber-Physical Systems (glossary)

(1) Cyber-Physical Systems (CPS) are integrations of computation with physical processes. Embedded computers and networks monitor and control the physical processes, usually with feedback loops where physical processes affect computations and vice versa. (Lee 2008, 363)

(2) ... in the past the science of computation has systematically abstracted away the physical world and vice versa. It is time to construct a Hybrid Systems Science that is simultaneously computational and physical, providing us with a unified framework for robust design flow with multi-scale dynamics and with integrated wired and wireless networking for managing the flows of mass, energy, and information in a coherent way. (Sha et al.)

(3) Cyber-physical systems (CPS) are physical and engineered systems whose operations are monitored, coordinated, controlled and integrated by a computing and communication core. (Rajkumar et al. 2010)

Source

(1) Lee, Edward A. "Cyber physical systems: Design challenges". *Object Oriented Real-Time Distributed Computing (ISORC)*, 2008 11th IEEE International Symposium on. 363-369. 2008.

(2) L. Sha, S. Gopalakrishnan, X. Liu and Q. Wang. "Cyber-Physical Systems: A New Frontier" *Sensor Networks, Ubiquitous and Trustworthy Computing*, 2008. SUTC '08. IEEE International Conference on, Taichung, 2008.

(3) Ragunathan (Raj) Rajkumar, Insup Lee, Lui Sha, and John Stankovic. "Cyber-physical systems: the next computing revolution". In *Proceedings of the 47th Design Automation Conference (DAC '10)*. ACM, New York, NY, USA, 731-736. 2010.

Discussion

Cyber-Physical Systems is still an emerging topic with no precise and universal agreed definition. Listed above are three descriptions of Cyber-Physical Systems from frequently referenced publications.

Cybernetics (glossary)

The study and modeling of communication, regulation and control in systems. (Wiener 1948)

Sources

Wiener, N. 1948. *Cybernetics: or Control and Communication in the Animal and the Machine*. Cambridge, MA, USA: The Massachusetts Institute of Technology Press.

Discussion

Cybernetics studies the flow of information through a system and how information is used by the system to control itself through feedback mechanisms. Early work in cybernetics in the 1940 was applied to electronic and mechanical networks, and was one of the disciplines used in the formation of early systems theory. It has since been used as a set of founding principles for all of the significant system disciplines.

Cycle (glossary)

A series of events that are regularly repeated in the same order; or move in or follow a regularly repeated sequence of events. (Oxford English Dictionary, 2020)

Sources

Oxford English Dictionary. 2020. s.v. "Cycle". "Cycles define and make things. Equally things contain Cycles." (Volk 1995) "A Temporal Pattern" (Mobus and Kalton 2015)

Discussion

Movement and evolution are fundamental in the dynamics of nature and thus cycles are evident, almost or even absolutely, everywhere. https://en.wikipedia.org/wiki/List_of_cycles

Due to its generality, Cycle is a key concept for Systemists, in our communication, in our thinking and our engagement with systems. The repetition of natural law and associated behavior in mechanical systems, and other examples of objective-subjective or system-context cycles are important in systems science. Cyclical phenomena are studied in several different ways in different sciences, e.g. dynamics, biology, sociology, cosmology etc.

Works Cited

Mobus G.E and M.C. Kalton. 2015. *Principles of System Science*. Springer.

Volk T (1996). *Metapatterns*. (1995) Columbia University Press.

Wikipedia.org. 2020. "List of Cycles". https://en.wikipedia.org/wiki/List_of_cycles

D

Data Center (glossary)

The data center is the department in an enterprise that houses and maintains back-end information technology (IT) systems and data stores—its mainframes, servers and databases. (Gartner 2012)

Source(s)

Gartner 2012. "Data Center". Accessed 25 July 2012. Available at: <https://www.gartner.com/it-glossary/data-center/>.

Discussion

In the days of large, centralized information technology (IT) operations, this department and all the systems resided in one physical place, hence the name data center.

With today's more distributed computing methods, single data center sites are still common, but are becoming less so. The term continues to be used to refer to the department that has responsibility for these systems, no matter how dispersed they are.

Decision Gate (glossary)

(1) A decision gate is an approval event (often associated with a review meeting). Entry and Exit criteria are established for each decision gate; continuation beyond the decision gate is contingent on the agreement of the decision-makers. (INCOSE 2011 p 362)

(2) A preplanned management event in the project cycle to demonstrate accomplishments, approve and baseline results, and approve the approach for continuing the project. (Also known as a control gate.) (Forsberg 2005, p 428)

Sources

(1) INCOSE. 2011. Systems Engineering Handbook, version 3.2.2. INCOSE-TP-2003-002-03.2.2.

(2) Forsberg, K., H. Mooz, H. Cotterman. 2005. Visualizing Project Management, 3rd Ed., Hoboken, NJ, USA: John Wiley and Sons.

Discussion

(1) The definition provided in the INCOSE Handbook is built on the language in ISO/IEC 15288:2015. Consequently it is at a high level of abstraction.

(2) The definition in "Visualizing Project Management" is tied to the project cycle and the project management issues related to it. The intent is to bring the systems engineering activities into the project context.

Decision Management (glossary)

Analysis of possible decisions using a formal evaluation process (glossary) that assesses identified alternatives against established criteria. (Created for SEBoK)

Source

This definition was developed for SEBoK v 1.0.

Discussion

Stakeholders should have input to the guidelines used to determine which issues should be included in the analysis.

Defense in depth (glossary)

A system resilience principle that states that a system should be capable of having two or more ways to address a single vulnerability. Jackson (2016)

Source

Jackson, Scott. 2016. "Principles for Resilient Design - A Guide for Understanding and Implementation." In IRGC Resource Guide on Resilience, edited by I. Linkov. University of Lausanne, Switzerland.

Discussion

Also called layered defence

Demonstration (glossary)

(1) A dynamic analysis technique that relies on observation of system or component behavior during execution, without need for post-execution analysis, to detect errors, violations of development standards, and other problems. (ISO/IEC 2010)

(2) A qualitative exhibition of functional performance against a set of test activities with pre-determined system stimuli designed to illustrate appropriate response or to show that operators can perform their allocated functions, usually accomplished with no or minimal instrumentation or test equipment. (Created for SEBoK)

Source

(1) ISO/IEC. 2010. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2010. Available from https://pascal.computer.org/sev_display/index.action.

(2) This definition was developed for the SEBoK.

Discussion

None.

Derived Requirement (glossary)

Constraint stated during the design activities which arise as a result of the selected solution (for example, a necessary mean or resource related to a technology, or an interface between two components of different sub-systems). (Faisandier 2012)

Source

(1) Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

(2) INCOSE. 1998. *INCOSE SE Terms Glossary*, version 0. Accessed September 12, 2012.

Discussion

None.

Descriptive Model (glossary)

(1) *A model that describes things as they are.* (Turban, et al 2010)

(2) *That depicts or describes how things actually work, and answers the question, "What is this?"*
(BusinessDictionary 2012)

Sources

(1) Turban, E., R. Sharda, and D. Delen. 2010. *Decision Support and Business Intelligence Systems*, 9th ed. Upper Saddle River, NJ, USA: Prentice-Hall.

(2) BusinessDictionary.com. 2012. s.v. "Descriptive Model." <https://www.businessdictionary.com/definition/descriptive-model.html> (accessed: September 12, 2012).

Discussion

Typical descriptive models may include those that describe the functional or physical architecture of a system, or the three dimensional geometric representation of a system.

Design (glossary)

(System) design includes activities to create concepts and models, and/or to conceive something (a system / a solution that answers an intended purpose) based on or using principles and concepts; the outcome of design activities is a coherent and purposeful set of models or representations using defined constructs and patterns (that implement principles and concepts). (ISO/IEC/IEEC 2009)

Sources

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

The definition provided above is a synthesis and adapted to systems of several understanding to the term "Design" as found in the sources cited.

In ISO/IEC/IEEE 24765:2010 - *Systems and software engineering – Vocabulary*, "design" means:

1. the process of defining the architecture, components, interfaces, and other characteristics of a system or component;
 2. the result of the process in (1);
 3. the process of defining the software architecture, components, modules, interfaces, and data for a software system to satisfy specified requirements;
 4. the process of conceiving, inventing, or contriving a scheme for turning a computer program specification into an operational program;
 5. activity that links requirements analysis to coding and debugging; or
 6. stage of documentation development that is concerned with determining what documentation will be provided in a product and what the nature of the documentation will be (ISO/IEC 26514:2008 Systems and software engineering--requirements for designers and developers of user documentation, 4.13)
-

Design Factor (glossary)

A solution parameter, characteristics, or relationship that influences the design at the system level.
(Singer et al. 2017)

Source

Singer, D., J. Strickland, N. Doerry, T. McKenney, and C. Whitcomb. 2017. "Set-Based Design." *SNAME T&R Bulletin SNAME (mt) Marine Technology Technical and Research Bulletin*.

Discussion

None.

Design Life (glossary)

The design life of a component or system is the period of time during which the item is expected by its designers to work within its specified parameters; in other words, the life expectancy of the item. It is the length of time between placement into service of a single item and that items on-set of wear-out. (wikipedia 2012)

Source

Wikipedia contributors. "Design life," *Wikipedia*, The Free Encyclopedia, https://en.wikipedia.org/w/index.php?title=Design_life&oldid=470916040 (accessed September 13, 2012).

Discussion

None.

Design Property (glossary)

A characteristic obtained during design through allocation of requirements, or estimate, analysis, study, calculation, simulation, etc. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

A design property is associated to a system element, a physical interface, a physical architecture. If the designed element complies with a requirement, the design property should equal the requirement. Otherwise one has to identify the difference or non conformance which treatment could conclude to modify the requirement, or the design, or identify a deviation.

DevOps (glossary)

The combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications and services at high velocity: evolving and improving products at a faster pace than organizations using traditional software development and infrastructure management processes. (Amazon)

Source

Available at: Amazon <https://aws.amazon.com/devops/what-is-devops/>.

Discussion

None.

Digital Elevation Model (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Digital Terrain Model (DTM) (glossary)

A digital elevation model (DEM) that incorporates the elevation of important topographic features on the land.

Note to entry: DTMs are comprised of mass points and breaklines that are irregularly spaced to better characterize the true shape of the bare-earth terrain. The net result of DTMs is that the distinctive terrain features are more clearly defined and precisely located, and contours generated from DTMs more closely approximate the real shape of the terrain.

Sources

ISO TC211. "Digital terrain model." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 1936. Available at: <https://isotc211.geolexica.org/concepts/1936/>^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/1936/>

Digital Twin (glossary)

A high-fidelity model of the system, which can be used to emulate the actual system. (SEBoK Original)

Source

SEBoK original.

Discussion

None.

SEBoK v. 2.1, 31 October 2019

Disposal (glossary)

The process of removing a system or component, ensuring proper handling of any environmentally sensitive materials, and sending the remainder to surplus storage or sale. (Mooz, et al 2003)

Source

Mooz, H., Forsberg, K., Cotterman, H. 2003. *Communicating Project Management*. Hoboken, NJ, USA: John Wiley and Sons. p. 159.

Discussion

It is critical to consider disposal from the very outset of the project. It is also not enough to consider only the end-of-life of the product(s) produced during the project. The process of producing a product will have byproducts which need to be managed and properly disposed of.

Disruption (glossary)

An interruption in the functionality of a system. Disruptions may be either internal or external. Internal disruptions may be due to human error, software error, or component failure. External disruptions may be caused by hostile attacks or natural phenomena, such as earthquakes, tsunamis, or hurricanes. Disruptions may also result from loss of resources (external) or from “system” errors, that is, failures caused by the interaction of two or more components that performed as designed (internal). (Jackson 2010)

Source

Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Hoboken, NJ, USA: John Wiley and Sons. p. 39-53.

Discussion

This book has a good discussion of the concept of disruptions. The word is used in other sources such as Hollnagel et al (Resilience Engineering: Concepts and Precepts, Ashgate Publishing Ltd, 2006, UK) and various government document.

Diversity (glossary)

The range of human differences, encompassing the characteristics that make one individual or group different from another. Diversity includes, but is not limited to, the following characteristics: race, ethnicity, culture, gender identity and expression, age, national origin, religious beliefs, work sector, physical ability, sexual orientation, socioeconomic status, education, marital status, language, physical appearance, and cognitive differences. (ABET 2020)

Source

ABET. 2020. "Diversity, Equity, and Inclusion." Accreditation Board for Engineering and Technology (ABET) website. <https://www.abet.org/about-abet/diversity-equity-and-inclusion/> Accessed: December 2020.

Discussion

See related terms

Domain (glossary)

A problem space. (IEEE 2010)

Sources

IEEE. 2010. *Standards for Information Technology*. New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1570-2010.

Discussion

Common usage includes "domain of application" and "domain engineering." In both cases a specific problem space is implied.

Drift Correction (glossary)

A resilience design principle that states that the drift of a system into a region of increasing risk should be monitored and appropriate responses should be formulated and executed – adapted from (Leveson et al. 2006)

In a resilience context according to Jackson and Ferris (2013) drift correction is a component principle of the adaptability grouping.

Sources

Leveson, N., N. Dulac, D. Zipkin, Cutcher-Gershenfeld, J. Carroll, and B. Barrett. 2006. "Engineering Resilience into a Safety-Critical System." In *Resilience Engineering: Concepts and Precepts*, edited by E. Hollnagel, D. D. Woods and N. Leveson. Aldershot, UK: Ashgate Publishing Limited.

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

This is a key resilience design principle. Although the definition above implies a gradual drift into a high risk state, it can also be interpreted to imply an immediate impending danger as was done by (Jackson 2010).

Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Edited by A. P. Sage, Wiley Series in Systems Engineering and Management. Hoboken, NJ, USA: John Wiley & Sons.

Drift Correction (glossary)

A resilience design principle that states that the drift of a system into a region of increasing risk should be monitored and appropriate responses should be formulated and executed – adapted from (Leveson et al. 2006)

In a resilience context according to Jackson and Ferris (2013) drift correction is a component principle of the adaptability grouping.

Sources

Leveson, N., N. Dulac, D. Zipkin, Cutcher-Gershenfeld, J. Carroll, and B. Barrett. 2006. "Engineering Resilience into a Safety-Critical System." In *Resilience Engineering: Concepts and Precepts*, edited by E. Hollnagel, D. D. Woods and N. Leveson. Aldershot, UK: Ashgate Publishing Limited.

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

This is a key resilience design principle. Although the definition above implies a gradual drift into a high risk state, it can also be interpreted to imply an immediate impending danger as was done by (Jackson 2010).

Jackson, S. 2010. *Architecting Resilient Systems: Accident Avoidance and Survival and Recovery from Disruptions*. Edited by A. P. Sage, Wiley Series in Systems Engineering and Management. Hoboken, NJ, USA: John Wiley & Sons.

Dualism (glossary)

(1) The division of something conceptually into two opposed or contrasted aspects, or the state of being so divided. (Stanford Encyclopedia of Philosophy 2011)

(2) In the history of thought, the idea that, for some particular domain, there are two fundamental kinds or categories of things or principles. (Stanford Encyclopedia of Philosophy 2011)

Sources

Stanford Encyclopedia of Philosophy. 2011. s.v. "Dualism." Accessed on September 11, 2012. Available at <https://plato.stanford.edu/entries/dualism/>.

Discussion

The yin yang concept in Chinese philosophy emphasizes the interaction between dual elements and their harmonization, ensuring a constant dynamic balance often through a cyclic dominance of one element and then the other, such as day and night (see Principles of Systems Thinking)

E

E-Services (glossary)

A collection of network-resident software services accessible via standardized protocols, whose functionality can be automatically discovered and integrated into applications or composed to form more complex services. (Hull et al. 2003)

Sources

Hull, R., M. Benedikt, V. Christiphides, J. Su. 2003. "E-Services: A Look Behind the Curtain." Proceedings of Symposium on Principles of Database Systems, 9-12 June, 2003, San Diego, CA.

Discussion

None.

Effectiveness (glossary)

(1) A systems effectiveness is a measure of its ability to perform the functions necessary to achieve goals or objectives, defined as the product of the number of combinations of behavior to reach a function and the efficiency of each combination. (Ackoff 1971)

(2) A combination of performance (how well a function is done in ideal conditions), availability (how often the function is there when needed) and survivability (how likely is it that the system will be able to use the function fully). (Hitchins 2007)

Sources

Ackoff, R.L. 1971. *Towards a System of Systems Concepts.* Management Science. 17(11).

Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology.* New York, NY, USA: Wiley

Discussion

(1) is a system science definition associated with the difference between behavior as system actions and function as combinations of behavior to provide external outcomes. Effectiveness is a measure of how well the system achieves its outcomes.

(2) is the systems engineering definition, relating effectiveness to achieving stakeholder needs.

Element (glossary)

Please see system element.

Ellipsoid (glossary)

Either (1) a geometric reference surface embedded in 3D Euclidean space represented by an ellipsoid of revolution where the rotation is about the polar axis, or (2) a reference ellipsoid, which is a geometric reference surface embedded in 3D Euclidean space formed by an ellipse that is rotated about a main axis.

Notes to Entry:

- For definition (1):
 1. For the Earth, the rotation is about the polar axis. This results in an oblate ellipsoid with midpoint of the foci located at the nominal center of the Earth.
 2. The two usual algorithms for latitude on an ellipsoid and on a sphere (such as used in spherical coordinates) are only equivalent if the ellipsoid is a sphere, having all radii equal in all directions. The problem is that a radial line from the center of a general ellipsoid does not always cross the surface of the ellipsoid orthogonally. In general, planar slices through the center do not intersect the surface orthogonally, and therefore the curves that correspond to the great circles of a sphere are not geodesics on the ellipsoid.
 3. The topology of the ellipsoid is inherited from the $\mathbb{R}^3$ space in which it is embedded. The difference is that metrics such as distance and direction on the ellipsoid are restricted to curves wholly on the surface and vectors tangent to the surface.
- For definition (2):
 1. For the Earth, the ellipsoid is bi-axial with rotation about the polar axis. This results in an oblate ellipsoid with the midpoint of the foci located at the nominal center of the Earth.

Sources

ISO TC211. Ellipsoid. In: ISO TC211 Multilingual Glossary. Version from 2020-06-02. Concept ID: 2382. Available at: <https://isotc211.geolexica.org/concepts/2382/>.

Discussion

None.

Ellipsoidal Height or Geodetic Height (glossary)

Ellipsoidal height, also known as geodetic height h , is the distance of a point from the reference ellipsoid along the perpendicular from the reference ellipsoid to this point, positive if upwards or outside of the reference ellipsoid.

Notes to Entry:

1. Only used as part of a three-dimensional ellipsoidal coordinate system or as part of a three-dimensional Cartesian coordinate system in a three-dimensional projected coordinate reference system, but never on its own.
2. Ellipsoidal height and geodetic height are registered with the ISO as two separate terms, but in practice are frequently used synonymously.

Sources

ISO TC211. "Ellipsoidal height." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2406. Available at: <https://isotc211.geolexica.org/concepts/2406/>.

Discussion

None.

Emergence (glossary)

The principle that whole entities exhibit properties which are meaningful only when attributed to the whole, not to its parts. Every model of a human activity system exhibits properties as a whole entity which derive from its component activities and their structure, but cannot be reduced to them.
(Checkland 1999)

Source

Checkland, P. B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

Discussion

There are other views of emergence which are not captured in Checkland's definition. Emergence has two aspects to it (like two sides of the same coin): One, for a given object, some of its properties may be identified as emergent with respect to a description of the object in terms of interacting elements. Two, if a set of given elements are allowed to interact, the resulting system may have properties that are not found in any one of the elements; these are the properties emerging as a result of the interaction and are therefore called the emergent properties of the system. In the first case, emergence arises as a result of the mode of description (no physical action or change involved), in the second case emergence arises as a result of a physical action or change (making the objects interact).

Emergent Behavior (glossary)

1. Behavior that is "in the process of coming into being or becoming prominent." (Apple 2011)
2. Behavior that is "arising unexpectedly or as a new or improved development." (Webster 1980)

Source(s)

- (1) Apple. 2011. "On-line Dictionary", version 2.2.3, 2005 - 2011
- (2) Webster's New World Dictionary. 1980. s.v. "Emergent Behavior."

Discussion

None.

Emergent Property (glossary)

Property of a system that is meaningful only at higher levels of the organization of a system and results from increased complexity at higher levels -- adapted from (Checkland 1999, p. 78)

Sources

Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York: John Wiley & Sons.

Discussion

Also, see definition of *emergence* and the Emergence and Complexity article.

Enabling System (glossary)

A system that complements a system-of-interest during its life cycle stages, but does not necessarily contribute directly to its function during its operation stage. (INCOSE 2012)

Source

INCOSE. 2012. *Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Discussion

An enabling system may support a System-of-Interest at any stage of its life. This may include direct maintenance and support systems, test, installation or training systems or project organization enabling systems such as project teams or information resources. See Enabling Systems Engineering.

Encapsulation (glossary)

(1) As a process, encapsulation means the act of enclosing one or more items within a (physical or logical) container. (Parnas 1972).

(2) A software development technique that consists of isolating a system function or a set of data and operations on those data within a module and providing precise specifications for the module. (IEEE 1990).

Sources

(1) Parnas,D. 1972. "On the Criteria To Be Used in Decomposing Systems Into Modules," *Communications of the ACM*, 5:(12) (December 1972) 1053-1058.

(2) IEEE. 1990. *Standard Glossary of Software Engineering*. Washington, DC, USA: Institute of Electrical and Electronics Engineers (IEEE). IEEE 610.12-1990.

Discussion

Much of the thinking in software engineering on abstraction, information hiding, and encapsulation was shaped by the classic 1972 paper by David Parnas. Parnas recommended that design decisions be encapsulated, especially decisions that are likely to change. This approach limits the rippling effect of a change in one element of a system so it does not result in changes spreading throughout the system.

Developments in Modular systems and in Object Oriented Programming and Design have largely been based on these ideas.

More generally System Encapsulation encloses system elements and their interactions from the external environment, and usually involves a system boundary that hides the internal from the external (see Concepts of Systems Thinking). Encapsulation can occur naturally in complex systems, as in the organs of the human body, or it can be used as a design strategy in man-made systems.

Engineered System (glossary)

(1) *An open, concrete system of technical or socio-technical elements which is the focus of a SE Life Cycle (glossary). Its characteristics include being created by and for people, having a purpose and satisfying key stakeholders' value propositions when considered as part of a broader System Context (glossary) . (Created for SEBoK)*

(2) *An engineered system is a system designed or adapted to interact with an anticipated operational environment to achieve one or more intended purposes while complying with applicable constraints.*

Sources

- (1) This definition was created for SEBoK v. 1.0.
- (2) INCOSE Fellows Briefing to INCOSE Board of Directors, January 2019.

Discussion

Definition (1) was adapted from the INCOSE Systems Engineering Handbook and systems science literature on open, concrete systems. See the Engineered Systems topic.

It is related to the definition proposed by Bartolomei et al (2006):

An engineering system is a complex socio-technical system that is designed, developed, and actively managed by humans in order to deliver value to stakeholders.

Definition (2) was created by the INCOSE Fellows Initiative on System and Systems Engineering Definitions. This was established in 2016, to review current INCOSE definitions of SYSTEM and SYSTEMS ENGINEERING and to recommend any changes necessary to align the definitions to a) current practice, and b) the aspirations of INCOSE's 2025 Vision. At the January 2019 INCOSE Board of Directors meeting, a new INCOSE definition for "system" was approved and is given above.

Expanding on this definition, the INCOSE Fellows state:

Thus, an "engineered system" is a system – not necessarily a technological one - which has been or will be "systems engineered" for a purpose.

Work Cited

Bartolomei, J.E., Hastings, D.E., de Neufville, R., and Rhodes, D.H. 2006. "Screening for Real Options "In" an Engineering System: A Step Towards Flexible Weapon System Development. " 4th Conference on Systems Engineering Research.v Los Angeles, CA, USA. April 2006. Available at [https:// esd. mit. edu/ wps/ esd-wp-2006-08.pdf](https://esd.mit.edu/wps/esd-wp-2006-08.pdf)^[1].

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

References

- [1] <https://esd.mit.edu/wps/esd-wp-2006-08.pdf>

Engineering (glossary)

(1) *The application of scientific knowledge to practical problems, or the creation of useful things. The traditional fields of mechanical engineering, electrical engineering, etc. are included in this definition.*

(Checkland 1999)

(2) *To (cleverly) arrange for something to happen.* (Checkland 1999)

Source

Checkland, P. B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

Discussion

None.

Engineering Change Management (glossary)

1. *Change Management: The comprehensive evaluation and approval or disapproval of a change that takes into consideration all effects of the change.* (Mooz 2003, p 116, 164)

2. *Engineering Change: An alteration in the configuration of a configuration item or other designated item after formal establishment of its configuration identification.* (Kossiakoff 2003, p 448)

Source(s)

Kossiakoff, A., and W. N. Sweet, 2003. *Systems Engineering: Principles and Practice*, J. Wiley & Sons, New Jersey. p. 448.

Mooz, H., K. Forsberg, and H. Cotterman, 2003. *Communicating Project Management*, J. Wiley & Sons, New Jersey. p. 116, 164.

Discussion

Engineering Change Management focuses on technical changes, or the technical aspect of a change, versus the business or budgetary aspect of the change. An engineering change notice is the formal release of an engineering revision to an approved baseline.

Engineering Change Proposal (ECP) (glossary)

An engineering change proposal (ECP) is the management tool used to propose a configuration change to a configuration item (CI) and the system's baselined performance requirements and configuration documentation established during design and development of the system (DoD 2001)

Sources

DoD. 2001. *MIL-HDBK-61A(SE) Configuration Management Handbook*, 7 February 2001. Accessed on 11 September 2012. Available at <https://acc.dau.mil/CommunityBrowser.aspx?id=142238>.

Discussion

None.

Engineering Coordinate Reference System (glossary)

A coordinate reference system based on an engineering datum.

Examples:

- Local engineering and architectural grids;
- coordinate reference system local to a ship or an orbiting spacecraft

Sources

ISO TC211. "Engineering coordinate reference system." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 706. Available at: <https://isotc211.geolexica.org/concepts/706/>^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/706/>

Enterprise (glossary)

(1) *one or more organizations sharing a definite mission, goals, and objectives to offer an output such as a product or service.* (ISO 15704 2000)

(2) *An organization (or cross organizational entity) supporting a defined business scope and mission that includes interdependent resources (people, organizations and technologies) that must coordinate their functions and share information in support of a common mission (or set of related missions).* (CIO Council 1999)

(3) *the term enterprise can be defined in one of two ways. The first is when the entity being considered is tightly bounded and directed by a single executive function. The second is when organizational boundaries are less well defined and where there may be multiple owners in terms of direction of the resources being employed. The common factor is that both entities exist to achieve specified outcomes.* (MOD 2004)

(4) *A complex, (adaptive) socio-technical system that comprises interdependent resources of people, processes, information, and technology that must interact with each other and their environment in support of a common mission.* (Giachetti 2010)

Source

(1) ISO 15704. 2000. *Industrial Automation Systems -- Requirements for Enterprise-Reference Architectures and Methodologies*. Geneva, Switzerland: International Organization for Standardization (ISO), ISO 15704:2000.

(2) CIO Council. 1999. *Federal Enterprise Architecture Framework (FEAF)*. Washington, DC, USA: Chief Information Officer (CIO) Council.

(3) MOD. 2004. *Ministry of Defence Architecture Framework (MODAF)*, version 2. London, UK: U.K. Ministry of Defence.

(4) Giachetti, R. E. 2010. *"Design of Enterprise Systems: Theory, Architecture, and Methods."* Boca Raton, FL, USA: CRC Press, Taylor and Francis Group.

Discussion

Definition (1) above is the SEBoK preferred definition. The focus of this definition is on purpose rather than structure, and the enterprise may be either enduring or transient, and either formally or informally constituted and governed. The simplest case of enterprise could be an individual and the largest a national or international undertaking.

Definition (2) is compatible with definition (1) and provides additional insight relevant to the practice of "enterprise architecting", which is closely related to Enterprise Systems Engineering.

Definition (3) highlights the fact that the word "enterprise" is sometimes used synonymously with "business". However in the SEBoK the term business is used to refer to a form of organization, while an enterprise often involves collaboration across organizational boundaries. Furthermore, we also use product and team to refer to two other forms of an organization. An enterprise *could* be at the business level, but some enterprises are at the project or team level. Therefore, as noted above, it is better to focus on the purpose rather than structure of an "enterprise" to facilitate the more proper use and better understanding of that term.

Definition (4) highlights the socio-technical nature of the enterprise and its ability to be adaptable to changing circumstances.

Enterprise Architecture (glossary)

- (1) *A rigorous description of the structure of an enterprise, its decomposition into subsystems, the relationships between the subsystems, the relationships with the external environment, the terminology to use, and the guiding principles for the design and evolution of an enterprise.* (Giachetti 2009)
- (2) *A strategic information asset base, which defines the business, the information necessary to operate the business, the technologies necessary to support the business operations, and the transitional processes necessary for implementing new technologies in response to the changing business needs. It is a representation or blueprint.* (CIO Council 1999)
- (3) *The formal description of the structure and function of the components of an enterprise, their interrelationships, and the principles and guidelines governing their design and evolution over time.* (MOD 2004)
- (4) *A discipline for proactively and holistically leading enterprise responses to disruptive forces by identifying and analyzing the execution of change toward desired business vision and outcomes. Enterprise architecture delivers value by presenting business and IT leaders with signature-ready recommendations for adjusting policies and projects to achieve target business outcomes that capitalize on relevant business disruptions. It is used to steer decision making toward the evolution of the future state architecture.* (Gartner 2013)
- (5) The practice of conducting enterprise analysis, design, planning, and implementation using a holistic approach for the successful development and execution of strategy. EA applies architecture principles and practices to guide organizations through the business, information, process, and technology changes necessary to execute their organization's strategies. (IEEE/ACM n.d. (b))

Source

- (1) Giachetti, R.E. 2009. *Design of Enterprise Systems: Theory, Architectures, and Methods*. Boca Raton, FL, USA: CRC Press.
- (2) CIO Council. 1999. *Federal Enterprise Architecture Framework (FEAF)*. Washington, DC, USA: Chief Information Officer (CIO) Council.
- (3) MOD. 2004. *Ministry of Defence Architecture Framework (MODAF)*, version 2. London, UK: U.K. Ministry of Defence.
- (4) Gartner IT Glossary. S.V. "enterprise architecture." Accessed 11 March 2013, available at: <https://www.gartner.com/it-glossary/enterprise-architecture-ea/>.
- (5) IEEE/ACM (n.d. (b)). *Guide to the Enterprise Information Technology Body of Knowledge* article on Enterprise Architecture. Available at: https://eitbokwiki.org/Enterprise_Architecture. Accessed on May 7, 2023.

Discussion

There are many different definitions of Enterprise Architecture which vary in the content and application of the architecture and in the importance of the various components of the architecture. The five definitions shown here reflect authoritative and expert sources.

Enterprise System (glossary)

A system context for which the system-of-interest is an enterprise. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

This definition was developed to help distinguish between an enterprise and an enterprise system in the SEBoK.

Enterprise Systems Engineering (ESE) (glossary)

The application of systems engineering principles, concepts, and methods to the planning, design, improvement, and operation of an enterprise. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

- (1) To enable more efficient and effective enterprise transformation, the enterprise needs to be looked at “as a system,” rather than as a collection of functions connected solely by information systems and shared facilities (Rouse 2009).
- (2) “The body of knowledge for enterprise systems engineering is evolving under such titles as enterprise engineering, Business (glossary) engineering, and Enterprise Architecture (glossary). Many systems and software engineering principles are applicable to enterprise engineering, but enterprise engineering’s unique complexities require additional principles. Enterprise engineering’s intent is to deliver a targeted level of enterprise performance in terms of shareholder value or customer satisfaction. Enterprise systems engineering methods include modeling; simulation; total quality management; change management; and bottleneck, cost, workflow, and value-added analysis.” (Joannou 2007)
- (3) A useful distinction between product system design and enterprise system design is that “enterprise design does not occur at a single point in time like the design of most systems. Instead, enterprises evolve over time and are constantly changing, or are constantly being designed.” (Giachetti 2010)

Works Cited

- Giachetti, R. E. 2010. *Design of Enterprise Systems: Theory, Architecture, and Methods*. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.
- Joannou, P. 2007. “Enterprise, Systems, and Software — the Need for Integration.” *Computer*, IEEE, May 2007.
- Rouse, W. B. 2009. “Engineering the Enterprise as a System.” In *Handbook of Systems Engineering and Management*. , eds. A. P. Sage, W. B. Rouse. 2nd ed. New York, NY, USA: Wiley and Sons, Inc.
-

Entropy (glossary)

(1) *A thermodynamic quantity representing the unavailability of a system's thermal energy for conversion into mechanical work, often interpreted as the degree of disorder or randomness in the system.* (Oxford English Dictionary Online 2012)

(2) *Lack of order or predictability; gradual decline into disorder.* (Oxford English Dictionary Online 2012)

(3) *Entropy is the tendency of systems to move towards disorder or disorganization. Negentropy describes the forces working in a system to hold off entropy.* (Hitchins 2007)

Sources

(1) & (2) Oxford English Dictionary. 2012. s.v. "Entropy."

(3) Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: John Wiley & Sons.

Discussion

Entropy has a specific mathematical definition related to transfer of energy (2nd law of thermodynamics) which in physics is also used to describing the loss of order or organization in a system over time.

Definiton (3) is the preferred Sysem Science view, work is put into to creating or organising a system, creating structure and order. If nothing more is done, the system will loss that order over time, unless more work (or energy) is added to the system during its life. This can be seen as an analogy for management, maintenance, training, repair, etc.

Environment (glossary)

(1) *Anything affecting a subject system or affected by a subject system through interactions with it, or anything sharing an interpretation of interactions with a subject system.* (IEEE 1175.1-2002 (R2007), 3.6)

(2) *The surroundings (natural or man-made) in which the system-of-interest is utilized and supported; or in which the system is being developed, produced or retired.* (INCOSE 2010)

Source

(1) IEEE. 2002. *IEEE Guide for CASE Tool Interconnections - Classification and Description*, 1175.1-2002.

(2) INCOSE. 2010. *INCOSE Systems Engineering Handbook*, version 3.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.

Discussion

(1) is a system science definition and can be applied to any system.

(2) is an engineered system definition, and distinguishes between the different environments that exist during the life of a system.

Equity (glossary)

The fair treatment, access, opportunity and advancement for all people, achieved by intentional focus on their disparate needs, conditions and abilities. Achieving equity requires understanding of historical and systemic patterns of disparity to address and eliminate barriers, and remove participation gaps as part of a comprehensive strategy to achieve equitable outcomes and social justice. (ABET 2020)

Source

ABET. 2020. "Diversity, Equity, and Inclusion." Accreditation Board for Engineering and Technology (ABET) website. <https://www.abet.org/about-abet/diversity-equity-and-inclusion/> Accessed: December 2020.

Discussion

See related terms

Ethics (glossary)

(1)*A system of moral principles.*(Dictionary.com 2011)

(2)*The rules of conduct recognized in respect to a particular class of human actions or a particular group, culture, etc.: e.g. medical ethics; Christian ethics; [engineering ethics].* (Dictionary.com 2011)

(3)*Moral principles, as of an individual: e.g. His ethics forbade betrayal of a confidence.* (Dictionary.com 2011)

(4)*The discipline dealing with what is good and bad and with moral duty and obligation.* (Merriam-Webster 2011)

Sources

Dictionary.com. s.v. "Ethics." Accessed July 18, 2011, <https://dictionary.reference.com/browse/ethics>

Merriam-Webster, Incorporated. s.v. "Ethics." accessed July 18, 2011, <https://www.merriam-webster.com/dictionary/ethics>

Discussion

Distinguishing between ethics and morals can seem a bit difficult. Though the two ideas are extremely closely-related, there is still a relatively simple way to keep the distinction clear. The idea of morals or morality relates closely to one's personal character and is the result of our upbringing, schooling, family and other influences leading us to know the difference between right and wrong. The term ethics, on the other hand draws more upon the idea of standards or codes of behavior. An individual's ethical responsibilities are significantly related to and guided by her profession or role in life.

Evolutionary (glossary)

A development method in which successive versions are produced to respond to discoveries surfaced by the previous versions. It is applied when requirements are uncertain and/or when technology experimentation is required. The alternative method is linear development. (Forsberg, Mooz, Cotterman 2005, p 423)

Sources

Forsberg, K., H. Mooz, H. Cotterman. 2005. *Visualizing Project Management*, 3rd ed. New York, NY, USA: J. Wiley & Sons.

Discussion

There is a substantial amount of discussion on evolutionary development in the section on Life Cycle Models.

Executable System Model (glossary)

A model that represents the time varying behavior of a system. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

None.

Extended Enterprise (glossary)

Wider organization' representing all associated entities - customers, employees, suppliers, distributors, etc. - who directly or indirectly, formally or informally, collaborate in the design, development, production, and delivery of a product to the end user. (BusinessDictionary.com 2012)

Source

BusinessDictionary.com. 2012. s.v. "Extended Enterprise." Accessed on 11 September 2012. Available at <https://www.businessdictionary.com/definition/extended-enterprise.html>.

Discussion

None.

F

Failure (glossary)

- (1) *Termination of the ability of a product to perform a required function or its inability to perform within previously specified limits. (ISO/IEC 2005, 4.2)*
- (2) *An event in which a system or system component does not perform a required function within specified limits. (ISO/IEC/IEEE 2009, 1)*
- (3) *The event in which any part of an item does not perform as required by its specification. The failure may occur at a value in excess of the minimum required in the specification, i.e., past design limits or beyond the margin of safety. (INCOSE 2012)*

Source

- (1) ISO/IEC. 2005. *Software Engineering - Software Product Quality Requirements and Evaluation (SQuaRE) - Guide to SQuaRE*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 25000:2005.
- (2) ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.
- (3) INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Discussion

None.

Failure Modes and Effects Criticality Analysis (glossary)

FMECA is a methodical process that provides identification of all the probable ways that parts, assemblies, and the system may fail, the causes for each failure, and the effect that the failure will have on the capability for the system to perform its mission is essential in the system design process. (DAU 2012)

Sources

DAU. 2012. "Defense Acquisition University, Acquisition Community Connection." Accessed on 11 September 2012. Available at <https://acc.dau.mil/CommunityBrowser.aspx?id=28905>.

Discussion

None.

Feature (glossary)

A distinctive attribute or aspect of something.

Source

Cambridge Online Dictionary. Accessed on May 11, 2022. Available: <https://dictionary.cambridge.org/us/dictionary/english/feature>.

Annotation

As examples, a geospatial feature is a description of a place on the earth, such as a mountain, lake, or city. A system design feature could be a function, quality, interface, or other distinctive attribute of the system design.

Feature attribute (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Feature Catalog (glossary)

A catalog containing definitions and descriptions of the feature types, feature attributes, and feature relationships occurring in one or more sets of geographic data, together with any feature operations that may be applied.

Source

ISO TC211. "Feature catalogue." In: ISO TC211 Multilingual Glossary. Version from 2020-06-02. Concept ID: 854. Available at: <https://isotc211.geolexica.org/concepts/854/>

Discussion

None.

Feature catalogue (glossary)

A catalog containing definitions and descriptions of the feature types, feature attributes, and feature relationships occurring in one or more sets of geographic data, together with any feature operations that may be applied.

Source

ISO TC211. "Feature catalogue." In: ISO TC211 Multilingual Glossary. Version from 2020-06-02. Concept ID: 854. Available at: <https://isotc211.geolexica.org/concepts/854/>

Annotation

None.

Federation of Systems (FoS) (glossary)

A system of systems that rates high on three dimensions of autonomy, heterogeneity, and dispersion. Each component system chooses of its own accord to participate in the FOS as it sees fit. It is a "coalition of the willing." (Krygiel 1999)

Source

Krygiel, A. J. 1999. *Behind the Wizard's Curtain: An Integration Environment for a System of Systems*. Arlington, VA, USA: C4ISR Cooperative Research Program (CCRP).

Discussion

None.

Flexibility (glossary)

The system's ability to restructure itself in response to external changes or pressures. (Woods 2006)

Sources

Woods, David D. 2006. "Essential Characteristics of Resilience," In *Resilience Engineering: Concepts and Precepts*, edited by E. Hollnagel, Woods, David D., and Leveson, Nancy. Aldershot, UK: Ashgate Publishing Limited.

Discussion

This is a key term in the context of resilience.

Form, Fit, Function, and Interface (F3I) (glossary)

(1) *A concept used for the update or upgrade of a system whereby a only a portion or subsystem of the entire system is replaced. The updated item is said to be Form/Fit/Function and interface compatible when it can be inserted into the existing system without impacting system operation. (Created for SEBoK)*

(2) *A mechanism to link design to performance requirements; i.e., replacing an item/system based on form, fit, function and interface characteristics. (DoD 2004)*

Source

(1) This definition was developed for the SEBoK.

(2) DoD. 2004. *Guidelines for Engineering, Manufacturing and Maintenance Documentation Requirements for Unique Identification (UID) Implementation*, version 1.0. Office of the Principal Deputy Under Secretary of Defense (Acquisition, Technology and Logistics). Accessed on 11 September 2012. Available at http://www.acq.osd.mil/dpap/Docs/uid/UIDEngDocGuidever4_120604.doc.

Discussion

F3I is an important concept regarding the transparency of a change to a portion of a system.

Framework (glossary)

Architectural Framework: Conventions, principles and practices for the description of architectures established within a specific domain of application and/or community of stakeholders. (ISO/IEC/IEEE 2011)

Source

ISO/IEC/IEEE. 2011. *Systems and software engineering - Architecture description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE), ISO/IEC/IEEE 42010.

Discussion

Examples are the Generalised Enterprise Reference Architecture and Methodologies (GERAM) (ISO 15704) and the Reference Model of Open Distributed Processing (RM-ODP) (ISO/IEC 10746).

Works Cited

ISO. 2000. *Industrial Automation Systems -- Requirements for Enterprise-Reference Architectures and Methodologies*. Geneva, Switzerland: International Organization for Standardization (ISO). ISO 15704:2000.

ISO/IEC. 1998. *Information Technology — Open Distributed Processing — Reference Model: Architecture*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 10746:1998.

Function (glossary)

(1) *A system outcomes which contribute to goals or objectives. To have a function, a system must be able to provide the outcome through two or more different combinations of elemental behavior. (Ackoff 1971)*

(2) *An action, a task, or an activity performed to achieve a desired outcome. (Hitchins 2007)*

(3) *A function is defined by the transformation of input flows to output flows, with defined performance. (Created for SEBoK)*

(4) *A broad work area encompassing multiple related disciplines (e.g., Engineering, Finance, Human Resources, etc.). (Created for SEBoK)*

Source

(1) Ackoff, R.L. 1971. "Towards a System of Systems Concepts". *Management Science*. 17(11).

(2) Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: John Wiley & Sons.

(3) and (4) These definitions were developed for the SEBoK.

Discussion

(1) is the system science definition. All systems have behavior, but to be able to function in a certain way they must have a certain richness of behavior. For example, fight, flight or hide are behaviors; while response to external threats is a function.

(2) is the more traditional Systems Engineering definition, in which functions are identified top down based on needs and then decomposed and allocated to lower level system.

(3) is a HW/SW or human system component definition, describing a function which can be performed by a single part of the system with identified performance.

(4) is a management definition, describing how similar functions are collected in organizational units.

Functional Architecture (glossary)

1. *A functional architecture is a set of functions and their sub-functions that defines the transformations of input flows into output flows performed by the system to achieve its mission.* (Created for SEBoK)
2. *The inter-related set of transformative processes and purposeful input-output tasks that a system performs, or can perform, on input(s) from external or internal sources in order to produce output(s) that supports the achievement of mission objectives; aspects of a system definition concerning the manner in which a system operates on, and/or in relation to, intangible and tangible inputs from the entities, users, and environments which exist around it in the surrounding system context through the transformation of inputs into outputs.* (Created for SEBoK)

Source

Definitions 1 and 2 were both developed for the SEBoK. Definition developed by BKCASE ^[1] for the v1.0 (2012) Logical Architecture article; Definition 2 developed by C. Singam for the v2.10 Functional Architecture article.

Discussion

Definition 1 is adapted from ISO/IEC/IEEE 24748 - 4 (past IEEE 1220, ISO/IEC 26702) as functional architecture; but it is here modified to emphasize on transformations performed. See also definition of behavioral architecture (glossary).

Works Cited ISO/IEC. 2010. *Systems and Software Engineering, Part 1: Guide for Life Cycle Management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 24748-1:2010.

References

- [1] https://sebokwiki.org/w/index.php?title=Logical_Architecture&direction=next&oldid=30959

Functional redundancy (glossary)

A system resilience principle that states that there should be two or more different ways to perform a critical task. Jackson and Ferris (2013)

Source

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

Also called design diversity by Leveson (1995)

G

Gazetteer (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

General System Theory (glossary)

General system theory (GST), attempts to formulate principles relevant to all open systems. GST is based on the idea that correspondence relationships (homologies) exist between systems from different disciplines. Thus, knowledge about one system should allow us to reason about other systems. (von Bertalanffy 1968)

Sources

von Bertalanffy, L. 1968. *General system theory: Foundations, development, applications*. Revised ed. New York, NY: Braziller.

Discussion

Ludwig von Bertalanffy developed a research approach based on open system theory (Bertalanffy 1950). He was one of a number of natural scientists who realized that the reductionist, closed system, approach could not be used to explain the behavior of an organism in its environment.

GST also implies a scientific approach, with identify laws and generalized theory to unify all science. Bertalanffy was cofounder, along with Kenneth Boulding (economist), Ralph Gerard (physiologist) and Anatol Rapoport (mathematician), of the Society for General Systems Research in 1957. This group is considered by many to be the founders of System Age Thinking (Flood 1999).

Work Cited

Bertalanffy, L. von. 1950. "The Theory of Open Systems in Physics and Biology". *Science*, New Series, 111(2872) (Jan 13): 23-29

Geodesic or Geodesic Line (glossary)

A curve on a surface with a zero-length tangential curvature vector.

Notes to Entry:

1. A geodesic's curvature vector is perpendicular to the surface, thus has the minimum curvature of any curve restricted to the surface. This is often defined as a minimal distance curve between two points, but this does not always suffice, since some points (especially on ellipsoids and spheres) are often joined by more than one geodesic. For example, on an ellipsoid the points with $(\varphi, \lambda) = (0, 0)$ and $(0, 180)$ are joined by four separate geodesic [2 polar (the shorter) and 2 equatorial]. The exponential map is only guaranteed to be one-to-one for a small area (depending on where the center is and how the surface is curved).
2. Geodesic and geodesic line are registered with the ISO as two separate terms, but in practice are frequently used synonymously.

Source

ISO TC211. "Geodesic." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2055. Available at: <https://isotc211.geolexica.org/concepts/2055/>.

Discussion

None.

Geodesy (glossary)

A science that studies the shape of the Earth in the gravitational field.

Sources

OSGeo. "Geodesy." In: *OSGeo Glossary*. Version 1.0. OSGeo, 2021-01-22. Concept ID: 126. Available at: <https://osgeo.geolexica.org/concepts/126/> ^[1].

Discussion

None.

References

[1] <https://osgeo.geolexica.org/concepts/126/>

Geodetic Coordinate System (glossary)

A geodetic coordinate system, or its synonym ellipsoidal coordinate system, is a coordinate system in which position is specified by geodetic latitude, geodetic longitude and (in the three-dimensional case) ellipsoidal height.

Note to entry: Geodetic coordinate system and ellipsoidal coordinate system are used synonymously but have separate entries in the ISO.

Source

ISO TC211. "Geodetic coordinate system." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 196. Available at: <https://isotc211.geolexica.org/concepts/196/>.

Discussion

None.

Geodetic Datum (glossary)

A reference frame or datum describing the relationship of a two- or three-dimensional coordinate system to the Earth.

Source

ISO TC211. "Geodetic datum." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 708. Available at: <https://isotc211.geolexica.org/concepts/708/> ^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/708/>

Geodetic Reference Frame (glossary)

A reference frame or datum describing the relationship of a two- or three-dimensional coordinate system to the Earth.

Sources

ISO TC211. "Geodetic reference frame." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2408. Available at: <https://isotc211.geolexica.org/concepts/2408/>^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/2408/>

Geographic coordinates (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Geographic Data (glossary)

A data with implicit or explicit reference to a location relative to the Earth.

Note to entry: Geographic information is also used as a term for information concerning phenomena implicitly or explicitly associated with a location relative to the Earth.

Sources

ISO TC211. "Geographic data." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 202. Available at: <https://isotc211.geolexica.org/concepts/202/> ^[1].

References

[1] <https://isotc211.geolexica.org/concepts/202/>

Geographic Identifier (glossary)

A spatial reference in the form of a label or code that identifies a location.

Example: 'Spain' is an example of a country name, 'SW1P 3AD' is an example of a postcode.

Sources

ISO TC211. "Geographic identifier." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 204. Available at: <https://isotc211.geolexica.org/concepts/204/> ^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/204/>

Geographic Imagery (glossary)

Imagery associated with a location relative to the Earth.

Sources

ISO TC211. "Geographic imagery." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 792. Available at: <https://isotc211.geolexica.org/concepts/792/> ^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/792/>

Geographic Information (glossary)

Information concerning phenomena implicitly or explicitly associated with a location relative to the Earth.

Sources

ISO TC211. "Digital terrain model." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 205. Available at: <https://isotc211.geolexica.org/concepts/205/> ^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/205/>

Geographic Information System (glossary)

An information system dealing with information concerning phenomena associated with location relative to the Earth.

Sources

ISO TC211. "Geographic information system." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 207. Available at: <https://isotc211.geolexica.org/concepts/207/>

Discussion

None.

Geoid (glossary)

The equipotential surface of the Earth's gravity field which is perpendicular to the direction of gravity and which best fits mean sea level either locally, regionally or globally.

Sources

ISO TC211. "Geoid." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2370. Available at: <https://isotc211.geolexica.org/concepts/2370/>.

Discussion

None.

GEOINT (glossary)

Geospatial Intelligence

Source

Abbreviation.

Annotation

None.

Geopositioning (glossary)

The determination of the geographic position of an object.

Sources

ISO TC211. "Geopositioning." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2467. Available at: <https://isotc211.geolexica.org/concepts/2467/>^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/2467/>

Glossary:Principle

A basic idea or rule that explains or controls how something happens or works. (Cambridge Dictionary 2020)

Source

Cambridge Dictionary. s.v. "Principle." accessed 12 May 2020, <https://dictionary.cambridge.org/dictionary/english/principle>.

Discussion

None.

Governance (glossary)

(1) *System by which organizations [or systems] are directed and controlled.* (ISO/IEC 2008, 1.6.2)

(2) *Organizational chains of responsibility, authority, and communication for executing measurement and control mechanisms to effectively drive the organization and enable people to perform roles their respective roles and responsibilities.* (Cantor 2006)

(3) *A decision-making process that defines the responsibility and authority of decision makers and stakeholders for identifying, defining, discussing, making, and implementing decisions in the face of complex problems, multiple stakeholders with diverse and conflicting objectives, and resource constraints. For engineering governance, the problem complexity usually includes significant technology complexity.* (Created for SEBoK)

Source

(1) ISO/IEC. 2008. *Corporate governance of information technology*. ISO/IEC 38500:2008. Accessed on 11 September 2012. Available at https://www.iso.org/iso/catalogue_detail?csnumber=51639.

(2) Cantor, M. 2006. "Estimation Variance and Governance." In IBM developerWorks. Accessed on 15 September 2011. Available at <https://www.ibm.com/developerworks/rational/library/mar06/cantor/>.

(3) This definition was developed for the SEBoK.

Discussion

None.

Gravity-Related Height (glossary)

The gravity-related height 'H' is a height that is dependent on the Earth's gravity field.

Notes to Entry:

1. This refers to, amongst others, orthometric height and Normal height, which are both approximations of the distance of a point above the mean sea level, but also may include Normal-orthometric heights, dynamic heights or geopotential numbers.
2. The distance from the reference surface may follow a curved line, not necessarily straight, as it is influenced by the direction of gravity.

Sources

ISO TC211. "Gravity-related height." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2371. Available at: <https://isotc211.geolexica.org/concepts/2371/> ^[1].

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/2371/>

Green Engineering (glossary)

Green engineering is the design, commercialization and use of processes and products that are feasible and economical while:Reducing the generation of pollution at the sourceMinimizing the risk to human health and the environment. (EPA 2012)EPA. 2012. "Green Engineering." United States Environmental Protection Agency. Accessed on 11 September 2012. Available at <https://www.epa.gov/oppt/greenengineering/>.None.

H

Habitability (glossary)

Habitability is the consideration of the characteristics of systems focused on satisfying personnel needs that are dependent upon physical environment. Habitability analyzes factors of working conditions and accommodations that are necessary to sustain the morale, safety, health, and comfort of the user population that contribute directly to personnel effectiveness and mission accomplishment. (USAF 2012)

Sources

USAF. 2012. "US Air Force HSI (Human-System Integration) 2012." Accessed on 11 September 2012. Available at <https://ww3.safahq.af.mil/organizations/afhsio/hsidomains.asp>.

Discussion

None.

Hard System (glossary)

System type characterized by the ability to define purpose, goals and missions that can be addressed via engineering methodologies in attempting to optimize a solution. (Checkland 1999)

Source

Checkland, P. B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

Discussion

None.

Hardware-in-the-Loop Testing (glossary)

A type of dynamic simulation that includes one or more actual system components operating in conjunction with simulated components. (Kossiakoff and Sweet 2003, 449)

Sources

Kossiakoff, A., and W. N. Sweet, 2003. *Systems Engineering: Principles and Practice*, Hoboken, NJ: J. Wiley & Sons.

Discussion

None.

Healthcare (glossary)

The maintenance and improvement of physical and mental health, especially through the provision of medical services. (Oxford Dictionaries)

Source(s)

Oxford Dictionaries. s.v. "healthcare." Accessed February 18, 2016. https://www.oxforddictionaries.com/us/definition/american_english/healthcare

Discussion

The term *healthcare* generally sometimes is used to refer only to the improvement of physical or mental health; i.e. to the delivery of medical care after there is an issue. This broader definition was chosen to encompass preventive approaches to healthcare as well.

Healthcare Systems Engineering (glossary)

The application of systems engineering or systems thinking approaches, methods, processes, and tools to healthcare systems, i.e. systems used to maintain and improve physical or mental health of individuals, as well as to population-level healthcare issues. (Created for SEBoK)

Source(s)

Created for the SEBoK.

Discussion

Some uses of the term *healthcare systems engineering* refer to systems engineering specifically within the healthcare delivery space; i.e. the provision of health-related services by medical professionals. However, there are applications for systems engineering throughout the healthcare sector, including in the development of medical devices and pharmaceuticals, in public health, systems biology, etc. The SEBoK Authors created this definition to reflect the wider applications of systems engineering which are beginning to occur and which will grow in future.

Height (glossary)

The distance of a point from a chosen reference surface positive upward along a line perpendicular to that surface.

Notes to entry:

1. A height below the reference surface will have a negative value.
2. Generalization of ellipsoidal height (h) and gravity-related height (H).
3. See also geodetic height which is used synonymously to ellipsoidal height.

Sources

ISO TC211. "Height." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2409. Available at: <https://isotc211.geolexica.org/concepts/2409/>.

Discussion

None.

Heuristic (glossary)

Widely accepted qualitative statements that, as judged from examples, add structure to ill-defined situations (Rechlin 1991)

Sources

Rechlin, Eberhardt. 1991. *Systems Architecting: Creating and Building Complex Systems*. Englewood Cliffs, NJ: CRC Press.

Discussion

Rechlin has at least five definitions of this term. However, they are not sufficiently different to warrant listing them all. This one is typical.

Hierarchy (glossary)

(1) A hierarchy is an arrangement of items (objects, names, values, categories, etc.) in which the items are represented as being "above," "below," or "at the same level as" one another. Levels in a hierarchy may also represent authority, control or ownership of lower levels (command structure). (Oxford English Dictionary 2012)

(2) Organisation of system elements into groups of (sub-)system wholes, based on relative strength of element cohesion. (Simon 1962)

(3) The hierarchy principle of systems, is the consequential concept of simultaneous multiple containment of one system by many containing systems. (Checkland 1999)

(4) An abstract view of a defined system that is developed to meet some need. The system results from an analysis that decomposes a system into constituent elements at two or more levels. Often based around a defined taxonomy of levels covering System, element, Subsystem, assembly, components and parts. (INCOSE 2011)

Source

(1) Oxford English Dictionary, s.v. "Hierarchy."

(2) Simon, H.A. 1962. "The Architecture of Complexity." *Proceedings of the American Philosophical Society*. 106(6) (December 12, 1962): 467-482.

(3) Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York: John Wiley & Sons.

(4) INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Discussion

(1) is a general dictionary definition.

(2) is a system science definition, which relates to the tendency of natural systems to form hierarchies and to the stability of hierarchical structures for man-made systems.

(3) Soft systems definition, allowing for a system to be associated with several hierarchical structure, one of the factors leading to multiple stakeholder viewpoints. This is the definition which should be used for problem systems.

(4) defines the man-made hierarchy created within a system. The use of a standard taxonomy defined against the types of technology and use of system elements can be useful as a common language for defining production of support processes. Care must be taken to avoid this leading to a reductionist (glossary) approach. This is the definition which should be used for hard solution systems.

High Reliability Organizations (HROs) (glossary)

Organizations that operate continuously under trying conditions and have fewer than their fair share of major accidents. (Weick and Sutcliffe 2007)

Sources

Weick, K. E. and K.M. Sutcliffe. 2007. *Managing the Unexpected: Resilient Performance in an Age of Uncertainty*, 2nd Edition. San Francisco, CA: Jossey-Bass.

Discussion

None.

Holism (glossary)

- (1) *The theory that parts of a whole are in intimate interconnection, such that they cannot exist or be understood independently of the whole.* (von Bertalanffy 1968)
- (2) *To take a Holistic view of a situation.* (von Bertalanffy 1968)

Sources

(1) and (2) von Bertalanffy, L. 1968. *General system theory: Foundations, development, applications*, Revised ed. New York, NY: Braziller.

Discussion

The term "Holistic (glossary)" is used to describe an approach based on holism.

Holistic (glossary)

See Holism (glossary)

Homeostasis (glossary)

- (1) *The ability or tendency of an organism or cell to maintain internal equilibrium by adjusting its physiological processes.* (The Free Dictionary)
- (2) *A Homeostatic system state is one where the system is static but its elements are dynamic. The system maintains its state by internal adjustments.* (Ackoff 1971)
- (3) *Homeostasis is the maintenance of suitable operating conditions for all contained systems.* (Hitchins 2007)

Sources

(1) The Free Dictionary. s.v. "Homeostasis." Available at: <https://www.thefreedictionary.com/homeostasis>.

Ackoff, R.L. 1971. "Towards a System of Systems Concepts." *Management Science*. 17(11), USA.

Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: John Wiley & Sons.

Discussion

- (1) dictionary definition for living organisms.
 - (2) is theoretical system science definition which applies to all system, conducting internal actions to hold themselves in the same state (see also Entropy (glossary)).
 - (3) applies to complex engineered systems and describes system activities or sub systems who's purpose to to maintain stable conditions inside the system to allow other system to operate, e.g. air conditioning or sound insulation.
-

Human Factors (glossary)

The systematic application of relevant information about human abilities, characteristics, behavior, motivation, and performance. It includes principles and applications in the areas of human related engineering, anthropometrics, ergonomics, job performance skills and aids, and human performance evaluation. (INCOSE 2011, 363)

Sources

INCOSE. 2011. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Discussion

None.

Human in the loop (glossary)

a system resilience principle that states that there should always be a human in the system when there is a need for human cognition. Jackson and Ferris (2013)

Source

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Human Survivability (glossary)

Human survivability addresses human-related characteristics of a system (e.g., life-support, body armor, helmets, plating, egress/ejection equipment, air bags, seat belts, electronic shielding, alarms, etc.) that reduce susceptibility of the total system to mission degradation or termination; injury or loss of life; and partial or complete loss of the system or any of its elements. These issues must be considered in the context of the full spectrum of anticipated operations and operational environments and for all people who will interact with the system (e.g., users/ customers, operators, maintainers, or other support personnel). (INCOSE 2011, 336)

Sources

INCOSE 2011. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.1. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.1.

Discussion

None.

Human Systems Integration (HSI) (glossary)

An interdisciplinary technical and management process for integrating human considerations with and across all system elements, an essential enabler to systems engineering practice. (ISO/IEC/IEEE 2011)

Sources

ISO/IEC/IEEE. 2011. *Systems and software engineering - Requirements engineering*. Geneva, Switzerland: International Organization for Standardization (ISO) / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers (IEEE), (IEC), ISO/IEC/IEEE 29148:2011, 4.1.11.

Discussion

None.

Hydrography (glossary)

*the science that **measures and describes** the physical features of bodies of water*

Source

NOAA.gov website. Accessed on May 11, 2022. Available: <https://oceanservice.noaa.gov/facts/hydrography.html>.

Annotation

None.

I

Implementation (glossary)

The process that actually yields the lowest level system elements in the system hierarchy (system breakdown structure).(DAU 2011)

Source

DAU. 2011. "4.2.3.2.5. Integration Process," in *Defense Acquisition Guidebook: Your Acquisition Policy and Discretionary Best Practice Guide*. Ft. Belvoir, VA: Defense Acquisition University (DAU). Accessed on 11 September 2012. Available at: [https:// acc. dau. mil/ CommunityBrowser.aspx?id=332973#4.2.3.2.5](https://acc.dau.mil/CommunityBrowser.aspx?id=332973#4.2.3.2.5).

Discussion

For a full discussion of the role and importance of system implementation in systems engineering see the System Implementation article.

In-Process Validation (glossary)

The practice of routinely getting user approval to guide each elaboration of the baseline. This technique keeps the user involved and committed to the incremental elaboration of the approach and the incremental verification results. (Mooz, Forsberg, and Cotterman 2003,198)

Sources

Mooz, H., K. Forsberg, H. Cotterman. 2003. *Communicating Project Management*. Hoboken, NJ: John Wiley and Sons.

Discussion

One reason that small software development projects, using Agile development principles, work so well is that the customer is embedded into the team. Agile teams are usually limited to a maximum of 20 people (although there are instances of teams of teams that have hundreds of people involved and working well). The in-process validation concept can effectively capture the customer involvement as a very positive force on large projects.

Inclusion (glossary)

The intentional, proactive, and continuing efforts and practices in which all members respect, support, and value others. An inclusive environment provides equitable access to opportunities and resources, empowers everyone to participate equally, and offers respect in words and actions for all. (ABET 2020)

Source

ABET. 2020. "Diversity, Equity, and Inclusion." Accreditation Board for Engineering and Technology (ABET) website. <https://www.abet.org/about-abet/diversity-equity-and-inclusion/> Accessed: December 2020.

Discussion

See related terms

Increment (glossary)

One of a series of planned additions and contributions. (Mooz, Forsberg, Cotterman 2003, 194)

Sources

Mooz, H., K. Forsberg, H. Cotterman. 2003. *Communicating Project Management*. Hoboken, NJ: John Wiley and Sons.

Discussion

See also incremental.

Incremental (glossary)

To operate using increments. (Mooz, Forsberg, and Cotterman 2003)

Sources

Mooz, H., K. Forsberg, H. Cotterman. 2003. *Communicating Project Management*. Hoboken, NJ: John Wiley and Sons.

Discussion

See also increment.

Industrial Engineering (glossary)

The design, improvement and installation of integrated systems of people, materials, information, equipment and energy. It draws upon specialized knowledge and skill in the mathematical, physical, and social sciences together with the principles and methods of engineering analysis and design, to specify, predict, and evaluate the results to be obtained from such systems (IIE 1992).

Sources

IIE. 1992. *Industrial Engineering Terminology, Revised Edition*. Institute of Industrial Engineers (IIE). Accessed 7 March 2012. Available at <https://www.iienet2.org/Details.aspx?id=645>.

Discussion

None.

Information and Communications Technologies (ICT) (glossary)

Information and communication technology...include computer hardware (computers, storage devices, printers, and other peripherals); computer software (operating systems, programming tools, utilities, applications, and internal software development); computer services (information technology consulting, computer and network systems integration, Web hosting, data processing services, and other services); and communications services (voice and data communications services) and wired and wireless communications equipment. (World Bank 2012)

Sources

World Bank. 2012. *Information and Communication Technology Expenditure*. Accessed on 11 September 2012. Available at <https://search.worldbank.org/data?qterm=ICT&language=EN>.

Discussion

None.

Information Category (glossary)

An element of a classification scheme describing the information needs of managers. 'Practical Software Management' defines seven of these:

- *Schedule and progress*
- *Resources and cost*
- *Product size and stability*
- *Product quality*
- *Process performance*
- *Technology effectiveness*
- *Customer satisfaction*

Source(s)

Card, D, and R. McIver. 2003. *Applying PSM to Enterprise Measurement*. Technical Report for US Army TACOM. Herndon, VA, USA: Software Productivity Consortium.

Discussion

None.

Information Need (glossary)

Insight necessary to manage objectives, goals, risks, and problems. (ISO/IEC/IEEE 2007)

Sources

ISO/IEC/IEEE. 2007. *Systems and software engineering - Measurement process*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronic Engineers (IEEE), ISO/IEC/IEEE 15939:2007, 3.12.

Discussion

None.

Information Technology (glossary)

Resources required to acquire, process, store and disseminate information. (ISO/IEC/IEEE 2008)

Sources

ISO/IEC/IEEE. 2008. *Corporate governance of information technology*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 38500:2008, 1.6.7.

Discussion

Note from SEVOCAB [1]: includes communication technology (CT) and the composite term information and communication technology (ICT).

Infrastructure (glossary)

The assets, systems, and networks, whether physical or virtual, whose incapacitation or destruction would have a debilitating effect on security, national economic security, public health or safety, or any combination thereof. -- adapted from Department of Homeland Security (DHS 2010, 46)

Sources

DHS. 2010. *Department of Homeland Security Risk Lexicon*, 2010 edition. Accessed on 11 September 2012. Available at <https://www.dhs.gov/xlibrary/assets/dhs-risk-lexicon-2010.pdf>.

Discussion

This term is discussed in a resilience context. That is also the context in which the *DHS Risk Lexicon* was written.

Input (glossary)

When used as a noun:

- (1) *Data received from an external source.* (ISO/IEC 2009)
- (2) *Pertaining to a device, process, or channel involved in receiving data from an external source.* (ISO/IEC 2009)
- (3) *In an IDEF0 model, that which is transformed by a function into output.* (ISO/IEC 2009)
- (4) *Any item, whether internal or external to the project that is required by a process before that process proceeds.* (ISO/IEC 2009)
- (5) *Data entered into an information processing system or any of its parts for storage or processing.* (ISO/IEC 2009)

When used as a verb:

- (6) *The process of entering data into an information processing system or any of its parts for storage or processing.* (ISO/IEC 2009)
- (7) *To receive data from an external source;* (ISO/IEC 2009)
- (8) *To provide data from an external source.* (ISO/IEC 2009)

Source

(1) - (8) ISO/IEC. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Accessed on 11 September 2012. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Input-Output Flow (glossary)

An input or an output, or a trigger of a function (used for input-output flow, control flow). An Input-Output Flow can be made of material and/or energy, and/or information. (Faisandier 2012)

Source

Faisandier, A. 2012. *Systems Opportunities and Requirements*. Belberaud, France: Sinergy'Com.

Discussion

See Logical Architecture Model Development article.

Installation (glossary)

Period of time in the system life cycle during which a product is integrated into its operational environment and tested in this environment to ensure that it performs as required. (Adapted from (ISO/IEC 2009))

Source

ISO/IEC. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21, 2009]. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Integrated Product Team (IPT) (glossary)

(1) *An interdisciplinary team assigned to an element of the system breakdown structure* (Created for SEBoK).

(2) *An interdisciplinary team* (Created for SEBoK).

Sources

This definition was developed for the SEBoK.

Discussion

The primary definition is adapted from Browning (2009) and corresponds to the traditional meaning of "Integrated Product Team". The second definition is not found in any reference but corresponds to the meaning that has evolved in many segments of industry. In industry the term "functional IPT" is also used to designate teams that are not assigned to an element of the system structural breakdown structure but rather represent disciplines that cross system element boundaries, such as reliability.

Works Cited

Browning, T.R. 2009. "Using the Design Structure Matrix to Design Program Organizations." In *Handbook of Systems Engineering and Management*, edited by A.P. Sage and W.B. Rouse. Hoboken, NJ: John Wiley & Sons, p. 1415-1416.

Integration (glossary)

A process that combines system elements to form complete or partial system configurations in order to create a product specified in the system requirements. (ISO/IEEE 2008)

Source

ISO/IEEE. 2008. *Systems and Software Engineering - Software Life Cycle Processes*. Geneva, Switzerland: International Organization for Standards (ISO)/Institute of Electrical & Electronics Engineers (IEEE) Computer Society, ISO/IEEE 12207:2008(E).

Discussion

For a full discussion of the role and importance of integration in systems engineering see the System Integration article.

Integrity (glossary)

[1] *the state of being whole and undivided. (Oxford Online Dictionary 2016)*

[2] Degree to which a system or component prevents unauthorized access to, or modification of, computer programs or data. *(ISO/IEC 2011)*

Sources

[1] Oxford On-line Dictionary (2016)

[2] ISO/IEC. 2011. *Systems and Software Engineering - Systems and Software Quality Requirements and Evaluation (SQuaRE) - Systems and Software Quality Models*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 25010:2011.

Discussion

In a resilience context integrity is an attribute of a system before, during and after an encounter with a threat.

Inter-Node Interaction (glossary)

The combination of communications, cooperation, and collaboration among the different nodes or elements of a system to act together to solve both local and strategic problems. Adapted from (Woods 2006)

According to Jackson and Ferris (2013) internode interaction is the primary principle of the integrity characteristic.

Sources

Woods, D.D. 2006. "Essential Characteristics of Resilience." In *Resilience Engineering: Concepts and Precepts*, edited by E. Hollnagel, Woods, David D., and Leveson, Nancy. Aldershot, UK: Ashgate Publishing Limited.

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

This is essentially the same concept discussed by Woods (2006) in a resilience context except that the term used by Woods is "cross-scale interaction."

Interface (glossary)

1. *A shared boundary between two functional units, defined by various characteristics pertaining to the functions, physical signal exchanges, and other characteristics.* (ISO/IEC 1993)
2. *A hardware or software component that connects two or more other components for the purpose of passing information from one to the other.* (ISO/IEC 1993)
3. *To connect two or more components for the purpose of passing information from one to the other.* (ISO/IEC/IEEE 2009)

Sources

From SEVOCAB (www.computer.org/sevocab):

(1) and (2) ISO/IEC. 1993. Information technology--Vocabulary--Part 1: Fundamental terms. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 2382-1:1993.

(3) ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

None.

Internal Coordinate Reference System (glossary)

A coordinate reference system having a datum specified with reference to the object itself.

Sources

ISO TC211. "Internal coordinate reference system." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 1004. Available at: <https://isotc211.geolexica.org/concepts/1004/>.

Discussion

None.

Internode Interaction (glossary)

a system resilience principle that states that every node, or element, of a system should be capable of communicating, cooperating, and collaborating with every other node. Jackson and Ferris (2013)

Source

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

In a resilience context the internode interaction principle supports the integrity attribute.

Interoperability (glossary)

(1) *Degree to which two or more systems, products or components can exchange information and use the information that has been exchanged.* (ISO/IEC 2009)

(2) *The ability for two or more ORBs to cooperate to deliver requests to the proper object.* (ISO/IEC 2003)

(3) *The capability to communicate, execute programs, and transfer data among various functional units in a manner that requires the user to have little or no knowledge of the unique characteristics of those units.* (ISO/IEC 1993)

Source

(1) ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

(2) ISO/IEC. 2003. *Information technology -- Open Distributed Processing -- Part 2: General Inter-ORB Protocol (GIOP)/Internet Inter-ORB Protocol (IIOP), 3.2.19*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 19500-2:2003.

(3) ISO/IEC. 1993. *Information technology--Vocabulary--Part 1: Fundamental terms*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 2382-1:1993.

Discussion

None.

Isomorphism (glossary)

(1) A formal or structural similarity between systems, patterns, processes, or principles occurring in different domains, such that knowledge about one may help illuminate or model the others. In general systems theory, isomorphies are recurring forms or processes found across otherwise different kinds of systems, supporting the search for domain-independent systems principles. (Bertalanffy 1968, Troncale 1982)

(2) A recurring system structure, pattern, process, or relationship that appears across different domains or scales and can therefore support cross-domain reasoning, analogy, modeling, and generalization in systems science and systems engineering. (SEBoK 2026)

Sources

- (1) Bertalanffy, L. von. (1968). General System Theory: Foundations, Development, Applications. George Braziller.
- (2) Troncale, L. (1982). Linkage propositions between systems isomorphies. In L. Troncale (Ed.), A General Survey of Systems Methodology: Vol. I. Conceptual and Mathematical Tools (pp. 27–38). Intersystems Publications.
- Giammarco, K., & Troncale, L. (2018). Modeling isomorphic systems processes using Monterey Phoenix. Systems, 6(2), 18.

Discussion

The classic grounding is Bertalanffy's argument that general system theory is possible because formally similar laws and structures recur across different scientific domains. Len Troncale then developed this further through the idea of systems isomorphies and later isomorphic systems processes, identifying recurring processes such as feedback, cycles, boundaries, hierarchy, networks, and self-organization.

Isomorphism is a related mathematical term. Isomorphism in systems science is often used more broadly than strict mathematical isomorphism.

Issue (glossary)

(1) *An area of concern that may impact the achievement of program/organizational objectives – a problem (existing), a risk (future uncertainty), or lack of information (existing).* (PSM 2010)

(2) *A concern that has a probability of occurrence equal to one, a consequence of occurrence greater than zero, and a time-frame in the future. An issue will occur and have a negative impact to the project, but it will not immediately occur.* (Conrow 2008)

Source

(1) PSM. 2010. *Practical Software and Systems Measurement: A Foundation for Objective Project Management*, version 4.0. Bethesda, MD, USA: Practical Software and Systems Measurement.

(2) Conrow, E. 2008. "Risk Analysis for Space Systems." Space Systems Engineering and Risk Management Symposium, 27-29 February, 2008, Los Angeles, CA, USA.

Discussion

Definition (2) defines issue in terms of probability of occurrence, consequence of occurrence, and time-frame; all of which are measurable.

Iteration (glossary)

The action or a process of iterating or repeating an action or process; a procedure in which repetition of a sequence of operations yields results successively closer to a desired result. (Oxford English Dictionary)

Sources

Oxford English Dictionary. s.v. "Iteration."

Discussion

This dictionary definition captures the essential idea of repeating an action in order to get closer to the answer we need.

Iteration is used as a generic term for successive application of a systems approach to the same problem situation, learning from each application, in order to progress towards greater stakeholder satisfaction.

K

Knowledge (glossary)

1. *acquaintance with facts, truths, or principles, as from study or investigation; general erudition: knowledge of many things.*
2. *acquaintance or familiarity gained by sight, experience, or report: a knowledge of human nature.*
3. *the fact or state of knowing; the perception of fact or truth; clear and certain mental apprehension.*

Source

Dictionary.com. 2012 "Knowledge." Available at <https://dictionary.reference.com/browse/knowledge> ^[1].

Discussion

None.

References

- [1] <https://dictionary.reference.com/browse/knowledge>
-

L

Land Cover (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Land Use (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Latent (glossary)

Description of unsafe conditions in a system that may be present for many years before they combine with local circumstances and active failures to penetrate the system's many layers of defenses. - adapted from (Reason 1997, 10)

Sources

Reason, J. 1997. *Managing the Risks of Organizational Accidents*. Aldershot, UK: Ashgate Publishing Limited.

Discussion

This term is applicable in the resilience context particularly with respect to the *drift correction design principle*.

Layer (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Leader (glossary)

A leader is one or more people who selects, equips, trains, and influences one or more follower(s) who have diverse gifts, abilities, and skills and focuses the follower(s) to the organization's mission and objectives causing the follower(s) to willingly and enthusiastically expend spiritual, emotional, and physical energy in a concerted coordinated effort to achieve the organizational mission and objectives.

Sources

Winston, B. E. and K. Patterson. 2006. "An Integrative Definition of Leadership." *International Journal of Leadership Studies*,: 1(2):6-66.

Discussion

Many definitions of leader exist such a the simple Merriam-Webster definition of "a person who leads as a guide, conductor." However, the definition provided above gives a more complete and elucidative definition that is more useful in the context of leaders of teams in organizations.

Lean Systems Engineering (LSE) (glossary)

(1) The application of lean principles, practices, and tools to SE to enhance the delivery of value to the system's stakeholders. (Oppenheim, Murman, and Secor 2011)

(2) Lean Systems Engineering (LSE) is the area of synergy between lean thinking and SE, with the goal to deliver the best life-cycle value for technically complex systems with minimal waste; under the lean SE philosophy, mission assurance is non-negotiable, and any task that is legitimately required for success must be included, but it should be well-planned and executed with minimal waste. (INCOSE 2012)

Sources

(1) Oppenheim, B., Murman, E., and Secor, D. 2011. "Lean Enablers for Systems Engineering." *Systems Engineering* 14(1): 29-55.

(2) INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

Discussion

None.

Lean Value Stream Mapping (glossary)

Map all end-to-end linked tasks, control/decision nodes, and the interconnecting flows necessary to realize customer value. During the mapping process, eliminate all non-value added activities, minimize all necessary non-value activities, and enable the remaining activities to flow without rework, backflow, or stopping. (Oppenheim 2011)

Sources

Oppenheim, B.W., 2011. *Lean for Systems Engineering With Lean Enablers for Systems Engineering*. Hoboken, NJ, USA: J. Wiley & Sons, Pg 17, 20.

Discussion

None.

Legacy System (glossary)

(1) *A system that has been in operation for a long time, and whose functions are too essential to be disrupted by upgrading or integration with another system. (Arbutus Software 2012)*

(2) *A legacy system is an old method, technology, computer system, or application program that continues to be used, typically because it still functions for the users' needs, even though newer technology or more efficient methods of performing a task are now available. A legacy system may include procedures or terminology which are no longer relevant in the current context, and may hinder or confuse understanding of the methods or technologies used. (Wikipedia Contributors 2012)*

Source

(1) Arbutus Software. "Legacy Systems." Accessed September 12, 2012. Available at: <https://www.arbutussoftware.com/data-source/legacy-systems/>.

(2) Wikipedia contributors. "Legacy system." *Wikipedia*, The Free Encyclopedia, https://en.wikipedia.org/w/index.php?title=Legacy_system&oldid=511592171 (accessed September 13, 2012).

Discussion

Seacord, R.C., D. Plakosh, and G.A. Lewis. 2003. *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Pearson Education.

discusses the difficulties of updating or modifying a legacy system's software

Leverage (glossary)

Leverage (of a solution) is the degree to which it satisfies the following two criteria (which are, in general, conflicting): (a) number of problem situations to which it applies, and (b) extent to which it provides the complete solution needed for the specific problem situations to which it applies. Leverage is the product of these two criteria. (Hybertson 2009).

Sources

Hybertson, D. 2009. *Model-Oriented Systems Engineering Science: A Unifying Framework for Traditional and Complex Systems*. Auerbach/CRC Press, Boca Raton, FL.

Discussion

Leverage describes the balance in system thinking between making specialised system to solve a single problem well vs making generalist system which can be applied to a wide range of problems (see Principles of Systems Thinking).

Life Cycle (glossary)

(1) The organized collection of activities, relationships and contracts which apply to a system-of-interest during its life. (Pyster 2009, 73)

(2) The evolution of a system, product, service, project or other human-made entity from conception through retirement. (ISO/IEC/IEEE 2015)

(3) Development (life) cycles start with user needs and end with system decommissioning and disposal. Project cycles contain three aspects: business, budget, and technical. (Mooz, Forsberg, Cotterman 2003, 259)

Source

(1) Pyster, A.(ed.). 2009. *Graduate Software Engineering 2009 (GSWE2009): Curriculum Guidelines for Graduate Degree Programs in Software Engineering*. Integrated Software & Systems Engineering Curriculum Project. Hoboken, NJ, USA: Stevens Institute of Technology, September 30, 2009.

(2) ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

(3) Mooz, H., K. Forsberg, H. Cotterman. 2003. *Communicating Project Management*. Hoboken, NJ, USA: John Wiley and Sons.

Discussion

For additional discussion of the different uses of "life cycle", see the Life Cycle Models article.

Life Cycle Cost (LCC) (glossary)

The total cost of implementation and ownership of a system over its useful life. It includes the cost of development, acquisition, operation, maintenance, support, and, where applicable, disposal. (Mooz, Forsberg, Cotterman 2003, 209)

Sources

Mooz, H., K. Forsberg, H. Cotterman. 2003. *Communicating Project Management*. Hoboken, NJ, USA: John Wiley and Sons.

Discussion

None.

Life Cycle Management (glossary)

The end-to-end management of the life cycle. (Created for SEBoK)

Sources

This definition was created for the SEBoK.

Discussion

See also Life Cycle (glossary).

Life Cycle Model (glossary)

A framework of processes and activities concerned with the Life Cycle (glossary) that may be organized into stages, which also acts as a common reference for communication and understanding (ISO/IEC 15288)

Sources

ISO/IEC. 2008. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2008.

Discussion

The life cycle models most commonly used in systems engineering are described in the Life Cycle Models article.

Life Cycle Process (glossary)

(1) *A process is a set of interrelated or interacting activities which transforms inputs into outputs (ISO/IEC 2008).*

(2) *Life cycle: evolution of a system, product, service, project or other human-made entity from conception through retirement. (ISO/IEC 2008)*

Sources

(1) and (2) ISO/IEC. 2008. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2008

Discussion

A life cycle process is a process that is applied during the application of the life cycle model. ISO 15288 (2008) lists and describes 25 life cycle processes for systems and software engineering. These are aggregated into four groups: agreement processes, technical processes, organizational project-enabling processes, and project processes.

Localized Capacity (glossary)

A design principle that allows a system to degrade gradually upon the encounter with a threat. It assumes that the capability of a system is concentrated in local nodes so that no more than one node fails at a time. -- adapted from (Woods 2006)

Sources

Woods, D.D. 2006. "Essential Characteristics of Resilience." In *Resilience Engineering: Concepts and Precepts*, edited by E. Hollnagel, Woods, D.D., and N. Leveson. Aldershot, UK: Ashgate Publishing Limited.

Discussion

This is a key design principle in the context of resilience to achieve the attribute of tolerance.

Location (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Location-Based Service (LBS) (glossary)

A service whose return or other property is dependent on the location of the client requesting the service or of some other thing, object or person.

Source

ISO TC211. "Location based service." In: ISO TC211 Multilingual Glossary. Version from 2020-06-02. Concept ID: 278. Available at: <https://isotc211.geolexica.org/concepts/278/> ^[1]

Discussion

None.

References

[1] <https://isotc211.geolexica.org/concepts/278/>

Logical Architecture (glossary)

The logical architecture of a system is composed of a set of related technical concepts and principles that support the logical operation of the system. It includes a functional architecture, a behavioral architecture, and a temporal architecture. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

The term "logical architecture" is used in ISO/IEC/IEEE 15288:2008 without normative reference.

For a full discussion of the role and importance of logical architecture in systems engineering see the Logical Architecture Model Development article.

Logistics (glossary)

The science of planning and implementing the acquisition and use of the resources necessary to sustain the operation of a system. (Cooke 1993)

Source

Cooke, W. 1993. "Integrated Logistics." *HUM - the Government Computer Magazine*. December 1993.

Discussion

None.

Loose Coupling (glossary)

The relationship among element in a system in which events in various elements can occur independently -- adapted from (Perrow 1999, 8)

Sources

Perrow, C. 1999. *Normal Accidents*. Princeton, NJ: Princeton University Press.

Discussion

Perrow is an authority on accidents in a system and the system characteristics that may cause them. This term is used in a resilience context.

Loss-Driven Systems Engineering (glossary)

An overarching system engineering process that addresses quality characteristics concerned with potential losses (such as resilience, safety, and security) in a holistic manner.

Sources

SEBoK original.

Discussion

Loss-driven systems engineering (LDSE) is an area of systems engineering that holistically addresses the quality characteristics concerned with loss (such as resilience, safety, and security).

M

Maintainability (glossary)

(1) *Maintainability is defined as the probability that a system or system element can be repaired in a defined environment with defined resources within a specified period of time. Increased maintainability implies shorter repair times.* (Created for SEBoK)

(2) *The American Society for Quality uses this definition. "Maintainability: The probability that a given maintenance action for an item under given usage conditions can be performed within a stated time interval when the maintenance is performed under stated conditions using stated procedures and resources."* (ASQ 2011) The definition above simplifies the ASQ definition.

Sources

(1) This definition was developed for the SEBoK.

(2) American Society for Quality. 2011. "Glossary: Maintainability." Accessed on 11 September 2012. Available at <https://asq.org/glossary/m.html>.

Discussion

None.

Maintenance (glossary)

The process of modifying a system or component after delivery to correct faults, improve performance or other attributes, or adapt to a changed environment.

Source

ISO/IEC/IEEE. 2010. *Systems and Software Engineering -- Vocabulary (SEVocab)* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronic Engineers (IEEE) 2009 ISO/IEC/IEEE 24765:2010 [database online]. Accessed on 11 September 2012. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Manpower (glossary)

Total supply of personnel available or engaged for a specific job or task. (WebFinance 2012)

Sources

WebFinance. 2012. "Glossary: Manpower." (online). Accessed on 11 September 2012. Available at: <https://www.businessdictionary.com/definition/manpower.html>.

Discussion

None.

Map (glossary)

A portrayal of geographic information as a digital image file suitable for display on a computer screen.

Sources

ISO TC211. "Map." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 282. Available at: <https://isotc211.geolexica.org/concepts/282/>.

Discussion

None.

Map Projection (glossary)

The coordinate conversion from an ellipsoidal coordinate system to a plane.

Sources

ISO TC211. "Map projection." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 618. Available at: <https://isotc211.geolexica.org/concepts/618/>.

Discussion

None.

Market Analysis (glossary)

Activities designed to determine the attractiveness of a market and understand its evolving opportunities and threats as they relate to the strengths and weaknesses of an organization. (NetMBA 2010)

Sources

NetMBA. 2010. "Market Analysis." (online). Florida, USA: NetMBA.com, Interent Center for Management and Business Administration, Inc. Accessed on 11 September 2012. Available at: <https://www.netmba.com/marketing/market/analysis/>

Discussion

None.

Mean Sea Level(glossary)

(1)*The average level of the surface of the sea over all stages of tide and seasonal variations.*

(2)*The average height of the surface of the sea at a tide station for all stages of the tide over a 19-year period, usually determined from hourly height readings measured from a fixed predetermined reference level.*

Note to definition (1): Mean sea level in a local context normally means mean sea level for the region calculated from observations at one or more points over a given period of time. Mean sea level in a global context differs from a global geoid by not more than 2 m.

Sources

ISO TC211. "Mean sea level." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 284. Available at: <https://isotc211.geolexica.org/concepts/284/>

ISO TC211. "Mean sea level." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 1562. Available at: <https://isotc211.geolexica.org/concepts/1562/>, referencing to IHO Hydrographic Dictionary S-32, Fifth Edition.

Discussion

None.

Measurable Concept (glossary)

Abstract relationship between attributes of entities and information needs. (ISO/IEC/IEEE 2007)

Sources

ISO/IEC/IEEE. 2007. *Systems and software engineering - Measurement process*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC/IEEE 15939:2007. 3.14

Discussion

None.

Measure (glossary)

When used as a noun:

(1) *A variable to which a value is assigned as the result of measurement. [The plural form “measures” is used to refer collectively to base measures, derived measures, and indicators.]* (PSM May 7, 2010; ISO/IEC/IEEE 2007)

When used as a verb:

(2) *To ascertain a quantitative or qualitative value for an entity based on a standard of comparison;* (PSM 2010)

(3) *To make a measurement.* (ISO/IEC 14598-1:1999).

Source

(1) PSM. 2010. *Practical Software and Systems Measurement: A Foundation for Objective Project Management*. version 4.0 ed. Bethesda, MD, USA: Practical Software and Systems Measurement (PSM).

ISO/IEC/IEEE. 2007. *Measurement Process*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical & Electronics Engineers (IEEE), ISO/IEC/IEEE 15939.

(2) PSM. 2010. *Practical Software and Systems Measurement: A Foundation for Objective Project Management*. version 4.0 ed. Bethesda, MD, USA: Practical Software and Systems Measurement (PSM).

(3) ISO/IEC. 1999. "Information technology -- Software product evaluation -- Part 1: General overview." Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 14598-1:1999.

Discussion

None.

Measure of Effectiveness (MoE) (glossary)

The metrics by which an acquirer will measure satisfaction with products produced by the technical effort. (IEEE 2005)

Sources

IEEE. 2005. *IEEE Standard for Application and Management of the Systems Engineering Process*. New York, NY, USA: Institute for Electrical and Electronics Engineers (IEEE). IEEE 1220-2005.

Discussion

None.

Measure of Performance (MoP) (glossary)

An engineering performance measure that provides design requirements that are necessary to satisfy an MOE (measure of effectiveness). (IEEE 2005)

Sources

IEEE. 2005. *IEEE Standard for Application and Management of the Systems Engineering Process*. New York, NY, USA: Institute for Electrical and Electronics Engineers (IEEE). IEEE 1220-2005.

Discussion

None.

Measurement (glossary)

A set of operations having the object of determining a value of a measure. (PSM 2010)

Source

PSM. 2010. *Practical Software and Systems Measurement: A Foundation for Objective Project Management*. version 4.0 ed. Bethesda, MD, USA: Practical Software and Systems Measurement (PSM).

Discussion

None.

Meta-model (glossary)

A model about a model.

Sources

Cited in Haskins, C. 2008. "Systems engineering analyzed, synthesized, and applied to sustainable industrial park development" (PhD diss., Norwegian University of Science and Technology, 2008).

Discussion

There are also alternative definitions of "meta-model" fashioned for specific applications of its use that have been defined in standards below:

(1) *A logical information model that specifies the modeling elements used within another (or the same) modeling notation. (IEEE 1175.1-2002 (R2007) IEEE Guide for CASE Tool Interconnections-Classification and Description, 3.8)*

(2) *A metamodel Vm for a subset of IDEFObject is a view of the constructs in the subset that is expressed using those constructs such that there exists a valid instance of Vm that is a description of Vm itself. (IEEE 1320.2-1998 (R2004) IEEE Standard for Conceptual Modeling Language Syntax and Semantics for IDEF1X97 (IDEFObject), 3.1.111)*

(3) *A model containing detailed definitions of the meta-entities, meta-relationships and meta-attributes whose instances appear in the model section of a CDIF transfer. (ISO/IEC 15474-1:2002 Information technology -- CDIF framework -- Part 1: Overview, 4.2)*

(4) *Specification of the concepts, relationships and rules that are used to define a methodology. (ISO/IEC 24744:2007 Software Engineering--Metamodel for Development Methodologies, 3.4)*

(5) *Model defining the concepts and their relations for some modeling notation. (ISO/IEC 15909-2:2011 Software and system engineering -- High-level Petri nets -- Part 2: Transfer format, 4.1.6)*

Works Cited

IEEE. 2002. *IEEE Guide for CASE Tool Interconnections-Classification and Description*. New York, NY, USA: Institute for Electrical and Electronics Engineers (IEEE). IEEE 1175.1-2002.

IEEE. 1998. *IEEE Standard for Conceptual Modeling Language Syntax and Semantics for IDEF1X97 (IDEFObject)*. New York, NY, USA: Institute for Electrical and Electronics Engineers (IEEE). IEEE 1320.2-1998

ISO/IEC. 2002. *Information technology -- CDIF framework -- Part 1: Overview*. Geneva, Switzerland: the International Organization for Standardization (ISO)/the International Electrotechnical Commission (IEC). ISO/IEC

15474-1:2002

ISO/IEC. 2007. *Software Engineering--Metamodel for Development Methodologies*. Geneva, Switzerland: the International Organization for Standardization (ISO)/the International Electrotechnical Commission (IEC). ISO/IEC 24744:2007

ISO/IEC. 2011. *Software and system engineering-- High-level Petri nets -- Part 2: Transfer format*. Geneva, Switzerland: the International Organization for Standardization (ISO)/the International Electrotechnical Commission (IEC). ISO/IEC 15909-2:2011

Metadata (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Metric (glossary)

Quantitative measure of the degree to which a system, component, or process possesses as given attribute." (ISO/IEC/IEEE 2010)

Sources

ISO/IEC/IEEE. 2010. *Systems and Software Engineering -- Vocabulary (SEVocab)* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronic Engineers (IEEE) 2009 ISO/IEC/IEEE 24765:2010 [database online]. Accessed on 11 September 2012. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Milestone (glossary)

- (1) *A significant point or event in the project.* (PMI 2008)
- (2) *A scheduled event used to measure progress.* (IEEE 1998)

Source

- (1) PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 4th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- (2) IEEE. 1998. *IEEE Standard for Software Project Management Plans*. New York, NY, USA: Institute of Electrical and Electronics Engineers Inc.

Discussion

None.

Mission (glossary)

The mission of a system is the top-level function of the system; the one that synthesizes all transformation of all inputs and solicitations into outputs and reactions. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

For a full discussion of the role and importance of a clear understanding of mission needs in systems engineering see the Business or Mission Analysis article.

Mission Analysis (glossary)

Examination and definition of the primary and secondary purposes of a system. - modified from (Blanchard and Fabryky 2011)

Sources

Modified from:

Blanchard, B. S., and W. J. Fabrycky. 2011. *Systems Engineering and Analysis*, 5th ed. Prentice-Hall International series in Industrial and Systems Engineering. Englewood Cliffs, NJ, USA: Prentice-Hall.

Discussion

Though Blanchard and Fabrycky (2011) do not define "mission analysis", their discussion of "mission definition" is relevant to mission analysis. Effectively, mission analysis is a set of activities designed to help answer the questions (Blanchard and Fabryky 2011):

- What is the system to accomplish? and
- How will the system accomplish its objectives?

The purpose of MA is to understand a problem or opportunity, analyze the solution space, and initiate the life cycle of a potential solution that could answer the problem or take advantage of an opportunity.

Mission Engineering (glossary)

Mission Engineering describes the application of systems engineering to the planning, analysis, and designing of missions, where the mission is the system of interest. (SEBoK Original)

Source

SEBoK original.

Discussion

None.

Model (glossary)

- (1) *A physical, mathematical, or otherwise logical representation of a system, entity, phenomenon, or process.* (DoD 1998)
- (2) *A representation of one or more concepts that may be realized in the physical world.* (Friedenthal, Moore, Steiner 2009)
- (3) *A simplified representation of a system at some particular point in time or space intended to promote understanding of the real system.* (Bellinger 2004)
- (4) *An abstraction of a system, aimed at understanding, communicating, explaining, or designing aspects of interest of that system* (Dori 2002)
- (5) *A selective representation of some system whose form and content are chosen based on a specific set of concerns. The model is related to the system by an explicit or implicit mapping.* (Object Management Group 2010)

Source

- (1) DoD. 1998. "DoD Modeling and Simulation (M&S) Glossary" in *DoD Manual 5000.59-M*. Arlington, VA, USA: US Department of Defense. January. P2.13.22. Available at <https://www.dtic.mil/whs/directives/corres/pdf/500059m.pdf>
- (2) Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.
- (3) Bellinger, G. 2004. *Modeling & Simulation: An Introduction*. Accessed on 11 September 2012. Available at <https://www.systems-thinking.org/modsim/modsim.htm>.
- (4) Dori, D. 2002. *Object-Process Methodology: A Holistic System Paradigm*. New York, NY, USA: Springer.
- (5) Object Management Group. 2010. *MDA Foundation Model*. OMG document number ORMSC/2010-09-06.

Discussion

See also model management, model validation, Model-Based Systems Engineering (MBSE) (glossary), modeling language and modeling tool.

Model Management (glossary)

The process of controlling models and the modeling environment including version, configuration, and variant management. (Friedenthal, et al 2009)

Source(s)

Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.

Discussion

None.

Model Transformation (glossary)

A mapping between two modeling languages that enables a model expressed in one modeling language to be expressed in whole or in part in the other modeling language. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

None.

Model Validation (glossary)

The process of ensuring the model correctly represents the domain or system-of-interest. (Friedenthal 2009)

Sources

Friedenthal, S., A. Moore, and R. Steiner. 2009. A Practical Guide to SysML: The Systems Modeling Language. Needham, MA: OMG Press.

Discussion

None.

Model-Based Systems Engineering (MBSE) (glossary)

The formalized application of modeling to support system requirements, design, analysis, verification and validation activities beginning in the conceptual design phase and continuing throughout development and later life cycle phases. (INCOSE 2007)

Sources

INCOSE. 2007. Systems Engineering Vision 2020. INCOSE-TP-2004-004-02 September, 2007.

Discussion

None.

Modeling Language (glossary)

A language in which a model is expressed. It includes syntax, rules and its semantics. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

See the knowledge area Representing Systems with Models for discussions on modeling systems.

Modeling Tool (glossary)

A tool such as a software application that is used to develop models. (Friedenthal 2009)

Sources

Friedenthal, S., A. Moore, R. Steiner, and M. Kaufman. 2012. *A Practical Guide to SysML: The Systems Modeling Language*, 2nd Edition. Needham, MA, USA: OMG Press.

Discussion

None.

Modularity (glossary)

(1) *Degree to which a system or computer program is composed of discrete components such that a change to one component has minimal impact on other components. (ISO/IEC 2011)*

(2) Software attributes that provide a structure of highly independent components. *(ISO/IEC/IEEE 2010)*

(3) In a resilience context modularity is a system resilience principle that states that the functionality of a system should be distributed through various nodes of that system so that if a single node is damaged or destroyed, the remaining nodes will continue to function. Jackson (2016) Modularity is a component principle in the tolerance attribute grouping. Jackson (2016)

Sources

(1) ISO/IEC. 2011. *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/ISO 25010:2011.

(2) ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

(3) Jackson, Scott. 2016. "Principles for Resilient Design - A Guide for Understanding and Implementation." In IRGC Resource Guide on Resilience, edited by I. Linkov. University of Lausanne, Switzerland: International Risk Governance Council (IRGC).

Discussion

also called localized capacity

Motion (glossary)

The change in the position of an object over time, represented by change of coordinate values with respect to a particular reference frame.

Example: This may be the motion of the position sensor mounted on a vehicle or other platform or motion of an object being tracked by a positioning system.

Sources

ISO TC211. "Motion." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 301. Available at: <https://isotc211.geolexica.org/concepts/301/>.

Discussion

None.

N

Natural System (glossary)

An open system whose elements, boundary, and relationships exist independently of human control.
(Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

This definition has been created to reflect the usage of this term in the SEBoK (See articles Engineered Systems and What is a System?). Examples of natural systems include: real number system, solar system, and planetary atmosphere circulation systems.

Nautical Chart (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Navigation (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Network (glossary)

An arrangement of nodes and interconnecting branches. (ISO/IEC 1993)

Source

ISO/IEC. 1993. *Information Technology -- Vocabulary -- Part 1: Fundamental Terms*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 2383-1: 1993.

Discussion

None.

Neutral State (glossary)

A resilience design principle that states that a system should be put into neutral if possible following a disruption. (Madni and Jackson 2009)

According to Jackson and Ferris (2013) neutral state is a component principle in the tolerance attribute grouping.

Sources

Madni, A. and S. Jackson. 2009. "Towards a conceptual framework for resilience engineering." *IEEE Systems Journal* 3(2):181-191.

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

This is a key resilience design principle.

Node (glossary)

An element of a system. (Created for SEBoK)

Sources

This definition was created for the SEBoK.

Discussion

There are also alternative definitions of "node" fashioned for specific applications of its use that have been defined in standards below:

(1) *In a diagram, a point, circle, or other geometric figure used to represent a state, event, or other item of interest.* (ISO/IEC/IEEE 24765:2010 Systems and software engineering--Vocabulary)

(2) *Configuration of engineering objects forming a single unit for the purpose of location in space, and which embodies a set of processing, storage and communication functions.* (ISO/IEC 10746-3:2009 Information technology -- Open Distributed Processing -- Reference Model: Architecture, 8.1.7)

(3) *Modeled function located within the hierarchical graph structure of an IDEF0 model by its designated node number.* (IEEE 1320.1-1998 (R2004) IEEE Standard for Functional Modeling Language - Syntax and Semantics for IDEF0, 2.1.80)

(4) *One of the defining points of a schedule network; a junction point joined to some or all of the other dependency lines.* (PMI 2008)

(5) *Vertex of a net graph, i.e., a place or a transition.* (ISO/IEC 15909-1:2004 Software and system engineering -- High-level Petri nets -- Part 1: Concepts, definitions and graphical notation, 2.1.16.2)

Works Cited

ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

ISO/IEC. 2009. *Information Technology -- Open Distributed Processing -- Reference Model: Architecture*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC).

ISO/IEC 10746-3:2009.

IEEE. 1998. *IEEE Standard for Functional Modeling Language - Syntax and Semantics for IDEF0*. New York, NY, USA: Institute for Electrical and Electronics Engineers (IEEE). IEEE 1320.1-1998.

ISO/IEC. 2004. *Software and system engineering -- High-level Petri nets -- Part 1: Concepts, definitions and graphical notation*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 15909-1:2004.

PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 4th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Non-Functional Requirements (glossary)

Quality attributes or characteristics that are desired in a system, that define how a system is supposed to be. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

Non-functional requirements contrast with functional requirements that define what the system must be able to do or perform. Typical non-functional requirements include availability, reliability, maintainability, safety, and security.

O

Occupational Health (glossary)

A robust process by which to design and develop systems that effectively and affordably integrate human capabilities and limitations. (USAF 2009)

Source(s)

USAF. 2009. *Air Force Human Systems Integration Handbook*. Brooks City-Base, TX, USA: US Air Force Directorate of Human Performance Integration. AFD-090121-054. Available at: <https://www.wpafb.af.mil/shared/media/document/AFD-090121-054.pdf>

Discussion

There has not been a standard usage for the term "Occupational Health". The term may also be seen as "human systems integration", "human-systems integration", "human system integration", and "human-system integration."

Ontology (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Open System (glossary)

A system which exchanges inputs and outputs with its environment. (von Bertalanffy 1968)

Source

von Bertalanffy, L. 1968. *General system theory: Foundations, development, applications*, Revised ed. New York, NY: Braziller.

Discussion

Open system theory considers an organism as a complex entity composed of many parts with an overall integrity, co-existing in an environment. In an open system the organisms structure is maintained, or adapts, through a continual exchange of energy and information with its environment.

Operational (glossary)

Of or relating to the routine functioning and activities of a business or an organization. (American Heritage Dictionary 2009)

Sources

American Heritage Dictionary, v2.1.3. 2009. "Operational."

Discussion

None.

Operational Capability (glossary)

The ability of a system to perform in the intended operational environment, particularly with respect to meeting the requirements of its stakeholders. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

See the article Enterprise Systems Engineering Background for discussions on operational capability of an enterprise.

Operational Concept (glossary)

Operational Concept: verbal and graphic statement of an organization's assumptions or intent in regard to an operation or series of operations of a system or a related set of systems. (ANSI/AIAA G-043-199)

Source

ISO/IEC/IEEE. 2011. Systems and software engineering - Requirements engineering. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission/ Institute of Electrical and Electronics Engineers (IEEE), (IEC), ISO/IEC/IEEE 29148.

Discussion

The operational concept is designed to give an overall picture of the operations using one or more specific systems, or set of related systems, in the organization's operational environment from the users' and operators' perspective. See also concept of operations.

Operational Environment (glossary)

The environment in which systems are deployed. The problem or opportunity in response to which the system has been developed, exists in this environment. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

The operational environment is a significant factor in defining the needed system capabilities, desired stakeholder outcomes and benefits, and constraints.

Operational Mode (glossary)

An operational situation/configuration of a system characterized by its active functions. Inside an operational mode the system can perform specific operational scenarios, and so activate the corresponding functions. (same as state) (Faisandier 2012)

Source(s)

Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Discussion

None.

Operational Scenario (glossary)

(1) *Description of an imagined sequence of events that includes the interaction of the product or service with its environment and users, as well as interaction among its product or service components. (ISO/IEC 2011)*

(2) *A set of actions or functions representing the dynamic of exchanges between the functions allowing the system to achieve a mission or a service. (Created for SEBoK)*

(3) *Stories which describe the expected utilization of the future system in terms of actions. (Created for SEBoK)*

Source

(1) ISO/IEC/IEEE. 2011. Systems and software engineering - Requirements engineering. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission/ Institute of Electrical and Electronics Engineers (IEEE), (IEC), ISO/IEC/IEEE 29148.

(2) and (3) These definitions were developed for the SEBoK.

Discussion

Operational scenarios are used to evaluate the requirements and design of the system and to verify and validate the system.

Operations Research (glossary)

***Operations Research (OR)** considers the use of technology by an organization. It is based on mathematical modeling and statistical analysis to optimize decisions on the deployment of the resources under an organizations control. It arises from military planning techniques developed during World War 2.*

***Operations Research and Management Science (ORMS)** was formalized in 1950 by **Ackoff** and **Churchman** applying the ideas and techniques of OR to organizations and organizational decisions . (Churchman et al, 1950)*

Source

Churchman, C.W., R.L. Ackoff, and E.L. Arnoff. 1950. *Introduction to Operations Research*. New York, NY, USA: Wiley.

Discussion

Operations research (OR) and operations research and management science (ORMS) are used interchangeably in many sources. The term operations analysis is used in the UK, especially in defense.

Opportunity (glossary)

(1) *an appropriate or favorable time or occasion; a situation or condition favorable for attainment of a goal; a good position, chance, or prospect, as for advancement or success.* (Dictionary.com 2012)

(2) *an outcome better than expected; a less negative outcome; a positive outcome.* (Conrow 2003)

Sources

(1) Dictionary.com. 2012. "Opportunity." Online dictionary.

(2) Conrow, E. 2003. *Effective Risk Management: Some Keys to Success*. 2nd ed. Reston, VA, USA: American Institute of Aeronautics and Astronautics (AIAA).

Discussion

(1) is a dictionary definition which associates opportunity with a situation in which a system intervention might lead to additional stakeholder benefit. This definition applies to the front end of the Systems Approach, and is used to broaden our view of Problem Situations.

2) Comes from Risk Management which uses opportunity as the likelihood and impact of a beneficial outcome. In risk management a variety of different classes of outcomes can be termed an opportunity, which may not be uniquely specified. Risk can be defined in terms of specific bounds on probability ($0 < P < 1$), consequence (> 0), and time-frame (future), as can issue [$P = 1$, $C > 0$, and time-frame (future)] and problem [$P = 1$, $C > 0$, and time-frame (present)]. However, similar definitions are unavailable for opportunity because of the inability to uniquely specify whether the case of $P = 1$ is included or excluded, consequence (can be positive or negative) and time-frame (present, future, with no analog to issue and problem).

Works Cited

Conrow, E. 2008. "Risk Analysis for Space Systems". Space Systems Engineering and Risk Management Symposium, 27-29 February, 2008, Los Angeles, CA, USA.

Organization (glossary)

A group of people and facilities with an arrangement of responsibilities, authorities and relationships.
(INCOSE 2011)

Sources

INCOSE. 2012. *Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE). INCOSE-TP-2003-002-03.2.2. p. 361.

Discussion

The INCOSE *Systems Engineering Handbook* is an established systems engineering source. The current INCOSE *Systems Engineering Handbook* version 3.2.1 has this definition as, "Person or a group of people and facilities with an arrangement of responsibilities, authorities and relationships." The earlier version 3 had, "A group of people and facilities with an arrangement of responsibilities, authorities and relationships." Other definitions of "organization" researched all relate to a group of people, not a single person, so the current version 3.2.1 is not cited. The version 3 definition is used since it is succinct, clear, and consistent with other sources.

Organizational (glossary)

Of or pertaining to an organization.

Sources

WebFinance. 2012. "Organizational" (online). Washington, DC, USA: WebFinance, Inc. Available at: <https://dictionary.reference.com/browse/organization>

Discussion

Corresponding adjective for Organization (glossary).

Organizational Capability (glossary)

The capability of an organization to perform systems engineering that contributes towards organizational value and purpose. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

None.

Orthoimage (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Outcome (glossary)

a) Process outcomes represent measurable and observable results or change of state achieved by executing a process. (Created for SEBoK)

b) Significant change of state or the meeting of specified constraints. e.g., requirements or goals. (ISO/IEC/IEEE DIS)

c) End result or consequence of a process or project. (PMI, 2021)

Source

a) This definition was developed for the SEBoK.

b) ISO/IEC/IEEE. DIS. *Systems and software engineering — Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765 DIS (Draft International Standard).

c) PMI. 2021. *A Guide to the Project Management Body of Knowledge, 7th Edition*. Project Management Institute.

Discussion

This definition was developed to support Process Description in the SEBoK.

Output (glossary)

When used as a noun:

- (1) *Data transmitted to an external destination.* (ISO/IEC 2009, 1)
- (2) *Pertaining to a device, process, or channel involved in transmitting data to an external destination.* (ISO/IEC 2009, 1)
- (3) *Pertaining to data transmitted to an external destination.* (ISO/IEC 2009, 1)

When used as a verb:

- (4) *To transmit data to an external destination.* (ISO/IEC 2009, 1)

Source

- (1) ISO/IEEE. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.
- (2) ISO/IEEE. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.
- (3) ISO/IEEE. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.
- (4) ISO/IEEE. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

P

Paradigm (glossary)

A view of the world that may or may not conflict with accepted scientific principles. -- adapted from (Vaughn 1997, 1996)

Sources

Vaughn, Diane 1997. *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. Chicago: University of Chicago Press. Original edition, 1996.

Discussion

Although the standard definition of *paradigm* is simply *example*, the word has come to mean in Vaughn's sense a way of thinking that may be positive or negative with respect to the Culture involved in the development and operation of a system, such as the Challenger. In a positive sense *paradigm* can mean the way of thinking about the systems engineering methodology itself.

Pattern (glossary)

(1) *An expression of an observed regularity.* (Alexander 1979)

(2) *A representation of similarities in a set or class of problems, solutions, or systems.* (Alexander 1979)

(3) *Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.* (Alexander 1979)

Sources

(1) - (3) Alexander, C. 1979. *The Timeless Way of Building*. New York, NY, USA: Oxford University Press.

Discussion

A full discussion of patterns and how they relate to systems thinking can be found in Patterns of Systems Thinking.

Personnel (glossary)

An individual expected to perform duties on behalf of the organization, including officers, employees and contractors. (ISO/IEC 2006)

Sources

ISO/IEC. 2006. *Information technology -- Software asset management -- Part 1: Process*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 19770-1:2006.

Discussion

None.

Physical Architecture (glossary)

(1) *A physical architecture is an arrangement of physical elements (system elements and physical interfaces) which provides the design solution for a product, service, or enterprise, and is intended to satisfy logical architecture elements and system requirements. It is implementable through technologies. (ISO/IEC 2010)*

(2) *An arrangement of physical elements which provides the design solution for a consumer product or life-cycle process intended to satisfy the requirements of the functional architecture and the requirement baseline. (ISO/IEC 2007)*

Source

(1) Adapted from ISO/IEC. 2010. *Systems and Software Engineering, Part 1: Guide for Life Cycle Management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 24748-1:2010.

(2) ISO/IEC. 2007. *Systems and Software Engineering -- Recommended Practice for Architectural Description of Software-Intensive Systems*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC FDIS 42010:2007.

Discussion

Definition (1) comes from the terms "design architecture" provided in ISO/IEC/IEEE 24748 - 4. It is adapted here to be consistent current terminology, in particular with logical architecture.

Definition (2) comes from ISO/IEC/IEEE 42010:2007 that is replaced by version 2011 in which this definition has been withdrawn.

For a full discussion of the role and importance of physical architecture in systems engineering see the Physical Architecture Model Development article.

Physical Interface (glossary)

A physical interface is a system element that binds physically two system elements. (Faisandier 2012)

Sources

Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Discussion

None.

Physical Model (glossary)

A model made of tangible components. (Sage and Rouse 2009)

Sources

Sage, A. and W. Rouse. 2009. *Handbook of Systems Engineering and Management*. Hoboken, NJ, USA: J. Wiley and Sons.

Discussion

None.

Physical redundancy (glossary)

a system resilience principle that states that the system should possess two or more independent and identical legs to perform critical tasks. Jackson (2016)

Source

Jackson, Scott. 2016. "Principles for Resilient Design - A Guide for Understanding and Implementation." In IRGC Resource Guide on Resilience, edited by I. Linkov. University of Lausanne, Switzerland: International Risk Governance Council (IRGC).

Discussion

Also called design redundancy by Leveson (1995)

Plan (glossary)

Information item that presents a systematic course of action for achieving a declared purpose, including when, how, and by whom specific activities are to be performed. (ISO/IEC/IEEE 2011)

Source

ISO/IEC/IEEE. 2011. "Section 5.16," in *Systems and software engineering - Content of the life-cycle information products (documentation)*. Geneva, Switzerland: International Organisation for Standardisation (ISO) / International Electrotechnical Commissions (IEC) / Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15289:2011.

Discussion

None.

Pluralist (glossary)

A problem situation involving participants in which "although their basic interests are compatible, they do not share the same values and beliefs. Space needs to be made available within which debate, disagreement, even conflict, can take place. If this is done, and all feel they have been involved in decision-making, then accommodations and compromises can be found. Participants will come to agree, at least temporarily, on productive ways forward and will act accordingly." (Jackson 2003, 19)

Sources

Jackson, M. 2003. *Systems Thinking: Creating Holisms for Managers*. Chichester, UK: Wiley.

Discussion

None.

Portfolio (glossary)

A collection of projects or programs and other work that are grouped together to facilitate effective management of that work to meet strategic business objectives. The projects or programs of the portfolio may not necessarily be interdependent or directly related. (PMI 2008)

Sources

PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 4th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Discussion

See also project and program.

Portfolio Management (glossary)

The centralized organization and administration of one or more collections of projects, which includes identifying, prioritizing, authorizing, managing, and controlling projects, programs, and other related work, to achieve specific strategic business objectives. (PMI 2008)

Source

PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 4th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Discussion

None.

Portrayal (glossary)

The presentation of information to humans.

Sources

ISO TC211. "Portrayal." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 342. Available at: <https://isotc211.geolexica.org/concepts/342/>.

Discussion

None.

Positional accuracy (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Postal address type (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Postmodernist (glossary)

Postmodernists believe that we live in a world of multiple truths that give rise to incommensurable interpretations of reality and think that we should promote difference rather than seek to subsume it in the quest for agreement and consensus. (Jackson 2003)

Sources

Jackson, M. 2003. *Systems Thinking: Creating Holisms for Managers*. Chichester, UK: Wiley.

Discussion

Postmodernism is defined here as it is used to describe an approach to systems in which all notions of problem or solution are rejected in favor of creating an environment in which people can be innovative and creative in response to changing situations.

Praxis (glossary)

(1) Praxis is the process of translating an idea into action (WordNet).

(2) In praxis there can be no prior knowledge of the right means by which we realize the end in a particular situation. For the end itself is only specified in deliberating about the means appropriate to a particular situation." (Bernstein 1983).

Sources

(1) WordNet 3.1. Accessed on 1 August 2012. Available at <https://wordnetweb.princeton.edu>.

(2) Bernstein, R.J. (1983). *Beyond Objectivism and Relativism: Science, hermeneutics and praxis*. Oxford: Basil Blackwell.

Discussion

The term Praxis has meaning in politic, education, philosophy and spiritualism. It appears in the writing of Aristotle and Marx, both of whom relate it to the translation of philosophical ideas into real-world action. Both also associate this in their different ways with moral notions of freedom of action and doing good by taking the right actions.

Praxis is also associated with a way of thinking in which practical considerations drive the development of theoretic approaches and visa versa (see 2 above).

Members of the Systems Science community have adopted the term to characterize work to develop an integrated approach to practice, which provides a framework for the use of appropriate elements of available theory as needed and helps to drive the development of that theory to resolve practical concerns as they arise.

Preliminary Design Review (PDR) (glossary)

A review conducted to evaluate the progress, technical adequacy, and risk resolution of the selected design approach for one or more configuration items; to determine each design's compatibility with the requirements for the configuration item; to evaluate the degree of definition and assess the technical risk associated with the selected manufacturing methods and processes; to establish the existence and compatibility of the physical and functional interfaces among the configuration items and other items of equipment, facilities, software and personnel; and, as applicable, to evaluate the preliminary operational and support documents. (ISO/IEC/IEEE 24765 2009)

Source

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

None.

Principle (glossary)

- 1) *a fundamental truth or proposition that serves as the foundation for a system of belief or behaviour or for a chain of reasoning. (Oxford Dictionaries Online 2012)*
- 2) *a rule or belief governing one's behavior. (Oxford Dictionaries Online 2012)*

Source

(1) and (2) Oxford Dictionaries Online S.v. "Principle" Accessed February 20, 2012. Available at <https://oxforddictionaries.com/definition/concept>.

Discussion

A principle is a rule of conduct or behavior. To take this further, a principle is a "basic generalization that is accepted as true and that can be used as a basis for reasoning or conduct." (WordWeb.com) A principle can also be thought of as a "basic truth or law or assumption." [ibid.]

Principles depend on concepts in order to state a "truth." Hence, principles and concepts go hand in hand; principles cannot exist without concepts and concepts are not very useful without principles to help guide the proper way to act (Lawson and Martin 2008).

Work Cited

Lawson, H., and J.N. Martin. 2008. "On the Use of Concepts and Principles for Improving Systems Engineering Practice". Proceedings of the 18th Annual International Council on Systems Engineering (INCOSE) International Symposium, 5-19 June 2008, Utrecht, The Netherlands.

Problem (glossary)

A concern that has a probability of occurrence equal to one, a consequence of occurrence greater than zero, and a time-frame that is current. A problem is an event that has occurred and has a negative impact to the project. (Conrow 2008)

Source

Conrow, E. 2008. "Risk Analysis for Space Systems". Space Systems Engineering and Risk Management Symposium, 27-29 February, 2008, Los Angeles, CA, USA.

Discussion

Defines problem in terms of probability of occurrence, consequence of occurrence, and time-frame; all of which are measurable.

Process (glossary)

(1) A process is a set of interrelated or interacting activities which transforms inputs into outputs (ISO 9000:2005)

(2) A set of activities, methods, practices, and transformations that people use to develop and maintain systems and associated products. (SEI 2007)

(3) Process Purpose: high level objective of performing the process and the likely outcomes of effective implementation of the process. NOTE The implementation of the process should provide tangible benefits to the stakeholders. (ISO/IEC 12207:2008)

(4) Process Outcome: observable result of the successful achievement of the process purpose (ISO/IEC 12207:2008)

Sources

(1) ISO/IEC. 2000. *International standards for quality management*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO 9000:2000.

(2) SEI. 2007. *Capability maturity model integrated (CMMI) for development*, version 1.2, Pittsburg, PA, USA: Software Engineering Institute (SEI)/Carnegie Mellon University (CMU).

(3) and (4) ISO/IEEE. 2008. *Systems and Software Engineering — Software Life Cycle Processes*. Geneva, Switzerland: International Organization for Standards (ISO)/Institute of Electrical & Electronics Engineers (IEEE) Computer Society, ISO/IEEE 12207:2008(E).

Discussion

These definitions are taken from the INCOSE Systems Engineering Handbook, and are based on standard ISO terminology.

Procurement (glossary)

The overarching function that describes the activities and processes to acquire goods and services.
(Purchasing Insight 2012)

Sources

Purchasing Insight. 2012. "Definition of Procurement – Procurement vs Purchasing." (online). New York, NY, USA: Purchasing Insight, Purchase to Pay, Purchasing & Procurement Process, Electronic Invoicing. Available at: <https://purchasinginsight.com/resources/what-is/definition-of-procurement-procurement-vs-purchasing/>

Discussion

"The terms "Purchasing" and "Procurement" are often used, incorrectly, interchangeably. Importantly, and distinct from "purchasing", procurement involves the activities involved in establishing fundamental requirements, sourcing activities such as market research and vendor evaluation and negotiation of contracts. It can also include the purchasing activities required to order and receive goods."

Product (glossary)

- (1) *A system considered from the point of view of a physical "system end product" (ANSI/EIA 2003) made of system elements that may include hardware, software, infrastructure and support services. The people and organizational aspects of the "whole system" of which the "product system" forms a part have to be considered in the design, but are provided by another organization.* (INCOSE UK Chapter 2010)
- (2) *An artifact that is produced, is quantifiable, and can be either an end item in itself or a component item.* (PMI 2008)

Source

- (1) INCOSE UK Chapter. 2010. *Applying Systems Engineering to In-Service Systems: Supplementary Guidance to the INCOSE Systems Engineering Handbook*. Version 3.2, issue 1.0. Foresgate, UK: International Council on Systems Engineering (INCOSE) UK Chapter: 10, 13, 23.
- (2) PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 4th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Discussion

This definition is a systems engineering definition, which is consistent with the concept of a system-of-interest (SoI) focused on the product system to be delivered to an acquirer who will use it to help deliver user outcomes.

Product System (glossary)

- (1) *A system context for which the system-of-interest (SoI) is a product.* (Created for SEBoK)
- (2) *A system considered from the point of view of a physical “system end product” (ANSI/EIA 2003) made of system elements that may include hardware, software, infrastructure and support services. The people and organizational aspects of the “whole system” of which the “product system” forms a part have to be considered in the design, but are provided by another organization.* (ISO/IEC/IEEE 15288 2008)

Sources

- (1) This definition was developed for the SEBoK.
- (2) This definition was extended from: ISO/IEC. 2008. Systems and Software Engineering -- System Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions. ISO/IEC/IEEE 15288:2008.

Discussion

Definition (1) is has been created for the SEBoK to distinguish from a product (glossary) from a product system.

Definition (2) is a systems engineering definition, which is consistent with the concept of a SoI focused on the product system to be delivered to an acquirer who will use it to help deliver user outcomes.

See the Engineered Systems discussion.

Program (glossary)

A group of related projects managed in a coordinated way to obtain benefits and control not available from managing them individually. Programs may include elements of related work outside of the scope of the discrete projects in the program.(PMI 2008)

Source

PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 4th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Discussion

The term "program" is also used as another name for a large project. "Program management" is sometimes used to denote management of a portfolio of projects within a business unit.

Program Management (glossary)

The application of knowledge, skills, tools, and techniques to a program to meet the program requirements and to obtain benefits and control not available by managing projects individually. (PMI 2013)

Source

PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

Discussion

See also program.

Project (glossary)

- (1) *A temporary endeavor undertaken to create a unique product, service, or result. (PMI 2013)*
- (2) *a development effort consisting of both technical and management activities for the purpose of engineering a system. (extract from (ANSI/EIA 2003))*
- (3) *endeavour with defined start and finish criteria undertaken to create a product or service in accordance with specified resources and requirements. (ISO/IEC/IEEE 2015)*

Source

- (1) PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- (2) ANSI/EIA. 2003. *Processes for Engineering a System*. Philadelphia, PA, USA: American National Standards Institute (ANSI)/Electronic Industries Association (EIA), ANSI/EIA-632-1998.
- (3) ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Discussion

These three definitions, taken together define the essence of a project: namely, a group of coordinated work activities that utilizes resources to achieve specified objectives within a prescribed time frame. Objectives include requirements for a product, service, or enterprise endeavor but objectives may be broader than technical requirements; for example, an objective may be to develop a new product or product line or to support a new type of service for new customers. In addition to providing technical leadership, systems engineers often manage the technical aspects of projects.

Project Management (glossary)

(1) *The application of knowledge, skills, tools, and techniques to project activities to meet the project requirements.* (PMI 2013)

(2) *The collection of work activities concerned with planning and estimating, measuring and controlling, leading and coordinating, and managing risk factors for a project.* (Fairley 2009)

Sources

(1) PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

(2) Fairley, R.E. 2009. *Managing and Leading Software Projects*. Hoboken, NJ, USA: John Wiley & Sons.

Discussion

See also project.

Prototype (glossary)

(1) **Prototype - hardware** - A specification-compliant production readiness demonstration model developed under engineering supervision that represents what manufacturing should replicate. All design engineering and production engineering must be complete, and the assembly must be under configuration control. (Fairley 2009, p 74-75), (Forsberg, Mooz, Cotterman 2005, p 432)

(2) **Prototype - software** - A term currently with multiple meanings. A "rapid prototype" is usually a software requirements demonstration model, which provides a simulated representation of the software functionality and operator interface. The model facilitates early buyer-developer agreement on the design approach. A software prototype may also be a technical demonstration model. Except with "Evolutionary Prototypes," the code is usually discarded once the model has served its purpose. (Forsberg, Mooz, Cotterman 2005, p 432)

(3) **Prototype - software** - Software prototypes are constructed to investigate a situation or to evaluate a proposed approach to solving a technical problem. ... When building a prototype, we keep the knowledge we have gained but we do not use the code in the deliverable version of the system (Fairley 2009, p 74-75)

Sources

(1) and (2) Forsberg, K., H. Mooz, H. Cotterman. 2005. *Visualizing Project Management*, 3rd Ed., John Wiley and Sons.

(1) and (3) Fairley, R. 2009. *Managing and Leading Software Projects*. Hoboken, NJ, USA: John Wiley & Sons.

Discussion

None.

Proving (glossary)

Subjecting a system or element to a test, experiment, comparison, analysis, or the like, to determine quality, amount, acceptability, characteristics, correctness, compliance with requirements, etc.
(Dictionary.com 2012)

Sources

Dictionary.com. 2012. "Prove." Available at <https://dictionary.reference.com/browse/prove> ^[1].

Discussion

This definition is the gerund of the Dictionary.com definition. In systems engineering, proving a system generally consists of verification and validation.

References

[1] <https://dictionary.reference.com/browse/prove>

Purpose (glossary)

What the system is for, and why the different stakeholders are willing to participate in the system lifecycle. (Blockley and Godfrey 2000)

Source

Blockley, D., and P. S. Godfrey. 2000. *Doing It Differently - Systems for Rethinking Construction*. Telford, UK: Thompson Telford, Ltd.

Discussion

None.

Purposeful (glossary)

A purposeful system is free to determine its own goals needed to achieve an outcome. (Ackoff 1971)

Sources

Ackoff, R.L. 1971. "Towards a System of Systems Concepts", *Management Science*, 17(11) USA.

Discussion

This is the theoretical system science definition. A goal is a specific task achievable in a set time, while an objective is a desired outcome which need to be achieved (or partially achieved) within a time limit. Purposful systems can be given a problem to solve. Humans, and sufficiently complex machines, are purposeful.

Purposive (glossary)

Purposive systems have multiple goals (pre-determined outcomes, over a set time period), with some shared outcome. (Ackoff 1971)

Sources

Ackoff, R.L. 1971. "Towards a System of Systems Concepts," *Management Science*, 17(11), USA.

Discussion

This is the theoretical system science definition. Purposive systems can be used to do specific things. Such a system may have set functions or some freedom to choose how to achieve the goal. If it has memory it may develop processes describing the behaviors needed for defined goals. Most machines or software systems are purposive; as are humans performing set tasks on demand.

Q

Qualification (glossary)

*Conducted to prove that the system design meets its requirements **with a predetermined margin** above expected operating conditions, for instance by using elevated environmental conditions for hardware. (INCOSE 2011,128)*

*Proof that the design will survive in its intended environment **with margin**. The process includes testing and analyzing hardware and software configuration items to prove that the design will survive the anticipated accumulation of acceptance test environments, plus its expected handling, storage, and operational environments plus a **specified qualification margin**. Qualification testing usually includes temperature, vibration, shock, humidity, software stress testing, and other selected environments. Qualification by similarity may be used if the item in question is sufficiently similar to a qualified item and the planned use is also sufficiently similar so as to not invalidate the previous qualification evidence and decision. (Mooz, Forsberg, Cotterman 2003, 270)*

Sources

- (1) INCOSE. 2011. Systems Engineering Handbook, version 3.2.1. INCOSE-TP-2003-002-03.2.1.
- (2) Mooz, H., K. Forsberg, K., H. Cotterman. 2003. *Communicating Project Management*. Hoboken, NJ, USA: John Wiley and Sons.

Discussion

The term "qualification" is not well understood, and when asked "Where do the qualification limits come from?" people often say "From the specification." It is the systems engineer's responsibility to determine what the qualification margins should be, with concurrence from the project manager and customer, and then put them into the specification. Lack of understanding of the significance of "Qualification" lead to the death of seven astronauts in the Challenger accident. According to testimony presented in the 1986 Rogers Commission Report on the Space Shuttle disaster, the question was asked in the launch commit meeting the night before the flight, "Are the O-Rings qualified to operate between 40° F and 90°F (+4°C and 32°C) or not?" There was no answer. Since the temperature was expected to be (and indeed was) 29°F (-2°C) at launch, no further discussion should have been needed and the launch should have been postponed. A rule: Never operate outside the qualified range.

Quality (glossary)

Ability of a product, service, system, component, or process to meet customer or user needs, expectations, or requirements. (ISO/IEC/IEEE 2010)

Sources

ISO/IEC/IEEE. 2010. *Systems and Software Engineering -- Vocabulary (SEVocab)* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronic Engineers (IEEE) 2009 ISO/IEC/IEEE 24765:2010 [database online]. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

R

Rationale (glossary)

Argument that provides the justification for the selection of an engineering element. (Faisandier 2012)

Sources

Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Discussion

None.

Recovery (glossary)

The level of functioning to which a system returns after experiencing a disruption. (DHS 2010)

Sources

DHS. 2010. "Risk and Resilience: Exploring the Relationship". In: Kahan, J. (ed.). Arlington, VA: Department of Homeland Security (DHS), p. A-20.

Discussion

Within the context of resilience, the Department of Homeland Security is regarded as the authoritative source for definitions and other concepts related to resilience.

Recursion (glossary)

The process of defining or generating a process or data structure in terms of itself. (ISO/IEC/IEEE 24765 2010)

Sources

ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

Discussion

In the systems engineering context, recursion is used in re-application of the same standard (in the case ISO/IEC 15288) as a system is decomposed into system elements and where the elements are systems as well. The standard is re-applied for each system-of-interest (SoI) on each level.

Reductionism (glossary)

(1) an approach to understanding the nature of complex things by reducing them to the interactions of their parts. (M'Pherson 1974)

(2) a view that a complex system is nothing but the sum of its parts, and that it can be fully understood by examination of those parts. (M'Pherson 1974)

Source

(1) and (2) M'Pherson, P, K. 1974. "A perspective on systems science and systems philosophy." *Futures* 6(3) (June 1974):219-239.

Discussion

None.

Redundancy (glossary)

In fault tolerance, the presence of auxiliary components in a system to perform the same or similar functions as other elements for the purpose of preventing or recovering from failures. (ISO/IEC/IEEE 24765:2010)

Sources

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

None.

Regularity (glossary)

A uniformity or similarity that exists in multiple entities or at multiple times. (von Bertalanffy 1968).

Sources

von Bertalanffy, L. 1968. *General System Theory: Foundations, Development, Applications*, Revised ed. New York, NY, USA: Braziller.

Discussion

The existence of similarities and differences between systems from different domains is one of fundamental ideas behind systems thinking (see Concepts of Systems Thinking)

Reliability (glossary)

The probability of a system or system element performing its intended function under stated conditions without failure for a given period of time. -- modified slightly from (American Society for Quality 2011)

Sources

American Society for Quality. 2011. Glossary: Reliability. Accessed on 11 September 2012. Available at <https://asq.org/glossary/r.html>.

Discussion

This definition is similar to the one found in the *Defense Acquisition Guidebook*, and the commonly used texts. (DAU 2010) It has four important parts. First, reliability is defined as a probability that a system or system element performs some function. That function must be explicitly defined. Second, the stated operating conditions (which can include environment, load, and any other factor affecting life) must be explicitly stated. Third, a failure must be defined. Last, the appropriate time scale must be specified. This time scale can be chronological, or it can be by cycles of operation, miles, landing, or any other measure that is related to the assumed failure mechanism.

Work Cited

DAU. 2010. *Defense Acquisition Guidebook (DAG)*. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense (DoD). February 19, 2010.

Remote sensing (glossary)

the process of detecting and monitoring the physical characteristics of an area by measuring its reflected and emitted radiation at a distance (typically from satellite or aircraft).

USGS. n.d. Accessed on May 15, 2022. Available: <https://www.usgs.gov/faqs/what-remote-sensing-and-what-it-used>

Annotation

None.

Reparability (glossary)

a system resilience principle which states that a system should have the capability of being brought up to partial or full functionality over a specified period of time and in a specified environment. Jackson and Ferris (2016)

Source

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Requirement (glossary)

Statement that identifies a product or process operational, functional, or design characteristic or constraint, which is unambiguous, testable or measurable, and necessary for product or process acceptability. (ISO/IEC 2007)*

*includes product, service, or enterprise.

Source

ISO/IEC. 2007. *Systems and Software Engineering -- Recommended Practice for Architectural Description of Software-Intensive Systems*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 42010:2007.

Discussion

A requirement is "a statement that identifies a system, product or process characteristic or constraint, which is unambiguous, clear, unique, consistent, stand-alone (not grouped), and verifiable, and is deemed necessary for stakeholder acceptability." (INCOSE 2010)

INCOSE. 2010. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. Version 3.2.1. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.1: 362.

Resilience (glossary)

1. *Ability to maintain capability in the face of adversity" (INCOSE)*
2. *Ability to adapt to changing conditions and prepare for, withstand, and rapidly recover from disruption.* (DHS 2010)
3. *Ability to provide required capability when faced with adversity .* (Brtis and McEvilley 2019)

Source

- 1) INCOSE Resilient Systems Working Group.
- 2) DHS. 2010. *Department of Homeland Security Risk Lexicon," September 2010. Accessed on 11 September 2012. Available at https://www.dhs.gov/xlibrary/assets/dhs_risk_lexicon.pdf.*
- 3) Brtis, J.S. and M.A. McEvilley. 2019. *Systems Engineering for Resilience*. The MITRE Corporation. MP 190495. Accessed April 2, 2021. Available at https://www.researchgate.net/publication/334549424_Systems_Engineering_for_Resilience.

Discussion

The above definition is the official definition of the US government. The same definition appears in both sources. Although the government's interest is in the resilience of infrastructure systems, the definition is sufficiently general to apply to any human-made system. The word has been applied in material sciences, ecology, and psychology. However, those definitions do not apply here. Some sources use the word "resiliency," but "resilience" is more common.

Resolution (of imagery) (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Restructuring (glossary)

a system resilience principle that states a system should be capable of restructuring itself. Jackson (2016)

Source

Jackson, Scott. 2016. "Principles for Resilient Design - A Guide for Understanding and Implementation." In IRGC Resource Guide on Resilience, edited by I. Linkov. University of Lausanne, Switzerland: International Risk Governance Council (IRGC).

Discussion

Also called reorganization.

Reverse Engineering (glossary)

(1) A process of documenting the functions and behaviors of a device or system by analysis and test. (Created for SEBoK)

(2) The reverse engineering process applied to a product, service or enterprise consists of performing a bottom up approach from physical aspects and properties to functional ones then to identify its characteristics as requirements. (Created for SEBoK)

(3) Reverse engineering is the process of discovering the technological principles of a device, object, or system through analysis of its structure, function, and operation. It often involves taking something (e.g., a mechanical device, electronic component, software program, or biological, chemical, or organic matter) apart and analyzing its workings in detail to be used in maintenance, or to try to make a new device or program that does the same thing without using or simply duplicating (without understanding) the original.' (Wikipedia Contributors 2012)'

Source

(1) and (2) These definitions were developed for the SEBoK.

(3) Wikipedia contributors, "Reverse engineering," *Wikipedia*, The Free Encyclopedia, https://en.wikipedia.org/w/index.php?title=Reverse_engineering&oldid=509727946 (accessed September 13, 2012).

Discussion

None.

Rhumb Line (glossary)

A rhumb line, or its synonym loxodrome (used in geometry navigation), is a curve which crosses meridians of longitude at a constant bearing.

Source

ISO TC211. "Rhumb line." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 2083. Available at: <https://isotc211.geolexica.org/concepts/2083/>.

Discussion

None.

Risk (glossary)

(1) Risk is a measure of the potential inability to achieve overall program objectives within defined cost, schedule, and technical constraints and has two components: The probability (or likelihood) of failing to achieve a particular outcome and The consequences (or impact) of failing to achieve that outcome. (DAU, 2003) A risk has a probability of occurrence that is greater than zero but less than one, a consequence of occurrence greater than zero, and a time-frame in the future. (Conrow 2008)

(2) In the domain of catastrophic risk analysis, such as for terrorist attacks or natural disasters, risk has three components: Threat (the probability that a specific target is attacked in a specific way during a specified period) Vulnerability (the probability that damage occurs given a threat), and Consequence (the magnitude and type of damage resulting from an attack or disaster). (Willis et al. 2005) (1) Conrow, E. 2008. Risk Analysis for Space Systems. Paper presented at Space Systems Engineering and Risk Management Symposium, 27-29 February, 2008, Los Angeles, CA, USA. (1) DAU. 2003. Risk Management Guide for DoD Acquisition: Fifth Edition. Ft. Belvoir, VA, USA: Defense Acquisition University (DAU)/U.S. Department of Defense, Fifth Edition, Version 2. (2) Willis, H.H., A.R. Morral, T.K. Kelly, and J.J. Medby. 2005. Estimating Terrorism Risk. Santa Monica, CA: The RAND Corporation, MG-388. Definition (1) or related definitions are in widespread project risk management use. The definition has been extended to include time-frame. Definition (2) or related definitions are in widespread use for catastrophic risk analysis (e.g., threat, disaster, information assurance). Definition (1) defines risk in terms of probability of occurrence, consequence of occurrence, and time-frame; all of which are measurable. Likewise, definition (2) defines risk in terms of threat, vulnerability, and consequence of occurrence; all of which are measurable.

Risk Management (glossary)

- (1) *An organized process for identifying and handling risk factors.* (ISO/IEC/IEEE 2017)
- (2) *an organized means of identifying and measuring risk (risk assessment) and developing, selecting, and managing options (risk analysis) for resolving (risk handling) these risks.* (ISO/IEC/IEEE 2017)
- (3) *organized, analytic process to identify what might cause harm or loss (identify risks); to assess and quantify the identified risks; and to develop and, if needed, implement an appropriate approach to prevent or handle causes of risk that could result in significant harm or loss.* (ISO/IEC/IEEE 2017)

Sources

(1) - (3) ISO/IEC/IEEE. 2017. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2017.

Discussion

None.

Robustness (glossary)

- [1] the inherent strength or resistance in a system to withstand external demands without degradation or loss of functionality. Jackson (2016)
- [2] the ability to resist capability degradations under adverse conditions. Brtis (2016)
- [3] The degree to which a system or component can function correctly in the presence of invalid inputs or stressful environmental conditions. (ISO/IEC/IEEE 2010)

Sources

- [1] Jackson, Scott. 2016. "Principles for Resilient Design - A Guide for Understanding and Implementation." In IRGC Resource Guide on Resilience, edited by I. Linkov. University of Lausanne, Switzerland: International Risk Governance Council (IRGC).
- [2] Brtis, John. 2016. *How to Think About Resilience in a DoD Context*. Colorado Springs, CO: MITRE Corporation.
- [3] ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

Discussion

This is a basic attribute of system resilience.

Routing (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Strongly suggest that there are at least two definitions: one in the geospatial context and one in the context of software routing.

S

Safety (glossary)

The expectation that a system does not, under defined conditions, lead to a state in which human life, health, property, or the environment is endangered. (ISO/IEC 1998)

Sources

ISO/IEC. 1998 *Information Technology — System and Software Integrity Levels* Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 15026:1998.

Discussion

None.

Satellite Positioning System (glossary)

A positioning system based upon receipt of signals broadcast from satellites.

Note to entry: In this context, satellite positioning implies the use of radio signals transmitted from "active" artificial objects orbiting the Earth and received by "passive" instruments on or near the Earth's surface to determine position, velocity, and/or attitude of an object. Examples are GPS and GLONASS.

Sources

ISO TC211. *Satellite positioning system*. In: ISO TC211 Multilingual Glossary. Version from 2020-06-02. Concept ID: 392. Available at: <https://isotc211.geolexica.org/concepts/392/>

Discussion

None.

Scalability (glossary)

The ability of something, especially a computer system, to adapt to increased demands. (Collins English Dictionary 2009)

Sources

Collins English Dictionary - Complete & Unabridged 10th Edition. 2009. s.v. "Scalability"

Discussion

None.

Scenario (glossary)

A set of actions / functions representing the dynamic of exchanges between the functions allowing the system to achieve a mission or a service. (Faisandier 2012)

Sources

Faisandier, A. 2012. *Systems Architecture and Design*. Belberaud, France: Sinergy'Com.

Discussion

None.

Scope (glossary)

- (1) *The behavior that system is expected to exhibit.* (ISO/IEC/IEEE 24765 2010)
- (2) *Work to be performed under a contract or subcontract in the completion of a project. Also called work scope.* (BusinessDictionary.com)
- (3) *The goals, processes, resources, facilities, activities, outputs and outcomes an organization is responsible for.* (Created for SEBoK)

Source

- (1) ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.
- (2) BusinessDictionary.com, s.v. "Scope." Available online at: <https://www.businessdictionary.com/definition/scope.html>. Accessed July 7, 2012.
- (3) This definition was developed for the SEBoK.

Discussion

Definition (1) is a typical systems engineering definition, outlining the range of behaviors/functions expected within a system. Definition (2) refers to the concept of "scope of work", as defined between an acquirer and a supplier. Definition (3) is important to the scope of an organization versus the scope of a system.

Security (glossary)

Protection against intentional subversion or forced failure. A composite of four attributes – confidentiality, integrity, availability, and accountability – plus aspects of a fifth, usability, all of which have the related issue of their assurance. NATO AEP-67 2010

Sources

NATO. 2010. *Engineering for System Assurance in NATO programs*. Washington, DC, USA: NATO Standardization Agency. DoD 5220.22M-NISPOM-NATO-AEP-67

Discussion

None.

Semantic Interoperability (glossary)

The ability to exchange data in such a way that the precise meaning of the data is readily accessible and the data itself can be translated by any system into a form that it understands. (Adapted from Heflin and Hendler 2000)

Sources

Adopted from: Heflin, J. and J. Hendler. 2000. "Semantic Operability on the Web". *Extreme Markup Languages*. Accessed 7 September 2012. Available at: <https://www.cs.umd.edu/projects/plus/SHOE/pubs/extreme2000.pdf>.

Discussion

None.

Semantics (glossary)

The relationships of symbols or groups of symbols to their meanings in a given language. (ISO/IEC/IEEE 2010)

Sources

ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

Discussion

None.

Service (glossary)

An activity required by one or more users who have agreed on the terms of outcomes and quality of service without details to how it is provided. A service is also, simply put, an act of help or assistance. In a more formal sense:

Services are activities that cause a transformation of the state of an entity (people, product, business, and region or nation) by mutually agreed terms between the service provider and the customer. (Chang 2010)

Source

Chang, C. M. 2010. *Service Systems Management and Engineering: Creating Strategic Differentiation and Operational Excellence*. New York, NY, USA: John Wiley & Sons, Inc. ISBN 978-0-470-42332-5.

Discussion

In the service-dominant logic (S-DL) for marketing, service is the application (through deeds, processes, and performances) of specialized operant resources (knowledge and skills) for the benefit of another entity or the entity itself. (Vargo and Akaka 2009)

In the field of service science, management, and engineering (SSME), the following is offered:

Services are processes, performances, or experiences that one person or organization does for the benefit of another – such as custom tailoring suit, cooking a dinner to order, driving a limousine, mounting a legal defense, setting a broken bone, teaching a class, or running a business's information technology infrastructure and applications. In all cases, service involves deployment of knowledge, skills, and competences that one person or organization has for the benefit of another (Vargo and Lusch 2004), often done as a single, customized job. And in all cases, service requires substantial input from the customer or client (Sampson 2001) – how else could your steak be customized for you unless you tell your waiter how you want it prepared? (https://en.wikipedia.org/wiki/Service_Science,_Management_and_Engineering)

Furthermore:

- Services are economic activities offered by one party to another, most commonly employing time-based performances to bring about desired results in recipients themselves or in objects or other assets for which purchasers have responsibility. In exchange for their money, time, and effort, service customers expect to obtain value from access to goods, labor, professional skills, facilities, networks, and systems; but they do not normally take ownership of any of the physical elements involved. (Lovelock & Wirtz 2007)
 - A service is a time-perishable, intangible experience performed for a customer acting in the role of a co-producer. (Fitzsimmons & Fitzsimmons 2007)
 - The application of competences (knowledge, skills and resources) by one entity for the benefit of another entity in a non-coercive (mutually agreed and mutually beneficial) manner.
 - Value co-creation interactions (typically with well-defined customer-provider entities as parties who initiate, directly or indirectly, front-stage and back-stage activities in anticipation of value results)
 - An economic activity offered by one party to another, most commonly employing time-based performances to bring about desired transformation results in recipients themselves or in objects or other assets for which purchasers are responsible. In exchange for their money, time and effort, service customers expect to obtain value from the access to goods, labour, professional skills, facilities, networks and systems; but they do not normally take ownership of any of the physical elements involved. (IfM and IBM 2008)
-

Works Cited Fitzsimmons, J.A. & M.J. Fitzsimmons. 2007. *Service Management: Operations, Strategy, Information Technology*, 6th ed. New York, NY, USA: McGraw-Hill.

IfM and IBM. 2008. "Succeeding through service innovation: A service perspective for education, research, business and government." University of Cambridge Institute for Manufacturing. Cambridge, UK, cited by Spohrer, J. and P. Maglio. 2010. "Service Science: Toward a Smarter Planet." In *Introduction to Service Engineering*. Ed. G Salvendy and W Karwowski. 3-30. Hoboken, NJ, USA: John Wiley & Sons, Inc.

Lovelock & Wirtz. 2007. *Services Marketing: People, Technology, Strategy*, 7th Edition. Upper Saddle River, NJ, USA: Prentice Hall.

Sampson, S.E. 2001. *Understanding Service Businesses*. New York, NY, USA: John Wiley.

Vargo, S.L. and R.F. Lusch. 2004. "The Four Service Marketing Myths – Remnants of a Goods-Based Manufacturing Model." *Journal of Service Research*. 6: 324-335.

Service Life Extension (SLE) (glossary)

Modification(s) to fielded systems undertaken to extend the life of the system beyond what was previously planned. (ExpertGlossary)

Sources

ExpertGlossary.com. s.v. "Service Life Extension." Accessed on 11 September 2012. Available at <https://www.expertglossary.com/find?q=service+life+extension>.

Discussion

None.

Service Science (glossary)

The application of scientific, engineering, and management competencies that a service-provider organization performs that creates value for the benefit of the client of customer. (Katzan 2008, vii)

Sources

Katzan, H. 2008. *Service Science*. Bloomington, IN, USA: iUniverse Books.

Discussion

None.

Service System (glossary)

A dynamic configuration of resources (people, technology, organizations and shared information) that creates and delivers value between the provider and the customer through services. (IfM and IBM 2008)

Sources

IfM and IBM. 2008. "Succeeding through Service Innovation: A Service Perspective for Education, Research, Business and Government." University of Cambridge Institute for Manufacturing. Cambridge, UK. cited by Spohrer, J. and P. Maglio. 2010. "Service Science: Toward a Smarter Planet." In *Introduction to Service Engineering*. Ed. G Salvendy and W Karwowski. 3-30. Hoboken, NJ, USA: John Wiley & Sons, Inc.

Discussion

This dynamic configuration is illustrated in the system coupling diagram (Lawson 2010) where a situation (need for a service) is met by (interacts with) a respondent system (service system) based upon the use of system assets. A service system can also be thought of as a collection of entities that performs the operations, administration, management and provisioning (OAM&P) of resources that together provide the opportunity to co-create value by both the service provider and the service consumer.

The Cambridge white paper defines a service system in this manner (IfM and IBM 2008):

Service systems are dynamic configurations of resources (people, technology, organisations and shared information) that can create and deliver service while balancing risk-taking and value co-creation. The dynamics are in part due to the ongoing adjustments and negotiations that occur in all systems involving people. People are the ultimate arbiters of value and risk in service systems (in part because people are legal entities with rights and responsibilities). Service systems are complex adaptive systems.

Works Cited

IfM and IBM. 2008. "Succeeding through service innovation: A service perspective for education, research, business and government." University of Cambridge Institute for Manufacturing. Cambridge, UK. cited by Spohrer, J. and P. Maglio. 2010. "Service Science: Toward a Smarter Planet." In *Introduction to Service Engineering*. Ed. G Salvendy and W Karwowski. 3-30. Hoboken, NJ, USA: John Wiley & Sons, Inc.

Lawson, H. 2010. *A Journey Through the Systems Landscape*. London, UK: College Publications, Kings College.

Service Systems Engineering (glossary)

(1) Service systems engineering (SSE) is the application of systems engineering principles and concepts in the development, delivery, operation, and life cycle management of service systems. The key focus of SSE is on the transactions between a service (glossary) provider and service consumers. The consumer can be an individual, an organization, or even an entire enterprise. (Created for SEBoK)

(2) A multidiscipline that addresses a service system from a life-cycle, cybernetic, and customer perspective. (Tien and Berg 2003)

Source

(1) This definition was developed for the SEBoK.

(2) Tien, J.M and D. Berg. 2003. "A Case for Service Systems Engineering." *Journal of Systems Science and Systems Engineering*. 12(1) (March 2003): 13-38.

Discussion

Though "service systems engineering" (SSE) is used commonly within the field of systems engineering, few formal definitions are provided within the literature. Tien and Berg (2003) provides definition (1), which includes the concepts that multiple disciplines must be utilized to manage customer needs and expectations of a service system throughout the systems life cycle. This aligns with the notion of a service as something akin to a support system for an additional system (e.g. a call center or information technology (IT) support system). The SEBoK authors, however, believe that a more holistic view of SSE is beneficial to the discussion and in understanding how it is addressed within the literature. The author team developed definition (1) to address these concerns.

Service Value Chain (glossary)

The progression of a service opportunity from beginning to end. (FieldSVC 2005)

Sources

FieldSVC. 2005. "The Service Value Chain." (online). USA: Field Service Value Chain (FieldSVC). Accessed on 11 September 2012. Available at: <https://www.fieldsvc.com/Files/White%20Paper%20-%20The%20Service%20Value%20Chain.pdf>.

Discussion

None.

Set Driver (glossary)

Fundamental design decisions that define the platform characteristics that enable current and future missions, e.g. airframe and engine for a UAV and structure and engine for a car. (Specking et al. 2018)

Source

Specking, E., C. Whitcomb, G. Parnell, S. Goerger, E. Kundeti, and N. Pohl. 2018. "Literature Review: Exploring the Role of Set-Based Design in Trade-off Analytics." *Naval Engineers Journal*. 130 (2): 51-62.

Discussion

None.

Set Modifiers (glossary)

Design decisions that are 'added on' to the platform and can be modified to adapt to new missions and scenarios, e.g., sensors, weapons, communications equipment for a UAV and tires, type of oil, navigation software for a car. (Specking et al. 2018)

Source

Specking, E., C. Whitcomb, G. Parnell, S. Goerger, E. Kundeti, and N. Pohl. 2018. "Literature Review: Exploring the Role of Set-Based Design in Trade-off Analytics." *Naval Engineers Journal*. 130 (2): 51-62.

Discussion

None.

Set-Based Design (glossary)

A complex design method that enables robust system design by 1) considering a large number of alternatives, 2) using experts who design from their own perspectives and use the intersection between their individual sets to optimize a design, and 3) establishing feasibility before making decisions. (Singer, Doerry, and Buckley 2009)

Source

Singer, D.J., N. Doerry, and M. E. Buckley. 2009. "What Is Set-Based Design?" *Naval Engineers Journal*. 121(4): 31–43.

Discussion

None.

Simulation (glossary)

(1) *A model that behaves or operates like a given system when provided a set of controlled inputs.*

(ISO/IEC/IEEE 24765:2010)

(2) *The process of developing or using a model as in (1).* (ISO/IEC/IEEE 24765:2010)

Sources

(1) and (2) ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

Discussion

None.

Simulator (glossary)

A device, computer program, or system that behaves or operates like a given system when provided a set of controlled inputs. (ISO/IEC 2010)

Sources

ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

Discussion

None.

Six Sigma (glossary)

Six Sigma is a rigorous and a systematic methodology that utilizes information (management by facts) and statistical analysis to measure and improve a company's operational performance, practices and systems by identifying and preventing 'defects' in manufacturing and service-related processes in order to anticipate and exceed expectations of all stakeholders to accomplish effectiveness. (iSixSigma 2012)

Sources

iSixSigma. 2012. "Dictionary: Six Sigma." Accessed 11 September 2012. Available at: <https://www.isixsigma.com/dictionary/six-sigma/>.

Discussion

None.

Social System (glossary)

An open system which includes human elements. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

Several definitions of "social systems" exist. For example, "the people in a society considered as a system organized by a characteristic pattern of relationships," (WordNet 2012) or "organized boundary systems of social roles, behaviors, and practices developed to maintain values and the mechanisms to regulate the practices and roles (Taber's Cyclopedic Medical Dictionary 2009). *These definitions, however, generally arise from sociology and do not take into account a "systems thinking" perspective. The definition provided above has been created for the SEBoK based on the systems science definition. See the Engineered Systems topic for additional discussion.*

Works Cited

Taber's Cyclopedic Medical Dictionary, 21st Edition, s.v. "social systems", 2009.

WordNet: A lexical database for English, version 3.1, s.v. "social system", 2012.

Sociotechnical System (glossary)

An engineered system which includes a combination of technical and human or natural elements.
(Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

The term "sociotechnical system" is used frequently in the literature, and almost all definitions state simply "a system incorporating humans and technology". The definition above has been created for the SEBoK alongside the definition of an "engineered system" - in the context of the SEBoK, sociotechnical systems are considered systems which are intentionally designed to incorporate human beings. See the Engineered Systems topic for additional information. For a brief overview of some common characteristics of sociotechnical systems, please see (ComputingCases.org 2012), a resource designed to support education in ethical issues in computing.

Works Cited

ComputingCases.org. Accessed 2012. "Why a Socio-Technical System?" Accessed September 10, 2012. Available at: https://computingcases.org/general_tools/sia/socio_tech_system.html.

Soft System (glossary)

A system that is characterized by extremely complex, problematical, and often mysterious phenomena, for which concrete goals cannot be established and which require learning in order to make improvement. Such systems are not limited to the social and political arenas; they also exist within and amongst enterprises where complex, often ill-defined patterns of behavior are observed that are limiting the enterprise's ability to improve. (Checkland 1999)

Source

Checkland, P. B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

Discussion

None.

Soft Systems Methodology (glossary)

Soft systems methodology (SSM) formalizes the ideas of a soft system approach using systemic thinking to expose the issues in a problem situation and guide interventions to reduce them. SSM provides a framework of ideas and models to help guide participants through this systemic thinking. (Checkland 1999)

Sources

Checkland, P.B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

Discussion

SSM is the most common example for a Problem Structuring Approach, see the Systems Science knowledge area for details.

Software (glossary)

All or part of the programs, procedures, rules, and associated documentation of an information processing system. (ISO/IEC 2382-1:1993)

Sources

ISO/IEC. 1993. *Information technology--Vocabulary--Part 1: Fundamental terms*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). ISO/IEC 2382-1:1993.

Discussion

None.

Software Assurance (glossary)

Level of confidence that software is free from vulnerabilities, either intentionally designed into the software or accidentally inserted at anytime during its lifecycle and that the software functions in the intended manner. (CNSS 2010)

Sources

CNSS. 2010. *National Information Assurance (IA) Glossary*. Fort George G. Meade, MD, USA: Committee on National Security Systems. CNSS Instruction No. 4009. 26 April 2010. Accessed on 11 September 2012. Available online at: https://www.cnss.gov/Assets/pdf/cnssi_4009.pdf.

Discussion

None.

Software Engineering (glossary)

(1) *The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (IEEE 1990)*

(2) *The study of approaches as in (1). (IEEE 1990)*

Source[s]

(1) and (2) IEEE. 1990. *IEEE Standard Glossary of Software Engineering Terminology*. Washington, DC: Institute of Electrical and Electronics Engineers. IEEE 610.12-1990.

Source

P. Bourque and R.E. Fairley (eds), *Guide to the Software Engineering Body of Knowledge*, Version 3.0, IEEE Computer Society, 2014; Available at <https://www.swebok.org>

Piscataway, NJ, USA: The Institute of Electrical and Electronic Engineers, Inc. (IEEE). Available at: <https://www.computer.org/portal/web/swebok>.

Discussion

The definitions listed above are also the accepted definitions used in the *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, which was published by the IEEE Computer Society in 2004.

Software System (glossary)

A software-intensive system for which software is the only component to be developed or modified.
(IEEE 1998)

Source

IEEE Standards Association. 1998. "IEEE 1362-1998 IEEE Guide for Information Technology-System Definition -Concept of Operation Document, 3.2." Accessed 25 July 2012. Available at <https://standards.ieee.org/findstds/standard/1362-1998.html> <https://standards.ieee.org/findstds/standard/1362-1998.html>.

Discussion

None.

Software-in-the-Loop Testing (glossary)

In software-in-the-loop testing, the actual Production Software Code is incorporated into the mathematical simulation that contains the models of the Physical System. This is done to permit inclusion of software functionality for which no model(s) exists, or to enable faster simulation times.
(Argonne National Lab 2012)

Sources

Argonne National Lab. 2012. *Model based design definition of terms*. Available at https://www.autonomie.net/references/model_based_design_defs_24c.html.^[1]

Discussion

This is one of three related techniques: model-in-the-loop, software-in-the-loop, and hardware-in-the-loop.

References

[1] https://www.autonomie.net/references/model_based_design_defs_24c.html

Solution (glossary)

A means of solving a problem or dealing with a difficult situation. (American Heritage Dictionary 2009 on-line)

Sources

American Heritage Dictionary, v2.1.3. 2009. s.v. "Solution."

Discussion

None.

Spatial Reference (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Spatial Reference System (glossary)

A system for identifying position in the real world.

Sources

ISO TC211. "Spatial reference system." In: *ISO TC211 Multilingual Glossary*. Version from 2020-06-02. Concept ID: 422. Available at: <https://isotc211.geolexica.org/concepts/422/>.

Discussion

None.

Spatial Reference (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

Specialty Engineering (glossary)

(1) A component of Systems Engineering that complements the technical activities required to deliver a project. It typically deals with engineering that affects the performance, safety, usability, cost certainty, constructability, deliverability and lifecycle of the product outside of the normal functional aspects of engineering. (INCOSE 2015)

(2) The collection of those narrow disciplines that are needed to engineer a complete system. (Elowitz 2006)

Source

(1) INCOSE. 2015. INCOSE Infrastructure Working Group. Available at: INCOSE https://www.incose.org/docs/default-source/Working-Groups/infrastructure-wg-documents/005-specialty-engineering-pamphlet.pdf?sfvrsn=9c2c82c6_6.

(2) Elowitz, M. 2006. "Specialty Engineering as an Element of Systems Engineering." Presentation to Enchantment Chapter, INCOSE. Albuquerque, NM, USA, 9 August 2006.

Discussion

The definition of "narrow disciplines" is context dependent, and can include traditional domain disciplines (electrical engineering, mechanical engineering, etc.) as well as integrative disciplines (human systems engineering, safety engineering, etc.).

Stage (glossary)

A period within the life cycle of an entity that relates to the state of its description or realization.
(ISO/IEC/IEEE 15288)

Source

ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Discussion

A lifecycle stage may also be referred to as a phase.

Stakeholder (glossary)

- (1) *Individual or organization having a right, share, claim, or interest in a system or in its possession of characteristics that meet their needs and expectations (ISO/IEC/IEEE 2015)*
- (2) *Individual or organization having a right, share, claim, or interest in a system or in its possession of characteristics that meet their needs and expectations; N.B. Stakeholders include, but are not limited to end users, end user organizations, supporters, developers, producers, trainers, maintainers, disposers, acquirers, customers, operators, supplier organizations and regulatory bodies. (ISO/IEC June 2010)*
- (3) *An individual, team, or organization (or classes thereof) with interests in, or concerns relative to, a system. (ISO/IEC 2007)*
- (4) *A stakeholder in an organisation is (by definition) any group or individual who can affect or is affected by the achievement of the organisation's objectives. (Freeman 1984)*

Source

- (1) ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission/Institute of Electrical and Electronics Engineers (IEC), ISO/IEC/IEEE 15288:2015 (E).
 - (2) ISO/IEC. June 2010. *Software and Systems Engineering -- Life Cycle Processes -- Requirements Engineering*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), ISO/IEC CD 29148.
 - (3) ISO/IEC. 2007. *Systems Engineering--Application and Management of the Systems Engineering Process*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC), ISO/IEC 26702:2007.
 - (4) Freeman, R.E. 1984. *Strategic Management: A stakeholder approach*, Boston, Pitman
-

Discussion

None.

Stakeholder Needs and Requirements (glossary)

(1a) *Stakeholder needs include elicitation of needs, wants, desires, expectations and perceived constraints of identified stakeholders. Needs concentrate on system purpose and behavior, and are described in the context of the operational environment and conditions. (ISO/IEC/IEEE 15288, 2023, Section 6.4.2)*

(1b) *Stakeholder requirements are transformed from the stakeholder needs to objectively adequate, structured and more formal statements. (ISO/IEC/IEEE 29148, 2018, Section 5.2.3)*

(1c) *Stakeholder requirements express the intended interaction the system will have with its operational environment and that are the reference against which each resulting operational capability is validated. The stakeholder requirements are defined considering the context of the system of interest with the interoperating systems and enabling systems. This also includes consideration of laws and regulations, environmental restrictions, and ethical values. (ISO/IEC/IEEE 15288, 2023, Section 6.4.2)*

Source

ISO/IEC/IEEE. 2023. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2023.

ISO/IEC/IEEE. 2018. *Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 29148:2018.

Discussion

Both definitions are taken from the introduction to the "Stakeholder Requirements Definition Process" in ISO/IEC 15288; there is no formal definition of the term "stakeholder requirement" in the standard.

For a full discussion of the role and importance of stakeholder needs in systems engineering see the Stakeholder Needs Definition article.

State (glossary)

(1) *The values of key system element qualities or properties at a given time.* (Ackoff 1971)

(2) *A description of the current or future potential, or health of the system.* (Ackoff 1971)

Source

(1) and (2) Ackoff, R.L. 1971. "Towards a System of Systems Concepts." *Management Science*. 17(11):USA.

Discussion

The first definition is a mathematical one, taken from the cybernetic world.

The second is used more practical, and is used by the system operations or sustainment community.

Statistical Process Control (glossary)

Statistically based analysis of a process and measures of process performance, which identify common and special causes of variation in process performance and maintain process performance within limits.
(ISO/IEC/IEEE 2009)

Sources

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

None.

Structure (glossary)

- (1) (*architectural structure*) *A physical or logical layout of the components of a system design and their internal and external connections.* (ISO/IEC 2009, 1)
- (2) (*Data structure*) *a physical or logical relationship among data elements, designed to support specific data manipulation functions.* (ISO/IEC 2009, 1)
- (3) (*Generalization structure*) *a connection between a superclass and one of its more specific, immediate subclasses.* (IEEE 1320.2-1998)
- (4) *The static existence of the system; namely its elements and their relationships.* (Created for SEBoK)

Source

- (1) and (2) ISO/IEC. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.
- (3) IEEE. 1998. *IEEE Standard for Application and Management of the Systems Engineering Process*. Washington, DC: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1220-1998.
- (4) This definition developed for the SEBoK.

Discussion

None.

Supplier (glossary)

- (1) *An organization or an individual that enters into an agreement with the acquirer for the supply of a product or a service.* (ISO/IEC 2015)
- (2) *Suppliers may be internal or external.* (ISO/IEC 2015)

Source

- (1) and (2) ISO/IEC/IEEE. 2015. *Systems and Software Engineering -- System Life Cycle Processes*. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Discussion

None.

Survivability (glossary)

Capability of a system to fulfill its mission, in a timely manner, in the presence of attacks (external), failures (internal), or accidents (environment). (Ellison et al. 1999)

Sources

Ellison, R.J., D.A. Fisher, R.C. Linger, H.F. Lipson, T. Longstaff, N.R. Mead 2009. *Survivable Network Systems: An Emerging Discipline*, Carnegie-Mellon Software Engineering Institute Technical Report CMU/SEI-97-TR-013, 1997 revised 1999.

Discussion

None.

Sustainment (glossary)

Activities performed to ensure that a product or service remains operational. (ISO/IEC 2009, 1)

Source

ISO/IEC. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Synectics (glossary)

A set of process tools which can be used successfully in a variety of situations, either in a specific sequence (as in the original invention model) or individually according to the needs of the situation. The result is a variety of meeting models and techniques for enhancing personal effectiveness. (Nolan 2003)

Sources

Nolan, V. 2003. "Whatever Happened to Synectics?" *Creativity and Innovation Management*. 12(1) (March 2003): 24.

Discussion

None.

Synergy (glossary)

(1) In an organization synergy is the ability of a group to outperform even its best individual member. (Buchanan and Huczynski, 1997).

(2) A construct or collection of different elements working together to produce results not obtainable by any of the elements alone. The elements, or parts, can include people, hardware, software, facilities, policies, documents: all things required to produce system-level results. (Blanchard 2004).

Sources

(1) Buchanan, D. and A. Huczynski. 1997. *Organizational behavior, introductory text*, 3rd edition, p. 276. Upper Saddle River, NJ, USA: Prentice Hall.

(2) Blanchard, B. 2004. *System Engineering Management*, p. 8. Hoboken, N, USAJ: John Wiley.

Discussion

Synergy is two or more things functioning together to produce a result not independently obtainable. The term synergy comes from the Greek word synergia meaning "working together". It is often used in business or other human activity systems to describe outcomes which can only be achieved by encouraging people or organizations to work together, often through encouraging friction or creative tensions between them. In medicine synergy is used to describe combinations of drugs which interact in ways that enhance or magnify one or more effects, or side-effects, of those drugs. This may give both positive and negative effects.

Definition (2) is typical of the use of the term synergy in systems engineering texts to describe how system behavior emerges from the interaction between elements. In this sense synergy is closely related to emergence.

Synthesis (glossary)

The combination of parts, elements, or diverse conceptions into a coherent whole; to put together
(INCOSE 1998, p. 236)

Sources

INCOSE. 1998. "INCOSE SE Terms Glossary". In *INCOSE Concepts and Terms*. Seattle, WA: International Council on Systems Engineering.

Discussion

None.

System (glossary)

(1) *A set of elements in interaction.* (von Bertalanffy 1968)

(2) *combination of interacting elements organized to achieve one or more stated purposes*
(ISO/IEC/IEEE 2015)

(3) A system is an arrangement of parts or elements that together exhibit behavior or meaning that the individual constituents do not. (INCOSE Fellows, 2019)

Source

(1) von Bertalanffy, L. 1968. *General System Theory: Foundations, Development, Applications*. Revised ed. New York, NY, USA: George Braziller, Inc.

(2) ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015. The second definition is an expanded version of the ISO/IEC/IEEE version.

(3) INCOSE Fellows Briefing to INCOSE Board of Directors, January 2019.

Discussion

Definition (1) is the System Science definition and applies to all systems: natural, social or technical.

Definition (2) is the recognized definition for systems engineers. Elements in this sense may include hardware, software, firmware, people, information, techniques, facilities, services, related natural artifacts and other support elements. This definition should be restricted to engineered systems which are created with a purpose which provides value to one or more beneficiaries.

Definition (3) was created by the INCOSE Fellows Initiative on System and Systems Engineering Definitions. This was established in 2016, to review current INCOSE definitions of SYSTEM and SYSTEMS ENGINEERING and to recommend any changes necessary to align the definitions to a) current practice, and b) the aspirations of INCOSE's 2025 Vision. At the January 2019 INCOSE Board of Directors meeting, a new INCOSE definition for "system" was approved and is given above.

Expanding on this definition, the INCOSE Fellows state:

Systems can be either physical or conceptual, or a combination of both. Systems in the physical universe are composed of matter and energy, may embody information encoded in matter-energy carriers, and exhibit observable

behavior. Conceptual systems are abstract systems of pure information, and do not directly exhibit behavior, but exhibit “meaning”. In both cases, the system's properties (as a whole) result, or emerge from:

- the parts or elements and their individual properties; AND
- the relationships and interactions between and among the parts, the system and its environment.

Note, definitions (1) and (2) remain applicable to the usage of this term in the current SEBoK, unless specifically stated. As the fellow initiative definition becomes more widely used we expect it to become the default definition in future releases of the SEBoK

See Introduction to Systems Fundamentals for a full discussion of the nature of systems in general and the scope of engineered systems of particular interest to systems engineering.

System Analysis (glossary)

A systematic investigation of a real or planned system to determine the information requirements and processes of the system and how these relate to each other and to any other system. (ISO/IEC/IEEE 2009)

Sources

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

For a full discussion of the role and importance of system analysis in systems engineering see the System Analysis article.

System Assurance (glossary)

...the justified confidence that the system functions as intended and is free of exploitable vulnerabilities, either intentionally or unintentionally designed or inserted as part of the system at any time during the life cycle... This confidence is achieved by system assurance activities, which include a planned, systematic set of multi-disciplinary activities to achieve the acceptable measures of system assurance and manage the risk of exploitable vulnerabilities. (NATO February 2010, p. 1)

Sources

NATO. 2010. Engineering for system assurance in NATO programs. Washington, DC: NATO Standardization Agency, DoD 5220.22M-NISPOM-NATO-AEP-67. February 2010.

Discussion

The NATO document is organized based on the life cycle processes in ISO/IEC 15288:2008 and provides process and technology guidance to improve system assurance.

System Boundary (glossary)

A distinction made by an observer which marks the difference between an entity he takes to be a system and its environment. (Checkland 1999, 312)

Source

Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.

Discussion

None.

System Breakdown Structure (glossary)

A hierarchy of elements, related life cycle processes, and personnel used to assign development teams, conduct technical reviews, and to partition out the assigned work and associated resource allocations to each of the tasks necessary to accomplish the objectives of the project. (IEEE 2005)

Sources

IEEE. 2005. *IEEE Standard for Application and Management of the Systems Engineering Process*. Washington, DC, USA: Institute of Electrical and Electronics Engineers. IEEE 1220-2005.

Discussion

None.

System Capability (glossary)

The ability of a system to execute a particular course of action or achieve a desired effect, under a specified set of conditions. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

In the SEBoK, the terms "capability" may be used in place of "system capability", implying a capability discussed within a systems context. Please see the discussion of capability. It is important to note that "system capability" is sometimes used to distinguish between an "organizational capability" within the SEBoK.

System Context (glossary)

(1) Describes the system relationships and environment, resolved around a selected system-of-interest. (Flood and Carson 1993)

(2) Diagram defining the highest level view of a system in its environment. (Flood and Carson 1993)

Source

(1) and (2) Flood, R.L. and E.R. Carson. 1993. *Dealing with complexity: An introduction to the theory and application of systems science*, 2nd ed. New York, NY, USA: Plenum Press.

Discussion

Definition (1) describes the reason for considering context.

Definitions (2) relates context to a context diagram. This is most common way in which context is expressed in systems engineering. A number of context diagram notations are available.

System Coupling Diagram (glossary)

A paradigm that conveys the essence of what systems are used for and how systems relate to each other as portrayed in the figure (Lawson 2010)

Sources

Lawson, H. 2010. *A Journey Through the Systems Landscape*, UK: College Publications, Kings College.

Discussion

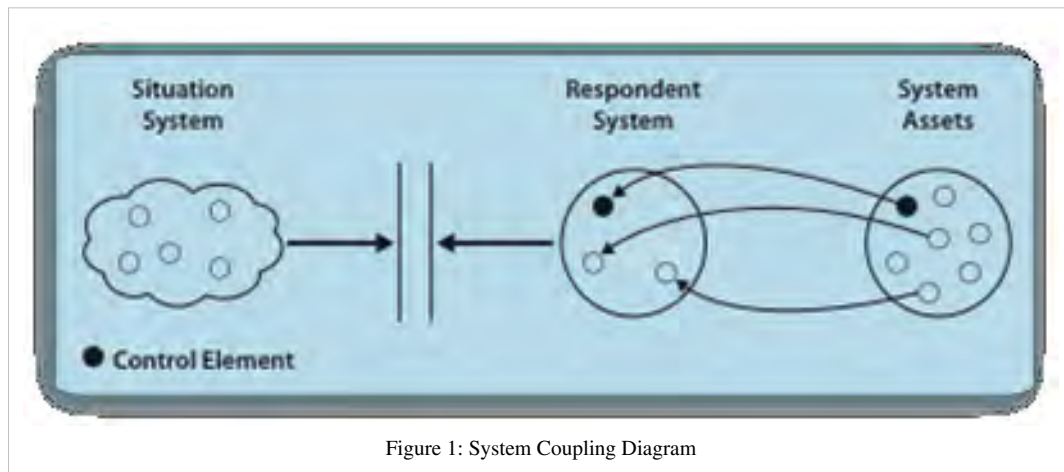


Figure 1: System Coupling Diagram

- Situation System – A problem or opportunity situation; either unplanned or planned. The situation may be the work of nature, be man-made, a combination of both or a postulated situation (Thematic System) that is to be used as a basis for deeper understanding and training (for example, business games or military exercises). Thus, the situation may be catastrophe, terrorist action, the capabilities needed for a new system, the next stage in a life cycle of a system. In all cases, in order to affect the situation a response is required.
- Respondent System – The system created to respond to the situation where the parallel bars indicate that this system interacts with the situation and transforms the situation to a new situation. A Respondent System, based upon the situation that is being treated can have several names such as Project, Program, Mission, Task Force, or

in a scientific context, Experiment. Note that one of the system elements of this system is a control element that directs the operation of the respondent system in its interaction with the situation. This element is based upon an instantiation of a Control System asset, for example a Command and Control System, or a control process of some form.

- **System Assets** – The sustained assets of an enterprise that are to be utilized in responding to situations. System assets must be adequately life cycle managed so that when instantiated in a Respondent System will perform their function. These are the systems that are the primary objects of an Enterprise and include their value added products or services as well as their infrastructure systems. Examples of assets include concrete systems such as produced products and/or services, facilities, instruments and tools as well as abstract systems such as theories, knowledge, processes and methods.

System Definition (glossary)

A set of core technical activities of systems engineering, including the activities that are completed primarily in the front-end portion of the system design. This consists of the definition of system requirements, the design of one or more logical and physical architectures, and analysis and selection between possible solution options. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

System definition is a knowledge area in the SEBoK that includes several front-end systems engineering activities, such as system requirements development; functional, logical and physical architectural design, and system analysis. This generally aligns with the "concept stage" as discussed in INCOSE (2012). In the SEBoK, system concept definition is used to discuss the assessment of the mission and stakeholder requirements.

Work Cited

INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

System Deployment and Use (glossary)

A set of core technical activities of systems engineering in ensure that the developed system is operationally acceptable and that the responsibility for the effective, efficient, and safe operations of the system is transferred to the owner. Considerations for deployment and use must be included throughout the system life cycle. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

System Deployment and Use is a SEBoK knowledge area; it includes systems engineering activities for system deployment and operation in its operational environment; and for the maintenance and logistics support of the system through life.

System Element (glossary)

A member of a set of elements that constitutes a system. A system element is a discrete part of a system that can be implemented to fulfill specified requirements. A system element can be hardware, software, data, humans, processes (e.g., processes for providing service to users), procedures (e.g., operator instructions), facilities, materials, and naturally occurring entities (e.g., water, organisms, minerals), or any combination. (ISO/IEC 15288:2015)

Source

ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Discussion

None.

System Hardware Assurance

- Lead Authors:
- Michael Bear, Donald Davidson, Shawn Fetterolf, Darin Leonhardt, Daniel Radack, Karen Johnson, and Elizabeth A. McDaniel
- Contributing Authors:
- Michael Berry, Brian Cohen, Diganta Das, Houman Homayoun, and Thomas McDermott

This article describes the discipline of hardware assurance, especially as it relates to systems engineering. It is part of the SE and Quality Attributes Knowledge Area.

Overview

System hardware assurance is a set of system security engineering activities (see System Security for more information) undertaken to quantify and increase the confidence that electronics function as intended and only as intended throughout their life cycle, and to manage identified risks. The term *hardware* refers to electronic components, sometimes called integrated circuits or chips. As products of multi-stage processes involving design, manufacturing and post-manufacturing, packaging, and test, they must function properly under a wide range of circumstances. Hardware components – alone and integrated into subcomponents, subsystems, and systems – have weaknesses and vulnerabilities enabling exploitation. Weaknesses are flaws, bugs, or errors in design, architecture, code, or implementation. Vulnerabilities are weaknesses that are exploitable in the context of use (Martin 2014).

Hardware assurance is conducted to minimize risks related to hardware that can enable adversarial exploitation and subversion of functionality, counterfeit production, and loss of technological advantage. Challenges include increasing levels of sophistication and complexity of hardware architectures, integrated circuits, operating systems, and application software, combined with supply chain risks, emergence of new attack surfaces, and reliance on global sources for some components and technologies.

After identifying concerns and applicable mitigations, hardware assurance offers a range of possible activities and processes. At the component level, hardware assurance focuses on the hardware itself and the supply chain used to design and manufacture it; at the subcomponent, subsystems, and system levels, hardware assurance incorporates the software and firmware integrated with the component.

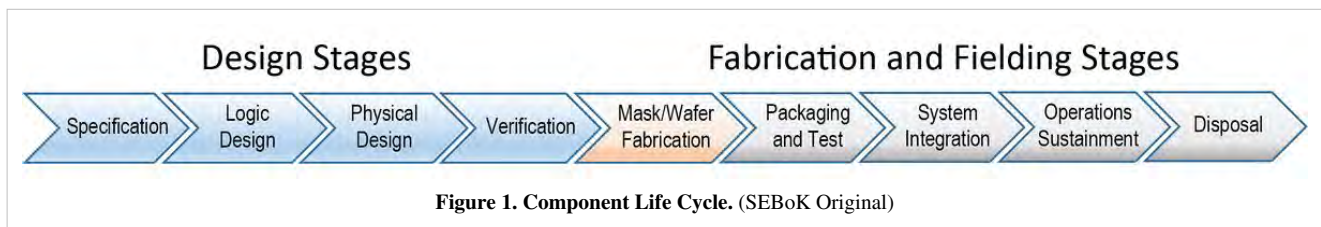
Engineering efforts to enhance trust in hardware have increased in response to complex hardware architectures, the increasing sophistication of adversarial attacks on hardware, and globalization of supply chains. These factors raise serious concerns about the security, confidentiality, integrity, and availability as well as the provenance and authenticity of hardware. The “root of trust” (NIST 2020) of a system is typically contained in the processes, steps, and layers of hardware components and across the systems engineering development cycle. System hardware assurance focuses on hardware components and their interconnections with software and firmware to reduce risks to proper function or other compromises of the hardware throughout the complete life cycle of components and systems. Advances in hardware assurance tools and techniques will strengthen designs, and enhance assurance during manufacturing, packaging, test, and deployment and operational use.

Life Cycle Concerns of Hardware Components

Hardware assurance should be applied at various stages of a component's life cycle from hardware architecture and design, through manufacturing and testing, and finally throughout its inclusion in a larger system. The need for hardware assurance then continues throughout its operational life including sustainment and disposal.

As semiconductor technology advances the complexity of electronic components, it increases the need to “bake-in” assurance. Risks created during architecture, design, and manufacturing are challenging to address during the operational phase. Risks associated with interconnections between and among chips are also a concern. Therefore, improving a hardware assurance posture must occur as early as possible in the life cycle, thereby reducing the cost and schedule impacts associated with “fixing” components later in the life cycle of the system.

A conceptual overview of the typical hardware life cycle (Figure 1) illustrates the phases of the life cycle of components, as well as the subsystems and systems in which they operate. In each phase multiple parties and processes contribute a large set of variables and corresponding attack surfaces. As a result, the potential exists for compromise of the hardware as well as the subcomponents and systems in which they operate; therefore, matching mitigations should be applied at the time the risks are identified.



Both the value of the hardware component and the associated cost of mitigating risks increase at each stage of the life cycle. Therefore, it is important to identify and mitigate vulnerabilities as early as possible. It takes longer to find and fix defects later, thereby increasing the complexity of replacing hardware with “corrected” designs that create system integration issues. In addition to cost savings, early correction and mitigation avoid delays in creating an operational system. It is essential to re-assess risks associated with hardware components throughout the life cycle periodically, especially as operational conditions change.

Hardware assurance during system sustainment is a novel challenge given legacy hardware and designs with their associated supply chains. In long-lived high-reliability systems, hardware assurance issues are compounded by obsolescence and diminished sourcing of components, thereby increasing concerns related to counterfeits and acquisitions from the gray market.

Function as Intended and Only as Intended

Exhaustive testing can check system functions against specifications and expectations; however, checking for unintended functions is problematic. Consumers have a reasonable expectation that a purchased product will perform as advertised and function properly (safely and securely, under specified conditions) – but consumers rarely consider if additional functions are built into the product. For example, a laptop with a web-conferencing capability comes with a webcam that will function properly when enabled, but what if the webcam also functions when turned off, thereby violating expectations of privacy? Given that a state-of-the-art semiconductor component might have billions of transistors, “hidden” functions might be exploitable by adversaries. The statement “function as intended and only intended” communicates the need to check for unintended functions.

Hardware specifications and information in the design phase are needed to validate that components function properly to support systems or missions. If an engineer creates specifications that support assurance that flow down the system development process, the concept of “function as intended” can be validated for the system and mission through accepted verification and validation processes. “Function only as intended” is also a consequence of capturing the requirements and specifications to assure the product is designed and developed without extra

functionality. For example, a Field Programmable Gate Array (FPGA) contains programmable logic that is highly configurable; however, the programmable circuitry might be susceptible to exploitation.

Given the specifications of a hardware component, specialized tools and processes can be used to determine with a high degree of confidence whether the component's performance meets specifications. Research efforts are underway to develop robust methods to validate that a component does not have capabilities that threaten assurance or that are not specified in the original design. Although tools and processes can test for known weaknesses, operational vulnerabilities, and deviations from expected performance, all states of possible anomalous behavior cannot currently be determined or predicted.

Data and information can be used to validate the component's function and should be collected from multiple sources including designers, developers, and members of the user community. Designers and developers can provide deep understanding of the component's intended function and provide tests used to verify its functional performance before fielding. The merging of component design and development information with extensive field data, including third-party evaluation, contributes to assurance that the component is performing specified functions and that no unintended functionality is observed.

Risks to Hardware

Modern systems depend on complex microelectronics, but advances in hardware without attention to associated risks can expose critical systems, their information, and the people who rely on them. "Hardware is evolving rapidly, thus creating fundamentally new attack surfaces, many of which will never be entirely secured". (Oberg 2020) Therefore, it is imperative that risk be modeled through a dynamic risk profile and be mitigated in depth across the entire profile. Hardware assurance requires extensible mitigations and strategies that can and do evolve as threats do. Hardware assurance methods seek to quantify and improve confidence that weaknesses that can become vulnerabilities that create risks are mitigated.

Most hardware components are commercially designed, manufactured, and inserted into larger assemblies by multi-national companies with global supply chains. Understanding the provenance and participants in complex global supply chains is fundamental to assessing risks associated with the components.

Operational risks that derive from unintentional or intentional features are differentiated based on the source of the feature. Three basic operational risk areas related to goods, products, or items are: failure to meet quality standards, maliciously tainted goods, and counterfeit hardware. Counterfeits are usually offered as legitimate products, but they are not. They may be refurbished or mock items made to appear as originals, re-marked products, the result of overproduction, or substandard production parts rejected by the legitimate producer. Counterfeit risks and substandard quality offer avenues for malware insertion and potential impacts to overall system performance and availability.

Failure to follow quality standards including safety and security standards, especially in design, can result in unintentional features or flaws being inadvertently introduced. These can occur through mistakes, omissions, or lack of understanding how features might be manipulated by future users for nefarious purposes. Features introduced intentionally for specific purposes can also make the hardware susceptible to espionage or control of the hardware at some point in its life cycle.

Quantify and Improve Confidence

The quantification of hardware assurance is a key technical challenge because of the complex interplay among designer, manufacturer and supply chains, and adversarial intent, as well as the challenge of defining "security" with respect to hardware function. Quantification is necessary to identify and manage hardware risks within program budgets and timeframes. It enables a determination of the required level of hardware assurance and whether quantification is achievable throughout the hardware's life cycle.

Current methods for quantifying hardware assurance are adapted from the fields of quality and reliability engineering, which use methods like Failure Mode and Effects Analysis (FMEA). (SAE 2021) FMEA is semi-quantitative and combines probabilistic hardware failure data and input from experts. Adapting FMEA to quantify hardware assurance is hampered when it relies on assigning probabilities to human behavior that may be motivated by money, malicious intent, etc. Expert opinion often varies when quantifying and weighting factors used in generating risk matrices and scores. In response, recent efforts are attempting to develop quantitative methods that reduce subjectivity.

Game theoretic analysis (game theory) is the creation of mathematical models of conflict and cooperation between intelligent and rational decision-makers. (Myerson 1991) Models include dynamic, as opposed to static, interactions between attackers and defenders that can quantify the risks associated with potential interactions among adversaries, hardware developers, and manufacturing processes. (Eames and Johnson 2017) Creation of the models forces one to define attack scenarios explicitly and to input detailed knowledge of hardware development and manufacturing processes. Outputs of the model may include a ranking of the most likely attacks to occur based on cost-benefit constraints on the attackers and defenders. (Graf 2017) The results can empower decision-makers to make quantitative trade-off decisions about hardware assurance.

Another quantification method that results in a confidence interval for detecting counterfeit/suspect microelectronics is presented in the SAE AS6171 standard. (SAE 2016) Confidence is based on knowing the types of defects associated with counterfeits, and the effectiveness of different tests to detect those defects. Along the same lines, a standard for hardware assurance might be developed to quantify the confidence interval by testing against a catalogue of known vulnerabilities, such as those documented in the MITRE Common Vulnerabilities and Exposures (CVE) list. (MITRE 2020)

Distributed ledger technology (DLT) is an example of an emerging technology that could enable a standardized approach for quantifying hardware assurance attributes such as data integrity, immutability, and traceability. DLT can be used in conjunction with manufacturing data (such as dimensional measurement, parametric testing, process monitoring, and defect mapping) to improve tamper resistance using component provenance and traceability data. DLT also enables new scenarios of cross-organizational data fusion, opening the door to new classes of hardware integrity checks.

Manage Risks

The selection of specific components for use in subsystems and systems should be the outcome of performance-risk and cost-benefit trade-off assessments in their intended context of use. The goal of risk management and mitigation planning is to select mitigations with the best overall operational risk reduction and the lowest cost impact. The required level of hardware assurance varies with the criticality of a component's use and the system in which it is used.

During a typical development life cycle of a system – architecture, design, code, and implementation – various types of problems can pose risks to the operational functionality of the hardware components provided. These risks include weaknesses or defects that are inadvertent (unintentional), as well as counterfeits that may be either inadvertent or intentionally injected into the supply chain for financial motivations or malicious components designed to change functionality.

Managing risk in the context of hardware assurance seeks to decrease the risk of weaknesses that create attack surfaces that can be exploited, while improving confidence that an implementation resists exploitation. Ideally, risk management reduces risk and maximizes assurance to an acceptable level. Often, risks are considered in the context of likelihood of consequences and the costs and effectiveness of mitigations. However, new operationally impactful risks are recognized continuously over the hardware life cycle and supply chains of components. At the same time hardware weaknesses are often exploited through software or firmware. Therefore, to maximize assurance and minimize operationally impactful risks mitigation-in-depth across all constituent components must be considered.

This highlights the need for a dynamic risk profile.

An example of a post-manufacturing mitigation involves a new hardware risk identified during field operation. A dynamic risk profile can be used to characterize the issue and identify possible resources to address the suspect component function. This profile can also be used to track and address risks throughout its life, including obsolescence-related risk. One means of mitigating this kind of hardware life cycle risk is the use of existing programmable logic.

Just as with software patches and updates, new attack surfaces on hardware may become exposed through the mitigation being applied, and they will likely take a long time to discover. In the example above, the programmable logic is updated to provide a new configuration to protect the hardware. In this context, access to hardware reconfiguration must be limited to authorized parties to prevent an unauthorized update that introduces weaknesses on purpose or by accident. While programmable logic may have mitigated a specific attack surface or type of weakness, additional mitigations are needed to minimize risk more completely. This is mitigation-in-depth – multiple mitigations building upon one another.

Throughout the entire supply chain, critical pieces of information can be inadvertently exposed. The exposure of such information directly enables the creation and exploitation of new attack surfaces. Therefore, the supply chain infrastructure must also be assessed for weaknesses, and the development, use, and maintenance of hardware components assured. The dynamic risk profile offers a framework to balance mitigations in the context of risk and cost throughout the complete hardware and system life cycles.

References

Works Cited

- Eames, B.K. and M.H. Johnson. 2017. "Trust Analysis in FPGA-based Systems." Proceeding of GOMACTech 2017, March 20-23, 2017, Reno, NV.
- Graf, J. 2017. "OpTrust: Software for Determining Optimal Test Coverage and Strategies for Trust." Proceedings of GOMACTech 2017, March 20-23, 2017, Reno, NV.
- Martin, R.A. 2014. "Non-Malicious Taint: Bad Hygiene is as Dangerous to the Mission as Malicious Intent." CrossTalk Magazine. 27(2).
- MITRE. 2020. "Common Vulnerabilities and Exposures." Last Updated December 11, 2020. Accessed March 31, 2021. Available at <https://cve.mitre.org/cve/>,
- Myerson, R.R. 1991. *Game Theory: Analysis of Conflict*. Cambridge, MA: Harvard University Press.
- NIST. 2020. Roots of Trust. Last Updated June 22, 2020. Accessed March 31, 2021. Available at <https://csrc.nist.gov/projects/hardware-roots-of-trust>.
- Oberg, J. 2020. Reducing Hardware Security Risk. Last Updated July 1, 2020. Accessed March 31, 2021. Available at: <https://semiengineering.com/reducing-hardware-security-risk/>.
- SAE. 2016. SAE AS6171, *Test Methods Standard: General Requirements, Suspect/Counterfeit, Electrical, Electronic, and Electromechanical Parts*. SAE International. Accessed March 31, 2021. Available at <https://www.sae.org/standards/content/as6171/>.
- SAE. 2021. SAE J1739_202101, *Potential Failure Mode and Effects Analysis (FMEA) Including Design FMEA, Supplemental FMEA-MSR, and Process FMEA*. SAE International. Last Updated January 13, 2021. Accessed on March 31, 2021. Available at https://www.sae.org/standards/content/j1739_202101/.
-

Primary References

- Bhunias, S. and M. Tehranipoor. 2018. *Hardware Security: A Hands-on Learning Approach*. Amsterdam, Netherlands: Elsevier Science.
- ENISA. 2017. *Hardware Threat Landscape and Good Practice Guide. Final Version 1.0*. European Union Agency for Cybersecurity. Accessed March 31, 2021. Available at <https://www.enisa.europa.eu/publications/hardware-threat-landscape>
- TAME Steering Committee. 2019. *Trusted and Assured Microelectronics Forum Working Group Reports*. Last Updated December 2019. Accessed March 31, 2021. Available at: <https://dforte.ece.ufl.edu/wp-content/uploads/sites/65/2020/08/TAME-Report-FINAL.pdf>.

Additional References

- DARPA. A DARPA Approach to Trusted Microelectronics. Accessed March 31, 2021. Available at: https://www.darpa.mil/attachments/Background_FINAL3.pdf ^[1].
- Fazzari, S. and R. Narumi. 2019. *New & Old Challenges for Trusted and Assured Microelectronics*. Accessed March 31, 2021. Available at: <https://apps.dtic.mil/dtic/tr/fulltext/u2/1076110.pdf>.
- IEEE. 2008-2020. IEEE International Symposium on Hardware Oriented Security and Trust (HOST). Annual symposium held since 2008 providing wealth of articles on hardware assurance.
- Martin, R. 2019. "Hardware Assurance and Weakness Collaboration and Sharing (HAWCS)." Proceedings of the 2019 Software and Supply Chain Assurance Forum, September 17-18, 2019 in McLean, VA. Accessed March 31, 2021. Available at https://csrc.nist.gov/CSRC/media/Projects/cyber-supply-chain-risk-management/documents/SSCA/Fall_2019/WedPM2.2_Robert_Martin.pdf.
- NDIA. 2017. Trusted Microelectronics Joint Working Group: Team 3 White Paper: Trustable Microelectronics Standard Products. Accessed March 31, 2021. Available at <https://www.ndia.org/-/media/sites/ndia/divisions/working-groups/tmjwg-documents/ndia-tm-jwg-team-3-white-paper-finalv3.ashx>.
- Regenscheid, A. 2019. NIST SP 800-193, *Platform Firmware Resiliency Guidelines*. Accessed March 31, 2021. Available at <https://csrc.nist.gov/publications/detail/sp/800-193/final>.
- Ross, R., V. Pillitteri, R. Graubart, D. Bodeau, R. McQuaid. 2019. NIST SP 800-160 Vol. 2, *Developing Cyber Resilient Systems – A Systems Security Engineering Approach*. Accessed March 31, 2021. Available at: <https://csrc.nist.gov/News/2019/sp-800-160-vol2-developing-cyber-resilient-systems>.

References

- [1] <https://www.darpa.mil/about-us/darpa-approach-to-trusted-microelectronics>

System of Systems (SoS) (glossary)

(1) *Two or more systems that are separately defined but operate together to perform a common goal.* (Checkland 1999)

(2) *an assemblage of components which individually may be regarded as systems, and which possess two additional properties: (a) Operational Independence of the Components: If the system-of-systems is disassembled into its component systems the component systems must be able to usefully operate independently. That is, the components fulfill customer-operator purposes on their own.*

(b) Managerial Independence of the Components: The component systems not only can operate independently, they do operate independently. The component systems are separately acquired and integrated but maintain a continuing operational existence independent of the system-of-systems. (Maier 1998, 267-284)

(3) *System-of-systems applies to a system-of-interest whose system elements are themselves systems; typically these entail large scale inter-disciplinary problems with multiple, heterogeneous, distributed systems.* (INCOSE 2012)

Source

(1) Checkland, P. B. 1999. *Systems Thinking, Systems Practice*. Chichester, UK: John Wiley & Sons Ltd.

(2) Maier, M. W. 1998. "Architecting principles for systems-of-systems." *Systems Engineering, the Journal of the International Council on Systems Engineering (INCOSE)* 1 (4).

(3) INCOSE. 2012. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2

Discussion

e the Systems of Systems Knowledge Area in Part 4: Applications of Systems Engineering.e

System Property (glossary)

Any named, measurable or observable attribute, quality or characteristic of a system or system element.
(OMG 2003)

Source

Object Management Group. 2003. "Property." In *SE Conceptual Model Semantic Dictionary (Draft 12_03-27-03)*. Available at https://syseng.omg.org/SE_Conceptual%20Model/Draft%2012/Concept%20Model%20Semantic%20Dictionary%2012th%20Draft%20Partitioned%203_27_03.xls.^[1]

Discussion

None

References

- [1] https://syseng.omg.org/SE_Conceptual%20Model/Draft%2012/Concept%20Model%20Semantic%20Dictionary%2012th%20Draft%20Partitioned%203_27_03.xls

System Realization (glossary)

The activities required to build a system, integrate disparate system elements, and ensure that a system both meets the needs of stakeholders and aligns with the requirements identified in the system definition stage. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

"System realization" is used in the SEBoK as a knowledge area (KA); it includes several systems engineering (SE) activities such as system implementation, integration, verification, and validation. (See System Realization KA.) This generally aligns with the "development" and "production" stages as discussed in the International Council on Systems Engineering (INCOSE) Systems Engineering Handbook (INCOSE 2012).

Work Cited

INCOSE. 2012. Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

System Requirement (glossary)

(1a) *A statement that transforms the stakeholder, user-oriented view of desired capabilities into a technical view, of a solution that meets the operational needs of the user. (ISO/IEC/IEEE, 2015, section 6.4.2)*

(1b) *The System Requirement Definition process creates a set of measurable system requirements that specify, from the supplier's perspective, what characteristics, attributes, and functional and performance requirements the system is to possess, in order to satisfy stakeholder requirements. (ISO/IEC/IEEE, 2015, section 6.4.2)*

Source

(1) ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015 (E).

Discussion

Both definitions are modified from the introduction to the Requirements Analysis process, there is no formal definition of the term "system requirement" in the standard.

The term (engineered system) has been substituted for the word (product) in definition 1a to better align with the terminology used in the SEBoK.

For a full discussion of the role and importance of system requirements in systems engineering see the System Requirements article.

System-of-Interest (glossary)

(1) *The system whose life cycle is under consideration. (ISO/IEC/IEEE 2015)*

(2) *The system of interest to an observer. (Bertalanffy 1968)*

Source

(1) ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC), Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015. The second definition is an expanded version of the ISO/IEC/IEEE version.

(2) von Bertalanffy, L. 1968. *General system theory: Foundations, development, applications*, revised ed. New York, NY: Braziller.

Discussion

(1) Is the common Systems Engineering definition. A system-of-interest is a collective set of all elements of any system considered by a lifecycle, this may include both operational or enabling systems. Strictly this definition is relevant for an Engineered System (glossary) of interest.

(2) Is the system science definition and applies to all systems irrespective of whether they have a purpose or lifecycle in an engineering sense.

Systemist (glossary)

A systemist (from Greek chēm(ia) alchemy; replacing chymist from Medieval Latin alchimista) is a scientist trained in the study of System Science.

Source

Daniel-Allegro B, Smith G.R. (2016). Exploring the branches of the system landscape, Les editions Allegro Brigitte D. ISBN 978-2-9538007-1-5.

Gary Smith (INCOSE International Workshop 2020) <https://docs.google.com/viewer?a=v&pid=sites&srcid=ZGVmYXVsdGRvbWFpbnc3dnMjAyMHxneDo1ODcwN2Q0NzdkMDA1YmRj>

Discussion

“We use this term for people who have the capabilities to understand complex situations, to cope with the unusual, to identify the right problem in order to solve it and who are able to take decisions keeping in mind a vision for the future” (Brigitte Daniel-Allegro and Gary Smith 2016). “The chemist uses their understanding of chemical elements when they interact with chemical substances. Likewise, in the future the systemist will use their understanding of systemics when they manipulate and transform systems” (Gary Smith 2020)

Systems Approach (glossary)

A set of principles for applying systems thinking to engineered system contexts. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

This definition has been created from a number of relevant sources on Systems Thinking, see the Systems Approach Applied to Engineered Systems knowledge area.

Systems Biology (glossary)

Systems biology is the computational and mathematical modeling of complex biological systems.

Source

https://en.wikipedia.org/wiki/Systems_biology

Discussion

An emerging engineering approach applied to biological scientific research, systems biology is a biology-based inter-disciplinary field of study that focuses on complex interactions within biological systems, using a holistic approach (holism instead of the more traditional reductionism) to biological research. Particularly from year 2000 onwards, the concept has been used widely in the biosciences in a variety of contexts. For example, the Human Genome Project is an example of applied systems thinking in biology which has led to new, collaborative ways of working on problems in the biological field of genetics. One of the outreaching aims of systems biology is to model and discover emergent properties, properties of cells, tissues and organisms functioning as a system whose theoretical description is only possible using techniques which fall under the remit of systems biology. These typically involve metabolic networks or cell signaling networks. (Wikipedia, "Systems Biology")

Systems Concept (glossary)

A mode of description, which explains an aspect of an object in terms of a set of interacting elements. The object can, in principle, be anything: a physical object, a body of work, an idea, or an enterprise. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

For a full discussion of the role and importance of systems concepts in systems engineering see the System Concept Definition article.

Systems Development (glossary)

A process that usually includes requirements analysis, system design, implementation, documentation and quality assurance. (ISO/IEC 1990)

Sources

ISO/IEC. 1990. *Information technology--Vocabulary--Part 20: System development*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC 2382-20:1990.

Discussion

None.

Systems Engineer (glossary)

A systems engineer is "a person who practices systems engineering" and whose systems engineering capabilities and experience include sustained practice, specialization, leadership or authority over systems engineering activities. Systems engineering activities may be conducted by any competent person regardless of job title or professional affiliation. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

Systems engineer is not a term in ISO/IEC/IEEE 24865:2010. The INCOSE website has a description but not a definition:

Systems engineers concentrate their efforts on the aspects of the engineering process (requirements definition, top-level functional designs, project management, life cycle cost analysis,...) that serve to organize and coordinate other engineering activities. The systems engineer is the primary interface between management, customers, suppliers, and specialty engineers in the systems development process.

Systems Engineering (glossary)

The definition of systems engineering has evolved over time. The current accepted definitions are found below:

(1) *Interdisciplinary approach governing the total technical and managerial effort required to transform a set of customer needs, expectations, and constraints into a solution and to support that solution throughout its life. (ISO/IEC/IEEE 2010)*

(2) *An interdisciplinary approach and means to enable the realization of successful systems. It focuses on defining customer needs and required functionality early in the development cycle, documenting requirements, then proceeding with design synthesis and system validation while considering the complete problem:*

- *Operations*
- *Performance*
- *Test*
- *Manufacturing*
- *Cost & Schedule*
- *Training & Support*
- *Disposal*

Systems engineering integrates all the disciplines and specialty groups into a team effort forming a structured development process that proceeds from concept to production to operation. Systems engineering considers both the business and the technical needs of all customers with the goal of providing a quality product that meets the user needs. (INCOSE 2012)

(3) *A transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems, using systems principles and concepts, and scientific, technological, and management methods.*

We use the terms “engineering” and “engineered” in their widest sense: “the action of working artfully to bring something about”. “Engineered systems” may be composed of any or all of people, products, services, information, processes, and natural elements. (INCOSE Fellows 2019)

Source

(1) ISO/IEC/IEEE. 2010. *Systems and Software Engineering - System and Software Engineering Vocabulary (SEVocab)*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2010.

(2) INCOSE. 2012. *INCOSE Systems Engineering Handbook*, version 3.2.2. San Diego, CA, USA: International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-002-03.2.2.

(3) INCOSE Fellows, Briefing to INCOSE Board of Directors, January 2019.

Discussion

There are many definitions of systems engineering. The SEBOK presents three of those definitions. Two come from foundational documents.

Note that both stress the interdisciplinary nature of systems engineering and its application throughout the life of the system.

In the Systems Engineering Overview article, the authors modified the INCOSE definition, saying traditional definitions of SE have emphasized sequential performance of SE activities, e.g., “documenting requirements, then proceeding with design synthesis ...”. (INCOSE 2012) The SEBoK authors depart from tradition to emphasize the inevitable intertwining of system requirements definition and system design in the following revised definition of SE:

Systems Engineering (SE) is an interdisciplinary approach and means to enable the realization of successful systems. It focuses on holistically and concurrently understanding stakeholder needs; exploring opportunities; documenting requirements; and synthesizing, verifying, validating, and evolving solutions while considering the complete problem, from system concept exploration through system disposal.

Definition (3) was created by the INCOSE Fellows Initiative on System and Systems Engineering Definitions. This was established in 2016, to review current INCOSE definitions of SYSTEM and SYSTEMS ENGINEERING and to recommend any changes necessary to align the definitions to a) current practice, and b) the aspirations of INCOSE's 2025 Vision. At the January 2019 INCOSE Board of Directors meeting, a new INCOSE definition for "system" was approved and is given above.

The Fellows Initiative on System and Systems Engineering Definitions was established in 2016, to review current INCOSE definitions of SYSTEM and SYSTEMS ENGINEERING and to recommend any changes necessary to align the definitions to a) current practice, and b) the aspirations of INCOSE's 2025 Vision. The INCOSE Fellows elaborated this definition by stating: We use the terms “engineering” and “engineered” in their widest sense: “the action of working artfully to bring something about”. “Engineered systems” may be composed of any or all of people, products, services, information, processes, and natural elements.

Systems Engineering Management (SEM) (glossary)

Managing the resources and assets allocated to perform systems engineering activities. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

None.

Systems Engineering Plan (SEP) (glossary)

A systems engineering plan (SEP) is a "living" document that captures a program's current and evolving systems engineering strategy and its relationship with the overall program management effort. The SEP purpose is to guide all technical aspects of the program. It should be established early in the Materiel Solution Analysis phase and updated continually. (DAU 2012)

Source

DAU 2012. "Systems Engineering Plan (SEP)." (online). Virginia, USA: ACQuipedia. Your Online Acquisition Encyclopedia. Accessed on 20 October 2019. Available at: <https://www.waru.edu/acquipedia-article/systems-engineering-plan-sep>.

Discussion

In many contexts, the Systems Engineering Plan (SEP) and Systems Engineering Management Plan (SEMP) are used interchangeably, to refer to the highest level technical plan. However, in the context of the U.S. Department of Defense the context, intent, and content of these documents are quite different:

- The SEP is a high-level plan that is made before the system acquisition and development begins. It is written by the government customer.
- The SEMP is the specific development plan written by the developer (or contractor).

Systems Integration (glossary)

A broad topic that includes hardware, software, and human systems and which uses an interdisciplinary approach for a structured, disciplined, and documented technical effort to simultaneously design and develop systems products and processes used to create cohesive systems. (Created for SEBoK)

Source

This definition was developed for the SEBoK.

Discussion

None.

Systems Science (glossary)

Systems science is a transdisciplinary field of formal inquiry concerned with the general principles, patterns, and processes by which systems form, persist, interact, adapt, and evolve across natural, social, engineered, and conceptual domains. It seeks to develop, test, and integrate concepts, models, and principles that support understanding of systems across disciplinary boundaries - "The systems sciences comprise a transdisciplinary area of formal inquiry aimed toward general theory development, testing, and validation." (Laszlo, Krippner 1998).

Source

Laszlo, A., and S. Krippner. 1998. "Systems Theories: Their Origins, Foundations, and Development." In *Systems Theories and A Priori Aspects of Perception*, edited by J. S. Jordan, 47–74. Amsterdam, Netherlands: Elsevier Science.

Discussion

Please see the Systems Science knowledge area.

Systems Thinking (glossary)

(1) *An epistemology which, when applied to human activity is based on four basic ideas: emergence, hierarchy, communication, and control as characteristics of systems. (Checkland 1999)*

(2) *A process of discovery and diagnosis – an inquiry into the governing processes underlying the problems and opportunities. (Senge 1990)*

(3) *A discipline for examining wholes, interrelationships, and patterns utilizing a specific set of tools and techniques. (Senge 1990)*

Source

(1) Checkland, P. 1999. *Systems Thinking, Systems Practice*. New York, NY, USA: John Wiley & Sons.

(2) & (3) Senge, P.M. 1990. *The fifth discipline: The Art & Practice of the Learning Organization*. New York, NY, USA: Doubleday Business.

Discussion

Definition (1) is the System Science view, defining system thinking as a "theory of knowledge, esp. with regard to its methods, validity, and scope", based around seeing the world as systems.

Definitions (2) and (3) focus more on systems thinking as a collection of methods for dealing with system problems. This aspect of systems thinking relates directly to the Systems Approach defined in the SEBoK.

See also the topic [What is Systems Thinking?](#)

T

Tailoring (glossary)

Adapted from notes/discussion of "tailoring guide" (ISO/IEC/IEEE 2009): Tailoring a process adapts the process description for a particular end. For example, a project creates its defined process by tailoring the organization's set of standard processes to meet the objectives, constraints, and environment of the project. (ISO/IEC/IEEE 2009)

Sources

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

None.

Task (glossary)

According to ISO/IEC/IEEE 24774, a task is defined as a "required, recommended, or permissible action, intended to contribute to the achievement of one or more outcomes of a process". (ISO/IEC/IEEE 2021)

Sources

ISO/IEC/IEEE. 2021. *Systems and software engineering — Life cycle management — Specification for process description*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24774:2021, Clause 3.20.

Discussion

None.

Team (glossary)

A group of people with a full set of complementary skills required to complete a task, job, or project. Team members (1) operate with a high degree of interdependence, (2) share authority and responsibility for self-management, (3) are accountable for the collective performance, and (4) work toward a common goal and shared reward(s). A team becomes more than just a collection of people when a strong sense of mutual commitment creates synergy, thus generating performance greater than the sum of the performances of its individual members. (WebFinance 2012)

Sources

WebFinance. 2012. "dictionary.com" (online). Washington, DC, USA: WebFinance, Inc. Accessed on 11 September 2012. Available at: <https://www.businessdictionary.com/definition/team.html>.

Discussion

This definition encompasses the key elements of "team." Other characteristics of systems engineering teams are a sense of shared commitment to achieving the end result, shared work products, and willingness to help one another. As pointed out in the definition, a team is more than a collection, or group of people. A systems engineering team is a group of individuals who cooperatively perform a collection of systems engineering tasks based on a shared vision and a common set of engineering objectives.

Technical Management (glossary)

The purpose of Technical Management is to steer the technical effort (commencing when the opportunity is identified) along the correct technical path to solution. Technical Management ensures integration (provides a bridge) between business / project management and all technical activity. Technical Management determines how the system approach can be carried out most effectively and efficiently in the attainment of value, cost targets and achievement of time-to-market. (INCOSE 2020)

Source

INCOSE. 2020. "PM-SE Integration Working Group." <https://www.incose.org/incose-member-resources/working-groups/process/pm-se-integration> ^[1]

Discussion

This is a primary topic of interest for the INCOSE PM/SE Integration Working Group.

SEBoK v. 2.3, released 30 October 2020

References

[1] <https://www.incose.org/incose-member-resources/working-groups/process/pm-se-integration>

Technical Performance Measure (TPM) (glossary)

(1) Measures of attributes of a system element within the system to determine how well the system or system element is satisfying specified requirements. (Roedler and Jones 2005, 1-65)

(2) A product design assessment, which estimates, through engineering analysis and tests, the values of essential performance parameters of the current design as effected by risk handling actions. (BMP 2012)

Source

(1) Roedler, G. J., and C. Jones. 2005. *Technical Measurement: A Collaborative Project of PSM, INCOSE, and Industry*. San Diego, CA, USA: Practical Software & Systems Management (PSM)/International Council on Systems Engineering (INCOSE), INCOSE-TP-2003-020-01.

(2) BMP. 2012. "2.8 Risk Monitoring." Accessed September 13, 2012. Available at: <https://www.bmpcoe.org/library/books/dsmc%20rmg%204th/69.html>.

Discussion

None.

Temporal Architecture (glossary)

A temporal classification of the functions of a system according to the frequency level of execution. Temporal architecture includes the definition of synchronous and asynchronous aspects of functions. (Created for SEBoK)

Sources

This definition was developed for SEBoK.

Discussion

The definition provided here is consistent with a multiple view of a system logical architecture that should include as a minimum functional, behavioral and temporal views.

Test Cases (glossary)

A set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement. (IEEE 2004)

Sources

IEEE. 2004. *IEEE Standard for Software Verification and Validation*. Washington, DC, USA: Institute of Electrical and Electronics Engineers (IEEE). IEEE 1012-2004.

Discussion

None.

Threat (glossary)

State of the system or system environment which can lead to adverse effects. (ISO/IEC/IEEE 2011)

Sources

ISO/IEC/IEEE. 2011. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765a:2011.

Discussion

None.

Tolerance (glossary)

a resilience attribute that describes the ability of the system to degrade gracefully in the face of a threat. (Jackson and Ferris, 2013)

How a system behaves near a boundary – whether the system gracefully degrades as stress or pressure increases or collapses quickly when pressure exceeds adaptive capacity (Woods 2006, 23)

Sources

Woods, D.D. 2006. "Essential Characteristics of Resilience." In *Resilience Engineering: Concepts and Precepts*, edited by E. Hollnagel, DD. Woods, and N. Leveson. Aldershot, UK: Ashgate Publishing Limited.

Jackson, Scott, and Timothy Ferris. 2013. "Resilience Principles for Engineered Systems." *Systems Engineering* 16 (2):152-164.

Discussion

None.

Total Ownership Cost (glossary)

The sum of all financial resources necessary to organize, sustain, and operate military forces sufficient to meet national goals in compliance with all laws, all policies applicable to DoD, all standards in effect for readiness, safety, and quality of life, and other official measures of performance in effect for DoD and its Components. (Miller 2007)

Sources

Miller, K. 2007. DoD Definition of Total Ownership Cost (TOC). Los Angeles: University of Southern California.

Discussion

The term "total ownership cost" is a term used primarily but not exclusively by the US Department of Defense (DoD). The quote was extracted from a presentation by Dr. Karen Miller a lecturer at the University of Southern California and Loyola Marymount University, both in Los Angeles. Dr. Miller is an authority on system affordability which includes the concept of total ownership cost.

Traceability (glossary)

The degree to which a relationship can be established between two or more products of the development process, especially products having a predecessor-successor or master-subordinate relationship to one another (IEEE 1233-1998 (R2002) IEEE Guide for Developing System Requirements Specifications, 3.1.9)

Source[s]

IEEE. 1998. *IEEE Guide for Developing System Requirements Specifications*. New York, NY: Institute of Electrical and Electronics Engineers (IEEE), IEEE 1233-1998.

Discussion

SEVOCAB lists other, similar definitions.

Training (glossary)

Provision of formal and informal learning activities. (ISO/IEC/IEEE 2009)

Sources

ISO/IEC/IEEE. 2009. *Systems and Software Engineering - System and Software Engineering Vocabulary*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 24765:2009.

Discussion

None.

Transition of Modes (glossary)

A transition of mode characterizes the transition (change of state) between two successive Operational Modes of a system. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

None.

Transport network (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

U

Uncertainty (glossary)

The result of not having accurate or sufficient knowledge of a situation. (ISO/IEC 2009, 1)

Source

ISO/IEC. 2009. *Systems and Software Engineering Vocabulary (SEVocab)* - ISO/IEC 24765. in International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) [database online]. Geneva, Switzerland, 2009 [cited December 21 2009]. Available from https://pascal.computer.org/sev_display/index.action.

Discussion

None.

Unitary (glossary)

A problem situation in which participants "have similar values, beliefs and interests. They share common purposes and are all involved, in one way or another, in decision-making about how to realize their agreed objectives." (Jackson 2003, 19)

Sources

Jackson, M. 2003. *Systems Thinking: Creating Holisms for Managers*. Chichester, UK: J. Wiley.

Discussion

None.

Use Case (glossary)

Description of the behavioral requirements of a system and its interaction with a user. (ISO/IEC/IEEE 2011)

Sources

ISO/IEC/IEEE. 2011. *Systems and software engineering: Developing user documentation in an agile environment*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/ Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 26515:2011.

Discussion

None.

Useful life (glossary)

The period of time during which an asset or property is expected to be usable for the purpose it was acquired. It may or may not correspond with the item's actual physical life or economic life. (WebFinance 2012)

Sources

WebFinance. 2012. "dictionary.com" (online). Washington, DC, USA: WebFinance, Inc. Accessed on 11 September 2012. Available at: <https://www.businessdictionary.com/definition/useful-life.html>

Discussion

None.

User (glossary)

Individual or group that interacts with a system or benefits from a system during its utilization.
(ISO/IEC/IEEE 2015)

Source

ISO/IEC/IEEE. 2015. Systems and Software Engineering -- System Life Cycle Processes. Geneva, Switzerland: International Organisation for Standardisation / International Electrotechnical Commissions / Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 15288:2015.

Discussion

None.

Utility network (glossary)

Insert definition here. (Reference YEAR)

Source

Insert full source info (Author Last, FI. YEAR. TITLE. Publisher. URL)

Annotation

If there is discussion around why this is the appropriate definition, include here. If not, please replace with "None".

V

Validation (glossary)

(1a) *Confirmation, through the provision of objective evidence, that the (stakeholder) requirements for a specific intended use or application have been fulfilled.* (ISO 9000:2015)

(1b) *The set of activities ensuring and gaining confidence that a system is able to accomplish its intended use, goals and objectives (i.e., meet stakeholder requirements) in the intended operational environment. The right system was built.* (ISO/IEEE 2015, 1, Section 6.4.8)

(2) *The assurance that a product, service, or system meets the needs of the customer and other identified stakeholders. It often involves acceptance and suitability with external customers. Contrast with verification.* (PMI 2013)

(3) *The process of providing evidence that the software and its associated products satisfy system requirements allocated to software at the end of each life cycle activity, solve the right problem, and satisfy intended use and user needs.* (IEEE 1012-2004, 3.1.35)

Source

- (1) ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015 (E).
- (2) PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).
- (3) IEEE. 2004. *IEEE Standard for Software Verification and Validation*. Institute of Electrical and Electronics Engineers (IEEE) Standards Association: 3.1.35. IEEE 1012-2004.

Discussion

Definition (1a) refers to the outcome of providing evidence that a particular system realization is validated (i.e., Does it satisfy the customer and user needs as stated and agreed?). The word (stakeholder) has been added to clarify the definition.

Definition (1b) is based on the introduction to the validation process and refers to the process of achieving validation through a set of activities conducted across a system's Life Cycle (glossary) to ensure that the system that has been built will serve its intended purpose. The term (engineered) system has been added to conform to SEBoK terminology.

Definition (3) refers to the validation of software components in terms of satisfying both allocated system requirements as well as user needs.

Validation builds on the activities and outcome of verification, a process that confirms that the system has been built correctly (i.e., Does it satisfy the system requirements?).

For a full discussion of the role and importance of validation in systems engineering see the System Validation article.

Value (glossary)

(1) *Value: the regard, merit, importance or worth given to something. It is the basis for showing a preference i.e. making a choice.* (Penguin Dictionary of Civil Engineering)

(2) *Numerical or categorical result assigned to a base measure, derived measure or indicator.* (PSM 2010; ISO/IEC/IEEE 2007)

(3) *A measure of worth (e.g., benefit divided by cost) of a specific product or service by a customer, and potentially other stakeholders;* (McManus 2005)

Note: "Values" are shared beliefs about what an organization or its members consider to be important.

(1) A personal and/or cultural value is an absolute or relative ethical value, the assumption of which can be the basis for ethical action. A value system is a set of consistent values and measures. (Wikipedia) (2)

Values are the principles or standards of behavior. The things to which we give value and which determine how we behave. Some important personal values are truth, honesty, trust, respect for others and for the environment, fairness, making what we do enjoyable for others with whom we interact, openness, competence, sustainability, balance, harmony, reasonableness. Wherever possible key performance indicators (KPIs) should reflect the relevant values within a process of all stakeholders. In this way you have a better chance of driving out waste and adding value to all of those involved. See ethics, worth. – see ethics, morals, codes of conduct. (Penguin Dictionary of Civil Engineering)

Source

(1) Penguin Dictionary of Civil Engineering

(2) PSM. 2010. *Practical Software and Systems Measurement: A Foundation for Objective Project Management*. version 4.0 ed. Bethesda, MD, USA: Practical Software and Systems Measurement (PSM).

ISO/IEC/IEEE. 2007. *Measurement Process*. Geneva, Switzerland: International Organization for Standards (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical & Electronics Engineers (IEEE), ISO/IEC/IEEE 15939.

(3) McManus, H. L. 2005. *Product Development Value Stream Mapping Manual*, release 1.0. Boston, MA, USA: Lean Aerospace Initiative (LAI)/Massachusetts Institute of Technology (MIT), PVDSM Manual 1.0.

Discussion

In systems engineering we generally use the notion of "value" to describe the amount of benefit some thing, feature or capability provides to various stakeholders. The term "worth" is often used in the same way, as a measure of benefit or "how much someone would pay for" something. From this viewpoint the McManus definition would be considered to be "value for money" rather than "value" per se.

In the SEBoK the need to clearly separate the notions of value and cost is stressed upon because the systems engineer needs to be clear on the distinction between value (or benefit), which is an attribute of the problem space and considers the optimum set of requirements to deliver customer satisfaction, and cost, which is an attribute of the solution, and may be different for different solutions to the same set of requirements. The concept of *value stream analysis* can be used to understand and optimize added value and associated costs through the value stream.

Systems Engineers also use "value" in the PSM sense of "the value of a given attribute".

Variety (glossary)

(1) *A measure of the connectivity between system elements* (Ashby 1956)

(2) *Interacting systems stability increases with variety, and with the degree of connectivity of that variety within the environment* (Hitchins 2007)

Sources

(1) Ashby, W.R. 1956. *Introduction to Cybernetics*. London, UK; J. Wiley.

(2) Hitchins, D. 2007. *Systems Engineering: A 21st Century Systems Methodology*. Hoboken, NJ, USA: Wiley .

Discussion

(1) Is a cybernetic term used to define the number of available connections between elements, and thus the potential routes for control actions.

(2) Is a systems engineering definition, in which variety of connections between elements is used as a way to increase the stability and potential of a system to achieve its goals. Hitchins refers to this as Connected Variety, and also discussed the limits on this connectivity applied by availability of time, space, resource.

Vee (V) Model (glossary)

The technical aspect of the project cycle is envisioned as a "Vee", starting with User needs on the upper left and ending with a User-validated system on the upper right. (Forsberg and Mooz 1991)

Source

Mooz, H. and K. Forsberg. 1991. "The Relationship of Systems Engineering to the Project Cycle", Joint Conference of INCOSE and the American Society for Engineering Management, 21-23 October 1991, Chattanooga, TN. Available at: <https://onlinelibrary.wiley.com/doi/abs/10.1002/j.2334-5837.1991.tb01484.x>. Accessed September 11, 2012.

Discussion

None.

Verification (glossary)

(1a) *Confirmation, through the provision of objective evidence, that specified (system) requirements have been fulfilled. (ISO/IEC 2008, section 4.38)*

(1b) *Verification is a set of activities that compares a system or system element against the required characteristics. This includes, but is not limited to, specified requirements, design description and the system itself. The system was built right. (ISO/IEC/IEEE 2015, 1, Section 6.4.6)*

(2) *The evaluation of whether or not a product, service, or system complies with a regulation, requirement, specification, or imposed condition. It is often an internal process. Contrast with validation. (PMI 2013)*

(3a) *The process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase. (IEEE 1012-2004, 3.1.36)*

(3b) *Process of providing objective evidence that the software and its associated products comply with requirements for all life cycle activities during each life cycle process, satisfy standards, practices, and conventions during life cycle processes, and successfully complete each life cycle activity and satisfy all the criteria for initiating succeeding life cycle activities. (IEEE 829-2008, 3.1.54)*

Source

(1) ISO/IEC/IEEE. 2015. *Systems and Software Engineering - System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC)/Institute of Electrical and Electronics Engineers (IEEE). ISO/IEC/IEEE 15288:2015 (E).

(2) PMI. 2013. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. 5th ed. Newtown Square, PA, USA: Project Management Institute (PMI).

(3) IEEE. 2004. *IEEE Standard for Software Verification and Validation*. Institute of Electrical and Electronics Engineers (IEEE) Standards Association: IEEE 1012-2004.

Discussion

Definition (1a) refers to the outcome of providing evidence that a particular system realization is verified (i.e. Does it satisfy the specified and agreed system requirements?). The word (system) has been added to clarify the definition.

Definition (1b) is based on the introduction to the verification process and refers to the process of achieving verification through a set of activities conducted across a system's Life Cycle (glossary) to ensure the system has been built correctly. The term (engineered) system has been added to conform to SEBoK terminology.

Definition (3) refers to verification at the end of each lifecycle stage that confirms that both software and systems have been developed in compliance with all standard practices and rules.

Verification supports the activities and outcome of validation, a process that ensures that the correct system has been built for its intended use (i.e., Does it satisfy the customer and user needs?).

For a full discussion of the role and importance of verification in systems engineering see the System Verification article.

Verification and Validation Action (glossary)

A verification and validation action describes what must be verified or validated (the element as reference), on which element, the expected result, the validation/verification technique to apply, on which level of decomposition. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

None.

Verification and Validation Configuration (glossary)

A grouping of all physical elements (system elements, Aggregates, Verification and Validation Tools) necessary to perform a Verification and Validation Procedure. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

None.

Verification and Validation Procedure (glossary)

A verification and validation procedure groups a set of Verification and Validation Actions performed together (as a scenario of tests) in a given Verification and Validation Configuration. (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

None.

Verification and Validation Tool (glossary)

A verification and validation tool is a device or physical tool used to perform verification and validation procedures (test bench, simulator, cap/stub, launcher, etc.). (Created for SEBoK)

Sources

This definition was developed for the SEBoK.

Discussion

None.

Viable (glossary)

(1) *Capable of working successfully; feasible.* (Oxford English Dictionary)

(3) *capable of living; especially : having attained such form and development as to be normally capable of surviving outside the mother's womb.* (Oxford English Dictionary)

(3) *A viable system is any system organised in such a way as to meet the demands of surviving in the changing environment.* (Beer 1967)

Sources

(1) & (2) Oxford English Dictionary, 2013., s.v. "viable."

(3) Beer, S. 1967. *Cybernetics and Management*, 2nd edition. London, UK: English Universities Press.

Discussion

(1) and (2) are dictionary definitons which illustrate the two basic ideas of able to do its job, and have reached sufficent maturity to be able to survive in its environment.

(3) Is taken from Beer's Viable Systems Model, which describes the things an organization needs to be viable.

View (glossary)

A representation of a system from the perspective of a viewpoint. (OMG 2010)

Sources

OMG. 2010. *OMG Systems Modeling Language (OMG SysML™), Version 1.3*. Needham, MA, USA: Object Management Group. Accessed on 11 September 2012. Available at https://www.omg.org/technology/documents/domain_spec_catalog.htm#OMGSysML - derived from IEEE 1471.

Discussion

None.

Viewpoint (glossary)

A viewpoint is a specification of the conventions and rules for constructing and using a view for the purpose of addressing a set of stakeholder concerns (OMG 2010)

Source(s)

OMG. 2010. OMG Systems Modeling Language (OMG SysML™), Version 1.3. Needham, MA, USA: Object Management Group. Available at https://www.omg.org/technology/documents/domain_spec_catalog.htm#OMGSysML - derived from IEEE 1471.

Discussion

None.

Vignette (glossary)

A small illustrative sketch. For the SEBoK, a vignette is a short implementation example written to become part of the SEBoK, where a case study introduction cites an existing, more extensive case study already available in the external literature. (Adapted from Webster's Online Dictionary)

Sources

Webster's Online Dictionary. s.v. "Vignette."

Discussion

The base definition is from "Webster's Online Dictionary" at, <https://www.websters-online-dictionary.org/definitions/vignette>. The additional explanation was developed for SEBoK.. This definition describes the approach for vignettes in Systems Engineering Implementation Examples.

W

White-Box System (glossary)

A system where the inner components or logic are available for inspection. (Ashby 1956)

Sources

Ashby, W.R. 1956. *Introduction to Cybernetics*. London, UK: Wiley.

Discussion

None.

Wicked Problem (glossary)

Wicked problems are ill-defined, ambiguous and associated with strong moral, political and professional issues. Since they are strongly stakeholder dependent, there is often little consensus about what the problem is, let alone how to resolve it. (Ritchey 2011)

Sources

Ritchey, T. 2001. *Wicked Problems: Structuring Social Messes with Morphological Analysis*. Stockholm, Sweden: Swedish Morphological Society. Available online at: <https://www.swemorph.com/pdf/wp.pdf>. Accessed July 7, 2012.

Discussion

None.

Acronyms

Acronyms

The following is a list of acronyms and abbreviations which are used within the SEBoK.

SEBoK Acronyms. (Table Developed for BKCASE)

Acronym	Definition
A	
AADL	Architecture Analysis and Design Language
AAS	Advanced Automation System
ABD	Architecture Block Diagram
ACM	Association for Computing Machinery
ACS	Automated Case Support
ADM	Architecture Development Method
AECL	Atomic Energy of Canada
AF	Architecture Framework
AF CSE	Air Force Center for Systems Engineering (USA)
AFI	Air Force Instruction (USA)
AFIT	Air Force Institute of Technology (USA)
AFRI	Air Force Research Lab (USA)
AHP	Analytical Hierarchy Process
AIA	Aerospace Industries Association
AIAA	American Institute of Aeronautics and Astronautics (USA)
AIT	(1) Analysis and Integration Team
	(2) Assembling, Integrating, and Testing
ALT	Accelerated Life Testing
ANSI	American National Standards Institute
AP-233	Application Protocol for Systems Engineering Data Exchange
API	Application Program Interface
APPEL	Academy Program/Project & Engineering Leadership
ASAECP	Aerospace Systems Architecting and Engineering Certificate Program
ASHE	American Society of Hospital Engineering
ASME	American Society of Mechanical Engineers
ASoT	Authoritative Source of Truth
ASQ	American Society for Quality
ASTM	American Society for Testing & Materials

AT&L	Acquisition, Technology, and Logistics (US DoD)
ATAM	Architecture Trademark Analysis Method
ATC	Air Traffic Control
B	
B2B	Business to Business
B2C	Business to Consumer
B2G	Business to Government
BCRM	Business Capability Road Map
BDD	Block Definition Diagram
BIM	Building Information Modeling
BIT	Built-in Test
BKCASE	Body of Knowledge and Curriculum to Advance Systems Engineering
BOK	Body of Knowledge
BPM	(1) Business Process Modeling
	(2) Business Process Management
BPMN	Business Process Modeling Notation
BSI	British Standards Institution
BSS	Business Support System
C	
C&A	Certification and Accreditation
C2	Command and Control
C4ISR	Command, Control, Communications, Intelligence, Information, Surveillance, and Reconnaissance
CAA	Certification and Accreditation
CAS	(1) Complex Adaptive System
	(2) Collision Avoidance System
CASE	Complex Adaptive Systems Engineering
CBK	Common Body of Knowledge
CCB	Change Control Board
CCR	Configuration Change Requirement
CDF	Cumulative Distribution Function
CDR	Critical Design Review
CDRL	Contractor Data Requirements List
CE	Compromising Emission
CI	Continuous Improvement
CIMOSA	Computer Integrated Manufacturing Open System Architecture
CIO	Chief Information Officer
CISSP	Certified Information Systems Security Professional
CM	Configuration Management
CMM	Capability Maturity Model

CMMI	Capability Maturity Model Integration
CO	Contract Officer
COCOMO	Constructive Cost Model
CONOPS	Concept of Operations
CorBoK	Core Body of Knowledge (see also GRCSE)
COSYSMO	Constructive Systems Engineering Cost Model
COTS	Commercial-off-the-Shelf
CPC	Corrosion Prevention and Control
CPI	Continuous Process Improvement
CRM	Customer Relationship Management
CSF	Critical Success Factor
CSI	Continuous Service Improvement
CST	Critical Systems Thinking
D	
D/C IM	Design/Construction Interface Manual
DAU	Defense Acquisition University (USA)
DCR	Design Certification Review
DE	Digital Engineering
DFD	Data Flow Diagram
DFDD	DGIWG Feature Data Dictionary
DFX	Design for "X"
DGWIG	Defence Geospatial Information Working Group
DHS	Department of Homeland Security (USA)
DIA	Denver International Airport
DIS	Distributed Interactive Simulation
DLA	Defense Logistics Agency
DMADV	Define, Measure, Analyze, Design, Verify
DMAIC	Define, Measure, Analyze, Improve, Control
DMSMS	Diminishing Manufacturing Sources and Material Shortages
DNA	Deoxyribonucleic Acid
DoC	Department of Commerce (USA)
DoD	Department of Defense (DoD)
DoDAF	Department of Defense Architecture Framework (USA)
DoDD	Department of Defense Directive (USA)
DoDI	Department of Defense Instruction (USA)
DoEd	Department of Education (USA)
DoJ	Department of Justice (USA)
DoL	Department of Labor (USA)
DoT	Department of Transportation (USA)

DSM	Design Structure Matrix
DTC	Design to Cost
DTM	Digital Terrain Model
E	
E&M	Electrical and Machinery
EA	Enterprise Architecture
EC	European Commission
ECHA	European Chemicals Agency
ECP	Engineering Change Proposal
ECR	Engineering Change Request
EDP	Evaluation and Discussion Point
EE&A	Enterprise Evaluation and Assessment
EESH	Energy, Environmental, Safety and Health Management
eFFBD	Enhanced Functional Flow Block Diagram
EIS	Environmental Impact Statement
EISA	Energy Independence and Security Act of 2007 (USA)
EM	Enterprise Modeling
EMC	Electromagnetic Capability
EMI	Electromagnetic Interference
EMP	Electromagnetic Pulse
EPA	Environmental Protection Agency (USA)
ERB	Engineering Review Board
ERP	Enterprise Resource Planning
ES	Engineered System
ESE	Enterprise Systems Engineering
e-TOM	Enhanced-Telecomm Operations Map
EuSEC	European Systems Engineering Conference
F	
F3I	Form, Fit, Function, and Interface
FAA	Federal Aviation Administration (USA)
FAST	Function Analysis System Technique
FBI	Federal Bureau of Investigation (USA)
FCC	Federal Communications Commission (USA)
FDA	Food and Drug Administration (USA)
FDIR	Failure Detection, Identification, and Recovery
FEAF	Federal Enterprise Architecture Framework (USA)
FEPP	Foreign Excess Personal Property
FFBD	Functional Flow Block Diagram
FMEA	Failure Modes and Effects Analysis

FMECA	Failure Modes, Effects, and Criticality Analysis
FONSI	Finding of No Significant Impact
FoS	Federation of Systems
FPMR	Federal Property Management Regulation
FRACA	Failure Reporting and Corrective Actions
FRACAS	Failure Reporting and Corrective Action System
FRP	Full-Rate Production
FUML	Foundational Subset for Executable UML Models

G

G2B	Government to Business
G2C	Government to Consumer
G2G	Government to Government
GAO	Government Accountability Office (USA)
GDP	Gross Domestic Product
GERAM	Generalize Enterprise Reference Architecture and Methodology
GGE	Geospatial/Geodetic Engineering
GIGO	Garbage In Garbage Out
GIS	Geographic Information System
GMDSS	Global Maritime Distress and Safety System
GNSS	Global Navigation Satellite System
GOTS	Government-off-the-Shelf
GPS	Global Positioning System
GQM	Goal-Question-Metric
GRCSE	Graduate Reference Curriculum for Systems Engineering
GST	General System Theory
GtNR	Guide to Needs and Requirements
GtVV	Guide to Verification and Validation
GtWR	Guide to Writing Requirements

H

HAZMAT	Hazardous Materials
HCI	Human-Computer Interaction
HEMP	High-altitude Electromagnetic Pulse
HFE	Human Factors Engineering
HLA	High-Level Architecture
HMI	Human-Machine Interface
HNA	Host Nation Approval
HNC	Host Nation Coordination
HRO	High Reliability Organization
HSI	Human Systems Integration

HST	Hubble Space Telescope
HWCI	Hardware Configuration Item
HWIL	Hardware in the Loop
I	
IBD	Internal Block Diagram
IBM	International Business Machines
ICM	Incremental Commitment Model
ICSM	Incremental Commitment Spiral Model
ICT	Information and Communications Technology
IDEF0	Integration Definition for Functional Modeling
IE	Industrial Engineering
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
IfM	Institute for Manufacturing
IFSR	International Federation for Systems Research
IHO	International Hydrographic Organization
IID	Incremental and Iterative Development
ILS	Integrated Logistics Support
IM	Information Management
IMS	Integrated Master Schedule
INCOSE	International Council on Systems Engineering
IOC	Initial Operating Capability
IP	(1) Internet Protocol
	(2) Intellectual Property
IPDP	Integrated Product Development Process
IPDT	Integrated Product Development Team
IPR	In-Progress Review
IPS	Integrated Product Support
IPT	Integrated Product Team
IPv6	Internet Protocol Version 6
IRAC	Interdepartment Radio Advisory Committee
IRL	Integration Readiness Level
ISDT	Integrated Service Development Team
ISO	International Organization for Standardization
ISOC	Internet Society
ISP	Internet Service Provider

ISSS	(1) International System Safety Society
	(2) International Society for the Systems Sciences
	(3) Initial Sector Suit System
IT	Information Technology
ITIL	Information Technology Infrastructure Library
ITS	Intelligent Transportation System
ITS-ETO	Intelligent Transport System-Emergency Transportation Operation
ITSM	Information Technology Services Management
ITU	International Telecommunications Union
K	
KA	Knowledge Area
KPI	Key Performance Indicator
KPP	Key Performance Parameter
KSAA	Knowledge, Skills, Abilities, and Attitudes
L	
LBS	Location Based Service
LCC	Life Cycle Cost
LCM	Life Cycle Management
LEfSE	Lean Enablers for Systems Engineering
LORA	Level of Repair Analysis
LRU	Line Replaceable Unit
LSE	(1) Lean Systems Engineering
	(2) Lead Systems Engineer
LSE WG	Lean Systems Engineering Working Group (INCOSE)
M	
MA	Mission Analysis
MARTE	Modeling and Analysis for Real-Time and Embedded Systems
MBE	Model-Based Engineering
MBRA	Model-Based Risk Analysis
MBSE	Model-Based Systems Engineering
MCDA	Multi-Criteria Decision Analysis
MDA®	Model-Driven Architecture
ME	Mission Engineering
MIT	Massachusetts Institute of Technology
MOD	Ministry of Defence
MODAF	Ministry of Defence Architecture Framework (UK)
MOE	Measure of Effectiveness
MOP	Measure of Performance

MOS	Mean Opinion Score
MOSES	Model Oriented Systems Engineering
MPT	(1) Methods, Processes, and Tools
	(2) Methodologies, Processes, and Technologies
MRAR	Mishap Risk Assessment Report
MRL	Manufacturing Readiness Level
MRM	Manufacturing Resource Management
MS	Meta-System
MSA	Measurement System Analysis
MSL	Mean Sea Level
MSRB	Management Safety Review Board
MSTI	Miniature Seeker Technology Integration
MTBF	Mean Time Between Failures
MTTR	Mean Time to Repair
	N
NAHB	National Association of Home Builders
NAO	National Audit Office (UK)
NASA	National Aeronautics and Space Administration (USA)
NATO	North Atlantic Treaty Organization
NDDOT	North Dakota Department of Transportation
NDI	Non-Developmental Item
NDIA	National Defense Industrial Association (NDIA)
NEPA	National Environmental Policy Act
NERC	North American Electric Reliability Corporation
NETMA	NATO Eurofighter and Tornado Management Agency
NIST	National Institute of Standards and Technology (USA)
NLP	Neuro-Linguistic Programming
NOAA	National Oceanographic and Atmospheric Agency
NPD	New Product Development
NPDP	New Product Development Process
NSA	National Security Agency
NSD	New service Development
NSoI	Narrower System-of-Interest
NSPE	National Society of Professional Engineers
NSTISSI	National Security Telecommunications Information Systems Security Issuance
NTIA	National Telecommunications and Information Administration (USA)
	O
O&S	Operations and Support
OAM&P	Operations, Administration, Management, and provisioning

OCR	Operations Commitment Review
OECD	Organization for Economic Co-Operative Development
OEM	Original Equipment Manufacturer
OGC	(1) Office of Government Commerce (UK)
	(2) Open Geospatial Consortium
OITL	Operator in the Loop
OMC	Operation/Maintenance/Closeout
OMG	Object Management Group
OODA	Observe, Orient, Decide, and Act
OPD	Object Process Diagram
OPL	Object Process Language
OPM	Object Process Methodology
OR	Operations Research
ORMS	Operations Research and Management Science
OSAT	On-site Acceptance Test
OSD	Office of the Secretary of Defense (USA)
OSHA	Occupational Safety and Health Administration (USA)
OSM	Office of Spectrum Management
OSS	Operations Support Systems
OT&E	Operational Test and Evaluation
OTP	Operations Technical Plan
OWL	Web Ontology Language
	P
P&L	Profit and Loss
P3I	Pre-Planned Production Improvement
PaaS	Platform as a System
PAPL	Program/Project Approved Parts List
PBD	Physical Block Diagram
PBSE	Pattern-Based Systems Engineering
P-CMM	People Capability Maturity Mode
PD	Product Development
PDCA	Plan, Do, Check, Act
PDF	Portable Document Format
PDR	Preliminary Design Review
PERA	Purdue Enterprise Reference Architecture
PESTEL	Political, Economic, Social, Technological, Environmental, and Legal
PEVQ	Perceptual Evaluation of Video Quality
PfM	Portfolio Management
PFR	Problem Failure Report

PHS&T	Packaging, Handling, Storage, and Transportation
PM	(1) Project Management
	(2) Project Manager
PMBOK™	Project Management Body of Knowledge
PM/FI	Performance Monitoring/Fault Location
PMI	Project Management Institute
PMP	(1) Project Management Professionals
	(2) Project Management Plan
PMP	Parts, Materials, and Processes
PMO	Project Management Office
PMPCB	PMP Control Board
PNT	Positioning, Navigation and Timing
POET	Political, Operational, Economic, and Technical
POM	Proof of Manufacture
PQMI	Process Quality Management and Improvement
PQT	Production Qualification Testing
PRR	Production Readiness Review
PSE	Product Systems Engineering
PSM	Practical Software and Systems Measurement
PSQM	Perceptual Speech Quality Measure
PSS	(1) Product-Service System
	(2) Problem Solution System
PT	Product Team
PUB	Public Utilities Board (Singapore)
Q	
QAS	Quality Assurance System
QFD	Quality Function Deployment
QoE	Quality of Experience
QoS	Quality of Service
QVT	Query View Transformations
R	
R&D	Research and Development
R&M	Reliability and Maintainability
RAA	Responsibility, Authority, and Accountability
RACI	Responsible, Accountable, Consulted, Informed
RAM	Reliability, Availability, and Maintainability
RAMS	Reliability, Availability, Maintainability, and Safety
RAMSS	Reliability, Availability, Maintainability, Safety, and Security

RARA	Roles, Authorities, Responsibilities, and Accountabilities
RB	Radio-communication Bureau
RBD	Reliability Block Diagram
RCM	Reliability Centered Maintenance
RCRA	Resource Conservation and Recovery Act (USA)
RDF	Resource Description Framework
REA	Responsible Engineering Authority
REACH	Registration, Evaluation, Authorization, and Restriction of Chemicals
REP	Risk Element Plan
ReqIF	Requirements Interchange Format
RF	Radio Frequency
RFP	Request For Proposal
RHP	Risk Handling Plan
RMP	Risk Management Plan
RMT	Reliability, Maintenance, and Testability
RNT	Resilient Navigation and Timing
RoHS	Restriction on Hazardous Substances (USA)
ROI	Return on Investment
ROSE	Risk-Oriented Systems Engineering
RR	Radio Regulations
RRB	Radio Regulations Board
RSG	Radio-communication Study Groups
RUP	Rational Unified Process
S	
SaaS	Software as a Service
SADT	Structured Analysis Design Technique
SAE	Society of Automotive Engineers
SAE	Society of Automotive Engineers
SART	System Analysis & Real Time
SBA	Service-Based Application
SBD	Set-Based Design
SBS	System Breakdown Structure
SCM	Software Configuration Management
ScSE	Service-Centric Systems Engineering
SD	Systems Dynamics
SDLC	Service Development Life Cycle
SDO	Standards Development Organization
SDP	Service Design Process
SE	Systems Engineering

SEAC	Systems Engineering Assessment and Control
SEBoK	Systems Engineering Body of Knowledge
SeCSE	Service Centric Systems Engineering
SEI	Software Engineering Institute
SEIT	Systems Engineering and Integration Team
SEM	Systems Engineering Management
SEMP	Systems Engineering Management Plan
SEP	Systems Engineering Plan
SFR	System Functional Review
SGIP	Smart Grid Interoperability Panel
SIS	Software-Intensive System
SKLTS	Standard Korean Light Transit System
SLA	Service Level Agreement
SLDC	Services Development Life Cycle
SLE	Service Life Extension
SLEP	Service Life Extension Program
SLM	Service Level Management
SLOC	Source Lines of Code
SLR	Service Level Requirement
SMC	Systems, Man, and Cybernetics
SOA	Service Oriented Architecture
SOAD	Service Oriented Analysis and Design/Decision Modeling
SoI	System-of-Interest
SOLE	Society of Logistics Engineers
SOMA	Service Oriented Modeling and Architecture
SOO	Statement of Objectives
SOP	Service Operations Plan
SoS	System of Systems
SoSE	System of Systems Engineering
SOSM	Systems of Systems Methodology
SOW	Statement of Work
SPAC	Spectrum Planning and Policy Advisory Committee
SPC	Statistical Process Control
SPM	Service Performance Measure
SRA	Strategic Rail Authority (UK)
SRD	Service Requirements Document
SRL	System Readiness Level
SRR	System Requirements Review
SSDP	Service Systems Development Process

SSE	(1) Service Systems Engineering
	(2) Security Systems Engineering
SSEMP	System Security Engineering Management Plan
SSM	Soft Systems Methodology
SSME	Service System Management and Engineering
SSP	(1) Source Selection Plan
	(2) System Security Plan
SSPP	System Safety Program Plan
SSS	Service Support Systems
STAMP	System-Theoretic Accident Model and Processes
STEEPLED	Social, Technical, Economic, Environmental, Political, Legal, Ethical and Demographic
STI	Science, Technology, and Industry Forum
STP	Strategic Technical Planning
SVR	System Verification Review
SWCI	Software Configuration Item
SwE	Software Engineering
SWEBoK	Software Engineering Body of Knowledge
SWG	Safety Working Group
SWIL	Software in the Loop
SWOT	Strengths, Weaknesses, Opportunities, and Threats Analysis
SwSE	Software Systems Engineering
SysML	Systems Modeling Language
T	
TCSE	Telecommunication Systems Engineering
TM	Technical Management
TOC	Total Ownership Cost
TOGAF	The Open Group Architecture Framework
TOVE	Toronto Virtual Enterprise (Canada)
TPM	Technical Performance Measure
TPS	Toyota Production System
TQM	Total Quality Management
TRL	Technology Readiness Level
TRR	Technical Readiness Review
TSE	Traditional Systems Engineering
TSI	Total Systems Intervention
TSP	Technology and Standards Planning
U	
UC	Use Case
UK	United Kingdom

UL	Underwriters Laboratory
UML	Unified Modeling Language
UN	United Nations
UPDM	Unified Profile for DoDAF and MODAF
US	United States
USAF	United States Air Force
USB	Universal Serial Bus
V	
V&V	Verification and Validation
VCF	Virtual Case File
VERAM	Virtual Enterprise Reference Architecture and Methodology
VHDL	VHSIC Hardware Description Language
VHSIC	Very-High-Speed Integration Circuit
VIN	Vehicle Identification Number
W	
WBS	Work Breakdown Structure
WCML	West Coast Main Line (UK)
WCRM	West Coast Route Modernisation (UK)
WGS84	World Geodetic System 1984
WIP	Work in Progress
WMO	World Meteorological Organization
WRC	World Radio-communication Conference
WS	Web Services 2.0
WSoI	Wider System-of-Interest
WS-PBEL	Web Services Business Process Execution Language
WWII	World War II
WWW	World Wide Web
X	
XMI	XML Metadata Interchange
XML	Extensible Markup Language

Article Sources and Contributors

Absorption (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76064> *Contributors:* Bkcase, Cdhoffman, Janthony, Mhaas, S.Jackson, WikiWorks, WikiWorks753, Wikiexpert

Abstract Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78354> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Kguillemette, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Abstract Syntax (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78278> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Kguillemette, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Abstraction (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78162> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, S.Jackson, Smenck2, WikiWorks, Wikiexpert

Acceptance Criteria (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76589> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Kforsberg, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Acceptance Sampling (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78517> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Gconover, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Acquirer (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76362> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Acquisition (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76150> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Acquisition Strategy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76119> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Activity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77796> *Contributors:* Cdhoffman, DEndler

Adaptability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76554> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, S.Jackson, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Address (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75982> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Adversity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75998> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Aggregate (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76033> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Agile (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76102> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Kforsberg, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Agility (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76392> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Smenck2, WikiWorks, Wikiexpert

Agreement (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76088> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Smenck2, WikiWorks, Wikiexpert

Analytical Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75992> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Smenck2, WikiWorks, Wikiexpert

Antipattern (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78440> *Contributors:* Apyster, Bkcase, Cdhoffman, Radcock, WikiWorks, WikiWorks753

Architecting (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76841> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Architecture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78344> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Eleach, Janthony, Radcock, Smenck2, WikiWorks, Wikiexpert

Architecture Framework (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78459> *Contributors:* Bkcase, Cdhoffman, Dholwell, Radcock, Rcloutier, WikiWorks

Assembly Procedure (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76184> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Assembly Tool (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76518> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Assessment Criterion (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76078> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Assessment Score (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75997> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Attribute (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78472> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Availability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78286> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Baseline (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78275> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Behavior (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76858> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Behavioral Architecture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76115> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, WikiWorks, Wikiexpert

Binding (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75993> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Black-Box System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76211> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Body of Knowledge (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78257> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Janthony, Kguillemette, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Boundary (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76068> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Business (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76788> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Business Process (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78415> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Business Process Outsourcing (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78289> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Call Center (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78298> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Capability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76786> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Eleach, Janthony, Radcock, Smenck2, WikiWorks, Wikiexpert

Capability Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76056> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Capacity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76060> *Contributors:* Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Cartesian coordinate system (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76826> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Cartography (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76599> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks

Case Study (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76021> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Catalogue (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76640> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Chaos (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76377> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Closed System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75994> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Coercive (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76499> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, Wikiexpert

Cognitive Bias (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=68183> *Contributors:* Bkcase, Cdhoffman

Cohesion (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78442> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Smenck2, WikiWorks, Wikiexpert

Competency (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78413> *Contributors:* Bkcase, Cdhoffman, Cnielsen, Hdavidz, Janthony, Radcock, Smenck2, WikiWorks, Wikiexpert

Complex (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76005> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Smenck2, WikiWorks, Wikiexpert

Complex Adaptive System (CAS) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76911> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Complexity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78562> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Gsmith, Radcock, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Complexity avoidance (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76829> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Component (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76311> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Computer Simulation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78322> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, Wikiexpert

Concept (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78428> *Contributors:* Apyster, Bkcase, Cdhoffman, Radcock, WikiWorks, WikiWorks753, Wikiexpert

Concept Definition (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75976> *Contributors:* Bkcase, Cdhoffman, Dhenry, Jtcarter, WikiWorks, WikiWorks753

Concept of Operations (ConOps) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75985> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Concrete Syntax (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78372> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Concurrent (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76082> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Concurrently (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76510> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, WikiWorks, Wikiexpert

Configuration (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78161> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Configuration Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78206> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Constraint (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76345> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Control (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78171> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Coordinate (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76002> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Cost (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76066> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Critical Design Review (CDR) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78146> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Critical Systems Thinking (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76587> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Culture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76007> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Customer (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78326> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jekstrom, Smenck2, WikiWorks, Wikiexpert

Cyber-Physical Systems (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76032> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Cybernetics (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76702> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Cycle (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78056> *Contributors:* Bkcase, Cdhoffman, Gconover, WikiWorks

Data Center (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78233> *Contributors:* Apyster, Bkcase, Cdhoffman, Jekstrom, Jgercken, Smenck2, WikiWorks, Wikiexpert

Decision Gate (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75991> *Contributors:* Apyster, Bkcase, Cdhoffman, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Decision Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76120> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Jgercken, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Defense in depth (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76116> *Contributors:* Bkcase, Cdhoffman, SJackson, WikiWorks

Demonstration (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78348> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Derived Requirement (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76155> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Smenck2, WikiWorks, Wikiexpert

Descriptive Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78143> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Design (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76823> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Design Factor (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76017> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Design Life (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78409> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Design Property (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=75978> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

DevOps (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76076> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhaas, WikiWorks

Digital Elevation Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76151> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Digital Terrain Model (DTM) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76171> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Digital Twin (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=57655> *Contributors:* Bkcase

Disposal (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76028> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Disruption (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76010> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Diversity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76394> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Domain (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76810> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Smenck2, WikiWorks, Wikiexpert

Drift Correction (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76097> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, SJackson, Smenck2, WikiWorks, Wikiexpert

Drift Correction (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76097> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, SJackson, Smenck2, WikiWorks, Wikiexpert

Dualism (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78225> *Contributors:* Apyster, Bkcase, Cdhoffman, Radcock, WikiWorks

E-Services (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76340> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Effectiveness (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76791> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Element (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76837> *Contributors:* Afaisandier, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Ellipsoid (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76070> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Ellipsoidal Height or Geodetic Height (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76354> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Emergence (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76117> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Emergent Behavior (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76069> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Emergent Property (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78583> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Enabling System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76106> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Encapsulation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77985> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Radcock, WikiWorks

Engineered System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78276> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Jgercken, Radcock, Rcloutier, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76130> *Contributors:* Bkcase, Cdhoffman, Dholwell, Jgercken, Smenck2, WikiWorks, Wikiexpert

Engineering Change Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76105> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Engineering Change Proposal (ECP) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76101> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Engineering Coordinate Reference System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76231> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Enterprise (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76305> *Contributors:* Bkcase, Cdhoffman, Dhenry, Janthony, Jmartin, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Enterprise Architecture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78187> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Eleach, Janthony, Jgercken, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Enterprise System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76482> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Enterprise Systems Engineering (ESE) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76477> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Entropy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78071> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Environment (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76240> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Equity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76245> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Ethics (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78247> *Contributors:* Asquires, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Evolutionary (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76536> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Executable System Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76222> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Extended Enterprise (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78208> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Failure (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76024> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Failure Modes and Effects Criticality Analysis (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76012> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Feature (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76047> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks

Feature attribute (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76387> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Feature Catalog (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76667> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Feature catalogue (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76084> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks

Federation of Systems (FoS) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76108> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Flexibility (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76006> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Form, Fit, Function, and Interface (F3I) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76020> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Framework (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76443> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, WikiWorks

Function (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76579> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Jgercken, Mhaas, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Functional Architecture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76475> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Csingam, Dhenry, Jgercken, Smenck2, WikiWorks, Wikiexpert

Functional redundancy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76348> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Gazetteer (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76447> *Contributors:* Bkcase, Cdhoffman, WikiWorks

General System Theory (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76581> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Geodesic or Geodesic Line (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76077> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geodesy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76787> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geodetic Coordinate System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76913> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geodetic Datum (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76285> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geodetic Reference Frame (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76148> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geographic coordinates (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76087> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Geographic Data (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=60724> *Contributors:* Mhaas

Geographic Identifier (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76881> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geographic Imagery (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76312> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geographic Information (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76361> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geographic Information System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76081> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Geoid (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76356> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

GEOINT (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76296> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks

Geopositioning (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76104> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Glossary:Principle *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76331> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Governance (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78258> *Contributors:* Apyster, Bkcase, Cdhoffman, Eleach, Jgercken, Smenck2, WikiWorks, Wikiexpert

Gravity-Related Height (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76113> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Green Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78237> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Smenck2, WikiWorks, Wikiexpert

Habitability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78391> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Hard System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76940> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Hardware-in-the-Loop Testing (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76303> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Healthcare (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78350> *Contributors:* Bkcase, Cdhoffman, Nicole.hutchison, WikiWorks

Healthcare Systems Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76615> *Contributors:* Bkcase, Cdhoffman, Nicole.hutchison, WikiWorks

Height (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76684> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Heuristic (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76201> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Hierarchy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76714> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

High Reliability Organizations (HROs) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76502> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Holism (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76182> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Holistic (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76884> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Homeostasis (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78374> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Human Factors (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76158> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Human in the loop (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76281> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Human Survivability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76893> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Human Systems Integration (HSI) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76287> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Hydrography (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76523> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks

Implementation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76541> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Eleach, Janthony, Jgercken, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert

In-Process Validation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76425> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Inclusion (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76205> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Increment (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76325> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Incremental (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76781> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Industrial Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78403> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, WikiWorks, Wikiexpert

Information and Communications Technologies (ICT) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78147> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Information Category (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76658> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Smenck2, WikiWorks, Wikiexpert

Information Need (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76203> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Information Technology (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78241> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Infrastructure (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78339> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Input (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78315> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Smenck2, WikiWorks, Wikiexpert

Input-Output Flow (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76885> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Smenck2, WikiWorks, Wikiexpert

Installation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78169> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Integrated Product Team (IPT) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76427> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Integration (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76473> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Jgercken, Radcock, Smenck2, WikiWorks, Wikiexpert

Integrity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76779> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, S.Jackson, Smenck2, WikiWorks, Wikiexpert

Inter-Node Interaction (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76496> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, S.Jackson, Smenck2, WikiWorks, Wikiexpert

Interface (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76403> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Internal Coordinate Reference System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76877> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Internode Interaction (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76242> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Interoperability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76385> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Isomorphism (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78830> *Contributors:* Cdhoffman

Issue (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76807> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Iteration (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76647> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Knowledge (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78399> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, WikiWorks, WikiWorks753

Land Cover (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76389> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Land Use (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76854> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Latent (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76720> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Layer (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76114> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Leader (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76156> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Lean Systems Engineering (LSE) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76528> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Jgercken, Smenck2, WikiWorks, Wikiexpert

Lean Value Stream Mapping (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76391> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Legacy System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78269> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Leverage (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76342> *Contributors:* Bkcase, Cdhoffman, Radcock, WikiWorks

Life Cycle (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76608> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Eleach, Jgercken, Radcock, Smenck2, WikiWorks, Wikiexpert

Life Cycle Cost (LCC) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76567> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Life Cycle Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76484> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Kforsberg, Smenck2, WikiWorks, Wikiexpert

Life Cycle Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76625> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Jgercken, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert

Life Cycle Process (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76507> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jgercken, Smenck2, WikiWorks, Wikiexpert

Localized Capacity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76586> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Location (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76192> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Location-Based Service (LBS) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76291> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Logical Architecture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76452> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Mhenshaw, Radcock, WikiWorks, Wikiexpert

Logistics (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76407> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Loose Coupling (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76856> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Loss-Driven Systems Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76092> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Maintainability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78224> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Maintenance (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78265> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Manpower (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78145> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Map (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76928> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Map Projection (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76755> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Market Analysis (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78317> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhenshaw, WikiWorks, Wikiexpert

Mean Sea Level(glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76867> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Measurable Concept (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76360> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Measure (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76512> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Measure of Effectiveness (MoE) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76121> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Measure of Performance (MoP) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76146> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Measurement (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76096> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Meta-model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76633> *Contributors:* Bkcase, Cdhoffman, Janthony, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Metadata (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76932> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Metric (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78282> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Milestone (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76490> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Mission (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76525> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Radcock, Smenck2, WikiWorks, Wikiexpert

Mission Analysis (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76349> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks, Wikiexpert

Mission Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76136> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78396> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Mhenshaw, Rmadachy, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Model Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76646> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Model Transformation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76198> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Model Validation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76134> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Model-Based Systems Engineering (MBSE) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76738> *Contributors:* Bkcase, Cdhoffman, Dholwell, Jgercken, Smenck2, WikiWorks, Wikiexpert

Modeling Language (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76709> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Modeling Tool (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76049> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Rmadachy, Smenck2, WikiWorks, Wikiexpert

Modularity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76915> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, S.Jackson, Smenck2, WikiWorks, Wikiexpert

Motion (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76899> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Natural System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77573> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Nautical Chart (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76759> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Navigation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76218> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Network (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76145> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Neutral State (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76924> *Contributors:* Apyster, Bkcase, Cdhoffman, S.Jackson, Smenck2, WikiWorks, Wikiexpert

Node (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76426> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Non-Functional Requirements (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76228> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Occupational Health (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78475> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Ontology (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76176> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Open System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76396> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Operational (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76619> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Operational Capability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76317> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Operational Concept (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76712> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Mhenshaw, WikiWorks, WikiWorks753

Operational Environment (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76585> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Operational Mode (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76614> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Operational Scenario (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76423> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Operations Research (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76768> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Opportunity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76479> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Organization (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76774> *Contributors:* Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

Organizational (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78305> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, WikiWorks753, Wikiexpert

Organizational Capability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76740> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Orthoimage (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76187> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Outcome (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77797> *Contributors:* Cdhoffman, DEndler

Output (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78416> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Paradigm (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76352> *Contributors:* Apyster, Bkcase, Cdhoffman, Smenck2, WikiWorks, Wikiexpert

Pattern (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76144> *Contributors:* Apyster, Bkcase, Cdhoffman, Radcock, WikiWorks

Personnel (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76195> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, Smenck2, WikiWorks, Wikiexpert

Physical Architecture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76588> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Eleach, Jgercken, Radcock, Smenck2, WikiWorks, Wikiexpert

Physical Interface (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76154> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Physical Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76652> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Physical redundancy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76736> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Plan (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76236> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Pluralist (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76616> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, WikiWorks, Wikiexpert

Portfolio (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76582> *Contributors:* Bkcase, Cdhoffman, Dhenry, WikiWorks, WikiWorks753, Wikiexpert

Portfolio Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76400> *Contributors:* Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Portrayal (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76669> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Positional accuracy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76179> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Postal address type (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76620> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Postmodernist (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76863> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks, Wikiexpert

Praxis (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78181> *Contributors:* Apyster, Bkcase, Cdhoffman, Gconover, Radcock, WikiWorks

Preliminary Design Review (PDR) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76689> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Principle (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78465> *Contributors:* Apyster, Bkcase, Cdhoffman, Radcock, WikiWorks, WikiWorks753, Wikiexpert

Problem (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76657> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, WikiWorks, Wikiexpert

Process (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76563> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Rmadachy, WikiWorks, Wikiexpert

Procurement (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78331> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Product (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76531> *Contributors:* Bkcase, Cdhoffman, Dholwell, Janthony, Jgercken, WikiWorks, WikiWorks753, Wikiexpert

Product System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77571> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jgercken, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

Program (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76341> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, WikiWorks, WikiWorks753, Wikiexpert

Program Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76055> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Janthony, Jgercken, Kguillemette, WikiWorks, WikiWorks753, Wikiexpert

Project (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76545> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Jgercken, Kguillemette, WikiWorks, Wikiexpert

Project Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76803> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Jgercken, Kguillemette, WikiWorks, WikiWorks753, Wikiexpert

Prototype (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76095> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, WikiWorks, Wikiexpert

Proving (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78430> *Contributors:* Bkcase, Cdhoffman, Dholwell, Jgercken, WikiWorks, WikiWorks753, Wikiexpert

Purpose (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76663> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Purposeful (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76058> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, WikiWorks, Wikiexpert

Purposive (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76927> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, WikiWorks, Wikiexpert

Qualification (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76299> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, WikiWorks, Wikiexpert

Quality (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78474> *Contributors:* Bkcase, Cdhoffman, Janthony, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

Rationale (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76517> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Recovery (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76402> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, WikiWorks, Wikiexpert

Recursion (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76623> *Contributors:* Bkcase, Cdhoffman, Jgercken, WikiWorks, Wikiexpert

Reductionism (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76243> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhenshaw, WikiWorks, Wikiexpert

Redundancy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76181> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Regularity (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76191> *Contributors:* Apyster, Bkcase, Cdhoffman, Radcock, WikiWorks

Reliability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78446> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, WikiWorks, Wikiexpert

Remote sensing (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76229> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks

Reparability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76466> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Requirement (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76704> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jgercken, WikiWorks, WikiWorks753, Wikiexpert

Resilience (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78347> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, S.Jackson, WikiWorks, Wikiexpert

Resolution (of imagery) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76282> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Restructuring (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76309> *Contributors:* Bkcase, Cdhoffman, S.Jackson, WikiWorks

Reverse Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78227> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Rhumb Line (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76252> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Risk (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76866> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks, Wikiexpert

Risk Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77764> *Contributors:* Bkcase, Cdhoffman, DEndler, Dholwell, Jgercken, WikiWorks, Wikiexpert

Robustness (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76898> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, S.Jackson, WikiWorks, Wikiexpert

Routing (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76690> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Safety (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76382> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

Satellite Positioning System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76323> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Scalability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76701> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Scenario (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76294> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Scope (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78358> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

Security (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76696> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Mhenshaw, WikiWorks, Wikiexpert

Semantic Interoperability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78345> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Semantics (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76895> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Service (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78340> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jgercken, WikiWorks, Wikiexpert

Service Life Extension (SLE) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78320> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Service Science (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76626> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Service System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76685> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, WikiWorks, WikiWorks753, Wikiexpert

Service Systems Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76874> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Janthony, Jgercken, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

Service Value Chain (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78448> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Set Driver (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76469> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Set Modifiers (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76059> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks

Set-Based Design (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76566> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Simulation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76897> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Simulator (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76272> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Six Sigma (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78419> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Social System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77562> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

Sociotechnical System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78156> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, Radcock, WikiWorks, WikiWorks753, Wikiexpert

Soft System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76472> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Soft Systems Methodology (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76601> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, WikiWorks, WikiWorks753, Wikiexpert

Software (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76172> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Software Assurance (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78213> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Software Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78217> *Contributors:* Bkcase, Cdhoffman, Kguillemette, WikiWorks

Software System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78228> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Software-in-the-Loop Testing (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78395> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, WikiWorks, Wikiexpert

Solution (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76276> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Spatial Reference (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76401> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Spatial Reference System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76278> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Spatial Reference (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76401> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Specialty Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76934> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, WikiWorks, Wikiexpert

Stage (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76861> *Contributors:* Bkcase, Cdhoffman, Dhenry, Jgercken, Kforsberg, Mhenshaw, WikiWorks, Wikiexpert

Stakeholder (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76550> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Stakeholder Needs and Requirements (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76521> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Eleach, Janthony, Mhenshaw, Radcock, WikiWorks, Wikiexpert

State (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76595> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks, Wikiexpert

Statistical Process Control (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76777> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Jgercken, WikiWorks, Wikiexpert

Structure (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78383> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Supplier (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76580> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, WikiWorks, Wikiexpert

Survivability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76207> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Sustainment (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78445> *Contributors:* Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Synectics (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76459> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Synergy (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76831> *Contributors:* Apyster, Bkcase, Cdhoffman, Radcock, WikiWorks, WikiWorks753

Synthesis (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76857> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78057> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Ddori, Dhenry, Dholwell, Gconover, Janthony, Jgercken, Radcock, Rcloutier, WikiWorks, WikiWorks753, Wikiexpert

System Analysis (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76260> *Contributors:* Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, Radcock, WikiWorks, Wikiexpert

System Assurance (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76612> *Contributors:* Asquires, Bkcase, Cdhoffman, WikiWorks, Wikiexpert

System Boundary (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76215> *Contributors:* Apyster, Bkcase, Cdhoffman, WikiWorks, WikiWorks753

System Breakdown Structure (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76638> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

System Capability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76072> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

System Context (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76600> *Contributors:* Apyster, Bkcase, Cdhoffman, Janthony, Jgercken, WikiWorks, Wikiexpert

System Coupling Diagram (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76288> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Smenck2, WikiWorks, Wikiexpert

System Definition (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76918> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Eleach, Janthony, Jcarter, Mhenshaw, Radcock, WikiWorks, Wikiexpert

System Deployment and Use (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76665> *Contributors:* Bkcase, Cdhoffman, Dhenry, WikiWorks

System Element (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76157> *Contributors:* Afaisandier, Bkcase, Cdhoffman, Ddori, Dhenry, Dholwell, Jgercken, Radcock, WikiWorks, Wikiexpert

System Hardware Assurance *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77494> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhaas, WikiWorks

System of Systems (SoS) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76722> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Jdahmann, WikiWorks, Wikiexpert

System Property (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78222> *Contributors:* Bkcase, Cdhoffman, Dholwell, Jgercken, WikiWorks, Wikiexpert

System Realization (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76850> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Janthony, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

System Requirement (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76705> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Janthony, Mhenshaw, Radcock, WikiWorks, Wikiexpert

System-of-Interest (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76878> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Radcock, Sackson, WikiWorks, WikiWorks753, Wikiexpert

Systemist (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76691> *Contributors:* Bkcase, Cdhoffman, Mhaas, WikiWorks

Systems Approach (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76694> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhenshaw, WikiWorks, WikiWorks753, Wikiexpert

Systems Biology (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76624> *Contributors:* Bkcase, Cdhoffman, Nicole.hutchison, WikiWorks

Systems Concept (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76530> *Contributors:* Apyster, Bkcase, Cdhoffman, Eleach, Jgercken, Mhenshaw, Radcock, WikiWorks, Wikiexpert

Systems Development (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76805> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Systems Engineer (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76737> *Contributors:* Apyster, Bkcase, Cdhoffman, Dholwell, Janthony, Jgercken, Mhenshaw, WikiWorks, Wikiexpert

Systems Engineering (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76725> *Contributors:* Bkcase, Cdhoffman, Dholwell, Janthony, Radcock, Rcloutier, WikiWorks, Wikiexpert

Systems Engineering Management (SEM) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76675> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Systems Engineering Plan (SEP) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78532> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Gconover, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Systems Integration (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76785> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Systems Science (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78852> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Gsmith, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Systems Thinking (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76274> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Dholwell, Mhaas, WikiWorks, Wikiexpert

Tailoring (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76202> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Task (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=77798> *Contributors:* Cdhoffman, DEndler

Team (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78432> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Technical Management (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=60287> *Contributors:* Bkcase

Technical Performance Measure (TPM) (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78148> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Temporal Architecture (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76653> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Mhaas, WikiWorks, Wikiexpert

Test Cases (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76489> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Threat (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76559> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Tolerance (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76061> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, SJackson, WikiWorks, Wikiexpert

Total Ownership Cost (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76789> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, WikiWorks, Wikiexpert

Traceability (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76917> *Contributors:* Bkcase, Cdhoffman, Dholwell, Mhaas, Smenck2, WikiWorks

Training (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76123> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Transition of Modes (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76618> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Transport network (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76799> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Uncertainty (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78200> *Contributors:* Bkcase, Cdhoffman, Dhenry, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Unitary (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76746> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Jgercken, Mhaas, WikiWorks, Wikiexpert

Use Case (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76796> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Useful life (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78357> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

User (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76160> *Contributors:* Bkcase, Cdhoffman, Dhenry, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Utility network (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76346> *Contributors:* Bkcase, Cdhoffman, WikiWorks

Validation (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76546> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Eleach, Janthony, Jgercken, Kforsberg, Kguillemette, Mhaas, Radcock, Rcloutier, WikiWorks, WikiWorks753, Wikiexpert

Value (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76535> *Contributors:* Apyster, Bkcase, Cdhoffman, Dhenry, Jgercken, Mhaas, Smenck2, WikiWorks, Wikiexpert

Variety (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76573> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, WikiWorks, Wikiexpert

Vee (V) Model (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76801> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Verification (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76368> *Contributors:* Asquires, Bkcase, Cdhoffman, Dhenry, Eleach, Janthony, Jgercken, Kforsberg, Kguillemette, Mhaas, Radcock, WikiWorks, WikiWorks753, Wikiexpert

Verification and Validation Action (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76639> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Verification and Validation Configuration (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76955> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Verification and Validation Procedure (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76943> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Verification and Validation Tool (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76942> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Viable (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76945> *Contributors:* Apyster, Bkcase, Cdhoffman, Jgercken, Mhaas, WikiWorks, Wikiexpert

View (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78471> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Viewpoint (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78346> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Vignette (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78381> *Contributors:* Apyster, Asquires, Bkcase, Cdhoffman, Dhenry, Hdavidz, Jgercken, Mhaas, WikiWorks, Wikiexpert

White-Box System (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=76948> *Contributors:* Apyster, Bkcase, Cdhoffman, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Wicked Problem (glossary) *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78296> *Contributors:* Bkcase, Cdhoffman, Jgercken, Mhaas, Mhenshaw, WikiWorks, Wikiexpert

Acronyms *Source:* <https://sebokwiki.org/draft/w/index.php?oldid=78511> *Contributors:* Afaisandier, Apyster, Bkcase, Cdhoffman, Ddorgan, Ddori, Dhenry, Gconover, Mhaas, Rlpineda, Smenck2, WikiWorks, Wikiexpert

Image Sources, Licenses and Contributors

File:052311_SJ_System_Coupling_Diagram.png *Source:* https://sebokwiki.org/draft/w/index.php?title=File:052311_SJ_System_Coupling_Diagram.png *License:* unknown *Contributors:* Mhenshaw, Smenck2

File:Component_Lifecycle_(rev_a)_-_MJB_ver2.jpg *Source:* [https://sebokwiki.org/draft/w/index.php?title=File:Component_Lifecycle_\(rev_a\)_-_MJB_ver2.jpg](https://sebokwiki.org/draft/w/index.php?title=File:Component_Lifecycle_(rev_a)_-_MJB_ver2.jpg) *License:* unknown *Contributors:* Bkcase